Physlib Monthly Report

1–31 October 2026

Contributors: Aayush Rajasekaran, Andrea Pari, Anoddy, artmountain, brentius, Cameron Lee, dl-eigenart, doxtor6, dwanith, Eduardo Nava Hernandez, Emil Kuban, ereinhardt8, Eric Wieser, Gregory Loges, Jaehyeon Shin, Joseph Tooby-Smith, Julian Popescu, Kim Morrison, Michal Mogielnicki, nateabr, Philippe Emile Christian KEVORKIAN, Rithwik Ranganathan, Ruth Wang, Samyak Rai, Scott Hughes, Sebastian Boehler, SourLen, Steven Nguyen, Tom Diem, Wouter Deconinck, Zhi Kai Pong

Reviewers: Alex Meiburg, Daniel Morrison, doxtor6, Gregory Loges, Joseph Tooby-Smith, Michal Mogielnicki, nateabr, Philippe Emile Christian KEVORKIAN, RaunakChhatwal, Samyak Rai, Tom Diem, Wouter Deconinck, Zhi Kai Pong

Generated 1 November 2026 from commit `a8823c7`.

Abstract

Physlib is a community library of formalised physics for the Lean 4 proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month's additions to it.

It covers 1–31 October 2026 on branch `master` of `leanprover-community/physlib`, between commits `b840c85` and `a8823c7`. Over that period 31 contributors landed 239 commits, changing 195,456 lines across 1,329 files and adding 10,965 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at GitHub compare.

Contents

1 Introduction **15**
- 1.1 Contributors . . . 15
- 1.2 Reviewers . . . 16
- 1.3 Modified subfolders . . . 16
- 1.4 New declarations by kind . . . 17

2 Details by file **18**
- 2.1 `Physlib.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.Basic` . . . 18
- 2.2 `Physlib.ClassicalMechanics.ConfigurationSpace.Basic` . . . 21
- 2.3 `Physlib.ClassicalMechanics.ConfigurationSpace.Variation` . . . 21
- 2.4 `Physlib.ClassicalMechanics.EulerLagrange` . . . 22
- 2.5 `Physlib.ClassicalMechanics.HamiltonianMechanics.PoissonBracket` . . . 23
- 2.6 `Physlib.ClassicalMechanics.HarmonicOscillator.Solution.Energy` . . . 25
- 2.7 `Physlib.ClassicalMechanics.HarmonicOscillator.Solution.SpecialPoints` 25
- 2.8 `Physlib.ClassicalMechanics.Lagrangian.Basic` . . . 27
- 2.9 `Physlib.ClassicalMechanics.Lagrangian.CriticalPoint` . . . 28
- 2.10 `Physlib.ClassicalMechanics.Lagrangian.FirstVariation` . . . 29
- 2.11 `Physlib.CondensedMatter.Crystal.Basic` . . . 30

2.12 Physlib.CondensedMatter.Crystal.Bonds . . . 31
2.13 Physlib.CondensedMatter.Crystal.Bravais.Basic . . . 32
2.14 Physlib.CondensedMatter.Crystal.Embedding . . . 33
2.15 Physlib.CondensedMatter.Crystal.PeriodicGraph . . . 35
2.16 Physlib.CondensedMatter.Crystal.Symmetry.Basic . . . 38
2.17 Physlib.CondensedMatter.Crystal.Symmetry.Isometries . . . 42
2.18 Physlib.CondensedMatter.Crystal.Symmetry.PointOperations . . . 43
2.19 Physlib.CondensedMatter.Crystal.Symmetry.SpaceGroup . . . 45
2.20 Physlib.CondensedMatter.Crystal.Symmetry.Species . . . 46
2.21 Physlib.CondensedMatter.TightBindingChain.Basic . . . 47
2.22 Physlib.Cosmology.FLRW.Age . . . 49
2.23 Physlib.Cosmology.FLRW.Basic . . . 49
2.24 Physlib.Cosmology.FLRW.DensityParameters . . . 52
2.25 Physlib.Cosmology.FLRW.DeSitter . . . 53
2.26 Physlib.Cosmology.FLRW.Distances . . . 54
2.27 Physlib.Cosmology.FLRW.EnergyConditions . . . 56
2.28 Physlib.Cosmology.FLRW.FriedmannSolution . . . 57
2.29 Physlib.Cosmology.FLRW.MatterContent . . . 59
2.30 Physlib.Cosmology.FLRW.ReducedHubble . . . 62
2.31 Physlib.Cosmology.FLRW.Solutions . . . 64
2.32 Physlib.Cosmology.FLRW.SpecificCases.EinsteinDeSitter . . . 64
2.33 Physlib.Cosmology.FLRW.SpecificCases.PowerLaw . . . 65
2.34 Physlib.Cosmology.FLRW.SpecificCases.RadiationDominated . . . 65
2.35 Physlib.Electromagnetism.Current.InfiniteWire . . . 66
2.36 Physlib.Electromagnetism.Distributional.Basic . . . 68
2.37 Physlib.Electromagnetism.Kinematics.ElectricField . . . 68
2.38 Physlib.Electromagnetism.Kinematics.MagneticField . . . 69
2.39 Physlib.Electromagnetism.Kinematics.VectorPotential . . . 69
2.40 Physlib.Electromagnetism.PointParticle.OneDimension . . . 69
2.41 Physlib.Electromagnetism.PointParticle.ThreeDimension . . . 71
2.42 Physlib.Electromagnetism.TemperedDistributional.Basic . . . 73
2.43 Physlib.Electromagnetism.TemperedDistributional.Dynamics.CurrentDensity 75
2.44 Physlib.Electromagnetism.TemperedDistributional.Dynamics.IsExtrema . . 75
2.45 Physlib.Electromagnetism.TemperedDistributional.Dynamics.KineticTerm 76
2.46 Physlib.Electromagnetism.TemperedDistributional.Dynamics.Lagrangian . 77
2.47 Physlib.Electromagnetism.TemperedDistributional.ElectricField . . . 79
2.48 Physlib.Electromagnetism.TemperedDistributional.FieldStrength . . . 79
2.49 Physlib.Electromagnetism.TemperedDistributional.MagneticField . . . 82
2.50 Physlib.Electromagnetism.TemperedDistributional.ScalarPotential . . . 83
2.51 Physlib.Electromagnetism.TemperedDistributional.VectorPotential . . . 83
2.52 Physlib.Mathematics.Calculus.Gradient . . . 83
2.53 Physlib.Mathematics.Calculus.ParametricIntegration . . . 85
2.54 Physlib.Mathematics.Calculus.PartialDerivatives . . . 85
2.55 Physlib.Mathematics.Calculus.Wirtinger.Basic . . . 86
2.56 Physlib.Mathematics.Calculus.Wirtinger.Coordinate . . . 86
2.57 Physlib.Mathematics.Calculus.Wirtinger.Hessian . . . 87
2.58 Physlib.Mathematics.Distribution.TemperedDistribution . . . 90
2.59 Physlib.Mathematics.ForMathlib.ComplexLinear . . . 91
2.60 Physlib.Mathematics.ForMathlib.Integration.Basic . . . 92
2.61 Physlib.Mathematics.ForMathlib.SesquilinearForm . . . 92
2.62 Physlib.Mathematics.ForMathlib.Trigonometry.Tanh . . . 93

2.63 Physlib.Mathematics.Groups.SO3.Basic . . . 93
2.64 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.Adjoint . . . 93
2.65 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.Basic . . . 97
2.66 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.Contractions . . 101
2.67 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.GellMann.Adjoint 105
2.68 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.GellMann.Basic . 106
2.69 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.GellMann.SpecialCases 111
2.70 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.KillingForm . . 112
2.71 Physlib.Mathematics.InnerProductSpace.Gaussian . . . 113
2.72 Physlib.Mathematics.KroneckerDelta.Contraction . . . 113
2.73 Physlib.Mathematics.VariationalCalculus.Interval . . . 114
2.74 Physlib.Meta.Pseudo . . . 114
2.75 Physlib.Optics.Frequency . . . 115
2.76 Physlib.Optics.Wavelength . . . 116
2.77 Physlib.Particles.SuperSymmetry.N1.Basic . . . 118
2.78 Physlib.Particles.SuperSymmetry.N1.Superpotential . . . 119
2.79 Physlib.ProbabilisticTheory.Channel.Basic . . . 119
2.80 Physlib.ProbabilisticTheory.Channel.Covariant . . . 121
2.81 Physlib.ProbabilisticTheory.Channel.Reversible . . . 122
2.82 Physlib.ProbabilisticTheory.Effect.Complement . . . 124
2.83 Physlib.ProbabilisticTheory.Effect.Metric . . . 124
2.84 Physlib.ProbabilisticTheory.LocalNet.Basic . . . 124
2.85 Physlib.ProbabilisticTheory.LocalNet.QuasilocalOrder . . . 127
2.86 Physlib.ProbabilisticTheory.State.Basic . . . 129
2.87 Physlib.ProbabilisticTheory.State.Convex . . . 130
2.88 Physlib.ProbabilisticTheory.State.Discrimination . . . 131
2.89 Physlib.ProbabilisticTheory.State.Local . . . 133
2.90 Physlib.ProbabilisticTheory.State.Metric . . . 135
2.91 Physlib.ProbabilisticTheory.State.Pairing . . . 137
2.92 Physlib.ProbabilisticTheory.State.Separation . . . 138
2.93 Physlib.ProbabilisticTheory.State.WeightEquivalence . . . 139
2.94 Physlib.ProbabilisticTheory.Weight.Basic . . . 140
2.95 Physlib.ProbabilisticTheory.Weight.Extension . . . 142
2.96 Physlib.QuantumMechanics.Blackbody.PlancksLaw . . . 143
2.97 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.MomentumStates . . . 146
2.98 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.PositionStates . . . 147
2.99 Physlib.QuantumMechanics.HilbertSpaces.TensorProducts.CompleteTensorProduct 147
2.100 Physlib.QuantumMechanics.Hydrogen.Basic . . . 148
2.101 Physlib.QuantumMechanics.Hydrogen.LaplaceRungeLenzVector . . . 149
2.102 Physlib.QuantumMechanics.Operators.AngularMomentum . . . 149
2.103 Physlib.QuantumMechanics.Operators.AngularMomentum.DimThree . . . 151
2.104 Physlib.QuantumMechanics.Operators.Multiplication . . . 156
2.105 Physlib.QuantumMechanics.Operators.SpectralTheory.SpectralMeasure . . 156
2.106 Physlib.QuantumMechanics.PoschlTeller.Scattering . . . 156
2.107 Physlib.Relativity.Causality.TimeLike . . . 158
2.108 Physlib.Relativity.CliffordAlgebra . . . 159
2.109 Physlib.Relativity.Fermions.Weyl.Contraction . . . 160
2.110 Physlib.Relativity.Fermions.Weyl.DualLeftHanded . . . 165
2.111 Physlib.Relativity.Fermions.Weyl.DualRightHanded . . . 166
2.112 Physlib.Relativity.Fermions.Weyl.LeftHanded . . . 167
2.113 Physlib.Relativity.Fermions.Weyl.Metric . . . 168

2.114 Physlib.Relativity.Fermions.Weyl.RightHanded . . . 171
2.115 Physlib.Relativity.Fermions.Weyl.Unit . . . 172
2.116 Physlib.Relativity.LorentzAlgebra.Basis . . . 175
2.117 Physlib.Relativity.LorentzGroup.Basic . . . 175
2.118 Physlib.Relativity.LorentzGroup.Boosts.Basic . . . 176
2.119 Physlib.Relativity.LorentzGroup.Restricted.FromBoostRotation . . . 176
2.120 Physlib.Relativity.PauliMatrices.AsTensor . . . 176
2.121 Physlib.Relativity.SL2C.Basic . . . 176
2.122 Physlib.Relativity.SL2C.CayleyKlein . . . 176
2.123 Physlib.Relativity.SL2C.Cover . . . 177
2.124 Physlib.Relativity.SpeedOfLight . . . 178
2.125 Physlib.Relativity.Tensors.Basic . . . 179
2.126 Physlib.Relativity.Tensors.ComplexTensor.Basic . . . 179
2.127 Physlib.Relativity.Tensors.ComplexTensor.Metrics.Basic . . . 181
2.128 Physlib.Relativity.Tensors.ComplexTensor.Metrics.Pre . . . 182
2.129 Physlib.Relativity.Tensors.ComplexTensor.Units.Basic . . . 183
2.130 Physlib.Relativity.Tensors.ComplexTensor.Units.Pre . . . 184
2.131 Physlib.Relativity.Tensors.ComplexTensor.Vector.Basic . . . 186
2.132 Physlib.Relativity.Tensors.ComplexTensor.Vector.Contraction . . . 189
2.133 Physlib.Relativity.Tensors.ComplexTensor.Vector.Modules . . . 190
2.134 Physlib.Relativity.Tensors.ComponentIdx.Insert . . . 193
2.135 Physlib.Relativity.Tensors.ComponentIdx.Pair . . . 194
2.136 Physlib.Relativity.Tensors.ComponentIdx.Product . . . 195
2.137 Physlib.Relativity.Tensors.Conjugation.Basic . . . 195
2.138 Physlib.Relativity.Tensors.Conjugation.Inverse . . . 199
2.139 Physlib.Relativity.Tensors.Constructors . . . 201
2.140 Physlib.Relativity.Tensors.Contraction.Basis . . . 202
2.141 Physlib.Relativity.Tensors.Contraction.CrossToEnd . . . 203
2.142 Physlib.Relativity.Tensors.Contraction.CrossToSlot . . . 205
2.143 Physlib.Relativity.Tensors.Contraction.RankTwo . . . 205
2.144 Physlib.Relativity.Tensors.Contraction.UnitTensorContraction . . . 207
2.145 Physlib.Relativity.Tensors.Dual . . . 207
2.146 Physlib.Relativity.Tensors.RealTensor.Basic . . . 208
2.147 Physlib.Relativity.Tensors.RealTensor.Metrics.Pre . . . 209
2.148 Physlib.Relativity.Tensors.RealTensor.ToComplex . . . 210
2.149 Physlib.Relativity.Tensors.RealTensor.Units.Pre . . . 211
2.150 Physlib.Relativity.Tensors.RealTensor.Vector.Pre.Contraction . . . 213
2.151 Physlib.Relativity.Tensors.RealTensor.Velocity.Basic . . . 214
2.152 Physlib.Relativity.Tensors.RealTensor.Velocity.Deriv . . . 214
2.153 Physlib.Relativity.Tensors.Reindexing . . . 215
2.154 Physlib.Relativity.Tensors.TensorSpecies.Basic . . . 215
2.155 Physlib.Relativity.Tensors.TensorSpecies.DualBasis . . . 216
2.156 Physlib.Relativity.Tensors.UnitTensor . . . 218
2.157 Physlib.SpaceAndTime.Space.Derivatives.Laplacian . . . 219
2.158 Physlib.SpaceAndTime.Space.IsDistBounded . . . 220
2.159 Physlib.SpaceAndTime.SpaceTime.Derivatives . . . 220
2.160 Physlib.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Basic . . . 222
2.161 Physlib.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Star . . . 224
2.162 Physlib.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.TaylorSeries . . . 225
2.163 Physlib.SpaceAndTime.SpaceTime.SpaceTimeDerivAlgebra.Action . . . 226
2.164 Physlib.SpaceAndTime.SpaceTime.SpaceTimeDerivAlgebra.Basic . . . 227

2.165 Physlib.SpaceAndTime.Time.Basic 230
2.166 Physlib.SpaceAndTime.Time.Derivatives 230
2.167 Physlib.SpaceAndTime.Time.ManifoldCurves 230
2.168 Physlib.SpaceAndTime.WaveVectorSpace.Basic 232
2.169 Physlib.Units.Basic 234
2.170 Physlib.Units.Constants 236
2.171 Physlib.Units.PositiveRealUnit 236
2.172 Physlib.Units.UnitDependent 237
2.173 Physlib.Units.WithDim.Basic 238
2.174 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Basic 239
2.175 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.Basic241
2.176 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.FieldStrength242
2.177 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.GaugeAction247
2.178 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.GaugeField252
2.179 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.JetDeriv253
2.180 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.LorentzAction257
2.181 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.MassDim260
2.182 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.MassWeightPoly261
2.183 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.Realization264
2.184 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGaugeCov.Basic268
2.185 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGaugeCov.Realization273
2.186 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Realization.Basic277
2.187 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Realization.FieldStrength278
2.188 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Realization.GaugeLaw281
2.189 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Realization.Symmetrized289
2.190 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Realization.TransformsAdjoint3
2.191 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.Basic . . 305
2.192 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.BosonGenerators306
2.193 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.BosonMatterField311
2.194 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.BosonModule315
2.195 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.FermionGenerators316
2.196 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.FermionMatterField321
2.197 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.FermionModule325
2.198 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.Basic . . 326
2.199 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.GaugeSector334
2.200 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.GaugeSectorRealization336
2.201 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.Realization338
2.202 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.Sector . . 345
2.203 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.SectorRealization351
2.204 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.Basic 356
2.205 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.CovariantDeriv367
2.206 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.GaugeAction 374
2.207 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.GaugeSector 377
2.208 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.GaugeSectorRealization379
2.209 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.Jet 381
2.210 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.JetRep . . . 385
2.211 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.LorentzAction390
2.212 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.MassWeight . 393
2.213 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.Realization 397
2.214 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.Sector . . . 402
2.215 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.SectorRealization407

2.216 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.TransformsIn 410
2.217 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.AdjointCoeff415
2.218 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.Basic . . 418
2.219 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.Factor . 420
2.220 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.MatrixJets422
2.221 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.MaurerCartan426
2.222 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.OfFactors428
2.223 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.Prod . . 434
2.224 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Adjoint436
2.225 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Algebra440
2.226 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Basic 441
2.227 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.GellMann446
2.228 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Invariants.Adjoint451
2.229 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Invariants.Basic454
2.230 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Invariants.BiFund461
2.231 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Invariants.FundAnti467
2.232 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Invariants.QuadFund469
2.233 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.TensorSpecies472
2.234 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.Truncation477
2.235 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.U1 . . . 482
2.236 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.Basic . . . 486
2.237 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.Charge . . . 486
2.238 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.CovariantDeriv488
2.239 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.InfinitesimalAction495
2.240 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.JetComponent.Basic499
2.241 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.JetComponent.GaugeAction506
2.242 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.JetComponent.TransformsIn510
2.243 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.LorentzCovariantDeriv512
2.244 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.MatrixRep.Basic515
2.245 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.MatrixRep.Constructions523
2.246 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.MatrixRep.Factors524
2.247 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.MatrixRep.Table525
2.248 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.Pi 535
2.249 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.Prod 538
2.250 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.TransformsIn540
2.251 PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Criterion . . . 542
2.252 PhyslibAlpha.ClassicalMechanics.CentralForce.AngularMomentum 542
2.253 PhyslibAlpha.ClassicalMechanics.CentralForce.Basic 545
2.254 PhyslibAlpha.ClassicalMechanics.LiouvilleIntegrable.Basic 546
2.255 PhyslibAlpha.CondensedMatter.Bloch.Algebraic 547
2.256 PhyslibAlpha.CondensedMatter.Bloch.FiniteBloch 550
2.257 PhyslibAlpha.CondensedMatter.Bloch.Hypercubic 551
2.258 PhyslibAlpha.CondensedMatter.Bloch.PointSymmetry 554
2.259 PhyslibAlpha.CondensedMatter.Bloch.RegularRep 559
2.260 PhyslibAlpha.CondensedMatter.Bloch.SiteRepresentation 562
2.261 PhyslibAlpha.CondensedMatter.Bloch.Spectrum 564
2.262 PhyslibAlpha.CondensedMatter.Crystal.Balls 566
2.263 PhyslibAlpha.CondensedMatter.Crystal.Basic 567
2.264 PhyslibAlpha.CondensedMatter.Crystal.Brillouin 568
2.265 PhyslibAlpha.CondensedMatter.Crystal.BrillouinCoordinates 570
2.266 PhyslibAlpha.CondensedMatter.Crystal.Examples.Basic 572

2.267 PhyslibAlpha.CondensedMatter.Crystal.Examples.Monoclinic 579
2.268 PhyslibAlpha.CondensedMatter.Crystal.Examples.Orthorhombic 581
2.269 PhyslibAlpha.CondensedMatter.Crystal.Examples.Rhombohedral 584
2.270 PhyslibAlpha.CondensedMatter.Crystal.Examples.TetragonalHexagonal . . 584
2.271 PhyslibAlpha.CondensedMatter.Crystal.Examples.ThreeDimensional 586
2.272 PhyslibAlpha.CondensedMatter.Crystal.Folner 591
2.273 PhyslibAlpha.CondensedMatter.Crystal.Geometry 592
2.274 PhyslibAlpha.CondensedMatter.Crystal.Graph 594
2.275 PhyslibAlpha.CondensedMatter.Crystal.InvariantState 597
2.276 PhyslibAlpha.CondensedMatter.Crystal.InvariantStateUncertainty 598
2.277 PhyslibAlpha.CondensedMatter.Crystal.ManyBody 599
2.278 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Basic 600
2.279 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Count 601
2.280 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Cubic 603
2.281 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Hypercubic 604
2.282 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Monoclinic 606
2.283 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Orthorhombic 607
2.284 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Rhombohedral 609
2.285 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Sublattices 610
2.286 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.TetragonalHexagonal . 612
2.287 PhyslibAlpha.CondensedMatter.Crystal.QuasiLocal 615
2.288 PhyslibAlpha.CondensedMatter.Crystal.Reciprocal 615
2.289 PhyslibAlpha.CondensedMatter.Crystal.Symmetry 617
2.290 PhyslibAlpha.CondensedMatter.IsingModel.Basic 619
2.291 PhyslibAlpha.CondensedMatter.IsingModel.Examples 623
2.292 PhyslibAlpha.CondensedMatter.QuasiLocal.Gibbs 628
2.293 PhyslibAlpha.CondensedMatter.QuasiLocal.GibbsSymmetry 631
2.294 PhyslibAlpha.CondensedMatter.QuasiLocal.Instances 633
2.295 PhyslibAlpha.CondensedMatter.QuasiLocal.Interaction 634
2.296 PhyslibAlpha.CondensedMatter.QuasiLocal.InvariantLimit 635
2.297 PhyslibAlpha.CondensedMatter.QuasiLocal.LimitState 636
2.298 PhyslibAlpha.CondensedMatter.QuasiLocal.Local 637
2.299 PhyslibAlpha.CondensedMatter.QuasiLocal.Symmetry 640
2.300 PhyslibAlpha.CondensedMatter.TightBindingChain.Cube.AxisCommutation . 642
2.301 PhyslibAlpha.CondensedMatter.TightBindingChain.Cube.Basic 643
2.302 PhyslibAlpha.CondensedMatter.TightBindingChain.Cube.MandelstamTamm . . 650
2.303 PhyslibAlpha.CondensedMatter.TightBindingChain.Cube.VolumetricQuantum 651
2.304 PhyslibAlpha.CondensedMatter.TightBindingChain.Current.Basic 652
2.305 PhyslibAlpha.CondensedMatter.TightBindingChain.Current.Eigenstates . . 655
2.306 PhyslibAlpha.CondensedMatter.TightBindingChain.Current.MandelstamTamm 656
2.307 PhyslibAlpha.CondensedMatter.TightBindingChain.Current.SpeedLimit . . 657
2.308 PhyslibAlpha.CondensedMatter.TightBindingChain.Current.TimeReversal . 659
2.309 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.Basic 660
2.310 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.FluctuationAngle663
2.311 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.LongChainLimit664
2.312 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.LongChainMonotonicity665
2.313 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.MandelstamTamm667
2.314 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.Saturation667
2.315 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.UncertaintyRatio669
2.316 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.Variances670
2.317 PhyslibAlpha.CondensedMatter.TightBindingChain.Open.LocalizedUncertainty671

2.318 PhyslibAlpha.CondensedMatter.TightBindingChain.VelocityBand.Basic . . 673
2.319 PhyslibAlpha.CondensedMatter.TightBindingChain.VelocityBand.Width . . 675
2.320 PhyslibAlpha.Electromagnetism.BoxChargeConservation 676
2.321 PhyslibAlpha.Electromagnetism.TemperedDistributional.WireJunction . . 678
2.322 PhyslibAlpha.Mathematics.AlgebraGeneration 680
2.323 PhyslibAlpha.Mathematics.AlgebraRepresentation 683
2.324 PhyslibAlpha.Mathematics.ExteriorAlgebra 688
2.325 PhyslibAlpha.Mathematics.ForMathlib.DataStructures.Matrix.Scalar . . . 692
2.326 PhyslibAlpha.Mathematics.ForMathlib.Fin 693
2.327 PhyslibAlpha.Mathematics.HomogeneousGenerators 693
2.328 PhyslibAlpha.Mathematics.InvariantReduction 696
2.329 PhyslibAlpha.Mathematics.LeviCivita.Basic 702
2.330 PhyslibAlpha.Mathematics.LieAlgebraUnit 703
2.331 PhyslibAlpha.Mathematics.LinearCombination 703
2.332 PhyslibAlpha.Mathematics.MeasureTheory.Pi 705
2.333 PhyslibAlpha.Mathematics.Modules.ConjModule 705
2.334 PhyslibAlpha.Mathematics.MultisetAntidiagonal 709
2.335 PhyslibAlpha.Mathematics.MvPolynomialTranslation 711
2.336 PhyslibAlpha.Mathematics.PolynomialEval 712
2.337 PhyslibAlpha.Mathematics.RepresentationDual 714
2.338 PhyslibAlpha.Mathematics.RepresentationProdMap 714
2.339 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.Basic 715
2.340 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.BivariateBasis . . . 716
2.341 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.Classification . . . 725
2.342 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.ClebschGordan . . . 730
2.343 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.CommutingActions . . 731
2.344 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.Construction 731
2.345 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.Schur 737
2.346 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.TensorDecomposition 739
2.347 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.TensorHighestWeight 743
2.348 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.TensorIntertwiner . 746
2.349 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.Weights 749
2.350 PhyslibAlpha.Mathematics.SpecialFunctions.Laguerre 751
2.351 PhyslibAlpha.Mathematics.SpecialFunctions.Walsh 754
2.352 PhyslibAlpha.Mathematics.SubalgebraRestriction 756
2.353 PhyslibAlpha.Mathematics.SymmetricAlgebra 757
2.354 PhyslibAlpha.Mathematics.TauCeti.Analysis.Bochner.DiscreteGroup . . . 762
2.355 PhyslibAlpha.Mathematics.TauCeti.Analysis.Bochner.LocallyCompactGroup 762
2.356 PhyslibAlpha.Mathematics.TauCeti.Analysis.CStarAlgebra.CharacterSpaceMeasure764
2.357 PhyslibAlpha.Mathematics.TauCeti.Analysis.CStarAlgebra.UnitaryCharacter765
2.358 PhyslibAlpha.Mathematics.TauCeti.Analysis.Fourier.Pontryagin.Continuity766
2.359 PhyslibAlpha.Mathematics.TauCeti.Analysis.Fourier.Pontryagin.DualEval 766
2.360 PhyslibAlpha.Mathematics.TauCeti.Analysis.Fourier.Pontryagin.Measure 766
2.361 PhyslibAlpha.Mathematics.TauCeti.Analysis.Fourier.Pontryagin.StronglyContinuous768
2.362 PhyslibAlpha.Mathematics.TauCeti.Analysis.Fourier.Pontryagin.Uniqueness770
2.363 PhyslibAlpha.Mathematics.TauCeti.Analysis.InnerProductSpace.BasisSpan 774
2.364 PhyslibAlpha.Mathematics.TauCeti.Analysis.InnerProductSpace.CourantFischer775
2.365 PhyslibAlpha.Mathematics.TauCeti.Analysis.InnerProductSpace.LinearCombination781
2.366 PhyslibAlpha.Mathematics.TauCeti.Analysis.Matrix.PosSemidef 781
2.367 PhyslibAlpha.Mathematics.TauCeti.Analysis.Normed.Operator.Dense . . . 782
2.368 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.AddGroup . 783

2.369 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.AdditiveCharacter789
2.370 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Basic . . . 790
2.371 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Continuity 793
2.372 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Function.Closure796
2.373 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Function.GNS799
2.374 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Function.Kernel802
2.375 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Kernel.Kolmogorov802
2.376 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Limits . . 805
2.377 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Normalize 806
2.378 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.PontryaginMeasure807
2.379 PhyslibAlpha.Mathematics.TauCeti.LinearAlgebra.Dimension.Submodule . . 807
2.380 PhyslibAlpha.Mathematics.TauCeti.LinearAlgebra.Matrix.PosSemidef . . . 808
2.381 PhyslibAlpha.Mathematics.TauCeti.MeasureTheory.Function.StronglyMeasurable.InnerRegular
2.382 PhyslibAlpha.Mathematics.TauCeti.MeasureTheory.Integral.PeakFunction 810
2.383 PhyslibAlpha.Mathematics.TauCeti.MeasureTheory.Measure.FiniteMeasureExt810
2.384 PhyslibAlpha.Mathematics.TauCeti.MeasureTheory.Measure.Regular 812
2.385 PhyslibAlpha.Mathematics.TauCeti.Order.Filter.SmallSets 812
2.386 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Integrated.Algebra812
2.387 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Integrated.Basic815
2.388 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Pontryagin.Basic818
2.389 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Pontryagin.Continuity8
2.390 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Subrepresentation820
2.391 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Unitary.Basic822
2.392 PhyslibAlpha.Mathematics.TauCeti.Topology.Algebra.ContinuousMonoidHom.Polish825
2.393 PhyslibAlpha.Mathematics.TauCeti.Topology.Algebra.PontryaginDual . . . 825
2.394 PhyslibAlpha.Mathematics.TensorProductComm 826
2.395 PhyslibAlpha.Particles.QED.Basic 826
2.396 PhyslibAlpha.Particles.QED.CurrentCoupling 847
2.397 PhyslibAlpha.Particles.QED.Evaluation 847
2.398 PhyslibAlpha.Particles.QED.FermionStatistics 851
2.399 PhyslibAlpha.Particles.QED.Fields 852
2.400 PhyslibAlpha.Particles.QED.FieldStrength 854
2.401 PhyslibAlpha.Particles.QED.GammaMatrices 855
2.402 PhyslibAlpha.Particles.QED.GaugeInvariance 857
2.403 PhyslibAlpha.Particles.QED.JetCompleteness 860
2.404 PhyslibAlpha.Particles.QED.Lagrangian 862
2.405 PhyslibAlpha.Particles.QED.LorentzInvariance 863
2.406 PhyslibAlpha.Particles.QED.MassDimension 867
2.407 PhyslibAlpha.Particles.StandardModel.Basic 869
2.408 PhyslibAlpha.Particles.StandardModel.CovRealization.Basic 871
2.409 PhyslibAlpha.Particles.StandardModel.CovRealization.FermionGaugeSector.Basic887
2.410 PhyslibAlpha.Particles.StandardModel.CovRealization.FermionGaugeSector.MassWeight888
2.411 PhyslibAlpha.Particles.StandardModel.CovRealization.GaugeHiggsSector.Basic890
2.412 PhyslibAlpha.Particles.StandardModel.CovRealization.GaugeHiggsSector.MassWeight891
2.413 PhyslibAlpha.Particles.StandardModel.CovRealization.Generators 895
2.414 PhyslibAlpha.Particles.StandardModel.CovRealization.MassWeight 902
2.415 PhyslibAlpha.Particles.StandardModel.CovRealization.MassWeight.Filtration907
2.416 PhyslibAlpha.Particles.StandardModel.CovRealization.MassWeight.Invariants912
2.417 PhyslibAlpha.Particles.StandardModel.CovRealization.MixedSector.Basic 919
2.418 PhyslibAlpha.Particles.StandardModel.CovRealization.Sectors 920
2.419 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.Basic928

2.420 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.Families.BarHiggs 930
2.421 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.Families.Higgs 940
2.422 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.Families.Symbols 951
2.423 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.GaugeWeight 962
2.424 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.MassDimEight 966
2.425 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.MassDimLTEight 974
2.426 PhyslibAlpha.Particles.StandardModel.Fermions.DownSinglet.Basic . . . 976
2.427 PhyslibAlpha.Particles.StandardModel.Fermions.DownSinglet.GaugeAlgebraAction 980
2.428 PhyslibAlpha.Particles.StandardModel.Fermions.JetAlgebra.Basic 984
2.429 PhyslibAlpha.Particles.StandardModel.Fermions.JetAlgebra.Species . . . 991
2.430 PhyslibAlpha.Particles.StandardModel.Fermions.LeptonDoublet.Basic . . 998
2.431 PhyslibAlpha.Particles.StandardModel.Fermions.LeptonDoublet.GaugeAlgebraAction 1003
2.432 PhyslibAlpha.Particles.StandardModel.Fermions.LeptonSinglet.Basic . . 1006
2.433 PhyslibAlpha.Particles.StandardModel.Fermions.LeptonSinglet.GaugeAlgebraAction 1010
2.434 PhyslibAlpha.Particles.StandardModel.Fermions.MatterField 1012
2.435 PhyslibAlpha.Particles.StandardModel.Fermions.QuarkDoublet.Basic . . . 1017
2.436 PhyslibAlpha.Particles.StandardModel.Fermions.QuarkDoublet.GaugeAlgebraAction 1022
2.437 PhyslibAlpha.Particles.StandardModel.Fermions.UpSinglet.Basic 1026
2.438 PhyslibAlpha.Particles.StandardModel.Fermions.UpSinglet.GaugeAlgebraAction 1030
2.439 PhyslibAlpha.Particles.StandardModel.FieldData 1034
2.440 PhyslibAlpha.Particles.StandardModel.GaugeAlgebra.Basic 1038
2.441 PhyslibAlpha.Particles.StandardModel.GaugeAlgebra.Basis 1043
2.442 PhyslibAlpha.Particles.StandardModel.GaugeAlgebra.JetGaugeAlgebra . . 1053
2.443 PhyslibAlpha.Particles.StandardModel.GaugeAlgebra.RootDecomposition . 1063
2.444 PhyslibAlpha.Particles.StandardModel.GaugeGroup.AdjointMatrix 1072
2.445 PhyslibAlpha.Particles.StandardModel.GaugeGroup.Basic 1073
2.446 PhyslibAlpha.Particles.StandardModel.GaugeGroup.GaugeWeight 1074
2.447 PhyslibAlpha.Particles.StandardModel.GaugeGroup.Invariants.Adjoint . . 1082
2.448 PhyslibAlpha.Particles.StandardModel.GaugeGroup.Invariants.Basic . . . 1088
2.449 PhyslibAlpha.Particles.StandardModel.GaugeGroup.Invariants.IsU1BiAdjoint 1095
2.450 PhyslibAlpha.Particles.StandardModel.GaugeGroup.JetGaugeGroup.Basic . 1096
2.451 PhyslibAlpha.Particles.StandardModel.GaugeGroup.JetGaugeGroup.Truncation 1099
2.452 PhyslibAlpha.Particles.StandardModel.GaugeGroup.LocalGaugeData . . . 1100
2.453 PhyslibAlpha.Particles.StandardModel.GaugeGroup.MaurerCartan.Basic . . 1103
2.454 PhyslibAlpha.Particles.StandardModel.GaugeGroup.SU2Conjugation . . . 1105
2.455 PhyslibAlpha.Particles.StandardModel.HiggsBoson.GaugeAlgebraAction . . 1105
2.456 PhyslibAlpha.Particles.StandardModel.HiggsBoson.JetAlgebra.Algebra . . 1109
2.457 PhyslibAlpha.Particles.StandardModel.HiggsBoson.JetAlgebra.Basic . . . 1110
2.458 PhyslibAlpha.Particles.StandardModel.HiggsBoson.MatterField 1112
2.459 PhyslibAlpha.Particles.StandardModel.InvariantReduction 1113
2.460 PhyslibAlpha.Particles.StandardModel.IsFermionSector.Basic 1116
2.461 PhyslibAlpha.Particles.StandardModel.IsFermionSector.Components . . . 1119
2.462 PhyslibAlpha.Particles.StandardModel.IsFermionSector.Deriv.Centre . . . 1127
2.463 PhyslibAlpha.Particles.StandardModel.IsFermionSector.Deriv.GaugeWeight 1128
2.464 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.Basic 1134
2.465 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.GaugeWeight 1137
2.466 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.KineticFamilies 1145
2.467 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.KineticTerms 1155
2.468 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.MassDimEight 1172
2.469 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.MassDimLTEight 1185
2.470 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.Basic 1192

2.471 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.Deriv.Centre . . .1195
2.472 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.Deriv.GaugeWeight 1196
2.473 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.MassWeight.Basic .1199
2.474 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.MassWeight.GaugeWeight1202
2.475 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.MassWeight.MassDimEight1206
2.476 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.MassWeight.MassDimLTEight1226
2.477 PhyslibAlpha.Particles.StandardModel.JetAlgebra.AlgebraRealization . .1229
2.478 PhyslibAlpha.Particles.StandardModel.JetAlgebra.Basic 1232
2.479 PhyslibAlpha.Particles.StandardModel.JetAlgebra.CovJetAlgebra.Basic .1234
2.480 PhyslibAlpha.Particles.StandardModel.JetAlgebra.CovJetAlgebra.Higgs .1244
2.481 PhyslibAlpha.Particles.StandardModel.JetAlgebra.CovJetAlgebra.Sectors1249
2.482 PhyslibAlpha.Particles.StandardModel.JetAlgebra.FieldAlgebra 1255
2.483 PhyslibAlpha.Particles.StandardModel.JetAlgebra.GaugeAction 1261
2.484 PhyslibAlpha.Particles.StandardModel.JetAlgebra.Generators 1264
2.485 PhyslibAlpha.Particles.StandardModel.JetAlgebra.Invariants 1276
2.486 PhyslibAlpha.Particles.StandardModel.JetAlgebra.JetDeriv 1278
2.487 PhyslibAlpha.Particles.StandardModel.JetAlgebra.LorentzAction 1283
2.488 PhyslibAlpha.Particles.StandardModel.JetAlgebra.MassWeightPoly . . .1286
2.489 PhyslibAlpha.Particles.StandardModel.JetAlgebra.Realization 1292
2.490 PhyslibAlpha.Particles.StandardModel.JetAlgebra.SectorEquiv.Basic . .1292
2.491 PhyslibAlpha.Particles.StandardModel.JetAlgebra.SectorEquiv.Structure1298
2.492 PhyslibAlpha.Particles.StandardModel.JetAlgebra.TransformsIn 1303
2.493 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.Basic 1308
2.494 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.GaugeAction . . .1310
2.495 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.JetDeriv 1313
2.496 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.LorentzAction . .1316
2.497 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.MassDim 1318
2.498 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.MassWeightPoly . .1319
2.499 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.Prod 1322
2.500 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.TransformsIn . . .1323
2.501 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.Basic 1324
2.502 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.GaugeAction . .1326
2.503 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.JetDeriv 1328
2.504 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.LorentzAction .1333
2.505 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.MassDim 1335
2.506 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.MassWeightPoly 1336
2.507 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.Prod 1340
2.508 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.TransformsIn . .1340
2.509 PhyslibAlpha.Particles.StandardModel.Model.Consistency 1342
2.510 PhyslibAlpha.Particles.StandardModel.Model.LeptonDoublet 1342
2.511 PhyslibAlpha.Particles.StandardModel.Realization.Basic 1342
2.512 PhyslibAlpha.Particles.StandardModel.Realization.Commutations 1349
2.513 PhyslibAlpha.Particles.StandardModel.Realization.CovariantDeriv . . .1364
2.514 PhyslibAlpha.Particles.StandardModel.Realization.CovFieldAlgebra.Basic1384
2.515 PhyslibAlpha.Particles.StandardModel.Realization.CovStandardModel . .1385
2.516 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.Basic . . .1407
2.517 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.DerivSubmodule.Basic1419
2.518 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.DerivSubmodule.Centre1420
2.519 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.DerivSubmodule.GaugeWeight1421
2.520 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.MassWeight.Basic1421
2.521 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.MassWeight.GaugeWeight1422

2.522 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.MassWeight.MassDimEight 1434
2.523 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.MassWeight.MassDimLTEight 1437
2.524 PhyslibAlpha.Particles.StandardModel.Realization.MassWeight.Basic . . 1440
2.525 PhyslibAlpha.Particles.StandardModel.Realization.MassWeight.Filtration 1443
2.526 PhyslibAlpha.ProbabilisticTheory.CStarAlgebra.GNSAutomorphism 1447
2.527 PhyslibAlpha.ProbabilisticTheory.CStarAlgebra.GNSSpectral 1450
2.528 PhyslibAlpha.ProbabilisticTheory.CStarAlgebra.Uncertainty 1450
2.529 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.JointSpectral.Basic . . 1453
2.530 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.OrthonormalMeasurement 1455
2.531 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.State.Vector 1457
2.532 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.State.VectorUncertainty 1458
2.533 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.Basic 1459
2.534 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.BoundedIntegralCommute 1459
2.535 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.ContinuousCalculus 1459
2.536 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.StarHomSpec.Commutant 1460
2.537 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.StarHomSpec.Functional 1461
2.538 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.StarHomSpec.SesquilinearForm 1463
2.539 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.StarHomSpec.SpectralMeasure 1470
2.540 PhyslibAlpha.ProbabilisticTheory.QuasiLocal.AlgebraNet 1476
2.541 PhyslibAlpha.ProbabilisticTheory.QuasiLocal.Limit 1477
2.542 PhyslibAlpha.ProbabilisticTheory.QuasiLocal.State 1479
2.543 PhyslibAlpha.ProbabilisticTheory.Representation.AbelianSpectral . . 1480
2.544 PhyslibAlpha.ProbabilisticTheory.Representation.Bands 1482
2.545 PhyslibAlpha.ProbabilisticTheory.Representation.Commutant 1483
2.546 PhyslibAlpha.ProbabilisticTheory.Representation.Floquet 1485
2.547 PhyslibAlpha.ProbabilisticTheory.Representation.Hopping 1488
2.548 PhyslibAlpha.ProbabilisticTheory.Representation.Koopman 1489
2.549 PhyslibAlpha.ProbabilisticTheory.Representation.MomentumSectors . . 1491
2.550 PhyslibAlpha.ProbabilisticTheory.Representation.Orthogonality 1494
2.551 PhyslibAlpha.ProbabilisticTheory.Representation.PointCovariance . . 1494
2.552 PhyslibAlpha.ProbabilisticTheory.Representation.SectorReduction . . 1496
2.553 PhyslibAlpha.ProbabilisticTheory.Representation.StructureFactor . . 1498
2.554 PhyslibAlpha.ProbabilisticTheory.Representation.VectorMeasure 1499
2.555 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.ComplexExtension 1500
2.556 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.CovarianceForm 1501
2.557 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.CovarianceFormUnitalMap 1504
2.558 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.GNS 1506
2.559 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.Invariant 1510
2.560 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.StateUncertainty 1513
2.561 PhyslibAlpha.ProbabilisticTheory.State.Metric 1515
2.562 PhyslibAlpha.QuantumMechanics.AngularMomentum.Addition 1516
2.563 PhyslibAlpha.QuantumMechanics.AngularMomentum.Basic 1519
2.564 PhyslibAlpha.QuantumMechanics.AngularMomentum.ClebschGordanCoefficients 1525
2.565 PhyslibAlpha.QuantumMechanics.AngularMomentum.ClebschGordanFormula . 1530
2.566 PhyslibAlpha.QuantumMechanics.AngularMomentum.CouplingSchemes 1535
2.567 PhyslibAlpha.QuantumMechanics.AngularMomentum.MultipoleSumRule . . . 1541
2.568 PhyslibAlpha.QuantumMechanics.AngularMomentum.NormalizedBasis 1541
2.569 PhyslibAlpha.QuantumMechanics.AngularMomentum.ProjectionTheorem . . 1543
2.570 PhyslibAlpha.QuantumMechanics.AngularMomentum.SelectionRules 1545
2.571 PhyslibAlpha.QuantumMechanics.AngularMomentum.SixJ 1548
2.572 PhyslibAlpha.QuantumMechanics.AngularMomentum.SixJSymmetry 1551

2.573 PhyslibAlpha.QuantumMechanics.AngularMomentum.SpinOrbit 1557
2.574 PhyslibAlpha.QuantumMechanics.AngularMomentum.Wigner3j 1558
2.575 PhyslibAlpha.QuantumMechanics.AngularMomentum.WignerEckart 1563
2.576 PhyslibAlpha.QuantumMechanics.AngularMomentum.WignerEckartIrreducible1565
2.577 PhyslibAlpha.QuantumMechanics.AngularMomentum.WignerEckartNormalized 1567
2.578 PhyslibAlpha.QuantumMechanics.Blackbody.RayleighJeans 1569
2.579 PhyslibAlpha.QuantumMechanics.Foundations.OntologicalModel 1570
2.580 PhyslibAlpha.QuantumMechanics.Foundations.PBR.Phases 1572
2.581 PhyslibAlpha.QuantumMechanics.HilbertSpaces.FiniteTarget.Operators . .1573
2.582 PhyslibAlpha.QuantumMechanics.HilbertSpaces.FiniteTarget.Product . . .1573
2.583 PhyslibAlpha.QuantumMechanics.HilbertSpaces.FiniteTarget.ProductState1575
2.584 PhyslibAlpha.QuantumMechanics.Hydrogen.Degeneracy 1579
2.585 PhyslibAlpha.QuantumMechanics.Hydrogen.Eigenfunctions 1584
2.586 PhyslibAlpha.QuantumMechanics.Hydrogen.FineStructure 1587
2.587 PhyslibAlpha.QuantumMechanics.Hydrogen.PauliSpectrum 1589
2.588 PhyslibAlpha.QuantumMechanics.Hydrogen.RadialCompleteness 1593
2.589 PhyslibAlpha.QuantumMechanics.Hydrogen.RadialFunctions 1595
2.590 PhyslibAlpha.QuantumMechanics.Hydrogen.RadialQuantization 1599
2.591 PhyslibAlpha.Relativity.Fermions.Dirac.MinimalDimension 1603
2.592 PhyslibAlpha.Relativity.Fermions.Weyl.BoostWeight 1604
2.593 PhyslibAlpha.Relativity.Fermions.Weyl.Metric 1605
2.594 PhyslibAlpha.Relativity.IsLorentzDeriv 1606
2.595 PhyslibAlpha.Relativity.LightConeDeriv 1609
2.596 PhyslibAlpha.Relativity.LorentzGroup.Basic 1610
2.597 PhyslibAlpha.Relativity.LorentzGroup.Boosts.Axis 1611
2.598 PhyslibAlpha.Relativity.LorentzGroup.FermionicParity 1611
2.599 PhyslibAlpha.Relativity.LorentzGroup.Invariants.AdjointClosed 1612
2.600 PhyslibAlpha.Relativity.LorentzGroup.Invariants.Basic 1614
2.601 PhyslibAlpha.Relativity.LorentzGroup.Invariants.Centre 1617
2.602 PhyslibAlpha.Relativity.LorentzGroup.Invariants.IsBiLeftWeyl 1618
2.603 PhyslibAlpha.Relativity.LorentzGroup.Invariants.IsLeftRightWeyl . . .1622
2.604 PhyslibAlpha.Relativity.LorentzGroup.Invariants.IsVectorLeftRightWeyl1623
2.605 PhyslibAlpha.Relativity.LorentzGroup.Invariants.LightCone 1628
2.606 PhyslibAlpha.Relativity.LorentzGroup.Invariants.LorentzCovariance . .1630
2.607 PhyslibAlpha.Relativity.LorentzGroup.Invariants.RankFour 1634
2.608 PhyslibAlpha.Relativity.LorentzGroup.Invariants.RankOne 1641
2.609 PhyslibAlpha.Relativity.LorentzGroup.Invariants.RankThree 1642
2.610 PhyslibAlpha.Relativity.LorentzGroup.Invariants.RankTwo 1643
2.611 PhyslibAlpha.Relativity.LorentzMix 1645
2.612 PhyslibAlpha.Relativity.MinkowskiMatrix 1649
2.613 PhyslibAlpha.Relativity.PauliMatrices.Anticommuting 1650
2.614 PhyslibAlpha.Relativity.PauliMatrices.SelfAdjoint 1651
2.615 PhyslibAlpha.Relativity.PauliMatrices.ToTensor 1652
2.616 PhyslibAlpha.Relativity.SL2C.AxisRotations 1652
2.617 PhyslibAlpha.Relativity.SL2C.Basic 1654
2.618 PhyslibAlpha.Relativity.Tensors.ComplexTensor.Vector.Basic 1655
2.619 PhyslibAlpha.Relativity.Tensors.Equivariant 1655
2.620 PhyslibAlpha.Relativity.Tensors.RealTensor.CoVector.Representation . .1660
2.621 PhyslibAlpha.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Basic 1660
2.622 PhyslibAlpha.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Jacobi 1664
2.623 PhyslibAlpha.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Matrix 1664

2.624 PhyslibAlpha.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Taylor 1669
2.625 PhyslibAlpha.SpaceAndTime.SpaceTime.SpaceTimeDerivAlgebra 1670
2.626 QuantumInfo.ClassicalInfo.Entropy 1679
2.627 QuantumInfo.ClassicalInfo.Prob 1679
2.628 QuantumInfo.ForMathlib.ExchangeRelations 1680
2.629 QuantumInfo.ForMathlib.HermitianMat.LogExp 1681
2.630 QuantumInfo.States.Mixed.MState 1681
2.631 QuantumInfo.States.Pure.Braket 1682
2.632 scripts.alpha.github.pr_summary 1683
2.633 scripts.alpha.lint.required.api_map_lint 1693
2.634 scripts.alpha.website.APIMapIndex 1699
2.635 scripts.dev.get_cache 1701
2.636 scripts.lint.required.auxillary-script-test 1701
2.637 scripts.lint.required.forMathlib_lint 1702
2.638 scripts.lint.required.module_doc_lint 1703
2.639 scripts.lint.required.theorem_lint 1705

1 Introduction

During Oct 2026, 31 contributors submitted 239 commits that changed 195,456 lines (+181,716 / -13,740) across 1,329 files spanning 65 top-level areas of the repository. Of those, 584 files were newly added and 56 were removed outright. This report catalogues 10,965 newly added Lean declarations: 8,085 lemmas, 2,261 defs, 281 instances, 213 abbrevs, 85 structures, 27 inductives, 7 classs, and 6 theorems. We're delighted to recognize the following first-time contributors this month: Aayush Rajasekaran, artmountain, brentius, Cameron Lee, dl-eigenart, dwanith, Emil Kuban, Jaehyeon Shin, Julian Popescu, Michal Mogielnicki, Ruth Wang, Scott Hughes, Sebastian Boehler, SourLen, and Steven Nguyen.

See the full compare diff and the list of commits on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Aayush Rajasekaran	16	1
Andrea Pari	3,155	14
Anoddy	518	1
artmountain	3,281	9
brentius	964	4
Cameron Lee	519	4
dl-eigenart	10,233	29
doxtor6	463	1
dwanith	230	1
Eduardo Nava Hernandez	8,539	43
Emil Kuban	221	2
ereinhardt8	15	1
Eric Wieser	5	1
Gregory Loges	476	3
Jaehyeon Shin	277	7
Joseph Tooby-Smith	133,219	34
Julian Popescu	45	2
Kim Morrison	6	2
Michal Mogielnicki	1,400	6
nateabr	1,715	8
Philippe Emile Christian KEVORKIAN	3,712	17
Rithwik Ranganathan	957	5
Ruth Wang	300	3
Samyak Rai	479	2
Scott Hughes	363	9
Sebastian Boehler	112	1
SourLen	65	1
Steven Nguyen	8	1
Tom Diem	39,493	21
Wouter Deconinck	203	2
Zhi Kai Pong	415	4

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Alex Meiburg	1
Daniel Morrison	19
doxtor6	1
Gregory Loges	8
Joseph Tooby-Smith	198
Michal Mogielnicki	2
nateabr	33
Philippe Emile Christian KEVORKIAN	1
RaunakChhatwal	1
Samyak Rai	1
Tom Diem	2
Wouter Deconinck	1
Zhi Kai Pong	4

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
`PhyslibAlpha/Particles`	139	+131	–	+59,775	-21
`PhyslibAlpha/ClassicalFieldTheory`	96	+77	–	+27,677	-23
`PhyslibAlpha/Mathematics`	105	+73	–	+15,954	-16
`PhyslibAlpha/CondensedMatter`	67	+65	–	+13,785	-17
`PhyslibAlpha/ProbabilisticTheory`	233	+31	–	+10,866	-110
`PhyslibAlpha/QuantumMechanics`	33	+29	–	+10,142	-6
`Physlib/Relativity`	89	+14	-6	+5,919	-2,393
`PhyslibAlpha/Relativity`	31	+30	–	+6,316	-0
`Physlib/Mathematics`	48	+13	-6	+3,386	-1,815
`Physlib/ClassicalMechanics`	39	+21	-7	+2,887	-1,674
`Physlib/QuantumMechanics`	37	+8	-6	+1,936	-822
`Physlib/Cosmology`	16	+9	-2	+2,318	-369
`Physlib/SpaceAndTime`	39	+13	-6	+2,063	-579
`Physlib/ProbabilisticTheory`	20	+16	–	+2,527	-40
`scripts/alpha`	3	+3	–	+2,353	-0
`PhyslibAlpha/SpaceAndTime`	12	+5	–	+2,087	-9
`scripts/lint`	33	+5	–	+1,842	-107
`Physlib/CondensedMatter`	11	+9	–	+1,858	-6
`QuantumInfo/ForMathlib`	43	+1	–	+804	-631
`Physlib/Electromagnetism`	26	+2	-1	+564	-600
`Physlib/Particles`	32	+4	-3	+520	-620
`Physlib/Units`	13	+2	-1	+718	-422
`PhyslibAlpha/ClassicalMechanics`	16	+3	–	+699	-92
`Physlib/Meta`	6	+2	-2	+306	-399
`scripts/MetaPrograms`	3	–	-3	+0	-698
`Physlib/ClassicalFieldTheory`	3	+2	-1	+565	-91
`scripts`	3	–	-2	+73	-494

PhyslibAlpha/Electromagnetism	2	+2	–	+561	-0
Physlib/QFT	25	+2	-2	+243	-265
QuantumInfo/States	9	–	–	+338	-157
PhyslibAlpha.lean	1	–	–	+448	-6
.github/workflows	10	+1	–	+325	-49
docs	4	–	–	+206	-120
Physlib/Optics	2	+2	–	+252	-0
QuantumInfo/ClassicalInfo	5	–	–	+128	-103
Physlib/StatisticalMechanics	7	+1	-1	+101	-114
QuantumInfo/Channels	6	–	–	+100	-110
Physlib/Thermodynamics	3	+1	-1	+92	-110
docs/build	3	+3	–	+182	-0
Physlib.lean	1	–	–	+128	-49
QuantumInfo/Entropy	4	–	–	+77	-79
QuantumInfo/ResourceTheory	3	–	–	+75	-74
Physlib/FluidDynamics	2	+1	-1	+53	-83
QuantumInfo/Capacity	1	–	–	+54	-64
scripts/github	4	+1	–	+95	-15
docs/CuratedNotes	5	–	-5	+0	-105
lakefile.toml	1	–	–	+54	-41
Physlib/LatticeQFT	1	+1	–	+89	-0
lake-manifest.json	1	–	–	+13	-63
scripts/dev	10	–	–	+51	-6
scripts/website	9	–	–	+9	-34
.github	1	+1	–	+38	-0
AGENTS.md	1	–	–	+21	-14
PhyslibAlpha	1	–	–	+24	-1
README.md	1	–	–	+5	-5
.vscode	1	–	–	+0	-9
QuantumInfo/Measurements	1	–	–	+4	-4
.github/ISSUE_TEMPLATE	1	–	–	+2	-1
.gitignore	1	–	–	+2	-1
Physlib/StringTheory	1	–	–	+1	-1
QuantumInfo/Operators	1	–	–	+1	-1
lake-cache.toml	1	–	–	+1	-1
lean-toolchain	1	–	–	+1	-1
.codespellignore	1	–	–	+1	-0
QuantumInfo.lean	1	–	–	+1	-0

1.4 New declarations by kind

Kind	Count
lemma	8,085
def	2,261
instance	281
abbrev	213
structure	85
inductive	27
class	7

theorem	6

2 Details by file

2.1 `Physlib.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.Basic`

Physlib/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/Basic.lean on GitHub.

structure LocalGaugeData [source]

Local gauge data. A gauge group `G₀` with Lie algebra $\mathfrak{g}$, its group of jets `GJ` with Lie algebra of jets $\mathfrak{g}J$, evaluation at the base point, formal derivatives, the adjoint action and the Maurer–Cartan form, subject to the identities used by the transformation laws of gauge and matter fields.

```
structure LocalGaugeData (G₀ : Type) [Group G₀] (𝔤 : Type) [LieRing 𝔤] [LieAlgebra ℝ 𝔤]
    (GJ : Type) [Group GJ] (𝔤J : Type) [LieRing 𝔤J] [LieAlgebra ℝ 𝔤J] where
  eval : GJ →* G₀
  ofConstant : G₀ →* GJ
  eval_ofConstant : ∀ g, eval (ofConstant g) = g
  evalLie : 𝔤J →ₗ⁅ℝ⁆ 𝔤
  ofConstantLie : 𝔤 →ₗ⁅ℝ⁆ 𝔤J
  evalLie_ofConstantLie : ∀ a, evalLie (ofConstantLie a) = a
  deriv : (Fin 1 ⊕ Fin 3) → 𝔤J →ₗ[ℝ] 𝔤J
  deriv_comm : ∀ (μ ν : Fin 1 ⊕ Fin 3) (a : 𝔤J), deriv μ (deriv ν a) = deriv ν (deriv μ a)
  deriv_bracket : ∀ (μ : Fin 1 ⊕ Fin 3) (x y : 𝔤J),
    deriv μ ⁅x, y⁆ = ⁅deriv μ x, y⁆ + ⁅x, deriv μ y⁆
  deriv_ofConstantLie : ∀ (μ : Fin 1 ⊕ Fin 3) (a : 𝔤), deriv μ (ofConstantLie a) = 0
  mulCoord : (Fin 1 ⊕ Fin 3) → 𝔤J →ₗ[ℝ] 𝔤J
  deriv_mulCoord : ∀ (μ ν : Fin 1 ⊕ Fin 3) (a : 𝔤J),
    deriv μ (mulCoord ν a) = mulCoord ν (deriv μ a) + if μ = ν then a else 0
  evalLie_mulCoord : ∀ (μ : Fin 1 ⊕ Fin 3) (a : 𝔤J), evalLie (mulCoord μ a) = 0
  mulCoord_bracket : ∀ (μ : Fin 1 ⊕ Fin 3) (a b : 𝔤J), ⁅mulCoord μ a, b⁆ = mulCoord μ ⁅a, b⁆
  adjointJ : Representation ℝ GJ 𝔤J
  adjointJ_bracket : ∀ (U : GJ) (x y : 𝔤J), adjointJ U ⁅x, y⁆ = ⁅adjointJ U x, adjointJ U y⁆
  adjoint : Representation ℝ G₀ 𝔤
  evalLie_adjointJ : ∀ (U : GJ) (x : 𝔤J), evalLie (adjointJ U x) = adjoint (eval U) (evalLie x)
  maurerCartan : GJ → (Fin 1 ⊕ Fin 3) → 𝔤J
  maurerCartan_ofConstant : ∀ (g : G₀) (μ : Fin 1 ⊕ Fin 3), maurerCartan (ofConstant g) μ = 0
  maurerCartan_cocycle : ∀ (U V : GJ) (μ : Fin 1 ⊕ Fin 3),
    maurerCartan (U * V) μ = maurerCartan U μ + adjointJ U (maurerCartan V μ)
  maurerCartan_structure : ∀ (U : GJ) (μ ν : Fin 1 ⊕ Fin 3),
    deriv μ (maurerCartan U ν) - deriv ν (maurerCartan U μ)
      + ⁅maurerCartan U μ, maurerCartan U ν⁆ = 0
  deriv_adjointJ : ∀ (U : GJ) (μ : Fin 1 ⊕ Fin 3) (x : 𝔤J),
    deriv μ (adjointJ U x) = adjointJ U (deriv μ x) - ⁅maurerCartan U μ, adjointJ U x⁆
```

lemma eval_surjective [source]

Every global gauge transformation is the value of a jet, namely its constant jet.

```
lemma eval_surjective : Function.Surjective jets.eval
```

lemma ofConstant_injective [source]

```
lemma ofConstant_injective : Function.Injective jets.ofConstant
```

lemma evalLie_surjective [source]

```
lemma evalLie_surjective : Function.Surjective jets.evalLie
```

lemma ofConstantLie_injective [source]

```
lemma ofConstantLie_injective : Function.Injective jets.ofConstantLie
```

lemma adjoint_bracket [source]

```
lemma adjoint_bracket (g : G₀) (a b : 𝔤) :
    jets.adjoint g ⁅a, b⁆ = ⁅jets.adjoint g a, jets.adjoint g b⁆
```

lemma deriv_adjointJ_ofConstant [source]

```
lemma deriv_adjointJ_ofConstant (g : G₀) (μ : Fin 1 ⊕ Fin 3) (x : 𝔤J) :
    jets.deriv μ (jets.adjointJ (jets.ofConstant g) x) =
      jets.adjointJ (jets.ofConstant g) (jets.deriv μ x)
```

instance instRightCommutativeCompDeriv [source]

Post-composition with `deriv` is right-commutative.

```
instance instRightCommutativeCompDeriv : RightCommutative
    (fun (D : 𝔤J →ₗ[ℝ] 𝔤J) (μ : Fin 1 ⊕ Fin 3) => D.comp (jets.deriv μ)) where
```

def iteratedDeriv [source]

The iterated formal derivative on the jet Lie algebra.

```
noncomputable def iteratedDeriv (μs : Multiset (Fin 1 ⊕ Fin 3)) : 𝔤J →ₗ[ℝ] 𝔤J
```

lemma iteratedDeriv_zero [source]

```
@[simp]
lemma iteratedDeriv_zero : jets.iteratedDeriv (0 : Multiset (Fin 1 ⊕ Fin 3)) = LinearMap.id
```

lemma iteratedDeriv_cons [source]

```
lemma iteratedDeriv_cons (μ : Fin 1 ⊕ Fin 3) (μs : Multiset (Fin 1 ⊕ Fin 3)) :
    jets.iteratedDeriv (μ ::ₘ μs) = (jets.deriv μ).comp (jets.iteratedDeriv μs)
```

lemma iteratedDeriv_add [source]

The iterated derivative is additive in the multiset.

```
lemma iteratedDeriv_add (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    jets.iteratedDeriv (s + t) = (jets.iteratedDeriv s).comp (jets.iteratedDeriv t)
```

lemma iteratedDeriv_singleton [source]

```
@[simp]
lemma iteratedDeriv_singleton (μ : Fin 1 ⊕ Fin 3) :
    jets.iteratedDeriv ({μ} : Multiset (Fin 1 ⊕ Fin 3)) = jets.deriv μ
```

lemma iteratedDeriv_cons_eq_comp_deriv [source]

```
lemma iteratedDeriv_cons_eq_comp_deriv (μ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    jets.iteratedDeriv (μ ::ₘ s) = (jets.iteratedDeriv s).comp (jets.deriv μ)
```

lemma iteratedDeriv_bracket [source]

The iterated Leibniz rule for the bracket.

```
lemma iteratedDeriv_bracket (s : Multiset (Fin 1 ⊕ Fin 3)) (a b : 𝔤J) :
    jets.iteratedDeriv s ⁅a, b⁆ =
      (s.antidiagonal.map fun p =>
        ⁅jets.iteratedDeriv p.1 a, jets.iteratedDeriv p.2 b⁆).sum
```

lemma iteratedDeriv_ofConstantLie_of_ne_zero [source]

```
lemma iteratedDeriv_ofConstantLie_of_ne_zero {p : Multiset (Fin 1 ⊕ Fin 3)} (hp : p ≠ 0)
    (a : 𝔤) : jets.iteratedDeriv p (jets.ofConstantLie a) = 0
```

lemma iteratedDeriv_adjointJ_ofConstant [source]

```
lemma iteratedDeriv_adjointJ_ofConstant (g : G₀) (s : Multiset (Fin 1 ⊕ Fin 3)) (x : 𝔤J) :
    jets.iteratedDeriv s (jets.adjointJ (jets.ofConstant g) x) =
      jets.adjointJ (jets.ofConstant g) (jets.iteratedDeriv s x)
```

lemma evalLie_iteratedDeriv_mulCoord [source]

```
lemma evalLie_iteratedDeriv_mulCoord (μ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (a : 𝔤J
    ) :
    jets.evalLie (jets.iteratedDeriv s (jets.mulCoord μ a)) =
      s.count μ • jets.evalLie (jets.iteratedDeriv (s.erase μ) a)
```

structure Hom [source]

A morphism of local gauge data.

`Hom` is the standard name for a morphism in Lean.

```
structure Hom (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) (jets' : LocalGaugeData G₀' 𝔤' GJ' 𝔤J') where
  group : G₀ →* G₀'
  groupJ : GJ →* GJ'
  eval_groupJ : ∀ U, jets'.eval (groupJ U) = group (jets.eval U)
  groupJ_ofConstant : ∀ g, groupJ (jets.ofConstant g) = jets'.ofConstant (group g)
  lie : 𝔤 →ₗ⁅ℝ⁆ 𝔤'
  lieJ : 𝔤J →ₗ⁅ℝ⁆ 𝔤J'
  lieJ_ofConstantLie : ∀ c, lieJ (jets.ofConstantLie c) = jets'.ofConstantLie (lie c)
  evalLie_lieJ : ∀ a, jets'.evalLie (lieJ a) = lie (jets.evalLie a)
  lieJ_deriv : ∀ μ a, lieJ (jets.deriv μ a) = jets'.deriv μ (lieJ a)
  lieJ_mulCoord : ∀ μ a, lieJ (jets.mulCoord μ a) = jets'.mulCoord μ (lieJ a)
```

```
  lieJ_adjointJ : ∀ U a, lieJ (jets.adjointJ U a) = jets'.adjointJ (groupJ U) (lieJ a)
  lieJ_maurerCartan : ∀ U μ, lieJ (jets.maurerCartan U μ) = jets'.maurerCartan (groupJ U) μ
```

lemma lie_adjoint [source]

A morphism intertwines the adjoint actions of the gauge groups.

```
lemma lie_adjoint (g : G₀) (x : 𝔤) :
    f.lie (jets.adjoint g x) = jets'.adjoint (f.group g) (f.lie x)
```

lemma lieJ_iteratedDeriv [source]

A morphism commutes with iterated derivatives.

```
lemma lieJ_iteratedDeriv (s : Multiset (Fin 1 ⊕ Fin 3)) (a : 𝔤J) :
    f.lieJ (jets.iteratedDeriv s a) = jets'.iteratedDeriv s (f.lieJ a)
```

def id [source]

The identity morphism of local gauge data.

```
protected def id (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) : Hom jets jets where
```

def comp [source]

The composite `g ∘ f` of morphisms of local gauge data.

```
def comp (g : Hom jets' jets'') : Hom jets jets'' where
```

2.2 Physlib.ClassicalMechanics.ConfigurationSpace.Basic

Physlib/ClassicalMechanics/ConfigurationSpace/Basic.lean on GitHub.

structure ConfigurationSpace [source]

A smooth manifold without boundary.

```
structure ConfigurationSpace (I : ModelWithCorners ℝ E H) where
  carrier : Type*
  [topologicalSpace : TopologicalSpace carrier]
  [chartedSpace : ChartedSpace H carrier]
  [isManifold : IsManifold I ∞ carrier]
  [boundarylessManifold : BoundarylessManifold I carrier]
```

instance CoeSort (ConfigurationSpace I) (Type _) [source]

```
instance : CoeSort (ConfigurationSpace I) (Type _)
```

2.3 Physlib.ClassicalMechanics.ConfigurationSpace.Variation

Physlib/ClassicalMechanics/ConfigurationSpace/Variation.lean on GitHub.

structure Variation [source]

A smooth variation of γ on [t₀, t₁] fixing γ t₀ and γ t₁.

```
@[ext]
structure Variation (γ : Time → M) (t₀ t₁ : Time) where
  toFun : C^∞ ⟮𝓘(ℝ, ℝ).prod 𝓘(ℝ, ℝ), ℝ × Time; I, M⟯
  apply_zero : ∀ s ∈ Set.Icc 0 (t₁ -ᵥ t₀), toFun (0, s +ᵥ t₀) = γ (s +ᵥ t₀)
  apply_left : ∀ s, toFun (s, t₀) = γ t₀
  apply_right : ∀ s, toFun (s, t₁) = γ t₁
```

lemma nonempty [source]

A smooth curve admits the constant variation.

```
lemma nonempty {γ : Time → M} {t₀ t₁ : Time} (hγ : ContMDiff 𝓘(ℝ, ℝ) I ∞ γ) :
    Nonempty (Variation I γ t₀ t₁)
```

lemma contDiff_comp_vadd [source]

(s, τ) ↦ h (s, τ +ᵥ t₀) is smooth.

```
lemma contDiff_comp_vadd {γ : Time → E} {t₀ t₁ : Time}
    (h : Variation (M := ⟨E⟩) 𝓘(ℝ, E) γ t₀ t₁) :
    ContDiff ℝ ∞ (fun p : ℝ × ℝ ↦ h.toFun (p.1, p.2 +ᵥ t₀))
```

lemma exists_bump [source]

For a smooth curve γ, smooth ψ supported in (0, t₁ -ᵥ t₀) and w : E, there exists a variation h of γ with h (s, τ +ᵥ t₀) = γ (τ +ᵥ t₀) + s • ψ τ • w.

```
lemma exists_bump {γ : Time → E} {t₀ t₁ : Time} (hγ : ContMDiff 𝓘(ℝ, ℝ) 𝓘(ℝ, E) ∞ γ)
    {ψ : ℝ → ℝ} (hψ : ContDiff ℝ ∞ ψ) (hs : tsupport ψ ⊆ Ioo 0 (t₁ -ᵥ t₀)) (w : E) :
    ∃ h : Variation (M := ⟨E⟩) 𝓘(ℝ, E) γ t₀ t₁,
      ∀ s τ, h.toFun (s, τ +ᵥ t₀) = γ (τ +ᵥ t₀) + s • (ψ τ • w)
```

2.4 Physlib.ClassicalMechanics.EulerLagrange

Physlib/ClassicalMechanics/EulerLagrange.lean on GitHub.

def canonicalMomentum [source]

The canonical momentum ∂L/∂q˙ of a curve q.

```
noncomputable def canonicalMomentum (q : ℝ → E) (τ : ℝ) : E →L[ℝ] ℝ
```

def eulerLagrange [source]

The Euler-Lagrange covector ∂L/∂q - dₜ(∂L/∂q˙) of a curve q; the covector counterpart of eulerLagrangeOp.

```
noncomputable def eulerLagrange (q : ℝ → E) (τ : ℝ) : E →L[ℝ] ℝ
```

lemma contDiff_eulerLagrange [source]

The Euler-Lagrange covector of a smooth curve is smooth.

```
lemma contDiff_eulerLagrange {q : ℝ → E} (hq : ContDiff ℝ ∞ q) :
    ContDiff ℝ ∞ (eulerLagrange S t₀ q)
```

lemma eulerLagrange_congr [source]

The Euler-Lagrange covector at `τ` depends only on the germ of the curve at `τ`.

```
lemma eulerLagrange_congr {q g : ℝ → E} {τ : ℝ} (h : q =ᶠ[𝓝 τ] g) :
    eulerLagrange S t₀ q τ = eulerLagrange S t₀ g τ
```

lemma eulerLagrange_congr_lagrangian [source]

The Euler-Lagrange covector only depends on the Lagrangian near the curve.

```
lemma eulerLagrange_congr_lagrangian {S' : LagrangianSystem 𝓘(ℝ, E) (E)} {q : ℝ → E} {τ : ℝ}
    {N : Set E} (hN : IsOpen N) (hq : ∀ᶠ σ in 𝓝 τ, q σ ∈ N)
    (h : ∀ p : ℝ × E × E, p.2.1 ∈ N → S.coord t₀ p = S'.coord t₀ p) :
    eulerLagrange S t₀ q τ = eulerLagrange S' t₀ q τ
```

2.5 Physlib.ClassicalMechanics.HamiltonianMechanics.PoissonBracket

Physlib/ClassicalMechanics/HamiltonianMechanics/PoissonBracket.lean on GitHub.

def poissonBracket [source]

The Poisson bracket of two phase-space functions `f g : X → X → ℝ`, each read as `f p q` for a momentum `p` and a position `q`, at a point `(p, q)` of phase space.

```
noncomputable def poissonBracket (f g : X → X → ℝ) (p q : X) : ℝ
```

lemma poissonBracket_eq_of_hasGradientAt [source]

The Poisson bracket of two phase-space functions whose partial gradients at `(p, q)` are known.

```
lemma poissonBracket_eq_of_hasGradientAt {f g : X → X → ℝ} {p q fp fq gp gq : X}
    (hfp : HasGradientAt (fun p' => f p' q) fp p) (hfq : HasGradientAt (fun q' => f p q') fq q)
    (hgp : HasGradientAt (fun p' => g p' q) gp p) (hgq : HasGradientAt (fun q' => g p q') gq q)
     :
    poissonBracket f g p q = ⟪fq, gp⟫_ℝ - ⟪fp, gq⟫_ℝ
```

lemma poissonBracket_antisymm [source]

The Poisson bracket is antisymmetric.

```
lemma poissonBracket_antisymm (f g : X → X → ℝ) (p q : X) :
    poissonBracket f g p q = -poissonBracket g f p q
```

lemma poissonBracket_self [source]

The Poisson bracket of a phase-space function with itself vanishes.

```
lemma poissonBracket_self (f : X → X → ℝ) (p q : X) : poissonBracket f f p q = 0
```

lemma poissonBracket_add_left [source]

The Poisson bracket is additive in its first argument, for functions differentiable in both phase-space directions.

```
lemma poissonBracket_add_left (f₁ f₂ g : X → X → ℝ) (p q : X)
    (hf₁p : DifferentiableAt ℝ (fun p' => f₁ p' q) p)
    (hf₂p : DifferentiableAt ℝ (fun p' => f₂ p' q) p)
    (hf₁q : DifferentiableAt ℝ (fun q' => f₁ p q') q)
    (hf₂q : DifferentiableAt ℝ (fun q' => f₂ p q') q) :
    poissonBracket (fun p' q' => f₁ p' q' + f₂ p' q') g p q =
      poissonBracket f₁ g p q + poissonBracket f₂ g p q
```

lemma poissonBracket_const_mul_left [source]

The Poisson bracket is homogeneous of degree one in its first argument, for a function differentiable in both phase-space directions.

```
lemma poissonBracket_const_mul_left (c : ℝ) (f g : X → X → ℝ) (p q : X)
    (hfp : DifferentiableAt ℝ (fun p' => f p' q) p)
    (hfq : DifferentiableAt ℝ (fun q' => f p q') q) :
    poissonBracket (fun p' q' => c * f p' q') g p q = c * poissonBracket f g p q
```

lemma poissonBracket_add_right [source]

The Poisson bracket is additive in its second argument.

```
lemma poissonBracket_add_right (f g₁ g₂ : X → X → ℝ) (p q : X)
    (hg₁p : DifferentiableAt ℝ (fun p' => g₁ p' q) p)
    (hg₂p : DifferentiableAt ℝ (fun p' => g₂ p' q) p)
    (hg₁q : DifferentiableAt ℝ (fun q' => g₁ p q') q)
    (hg₂q : DifferentiableAt ℝ (fun q' => g₂ p q') q) :
    poissonBracket f (fun p' q' => g₁ p' q' + g₂ p' q') p q =
      poissonBracket f g₁ p q + poissonBracket f g₂ p q
```

lemma poissonBracket_const_mul_right [source]

The Poisson bracket is homogeneous of degree one in its second argument.

```
lemma poissonBracket_const_mul_right (c : ℝ) (f g : X → X → ℝ) (p q : X)
    (hgp : DifferentiableAt ℝ (fun p' => g p' q) p)
    (hgq : DifferentiableAt ℝ (fun q' => g p q') q) :
    poissonBracket f (fun p' q' => c * g p' q') p q = c * poissonBracket f g p q
```

lemma poissonBracket_mul_left [source]

The Poisson bracket satisfies the Leibniz rule in its first argument.

```
lemma poissonBracket_mul_left (f₁ f₂ g : X → X → ℝ) (p q : X)
    (hf₁p : DifferentiableAt ℝ (fun p' => f₁ p' q) p)
```

```
    (hf₂p : DifferentiableAt ℝ (fun p' => f₂ p' q) p)
    (hf₁q : DifferentiableAt ℝ (fun q' => f₁ p q') q)
    (hf₂q : DifferentiableAt ℝ (fun q' => f₂ p q') q) :
    poissonBracket (fun p' q' => f₁ p' q' * f₂ p' q') g p q =
      poissonBracket f₁ g p q * f₂ p q + f₁ p q * poissonBracket f₂ g p q
```

lemma poissonBracket_mul_right [source]

The Poisson bracket satisfies the Leibniz rule in its second argument.

```
lemma poissonBracket_mul_right (f g₁ g₂ : X → X → ℝ) (p q : X)
    (hg₁p : DifferentiableAt ℝ (fun p' => g₁ p' q) p)
    (hg₂p : DifferentiableAt ℝ (fun p' => g₂ p' q) p)
    (hg₁q : DifferentiableAt ℝ (fun q' => g₁ p q') q)
    (hg₂q : DifferentiableAt ℝ (fun q' => g₂ p q') q) :
    poissonBracket f (fun p' q' => g₁ p' q' * g₂ p' q') p q =
      poissonBracket f g₁ p q * g₂ p q + g₁ p q * poissonBracket f g₂ p q
```

lemma deriv_comp_eq_inner_gradient_add [source]

The time derivative of a phase-space function `f` composed with trajectories `p q : Time → X` is the sum of the gradients of `f` paired with the velocities `∂ₜ p` and `∂ₜ q`.

```
lemma deriv_comp_eq_inner_gradient_add (f : X → X → ℝ) (p q : Time → X) (t : Time)
    (hp : DifferentiableAt ℝ p t) (hq : DifferentiableAt ℝ q t)
    (hf : DifferentiableAt ℝ (Function.uncurry f) (p t, q t)) :
    ∂ₜ (fun t => f (p t) (q t)) t =
      ⟪gradient (fun p' => f p' (q t)) (p t), ∂ₜ p t⟫_ℝ +
      ⟪gradient (fun q' => f (p t) q') (q t), ∂ₜ q t⟫_ℝ
```

theorem deriv_comp_eq_poissonBracket [source]

Along a solution of Hamilton's equations for a Hamiltonian `H`, the time derivative of a phase-space observable `f` is its Poisson bracket with `H t`.

```
theorem deriv_comp_eq_poissonBracket (H : Time → X → X → ℝ) (p q : Time → X) (f : X → X → ℝ)
    (t : Time) (heq : hamiltonEqOp H p q = 0)
    (hp : DifferentiableAt ℝ p t) (hq : DifferentiableAt ℝ q t)
    (hf : DifferentiableAt ℝ (Function.uncurry f) (p t, q t)) :
    ∂ₜ (fun t => f (p t) (q t)) t = poissonBracket f (H t) (p t) (q t)
```

2.6 Physlib.ClassicalMechanics.HarmonicOscillator.Solution.Energy

Physlib/ClassicalMechanics/HarmonicOscillator/Solution/Energy.lean on GitHub.

lemma trajectory_energy [source]

```
lemma trajectory_energy (IC : InitialConditions) : S.energy (IC.trajectory S) =
    fun _ => 1/2 * (S.m * ‖IC.v₀‖ ^2 + S.k * ‖IC.x₀‖ ^ 2)
```

2.7 Physlib.ClassicalMechanics.HarmonicOscillator.Solution.SpecialPoints

Physlib/ClassicalMechanics/HarmonicOscillator/Solution/SpecialPoints.lean on GitHub.

lemma trajectory_eq_cos [source]

Every trajectory of the harmonic oscillator is a single shifted cosine after converting its initial conditions to amplitude-phase form.

```
lemma trajectory_eq_cos (IC : InitialConditions) (t : Time) :
    IC.trajectory S t =
      EuclideanSpace.single 0 ((AmplitudePhase.fromInitialConditions S IC).A *
        cos (S.ω * t - (AmplitudePhase.fromInitialConditions S IC).φ))
```

lemma trajectory_velocity_eq_sin [source]

The velocity of every trajectory is the corresponding shifted sine in amplitude-phase form.

```
lemma trajectory_velocity_eq_sin (IC : InitialConditions) (t : Time) :
    ∂ₜ (IC.trajectory S) t =
      EuclideanSpace.single 0 (-((AmplitudePhase.fromInitialConditions S IC).A * S.ω *
        sin (S.ω * t.val - (AmplitudePhase.fromInitialConditions S IC).φ)))
```

lemma trajectory_velocity_eq_zero_iff_sin_eq_zero [source]

For nonzero amplitude, the velocity vanishes exactly when the sine factor in amplitude-phase form vanishes.

```
lemma trajectory_velocity_eq_zero_iff_sin_eq_zero (IC : InitialConditions)
    (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
    ∂ₜ (IC.trajectory S) t = 0 ↔
      sin (S.ω * t.val - (AmplitudePhase.fromInitialConditions S IC).φ) = 0
```

lemma trajectory_velocity_eq_zero_iff_exists_int [source]

For nonzero amplitude, the velocity is zero exactly at phase times $\varphi + n\pi$.

```
lemma trajectory_velocity_eq_zero_iff_exists_int (IC : InitialConditions)
    (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
    ∂ₜ (IC.trajectory S) t = 0 ↔
      ∃ n : ℤ,
        (t : ℝ) =
          ((AmplitudePhase.fromInitialConditions S IC).φ + n * π) / S.ω
```

lemma trajectory_velocity_eq_zero_iff_norm_eq_amplitude [source]

The velocity vanishes exactly when the trajectory has norm equal to the amplitude.

```
lemma trajectory_velocity_eq_zero_iff_norm_eq_amplitude (IC : InitialConditions)
    (t : Time) :
    ∂ₜ (IC.trajectory S) t = 0 ↔
      ‖IC.trajectory S t‖ = (AmplitudePhase.fromInitialConditions S IC).A
```

lemma trajectory_eq_zero_iff_cos_eq_zero [source]

For nonzero amplitude, the trajectory passes through zero exactly when the cosine factor in amplitude-phase form vanishes.

```
lemma trajectory_eq_zero_iff_cos_eq_zero (IC : InitialConditions)
    (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
    IC.trajectory S t = 0 ↔
      cos (S.ω * t.val - (AmplitudePhase.fromInitialConditions S IC).φ) = 0
```

lemma trajectory_eq_zero_iff_exists_int [source]

For nonzero amplitude, the trajectory passes through zero exactly at phase times `φ + (2n + 1)π / 2`.

```
lemma trajectory_eq_zero_iff_exists_int (IC : InitialConditions)
    (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
    IC.trajectory S t = 0 ↔
      ∃ n : ℤ,
        (t : ℝ) =
          ((AmplitudePhase.fromInitialConditions S IC).φ + (2 * n + 1) * π / 2) / S.ω
```

2.8 Physlib.ClassicalMechanics.Lagrangian.Basic

Physlib/ClassicalMechanics/Lagrangian/Basic.lean on GitHub.

instance FunLike (LagrangianSystem I M) (Time × TangentBundle I M) ℝ [source]

```
instance : FunLike (LagrangianSystem I M) (Time × TangentBundle I M) ℝ where
```

def ofNormedSpace [source]

The Lagrangian system on a normed space `E` given by a smooth `L(t, q, q˙)`.

```
def ofNormedSpace (L : Time × E × E → ℝ)
    (hL : ContMDiff (𝓘(ℝ, ℝ).prod 𝓘(ℝ, E × E)) 𝓘(ℝ, ℝ) ∞ L) :
    LagrangianSystem 𝓘(ℝ, E) ⟨E⟩
```

def coord [source]

`L` in the time coordinate `τ ↦ τ +ᵥ t₀`.

```
noncomputable def coord (S : LagrangianSystem 𝓘(ℝ, E) ⟨E⟩) (t₀ : Time) (p : ℝ × E × E) : ℝ
```

lemma contDiff_coord [source]

`S.coord t₀` is smooth.

```
lemma contDiff_coord (S : LagrangianSystem 𝓘(ℝ, E) ⟨E⟩) (t₀ : Time) :
    ContDiff ℝ ∞ (S.coord t₀)
```

def action [source]

The action `∫ L(t, γ, γ˙) dt` over `[t₀, t₁]`.

```
noncomputable def action (γ : Time → M) (t₀ t₁ : Time) : ℝ
```

lemma action_eq_zero_of_vsub_nonpos [source]

The action over an empty or one-point interval vanishes.

```
lemma action_eq_zero_of_vsub_nonpos (h : t₁ -ᵥ t₀ ≤ 0) : S.action γ t₀ t₁ = 0
```

lemma action_eq_integral_coord [source]

The action is the integral over `[0, t₁ -ᵥ t₀]` of `S.coord t₀ (s, q s, q' s)`, where `q τ = γ (τ +ᵥ t₀)` and `q' = deriv q`.

```
lemma action_eq_integral_coord (S : LagrangianSystem 𝓘(ℝ, E) ⟨E⟩) (γ : Time → E) (t₀ t₁ : Time)
    :
    S.action γ t₀ t₁ = ∫ s in Set.Icc 0 (t₁ -ᵥ t₀),
      S.coord t₀ (s, γ (s +ᵥ t₀), deriv (fun τ : ℝ ↦ γ (τ +ᵥ t₀)) s)
```

2.9 Physlib.ClassicalMechanics.Lagrangian.CriticalPoint

Physlib/ClassicalMechanics/Lagrangian/CriticalPoint.lean on GitHub.

def IsCriticalPoint [source]

`γ` is smooth and every variation of `γ` fixing the endpoints has zero first variation of the action, i.e. `d/ds S[γ_s]` vanishes at `s = 0` (Hamilton's principle).

```
def IsCriticalPoint (γ : Time → M) (t₀ t₁ : Time) : Prop
```

lemma isCriticalPoint_of_vsub_nonpos [source]

Every smooth curve is critical over an empty or one-point interval.

```
lemma isCriticalPoint_of_vsub_nonpos (hγ : ContMDiff 𝓘(ℝ, ℝ) I ∞ γ) (h : t₁ -ᵥ t₀ ≤ 0) :
    S.IsCriticalPoint γ t₀ t₁
```

lemma isCriticalPoint_of_eulerLagrange_eq_zero [source]

If `q σ = γ (σ +ᵥ t₀)` is smooth and `eulerLagrange S t₀ q = 0` on `(0, t₁ -ᵥ t₀)`, then `γ` is a critical point.

```
lemma isCriticalPoint_of_eulerLagrange_eq_zero
    (hγ : ContDiff ℝ ∞ (fun σ : ℝ ↦ γ (σ +ᵥ t₀))) (hEq : ∀ τ ∈ Ioo 0 (t₁ -ᵥ t₀),
      eulerLagrange S t₀ (fun σ ↦ γ (σ +ᵥ t₀)) τ = 0) : S.IsCriticalPoint γ t₀ t₁
```

lemma eulerLagrange_eq_zero_of_isCriticalPoint [source]

A critical point `γ` has `eulerLagrange S t₀ q = 0` on `[0, t₁ -ᵥ t₀]`, `q σ = γ (σ +ᵥ t₀)`.

```
lemma eulerLagrange_eq_zero_of_isCriticalPoint (ht : t₁ -ᵥ t₀ ≠ 0)
    (hc : S.IsCriticalPoint γ t₀ t₁) : ∀ τ ∈ Icc 0 (t₁ -ᵥ t₀),
      eulerLagrange S t₀ (fun σ : ℝ ↦ γ (σ +ᵥ t₀)) τ = 0
```

lemma isCriticalPoint_iff_eulerLagrange [source]

For `t₁ -ᵥ t₀ ≠ 0`, `γ` is a critical point iff `q σ = γ (σ +ᵥ t₀)` is smooth and `eulerLagrange S t₀ q = 0` on `[0, t₁ -ᵥ t₀]`.

```
lemma isCriticalPoint_iff_eulerLagrange (ht : t₁ -ᵥ t₀ ≠ 0) :
    S.IsCriticalPoint γ t₀ t₁ ↔ ContDiff ℝ ∞ (fun σ : ℝ ↦ γ (σ +ᵥ t₀)) ∧
      ∀ τ ∈ Icc 0 (t₁ -ᵥ t₀), eulerLagrange S t₀ (fun σ : ℝ ↦ γ (σ +ᵥ t₀)) τ = 0
```

2.10 Physlib.ClassicalMechanics.Lagrangian.FirstVariation

Physlib/ClassicalMechanics/Lagrangian/FirstVariation.lean on GitHub.

lemma fderiv_action_integrand_apply [source]

The `s`-derivative of the integrand of the action splits into the Euler-Lagrange term and a total `τ`-derivative.

```
lemma fderiv_action_integrand_apply (S : LagrangianSystem 𝓘(ℝ, E) (E)) (t₀ : Time)
    (hH : ContDiff ℝ ∞ H) (τ : ℝ) :
    fderiv ℝ (fun p : ℝ × ℝ ↦ S.coord t₀ (p.2, H p, fderiv ℝ H p (0, 1))) (0, τ) (1, 0) =
      eulerLagrange S t₀ (fun σ ↦ H (0, σ)) τ (fderiv ℝ H (0, τ) (1, 0)) +
        (deriv (canonicalMomentum S t₀ fun σ ↦ H (0, σ)) τ
            (fderiv ℝ H (0, τ) (1, 0)) +
          canonicalMomentum S t₀ (fun σ ↦ H (0, σ)) τ
            (deriv (fun σ ↦ fderiv ℝ H (0, σ) (1, 0)) τ))
```

lemma contDiff_action_integrand [source]

The integrand `L(τ, H, ∂τ H)` of the action is smooth.

```
lemma contDiff_action_integrand (S : LagrangianSystem 𝓘(ℝ, E) (E)) (t₀ : Time)
    (hH : ContDiff ℝ ∞ H) :
    ContDiff ℝ ∞ fun p : ℝ × ℝ ↦ S.coord t₀ (p.2, H p, fderiv ℝ H p (0, 1))
```

lemma hasDerivAt_integral_variation [source]

First variation of `∫ L(τ, H, ∂τ H) dτ` along a variation fixing the endpoints.

```
lemma hasDerivAt_integral_variation (S : LagrangianSystem 𝓘(ℝ, E) (E)) (t₀ : Time)
    (hH : ContDiff ℝ ∞ H)
    (T : ℝ) (h0 : ∀ s, H (s, 0) = H (0, 0)) (hT : ∀ s, H (s, T) = H (0, T)) :
    HasDerivAt
      (fun s ↦ ∫ τ in (0 : ℝ)..T, S.coord t₀ (τ, H (s, τ), fderiv ℝ H (s, τ) (0, 1)))
      (∫ τ in (0 : ℝ)..T,
        eulerLagrange S t₀ (fun σ ↦ H (0, σ)) τ (fderiv ℝ H (0, τ) (1, 0))) 0
```

lemma action_variation_eq_integral [source]

The action of `t ↦ h (s, t)` is the integral over `[0, t₁ -ᵥ t₀]` of `S.coord t₀ (τ, q τ, q' τ)`, where `q τ = h (s, τ +ᵥ t₀)` and `q' = deriv q`.

```
lemma action_variation_eq_integral (S : LagrangianSystem 𝓘(ℝ, E) (E)) {γ : Time → E}
    {t₀ t₁ : Time} (h : Variation (M := (E)) 𝓘(ℝ, E) γ t₀ t₁) (ht : 0 ≤ t₁ -ᵥ t₀) (s : ℝ) :
    S.action (fun t ↦ h.toFun (s, t)) t₀ t₁ =
      ∫ τ in (0 : ℝ)..(t₁ -ᵥ t₀), S.coord t₀ (τ, h.toFun (s, τ +ᵥ t₀),
```

```
      fderiv ℝ (fun p : ℝ × ℝ ↦ h.toFun (p.1, p.2 +ᵥ t₀)) (s, τ) (0, 1))
```

lemma hasDerivAt_action_variation [source]

The `s`-derivative at `0` of the action of `t ↦ h (s, t)` is the integral over `[0, t₁ -ᵥ t₀]` of `eulerLagrange S t₀ q τ` applied to the derivative of `s ↦ h (s, τ +ᵥ t₀)` at `0`, where `q σ = h (0, σ +ᵥ t₀)`.

```
lemma hasDerivAt_action_variation (S : LagrangianSystem 𝓘(ℝ, E) (E)) {γ : Time → E}
    {t₀ t₁ : Time} (h : Variation (M := (E)) 𝓘(ℝ, E) γ t₀ t₁) (ht : 0 ≤ t₁ -ᵥ t₀) :
    HasDerivAt (fun s ↦ S.action (fun t ↦ h.toFun (s, t)) t₀ t₁)
      (∫ τ in (0 : ℝ)..(t₁ -ᵥ t₀),
        eulerLagrange S t₀ (fun σ ↦ h.toFun (0, σ +ᵥ t₀)) τ
          (fderiv ℝ (fun p : ℝ × ℝ ↦ h.toFun (p.1, p.2 +ᵥ t₀)) (0, τ) (1, 0))) 0
```

2.11 Physlib.CondensedMatter.Crystal.Basic

Physlib/CondensedMatter/Crystal/Basic.lean on GitHub.

structure Crystal [source]

A crystal with finitely generated abelian translation group `G` and a finite, nonempty unit cell. The sites are one copy of each unit-cell label in every translated cell.

```
structure Crystal (G : Type*) [AddCommGroup G] [AddGroup.FG G] where
  UnitCell : Type*
  [finiteUnitCell : Finite UnitCell]
  [nonemptyUnitCell : Nonempty UnitCell]
```

abbrev ofUnitCell [source]

Construct a crystal by repeating a unit cell with labels `F`. The group `G` indexes translated cells; `(n, i)` is the site with label `i` in cell `n`.

```
abbrev ofUnitCell (G : Type*) (F : Type*) [AddCommGroup G] [AddGroup.FG G]
    [Finite F] [Nonempty F] : Crystal G where
```

abbrev ref [source]

The site with label `i` in the reference cell, the cell `0`.

```
abbrev Site.ref {C : Crystal G} (i : C.UnitCell) : C.Site
```

def siteEquiv [source]

A site is a pair of its unit-cell label and its cell.

```
def siteEquiv (C : Crystal G) : C.Site ≃ C.UnitCell × G where
```

instance DecidableEq C.Site [source]

Sites can be compared when cells and unit-cell labels can be compared.

```
instance (C : Crystal G) [DecidableEq G] [DecidableEq C.UnitCell] : DecidableEq C.Site
```

instance Fintype C.Site [source]

A finite set of cells and a finite unit cell give a finite set of sites.

```
instance (C : Crystal G) [Fintype G] [Fintype C.UnitCell] : Fintype C.Site
```

instance Finite C.Site [source]

A finite translation group, as for periodic boundary conditions, gives a finite crystal.

```
instance (C : Crystal G) [Finite G] : Finite C.Site
```

instance Nonempty C.Site [source]

Every crystal has a site in its reference cell.

```
instance (C : Crystal G) : Nonempty C.Site
```

instance AddAction G C.Site [source]

Translate a site to another cell, preserving its unit-cell label.

```
instance (C : Crystal G) : AddAction G C.Site where
```

lemma vadd_def [source]

Translation changes a site's cell and preserves its label.

```
lemma Site.vadd_def {C : Crystal G} (a : G) (x : C.Site) :
    a +ᵥ x = ⟨a + x.cell, x.label⟩
```

instance IsCancelVAdd G C.Site [source]

Translation acts freely: translations are injective on sites, and different translations move a site to different sites.

```
instance (C : Crystal G) : IsCancelVAdd G C.Site where
```

lemma mem_orbit_iff_label_eq [source]

The translates of a site are exactly the sites with the same label.

```
lemma Site.mem_orbit_iff_label_eq {C : Crystal G} (x y : C.Site) :
    y ∈ AddAction.orbit G x ↔ x.label = y.label
```

abbrev Species [source]

A species assignment gives each label of the unit cell a species in `Q`, for example a chemical element. The same assignment holds in every cell.

```
abbrev Species (C : Crystal G) (Q : Type*)
```

2.12 Physlib.CondensedMatter.Crystal.Bonds

Physlib/CondensedMatter/Crystal/Bonds.lean on GitHub.

structure Bond [source]

A bond from `source` in one cell to `target` in the cell displaced by `displacement`.

```
structure Bond (C : Crystal G) where
  source : C.UnitCell
  target : C.UnitCell
  displacement : G
```

instance DecidableEq (Bond C) [source]

Bonds can be compared when cells and unit-cell labels can be compared.

```
instance {C : Crystal G} [DecidableEq G] [DecidableEq C.UnitCell] : DecidableEq (Bond C)
```

def Links [source]

Site `x` is the source and site `y` is the target of this bond.

```
def Links (b : Bond C) (x y : C.Site) : Prop
```

lemma links_reference [source]

Every bond links the reference site of its source to the site its displacement points to.

```
lemma links_reference (b : Bond C) : b.Links (Site.ref b.source) ⟨b.displacement, b.target⟩
```

lemma eq_of_links [source]

Two bonds linking the same pair of sites are equal.

```
lemma eq_of_links {b b' : Bond C} {x y : C.Site} (h : b.Links x y) (h' : b'.Links x y) :
    b = b'
```

lemma links_vadd [source]

Translating the endpoints gives another copy of the same bond.

```
lemma links_vadd (b : Bond C) (a : G) (x y : C.Site) :
    b.Links (a +ᵥ x) (a +ᵥ y) ↔ b.Links x y
```

instance DecidableRel b.Links [source]

The source and target of a bond can be checked from their site addresses.

```
instance [DecidableEq G] [DecidableEq C.UnitCell] (b : Bond C) : DecidableRel b.Links
```

2.13 Physlib.CondensedMatter.Crystal.Bravais.Basic

Physlib/CondensedMatter/Crystal/Bravais/Basic.lean on GitHub.

def bravais [source]

The Bravais lattice of dimension `d`: the translation group is $\mathbb{Z}^d$ and the unit cell has a single site.

```
def bravais (d : ℕ) : Crystal (Fin d → ℤ)
```

def ofVectors [source]

The embedding of the Bravais lattice of dimension `d` given by `d` primitive vectors.

```
def ofVectors (v : Fin d → E) : Embedding (bravais d) E where
```

lemma position_ofVectors [source]

The position of a site is the integer combination of the primitive vectors given by its cell.

```
lemma position_ofVectors (v : Fin d → E) (x : (bravais d).Site) :
    (ofVectors v).position x = ∑ j, x.cell j • v j
```

lemma isSpaceFilling_ofVectors_iff [source]

The embedding of a Bravais lattice is space filling exactly when the primitive vectors span the space.

```
lemma isSpaceFilling_ofVectors_iff (v : Fin d → E) :
    (ofVectors v).IsSpaceFilling ↔ Submodule.span ℝ (Set.range v) = ⊤
```

lemma position_ofVectors_injective [source]

Linearly independent primitive vectors give distinct positions to distinct sites.

```
lemma position_ofVectors_injective {v : Fin d → E} (hv : LinearIndependent ℝ v) :
    Function.Injective (ofVectors v).position
```

lemma finrank_eq_of_ofVectors [source]

A space-filling embedding of a Bravais lattice with linearly independent primitive vectors lives in a space of dimension `d`.

```
lemma finrank_eq_of_ofVectors {v : Fin d → E} (hv : LinearIndependent ℝ v)
    (h : (ofVectors v).IsSpaceFilling) : Module.finrank ℝ E = d
```

2.14 Physlib.CondensedMatter.Crystal.Embedding

`Physlib/CondensedMatter/Crystal/Embedding.lean` on GitHub.

structure Embedding [source]

An embedding of a crystal in a normed group `E`. It assigns a vector to each translation and a position to each site of the reference cell.

```
structure Embedding (C : Crystal G) (E : Type*) [NormedAddCommGroup E] where
  vector : G →+ E
  basis : C.UnitCell → E
```

def position [source]

The site `(n, i)` sits at the vector of `n` plus the position of the label `i`.

```
def position (x : C.Site) : E
```

lemma position_vadd [source]

Translating a site by `a` translates its position by the vector of `a`.

```
lemma position_vadd (a : G) (x : C.Site) : R.position (a +ᵥ x) = R.vector a + R.position x
```

abbrev action [source]

The translation group acts on the space by its vectors.

```
abbrev action : AddAction G E
```

def positionHom [source]

The positions commute with the translations.

```
def positionHom : letI
```

lemma dist_position_vadd [source]

Translating two sites does not change the distance between their positions.

```
lemma dist_position_vadd (a : G) (x y : C.Site) :
    Dist.dist (R.position (a +ᵥ x)) (R.position (a +ᵥ y)) =
      Dist.dist (R.position x) (R.position y)
```

def IsSpaceFilling [source]

The embedding is space filling if the vectors of the translations span the space, so that the crystal repeats in every direction.

```
def IsSpaceFilling : Prop
```

lemma isSpaceFilling_iff [source]

An embedding fills the space exactly when the vectors of a generating set of the translation group span it.

```
lemma isSpaceFilling_iff {S : Set G} (hS : AddSubgroup.closure S = ⊤) :
    R.IsSpaceFilling ↔ Submodule.span ℝ (R.vector '' S) = ⊤
```

lemma finiteDimensional [source]

A space-filling embedding is in a finite-dimensional space.

```
lemma IsSpaceFilling.finiteDimensional (h : R.IsSpaceFilling) : FiniteDimensional ℝ E
```

lemma finrank_eq_rank [source]

The dimension equals the rank of the translation group when the vectors of a basis of the group are linearly independent over $\mathbb{R}$.

```
lemma IsSpaceFilling.finrank_eq_rank (h : R.IsSpaceFilling) {ι : Type*} [Fintype ι]
    (B : Module.Basis ι ℤ G) (hB : LinearIndependent ℝ (R.vector ∘ B)) :
    Module.finrank ℝ E = Module.finrank ℤ G
```

2.15 Physlib.CondensedMatter.Crystal.PeriodicGraph

Physlib/CondensedMatter/Crystal/PeriodicGraph.lean on GitHub.

structure PeriodicGraph [source]

A periodic pattern of undirected connections between crystal sites, with finitely many neighbors at each site.

```
structure PeriodicGraph (C : Crystal G) where
  toSimpleGraph : SimpleGraph C.Site
  adj_vadd : ∀ (a : G) (x y : C.Site),
    toSimpleGraph.Adj (a +ᵥ x) (a +ᵥ y) ↔ toSimpleGraph.Adj x y
  [locallyFinite : toSimpleGraph.LocallyFinite]
```

def translateIso [source]

A translation is an automorphism of the graph.

```
def translateIso (g : PeriodicGraph C) (a : G) : g.toSimpleGraph ≃g g.toSimpleGraph where
```

lemma degree_vadd [source]

The degree, the number of neighbors, is the same for all translates of a site.

```
lemma degree_vadd (g : PeriodicGraph C) (a : G) (x : C.Site) :
    g.toSimpleGraph.degree (a +ᵥ x) = g.toSimpleGraph.degree x
```

lemma degree_cell [source]

The degree of a site depends only on its label.

```
lemma degree_cell (g : PeriodicGraph C) (n : G) (i : C.UnitCell) :
    g.toSimpleGraph.degree ⟨n, i⟩ = g.toSimpleGraph.degree (Site.ref i)
```

def graphOfBonds [source]

The undirected graph generated by translating every bond in the set.

```
def graphOfBonds (bonds : Finset (Bond C)) : SimpleGraph C.Site
```

lemma adj_graphOfBonds [source]

Two sites are adjacent when they are different and joined by a bond in either direction.

```
lemma adj_graphOfBonds (bonds : Finset (Bond C)) (x y : C.Site) :
    (graphOfBonds bonds).Adj x y ↔ x ≠ y ∧ ∃ b ∈ bonds, b.Links x y ∨ b.Links y x
```

lemma adj_iff [source]

Adjacency depends only on the labels of the sites and the difference of their cells.

```
lemma adj_iff (g : PeriodicGraph C) (x y : C.Site) :
    g.toSimpleGraph.Adj x y ↔
      g.toSimpleGraph.Adj (Site.ref x.label) ⟨y.cell - x.cell, y.label⟩
```

instance DecidableRel (graphOfBonds bonds).Adj [source]

Connections generated by a finite set of bonds can be decided.

```
instance (bonds : Finset (Bond C)) : DecidableRel (graphOfBonds bonds).Adj
```

def candidates [source]

The sites reached from `x` by following a bond forwards or backwards.

```
def candidates (bonds : Finset (Bond C)) (x : C.Site) : Finset C.Site
```

def neighborsOfBonds [source]

The neighbors of a site in the graph generated by finitely many bonds.

```
def neighborsOfBonds (bonds : Finset (Bond C)) (x : C.Site) : Finset C.Site
```

lemma mem_neighborsOfBonds [source]

The enumerated neighbors are exactly the adjacent sites.

```
lemma mem_neighborsOfBonds (bonds : Finset (Bond C)) (x y : C.Site) :
    y ∈ neighborsOfBonds bonds x ↔ (graphOfBonds bonds).Adj x y
```

def ofBonds [source]

The periodic graph obtained by repeating the given bonds in every cell.

```
def ofBonds (bonds : Finset (Bond C)) : PeriodicGraph C where
```

def bonds [source]

The bonds that join a site of the reference cell to one of its neighbors.

```
def bonds (g : PeriodicGraph C) : Finset (Bond C)
```

lemma mem_bonds [source]

A bond of `g.bonds` joins a site of the reference cell to one of its neighbors.

```
lemma mem_bonds (g : PeriodicGraph C) (b : Bond C) :
    b ∈ g.bonds ↔ g.toSimpleGraph.Adj (Site.ref b.source) (b.displacement, b.target)
```

lemma adj_of_links [source]

Sites joined by a bond of `g.bonds` are adjacent in `g`.

```
lemma adj_of_links (g : PeriodicGraph C) {b : Bond C} (hb : b ∈ g.bonds) {x y : C.Site}
    (h : b.Links x y) : g.toSimpleGraph.Adj x y
```

lemma adj_iff_mem_bonds [source]

Sites joined by a bond are adjacent exactly when the bond is one of `g.bonds`.

```
lemma adj_iff_mem_bonds (g : PeriodicGraph C) {b : Bond C} {x y : C.Site} (h : b.Links x y) :
    g.toSimpleGraph.Adj x y ↔ b ∈ g.bonds
```

lemma toSimpleGraph_eq_graphOfBonds [source]

Every periodic graph is generated by the finitely many bonds leaving the reference cell.

```
lemma toSimpleGraph_eq_graphOfBonds (g : PeriodicGraph C) :
    g.toSimpleGraph = graphOfBonds g.bonds
```

lemma eq_ofBonds [source]

Every periodic graph is generated by the finitely many bonds leaving the reference cell.

```
lemma eq_ofBonds (g : PeriodicGraph C) : g = ofBonds g.bonds
```

def graphAutomorphisms [source]

The automorphism group of the underlying graph, without requiring crystal symmetry.

```
def graphAutomorphisms (g : PeriodicGraph C) : Subgroup (Equiv.Perm C.Site) where
```

def PreservesBonds [source]

A crystal symmetry maps every bond of the reference cell to another such bond.

```
def PreservesBonds (g : PeriodicGraph C) (φ : Symmetry C) : Prop
```

lemma preservesBonds_iff_smul_eq [source]

A symmetry preserves the bonds exactly when it permutes the set of reference-cell bonds.

```
lemma preservesBonds_iff_smul_eq : g.PreservesBonds φ ↔ φ • g.bonds = g.bonds
```

lemma map_adj [source]

Preserving the reference-cell bonds implies preserving adjacency in the forward direction.

```
lemma PreservesBonds.map_adj (h : g.PreservesBonds φ) {x y : C.Site}
    (hxy : g.toSimpleGraph.Adj x y) : g.toSimpleGraph.Adj (φ x) (φ y)
```

lemma inv [source]

The inverse of a symmetry that preserves the bonds preserves them as well.

```
lemma PreservesBonds.inv (h : g.PreservesBonds φ) : g.PreservesBonds φ⁻¹
```

2.16 Physlib.CondensedMatter.Crystal.Symmetry.Basic

Physlib/CondensedMatter/Crystal/Symmetry/Basic.lean on GitHub.

structure Symmetry [source]

A symmetry of a crystal: an automorphism of the action of the translation group on the sites. It is an automorphism `lattice` of the translation group together with a bijection of the sites that carries the action along.

```
structure Symmetry (C : Crystal G) where
  lattice : G ≃+ G
  toHom : AddActionHom ⇑lattice C.Site C.Site
  bijective : Function.Bijective toHom
```

lemma ext' [source]

Symmetries with the same action on the sites are equal, since translations act freely.

```
lemma ext' {φ ψ : Symmetry C} (h : ∀ x, φ.toHom x = ψ.toHom x) : φ = ψ
```

instance FunLike (Symmetry C) C.Site C.Site [source]

A symmetry acts on the sites.

```
instance : FunLike (Symmetry C) C.Site C.Site where
```

lemma map_vadd [source]

Mapping a translated site is translating the mapped site by the image of the vector.

```
lemma map_vadd (φ : Symmetry C) (a : G) (x : C.Site) : φ (a +ᵥ x) = φ.lattice a +ᵥ φ x
```

lemma apply_eq [source]

A symmetry moves the site `⟨n, i⟩` by the lattice automorphism from the image of `⟨0, i⟩`.

```
lemma apply_eq (φ : Symmetry C) (n : G) (i : C.UnitCell) :
    φ ⟨n, i⟩ = ⟨φ.lattice n + (φ (Site.ref i)).cell, (φ (Site.ref i)).label⟩
```

def toEquiv [source]

A symmetry as a permutation of the sites.

```
noncomputable def toEquiv (φ : Symmetry C) : C.Site ≃ C.Site
```

lemma comp_map_vadd [source]

Composing equivariant maps gives an equivariant map.

```
lemma comp_map_vadd (φ ψ : Symmetry C) (a : G) (x : C.Site) :
    (φ ∘ ψ) (a +ᵥ x) = (ψ.lattice.trans φ.lattice) a +ᵥ (φ ∘ ψ) x
```

lemma id_map_vadd [source]

The identity is equivariant for the identity automorphism.

```
lemma id_map_vadd (a : G) (x : C.Site) : id (a +ᵥ x) = AddEquiv.refl G a +ᵥ id x
```

lemma symm_map_vadd [source]

The inverse of an equivariant bijection is equivariant for the inverse automorphism.

```
lemma symm_map_vadd (φ : Symmetry C) (a : G) (x : C.Site) :
    φ.toEquiv.symm (a +ᵥ x) = φ.lattice.symm a +ᵥ φ.toEquiv.symm x
```

instance Group (Symmetry C) [source]

Symmetries form a group under composition.

```
noncomputable instance (C : Crystal G) : Group (Symmetry C) where
```

def toPerm [source]

The permutation of the sites underlying a symmetry.

```
noncomputable def toPerm (C : Crystal G) : Symmetry C →* Equiv.Perm C.Site where
```

lemma label_apply [source]

The label of the image of `⟨n, i⟩` does not depend on the cell `n`.

```
lemma label_apply (φ : Symmetry C) (n : G) (i : C.UnitCell) :
    (φ ⟨n, i⟩).label = (φ (Site.ref i)).label
```

instance MulAction (Symmetry C) C.UnitCell [source]

A symmetry permutes the labels: it sends `i` to the label of the image of `⟨0, i⟩`.

```
instance : MulAction (Symmetry C) C.UnitCell where
```

def labelHom [source]

The permutation of the labels by a symmetry, as a homomorphism.

```
noncomputable def labelHom (C : Crystal G) : Symmetry C →* Equiv.Perm C.UnitCell
```

lemma labelHom_apply [source]

The label permutation of a symmetry sends `i` to the label of the image of `(0, i)`.

```
lemma labelHom_apply (φ : Symmetry C) (i : C.UnitCell) :
    labelHom C φ i = (φ (Site.ref i)).label
```

abbrev Offsets [source]

One lattice vector for each unit-cell label. Every symmetry has such offsets, and so does every shift.

```
abbrev Offsets (C : Crystal G)
```

def offset [source]

The offsets of a symmetry: at the label `i` the cell in which the image of the reference site of `i` lies, as a lattice vector in `G`.

```
def offset (φ : Symmetry C) : Offsets C
```

lemma apply_eq_components [source]

A symmetry sends the site `(n, i)` to the cell `A n + bᵢ` with label `σ i`, where `A` is its lattice automorphism, `σ` its label permutation and `bᵢ` its offset at `i`.

```
lemma apply_eq_components (φ : Symmetry C) (x : C.Site) :
    φ x = (φ.lattice x.cell + φ.offset x.label, labelHom C φ x.label)
```

lemma cell_apply [source]

The cell of the image of `(n, i)` is `A n + bᵢ`.

```
lemma cell_apply (φ : Symmetry C) (x : C.Site) :
    (φ x).cell = φ.lattice x.cell + φ.offset x.label
```

lemma vadd_map_vadd [source]

Translations commute, so translating is equivariant for the identity automorphism.

```
lemma vadd_map_vadd (c b : G) (x : C.Site) : c +ᵥ (b +ᵥ x) = AddEquiv.refl G b +ᵥ (c +ᵥ x)
```

def translationHom [source]

Translation as a homomorphism, with the lattice written multiplicatively. It is only used to form the subgroup of translations; use `translation`.

```
noncomputable def translationHom (C : Crystal G) : Multiplicative G →* Symmetry C where
```

def translation [source]

Translation by a lattice vector, as a symmetry.

```
noncomputable def translation (C : Crystal G) (a : G) : Symmetry C
```

lemma translation_add [source]

Translations add.

```
lemma translation_add (a b : G) : translation C (a + b) = translation C a * translation C b
```

lemma translation_zero [source]

Translation by zero is the identity.

```
lemma translation_zero : translation C 0 = 1
```

def translations [source]

The translations as a subgroup of the symmetries.

```
noncomputable def translations (C : Crystal G) : Subgroup (Symmetry C)
```

lemma conj_translation [source]

Conjugating a translation by a symmetry translates by the image of the vector.

```
lemma conj_translation (φ : Symmetry C) (a : G) :
    φ * translation C a * φ⁻¹ = translation C (φ.lattice a)
```

instance (translations C).Normal [source]

The translations are a normal subgroup of the symmetries.

```
instance : (translations C).Normal where
```

def mapBond [source]

The image of a bond under a crystal symmetry, with displacement $A\,\delta + b_j - b_i$.

```
noncomputable def mapBond (b : Bond C) : Bond C where
```

lemma mapBond_links [source]

A symmetry maps sites connected by a bond to sites connected by its image.

```
lemma mapBond_links {b : Bond C} {x y : C.Site} (h : b.Links x y) :
    (φ.mapBond b).Links (φ x) (φ y)
```

lemma mapBond_mul [source]

The image of a bond under a product is the iterated image.

```
lemma mapBond_mul (ψ : Symmetry C) (b : Bond C) :
    (φ * ψ).mapBond b = φ.mapBond (ψ.mapBond b)
```

lemma mapBond_one [source]

The identity symmetry fixes every bond.

```
lemma mapBond_one (b : Bond C) : (1 : Symmetry C).mapBond b = b
```

instance MulAction (Symmetry C) (Bond C) [source]

The crystal symmetries act on the bonds.

```
noncomputable instance : MulAction (Symmetry C) (Bond C) where
```

2.17 Physlib.CondensedMatter.Crystal.Symmetry.Isometries

Physlib/CondensedMatter/Crystal/Symmetry/Isometries.lean on GitHub.

def isometries [source]

The crystal symmetries that preserve distances between all pairs of site positions.

```
def isometries : Subgroup (Symmetry C) where
```

lemma mem_isometries_iff [source]

A symmetry is an isometry exactly when it preserves every distance between site positions.

```
lemma mem_isometries_iff : φ ∈ R.isometries ↔
    ∀ x y, dist (R.position (φ x)) (R.position (φ y)) = dist (R.position x) (R.position y)
```

lemma translations_le_isometries [source]

Translating both sites by the same lattice vector preserves their distance.

```
lemma translations_le_isometries : Symmetry.translations C ≤ R.isometries
```

lemma site_eq_vadd [source]

A site is the lattice translate of the site with the same label in the reference cell.

```
lemma site_eq_vadd (x : C.Site) : x = x.cell +ᵥ (Site.ref x.label)
```

lemma mem_isometries_iff_reference [source]

A symmetry is an isometry exactly when it preserves the distance from every site `(d, i)` to every site `(0, j)` of the reference cell.

```
lemma mem_isometries_iff_reference : φ ∈ R.isometries ↔ ∀ (i j : C.UnitCell) (d : G),
    dist (R.position (φ ⟨d, i⟩)) (R.position (φ (Site.ref j))) =
      dist (R.position ⟨d, i⟩) (R.position (Site.ref j))
```

2.18 `Physlib.CondensedMatter.Crystal.Symmetry.PointOperations`

`Physlib/CondensedMatter/Crystal/Symmetry/PointOperations.lean` on GitHub.

`def componentsEquiv` [source]

The bijection of the sites `⟨n, i⟩ ↦ ⟨A n + bᵢ, σ i⟩` given by a lattice automorphism `A`, a label permutation `σ` and offsets `b`.

```
def componentsEquiv (C : Crystal G) (A : G ≃+ G) (σ : Equiv.Perm C.UnitCell)
    (b : Offsets C) :
    C.Site ≃ C.Site where
```

`def ofComponents` [source]

The symmetry with lattice automorphism `A`, label permutation `σ` and offsets `b`. It is equivariant for `A`, since translating `⟨n, i⟩` by `a` translates its image by `A a`.

```
def ofComponents (C : Crystal G) (A : G ≃+ G) (σ : Equiv.Perm C.UnitCell)
    (b : Offsets C) :
    Symmetry C
```

`lemma eq_ofComponents` [source]

Every symmetry is determined by its components: it equals `ofComponents` of its lattice automorphism, label permutation and offsets.

```
lemma eq_ofComponents (φ : Symmetry C) :
    φ = ofComponents C φ.lattice (labelHom C φ) φ.offset
```

`lemma ofComponents_mul` [source]

The product of two symmetries in components: `(A, σ, b) * (A', σ', b') = (A + A', σ * σ', A ∘ b' + b ∘ σ')`, where `A + A'` is the composite `A ∘ A'` of lattice automorphisms.

```
lemma ofComponents_mul :
    ofComponents C A σ b * ofComponents C B τ b' =
      ofComponents C (A + B) (σ * τ) (A ∘ b' + b ∘ τ)
```

`def linear` [source]

The point operation with lattice automorphism `A` and label permutation `σ`: it sends `⟨n, i⟩` to `⟨A n, σ i⟩` and has no offsets.

```
def linear (C : Crystal G) (A : G ≃+ G) (σ : Equiv.Perm C.UnitCell) : Symmetry C
```

`lemma linear_add_mul` [source]

Point operations compose through their components: `linear (A + B) (σ * τ) = linear A σ * linear B τ`.

```
lemma linear_add_mul :
    linear C (A + B) (σ * τ) = linear C A σ * linear C B τ
```

def pointOp [source]

The point operation of a symmetry: the point operation with the same lattice automorphism and label permutation, dropping the shifts. It is a homomorphism. For $\mathbb{Z}^d$ it is the rotation or reflection of the lattice.

```
noncomputable def pointOp (C : Crystal G) : Symmetry C →* Symmetry C where
```

def shift [source]

The shift by `b`: it moves every site with label `i` by the lattice vector b_i and fixes the labels and the lattice.

```
def shift (C : Crystal G) (b : Offsets C) : Symmetry C
```

lemma shift_const [source]

A shift by the same vector `a` on every label is the translation by `a`.

```
lemma shift_const (a : G) : shift C (Function.const C.UnitCell a) = translation C a
```

lemma shift_add [source]

Shifts compose by adding their vectors: `shift (b + b') = shift b * shift b'`.

```
lemma shift_add : shift C (b + b') = shift C b * shift C b'
```

lemma shift_mul_linear [source]

The point operation `(A, σ)` followed by the shift `b` is the symmetry with components `` `(A, σ, b ∘ σ)` ``.

```
lemma shift_mul_linear :
    shift C b * linear C A σ = ofComponents C A σ (b ∘ σ)
```

lemma shift_mul_linear_mul [source]

The product in the decomposition into shifts and point operations: `` `(b, A, σ) * (b', B, τ) = (b + A ∘ b' ∘ σ⁻¹, A + B, σ * τ)`, where `A + B` is the composite `A ∘ B` ``.

```
lemma shift_mul_linear_mul :
    (shift C b * linear C A σ) * (shift C b' * linear C B τ) =
      shift C (b + A ∘ b' ∘ σ.symm) * linear C (A + B) (σ * τ)
```

lemma eq_shift_mul_linear [source]

Every symmetry is a point operation followed by a shift, with shift vectors `offset ∘ σ⁻¹`.

```
lemma eq_shift_mul_linear (φ : Symmetry C) :
    φ = shift C (φ.offset ∘ (labelHom C φ).symm) * linear C φ.lattice (labelHom C φ)
```

lemma exists_eq_shift_mul_pointOp [source]

Every symmetry is its point operation followed by a shift.

```
lemma exists_eq_shift_mul_pointOp (φ : Symmetry C) : ∃ b, φ = shift C b * pointOp C φ
```

lemma pointOp_eq_one_iff [source]

A symmetry has trivial point operation exactly when it is a shift.

```
lemma pointOp_eq_one_iff (φ : Symmetry C) : pointOp C φ = 1 ↔ ∃ b, φ = shift C b
```

lemma pointOp_translation [source]

A translation has trivial point operation.

```
lemma pointOp_translation (a : G) : pointOp C (translation C a) = 1
```

def shiftAction [source]

The action of the point operations on the shifts: a lattice automorphism `A` and a label permutation `σ` send `b` to `A ∘ b ∘ σ⁻¹`.

```
def shiftAction (C : Crystal G) :
    Multiplicative (AddAut G) × Equiv.Perm C.UnitCell →*
      MulAut (Multiplicative (Offsets C)) where
```

def semidirectEquiv [source]

The symmetry group of a crystal is isomorphic to the semidirect product of the shifts, `UnitCell → G, by the point operations, Aut G × Perm UnitCell, acting through shiftAction`.

```
noncomputable def semidirectEquiv (C : Crystal G) :
    Multiplicative (Offsets C) ⋊[shiftAction C]
      (Multiplicative (AddAut G) × Equiv.Perm C.UnitCell) ≃* Symmetry C where
```

2.19 Physlib.CondensedMatter.Crystal.Symmetry.SpaceGroup

Physlib/CondensedMatter/Crystal/Symmetry/SpaceGroup.lean on GitHub.

def spaceGroup [source]

The subgroup of crystal symmetries preserving adjacency.

```
noncomputable def spaceGroup (g : PeriodicGraph C) : Subgroup (Symmetry C)
```

lemma mem_spaceGroup_iff [source]

A crystal symmetry belongs to the space group exactly when it preserves adjacency.

```
lemma mem_spaceGroup_iff :
    φ ∈ g.spaceGroup ↔ ∀ x y, g.toSimpleGraph.Adj (φ x) (φ y) ↔ g.toSimpleGraph.Adj x y
```

lemma translation_mem_spaceGroup [source]

Every lattice translation belongs to the space group.

```
lemma translation_mem_spaceGroup (a : G) : Symmetry.translation C a ∈ g.spaceGroup
```

lemma translations_le_spaceGroup [source]

The translation subgroup is contained in the space group.

```
lemma translations_le_spaceGroup : Symmetry.translations C ≤ g.spaceGroup
```

def pointGroup [source]

The image of the space group under the point-operation homomorphism.

```
noncomputable def pointGroup (g : PeriodicGraph C) : Subgroup (Symmetry C)
```

lemma mem_pointGroup_iff [source]

A symmetry is in the point group exactly when it is a point operation of a space-group element.

```
lemma mem_pointGroup_iff :
    ψ ∈ g.pointGroup ↔ ∃ φ ∈ g.spaceGroup, Symmetry.pointOp C φ = ψ
```

lemma of_mem_spaceGroup [source]

Every space-group symmetry preserves the reference-cell bonds.

```
lemma PreservesBonds.of_mem_spaceGroup (hφ : φ ∈ g.spaceGroup) : g.PreservesBonds φ
```

lemma mem_spaceGroup_iff_preservesBonds [source]

A crystal symmetry preserves adjacency exactly when it preserves the reference-cell bonds.

```
lemma mem_spaceGroup_iff_preservesBonds : φ ∈ g.spaceGroup ↔ g.PreservesBonds φ
```

lemma spaceGroup_eq_stabilizer [source]

The space group is the stabilizer of the set of reference-cell bonds.

```
lemma spaceGroup_eq_stabilizer :
    g.spaceGroup = MulAction.stabilizer (Symmetry C) g.bonds
```

2.20 Physlib.CondensedMatter.Crystal.Symmetry.Species

Physlib/CondensedMatter/Crystal/Symmetry/Species.lean on GitHub.

def speciesGroup [source]

The crystal symmetries that preserve the species assignment `q`.

```
def speciesGroup (q : Species C Q) : Subgroup (Symmetry C) where
```

lemma mem_speciesGroup_iff [source]

A symmetry preserves `q` exactly when it sends every label to one of the same species.

```
lemma mem_speciesGroup_iff {q : Species C Q} {φ : Symmetry C} :
    φ ∈ speciesGroup q ↔ ∀ i, q (φ • i) = q i
```

lemma speciesGroup_const [source]

Every crystal symmetry preserves a constant species assignment.

```
lemma speciesGroup_const (c : Q) : speciesGroup (C := C) (Function.const _ c) = ⊤
```

lemma ker_labelHom_le_speciesGroup [source]

A symmetry that fixes every unit-cell label preserves any species assignment.

```
lemma ker_labelHom_le_speciesGroup (q : Species C Q) : (labelHom C).ker ≤ speciesGroup q
```

lemma speciesGroup_eq_ker_labelHom [source]

With distinct species on distinct labels, preserving species requires fixing every label.

```
lemma speciesGroup_eq_ker_labelHom {q : Species C Q} (hq : Function.Injective q) :
    speciesGroup q = (labelHom C).ker
```

lemma speciesGroup_le_comp [source]

Applying `f` to the species values can only enlarge the species group.

```
lemma speciesGroup_le_comp (q : Species C Q) {Q' : Type*} (f : Q → Q') :
    speciesGroup q ≤ speciesGroup (f ∘ q)
```

lemma shift_mem_speciesGroup [source]

Every shift fixes the unit-cell labels and so preserves species.

```
lemma shift_mem_speciesGroup (q : Species C Q) (b : C.UnitCell → G) :
    shift C b ∈ speciesGroup q
```

lemma translations_le_speciesGroup [source]

Every translation preserves any species assignment.

```
lemma translations_le_speciesGroup (q : Species C Q) : translations C ≤ speciesGroup q
```

2.21 Physlib.CondensedMatter.TightBindingChain.Basic

Physlib/CondensedMatter/TightBindingChain/Basic.lean on GitHub.

lemma finrank_hilbertSpace [source]

The Hilbert space of the chain has dimension `N`.

```
lemma finrank_hilbertSpace : Module.finrank ℂ T.HilbertSpace = T.N
```

def quantaWaveNumber [source]

The quantized wavenumber `2 π / (a N) * (n - ⌊N/2⌋)` of the index `n`.

```
noncomputable def quantaWaveNumber (n : Fin T.N) : T.QuantaWaveNumber
```

lemma quantaWaveNumber_injective [source]

Distinct indices give distinct quantized wavenumbers.

```
lemma quantaWaveNumber_injective : Function.Injective T.quantaWaveNumber
```

lemma energyEigenstate_ne_zero [source]

An energy eigenstate is not zero.

```
lemma energyEigenstate_ne_zero (k : T.QuantaWaveNumber) : T.energyEigenstate k ≠ 0
```

lemma energyEigenstate_inner_self [source]

The inner product of an energy eigenstate with itself is `N`.

```
lemma energyEigenstate_inner_self (k : T.QuantaWaveNumber) :
    ⟪T.energyEigenstate k, T.energyEigenstate k⟫_ℂ = T.N
```

lemma energyEigenstate_linearIndependent [source]

The energy eigenstates are linearly independent.

```
lemma energyEigenstate_linearIndependent :
    LinearIndependent ℂ (fun n : Fin T.N => T.energyEigenstate (T.quantaWaveNumber n))
```

lemma energyEigenstate_span [source]

The energy eigenstates span the Hilbert space.

```
lemma energyEigenstate_span :
    ⊤ ≤ Submodule.span ℂ (Set.range fun n : Fin T.N =>
      T.energyEigenstate (T.quantaWaveNumber n))
```

lemma energyEigenstate_orthonormal [source]

The normalized energy eigenstates `N ^ (-1/2) • energyEigenstate k` are orthonormal.

```
lemma energyEigenstate_orthonormal :
    Orthonormal ℂ (fun n : Fin T.N =>
      (((Real.sqrt T.N)⁻¹ : ℝ) : ℂ) • T.energyEigenstate (T.quantaWaveNumber n))
```

def energyEigenbasis [source]

The normalized energy eigenstates form an orthonormal basis of the Hilbert space.

```
noncomputable def energyEigenbasis : OrthonormalBasis (Fin T.N) ℂ T.HilbertSpace
```

lemma energyEigenbasis_apply [source]

The n-th vector of the basis is the normalized energy eigenstate of the n-th wavenumber.

```
@[simp]
lemma energyEigenbasis_apply (n : Fin T.N) :
    T.energyEigenbasis n =
      (((Real.sqrt T.N)⁻¹ : ℝ) : ℂ) • T.energyEigenstate (T.quantaWaveNumber n)
```

lemma hasEigenvalue_hamiltonian_iff [source]

The eigenvalues of the Hamiltonian are exactly the energies `E0 - 2 t cos (k a)` of the quantized wavenumbers.

```
lemma hasEigenvalue_hamiltonian_iff (E : ℂ) :
    Module.End.HasEigenvalue T.hamiltonian E ↔
      ∃ k : T.QuantaWaveNumber, E = T.energyEigenvalue k
```

2.22 Physlib.Cosmology.FLRW.Age

Physlib/Cosmology/FLRW/Age.lean on GitHub.

def age [source]

The age of the universe `t₀ = H₀⁻¹ ∫₀¹ dx / (x E(x))` for a reduced Hubble function `E`: the time elapsed since `a = 0`, obtained by integrating `dt = da / (a H)` with `H = H₀ E(a / a₀)` (interval integral; no convergence is asserted here).

```
noncomputable def age (H₀ : ℝ) (E : ℝ → ℝ) : ℝ
```

lemma age_eq [source]

The age is proportional to `1 / H₀`.

```
lemma age_eq (H₀ : ℝ) (E : ℝ → ℝ) : age H₀ E = (1 / H₀) * age 1 E
```

2.23 Physlib.Cosmology.FLRW.Basic

Physlib/Cosmology/FLRW/Basic.lean on GitHub.

def curvature [source]

The curvature parameter of the spatial geometry, the constant `k` of the first-order Friedmann equation `FLRW.FriedmannEquation.FirstOrderFriedmann: - 1 / k ^ 2` for `Spherical k`, whose profile `S` is that of a sphere of radius `|k|`, `- 0` for `Flat`, `- -1 / k ^ 2` for `Saddle k`, whose profile `S` is that of a hyperbolic space of curvature radius `|k|`.

```
noncomputable def curvature : SpatialGeometry → ℝ
  | Spherical k _ => 1 / k ^ 2
  | Flat => 0
  | Saddle k _ => -1 / k ^ 2
```

lemma curvature_flat [source]

```
@[simp]
lemma curvature_flat : Flat.curvature = 0
```

lemma curvature_spherical_pos [source]

The spherical geometry has a positive curvature parameter.

```
lemma curvature_spherical_pos (k : ℝ) (h : k < 0) : 0 < (Spherical k h).curvature
```

lemma curvature_saddle_neg [source]

The saddle geometry has a negative curvature parameter.

```
lemma curvature_saddle_neg (k : ℝ) (h : k > 0) : (Saddle k h).curvature < 0
```

structure CosmicFluid [source]

The cosmic fluid filling an FLRW model, described by its total energy density `ρ` and its pressure `p`, both functions of cosmic time, as in the first- and second-order Friedmann equations `FLRW.FriedmannEquation.FirstOrderFriedmann` and `FLRW.FriedmannEquation.SecondOrderFriedma`

```
structure CosmicFluid where
  ρ : Time → ℝ
  p : Time → ℝ
```

def empty [source]

The empty cosmic fluid, with zero energy density and zero pressure at all times.

```
def CosmicFluid.empty : CosmicFluid
```

def cosmologicalConstantDensity [source]

The density `ρ_Λ = Λ c² / (8 π G)` associated with the cosmological constant.

```
noncomputable def FLRW.FriedmannEquation.cosmologicalConstantDensity (Λ G c : ℝ) : ℝ
```

def cosmologicalConstantPressure [source]

The pressure `p_Λ = - ρ_Λ c²` associated with the cosmological constant.

```
noncomputable def FLRW.FriedmannEquation.cosmologicalConstantPressure (Λ G c : ℝ) : ℝ
```

def addCosmologicalConstant [source]

The cosmic fluid `F` with the cosmological constant folded in as a fluid.

```
noncomputable def CosmicFluid.addCosmologicalConstant (F : CosmicFluid) (Λ G c : ℝ) :
    CosmicFluid
```

lemma a_ne_zero [source]

```
lemma a_ne_zero {t : Time} (ht : t ∈ U.I) : U.a t ≠ 0
```

lemma differentiableAt_a [source]

```
lemma differentiableAt_a {t : Time} (ht : t ∈ U.I) : DifferentiableAt ℝ U.a t
```

lemma differentiableAt_deriv_a [source]

```
lemma differentiableAt_deriv_a {t : Time} (ht : t ∈ U.I) : DifferentiableAt ℝ (∂ₜ U.a) t
```

lemma mk_mem_I_of_mem_Icc [source]

The times between two times of the interval `U.I` of an FLRW model are in `U.I`.

```
lemma mk_mem_I_of_mem_Icc (U : FLRW) {σ τ x : ℝ} (hσ : (⟨σ⟩ : Time) ∈ U.I)
    (hτ : (⟨τ⟩ : Time) ∈ U.I) (hx : x ∈ Set.Icc σ τ) : (⟨x⟩ : Time) ∈ U.I
```

lemma continuousOn_deriv_scaleFactor_mk [source]

Along the time coordinate, the derivative of the scale factor is continuous on any segment of the interval `U.I`.

```
lemma continuousOn_deriv_scaleFactor_mk (U : FLRW) {σ τ : ℝ} (hσ : (⟨σ⟩ : Time) ∈ U.I)
    (hτ : (⟨τ⟩ : Time) ∈ U.I) : ContinuousOn (fun x : ℝ => ∂ₜ U.a ⟨x⟩) (Set.Icc σ τ)
```

lemma scaleFactor_le_tangent [source]

If the scale factor decelerates on the interval (`∂ₜ ∂ₜ a ≤ 0` on `U.I`), it lies below its tangent line at any time `t₀` of `U.I`: `a t ≤ a t₀ + ∂ₜ a t₀ (t - t₀)` for `t ∈ U.I`. The derivative is antitone on the segment between `t` and `t₀`, which lies in `U.I`, and the mean value inequality concludes.

```
lemma scaleFactor_le_tangent (U : FLRW) (hdec : ∀ s ∈ U.I, ∂ₜ (∂ₜ U.a) s ≤ 0) {t₀ t : Time}
    (ht₀ : t₀ ∈ U.I) (ht : t ∈ U.I) : U.a t ≤ U.a t₀ + ∂ₜ U.a t₀ * (t.val - t₀.val)
```

lemma lt_of_mem_I_of_decelerating [source]

The Big-Bang bound for an FLRW model: if the scale factor decelerates on `U.I` and the model expands at `t₀ ∈ U.I`, every time of `U.I` is later than `t₀ - a t₀ / ∂ₜ a t₀`. Earlier, the tangent line, and hence the scale factor, would be non-positive.

```
lemma lt_of_mem_I_of_decelerating (U : FLRW) (hdec : ∀ s ∈ U.I, ∂ₜ (∂ₜ U.a) s ≤ 0)
    {t₀ : Time} (ht₀ : t₀ ∈ U.I) (h0 : 0 < ∂ₜ U.a t₀) {t : Time} (ht : t ∈ U.I) :
    t₀.val - U.a t₀ / ∂ₜ U.a t₀ < t.val
```

lemma bddBelow_I_of_decelerating [source]

Hence the interval of an FLRW model that decelerates on `U.I` and expands at one of its times is bounded below.

```
lemma bddBelow_I_of_decelerating (U : FLRW) (hdec : ∀ s ∈ U.I, ∂ₜ (∂ₜ U.a) s ≤ 0) {t₀ : Time}
    (ht₀ : t₀ ∈ U.I) (h0 : 0 < ∂ₜ U.a t₀) : BddBelow U.I
```

structure FriedmannSolution [source]

The proposition that the FLRW model `U` filled with the cosmic fluid `F` solves the Friedmann equations: the first- and second-order Friedmann equations, with cosmological constant `Λ`, Newton's constant `G` and speed of light `c`, hold at every time of the interval `U.I`, the curvature parameter being the one of the spatial geometry. The continuity equation of the fluid is not part of it: it follows from the two Friedmann equations (`FLRW.FriedmannSolution.continuityEquation` in `Physlib.Cosmology.FLRW.MatterContent`).

```
structure FriedmannSolution (U : FLRW) (F : CosmicFluid) (Λ G c : ℝ) : Prop where
  firstOrder : ∀ t ∈ U.I, FirstOrderFriedmann U.a F.ρ U.geometry.curvature Λ G c t
  secondOrder : ∀ t ∈ U.I, SecondOrderFriedmann U.a F.ρ F.p Λ G c t
```

2.24 Physlib.Cosmology.FLRW.DensityParameters

Physlib/Cosmology/FLRW/DensityParameters.lean on GitHub.

def criticalDensity [source]

The critical density `ρ_cr = 3 H² / (8 π G)` of an FLRW model at time `t`, where `H` is the Hubble parameter of its scale factor. It is the density of a spatially flat universe without cosmological constant expanding at the rate `H`.

```
noncomputable def criticalDensity (G : ℝ) (t : Time) : ℝ
```

def densityParameter [source]

The density parameter `Ω = ρ / ρ_cr` of a cosmic fluid in an FLRW model.

```
noncomputable def densityParameter (F : CosmicFluid) (G : ℝ) (t : Time) : ℝ
```

def curvatureDensityParameter [source]

The curvature density parameter `Ω_K = - k c² / (H² a²)` of an FLRW model, where `k` is the curvature parameter of its spatial geometry. When `H`, `a` and `c` are nonzero, it is positive for a saddle geometry, negative for a spherical one and zero for the flat one.

```
noncomputable def curvatureDensityParameter (c : ℝ) (t : Time) : ℝ
```

def lambdaDensityParameter [source]

The density parameter `Ω_Λ = Λ c² / (3 H²)` of the cosmological constant in an FLRW model.

```
noncomputable def lambdaDensityParameter (Λ c : ℝ) (t : Time) : ℝ
```

lemma lambdaDensityParameter_eq_densityParameter [source]

For `G ≠ 0`, `Ω_Λ` is the density parameter of the empty fluid with the cosmological constant folded in.

```
lemma lambdaDensityParameter_eq_densityParameter {Λ G c : ℝ} (hG : G ≠ 0) (t : Time) :
    U.lambdaDensityParameter Λ c t
      = U.densityParameter (CosmicFluid.empty.addCosmologicalConstant Λ G c) G t
```

lemma densityParameter_addCosmologicalConstant [source]

For `G ≠ 0`, folding the cosmological constant into a cosmic fluid adds `Ω_Λ` to its density parameter.

```
lemma densityParameter_addCosmologicalConstant (F : CosmicFluid) {Λ G c : ℝ} (hG : G ≠ 0)
    (t : Time) :
    U.densityParameter (F.addCosmologicalConstant Λ G c) G t
      = U.densityParameter F G t + U.lambdaDensityParameter Λ c t
```

lemma densityParameter_add_lambda_add_curvature [source]

The closure relation `Ω + Ω_Λ + Ω_K = 1` for a solution of the Friedmann equations, at the times of the interval of the model at which `H ≠ 0`.

```
lemma densityParameter_add_lambda_add_curvature (h : U.FriedmannSolution F Λ G c) {t : Time}
    (ht : t ∈ U.I) (hH : hubbleConstant U.a t ≠ 0) :
    U.densityParameter F G t + U.lambdaDensityParameter Λ c t
      + U.curvatureDensityParameter c t = 1
```

lemma densityParameter_add_curvature [source]

The closure relation `Ω + Ω_K = 1` for a solution of the Friedmann equations without cosmological constant, at the times of the interval of the model at which `H ≠ 0`.

```
lemma densityParameter_add_curvature (h : U.FriedmannSolution F 0 G c) {t : Time} (ht : t ∈ U.I
    )
    (hH : hubbleConstant U.a t ≠ 0) :
    U.densityParameter F G t + U.curvatureDensityParameter c t = 1
```

2.25 Physlib.Cosmology.FLRW.DeSitter

Physlib/Cosmology/FLRW/DeSitter.lean on GitHub.

def deSitter [source]

The de Sitter model, defined at all times: the FLRW model with the scale factor `deSitterScaleFactor a₀ σ Λ c`, `a₀ > 0`, and the flat spatial geometry.

```
noncomputable def _root_.Cosmology.FLRW.deSitter (a₀ σ Λ c : ℝ) (ha₀ : 0 < a₀) : FLRW where
```

lemma friedmannSolution_deSitter [source]

The de Sitter model filled with the empty cosmic fluid solves the Friedmann equations with cosmological constant `Λ > 0`, for `σ = ±1`.

```
lemma _root_.Cosmology.FLRW.friedmannSolution_deSitter {a₀ σ Λ : ℝ} (G c : ℝ) (hΛ : 0 < Λ)
    (ha₀ : 0 < a₀) (hσ : σ = 1 ∨ σ = -1) :
    (FLRW.deSitter a₀ σ Λ c ha₀).FriedmannSolution CosmicFluid.empty Λ G c
```

lemma not_bddBelow_deSitter_I [source]

The de Sitter model, which solves the Friedmann equations with `0 < Λ` (`FLRW.friedmannSolution_deSitter`), is defined at all times: its interval is not bounded below.

```
lemma _root_.Cosmology.FLRW.not_bddBelow_deSitter_I (a₀ σ Λ c : ℝ) (ha₀ : 0 < a₀) :
    ¬ BddBelow (FLRW.deSitter a₀ σ Λ c ha₀).I
```

2.26 Physlib.Cosmology.FLRW.Distances

Physlib/Cosmology/FLRW/Distances.lean on GitHub.

def comovingDistance [source]

The line-of-sight comoving distance `χ = c ∫_t^{t₀} dτ / a(τ)` of a source emitting at `t` and observed at `t₀`.

```
noncomputable def comovingDistance (c : ℝ) (t t₀ : Time) : ℝ
```

lemma deriv_comovingDistance [source]

For `t` and `t₀` in the interval of the model, `∂ₜ χ = - c / a`.

```
lemma deriv_comovingDistance {c : ℝ} {t₀ t : Time} (ht₀ : t₀ ∈ U.I) (ht : t ∈ U.I) :
    ∂ₜ (fun s => U.comovingDistance c s t₀) t = -c / U.a t
```

def redshift [source]

The redshift `z` of a source emitting at `t` and observed at `t₀`, with `1 + z = a(t₀) / a(t)`.

```
noncomputable def redshift (t₀ t : Time) : ℝ
```

lemma one_add_redshift [source]

```
lemma one_add_redshift (t₀ t : Time) : 1 + U.redshift t₀ t = U.a t₀ / U.a t
```

lemma deriv_redshift [source]

For `t` in the interval of the model, `∂ₜ z = -(1 + z) H`.

```
lemma deriv_redshift {t₀ t : Time} (ht : t ∈ U.I) :
    ∂ₜ (U.redshift t₀) t = -(1 + U.redshift t₀ t) * hubbleConstant U.a t
```

lemma deriv_scaleFactor_eq_mul_hubbleConstant [source]

For `t` in the interval of the model, `∂ₜ a = a H`.

```
lemma deriv_scaleFactor_eq_mul_hubbleConstant {t : Time} (ht : t ∈ U.I) :
    ∂ₜ U.a t = U.a t * hubbleConstant U.a t
```

lemma deriv_comovingDistance_eq_mul_deriv_redshift [source]

For `t` and `t₀` in the interval of the model with `H(t) ≠ 0`, `∂ₜ χ = (c / (a(t₀) H)) ∂ₜ z`. For `a(t₀) = 1`, this is the change of variables `dχ = c dz / H`.

```
lemma deriv_comovingDistance_eq_mul_deriv_redshift {c : ℝ} {t₀ t : Time} (ht₀ : t₀ ∈ U.I)
    (ht : t ∈ U.I) (hH : hubbleConstant U.a t ≠ 0) :
    ∂ₜ (fun s => U.comovingDistance c s t₀) t
      = c / (U.a t₀ * hubbleConstant U.a t) * ∂ₜ (U.redshift t₀) t
```

def properDistance [source]

The proper distance `D = a χ` at fixed comoving distance `χ`.

```
noncomputable def properDistance (χ : ℝ) (t : Time) : ℝ
```

lemma deriv_properDistance [source]

The Hubble-Lemaître law `∂ₜ D = H D`, for `t` in the interval of the model.

```
lemma deriv_properDistance {χ : ℝ} {t : Time} (ht : t ∈ U.I) :
    ∂ₜ (U.properDistance χ) t = hubbleConstant U.a t * U.properDistance χ t
```

def hubbleRadius [source]

The Hubble radius `R_H = c / H`.

```
noncomputable def hubbleRadius (c : ℝ) (t : Time) : ℝ
```

lemma deriv_properDistance_eq_iff [source]

For `t` in the interval of the model with `H(t) ≠ 0`, the recession velocity `∂ₜ D` equals `c` exactly at the Hubble radius.

```
lemma deriv_properDistance_eq_iff {χ c : ℝ} {t : Time} (ht : t ∈ U.I)
    (hH : hubbleConstant U.a t ≠ 0) :
    ∂ₜ (U.properDistance χ) t = c ↔ U.properDistance χ t = U.hubbleRadius c t
```

def transverseComovingDistance [source]

The transverse comoving distance `r(χ) = S(χ)` for the spatial geometry of an FLRW model.

```
noncomputable def transverseComovingDistance (χ : ℝ) : ℝ
```

lemma transverseComovingDistance_of_curvature_pos [source]

For a positive curvature parameter `K`, `r(χ) = (1 / √K) sin (√K χ)`.

```
lemma transverseComovingDistance_of_curvature_pos (hK : 0 < U.geometry.curvature) (χ : ℝ) :
    U.transverseComovingDistance χ
      = 1 / √U.geometry.curvature * Real.sin (√U.geometry.curvature * χ)
```

lemma transverseComovingDistance_of_curvature_eq_zero [source]

For a zero curvature parameter, `r(χ) = χ`.

```
lemma transverseComovingDistance_of_curvature_eq_zero (hK : U.geometry.curvature = 0) (χ : ℝ) :
    U.transverseComovingDistance χ = χ
```

lemma transverseComovingDistance_of_curvature_neg [source]

For a negative curvature parameter `K`, `r(χ) = (1 / √(-K)) sinh (√(-K) χ)`.

```
lemma transverseComovingDistance_of_curvature_neg (hK : U.geometry.curvature < 0) (χ : ℝ) :
    U.transverseComovingDistance χ
      = 1 / √(-U.geometry.curvature) * Real.sinh (√(-U.geometry.curvature) * χ)
```

def particleHorizon [source]

The particle horizon at `t`, the integral of `c / a` over the times of the interval of the model before `t`. It is a Lebesgue integral, equal to `0` when the integrand is not integrable.

```
noncomputable def particleHorizon (c : ℝ) (t : Time) : ℝ
```

def eventHorizon [source]

The event horizon at `t`, the integral of `c / a` over the times of the interval of the model after `t`. It is a Lebesgue integral, equal to `0` when the integrand is not integrable.

```
noncomputable def eventHorizon (c : ℝ) (t : Time) : ℝ
```

lemma particleHorizon_einsteinDeSitter [source]

The particle horizon of the Einstein-de Sitter model at `t > 0` is `3 c t₀^(2/3) t^(1/3)`.

```
lemma particleHorizon_einsteinDeSitter {t₀ : Time} {c : ℝ} (ht₀ : 0 < t₀.val) {t : Time}
    (ht : 0 < t.val) :
    (einsteinDeSitter t₀ ht₀).particleHorizon c t
      = 3 * c * t₀.val ^ (2 / 3 : ℝ) * t.val ^ (1 / 3 : ℝ)
```

2.27 Physlib.Cosmology.FLRW.EnergyConditions

Physlib/Cosmology/FLRW/EnergyConditions.lean on GitHub.

def NullEnergyCondition [source]

The null energy condition at time `t`, `0 ≤ ρ + p / c²`, for the total energy density `ρ` and the pressure `p` of the cosmic fluid, `c` being the speed of light.

```
def NullEnergyCondition : Prop
```

def WeakEnergyCondition [source]

The weak energy condition at time `t`, `0 ≤ ρ ∧ 0 ≤ ρ + p / c²`, for the total energy density `ρ` and the pressure `p` of the cosmic fluid, `c` being the speed of light.

```
def WeakEnergyCondition : Prop
```

def StrongEnergyCondition [source]

The strong energy condition at time `t`, `0 ≤ ρ + p / c² ∧ 0 ≤ ρ + 3 p / c²`, for the total energy density `ρ` and the pressure `p` of the cosmic fluid, `c` being the speed of light.

```
def StrongEnergyCondition : Prop
```

def DominantEnergyCondition [source]

The dominant energy condition at time `t`, `0 ≤ ρ ∧ |p| ≤ ρ c²`, for the total energy density `ρ` and the pressure `p` of the cosmic fluid, `c` being the speed of light.

```
def DominantEnergyCondition : Prop
```

lemma weakEnergyCondition [source]

For `c ≠ 0`, the dominant energy condition implies the weak energy condition.

```
lemma DominantEnergyCondition.weakEnergyCondition (hc : c ≠ 0)
    (h : F.DominantEnergyCondition c t) : F.WeakEnergyCondition c t
```

lemma WeakEnergyCondition.nullEnergyCondition [source]

The weak energy condition implies the null energy condition.

```
lemma WeakEnergyCondition.nullEnergyCondition (h : F.WeakEnergyCondition c t) :
    F.NullEnergyCondition c t
```

lemma StrongEnergyCondition.nullEnergyCondition [source]

The strong energy condition implies the null energy condition.

```
lemma StrongEnergyCondition.nullEnergyCondition (h : F.StrongEnergyCondition c t) :
    F.NullEnergyCondition c t
```

2.28 Physlib.Cosmology.FLRW.FriedmannSolution

Physlib/Cosmology/FLRW/FriedmannSolution.lean on GitHub.

lemma friedmannSolution_iff_addCosmologicalConstant [source]

An FLRW model filled with the cosmic fluid `F` solves the Friedmann equations with the cosmological constant `Λ` if and only if it solves them without cosmological constant for the fluid `F` with `Λ` folded in.

```
lemma _root_.Cosmology.FLRW.friedmannSolution_iff_addCosmologicalConstant {U : FLRW}
    {F : CosmicFluid} {Λ G c : ℝ} (hG : G ≠ 0) (hc : c ≠ 0) :
    U.FriedmannSolution F Λ G c ↔ U.FriedmannSolution (F.addCosmologicalConstant Λ G c) 0 G c
```

lemma deriv_deriv_scaleFactor_pos_iff [source]

On the interval of a solution without cosmological constant, the second derivative `∂ₜ ∂ₜ a` of the scale factor is positive (the expansion accelerates) if and only if `ρ + 3 p / c² < 0`, that is, if and only if the strong energy condition of the cosmic fluid fails through its second inequality.

```
lemma deriv_deriv_scaleFactor_pos_iff (h : U.FriedmannSolution F 0 G c) (hG : 0 < G) {t : Time}
    (ht : t ∈ U.I) : 0 < ∂ₜ (∂ₜ U.a) t ↔ F.ρ t + 3 * F.p t / c ^ 2 < 0
```

lemma deriv_deriv_scaleFactor_pos_iff_lambda [source]

With cosmological constant `Λ`, on the interval of a solution, `∂ₜ ∂ₜ a` is positive if and only if the total fluid, `Λ` folded in as the `w = -1` component, violates `0 ≤ ρ + 3 p / c²`.

```
lemma deriv_deriv_scaleFactor_pos_iff_lambda (h : U.FriedmannSolution F Λ G c) (hG : 0 < G)
    (hc : c ≠ 0) {t : Time} (ht : t ∈ U.I) :
    0 < ∂ₜ (∂ₜ U.a) t ↔ (F.ρ t + cosmologicalConstantDensity Λ G c)
      + 3 * (F.p t + cosmologicalConstantPressure Λ G c) / c ^ 2 < 0
```

lemma decelerationParameter_eq_sum [source]

For a solution of the Friedmann equations without cosmological constant whose cosmic fluid is a sum of barotropic components of pressure `w_i ρ_i c²`, the deceleration parameter is `q = ½ Σ_i Ω_i (1 + 3 w_i)` at the times of the interval of the model at which `H ≠ 0`, for `G ≠ 0` and `c ≠ 0`.

```
lemma decelerationParameter_eq_sum {n : ℕ} {Fi : Fin n → CosmicFluid} {w : Fin n → ℝ}
    (h : U.FriedmannSolution F 0 G c) (hG : G ≠ 0) (hc : c ≠ 0)
    (hρ : ∀ s, F.ρ s = ∑ i, (Fi i).ρ s) (hp : ∀ s, F.p s = ∑ i, (Fi i).p s)
    (hw : ∀ i, (Fi i).p = barotropicPressure (w i) (Fi i).ρ c) {t : Time} (ht : t ∈ U.I)
    (hH : hubbleConstant U.a t ≠ 0) :
    decelerationParameter U.a t
      = 1 / 2 * ∑ i, U.densityParameter (Fi i) G t * (1 + 3 * w i)
```

lemma decelerationParameter_lambdaCDM [source]

For a solution of the Friedmann equations whose cosmic fluid is the sum of matter (zero pressure) and radiation (pressure `ρ_r c² / 3`), the deceleration parameter is `q = ½ (Ω_m + 2 Ω_r - 2 Ω_Λ)` at the times of the interval of the model, for `c ≠ 0` (the ΛCDM case).

```
lemma decelerationParameter_lambdaCDM {Fm Fr : CosmicFluid} (h : U.FriedmannSolution F Λ G c)
    (hc : c ≠ 0) (hρ : ∀ s, F.ρ s = Fm.ρ s + Fr.ρ s) (hp : ∀ s, F.p s = Fm.p s + Fr.p s)
    (hm : Fm.p = barotropicPressure 0 Fm.ρ c) (hr : Fr.p = barotropicPressure (1 / 3) Fr.ρ c)
    {t : Time} (ht : t ∈ U.I) :
    decelerationParameter U.a t = 1 / 2 * (U.densityParameter Fm G t
      + 2 * U.densityParameter Fr G t - 2 * U.lambdaDensityParameter Λ c t)
```

lemma deriv_scaleFactor_ne_zero [source]

For a spatial geometry of non-positive curvature parameter, `0 ≤ Λ` and a positive density at a time `t` of the interval, the first-order Friedmann equation forces `0 < H²` at `t`: the derivative `∂ₜ a` of the scale factor does not vanish at `t`.

```
lemma deriv_scaleFactor_ne_zero (h : U.FriedmannSolution F Λ G c)
    (hk : U.geometry.curvature ≤ 0) (hΛ : 0 ≤ Λ) (hG : 0 < G) {t : Time} (ht : t ∈ U.I)
    (hρ : 0 < F.ρ t) : ∂ₜ U.a t ≠ 0
```

lemma deriv_scaleFactor_pos [source]

If moreover the density is positive on the interval and the model expands at one time `t₀` of `U.I`, it expands at all times of `U.I`: with a non-positive curvature parameter, `0 ≤ Λ` and a positive density, an expanding model expands forever.

```
lemma deriv_scaleFactor_pos (h : U.FriedmannSolution F Λ G c) (hk : U.geometry.curvature ≤ 0)
    (hΛ : 0 ≤ Λ) (hG : 0 < G) (hρ : ∀ s ∈ U.I, 0 < F.ρ s) {t₀ : Time} (ht₀ : t₀ ∈ U.I)
    (h0 : 0 < ∂ₜ U.a t₀) {t : Time} (ht : t ∈ U.I) : 0 < ∂ₜ U.a t
```

lemma deriv_deriv_scaleFactor_nonpos [source]

With `0 ≤ ρ`, `0 ≤ p` and `Λ = 0` at a time `t` of the interval, the second-order Friedmann equation gives `∂ₜ ∂ₜ a ≤ 0` at `t`: the expansion decelerates.

```
lemma deriv_deriv_scaleFactor_nonpos (h : U.FriedmannSolution F 0 G c) (hG : 0 < G)
    (hc : c ≠ 0) {t : Time} (ht : t ∈ U.I) (hρ : 0 ≤ F.ρ t) (hp : 0 ≤ F.p t) :
    ∂ₜ (∂ₜ U.a) t ≤ 0
```

lemma lt_of_mem_I [source]

The Big-Bang bound for a solution of the Friedmann equations with `0 ≤ ρ`, `0 ≤ p` and `Λ = 0` on its interval: if the model expands at `t₀ ∈ U.I`, every time of `U.I` is later than `t₀ - 1 / H(t₀)`, the Big Bang being at most `1 / H(t₀)` in the past.

```
lemma lt_of_mem_I (h : U.FriedmannSolution F 0 G c) (hG : 0 < G) (hc : c ≠ 0)
    (hρ : ∀ s ∈ U.I, 0 ≤ F.ρ s) (hp : ∀ s ∈ U.I, 0 ≤ F.p s) {t₀ : Time} (ht₀ : t₀ ∈ U.I)
    (h0 : 0 < ∂ₜ U.a t₀) {t : Time} (ht : t ∈ U.I) :
    t₀.val - 1 / hubbleConstant U.a t₀ < t.val
```

lemma bddBelow_I [source]

Under the same assumptions, the interval of the model is bounded below.

```
lemma bddBelow_I (h : U.FriedmannSolution F 0 G c) (hG : 0 < G) (hc : c ≠ 0)
    (hρ : ∀ s ∈ U.I, 0 ≤ F.ρ s) (hp : ∀ s ∈ U.I, 0 ≤ F.p s) {t₀ : Time} (ht₀ : t₀ ∈ U.I)
    (h0 : 0 < ∂ₜ U.a t₀) : BddBelow U.I
```

2.29 Physlib.Cosmology.FLRW.MatterContent

Physlib/Cosmology/FLRW/MatterContent.lean on GitHub.

def ContinuityEquation [source]

The continuity equation of the cosmic fluid at the instant `t`: `∂ₜ ρ + 3 H (ρ + p / c²) = 0`, with `H = hubbleConstant a`.

```
def ContinuityEquation (a ρ p : Time → ℝ) (c : ℝ) (t : Time) : Prop
```

lemma deriv_firstOrderFriedmann [source]

Differentiating the first-order Friedmann equation, assumed at all times near `t`, at `t`: `2 H (a''/a - H²) = (8 π G / 3) ρ' + 2 k c² H / a²`.

```
lemma deriv_firstOrderFriedmann {a ρ : Time → ℝ} {k Λ G c : ℝ} {t : Time} (ha : a t ≠ 0)
    (hd1 : DifferentiableAt ℝ a t) (hd2 : DifferentiableAt ℝ (∂ₜ a) t)
    (hdρ : DifferentiableAt ℝ ρ t) (hF1 : ∀ᶠ s in 𝒩 t, FirstOrderFriedmann a ρ k Λ G c s) :
    2 * (∂ₜ a t / a t) * (∂ₜ (∂ₜ a) t / a t - (∂ₜ a t / a t) ^ 2)
      = 8 * π * G / 3 * ∂ₜ ρ t + 2 * k * c ^ 2 * (∂ₜ a t / a t) / (a t) ^ 2
```

lemma continuityEquation_of_friedmann [source]

The continuity equation follows from the first-order Friedmann equation, assumed at all times near `t`, and the second-order Friedmann equation at `t`; `a` must be twice differentiable and `ρ` differentiable at `t`.

```
lemma continuityEquation_of_friedmann {a ρ p : Time → ℝ} {k Λ G c : ℝ} {t : Time}
    (hG : G ≠ 0) (ha : a t ≠ 0) (hd1 : DifferentiableAt ℝ a t)
    (hd2 : DifferentiableAt ℝ (∂ₜ a) t)
    (hdρ : DifferentiableAt ℝ ρ t) (hF1 : ∀ᶠ s in 𝓝 t, FirstOrderFriedmann a ρ k Λ G c s)
    (hF2 : SecondOrderFriedmann a ρ p Λ G c t) :
    ContinuityEquation a ρ p c t
```

lemma secondOrderFriedmann_of_continuityEquation [source]

The second-order Friedmann equation at `t` follows from the first-order Friedmann equation, assumed at all times near `t`, and the continuity equation at `t`, provided `∂ₜ a t ≠ 0`. Together with `continuityEquation_of_friedmann`, the three equations are not independent.

```
lemma secondOrderFriedmann_of_continuityEquation {a ρ p : Time → ℝ} {k Λ G c : ℝ}
    {t : Time} (ha : a t ≠ 0) (ha' : ∂ₜ a t ≠ 0) (hd1 : DifferentiableAt ℝ a t)
    (hd2 : DifferentiableAt ℝ (∂ₜ a) t) (hdρ : DifferentiableAt ℝ ρ t)
    (hF1 : ∀ᶠ s in 𝓝 t, FirstOrderFriedmann a ρ k Λ G c s)
    (hC : ContinuityEquation a ρ p c t) :
    SecondOrderFriedmann a ρ p Λ G c t
```

lemma continuityEquation [source]

If the FLRW model `U` filled with the cosmic fluid `F` solves the Friedmann equations, the continuity equation holds at every time of the interval `U.I` at which the energy density is differentiable: it is obtained from the two Friedmann equations through `continuityEquation_of_friedmann`.

```
lemma _root_.Cosmology.FLRW.FriedmannSolution.continuityEquation {U : FLRW} {F : CosmicFluid}
    {Λ G c : ℝ} (h : U.FriedmannSolution F Λ G c) (hG : G ≠ 0) {t : Time} (ht : t ∈ U.I)
    (hdρ : DifferentiableAt ℝ F.ρ t) :
    ContinuityEquation U.a F.ρ F.p c t
```

def barotropicPressure [source]

The barotropic pressure `p = w ρ c²` with constant equation-of-state parameter `w`.

```
noncomputable def barotropicPressure (w : ℝ) (ρ : Time → ℝ) (c : ℝ) : Time → ℝ
```

lemma deriv_of_continuityEquation_barotropic [source]

Under the barotropic continuity equation, `∂ₜ ρ = -3 (1 + w) H ρ`.

```
lemma deriv_of_continuityEquation_barotropic {a ρ : Time → ℝ} {w c : ℝ} {t : Time} (hc : c ≠ 0)
    (hC : ContinuityEquation a ρ (barotropicPressure w ρ c) c t) :
    ∂ₜ ρ t = -3 * (1 + w) * hubbleConstant a t * ρ t
```

lemma density_scaling [source]

The density scaling law: under the barotropic continuity equation at all times, with `a` and `ρ` differentiable and `a > 0`, `ρ t = ρ t₀ (a t / a t₀)^(-3(1+w))`. The proof shows that `ρ a^(3(1+w))` has zero derivative along the time chart.

```
lemma density_scaling {a ρ : Time → ℝ} {w c : ℝ} (hc : c ≠ 0) (hd1 : Differentiable ℝ a)
    (hdρ : Differentiable ℝ ρ) (hapos : ∀ s, 0 < a s)
    (hC : ∀ s, ContinuityEquation a ρ (barotropicPressure w ρ c) c s) (t t₀ : Time) :
    ρ t = ρ t₀ * (a t / a t₀) ^ (-(3 * (1 + w)))
```

lemma continuityEquation_of_scaling [source]

Conversely, `ρ = ρ₀ (a / a₀)^(-3(1+w))` satisfies the barotropic continuity equation at all times, for `a` differentiable and positive.

```
lemma continuityEquation_of_scaling {a : Time → ℝ} {w c ρ₀ a₀ : ℝ} (hc : c ≠ 0) (ha₀ : a₀ ≠ 0)
    (hd1 : Differentiable ℝ a) (hapos : ∀ s, 0 < a s) (t : Time) :
    ContinuityEquation a (fun s => ρ₀ * (a s / a₀) ^ (-(3 * (1 + w))))
      (barotropicPressure w (fun s => ρ₀ * (a s / a₀) ^ (-(3 * (1 + w)))) c) c t
```

lemma density_scaling_dust [source]

Dust, `w = 0`: `ρ ∝ a⁻³`.

```
lemma density_scaling_dust {a ρ : Time → ℝ} {c : ℝ} (hc : c ≠ 0) (hd1 : Differentiable ℝ a)
    (hdρ : Differentiable ℝ ρ) (hapos : ∀ s, 0 < a s)
    (hC : ∀ s, ContinuityEquation a ρ (barotropicPressure 0 ρ c) c s) (t t₀ : Time) :
    ρ t = ρ t₀ * (a t₀ / a t) ^ 3
```

lemma density_scaling_radiation [source]

Radiation, `w = 1/3`: `ρ ∝ a⁻⁴`.

```
lemma density_scaling_radiation {a ρ : Time → ℝ} {c : ℝ} (hc : c ≠ 0) (hd1 : Differentiable ℝ a
    )
    (hdρ : Differentiable ℝ ρ) (hapos : ∀ s, 0 < a s)
    (hC : ∀ s, ContinuityEquation a ρ (barotropicPressure (1 / 3) ρ c) c s) (t t₀ : Time) :
    ρ t = ρ t₀ * (a t₀ / a t) ^ 4
```

lemma density_scaling_vacuum [source]

Vacuum energy, `w = -1`: `ρ` is constant.

```
lemma density_scaling_vacuum {a ρ : Time → ℝ} {c : ℝ} (hc : c ≠ 0) (hd1 : Differentiable ℝ a)
    (hdρ : Differentiable ℝ ρ) (hapos : ∀ s, 0 < a s)
    (hC : ∀ s, ContinuityEquation a ρ (barotropicPressure (-1) ρ c) c s) (t t₀ : Time) :
    ρ t = ρ t₀
```

lemma firstOrderFriedmann_iff_lambdaFluid [source]

The first-order Friedmann equation with `Λ` is the first-order Friedmann equation without `Λ` for the density `ρ + ρ_Λ`.

```
lemma firstOrderFriedmann_iff_lambdaFluid {a ρ : Time → ℝ} {k Λ G c : ℝ} {t : Time}
    (hG : G ≠ 0) :
```

```
    FirstOrderFriedmann a ρ k Λ G c t ↔
      FirstOrderFriedmann a (fun s => ρ s + cosmologicalConstantDensity Λ G c) k 0 G c t
```

lemma secondOrderFriedmann_iff_lambdaFluid [source]

The second-order Friedmann equation with `Λ` is the second-order Friedmann equation without `Λ` for the density `ρ + ρ_Λ` and the pressure `p + p_Λ`.

```
lemma secondOrderFriedmann_iff_lambdaFluid {a ρ p : Time → ℝ} {Λ G c : ℝ} {t : Time}
    (hG : G ≠ 0) (hc : c ≠ 0) :
    SecondOrderFriedmann a ρ p Λ G c t ↔
      SecondOrderFriedmann a (fun s => ρ s + cosmologicalConstantDensity Λ G c)
        (fun s => p s + cosmologicalConstantPressure Λ G c) 0 G c t
```

lemma cosmologicalConstantPressure_eq_barotropic [source]

`p_Λ` is the barotropic pressure of `ρ_Λ` with `w = -1`.

```
lemma cosmologicalConstantPressure_eq_barotropic (Λ G c : ℝ) (t : Time) :
    cosmologicalConstantPressure Λ G c
      = barotropicPressure (-1) (fun _ => cosmologicalConstantDensity Λ G c) c t
```

2.30 Physlib.Cosmology.FLRW.ReducedHubble

Physlib/Cosmology/FLRW/ReducedHubble.lean on GitHub.

def reducedHubble [source]

The reduced Hubble function `E(x) = √(Ω_Λ + Ω_m x⁻³ + Ω_r x⁻⁴ + Ω_K x⁻²)` of the standard cosmological model: the Hubble parameter in units of its value `H₀` at the reference time, as a function of the scale factor `x = a / a₀` in units of its reference value. The four terms are the shares of the cosmological constant, matter, radiation and curvature at the reference time, each diluted by its own power of `x`.

```
noncomputable def reducedHubble (ΩΛ Ωm Ωr ΩK x : ℝ) : ℝ
```

def equalityScaleFactorRadiationMatter [source]

The radiation-matter equality scale factor `a_eq = Ω_r / Ω_m`: the value of `x = a / a₀` at which the matter and radiation terms of `E²` are equal; radiation dominates before it.

```
noncomputable def equalityScaleFactorRadiationMatter (Ωr Ωm : ℝ) : ℝ
```

def equalityScaleFactorMatterLambda [source]

The matter-Λ equality scale factor `a_Λ = (Ω_m / Ω_Λ)^(1/3)`: the value of `x = a / a₀` at which the matter term of `E²` equals `Ω_Λ`; the cosmological constant dominates after it.

```
noncomputable def equalityScaleFactorMatterLambda (Ωm ΩΛ : ℝ) : ℝ
```

lemma equalityScaleFactorRadiationMatter_spec [source]

At `a_eq`, the matter and radiation terms of `E²` are equal.

```
lemma equalityScaleFactorRadiationMatter_spec {Ωr Ωm : ℝ} (hr : 0 < Ωr) (hm : 0 < Ωm) :
    Ωm * equalityScaleFactorRadiationMatter Ωr Ωm ^ (-3 : ℝ)
      = Ωr * equalityScaleFactorRadiationMatter Ωr Ωm ^ (-4 : ℝ)
```

lemma equalityScaleFactorMatterLambda_spec [source]

At `a_Λ`, the matter term of `E²` equals `Ω_Λ`.

```
lemma equalityScaleFactorMatterLambda_spec {Ωm ΩΛ : ℝ} (hm : 0 < Ωm) (hL : 0 < ΩΛ) :
    Ωm * equalityScaleFactorMatterLambda Ωm ΩΛ ^ (-3 : ℝ) = ΩΛ
```

lemma sq_hubbleConstant_eq_mul_densityParameters [source]

For a solution of the Friedmann equations whose density is that of matter plus that of radiation, take two times `t` and `t₀` of the interval of the model with `H(t₀) ≠ 0` at which `ρ_m(t) = ρ_m(t₀) (a(t₀) / a(t))³` and `ρ_r(t) = ρ_r(t₀) (a(t₀) / a(t))⁴`. Then `H² = H₀² (Ω_Λ + Ω_m x⁻³ + Ω_r x⁻⁴ + Ω_K x⁻²)` with `x = a(t) / a(t₀)` and the density parameters taken at `t₀`.

```
lemma sq_hubbleConstant_eq_mul_densityParameters (h : U.FriedmannSolution F Λ G c)
    (hρ : ∀ s, F.ρ s = Fm.ρ s + Fr.ρ s) {t t₀ : Time} (ht : t ∈ U.I) (ht₀ : t₀ ∈ U.I)
    (hH₀ : hubbleConstant U.a t₀ ≠ 0)
    (hm : Fm.ρ t = Fm.ρ t₀ * (U.a t₀ / U.a t) ^ 3) (hr : Fr.ρ t = Fr.ρ t₀ * (U.a t₀ / U.a t) ^
    4) :
    hubbleConstant U.a t ^ 2 = hubbleConstant U.a t₀ ^ 2
      * (U.lambdaDensityParameter Λ c t₀ + U.densityParameter Fm G t₀ * (U.a t / U.a t₀) ^ (-3
    : ℝ)
        + U.densityParameter Fr G t₀ * (U.a t / U.a t₀) ^ (-4 : ℝ)
        + U.curvatureDensityParameter c t₀ * (U.a t / U.a t₀) ^ (-2 : ℝ))
```

lemma sq_hubbleConstant_eq_reducedHubble [source]

`H² = H₀² E(a / a₀)²`, under the hypotheses of `sq_hubbleConstant_eq_mul_densityParameters`.

```
lemma sq_hubbleConstant_eq_reducedHubble (h : U.FriedmannSolution F Λ G c)
    (hρ : ∀ s, F.ρ s = Fm.ρ s + Fr.ρ s) {t t₀ : Time} (ht : t ∈ U.I) (ht₀ : t₀ ∈ U.I)
    (hH₀ : hubbleConstant U.a t₀ ≠ 0)
    (hm : Fm.ρ t = Fm.ρ t₀ * (U.a t₀ / U.a t) ^ 3) (hr : Fr.ρ t = Fr.ρ t₀ * (U.a t₀ / U.a t) ^
    4) :
    hubbleConstant U.a t ^ 2 = hubbleConstant U.a t₀ ^ 2
      * reducedHubble (U.lambdaDensityParameter Λ c t₀) (U.densityParameter Fm G t₀)
        (U.densityParameter Fr G t₀) (U.curvatureDensityParameter c t₀) (U.a t / U.a t₀) ^ 2
```

lemma hubbleConstant_eq_reducedHubble [source]

`H = H₀ E(a / a₀)` for `0 < H₀` and `0 ≤ H`, under the hypotheses of `sq_hubbleConstant_eq_reducedHubble`.

```
lemma hubbleConstant_eq_reducedHubble (h : U.FriedmannSolution F Λ G c)
    (hρ : ∀ s, F.ρ s = Fm.ρ s + Fr.ρ s) {t t₀ : Time} (ht : t ∈ U.I) (ht₀ : t₀ ∈ U.I)
    (hH₀ : 0 < hubbleConstant U.a t₀) (hH : 0 ≤ hubbleConstant U.a t)
    (hm : Fm.ρ t = Fm.ρ t₀ * (U.a t₀ / U.a t) ^ 3) (hr : Fr.ρ t = Fr.ρ t₀ * (U.a t₀ / U.a t) ^
    4) :
```

```
    hubbleConstant U.a t = hubbleConstant U.a t₀
      * reducedHubble (U.lambdaDensityParameter Λ c t₀) (U.densityParameter Fm G t₀)
        (U.densityParameter Fr G t₀) (U.curvatureDensityParameter c t₀) (U.a t / U.a t₀)
```

2.31 Physlib.Cosmology.FLRW.Solutions

Physlib/Cosmology/FLRW/Solutions.lean on GitHub.

def milne [source]

The Milne model, defined on the interval `I = {t | 0 < t.val}` after the Big Bang at `t.val = 0`: the FLRW model with the scale factor `milneScaleFactor c`, `c > 0`, and the saddle geometry `Saddle 1` of curvature parameter `-1`.

```
noncomputable def _root_.Cosmology.FLRW.milne (c : ℝ) (hc : 0 < c) : FLRW where
```

lemma friedmannSolution_milne [source]

The Milne model filled with the empty cosmic fluid solves the Friedmann equations with `Λ = 0` on its interval `t > 0`.

```
lemma _root_.Cosmology.FLRW.friedmannSolution_milne (G : ℝ) {c : ℝ} (hc : 0 < c) :
    (FLRW.milne c hc).FriedmannSolution CosmicFluid.empty 0 G c
```

2.32 Physlib.Cosmology.FLRW.SpecificCases.EinsteinDeSitter

Physlib/Cosmology/FLRW/SpecificCases/EinsteinDeSitter.lean on GitHub.

def einsteinDeSitter [source]

The Einstein-de Sitter model, the flat power-law model with `n = 2/3`.

```
noncomputable def FLRW.einsteinDeSitter (t₀ : Time) (ht₀ : 0 < t₀.val) : FLRW
```

lemma einsteinDeSitter_a [source]

The scale factor of the Einstein-de Sitter model is `einsteinDeSitterScaleFactor t₀`.

```
lemma FLRW.einsteinDeSitter_a {t₀ : Time} (ht₀ : 0 < t₀.val) :
    (FLRW.einsteinDeSitter t₀ ht₀).a = einsteinDeSitterScaleFactor t₀
```

lemma friedmannSolution_einsteinDeSitter [source]

The Einstein-de Sitter model, filled with dust of density `3 n² / (8 π G t²)` with `n = 2/3`, that is `1 / (6 π G t²)`, and zero pressure, solves the Friedmann equations without cosmological constant on its interval.

```
lemma FLRW.friedmannSolution_einsteinDeSitter {t₀ : Time} (ht₀ : 0 < t₀.val)
    {G c : ℝ} (hG : 0 < G) (hc : 0 < c) :
    (FLRW.einsteinDeSitter t₀ ht₀).FriedmannSolution
      (CosmicFluid.powerLaw (2 / 3) 0 G c) 0 G c
```

2.33 Physlib.Cosmology.FLRW.SpecificCases.PowerLaw

Physlib/Cosmology/FLRW/SpecificCases/PowerLaw.lean on GitHub.

def FLRW.powerLaw [source]

The spatially flat power-law model `a(t) = (t / t₀) ^ n` on the interval `t > 0`, after the origin `t.val = 0` of the time chart (for `n > 0`, the Big Bang), for a normalisation epoch `t₀ > 0`.

```
noncomputable def FLRW.powerLaw (t₀ : Time) (ht₀ : 0 < t₀.val) (n : ℝ) : FLRW where
```

def CosmicFluid.powerLaw [source]

The cosmic fluid of the flat power-law solution: the density `ρ = 3 n² / (8 π G t²)` imposed by the first-order Friedmann equation, and the pressure `p = w ρ c²`.

```
noncomputable def CosmicFluid.powerLaw (n w G c : ℝ) : CosmicFluid
```

lemma friedmannSolution_powerLaw [source]

The flat power-law model with `n ≠ 0`, filled with the fluid `CosmicFluid.powerLaw n w G c` where `1 + 3 w = 2 (1 - n) / n`, solves the Friedmann equations without cosmological constant on its interval `t > 0`.

```
lemma FLRW.friedmannSolution_powerLaw {t₀ : Time} (ht₀ : 0 < t₀.val)
    {n w G c : ℝ} (hG : 0 < G) (hc : 0 < c) (hn : n ≠ 0) (hw : 1 + 3 * w = 2 * (1 - n) / n) :
    (FLRW.powerLaw t₀ ht₀ n).FriedmannSolution (CosmicFluid.powerLaw n w G c) 0 G c
```

2.34 Physlib.Cosmology.FLRW.SpecificCases.RadiationDominated

Physlib/Cosmology/FLRW/SpecificCases/RadiationDominated.lean on GitHub.

def radiationDominated [source]

The radiation-dominated model, the flat power-law model with `n = 1/2`.

```
noncomputable def FLRW.radiationDominated (t₀ : Time) (ht₀ : 0 < t₀.val) : FLRW
```

lemma radiationDominated_a [source]

The scale factor of the radiation-dominated model is `radiationScaleFactor t₀`.

```
lemma FLRW.radiationDominated_a {t₀ : Time} (ht₀ : 0 < t₀.val) :
    (FLRW.radiationDominated t₀ ht₀).a = radiationScaleFactor t₀
```

lemma friedmannSolution_radiationDominated [source]

The radiation-dominated model, filled with radiation of density `3 n² / (8 π G t²)` with `n = 1/2`, that is `3 / (32 π G t²)`, and pressure `ρ c² / 3`, solves the Friedmann equations without cosmological constant on its interval.

```
lemma FLRW.friedmannSolution_radiationDominated {t₀ : Time} (ht₀ : 0 < t₀.val)
    {G c : ℝ} (hG : 0 < G) (hc : 0 < c) :
    (FLRW.radiationDominated t₀ ht₀).FriedmannSolution
```

```
      (CosmicFluid.powerLaw (1 / 2) (1 / 3) G c) 0 G c
```

2.35 Physlib.Electromagnetism.Current.InfiniteWire

Physlib/Electromagnetism/Current/InfiniteWire.lean on GitHub.

def wireCurrentDensity [source]

The current density associated with an infinite wire carrying a current `I` along the `x`-axis.

```
noncomputable def wireCurrentDensity (c : SpeedOfLight) :
    ℝ →ₗ[ℝ] TempDistLorentzCurrentDensity 3 where
```

lemma wireCurrentDensity_chargeDesnity [source]

```
@[simp]
lemma wireCurrentDensity_chargeDesnity (c : SpeedOfLight) (I : ℝ) :
    (wireCurrentDensity c I).chargeDensity c = 0
```

lemma wireCurrentDensity_currentDensity_fst [source]

```
lemma wireCurrentDensity_currentDensity_fst (c : SpeedOfLight) (I : ℝ)
    (η : 𝓢(Time × Space 3, ℝ)) :
    (wireCurrentDensity c I).currentDensity c η 0 =
    (constantTime <|
    constantSliceDist 0 <|
    I • diracDelta ℝ 0) η
```

lemma wireCurrentDensity_currentDensity_snd [source]

```
@[simp]
lemma wireCurrentDensity_currentDensity_snd (c : SpeedOfLight) (I : ℝ)
    (ε : 𝓢(Time × Space 3, ℝ)) :
    (wireCurrentDensity c I).currentDensity c ε 1 = 0
```

lemma wireCurrentDensity_currentDensity_thrd [source]

```
@[simp]
lemma wireCurrentDensity_currentDensity_thrd (c : SpeedOfLight) (I : ℝ)
    (ε : 𝓢(Time × Space 3, ℝ)) :
    (wireCurrentDensity c I).currentDensity c ε 2 = 0
```

def infiniteWire [source]

The electromagnetic potential of an infinite wire along the x-axis carrying a current `I`.

```
noncomputable def infiniteWire (𝓕 : FreeSpace) (I : ℝ) :
    TempDistElectromagneticPotential 3
```

lemma infiniteWire_scalarPotential [source]

```
@[simp]
lemma infiniteWire_scalarPotential (𝓕 : FreeSpace) (I : ℝ) :
    (infiniteWire 𝓕 I).scalarPotential 𝓕.c = 0
```

lemma infiniteWire_vectorPotential [source]

```
lemma infiniteWire_vectorPotential (𝓕 : FreeSpace) (I : ℝ) :
    (infiniteWire 𝓕 I).vectorPotential 𝓕.c =
    (constantTime <|
    constantSliceDist 0
    ((- I * 𝓕.μ₀ / (2 * Real.pi)) • distOfFunction (fun (x : Space 2) =>
      Real.log ‖x‖ • EuclideanSpace.single 0 (1 : ℝ))
    (by apply (IsDistBounded.log_norm (by norm_num)).smul_const)))
```

lemma infiniteWire_vectorPotential_fst [source]

```
lemma infiniteWire_vectorPotential_fst (𝓕 : FreeSpace) (I : ℝ)(η : 𝓢(Time × Space 3, ℝ)) :
    (infiniteWire 𝓕 I).vectorPotential 𝓕.c η 0 =
    (constantTime <|
    constantSliceDist 0 <|
    (- I * 𝓕.μ₀ / (2 * Real.pi)) • distOfFunction (fun (x : Space 2) => Real.log ‖x‖)
    (IsDistBounded.log_norm)) η
```

lemma infiniteWire_vectorPotential_snd [source]

```
@[simp]
lemma infiniteWire_vectorPotential_snd (𝓕 : FreeSpace) (I : ℝ) :
    (infiniteWire 𝓕 I).vectorPotential 𝓕.c η 1 = 0
```

lemma infiniteWire_vectorPotential_thrd [source]

```
@[simp]
lemma infiniteWire_vectorPotential_thrd (𝓕 : FreeSpace) (I : ℝ) :
    (infiniteWire 𝓕 I).vectorPotential 𝓕.c η 2 = 0
```

lemma infiniteWire_vectorPotential_distTimeDeriv [source]

```
@[simp]
lemma infiniteWire_vectorPotential_distTimeDeriv (𝓕 : FreeSpace) (I : ℝ) :
    distTimeDeriv ((infiniteWire 𝓕 I).vectorPotential 𝓕.c) = 0
```

lemma infiniteWire_vectorPotential_distSpaceDeriv_0 [source]

```
@[simp]
lemma infiniteWire_vectorPotential_distSpaceDeriv_0 (𝓕 : FreeSpace) (I : ℝ) :
    distSpaceDeriv 0 ((infiniteWire 𝓕 I).vectorPotential 𝓕.c) = 0
```

lemma infiniteWire_electricField [source]

```
@[simp]
lemma infiniteWire_electricField (𝓕 : FreeSpace) (I : ℝ) :
```

```
    (infiniteWire 𝓕 I).electricField 𝓕.c = 0
```

lemma infiniteWire_isExterma [source]

```
lemma infiniteWire_isExterma {𝓕 : FreeSpace} {I : ℝ} :
    IsExtrema 𝓕 (infiniteWire 𝓕 I) (wireCurrentDensity 𝓕.c I)
```

2.36 Physlib.Electromagnetism.Distributional.Basic

Physlib/Electromagnetism/Distributional/Basic.lean on GitHub.

def valEquiv [source]

The equivalence between an electromagnetic potential and its underlying distribution.

```
def valEquiv {d : ℕ} :
    DistElectromagneticPotential d ≃
      𝓓'(⟨Set.univ (α := SpaceTime d), isOpen_univ⟩, Lorentz.Vector d) where
```

instance AddCommGroup (DistElectromagneticPotential d) [source]

```
noncomputable instance {d : ℕ} : AddCommGroup (DistElectromagneticPotential d)
```

instance Module ℝ (DistElectromagneticPotential d) [source]

```
noncomputable instance {d : ℕ} : Module ℝ (DistElectromagneticPotential d)
```

def valLinearEquiv [source]

The linear equivalence between an electromagnetic potential and its underlying distribution.

```
noncomputable def valLinearEquiv {d : ℕ} :
    DistElectromagneticPotential d ≃ₗ[ℝ]
      𝓓'(⟨Set.univ (α := SpaceTime d), isOpen_univ⟩, Lorentz.Vector d)
```

def ofFunction [source]

The distribution induced by a spacetime function.

```
noncomputable def ofFunction {d} (f : SpaceTime d → Lorentz.Vector d) :
    DistElectromagneticPotential d where
```

2.37 Physlib.Electromagnetism.Kinematics.ElectricField

Physlib/Electromagnetism/Kinematics/ElectricField.lean on GitHub.

lemma ofScalarPotential_electricField [source]

A scalar-only potential produces the negative spatial gradient as its electric field.

```
@[simp]
lemma ofScalarPotential_electricField {d} (c : SpeedOfLight)
    (φ : Time → Space d → ℝ) :
    (ofScalarPotential c φ).electricField c = fun t x => - ∇ (φ t) x
```

2.38 Physlib.Electromagnetism.Kinematics.MagneticField

Physlib/Electromagnetism/Kinematics/MagneticField.lean on GitHub.

lemma ofScalarPotential_magneticField [source]

A scalar-only potential produces no magnetic field.

```
@[simp]
lemma ofScalarPotential_magneticField (c : SpeedOfLight)
    (φ : Time → Space 3 → ℝ) :
    (ofScalarPotential c φ).magneticField c = 0
```

2.39 Physlib.Electromagnetism.Kinematics.VectorPotential

Physlib/Electromagnetism/Kinematics/VectorPotential.lean on GitHub.

lemma contDiff_vectorPotential_ofElectromagneticField_space [source]

At a fixed time, the vector potential of `ofElectromagneticField` is `C^n` in space if the magnetic field is `C^n` in space at that time.

```
@[fun_prop]
lemma contDiff_vectorPotential_ofElectromagneticField_space {n : ℕ} (c : SpeedOfLight)
    (E : Time → Space 3 → EuclideanSpace ℝ (Fin 3))
    (B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) (t : Time) (hB : ContDiff ℝ n (B t)) :
    ContDiff ℝ n ((ofElectromagneticField c E B).vectorPotential c t)
```

2.40 Physlib.Electromagnetism.PointParticle.OneDimension

Physlib/Electromagnetism/PointParticle/OneDimension.lean on GitHub.

def oneDimPointParticleCurrentDensity [source]

The current density of a point particle stationary at the origin of 1d space.

```
noncomputable def oneDimPointParticleCurrentDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space 1) :
    TempDistLorentzCurrentDensity 1
```

lemma oneDimPointParticleCurrentDensity_eq_distTranslate [source]

```
lemma oneDimPointParticleCurrentDensity_eq_distTranslate (c : SpeedOfLight) (q : ℝ) (r₀ : Space
     1) :
    oneDimPointParticleCurrentDensity c q r₀ = ((SpaceTime.distTimeSlice c).symm <|
    constantTime <|
    distTranslate (basis.repr r₀) <|
    ((c * q) • diracDelta' ℝ 0 (Lorentz.Vector.basis (Sum.inl 0))))
```

lemma oneDimPointParticleCurrentDensity_currentDensity [source]

```
@[simp]
lemma oneDimPointParticleCurrentDensity_currentDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space
    1) :
    (oneDimPointParticleCurrentDensity c q r₀).currentDensity c = 0
```

lemma oneDimPointParticleCurrentDensity_chargeDensity [source]

```
@[simp]
lemma oneDimPointParticleCurrentDensity_chargeDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space 1)
     :
    (oneDimPointParticleCurrentDensity c q r₀).chargeDensity c =
    constantTime (q • diracDelta ℝ r₀)
```

def oneDimPointParticle [source]

The electromagnetic potential of a point particle stationary at r_0 of 1d space.

```
noncomputable def oneDimPointParticle (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 1) :
    TempDistElectromagneticPotential 1
```

lemma oneDimPointParticle_eq_distTranslate [source]

```
lemma oneDimPointParticle_eq_distTranslate (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 1) :
    oneDimPointParticle 𝓕 q r₀ = ((SpaceTime.distTimeSlice 𝓕.c).symm <|
    constantTime <|
    distTranslate (basis.repr r₀) <|
    distOfFunction (fun x => ((- (q * 𝓕.μ₀ * 𝓕.c)/ 2) * ‖x‖) • Lorentz.Vector.basis (Sum.inl
    0))
      (by fun_prop))
```

lemma oneDimPointParticle_vectorPotential [source]

```
@[simp]
lemma oneDimPointParticle_vectorPotential (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 1) :
    (oneDimPointParticle 𝓕 q r₀).vectorPotential 𝓕.c = 0
```

lemma oneDimPointParticle_scalarPotential [source]

```
lemma oneDimPointParticle_scalarPotential (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 1) :
    (oneDimPointParticle 𝓕 q r₀).scalarPotential 𝓕.c =
    Space.constantTime (distOfFunction (fun x =>
      - ((q * 𝓕.μ₀ * 𝓕.c ^ 2)/(2)) • ‖x-r₀‖) (by fun_prop))
```

lemma oneDimPointParticle_electricField [source]

```
lemma oneDimPointParticle_electricField (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 1) :
    (oneDimPointParticle 𝓕 q r₀).electricField 𝓕.c =
    ((q * 𝓕.μ₀ * 𝓕.c ^ 2) / 2) • constantTime (distOfFunction (fun x : Space 1 =>
      ‖x - r₀‖ ^ (- 1 : ℤ) • basis.repr (x - r₀))
      ((IsDistBounded.zpow_smul_repr_self (- 1 : ℤ) (by omega)).comp_sub_right r₀))
```

lemma oneDimPointParticle_electricField_timeDeriv [source]

```
@[simp]
lemma oneDimPointParticle_electricField_timeDeriv (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 1) :
    Space.distTimeDeriv ((oneDimPointParticle 𝓕 q r₀).electricField 𝓕.c) = 0
```

lemma oneDimPointParticle_magneticFieldMatrix [source]

```
lemma oneDimPointParticle_magneticFieldMatrix (q : ℝ) (r₀ : Space 1) :
    (oneDimPointParticle 𝓕 q r₀).magneticFieldMatrix 𝓕.c = 0
```

lemma oneDimPointParticle_div_electricField [source]

```
lemma oneDimPointParticle_div_electricField {𝓕} (q : ℝ) (r₀ : Space 1) :
    distSpaceDiv ((oneDimPointParticle 𝓕 q r₀).electricField 𝓕.c) =
    (𝓕.μ₀ * 𝓕.c ^ 2) • constantTime (q • diracDelta ℝ r₀)
```

lemma oneDimPointParticle_isExterma [source]

```
lemma oneDimPointParticle_isExterma (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 1) :
    (oneDimPointParticle 𝓕 q r₀).IsExtrema 𝓕 (oneDimPointParticleCurrentDensity 𝓕.c q r₀)
```

2.41 Physlib.Electromagnetism.PointParticle.ThreeDimension

Physlib/Electromagnetism/PointParticle/ThreeDimension.lean on GitHub.

def threeDimPointParticleCurrentDensity [source]

The current density of a point particle stationary at a point `r₀` of 3d space.

```
noncomputable def threeDimPointParticleCurrentDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space 3)
     :
    TempDistLorentzCurrentDensity 3
```

lemma threeDimPointParticleCurrentDensity_eq_distTranslate [source]

```
lemma threeDimPointParticleCurrentDensity_eq_distTranslate (c : SpeedOfLight) (q : ℝ)
    (r₀ : Space 3) :
    threeDimPointParticleCurrentDensity c q r₀ = ((SpaceTime.distTimeSlice c).symm <|
    constantTime <|
    distTranslate (basis.repr r₀) <|
    ((c * q) • diracDelta' ℝ 0 (Lorentz.Vector.basis (Sum.inl 0))))
```

lemma threeDimPointParticleCurrentDensity_chargeDensity [source]

```
@[simp]
lemma threeDimPointParticleCurrentDensity_chargeDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space
    3) :
    (threeDimPointParticleCurrentDensity c q r₀).chargeDensity c =
    constantTime (q • diracDelta ℝ r₀)
```

lemma threeDimPointParticleCurrentDensity_currentDensity [source]

```
@[simp]
lemma threeDimPointParticleCurrentDensity_currentDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space
     3) :
    (threeDimPointParticleCurrentDensity c q r₀).currentDensity c = 0
```

def threeDimPointParticle [source]

The electromagnetic potential of a point particle stationary at r_0 of 3d space.

```
noncomputable def threeDimPointParticle (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    TempDistElectromagneticPotential 3
```

lemma threeDimPointParticle_eq_distTranslate [source]

```
lemma threeDimPointParticle_eq_distTranslate (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    threeDimPointParticle 𝓕 q r₀ = ((SpaceTime.distTimeSlice 𝓕.c).symm <|
    constantTime <|
    distTranslate (basis.repr r₀) <|
    distOfFunction (fun x => (((q * 𝓕.μ₀ * 𝓕.c)/ (4 * π))* ‖x‖⁻¹) •
      Lorentz.Vector.basis (Sum.inl 0))
      (((IsDistBounded.inv (by norm_num)).const_mul_fun _).smul_const _))
```

lemma threeDimPointParticle_scalarPotential [source]

```
lemma threeDimPointParticle_scalarPotential (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    (threeDimPointParticle 𝓕 q r₀).scalarPotential 𝓕.c =
    Space.constantTime (distOfFunction (fun x => (q/ (4 * π * 𝓕.ε₀))• ‖x - r₀‖⁻¹)
      (((IsDistBounded.inv_shift _).const_mul_fun _)))
```

lemma threeDimPointParticle_vectorPotential [source]

```
@[simp]
lemma threeDimPointParticle_vectorPotential (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    (threeDimPointParticle 𝓕 q r₀).vectorPotential 𝓕.c = 0
```

lemma threeDimPointParticle_electricField [source]

```
lemma threeDimPointParticle_electricField (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    (threeDimPointParticle 𝓕 q r₀).electricField 𝓕.c =
    (q/ (4 * π * 𝓕.ε₀)) • constantTime (distOfFunction (fun x : Space 3 =>
      ‖x - r₀‖ ^ (- 3 : ℤ) • basis.repr (x - r₀))
      ((IsDistBounded.zpow_smul_repr_self (- 3 : ℤ) (by omega)).comp_sub_right r₀))
```

lemma threeDimPointParticle_electricField_timeDeriv [source]

```
@[simp]
lemma threeDimPointParticle_electricField_timeDeriv (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    Space.distTimeDeriv ((threeDimPointParticle 𝓕 q r₀).electricField 𝓕.c) = 0
```

lemma threeDimPointParticle_magneticFieldMatrix [source]

```
@[simp]
lemma threeDimPointParticle_magneticFieldMatrix (q : ℝ) (r₀ : Space 3) :
    (threeDimPointParticle 𝓕 q r₀).magneticFieldMatrix 𝓕.c = 0
```

lemma threeDimPointParticle_div_electricField [source]

```
lemma threeDimPointParticle_div_electricField {𝓕} (q : ℝ) (r₀ : Space 3) :
    distSpaceDiv ((threeDimPointParticle 𝓕 q r₀).electricField 𝓕.c) =
    (1/𝓕.ε₀) • constantTime (q • diracDelta ℝ r₀)
```

lemma threeDimPointParticle_isExterma [source]

```
lemma threeDimPointParticle_isExterma (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    (threeDimPointParticle 𝓕 q r₀).IsExtrema 𝓕 (threeDimPointParticleCurrentDensity 𝓕.c q r₀)
```

2.42 Physlib.Electromagnetism.TemperedDistributional.Basic

Physlib/Electromagnetism/TemperedDistributional/Basic.lean on GitHub.

abbrev TempDistElectromagneticPotential [source]

The electromagnetic potential as a distribution and as a tensor `A^μ`.

```
noncomputable abbrev TempDistElectromagneticPotential (d : ℕ := 3)
```

def ofScalarPotential [source]

The creation of an electromagnetic potential from a scalar potential.

```
noncomputable def ofScalarPotential {d} (c : SpeedOfLight) :
    ((Time × Space d) →d[ℝ] ℝ) →ₗ[ℝ] TempDistElectromagneticPotential d where
```

def ofStaticScalarPotential [source]

The creation of an electromagnetic potential from a static scalar potential.

```
noncomputable def ofStaticScalarPotential {d} (c : SpeedOfLight) :
    ((Space d) →d[ℝ] ℝ) →ₗ[ℝ] TempDistElectromagneticPotential d
```

def ofStaticScalarPotentialFunction [source]

The creation of an electromagnetic potential from a static scalar-potential function.

If the function is distribution-bounded, it is first promoted to a distribution using `Space.distOfFunction`. Otherwise the constructor returns zero.

```
noncomputable def ofStaticScalarPotentialFunction {d} (c : SpeedOfLight)
    (φ : Space d → ℝ) : TempDistElectromagneticPotential d
```

lemma ofStaticScalarPotentialFunction_of_isDistBounded [source]

```
@[simp]
lemma ofStaticScalarPotentialFunction_of_isDistBounded {d} (c : SpeedOfLight)
    (φ : Space d → ℝ) (hφ : Space.IsDistBounded φ) :
    ofStaticScalarPotentialFunction c φ =
      ofStaticScalarPotential c (Space.distOfFunction φ hφ)
```

lemma ofStaticScalarPotentialFunction_eq_zero_of_not_isDistBounded [source]

```
@[simp]
lemma ofStaticScalarPotentialFunction_eq_zero_of_not_isDistBounded {d}
    (c : SpeedOfLight) (φ : Space d → ℝ) (hφ : ¬ Space.IsDistBounded φ) :
    ofStaticScalarPotentialFunction c φ = 0
```

def ofVectorPotential [source]

The creation of an electromagnetic potential from a vector potential.

```
noncomputable def ofVectorPotential {d} (c : SpeedOfLight) :
    ((Time × Space d) →d[ℝ] EuclideanSpace ℝ (Fin d)) →ₗ[ℝ]
    TempDistElectromagneticPotential d where
```

def ofStaticVectorPotential [source]

The creation of an electromagnetic potential from a static vector potential.

```
noncomputable def ofStaticVectorPotential {d} (c : SpeedOfLight) :
    ((Space d) →d[ℝ] EuclideanSpace ℝ (Fin d)) →ₗ[ℝ] TempDistElectromagneticPotential d
```

lemma distTensorDeriv_eq_sum_sum [source]

```
lemma distTensorDeriv_eq_sum_sum {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    distTensorDeriv A ε =∑ μ, ∑ ν, (SpaceTime.distDeriv μ A ε ν) •
      Lorentz.CoVector.basis μ ⊗ₜ[ℝ] Lorentz.Vector.basis ν
```

lemma distTensorDeriv_basis_repr_apply [source]

```
@[simp]
lemma distTensorDeriv_basis_repr_apply {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d)}
    (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    (Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr (distTensorDeriv A ε) μν =
    distDeriv μν.1 A ε μν.2
```

lemma toTensor_distTensorDeriv_basis_repr_apply [source]

```
lemma toTensor_distTensorDeriv_basis_repr_apply {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) (b : ComponentIdx (S := realLorentzTensor d)
      (Fin.append ![Color.down] ![Color.up])) :
    (Tensor.basis _).repr (Tensorial.toTensor (distTensorDeriv A ε)) b =
    distDeriv (b 0) A ε (b 1)
```

2.43 Physlib.Electromagnetism.TemperedDistributional.Dynamics.CurrentDensity

Physlib/Electromagnetism/TemperedDistributional/Dynamics/CurrentDensity.lean on GitHub.

abbrev TempDistLorentzCurrentDensity [source]

The Lorentz current density, also called four-current as a distribution.

```
abbrev TempDistLorentzCurrentDensity (d : ℕ := 3)
```

def chargeDensity [source]

The charge density underlying a Lorentz current density which is a distribution.

```
noncomputable def chargeDensity {d : ℕ} (c : SpeedOfLight) :
    (TempDistLorentzCurrentDensity d) →ₗ[ℝ] (Time × Space d) →d[ℝ] ℝ where
```

def currentDensity [source]

The underlying (non-Lorentz) current density associated with a distributive Lorentz current density.

```
noncomputable def currentDensity (c : SpeedOfLight) :
    TempDistLorentzCurrentDensity d →ₗ[ℝ] (Time × Space d) →d[ℝ] EuclideanSpace ℝ (Fin d) where
```

2.44 Physlib.Electromagnetism.TemperedDistributional.Dynamics.IsExtrema

Physlib/Electromagnetism/TemperedDistributional/Dynamics/IsExtrema.lean on GitHub.

def IsExtrema [source]

The proposition on an electromagnetic potential, corresponding to the statement that it is an extrema of the lagrangian.

```
def IsExtrema {d} (ℱ : FreeSpace)
    (A : TempDistElectromagneticPotential d)
    (J : TempDistLorentzCurrentDensity d) : Prop
```

lemma isExtrema_iff_gradLagrangian [source]

```
lemma isExtrema_iff_gradLagrangian {ℱ : FreeSpace}
    (A : TempDistElectromagneticPotential d)
    (J : TempDistLorentzCurrentDensity d) :
    IsExtrema ℱ A J ↔ A.gradLagrangian ℱ J = 0
```

lemma isExtrema_iff_components [source]

```
lemma isExtrema_iff_components {ℱ : FreeSpace}
    (A : TempDistElectromagneticPotential d)
    (J : TempDistLorentzCurrentDensity d) :
    IsExtrema ℱ A J ↔ (∀ ε, A.gradLagrangian ℱ J ε (Sum.inl 0) = 0)
    ∧ (∀ ε i, A.gradLagrangian ℱ J ε (Sum.inr i) = 0)
```

lemma isExtrema_iff_space_time [source]

```
lemma isExtrema_iff_space_time {𝓕 : FreeSpace}
    (A : TempDistElectromagneticPotential d)
    (J : TempDistLorentzCurrentDensity d) :
    IsExtrema 𝓕 A J ↔
      (∀ ε, distSpaceDiv (A.electricField 𝓕.c) ε = (1/𝓕.ε₀) * (J.chargeDensity 𝓕.c) ε) ∧
      (∀ ε i, 𝓕.μ₀ * 𝓕.ε₀ * (Space.distTimeDeriv (A.electricField 𝓕.c)) ε i -
      ∑ j, ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
        ((Space.distSpaceDeriv j (A.magneticFieldMatrix 𝓕.c)) ε) (j, i) +
      𝓕.μ₀ * J.currentDensity 𝓕.c ε i = 0)
```

lemma isExtrema_iff_vectorPotential [source]

```
lemma isExtrema_iff_vectorPotential {𝓕 : FreeSpace}
    (A : TempDistElectromagneticPotential d)
    (J : TempDistLorentzCurrentDensity d) :
    IsExtrema 𝓕 A J ↔
      (∀ ε, distSpaceDiv (A.electricField 𝓕.c) ε = (1/𝓕.ε₀) * (J.chargeDensity 𝓕.c) ε) ∧
      (∀ ε i, 𝓕.μ₀ * 𝓕.ε₀ * distTimeDeriv (A.electricField 𝓕.c) ε i -
      (∑ x, -(distSpaceDeriv x (distSpaceDeriv x (A.vectorPotential 𝓕.c)) ε i
        - distSpaceDeriv x (distSpaceDeriv i (A.vectorPotential 𝓕.c)) ε x)) +
      𝓕.μ₀ * J.currentDensity 𝓕.c ε i = 0)
```

lemma isExterma_iff_tensor [source]

```
lemma isExterma_iff_tensor {𝓕 : FreeSpace}
    (A : TempDistElectromagneticPotential d)
    (J : TempDistLorentzCurrentDensity d) :
    IsExtrema 𝓕 A J ↔ ∀ ε,
    {((1/ 𝓕.μ₀ : ℝ) • distTensorDeriv A.fieldStrength ε | κ κ ν') + - (J ε | ν')}^T = 0
```

lemma isExterma_equivariant [source]

```
lemma isExterma_equivariant {𝓕 : FreeSpace}
    (A : TempDistElectromagneticPotential d)
    (J : TempDistLorentzCurrentDensity d) (Λ : LorentzGroup d) :
    IsExtrema 𝓕 (Λ • A) (Λ • J) ↔ IsExtrema 𝓕 A J
```

2.45 Physlib.Electromagnetism.TemperedDistributional.Dynamics.KineticTerm

Physlib/Electromagnetism/TemperedDistributional/Dynamics/KineticTerm.lean on GitHub.

def gradKineticTerm [source]

The gradient of the kinetic term for an Electromagnetic potential which is a distribution.

```
noncomputable def gradKineticTerm {d} (𝓕 : FreeSpace) :
    TempDistElectromagneticPotential d →ₗ[ℝ] (SpaceTime d) →d[ℝ] Lorentz.Vector d where
```

lemma gradKineticTerm_eq_sum_sum [source]

```
lemma gradKineticTerm_eq_sum_sum {d} {𝓕 : FreeSpace}
    (A : TempDistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) :
```

```
    A.gradKineticTerm 𝓕 ε = ∑ ν, ∑ μ,
        (1 / (𝓕.μ₀) * (η μ μ * η ν ν * distDeriv μ (distDeriv μ A) ε ν -
        distDeriv μ (distDeriv ν A) ε μ)) • Lorentz.Vector.basis ν
```

lemma gradKineticTerm_eq_fieldStrength [source]

```
lemma gradKineticTerm_eq_fieldStrength {d} {𝓕 : FreeSpace} (A :
    TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    A.gradKineticTerm 𝓕 ε = ∑ ν, (1/𝓕.μ₀ * η ν ν) •
    (∑ μ, ((Vector.basis.tensorProduct Vector.basis).repr
      (distDeriv μ (A.fieldStrength) ε) (μ, ν))) • Lorentz.Vector.basis ν
```

lemma gradKineticTerm_sum_inl_eq [source]

```
lemma gradKineticTerm_sum_inl_eq {d} {𝓕 : FreeSpace}
    (A : TempDistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) :
    A.gradKineticTerm 𝓕 ε (Sum.inl 0) =
    (1/(𝓕.μ₀ * 𝓕.c) * (distTimeSlice 𝓕.c).symm (Space.distSpaceDiv (A.electricField 𝓕.c)) ε)
```

lemma gradKineticTerm_sum_inr_eq [source]

```
lemma gradKineticTerm_sum_inr_eq {d} {𝓕 : FreeSpace}
    (A : TempDistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) (i : Fin d) :
    A.gradKineticTerm 𝓕 ε (Sum.inr i) =
    (𝓕.μ₀⁻¹ * (1 / 𝓕.c ^ 2 * (distTimeSlice 𝓕.c).symm
      (Space.distTimeDeriv (A.electricField 𝓕.c)) ε i -
      ∑ j, ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
        ((distTimeSlice 𝓕.c).symm (Space.distSpaceDeriv j
          (A.magneticFieldMatrix 𝓕.c)) ε) (j, i)))
```

lemma gradKineticTerm_eq_distTensorDeriv [source]

```
lemma gradKineticTerm_eq_distTensorDeriv {d} {𝓕 : FreeSpace}
    (A : TempDistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) (ν : Fin 1 ⊕ Fin d) :
    A.gradKineticTerm 𝓕 ε ν = η ν ν * ((Tensorial.toTensor (M := Lorentz.Vector d)).symm
    (permT id (IsReindexing.auto) {(1/ 𝓕.μ₀ : ℝ) •
    distTensorDeriv A.fieldStrength ε | κ κ ν'}ᵀ)) ν
```

2.46 Physlib.Electromagnetism.TemperedDistributional.Dynamics.Lagrangian

Physlib/Electromagnetism/TemperedDistributional/Dynamics/Lagrangian.lean on GitHub.

def gradFreeCurrentPotential [source]

The variational gradient of the free current potential for distributional potentials.

```
noncomputable def gradFreeCurrentPotential {d} :
    TempDistLorentzCurrentDensity d →ₗ[ℝ] ((SpaceTime d) →d[ℝ] Lorentz.Vector d) where
```

lemma gradFreeCurrentPotential_eq_sum_basis [source]

```
lemma gradFreeCurrentPotential_eq_sum_basis {d}
    (J : TempDistLorentzCurrentDensity d) (ε : 𝓢(SpaceTime d, ℝ)) :
```

```
    (gradFreeCurrentPotential J) ε =
    (∑ μ, (η μ μ • (J ε μ) • Lorentz.Vector.basis μ))
```

lemma gradFreeCurrentPotential_sum_inl_0 [source]

```
lemma gradFreeCurrentPotential_sum_inl_0 (𝓕 : FreeSpace) {d}
    (J : TempDistLorentzCurrentDensity d) (ε : 𝓢(SpaceTime d, ℝ)) :
    (gradFreeCurrentPotential J) ε (Sum.inl 0) =
    𝓕.c * (distTimeSlice 𝓕.c).symm (J.chargeDensity 𝓕.c) ε
```

lemma gradFreeCurrentPotential_sum_inr_i [source]

```
lemma gradFreeCurrentPotential_sum_inr_i (𝓕 : FreeSpace) {d}
    (J : TempDistLorentzCurrentDensity d) (ε : 𝓢(SpaceTime d, ℝ)) (i : Fin d) :
    (gradFreeCurrentPotential J) ε (Sum.inr i) =
    - (distTimeSlice 𝓕.c).symm (J.currentDensity 𝓕.c) ε i
```

lemma gradFreeCurrentPotential_eq_tensor [source]

```
lemma gradFreeCurrentPotential_eq_tensor {d}
    (J : TempDistLorentzCurrentDensity d) (ε : 𝓢(SpaceTime d, ℝ))
    (ν : Fin 1 ⊕ Fin d) :
    gradFreeCurrentPotential J ε ν = η ν ν * ((Tensorial.toTensor (M := Lorentz.Vector d)).symm
    (permT id (IsReindexing.auto) {J ε | ν'}ᵀ)) ν
```

def gradLagrangian [source]

The variational gradient of lagrangian for an electromagnetic potential which is a distribution.

```
noncomputable def gradLagrangian {d} (𝓕 : FreeSpace) (A : TempDistElectromagneticPotential d)
    (J : TempDistLorentzCurrentDensity d) : ((SpaceTime d) →d[ℝ] Lorentz.Vector d)
```

lemma gradLagrangian_sum_inl_0 [source]

```
lemma gradLagrangian_sum_inl_0 {𝓕 : FreeSpace}
    (A : TempDistElectromagneticPotential d) (J : TempDistLorentzCurrentDensity d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    A.gradLagrangian 𝓕 J ε (Sum.inl 0) =
    (1/(𝓕.μ₀ * 𝓕.c) * (distTimeSlice 𝓕.c).symm (Space.distSpaceDiv (A.electricField 𝓕.c)) ε)
    - 𝓕.c * (distTimeSlice 𝓕.c).symm (J.chargeDensity 𝓕.c) ε
```

lemma gradLagrangian_sum_inr_i [source]

```
lemma gradLagrangian_sum_inr_i {𝓕 : FreeSpace}
    (A : TempDistElectromagneticPotential d) (J : TempDistLorentzCurrentDensity d)
    (ε : 𝓢(SpaceTime d, ℝ)) (i : Fin d) :
    A.gradLagrangian 𝓕 J ε (Sum.inr i) =
    𝓕.μ₀⁻¹ * (1 / 𝓕.c ^ 2 *
      (distTimeSlice 𝓕.c).symm (Space.distTimeDeriv (A.electricField 𝓕.c)) ε i -
      ∑ j, ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
        ((distTimeSlice 𝓕.c).symm (Space.distSpaceDeriv j (A.magneticFieldMatrix 𝓕.c)) ε) (j,
    i)) +
    (distTimeSlice 𝓕.c).symm (J.currentDensity 𝓕.c) ε i
```

lemma gradLagrangian_eq_tensor [source]

```
lemma gradLagrangian_eq_tensor {𝓕 : FreeSpace}
    (A : TempDistElectromagneticPotential d) (J : TempDistLorentzCurrentDensity d)
    (ε : 𝓢(SpaceTime d, ℝ)) (ν : Fin 1 ⊕ Fin d) :
    A.gradLagrangian 𝓕 J ε ν =
    η ν ν * ((Tensorial.toTensor (M := Lorentz.Vector d)).symm
    (permT id (IsReindexing.auto) {((1/ 𝓕.μ₀ : ℝ) • (distTensorDeriv A.fieldStrength ε) | κ κ
    ν') +
    - (J ε | ν')}ᵀ)) ν
```

2.47 Physlib.Electromagnetism.TemperedDistributional.ElectricField

Physlib/Electromagnetism/TemperedDistributional/ElectricField.lean on GitHub.

def electricField [source]

The electric field of an electromagnetic potential which is a distribution.

```
noncomputable def electricField {d} (c : SpeedOfLight) :
    TempDistElectromagneticPotential d →ₗ[ℝ]
    (Time × Space d) →d[ℝ] EuclideanSpace ℝ (Fin d) where
```

lemma electricField_eq_fieldStrength [source]

```
lemma electricField_eq_fieldStrength {d} {c : SpeedOfLight}
    (A : TempDistElectromagneticPotential d) (ε : 𝓢(Time × Space d, ℝ))
    (i : Fin d) : A.electricField c ε i = - c * (Vector.basis.tensorProduct Vector.basis).repr
      (distTimeSlice c (A.fieldStrength) ε) (Sum.inl 0, Sum.inr i)
```

2.48 Physlib.Electromagnetism.TemperedDistributional.FieldStrength

Physlib/Electromagnetism/TemperedDistributional/FieldStrength.lean on GitHub.

def fieldStrengthAux [source]

An auxiliary definition for the field strength of an electromagnetic potential based on a distribution. On Schwartz maps this has the same value as the field strength tensor, but no linearity or continuous properties built in.

```
noncomputable def fieldStrengthAux {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) : Lorentz.Vector d ⊗[ℝ] Lorentz.Vector d
```

lemma fieldStrengthAux_eq_add [source]

```
lemma fieldStrengthAux_eq_add {d} (A : TempDistElectromagneticPotential d) (ε : 𝓢(SpaceTime d,
    ℝ)) :
    fieldStrengthAux A ε =
    Tensorial.toTensor.symm
      (permT id (IsReindexing.auto) {(η d | μ μ' ⊗ distTensorDeriv A ε | μ' ν)}ᵀ)
    - Tensorial.toTensor.symm (permT ![1, 0] (IsReindexing.auto)
      {(η d | μ μ' ⊗ distTensorDeriv A ε | μ' ν)}ᵀ)
```

lemma toTensor_fieldStrengthAux [source]

```
lemma toTensor_fieldStrengthAux {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    Tensorial.toTensor (fieldStrengthAux A ε) =
    (permT id (IsReindexing.auto) {(η d | μ μ' ⊗ distTensorDeriv A ε | μ' ν)}^T)
    - (permT ![1, 0] (IsReindexing.auto)
      {(η d | μ μ' ⊗ distTensorDeriv A ε | μ' ν)}^T)
```

lemma toTensor_fieldStrengthAux_basis_repr [source]

```
lemma toTensor_fieldStrengthAux_basis_repr {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ))
    (b : ComponentIdx (S := realLorentzTensor d) (Fin.append ![Color.up] ![Color.up])) :
    (Tensor.basis _).repr (Tensorial.toTensor (fieldStrengthAux A ε)) b =
    ∑ κ, (η (b 0) κ * SpaceTime.distDeriv κ A ε (b 1) -
      η (b 1) κ * SpaceTime.distDeriv κ A ε (b 0))
```

lemma fieldStrengthAux_tensor_basis_eq_basis [source]

```
lemma fieldStrengthAux_tensor_basis_eq_basis {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ))
    (b : ComponentIdx (S := realLorentzTensor d) (Fin.append ![Color.up] ![Color.up])) :
    (Tensor.basis _).repr (Tensorial.toTensor (A.fieldStrengthAux ε)) b =
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.fieldStrengthAux ε)
      (b 0, b 1)
```

lemma fieldStrengthAux_basis_repr_apply [source]

```
lemma fieldStrengthAux_basis_repr_apply {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d)}
    (A : TempDistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) :
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.fieldStrengthAux ε) μν =
    ∑ κ, ((η μν.1 κ * distDeriv κ A ε μν.2) - η μν.2 κ * distDeriv κ A ε μν.1)
```

lemma fieldStrengthAux_basis_repr_apply_eq_single [source]

```
lemma fieldStrengthAux_basis_repr_apply_eq_single {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d)}
    (A : TempDistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) :
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.fieldStrengthAux ε) μν =
    ((η μν.1 μν.1 * distDeriv μν.1 A ε μν.2) - η μν.2 μν.2 * distDeriv μν.2 A ε μν.1)
```

lemma fieldStrengthAux_eq_basis [source]

```
lemma fieldStrengthAux_eq_basis {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    (A.fieldStrengthAux ε) = ∑ μ, ∑ ν,
      ((η μ μ * distDeriv μ A ε ν) - η ν ν * distDeriv ν A ε μ)
      • Lorentz.Vector.basis μ ⊗ₜ[ℝ] Lorentz.Vector.basis ν
```

def fieldStrength [source]

The field strength of an electromagnetic potential which is a distribution.

```
noncomputable def fieldStrength {d} :
    TempDistElectromagneticPotential d →ₗ[ℝ]
    (SpaceTime d) →d[ℝ] Lorentz.Vector d ⊗[ℝ] Lorentz.Vector d where
```

lemma fieldStrength_eq_fieldStrengthAux [source]

```
lemma fieldStrength_eq_fieldStrengthAux {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    A.fieldStrength ε = A.fieldStrengthAux ε
```

lemma fieldStrength_eq_basis [source]

```
lemma fieldStrength_eq_basis {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    A.fieldStrength ε = ∑ μ, ∑ ν,
      ((η μ μ * distDeriv μ A ε ν) - η ν ν * distDeriv ν A ε μ)
      • Lorentz.Vector.basis μ ⊗ₜ[ℝ] Lorentz.Vector.basis ν
```

lemma fieldStrength_basis_repr_eq_single [source]

```
lemma fieldStrength_basis_repr_eq_single {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d)}
    (A : TempDistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) :
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.fieldStrength ε) μν =
    ((η μν.1 μν.1 * distDeriv μν.1 A ε μν.2) - η μν.2 μν.2 * distDeriv μν.2 A ε μν.1)
```

lemma fieldStrength_diag_zero [source]

```
@[simp]
lemma fieldStrength_diag_zero {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) (μ : Fin 1 ⊕ Fin d) :
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr
    (A.fieldStrength ε) (μ, μ) = 0
```

lemma distDeriv_fieldStrength_diag_zero [source]

```
@[simp]
lemma distDeriv_fieldStrength_diag_zero {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) (μ ν : Fin 1 ⊕ Fin d) :
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr
    (distDeriv ν A.fieldStrength ε) (μ, μ) = 0
```

lemma fieldStrength_antisymmetric_basis [source]

```
lemma fieldStrength_antisymmetric_basis {d} (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) (μ ν : Fin 1 ⊕ Fin d) :
    (Vector.basis.tensorProduct Vector.basis).repr
    (A.fieldStrength ε) (μ, ν) = - (Vector.basis.tensorProduct Vector.basis).repr
    (A.fieldStrength ε) (ν, μ)
```

lemma fieldStrength_equivariant [source]

```
lemma fieldStrength_equivariant {d} (A : TempDistElectromagneticPotential d)
    (Λ : LorentzGroup d) :
    (Λ • A).fieldStrength = Λ • A.fieldStrength
```

2.49 Physlib.Electromagnetism.TemperedDistributional.MagneticField

Physlib/Electromagnetism/TemperedDistributional/MagneticField.lean on GitHub.

def magneticFieldMatrix [source]

The magnetic field matrix of an electromagnetic potential which is a distribution.

```
noncomputable def magneticFieldMatrix {d} (c : SpeedOfLight) :
    TempDistElectromagneticPotential d →ₗ[ℝ]
    (Time × Space d) →d[ℝ] (EuclideanSpace ℝ (Fin d) ⊗[ℝ] EuclideanSpace ℝ (Fin d)) where
```

lemma magneticFieldMatrix_eq_vectorPotential [source]

```
lemma magneticFieldMatrix_eq_vectorPotential {c : SpeedOfLight}
    (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(Time × Space d, ℝ)) :
    A.magneticFieldMatrix c ε = ∑ i, ∑ j,
    (Space.distSpaceDeriv j (A.vectorPotential c) ε i -
      Space.distSpaceDeriv i (A.vectorPotential c) ε j) •
    EuclideanSpace.basisFun (Fin d) ℝ i ⊗ₜ[ℝ] EuclideanSpace.basisFun (Fin d) ℝ j
```

lemma magneticFieldMatrix_basis_repr_eq_vector_potential [source]

```
lemma magneticFieldMatrix_basis_repr_eq_vector_potential {c : SpeedOfLight}
    (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(Time × Space d, ℝ)) (i j : Fin d) :
    ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
        (A.magneticFieldMatrix c ε) (i, j) =
      Space.distSpaceDeriv j (A.vectorPotential c) ε i -
      Space.distSpaceDeriv i (A.vectorPotential c) ε j
```

lemma magneticFieldMatrix_distSpaceDeriv_basis_repr_eq_vector_potential [source]

```
lemma magneticFieldMatrix_distSpaceDeriv_basis_repr_eq_vector_potential {c : SpeedOfLight}
    (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(Time × Space d, ℝ)) (i j k : Fin d) :
    ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
    (Space.distSpaceDeriv k (A.magneticFieldMatrix c) ε) (i, j) =
    Space.distSpaceDeriv k (Space.distSpaceDeriv j (A.vectorPotential c)) ε i -
    Space.distSpaceDeriv k (Space.distSpaceDeriv i (A.vectorPotential c)) ε j
```

lemma magneticFieldMatrix_basis_repr_eq_fieldStrength [source]

```
lemma magneticFieldMatrix_basis_repr_eq_fieldStrength {c : SpeedOfLight}
    (A : TempDistElectromagneticPotential d)
    (ε : 𝓢(Time × Space d, ℝ)) (i j : Fin d) :
    ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
        (A.magneticFieldMatrix c ε) (i, j) =
      (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr
```

```
        (distTimeSlice c A.fieldStrength ε) (Sum.inr i, Sum.inr j)
```

lemma magneticFieldMatrix_one_dim_eq_zero [source]

```
@[simp]
lemma magneticFieldMatrix_one_dim_eq_zero {c : SpeedOfLight}
    (A : TempDistElectromagneticPotential 1) :
    A.magneticFieldMatrix c = 0
```

2.50 Physlib.Electromagnetism.TemperedDistributional.ScalarPotential

Physlib/Electromagnetism/TemperedDistributional/ScalarPotential.lean on GitHub.

def scalarPotential [source]

The scalar potential of an electromagnetic potential which is a distribution.

```
noncomputable def scalarPotential {d} (c : SpeedOfLight) :
    TempDistElectromagneticPotential d →ₗ[ℝ]
    (Time × Space d) →d[ℝ] ℝ where
```

2.51 Physlib.Electromagnetism.TemperedDistributional.VectorPotential

Physlib/Electromagnetism/TemperedDistributional/VectorPotential.lean on GitHub.

def vectorPotential [source]

The vector potential of an electromagnetic potential which is a distribution.

```
noncomputable def vectorPotential {d} (c : SpeedOfLight) :
    TempDistElectromagneticPotential d →ₗ[ℝ]
    (Time × Space d) →d[ℝ] EuclideanSpace ℝ (Fin d) where
```

2.52 Physlib.Mathematics.Calculus.Gradient

Physlib/Mathematics/Calculus/Gradient.lean on GitHub.

lemma add [source]

The gradient of a sum is the sum of the gradients.

```
lemma HasGradientAt.add (hf : HasGradientAt f f' x) (hg : HasGradientAt g g' x) :
    HasGradientAt (fun y => f y + g y) (f' + g') x
```

lemma sub [source]

The gradient of a difference is the difference of the gradients.

```
lemma HasGradientAt.sub (hf : HasGradientAt f f' x) (hg : HasGradientAt g g' x) :
    HasGradientAt (fun y => f y - g y) (f' - g') x
```

lemma mul [source]

The gradient of a product is given by the Leibniz rule.

```
lemma HasGradientAt.mul (hf : HasGradientAt f f' x) (hg : HasGradientAt g g' x) :
    HasGradientAt (fun y => f y * g y) (f x • g' + g x • f') x
```

lemma const_mul [source]

The gradient of a constant multiple is the constant multiple of the gradient.

```
lemma HasGradientAt.const_mul (c : ℝ) (hf : HasGradientAt f f' x) :
    HasGradientAt (fun y => c * f y) (c • f') x
```

lemma gradient_add [source]

The gradient of a sum of differentiable functions is the sum of their gradients.

```
lemma gradient_add {f g : F → ℝ} {x : F} (hf : DifferentiableAt ℝ f x)
    (hg : DifferentiableAt ℝ g x) :
    gradient (fun y => f y + g y) x = gradient f x + gradient g x
```

lemma comp_hasGradientAt [source]

The gradient of $y \mapsto \varphi\ (f\ y)$ is φ' times the gradient of f, for φ' the derivative of φ at $f\ x$.

```
lemma HasDerivAt.comp_hasGradientAt {φ : ℝ → ℝ} {φ' : ℝ} (hf : HasGradientAt f f' x)
    (hφ : HasDerivAt φ φ' (f x)) : HasGradientAt (fun y => φ (f y)) (φ' • f') x
```

lemma hasGradientAt_inner_left [source]

The gradient of $y \mapsto \langle\langle a,\ y\rangle\rangle$ is a.

```
lemma hasGradientAt_inner_left (a x : F) : HasGradientAt (fun y : F => ⟪a, y⟫_ℝ) a x
```

lemma hasGradientAt_inner_right [source]

The gradient of $y \mapsto \langle\langle y,\ a\rangle\rangle$ is a.

```
lemma hasGradientAt_inner_right (a x : F) : HasGradientAt (fun y : F => ⟪y, a⟫_ℝ) a x
```

lemma hasGradientAt_inner_self [source]

The gradient of $y \mapsto \langle\langle y,\ y\rangle\rangle$ is $2 \bullet y$.

```
lemma hasGradientAt_inner_self (x : F) : HasGradientAt (fun y : F => ⟪y, y⟫_ℝ) ((2 : ℝ) • x) x
```

lemma hasGradientAt_norm [source]

Away from the origin, the gradient of the norm is the unit vector $\|x\|^{-1} \bullet x$.

```
lemma hasGradientAt_norm (hx : x ≠ 0) : HasGradientAt (fun y : F => ‖y‖) (‖x‖⁻¹ • x) x
```

lemma hasGradientAt_coord [source]

The gradient of the `i`-th coordinate functional on Euclidean space is the `i`-th basis vector.

```
lemma hasGradientAt_coord {ι : Type*} [Fintype ι] [DecidableEq ι] (i : ι)
    (x : EuclideanSpace ℝ ι) :
    HasGradientAt (fun y : EuclideanSpace ℝ ι => y i) (EuclideanSpace.single i 1) x
```

2.53 Physlib.Mathematics.Calculus.ParametricIntegration

Physlib/Mathematics/Calculus/ParametricIntegration.lean on GitHub.

lemma fderiv_apply_parametric_intervalIntegral [source]

```
lemma fderiv_apply_parametric_intervalIntegral
    {F : M → ℝ → N} (hf : ContDiff ℝ 1 ↿F) (x₀ : M) (v : M) :
    fderiv ℝ (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume)) x₀ v =
      ∫ (t : ℝ) in 0..1, fderiv ℝ (F · t) x₀ v ∂(volume)
```

lemma fderiv_parametric_intervalIntegral [source]

```
lemma fderiv_parametric_intervalIntegral
    {F : M → ℝ → N} (hf : ContDiff ℝ 1 ↿F) (x₀ : M) :
    fderiv ℝ (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume)) =
      fun x => ∫ (t : ℝ) in 0..1, fderiv ℝ (F · t) x ∂(volume)
```

lemma hasDerivAt_intervalIntegral_of_contDiff [source]

Differentiation under the integral sign for a `C¹` integrand.

```
lemma hasDerivAt_intervalIntegral_of_contDiff {F : ℝ × ℝ → N}
    (hF : ContDiff ℝ 1 F) (a b x₀ : ℝ) :
    HasDerivAt (fun x ↦ ∫ t in a..b, F (x, t))
      (∫ t in a..b, fderiv ℝ F (x₀, t) (1, 0)) x₀
```

2.54 Physlib.Mathematics.Calculus.PartialDerivatives

Physlib/Mathematics/Calculus/PartialDerivatives.lean on GitHub.

lemma fderiv_partial_snd [source]

The derivative in the middle argument of `f` as a restriction of the total derivative.

```
lemma fderiv_partial_snd {f : X × Y × Z → F} {a : X} {y : Y} {z : Z}
    (hf : DifferentiableAt ℝ f (a, y, z)) :
    fderiv ℝ (fun y' ↦ f (a, y', z)) y = (fderiv ℝ f (a, y, z)).comp
      ((0 : Y →L[ℝ] X).prod ((ContinuousLinearMap.id ℝ Y).prod (0 : Y →L[ℝ] Z)))
```

lemma fderiv_partial_thd [source]

The derivative in the last argument of `f` as a restriction of the total derivative.

```
lemma fderiv_partial_thd {f : X × Y × Z → F} {a : X} {y : Y} {z : Z}
    (hf : DifferentiableAt ℝ f (a, y, z)) :
```

```
    fderiv ℝ (fun z' ↦ f (a, y, z')) z = (fderiv ℝ f (a, y, z)).comp
      ((0 : Z →L[ℝ] X).prod ((0 : Z →L[ℝ] Y).prod (ContinuousLinearMap.id ℝ Z)))
```

lemma fderiv_apply_zero_prod [source]

The total derivative of `f` on `(0, u, w)` splits into its middle and last partials.

```
lemma fderiv_apply_zero_prod {f : X × Y × Z → F} {a : X} {y : Y} {z : Z}
    (hf : DifferentiableAt ℝ f (a, y, z)) (u : Y) (w : Z) :
    fderiv ℝ f (a, y, z) (0, u, w) =
      fderiv ℝ (fun y' ↦ f (a, y', z)) y u + fderiv ℝ (fun z' ↦ f (a, y, z')) z w
```

lemma deriv_fderiv_apply_comm [source]

Mixed partial derivatives of a two-parameter family, `C²` at the point, commute.

```
lemma deriv_fderiv_apply_comm {s τ : ℝ} (hH : ContDiffAt ℝ 2 H (s, τ)) :
    deriv (fun σ ↦ fderiv ℝ H (s, σ) (1, 0)) τ =
      fderiv ℝ (fun p ↦ fderiv ℝ H p (0, 1)) (s, τ) (1, 0)
```

2.55 Physlib.Mathematics.Calculus.Wirtinger.Basic

Physlib/Mathematics/Calculus/Wirtinger/Basic.lean on GitHub.

lemma clinear_of_dWirtingerAntiDir_eq_zero [source]

The converse of `dWirtingerAntiDir_eq_zero_of_clinear`. It needs no differentiability.

```
lemma clinear_of_dWirtingerAntiDir_eq_zero {v : V} (h : dWirtingerAntiDir f v u = 0) :
    fderiv ℝ f u (Complex.I • v) = Complex.I • fderiv ℝ f u v
```

theorem differentiableAt_complex_iff_dWirtingerAntiDir_eq_zero [source]

***Cauchy–Riemann.** `f` is `ℂ`-differentiable at `u` iff it is real-differentiable there and every anti-holomorphic Wirtinger derivative vanishes. Real differentiability is needed: `fderiv` is `0` where `f` is not differentiable.*

```
theorem differentiableAt_complex_iff_dWirtingerAntiDir_eq_zero (f : V → ℂ) (u : V) :
    DifferentiableAt ℂ f u ↔
      DifferentiableAt ℝ f u ∧ ∀ v, dWirtingerAntiDir f v u = 0
```

2.56 Physlib.Mathematics.Calculus.Wirtinger.Coordinate

Physlib/Mathematics/Calculus/Wirtinger/Coordinate.lean on GitHub.

theorem differentiableAt_complex_iff_dWirtingerAntiCoord_eq_zero [source]

***Cauchy–Riemann in coordinates**, the form `∂̄_I W = 0` used in physics. Coordinate directions suffice by `LinearMap.commutesI_of_basis`.*

```
theorem differentiableAt_complex_iff_dWirtingerAntiCoord_eq_zero
    (f : (ι → ℂ) → ℂ) (u : ι → ℂ) :
    DifferentiableAt ℂ f u ↔
      DifferentiableAt ℝ f u ∧ ∀ I, dWirtingerAntiCoord f I u = 0
```

lemma dWirtingerCoord_comp_clm_zero [source]

If the inner continuous $\mathbb{R}$-linear map `P` kills the coordinate column `c`, the holomorphic coordinate Wirtinger derivative of `h ∘ P` along `c` vanishes.

```
lemma dWirtingerCoord_comp_clm_zero {C_s : Type} [Fintype C_s]
    (P : (D → ℂ) →L[ℝ] (C_s → ℂ)) (h : (C_s → ℂ) → ℂ) (c : D) (u : D → ℂ)
    (hh : DifferentiableAt ℝ h (P u))
    (h1 : P (Pi.single c 1) = 0) (hI : P (Pi.single c Complex.I) = 0) :
    dWirtingerCoord (fun v => h (P v)) c u = 0
```

lemma dWirtingerCoord_comp_clm_col [source]

If the inner continuous $\mathbb{R}$-linear map `P` reads the coordinate column `c` as the sector coordinate `a₀`, the holomorphic coordinate Wirtinger derivative of `h ∘ P` along `c` is that of `h` along `a₀`, evaluated at `P u`.

```
lemma dWirtingerCoord_comp_clm_col {C_s : Type} [Fintype C_s] [DecidableEq C_s]
    (P : (D → ℂ) →L[ℝ] (C_s → ℂ)) (h : (C_s → ℂ) → ℂ) (c : D) (a₀ : C_s)
    (u : D → ℂ) (hh : DifferentiableAt ℝ h (P u))
    (h1 : P (Pi.single c 1) = Pi.single a₀ 1)
    (hI : P (Pi.single c Complex.I) = Pi.single a₀ Complex.I) :
    dWirtingerCoord (fun v => h (P v)) c u = dWirtingerCoord h a₀ (P u)
```

lemma dWirtingerAntiCoord_comp_clm_zero [source]

Anti-holomorphic mirror of `dWirtingerCoord_comp_clm_zero`.

```
lemma dWirtingerAntiCoord_comp_clm_zero {C_s : Type} [Fintype C_s]
    (P : (D → ℂ) →L[ℝ] (C_s → ℂ)) (h : (C_s → ℂ) → ℂ) (c : D) (u : D → ℂ)
    (hh : DifferentiableAt ℝ h (P u))
    (h1 : P (Pi.single c 1) = 0) (hI : P (Pi.single c Complex.I) = 0) :
    dWirtingerAntiCoord (fun v => h (P v)) c u = 0
```

lemma dWirtingerAntiCoord_comp_clm_col [source]

Anti-holomorphic mirror of `dWirtingerCoord_comp_clm_col`.

```
lemma dWirtingerAntiCoord_comp_clm_col {C_s : Type} [Fintype C_s] [DecidableEq C_s]
    (P : (D → ℂ) →L[ℝ] (C_s → ℂ)) (h : (C_s → ℂ) → ℂ) (c : D) (a₀ : C_s)
    (u : D → ℂ) (hh : DifferentiableAt ℝ h (P u))
    (h1 : P (Pi.single c 1) = Pi.single a₀ 1)
    (hI : P (Pi.single c Complex.I) = Pi.single a₀ Complex.I) :
    dWirtingerAntiCoord (fun v => h (P v)) c u = dWirtingerAntiCoord h a₀ (P u)
```

2.57 Physlib.Mathematics.Calculus.Wirtinger.Hessian

Physlib/Mathematics/Calculus/Wirtinger/Hessian.lean on GitHub.

def hessianMatrixOf [source]

The mixed Wirtinger Hessian of `f` at `u`: the matrix with `(I, J)` entry `∂_I ∂⁻_J f`.

```
noncomputable def hessianMatrixOf (f : (ι → ℂ) → ℂ) (u : ι → ℂ) : Matrix ι ι ℂ
```

abbrev hessianMatrixOfR [source]

The mixed Wirtinger Hessian of a real-valued `K`, through `Complex.ofReal`. An `abbrev`, so lemmas about `hessianMatrixOf` apply to it.

```
noncomputable abbrev hessianMatrixOfR (K : (ι → ℂ) → ℝ) (u : ι → ℂ) : Matrix ι ι ℂ
```

lemma hessianMatrixOfR_isHermitian [source]

The mixed Wirtinger Hessian of a real function `C²` at `u` is Hermitian, `(g_{JĪ})^* = g_{IJ¯}`.

```
lemma hessianMatrixOfR_isHermitian {K : (ι → ℂ) → ℝ} {u : ι → ℂ} (hK : ContDiffAt ℝ 2 K u) :
    Matrix.IsHermitian (hessianMatrixOfR K u)
```

def restrict [source]

The restriction `q ↦ q ∘ Sigma.mk k` to sector `k`, as a continuous ℝ-linear map.

```
noncomputable def restrict (k : K) : ((Σ k, ι k) → ℂ) →L[ℝ] (ι k → ℂ)
```

lemma restrict_single_same [source]

`restrict k` sends the basis vector of field `⟨k, a'⟩` to the basis vector of field `a'`.

```
lemma restrict_single_same (k : K) (a' : ι k) (z : ℂ) :
    restrict k (Pi.single (⟨k, a'⟩ : Σ k, ι k) z) = Pi.single a' z
```

lemma restrict_single_ne [source]

`restrict k` sends the basis vector of a field of another sector to zero.

```
lemma restrict_single_ne {k k' : K} (h : k ≠ k') (a' : ι k') (z : ℂ) :
    restrict k (Pi.single (⟨k', a'⟩ : Σ k, ι k) z) = 0
```

lemma dWirtingerCoord_block_own [source]

`∂_⟨k, a⟩` of a function of sector `k` alone is its `∂_a`.

```
lemma dWirtingerCoord_block_own {k : K} (g : (ι k → ℂ) → ℂ) (a : ι k)
    (u : (Σ k, ι k) → ℂ) (hg : DifferentiableAt ℝ g (u ∘ Sigma.mk k)) :
    dWirtingerCoord (fun q => g (q ∘ Sigma.mk k)) ⟨k, a⟩ u
      = dWirtingerCoord g a (u ∘ Sigma.mk k)
```

lemma dWirtingerCoord_block_ne [source]

`∂_⟨k, a⟩` of a function of another sector vanishes.

```
lemma dWirtingerCoord_block_ne {k k' : K} (h : k ≠ k') (g : (ι k' → ℂ) → ℂ) (a : ι k)
    (u : (Σ k, ι k) → ℂ) (hg : DifferentiableAt ℝ g (u ∘ Sigma.mk k')) :
    dWirtingerCoord (fun q => g (q ∘ Sigma.mk k')) ⟨k, a⟩ u = 0
```

lemma dWirtingerAntiCoord_block_own [source]

Anti-holomorphic mirror of `dWirtingerCoord_block_own`.

```
lemma dWirtingerAntiCoord_block_own {k : K} (g : (ι k → ℂ) → ℂ) (a : ι k)
    (u : (Σ k, ι k) → ℂ) (hg : DifferentiableAt ℝ g (u ∘ Sigma.mk k)) :
    dWirtingerAntiCoord (fun q => g (q ∘ Sigma.mk k)) (⟨k, a⟩ : Σ k, ι k) u
      = dWirtingerAntiCoord g a (u ∘ Sigma.mk k)
```

lemma dWirtingerAntiCoord_block_ne [source]

Anti-holomorphic mirror of `dWirtingerCoord_block_ne`.

```
lemma dWirtingerAntiCoord_block_ne {k k' : K} (h : k ≠ k') (g : (ι k' → ℂ) → ℂ) (a : ι k)
    (u : (Σ k, ι k) → ℂ) (hg : DifferentiableAt ℝ g (u ∘ Sigma.mk k')) :
    dWirtingerAntiCoord (fun q => g (q ∘ Sigma.mk k')) (⟨k, a⟩ : Σ k, ι k) u = 0
```

lemma dWirtingerCoord_block_sum [source]

`∂_⟨k₀, a⟩` of a sum of sector functions is the `∂_a` of the sector-`k₀` function.

```
lemma dWirtingerCoord_block_sum (f : ∀ k, (ι k → ℂ) → ℂ) (k₀ : K) (a : ι k₀)
    (u : (Σ k, ι k) → ℂ) (hf : ∀ k, DifferentiableAt ℝ (f k) (u ∘ Sigma.mk k)) :
    dWirtingerCoord (fun q => ∑ k, f k (q ∘ Sigma.mk k)) ⟨k₀, a⟩ u
      = dWirtingerCoord (f k₀) a (u ∘ Sigma.mk k₀)
```

lemma dWirtingerAntiCoord_block_sum [source]

`∂̄_⟨k₀, a⟩` of a sum of sector functions is the `∂̄_a` of the sector-`k₀` function.

```
lemma dWirtingerAntiCoord_block_sum (f : ∀ k, (ι k → ℂ) → ℂ) (k₀ : K) (a : ι k₀)
    (u : (Σ k, ι k) → ℂ) (hf : ∀ k, DifferentiableAt ℝ (f k) (u ∘ Sigma.mk k)) :
    dWirtingerAntiCoord (fun q => ∑ k, f k (q ∘ Sigma.mk k)) ⟨k₀, a⟩ u
      = dWirtingerAntiCoord (f k₀) a (u ∘ Sigma.mk k₀)
```

lemma hessianMatrixOf_sum_eq_blockDiagonal' [source]

The Hessian of a sum of sector functions is the block-diagonal matrix of the sector Hessians.

```
lemma hessianMatrixOf_sum_eq_blockDiagonal' (f : ∀ k, (ι k → ℂ) → ℂ) (u : (Σ k, ι k) → ℂ)
    (hf : ∀ k, ContDiffAt ℝ 2 (f k) (u ∘ Sigma.mk k)) :
    hessianMatrixOf (fun q => ∑ k, f k (q ∘ Sigma.mk k)) u
      = blockDiagonal' (fun k => hessianMatrixOf (f k) (u ∘ Sigma.mk k))
```

def precompEquiv [source]

The relabelling `q ↦ q ∘ ε.symm` along `ε`, as a continuous $\mathbb{R}$-linear map.

```
noncomputable def precompEquiv (ε : ι ≃ ι') : (ι → ℂ) →L[ℝ] (ι' → ℂ)
```

lemma precompEquiv_single [source]

`precompEquiv ε` sends the basis vector of field `c` to the basis vector of field `ε c`.

```
lemma precompEquiv_single (ε : ι ≃ ι') (c : ι) (z : ℂ) :
    precompEquiv ε (Pi.single c z) = Pi.single (ε c) z
```

lemma hessianMatrixOf_comp_equiv [source]

Relabelling the coordinates by `ε` reindexes the Hessian by `ε`.

```
lemma hessianMatrixOf_comp_equiv (ε : ι ≃ ι') (H : (ι' → ℂ) → ℂ) (u : ι → ℂ)
    (hH : ContDiffAt ℝ 2 H (u ∘ ε.symm)) :
    hessianMatrixOf (fun q => H (q ∘ ε.symm)) u
      = (hessianMatrixOf H (u ∘ ε.symm)).submatrix ε ε
```

def restrictInl [source]

The restriction `q ↦ q ∘ Sum.inl` to the first sector, as a continuous $\mathbb{R}$-linear map.

```
noncomputable def restrictInl : (ι₁ ⊕ ι₂ → ℂ) →L[ℝ] (ι₁ → ℂ)
```

def restrictInr [source]

The restriction `q ↦ q ∘ Sum.inr` to the second sector, as a continuous $\mathbb{R}$-linear map.

```
noncomputable def restrictInr : (ι₁ ⊕ ι₂ → ℂ) →L[ℝ] (ι₂ → ℂ)
```

lemma hessianMatrixOf_add_eq_fromBlocks [source]

The Hessian of `f₁ (q ∘ Sum.inl) + f₂ (q ∘ Sum.inr)` is `fromBlocks` of the two sector Hessians.

```
lemma hessianMatrixOf_add_eq_fromBlocks
    (f₁ : (ι₁ → ℂ) → ℂ) (f₂ : (ι₂ → ℂ) → ℂ) (u : ι₁ ⊕ ι₂ → ℂ)
    (hf₁ : ContDiffAt ℝ 2 f₁ (u ∘ Sum.inl)) (hf₂ : ContDiffAt ℝ 2 f₂ (u ∘ Sum.inr)) :
    hessianMatrixOf (fun q => f₁ (q ∘ Sum.inl) + f₂ (q ∘ Sum.inr)) u
      = Matrix.fromBlocks (hessianMatrixOf f₁ (u ∘ Sum.inl)) 0 0
          (hessianMatrixOf f₂ (u ∘ Sum.inr))
```

2.58 Physlib.Mathematics.Distribution.TemperedDistribution

Physlib/Mathematics/Distribution/TemperedDistribution.lean on GitHub.

lemma integrable_eval_of_bound [source]

Evaluations of a distribution family with an integrable seminorm bound are integrable.

```
lemma integrable_eval_of_bound (η : 𝓢(E, 𝕜)) : Integrable (fun a => u a η) μ
```

def integralOfBound [source]

The weak integral of a family of distributions with an integrable Schwartz seminorm bound. It acts on a test function by integrating the evaluations of the family.

```
def integralOfBound : E →d[𝕜] F
```

lemma integralOfBound_apply [source]

Evaluation of the weak integral is the integral of the evaluations.

```
@[simp]
lemma integralOfBound_apply (η : 𝓢(E, 𝕜)) :
    integralOfBound u s b hb hmeas hbound hb_nonneg η = ∫ a, u a η ∂μ
```

lemma integrable_fderivD_eval_of_bound [source]

Differentiated evaluations of a family with an integrable seminorm bound are integrable.

```
lemma integrable_fderivD_eval_of_bound (η : 𝓢(E, 𝕜)) (v : E) :
    Integrable (fun a => fderivD 𝕜 (u a) η v) μ
```

lemma fderivD_integralOfBound_apply [source]

Distributional differentiation commutes with the weak integral after evaluation at a test function and a direction.

```
lemma fderivD_integralOfBound_apply (η : 𝓢(E, 𝕜)) (v : E) :
    fderivD 𝕜 (integralOfBound u s b hb hmeas hbound hb_nonneg) η v =
      ∫ a, fderivD 𝕜 (u a) η v ∂μ
```

2.59 Physlib.Mathematics.ForMathlib.ComplexLinear

Physlib/Mathematics/ForMathlib/ComplexLinear.lean on GitHub.

lemma map_smul_of_commutesI [source]

An $\mathbb{R}$-linear map commuting with `i` along `v` commutes with every complex scalar along `v`.

```
lemma map_smul_of_commutesI (L : V →ₗ[ℝ] E) {v : V}
    (h : L (Complex.I • v) = Complex.I • L v) (c : ℂ) :
    L (c • v) = c • L v
```

lemma commutesI_of_basis [source]

An $\mathbb{R}$-linear map commuting with `i` on a $\mathbb{C}$-basis commutes with `i` everywhere.

```
lemma commutesI_of_basis {ι : Type*} [Fintype ι] (L : V →ₗ[ℝ] E) (b : Module.Basis ι ℂ V)
    (h : ∀ i, L (Complex.I • b i) = Complex.I • L (b i)) (v : V) :
    L (Complex.I • v) = Complex.I • L v
```

def complexOfCommutesI [source]

A continuous $\mathbb{R}$-linear map commuting with `i`, as a continuous $\mathbb{C}$-linear map.

```
def complexOfCommutesI (L : V →L[ℝ] E)
    (h : ∀ v, L (Complex.I • v) = Complex.I • L v) : V →L[ℂ] E where
```

lemma complexOfCommutesI_apply [source]

```
@[simp]
lemma complexOfCommutesI_apply (L : V →L[ℝ] E)
    (h : ∀ v, L (Complex.I • v) = Complex.I • L v) (v : V) :
    complexOfCommutesI L h v = L v
```

2.60 Physlib.Mathematics.ForMathlib.Integration.Basic

Physlib/Mathematics/ForMathlib/Integration/Basic.lean on GitHub.

lemma integral_withDensity_ofReal_eq_integral_smul [source]

Integrating `g` against the measure with real-valued density `f` (taken through `ENNReal.ofReal`) is integrating `f x • g x` against the original measure, for measurable `f` that is everywhere nonnegative. This is the ℝ-valued form of `integral_withDensity_eq_integral_smul`, which takes an `NNReal`-valued density.

```
lemma integral_withDensity_ofReal_eq_integral_smul {α : Type*} [MeasurableSpace α]
    {μ : Measure α} {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    {f : α → ℝ} (hf : Measurable f) (hf0 : ∀ x, 0 ≤ f x) (g : α → E) :
    ∫ x, g x ∂(μ.withDensity fun x => ENNReal.ofReal (f x)) = ∫ x, f x • g x ∂μ
```

2.61 Physlib.Mathematics.ForMathlib.SesquilinearForm

Physlib/Mathematics/ForMathlib/SesquilinearForm.lean on GitHub.

structure IsPosDef [source]

*A sesquilinear form `B` is **positive-definite** if it is symmetric and `0 < B x x` for every nonzero `x`.*

```
structure IsPosDef [LT R] (B : M →ₛₗ[I₁] M →ₗ[R] R) : Prop extends isSymm : B.IsSymm where
  pos : ∀ x, x ≠ 0 → 0 < B x x
```

lemma isPosDef_def [source]

```
lemma isPosDef_def [LT R] {B : M →ₛₗ[I₁] M →ₗ[R] R} :
    B.IsPosDef ↔ B.IsSymm ∧ ∀ x, x ≠ 0 → 0 < B x x
```

lemma isPosSemidef [source]

A positive-definite form is positive-semidefinite.

```
lemma IsPosDef.isPosSemidef (hB : B.IsPosDef) : B.IsPosSemidef where
```

lemma nondegenerate [source]

A positive-definite form is nondegenerate: a nonzero `x` is not orthogonal to itself.

```
lemma IsPosDef.nondegenerate (hB : B.IsPosDef) : B.Nondegenerate
```

lemma isPosDef_iff_posDef_toMatrix [source]

A sesquilinear form is positive-definite exactly when its matrix in a basis is.

```
lemma isPosDef_iff_posDef_toMatrix (b : Basis n R M) : B.IsPosDef ↔ (toMatrix₂ b b B).PosDef
```

2.62 Physlib.Mathematics.ForMathlib.Trigonometry.Tanh

Physlib/Mathematics/ForMathlib/Trigonometry/Tanh.lean on GitHub.

lemma hasDerivAt_tanh [source]

The derivative of tanh at `x` is `1 - tanh(x)^2`

```
lemma hasDerivAt_tanh (x : ℝ) : HasDerivAt Real.tanh (1 - Real.tanh x ^ 2) x
```

lemma tendsto_tanh_atTop [source]

tanh(x) tends to `1` as `x → ∞`

```
lemma tendsto_tanh_atTop : Filter.Tendsto Real.tanh Filter.atTop (nhds 1)
```

lemma tendsto_tanh_atBot [source]

tanh(x) tends to `-1` as `x → -∞`

```
lemma tendsto_tanh_atBot : Filter.Tendsto Real.tanh Filter.atBot (nhds (-1))
```

2.63 Physlib.Mathematics.Groups.SO3.Basic

Physlib/Mathematics/Groups/SO3/Basic.lean on GitHub.

lemma eq_of_apply_two [source]

A rotation fixing the third axis is a rotation in the first two coordinates.

```
lemma eq_of_apply_two (A : SO(3)) (hA : ∀ i, A.1 i 2 = if i = 2 then 1 else 0) :
    A.1 = !![A.1 0 0, A.1 0 1, 0; -A.1 0 1, A.1 0 0, 0; 0, 0, 1]
```

2.64 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.Adjoint

Physlib/Mathematics/Groups/SpecialUnitary/LieAlgebra/Adjoint.lean on GitHub.

def adjoint [source]

The adjoint representation `x ↦ g x g†` of `SU(n)` on its real Lie algebra `su(n)`.

```
noncomputable def adjoint {n : ℕ} :
    Representation ℝ (specialUnitaryGroup (Fin n) ℂ) (SULieAlgebra n ℂ)
```

lemma adjoint_val [source]

The underlying matrix of `adjoint g x` is `g x g†`.

```
@[simp]
lemma adjoint_val {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ) (x : SULieAlgebra n ℂ) :
    (adjoint g x).1 = g.1 * x.1 * star g.1
```

lemma adjoint_lie [source]

The adjoint representation preserves the bracket, so `SU(n)` acts on `su(n)` by Lie algebra automorphisms.

```
lemma adjoint_lie {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ) (x y : SULieAlgebra n ℂ) :
    adjoint g ⁅x, y⁆ = ⁅adjoint g x, adjoint g y⁆
```

lemma adjoint_eq_one_iff [source]

The kernel of the adjoint representation is the centre of `SU(n)`: `g` acts trivially exactly when it is a scalar matrix `c • 1`.

```
lemma adjoint_eq_one_iff {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ) :
    adjoint g = 1 ↔ ∃ c : ℂ, g.1 = c • 1
```

lemma adjoint_eq_iff [source]

Two elements of `SU(n)` have the same adjoint action exactly when they differ by a scalar: the adjoint representation is faithful on `SU(n)` modulo its centre.

```
lemma adjoint_eq_iff {n : ℕ} (g h : specialUnitaryGroup (Fin n) ℂ) :
    adjoint g = adjoint h ↔ ∃ c : ℂ, g.1 = c • h.1
```

def adjointℂ [source]

The adjoint representation of `SU(n)` on the complexification `ℂ ⊗[ℝ] su(n)`: the complexification of the adjoint representation `adjoint` on `su(n)`.

```
noncomputable def adjointℂ {n : ℕ} :
    Representation ℂ (specialUnitaryGroup (Fin n) ℂ) (Complexification n)
```

lemma adjointℂ_tmul [source]

```
@[simp]
lemma adjointℂ_tmul {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ) (z : ℂ) (x : SULieAlgebra n ℂ)
    :
    adjointℂ g (z ⊗ₜ x) = z ⊗ₜ adjoint g x
```

lemma adjointℂ_lie [source]

The complex adjoint representation preserves the bracket of the complexification, so `SU(n)` acts on `ℂ ⊗[ℝ] su(n)` by Lie algebra automorphisms.

```
lemma adjointℂ_lie {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ) (A B : Complexification n) :
    adjointℂ g ⁅A, B⁆ = ⁅adjointℂ g A, adjointℂ g B⁆
```

lemma adjointℂ_eq_one_iff [source]

The kernel of the complex adjoint representation is the centre of `SU(n)`, as for the real adjoint representation.

```
lemma adjointℂ_eq_one_iff {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ) :
    adjointℂ g = 1 ↔ ∃ c : ℂ, g.1 = c • 1
```

lemma adjointℂ_eq_iff [source]

Two elements of `SU(n)` have the same complex adjoint action exactly when they differ by a scalar, as for the real adjoint representation.

```
lemma adjointℂ_eq_iff {n : ℕ} (g h : specialUnitaryGroup (Fin n) ℂ) :
    adjointℂ g = adjointℂ h ↔ ∃ c : ℂ, g.1 = c • h.1
```

lemma toMatrixℂ_adjointℂ [source]

The adjoint representation conjugates the matrix, `M ↦ g M g†`.

```
@[simp]
lemma toMatrixℂ_adjointℂ {n} (g : specialUnitaryGroup (Fin n) ℂ) (A : Complexification n) :
    toMatrixℂ (adjointℂ g A) = g.1 * toMatrixℂ A * star g.1
```

lemma exists_toMatrixℂ_adjointℂ_eq_unitary [source]

Conjugation by any unitary matrix is the adjoint action of an element of `SU(n)`: a unitary matrix agrees with an element of `SU(n)` up to a phase, which conjugation does not see.

```
lemma exists_toMatrixℂ_adjointℂ_eq_unitary {n : ℕ} (U : unitaryGroup (Fin n) ℂ) :
    ∃ g : specialUnitaryGroup (Fin n) ℂ, ∀ A : Complexification n,
      toMatrixℂ (adjointℂ g A) = U.1 * toMatrixℂ A * star U.1
```

lemma exists_toMatrixℂ_adjointℂ_apply_perm [source]

The adjoint action realises any permutation `σ` of the coordinates: some element of `SU(n)` moves the entry `(x, y)` of every matrix to `(σ x, σ y)`.

```
lemma exists_toMatrixℂ_adjointℂ_apply_perm {n : ℕ} (σ : Equiv.Perm (Fin n)) :
    ∃ g : specialUnitaryGroup (Fin n) ℂ, ∀ A x y,
      toMatrixℂ (adjointℂ g A) (σ x) (σ y) = toMatrixℂ A x y
```

lemma toMatrixℂ_adjointℂ_apply_of_diagonal [source]

The adjoint action of a diagonal element of `SU(n)` scales the entry at `(x, y)` of the matrix by `d x * star (d y)`.

```
lemma toMatrixℂ_adjointℂ_apply_of_diagonal {n} {g : specialUnitaryGroup (Fin n) ℂ}
    {d : Fin n → ℂ} (hg : g.1 = diagonal d) (A : Complexification n) (x y : Fin n) :
    toMatrixℂ (adjointℂ g A) x y = d x * star (d y) * toMatrixℂ A x y
```

lemma toMatrixℂ_adjointℂ_trace [source]

```
lemma toMatrixℂ_adjointℂ_trace {n} (g : specialUnitaryGroup (Fin n) ℂ) (A : Complexification n)
    :
    (toMatrixℂ (adjointℂ g A)).trace = (toMatrixℂ A).trace
```

lemma exists_adjointℂ_singleℂ_eq_singleℂ [source]

All the off-diagonal matrix units `singleℂ` lie in one adjoint orbit

```
lemma exists_adjointℂ_singleℂ_eq_singleℂ {n : ℕ} {i j p q : Fin n} (hij : i ≠ j) (hpq : p ≠ q)
    :
    ∃ g : specialUnitaryGroup (Fin n) ℂ, adjointℂ g (singleℂ i j hij) = singleℂ p q hpq
```

lemma singleℂ_mem_span_range_adjointℂ [source]

If the matrix of `A` has a non-zero entry at the off-diagonal position `(p, q)`, then `singleℂ p q h` lies in the span of the adjoint orbit of `A`.

```
lemma singleℂ_mem_span_range_adjointℂ {n : ℕ} {p q : Fin n} (h : p ≠ q)
    {A : Complexification n} (hA : toMatrixℂ A p q ≠ 0) :
    singleℂ p q h ∈ Submodule.span ℂ (Set.range fun g => adjointℂ g A)
```

lemma exists_adjointℂ_eq_of_toMatrixℂ_eq_diagonal [source]

The adjoint action of some element of `SU(n)` — the rotation by `π / 4` in the `(p, q)`-plane — changes an element whose matrix is the diagonal matrix `d` only in the `(p, q)`-block, by `(d p - d q) / 2` times `singleℂ p q h + singleℂ q p h.symm - diagDiffℂ p q`.

```
lemma exists_adjointℂ_eq_of_toMatrixℂ_eq_diagonal {n : ℕ} {p q : Fin n} (h : p ≠ q)
    {A : Complexification n} {d : Fin n → ℂ} (hA : toMatrixℂ A = diagonal d) :
    ∃ g : specialUnitaryGroup (Fin n) ℂ, adjointℂ g A =
      A + ((d p - d q) / 2) • (singleℂ p q h + singleℂ q p h.symm - diagDiffℂ p q)
```

lemma exists_diagDiffℂ_eq_adjointℂ_singleℂ_add_singleℂ [source]

The difference of two diagonal entries lies in the adjoint orbit of the sum of the two elements supported at the corresponding off-diagonal entries.

```
lemma exists_diagDiffℂ_eq_adjointℂ_singleℂ_add_singleℂ {n : ℕ} {p q : Fin n} (h : p ≠ q) :
    ∃ g : specialUnitaryGroup (Fin n) ℂ,
      diagDiffℂ p q = adjointℂ g (singleℂ p q h + singleℂ q p h.symm)
```

lemma exists_singleℂ_mem_span_range_adjointℂ [source]

The span of the adjoint orbit of a non-zero element contains an element supported at a single off-diagonal entry.

```
lemma exists_singleℂ_mem_span_range_adjointℂ {n : ℕ} {A : Complexification n} (hA : A ≠ 0) :
    ∃ (p q : Fin n) (h : p ≠ q),
      singleℂ p q h ∈ Submodule.span ℂ (Set.range fun g => adjointℂ g A)
```

lemma isIrreducible_adjointℂ [source]

The complex adjoint representation of `SU(n)` is irreducible for `2 ≤ n`.

```
lemma isIrreducible_adjointℂ {n : ℕ} (hn : 2 ≤ n) : (adjointℂ (n := n)).IsIrreducible
```

lemma end_eq_smul_id_of_commute_adjointℂ [source]

An endomorphism of the complexification commuting with the adjoint action is a scalar.

```
lemma end_eq_smul_id_of_commute_adjointℂ {n : ℕ} {T : Module.End ℂ (Complexification n)}
    (hT : ∀ g, T ∘ₗ adjointℂ g = adjointℂ g ∘ₗ T) : ∃ μ : ℂ, T = μ • LinearMap.id
```

2.65 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.Basic

Physlib/Mathematics/Groups/SpecialUnitary/LieAlgebra/Basic.lean on GitHub.

abbrev submodule [source]

The traceless hermitian `n × n` matrices with entries in `R`, as a real subspace.

```
abbrev SULieAlgebra.submodule (n : ℕ) (R : Type*) [CommRing R] [StarRing R] [Algebra ℝ R]
    [StarModule ℝ R] : Submodule ℝ (Matrix (Fin n) (Fin n) R)
```

abbrev SULieAlgebra [source]

Traceless hermitian `n × n` matrices with entries in `R`, as a real Lie algebra; `SULieAlgebra n ℂ` is `su(n)`.

```
abbrev SULieAlgebra (n : ℕ) (R : Type*) [CommRing R] [StarRing R] [Algebra ℝ R]
    [StarModule ℝ R] : Type _
```

def ofMatrix [source]

The element given by a traceless hermitian matrix.

```
def ofMatrix (A : Matrix (Fin n) (Fin n) R) (hA : star A = A) (hT : A.trace = 0) :
    SULieAlgebra n R
```

lemma val_ofMatrix [source]

The underlying matrix of `ofMatrix A hA hT` is `A`.

```
@[simp]
lemma val_ofMatrix (A : Matrix (Fin n) (Fin n) R) (hA : star A = A) (hT : A.trace = 0) :
    (ofMatrix A hA hT).1 = A
```

lemma star_val_eq [source]

The underlying matrix of an element is hermitian.

```
lemma star_val_eq (x : SULieAlgebra n R) : star x.1 = x.1
```

lemma trace_val_eq_zero [source]

The underlying matrix of an element is traceless.

```
lemma trace_val_eq_zero (x : SULieAlgebra n R) : x.1.trace = 0
```

def conj [source]

The conjugation representation of the unitary group on `SULieAlgebra n R`: `x ↦ U x U†`.

```
noncomputable def conj : Representation ℝ (unitaryGroup (Fin n) R) (SULieAlgebra n R) where
```

lemma val_conj_apply [source]

The underlying matrix of `conj U x` is `U x U†`.

```
@[simp]
lemma val_conj_apply (U : unitaryGroup (Fin n) R) (x : SULieAlgebra n R) :
    (conj U x).1 = U.1 * x.1 * star U.1
```

instance Bracket (SULieAlgebra n R) (SULieAlgebra n R) [source]

The bracket `⁅x, y⁆ = i (x y - y x)`.

```
noncomputable instance : Bracket (SULieAlgebra n R) (SULieAlgebra n R) where
```

lemma val_bracket [source]

The underlying matrix of `⁅x, y⁆` is `i (x y - y x)`.

```
@[simp]
lemma val_bracket (x y : SULieAlgebra n R) :
    ⁅x, y⁆.1 = Complex.I • (x.1 * y.1 - y.1 * x.1)
```

instance LieRing (SULieAlgebra n R) [source]

`⁅x, y⁆ = i (x y - y x)` makes `SULieAlgebra n R` a Lie ring.

```
noncomputable instance : LieRing (SULieAlgebra n R) where
```

instance LieAlgebra ℝ (SULieAlgebra n R) [source]

The bracket is ℝ-bilinear, so `SULieAlgebra n R` is a real Lie algebra.

```
noncomputable instance : LieAlgebra ℝ (SULieAlgebra n R) where
```

lemma conj_lie [source]

Conjugation by a unitary matrix preserves the bracket, so `conj` acts by Lie algebra automorphisms.

```
lemma conj_lie (U : unitaryGroup (Fin n) R) (x y : SULieAlgebra n R) :
    conj U ⁅x, y⁆ = ⁅conj U x, conj U y⁆
```

abbrev Complexification [source]

The complexification `ℂ ⊗[ℝ] su(n)` of `su(n)`, a complex Lie algebra.

```
abbrev Complexification (n : ℕ) : Type
```

def toMatrixℂ [source]

The underlying matrix of an element of the complexification, `z ⊗ x ↦ z x`.

```
noncomputable def toMatrixℂ : Complexification n →ₗ[ℂ] Matrix (Fin n) (Fin n) ℂ
```

lemma toMatrixℂ_tmul [source]

```
@[simp]
lemma toMatrixℂ_tmul (z : ℂ) (x : SULieAlgebra n ℂ) : toMatrixℂ (z ⊗ₜ x) = z • x.1
```

lemma trace_toMatrixℂ [source]

The underlying matrix of an element of the complexification is traceless.

```
lemma trace_toMatrixℂ (A : Complexification n) : (toMatrixℂ A).trace = 0
```

lemma toMatrixℂ_lie [source]

The matrix of a bracket is `i` times the commutator of the matrices, `[A, B] ↦ i (A B - B A)`, as in `su(n)` itself.

```
lemma toMatrixℂ_lie (A B : Complexification n) :
    toMatrixℂ ⁅A, B⁆ = Complex.I • (toMatrixℂ A * toMatrixℂ B - toMatrixℂ B * toMatrixℂ A)
```

def ofTracelessℂ [source]

The element `1 ⊗ ℜ M + i ⊗ ℑ M` of the complexification with the traceless matrix `M`.

```
noncomputable def ofTracelessℂ (M : Matrix (Fin n) (Fin n) ℂ) (hM : M.trace = 0) :
    Complexification n
```

lemma toMatrixℂ_ofTracelessℂ [source]

The matrix of `ofTracelessℂ M` is `M = ℜ M + i ℑ M`.

```
@[simp]
lemma toMatrixℂ_ofTracelessℂ (M : Matrix (Fin n) (Fin n) ℂ) (hM : M.trace = 0) :
    toMatrixℂ (ofTracelessℂ M hM) = M
```

lemma ofTracelessℂ_add [source]

`ofTracelessℂ` is additive.

```
lemma ofTracelessℂ_add (M N : Matrix (Fin n) (Fin n) ℂ) (hM : M.trace = 0) (hN : N.trace = 0)
    (hMN : (M + N).trace = 0) :
    ofTracelessℂ (M + N) hMN = ofTracelessℂ M hM + ofTracelessℂ N hN
```

lemma ofTracelessℂ_toMatrixℂ [source]

An element of the complexification is recovered from its matrix.

```
@[simp]
lemma ofTracelessℂ_toMatrixℂ (A : Complexification n) :
    ofTracelessℂ (toMatrixℂ A) (trace_toMatrixℂ A) = A
```

lemma toMatrixℂ_injective [source]

An element of the complexification is determined by its matrix.

```
lemma toMatrixℂ_injective : Function.Injective (toMatrixℂ (n := n))
```

lemma range_toMatrixℂ [source]

The matrices of elements of the complexification are exactly the traceless matrices.

```
lemma range_toMatrixℂ :
    LinearMap.range (toMatrixℂ (n := n)) = LinearMap.ker (Matrix.traceLinearMap (Fin n) ℂ ℂ)
```

lemma commute_of_forall_commute_val [source]

A matrix commuting with every element of `su(n)` commutes with every matrix: it commutes with the traceless matrices, which `su(n)` spans over `ℂ`, and with the identity.

```
lemma commute_of_forall_commute_val {M : Matrix (Fin n) (Fin n) ℂ}
    (h : ∀ x : SULieAlgebra n ℂ, Commute M x.1) (N : Matrix (Fin n) (Fin n) ℂ) : Commute M N
```

def equivTraceKer [source]

The linear equivalence between the complexification of `su(n)` and the traceless complex matrices, sending an element to its matrix.

```
noncomputable def equivTraceKer :
    Complexification n ≃ₗ[ℂ] LinearMap.ker (Matrix.traceLinearMap (Fin n) ℂ ℂ) where
```

lemma val_equivTraceKer [source]

The matrix of `equivTraceKer A` is the matrix of `A`.

```
@[simp]
lemma val_equivTraceKer (A : Complexification n) : (equivTraceKer A).1 = toMatrixℂ A
```

lemma toMatrixℂ_equivTraceKer_symm [source]

The matrix of `equivTraceKer.symm M` is `M`.

```
@[simp]
lemma toMatrixℂ_equivTraceKer_symm (M : LinearMap.ker (traceLinearMap (Fin n) ℂ ℂ)) :
    toMatrixℂ (equivTraceKer.symm M) = M.1
```

def singleℂ [source]

The element of the complexification supported at the single off-diagonal entry `(p, q)`.

```
noncomputable def singleℂ (p q : Fin n) (h : p ≠ q) : Complexification n
```

def diagDiffℂ [source]

The element of the complexification corresponding to the difference of diagonal entries `(p, p) - (q, q)`.

```
noncomputable def diagDiffℂ (p q : Fin n) : Complexification n
```

lemma submodule_eq_top_of_singleℂ_mem_of_diagDiffℂ_mem [source]

A submodule of the complexification containing every element supported at a single off-diagonal entry, and every difference of two diagonal entries, is everything.

```
lemma submodule_eq_top_of_singleℂ_mem_of_diagDiffℂ_mem {n : ℕ}
    {W : Submodule ℂ (Complexification n)}
    (hoff : ∀ (p q : Fin n) (h : p ≠ q), singleℂ p q h ∈ W)
    (hdiag : ∀ p q : Fin n, diagDiffℂ p q ∈ W) :
    W = ⊤
```

lemma singleℂ_diagDiffℂ_span_eq_top [source]

The elements supported at a single off-diagonal entry, together with the differences of two diagonal entries, span the complexification.

```
lemma singleℂ_diagDiffℂ_span_eq_top {n : ℕ} :
    Submodule.span ℂ ({A : Complexification n | ∃ (p q : Fin n) (h : p ≠ q), A = singleℂ p q h}
     ∪
      {A : Complexification n | ∃ p q : Fin n, A = diagDiffℂ p q}) = ⊤
```

lemma finrank_complexification [source]

The complexification `ℂ ⊗[ℝ] su(n)` has complex dimension `n² - 1`: it is the traceless matrices, and the trace is onto for `0 < n`.

```
lemma finrank_complexification : Module.finrank ℂ (Complexification n) = n ^ 2 - 1
```

lemma finrank_eq [source]

`su(n)` has real dimension `n² - 1`, the dimension of its complexification.

```
lemma finrank_eq : Module.finrank ℝ (SULieAlgebra n ℂ) = n ^ 2 - 1
```

2.66 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.Contractions

`Physlib/Mathematics/Groups/SpecialUnitary/LieAlgebra/Contractions.lean` on GitHub.

def adjointContr [source]

The contraction of two real adjoint indices: the trace form `x ⊗ y ↦ tr (x y)` on `su(n)`, which is real since `x` and `y` are hermitian.

```
noncomputable def adjointContr {n : ℕ} : ((adjoint (n := n)).tprod adjoint).IntertwiningMap
    (Representation.trivial ℝ (specialUnitaryGroup (Fin n) ℂ) ℝ) where
```

lemma adjointContr_tmul [source]

The real contraction is the real part of the trace, `x ⊗ y ↦ re tr (x y)`.

```
@[simp]
lemma adjointContr_tmul {n : ℕ} (x y : SULieAlgebra n ℂ) :
    adjointContr (x ⊗ₜ y) = (x.1 * y.1).trace.re
```

lemma adjointContr_adjoint [source]

The adjoint representation preserves the real contraction, so `SU(n)` acts on `su(n)` by isometries.

```
lemma adjointContr_adjoint {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ) (x y : SULieAlgebra n ℂ)
    :
    adjointContr (adjoint g x ⊗ₜ adjoint g y) = adjointContr (x ⊗ₜ y)
```

lemma ofReal_adjointContr_tmul [source]

The trace `tr (x y)` of two elements of `su(n)` is real, equal to their contraction.

```
lemma ofReal_adjointContr_tmul {n : ℕ} (x y : SULieAlgebra n ℂ) :
    (adjointContr (x ⊗ₜ y) : ℂ) = (x.1 * y.1).trace
```

lemma adjointContr_symm [source]

```
lemma adjointContr_symm {n : ℕ} (x y : SULieAlgebra n ℂ) :
    adjointContr (x ⊗ₜ y) = adjointContr (y ⊗ₜ x)
```

lemma adjointContr_lie [source]

The real contraction is invariant under the bracket, `tr ([x, y] z) = tr (x [y, z])`; in terms of generators, the structure constants `tr (T^a [T^b, T^c])` are totally antisymmetric.

```
lemma adjointContr_lie {n : ℕ} (x y z : SULieAlgebra n ℂ) :
    adjointContr ([x, y] ⊗ₜ z) = adjointContr (x ⊗ₜ [y, z])
```

lemma adjointContr_self_nonneg [source]

The real contraction is positive semidefinite: `tr (x x) = tr (x x†) ≥ 0`.

```
lemma adjointContr_self_nonneg {n : ℕ} (x : SULieAlgebra n ℂ) : 0 ≤ adjointContr (x ⊗ₜ x)
```

lemma adjointContr_self_eq_zero_iff [source]

The real contraction of `x` with itself vanishes only for `x = 0`.

```
lemma adjointContr_self_eq_zero_iff {n : ℕ} (x : SULieAlgebra n ℂ) :
    adjointContr (x ⊗ₜ x) = 0 ↔ x = 0
```

lemma adjointContr_self_pos [source]

The real contraction is positive definite, making it an inner product on `su(n)`.

```
lemma adjointContr_self_pos {n : ℕ} {x : SULieAlgebra n ℂ} (hx : x ≠ 0) :
    0 < adjointContr (x ⊗ₜ x)
```

lemma adjointContr_separating_left [source]

```
lemma adjointContr_separating_left {n : ℕ} (x : SULieAlgebra n ℂ)
    (hx : ∀ y, adjointContr (x ⊗ₜ y) = 0) : x = 0
```

lemma adjointContr_separating_right [source]

```
lemma adjointContr_separating_right {n : ℕ} (y : SULieAlgebra n ℂ)
    (hy : ∀ x, adjointContr (x ⊗ₜ y) = 0) : y = 0
```

lemma adjointContr_flip_injective [source]

The real contraction separates `su(n)`: `y ↦ (x ↦ re tr (x y))` is injective.

```
lemma adjointContr_flip_injective {n : ℕ} :
    Function.Injective (TensorProduct.curry (adjointContr (n := n)).toLinearMap).flip
```

lemma adjointContr_isSymm [source]

The real contraction, as a bilinear form, is symmetric.

```
lemma adjointContr_isSymm {n : ℕ} :
    LinearMap.BilinForm.IsSymm (TensorProduct.curry (adjointContr (n := n)).toLinearMap)
```

lemma adjointContr_nondegenerate [source]

The real contraction, as a bilinear form, is nondegenerate.

```
lemma adjointContr_nondegenerate {n : ℕ} :
    (TensorProduct.curry (adjointContr (n := n)).toLinearMap).Nondegenerate
```

def adjointℂContr [source]

The contraction of two complex adjoint indices: the complexification of the real contraction `adjointContr`.

```
noncomputable def adjointℂContr {n : ℕ} : ((adjointℂ (n := n)).tprod adjointℂ).IntertwiningMap
    (Representation.trivial ℂ (specialUnitaryGroup (Fin n) ℂ) ℂ) where
```

lemma adjointℂContr_adjointℂ [source]

The adjoint representation preserves the complex contraction.

```
lemma adjointℂContr_adjointℂ {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ)
    (A B : Complexification n) :
    adjointℂContr (adjointℂ g A ⊗ₜ adjointℂ g B) = adjointℂContr (A ⊗ₜ B)
```

lemma adjointℂContr_tmul [source]

The contraction is the trace form, `A ⊗ B ↦ tr (A B)`.

```
lemma adjointℂContr_tmul (A B : Complexification n) :
    adjointℂContr (A ⊗ₜ B) = (toMatrixℂ A * toMatrixℂ B).trace
```

lemma adjointℂContr_one_tmul [source]

On the real elements `1 ⊗ x` of the complexification the complex contraction is the real contraction, so the two agree on real gauge fields.

```
lemma adjointℂContr_one_tmul {n : ℕ} (x y : SULieAlgebra n ℂ) :
    adjointℂContr ((1 ⊗ₜ x) ⊗ₜ (1 ⊗ₜ y)) = adjointContr (x ⊗ₜ y)
```

lemma adjointℂContr_symm [source]

```
lemma adjointℂContr_symm {n} (A B : Complexification n) :
    adjointℂContr (A ⊗ₜ B) = adjointℂContr (B ⊗ₜ A)
```

lemma adjointℂContr_lie [source]

The complex contraction is invariant under the bracket, `tr ([A, B] C) = tr (A [B, C])`.

```
lemma adjointℂContr_lie {n} (A B C : Complexification n) :
    adjointℂContr ([A, B] ⊗ₜ C) = adjointℂContr (A ⊗ₜ [B, C])
```

lemma adjointℂContr_separating_left [source]

```
lemma adjointℂContr_separating_left {n} (A : Complexification n)
    (hA : ∀ B, adjointℂContr (A ⊗ₜ B) = 0) : A = 0
```

lemma adjointℂContr_separating_right [source]

```
lemma adjointℂContr_separating_right {n} (B : Complexification n)
    (hB : ∀ A, adjointℂContr (A ⊗ₜ B) = 0) : B = 0
```

lemma adjointℂContr_flip_injective [source]

The contraction separates the complexification: `B ↦ (A ↦ tr (A B))` is injective.

```
lemma adjointℂContr_flip_injective :
    Function.Injective (TensorProduct.curry (adjointℂContr (n := n)).toLinearMap).flip
```

lemma adjointℂContr_isSymm [source]

The complex contraction, as a bilinear form, is symmetric.

```
lemma adjointℂContr_isSymm {n : ℕ} :
    LinearMap.BilinForm.IsSymm (TensorProduct.curry (adjointℂContr (n := n)).toLinearMap)
```

lemma adjointℂContr_nondegenerate [source]

The complex contraction, as a bilinear form, is nondegenerate.

```
lemma adjointℂContr_nondegenerate {n : ℕ} :
    (TensorProduct.curry (adjointℂContr (n := n)).toLinearMap).Nondegenerate
```

lemma exists_eq_adjointℂContr_tmul [source]

Every bilinear form on the complexification is the contraction against an endomorphism, the contraction being nondegenerate.

```
lemma exists_eq_adjointℂContr_tmul {n : ℕ}
    (B : Complexification n ⊗[ℂ] Complexification n →ₗ[ℂ] ℂ) :
    ∃ T : Module.End ℂ (Complexification n), ∀ A C, B (A ⊗ₜ C) = adjointℂContr (T A ⊗ₜ C)
```

lemma adjointℂ_comp_eq_of_adjointℂContr_tmul [source]

The endomorphism representing an invariant bilinear form commutes with the adjoint action.

```
lemma adjointℂ_comp_eq_of_adjointℂContr_tmul {n : ℕ}
    (B : ((adjointℂ (n := n)).tprod adjointℂ).IntertwiningMap
      (Representation.trivial ℂ (specialUnitaryGroup (Fin n) ℂ) ℂ))
    {T : Module.End ℂ (Complexification n)} (hT : ∀ A C, B (A ⊗ₜ C) = adjointℂContr (T A ⊗ₜ C))
    (g : specialUnitaryGroup (Fin n) ℂ) :
    T ∘ₗ adjointℂ g = adjointℂ g ∘ₗ T
```

lemma intertwiningMap_eq_smul_adjointℂContr [source]

Every intertwiner `adj ⊗ adj → ℂ` is a multiple of the contraction: up to scale, the contraction is the unique `SU(n)`-invariant bilinear form on the complexification.

```
lemma intertwiningMap_eq_smul_adjointℂContr {n : ℕ}
    (B : ((adjointℂ (n := n)).tprod adjointℂ).IntertwiningMap
      (Representation.trivial ℂ (specialUnitaryGroup (Fin n) ℂ) ℂ)) :
    ∃ z : ℂ, B = z • adjointℂContr
```

2.67 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.GellMann.Adjoint

Physlib/Mathematics/Groups/SpecialUnitary/LieAlgebra/GellMann/Adjoint.lean on GitHub.

def adjointGellMannBasis [source]

The matrix of the adjoint action of `g` in the Gell-Mann basis, a real matrix, as a monoid homomorphism from `SU(n)`.

```
noncomputable def adjointGellMannBasis {n : ℕ} :
    specialUnitaryGroup (Fin n) ℂ →* Matrix (GellMannIndex n) (GellMannIndex n) ℝ
```

lemma adjointGellMannBasis_apply [source]

The entries of the matrix of the adjoint action, `re tr (λ_i g λ_j g†) / 2`.

```
lemma adjointGellMannBasis_apply {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ)
    (i j : GellMannIndex n) : adjointGellMannBasis g i j =
      ((gellMannMatrices i).1 * (g.1 * (gellMannMatrices j).1 * star g.1)).trace.re / 2
```

lemma adjointGellMannBasis_inv [source]

The matrix of the adjoint action of `g⁻¹` is the transpose of that of `g`, the adjoint action preserving the contraction.

```
lemma adjointGellMannBasis_inv {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ) :
    adjointGellMannBasis g⁻¹ = (adjointGellMannBasis g)ᵀ
```

lemma adjointGellMannBasis_transpose_mul_self [source]

The matrix of the adjoint action is orthogonal.

```
lemma adjointGellMannBasis_transpose_mul_self {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ) :
    (adjointGellMannBasis g)ᵀ * adjointGellMannBasis g = 1
```

lemma toMatrix_adjointℂ_gellMannBasisℂ [source]

The matrix of the complex adjoint action in the complex Gell-Mann basis is the real matrix of the adjoint action.

```
lemma toMatrix_adjointℂ_gellMannBasisℂ {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ) :
    LinearMap.toMatrix gellMannBasisℂ gellMannBasisℂ (adjointℂ g) =
      (adjointGellMannBasis g).map (algebraMap ℝ ℂ)
```

lemma adjoint_gellMannMatrices [source]

The adjoint action on a Gell-Mann matrix is a combination of Gell-Mann matrices with coefficients the entries of the matrix of the adjoint action, `g λ_j g† = ∑ i, R_ij(g) λ_i`.

```
lemma adjoint_gellMannMatrices {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ) (j : GellMannIndex n
    ) :
    adjoint g (gellMannMatrices j) = ∑ i, adjointGellMannBasis g i j • gellMannMatrices i
```

lemma gellMannBasis_repr_adjoint [source]

The adjoint action on the coordinates in the Gell-Mann basis is multiplication by the matrix of the adjoint action, `A^i ↦ R_ij(g) A^j`.

```
lemma gellMannBasis_repr_adjoint {n : ℕ} (g : specialUnitaryGroup (Fin n) ℂ)
    (x : SULieAlgebra n ℂ) :
    gellMannBasis.repr (adjoint g x) = adjointGellMannBasis g *ᵥ gellMannBasis.repr x
```

2.68 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.GellMann.Basic

Physlib/Mathematics/Groups/SpecialUnitary/LieAlgebra/GellMann/Basic.lean on GitHub.

inductive GellMannIndex [source]

The indexing type of the Gell-Mann matrices.

```
inductive GellMannIndex (n : ℕ)
  | symm (p : Fin n) (q : Fin n) (h : p < q) : GellMannIndex n
  | antisymm (p : Fin n) (q : Fin n) (h : p < q) : GellMannIndex n
  | diag (l : Fin (n - 1)) : GellMannIndex n
```

lemma gellMannIndex_card [source]

There are `n² - 1` generalized Gell-Mann matrices: `n (n - 1) / 2` symmetric ones, as many antisymmetric ones, and `n - 1` diagonal ones.

```
lemma gellMannIndex_card {n : ℕ} : Fintype.card (GellMannIndex n) = n ^ 2 - 1
```

def gellMannMatrices [source]

The generalized Gell-Mann matrices, as elements of `su(n)`:

- for `p < q`, the symmetric matrix `E_pq + E_qp`, - for `p < q`, the antisymmetric matrix `-i E_pq + i E_qp`, - for `l : Fin (n - 1)`, with `m = l + 1`, the diagonal matrix `√(2 / (m (m + 1))) (E_00 + ⋯ + E_(m-1)(m-1) - m E_mm)`.

```
noncomputable def gellMannMatrices {n : ℕ} : GellMannIndex n → SULieAlgebra n ℂ
  | .symm p q h => ofMatrix (single p q 1 + single q p 1)
      (by simp [star_eq_conjTranspose, conjTranspose_single, add_comm])
      (by rw [trace_add, trace_single_eq_of_ne _ _ _ h.ne, trace_single_eq_of_ne _ _ _ h.ne',
        add_zero])
  | .antisymm p q h => ofMatrix (single p q (-Complex.I) + single q p Complex.I)
      (by simp [star_eq_conjTranspose, conjTranspose_single, add_comm])
      (by rw [trace_add, trace_single_eq_of_ne _ _ _ h.ne, trace_single_eq_of_ne _ _ _ h.ne',
        add_zero])
  | .diag l => ofMatrix (diagonal fun x => (√(2 / ((l.1 + 1) * (l.1 + 2))) : ℝ) *
      if x.1 ≤ l.1 then 1 else if x.1 = l.1 + 1 then -(l.1 + 1 : ℂ) else 0)
      (by
        ext x y
        simp only [star_apply, diagonal_apply]
        split_ifs <;> simp_all [Complex.conj_ofReal]
        omega)
      (by
        have hl := l.2
        rw [trace_diagonal, ← Finset.mul_sum, Fin.sum_univ_eq_sum_range (fun x =>
          if x ≤ l.1 then (1 : ℂ) else if x = l.1 + 1 then -(l.1 + 1 : ℂ) else 0),
          ← Finset.sum_range_add_sum_Ico _ (show l.1 + 2 ≤ n by omega), Finset.sum_range_succ,
          Finset.sum_congr (s₁ := Finset.range (l.1 + 1)) rfl fun x hx =>
            ite_eq_left (Nat.lt_succ_iff.1 (Finset.mem_range.1 hx)),
          Finset.sum_eq_zero (s := Finset.Ico (l.1 + 2) n) fun x hx => by
            have := (Finset.mem_Ico.1 hx).1
            rw [ite_eq_right (by omega), ite_eq_right (by omega)]]
        simp)
```

lemma gellMannMatrices_symm_isSymm [source]

The symmetric generalized Gell-Mann matrices are symmetric.

```
lemma gellMannMatrices_symm_isSymm {n : ℕ} (p q : Fin n) (h : p < q) :
```

```
    Matrix.IsSymm (gellMannMatrices (.symm p q h)).1
```

lemma gellMannMatrices_antisymm_transpose [source]

The antisymmetric generalized Gell-Mann matrices are antisymmetric.

```
lemma gellMannMatrices_antisymm_transpose {n : ℕ} (p q : Fin n) (h : p < q) :
    (gellMannMatrices (.antisymm p q h)).1^T = -(gellMannMatrices (.antisymm p q h)).1
```

lemma gellMannMatrices_diag_isDiag [source]

The diagonal generalized Gell-Mann matrices are diagonal.

```
lemma gellMannMatrices_diag_isDiag {n : ℕ} (l : Fin (n - 1)) :
    Matrix.IsDiag (gellMannMatrices (.diag l)).1
```

lemma gellMannMatrices_symm_val_eq [source]

```
lemma gellMannMatrices_symm_val_eq {n : ℕ} (p q : Fin n) (h : p < q) :
    (gellMannMatrices (.symm p q h)).1 = single p q 1 + single q p 1
```

lemma gellMannMatrices_antisymm_val_eq [source]

```
lemma gellMannMatrices_antisymm_val_eq {n : ℕ} (p q : Fin n) (h : p < q) :
    (gellMannMatrices (.antisymm p q h)).1 = single p q (-Complex.I) + single q p Complex.I
```

lemma gellMannMatrices_diag_val_eq [source]

```
lemma gellMannMatrices_diag_val_eq {n : ℕ} (l : Fin (n - 1)) :
    (gellMannMatrices (.diag l)).1 = diagonal fun x => (√(2 / ((l.1 + 1) * (l.1 + 2))) : ℝ) *
      if x.1 ≤ l.1 then 1 else if x.1 = l.1 + 1 then -(l.1 + 1 : ℂ) else 0
```

lemma adjointContr_gellMannMatrices_symm_antisymm_eq_zero [source]

```
lemma adjointContr_gellMannMatrices_symm_antisymm_eq_zero {n : ℕ} (p q : Fin n) (h : p < q)
    (p' q' : Fin n) (h' : p' < q') :
    adjointContr (gellMannMatrices (.symm p q h) ⊗ₜ gellMannMatrices (.antisymm p' q' h')) = 0
```

lemma adjointContr_gellMannMatrices_symm_diag_eq_zero [source]

```
lemma adjointContr_gellMannMatrices_symm_diag_eq_zero {n : ℕ} (p q : Fin n) (h : p < q)
    (l : Fin (n - 1)) :
    adjointContr (gellMannMatrices (.symm p q h) ⊗ₜ gellMannMatrices (.diag l)) = 0
```

lemma adjointContr_gellMannMatrices_antisymm_symm_eq_zero [source]

```
lemma adjointContr_gellMannMatrices_antisymm_symm_eq_zero {n : ℕ} (p q : Fin n) (h : p < q)
    (p' q' : Fin n) (h' : p' < q') :
    adjointContr (gellMannMatrices (.antisymm p q h) ⊗ₜ gellMannMatrices (.symm p' q' h')) = 0
```

lemma adjointContr_gellMannMatrices_antisymm_diag_eq_zero [source]

```
lemma adjointContr_gellMannMatrices_antisymm_diag_eq_zero {n : ℕ} (p q : Fin n) (h : p < q)
    (l : Fin (n - 1)) :
    adjointContr (gellMannMatrices (.antisymm p q h) ⊗ₜ gellMannMatrices (.diag l)) = 0
```

lemma adjointContr_gellMannMatrices_diag_symm_eq_zero [source]

```
lemma adjointContr_gellMannMatrices_diag_symm_eq_zero {n : ℕ} (l : Fin (n - 1)) (p q : Fin n)
    (h : p < q) :
    adjointContr (gellMannMatrices (.diag l) ⊗ₜ gellMannMatrices (.symm p q h)) = 0
```

lemma adjointContr_gellMannMatrices_diag_antisymm_eq_zero [source]

```
lemma adjointContr_gellMannMatrices_diag_antisymm_eq_zero {n : ℕ} (l : Fin (n - 1)) (p q : Fin
    n)
    (h : p < q) :
    adjointContr (gellMannMatrices (.diag l) ⊗ₜ gellMannMatrices (.antisymm p q h)) = 0
```

lemma adjointContr_gellMannMatrices_symm_orthogonal [source]

```
lemma adjointContr_gellMannMatrices_symm_orthogonal {n : ℕ} (p q : Fin n) (h : p < q)
    (p' q' : Fin n) (h' : p' < q') :
    adjointContr (gellMannMatrices (.symm p q h) ⊗ₜ gellMannMatrices (.symm p' q' h')) =
      if p = p' ∧ q = q' then 2 else 0
```

lemma adjointContr_gellMannMatrices_antisymm_orthogonal [source]

```
lemma adjointContr_gellMannMatrices_antisymm_orthogonal {n : ℕ} (p q : Fin n) (h : p < q)
    (p' q' : Fin n) (h' : p' < q') :
    adjointContr (gellMannMatrices (.antisymm p q h) ⊗ₜ gellMannMatrices (.antisymm p' q' h'))
    =
      if p = p' ∧ q = q' then 2 else 0
```

lemma adjointContr_gellMannMatrices_diag_self [source]

The diagonal Gell-Mann matrices are normalized, `tr (λ_l λ_l) = 2`.

```
lemma adjointContr_gellMannMatrices_diag_self {n : ℕ} (l : Fin (n - 1)) :
    adjointContr (gellMannMatrices (.diag l) ⊗ₜ gellMannMatrices (.diag l)) = 2
```

lemma adjointContr_gellMannMatrices_diag_orthogonal [source]

```
lemma adjointContr_gellMannMatrices_diag_orthogonal {n : ℕ} (l l' : Fin (n - 1)) :
    adjointContr (gellMannMatrices (.diag l) ⊗ₜ gellMannMatrices (.diag l')) =
      if l = l' then 2 else 0
```

lemma adjointContr_gellMannMatrices [source]

The generalized Gell-Mann matrices are orthogonal under the trace pairing, with normalization `tr (λ_i λ_j) = 2 δ_ij`.

```
lemma adjointContr_gellMannMatrices {n : ℕ} (i j : GellMannIndex n) :
    adjointContr (gellMannMatrices i ⊗ₜ gellMannMatrices j) = if i = j then 2 else 0
```

lemma linearIndependent_gellMannMatrices [source]

The generalized Gell-Mann matrices are linearly independent over $\mathbb{R}$, being orthogonal under the contraction.

```
lemma linearIndependent_gellMannMatrices {n : ℕ} :
    LinearIndependent ℝ (gellMannMatrices (n := n))
```

def gellMannBasis [source]

The generalized Gell-Mann basis of `su(n)`.

```
noncomputable def gellMannBasis {n : ℕ} : Module.Basis (GellMannIndex n) ℝ (SULieAlgebra n ℂ)
```

lemma gellMannBasis_apply [source]

```
@[simp]
lemma gellMannBasis_apply {n : ℕ} (i : GellMannIndex n) : gellMannBasis i = gellMannMatrices i
```

lemma gellMannBasis_repr_apply [source]

The coordinates in the Gell-Mann basis are the contractions with the Gell-Mann matrices, `x = ∑ (adjointContr (λ_i ⊗ x) / 2) λ_i`.

```
lemma gellMannBasis_repr_apply {n : ℕ} (x : SULieAlgebra n ℂ) (i : GellMannIndex n) :
    gellMannBasis.repr x i = adjointContr (gellMannMatrices i ⊗ₜ x) / 2
```

def gellMannBasisℂ [source]

The generalized Gell-Mann basis `1 ⊗ λ_i` of the complexification `ℂ ⊗[ℝ] su(n)`.

```
noncomputable def gellMannBasisℂ {n : ℕ} : Module.Basis (GellMannIndex n) ℂ (Complexification n
    )
```

lemma gellMannBasisℂ_apply [source]

```
lemma gellMannBasisℂ_apply {n : ℕ} (i : GellMannIndex n) :
    gellMannBasisℂ i = 1 ⊗ₜ gellMannMatrices i
```

lemma toMatrixℂ_gellMannBasisℂ [source]

```
@[simp]
lemma toMatrixℂ_gellMannBasisℂ {n : ℕ} (i : GellMannIndex n) :
    toMatrixℂ (gellMannBasisℂ i) = (gellMannMatrices i).1
```

lemma gellMannBasisℂ_repr_apply [source]

The coordinates in the complex Gell-Mann basis are the contractions with the Gell-Mann matrices, `A = ∑ (adjointℂContr (λ_i ⊗ A) / 2) λ_i`.

```
lemma gellMannBasisℂ_repr_apply {n : ℕ} (A : Complexification n) (i : GellMannIndex n) :
    gellMannBasisℂ.repr A i = adjointℂContr (gellMannBasisℂ i ⊗ₜ A) / 2
```

def gellMannStructureConst [source]

The structure constants of `su(n)` in the Gell-Mann basis, with the physics normalisation `[λ_a, λ_b] = 2 i ∑ c, f_abc λ_c`, read off by the contraction: `f_abc = tr ([λ_a, λ_b] λ_c) / (4 i) = -(1 / 4) adjointContr ([λ_a, λ_b] ⊗ λ_c)`.

```
noncomputable def gellMannStructureConst {n : ℕ} (a b c : GellMannIndex n) : ℝ
```

lemma lie_gellMannMatrices [source]

The bracket of two Gell-Mann matrices in terms of the structure constants, `[λ_a, λ_b] = -2 ∑ c, f_abc λ_c`, the bracket `[x, y] = i (x y - y x)` carrying a factor `i`.

```
lemma lie_gellMannMatrices {n : ℕ} (a b : GellMannIndex n) :
    ⁅gellMannMatrices a, gellMannMatrices b⁆ =
      ∑ c, (-2 * gellMannStructureConst a b c) • gellMannMatrices c
```

lemma gellMannStructureConst_swap [source]

The structure constants are antisymmetric in their first two indices.

```
lemma gellMannStructureConst_swap {n : ℕ} (a b c : GellMannIndex n) :
    gellMannStructureConst b a c = -gellMannStructureConst a b c
```

lemma gellMannStructureConst_cycle [source]

The structure constants are invariant under cyclic permutations of their indices, the contraction being invariant under the bracket. With `gellMannStructureConst_swap`, they are totally antisymmetric.

```
lemma gellMannStructureConst_cycle {n : ℕ} (a b c : GellMannIndex n) :
    gellMannStructureConst b c a = gellMannStructureConst a b c
```

2.69 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.GellMann.SpecialCases

Physlib/Mathematics/Groups/SpecialUnitary/LieAlgebra/GellMann/SpecialCases.lean on GitHub.

def equivTwo [source]

For `n = 2`, the labelling of the generalized Gell-Mann matrices as the Pauli matrices `σ₁, σ₂, σ₃` (as `0, 1, 2`): the symmetric, antisymmetric and diagonal matrices.

```
def GellMannIndex.equivTwo : GellMannIndex 2 ≃ Fin 3 where
```

lemma gellMannMatrices_two [source]

For `n = 2` the generalized Gell-Mann matrices are the Pauli matrices `σ₁, σ₂, σ₃`, labelled by `GellMannIndex.equivTwo`.

```
lemma gellMannMatrices_two (i : Fin 3) :
    (gellMannMatrices (GellMannIndex.equivTwo.symm i)).1 = PauliMatrix.pauliMatrix (Sum.inr i)
```

def equivThree [source]

For `n = 3`, the labelling of the Gell-Mann matrices in the standard order `λ₁, …, λ₈` (as `0, …, 7`): `λ₁, λ₂, λ₃` are the symmetric, antisymmetric and diagonal matrices of the first two coordinates, `λ₄, λ₅` the symmetric and antisymmetric ones of the coordinates `0, 2`, `λ₆, λ₇` those of the coordinates `1, 2`, and `λ₈` the second diagonal matrix.

```
def GellMannIndex.equivThree : GellMannIndex 3 ≃ Fin 8 where
```

lemma gellMannMatrices_three [source]

For `n = 3` the generalized Gell-Mann matrices are the Gell-Mann matrices `λ₁, …, λ₈`, labelled in the standard order by `GellMannIndex.equivThree`.

```
lemma gellMannMatrices_three :
    (fun i => (gellMannMatrices (GellMannIndex.equivThree.symm i)).1) =
    ![!![0, 1, 0; 1, 0, 0; 0, 0, 0],
      !![0, -Complex.I, 0; Complex.I, 0, 0; 0, 0, 0],
      !![1, 0, 0; 0, -1, 0; 0, 0, 0],
      !![0, 0, 1; 0, 0, 0; 1, 0, 0],
      !![0, 0, -Complex.I; 0, 0, 0; Complex.I, 0, 0],
      !![0, 0, 0; 0, 0, 1; 0, 1, 0],
      !![0, 0, 0; 0, 0, -Complex.I; 0, Complex.I, 0],
      (1 / Real.sqrt 3 : ℂ) • !![1, 0, 0; 0, 1, 0; 0, 0, -2]]
```

2.70 Physlib.Mathematics.Groups.SpecialUnitary.LieAlgebra.KillingForm

Physlib/Mathematics/Groups/SpecialUnitary/LieAlgebra/KillingForm.lean on GitHub.

lemma killingForm_eq_trace [source]

The Killing form of the complexification is the trace form of the matrices, `K(A, B) = -2n tr (A B)`. The sign comes from the factor `i` in the bracket, which makes `ad A ∘ ad B` the map `M ↦ -[A, [B, M]]`.

```
lemma killingForm_eq_trace {n : ℕ} (A B : Complexification n) :
    killingForm ℂ (Complexification n) A B = - (2 * n) * (toMatrixℂ A * toMatrixℂ B).trace
```

lemma adjointℂContr_eq_killingForm [source]

The contraction is the Killing form on the complexified Lie algebra, up to a factor of `-1/(2n)`.

```
lemma adjointℂContr_eq_killingForm {n : ℕ} (A B : Complexification n) :
    adjointℂContr (A ⊗ₜ B) = -(1 / (2 * (n : ℂ))) * killingForm ℂ (Complexification n) A B
```

lemma adjointContr_eq_killingForm [source]

The contraction is the Killing form on the Lie algebra, up to a factor of `-1/(2n)`.

```
lemma adjointContr_eq_killingForm {n : ℕ} (x y : SULieAlgebra n ℂ) :
    adjointContr (x ⊗ₜ y) = -(1 / (2 * (n : ℝ))) * killingForm ℝ (SULieAlgebra n ℂ) x y
```

2.71 Physlib.Mathematics.InnerProductSpace.Gaussian

Physlib/Mathematics/InnerProductSpace/Gaussian.lean on GitHub.

lemma exp_mul_gaussian [source]

Multiplication by a real linear exponential shifts and rescales a general Gaussian.

```
lemma exp_mul_gaussian (v : E) :
    (ofReal (rexp (inner ℝ v x)) : 𝕜) * gaussian 𝕜 B x₀ x =
      (ofReal (rexp (inner ℝ v x₀ + 2⁻¹ * ‖B.toContinuousLinearMap.adjoint v‖ ^ 2)) : 𝕜) *
        gaussian 𝕜 B (x₀ + B (B.toContinuousLinearMap.adjoint v)) x
```

def gaussianExpMul [source]

The product of `gaussian 𝕜 B x₀` with the real exponential `exp ⟪v, x⟫`, as a Schwartz map. It has center `x₀ + B (Bᵀ v)` and the same covariance, with scale factor `exp (⟪v, x₀⟫ + 2⁻¹ * ‖Bᵀ v‖ ^ 2)`.

```
def gaussianExpMul (v : E) : 𝒮(E, 𝕜)
```

lemma gaussianExpMul_apply [source]

Evaluation of the bundled product agrees with pointwise multiplication by the exponential.

```
@[simp]
lemma gaussianExpMul_apply (v : E) :
    gaussianExpMul 𝕜 B x₀ v x = (ofReal (rexp (inner ℝ v x)) : 𝕜) * gaussian 𝕜 B x₀ x
```

2.72 Physlib.Mathematics.KroneckerDelta.Contraction

Physlib/Mathematics/KroneckerDelta/Contraction.lean on GitHub.

lemma sum_generalizedKroneckerDelta_self_card [source]

***Full contraction over an arbitrary finite type.** Summing `δ^{h}_{h}` over all `k`-tuples `h` of indices gives the falling product `card α · (card α - 1) ⋯ (card α - k + 1)`.*

```
lemma sum_generalizedKroneckerDelta_self_card (k : ℕ) :
    ∑ h : Fin k → α, generalizedKroneckerDelta h h
      = ∏ j ∈ Finset.range k, ((Fintype.card α : ℤ) - j)
```

lemma sum_generalizedKroneckerDelta_cons_card [source]

***Single contraction over an arbitrary finite type.** Contracting the last `k` of `k+1` index pairs leaves one free pair `σ, τ`, with the falling product `(card α - 1) (card α - 2) ⋯`.*

```
lemma sum_generalizedKroneckerDelta_cons_card (σ τ : α) (k : ℕ) :
    ∑ h : Fin k → α,
        generalizedKroneckerDelta (Fin.cons σ h) (Fin.cons τ h)
      = (∏ j ∈ Finset.range k, ((Fintype.card α : ℤ) - 1 - j))
        * ((kroneckerDelta σ τ : ℕ) : ℤ)
```

lemma sum_generalizedKroneckerDelta_cons₂_card [source]

***Double contraction over an arbitrary finite type.** Contracting the last `k` of `k+2` index pairs leaves two free pairs, with value a `2×2` generalized Kronecker delta times the falling product `(card α - 2) (card α - 3) ⋯`.*

```
lemma sum_generalizedKroneckerDelta_cons₂_card (ρ σ τ ω : α) (k : ℕ) :
    ∑ h : Fin k → α,
      generalizedKroneckerDelta (Fin.cons ρ (Fin.cons σ h)) (Fin.cons τ (Fin.cons ω h))
    = (∏ j ∈ Finset.range k, ((Fintype.card α : ℤ) - 2 - j))
      * generalizedKroneckerDelta ![ρ, σ] ![τ, ω]
```

lemma sum_generalizedKroneckerDelta_self_eq_factorial [source]

When the index type has exactly `k` elements, the full contraction is `k!`.

```
lemma sum_generalizedKroneckerDelta_self_eq_factorial {k : ℕ} (hk : Fintype.card α = k) :
    ∑ h : Fin k → α, generalizedKroneckerDelta h h = (k.factorial : ℤ)
```

2.73 Physlib.Mathematics.VariationalCalculus.Interval

Physlib/Mathematics/VariationalCalculus/Interval.lean on GitHub.

lemma eqOn_Icc_zero_of_integral_contDiff_smul_eq_zero [source]

If `f : ℝ → V →L[ℝ] ℝ` is continuous on `[a, b]` and `∫ x in a..b, f x (ψ x • v) = 0` for every smooth `ψ` supported in `(a, b)` and every `v : V`, then `f = 0` on `[a, b]`.

```
lemma eqOn_Icc_zero_of_integral_contDiff_smul_eq_zero {f : ℝ → V →L[ℝ] ℝ} {a b : ℝ}
    (hf : ContinuousOn f (Icc a b)) (hab : a < b)
    (hint : ∀ ψ : ℝ → ℝ, ContDiff ℝ ∞ ψ → tsupport ψ ⊆ Ioo a b →
      ∀ v : V, ∫ x in a..b, f x (ψ x • v) = 0) :
    ∀ x ∈ Icc a b, f x = 0
```

2.74 Physlib.Meta.Pseudo

Physlib/Meta/Pseudo.lean on GitHub.

structure State [source]

A structure used for collecting names of results dependent on the `Lean.ofReduceBool` axiom.

```
structure State where
  visited : NameSet := {}
  containsOfReduceBool : NameSet := {}
```

abbrev M [source]

A monad used for collecting names of results dependent on the `Lean.ofReduceBool` axiom.

```
abbrev M
```

def collect [source]

Given a `c : Name` updating the monad `M` based on which results which `c` uses depend on the `Lean.ofReduceBool` axiom.

```
partial def collect (c : Name) (parents : NameSet) : M Unit
```

def allOfReduceBool [source]

Given a `c : Array Name` updating the monad `M` based on which results depend on the `Lean.ofReduceBool` axiom.

```
partial def allOfReduceBool (c : Array Name) : M Unit
```

def collectOfReduceBool [source]

Given a `c : Array Name` the names of all results used to defined the results in `c` with the `Lean.ofReduceBool` axiom.

```
def collectOfReduceBool (c : Array Name) : CoreM (Array Name)
```

def libraryImports [source]

The files of the libraries with root modules `libs`, as the imports of those roots.

```
def libraryImports (libs : Array Name) : IO (Array Import)
```

def libraryUserConsts [source]

The user defined constants of the libraries with root modules `libs`.

```
def libraryUserConsts (libs : Array Name) : CoreM (Array ConstantInfo)
```

def allWithOfReduceBool [source]

The user defined constants of the libraries `libs` depending on the `Lean.ofReduceBool` axiom.

```
def allWithOfReduceBool (libs : Array Name) : CoreM (Array Name)
```

def pseudoTest [source]

Checks, over the libraries with root modules `libs`, that the results attributed `pseudo` are exactly those depending on the `Lean.ofReduceBool` axiom. Returns whether the test passed.

```
unsafe def pseudoTest (libs : Array Name) : MetaM Bool
```

2.75 Physlib.Optics.Frequency

`Physlib/Optics/Frequency.lean` on GitHub.

abbrev Frequency [source]

The type `Frequency` represents the measure of of how often sinusodial components of a wave repeat per unit time, i.e. a nonnegative real number carrying the dimension of inverse time, in a given (but arbitrary) set of units.

```
abbrev Frequency : Type
```

2.76 Physlib.Optics.Wavelength

Physlib/Optics/Wavelength.lean on GitHub.

abbrev Wavelength [source]

The type `Wavelength` represents the distance between consecutive corresponding points of the same phase on the wave, such as two adjacent crests, troughs, or zero crossings, i.e. a nonnegative real number carrying the dimension of length.

```
abbrev Wavelength : Type
```

def ofFrequency [source]

The wavelength of a wave of frequency `ν` propagating at a speed `c`, given by `λ = c / ν`.

```
noncomputable def ofFrequency (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (ν : Frequency) (_ν_pos : 0 < ν.val
    ) :
    Wavelength
```

lemma ofFrequency_val [source]

```
@[simp]
lemma ofFrequency_val (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (ν : Frequency) (hν : 0 < ν.val) :
    (ofFrequency c ν hν).val = c.val / ν.val
```

lemma ofFrequency_pos [source]

Positivity of the wavelength associated with a positive frequency and a positive speed.

```
lemma ofFrequency_pos (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (hc : 0 < c.val) (ν : Frequency)
    (hν : 0 < ν.val) : 0 < (ofFrequency c ν hν).val
```

lemma ofFrequency_mul [source]

The wavelength associated with a frequency satisfies `λ * ν = c`.

```
lemma ofFrequency_mul (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (ν : Frequency) (hν : 0 < ν.val) :
    (ofFrequency c ν hν).val * ν.val = c.val
```

lemma ofFrequency_scaleUnit [source]

The wavelength associated with a frequency scales with units as a wavelength should: changing the units from `u1` to `u2` rescales the resulting wavelength by the scale factor of the dimension of length.

```
lemma ofFrequency_scaleUnit (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (ν : Frequency) (hν : 0 < ν.val)
    (u1 u2 : LTMCTUnitChoices) :
    ofFrequency (scaleUnit u1 u2 c) (scaleUnit u1 u2 ν) (WithDim.scaleUnit_val_pos ν hν u1 u2)
    =
      scaleUnit u1 u2 (ofFrequency c ν hν)
```

def ofWavelength [source]

The frequency of a wave of wavelength `w` propagating at a speed `c`, given by `ν = c / w`. We can't use `λ` due to Lean anon function.

```
noncomputable def ofWavelength (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (w : Wavelength) (_w_pos : 0 < w.
    val) :
    Frequency
```

lemma ofWavelength_val [source]

```
@[simp]
lemma ofWavelength_val (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (w : Wavelength) (w_pos : 0 < w.val) :
    (ofWavelength c w w_pos).val = c.val / w.val
```

lemma ofWavelength_pos [source]

Positivity of the frequency associated with a positive wavelength and a positive speed.

```
lemma ofWavelength_pos (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (c_pos : 0 < c.val) (w : Wavelength)
    (hw : 0 < w.val) : 0 < (ofWavelength c w hw).val
```

lemma ofWavelength_mul [source]

The frequency associated with a wavelength satisfies `ν * λ = c`.

```
lemma ofWavelength_mul (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (w : Wavelength) (w_pos : 0 < w.val) :
    (ofWavelength c w w_pos).val * w.val = c.val
```

lemma ofWavelength_scaleUnit [source]

The frequency associated with a wavelength scales with units as a frequency should: changing the units from `u1` to `u2` rescales the resulting frequency by the scale factor of the dimension of inverse time.

```
lemma ofWavelength_scaleUnit (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (w : Wavelength) (w_pos : 0 < w.val)
    (u1 u2 : LTMCTUnitChoices) :
    ofWavelength (scaleUnit u1 u2 c) (scaleUnit u1 u2 w)
      (WithDim.scaleUnit_val_pos w w_pos u1 u2) =
      scaleUnit u1 u2 (ofWavelength c w w_pos)
```

lemma ofWavelength_ofFrequency [source]

Converting a wavelength to a frequency and back recovers the wavelength.

```
@[simp]
lemma ofWavelength_ofFrequency (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (hc : 0 < c.val)
    (ν : Frequency) (hν : 0 < ν.val) :
```

```
    ofWavelength c (Wavelength.ofFrequency c ν hν)
      (Wavelength.ofFrequency_pos c hc ν hν) = ν
```

lemma ofFrequency_ofWavelength [source]

Converting a frequency to a wavelength and back recovers the frequency.

```
@[simp]
lemma ofFrequency_ofWavelength (c : WithDim (L⌈ * T⌈⁻¹) ℝ≥0) (hc : 0 < c.val)
    (w : Wavelength) (hw : 0 < w.val) :
    ofFrequency c (Frequency.ofWavelength c w hw)
      (Frequency.ofWavelength_pos c hc w hw) = w
```

def SpeedEqWavelengthMulFrequency [source]

The relation `c = λ * ν` between a speed `c`, a wavelength `λ` and a frequency `ν`, stated on real-valued quantities carrying the dimensions of speed, length and inverse time respectively.

```
def SpeedEqWavelengthMulFrequency (c : WithDim (L⌈ * T⌈⁻¹) ℝ) (w : WithDim L⌈ ℝ)
    (ν : WithDim T⌈⁻¹ ℝ) : Prop
```

lemma speedEqWavelengthMulFrequency_isDimensionallyCorrect [source]

The relation `c = λ * ν` is dimensionally correct: it holds in one system of units if and only if it holds in every system of units.

```
lemma speedEqWavelengthMulFrequency_isDimensionallyCorrect :
    IsDimensionallyCorrect SpeedEqWavelengthMulFrequency
```

2.77 Physlib.Particles.SuperSymmetry.N1.Basic

Physlib/Particles/SuperSymmetry/N1/Basic.lean on GitHub.

lemma chiralTensor_contr_chiralBasis [source]

The chiral contraction of two basis vectors is the Kronecker δ.

```
lemma chiralTensor_contr_chiralBasis (d : ChiralColor) (x₁ x₂ : ι) :
    (chiralTensor (ι := ι)).contr d
        (chiralBasis (ι := ι) d x₁ ⊗ₜ[ℂ] chiralBasis (ι := ι) ((chiralTensor (ι := ι)).τ d) x₂)
      = if x₁ = x₂ then 1 else 0
```

instance instHasContrDualBases [source]

The chiral species has dual bases at dual colours, matched by the identity on `ι`.

```
instance chiralTensor.instHasContrDualBases :
    TensorSpecies.HasContrDualBases (chiralTensor (ι := ι)).toTensorSpecies where
```

lemma contrDualIdxEquiv_eq_refl [source]

The dual-label matching of the chiral species is the identity.

```
@[simp]
lemma contrDualIdxEquiv_eq_refl (c : ChiralColor) :
    TensorSpecies.HasContrDualBases.contrDualIdxEquiv (chiralTensor (ι := ι)).toTensorSpecies c
     =
      Equiv.refl ι
```

2.78 Physlib.Particles.SuperSymmetry.N1.Superpotential

Physlib/Particles/SuperSymmetry/N1/Superpotential.lean on GitHub.

structure Superpotential [source]

A superpotential on the domain `D`: a function `W` of the chiral configuration, holomorphic on `D`.

```
structure Superpotential (ι : Type) [Fintype ι] [DecidableEq ι]
    (D : Set (ChiralScalarConfiguration ι)) where
  W : ChiralScalarConfiguration ι → ℂ
  holomorphic : ∀ u ∈ D, DifferentiableAt ℂ W u
```

instance CoeFun (Superpotential ι D) (fun _ => ChiralScalarConfiguration ι → ℂ) [source]

A superpotential is applied as the function `W`.

```
instance : CoeFun (Superpotential ι D) (fun _ => ChiralScalarConfiguration ι → ℂ)
```

def dW [source]

The gradient `∂_I W` of the superpotential, a covector of colour `chiralDown`.

```
def dW (W : Superpotential ι D) (u : ChiralScalarConfiguration ι) :
    (chiralTensor (ι := ι)).Tensor ![chiralDown]
```

lemma repr_dW [source]

The `I`-th component of the gradient is `∂_I W`.

```
lemma repr_dW (W : Superpotential ι D) (u : ChiralScalarConfiguration ι) (I : ι) :
    (basis (S := (chiralTensor (ι := ι)).toTensorSpecies) ![chiralDown]).repr (W.dW u) ![I] =
      dWirtingerCoord W I u
```

2.79 Physlib.ProbabilisticTheory.Channel.Basic

Physlib/ProbabilisticTheory/Channel/Basic.lean on GitHub.

structure UnitalPositiveLinearMap [source]

A positive linear map preserving `1`.

```
@[ext]
structure UnitalPositiveLinearMap (R E F : Type*) [Semiring R]
    [AddCommMonoid E] [PartialOrder E] [AddCommMonoid F] [PartialOrder F]
    [Module R E] [Module R F] [One E] [One F] extends E →ₚ[R] F, OneHom E F
```

instance FunLike (E →ₚ₁[R] F) E F [source]

```
instance : FunLike (E →ₚ₁[R] F) E F where
```

instance LinearMapClass (E →ₚ₁[R] F) R E F [source]

```
instance : LinearMapClass (E →ₚ₁[R] F) R E F where
```

instance OrderHomClass (E →ₚ₁[R] F) E F [source]

```
instance : OrderHomClass (E →ₚ₁[R] F) E F where
```

instance OneHomClass (E →ₚ₁[R] F) E F [source]

```
instance : OneHomClass (E →ₚ₁[R] F) E F where
```

instance Coe (E →ₚ₁[R] F) (E →ₚ[R] F) [source]

```
instance : Coe (E →ₚ₁[R] F) (E →ₚ[R] F)
```

lemma toPositiveLinearMap_apply [source]

```
@[simp]
lemma toPositiveLinearMap_apply (f : E →ₚ₁[R] F) (x : E) : f.toPositiveLinearMap x = f x
```

lemma toPositiveLinearMap_injective [source]

Unital positive linear maps are determined by their underlying positive linear map.

```
lemma toPositiveLinearMap_injective :
    Function.Injective (toPositiveLinearMap (R := R) (E := E) (F := F))
```

lemma toLinearMap_injective [source]

Unital positive linear maps are determined by their underlying linear map.

```
lemma toLinearMap_injective : Function.Injective (fun f : E →ₚ₁[R] F => f.toLinearMap)
```

def id [source]

The identity as a unital positive linear map.

```
def id : E →ₚ₁[R] E where
```

def comp [source]

The composition of unital positive linear maps.

```
def comp (g : F →ₚ₁[R] G) (f : E →ₚ₁[R] F) : E →ₚ₁[R] G where
```

instance Monoid (E →ₚ₁[R] E) [source]

Unital positive linear maps from a space to itself form a monoid under composition, with the identity as unit.

```
instance : Monoid (E →ₚ₁[R] E) where
```

def ofLinearMap [source]

Bundle a linear map after proving only positivity and preservation of 1.

```
def ofLinearMap (f : E →ₗ[R] F) (hpos : ∀ x, 0 ≤ x → 0 ≤ f x) (hone : f 1 = 1) : E →ₚ₁[R] F
    where
```

def ofPositiveLinearMap [source]

Bundle a positive linear map after proving only that it preserves 1.

```
def ofPositiveLinearMap (f : E →ₚ[R] F) (hone : f 1 = 1) : E →ₚ₁[R] F where
```

lemma ofPositiveLinearMap_apply [source]

```
@[simp]
lemma ofPositiveLinearMap_apply (f : E →ₚ[R] F) (hone : f 1 = 1) (x : E) :
    ofPositiveLinearMap f hone x = f x
```

instance SMul NNReal (E →ₚ[R] F) [source]

```
instance : SMul NNReal (E →ₚ[R] F) where
```

lemma nnreal_smul_apply [source]

```
@[simp]
lemma nnreal_smul_apply (c : NNReal) (f : E →ₚ[R] F) (x : E) : (c • f) x = c • f x
```

2.80 Physlib.ProbabilisticTheory.Channel.Covariant

Physlib/ProbabilisticTheory/Channel/Covariant.lean on GitHub.

def IsIntertwining [source]

A channel intertwines two group actions if it commutes with their transformations.

```
def IsIntertwining (Φ : E →ₚ₁[ℝ] F) : Prop
```

lemma isIntertwining_id [source]

The identity channel intertwines a group action with itself.

```
lemma isIntertwining_id : IsIntertwining ρ ρ (.id ℝ E)
```

lemma comp [source]

The composition of intertwining channels also intertwines the group actions.

```
lemma comp {H : Type*} [OrderUnitSpace H] {τ : G →* Reversible H H}
    {Φ : E →ₚ₁[ℝ] F} {Ψ : F →ₚ₁[ℝ] H} (hΦ : IsIntertwining ρ σ Φ)
    (hΨ : IsIntertwining σ τ Ψ) : IsIntertwining ρ τ (Ψ.comp Φ)
```

lemma comap_isInvariant [source]

Pulling back an invariant state along an intertwining channel preserves invariance.

```
lemma comap_isInvariant {Φ : E →ₚ₁[ℝ] F} (h : IsIntertwining ρ σ Φ)
    {ω : 𝒮[ℝ, F]} (hω : IsInvariant σ ω) :
    IsInvariant ρ (UnitalPositiveLinearMap.comap Φ ω)
```

lemma isIntertwining_transport [source]

A reversible transformation intertwines an action with its transport.

```
lemma isIntertwining_transport :
    IsIntertwining ρ (e.transport ρ) e.toChannel
```

lemma isIntertwining_iff_eq_transport [source]

A reversible transformation intertwines two actions exactly when the target action is the transport of the source action.

```
lemma isIntertwining_iff_eq_transport :
    IsIntertwining ρ σ e.toChannel ↔ σ = e.transport ρ
```

2.81 Physlib.ProbabilisticTheory.Channel.Reversible

Physlib/ProbabilisticTheory/Channel/Reversible.lean on GitHub.

structure Reversible [source]

A reversible transformation between observable spaces: a linear bijection that preserves and reflects the order and preserves the order unit. Both it and its inverse are channels.

```
structure Reversible (E F : Type*) [OrderUnitSpace E] [OrderUnitSpace F]
    extends E ≃ₗ[ℝ] F where
  map_le_map_iff' : ∀ {x y : E}, toLinearEquiv x ≤ toLinearEquiv y ↔ x ≤ y
  map_one' : toLinearEquiv 1 = 1
```

instance FunLike (Reversible E F) E F [source]

```
instance : FunLike (Reversible E F) E F where
```

lemma ext [source]

```
@[ext]
lemma ext {e f : Reversible E F} (h : ∀ x, e x = f x) : e = f
```

def refl [source]

The identity reversible transformation.

```
def refl (E : Type*) [OrderUnitSpace E] : Reversible E E where
```

def symm [source]

The inverse of a reversible transformation.

```
def symm (e : Reversible E F) : Reversible F E where
```

def trans [source]

Composition of reversible transformations, applying e_1 before e_2.

```
def trans (e₁ : Reversible E F) (e₂ : Reversible F H) : Reversible E H where
```

instance automorphismGroup [source]

Reversible transformations of a single space form a group under composition.

```
instance automorphismGroup : Group (Reversible E E) where
```

def toChannel [source]

A reversible transformation regarded as a channel.

```
def toChannel (e : Reversible E F) : E →ₚ₁[ℝ] F
```

def toChannelHom [source]

Regarding reversible transformations as channels is a homomorphism of monoids.

```
def toChannelHom : Reversible E E →* (E →ₚ₁[ℝ] E) where
```

instance instMulActionState [source]

The action on states induced by the inverse of a reversible transformation.

```
instance instMulActionState : MulAction (Reversible E E) (𝓢[ℝ, E]) where
```

lemma smul_state_apply [source]

```
lemma smul_state_apply (φ : Reversible E E) (ω : 𝓢[ℝ, E]) (x : E) :
    (φ • ω) x = ω (φ.symm x)
```

def conj [source]

Conjugation by a reversible transformation transports automorphisms from E to F.

```
def conj : Reversible E E →* Reversible F F where
```

def transport [source]

Transport a group action from `E` to `F` along a reversible transformation.

```
def transport : G →* Reversible F F
```

def IsInvariant [source]

A state is invariant if it is fixed by every transformation of the group action.

```
def IsInvariant {G E : Type*} [Group G] [OrderUnitSpace E] (ρ : G →* Reversible E E)
    (ω : 𝒮[ℝ, E]) : Prop
```

2.82 Physlib.ProbabilisticTheory.Effect.Complement

Physlib/ProbabilisticTheory/Effect/Complement.lean on GitHub.

lemma coe_complement [source]

```
@[simp]
lemma coe_complement (e : Effect E) : ((complement e : Effect E) : E) = 1 - e
```

2.83 Physlib.ProbabilisticTheory.Effect.Metric

Physlib/ProbabilisticTheory/Effect/Metric.lean on GitHub.

def effectEquiv [source]

Effects correspond to points of the order-unit-norm ball by doubling and re-centering at the order unit: `e ↦ 2 • e - 1`, with inverse `A ↦ (1 + A) / 2`.

```
noncomputable def effectEquiv : Effect E ≃ {A : E // orderUnitNorm A ≤ 1} where
```

2.84 Physlib.ProbabilisticTheory.LocalNet.Basic

Physlib/ProbabilisticTheory/LocalNet/Basic.lean on GitHub.

structure LocalNet [source]

The observables of each subsystem, with channels into larger subsystems.

```
@[ext]
structure LocalNet (R : Type*) [Preorder R] (A : R → Type*)
    [∀ X, OrderUnitSpace (A X)] where
  incl : ∀ {X Y : R}, X ≤ Y → A X →ₚ₁[ℝ] A Y
  incl_refl : ∀ X : R, incl (le_refl X) = .id ℝ (A X)
  incl_trans : ∀ {X Y Z : R} (h₁ : X ≤ Y) (h₂ : Y ≤ Z) (h₃ : X ≤ Z),
    incl h₃ = (incl h₂).comp (incl h₁)
```

lemma incl_trans_apply [source]

Including an observable through an intermediate subsystem agrees with direct inclusion.

```
lemma incl_trans_apply (h₁ : X ≤ Y) (h₂ : Y ≤ Z) (h₃ : X ≤ Z) (a : A X) :
    N.incl h₃ a = N.incl h₂ (N.incl h₁ a)
```

class IsOrderEmbedding [source]

The inclusion maps are order embeddings if each reflects positivity.

```
class IsOrderEmbedding (N : LocalNet R A) : Prop where
  nonneg_of_incl_nonneg : ∀ {X Y : R} (h : X ≤ Y) {a : A X}, 0 ≤ N.incl h a → 0 ≤ a
```

lemma map_le_map_iff [source]

Inclusion in an order-embedding net preserves and reflects inequalities.

```
lemma map_le_map_iff [N.IsOrderEmbedding] (h : X ≤ Y) {a b : A X} :
    N.incl h a ≤ N.incl h b ↔ a ≤ b
```

lemma incl_injective [source]

The inclusion maps of an order-embedding net are injective.

```
lemma incl_injective [N.IsOrderEmbedding] (h : X ≤ Y) : Function.Injective (N.incl h)
```

abbrev inclLin [source]

The inclusions as linear maps: the transition maps of the direct limit defining `N.Quasilocal`.

```
abbrev inclLin (N : LocalNet R A) {X Y : R} (h : X ≤ Y) : A X →ₗ[ℝ] A Y
```

instance directedSystem [source]

The inclusions form a directed system: including a subsystem into itself does nothing, and successive inclusions compose.

```
instance directedSystem (N : LocalNet R A) : DirectedSystem A fun _ _ h => N.inclLin h where
```

def Quasilocal [source]

The algebraic direct limit of the local observable spaces.

```
def Quasilocal (N : LocalNet R A) : Type _
```

instance AddCommGroup N.Quasilocal [source]

Quasilocal observables add, as elements of a direct limit of vector spaces.

```
noncomputable instance : AddCommGroup N.Quasilocal
```

instance Module ℝ N.Quasilocal [source]

Quasilocal observables can be scaled by real numbers.

```
noncomputable instance : Module ℝ N.Quasilocal
```

def toQuasilocal [source]

Regard an observable of subsystem `i` as a quasilocal observable.

```
noncomputable def toQuasilocal (i : R) : A i →ₗ[ℝ] N.Quasilocal
```

lemma toQuasilocal_incl [source]

Including an observable in a larger subsystem does not change its quasilocal representation.

```
lemma toQuasilocal_incl {i j : R} (h : i ≤ j) (a : A i) :
    toQuasilocal N j (N.incl h a) = toQuasilocal N i a
```

lemma toQuasilocal_eq_of_incl_eq [source]

Observables that agree in a common larger subsystem represent the same quasilocal observable.

```
lemma toQuasilocal_eq_of_incl_eq {i j k : R} (hik : i ≤ k) (hjk : j ≤ k) {a : A i} {b : A j}
    (h : N.incl hik a = N.incl hjk b) : toQuasilocal N i a = toQuasilocal N j b
```

lemma exists_incl_eq_of_toQuasilocal_eq [source]

Observables representing the same quasilocal observable agree in a common larger subsystem.

```
lemma exists_incl_eq_of_toQuasilocal_eq {i j : R} {a : A i} {b : A j}
    (h : toQuasilocal N i a = toQuasilocal N j b) :
    ∃ (k : R) (hik : i ≤ k) (hjk : j ≤ k), N.incl hik a = N.incl hjk b
```

lemma toQuasilocal_eq_iff [source]

Two local observables represent the same quasilocal observable exactly when they agree in a common larger subsystem.

```
lemma toQuasilocal_eq_iff {i j : R} (a : A i) (b : A j) :
    toQuasilocal N i a = toQuasilocal N j b ↔
      ∃ (k : R) (hik : i ≤ k) (hjk : j ≤ k), N.incl hik a = N.incl hjk b
```

lemma toQuasilocal_injective [source]

Order-embedding inclusions keep distinct local observables distinct in the quasilocal space.

```
lemma toQuasilocal_injective [N.IsOrderEmbedding] (i : R) :
    Function.Injective (toQuasilocal N i)
```

lemma exists_toQuasilocal_eq [source]

Every quasilocal observable has a representative in some subsystem.

```
lemma exists_toQuasilocal_eq (x : N.Quasilocal) : ∃ (i : R) (a : A i), toQuasilocal N i a = x
```

lemma exists_toQuasilocal_eq₂ [source]

Any two quasilocal observables have representatives in a common subsystem.

```
lemma exists_toQuasilocal_eq₂ (x y : N.Quasilocal) :
    ∃ (i : R) (a b : A i), toQuasilocal N i a = x ∧ toQuasilocal N i b = y
```

2.85 Physlib.ProbabilisticTheory.LocalNet.QuasilocalOrder

Physlib/ProbabilisticTheory/LocalNet/QuasilocalOrder.lean on GitHub.

def positive [source]

Global observables with a positive local representative.

```
def positive : Set N.Quasilocal
```

lemma toQuasilocal_mem_pos [source]

A positive local observable is positive globally.

```
lemma toQuasilocal_mem_pos {i : R} {a : A i} (ha : 0 ≤ a) :
    toQuasilocal N i a ∈ positive N
```

lemma smul_mem_pos [source]

Nonnegative scaling preserves global positivity.

```
lemma smul_mem_pos {c : ℝ} (hc : 0 ≤ c) {x : N.Quasilocal}
    (hx : x ∈ positive N) : c • x ∈ positive N
```

lemma add_mem_pos [source]

The sum of positive global observables is positive.

```
lemma add_mem_pos {x y : N.Quasilocal} (hx : x ∈ positive N) (hy : y ∈ positive N) :
    x + y ∈ positive N
```

lemma eq_zero_of_mem_pos_of_neg_mem_pos [source]

A global observable that is both positive and negative is zero.

```
lemma eq_zero_of_mem_pos_of_neg_mem_pos {x : N.Quasilocal}
    (hx : x ∈ positive N) (hx' : -x ∈ positive N) : x = 0
```

instance One N.Quasilocal [source]

The common global image of the local units.

```
noncomputable instance : One N.Quasilocal
```

lemma one_eq_toQuasilocal [source]

The global unit is represented by the unit of the chosen subsystem.

```
lemma one_eq_toQuasilocal :
    (1 : N.Quasilocal) = toQuasilocal N (Classical.arbitrary R) 1
```

lemma toQuasilocal_one [source]

Every local unit represents the global unit.

```
lemma toQuasilocal_one (i : R) : toQuasilocal N i 1 = (1 : N.Quasilocal)
```

def posCone [source]

The cone of positive global observables.

```
noncomputable def posCone : AddGroupCone N.Quasilocal where
```

instance PartialOrder N.Quasilocal [source]

The order on global observables induced by local positivity.

```
noncomputable instance : PartialOrder N.Quasilocal
```

lemma le_iff [source]

$x \le y$ exactly when $y - x$ has a positive local representative.

```
lemma le_iff {x y : N.Quasilocal} : x ≤ y ↔ y - x ∈ positive N
```

lemma nonneg_iff [source]

A global observable is nonnegative exactly when it belongs to `positive N`.

```
lemma nonneg_iff {x : N.Quasilocal} : 0 ≤ x ↔ x ∈ positive N
```

lemma toQuasilocal_nonneg_iff [source]

A local observable is positive globally exactly when it is positive locally.

```
lemma toQuasilocal_nonneg_iff (i : R) (a : A i) : 0 ≤ toQuasilocal N i a ↔ 0 ≤ a
```

instance IsOrderedAddMonoid N.Quasilocal [source]

Adding the same global observable preserves a comparison.

```
instance : IsOrderedAddMonoid N.Quasilocal
```

instance PosSMulMono ℝ N.Quasilocal [source]

Nonnegative scaling preserves comparisons between global observables.

```
instance : PosSMulMono ℝ N.Quasilocal where
```

instance OrderedVectorSpace N.Quasilocal [source]

Global observables form an ordered real vector space.

```
noncomputable instance : OrderedVectorSpace N.Quasilocal where
```

lemma toQuasilocal_le_iff [source]

A comparison of local observables has the same meaning globally.

```
lemma toQuasilocal_le_iff (i : R) (a b : A i) :
    toQuasilocal N i a ≤ toQuasilocal N i b ↔ a ≤ b
```

instance OrderUnitSpace N.Quasilocal [source]

The global observable space inherits an order unit from its subsystems.

```
noncomputable instance : OrderUnitSpace N.Quasilocal where
```

def toQuasilocalChannel [source]

Regard a subsystem observable as a global observable by a channel.

```
noncomputable def toQuasilocalChannel (i : R) : A i →ₚ₁[ℝ] N.Quasilocal
```

instance ArchimedeanOrderUnitSpace N.Quasilocal [source]

The global observable space is Archimedean when every subsystem is Archimedean.

```
noncomputable instance : ArchimedeanOrderUnitSpace N.Quasilocal where
```

2.86 Physlib.ProbabilisticTheory.State.Basic

Physlib/ProbabilisticTheory/State/Basic.lean on GitHub.

def comap [source]

The state on `E` that evaluates an observable by first sending it through a channel `f` and then evaluating the state `ω` on the result.

```
def comap (f : E →ₚ₁[ℝ] F) (ω : 𝒮[ℝ, F]) : 𝒮[ℝ, E] where
```

lemma comap_apply [source]

```
@[simp]
lemma comap_apply (f : E →ₚ₁[ℝ] F) (ω : 𝒮[ℝ, F]) (x : E) : comap f ω x = ω (f x)
```

2.87 `Physlib.ProbabilisticTheory.State.Convex`

`Physlib/ProbabilisticTheory/State/Convex.lean` on GitHub.

`def mix` [source]

Randomize between two states with probability `t` of choosing the first, so `t • ω + σ t • φ` with `σ t = 1 - t`.

```
def mix (ω φ : 𝒮[ℝ, E]) (t : unitInterval) : 𝒮[ℝ, E]
```

`lemma mix_apply` [source]

Evaluation of a mixture is the pointwise convex combination.

```
@[simp]
lemma mix_apply (ω φ : 𝒮[ℝ, E]) (t : unitInterval) (A : E) :
    mix ω φ t A = (t : ℝ) * ω A + (1 - (t : ℝ)) * φ A
```

`lemma toLinearMap_mix` [source]

The underlying linear map of a mixture is the corresponding combination of linear maps.

```
lemma toLinearMap_mix (ω φ : 𝒮[ℝ, E]) (t : unitInterval) :
    (mix ω φ t).toLinearMap = (t : ℝ) • ω.toLinearMap + (1 - (t : ℝ)) • φ.toLinearMap
```

`lemma mix_symm` [source]

Swapping the two states swaps the weights: `σ` is the symmetry of the interval.

```
lemma mix_symm (ω φ : 𝒮[ℝ, E]) (t : unitInterval) : mix ω φ t = mix φ ω (σ t)
```

`def stateSpace` [source]

States embedded into the algebraic dual.

```
def stateSpace : Set (E →ₗ[ℝ] ℝ)
```

`lemma stateSpace_convex` [source]

The state space is convex in the algebraic dual.

```
lemma stateSpace_convex : Convex ℝ (stateSpace (E := E))
```

`def IsPure` [source]

A state is pure when it is an extreme point of the state space.

```
def IsPure (ω : 𝒮[ℝ, E]) : Prop
```

def IsMixed [source]

A state is mixed when it isn't pure.

```
def IsMixed (ω : 𝒮[ℝ, E]) : Prop
```

lemma isPure_iff_forall_mem_openSegment [source]

A state is pure exactly when any open segment containing it has both endpoints equal to it.

```
lemma isPure_iff_forall_mem_openSegment {ω : 𝒮[ℝ, E]} :
    ω.IsPure ↔ ∀ φ ψ : 𝒮[ℝ, E],
      ω.toLinearMap ∈ openSegment ℝ φ.toLinearMap ψ.toLinearMap → φ = ω ∧ ψ = ω
```

lemma isMixed_iff_exists_mem_openSegment [source]

A state is mixed exactly when it lies in an open segment with a different endpoint.

```
lemma isMixed_iff_exists_mem_openSegment {ω : 𝒮[ℝ, E]} :
    ω.IsMixed ↔ ∃ φ ψ : 𝒮[ℝ, E],
      ω.toLinearMap ∈ openSegment ℝ φ.toLinearMap ψ.toLinearMap ∧ (φ ≠ ω ∨ ψ ≠ ω)
```

lemma mem_openSegment_iff_exists_mix [source]

Open segments between states consist exactly of their genuine mixtures.

```
lemma mem_openSegment_iff_exists_mix (ω φ ψ : 𝒮[ℝ, E]) :
    ω.toLinearMap ∈ openSegment ℝ φ.toLinearMap ψ.toLinearMap ↔
      ∃ t : unitInterval, 0 < t ∧ t < 1 ∧ mix φ ψ t = ω
```

lemma isPure_iff_forall_mix_eq [source]

A state is pure exactly when every genuine binary mixture producing it is trivial.

```
lemma isPure_iff_forall_mix_eq {ω : 𝒮[ℝ, E]} :
    ω.IsPure ↔ ∀ φ ψ t, 0 < t → t < 1 → mix φ ψ t = ω → φ = ω ∧ ψ = ω
```

2.88 Physlib.ProbabilisticTheory.State.Discrimination

Physlib/ProbabilisticTheory/State/Discrimination.lean on GitHub.

def successProb [source]

Probability of guessing right between $ω_0$ (prior `p`) and $ω_1$ (prior `1 - p`) using test `e`: guess $ω_0$ on a click, $ω_1$ otherwise.

```
def successProb (ω₀ ω₁ : 𝒮[ℝ, E]) (p : unitInterval) (e : Effect E) : ℝ
```

def advantage [source]

How much test `e` improves on the baseline of always guessing $ω_1$.

```
def advantage (ω₀ ω₁ : 𝒮[ℝ, E]) (p : unitInterval) (e : Effect E) : ℝ
```

lemma successProb_eq_add_advantage [source]

Success probability equals the baseline `1 - p` plus the advantage of test `e`.

```
lemma successProb_eq_add_advantage (ω₀ ω₁ : 𝒮[ℝ, E]) (p : unitInterval) (e : Effect E) :
    successProb ω₀ ω₁ p e = (1 - p) + advantage ω₀ ω₁ p e
```

lemma advantage_le [source]

No test's advantage beats the prior weight `p` of the state it favors.

```
lemma advantage_le (ω₀ ω₁ : 𝒮[ℝ, E]) (p : unitInterval) (e : Effect E) :
    advantage ω₀ ω₁ p e ≤ p
```

lemma bddAbove_advantage [source]

Advantages of tests are bounded above.

```
lemma bddAbove_advantage (ω₀ ω₁ : 𝒮[ℝ, E]) (p : unitInterval) :
    BddAbove (Set.range (advantage ω₀ ω₁ p))
```

def optimalSuccessProb [source]

The best a single test can do.

```
noncomputable def optimalSuccessProb (ω₀ ω₁ : 𝒮[ℝ, E]) (p : unitInterval) : ℝ
```

lemma optimalSuccessProb_eq [source]

The Helstrom bound: optimal success probability is the baseline `1 - p` plus the best advantage any test can give.

```
lemma optimalSuccessProb_eq (ω₀ ω₁ : 𝒮[ℝ, E]) (p : unitInterval) :
    optimalSuccessProb ω₀ ω₁ p = (1 - p) + ⨆ e : Effect E, advantage ω₀ ω₁ p e
```

lemma sub_complement_eq_neg_sub [source]

Complementing an effect negates `ω₀ e - ω₁ e`.

```
lemma sub_complement_eq_neg_sub (ω₀ ω₁ : 𝒮[ℝ, E]) (e : Effect E) :
    ω₀ (complement e) - ω₁ (complement e) = -(ω₀ e - ω₁ e)
```

lemma bddAbove_abs_sub [source]

The differences `|ω₀ e - ω₁ e|` over effects are bounded above.

```
lemma bddAbove_abs_sub (ω₀ ω₁ : 𝒮[ℝ, E]) :
    BddAbove (Set.range fun e : Effect E => |ω₀ e - ω₁ e|)
```

lemma bddAbove_sub [source]

The differences `ω₀ e - ω₁ e` over effects are bounded above.

```
lemma bddAbove_sub (ω₀ ω₁ : 𝒮[ℝ, E]) :
    BddAbove (Set.range fun e : Effect E => ω₀ e - ω₁ e)
```

lemma ciSup_sub_eq_ciSup_abs_sub [source]

The supremum of the state-value difference equals that of its absolute value: complementing an effect flips the sign.

```
lemma ciSup_sub_eq_ciSup_abs_sub (ω₀ ω₁ : 𝒮[ℝ, E]) :
    (⨆ e : Effect E, (ω₀ e - ω₁ e)) = ⨆ e : Effect E, |ω₀ e - ω₁ e|
```

lemma dist_eq_ciSup_abs_sub [source]

The state distance is the supremum of the differences on recentered effects.

```
lemma dist_eq_ciSup_abs_sub (ω₀ ω₁ : 𝒮[ℝ, E]) :
    dist ω₀ ω₁ =
      ⨆ e : Effect E, |ω₀ (effectEquiv e) - ω₁ (effectEquiv e)|
```

lemma dist_eq_two_mul_ciSup_sub [source]

The state distance is twice the largest state-value difference over all effects.

```
lemma dist_eq_two_mul_ciSup_sub (ω₀ ω₁ : 𝒮[ℝ, E]) :
    dist ω₀ ω₁ = 2 * ⨆ e : Effect E, (ω₀ e - ω₁ e)
```

lemma optimalSuccessProb_half_half_eq [source]

For equal priors, the Helstrom bound is `1/2` plus a quarter of the state distance.

```
lemma optimalSuccessProb_half_half_eq (ω₀ ω₁ : 𝒮[ℝ, E]) :
    optimalSuccessProb ω₀ ω₁ (1 / 2, by norm_num, by norm_num) = 1 / 2 + dist ω₀ ω₁ / 4
```

2.89 Physlib.ProbabilisticTheory.State.Local

Physlib/ProbabilisticTheory/State/Local.lean on GitHub.

def restrict [source]

Restrict a state of `Y` to `X` by pulling it back along the observable inclusion channel.

```
def restrict (h : X ≤ Y) (ω : 𝒮[ℝ, A Y]) : 𝒮[ℝ, A X]
```

lemma restrict_apply [source]

Restriction evaluates an observable through its inclusion into the larger subsystem.

```
@[simp]
lemma restrict_apply (h : X ≤ Y) (ω : 𝒮[ℝ, A Y]) (a : A X) :
    N.restrict h ω a = ω (N.incl h a)
```

lemma restrict_refl [source]

Restriction to the same subsystem is the identity.

```
@[simp]
lemma restrict_refl (ω : 𝒮[ℝ, A X]) : N.restrict (le_refl X) ω = ω
```

lemma restrict_restrict [source]

Successive restrictions agree with direct restriction.

```
lemma restrict_restrict (h₁ : X ≤ Y) (h₂ : Y ≤ Z) (h₃ : X ≤ Z) (ω : 𝒮[ℝ, A Z]) :
    N.restrict h₁ (N.restrict h₂ ω) = N.restrict h₃ ω
```

structure CompatibleStates [source]

A state on each subsystem whose restrictions agree with the states of smaller subsystems.

```
@[ext]
structure CompatibleStates (N : LocalNet R A) where
  state : ∀ X, 𝒮[ℝ, A X]
  compat : ∀ {X Y : R} (h : X ≤ Y), N.restrict h (state Y) = state X
```

lemma compat_apply [source]

An observable has the same expectation in every subsystem containing it.

```
lemma CompatibleStates.compat_apply {N : LocalNet R A} (ω : N.CompatibleStates)
    {X Y : R} (h : X ≤ Y) (a : A X) :
    ω.state Y (N.incl h a) = ω.state X a
```

instance DecidableEq R [source]

Decidable equality used internally by the direct-limit construction.

```
noncomputable local instance : DecidableEq R
```

def lift [source]

The expectation functional of `LocalNet.CompatibleStates` on the whole system.

```
noncomputable def lift (σ : N.CompatibleStates) : N.Quasilocal →ₗ[ℝ] ℝ
```

lemma lift_apply [source]

The expectation functional of `LocalNet.CompatibleStates` evaluates a quasilocal observable in any subsystem representing it.

```
lemma lift_apply (σ : N.CompatibleStates) (i : R) (a : A i) :
    lift σ (toQuasilocal N i a) = σ.state i a
```

def toState [source]

The state of the whole system defined by `LocalNet.CompatibleStates`.

```
noncomputable def toState (σ : N.CompatibleStates) : 𝒮[ℝ, N.Quasilocal]
```

lemma toState_apply [source]

On a quasilocal observable represented in the subsystem `i`, `toState σ` is the state of `i`.

```
lemma toState_apply (σ : N.CompatibleStates) (i : R) (a : A i) :
    toState σ (toQuasilocal N i a) = σ.state i a
```

def stateEquiv [source]

A state of the whole system is the same as `LocalNet.CompatibleStates`.

```
noncomputable def stateEquiv : 𝒮[ℝ, N.Quasilocal] ≃ N.CompatibleStates where
```

2.90 Physlib.ProbabilisticTheory.State.Metric

Physlib/ProbabilisticTheory/State/Metric.lean on GitHub.

lemma apply_le_one [source]

A state takes values at most `1` on observables bounded by the unit.

```
lemma apply_le_one {F : Type*} [OrderUnitSpace F] (ω : 𝒮[ℝ, F]) {A : F} (hA : A ≤ 1) :
    ω A ≤ 1
```

lemma apply_le_orderUnitNorm [source]

A state never overshoots the order-unit norm.

```
lemma apply_le_orderUnitNorm (ω : 𝒮[ℝ, E]) (A : E) : ω A ≤ orderUnitNorm A
```

lemma abs_apply_le_orderUnitNorm [source]

A state's values are bounded by the order-unit norm in both directions.

```
lemma abs_apply_le_orderUnitNorm (ω : 𝒮[ℝ, E]) (A : E) : |ω A| ≤ orderUnitNorm A
```

lemma abs_apply_sub_apply_le_two [source]

Two states' predictions on any observable of order-unit norm at most `1` never differ by more than `2`.

```
lemma abs_apply_sub_apply_le_two (ω φ : 𝒮[ℝ, E]) {A : E} (hA : orderUnitNorm A ≤ 1) :
    |ω A - φ A| ≤ 2
```

lemma dist_bddAbove [source]

The values `|ω A - φ A|` of two states, restricted to the order-unit-norm unit ball, are bounded by `2`.

```
lemma dist_bddAbove (ω φ : 𝒮[ℝ, E]) :
    BddAbove (Set.range fun A : {A : E // orderUnitNorm A ≤ 1} => |ω A - φ A|)
```

instance instNonemptyOrderUnitBall [source]

```
instance instNonemptyOrderUnitBall : Nonempty {A : E // orderUnitNorm A ≤ 1}
```

def dist [source]

The operator-norm distance between two states: how far apart their predictions can get on an observable of order-unit norm at most `1`.

```
noncomputable def dist (ω φ : 𝒮[ℝ, E]) : ℝ
```

lemma dist_nonneg [source]

Distances between states are nonnegative.

```
lemma dist_nonneg (ω φ : 𝒮[ℝ, E]) : 0 ≤ dist ω φ
```

lemma dist_le_two [source]

The whole state space has diameter at most `2`: it's a bounded metric space.

```
lemma dist_le_two (ω φ : 𝒮[ℝ, E]) : dist ω φ ≤ 2
```

lemma dist_self [source]

```
@[simp]
lemma dist_self (ω : 𝒮[ℝ, E]) : dist ω ω = 0
```

lemma dist_comm [source]

The state distance is symmetric.

```
lemma dist_comm (ω φ : 𝒮[ℝ, E]) : dist ω φ = dist φ ω
```

lemma dist_triangle [source]

The state distance satisfies the triangle inequality.

```
lemma dist_triangle (ω φ ψ : 𝒮[ℝ, E]) : dist ω ψ ≤ dist ω φ + dist φ ψ
```

lemma eq_of_dist_eq_zero [source]

`dist` separates states: two states at distance `0` are equal.

```
lemma eq_of_dist_eq_zero {ω φ : 𝒮[ℝ, E]} (h : dist ω φ = 0) : ω = φ
```

lemma apply_effectEquiv [source]

A state's value at a doubled, re-centered effect (`effectEquiv`) is twice its value at the effect, minus one.

```
lemma apply_effectEquiv (ψ : 𝒮[ℝ, E]) (e : Effect E) :
    ψ (effectEquiv e) = 2 * ψ e - 1
```

instance instMetricSpace [source]

States, metrized by the operator norm induced by the order-unit norm on `E`.

```
noncomputable instance instMetricSpace : MetricSpace (𝒮[ℝ, E]) where
```

def distToPure [source]

The infimum distance to the set of pure states, using `Metric.infDist`.

```
noncomputable def distToPure (ω : 𝒮[ℝ, E]) : ℝ
```

lemma distToPure_nonneg [source]

The distance to the set of pure states is nonnegative.

```
lemma distToPure_nonneg (ω : 𝒮[ℝ, E]) : 0 ≤ distToPure ω
```

lemma distToPure_eq_zero_of_isPure [source]

A pure state is at distance `0` from the set of pure states.

```
lemma distToPure_eq_zero_of_isPure {ω : 𝒮[ℝ, E]} (h : IsPure ω) : distToPure ω = 0
```

2.91 Physlib.ProbabilisticTheory.State.Pairing

Physlib/ProbabilisticTheory/State/Pairing.lean on GitHub.

lemma apply_mem_Icc [source]

State–effect evaluation takes values in `[0, 1]`.

```
lemma apply_mem_Icc (ω : 𝒮[ℝ, E]) (e : Effect E) : ω e ∈ Set.Icc 0 1
```

lemma apply_mix [source]

Evaluation is affine in the effect argument.

```
lemma apply_mix (ω : 𝒮[ℝ, E]) (e f : Effect E) (t : unitInterval) :
    ω (Effect.mix e f t) = (t : ℝ) * ω e + (1 - (t : ℝ)) * ω f
```

lemma ext_of_effect_eq_of_nonneg [source]

Two states that agree on all effects agree on every nonnegative element.

```
lemma ext_of_effect_eq_of_nonneg {ω φ : 𝒮[ℝ, E]}
    (h : ∀ e : Effect E, ω e = φ e) {B : E} (hB : 0 ≤ B) : ω B = φ B
```

lemma ext_of_effect_eq [source]

A state is determined by its values on effects.

```
lemma ext_of_effect_eq {ω φ : 𝒮[ℝ, E]}
    (h : ∀ e : Effect E, ω e = φ e) : ω = φ
```

lemma injective_apply_effect [source]

Evaluation on effects is injective on states.

```
lemma injective_apply_effect :
    Function.Injective (fun (ω : 𝓢[ℝ, E]) (e : Effect E) => ω e)
```

lemma dist_eq_sSup_abs_apply [source]

The distance between two effects is the supremum of the difference in their evaluations over all states.

```
lemma dist_eq_sSup_abs_apply [Nontrivial E] (e f : Effect E) :
    Dist.dist e f = sSup (Set.range fun ω : 𝓢[ℝ, E] => |ω e - ω f|)
```

lemma ext_of_forall_apply_eq [source]

An effect is determined by its values under all states.

```
lemma ext_of_forall_apply_eq {e f : Effect E}
    (h : ∀ ω : 𝓢[ℝ, E], ω e = ω f) : e = f
```

lemma injective_apply_state [source]

Evaluation by states is injective on effects.

```
lemma injective_apply_state :
    Function.Injective (fun (e : Effect E) (ω : 𝓢[ℝ, E]) => ω e)
```

2.92 Physlib.ProbabilisticTheory.State.Separation

Physlib/ProbabilisticTheory/State/Separation.lean on GitHub.

lemma apply_eq_zero_of_apply_one_eq_zero [source]

A positive functional vanishing at the order unit vanishes everywhere.

```
lemma apply_eq_zero_of_apply_one_eq_zero [OrderUnitSpace E] {p : E →ₚ[ℝ] ℝ}
    (h1 : p (1 : E) = 0) (A : E) : p A = 0
```

lemma exists_apply_neg_of_not_nonneg [source]

Every element outside the positive cone is strictly separated from it by a state.

```
lemma exists_apply_neg_of_not_nonneg {A : E} (hA : ¬ 0 ≤ A) : ∃ ω : 𝓢[ℝ, E], ω A < 0
```

lemma nonneg_iff_forall_state_nonneg [source]

An element is positive iff every state assigns it a nonnegative value.

```
lemma nonneg_iff_forall_state_nonneg (A : E) : 0 ≤ A ↔ ∀ ω : 𝓢[ℝ, E], 0 ≤ ω A
```

instance instNonemptyState [source]

Every nontrivial Archimedean order-unit space has a state.

```
instance instNonemptyState [Nontrivial E] : Nonempty (𝒮[ℝ, E])
```

lemma orderUnitNorm_le_iff_forall_abs_apply_le [source]

The order-unit norm is at most `r` iff every state predicts an absolute value at most `r`.

```
lemma orderUnitNorm_le_iff_forall_abs_apply_le [Nontrivial E] (A : E) (r : ℝ) :
    orderUnitNorm A ≤ r ↔ ∀ ω : 𝒮[ℝ, E], |ω A| ≤ r
```

lemma sSup_abs_apply_eq_orderUnitNorm [source]

The order-unit norm is the supremum of `|ω A|` over all states `ω`.

```
lemma sSup_abs_apply_eq_orderUnitNorm [Nontrivial E] (A : E) :
    sSup (Set.range fun ω : 𝒮[ℝ, E] => |ω A|) = orderUnitNorm A
```

lemma orderUnitNorm_one [source]

The order unit has norm exactly `1`.

```
@[simp]
lemma orderUnitNorm_one [Nontrivial E] : orderUnitNorm (1 : E) = 1
```

lemma ext_of_forall_apply_eq [source]

States separate points: if `ω A = ω B` for every state `ω`, then `A = B`.

```
lemma ext_of_forall_apply_eq {A B : E} (h : ∀ ω : 𝒮[ℝ, E], ω A = ω B) : A = B
```

2.93 Physlib.ProbabilisticTheory.State.WeightEquivalence

Physlib/ProbabilisticTheory/State/WeightEquivalence.lean on GitHub.

def toState [source]

A finite normalized weight extends to a state.

```
noncomputable def toState (hw : w.IsState) : 𝒮[ℝ, E]
```

lemma toState_apply [source]

The extended state agrees with the weight on positive observables.

```
@[simp]
lemma toState_apply (hw : w.IsState) (A : PosCone E) : hw.toState A = (w A).toReal
```

def toWeight [source]

The weight induced by a state: `ENNReal.ofReal` applied to the state's values on the positive cone, where they are automatically nonnegative.

```
noncomputable def toWeight (s : 𝒮[ℝ, E]) : Weight E where
```

lemma toWeight_apply [source]

```
@[simp]
lemma toWeight_apply (s : 𝒮[ℝ, E]) (A : PosCone E) : s.toWeight A = ENNReal.ofReal (s (A : E))
```

lemma toReal_toWeight_apply [source]

The weight induced by a state agrees with the state itself on the positive cone.

```
lemma toReal_toWeight_apply (s : 𝒮[ℝ, E]) (A : PosCone E) :
    (s.toWeight A).toReal = s (A : E)
```

lemma toWeight_isState [source]

The weight induced by a state is finite and normalized.

```
lemma toWeight_isState (s : 𝒮[ℝ, E]) : s.toWeight.IsState where
```

def stateEquiv [source]

Finite normalized weights correspond exactly to states.

```
noncomputable def stateEquiv : {w : Weight E // w.IsState} ≃ 𝒮[ℝ, E] where
```

2.94 Physlib.ProbabilisticTheory.Weight.Basic

Physlib/ProbabilisticTheory/Weight/Basic.lean on GitHub.

abbrev Weight [source]

An extended nonnegative linear functional on the positive cone.

```
abbrev Weight (E : Type*) [OrderedVectorSpace E]
```

def IsFaithful [source]

Only the zero positive element has weight zero.

```
def IsFaithful (w : Weight E) : Prop
```

def IsFinite [source]

A weight has no infinite values.

```
def IsFinite (w : Weight E) : Prop
```

def IsSemifinite [source]

A weight is the supremum of its finite values below each positive element.

```
def IsSemifinite (w : Weight E) : Prop
```

lemma monotone [source]

Weights are monotone on the positive cone.

```
lemma monotone (w : Weight E) : Monotone w
```

lemma isSemifinite [source]

A finite weight is semifinite.

```
lemma IsFinite.isSemifinite {w : Weight E} (hw : w.IsFinite) : w.IsSemifinite
```

lemma toReal_map_add [source]

A finite weight's real value is additive.

```
lemma IsFinite.toReal_map_add {w : Weight E} (hw : w.IsFinite) (A B : PosCone E) :
    (w (A + B)).toReal = (w A).toReal + (w B).toReal
```

lemma toReal_map_nnreal_smul [source]

A weight's real value scales linearly under nonnegative real scaling.

```
lemma toReal_map_nnreal_smul (w : Weight E) (k : ℝ≥0) (A : PosCone E) :
    (w (k • A)).toReal = k * (w A).toReal
```

def IsNormal [source]

A weight is normal when it preserves least upper bounds of increasing sequences in the positive cone.

```
def IsNormal (w : Weight E) : Prop
```

structure IsState [source]

A weight that's finite everywhere and gives the certain outcome weight exactly `1`.

```
structure IsState (w : Weight E) : Prop where
  finite : w.IsFinite
  normalized : w 1 = 1
```

def normalize [source]

Rescale a weight by the inverse of its value at the order unit.

```
noncomputable def normalize (w : Weight E) : Weight E where
```

lemma normalize_isFinite [source]

Normalizing a finite weight that's nonzero at the order unit keeps it finite.

```
lemma IsFinite.normalize_isFinite {w : Weight E} (hw : w.IsFinite) (h : w 1 ≠ 0) :
    (normalize w).IsFinite
```

lemma normalize_isState [source]

Normalizing a finite weight that's nonzero at the order unit makes it a state: the order unit is scaled to weight exactly `1`.

```
lemma IsFinite.normalize_isState {w : Weight E} (hw : w.IsFinite) (h : w 1 ≠ 0) :
    (normalize w).IsState where
```

2.95 Physlib.ProbabilisticTheory.Weight.Extension

Physlib/ProbabilisticTheory/Weight/Extension.lean on GitHub.

lemma coe_nnreal_smul [source]

```
@[simp]
lemma coe_nnreal_smul (t : ℝ≥0) (P : PosCone E) : ((t • P : PosCone E) : E) = (t : ℝ) • (P : E)
```

lemma map_sub_eq_map_sub [source]

`P - Q ↦ f P - f Q` does not depend on how an element is written as a difference.

```
lemma map_sub_eq_map_sub {P Q P' Q' : PosCone E} (h : (P : E) - Q = P' - Q') :
    f P - f Q = f P' - f Q'
```

lemma exists_sub [source]

In an order unit space, every element is a difference of positive elements.

```
lemma exists_sub (A : E) : ∃ P Q : PosCone E, (P : E) - Q = A
```

def extendFun [source]

The value of the extension, computed from a chosen decomposition `A = P - Q`.

```
noncomputable def extendFun (A : E) : ℝ
```

lemma extendFun_eq [source]

```
lemma extendFun_eq {A : E} {P Q : PosCone E} (h : (P : E) - Q = A) :
    extendFun f A = f P - f Q
```

def extend [source]

The `ℝ`-linear extension of an `ℝ≥0`-linear map on an order unit space's positive cone.

```
noncomputable def extend : E →ₗ[ℝ] ℝ where
```

lemma extend_apply [source]

```
@[simp]
lemma extend_apply {P Q : PosCone E} : extend f ((P : E) - Q) = f P - f Q
```

lemma extend_coe [source]

```
@[simp]
lemma extend_coe (P : PosCone E) : extend f (P : E) = f P
```

def toReal [source]

A finite weight, read as a real-valued $\mathbb{R}_{\geq 0}$-linear map on the positive cone.

```
noncomputable def IsFinite.toReal (hw : w.IsFinite) : PosCone E →ₗ[ℝ≥0] ℝ where
```

lemma toReal_apply [source]

```
@[simp]
lemma IsFinite.toReal_apply (hw : w.IsFinite) (A : PosCone E) : hw.toReal A = (w A).toReal
```

def toLinearMap [source]

The $\mathbb{R}$-linear map extending a finite weight.

```
noncomputable def IsFinite.toLinearMap (hw : w.IsFinite) : E →ₗ[ℝ] ℝ
```

lemma toLinearMap_coe [source]

```
@[simp]
lemma IsFinite.toLinearMap_coe (hw : w.IsFinite) (A : PosCone E) :
    hw.toLinearMap (A : E) = (w A).toReal
```

2.96 Physlib.QuantumMechanics.Blackbody.PlancksLaw

Physlib/QuantumMechanics/Blackbody/PlancksLaw.lean on GitHub.

structure BlackBody [source]

An idealized black body in thermal equilibrium at temperature `T`.

```
structure BlackBody where
  T : Temperature
```

def spectralRadiance [source]

The spectral radiance per unit frequency of blackbody radiation at frequency `ν` for a black body `B` and speed of light `c`:

`B(ν, T) = 2 h ν³ / c² · 1 / (e^{h ν / (k_B T)} - 1)`

By the homogeneity and isotropy of blackbody radiation, the spectral radiance is independent of position and direction, so it depends only on frequency and temperature.

Extended by zero outside the physical domain `0 < ν`.

```
noncomputable def spectralRadiance (B : BlackBody) (c : SpeedOfLight) (ν : Frequency) : ℝ
```

lemma spectralRadiance_pos [source]

The spectral radiance of blackbody radiation is positive for positive frequency and positive temperature.

```
lemma spectralRadiance_pos (B : BlackBody) (c : SpeedOfLight) (ν : Frequency)
    (hν : 0 < ν.val) (hT : 0 < (B.T : ℝ)) : 0 < B.spectralRadiance c ν
```

lemma spectralRadiance_nonneg [source]

The spectral radiance per unit frequency is non-negative on the physical domain.

```
lemma spectralRadiance_nonneg (B : BlackBody) (c : SpeedOfLight) (ν : Frequency)
    (hν : 0 < ν.val) (hT : 0 < (B.T : ℝ)) : 0 ≤ B.spectralRadiance c ν
```

lemma spectralRadiance_absZero [source]

The spectral radiance vanishes at absolute zero temperature.

```
lemma spectralRadiance_absZero (c : SpeedOfLight) (ν : Frequency) :
    spectralRadiance ⟨0⟩ c ν = 0
```

lemma spectralRadiance_zeroFreq [source]

The spectral radiance vanishes at zero frequency.

```
lemma spectralRadiance_zeroFreq (B : BlackBody) (c : SpeedOfLight) :
    B.spectralRadiance c 0 = 0
```

lemma spectralRadiance_eq_zero_of_nonpos_freq [source]

The spectral radiance vanishes when frequency is non-positive.

```
lemma spectralRadiance_eq_zero_of_nonpos_freq (B : BlackBody) (c : SpeedOfLight) (ν : Frequency
    )
    (hν : ν.val ≤ 0) : B.spectralRadiance c ν = 0
```

def spectralRadianceWave [source]

Spectral radiance per unit wavelength of blackbody radiation at wavelength `λ` for a black body `B` and speed of light `c`:

`B(λ, T) = 2 h c² / λ⁵ · 1 / (e ^ (h c / (λ kB T)) - 1)`,

extended by zero outside the physical domain `0 < λ`.

```
noncomputable def spectralRadianceWave (B : BlackBody) (c : SpeedOfLight) (lam : Wavelength) :
    ℝ
```

lemma spectralRadianceWave_pos [source]

The spectral radiance per unit wavelength is positive for positive wavelength and positive temperature.

```
lemma spectralRadianceWave_pos (B : BlackBody) (c : SpeedOfLight) (lam : Wavelength)
    (hlam : 0 < lam.val) (hT : 0 < (B.T : ℝ)) : 0 < B.spectralRadianceWave c lam
```

lemma spectralRadianceWave_nonneg [source]

The spectral radiance per unit wavelength is non-negative on the physical domain.

```
lemma spectralRadianceWave_nonneg (B : BlackBody) (c : SpeedOfLight) (lam : Wavelength)
    (hlam : 0 < lam.val) (hT : 0 < (B.T : ℝ)) : 0 ≤ B.spectralRadianceWave c lam
```

lemma spectralRadianceWave_absZero [source]

The spectral radiance per unit wavelength vanishes at absolute zero.

```
lemma spectralRadianceWave_absZero (c : SpeedOfLight) (lam : Wavelength) :
    spectralRadianceWave ⟨0⟩ c lam = 0
```

lemma spectralRadianceWave_zeroWave [source]

The spectral radiance per unit wavelength vanishes at zero wavelength.

```
lemma spectralRadianceWave_zeroWave (B : BlackBody) (c : SpeedOfLight) :
    B.spectralRadianceWave c 0 = 0
```

lemma spectralRadianceWave_eq_zero_of_nonpos_wave [source]

The spectral radiance per unit wavelength vanishes when wavelength is non-positive.

```
lemma spectralRadianceWave_eq_zero_of_nonpos_wave (B : BlackBody) (c : SpeedOfLight)
    (lam : Wavelength) (hlam : lam.val ≤ 0) : B.spectralRadianceWave c lam = 0
```

lemma spectralRadianceWave_eq_spectralRadiance [source]

Correspondence between the wavelength and frequency forms of Planck's law: `B(λ, T) = (c / λ²) B(ν = c / λ, T)`, where the frequency `c / λ` is given by `Frequency.ofWavelength`.

```
lemma spectralRadianceWave_eq_spectralRadiance (B : BlackBody) (c : SpeedOfLight)
    (lam : Wavelength) (hlam : 0 < lam.val) (hT : 0 < (B.T : ℝ)) :
    B.spectralRadianceWave c lam
      = ((c : ℝ) / (lam.val : ℝ) ^ 2) * B.spectralRadiance c
          (Frequency.ofWavelength c.toWithDim lam hlam)
```

def firstRadiationConstant [source]

The first radiation constant `c₁L = 2 h c²`.

```
noncomputable def firstRadiationConstant (c : SpeedOfLight) : ℝ
```

lemma firstRadiationConstant_pos [source]

The first radiation constant is positive.

```
lemma firstRadiationConstant_pos (c : SpeedOfLight) :
    0 < firstRadiationConstant c
```

lemma firstRadiationConstant_eq [source]

The first radiation constant equals `2 * h * c ^ 2`.

```
lemma firstRadiationConstant_eq (c : SpeedOfLight) :
    firstRadiationConstant c = 2 * h * (c : ℝ) ^ 2
```

def secondRadiationConstant [source]

The second radiation constant `c₂ = h c / kB`.

```
noncomputable def secondRadiationConstant (c : SpeedOfLight) : ℝ
```

lemma secondRadiationConstant_pos [source]

The second radiation constant is positive.

```
lemma secondRadiationConstant_pos (c : SpeedOfLight) :
    0 < secondRadiationConstant c
```

lemma secondRadiationConstant_eq [source]

The second radiation constant equals `h * c / kB`.

```
lemma secondRadiationConstant_eq (c : SpeedOfLight) :
    secondRadiationConstant c = (h : ℝ) * (c : ℝ) / kB
```

lemma spectralRadianceWave_eq_constants [source]

Planck's law per unit wavelength in terms of the radiation constants: `B(λ, T) = (c₁L / λ⁵) / (e ^ (c₂ / (λ T)) - 1)`.

```
lemma spectralRadianceWave_eq_constants (B : BlackBody) (c : SpeedOfLight) (lam : Wavelength)
    (hlam : 0 < lam.val) (hT : 0 < (B.T : ℝ)) :
    B.spectralRadianceWave c lam
      = firstRadiationConstant c / (lam.val : ℝ) ^ 5
        / (Real.exp (secondRadiationConstant c / ((lam.val : ℝ) * (B.T : ℝ))) - 1)
```

2.97 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.MomentumStates

Physlib/QuantumMechanics/HilbertSpaces/SpaceD/MomentumStates.lean on GitHub.

lemma momentumState_comp_momentumCLM [source]

The momentum state `momentumState p` is a generalized eigenvector of `𝐩 i` with eigenvalue `ℏ pᵢ`, in the bra form `⟨p| 𝐩ᵢ = ℏ pᵢ ⟨p|` as functionals on Schwartz space. Since `𝐩 i` is symmetric and `ℏ pᵢ` is real, this is equivalent to the ket form `𝐩ᵢ |p⟩ = ℏ pᵢ |p⟩`.

```
lemma momentumState_comp_momentumCLM (p : Space d) (i : Fin d) :
    momentumState p ∘L 𝐩 i = (ℏ * p i : ℂ) • momentumState p
```

2.98 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.PositionStates

Physlib/QuantumMechanics/HilbertSpaces/SpaceD/PositionStates.lean on GitHub.

lemma positionState_comp_positionCLM [source]

The position state `positionState x` is a generalized eigenvector of `𝐱 i` with eigenvalue x_i, in the bra form `⟨x| 𝐱ᵢ = xᵢ ⟨x|` as functionals on Schwartz space. Since `𝐱 i` is symmetric and x_i is real, this is equivalent to the ket form `𝐱ᵢ |x⟩ = xᵢ |x⟩`.

```
lemma positionState_comp_positionCLM (x : Space d) (i : Fin d) :
    positionState x ∘L 𝐱 i = (x i : ℂ) • positionState x
```

2.99 Physlib.QuantumMechanics.HilbertSpaces.TensorProducts.CompleteTensorProduct

Physlib/QuantumMechanics/HilbertSpaces/TensorProducts/CompleteTensorProduct.lean on GitHub.

def tIncl₃ₗᵢ [source]

The embedding of the algebraic threefold tensor product `E ⊗[𝕜] F ⊗[𝕜] G` into the complete tensor product `E ⊗ₕ[𝕜] F ⊗ₕ[𝕜] G` as a linear isometry: `E ⊗[𝕜] F` is first embedded into `E ⊗ₕ[𝕜] F`, then `(E ⊗ₕ[𝕜] F) ⊗[𝕜] G` is embedded into its completion.

```
def tIncl₃ₗᵢ : E ⊗[𝕜] F ⊗[𝕜] G →ₗᵢ[𝕜] E ⊗ₕ[𝕜] F ⊗ₕ[𝕜] G
```

def tIncl₃'ₗᵢ [source]

The embedding of the algebraic threefold tensor product `E ⊗[𝕜] (F ⊗[𝕜] G)` into the complete tensor product `E ⊗ₕ[𝕜] (F ⊗ₕ[𝕜] G)` as a linear isometry: `F ⊗[𝕜] G` is first embedded into `F ⊗ₕ[𝕜] G`, then `E ⊗[𝕜] (F ⊗ₕ[𝕜] G)` is embedded into its completion.

```
def tIncl₃'ₗᵢ : E ⊗[𝕜] (F ⊗[𝕜] G) →ₗᵢ[𝕜] E ⊗ₕ[𝕜] (F ⊗ₕ[𝕜] G)
```

lemma tIncl₃ₗᵢ_tmul [source]

On a simple tensor `x ⊗ₜ y ⊗ₜ z`, the embedding `tIncl₃ₗᵢ` is the nested coercion.

```
@[simp]
lemma tIncl₃ₗᵢ_tmul (x : E) (y : F) (z : G) :
    tIncl₃ₗᵢ (x ⊗ₜ[𝕜] y ⊗ₜ[𝕜] z) = (x ⊗ₜ[𝕜] y ⊗ₜ[𝕜] z : (E ⊗ₕ[𝕜] F) ⊗[𝕜] G)
```

lemma tIncl₃'ₗᵢ_tmul [source]

On a simple tensor `x ⊗ₜ (y ⊗ₜ z)`, the embedding `tIncl₃'ₗᵢ` is the nested coercion.

```
@[simp]
lemma tIncl₃'ₗᵢ_tmul (x : E) (y : F) (z : G) :
    tIncl₃'ₗᵢ (x ⊗ₜ[𝕜] (y ⊗ₜ[𝕜] z)) = (x ⊗ₜ[𝕜] (y ⊗ₜ[𝕜] z) : E ⊗[𝕜] (F ⊗ₕ[𝕜] G))
```

lemma assoc_tIncl₃'ₗᵢ [source]

CompleteTensorProduct.assoc acting on an element of the algebraic threefold tensor product reduces to TensorProduct.assoc.

```
@[simp]
lemma assoc_tIncl₃'ₗᵢ (x : E ⊗[𝕜] F ⊗[𝕜] G) :
    assoc 𝕜 E F G (tIncl₃'ₗᵢ x) = tIncl₃'ₗᵢ (TensorProduct.assoc 𝕜 E F G x)
```

lemma assoc_symm_tIncl₃'ₗᵢ [source]

The inverse of CompleteTensorProduct.assoc acting on an element of the algebraic threefold tensor product reduces to the inverse of TensorProduct.assoc.

```
@[simp]
lemma assoc_symm_tIncl₃'ₗᵢ (x : E ⊗[𝕜] (F ⊗[𝕜] G)) :
    (assoc 𝕜 E F G).symm (tIncl₃'ₗᵢ x) = tIncl₃'ₗᵢ ((TensorProduct.assoc 𝕜 E F G).symm x)
```

lemma assoc_tmul [source]

CompleteTensorProduct.assoc on simple tensors (c.f. TensorProduct.assoc_tmul).

```
@[simp]
lemma assoc_tmul (x : E) (y : F) (z : G) :
    assoc 𝕜 E F G (x ⊗ₜ[𝕜] y ⊗ₜ[𝕜] z : (E ⊗ₕ[𝕜] F) ⊗[𝕜] G) =
      (x ⊗ₜ[𝕜] (y ⊗ₜ[𝕜] z) : E ⊗[𝕜] (F ⊗ₕ[𝕜] G))
```

lemma assoc_symm_tmul [source]

The inverse of CompleteTensorProduct.assoc on simple tensors (c.f. TensorProduct.assoc_symm_tmul).

```
@[simp]
lemma assoc_symm_tmul (x : E) (y : F) (z : G) :
    (assoc 𝕜 E F G).symm (x ⊗ₜ[𝕜] (y ⊗ₜ[𝕜] z) : E ⊗[𝕜] (F ⊗ₕ[𝕜] G)) =
      (x ⊗ₜ[𝕜] y ⊗ₜ[𝕜] z : (E ⊗ₕ[𝕜] F) ⊗[𝕜] G)
```

2.100 Physlib.QuantumMechanics.Hydrogen.Basic

Physlib/QuantumMechanics/Hydrogen/Basic.lean on GitHub.

lemma polyBddSchwartzSubmodule_le_hamiltonianOperator_domain [source]

*Polynomially-bounded Schwartz maps f with ‖x‖ ^ (-k) * ‖f x‖ ≤ Cₖ for all k ≤ ⌊2 - d/2⌋ lie in the domain of the Hamiltonian. For d ≥ 3 these are all Schwartz maps.*

```
lemma polyBddSchwartzSubmodule_le_hamiltonianOperator_domain :
    PolyBddSchwartzSubmodule d ⌊1 - d / 2 - (-1 : ℝ)⌋.toNat ≤
      H.hamiltonianOperator.domain
```

lemma hamiltonianOperator_hasDenseDomain [source]

The Hamiltonian of the hydrogen atom is densely defined, in any dimension.

```
lemma hamiltonianOperator_hasDenseDomain : H.hamiltonianOperator.HasDenseDomain
```

2.101 `Physlib.QuantumMechanics.Hydrogen.LaplaceRungeLenzVector`

Physlib/QuantumMechanics/Hydrogen/LaplaceRungeLenzVector.lean on GitHub.

lemma angularMomentum_cyclic_lrl [source]

$\mathbf{L}_{i\,j}\mathbf{A}(\varepsilon)_k + \mathbf{L}_{j\,k}\mathbf{A}(\varepsilon)_i + \mathbf{L}_{k\,i}\mathbf{A}(\varepsilon)_j = 0$; *in three dimensions this is* $\mathbf{L} \cdot \mathbf{A}(\varepsilon) = 0$. *The identity is trivial unless* `i`, `j`, `k` *are pairwise distinct, so it has no content in fewer than three dimensions.*

```
lemma angularMomentum_cyclic_lrl (ε : ℝˣ) (i j k : Fin d) :
    L i j ∘L H.lrlOperator ε k + L j k ∘L H.lrlOperator ε i + L k i ∘L H.lrlOperator ε j = 0
```

lemma lrl_cyclic_angularMomentum [source]

$\mathbf{A}(\varepsilon)_k\mathbf{L}_{i\,j} + \mathbf{A}(\varepsilon)_i\mathbf{L}_{j\,k} + \mathbf{A}(\varepsilon)_j\mathbf{L}_{k\,i} = 0$; *in three dimensions this is* $\mathbf{A}(\varepsilon) \cdot \mathbf{L} = 0$. *The identity is trivial unless* `i`, `j`, `k` *are pairwise distinct, so it has no content in fewer than three dimensions.*

```
lemma lrl_cyclic_angularMomentum (ε : ℝˣ) (i j k : Fin d) :
    H.lrlOperator ε k ∘L L i j + H.lrlOperator ε i ∘L L j k + H.lrlOperator ε j ∘L L k i = 0
```

2.102 `Physlib.QuantumMechanics.Operators.AngularMomentum`

Physlib/QuantumMechanics/Operators/AngularMomentum.lean on GitHub.

lemma angularMomentumCLM_inner [source]

The component symmetry identity, expressed directly on Schwartz maps.

```
lemma angularMomentumCLM_inner (f g : 𝒮(Space d, ℂ)) :
    ⟪(schwartzEquiv volume (L i j f) : SpaceDHilbertSpace d), schwartzEquiv volume g⟫_ℂ =
      ⟪(schwartzEquiv volume f : SpaceDHilbertSpace d), schwartzEquiv volume (L i j g)⟫_ℂ
```

def angularMomentumSqOperator [source]

Angular momentum squared on the Hilbert space, with the invariant Schwartz domain. This transports the existing Schwartz operator $\mathbf{L}^2$, *including its factor* `1/2`.

```
def angularMomentumSqOperator : SpaceDHilbertSpace d →ₗ.[ℂ] SpaceDHilbertSpace d where
```

lemma angularMomentumSqOperator_domain_eq [source]

```
lemma angularMomentumSqOperator_domain_eq :
    (angularMomentumSqOperator (d := d)).domain = SchwartzSubmodule d
```

lemma angularMomentumSqOperator_apply [source]

```
lemma angularMomentumSqOperator_apply (ψ : SchwartzSubmodule d) : angularMomentumSqOperator ψ =
    schwartzEquiv volume (L² ((schwartzEquiv volume).symm ψ))
```

lemma angularMomentumSqOperator_apply_ae [source]

```
lemma angularMomentumSqOperator_apply_ae (ψ : SchwartzSubmodule d) :
    angularMomentumSqOperator ψ =ᵐ[volume] 𝐋² ((schwartzEquiv volume).symm ψ)
```

lemma angularMomentumSqOperator_range [source]

```
lemma angularMomentumSqOperator_range (ψ : SchwartzSubmodule d) :
    angularMomentumSqOperator ψ ∈ SchwartzSubmodule d
```

lemma angularMomentumSqOperator_apply_eq_sum [source]

The squared operator is the sum of squares of Hilbert-space components on their common invariant domain. The inner applications land in Schwartz by `angularMomentumOperator_range`.

```
lemma angularMomentumSqOperator_apply_eq_sum (ψ : SchwartzSubmodule d) :
    angularMomentumSqOperator ψ = (2 : ℂ)⁻¹ • ∑ i, ∑ j,
      angularMomentumOperator i j
        (angularMomentumOperator i j ψ, angularMomentumOperator_range i j ψ)
```

lemma angularMomentumSqOperator_hasDenseDomain [source]

```
lemma angularMomentumSqOperator_hasDenseDomain :
    (angularMomentumSqOperator (d := d)).HasDenseDomain
```

lemma angularMomentumSqr_inner [source]

Symmetry of $\mathbf{L}^2$ *follows by moving each component across the inner product twice.*

```
lemma angularMomentumSqr_inner (f g : 𝓢(Space d, ℂ)) :
    ⟪(schwartzEquiv volume (𝐋² f) : SpaceDHilbertSpace d), schwartzEquiv volume g⟫_ℂ =
      ⟪(schwartzEquiv volume f : SpaceDHilbertSpace d), schwartzEquiv volume (𝐋² g)⟫_ℂ
```

lemma angularMomentumSqOperator_isSymmetric [source]

Angular momentum squared is symmetric on the Schwartz domain.

```
lemma angularMomentumSqOperator_isSymmetric : (angularMomentumSqOperator (d := d)).IsSymmetric
```

lemma angularMomentumSqOperator_isUnbounded [source]

In Physlib terminology, the squared operator is densely defined and closable.

```
lemma angularMomentumSqOperator_isUnbounded : (angularMomentumSqOperator (d := d)).IsUnbounded
```

lemma angularMomentum_cyclic_position [source]

$\mathbf{L}_{ij}\mathbf{x}_k + \mathbf{L}_{jk}\mathbf{x}_i + \mathbf{L}_{ki}\mathbf{x}_j = 0$*; in three dimensions this is* $\mathbf{L} \cdot \mathbf{x} = 0$*. The identity is trivial unless* `i`*,* `j`*,* `k` *are pairwise distinct, so it has no content in fewer than three dimensions.*

```
lemma angularMomentum_cyclic_position :
    𝐋 i j ∘L 𝐱 k + 𝐋 j k ∘L 𝐱 i + 𝐋 k i ∘L 𝐱 j = 0
```

lemma angularMomentum_cyclic_momentum [source]

$\mathbf{L}_{ij}\mathbf{p}_k + \mathbf{L}_{jk}\mathbf{p}_i + \mathbf{L}_{ki}\mathbf{p}_j = 0$; *in three dimensions this is* $\mathbf{L} \cdot \mathbf{p} = 0$. *The identity is trivial unless* `i, j, k` *are pairwise distinct, so it has no content in fewer than three dimensions.*

```
lemma angularMomentum_cyclic_momentum :
    L i j ∘L p k + L j k ∘L p i + L k i ∘L p j = 0
```

lemma angularMomentum_commutation_positionDotMomentum [source]

```
@[simp]
lemma angularMomentum_commutation_positionDotMomentum : ⁅L i j, x[d] ⬝ᵥ p⁆ = 0
```

2.103 Physlib.QuantumMechanics.Operators.AngularMomentum.DimThree

Physlib/QuantumMechanics/Operators/AngularMomentum/DimThree.lean on GitHub.

def angularMomentumDimThreeOperator [source]

The Cartesian Hilbert-space components, ordered as `x, y, z`.

```
def angularMomentumDimThreeOperator (i : Fin 3) :
    SpaceDHilbertSpace 3 →ₗ[ℂ] SpaceDHilbertSpace 3 where
```

lemma angularMomentumDimThreeCLM_zero_eq [source]

```
lemma angularMomentumDimThreeCLM_zero_eq : L₃ 0 = L 1 2
```

lemma angularMomentumDimThreeCLM_one_eq [source]

```
lemma angularMomentumDimThreeCLM_one_eq : L₃ 1 = L 2 0
```

lemma angularMomentumDimThreeCLM_two_eq [source]

```
lemma angularMomentumDimThreeCLM_two_eq : L₃ 2 = L 0 1
```

lemma angularMomentumDimThreeOperator_domain_eq [source]

```
lemma angularMomentumDimThreeOperator_domain_eq : (ℒ₃ i).domain = SchwartzSubmodule 3
```

lemma angularMomentumDimThreeOperator_apply [source]

```
lemma angularMomentumDimThreeOperator_apply (ψ : SchwartzSubmodule 3) :
    ℒ₃ i ψ = schwartzEquiv volume (L₃ i ((schwartzEquiv volume).symm ψ))
```

lemma angularMomentumDimThreeOperator_apply_ae [source]

```
lemma angularMomentumDimThreeOperator_apply_ae (ψ : SchwartzSubmodule 3) :
    ℒ₃ i ψ =ᵐ[volume] L₃ i ((schwartzEquiv volume).symm ψ)
```

lemma angularMomentumDimThreeOperator_range [source]

```
lemma angularMomentumDimThreeOperator_range (ψ : SchwartzSubmodule 3) :
    ℒ₃ i ψ ∈ SchwartzSubmodule 3
```

lemma angularMomentumDimThreeOperator_apply_apply [source]

Composing two components stays on the common Schwartz domain.

```
lemma angularMomentumDimThreeOperator_apply_apply (j : Fin 3) (ψ : SchwartzSubmodule 3) :
    ℒ₃ i ⟨ℒ₃ j ψ, angularMomentumDimThreeOperator_range j ψ⟩ =
      schwartzEquiv volume (𝐋₃ i (𝐋₃ j ((schwartzEquiv volume).symm ψ)))
```

lemma angularMomentumDimThreeOperator_hasDenseDomain [source]

```
lemma angularMomentumDimThreeOperator_hasDenseDomain :
    (ℒ₃ i).HasDenseDomain
```

lemma angularMomentumDimThreeOperator_isSymmetric [source]

```
lemma angularMomentumDimThreeOperator_isSymmetric : (ℒ₃ i).IsSymmetric
```

lemma angularMomentumDimThreeOperator_isUnbounded [source]

```
lemma angularMomentumDimThreeOperator_isUnbounded : (ℒ₃ i).IsUnbounded
```

lemma angularMomentumOperatorSqr_eq_sum_dimThree [source]

The antisymmetric-index definition agrees with the Cartesian sum of squares.

```
lemma angularMomentumOperatorSqr_eq_sum_dimThree : 𝐋²[3] = ∑ i : Fin 3, 𝐋₃ i ∘L 𝐋₃ i
```

lemma angularMomentumSqOperator_apply_eq_sum_dimThree [source]

The same Cartesian sum of squares, evaluated on a Hilbert-space Schwartz vector.

```
lemma angularMomentumSqOperator_apply_eq_sum_dimThree (ψ : SchwartzSubmodule 3) :
    angularMomentumSqOperator ψ = ∑ i : Fin 3, ℒ₃ i
      ⟨ℒ₃ i ψ, angularMomentumDimThreeOperator_range i ψ⟩
```

lemma angularMomentumDimThree_commutation [source]

The Cartesian angular momentum algebra, with the right-handed Levi-Civita symbol.

```
lemma angularMomentumDimThree_commutation (j : Fin 3) : ⁅𝐋₃ i, 𝐋₃ j⁆ =
    (I * ℏ) • ∑ k : Fin 3, (leviCivitaSymbol ![i, j, k] : ℂ) • 𝐋₃ k
```

lemma angularMomentumDimThree_commutation_xy [source]

[Lx, Ly] = iℏ Lz.

```
lemma angularMomentumDimThree_commutation_xy : ⁅𝐋₃ 0, 𝐋₃ 1⁆ = (I * ℏ) • 𝐋₃ 2
```

lemma angularMomentumDimThree_commutation_yz [source]

[Ly, Lz] = iℏ Lx.

```
lemma angularMomentumDimThree_commutation_yz : ⁅𝐋₃ 1, 𝐋₃ 2⁆ = (I * ℏ) • 𝐋₃ 0
```

lemma angularMomentumDimThree_commutation_zx [source]

[Lz, Lx] = iℏ Ly.

```
lemma angularMomentumDimThree_commutation_zx : ⁅𝐋₃ 2, 𝐋₃ 0⁆ = (I * ℏ) • 𝐋₃ 1
```

lemma angularMomentumSqr_commutation_dimThree [source]

Angular momentum squared commutes with every Cartesian component on Schwartz maps.

```
lemma angularMomentumSqr_commutation_dimThree : ⁅𝐋²[3], 𝐋₃ i⁆ = 0
```

lemma angularMomentumDimThreeOperator_commutation [source]

The Cartesian commutation algebra on the common Hilbert-space Schwartz domain.

```
lemma angularMomentumDimThreeOperator_commutation (j : Fin 3) (ψ : SchwartzSubmodule 3) :
    ℒ₃ i ⟨ℒ₃ j ψ, angularMomentumDimThreeOperator_range j ψ⟩ -
      ℒ₃ j ⟨ℒ₃ i ψ, angularMomentumDimThreeOperator_range i ψ⟩ =
      (I * ℏ) • ∑ k : Fin 3, (leviCivitaSymbol ![i, j, k] : ℂ) • ℒ₃ k ψ
```

lemma angularMomentumSqOperator_commutation_dimThree [source]

[L², Li] = 0 on the common invariant Hilbert-space domain.

```
lemma angularMomentumSqOperator_commutation_dimThree (ψ : SchwartzSubmodule 3) :
    angularMomentumSqOperator ⟨ℒ₃ i ψ, angularMomentumDimThreeOperator_range i ψ⟩ -
      ℒ₃ i ⟨angularMomentumSqOperator ψ, angularMomentumSqOperator_range ψ⟩ = 0
```

def angularMomentumRaisingCLM [source]

The angular momentum raising operator L₊ = Lx + i Ly on Schwartz maps.

```
def angularMomentumRaisingCLM : 𝓢(Space 3, ℂ) →L[ℂ] 𝓢(Space 3, ℂ)
```

def angularMomentumLoweringCLM [source]

The angular momentum lowering operator L₋ = Lx - i Ly on Schwartz maps.

```
def angularMomentumLoweringCLM : 𝓢(Space 3, ℂ) →L[ℂ] 𝓢(Space 3, ℂ)
```

lemma angularMomentumZ_commutation_raising [source]

[Lz, L₊] = ℏ L₊.

```
lemma angularMomentumZ_commutation_raising : ⁅𝐋₃ 2, 𝐋⁺⁆ = (ℏ : ℂ) • 𝐋⁺
```

lemma angularMomentumZ_commutation_lowering [source]

[Lz, L-] = -ħ L-.

```
lemma angularMomentumZ_commutation_lowering : ⁅𝐋₃ 2, 𝐋⁻⁆ = -(ℏ : ℂ) • 𝐋⁻
```

lemma angularMomentumSqr_commutation_raising [source]

Angular momentum squared commutes with the raising operator.

```
lemma angularMomentumSqr_commutation_raising : ⁅𝐋²[3], 𝐋⁺⁆ = 0
```

lemma angularMomentumSqr_commutation_lowering [source]

Angular momentum squared commutes with the lowering operator.

```
lemma angularMomentumSqr_commutation_lowering : ⁅𝐋²[3], 𝐋⁻⁆ = 0
```

def angularMomentumRaisingOperator [source]

The raising operator on the Hilbert space, with Schwartz domain.

```
def angularMomentumRaisingOperator : SpaceDHilbertSpace 3 →ₗ.[ℂ] SpaceDHilbertSpace 3 where
```

def angularMomentumLoweringOperator [source]

The lowering operator on the Hilbert space, with Schwartz domain.

```
def angularMomentumLoweringOperator : SpaceDHilbertSpace 3 →ₗ.[ℂ] SpaceDHilbertSpace 3 where
```

lemma angularMomentumRaisingOperator_domain_eq [source]

```
lemma angularMomentumRaisingOperator_domain_eq : (ℒ⁺).domain = SchwartzSubmodule 3
```

lemma angularMomentumLoweringOperator_domain_eq [source]

```
lemma angularMomentumLoweringOperator_domain_eq : (ℒ⁻).domain = SchwartzSubmodule 3
```

lemma angularMomentumRaisingOperator_apply [source]

```
lemma angularMomentumRaisingOperator_apply (ψ : SchwartzSubmodule 3) :
    ℒ⁺ ψ = schwartzEquiv volume (𝐋⁺ ((schwartzEquiv volume).symm ψ))
```

lemma angularMomentumLoweringOperator_apply [source]

```
lemma angularMomentumLoweringOperator_apply (ψ : SchwartzSubmodule 3) :
    ℒ⁻ ψ = schwartzEquiv volume (𝐋⁻ ((schwartzEquiv volume).symm ψ))
```

lemma angularMomentumRaisingOperator_apply_eq [source]

```
lemma angularMomentumRaisingOperator_apply_eq (ψ : SchwartzSubmodule 3) :
    ℒ⁺ ψ = ℒ₃ 0 ψ + I • ℒ₃ 1 ψ
```

lemma angularMomentumLoweringOperator_apply_eq [source]

```
lemma angularMomentumLoweringOperator_apply_eq (ψ : SchwartzSubmodule 3) :
    ℒ⁻ ψ = ℒ₃ 0 ψ - I • ℒ₃ 1 ψ
```

lemma angularMomentumRaisingOperator_apply_ae [source]

```
lemma angularMomentumRaisingOperator_apply_ae (ψ : SchwartzSubmodule 3) :
    ℒ⁺ ψ =ᵐ[volume] 𝐋⁺ ((schwartzEquiv volume).symm ψ)
```

lemma angularMomentumLoweringOperator_apply_ae [source]

```
lemma angularMomentumLoweringOperator_apply_ae (ψ : SchwartzSubmodule 3) :
    ℒ⁻ ψ =ᵐ[volume] 𝐋⁻ ((schwartzEquiv volume).symm ψ)
```

lemma angularMomentumRaisingOperator_range [source]

```
lemma angularMomentumRaisingOperator_range (ψ : SchwartzSubmodule 3) :
    ℒ⁺ ψ ∈ SchwartzSubmodule 3
```

lemma angularMomentumLoweringOperator_range [source]

```
lemma angularMomentumLoweringOperator_range (ψ : SchwartzSubmodule 3) :
    ℒ⁻ ψ ∈ SchwartzSubmodule 3
```

lemma angularMomentumRaisingOperator_hasDenseDomain [source]

```
lemma angularMomentumRaisingOperator_hasDenseDomain : (ℒ⁺).HasDenseDomain
```

lemma angularMomentumLoweringOperator_hasDenseDomain [source]

```
lemma angularMomentumLoweringOperator_hasDenseDomain : (ℒ⁻).HasDenseDomain
```

lemma angularMomentumZOperator_commutation_raising [source]

`[Lz, L₊] ψ = ℏ L₊ ψ` for every Schwartz vector in the Hilbert space.

```
lemma angularMomentumZOperator_commutation_raising (ψ : SchwartzSubmodule 3) :
    ℒ₃ 2 ⟨ℒ⁺ ψ, angularMomentumRaisingOperator_range ψ⟩ -
      ℒ⁺ ⟨ℒ₃ 2 ψ, angularMomentumDimThreeOperator_range 2 ψ⟩ = (ℏ : ℂ) • ℒ⁺ ψ
```

lemma angularMomentumZOperator_commutation_lowering [source]

`[Lz, L₋] ψ = -ℏ L₋ ψ` for every Schwartz vector in the Hilbert space.

```
lemma angularMomentumZOperator_commutation_lowering (ψ : SchwartzSubmodule 3) :
    ℒ₃ 2 ⟨ℒ⁻ ψ, angularMomentumLoweringOperator_range ψ⟩ -
      ℒ⁻ ⟨ℒ₃ 2 ψ, angularMomentumDimThreeOperator_range 2 ψ⟩ = -(ℏ : ℂ) • ℒ⁻ ψ
```

lemma angularMomentumSqOperator_commutation_raising [source]

`[L², L₊] ψ = 0` on the common invariant Schwartz domain.

```
lemma angularMomentumSqOperator_commutation_raising (ψ : SchwartzSubmodule 3) :
    ℒ² ⟨ℒ⁺ ψ, angularMomentumRaisingOperator_range ψ⟩ -
      ℒ⁺ ⟨ℒ² ψ, angularMomentumSqOperator_range ψ⟩ = 0
```

lemma angularMomentumSqOperator_commutation_lowering [source]

`[L², L₋] ψ = 0` on the common invariant Schwartz domain.

```
lemma angularMomentumSqOperator_commutation_lowering (ψ : SchwartzSubmodule 3) :
    ℒ² ⟨ℒ⁻ ψ, angularMomentumLoweringOperator_range ψ⟩ -
      ℒ⁻ ⟨ℒ² ψ, angularMomentumSqOperator_range ψ⟩ = 0
```

2.104 Physlib.QuantumMechanics.Operators.Multiplication

Physlib/QuantumMechanics/Operators/Multiplication.lean on GitHub.

lemma mem_mulOperator_domain_of_ae_eq_indicator [source]

A state which is μ-a.e. equal to the indicator of a set on which `f` is μ-a.e. bounded lies in the domain of `𝓜 μ f`.

```
lemma mem_mulOperator_domain_of_ae_eq_indicator {μ : Measure (Space d)} {f : Space d → ℂ}
    (hf : AEStronglyMeasurable f μ) {s : Set (Space d)} {c : ℝ}
    (hfc : ∀ᵐ x ∂μ, x ∈ s → ‖f x‖ ≤ c) {ψ : SpaceDHilbertSpace d μ}
    (hψ : ψ =ᵐ[μ] s.indicator 1) : ψ ∈ (𝓜 μ f).domain
```

lemma mulOperator_eq_iff_ae_eq [source]

For a μ-a.e. strongly measurable `f` and σ-finite μ, the multiplication operator determines the function up to μ-a.e. equality. No measurability of `g` is needed.

```
lemma mulOperator_eq_iff_ae_eq {μ : Measure (Space d)} [SigmaFinite μ] {f g : Space d → ℂ}
    (hf : AEStronglyMeasurable f μ) : 𝓜 μ f = 𝓜 μ g ↔ f =ᵐ[μ] g
```

2.105 Physlib.QuantumMechanics.Operators.SpectralTheory.SpectralMeasure

Physlib/QuantumMechanics/Operators/SpectralTheory/SpectralMeasure.lean on GitHub.

instance HasUniqueDiv (H →L[ℂ] H) [source]

```
instance (H : Type*) [SeminormedAddCommGroup H] [InnerProductSpace ℂ H] :
    HasUniqueDiv (H →L[ℂ] H) where
```

2.106 Physlib.QuantumMechanics.PoschlTeller.Scattering

Physlib/QuantumMechanics/PoschlTeller/Scattering.lean on GitHub.

def polyDeriv [source]

The derivative acting on the polynomial of a wave `p(tanh (κ x)) e^{i k x}`, `p ↦ κ (1 - X²) p' + i k p`, so that `∂ₓ (p(tanh (κ x)) e^{i k x}) = (polyDeriv k p)(tanh (κ x)) e^{i k x}`.

```
def polyDeriv (k : ℝ) (p : ℂ[X]) : ℂ[X]
```

def polyRaise [source]

The raising operator `-∂ₓ + n κ tanh (κ x)` acting on the polynomial of a wave `p(tanh (κ x)) e^{i k x}`.

```
def polyRaise (k : ℝ) (n : ℕ) (p : ℂ[X]) : ℂ[X]
```

def polyReducedHamiltonian [source]

The operator `-∂ₓ² - n (n + 1) κ² / cosh² (κ x)` acting on the polynomial of a wave `p(tanh (κ x)) e^{i k x}`. This is `2 m / ħ²` times the Hamiltonian of the Pöschl-Teller system with family number `n`.

```
def polyReducedHamiltonian (k : ℝ) (n : ℕ) (p : ℂ[X]) : ℂ[X]
```

lemma polyReducedHamiltonian_polyRaise [source]

The raising operator intertwines the Hamiltonians with family numbers `n` and `n + 1`.

```
lemma polyReducedHamiltonian_polyRaise (k : ℝ) (n : ℕ) (p : ℂ[X]) :
    Q.polyReducedHamiltonian k (n + 1) (Q.polyRaise k (n + 1) p) =
      Q.polyRaise k (n + 1) (Q.polyReducedHamiltonian k n p)
```

lemma polyDeriv_C [source]

The derivative of a constant multiple of the plane wave `c e^{i k x}` is `i k c e^{i k x}`.

```
lemma polyDeriv_C (k : ℝ) (c : ℂ) : Q.polyDeriv k (C c) = C (k * I * c)
```

lemma polyRaise_C_mul [source]

The raising operator is linear with respect to constant multiples.

```
lemma polyRaise_C_mul (k : ℝ) (n : ℕ) (c : ℂ) (p : ℂ[X]) :
    Q.polyRaise k n (C c * p) = C c * Q.polyRaise k n p
```

def polyScattering [source]

The polynomial `Pₖ` of the scattering state `Pₖ(tanh (κ x)) e^{i k x}` of the Pöschl-Teller system with family number `n`, obtained by applying the raising operators with `1, …, n` to `1`.

```
def polyScattering (k : ℝ) : ℕ → ℂ[X]
  | 0 => 1
  | n + 1 => Q.polyRaise k (n + 1) (polyScattering k n)
```

lemma polyReducedHamiltonian_polyScattering [source]

The scattering polynomial with family number `n` is an eigenvector of the reduced Hamiltonian with eigenvalue `k²`.

```
lemma polyReducedHamiltonian_polyScattering (k : ℝ) (n : ℕ) :
    Q.polyReducedHamiltonian k n (Q.polyScattering k n) =
      C ((k : ℂ) ^ 2) * Q.polyScattering k n
```

lemma eval_one_polyRaise [source]

At `tanh (κ x) = 1` the raising operator acts as multiplication by `n κ - i k`.

```
lemma eval_one_polyRaise (k : ℝ) (n : ℕ) (p : ℂ[X]) :
    (Q.polyRaise k n p).eval 1 = (n * Q.κ - k * I) * p.eval 1
```

lemma eval_neg_one_polyRaise [source]

At `tanh (κ x) = -1` the raising operator acts as multiplication by `-n κ - i k`.

```
lemma eval_neg_one_polyRaise (k : ℝ) (n : ℕ) (p : ℂ[X]) :
    (Q.polyRaise k n p).eval (-1) = (-(n * Q.κ) - k * I) * p.eval (-1)
```

lemma polyScattering_eval_one [source]

The value at `1` of the scattering polynomial is `∏ₙ (n κ - i k)`.

```
lemma polyScattering_eval_one (k : ℝ) (n : ℕ) :
    (Q.polyScattering k n).eval 1 = ∏ j ∈ Finset.range n, ((j + 1) * Q.κ - k * I)
```

lemma polyScattering_eval_neg_one [source]

The value at `-1` of the scattering polynomial is `∏ₙ (-n κ - i k)`.

```
lemma polyScattering_eval_neg_one (k : ℝ) (n : ℕ) :
    (Q.polyScattering k n).eval (-1) = ∏ j ∈ Finset.range n, (-((j + 1) * Q.κ) - k * I)
```

lemma polyScattering_eval_one_ne_zero [source]

The value at `1` of the scattering polynomial is non-zero.

```
lemma polyScattering_eval_one_ne_zero (k : ℝ) (n : ℕ) : (Q.polyScattering k n).eval 1 ≠ 0
```

lemma polyScattering_eval_neg_one_ne_zero [source]

The value at `-1` of the scattering polynomial is non-zero.

```
lemma polyScattering_eval_neg_one_ne_zero (k : ℝ) (n : ℕ) :
    (Q.polyScattering k n).eval (-1) ≠ 0
```

2.107 `Physlib.Relativity.Causality.TimeLike`

`Physlib/Relativity/Causality/TimeLike.lean` on GitHub.

lemma minkowskiProduct_ne_zero_of_timeLike [source]

Two timelike vectors have nonzero Minkowski inner product.

```
lemma minkowskiProduct_ne_zero_of_timeLike {d : ℕ} {u v : Vector d}
    (hu : causalCharacter u = .timeLike) (hv : causalCharacter v = .timeLike) :
    ⟪u, v⟫ₘ ≠ 0
```

lemma minkowskiProduct_self_neg_of_timeLike_orthogonal [source]

A nonzero vector orthogonal to a timelike vector has negative Minkowski self-product.

```
lemma minkowskiProduct_self_neg_of_timeLike_orthogonal {d : ℕ} {u v : Lorentz.Vector d}
    (hu : causalCharacter u = .timeLike) (hv : v ≠ 0) (horth : ⟪u, v⟫ₘ = 0) :
    ⟪v, v⟫ₘ < 0
```

2.108 Physlib.Relativity.CliffordAlgebra

Physlib/Relativity/CliffordAlgebra.lean on GitHub.

lemma γ0_eq_matrix [source]

The entries of γ0.

```
lemma γ0_eq_matrix : γ0 = !![1, 0, 0, 0; 0, 1, 0, 0; 0, 0, -1, 0; 0, 0, 0, -1]
```

lemma γ1_eq_matrix [source]

The entries of γ1.

```
lemma γ1_eq_matrix : γ1 = !![0, 0, 0, 1; 0, 0, 1, 0; 0, -1, 0, 0; -1, 0, 0, 0]
```

lemma γ2_eq_matrix [source]

The entries of γ2.

```
lemma γ2_eq_matrix : γ2 = !![0, 0, 0, -I; 0, 0, I, 0; 0, I, 0, 0; -I, 0, 0, 0]
```

lemma γ3_eq_matrix [source]

The entries of γ3.

```
lemma γ3_eq_matrix : γ3 = !![0, 0, 1, 0; 0, 0, 0, -1; -1, 0, 0, 0; 0, 1, 0, 0]
```

lemma γ_mem_unitary [source]

The gamma matrices in the Dirac representation are unitary.

```
lemma γ_mem_unitary (μ : Fin 1 ⊕ Fin 3) :
    endEquivMatrix (gamma μ) ∈ Matrix.unitaryGroup (Fin 4) ℂ
```

def γProd [source]

The product, with increasing indices, of the gamma matrices in the Dirac representation whose index lies in `s`; the empty product is `1`. The index `i : Fin 4` labels the image of `CliffordAlgebra.ι diracForm (Pi.single i 1)` under `ofCliffordAlgebra`.

```
def γProd (s : Finset (Fin 4)) : Matrix (Fin 4) (Fin 4) ℂ
```

lemma γProd_mem_diracAlgebra [source]

Each product `γProd s` lies in `diracAlgebra`.

```
lemma γProd_mem_diracAlgebra (s : Finset (Fin 4)) : γProd s ∈ diracAlgebra
```

lemma γProd_singleton [source]

The product `γProd {i}` is the gamma matrix of index `i`.

```
lemma γProd_singleton (i : Fin 4) :
    γProd {i} = endEquivMatrix (gamma (finSumFinEquiv.symm i))
```

lemma γ_mul_γProd [source]

The gamma matrix of index `j` commutes past `γProd s` up to the sign `(-1) ^ #(s.erase j)`, as it anticommutes with the gamma matrices of the other indices.

```
lemma γ_mul_γProd (j : Fin 4) (s : Finset (Fin 4)) :
    endEquivMatrix (gamma (finSumFinEquiv.symm j)) * γProd s =
      (-1 : ℂ) ^ (s.erase j).card • (γProd s * endEquivMatrix (gamma (finSumFinEquiv.symm j)))
```

lemma γProd_mem_unitary [source]

Each product `γProd s` is unitary.

```
lemma γProd_mem_unitary (s : Finset (Fin 4)) : γProd s ∈ Matrix.unitaryGroup (Fin 4) ℂ
```

lemma trace_star_γProd_mul_γProd [source]

The products `γProd s` are orthogonal for the trace form `tr (star A * B)`, each with square norm `4`.

```
lemma trace_star_γProd_mul_γProd (t s : Finset (Fin 4)) :
    (star (γProd t) * γProd s).trace = if t = s then 4 else 0
```

lemma linearIndependent_γProd [source]

The 16 products `γProd s` are linearly independent over $\mathbb{R}$, since they are orthogonal for the trace form `tr (star A * B)`.

```
lemma linearIndependent_γProd : LinearIndependent ℝ γProd
```

2.109 Physlib.Relativity.Fermions.Weyl.Contraction

`Physlib/Relativity/Fermions/Weyl/Contraction.lean` on GitHub.

def LeftHandedWeyl.contrBi [source]

The bilinear map `(ψ, φ) ↦ ψ^a φ_a` underlying `LeftHandedWeyl.contr`.

```
def LeftHandedWeyl.contrBi : LeftHandedWeyl →ₗ[ℂ] DualLeftHandedWeyl →ₗ[ℂ] ℂ where
```

def LeftHandedWeyl.contr [source]

The contraction `ψ^a φ_a` of a left-handed with a dual-left-handed Weyl fermion, the dot product of their components, as an intertwining map to the trivial representation.

```
def LeftHandedWeyl.contr : (LeftHandedWeyl.rep.tprod DualLeftHandedWeyl.rep).IntertwiningMap
    (Representation.trivial ℂ SL(2,ℂ) ℂ) where
```

lemma LeftHandedWeyl.contr_tmul [source]

```
lemma LeftHandedWeyl.contr_tmul (ψ : LeftHandedWeyl)
    (φ : DualLeftHandedWeyl) :
    LeftHandedWeyl.contr (ψ ⊗ₜ φ) = ψ.toFin2ℂ ·ᵥ φ.toFin2ℂ
```

lemma LeftHandedWeyl.contr_basis [source]

```
lemma LeftHandedWeyl.contr_basis (i j : Fin 2) :
    LeftHandedWeyl.contr (LeftHandedWeyl.basis i ⊗ₜ DualLeftHandedWeyl.basis j) =
      if i.1 = j.1 then (1 : ℂ) else 0
```

lemma LeftHandedWeyl.contr_tmul_basis [source]

Contracting with the `j`-th basis vector picks out the `j`-th component of `ψ`.

```
lemma LeftHandedWeyl.contr_tmul_basis (ψ : LeftHandedWeyl) (j : Fin 2) :
    LeftHandedWeyl.contr (ψ ⊗ₜ DualLeftHandedWeyl.basis j) = ψ.val j
```

def DualLeftHandedWeyl.contrBi [source]

The bilinear map `(φ, ψ) ↦ φ_a ψ^a` underlying `DualLeftHandedWeyl.contr`.

```
def DualLeftHandedWeyl.contrBi : DualLeftHandedWeyl →ₗ[ℂ] LeftHandedWeyl →ₗ[ℂ] ℂ where
```

def DualLeftHandedWeyl.contr [source]

The contraction `φ_a ψ^a` of a dual-left-handed with a left-handed Weyl fermion, the dot product of their components, as an intertwining map to the trivial representation.

```
def DualLeftHandedWeyl.contr : (DualLeftHandedWeyl.rep.tprod LeftHandedWeyl.rep).
    IntertwiningMap
    (Representation.trivial ℂ SL(2,ℂ) ℂ) where
```

lemma DualLeftHandedWeyl.contr_tmul [source]

```
lemma DualLeftHandedWeyl.contr_tmul (φ : DualLeftHandedWeyl) (ψ : LeftHandedWeyl) :
    DualLeftHandedWeyl.contr (φ ⊗ₜ ψ) = φ.toFin2ℂ ·ᵥ ψ.toFin2ℂ
```

lemma DualLeftHandedWeyl.contr_basis [source]

```
lemma DualLeftHandedWeyl.contr_basis (i j : Fin 2) :
    DualLeftHandedWeyl.contr (DualLeftHandedWeyl.basis i ⊗ₜ LeftHandedWeyl.basis j) =
      if i.1 = j.1 then (1 : ℂ) else 0
```

lemma LeftHandedWeyl.contr_tmul_symm [source]

```
lemma LeftHandedWeyl.contr_tmul_symm (ψ : LeftHandedWeyl) (φ : DualLeftHandedWeyl) :
    LeftHandedWeyl.contr (ψ ⊗ₜ[ℂ] φ) = DualLeftHandedWeyl.contr (φ ⊗ₜ[ℂ] ψ)
```

lemma DualLeftHandedWeyl.contr_tmul_symm [source]

```
lemma DualLeftHandedWeyl.contr_tmul_symm (φ : DualLeftHandedWeyl) (ψ : LeftHandedWeyl) :
    DualLeftHandedWeyl.contr (φ ⊗ₜ[ℂ] ψ) = LeftHandedWeyl.contr (ψ ⊗ₜ[ℂ] φ)
```

lemma LeftHandedWeyl.eq_zero_of_contr_tmul_eq_zero [source]

The contraction is nondegenerate: if `ψ^a φ_a = 0` for every `φ` then `ψ = 0`.

```
lemma LeftHandedWeyl.eq_zero_of_contr_tmul_eq_zero (ψ : LeftHandedWeyl)
    (h : ∀ φ, LeftHandedWeyl.contr (ψ ⊗ₜ φ) = 0) : ψ = 0
```

lemma DualLeftHandedWeyl.eq_zero_of_contr_tmul_eq_zero [source]

The contraction is nondegenerate: if `φ_a ψ^a = 0` for every `ψ` then `φ = 0`.

```
lemma DualLeftHandedWeyl.eq_zero_of_contr_tmul_eq_zero (φ : DualLeftHandedWeyl)
    (h : ∀ ψ, DualLeftHandedWeyl.contr (φ ⊗ₜ ψ) = 0) : φ = 0
```

instance LeftHandedWeyl.contrBi_isPerfPair [source]

The pairing `(ψ, φ) ↦ ψ^a φ_a` is perfect. It identifies dual-left-handed Weyl fermions with the dual of left-handed Weyl fermions, and vice versa.

```
instance LeftHandedWeyl.contrBi_isPerfPair : LeftHandedWeyl.contrBi.IsPerfPair
```

instance DualLeftHandedWeyl.contrBi_isPerfPair [source]

The pairing `(φ, ψ) ↦ φ_a ψ^a` is perfect. It identifies left-handed Weyl fermions with the dual of dual-left-handed Weyl fermions, and vice versa.

```
instance DualLeftHandedWeyl.contrBi_isPerfPair : DualLeftHandedWeyl.contrBi.IsPerfPair
```

lemma LeftHandedWeyl.eq_smul_contr [source]

Every `SL(2,ℂ)`-invariant pairing of left-handed with dual-left-handed Weyl fermions is a multiple of `LeftHandedWeyl.contr`.

```
lemma LeftHandedWeyl.eq_smul_contr
    (B : (LeftHandedWeyl.rep.tprod DualLeftHandedWeyl.rep).IntertwiningMap
      (Representation.trivial ℂ SL(2,ℂ) ℂ)) :
    ∃ z : ℂ, B = z • LeftHandedWeyl.contr
```

lemma DualLeftHandedWeyl.eq_smul_contr [source]

Every `SL(2,ℂ)`-invariant pairing of dual-left-handed with left-handed Weyl fermions is a multiple of `DualLeftHandedWeyl.contr`.

```
lemma DualLeftHandedWeyl.eq_smul_contr
    (B : (DualLeftHandedWeyl.rep.tprod LeftHandedWeyl.rep).IntertwiningMap
      (Representation.trivial ℂ SL(2,ℂ) ℂ)) :
    ∃ z : ℂ, B = z • DualLeftHandedWeyl.contr
```

def RightHandedWeyl.contrBi [source]

The bilinear map `(ψ, φ) ↦ ψ^{dot a} φ_{dot a}` underlying `RightHandedWeyl.contr`.

```
def RightHandedWeyl.contrBi : RightHandedWeyl →ₗ[ℂ] DualRightHandedWeyl →ₗ[ℂ] ℂ where
```

def RightHandedWeyl.contr [source]

The contraction `ψ^{dot a} φ_{dot a}` of a right-handed with a dual-right-handed Weyl fermion, the dot product of their components, as an intertwining map to the trivial representation.

```
def RightHandedWeyl.contr : (RightHandedWeyl.rep.tprod DualRightHandedWeyl.rep).IntertwiningMap
    (Representation.trivial ℂ SL(2,ℂ) ℂ) where
```

lemma RightHandedWeyl.contr_tmul [source]

```
lemma RightHandedWeyl.contr_tmul (ψ : RightHandedWeyl)
    (φ : DualRightHandedWeyl) :
    RightHandedWeyl.contr (ψ ⊗ₜ φ) = ψ.toFin2ℂ ⬝ᵥ φ.toFin2ℂ
```

lemma RightHandedWeyl.contr_basis [source]

```
lemma RightHandedWeyl.contr_basis (i j : Fin 2) :
    RightHandedWeyl.contr (RightHandedWeyl.basis i ⊗ₜ DualRightHandedWeyl.basis j) =
    if i.1 = j.1 then (1 : ℂ) else 0
```

lemma RightHandedWeyl.contr_tmul_basis [source]

Contracting with the `j`-th basis vector picks out the `j`-th component.

```
lemma RightHandedWeyl.contr_tmul_basis (ψ : RightHandedWeyl) (j : Fin 2) :
    RightHandedWeyl.contr (ψ ⊗ₜ DualRightHandedWeyl.basis j) = ψ.val j
```

def DualRightHandedWeyl.contrBi [source]

The bilinear map `(φ, ψ) ↦ φ_{dot a} ψ^{dot a}` underlying `DualRightHandedWeyl.contr`.

```
def DualRightHandedWeyl.contrBi : DualRightHandedWeyl →ₗ[ℂ] RightHandedWeyl →ₗ[ℂ] ℂ where
```

def DualRightHandedWeyl.contr [source]

The contraction `φ_{dot a} ψ^{dot a}` of a dual-right-handed with a right-handed Weyl fermion, the dot product of their components, as an intertwining map to the trivial representation.

```
def DualRightHandedWeyl.contr : (DualRightHandedWeyl.rep.tprod RightHandedWeyl.rep).
    IntertwiningMap
    (Representation.trivial ℂ SL(2,ℂ) ℂ) where
```

lemma DualRightHandedWeyl.contr_tmul [source]

```
lemma DualRightHandedWeyl.contr_tmul (φ : DualRightHandedWeyl)
    (ψ : RightHandedWeyl) :
    DualRightHandedWeyl.contr (φ ⊗ₜ ψ) = φ.toFin2ℂ ·ᵥ ψ.toFin2ℂ
```

lemma DualRightHandedWeyl.contr_basis [source]

```
lemma DualRightHandedWeyl.contr_basis (i j : Fin 2) :
    DualRightHandedWeyl.contr (DualRightHandedWeyl.basis i ⊗ₜ RightHandedWeyl.basis j) =
    if i.1 = j.1 then (1 : ℂ) else 0
```

lemma RightHandedWeyl.contr_tmul_symm [source]

```
lemma RightHandedWeyl.contr_tmul_symm (ψ : RightHandedWeyl) (φ : DualRightHandedWeyl) :
    RightHandedWeyl.contr (ψ ⊗ₜ[ℂ] φ) = DualRightHandedWeyl.contr (φ ⊗ₜ[ℂ] ψ)
```

lemma DualRightHandedWeyl.contr_tmul_symm [source]

```
lemma DualRightHandedWeyl.contr_tmul_symm (φ : DualRightHandedWeyl) (ψ : RightHandedWeyl) :
    DualRightHandedWeyl.contr (φ ⊗ₜ[ℂ] ψ) = RightHandedWeyl.contr (ψ ⊗ₜ[ℂ] φ)
```

lemma RightHandedWeyl.eq_zero_of_contr_tmul_eq_zero [source]

The contraction is nondegenerate: if `ψ^{dot a} φ_{dot a} = 0` for every `φ` then `ψ = 0`.

```
lemma RightHandedWeyl.eq_zero_of_contr_tmul_eq_zero (ψ : RightHandedWeyl)
    (h : ∀ φ, RightHandedWeyl.contr (ψ ⊗ₜ φ) = 0) : ψ = 0
```

lemma DualRightHandedWeyl.eq_zero_of_contr_tmul_eq_zero [source]

The contraction is nondegenerate: if `φ_{dot a} ψ^{dot a} = 0` for every `ψ` then `φ = 0`.

```
lemma DualRightHandedWeyl.eq_zero_of_contr_tmul_eq_zero (φ : DualRightHandedWeyl)
    (h : ∀ ψ, DualRightHandedWeyl.contr (φ ⊗ₜ ψ) = 0) : φ = 0
```

instance RightHandedWeyl.contrBi_isPerfPair [source]

The pairing `(ψ, φ) ↦ ψ^{dot a} φ_{dot a}` is perfect. It identifies dual-right-handed Weyl fermions with the dual of right-handed Weyl fermions, and vice versa.

```
instance RightHandedWeyl.contrBi_isPerfPair : RightHandedWeyl.contrBi.IsPerfPair
```

instance DualRightHandedWeyl.contrBi_isPerfPair [source]

The pairing `(φ, ψ) ↦ φ_{dot a} ψ^{dot a}` is perfect. It identifies right-handed Weyl fermions with the dual of dual-right-handed Weyl fermions, and vice versa.

```
instance DualRightHandedWeyl.contrBi_isPerfPair : DualRightHandedWeyl.contrBi.IsPerfPair
```

lemma RightHandedWeyl.eq_smul_contr [source]

Every `SL(2,ℂ)`-invariant pairing of right-handed with dual-right-handed Weyl fermions is a multiple of `RightHandedWeyl.contr`.

```
lemma RightHandedWeyl.eq_smul_contr
    (B : (RightHandedWeyl.rep.tprod DualRightHandedWeyl.rep).IntertwiningMap
      (Representation.trivial ℂ SL(2,ℂ) ℂ)) :
    ∃ z : ℂ, B = z • RightHandedWeyl.contr
```

lemma DualRightHandedWeyl.eq_smul_contr [source]

Every `SL(2,ℂ)`-invariant pairing of dual-right-handed with right-handed Weyl fermions is a multiple of `DualRightHandedWeyl.contr`.

```
lemma DualRightHandedWeyl.eq_smul_contr
    (B : (DualRightHandedWeyl.rep.tprod RightHandedWeyl.rep).IntertwiningMap
      (Representation.trivial ℂ SL(2,ℂ) ℂ)) :
    ∃ z : ℂ, B = z • DualRightHandedWeyl.contr
```

2.110 Physlib.Relativity.Fermions.Weyl.DualLeftHanded

Physlib/Relativity/Fermions/Weyl/DualLeftHanded.lean on GitHub.

instance Nontrivial DualLeftHandedWeyl [source]

```
instance : Nontrivial DualLeftHandedWeyl
```

instance FiniteDimensional ℂ DualLeftHandedWeyl [source]

```
instance : FiniteDimensional ℂ DualLeftHandedWeyl
```

lemma finrank_eq_two [source]

Dual left-handed Weyl fermions form a two-dimensional complex vector space.

```
@[simp]
lemma finrank_eq_two : Module.finrank ℂ DualLeftHandedWeyl = 2
```

lemma rep_apply [source]

```
lemma rep_apply (M : SL(2,ℂ)) (ψ : DualLeftHandedWeyl) : rep M ψ = ⟨(M.1⁻¹)ᵀ *ᵥ ψ.1⟩
```

lemma eq_zero_of_rep_eq_smul [source]

The representation `rep` has no common eigenvector.

```
lemma eq_zero_of_rep_eq_smul (ψ : DualLeftHandedWeyl)
    (h : ∀ M : SL(2,ℂ), ∃ c : ℂ, rep M ψ = c • ψ) : ψ = 0
```

lemma eq_bot_or_eq_top_of_rep_mem [source]

A submodule of `DualLeftHandedWeyl` stable under `rep` is either `⊥` or everything.

```
lemma eq_bot_or_eq_top_of_rep_mem (p : Submodule ℂ DualLeftHandedWeyl)
    (h : ∀ (M : SL(2,ℂ)) (ψ : DualLeftHandedWeyl), ψ ∈ p → rep M ψ ∈ p) :
    p = ⊥ ∨ p = ⊤
```

instance rep.IsIrreducible [source]

The representation `rep` of `SL(2, ℂ)` is irreducible.

```
instance : rep.IsIrreducible where
```

lemma end_eq_smul_id_of_commute_rep [source]

Schur's lemma for dual left-handed Weyl fermions.

```
lemma end_eq_smul_id_of_commute_rep {T : Module.End ℂ DualLeftHandedWeyl}
    (hT : ∀ M : SL(2,ℂ), T ∘ₗ rep M = rep M ∘ₗ T) : ∃ μ : ℂ, T = μ • LinearMap.id
```

2.111 Physlib.Relativity.Fermions.Weyl.DualRightHanded

Physlib/Relativity/Fermions/Weyl/DualRightHanded.lean on GitHub.

instance Nontrivial DualRightHandedWeyl [source]

```
instance : Nontrivial DualRightHandedWeyl
```

instance FiniteDimensional ℂ DualRightHandedWeyl [source]

```
instance : FiniteDimensional ℂ DualRightHandedWeyl
```

lemma finrank_eq_two [source]

Dual right-handed Weyl fermions form a two-dimensional complex vector space.

```
@[simp]
lemma finrank_eq_two : Module.finrank ℂ DualRightHandedWeyl = 2
```

lemma eq_zero_of_rep_eq_smul [source]

The representation `rep` has no common eigenvector.

```
lemma eq_zero_of_rep_eq_smul (ψ : DualRightHandedWeyl)
    (h : ∀ M : SL(2,ℂ), ∃ c : ℂ, rep M ψ = c • ψ) : ψ = 0
```

lemma eq_bot_or_eq_top_of_rep_mem [source]

A submodule of `DualRightHandedWeyl` stable under `rep` is either `⊥` or everything.

```
lemma eq_bot_or_eq_top_of_rep_mem (p : Submodule ℂ DualRightHandedWeyl)
    (h : ∀ (M : SL(2,ℂ)) (ψ : DualRightHandedWeyl), ψ ∈ p → rep M ψ ∈ p) :
    p = ⊥ ∨ p = ⊤
```

instance rep.IsIrreducible [source]

The representation `rep` of `SL(2, ℂ)` is irreducible.

```
instance : rep.IsIrreducible where
```

lemma end_eq_smul_id_of_commute_rep [source]

Schur's lemma for dual right-handed Weyl fermions.

```
lemma end_eq_smul_id_of_commute_rep {T : Module.End ℂ DualRightHandedWeyl}
    (hT : ∀ M : SL(2,ℂ), T ∘ₗ rep M = rep M ∘ₗ T) : ∃ μ : ℂ, T = μ • LinearMap.id
```

2.112 Physlib.Relativity.Fermions.Weyl.LeftHanded

Physlib/Relativity/Fermions/Weyl/LeftHanded.lean on GitHub.

instance Nontrivial LeftHandedWeyl [source]

```
instance : Nontrivial LeftHandedWeyl
```

instance FiniteDimensional ℂ LeftHandedWeyl [source]

```
instance : FiniteDimensional ℂ LeftHandedWeyl
```

lemma finrank_eq_two [source]

Left-handed Weyl fermions form a two-dimensional complex vector space.

```
@[simp]
lemma finrank_eq_two : Module.finrank ℂ LeftHandedWeyl = 2
```

lemma eq_zero_of_rep_eq_smul [source]

The representation `rep` has no common eigenvector.

```
lemma eq_zero_of_rep_eq_smul (ψ : LeftHandedWeyl)
    (h : ∀ M : SL(2,ℂ), ∃ c : ℂ, rep M ψ = c • ψ) : ψ = 0
```

lemma eq_bot_or_eq_top_of_rep_mem [source]

A submodule of `LeftHandedWeyl` stable under `rep` is either `⊥` or everything.

```
lemma eq_bot_or_eq_top_of_rep_mem (p : Submodule ℂ LeftHandedWeyl)
    (h : ∀ (M : SL(2,ℂ)) (ψ : LeftHandedWeyl), ψ ∈ p → rep M ψ ∈ p) :
    p = ⊥ ∨ p = ⊤
```

instance rep.IsIrreducible [source]

The representation `rep` of `SL(2, ℂ)` is irreducible.

```
instance : rep.IsIrreducible where
```

lemma end_eq_smul_id_of_commute_rep [source]

Schur's lemma for left-handed Weyl fermions.

```
lemma end_eq_smul_id_of_commute_rep {T : Module.End ℂ LeftHandedWeyl}
    (hT : ∀ M : SL(2,ℂ), T ∘ₗ rep M = rep M ∘ₗ T) : ∃ μ : ℂ, T = μ • LinearMap.id
```

2.113 Physlib.Relativity.Fermions.Weyl.Metric

Physlib/Relativity/Fermions/Weyl/Metric.lean on GitHub.

def LeftHandedWeyl.metricVal [source]

The metric ε^{aa} as an element of `(leftHanded ⊗ leftHanded).V`.

```
def LeftHandedWeyl.metricVal : LeftHandedWeyl ⊗[ℂ] LeftHandedWeyl
```

lemma LeftHandedWeyl.metricVal_expand_tmul [source]

Expansion of `LeftHandedWeyl.metricVal` into the left basis.

```
lemma LeftHandedWeyl.metricVal_expand_tmul : LeftHandedWeyl.metricVal =
    - LeftHandedWeyl.basis 0 ⊗ₜ[ℂ] LeftHandedWeyl.basis 1 +
      LeftHandedWeyl.basis 1 ⊗ₜ[ℂ] LeftHandedWeyl.basis 0
```

lemma LeftHandedWeyl.metricVal_expand_tmul' [source]

```
lemma LeftHandedWeyl.metricVal_expand_tmul' : LeftHandedWeyl.metricVal =
    LeftHandedWeyl.basis 1 ⊗ₜ[ℂ] LeftHandedWeyl.basis 0
    - LeftHandedWeyl.basis 0 ⊗ₜ[ℂ] LeftHandedWeyl.basis 1
```

def LeftHandedWeyl.metric [source]

The metric ε^{aa} as a morphism `⊮_ (Rep ℂ SL(2,ℂ)) ⟶ leftHanded ⊗ leftHanded`, making manifest its invariance under the action of `SL(2,ℂ)`.

```
def LeftHandedWeyl.metric : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
    (LeftHandedWeyl.rep.tprod LeftHandedWeyl.rep) where
```

lemma LeftHandedWeyl.metric_apply_one [source]

```
lemma LeftHandedWeyl.metric_apply_one :
    LeftHandedWeyl.metric (1 : ℂ) = LeftHandedWeyl.metricVal
```

def DualLeftHandedWeyl.metricVal [source]

The metric ε_{aa} as an element of `(dualLeftHanded ⊗ dualLeftHanded).V`.

```
def DualLeftHandedWeyl.metricVal : (DualLeftHandedWeyl ⊗[ℂ] DualLeftHandedWeyl)
```

lemma DualLeftHandedWeyl.metricVal_expand_tmul [source]

Expansion of `DualLeftHandedWeyl.metricVal` into the left basis.

```
lemma DualLeftHandedWeyl.metricVal_expand_tmul : DualLeftHandedWeyl.metricVal =
    DualLeftHandedWeyl.basis 0 ⊗ₜ[ℂ] DualLeftHandedWeyl.basis 1 -
      DualLeftHandedWeyl.basis 1 ⊗ₜ[ℂ] DualLeftHandedWeyl.basis 0
```

def DualLeftHandedWeyl.metric [source]

The metric `εₐₐ` as a morphism `𝓕.V ⟶ (Rep ℂ SL(2,ℂ)) ⟶ dualLeftHanded ⊗ dualLeftHanded`, making manifest its invariance under the action of `SL(2,ℂ)`.

```
def DualLeftHandedWeyl.metric : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
    (DualLeftHandedWeyl.rep.tprod DualLeftHandedWeyl.rep) where
```

lemma DualLeftHandedWeyl.metric_apply_one [source]

```
lemma DualLeftHandedWeyl.metric_apply_one :
    DualLeftHandedWeyl.metric (1 : ℂ) = DualLeftHandedWeyl.metricVal
```

def RightHandedWeyl.metricVal [source]

The metric `ε^{dot a}^{dot a}` as an element of `(rightHanded ⊗ rightHanded).V`.

```
def RightHandedWeyl.metricVal : (RightHandedWeyl ⊗[ℂ] RightHandedWeyl)
```

lemma RightHandedWeyl.metricVal_expand_tmul [source]

Expansion of `RightHandedWeyl.metricVal` into the left basis.

```
lemma RightHandedWeyl.metricVal_expand_tmul : RightHandedWeyl.metricVal =
    - RightHandedWeyl.basis 0 ⊗ₜ[ℂ] RightHandedWeyl.basis 1 +
      RightHandedWeyl.basis 1 ⊗ₜ[ℂ] RightHandedWeyl.basis 0
```

lemma RightHandedWeyl.metricVal_expand_tmul' [source]

```
lemma RightHandedWeyl.metricVal_expand_tmul' : RightHandedWeyl.metricVal =
    RightHandedWeyl.basis 1 ⊗ₜ[ℂ] RightHandedWeyl.basis 0
    - RightHandedWeyl.basis 0 ⊗ₜ[ℂ] RightHandedWeyl.basis 1
```

def RightHandedWeyl.metric [source]

The metric `ε^{dot a}^{dot a}` as a morphism `𝓕.V ⟶ (Rep ℂ SL(2,ℂ)) ⟶ rightHanded ⊗ rightHanded`, making manifest its invariance under the action of `SL(2,ℂ)`.

```
def RightHandedWeyl.metric : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
    (RightHandedWeyl.rep.tprod RightHandedWeyl.rep) where
```

lemma RightHandedWeyl.metric_apply_one [source]

```
lemma RightHandedWeyl.metric_apply_one :
    RightHandedWeyl.metric (1 : ℂ) = RightHandedWeyl.metricVal
```

def DualRightHandedWeyl.metricVal [source]

The metric `ε_{dot a}_{dot a}` as an element of `(dualRightHanded ⊗ dualRightHanded).V`.

```
def DualRightHandedWeyl.metricVal : DualRightHandedWeyl ⊗[ℂ] DualRightHandedWeyl
```

lemma DualRightHandedWeyl.metricVal_expand_tmul [source]

Expansion of `RightHandedWeyl.metricVal` into the left basis.

```
lemma DualRightHandedWeyl.metricVal_expand_tmul : DualRightHandedWeyl.metricVal =
    DualRightHandedWeyl.basis 0 ⊗ₜ[ℂ] DualRightHandedWeyl.basis 1 -
      DualRightHandedWeyl.basis 1 ⊗ₜ[ℂ] DualRightHandedWeyl.basis 0
```

def DualRightHandedWeyl.metric [source]

The metric `ε_{dot a}_{dot a}` as a morphism `↑_ (Rep ℂ SL(2,ℂ)) ⟶ dualRightHanded ⊗ dualRightHanded`, making manifest its invariance under the action of `SL(2,ℂ)`.

```
def DualRightHandedWeyl.metric : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
    (DualRightHandedWeyl.rep.tprod DualRightHandedWeyl.rep) where
```

lemma DualRightHandedWeyl.metric_apply_one [source]

```
lemma DualRightHandedWeyl.metric_apply_one :
    DualRightHandedWeyl.metric (1 : ℂ) = DualRightHandedWeyl.metricVal
```

lemma LeftHandedWeyl.contr_metric [source]

```
lemma LeftHandedWeyl.contr_metric :
    (TensorProduct.comm ℂ _ _ <|
      (TensorProduct.lid ℂ _).lTensor _ <|
      (LeftHandedWeyl.contr.toLinearMap.rTensor _).lTensor _ <|
      (TensorProduct.assoc ℂ _ _ _).symm.toLinearMap.lTensor _<|
      TensorProduct.assoc ℂ _ _ (_ ⊗[ℂ] _) <|
      (LeftHandedWeyl.metric 1) ⊗ₜ[ℂ] (DualLeftHandedWeyl.metric 1)) =
      LeftHandedWeyl.unit (1 : ℂ)
```

lemma DualLeftHandedWeyl.contr_metric [source]

```
lemma DualLeftHandedWeyl.contr_metric :
    (TensorProduct.comm ℂ _ _ <|
    (TensorProduct.lid ℂ _).lTensor _ <|
    (DualLeftHandedWeyl.contr.toLinearMap.rTensor _).lTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm.toLinearMap.lTensor _<|
    TensorProduct.assoc ℂ _ _ (_ ⊗[ℂ] _) <|
    (DualLeftHandedWeyl.metric 1) ⊗ₜ[ℂ] (LeftHandedWeyl.metric 1)) =
    DualLeftHandedWeyl.unit (1 : ℂ)
```

lemma RightHandedWeyl.contr_metric [source]

```
lemma RightHandedWeyl.contr_metric :
    (TensorProduct.comm ℂ _ _ <|
    (TensorProduct.lid ℂ _).lTensor _ <|
    (RightHandedWeyl.contr.toLinearMap.rTensor _).lTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm.toLinearMap.lTensor _<|
    TensorProduct.assoc ℂ _ _ (_ ⊗[ℂ] _) <|
    (RightHandedWeyl.metric 1) ⊗ₜ[ℂ] (DualRightHandedWeyl.metric 1)) =
    RightHandedWeyl.unit (1 : ℂ)
```

lemma DualRightHandedWeyl.contr_metric [source]

```
lemma DualRightHandedWeyl.contr_metric :
    (TensorProduct.comm ℂ _ _ <|
      (TensorProduct.lid ℂ _).lTensor _ <|
      (DualRightHandedWeyl.contr.toLinearMap.rTensor _).lTensor _ <|
      (TensorProduct.assoc ℂ _ _ _).symm.toLinearMap.lTensor _<|
      TensorProduct.assoc ℂ _ _ (_ ⊗[ℂ] _) <|
      (DualRightHandedWeyl.metric 1) ⊗ₜ[ℂ] (RightHandedWeyl.metric 1)) =
      DualRightHandedWeyl.unit (1 : ℂ)
```

2.114 Physlib.Relativity.Fermions.Weyl.RightHanded

Physlib/Relativity/Fermions/Weyl/RightHanded.lean on GitHub.

instance Nontrivial RightHandedWeyl [source]

```
instance : Nontrivial RightHandedWeyl
```

instance FiniteDimensional ℂ RightHandedWeyl [source]

```
instance : FiniteDimensional ℂ RightHandedWeyl
```

lemma finrank_eq_two [source]

Right-handed Weyl fermions form a two-dimensional complex vector space.

```
@[simp]
lemma finrank_eq_two : Module.finrank ℂ RightHandedWeyl = 2
```

lemma eq_zero_of_rep_eq_smul [source]

The representation `rep` has no common eigenvector.

```
lemma eq_zero_of_rep_eq_smul (ψ : RightHandedWeyl)
    (h : ∀ M : SL(2,ℂ), ∃ c : ℂ, rep M ψ = c • ψ) : ψ = 0
```

lemma eq_bot_or_eq_top_of_rep_mem [source]

A submodule of `RightHandedWeyl` stable under `rep` is either `⊥` or everything.

```
lemma eq_bot_or_eq_top_of_rep_mem (p : Submodule ℂ RightHandedWeyl)
    (h : ∀ (M : SL(2,ℂ)) (ψ : RightHandedWeyl), ψ ∈ p → rep M ψ ∈ p) :
    p = ⊥ ∨ p = ⊤
```

instance rep.IsIrreducible [source]

The representation `rep` of `SL(2, ℂ)` is irreducible.

```
instance : rep.IsIrreducible where
```

lemma end_eq_smul_id_of_commute_rep [source]

Schur's lemma for right-handed Weyl fermions.

```
lemma end_eq_smul_id_of_commute_rep {T : Module.End ℂ RightHandedWeyl}
    (hT : ∀ M : SL(2,ℂ), T ∘ₗ rep M = rep M ∘ₗ T) : ∃ μ : ℂ, T = μ • LinearMap.id
```

2.115 Physlib.Relativity.Fermions.Weyl.Unit

Physlib/Relativity/Fermions/Weyl/Unit.lean on GitHub.

def DualLeftHandedWeyl.unitVal [source]

The left-dual-left unit $\delta^a{}_a$ as an element of `(leftHanded ⊗ dualLeftHanded).V`.

```
def DualLeftHandedWeyl.unitVal : (LeftHandedWeyl ⊗[ℂ] DualLeftHandedWeyl)
```

lemma DualLeftHandedWeyl.unitVal_expand_tmul [source]

Expansion of `DualLeftHandedWeyl.unitVal` into the basis.

```
lemma DualLeftHandedWeyl.unitVal_expand_tmul : DualLeftHandedWeyl.unitVal =
    LeftHandedWeyl.basis 0 ⊗ₜ[ℂ] DualLeftHandedWeyl.basis 0 +
      LeftHandedWeyl.basis 1 ⊗ₜ[ℂ] DualLeftHandedWeyl.basis 1
```

def DualLeftHandedWeyl.unit [source]

The left-dual-left unit $\delta^a{}_a$ as a morphism `↥_ (Rep ℂ SL(2,ℂ)) ⟶ leftHanded ⊗ dualLeftHanded` , manifesting the invariance under the `SL(2,ℂ)` action.

```
def DualLeftHandedWeyl.unit : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
    (LeftHandedWeyl.rep.tprod DualLeftHandedWeyl.rep) where
```

lemma DualLeftHandedWeyl.unit_apply_one [source]

```
lemma DualLeftHandedWeyl.unit_apply_one :
    DualLeftHandedWeyl.unit (1 : ℂ) = DualLeftHandedWeyl.unitVal
```

def LeftHandedWeyl.unitVal [source]

The dual-left-left unit $\delta_a{}^a$ as an element of `(dualLeftHanded ⊗ leftHanded).V`.

```
def LeftHandedWeyl.unitVal : (DualLeftHandedWeyl ⊗[ℂ] LeftHandedWeyl)
```

lemma LeftHandedWeyl.unitVal_expand_tmul [source]

Expansion of `LeftHandedWeyl.unitVal` into the basis.

```
lemma LeftHandedWeyl.unitVal_expand_tmul : LeftHandedWeyl.unitVal =
    DualLeftHandedWeyl.basis 0 ⊗ₜ[ℂ] LeftHandedWeyl.basis 0 +
      DualLeftHandedWeyl.basis 1 ⊗ₜ[ℂ] LeftHandedWeyl.basis 1
```

def LeftHandedWeyl.unit [source]

The dual-left-left unit `δₐᵃ` *as a morphism* `↾_ (Rep ℂ SL(2,ℂ)) ⟶ dualLeftHanded ⊗ leftHanded , manifesting the invariance under the SL(2,ℂ) ` *action.*

```
def LeftHandedWeyl.unit :
    (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
      (DualLeftHandedWeyl.rep.tprod LeftHandedWeyl.rep) where
```

lemma LeftHandedWeyl.unit_apply_one [source]

Applying the morphism `LeftHandedWeyl.unit` *to* `1` *returns* `LeftHandedWeyl.unitVal`.

```
lemma LeftHandedWeyl.unit_apply_one : LeftHandedWeyl.unit (1 : ℂ) = LeftHandedWeyl.unitVal
```

def DualRightHandedWeyl.unitVal [source]

The right-dual-right unit `δ^{dot a}_{dot a}` *as an element of* `(rightHanded ⊗ dualRightHanded).V`.

```
def DualRightHandedWeyl.unitVal : RightHandedWeyl ⊗[ℂ] DualRightHandedWeyl
```

lemma DualRightHandedWeyl.unitVal_expand_tmul [source]

Expansion of `DualRightHandedWeyl.unitVal` *into the basis.*

```
lemma DualRightHandedWeyl.unitVal_expand_tmul : DualRightHandedWeyl.unitVal =
    RightHandedWeyl.basis 0 ⊗ₜ[ℂ] DualRightHandedWeyl.basis 0 +
      RightHandedWeyl.basis 1 ⊗ₜ[ℂ] DualRightHandedWeyl.basis 1
```

def DualRightHandedWeyl.unit [source]

The right-dual-right unit `δ^{dot a}_{dot a}` *as a morphism* `↾_ (Rep ℂ SL(2,ℂ)) ⟶ rightHanded ⊗ dualRightHanded`, *manifesting the invariance under the* `SL(2,ℂ)` *action.*

```
def DualRightHandedWeyl.unit : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
    (RightHandedWeyl.rep.tprod DualRightHandedWeyl.rep) where
```

lemma DualRightHandedWeyl.unit_apply_one [source]

```
lemma DualRightHandedWeyl.unit_apply_one :
    DualRightHandedWeyl.unit (1 : ℂ) = DualRightHandedWeyl.unitVal
```

def RightHandedWeyl.unitVal [source]

The dual-right-right unit `δ_{dot a}^{dot a}` *as an element of* `(rightHanded ⊗ dualRightHanded).V`.

```
def RightHandedWeyl.unitVal : (DualRightHandedWeyl ⊗[ℂ] RightHandedWeyl)
```

lemma RightHandedWeyl.unitVal_expand_tmul [source]

Expansion of `RightHandedWeyl.unitVal` into the basis.

```
lemma RightHandedWeyl.unitVal_expand_tmul : RightHandedWeyl.unitVal =
    DualRightHandedWeyl.basis 0 ⊗ₜ[ℂ] RightHandedWeyl.basis 0 +
      DualRightHandedWeyl.basis 1 ⊗ₜ[ℂ] RightHandedWeyl.basis 1
```

def RightHandedWeyl.unit [source]

The dual-right-right unit `δ_{dot a}^{dot a}` as a morphism `𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ dualRightHanded ⊗ rightHanded`, manifesting the invariance under the `SL(2,ℂ)` action.

```
def RightHandedWeyl.unit : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
    (DualRightHandedWeyl.rep.tprod RightHandedWeyl.rep) where
```

lemma RightHandedWeyl.unit_apply_one [source]

```
lemma RightHandedWeyl.unit_apply_one : RightHandedWeyl.unit (1 : ℂ) = RightHandedWeyl.unitVal
```

lemma LeftHandedWeyl.contr_unit [source]

Contraction on the right with `LeftHandedWeyl.unit` does nothing.

```
lemma LeftHandedWeyl.contr_unit (x : LeftHandedWeyl) :
    (TensorProduct.lid ℂ _ <|
    LeftHandedWeyl.contr.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm <|
    x ⊗ₜ[ℂ] (LeftHandedWeyl.unit (1 : ℂ))) = x
```

lemma DualLeftHandedWeyl.contr_unit [source]

Contraction on the right with `DualLeftHandedWeyl.unit` does nothing.

```
lemma DualLeftHandedWeyl.contr_unit (x : DualLeftHandedWeyl) :
    (TensorProduct.lid ℂ _ <|
    DualLeftHandedWeyl.contr.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm <|
    x ⊗ₜ[ℂ] (DualLeftHandedWeyl.unit (1 : ℂ))) = x
```

lemma RightHandedWeyl.contr_unit [source]

Contraction on the right with `RightHandedWeyl.unit` does nothing.

```
lemma RightHandedWeyl.contr_unit (x : RightHandedWeyl) :
    (TensorProduct.lid ℂ _ <|
    RightHandedWeyl.contr.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm <|
    x ⊗ₜ[ℂ] (RightHandedWeyl.unit (1 : ℂ))) = x
```

lemma DualRightHandedWeyl.contr_unit [source]

Contraction on the right with `DualRightHandedWeyl.unit` does nothing.

```
lemma DualRightHandedWeyl.contr_unit (x : DualRightHandedWeyl) :
    (TensorProduct.lid ℂ _ <|
    DualRightHandedWeyl.contr.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm <|
    x ⊗ₜ[ℂ] (DualRightHandedWeyl.unit (1 : ℂ))) = x
```

lemma LeftHandedWeyl.unit_symm [source]

```
lemma LeftHandedWeyl.unit_symm :
    LeftHandedWeyl.unit (1 : ℂ) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMap
    (TensorProduct.comm ℂ _ _ (DualLeftHandedWeyl.unit (1 : ℂ)))
```

lemma DualLeftHandedWeyl.unit_symm [source]

```
lemma DualLeftHandedWeyl.unit_symm :
    DualLeftHandedWeyl.unit (1 : ℂ) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMap
      (TensorProduct.comm ℂ _ _ (LeftHandedWeyl.unit (1 : ℂ)))
```

lemma RightHandedWeyl.unit_symm [source]

```
lemma RightHandedWeyl.unit_symm :
    RightHandedWeyl.unit (1 : ℂ) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMap
      (TensorProduct.comm ℂ _ _ (DualRightHandedWeyl.unit (1 : ℂ)))
```

lemma DualRightHandedWeyl.unit_symm [source]

```
lemma DualRightHandedWeyl.unit_symm :
    DualRightHandedWeyl.unit (1 : ℂ) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMap
      (TensorProduct.comm ℂ _ _ (RightHandedWeyl.unit (1 : ℂ)))
```

2.116 Physlib.Relativity.LorentzAlgebra.Basis

Physlib/Relativity/LorentzAlgebra/Basis.lean on GitHub.

lemma hasDerivAt_boost_tanh_zero [source]

The tangent at zero of the finite boost curve parametrized by rapidity is the negative of the named boost generator.

```
lemma hasDerivAt_boost_tanh_zero (i : Fin 3) :
    HasDerivAt (fun r : ℝ => (LorentzGroup.boost i (Real.tanh r) (Real.abs_tanh_lt_one r)).1)
      (-boostGenerator i) 0
```

2.117 Physlib.Relativity.LorentzGroup.Basic

Physlib/Relativity/LorentzGroup/Basic.lean on GitHub.

lemma star_toComplex_apply [source]

The entries of `toComplex Λ` are real.

```
@[simp]
lemma star_toComplex_apply (Λ : LorentzGroup d) (i j : Fin 1 ⊕ Fin d) :
    star (toComplex Λ i j) = toComplex Λ i j
```

2.118 Physlib.Relativity.LorentzGroup.Boosts.Basic

Physlib/Relativity/LorentzGroup/Boosts/Basic.lean on GitHub.

lemma γ_tanh [source]

The Lorentz factor expressed in terms of rapidity.

```
lemma γ_tanh (r : ℝ) : γ (Real.tanh r) = Real.cosh r
```

2.119 Physlib.Relativity.LorentzGroup.Restricted.FromBoostRotation

Physlib/Relativity/LorentzGroup/Restricted/FromBoostRotation.lean on GitHub.

lemma toVelocity_val [source]

```
lemma toVelocity_val {d} (Λ : LorentzGroup.restricted d) :
    (toVelocity Λ).1 = toVector Λ
```

2.120 Physlib.Relativity.PauliMatrices.AsTensor

Physlib/Relativity/PauliMatrices/AsTensor.lean on GitHub.

lemma asTensor_expand_complexVector_basis [source]

The expansion of `asTensor` into ComplexVector.basis basis vectors .

```
lemma asTensor_expand_complexVector_basis : asTensor =
    ComplexVector.basis (Sum.inl 0) ⊗ₜ leftRightToMatrix.symm (pauliBasis (Sum.inl 0))
    + ComplexVector.basis (Sum.inr 0) ⊗ₜ leftRightToMatrix.symm (pauliBasis (Sum.inr 0))
    + ComplexVector.basis (Sum.inr 1) ⊗ₜ leftRightToMatrix.symm (pauliBasis (Sum.inr 1))
    + ComplexVector.basis (Sum.inr 2) ⊗ₜ leftRightToMatrix.symm (pauliBasis (Sum.inr 2))
```

2.121 Physlib.Relativity.SL2C.Basic

Physlib/Relativity/SL2C/Basic.lean on GitHub.

lemma toLorentzGroup_eq_one_iff [source]

The kernel of the homomorphism from `SL(2, ℂ)` to the Lorentz group is `{1, -1}`.

```
lemma toLorentzGroup_eq_one_iff (M : SL(2, ℂ)) :
    toLorentzGroup M = 1 ↔ M.1 = 1 ∨ M.1 = -1
```

2.122 Physlib.Relativity.SL2C.CayleyKlein

Physlib/Relativity/SL2C/CayleyKlein.lean on GitHub.

def cayleyKlein [source]

The element `!![a, -conj b; b, conj a]` of `SL(2, ℂ)`, given by its Cayley–Klein parameters `a b` with `‖a‖² + ‖b‖² = 1`.

```
def cayleyKlein (a b : ℂ) (h : normSq a + normSq b = 1) : SL(2, ℂ)
```

lemma cayleyKlein_mem_unitaryGroup [source]

A Cayley–Klein element is unitary.

```
lemma cayleyKlein_mem_unitaryGroup (a b : ℂ) (h : normSq a + normSq b = 1) :
    (cayleyKlein a b h).1 ∈ unitaryGroup (Fin 2) ℂ
```

lemma toLorentzGroup_cayleyKlein_mem_rotations [source]

The image of a Cayley–Klein element is a rotation.

```
lemma toLorentzGroup_cayleyKlein_mem_rotations (a b : ℂ) (h : normSq a + normSq b = 1) :
    toLorentzGroup (cayleyKlein a b h) ∈ LorentzGroup.Rotations 3
```

def cayleyKleinRotation [source]

The rotation of the Cayley–Klein parameters `a b`, as an element of the Lorentz rotation subgroup.

```
def cayleyKleinRotation (a b : ℂ) (h : normSq a + normSq b = 1) : LorentzGroup.Rotations 3
```

lemma cayleyKleinRotation_inr_inr [source]

The spatial block of the rotation of the Cayley–Klein parameters `a b`.

```
lemma cayleyKleinRotation_inr_inr (a b : ℂ) (h : normSq a + normSq b = 1) (i j : Fin 3) :
    (cayleyKleinRotation a b h).1.1 (Sum.inr i) (Sum.inr j) =
      !![(a ^ 2 - b ^ 2).re, (a ^ 2 - b ^ 2).im, 2 * (a * starRingEnd ℂ b).re;
        -(a ^ 2 + b ^ 2).im, (a ^ 2 + b ^ 2).re, -2 * (a * starRingEnd ℂ b).im;
        -2 * (a * b).re, -2 * (a * b).im, normSq a - normSq b] i j
```

lemma exists_cayleyKleinRotation_apply_two [source]

Every unit vector is the image of the third axis under some Cayley–Klein rotation.

```
lemma exists_cayleyKleinRotation_apply_two (n : Fin 3 → ℝ)
    (hn : n 0 ^ 2 + n 1 ^ 2 + n 2 ^ 2 = 1) :
    ∃ a b, ∃ h : normSq a + normSq b = 1,
      ∀ i, (cayleyKleinRotation a b h).1.1 (Sum.inr i) (Sum.inr 2) = n i
```

2.123 `Physlib.Relativity.SL2C.Cover`

`Physlib/Relativity/SL2C/Cover.lean` on GitHub.

lemma exists_unitary_toLorentzGroup_eq [source]

Every rotation is the image of a unitary element of `SL(2, ℂ)`.

```
lemma exists_unitary_toLorentzGroup_eq (Λ : LorentzGroup.Rotations 3) :
    ∃ M : SL(2, ℂ), M.1 ∈ unitaryGroup (Fin 2) ℂ ∧ toLorentzGroup M = Λ
```

lemma exists_toVector_toLorentzGroup_eq [source]

Every velocity is the image of the time axis under some element of `SL(2, ℂ)`.

```
lemma exists_toVector_toLorentzGroup_eq (u : Velocity 3) :
    ∃ M : SL(2, ℂ), LorentzGroup.toVector (toLorentzGroup M) = u.1
```

lemma toRestrictedLorentzGroup_surjective [source]

The homomorphism from `SL(2, ℂ)` to the restricted Lorentz group is surjective.

```
lemma toRestrictedLorentzGroup_surjective : Function.Surjective toRestrictedLorentzGroup
```

2.124 Physlib.Relativity.SpeedOfLight

Physlib/Relativity/SpeedOfLight.lean on GitHub.

instance instPositiveRealUnitCore [source]

```
instance instPositiveRealUnitCore : PositiveRealUnitCore SpeedOfLight where
```

lemma positiveRealUnitCore_val [source]

The magnitude supplied to `PositiveRealUnitCore` is the underlying value, so that the generic positive-real lemmas apply to `SpeedOfLight`.

```
@[simp]
lemma positiveRealUnitCore_val (c : SpeedOfLight) : PositiveRealUnitCore.val c = c.val
```

def toWithDim [source]

The speed of light as a quantity carrying the dimension of length over time, in nonnegative real values. The underlying value is the same as that of the `SpeedOfLight`.

```
def toWithDim (c : SpeedOfLight) : WithDim (L⌈ * T⌈⁻¹) ℝ≥0
```

lemma toWithDim_val [source]

The underlying value of the speed of light as a dimensioned quantity agrees with the original value.

```
@[simp]
lemma toWithDim_val (c : SpeedOfLight) : (c.toWithDim.val : ℝ) = c.val
```

lemma toWithDim_val_pos [source]

The speed of light as a dimensioned quantity is positive.

```
@[simp]
lemma toWithDim_val_pos (c : SpeedOfLight) : 0 < c.toWithDim.val
```

instance UnitDependent SpeedOfLight [source]

The speed of light scales under a change of units by the scale factor of the dimension of length over time.

```
noncomputable instance : UnitDependent SpeedOfLight where
```

lemma scaleUnit_val [source]

Scaling a speed of light multiplies its value by the scale factor of the dimension of length over time.

```
@[simp]
lemma scaleUnit_val (c : SpeedOfLight) (u1 u2 : LTMCTUnitChoices) :
    (scaleUnit u1 u2 c).val = (u1.dimScale u2 (L⌈ * T⌈⁻¹) : ℝ) * c.val
```

lemma toWithDim_scaleUnit [source]

The map `toWithDim` commutes with changes of units.

```
@[simp]
lemma toWithDim_scaleUnit (c : SpeedOfLight) (u1 u2 : LTMCTUnitChoices) :
    (scaleUnit u1 u2 c).toWithDim = scaleUnit u1 u2 c.toWithDim
```

2.125 Physlib.Relativity.Tensors.Basic

Physlib/Relativity/Tensors/Basic.lean on GitHub.

lemma permT_basis_repr_of_id [source]

A reindexing whose slot map is pointwise the identity only transports the labels.

```
lemma permT_basis_repr_of_id {n : ℕ} {c c1 : Fin n → C} {σ : Fin n → Fin n}
    (hσ : ∀ i, σ i = i) (h : IsReindexing c c1 σ) (t : S.Tensor c)
    (φ : ComponentIdx (S := S) c1) :
    (basis c1).repr (permT σ h t) φ =
    (basis c).repr t (fun i => basisIdxCongr (by rw [← h.2 i, hσ]) (φ i))
```

2.126 Physlib.Relativity.Tensors.ComplexTensor.Basic

Physlib/Relativity/Tensors/ComplexTensor/Basic.lean on GitHub.

abbrev basisIdx [source]

The type indexing the standard basis of each complex Lorentz vector space: `Fin 2` for the Weyl colors and `Fin 1 ⊕ Fin 3` for the vector colors.

```
abbrev basisIdx : Color → Type
  | Color.upL => Fin 2
  | Color.downL => Fin 2
  | Color.upR => Fin 2
  | Color.downR => Fin 2
  | Color.up => Fin 1 ⊕ Fin 3
  | Color.down => Fin 1 ⊕ Fin 3
```

instance basisIdxFintype [source]

```
instance basisIdxFintype : ∀ c, Fintype (basisIdx c)
  | Color.upL => inferInstance
  | Color.downL => inferInstance
  | Color.upR => inferInstance
  | Color.downR => inferInstance
  | Color.up => inferInstance
  | Color.down => inferInstance
```

instance basisIdxDecidableEq [source]

```
instance basisIdxDecidableEq : ∀ c, DecidableEq (basisIdx c)
  | Color.upL => inferInstance
  | Color.downL => inferInstance
  | Color.upR => inferInstance
  | Color.downR => inferInstance
  | Color.up => inferInstance
  | Color.down => inferInstance
```

instance instWithMetric [source]

The metrics of the complex Lorentz tensors: ε on the four Weyl colors and η on the two vector colors.

```
instance instWithMetric : complexLorentzTensor.WithMetric where
```

def basisIdxTauEquiv [source]

A color and its dual carry the same basis index type.

```
def basisIdxTauEquiv : (c : complexLorentzTensor.Color) →
    basisIdx (complexLorentzTensor.τ c) ≃ basisIdx c
  | Color.upL => Equiv.refl _
  | Color.downL => Equiv.refl _
  | Color.upR => Equiv.refl _
  | Color.downR => Equiv.refl _
  | Color.up => Equiv.refl _
  | Color.down => Equiv.refl _
```

lemma isContrDualMatching_basisIdxTauEquiv [source]

At every complex Lorentz color, the bases at the color and its dual are dual bases, with labels matched by the identity.

```
lemma isContrDualMatching_basisIdxTauEquiv (c : complexLorentzTensor.Color) :
    IsContrDualMatching complexLorentzTensor c (basisIdxTauEquiv c)
```

instance HasContrDualBases complexLorentzTensor [source]

Complex Lorentz tensors have dual bases under contraction, with the identity matching.

```
instance : HasContrDualBases complexLorentzTensor where
```

lemma contrDualIdxEquiv_eq_basisIdxTauEquiv [source]

The dual-label matching of complex Lorentz tensors is the identity.

```
@[simp]
lemma contrDualIdxEquiv_eq_basisIdxTauEquiv (c : complexLorentzTensor.Color) :
    HasContrDualBases.contrDualIdxEquiv complexLorentzTensor c = basisIdxTauEquiv c
```

def bar [source]

Complex conjugation of colours: it swaps handedness and fixes the vector colours.

```
def Color.bar : Color → Color
  | Color.upL => Color.upR
  | Color.downL => Color.downR
  | Color.upR => Color.upL
  | Color.downR => Color.downL
  | Color.up => Color.up
  | Color.down => Color.down
```

def complexLorentzConjTensor [source]

The complex Lorentz tensors as a conjugation species: conjugation swaps left- and right-handed Weyl colours and fixes the vector colours.

```
def complexLorentzConjTensor : ConjTensorSpecies ℂ Color SL(2, ℂ) modules
    basisIdx rep basis where
```

2.127 Physlib.Relativity.Tensors.ComplexTensor.Metrics.Basic

Physlib/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean on GitHub.

lemma coMetric_eq_complexCoVector_basis [source]

```
lemma coMetric_eq_complexCoVector_basis : η' =
    fromPairT (ComplexCoVector.basis (Sum.inl 0) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inl 0))
    - fromPairT (ComplexCoVector.basis (Sum.inr 0) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inr 0))
    - fromPairT (ComplexCoVector.basis (Sum.inr 1) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inr 1))
    - fromPairT (ComplexCoVector.basis (Sum.inr 2) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inr 2))
```

lemma coMetric_eq_complexCoVector_basisFin4 [source]

```
lemma coMetric_eq_complexCoVector_basisFin4 : η' =
    fromPairT (ComplexCoVector.basisFin4 0 ⊗ₜ[ℂ] ComplexCoVector.basisFin4 0)
    - fromPairT (ComplexCoVector.basisFin4 1 ⊗ₜ[ℂ] ComplexCoVector.basisFin4 1)
    - fromPairT (ComplexCoVector.basisFin4 2 ⊗ₜ[ℂ] ComplexCoVector.basisFin4 2)
    - fromPairT (ComplexCoVector.basisFin4 3 ⊗ₜ[ℂ] ComplexCoVector.basisFin4 3)
```

lemma contrMetric_eq_complexVector_basis [source]

```
lemma contrMetric_eq_complexVector_basis : η =
    fromPairT (ComplexVector.basis (Sum.inl 0) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inl 0))
    - fromPairT (ComplexVector.basis (Sum.inr 0) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inr 0))
    - fromPairT (ComplexVector.basis (Sum.inr 1) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inr 1))
    - fromPairT (ComplexVector.basis (Sum.inr 2) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inr 2))
```

lemma contrMetric_eq_complexVector_basisFin4 [source]

```
lemma contrMetric_eq_complexVector_basisFin4 : η =
    fromPairT (ComplexVector.basisFin4 0 ⊗ₜ[ℂ] ComplexVector.basisFin4 0)
    - fromPairT (ComplexVector.basisFin4 1 ⊗ₜ[ℂ] ComplexVector.basisFin4 1)
    - fromPairT (ComplexVector.basisFin4 2 ⊗ₜ[ℂ] ComplexVector.basisFin4 2)
    - fromPairT (ComplexVector.basisFin4 3 ⊗ₜ[ℂ] ComplexVector.basisFin4 3)
```

2.128 Physlib.Relativity.Tensors.ComplexTensor.Metrics.Pre

Physlib/Relativity/Tensors/ComplexTensor/Metrics/Pre.lean on GitHub.

def ComplexVector.metricVal [source]

The metric η^{aa} as an element of `(complexContr ⊗ complexContr).V`.

```
def ComplexVector.metricVal : (ComplexVector ⊗[ℂ] ComplexVector)
```

lemma ComplexVector.metricVal_expand_tmul [source]

The expansion of `ComplexVector.metricVal` into basis vectors.

```
lemma ComplexVector.metricVal_expand_tmul : ComplexVector.metricVal =
    ComplexVector.basis (Sum.inl 0) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inl 0)
    - ComplexVector.basis (Sum.inr 0) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inr 0)
    - ComplexVector.basis (Sum.inr 1) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inr 1)
    - ComplexVector.basis (Sum.inr 2) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inr 2)
```

def ComplexVector.metric [source]

The metric η^{aa} as a morphism `↾_ (Rep ℂ SL(2,ℂ)) ⟶ complexContr ⊗ complexContr`, making its invariance under the action of `SL(2,ℂ)`.

```
def ComplexVector.metric : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
    (ComplexVector.SL2CRep.tprod ComplexVector.SL2CRep) where
```

lemma ComplexVector.metric_apply_one [source]

```
lemma ComplexVector.metric_apply_one : ComplexVector.metric (1 : ℂ) = ComplexVector.metricVal
```

def ComplexCoVector.metricVal [source]

The metric η_{ii} as an element of `(complexCo ⊗ complexCo).V`.

```
def ComplexCoVector.metricVal : (ComplexCoVector ⊗[ℂ] ComplexCoVector)
```

lemma ComplexCoVector.metricVal_expand_tmul [source]

The expansion of `ComplexCoVector.metricVal` into basis vectors.

```
lemma ComplexCoVector.metricVal_expand_tmul : ComplexCoVector.metricVal =
    ComplexCoVector.basis (Sum.inl 0) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inl 0)
    - ComplexCoVector.basis (Sum.inr 0) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inr 0)
    - ComplexCoVector.basis (Sum.inr 1) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inr 1)
    - ComplexCoVector.basis (Sum.inr 2) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inr 2)
```

def ComplexCoVector.metric [source]

The metric η_{ii} as a morphism $\mathbb{1} \to$ (Rep ℂ SL(2,ℂ)) $\otimes$ complexCo $\otimes$ complexCo, making its invariance under the action of SL(2,ℂ).

```
def ComplexCoVector.metric : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
    (ComplexCoVector.SL2CRep.tprod ComplexCoVector.SL2CRep) where
```

lemma ComplexCoVector.metric_apply_one [source]

```
lemma ComplexCoVector.metric_apply_one :
    ComplexCoVector.metric (1 : ℂ) = ComplexCoVector.metricVal
```

lemma ComplexVector.contr_metric [source]

```
lemma ComplexVector.contr_metric :
    (TensorProduct.comm ℂ _ _ <|
      (TensorProduct.lid ℂ _).lTensor _ <|
      (ComplexVector.contr.toLinearMap.rTensor _).lTensor _ <|
      (TensorProduct.assoc ℂ _ _ _).symm.toLinearMap.lTensor _<|
      TensorProduct.assoc ℂ _ _ (_ ⊗[ℂ] _) <|
      (ComplexVector.metric 1) ⊗ₜ[ℂ] (ComplexCoVector.metric 1)) = ComplexVector.unit (1 : ℝ)
```

lemma ComplexCoVector.contr_metric [source]

```
lemma ComplexCoVector.contr_metric :
    (TensorProduct.comm ℂ _ _ <|
      (TensorProduct.lid ℂ _).lTensor _ <|
      (ComplexCoVector.contr.toLinearMap.rTensor _).lTensor _ <|
      (TensorProduct.assoc ℂ _ _ _).symm.toLinearMap.lTensor _<|
      TensorProduct.assoc ℂ _ _ (_ ⊗[ℂ] _) <|
      (ComplexCoVector.metric 1) ⊗ₜ[ℂ] (ComplexVector.metric 1)) =
      ComplexCoVector.unit (1 : ℝ)
```

2.129 Physlib.Relativity.Tensors.ComplexTensor.Units.Basic

Physlib/Relativity/Tensors/ComplexTensor/Units/Basic.lean on GitHub.

lemma coContrUnit_eq_complexCoVector_basis_complexVector_basis [source]

```
lemma coContrUnit_eq_complexCoVector_basis_complexVector_basis : δ' =
    ∑ i, fromPairT (ComplexCoVector.basis i ⊗ₜ[ℂ] ComplexVector.basis i)
```

lemma coContrUnit_eq_complexCoVector_basisFin4_complexVector_basisFin4 [source]

```
lemma coContrUnit_eq_complexCoVector_basisFin4_complexVector_basisFin4 : δ' =
    ∑ i, fromPairT (ComplexCoVector.basisFin4 i ⊗ₜ[ℂ] ComplexVector.basisFin4 i)
```

lemma contrCoUnit_eq_complexVector_basis_complexCoVector_basis [source]

```
lemma contrCoUnit_eq_complexVector_basis_complexCoVector_basis : δ =
    ∑ i, fromPairT (ComplexVector.basis i ⊗ₜ[ℂ] ComplexCoVector.basis i)
```

lemma contrCoUnit_eq_complexVector_basisFin4_complexCoVector_basisFin4 [source]

```
lemma contrCoUnit_eq_complexVector_basisFin4_complexCoVector_basisFin4 : δ =
    ∑ i, fromPairT (ComplexVector.basisFin4 i ⊗ₜ[ℂ] ComplexCoVector.basisFin4 i)
```

2.130 Physlib.Relativity.Tensors.ComplexTensor.Units.Pre

Physlib/Relativity/Tensors/ComplexTensor/Units/Pre.lean on GitHub.

def ComplexCoVector.unitVal [source]

The contra-co unit for complex lorentz vectors. Usually denoted $\delta^{i}{}_{i}$.

```
def ComplexCoVector.unitVal : ComplexVector ⊗[ℂ] ComplexCoVector
```

lemma ComplexCoVector.unitVal_expand_tmul [source]

Expansion of `ComplexCoVector.unitVal` into basis.

```
lemma ComplexCoVector.unitVal_expand_tmul : ComplexCoVector.unitVal =
    ComplexVector.basis (Sum.inl 0) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inl 0)
    + ComplexVector.basis (Sum.inr 0) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inr 0)
    + ComplexVector.basis (Sum.inr 1) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inr 1)
    + ComplexVector.basis (Sum.inr 2) ⊗ₜ[ℂ] ComplexCoVector.basis (Sum.inr 2)
```

lemma ComplexCoVector.unitVal_eq_sum_tmul [source]

```
lemma ComplexCoVector.unitVal_eq_sum_tmul : ComplexCoVector.unitVal =
    ∑ i, ComplexVector.basis i ⊗ₜ[ℂ] ComplexCoVector.basis i
```

def ComplexCoVector.unit [source]

The contra-co unit for complex lorentz vectors as a morphism `𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ complexContr ⊗ complexCo`, manifesting the invariance under the `SL(2, ℂ)` action.

```
def ComplexCoVector.unit : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
    (ComplexVector.SL2CRep.tprod ComplexCoVector.SL2CRep) where
```

lemma ComplexCoVector.unit_apply_one [source]

```
lemma ComplexCoVector.unit_apply_one : ComplexCoVector.unit (1 : ℂ) = ComplexCoVector.unitVal
```

def ComplexVector.unitVal [source]

The co-contra unit for complex lorentz vectors. Usually denoted $\delta_{i}{}^{i}$.

```
def ComplexVector.unitVal : ComplexCoVector ⊗[ℂ] ComplexVector
```

lemma ComplexVector.unitVal_expand_tmul [source]

Expansion of `ComplexVector.unitVal` into basis.

```
lemma ComplexVector.unitVal_expand_tmul : ComplexVector.unitVal =
    ComplexCoVector.basis (Sum.inl 0) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inl 0)
    + ComplexCoVector.basis (Sum.inr 0) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inr 0)
    + ComplexCoVector.basis (Sum.inr 1) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inr 1)
    + ComplexCoVector.basis (Sum.inr 2) ⊗ₜ[ℂ] ComplexVector.basis (Sum.inr 2)
```

lemma ComplexVector.unitVal_eq_sum_tmul [source]

```
lemma ComplexVector.unitVal_eq_sum_tmul : ComplexVector.unitVal =
    ∑ i, ComplexCoVector.basis i ⊗ₜ[ℂ] ComplexVector.basis i
```

def ComplexVector.unit [source]

The co-contra unit for complex lorentz vectors as a morphism `𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ complexCo ⊗ complexContr`, manifesting the invariance under the `SL(2, ℂ)` action.

```
def ComplexVector.unit : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
    (ComplexCoVector.SL2CRep.tprod ComplexVector.SL2CRep) where
```

lemma ComplexVector.unit_apply_one [source]

```
lemma ComplexVector.unit_apply_one : ComplexVector.unit (1 : ℂ) = ComplexVector.unitVal
```

lemma ComplexCoVector.contr_unit [source]

Contraction on the right with `ComplexCoVector.unit` does nothing.

```
lemma ComplexCoVector.contr_unit (x : ComplexCoVector) :
    (TensorProduct.lid ℂ _ <|
    ComplexCoVector.contr.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm <|
    x ⊗ₜ[ℂ] (ComplexCoVector.unit (1 : ℂ))) = x
```

lemma ComplexVector.contr_unit [source]

Contraction on the right with `ComplexVector.unit`.

```
lemma ComplexVector.contr_unit (x : ComplexVector) :
    (TensorProduct.lid ℂ _ <|
    ComplexVector.contr.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm <|
    x ⊗ₜ[ℂ] (ComplexVector.unit (1 : ℂ))) = x
```

lemma ComplexCoVector.unit_symm [source]

```
lemma ComplexCoVector.unit_symm :
    ComplexCoVector.unit (1 : ℂ) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMap
      (TensorProduct.comm ℂ _ _ (ComplexVector.unit (1 : ℂ)))
```

lemma ComplexVector.unit_symm [source]

```
lemma ComplexVector.unit_symm :
    (ComplexVector.unit (1 : ℂ)) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMap
      (TensorProduct.comm ℂ _ _ (ComplexCoVector.unit (1 : ℂ)))
```

2.131 Physlib.Relativity.Tensors.ComplexTensor.Vector.Basic

Physlib/Relativity/Tensors/ComplexTensor/Vector/Basic.lean on GitHub.

def ComplexVector.basis [source]

The standard basis of complex contravariant Lorentz vectors.

```
def ComplexVector.basis : Basis (Fin 1 ⊕ Fin 3) ℂ ComplexVector
```

lemma ComplexVector.basis_toFin13ℂ [source]

```
@[simp]
lemma ComplexVector.basis_toFin13ℂ (i :Fin 1 ⊕ Fin 3) :
    (ComplexVector.basis i).toFin13ℂ = Pi.single i 1
```

lemma ComplexVector.basis_ρ_apply [source]

```
@[simp]
lemma ComplexVector.basis_ρ_apply (M : SL(2,ℂ)) (i j : Fin 1 ⊕ Fin 3) :
    (LinearMap.toMatrix ComplexVector.basis ComplexVector.basis) (ComplexVector.SL2CRep M) i j
    =
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M)) i j
```

lemma basis_ρ_val [source]

```
lemma ComplexVector.basis_ρ_val (M : SL(2,ℂ)) (v : ComplexVector) :
    ((ComplexVector.SL2CRep M) v).val =
    LorentzGroup.toComplex (SL2C.toLorentzGroup M) *ᵥ v.val
```

def ComplexVector.basisFin4 [source]

The standard basis of complex contravariant Lorentz vectors indexed by `Fin 4`.

```
def ComplexVector.basisFin4 : Basis (Fin 4) ℂ ComplexVector
```

lemma ComplexVector.basisFin4_eq_reindex [source]

```
lemma ComplexVector.basisFin4_eq_reindex :
    ComplexVector.basisFin4 = ComplexVector.basis.reindex finSumFinEquiv
```

lemma ComplexVector.basis_reindex_apply_eq_fin4 [source]

```
lemma ComplexVector.basis_reindex_apply_eq_fin4 (j : Fin 4) :
    (ComplexVector.basis.reindex finSumFinEquiv) j = ComplexVector.basisFin4 j
```

lemma ComplexVector.basisFin4_apply_zero [source]

```
@[simp]
lemma ComplexVector.basisFin4_apply_zero :
    ComplexVector.basisFin4 0 = ComplexVector.basis (Sum.inl 0)
```

lemma ComplexVector.basisFin4_apply_one [source]

```
@[simp]
lemma ComplexVector.basisFin4_apply_one :
    ComplexVector.basisFin4 1 = ComplexVector.basis (Sum.inr 0)
```

lemma ComplexVector.basisFin4_apply_two [source]

```
@[simp]
lemma ComplexVector.basisFin4_apply_two :
    ComplexVector.basisFin4 2 = ComplexVector.basis (Sum.inr 1)
```

lemma ComplexVector.basisFin4_apply_three [source]

```
@[simp]
lemma ComplexVector.basisFin4_apply_three :
    ComplexVector.basisFin4 3 = ComplexVector.basis (Sum.inr 2)
```

lemma basisFin4_apply_succ [source]

```
@[simp]
lemma ComplexVector.basisFin4_apply_succ (i : Fin 3) :
    ComplexVector.basisFin4 i.succ = ComplexVector.basis (Sum.inr i)
```

def ComplexCoVector.basis [source]

The standard basis of complex covariant Lorentz vectors.

```
def ComplexCoVector.basis : Basis (Fin 1 ⊕ Fin 3) ℂ ComplexCoVector
```

lemma ComplexCoVector.basis_toFin13ℂ [source]

```
@[simp]
lemma ComplexCoVector.basis_toFin13ℂ (i :Fin 1 ⊕ Fin 3) :
    (ComplexCoVector.basis i).toFin13ℂ = Pi.single i 1
```

lemma ComplexCoVector.basis_ρ_apply [source]

```
@[simp]
lemma ComplexCoVector.basis_ρ_apply (M : SL(2,ℂ)) (i j : Fin 1 ⊕ Fin 3) :
    (LinearMap.toMatrix ComplexCoVector.basis ComplexCoVector.basis)
      (ComplexCoVector.SL2CRep M) i j =
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹ᵀ i j
```

lemma SL2CRep_val [source]

```
lemma ComplexCoVector.SL2CRep_val (M : SL(2,ℂ)) (v : ComplexCoVector) :
    ((ComplexCoVector.SL2CRep M) v).val =
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻¹ᵀ *ᵥ v.val
```

def ComplexCoVector.basisFin4 [source]

The standard basis of complex covariant Lorentz vectors indexed by `Fin 4`.

```
def ComplexCoVector.basisFin4 : Basis (Fin 4) ℂ ComplexCoVector
```

lemma ComplexCoVector.basisFin4_eq_reindex [source]

```
lemma ComplexCoVector.basisFin4_eq_reindex :
    ComplexCoVector.basisFin4 = ComplexCoVector.basis.reindex finSumFinEquiv
```

lemma ComplexCoVector.basis_reindex_apply_eq_fin4 [source]

```
lemma ComplexCoVector.basis_reindex_apply_eq_fin4 (j : Fin 4) :
    (ComplexCoVector.basis.reindex finSumFinEquiv) j = ComplexCoVector.basisFin4 j
```

lemma ComplexCoVector.basisFin4_apply_zero [source]

```
@[simp]
lemma ComplexCoVector.basisFin4_apply_zero :
    ComplexCoVector.basisFin4 0 = ComplexCoVector.basis (Sum.inl 0)
```

lemma ComplexCoVector.basisFin4_apply_one [source]

```
@[simp]
lemma ComplexCoVector.basisFin4_apply_one :
    ComplexCoVector.basisFin4 1 = ComplexCoVector.basis (Sum.inr 0)
```

lemma ComplexCoVector.basisFin4_apply_two [source]

```
@[simp]
lemma ComplexCoVector.basisFin4_apply_two :
    ComplexCoVector.basisFin4 2 = ComplexCoVector.basis (Sum.inr 1)
```

lemma ComplexCoVector.basisFin4_apply_three [source]

```
@[simp]
lemma ComplexCoVector.basisFin4_apply_three :
    ComplexCoVector.basisFin4 3 = ComplexCoVector.basis (Sum.inr 2)
```

lemma ComplexVector.basis_of_real [source]

```
lemma ComplexVector.basis_of_real (i : Fin 1 ⊕ Fin 3) :
    (ComplexVector.basis i) = inclCongrRealLorentz (ContrMod.stdBasis i)
```

lemma ComplexCoVector.basis_of_real [source]

```
lemma ComplexCoVector.basis_of_real (i : Fin 1 ⊕ Fin 3) :
    (ComplexCoVector.basis i) = inclCoRealLorentz (CoMod.stdBasis i)
```

2.132 Physlib.Relativity.Tensors.ComplexTensor.Vector.Contraction

Physlib/Relativity/Tensors/ComplexTensor/Vector/Contraction.lean on GitHub.

def ComplexVector.contrBi [source]

The bi-linear map corresponding to contraction of a contravariant Lorentz vector with a covariant Lorentz vector.

```
def ComplexVector.contrBi : ComplexVector →ₗ[ℂ] ComplexCoVector →ₗ[ℂ] ℂ where
```

def ComplexCoVector.contrBi [source]

The bi-linear map corresponding to contraction of a covariant Lorentz vector with a contravariant Lorentz vector.

```
def ComplexCoVector.contrBi : ComplexCoVector →ₗ[ℂ] ComplexVector →ₗ[ℂ] ℂ where
```

def ComplexVector.contr [source]

The linear map from complexContr ⊗ complexCo to ℂ given by summing over components of contravariant Lorentz vector and covariant Lorentz vector in the standard basis (i.e. the dot product). In terms of index notation this is the contraction is $\psi^i \varphi_i$.

```
def ComplexVector.contr : (ComplexVector.SL2CRep.tprod ComplexCoVector.SL2CRep).IntertwiningMap
    (Representation.trivial ℂ SL(2,ℂ) ℂ) where
```

lemma ComplexVector.contr_tmul [source]

```
lemma ComplexVector.contr_tmul (ψ : ComplexVector) (φ : ComplexCoVector) :
    ComplexVector.contr (ψ ⊗ₜ φ) = ψ.toFin13ℂ ⬝ᵥ φ.toFin13ℂ
```

lemma ComplexVector.contr_basis [source]

```
lemma ComplexVector.contr_basis (i j : Fin 4) :
    ComplexVector.contr (ComplexVector.basisFin4 i ⊗ₜ ComplexCoVector.basisFin4 j) =
    if i.1 = j.1 then (1 : ℂ) else 0
```

lemma ComplexVector.contr_basis' [source]

```
lemma ComplexVector.contr_basis' (i j : Fin 1 ⊕ Fin 3) :
    ComplexVector.contr (ComplexVector.basis i ⊗ₜ ComplexCoVector.basis j) =
    if i = j then (1 : ℂ) else 0
```

def ComplexCoVector.contr [source]

The linear map from complexCo ⊗ complexContr to ℂ given by summing over components of covariant Lorentz vector and contravariant Lorentz vector in the standard basis (i.e. the dot product). In terms of index notation this is the contraction is $\varphi_i \psi^i$.

```
def ComplexCoVector.contr : (ComplexCoVector.SL2CRep.tprod ComplexVector.SL2CRep).
    IntertwiningMap
    (Representation.trivial ℂ SL(2,ℂ) ℂ) where
```

lemma ComplexCoVector.contr_tmul [source]

```
lemma ComplexCoVector.contr_tmul (φ : ComplexCoVector) (ψ : ComplexVector) :
    ComplexCoVector.contr (φ ⊗ₜ ψ) = φ.toFin13ℂ ⬝ᵥ ψ.toFin13ℂ
```

lemma ComplexCoVector.contr_basis [source]

```
lemma ComplexCoVector.contr_basis (i j : Fin 4) :
    ComplexCoVector.contr (ComplexCoVector.basisFin4 i ⊗ₜ ComplexVector.basisFin4 j) =
    if i.1 = j.1 then (1 : ℂ) else 0
```

lemma ComplexCoVector.contr_basis' [source]

```
lemma ComplexCoVector.contr_basis' (i j : Fin 1 ⊕ Fin 3) :
    ComplexCoVector.contr (ComplexCoVector.basis i ⊗ₜ ComplexVector.basis j) =
    if i = j then (1 : ℂ) else 0
```

lemma ComplexVector.contr_tmul_symm [source]

```
lemma ComplexVector.contr_tmul_symm (φ : ComplexVector) (ψ : ComplexCoVector) :
    ComplexVector.contr (φ ⊗ₜ ψ) = ComplexCoVector.contr (ψ ⊗ₜ φ)
```

lemma ComplexCoVector.contr_tmul_symm [source]

```
lemma ComplexCoVector.contr_tmul_symm (φ : ComplexCoVector) (ψ : ComplexVector) :
    ComplexCoVector.contr (φ ⊗ₜ ψ) = ComplexVector.contr (ψ ⊗ₜ φ)
```

2.133 Physlib.Relativity.Tensors.ComplexTensor.Vector.Modules

Physlib/Relativity/Tensors/ComplexTensor/Vector/Modules.lean on GitHub.

structure ComplexVector [source]

The module for contravariant (up-index) complex Lorentz vectors.

```
structure ComplexVector where
  val : Fin 1 ⊕ Fin 3 → ℂ
```

def ComplexVector.toFin13ℂFun [source]

The equivalence between `ComplexVector` and `Fin 1 ⊕ Fin 3 → ℂ`.

```
def toFin1ℂFun : ComplexVector ≃ (Fin 1 ⊕ Fin 3 → ℂ) where
```

instance AddCommMonoid ComplexVector [source]

The instance of `AddCommMonoid` on `ComplexVector` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.

```
instance : AddCommMonoid ComplexVector
```

instance AddCommGroup ComplexVector [source]

The instance of `AddCommGroup` on `ComplexVector` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.

```
instance : AddCommGroup ComplexVector
```

def ComplexVector.toFin1ℂAddEquiv [source]

The additive equivalence between `ComplexVector` and `Fin 1 ⊕ Fin 3 → ℂ`.

```
def toFin1ℂAddEquiv : ComplexVector ≃+ (Fin 1 ⊕ Fin 3 → ℂ)
```

instance Module ℂ ComplexVector [source]

The instance of `Module` on `ComplexVector` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.

```
instance : Module ℂ ComplexVector
```

lemma ComplexVector.ext [source]

```
@[ext]
lemma ext (ψ ψ' : ComplexVector) (h : ψ.val = ψ'.val) : ψ = ψ'
```

lemma ComplexVector.val_add [source]

```
@[simp]
lemma val_add (ψ ψ' : ComplexVector) : (ψ + ψ').val = ψ.val + ψ'.val
```

lemma ComplexVector.val_smul [source]

```
@[simp]
lemma val_smul (r : ℂ) (ψ : ComplexVector) : (r • ψ).val = r • ψ.val
```

def ComplexVector.toFin1ℂEquiv [source]

The linear equivalence between `ComplexVector` and `(Fin 1 ⊕ Fin 3 → ℂ)`.

```
@[simps!]
def toFin1ℂEquiv : ComplexVector ≃ₗ[ℂ] (Fin 1 ⊕ Fin 3 → ℂ)
```

abbrev ComplexVector.toFin13ℂ [source]

The underlying element of `Fin 1 ⊕ Fin 3 → ℂ` of a element in `ComplexVector` defined through the linear equivalence `toFin13ℂEquiv`.

```
abbrev toFin13ℂ (ψ : ComplexVector)
```

def ComplexVector.lorentzGroupRep [source]

The representation of the Lorentz group on `ComplexVector`.

```
def lorentzGroupRep : Representation ℂ (LorentzGroup 3) ComplexVector where
```

def ComplexVector.SL2CRep [source]

The representation of the SL(2, ℂ) on `ComplexVector` induced by the representation of the Lorentz group.

```
def SL2CRep : Representation ℂ SL(2, ℂ) ComplexVector
```

structure ComplexCoVector [source]

The module for covariant (up-index) complex Lorentz vectors.

```
structure ComplexCoVector where
  val : Fin 1 ⊕ Fin 3 → ℂ
```

def ComplexCoVector.toFin13ℂFun [source]

The equivalence between `ComplexCoVector` and `Fin 1 ⊕ Fin 3 → ℂ`.

```
def toFin13ℂFun : ComplexCoVector ≃ (Fin 1 ⊕ Fin 3 → ℂ) where
```

instance AddCommMonoid ComplexCoVector [source]

The instance of `AddCommMonoid` on `ComplexCoVector` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.

```
instance : AddCommMonoid ComplexCoVector
```

instance AddCommGroup ComplexCoVector [source]

The instance of `AddCommGroup` on `ComplexCoVector` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.

```
instance : AddCommGroup ComplexCoVector
```

def ComplexCoVector.toFin13ℂAddEquiv [source]

The additive equivalence between `ComplexCoVector` and `Fin 1 ⊕ Fin 3 → ℂ`.

```
def toFin13ℂAddEquiv : ComplexCoVector ≃+ (Fin 1 ⊕ Fin 3 → ℂ)
```

instance Module ℂ ComplexCoVector [source]

The instance of `Module` on `ComplexCoVector` defined via its equivalence with `Fin 1 ⊕ Fin 3 → ℂ`.

```
instance : Module ℂ ComplexCoVector
```

lemma ComplexCoVector.ext [source]

```
@[ext]
lemma ext (ψ ψ' : ComplexCoVector) (h : ψ.val = ψ'.val) : ψ = ψ'
```

lemma ComplexCoVector.val_add [source]

```
@[simp]
lemma val_add (ψ ψ' : ComplexCoVector) : (ψ + ψ').val = ψ.val + ψ'.val
```

lemma ComplexCoVector.val_smul [source]

```
@[simp]
lemma val_smul (r : ℂ) (ψ : ComplexCoVector) : (r • ψ).val = r • ψ.val
```

def ComplexCoVector.toFin13ℂEquiv [source]

The linear equivalence between `ComplexCoVector` and `(Fin 1 ⊕ Fin 3 → ℂ)`.

```
@[simps!]
def toFin13ℂEquiv : ComplexCoVector ≃ₗ[ℂ] (Fin 1 ⊕ Fin 3 → ℂ)
```

abbrev ComplexCoVector.toFin13ℂ [source]

The underlying element of `Fin 1 ⊕ Fin 3 → ℂ` of a element in `ComplexCoVector` defined through the linear equivalence `toFin13ℂEquiv`.

```
abbrev toFin13ℂ (ψ : ComplexCoVector)
```

def ComplexCoVector.lorentzGroupRep [source]

The representation of the Lorentz group on `ComplexCoVector`.

```
def lorentzGroupRep : Representation ℂ (LorentzGroup 3) ComplexCoVector where
```

def ComplexCoVector.SL2CRep [source]

The representation of the $SL(2, \mathbb{C})$ on `ComplexCoVector` induced by the representation of the Lorentz group.

```
def SL2CRep : Representation ℂ SL(2, ℂ) ComplexCoVector
```

2.134 Physlib.Relativity.Tensors.ComponentIdx.Insert

`Physlib/Relativity/Tensors/ComponentIdx/Insert.lean` on GitHub.

def insert [source]

Splitting a component index at a named slot: the label at slot `i`, together with a component index for the colours of the remaining slots. The dependent counterpart of `Fin.cons`, and the single-slot analogue of the two-slot `DropPairSection.ofFinEquiv`.

```
def insert {n : ℕ} {c : Fin (n + 1) → C} (i : Fin (n + 1)) :
    basisIdx (c i) × ComponentIdx (S := S) (c ∘ i.succAbove) ≃ ComponentIdx (S := S) c
```

lemma insert_apply_self [source]

```
@[simp]
lemma insert_apply_self {n : ℕ} {c : Fin (n + 1) → C} (i : Fin (n + 1))
    (x : basisIdx (c i) × ComponentIdx (S := S) (c ∘ i.succAbove)) :
    insert (S := S) i x i = x.1
```

lemma insert_apply_succAbove [source]

```
@[simp]
lemma insert_apply_succAbove {n : ℕ} {c : Fin (n + 1) → C} (i : Fin (n + 1))
    (x : basisIdx (c i) × ComponentIdx (S := S) (c ∘ i.succAbove)) (m : Fin n) :
    insert (S := S) i x (i.succAbove m) = x.2 m
```

2.135 Physlib.Relativity.Tensors.ComponentIdx.Pair

Physlib/Relativity/Tensors/ComponentIdx/Pair.lean on GitHub.

def pair [source]

The equivalence between component indices for two colors and pairs of basis indices of those colors.

```
def ComponentIdx.pair {cs : Fin 2 → C} :
    ComponentIdx (S := S) cs ≃ basisIdx (cs 0) × basisIdx (cs 1)
```

lemma pair_apply [source]

```
lemma ComponentIdx.pair_apply {cs : Fin 2 → C} (b : ComponentIdx (S := S) cs) :
    ComponentIdx.pair (S := S) b = (b 0, b 1)
```

lemma pair_symm_apply_zero [source]

```
@[simp]
lemma ComponentIdx.pair_symm_apply_zero {cs : Fin 2 → C}
    (x : basisIdx (cs 0) × basisIdx (cs 1)) :
    (ComponentIdx.pair (S := S) (cs := cs)).symm x 0 = x.1
```

lemma pair_symm_apply_one [source]

```
@[simp]
lemma ComponentIdx.pair_symm_apply_one {cs : Fin 2 → C}
    (x : basisIdx (cs 0) × basisIdx (cs 1)) :
    (ComponentIdx.pair (S := S) (cs := cs)).symm x 1 = x.2
```

2.136 Physlib.Relativity.Tensors.ComponentIdx.Product

Physlib/Relativity/Tensors/ComponentIdx/Product.lean on GitHub.

lemma prod_apply_fst [source]

```
lemma ComponentIdx.prod_apply_fst {n1 n2 : ℕ} {c : Fin n1 → C} {c1 : Fin n2 → C}
    (p : ComponentIdx (S := S) (Fin.append c c1)) (i : Fin n1) :
    p.prod.1 i =
      basisIdxCongr (by simp) (p (Fin.castAdd n2 i))
```

lemma prod_apply_snd [source]

```
lemma ComponentIdx.prod_apply_snd {n1 n2 : ℕ} {c : Fin n1 → C} {c1 : Fin n2 → C}
    (p : ComponentIdx (S := S) (Fin.append c c1)) (i : Fin n2) :
    p.prod.2 i =
      basisIdxCongr (by simp) (p (Fin.natAdd n1 i))
```

lemma prod_symm_heq_castAdd [source]

`ComponentIdx.prod_symm_castAdd` at a slot only propositionally equal to `Fin.castAdd n2 a`, up to `HEq`.

```
lemma ComponentIdx.prod_symm_heq_castAdd {n1 n2 : ℕ} {c : Fin n1 → C} {c1 : Fin n2 → C}
    (p : ComponentIdx (S := S) c) (q : ComponentIdx (S := S) c1) {x : Fin (n1 + n2)} {a : Fin
    n1}
    (hx : x = Fin.castAdd n2 a) : ComponentIdx.prod.symm (p, q) x ≍ p a
```

lemma prod_symm_heq_natAdd [source]

`ComponentIdx.prod_symm_natAdd` at a slot only propositionally equal to `Fin.natAdd n1 a`, up to `HEq`.

```
lemma ComponentIdx.prod_symm_heq_natAdd {n1 n2 : ℕ} {c : Fin n1 → C} {c1 : Fin n2 → C}
    (p : ComponentIdx (S := S) c) (q : ComponentIdx (S := S) c1) {x : Fin (n1 + n2)} {a : Fin
    n2}
    (hx : x = Fin.natAdd n1 a) : ComponentIdx.prod.symm (p, q) x ≍ q a
```

2.137 Physlib.Relativity.Tensors.Conjugation.Basic

Physlib/Relativity/Tensors/Conjugation/Basic.lean on GitHub.

lemma bar_tau_of_bar [source]

If `bar a = d`, then `bar (τ d) = τ a`.

```
lemma bar_tau_of_bar {a d : C} (hbar : S.bar a = d) : S.bar (S.τ d) = S.τ a
```

lemma toMatrix_conjT [source]

Conjugating a rank-two tensor conjugates each component and bars both indices: `(T¯)_{ĪJ¯} = (T_{IJ})*`, where barring a barred index removes the bar. For example, `ψ_α χ_β` goes to `ψ¯_α˙ χ¯_β˙`, and a tensor `M_{IJ¯}` goes to `(M¯)_{ĪJ} = (M_{IJ¯})*`.

```
lemma toMatrix_conjT {cs : Fin 2 → C} (t : S.Tensor cs) :
    toMatrix (S.conjT t) = ((toMatrix t).map star).submatrix
      (Equiv.cast (S.barIdx_eq (cs 0))) (Equiv.cast (S.barIdx_eq (cs 1)))
```

def conjSingleT [source]

Conjugation of a vector, `vᴵ ↦ v¯^Ī = (vᴵ)*`, landing at the conjugate colour. For example, a left-handed Weyl spinor `ψ_α` goes to the right-handed `ψ¯_α̇`.

```
def conjSingleT (x : C) : S.Tensor ![x] →ₛₗ[starRingEnd k] S.Tensor ![S.bar x]
```

lemma conjSingleT_basis [source]

Conjugation sends the basis vector `e_I` to `ē_Ī`, the basis vector of the conjugate colour with the same label.

```
lemma conjSingleT_basis (x : C) (z : basisIdx x) :
    S.conjSingleT x (basis ![x] (ComponentIdx.single.symm z)) =
      basis ![S.bar x] (ComponentIdx.single.symm (cast (S.barIdx_eq x).symm z))
```

def toSquareMatrix [source]

The matrix `(M_{IJ¯})`, with entries unchanged, regarded as a square matrix over the labels of `I`. Lean needs this step because the labels of a colour and of its conjugate are different types, identified by `barIdx_eq`; only for a square matrix are Hermiticity, positive-definiteness and the inverse defined. Convention: the unbarred index is the row and the barred index the column, entry `(I, J)` being `M_{IJ¯}`, as physicists write the matrix. For a Kähler metric `g_{IJ¯}` it is the usual matrix `(g_{IJ¯})`.

```
def toSquareMatrix {c : Fin 2 → C} (M : S.Tensor c) (hbar : S.bar (c 0) = c 1 := by rfl) :
    Matrix (basisIdx (c 0)) (basisIdx (c 0)) k
```

lemma toSquareMatrix_apply [source]

```
lemma toSquareMatrix_apply {c : Fin 2 → C} (hbar : S.bar (c 0) = c 1) (M : S.Tensor c)
    (i j : basisIdx (c 0)) :
    S.toSquareMatrix M hbar i j =
      toMatrix M i ((Equiv.cast (hbar ▸ S.barIdx_eq (c 0))).symm j)
```

lemma toMatrix_eq_submatrix_toSquareMatrix [source]

`toMatrix M` is `toSquareMatrix M` with its columns labelled by `J¯` instead of `J`.

```
lemma toMatrix_eq_submatrix_toSquareMatrix {c : Fin 2 → C} (hbar : S.bar (c 0) = c 1)
    (M : S.Tensor c) :
    toMatrix M =
      (S.toSquareMatrix M hbar).submatrix id (Equiv.cast (hbar ▸ S.barIdx_eq (c 0)))
```

lemma conjT_eq_permT_swap_iff_toSquareMatrix_isHermitian [source]

For `M_{IJ¯}`, the conjugate `(M¯)_{ĪJ} = (M_{IJ¯})*` and the swapped tensor `M_{JĪ}` have the same index types, a barred index then an ordinary one. They are equal,

`(M_{IJ¯})* = M_{JĪ}`, exactly when the matrix `(M_{IJ¯})` is Hermitian. A Kähler metric satisfies this.

```
lemma conjT_eq_permT_swap_iff_toSquareMatrix_isHermitian {c : Fin 2 → C}
    (hbar : S.bar (c 0) = c 1) (M : S.Tensor c) :
    S.conjT M = permT ![1, 0] (IsReindexing.swap_of_eq hbar.symm
      ((congrArg S.bar hbar).symm.trans (S.bar_involution (c 0))).symm) M ↔
    (S.toSquareMatrix M hbar).IsHermitian
```

lemma toMatrix_contractInnerT_eq_toSquareMatrix_mul [source]

The matrix of `g_{IJ¯} t^{J¯K}` is the product of the square matrix `(g_{IJ¯})` and the matrix `(t^{J¯K})`, with the rows of `(t^{J¯K})` labelled by `J`.

```
lemma toMatrix_contractInnerT_eq_toSquareMatrix_mul [HasContrDualBases S.toTensorSpecies]
    {cs ds : Fin 2 → C} (hbar : S.bar (cs 0) = cs 1) (h : S.τ (cs 1) = ds 0) (M : S.Tensor cs)
    (t : S.Tensor ds) :
    toMatrix (contractInnerT h M t) =
      S.toSquareMatrix M hbar * (toMatrix t).submatrix
        (dualIdxEquiv h ∘ (Equiv.cast (hbar ▸ S.barIdx_eq (cs 0))).symm)
        (id : basisIdx (ds 1) → _)
```

lemma isContrDualMatching_bar [source]

The conjugate bases are dual: if `e_α` contracted with `e^β` is `δ_α^β`, then `ē_α˙` contracted with `ē^β˙` is `δ_α˙^β˙`.

```
lemma isContrDualMatching_bar [HasContrDualBases S.toTensorSpecies] (d : C) :
    IsContrDualMatching S.toTensorSpecies (S.bar d)
      ((basisIdxCongr (S.bar_tau d)).symm.trans ((Equiv.cast (S.barIdx_eq (S.τ d))).trans
        ((contrDualIdxEquiv S.toTensorSpecies d).trans (Equiv.cast (S.barIdx_eq d)).symm)))
```

lemma dualIdxEquiv_bar [source]

`dualIdxEquiv` sends a lower label `α` to the upper label `α` it contracts with to `1`. Conjugation commutes with this: the dotted label `α˙` pairs with the dotted upper label `α˙`.

```
lemma dualIdxEquiv_bar [Nontrivial k] [HasContrDualBases S.toTensorSpecies] {c1 c2 c3 : C}
    (h : S.τ c1 = c2) (h' : S.τ (S.bar c1) = c3) (hbar : S.bar c2 = c3) (x : basisIdx c1) :
    dualIdxEquiv h' (cast (S.barIdx_eq c1).symm x) =
      cast (hbar ▸ S.barIdx_eq c2).symm (dualIdxEquiv h x)
```

lemma dualIdxEquiv_bar_tau [source]

Removing a bar and lowering an index commute on labels. From a label of the upper barred index `J¯`, removing the bar and then lowering gives the same label of the lower index `J` as lowering and then removing the bar. ` upper J¯ (τ d) → remove bar → upper J (τ a) ↓ lower ↓ lower lower J¯ (d) → remove bar → lower J (a) ` The left-hand side of the statement goes across the top and down, the right-hand side down and across the bottom.

```
lemma dualIdxEquiv_bar_tau [Nontrivial k] [HasContrDualBases S.toTensorSpecies] {a d : C}
    (hbar : S.bar a = d) (j : basisIdx (S.τ d)) :
    dualIdxEquiv (S.τ_τ_apply a)
```

```
    ((Equiv.cast (S.bar_tau_of_bar hbar ▸ S.barIdx_eq (S.τ d))).symm j) =
  Equiv.cast (hbar ▸ S.barIdx_eq a) ((dualIdxEquiv (rfl : S.τ d = S.τ d)).symm j)
```

def toSesqForm [source]

The Hermitian form `⟨s, t⟩ = M(t, s̄) = M_{IJ̄} t^I s̄^J̄` of a tensor `M_{IJ̄}`, on vectors `s, t`. Convention: as in Mathlib it is conjugate-linear in its left argument `s`; `s̄` fills the second, barred slot of `M` and `t` the first, so the argument order is opposite to the slot order.

```
def toSesqForm (x : C) {c : Fin 2 → C} (M : S.Tensor c) (hbar : S.bar (c 0) = c 1 := by rfl)
    (hx : S.τ x = c 0 := by rfl) : S.Tensor ![x] →ₗ⋆[k] S.Tensor ![x] →ₗ[k] k
```

lemma toMatrix₂_toSesqForm [source]

Since the left argument of the form fills the second slot of `M` (`toSesqForm`), `⟨e_I, e_J⟩ = M(e_J, ē_I) = M_{JĪ}`: the matrix of the form is the transpose of `(M_{IJ̄})`. For a Kähler metric, `⟨∂_I, ∂_J⟩ = ∂_J ∂_Ī K = g_{JĪ}`.

```
lemma toMatrix₂_toSesqForm [HasContrDualBases S.toTensorSpecies] {x : C} {c : Fin 2 → C}
    (hbar : S.bar (c 0) = c 1) (hx : S.τ x = c 0) (M : S.Tensor c) :
    LinearMap.toMatrix₂ ((basis ![x]).reindex (ComponentIdx.single.trans (dualIdxEquiv hx)))
        ((basis ![x]).reindex (ComponentIdx.single.trans (dualIdxEquiv hx)))
        (S.toSesqForm x M hbar hx) = (S.toSquareMatrix M hbar)^T
```

lemma isSymm_toSesqForm_iff_toSquareMatrix_isHermitian [source]

`⟨s, t⟩* = ⟨t, s⟩` for all `s, t` exactly when `(M_{IJ̄})* = M_{JĪ}`.

```
lemma isSymm_toSesqForm_iff_toSquareMatrix_isHermitian [HasContrDualBases S.toTensorSpecies]
    {x : C} {c : Fin 2 → C} (hbar : S.bar (c 0) = c 1) (hx : S.τ x = c 0) (M : S.Tensor c) :
    (S.toSesqForm x M hbar hx).IsSymm ↔ (S.toSquareMatrix M hbar).IsHermitian
```

lemma isSymm_toSesqForm_iff_conjT_eq_permT [source]

`⟨s, t⟩* = ⟨t, s⟩` for all `s, t` exactly when the conjugate of `M` equals `M` with its two indices swapped, `(M_{IJ̄})* = M_{JĪ}`.

```
lemma isSymm_toSesqForm_iff_conjT_eq_permT [HasContrDualBases S.toTensorSpecies]
    {x : C} {c : Fin 2 → C} (hbar : S.bar (c 0) = c 1) (hx : S.τ x = c 0) (M : S.Tensor c) :
    (S.toSesqForm x M hbar hx).IsSymm ↔ S.conjT M = permT ![1, 0] (IsReindexing.swap_of_eq hbar.
    symm
      ((congrArg S.bar hbar).symm.trans (S.bar_involution (c 0))).symm) M
```

lemma isPosDef_toSesqForm_iff_toSquareMatrix_posDef [source]

The form is positive-definite, Hermitian with `M_{IJ̄} t^I t̄^J̄ > 0` for every `t ≠ 0`, exactly when the matrix `(M_{IJ̄})` is. A Kähler metric is positive-definite, which makes kinetic terms positive.

```
lemma isPosDef_toSesqForm_iff_toSquareMatrix_posDef [HasContrDualBases S.toTensorSpecies]
    {x : C} {c : Fin 2 → C} (hbar : S.bar (c 0) = c 1) (hx : S.τ x = c 0) (M : S.Tensor c) :
    (S.toSesqForm x M hbar hx).IsPosDef ↔ (S.toSquareMatrix M hbar).PosDef
```

lemma isUnit_det_toSquareMatrix_of_isPosDef [source]

Over a field, the matrix `(M_{IJ¯})` of a positive-definite tensor is invertible.

```
lemma isUnit_det_toSquareMatrix_of_isPosDef [HasContrDualBases S.toTensorSpecies] {x : C}
    {c : Fin 2 → C} (hbar : S.bar (c 0) = c 1) (hx : S.τ x = c 0) {M : S.Tensor c}
    (hM : (S.toSesqForm x M hbar hx).IsPosDef) :
    IsUnit (S.toSquareMatrix M hbar).det
```

2.138 Physlib.Relativity.Tensors.Conjugation.Inverse

Physlib/Relativity/Tensors/Conjugation/Inverse.lean on GitHub.

def inverseT [source]

The inverse `g^{J¯I}` of a tensor `g_{IJ¯}`. Its matrix, with row `J` and column `I`, is `(g_{IJ¯})⁻¹`, and it is zero when `(g_{IJ¯})` is not invertible.

```
def inverseT {c : Fin 2 → C} (M : S.Tensor c) (hbar : S.bar (c 0) = c 1 := by rfl) :
    S.Tensor ![S.τ (c 1), S.τ (c 0)]
```

lemma toMatrix_inverseT [source]

The component `g^{J¯I}` is the entry `(J, I)` of `(g_{IJ¯})⁻¹`. The upper indices `J¯` and `I` are labelled by the lower indices `J` and `I` they contract with.

```
lemma toMatrix_inverseT {c : Fin 2 → C} (hbar : S.bar (c 0) = c 1) (M : S.Tensor c) :
    toMatrix (S.inverseT M hbar) =
      (S.toSquareMatrix M hbar)⁻¹.submatrix
        (Equiv.cast (hbar ▸ S.barIdx_eq (c 0)) ∘
          (dualIdxEquiv (rfl : S.τ (c 1) = S.τ (c 1))).symm)
        (dualIdxEquiv (S.τ_τ_apply (c 0)))
```

lemma toSquareMatrix_inverseT [source]

The matrix `(g^{J¯I})`, row `J` from the first, barred index and column `I` from the second, is `(g_{IJ¯})⁻¹`.

```
lemma toSquareMatrix_inverseT {c : Fin 2 → C} (hbar : S.bar (c 0) = c 1) (M : S.Tensor c) :
    S.toSquareMatrix (S.inverseT M hbar) (S.bar_tau_of_bar hbar) =
      (S.toSquareMatrix M hbar)⁻¹.submatrix
        (Equiv.cast (hbar ▸ S.barIdx_eq (c 0)) ∘
          (dualIdxEquiv (rfl : S.τ (c 1) = S.τ (c 1))).symm)
        (Equiv.cast (hbar ▸ S.barIdx_eq (c 0)) ∘
          (dualIdxEquiv (rfl : S.τ (c 1) = S.τ (c 1))).symm)
```

lemma isSymm_toSesqForm_inverseT [source]

The inverse of a Hermitian tensor is Hermitian. If `(g_{IJ¯})* = g_{JĪ}`, then `(g^{J¯I})* = g^{Ī J}`.

```
lemma isSymm_toSesqForm_inverseT {x y : C} {c : Fin 2 → C} (hbar : S.bar (c 0) = c 1)
    (hx : S.τ x = c 0) (hy : S.τ y = S.τ (c 1)) {M : S.Tensor c}
    (hM : (S.toSesqForm x M hbar hx).IsSymm) :
    (S.toSesqForm y (S.inverseT M hbar) (S.bar_tau_of_bar hbar) hy).IsSymm
```

lemma isPosDef_toSesqForm_inverseT [source]

The inverse of a positive-definite tensor is positive-definite, so `g^{J¯I} t_J¯ t¯_I > 0` for every covector `t ≠ 0`.

```
lemma isPosDef_toSesqForm_inverseT {x y : C} {c : Fin 2 → C} (hbar : S.bar (c 0) = c 1)
    (hx : S.τ x = c 0) (hy : S.τ y = S.τ (c 1)) {M : S.Tensor c}
    (hM : (S.toSesqForm x M hbar hx).IsPosDef) :
    (S.toSesqForm y (S.inverseT M hbar) (S.bar_tau_of_bar hbar) hy).IsPosDef
```

lemma contractInnerT_inverseT [source]

If the matrix `(g_{IJ¯})` is invertible, `g_{IJ¯} g^{J¯K} = δ_I^K`. The first colour of the unit tensor is renamed from `τ (τ (cs 0))` to `cs 0`.

```
lemma contractInnerT_inverseT {cs : Fin 2 → C} (hbar : S.bar (cs 0) = cs 1) (M : S.Tensor cs)
    (hM : IsUnit (S.toSquareMatrix M hbar).det) :
    contractInnerT rfl M (S.inverseT M hbar) =
      permT id (IsReindexing.unitTensor_of_tau_eq (S.τ_τ_apply (cs 0)))
        (unitTensor (S := S.toTensorSpecies) (S.τ (cs 0)))
```

lemma inverseT_contractInnerT [source]

If the matrix `(g_{IJ¯})` is invertible, `g^{J¯I} g_{IK¯} = δ^J¯_K¯`.

```
lemma inverseT_contractInnerT {cs : Fin 2 → C} (hbar : S.bar (cs 0) = cs 1) (M : S.Tensor cs)
    (hM : IsUnit (S.toSquareMatrix M hbar).det) :
    contractInnerT (S.τ_τ_apply (cs 0)) (S.inverseT M hbar) M =
      unitTensor (S := S.toTensorSpecies) (cs 1)
```

lemma eq_inverseT_of_contractInnerT_eq_unitTensor [source]

If `(g_{IJ¯})` is invertible and `g_{IJ¯} t^{J¯K} = δ_I^K`, then `t` is the inverse `g^{J¯K}`.

```
lemma eq_inverseT_of_contractInnerT_eq_unitTensor {cs : Fin 2 → C} (hbar : S.bar (cs 0) = cs 1)
    (M : S.Tensor cs) (hM : IsUnit (S.toSquareMatrix M hbar).det)
    {t : S.Tensor ![S.τ (cs 1), S.τ (cs 0)]}
    (ht : contractInnerT rfl M t =
      permT id (IsReindexing.unitTensor_of_tau_eq (S.τ_τ_apply (cs 0)))
        (unitTensor (S := S.toTensorSpecies) (S.τ (cs 0)))) :
    t = S.inverseT M hbar
```

def toConjDualAtIndex [source]

The linear equivalence that contracts slot `i` of a tensor with the first index of `M`, when the matrix `(M_{IJ¯})` is invertible. Slot `i` changes from `τ (cs 0)` to its conjugate dual `cs 1`, and the inverse contracts with `inverseT M`. For a metric `g_{IJ¯}` this lowers an unbarred index to a barred one, `t_{J¯} = g_{IJ¯} t^I`.

```
def toConjDualAtIndex {cs : Fin 2 → C} (M : S.Tensor cs) (hbar : S.bar (cs 0) = cs 1)
    (hM : IsUnit (S.toSquareMatrix M hbar).det) :
    {n : ℕ} → {c : Fin n → C} → (i : Fin n) → (he : c i = S.τ (cs 0)) →
      S.Tensor c ≃ₗ[k] S.Tensor (Function.update c i (cs 1))
  | 0, _, i, _ => i.elim0
  | _ + 1, _, i, he =>
```

```
    crossToSlotEquiv i he ((congrArg S.τ he).trans (S.τ_τ_apply (cs 0))) rfl M (S.inverseT M
    hbar)
      (S.contractInnerT_inverseT hbar M hM)
      ((S.inverseT_contractInnerT hbar M hM).trans (permT_id_self _).symm)
```

lemma toConjDualAtIndex_apply [source]

`toConjDualAtIndex` is the contraction `crossToSlot` of slot `i` with the first index of `M`.

```
@[simp]
lemma toConjDualAtIndex_apply {nA : ℕ} {c : Fin (nA + 1) → C} {cs : Fin 2 → C} (M : S.Tensor cs
    )
    (hbar : S.bar (cs 0) = cs 1) (hM : IsUnit (S.toSquareMatrix M hbar).det)
    (i : Fin (nA + 1)) (he : c i = S.τ (cs 0)) (t : S.Tensor c) :
    S.toConjDualAtIndex M hbar hM i he t =
      crossToSlot i 0 ((congrArg S.τ he).trans (S.τ_τ_apply (cs 0))) M t
```

lemma toConjDualAtIndex_symm_apply [source]

The inverse of `toConjDualAtIndex` is the contraction `crossToSlotInv` of slot `i` with the first index of `inverseT M`.

```
@[simp]
lemma toConjDualAtIndex_symm_apply {nA : ℕ} {c : Fin (nA + 1) → C} {cs : Fin 2 → C}
    (M : S.Tensor cs) (hbar : S.bar (cs 0) = cs 1) (hM : IsUnit (S.toSquareMatrix M hbar).det)
    (i : Fin (nA + 1)) (he : c i = S.τ (cs 0)) (t : S.Tensor (Function.update c i (cs 1))) :
    (S.toConjDualAtIndex M hbar hM i he).symm t =
      crossToSlotInv (S := S.toTensorSpecies) (d := cs 1) i he rfl (S.inverseT M hbar) t
```

2.139 `Physlib.Relativity.Tensors.Constructors`

`Physlib/Relativity/Tensors/Constructors.lean` on GitHub.

def fromMatrixT [source]

Rank-two tensors and their component matrices, as a linear equivalence: `fromMatrixT c0 c1 M` is the tensor with component matrix `M`.

```
noncomputable def fromMatrixT (c0 c1 : C) :
    Matrix (basisIdx c0) (basisIdx c1) k ≃ₗ[k] S.Tensor ![c0, c1] where
```

lemma fromMatrixT_basis_repr [source]

```
lemma fromMatrixT_basis_repr (c0 c1 : C) (M : Matrix (basisIdx c0) (basisIdx c1) k)
    (φ : ComponentIdx ![c0, c1]) :
    (basis ![c0, c1]).repr (fromMatrixT (S := S) c0 c1 M) φ = M (φ 0) (φ 1)
```

def toMatrix [source]

The component matrix of a rank-two tensor: the `(x, y)` entry is `repr t (ComponentIdx.pair.symm (x, y))`.

```
noncomputable def toMatrix {cs : Fin 2 → C} (t : S.Tensor cs) :
    Matrix (basisIdx (cs 0)) (basisIdx (cs 1)) k
```

lemma toMatrix_apply [source]

```
@[simp]
lemma toMatrix_apply {cs : Fin 2 → C} (t : S.Tensor cs)
    (x : basisIdx (cs 0)) (y : basisIdx (cs 1)) :
    toMatrix t x y = (basis cs).repr t (ComponentIdx.pair.symm (x, y))
```

lemma toMatrix_apply_cons [source]

`toMatrix_apply` at a `cons` colour list `![c0, c1]`, where the labels are named by the colours.

```
@[simp]
lemma toMatrix_apply_cons {c0 c1 : C} (t : S.Tensor ![c0, c1])
    (x : basisIdx c0) (y : basisIdx c1) :
    toMatrix t x y = (basis ![c0, c1]).repr t (ComponentIdx.pair.symm (x, y))
```

lemma toMatrix_fromMatrixT [source]

```
@[simp]
lemma toMatrix_fromMatrixT (c0 c1 : C) (M : Matrix (basisIdx c0) (basisIdx c1) k) :
    toMatrix (fromMatrixT (S := S) c0 c1 M) = M
```

lemma fromMatrixT_toMatrix [source]

```
@[simp]
lemma fromMatrixT_toMatrix {c0 c1 : C} (t : S.Tensor ![c0, c1]) :
    fromMatrixT (S := S) c0 c1 (toMatrix t) = t
```

lemma toMatrix_injective [source]

Rank-two tensors with equal component matrices are equal.

```
lemma toMatrix_injective {cs : Fin 2 → C} :
    Function.Injective (toMatrix (S := S) (cs := cs))
```

lemma toMatrix_permT_swap [source]

Swapping the two slots of a rank-two tensor transposes its component matrix.

```
lemma toMatrix_permT_swap {c c1 : Fin 2 → C} (h0 : c 1 = c1 0) (h1 : c 0 = c1 1)
    (t : S.Tensor c) :
    toMatrix (permT ![1, 0] (IsReindexing.swap_of_eq h0 h1) t) =
      (toMatrix t).transpose.submatrix (basisIdxCongr h0.symm) (basisIdxCongr h1.symm)
```

2.140 `Physlib.Relativity.Tensors.Contraction.Basis`

`Physlib/Relativity/Tensors/Contraction/Basis.lean` on GitHub.

lemma contrPCoeff_basisVector [source]

For a species with dual bases (`HasContrDualBases`), the contraction coefficient of a basis vector is the Kronecker δ of its labels at `i` and `j`, the label at `j` read as a label at the color `c i`. The per-species `contrPCoeff_basis` lemmas follow from it.

```
lemma Pure.contrPCoeff_basisVector [HasContrDualBases S] {n : ℕ} {c : Fin n → C} (i j : Fin n)
    (hij : i ≠ j ∧ S.τ (c i) = c j) (φ : ComponentIdx (S := S) c) :
    Pure.contrPCoeff i j hij (Pure.basisVector c φ) =
      if φ i = HasContrDualBases.contrDualIdxEquiv S (c i) (basisIdxCongr hij.2.symm (φ j))
    then 1
      else 0
```

lemma contrT_basis_repr_apply_eq_sum_dual [source]

For a species with dual bases (`HasContrDualBases`), a component of a contraction is a single sum over the labels at the first contracted index: the δ law (`HasContrDualBases.contr_basis_eq_ite`) fixes the label at the second. This collapses the double sum of `contrT_basis_repr_apply_eq_sum_fin`.

```
lemma contrT_basis_repr_apply_eq_sum_dual [HasContrDualBases S] {n : ℕ} {c : Fin (n + 1 + 1) →
    C}
    {i j : Fin (n + 1 + 1)} (h : i ≠ j ∧ S.τ (c i) = c j) (t : Tensor S c)
    (φ : ComponentIdx (c ∘ Fin.succSuccAbove i j)) :
    (basis (c ∘ Fin.succSuccAbove i j)).repr (contrT n i j h t) φ =
    ∑ x : basisIdx (c i), (basis c).repr t
      (DropPairSection.ofFinEquiv h.1 φ
        (x, basisIdxCongr h.2 ((HasContrDualBases.contrDualIdxEquiv S (c i)).symm x))).1
```

2.141 Physlib.Relativity.Tensors.Contraction.CrossToEnd

Physlib/Relativity/Tensors/Contraction/CrossToEnd.lean on GitHub.

lemma crossToEnd_basis [source]

Cross contraction of two basis tensors: the contraction of the two contracted basis vectors times the basis tensor of the surviving labels. These values determine `crossToEnd i j hc`, a bilinear map in the two tensors.

```
lemma crossToEnd_basis {nA nB : ℕ} {cA : Fin (nA + 1) → C} {cB : Fin (nB + 1) → C}
    (i : Fin (nA + 1)) (j : Fin (nB + 1)) (hc : S.τ (cA i) = cB j)
    (φA : ComponentIdx (S := S) cA) (φB : ComponentIdx (S := S) cB) :
    crossToEnd i j hc (basis cA φA) (basis cB φB) =
      S.contr (cA i) (b (cA i) (φA i) ⊗ₜ[k] b (S.τ (cA i)) (basisIdxCongr hc.symm (φB j))) •
        basis (Fin.append (cA ∘ i.succAbove) (cB ∘ j.succAbove))
          (ComponentIdx.prod.symm (fun m => φA (i.succAbove m), fun m => φB (j.succAbove m)))
```

lemma crossToEnd_basis_repr_eq_sum_fin [source]

The component formula for a cross contraction at arbitrary slots and ranks, for any species: each contracted label is summed over its own slot, with the contraction of the two basis vectors as coefficient. The counterpart of `contrT_basis_repr_apply_eq_sum_fin` for two tensors.

```
lemma crossToEnd_basis_repr_eq_sum_fin {nA nB : ℕ} {cA : Fin (nA + 1) → C}
    {cB : Fin (nB + 1) → C} (i : Fin (nA + 1)) (j : Fin (nB + 1)) (hc : S.τ (cA i) = cB j)
    (t1 : Tensor S cA) (t2 : Tensor S cB)
    (φ : ComponentIdx (S := S) (Fin.append (cA ∘ i.succAbove) (cB ∘ j.succAbove))) :
    (basis (Fin.append (cA ∘ i.succAbove) (cB ∘ j.succAbove))).repr
        (crossToEnd i j hc t1 t2) φ =
      ∑ x₁ : basisIdx (cA i), ∑ x₂ : basisIdx (cB j),
        ((basis cA).repr t1 (ComponentIdx.insert i (x₁, φ.prod.1)) *
```

```
      (basis cB).repr t2 (ComponentIdx.insert j (x₂, φ.prod.2))) *
    S.contr (cA i) (b (cA i) x₁ ⊗ₜ[k]
      b (S.τ (cA i)) (basisIdxCongr (by rw [hc]) x₂))
```

lemma crossToEnd_basis_repr_eq_sum_dual [source]

The component formula for a cross contraction at arbitrary slots and ranks, for a species with contraction-dual bases: the contraction coefficient is `δ` and one sum remains, the contracted label of `t2` matched to that of `t1` by `contrDualIdxEquiv`.

```
lemma crossToEnd_basis_repr_eq_sum_dual [HasContrDualBases S] {nA nB : ℕ} {cA : Fin (nA + 1) →
    C}
    {cB : Fin (nB + 1) → C} (i : Fin (nA + 1)) (j : Fin (nB + 1)) (hc : S.τ (cA i) = cB j)
    (t1 : Tensor S cA) (t2 : Tensor S cB)
    (φ : ComponentIdx (S := S) (Fin.append (cA ∘ i.succAbove) (cB ∘ j.succAbove))) :
    (basis (Fin.append (cA ∘ i.succAbove) (cB ∘ j.succAbove))).repr
        (crossToEnd i j hc t1 t2) φ =
      ∑ x : basisIdx (cA i),
        (basis cA).repr t1 (ComponentIdx.insert i (x, φ.prod.1)) *
        (basis cB).repr t2 (ComponentIdx.insert j
          (basisIdxCongr hc ((HasContrDualBases.contrDualIdxEquiv S (cA i)).symm x),
            φ.prod.2))
```

lemma crossToEnd_pair_basis_repr [source]

The component formula for cross-contracting the last slot of a rank-two tensor with the first slot of another. The surviving labels are supplied as a dependent pair, so the formula applies when the species has colour-dependent basis-index types.

```
lemma crossToEnd_pair_basis_repr [HasContrDualBases S] {c0 c1 c2 c3 : C} (h : S.τ c1 = c2)
    (t1 : Tensor S ![c0, c1]) (t2 : Tensor S ![c2, c3])
    (y0 : basisIdx c0) (y3 : basisIdx c3) :
    (basis (Fin.append (![c0, c1] ∘ (Fin.last 1).succAbove)
        (![c2, c3] ∘ (0 : Fin 2).succAbove))).repr
      (crossToEnd (Fin.last 1) (0 : Fin 2) h t1 t2)
      (ComponentIdx.pair.symm (y0, y3)) =
      ∑ x : basisIdx c1,
        (basis ![c0, c1]).repr t1 (ComponentIdx.pair.symm (y0, x)) *
        (basis ![c2, c3]).repr t2
          (ComponentIdx.pair.symm
            (basisIdxCongr h ((HasContrDualBases.contrDualIdxEquiv S c1).symm x), y3))
```

lemma toMatrix_crossToEnd_pair [source]

Cross contraction of two rank-two tensors is the product of their component matrices, the middle labels matched by `HasContrDualBases.dualIdxEquiv`.

```
lemma toMatrix_crossToEnd_pair [HasContrDualBases S] {c0 c1 c2 c3 : C} (h : S.τ c1 = c2)
    (t1 : S.Tensor ![c0, c1]) (t2 : S.Tensor ![c2, c3]) :
    toMatrix (crossToEnd (Fin.last 1) (0 : Fin 2) h t1 t2) =
      toMatrix t1 * (toMatrix t2).submatrix
        (fun x : basisIdx (![c0, c1] 1) => HasContrDualBases.dualIdxEquiv h x) id
```

2.142 Physlib.Relativity.Tensors.Contraction.CrossToSlot

Physlib/Relativity/Tensors/Contraction/CrossToSlot.lean on GitHub.

lemma crossToSlot_basis_repr_eq_sum_fin [source]

The component formula for a cross contraction into an arbitrary slot, for a species whose bases are not assumed adapted to the contraction: each contracted label is summed over its own slot and the contraction of the two basis vectors remains an opaque coefficient. Rotating the survivor back to its slot only transports the surviving labels, so this is `crossToEnd_basis_repr_eq_sum_fin` read through the cycle.

```
lemma crossToSlot_basis_repr_eq_sum_fin {nA : ℕ} {c : Fin (nA + 1) → C}
    {cM : Fin 2 → C} (i : Fin (nA + 1)) (j : Fin 2) (hc : S.τ (c i) = cM j)
    (M : S.Tensor cM) (t : S.Tensor c)
    (φ : ComponentIdx (S := S) (Function.update c i (cM (j.succAbove 0)))) :
    (basis (Function.update c i (cM (j.succAbove 0)))).repr
        (crossToSlot i j hc M t) φ =
      ∑ x₁ : basisIdx (c i), ∑ x₂ : basisIdx (cM j),
        ((basis c).repr t (ComponentIdx.insert i
            (x₁, fun m => basisIdxCongr (by simp [Function.update_of_ne]) (φ (i.succAbove m))))
     *
          (basis cM).repr M (ComponentIdx.insert j
            (x₂, fun m => basisIdxCongr (by fin_cases m; simp) (φ i)))) *
        S.contr (c i) (b (c i) x₁ ⊗ₜ[k]
          b (S.τ (c i)) (basisIdxCongr (by rw [hc]) x₂))
```

lemma crossToSlot_basis_repr_eq_sum_dual [source]

The component formula for a cross contraction into an arbitrary slot, for a species with contraction-dual bases: the contraction coefficient is `δ` and one sum remains, the contracted label of `M` matched to that of `t` by `contrDualIdxEquiv`.

```
lemma crossToSlot_basis_repr_eq_sum_dual [HasContrDualBases S] {nA : ℕ} {c : Fin (nA + 1) → C}
    {cM : Fin 2 → C} (i : Fin (nA + 1)) (j : Fin 2) (hc : S.τ (c i) = cM j)
    (M : S.Tensor cM) (t : S.Tensor c)
    (φ : ComponentIdx (S := S) (Function.update c i (cM (j.succAbove 0)))) :
    (basis (Function.update c i (cM (j.succAbove 0)))).repr
        (crossToSlot i j hc M t) φ =
      ∑ x : basisIdx (c i),
        (basis c).repr t (ComponentIdx.insert i
          (x, fun m => basisIdxCongr (by simp [Function.update_of_ne]) (φ (i.succAbove m)))) *
        (basis cM).repr M (ComponentIdx.insert j
          (basisIdxCongr hc ((HasContrDualBases.contrDualIdxEquiv S (c i)).symm x),
            fun m => basisIdxCongr (by fin_cases m; simp) (φ i)))
```

2.143 Physlib.Relativity.Tensors.Contraction.RankTwo

Physlib/Relativity/Tensors/Contraction/RankTwo.lean on GitHub.

def toLinearMap₂ [source]

The bilinear map `t, s ↦ M_{μν} t^μ s^ν` of a rank-two tensor: `t` fills the first index of `M`, `s` the second. For the Minkowski metric this is the inner product `η_{μν} t^μ s^ν`.

```
noncomputable def toLinearMap₂ (x y : C) {cM : Fin 2 → C} (M : Tensor S cM)
    (hx : S.τ x = cM 0 := by rfl) (hy : S.τ y = cM 1 := by rfl) :
```

```
    Tensor S ![x] →ₗ[k] Tensor S ![y] →ₗ[k] k
```

lemma toLinearMap₂_apply [source]

In components, the bilinear map is `∑_{μ,ν} M_{μν} t^u s^v`.

```
lemma toLinearMap₂_apply [HasContrDualBases S] {x y : C} {cM : Fin 2 → C} (hx : S.τ x = cM 0)
    (hy : S.τ y = cM 1) (M : Tensor S cM) (t : Tensor S ![x]) (s : Tensor S ![y]) :
    toLinearMap₂ x y M hx hy t s =
      ∑ w : basisIdx y, ∑ z : basisIdx x,
        (basis ![x]).repr t (ComponentIdx.single.symm z) *
          toMatrix M (HasContrDualBases.dualIdxEquiv hx z) (HasContrDualBases.dualIdxEquiv hy w
    ) *
          (basis ![y]).repr s (ComponentIdx.single.symm w)
```

lemma toMatrix₂_toLinearMap₂ [source]

On basis vectors, the bilinear map gives `M(e_μ, e_ν) = M_{μν}`*, so its matrix is the component matrix* `(M_{μν})`.

```
lemma toMatrix₂_toLinearMap₂ [HasContrDualBases S] {x y : C} {cM : Fin 2 → C}
    (hx : S.τ x = cM 0) (hy : S.τ y = cM 1) (M : Tensor S cM) :
    LinearMap.toMatrix₂
      ((basis ![x]).reindex (ComponentIdx.single.trans (HasContrDualBases.dualIdxEquiv hx)))
      ((basis ![y]).reindex (ComponentIdx.single.trans (HasContrDualBases.dualIdxEquiv hy)))
      (toLinearMap₂ x y M hx hy) = toMatrix M
```

def contractInnerT [source]

The contraction `A_{μν} B^{νρ}` *of the second index of* `A` *with the first index of* `B`*, whose colours are dual by* `h`*. The result has the indices* `μ` *and* `ρ`*, in that order.*

```
noncomputable def contractInnerT {cs ds : Fin 2 → C} (h : S.τ (cs 1) = ds 0) :
    Tensor S cs →ₗ[k] Tensor S ds →ₗ[k] Tensor S ![cs 0, ds 1]
```

lemma contractInnerT_apply [source]

`contractInnerT` *is* `crossToEnd` *of the last index of* `A` *with the first index of* `B`*, with the colours of the result renamed to* `![cs 0, ds 1]`.

```
lemma contractInnerT_apply {cs ds : Fin 2 → C} (h : S.τ (cs 1) = ds 0) (a : Tensor S cs)
    (b : Tensor S ds) :
    contractInnerT h a b = permT id (IsReindexing.on_id.mpr fun j => by fin_cases j <;> rfl)
      (crossToEnd (Fin.last 1) (0 : Fin 2) h a b)
```

lemma crossToEnd_eq_permT_contractInnerT [source]

`crossToEnd` *of the last index of* `A` *with the first index of* `B` *is* `contractInnerT` *with its colours renamed. This is* `contractInnerT_apply` *read from right to left.*

```
lemma crossToEnd_eq_permT_contractInnerT {cs ds : Fin 2 → C} (h : S.τ (cs 1) = ds 0)
    (a : Tensor S cs) (b : Tensor S ds) :
    crossToEnd (Fin.last 1) (0 : Fin 2) h a b =
      permT id (IsReindexing.on_id.mpr fun j => by fin_cases j <;> rfl) (contractInnerT h a b)
```

lemma toMatrix_contractInnerT [source]

The matrix of `A_{μν} B^{νρ}` is the matrix product of `(A_{μν})` and `(B^{νρ})`, with the labels of `ν` in `B` matched to those in `A` by `dualIdxEquiv`. This is `toMatrix_crossToEnd_pair` at general colour lists.

```
lemma toMatrix_contractInnerT [HasContrDualBases S] {cs ds : Fin 2 → C} (h : S.τ (cs 1) = ds 0)
    (a : Tensor S cs) (b : Tensor S ds) :
    toMatrix (contractInnerT h a b) =
      toMatrix a * (toMatrix b).submatrix (HasContrDualBases.dualIdxEquiv h) id
```

lemma contractInnerT_right_injective [source]

If the matrix `(M_{μν})`, made square by the label bijection `e`, is invertible, then `t ↦ M_{μν} t^{νρ}` is injective.

```
lemma contractInnerT_right_injective [HasContrDualBases S] {cs ds : Fin 2 → C}
    (h : S.τ (cs 1) = ds 0) (e : basisIdx (cs 0) ≃ basisIdx (cs 1)) (M : Tensor S cs)
    (hM : IsUnit ((toMatrix M).submatrix id e).det) :
    Function.Injective (contractInnerT h M)
```

lemma contractInnerT_equivariant [source]

`contractInnerT` commutes with the action of `G`. For a Lorentz transformation `Λ`, `(Λ A)_{μν} (Λ B)^{νρ} = Λ (A_{μν} B^{νρ})`.

```
lemma contractInnerT_equivariant {cs ds : Fin 2 → C} (h : S.τ (cs 1) = ds 0) (g : G)
    (a : Tensor S cs) (b : Tensor S ds) :
    contractInnerT h (g • a) (g • b) = g • contractInnerT h a b
```

2.144 Physlib.Relativity.Tensors.Contraction.UnitTensorContraction

Physlib/Relativity/Tensors/Contraction/UnitTensorContraction.lean on GitHub.

lemma contractInnerT_eq_unitTensor_comm [source]

If the label sets of the two indices of `M` are in bijection, `M_{μν} M'^{νρ} = δ_μ^ρ` implies `M'^{νρ} M_{ρσ} = δ^ν_σ`. Without the bijection `e` this fails, since `A B = 1` does not imply `B A = 1` for rectangular matrices.

```
lemma contractInnerT_eq_unitTensor_comm [HasContrDualBases S] {cs ds : Fin 2 → C}
    (hb : S.τ (cs 1) = ds 0) (ha : S.τ (ds 1) = cs 0) (e : basisIdx (cs 0) ≃ basisIdx (cs 1))
    (M : Tensor S cs) (M' : Tensor S ds)
    (h : contractInnerT hb M M' =
      permT id (IsReindexing.unitTensor_of_tau_eq ha) (unitTensor (S := S) (ds 1))) :
    contractInnerT ha M' M =
      permT id (IsReindexing.unitTensor_of_tau_eq hb) (unitTensor (S := S) (cs 1))
```

2.145 Physlib.Relativity.Tensors.Dual

Physlib/Relativity/Tensors/Dual.lean on GitHub.

lemma contractInnerT_dual_metricTensor_metricTensor [source]

The metric at `τ c` contracted with the metric at `c` is the unit tensor at `c`, `η^{μν} η_{νρ} = δ^μ_ρ`.

```
lemma contractInnerT_dual_metricTensor_metricTensor {c : C} :
    contractInnerT (S.τ_τ_apply c) (metricTensor (S := S) (S.τ c)) (metricTensor (S := S) c) =
      permT (id : Fin 2 → Fin 2) (IsReindexing.unitTensor_of_tau_eq rfl)
        (unitTensor (S := S) c)
```

lemma contractInnerT_metricTensor_metricTensor_eq_dual_unit [source]

The metric at `c` contracted with the metric at `τ c` is the unit tensor at `τ c`, `η_{μν} η^{νρ} = δ_μ^ρ`.

```
lemma contractInnerT_metricTensor_metricTensor_eq_dual_unit {c : C} :
    contractInnerT rfl (metricTensor (S := S) c) (metricTensor (S := S) (S.τ c)) =
      permT (id : Fin 2 → Fin 2) (IsReindexing.unitTensor_of_tau_eq (S.τ_τ_apply c))
        (unitTensor (S := S) (S.τ c))
```

2.146 Physlib.Relativity.Tensors.RealTensor.Basic

Physlib/Relativity/Tensors/RealTensor/Basic.lean on GitHub.

def realLorentzConjTensor [source]

The real Lorentz tensors as a conjugation species: over `ℝ` conjugation is trivial, so every colour is its own conjugate.

```
def realLorentzConjTensor (d : ℕ := 3) : ConjTensorSpecies ℝ Color (LorentzGroup d)
    (modules d) (fun _ => Fin 1 ⊕ Fin d)
    (fun | Color.up => Lorentz.ContrMod.rep | Color.down => Lorentz.CoMod.rep)
    (fun | Color.up => Lorentz.contrBasis d | Color.down => Lorentz.coBasis d) where
```

instance instWithMetric [source]

The Minkowski metric of the real Lorentz tensors, on both vector colors.

```
instance instWithMetric (d : ℕ) : (realLorentzTensor d).WithMetric where
```

lemma isContrDualMatching_refl [source]

At every real Lorentz color, the bases at the color and its dual are dual bases, with labels matched by the identity.

```
lemma isContrDualMatching_refl (c : realLorentzTensor.Color) :
    IsContrDualMatching (realLorentzTensor d) c (Equiv.refl _)
```

instance HasContrDualBases (realLorentzTensor d) [source]

Real Lorentz tensors have dual bases under contraction, with the identity matching.

```
instance : HasContrDualBases (realLorentzTensor d) where
```

lemma contrDualIdxEquiv_eq_refl [source]

The dual-label matching of real Lorentz tensors is the identity.

```
@[simp]
lemma contrDualIdxEquiv_eq_refl (c : realLorentzTensor.Color) :
    HasContrDualBases.contrDualIdxEquiv (realLorentzTensor d) c = Equiv.refl _
```

2.147 Physlib.Relativity.Tensors.RealTensor.Metrics.Pre

Physlib/Relativity/Tensors/RealTensor/Metrics/Pre.lean on GitHub.

def ContrMod.metricVal [source]

The metric η^{aa} as an element of `(ContrMod d ⊗[ℝ] ContrMod d)`.

```
def ContrMod.metricVal (d : ℕ := 3) : ContrMod d ⊗[ℝ] ContrMod d
```

lemma ContrMod.metricVal_expand_tmul_minkowskiMatrix [source]

```
lemma ContrMod.metricVal_expand_tmul_minkowskiMatrix {d : ℕ} : ContrMod.metricVal d =
    ∑ i, (minkowskiMatrix i i) • (contrBasis d i ⊗ₜ[ℝ] contrBasis d i)
```

lemma ContrMod.metricVal_expand_tmul [source]

Expansion of `ContrMod.metricVal` into basis.

```
lemma ContrMod.metricVal_expand_tmul {d : ℕ} : ContrMod.metricVal d =
    contrBasis d (Sum.inl 0) ⊗ₜ[ℝ] contrBasis d (Sum.inl 0) -
    ∑ i, contrBasis d (Sum.inr i) ⊗ₜ[ℝ] contrBasis d (Sum.inr i)
```

def ContrMod.metric [source]

The metric η^{aa} as a morphism `⊮_ (Rep ℝ (LorentzGroup d)) ⟶ ContrMod.rep ⊗ ContrMod.rep`, making its invariance under the action of `LorentzGroup d`.

```
def ContrMod.metric (d : ℕ := 3) :
    (Representation.trivial ℝ (LorentzGroup d) ℝ).IntertwiningMap
    ((ContrMod.rep).tprod (ContrMod.rep)) where
```

lemma ContrMod.metric_apply_one [source]

```
lemma ContrMod.metric_apply_one {d : ℕ} : (ContrMod.metric d) (1 : ℝ) = ContrMod.metricVal d
```

def CoMod.metricVal [source]

The metric η_{ii} as an element of `(CoMod d ⊗[ℝ] CoMod d)`.

```
def CoMod.metricVal (d : ℕ := 3) : CoMod d ⊗[ℝ] CoMod d
```

lemma CoMod.metricVal_expand_tmul_minkowskiMatrix [source]

```
lemma CoMod.metricVal_expand_tmul_minkowskiMatrix {d : ℕ} : CoMod.metricVal d =
    ∑ i, (minkowskiMatrix i i) • (coBasis d i ⊗ₜ[ℝ] coBasis d i)
```

lemma CoMod.metricVal_expand_tmul [source]

Expansion of `ContrMod.metricVal` into basis.

```
lemma CoMod.metricVal_expand_tmul {d : ℕ} : CoMod.metricVal d =
    coBasis d (Sum.inl 0) ⊗ₜ[ℝ] coBasis d (Sum.inl 0) -
    ∑ i, coBasis d (Sum.inr i) ⊗ₜ[ℝ] coBasis d (Sum.inr i)
```

def CoMod.metric [source]

The metric η_{ii} as a morphism `⊮_ (Rep ℂ (LorentzGroup d))) ⟶ CoMod.rep ⊗ CoMod.rep`, making its invariance under the action of `LorentzGroup d`.

```
def CoMod.metric (d : ℕ := 3) : (Representation.trivial ℝ (LorentzGroup d) ℝ).IntertwiningMap
    ((CoMod.rep).tprod (CoMod.rep)) where
```

lemma CoMod.metric_apply_one [source]

```
lemma CoMod.metric_apply_one {d : ℕ} : (CoMod.metric d) (1 : ℝ) = CoMod.metricVal d
```

lemma ContrMod.contr_metric [source]

```
lemma ContrMod.contr_metric {d : ℕ} :
    (TensorProduct.comm ℝ _ _ <|
      (TensorProduct.lid ℝ _).lTensor _ <|
      (ContrMod.contr.toLinearMap.rTensor (CoMod d)).lTensor (ContrMod d) <|
      (TensorProduct.assoc ℝ (ContrMod d) (CoMod d) (CoMod d)).symm.toLinearMap.lTensor
        (ContrMod d) <|
      TensorProduct.assoc ℝ (ContrMod d) (ContrMod d) ((CoMod d) ⊗[ℝ] (CoMod d)) <|
      (ContrMod.metric d 1) ⊗ₜ[ℝ] (CoMod.metric d 1)) = ContrMod.unit d (1 : ℝ)
```

lemma CoMod.contr_metric [source]

```
lemma CoMod.contr_metric {d : ℕ} :
    (TensorProduct.comm ℝ _ _ <|
    (TensorProduct.lid ℝ _).lTensor _ <|
    (CoMod.contr.toLinearMap.rTensor (ContrMod d)).lTensor (CoMod d) <|
    (TensorProduct.assoc ℝ (CoMod d) (ContrMod d) (ContrMod d)).symm.toLinearMap.lTensor
      (CoMod d) <|
    TensorProduct.assoc ℝ (CoMod d) (CoMod d) ((ContrMod d) ⊗[ℝ] (ContrMod d)) <|
    (CoMod.metric d 1) ⊗ₜ[ℝ] (ContrMod.metric d 1)) = CoMod.unit d (1 : ℝ)
```

2.148 Physlib.Relativity.Tensors.RealTensor.ToComplex

`Physlib/Relativity/Tensors/RealTensor/ToComplex.lean` on GitHub.

def basisIdxColorToComplex [source]

The basis index type of a complexified real color is `Fin 1 ⊕ Fin 3`, the basis index type of the real color.

```
def basisIdxColorToComplex : (c : realLorentzTensor.Color) →
    complexLorentzTensor.basisIdx (colorToComplex c) ≃ Fin 1 ⊕ Fin 3
  | .up => Equiv.refl _
  | .down => Equiv.refl _
```

lemma basisIdxCongr_basisIdxColorToComplex_symm [source]

```
@[simp]
lemma basisIdxCongr_basisIdxColorToComplex_symm {c c' : realLorentzTensor.Color}
    (h : colorToComplex c = colorToComplex c') (x : Fin 1 ⊕ Fin 3) :
    basisIdxCongr (basisIdx := complexLorentzTensor.basisIdx) h
      ((basisIdxColorToComplex c).symm x) = (basisIdxColorToComplex c').symm x
```

lemma basisIdxTauEquiv_basisIdxCongr_basisIdxColorToComplex_symm [source]

```
@[simp]
lemma basisIdxTauEquiv_basisIdxCongr_basisIdxColorToComplex_symm
    {c c' : realLorentzTensor.Color}
    (h : colorToComplex c' = complexLorentzTensor.τ (colorToComplex c)) (x : Fin 1 ⊕ Fin 3) :
    basisIdxTauEquiv (colorToComplex c) (basisIdxCongr h ((basisIdxColorToComplex c').symm x))
    =
      (basisIdxColorToComplex c).symm x
```

2.149 Physlib.Relativity.Tensors.RealTensor.Units.Pre

Physlib/Relativity/Tensors/RealTensor/Units/Pre.lean on GitHub.

def CoMod.unitVal [source]

The contra-co unit for complex lorentz vectors. Usually denoted $\delta^i{}_i$.

```
def CoMod.unitVal (d : ℕ := 3) : ContrMod d ⊗[ℝ] CoMod d
```

lemma CoMod.unitVal_expand_tmul [source]

Expansion of `CoMod.unitVal` into basis.

```
lemma CoMod.unitVal_expand_tmul {d : ℕ} : CoMod.unitVal d =
    ∑ i, contrBasis d i ⊗ₜ[ℝ] coBasis d i
```

def CoMod.unit [source]

The contra-co unit for complex lorentz vectors as a morphism $\mathbb{1} \longrightarrow$ `complexContr ⊗ complexCo`, manifesting the invariance under the `SL(2, ℂ)` action.

```
def CoMod.unit (d : ℕ := 3) :
  (Representation.trivial ℝ (LorentzGroup d) ℝ).IntertwiningMap
    ((ContrMod.rep).tprod (CoMod.rep)) where
```

lemma CoMod.unit_apply_one [source]

```
lemma CoMod.unit_apply_one {d : ℕ} : (CoMod.unit d) (1 : ℝ) = CoMod.unitVal d
```

def ContrMod.unitVal [source]

The co-contra unit for complex lorentz vectors. Usually denoted $\delta_i{}^i$.

```
def ContrMod.unitVal (d : ℕ := 3) : CoMod d ⊗[ℝ] ContrMod d
```

lemma ContrMod.unitVal_expand_tmul [source]

Expansion of `ContrMod.unitVal` into basis.

```
lemma ContrMod.unitVal_expand_tmul {d : ℕ} : ContrMod.unitVal d =
    ∑ i, coBasis d i ⊗ₜ[ℝ] contrBasis d i
```

def ContrMod.unit [source]

The co-contra unit for complex lorentz vectors as a morphism `𝟙_ (Rep ℝ (LorentzGroup d)) ⟶ CoMod.rep ⊗ ContrMod.rep`, manifesting the invariance under the `LorentzGroup d` action.

```
def ContrMod.unit (d : ℕ) : (Representation.trivial ℝ (LorentzGroup d) ℝ).IntertwiningMap
    ((CoMod.rep).tprod (ContrMod.rep)) where
```

lemma ContrMod.unit_apply_one [source]

```
lemma ContrMod.unit_apply_one {d : ℕ} : (ContrMod.unit d) (1 : ℝ) = ContrMod.unitVal d
```

lemma CoMod.contr_unit [source]

Contraction on the right with `CoMod.unit` does nothing.

```
lemma CoMod.contr_unit {d : ℕ} (x : CoMod d) :
    (TensorProduct.lid ℝ _ <|
    CoMod.contr.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℝ (CoMod d) (ContrMod d) (CoMod d)).symm <|
    x ⊗ₜ[ℝ] (CoMod.unit d (1 : ℝ))) = x
```

lemma ContrMod.contr_unit [source]

Contraction on the right with `ContrMod.unit` does nothing.

```
lemma ContrMod.contr_unit {d : ℕ} (x : ContrMod d) :
    (TensorProduct.lid ℝ _ <|
    ContrMod.contr.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℝ (ContrMod d) (CoMod d) (ContrMod d)).symm <|
    x ⊗ₜ[ℝ] (ContrMod.unit d (1 : ℝ))) = x
```

lemma CoMod.unit_symm [source]

```
lemma CoMod.unit_symm {d : ℕ} :
    (CoMod.unit d) (1 : ℝ) = LinearMap.lTensor _ (LinearEquiv.cast (by rfl)).toLinearMap
      (TensorProduct.comm ℝ _ _ ((ContrMod.unit d) (1 : ℝ)))
```

lemma ContrMod.unit_symm [source]

```
lemma ContrMod.unit_symm {d : ℕ} :
    (ContrMod.unit d) (1 : ℝ) = LinearMap.lTensor _ (LinearEquiv.cast (by simp)).toLinearMap
    (TensorProduct.comm ℝ _ _ ((CoMod.unit d) (1 : ℝ)))
```

2.150 `Physlib.Relativity.Tensors.RealTensor.Vector.Pre.Contraction`

Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Contraction.lean on GitHub.

def ContrMod.contr [source]

The linear map from ContrMod $d \otimes$ CoMod d to $\mathbb{R}$ given by summing over components of contravariant Lorentz vector and covariant Lorentz vector in the standard basis (i.e. the dot product). In terms of index notation this is the contraction is $\psi^i \varphi_i$.

```
def ContrMod.contr : ((ContrMod.rep).tprod (CoMod.rep)).IntertwiningMap
    (Representation.trivial ℝ (LorentzGroup d) ℝ) where
```

lemma ContrMod.contr_tmul [source]

```
lemma ContrMod.contr_tmul (ψ : ContrMod d) (φ : CoMod d) :
    ⟪ψ, φ⟫ₘ = ψ.toFin1dℝ ⬝ᵥ φ.toFin1dℝ
```

def CoMod.contr [source]

The linear map from CoMod $d \otimes$ ContrMod d to $\mathbb{R}$ given by summing over components of contravariant Lorentz vector and covariant Lorentz vector in the standard basis (i.e. the dot product). In terms of index notation this is the contraction is $\psi^i \varphi_i$.

```
def CoMod.contr : ((CoMod.rep (d := d)).tprod (ContrMod.rep (d := d))).IntertwiningMap
    (Representation.trivial ℝ (LorentzGroup d) ℝ) where
```

lemma CoMod.contr_tmul [source]

```
lemma CoMod.contr_tmul (φ : CoMod d) (ψ : ContrMod d) :
    ⟪φ, ψ⟫ₘ = φ.toFin1dℝ ⬝ᵥ ψ.toFin1dℝ
```

lemma ContrMod.contr_tmul_symm [source]

```
lemma ContrMod.contr_tmul_symm (φ : ContrMod d) (ψ : CoMod d) : ⟪φ, ψ⟫ₘ = ⟪ψ, φ⟫ₘ
```

lemma CoMod.contr_tmul_symm [source]

```
lemma CoMod.contr_tmul_symm (φ : CoMod d) (ψ : ContrMod d) : ⟪φ, ψ⟫ₘ = ⟪ψ, φ⟫ₘ
```

lemma ContrMod.contr_basis [source]

```
lemma ContrMod.contr_basis {d : ℕ} (i j : Fin 1 ⊕ Fin d) :
    ContrMod.contr (contrBasis d i ⊗ₜ coBasis d j) = if i = j then (1 : ℝ) else 0
```

lemma CoMod.contr_basis [source]

```
lemma CoMod.contr_basis {d : ℕ} (i j : Fin 1 ⊕ Fin d) :
    CoMod.contr (coBasis d i ⊗ₜ[ℝ] contrBasis d j) = if i = j then (1 : ℝ) else 0
```

2.151 Physlib.Relativity.Tensors.RealTensor.Velocity.Basic

Physlib/Relativity/Tensors/RealTensor/Velocity/Basic.lean on GitHub.

def ofSpatialPart [source]

The unit future four-velocity with spatial part `p` and time component `Real.sqrt (1 + ‖p‖ ^ 2)`. The spatial part is unrestricted.

```
noncomputable def ofSpatialPart (p : EuclideanSpace ℝ (Fin d)) : Velocity d
```

lemma spatialPart_ofSpatialPart [source]

```
@[simp]
lemma spatialPart_ofSpatialPart (p : EuclideanSpace ℝ (Fin d)) :
    (ofSpatialPart p).1.spatialPart = p
```

lemma timeComponent_ofSpatialPart [source]

```
@[simp]
lemma timeComponent_ofSpatialPart (p : EuclideanSpace ℝ (Fin d)) :
    (ofSpatialPart p).1.timeComponent = Real.sqrt (1 + ‖p‖ ^ 2)
```

lemma ofSpatialPart_spatialPart [source]

A future unit four-velocity is determined by its spatial part.

```
@[simp]
lemma ofSpatialPart_spatialPart (u : Velocity d) : ofSpatialPart u.1.spatialPart = u
```

lemma continuous_ofSpatialPart [source]

The spatial-coordinate parametrization of future unit four-velocities is continuous.

```
@[fun_prop]
lemma continuous_ofSpatialPart : Continuous (ofSpatialPart (d := d))
```

lemma norm_spatialPart_le_of_orthogonal [source]

A bound on the proper norm of a vector orthogonal to a unit future four-velocity controls its spatial norm in the chosen frame.

```
lemma norm_spatialPart_le_of_orthogonal (u : Velocity d) (A : Vector d)
    {C : ℝ} (hC : 0 ≤ C) (horth : ⟪u.1, A⟫ₘ = 0) (hacc : -⟪A, A⟫ₘ ≤ C ^ 2) :
    ‖A.spatialPart‖ ≤ C * u.1.timeComponent
```

2.152 Physlib.Relativity.Tensors.RealTensor.Velocity.Deriv

Physlib/Relativity/Tensors/RealTensor/Velocity/Deriv.lean on GitHub.

lemma minkowskiProduct_eq_zero_of_hasDerivAt [source]

The derivative of a curve of unit future-directed four-velocities is Minkowski orthogonal to its value.

```
lemma minkowskiProduct_eq_zero_of_hasDerivAt {d : ℕ} {u : ℝ → Velocity d}
    {a : Vector d} {t : ℝ} (hu : HasDerivAt (fun s => (u s).1) a t) :
    ⟪(u t).1, a⟫ₘ = 0
```

lemma hasDerivAt_ofSpatialPart [source]

The ambient derivative of the unit future-directed four-velocity determined by a differentiable spatial-part curve.

```
lemma hasDerivAt_ofSpatialPart {d : ℕ} {p : ℝ → EuclideanSpace ℝ (Fin d)}
    {p' : EuclideanSpace ℝ (Fin d)} {t : ℝ} (hp : HasDerivAt p p' t) :
    HasDerivAt (fun s => (ofSpatialPart (p s)).1)
      (Vector.ofTemporalComponent (inner ℝ (p t) p' / Real.sqrt (1 + ‖p t‖ ^ 2)) +
        Vector.ofSpatialComponent p') t
```

2.153 Physlib.Relativity.Tensors.Reindexing

Physlib/Relativity/Tensors/Reindexing.lean on GitHub.

lemma swap_of_eq [source]

The swap `![1, 0]` reindexes two rank-two colour lists whose colours match crosswise; `swap` with the target colours given by equations.

```
lemma swap_of_eq {c c1 : Fin 2 → C} (h0 : c 1 = c1 0) (h1 : c 0 = c1 1) :
    IsReindexing c c1 ![1, 0]
```

2.154 Physlib.Relativity.Tensors.TensorSpecies.Basic

Physlib/Relativity/Tensors/TensorSpecies/Basic.lean on GitHub.

lemma basisIdxCongr_heq [source]

A transported label is heterogeneously equal to the original.

```
lemma basisIdxCongr_heq {c c1 : C} (h : c = c1) (x : basisIdx c) : basisIdxCongr h x ≍ x
```

lemma basisIdxCongr_const [source]

For a species whose every colour carries the same label type `I`, a label transport is the identity.

```
@[simp]
lemma basisIdxCongr_const {I : Type} {c c1 : C} (h : c = c1) :
    basisIdxCongr (basisIdx := fun _ : C => I) h = Equiv.refl I
```

lemma contr_basis_congr [source]

The contraction of two basis vectors does not change when their colour is transported along an equality and their labels with it.

```
lemma contr_basis_congr {a a' : C} (h : a = a') (x : basisIdx a) (y : basisIdx (S.τ a)) :
    S.contr a (basis a x ⊗ₜ[k] basis (S.τ a) y) =
      S.contr a' (basis a' (basisIdxCongr h x) ⊗ₜ[k]
```

```
      basis (S.τ a') (basisIdxCongr (congrArg S.τ h) y))
```

lemma equivCast_basis [source]

map_basis_eq with the cast spelled Equiv.cast, the form contr_tmul_symm applies to its first vector.

```
lemma equivCast_basis {c c1 : C} (h : c = c1) (i : basisIdx c) :
    Equiv.cast (congrArg V h) (basis c i) = basis c1 (basisIdxCongr h i)
```

class WithMetric [source]

Instance of a metric on a tensor species. For each color c an invariant tensor in V c ⊗ V c, whose contraction against the metric of the dual color τ c is the unit. Not every species has one: it requires every color to be self-dual, which fails for the fundamental of SU(N), 3 ≤ N.

```
class WithMetric {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
    {V : C → Type} [∀ c, AddCommGroup (V c)] [∀ c, Module k (V c)]
    {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
    {rep : (c : C) → Representation k G (V c)} {basis : (c : C) → Basis (basisIdx c) k (V c)}
    (S : TensorSpecies k C G V basisIdx rep basis) where
  metric : (c : C) → ((Representation.trivial k G k)).IntertwiningMap ((rep c).tprod (rep c))
  contr_metric : ∀ c,
    (TensorProduct.comm k _ _ <|
      (TensorProduct.lid k _).lTensor _ <|
      ((S.contr c).toLinearMap.rTensor (V (S.τ c))).lTensor (V c) <|
      (TensorProduct.assoc k (V c) (V (S.τ c)) (V (S.τ c))).symm.toLinearMap.lTensor (V c) <|
      TensorProduct.assoc k (V c) (V c) (V (S.τ c) ⊗[k] V (S.τ c)) <|
      (metric c 1) ⊗ₜ[k] (metric (S.τ c) 1)) = S.unit c (1 : k)
```

abbrev metric [source]

The metric of a species with a metric, at the color c.

```
abbrev metric [S.WithMetric] (c : C) :
    ((Representation.trivial k G k)).IntertwiningMap ((rep c).tprod (rep c))
```

lemma contr_metric [source]

On contracting metrics we get the unit.

```
lemma contr_metric [S.WithMetric] (c : C) :
    (TensorProduct.comm k _ _ <|
      (TensorProduct.lid k _).lTensor _ <|
      ((S.contr c).toLinearMap.rTensor (V (S.τ c))).lTensor (V c) <|
      (TensorProduct.assoc k (V c) (V (S.τ c)) (V (S.τ c))).symm.toLinearMap.lTensor (V c) <|
      TensorProduct.assoc k (V c) (V c) (V (S.τ c) ⊗[k] V (S.τ c)) <|
      (S.metric c 1) ⊗ₜ[k] (S.metric (S.τ c) 1)) = S.unit c (1 : k)
```

2.155 Physlib.Relativity.Tensors.TensorSpecies.DualBasis

Physlib/Relativity/Tensors/TensorSpecies/DualBasis.lean on GitHub.

def IsContrDualMatching [source]

A matching `e` of the basis labels at `S.τ c` with those at `c` satisfies the δ law: contracting a basis vector at `c` with one at `S.τ c` gives `1` if `e` matches their labels and `0` otherwise.

```
def IsContrDualMatching (S : TensorSpecies k C G V basisIdx rep b) (c : C)
    (e : basisIdx (S.τ c) ≃ basisIdx c) : Prop
```

lemma unique [source]

Over a nontrivial ring the δ law determines the matching: the label matched with x_2 is the one whose basis vector pairs with `b (S.τ c) x₂` to `1`.

```
lemma IsContrDualMatching.unique [Nontrivial k] {S : TensorSpecies k C G V basisIdx rep b}
    {c : C} {e e' : basisIdx (S.τ c) ≃ basisIdx c}
    (he : IsContrDualMatching S c e) (he' : IsContrDualMatching S c e') : e = e'
```

class HasContrDualBases [source]

A tensor species whose basis at `S.τ c` is dual to its basis at `c` under the pairing `S.contr c`: for every color some matching of the labels satisfies the δ law.

```
class HasContrDualBases (S : TensorSpecies k C G V basisIdx rep b) : Prop where
  exists_matching : ∀ c, ∃ e, IsContrDualMatching S c e
```

def contrDualIdxEquiv [source]

A matching of the basis labels at `S.τ c` with those at `c` satisfying the δ law, chosen by `Classical.choose`. Over a nontrivial ring it is the only one (`contrDualIdxEquiv_eq_of_isContrDualMatchin`

```
noncomputable def contrDualIdxEquiv (c : C) : basisIdx (S.τ c) ≃ basisIdx c
```

lemma contr_basis_eq_ite [source]

The δ law for `contrDualIdxEquiv`.

```
lemma contr_basis_eq_ite (c : C) (x₁ : basisIdx c) (x₂ : basisIdx (S.τ c)) :
    S.contr c (b c x₁ ⊗ₜ[k] b (S.τ c) x₂) =
      if x₁ = contrDualIdxEquiv S c x₂ then 1 else 0
```

lemma contrDualIdxEquiv_eq_of_isContrDualMatching [source]

Any matching satisfying the δ law is `contrDualIdxEquiv`. This is how a concrete species identifies its matching.

```
lemma contrDualIdxEquiv_eq_of_isContrDualMatching [Nontrivial k] {c : C}
    {e : basisIdx (S.τ c) ≃ basisIdx c} (he : IsContrDualMatching S c e) :
    contrDualIdxEquiv S c = e
```

def dualIdxEquiv [source]

The equivalence between labels of dual indices: a label at `c1` carried across the variance dual by `contrDualIdxEquiv`, then along `h`.

```
noncomputable def dualIdxEquiv {c1 c2 : C} (h : S.τ c1 = c2) : basisIdx c1 ≃ basisIdx c2
```

lemma dualIdxEquiv_apply [source]

```
@[simp]
lemma dualIdxEquiv_apply {c1 c2 : C} (h : S.τ c1 = c2) (x : basisIdx c1) :
    dualIdxEquiv h x = basisIdxCongr h ((contrDualIdxEquiv S c1).symm x)
```

lemma contrDualIdxEquiv_tau [source]

The matching at `S.τ c` is the inverse of the one at `c`, once `S.τ (S.τ c)` is identified with `c` by `S.τ_τ_apply`. The inverse matching satisfies the δ law at `S.τ c` by `contr_tmul_symm`, so uniqueness identifies the two.

```
lemma contrDualIdxEquiv_tau [Nontrivial k] (c : C) (x : basisIdx (S.τ (S.τ c))) :
    contrDualIdxEquiv S (S.τ c) x =
      (contrDualIdxEquiv S c).symm (basisIdxCongr (S.τ_τ_apply c) x)
```

lemma dualIdxEquiv_symm [source]

The inverse of `dualIdxEquiv` from `c1` to `c2` is `dualIdxEquiv` from `c2` to `c1`.

```
lemma dualIdxEquiv_symm [Nontrivial k] {c1 c2 : C} (h : S.τ c1 = c2) (h' : S.τ c2 = c1) :
    (dualIdxEquiv h).symm = dualIdxEquiv h'
```

lemma contr_tmul_basis_eq_dualBasis [source]

The functional `v ↦ S.contr c (v ⊗ₜ b (S.τ c) x)` is the element of `(b c).dualBasis` at the label matched with `x`.

```
lemma contr_tmul_basis_eq_dualBasis (c : C) (x : basisIdx (S.τ c)) (v : V c) :
    S.contr c (v ⊗ₜ[k] b (S.τ c) x) = (b c).dualBasis (contrDualIdxEquiv S c x) v
```

2.156 Physlib.Relativity.Tensors.UnitTensor

Physlib/Relativity/Tensors/UnitTensor.lean on GitHub.

lemma unitTensor_pair_basis_repr [source]

For a species with dual bases at dual colours, the unit tensor's components are a Kronecker δ, the first label carried across the variance dual by `contrDualIdxEquiv`.

```
lemma unitTensor_pair_basis_repr [HasContrDualBases S] (c : C) (J y : basisIdx c) :
    (basis ![S.τ c, c]).repr (unitTensor c)
        (ComponentIdx.pair.symm ((HasContrDualBases.contrDualIdxEquiv S c).symm J, y)) =
      if J = y then 1 else 0
```

lemma unitTensor_basis_repr_eq_ite [source]

`unitTensor_pair_basis_repr` at an arbitrary component index.

```
lemma unitTensor_basis_repr_eq_ite [HasContrDualBases S] (c : C)
    (φ : ComponentIdx (S := S) ![S.τ c, c]) :
    (basis ![S.τ c, c]).repr (unitTensor c) φ =
      if HasContrDualBases.contrDualIdxEquiv S c (φ 0) = φ 1 then 1 else 0
```

lemma toMatrix_permT_unitTensor [source]

The component matrix of a unit tensor recoloured without moving its slots is the identity, with the labels transported back to the unit tensor's colours.

```
lemma toMatrix_permT_unitTensor [HasContrDualBases S] (c : C) {cs : Fin 2 → C}
    {σ : Fin 2 → Fin 2} (hσ : ∀ i, σ i = i) (hperm : IsReindexing ![S.τ c, c] cs σ) :
    toMatrix (permT σ hperm (unitTensor (S := S) c)) =
      (1 : Matrix (basisIdx c) (basisIdx c) k).submatrix
        (fun x => HasContrDualBases.contrDualIdxEquiv S c
          (basisIdxCongr (by rw [← hperm.2 0, hσ]; rfl) x))
        (fun y => basisIdxCongr (by rw [← hperm.2 1, hσ]; rfl) y)
```

lemma toMatrix_unitTensor [source]

The unit tensor's component matrix is the identity, carried along `contrDualIdxEquiv`.

```
lemma toMatrix_unitTensor [HasContrDualBases S] (c : C) :
    toMatrix (unitTensor (S := S) c) =
      (1 : Matrix (basisIdx c) (basisIdx c) k).submatrix
        ⇑(HasContrDualBases.contrDualIdxEquiv S c) id
```

lemma unitTensor_of_tau_eq [source]

If `τ e = a`, the identity is a reindexing from `![τ e, e]` to `![a, e]`, so `permT` renames the colours of `unitTensor e` to `![a, e]`.

```
lemma IsReindexing.unitTensor_of_tau_eq {a e : C} (h : S.τ e = a) :
    IsReindexing (![S.τ e, e] : Fin 2 → C) ![a, e] (id : Fin 2 → Fin 2)
```

lemma toMatrix_permT_unitTensor_of_tau_eq [source]

The matrix of `unitTensor e`, with its colours renamed to `![a, e]`, is the identity matrix with its rows relabelled by `dualIdxEquiv h`.

```
lemma toMatrix_permT_unitTensor_of_tau_eq [HasContrDualBases S] {a e : C} (h : S.τ e = a) :
    toMatrix (permT id (IsReindexing.unitTensor_of_tau_eq h) (unitTensor (S := S) e)) =
      (1 : Matrix (basisIdx e) (basisIdx e) k).submatrix (dualIdxEquiv h).symm id
```

2.157 Physlib.SpaceAndTime.Space.Derivatives.Laplacian

Physlib/SpaceAndTime/Space/Derivatives/Laplacian.lean on GitHub.

lemma laplacian_comp_linearIsometryEquiv [source]

The Laplacian of `f ∘ R` at `x` is the Laplacian of `f` at `R x`, for a linear isometry `R` of space.

```
lemma laplacian_comp_linearIsometryEquiv {d : ℕ} (f : Space d → ℝ) (R : Space d ≃ₗᵢ[ℝ] Space d)
    (x : Space d) (hf : DifferentiableAt ℝ (fderiv ℝ f) (R x)) :
    Δ (fun y => f (R y)) x = Δ f (R x)
```

lemma laplacianVec_comp_linearIsometryEquiv [source]

The vector Laplacian of `f ∘ R` at `x` is the vector Laplacian of `f` at `R x`, for a linear isometry `R` of space.

```
lemma laplacianVec_comp_linearIsometryEquiv {d : ℕ} (f : Space d → EuclideanSpace ℝ (Fin d))
    (R : Space d ≃ₗᵢ[ℝ] Space d) (x : Space d)
    (hf : ∀ i, DifferentiableAt ℝ (fderiv ℝ (fun y => f y i)) (R x)) :
    Δᵥ (fun y => f (R y)) x = Δᵥ f (R x)
```

2.158 Physlib.SpaceAndTime.Space.IsDistBounded

Physlib/SpaceAndTime/Space/IsDistBounded.lean on GitHub.

instance Measure.HasTemperateGrowth (μ1.prod μ2) [source]

```
instance {D1 : Type} [NormedAddCommGroup D1] [MeasurableSpace D1]
    {D2 : Type} [NormedAddCommGroup D2] [MeasurableSpace D2]
    (μ1 : Measure D1) (μ2 : Measure D2)
    [Measure.HasTemperateGrowth μ1] [Measure.HasTemperateGrowth μ2]
    [OpensMeasurableSpace (D1 × D2)] :
    Measure.HasTemperateGrowth (μ1.prod μ2) where
```

2.159 Physlib.SpaceAndTime.SpaceTime.Derivatives

Physlib/SpaceAndTime/SpaceTime/Derivatives.lean on GitHub.

lemma deriv_add [source]

Derivatives on spacetime distribute over addition.

```
@[to_fun]
lemma deriv_add {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ}
    (μ : Fin 1 ⊕ Fin d) (f g : SpaceTime d → M) (hf : Differentiable ℝ f)
    (hg : Differentiable ℝ g) :
    ∂_ μ (f + g) = ∂_ μ f + ∂_ μ g
```

lemma deriv_const_smul [source]

Derivatives on spacetime commute with multiplication by a constant.

```
lemma deriv_const_smul {M R : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] [Semiring R]
    [Module R M] [SMulCommClass ℝ R M] [ContinuousConstSMul R M] {d : ℕ}
    (μ : Fin 1 ⊕ Fin d) (c : R) {f : SpaceTime d → M} (hf : Differentiable ℝ f) :
    ∂_ μ (c • f) = c • ∂_ μ f
```

lemma foldl_deriv_perm [source]

For a smooth function, iterated derivatives along a list of directions do not depend on the order of the list.

```
lemma foldl_deriv_perm {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ}
    {l₁ l₂ : List (Fin 1 ⊕ Fin d)} (h : l₁.Perm l₂) {f : SpaceTime d → M}
    (hf : ContDiff ℝ ∞ f) :
    l₁.foldl (fun g μ => ∂_ μ g) f = l₂.foldl (fun g μ => ∂_ μ g) f
```

def iteratedDeriv [source]

The iterated derivative of `f`, differentiating once along each element of `s`.

```
noncomputable def iteratedDeriv {M : Type} [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
    {d : ℕ} (s : Multiset (Fin 1 ⊕ Fin d)) (f : SpaceTime d → M) : SpaceTime d → M
```

lemma iteratedDeriv_zero [source]

```
@[simp]
lemma iteratedDeriv_zero {M : Type} [AddCommGroup M] [Module ℝ M] [TopologicalSpace M] {d : ℕ}
    (f : SpaceTime d → M) :
    iteratedDeriv 0 f = f
```

lemma iteratedDeriv_singleton [source]

```
@[simp]
lemma iteratedDeriv_singleton {M : Type} [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
    {d : ℕ} (μ : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) :
    iteratedDeriv {μ} f = ∂_ μ f
```

lemma contDiff_iteratedDeriv [source]

Iterated derivatives of a smooth function are smooth.

```
lemma contDiff_iteratedDeriv {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ}
    (s : Multiset (Fin 1 ⊕ Fin d)) {f : SpaceTime d → M} (hf : ContDiff ℝ ∞ f) :
    ContDiff ℝ ∞ (iteratedDeriv s f)
```

lemma iteratedDeriv_cons [source]

An iterated derivative of a smooth function may begin with any of its directions.

```
lemma iteratedDeriv_cons {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ}
    (μ : Fin 1 ⊕ Fin d) (s : Multiset (Fin 1 ⊕ Fin d)) {f : SpaceTime d → M}
    (hf : ContDiff ℝ ∞ f) :
    iteratedDeriv (μ ::ₘ s) f = iteratedDeriv s (∂_ μ f)
```

lemma iteratedDeriv_add [source]

Iterated derivatives of smooth functions distribute over addition.

```
lemma iteratedDeriv_add {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ}
    (s : Multiset (Fin 1 ⊕ Fin d)) {f g : SpaceTime d → M} (hf : ContDiff ℝ ∞ f)
    (hg : ContDiff ℝ ∞ g) :
    iteratedDeriv s (f + g) = iteratedDeriv s f + iteratedDeriv s g
```

lemma iteratedDeriv_const_smul [source]

Iterated derivatives of smooth functions commute with multiplication by a constant.

```
lemma iteratedDeriv_const_smul {M R : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] [Semiring
    R]
    [Module R M] [SMulCommClass ℝ R M] [ContinuousConstSMul R M] {d : ℕ}
    (s : Multiset (Fin 1 ⊕ Fin d)) (c : R) {f : SpaceTime d → M} (hf : ContDiff ℝ ∞ f) :
```

```
    iteratedDeriv s (c • f) = c • iteratedDeriv s f
```

2.160 Physlib.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Basic

Physlib/SpaceAndTime/SpaceTime/SpaceTimeAlgebra/Basic.lean on GitHub.

abbrev SpaceTimeAlgebra [source]

Formal power series in the four spacetime directions, with complex coefficients.

```
abbrev SpaceTimeAlgebra : Type
```

lemma pderiv_comm [source]

Formal partial derivatives commute.

```
lemma pderiv_comm (μ ν : Fin 1 ⊕ Fin 3) (f : SpaceTimeAlgebra) :
    pderiv μ (pderiv ν f) = pderiv ν (pderiv μ f)
```

instance RightCommutative (fun (f : SpaceTimeAlgebra) (μ : Fin 1 ⊕ Fin 3) => pde… [source]

Formal partial differentiation can be iterated over a multiset of directions.

```
instance : RightCommutative (fun (f : SpaceTimeAlgebra) (μ : Fin 1 ⊕ Fin 3) => pderiv μ f)
    where
```

def iteratedPDeriv [source]

The iterated formal partial derivative `∂^s f`, differentiating once along each element of `s`.

```
noncomputable def iteratedPDeriv (s : Multiset (Fin 1 ⊕ Fin 3)) (f : SpaceTimeAlgebra) :
    SpaceTimeAlgebra
```

lemma iteratedPDeriv_zero [source]

```
@[simp]
lemma iteratedPDeriv_zero (f : SpaceTimeAlgebra) : iteratedPDeriv 0 f = f
```

lemma iteratedPDeriv_cons [source]

```
@[simp]
lemma iteratedPDeriv_cons (μ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (f : SpaceTimeAlgebra) :
    iteratedPDeriv (μ ::ₘ s) f = iteratedPDeriv s (pderiv μ f)
```

lemma iteratedPDeriv_singleton [source]

```
@[simp]
lemma iteratedPDeriv_singleton (μ : Fin 1 ⊕ Fin 3) (f : SpaceTimeAlgebra) :
    iteratedPDeriv {μ} f = pderiv μ f
```

lemma iteratedPDeriv_add [source]

```
lemma iteratedPDeriv_add (s : Multiset (Fin 1 ⊕ Fin 3)) (f g : SpaceTimeAlgebra) :
    iteratedPDeriv s (f + g) = iteratedPDeriv s f + iteratedPDeriv s g
```

lemma constantCoeff_iteratedPDeriv [source]

The base-point value of `∂^s f` is `s!` times the coefficient of `x^s` in `f`.

```
lemma constantCoeff_iteratedPDeriv (s : Multiset (Fin 1 ⊕ Fin 3)) (f : SpaceTimeAlgebra) :
    constantCoeff (iteratedPDeriv s f) =
      ((∏ ν, (s.count ν).factorial : ℕ) : ℂ) * coeff (Multiset.toFinsupp s) f
```

lemma iteratedPDeriv_mul [source]

The all-orders Leibniz rule for iterated formal partial derivatives.

```
lemma iteratedPDeriv_mul (s : Multiset (Fin 1 ⊕ Fin 3)) (f g : SpaceTimeAlgebra) :
    iteratedPDeriv s (f * g) =
      (s.antidiagonal.map fun p => iteratedPDeriv p.1 f * iteratedPDeriv p.2 g).sum
```

lemma constantCoeff_iteratedPDeriv_mul [source]

The Leibniz rule for the base-point derivative values of a product.

```
lemma constantCoeff_iteratedPDeriv_mul (s : Multiset (Fin 1 ⊕ Fin 3)) (f g : SpaceTimeAlgebra)
    :
    constantCoeff (iteratedPDeriv s (f * g)) =
      (s.antidiagonal.map fun p =>
        constantCoeff (iteratedPDeriv p.1 f) * constantCoeff (iteratedPDeriv p.2 g)).sum
```

lemma ext_of_constantCoeff_iteratedPDeriv [source]

Series whose iterated derivatives have the same base-point values are equal.

```
lemma ext_of_constantCoeff_iteratedPDeriv {f g : SpaceTimeAlgebra}
    (h : ∀ s, constantCoeff (iteratedPDeriv s f) = constantCoeff (iteratedPDeriv s g)) :
    f = g
```

lemma eq_C_of_pderiv_eq_zero [source]

A series with vanishing first derivatives is constant.

```
lemma eq_C_of_pderiv_eq_zero {f : SpaceTimeAlgebra} (hf : ∀ μ, pderiv μ f = 0) :
    f = C (constantCoeff f)
```

def ofDerivValues [source]

The series whose base-point derivative values are `F`.

```
noncomputable def ofDerivValues (F : Multiset (Fin 1 ⊕ Fin 3) → ℂ) : SpaceTimeAlgebra
```

lemma coeff_ofDerivValues [source]

```
lemma coeff_ofDerivValues (F : Multiset (Fin 1 ⊕ Fin 3) → ℂ) (m : (Fin 1 ⊕ Fin 3) →₀ ℕ) :
    coeff m (ofDerivValues F) =
      ((∏ ν, (m ν).factorial : ℕ) : ℂ)⁻¹ * F (Finsupp.toMultiset m)
```

lemma constantCoeff_iteratedPDeriv_ofDerivValues [source]

The base-point derivative values of `ofDerivValues F` are `F`.

```
lemma constantCoeff_iteratedPDeriv_ofDerivValues (F : Multiset (Fin 1 ⊕ Fin 3) → ℂ)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    constantCoeff (iteratedPDeriv s (ofDerivValues F)) = F s
```

lemma ofDerivValues_constantCoeff_iteratedPDeriv [source]

A series is recovered from its base-point derivative values by `ofDerivValues`.

```
lemma ofDerivValues_constantCoeff_iteratedPDeriv (f : SpaceTimeAlgebra) :
    ofDerivValues (fun s => constantCoeff (iteratedPDeriv s f)) = f
```

def derivValuesEquiv [source]

The linear equivalence between series and their base-point derivative values.

```
noncomputable def derivValuesEquiv : SpaceTimeAlgebra ≃ₗ[ℂ] (Multiset (Fin 1 ⊕ Fin 3) → ℂ)
```

lemma derivValuesEquiv_apply [source]

```
@[simp]
lemma derivValuesEquiv_apply (f : SpaceTimeAlgebra) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    derivValuesEquiv f s = constantCoeff (iteratedPDeriv s f)
```

lemma derivValuesEquiv_symm_apply [source]

```
@[simp]
lemma derivValuesEquiv_symm_apply (F : Multiset (Fin 1 ⊕ Fin 3) → ℂ) :
    derivValuesEquiv.symm F = ofDerivValues F
```

2.161 Physlib.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Star

Physlib/SpaceAndTime/SpaceTime/SpaceTimeAlgebra/Star.lean on GitHub.

instance Star SpaceTimeAlgebra [source]

The star of a series in the spacetime algebra conjugates each of its coefficients.

```
instance : Star SpaceTimeAlgebra where
```

lemma coeff_star [source]

```
@[simp]
lemma coeff_star (n : (Fin 1 ⊕ Fin 3) →₀ ℕ) (f : SpaceTimeAlgebra) :
    coeff n (star f) = star (coeff n f)
```

lemma constantCoeff_star [source]

```
@[simp]
lemma constantCoeff_star (f : SpaceTimeAlgebra) :
    constantCoeff (star f) = star (constantCoeff f)
```

instance StarRing SpaceTimeAlgebra [source]

The spacetime algebra is a star ring under coefficientwise conjugation.

```
instance : StarRing SpaceTimeAlgebra where
```

lemma star_C [source]

```
@[simp]
lemma star_C (a : ℂ) : star (C a : SpaceTimeAlgebra) = C (star a)
```

lemma star_X [source]

```
@[simp]
lemma star_X (μ : Fin 1 ⊕ Fin 3) : star (X μ : SpaceTimeAlgebra) = X μ
```

instance StarModule ℝ SpaceTimeAlgebra [source]

Conjugation fixes real scalars, `star (r • f) = r • star f`.

```
instance : StarModule ℝ SpaceTimeAlgebra where
```

instance StarModule ℂ SpaceTimeAlgebra [source]

Conjugation conjugates complex scalars, `star (c • f) = star c • star f`.

```
instance : StarModule ℂ SpaceTimeAlgebra where
```

lemma pderiv_star [source]

Conjugation commutes with formal partial derivatives.

```
lemma pderiv_star (μ : Fin 1 ⊕ Fin 3) (f : SpaceTimeAlgebra) :
    pderiv μ (star f) = star (pderiv μ f)
```

lemma iteratedPDeriv_star [source]

Conjugation commutes with iterated formal partial derivatives.

```
lemma iteratedPDeriv_star (s : Multiset (Fin 1 ⊕ Fin 3)) (f : SpaceTimeAlgebra) :
    iteratedPDeriv s (star f) = star (iteratedPDeriv s f)
```

2.162 Physlib.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.TaylorSeries

Physlib/SpaceAndTime/SpaceTime/SpaceTimeAlgebra/TaylorSeries.lean on GitHub.

def taylorSeries [source]

The Taylor series at x_0 of a complex-valued function on spacetime.

```
noncomputable def taylorSeries (x₀ : SpaceTime) (f : SpaceTime → ℂ) : SpaceTimeAlgebra
```

lemma coeff_taylorSeries [source]

```
lemma coeff_taylorSeries (x₀ : SpaceTime) (f : SpaceTime → ℂ) (m : (Fin 1 ⊕ Fin 3) →₀ ℕ) :
    coeff m (taylorSeries x₀ f) =
      ((∏ ν, (m ν).factorial : ℕ) : ℂ)⁻¹ * iteratedDeriv (Finsupp.toMultiset m) f x₀
```

lemma constantCoeff_iteratedPDeriv_taylorSeries [source]

The base-point derivative values of `taylorSeries x₀ f` are the derivatives of `f` at x_0.

```
lemma constantCoeff_iteratedPDeriv_taylorSeries (x₀ : SpaceTime) (f : SpaceTime → ℂ)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    constantCoeff (iteratedPDeriv s (taylorSeries x₀ f)) = iteratedDeriv s f x₀
```

lemma taylorSeries_add [source]

```
lemma taylorSeries_add (x₀ : SpaceTime) {f g : SpaceTime → ℂ} (hf : ContDiff ℝ ∞ f)
    (hg : ContDiff ℝ ∞ g) :
    taylorSeries x₀ (f + g) = taylorSeries x₀ f + taylorSeries x₀ g
```

lemma taylorSeries_const_smul [source]

```
lemma taylorSeries_const_smul (x₀ : SpaceTime) (c : ℂ) {f : SpaceTime → ℂ}
    (hf : ContDiff ℝ ∞ f) :
    taylorSeries x₀ (c • f) = c • taylorSeries x₀ f
```

lemma taylorSeries_deriv [source]

The Taylor series of a derivative is the formal partial derivative of the Taylor series.

```
lemma taylorSeries_deriv (x₀ : SpaceTime) (μ : Fin 1 ⊕ Fin 3) {f : SpaceTime → ℂ}
    (hf : ContDiff ℝ ∞ f) :
    taylorSeries x₀ (∂_ μ f) = pderiv μ (taylorSeries x₀ f)
```

2.163 Physlib.SpaceAndTime.SpaceTime.SpaceTimeDerivAlgebra.Action

Physlib/SpaceAndTime/SpaceTime/SpaceTimeDerivAlgebra/Action.lean on GitHub.

def seriesAction [source]

The action of series on derivative symbols, defined on monomials by the product rule.

```
noncomputable def seriesAction : SpaceTimeAlgebra →+* Module.End ℂ SpaceTimeDerivAlgebraℂ where
```

lemma seriesAction_basis [source]

The action of χ on a monomial ∂^s is its Leibniz expansion.

```
lemma seriesAction_basis (χ : SpaceTimeAlgebra) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    seriesAction χ ∂^[s] =
      (s.antidiagonal.map fun t => constantCoeff (iteratedPDeriv t.1 χ) • ∂^[t.2]).sum
```

lemma pairing_seriesAction [source]

The action of χ multiplies by χ before applying the symbol.

```
lemma pairing_seriesAction (χ : SpaceTimeAlgebra) (p : SpaceTimeDerivAlgebraℂ)
    (f : SpaceTimeAlgebra) :
    pairing (seriesAction χ p) f = pairing p (χ * f)
```

lemma seriesAction_C [source]

The constant series C c acts as scalar multiplication by c.

```
@[simp]
lemma seriesAction_C (c : ℂ) : seriesAction (C c) = c • LinearMap.id
```

lemma seriesAction_apply_one [source]

A series acts on the unit symbol by its base-point value.

```
@[simp]
lemma seriesAction_apply_one (χ : SpaceTimeAlgebra) :
    seriesAction χ 1 = constantCoeff χ • 1
```

lemma seriesAction_basis_singleton [source]

A series χ sends ∂_μ to χ(x₀) ∂_μ + (∂_μ χ)(x₀), the product rule.

```
lemma seriesAction_basis_singleton (χ : SpaceTimeAlgebra) (μ : Fin 1 ⊕ Fin 3) :
    seriesAction χ ∂[μ] = constantCoeff χ • ∂[μ] + constantCoeff (pderiv μ χ) • 1
```

lemma seriesAction_mul_basis_singleton [source]

The action commutes with multiplication by ∂_μ up to the action of pderiv μ χ.

```
lemma seriesAction_mul_basis_singleton (χ : SpaceTimeAlgebra) (p : SpaceTimeDerivAlgebraℂ)
    (μ : Fin 1 ⊕ Fin 3) :
    seriesAction χ (p * ∂[μ]) = seriesAction χ p * ∂[μ] + seriesAction (pderiv μ χ) p
```

2.164 Physlib.SpaceAndTime.SpaceTime.SpaceTimeDerivAlgebra.Basic

Physlib/SpaceAndTime/SpaceTime/SpaceTimeDerivAlgebra/Basic.lean on GitHub.

abbrev SpaceTimeDerivAlgebraℂ [source]

The algebra of derivative symbols on spacetime, the polynomials in four commuting symbols ∂_μ.

```
abbrev SpaceTimeDerivAlgebra𝕜 : Type
```

def basis [source]

The monomial basis of the derivative algebra, in which `basis s` is the monomial `∂^s`.

```
noncomputable def basis : Module.Basis (Multiset (Fin 1 ⊕ Fin 3)) 𝕜 SpaceTimeDerivAlgebra𝕜
```

lemma basis_apply [source]

The monomial `∂^s` is the polynomial monomial whose exponents are the multiplicities in `s`.

```
lemma basis_apply (s : Multiset (Fin 1 ⊕ Fin 3)) :
    ∂^[s] = (SymmetricAlgebra.equivMvPolynomial Lorentz.ComplexCoVector.basis.dualBasis).symm
      (MvPolynomial.monomial (Multiset.toFinsupp s) 1)
```

lemma basis_zero [source]

The empty monomial is the unit.

```
@[simp]
lemma basis_zero : ∂^[0] = 1
```

lemma basis_singleton [source]

The monomial of a single direction `μ` is the derivative symbol `∂_μ`.

```
lemma basis_singleton (μ : Fin 1 ⊕ Fin 3) :
    ∂[μ] = SymmetricAlgebra.ι 𝕜 _ (Lorentz.ComplexCoVector.basis.dualBasis μ)
```

lemma basis_mul [source]

Monomials multiply by adding multisets, `∂^s ∂^t = ∂^(s + t)`.

```
lemma basis_mul (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    ∂^[s] * ∂^[t] = ∂^[s + t]
```

def pairing [source]

The pairing of a symbol `P` with a series `f`, given by the value of `P f` at the base point.

```
noncomputable def pairing : SpaceTimeDerivAlgebra𝕜 →ₗ[𝕜] SpaceTimeAlgebra →ₗ[𝕜] 𝕜
```

lemma pairing_basis [source]

The monomial `∂^s` pairs with `f` to the base-point value `(∂^s f)(x₀)`.

```
lemma pairing_basis (s : Multiset (Fin 1 ⊕ Fin 3)) (f : SpaceTimeAlgebra) :
    pairing ∂^[s] f = constantCoeff (iteratedPDeriv s f)
```

lemma pairing_one [source]

The unit symbol pairs with `f` to its base-point value `f(x₀)`.

```
lemma pairing_one (f : SpaceTimeAlgebra) : pairing 1 f = constantCoeff f
```

lemma pairing_monomial [source]

Pairing a symbol with the series `x^s` gives `s!` times the coefficient of `∂^s` in the symbol.

```
lemma pairing_monomial (p : SpaceTimeDerivAlgebraℂ) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    pairing p (monomial (Multiset.toFinsupp s) 1) =
      ((∏ ν, (s.count ν).factorial : ℕ) : ℂ) * basis.repr p s
```

lemma pairing_injective [source]

The pairing separates derivative symbols.

```
lemma pairing_injective : Function.Injective pairing
```

lemma ext_of_pairing_eq [source]

Symbols that pair equally with every series are equal.

```
lemma ext_of_pairing_eq {p q : SpaceTimeDerivAlgebraℂ} (h : ∀ f, pairing p f = pairing q f) :
    p = q
```

lemma pairing_mul_basis_singleton [source]

Multiplying a symbol by `∂_μ` corresponds to applying `pderiv μ` to the series.

```
lemma pairing_mul_basis_singleton (p : SpaceTimeDerivAlgebraℂ) (μ : Fin 1 ⊕ Fin 3)
    (f : SpaceTimeAlgebra) :
    pairing (p * ∂[μ]) f = pairing p (pderiv μ f)
```

lemma pairing_basis_mul [source]

Pairing `∂^s` with a product `χ f` expands by the Leibniz rule.

```
lemma pairing_basis_mul (s : Multiset (Fin 1 ⊕ Fin 3)) (χ f : SpaceTimeAlgebra) :
    pairing ∂^[s] (χ * f) =
      (s.antidiagonal.map fun t =>
        constantCoeff (iteratedPDeriv t.1 χ) * pairing ∂^[t.2] f).sum
```

lemma pairing_constr_leibniz [source]

Replacing each monomial `∂^s` of `p` by its Leibniz expansion with `χ` gives a symbol that pairs with `f` as `p` pairs with `χ f`.

```
lemma pairing_constr_leibniz (χ : SpaceTimeAlgebra) (p : SpaceTimeDerivAlgebraℂ)
    (f : SpaceTimeAlgebra) :
    pairing (basis.constr ℂ (fun s => (s.antidiagonal.map fun t =>
      constantCoeff (iteratedPDeriv t.1 χ) • ∂^[t.2]).sum) p) f = pairing p (χ * f)
```

2.165 Physlib.SpaceAndTime.Time.Basic

Physlib/SpaceAndTime/Time/Basic.lean on GitHub.

lemma chartAt_apply [source]

The chart of `Time` at `t₀` is `t ↦ t -ᵥ t₀`.

```
@[simp]
lemma chartAt_apply (t₀ t : Time) : chartAt ℝ t₀ t = t -ᵥ t₀
```

lemma chartAt_symm_apply [source]

The inverse of the chart of `Time` at `t₀` is `τ ↦ τ +ᵥ t₀`.

```
@[simp]
lemma chartAt_symm_apply (t₀ : Time) (τ : ℝ) : (chartAt ℝ t₀).symm τ = τ +ᵥ t₀
```

lemma extChartAt_apply [source]

The extended chart of `Time` at `t₀` is `t ↦ t -ᵥ t₀`.

```
lemma extChartAt_apply (t₀ t : Time) : extChartAt 𝓘(ℝ, ℝ) t₀ t = t -ᵥ t₀
```

lemma contMDiff_vadd [source]

The map `τ ↦ τ +ᵥ t₀` from ℝ to `Time` is smooth.

```
lemma contMDiff_vadd (t₀ : Time) (n : ℕ∞ω) :
    ContMDiff 𝓘(ℝ, ℝ) 𝓘(ℝ, ℝ) n (fun τ : ℝ ↦ τ +ᵥ t₀)
```

lemma contMDiff_vsub [source]

The map `t ↦ t -ᵥ t₀` from `Time` to ℝ is smooth.

```
lemma contMDiff_vsub (t₀ : Time) (n : ℕ∞ω) :
    ContMDiff 𝓘(ℝ, ℝ) 𝓘(ℝ, ℝ) n (fun t : Time ↦ t -ᵥ t₀)
```

2.166 Physlib.SpaceAndTime.Time.Derivatives

Physlib/SpaceAndTime/Time/Derivatives.lean on GitHub.

lemma hasDerivAt_mk_of_differentiableAt [source]

A curve `f : Time → ℝ` differentiable at `t` gives the curve `σ ↦ f ⟨σ⟩` on ℝ, whose Mathlib derivative at `t.val` is the time derivative `∂ₜ f t`.

```
lemma hasDerivAt_mk_of_differentiableAt {f : Time → ℝ} {t : Time}
    (hf : DifferentiableAt ℝ f t) :
    HasDerivAt (fun σ : ℝ => f ⟨σ⟩) (∂ₜ f t) t.val
```

2.167 Physlib.SpaceAndTime.Time.ManifoldCurves

Physlib/SpaceAndTime/Time/ManifoldCurves.lean on GitHub.

lemma continuousAt_of_continuousAt_vadd [source]

Continuity of the coordinate curve at `0` gives continuity at `x`.

```
private lemma continuousAt_of_continuousAt_vadd {X : Type*} [TopologicalSpace X]
    {γ : Time → X} {x : Time} (h : ContinuousAt (fun τ : ℝ ↦ γ (τ +ᵥ x)) 0) : ContinuousAt γ x
```

lemma contMDiffAt_iff_contDiffAt_vadd [source]

A curve into a normed space is `C^n` at `x` iff its coordinate curve is `C^n` at `0`.

```
lemma contMDiffAt_iff_contDiffAt_vadd (γ : Time → F) (x : Time) (n : ℕ∞ω) :
    ContMDiffAt 𝓘(ℝ, ℝ) 𝓘(ℝ, F) n γ x ↔ ContDiffAt ℝ n (fun τ : ℝ ↦ γ (τ +ᵥ x)) 0
```

lemma mdifferentiableAt_iff_differentiableAt_vadd [source]

A curve into a normed space is differentiable at `x` iff its coordinate curve is at `0`.

```
lemma mdifferentiableAt_iff_differentiableAt_vadd (γ : Time → F) (x : Time) :
    MDifferentiableAt 𝓘(ℝ, ℝ) 𝓘(ℝ, F) γ x ↔ DifferentiableAt ℝ (fun τ : ℝ ↦ γ (τ +ᵥ x)) 0
```

lemma vadd_vadd_comp [source]

Shifting the base instant by `s` shifts the coordinate curve by `s`.

```
private lemma vadd_vadd_comp {X : Type*} (γ : Time → X) (t₀ : Time) (s : ℝ) :
    (fun τ : ℝ ↦ γ (τ +ᵥ (s +ᵥ t₀))) = fun τ ↦ (fun σ : ℝ ↦ γ (σ +ᵥ t₀)) (τ + s)
```

lemma contMDiff_iff_contDiff_vadd [source]

A curve `γ : Time → F` into a normed space is `C^n` iff the coordinate curve `τ ↦ γ (τ +ᵥ t₀)` is `C^n`.

```
lemma contMDiff_iff_contDiff_vadd (γ : Time → F) (t₀ : Time) (n : ℕ∞ω) :
    ContMDiff 𝓘(ℝ, ℝ) 𝓘(ℝ, F) n γ ↔ ContDiff ℝ n (fun τ : ℝ ↦ γ (τ +ᵥ t₀))
```

lemma mdifferentiableAt_vadd_iff [source]

A curve into a normed space is differentiable at `s +ᵥ t₀` iff its coordinate curve based at `t₀` is differentiable at `s`.

```
lemma mdifferentiableAt_vadd_iff (γ : Time → F) (t₀ : Time) (s : ℝ) :
    MDifferentiableAt 𝓘(ℝ, ℝ) 𝓘(ℝ, F) γ (s +ᵥ t₀) ↔
      DifferentiableAt ℝ (fun τ : ℝ ↦ γ (τ +ᵥ t₀)) s
```

lemma writtenInExtChartAt_eq_vadd [source]

A curve read in the extended chart at `x` is its coordinate curve based at `x`.

```
lemma writtenInExtChartAt_eq_vadd (γ : Time → F) (x : Time) :
    writtenInExtChartAt 𝓘(ℝ, ℝ) 𝓘(ℝ, F) x γ = fun τ : ℝ ↦ γ (τ +ᵥ x)
```

lemma mfderiv_eq_deriv_vadd [source]

The manifold derivative of a curve into a normed space at `s +ᵥ t₀` applied to `1` equals the ordinary derivative of the coordinate curve at `s`.

```
lemma mfderiv_eq_deriv_vadd (γ : Time → F) (t₀ : Time) (s : ℝ) :
    mfderiv 𝓘(ℝ, ℝ) 𝓘(ℝ, F) γ (s +ᵥ t₀) (1 : ℝ) = deriv (fun τ : ℝ ↦ γ (τ +ᵥ t₀)) s
```

lemma tangentMap_eq_deriv_vadd [source]

The tangent map of a curve into a normed space sends `⟨s +ᵥ t₀, 1⟩` to the point and the ordinary derivative of the coordinate curve.

```
lemma tangentMap_eq_deriv_vadd (γ : Time → F) (t₀ : Time) (s : ℝ) :
    tangentMap 𝓘(ℝ, ℝ) 𝓘(ℝ, F) γ ⟨s +ᵥ t₀, (1 : ℝ)⟩ =
      ⟨γ (s +ᵥ t₀), deriv (fun τ : ℝ ↦ γ (τ +ᵥ t₀)) s⟩
```

2.168 Physlib.SpaceAndTime.WaveVectorSpace.Basic

Physlib/SpaceAndTime/WaveVectorSpace/Basic.lean on GitHub.

structure WaveVectorSpace [source]

The type `WaveVectorSpace d` is the real vector space which corresponds to `d`-dimensional (angular) wave vectors in `Space d` with a given (but arbitrary) choice of inverse length unit.

The default value of `d` is `3`. Thus `WaveVectorSpace = WaveVectorSpace 3`

```
structure WaveVectorSpace (d : ℕ := 3) where
  val : Fin d → ℝ
```

lemma eq_of_val [source]

```
lemma eq_of_val (h : k₁.val = k₂.val) : k₁ = k₂
```

lemma val_eq_iff [source]

```
@[simp]
lemma val_eq_iff : k₁.val = k₂.val ↔ k₁ = k₂
```

instance CoeFun (WaveVectorSpace d) (fun _ ↦ Fin d → ℝ) [source]

```
instance : CoeFun (WaveVectorSpace d) (fun _ ↦ Fin d → ℝ) where
```

lemma eq_of_apply [source]

```
@[ext]
lemma eq_of_apply (h : ∀ i, k₁ i = k₂ i) : k₁ = k₂
```

instance Subsingleton (WaveVectorSpace 0) [source]

```
instance : Subsingleton (WaveVectorSpace 0) where
```

instance Zero (WaveVectorSpace d) [source]

```
instance : Zero (WaveVectorSpace d) where
```

lemma zero_val [source]

```
@[simp]
lemma zero_val : (0 : WaveVectorSpace d).val = 0
```

instance Add (WaveVectorSpace d) [source]

```
instance : Add (WaveVectorSpace d) where
```

lemma add_val [source]

```
@[simp]
lemma add_val : (k₁ + k₂).val = k₁.val + k₂.val
```

instance Sub (WaveVectorSpace d) [source]

```
instance : Sub (WaveVectorSpace d) where
```

lemma sub_val [source]

```
@[simp]
lemma sub_val : (k₁ - k₂).val = k₁.val - k₂.val
```

instance Neg (WaveVectorSpace d) [source]

```
instance : Neg (WaveVectorSpace d) where
```

lemma neg_val [source]

```
@[simp]
lemma neg_val : (-k).val = -k.val
```

instance AddCommGroup (WaveVectorSpace d) [source]

```
instance : AddCommGroup (WaveVectorSpace d) where
```

instance SMul ℝ (WaveVectorSpace d) [source]

```
instance : SMul ℝ (WaveVectorSpace d) where
```

lemma smul_val [source]

```
@[simp]
lemma smul_val : (c • k).val = c • k.val
```

instance instModuleReal [source]

```
instance instModuleReal : Module ℝ (WaveVectorSpace d) where
```

instance Inner ℝ (WaveVectorSpace d) [source]

```
instance : Inner ℝ (WaveVectorSpace d) where
```

lemma inner_apply [source]

```
lemma inner_apply : ⟪k₁, k₂⟫ = ∑ i, k₁ i * k₂ i
```

instance InnerProductSpace.Core ℝ (WaveVectorSpace d) [source]

```
instance : InnerProductSpace.Core ℝ (WaveVectorSpace d) where
```

instance instNormedAddCommGroup [source]

```
noncomputable instance instNormedAddCommGroup : NormedAddCommGroup (WaveVectorSpace d)
```

instance instInnerProductSpaceReal [source]

```
noncomputable instance instInnerProductSpaceReal : InnerProductSpace ℝ (WaveVectorSpace d)
```

lemma norm_eq [source]

```
lemma norm_eq : ‖k‖ = √(∑ i, k i ^ 2)
```

lemma norm_sq_eq [source]

```
lemma norm_sq_eq : ‖k‖ ^ 2 = ∑ i, k i ^ 2
```

def equivEuclideanSpace [source]

The linear isometric equivalence between `WaveVectorSpace d` and $\mathbb{R}^d$.

```
def equivEuclideanSpace : WaveVectorSpace d ≃ₗᵢ[ℝ] EuclideanSpace ℝ (Fin d) where
```

lemma finrank_waveVectorSpace [source]

```
@[simp]
lemma finrank_waveVectorSpace : Module.finrank ℝ (WaveVectorSpace d) = d
```

2.169 Physlib.Units.Basic

Physlib/Units/Basic.lean on GitHub.

def dimScaleUnits [source]

The scaling factor `dimScale u1 u2 d` regarded as a unit of `ℝ≥0`, that is as a strictly positive real. Dimensionful quantities are scaled by this rather than by `dimScale u1 u2 d` itself, so that a type whose elements are strictly positive, such as `SpeedOfLight`, can carry a dimension: no action of `ℝ≥0` on such a type can be compatible with the magnitude, since compatibility at `a = 0` would give `val (0 • m) = 0` while `val` is strictly positive.

```
noncomputable def dimScaleUnits (u1 u2 : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
    ℝ≥0ˣ
```

lemma dimScaleUnits_coe [source]

```
@[simp]
lemma dimScaleUnits_coe (u1 u2 : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
    ((dimScaleUnits u1 u2 d : ℝ≥0)) = dimScale u1 u2 d
```

lemma dimScaleUnits_transitive [source]

```
@[simp]
lemma dimScaleUnits_transitive (u1 u2 u3 : LTMCTUnitChoices)
    (d : Dimension LTMCTDimensionBase) :
    dimScaleUnits u1 u2 d * dimScaleUnits u2 u3 d = dimScaleUnits u1 u3 d
```

lemma dimScaleUnits_smul [source]

Scaling by `dimScaleUnits` agrees with scaling by `dimScale` whenever the type also carries an action of `ℝ≥0`, so that statements phrased either way normalise to the same form.

```
lemma dimScaleUnits_smul {M : Type} [MulAction ℝ≥0 M] (u1 u2 : LTMCTUnitChoices)
    (d : Dimension LTMCTDimensionBase) (m : M) :
    dimScaleUnits u1 u2 d • m = dimScale u1 u2 d • m
```

lemma dimScaleUnits_mul_dim [source]

```
@[simp]
lemma dimScaleUnits_mul_dim (u1 u2 : LTMCTUnitChoices)
    (d1 d2 : Dimension LTMCTDimensionBase) :
    dimScaleUnits u1 u2 (d1 * d2) = dimScaleUnits u1 u2 d1 * dimScaleUnits u1 u2 d2
```

lemma dimScaleUnits_self [source]

```
@[simp]
lemma dimScaleUnits_self (u : LTMCTUnitChoices) (d : Dimension LTMCTDimensionBase) :
    dimScaleUnits u u d = 1
```

instance MulAction ℝ≥0ˣ (Dimensionful M) [source]

```
instance {M : Type} [CarriesDimension M] : MulAction ℝ≥0ˣ (Dimensionful M) where
```

instance MulAction ℝ≥0 (Dimensionful M) [source]

When `M` itself carries an action of `ℝ≥0`, a dimensionful quantity of `M` may be scaled by any non-negative real, not only by a strictly positive one.

```
instance {M : Type} [HasDim M] [MulAction ℝ≥0 M] :
    MulAction ℝ≥0 (Dimensionful M) where
```

lemma smul_apply_units [source]

```
@[simp]
lemma Dimensionful.smul_apply_units {M : Type} [CarriesDimension M]
    (a : ℝ≥0ˣ) (f : Dimensionful M) (u : LTMCTUnitChoices) :
    (a • f).1 u = a • f.1 u
```

lemma toDimensionful_apply_apply_units [source]

The value of `toDimensionful`, scaled by `dimScaleUnits`. This is the general form, holding for every type carrying a dimension.

```
lemma CarriesDimension.toDimensionful_apply_apply_units
    {M : Type} [CarriesDimension M] (u1 u2 : LTMCTUnitChoices) (m : M) :
    (toDimensionful u1 m).1 u2 = (u1.dimScaleUnits u2 (dim M)) • m
```

2.170 Physlib.Units.Constants

Physlib/Units/Constants.lean on GitHub.

instance HasDim SpeedOfLight [source]

The speed of light has the dimension of a length divided by a time.

```
instance : HasDim SpeedOfLight where
```

def speedOfLight [source]

The speed of light in our world, `299792458 m s⁻¹`, which is exact in SI units.

```
noncomputable def speedOfLight : Dimensionful SpeedOfLight
```

lemma speedOfLight_in_SI [source]

```
@[simp]
lemma speedOfLight_in_SI : speedOfLight SI = ⟨299792458, by norm_num⟩
```

2.171 Physlib.Units.PositiveRealUnit

Physlib/Units/PositiveRealUnit.lean on GitHub.

def unitsSMul [source]

The rescaling of a unit magnitude by a strictly positive real, given as a unit of `ℝ≥0`. This is `scale` at a factor known to be positive, so the `scale` lemmas may be applied to `u • x` after rewriting with `units_smul_eq_scale`.

```
def unitsSMul (u : ℝ≥0ˣ) (x : U) : U
```

instance instMulActionUnitsNNReal [source]

A unit magnitude may be rescaled by a strictly positive real, that is by a unit of `ℝ≥0`. No action of `ℝ≥0` on `U` can be compatible with the magnitude: taking `a > 0` and `b = 0` in `mul_smul` would force `0 • x` to be a fixed point of every rescaling, and no element of `U` is.

```
instance instMulActionUnitsNNReal : MulAction ℝ≥0ˣ U where
```

lemma units_smul_eq_scale [source]

A rescaling by a unit of `ℝ≥0` is the rescaling by its underlying positive factor. Rewriting with this makes the `scale` lemmas applicable; `simp` does not see through `HSMul.hSMul` on its own.

```
lemma units_smul_eq_scale (u : ℝ≥0ˣ) (x : U) :
    u • x = scale ((u : ℝ≥0) : ℝ) x (by
      have hu : ((u : ℝ≥0) : ℝ) ≠ 0 := by simp [Units.ne_zero u]
      exact lt_of_le_of_ne (by positivity) (Ne.symm hu))
```

lemma val_units_smul [source]

The magnitude of a rescaling by a unit of `ℝ≥0`.

```
@[simp]
lemma val_units_smul (u : ℝ≥0ˣ) (x : U) :
    val (u • x) = ((u : ℝ≥0) : ℝ) * val x
```

2.172 Physlib.Units.UnitDependent

Physlib/Units/UnitDependent.lean on GitHub.

lemma dimScaleUnits_scaleUnit [source]

```
@[simp]
lemma LTMCTUnitChoices.dimScaleUnits_scaleUnit {u1 u2 u : LTMCTUnitChoices}
    (d : Dimension LTMCTDimensionBase) :
    u.dimScaleUnits (scaleUnit u1 u2 u) d = u1.dimScaleUnits u2 d
```

lemma of_scaleUnit_units [source]

```
lemma Dimensionful.of_scaleUnit_units {M : Type} [CarriesDimension M] {u1 u2 u :
    LTMCTUnitChoices}
    (c : Dimensionful M) :
    c.1 (scaleUnit u1 u2 u) =
    u1.dimScaleUnits u2 (dim M) • c.1 (u)
```

instance UnitDependent M1 [source]

```
noncomputable instance {M1 : Type} [CarriesDimension M1] : UnitDependent M1 where
```

lemma scaleUnit_apply_units [source]

```
lemma HasDim.scaleUnit_apply_units {M : Type} [CarriesDimension M]
    (u1 u2 : LTMCTUnitChoices) (m : M) :
    scaleUnit u1 u2 m = (u1.dimScaleUnits u2 (dim M)) • m
```

instance MulUnitDependent M1 [source]

A type carrying a dimension which also has a `MulAction ℝ≥0` compatible with the scaling by units is `MulUnitDependent`. A type whose elements are strictly positive carries no action of `ℝ≥0` compatible with its magnitude, and so is `UnitDependent` only.

```
noncomputable instance {M1 : Type} [HasDim M1] [MulAction ℝ≥0 M1] : MulUnitDependent M1 where
```

2.173 Physlib.Units.WithDim.Basic

Physlib/Units/WithDim/Basic.lean on GitHub.

lemma smul_val_units [source]

The underlying value of a scaling by a unit of `ℝ≥0`, that is by a strictly positive real.

```
@[simp]
lemma smul_val_units {B : Type} [DimensionBasis B] {d : Dimension B} {M : Type}
    [MulAction ℝ≥0 M] (a : ℝ≥0ˣ) (m : WithDim d M) :
    (a • m).val = (a : ℝ≥0) • m.val
```

def ofPositiveRealUnit [source]

A positive-real quantity which carries a dimension, regarded as a dimension-tagged real, so that it can be used alongside the other dimension-tagged quantities. The dimension is read off the `HasDim` instance, so the two cannot drift apart.

```
def ofPositiveRealUnit {U : Type} [PositiveRealUnitCore U] [HasDim U] (x : U) :
    WithDim (dim U) ℝ
```

lemma ofPositiveRealUnit_val [source]

```
@[simp]
lemma ofPositiveRealUnit_val {U : Type} [PositiveRealUnitCore U] [HasDim U] (x : U) :
    (ofPositiveRealUnit x).val = PositiveRealUnitCore.val x
```

lemma ofPositiveRealUnit_units_smul [source]

```
@[simp]
lemma ofPositiveRealUnit_units_smul {U : Type} [PositiveRealUnitCore U] [HasDim U]
    (a : ℝ≥0ˣ) (x : U) :
    ofPositiveRealUnit (a • x) = a • ofPositiveRealUnit x
```

lemma ofPositiveRealUnit_of_scaleUnit [source]

A dimensionful positive-real quantity, such as the speed of light, may be tagged with its dimension either before or after a change of units: `ofPositiveRealUnit` commutes with `scaleUnit`.

```
lemma ofPositiveRealUnit_of_scaleUnit {U : Type} [PositiveRealUnitCore U] [HasDim U]
    {u1 u2 u : LTMCTUnitChoices} (c : Dimensionful U) :
    ofPositiveRealUnit (c.1 (UnitDependent.scaleUnit u1 u2 u)) =
    UnitDependent.scaleUnit u1 u2 (ofPositiveRealUnit (c.1 u))
```

lemma scaleUnit_val_pos [source]

Scaling by a change of units preserves positivity of the value of a dimensionful quantity.

```
lemma scaleUnit_val_pos {d : Dimension LTMCTDimensionBase} (m : WithDim d ℝ≥0) (hm : 0 < m.val)
    (u1 u2 : LTMCTUnitChoices) : 0 < (scaleUnit u1 u2 m).val
```

2.174 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Basic

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/Basic.lean on GitHub.

structure GaugeBoson [source]

The target vector space of the gauge-boson field: a spacetime covector with values in the gauge algebra. Its components are the fields `A_μ^a`; here the adjoint index is kept abstract, as the gauge-algebra factor.

```
@[ext]
structure GaugeBoson where
  val : Lorentz.CoVector ⊗[ℝ] 𝔤
```

def valEquiv [source]

Identifies a gauge boson with its underlying tensor-product value.

```
def valEquiv : (GaugeBoson 𝔤) ≃ Lorentz.CoVector ⊗[ℝ] 𝔤 where
```

instance AddCommGroup (GaugeBoson 𝔤) [source]

```
noncomputable instance : AddCommGroup (GaugeBoson 𝔤)
```

instance Module ℝ (GaugeBoson 𝔤) [source]

```
noncomputable instance : Module ℝ (GaugeBoson 𝔤)
```

def valLinEquiv [source]

The linear identification with the underlying tensor product.

```
def valLinEquiv : (GaugeBoson 𝔤) ≃ₗ[ℝ] Lorentz.CoVector ⊗[ℝ] 𝔤 where
```

lemma valLinEquiv_apply [source]

```
@[simp]
lemma valLinEquiv_apply (v : (GaugeBoson 𝔤)) : (valLinEquiv 𝔤) v = v.val
```

lemma valLinEquiv_symm_apply [source]

```
lemma valLinEquiv_symm_apply (m : Lorentz.CoVector ⊗[ℝ] 𝔤) :
    (valLinEquiv 𝔤).symm m = ⟨m⟩
```

lemma val_add [source]

```
@[simp]
lemma val_add (v₁ v₂ : (GaugeBoson 𝔤)) : (v₁ + v₂).val = v₁.val + v₂.val
```

lemma val_smul [source]

```
@[simp]
lemma val_smul (r : ℝ) (v : (GaugeBoson 𝔤)) : (r • v).val = r • v.val
```

instance Module.Finite ℝ (GaugeBoson 𝔤) [source]

```
instance : Module.Finite ℝ (GaugeBoson 𝔤)
```

def repLorentzGroup [source]

The Lorentz action on the gauge-boson target space: the covector action on the spacetime index, and the trivial action on the gauge-algebra factor.

```
noncomputable def repLorentzGroup : Representation ℝ SL(2,ℂ) (GaugeBoson 𝔤) where
```

def repValue [source]

The global gauge action on the gauge-boson target space: the adjoint action on the gauge-algebra factor, and the trivial action on the spacetime index.

```
noncomputable def repValue {G₀ : Type} [Monoid G₀] (ρ : Representation ℝ G₀ 𝔤) :
    Representation ℝ G₀ (GaugeBoson 𝔤) where
```

abbrev JetComponentSpace [source]

The jet component space of the gauge-boson field: the span of the component functions `∂_s A_μ^φ`. The `SpaceTimeDerivAlgebra`ℝ factor carries the derivative label `s`, and the dual factor the spacetime and adjoint indices — the latter as an abstract covector on the gauge algebra, with no basis chosen. Unlike a matter field, the gauge boson is real, so there is no conjugate half.

```
abbrev JetComponentSpace : Type
```

def componentDual [source]

The covector on the gauge-boson target space pairing the spacetime index against a covector `ω` and the adjoint index against `φ`.

```
noncomputable def componentDual :
    Module.Dual ℝ Lorentz.CoVector →ₗ[ℝ]
      Module.Dual ℝ 𝔤 →ₗ[ℝ] Module.Dual ℝ (GaugeBoson 𝔤) where
```

lemma componentDual_apply_val_tmul [source]

```
@[simp]
lemma componentDual_apply_val_tmul (ω : Module.Dual ℝ Lorentz.CoVector)
    (φ : Module.Dual ℝ 𝔤) (v : Lorentz.CoVector) (a : 𝔤) :
    (componentDual 𝔤) ω φ (v ⊗ₜ[ℝ] a) = ω v * φ a
```

2.175 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.Basic

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/LocalGauge/Basic.lean on GitHub.

abbrev LocalGaugeFieldAlgebra [source]

The local gauge field algebra, the jet algebra of the gauge bosons: the free commutative algebra on the component functions `∂_s A_μ^φ` of the gauge-boson field, realized as the symmetric algebra on the jet component space. The commutativity of the product is the Bose statistics of the gauge fields.

```
abbrev LocalGaugeFieldAlgebra : Type
```

def ofComponent [source]

The undifferentiated component function `A^φ` of the gauge-boson field along a covector `φ` on the target space.

```
noncomputable def ofComponent : Module.Dual ℝ (GaugeBoson 𝔤) →ₗ[ℝ] (LocalGaugeFieldAlgebra 𝔤)
```

lemma ofComponent_apply [source]

```
lemma ofComponent_apply (φ : Module.Dual ℝ (GaugeBoson 𝔤)) :
    (ofComponent 𝔤) φ = SymmetricAlgebra.ι ℝ _ ((1 : SpaceTimeDerivAlgebraℝ) ⊗ₜ[ℝ] φ)
```

def ofA [source]

***The component function `A_μ^φ` of the gauge-boson field**: the spacetime index `μ` paired against the Lorentz coordinate basis, the adjoint index against the abstract covector `φ` on the gauge algebra. These are the generators the ambient theory sees; no basis of the gauge algebra is involved.*

```
noncomputable def ofA (μ : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] (LocalGaugeFieldAlgebra 𝔤)
```

lemma ofA_apply [source]

```
lemma ofA_apply (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    (ofA 𝔤) μ φ = (ofComponent 𝔤) ((GaugeBoson.componentDual 𝔤)
      (Lorentz.CoVector.basis.dualBasis μ) φ)
```

lemma adjoin_ι_eq_top [source]

The jet algebra is generated by the component functions.

```
lemma adjoin_ι_eq_top :
    Algebra.adjoin ℝ (Set.range (SymmetricAlgebra.ι ℝ (GaugeBoson.JetComponentSpace 𝔤))) = ⊤
```

2.176 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.FieldStrength

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/LocalGauge/FieldStrength.lean on GitHub.

def derivA [source]

The derivative symbols `∂_s A_μ^φ` of the local gauge field algebra, as a family over the derivative multiset `s` and the spacetime index `μ`: the iterated total derivative of the gauge-field generator. The field strength and its covariant derivatives are built out of this family.

```
noncomputable def derivA (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] LocalGaugeFieldAlgebra 𝔤
```

lemma derivA_apply [source]

```
lemma derivA_apply (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) : derivA 𝔤 s μ φ = iteratedJetDeriv 𝔤 s (ofA 𝔤 μ φ)
```

lemma derivA_zero [source]

```
@[simp]
lemma derivA_zero (μ : Fin 1 ⊕ Fin 3) : derivA 𝔤 0 μ = ofA 𝔤 μ
```

lemma derivA_eq_iteratedJetDeriv [source]

The derivative symbols are the iterated derivatives of the undifferentiated symbol.

```
lemma derivA_eq_iteratedJetDeriv (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) : derivA 𝔤 s μ φ = iteratedJetDeriv 𝔤 s (derivA 𝔤 0 μ φ)
```

lemma gaugeField_eq_one_tmul_derivA [source]

The complexified symbols `gaugeField` are the images of the real symbols.

```
lemma gaugeField_eq_one_tmul_derivA (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) : gaugeField 𝔤 s μ φ = (1 : ℂ) ⊗ₜ[ℝ] derivA 𝔤 s μ φ
```

lemma algHom_ext [source]

Two algebra maps out of the local gauge field algebra agreeing on the derivative symbols `∂_s A_μ^φ` are equal.

```
lemma algHom_ext {B : Type} [Semiring B] [Algebra ℝ B] {f g : LocalGaugeFieldAlgebra 𝔤 →ₐ[ℝ] B}
    (h : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
      f (derivA 𝔤 s μ φ) = g (derivA 𝔤 s μ φ)) : f = g
```

def fieldStrength [source]

The field strength `F_μν = ∂_μ A_ν − ∂_ν A_μ + ⁅A_μ, A_ν⁆` of the local gauge field algebra: the underived field strength of the family of derivative symbols, in the conventions of `GaugeAlgebraRealization.fieldStrength`, where the bracket of the gauge algebra carries the physicists' factor of `i`.

```
noncomputable def fieldStrength (μ ν : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] LocalGaugeFieldAlgebra 𝔤
```

lemma fieldStrength_apply [source]

The field strength written in the generators of the algebra.

```
lemma fieldStrength_apply (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    fieldStrength 𝔤 μ ν φ = jetDeriv 𝔤 μ (ofA 𝔤 ν φ) - jetDeriv 𝔤 ν (ofA 𝔤 μ φ)
      + bracketFam (ofA 𝔤 μ) (ofA 𝔤 ν) φ
```

lemma fieldStrength_swap [source]

The field strength is antisymmetric in its two covector indices.

```
lemma fieldStrength_swap (μ ν : Fin 1 ⊕ Fin 3) :
    fieldStrength 𝔤 ν μ = - fieldStrength 𝔤 μ ν
```

def covDerivFieldStrength [source]

The iterated covariant derivative `∇_{l₁} ⋯ ∇_{lₙ} F_μν` of the field strength along an ordered list of directions, with `∇_ρ F = ∂_ρ F + ⁅A_ρ, F⁆` the covariant derivative in the adjoint. Covariant derivatives do not commute, so the iteration is indexed by a list and not by a multiset; the empty list gives the field strength itself.

```
noncomputable def covDerivFieldStrength (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] LocalGaugeFieldAlgebra 𝔤
```

lemma covDerivFieldStrength_nil [source]

Zero covariant derivatives: the field strength itself.

```
@[simp]
lemma covDerivFieldStrength_nil (μ ν : Fin 1 ⊕ Fin 3) :
    covDerivFieldStrength 𝔤 [] μ ν = fieldStrength 𝔤 μ ν
```

lemma iteratedJetDeriv_bracketFam [source]

The Leibniz rule of the iterated total derivative on a bracket of families: the antidiagonal convolution of the iterated derivatives of the two factors.

```
lemma iteratedJetDeriv_bracketFam (s : Multiset (Fin 1 ⊕ Fin 3))
    (f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] LocalGaugeFieldAlgebra 𝔤) (φ : Module.Dual ℝ 𝔤) :
    iteratedJetDeriv 𝔤 s (bracketFam f g φ) = (s.antidiagonal.map fun p =>
      bracketFam (iteratedJetDeriv 𝔤 p.1 ∘ₗ f) (iteratedJetDeriv 𝔤 p.2 ∘ₗ g) φ).sum
```

lemma bracketFamConv_derivA [source]

If a family consists of the iterated derivatives of its base value, so does its derived bracket against the gauge field: the Leibniz convolution is the iterated derivative of the bracket of the base values.

```
lemma bracketFamConv_derivA (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ]
    LocalGaugeFieldAlgebra 𝔤)
    (hF : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤),
      F s φ = iteratedJetDeriv 𝔤 s (F 0 φ))
    (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    bracketFamConv (derivA 𝔤) ρ F s φ
      = iteratedJetDeriv 𝔤 s (bracketFam (ofA 𝔤 ρ) (F 0) φ)
```

lemma covDerivAdjoint_derivA [source]

The covariant derivative of a family of iterated derivatives is again a family of iterated derivatives.

```
lemma covDerivAdjoint_derivA (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ]
    LocalGaugeFieldAlgebra 𝔤)
    (hF : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤),
      F s φ = iteratedJetDeriv 𝔤 s (F 0 φ))
    (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    covDerivAdjoint (derivA 𝔤) F ρ s φ
      = iteratedJetDeriv 𝔤 s (covDerivAdjoint (derivA 𝔤) F ρ 0 φ)
```

lemma fieldStrength_derivA [source]

The derivative symbols of the field strength are the iterated derivatives of the field strength.

```
lemma fieldStrength_derivA (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℝ 𝔤) :
    GaugeAlgebraRealization.fieldStrength (derivA 𝔤) μ ν s φ
      = iteratedJetDeriv 𝔤 s (fieldStrength 𝔤 μ ν φ)
```

lemma iteratedCovDerivAdjoint_fieldStrength_derivA [source]

The derivative symbols of the covariant derivatives of the field strength are the iterated derivatives of their base values.

```
lemma iteratedCovDerivAdjoint_fieldStrength_derivA (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    iteratedCovDerivAdjoint (derivA 𝔤) l (GaugeAlgebraRealization.fieldStrength (derivA 𝔤) μ ν)
        s φ
```

```
    = iteratedJetDeriv 𝔤 s (covDerivFieldStrength 𝔤 l μ ν φ)
```

lemma covDerivFieldStrength_cons [source]

The covariant derivative `∇_ρ F = ∂_ρ F + [A_ρ, F]` peeled off the front of the list: the recursion on elements of the algebra.

```
lemma covDerivFieldStrength_cons (ρ : Fin 1 ⊕ Fin 3) (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    covDerivFieldStrength 𝔤 (ρ :: l) μ ν φ
      = jetDeriv 𝔤 ρ (covDerivFieldStrength 𝔤 l μ ν φ)
        + bracketFam (ofA 𝔤 ρ) (covDerivFieldStrength 𝔤 l μ ν) φ
```

lemma one_tmul_iteratedCovDerivAdjoint_fieldStrength [source]

The complexified covariant tower of the identity realization is the image of the real one.

```
lemma one_tmul_iteratedCovDerivAdjoint_fieldStrength (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    (1 : ℂ) ⊗ₜ[ℝ] iteratedCovDerivAdjoint (derivA 𝔤) l
        (GaugeAlgebraRealization.fieldStrength (derivA 𝔤) μ ν) s φ
      = iteratedCovDerivAdjoint (gaugeField 𝔤) l
        (GaugeAlgebraRealization.fieldStrength (gaugeField 𝔤) μ ν) s φ
```

lemma repJet_iteratedCovDerivAdjoint_fieldStrength [source]

The gauge law of the covariant derivatives of the field strength at every derivative order: a jet acts by the Leibniz convolution of the dual adjoint Taylor coefficients of `U⁻¹` against lower derivative symbols, with no Maurer–Cartan shift.

```
lemma repJet_iteratedCovDerivAdjoint_fieldStrength (U : GJ) (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    repJet jets U (iteratedCovDerivAdjoint (derivA 𝔤) l
        (GaugeAlgebraRealization.fieldStrength (derivA 𝔤) μ ν) s φ)
      = (s.antidiagonal.map fun p => iteratedCovDerivAdjoint (derivA 𝔤) l
          (GaugeAlgebraRealization.fieldStrength (derivA 𝔤) μ ν) p.2
          (jets.adjointDualCoeff U⁻¹ p.1 φ)).sum
```

lemma repJet_covDerivFieldStrength [source]

The gauge law of the covariant derivatives of the field strength: a jet acts through the zeroth dual adjoint Taylor coefficient of `U⁻¹` on the adjoint index alone.

```
lemma repJet_covDerivFieldStrength (U : GJ) (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repJet jets U (covDerivFieldStrength 𝔤 l μ ν φ)
      = covDerivFieldStrength 𝔤 l μ ν (jets.adjointDualCoeff U⁻¹ 0 φ)
```

lemma repJet_covDerivFieldStrength_eval [source]

On the covariant derivatives of the field strength the local gauge action factors through evaluation: a jet `U` acts through the dual base-point adjoint action of the value

`jets.eval U⁻¹` of its inverse, and the derivatives of the jet are not seen. The covariant expressions are gauge covariant, not gauge invariant.

```
lemma repJet_covDerivFieldStrength_eval (U : GJ) (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repJet jets U (covDerivFieldStrength 𝔤 l μ ν φ)
      = covDerivFieldStrength 𝔤 l μ ν ((jets.adjoint (jets.eval U⁻¹)).dualMap φ)
```

lemma repJet_fieldStrength [source]

The field strength transforms in the adjoint.

```
lemma repJet_fieldStrength (U : GJ) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repJet jets U (fieldStrength 𝔤 μ ν φ)
      = fieldStrength 𝔤 μ ν ((jets.adjoint (jets.eval U⁻¹)).dualMap φ)
```

lemma repLorentzGroup_comp_ofA [source]

The Lorentz action on the gauge-field generator, as a linear map.

```
lemma repLorentzGroup_comp_ofA (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) :
    (repLorentzGroup 𝔤 Λ) ∘ₗ ofA 𝔤 μ = ∑ a, L[Λ] a μ • ofA 𝔤 a
```

lemma repLorentzGroup_derivA [source]

The Lorentz law of the derivative symbols, read off the complexified law `repLorentz_gaugeField` along the injective `x ↦ 1 ⊗ₜ x`.

```
lemma repLorentzGroup_derivA (Λ : SL(2,ℂ)) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repLorentzGroup 𝔤 Λ (derivA 𝔤 (List.ofFn l) μ φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ i, L[Λ] (p i) (l i)) •
          ∑ a, L[Λ] a μ • derivA 𝔤 (List.ofFn p) a φ
```

lemma repLorentzGroup_bracketFam_ofA [source]

The Lorentz action passes through the bracket of a gauge-field generator against a family, mixing the covector index of the generator.

```
lemma repLorentzGroup_bracketFam_ofA (Λ : SL(2,ℂ)) (ρ : Fin 1 ⊕ Fin 3)
    (F : Module.Dual ℝ 𝔤 →ₗ[ℝ] LocalGaugeFieldAlgebra 𝔤) (φ : Module.Dual ℝ 𝔤) :
    repLorentzGroup 𝔤 Λ (bracketFam (ofA 𝔤 ρ) F φ)
      = ∑ a, L[Λ] a ρ • bracketFam (ofA 𝔤 a) ((repLorentzGroup 𝔤 Λ) ∘ₗ F) φ
```

lemma repLorentzGroup_fieldStrength [source]

The Lorentz law of the field strength: both covector indices mix by the columns of the Lorentz matrix, the adjoint index is untouched.

```
lemma repLorentzGroup_fieldStrength (Λ : SL(2,ℂ)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repLorentzGroup 𝔤 Λ (fieldStrength 𝔤 μ ν φ)
      = ∑ a, L[Λ] a μ • ∑ b, L[Λ] b ν • fieldStrength 𝔤 a b φ
```

lemma repLorentzGroup_covDerivFieldStrength [source]

The Lorentz law of the covariant derivatives of the field strength: every covariant slot and both covector indices of the field strength mix by the columns of the Lorentz matrix, the adjoint index is untouched.

```
lemma repLorentzGroup_covDerivFieldStrength (Λ : SL(2,ℂ)) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repLorentzGroup 𝔤 Λ (covDerivFieldStrength 𝔤 (List.ofFn l) μ ν φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ i, L[Λ] (p i) (l i)) •
          ∑ a, L[Λ] a μ • ∑ b, L[Λ] b ν • covDerivFieldStrength 𝔤 (List.ofFn p) a b φ
```

2.177 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.GaugeAction

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/LocalGauge/GaugeAction.lean on GitHub.

def adjointTransport [source]

The adjoint transport on the gauge-boson target space at `p` derivatives: the adjoint Taylor coefficient on the gauge-algebra factor, the identity on the spacetime index.

```
noncomputable def adjointTransport (U : GJ) (p : Multiset (Fin 1 ⊕ Fin 3)) :
    (GaugeBoson 𝔤) →ₗ[ℝ] (GaugeBoson 𝔤)
```

lemma adjointTransport_mk_tmul [source]

```
lemma adjointTransport_mk_tmul (U : GJ) (p : Multiset (Fin 1 ⊕ Fin 3))
    (v : Lorentz.CoVector) (a : 𝔤) :
    adjointTransport jets U p ⟨v ⊗ₜ[ℝ] a⟩ = ⟨v ⊗ₜ[ℝ] jets.adjointCoeff U p a⟩
```

lemma adjointTransport_one [source]

The adjoint transport at the identity: only the base point survives.

```
lemma adjointTransport_one (p : Multiset (Fin 1 ⊕ Fin 3)) :
    adjointTransport jets 1 p = if p = 0 then LinearMap.id else 0
```

lemma adjointTransport_mul [source]

The adjoint transport of a product: the antidiagonal convolution of transports.

```
lemma adjointTransport_mul (U V : GJ) (p : Multiset (Fin 1 ⊕ Fin 3)) :
    adjointTransport jets (U * V) p
      = (p.antidiagonal.map fun r =>
          adjointTransport jets U r.1 ∘ₗ adjointTransport jets V r.2).sum
```

lemma dualMap_adjointTransport_componentDual [source]

The dual transport carries a component covector to the component covector of the transported adjoint index: the spacetime slot is untouched.

```
lemma dualMap_adjointTransport_componentDual (U : GJ)
    (p : Multiset (Fin 1 ⊕ Fin 3)) (ω : Module.Dual ℝ Lorentz.CoVector)
    (φ : Module.Dual ℝ 𝔤) :
    (adjointTransport jets U p).dualMap ((componentDual 𝔤) ω φ)
```

```
    = (componentDual 𝔤) ω (φ ∘ₗ jets.adjointCoeff U p)
```

def transportFun [source]

The value of the transport on the derivative symbol at `s`: the all-orders Leibniz convolution of the dual adjoint transports against lower derivative symbols.

```
noncomputable def transportFun (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℝ (GaugeBoson 𝔤) →ₗ[ℝ] (GaugeBoson.JetComponentSpace 𝔤)
```

def transport [source]

The linear part of the gauge action on the jet component space: on a component function `∂_s A^ψ` it is the all-orders Leibniz convolution of the Taylor coefficients of the adjoint action of `U` against the lower component functions.

```
noncomputable def transport (U : GJ) :
    (GaugeBoson.JetComponentSpace 𝔤) →ₗ[ℝ] (GaugeBoson.JetComponentSpace 𝔤)
```

lemma transport_basis_tmul [source]

```
lemma transport_basis_tmul (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3))
    (ψ : Module.Dual ℝ (GaugeBoson 𝔤)) :
    transport jets U (SpaceTimeDerivAlgebraℝ.basisMultiset s ⊗ₜ[ℝ] ψ)
      = (s.antidiagonal.map fun p =>
          SpaceTimeDerivAlgebraℝ.basisMultiset p.2 ⊗ₜ[ℝ]
            (GaugeBoson.adjointTransport jets U p.1).dualMap ψ).sum
```

lemma ext_of_basis [source]

Two maps out of the jet component space agree if they agree on the components `∂_s A^ψ` with `s` a derivative multiset and `ψ` an arbitrary covector.

```
lemma _root_.GaugeBoson.JetComponentSpace.ext_of_basis
    {M : Type*} [AddCommMonoid M] [Module ℝ M]
    {F G : (GaugeBoson.JetComponentSpace 𝔤) →ₗ[ℝ] M}
    (h : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (ψ : Module.Dual ℝ (GaugeBoson 𝔤)),
      F (SpaceTimeDerivAlgebraℝ.basisMultiset s ⊗ₜ[ℝ] ψ)
        = G (SpaceTimeDerivAlgebraℝ.basisMultiset s ⊗ₜ[ℝ] ψ)) : F = G
```

lemma transport_one [source]

The transport of the identity is the identity.

```
lemma transport_one : transport jets (1 : GJ) = LinearMap.id
```

lemma transport_mul [source]

The transport is an anti-homomorphism: the transport of a product is the reverse composite. Composed with the inverse, it becomes the linear part of the gauge representation.

```
lemma transport_mul (U V : GJ) :
    transport jets (U * V) = transport jets V ∘ₗ transport jets U
```

def mcBosonCoeff [source]

The Taylor coefficient of the Maurer–Cartan form of `U` at the derivative multiset `s`, packaged as a gauge boson: the spacetime index runs over the coordinate directions, the adjoint index over the base-point Taylor coefficients of the Maurer–Cartan form.

```
noncomputable def mcBosonCoeff (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (GaugeBoson 𝔤)
```

lemma mcBosonCoeff_one [source]

```
@[simp]
lemma mcBosonCoeff_one (s : Multiset (Fin 1 ⊕ Fin 3)) : mcBosonCoeff jets 1 s = 0
```

lemma mcBosonCoeff_mul [source]

The Maurer–Cartan Taylor coefficients of a product: the cocycle identity, with the adjoint transport convoluted in by the Taylor–Leibniz theorem.

```
lemma mcBosonCoeff_mul (U V : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    mcBosonCoeff jets (U * V) s
      = mcBosonCoeff jets U s
        + (s.antidiagonal.map fun p =>
            GaugeBoson.adjointTransport jets U p.1 (mcBosonCoeff jets V p.2)).sum
```

def mcShift [source]

The Maurer–Cartan shift: the linear functional on the component space pairing a component `∂_s A^ψ` with the Taylor coefficient of the Maurer–Cartan form of `U`. It is the constant part of the affine gauge action.

```
noncomputable def mcShift (U : GJ) : (GaugeBoson.JetComponentSpace 𝔤) →ₗ[ℝ] ℝ
```

lemma mcShift_basis_tmul [source]

```
lemma mcShift_basis_tmul (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3))
    (ψ : Module.Dual ℝ (GaugeBoson 𝔤)) :
    mcShift jets U (SpaceTimeDerivAlgebraℝ.basisMultiset s ⊗ₜ[ℝ] ψ)
      = ψ (mcBosonCoeff jets U s)
```

lemma mcShift_one [source]

```
@[simp]
lemma mcShift_one : mcShift jets (1 : GJ) = 0
```

lemma mcShift_mul [source]

The cocycle identity for the Maurer–Cartan shift.

```
lemma mcShift_mul (U V : GJ) :
    mcShift jets (U * V) = mcShift jets V ∘ₗ transport jets U + mcShift jets U
```

def gaugeGen [source]

The affine action of a jet of gauge transformations on the generators of the jet algebra: the transported component plus the Maurer–Cartan shift, both of `U⁻¹` — the contragredient convention for an action on component functions.

```
noncomputable def gaugeGen (U : GJ) :
    (GaugeBoson.JetComponentSpace 𝔤) →ₗ[ℝ] (LocalGaugeFieldAlgebra 𝔤)
```

lemma gaugeGen_apply [source]

```
lemma gaugeGen_apply (U : GJ) (x : (GaugeBoson.JetComponentSpace 𝔤)) :
    gaugeGen jets U x = SymmetricAlgebra.ι ℝ _ (transport jets U⁻¹ x)
      + algebraMap ℝ (LocalGaugeFieldAlgebra 𝔤) (mcShift jets U⁻¹ x)
```

def repJet [source]

The action of the jet gauge group on the gauge-boson jet algebra: the substitution homomorphism determined by the affine action on the generators, `∂_s A^ψ` going to its transported convolution plus the Maurer–Cartan shift of `U⁻¹`.

```
noncomputable def repJet : Representation ℝ GJ (LocalGaugeFieldAlgebra 𝔤) where
```

def repJetAlgHom [source]

The action of `U` as an algebra homomorphism: a jet of gauge transformations acts on a Lagrangian term factor by factor.

```
noncomputable def repJetAlgHom (U : GJ) :
    (LocalGaugeFieldAlgebra 𝔤) →ₐ[ℝ] (LocalGaugeFieldAlgebra 𝔤)
```

lemma repJet_ι [source]

```
@[simp]
lemma repJet_ι (U : GJ) (x : (GaugeBoson.JetComponentSpace 𝔤)) :
    repJet jets U (SymmetricAlgebra.ι ℝ _ x)
      = SymmetricAlgebra.ι ℝ _ (transport jets U⁻¹ x)
        + algebraMap ℝ (LocalGaugeFieldAlgebra 𝔤) (mcShift jets U⁻¹ x)
```

lemma repJet_apply_one [source]

```
@[simp]
lemma repJet_apply_one (U : GJ) :
    repJet jets U (1 : (LocalGaugeFieldAlgebra 𝔤)) = 1
```

lemma repJet_apply_mul [source]

```
lemma repJet_apply_mul (U : GJ) (x y : (LocalGaugeFieldAlgebra 𝔤)) :
    repJet jets U (x * y) = repJet jets U x * repJet jets U y
```

lemma repJet_algebraMap [source]

```
@[simp]
lemma repJet_algebraMap (U : GJ) (r : ℝ) :
    repJet jets U (algebraMap ℝ (LocalGaugeFieldAlgebra 𝔤) r)
      = algebraMap ℝ (LocalGaugeFieldAlgebra 𝔤) r
```

lemma componentDual_dualBasis_mcBosonCoeff [source]

The component covector at μ picks the μ-th Maurer–Cartan Taylor coefficient out of the shift.

```
lemma componentDual_dualBasis_mcBosonCoeff (W : GJ)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    (GaugeBoson.componentDual 𝔤) (Lorentz.CoVector.basis.dualBasis μ) φ (mcBosonCoeff jets W s)
      = φ (jets.evalLie (jets.iteratedDeriv s
          (jets.maurerCartan W μ)))
```

lemma repJet_iteratedJetDeriv_ofA [source]

The transformation law of the derivative generators, in the form used by `GaugeAlgebraRealization`: a jet of gauge transformations acts on `∂_s A_μ^φ` by the all-orders Leibniz convolution of the adjoint Taylor coefficients of `U⁻¹` against lower generators, plus the Taylor coefficient of the Maurer–Cartan form of `U⁻¹`.

```
lemma repJet_iteratedJetDeriv_ofA (U : GJ)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repJet jets U ((iteratedJetDeriv 𝔤) s ((ofA 𝔤) μ φ))
      = (s.antidiagonal.map fun p =>
          (iteratedJetDeriv 𝔤) p.2 ((ofA 𝔤) μ (jets.adjointDualCoeff U⁻¹ p.1 φ))).sum
        + algebraMap ℝ (LocalGaugeFieldAlgebra 𝔤)
            (φ (jets.evalLie (jets.iteratedDeriv s
              (jets.maurerCartan U⁻¹ μ))))
```

def complexRepJet [source]

The action of the jet gauge group on the complexified gauge-boson jet algebra, by base change.

```
noncomputable def complexRepJet :
    Representation ℂ GJ (ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝔤)) where
```

lemma complexRepJet_tmul [source]

```
@[simp]
lemma complexRepJet_tmul (U : GJ) (z : ℂ) (x : (LocalGaugeFieldAlgebra 𝔤)) :
    complexRepJet jets U (z ⊗ₜ[ℝ] x) = z ⊗ₜ[ℝ] repJet jets U x
```

def complexRepJetAlgHom [source]

The action of a jet on the complexified gauge-boson jet algebra, as an algebra endomorphism: the base change of `repJetAlgHom`.

```
noncomputable def complexRepJetAlgHom (U : GJ) :
    ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝔤) →ₐ[ℂ] ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝔤)
```

lemma complexRepJet_apply [source]

```
lemma complexRepJet_apply (U : GJ) (x : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝔤)) :
    complexRepJet jets U x = complexRepJetAlgHom jets U x
```

lemma complexRepJet_apply_mul [source]

```
lemma complexRepJet_apply_mul (U : GJ)
    (x y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝔤)) :
    complexRepJet jets U (x * y)
      = complexRepJet jets U x * complexRepJet jets U y
```

lemma iteratedD_complexJetDeriv_one_tmul [source]

The iterated complexified derivative of a real element is the complexification of the iterated real derivative.

```
lemma iteratedD_complexJetDeriv_one_tmul (s : Multiset (Fin 1 ⊕ Fin 3))
    (x : (LocalGaugeFieldAlgebra 𝔤)) :
    Lorentz.iteratedD (complexJetDeriv 𝔤) complexJetDeriv_comm s ((1 : ℂ) ⊗ₜ[ℝ] x)
      = (1 : ℂ) ⊗ₜ[ℝ] (iteratedJetDeriv 𝔤) s x
```

lemma one_tmul_algebraMap [source]

A real scalar in the complexified jet algebra is the corresponding complex scalar.

```
lemma one_tmul_algebraMap (r : ℝ) :
    (1 : ℂ) ⊗ₜ[ℝ] (algebraMap ℝ (LocalGaugeFieldAlgebra 𝔤) r)
      = algebraMap ℂ (ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝔤)) ((r : ℝ) : ℂ)
```

lemma complexRepJet_iteratedD_one_tmul_ofA [source]

The transformation law of the derivative generators on the complexification: the form consumed by the laws of a `GaugeAlgebraRealization`.

```
lemma complexRepJet_iteratedD_one_tmul_ofA (U : GJ)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    complexRepJet jets U (Lorentz.iteratedD (complexJetDeriv 𝔤) complexJetDeriv_comm s
        ((1 : ℂ) ⊗ₜ[ℝ] (ofA 𝔤) μ φ))
      = (s.antidiagonal.map fun p =>
          Lorentz.iteratedD (complexJetDeriv 𝔤) complexJetDeriv_comm p.2
            ((1 : ℂ) ⊗ₜ[ℝ] (ofA 𝔤) μ (jets.adjointDualCoeff U⁻¹ p.1 φ))).sum
        + algebraMap ℂ (ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝔤))
            (((φ (jets.evalLie (jets.iteratedDeriv s
              (jets.maurerCartan U⁻¹ μ))) : ℝ)) : ℂ)
```

2.178 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.GaugeField

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/LocalGauge/GaugeField.lean on GitHub.

def gaugeField [source]

The gauge-field derivative symbols of the complexified gauge-boson jet algebra, as a family over the derivative multiset, the spacetime index and the dual of the gauge algebra — the form consumed by the abstract covariance machinery.

```
noncomputable def gaugeField (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝔤)
```

lemma gaugeField_apply [source]

```
@[simp]
lemma gaugeField_apply (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    (gaugeField 𝔤) s μ φ = Lorentz.iteratedD (complexJetDeriv 𝔤) complexJetDeriv_comm s
      ((1 : ℂ) ⊗ₜ[ℝ] (ofA 𝔤) μ φ)
```

lemma repLorentz_gaugeField [source]

The Lorentz law of the jet algebra: the symbol `∂_s A_μ^φ` carries one covector index, and each derivative slot transforms as a covector, by `IsLorentzDeriv`.

```
lemma repLorentz_gaugeField (Λ : SL(2,ℂ)) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    (complexRepLorentzGroup 𝔤) Λ (gaugeField 𝔤 (List.ofFn l) μ φ) =
      ∑ (p : Fin n → (Fin 1 ⊕ Fin 3)),
        (∏ (i : Fin n), (((SL2C.toLorentzGroup Λ).1 (p i) (l i) : ℝ) : ℂ)) •
      ∑ a, (((Lorentz.SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
        gaugeField 𝔤 (List.ofFn p) a φ
```

lemma repJet_gaugeField [source]

The gauge law of the jet algebra: a jet `U` acts on `∂_s A_μ^φ` by the Leibniz convolution of the dual adjoint Taylor coefficients of `U⁻¹` against lower symbols, plus the base-point value of the `s`-th derivative of the Maurer–Cartan form of `U⁻¹`.

```
lemma repJet_gaugeField (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    complexRepJet jets U (gaugeField 𝔤 s μ φ) =
      (s.antidiagonal.map fun p => gaugeField 𝔤 p.2 μ (jets.adjointDualCoeff U⁻¹ p.1 φ)).sum
      + algebraMap ℂ (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤)
          (φ (jets.evalLie (jets.iteratedDeriv s (jets.maurerCartan U⁻¹ μ))))
```

2.179 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.JetDeriv

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/LocalGauge/JetDeriv.lean on GitHub.

def JetComponentSpace.jetDeriv [source]

The derivative of components in the jet component space, in the direction `ν`: the shift `∂_s A_μ^φ ↦ ∂_{s + {ν}} A_μ^φ` of the derivative label, i.e. right multiplication by the degree-one symbol `∂_ν` on the `SpaceTimeDerivAlgebraℝ` factor.

```
noncomputable def JetComponentSpace.jetDeriv (ν : Fin 1 ⊕ Fin 3) :
    (JetComponentSpace 𝔤) →ₗ[ℝ] (JetComponentSpace 𝔤)
```

lemma jetDeriv_tmul [source]

```
@[simp]
lemma JetComponentSpace.jetDeriv_tmul (ν : Fin 1 ⊕ Fin 3) (a : SpaceTimeDerivAlgebraℝ)
```

```
    (φ : Module.Dual ℝ (GaugeBoson 𝔤)) :
    (JetComponentSpace.jetDeriv 𝔤) ν (a ⊗ₜ[ℝ] φ)
      = (a * SpaceTimeDerivAlgebraℝ.basisMultiset
          ({ν} : Multiset (Fin 1 ⊕ Fin 3))) ⊗ₜ[ℝ] φ
```

lemma JetComponentSpace.jetDeriv_comm [source]

Total derivatives commute on the component space: the derivative labels live in a symmetric algebra.

```
lemma JetComponentSpace.jetDeriv_comm (μ ν : Fin 1 ⊕ Fin 3) :
    ((JetComponentSpace.jetDeriv 𝔤) μ).comp ((JetComponentSpace.jetDeriv 𝔤) ν)
      = ((JetComponentSpace.jetDeriv 𝔤) ν).comp ((JetComponentSpace.jetDeriv 𝔤) μ)
```

def LocalGaugeFieldAlgebra.jetDeriv [source]

The formal total spacetime derivative on the gauge-boson jet algebra in the direction `ν`: the derivation extending the shift `∂_s A_μ^φ ↦ ∂_{s + {ν}} A_μ^φ` of the component functions.

```
noncomputable def jetDeriv (ν : Fin 1 ⊕ Fin 3) :
    (LocalGaugeFieldAlgebra 𝔤) →ₗ[ℝ] (LocalGaugeFieldAlgebra 𝔤)
```

lemma jetDeriv_ι [source]

```
@[simp]
lemma jetDeriv_ι (ν : Fin 1 ⊕ Fin 3) (x : (GaugeBoson.JetComponentSpace 𝔤)) :
    (jetDeriv 𝔤) ν (SymmetricAlgebra.ι ℝ _ x)
      = SymmetricAlgebra.ι ℝ _ ((GaugeBoson.JetComponentSpace.jetDeriv 𝔤) ν x)
```

lemma jetDeriv_one [source]

```
@[simp]
lemma jetDeriv_one (ν : Fin 1 ⊕ Fin 3) : (jetDeriv 𝔤) ν (1 : (LocalGaugeFieldAlgebra 𝔤)) = 0
```

lemma jetDeriv_algebraMap [source]

```
@[simp]
lemma jetDeriv_algebraMap (ν : Fin 1 ⊕ Fin 3) (r : ℝ) :
    (jetDeriv 𝔤) ν (algebraMap ℝ (LocalGaugeFieldAlgebra 𝔤) r) = 0
```

lemma jetDeriv_mul [source]

The total derivative is a derivation: the Leibniz rule.

```
lemma jetDeriv_mul (ν : Fin 1 ⊕ Fin 3) (x y : (LocalGaugeFieldAlgebra 𝔤)) :
    (jetDeriv 𝔤) ν (x * y) = (jetDeriv 𝔤) ν x * y + x * (jetDeriv 𝔤) ν y
```

lemma jetDeriv_comm_apply [source]

Mixed partials agree on the jet algebra.

```
lemma jetDeriv_comm_apply (μ ν : Fin 1 ⊕ Fin 3) (x : (LocalGaugeFieldAlgebra 𝔤)) :
    (jetDeriv 𝔤) μ ((jetDeriv 𝔤) ν x) = (jetDeriv 𝔤) ν ((jetDeriv 𝔤) μ x)
```

lemma LocalGaugeFieldAlgebra.jetDeriv_comm [source]

```
lemma jetDeriv_comm (μ ν : Fin 1 ⊕ Fin 3) :
    ((jetDeriv 𝔤) μ).comp ((jetDeriv 𝔤) ν) = ((jetDeriv 𝔤) ν).comp ((jetDeriv 𝔤) μ)
```

instance RightCommutative (fun (D : (LocalGaugeFieldAlgebra 𝔤) →ₗ[ℝ] (LocalGaug… [source]

```
instance : RightCommutative
    (fun (D : (LocalGaugeFieldAlgebra 𝔤) →ₗ[ℝ] (LocalGaugeFieldAlgebra 𝔤)) (μ : Fin 1 ⊕ Fin 3)
    =>
      D.comp ((jetDeriv 𝔤) μ)) where
```

def iteratedJetDeriv [source]

The iterated total derivative `∂_s` along a multiset `s` of directions, well defined because the directional derivatives commute.

```
noncomputable def iteratedJetDeriv (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (LocalGaugeFieldAlgebra 𝔤) →ₗ[ℝ] (LocalGaugeFieldAlgebra 𝔤)
```

lemma iteratedJetDeriv_zero [source]

```
@[simp]
lemma iteratedJetDeriv_zero :
    (iteratedJetDeriv 𝔤) (0 : Multiset (Fin 1 ⊕ Fin 3)) = LinearMap.id
```

lemma foldl_comp_eq [source]

Any initial map factors out of the fold defining the iterated derivative.

```
lemma foldl_comp_eq (s : Multiset (Fin 1 ⊕ Fin 3)) :
    ∀ D : (LocalGaugeFieldAlgebra 𝔤) →ₗ[ℝ] (LocalGaugeFieldAlgebra 𝔤),
      s.foldl (fun D μ => D.comp ((jetDeriv 𝔤) μ)) D = D ∘ₗ (iteratedJetDeriv 𝔤) s
```

lemma iteratedJetDeriv_cons [source]

```
lemma iteratedJetDeriv_cons (μ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (iteratedJetDeriv 𝔤) (μ ::ₘ s) = (jetDeriv 𝔤) μ ∘ₗ (iteratedJetDeriv 𝔤) s
```

lemma iteratedJetDeriv_singleton [source]

```
@[simp]
lemma iteratedJetDeriv_singleton (μ : Fin 1 ⊕ Fin 3) :
    (iteratedJetDeriv 𝔤) ({μ} : Multiset (Fin 1 ⊕ Fin 3)) = (jetDeriv 𝔤) μ
```

lemma iteratedJetDeriv_add [source]

Differentiating along `s + t` is differentiating along `t` and then along `s`.

```
lemma iteratedJetDeriv_add (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    (iteratedJetDeriv 𝔤) (s + t) = (iteratedJetDeriv 𝔤) t ∘ₗ (iteratedJetDeriv 𝔤) s
```

lemma iteratedJetDeriv_cons' [source]

The companion of `iteratedJetDeriv_cons`, peeling the new derivative on the inside: the derivatives commute, so the extra direction may equally be applied first.

```
lemma iteratedJetDeriv_cons' (μ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (iteratedJetDeriv 𝔤) (μ ::ₘ s) = (iteratedJetDeriv 𝔤) s ∘ₗ (jetDeriv 𝔤) μ
```

lemma iteratedJetDeriv_ι [source]

On a component function the iterated derivative writes the derivative symbol `∂_s` into the derivative label.

```
lemma iteratedJetDeriv_ι (s : Multiset (Fin 1 ⊕ Fin 3)) (a : SpaceTimeDerivAlgebraℝ)
    (φ : Module.Dual ℝ (GaugeBoson 𝔤)) :
    (iteratedJetDeriv 𝔤) s (SymmetricAlgebra.ι ℝ _ (a ⊗ₜ[ℝ] φ))
      = SymmetricAlgebra.ι ℝ _
          ((a * SpaceTimeDerivAlgebraℝ.basisMultiset s) ⊗ₜ[ℝ] φ)
```

lemma iteratedJetDeriv_ofA [source]

***The derivative generator `∂_s A_μ^φ`**: the iterated derivative of the gauge-field component function.*

```
lemma iteratedJetDeriv_ofA (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    (iteratedJetDeriv 𝔤) s ((ofA 𝔤) μ φ)
      = SymmetricAlgebra.ι ℝ _
          (SpaceTimeDerivAlgebraℝ.basisMultiset s ⊗ₜ[ℝ]
            (GaugeBoson.componentDual 𝔤) (Lorentz.CoVector.basis.dualBasis μ) φ)
```

lemma dual_eq_sum_componentDual [source]

Every covector on the gauge-boson target space decomposes along the Lorentz coordinate directions into component covectors: the spacetime index is expanded in the coordinate basis, while the adjoint index stays abstract.

```
lemma _root_.GaugeBoson.dual_eq_sum_componentDual
    (ψ : Module.Dual ℝ (GaugeBoson 𝔤)) :
    ψ = ∑ μ, (GaugeBoson.componentDual 𝔤) (Lorentz.CoVector.basis.dualBasis μ)
      (ψ ∘ₗ (GaugeBoson.valLinEquiv 𝔤).symm.toLinearMap ∘ₗ
        TensorProduct.mk ℝ Lorentz.CoVector 𝔤 (Lorentz.CoVector.basis μ))
```

lemma adjoin_iteratedJetDeriv_eq_top [source]

***The jet algebra is generated by the gauge fields and their derivatives.** Every element is a polynomial in the derivative generators `∂_s A_μ^φ` — nothing else is available to write down for the gauge sector of a Lagrangian.*

```
lemma adjoin_iteratedJetDeriv_eq_top :
    Algebra.adjoin ℝ
```

```
    (⋃ s : Multiset (Fin 1 ⊕ Fin 3), ⋃ μ : Fin 1 ⊕ Fin 3,
      Set.range (fun φ : Module.Dual ℝ 𝔤 => (iteratedJetDeriv 𝔤) s ((ofA 𝔤) μ φ)))
    = (⊤ : Subalgebra ℝ (LocalGaugeFieldAlgebra 𝔤))
```

def complexJetDeriv [source]

The formal total derivative on the complexified gauge-boson jet algebra, by base change. This is the derivative the ambient Lagrangian theory uses.

```
noncomputable def complexJetDeriv (ν : Fin 1 ⊕ Fin 3) :
    ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝔤) →ₗ[ℂ] ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝔤)
```

lemma complexJetDeriv_tmul [source]

```
@[simp]
lemma complexJetDeriv_tmul (ν : Fin 1 ⊕ Fin 3) (z : ℂ) (x : (LocalGaugeFieldAlgebra 𝔤)) :
    (complexJetDeriv 𝔤) ν (z ⊗ₜ[ℝ] x) = z ⊗ₜ[ℝ] (jetDeriv 𝔤) ν x
```

lemma complexJetDeriv_mul [source]

The Leibniz rule on the complexification.

```
lemma complexJetDeriv_mul (ν : Fin 1 ⊕ Fin 3) (x y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝔤)) :
    (complexJetDeriv 𝔤) ν (x * y)
      = (complexJetDeriv 𝔤) ν x * y + x * (complexJetDeriv 𝔤) ν y
```

lemma complexJetDeriv_comm [source]

The complexified total derivatives commute.

```
lemma complexJetDeriv_comm (μ ν : Fin 1 ⊕ Fin 3) :
    ((complexJetDeriv 𝔤) μ).comp ((complexJetDeriv 𝔤) ν)
      = ((complexJetDeriv 𝔤) ν).comp ((complexJetDeriv 𝔤) μ)
```

2.180 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.LorentzAction

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/LocalGauge/LorentzAction.lean on GitHub.

def JetComponentSpace.repLorentzGroup [source]

The Lorentz action on the jet component space of the gauge bosons: the derivative label transforms in `SpaceTimeDerivAlgebra`ℝ, the target index contragrediently.

```
noncomputable def JetComponentSpace.repLorentzGroup :
    Representation ℝ SL(2,ℂ) (JetComponentSpace 𝔤)
```

lemma repLorentzGroup_basis_singleton [source]

The Lorentz action on the singleton derivative symbol: the derivative slot transforms by the columns of the Lorentz matrix.

```
lemma _root_.SpaceTimeDerivAlgebraℝ.repLorentzGroup_basis_singleton
    (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) :
    SpaceTimeDerivAlgebraℝ.repLorentzGroup Λ
```

```
    (SpaceTimeDerivAlgebraℝ.basisMultiset ({μ} : Multiset (Fin 1 ⊕ Fin 3))) =
  ∑ a, ((Lorentz.SL2C.toLorentzGroup Λ).1 a μ) •
    SpaceTimeDerivAlgebraℝ.basisMultiset ({a} : Multiset (Fin 1 ⊕ Fin 3))
```

lemma JetComponentSpace.repLorentzGroup_jetDeriv [source]

***The derivative shift is a Lorentz vector on the component space**: appending ∂_μ and then acting is acting and then appending the transformed ∂_μ.*

```
lemma JetComponentSpace.repLorentzGroup_jetDeriv (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3)
    (v : (JetComponentSpace 𝓖)) :
    (JetComponentSpace.repLorentzGroup 𝓖) Λ ((JetComponentSpace.jetDeriv 𝓖) μ v) =
      ∑ a, ((Lorentz.SL2C.toLorentzGroup Λ).1 a μ) •
        (JetComponentSpace.jetDeriv 𝓖) a ((JetComponentSpace.repLorentzGroup 𝓖) Λ v)
```

def LocalGaugeFieldAlgebra.repLorentzGroup [source]

***The Lorentz action on the gauge-boson jet algebra**: the symmetric-algebra functor applied to the Lorentz action on the jet component space.*

```
noncomputable def repLorentzGroup : Representation ℝ SL(2,ℂ) (LocalGaugeFieldAlgebra 𝓖) where
```

lemma repLorentzGroup_apply [source]

```
lemma repLorentzGroup_apply (Λ : SL(2,ℂ)) (x : (LocalGaugeFieldAlgebra 𝓖)) :
    (repLorentzGroup 𝓖) Λ x =
      SymmetricAlgebra.map ((GaugeBoson.JetComponentSpace.repLorentzGroup 𝓖) Λ) x
```

lemma repLorentzGroup_apply_one [source]

```
@[simp]
lemma repLorentzGroup_apply_one (Λ : SL(2,ℂ)) :
    (repLorentzGroup 𝓖) Λ (1 : (LocalGaugeFieldAlgebra 𝓖)) = 1
```

lemma repLorentzGroup_apply_mul [source]

```
lemma repLorentzGroup_apply_mul (Λ : SL(2,ℂ)) (x y : (LocalGaugeFieldAlgebra 𝓖)) :
    (repLorentzGroup 𝓖) Λ (x * y) = (repLorentzGroup 𝓖) Λ x * (repLorentzGroup 𝓖) Λ y
```

lemma repLorentzGroup_ι [source]

```
@[simp]
lemma repLorentzGroup_ι (Λ : SL(2,ℂ)) (v : (GaugeBoson.JetComponentSpace 𝓖)) :
    (repLorentzGroup 𝓖) Λ (SymmetricAlgebra.ι ℝ _ v) =
      SymmetricAlgebra.ι ℝ _ ((GaugeBoson.JetComponentSpace.repLorentzGroup 𝓖) Λ v)
```

lemma LocalGaugeFieldAlgebra.repLorentzGroup_jetDeriv [source]

The total derivative on the gauge-boson jet algebra is a Lorentz vector.

```
lemma repLorentzGroup_jetDeriv (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) (x : (LocalGaugeFieldAlgebra
    𝓖)) :
    (repLorentzGroup 𝓖) Λ ((jetDeriv 𝓖) μ x) =
```

```
    ∑ a, ((Lorentz.SL2C.toLorentzGroup Λ).1 a μ) •
      (jetDeriv 𝓰) a ((repLorentzGroup 𝓰) Λ x)
```

def complexRepLorentzGroup [source]

The Lorentz action on the complexified gauge-boson jet algebra, by base change.

```
noncomputable def complexRepLorentzGroup :
    Representation ℂ SL(2,ℂ) (ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝓰)) where
```

lemma complexRepLorentzGroup_tmul [source]

```
@[simp]
lemma complexRepLorentzGroup_tmul (Λ : SL(2,ℂ)) (z : ℂ) (x : (LocalGaugeFieldAlgebra 𝓰)) :
    (complexRepLorentzGroup 𝓰) Λ (z ⊗ₜ[ℝ] x) = z ⊗ₜ[ℝ] (repLorentzGroup 𝓰) Λ x
```

def complexRepLorentzGroupAlgHom [source]

The action of a Lorentz transformation on the complexified gauge-boson jet algebra, as an algebra endomorphism: the base change of the symmetric-algebra functor applied to the action on the jet component space.

```
noncomputable def complexRepLorentzGroupAlgHom (Λ : SL(2,ℂ)) :
    ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝓰) →ₐ[ℂ] ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝓰)
```

lemma complexRepLorentzGroup_apply [source]

```
lemma complexRepLorentzGroup_apply (Λ : SL(2,ℂ)) (x : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝓰)) :
    (complexRepLorentzGroup 𝓰) Λ x = complexRepLorentzGroupAlgHom 𝓰 Λ x
```

lemma complexRepLorentzGroup_apply_mul [source]

```
lemma complexRepLorentzGroup_apply_mul (Λ : SL(2,ℂ)) (x y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝓰))
    :
    (complexRepLorentzGroup 𝓰) Λ (x * y)
      = (complexRepLorentzGroup 𝓰) Λ x * (complexRepLorentzGroup 𝓰) Λ y
```

lemma complexRepLorentzGroup_jetDeriv [source]

The complexified total derivative is a Lorentz vector.

```
lemma complexRepLorentzGroup_jetDeriv (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3)
    (x : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra 𝓰)) :
    (complexRepLorentzGroup 𝓰) Λ ((complexJetDeriv 𝓰) μ x) =
      ∑ a, (((Lorentz.SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
        (complexJetDeriv 𝓰) a ((complexRepLorentzGroup 𝓰) Λ x)
```

instance instIsLorentzDeriv [source]

The complexified total derivatives form a Lorentz derivative, giving access to the boost-weight machinery.

```
instance instIsLorentzDeriv :
    Lorentz.IsLorentzDeriv (complexRepLorentzGroup 𝔤) (complexJetDeriv 𝔤) where
```

lemma repLorentzGroup_dual_componentDual [source]

The contragredient Lorentz action passes through a component covector to its spacetime slot: the adjoint index is Lorentz-inert.

```
lemma _root_.GaugeBoson.repLorentzGroup_dual_componentDual (Λ : SL(2,ℂ))
    (ω : Module.Dual ℝ Lorentz.CoVector) (φ : Module.Dual ℝ 𝔤) :
    (GaugeBoson.repLorentzGroup 𝔤).dual Λ ((GaugeBoson.componentDual 𝔤) ω φ)
      = (GaugeBoson.componentDual 𝔤) (Lorentz.CoVector.sl2Rep.dual Λ ω) φ
```

lemma repLorentzGroup_ofA [source]

***The gauge field is a Lorentz covector**: the generator `A_μ^φ` mixes into the `A_a^φ` by the columns of the Lorentz matrix, with the adjoint index untouched.*

```
lemma repLorentzGroup_ofA (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    (repLorentzGroup 𝔤) Λ ((ofA 𝔤) μ φ)
      = ∑ a, ((Lorentz.SL2C.toLorentzGroup Λ).1 a μ) • (ofA 𝔤) a φ
```

lemma complexRepLorentzGroup_one_tmul_ofA [source]

The Lorentz law of the gauge-field generators on the complexification.

```
lemma complexRepLorentzGroup_one_tmul_ofA (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    (complexRepLorentzGroup 𝔤) Λ ((1 : ℂ) ⊗ₜ[ℝ] (ofA 𝔤) μ φ)
      = ∑ a, (((Lorentz.SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
          ((1 : ℂ) ⊗ₜ[ℝ] (ofA 𝔤) a φ)
```

2.181 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.MassDim

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/LocalGauge/MassDim.lean on GitHub.

def JetComponentSpace.massWeightScale [source]

The mass-weight scaling on the jet component space of the gauge bosons: the generator `∂_s A_μ^φ` is scaled by `c ^ (2 + 2 |s|)`, through the derivative-degree scaling `SpaceTimeDerivAlgebraℝ.gradeScale` on the derivative label.

```
noncomputable def JetComponentSpace.massWeightScale (c : ℝ) :
    (JetComponentSpace 𝔤) →ₗ[ℝ] (JetComponentSpace 𝔤)
```

lemma massWeightScale_tmul [source]

```
lemma JetComponentSpace.massWeightScale_tmul (c : ℝ) (a : SpaceTimeDerivAlgebraℝ)
    (φ : Module.Dual ℝ (GaugeBoson 𝔤)) :
    (JetComponentSpace.massWeightScale 𝔤) c (a ⊗ₜ[ℝ] φ)
      = c ^ 2 • (SpaceTimeDerivAlgebraℝ.gradeScale (c ^ 2) a ⊗ₜ[ℝ] φ)
```

lemma JetComponentSpace.massWeightScale_jetDeriv [source]

***The derivative shift carries mass weight two** on the component space.*

```
lemma JetComponentSpace.massWeightScale_jetDeriv (c : ℝ) (μ : Fin 1 ⊕ Fin 3)
    (v : (JetComponentSpace 𝓖)) :
    (JetComponentSpace.massWeightScale 𝓖) c ((JetComponentSpace.jetDeriv 𝓖) μ v)
      = c ^ 2 • (JetComponentSpace.jetDeriv 𝓖) μ ((JetComponentSpace.massWeightScale 𝓖) c v)
```

def LocalGaugeFieldAlgebra.massWeightScale [source]

***The mass-weight scaling on the gauge-boson jet algebra**: the algebra endomorphism scaling the generator `∂_s A_μ^φ` by `c ^ (2 + 2 |s|)`, the functorial lift of the scaling on the jet component space.*

```
noncomputable def massWeightScale (c : ℝ) :
    (LocalGaugeFieldAlgebra 𝓖) →ₐ[ℝ] (LocalGaugeFieldAlgebra 𝓖)
```

lemma massWeightScale_ι [source]

```
@[simp]
lemma massWeightScale_ι (c : ℝ) (x : (GaugeBoson.JetComponentSpace 𝓖)) :
    (massWeightScale 𝓖) c (SymmetricAlgebra.ι ℝ _ x)
      = SymmetricAlgebra.ι ℝ _ ((GaugeBoson.JetComponentSpace.massWeightScale 𝓖) c x)
```

lemma massWeightScale_ofA [source]

***The gauge field carries mass weight two** — mass dimension one.*

```
@[simp]
lemma massWeightScale_ofA (c : ℝ) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓖) :
    (massWeightScale 𝓖) c ((ofA 𝓖) μ φ) = c ^ 2 • (ofA 𝓖) μ φ
```

lemma LocalGaugeFieldAlgebra.massWeightScale_jetDeriv [source]

A total derivative adds mass weight two.

```
lemma massWeightScale_jetDeriv (c : ℝ) (μ : Fin 1 ⊕ Fin 3) (x : (LocalGaugeFieldAlgebra 𝓖)) :
    (massWeightScale 𝓖) c ((jetDeriv 𝓖) μ x) = c ^ 2 • (jetDeriv 𝓖) μ
      ((massWeightScale 𝓖) c x)
```

lemma massWeightScale_iteratedJetDeriv [source]

The iterated derivative `∂_s` adds mass weight `2 |s|`.

```
lemma massWeightScale_iteratedJetDeriv (c : ℝ) (s : Multiset (Fin 1 ⊕ Fin 3))
    (x : (LocalGaugeFieldAlgebra 𝓖)) :
    (massWeightScale 𝓖) c ((iteratedJetDeriv 𝓖) s x)
      = c ^ (2 * Multiset.card s) • (iteratedJetDeriv 𝓖) s ((massWeightScale 𝓖) c x)
```

2.182 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.MassWeightPoly

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/LocalGauge/MassWeightPoly.lean on GitHub.

def monomial$_l$ [source]

The monomial map into polynomials over the complexified jet algebra, as a map of $\mathbb{R}$-modules rather than of modules over the complexified jet algebra.

```
noncomputable def monomial_l (n : ℕ) :
    (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) →_l[ℝ]
      Polynomial (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤)
```

lemma monomial$_l$_apply [source]

```
@[simp]
lemma monomial_l_apply (n : ℕ) (x : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) :
    (monomial_l (𝔤 := 𝔤)) n x = Polynomial.monomial n x
```

def ιComplex [source]

A component function, viewed inside the complexified jet algebra: the generator `∂_s A_μ^φ` tensored with the scalar one.

```
noncomputable def ιComplex :
    (GaugeBoson.JetComponentSpace 𝔤) →_l[ℝ] ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤
```

lemma ιComplex_apply [source]

```
@[simp]
lemma ιComplex_apply (x : (GaugeBoson.JetComponentSpace 𝔤)) :
    (ιComplex (𝔤 := 𝔤)) x =
      (1 : ℂ) ⊗ₜ[ℝ] SymmetricAlgebra.ι ℝ (GaugeBoson.JetComponentSpace 𝔤) x
```

def jetComponentPoly [source]

The mass-weight polynomial of a component function: the linear map sending the symbol `∂_s A^φ` to `X ^ (2 + 2 |s|)` times itself, read off from the multiset basis of the real derivative symbols.

```
noncomputable def jetComponentPoly :
    (GaugeBoson.JetComponentSpace 𝔤) →_l[ℝ]
      Polynomial (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤)
```

lemma jetComponentPoly_basisMultiset_tmul [source]

On the symbol `∂_s A^φ` the component map is the monomial of degree `2 + 2 |s|`: the gauge field contributes two and each derivative two more.

```
lemma jetComponentPoly_basisMultiset_tmul (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℝ (GaugeBoson 𝔤)) :
    (jetComponentPoly (𝔤 := 𝔤)) (SpaceTimeDerivAlgebraℝ.basisMultiset s ⊗ₜ[ℝ] φ) =
      Polynomial.monomial (2 + 2 * Multiset.card s)
        (ιComplex (SpaceTimeDerivAlgebraℝ.basisMultiset s ⊗ₜ[ℝ] φ))
```

def massWeightPoly [source]

The mass-weight polynomial on the gauge-boson jet algebra: the $\mathbb{R}$-algebra map sending a generator of mass weight `n` to `X ^ n` times its image in the complexification. It needs no side condition because the target is commutative.

```
noncomputable def massWeightPoly :
    LocalGaugeFieldAlgebra 𝓖 →ₐ[ℝ]
      Polynomial (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝓖)
```

lemma massWeightPoly_ι [source]

On a component function the mass-weight polynomial is the component-function map.

```
@[simp]
lemma massWeightPoly_ι (x : (GaugeBoson.JetComponentSpace 𝓖)) :
    (massWeightPoly (𝓖 := 𝓖)) (SymmetricAlgebra.ι ℝ (GaugeBoson.JetComponentSpace 𝓖) x) =
      jetComponentPoly x
```

lemma massWeightPoly_iteratedJetDeriv_ofA [source]

The generator `∂_s A_μ^φ` is a monomial eigenvector of mass weight `2 + 2 |s|`.

```
@[simp]
lemma massWeightPoly_iteratedJetDeriv_ofA (s : Multiset (Fin 1 ⊕ Fin 3))
    (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓖) :
    (massWeightPoly (𝓖 := 𝓖)) (LocalGaugeFieldAlgebra.iteratedJetDeriv 𝓖 s
        (LocalGaugeFieldAlgebra.ofA 𝓖 μ φ)) =
      Polynomial.monomial (2 + 2 * Multiset.card s) ((1 : ℂ) ⊗ₜ[ℝ]
        LocalGaugeFieldAlgebra.iteratedJetDeriv 𝓖 s
          (LocalGaugeFieldAlgebra.ofA 𝓖 μ φ))
```

lemma massWeightPoly_ofA [source]

The undifferentiated gauge field has mass weight two — mass dimension one.

```
lemma massWeightPoly_ofA (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓖) :
    (massWeightPoly (𝓖 := 𝓖)) (LocalGaugeFieldAlgebra.ofA 𝓖 μ φ) =
      Polynomial.monomial 2 ((1 : ℂ) ⊗ₜ[ℝ] LocalGaugeFieldAlgebra.ofA 𝓖 μ φ)
```

def complexMassWeightPoly [source]

The mass-weight polynomial on the complexified gauge-boson jet algebra: the $\mathbb{C}$-algebra map obtained from the real one by extending the scalars, the grading the jet algebra of the Standard Model sees on its gauge sector.

```
noncomputable def complexMassWeightPoly :
    (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝓖) →ₐ[ℂ]
      Polynomial (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝓖)
```

lemma complexMassWeightPoly_tmul [source]

On a pure tensor the complexified grading is the scalar times the real grading.

```
lemma complexMassWeightPoly_tmul (z : ℂ) (x : LocalGaugeFieldAlgebra 𝓖) :
    (complexMassWeightPoly (𝓖 := 𝓖)) (z ⊗ₜ[ℝ] x) =
```

```
      Polynomial.C (z ⊗ₜ[ℝ] (1 : LocalGaugeFieldAlgebra 𝔤)) * massWeightPoly x
```

lemma complexMassWeightPoly_tmul_iteratedJetDeriv_ofA [source]

The generator `∂_s A_μ^φ` of the complexified jet algebra, with any complex coefficient, is a monomial eigenvector of mass weight `2 + 2 |s|`.

```
@[simp]
lemma complexMassWeightPoly_tmul_iteratedJetDeriv_ofA (z : ℂ)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    (complexMassWeightPoly (𝔤 := 𝔤)) (z ⊗ₜ[ℝ] LocalGaugeFieldAlgebra.iteratedJetDeriv 𝔤 s
        (LocalGaugeFieldAlgebra.ofA 𝔤 μ φ)) =
      Polynomial.monomial (2 + 2 * Multiset.card s) (z ⊗ₜ[ℝ]
        LocalGaugeFieldAlgebra.iteratedJetDeriv 𝔤 s
          (LocalGaugeFieldAlgebra.ofA 𝔤 μ φ))
```

lemma jetComponentPoly_eval_one [source]

Setting the formal variable to one collapses the component map back to the component function it graded. The derivative monomials span, so it is enough to check this on the multiset basis.

```
lemma jetComponentPoly_eval_one (x : (GaugeBoson.JetComponentSpace 𝔤)) :
    ((jetComponentPoly (𝔤 := 𝔤)) x).eval 1 = ιComplex x
```

lemma massWeightPoly_eval_one [source]

Setting the formal variable to one recovers the original element, viewed in the complexification.

```
lemma massWeightPoly_eval_one (x : LocalGaugeFieldAlgebra 𝔤) :
    ((massWeightPoly (𝔤 := 𝔤)) x).eval 1 =
      Algebra.TensorProduct.includeRight (R := ℝ) (A := ℂ)
        (B := LocalGaugeFieldAlgebra 𝔤) x
```

lemma complexMassWeightPoly_eval_one [source]

Setting the formal variable to one recovers the original element of the complexified jet algebra: the mass-weight pieces sum back to it.

```
lemma complexMassWeightPoly_eval_one (y : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) :
    ((complexMassWeightPoly (𝔤 := 𝔤)) y).eval 1 = y
```

lemma complexMassWeightPoly_injective [source]

The mass-weight polynomial on the complexification is injective: an element is recovered from its graded pieces.

```
lemma complexMassWeightPoly_injective : Function.Injective (complexMassWeightPoly (𝔤 := 𝔤))
```

2.183 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGauge.Realization

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/LocalGauge/Realization.lean on GitHub.

abbrev Realization [source]

A real algebra `B` carrying the gauge bosons of the package `jets`: a real algebra map out of the local gauge field algebra, equivariant for the jet gauge group and the Lorentz group, both acting on the whole of `B` by algebra endomorphisms. It is built from the fields `toAlgHom`, `map_fst`, `map_snd`, `fst_mul`, `snd_mul` of `Representation.EquivariantAlgHom`, which the lemmas `map_repJet`, `map_repLorentz`, `repJet_mul` and `repLorentz_mul` name.

```
abbrev Realization (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) (B : Type) [Ring B] [Algebra ℝ B]
    (repJet : Representation ℝ GJ B) (repLorentz : Representation ℝ SL(2,ℂ) B)
```

def id [source]

The local gauge field algebra realized in itself, by the identity.

```
noncomputable def id : Realization jets (LocalGaugeFieldAlgebra 𝔤)
    (LocalGaugeFieldAlgebra.repJet jets) (repLorentzGroup 𝔤)
```

lemma id_toAlgHom [source]

```
@[simp]
lemma id_toAlgHom : (id jets).toAlgHom = AlgHom.id ℝ (LocalGaugeFieldAlgebra 𝔤)
```

lemma map_repJet [source]

The map is equivariant for the jet gauge group.

```
lemma map_repJet (U : GJ) (x : LocalGaugeFieldAlgebra 𝔤) :
    h.toAlgHom (LocalGaugeFieldAlgebra.repJet jets U x) = repJet U (h.toAlgHom x)
```

lemma map_repLorentz [source]

The map is equivariant for the Lorentz group.

```
lemma map_repLorentz (Λ : SL(2,ℂ)) (x : LocalGaugeFieldAlgebra 𝔤) :
    h.toAlgHom (LocalGaugeFieldAlgebra.repLorentzGroup 𝔤 Λ x) = repLorentz Λ (h.toAlgHom x)
```

lemma repJet_mul [source]

The jet gauge group acts on the whole of `B` by algebra endomorphisms.

```
lemma repJet_mul (U : GJ) (b₁ b₂ : B) : repJet U (b₁ * b₂) = repJet U b₁ * repJet U b₂
```

lemma repLorentz_mul [source]

The Lorentz group acts on the whole of `B` by algebra endomorphisms.

```
lemma repLorentz_mul (Λ : SL(2,ℂ)) (b₁ b₂ : B) :
    repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂
```

def A [source]

The derivative symbols `∂_s A_μ^φ` of a realization: the images of the algebra's own symbols `derivA`.

```
noncomputable def A (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] B
```

lemma A_apply [source]

```
lemma A_apply (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    h.A s μ φ = h.toAlgHom (derivA 𝔤 s μ φ)
```

lemma id_A [source]

```
@[simp]
lemma id_A : (id jets).A = derivA 𝔤
```

lemma commute_A [source]

```
lemma commute_A (p q : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ ψ : Module.Dual ℝ 𝔤) : Commute (h.A p μ φ) (h.A q ν ψ)
```

lemma ext_A [source]

Two realizations with the same derivative symbols are equal.

```
lemma ext_A {h₁ h₂ : Realization jets B repJet repLorentz}
    (hA : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
      h₁.A s μ φ = h₂.A s μ φ) : h₁ = h₂
```

lemma gauge_apply_deriv [source]

The gauge law: a jet `U` acts on `∂_s A_μ^φ` by the Leibniz convolution of the dual adjoint Taylor coefficients of `U⁻¹` against lower symbols, plus the base-point value of the `s`-th derivative of the Maurer–Cartan form of `U⁻¹`.

```
lemma gauge_apply_deriv (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repJet U (h.A s μ φ) =
      (s.antidiagonal.map fun p => h.A p.2 μ (jets.adjointDualCoeff U⁻¹ p.1 φ)).sum
      + algebraMap ℝ B (φ (jets.evalLie (jets.iteratedDeriv s (jets.maurerCartan U⁻¹ μ))))
```

lemma lorentz_apply [source]

The Lorentz law: the symbol carries one covector index, and each derivative slot transforms as a covector.

```
lemma lorentz_apply (Λ : SL(2,ℂ)) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repLorentz Λ (h.A (List.ofFn l) μ φ) =
      ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ i, ((SL2C.toLorentzGroup Λ).1 (p i) (l i) : ℝ)) •
        ∑ a, ((SL2C.toLorentzGroup Λ).1 a μ : ℝ) • h.A (List.ofFn p) a φ
```

lemma toAlgHom_covDerivFieldStrength [source]

The map of a realization carries the covariant derivatives of the field strength to those of its symbols.

```
lemma toAlgHom_covDerivFieldStrength (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    h.toAlgHom (covDerivFieldStrength 𝔤 l μ ν φ)
      = GaugeAlgebraRealization.iteratedCovDerivAdjoint h.A l
        (GaugeAlgebraRealization.fieldStrength h.A μ ν) 0 φ
```

lemma toAlgHom_fieldStrength [source]

```
lemma toAlgHom_fieldStrength (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    h.toAlgHom (fieldStrength 𝔤 μ ν φ) = GaugeAlgebraRealization.fieldStrength h.A μ ν 0 φ
```

def toGaugeAlgebraRealization [source]

The complexification of a real realization in a complex algebra, by `AlgHom.liftEquiv`; the symbols are unchanged.

```
noncomputable def toGaugeAlgebraRealization
    (h : Realization jets B (repJet.restrictScalars ℝ) (repLorentz.restrictScalars ℝ)) :
    GaugeAlgebraRealization jets B repJet repLorentz where
```

lemma toGaugeAlgebraRealization_A [source]

```
@[simp]
lemma toGaugeAlgebraRealization_A
    (h : Realization jets B (repJet.restrictScalars ℝ) (repLorentz.restrictScalars ℝ)) :
    h.toGaugeAlgebraRealization.A = h.A
```

lemma toGaugeAlgebraRealization_toAlgHom_one_tmul [source]

```
lemma toGaugeAlgebraRealization_toAlgHom_one_tmul
    (h : Realization jets B (repJet.restrictScalars ℝ) (repLorentz.restrictScalars ℝ))
    (x : LocalGaugeFieldAlgebra 𝔤) :
    h.toGaugeAlgebraRealization.toAlgHom ((1 : ℂ) ⊗ₜ[ℝ] x) = h.toAlgHom x
```

def toRealization [source]

The restriction of scalars of a complex realization: the algebra map precomposed with $x \mapsto 1 \otimes_t x$.

```
noncomputable def _root_.GaugeAlgebraRealization.toRealization
    (h : GaugeAlgebraRealization jets B repJet repLorentz) :
    Realization jets B (repJet.restrictScalars ℝ) (repLorentz.restrictScalars ℝ) where
```

lemma toRealization_toAlgHom_apply [source]

```
@[simp]
lemma _root_.GaugeAlgebraRealization.toRealization_toAlgHom_apply
    (h : GaugeAlgebraRealization jets B repJet repLorentz) (x : LocalGaugeFieldAlgebra 𝔤) :
    h.toRealization.toAlgHom x = h.toAlgHom ((1 : ℂ) ⊗ₜ[ℝ] x)
```

lemma toRealization_A [source]

```
@[simp]
lemma _root_.GaugeAlgebraRealization.toRealization_A
    (h : GaugeAlgebraRealization jets B repJet repLorentz) : h.toRealization.A = h.A
```

lemma toRealization_id_toAlgHom [source]

```
lemma _root_.GaugeAlgebraRealization.toRealization_id_toAlgHom :
    (GaugeAlgebraRealization.id jets).toRealization.toAlgHom
      = (Algebra.TensorProduct.includeRight :
          LocalGaugeFieldAlgebra 𝓰 →ₐ[ℝ] ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝓰)
```

def equivGaugeAlgebraRealization [source]

For a complex target, real realizations are the complex realizations.

```
noncomputable def equivGaugeAlgebraRealization :
    Realization jets B (repJet.restrictScalars ℝ) (repLorentz.restrictScalars ℝ)
      ≃ GaugeAlgebraRealization jets B repJet repLorentz where
```

2.184 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGaugeCov.Basic

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/LocalGaugeCov/Basic.lean on GitHub.

def LocalGaugeCovFieldAlgebra [source]

The covariant field algebra of the gauge bosons: the real unital subalgebra of the local gauge field algebra generated by the field strength and its iterated covariant derivatives ∇_{l₁} ⋯ ∇_{lₙ} F_μν^φ, along all ordered lists of directions.

```
noncomputable def LocalGaugeCovFieldAlgebra : Subalgebra ℝ (LocalGaugeFieldAlgebra 𝓰)
```

lemma mem_tower_iff [source]

The generating set, as the covariant derivatives of the field strength.

```
lemma mem_tower_iff (x : LocalGaugeFieldAlgebra 𝓰) :
    x ∈ GaugeAlgebraRealization.tower (derivA 𝓰) ↔
      ∃ (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓰),
        x = covDerivFieldStrength 𝓰 l μ ν φ
```

lemma covDerivFieldStrength_mem [source]

```
lemma covDerivFieldStrength_mem (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝓰) : covDerivFieldStrength 𝓰 l μ ν φ ∈ LocalGaugeCovFieldAlgebra 𝓰
```

lemma fieldStrength_mem [source]

```
lemma fieldStrength_mem (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓰) :
    fieldStrength 𝓰 μ ν φ ∈ LocalGaugeCovFieldAlgebra 𝓰
```

lemma adjoin_preimage_tower_eq_top [source]

Inside the covariant field algebra, the preimage of the generating tower under the inclusion generates the whole real subalgebra.

```
lemma adjoin_preimage_tower_eq_top :
    Algebra.adjoin ℝ ((Subtype.val : LocalGaugeCovFieldAlgebra 𝓖 → LocalGaugeFieldAlgebra 𝓖)
      ⁻¹' GaugeAlgebraRealization.tower (derivA 𝓖)) = ⊤
```

lemma induction [source]

The generation principle: a property holding on the covariant derivatives of the field strength and on the scalars, and closed under sums and products, holds on the whole covariant field algebra.

```
lemma induction {P : LocalGaugeFieldAlgebra 𝓖 → Prop} {x : LocalGaugeFieldAlgebra 𝓖}
    (hx : x ∈ LocalGaugeCovFieldAlgebra 𝓖)
    (hgen : ∀ (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓖),
      P (covDerivFieldStrength 𝓖 l μ ν φ))
    (halg : ∀ r : ℝ, P (algebraMap ℝ (LocalGaugeFieldAlgebra 𝓖) r))
    (hadd : ∀ x y, P x → P y → P (x + y))
    (hmul : ∀ x y, P x → P y → P (x * y)) : P x
```

lemma mapsTo [source]

An algebra endomorphism carrying the generators into the covariant field algebra carries the whole covariant field algebra into itself.

```
lemma mapsTo (f : LocalGaugeFieldAlgebra 𝓖 →ₐ[ℝ] LocalGaugeFieldAlgebra 𝓖)
    (hgen : ∀ (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓖),
      f (covDerivFieldStrength 𝓖 l μ ν φ) ∈ LocalGaugeCovFieldAlgebra 𝓖)
    {x : LocalGaugeFieldAlgebra 𝓖} (hx : x ∈ LocalGaugeCovFieldAlgebra 𝓖) :
    f x ∈ LocalGaugeCovFieldAlgebra 𝓖
```

def covF [source]

The generators ∇_l F_μν^φ of the covariant field algebra, as elements of it.

```
noncomputable def covF (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ 𝓖 →ₗ[ℝ] LocalGaugeCovFieldAlgebra 𝓖 where
```

lemma coe_covF [source]

```
@[simp]
lemma coe_covF (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓖) :
    (covF 𝓖 l μ ν φ : LocalGaugeFieldAlgebra 𝓖) = covDerivFieldStrength 𝓖 l μ ν φ
```

lemma coe_covF_nil [source]

```
lemma coe_covF_nil (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓖) :
    (covF 𝓖 [] μ ν φ : LocalGaugeFieldAlgebra 𝓖) = fieldStrength 𝓖 μ ν φ
```

lemma val_comp_covF [source]

```
lemma val_comp_covF (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    (LocalGaugeCovFieldAlgebra 𝔤).val.toLinearMap ∘ₗ covF 𝔤 l μ ν
      = covDerivFieldStrength 𝔤 l μ ν
```

lemma algHom_ext [source]

Two algebra maps out of the covariant field algebra agreeing on the generators are equal. This is uniqueness only: the covariant field algebra is not free on the generators, so an assignment of their images does not by itself define a map.

```
lemma algHom_ext {B : Type} [Semiring B] [Algebra ℝ B]
    {f g : LocalGaugeCovFieldAlgebra 𝔤 →ₐ[ℝ] B}
    (h : ∀ (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
      f (covF 𝔤 l μ ν φ) = g (covF 𝔤 l μ ν φ)) : f = g
```

lemma repJet_mem [source]

The jet gauge group preserves the covariant field algebra: a jet rotates the adjoint index of each generator through the value of its inverse.

```
lemma repJet_mem (U : GJ) {x : LocalGaugeFieldAlgebra 𝔤}
    (hx : x ∈ LocalGaugeCovFieldAlgebra 𝔤) :
    LocalGaugeFieldAlgebra.repJet jets U x ∈ LocalGaugeCovFieldAlgebra 𝔤
```

lemma repLorentzGroup_mem [source]

The Lorentz group preserves the covariant field algebra: the generators mix among themselves by the columns of the Lorentz matrix.

```
lemma repLorentzGroup_mem (Λ : SL(2,ℂ)) {x : LocalGaugeFieldAlgebra 𝔤}
    (hx : x ∈ LocalGaugeCovFieldAlgebra 𝔤) :
    LocalGaugeFieldAlgebra.repLorentzGroup 𝔤 Λ x ∈ LocalGaugeCovFieldAlgebra 𝔤
```

def repJet [source]

The action of the jet gauge group on the covariant field algebra, restricted from the local gauge field algebra.

```
noncomputable def repJet : Representation ℝ GJ (LocalGaugeCovFieldAlgebra 𝔤)
```

lemma coe_repJet [source]

```
@[simp]
lemma coe_repJet (U : GJ) (x : LocalGaugeCovFieldAlgebra 𝔤) :
    (repJet jets U x : LocalGaugeFieldAlgebra 𝔤) = LocalGaugeFieldAlgebra.repJet jets U x
```

lemma repJet_apply_mul [source]

```
lemma repJet_apply_mul (U : GJ) (x y : LocalGaugeCovFieldAlgebra 𝔤) :
    repJet jets U (x * y) = repJet jets U x * repJet jets U y
```

def repLorentzGroup [source]

The action of the Lorentz group on the covariant field algebra, restricted from the local gauge field algebra.

```
noncomputable def repLorentzGroup : Representation ℝ SL(2,ℂ) (LocalGaugeCovFieldAlgebra 𝔤)
```

lemma coe_repLorentzGroup [source]

```
@[simp]
lemma coe_repLorentzGroup (Λ : SL(2,ℂ)) (x : LocalGaugeCovFieldAlgebra 𝔤) :
    (repLorentzGroup 𝔤 Λ x : LocalGaugeFieldAlgebra 𝔤)
      = LocalGaugeFieldAlgebra.repLorentzGroup 𝔤 Λ x
```

lemma repLorentzGroup_apply_mul [source]

```
lemma repLorentzGroup_apply_mul (Λ : SL(2,ℂ)) (x y : LocalGaugeCovFieldAlgebra 𝔤) :
    repLorentzGroup 𝔤 Λ (x * y) = repLorentzGroup 𝔤 Λ x * repLorentzGroup 𝔤 Λ y
```

def repValue [source]

The action of the ordinary gauge group on the covariant field algebra: the jet action at the constant jets.

```
noncomputable def repValue : Representation ℝ G₀ (LocalGaugeCovFieldAlgebra 𝔤)
```

lemma coe_repValue [source]

```
@[simp]
lemma coe_repValue (g : G₀) (x : LocalGaugeCovFieldAlgebra 𝔤) :
    (repValue jets g x : LocalGaugeFieldAlgebra 𝔤)
      = LocalGaugeFieldAlgebra.repJet jets (jets.ofConstant g) x
```

lemma repValue_apply_mul [source]

```
lemma repValue_apply_mul (g : G₀) (x y : LocalGaugeCovFieldAlgebra 𝔤) :
    repValue jets g (x * y) = repValue jets g x * repValue jets g y
```

lemma repJet_eq_repValue_eval [source]

The action of the jet gauge group on the covariant field algebra factors through evaluation: a jet acts as the constant jet of its value. The derivatives of a gauge transformation act trivially on covariant expressions.

```
lemma repJet_eq_repValue_eval (U : GJ) (x : LocalGaugeCovFieldAlgebra 𝔤) :
    repJet jets U x = repValue jets (jets.eval U) x
```

lemma repValue_covF [source]

The ordinary gauge group rotates the adjoint index of a generator through the dual adjoint action of the inverse.

```
lemma repValue_covF (g : G₀) (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝓖) :
    repValue jets g (covF 𝓖 l μ ν φ) = covF 𝓖 l μ ν ((jets.adjoint g⁻¹).dualMap φ)
```

lemma repJet_covF [source]

A jet acts on a generator through the value of its inverse alone.

```
lemma repJet_covF (U : GJ) (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝓖) :
    repJet jets U (covF 𝓖 l μ ν φ)
      = covF 𝓖 l μ ν ((jets.adjoint (jets.eval U⁻¹)).dualMap φ)
```

lemma repLorentzGroup_covF [source]

The Lorentz law of the generators: every covariant slot and both covector indices mix by the columns of the Lorentz matrix.

```
lemma repLorentzGroup_covF (Λ : SL(2,ℂ)) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓖) :
    repLorentzGroup 𝓖 Λ (covF 𝓖 (List.ofFn l) μ ν φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ i, ((SL2C.toLorentzGroup Λ).1 (p i) (l i) : ℝ)) •
          ∑ a, ((SL2C.toLorentzGroup Λ).1 a μ : ℝ) • ∑ b, ((SL2C.toLorentzGroup Λ).1 b ν : ℝ) •
            covF 𝓖 (List.ofFn p) a b φ
```

def complexVal [source]

The complexified covariant field algebra inside the complexified local gauge field algebra: the base change of `Subalgebra.val` along $\mathbb{R} \to \mathbb{C}$.

```
noncomputable def complexVal :
    (ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝓖) →ₐ[ℂ] ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝓖
```

lemma complexVal_tmul [source]

```
@[simp]
lemma complexVal_tmul (z : ℂ) (x : LocalGaugeCovFieldAlgebra 𝓖) :
    complexVal 𝓖 (z ⊗ₜ[ℝ] x) = z ⊗ₜ[ℝ] (x : LocalGaugeFieldAlgebra 𝓖)
```

lemma complexVal_injective [source]

```
lemma complexVal_injective : Function.Injective (complexVal 𝓖)
```

def complexRepValue [source]

The action of the ordinary gauge group on the complexified covariant field algebra, by base change.

```
noncomputable def complexRepValue :
    Representation ℂ G₀ (ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝓖)
```

lemma complexRepValue_tmul [source]

```
@[simp]
lemma complexRepValue_tmul (g : G₀) (z : ℂ) (x : LocalGaugeCovFieldAlgebra 𝔤) :
    complexRepValue jets g (z ⊗ₜ[ℝ] x) = z ⊗ₜ[ℝ] repValue jets g x
```

def complexRepLorentzGroup [source]

The Lorentz action on the complexified covariant field algebra, by base change.

```
noncomputable def complexRepLorentzGroup :
    Representation ℂ SL(2,ℂ) (ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝔤)
```

lemma complexRepLorentzGroup_tmul [source]

```
@[simp]
lemma complexRepLorentzGroup_tmul (Λ : SL(2,ℂ)) (z : ℂ) (x : LocalGaugeCovFieldAlgebra 𝔤) :
    complexRepLorentzGroup 𝔤 Λ (z ⊗ₜ[ℝ] x) = z ⊗ₜ[ℝ] repLorentzGroup 𝔤 Λ x
```

lemma complexVal_complexRepValue [source]

Along the complexified inclusion the ordinary gauge group acts as the constant jets.

```
lemma complexVal_complexRepValue (g : G₀) (y : ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝔤) :
    complexVal 𝔤 (complexRepValue jets g y)
      = LocalGaugeFieldAlgebra.complexRepJet jets (jets.ofConstant g) (complexVal 𝔤 y)
```

lemma complexVal_complexRepLorentzGroup [source]

Along the complexified inclusion the Lorentz action is the ambient one.

```
lemma complexVal_complexRepLorentzGroup (Λ : SL(2,ℂ))
    (y : ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝔤) :
    complexVal 𝔤 (complexRepLorentzGroup 𝔤 Λ y)
      = LocalGaugeFieldAlgebra.complexRepLorentzGroup 𝔤 Λ (complexVal 𝔤 y)
```

2.185 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.LocalGaugeCov.Realization

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/LocalGaugeCov/Realization.lean on GitHub.

abbrev Realization [source]

A real algebra `B` carrying the covariant gauge-boson tower of the package `jets`: a real algebra map out of the covariant field algebra, equivariant for the ordinary gauge group and the Lorentz group, both acting on the whole of `B` by algebra endomorphisms. It is built from the fields `toAlgHom`, `map_fst`, `map_snd`, `fst_mul`, `snd_mul` of `Representation.EquivariantAlgHom`, which the lemmas `map_repValue`, `map_repLorentz`, `repGauge_mul` and `repLorentz_mul` name.

```
abbrev Realization (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) (B : Type) [Ring B] [Algebra ℝ B]
    (repGauge : Representation ℝ G₀ B) (repLorentz : Representation ℝ SL(2,ℂ) B)
```

def id [source]

The covariant field algebra realized in itself, by the identity.

```
noncomputable def id : Realization jets (LocalGaugeCovFieldAlgebra 𝓖) (repValue jets)
    (repLorentzGroup 𝓖)
```

lemma id_toAlgHom [source]

```
@[simp]
lemma id_toAlgHom : (id jets).toAlgHom = AlgHom.id ℝ (LocalGaugeCovFieldAlgebra 𝓖)
```

lemma map_repValue [source]

The map is equivariant for the ordinary gauge group.

```
lemma map_repValue (g : G₀) (x : LocalGaugeCovFieldAlgebra 𝓖) :
    k.toAlgHom (repValue jets g x) = repGauge g (k.toAlgHom x)
```

lemma map_repLorentz [source]

The map is equivariant for the Lorentz group.

```
lemma map_repLorentz (Λ : SL(2,ℂ)) (x : LocalGaugeCovFieldAlgebra 𝓖) :
    k.toAlgHom (repLorentzGroup 𝓖 Λ x) = repLorentz Λ (k.toAlgHom x)
```

lemma repGauge_mul [source]

The ordinary gauge group acts on the whole of `B` by algebra endomorphisms.

```
lemma repGauge_mul (g : G₀) (b₁ b₂ : B) :
    repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂
```

lemma repLorentz_mul [source]

The Lorentz group acts on the whole of `B` by algebra endomorphisms.

```
lemma repLorentz_mul (Λ : SL(2,ℂ)) (b₁ b₂ : B) :
    repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂
```

def F [source]

The covariant tower `∇_l F_μν^φ` of a realization: the images of the generators of the covariant field algebra.

```
noncomputable def F (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ 𝓖 →ₗ[ℝ] B
```

lemma F_apply [source]

```
lemma F_apply (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓖) :
    k.F l μ ν φ = k.toAlgHom (covF 𝓖 l μ ν φ)
```

lemma id_F [source]

```
@[simp]
lemma id_F : (id jets).F = covF 𝔤
```

lemma F_nil [source]

```
lemma F_nil (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    k.F [] μ ν φ = k.toAlgHom (fieldStrength 𝔤 μ ν φ, fieldStrength_mem μ ν φ)
```

lemma commute_F [source]

```
lemma commute_F (l l' : List (Fin 1 ⊕ Fin 3)) (μ ν μ' ν' : Fin 1 ⊕ Fin 3)
    (φ ψ : Module.Dual ℝ 𝔤) : Commute (k.F l μ ν φ) (k.F l' μ' ν' ψ)
```

lemma ext_F [source]

Two realizations with the same covariant tower are equal. This is uniqueness only: a tower in `B` need not come from a realization.

```
lemma ext_F {k₁ k₂ : Realization jets B repGauge repLorentz}
    (hF : ∀ (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
      k₁.F l μ ν φ = k₂.F l μ ν φ) : k₁ = k₂
```

lemma gauge_F [source]

The gauge law of the covariant tower: the adjoint index rotates through the dual adjoint action of the inverse.

```
lemma gauge_F (g : G₀) (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repGauge g (k.F l μ ν φ) = k.F l μ ν ((jets.adjoint g⁻¹).dualMap φ)
```

lemma lorentz_F [source]

The Lorentz law of the covariant tower: every covariant slot and both covector indices mix by the columns of the Lorentz matrix.

```
lemma lorentz_F (Λ : SL(2,ℂ)) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repLorentz Λ (k.F (List.ofFn l) μ ν φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ i, ((SL2C.toLorentzGroup Λ).1 (p i) (l i) : ℝ)) •
          ∑ a, ((SL2C.toLorentzGroup Λ).1 a μ : ℝ) • ∑ b, ((SL2C.toLorentzGroup Λ).1 b ν : ℝ) •
            k.F (List.ofFn p) a b φ
```

lemma map_repJet [source]

A covariant realization intertwines the jet action with the `G₀` action at the value of the jet.

```
lemma map_repJet (U : GJ) (x : LocalGaugeCovFieldAlgebra 𝔤) :
    k.toAlgHom (repJet jets U x) = repGauge (jets.eval U) (k.toAlgHom x)
```

def restrict [source]

The restriction of a realization of the local gauge field algebra to the covariant field algebra, along the inclusion; the ordinary gauge group acts on the target as the constant jets.

```
noncomputable def restrict :
    LocalGaugeCovFieldAlgebra.Realization jets B (repJet.comp jets.ofConstant) repLorentz
```

lemma restrict_toAlgHom_apply [source]

```
@[simp]
lemma restrict_toAlgHom_apply (x : LocalGaugeCovFieldAlgebra 𝓰) :
    h.restrict.toAlgHom x = h.toAlgHom x
```

lemma restrict_id_toAlgHom [source]

```
lemma restrict_id_toAlgHom :
    (id jets).restrict.toAlgHom = (LocalGaugeCovFieldAlgebra 𝓰).val
```

lemma restrict_F [source]

```
lemma restrict_F (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓰) :
    h.restrict.F l μ ν φ = h.toAlgHom (covDerivFieldStrength 𝓰 l μ ν φ)
```

lemma restrict_F_eq_iteratedCovDerivAdjoint [source]

The covariant tower of a restriction is the covariant tower of the symbols of `B`.

```
lemma restrict_F_eq_iteratedCovDerivAdjoint (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝓰) :
    h.restrict.F l μ ν φ = GaugeAlgebraRealization.iteratedCovDerivAdjoint h.A l
      (GaugeAlgebraRealization.fieldStrength h.A μ ν) 0 φ
```

lemma restrict_F_nil [source]

```
lemma restrict_F_nil (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓰) :
    h.restrict.F [] μ ν φ = GaugeAlgebraRealization.fieldStrength h.A μ ν 0 φ
```

lemma restrict_map_repJet [source]

```
lemma restrict_map_repJet (U : GJ) (x : LocalGaugeCovFieldAlgebra 𝓰) :
    h.restrict.toAlgHom (LocalGaugeCovFieldAlgebra.repJet jets U x)
      = repJet U (h.restrict.toAlgHom x)
```

lemma repJet_restrict_toAlgHom [source]

On the image of the covariant field algebra a jet acts as the constant jet of its value; nothing is assumed about the jet action elsewhere in `B`.

```
lemma repJet_restrict_toAlgHom (U : GJ) (x : LocalGaugeCovFieldAlgebra 𝓰) :
    repJet U (h.restrict.toAlgHom x)
      = repJet (jets.ofConstant (jets.eval U)) (h.restrict.toAlgHom x)
```

2.186 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Realization.Basic

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/Realization/Basic.lean on GitHub.

structure GaugeAlgebraRealization [source]

An algebra `B` carrying the gauge bosons of the package `jets`: an algebra map out of the complexified gauge-boson jet algebra, equivariant for the jet gauge group and the Lorentz group, with both groups acting on the whole of `B` by algebra endomorphisms. The gauge-field symbols of `B` are the images of the jet algebra's symbols, `GaugeAlgebraRealization.A`, and they satisfy the jet algebra's laws by transport.

```
structure GaugeAlgebraRealization (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) (B : Type) [Ring B]
    [Algebra ℂ B] (repJet : Representation ℂ GJ B) (repLorentz : Representation ℂ SL(2,ℂ) B)
    where
  toAlgHom : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤 →ₐ[ℂ] B
  A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B
  A_eq : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
    A s μ φ = toAlgHom (LocalGaugeFieldAlgebra.gaugeField 𝔤 s μ φ)
  map_repJet : ∀ (U : GJ) (x : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤),
    toAlgHom (LocalGaugeFieldAlgebra.complexRepJet jets U x) = repJet U (toAlgHom x)
  map_repLorentz : ∀ (Λ : SL(2,ℂ)) (x : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤),
    toAlgHom (LocalGaugeFieldAlgebra.complexRepLorentzGroup 𝔤 Λ x) = repLorentz Λ (toAlgHom x)
  repJet_mul : ∀ (U : GJ) (b₁ b₂ : B), repJet U (b₁ * b₂) = repJet U b₁ * repJet U b₂
  repLorentz_mul : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
    repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂
```

def id [source]

The gauge-boson jet algebra realized in itself, by the identity.

```
noncomputable def id : GaugeAlgebraRealization jets (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤)
    (complexRepJet jets) (complexRepLorentzGroup 𝔤) where
```

lemma A_apply [source]

```
lemma A_apply (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    h.A s μ φ = h.toAlgHom (gaugeField 𝔤 s μ φ)
```

lemma id_A [source]

```
@[simp]
lemma id_A : (GaugeAlgebraRealization.id jets).A = gaugeField 𝔤
```

lemma ext [source]

A realization is determined by its algebra map.

```
lemma ext {h₁ h₂ : GaugeAlgebraRealization jets B repJet repLorentz}
    (h : h₁.toAlgHom = h₂.toAlgHom) : h₁ = h₂
```

lemma commute_A [source]

The gauge-field symbols of a realization commute, being images of a commutative algebra.

```
lemma commute_A (p q : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ ψ : Module.Dual ℝ 𝔤) : Commute (h.A p μ φ) (h.A q ν ψ)
```

lemma lorentz_apply [source]

The Lorentz law: the gauge-field symbol carries one covector index, and each derivative slot transforms as a covector.

```
lemma lorentz_apply (Λ : SL(2,ℂ)) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repLorentz Λ (h.A (List.ofFn l) μ φ) =
      ∑ (p : Fin n → (Fin 1 ⊕ Fin 3)),
        (∏ (i : Fin n), (((SL2C.toLorentzGroup Λ).1 (p i) (l i) : ℝ) : ℂ)) •
      ∑ a, (((Lorentz.SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) • h.A (List.ofFn p) a φ
```

lemma gauge_apply_deriv [source]

The gauge law: a jet `U` acts on the derivative symbol `∂_s A_μ^φ` by the Leibniz convolution of the dual adjoint Taylor coefficients of `U⁻¹` against lower symbols (the multiset antidiagonal carrying the multinomial coefficients), plus the base-point value of the `s`-th derivative of the Maurer–Cartan form of `U⁻¹`.

```
lemma gauge_apply_deriv (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repJet U (h.A s μ φ) =
      (s.antidiagonal.map fun p => h.A p.2 μ (jets.adjointDualCoeff U⁻¹ p.1 φ)).sum
      + algebraMap ℂ B (φ (jets.evalLie (jets.iteratedDeriv s (jets.maurerCartan U⁻¹ μ))))
```

lemma gauge_mul [source]

The gauge action preserves products: gauge transformations act on the algebra of local expressions as algebra homomorphisms.

```
lemma gauge_mul (U : GJ) (b₁ b₂ : B) : repJet U (b₁ * b₂) = repJet U b₁ * repJet U b₂
```

2.187 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Realization.FieldStrength

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/Realization/FieldStrength.lean on GitHub.

def fieldStrength [source]

The field strength `F_μν = ∂_μ A_ν − ∂_ν A_μ + [A_μ, A_ν]` of a family of gauge-field symbols, as a family of derivative symbols: the `s`-th derivative has the derivative terms through the shifted symbols `A (μ ::ₘ s) ν`, the commutator term through the Leibniz convolution `commutatorFam`. This is the physicists' `F_μν^a = ∂_μ A_ν^a − ∂_ν A_μ^a + f^a_{bc} A_μ^b A_ν^c`: the gauge-algebra bracket already carries the physicists' factor of `i`, so no explicit factor appears — the same normalization as in the structural equation of the Maurer–Cartan form, which is exactly what makes the field strength transform without inhomogeneous terms (`repGauge_fieldStrength`).

```
noncomputable def fieldStrength
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] B
```

lemma fieldStrength_apply [source]

```
@[simp]
lemma fieldStrength_apply
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    fieldStrength A μ ν s φ = A (μ ::ₘ s) ν φ - A (ν ::ₘ s) μ φ + commutatorFam A μ ν s φ
```

lemma fieldStrength_zero [source]

The underived field strength: derivative symbols on singletons, plus the plain commutator.

```
lemma fieldStrength_zero
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (μ ν : Fin 1 ⊕ Fin 3) :
    fieldStrength A μ ν 0 = A {μ} ν - A {ν} μ + commutator A μ ν
```

lemma pair_eq_fieldStrength_sub_commutatorFam [source]

The antisymmetrized pair of derivative symbols is the field strength minus its commutator term.

```
lemma pair_eq_fieldStrength_sub_commutatorFam
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (ν μ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    A (ν ::ₘ s) μ - A (μ ::ₘ s) ν = fieldStrength A ν μ s - commutatorFam A ν μ s
```

lemma bracketFam_swap_of_commute [source]

The bracket of two component families with commuting values is antisymmetric: in the basis expansion the structure constants are antisymmetric in the two gauge indices, and the two field factors of each term may be exchanged.

```
lemma bracketFam_swap_of_commute {f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B}
    (hfg : ∀ φ ψ, Commute (f φ) (g ψ)) :
    bracketFam g f = - bracketFam f g
```

lemma commutatorFam_swap [source]

The derived commutator term is antisymmetric in its two directions when the symbols of the gauge field commute: swapping the two parts of the antidiagonal matches the Leibniz convolution with the swapped one termwise.

```
lemma commutatorFam_swap
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (hA : ∀ (s s' : Multiset (Fin 1 ⊕ Fin 3)) (μ μ' : Fin 1 ⊕ Fin 3)
      (ψ ψ' : Module.Dual ℝ 𝔤), Commute (A s μ ψ) (A s' μ' ψ'))
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    commutatorFam A ν μ s = - commutatorFam A μ ν s
```

lemma fieldStrength_swap [source]

The field strength is antisymmetric in its two covector indices when the symbols of the gauge field commute: the two derivative terms swap outright, the commutator term by `commutatorFam_swap`.

```
lemma fieldStrength_swap
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (hA : ∀ (s s' : Multiset (Fin 1 ⊕ Fin 3)) (μ μ' : Fin 1 ⊕ Fin 3)
      (ψ ψ' : Module.Dual ℝ 𝔤), Commute (A s μ ψ) (A s' μ' ψ'))
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    fieldStrength A ν μ s = - fieldStrength A μ ν s
```

lemma fieldStrength_map [source]

The field strength is natural in the algebra: a multiplicative linear map carries the field strength of a family to the field strength of its image.

```
lemma fieldStrength_map {B' : Type} [Ring B'] [Module ℝ B'] [SMulCommClass ℝ B' B']
    [IsScalarTower ℝ B' B'] (Φ : B →ₗ[ℝ] B') (hΦ : ∀ b₁ b₂, Φ (b₁ * b₂) = Φ b₁ * Φ b₂)
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    fieldStrength (fun p ρ => Φ ∘ₗ A p ρ) μ ν s = Φ ∘ₗ fieldStrength A μ ν s
```

lemma repGauge_fieldStrength [source]

***The field strength transforms in the adjoint, at every derivative order**: under a gauge jet `U` every derivative symbol of `F_μν` transforms by the pure Leibniz convolution of the dual adjoint action over the multiset antidiagonal — the exact analogue of `gauge_apply_deriv` with *no* Maurer–Cartan shift, since the field strength transforms homogeneously. The `κ`-into-the-adjoint splittings of the derivative terms (`repGauge_cons_apply`) cancel the `ad` cross-term convolutions of the commutator (`repGauge_commutatorFam`) through the coassociativity and swap of the antidiagonal, and the derived Maurer–Cartan shifts cancel the bracket-shift convolution through the all-orders structural equation.*

```
lemma repGauge_fieldStrength
    (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repGauge U (fieldStrength h.A μ ν s φ) =
      (s.antidiagonal.map fun p =>
        fieldStrength h.A μ ν p.2 (jets.adjointDualCoeff U⁻¹ p.1 φ)).sum
```

lemma transformsInAdjoint_fieldStrength [source]

***The field strength is an adjoint gauge tensor**: the packaging of `repGauge_fieldStrength` as `TransformsInAdjoint` — the base case of the covariant-derivative recursion `TransformsInAdjoint.covDeri`*

```
lemma transformsInAdjoint_fieldStrength
    (μ ν : Fin 1 ⊕ Fin 3) : TransformsInAdjoint jets repGauge (fieldStrength h.A μ ν)
```

lemma repGauge_fieldStrength_zero [source]

The underived transformation law: at `s = 0` the Leibniz convolution collapses to the homogeneous law — the field strength transforms by the base-point dual adjoint action of `U⁻¹` on the adjoint index.

```
lemma repGauge_fieldStrength_zero
    (U : GJ) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repGauge U (fieldStrength h.A μ ν 0 φ) =
      fieldStrength h.A μ ν 0 (jets.adjointDualCoeff U⁻¹ 0 φ)
```

2.188 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Realization.GaugeLaw

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/Realization/GaugeLaw.lean on GitHub.

def dualPairEquiv [source]

The canonical equivalence, through finite-dimensional duality, between algebra-valued fields `B ⊗ 𝔤` and their component families `φ ↦ A^φ`: the element `b ⊗ a` corresponds to the family `φ ↦ φ(a) b`.

```
noncomputable def dualPairEquiv :
    (B ⊗[ℝ] 𝔤) ≃ₗ[ℝ] (Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
```

def tensorBracket [source]

The bracket of two algebra-valued fields: multiplication in `B` on the first factors, the Lie bracket of the gauge algebra on the second, so that on pure tensors `⁅b₁ ⊗ a₁, b₂ ⊗ a₂⁆ = (b₁ b₂) ⊗ ⁅a₁, a₂⁆`.

```
noncomputable def tensorBracket :
    (B ⊗[ℝ] 𝔤) →ₗ[ℝ] (B ⊗[ℝ] 𝔤) →ₗ[ℝ] B ⊗[ℝ] 𝔤
```

def commutator [source]

The commutator term `⁅A_μ, A_ν⁆` of the field strength, as a component family: the physicists' `f^a_{bc} A_μ^b A_ν^c` contracted with a dual adjoint vector, but basis-free — the two fields are assembled into `B ⊗ 𝔤` by `dualPairEquiv.symm`, bracketed there by `tensorBracket`, and read back out as components.

```
noncomputable def commutator
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (μ ν : Fin 1 ⊕ Fin 3) : Module.Dual ℝ 𝔤 →ₗ[ℝ] B
```

lemma repGauge_apply [source]

The gauge transformation of the underived symbol `A_μ^φ`: the special case `s = 0` of `gauge_apply_deriv`, with no Leibniz convolution left over — the dual adjoint action of the value of `U⁻¹` plus the Maurer–Cartan shift.

```
lemma repGauge_apply (U : GJ)
    (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repGauge U (h.A 0 μ φ) = h.A 0 μ (jets.adjointDualCoeff U⁻¹ ∅ φ) +
      algebraMap ℂ B (φ (jets.evalLie (jets.maurerCartan U⁻¹ μ)))
```

lemma repGauge_deriv_apply [source]

The gauge transformation of the once-derived symbol `∂_ρ A_σ`: the case `s = {ρ}` of `gauge_apply_deriv` — the two Leibniz splittings of one derivative, plus the base-point value of the derived Maurer–Cartan form.

```
lemma repGauge_deriv_apply
    (U : GJ) (ρ σ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repGauge U (h.A {ρ} σ φ) =
      h.A {ρ} σ (jets.adjointDualCoeff U⁻¹ 0 φ) + h.A 0 σ (jets.adjointDualCoeff U⁻¹ {ρ} φ) +
      algebraMap ℂ B (φ (jets.evalLie
        (jets.deriv ρ (jets.maurerCartan U⁻¹ σ))))
```

lemma dualPairEquiv_tmul [source]

```
@[simp]
lemma dualPairEquiv_tmul (b : B) (a : 𝔤) (φ : Module.Dual ℝ 𝔤) :
    dualPairEquiv (b ⊗ₜ[ℝ] a) φ = φ a • b
```

lemma tensorBracket_tmul [source]

```
@[simp]
lemma tensorBracket_tmul (b₁ b₂ : B) (a₁ a₂ : 𝔤) :
    tensorBracket (b₁ ⊗ₜ[ℝ] a₁) (b₂ ⊗ₜ[ℝ] a₂) = (b₁ * b₂) ⊗ₜ[ℝ] ⁅a₁, a₂⁆
```

lemma dualPairEquiv_map_left [source]

```
lemma dualPairEquiv_map_left (Φ : B →ₗ[ℝ] B) (t : B ⊗[ℝ] 𝔤)
    (φ : Module.Dual ℝ 𝔤) :
    dualPairEquiv ((TensorProduct.map Φ LinearMap.id) t) φ = Φ (dualPairEquiv t φ)
```

lemma dualPairEquiv_map_right [source]

```
lemma dualPairEquiv_map_right (T : 𝔤 →ₗ[ℝ] 𝔤)
    (t : B ⊗[ℝ] 𝔤) (φ : Module.Dual ℝ 𝔤) :
    dualPairEquiv ((TensorProduct.map LinearMap.id T) t) φ =
      dualPairEquiv t (T.dualMap φ)
```

lemma symm_comp_left [source]

```
lemma symm_comp_left (Φ : B →ₗ[ℝ] B) (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    dualPairEquiv.symm (Φ ∘ₗ f) =
      (TensorProduct.map Φ LinearMap.id) (dualPairEquiv.symm f)
```

lemma symm_comp_right [source]

```
lemma symm_comp_right (T : 𝔤 →ₗ[ℝ] 𝔤)
    (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    dualPairEquiv.symm (f ∘ₗ T.dualMap) =
      (TensorProduct.map LinearMap.id T) (dualPairEquiv.symm f)
```

lemma tensorBracket_map_left [source]

```
lemma tensorBracket_map_left (Φ : B →ₗ[ℝ] B)
    (hΦ : ∀ b₁ b₂, Φ (b₁ * b₂) = Φ b₁ * Φ b₂) (s t : B ⊗[ℝ] 𝔤) :
```

```
    tensorBracket ((TensorProduct.map Φ LinearMap.id) s)
        ((TensorProduct.map Φ LinearMap.id) t) =
      (TensorProduct.map Φ LinearMap.id) (tensorBracket s t)
```

lemma tensorBracket_map_right [source]

```
lemma tensorBracket_map_right (T : 𝔤 →ₗ[ℝ] 𝔤)
    (hT : ∀ a b, T ⁅a, b⁆ = ⁅T a, T b⁆) (s t : B ⊗[ℝ] 𝔤) :
    tensorBracket ((TensorProduct.map LinearMap.id T) s)
        ((TensorProduct.map LinearMap.id T) t) =
      (TensorProduct.map LinearMap.id T) (tensorBracket s t)
```

lemma tensorBracket_one_right [source]

```
lemma tensorBracket_one_right (c : 𝔤) (s : B ⊗[ℝ] 𝔤) :
    tensorBracket s ((1 : B) ⊗ₜ[ℝ] c) =
      -(TensorProduct.map LinearMap.id (LieAlgebra.ad ℝ 𝔤 c)) s
```

lemma tensorBracket_one_left [source]

```
lemma tensorBracket_one_left (c : 𝔤) (t : B ⊗[ℝ] 𝔤) :
    tensorBracket ((1 : B) ⊗ₜ[ℝ] c) t =
      (TensorProduct.map LinearMap.id (LieAlgebra.ad ℝ 𝔤 c)) t
```

lemma dualPairEquiv_one_tmul [source]

```
lemma dualPairEquiv_one_tmul (c : 𝔤) (φ : Module.Dual ℝ 𝔤) :
    dualPairEquiv ((1 : B) ⊗ₜ[ℝ] c) φ = algebraMap ℂ B (φ c)
```

lemma repGauge_commutator [source]

The gauge transformation law of the commutator term: writing the field law as `A_μ ↦ Ad₀ A_μ + c_μ` with `Ad₀` the base-point adjoint of `U₀⁻¹` and `c_μ = maurerCartan(U⁻¹)_μ|₀` the constant Maurer–Cartan shift, bilinearity of the bracket gives

`⁅A_μ, A_ν⁆ ↦ Ad₀ ⁅A_μ, A_ν⁆ + ⁅Ad₀ A_μ, c_ν⁆ + ⁅c_μ, Ad₀ A_ν⁆ + ⁅c_μ, c_ν⁆`:

the adjoint-transported commutator, two cross terms linear in the field (the bracket against `c` acting on the dual index through `ad`), and the constant commutator of the two Maurer–Cartan shifts. Uses that the gauge action is by algebra homomorphisms (`gauge_mul`) and that the base-point adjoint transport is a morphism of Lie algebras.

```
lemma repGauge_commutator
    (U : GJ) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repGauge U (commutator h.A μ ν φ) =
      commutator h.A μ ν (jets.adjointDualCoeff U⁻¹ 0 φ)
      - h.A 0 μ (jets.adjointDualCoeff U⁻¹ 0 (φ ∘ₗ LieAlgebra.ad ℝ 𝔤
          (jets.evalLie (jets.maurerCartan U⁻¹ ν))))
      + h.A 0 ν (jets.adjointDualCoeff U⁻¹ 0 (φ ∘ₗ LieAlgebra.ad ℝ 𝔤
          (jets.evalLie (jets.maurerCartan U⁻¹ μ))))
      + algebraMap ℂ B (φ ⁅jets.evalLie (jets.maurerCartan U⁻¹ μ),
          jets.evalLie (jets.maurerCartan U⁻¹ ν)⁆)
```

lemma repGauge_deriv_deriv_apply [source]

The gauge transformation of the twice-derived symbol `∂_ρ ∂_σ A_τ`: the case `s = ρ ::ₘ {σ}` of `gauge_apply_deriv` — the four Leibniz splittings of two derivatives, plus the base-point value of the twice-derived Maurer–Cartan form.

```
lemma repGauge_deriv_deriv_apply
    (U : GJ) (ρ σ τ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repGauge U (h.A (ρ ::ₘ {σ}) τ φ) =
      h.A (ρ ::ₘ {σ}) τ (jets.adjointDualCoeff U⁻¹ 0 φ)
      + h.A {ρ} τ (jets.adjointDualCoeff U⁻¹ {σ} φ)
      + h.A {σ} τ (jets.adjointDualCoeff U⁻¹ {ρ} φ)
      + h.A 0 τ (jets.adjointDualCoeff U⁻¹ (ρ ::ₘ {σ}) φ)
      + algebraMap ℂ B (φ (jets.evalLie (jets.deriv ρ
          (jets.deriv σ (jets.maurerCartan U⁻¹ τ)))))
```

def bracketFam [source]

The bracket of two arbitrary component families, generalizing `commutator` (which is the case of two field symbols): assemble into `B ⊗ 𝔤` by `dualPairEquiv.symm`, bracket by `tensorBracket`, read back out as components.

```
noncomputable def bracketFam (f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] B
```

lemma commutator_eq_bracketFam [source]

```
lemma commutator_eq_bracketFam
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (μ ν : Fin 1 ⊕ Fin 3) : commutator A μ ν = bracketFam (A 0 μ) (A 0 ν)
```

def commutatorFam [source]

***The derived commutator family**: the `s`-derivative of the commutator term, given by the Leibniz convolution of the derivative symbols over the multiset antidiagonal. With the derivative symbols as primitives this convolution is the definition; for `s = 0` it is the commutator itself (`commutatorFam_zero`).*

```
noncomputable def commutatorFam
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] B
```

lemma commutatorFam_zero [source]

```
lemma commutatorFam_zero
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (μ ν : Fin 1 ⊕ Fin 3) : commutatorFam A μ ν 0 = commutator A μ ν
```

lemma bracketFam_add_left [source]

```
lemma bracketFam_add_left (f₁ f₂ g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam (f₁ + f₂) g = bracketFam f₁ g + bracketFam f₂ g
```

lemma bracketFam_add_right [source]

```
lemma bracketFam_add_right (f g₁ g₂ : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam f (g₁ + g₂) = bracketFam f g₁ + bracketFam f g₂
```

lemma bracketFam_smul_left [source]

```
lemma bracketFam_smul_left (c : ℝ) (f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam (c • f) g = c • bracketFam f g
```

lemma bracketFam_smul_right [source]

```
lemma bracketFam_smul_right (c : ℝ) (f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam f (c • g) = c • bracketFam f g
```

lemma bracketFam_apply_eq_sum [source]

The bracket of two component families expanded through a basis of the gauge algebra: the physicists' `f^a_{bc} f^b g^c`, with `φ⁅e_j, e_k⁆` the structure constants contracted with the dual vector.

```
lemma bracketFam_apply_eq_sum (f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (φ : Module.Dual ℝ 𝔤) :
    bracketFam f g φ = ∑ j, ∑ k,
      φ ⁅Module.Free.chooseBasis ℝ 𝔤 j,
          Module.Free.chooseBasis ℝ 𝔤 k⁆ •
        (f ((Module.Free.chooseBasis ℝ 𝔤).coord j) *
          g ((Module.Free.chooseBasis ℝ 𝔤).coord k))
```

lemma bracketFam_apply_eq_zero_of_map_lie [source]

A dual vector vanishing on every bracket of the gauge algebra, such as the coordinate of an abelian factor, vanishes on the bracket of any two families: every structure constant `φ⁅e_j, e_k⁆` it sees is zero.

```
lemma bracketFam_apply_eq_zero_of_map_lie {φ : Module.Dual ℝ 𝔤}
    (hφ : ∀ x y : 𝔤, φ ⁅x, y⁆ = 0) (f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam f g φ = 0
```

lemma bracketFam_map [source]

The bracket of families is natural in the algebra: a multiplicative linear map carries the bracket of two families to the bracket of their images.

```
lemma bracketFam_map {B' : Type} [Ring B'] [Module ℝ B'] [SMulCommClass ℝ B' B']
    [IsScalarTower ℝ B' B'] (Φ : B →ₗ[ℝ] B') (hΦ : ∀ b₁ b₂, Φ (b₁ * b₂) = Φ b₁ * Φ b₂)
    (f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam (Φ ∘ₗ f) (Φ ∘ₗ g) = Φ ∘ₗ bracketFam f g
```

lemma commutatorFam_map [source]

The derived commutator family is natural in the algebra.

```
lemma commutatorFam_map {B' : Type} [Ring B'] [Module ℝ B'] [SMulCommClass ℝ B' B']
    [IsScalarTower ℝ B' B'] (Φ : B →ₗ[ℝ] B') (hΦ : ∀ b₁ b₂, Φ (b₁ * b₂) = Φ b₁ * Φ b₂)
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    commutatorFam (fun p ρ => Φ ∘ₗ A p ρ) μ ν s = Φ ∘ₗ commutatorFam A μ ν s
```

lemma bracketFam_comp_dualMap [source]

The bracket of families against a common Lie-algebra morphism on the dual index.

```
lemma bracketFam_comp_dualMap (T : 𝔤 →ₗ[ℝ] 𝔤)
    (hT : ∀ a b, T ⁅a, b⁆ = ⁅T a, T b⁆) (f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam (f ∘ₗ T.dualMap) (g ∘ₗ T.dualMap) = bracketFam f g ∘ₗ T.dualMap
```

lemma tensorBracket_map_left_derivation [source]

`tensorBracket` is a derivation in the algebra factor: for `Δ` satisfying the Leibniz rule on `B`, applying `Δ ⊗ id` to a bracket distributes over the two arguments.

```
lemma tensorBracket_map_left_derivation (Δ : B →ₗ[ℝ] B)
    (hΔ : ∀ b₁ b₂, Δ (b₁ * b₂) = Δ b₁ * b₂ + b₁ * Δ b₂) (s t : B ⊗[ℝ] 𝔤) :
    (TensorProduct.map Δ LinearMap.id) (tensorBracket s t) =
      tensorBracket ((TensorProduct.map Δ LinearMap.id) s) t +
      tensorBracket s ((TensorProduct.map Δ LinearMap.id) t)
```

lemma bracketFam_derivation [source]

The bracket of families under a derivation of the algebra: the Leibniz rule, in family form.

```
lemma bracketFam_derivation (Δ : B →ₗ[ℝ] B)
    (hΔ : ∀ b₁ b₂, Δ (b₁ * b₂) = Δ b₁ * b₂ + b₁ * Δ b₂) (f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    Δ ∘ₗ bracketFam f g = bracketFam (Δ ∘ₗ f) g + bracketFam f (Δ ∘ₗ g)
```

lemma tensorBracket_map_right_derivation [source]

`tensorBracket` under a relative derivation on the Lie factor: if `T₁ ⁅a, b⁆ = ⁅T₁ a, T₀ b⁆ + ⁅T₀ a, T₁ b⁆`, the two mixed brackets sum to the `T₁`-image of the bracket. This is how the once-derived adjoint transport distributes over the commutator.

```
lemma tensorBracket_map_right_derivation (T₀ T₁ : 𝔤 →ₗ[ℝ] 𝔤)
    (hT : ∀ a b, T₁ ⁅a, b⁆ = ⁅T₁ a, T₀ b⁆ + ⁅T₀ a, T₁ b⁆) (s t : B ⊗[ℝ] 𝔤) :
    tensorBracket ((TensorProduct.map LinearMap.id T₁) s)
        ((TensorProduct.map LinearMap.id T₀) t) +
      tensorBracket ((TensorProduct.map LinearMap.id T₀) s)
        ((TensorProduct.map LinearMap.id T₁) t) =
      (TensorProduct.map LinearMap.id T₁) (tensorBracket s t)
```

lemma bracketFam_dualMap_derivation [source]

The family-level form of `tensorBracket_map_right_derivation`: a relative derivation on the dual index distributes over the bracket of families.

```
lemma bracketFam_dualMap_derivation (T₀ T₁ : 𝔤 →ₗ[ℝ] 𝔤)
    (hT : ∀ a b, T₁ ⁅a, b⁆ = ⁅T₁ a, T₀ b⁆ + ⁅T₀ a, T₁ b⁆)
    (f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam (f ∘ₗ T₁.dualMap) (g ∘ₗ T₀.dualMap) +
      bracketFam (f ∘ₗ T₀.dualMap) (g ∘ₗ T₁.dualMap) =
      bracketFam f g ∘ₗ T₁.dualMap
```

lemma repGauge_bracketFam [source]

The gauge transformation of the bracket of two component families with affine transformation laws `f ↦ f' + φ(c_f)·1` and `g ↦ g' + φ(c_g)·1`: the bracket of the transformed families, two `ad` cross terms, and the constant bracket `⁅c_f, c_g⁆`. Pure bilinearity, with `tensorBracket_one_left/right` computing the cross terms; `repGauge_commutator` is the special case of two field symbols.

```
lemma repGauge_bracketFam
    (U : GJ) {f g f' g' : Module.Dual ℝ 𝔤 →ₗ[ℝ] B}
    {cf cg : 𝔤}
    (hf : ∀ ψ : Module.Dual ℝ 𝔤,
      repGauge U (f ψ) = f' ψ + algebraMap ℂ B (ψ cf))
    (hg : ∀ ψ : Module.Dual ℝ 𝔤,
      repGauge U (g ψ) = g' ψ + algebraMap ℂ B (ψ cg))
    (φ : Module.Dual ℝ 𝔤) :
    repGauge U (bracketFam f g φ) =
      bracketFam f' g' φ
      + g' (φ ∘ₗ LieAlgebra.ad ℝ 𝔤 cf)
      - f' (φ ∘ₗ LieAlgebra.ad ℝ 𝔤 cg)
      + algebraMap ℂ B (φ ⁅cf, cg⁆)
```

lemma commutatorFam_mem [source]

Every derived commutator term is a polynomial in derivative symbols of order at most that of the derivative: each Leibniz splitting contributes a product of two lower-order symbols.

```
lemma commutatorFam_mem
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (s' : Multiset (Fin 1 ⊕ Fin 3)) (ν lam : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    commutatorFam A ν lam s' φ ∈
      Algebra.adjoin ℂ {b : B | ∃ (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
        (φ : Module.Dual ℝ 𝔤), p.card ≤ s'.card ∧ b = A p μ φ}
```

lemma bracketFam_zero_left [source]

```
lemma bracketFam_zero_left (g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam 0 g = 0
```

lemma bracketFam_zero_right [source]

```
lemma bracketFam_zero_right (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam f 0 = 0
```

lemma bracketFam_sum_left [source]

```
lemma bracketFam_sum_left (S : Multiset (Module.Dual ℝ 𝔤 →ₗ[ℝ] B))
    (g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam S.sum g = (S.map fun f => bracketFam f g).sum
```

lemma bracketFam_sum_right [source]

```
lemma bracketFam_sum_right (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (S : Multiset (Module.Dual ℝ 𝔤 →ₗ[ℝ] B)) :
    bracketFam f S.sum = (S.map fun g => bracketFam f g).sum
```

lemma bracketFam_finset_sum_left [source]

```
lemma bracketFam_finset_sum_left {ι : Type*} (S : Finset ι)
    (f : ι → Module.Dual ℝ 𝔤 →ₗ[ℝ] B) (g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam (∑ i ∈ S, f i) g = ∑ i ∈ S, bracketFam (f i) g
```

lemma bracketFam_finset_sum_right [source]

```
lemma bracketFam_finset_sum_right {ι : Type*} (S : Finset ι)
    (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) (g : ι → Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    bracketFam f (∑ i ∈ S, g i) = ∑ i ∈ S, bracketFam f (g i)
```

lemma tensorBracket_map_right_antidiagonal [source]

`tensorBracket` under an antidiagonal family of transports on the Lie factor: if `T x` distributes over the bracket as the antidiagonal convolution of the `T m`, so does `id ⊗ T x` over `tensorBracket`.

```
lemma tensorBracket_map_right_antidiagonal
    (T : Multiset (Fin 1 ⊕ Fin 3) → 𝔤 →ₗ[ℝ] 𝔤)
    (x : Multiset (Fin 1 ⊕ Fin 3))
    (hT : ∀ a b : 𝔤, T x ⁅a, b⁆ =
      (x.antidiagonal.map fun p => ⁅T p.1 a, T p.2 b⁆).sum)
    (s t : B ⊗[ℝ] 𝔤) :
    (x.antidiagonal.map fun p =>
      tensorBracket ((TensorProduct.map LinearMap.id (T p.1)) s)
        ((TensorProduct.map LinearMap.id (T p.2)) t)).sum =
      (TensorProduct.map LinearMap.id (T x)) (tensorBracket s t)
```

lemma bracketFam_adjointDualCoeff [source]

The bracket of families against an iterated dual adjoint coefficient: the antidiagonal convolution — the all-orders form of `bracketFam_comp_dualMap` and `bracketFam_dualMap_derivation`.

```
lemma bracketFam_adjointDualCoeff (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3))
    (f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) (φ : Module.Dual ℝ 𝔤) :
    bracketFam f g (jets.adjointDualCoeff U x φ) =
      (x.antidiagonal.map fun p =>
        bracketFam (f ∘ₗ jets.adjointDualCoeff U p.1)
          (g ∘ₗ jets.adjointDualCoeff U p.2) φ).sum
```

lemma repGauge_cons_apply [source]

The `κ ::ₘ s` case of `gauge_apply_deriv` with the extra derivative traced through: the Leibniz splittings where `κ` stays a derivative, minus (by `LocalGaugeData.adjointDualCoeff_cons`) the splittings where `κ` hits the adjoint — an `ad` of the derived Maurer–Cartan form — plus the derived Maurer–Cartan shift.

```
lemma repGauge_cons_apply
    (U : GJ) (κ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (τ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repGauge U (h.A (κ ::ₘ s) τ φ) =
      (s.antidiagonal.map fun p =>
        h.A (κ ::ₘ p.2) τ (jets.adjointDualCoeff U⁻¹ p.1 φ)).sum
      - (s.antidiagonal.map fun p =>
          (p.1.antidiagonal.map fun q =>
            h.A p.2 τ (jets.adjointDualCoeff U⁻¹ q.2
              (φ ∘ₗ LieAlgebra.ad ℝ 𝔤 (jets.evalLie
                (jets.iteratedDeriv q.1 (jets.maurerCartan U⁻¹ κ)))))).sum).sum
      + algebraMap ℂ B (φ (jets.evalLie (jets.iteratedDeriv (κ ::ₘ s)
          (jets.maurerCartan U⁻¹ τ))))
```

lemma repGauge_commutatorFam [source]

The all-orders gauge transformation of the derived commutator term: the Leibniz convolution of the transformed commutator, the two `ad` cross-term convolutions, and the convolution of Maurer–Cartan bracket shifts. This is `repGauge_commutator` at every derivative order simultaneously; the regrouping of the four-fold splitting is `Multiset.sum_antidiagonal_exchange`.

```
lemma repGauge_commutatorFam
    (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repGauge U (commutatorFam h.A μ ν s φ) =
      (s.antidiagonal.map fun p =>
        commutatorFam h.A μ ν p.2 (jets.adjointDualCoeff U⁻¹ p.1 φ)).sum
      + (s.antidiagonal.map fun p =>
          (p.2.antidiagonal.map fun r =>
            h.A r.2 ν (jets.adjointDualCoeff U⁻¹ r.1
              (φ ∘ₗ LieAlgebra.ad ℝ 𝔤 (jets.evalLie
                (jets.iteratedDeriv p.1 (jets.maurerCartan U⁻¹ μ)))))).sum).sum
      - (s.antidiagonal.map fun p =>
          (p.1.antidiagonal.map fun q =>
            h.A q.2 μ (jets.adjointDualCoeff U⁻¹ q.1
              (φ ∘ₗ LieAlgebra.ad ℝ 𝔤 (jets.evalLie
                (jets.iteratedDeriv p.2 (jets.maurerCartan U⁻¹ ν)))))).sum).sum
      + (s.antidiagonal.map fun p =>
          algebraMap ℂ B (φ ⁅jets.evalLie (jets.iteratedDeriv p.1
            (jets.maurerCartan U⁻¹ μ)),
            jets.evalLie (jets.iteratedDeriv p.2
              (jets.maurerCartan U⁻¹ ν))⁆)).sum
```

2.189 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Realization.Symmetrized

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/Realization/Symmetrized.lean on GitHub.

def symmetrizedDeriv [source]

The symmetrized derivative symbol `sym(∂_s A)^φ`: the average over the directions `μ ∈ s` of the symbols `∂_{s-μ} A_μ^φ`, so that the direction of the gauge field is symmetrized into the derivative multiset.

```
noncomputable def symmetrizedDeriv (s : Multiset (Fin 1 ⊕ Fin 3))
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (φ : Module.Dual ℝ 𝔤) : B
```

lemma symmetrizedDeriv_singleton [source]

```
@[simp]
lemma symmetrizedDeriv_singleton (μ : Fin 1 ⊕ Fin 3)
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (φ : Module.Dual ℝ 𝔤) : symmetrizedDeriv ({μ}) A φ = A 0 μ φ
```

lemma symmetrizedDeriv_empty [source]

```
@[simp]
lemma symmetrizedDeriv_empty
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (φ : Module.Dual ℝ 𝔤) : symmetrizedDeriv 0 A φ = 0
```

lemma deriv_sub_symmetrizedDeriv_eq_sum [source]

The symmetrization defect of a symbol: the average of its differences with the symbols in which one derivative direction has been exchanged with the field direction.

```
lemma deriv_sub_symmetrizedDeriv_eq_sum (s : Multiset (Fin 1 ⊕ Fin 3))
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (φ : Module.Dual ℝ 𝔤) (μ : Fin 1 ⊕ Fin 3) :
    A s μ φ - symmetrizedDeriv (μ ::ₘ s) A φ =
      ((1/(s.card + 1 : ℝ)) • ((s.map fun ν => A s μ φ - A (μ ::ₘ s - {ν}) ν φ).sum))
```

def symmetrizedDeriv$_l$ [source]

The symmetrized symbols are linear in the dual index.

```
noncomputable def symmetrizedDeriv_l (s : Multiset (Fin 1 ⊕ Fin 3))
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] B where
```

lemma symmetrizedDeriv_eq_sum_coord [source]

The expansion of a symmetrized symbol in a basis of the gauge algebra.

```
lemma symmetrizedDeriv_eq_sum_coord {ι : Type} [Fintype ι] (bv : Module.Basis ι ℝ 𝔤)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    symmetrizedDeriv s A φ = ∑ j, φ (bv j) • symmetrizedDeriv s A (bv.coord j)
```

def iteratedCovDerivAdjoint [source]

*The iterated covariant derivative `𝒟_l F` of an adjoint family of derivative symbols along a *list* of directions: covariant derivatives do not commute (their commutator is an `ad F` term), so the iteration is order-dependent and indexed by a list. The result*

is again a family of derivative symbols; the underived covariant tower is its value at the empty multiset.

```
noncomputable def iteratedCovDerivAdjoint
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    List (Fin 1 ⊕ Fin 3) →
      (Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B) →
      Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B
  | [], F => F
  | ρ :: l, F => covDerivAdjoint A (iteratedCovDerivAdjoint A l F) ρ
```

lemma iteratedCovDerivAdjoint_map [source]

The iterated covariant derivative is natural in the algebra.

```
lemma iteratedCovDerivAdjoint_map {B' : Type} [Ring B'] [Module ℝ B'] [SMulCommClass ℝ B' B']
    [IsScalarTower ℝ B' B'] (Φ : B →ₗ[ℝ] B') (hΦ : ∀ b₁ b₂, Φ (b₁ * b₂) = Φ b₁ * Φ b₂)
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (l : List (Fin 1 ⊕ Fin 3)) (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    iteratedCovDerivAdjoint (fun p σ => Φ ∘ₗ A p σ) l (fun p => Φ ∘ₗ F p) =
      fun p => Φ ∘ₗ iteratedCovDerivAdjoint A l F p
```

lemma iteratedCovDerivAdjoint_fieldStrength_map [source]

The covariant tower of the field strength is natural in the algebra: the tower of the image family is the image of the tower.

```
lemma iteratedCovDerivAdjoint_fieldStrength_map {B' : Type} [Ring B'] [Module ℝ B']
    [SMulCommClass ℝ B' B'] [IsScalarTower ℝ B' B'] (Φ : B →ₗ[ℝ] B')
    (hΦ : ∀ b₁ b₂, Φ (b₁ * b₂) = Φ b₁ * Φ b₂)
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    iteratedCovDerivAdjoint (fun p σ => Φ ∘ₗ A p σ) l
        (fieldStrength (fun p σ => Φ ∘ₗ A p σ) μ ν) 0 φ
      = Φ (iteratedCovDerivAdjoint A l (fieldStrength A μ ν) 0 φ)
```

lemma iteratedCovDerivAdjoint_apply_of_map_lie [source]

At a dual vector vanishing on every bracket, such as the coordinate of an abelian factor, the iterated covariant derivative is the ordinary derivative along the multiset of its directions: no bracket correction survives at any order.

```
lemma iteratedCovDerivAdjoint_apply_of_map_lie {φ : Module.Dual ℝ 𝔤}
    (hφ : ∀ x y : 𝔤, φ ⁅x, y⁆ = 0)
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (l : List (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    iteratedCovDerivAdjoint A l F s φ = F (↑l + s) φ
```

lemma iteratedCovDerivAdjoint_apply_perm_of_map_lie [source]

Hence at such a dual vector the covariant derivatives commute: the iterated covariant derivative depends only on the multiset of its directions.

```
lemma iteratedCovDerivAdjoint_apply_perm_of_map_lie {φ : Module.Dual ℝ 𝔤}
    (hφ : ∀ x y : 𝔤, φ ⁅x, y⁆ = 0)
```

```
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    {l l' : List (Fin 1 ⊕ Fin 3)} (hl : l.Perm l') (s : Multiset (Fin 1 ⊕ Fin 3)) :
    iteratedCovDerivAdjoint A l F s φ = iteratedCovDerivAdjoint A l' F s φ
```

abbrev symbolsLE [source]

The derivative symbols `∂_p A_μ^φ` with at most `n` derivatives.

```
abbrev symbolsLE (n : ℕ) : Set B
```

abbrev symbols [source]

All derivative symbols `∂_p A_μ^φ`.

```
abbrev symbols : Set B
```

abbrev symSymbolsLE [source]

The symmetrized symbols `sym(∂_r A)^φ` with at least one and at most `n` derivatives.

```
abbrev symSymbolsLE (n : ℕ) : Set B
```

abbrev towerLT [source]

The covariant derivatives `𝒟_l F_{ν λ}^φ` of the field strength with fewer than `n` derivatives.

```
abbrev towerLT (n : ℕ) : Set B
```

abbrev tower [source]

All covariant derivatives `𝒟_l F_{ν λ}^φ` of the field strength.

```
abbrev tower : Set B
```

lemma symbolsLE_mono [source]

```
lemma symbolsLE_mono {n m : ℕ} (hnm : n ≤ m) : symbolsLE A n ⊆ symbolsLE A m
```

lemma symbolsLE_subset_symbols [source]

```
lemma symbolsLE_subset_symbols (n : ℕ) : symbolsLE A n ⊆ symbols A
```

lemma symSymbolsLE_mono [source]

```
lemma symSymbolsLE_mono {n m : ℕ} (hnm : n ≤ m) : symSymbolsLE A n ⊆ symSymbolsLE A m
```

lemma towerLT_mono [source]

```
lemma towerLT_mono {n m : ℕ} (hnm : n ≤ m) : towerLT A n ⊆ towerLT A m
```

lemma towerLT_subset_tower [source]

```
lemma towerLT_subset_tower (n : ℕ) : towerLT A n ≤ tower A
```

lemma adjoin_symbols_mono [source]

Symbol subalgebras are monotone in the order bound.

```
lemma adjoin_symbols_mono {n m : ℕ} (hnm : n ≤ m) :
    Algebra.adjoin ℂ (symbolsLE A n) ≤ Algebra.adjoin ℂ (symbolsLE A m)
```

lemma symmetrizedDeriv_mem_adjoin_symbolsLE [source]

A symmetrized symbol with `r` derivatives is a polynomial in the symbols with fewer than `r` derivatives.

```
lemma symmetrizedDeriv_mem_adjoin_symbolsLE (r : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℝ 𝔤) :
    symmetrizedDeriv r A φ ∈ Algebra.adjoin ℂ (symbolsLE A (r.card - 1))
```

lemma bracketFam_mem_adjoin_symbols [source]

The bracket of two component families whose components are order-`n` symbol polynomials is again an order-`n` symbol polynomial, componentwise.

```
lemma bracketFam_mem_adjoin_symbols {n : ℕ} {f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B}
    (hf : ∀ ψ, f ψ ∈ Algebra.adjoin ℂ (symbolsLE A n))
    (hg : ∀ ψ, g ψ ∈ Algebra.adjoin ℂ (symbolsLE A n)) (φ : Module.Dual ℝ 𝔤) :
    bracketFam f g φ ∈ Algebra.adjoin ℂ (symbolsLE A n)
```

lemma fieldStrength_mem_adjoin_symbols [source]

Every derivative symbol of the field strength is a symbol polynomial of order one higher than the number of derivatives.

```
lemma fieldStrength_mem_adjoin_symbols (q : Multiset (Fin 1 ⊕ Fin 3)) (ν lam : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    fieldStrength A ν lam q φ ∈ Algebra.adjoin ℂ (symbolsLE A (q.card + 1))
```

lemma iteratedCovDerivAdjoint_sub_mem [source]

Unitriangularity of the covariant tower: the covariant and plain derivative symbols of the field strength differ by a polynomial in lower-order symbols. Stated at every derivative multiset `s`, as needed for the induction: the covariant derivative shifts the family index.

```
lemma iteratedCovDerivAdjoint_sub_mem (l : List (Fin 1 ⊕ Fin 3)) (ν lam : Fin 1 ⊕ Fin 3)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    iteratedCovDerivAdjoint A l (fieldStrength A ν lam) s φ -
        fieldStrength A ν lam (Multiset.ofList l + s) φ ∈
      Algebra.adjoin ℂ (symbolsLE A (l.length + s.card))
```

lemma iteratedCovDerivAdjoint_fieldStrength_mem_adjoin_symbols [source]

The underived covariant field-strength tower consists of polynomials in the gauge-field symbols.

```
lemma iteratedCovDerivAdjoint_fieldStrength_mem_adjoin_symbols
    (l : List (Fin 1 ⊕ Fin 3)) (ν lam : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    iteratedCovDerivAdjoint A l (fieldStrength A ν lam) 0 φ ∈ Algebra.adjoin ℂ (symbols A)
```

lemma symbol_mem_symFieldAdjoin_sup [source]

The inductive step of the generation theorem: every derivative symbol of order `n + 1` lies in the subalgebra generated by its symmetrization, the covariant derivatives of the field strength of order `n`, and the symbols of order at most `n`.

```
lemma symbol_mem_symFieldAdjoin_sup (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    A s μ φ ∈ Algebra.adjoin ℂ (symSymbolsLE A (s.card + 1) ∪ towerLT A s.card) ⊔
      Algebra.adjoin ℂ (symbolsLE A (s.card - 1))
```

lemma symbolAdjoin_eq_symFieldAdjoin [source]

The generation theorem, by strong induction on the order: the derivative symbols of order at most `n` and the symmetrized symbols together with the covariant field-strength tower generate the same subalgebra,

`adjoin({ ∂_p A : |p| ≤ n }) = adjoin({ sym(∂_p A) : |p| ≤ n + 1 } ∪ { 𝒟_q F : |q| < n }).`

```
lemma symbolAdjoin_eq_symFieldAdjoin (n : ℕ) :
    Algebra.adjoin ℂ (symbolsLE A n) =
      Algebra.adjoin ℂ (symSymbolsLE A (n + 1) ∪ towerLT A n)
```

lemma symbolAdjoin_eq_symFieldAdjoin_top [source]

The generation theorem, unbounded version: the derivative symbols of all orders, and the symmetrized symbols together with the full covariant field-strength tower, generate the same subalgebra of local expressions,

`adjoin({ ∂_p A }) = adjoin({ sym(∂_p A) } ∪ { 𝒟_q F }).`

```
lemma symbolAdjoin_eq_symFieldAdjoin_top :
    Algebra.adjoin ℂ (symbols A) =
      Algebra.adjoin ℂ ({b | ∃ (r : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤),
        r ≠ 0 ∧ b = symmetrizedDeriv r A φ} ∪ tower A)
```

lemma symbolAdjoin_union_eq_symFieldAdjoin_union [source]

The generation theorem relativized to an arbitrary set `S` of extra generators.

```
lemma symbolAdjoin_union_eq_symFieldAdjoin_union (n : ℕ) (S : Set B) :
    Algebra.adjoin ℂ (symbolsLE A n ∪ S) =
      Algebra.adjoin ℂ ((symSymbolsLE A (n + 1) ∪ towerLT A n) ∪ S)
```

lemma exists_le_of_mem_adjoin_symbols_union [source]

Finite order bound: membership in the subalgebra generated by all symbols and `S` uses only finitely many generators, hence symbols of some bounded order.

```
lemma exists_le_of_mem_adjoin_symbols_union (S : Set B) {x : B}
    (hx : x ∈ Algebra.adjoin ℂ (symbols A ∪ S)) :
    ∃ n : ℕ, x ∈ Algebra.adjoin ℂ (symbolsLE A n ∪ S)
```

lemma repGauge_symmetrizedDeriv [source]

The gauge transformation of the symmetrized derivatives: averaging the transformation law `gauge_apply_deriv` of the individual derivative symbols over the multiset `s`, the homogeneous part is the symmetrized adjoint convolution and the inhomogeneous Maurer–Cartan shifts average to exactly the base-point value of the symmetrized Maurer–Cartan form of `U⁻¹`:

`U • sym(∂_s h.A)^φ = (1/|s|) ∑_{μ ∈ s} ∑_{x+y=s−μ} ∂_y A_μ^{∂_x Ad*(U⁻¹) φ} + φ( sym(ω(U⁻¹))_s |₀ )`.

```
lemma repGauge_symmetrizedDeriv
    (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    repGauge U (symmetrizedDeriv s h.A φ) =
      (1/(s.card : ℝ)) • (s.map fun μ =>
        ((s - {μ}).antidiagonal.map fun p =>
          h.A p.2 μ (jets.adjointDualCoeff U⁻¹ p.1 φ)).sum).sum
      + algebraMap ℂ B (φ (jets.evalLie (jets.symmetrizedMaurerCartanForm U⁻¹ s)))
```

lemma repGauge_symmetrizedDeriv_truncationKer [source]

The action of a pure jet on the symmetrized derivatives is through the symmetrized Maurer–Cartan data: for a gauge jet `U` with identity value, the inhomogeneous shift of `sym(∂_s h.A)^φ` is the pairing of `φ` with the symmetrized Maurer–Cartan coefficient of `U⁻¹` at `s`, the very data that classifies pure jets.

```
lemma repGauge_symmetrizedDeriv_truncationKer
    (U : jets.truncationKer 0) (s : Multiset (Fin 1 ⊕ Fin 3)) (hs : s ≠ 0)
    (φ : Module.Dual ℝ 𝔤) :
    repGauge U.1 (symmetrizedDeriv s h.A φ) =
      (1/(s.card : ℝ)) • (s.map fun μ =>
        ((s - {μ}).antidiagonal.map fun p =>
          h.A p.2 μ (jets.adjointDualCoeff (U.1)⁻¹ p.1 φ)).sum).sum
      + algebraMap ℂ B (φ (jets.symmetrizedMaurerCartanCoeff U⁻¹ ⟨s, hs⟩))
```

lemma exists_repGauge_symmetrizedDeriv_shift [source]

The pure jets realize arbitrary translations of the symmetrized derivative coordinates: for any prescribed family `c` of gauge-algebra values there is a pure jet `U` whose action shifts every symmetrized symbol by exactly `φ (c s)`, by the freeness of the symmetrized Maurer–Cartan data.

```
lemma exists_repGauge_symmetrizedDeriv_shift [jets.Free]
    (c : {r : Multiset (Fin 1 ⊕ Fin 3) // r ≠ 0} → 𝔤) :
    ∃ U : jets.truncationKer 0,
      ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (hs : s ≠ 0) (φ : Module.Dual ℝ 𝔤),
        repGauge U.1 (symmetrizedDeriv s h.A φ) =
```

```
        (1/(s.card : ℝ)) • (s.map fun μ =>
          ((s - {μ}).antidiagonal.map fun p =>
            h.A p.2 μ (jets.adjointDualCoeff (U.1)⁻¹ p.1 φ)).sum).sum
        + algebraMap ℂ B (φ (c ⟨s, hs⟩))
```

lemma repGauge_symmetrizedDeriv_translation [source]

Pure translation: when all positive dual adjoint coefficients of `U⁻¹` below the order of `s` vanish, the adjoint convolution in the transformation of the symmetrized symbol collapses to the symbol itself, and the action is an honest translation by the symmetrized Maurer–Cartan coefficient.

```
lemma repGauge_symmetrizedDeriv_translation
    (U : jets.truncationKer 0) (s : Multiset (Fin 1 ⊕ Fin 3)) (hs : s ≠ 0)
    (hU : ∀ x : Multiset (Fin 1 ⊕ Fin 3), x ≠ 0 → x.card < s.card →
      jets.adjointDualCoeff (U.1)⁻¹ x = 0)
    (φ : Module.Dual ℝ 𝔤) :
    repGauge U.1 (symmetrizedDeriv s h.A φ) =
      symmetrizedDeriv s h.A φ +
        algebraMap ℂ B (φ (jets.symmetrizedMaurerCartanCoeff U⁻¹ ⟨s, hs⟩))
```

lemma exists_translation_of_support [source]

Realization of top-order translations: any coefficient family supported at exactly order `N` is realized by a pure jet trivial to order `N - 1`, by freeness of the symmetrized Maurer–Cartan data together with Maurer–Cartan triangularity.

```
lemma exists_translation_of_support [jets.Free] (N : ℕ)
    (c : {r : Multiset (Fin 1 ⊕ Fin 3) // r ≠ 0} → 𝔤) (hcN : ∀ r, r.1.card ≠ N → c r = 0) :
    ∃ U : jets.truncationKer 0,
      jets.symmetrizedMaurerCartanCoeff U⁻¹ = c ∧
      ∀ x : Multiset (Fin 1 ⊕ Fin 3), x ≠ 0 → x.card < N →
        jets.adjointDualCoeff (U.1)⁻¹ x = 0
```

lemma smul_mem_center [source]

Real multiples of central elements are central.

```
lemma smul_mem_center (r : ℝ) {x : B} (hx : x ∈ Subring.center B) :
    r • x ∈ Subring.center B
```

lemma bracketFam_mem_center [source]

The bracket of component families with central components is central.

```
lemma bracketFam_mem_center {f g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B}
    (hf : ∀ ψ, f ψ ∈ Subring.center B) (hg : ∀ ψ, g ψ ∈ Subring.center B)
    (φ : Module.Dual ℝ 𝔤) : bracketFam f g φ ∈ Subring.center B
```

lemma commutatorFam_mem_center [source]

The derived commutator terms of central symbols are central.

```
lemma commutatorFam_mem_center
    (hc : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
```

```
    A p μ φ ∈ Subring.center B)
  (ν lam : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
  commutatorFam A ν lam s φ ∈ Subring.center B
```

lemma iteratedCovDerivAdjoint_fieldStrength_mem_center [source]

If the derivative symbols of the gauge field are central, so are all derivative symbols of the covariant derivatives of the field strength.

```
lemma iteratedCovDerivAdjoint_fieldStrength_mem_center
  (hc : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
    A p μ φ ∈ Subring.center B)
  (l : List (Fin 1 ⊕ Fin 3)) (ν lam : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
  (φ : Module.Dual ℝ 𝔤) :
  iteratedCovDerivAdjoint A l (fieldStrength A ν lam) s φ ∈ Subring.center B
```

lemma symmetrizedDeriv_mem_center [source]

If the derivative symbols of the gauge field are central, so are the symmetrized symbols.

```
lemma symmetrizedDeriv_mem_center
  (hc : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
    A p μ φ ∈ Subring.center B)
  (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
  symmetrizedDeriv s A φ ∈ Subring.center B
```

lemma repGauge_eq_of_mem_truncationKer_zero [source]

Anything that transforms in the adjoint is invariant under the pure jets: at `s = 0` the transformation law is the dual adjoint action of the base-point value `U₀⁻¹ = 1`.

```
lemma TransformsInAdjoint.repGauge_eq_of_mem_truncationKer_zero
  {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B}
  (hF : TransformsInAdjoint jets repGauge F) (U : jets.truncationKer 0)
  (φ : Module.Dual ℝ 𝔤) : repGauge U.1 (F 0 φ) = F 0 φ
```

lemma transformsInAdjoint_iteratedCovDerivAdjoint [source]

Every iterated covariant derivative of the field strength is an adjoint gauge tensor: the recursion of `TransformsInAdjoint.covDerivAdjoint` over the list of directions, from the base case `transformsInAdjoint_fieldStrength`.

```
lemma transformsInAdjoint_iteratedCovDerivAdjoint (l : List (Fin 1 ⊕ Fin 3))
  (ν lam : Fin 1 ⊕ Fin 3) :
  TransformsInAdjoint jets repGauge
    (iteratedCovDerivAdjoint h.A l (fieldStrength h.A ν lam))
```

lemma repGauge_iteratedCovDerivAdjoint_fieldStrength_of_mem_truncationKer_zero [source]

The covariant derivatives of the field strength are invariant under the pure jets.

```
lemma repGauge_iteratedCovDerivAdjoint_fieldStrength_of_mem_truncationKer_zero
  (U : jets.truncationKer 0)
  (l : List (Fin 1 ⊕ Fin 3)) (ν lam : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
  repGauge U.1 (iteratedCovDerivAdjoint h.A l (fieldStrength h.A ν lam) 0 φ) =
```

```
    iteratedCovDerivAdjoint h.A l (fieldStrength h.A ν lam) 0 φ
```

lemma repGauge_one [source]

The gauge action fixes the unit, being multiplicative and invertible.

```
lemma repGauge_one (U : GJ) :
    repGauge U (1 : B) = 1
```

def repGaugeRingHom [source]

The gauge action of a jet as a ring endomorphism of the algebra of local expressions.

```
def repGaugeRingHom (U : GJ) : B →+* B where
```

lemma repGaugeRingHom_apply [source]

```
@[simp]
lemma repGaugeRingHom_apply (U : GJ) (x : B) :
    repGaugeRingHom h U x = repGauge U x
```

lemma commute_of_mem_adjoin [source]

Commutation with a generating set extends to the generated subalgebra.

```
lemma commute_of_mem_adjoin {X : Set B} {y : B} (hX : ∀ x ∈ X, Commute x y)
    {r : B} (hr : r ∈ Algebra.adjoin ℂ X) : Commute r y
```

lemma commute_symmetrizedDeriv_right [source]

Anything commuting with all gauge-field symbols commutes with the symmetrized symbols.

```
lemma commute_symmetrizedDeriv_right {y : B}
    (hy : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
      Commute y (A p μ φ))
    (r : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    Commute y (symmetrizedDeriv r A φ)
```

lemma commute_symmetrizedDeriv [source]

For a bosonic gauge field the symmetrized symbols commute with each other.

```
lemma commute_symmetrizedDeriv
    (hcomm : ∀ (p q : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
      (φ ψ : Module.Dual ℝ 𝔤), Commute (A p μ φ) (A q ν ψ))
    (r r' : Multiset (Fin 1 ⊕ Fin 3)) (φ φ' : Module.Dual ℝ 𝔤) :
    Commute (symmetrizedDeriv r A φ) (symmetrizedDeriv r' A φ')
```

lemma eq_zero_of_forall_sum_smul_pow_eq_zero [source]

A `B`-valued polynomial function of one real variable that vanishes identically has vanishing coefficients: pair with real-linear functionals, which separate points, and use that a real polynomial vanishing everywhere is zero.

```
lemma eq_zero_of_forall_sum_smul_pow_eq_zero {n : ℕ} {b : ℕ → B}
    (h : ∀ t : ℝ, ∑ m ∈ Finset.range n, t ^ m • b m = 0) :
    ∀ m ∈ Finset.range n, b m = 0
```

lemma exists_polynomial_rep [source]

Any element of the subalgebra generated by a subalgebra `R` and a single element `y` commuting with `R` is a polynomial in `y` with coefficients in `R`: the subalgebra is the image of `R[X]` under evaluation at `y`.

```
lemma exists_polynomial_rep (R : Subalgebra ℂ B) (y : B) (hy : ∀ r ∈ R, Commute r y)
    {x : B} (hx : x ∈ R ⊔ Algebra.adjoin ℂ {y}) :
    ∃ (n : ℕ) (r : ℕ → B), (∀ k, r k ∈ R) ∧ x = ∑ k ∈ Finset.range n, r k * y ^ k
```

lemma map_mul_pow_eq_sum [source]

The binomial expansion of the translate of a single monomial `r * y ^ k` under an endomorphism fixing `r` and shifting `y` by a real scalar `t`.

```
lemma map_mul_pow_eq_sum (Φ : B →+* B) {r y : B} (hr : Φ r = r) {t : ℝ}
    (hy : Φ y = y + algebraMap ℂ B (t : ℂ)) (k : ℕ) :
    Φ (r * y ^ k) =
      ∑ j ∈ Finset.range (k + 1), t ^ j • ((k.choose j : ℂ) • (r * y ^ (k - j)))
```

lemma mem_of_translationInvariant_single [source]

The single-variable extraction: an element of `R[y]` invariant under a family of ring endomorphisms fixing `R` pointwise and translating `y` by arbitrary real scalars lies in `R`. Invariance forces the top coefficient of any chosen polynomial representation to vanish, by expanding the translated polynomial in powers of the shift.

```
lemma mem_of_translationInvariant_single (R : Subalgebra ℂ B) (y : B)
    (hy : ∀ r ∈ R, Commute r y) (Φ : ℝ → B →+* B)
    (hΦR : ∀ t : ℝ, ∀ z ∈ R, Φ t z = z)
    (hΦy : ∀ t : ℝ, Φ t y = y + algebraMap ℂ B (t : ℂ))
    {x : B} (hx : x ∈ R ⊔ Algebra.adjoin ℂ {y}) (hinv : ∀ t, Φ t x = x) : x ∈ R
```

lemma mem_of_translationInvariant [source]

The extraction theorem: if a family of ring endomorphisms fixes a subalgebra `R` pointwise and translates finitely many commuting elements `y i`, each commuting with `R`, by arbitrary prescribable scalars, then an element of the subalgebra generated by `R` and the `y i` that is invariant under the whole family lies in `R`. The variables are eliminated one at a time by `mem_of_translationInvariant_single`.

```
lemma mem_of_translationInvariant {ι : Type} [Fintype ι]
    (R : Subalgebra ℂ B) (y : ι → B)
    (hyR : ∀ i, ∀ r ∈ R, Commute r (y i)) (hyy : ∀ i j, Commute (y i) (y j))
    (Φ : (ι → ℝ) → (B →+* B)) (hΦR : ∀ t, ∀ z ∈ R, Φ t z = z)
    (hΦy : ∀ t i, Φ t (y i) = y i + algebraMap ℂ B (t i))
    {x : B} (hx : x ∈ R ⊔ Algebra.adjoin ℂ (Set.range y)) (hinv : ∀ t, Φ t x = x) :
    x ∈ R
```

lemma repGauge_eq_of_mem_adjoin_symSymbolsLE [source]

h.A pure jet trivial to order `m` fixes every generator of order at most `m`: the symmetrized symbols with at most `m` derivatives, the covariant tower and `S`.

```
lemma repGauge_eq_of_mem_adjoin_symSymbolsLE {U : jets.truncationKer 0}
    (hU : ∀ x : Multiset (Fin 1 ⊕ Fin 3), x ≠ 0 → x.card ≤ m →
      jets.adjointDualCoeff (U.1)⁻¹ x = 0)
    (hUsym : ∀ (r : Multiset (Fin 1 ⊕ Fin 3)) (hr : r ≠ 0), r.card ≤ m →
      jets.symmetrizedMaurerCartanCoeff U⁻¹ ⟨r, hr⟩ = 0)
    {z : B} (hz : z ∈ Algebra.adjoin ℂ (symSymbolsLE h.A m ∪ (tower h.A ∪ S))) :
    repGauge U.1 z = z
```

lemma commute_symmetrizedDeriv_of_mem_adjoin_symSymbolsLE [source]

Every element of the order-`m` subalgebra commutes with every symmetrized symbol.

```
lemma commute_symmetrizedDeriv_of_mem_adjoin_symSymbolsLE
    {z : B} (hz : z ∈ Algebra.adjoin ℂ (symSymbolsLE h.A m ∪ (tower h.A ∪ S)))
    (r : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    Commute z (symmetrizedDeriv r h.A φ)
```

def topCoord [source]

The top-order coordinates: the symmetrized symbols of order `m + 1` in the basis `bv`.

```
noncomputable def topCoord : Sym (Fin 1 ⊕ Fin 3) (m + 1) × ι → B
```

def shiftFamily [source]

The shift family supported at order `m + 1` with coordinates `t` in the basis `bv`.

```
noncomputable def shiftFamily (t : Sym (Fin 1 ⊕ Fin 3) (m + 1) × ι → ℝ) :
    {r : Multiset (Fin 1 ⊕ Fin 3) // r ≠ 0} → 𝔤
```

lemma shiftFamily_eq_zero_of_card_ne [source]

```
lemma shiftFamily_eq_zero_of_card_ne (t : Sym (Fin 1 ⊕ Fin 3) (m + 1) × ι → ℝ)
    (r : {r : Multiset (Fin 1 ⊕ Fin 3) // r ≠ 0}) (hr : r.1.card ≠ m + 1) :
    shiftFamily m bv t r = 0
```

lemma coord_shiftFamily [source]

```
lemma coord_shiftFamily (t : Sym (Fin 1 ⊕ Fin 3) (m + 1) × ι → ℝ)
    (ps : Sym (Fin 1 ⊕ Fin 3) (m + 1)) (hp0 : (ps : Multiset (Fin 1 ⊕ Fin 3)) ≠ 0) (j : ι) :
    bv.coord j (shiftFamily m bv t ⟨ps, hp0⟩) = t (ps, j)
```

lemma commute_topCoord_of_mem_adjoin_symSymbolsLE [source]

The top-order coordinates commute with the order-`m` subalgebra.

```
lemma commute_topCoord_of_mem_adjoin_symSymbolsLE (p : Sym (Fin 1 ⊕ Fin 3) (m + 1) × ι)
    {z : B} (hz : z ∈ Algebra.adjoin ℂ (symSymbolsLE h.A m ∪ (tower h.A ∪ S))) :
    Commute z (topCoord h.A m bv p)
```

lemma mem_sup_adjoin_range_topCoord [source]

An element of the order-`m + 1` subalgebra lies in the sup of the order-`m` subalgebra and the subalgebra generated by the top-order coordinates.

```
lemma mem_sup_adjoin_range_topCoord {z : B}
    (hz : z ∈ Algebra.adjoin ℂ (symSymbolsLE h.A (m + 1) ∪ (tower h.A ∪ S))) :
    z ∈ Algebra.adjoin ℂ (symSymbolsLE h.A m ∪ (tower h.A ∪ S)) ⊔
      Algebra.adjoin ℂ (Set.range (topCoord h.A m bv))
```

lemma repGauge_topCoord [source]

The pure jet realizing the shift family `t` translates each top-order coordinate by the corresponding value of `t`.

```
lemma repGauge_topCoord {U : jets.truncationKer 0}
    (t : Sym (Fin 1 ⊕ Fin 3) (m + 1) × ι → ℝ)
    (hU1 : jets.symmetrizedMaurerCartanCoeff U⁻¹ = shiftFamily m bv t)
    (hU2 : ∀ x : Multiset (Fin 1 ⊕ Fin 3), x ≠ 0 → x.card < m + 1 →
      jets.adjointDualCoeff (U.1)⁻¹ x = 0)
    (p : Sym (Fin 1 ⊕ Fin 3) (m + 1) × ι) :
    repGauge U.1 (topCoord h.A m bv p) = topCoord h.A m bv p + algebraMap ℂ B ((t p : ℝ) : ℂ)
```

lemma mem_adjoin_symSymbolsLE_of_repGauge_eq [source]

The descent: an element of the order-`m + 1` subalgebra fixed by all pure jets lies in the order-`m` subalgebra. The pure jets realizing the shift families at order `m + 1` fix the order-`m` subalgebra and translate the top-order coordinates by arbitrary real scalars, so `mem_of_translationInvariant` applies.

```
lemma mem_adjoin_symSymbolsLE_of_repGauge_eq [jets.Free] {z : B}
    (hz : z ∈ Algebra.adjoin ℂ (symSymbolsLE h.A (m + 1) ∪ (tower h.A ∪ S)))
    (hzinv : ∀ U : jets.truncationKer 0, repGauge U.1 z = z) :
    z ∈ Algebra.adjoin ℂ (symSymbolsLE h.A m ∪ (tower h.A ∪ S))
```

lemma GaugeAlgebraRealization.invariant_mem_adjoin_fieldStrength [source]

The classification of invariants: a gauge-invariant element of the subalgebra generated by the gauge-field symbols and a set `S` of `truncationKer 0`-fixed elements is a polynomial in the covariant derivatives of the field strength and the elements of `S`. Requires only that the gauge-field symbols commute with each other (the gauge field is bosonic) and with the elements of `S`, nothing about the rest of `B`, and no independence hypothesis.

```
lemma invariant_mem_adjoin_fieldStrength [jets.Free] (S : Set B)
    (hcS : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
      (φ : Module.Dual ℝ 𝔤), ∀ y ∈ S, Commute y (h.A p μ φ))
    (hS : ∀ y ∈ S, ∀ U : jets.truncationKer 0, repGauge U.1 y = y)
    {x : B} (hx : x ∈ Algebra.adjoin ℂ (symbols h.A ∪ S))
    (hinv : ∀ U : GJ, repGauge U x = x) :
    x ∈ Algebra.adjoin ℂ (tower h.A ∪ S)
```

lemma LocalGaugeFieldAlgebra.invariant_mem_adjoin_fieldStrength [source]

The classification of gauge invariants of the gauge-boson jet algebra: for a free package, a gauge-invariant element of the subalgebra generated by the gauge-field symbols `∂_s A_μ^φ` and a set `S` of elements fixed by the pure jets is a polynomial in the covariant derivatives of the field strength and the elements of `S`. This is the case of the identity realization; the commutation of `S` with the symbols is automatic in the commutative jet algebra.

```
lemma invariant_mem_adjoin_fieldStrength [jets.Free] (S : Set (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤)
    )
    (hS : ∀ y ∈ S, ∀ U : jets.truncationKer 0, complexRepJet jets U.1 y = y)
    {x : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤}
    (hx : x ∈ Algebra.adjoin ℂ ({b : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤 |
      ∃ (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
      b = gaugeField 𝔤 p μ φ} ∪ S))
    (hinv : ∀ U : GJ, complexRepJet jets U x = x) :
    x ∈ Algebra.adjoin ℂ ({b : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤 |
      ∃ (l : List (Fin 1 ⊕ Fin 3)) (ν lam : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
      b = GaugeAlgebraRealization.iteratedCovDerivAdjoint (gaugeField 𝔤) l
        (GaugeAlgebraRealization.fieldStrength (gaugeField 𝔤) ν lam) 0 φ} ∪ S)
```

2.190 `PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeBoson.Realization.TransformsAdjoin`

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeBoson/Realization/TransformsAdjoint.lean on GitHub.

`def TransformsInAdjoint` [source]

*A family of derivative symbols `F` *transforms in the adjoint* (is an adjoint gauge tensor) for the gauge representation `repGauge` when each symbol `[∂_s F^φ]` transforms by the Leibniz convolution of the dual adjoint coefficients against lower symbols — the shape of `gauge_apply_deriv` with no Maurer–Cartan shift. At `s = 0` this is the homogeneous law `U • F^φ = F^{Ad₀^* φ}`.*

```
def TransformsInAdjoint (repGauge : Representation ℂ GJ B)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B) : Prop
```

`lemma repGauge_zero` [source]

An adjoint gauge tensor transforms at the base point through the dual adjoint coefficient of the value of the gauge jet alone: the antidiagonal of the empty multiset has a single term.

```
lemma TransformsInAdjoint.repGauge_zero
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B}
    (hF : TransformsInAdjoint jets repGauge F) (U : GJ) (φ : Module.Dual ℝ 𝔤) :
    repGauge U (F 0 φ) = F 0 (jets.adjointDualCoeff U⁻¹ 0 φ)
```

`def bracketFamConv` [source]

***The derived bracket family** `[A_ρ, F]`: the `s`-derivative of the bracket of the gauge field against a family, given by the Leibniz convolution of the derivative symbols over the multiset antidiagonal.*

```
noncomputable def bracketFamConv
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (ρ : Fin 1 ⊕ Fin 3)
```

```
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) : Module.Dual ℝ 𝔤 →ₗ[ℝ] B
```

def GaugeAlgebraRealization.covDerivAdjoint [source]

The covariant derivative `∇_ρ F = [∂_ρ F] + [A_ρ, F]` of an adjoint-valued family of derivative symbols: the extra derivative on the symbol plus the derived bracket against the gauge field. The gauge-algebra bracket carries the physicists' `i`, so in matrix terms this is `∂_ρ F + i [A_ρ, F]` — the adjoint-representation covariant derivative in the same `D = ∂ + i A` convention as the field strength. It preserves `TransformsInAdjoint` (`TransformsInAdjoint.covDerivAdjoint`).

```
noncomputable def covDerivAdjoint
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] B
```

lemma covDerivAdjoint_apply [source]

```
@[simp]
lemma covDerivAdjoint_apply
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤) :
    covDerivAdjoint A F ρ s φ = F (ρ ::ₘ s) φ + bracketFamConv A ρ F s φ
```

lemma covDerivAdjoint_apply_of_map_lie [source]

At a dual vector vanishing on every bracket the derived bracket term drops out and the covariant derivative is the ordinary one, `(∇_ρ F)^φ = ∂_ρ F^φ`.

```
lemma covDerivAdjoint_apply_of_map_lie {φ : Module.Dual ℝ 𝔤}
    (hφ : ∀ x y : 𝔤, φ ⁅x, y⁆ = 0)
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    covDerivAdjoint A F ρ s φ = F (ρ ::ₘ s) φ
```

lemma bracketFamConv_map [source]

The derived bracket family is natural in the algebra.

```
lemma bracketFamConv_map {B' : Type} [Ring B'] [Module ℝ B'] [SMulCommClass ℝ B' B']
    [IsScalarTower ℝ B' B'] (Φ : B →ₗ[ℝ] B') (hΦ : ∀ b₁ b₂, Φ (b₁ * b₂) = Φ b₁ * Φ b₂)
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (ρ : Fin 1 ⊕ Fin 3) (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    bracketFamConv (fun p σ => Φ ∘ₗ A p σ) ρ (fun p => Φ ∘ₗ F p) s =
      Φ ∘ₗ bracketFamConv A ρ F s
```

lemma covDerivAdjoint_map [source]

The covariant derivative is natural in the algebra.

```
lemma covDerivAdjoint_map {B' : Type} [Ring B'] [Module ℝ B'] [SMulCommClass ℝ B' B']
    [IsScalarTower ℝ B' B'] (Φ : B →ₗ[ℝ] B') (hΦ : ∀ b₁ b₂, Φ (b₁ * b₂) = Φ b₁ * Φ b₂)
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    covDerivAdjoint (fun p σ => Φ ∘ₗ A p σ) (fun p => Φ ∘ₗ F p) ρ s =
      Φ ∘ₗ covDerivAdjoint A F ρ s
```

lemma repGauge_cons [source]

If `F` transforms in the adjoint, so do its `κ ::ₘ s`-derived symbols with the extra derivative traced through `LocalGaugeData.adjointDualCoeff_cons`: the Leibniz splittings where `κ` stays a derivative, minus the convolution where `κ` hits the adjoint — an `ad` of the derived Maurer–Cartan form.

```
lemma TransformsInAdjoint.repGauge_cons
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B}
    (hF : TransformsInAdjoint jets repGauge F)
    (U : GJ) (κ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℝ 𝔤) :
    repGauge U (F (κ ::ₘ s) φ) =
      (s.antidiagonal.map fun p =>
        F (κ ::ₘ p.2) (jets.adjointDualCoeff U⁻¹ p.1 φ)).sum
      - (s.antidiagonal.map fun p =>
          (p.1.antidiagonal.map fun q =>
            F p.2 (jets.adjointDualCoeff U⁻¹ q.2
              (φ ∘ₗ LieAlgebra.ad ℝ 𝔤 (jets.evalLie
                (jets.iteratedDeriv q.1
                  (jets.maurerCartan U⁻¹ κ))))))).sum).sum
```

lemma repGauge_bracketFamConv [source]

The all-orders gauge transformation of the derived bracket `[A_ρ, F]` against an adjoint gauge tensor `F`: since `F` transforms homogeneously, only one `ad` cross-term convolution survives — the analogue of `repGauge_commutatorFam` with a gauge tensor in the second slot.

```
lemma TransformsInAdjoint.repGauge_bracketFamConv
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B}
    (hF : TransformsInAdjoint jets repGauge F)
    (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) (ρ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repGauge U (bracketFamConv h.A ρ F s φ) =
      (s.antidiagonal.map fun p =>
        bracketFamConv h.A ρ F p.2 (jets.adjointDualCoeff U⁻¹ p.1 φ)).sum
      + (s.antidiagonal.map fun p =>
          (p.2.antidiagonal.map fun r =>
            F r.2 (jets.adjointDualCoeff U⁻¹ r.1
              (φ ∘ₗ LieAlgebra.ad ℝ 𝔤 (jets.evalLie
                (jets.iteratedDeriv p.1
                  (jets.maurerCartan U⁻¹ ρ))))))).sum).sum
```

lemma TransformsInAdjoint.covDerivAdjoint [source]

***Adjoint gauge tensors are closed under the covariant derivative**: if `F` transforms in the adjoint, so does `∇_ρ F = [∂_ρ F] + [A_ρ, F]`. The single inhomogeneous convolution of `[∂_{ρ ::ₘ s} F]` (`TransformsInAdjoint.repGauge_cons`) cancels the single `ad` cross-term convolution of `[A_ρ, F]` (`TransformsInAdjoint.repGauge_bracketFamConv`) through the coassociativity of the antidiagonal; no structural equation is needed. Together with `transformsInAdjoint_fieldStrength` this makes every iterated covariant derivative of the field strength an adjoint gauge tensor, by recursion.*

```
lemma TransformsInAdjoint.covDerivAdjoint
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B}
    (hF : TransformsInAdjoint jets repGauge F) (ρ : Fin 1 ⊕ Fin 3) :
    TransformsInAdjoint jets repGauge (GaugeAlgebraRealization.covDerivAdjoint h.A F ρ)
```

2.191 `PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.Basic`

`PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeFieldData/Basic.lean` on GitHub.

`structure GaugeFieldData` [source]

The field data of a gauge theory. Over a gauge context, given by a jet gauge group `GJ` with global group `G₀`, a finite-dimensional real gauge algebra `𝔤` with jet algebra `𝔤J` and a local-gauge-data package `jets` over them, it records a finite family of fermionic species and a finite family of bosonic species, each given by an existing `MatterField jets`. The species types are `Fintype` rather than merely `Finite`, so that a theory may be summed over its species: this is what lets the several fermionic multiplets be assembled into the single fermionic matter field of `GaugeFieldData.fermionMatterField`.

Nothing is repeated from `MatterField`, whose fields already carry the value space, the Lorentz representation, the local-gauge-data action, the mass weight and the declared statistic of a species. The fermionic family supplies the generators of the exterior-algebra factor of the local field algebra `GaugeFieldData.LocalFieldAlgebra`, and the bosonic family those of its symmetric-algebra factor; the fields `fermion_statistic` and `boson_statistic` require every entry of each family to be declared with the corresponding statistics. Nothing is repeated from the gauge context either, and the gauge bosons are not a species, since their generator space is determined by `𝔤` alone.

This is data before a Lagrangian. Collecting representations of the several species does not assert that they are jointly consistent. Gauge-Lorentz compatibility, factorization of the jet action through its global value, and richness of the jet group are separate conditions, none of them imposed here.

```
structure GaugeFieldData {G₀ : Type} [Group G₀] {𝔤 : Type} [LieRing 𝔤] [LieAlgebra ℝ 𝔤]
    [Module.Finite ℝ 𝔤] {GJ : Type} [Group GJ] {𝔤J : Type} [LieRing 𝔤J] [LieAlgebra ℝ 𝔤J]
    (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) where
  FermionSpecies : Type
  [decidableEqFermionSpecies : DecidableEq FermionSpecies]
  [fintypeFermionSpecies : Fintype FermionSpecies]
  fermion : FermionSpecies → MatterField jets
  fermion_statistic : ∀ i, (fermion i).statistic = .fermionic
  BosonSpecies : Type
  [decidableEqBosonSpecies : DecidableEq BosonSpecies]
  [fintypeBosonSpecies : Fintype BosonSpecies]
  boson : BosonSpecies → MatterField jets
  boson_statistic : ∀ j, (boson j).statistic = .bosonic
```

abbrev FermionValue [source]

The value space of a fermionic species.

```
abbrev FermionValue (i : T.FermionSpecies) : Type
```

abbrev BosonValue [source]

The value space of a bosonic species.

```
abbrev BosonValue (j : T.BosonSpecies) : Type
```

def PureJetsActTrivially [source]

Pure gauge jets act trivially at the base point on every species of the datum.

```
def PureJetsActTrivially : Prop
```

def GaugeLorentzCompatible [source]

The infinitesimal gauge action of every species of the datum commutes with its Lorentz representation.

```
def GaugeLorentzCompatible : Prop
```

2.192 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.BosonGenerators

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeFieldData/BosonGenerators.lean on GitHub.

def BosonGenerators [source]

The bosonic generator space of the datum, holding the component functions `∂_s φ_α` and their conjugates of every bosonic species at once, as a direct sum over the species. A component function of the theory is a finitely supported family of component functions of the species.

The direct sum, rather than a single component space on the product of the value spaces, is what lets the species carry different mass weights: the scaling of one component space is natural in the value space — `JetComponentSpace.comap_comp_massWeightScale` — and so cannot tell the species apart. When the weights do agree the two descriptions coincide, which is section C below.

This `def` and its explicit instances reduce instance-term expansion in the symmetric algebra and tensor products built on it. Use the inclusion, assembly and extensionality API in downstream proofs; unfold the direct-sum representation explicitly when necessary.

```
def BosonGenerators : Type
```

instance AddCommGroup T.BosonGenerators [source]

```
instance : AddCommGroup T.BosonGenerators
```

instance Module ℂ T.BosonGenerators [source]

```
instance : Module ℂ T.BosonGenerators
```

def inclBoson [source]

The inclusion of the component space of one bosonic species into the bosonic generator space.

```
def inclBoson (i : T.BosonSpecies) :
    JetComponentSpace (T.boson i) →ₗ[ℂ] T.BosonGenerators
```

def assembleBoson [source]

The assembly of a species-wise family of linear maps out of the bosonic generator space into a common target.

```
def assembleBoson (f : ∀ i, JetComponentSpace (T.boson i) →ₗ[ℂ] N) :
    T.BosonGenerators →ₗ[ℂ] N
```

lemma assembleBoson_inclBoson [source]

```
@[simp]
lemma assembleBoson_inclBoson (f : ∀ i, JetComponentSpace (T.boson i) →ₗ[ℂ] N)
    (i : T.BosonSpecies) (x : JetComponentSpace (T.boson i)) :
    T.assembleBoson f (T.inclBoson i x) = f i x
```

lemma bosonGenerators_hom_ext [source]

Two linear maps out of the bosonic generator space agreeing on every species are equal.

```
lemma bosonGenerators_hom_ext {F F' : T.BosonGenerators →ₗ[ℂ] N}
    (h : ∀ i x, F (T.inclBoson i x) = F' (T.inclBoson i x)) : F = F'
```

def repLorentzBoson [source]

The Lorentz action on the bosonic generator space, acting on each species through the Lorentz representation of its matter field.

```
noncomputable def repLorentzBoson : Representation ℂ SL(2,ℂ) T.BosonGenerators where
```

lemma repLorentzBoson_inclBoson [source]

```
@[simp]
lemma repLorentzBoson_inclBoson (Λ : SL(2,ℂ)) (i : T.BosonSpecies)
    (x : JetComponentSpace (T.boson i)) :
    T.repLorentzBoson Λ (T.inclBoson i x)
      = T.inclBoson i (JetComponentSpace.repLorentzGroup (T.boson i) Λ x)
```

def repJetBoson [source]

The action of the jet gauge group on the bosonic generator space, acting on each species through the fibrewise jet action of its matter field. Both the fibrewise hypothesis and the finite dimensionality of the value space that `JetComponentSpace.repJet` needs are already fields of `MatterField`.

```
noncomputable def repJetBoson : Representation ℂ GJ T.BosonGenerators where
```

lemma repJetBoson_inclBoson [source]

```
@[simp]
lemma repJetBoson_inclBoson (U : GJ) (i : T.BosonSpecies)
    (x : JetComponentSpace (T.boson i)) :
    T.repJetBoson U (T.inclBoson i x)
      = T.inclBoson i
        (JetComponentSpace.repJet (T.boson i) U x)
```

def massWeightScaleBoson [source]

The mass-weight scaling on the bosonic generator space, with the weight of each species taken from its matter field. Species of different weight scale differently, which is the property the direct-sum generator space was chosen to have.

```
noncomputable def massWeightScaleBoson (c : ℂ) :
    T.BosonGenerators →ₗ[ℂ] T.BosonGenerators
```

lemma massWeightScaleBoson_inclBoson [source]

On the summand of a species the scaling is that species' own mass-weight scaling, with the weight recorded in its matter field.

```
@[simp]
lemma massWeightScaleBoson_inclBoson (c : ℂ) (i : T.BosonSpecies)
    (x : JetComponentSpace (T.boson i)) :
    T.massWeightScaleBoson c (T.inclBoson i x)
      = T.inclBoson i
        (JetComponentSpace.massWeightScale (T.boson i).massWeight c x)
```

lemma massWeightScaleBoson_inclBoson_basis_tmul [source]

A component function `∂_s φ_α` of a species scales by `c ^ (w + 2 |s|)`, where `w` is the mass weight of that species. There is one factor of `c` per unit of mass dimension of the field and two per derivative.

```
lemma massWeightScaleBoson_inclBoson_basis_tmul (c : ℂ) (i : T.BosonSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.BosonValue i)) :
    T.massWeightScaleBoson c (T.inclBoson i
        ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace (T.boson i)))
      = c ^ ((T.boson i).massWeight + 2 * Multiset.card s) • T.inclBoson i
          ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace (T.boson i))
```

lemma massWeightScaleBoson_inclBoson_basis_tmul_conj [source]

The conjugate component functions of a species scale with the same weight as its unconjugated ones.

```
lemma massWeightScaleBoson_inclBoson_basis_tmul_conj (c : ℂ) (i : T.BosonSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue i))) :
    T.massWeightScaleBoson c (T.inclBoson i
        ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace (T.boson i)))
      = c ^ ((T.boson i).massWeight + 2 * Multiset.card s) • T.inclBoson i
          ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace (T.boson i))
```

def jetDerivBoson [source]

The ordinary derivative shift on the bosonic generator space, acting on each species' component functions by appending ∂_μ to the derivative label.

```
noncomputable def jetDerivBoson (μ : Fin 1 ⊕ Fin 3) :
    T.BosonGenerators →ₗ[ℂ] T.BosonGenerators
```

lemma jetDerivBoson_inclBoson [source]

```
@[simp]
lemma jetDerivBoson_inclBoson (μ : Fin 1 ⊕ Fin 3) (i : T.BosonSpecies)
    (x : JetComponentSpace (T.boson i)) :
    T.jetDerivBoson μ (T.inclBoson i x)
      = T.inclBoson i (JetComponentSpace.jetDeriv μ x)
```

lemma jetDerivBoson_comm [source]

Mixed partials agree on the bosonic generator space, because they do on each species.

```
lemma jetDerivBoson_comm (μ ν : Fin 1 ⊕ Fin 3) :
    (T.jetDerivBoson μ).comp (T.jetDerivBoson ν)
      = (T.jetDerivBoson ν).comp (T.jetDerivBoson μ)
```

lemma repLorentzBoson_jetDerivBoson [source]

The derivative shift is a Lorentz vector on the bosonic generator space: it is one on each species, and both operations are species-diagonal.

```
lemma repLorentzBoson_jetDerivBoson (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3)
    (x : T.BosonGenerators) :
    T.repLorentzBoson Λ (T.jetDerivBoson μ x)
      = ∑ a, (((Lorentz.SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
          T.jetDerivBoson a (T.repLorentzBoson Λ x)
```

lemma massWeightScaleBoson_jetDerivBoson [source]

The derivative carries mass weight two on the bosonic generator space, whatever the weights of the species: the shift adds two units of mass dimension to every component function alike.

```
lemma massWeightScaleBoson_jetDerivBoson (c : ℂ) (μ : Fin 1 ⊕ Fin 3) :
    (T.massWeightScaleBoson c).comp (T.jetDerivBoson μ)
      = c ^ 2 • (T.jetDerivBoson μ).comp (T.massWeightScaleBoson c)
```

def bosonGeneratorsEquiv [source]

***The bosonic generator space is the component space of the bosonic matter field.** The direct sum over the species of their component spaces is, the species type being finite, the same thing as the space of component functions of the single field `T.bosonMatterField w h` — the presentation of the boson content used in writing a theory down. The shared weight `w` enters only because a component space is now taken of a matter field, and the only matter field on `T.BosonModule` is that one; the underlying identification does not use it.*

```
noncomputable def bosonGeneratorsEquiv (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w) :
    T.BosonGenerators ≃ₗ[ℂ] JetComponentSpace (T.bosonMatterField w h)
```

lemma bosonGeneratorsEquiv_inclBoson [source]

***A species sits inside the bosonic generators as the pullback along the projection onto it.** A component function of the multiplet `i` becomes the component function of the whole field whose target covector is supported on that multiplet.*

```
@[simp]
lemma bosonGeneratorsEquiv_inclBoson (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w)
    (i : T.BosonSpecies) (x : JetComponentSpace (T.boson i)) :
    T.bosonGeneratorsEquiv w h (T.inclBoson i x)
      = JetComponentSpace.comap
        (T.projBosonField w h i) x
```

lemma bosonFieldComponents_hom_ext [source]

Two linear maps out of the component space of the bosonic matter field agree as soon as they agree on every species, the species pullbacks spanning it. This is the counterpart, on the single-field side of the identification, of `bosonGenerators_hom_ext`.

```
lemma bosonFieldComponents_hom_ext {N : Type} [AddCommGroup N] [Module ℂ N]
    (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w)
    {F F' : JetComponentSpace (T.bosonMatterField w h) →ₗ[ℂ] N}
    (hs : ∀ i x, F (JetComponentSpace.comap
        (T.projBosonField w h i) x)
      = F' (JetComponentSpace.comap
        (T.projBosonField w h i) x)) : F = F'
```

lemma bosonGeneratorsEquiv_repLorentzBoson [source]

***The identification is Lorentz-equivariant.** The species-diagonal Lorentz action on the generator space is the Lorentz action on the component functions of the single field: each species is a subrepresentation of the bosonic matter field, so pulling back along the projection onto it commutes with the two actions.*

```
lemma bosonGeneratorsEquiv_repLorentzBoson (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w)
    (Λ : SL(2,ℂ)) (y : T.BosonGenerators) :
    T.bosonGeneratorsEquiv w h (T.repLorentzBoson Λ y)
      = JetComponentSpace.repLorentzGroup (T.bosonMatterField w h) Λ
        (T.bosonGeneratorsEquiv w h y)
```

lemma bosonGeneratorsEquiv_repJetBoson [source]

The identification is equivariant for the jet gauge action. The species-diagonal action of the jets of gauge transformations on the generator space is the action on the component functions of the single boson field, for the same reason as in the fermionic case: each species is a subrepresentation of the bosonic module. The common mass weight enters only through the packaging of the bosonic module as a matter field; both halves of the component space, with every derivative label, are covered.

```
lemma bosonGeneratorsEquiv_repJetBoson (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w)
    (U : GJ) (y : T.BosonGenerators) :
    T.bosonGeneratorsEquiv w h (T.repJetBoson U y)
      = JetComponentSpace.repJet (T.bosonMatterField w h) U
        (T.bosonGeneratorsEquiv w h y)
```

lemma bosonGeneratorsEquiv_massWeightScaleBoson [source]

***The identification carries the species-wise mass-weight scaling to a single scaling.** With one weight `w` shared by every species, the scaling that acts on each species through its own weight is the scaling of weight `w` on the component functions of the one field: `comap` is natural in the value space, so it does not see which species a generator came from.*

```
lemma bosonGeneratorsEquiv_massWeightScaleBoson (w : ℕ)
    (h : ∀ i, (T.boson i).massWeight = w) (c : ℂ) (y : T.BosonGenerators) :
    T.bosonGeneratorsEquiv w h (T.massWeightScaleBoson c y)
      = JetComponentSpace.massWeightScale w c (T.bosonGeneratorsEquiv w h y)
```

lemma bosonGeneratorsEquiv_jetDerivBoson [source]

The identification carries the derivative shift across. The species-diagonal shift of the derivative label on the generator space is the shift on the component functions of the single boson field: `comap` is natural in the value space, and the shift touches only the derivative label, so neither operation sees which species a generator came from. The common mass weight enters only through the packaging of the bosonic module as a matter field.

```
lemma bosonGeneratorsEquiv_jetDerivBoson (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w)
    (μ : Fin 1 ⊕ Fin 3) (y : T.BosonGenerators) :
    T.bosonGeneratorsEquiv w h (T.jetDerivBoson μ y)
      = JetComponentSpace.jetDeriv μ (T.bosonGeneratorsEquiv w h y)
```

2.193 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.BosonMatterField

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeFieldData/BosonMatterField.lean on GitHub.

def bosonMatterField [source]

***The bosonic matter field of a gauge theory**: the direct sum of the bosonic species, valued in `T.BosonModule`, with the Lorentz group, the jets of gauge transformations and the gauge algebra all acting species by species. It exists only for a family degenerate in mass dimension: `w` is the common mass weight of the species and `h` the proof that they all have it, which for the Standard Model, whose only scalar is the Higgs doublet, is weight two.*

```
noncomputable def bosonMatterField (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w) :
    MatterField jets
```

lemma bosonMatterField_V [source]

```
lemma bosonMatterField_V (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w) :
    (T.bosonMatterField w h).V = T.BosonModule
```

lemma bosonMatterField_massWeight [source]

The bosonic matter field carries the common mass weight of the species.

```
@[simp]
lemma bosonMatterField_massWeight (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w) :
    (T.bosonMatterField w h).massWeight = w
```

lemma bosonMatterField_statistic [source]

The bosonic matter field is bosonic, as every bosonic species is declared to be.

```
@[simp]
lemma bosonMatterField_statistic (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w) :
    (T.bosonMatterField w h).statistic = .bosonic
```

def repLorentzBosonModule [source]

***The Lorentz action on the bosonic module**, acting species by species. This is the Lorentz representation of the bosonic matter field, typed on `T.BosonModule` itself so that it can be spoken of without fixing a common mass weight.*

```
noncomputable def repLorentzBosonModule : Representation ℂ SL(2,ℂ) T.BosonModule
```

lemma repLorentzBosonModule_apply [source]

```
@[simp]
lemma repLorentzBosonModule_apply (Λ : SL(2,ℂ)) (v : T.BosonModule) (i : T.BosonSpecies) :
    T.repLorentzBosonModule Λ v i = (T.boson i).repLorentz Λ (v i)
```

def repJetBosonModule [source]

The action of the jets of gauge transformations on the jets of the bosonic module, acting species by species, and typed on `T.BosonModule` itself so that it can be spoken of without fixing a common mass weight.

```
noncomputable def repJetBosonModule :
    Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] T.BosonModule)
```

lemma repJetBosonModule_smul [source]

The jet gauge action on the bosonic module is fibrewise, as each species is.

```
lemma repJetBosonModule_smul (U : GJ) (χ : SpaceTimeAlgebra)
    (z : SpaceTimeAlgebra ⊗[ℂ] T.BosonModule) :
    T.repJetBosonModule U (χ • z) = χ • T.repJetBosonModule U z
```

lemma lTensor_projBosonValue_repJetBosonModule [source]

A bosonic species is a subrepresentation of the jet gauge action on the bosonic module: the projection onto its value space intertwines the two actions on the jets.

```
lemma lTensor_projBosonValue_repJetBosonModule (j : T.BosonSpecies) (U : GJ) :
    (LinearMap.lTensor SpaceTimeAlgebra (T.projBosonValue j)).comp (T.repJetBosonModule U)
      = ((T.boson j).repJet U).comp
        (LinearMap.lTensor SpaceTimeAlgebra (T.projBosonValue j))
```

lemma bosonMatterField_repJet [source]

The jet gauge action of the bosonic matter field is that of the bosonic module.

```
lemma bosonMatterField_repJet (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w) :
    (T.bosonMatterField w h).repJet = T.repJetBosonModule
```

lemma bosonMatterField_repLorentz [source]

The Lorentz action of the bosonic matter field is that of the bosonic module.

```
lemma bosonMatterField_repLorentz (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w) :
    (T.bosonMatterField w h).repLorentz = T.repLorentzBosonModule
```

lemma bosonMatterField_repLorentz_apply [source]

The Lorentz group acts on a bosonic configuration species by species.

```
@[simp]
lemma bosonMatterField_repLorentz_apply (w : ℕ)
    (h : ∀ i, (T.boson i).massWeight = w) (Λ : SL(2,ℂ)) (v : T.BosonModule)
    (i : T.BosonSpecies) :
    (T.bosonMatterField w h).repLorentz Λ v i = (T.boson i).repLorentz Λ (v i)
```

lemma bosonMatterField_repAlgebra_apply [source]

The gauge algebra acts on a bosonic configuration species by species.

```
@[simp]
lemma bosonMatterField_repAlgebra_apply (w : ℕ)
    (h : ∀ i, (T.boson i).massWeight = w) (c : 𝔤) (v : T.BosonModule)
    (i : T.BosonSpecies) :
    (T.bosonMatterField w h).repAlgebra c v i = (T.boson i).repAlgebra c (v i)
```

lemma bosonMatterField_repJet_apply [source]

The jets of gauge transformations act on the jets of the bosonic field species by species, through the identification `jetPiEquiv` of the jets of the bosonic module with the family of the jets of the species.

```
lemma bosonMatterField_repJet_apply (w : ℕ)
    (h : ∀ i, (T.boson i).massWeight = w) (U : GJ)
    (z : SpaceTimeAlgebra ⊗[ℂ] T.BosonModule) :
    (T.bosonMatterField w h).repJet U z =
      (jetPiEquiv T.BosonValue).symm
        (fun i => (T.boson i).repJet U (jetPiEquiv T.BosonValue z i))
```

lemma repCoeff_bosonMatterField [source]

The base-point Taylor coefficients of the bosonic jet action are those of the species, index by index.

```
lemma repCoeff_bosonMatterField (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w) (U : GJ)
    (x : Multiset (Fin 1 ⊕ Fin 3)) :
    GaugeAlgebraRealization.repCoeff (T.bosonMatterField w h).repJet U x =
      LinearMap.piMap fun i =>
        GaugeAlgebraRealization.repCoeff (T.boson i).repJet U x
```

lemma bosonMatterField_repLorentz_inclBosonValue [source]

***A bosonic species is a Lorentz subrepresentation of the bosonic matter field**: including a value of one species and then transforming is transforming and then including. Assembling the species into one field therefore loses no Lorentz information.*

```
lemma bosonMatterField_repLorentz_inclBosonValue (w : ℕ)
    (h : ∀ i, (T.boson i).massWeight = w) (Λ : SL(2,ℂ)) (i : T.BosonSpecies)
    (x : T.BosonValue i) :
    (T.bosonMatterField w h).repLorentz Λ (T.inclBosonValue i x)
      = T.inclBosonValue i ((T.boson i).repLorentz Λ x)
```

lemma bosonMatterField_repAlgebra_inclBosonValue [source]

***A bosonic species is a subrepresentation of the gauge algebra action** on the bosonic matter field, for the same reason: the gauge algebra does not mix the species.*

```
lemma bosonMatterField_repAlgebra_inclBosonValue (w : ℕ)
    (h : ∀ i, (T.boson i).massWeight = w) (c : 𝔤) (i : T.BosonSpecies)
    (x : T.BosonValue i) :
    (T.bosonMatterField w h).repAlgebra c (T.inclBosonValue i x)
      = T.inclBosonValue i ((T.boson i).repAlgebra c x)
```

lemma finrank_bosonMatterField [source]

The dimension of the bosonic matter field is the sum of the dimensions of the species.

```
lemma finrank_bosonMatterField (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w) :
    Module.finrank ℂ (T.bosonMatterField w h).V
      = ∑ i, Module.finrank ℂ (T.BosonValue i)
```

abbrev projBosonField [source]

The projection of the bosonic matter field onto one species, typed as a map out of `(T.bosonMatterField w h).V` rather than out of `T.BosonModule`. The two are the

same type by definition, but naming the first keeps unification from having to unfold the direct sum every time the projection meets the matter field.

```
noncomputable abbrev projBosonField (w : ℕ) (h : ∀ i, (T.boson i).massWeight = w)
    (i : T.BosonSpecies) : (T.bosonMatterField w h).V →ₗ[ℂ] (T.boson i).V
```

2.194 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.BosonModule

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeFieldData/BosonModule.lean on GitHub.

abbrev BosonModule [source]

***The bosonic module of a gauge theory**: the product, over the bosonic species, of their value spaces. One element of it is a value of the entire bosonic content of the theory — the column of all the boson fields — as opposed to `GaugeFieldData.BosonGenerators`, which is a space of component *functions* and is infinite-dimensional. Since the species type is finite the product is also their direct sum.*

```
abbrev BosonModule : Type
```

abbrev projBosonValue [source]

The component of a bosonic configuration in one species.

```
abbrev projBosonValue (i : T.BosonSpecies) :
    T.BosonModule →ₗ[ℂ] T.BosonValue i
```

abbrev inclBosonValue [source]

The inclusion of one bosonic species into the bosonic module, extending a value of that species by zero in every other.

```
abbrev inclBosonValue (i : T.BosonSpecies) :
    T.BosonValue i →ₗ[ℂ] T.BosonModule
```

lemma projBosonValue_apply [source]

```
@[simp]
lemma projBosonValue_apply (i : T.BosonSpecies) (v : T.BosonModule) :
    T.projBosonValue i v = v i
```

lemma projBosonValue_inclBosonValue_self [source]

```
lemma projBosonValue_inclBosonValue_self (i : T.BosonSpecies)
    (v : T.BosonValue i) : T.projBosonValue i (T.inclBosonValue i v) = v
```

lemma projBosonValue_inclBosonValue_of_ne [source]

A value of one species has no component in any other species: the inclusions of distinct species have disjoint supports.

```
lemma projBosonValue_inclBosonValue_of_ne {i j : T.BosonSpecies} (h : i ≠ j)
    (v : T.BosonValue j) : T.projBosonValue i (T.inclBosonValue j v) = 0
```

lemma sum_inclBosonValue_proj [source]

***A bosonic configuration is the sum of its species components**. The product over the species is their direct sum, the species type being finite, so nothing is lost in describing the bosonic content of the theory by one module.*

```
lemma sum_inclBosonValue_proj (v : T.BosonModule) :
    ∑ i, T.inclBosonValue i (v i) = v
```

lemma bosonModule_hom_ext [source]

Two linear maps out of the bosonic module agreeing on every species are equal.

```
lemma bosonModule_hom_ext {N : Type} [AddCommGroup N] [Module ℂ N]
    {F F' : T.BosonModule →ₗ[ℂ] N}
    (h : ∀ i, F.comp (T.inclBosonValue i) = F'.comp (T.inclBosonValue i)) : F = F'
```

lemma finrank_bosonModule [source]

***The dimension of the bosonic module is the sum of the dimensions of the species**, each species contributing the number of complex components of its multiplet.*

```
lemma finrank_bosonModule :
    Module.finrank ℂ T.BosonModule = ∑ i, Module.finrank ℂ (T.BosonValue i)
```

2.195 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.FermionGenerators

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeFieldData/FermionGenerators.lean on GitHub.

def FermionGenerators [source]

The fermionic generator space of the datum, holding the component functions `∂_s ψ_α` and their conjugates of every fermionic species at once, as a direct sum over the species. A component function of the theory is a finitely supported family of component functions of the species.

The direct sum, rather than a single component space on the product of the value spaces, is what lets the species carry different mass weights: the scaling of one component space is natural in the value space — `JetComponentSpace.comap_comp_massWeightScale` — and so cannot tell the species apart. When the weights do agree the two descriptions coincide, which is section C below.

This `def` and its explicit instances reduce instance-term expansion in the exterior algebra and tensor products built on it. Use the inclusion, assembly and extensionality API in downstream proofs; unfold the direct-sum representation explicitly when necessary.

```
def FermionGenerators : Type
```

instance AddCommGroup T.FermionGenerators [source]

```
instance : AddCommGroup T.FermionGenerators
```

instance Module ℂ T.FermionGenerators [source]

```
instance : Module ℂ T.FermionGenerators
```

def inclFermion [source]

The inclusion of the component space of one fermionic species into the fermionic generator space.

```
def inclFermion (i : T.FermionSpecies) :
    JetComponentSpace (T.fermion i) →ₗ[ℂ] T.FermionGenerators
```

def assembleFermion [source]

The assembly of a species-wise family of linear maps out of the fermionic generator space into a common target.

```
def assembleFermion (f : ∀ i, JetComponentSpace (T.fermion i) →ₗ[ℂ] N) :
    T.FermionGenerators →ₗ[ℂ] N
```

lemma assembleFermion_inclFermion [source]

```
@[simp]
lemma assembleFermion_inclFermion (f : ∀ i, JetComponentSpace (T.fermion i) →ₗ[ℂ] N)
    (i : T.FermionSpecies) (x : JetComponentSpace (T.fermion i)) :
    T.assembleFermion f (T.inclFermion i x) = f i x
```

lemma fermionGenerators_hom_ext [source]

Two linear maps out of the fermionic generator space agreeing on every species are equal.

```
lemma fermionGenerators_hom_ext {F F' : T.FermionGenerators →ₗ[ℂ] N}
    (h : ∀ i x, F (T.inclFermion i x) = F' (T.inclFermion i x)) : F = F'
```

def repLorentzFermion [source]

The Lorentz action on the fermionic generator space, acting on each species through the Lorentz representation of its matter field.

```
noncomputable def repLorentzFermion : Representation ℂ SL(2,ℂ) T.FermionGenerators where
```

lemma repLorentzFermion_inclFermion [source]

```
@[simp]
lemma repLorentzFermion_inclFermion (Λ : SL(2,ℂ)) (i : T.FermionSpecies)
    (x : JetComponentSpace (T.fermion i)) :
    T.repLorentzFermion Λ (T.inclFermion i x)
      = T.inclFermion i (JetComponentSpace.repLorentzGroup (T.fermion i) Λ x)
```

def repJetFermion [source]

The action of the jet gauge group on the fermionic generator space, acting on each species through the fibrewise jet action of its matter field. Both the fibrewise hypothesis and the finite dimensionality of the value space that `JetComponentSpace.repJet` needs are already fields of `MatterField`.

```
noncomputable def repJetFermion : Representation ℂ GJ T.FermionGenerators where
```

lemma repJetFermion_inclFermion [source]

```
@[simp]
lemma repJetFermion_inclFermion (U : GJ) (i : T.FermionSpecies)
    (x : JetComponentSpace (T.fermion i)) :
    T.repJetFermion U (T.inclFermion i x)
      = T.inclFermion i
        (JetComponentSpace.repJet (T.fermion i) U x)
```

def massWeightScaleFermion [source]

The mass-weight scaling on the fermionic generator space, with the weight of each species taken from its matter field. Species of different weight scale differently, which is the property the direct-sum generator space was chosen to have.

```
noncomputable def massWeightScaleFermion (c : ℂ) :
    T.FermionGenerators →ₗ[ℂ] T.FermionGenerators
```

lemma massWeightScaleFermion_inclFermion [source]

On the summand of a species the scaling is that species' own mass-weight scaling, with the weight recorded in its matter field.

```
@[simp]
lemma massWeightScaleFermion_inclFermion (c : ℂ) (i : T.FermionSpecies)
    (x : JetComponentSpace (T.fermion i)) :
    T.massWeightScaleFermion c (T.inclFermion i x)
      = T.inclFermion i
        (JetComponentSpace.massWeightScale (T.fermion i).massWeight c x)
```

lemma massWeightScaleFermion_inclFermion_basis_tmul [source]

A component function `∂_s ψ_α` of a species scales by `c ^ (w + 2 |s|)`, where `w` is the mass weight of that species. There is one factor of `c` per unit of mass dimension of the field and two per derivative.

```
lemma massWeightScaleFermion_inclFermion_basis_tmul (c : ℂ) (i : T.FermionSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.FermionValue i)) :
    T.massWeightScaleFermion c (T.inclFermion i
        ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace (T.fermion i)))
      = c ^ ((T.fermion i).massWeight + 2 * Multiset.card s) • T.inclFermion i
          ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace (T.fermion i))
```

lemma massWeightScaleFermion_inclFermion_basis_tmul_conj [source]

The conjugate component functions of a species scale with the same weight as its unconjugated ones.

```
lemma massWeightScaleFermion_inclFermion_basis_tmul_conj (c : ℂ) (i : T.FermionSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.massWeightScaleFermion c (T.inclFermion i
        ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace (T.fermion i)))
      = c ^ ((T.fermion i).massWeight + 2 * Multiset.card s) • T.inclFermion i
          ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace (T.fermion i))
```

def jetDerivFermion [source]

The ordinary derivative shift on the fermionic generator space, acting on each species' component functions by appending ∂_μ to the derivative label.

```
noncomputable def jetDerivFermion (μ : Fin 1 ⊕ Fin 3) :
    T.FermionGenerators →ₗ[ℂ] T.FermionGenerators
```

lemma jetDerivFermion_inclFermion [source]

```
@[simp]
lemma jetDerivFermion_inclFermion (μ : Fin 1 ⊕ Fin 3) (i : T.FermionSpecies)
    (x : JetComponentSpace (T.fermion i)) :
    T.jetDerivFermion μ (T.inclFermion i x)
      = T.inclFermion i (JetComponentSpace.jetDeriv μ x)
```

lemma jetDerivFermion_comm [source]

Mixed partials agree on the fermionic generator space, because they do on each species.

```
lemma jetDerivFermion_comm (μ ν : Fin 1 ⊕ Fin 3) :
    (T.jetDerivFermion μ).comp (T.jetDerivFermion ν)
      = (T.jetDerivFermion ν).comp (T.jetDerivFermion μ)
```

lemma repLorentzFermion_jetDerivFermion [source]

The derivative shift is a Lorentz vector on the fermionic generator space: it is one on each species, and both operations are species-diagonal.

```
lemma repLorentzFermion_jetDerivFermion (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3)
    (x : T.FermionGenerators) :
    T.repLorentzFermion Λ (T.jetDerivFermion μ x)
      = ∑ a, (((Lorentz.SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
          T.jetDerivFermion a (T.repLorentzFermion Λ x)
```

lemma massWeightScaleFermion_jetDerivFermion [source]

The derivative carries mass weight two on the fermionic generator space, whatever the weights of the species: the shift adds two units of mass dimension to every component function alike.

```
lemma massWeightScaleFermion_jetDerivFermion (c : ℂ) (μ : Fin 1 ⊕ Fin 3) :
    (T.massWeightScaleFermion c).comp (T.jetDerivFermion μ)
      = c ^ 2 • (T.jetDerivFermion μ).comp (T.massWeightScaleFermion c)
```

def fermionGeneratorsEquiv [source]

***The fermionic generator space is the component space of the fermionic matter field.** The direct sum over the species of their component spaces is, the species type being finite, the same thing as the space of component functions of the single field `T.fermionMatterField w h` — the presentation of the fermion content used in writing a theory down. The shared weight `w` enters only because a component space is now taken of a matter field, and the only matter field on `T.FermionModule` is that one; the underlying identification does not use it.*

```
noncomputable def fermionGeneratorsEquiv (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w) :
    T.FermionGenerators ≃ₗ[ℂ] JetComponentSpace (T.fermionMatterField w h)
```

lemma fermionGeneratorsEquiv_inclFermion [source]

***A species sits inside the fermionic generators as the pullback along the projection onto it.** A component function of the multiplet `i` becomes the component function of the whole field whose target covector is supported on that multiplet.*

```
@[simp]
lemma fermionGeneratorsEquiv_inclFermion (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w)
    (i : T.FermionSpecies) (x : JetComponentSpace (T.fermion i)) :
    T.fermionGeneratorsEquiv w h (T.inclFermion i x)
      = JetComponentSpace.comap
        (T.projFermionField w h i) x
```

lemma fermionFieldComponents_hom_ext [source]

Two linear maps out of the component space of the fermionic matter field agree as soon as they agree on every species, the species pullbacks spanning it. This is the counterpart, on the single-field side of the identification, of `fermionGenerators_hom_ext`.

```
lemma fermionFieldComponents_hom_ext {N : Type} [AddCommGroup N] [Module ℂ N]
    (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w)
    {F F' : JetComponentSpace (T.fermionMatterField w h) →ₗ[ℂ] N}
    (hs : ∀ i x, F (JetComponentSpace.comap
        (T.projFermionField w h i) x)
      = F' (JetComponentSpace.comap
        (T.projFermionField w h i) x)) : F = F'
```

lemma fermionGeneratorsEquiv_repLorentzFermion [source]

***The identification is Lorentz-equivariant.** The species-diagonal Lorentz action on the generator space is the Lorentz action on the component functions of the single field: each species is a subrepresentation of the fermionic matter field, so pulling back along the projection onto it commutes with the two actions.*

```
lemma fermionGeneratorsEquiv_repLorentzFermion (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w)
    (Λ : SL(2,ℂ)) (y : T.FermionGenerators) :
    T.fermionGeneratorsEquiv w h (T.repLorentzFermion Λ y)
      = JetComponentSpace.repLorentzGroup (T.fermionMatterField w h) Λ
        (T.fermionGeneratorsEquiv w h y)
```

lemma fermionGeneratorsEquiv_repJetFermion [source]

The identification is equivariant for the jet gauge action. The species-diagonal action of the jets of gauge transformations on the generator space is the action on the component functions of the single fermion field: each species is a subrepresentation of the fermionic module, so pulling back along the projection onto it commutes with the two actions.

The common mass weight enters only through the packaging of the fermionic module as the matter field `T.fermionMatterField w h`; the gauge action itself is `repJetFermionModule`, which exists whatever the weights are. Both halves of the component space are covered, the conjugate one included, and every derivative label with them: this is `JetComponentSpace.comap_comp_repJet` at the species projection.

```
lemma fermionGeneratorsEquiv_repJetFermion (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w)
    (U : GJ) (y : T.FermionGenerators) :
    T.fermionGeneratorsEquiv w h (T.repJetFermion U y)
      = JetComponentSpace.repJet (T.fermionMatterField w h) U
        (T.fermionGeneratorsEquiv w h y)
```

lemma fermionGeneratorsEquiv_massWeightScaleFermion [source]

***The identification carries the species-wise mass-weight scaling to a single scaling.** With one weight `w` shared by every species, the scaling that acts on each species through its own weight is the scaling of weight `w` on the component functions of the one field: `comap` is natural in the value space, so it does not see which species a generator came from.*

```
lemma fermionGeneratorsEquiv_massWeightScaleFermion (w : ℕ)
    (h : ∀ i, (T.fermion i).massWeight = w) (c : ℂ) (y : T.FermionGenerators) :
    T.fermionGeneratorsEquiv w h (T.massWeightScaleFermion c y)
      = JetComponentSpace.massWeightScale w c (T.fermionGeneratorsEquiv w h y)
```

lemma fermionGeneratorsEquiv_jetDerivFermion [source]

The identification carries the derivative shift across. The species-diagonal shift of the derivative label on the generator space is the shift on the component functions of the single fermion field: `comap` is natural in the value space, and the shift touches only the derivative label, so neither operation sees which species a generator came from. The common mass weight enters only through the packaging of the fermionic module as a matter field.

```
lemma fermionGeneratorsEquiv_jetDerivFermion (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w)
    (μ : Fin 1 ⊕ Fin 3) (y : T.FermionGenerators) :
    T.fermionGeneratorsEquiv w h (T.jetDerivFermion μ y)
      = JetComponentSpace.jetDeriv μ (T.fermionGeneratorsEquiv w h y)
```

2.196 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.FermionMatterField

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeFieldData/FermionMatterField.lean on GitHub.

def fermionMatterField [source]

***The fermionic matter field of a gauge theory**: the direct sum of the fermionic species, valued in `T.FermionModule`, with the Lorentz group, the jets of gauge transformations and the gauge algebra all acting species by species. It exists only for a*

family degenerate in mass dimension: `w` is the common mass weight of the species and `h` the proof that they all have it, which for the Standard Model, and for any theory whose fermions are Weyl spinors, is weight three.

```
noncomputable def fermionMatterField (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w) :
    MatterField jets
```

lemma fermionMatterField_V [source]

```
lemma fermionMatterField_V (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w) :
    (T.fermionMatterField w h).V = T.FermionModule
```

lemma fermionMatterField_massWeight [source]

The fermionic matter field carries the common mass weight of the species.

```
@[simp]
lemma fermionMatterField_massWeight (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w) :
    (T.fermionMatterField w h).massWeight = w
```

lemma fermionMatterField_statistic [source]

The fermionic matter field is fermionic, as every fermionic species is declared to be.

```
@[simp]
lemma fermionMatterField_statistic (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w) :
    (T.fermionMatterField w h).statistic = .fermionic
```

def repLorentzFermionModule [source]

***The Lorentz action on the fermionic module**, acting species by species. This is the Lorentz representation of the fermionic matter field, typed on `T.FermionModule` itself so that it can be spoken of without fixing a common mass weight.*

```
noncomputable def repLorentzFermionModule : Representation ℂ SL(2,ℂ) T.FermionModule
```

lemma repLorentzFermionModule_apply [source]

```
@[simp]
lemma repLorentzFermionModule_apply (Λ : SL(2,ℂ)) (v : T.FermionModule) (i : T.FermionSpecies)
    :
    T.repLorentzFermionModule Λ v i = (T.fermion i).repLorentz Λ (v i)
```

def repJetFermionModule [source]

The action of the jets of gauge transformations on the jets of the fermionic module, acting species by species. Like `repLorentzFermionModule` it is typed on `T.FermionModule` itself, so that it can be spoken of without fixing a common mass weight; the gauge action of a theory whose fermions carry different mass dimensions is perfectly well defined, only its packaging as one `MatterField` is not.

```
noncomputable def repJetFermionModule :
    Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] T.FermionModule)
```

lemma repJetFermionModule_smul [source]

The jet gauge action on the fermionic module is fibrewise, as each species is.

```
lemma repJetFermionModule_smul (U : GJ) (χ : SpaceTimeAlgebra)
    (z : SpaceTimeAlgebra ⊗[ℂ] T.FermionModule) :
    T.repJetFermionModule U (χ • z) = χ • T.repJetFermionModule U z
```

lemma lTensor_projFermionValue_repJetFermionModule [source]

A fermionic species is a subrepresentation of the jet gauge action on the fermionic module: the projection onto its value space intertwines the two actions on the jets.

```
lemma lTensor_projFermionValue_repJetFermionModule (i : T.FermionSpecies) (U : GJ) :
    (LinearMap.lTensor SpaceTimeAlgebra (T.projFermionValue i)).comp (T.repJetFermionModule U)
      = ((T.fermion i).repJet U).comp
        (LinearMap.lTensor SpaceTimeAlgebra (T.projFermionValue i))
```

lemma fermionMatterField_repJet [source]

The jet gauge action of the fermionic matter field is that of the fermionic module.

```
lemma fermionMatterField_repJet (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w) :
    (T.fermionMatterField w h).repJet = T.repJetFermionModule
```

lemma fermionMatterField_repLorentz [source]

The Lorentz action of the fermionic matter field is that of the fermionic module.

```
lemma fermionMatterField_repLorentz (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w) :
    (T.fermionMatterField w h).repLorentz = T.repLorentzFermionModule
```

lemma fermionMatterField_repLorentz_apply [source]

The Lorentz group acts on a fermionic configuration species by species.

```
@[simp]
lemma fermionMatterField_repLorentz_apply (w : ℕ)
    (h : ∀ i, (T.fermion i).massWeight = w) (Λ : SL(2,ℂ)) (v : T.FermionModule)
    (i : T.FermionSpecies) :
    (T.fermionMatterField w h).repLorentz Λ v i = (T.fermion i).repLorentz Λ (v i)
```

lemma fermionMatterField_repAlgebra_apply [source]

The gauge algebra acts on a fermionic configuration species by species.

```
@[simp]
lemma fermionMatterField_repAlgebra_apply (w : ℕ)
    (h : ∀ i, (T.fermion i).massWeight = w) (c : 𝔤) (v : T.FermionModule)
    (i : T.FermionSpecies) :
    (T.fermionMatterField w h).repAlgebra c v i = (T.fermion i).repAlgebra c (v i)
```

lemma fermionMatterField_repJet_apply [source]

The jets of gauge transformations act on the jets of the fermionic field species by species, through the identification `jetPiEquiv` of the jets of the fermionic module with the family of the jets of the species.

```
lemma fermionMatterField_repJet_apply (w : ℕ)
    (h : ∀ i, (T.fermion i).massWeight = w) (U : GJ)
    (z : SpaceTimeAlgebra ⊗[ℂ] T.FermionModule) :
    (T.fermionMatterField w h).repJet U z =
      (jetPiEquiv T.FermionValue).symm
        (fun i => (T.fermion i).repJet U (jetPiEquiv T.FermionValue z i))
```

lemma repCoeff_fermionMatterField [source]

The base-point Taylor coefficients of the fermionic jet action are those of the species, index by index.

```
lemma repCoeff_fermionMatterField (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w) (U : GJ)
    (x : Multiset (Fin 1 ⊕ Fin 3)) :
    GaugeAlgebraRealization.repCoeff (T.fermionMatterField w h).repJet U x =
      LinearMap.piMap fun i =>
        GaugeAlgebraRealization.repCoeff (T.fermion i).repJet U x
```

lemma fermionMatterField_repLorentz_inclFermionValue [source]

***A fermionic species is a Lorentz subrepresentation of the fermionic matter field**: including a value of one species and then transforming is transforming and then including. Assembling the species into one field therefore loses no Lorentz information.*

```
lemma fermionMatterField_repLorentz_inclFermionValue (w : ℕ)
    (h : ∀ i, (T.fermion i).massWeight = w) (Λ : SL(2,ℂ)) (i : T.FermionSpecies)
    (x : T.FermionValue i) :
    (T.fermionMatterField w h).repLorentz Λ (T.inclFermionValue i x)
      = T.inclFermionValue i ((T.fermion i).repLorentz Λ x)
```

lemma fermionMatterField_repAlgebra_inclFermionValue [source]

***A fermionic species is a subrepresentation of the gauge algebra action** on the fermionic matter field, for the same reason: the gauge algebra does not mix the species.*

```
lemma fermionMatterField_repAlgebra_inclFermionValue (w : ℕ)
    (h : ∀ i, (T.fermion i).massWeight = w) (c : 𝔤) (i : T.FermionSpecies)
    (x : T.FermionValue i) :
    (T.fermionMatterField w h).repAlgebra c (T.inclFermionValue i x)
      = T.inclFermionValue i ((T.fermion i).repAlgebra c x)
```

lemma finrank_fermionMatterField [source]

The dimension of the fermionic matter field is the sum of the dimensions of the species.

```
lemma finrank_fermionMatterField (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w) :
    Module.finrank ℂ (T.fermionMatterField w h).V
      = ∑ i, Module.finrank ℂ (T.FermionValue i)
```

abbrev projFermionField [source]

The projection of the fermionic matter field onto one species, typed as a map out of `(T.fermionMatterField w h).V` rather than out of `T.FermionModule`. The two are the same type by definition, but naming the first keeps unification from having to unfold the direct sum every time the projection meets the matter field.

```
noncomputable abbrev projFermionField (w : ℕ) (h : ∀ i, (T.fermion i).massWeight = w)
    (i : T.FermionSpecies) : (T.fermionMatterField w h).V →ₗ[ℂ] (T.fermion i).V
```

2.197 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.GaugeFieldData.FermionModule

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/GaugeFieldData/FermionModule.lean on GitHub.

abbrev FermionModule [source]

***The fermionic module of a gauge theory**: the product, over the fermionic species, of their value spaces. One element of it is a value of the entire fermionic content of the theory — the column of all the fermion fields — as opposed to `GaugeFieldData.FermionGenerators`, which is a space of component *functions* and is infinite-dimensional. Since the species type is finite the product is also their direct sum.*

```
abbrev FermionModule : Type
```

abbrev projFermionValue [source]

The component of a fermionic configuration in one species.

```
abbrev projFermionValue (i : T.FermionSpecies) :
    T.FermionModule →ₗ[ℂ] T.FermionValue i
```

abbrev inclFermionValue [source]

The inclusion of one fermionic species into the fermionic module, extending a value of that species by zero in every other.

```
abbrev inclFermionValue (i : T.FermionSpecies) :
    T.FermionValue i →ₗ[ℂ] T.FermionModule
```

lemma projFermionValue_apply [source]

```
@[simp]
lemma projFermionValue_apply (i : T.FermionSpecies) (v : T.FermionModule) :
    T.projFermionValue i v = v i
```

lemma projFermionValue_inclFermionValue_self [source]

```
lemma projFermionValue_inclFermionValue_self (i : T.FermionSpecies)
    (v : T.FermionValue i) : T.projFermionValue i (T.inclFermionValue i v) = v
```

lemma projFermionValue_inclFermionValue_of_ne [source]

A value of one species has no component in any other species: the inclusions of distinct species have disjoint supports.

```
lemma projFermionValue_inclFermionValue_of_ne {i j : T.FermionSpecies} (h : i ≠ j)
    (v : T.FermionValue j) : T.projFermionValue i (T.inclFermionValue j v) = 0
```

lemma sum_inclFermionValue_proj [source]

***A fermionic configuration is the sum of its species components**. The product over the species is their direct sum, the species type being finite, so nothing is lost in describing the fermionic content of the theory by one module.*

```
lemma sum_inclFermionValue_proj (v : T.FermionModule) :
    ∑ i, T.inclFermionValue i (v i) = v
```

lemma fermionModule_hom_ext [source]

Two linear maps out of the fermionic module agreeing on every species are equal.

```
lemma fermionModule_hom_ext {N : Type} [AddCommGroup N] [Module ℂ N]
    {F F' : T.FermionModule →ₗ[ℂ] N}
    (h : ∀ i, F.comp (T.inclFermionValue i) = F'.comp (T.inclFermionValue i)) : F = F'
```

lemma finrank_fermionModule [source]

***The dimension of the fermionic module is the sum of the dimensions of the species**, each species contributing the number of complex components of its multiplet.*

```
lemma finrank_fermionModule :
    Module.finrank ℂ T.FermionModule = ∑ i, Module.finrank ℂ (T.FermionValue i)
```

2.198 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.Basic

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalCovField/Basic.lean on GitHub.

def covGenerators [source]

The covariant generators of the local field algebra: the included field-strength tower and the covariant matter towers of every species, unconjugated and conjugate, along all ordered tuples of directions.

```
def covGenerators : Set T.LocalFieldAlgebra
```

def LocalCovFieldAlgebra [source]

The covariant field algebra of a field datum: the complex unital subalgebra of its local field algebra generated by the covariant matter towers of every species, their conjugates, and the included field-strength tower.

```
noncomputable def LocalCovFieldAlgebra : Subalgebra ℂ T.LocalFieldAlgebra
```

lemma covDerivFieldStrength_mem [source]

```
lemma covDerivFieldStrength_mem (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) : T.covDerivFieldStrength l μ ν φ ∈ T.LocalCovFieldAlgebra
```

lemma covDerivFermion_mem [source]

```
lemma covDerivFermion_mem (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i)) : T.covDerivFermion i l φ ∈ T.LocalCovFieldAlgebra
```

lemma covDerivConjFermion_mem [source]

```
lemma covDerivConjFermion_mem (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.covDerivConjFermion i l φ ∈ T.LocalCovFieldAlgebra
```

lemma covDerivBoson_mem [source]

```
lemma covDerivBoson_mem (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)) : T.covDerivBoson j l φ ∈ T.LocalCovFieldAlgebra
```

lemma covDerivConjBoson_mem [source]

```
lemma covDerivConjBoson_mem (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    T.covDerivConjBoson j l φ ∈ T.LocalCovFieldAlgebra
```

lemma fermionSymbol_zero_mem [source]

The undifferentiated matter symbols lie in the covariant field algebra, as the case of zero covariant derivatives.

```
lemma fermionSymbol_zero_mem (i : T.FermionSpecies) (φ : Module.Dual ℂ (T.FermionValue i)) :
    T.fermionSymbol i 0 φ ∈ T.LocalCovFieldAlgebra
```

lemma conjFermionSymbol_zero_mem [source]

```
lemma conjFermionSymbol_zero_mem (i : T.FermionSpecies)
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.conjFermionSymbol i 0 φ ∈ T.LocalCovFieldAlgebra
```

lemma bosonSymbol_zero_mem [source]

```
lemma bosonSymbol_zero_mem (j : T.BosonSpecies) (φ : Module.Dual ℂ (T.BosonValue j)) :
    T.bosonSymbol j 0 φ ∈ T.LocalCovFieldAlgebra
```

lemma conjBosonSymbol_zero_mem [source]

```
lemma conjBosonSymbol_zero_mem (j : T.BosonSpecies)
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    T.conjBosonSymbol j 0 φ ∈ T.LocalCovFieldAlgebra
```

lemma covGenerators_cases [source]

Case analysis on the covariant generators.

```
lemma covGenerators_cases {P : T.LocalFieldAlgebra → Prop} {b : T.LocalFieldAlgebra}
    (hb : b ∈ T.covGenerators)
    (hF : ∀ (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
      P (T.covDerivFieldStrength l μ ν φ))
    (hψ : ∀ (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (T.FermionValue i)), P (T.covDerivFermion i l φ))
    (hψc : ∀ (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))), P (T.covDerivConjFermion i l φ))
    (hφ : ∀ (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (T.BosonValue j)), P (T.covDerivBoson j l φ))
    (hφc : ∀ (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))), P (T.covDerivConjBoson j l φ)) :
    P b
```

lemma induction [source]

The generation principle: a property holding on the covariant generators and on the scalars, and closed under sums and products, holds on the whole covariant field algebra.

```
lemma induction {P : T.LocalFieldAlgebra → Prop} {x : T.LocalFieldAlgebra}
    (hx : x ∈ T.LocalCovFieldAlgebra)
    (hF : ∀ (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
      P (T.covDerivFieldStrength l μ ν φ))
    (hψ : ∀ (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (T.FermionValue i)), P (T.covDerivFermion i l φ))
    (hψc : ∀ (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))), P (T.covDerivConjFermion i l φ))
    (hφ : ∀ (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (T.BosonValue j)), P (T.covDerivBoson j l φ))
    (hφc : ∀ (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))), P (T.covDerivConjBoson j l φ))
    (halg : ∀ z : ℂ, P (z • (1 : T.LocalFieldAlgebra)))
    (hadd : ∀ x y, P x → P y → P (x + y))
    (hmul : ∀ x y, P x → P y → P (x * y)) : P x
```

lemma mapsTo [source]

An algebra endomorphism carrying the covariant generators into the covariant field algebra carries the whole covariant field algebra into itself.

```
lemma mapsTo (f : T.LocalFieldAlgebra →ₐ[ℂ] T.LocalFieldAlgebra)
    (hgen : ∀ b ∈ T.covGenerators, f b ∈ T.LocalCovFieldAlgebra)
    {x : T.LocalFieldAlgebra} (hx : x ∈ T.LocalCovFieldAlgebra) :
    f x ∈ T.LocalCovFieldAlgebra
```

lemma algHom_ext [source]

Two algebra maps out of the covariant field algebra agreeing on the covariant generators are equal. This is uniqueness only: the covariant field algebra is not free on its generators, so an assignment of their images does not by itself define a map.

```
lemma algHom_ext {B : Type} [Semiring B] [Algebra ℂ B]
```

```
    {f g : 𝓣.LocalCovFieldAlgebra →ₐ[ℂ] B}
    (h : ∀ b (hb : b ∈ T.covGenerators), f ⟨b, Algebra.subset_adjoin hb⟩
      = g ⟨b, Algebra.subset_adjoin hb⟩) : f = g
```

def covFieldStrength [source]

The included field-strength tower `∇_l F_μν^φ`, as elements of the covariant field algebra.

```
noncomputable def covFieldStrength (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) : T.LocalCovFieldAlgebra
```

def covFermion [source]

The covariant tower `∇_l ψ^φ` of a fermionic species, as elements of the covariant field algebra.

```
noncomputable def covFermion (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (T.FermionValue i) →ₗ[ℂ] T.LocalCovFieldAlgebra where
```

def covConjFermion [source]

The conjugate covariant tower of a fermionic species, as elements of the covariant field algebra.

```
noncomputable def covConjFermion (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule (T.FermionValue i)) →ₗ[ℂ] T.LocalCovFieldAlgebra where
```

def covBoson [source]

The covariant tower of a bosonic species, as elements of the covariant field algebra.

```
noncomputable def covBoson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (T.BosonValue j) →ₗ[ℂ] T.LocalCovFieldAlgebra where
```

def covConjBoson [source]

The conjugate covariant tower of a bosonic species, as elements of the covariant field algebra.

```
noncomputable def covConjBoson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule (T.BosonValue j)) →ₗ[ℂ] T.LocalCovFieldAlgebra where
```

lemma coe_covFieldStrength [source]

```
@[simp]
lemma coe_covFieldStrength (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    (covFieldStrength T l μ ν φ : T.LocalFieldAlgebra) = T.covDerivFieldStrength l μ ν φ
```

lemma coe_covFermion [source]

```
@[simp]
lemma coe_covFermion (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i)) :
    (covFermion T i l φ : T.LocalFieldAlgebra) = T.covDerivFermion i l φ
```

lemma coe_covConjFermion [source]

```
@[simp]
lemma coe_covConjFermion (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    (covConjFermion T i l φ : T.LocalFieldAlgebra) = T.covDerivConjFermion i l φ
```

lemma coe_covBoson [source]

```
@[simp]
lemma coe_covBoson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)) :
    (covBoson T j l φ : T.LocalFieldAlgebra) = T.covDerivBoson j l φ
```

lemma coe_covConjBoson [source]

```
@[simp]
lemma coe_covConjBoson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    (covConjBoson T j l φ : T.LocalFieldAlgebra) = T.covDerivConjBoson j l φ
```

lemma algHom_ext_towers [source]

Two algebra maps out of the covariant field algebra agreeing on the five towers are equal. Like `GaugeFieldData.LocalCovFieldAlgebra.algHom_ext` this is uniqueness only.

```
lemma algHom_ext_towers {B : Type} [Semiring B] [Algebra ℂ B]
    {f g : 𝓩T.LocalCovFieldAlgebra →ₐ[ℂ] B}
    (hF : ∀ (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
      f (covFieldStrength T l μ ν φ) = g (covFieldStrength T l μ ν φ))
    (hψ : ∀ (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (T.FermionValue i)),
      f (covFermion T i l φ) = g (covFermion T i l φ))
    (hψc : ∀ (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))),
      f (covConjFermion T i l φ) = g (covConjFermion T i l φ))
    (hφ : ∀ (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (T.BosonValue j)),
      f (covBoson T j l φ) = g (covBoson T j l φ))
    (hφc : ∀ (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))),
      f (covConjBoson T j l φ) = g (covConjBoson T j l φ)) : f = g
```

lemma repJet_mem [source]

The jet gauge group preserves the covariant field algebra: a jet carries each generator to a generator of the same tower.

```
lemma repJet_mem (U : GJ) {x : T.LocalFieldAlgebra} (hx : x ∈ T.LocalCovFieldAlgebra) :
    T.repJet U x ∈ T.LocalCovFieldAlgebra
```

lemma repLorentzGroup_mem [source]

The Lorentz group preserves the covariant field algebra under `GaugeFieldData.GaugeLorentzCompatible`: the generators mix among themselves.

```
lemma repLorentzGroup_mem (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ))
    {x : T.LocalFieldAlgebra} (hx : x ∈ T.LocalCovFieldAlgebra) :
    T.repLorentzGroup Λ x ∈ T.LocalCovFieldAlgebra
```

def repJet [source]

The action of the jet gauge group on the covariant field algebra, restricted from the local field algebra.

```
noncomputable def repJet : Representation ℂ GJ T.LocalCovFieldAlgebra
```

lemma coe_repJet [source]

```
@[simp]
lemma coe_repJet (U : GJ) (x : T.LocalCovFieldAlgebra) :
    (repJet T U x : T.LocalFieldAlgebra) = T.repJet U x
```

lemma repJet_apply_mul [source]

```
lemma repJet_apply_mul (U : GJ) (x y : T.LocalCovFieldAlgebra) :
    repJet T U (x * y) = repJet T U x * repJet T U y
```

def repLorentzGroup [source]

The action of the Lorentz group on the covariant field algebra, restricted from the local field algebra under `GaugeFieldData.GaugeLorentzCompatible`.

```
noncomputable def repLorentzGroup (hGL : T.GaugeLorentzCompatible) :
    Representation ℂ SL(2,ℂ) T.LocalCovFieldAlgebra
```

lemma coe_repLorentzGroup [source]

```
@[simp]
lemma coe_repLorentzGroup (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ))
    (x : T.LocalCovFieldAlgebra) :
    (repLorentzGroup T hGL Λ x : T.LocalFieldAlgebra) = T.repLorentzGroup Λ x
```

lemma repLorentzGroup_apply_mul [source]

```
lemma repLorentzGroup_apply_mul (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ))
    (x y : T.LocalCovFieldAlgebra) :
    repLorentzGroup T hGL Λ (x * y) = repLorentzGroup T hGL Λ x * repLorentzGroup T hGL Λ y
```

def repValue [source]

The action of the ordinary gauge group on the covariant field algebra: the jet action at the constant jets.

```
noncomputable def repValue : Representation ℂ G₀ T.LocalCovFieldAlgebra
```

lemma coe_repValue [source]

```
@[simp]
lemma coe_repValue (g : G₀) (x : T.LocalCovFieldAlgebra) :
    (repValue T g x : T.LocalFieldAlgebra) = T.repJet (jets.ofConstant g) x
```

lemma repValue_apply_mul [source]

```
lemma repValue_apply_mul (g : G₀) (x y : T.LocalCovFieldAlgebra) :
    repValue T g (x * y) = repValue T g x * repValue T g y
```

lemma repJet_eq_ofConstant_eval_of_mem [source]

Under `GaugeFieldData.PureJetsActTrivially`, a jet acts on every element of the covariant field algebra as the constant jet of its value, stated in the local field algebra.

```
lemma repJet_eq_ofConstant_eval_of_mem (hP : T.PureJetsActTrivially) (U : GJ)
    {x : T.LocalFieldAlgebra} (hx : x ∈ T.LocalCovFieldAlgebra) :
    T.repJet U x = T.repJet (jets.ofConstant (jets.eval U)) x
```

lemma repJet_eq_repValue_eval [source]

Under `GaugeFieldData.PureJetsActTrivially`, the action of the jet gauge group on the covariant field algebra factors through evaluation: a jet acts as the constant jet of its value, so the derivatives of a gauge transformation act trivially on covariant expressions.

```
lemma repJet_eq_repValue_eval (hP : T.PureJetsActTrivially) (U : GJ)
    (x : T.LocalCovFieldAlgebra) : repJet T U x = repValue T (jets.eval U) x
```

lemma repValue_covFieldStrength [source]

The ordinary gauge group rotates the adjoint index of the field-strength tower through the dual adjoint action of the inverse.

```
lemma repValue_covFieldStrength (g : G₀) (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repValue T g (covFieldStrength T l μ ν φ)
      = covFieldStrength T l μ ν ((jets.adjoint g⁻¹).dualMap φ)
```

lemma repJet_covFieldStrength [source]

A jet acts on the field-strength tower through the value of its inverse alone.

```
lemma repJet_covFieldStrength (U : GJ) (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    repJet T U (covFieldStrength T l μ ν φ)
      = covFieldStrength T l μ ν ((jets.adjoint (jets.eval U⁻¹)).dualMap φ)
```

lemma repJet_covFermion [source]

A jet acts on the covariant tower of a fermionic species through the zeroth dual Taylor coefficient of its inverse on the value index.

```
lemma repJet_covFermion (U : GJ) (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i)) :
    repJet T U (covFermion T i l φ)
      = covFermion T i l (repDualCoeff (T.fermion i).repJet U⁻¹ 0 φ)
```

lemma repJet_covConjFermion [source]

```
lemma repJet_covConjFermion (U : GJ) (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    repJet T U (covConjFermion T i l φ)
      = covConjFermion T i l
          (repDualCoeff (JetComponentSpace.repConj (T.fermion i).repJet) U⁻¹ 0 φ)
```

lemma repJet_covBoson [source]

```
lemma repJet_covBoson (U : GJ) (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)) :
    repJet T U (covBoson T j l φ) = covBoson T j l (repDualCoeff (T.boson j).repJet U⁻¹ 0 φ)
```

lemma repJet_covConjBoson [source]

```
lemma repJet_covConjBoson (U : GJ) (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    repJet T U (covConjBoson T j l φ)
      = covConjBoson T j l
          (repDualCoeff (JetComponentSpace.repConj (T.boson j).repJet) U⁻¹ 0 φ)
```

lemma repLorentzGroup_covFieldStrength [source]

The Lorentz law of the field-strength tower in the covariant field algebra: every covariant slot and both covector indices mix by the columns of the Lorentz matrix.

```
lemma repLorentzGroup_covFieldStrength (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ))
    {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repLorentzGroup T hGL Λ (covFieldStrength T (List.ofFn l) μ ν φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ k, L[Λ] (p k) (l k)) •
          ∑ a, L[Λ] a μ • ∑ b, L[Λ] b ν • covFieldStrength T (List.ofFn p) a b φ
```

lemma repLorentzGroup_covFermion [source]

The Lorentz law of the covariant tower of a fermionic species in the covariant field algebra: every covariant slot mixes by the columns of the Lorentz matrix and the value index transforms contragrediently.

```
lemma repLorentzGroup_covFermion (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ))
    (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i)) :
```

```
    repLorentzGroup T hGL Λ (covFermion T i l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ k, L[Λ] (p k) (l k)) •
          covFermion T i p ((T.fermion i).repLorentz.dual Λ φ)
```

lemma repLorentzGroup_covConjFermion [source]

```
lemma repLorentzGroup_covConjFermion (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ))
    (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    repLorentzGroup T hGL Λ (covConjFermion T i l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ k, L[Λ] (p k) (l k)) •
          covConjFermion T i p ((T.fermion i).repLorentz.conj.dual Λ φ)
```

lemma repLorentzGroup_covBoson [source]

```
lemma repLorentzGroup_covBoson (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ))
    (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)) :
    repLorentzGroup T hGL Λ (covBoson T j l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ k, L[Λ] (p k) (l k)) •
          covBoson T j p ((T.boson j).repLorentz.dual Λ φ)
```

lemma repLorentzGroup_covConjBoson [source]

```
lemma repLorentzGroup_covConjBoson (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ))
    (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    repLorentzGroup T hGL Λ (covConjBoson T j l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ k, L[Λ] (p k) (l k)) •
          covConjBoson T j p ((T.boson j).repLorentz.conj.dual Λ φ)
```

2.199 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.GaugeSector

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalCovField/GaugeSector.lean on GitHub.

def includeCovConnection [source]

The complexified covariant gauge-only algebra inside the local field algebra: the complexification of the real covariant subalgebra inclusion, followed by the connection inclusion.

```
noncomputable def includeCovConnection :
    (ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝔤) →ₐ[ℂ] T.LocalFieldAlgebra
```

lemma includeCovConnection_apply [source]

```
@[simp]
lemma includeCovConnection_apply (y : ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝔤) :
    T.includeCovConnection y
      = T.includeConnection (LocalGaugeCovFieldAlgebra.complexVal 𝔤 y)
```

lemma includeCovConnection_one_tmul [source]

```
lemma includeCovConnection_one_tmul (x : LocalGaugeCovFieldAlgebra 𝔤) :
    T.includeCovConnection ((1 : ℂ) ⊗ₜ[ℝ] x)
      = T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ] (x : LocalGaugeFieldAlgebra 𝔤))
```

lemma includeCovConnection_injective [source]

The complexified covariant gauge-only algebra embeds in the local field algebra.

```
lemma includeCovConnection_injective : Function.Injective T.includeCovConnection
```

lemma range_includeCovConnection [source]

The range of the inclusion is the covariant gauge sector.

```
lemma range_includeCovConnection :
    T.includeCovConnection.range = T.CovSectorAlgebra {FieldCategory.gauge}
```

lemma includeCovConnection_mem_covGaugeSector [source]

```
lemma includeCovConnection_mem_covGaugeSector (y : ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝔤) :
    T.includeCovConnection y ∈ T.CovSectorAlgebra {FieldCategory.gauge}
```

def covGaugeSectorEquiv [source]

The covariant gauge sector is the complexification of the covariant gauge-only algebra: the inclusion, corestricted to its range.

```
noncomputable def covGaugeSectorEquiv :
    (ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝔤) ≃ₐ[ℂ] ↥(T.CovSectorAlgebra {FieldCategory.gauge})
```

lemma coe_covGaugeSectorEquiv [source]

In the local field algebra the equivalence is the canonical inclusion.

```
@[simp]
lemma coe_covGaugeSectorEquiv (y : ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝔤) :
    (T.covGaugeSectorEquiv y : T.LocalFieldAlgebra) = T.includeCovConnection y
```

lemma covGaugeSectorEquiv_symm_includeCovConnection [source]

The inverse recovers an element from its inclusion.

```
lemma covGaugeSectorEquiv_symm_includeCovConnection (y : ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝔤)
    (hy : T.includeCovConnection y ∈ T.CovSectorAlgebra {FieldCategory.gauge}) :
    T.covGaugeSectorEquiv.symm ⟨T.includeCovConnection y, hy⟩ = y
```

lemma coe_covGaugeSectorEquiv_one_tmul_covF [source]

A generator, embedded with the scalar `1`, is the included tower element with the same derivative labels, spacetime indices and adjoint covector.

```
lemma coe_covGaugeSectorEquiv_one_tmul_covF (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    (T.covGaugeSectorEquiv ((1 : ℂ) ⊗ₜ[ℝ] LocalGaugeCovFieldAlgebra.covF 𝔤 l μ ν φ) :
        T.LocalFieldAlgebra) = T.covDerivFieldStrength l μ ν φ
```

lemma coe_covGaugeSectorEquiv_one_tmul_covF_nil [source]

Zero covariant derivatives: the included field strength itself.

```
lemma coe_covGaugeSectorEquiv_one_tmul_covF_nil (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    (T.covGaugeSectorEquiv ((1 : ℂ) ⊗ₜ[ℝ] LocalGaugeCovFieldAlgebra.covF 𝔤 [] μ ν φ) :
        T.LocalFieldAlgebra)
      = T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ] LocalGaugeFieldAlgebra.fieldStrength 𝔤 μ ν φ)
```

lemma coe_covGaugeSectorEquiv_tmul [source]

`z ⊗ₜ x` maps to `z` times the image of `1 ⊗ₜ x`.

```
lemma coe_covGaugeSectorEquiv_tmul (z : ℂ) (x : LocalGaugeCovFieldAlgebra 𝔤) :
    (T.covGaugeSectorEquiv (z ⊗ₜ[ℝ] x) : T.LocalFieldAlgebra)
      = z • (T.covGaugeSectorEquiv ((1 : ℂ) ⊗ₜ[ℝ] x) : T.LocalFieldAlgebra)
```

lemma covGaugeSectorEquiv_complexRepValue [source]

The equivalence intertwines the complexified gauge-only action of the ordinary gauge group with the value action of the covariant gauge sector.

```
lemma covGaugeSectorEquiv_complexRepValue (g : G₀) (y : ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝔤) :
    T.covGaugeSectorEquiv (LocalGaugeCovFieldAlgebra.complexRepValue jets g y)
      = CovSectorAlgebra.repValue T {FieldCategory.gauge} g (T.covGaugeSectorEquiv y)
```

lemma covGaugeSectorEquiv_complexRepLorentzGroup [source]

The equivalence intertwines the complexified gauge-only Lorentz action with the Lorentz action of the covariant gauge sector.

```
lemma covGaugeSectorEquiv_complexRepLorentzGroup (hGL : T.GaugeLorentzCompatible)
    (Λ : SL(2,ℂ)) (y : ℂ ⊗[ℝ] LocalGaugeCovFieldAlgebra 𝔤) :
    T.covGaugeSectorEquiv (LocalGaugeCovFieldAlgebra.complexRepLorentzGroup 𝔤 Λ y)
      = CovSectorAlgebra.repLorentzGroup T {FieldCategory.gauge} hGL Λ
          (T.covGaugeSectorEquiv y)
```

2.200 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.GaugeSectorRealization

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalCovField/GaugeSectorRealization.lean on GitHub.

def covGaugeSectorRealizationEquiv [source]

Realizations of the covariant gauge sector are the real realizations of the covariant gauge-only algebra, the target keeping its complex actions restricted to `ℝ`.

```
noncomputable def covGaugeSectorRealizationEquiv (hGL : T.GaugeLorentzCompatible) :
    CovSectorAlgebra.Realization T {FieldCategory.gauge} hGL B repGauge repLorentz
```

```
      ≃ LocalGaugeCovFieldAlgebra.Realization jets B (repGauge.restrictScalars ℝ)
          (repLorentz.restrictScalars ℝ)
```

lemma covGaugeSectorRealizationEquiv_toAlgHom [source]

```
lemma covGaugeSectorRealizationEquiv_toAlgHom
    (k : CovSectorAlgebra.Realization T {FieldCategory.gauge} hGL B repGauge repLorentz) :
    (T.covGaugeSectorRealizationEquiv hGL k).toAlgHom
      = ((k.toAlgHom.comp T.covGaugeSectorEquiv.toAlgHom).restrictScalars ℝ).comp
          Algebra.TensorProduct.includeRight
```

lemma covGaugeSectorRealizationEquiv_toAlgHom_apply [source]

```
@[simp]
lemma covGaugeSectorRealizationEquiv_toAlgHom_apply
    (k : CovSectorAlgebra.Realization T {FieldCategory.gauge} hGL B repGauge repLorentz)
    (x : LocalGaugeCovFieldAlgebra 𝔤) :
    (T.covGaugeSectorRealizationEquiv hGL k).toAlgHom x
      = k.toAlgHom (T.covGaugeSectorEquiv ((1 : ℂ) ⊗ₜ[ℝ] x))
```

lemma covGaugeSectorRealizationEquiv_F [source]

The field-strength tower of the corresponding real realization is the sector tower, with the same derivative labels, spacetime indices and adjoint covector.

```
lemma covGaugeSectorRealizationEquiv_F
    (k : CovSectorAlgebra.Realization T {FieldCategory.gauge} hGL B repGauge repLorentz)
    (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    (T.covGaugeSectorRealizationEquiv hGL k).F l μ ν φ
      = k.toAlgHom
          (T.covGaugeSectorEquiv ((1 : ℂ) ⊗ₜ[ℝ] LocalGaugeCovFieldAlgebra.covF 𝔤 l μ ν φ))
```

lemma covGaugeSectorRealizationEquiv_symm_toAlgHom [source]

```
lemma covGaugeSectorRealizationEquiv_symm_toAlgHom
    (h : LocalGaugeCovFieldAlgebra.Realization jets B (repGauge.restrictScalars ℝ)
      (repLorentz.restrictScalars ℝ)) :
    ((T.covGaugeSectorRealizationEquiv hGL).symm h).toAlgHom
      = (AlgHom.liftEquiv ℝ ℂ (LocalGaugeCovFieldAlgebra 𝔤) B h.toAlgHom).comp
          T.covGaugeSectorEquiv.symm.toAlgHom
```

lemma covGaugeSectorRealizationEquiv_symm_toAlgHom_apply [source]

```
@[simp]
lemma covGaugeSectorRealizationEquiv_symm_toAlgHom_apply
    (h : LocalGaugeCovFieldAlgebra.Realization jets B (repGauge.restrictScalars ℝ)
      (repLorentz.restrictScalars ℝ)) (x : T.CovSectorAlgebra {FieldCategory.gauge}) :
    ((T.covGaugeSectorRealizationEquiv hGL).symm h).toAlgHom x
      = AlgHom.liftEquiv ℝ ℂ (LocalGaugeCovFieldAlgebra 𝔤) B h.toAlgHom
          (T.covGaugeSectorEquiv.symm x)
```

lemma covGaugeSectorRealizationEquiv_symm_toAlgHom_covF [source]

The sector towers of the corresponding sector realization are the tower of the real realization; at `l = []` this is the field strength.

```
lemma covGaugeSectorRealizationEquiv_symm_toAlgHom_covF
    (h : LocalGaugeCovFieldAlgebra.Realization jets B (repGauge.restrictScalars ℝ)
      (repLorentz.restrictScalars ℝ)) (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    ((T.covGaugeSectorRealizationEquiv hGL).symm h).toAlgHom
        (T.covGaugeSectorEquiv ((1 : ℂ) ⊗ₜ[ℝ] LocalGaugeCovFieldAlgebra.covF 𝔤 l μ ν φ))
      = h.F l μ ν φ
```

lemma covGaugeSectorRealizationEquiv_restrictCovSector [source]

Restricting a realization of the local field algebra to the covariant gauge sector and passing to the gauge-only side gives the covariant restriction of its ordinary gauge-only realization.

```
lemma covGaugeSectorRealizationEquiv_restrictCovSector
    {repJet : Representation ℂ GJ B} (h : Realization T B repJet repLorentz) :
    T.covGaugeSectorRealizationEquiv hGL (h.restrictCovSector hGL {FieldCategory.gauge})
      = (T.gaugeSectorRealizationEquiv (h.restrictSector {FieldCategory.gauge})).restrict
```

2.201 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.Realization

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalCovField/Realization.lean on GitHub.

abbrev Realization [source]

A complex algebra `B` carrying the covariant expressions of the datum `T`: a complex algebra map out of the covariant field algebra, equivariant for the ordinary gauge group and the Lorentz group, both acting on the whole of `B` by algebra endomorphisms. The Lorentz action of the source needs `GaugeFieldData.GaugeLorentzCompatible`, which is a parameter; no other species condition is used. It is built from the fields `toAlgHom`, `map_fst`, `map_snd`, `fst_mul`, `snd_mul` of `Representation.EquivariantAlgHom`, which the lemmas `map_repValue`, `map_repLorentz`, `repGauge_mul` and `repLorentz_mul` name.

```
abbrev Realization (T : GaugeFieldData jets) (hGL : T.GaugeLorentzCompatible) (B : Type)
    [Semiring B] [Algebra ℂ B] (repGauge : Representation ℂ G₀ B)
    (repLorentz : Representation ℂ SL(2,ℂ) B)
```

def id [source]

The covariant field algebra realized in itself, by the identity.

```
noncomputable def id : Realization T hGL (↥T.LocalCovFieldAlgebra) (repValue T)
    (repLorentzGroup T hGL)
```

lemma id_toAlgHom [source]

```
@[simp]
lemma id_toAlgHom : (id T hGL).toAlgHom = AlgHom.id ℂ ↥T.LocalCovFieldAlgebra
```

lemma map_repValue [source]

The map is equivariant for the ordinary gauge group.

```
lemma map_repValue (g : G₀) (x : 𝓣.LocalCovFieldAlgebra) :
    k.toAlgHom (repValue T g x) = repGauge g (k.toAlgHom x)
```

lemma map_repLorentz [source]

The map is equivariant for the Lorentz group.

```
lemma map_repLorentz (Λ : SL(2,ℂ)) (x : 𝓣.LocalCovFieldAlgebra) :
    k.toAlgHom (repLorentzGroup T hGL Λ x) = repLorentz Λ (k.toAlgHom x)
```

lemma repGauge_mul [source]

The ordinary gauge group acts on the whole of `B` by algebra endomorphisms.

```
lemma repGauge_mul (g : G₀) (b₁ b₂ : B) :
    repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂
```

lemma repLorentz_mul [source]

The Lorentz group acts on the whole of `B` by algebra endomorphisms.

```
lemma repLorentz_mul (Λ : SL(2,ℂ)) (b₁ b₂ : B) :
    repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂
```

def fieldStrength [source]

The realized field-strength tower `∇_l F_μν^φ`.

```
noncomputable def fieldStrength (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝓰) : B
```

def fermion [source]

The realized covariant tower `∇_l ψ^φ` of a fermionic species.

```
noncomputable def fermion (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (T.FermionValue i) →ₗ[ℂ] B
```

def conjFermion [source]

The realized conjugate covariant tower of a fermionic species.

```
noncomputable def conjFermion (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule (T.FermionValue i)) →ₗ[ℂ] B
```

def boson [source]

The realized covariant tower of a bosonic species.

```
noncomputable def boson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (T.BosonValue j) →ₗ[ℂ] B
```

def conjBoson [source]

The realized conjugate covariant tower of a bosonic species.

```
noncomputable def conjBoson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule (T.BosonValue j)) →ₗ[ℂ] B
```

lemma fermion_apply [source]

```
lemma fermion_apply (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i)) :
    k.fermion i l φ = k.toAlgHom (covFermion T i l φ)
```

lemma conjFermion_apply [source]

```
lemma conjFermion_apply (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    k.conjFermion i l φ = k.toAlgHom (covConjFermion T i l φ)
```

lemma boson_apply [source]

```
lemma boson_apply (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)) :
    k.boson j l φ = k.toAlgHom (covBoson T j l φ)
```

lemma conjBoson_apply [source]

```
lemma conjBoson_apply (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    k.conjBoson j l φ = k.toAlgHom (covConjBoson T j l φ)
```

lemma id_fermion [source]

```
@[simp]
lemma id_fermion (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    (id T hGL).fermion i l = covFermion T i l
```

lemma id_fieldStrength [source]

```
@[simp]
lemma id_fieldStrength (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    (id T hGL).fieldStrength l μ ν = covFieldStrength T l μ ν
```

lemma ext_towers [source]

Two realizations with the same five towers are equal. This is uniqueness only: the covariant field algebra is not free on its towers, so an assignment of the tower images does not by itself define a realization.

```
lemma ext_towers {k₁ k₂ : Realization T hGL B repGauge repLorentz}
    (hF : ∀ (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤),
      k₁.fieldStrength l μ ν φ = k₂.fieldStrength l μ ν φ)
    (hψ : ∀ (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
```

```
    (φ : Module.Dual ℂ (T.FermionValue i)), k₁.fermion i l φ = k₂.fermion i l φ)
  (hψc : ∀ (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))),
    k₁.conjFermion i l φ = k₂.conjFermion i l φ)
  (hφ : ∀ (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)), k₁.boson j l φ = k₂.boson j l φ)
  (hφc : ∀ (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))),
    k₁.conjBoson j l φ = k₂.conjBoson j l φ) : k₁ = k₂
```

lemma gauge_fieldStrength [source]

The gauge law of the realized field-strength tower: the adjoint index rotates through the dual adjoint action of the inverse.

```
lemma gauge_fieldStrength (g : G₀) (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
  (φ : Module.Dual ℝ 𝔤) :
  repGauge g (k.fieldStrength l μ ν φ)
    = k.fieldStrength l μ ν ((jets.adjoint g⁻¹).dualMap φ)
```

lemma gauge_fermion [source]

The gauge law of a realized matter tower: the value index rotates through the zeroth dual Taylor coefficient of the inverse constant jet.

```
lemma gauge_fermion (g : G₀) (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
  (φ : Module.Dual ℂ (T.FermionValue i)) :
  repGauge g (k.fermion i l φ)
    = k.fermion i l (repDualCoeff (T.fermion i).repJet (jets.ofConstant g)⁻¹ 0 φ)
```

lemma gauge_conjFermion [source]

```
lemma gauge_conjFermion (g : G₀) (i : T.FermionSpecies) {n : ℕ}
  (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
  repGauge g (k.conjFermion i l φ) = k.conjFermion i l
    (repDualCoeff (JetComponentSpace.repConj (T.fermion i).repJet)
      (jets.ofConstant g)⁻¹ 0 φ)
```

lemma gauge_boson [source]

```
lemma gauge_boson (g : G₀) (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
  (φ : Module.Dual ℂ (T.BosonValue j)) :
  repGauge g (k.boson j l φ)
    = k.boson j l (repDualCoeff (T.boson j).repJet (jets.ofConstant g)⁻¹ 0 φ)
```

lemma gauge_conjBoson [source]

```
lemma gauge_conjBoson (g : G₀) (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
  (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
  repGauge g (k.conjBoson j l φ) = k.conjBoson j l
    (repDualCoeff (JetComponentSpace.repConj (T.boson j).repJet)
      (jets.ofConstant g)⁻¹ 0 φ)
```

lemma lorentz_fieldStrength [source]

The Lorentz law of the realized field-strength tower: every covariant slot and both covector indices mix by the columns of the Lorentz matrix.

```
lemma lorentz_fieldStrength (Λ : SL(2,ℂ)) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    repLorentz Λ (k.fieldStrength (List.ofFn l) μ ν φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ a, L[Λ] (p a) (l a)) •
          ∑ a, L[Λ] a μ • ∑ b, L[Λ] b ν • k.fieldStrength (List.ofFn p) a b φ
```

lemma lorentz_fermion [source]

The Lorentz law of a realized matter tower: every covariant slot mixes by the columns of the Lorentz matrix and the value index transforms contragrediently.

```
lemma lorentz_fermion (Λ : SL(2,ℂ)) (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.FermionValue i)) :
    repLorentz Λ (k.fermion i l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ a, L[Λ] (p a) (l a)) •
          k.fermion i p ((T.fermion i).repLorentz.dual Λ φ)
```

lemma lorentz_conjFermion [source]

```
lemma lorentz_conjFermion (Λ : SL(2,ℂ)) (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    repLorentz Λ (k.conjFermion i l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ a, L[Λ] (p a) (l a)) •
          k.conjFermion i p ((T.fermion i).repLorentz.conj.dual Λ φ)
```

lemma lorentz_boson [source]

```
lemma lorentz_boson (Λ : SL(2,ℂ)) (j : T.BosonSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.BosonValue j)) :
    repLorentz Λ (k.boson j l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ a, L[Λ] (p a) (l a)) •
          k.boson j p ((T.boson j).repLorentz.dual Λ φ)
```

lemma lorentz_conjBoson [source]

```
lemma lorentz_conjBoson (Λ : SL(2,ℂ)) (j : T.BosonSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    repLorentz Λ (k.conjBoson j l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ a, L[Λ] (p a) (l a)) •
          k.conjBoson j p ((T.boson j).repLorentz.conj.dual Λ φ)
```

def restrict [source]

The restriction of a realization of the local field algebra to the covariant field algebra, along the inclusion; the ordinary gauge group acts on the target through the constant jets. No species condition beyond `hGL` is used.

```
noncomputable def restrict :
    LocalCovFieldAlgebra.Realization T hGL B (repJet.comp jets.ofConstant) repLorentz
```

lemma restrict_toAlgHom [source]

```
lemma restrict_toAlgHom : (h.restrict hGL).toAlgHom = h.toAlgHom.comp T.LocalCovFieldAlgebra.
    val
```

lemma restrict_toAlgHom_apply [source]

```
@[simp]
lemma restrict_toAlgHom_apply (x : ↥T.LocalCovFieldAlgebra) :
    (h.restrict hGL).toAlgHom x = h.toAlgHom x
```

lemma restrict_id_toAlgHom [source]

```
lemma restrict_id_toAlgHom :
    ((id T).restrict hGL).toAlgHom = T.LocalCovFieldAlgebra.val
```

lemma restrict_fieldStrength [source]

```
lemma restrict_fieldStrength (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    (h.restrict hGL).fieldStrength l μ ν φ = h.toAlgHom (T.covDerivFieldStrength l μ ν φ)
```

lemma restrict_fermion [source]

```
lemma restrict_fermion (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i)) :
    (h.restrict hGL).fermion i l φ = h.toAlgHom (T.covDerivFermion i l φ)
```

lemma restrict_conjFermion [source]

```
lemma restrict_conjFermion (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    (h.restrict hGL).conjFermion i l φ = h.toAlgHom (T.covDerivConjFermion i l φ)
```

lemma restrict_boson [source]

```
lemma restrict_boson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)) :
    (h.restrict hGL).boson j l φ = h.toAlgHom (T.covDerivBoson j l φ)
```

lemma restrict_conjBoson [source]

```
lemma restrict_conjBoson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    (h.restrict hGL).conjBoson j l φ = h.toAlgHom (T.covDerivConjBoson j l φ)
```

lemma restrict_fieldStrength_eq_iteratedCovDerivAdjoint [source]

The field-strength tower of a restriction is the covariant tower of the realized gauge-boson symbols.

```
lemma restrict_fieldStrength_eq_iteratedCovDerivAdjoint (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    (h.restrict hGL).fieldStrength l μ ν φ
      = GaugeAlgebraRealization.iteratedCovDerivAdjoint h.gaugeRealization.A l
        (GaugeAlgebraRealization.fieldStrength h.gaugeRealization.A μ ν) 0 φ
```

lemma restrict_fermion_eq_covDerivIter [source]

A matter tower of a restriction is the covariant tower of the realized symbols of the species, computed against the realized gauge-boson symbols.

```
lemma restrict_fermion_eq_covDerivIter (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.FermionValue i)) :
    (h.restrict hGL).fermion i l φ
      = GaugeAlgebraRealization.covDerivIter h.gaugeRealization.A (T.fermion i).repAlgebra
        (h.fermionSymbol i) n l 0 φ
```

lemma restrict_conjFermion_eq_covDerivIter [source]

```
lemma restrict_conjFermion_eq_covDerivIter (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    (h.restrict hGL).conjFermion i l φ
      = GaugeAlgebraRealization.covDerivIter h.gaugeRealization.A
        (LocalGaugeData.actionConj (T.fermion i).repAlgebra) (h.conjFermionSymbol i) n l 0 φ
```

lemma restrict_boson_eq_covDerivIter [source]

```
lemma restrict_boson_eq_covDerivIter (j : T.BosonSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.BosonValue j)) :
    (h.restrict hGL).boson j l φ
      = GaugeAlgebraRealization.covDerivIter h.gaugeRealization.A (T.boson j).repAlgebra
        (h.bosonSymbol j) n l 0 φ
```

lemma restrict_conjBoson_eq_covDerivIter [source]

```
lemma restrict_conjBoson_eq_covDerivIter (j : T.BosonSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    (h.restrict hGL).conjBoson j l φ
      = GaugeAlgebraRealization.covDerivIter h.gaugeRealization.A
        (LocalGaugeData.actionConj (T.boson j).repAlgebra) (h.conjBosonSymbol j) n l 0 φ
```

lemma restrict_map_repJet [source]

Full jet equivariance survives restriction, with no species condition.

```
lemma restrict_map_repJet (U : GJ) (x : ɪT.LocalCovFieldAlgebra) :
    (h.restrict hGL).toAlgHom (LocalCovFieldAlgebra.repJet T U x)
      = repJet U ((h.restrict hGL).toAlgHom x)
```

lemma repJet_restrict_toAlgHom [source]

Under `GaugeFieldData.PureJetsActTrivially`, a jet acts on the realized covariant image as the constant jet of its value; nothing is assumed about the jet action elsewhere in `B`.

```
lemma repJet_restrict_toAlgHom (hP : T.PureJetsActTrivially) (U : GJ)
    (x : ɪT.LocalCovFieldAlgebra) :
    repJet U ((h.restrict hGL).toAlgHom x)
      = repJet (jets.ofConstant (jets.eval U)) ((h.restrict hGL).toAlgHom x)
```

lemma map_repJet [source]

Under `GaugeFieldData.PureJetsActTrivially`, a covariant realization intertwines the source jet action with the target action of the ordinary gauge group at the value of the jet.

```
lemma map_repJet (k : Realization T hGL B repGauge repLorentz) (hP : T.PureJetsActTrivially)
    (U : GJ) (x : ɪT.LocalCovFieldAlgebra) :
    k.toAlgHom (repJet T U x) = repGauge (jets.eval U) (k.toAlgHom x)
```

2.202 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.Sector

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalCovField/Sector.lean on GitHub.

def covCategoryGenerators [source]

The covariant generators of one field category.

```
def covCategoryGenerators : FieldCategory → Set T.LocalFieldAlgebra
  | .fermion => ⋃ i, ⋃ n : ℕ, ⋃ l : Fin n → (Fin 1 ⊕ Fin 3),
      Set.range (T.covDerivFermion i l) ∪ Set.range (T.covDerivConjFermion i l)
  | .gauge => ⋃ l : List (Fin 1 ⊕ Fin 3), ⋃ μ, ⋃ ν, Set.range (T.covDerivFieldStrength l μ ν)
  | .scalar => ⋃ j, ⋃ n : ℕ, ⋃ l : Fin n → (Fin 1 ⊕ Fin 3),
      Set.range (T.covDerivBoson j l) ∪ Set.range (T.covDerivConjBoson j l)
```

def covSectorGenerators [source]

The covariant generators selected by a sector.

```
def covSectorGenerators (S : Finset FieldCategory) : Set T.LocalFieldAlgebra
```

lemma covSectorGenerators_mono [source]

```
lemma covSectorGenerators_mono {S S' : Finset FieldCategory} (h : S ⊆ S') :
    T.covSectorGenerators S ⊆ T.covSectorGenerators S'
```

lemma covSectorGenerators_empty [source]

```
@[simp]
lemma covSectorGenerators_empty : T.covSectorGenerators ∅ = ∅
```

lemma covSectorGenerators_union [source]

```
lemma covSectorGenerators_union (S S' : Finset FieldCategory) :
    T.covSectorGenerators (S ∪ S') = T.covSectorGenerators S ∪ T.covSectorGenerators S'
```

lemma covSectorGenerators_univ [source]

The full sector selects the covariant generators of the covariant field algebra.

```
lemma covSectorGenerators_univ : T.covSectorGenerators Finset.univ = T.covGenerators
```

def CovSectorAlgebra [source]

The covariant sector algebra of a sector `S`: the subalgebra of the local field algebra generated by the covariant towers of the selected categories.

```
noncomputable def CovSectorAlgebra (S : Finset FieldCategory) :
    Subalgebra ℂ T.LocalFieldAlgebra
```

lemma covDerivFieldStrength_mem [source]

```
lemma covDerivFieldStrength_mem (hS : FieldCategory.gauge ∈ S) (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    T.covDerivFieldStrength l μ ν φ ∈ T.CovSectorAlgebra S
```

lemma covDerivFermion_mem [source]

```
lemma covDerivFermion_mem (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.FermionValue i)) :
    T.covDerivFermion i l φ ∈ T.CovSectorAlgebra S
```

lemma covDerivConjFermion_mem [source]

```
lemma covDerivConjFermion_mem (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.covDerivConjFermion i l φ ∈ T.CovSectorAlgebra S
```

lemma covDerivBoson_mem [source]

```
lemma covDerivBoson_mem (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.BosonValue j)) :
    T.covDerivBoson j l φ ∈ T.CovSectorAlgebra S
```

lemma covDerivConjBoson_mem [source]

```
lemma covDerivConjBoson_mem (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    T.covDerivConjBoson j l φ ∈ T.CovSectorAlgebra S
```

lemma fieldStrength_mem [source]

The included field strength, as the tower of empty length.

```
lemma fieldStrength_mem (hS : FieldCategory.gauge ∈ S) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ] LocalGaugeFieldAlgebra.fieldStrength 𝔤 μ ν φ)
      ∈ T.CovSectorAlgebra S
```

lemma fermionSymbol_zero_mem [source]

The undifferentiated matter symbols, as the towers of length zero.

```
lemma fermionSymbol_zero_mem (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies)
    (φ : Module.Dual ℂ (T.FermionValue i)) : T.fermionSymbol i 0 φ ∈ T.CovSectorAlgebra S
```

lemma conjFermionSymbol_zero_mem [source]

```
lemma conjFermionSymbol_zero_mem (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies)
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.conjFermionSymbol i 0 φ ∈ T.CovSectorAlgebra S
```

lemma bosonSymbol_zero_mem [source]

```
lemma bosonSymbol_zero_mem (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies)
    (φ : Module.Dual ℂ (T.BosonValue j)) : T.bosonSymbol j 0 φ ∈ T.CovSectorAlgebra S
```

lemma conjBosonSymbol_zero_mem [source]

```
lemma conjBosonSymbol_zero_mem (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies)
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    T.conjBosonSymbol j 0 φ ∈ T.CovSectorAlgebra S
```

lemma covSectorGenerators_cases [source]

Case analysis on the covariant generators of a sector.

```
lemma covSectorGenerators_cases {P : T.LocalFieldAlgebra → Prop} {b : T.LocalFieldAlgebra}
    (hb : b ∈ T.covSectorGenerators S)
    (hF : FieldCategory.gauge ∈ S → ∀ (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
      (φ : Module.Dual ℝ 𝔤), P (T.covDerivFieldStrength l μ ν φ))
    (hψ : FieldCategory.fermion ∈ S → ∀ (i : T.FermionSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.FermionValue i)),
      P (T.covDerivFermion i l φ))
    (hψc : FieldCategory.fermion ∈ S → ∀ (i : T.FermionSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))),
      P (T.covDerivConjFermion i l φ))
    (hφ : FieldCategory.scalar ∈ S → ∀ (j : T.BosonSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.BosonValue j)),
      P (T.covDerivBoson j l φ))
    (hφc : FieldCategory.scalar ∈ S → ∀ (j : T.BosonSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))),
      P (T.covDerivConjBoson j l φ)) : P b
```

lemma induction [source]

A property holding on the selected towers and the scalars and closed under sums and products holds on the covariant sector algebra.

```
lemma induction {P : T.LocalFieldAlgebra → Prop} {x : T.LocalFieldAlgebra}
    (hx : x ∈ T.CovSectorAlgebra S)
    (hF : FieldCategory.gauge ∈ S → ∀ (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
      (φ : Module.Dual ℝ 𝔤), P (T.covDerivFieldStrength l μ ν φ))
    (hψ : FieldCategory.fermion ∈ S → ∀ (i : T.FermionSpecies) {n : ℕ}
```

```
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.FermionValue i)),
    P (T.covDerivFermion i l φ))
  (hψc : FieldCategory.fermion ∈ S → ∀ (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))),
    P (T.covDerivConjFermion i l φ))
  (hφ : FieldCategory.scalar ∈ S → ∀ (j : T.BosonSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.BosonValue j)),
    P (T.covDerivBoson j l φ))
  (hφc : FieldCategory.scalar ∈ S → ∀ (j : T.BosonSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))),
    P (T.covDerivConjBoson j l φ))
  (halg : ∀ z : ℂ, P (z • (1 : T.LocalFieldAlgebra)))
  (hadd : ∀ x y, P x → P y → P (x + y))
  (hmul : ∀ x y, P x → P y → P (x * y)) : P x
```

lemma algHom_ext [source]

Two algebra maps out of a covariant sector algebra agreeing on the selected towers are equal. This is uniqueness only: the algebra is not free on its towers, so an assignment on them need not extend to a map.

```
lemma algHom_ext {B : Type} [Semiring B] [Algebra ℂ B]
    {f g : 𝕥(T.CovSectorAlgebra S) →ₐ[ℂ] B}
    (hF : ∀ (hS : FieldCategory.gauge ∈ S) (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
      (φ : Module.Dual ℝ 𝔤),
      f ⟨T.covDerivFieldStrength l μ ν φ, covDerivFieldStrength_mem hS l μ ν φ⟩
        = g ⟨T.covDerivFieldStrength l μ ν φ, covDerivFieldStrength_mem hS l μ ν φ⟩)
    (hψ : ∀ (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.FermionValue i)),
      f ⟨T.covDerivFermion i l φ, covDerivFermion_mem hS i l φ⟩
        = g ⟨T.covDerivFermion i l φ, covDerivFermion_mem hS i l φ⟩)
    (hψc : ∀ (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))),
      f ⟨T.covDerivConjFermion i l φ, covDerivConjFermion_mem hS i l φ⟩
        = g ⟨T.covDerivConjFermion i l φ, covDerivConjFermion_mem hS i l φ⟩)
    (hφ : ∀ (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.BosonValue j)),
      f ⟨T.covDerivBoson j l φ, covDerivBoson_mem hS j l φ⟩
        = g ⟨T.covDerivBoson j l φ, covDerivBoson_mem hS j l φ⟩)
    (hφc : ∀ (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))),
      f ⟨T.covDerivConjBoson j l φ, covDerivConjBoson_mem hS j l φ⟩
        = g ⟨T.covDerivConjBoson j l φ, covDerivConjBoson_mem hS j l φ⟩) : f = g
```

lemma mono [source]

```
lemma mono {S S' : Finset FieldCategory} (h : S ⊆ S') :
    T.CovSectorAlgebra S ≤ T.CovSectorAlgebra S'
```

lemma empty [source]

The empty covariant sector algebra is the image of the scalars.

```
@[simp]
lemma empty : T.CovSectorAlgebra ∅ = ⊥
```

lemma univ [source]

```
@[simp]
lemma univ : T.CovSectorAlgebra Finset.univ = T.LocalCovFieldAlgebra
```

lemma union [source]

```
lemma union (S S' : Finset FieldCategory) :
    T.CovSectorAlgebra (S ∪ S') = T.CovSectorAlgebra S ⨯ T.CovSectorAlgebra S'
```

lemma le_localCovFieldAlgebra [source]

```
lemma le_localCovFieldAlgebra : T.CovSectorAlgebra S ≤ T.LocalCovFieldAlgebra
```

lemma repJet_mem [source]

```
lemma repJet_mem (U : GJ) {x : T.LocalFieldAlgebra} (hx : x ∈ T.CovSectorAlgebra S) :
    T.repJet U x ∈ T.CovSectorAlgebra S
```

lemma repLorentzGroup_mem [source]

```
lemma repLorentzGroup_mem (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ))
    {x : T.LocalFieldAlgebra} (hx : x ∈ T.CovSectorAlgebra S) :
    T.repLorentzGroup Λ x ∈ T.CovSectorAlgebra S
```

def repJet [source]

The jet gauge group acting on a covariant sector algebra, by restriction.

```
noncomputable def repJet : Representation ℂ GJ (T.CovSectorAlgebra S)
```

def repLorentzGroup [source]

The Lorentz group acting on a covariant sector algebra, by restriction.

```
noncomputable def repLorentzGroup (hGL : T.GaugeLorentzCompatible) :
    Representation ℂ SL(2,ℂ) (T.CovSectorAlgebra S)
```

lemma coe_repJet [source]

```
@[simp]
lemma coe_repJet (U : GJ) (x : T.CovSectorAlgebra S) :
    (repJet T S U x : T.LocalFieldAlgebra) = T.repJet U x
```

lemma repJet_apply_mul [source]

```
lemma repJet_apply_mul (U : GJ) (x y : T.CovSectorAlgebra S) :
    repJet T S U (x * y) = repJet T S U x * repJet T S U y
```

lemma coe_repLorentzGroup [source]

```
@[simp]
lemma coe_repLorentzGroup (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ))
    (x : T.CovSectorAlgebra S) :
    (repLorentzGroup T S hGL Λ x : T.LocalFieldAlgebra) = T.repLorentzGroup Λ x
```

lemma repLorentzGroup_apply_mul [source]

```
lemma repLorentzGroup_apply_mul (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ))
    (x y : T.CovSectorAlgebra S) :
    repLorentzGroup T S hGL Λ (x * y)
      = repLorentzGroup T S hGL Λ x * repLorentzGroup T S hGL Λ y
```

def repValue [source]

The ordinary gauge group acting on a covariant sector algebra, through the constant jets.

```
noncomputable def repValue : Representation ℂ G₀ (T.CovSectorAlgebra S)
```

lemma coe_repValue [source]

```
@[simp]
lemma coe_repValue (g : G₀) (x : T.CovSectorAlgebra S) :
    (repValue T S g x : T.LocalFieldAlgebra) = T.repJet (jets.ofConstant g) x
```

lemma repValue_apply_mul [source]

```
lemma repValue_apply_mul (g : G₀) (x y : T.CovSectorAlgebra S) :
    repValue T S g (x * y) = repValue T S g x * repValue T S g y
```

lemma repJet_eq_repValue_eval [source]

Under `GaugeFieldData.PureJetsActTrivially`, the jet action factors through evaluation: `LocalCovFieldAlgebra.repJet_eq_ofConstant_eval_of_mem` read on the sector.

```
lemma repJet_eq_repValue_eval (hP : T.PureJetsActTrivially) (U : GJ)
    (x : T.CovSectorAlgebra S) : repJet T S U x = repValue T S (jets.eval U) x
```

lemma inclusion_repJet [source]

```
lemma inclusion_repJet {S' : Finset FieldCategory} (h : S ⊆ S') (U : GJ)
    (x : T.CovSectorAlgebra S) :
    Subalgebra.inclusion (mono T h) (repJet T S U x)
      = repJet T S' U (Subalgebra.inclusion (mono T h) x)
```

lemma inclusion_repLorentzGroup [source]

```
lemma inclusion_repLorentzGroup {S' : Finset FieldCategory} (h : S ⊆ S')
    (hGL : T.GaugeLorentzCompatible) (Λ : SL(2,ℂ)) (x : T.CovSectorAlgebra S) :
    Subalgebra.inclusion (mono T h) (repLorentzGroup T S hGL Λ x)
      = repLorentzGroup T S' hGL Λ (Subalgebra.inclusion (mono T h) x)
```

lemma inclusion_repValue [source]

```
lemma inclusion_repValue {S' : Finset FieldCategory} (h : S ⊆ S') (g : G₀)
    (x : T.CovSectorAlgebra S) :
    Subalgebra.inclusion (mono T h) (repValue T S g x)
      = repValue T S' g (Subalgebra.inclusion (mono T h) x)
```

lemma inclusion_localCovFieldAlgebra_repJet [source]

```
lemma inclusion_localCovFieldAlgebra_repJet (U : GJ) (x : T.CovSectorAlgebra S) :
    Subalgebra.inclusion (le_localCovFieldAlgebra T S) (repJet T S U x)
      = LocalCovFieldAlgebra.repJet T U (Subalgebra.inclusion (le_localCovFieldAlgebra T S) x)
```

lemma inclusion_localCovFieldAlgebra_repLorentzGroup [source]

```
lemma inclusion_localCovFieldAlgebra_repLorentzGroup (hGL : T.GaugeLorentzCompatible)
    (Λ : SL(2,ℂ)) (x : T.CovSectorAlgebra S) :
    Subalgebra.inclusion (le_localCovFieldAlgebra T S) (repLorentzGroup T S hGL Λ x)
      = LocalCovFieldAlgebra.repLorentzGroup T hGL Λ
          (Subalgebra.inclusion (le_localCovFieldAlgebra T S) x)
```

lemma inclusion_localCovFieldAlgebra_repValue [source]

```
lemma inclusion_localCovFieldAlgebra_repValue (g : G₀) (x : T.CovSectorAlgebra S) :
    Subalgebra.inclusion (le_localCovFieldAlgebra T S) (repValue T S g x)
      = LocalCovFieldAlgebra.repValue T g (Subalgebra.inclusion (le_localCovFieldAlgebra T S) x
    )
```

2.203 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalCovField.SectorRealization

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalCovField/SectorRealization.lean on GitHub.

abbrev Realization [source]

A complex algebra `B` carrying the covariant sector `S` of the datum `T`: a complex algebra map out of the covariant sector algebra, equivariant for the ordinary gauge group and the Lorentz group, both acting on the whole of `B` by algebra endomorphisms. The source Lorentz action needs `hGL`; no other species condition is used. The lemmas `map_repValue`, `map_repLorentz`, `repGauge_mul` and `repLorentz_mul` name its fields.

```
abbrev Realization (T : GaugeFieldData jets) (S : Finset FieldCategory)
    (hGL : T.GaugeLorentzCompatible) (B : Type) [Semiring B] [Algebra ℂ B]
    (repGauge : Representation ℂ G₀ B) (repLorentz : Representation ℂ SL(2,ℂ) B)
```

def id [source]

A covariant sector algebra realized in itself, by the identity.

```
noncomputable def id :
    Realization T S hGL (T.CovSectorAlgebra S) (repValue T S) (repLorentzGroup T S hGL)
```

lemma id_toAlgHom [source]

```
@[simp]
lemma id_toAlgHom : (id T S hGL).toAlgHom = AlgHom.id ℂ (T.CovSectorAlgebra S)
```

lemma map_repValue [source]

```
lemma map_repValue (g : G₀) (x : T.CovSectorAlgebra S) :
    k.toAlgHom (repValue T S g x) = repGauge g (k.toAlgHom x)
```

lemma map_repLorentz [source]

```
lemma map_repLorentz (Λ : SL(2,ℂ)) (x : T.CovSectorAlgebra S) :
    k.toAlgHom (repLorentzGroup T S hGL Λ x) = repLorentz Λ (k.toAlgHom x)
```

lemma repGauge_mul [source]

```
lemma repGauge_mul (g : G₀) (b₁ b₂ : B) :
    repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂
```

lemma repLorentz_mul [source]

```
lemma repLorentz_mul (Λ : SL(2,ℂ)) (b₁ b₂ : B) :
    repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂
```

lemma ext_towers [source]

A sector realization is determined by its images of the selected towers; uniqueness only, the algebra not being free on its towers.

```
lemma ext_towers {k₁ k₂ : Realization T S hGL B repGauge repLorentz}
    (hF : ∀ (hS : FieldCategory.gauge ∈ S) (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
      (φ : Module.Dual ℝ 𝔤),
      k₁.toAlgHom ⟨T.covDerivFieldStrength l μ ν φ, covDerivFieldStrength_mem hS l μ ν φ⟩
        = k₂.toAlgHom ⟨T.covDerivFieldStrength l μ ν φ, covDerivFieldStrength_mem hS l μ ν φ⟩)
    (hψ : ∀ (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.FermionValue i)),
      k₁.toAlgHom ⟨T.covDerivFermion i l φ, covDerivFermion_mem hS i l φ⟩
        = k₂.toAlgHom ⟨T.covDerivFermion i l φ, covDerivFermion_mem hS i l φ⟩)
    (hψc : ∀ (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))),
      k₁.toAlgHom ⟨T.covDerivConjFermion i l φ, covDerivConjFermion_mem hS i l φ⟩
        = k₂.toAlgHom ⟨T.covDerivConjFermion i l φ, covDerivConjFermion_mem hS i l φ⟩)
    (hφ : ∀ (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.BosonValue j)),
      k₁.toAlgHom ⟨T.covDerivBoson j l φ, covDerivBoson_mem hS j l φ⟩
        = k₂.toAlgHom ⟨T.covDerivBoson j l φ, covDerivBoson_mem hS j l φ⟩)
    (hφc : ∀ (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies) {n : ℕ}
      (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))),
      k₁.toAlgHom ⟨T.covDerivConjBoson j l φ, covDerivConjBoson_mem hS j l φ⟩
        = k₂.toAlgHom ⟨T.covDerivConjBoson j l φ, covDerivConjBoson_mem hS j l φ⟩) :
    k₁ = k₂
```

lemma map_repJet [source]

Under `GaugeFieldData.PureJetsActTrivially` the jet action factors through evaluation: a jet acts on the realized image as its value.

```
lemma map_repJet (hP : T.PureJetsActTrivially) (U : GJ) (x : T.CovSectorAlgebra S) :
    k.toAlgHom (repJet T S U x) = repGauge (jets.eval U) (k.toAlgHom x)
```

def restrict [source]

The restriction of a covariant sector realization to a smaller sector.

```
noncomputable def restrict (k : Realization T S' hGL B repGauge repLorentz) (hS : S ⊆ S') :
    Realization T S hGL B repGauge repLorentz
```

lemma restrict_toAlgHom [source]

```
lemma restrict_toAlgHom (k : Realization T S' hGL B repGauge repLorentz) (hS : S ⊆ S') :
    (k.restrict hS).toAlgHom = k.toAlgHom.comp (Subalgebra.inclusion (mono T hS))
```

lemma restrict_toAlgHom_apply [source]

```
@[simp]
lemma restrict_toAlgHom_apply (k : Realization T S' hGL B repGauge repLorentz) (hS : S ⊆ S')
    (x : T.CovSectorAlgebra S) :
    (k.restrict hS).toAlgHom x = k.toAlgHom (Subalgebra.inclusion (mono T hS) x)
```

lemma restrict_toAlgHom_mk [source]

```
lemma restrict_toAlgHom_mk (k : Realization T S' hGL B repGauge repLorentz) (hS : S ⊆ S')
    (x : T.LocalFieldAlgebra) (hx : x ∈ T.CovSectorAlgebra S) :
    (k.restrict hS).toAlgHom ⟨x, hx⟩ = k.toAlgHom ⟨x, mono T hS hx⟩
```

lemma restrict_id_toAlgHom [source]

```
lemma restrict_id_toAlgHom (hS : S ⊆ S') :
    ((id T S' hGL).restrict hS).toAlgHom = Subalgebra.inclusion (mono T hS)
```

lemma restrict_restrict [source]

```
lemma restrict_restrict {S'' : Finset FieldCategory}
    (k : Realization T S'' hGL B repGauge repLorentz) (hS' : S' ⊆ S'') (hS : S ⊆ S') :
    (k.restrict hS').restrict hS = k.restrict (hS.trans hS')
```

def restrictSector [source]

The restriction of a realization of the covariant field algebra to a covariant sector.

```
noncomputable def restrictSector : CovSectorAlgebra.Realization T S hGL B repGauge repLorentz
```

lemma restrictSector_toAlgHom [source]

```
lemma restrictSector_toAlgHom :
    (k.restrictSector S).toAlgHom
      = k.toAlgHom.comp (Subalgebra.inclusion (CovSectorAlgebra.le_localCovFieldAlgebra T S))
```

lemma restrictSector_toAlgHom_apply [source]

```
@[simp]
lemma restrictSector_toAlgHom_apply (x : T.CovSectorAlgebra S) :
    (k.restrictSector S).toAlgHom x
      = k.toAlgHom (Subalgebra.inclusion (CovSectorAlgebra.le_localCovFieldAlgebra T S) x)
```

lemma restrictSector_id_toAlgHom [source]

```
lemma restrictSector_id_toAlgHom :
    ((id T hGL).restrictSector S).toAlgHom
      = Subalgebra.inclusion (CovSectorAlgebra.le_localCovFieldAlgebra T S)
```

lemma restrictSector_fieldStrength [source]

```
lemma restrictSector_fieldStrength (hS : FieldCategory.gauge ∈ S) (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    (k.restrictSector S).toAlgHom
      ⟨T.covDerivFieldStrength l μ ν φ, CovSectorAlgebra.covDerivFieldStrength_mem hS l μ ν φ⟩
      = k.fieldStrength l μ ν φ
```

lemma restrictSector_fermion [source]

```
lemma restrictSector_fermion (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.FermionValue i)) :
    (k.restrictSector S).toAlgHom
      ⟨T.covDerivFermion i l φ, CovSectorAlgebra.covDerivFermion_mem hS i l φ⟩
      = k.fermion i l φ
```

lemma restrictSector_conjFermion [source]

```
lemma restrictSector_conjFermion (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies)
    {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    (k.restrictSector S).toAlgHom
      ⟨T.covDerivConjFermion i l φ, CovSectorAlgebra.covDerivConjFermion_mem hS i l φ⟩
      = k.conjFermion i l φ
```

lemma restrictSector_boson [source]

```
lemma restrictSector_boson (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.BosonValue j)) :
    (k.restrictSector S).toAlgHom
      ⟨T.covDerivBoson j l φ, CovSectorAlgebra.covDerivBoson_mem hS j l φ⟩
      = k.boson j l φ
```

lemma restrictSector_conjBoson [source]

```
lemma restrictSector_conjBoson (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    (k.restrictSector S).toAlgHom
      ⟨T.covDerivConjBoson j l φ, CovSectorAlgebra.covDerivConjBoson_mem hS j l φ⟩
      = k.conjBoson j l φ
```

lemma restrict_restrictSector [source]

```
lemma restrict_restrictSector {S' : Finset FieldCategory} (hS : S ⊆ S') :
    (k.restrictSector S').restrict hS = k.restrictSector S
```

def restrictCovSector [source]

The restriction of a realization of the local field algebra to a covariant sector; the ordinary gauge group acts on the target through the constant jets.

```
noncomputable def restrictCovSector :
    CovSectorAlgebra.Realization T S hGL B (repJet.comp jets.ofConstant) repLorentz
```

lemma restrictCovSector_toAlgHom [source]

```
lemma restrictCovSector_toAlgHom :
    (h.restrictCovSector hGL S).toAlgHom = h.toAlgHom.comp (T.CovSectorAlgebra S).val
```

lemma restrictCovSector_toAlgHom_apply [source]

```
@[simp]
lemma restrictCovSector_toAlgHom_apply (x : T.CovSectorAlgebra S) :
    (h.restrictCovSector hGL S).toAlgHom x = h.toAlgHom x
```

lemma restrictCovSector_map_repJet [source]

Jet equivariance survives restriction without `PureJetsActTrivially`.

```
lemma restrictCovSector_map_repJet (U : GJ) (x : T.CovSectorAlgebra S) :
    (h.restrictCovSector hGL S).toAlgHom (CovSectorAlgebra.repJet T S U x)
      = repJet U ((h.restrictCovSector hGL S).toAlgHom x)
```

lemma restrictSector_restrict [source]

Restriction through the covariant field algebra is the direct restriction.

```
lemma restrictSector_restrict :
    (h.restrict hGL).restrictSector S = h.restrictCovSector hGL S
```

lemma restrict_restrictCovSector [source]

```
lemma restrict_restrictCovSector {S' : Finset FieldCategory} (hS : S ⊆ S') :
    (h.restrictCovSector hGL S').restrict hS = h.restrictCovSector hGL S
```

2.204 `PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.Basic`

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/Basic.lean on GitHub.

abbrev LocalFieldAlgebra [source]

The local field algebra `J(T)` of a gauge-field datum, in which the local expressions of the theory, such as Lagrangian terms, currents and field strengths, live before any of them is selected. It is the exterior algebra of the fermionic generators tensored with the symmetric algebra of the bosonic generators, tensored with the complexified symmetric algebra of the connection generators.

All the fermionic species share one exterior algebra, so their generators anticommute across species and not only within one. The factor order matches `StandardModel.JetAlgebra`.

```
abbrev LocalFieldAlgebra : Type
```

def includeFermion [source]

The inclusion of the fermionic factor.

```
noncomputable def includeFermion :
    ExteriorAlgebra ℂ T.FermionGenerators →ₐ[ℂ] T.LocalFieldAlgebra
```

def includeBoson [source]

The inclusion of the bosonic factor.

```
noncomputable def includeBoson :
    SymmetricAlgebra ℂ T.BosonGenerators →ₐ[ℂ] T.LocalFieldAlgebra
```

def includeConnection [source]

The inclusion of the complexified connection factor.

```
noncomputable def includeConnection :
    (ℂ ⊗[ℝ] SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤)) →ₐ[ℂ]
      T.LocalFieldAlgebra
```

abbrev MatterAlgebra [source]

The matter factor of the local field algebra, on which the two matter inclusions land.

```
abbrev MatterAlgebra : Type
```

lemma one_matterAlgebra [source]

The unit of the matter factor is the tensor of the two units. It is recorded here, at an abstract datum, because unfolding it at a concrete one has to see through the symmetric algebra's ring congruence, which is not exposed, and is prohibitively slow.

```
lemma one_matterAlgebra :
    (1 : T.MatterAlgebra)
      = (1 : ExteriorAlgebra ℂ T.FermionGenerators) ⊗ₜ[ℂ]
        (1 : SymmetricAlgebra ℂ T.BosonGenerators)
```

lemma includeFermion_apply [source]

The fermionic inclusion, written out as a pure tensor with units in the other two factors. Recorded at an abstract datum for the same reason.

```
lemma includeFermion_apply (a : ExteriorAlgebra ℂ T.FermionGenerators) :
    T.includeFermion a
      = ((a ⊗ₜ[ℂ] (1 : SymmetricAlgebra ℂ T.BosonGenerators))
          ⊗ₜ[ℂ] (1 : ℂ ⊗[ℝ] SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤)) :
        T.LocalFieldAlgebra)
```

lemma includeBoson_apply [source]

The bosonic inclusion, written out as a pure tensor.

```
lemma includeBoson_apply (b : SymmetricAlgebra ℂ T.BosonGenerators) :
    T.includeBoson b
      = (((1 : ExteriorAlgebra ℂ T.FermionGenerators) ⊗ₜ[ℂ] b)
          ⊗ₜ[ℂ] (1 : ℂ ⊗[ℝ] SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤)) :
        T.LocalFieldAlgebra)
```

lemma includeConnection_apply [source]

The connection inclusion, written out as a pure tensor.

```
lemma includeConnection_apply
    (c : ℂ ⊗[ℝ] SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤)) :
    T.includeConnection c
      = ((1 : T.MatterAlgebra) ⊗ₜ[ℂ] c : T.LocalFieldAlgebra)
```

def ιFermionTotal [source]

All the fermionic generators of the datum at once, the whole fermionic generator space inside the algebra. Fermi statistics is a condition on this map, not on the species maps separately.

```
noncomputable def ιFermionTotal : T.FermionGenerators →ₗ[ℂ] T.LocalFieldAlgebra
```

def ιBosonTotal [source]

All the bosonic generators of the datum at once.

```
noncomputable def ιBosonTotal : T.BosonGenerators →ₗ[ℂ] T.LocalFieldAlgebra
```

def ιConnection [source]

The connection generators inside the local field algebra. They are only real-linear, the connection generator space being a real vector space, complexified inside the algebra.

```
noncomputable def ιConnection :
    GaugeBoson.JetComponentSpace 𝔤 →ₗ[ℝ] T.LocalFieldAlgebra
```

def ιFermion [source]

The generators of one fermionic species inside the local field algebra.

```
noncomputable def ιFermion (i : T.FermionSpecies) :
    JetComponentSpace (T.fermion i) →ₗ[ℂ] T.LocalFieldAlgebra
```

def ιBoson [source]

The generators of one bosonic species inside the local field algebra.

```
noncomputable def ιBoson (j : T.BosonSpecies) :
    JetComponentSpace (T.boson j) →ₗ[ℂ] T.LocalFieldAlgebra
```

lemma ιFermionTotal_apply [source]

```
lemma ιFermionTotal_apply (v : T.FermionGenerators) :
    T.ιFermionTotal v
      = (ExteriorAlgebra.ι ℂ v ⊗ₜ[ℂ] (1 : SymmetricAlgebra ℂ T.BosonGenerators))
          ⊗ₜ[ℂ] (1 : ℂ ⊗[ℝ] SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤))
```

lemma ιBosonTotal_apply [source]

```
lemma ιBosonTotal_apply (v : T.BosonGenerators) :
    T.ιBosonTotal v
      = ((1 : ExteriorAlgebra ℂ T.FermionGenerators) ⊗ₜ[ℂ]
            SymmetricAlgebra.ι ℂ T.BosonGenerators v)
          ⊗ₜ[ℂ] (1 : ℂ ⊗[ℝ] SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤))
```

lemma ιConnection_apply [source]

```
lemma ιConnection_apply (v : GaugeBoson.JetComponentSpace 𝔤) :
    T.ιConnection v
      = (1 : ExteriorAlgebra ℂ T.FermionGenerators ⊗[ℂ] SymmetricAlgebra ℂ T.BosonGenerators)
          ⊗ₜ[ℂ] ((1 : ℂ) ⊗ₜ[ℝ]
            SymmetricAlgebra.ι ℝ (GaugeBoson.JetComponentSpace 𝔤) v)
```

lemma ιFermion_apply [source]

```
lemma ιFermion_apply (i : T.FermionSpecies) (x : JetComponentSpace (T.fermion i)) :
    T.ιFermion i x = T.ιFermionTotal (T.inclFermion i x)
```

lemma ιBoson_apply [source]

```
lemma ιBoson_apply (j : T.BosonSpecies) (y : JetComponentSpace (T.boson j)) :
    T.ιBoson j y = T.ιBosonTotal (T.inclBoson j y)
```

lemma ιFermionTotal_mul_self [source]

A fermionic generator squares to zero. This is Fermi statistics, in the form that implies anticommutation across the whole fermionic generator space by polarization.

```
@[simp]
lemma ιFermionTotal_mul_self (v : T.FermionGenerators) :
    T.ιFermionTotal v * T.ιFermionTotal v = 0
```

lemma ιFermionTotal_mul_swap [source]

Two fermionic generators anticommute, including generators of two different species, which share the one exterior algebra. This is not an extra relation, being `ιFermionTotal_mul_self` polarized.

```
lemma ιFermionTotal_mul_swap (v w : T.FermionGenerators) :
    T.ιFermionTotal v * T.ιFermionTotal w = -(T.ιFermionTotal w * T.ιFermionTotal v)
```

lemma ιBosonTotal_commute [source]

Two bosonic generators commute.

```
lemma ιBosonTotal_commute (v w : T.BosonGenerators) :
    Commute (T.ιBosonTotal v) (T.ιBosonTotal w)
```

lemma ιConnection_commute [source]

Two connection generators commute.

```
lemma ιConnection_commute (v w : GaugeBoson.JetComponentSpace 𝔤) :
    Commute (T.ιConnection v) (T.ιConnection w)
```

lemma ιBosonTotal_commute_ιConnection [source]

A bosonic generator commutes with a connection generator.

```
lemma ιBosonTotal_commute_ιConnection (v : T.BosonGenerators)
    (w : GaugeBoson.JetComponentSpace 𝔤) :
    Commute (T.ιBosonTotal v) (T.ιConnection w)
```

lemma ιBosonTotal_commute_ιFermionTotal [source]

A bosonic generator commutes with a fermionic generator, bosons being even.

```
lemma ιBosonTotal_commute_ιFermionTotal (v : T.BosonGenerators) (w : T.FermionGenerators) :
    Commute (T.ιBosonTotal v) (T.ιFermionTotal w)
```

lemma ιConnection_commute_ιFermionTotal [source]

A connection generator commutes with a fermionic generator, the connection being even.

```
lemma ιConnection_commute_ιFermionTotal (v : GaugeBoson.JetComponentSpace 𝔤)
    (w : T.FermionGenerators) : Commute (T.ιConnection v) (T.ιFermionTotal w)
```

lemma includeConnection_commute [source]

The connection factor is central: its elements commute with the whole algebra, the factor being commutative and tensored in as the right factor.

```
lemma includeConnection_commute
    (c : ℂ ⊗[ℝ] SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤)) (x : T.LocalFieldAlgebra)
    :
    Commute (T.includeConnection c) x
```

lemma ιFermion_mul_self [source]

A fermionic generator of a species squares to zero.

```
@[simp]
lemma ιFermion_mul_self (i : T.FermionSpecies) (x : JetComponentSpace (T.fermion i)) :
    T.ιFermion i x * T.ιFermion i x = 0
```

lemma ιFermion_mul_swap [source]

The generators of two fermionic species of the datum anticommute. The species enter one exterior algebra through different summands of the fermionic generator space, so this is ordinary exterior anticommutation and not an extra relation; it is what separate exterior algebras joined by an ordinary tensor product would lose.

```
lemma ιFermion_mul_swap (i j : T.FermionSpecies) (x : JetComponentSpace (T.fermion i))
    (y : JetComponentSpace (T.fermion j)) :
    T.ιFermion i x * T.ιFermion j y = -(T.ιFermion j y * T.ιFermion i x)
```

lemma ιBoson_commute [source]

Bosonic generators commute, across species as well as within one.

```
lemma ιBoson_commute (i j : T.BosonSpecies) (x : JetComponentSpace (T.boson i))
    (y : JetComponentSpace (T.boson j)) :
    Commute (T.ιBoson i x) (T.ιBoson j y)
```

lemma ιBoson_commute_ιFermion [source]

A bosonic generator commutes with a fermionic one, bosons being even.

```
lemma ιBoson_commute_ιFermion (j : T.BosonSpecies) (i : T.FermionSpecies)
    (y : JetComponentSpace (T.boson j)) (x : JetComponentSpace (T.fermion i)) :
    Commute (T.ιBoson j y) (T.ιFermion i x)
```

lemma ιBoson_commute_ιConnection [source]

A bosonic generator commutes with a connection generator.

```
lemma ιBoson_commute_ιConnection (j : T.BosonSpecies)
    (y : JetComponentSpace (T.boson j)) (v : GaugeBoson.JetComponentSpace 𝔤) :
    Commute (T.ιBoson j y) (T.ιConnection v)
```

lemma ιConnection_commute_ιFermion [source]

A connection generator commutes with a fermionic one, the connection being even.

```
lemma ιConnection_commute_ιFermion (v : GaugeBoson.JetComponentSpace 𝔤)
    (i : T.FermionSpecies) (x : JetComponentSpace (T.fermion i)) :
    Commute (T.ιConnection v) (T.ιFermion i x)
```

def generators [source]

The generators of the local field algebra, the images of the three generator spaces.

```
noncomputable def generators : Set T.LocalFieldAlgebra
```

lemma adjoin_matter_generators_eq_top [source]

The matter factor `Λ_ℂ E_f(T) ⊗[ℂ] Sym_ℂ E_b(T)` is generated by the images of the fermionic and bosonic generators.

```
lemma adjoin_matter_generators_eq_top :
    Algebra.adjoin ℂ
        (Set.range (fun v : T.FermionGenerators =>
            ExteriorAlgebra.ι ℂ v ⊗ₜ[ℂ] (1 : SymmetricAlgebra ℂ T.BosonGenerators))
          ∪ Set.range (fun v : T.BosonGenerators =>
            (1 : ExteriorAlgebra ℂ T.FermionGenerators) ⊗ₜ[ℂ]
              SymmetricAlgebra.ι ℂ T.BosonGenerators v)) = ⊤
```

lemma adjoin_generators_eq_top [source]

The local field algebra is generated by its three families of generators. Every element is a polynomial in the fermionic, bosonic and connection symbols; this is the algebraic form of "every local expression is a polynomial in the fields and their derivatives", and it is what forces uniqueness in the universal property.

```
lemma adjoin_generators_eq_top : Algebra.adjoin ℂ T.generators = ⊤
```

lemma algHom_ext_generators [source]

Two algebra maps out of the local field algebra agreeing on the whole of the three generator spaces are equal. The target is not assumed commutative, so this cannot be deduced from `SymmetricAlgebra.algHom_ext`; it comes from generation.

```
lemma algHom_ext_generators {B : Type*} [Semiring B] [Algebra ℂ B]
    {Φ Ψ : T.LocalFieldAlgebra →ₐ[ℂ] B}
    (hf : ∀ v, Φ (T.ιFermionTotal v) = Ψ (T.ιFermionTotal v))
    (hb : ∀ v, Φ (T.ιBosonTotal v) = Ψ (T.ιBosonTotal v))
    (ha : ∀ v, Φ (T.ιConnection v) = Ψ (T.ιConnection v)) : Φ = Ψ
```

lemma algHom_ext [source]

Two algebra maps out of `J(T)` are equal as soon as they agree on the generators of every species and on the connection generators. A linear map out of the fermionic generator space is determined by its restrictions to the species, so the species-wise hypotheses already give the hypotheses of `algHom_ext_generators`.

```
lemma algHom_ext {B : Type*} [Semiring B] [Algebra ℂ B] {Φ Ψ : T.LocalFieldAlgebra →ₐ[ℂ] B}
    (hf : ∀ i x, Φ (T.ιFermion i x) = Ψ (T.ιFermion i x))
    (hb : ∀ j y, Φ (T.ιBoson j y) = Ψ (T.ιBoson j y))
    (ha : ∀ v, Φ (T.ιConnection v) = Ψ (T.ιConnection v)) : Φ = Ψ
```

structure Assignment [source]

A compatible assignment of the generators of the local field algebra of a datum in a complex algebra `B`, given by one linear map per fermionic species, one per bosonic species and one real-linear map on the connection generators, subject to exactly

the relations of section A.2. The algebra `B` is not assumed commutative, and the commutation conditions are imposed only between the assigned images, not between an image and an arbitrary element of `B`.

`fermion_mul_swap` is not redundant. The square-zero condition the algebra imposes is on the whole fermionic generator space, and by `DirectSum.mul_self_iff_lof` that condition is equivalent to `fermion_mul_self` together with `fermion_mul_swap`; species-wise square-zero alone leaves the cross terms of `(x + y) * (x + y)` for `x` and `y` in different species.

```
@[ext]
structure Assignment (B : Type*) [Ring B] [Algebra ℂ B] where
  fermion : ∀ i, JetComponentSpace (T.fermion i) →ₗ[ℂ] B
  boson : ∀ j, JetComponentSpace (T.boson j) →ₗ[ℂ] B
  connection : GaugeBoson.JetComponentSpace 𝔤 →ₗ[ℝ] B
  fermion_mul_self : ∀ i x, fermion i x * fermion i x = 0
  fermion_mul_swap : ∀ i j x y, fermion i x * fermion j y = -(fermion j y * fermion i x)
  boson_commute : ∀ i j x y, Commute (boson i x) (boson j y)
  connection_commute : ∀ v w, Commute (connection v) (connection w)
  boson_commute_connection : ∀ j y w, Commute (boson j y) (connection w)
  boson_commute_fermion : ∀ j i y x, Commute (boson j y) (fermion i x)
  connection_commute_fermion : ∀ v i x, Commute (connection v) (fermion i x)
```

lemma assemble_mul_self_iff [source]

Species-wise square-zero is not square-zero. The condition the local field algebra imposes on the assembled fermionic map is equivalent to the two fields `Assignment.fermion_mul_self` and `Assignment.fermion_mul_swap` together; the second is vacuous only for a family with at most one species.

```
lemma assemble_mul_self_iff {B : Type*} [Ring B] [Algebra ℂ B]
    (f : ∀ i, JetComponentSpace (T.fermion i) →ₗ[ℂ] B) :
    (∀ v, T.assembleFermion f v * T.assembleFermion f v = 0)
      ↔ ((∀ i x, f i x * f i x = 0) ∧
        ∀ i j x y, f i x * f j y = -(f j y * f i x))
```

def fermionTotal [source]

The images of all the fermionic generators at once, assembled from the species.

```
def fermionTotal : T.FermionGenerators →ₗ[ℂ] B
```

def bosonTotal [source]

The images of all the bosonic generators at once.

```
def bosonTotal : T.BosonGenerators →ₗ[ℂ] B
```

lemma fermionTotal_inclFermion [source]

```
@[simp]
lemma fermionTotal_inclFermion (i : T.FermionSpecies)
    (x : JetComponentSpace (T.fermion i)) :
    d.fermionTotal (T.inclFermion i x) = d.fermion i x
```

lemma bosonTotal_inclBoson [source]

```
@[simp]
lemma bosonTotal_inclBoson (j : T.BosonSpecies) (y : JetComponentSpace (T.boson j)) :
    d.bosonTotal (T.inclBoson j y) = d.boson j y
```

lemma fermionTotal_mul_self [source]

The assembled fermionic images square to zero, which is more than the species-wise condition and needs the cross-species anticommutation as well.

```
lemma fermionTotal_mul_self (v : T.FermionGenerators) :
    d.fermionTotal v * d.fermionTotal v = 0
```

lemma fermionTotal_mul_swap [source]

The images of two fermionic generators anticommute. Only `fermion_mul_self` and `fermion_mul_swap` are assumed, and only species by species; because their conjunction is square-zero on the whole generator space, polarization yields the relation for arbitrary vectors, sums across species included.

```
lemma fermionTotal_mul_swap (v w : T.FermionGenerators) :
    d.fermionTotal v * d.fermionTotal w = -(d.fermionTotal w * d.fermionTotal v)
```

lemma bosonTotal_commute [source]

The assembled bosonic images commute pairwise.

```
lemma bosonTotal_commute (v w : T.BosonGenerators) :
    Commute (d.bosonTotal v) (d.bosonTotal w)
```

lemma bosonTotal_commute_connection [source]

The assembled bosonic images commute with the connection images.

```
lemma bosonTotal_commute_connection (v : T.BosonGenerators)
    (w : GaugeBoson.JetComponentSpace 𝔤) : Commute (d.bosonTotal v) (d.connection w)
```

lemma bosonTotal_commute_fermionTotal [source]

The assembled bosonic images commute with the assembled fermionic images.

```
lemma bosonTotal_commute_fermionTotal (v : T.BosonGenerators) (w : T.FermionGenerators) :
    Commute (d.bosonTotal v) (d.fermionTotal w)
```

lemma connection_commute_fermionTotal [source]

The connection images commute with the assembled fermionic images.

```
lemma connection_commute_fermionTotal (v : GaugeBoson.JetComponentSpace 𝔤)
    (w : T.FermionGenerators) : Commute (d.connection v) (d.fermionTotal w)
```

def evenGenerators [source]

The even generator images, the bosonic and connection images together.

```
def evenGenerators : Set B
```

def evenSubalgebra [source]

The commutative subalgebra of `B` generated by the even images. The bosonic and connection assignments are lifted into it, because `SymmetricAlgebra.lift` requires a commutative target while `B` itself need not be commutative.

```
def evenSubalgebra : Subalgebra ℂ B
```

instance IsMulCommutative d.evenSubalgebra [source]

```
instance : IsMulCommutative d.evenSubalgebra
```

def bosonGenerator [source]

The bosonic assignment, corestricted to the commutative subalgebra of even images.

```
def bosonGenerator : T.BosonGenerators →ₗ[ℂ] d.evenSubalgebra where
```

def connectionGenerator [source]

The connection assignment, corestricted to the commutative subalgebra of even images.

```
noncomputable def connectionGenerator :
    GaugeBoson.JetComponentSpace 𝔤 →ₗ[ℝ] d.evenSubalgebra where
```

def fermionHom [source]

The algebra map out of the fermionic factor, the exterior lift of the assembled fermionic images, which already accepts an associative, possibly noncommutative, target.

```
noncomputable def fermionHom : ExteriorAlgebra ℂ T.FermionGenerators →ₐ[ℂ] B
```

def bosonHom [source]

The algebra map out of the bosonic factor, the symmetric lift into the commutative subalgebra of even images, followed by its inclusion.

```
noncomputable def bosonHom : SymmetricAlgebra ℂ T.BosonGenerators →ₐ[ℂ] B
```

def connectionHomReal [source]

The real algebra map out of the real connection factor.

```
noncomputable def connectionHomReal :
    SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤) →ₐ[ℝ] B
```

def connectionHom [source]

The algebra map out of the complexified connection factor, obtained from `connectionHomReal` by the universal property of extension of scalars, which, unlike the symmetric lift, accepts a noncommutative target.

```
noncomputable def connectionHom :
    (ℂ ⊗[ℝ] SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤)) →ₐ[ℂ] B
```

lemma fermionHom_ι [source]

```
@[simp]
lemma fermionHom_ι (v : T.FermionGenerators) :
    d.fermionHom (ExteriorAlgebra.ι ℂ v) = d.fermionTotal v
```

lemma bosonHom_ι [source]

```
@[simp]
lemma bosonHom_ι (v : T.BosonGenerators) :
    d.bosonHom (SymmetricAlgebra.ι ℂ T.BosonGenerators v) = d.bosonTotal v
```

lemma connectionHomReal_ι [source]

```
@[simp]
lemma connectionHomReal_ι (v : GaugeBoson.JetComponentSpace 𝔤) :
    d.connectionHomReal (SymmetricAlgebra.ι ℝ (GaugeBoson.JetComponentSpace 𝔤) v)
      = d.connection v
```

lemma connectionHom_tmul [source]

```
@[simp]
lemma connectionHom_tmul (z : ℂ)
    (p : SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤)) :
    d.connectionHom (z ⊗ₜ[ℝ] p) = z • d.connectionHomReal p
```

lemma fermionHom_commute_bosonHom [source]

```
lemma fermionHom_commute_bosonHom (x : ExteriorAlgebra ℂ T.FermionGenerators)
    (y : SymmetricAlgebra ℂ T.BosonGenerators) :
    Commute (d.fermionHom x) (d.bosonHom y)
```

lemma fermionHom_commute_connectionHom [source]

```
lemma fermionHom_commute_connectionHom (x : ExteriorAlgebra ℂ T.FermionGenerators)
    (y : ℂ ⊗[ℝ] SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤)) :
    Commute (d.fermionHom x) (d.connectionHom y)
```

lemma bosonHom_commute_connectionHom [source]

```
lemma bosonHom_commute_connectionHom (x : SymmetricAlgebra ℂ T.BosonGenerators)
    (y : ℂ ⊗[ℝ] SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤)) :
    Commute (d.bosonHom x) (d.connectionHom y)
```

def matterHom [source]

The algebra map out of the matter factor `Λ_ℂ E_f(T) ⊗[ℂ] Sym_ℂ E_b(T)`.

```
noncomputable def matterHom :
    (ExteriorAlgebra ℂ T.FermionGenerators ⊗[ℂ] SymmetricAlgebra ℂ T.BosonGenerators) →ₐ[ℂ]
      B
```

lemma matterHom_commute_connectionHom [source]

```
lemma matterHom_commute_connectionHom
    (x : ExteriorAlgebra ℂ T.FermionGenerators ⊗[ℂ] SymmetricAlgebra ℂ T.BosonGenerators)
    (y : ℂ ⊗[ℝ] SymmetricAlgebra ℝ (GaugeBoson.JetComponentSpace 𝔤)) :
    Commute (d.matterHom x) (d.connectionHom y)
```

def lift [source]

The realization arrow of the datum: the algebra homomorphism `J(T) →ₐ[ℂ] B` induced by a compatible assignment.

```
noncomputable def lift : T.LocalFieldAlgebra →ₐ[ℂ] B
```

lemma lift_ιFermionTotal [source]

```
@[simp]
lemma lift_ιFermionTotal (v : T.FermionGenerators) :
    d.lift (T.ιFermionTotal v) = d.fermionTotal v
```

lemma lift_ιBosonTotal [source]

```
@[simp]
lemma lift_ιBosonTotal (v : T.BosonGenerators) :
    d.lift (T.ιBosonTotal v) = d.bosonTotal v
```

lemma lift_ιConnection [source]

```
@[simp]
lemma lift_ιConnection (v : GaugeBoson.JetComponentSpace 𝔤) :
    d.lift (T.ιConnection v) = d.connection v
```

lemma lift_ιFermion [source]

```
@[simp]
lemma lift_ιFermion (i : T.FermionSpecies) (x : JetComponentSpace (T.fermion i)) :
    d.lift (T.ιFermion i x) = d.fermion i x
```

lemma lift_ιBoson [source]

```
@[simp]
lemma lift_ιBoson (j : T.BosonSpecies) (y : JetComponentSpace (T.boson j)) :
    d.lift (T.ιBoson j y) = d.boson j y
```

lemma existsUnique_algHom [source]

The mapping-out universal property of the local field algebra of a datum. A compatible assignment of the generators of every species, and of the connection, in an arbitrary, in particular noncommutative, complex algebra `B` extends to one and only one complex algebra homomorphism `J(T) →ₐ[ℂ] B`. Injectivity is neither claimed nor wanted, since a realization may identify local expressions.

```
lemma existsUnique_algHom :
    ∃! Φ : T.LocalFieldAlgebra →ₐ[ℂ] B,
      (∀ i x, Φ (T.ιFermion i x) = d.fermion i x) ∧
        (∀ j y, Φ (T.ιBoson j y) = d.boson j y) ∧
          (∀ v, Φ (T.ιConnection v) = d.connection v)
```

def comp [source]

The compatible assignment obtained by pushing one forward along an algebra map, every relation being preserved by an algebra homomorphism.

```
noncomputable def comp {C : Type*} [Ring C] [Algebra ℂ C] (φ : B →ₐ[ℂ] C) :
    T.Assignment C where
```

def canonicalAssignment [source]

The tautological assignment, with the generators of the local field algebra assigned to themselves. Its relations are those of section A.2.

```
noncomputable def canonicalAssignment : T.Assignment T.LocalFieldAlgebra where
```

def liftEquiv [source]

Compatible assignments in `B` are the same thing as algebra maps into `B`. This is the universal property as an equivalence of types rather than a linear equivalence, neither side being a module in a way the other respects.

```
noncomputable def liftEquiv (B : Type*) [Ring B] [Algebra ℂ B] :
    T.Assignment B ≃ (T.LocalFieldAlgebra →ₐ[ℂ] B) where
```

lemma range_lift [source]

The range of the induced map is the complex subalgebra generated by the images of the species and of the connection. No injectivity is claimed or required, since a realization of a field theory may identify local expressions.

```
lemma range_lift :
    d.lift.range
      = Algebra.adjoin ℂ
        ((⋃ i, Set.range (d.fermion i)) ∪ (⋃ j, Set.range (d.boson j))
          ∪ Set.range d.connection)
```

2.205 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.CovariantDeriv

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/CovariantDeriv.lean on GitHub.

def covDerivFermion [source]

The iterated covariant derivative `∇_{l 0} ⋯ ∇_{l (n-1)} ψ` of a fermionic species along an ordered tuple of directions, as a family over the covectors of its value space.

```
noncomputable def covDerivFermion (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    :
    Module.Dual ℂ (T.FermionValue i) →ₗ[ℂ] T.LocalFieldAlgebra
```

def covDerivConjFermion [source]

The iterated covariant derivative of the conjugate components of a fermionic species, through the conjugate infinitesimal action.

```
noncomputable def covDerivConjFermion (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule (T.FermionValue i)) →ₗ[ℂ] T.LocalFieldAlgebra
```

def covDerivBoson [source]

The iterated covariant derivative of a bosonic species.

```
noncomputable def covDerivBoson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (T.BosonValue j) →ₗ[ℂ] T.LocalFieldAlgebra
```

def covDerivConjBoson [source]

The iterated covariant derivative of the conjugate components of a bosonic species.

```
noncomputable def covDerivConjBoson (j : T.BosonSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule (T.BosonValue j)) →ₗ[ℂ] T.LocalFieldAlgebra
```

lemma covDerivFermion_zero [source]

Zero covariant derivatives: the undifferentiated symbol.

```
lemma covDerivFermion_zero (i : T.FermionSpecies) (l : Fin 0 → (Fin 1 ⊕ Fin 3)) :
    T.covDerivFermion i l = T.fermionSymbol i 0
```

lemma covDerivConjFermion_zero [source]

```
lemma covDerivConjFermion_zero (i : T.FermionSpecies) (l : Fin 0 → (Fin 1 ⊕ Fin 3)) :
    T.covDerivConjFermion i l = T.conjFermionSymbol i 0
```

lemma covDerivBoson_zero [source]

```
lemma covDerivBoson_zero (j : T.BosonSpecies) (l : Fin 0 → (Fin 1 ⊕ Fin 3)) :
    T.covDerivBoson j l = T.bosonSymbol j 0
```

lemma covDerivConjBoson_zero [source]

```
lemma covDerivConjBoson_zero (j : T.BosonSpecies) (l : Fin 0 → (Fin 1 ⊕ Fin 3)) :
    T.covDerivConjBoson j l = T.conjBosonSymbol j 0
```

lemma covDerivFermion_succ [source]

One more covariant derivative, peeled off the front of the tuple: the derivative symbol `[∂_{l 0} ∇_{l'} ψ]` of the lower tower plus the derived action `A_{l 0} · ∇_{l'} ψ` of the gauge field on its value index, both read off the lower tower as a family.

```
lemma covDerivFermion_succ (i : T.FermionSpecies) {n : ℕ} (l : Fin (n + 1) → (Fin 1 ⊕ Fin 3)) :
    T.covDerivFermion i l
      = covDerivIter T.gaugeRealization.A (T.fermion i).repAlgebra (T.fermionSymbol i) n
          (fun k => l k.succ) {l 0}
        + actionFamConv T.gaugeRealization.A (T.fermion i).repAlgebra (l 0)
          (covDerivIter T.gaugeRealization.A (T.fermion i).repAlgebra (T.fermionSymbol i) n
            fun k => l k.succ) 0
```

lemma covDerivConjFermion_succ [source]

```
lemma covDerivConjFermion_succ (i : T.FermionSpecies) {n : ℕ}
    (l : Fin (n + 1) → (Fin 1 ⊕ Fin 3)) :
    T.covDerivConjFermion i l
      = covDerivIter T.gaugeRealization.A (LocalGaugeData.actionConj (T.fermion i).repAlgebra)
          (T.conjFermionSymbol i) n (fun k => l k.succ) {l 0}
        + actionFamConv T.gaugeRealization.A (LocalGaugeData.actionConj (T.fermion i).
    repAlgebra)
          (l 0) (covDerivIter T.gaugeRealization.A
            (LocalGaugeData.actionConj (T.fermion i).repAlgebra) (T.conjFermionSymbol i) n
            fun k => l k.succ) 0
```

lemma covDerivBoson_succ [source]

```
lemma covDerivBoson_succ (j : T.BosonSpecies) {n : ℕ} (l : Fin (n + 1) → (Fin 1 ⊕ Fin 3)) :
    T.covDerivBoson j l
      = covDerivIter T.gaugeRealization.A (T.boson j).repAlgebra (T.bosonSymbol j) n
          (fun k => l k.succ) {l 0}
        + actionFamConv T.gaugeRealization.A (T.boson j).repAlgebra (l 0)
          (covDerivIter T.gaugeRealization.A (T.boson j).repAlgebra (T.bosonSymbol j) n
            fun k => l k.succ) 0
```

lemma covDerivConjBoson_succ [source]

```
lemma covDerivConjBoson_succ (j : T.BosonSpecies) {n : ℕ}
    (l : Fin (n + 1) → (Fin 1 ⊕ Fin 3)) :
    T.covDerivConjBoson j l
      = covDerivIter T.gaugeRealization.A (LocalGaugeData.actionConj (T.boson j).repAlgebra)
          (T.conjBosonSymbol j) n (fun k => l k.succ) {l 0}
        + actionFamConv T.gaugeRealization.A (LocalGaugeData.actionConj (T.boson j).repAlgebra)
          (l 0) (covDerivIter T.gaugeRealization.A
            (LocalGaugeData.actionConj (T.boson j).repAlgebra) (T.conjBosonSymbol j) n
            fun k => l k.succ) 0
```

def covDerivFieldStrength [source]

The field strength and its ordered covariant derivatives `∇_l F_μν^φ`, included from the real gauge-only algebra through the connection factor.

```
noncomputable def covDerivFieldStrength (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] T.LocalFieldAlgebra
```

lemma covDerivFieldStrength_apply [source]

```
lemma covDerivFieldStrength_apply (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    T.covDerivFieldStrength l μ ν φ
      = T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ]
          LocalGaugeFieldAlgebra.covDerivFieldStrength 𝔤 l μ ν φ)
```

lemma covDerivFieldStrength_nil [source]

Zero covariant derivatives: the included field strength.

```
lemma covDerivFieldStrength_nil (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    T.covDerivFieldStrength [] μ ν φ
      = T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ] LocalGaugeFieldAlgebra.fieldStrength 𝔤 μ ν φ)
```

lemma covDerivFieldStrength_eq_iteratedCovDerivAdjoint [source]

The included tower is the covariant tower of the gauge-field symbols of `J(T)` computed by the realization theory, the tower being natural along the inclusion of the connection factor.

```
lemma covDerivFieldStrength_eq_iteratedCovDerivAdjoint (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    T.covDerivFieldStrength l μ ν φ
      = iteratedCovDerivAdjoint T.gaugeRealization.A l
          (fieldStrength T.gaugeRealization.A μ ν) 0 φ
```

lemma repJet_covDerivFermion [source]

The gauge law of the covariant tower of a fermionic species: a jet acts through the zeroth dual Taylor coefficient of its inverse on the value index alone.

```
lemma repJet_covDerivFermion (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i)) :
    T.repJet U (T.covDerivFermion i l φ)
      = T.covDerivFermion i l (repDualCoeff (T.fermion i).repJet U⁻¹ 0 φ)
```

lemma repJet_covDerivConjFermion [source]

The gauge law of the conjugate covariant tower of a fermionic species, through the conjugate representation.

```
lemma repJet_covDerivConjFermion (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.repJet U (T.covDerivConjFermion i l φ)
      = T.covDerivConjFermion i l
          (repDualCoeff (JetComponentSpace.repConj (T.fermion i).repJet) U⁻¹ 0 φ)
```

lemma repJet_covDerivBoson [source]

The gauge law of the covariant tower of a bosonic species.

```
lemma repJet_covDerivBoson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)) :
    T.repJet U (T.covDerivBoson j l φ)
      = T.covDerivBoson j l (repDualCoeff (T.boson j).repJet U⁻¹ 0 φ)
```

lemma repJet_covDerivConjBoson [source]

The gauge law of the conjugate covariant tower of a bosonic species.

```
lemma repJet_covDerivConjBoson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    T.repJet U (T.covDerivConjBoson j l φ)
      = T.covDerivConjBoson j l
          (repDualCoeff (JetComponentSpace.repConj (T.boson j).repJet) U⁻¹ 0 φ)
```

lemma repJet_covDerivFieldStrength [source]

The gauge law of the included field-strength tower: a jet acts through the dual adjoint action of the value of its inverse, from the gauge-only law.

```
lemma repJet_covDerivFieldStrength (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    T.repJet U (T.covDerivFieldStrength l μ ν φ)
      = T.covDerivFieldStrength l μ ν ((jets.adjoint (jets.eval U⁻¹)).dualMap φ)
```

lemma repJet_covDerivFermion_ofConstant_eval [source]

Under `MatterField.PureJetsActTrivially` for the species, the jet action on its covariant tower factors through evaluation: a jet acts as the constant jet of its value.

```
lemma repJet_covDerivFermion_ofConstant_eval (i : T.FermionSpecies)
    (hi : (T.fermion i).PureJetsActTrivially) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i)) :
    T.repJet U (T.covDerivFermion i l φ)
      = T.repJet (jets.ofConstant (jets.eval U)) (T.covDerivFermion i l φ)
```

lemma repJet_covDerivConjFermion_ofConstant_eval [source]

```
lemma repJet_covDerivConjFermion_ofConstant_eval (i : T.FermionSpecies)
    (hi : (T.fermion i).PureJetsActTrivially) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.repJet U (T.covDerivConjFermion i l φ)
      = T.repJet (jets.ofConstant (jets.eval U)) (T.covDerivConjFermion i l φ)
```

lemma repJet_covDerivBoson_ofConstant_eval [source]

```
lemma repJet_covDerivBoson_ofConstant_eval (j : T.BosonSpecies)
    (hj : (T.boson j).PureJetsActTrivially) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)) :
    T.repJet U (T.covDerivBoson j l φ)
      = T.repJet (jets.ofConstant (jets.eval U)) (T.covDerivBoson j l φ)
```

lemma repJet_covDerivConjBoson_ofConstant_eval [source]

```
lemma repJet_covDerivConjBoson_ofConstant_eval (j : T.BosonSpecies)
    (hj : (T.boson j).PureJetsActTrivially) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    T.repJet U (T.covDerivConjBoson j l φ)
      = T.repJet (jets.ofConstant (jets.eval U)) (T.covDerivConjBoson j l φ)
```

lemma repJet_covDerivFieldStrength_ofConstant_eval [source]

The jet action on the field-strength tower factors through evaluation, with no condition.

```
lemma repJet_covDerivFieldStrength_ofConstant_eval (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    T.repJet U (T.covDerivFieldStrength l μ ν φ)
      = T.repJet (jets.ofConstant (jets.eval U)) (T.covDerivFieldStrength l μ ν φ)
```

lemma repLorentzGroup_covDerivFermion [source]

The Lorentz law of the covariant tower of a fermionic species, under `MatterField.GaugeLorentzCompatible` for the species: every covariant slot mixes by the columns of the Lorentz matrix and the value index transforms contragrediently.

```
lemma repLorentzGroup_covDerivFermion (i : T.FermionSpecies)
    (hi : (T.fermion i).GaugeLorentzCompatible) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i)) :
    T.repLorentzGroup Λ (T.covDerivFermion i l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ k, L[Λ] (p k) (l k)) •
          T.covDerivFermion i p ((T.fermion i).repLorentz.dual Λ φ)
```

lemma repLorentzGroup_covDerivConjFermion [source]

The Lorentz law of the conjugate covariant tower of a fermionic species.

```
lemma repLorentzGroup_covDerivConjFermion (i : T.FermionSpecies)
    (hi : (T.fermion i).GaugeLorentzCompatible) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.repLorentzGroup Λ (T.covDerivConjFermion i l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ k, L[Λ] (p k) (l k)) •
          T.covDerivConjFermion i p ((T.fermion i).repLorentz.conj.dual Λ φ)
```

lemma repLorentzGroup_covDerivBoson [source]

The Lorentz law of the covariant tower of a bosonic species.

```
lemma repLorentzGroup_covDerivBoson (j : T.BosonSpecies)
    (hj : (T.boson j).GaugeLorentzCompatible) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)) :
    T.repLorentzGroup Λ (T.covDerivBoson j l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ k, L[Λ] (p k) (l k)) •
          T.covDerivBoson j p ((T.boson j).repLorentz.dual Λ φ)
```

lemma repLorentzGroup_covDerivConjBoson [source]

The Lorentz law of the conjugate covariant tower of a bosonic species.

```
lemma repLorentzGroup_covDerivConjBoson (j : T.BosonSpecies)
    (hj : (T.boson j).GaugeLorentzCompatible) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    T.repLorentzGroup Λ (T.covDerivConjBoson j l φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ k, L[Λ] (p k) (l k)) •
          T.covDerivConjBoson j p ((T.boson j).repLorentz.conj.dual Λ φ)
```

lemma includeConnection_one_tmul_real_smul [source]

A real scalar acting on the inclusion of a real element of the gauge-only algebra is the same complex scalar acting on it.

```
private lemma includeConnection_one_tmul_real_smul (r : ℝ) (x : LocalGaugeFieldAlgebra 𝔤) :
    T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ] (r • x))
      = ((r : ℝ) : ℂ) • T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ] x)
```

lemma repLorentzGroup_covDerivFieldStrength [source]

The Lorentz law of the field-strength tower, with no condition: every covariant slot and both covector indices mix by the columns of the Lorentz matrix.

```
lemma repLorentzGroup_covDerivFieldStrength {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    T.repLorentzGroup Λ (T.covDerivFieldStrength (List.ofFn l) μ ν φ)
      = ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ k, L[Λ] (p k) (l k)) •
          ∑ a, L[Λ] a μ • ∑ b, L[Λ] b ν • T.covDerivFieldStrength (List.ofFn p) a b φ
```

lemma repLorentzGroup_covDerivFermion_mem [source]

Under `MatterField.GaugeLorentzCompatible` for the species, a Lorentz transformation carries an element of the covariant tower of a fermionic species into any subalgebra containing the tower of that length.

```
lemma repLorentzGroup_covDerivFermion_mem (i : T.FermionSpecies)
    (hi : (T.fermion i).GaugeLorentzCompatible) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i))
    (hS : ∀ (p : Fin n → (Fin 1 ⊕ Fin 3)) (ψ : Module.Dual ℂ (T.FermionValue i)),
      T.covDerivFermion i p ψ ∈ S) :
    T.repLorentzGroup Λ (T.covDerivFermion i l φ) ∈ S
```

lemma repLorentzGroup_covDerivConjFermion_mem [source]

```
lemma repLorentzGroup_covDerivConjFermion_mem (i : T.FermionSpecies)
    (hi : (T.fermion i).GaugeLorentzCompatible) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i)))
    (hS : ∀ (p : Fin n → (Fin 1 ⊕ Fin 3)) (ψ : Module.Dual ℂ (ConjModule (T.FermionValue i))),
      T.covDerivConjFermion i p ψ ∈ S) :
    T.repLorentzGroup Λ (T.covDerivConjFermion i l φ) ∈ S
```

lemma repLorentzGroup_covDerivBoson_mem [source]

```
lemma repLorentzGroup_covDerivBoson_mem (j : T.BosonSpecies)
    (hj : (T.boson j).GaugeLorentzCompatible) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
```

```
    (φ : Module.Dual ℂ (T.BosonValue j))
    (hS : ∀ (p : Fin n → (Fin 1 ⊕ Fin 3)) (ψ : Module.Dual ℂ (T.BosonValue j)),
      T.covDerivBoson j p ψ ∈ S) :
    T.repLorentzGroup Λ (T.covDerivBoson j l φ) ∈ S
```

lemma repLorentzGroup_covDerivConjBoson_mem [source]

```
lemma repLorentzGroup_covDerivConjBoson_mem (j : T.BosonSpecies)
    (hj : (T.boson j).GaugeLorentzCompatible) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j)))
    (hS : ∀ (p : Fin n → (Fin 1 ⊕ Fin 3)) (ψ : Module.Dual ℂ (ConjModule (T.BosonValue j))),
      T.covDerivConjBoson j p ψ ∈ S) :
    T.repLorentzGroup Λ (T.covDerivConjBoson j l φ) ∈ S
```

lemma repLorentzGroup_covDerivFieldStrength_mem [source]

A Lorentz transformation carries an element of the field-strength tower into any subalgebra containing the whole tower of its adjoint covector, with no condition.

```
lemma repLorentzGroup_covDerivFieldStrength_mem (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤)
    (hS : ∀ (l' : List (Fin 1 ⊕ Fin 3)) (a b : Fin 1 ⊕ Fin 3),
      T.covDerivFieldStrength l' a b φ ∈ S) :
    T.repLorentzGroup Λ (T.covDerivFieldStrength l μ ν φ) ∈ S
```

2.206 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.GaugeAction

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/GaugeAction.lean on GitHub.

def gaugeConnection [source]

The connection generators after the action of a jet: the gauge-only action on the degree-one element, included into the local field algebra.

```
noncomputable def gaugeConnection (U : GJ) :
    GaugeBoson.JetComponentSpace 𝔤 →ₗ[ℝ] T.LocalFieldAlgebra
```

def gaugeAssignment [source]

The assignment of the generators defining the action of a jet: each matter species acts on its own component functions, the connection generators through the gauge-only action. The relations hold because the images are generators of the same kind, or lie in the central connection factor.

```
noncomputable def gaugeAssignment (U : GJ) : T.Assignment T.LocalFieldAlgebra where
```

def repJetAlgHom [source]

The action of a jet on the local field algebra, as an algebra endomorphism: the lift of its gauge assignment.

```
noncomputable def repJetAlgHom (U : GJ) : T.LocalFieldAlgebra →ₐ[ℂ] T.LocalFieldAlgebra
```

lemma repJetAlgHom_ιFermion [source]

```
@[simp]
lemma repJetAlgHom_ιFermion (U : GJ) (i : T.FermionSpecies)
    (x : JetComponentSpace (T.fermion i)) :
    T.repJetAlgHom U (T.ιFermion i x)
      = T.ιFermion i (JetComponentSpace.repJet (T.fermion i) U x)
```

lemma repJetAlgHom_ιBoson [source]

```
@[simp]
lemma repJetAlgHom_ιBoson (U : GJ) (j : T.BosonSpecies) (y : JetComponentSpace (T.boson j)) :
    T.repJetAlgHom U (T.ιBoson j y) = T.ιBoson j (JetComponentSpace.repJet (T.boson j) U y)
```

lemma repJetAlgHom_ιConnection [source]

```
@[simp]
lemma repJetAlgHom_ιConnection (U : GJ) (v : GaugeBoson.JetComponentSpace 𝔤) :
    T.repJetAlgHom U (T.ιConnection v)
      = T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ]
          LocalGaugeFieldAlgebra.repJet jets U (SymmetricAlgebra.ι ℝ _ v))
```

lemma repJetAlgHom_comp_includeConnection [source]

The action of a jet restricts to the complexified gauge-only action on the connection factor, as an equation of algebra maps out of the complexified gauge-only algebra: it is enough to compare them on the real generators, where it is the computation rule of the lift.

```
lemma repJetAlgHom_comp_includeConnection (U : GJ) :
    (T.repJetAlgHom U).comp T.includeConnection
      = T.includeConnection.comp (LocalGaugeFieldAlgebra.complexRepJetAlgHom jets U)
```

lemma repJetAlgHom_includeConnection [source]

The action of a jet restricts to the complexified gauge-only action on the connection factor.

```
lemma repJetAlgHom_includeConnection (U : GJ) (y : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) :
    T.repJetAlgHom U (T.includeConnection y)
      = T.includeConnection (LocalGaugeFieldAlgebra.complexRepJet jets U y)
```

lemma repJetAlgHom_includeConnection_one_tmul [source]

On the real gauge-only algebra, included, a jet acts through the gauge-only action.

```
lemma repJetAlgHom_includeConnection_one_tmul (U : GJ) (x : LocalGaugeFieldAlgebra 𝔤) :
    T.repJetAlgHom U (T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ] x))
      = T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ] LocalGaugeFieldAlgebra.repJet jets U x)
```

lemma repJetAlgHom_one [source]

The identity jet acts as the identity.

```
lemma repJetAlgHom_one : T.repJetAlgHom 1 = AlgHom.id ℂ T.LocalFieldAlgebra
```

lemma repJetAlgHom_mul [source]

The action of a product of jets is the composite of the actions.

```
lemma repJetAlgHom_mul (U V : GJ) :
    T.repJetAlgHom (U * V) = (T.repJetAlgHom U).comp (T.repJetAlgHom V)
```

def repJet [source]

The action of the jet gauge group on the local field algebra.

```
noncomputable def repJet : Representation ℂ GJ T.LocalFieldAlgebra where
```

lemma repJet_apply [source]

```
lemma repJet_apply (U : GJ) (x : T.LocalFieldAlgebra) :
    T.repJet U x = T.repJetAlgHom U x
```

lemma repJet_apply_mul [source]

```
lemma repJet_apply_mul (U : GJ) (x y : T.LocalFieldAlgebra) :
    T.repJet U (x * y) = T.repJet U x * T.repJet U y
```

lemma repJet_apply_one [source]

```
lemma repJet_apply_one (U : GJ) : T.repJet U (1 : T.LocalFieldAlgebra) = 1
```

lemma repJet_ιFermion [source]

```
@[simp]
lemma repJet_ιFermion (U : GJ) (i : T.FermionSpecies) (x : JetComponentSpace (T.fermion i)) :
    T.repJet U (T.ιFermion i x) = T.ιFermion i (JetComponentSpace.repJet (T.fermion i) U x)
```

lemma repJet_ιBoson [source]

```
@[simp]
lemma repJet_ιBoson (U : GJ) (j : T.BosonSpecies) (y : JetComponentSpace (T.boson j)) :
    T.repJet U (T.ιBoson j y) = T.ιBoson j (JetComponentSpace.repJet (T.boson j) U y)
```

lemma repJet_ιConnection [source]

```
@[simp]
lemma repJet_ιConnection (U : GJ) (v : GaugeBoson.JetComponentSpace 𝔤) :
    T.repJet U (T.ιConnection v)
      = T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ]
          LocalGaugeFieldAlgebra.repJet jets U (SymmetricAlgebra.ι ℝ _ v))
```

lemma repJet_includeConnection [source]

The jet gauge action restricts to the complexified gauge-only action on the connection factor.

```
lemma repJet_includeConnection (U : GJ) (y : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) :
    T.repJet U (T.includeConnection y)
      = T.includeConnection (LocalGaugeFieldAlgebra.complexRepJet jets U y)
```

lemma repJet_ιConnection_affine [source]

The affine gauge law of the connection generators: a jet carries a connection generator to the transported generator of its inverse plus the Maurer–Cartan shift, the gauge field being a connection and not a tensor. This is the generator-level form of `GaugeFieldData.repJet_ιConnection`, with no reference to the connection factor.

```
lemma repJet_ιConnection_affine (U : GJ) (v : GaugeBoson.JetComponentSpace 𝔤) :
    T.repJet U (T.ιConnection v)
      = T.ιConnection (LocalGaugeFieldAlgebra.transport jets U⁻¹ v)
        + (LocalGaugeFieldAlgebra.mcShift jets U⁻¹ v : ℂ) • (1 : T.LocalFieldAlgebra)
```

2.207 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.GaugeSector

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/GaugeSector.lean on GitHub.

def connectionAssignment [source]

The assignment sending every matter generator to zero and keeping the connection generators. Its relations hold in the commutative target.

```
noncomputable def connectionAssignment :
    T.Assignment (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) where
```

def connectionRetraction [source]

The retraction of the local field algebra onto the complexified gauge-only algebra: it forgets the matter symbols.

```
noncomputable def connectionRetraction :
    T.LocalFieldAlgebra →ₐ[ℂ] ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤
```

lemma connectionRetraction_ιConnection [source]

```
@[simp]
lemma connectionRetraction_ιConnection (v : GaugeBoson.JetComponentSpace 𝔤) :
    T.connectionRetraction (T.ιConnection v)
      = (1 : ℂ) ⊗ₜ[ℝ] SymmetricAlgebra.ι ℝ (GaugeBoson.JetComponentSpace 𝔤) v
```

lemma connectionRetraction_comp_includeConnection [source]

The retraction is a left inverse of the connection inclusion.

```
lemma connectionRetraction_comp_includeConnection :
    T.connectionRetraction.comp T.includeConnection
      = AlgHom.id ℂ (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤)
```

lemma connectionRetraction_includeConnection [source]

```
lemma connectionRetraction_includeConnection (y : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) :
    T.connectionRetraction (T.includeConnection y) = y
```

lemma includeConnection_injective [source]

The complexified gauge-only algebra embeds in the local field algebra.

```
lemma includeConnection_injective : Function.Injective T.includeConnection
```

lemma range_includeConnection [source]

The range of the connection inclusion is the gauge sector.

```
lemma range_includeConnection :
    T.includeConnection.range = T.SectorAlgebra {FieldCategory.gauge}
```

lemma includeConnection_mem_gaugeSector [source]

```
lemma includeConnection_mem_gaugeSector (y : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) :
    T.includeConnection y ∈ T.SectorAlgebra {FieldCategory.gauge}
```

def gaugeSectorEquiv [source]

The gauge sector is the complexification of the gauge-only algebra: the connection inclusion, corestricted to its range. The source is complexified because the gauge-only algebra is real while the sector is a complex subalgebra.

```
noncomputable def gaugeSectorEquiv :
    (ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) ≃ₐ[ℂ] ↥(T.SectorAlgebra {FieldCategory.gauge})
```

lemma coe_gaugeSectorEquiv [source]

In the local field algebra the equivalence is the canonical inclusion.

```
@[simp]
lemma coe_gaugeSectorEquiv (y : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) :
    (T.gaugeSectorEquiv y : T.LocalFieldAlgebra) = T.includeConnection y
```

lemma gaugeSectorEquiv_symm_includeConnection [source]

The inverse recovers an element from its inclusion.

```
lemma gaugeSectorEquiv_symm_includeConnection (y : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤)
    (hy : T.includeConnection y ∈ T.SectorAlgebra {FieldCategory.gauge}) :
    T.gaugeSectorEquiv.symm ⟨T.includeConnection y, hy⟩ = y
```

lemma coe_gaugeSectorEquiv_one_tmul_ι [source]

A real gauge-only generator, embedded with the scalar `1`, is the corresponding connection symbol.

```
lemma coe_gaugeSectorEquiv_one_tmul_ι (v : GaugeBoson.JetComponentSpace 𝔤) :
    (T.gaugeSectorEquiv ((1 : ℂ) ⊗ₜ[ℝ]
        SymmetricAlgebra.ι ℝ (GaugeBoson.JetComponentSpace 𝔤) v) : T.LocalFieldAlgebra)
      = T.ιConnection v
```

lemma coe_gaugeSectorEquiv_tmul [source]

`z ⊗ₜ x` maps to `z` times the image of `1 ⊗ₜ x`.

```
lemma coe_gaugeSectorEquiv_tmul (z : ℂ) (x : LocalGaugeFieldAlgebra 𝔤) :
    (T.gaugeSectorEquiv (z ⊗ₜ[ℝ] x) : T.LocalFieldAlgebra)
      = z • (T.gaugeSectorEquiv ((1 : ℂ) ⊗ₜ[ℝ] x) : T.LocalFieldAlgebra)
```

lemma coe_gaugeSectorEquiv_gaugeField [source]

The gauge-field symbol `∂_s A_μ^φ`, for any derivative multiset, spacetime index and adjoint covector, is the connection symbol with the same labels.

```
lemma coe_gaugeSectorEquiv_gaugeField (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    (T.gaugeSectorEquiv (LocalGaugeFieldAlgebra.gaugeField 𝔤 s μ φ) : T.LocalFieldAlgebra)
      = T.gaugeRealization.A s μ φ
```

lemma gaugeSectorEquiv_complexRepJet [source]

The equivalence intertwines the complexified gauge-only jet action with the restricted jet action of the gauge sector.

```
lemma gaugeSectorEquiv_complexRepJet (U : GJ) (y : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) :
    T.gaugeSectorEquiv (LocalGaugeFieldAlgebra.complexRepJet jets U y)
      = SectorAlgebra.repJet T {FieldCategory.gauge} U (T.gaugeSectorEquiv y)
```

lemma gaugeSectorEquiv_complexRepLorentzGroup [source]

The equivalence intertwines the complexified gauge-only Lorentz action with the restricted Lorentz action of the gauge sector.

```
lemma gaugeSectorEquiv_complexRepLorentzGroup (Λ : SL(2,ℂ))
    (y : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) :
    T.gaugeSectorEquiv (LocalGaugeFieldAlgebra.complexRepLorentzGroup 𝔤 Λ y)
      = SectorAlgebra.repLorentzGroup T {FieldCategory.gauge} Λ (T.gaugeSectorEquiv y)
```

2.208 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.GaugeSectorRealization

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/GaugeSectorRealization.lean on GitHub.

def gaugeSectorRealizationEquiv [source]

Realizations of the ordinary gauge sector are the real realizations of the gauge-only algebra, the target keeping its complex actions restricted to `ℝ`.

```
noncomputable def gaugeSectorRealizationEquiv :
    SectorAlgebra.Realization T {FieldCategory.gauge} B repJet repLorentz
      ≃ LocalGaugeFieldAlgebra.Realization jets B (repJet.restrictScalars ℝ)
```

```
          (repLorentz.restrictScalars ℝ)
```

lemma gaugeSectorRealizationEquiv_toAlgHom [source]

```
lemma gaugeSectorRealizationEquiv_toAlgHom
    (f : SectorAlgebra.Realization T {FieldCategory.gauge} B repJet repLorentz) :
    (T.gaugeSectorRealizationEquiv f).toAlgHom
      = ((f.toAlgHom.comp T.gaugeSectorEquiv.toAlgHom).restrictScalars ℝ).comp
          Algebra.TensorProduct.includeRight
```

lemma gaugeSectorRealizationEquiv_toAlgHom_apply [source]

```
@[simp]
lemma gaugeSectorRealizationEquiv_toAlgHom_apply
    (f : SectorAlgebra.Realization T {FieldCategory.gauge} B repJet repLorentz)
    (x : LocalGaugeFieldAlgebra 𝓰) :
    (T.gaugeSectorRealizationEquiv f).toAlgHom x
      = f.toAlgHom (T.gaugeSectorEquiv ((1 : ℂ) ⊗ₜ[ℝ] x))
```

lemma gaugeSectorRealizationEquiv_A [source]

The gauge symbols of the corresponding real realization are the sector symbols.

```
lemma gaugeSectorRealizationEquiv_A
    (f : SectorAlgebra.Realization T {FieldCategory.gauge} B repJet repLorentz)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝓰) :
    (T.gaugeSectorRealizationEquiv f).A s μ φ
      = f.toAlgHom (T.gaugeSectorEquiv (LocalGaugeFieldAlgebra.gaugeField 𝓰 s μ φ))
```

lemma gaugeSectorRealizationEquiv_symm_toAlgHom [source]

```
lemma gaugeSectorRealizationEquiv_symm_toAlgHom
    (h : LocalGaugeFieldAlgebra.Realization jets B (repJet.restrictScalars ℝ)
      (repLorentz.restrictScalars ℝ)) :
    (T.gaugeSectorRealizationEquiv.symm h).toAlgHom
      = (AlgHom.liftEquiv ℝ ℂ (LocalGaugeFieldAlgebra 𝓰) B h.toAlgHom).comp
          T.gaugeSectorEquiv.symm.toAlgHom
```

lemma gaugeSectorRealizationEquiv_symm_toAlgHom_apply [source]

```
@[simp]
lemma gaugeSectorRealizationEquiv_symm_toAlgHom_apply
    (h : LocalGaugeFieldAlgebra.Realization jets B (repJet.restrictScalars ℝ)
      (repLorentz.restrictScalars ℝ)) (x : T.SectorAlgebra {FieldCategory.gauge}) :
    (T.gaugeSectorRealizationEquiv.symm h).toAlgHom x
      = AlgHom.liftEquiv ℝ ℂ (LocalGaugeFieldAlgebra 𝓰) B h.toAlgHom
          (T.gaugeSectorEquiv.symm x)
```

lemma gaugeSectorRealizationEquiv_symm_toAlgHom_gaugeField [source]

The sector symbols of the corresponding sector realization are the gauge symbols.

```
lemma gaugeSectorRealizationEquiv_symm_toAlgHom_gaugeField
    (h : LocalGaugeFieldAlgebra.Realization jets B (repJet.restrictScalars ℝ)
```

```
    (repLorentz.restrictScalars ℝ)) (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
  (φ : Module.Dual ℝ g) :
  (T.gaugeSectorRealizationEquiv.symm h).toAlgHom
      (T.gaugeSectorEquiv (LocalGaugeFieldAlgebra.gaugeField g s μ φ)) = h.A s μ φ
```

2.209 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.Jet

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/Jet.lean on GitHub.

def jetOfConstant [source]

The inclusion of constants into `V`-valued jets: `v ↦ 1 ⊗ v`.

```
noncomputable def jetOfConstant : V →ₗ[ℂ] SpaceTimeAlgebra ⊗[ℂ] V
```

lemma jetOfConstant_apply [source]

```
@[simp]
lemma jetOfConstant_apply (v : V) : jetOfConstant v = (1 : SpaceTimeAlgebra) ⊗ₜ[ℂ] v
```

def jetDeriv [source]

The formal derivative on `V`-valued jets in the direction `μ`, acting on the jet factor.

```
noncomputable def jetDeriv (μ : Fin 1 ⊕ Fin 3) :
    SpaceTimeAlgebra ⊗[ℂ] V →ₗ[ℂ] SpaceTimeAlgebra ⊗[ℂ] V
```

lemma jetDeriv_tmul [source]

```
@[simp]
lemma jetDeriv_tmul (μ : Fin 1 ⊕ Fin 3) (f : SpaceTimeAlgebra) (v : V) :
    jetDeriv μ (f ⊗ₜ[ℂ] v) = pderiv μ f ⊗ₜ[ℂ] v
```

lemma jetDeriv_comm [source]

Formal derivatives on `V`-valued jets commute, since the partial derivatives of jets do.

```
lemma jetDeriv_comm (μ ν : Fin 1 ⊕ Fin 3) :
    (jetDeriv (V := V) μ).comp (jetDeriv ν) = (jetDeriv ν).comp (jetDeriv μ)
```

instance RightCommutative (fun (L : SpaceTimeAlgebra ⊗[ℂ] V →ₗ[ℂ] SpaceTimeAlgeb… [source]

Post-composition with `jetDeriv` is right-commutative, which is what allows iterated derivatives to be indexed by a `Multiset` of directions.

```
instance : RightCommutative (fun (L : SpaceTimeAlgebra ⊗[ℂ] V →ₗ[ℂ] SpaceTimeAlgebra ⊗[ℂ] V)
    (μ : Fin 1 ⊕ Fin 3) => L.comp (jetDeriv μ)) where
```

def jetIteratedDeriv [source]

The iterated formal derivative on `V`-valued jets, in the (unordered) directions given by the multiset `μs`.

```
noncomputable def jetIteratedDeriv (μs : Multiset (Fin 1 ⊕ Fin 3)) :
    SpaceTimeAlgebra ⊗[ℂ] V →ₗ[ℂ] SpaceTimeAlgebra ⊗[ℂ] V
```

lemma jetIteratedDeriv_zero [source]

```
@[simp]
lemma jetIteratedDeriv_zero :
    jetIteratedDeriv (V := V) (0 : Multiset (Fin 1 ⊕ Fin 3)) = LinearMap.id
```

lemma jetIteratedDeriv_cons [source]

```
lemma jetIteratedDeriv_cons (μ : Fin 1 ⊕ Fin 3) (μs : Multiset (Fin 1 ⊕ Fin 3)) :
    jetIteratedDeriv (V := V) (μ ::ₘ μs) = (jetDeriv μ).comp (jetIteratedDeriv μs)
```

lemma jetIteratedDeriv_add [source]

The iterated derivative is additive in the multiset of directions.

```
lemma jetIteratedDeriv_add (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    jetIteratedDeriv (V := V) (s + t) =
      (jetIteratedDeriv s).comp (jetIteratedDeriv t)
```

lemma jetIteratedDeriv_singleton [source]

```
@[simp]
lemma jetIteratedDeriv_singleton (μ : Fin 1 ⊕ Fin 3) :
    jetIteratedDeriv (V := V) ({μ} : Multiset (Fin 1 ⊕ Fin 3)) = jetDeriv μ
```

def jetEval [source]

Evaluation of a `V`-valued jet at the base point: `f ⊗ v ↦ (constant coefficient of f) • v`. This is a retraction of `jetOfConstant`.

```
noncomputable def jetEval : SpaceTimeAlgebra ⊗[ℂ] V →ₗ[ℂ] V
```

lemma jetEval_tmul [source]

```
@[simp]
lemma jetEval_tmul (f : SpaceTimeAlgebra) (v : V) :
    jetEval (f ⊗ₜ[ℂ] v) = constantCoeff f • v
```

lemma jetEval_jetOfConstant [source]

```
lemma jetEval_jetOfConstant (v : V) : jetEval (jetOfConstant v) = v
```

lemma jetEval_smul [source]

Evaluation at the base point is semilinear over the jet ring: a scalar jet acts through its constant coefficient.

```
lemma jetEval_smul (f : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V) :
    jetEval (f • z) = constantCoeff f • jetEval z
```

abbrev jetProdEquiv [source]

***The jets of a product are the product of the jets**: `SpaceTimeAlgebra ⊗ (V × W)` splits as `(SpaceTimeAlgebra ⊗ V) × (SpaceTimeAlgebra ⊗ W)`, the jet-ring factor being shared.*

```
noncomputable abbrev jetProdEquiv :
    SpaceTimeAlgebra ⊗[ℂ] (V × W) ≃ₗ[ℂ] (SpaceTimeAlgebra ⊗[ℂ] V) × (SpaceTimeAlgebra ⊗[ℂ] W)
```

lemma jetProdEquiv_jetOfConstant [source]

```
@[simp]
lemma jetProdEquiv_jetOfConstant (v : V) (w : W) :
    jetProdEquiv (jetOfConstant (v, w)) = (jetOfConstant v, jetOfConstant w)
```

lemma jetProdEquiv_jetDeriv [source]

```
lemma jetProdEquiv_jetDeriv (μ : Fin 1 ⊕ Fin 3) (z : SpaceTimeAlgebra ⊗[ℂ] (V × W)) :
    jetProdEquiv (jetDeriv μ z) =
      (jetDeriv μ (jetProdEquiv z).1, jetDeriv μ (jetProdEquiv z).2)
```

lemma jetProdEquiv_jetIteratedDeriv [source]

```
lemma jetProdEquiv_jetIteratedDeriv (s : Multiset (Fin 1 ⊕ Fin 3))
    (z : SpaceTimeAlgebra ⊗[ℂ] (V × W)) :
    jetProdEquiv (jetIteratedDeriv s z) =
      (jetIteratedDeriv s (jetProdEquiv z).1, jetIteratedDeriv s (jetProdEquiv z).2)
```

lemma jetProdEquiv_smul [source]

The identification is `SpaceTimeAlgebra`-linear: multiplication by a scalar jet acts on both components.

```
lemma jetProdEquiv_smul (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] (V × W)) :
    jetProdEquiv (χ • z) = (χ • (jetProdEquiv z).1, χ • (jetProdEquiv z).2)
```

lemma jetProdEquiv_symm_smul [source]

```
lemma jetProdEquiv_symm_smul (χ : SpaceTimeAlgebra) (a : SpaceTimeAlgebra ⊗[ℂ] V)
    (b : SpaceTimeAlgebra ⊗[ℂ] W) :
    (jetProdEquiv (V := V) (W := W)).symm (χ • a, χ • b)
      = χ • (jetProdEquiv (V := V) (W := W)).symm (a, b)
```

lemma jetEval_prod [source]

```
lemma jetEval_prod (z : SpaceTimeAlgebra ⊗[ℂ] (V × W)) :
    jetEval z = (jetEval (jetProdEquiv z).1, jetEval (jetProdEquiv z).2)
```

abbrev jetPiEquiv [source]

***The jets of a finite product are the product of the jets**: `SpaceTimeAlgebra ⊗ (∀ i, E i)` splits as `∀ i, SpaceTimeAlgebra ⊗ E i`, the jet-ring factor being shared.*

```
noncomputable abbrev jetPiEquiv :
    SpaceTimeAlgebra ⊗[ℂ] (∀ i, E i) ≃ₗ[ℂ] ∀ i, SpaceTimeAlgebra ⊗[ℂ] E i
```

lemma jetPiEquiv_jetOfConstant [source]

```
lemma jetPiEquiv_jetOfConstant (v : ∀ i, E i) :
    jetPiEquiv E (jetOfConstant v) = fun i => jetOfConstant (v i)
```

lemma jetPiEquiv_jetDeriv [source]

```
lemma jetPiEquiv_jetDeriv (μ : Fin 1 ⊕ Fin 3) (z : SpaceTimeAlgebra ⊗[ℂ] (∀ i, E i)) (i : ι) :
    jetPiEquiv E (jetDeriv μ z) i = jetDeriv μ (jetPiEquiv E z i)
```

lemma jetPiEquiv_jetIteratedDeriv [source]

```
lemma jetPiEquiv_jetIteratedDeriv (s : Multiset (Fin 1 ⊕ Fin 3))
    (z : SpaceTimeAlgebra ⊗[ℂ] (∀ i, E i)) (i : ι) :
    jetPiEquiv E (jetIteratedDeriv s z) i = jetIteratedDeriv s (jetPiEquiv E z i)
```

lemma jetPiEquiv_smul [source]

The identification is `SpaceTimeAlgebra`-linear: multiplication by a scalar jet acts on every component.

```
lemma jetPiEquiv_smul (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] (∀ i, E i)) (i : ι) :
    jetPiEquiv E (χ • z) i = χ • (jetPiEquiv E z i)
```

lemma jetPiEquiv_symm_smul [source]

```
lemma jetPiEquiv_symm_smul (χ : SpaceTimeAlgebra) (a : ∀ i, SpaceTimeAlgebra ⊗[ℂ] E i) :
    (jetPiEquiv E).symm (fun i => χ • a i) = χ • (jetPiEquiv E).symm a
```

lemma jetEval_pi [source]

```
lemma jetEval_pi (z : SpaceTimeAlgebra ⊗[ℂ] (∀ i, E i)) (i : ι) :
    jetEval z i = jetEval (jetPiEquiv E z i)
```

lemma jetPiEquiv_eq_lTensor_proj [source]

The component of the splitting is the projection on the value factor. Reading off the summand `i` of a jet of a `(∀ i, E i)`-valued field is applying the projection onto that summand to the value factor, the jet-ring factor being untouched. This is the form in which the splitting meets the naturality statements, which are all phrased in terms of linear maps of value spaces.

```
lemma jetPiEquiv_eq_lTensor_proj (z : SpaceTimeAlgebra ⊗[ℂ] (∀ i, E i)) (i : ι) :
    jetPiEquiv E z i = LinearMap.lTensor SpaceTimeAlgebra (LinearMap.proj i) z
```

lemma jetPi_hom_ext [source]

Two maps into the jets of a product agree as soon as their species components do. The splitting is an equivalence, so a jet of a `(∀ i, E i)`-valued field is determined by its summands.

```
lemma jetPi_hom_ext {N : Type} [AddCommGroup N] [Module ℂ N]
    {A B : N →ₗ[ℂ] SpaceTimeAlgebra ⊗[ℂ] (∀ i, E i)}
    (h : ∀ i, (LinearMap.lTensor SpaceTimeAlgebra (LinearMap.proj i)).comp A
      = (LinearMap.lTensor SpaceTimeAlgebra (LinearMap.proj i)).comp B) : A = B
```

2.210 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.JetRep

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/JetRep.lean on GitHub.

lemma rep_tmul_of_smul_comm [source]

***A fibrewise action is determined by its values on constant jets.** If the gauge action commutes with multiplication by scalar jets — the statement that it acts on the values of the field, over the identity on spacetime — then its value on a general jet `f ⊗ₜ v` is the constant-jet value `rep U (1 ⊗ₜ v)` scaled by `f`.*

```
lemma rep_tmul_of_smul_comm
    {rep : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V)}
    (hlin : ∀ (U : G) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V),
      rep U (χ • z) = χ • rep U z)
    (U : G) (f : SpaceTimeAlgebra) (v : V) :
    rep U (f ⊗ₜ[ℂ] v) = f • rep U (jetOfConstant v)
```

lemma lift_mul_tmul [source]

***The canonical evaluation is a right module map.** Writing `ev` for the canonical `SpaceTimeAlgebra ⊗ End V →ₗ[SpaceTimeAlgebra] (V →ₗ SpaceTimeAlgebra ⊗ V)`, `g ⊗ T ↦ (v ↦ g ⊗ₜ T v)`, multiplying on the right by `b ⊗ T` applies `T` to the argument and scales the value by `b`.*

```
lemma lift_mul_tmul (x : SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V)
    (b : SpaceTimeAlgebra) (T : Module.End ℂ V) (v : V) :
    TensorProduct.lift ((LinearMap.llcomp ℂ V V (SpaceTimeAlgebra ⊗[ℂ] V)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra V)) (x * (b ⊗ₜ[ℂ] T)) v
      = b • TensorProduct.lift ((LinearMap.llcomp ℂ V V (SpaceTimeAlgebra ⊗[ℂ] V)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra V)) x (T v)
```

lemma rep_lift_of_smul_comm [source]

***A fibrewise action is the `SpaceTimeAlgebra`-linear extension of its coefficient.** If the element `x` of `SpaceTimeAlgebra ⊗ End V` records `rep U` on constant jets, then `rep U` agrees with left multiplication by `x` on every coefficient `y`.*

```
lemma rep_lift_of_smul_comm
    {rep : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V)}
    (hlin : ∀ (U : G) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V),
      rep U (χ • z) = χ • rep U z)
    (U : G) (x : SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V)
    (hx : ∀ v : V, TensorProduct.lift ((LinearMap.llcomp ℂ V V (SpaceTimeAlgebra ⊗[ℂ] V)).comp
      (TensorProduct.mk ℂ SpaceTimeAlgebra V)) x v = rep U (jetOfConstant v))
```

```
    (y : SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V) (v : V) :
    rep U (TensorProduct.lift ((LinearMap.llcomp ℂ V V (SpaceTimeAlgebra ⊗[ℂ] V)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra V)) y v)
      = TensorProduct.lift ((LinearMap.llcomp ℂ V V (SpaceTimeAlgebra ⊗[ℂ] V)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra V)) (x * y) v
```

lemma coeff_mul_of_smul_comm [source]

***The coefficient of a fibrewise action is multiplicative.** Recording `rep` on constant jets as a family `c` in `SpaceTimeAlgebra ⊗ End V`, group multiplication becomes multiplication in that algebra. This is the identity that makes the induced action on the symbols a representation, and it needs no basis.*

```
lemma coeff_mul_of_smul_comm
    {rep : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V)}
    (hlin : ∀ (U : G) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V),
      rep U (χ • z) = χ • rep U z)
    (c : G → SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V)
    (hc : ∀ (U : G) (v : V),
      TensorProduct.lift ((LinearMap.llcomp ℂ V V (SpaceTimeAlgebra ⊗[ℂ] V)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra V)) (c U) v = rep U (jetOfConstant v))
    (U W : G) (v : V) :
    TensorProduct.lift ((LinearMap.llcomp ℂ V V (SpaceTimeAlgebra ⊗[ℂ] V)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra V)) (c U * c W) v
      = rep (U * W) (jetOfConstant v)
```

lemma symbolAction_mul [source]

***The symbol action of a coefficient is an anti-homomorphism.** Let `θ` send a coefficient `g ⊗ T` in `SpaceTimeAlgebra ⊗ End V` to the endomorphism `jetRingAction g ⊗ T^T` of the symbol space `SpaceTimeDerivAlgebraℂ ⊗ Dual V`. Then `θ` reverses products: the jet-ring factor is multiplicative (`jetRingAction_mul`, and `SpaceTimeAlgebra` is commutative) while the target factor is contravariant (`Module.Dual.transpose_comp`). Composed with `U ↦ U⁻¹` this is exactly what makes the induced action a representation, with no induction over the antidiagonal.*

```
lemma symbolAction_mul
    (θ : (SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V) →ₗ[ℂ]
      Module.End ℂ (SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ V))
    (hθ : ∀ (g : SpaceTimeAlgebra) (T : Module.End ℂ V),
      θ (g ⊗ₜ[ℂ] T) = TensorProduct.map (SpaceTimeDerivAlgebraℂ.jetRingAction g)
        (Module.Dual.transpose T))
    (x y : SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V) :
    θ (x * y) = θ y ∘ₗ θ x
```

lemma lift_congr_leftComm [source]

***The coefficient of a linear map, canonically.** For finite-dimensional `V` the canonical `SpaceTimeAlgebra ⊗ (V →ₗ W) → (V →ₗ SpaceTimeAlgebra ⊗ W)` is inverted by reassociating the contraction `Dual V ⊗ (SpaceTimeAlgebra ⊗ W) ≃ SpaceTimeAlgebra ⊗ (Dual V ⊗ W) ≃ SpaceTimeAlgebra ⊗ (V →ₗ W). This is the finite-rank input, obtained from dualTensorHomEquiv` rather than from a basis. Only the source `V` has to be finite-dimensional; the target is arbitrary, which is what lets the naturality statements below compare two different value spaces.*

```
lemma lift_congr_leftComm [Module.Free ℂ V] [Module.Finite ℂ V]
    (G : Module.Dual ℂ V ⊗[ℂ] (SpaceTimeAlgebra ⊗[ℂ] W)) (v : V) :
    TensorProduct.lift ((LinearMap.llcomp ℂ V W (SpaceTimeAlgebra ⊗[ℂ] W)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra W))
        ((TensorProduct.congr (LinearEquiv.refl ℂ SpaceTimeAlgebra) (dualTensorHomEquiv ℂ V W))
          (TensorProduct.leftComm ℂ (Module.Dual ℂ V) SpaceTimeAlgebra W G)) v
      = dualTensorHom ℂ V (SpaceTimeAlgebra ⊗[ℂ] W) G v
```

def repConj [source]

***The conjugate jet action.** Given a gauge action on the jets of a `V`-valued field, this is the induced action on the jets of the *conjugate* field.*

It is `Representation.conj rep` — the same underlying maps, read on `ConjModule` — carried across the identification

`ConjModule (SpaceTimeAlgebra ⊗[ℂ] V) ≃ₗ[ℂ] SpaceTimeAlgebra ⊗[ℂ] ConjModule V`

which is `ConjModule.tensorEquiv` (conjugation is monoidal) followed by `SpaceTimeAlgebra.starConjEquiv` on the jet-ring factor (the real structure of the jet ring). On pure tensors the composite is `f ⊗ₜ v ↦ star f ⊗ₜ v`, so `repConj` carries the conjugate gauge matrix — the physicists' `ψ̄ ↦ ψ̄ U†`.

Being a representation is free: `LinearEquiv.conjRingEquiv` is a ring equivalence of endomorphism rings, hence multiplicative.

```
noncomputable def repConj (rep : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V)) :
    Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] ConjModule V) where
```

lemma repConj_apply_tmul [source]

On pure tensors the conjugate jet action conjugates the jet factor: it is `rep` evaluated at `star f ⊗ₜ v`, read back through the same identification.

```
lemma repConj_apply_tmul (rep : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V))
    (U : G) (f : SpaceTimeAlgebra) (v : V) :
    repConj rep U (f ⊗ₜ[ℂ] conjEquiv (k := ℂ) (M := V) v)
      = ((ConjModule.tensorEquiv (k := ℂ) (M := SpaceTimeAlgebra) (N := V)).symm.trans
          (TensorProduct.congr SpaceTimeAlgebra.starConjEquiv (LinearEquiv.refl ℂ (ConjModule V
    ))))
        (conjEquiv (k := ℂ) (M := SpaceTimeAlgebra ⊗[ℂ] V) (rep U (star f ⊗ₜ[ℂ] v)))
```

lemma tensorEquiv_congr_conjEquiv_smul [source]

***The identification conjugates the jet-ring action.** Carrying a `V`-valued jet over to the conjugate side turns multiplication by `star χ` into multiplication by `χ`: the `star` on the jet-ring factor is exactly what absorbs the conjugation.*

```
lemma tensorEquiv_congr_conjEquiv_smul (χ : SpaceTimeAlgebra) (y : SpaceTimeAlgebra ⊗[ℂ] V) :
    ((ConjModule.tensorEquiv (k := ℂ) (M := SpaceTimeAlgebra) (N := V)).symm.trans
        (TensorProduct.congr SpaceTimeAlgebra.starConjEquiv (LinearEquiv.refl ℂ (ConjModule V))
    ))
          (conjEquiv (k := ℂ) (M := SpaceTimeAlgebra ⊗[ℂ] V) (star χ • y))
      = χ • ((ConjModule.tensorEquiv (k := ℂ) (M := SpaceTimeAlgebra) (N := V)).symm.trans
        (TensorProduct.congr SpaceTimeAlgebra.starConjEquiv (LinearEquiv.refl ℂ (ConjModule V))
    ))
```

```
        (conjEquiv (k := ℂ) (M := SpaceTimeAlgebra ⊗[ℂ] V) y)
```

lemma repConj_apply_conjEquiv [source]

The conjugate jet action is the original one, read through the identification. On a jet carried over to the conjugate side, `repConj rep U` is `rep U` applied on the original side and carried over again. Everything about `repConj` beyond its definition follows from this.

```
lemma repConj_apply_conjEquiv (rep : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V)) (U : G)
    (y : SpaceTimeAlgebra ⊗[ℂ] V) :
    repConj rep U (((ConjModule.tensorEquiv (k := ℂ) (M := SpaceTimeAlgebra) (N := V)).symm.
    trans
        (TensorProduct.congr SpaceTimeAlgebra.starConjEquiv (LinearEquiv.refl ℂ (ConjModule V))
    ))
        (conjEquiv (k := ℂ) (M := SpaceTimeAlgebra ⊗[ℂ] V) y))
      = ((ConjModule.tensorEquiv (k := ℂ) (M := SpaceTimeAlgebra) (N := V)).symm.trans
          (TensorProduct.congr SpaceTimeAlgebra.starConjEquiv (LinearEquiv.refl ℂ (ConjModule V
    ))))
        (conjEquiv (k := ℂ) (M := SpaceTimeAlgebra ⊗[ℂ] V) (rep U y))
```

lemma repConj_smul_comm [source]

***The conjugate jet action is fibrewise-linear whenever the original is.** This is what lets the coefficient machinery of `coeff_mul_of_smul_comm` be instantiated at `ConjModule V`, giving the conjugate half of the symbol action.*

```
lemma repConj_smul_comm
    {rep : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V)}
    (hlin : ∀ (U : G) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V),
      rep U (χ • z) = χ • rep U z)
    (U : G) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] ConjModule V) :
    repConj rep U (χ • z) = χ • repConj rep U z
```

lemma lTensor_conjEquiv_naturality [source]

The identification of the conjugate of the jets with the jets of the conjugate is natural in the value space. A linear map of value spaces acts on either side by the same underlying map, so carrying a jet over to the conjugate side commutes with it.

```
lemma lTensor_conjEquiv_naturality (f : V →ₗ[ℂ] W) (y : SpaceTimeAlgebra ⊗[ℂ] V) :
    (LinearMap.lTensor SpaceTimeAlgebra (ConjModule.map (k := ℂ) f))
        (((ConjModule.tensorEquiv (k := ℂ) (M := SpaceTimeAlgebra) (N := V)).symm.trans
          (TensorProduct.congr SpaceTimeAlgebra.starConjEquiv (LinearEquiv.refl ℂ (ConjModule V
    ))))
          (conjEquiv (k := ℂ) (M := SpaceTimeAlgebra ⊗[ℂ] V) y))
      = ((ConjModule.tensorEquiv (k := ℂ) (M := SpaceTimeAlgebra) (N := W)).symm.trans
          (TensorProduct.congr SpaceTimeAlgebra.starConjEquiv (LinearEquiv.refl ℂ (ConjModule W
    ))))
        (conjEquiv (k := ℂ) (M := SpaceTimeAlgebra ⊗[ℂ] W)
          ((LinearMap.lTensor SpaceTimeAlgebra f) y))
```

lemma lTensor_comp_repConj [source]

The conjugate jet action is natural in the value space. A linear map of value spaces intertwining two fibrewise jet actions intertwines the actions on the jets of the conjugate fields, through the induced map of conjugate modules. This is what supplies the conjugate half of the naturality of the gauge action on the jet component space, which does not follow from the unconjugated half.

```
lemma lTensor_comp_repConj (repV : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V))
    (repW : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] W)) (f : V →ₗ[ℂ] W)
    (hf : ∀ U : G, (LinearMap.lTensor SpaceTimeAlgebra f).comp (repV U)
      = (repW U).comp (LinearMap.lTensor SpaceTimeAlgebra f)) (U : G) :
    (LinearMap.lTensor SpaceTimeAlgebra (ConjModule.map (k := ℂ) f)).comp (repConj repV U)
      = (repConj repW U).comp (LinearMap.lTensor SpaceTimeAlgebra (ConjModule.map (k := ℂ) f))
```

lemma lift_injective

[source]

***The coefficient is determined by its action on constants.** For finite-dimensional `V` the canonical evaluation `SpaceTimeAlgebra ⊗ (V →ₗ W) → (V →ₗ SpaceTimeAlgebra ⊗ W)` is injective.*

```
lemma lift_injective [Module.Free ℂ V] [Module.Finite ℂ V]
    {x y : SpaceTimeAlgebra ⊗[ℂ] (V →ₗ[ℂ] W)}
    (h : ∀ v : V, TensorProduct.lift ((LinearMap.llcomp ℂ V W (SpaceTimeAlgebra ⊗[ℂ] W)).comp
      (TensorProduct.mk ℂ SpaceTimeAlgebra W)) x v = TensorProduct.lift ((LinearMap.llcomp ℂ V
    W
        (SpaceTimeAlgebra ⊗[ℂ] W)).comp
      (TensorProduct.mk ℂ SpaceTimeAlgebra W)) y v) : x = y
```

def jetCoeff

[source]

***The coefficient of a fibrewise gauge action.** For finite-dimensional `V`, the restriction of `rep U` to constant jets is an element of `SpaceTimeAlgebra ⊗ End V` — a matrix of power series, obtained canonically from `dualTensorHomEquiv` rather than from a basis.*

```
noncomputable def jetCoeff [Module.Free ℂ V] [Module.Finite ℂ V]
    (rep : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V)) (U : G) :
    SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V
```

lemma jetCoeff_spec

[source]

The coefficient reproduces `rep U` on constant jets.

```
lemma jetCoeff_spec [Module.Free ℂ V] [Module.Finite ℂ V]
    (rep : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V)) (U : G) (v : V) :
    TensorProduct.lift ((LinearMap.llcomp ℂ V V (SpaceTimeAlgebra ⊗[ℂ] V)).comp
      (TensorProduct.mk ℂ SpaceTimeAlgebra V)) (jetCoeff rep U) v = rep U (jetOfConstant v)
```

lemma lift_lTensor_llcomp

[source]

Postcomposing a coefficient with `f` evaluates as applying `f` to the value.

```
lemma lift_lTensor_llcomp (f : V →ₗ[ℂ] W) (x : SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V) (v : V) :
    TensorProduct.lift ((LinearMap.llcomp ℂ V W (SpaceTimeAlgebra ⊗[ℂ] W)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra W))
        (LinearMap.lTensor SpaceTimeAlgebra (LinearMap.llcomp ℂ V V W f) x) v
```

```
    = LinearMap.lTensor SpaceTimeAlgebra f
      (TensorProduct.lift ((LinearMap.llcomp ℂ V V (SpaceTimeAlgebra ⊗[ℂ] V)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra V)) x v)
```

lemma lift_lTensor_lcomp [source]

Precomposing a coefficient with `f` evaluates as evaluating at the image of the argument.

```
lemma lift_lTensor_lcomp (f : V →ₗ[ℂ] W) (y : SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ W) (v : V) :
    TensorProduct.lift ((LinearMap.llcomp ℂ V W (SpaceTimeAlgebra ⊗[ℂ] W)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra W))
        (LinearMap.lTensor SpaceTimeAlgebra (LinearMap.lcomp ℂ W f) y) v
      = TensorProduct.lift ((LinearMap.llcomp ℂ W W (SpaceTimeAlgebra ⊗[ℂ] W)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra W)) y (f v)
```

lemma jetCoeff_naturality [source]

The coefficient of a fibrewise action is natural in the value space. If `f` maps the values of one field to the values of another and intertwines their jet actions, then the coefficient of the first followed by `f` is `f` followed by the coefficient of the second, as elements of `SpaceTimeAlgebra ⊗ (V →ₗ W)`. Both value spaces have to be finite-dimensional, since both coefficients have to exist; no fibrewise-linearity is used here, the coefficient being defined for any action.

```
lemma jetCoeff_naturality [Module.Free ℂ V] [Module.Finite ℂ V]
    [Module.Free ℂ W] [Module.Finite ℂ W]
    (repV : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V))
    (repW : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] W)) (f : V →ₗ[ℂ] W)
    (hf : ∀ U : G, (LinearMap.lTensor SpaceTimeAlgebra f).comp (repV U)
      = (repW U).comp (LinearMap.lTensor SpaceTimeAlgebra f)) (U : G) :
    LinearMap.lTensor SpaceTimeAlgebra (LinearMap.llcomp ℂ V V W f) (jetCoeff repV U)
      = LinearMap.lTensor SpaceTimeAlgebra (LinearMap.lcomp ℂ W f) (jetCoeff repW U)
```

2.211 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.LorentzAction

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/LorentzAction.lean on GitHub.

def lorentzConnection [source]

The connection generators after a Lorentz transformation: the gauge-only action on the degree-one element, included into the local field algebra.

```
noncomputable def lorentzConnection (Λ : SL(2,ℂ)) :
    GaugeBoson.JetComponentSpace 𝔤 →ₗ[ℝ] T.LocalFieldAlgebra
```

def lorentzAssignment [source]

The assignment of the generators defining the action of a Lorentz transformation: each matter species acts on its own component functions, the connection generators through the gauge-only action.

```
noncomputable def lorentzAssignment (Λ : SL(2,ℂ)) : T.Assignment T.LocalFieldAlgebra where
```

def repLorentzAlgHom [source]

The action of a Lorentz transformation on the local field algebra, as an algebra endomorphism: the lift of its Lorentz assignment.

```
noncomputable def repLorentzAlgHom (Λ : SL(2,ℂ)) :
    T.LocalFieldAlgebra →ₐ[ℂ] T.LocalFieldAlgebra
```

lemma repLorentzAlgHom_ιFermion [source]

```
@[simp]
lemma repLorentzAlgHom_ιFermion (Λ : SL(2,ℂ)) (i : T.FermionSpecies)
    (x : JetComponentSpace (T.fermion i)) :
    T.repLorentzAlgHom Λ (T.ιFermion i x)
      = T.ιFermion i (JetComponentSpace.repLorentzGroup (T.fermion i) Λ x)
```

lemma repLorentzAlgHom_ιBoson [source]

```
@[simp]
lemma repLorentzAlgHom_ιBoson (Λ : SL(2,ℂ)) (j : T.BosonSpecies)
    (y : JetComponentSpace (T.boson j)) :
    T.repLorentzAlgHom Λ (T.ιBoson j y)
      = T.ιBoson j (JetComponentSpace.repLorentzGroup (T.boson j) Λ y)
```

lemma repLorentzAlgHom_ιConnection [source]

```
@[simp]
lemma repLorentzAlgHom_ιConnection (Λ : SL(2,ℂ)) (v : GaugeBoson.JetComponentSpace 𝔤) :
    T.repLorentzAlgHom Λ (T.ιConnection v)
      = T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ]
          LocalGaugeFieldAlgebra.repLorentzGroup 𝔤 Λ (SymmetricAlgebra.ι ℝ _ v))
```

lemma repLorentzAlgHom_comp_includeConnection [source]

The action of a Lorentz transformation restricts to the complexified gauge-only action on the connection factor, as an equation of algebra maps out of the complexified gauge-only algebra, compared on the real generators.

```
lemma repLorentzAlgHom_comp_includeConnection (Λ : SL(2,ℂ)) :
    (T.repLorentzAlgHom Λ).comp T.includeConnection
      = T.includeConnection.comp (LocalGaugeFieldAlgebra.complexRepLorentzGroupAlgHom 𝔤 Λ)
```

lemma repLorentzAlgHom_includeConnection [source]

The action of a Lorentz transformation restricts to the complexified gauge-only action on the connection factor.

```
lemma repLorentzAlgHom_includeConnection (Λ : SL(2,ℂ)) (y : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) :
    T.repLorentzAlgHom Λ (T.includeConnection y)
      = T.includeConnection (LocalGaugeFieldAlgebra.complexRepLorentzGroup 𝔤 Λ y)
```

lemma repLorentzAlgHom_includeConnection_one_tmul [source]

On the real gauge-only algebra, included, a Lorentz transformation acts through the gauge-only action.

```
lemma repLorentzAlgHom_includeConnection_one_tmul (Λ : SL(2,ℂ)) (x : LocalGaugeFieldAlgebra 𝔤)
    :
    T.repLorentzAlgHom Λ (T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ] x))
      = T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ] LocalGaugeFieldAlgebra.repLorentzGroup 𝔤 Λ x)
```

lemma repLorentzAlgHom_one [source]

The identity transformation acts as the identity.

```
lemma repLorentzAlgHom_one : T.repLorentzAlgHom 1 = AlgHom.id ℂ T.LocalFieldAlgebra
```

lemma repLorentzAlgHom_mul [source]

The action of a product of transformations is the composite of the actions.

```
lemma repLorentzAlgHom_mul (Λ₁ Λ₂ : SL(2,ℂ)) :
    T.repLorentzAlgHom (Λ₁ * Λ₂) = (T.repLorentzAlgHom Λ₁).comp (T.repLorentzAlgHom Λ₂)
```

def repLorentzGroup [source]

The action of the Lorentz group on the local field algebra.

```
noncomputable def repLorentzGroup : Representation ℂ SL(2,ℂ) T.LocalFieldAlgebra where
```

lemma repLorentzGroup_apply [source]

```
lemma repLorentzGroup_apply (Λ : SL(2,ℂ)) (x : T.LocalFieldAlgebra) :
    T.repLorentzGroup Λ x = T.repLorentzAlgHom Λ x
```

lemma repLorentzGroup_apply_mul [source]

```
lemma repLorentzGroup_apply_mul (Λ : SL(2,ℂ)) (x y : T.LocalFieldAlgebra) :
    T.repLorentzGroup Λ (x * y) = T.repLorentzGroup Λ x * T.repLorentzGroup Λ y
```

lemma repLorentzGroup_apply_one [source]

```
lemma repLorentzGroup_apply_one (Λ : SL(2,ℂ)) :
    T.repLorentzGroup Λ (1 : T.LocalFieldAlgebra) = 1
```

lemma repLorentzGroup_ιFermion [source]

```
@[simp]
lemma repLorentzGroup_ιFermion (Λ : SL(2,ℂ)) (i : T.FermionSpecies)
    (x : JetComponentSpace (T.fermion i)) :
    T.repLorentzGroup Λ (T.ιFermion i x)
      = T.ιFermion i (JetComponentSpace.repLorentzGroup (T.fermion i) Λ x)
```

lemma repLorentzGroup_ιBoson [source]

```
@[simp]
lemma repLorentzGroup_ιBoson (Λ : SL(2,ℂ)) (j : T.BosonSpecies)
    (y : JetComponentSpace (T.boson j)) :
    T.repLorentzGroup Λ (T.ιBoson j y)
      = T.ιBoson j (JetComponentSpace.repLorentzGroup (T.boson j) Λ y)
```

lemma repLorentzGroup_ιConnection [source]

```
@[simp]
lemma repLorentzGroup_ιConnection (Λ : SL(2,ℂ)) (v : GaugeBoson.JetComponentSpace 𝔤) :
    T.repLorentzGroup Λ (T.ιConnection v)
      = T.includeConnection ((1 : ℂ) ⊗ₜ[ℝ]
          LocalGaugeFieldAlgebra.repLorentzGroup 𝔤 Λ (SymmetricAlgebra.ι ℝ _ v))
```

lemma repLorentzGroup_includeConnection [source]

The Lorentz action restricts to the complexified gauge-only action on the connection factor.

```
lemma repLorentzGroup_includeConnection (Λ : SL(2,ℂ)) (y : ℂ ⊗[ℝ] LocalGaugeFieldAlgebra 𝔤) :
    T.repLorentzGroup Λ (T.includeConnection y)
      = T.includeConnection (LocalGaugeFieldAlgebra.complexRepLorentzGroup 𝔤 Λ y)
```

lemma repLorentzGroup_ιConnection_eq [source]

The Lorentz law of the connection generators: a transformation carries a connection generator to the generator of the transformed component, with no shift. This is the generator-level form of `GaugeFieldData.repLorentzGroup_ιConnection`, with no reference to the connection factor.

```
lemma repLorentzGroup_ιConnection_eq (Λ : SL(2,ℂ)) (v : GaugeBoson.JetComponentSpace 𝔤) :
    T.repLorentzGroup Λ (T.ιConnection v)
      = T.ιConnection (GaugeBoson.JetComponentSpace.repLorentzGroup 𝔤 Λ v)
```

2.212 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.MassWeight

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/MassWeight.lean on GitHub.

def massWeightAssignment [source]

The assignment scaling each generator by `c` to its mass weight: a component function `∂_s φ_α` of a species of mass weight `w` by `c ^ (w + 2 |s|)`, a connection generator `∂_s A_μ` by `c ^ (2 + 2 |s|)`. The relations hold because the images are generators of the same kind.

```
noncomputable def massWeightAssignment (c : ℝ) : T.Assignment T.LocalFieldAlgebra where
```

def massWeightScale [source]

***The mass-weight scaling of the local field algebra**: the algebra endomorphism scaling each generator by `c` to its mass weight.*

```
noncomputable def massWeightScale (c : ℝ) : T.LocalFieldAlgebra →ₐ[ℂ] T.LocalFieldAlgebra
```

lemma massWeightScale_ιFermion [source]

```
@[simp]
lemma massWeightScale_ιFermion (c : ℝ) (i : T.FermionSpecies)
    (x : JetComponentSpace (T.fermion i)) :
    T.massWeightScale c (T.ιFermion i x)
      = T.ιFermion i (JetComponentSpace.massWeightScale (T.fermion i).massWeight (c : ℂ) x)
```

lemma massWeightScale_ιBoson [source]

```
@[simp]
lemma massWeightScale_ιBoson (c : ℝ) (j : T.BosonSpecies)
    (y : JetComponentSpace (T.boson j)) :
    T.massWeightScale c (T.ιBoson j y)
      = T.ιBoson j (JetComponentSpace.massWeightScale (T.boson j).massWeight (c : ℂ) y)
```

lemma massWeightScale_ιConnection [source]

```
@[simp]
lemma massWeightScale_ιConnection (c : ℝ) (v : GaugeBoson.JetComponentSpace 𝔤) :
    T.massWeightScale c (T.ιConnection v)
      = T.ιConnection (GaugeBoson.JetComponentSpace.massWeightScale 𝔤 c v)
```

def massWeightSubmodule [source]

***The weight-n piece** of the local field algebra: the elements scaled by `c ^ n` under every mass-weight scaling: the equaliser of the scalings and the scalars.*

```
noncomputable def massWeightSubmodule (n : ℕ) : Submodule ℂ T.LocalFieldAlgebra
```

lemma mem_massWeightSubmodule_iff [source]

```
lemma mem_massWeightSubmodule_iff {n : ℕ} {x : T.LocalFieldAlgebra} :
    x ∈ T.massWeightSubmodule n ↔ ∀ c : ℝ, T.massWeightScale c x = ((c : ℂ) ^ n) • x
```

def massWeightSubmoduleLE [source]

***The mass-weight filtration**: the join of the pieces of weight at most `w`.*

```
noncomputable def massWeightSubmoduleLE (w : ℕ) : Submodule ℂ T.LocalFieldAlgebra
```

lemma massWeightSubmodule_le_massWeightSubmoduleLE [source]

```
lemma massWeightSubmodule_le_massWeightSubmoduleLE {k w : ℕ} (h : k ≤ w) :
    T.massWeightSubmodule k ≤ T.massWeightSubmoduleLE w
```

def gaugeInvariants [source]

***The gauge invariants**: the elements fixed by every jet of gauge transformations.*

```
noncomputable def gaugeInvariants : Submodule ℂ T.LocalFieldAlgebra
```

lemma mem_gaugeInvariants_iff [source]

```
lemma mem_gaugeInvariants_iff {x : T.LocalFieldAlgebra} :
    x ∈ T.gaugeInvariants ↔ ∀ U : GJ, T.repJet U x = x
```

def lorentzInvariants [source]

***The Lorentz invariants**: the elements fixed by every Lorentz transformation.*

```
noncomputable def lorentzInvariants : Submodule ℂ T.LocalFieldAlgebra
```

lemma mem_lorentzInvariants_iff [source]

```
lemma mem_lorentzInvariants_iff {x : T.LocalFieldAlgebra} :
    x ∈ T.lorentzInvariants ↔ ∀ Λ : SL(2,ℂ), T.repLorentzGroup Λ x = x
```

def invariantsLE [source]

***The invariants of mass weight at most `w`**: the gauge and Lorentz invariants in the filtration. A Lagrangian of mass dimension at most `w / 2` is an element of it.*

```
noncomputable def invariantsLE (w : ℕ) : Submodule ℂ T.LocalFieldAlgebra
```

lemma mem_invariantsLE_iff [source]

```
lemma mem_invariantsLE_iff {w : ℕ} {x : T.LocalFieldAlgebra} :
    x ∈ T.invariantsLE w ↔ x ∈ T.massWeightSubmoduleLE w
      ∧ (∀ U : GJ, T.repJet U x = x) ∧ ∀ Λ : SL(2,ℂ), T.repLorentzGroup Λ x = x
```

lemma invariantsLE_eq_iff [source]

A description of the invariants of mass weight at most `w`, element by element: an element of the filtration fixed by both groups is exactly an element of `Q`.

```
lemma invariantsLE_eq_iff (w : ℕ) (Q : Submodule ℂ T.LocalFieldAlgebra) :
    T.invariantsLE w = Q ↔ ∀ x : T.LocalFieldAlgebra,
      (x ∈ T.massWeightSubmoduleLE w ∧ (∀ U : GJ, T.repJet U x = x)
        ∧ ∀ Λ : SL(2,ℂ), T.repLorentzGroup Λ x = x) ↔ x ∈ Q
```

lemma invariantsLE_inf_eq_iff [source]

A description of the invariants of mass weight at most `w` lying in a submodule `A`, element by element.

```
lemma invariantsLE_inf_eq_iff (w : ℕ) (A Q : Submodule ℂ T.LocalFieldAlgebra) :
    T.invariantsLE w ⊓ A = Q ↔ ∀ x : T.LocalFieldAlgebra,
      (x ∈ T.massWeightSubmoduleLE w ∧ x ∈ A ∧ (∀ U : GJ, T.repJet U x = x)
        ∧ ∀ Λ : SL(2,ℂ), T.repLorentzGroup Λ x = x) ↔ x ∈ Q
```

def bosonNormSq [source]

***The contraction φ† φ** of a bosonic species with its conjugate, through a basis of its value space: the sum over the basis of the conjugate coordinate times the coordinate. For a scalar in a unitary representation it is the mass term.*

```
noncomputable def bosonNormSq (j : T.BosonSpecies) {ι : Type} [Fintype ι] [DecidableEq ι]
    (b : Module.Basis ι ℂ (T.BosonValue j)) : T.LocalFieldAlgebra
```

lemma mem_massWeightSubmodule_apply_iff [source]

The mass-weight scaling is respected: the weight-n piece is carried onto the weight-n piece.

```
lemma mem_massWeightSubmodule_apply_iff
    (hscale : ∀ (c : ℝ) x, e (T.massWeightScale c x) = T'.massWeightScale c (e x))
    (n : ℕ) (x : T.LocalFieldAlgebra) :
    e x ∈ T'.massWeightSubmodule n ↔ x ∈ T.massWeightSubmodule n
```

lemma massWeightSubmodule_map [source]

```
lemma massWeightSubmodule_map
    (hscale : ∀ (c : ℝ) x, e (T.massWeightScale c x) = T'.massWeightScale c (e x)) (n : ℕ) :
    (T.massWeightSubmodule n).map (e : T.LocalFieldAlgebra →ₗ[ℂ] T'.LocalFieldAlgebra)
      = T'.massWeightSubmodule n
```

lemma massWeightSubmoduleLE_map [source]

```
lemma massWeightSubmoduleLE_map
    (hscale : ∀ (c : ℝ) x, e (T.massWeightScale c x) = T'.massWeightScale c (e x)) (w : ℕ) :
    (T.massWeightSubmoduleLE w).map (e : T.LocalFieldAlgebra →ₗ[ℂ] T'.LocalFieldAlgebra)
      = T'.massWeightSubmoduleLE w
```

lemma gaugeInvariants_map [source]

The gauge invariants are carried onto the gauge invariants.

```
lemma gaugeInvariants_map (hjet : ∀ (U : GJ) x, e (T.repJet U x) = T'.repJet U (e x)) :
    T.gaugeInvariants.map (e : T.LocalFieldAlgebra →ₗ[ℂ] T'.LocalFieldAlgebra)
      = T'.gaugeInvariants
```

lemma lorentzInvariants_map [source]

The Lorentz invariants are carried onto the Lorentz invariants.

```
lemma lorentzInvariants_map
    (hlor : ∀ (Λ : SL(2,ℂ)) x, e (T.repLorentzGroup Λ x) = T'.repLorentzGroup Λ (e x)) :
    T.lorentzInvariants.map (e : T.LocalFieldAlgebra →ₗ[ℂ] T'.LocalFieldAlgebra)
      = T'.lorentzInvariants
```

lemma mem_massWeightSubmoduleLE_apply_iff [source]

The filtration is respected, element by element.

```
lemma mem_massWeightSubmoduleLE_apply_iff
    (hscale : ∀ (c : ℝ) x, e (T.massWeightScale c x) = T'.massWeightScale c (e x)) (w : ℕ)
    (x : T.LocalFieldAlgebra) :
    e x ∈ T'.massWeightSubmoduleLE w ↔ x ∈ T.massWeightSubmoduleLE w
```

lemma mem_invariantsLE_apply_iff [source]

The invariants of mass weight at most `w` are respected, element by element.

```
lemma mem_invariantsLE_apply_iff (hjet : ∀ (U : GJ) x, e (T.repJet U x) = T'.repJet U (e x))
    (hlor : ∀ (Λ : SL(2,ℂ)) x, e (T.repLorentzGroup Λ x) = T'.repLorentzGroup Λ (e x))
    (hscale : ∀ (c : ℝ) x, e (T.massWeightScale c x) = T'.massWeightScale c (e x)) (w : ℕ)
    (x : T.LocalFieldAlgebra) :
    e x ∈ T'.invariantsLE w ↔ x ∈ T.invariantsLE w
```

lemma invariantsLE_map [source]

***The invariants of mass weight at most `w` are carried onto the invariants of mass weight at most `w`** by an isomorphism respecting the two actions and the scaling.*

```
lemma invariantsLE_map (hjet : ∀ (U : GJ) x, e (T.repJet U x) = T'.repJet U (e x))
    (hlor : ∀ (Λ : SL(2,ℂ)) x, e (T.repLorentzGroup Λ x) = T'.repLorentzGroup Λ (e x))
    (hscale : ∀ (c : ℝ) x, e (T.massWeightScale c x) = T'.massWeightScale c (e x)) (w : ℕ) :
    (T.invariantsLE w).map (e : T.LocalFieldAlgebra →ₗ[ℂ] T'.LocalFieldAlgebra)
      = T'.invariantsLE w
```

2.213 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.Realization

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/Realization.lean on GitHub.

abbrev Realization [source]

A complex algebra `B` carrying the fields of the datum `T`: a complex algebra map out of the local field algebra, equivariant for the jet gauge group and the Lorentz group, both acting on the whole of `B` by algebra endomorphisms. No commutativity, injectivity or surjectivity is assumed, and no derivative operator on `B` is involved. It is built from the fields `toAlgHom`, `map_fst`, `map_snd`, `fst_mul`, `snd_mul` of `Representation.EquivariantAlgHom`, which the lemmas `map_repJet`, `map_repLorentz`, `repJet_mul` and `repLorentz_mul` name.

```
abbrev Realization (T : GaugeFieldData jets) (B : Type) [Ring B] [Algebra ℂ B]
    (repJet : Representation ℂ GJ B) (repLorentz : Representation ℂ SL(2,ℂ) B)
```

def id [source]

The local field algebra realized in itself, by the identity.

```
noncomputable def id : Realization T T.LocalFieldAlgebra T.repJet T.repLorentzGroup
```

lemma id_toAlgHom [source]

```
@[simp]
lemma id_toAlgHom : (id T).toAlgHom = AlgHom.id ℂ T.LocalFieldAlgebra
```

lemma ext_generators [source]

Two realizations agreeing on the generators of every species and on the connection generators are equal.

```
lemma ext_generators {h₁ h₂ : Realization T B repJet repLorentz}
    (hf : ∀ i x, h₁.toAlgHom (T.ιFermion i x) = h₂.toAlgHom (T.ιFermion i x))
    (hb : ∀ j y, h₁.toAlgHom (T.ιBoson j y) = h₂.toAlgHom (T.ιBoson j y))
    (ha : ∀ v, h₁.toAlgHom (T.ιConnection v) = h₂.toAlgHom (T.ιConnection v)) : h₁ = h₂
```

lemma map_repJet [source]

The map is equivariant for the jet gauge group.

```
lemma map_repJet (U : GJ) (x : T.LocalFieldAlgebra) :
    h.toAlgHom (T.repJet U x) = repJet U (h.toAlgHom x)
```

lemma map_repLorentz [source]

The map is equivariant for the Lorentz group.

```
lemma map_repLorentz (Λ : SL(2,ℂ)) (x : T.LocalFieldAlgebra) :
    h.toAlgHom (T.repLorentzGroup Λ x) = repLorentz Λ (h.toAlgHom x)
```

lemma repJet_mul [source]

The jet gauge group acts on the whole of `B` by algebra endomorphisms.

```
lemma repJet_mul (U : GJ) (b₁ b₂ : B) : repJet U (b₁ * b₂) = repJet U b₁ * repJet U b₂
```

lemma repLorentz_mul [source]

The Lorentz group acts on the whole of `B` by algebra endomorphisms.

```
lemma repLorentz_mul (Λ : SL(2,ℂ)) (b₁ b₂ : B) :
    repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂
```

def toAssignment [source]

The generator images of a realization, as a compatible assignment: the inverse of the universal property applied to its algebra map.

```
noncomputable def toAssignment : T.Assignment B
```

lemma toAssignment_fermion [source]

```
lemma toAssignment_fermion (i : T.FermionSpecies) (x : JetComponentSpace (T.fermion i)) :
    h.toAssignment.fermion i x = h.toAlgHom (T.ιFermion i x)
```

lemma toAssignment_boson [source]

```
lemma toAssignment_boson (j : T.BosonSpecies) (y : JetComponentSpace (T.boson j)) :
    h.toAssignment.boson j y = h.toAlgHom (T.ιBoson j y)
```

lemma toAssignment_connection [source]

```
lemma toAssignment_connection (v : GaugeBoson.JetComponentSpace 𝔤) :
    h.toAssignment.connection v = h.toAlgHom (T.ιConnection v)
```

lemma lift_toAssignment [source]

```
@[simp]
lemma lift_toAssignment : h.toAssignment.lift = h.toAlgHom
```

def gaugeRealization [source]

The gauge bosons of a realization: the connection factor of `J(T)` carried into `B`, so that the gauge-boson symbol theory applies to the images.

```
noncomputable def gaugeRealization : GaugeAlgebraRealization jets B repJet repLorentz where
```

lemma gaugeRealization_A [source]

```
lemma gaugeRealization_A (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    h.gaugeRealization.A s μ φ = h.toAlgHom (T.gaugeRealization.A s μ φ)
```

lemma id_gaugeRealization_A [source]

```
@[simp]
lemma id_gaugeRealization_A : (id T).gaugeRealization.A = T.gaugeRealization.A
```

lemma toAlgHom_covDerivFieldStrength [source]

The map of a realization carries the included field-strength tower to the covariant tower of the realized gauge-boson symbols.

```
lemma toAlgHom_covDerivFieldStrength (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    h.toAlgHom (T.covDerivFieldStrength l μ ν φ)
      = GaugeAlgebraRealization.iteratedCovDerivAdjoint h.gaugeRealization.A l
        (GaugeAlgebraRealization.fieldStrength h.gaugeRealization.A μ ν) 0 φ
```

def fermionSymbol [source]

The realized derivative symbols of a fermionic species.

```
noncomputable def fermionSymbol (i : T.FermionSpecies) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (T.FermionValue i) →ₗ[ℂ] B
```

def conjFermionSymbol [source]

The realized conjugate derivative symbols of a fermionic species.

```
noncomputable def conjFermionSymbol (i : T.FermionSpecies) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule (T.FermionValue i)) →ₗ[ℂ] B
```

def bosonSymbol [source]

The realized derivative symbols of a bosonic species.

```
noncomputable def bosonSymbol (j : T.BosonSpecies) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (T.BosonValue j) →ₗ[ℂ] B
```

def conjBosonSymbol [source]

The realized conjugate derivative symbols of a bosonic species.

```
noncomputable def conjBosonSymbol (j : T.BosonSpecies) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule (T.BosonValue j)) →ₗ[ℂ] B
```

lemma toAlgHom_covDerivFermion [source]

The realized covariant tower of a fermionic species is the covariant tower of its realized symbols, computed against the realized gauge-boson symbols. No derivative operator on `B` is involved: both sides are the same finite algebraic expression.

```
lemma toAlgHom_covDerivFermion (i : T.FermionSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i)) :
    h.toAlgHom (T.covDerivFermion i l φ)
      = GaugeAlgebraRealization.covDerivIter h.gaugeRealization.A (T.fermion i).repAlgebra
        (h.fermionSymbol i) n l 0 φ
```

lemma toAlgHom_covDerivConjFermion [source]

```
lemma toAlgHom_covDerivConjFermion (i : T.FermionSpecies) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    h.toAlgHom (T.covDerivConjFermion i l φ)
      = GaugeAlgebraRealization.covDerivIter h.gaugeRealization.A
        (LocalGaugeData.actionConj (T.fermion i).repAlgebra) (h.conjFermionSymbol i) n l 0 φ
```

lemma toAlgHom_covDerivBoson [source]

```
lemma toAlgHom_covDerivBoson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)) :
    h.toAlgHom (T.covDerivBoson j l φ)
      = GaugeAlgebraRealization.covDerivIter h.gaugeRealization.A (T.boson j).repAlgebra
        (h.bosonSymbol j) n l 0 φ
```

lemma toAlgHom_covDerivConjBoson [source]

```
lemma toAlgHom_covDerivConjBoson (j : T.BosonSpecies) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    h.toAlgHom (T.covDerivConjBoson j l φ)
      = GaugeAlgebraRealization.covDerivIter h.gaugeRealization.A
        (LocalGaugeData.actionConj (T.boson j).repAlgebra) (h.conjBosonSymbol j) n l 0 φ
```

structure IsEquivariant [source]

The generator-level transformation laws of an assignment: each matter species transforms by the jet action of its own component space, and the connection generators

transform affinely, by the transport of the inverse jet plus its Maurer–Cartan shift. The Lorentz laws are all linear. These are the laws the generators of `J(T)` themselves satisfy.

```
structure Assignment.IsEquivariant {B : Type} [Ring B] [Algebra ℂ B] (d : T.Assignment B)
    (repJet : Representation ℂ GJ B) (repLorentz : Representation ℂ SL(2,ℂ) B) : Prop where
  repJet_fermion : ∀ (U : GJ) (i : T.FermionSpecies) (x : JetComponentSpace (T.fermion i)),
    repJet U (d.fermion i x) = d.fermion i (JetComponentSpace.repJet (T.fermion i) U x)
  repJet_boson : ∀ (U : GJ) (j : T.BosonSpecies) (y : JetComponentSpace (T.boson j)),
    repJet U (d.boson j y) = d.boson j (JetComponentSpace.repJet (T.boson j) U y)
  repJet_connection : ∀ (U : GJ) (v : GaugeBoson.JetComponentSpace 𝔤),
    repJet U (d.connection v) = d.connection (LocalGaugeFieldAlgebra.transport jets U⁻¹ v)
      + (LocalGaugeFieldAlgebra.mcShift jets U⁻¹ v : ℂ) • (1 : B)
  repLorentz_fermion : ∀ (Λ : SL(2,ℂ)) (i : T.FermionSpecies)
    (x : JetComponentSpace (T.fermion i)), repLorentz Λ (d.fermion i x)
      = d.fermion i (JetComponentSpace.repLorentzGroup (T.fermion i) Λ x)
  repLorentz_boson : ∀ (Λ : SL(2,ℂ)) (j : T.BosonSpecies)
    (y : JetComponentSpace (T.boson j)), repLorentz Λ (d.boson j y)
      = d.boson j (JetComponentSpace.repLorentzGroup (T.boson j) Λ y)
  repLorentz_connection : ∀ (Λ : SL(2,ℂ)) (v : GaugeBoson.JetComponentSpace 𝔤),
    repLorentz Λ (d.connection v)
      = d.connection (GaugeBoson.JetComponentSpace.repLorentzGroup 𝔤 Λ v)
```

lemma toAssignment_isEquivariant [source]

The assignment of a realization is equivariant: the laws are those of the generators of `J(T)`, transported along the map.

```
lemma _root_.GaugeFieldData.Realization.toAssignment_isEquivariant
    (h : Realization T B repJet repLorentz) :
    h.toAssignment.IsEquivariant repJet repLorentz where
```

lemma lift_comp_repJetAlgHom [source]

The lift of an equivariant assignment intertwines the jet gauge actions, as an equality of algebra maps: both sides agree on the generators of every species and on the connection generators.

```
lemma IsEquivariant.lift_comp_repJetAlgHom {d : T.Assignment B}
    (hd : d.IsEquivariant repJet repLorentz)
    (hJ : ∀ (U : GJ) (b₁ b₂ : B), repJet U (b₁ * b₂) = repJet U b₁ * repJet U b₂) (U : GJ) :
    d.lift.comp (T.repJetAlgHom U) = (repJet.toAlgHom hJ U).comp d.lift
```

lemma lift_comp_repLorentzAlgHom [source]

The lift of an equivariant assignment intertwines the Lorentz actions, as an equality of algebra maps.

```
lemma IsEquivariant.lift_comp_repLorentzAlgHom {d : T.Assignment B}
    (hd : d.IsEquivariant repJet repLorentz)
    (hL : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
      repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂) (Λ : SL(2,ℂ)) :
    d.lift.comp (T.repLorentzAlgHom Λ) = (repLorentz.toAlgHom hL Λ).comp d.lift
```

def toRealization [source]

An equivariant assignment lifts to a realization: each equivariance law is an equality of two algebra maps agreeing on the generators, so it follows from the generator laws by the universal property.

```
noncomputable def IsEquivariant.toRealization {d : T.Assignment B}
    (hd : d.IsEquivariant repJet repLorentz)
    (hJ : ∀ (U : GJ) (b₁ b₂ : B), repJet U (b₁ * b₂) = repJet U b₁ * repJet U b₂)
    (hL : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
      repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂) :
    Realization T B repJet repLorentz where
```

lemma toRealization_toAlgHom [source]

```
@[simp]
lemma IsEquivariant.toRealization_toAlgHom {d : T.Assignment B}
    (hd : d.IsEquivariant repJet repLorentz)
    (hJ : ∀ (U : GJ) (b₁ b₂ : B), repJet U (b₁ * b₂) = repJet U b₁ * repJet U b₂)
    (hL : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
      repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂) :
    (hd.toRealization hJ hL).toAlgHom = d.lift
```

def realizationEquivAssignment [source]

The mapping-out universal property of `J(T)` in equivariant form: for target actions by algebra endomorphisms, realizations of `T` in `B` are exactly the equivariant compatible assignments of its generators.

```
noncomputable def realizationEquivAssignment {B : Type} [Ring B] [Algebra ℂ B]
    {repJet : Representation ℂ GJ B} {repLorentz : Representation ℂ SL(2,ℂ) B}
    (hJ : ∀ (U : GJ) (b₁ b₂ : B), repJet U (b₁ * b₂) = repJet U b₁ * repJet U b₂)
    (hL : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
      repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂) :
    {d : T.Assignment B // d.IsEquivariant repJet repLorentz}
      ≃ Realization T B repJet repLorentz where
```

2.214 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.Sector

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/Sector.lean on GitHub.

inductive FieldCategory [source]

The categories of fields a sector may select.

```
inductive FieldCategory where
  | fermion : FieldCategory
  | gauge : FieldCategory
  | scalar : FieldCategory
```

instance Fintype FieldCategory [source]

The three categories exhaust `FieldCategory`.

```
instance : Fintype FieldCategory where
```

def categoryGenerators [source]

The ordinary generators of one field category.

```
def categoryGenerators : FieldCategory → Set T.LocalFieldAlgebra
  | .fermion => Set.range T.ιFermionTotal
  | .gauge => Set.range T.ιConnection
  | .scalar => Set.range T.ιBosonTotal
```

def sectorGenerators [source]

The ordinary generators selected by a sector.

```
def sectorGenerators (S : Finset FieldCategory) : Set T.LocalFieldAlgebra
```

lemma sectorGenerators_mono [source]

```
lemma sectorGenerators_mono {S S' : Finset FieldCategory} (h : S ⊆ S') :
    T.sectorGenerators S ⊆ T.sectorGenerators S'
```

lemma sectorGenerators_empty [source]

```
@[simp]
lemma sectorGenerators_empty : T.sectorGenerators ∅ = ∅
```

lemma sectorGenerators_union [source]

```
lemma sectorGenerators_union (S S' : Finset FieldCategory) :
    T.sectorGenerators (S ∪ S') = T.sectorGenerators S ∪ T.sectorGenerators S'
```

lemma sectorGenerators_univ [source]

The full sector selects the generators of the local field algebra.

```
lemma sectorGenerators_univ : T.sectorGenerators Finset.univ = T.generators
```

def SectorAlgebra [source]

The ordinary sector algebra of a sector `S`: the subalgebra of the local field algebra generated by the ordinary generators of the selected categories.

```
noncomputable def SectorAlgebra (S : Finset FieldCategory) : Subalgebra ℂ T.LocalFieldAlgebra
```

lemma ιFermionTotal_mem [source]

```
lemma ιFermionTotal_mem (hS : FieldCategory.fermion ∈ S) (v : T.FermionGenerators) :
    T.ιFermionTotal v ∈ T.SectorAlgebra S
```

lemma ιFermion_mem [source]

```
lemma ιFermion_mem (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies)
    (x : JetComponentSpace (T.fermion i)) : T.ιFermion i x ∈ T.SectorAlgebra S
```

lemma fermionSymbol_mem [source]

```
lemma fermionSymbol_mem (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.FermionValue i)) :
    T.fermionSymbol i s φ ∈ T.SectorAlgebra S
```

lemma conjFermionSymbol_mem [source]

```
lemma conjFermionSymbol_mem (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.conjFermionSymbol i s φ ∈ T.SectorAlgebra S
```

lemma ιBosonTotal_mem [source]

```
lemma ιBosonTotal_mem (hS : FieldCategory.scalar ∈ S) (v : T.BosonGenerators) :
    T.ιBosonTotal v ∈ T.SectorAlgebra S
```

lemma ιBoson_mem [source]

```
lemma ιBoson_mem (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies)
    (y : JetComponentSpace (T.boson j)) : T.ιBoson j y ∈ T.SectorAlgebra S
```

lemma bosonSymbol_mem [source]

```
lemma bosonSymbol_mem (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.BosonValue j)) :
    T.bosonSymbol j s φ ∈ T.SectorAlgebra S
```

lemma conjBosonSymbol_mem [source]

```
lemma conjBosonSymbol_mem (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    T.conjBosonSymbol j s φ ∈ T.SectorAlgebra S
```

lemma ιConnection_mem [source]

```
lemma ιConnection_mem (hS : FieldCategory.gauge ∈ S) (v : GaugeBoson.JetComponentSpace g) :
    T.ιConnection v ∈ T.SectorAlgebra S
```

lemma sectorGenerators_cases [source]

Case analysis on the generators of a sector.

```
lemma sectorGenerators_cases {P : T.LocalFieldAlgebra → Prop} {b : T.LocalFieldAlgebra}
    (hb : b ∈ T.sectorGenerators S)
    (hf : FieldCategory.fermion ∈ S → ∀ v, P (T.ιFermionTotal v))
    (hs : FieldCategory.scalar ∈ S → ∀ v, P (T.ιBosonTotal v))
    (hg : FieldCategory.gauge ∈ S → ∀ v, P (T.ιConnection v)) : P b
```

lemma induction [source]

A property holding on the selected generators and the scalars and closed under sums and products holds on the sector algebra.

```
lemma induction {P : T.LocalFieldAlgebra → Prop} {x : T.LocalFieldAlgebra}
    (hx : x ∈ T.SectorAlgebra S)
    (hf : FieldCategory.fermion ∈ S → ∀ v, P (T.ιFermionTotal v))
    (hs : FieldCategory.scalar ∈ S → ∀ v, P (T.ιBosonTotal v))
    (hg : FieldCategory.gauge ∈ S → ∀ v, P (T.ιConnection v))
    (halg : ∀ z : ℂ, P (z • (1 : T.LocalFieldAlgebra)))
    (hadd : ∀ x y, P x → P y → P (x + y))
    (hmul : ∀ x y, P x → P y → P (x * y)) : P x
```

lemma algHom_ext [source]

Two algebra maps out of a sector algebra agreeing on the selected generators are equal. This is uniqueness only: a sector algebra is not free on its generators.

```
lemma algHom_ext {B : Type} [Semiring B] [Algebra ℂ B]
    {f g : 𝓐(T.SectorAlgebra S) →ₐ[ℂ] B}
    (hf : ∀ (hS : FieldCategory.fermion ∈ S) (v : T.FermionGenerators),
      f ⟨T.ιFermionTotal v, ιFermionTotal_mem hS v⟩ = g ⟨T.ιFermionTotal v, ιFermionTotal_mem
    hS v⟩)
    (hs : ∀ (hS : FieldCategory.scalar ∈ S) (v : T.BosonGenerators),
      f ⟨T.ιBosonTotal v, ιBosonTotal_mem hS v⟩ = g ⟨T.ιBosonTotal v, ιBosonTotal_mem hS v⟩)
    (hg : ∀ (hS : FieldCategory.gauge ∈ S) (v : GaugeBoson.JetComponentSpace 𝔤),
      f ⟨T.ιConnection v, ιConnection_mem hS v⟩ = g ⟨T.ιConnection v, ιConnection_mem hS v⟩) :
    f = g
```

lemma mono [source]

```
lemma mono {S S' : Finset FieldCategory} (h : S ⊆ S') :
    T.SectorAlgebra S ≤ T.SectorAlgebra S'
```

lemma empty [source]

The empty sector algebra is the image of the scalars.

```
@[simp]
lemma empty : T.SectorAlgebra ∅ = ⊥
```

lemma univ [source]

```
@[simp]
lemma univ : T.SectorAlgebra Finset.univ = ⊤
```

lemma union [source]

```
lemma union (S S' : Finset FieldCategory) :
    T.SectorAlgebra (S ∪ S') = T.SectorAlgebra S ⊔ T.SectorAlgebra S'
```

lemma repJet_mem [source]

```
lemma repJet_mem (U : GJ) {x : T.LocalFieldAlgebra} (hx : x ∈ T.SectorAlgebra S) :
    T.repJet U x ∈ T.SectorAlgebra S
```

lemma repLorentzGroup_mem [source]

```
lemma repLorentzGroup_mem (Λ : SL(2,ℂ)) {x : T.LocalFieldAlgebra} (hx : x ∈ T.SectorAlgebra S)
    :
    T.repLorentzGroup Λ x ∈ T.SectorAlgebra S
```

def repJet [source]

The jet gauge group acting on a sector algebra, by restriction.

```
noncomputable def repJet : Representation ℂ GJ (T.SectorAlgebra S)
```

def repLorentzGroup [source]

The Lorentz group acting on a sector algebra, by restriction.

```
noncomputable def repLorentzGroup : Representation ℂ SL(2,ℂ) (T.SectorAlgebra S)
```

lemma coe_repJet [source]

```
@[simp]
lemma coe_repJet (U : GJ) (x : T.SectorAlgebra S) :
    (repJet T S U x : T.LocalFieldAlgebra) = T.repJet U x
```

lemma repJet_apply_mul [source]

```
lemma repJet_apply_mul (U : GJ) (x y : T.SectorAlgebra S) :
    repJet T S U (x * y) = repJet T S U x * repJet T S U y
```

lemma coe_repLorentzGroup [source]

```
@[simp]
lemma coe_repLorentzGroup (Λ : SL(2,ℂ)) (x : T.SectorAlgebra S) :
    (repLorentzGroup T S Λ x : T.LocalFieldAlgebra) = T.repLorentzGroup Λ x
```

lemma repLorentzGroup_apply_mul [source]

```
lemma repLorentzGroup_apply_mul (Λ : SL(2,ℂ)) (x y : T.SectorAlgebra S) :
    repLorentzGroup T S Λ (x * y) = repLorentzGroup T S Λ x * repLorentzGroup T S Λ y
```

lemma inclusion_repJet [source]

```
lemma inclusion_repJet {S' : Finset FieldCategory} (h : S ⊆ S') (U : GJ) (x : T.SectorAlgebra S
    ) :
    Subalgebra.inclusion (mono T h) (repJet T S U x)
      = repJet T S' U (Subalgebra.inclusion (mono T h) x)
```

lemma inclusion_repLorentzGroup [source]

```
lemma inclusion_repLorentzGroup {S' : Finset FieldCategory} (h : S ⊆ S') (Λ : SL(2,ℂ))
    (x : T.SectorAlgebra S) :
    Subalgebra.inclusion (mono T h) (repLorentzGroup T S Λ x)
      = repLorentzGroup T S' Λ (Subalgebra.inclusion (mono T h) x)
```

2.215 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.SectorRealization

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/SectorRealization.lean on GitHub.

abbrev Realization [source]

A complex algebra `B` carrying the ordinary sector `S` of the datum `T`: a complex algebra map out of the sector algebra, equivariant for the jet gauge group and the Lorentz group, both acting on the whole of `B` by algebra endomorphisms. The lemmas `map_repJet`, `map_repLorentz`, `repJet_mul` and `repLorentz_mul` name its fields.

```
abbrev Realization (T : GaugeFieldData jets) (S : Finset FieldCategory) (B : Type)
    [Semiring B] [Algebra ℂ B] (repJet : Representation ℂ GJ B)
    (repLorentz : Representation ℂ SL(2,ℂ) B)
```

def id [source]

A sector algebra realized in itself, by the identity.

```
noncomputable def id :
    Realization T S (T.SectorAlgebra S) (SectorAlgebra.repJet T S)
      (SectorAlgebra.repLorentzGroup T S)
```

lemma id_toAlgHom [source]

```
@[simp]
lemma id_toAlgHom : (id T S).toAlgHom = AlgHom.id ℂ (T.SectorAlgebra S)
```

lemma map_repJet [source]

```
lemma map_repJet (U : GJ) (x : T.SectorAlgebra S) :
    h.toAlgHom (SectorAlgebra.repJet T S U x) = repJet U (h.toAlgHom x)
```

lemma map_repLorentz [source]

```
lemma map_repLorentz (Λ : SL(2,ℂ)) (x : T.SectorAlgebra S) :
    h.toAlgHom (SectorAlgebra.repLorentzGroup T S Λ x) = repLorentz Λ (h.toAlgHom x)
```

lemma repJet_mul [source]

```
lemma repJet_mul (U : GJ) (b₁ b₂ : B) : repJet U (b₁ * b₂) = repJet U b₁ * repJet U b₂
```

lemma repLorentz_mul [source]

```
lemma repLorentz_mul (Λ : SL(2,ℂ)) (b₁ b₂ : B) :
    repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂
```

lemma ext_generators [source]

A sector realization is determined by its images of the selected generators. Only uniqueness is asserted, not the extension of an arbitrary assignment.

```
lemma ext_generators {h₁ h₂ : Realization T S B repJet repLorentz}
    (hf : ∀ (hS : FieldCategory.fermion ∈ S) (v : T.FermionGenerators),
      h₁.toAlgHom ⟨T.ιFermionTotal v, ιFermionTotal_mem hS v⟩
        = h₂.toAlgHom ⟨T.ιFermionTotal v, ιFermionTotal_mem hS v⟩)
    (hs : ∀ (hS : FieldCategory.scalar ∈ S) (v : T.BosonGenerators),
      h₁.toAlgHom ⟨T.ιBosonTotal v, ιBosonTotal_mem hS v⟩
        = h₂.toAlgHom ⟨T.ιBosonTotal v, ιBosonTotal_mem hS v⟩)
    (hg : ∀ (hS : FieldCategory.gauge ∈ S) (v : GaugeBoson.JetComponentSpace 𝔤),
      h₁.toAlgHom ⟨T.ιConnection v, ιConnection_mem hS v⟩
        = h₂.toAlgHom ⟨T.ιConnection v, ιConnection_mem hS v⟩) : h₁ = h₂
```

def restrict [source]

The restriction of a sector realization to a smaller sector.

```
noncomputable def restrict (h : Realization T S' B repJet repLorentz) (hS : S ⊆ S') :
    Realization T S B repJet repLorentz
```

lemma restrict_toAlgHom [source]

```
lemma restrict_toAlgHom (h : Realization T S' B repJet repLorentz) (hS : S ⊆ S') :
    (h.restrict hS).toAlgHom = h.toAlgHom.comp (Subalgebra.inclusion (mono T hS))
```

lemma restrict_toAlgHom_apply [source]

```
@[simp]
lemma restrict_toAlgHom_apply (h : Realization T S' B repJet repLorentz) (hS : S ⊆ S')
    (x : T.SectorAlgebra S) :
    (h.restrict hS).toAlgHom x = h.toAlgHom (Subalgebra.inclusion (mono T hS) x)
```

lemma restrict_toAlgHom_mk [source]

```
lemma restrict_toAlgHom_mk (h : Realization T S' B repJet repLorentz) (hS : S ⊆ S')
    (x : T.LocalFieldAlgebra) (hx : x ∈ T.SectorAlgebra S) :
    (h.restrict hS).toAlgHom ⟨x, hx⟩ = h.toAlgHom ⟨x, mono T hS hx⟩
```

lemma restrict_id_toAlgHom [source]

```
lemma restrict_id_toAlgHom (hS : S ⊆ S') :
    ((id T S').restrict hS).toAlgHom = Subalgebra.inclusion (mono T hS)
```

lemma restrict_restrict [source]

```
lemma restrict_restrict {S'' : Finset FieldCategory} (h : Realization T S'' B repJet repLorentz
    )
    (hS' : S' ⊆ S'') (hS : S ⊆ S') :
    (h.restrict hS').restrict hS = h.restrict (hS.trans hS')
```

def restrictSector [source]

The restriction of a realization of the local field algebra to a sector.

```
noncomputable def restrictSector : SectorAlgebra.Realization T S B repJet repLorentz
```

lemma restrictSector_toAlgHom [source]

```
lemma restrictSector_toAlgHom :
    (h.restrictSector S).toAlgHom = h.toAlgHom.comp (T.SectorAlgebra S).val
```

lemma restrictSector_toAlgHom_apply [source]

```
@[simp]
lemma restrictSector_toAlgHom_apply (x : T.SectorAlgebra S) :
    (h.restrictSector S).toAlgHom x = h.toAlgHom x
```

lemma restrictSector_id_toAlgHom [source]

```
lemma restrictSector_id_toAlgHom :
    ((id T).restrictSector S).toAlgHom = (T.SectorAlgebra S).val
```

lemma restrictSector_fermionSymbol [source]

```
lemma restrictSector_fermionSymbol (hS : FieldCategory.fermion ∈ S) (i : T.FermionSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.FermionValue i)) :
    (h.restrictSector S).toAlgHom ⟨T.fermionSymbol i s φ, SectorAlgebra.fermionSymbol_mem hS i
    s φ⟩
      = h.fermionSymbol i s φ
```

lemma restrictSector_conjFermionSymbol [source]

```
lemma restrictSector_conjFermionSymbol (hS : FieldCategory.fermion ∈ S)
    (i : T.FermionSpecies) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    (h.restrictSector S).toAlgHom
      ⟨T.conjFermionSymbol i s φ, SectorAlgebra.conjFermionSymbol_mem hS i s φ⟩
      = h.conjFermionSymbol i s φ
```

lemma restrictSector_bosonSymbol [source]

```
lemma restrictSector_bosonSymbol (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (T.BosonValue j)) :
    (h.restrictSector S).toAlgHom ⟨T.bosonSymbol j s φ, SectorAlgebra.bosonSymbol_mem hS j s φ⟩
      = h.bosonSymbol j s φ
```

lemma restrictSector_conjBosonSymbol [source]

```
lemma restrictSector_conjBosonSymbol (hS : FieldCategory.scalar ∈ S) (j : T.BosonSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    (h.restrictSector S).toAlgHom
      ⟨T.conjBosonSymbol j s φ, SectorAlgebra.conjBosonSymbol_mem hS j s φ⟩
      = h.conjBosonSymbol j s φ
```

lemma restrict_restrictSector [source]

```
lemma restrict_restrictSector {S' : Finset FieldCategory} (hS : S ⊆ S') :
    (h.restrictSector S').restrict hS = h.restrictSector S
```

2.216 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalField.TransformsIn

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalField/TransformsIn.lean on GitHub.

def gaugeRealization [source]

The connection factor as a realization of the gauge bosons in `J(T)`: the inclusion of the complexified gauge-only algebra, equivariant for both actions, with the included gauge-field symbols as its symbols.

```
noncomputable def gaugeRealization :
    GaugeAlgebraRealization jets T.LocalFieldAlgebra T.repJet T.repLorentzGroup where
```

lemma gaugeRealization_toAlgHom [source]

```
lemma gaugeRealization_toAlgHom : T.gaugeRealization.toAlgHom = T.includeConnection
```

lemma gaugeRealization_A [source]

```
lemma gaugeRealization_A (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ 𝔤) :
    T.gaugeRealization.A s μ φ
      = T.includeConnection (LocalGaugeFieldAlgebra.gaugeField 𝔤 s μ φ)
```

def fermionSymbolMap [source]

The unconjugated symbols of a fermionic species, as a linear map on the unconjugated half of its component space.

```
noncomputable def fermionSymbolMap (i : T.FermionSpecies) :
    SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (T.FermionValue i) →ₗ[ℂ] T.LocalFieldAlgebra
```

def conjFermionSymbolMap [source]

The conjugate symbols of a fermionic species, as a linear map on the conjugate half of its component space.

```
noncomputable def conjFermionSymbolMap (i : T.FermionSpecies) :
    SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule (T.FermionValue i)) →ₗ[ℂ]
      T.LocalFieldAlgebra
```

def bosonSymbolMap [source]

The unconjugated symbols of a bosonic species, as a linear map on the unconjugated half of its component space.

```
noncomputable def bosonSymbolMap (j : T.BosonSpecies) :
    SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (T.BosonValue j) →ₗ[ℂ] T.LocalFieldAlgebra
```

def conjBosonSymbolMap [source]

The conjugate symbols of a bosonic species, as a linear map on the conjugate half of its component space.

```
noncomputable def conjBosonSymbolMap (j : T.BosonSpecies) :
    SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule (T.BosonValue j)) →ₗ[ℂ]
      T.LocalFieldAlgebra
```

def fermionSymbol [source]

The derivative symbols `∂_s ψ^φ` of a fermionic species: the family over the derivative multiset `s`, indexed by the covectors of the value space.

```
noncomputable def fermionSymbol (i : T.FermionSpecies) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (T.FermionValue i) →ₗ[ℂ] T.LocalFieldAlgebra
```

def conjFermionSymbol [source]

The conjugate derivative symbols `∂_s ψ̄^φ` of a fermionic species, indexed by the covectors of the conjugate value space.

```
noncomputable def conjFermionSymbol (i : T.FermionSpecies) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule (T.FermionValue i)) →ₗ[ℂ] T.LocalFieldAlgebra
```

def bosonSymbol [source]

The derivative symbols `∂_s φ^χ` of a bosonic species.

```
noncomputable def bosonSymbol (j : T.BosonSpecies) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (T.BosonValue j) →ₗ[ℂ] T.LocalFieldAlgebra
```

def conjBosonSymbol [source]

The conjugate derivative symbols `∂_s φ̄^χ` of a bosonic species.

```
noncomputable def conjBosonSymbol (j : T.BosonSpecies) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule (T.BosonValue j)) →ₗ[ℂ] T.LocalFieldAlgebra
```

lemma fermionSymbol_apply [source]

```
lemma fermionSymbol_apply (i : T.FermionSpecies) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.FermionValue i)) :
    T.fermionSymbol i s φ = T.ιFermion i (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0)
```

lemma conjFermionSymbol_apply [source]

```
lemma conjFermionSymbol_apply (i : T.FermionSpecies) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.conjFermionSymbol i s φ = T.ιFermion i (0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ)
```

lemma bosonSymbol_apply [source]

```
lemma bosonSymbol_apply (j : T.BosonSpecies) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (T.BosonValue j)) :
    T.bosonSymbol j s φ = T.ιBoson j (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0)
```

lemma conjBosonSymbol_apply [source]

```
lemma conjBosonSymbol_apply (j : T.BosonSpecies) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    T.conjBosonSymbol j s φ = T.ιBoson j (0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ)
```

lemma transformsIn_of_repDual [source]

The transformation law of a family of symbols built from a linear map intertwining the jet gauge action with `JetComponentSpace.repDual`.

```
private lemma transformsIn_of_repDual (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (hlin : ∀ (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V), rep U
        (χ • z) = χ • rep U z)
    (Φ : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ V →ₗ[ℂ] T.LocalFieldAlgebra)
    (hΦ : ∀ (U : GJ) (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ V),
      T.repJet U (Φ x) = Φ (JetComponentSpace.repDual rep hlin U x)) :
    LocalGaugeData.TransformsIn (B := T.LocalFieldAlgebra) T.repJet rep
      (fun s => Φ ∘ₗ TensorProduct.mk ℂ _ _ (SpaceTimeDerivAlgebraℂ.basis s))
```

lemma repJet_fermionSymbolMap [source]

The unconjugated fermionic symbol map intertwines the jet gauge actions.

```
lemma repJet_fermionSymbolMap (i : T.FermionSpecies) (U : GJ)
    (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (T.FermionValue i)) :
    T.repJet U (T.fermionSymbolMap i x)
      = T.fermionSymbolMap i
        (JetComponentSpace.repDual (T.fermion i).repJet (T.fermion i).repJet_smul U x)
```

lemma repJet_conjFermionSymbolMap [source]

The conjugate fermionic symbol map intertwines the jet gauge actions.

```
lemma repJet_conjFermionSymbolMap (i : T.FermionSpecies) (U : GJ)
    (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.repJet U (T.conjFermionSymbolMap i x)
      = T.conjFermionSymbolMap i
        (JetComponentSpace.repDual (JetComponentSpace.repConj (T.fermion i).repJet)
          (JetComponentSpace.repConj_smul_comm (T.fermion i).repJet_smul) U x)
```

lemma repJet_bosonSymbolMap [source]

The unconjugated bosonic symbol map intertwines the jet gauge actions.

```
lemma repJet_bosonSymbolMap (j : T.BosonSpecies) (U : GJ)
    (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (T.BosonValue j)) :
    T.repJet U (T.bosonSymbolMap j x)
      = T.bosonSymbolMap j
        (JetComponentSpace.repDual (T.boson j).repJet (T.boson j).repJet_smul U x)
```

lemma repJet_conjBosonSymbolMap [source]

The conjugate bosonic symbol map intertwines the jet gauge actions.

```
lemma repJet_conjBosonSymbolMap (j : T.BosonSpecies) (U : GJ)
    (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    T.repJet U (T.conjBosonSymbolMap j x)
      = T.conjBosonSymbolMap j
        (JetComponentSpace.repDual (JetComponentSpace.repConj (T.boson j).repJet)
          (JetComponentSpace.repConj_smul_comm (T.boson j).repJet_smul) U x)
```

lemma transformsIn_fermionSymbol [source]

The symbols of a fermionic species transform in the jet gauge representation of the species.

```
lemma transformsIn_fermionSymbol (i : T.FermionSpecies) :
    LocalGaugeData.TransformsIn (B := T.LocalFieldAlgebra) T.repJet (T.fermion i).repJet
      (T.fermionSymbol i)
```

lemma transformsIn_conjFermionSymbol [source]

The conjugate symbols of a fermionic species transform in the conjugate of the jet gauge representation of the species.

```
lemma transformsIn_conjFermionSymbol (i : T.FermionSpecies) :
    LocalGaugeData.TransformsIn (B := T.LocalFieldAlgebra) T.repJet
      (JetComponentSpace.repConj (T.fermion i).repJet) (T.conjFermionSymbol i)
```

lemma transformsIn_bosonSymbol [source]

The symbols of a bosonic species transform in the jet gauge representation of the species.

```
lemma transformsIn_bosonSymbol (j : T.BosonSpecies) :
    LocalGaugeData.TransformsIn (B := T.LocalFieldAlgebra) T.repJet (T.boson j).repJet
      (T.bosonSymbol j)
```

lemma transformsIn_conjBosonSymbol [source]

The conjugate symbols of a bosonic species transform in the conjugate of the jet gauge representation of the species.

```
lemma transformsIn_conjBosonSymbol (j : T.BosonSpecies) :
    LocalGaugeData.TransformsIn (B := T.LocalFieldAlgebra) T.repJet
      (JetComponentSpace.repConj (T.boson j).repJet) (T.conjBosonSymbol j)
```

lemma isLorentzDerivTransforms_of_tprod [source]

The Lorentz law of a family of symbols built from a linear map intertwining the Lorentz action with the tensor product of the action on derivative labels and a representation on the covectors.

```
private lemma isLorentzDerivTransforms_of_tprod (rep : Representation ℂ SL(2,ℂ) V)
    (Φ : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ V →ₗ[ℂ] T.LocalFieldAlgebra)
    (hΦ : ∀ (Λ : SL(2,ℂ)) (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ V),
```

```
    T.repLorentzGroup Λ (Φ x)
      = Φ ((SpaceTimeDerivAlgebraℂ.repLorentzGroup.tprod rep.dual) Λ x)) :
  IsLorentzDerivTransforms (A := T.LocalFieldAlgebra) T.repLorentzGroup rep
    (fun s => Φ ∘ₗ TensorProduct.mk ℂ _ _ (SpaceTimeDerivAlgebraℂ.basis s))
```

lemma repLorentzGroup_fermionSymbolMap [source]

The unconjugated fermionic symbol map intertwines the Lorentz actions.

```
lemma repLorentzGroup_fermionSymbolMap (i : T.FermionSpecies) (Λ : SL(2,ℂ))
    (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (T.FermionValue i)) :
    T.repLorentzGroup Λ (T.fermionSymbolMap i x)
      = T.fermionSymbolMap i
        ((SpaceTimeDerivAlgebraℂ.repLorentzGroup.tprod (T.fermion i).repLorentz.dual) Λ x)
```

lemma repLorentzGroup_conjFermionSymbolMap [source]

The conjugate fermionic symbol map intertwines the Lorentz actions.

```
lemma repLorentzGroup_conjFermionSymbolMap (i : T.FermionSpecies) (Λ : SL(2,ℂ))
    (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule (T.FermionValue i))) :
    T.repLorentzGroup Λ (T.conjFermionSymbolMap i x)
      = T.conjFermionSymbolMap i
        ((SpaceTimeDerivAlgebraℂ.repLorentzGroup.tprod
            (T.fermion i).repLorentz.conj.dual) Λ x)
```

lemma repLorentzGroup_bosonSymbolMap [source]

The unconjugated bosonic symbol map intertwines the Lorentz actions.

```
lemma repLorentzGroup_bosonSymbolMap (j : T.BosonSpecies) (Λ : SL(2,ℂ))
    (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (T.BosonValue j)) :
    T.repLorentzGroup Λ (T.bosonSymbolMap j x)
      = T.bosonSymbolMap j
        ((SpaceTimeDerivAlgebraℂ.repLorentzGroup.tprod (T.boson j).repLorentz.dual) Λ x)
```

lemma repLorentzGroup_conjBosonSymbolMap [source]

The conjugate bosonic symbol map intertwines the Lorentz actions.

```
lemma repLorentzGroup_conjBosonSymbolMap (j : T.BosonSpecies) (Λ : SL(2,ℂ))
    (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule (T.BosonValue j))) :
    T.repLorentzGroup Λ (T.conjBosonSymbolMap j x)
      = T.conjBosonSymbolMap j
        ((SpaceTimeDerivAlgebraℂ.repLorentzGroup.tprod (T.boson j).repLorentz.conj.dual) Λ x)
```

lemma isLorentzDerivTransforms_fermionSymbol [source]

The symbols of a fermionic species transform as the derivative symbols of a field in the Lorentz representation of the species.

```
lemma isLorentzDerivTransforms_fermionSymbol (i : T.FermionSpecies) :
    IsLorentzDerivTransforms (A := T.LocalFieldAlgebra) T.repLorentzGroup
      (T.fermion i).repLorentz (T.fermionSymbol i)
```

lemma isLorentzDerivTransforms_conjFermionSymbol [source]

The conjugate symbols of a fermionic species transform as the derivative symbols of a field in the conjugate of the Lorentz representation of the species.

```
lemma isLorentzDerivTransforms_conjFermionSymbol (i : T.FermionSpecies) :
    IsLorentzDerivTransforms (A := T.LocalFieldAlgebra) T.repLorentzGroup
      (T.fermion i).repLorentz.conj (T.conjFermionSymbol i)
```

lemma isLorentzDerivTransforms_bosonSymbol [source]

The symbols of a bosonic species transform as the derivative symbols of a field in the Lorentz representation of the species.

```
lemma isLorentzDerivTransforms_bosonSymbol (j : T.BosonSpecies) :
    IsLorentzDerivTransforms (A := T.LocalFieldAlgebra) T.repLorentzGroup
      (T.boson j).repLorentz (T.bosonSymbol j)
```

lemma isLorentzDerivTransforms_conjBosonSymbol [source]

The conjugate symbols of a bosonic species transform as the derivative symbols of a field in the conjugate of the Lorentz representation of the species.

```
lemma isLorentzDerivTransforms_conjBosonSymbol (j : T.BosonSpecies) :
    IsLorentzDerivTransforms (A := T.LocalFieldAlgebra) T.repLorentzGroup
      (T.boson j).repLorentz.conj (T.conjBosonSymbol j)
```

2.217 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.AdjointCoeff

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/AdjointCoeff.lean on GitHub.

def adjointCoeff [source]

The physicists' `∂_x (Ad_U)^a_b|₀`: include a constant gauge algebra element into jets, act by the adjoint of `U`, differentiate `x` times, and evaluate at the base point. For `x = 0` this is the adjoint action of the value of `U`; for `x ≠ 0` it sees the derivatives of the gauge transformation.

```
noncomputable def adjointCoeff (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) : 𝔤 →ₗ[ℝ] 𝔤
```

lemma adjointCoeff_apply [source]

```
lemma adjointCoeff_apply (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) (a : 𝔤) :
    jets.adjointCoeff U x a =
      jets.evalLie (jets.iteratedDeriv x (jets.adjointJ U (jets.ofConstantLie a)))
```

lemma adjointCoeff_zero [source]

The zeroth coefficient is the adjoint action of the value of the jet.

```
@[simp]
lemma adjointCoeff_zero (U : GJ) : jets.adjointCoeff U 0 = jets.adjoint (jets.eval U)
```

lemma adjointCoeff_zero_of_eval_eq_one [source]

A jet with trivial value has trivial zeroth coefficient.

```
lemma adjointCoeff_zero_of_eval_eq_one {U : GJ} (hU : jets.eval U = 1) :
    jets.adjointCoeff U 0 = LinearMap.id
```

lemma adjointCoeff_one [source]

The coefficients of the identity jet: only the base point survives.

```
lemma adjointCoeff_one (p : Multiset (Fin 1 ⊕ Fin 3)) :
    jets.adjointCoeff (1 : GJ) p = if p = 0 then LinearMap.id else 0
```

lemma adjointCoeff_lie [source]

The coefficients are derivations of the bracket up to convolution, by the iterated Leibniz rule for the jet bracket.

```
lemma adjointCoeff_lie (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) (a b : 𝔤) :
    jets.adjointCoeff U x ⁅a, b⁆ =
      (x.antidiagonal.map fun p => ⁅jets.adjointCoeff U p.1 a, jets.adjointCoeff U p.2 b⁆).sum
```

lemma deriv_adjoint_ofConstantLie [source]

One derivative of the adjoint action on a constant is minus the bracket with the Maurer–Cartan form: the Leibniz rule `deriv_adjoint` with the constant's derivative killed.

```
lemma deriv_adjoint_ofConstantLie (U : GJ) (μ : Fin 1 ⊕ Fin 3) (a : 𝔤) :
    jets.deriv μ (jets.adjointJ U (jets.ofConstantLie a)) =
      -⁅jets.maurerCartan U μ, jets.adjointJ U (jets.ofConstantLie a)⁆
```

lemma adjointCoeff_cons [source]

One more derivative of a coefficient: differentiating the adjoint once produces minus `ad` of the Maurer–Cartan form, and the remaining derivatives distribute over the bracket by the Leibniz rule.

```
lemma adjointCoeff_cons (U : GJ) (μ : Fin 1 ⊕ Fin 3) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    jets.adjointCoeff U (μ ::ₘ x) =
      -((x.antidiagonal.map fun p =>
        LieAlgebra.ad ℝ 𝔤 (jets.evalLie (jets.iteratedDeriv p.1 (jets.maurerCartan U μ))) ∘ₗ
          jets.adjointCoeff U p.2).sum)
```

lemma evalLie_iteratedDeriv_adjoint_cons [source]

The inductive step of the Taylor–Leibniz theorem: the rule for `μ ::ₘ s` derivatives follows from the rule for every sub-multiset of `s`. Peeling off `∂_μ` by the Leibniz rule `deriv_adjoint` leaves `Ad_U (∂_μ Y)`, handled by the rule for `s`, and a bracket with the Maurer–Cartan form, handled by the rule for the parts of `s`; on the other side the coefficients at `μ ::ₘ p` unfold by `adjointCoeff_cons`, and the two triple sums agree by coassociativity of the antidiagonal.

```
lemma evalLie_iteratedDeriv_adjoint_cons (U : GJ) (μ : Fin 1 ⊕ Fin 3)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (Y : 𝔤J)
    (ih : ∀ t ≤ s, ∀ Z : 𝔤J, jets.evalLie (jets.iteratedDeriv t (jets.adjointJ U Z)) =
      (t.antidiagonal.map fun p =>
        jets.adjointCoeff U p.1 (jets.evalLie (jets.iteratedDeriv p.2 Z))).sum) :
    jets.evalLie (jets.iteratedDeriv (μ ::ₘ s) (jets.adjointJ U Y)) =
      ((μ ::ₘ s).antidiagonal.map fun p =>
        jets.adjointCoeff U p.1 (jets.evalLie (jets.iteratedDeriv p.2 Y))).sum
```

lemma evalLie_iteratedDeriv_adjointJ [source]

The Taylor–Leibniz theorem for the adjoint action: the base-point Taylor coefficients of `Ad_U Y` are the antidiagonal convolution of the coefficients `adjointCoeff U` of `Ad_U` with those of `Y`. For a matrix group this is the Leibniz rule for products of matrices of power series; here it follows from the single-derivative Leibniz rule `deriv_adjoint` by strong induction on the number of derivatives.

```
lemma evalLie_iteratedDeriv_adjointJ (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) (Y : 𝔤J) :
    jets.evalLie (jets.iteratedDeriv x (jets.adjointJ U Y)) =
      (x.antidiagonal.map fun p =>
        jets.adjointCoeff U p.1 (jets.evalLie (jets.iteratedDeriv p.2 Y))).sum
```

lemma evalLie_iteratedDeriv_adjoint_eq_zero [source]

The base-point Taylor data of `Ad_U Y` vanish up to a given order whenever those of `Y` do.

```
lemma evalLie_iteratedDeriv_adjoint_eq_zero (U : GJ) {Y : 𝔤J} {s : Multiset (Fin 1 ⊕ Fin 3)}
    (h : ∀ q ≤ s, jets.evalLie (jets.iteratedDeriv q Y) = 0) :
    jets.evalLie (jets.iteratedDeriv s (jets.adjointJ U Y)) = 0
```

lemma adjointCoeff_mul [source]

The coefficients are multiplicative up to convolution: the coefficient of a product of jets is the antidiagonal convolution of the coefficients of the factors.

```
lemma adjointCoeff_mul (U V : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    jets.adjointCoeff (U * V) x =
      (x.antidiagonal.map fun p => jets.adjointCoeff U p.1 ∘ₗ jets.adjointCoeff V p.2).sum
```

def adjointDualCoeff [source]

The physicists' `∂_x (Ad_U)^a_b|₀` acting on the dual adjoint index of a gauge-field symbol: the transpose of `adjointCoeff U x`.

```
noncomputable def adjointDualCoeff (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℝ 𝔤 →ₗ[ℝ] Module.Dual ℝ 𝔤
```

lemma adjointDualCoeff_apply [source]

```
lemma adjointDualCoeff_apply (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ 𝔤)
    (a : 𝔤) : jets.adjointDualCoeff U x φ a = φ (jets.adjointCoeff U x a)
```

lemma adjointDualCoeff_zero [source]

The zeroth dual coefficient is the dual of the adjoint action of the value of the jet.

```
lemma adjointDualCoeff_zero (U : GJ) :
    jets.adjointDualCoeff U 0 = (jets.adjoint (jets.eval U)).dualMap
```

lemma adjointDualCoeff_zero_of_eval_eq_one [source]

A jet with trivial value has trivial zeroth dual coefficient.

```
lemma adjointDualCoeff_zero_of_eval_eq_one {U : GJ} (hU : jets.eval U = 1) :
    jets.adjointDualCoeff U 0 = LinearMap.id
```

lemma adjointDualCoeff_cons [source]

The dual form of `adjointCoeff_cons`: one more derivative of a dual coefficient is minus the antidiagonal convolution of lower dual coefficients against `ad` of the derived Maurer–Cartan form.

```
lemma adjointDualCoeff_cons (U : GJ) (μ : Fin 1 ⊕ Fin 3) (x : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℝ 𝔤) :
    jets.adjointDualCoeff U (μ ::ₘ x) φ =
      -((x.antidiagonal.map fun p =>
        jets.adjointDualCoeff U p.2 (φ ∘ₗ LieAlgebra.ad ℝ 𝔤
          (jets.evalLie (jets.iteratedDeriv p.1 (jets.maurerCartan U μ))))).sum)
```

lemma adjointDualCoeff_singleton [source]

The dual coefficient at a single derivative is minus the underived coefficient pre-composed with `ad` of the base-point Maurer–Cartan form. This is what cancels the Leibniz cross terms of the gauge law against the commutator cross terms in the field strength.

```
lemma adjointDualCoeff_singleton (U : GJ) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    jets.adjointDualCoeff U {μ} φ =
      -jets.adjointDualCoeff U 0 (φ ∘ₗ LieAlgebra.ad ℝ 𝔤
        (jets.evalLie (jets.maurerCartan U μ)))
```

lemma adjointDualCoeff_pair [source]

The dual coefficient at two derivatives: the underived coefficient against `ad` of the derived Maurer–Cartan form, and the once-derived coefficient against `ad` of the Maurer–Cartan form itself.

```
lemma adjointDualCoeff_pair (U : GJ) (ρ μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ 𝔤) :
    jets.adjointDualCoeff U (ρ ::ₘ {μ}) φ =
      -jets.adjointDualCoeff U 0 (φ ∘ₗ LieAlgebra.ad ℝ 𝔤
        (jets.evalLie (jets.deriv ρ (jets.maurerCartan U μ))))
      - jets.adjointDualCoeff U {ρ} (φ ∘ₗ LieAlgebra.ad ℝ 𝔤
        (jets.evalLie (jets.maurerCartan U μ)))
```

2.218 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.Basic

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/Basic.lean on GitHub.

lemma maurerCartan_one [source]

The Maurer–Cartan form of the identity vanishes: the cocycle law at `1 * 1 = 1`.

```
@[simp]
lemma maurerCartan_one (μ : Fin 1 ⊕ Fin 3) : jets.maurerCartan 1 μ = 0
```

lemma maurerCartan_inv [source]

The Maurer–Cartan form of an inverse: `ω_μ(U⁻¹) = - Ad_{U⁻¹} ω_μ(U)`, the cocycle law applied to `U⁻¹ U = 1`.

```
lemma maurerCartan_inv (U : GJ) (μ : Fin 1 ⊕ Fin 3) :
    jets.maurerCartan U⁻¹ μ = - jets.adjointJ U⁻¹ (jets.maurerCartan U μ)
```

lemma evalLie_adjointJ_ofConstantLie [source]

At the base point, the adjoint action of a jet on a constant is the adjoint action of its value.

```
lemma evalLie_adjointJ_ofConstantLie (U : GJ) (a : 𝔤) :
    jets.evalLie (jets.adjointJ U (jets.ofConstantLie a)) = jets.adjoint (jets.eval U) a
```

lemma maurerCartan_eq_of_deriv_adjointJ [source]

The Maurer–Cartan form is determined by the Leibniz rule, up to the centre. Since `adjointJ U` is invertible, `deriv_adjointJ` says exactly that the inner derivation `⁅maurerCartan U μ, ·⁆` is `adjointJ U ∘ deriv μ ∘ adjointJ U⁻¹ − deriv μ`; so any other form obeying the same rule differs from it by something acting trivially in the adjoint representation of `𝔤J`. It follows that the Maurer–Cartan form is redundant data exactly when that representation is faithful — which it is not for the Standard Model, whose jet gauge algebra has a central `u(1)` factor. That is why `maurerCartan` is a field of the structure rather than a construction from the rest of it.

```
lemma maurerCartan_eq_of_deriv_adjointJ
    (hfaithful : ∀ x y : 𝔤J, (∀ z : 𝔤J, ⁅x, z⁆ = ⁅y, z⁆) → x = y)
    (ω : GJ → (Fin 1 ⊕ Fin 3) → 𝔤J)
    (hω : ∀ (U : GJ) (μ : Fin 1 ⊕ Fin 3) (x : 𝔤J),
      jets.deriv μ (jets.adjointJ U x)
        = jets.adjointJ U (jets.deriv μ x) - ⁅ω U μ, jets.adjointJ U x⁆) :
    ω = jets.maurerCartan
```

lemma evalLie_iteratedDeriv_bracket_congr [source]

The base-point Taylor data of a bracket is determined by that of its arguments. If the base-point values of the iterated derivatives of `a` and `b` along sub-multisets of `w` agree with those of `a'` and `b'`, then so do those of the brackets: the iterated Leibniz rule expands the bracket over the antidiagonal of `w`, whose parts are all sub-multisets of `w`.

```
lemma evalLie_iteratedDeriv_bracket_congr (w : Multiset (Fin 1 ⊕ Fin 3)) (a b a' b' : 𝔤J)
    (ha : ∀ p ≤ w, jets.evalLie (jets.iteratedDeriv p a)
      = jets.evalLie (jets.iteratedDeriv p a'))
    (hb : ∀ p ≤ w, jets.evalLie (jets.iteratedDeriv p b)
      = jets.evalLie (jets.iteratedDeriv p b')) :
```

```
    jets.evalLie (jets.iteratedDeriv w [a, b])
      = jets.evalLie (jets.iteratedDeriv w [a', b'])
```

class Faithful [source]

A package is faithful when an element of `gJ` is determined by the base-point values of its iterated derivatives (Taylor determinacy) and a jet with vanishing Maurer–Cartan form is the constant jet of its value.

```
class Faithful (jets : LocalGaugeData G₀ g GJ gJ) : Prop where
  ext_of_evalLie_iteratedDeriv : ∀ {x y : gJ},
    (∀ s : Multiset (Fin 1 ⊕ Fin 3),
      jets.evalLie (jets.iteratedDeriv s x) = jets.evalLie (jets.iteratedDeriv s y)) → x = y
  eq_ofConstant_of_maurerCartan_eq_zero : ∀ {U : GJ},
    jets.maurerCartan U = 0 → U = jets.ofConstant (jets.eval U)
```

lemma ext_of_evalLie_iteratedDeriv [source]

Taylor determinacy of a faithful package, in the form of an extensionality lemma.

```
lemma ext_of_evalLie_iteratedDeriv [jets.Faithful] {x y : gJ}
    (h : ∀ s : Multiset (Fin 1 ⊕ Fin 3),
      jets.evalLie (jets.iteratedDeriv s x) = jets.evalLie (jets.iteratedDeriv s y)) :
    x = y
```

2.219 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.Factor

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/Factor.lean on GitHub.

structure U1Factor [source]

***A `U(1)` factor** of the local gauge data: a unitary jet `u U` attached to each gauge jet, with the corresponding components `φ c` of the gauge algebra and `φJ a` of its jets, related by the Maurer–Cartan form `φJ (ω_μ U) = i (∂_μ u) u⁻¹` and invariant under the adjoint action.*

```
structure U1Factor (jets : LocalGaugeData G₀ g GJ gJ) where
  u : GJ →* unitary SpaceTimeAlgebra
  φ : g →ₗ[ℝ] ℂ
  φJ : gJ → SpaceTimeAlgebra
  φJ_ofConstantLie : ∀ c, φJ (jets.ofConstantLie c) = C (φ c)
  φJ_cc_foldl : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (a : gJ),
    constantCoeff (SpaceTimeAlgebra.iteratedPDeriv p (φJ a))
      = φ (jets.evalLie (jets.iteratedDeriv p a))
  φJ_maurerCartan : ∀ (U : GJ) (μ : Fin 1 ⊕ Fin 3),
    φJ (jets.maurerCartan U μ)
      = Complex.I • (pderiv μ (u U : SpaceTimeAlgebra) * star (u U : SpaceTimeAlgebra))
  φJ_adjoint : ∀ (U : GJ) (c : g),
    φJ (jets.adjointJ U (jets.ofConstantLie c)) = φJ (jets.ofConstantLie c)
```

structure SUFactor [source]

***An `SU(n)` factor** of the local gauge data: a unitary matrix of jets `u U` attached to each gauge jet, with the corresponding matrix components `φ c` of the gauge algebra*

and `φJ a` of its jets, related by the Maurer–Cartan form `φJ (ω_μ U) = i (∂_μ u) u⁻¹` and transforming by conjugation under the adjoint action.

```
structure SUFactor (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) (n : Type) [Fintype n] [DecidableEq n]
    where
  u : GJ →* Matrix n n SpaceTimeAlgebra
  u_unitary : ∀ U, star (u U) * u U = 1
  φ : 𝔤 →ₗ[ℝ] Matrix n n ℂ
  φJ : 𝔤J → Matrix n n SpaceTimeAlgebra
  φJ_ofConstantLie : ∀ c, φJ (jets.ofConstantLie c) = (φ c).map (C : ℂ → SpaceTimeAlgebra)
  φJ_cc_foldl : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (a : 𝔤J),
    ((φJ a).map fun f => constantCoeff (SpaceTimeAlgebra.iteratedPDeriv p f))
      = φ (jets.evalLie (jets.iteratedDeriv p a))
  φJ_maurerCartan : ∀ (U : GJ) (μ : Fin 1 ⊕ Fin 3),
    φJ (jets.maurerCartan U μ)
      = Complex.I • (((u U).map fun f => pderiv μ f) * star (u U))
  φJ_adjoint : ∀ (U : GJ) (c : 𝔤),
    φJ (jets.adjointJ U (jets.ofConstantLie c)) = u U * φJ (jets.ofConstantLie c) * star (u U)
```

`inductive Factor` [source]

***A factor of the gauge group**, presented in the local gauge data: a `U(1)` factor or an `SU(n)` factor.*

```
inductive Factor (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J)
  | U1 (F : U1Factor jets)
  | SU {n : ℕ} (F : SUFactor jets (Fin n))
```

`abbrev Factors` [source]

A gauge group presented by its factors.

```
abbrev Factors (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) : Type
```

`def U1Factor.comap` [source]

A `U(1)` factor pulled back along a morphism.

```
noncomputable def U1Factor.comap (F : U1Factor jets') (h : Hom jets jets') : U1Factor jets
    where
```

`def SUFactor.comap` [source]

An `SU(n)` factor pulled back along a morphism.

```
noncomputable def SUFactor.comap {n : Type} [Fintype n] [DecidableEq n] (F : SUFactor jets' n)
    (h : Hom jets jets') : SUFactor jets n where
```

`abbrev Factor.comap` [source]

A factor pulled back along a morphism.

```
noncomputable abbrev Factor.comap (h : Hom jets jets') : Factor jets' → Factor jets
  | .U1 F => .U1 (F.comap h)
  | .SU F => .SU (F.comap h)
```

abbrev Factors.comap [source]

A list of factors pulled back along a morphism. Written by recursion rather than as `List.map`, so that it unfolds at reducible transparency, as the charge tuples of a table over `Factors.factors` require.

```
noncomputable abbrev Factors.comap (h : Hom jets jets') : Factors jets' → Factors jets
  | [] => []
  | F :: Fs => F.comap h :: Factors.comap h Fs
```

2.220 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.MatrixJets

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/MatrixJets.lean on GitHub.

structure MatrixJets [source]

***A presentation of gauge jets by matrices**: the value group, its Lie algebra, the jet group and its Lie algebra of jets embed into `κ × κ` matrices, and every structure map of the local gauge data is what it is for matrices: the entrywise constant coefficient, inclusion of constants, derivative and coordinate multiplication, conjugation for the adjoint actions and `i (∂_μ U) U†` for the Maurer–Cartan form.*

```
structure MatrixJets (κ : Type) [Fintype κ] [DecidableEq κ] (G₀ : Type) [Group G₀]
    (𝔤 : Type) [LieRing 𝔤] [LieAlgebra ℝ 𝔤] (GJ : Type) [Group GJ]
    (𝔤J : Type) [LieRing 𝔤J] [LieAlgebra ℝ 𝔤J] where
  toMat₀ : G₀ →* Matrix κ κ ℂ
  toMat₀_injective : Function.Injective toMat₀
  toMatJ : GJ →* Matrix κ κ SpaceTimeAlgebra
  toMatJ_injective : Function.Injective toMatJ
  toMatJ_mul_star : ∀ U, toMatJ U * star (toMatJ U) = 1
  star_toMatJ_mul : ∀ U, star (toMatJ U) * toMatJ U = 1
  lie₀ : 𝔤 →ₗ[ℝ] Matrix κ κ ℂ
  lie₀_injective : Function.Injective lie₀
  lie₀_bracket : ∀ a b, lie₀ ⁅a, b⁆ = Complex.I • (lie₀ a * lie₀ b - lie₀ b * lie₀ a)
  lieJ : 𝔤J →ₗ[ℝ] Matrix κ κ SpaceTimeAlgebra
  lieJ_injective : Function.Injective lieJ
  lieJ_bracket : ∀ a b, lieJ ⁅a, b⁆ = Complex.I • (lieJ a * lieJ b - lieJ b * lieJ a)
  eval : GJ →* G₀
  toMat₀_eval : ∀ U, toMat₀ (eval U) = (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix (
    toMatJ U)
  ofConstant : G₀ →* GJ
  toMatJ_ofConstant : ∀ g, toMatJ (ofConstant g) = (C : ℂ →+* SpaceTimeAlgebra).mapMatrix (
    toMat₀ g)
  evalLie : 𝔤J →ₗ[ℝ] 𝔤
  lie₀_evalLie : ∀ a, lie₀ (evalLie a) = (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix (
    lieJ a)
  ofConstantLie : 𝔤 →ₗ[ℝ] 𝔤J
  lieJ_ofConstantLie : ∀ a, lieJ (ofConstantLie a) = (C : ℂ →+* SpaceTimeAlgebra).mapMatrix (
    lie₀ a)
  deriv : (Fin 1 ⊕ Fin 3) → 𝔤J →ₗ[ℝ] 𝔤J
  lieJ_deriv : ∀ μ a, lieJ (deriv μ a) = (lieJ a).map (pderiv μ)
  coord : (Fin 1 ⊕ Fin 3) → 𝔤J →ₗ[ℝ] 𝔤J
  lieJ_coord : ∀ μ a, lieJ (coord μ a) = (X μ : SpaceTimeAlgebra) • lieJ a
  adjoint : Representation ℝ GJ 𝔤J
  lieJ_adjoint : ∀ U a, lieJ (adjoint U a) = toMatJ U * lieJ a * star (toMatJ U)
  adjointValue : Representation ℝ G₀ 𝔤
  lie₀_adjointValue : ∀ g a, lie₀ (adjointValue g a) = toMat₀ g * lie₀ a * star (toMat₀ g)
```

```
  maurerCartan : GJ → (Fin 1 ⊕ Fin 3) → 𝔤J
  lieJ_maurerCartan : ∀ U μ, lieJ (maurerCartan U μ)
    = Complex.I • ((toMatJ U).map (pderiv μ) * star (toMatJ U))
```

lemma eval_ofConstant [source]

```
lemma eval_ofConstant (g : G₀) : M.eval (M.ofConstant g) = g
```

lemma evalLie_lie [source]

```
lemma evalLie_lie (a b : 𝔤J) : M.evalLie ⁅a, b⁆ = ⁅M.evalLie a, M.evalLie b⁆
```

lemma ofConstantLie_lie [source]

```
lemma ofConstantLie_lie (a b : 𝔤) :
    M.ofConstantLie ⁅a, b⁆ = ⁅M.ofConstantLie a, M.ofConstantLie b⁆
```

lemma evalLie_ofConstantLie [source]

```
lemma evalLie_ofConstantLie (a : 𝔤) : M.evalLie (M.ofConstantLie a) = a
```

lemma deriv_comm [source]

```
lemma deriv_comm (μ ν : Fin 1 ⊕ Fin 3) (a : 𝔤J) :
    M.deriv μ (M.deriv ν a) = M.deriv ν (M.deriv μ a)
```

lemma deriv_bracket [source]

```
lemma deriv_bracket (μ : Fin 1 ⊕ Fin 3) (x y : 𝔤J) :
    M.deriv μ ⁅x, y⁆ = ⁅M.deriv μ x, y⁆ + ⁅x, M.deriv μ y⁆
```

lemma deriv_ofConstantLie [source]

```
lemma deriv_ofConstantLie (μ : Fin 1 ⊕ Fin 3) (a : 𝔤) : M.deriv μ (M.ofConstantLie a) = 0
```

lemma deriv_coord [source]

```
lemma deriv_coord (μ ν : Fin 1 ⊕ Fin 3) (a : 𝔤J) :
    M.deriv μ (M.coord ν a) = M.coord ν (M.deriv μ a) + if μ = ν then a else 0
```

lemma evalLie_coord [source]

```
lemma evalLie_coord (μ : Fin 1 ⊕ Fin 3) (a : 𝔤J) : M.evalLie (M.coord μ a) = 0
```

lemma coord_lie [source]

```
lemma coord_lie (μ : Fin 1 ⊕ Fin 3) (a b : 𝔤J) : ⁅M.coord μ a, b⁆ = M.coord μ ⁅a, b⁆
```

lemma conj_mul_conj [source]

Conjugation by a unitary matrix respects products.

```
lemma conj_mul_conj (U : GJ) (A B : Matrix κ κ SpaceTimeAlgebra) :
    (M.toMatJ U * A * star (M.toMatJ U)) * (M.toMatJ U * B * star (M.toMatJ U))
      = M.toMatJ U * (A * B) * star (M.toMatJ U)
```

lemma adjoint_lie [source]

```
lemma adjoint_lie (U : GJ) (x y : 𝔤J) :
    M.adjoint U ⁅x, y⁆ = ⁅M.adjoint U x, M.adjoint U y⁆
```

lemma evalLie_adjoint [source]

```
lemma evalLie_adjoint (U : GJ) (x : 𝔤J) :
    M.evalLie (M.adjoint U x) = M.adjointValue (M.eval U) (M.evalLie x)
```

lemma maurerCartan_ofConstant [source]

```
lemma maurerCartan_ofConstant (g : G₀) (μ : Fin 1 ⊕ Fin 3) :
    M.maurerCartan (M.ofConstant g) μ = 0
```

lemma maurerCartan_cocycle [source]

```
lemma maurerCartan_cocycle (U V : GJ) (μ : Fin 1 ⊕ Fin 3) :
    M.maurerCartan (U * V) μ = M.maurerCartan U μ + M.adjoint U (M.maurerCartan V μ)
```

lemma maurerCartan_structure [source]

```
lemma maurerCartan_structure (U : GJ) (μ ν : Fin 1 ⊕ Fin 3) :
    M.deriv μ (M.maurerCartan U ν) - M.deriv ν (M.maurerCartan U μ)
      + ⁅M.maurerCartan U μ, M.maurerCartan U ν⁆ = 0
```

lemma deriv_adjoint [source]

```
lemma deriv_adjoint (U : GJ) (μ : Fin 1 ⊕ Fin 3) (x : 𝔤J) :
    M.deriv μ (M.adjoint U x) = M.adjoint U (M.deriv μ x)
      - ⁅M.maurerCartan U μ, M.adjoint U x⁆
```

def evalLieHom [source]

Evaluation of Lie algebra jets, as a morphism of Lie algebras.

```
noncomputable def evalLieHom : 𝔤J →ₗ⁅ℝ⁆ 𝔤 where
```

lemma evalLieHom_apply [source]

```
@[simp]
lemma evalLieHom_apply (a : 𝔤J) : M.evalLieHom a = M.evalLie a
```

def toLocalGaugeData [source]

The local gauge data presented by matrices of jets.

```
noncomputable def toLocalGaugeData : LocalGaugeData G₀ 𝔤 GJ 𝔤J where
```

lemma lieJ_iteratedDeriv [source]

The iterated derivative is the entrywise iterated formal derivative.

```
lemma lieJ_iteratedDeriv (s : Multiset (Fin 1 ⊕ Fin 3)) (a : 𝔤J) :
    M.lieJ (M.toLocalGaugeData.iteratedDeriv s a)
      = (M.lieJ a).map fun f => SpaceTimeAlgebra.iteratedPDeriv s f
```

lemma lie₀_evalLie_iteratedDeriv [source]

The base-point value of the iterated derivative, entrywise.

```
lemma lie₀_evalLie_iteratedDeriv (s : Multiset (Fin 1 ⊕ Fin 3)) (a : 𝔤J) :
    M.lie₀ (M.toLocalGaugeData.evalLie (M.toLocalGaugeData.iteratedDeriv s a))
      = (M.lieJ a).map fun f => constantCoeff (SpaceTimeAlgebra.iteratedPDeriv s f)
```

lemma faithful [source]

***A package presented by matrices is faithful**: a jet is determined entrywise by the constant coefficients of its derivatives, and a jet with vanishing Maurer–Cartan form has constant entries.*

```
lemma faithful : M.toLocalGaugeData.Faithful where
```

def suFactor [source]

***The canonical `SU`-type factor** of a package presented by matrices: the matrices of jets themselves.*

```
noncomputable def suFactor : SUFactor M.toLocalGaugeData κ where
```

lemma lieJ_radial [source]

The radial component of the Maurer–Cartan form, in matrices: `∑ μ, x μ • I • (∂ μ U) U†`.

```
lemma lieJ_radial (U : GJ) :
    M.lieJ (M.toLocalGaugeData.radial U) =
      ∑ μ, (X μ : SpaceTimeAlgebra) •
          (Complex.I • ((M.toMatJ U).map (pderiv μ) * star (M.toMatJ U)))
```

lemma mem_truncationKer_zero_iff [source]

A gauge jet is pure exactly when its matrix is the identity at the base point.

```
lemma mem_truncationKer_zero_iff (U : GJ) :
    U ∈ M.toLocalGaugeData.truncationKer 0 ↔
      (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix (M.toMatJ U) = 1
```

lemma free [source]

***A criterion for freeness.** A presentation by matrices is free when every matrix of jets built entrywise from Taylor data in $\mathfrak{g}$ is a Lie algebra jet, the Lie algebra jets are hermitian, and every unitary solution `V` of `E V = -i P V, V(0) = 1`, for a Lie algebra jet `P`, is a gauge jet.*

```
lemma free
    (hTaylor : ∀ c : Multiset (Fin 1 ⊕ Fin 3) → 𝔤,
      ∃ Y, M.lieJ Y = SpaceTimeAlgebra.taylorMatrix fun s => M.lie₀ (c s))
    (hherm : ∀ a, star (M.lieJ a) = M.lieJ a)
    (hlift : ∀ (ρ : 𝔤J) (V : Matrix κ κ SpaceTimeAlgebra),
      (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix V = 1 → V * star V = 1 →
      ∑ μ, (X μ : SpaceTimeAlgebra) • V.map (pderiv μ) = ((-Complex.I) • M.lieJ ρ) * V →
      ∃ U, M.toMatJ U = V) :
    M.toLocalGaugeData.Free where
```

2.221 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.MaurerCartan

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/MaurerCartan.lean on GitHub.

lemma evalLie_iteratedDeriv_maurerCartan_structure [source]

The all-orders structural equation of the Maurer–Cartan form, at the base point: the `s`-th derivative of `∂_μ ω_ν − ∂_ν ω_μ + ⁅ω_μ, ω_ν⁆ = 0`, with the bracket expanded by the iterated Leibniz rule.

```
lemma evalLie_iteratedDeriv_maurerCartan_structure
    (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    jets.evalLie (jets.iteratedDeriv (μ ::ₘ s) (jets.maurerCartan U ν)) =
      jets.evalLie (jets.iteratedDeriv (ν ::ₘ s) (jets.maurerCartan U μ))
      - (s.antidiagonal.map fun p =>
          ⁅jets.evalLie (jets.iteratedDeriv p.1 (jets.maurerCartan U μ)),
            jets.evalLie (jets.iteratedDeriv p.2 (jets.maurerCartan U ν))⁆).sum
```

def symmetrizedMaurerCartanForm [source]

The symmetrized Maurer–Cartan form `ω̃_r(U) = (1/|r|) ∑_{μ ∈ r} ∂_{r − {μ}} ω_μ(U)`: the average, over the directions of `r`, of the Maurer–Cartan form in one direction differentiated along the remaining ones.

```
noncomputable def symmetrizedMaurerCartanForm (U : GJ) (r : Multiset (Fin 1 ⊕ Fin 3)) : 𝔤J
```

lemma symmetrizedMaurerCartanForm_apply_zero [source]

```
@[simp]
lemma symmetrizedMaurerCartanForm_apply_zero (U : GJ) :
    jets.symmetrizedMaurerCartanForm U 0 = 0
```

lemma symmetrizedMaurerCartanForm_one [source]

```
@[simp]
lemma symmetrizedMaurerCartanForm_one : jets.symmetrizedMaurerCartanForm 1 = 0
```

lemma symmetrizedMaurerCartanForm_ofConstant [source]

```
@[simp]
lemma symmetrizedMaurerCartanForm_ofConstant (g : G₀) :
    jets.symmetrizedMaurerCartanForm (jets.ofConstant g) = 0
```

lemma symmetrizedMaurerCartanForm_singleton [source]

```
@[simp]
lemma symmetrizedMaurerCartanForm_singleton (U : GJ) (μ : Fin 1 ⊕ Fin 3) :
    jets.symmetrizedMaurerCartanForm U {μ} = jets.maurerCartan U μ
```

lemma symmetrizedMaurerCartanForm_cons [source]

The recursion for the symmetrized Maurer–Cartan form: peeling one direction off the multiset.

```
lemma symmetrizedMaurerCartanForm_cons (U : GJ) (μ : Fin 1 ⊕ Fin 3)
    (r : Multiset (Fin 1 ⊕ Fin 3)) : jets.symmetrizedMaurerCartanForm U (μ ::ₘ r) =
    (1/(r.card + 1 : ℝ) : ℝ) • (jets.iteratedDeriv r (jets.maurerCartan U μ))
    + ((r.card : ℝ)/(r.card + 1 : ℝ)) •
      jets.deriv μ (jets.symmetrizedMaurerCartanForm U r)
```

lemma iteratedDeriv_maurerCartan_eq_symmetrized_add [source]

The symmetrization defect of the Maurer–Cartan form: an iterated derivative of `ω` is the corresponding symmetrized form plus an average of iterated derivatives of brackets of `ω` in strictly fewer directions. This is the structural equation `maurerCartan_structure` used to trade the antisymmetric part for lower-order data.

```
lemma iteratedDeriv_maurerCartan_eq_symmetrized_add (U : GJ)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    jets.iteratedDeriv s (jets.maurerCartan U μ) =
      jets.symmetrizedMaurerCartanForm U (μ ::ₘ s) +
      (1/(s.card + 1 : ℝ)) • (s.map fun ν =>
        jets.iteratedDeriv (s.erase ν)
          [jets.maurerCartan U μ, jets.maurerCartan U ν]).sum
```

lemma evalLie_iteratedDeriv_maurerCartan_eq_zero_of_symmetrized_eq_zero [source]

Maurer–Cartan triangularity, base-point half: if the base-point symmetrized Maurer–Cartan data of `U` vanish in every nonempty multiset of at most `n` directions, then so do all its base-point Maurer–Cartan Taylor coefficients below order `n`. The induction is on the order: the symmetrization defect `iteratedDeriv_maurerCartan_eq_symmetrized_add` expresses `∂_s ω_μ` through the symmetrized form, which vanishes by hypothesis, and brackets of `ω`s differentiated strictly fewer times, which vanish by the inductive hypothesis through `evalLie_iteratedDeriv_bracket_congr`.

```
lemma evalLie_iteratedDeriv_maurerCartan_eq_zero_of_symmetrized_eq_zero (U : GJ) {n : ℕ}
    (h : ∀ r : Multiset (Fin 1 ⊕ Fin 3), r ≠ 0 → r.card ≤ n →
      jets.evalLie (jets.symmetrizedMaurerCartanForm U r) = 0)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (hs : s.card < n) :
    jets.evalLie (jets.iteratedDeriv s (jets.maurerCartan U μ)) = 0
```

lemma evalLie_iteratedDeriv_maurerCartan_eq_of_symmetrized_eq [source]

Determination step: if the base-point symmetrized Maurer–Cartan data of `U` and `V` agree, and their Maurer–Cartan Taylor data agree in fewer than `n` directions, then they agree in `n` directions.

```
lemma evalLie_iteratedDeriv_maurerCartan_eq_of_symmetrized_eq (U V : GJ) (n : ℕ)
    (hsym : ∀ r, jets.evalLie (jets.symmetrizedMaurerCartanForm U r) =
      jets.evalLie (jets.symmetrizedMaurerCartanForm V r))
    (ih : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3), s.card < n →
      jets.evalLie (jets.iteratedDeriv s (jets.maurerCartan U μ)) =
        jets.evalLie (jets.iteratedDeriv s (jets.maurerCartan V μ)))
    (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (hs : s.card = n) :
    jets.evalLie (jets.iteratedDeriv s (jets.maurerCartan U μ)) =
      jets.evalLie (jets.iteratedDeriv s (jets.maurerCartan V μ))
```

lemma evalLie_iteratedDeriv_maurerCartan_eq_of_symmetrized_eq_all [source]

***The symmetrized data determine the Maurer–Cartan Taylor data**: if the base-point symmetrized Maurer–Cartan data of `U` and `V` agree, so do all base-point Taylor coefficients of their Maurer–Cartan forms. Strong induction on the order, with `evalLie_iteratedDeriv_maurerCartan_eq_of_symmetrized_eq` as the step.*

```
lemma evalLie_iteratedDeriv_maurerCartan_eq_of_symmetrized_eq_all (U V : GJ)
    (hsym : ∀ r, jets.evalLie (jets.symmetrizedMaurerCartanForm U r) =
      jets.evalLie (jets.symmetrizedMaurerCartanForm V r))
    (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    jets.evalLie (jets.iteratedDeriv s (jets.maurerCartan U μ)) =
      jets.evalLie (jets.iteratedDeriv s (jets.maurerCartan V μ))
```

lemma maurerCartan_eq_zero_iff [source]

In a faithful package, the Maurer–Cartan form vanishes exactly on the constant jets.

```
lemma maurerCartan_eq_zero_iff [jets.Faithful] (U : GJ) :
    jets.maurerCartan U = 0 ↔ U = jets.ofConstant (jets.eval U)
```

2.222 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.OfFactors

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/OfFactors.lean on GitHub.

inductive FactorSpec [source]

***A factor symbol**: `U(1)` or `SU(n)`.*

```
inductive FactorSpec
  | U1
  | SU (n : ℕ)
  deriving DecidableEq, Repr
```

def FactorSpec.G [source]

The group of jets of a factor.

```
def G : FactorSpec → Type
  | .U1 => JetU1
```

```
  | .SU n => JetSU n
```

def FactorSpec.𝔤 [source]

The Lie algebra of a factor.

```
def 𝔤 : FactorSpec → Type
  | .U1 => U1Algebra
  | .SU n => SUAlgebra n
```

def FactorSpec.G₀ [source]

The group of a factor.

```
def G₀ : FactorSpec → Type
  | .U1 => _root_.U1
  | .SU n => _root_.SU n
```

def FactorSpec.𝔤J [source]

The jets of the Lie algebra of a factor.

```
def 𝔤J : FactorSpec → Type
  | .U1 => JetU1Algebra
  | .SU n => JetSUAlgebra n
```

instance FactorSpec.instGroupG [source]

```
noncomputable instance instGroupG : (f : FactorSpec) → Group f.G
  | .U1 => inferInstanceAs (Group JetU1)
  | .SU n => inferInstanceAs (Group (JetSU n))
```

instance FactorSpec.instGroupG₀ [source]

```
noncomputable instance instGroupG₀ : (f : FactorSpec) → Group f.G₀
  | .U1 => inferInstanceAs (Group _root_.U1)
  | .SU n => inferInstanceAs (Group (_root_.SU n))
```

instance FactorSpec.instLieRing𝔤 [source]

```
noncomputable instance instLieRing𝔤 : (f : FactorSpec) → LieRing f.𝔤
  | .U1 => inferInstanceAs (LieRing U1Algebra)
  | .SU n => inferInstanceAs (LieRing (SUAlgebra n))
```

instance FactorSpec.instLieAlgebra𝔤 [source]

```
noncomputable instance instLieAlgebra𝔤 : (f : FactorSpec) → LieAlgebra ℝ f.𝔤
  | .U1 => inferInstanceAs (LieAlgebra ℝ U1Algebra)
  | .SU n => inferInstanceAs (LieAlgebra ℝ (SUAlgebra n))
```

instance FactorSpec.instLieRinggJ [source]

```
noncomputable instance instLieRinggJ : (f : FactorSpec) → LieRing f.gJ
  | .U1 => inferInstanceAs (LieRing JetU1Algebra)
  | .SU n => inferInstanceAs (LieRing (JetSUAlgebra n))
```

instance FactorSpec.instLieAlgebragJ [source]

```
noncomputable instance instLieAlgebragJ : (f : FactorSpec) → LieAlgebra ℝ f.gJ
  | .U1 => inferInstanceAs (LieAlgebra ℝ JetU1Algebra)
  | .SU n => inferInstanceAs (LieAlgebra ℝ (JetSUAlgebra n))
```

instance FactorSpec.instFiniteg [source]

```
noncomputable instance instFiniteg : (f : FactorSpec) → Module.Finite ℝ f.g
  | .U1 => inferInstanceAs (Module.Finite ℝ U1Algebra)
  | .SU n => inferInstanceAs (Module.Finite ℝ (SUAlgebra n))
```

abbrev data [source]

The local gauge data of a factor.

```
noncomputable abbrev data : (f : FactorSpec) → LocalGaugeData f.G₀ f.g f.G f.gJ
  | .U1 => u1
  | .SU n => su n
```

abbrev factor [source]

The canonical factor of the local gauge data of a factor symbol.

```
noncomputable abbrev factor : (f : FactorSpec) → Factor f.data
  | .U1 => .U1 u1Factor
  | .SU n => .SU (suFactor n)
```

instance instFaithfulData [source]

```
noncomputable instance instFaithfulData : (f : FactorSpec) → f.data.Faithful
  | .U1 => inferInstanceAs u1.Faithful
  | .SU n => inferInstanceAs (su n).Faithful
```

instance instFreeData [source]

```
noncomputable instance instFreeData : (f : FactorSpec) → f.data.Free
  | .U1 => inferInstanceAs u1.Free
  | .SU n => inferInstanceAs (su n).Free
```

def OfFactors.G [source]

The group of jets of a list of factors.

```
def G : List FactorSpec → Type
  | [] => Unit
  | [f] => f.G
  | f :: g :: gs => f.G × G (g :: gs)
```

def OfFactors.𝔤 [source]

The Lie algebra of a list of factors.

```
def 𝔤 : List FactorSpec → Type
  | [] => Unit
  | [f] => f.𝔤
  | f :: g :: gs => f.𝔤 × 𝔤 (g :: gs)
```

def OfFactors.G₀ [source]

The group of a list of factors.

```
def G₀ : List FactorSpec → Type
  | [] => Unit
  | [f] => f.G₀
  | f :: g :: gs => f.G₀ × G₀ (g :: gs)
```

def OfFactors.𝔤J [source]

The jets of the Lie algebra of a list of factors.

```
def 𝔤J : List FactorSpec → Type
  | [] => Unit
  | [f] => f.𝔤J
  | f :: g :: gs => f.𝔤J × 𝔤J (g :: gs)
```

def OfFactors.instGroupG [source]

The group structure on the jets of a list of factors, by recursion on the list.

```
@[instance_reducible]
noncomputable def instGroupG : (Γ : List FactorSpec) → Group (G Γ)
  | [] => inferInstanceAs (Group Unit)
  | [f] => inferInstanceAs (Group f.G)
  | f :: g :: gs =>
    letI
```

instance Group (G Γ) [source]

```
noncomputable instance (Γ : List FactorSpec) : Group (G Γ)
```

def OfFactors.instGroupG₀ [source]

The group structure on a list of factors, by recursion on the list.

```
@[instance_reducible]
noncomputable def instGroupG₀ : (Γ : List FactorSpec) → Group (G₀ Γ)
  | [] => inferInstanceAs (Group Unit)
  | [f] => inferInstanceAs (Group f.G₀)
  | f :: g :: gs =>
    letI
```

instance Group (G₀ Γ) [source]

```
noncomputable instance (Γ : List FactorSpec) : Group (G₀ Γ)
```

def OfFactors.instLieRing𝔤 [source]

The Lie ring structure on the Lie algebra of a list of factors, by recursion.

```
@[instance_reducible]
noncomputable def instLieRing𝔤 : (Γ : List FactorSpec) → LieRing (𝔤 Γ)
  | [] => inferInstanceAs (LieRing Unit)
  | [f] => inferInstanceAs (LieRing f.𝔤)
  | f :: g :: gs =>
    letI
```

instance LieRing (𝔤 Γ) [source]

```
noncomputable instance (Γ : List FactorSpec) : LieRing (𝔤 Γ)
```

def OfFactors.instLieAlgebra𝔤 [source]

The real Lie algebra structure on the Lie algebra of a list of factors, by recursion.

```
@[instance_reducible]
noncomputable def instLieAlgebra𝔤 : (Γ : List FactorSpec) → LieAlgebra ℝ (𝔤 Γ)
  | [] => inferInstanceAs (LieAlgebra ℝ Unit)
  | [f] => inferInstanceAs (LieAlgebra ℝ f.𝔤)
  | f :: g :: gs =>
    letI
```

instance LieAlgebra ℝ (𝔤 Γ) [source]

```
noncomputable instance (Γ : List FactorSpec) : LieAlgebra ℝ (𝔤 Γ)
```

def OfFactors.instLieRing𝔤J [source]

The Lie ring structure on the jets of the Lie algebra of a list of factors, by recursion.

```
@[instance_reducible]
noncomputable def instLieRing𝔤J : (Γ : List FactorSpec) → LieRing (𝔤J Γ)
  | [] => inferInstanceAs (LieRing Unit)
  | [f] => inferInstanceAs (LieRing f.𝔤J)
  | f :: g :: gs =>
    letI
```

instance LieRing (𝔤J Γ) [source]

```
noncomputable instance (Γ : List FactorSpec) : LieRing (𝔤J Γ)
```

def OfFactors.instLieAlgebra𝔤J [source]

The real Lie algebra structure on the jets of the Lie algebra of a list of factors.

```
@[instance_reducible]
noncomputable def instLieAlgebragJ : (Γ : List FactorSpec) → LieAlgebra ℝ (gJ Γ)
  | [] => inferInstanceAs (LieAlgebra ℝ Unit)
  | [f] => inferInstanceAs (LieAlgebra ℝ f.gJ)
  | f :: g :: gs =>
    letI
```

instance LieAlgebra ℝ (gJ Γ) [source]

```
noncomputable instance (Γ : List FactorSpec) : LieAlgebra ℝ (gJ Γ)
```

instance OfFactors.instFinitegₗ [source]

```
instance instFinitegₗ : (Γ : List FactorSpec) → Module.Finite ℝ (gₗ Γ)
  | [] => inferInstanceAs (Module.Finite ℝ Unit)
  | [f] => inferInstanceAs (Module.Finite ℝ f.gₗ)
  | f :: g :: gs =>
    letI
```

def trivial [source]

The trivial local gauge data, of the empty list of factors.

```
noncomputable def trivial : LocalGaugeData Unit Unit Unit Unit where
```

instance instFaithfulTrivial [source]

```
instance instFaithfulTrivial : trivial.Faithful where
```

instance instFreeTrivial [source]

```
instance instFreeTrivial : trivial.Free where
```

def ofFactors [source]

***The local gauge data of a list of factors**: the product, in the order of the list, of the local gauge data of the factors.*

```
noncomputable def ofFactors : (Γ : List FactorSpec) → LocalGaugeData (G₀ Γ) (gₗ Γ) (G Γ) (gJ Γ)
  | [] => trivial
  | [f] => f.data
  | f :: g :: gs => f.data.prod (ofFactors (g :: gs))
```

lemma ofFactors_nil [source]

```
@[simp]
lemma ofFactors_nil : ofFactors [] = trivial
```

lemma ofFactors_singleton [source]

```
@[simp]
lemma ofFactors_singleton (f : FactorSpec) : ofFactors [f] = f.data
```

lemma ofFactors_cons_cons [source]

```
@[simp]
lemma ofFactors_cons_cons (f g : FactorSpec) (gs : List FactorSpec) :
    ofFactors (f :: g :: gs) = f.data.prod (ofFactors (g :: gs))
```

instance instFaithfulOfFactors [source]

The local gauge data of a list of factors is faithful.

```
noncomputable instance instFaithfulOfFactors : (Γ : List FactorSpec) → (ofFactors Γ).Faithful
  | [] => inferInstanceAs trivial.Faithful
  | [f] => inferInstanceAs f.data.Faithful
  | f :: g :: gs =>
    letI
```

instance instFreeOfFactors [source]

The local gauge data of a list of factors is free.

```
noncomputable instance instFreeOfFactors : (Γ : List FactorSpec) → (ofFactors Γ).Free
  | [] => inferInstanceAs trivial.Free
  | [f] => inferInstanceAs f.data.Free
  | f :: g :: gs =>
    letI
```

abbrev factors [source]

***The canonical factors** of the local gauge data of a list of factors.*

```
noncomputable abbrev Factors.factors : (Γ : List FactorSpec) → Factors (ofFactors Γ)
  | [] => []
  | [f] => [f.factor]
  | f :: g :: gs =>
    f.factor.comap Hom.fst :: (Factors.factors (g :: gs)).comap Hom.snd
```

2.223 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.Prod

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/Prod.lean on GitHub.

def prod [source]

***The product of two local gauge data**: every structure map acts componentwise.*

```
noncomputable def prod : LocalGaugeData (G₀₁ × G₀₂) (𝔤₁ × 𝔤₂) (G₁ × G₂) (𝔤J₁ × 𝔤J₂) where
```

lemma prod_eval [source]

```
@[simp]
lemma prod_eval (U : G₁ × G₂) : (j₁.prod j₂).eval U = (j₁.eval U.1, j₂.eval U.2)
```

lemma prod_ofConstant [source]

```
@[simp]
lemma prod_ofConstant (g : G₀₁ × G₀₂) :
```

```
    (j₁.prod j₂).ofConstant g = (j₁.ofConstant g.1, j₂.ofConstant g.2)
```

lemma prod_evalLie [source]

```
@[simp]
lemma prod_evalLie (a : 𝔤J₁ × 𝔤J₂) : (j₁.prod j₂).evalLie a = (j₁.evalLie a.1, j₂.evalLie a.2)
```

lemma prod_ofConstantLie [source]

```
@[simp]
lemma prod_ofConstantLie (a : 𝔤₁ × 𝔤₂) :
    (j₁.prod j₂).ofConstantLie a = (j₁.ofConstantLie a.1, j₂.ofConstantLie a.2)
```

lemma prod_deriv [source]

```
@[simp]
lemma prod_deriv (μ : Fin 1 ⊕ Fin 3) (a : 𝔤J₁ × 𝔤J₂) :
    (j₁.prod j₂).deriv μ a = (j₁.deriv μ a.1, j₂.deriv μ a.2)
```

lemma prod_adjointJ [source]

```
@[simp]
lemma prod_adjointJ (U : G₁ × G₂) (a : 𝔤J₁ × 𝔤J₂) :
    (j₁.prod j₂).adjointJ U a = (j₁.adjointJ U.1 a.1, j₂.adjointJ U.2 a.2)
```

lemma prod_maurerCartan [source]

```
@[simp]
lemma prod_maurerCartan (U : G₁ × G₂) (μ : Fin 1 ⊕ Fin 3) :
    (j₁.prod j₂).maurerCartan U μ = (j₁.maurerCartan U.1 μ, j₂.maurerCartan U.2 μ)
```

lemma prod_iteratedDeriv [source]

```
@[simp]
lemma prod_iteratedDeriv (s : Multiset (Fin 1 ⊕ Fin 3)) (a : 𝔤J₁ × 𝔤J₂) :
    (j₁.prod j₂).iteratedDeriv s a = (j₁.iteratedDeriv s a.1, j₂.iteratedDeriv s a.2)
```

def fst [source]

The first projection, as a morphism of local gauge data.

```
noncomputable def Hom.fst : Hom (j₁.prod j₂) j₁ where
```

def snd [source]

The second projection, as a morphism of local gauge data.

```
noncomputable def Hom.snd : Hom (j₁.prod j₂) j₂ where
```

instance instFaithfulProd [source]

The product of two faithful packages is faithful.

```
instance instFaithfulProd [j₁.Faithful] [j₂.Faithful] : (j₁.prod j₂).Faithful where
```

lemma prod_mulCoord [source]

```
@[simp]
lemma prod_mulCoord (μ : Fin 1 ⊕ Fin 3) (a : 𝔤J₁ × 𝔤J₂) :
    (j₁.prod j₂).mulCoord μ a = (j₁.mulCoord μ a.1, j₂.mulCoord μ a.2)
```

lemma prod_radial [source]

The radial component of the Maurer–Cartan form of a product jet is the pair of the radial components of its factors.

```
@[simp]
lemma prod_radial (U : G₁ × G₂) :
    (j₁.prod j₂).radial U = (j₁.radial U.1, j₂.radial U.2)
```

lemma mem_prod_truncationKer_iff [source]

A product jet lies in the `n`-th truncation kernel exactly when both of its factors do.

```
lemma mem_prod_truncationKer_iff (n : ℕ) (U : G₁ × G₂) :
    U ∈ (j₁.prod j₂).truncationKer n ↔
      U.1 ∈ j₁.truncationKer n ∧ U.2 ∈ j₂.truncationKer n
```

instance instFreeProd [source]

The product of two free packages is free: Taylor data and radial components are realized factor by factor and paired.

```
instance instFreeProd [j₁.Free] [j₂.Free] : (j₁.prod j₂).Free where
```

2.224 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Adjoint

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/SU/Adjoint.lean on GitHub.

def adjBasis [source]

The generalized Gell-Mann basis `1 ⊗ λ_a` of the complexified Lie algebra `ℂ ⊗[ℝ] su(N)`.

```
noncomputable def adjBasis : Basis (GellMann.Index N) ℂ (SUAlgebraComplexified N)
```

lemma adjBasis_apply [source]

```
lemma adjBasis_apply (a : GellMann.Index N) :
    adjBasis N a = (1 : ℂ) ⊗ₜ[ℝ] GellMann.realBasis a
```

lemma val_inv_mul_val [source]

```
lemma val_inv_mul_val (g : SU N) : (g⁻¹).1 * g.1 = 1
```

lemma val_inv [source]

```
lemma val_inv (g : SU N) : (g⁻¹).1 = star g.1
```

def adjRep [source]

The adjoint representation on the complexified Lie algebra: the complexification of the adjoint action `X ↦ g X g⁻¹` on `su(N)`.

```
noncomputable def adjRep : Representation ℂ (SU N) (SUAlgebraComplexified N) where
```

def adjMat [source]

The matrix of an element of the complexified Lie algebra, `z ⊗ X ↦ z X`: a traceless complex matrix.

```
noncomputable def adjMat : SUAlgebraComplexified N →ₗ[ℂ] Matrix (Fin N) (Fin N) ℂ
```

lemma adjMat_tmul [source]

```
@[simp]
lemma adjMat_tmul (z : ℂ) (X : SUAlgebra N) : adjMat N (z ⊗ₜ X) = z • X.1
```

lemma adjMat_adjBasis [source]

```
@[simp]
lemma adjMat_adjBasis (a : GellMann.Index N) : adjMat N (adjBasis N a) = GellMann.matrix a
```

lemma trace_adjMat [source]

The matrix of an element of the complexified Lie algebra is traceless.

```
lemma trace_adjMat (A : SUAlgebraComplexified N) : (adjMat N A).trace = 0
```

lemma adjMat_adjRep [source]

The adjoint representation moves the matrix by conjugation.

```
@[simp]
lemma adjMat_adjRep (g : SU N) (A : SUAlgebraComplexified N) :
    adjMat N (adjRep N g A) = g.1 * adjMat N A * (g⁻¹).1
```

lemma adjBasis_repr_apply [source]

The coordinates in the Gell-Mann basis are read off by the trace form, `A = ∑ (tr (λ_a A) / 2) λ_a`.

```
lemma adjBasis_repr_apply (A : SUAlgebraComplexified N) (a : GellMann.Index N) :
    (adjBasis N).repr A a = (GellMann.matrix a * adjMat N A).trace / 2
```

lemma adjMat_injective [source]

An element of the complexified Lie algebra is determined by its matrix.

```
lemma adjMat_injective : Function.Injective (adjMat N)
```

def ofTraceless [source]

The element of the complexified Lie algebra with a given traceless matrix.

```
noncomputable def ofTraceless (M : Matrix (Fin N) (Fin N) ℂ) : SUAlgebraComplexified N
```

lemma adjMat_ofTraceless [source]

Every traceless matrix is the matrix of an element of the complexified Lie algebra.

```
@[simp]
lemma adjMat_ofTraceless {M : Matrix (Fin N) (Fin N) ℂ} (hM : M.trace = 0) :
    adjMat N (ofTraceless M) = M
```

abbrev adjMatrix [source]

The matrix of the adjoint action of `g` in the Gell-Mann basis.

```
noncomputable abbrev adjMatrix (g : SU N) : Matrix (GellMann.Index N) (GellMann.Index N) ℂ
```

lemma adjMatrix_apply [source]

The entries of the matrix of the adjoint action, `tr (λ_a g λ_b g⁻¹) / 2`.

```
lemma adjMatrix_apply (g : SU N) (a b : GellMann.Index N) :
    adjMatrix g a b = (GellMann.matrix a * (g.1 * GellMann.matrix b * (g⁻¹).1)).trace / 2
```

lemma star_adjMatrix_apply [source]

The matrix of the adjoint action is real, the Gell-Mann matrices being hermitian.

```
lemma star_adjMatrix_apply (g : SU N) (a b : GellMann.Index N) :
    star (adjMatrix g a b) = adjMatrix g a b
```

lemma adjMatrix_inv [source]

The matrix of the adjoint action of `g⁻¹` is the transpose of that of `g`, by the cyclicity of the trace.

```
lemma adjMatrix_inv (g : SU N) : adjMatrix g⁻¹ = (adjMatrix g)ᵀ
```

lemma adjMatrix_inv_mul [source]

The matrix of the adjoint action of `g⁻¹` times that of `g` is the identity.

```
lemma adjMatrix_inv_mul (g : SU N) : adjMatrix g⁻¹ * adjMatrix g = 1
```

def traceForm [source]

The trace form `A ⊗ B ↦ tr (A B)` on the complexified Lie algebra.

```
noncomputable def traceForm : LinearMap.BilinForm ℂ (SUAlgebraComplexified N)
```

lemma traceForm_apply [source]

```
@[simp]
lemma traceForm_apply (A B : SUAlgebraComplexified N) :
    traceForm N A B = (adjMat N A * adjMat N B).trace
```

lemma traceForm_isSymm [source]

The trace form is symmetric.

```
lemma traceForm_isSymm : (traceForm N).IsSymm
```

lemma traceForm_adjBasis [source]

The trace form against a Gell-Mann basis vector is twice the coordinate.

```
lemma traceForm_adjBasis (A : SUAlgebraComplexified N) (a : GellMann.Index N) :
    traceForm N A (adjBasis N a) = 2 * (adjBasis N).repr A a
```

lemma traceForm_separatingLeft [source]

An element orthogonal to every element under the trace form vanishes: its coordinates are its trace pairings with the Gell-Mann basis.

```
lemma traceForm_separatingLeft (A : SUAlgebraComplexified N) (hA : ∀ B, traceForm N A B = 0) :
    A = 0
```

lemma traceForm_nondegenerate [source]

The trace form is nondegenerate on the complexified Lie algebra.

```
lemma traceForm_nondegenerate : (traceForm N).Nondegenerate
```

def adjContr [source]

The contraction of two adjoint indices, the trace form `A ⊗ B ↦ tr (A B)`.

```
noncomputable def adjContr : ((adjRep N).tprod (adjRep N)).IntertwiningMap
    (Representation.trivial ℂ (SU N) ℂ) where
```

lemma adjContr_flip_injective [source]

The trace form separates the complexified Lie algebra.

```
lemma adjContr_flip_injective :
    Function.Injective (TensorProduct.curry (adjContr N).toLinearMap).flip
```

2.225 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Algebra

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/SU/Algebra.lean on GitHub.

abbrev submodule [source]

The submodule of traceless hermitian matrices.

```
abbrev SUAlgebraOver.submodule (R : Type) [CommRing R] [StarRing R] [Algebra ℝ R]
    [StarModule ℝ R] (n : ℕ) : Submodule ℝ (Matrix (Fin n) (Fin n) R)
```

abbrev SUAlgebraOver [source]

***The Lie algebra `su(n)` over `R`**: traceless hermitian `n × n` matrices with entries in `R`, a real Lie algebra with bracket `i (a b − b a)`.*

```
abbrev SUAlgebraOver (R : Type) [CommRing R] [StarRing R] [Algebra ℝ R] [StarModule ℝ R]
    (n : ℕ) : Type
```

lemma mem_iff [source]

```
lemma mem_iff (A : Matrix (Fin n) (Fin n) R) :
    A ∈ submodule R n ↔ star A = A ∧ A.trace = 0
```

def ofMatrix [source]

An element from a traceless hermitian matrix.

```
def ofMatrix (A : Matrix (Fin n) (Fin n) R) (hA : star A = A) (hT : A.trace = 0) :
    SUAlgebraOver R n
```

lemma ofMatrix_val [source]

```
@[simp]
lemma ofMatrix_val (A : Matrix (Fin n) (Fin n) R) (hA : star A = A) (hT : A.trace = 0) :
    (ofMatrix A hA hT).1 = A
```

lemma star_val [source]

```
lemma star_val (a : SUAlgebraOver R n) : star a.1 = a.1
```

lemma trace_val [source]

```
lemma trace_val (a : SUAlgebraOver R n) : a.1.trace = 0
```

lemma ext [source]

```
@[ext]
lemma ext {a b : SUAlgebraOver R n} (h : a.1 = b.1) : a = b
```

def conj [source]

***The conjugation representation** of the unitary group on `su(n)`: `a ↦ U a U†`.*

```
noncomputable def conj : Representation ℝ (unitaryGroup (Fin n) R) (SUAlgebraOver R n) where
```

lemma conj_apply_val [source]

```
@[simp]
lemma conj_apply_val (U : unitaryGroup (Fin n) R) (a : SUAlgebraOver R n) :
    (conj U a).1 = U.1 * a.1 * star U.1
```

instance Bracket (SUAlgebraOver R n) (SUAlgebraOver R n) [source]

The bracket `i (a b − b a)`.

```
noncomputable instance : Bracket (SUAlgebraOver R n) (SUAlgebraOver R n) where
```

lemma bracket_val [source]

```
@[simp]
lemma bracket_val (a b : SUAlgebraOver R n) :
    ⁅a, b⁆.1 = Complex.I • (a.1 * b.1 - b.1 * a.1)
```

instance LieRing (SUAlgebraOver R n) [source]

```
noncomputable instance : LieRing (SUAlgebraOver R n) where
```

instance LieAlgebra ℝ (SUAlgebraOver R n) [source]

```
noncomputable instance : LieAlgebra ℝ (SUAlgebraOver R n) where
```

abbrev SUAlgebraComplexified [source]

***The complexified Lie algebra `su(n)_ℂ = ℂ ⊗[ℝ] su(n)`**, a complex vector space.*

```
abbrev SUAlgebraComplexified (n : ℕ) : Type
```

2.226 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Basic

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/SU/Basic.lean on GitHub.

abbrev SU [source]

The gauge group `SU(n)`.

```
abbrev SU (n : ℕ) : Type
```

abbrev JetSU [source]

Jets of `SU(n)` gauge transformations: special unitary matrices of formal power series.

```
abbrev JetSU (n : ℕ) : Type
```

abbrev SUAlgebra [source]

The Lie algebra `su(n)`: traceless hermitian matrices.

```
abbrev SUAlgebra (n : ℕ) : Type
```

abbrev JetSUAlgebra [source]

Jets of the Lie algebra `su(n)`: traceless hermitian matrices of formal power series.

```
abbrev JetSUAlgebra (n : ℕ) : Type
```

lemma val_mul_star [source]

```
lemma val_mul_star (U : JetSU n) : U.1 * star U.1 = 1
```

lemma star_mul_val [source]

```
lemma star_mul_val (U : JetSU n) : star U.1 * U.1 = 1
```

lemma det_val [source]

```
lemma det_val (U : JetSU n) : U.1.det = 1
```

lemma star_val_eq_adjugate [source]

For a special unitary matrix, the conjugate transpose is the adjugate.

```
lemma star_val_eq_adjugate (U : JetSU n) : star U.1 = U.1.adjugate
```

def eval [source]

Evaluation of a jet of an `SU(n)` gauge transformation at the base point: the entrywise constant coefficient.

```
noncomputable def eval : JetSU n →* SU n where
```

lemma eval_val [source]

```
@[simp]
lemma eval_val (U : JetSU n) : (eval U).1 =
    (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix U.1
```

def ofConstant [source]

The jet of a constant `SU(n)` gauge transformation: the entrywise inclusion of constants.

```
noncomputable def ofConstant : SU n →* JetSU n where
```

lemma ofConstant_val [source]

```
@[simp]
lemma ofConstant_val (u : SU n) : (ofConstant u).1 =
    (C : ℂ →+* SpaceTimeAlgebra).mapMatrix u.1
```

lemma trace_mcMatrix [source]

The Maurer–Cartan matrix `i (∂_μ U) U†` is traceless, by Jacobi's formula and `det U = 1`.

```
lemma trace_mcMatrix (μ : Fin 1 ⊕ Fin 3) (U : JetSU n) :
    (Complex.I • (U.1.map (pderiv μ) * star U.1)).trace = 0
```

def deriv [source]

The formal derivative in the direction `μ`, entrywise.

```
noncomputable def deriv (μ : Fin 1 ⊕ Fin 3) : JetSUAlgebra n →ₗ[ℝ] JetSUAlgebra n where
```

lemma deriv_val [source]

```
@[simp]
lemma deriv_val (μ : Fin 1 ⊕ Fin 3) (a : JetSUAlgebra n) :
    (deriv μ a).1 = a.1.map (pderiv μ)
```

def coord [source]

Multiplication by the coordinate `x_μ`, entrywise.

```
noncomputable def coord (μ : Fin 1 ⊕ Fin 3) : JetSUAlgebra n →ₗ[ℝ] JetSUAlgebra n where
```

lemma coord_val [source]

```
@[simp]
lemma coord_val (μ : Fin 1 ⊕ Fin 3) (a : JetSUAlgebra n) :
    (coord μ a).1 = (X μ : SpaceTimeAlgebra) • a.1
```

lemma star_mapMatrix_constantCoeff [source]

```
lemma star_mapMatrix_constantCoeff (a : JetSUAlgebra n) :
    star ((constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix a.1)
      = (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix a.1
```

lemma trace_mapMatrix_constantCoeff [source]

```
lemma trace_mapMatrix_constantCoeff (a : JetSUAlgebra n) :
    ((constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix a.1).trace = 0
```

def evalLie [source]

Evaluation at the base point: the entrywise constant coefficient.

```
noncomputable def evalLie : JetSUAlgebra n →ₗ[ℝ] SUAlgebra n where
```

lemma evalLie_val [source]

```
@[simp]
lemma evalLie_val (a : JetSUAlgebra n) :
    (evalLie a).1 = (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix a.1
```

lemma star_mapMatrix_C [source]

```
lemma star_mapMatrix_C (a : SUAlgebra n) :
    star ((C : ℂ →+* SpaceTimeAlgebra).mapMatrix a.1) =
        (C : ℂ →+* SpaceTimeAlgebra).mapMatrix a.1
```

lemma trace_mapMatrix_C [source]

```
lemma trace_mapMatrix_C (a : SUAlgebra n) :
    ((C : ℂ →+* SpaceTimeAlgebra).mapMatrix a.1).trace = 0
```

def ofConstantLie [source]

A constant as a jet: the entrywise constant power series.

```
noncomputable def ofConstantLie : SUAlgebra n →ₗ[ℝ] JetSUAlgebra n where
```

lemma ofConstantLie_val [source]

```
@[simp]
lemma ofConstantLie_val (a : SUAlgebra n) :
    (ofConstantLie a).1 = (C : ℂ →+* SpaceTimeAlgebra).mapMatrix a.1
```

def adjoint [source]

The adjoint action `a ↦ U a U†` of the jets of `SU(n)` on the jets of `su(n)`.

```
noncomputable def adjoint : Representation ℝ (JetSU n) (JetSUAlgebra n)
```

lemma adjoint_val [source]

```
@[simp]
lemma adjoint_val (U : JetSU n) (a : JetSUAlgebra n) :
    (adjoint U a).1 = U.1 * a.1 * star U.1
```

def adjointValue [source]

The adjoint action `a ↦ U a U†` of `SU(n)` on `su(n)`.

```
noncomputable def adjointValue : Representation ℝ (SU n) (SUAlgebra n)
```

lemma adjointValue_val [source]

```
@[simp]
lemma adjointValue_val (U : SU n) (a : SUAlgebra n) :
    (adjointValue U a).1 = U.1 * a.1 * star U.1
```

def mc [source]

The Maurer–Cartan form `i (∂_μ U) U†` of an `SU(n)` gauge jet.

```
noncomputable def mc (U : JetSU n) (μ : Fin 1 ⊕ Fin 3) : JetSUAlgebra n
```

lemma mc_val [source]

```
@[simp]
lemma mc_val (U : JetSU n) (μ : Fin 1 ⊕ Fin 3) :
    (mc U μ).1 = Complex.I • (U.1.map (pderiv μ) * star U.1)
```

def suMatrixJets [source]

***The presentation of `SU(n)` by matrices of jets**: the carriers are subtypes of matrices and every structure map is the matrix one.*

```
noncomputable def suMatrixJets (n : ℕ) :
    MatrixJets (Fin n) (SU n) (SUAlgebra n) (JetSU n) (JetSUAlgebra n) where
```

def su [source]

***The local gauge data of `SU(n)`**: special unitary jets, traceless hermitian jets with the bracket `i (a b - b a)` and the conjugation action, and the Maurer–Cartan form `i (∂_μ U) U⁻¹`.*

```
noncomputable def su (n : ℕ) : LocalGaugeData (SU n) (SUAlgebra n) (JetSU n) (JetSUAlgebra n)
```

def suFactor [source]

The canonical `SU(n)` factor of the local gauge data of `SU(n)`.

```
noncomputable def suFactor (n : ℕ) : SUFactor (su n) (Fin n)
```

instance instFaithfulSU [source]

The local gauge data of `SU(n)` is faithful.

```
instance instFaithfulSU : (su n).Faithful
```

instance instFreeSU [source]

The local gauge data of `SU(n)` is free. Traceless hermitian Taylor data give a traceless hermitian matrix of jets, and the unitary Euler transport of a traceless hermitian jet has unit determinant by Jacobi's formula.

```
instance instFreeSU : (su n).Free
```

2.227 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.GellMann

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/SU/GellMann.lean on GitHub.

abbrev Index [source]

The labels of the generalized Gell-Mann matrices: a pair `j < k` for each symmetric and each antisymmetric matrix, and `l : Fin (N - 1)` for the diagonal matrix of size `n = l + 1`.

```
abbrev Index (N : ℕ) : Type
```

def diagProfile [source]

The diagonal profile of size `n = l + 1`: `1` on the first `n` entries, `-n` on the next, and `0` beyond.

```
def diagProfile (l : Fin (N - 1)) (m : Fin N) : ℂ
```

def diagNorm [source]

The normalization `√(2 / (n (n + 1)))` of the diagonal matrix of size `n = l + 1`.

```
noncomputable def diagNorm (l : Fin (N - 1)) : ℂ
```

def matrix [source]

The generalized Gell-Mann matrices.

```
noncomputable def matrix : Index N → Matrix (Fin N) (Fin N) ℂ
  | .inl p => single p.1.1 p.1.2 1 + single p.1.2 p.1.1 1
  | .inr (.inl p) => single p.1.1 p.1.2 (-Complex.I) + single p.1.2 p.1.1 Complex.I
  | .inr (.inr l) => diagNorm l • diagonal (diagProfile l)
```

def profile [source]

The profile as a function of the position, `1` below `n`, `-n` at `n`, `0` above.

```
private def profile (n m : ℕ) : ℂ
```

lemma sum_range_profile [source]

```
private lemma sum_range_profile {n : ℕ} :
    ∀ M, n < M → ∑ m ∈ Finset.range M, profile n m = 0
```

lemma sum_diagProfile [source]

```
lemma sum_diagProfile (l : Fin (N - 1)) : ∑ m, diagProfile l m = 0
```

lemma profile_mul_of_lt [source]

```
private lemma profile_mul_of_lt {n n' : ℕ} (h : n < n') (m : ℕ) :
    profile n m * profile n' m = profile n m
```

lemma profile_mul_self [source]

```
private lemma profile_mul_self (n m : ℕ) :
    profile n m * profile n m = (if m < n then 1 else 0) + (if m = n then (n : ℂ) ^ 2 else 0)
```

lemma sum_range_indicator_lt [source]

```
private lemma sum_range_indicator_lt {n : ℕ} :
    ∀ M, n ≤ M → ∑ m ∈ Finset.range M, (if m < n then (1 : ℂ) else 0) = n
```

lemma sum_diagProfile_mul [source]

The diagonal profiles are orthogonal, with ∑ d_l (m)² = n (n + 1) for n = l + 1.

```
lemma sum_diagProfile_mul (l l' : Fin (N - 1)) :
    ∑ m, diagProfile l m * diagProfile l' m
      = if l = l' then ((l.val + 1 : ℕ) : ℂ) * ((l.val + 2 : ℕ) : ℂ) else 0
```

lemma diagNorm_mul_self [source]

The square of the normalization of the diagonal matrix of size n = l + 1 is 2 / (n (n + 1)).

```
lemma diagNorm_mul_self (l : Fin (N - 1)) :
    diagNorm l * diagNorm l * (((l.val + 1 : ℕ) : ℂ) * ((l.val + 2 : ℕ) : ℂ)) = 2
```

lemma trace_matrix_inl_mul [source]

The trace of a symmetric Gell-Mann matrix against X.

```
lemma trace_matrix_inl_mul (p : {p : Fin N × Fin N // p.1 < p.2}) (X : Matrix (Fin N) (Fin N)
    ℂ) :
    (matrix (.inl p) * X).trace = X p.1.2 p.1.1 + X p.1.1 p.1.2
```

lemma trace_matrix_inr_inl_mul [source]

The trace of an antisymmetric Gell-Mann matrix against X.

```
lemma trace_matrix_inr_inl_mul (p : {p : Fin N × Fin N // p.1 < p.2})
    (X : Matrix (Fin N) (Fin N) ℂ) :
    (matrix (.inr (.inl p)) * X).trace
      = -Complex.I * X p.1.2 p.1.1 + Complex.I * X p.1.1 p.1.2
```

lemma trace_matrix_inr_inr_mul [source]

The trace of a diagonal Gell-Mann matrix against X.

```
lemma trace_matrix_inr_inr_mul (l : Fin (N - 1)) (X : Matrix (Fin N) (Fin N) ℂ) :
    (matrix (.inr (.inr l)) * X).trace = diagNorm l * ∑ m, diagProfile l m * X m m
```

lemma matrix_inl_apply [source]

The entries of a symmetric Gell-Mann matrix.

```
lemma matrix_inl_apply (p : {p : Fin N × Fin N // p.1 < p.2}) (m n : Fin N) :
    matrix (.inl p) m n
      = (if p.1.1 = m ∧ p.1.2 = n then 1 else 0) + (if p.1.2 = m ∧ p.1.1 = n then 1 else 0)
```

lemma matrix_inr_inl_apply [source]

The entries of an antisymmetric Gell-Mann matrix.

```
lemma matrix_inr_inl_apply (p : {p : Fin N × Fin N // p.1 < p.2}) (m n : Fin N) :
    matrix (.inr (.inl p)) m n = (if p.1.1 = m ∧ p.1.2 = n then -Complex.I else 0)
      + (if p.1.2 = m ∧ p.1.1 = n then Complex.I else 0)
```

lemma matrix_inr_inr_apply [source]

The entries of a diagonal Gell-Mann matrix.

```
lemma matrix_inr_inr_apply (l : Fin (N - 1)) (m n : Fin N) :
    matrix (.inr (.inr l)) m n = if m = n then diagNorm l * diagProfile l m else 0
```

lemma not_cross [source]

Two ordered pairs `j < k` and `j' < k'` never match crosswise.

```
private lemma not_cross {j k j' k' : Fin N} (h : j < k) (h' : j' < k') :
    ¬ (j' = k ∧ k' = j) ∧ ¬ (k' = j ∧ j' = k)
```

lemma trace_matrix_inl_mul_matrix [source]

The symmetric Gell-Mann matrices are orthogonal to all others and have norm `2`.

```
lemma trace_matrix_inl_mul_matrix (p : {p : Fin N × Fin N // p.1 < p.2}) (b : Index N) :
    (matrix (.inl p) * matrix b).trace = if .inl p = b then 2 else 0
```

lemma trace_matrix_inr_inl_mul_matrix [source]

The antisymmetric Gell-Mann matrices are orthogonal to all others and have norm `2`.

```
lemma trace_matrix_inr_inl_mul_matrix (p : {p : Fin N × Fin N // p.1 < p.2}) (b : Index N) :
    (matrix (.inr (.inl p)) * matrix b).trace = if .inr (.inl p) = b then 2 else 0
```

lemma trace_matrix_inr_inr_mul_matrix [source]

The diagonal Gell-Mann matrices are orthogonal to all others and have norm `2`.

```
lemma trace_matrix_inr_inr_mul_matrix (l : Fin (N - 1)) (b : Index N) :
    (matrix (.inr (.inr l)) * matrix b).trace = if .inr (.inr l) = b then 2 else 0
```

lemma trace_matrix_mul_matrix [source]

The generalized Gell-Mann matrices are orthogonal under the trace form: `tr (λ_a λ_b) = 2 δ_ab`.

```
lemma trace_matrix_mul_matrix (a b : Index N) :
    (matrix a * matrix b).trace = if a = b then 2 else 0
```

lemma trace_matrix [source]

The generalized Gell-Mann matrices are traceless.

```
lemma trace_matrix (a : Index N) : (matrix a).trace = 0
```

lemma conjTranspose_matrix [source]

The generalized Gell-Mann matrices are hermitian.

```
lemma conjTranspose_matrix (a : Index N) : (matrix a)^H = matrix a
```

def traceless [source]

The generalized Gell-Mann matrices as traceless matrices.

```
noncomputable def traceless (a : Index N) :
    ↥(LinearMap.ker (Matrix.traceLinearMap (Fin N) ℂ ℂ))
```

lemma linearIndependent_traceless [source]

The generalized Gell-Mann matrices are linearly independent, by orthogonality.

```
lemma linearIndependent_traceless : LinearIndependent ℂ (traceless (N := N))
```

lemma two_mul_card_lt_add [source]

There are $N^2 - 1$ ordered pairs with $j < k$, counted twice, and $N - 1$ diagonal labels: $2 \#\{j < k\} + N = N^2$.

```
lemma two_mul_card_lt_add :
    2 * Fintype.card {p : Fin N × Fin N // p.1 < p.2} + N = N * N
```

lemma finrank_traceless [source]

The traceless $N \times N$ matrices have dimension $N^2 - 1$.

```
lemma finrank_traceless :
    finrank ℂ ↥(LinearMap.ker (Matrix.traceLinearMap (Fin N) ℂ ℂ)) = N * N - 1
```

lemma card_index [source]

```
lemma card_index : Fintype.card (Index N)
    = finrank ℂ ↥(LinearMap.ker (Matrix.traceLinearMap (Fin N) ℂ ℂ))
```

def basis [source]

***The generalized Gell-Mann basis** of the traceless complex $N \times N$ matrices.*

```
noncomputable def basis : Basis (Index N) ℂ ↥(LinearMap.ker (Matrix.traceLinearMap (Fin N) ℂ ℂ)
    )
```

lemma basis_apply_val [source]

```
@[simp]
lemma basis_apply_val (a : Index N) : (basis a).1 = matrix a
```

lemma basis_repr_apply [source]

The coordinates of a traceless matrix in the Gell-Mann basis are read off by the trace form, `x = ∑ (tr (λ_a x) / 2) λ_a`.

```
lemma basis_repr_apply (x : ↥(LinearMap.ker (Matrix.traceLinearMap (Fin N) ℂ ℂ))) (a : Index N)
     :
    basis.repr x a = (matrix a * x.1).trace / 2
```

def hermitian [source]

The generalized Gell-Mann matrices as elements of the real Lie algebra `su(N)`.

```
noncomputable def hermitian (a : Index N) : SUAlgebraOver ℂ N
```

lemma linearIndependent_hermitian [source]

The generalized Gell-Mann matrices are linearly independent over the reals.

```
lemma linearIndependent_hermitian : LinearIndependent ℝ (hermitian (N := N))
```

lemma trace_mul_ofReal_re [source]

The trace pairing of two hermitian matrices is real.

```
lemma trace_mul_ofReal_re {A H : Matrix (Fin N) (Fin N) ℂ} (hA : A^H = A) (hH : H^H = H) :
    (((A * H).trace.re : ℝ) : ℂ) = (A * H).trace
```

lemma eq_sum_re_trace_smul_hermitian [source]

A traceless hermitian matrix is the real combination `∑ (tr (λ_a H) / 2) λ_a` of the generalized Gell-Mann matrices.

```
lemma eq_sum_re_trace_smul_hermitian (H : SUAlgebraOver ℂ N) :
    ∑ a, ((matrix a * H.1).trace.re / 2) • hermitian a = H
```

def realBasis [source]

***The generalized Gell-Mann basis** of the real Lie algebra `su(N)`.*

```
noncomputable def realBasis : Basis (Index N) ℝ (SUAlgebraOver ℂ N)
```

lemma realBasis_apply_val [source]

```
@[simp]
lemma realBasis_apply_val (a : Index N) : (realBasis a).1 = matrix a
```

2.228 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Invariants.Adjoint

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/SU/Invariants/Adjoint.lean on GitHub.

lemma eq_zero_of_adjRep_eq_self [source]

An invariant element of the complexified Lie algebra is zero: its matrix commutes with every element of `SU(N)`, so it is a scalar, and its trace vanishes.

```
lemma eq_zero_of_adjRep_eq_self {A : SUAlgebraComplexified N} (hA : ∀ g : SU N, adjRep N g A =
    A) :
    A = 0
```

lemma eq_zero_of_invariant_adj [source]

An invariant tensor with one adjoint index is zero.

```
lemma eq_zero_of_invariant_adj (t : SuT[N, .adj]) (ht : ∀ g : SU N, g • t = t) : t = 0
```

def adjPairIdx [source]

The component indices of a tensor with two adjoint indices, as the pair of their Gell-Mann labels.

```
def adjPairIdx : ComponentIdx (S := suTensor N) ![.adj, .adj] ≃ (Fin 2 → GellMann.Index N)
    where
```

lemma basis_repr_smul_adjPair [source]

The components of `g • t` for a tensor with two adjoint indices: the matrix of components `C` is moved to `M C Mᵀ`.

```
lemma basis_repr_smul_adjPair (g : SU N) (t : SuT[N, .adj, .adj])
    (n : Fin 2 → GellMann.Index N) :
    (Tensor.basis _).repr (g • t) (adjPairIdx.symm n)
      = ∑ x, ∑ y, adjMatrix g (n 0) x * adjMatrix g (n 1) y
        * (Tensor.basis _).repr t (adjPairIdx.symm ![x, y])
```

def adjUnitTensor [source]

The unit tensor of the adjoint color, `∑ λ_a ⊗ λ_a / 2`, with two adjoint indices.

```
noncomputable def adjUnitTensor : SuT[N, .adj, .adj]
```

lemma adjUnitTensor_invariant [source]

The unit tensor of the adjoint color is invariant.

```
lemma adjUnitTensor_invariant (g : SU N) : g • adjUnitTensor N = adjUnitTensor N
```

lemma basis_repr_adjUnitTensor [source]

The components of the unit tensor of the adjoint color: `δ / 2`, the Gell-Mann matrices being orthogonal with `tr (λ_a λ_b) = 2 δ_ab`.

```
lemma basis_repr_adjUnitTensor (n : Fin 2 → GellMann.Index N) :
    (Tensor.basis _).repr (adjUnitTensor N) (adjPairIdx.symm n)
      = if n 0 = n 1 then 1 / 2 else 0
```

lemma exists_eq_smul_adjUnitTensor_of_invariant [source]

An invariant tensor with two adjoint indices is a multiple of the unit tensor of the adjoint color. Its matrix of components commutes with the matrices of the adjoint action, so it is the matrix of an endomorphism commuting with the adjoint action, which is scalar.

```
lemma exists_eq_smul_adjUnitTensor_of_invariant (t : SuT[N, .adj, .adj])
    (ht : ∀ g : SU N, g • t = t) : ∃ a : ℂ, t = a • adjUnitTensor N
```

lemma reducesInvariantsTo_bot_of_isEquivariant_adj [source]

For an equivariant map out of the tensors with one adjoint index, the invariants of the range reduce to `⊥`.

```
lemma reducesInvariantsTo_bot_of_isEquivariant_adj {f : SuT[N, .adj] →ₗ[ℂ] B}
    (hf : (suTensor N).IsEquivariant ![.adj] ρ f) :
    ReducesInvariantsTo (fun g : SU N => ρ g) (LinearMap.range f) ⊥
```

def invariantReductionToAdjUnitImage [source]

For an equivariant map `f` out of the tensors with two adjoint indices, the invariants of the range reduce to the span of the image of the unit tensor.

```
noncomputable def invariantReductionToAdjUnitImage {f : SuT[N, .adj, .adj] →ₗ[ℂ] B}
    (hf : (suTensor N).IsEquivariant ![.adj, .adj] ρ f) :
    InvariantReductionToSpan (fun g : SU N => ρ g) (LinearMap.range f)
```

def adjIdx [source]

The component indices of a tensor with one adjoint index, as its Gell-Mann label.

```
def adjIdx : ComponentIdx (S := suTensor N) ![.adj] ≃ GellMann.Index N where
```

def adjMap [source]

The linear map out of the tensors with one adjoint index sending the basis tensor with label `a` to `T a`.

```
noncomputable def adjMap (T : GellMann.Index N → B) : SuT[N, .adj] →ₗ[ℂ] B
```

lemma adjMap_smul_of_law [source]

A linear map moving a family indexed by one adjoint label by the matrix of the adjoint action of `g` intertwines the map of the family with the action of `g`.

```
lemma adjMap_smul_of_law (T : GellMann.Index N → B) {σ : B →ₗ[ℂ] B} (g : SU N)
    (hσ : ∀ a : GellMann.Index N, σ (T a) = ∑ b, adjMatrix g b a • T b) (t : SuT[N, .adj]) :
    σ (adjMap T t) = adjMap T (g • t)
```

lemma isEquivariant_adjMap [source]

The map of a family indexed by one adjoint label is equivariant when the family moves by the matrix of the adjoint action, the summed label first.

```
lemma isEquivariant_adjMap (T : GellMann.Index N → B)
    (hT : ∀ (g : SU N) (a : GellMann.Index N), ρ g (T a) = ∑ b, adjMatrix g b a • T b) :
    (suTensor N).IsEquivariant ![.adj] ρ (adjMap T)
```

lemma reducesInvariantsTo_bot_span_of_adjMap [source]

For a family indexed by one adjoint label whose map is equivariant, the invariants of the span of the family reduce to ⊥.

```
lemma reducesInvariantsTo_bot_span_of_adjMap {T : GellMann.Index N → B}
    (hT : (suTensor N).IsEquivariant ![.adj] ρ (adjMap T)) :
    ReducesInvariantsTo (fun g : SU N => ρ g) (Submodule.span ℂ (Set.range T)) ⊥
```

def adjPairMap [source]

The linear map out of the tensors with two adjoint indices sending the basis tensor with labels `n` to `T n`.

```
noncomputable def adjPairMap (T : (Fin 2 → GellMann.Index N) → B) :
    SuT[N, .adj, .adj] →ₗ[ℂ] B
```

lemma adjPairMap_two_smul_adjUnitTensor [source]

The map of a family sends twice the unit tensor to the trace contraction `∑ a, T ![a, a]`.

```
lemma adjPairMap_two_smul_adjUnitTensor (T : (Fin 2 → GellMann.Index N) → B) :
    adjPairMap T ((2 : ℂ) • adjUnitTensor N) = ∑ a, T ![a, a]
```

lemma adjPairMap_smul_of_law [source]

A linear map moving a family indexed by two adjoint labels by the matrix of the adjoint action of `g` on each label intertwines the map of the family with the action of `g`.

```
lemma adjPairMap_smul_of_law (T : (Fin 2 → GellMann.Index N) → B) {σ : B →ₗ[ℂ] B} (g : SU N)
    (hσ : ∀ l : Fin 2 → GellMann.Index N, σ (T l)
      = ∑ a : Fin 2 → GellMann.Index N, (adjMatrix g (a 0) (l 0) * adjMatrix g (a 1) (l 1)) • T
     a)
    (t : SuT[N, .adj, .adj]) :
    σ (adjPairMap T t) = adjPairMap T (g • t)
```

lemma isEquivariant_adjPairMap [source]

The map of a family indexed by two adjoint labels is equivariant when the family moves by the matrix of the adjoint action on each label, the summed label first.

```
lemma isEquivariant_adjPairMap (T : (Fin 2 → GellMann.Index N) → B)
    (hT : ∀ (g : SU N) (l : Fin 2 → GellMann.Index N), ρ g (T l)
      = ∑ a : Fin 2 → GellMann.Index N, (adjMatrix g (a 0) (l 0) * adjMatrix g (a 1) (l 1)) • T
     a) :
    (suTensor N).IsEquivariant ![.adj, .adj] ρ (adjPairMap T)
```

def invariantReductionToTrace [source]

For a family indexed by two adjoint labels whose map is equivariant, the invariants of the span of the family reduce to the span of the trace contraction ∑ `a, T ![a, a]`.

```
noncomputable def invariantReductionToTrace {T : (Fin 2 → GellMann.Index N) → B}
    (hT : (suTensor N).IsEquivariant ![.adj, .adj] ρ (adjPairMap T)) :
    InvariantReductionToSpan (fun g : SU N => ρ g) (Submodule.span ℂ (Set.range T))
```

2.229 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Invariants.Basic

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/SU/Invariants/Basic.lean on GitHub.

def planeInclusion [source]

The inclusion of the plane of the coordinates `p` and `q`: the `N × 2` matrix whose columns are the basis vectors `e_p` and `e_q`.

```
def planeInclusion (p q : Fin N) : Matrix (Fin N) (Fin 2) ℂ
```

lemma planeInclusion_apply [source]

```
@[simp]
lemma planeInclusion_apply (p q : Fin N) (x : Fin N) (i : Fin 2) :
    planeInclusion p q x i = if x = ![p, q] i then 1 else 0
```

lemma planeInclusion_transpose_mul [source]

The inclusion of the plane of two distinct coordinates is an isometry.

```
lemma planeInclusion_transpose_mul {p q : Fin N} (hpq : p ≠ q) :
    (planeInclusion p q)ᵀ * planeInclusion p q = 1
```

lemma conjTranspose_planeInclusion [source]

The inclusion of the plane has real entries, so its conjugate transpose is its transpose.

```
lemma conjTranspose_planeInclusion (p q : Fin N) :
    (planeInclusion p q)ᴴ = (planeInclusion p q)ᵀ
```

def planeMatrix [source]

The matrix acting as `U` on the plane of the coordinates `p` and `q` and as the identity on the other coordinates.

```
def planeMatrix (p q : Fin N) (U : Matrix (Fin 2) (Fin 2) ℂ) : Matrix (Fin N) (Fin N) ℂ
```

lemma planeMatrix_one [source]

The plane matrix of the identity is the identity.

```
@[simp]
lemma planeMatrix_one : planeMatrix p q 1 = 1
```

lemma planeMatrix_mul [source]

The plane matrices multiply as the matrices on the plane.

```
lemma planeMatrix_mul (hpq : p ≠ q) (U V : Matrix (Fin 2) (Fin 2) ℂ) :
    planeMatrix p q U * planeMatrix p q V = planeMatrix p q (U * V)
```

lemma conjTranspose_planeMatrix [source]

The conjugate transpose of a plane matrix is the plane matrix of the conjugate transpose.

```
lemma conjTranspose_planeMatrix (U : Matrix (Fin 2) (Fin 2) ℂ) :
    (planeMatrix p q U)ᴴ = planeMatrix p q Uᴴ
```

lemma det_planeMatrix [source]

A plane matrix has the determinant of the matrix on the plane.

```
lemma det_planeMatrix (hpq : p ≠ q) (U : Matrix (Fin 2) (Fin 2) ℂ) :
    (planeMatrix p q U).det = U.det
```

lemma planeMatrix_mul_planeInclusion [source]

A plane matrix moves the plane as the matrix on the plane does.

```
lemma planeMatrix_mul_planeInclusion (hpq : p ≠ q) (U : Matrix (Fin 2) (Fin 2) ℂ) :
    planeMatrix p q U * planeInclusion p q = planeInclusion p q * U
```

lemma transpose_planeInclusion_mul_planeMatrix [source]

The transpose of the inclusion of the plane intertwines a plane matrix with the matrix on the plane.

```
lemma transpose_planeInclusion_mul_planeMatrix (hpq : p ≠ q) (U : Matrix (Fin 2) (Fin 2) ℂ) :
    (planeInclusion p q)ᵀ * planeMatrix p q U = U * (planeInclusion p q)ᵀ
```

lemma planeMatrix_conj [source]

Conjugating a matrix supported on the plane by a plane matrix conjugates it on the plane.

```
lemma planeMatrix_conj (hpq : p ≠ q) (U M : Matrix (Fin 2) (Fin 2) ℂ) :
    planeMatrix p q U * (planeInclusion p q * M * (planeInclusion p q)ᵀ) * (planeMatrix p q U)
    ᴴ
```

```
    = planeInclusion p q * (U * M * U^H) * (planeInclusion p q)^T
```

lemma planeBlock_conj [source]

The block on the plane of a matrix conjugated by a plane matrix is the conjugate of its block.

```
lemma planeBlock_conj (hpq : p ≠ q) (U : Matrix (Fin 2) (Fin 2) ℂ) (C : Matrix (Fin N) (Fin N)
    ℂ) :
    (planeInclusion p q)^T * (planeMatrix p q U * C * (planeMatrix p q U)^H) * planeInclusion p
    q
      = U * ((planeInclusion p q)^T * C * planeInclusion p q) * U^H
```

lemma planeBlock_apply [source]

The entries of the block on the plane are the entries of the matrix at the two coordinates.

```
@[simp]
lemma planeBlock_apply (C : Matrix (Fin N) (Fin N) ℂ) (i j : Fin 2) :
    ((planeInclusion p q)^T * C * planeInclusion p q) i j = C (![p, q] i) (![p, q] j)
```

lemma planeMatrix_diagonal [source]

A plane matrix of a diagonal matrix is diagonal.

```
lemma planeMatrix_diagonal (hpq : p ≠ q) (a b : ℂ) :
    planeMatrix p q (diagonal ![a, b])
      = diagonal fun x => if x = p then a else if x = q then b else 1
```

def planeEmbed [source]

`SU(2)` acting on the plane of the coordinates `p` and `q` and trivially on the others: a homomorphism from `SU(2)` to `SU(N)`.

```
noncomputable def planeEmbed (p q : Fin N) (hpq : p ≠ q) : SU 2 →* SU N where
```

lemma planeEmbed_val [source]

```
@[simp]
lemma planeEmbed_val (hpq : p ≠ q) (U : SU 2) : (planeEmbed p q hpq U).1 = planeMatrix p q U.1
```

def diagPhase [source]

The diagonal element `diag (u, ū)` of `SU(2)`, for a complex number `u` of modulus one.

```
noncomputable def diagPhase (u : ℂ) (hu : u * star u = 1) : SU 2
```

lemma diagPhase_val [source]

```
@[simp]
lemma diagPhase_val (u : ℂ) (hu : u * star u = 1) :
    (diagPhase u hu).1 = diagonal ![u, star u]
```

def rotation [source]

The rotation `!![c, -s; s, c]` of `SU(2)`, for real `c` and `s` with `c² + s² = 1`.

```
noncomputable def rotation (c s : ℝ) (h : c ^ 2 + s ^ 2 = 1) : SU 2
```

lemma rotation_val [source]

```
@[simp]
lemma rotation_val (c s : ℝ) (h : c ^ 2 + s ^ 2 = 1) :
    (rotation c s h).1 = !![(c : ℂ), -s; s, c]
```

lemma planeBlock_eq_of_conj_eq [source]

A matrix fixed by conjugation by a plane matrix has its block on the plane fixed by conjugation by the matrix on the plane.

```
lemma planeBlock_eq_of_conj_eq (hpq : p ≠ q) {U : Matrix (Fin 2) (Fin 2) ℂ}
    {C : Matrix (Fin N) (Fin N) ℂ} (hC : planeMatrix p q U * C * (planeMatrix p q U)ᴴ = C) :
    U * ((planeInclusion p q)ᵀ * C * planeInclusion p q) * Uᴴ
      = (planeInclusion p q)ᵀ * C * planeInclusion p q
```

lemma conj_eq_of_commute [source]

A matrix commuting with every element of `SU(N)` is fixed by conjugation.

```
lemma conj_eq_of_commute {C : Matrix (Fin N) (Fin N) ℂ} (hC : ∀ g : SU N, g.1 * C = C * g.1)
    (g : SU N) : g.1 * C * g.1ᴴ = C
```

lemma apply_eq_of_commute [source]

A matrix commuting with every element of `SU(N)` is diagonal, with equal diagonal entries.

```
lemma apply_eq_of_commute {C : Matrix (Fin N) (Fin N) ℂ} (hC : ∀ g : SU N, g.1 * C = C * g.1)
    (hpq : p ≠ q) : C p q = 0 ∧ C p p = C q q
```

lemma eq_smul_one_of_commute [source]

A matrix commuting with every element of `SU(N)` is scalar.

```
lemma eq_smul_one_of_commute {C : Matrix (Fin N) (Fin N) ℂ}
    (hC : ∀ g : SU N, g.1 * C = C * g.1) : ∃ z : ℂ, C = z • 1
```

def offDiag [source]

The matrix unit at `(x, y)` off the diagonal, as an element of the complexified Lie algebra, and zero on the diagonal.

```
noncomputable def offDiag (x y : Fin N) : SUAlgebraComplexified N
```

def diagDiff [source]

The difference of the diagonal matrix units at `x` and at `y`, as an element of the complexified Lie algebra.

```
noncomputable def diagDiff (x y : Fin N) : SUAlgebraComplexified N
```

lemma adjMat_offDiag [source]

```
lemma adjMat_offDiag (x y : Fin N) :
    adjMat N (offDiag x y) = if x = y then 0 else single x y 1
```

lemma adjMat_offDiag_of_ne [source]

```
lemma adjMat_offDiag_of_ne {x y : Fin N} (h : x ≠ y) : adjMat N (offDiag x y) = single x y 1
```

lemma offDiag_self [source]

```
@[simp]
lemma offDiag_self (x : Fin N) : offDiag x x = 0
```

lemma adjMat_diagDiff [source]

```
@[simp]
lemma adjMat_diagDiff (x y : Fin N) : adjMat N (diagDiff x y) = single x x 1 - single y y 1
```

lemma diagDiff_self [source]

```
@[simp]
lemma diagDiff_self (x : Fin N) : diagDiff x x = 0
```

lemma planeInclusion_single [source]

The inclusion of the plane carries the matrix units of the plane to those of the coordinates.

```
lemma planeInclusion_single (p q : Fin N) (i j : Fin 2) :
    planeInclusion p q * single i j (1 : ℂ) * (planeInclusion p q)^T
      = single (![p, q] i) (![p, q] j) 1
```

lemma adjMat_adjRep_planeEmbed [source]

The adjoint action of an element of `SU(2)` embedded in a plane on a matrix supported on the plane.

```
lemma adjMat_adjRep_planeEmbed {p q : Fin N} (hpq : p ≠ q) (U : SU 2)
    (A : SUAlgebraComplexified N) (M : Matrix (Fin 2) (Fin 2) ℂ)
    (hA : adjMat N A = planeInclusion p q * M * (planeInclusion p q)^T) :
    adjMat N (adjRep N (planeEmbed p q hpq U) A)
      = planeInclusion p q * (U.1 * M * U.1^H) * (planeInclusion p q)^T
```

def sqrtI [source]

The phase `(1 + i) / √2`, a square root of `i` of modulus one.

```
noncomputable def sqrtI : ℂ
```

lemma sqrt_two_mul_sqrt_two [source]

```
lemma sqrt_two_mul_sqrt_two : (Real.sqrt 2 : ℂ) * (Real.sqrt 2 : ℂ) = 2
```

lemma sqrt_two_ne_zero [source]

```
lemma sqrt_two_ne_zero : (Real.sqrt 2 : ℂ) ≠ 0
```

lemma star_sqrtI [source]

```
lemma star_sqrtI : star sqrtI = (1 - Complex.I) / (Real.sqrt 2 : ℂ)
```

lemma sqrtI_mul_star [source]

```
lemma sqrtI_mul_star : sqrtI * star sqrtI = 1
```

lemma sqrtI_mul_self [source]

```
lemma sqrtI_mul_self : sqrtI * sqrtI = Complex.I
```

lemma star_sqrtI_mul_self [source]

```
lemma star_sqrtI_mul_self : star sqrtI * star sqrtI = -Complex.I
```

lemma sqrtI_ne_I [source]

```
lemma sqrtI_ne_I : sqrtI ≠ Complex.I
```

lemma star_sqrtI_ne_I [source]

```
lemma star_sqrtI_ne_I : star sqrtI ≠ Complex.I
```

lemma adjMat_adjRep_apply_of_diagonal [source]

The adjoint action of a diagonal matrix of `SU(N)` scales the entry at `(x, y)` by `d x * star (d y)`.

```
lemma adjMat_adjRep_apply_of_diagonal {g : SU N} {d : Fin N → ℂ} (hg : g.1 = diagonal d)
    (A : SUAlgebraComplexified N) (x y : Fin N) :
    adjMat N (adjRep N g A) x y = d x * adjMat N A x y * star (d y)
```

lemma map_offDiag_eq_smul [source]

For `p ≠ q`, an endomorphism commuting with the adjoint action sends the matrix unit at `(p, q)` to a multiple of itself. The phase `diag (ω, ω⁻)` in the plane of `p` and `q`, with `ω² = i`, multiplies that matrix unit by `i` and no other entry by `i`.

```
lemma map_offDiag_eq_smul (hL : ∀ (g : SU N) (A : SUAlgebraComplexified N),
    L (adjRep N g A) = adjRep N g (L A)) {p q : Fin N} (hpq : p ≠ q) :
    L (offDiag p q) = adjMat N (L (offDiag p q)) p q • offDiag p q
```

lemma adjRep_quarterTurn_offDiag [source]

The quarter turn in the plane of `p` and `q` sends the matrix unit at `(p, q)` to minus the one at `(q, p)`.

```
lemma adjRep_quarterTurn_offDiag {p q : Fin N} (hpq : p ≠ q) :
    adjRep N (planeEmbed p q hpq (rotation 0 1 (by norm_num))) (offDiag p q) = -offDiag q p
```

lemma coeff_offDiag_swap [source]

The coefficients of the matrix units at `(p, q)` and at `(q, p)` agree.

```
lemma coeff_offDiag_swap (hL : ∀ (g : SU N) (A : SUAlgebraComplexified N),
    L (adjRep N g A) = adjRep N g (L A)) {p q : Fin N} (hpq : p ≠ q) :
    adjMat N (L (offDiag q p)) q p = adjMat N (L (offDiag p q)) p q
```

def invSqrtTwo [source]

The cosine of an eighth of a turn, `1 / √2`.

```
noncomputable def invSqrtTwo : ℝ
```

lemma invSqrtTwo_sq_add [source]

```
lemma invSqrtTwo_sq_add : invSqrtTwo ^ 2 + invSqrtTwo ^ 2 = 1
```

lemma invSqrtTwo_mul_self [source]

```
lemma invSqrtTwo_mul_self : (invSqrtTwo : ℂ) * invSqrtTwo = 1 / 2
```

lemma adjRep_eighthTurn_offDiag [source]

The eighth turn in the plane of `p` and `q` sends the sum of the matrix units at `(p, q)` and at `(q, p)` to minus the difference of the diagonal matrix units at `p` and at `q`.

```
lemma adjRep_eighthTurn_offDiag {p q : Fin N} (hpq : p ≠ q) :
    adjRep N (planeEmbed p q hpq (rotation invSqrtTwo invSqrtTwo invSqrtTwo_sq_add))
      (offDiag p q + offDiag q p) = -diagDiff p q
```

lemma map_diagDiff_eq_smul [source]

For `p ≠ q`, an endomorphism commuting with the adjoint action scales the difference of the diagonal matrix units at `p` and at `q` by the coefficient of the matrix unit at `(p, q)`.

```
lemma map_diagDiff_eq_smul (hL : ∀ (g : SU N) (A : SUAlgebraComplexified N),
    L (adjRep N g A) = adjRep N g (L A)) {p q : Fin N} (hpq : p ≠ q) :
    L (diagDiff p q) = adjMat N (L (offDiag p q)) p q • diagDiff p q
```

lemma coeff_offDiag_eq [source]

For distinct `p`, `q` and `r`, the coefficients of the matrix units at `(p, q)` and at `(p, r)` agree: the difference of the diagonal units at `p` and `r` is the sum of those at `p` and `q` and at `q` and `r`.

```
lemma coeff_offDiag_eq (hL : ∀ (g : SU N) (A : SUAlgebraComplexified N),
    L (adjRep N g A) = adjRep N g (L A)) {p q r : Fin N} (hpq : p ≠ q) (hpr : p ≠ r)
    (hqr : q ≠ r) : adjMat N (L (offDiag p r)) p r = adjMat N (L (offDiag p q)) p q
```

lemma exists_coeff_offDiag_eq [source]

All the matrix units off the diagonal have the same coefficient under an endomorphism commuting with the adjoint action.

```
lemma exists_coeff_offDiag_eq (hL : ∀ (g : SU N) (A : SUAlgebraComplexified N),
    L (adjRep N g A) = adjRep N g (L A)) :
    ∃ μ : ℂ, ∀ x y : Fin N, x ≠ y → adjMat N (L (offDiag x y)) x y = μ
```

lemma eq_sum_offDiag_add_sum_diagDiff [source]

An element of the complexified Lie algebra is the combination of the entries of its matrix off the diagonal against the matrix units, and of its diagonal entries against the differences of the diagonal matrix units at a fixed coordinate.

```
lemma eq_sum_offDiag_add_sum_diagDiff (A : SUAlgebraComplexified N) (x₀ : Fin N) :
    A = ∑ x, ∑ y, adjMat N A x y • offDiag x y + ∑ x, adjMat N A x x • diagDiff x x₀
```

lemma eq_smul_id_of_commute_adjRep [source]

An endomorphism of the complexified Lie algebra commuting with the adjoint action of every element of `SU(N)` is scalar: a form of Schur's lemma for the adjoint representation.

```
lemma eq_smul_id_of_commute_adjRep (hL : ∀ (g : SU N) (A : SUAlgebraComplexified N),
    L (adjRep N g A) = adjRep N g (L A)) :
    ∃ z : ℂ, L = z • LinearMap.id
```

2.230 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Invariants.BiFund

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/SU/Invariants/BiFund.lean on GitHub.

lemma basis_repr_smul_fund [source]

The components of `g • t` for a tensor with `n` fundamental indices: one factor of `g` per index.

```
lemma basis_repr_smul_fund {n : ℕ} (g : SU N) (t : (suTensor N).Tensor fun _ : Fin n => .fund)
    (φ : Fin n → Fin N) :
    (Tensor.basis _).repr (g • t) φ
```

```
    = ∑ ψ : Fin n → Fin N, (∏ i, g.1 (φ i) (ψ i)) * (Tensor.basis _).repr t ψ
```

lemma basis_repr_smul_antiFund [source]

The components of `g • t` for a tensor with `n` anti-fundamental indices: one factor of the complex conjugate of `g` per index.

```
lemma basis_repr_smul_antiFund {n : ℕ} (g : SU N)
    (t : (suTensor N).Tensor fun _ : Fin n => .antiFund) (φ : Fin n → Fin N) :
    (Tensor.basis _).repr (g • t) φ = ∑ ψ : Fin n → Fin N,
      (∏ i, starRingEnd ℂ (g.1 (φ i) (ψ i))) * (Tensor.basis _).repr t ψ
```

lemma sum_fin_two_arrow [source]

A sum over pairs of labels is a double sum.

```
lemma sum_fin_two_arrow {M : Type*} [AddCommMonoid M] (F : (Fin 2 → Fin N) → M) :
    ∑ ψ : Fin 2 → Fin N, F ψ = ∑ x, ∑ y, F ![x, y]
```

def epsilonFund [source]

The antisymmetric symbol `ε^{ab}` with two fundamental indices of `SU(2)`.

```
noncomputable def epsilonFund : (suTensor 2).Tensor fun _ : Fin 2 => .fund
```

def epsilonAntiFund [source]

The antisymmetric symbol `ε_{ab}` with two anti-fundamental indices of `SU(2)`.

```
noncomputable def epsilonAntiFund : (suTensor 2).Tensor fun _ : Fin 2 => .antiFund
```

lemma basis_repr_epsilonFund [source]

The components of the antisymmetric symbol.

```
lemma basis_repr_epsilonFund (φ : Fin 2 → Fin 2) :
    (Tensor.basis _).repr epsilonFund φ
      = (if φ = ![0, 1] then 1 else 0) - (if φ = ![1, 0] then 1 else 0)
```

lemma basis_repr_epsilonAntiFund [source]

The components of the antisymmetric symbol.

```
lemma basis_repr_epsilonAntiFund (φ : Fin 2 → Fin 2) :
    (Tensor.basis _).repr epsilonAntiFund φ
      = (if φ = ![0, 1] then 1 else 0) - (if φ = ![1, 0] then 1 else 0)
```

lemma eq_smul_epsilon_of_fixed [source]

A function of two labels fixed by a diagonal matrix with entries squaring to `-1` and by the quarter turn, acting by one factor of `M g` per label, is a multiple of the antisymmetric symbol.

```
lemma eq_smul_epsilon_of_fixed (M : SU 2 → Matrix (Fin 2) (Fin 2) ℂ) {u v : ℂ}
    (hu : u * u = -1) (hv : v * v = -1)
    (hD : M (diagPhase Complex.I (by simp)) = diagonal ![u, v])
    (hR : M (rotation 0 1 (by norm_num)) = !![0, -1; 1, 0]) {c : (Fin 2 → Fin 2) → ℂ}
    (hc : ∀ (g : SU 2) (φ : Fin 2 → Fin 2),
      c φ = ∑ ψ : Fin 2 → Fin 2, (∏ i, M g (φ i) (ψ i)) * c ψ) (φ : Fin 2 → Fin 2) :
    c φ = c ![0, 1] * ((if φ = ![0, 1] then 1 else 0) - (if φ = ![1, 0] then 1 else 0))
```

lemma exists_eq_smul_epsilonFund_of_invariant [source]

An invariant tensor with two fundamental indices of `SU(2)` is a multiple of the antisymmetric symbol.

```
lemma exists_eq_smul_epsilonFund_of_invariant (t : (suTensor 2).Tensor fun _ : Fin 2 => .fund)
    (ht : ∀ g : SU 2, g • t = t) : ∃ a : ℂ, t = a • epsilonFund
```

lemma exists_eq_smul_epsilonAntiFund_of_invariant [source]

An invariant tensor with two anti-fundamental indices of `SU(2)` is a multiple of the antisymmetric symbol.

```
lemma exists_eq_smul_epsilonAntiFund_of_invariant
    (t : (suTensor 2).Tensor fun _ : Fin 2 => .antiFund) (ht : ∀ g : SU 2, g • t = t) :
    ∃ a : ℂ, t = a • epsilonAntiFund
```

lemma det_fin_two_eq_one [source]

The determinant of an element of `SU(2)`, written out.

```
lemma det_fin_two_eq_one (g : SU 2) : g.1 0 0 * g.1 1 1 - g.1 0 1 * g.1 1 0 = 1
```

lemma epsilonFund_invariant [source]

The antisymmetric symbol with two fundamental indices is invariant: `g ⊗ g` scales it by `det g = 1`.

```
lemma epsilonFund_invariant (g : SU 2) : g • epsilonFund = epsilonFund
```

lemma epsilonAntiFund_invariant [source]

The antisymmetric symbol with two anti-fundamental indices is invariant: `ḡ ⊗ ḡ` scales it by the conjugate of `det g = 1`.

```
lemma epsilonAntiFund_invariant (g : SU 2) : g • epsilonAntiFund = epsilonAntiFund
```

def rootOfUnity [source]

The primitive `N`-th root of unity `exp (2 π i / N)`.

```
noncomputable def rootOfUnity (N : ℕ) : ℂ
```

lemma rootOfUnity_isPrimitiveRoot [source]

```
lemma rootOfUnity_isPrimitiveRoot (hN : N ≠ 0) : IsPrimitiveRoot (rootOfUnity N) N
```

lemma rootOfUnity_mul_star [source]

```
lemma rootOfUnity_mul_star : rootOfUnity N * star (rootOfUnity N) = 1
```

def centre [source]

The centre element `ω 1` of `SU(N)`, for `ω` the primitive `N`-th root of unity.

```
noncomputable def centre (N : ℕ) (hN : N ≠ 0) : SU N
```

lemma basis_repr_centre_smul_fund [source]

The centre element scales the components of a tensor with two fundamental indices by `ω²`.

```
lemma basis_repr_centre_smul_fund (hN : N ≠ 0) (t : (suTensor N).Tensor fun _ : Fin 2 => .fund)
    (φ : Fin 2 → Fin N) :
    (Tensor.basis _).repr (centre N hN • t) φ
      = rootOfUnity N ^ 2 * (Tensor.basis _).repr t φ
```

lemma basis_repr_centre_smul_antiFund [source]

The centre element scales the components of a tensor with two anti-fundamental indices by `ω⁻²`.

```
lemma basis_repr_centre_smul_antiFund (hN : N ≠ 0)
    (t : (suTensor N).Tensor fun _ : Fin 2 => .antiFund) (φ : Fin 2 → Fin N) :
    (Tensor.basis _).repr (centre N hN • t) φ
      = starRingEnd ℂ (rootOfUnity N) ^ 2 * (Tensor.basis _).repr t φ
```

lemma rootOfUnity_sq_ne_one [source]

For `N ≥ 3`, `ω² ≠ 1`.

```
lemma rootOfUnity_sq_ne_one (hN : 3 ≤ N) : rootOfUnity N ^ 2 ≠ 1
```

lemma eq_zero_of_invariant_fundPair [source]

For `N ≥ 3` the only invariant tensor with two fundamental indices is zero: the centre element scales it by `ω² ≠ 1`.

```
lemma eq_zero_of_invariant_fundPair (hN : 3 ≤ N) (t : (suTensor N).Tensor fun _ : Fin 2 => .
    fund)
    (ht : ∀ g : SU N, g • t = t) : t = 0
```

lemma eq_zero_of_invariant_antiFundPair [source]

For `N ≥ 3` the only invariant tensor with two anti-fundamental indices is zero: the centre element scales it by `ω⁻² ≠ 1`.

```
lemma eq_zero_of_invariant_antiFundPair (hN : 3 ≤ N)
    (t : (suTensor N).Tensor fun _ : Fin 2 => .antiFund) (ht : ∀ g : SU N, g • t = t) :
    t = 0
```

def fundMap [source]

The linear map out of the tensors with `n` fundamental indices sending the basis tensor with labels `l` to `T l`.

```
noncomputable def fundMap {n : ℕ} (T : (Fin n → Fin N) → B) :
    (suTensor N).Tensor (fun _ : Fin n => .fund) →ₗ[ℂ] B
```

def antiFundMap [source]

The linear map out of the tensors with `n` anti-fundamental indices sending the basis tensor with labels `l` to `T l`.

```
noncomputable def antiFundMap {n : ℕ} (T : (Fin n → Fin N) → B) :
    (suTensor N).Tensor (fun _ : Fin n => .antiFund) →ₗ[ℂ] B
```

lemma fundMap_basis [source]

```
@[simp]
lemma fundMap_basis {n : ℕ} (T : (Fin n → Fin N) → B) (l : Fin n → Fin N) :
    fundMap T (Tensor.basis (S := suTensor N) (fun _ : Fin n => Color.fund) l) = T l
```

lemma antiFundMap_basis [source]

```
@[simp]
lemma antiFundMap_basis {n : ℕ} (T : (Fin n → Fin N) → B) (l : Fin n → Fin N) :
    antiFundMap T (Tensor.basis (S := suTensor N) (fun _ : Fin n => Color.antiFund) l) = T l
```

lemma fundMap_smul_of_law [source]

A linear map moving a family with `n` fundamental labels by one factor of `g` per label intertwines the map of the family with the action of `g`.

```
lemma fundMap_smul_of_law {n : ℕ} (T : (Fin n → Fin N) → B) {σ : B →ₗ[ℂ] B} (g : SU N)
    (hσ : ∀ l : Fin n → Fin N, σ (T l) = ∑ a : Fin n → Fin N, (∏ i, g.1 (a i) (l i)) • T a)
    (t : (suTensor N).Tensor fun _ : Fin n => .fund) :
    σ (fundMap T t) = fundMap T (g • t)
```

lemma antiFundMap_smul_of_law [source]

A linear map moving a family with `n` anti-fundamental labels by one factor of the complex conjugate of `g` per label intertwines the map of the family with the action of `g`.

```
lemma antiFundMap_smul_of_law {n : ℕ} (T : (Fin n → Fin N) → B) {σ : B →ₗ[ℂ] B} (g : SU N)
    (hσ : ∀ l : Fin n → Fin N, σ (T l)
      = ∑ a : Fin n → Fin N, (∏ i, starRingEnd ℂ (g.1 (a i) (l i))) • T a)
    (t : (suTensor N).Tensor fun _ : Fin n => .antiFund) :
    σ (antiFundMap T t) = antiFundMap T (g • t)
```

lemma isEquivariant_fundMap [source]

The map of a family with `n` fundamental labels is equivariant when the family moves by one factor of `g` per label, the summed label first.

```
lemma isEquivariant_fundMap {n : ℕ} (T : (Fin n → Fin N) → B)
    (hT : ∀ (g : SU N) (l : Fin n → Fin N), ρ g (T l)
      = ∑ a : Fin n → Fin N, (∏ i, g.1 (a i) (l i)) • T a) :
    (suTensor N).IsEquivariant (fun _ => .fund) ρ (fundMap T)
```

lemma isEquivariant_antiFundMap [source]

The map of a family with `n` anti-fundamental labels is equivariant when the family moves by one factor of the complex conjugate of `g` per label, the summed label first.

```
lemma isEquivariant_antiFundMap {n : ℕ} (T : (Fin n → Fin N) → B)
    (hT : ∀ (g : SU N) (l : Fin n → Fin N), ρ g (T l)
      = ∑ a : Fin n → Fin N, (∏ i, starRingEnd ℂ (g.1 (a i) (l i))) • T a) :
    (suTensor N).IsEquivariant (fun _ => .antiFund) ρ (antiFundMap T)
```

lemma fundMap_epsilonFund [source]

The map of a family with two fundamental labels sends the antisymmetric symbol to the epsilon contraction.

```
lemma fundMap_epsilonFund (T : (Fin 2 → Fin 2) → B) :
    fundMap T epsilonFund = T ![0, 1] - T ![1, 0]
```

lemma antiFundMap_epsilonAntiFund [source]

The map of a family with two anti-fundamental labels sends the antisymmetric symbol to the epsilon contraction.

```
lemma antiFundMap_epsilonAntiFund (T : (Fin 2 → Fin 2) → B) :
    antiFundMap T epsilonAntiFund = T ![0, 1] - T ![1, 0]
```

def invariantReductionToEpsilonFund [source]

For a family with two fundamental labels of `SU(2)` whose map is equivariant, the invariants of the span reduce to the span of the epsilon contraction `T ![0, 1] - T ![1, 0]`.

```
noncomputable def invariantReductionToEpsilonFund {ρ : Representation ℂ (SU 2) B}
    {T : (Fin 2 → Fin 2) → B} (hT : (suTensor 2).IsEquivariant (fun _ => .fund) ρ (fundMap T))
    :
    InvariantReductionToSpan (fun g : SU 2 => ρ g) (Submodule.span ℂ (Set.range T))
```

def invariantReductionToEpsilonAntiFund [source]

For a family with two anti-fundamental labels of `SU(2)` whose map is equivariant, the invariants of the span reduce to the span of the epsilon contraction `T ![0, 1] - T ![1, 0]`.

```
noncomputable def invariantReductionToEpsilonAntiFund {ρ : Representation ℂ (SU 2) B}
    {T : (Fin 2 → Fin 2) → B}
```

```
    (hT : (suTensor 2).IsEquivariant (fun _ => .antiFund) ρ (antiFundMap T)) :
    InvariantReductionToSpan (fun g : SU 2 => ρ g) (Submodule.span ℂ (Set.range T))
```

lemma reducesInvariantsTo_bot_span_of_fundMap [source]

For `N ≥ 3` and a family with two fundamental labels whose map is equivariant, the invariants of the span reduce to `⊥`.

```
lemma reducesInvariantsTo_bot_span_of_fundMap (hN : 3 ≤ N) {T : (Fin 2 → Fin N) → B}
    (hT : (suTensor N).IsEquivariant (fun _ => .fund) ρ (fundMap T)) :
    ReducesInvariantsTo (fun g : SU N => ρ g) (Submodule.span ℂ (Set.range T)) ⊥
```

lemma reducesInvariantsTo_bot_span_of_antiFundMap [source]

For `N ≥ 3` and a family with two anti-fundamental labels whose map is equivariant, the invariants of the span reduce to `⊥`.

```
lemma reducesInvariantsTo_bot_span_of_antiFundMap (hN : 3 ≤ N) {T : (Fin 2 → Fin N) → B}
    (hT : (suTensor N).IsEquivariant (fun _ => .antiFund) ρ (antiFundMap T)) :
    ReducesInvariantsTo (fun g : SU N => ρ g) (Submodule.span ℂ (Set.range T)) ⊥
```

2.231 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Invariants.FundAnti

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/SU/Invariants/FundAnti.lean on GitHub.

def fundAntiFundIdx [source]

The component indices of a tensor with a fundamental and an anti-fundamental index, as the pair of their labels.

```
def fundAntiFundIdx : ComponentIdx (S := suTensor N) ![.fund, .antiFund] ≃ (Fin 2 → Fin N)
    where
```

def delta [source]

The Kronecker delta `δ^a_b`, the unit tensor of the anti-fundamental color, with a fundamental and an anti-fundamental index.

```
noncomputable def delta : SuT[N, .fund, .antiFund]
```

lemma delta_invariant [source]

The Kronecker delta is invariant.

```
lemma delta_invariant (g : SU N) : g • delta N = delta N
```

lemma basis_repr_delta [source]

The components of the Kronecker delta.

```
lemma basis_repr_delta (n : Fin 2 → Fin N) :
    (Tensor.basis _).repr (delta N) (fundAntiFundIdx.symm n) = if n 0 = n 1 then 1 else 0
```

lemma basis_repr_smul_fundAntiFund [source]

The components of `g • t` for a tensor with a fundamental and an anti-fundamental index: the matrix of components `C` is moved to `g C g⁻¹`.

```
lemma basis_repr_smul_fundAntiFund (g : SU N) (t : SuT[N, .fund, .antiFund])
    (n : Fin 2 → Fin N) :
    (Tensor.basis _).repr (g • t) (fundAntiFundIdx.symm n)
      = ∑ x, ∑ y, g.1 (n 0) x * (g⁻¹).1 y (n 1)
        * (Tensor.basis _).repr t (fundAntiFundIdx.symm ![x, y])
```

lemma exists_eq_smul_delta_of_invariant [source]

An invariant tensor with a fundamental and an anti-fundamental index is a multiple of the Kronecker delta: its matrix of components commutes with every element of `SU(N)`.

```
lemma exists_eq_smul_delta_of_invariant (t : SuT[N, .fund, .antiFund])
    (ht : ∀ g : SU N, g • t = t) : ∃ a : ℂ, t = a • delta N
```

def invariantReductionToDeltaImage [source]

For an equivariant map `f` out of the tensors with a fundamental and an anti-fundamental index, the invariants of the range of `f` reduce to the span of the image `f (delta N)` of the Kronecker delta.

```
noncomputable def invariantReductionToDeltaImage
    (hf : (suTensor N).IsEquivariant ![.fund, .antiFund] ρ f) :
    InvariantReductionToSpan (fun g : SU N => ρ g) (LinearMap.range f)
```

def fundAntiFundMap [source]

The linear map out of the tensors with a fundamental and an anti-fundamental index sending the basis tensor with labels `n` to `T n`.

```
noncomputable def fundAntiFundMap (T : (Fin 2 → Fin N) → B) :
    SuT[N, .fund, .antiFund] →ₗ[ℂ] B
```

lemma fundAntiFundMap_delta [source]

The map of a family sends the Kronecker delta to the contraction `∑ a, T ![a, a]`.

```
lemma fundAntiFundMap_delta (T : (Fin 2 → Fin N) → B) :
    fundAntiFundMap T (delta N) = ∑ a, T ![a, a]
```

lemma fundAntiFundMap_smul_of_law [source]

A linear map moving a family as `g` moves the components of a tensor with a fundamental and an anti-fundamental index intertwines the map of the family with the action of `g`: a factor of `g` on the first index and of its complex conjugate on the second, the summed index first.

```
lemma fundAntiFundMap_smul_of_law (T : (Fin 2 → Fin N) → B) {σ : B →ₗ[ℂ] B} (g : SU N)
    (hσ : ∀ l : Fin 2 → Fin N, σ (T l)
      = ∑ a : Fin 2 → Fin N, (g.1 (a 0) (l 0) * starRingEnd ℂ (g.1 (a 1) (l 1))) • T a)
```

```
    (t : SuT[N, .fund, .antiFund]) :
    σ (fundAntiFundMap T t) = fundAntiFundMap T (g • t)
```

lemma isEquivariant_fundAntiFundMap [source]

The map of a family is equivariant when the family moves as the components of a tensor with a fundamental and an anti-fundamental index.

```
lemma isEquivariant_fundAntiFundMap (T : (Fin 2 → Fin N) → B)
    (hT : ∀ (g : SU N) (l : Fin 2 → Fin N), ρ g (T l)
      = ∑ a : Fin 2 → Fin N, (g.1 (a 0) (l 0) * starRingEnd ℂ (g.1 (a 1) (l 1))) • T a) :
    (suTensor N).IsEquivariant ![.fund, .antiFund] ρ (fundAntiFundMap T)
```

def invariantReductionToDelta [source]

For a family whose map is equivariant, the invariants of the span of the family reduce to the span of the delta contraction `∑ a, T ![a, a]`.

```
noncomputable def invariantReductionToDelta {T : (Fin 2 → Fin N) → B}
    (hT : (suTensor N).IsEquivariant ![.fund, .antiFund] ρ (fundAntiFundMap T)) :
    InvariantReductionToSpan (fun g : SU N => ρ g) (Submodule.span ℂ (Set.range T))
```

2.232 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.Invariants.QuadFund

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/SU/Invariants/QuadFund.lean on GitHub.

def epsilon [source]

The antisymmetric symbol of `SU(2)` as a function of two labels.

```
def epsilon (a b : Fin 2) : ℂ
```

lemma sum_mul_epsilon [source]

The antisymmetric symbol is invariant: `∑ g a x g b y ε x y = det g ε a b`.

```
lemma sum_mul_epsilon (g : SU 2) (a b : Fin 2) :
    ∑ x, ∑ y, g.1 a x * g.1 b y * epsilon x y = epsilon a b
```

def epsilonPair₁₂ [source]

The pairing `ε^{ab} ε^{cd}` of the first index with the second and the third with the fourth.

```
noncomputable def epsilonPair₁₂ : (suTensor 2).Tensor fun _ : Fin 4 => .fund
```

def epsilonPair₁₃ [source]

The pairing `ε^{ac} ε^{bd}` of the first index with the third and the second with the fourth.

```
noncomputable def epsilonPair₁₃ : (suTensor 2).Tensor fun _ : Fin 4 => .fund
```

lemma basis_repr_epsilonPair₁₂ [source]

```
lemma basis_repr_epsilonPair₁₂ (l : Fin 4 → Fin 2) :
    (Tensor.basis _).repr epsilonPair₁₂ l = epsilon (l 0) (l 1) * epsilon (l 2) (l 3)
```

lemma basis_repr_epsilonPair₁₃ [source]

```
lemma basis_repr_epsilonPair₁₃ (l : Fin 4 → Fin 2) :
    (Tensor.basis _).repr epsilonPair₁₃ l = epsilon (l 0) (l 2) * epsilon (l 1) (l 3)
```

lemma sum_fin_four_arrow [source]

A sum over four labels is a fourfold sum.

```
lemma sum_fin_four_arrow {M : Type*} [AddCommMonoid M] (F : (Fin 4 → Fin 2) → M) :
    ∑ ψ : Fin 4 → Fin 2, F ψ = ∑ x, ∑ y, ∑ z, ∑ w, F ![x, y, z, w]
```

lemma sum_mul_epsilon_mul_epsilon [source]

A product of two invariant pairings of labels is invariant.

```
lemma sum_mul_epsilon_mul_epsilon (g : SU 2) (l : Fin 4 → Fin 2) (i j k m : Fin 4)
    (hijkm : ∀ ψ : Fin 4 → Fin 2, (∏ r, g.1 (l r) (ψ r))
      = g.1 (l i) (ψ i) * g.1 (l j) (ψ j) * (g.1 (l k) (ψ k) * g.1 (l m) (ψ m)))
    (F : Fin 2 → Fin 2 → Fin 2 → Fin 2 → Fin 4 → Fin 2) (hF : ∀ x y z w, F x y z w i = x ∧
      F x y z w j = y ∧ F x y z w k = z ∧ F x y z w m = w)
    (hsum : ∀ G : (Fin 4 → Fin 2) → ℂ, ∑ ψ, G ψ = ∑ x, ∑ y, ∑ z, ∑ w, G (F x y z w)) :
    ∑ ψ : Fin 4 → Fin 2, (∏ r, g.1 (l r) (ψ r)) * (epsilon (ψ i) (ψ j) * epsilon (ψ k) (ψ m))
      = epsilon (l i) (l j) * epsilon (l k) (l m)
```

lemma epsilonPair₁₂_invariant [source]

The pairing `ε^{ab} ε^{cd}` is invariant.

```
lemma epsilonPair₁₂_invariant (g : SU 2) : g • epsilonPair₁₂ = epsilonPair₁₂
```

lemma epsilonPair₁₃_invariant [source]

The pairing `ε^{ac} ε^{bd}` is invariant.

```
lemma epsilonPair₁₃_invariant (g : SU 2) : g • epsilonPair₁₃ = epsilonPair₁₃
```

lemma eq_zero_of_not_dvd [source]

The phase `diag (ω, ω⁻)`, with `ω` a primitive eighth root of unity, scales the component at `l` by `ω` to the power of the number of zeros of `l` plus seven times the number of ones, so the component vanishes unless that power is a multiple of eight.

```
lemma eq_zero_of_not_dvd (l : Fin 4 → Fin 2) (hl : ¬ 8 ∣ ∑ i, ![1, 7] (l i)) : c l = 0
```

lemma quarterTurn_relations [source]

The quarter turn equates each balanced component with its complement.

```
lemma quarterTurn_relations :
    c ![0, 0, 1, 1] = c ![1, 1, 0, 0] ∧ c ![0, 1, 0, 1] = c ![1, 0, 1, 0] ∧
      c ![0, 1, 1, 0] = c ![1, 0, 0, 1]
```

lemma unbalanced_eq_zero [source]

The ten unbalanced components vanish.

```
lemma unbalanced_eq_zero :
    c ![0, 0, 0, 0] = 0 ∧ c ![0, 0, 0, 1] = 0 ∧ c ![0, 0, 1, 0] = 0 ∧ c ![0, 1, 0, 0] = 0 ∧
      c ![1, 0, 0, 0] = 0 ∧ c ![0, 1, 1, 1] = 0 ∧ c ![1, 0, 1, 1] = 0 ∧ c ![1, 1, 0, 1] = 0 ∧
      c ![1, 1, 1, 0] = 0 ∧ c ![1, 1, 1, 1] = 0
```

lemma sum_balanced_eq_zero [source]

The eighth turn carries the component at `![0, 0, 0, 0]` to a quarter of the sum of the balanced components, so that sum vanishes.

```
lemma sum_balanced_eq_zero :
    c ![0, 0, 1, 1] + c ![0, 1, 0, 1] + c ![0, 1, 1, 0] + c ![1, 0, 0, 1] + c ![1, 0, 1, 0]
      + c ![1, 1, 0, 0] = 0
```

lemma eq_add_of_pairings [source]

An invariant function of four labels is a combination of the two pairings, with the components at `![0, 1, 0, 1]` and `![0, 0, 1, 1]` as coefficients.

```
lemma eq_add_of_pairings (l : Fin 4 → Fin 2) :
    c l = c ![0, 1, 0, 1] * (epsilon (l 0) (l 1) * epsilon (l 2) (l 3))
      + c ![0, 0, 1, 1] * (epsilon (l 0) (l 2) * epsilon (l 1) (l 3))
```

lemma mem_span_epsilonPairs_of_invariant [source]

An invariant tensor with four fundamental indices of `SU(2)` is a combination of the two pairings of its indices by the antisymmetric symbol.

```
lemma mem_span_epsilonPairs_of_invariant (t : (suTensor 2).Tensor fun _ : Fin 4 => .fund)
    (ht : ∀ g : SU 2, g • t = t) :
    t ∈ Submodule.span ℂ {epsilonPair₁₂, epsilonPair₁₃}
```

lemma reducesInvariantsTo_span_map_epsilonPairs [source]

For an equivariant map `f` out of the tensors with four fundamental indices of `SU(2)`, the invariants of the range reduce to the span of the images of the two pairings.

```
lemma reducesInvariantsTo_span_map_epsilonPairs
    {f : (suTensor 2).Tensor (fun _ : Fin 4 => .fund) →ₗ[ℂ] B}
    (hf : (suTensor 2).IsEquivariant (fun _ => .fund) ρ f) :
    ReducesInvariantsTo (fun g : SU 2 => ρ g) (LinearMap.range f)
      (Submodule.span ℂ {f epsilonPair₁₂, f epsilonPair₁₃})
```

lemma fundMap_epsilonPair₁₂ [source]

The map of a family sends the first pairing to the contraction pairing the first label with the second and the third with the fourth.

```
lemma fundMap_epsilonPair₁₂ (T : (Fin 4 → Fin 2) → B) :
    fundMap T epsilonPair₁₂
      = T ![0, 1, 0, 1] - T ![0, 1, 1, 0] - T ![1, 0, 0, 1] + T ![1, 0, 1, 0]
```

lemma fundMap_epsilonPair₁₃ [source]

The map of a family sends the second pairing to the contraction pairing the first label with the third and the second with the fourth.

```
lemma fundMap_epsilonPair₁₃ (T : (Fin 4 → Fin 2) → B) :
    fundMap T epsilonPair₁₃
      = T ![0, 0, 1, 1] - T ![0, 1, 1, 0] - T ![1, 0, 0, 1] + T ![1, 1, 0, 0]
```

lemma reducesInvariantsTo_span_epsilonContractions [source]

For a family with four fundamental labels of `SU(2)` whose map is equivariant, the invariants of the span of the family reduce to the span of its two epsilon contractions.

```
lemma reducesInvariantsTo_span_epsilonContractions {T : (Fin 4 → Fin 2) → B}
    (hT : (suTensor 2).IsEquivariant (fun _ => .fund) ρ (fundMap T)) :
    ReducesInvariantsTo (fun g : SU 2 => ρ g) (Submodule.span ℂ (Set.range T))
      (Submodule.span ℂ
        {T ![0, 1, 0, 1] - T ![0, 1, 1, 0] - T ![1, 0, 0, 1] + T ![1, 0, 1, 0],
          T ![0, 0, 1, 1] - T ![0, 1, 1, 0] - T ![1, 0, 0, 1] + T ![1, 1, 0, 0]})
```

2.233 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.SU.TensorSpecies

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/SU/TensorSpecies.lean on GitHub.

inductive Color [source]

The colors of the complex tensors of `SU(N)`.

```
inductive Color
  | fund : Color
  | antiFund : Color
  | adj : Color
  deriving DecidableEq
```

abbrev modules [source]

The carriers of the three colors.

```
abbrev modules : Color → Type
  | .fund => Fin N → ℂ
  | .antiFund => Fin N → ℂ
  | .adj => SUAlgebraComplexified N
```

instance modulesAddCommGroup [source]

```
noncomputable instance modulesAddCommGroup : ∀ c, AddCommGroup (modules N c)
  | .fund => inferInstance
  | .antiFund => inferInstance
  | .adj => inferInstance
```

instance modulesModule [source]

```
noncomputable instance modulesModule : ∀ c, Module ℂ (modules N c)
  | .fund => inferInstance
  | .antiFund => inferInstance
  | .adj => inferInstance
```

abbrev basisIdx [source]

The labels of the basis vectors: `Fin N` for the fundamental and anti-fundamental colors, and the labels of the generalized Gell-Mann matrices for the adjoint.

```
abbrev basisIdx : Color → Type
  | .fund => Fin N
  | .antiFund => Fin N
  | .adj => GellMann.Index N
```

instance basisIdxFintype [source]

```
instance basisIdxFintype : ∀ c, Fintype (basisIdx N c)
  | .fund => inferInstance
  | .antiFund => inferInstance
  | .adj => inferInstance
```

instance basisIdxDecidableEq [source]

```
instance basisIdxDecidableEq : ∀ c, DecidableEq (basisIdx N c)
  | .fund => inferInstance
  | .antiFund => inferInstance
  | .adj => inferInstance
```

abbrev basis [source]

The bases of the carriers: the standard basis for the fundamental and anti-fundamental colors, and the generalized Gell-Mann matrices for the adjoint.

```
noncomputable abbrev basis : (c : Color) → Basis (basisIdx N c) ℂ (modules N c)
  | .fund => Pi.basisFun ℂ (Fin N)
  | .antiFund => Pi.basisFun ℂ (Fin N)
  | .adj => adjBasis N
```

def fundRep [source]

The fundamental representation, `v ↦ g v`.

```
noncomputable def fundRep : Representation ℂ (SU N) (Fin N → ℂ) where
```

def antiFundRep [source]

The anti-fundamental representation, $v \mapsto (g^{-1})^T v$, the dual of the fundamental one. For unitary g, $(g^{-1})^T$ is the complex conjugate of g.

```
noncomputable def antiFundRep : Representation ℂ (SU N) (Fin N → ℂ) where
```

abbrev rep [source]

The representations of the three colors.

```
noncomputable abbrev rep : (c : Color) → Representation ℂ (SU N) (modules N c)
  | .fund => fundRep N
  | .antiFund => antiFundRep N
  | .adj => adjRep N
```

def fundContr [source]

The contraction of a fundamental against an anti-fundamental index, the dot product $v \otimes w \mapsto \sum v_i w_i$.

```
noncomputable def fundContr : ((fundRep N).tprod (antiFundRep N)).IntertwiningMap
    (Representation.trivial ℂ (SU N) ℂ) where
```

def antiFundContr [source]

The contraction of an anti-fundamental against a fundamental index, the dot product $w \otimes v \mapsto \sum w_i v_i$.

```
noncomputable def antiFundContr : ((antiFundRep N).tprod (fundRep N)).IntertwiningMap
    (Representation.trivial ℂ (SU N) ℂ) where
```

def coevalMap [source]

The endomorphism $x \mapsto \sum \mathit{contr}\ (x \otimes w_i)\ v_i$ of V attached to $\sum w_i \otimes v_i \in W \otimes V$.

```
noncomputable def coevalMap (contr : (ρ.tprod σ).IntertwiningMap
    (Representation.trivial ℂ (SU N) ℂ)) : W ⊗[ℂ] V →ₗ[ℂ] V →ₗ[ℂ] V
```

lemma coevalMap_tmul [source]

```
@[simp]
lemma coevalMap_tmul (w : W) (v x : V) :
    coevalMap contr (w ⊗ₜ v) x = contr (x ⊗ₜ w) • v
```

lemma lid_rTensor_assoc_symm_eq_coevalMap [source]

The contraction of x against the first factor of U is $\mathit{coevalMap}\ \mathit{contr}\ U\ x$.

```
lemma lid_rTensor_assoc_symm_eq_coevalMap (x : V) (U : W ⊗[ℂ] V) :
    TensorProduct.lid ℂ V (contr.toLinearMap.rTensor V
      ((TensorProduct.assoc ℂ V W V).symm (x ⊗ₜ[ℂ] U))) = coevalMap contr U x
```

lemma coevalMap_injective [source]

`coevalMap` is injective when the contraction separates `W`.

```
lemma coevalMap_injective [FiniteDimensional ℂ V]
    (h : Function.Injective (TensorProduct.curry contr.toLinearMap).flip) :
    Function.Injective (coevalMap contr)
```

lemma coevalMap_map [source]

Transforming both factors of `U` conjugates `coevalMap contr U`, by the invariance of the contraction.

```
lemma coevalMap_map (g : SU N) (U : W ⊗[ℂ] V) :
    coevalMap contr (TensorProduct.map (σ g) (ρ g) U)
      = ρ g ∘ₗ coevalMap contr U ∘ₗ ρ g⁻¹
```

lemma map_eq_self_of_coevalMap_eq_id [source]

An element whose `coevalMap` is the identity is invariant, when the contraction separates `W`.

```
lemma map_eq_self_of_coevalMap_eq_id [FiniteDimensional ℂ V]
    (h : Function.Injective (TensorProduct.curry contr.toLinearMap).flip) {U : W ⊗[ℂ] V}
    (hU : coevalMap contr U = LinearMap.id) (g : SU N) :
    TensorProduct.map (σ g) (ρ g) U = U
```

def unitOf [source]

The unit intertwining map `a ↦ a • U` of an invariant element `U ∈ W ⊗ V`.

```
noncomputable def unitOf {U : W ⊗[ℂ] V} (hU : ∀ g : SU N, TensorProduct.map (σ g) (ρ g) U = U)
    :
    (Representation.trivial ℂ (SU N) ℂ).IntertwiningMap (σ.tprod ρ) where
```

lemma unitOf_apply [source]

```
@[simp]
lemma unitOf_apply {U : W ⊗[ℂ] V} (hU : ∀ g : SU N, TensorProduct.map (σ g) (ρ g) U = U)
    (a : ℂ) : unitOf hU a = a • U
```

def pairUnitVal [source]

The element $\sum e_i \otimes e_i$ of $\mathbb{C}^N \otimes \mathbb{C}^N$, the unit of the fundamental and of the anti-fundamental color.

```
noncomputable def pairUnitVal : (Fin N → ℂ) ⊗[ℂ] (Fin N → ℂ)
```

lemma comm_pairUnitVal [source]

```
lemma comm_pairUnitVal : TensorProduct.comm ℂ _ _ (pairUnitVal N) = pairUnitVal N
```

lemma dotProduct_flip_injective [source]

The dot product separates vectors.

```
lemma dotProduct_flip_injective : Function.Injective
    (TensorProduct.curry (TensorProduct.lift (dotProductBilin ℂ ℂ) :
      (Fin N → ℂ) ⊗[ℂ] (Fin N → ℂ) →ₗ[ℂ] ℂ)).flip
```

lemma coevalMap_pairUnitVal [source]

Contracting against $\sum e_i \otimes e_i$ by the dot product is the identity.

```
lemma coevalMap_pairUnitVal {ρ σ : Representation ℂ (SU N) (Fin N → ℂ)}
    (contr : (ρ.tprod σ).IntertwiningMap (Representation.trivial ℂ (SU N) ℂ))
    (h : contr.toLinearMap = TensorProduct.lift (dotProductBilin ℂ ℂ)) :
    coevalMap contr (pairUnitVal N) = LinearMap.id
```

def fundUnit [source]

The unit of the fundamental color, $\sum e_i \otimes e_i$ in anti-fundamental $\otimes$ fundamental.

```
noncomputable def fundUnit : (Representation.trivial ℂ (SU N) ℂ).IntertwiningMap
    ((antiFundRep N).tprod (fundRep N))
```

def antiFundUnit [source]

The unit of the anti-fundamental color, $\sum e_i \otimes e_i$ in fundamental $\otimes$ anti-fundamental.

```
noncomputable def antiFundUnit : (Representation.trivial ℂ (SU N) ℂ).IntertwiningMap
    ((fundRep N).tprod (antiFundRep N))
```

def adjUnitVal [source]

The element $\sum b_a \otimes b^a$ of su(N)_ℂ ⊗ su(N)_ℂ, for the Gell-Mann basis b and its dual basis b^a under the trace form: the unit of the adjoint color.

```
noncomputable def adjUnitVal : (SUAlgebraComplexified N) ⊗[ℂ] (SUAlgebraComplexified N)
```

lemma coevalMap_adjUnitVal [source]

```
lemma coevalMap_adjUnitVal : coevalMap (adjContr N) (adjUnitVal N) = LinearMap.id
```

lemma coevalMap_comm_adjUnitVal [source]

```
lemma coevalMap_comm_adjUnitVal :
    coevalMap (adjContr N) (TensorProduct.comm ℂ _ _ (adjUnitVal N)) = LinearMap.id
```

lemma comm_adjUnitVal [source]

The unit of the adjoint color is symmetric.

```
lemma comm_adjUnitVal : TensorProduct.comm ℂ _ _ (adjUnitVal N) = adjUnitVal N
```

def adjUnit [source]

The unit of the adjoint color, ∑ b_a ⊗ b^a.

```
noncomputable def adjUnit : (Representation.trivial ℂ (SU N) ℂ).IntertwiningMap
    ((adjRep N).tprod (adjRep N))
```

def suTensor [source]

The complex tensor species of SU(N), with fundamental, anti-fundamental and adjoint indices. The fundamental and anti-fundamental colors are dual to each other, the adjoint color dual to itself.

```
noncomputable def suTensor (N : ℕ) : TensorSpecies ℂ suTensor.Color (SU N)
    (suTensor.modules N) (suTensor.basisIdx N) (suTensor.rep N)
    (suTensor.basis N) where
```

lemma toMatrix_fundRep [source]

```
lemma toMatrix_fundRep (g : SU N) :
    LinearMap.toMatrix (Pi.basisFun ℂ (Fin N)) (Pi.basisFun ℂ (Fin N)) (fundRep N g) = g.1
```

lemma toMatrix_antiFundRep [source]

```
lemma toMatrix_antiFundRep (g : SU N) :
    LinearMap.toMatrix (Pi.basisFun ℂ (Fin N)) (Pi.basisFun ℂ (Fin N)) (antiFundRep N g)
      = (g⁻¹).1^T
```

lemma toMatrix_rep_inv [source]

On every color, the matrix of g^{-1} is the conjugate transpose of the matrix of g.

```
lemma toMatrix_rep_inv (k : Color) (g : SU N) :
    LinearMap.toMatrix (basis N k) (basis N k) (rep N k g⁻¹)
      = (LinearMap.toMatrix (basis N k) (basis N k) (rep N k g))^H
```

lemma isAdjointClosed [source]

Every list of colors of the SU(N) tensors is closed under adjoints, with g' = g^{-1}.

```
lemma isAdjointClosed {n : ℕ} (c : Fin n → Color) : (suTensor N).IsAdjointClosed c
```

2.234 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.Truncation

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/Truncation.lean on GitHub.

def truncationKer [source]

The jets trivial to order n: value the identity, and base-point Taylor coefficients of the Maurer–Cartan form vanishing below order n. Closure under products and inverses is the cocycle law together with the Taylor–Leibniz theorem for the adjoint action.

```
noncomputable def truncationKer (n : ℕ) : Subgroup GJ where
```

lemma mem_truncationKer_zero_iff [source]

The zeroth truncation kernel is the group of pure jets, those with identity value.

```
lemma mem_truncationKer_zero_iff {U : GJ} : U ∈ jets.truncationKer 0 ↔ jets.eval U = 1
```

lemma adjointCoeff_eq_zero_of_mem_truncationKer [source]

Deep jets kill the positive adjoint coefficients: for a jet trivial to order `n`, the adjoint coefficients of order between `1` and `n` vanish. One derivative of the adjoint is `ad` of the Maurer–Cartan form, whose base-point data vanish below order `n`.

```
lemma adjointCoeff_eq_zero_of_mem_truncationKer {n : ℕ} {U : GJ} (hU : U ∈ jets.truncationKer n
    )
    {x : Multiset (Fin 1 ⊕ Fin 3)} (hx : x ≠ 0) (hxn : x.card ≤ n) :
    jets.adjointCoeff U x = 0
```

lemma adjointDualCoeff_eq_zero_of_mem_truncationKer [source]

The dual form of `adjointCoeff_eq_zero_of_mem_truncationKer`.

```
lemma adjointDualCoeff_eq_zero_of_mem_truncationKer {n : ℕ} {U : GJ}
    (hU : U ∈ jets.truncationKer n) {x : Multiset (Fin 1 ⊕ Fin 3)} (hx : x ≠ 0)
    (hxn : x.card ≤ n) : jets.adjointDualCoeff U x = 0
```

lemma adjointCoeff_mul_of_mem_truncationKer_right [source]

Up to order `n`, a jet trivial to order `n` is invisible on the right of a product.

```
lemma adjointCoeff_mul_of_mem_truncationKer_right (g : GJ) {n : ℕ} {U : GJ}
    (hU : U ∈ jets.truncationKer n) {x : Multiset (Fin 1 ⊕ Fin 3)} (hxn : x.card ≤ n) :
    jets.adjointCoeff (g * U) x = jets.adjointCoeff g x
```

lemma adjointCoeff_conj_of_mem_truncationKer [source]

Up to order `n`, a conjugate of a jet trivial to order `n` has the adjoint coefficients of the identity.

```
lemma adjointCoeff_conj_of_mem_truncationKer (g : GJ) {n : ℕ} {U : GJ}
    (hU : U ∈ jets.truncationKer n) {x : Multiset (Fin 1 ⊕ Fin 3)} (hxn : x.card ≤ n) :
    jets.adjointCoeff (g * U * g⁻¹) x = jets.adjointCoeff 1 x
```

lemma evalLie_iteratedDeriv_adjoint_conj_of_mem_truncationKer [source]

Up to order `n`, a conjugate of a jet trivial to order `n` acts trivially on the base-point Taylor data of the jet Lie algebra.

```
lemma evalLie_iteratedDeriv_adjoint_conj_of_mem_truncationKer (g : GJ) {n : ℕ} {U : GJ}
    (hU : U ∈ jets.truncationKer n) {s : Multiset (Fin 1 ⊕ Fin 3)} (hs : s.card ≤ n)
    (Y : 𝔤J) : jets.evalLie (jets.iteratedDeriv s (jets.adjointJ (g * U * g⁻¹) Y)) =
      jets.evalLie (jets.iteratedDeriv s Y)
```

lemma maurerCartan_conj [source]

The Maurer–Cartan form of a conjugate, by the cocycle law: the conjugating jet contributes its own form and its transport by the conjugate.

```
lemma maurerCartan_conj (g U : GJ) (μ : Fin 1 ⊕ Fin 3) :
    jets.maurerCartan (g * U * g⁻¹) μ =
      jets.maurerCartan g μ + jets.adjointJ g (jets.maurerCartan U μ)
        - jets.adjointJ (g * U * g⁻¹) (jets.maurerCartan g μ)
```

instance truncationKer_normal [source]

The truncation kernels are normal subgroups: conjugating a jet trivial to order `n` gives a jet trivial to order `n`.

```
instance truncationKer_normal (n : ℕ) : (jets.truncationKer n).Normal where
```

def truncationProjZero [source]

The projection of a jet onto the pure jets, stripping its value: `U ↦ U · (U₀)⁻¹`. This is not a group homomorphism; it is the cocycle of the splitting of `GJ` by the constant jets.

```
noncomputable def truncationProjZero (U : GJ) : jets.truncationKer 0
```

lemma coe_truncationProjZero [source]

```
@[simp]
lemma coe_truncationProjZero (U : GJ) :
    (jets.truncationProjZero U : GJ) = U * (jets.ofConstant (jets.eval U))⁻¹
```

lemma eq_truncationProjZero_mul_ofConstant [source]

Every jet is its pure part times the constant jet of its value.

```
lemma eq_truncationProjZero_mul_ofConstant (U : GJ) :
    U = jets.truncationProjZero U * jets.ofConstant (jets.eval U)
```

lemma truncationProjZero_eq_one_iff [source]

The pure part of a jet is trivial exactly when the jet is constant.

```
lemma truncationProjZero_eq_one_iff {U : GJ} :
    jets.truncationProjZero U = 1 ↔ U = jets.ofConstant (jets.eval U)
```

lemma truncationProjZero_ofConstant [source]

```
@[simp]
lemma truncationProjZero_ofConstant (g : G₀) :
    jets.truncationProjZero (jets.ofConstant g) = 1
```

lemma maurerCartan_truncationProjZero [source]

Stripping the value of a jet does not change its Maurer–Cartan form: by the cocycle law, right multiplication by a constant jet drops out.

```
lemma maurerCartan_truncationProjZero (U : GJ) (μ : Fin 1 ⊕ Fin 3) :
    jets.maurerCartan (jets.truncationProjZero U) μ = jets.maurerCartan U μ
```

def symmetrizedMaurerCartanCoeff [source]

The symmetrized Maurer–Cartan data of a pure jet: the base-point values of its symmetrized Maurer–Cartan forms, indexed by nonempty multisets of directions. Total symmetry is automatic from the multiset indexing.

```
noncomputable def symmetrizedMaurerCartanCoeff (U : jets.truncationKer 0)
    (r : {r : Multiset (Fin 1 ⊕ Fin 3) // r ≠ 0}) : 𝔤
```

lemma symmetrizedMaurerCartanCoeff_apply [source]

```
lemma symmetrizedMaurerCartanCoeff_apply (U : jets.truncationKer 0)
    (r : {r : Multiset (Fin 1 ⊕ Fin 3) // r ≠ 0}) :
    jets.symmetrizedMaurerCartanCoeff U r =
      jets.evalLie (jets.symmetrizedMaurerCartanForm U.1 r.1)
```

lemma mem_truncationKer_of_symmetrizedMaurerCartanCoeff_eq_zero [source]

Maurer–Cartan triangularity: a pure jet whose symmetrized Maurer–Cartan data vanish up to order `n` is trivial to order `n`. The symmetrized data control the Taylor data of the Maurer–Cartan form through the symmetrization defect.

```
lemma mem_truncationKer_of_symmetrizedMaurerCartanCoeff_eq_zero (U : jets.truncationKer 0)
    (n : ℕ) (h : ∀ (r : Multiset (Fin 1 ⊕ Fin 3)) (hr : r ≠ 0), r.card ≤ n →
      jets.symmetrizedMaurerCartanCoeff U ⟨r, hr⟩ = 0) :
    U.1 ∈ jets.truncationKer n
```

lemma symmetrizedMaurerCartanCoeff_eq_zero_of_mem_truncationKer [source]

Conversely, the symmetrized Maurer–Cartan data of a jet trivial to order `n` vanish up to order `n`: each term of the symmetrized form carries fewer than `n` derivatives.

```
lemma symmetrizedMaurerCartanCoeff_eq_zero_of_mem_truncationKer {U : jets.truncationKer 0}
    {n : ℕ} (hU : U.1 ∈ jets.truncationKer n) {r : Multiset (Fin 1 ⊕ Fin 3)} (hr : r ≠ 0)
    (hrn : r.card ≤ n) : jets.symmetrizedMaurerCartanCoeff U ⟨r, hr⟩ = 0
```

lemma mem_truncationKer_iff_symmetrizedMaurerCartanCoeff_eq_zero [source]

The truncation filtration through the symmetrized Maurer–Cartan data: a pure jet is trivial to order `n` exactly when its symmetrized data vanish up to order `n`.

```
lemma mem_truncationKer_iff_symmetrizedMaurerCartanCoeff_eq_zero (U : jets.truncationKer 0)
    (n : ℕ) : U.1 ∈ jets.truncationKer n ↔
      ∀ (r : Multiset (Fin 1 ⊕ Fin 3)) (hr : r ≠ 0), r.card ≤ n →
        jets.symmetrizedMaurerCartanCoeff U ⟨r, hr⟩ = 0
```

lemma maurerCartan_injOn_truncationKer_zero [source]

A pure jet of a faithful package is determined by its Maurer–Cartan form. By the cocycle and inverse laws `ω(V⁻¹ U) = Ad_{V⁻¹}(ω(U) − ω(V)) = 0`, so `V⁻¹ U` is the constant jet of its value, which is the identity.

```
lemma maurerCartan_injOn_truncationKer_zero {U V : GJ} (hU : U ∈ jets.truncationKer 0)
    (hV : V ∈ jets.truncationKer 0) (h : jets.maurerCartan U = jets.maurerCartan V) :
    U = V
```

lemma symmetrizedMaurerCartanCoeff_injective [source]

A pure jet of a faithful package is determined by its symmetrized Maurer–Cartan data. The symmetrized data determine all base-point Taylor data of the Maurer–Cartan form (`evalLie_iteratedDeriv_maurerCartan_eq_of_symmetrized_eq_all`), hence the form itself by Taylor determinacy, hence the jet by `maurerCartan_injOn_truncationKer_zero`.

```
lemma symmetrizedMaurerCartanCoeff_injective :
    Function.Injective jets.symmetrizedMaurerCartanCoeff
```

def radial [source]

The radial component `∑_μ x_μ ω_μ(U)` of the Maurer–Cartan form of a jet.

```
noncomputable def radial (U : GJ) : 𝔤J
```

lemma symmetrizedMaurerCartanCoeff_eq_evalLie_iteratedDeriv_radial [source]

The symmetrized Maurer–Cartan data are the Taylor data of the radial component: `sym(ω(U))_r|₀ = (1/|r|) (∂_r ρ(U))|₀`.

```
lemma symmetrizedMaurerCartanCoeff_eq_evalLie_iteratedDeriv_radial (U : jets.truncationKer 0)
    (r : {r : Multiset (Fin 1 ⊕ Fin 3) // r ≠ 0}) :
    jets.symmetrizedMaurerCartanCoeff U r =
      (1 / (r.1.card : ℝ)) • jets.evalLie (jets.iteratedDeriv r.1 (jets.radial U.1))
```

class Free [source]

A package is free when it is faithful and its jets are honest formal power series in the coordinates: every family of base-point Taylor data is realized by an element of `𝔤J` (Taylor completeness), and every element vanishing at the base point is the radial component of the Maurer–Cartan form of a pure jet (radial integrability, the solution of the Euler equation `∑_μ x_μ ∂_μ U = −i ρ U` with `U(0) = 1`). Both hold for the full jet group of any matrix group. Freeness makes the symmetrized Maurer–Cartan data free coordinates on the pure jets, `symmetrizedMaurerCartanCoeff_bijective`; like `Faithful` it is recorded separately from the structure because the covariance theory does not need it, only the classification of invariants does.

```
class Free (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) : Prop extends Faithful jets where
  exists_evalLie_iteratedDeriv_eq : ∀ c : Multiset (Fin 1 ⊕ Fin 3) → 𝔤,
    ∃ Y : 𝔤J, ∀ s, jets.evalLie (jets.iteratedDeriv s Y) = c s
  exists_radial_eq : ∀ ρ : 𝔤J, jets.evalLie ρ = 0 → ∃ U : jets.truncationKer 0, jets.radial U.1
     = ρ
```

lemma exists_evalLie_iteratedDeriv_eq [source]

Taylor completeness of a free package.

```
lemma exists_evalLie_iteratedDeriv_eq (c : Multiset (Fin 1 ⊕ Fin 3) → 𝔤) :
    ∃ Y : 𝔤J, ∀ s, jets.evalLie (jets.iteratedDeriv s Y) = c s
```

lemma exists_radial_eq [source]

Radial integrability of a free package.

```
lemma exists_radial_eq {ρ : 𝔤J} (hρ : jets.evalLie ρ = 0) :
    ∃ U : jets.truncationKer 0, jets.radial U.1 = ρ
```

lemma symmetrizedMaurerCartanCoeff_surjective [source]

Every family of symmetrized Maurer–Cartan data is realized by a pure jet: realize the data, rescaled by the order, as the Taylor data of an element `ρ` vanishing at the base point, and integrate `ρ` to a pure jet.

```
lemma symmetrizedMaurerCartanCoeff_surjective :
    Function.Surjective jets.symmetrizedMaurerCartanCoeff
```

lemma symmetrizedMaurerCartanCoeff_bijective [source]

The symmetrized Maurer–Cartan data are free coordinates on the pure jets of a free package.

```
lemma symmetrizedMaurerCartanCoeff_bijective :
    Function.Bijective jets.symmetrizedMaurerCartanCoeff
```

2.235 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.LocalGaugeData.U1

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/LocalGaugeData/U1.lean on GitHub.

abbrev U1 [source]

The gauge group `U(1)`.

```
abbrev U1 : Type
```

abbrev JetU1 [source]

Jets of `U(1)` gauge transformations: unitary formal power series.

```
abbrev JetU1 : Type
```

abbrev U1AlgebraOver [source]

The Lie algebra `u(1)` over a `*`-ring: the self-adjoint elements, with vanishing bracket.

```
abbrev U1AlgebraOver (R : Type) [Ring R] [StarRing R] : Type
```

abbrev U1Algebra [source]

The Lie algebra `u(1)`: the self-adjoint (real) scalars.

```
abbrev U1Algebra : Type
```

abbrev JetU1Algebra [source]

Jets of the Lie algebra `u(1)`: the self-adjoint formal power series.

```
abbrev JetU1Algebra : Type
```

instance Bracket (U1AlgebraOver R) (U1AlgebraOver R) [source]

```
instance : Bracket (U1AlgebraOver R) (U1AlgebraOver R)
```

lemma bracket_eq_zero [source]

```
@[simp]
lemma bracket_eq_zero (a b : U1AlgebraOver R) : ⁅a, b⁆ = 0
```

instance LieRing (U1AlgebraOver R) [source]

```
instance : LieRing (U1AlgebraOver R) where
```

instance LieAlgebra ℝ (U1AlgebraOver R) [source]

```
instance [Algebra ℝ R] [StarModule ℝ R] : LieAlgebra ℝ (U1AlgebraOver R) where
```

instance Module.Finite ℝ U1Algebra [source]

```
instance : Module.Finite ℝ U1Algebra
```

def eval [source]

Evaluation of a jet of a `U(1)` gauge transformation at the base point.

```
noncomputable def eval : JetU1 →* U1 where
```

lemma eval_val [source]

```
@[simp]
lemma eval_val (u : JetU1) : (eval u : ℂ) = constantCoeff (u : SpaceTimeAlgebra)
```

def ofConstant [source]

The jet of a constant `U(1)` gauge transformation.

```
noncomputable def ofConstant : U1 →* JetU1 where
```

lemma ofConstant_val [source]

```
@[simp]
lemma ofConstant_val (u : U1) : (ofConstant u : SpaceTimeAlgebra) = C (u : ℂ)
```

def deriv [source]

The formal derivative of a `u(1)` jet.

```
noncomputable def deriv (μ : Fin 1 ⊕ Fin 3) : JetU1Algebra →ₗ[ℝ] JetU1Algebra where
```

lemma deriv_val [source]

```
@[simp]
lemma deriv_val (μ : Fin 1 ⊕ Fin 3) (a : JetU1Algebra) :
    (deriv μ a : SpaceTimeAlgebra) = pderiv μ a
```

def coord [source]

Multiplication of a `u(1)` jet by the coordinate `x_μ`.

```
noncomputable def coord (μ : Fin 1 ⊕ Fin 3) : JetU1Algebra →ₗ[ℝ] JetU1Algebra where
```

lemma coord_val [source]

```
@[simp]
lemma coord_val (μ : Fin 1 ⊕ Fin 3) (a : JetU1Algebra) :
    (coord μ a : SpaceTimeAlgebra) = (X μ : SpaceTimeAlgebra) * a
```

def evalLie [source]

Evaluation of a `u(1)` jet at the base point.

```
noncomputable def evalLie : JetU1Algebra →ₗ[ℝ] U1Algebra where
```

lemma evalLie_val [source]

```
@[simp]
lemma evalLie_val (a : JetU1Algebra) : (evalLie a : ℂ) = constantCoeff (a : SpaceTimeAlgebra)
```

def ofConstantLie [source]

A constant as a `u(1)` jet.

```
noncomputable def ofConstantLie : U1Algebra →ₗ[ℝ] JetU1Algebra where
```

lemma ofConstantLie_val [source]

```
@[simp]
lemma ofConstantLie_val (a : U1Algebra) : (ofConstantLie a : SpaceTimeAlgebra) = C (a : ℂ)
```

lemma star_mcVal [source]

The Maurer–Cartan scalar `i (∂_μ u) u⁻¹` of a unitary jet is self-adjoint.

```
lemma star_mcVal (u : JetU1) (μ : Fin 1 ⊕ Fin 3) :
    star (Complex.I • (pderiv μ (u : SpaceTimeAlgebra) * star (u : SpaceTimeAlgebra)))
      = Complex.I • (pderiv μ (u : SpaceTimeAlgebra) * star (u : SpaceTimeAlgebra))
```

def mc [source]

The Maurer–Cartan form `i (∂_μ u) u⁻¹` of a `U(1)` jet.

```
noncomputable def mc (u : JetU1) (μ : Fin 1 ⊕ Fin 3) : JetU1Algebra
```

lemma mc_val [source]

```
@[simp]
lemma mc_val (u : JetU1) (μ : Fin 1 ⊕ Fin 3) :
    (mc u μ : SpaceTimeAlgebra) = Complex.I •
        (pderiv μ (u : SpaceTimeAlgebra) * star (u : SpaceTimeAlgebra))
```

def u1MatrixJets [source]

The presentation of `U(1)` by `1 × 1` matrices of jets.

```
noncomputable def u1MatrixJets : MatrixJets (Fin 1) U1 U1Algebra JetU1 JetU1Algebra where
```

def u1 [source]

***The local gauge data of `U(1)`**: unitary jets, self-adjoint scalar jets with vanishing bracket and trivial adjoint action, and the Maurer–Cartan form `i (∂_μ u) u⁻¹`.*

```
noncomputable def u1 : LocalGaugeData U1 U1Algebra JetU1 JetU1Algebra
```

lemma u1_iteratedDeriv_val [source]

The iterated derivative on `u(1)` jets is the iterated formal derivative.

```
lemma u1_iteratedDeriv_val (s : Multiset (Fin 1 ⊕ Fin 3)) (a : JetU1Algebra) :
    (u1.iteratedDeriv s a : SpaceTimeAlgebra) =
      SpaceTimeAlgebra.iteratedPDeriv s (a : SpaceTimeAlgebra)
```

instance instFaithfulU1 [source]

The local gauge data of `U(1)` is faithful.

```
instance instFaithfulU1 : u1.Faithful
```

instance instFreeU1 [source]

The local gauge data of `U(1)` is free. Real Taylor data give a self-adjoint jet, and the unitary Euler transport of a `1 × 1` matrix is a unitary jet.

```
instance instFreeU1 : u1.Free
```

def u1Factor [source]

The canonical `U(1)` factor of the local gauge data of `U(1)`.

```
noncomputable def u1Factor : U1Factor u1 where
```

2.236 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.Basic

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/Basic.lean on GitHub.

structure MatterField [source]

***A matter field** of a gauge theory over the gauge context `jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J`: a finite-dimensional complex target space `V`, the Lorentz representation on `V`, a fibrewise action of `GJ` on the jets `SpaceTimeAlgebra ⊗[ℂ] V` of the field, the infinitesimal action of the gauge algebra generating it, the mass weight of the field (in the units in which a derivative has weight `2`), and its declared statistics.*

```
structure MatterField {G₀ : Type} [Group G₀] {𝔤 : Type} [LieRing 𝔤] [LieAlgebra ℝ 𝔤]
    {GJ : Type} [Group GJ] {𝔤J : Type} [LieRing 𝔤J] [LieAlgebra ℝ 𝔤J]
    (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) where
  V : Type
  [instAddCommGroup : AddCommGroup V]
  [instModule : Module ℂ V]
  [instFree : Module.Free ℂ V]
  [instFinite : Module.Finite ℂ V]
  repLorentz : Representation ℂ SL(2,ℂ) V
  repJet : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V)
  repAlgebra : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V
  repJet_smul : ∀ (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V), repJet U (χ •
    z) =
      χ • repJet U z
  repAlgebra_isInfinitesimalAction : jets.IsInfinitesimalActionOf repAlgebra repJet
  massWeight : ℕ
  statistic : FieldStatistic
```

def PureJetsActTrivially [source]

Pure gauge jets act trivially at the base point: a jet with trivial value has identity zeroth Taylor coefficient on the value space. With the fibrewise law, the base-point value of the jet action is then a function of the value of the jet.

```
def PureJetsActTrivially : Prop
```

def GaugeLorentzCompatible [source]

The infinitesimal gauge action commutes with the Lorentz representation on the value space.

```
def GaugeLorentzCompatible : Prop
```

2.237 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.Charge

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/Charge.lean on GitHub.

def chargePow [source]

The unitary power series `U ^ n` of a `U(1)` jet, for an integer charge `n`.

```
noncomputable def chargePow (n : ℤ) (U : unitary SpaceTimeAlgebra) : SpaceTimeAlgebra
```

lemma chargePow_one [source]

```
lemma chargePow_one (n : ℤ) : chargePow n 1 = 1
```

lemma chargePow_mul [source]

```
lemma chargePow_mul (n : ℤ) (U W : unitary SpaceTimeAlgebra) :
    chargePow n (U * W) = chargePow n U * chargePow n W
```

def chargeRep [source]

***The charge-`n` action of `U(1)` jets on the jets of a `V`-valued field**: multiplication of the jet-ring factor by `U ^ n`.*

```
noncomputable def chargeRep (n : ℤ) (V : Type) [AddCommGroup V] [Module ℂ V] :
    Representation ℂ (unitary SpaceTimeAlgebra) (SpaceTimeAlgebra ⊗[ℂ] V) where
```

lemma chargeRep_tmul [source]

```
lemma chargeRep_tmul (n : ℤ) (U : unitary SpaceTimeAlgebra) (f : SpaceTimeAlgebra) (v : V) :
    chargeRep n V U (f ⊗ₜ[ℂ] v) = (chargePow n U * f) ⊗ₜ[ℂ] v
```

lemma chargeRep_smul [source]

***The charge action is fibrewise**: it commutes with multiplication by scalar jets.*

```
lemma chargeRep_smul (n : ℤ) (U : unitary SpaceTimeAlgebra) (χ : SpaceTimeAlgebra)
    (z : SpaceTimeAlgebra ⊗[ℂ] V) :
    chargeRep n V U (χ • z) = χ • chargeRep n V U z
```

def charged [source]

***The charged matter field**: a field with values in `V`, Lorentz representation `repLorentz`, electric charge `n`, mass weight `w` and declared statistic `σ`, as a matter field for the jets of `U(1)`, in any gauge context `jets` whose jet group is `unitary SpaceTimeAlgebra`.*

The infinitesimal action `act` of the gauge algebra is supplied, not constructed. For a charge-`n` field it is `c ↦ (i n φ(c)) • id` for the functional `φ` reading off the `u(1)` component of `c`, and no such functional is available: `jets` relates the gauge algebra `𝔤` to the jets only through `evalLie` and `maurerCartan`, neither of which identifies a `u(1)` direction in an arbitrary `𝔤`. The rest of the data does not depend on `jets` beyond its jet group, so it is supplied polymorphically.

```
noncomputable def charged {𝔤 : Type} [LieRing 𝔤] [LieAlgebra ℝ 𝔤]
    {G₀ : Type} [Group G₀] {𝔤J : Type} [LieRing 𝔤J] [LieAlgebra ℝ 𝔤J]
    (jets : LocalGaugeData G₀ 𝔤 (unitary SpaceTimeAlgebra) 𝔤J) [Module.Free ℂ V] [Module.Finite
     ℂ V]
```

```
    (repLorentz : Representation ℂ SL(2,ℂ) V) (n : ℤ) (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (hact : jets.IsInfinitesimalActionOf act (chargeRep n V)) (w : ℕ) (σ : FieldStatistic) :
    MatterField jets where
```

lemma charged_V [source]

```
@[simp]
lemma charged_V {𝔤 : Type} [LieRing 𝔤] [LieAlgebra ℝ 𝔤]
    {G₀ : Type} [Group G₀] {𝔤J : Type} [LieRing 𝔤J] [LieAlgebra ℝ 𝔤J]
    (jets : LocalGaugeData G₀ 𝔤 (unitary SpaceTimeAlgebra) 𝔤J) [Module.Free ℂ V] [Module.Finite
     ℂ V]
    (repLorentz : Representation ℂ SL(2,ℂ) V) (n : ℤ) (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (hact : jets.IsInfinitesimalActionOf act (chargeRep n V)) (w : ℕ) (σ : FieldStatistic) :
    (charged jets repLorentz n act hact w σ).V = V
```

2.238 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.CovariantDeriv

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/CovariantDeriv.lean on GitHub.

def repCoeff [source]

The base-point Taylor coefficient of the representation: include the constant vector into `V`-valued jets, act by `rep U`, differentiate `x` times, evaluate at the base point. The composite is complex-linear: the physicists' `∂_x (rep U)^i_j|₀` as a ℂ-linear map on the value space.

```
noncomputable def repCoeff (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) : V →ₗ[ℂ] V
```

def repDualCoeff [source]

The physicists' `∂_x (rep U)^j_i|₀` acting on the complex dual index of a matter-field symbol: the transpose of `repCoeff`. This is the analogue of `adjointDualCoeff` for a general representation of the jet gauge group; for `x = 0` it is the dual (contragredient) action of the value of `U`, and for `x ≠ 0` it sees the derivatives of the gauge transformation.

```
noncomputable def repDualCoeff (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ V →ₗ[ℂ] Module.Dual ℂ V
```

lemma jetEval_rep_of_smul [source]

For a fibrewise representation, evaluating the transform of a jet at the base point is the zeroth Taylor coefficient of the transform of its base-point value: the jet ring factor passes through the action and is then evaluated.

```
lemma jetEval_rep_of_smul (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (hlin : ∀ (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V), rep U
        (χ • z) = χ • rep U z)
    (U : GJ) (z : SpaceTimeAlgebra ⊗[ℂ] V) :
    jetEval (rep U z) = repCoeff rep U 0 (jetEval z)
```

lemma repCoeff_zero_mul [source]

The zeroth Taylor coefficients of a fibrewise representation are multiplicative: they form a representation of the jet gauge group on the value space.

```
lemma repCoeff_zero_mul (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (hlin : ∀ (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V), rep U
        (χ • z) = χ • rep U z)
    (U W : GJ) : repCoeff rep (U * W) 0 = repCoeff rep U 0 ∘ₗ repCoeff rep W 0
```

def tensorAction [source]

The action of an adjoint-valued field on a matter field at the tensor level: multiplication in `B` on the first factors, the $\mathbb{C}$-linear infinitesimal action `act` of the gauge algebra on `V` on the second, so that on pure tensors `(b₁ ⊗ c) · (b₂ ⊗ v) = (b₁ b₂) ⊗ act c v`.

```
noncomputable def tensorAction (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V) :
    (B ⊗[ℝ] 𝔤) →ₗ[ℝ] (B ⊗[ℂ] V) →ₗ[ℂ] B ⊗[ℂ] V
```

lemma tensorAction_tmul [source]

```
@[simp]
lemma tensorAction_tmul (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V) (b₁ b₂ : B)
    (c : 𝔤) (v : V) :
    tensorAction act (b₁ ⊗ₜ[ℝ] c) (b₂ ⊗ₜ[ℂ] v) = (b₁ * b₂) ⊗ₜ[ℂ] act c v
```

lemma tensorAction_map_left [source]

```
lemma tensorAction_map_left (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V) (Φ : B →ₗ[ℂ] B)
    (hΦ : ∀ b₁ b₂, Φ (b₁ * b₂) = Φ b₁ * Φ b₂) (s : B ⊗[ℝ] 𝔤)
    (t : B ⊗[ℂ] V) :
    tensorAction act ((TensorProduct.map (Φ.restrictScalars ℝ) LinearMap.id) s)
        ((TensorProduct.map Φ LinearMap.id) t) =
      (TensorProduct.map Φ LinearMap.id) (tensorAction act s t)
```

lemma tensorAction_one_left [source]

```
lemma tensorAction_one_left (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V) (c : 𝔤)
    (t : B ⊗[ℂ] V) :
    tensorAction act ((1 : B) ⊗ₜ[ℝ] c) t =
      (TensorProduct.map LinearMap.id (act c)) t
```

lemma tensorAction_map_right_antidiagonal [source]

`tensorAction` under an antidiagonal pair of transport families: if the `V`-transports intertwine `act` with the $\mathfrak{g}$-transports as an antidiagonal convolution, so do `id ⊗ ·` over `tensorAction`.

```
lemma tensorAction_map_right_antidiagonal (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (Tg : Multiset (Fin 1 ⊕ Fin 3) → 𝔤 →ₗ[ℝ] 𝔤)
    (Tv : Multiset (Fin 1 ⊕ Fin 3) → V →ₗ[ℂ] V) (x : Multiset (Fin 1 ⊕ Fin 3))
    (hT : ∀ (c : 𝔤) (w : V), Tv x (act c w) =
      (x.antidiagonal.map fun p => act (Tg p.1 c) (Tv p.2 w)).sum)
    (s : B ⊗[ℝ] 𝔤) (t : B ⊗[ℂ] V) :
```

```
    (x.antidiagonal.map fun p =>
      tensorAction act ((TensorProduct.map LinearMap.id (Tg p.1)) s)
        ((TensorProduct.map LinearMap.id (Tv p.2)) t)).sum =
      (TensorProduct.map LinearMap.id (Tv x)) (tensorAction act s t)
```

def dualPairEquivC [source]

The canonical equivalence between matter fields `B ⊗[ℂ] V` and their component families `φ ↦ F^φ` over the complex dual — `dualPairEquiv` for a general finite-dimensional complex value space.

```
noncomputable def dualPairEquivC : (B ⊗[ℂ] V) ≃ₗ[ℂ] (Module.Dual ℂ V →ₗ[ℂ] B)
```

lemma dualPairEquivC_tmul [source]

```
@[simp]
lemma dualPairEquivC_tmul (b : B) (v : V) (φ : Module.Dual ℂ V) :
    dualPairEquivC (b ⊗ₜ[ℂ] v) φ = φ v • b
```

lemma dualPairEquivC_map_left [source]

```
lemma dualPairEquivC_map_left (Φ : B →ₗ[ℂ] B) (t : B ⊗[ℂ] V)
    (φ : Module.Dual ℂ V) :
    dualPairEquivC ((TensorProduct.map Φ LinearMap.id) t) φ =
      Φ (dualPairEquivC t φ)
```

lemma dualPairEquivC_map_right [source]

```
lemma dualPairEquivC_map_right (T : V →ₗ[ℂ] V) (t : B ⊗[ℂ] V)
    (φ : Module.Dual ℂ V) :
    dualPairEquivC ((TensorProduct.map LinearMap.id T) t) φ =
      dualPairEquivC t (T.dualMap φ)
```

lemma symm_comp_left_C [source]

```
lemma symm_comp_left_C (Φ : B →ₗ[ℂ] B) (g : Module.Dual ℂ V →ₗ[ℂ] B) :
    dualPairEquivC.symm (Φ ∘ₗ g) =
      (TensorProduct.map Φ LinearMap.id) (dualPairEquivC.symm g)
```

lemma symm_comp_right_C [source]

```
lemma symm_comp_right_C (T : V →ₗ[ℂ] V) (g : Module.Dual ℂ V →ₗ[ℂ] B) :
    dualPairEquivC.symm (g ∘ₗ T.dualMap) =
      (TensorProduct.map LinearMap.id T) (dualPairEquivC.symm g)
```

def actionFam [source]

The action of an adjoint-indexed component family on a matter one, through the infinitesimal action `act`: assemble both into fields, act by `tensorAction`, read back out as components. This is the physicists' `f^a (T_a)^i_j g^j` with `T = act`, basis-free.

```
noncomputable def actionFam (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) (g : Module.Dual ℂ V →ₗ[ℂ] B) :
```

```
    Module.Dual ℂ V →ₗ[ℂ] B
```

lemma actionFam_add_left [source]

```
lemma actionFam_add_left (f₁ f₂ : Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (g : Module.Dual ℂ V →ₗ[ℂ] B) :
    actionFam act (f₁ + f₂) g = actionFam act f₁ g + actionFam act f₂ g
```

lemma actionFam_add_right [source]

```
lemma actionFam_add_right (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (g₁ g₂ : Module.Dual ℂ V →ₗ[ℂ] B) :
    actionFam act f (g₁ + g₂) = actionFam act f g₁ + actionFam act f g₂
```

lemma actionFam_zero_left [source]

```
lemma actionFam_zero_left (g : Module.Dual ℂ V →ₗ[ℂ] B) :
    actionFam act 0 g = 0
```

lemma actionFam_zero_right [source]

```
lemma actionFam_zero_right (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    actionFam act f 0 = 0
```

lemma actionFam_sum_left [source]

```
lemma actionFam_sum_left (S : Multiset (Module.Dual ℝ 𝔤 →ₗ[ℝ] B))
    (g : Module.Dual ℂ V →ₗ[ℂ] B) :
    actionFam act S.sum g = (S.map fun f => actionFam act f g).sum
```

lemma actionFam_sum_right [source]

```
lemma actionFam_sum_right (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (S : Multiset (Module.Dual ℂ V →ₗ[ℂ] B)) :
    actionFam act f S.sum = (S.map fun g => actionFam act f g).sum
```

lemma repGauge_actionFam [source]

The gauge transformation of the action of an affinely-transforming adjoint-indexed family on a linearly-transforming matter family: the action of the transformed families plus one `act`-type cross term. This is `repGauge_bracketFam` with a homogeneous second slot and the bracket replaced by a general action.

```
lemma repGauge_actionFam
    (U : GJ) {f f' : Module.Dual ℝ 𝔤 →ₗ[ℝ] B}
    {g g' : Module.Dual ℂ V →ₗ[ℂ] B} {cf : 𝔤}
    (hf : ∀ ψ : Module.Dual ℝ 𝔤,
      repGauge U (f ψ) = f' ψ + algebraMap ℂ B (ψ cf))
    (hg : ∀ ψ : Module.Dual ℂ V, repGauge U (g ψ) = g' ψ)
    (φ : Module.Dual ℂ V) :
    repGauge U (actionFam act f g φ) =
      actionFam act f' g' φ + g' (φ ∘ₗ act cf)
```

def actionFamConv [source]

The derived action family `A_ρ · F`: the `s`-derivative of the action of the gauge field on a matter family, given by the Leibniz convolution of the derivative symbols over the multiset antidiagonal — the matter analogue of `bracketFamConv`.

```
noncomputable def actionFamConv
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V) (ρ : Fin 1 ⊕ Fin 3)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) : Module.Dual ℂ V →ₗ[ℂ] B
```

def covDerivAction [source]

The covariant derivative `∇_ρ F = [∂_ρ F] + A_ρ · F` of a matter family of derivative symbols, in the single direction `ρ`: the extra derivative on the symbol plus the derived action of the gauge field on the value index. With the physicists' factor of `i` absorbed into `act` (as it is in the gauge-algebra bracket), this is `∂_ρ F + i A_ρ^a T_a F` in the `D = ∂ + i A` convention.

```
noncomputable def covDerivAction
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) : Module.Dual ℂ V →ₗ[ℂ] B
```

lemma covDerivAction_apply [source]

```
@[simp]
lemma covDerivAction_apply
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    covDerivAction A act F ρ s φ = F (ρ ::ₘ s) φ + actionFamConv A act ρ F s φ
```

def covDerivIter [source]

***The iterated covariant derivative** `∇_{l 0} ⋯ ∇_{l (n-1)} F` of a matter family along an ordered tuple of directions: covariant derivatives do not commute (their commutator is the action of the field strength), so the iteration is order-dependent and indexed by `(n : ℕ)` and `l : Fin n → (Fin 1 ⊕ Fin 3)` — the same ordered-tuple indexing as the derivative labels of `IsHiggsAlgebraValued`. The result is again a family of derivative symbols; the physical iterated covariant derivative is its value at the empty multiset.*

```
noncomputable def covDerivIter
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B) :
    (n : ℕ) → (Fin n → (Fin 1 ⊕ Fin 3)) →
      Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B
  | 0, _ => F
  | n + 1, l => covDerivAction A act (covDerivIter A act F n fun i => l i.succ) (l 0)
```

lemma covDerivIter_zero [source]

```
@[simp]
lemma covDerivIter_zero (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (l : Fin 0 → (Fin 1 ⊕ Fin 3)) :
    covDerivIter A act F 0 l = F
```

lemma covDerivIter_succ [source]

```
@[simp]
lemma covDerivIter_succ (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    {n : ℕ} (l : Fin (n + 1) → (Fin 1 ⊕ Fin 3)) :
    covDerivIter A act F (n + 1) l =
      covDerivAction A act (covDerivIter A act F n fun i => l i.succ) (l 0)
```

lemma dualPairEquiv_symm_eq_sum [source]

Decomposition of an assembled adjoint-indexed family along a basis of the gauge algebra: the components against the dual basis, tensored with the basis vectors.

```
lemma dualPairEquiv_symm_eq_sum {ι : Type*} [Fintype ι]
    (bW : Module.Basis ι ℝ 𝔤)
    (g : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) :
    dualPairEquiv.symm g = ∑ i, g (bW.coord i) ⊗ₜ[ℝ] bW i
```

lemma dualPairEquivC_symm_eq_sum [source]

Decomposition of an assembled matter family along a basis of the value space.

```
lemma dualPairEquivC_symm_eq_sum {ι : Type*} [Fintype ι] (bW : Module.Basis ι ℂ V)
    (g : Module.Dual ℂ V →ₗ[ℂ] B) :
    dualPairEquivC.symm g = ∑ i, g (bW.coord i) ⊗ₜ[ℂ] bW i
```

lemma actionFam_apply_mem [source]

The value of an action of families lies in any subalgebra containing the values of both families: the action is a finite sum of products of components.

```
lemma actionFam_apply_mem {act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V} {P : Subalgebra ℂ B}
    {f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B} {g : Module.Dual ℂ V →ₗ[ℂ] B}
    (hf : ∀ ψ, f ψ ∈ P) (hg : ∀ χ, g χ ∈ P) (φ : Module.Dual ℂ V) :
    actionFam act f g φ ∈ P
```

lemma covDerivIter_sub_mem [source]

***Unitriangularity of the covariant matter tower**: the covariant and plain derivative symbols of a matter family differ by an element of the subalgebra generated by the gauge-field symbols and the strictly lower-order matter symbols. Stated at every derivative multiset `s`, as needed for the induction.*

```
lemma covDerivIter_sub_mem (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3))
```

```
    (φ : Module.Dual ℂ V) :
    covDerivIter A act F n l s φ - F (List.ofFn l + s) φ ∈
      Algebra.adjoin ℂ
        ({b : B | ∃ (u : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
            (ψ : Module.Dual ℝ 𝔤), b = A u μ ψ} ∪
          {b : B | ∃ (t : Multiset (Fin 1 ⊕ Fin 3)) (χ : Module.Dual ℂ V),
            t.card < n + s.card ∧ b = F t χ})
```

lemma covDerivIter_mem_adjoin_symbols [source]

Every derivative symbol of the covariant tower is a polynomial in the gauge-field symbols and the matter symbols.

```
lemma covDerivIter_mem_adjoin_symbols (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ V) :
    covDerivIter A act F n l s φ ∈ Algebra.adjoin ℂ
      ({b : B | ∃ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
          (ψ : Module.Dual ℝ 𝔤), b = A s μ ψ} ∪
        {b : B | ∃ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V),
          b = F s φ})
```

lemma adjoin_symbols_eq_adjoin_covDerivIter [source]

***The span lemma**: the algebra of symbols generated by the gauge field together with a matter family's *derivative* symbols equals the one generated by the gauge field together with the matter family's *covariant* derivative tower. The correction ∇_ρ − ∂_ρ is the derived action of the gauge field — a sum of products of symbols, absorbed by the algebra structure.*

```
lemma adjoin_symbols_eq_adjoin_covDerivIter (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B) :
    Algebra.adjoin ℂ
      ({b : B | ∃ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
          (ψ : Module.Dual ℝ 𝔤), b = A s μ ψ} ∪
        {b : B | ∃ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V),
          b = F s φ}) =
    Algebra.adjoin ℂ
      ({b : B | ∃ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
          (ψ : Module.Dual ℝ 𝔤), b = A s μ ψ} ∪
        {b : B | ∃ (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V),
          b = covDerivIter A act F n l 0 φ})
```

lemma actionFam_map [source]

The action of an adjoint-indexed family on a matter family is natural in the algebra.

```
lemma actionFam_map (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V) (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (g : Module.Dual ℂ V →ₗ[ℂ] B) :
    actionFam act (Φ.restrictScalars ℝ ∘ₗ f) (Φ ∘ₗ g) = Φ ∘ₗ actionFam act f g
```

lemma actionFamConv_map [source]

The derived action family is natural in the algebra.

```
lemma actionFamConv_map
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V) (ρ : Fin 1 ⊕ Fin 3)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    actionFamConv (fun p σ => Φ.restrictScalars ℝ ∘ₗ A p σ) act ρ (fun p => Φ ∘ₗ F p) s
      = Φ ∘ₗ actionFamConv A act ρ F s
```

lemma covDerivAction_map [source]

The covariant derivative of a matter family is natural in the algebra.

```
lemma covDerivAction_map
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    covDerivAction (fun p σ => Φ.restrictScalars ℝ ∘ₗ A p σ) act (fun p => Φ ∘ₗ F p) ρ s
      = Φ ∘ₗ covDerivAction A act F ρ s
```

lemma covDerivIter_map [source]

The iterated covariant derivative of a matter family is natural in the algebra: the image of the tower is the tower of the image families, at every ordered tuple of directions and every derivative multiset.

```
lemma covDerivIter_map
    (A : Multiset (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    covDerivIter (fun p σ => Φ.restrictScalars ℝ ∘ₗ A p σ) act (fun p => Φ ∘ₗ F p) n l
      = fun s => Φ ∘ₗ covDerivIter A act F n l s
```

2.239 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.InfinitesimalAction

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/InfinitesimalAction.lean on GitHub.

structure IsInfinitesimalActionOf [source]

*`act` is the *infinitesimal action* of the gauge algebra underlying the representation `rep` of the jet gauge group, when the base-point Taylor coefficients of `rep` satisfy the two laws forced by `rep` being generated by `act`:*

** `repCoeff_cons` — the Leibniz rule in the Maurer–Cartan form: differentiating the representation once produces minus the action of the Maurer–Cartan form, with the remaining derivatives distributed over the antidiagonal (for the adjoint representation this is `LocalGaugeData.adjointCoeff_cons`); * `repCoeff_act` — the transports of `rep` intertwine `act` with the adjoint transports, as an antidiagonal convolution (for the adjoint representation this is `LocalGaugeData.adjointCoeff_lie`); at `x = 0` it is the classical equivariance `rep(U)|₀ ∘ act c = act (Ad(U) c)|₀ ∘ rep(U)|₀`.*

These are exactly the identities consumed by the proof that the covariant derivative `covDerivAction` preserves `TransformsIn`.

```
structure IsInfinitesimalActionOf (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J)
    (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V)
    (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V)) : Prop where
  repCoeff_cons : ∀ (U : GJ) (μ : Fin 1 ⊕ Fin 3) (x : Multiset (Fin 1 ⊕ Fin 3)),
    repCoeff rep U (μ ::ₘ x) =
      -((x.antidiagonal.map fun p =>
        act (jets.evalLie (jets.iteratedDeriv p.1 (jets.maurerCartan U μ))) ∘ₗ
          repCoeff rep U p.2).sum)
  repCoeff_act : ∀ (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) (c : 𝔤),
    repCoeff rep U x ∘ₗ act c =
      ((x.antidiagonal.map fun p =>
        act (jets.adjointCoeff U p.1 c) ∘ₗ repCoeff rep U p.2).sum)
```

lemma repDualCoeff_cons [source]

The dual form of the Leibniz law: the once-more-derived dual coefficient is minus the antidiagonal convolution of dual coefficients against `act` of the derived Maurer–Cartan form — the analogue of `LocalGaugeData.adjointDualCoeff_cons`.

```
lemma IsInfinitesimalActionOf.repDualCoeff_cons
    {act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V}
    {rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V)}
    (h : IsInfinitesimalActionOf jets act rep) (U : GJ) (μ : Fin 1 ⊕ Fin 3)
    (x : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    repDualCoeff rep U (μ ::ₘ x) φ =
      -((x.antidiagonal.map fun p =>
        repDualCoeff rep U p.2 (φ ∘ₗ act (jets.evalLie
          (jets.iteratedDeriv p.1 (jets.maurerCartan U μ))))).sum)
```

lemma actionFam_repDualCoeff [source]

The action of families against the dual representation coefficients: the antidiagonal convolution mixing the adjoint transport on the field slot with the representation transport on the matter slot — the family-level form of `IsInfinitesimalActionOf.repCoeff_act`, and the analogue of `bracketFam_adjointDualCoeff`.

```
lemma IsInfinitesimalActionOf.actionFam_repDualCoeff
    (h : IsInfinitesimalActionOf jets act rep) (U : GJ)
    (x : Multiset (Fin 1 ⊕ Fin 3)) (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B)
    (g : Module.Dual ℂ V →ₗ[ℂ] B) (φ : Module.Dual ℂ V) :
    actionFam act f g (repDualCoeff rep U x φ) =
      (x.antidiagonal.map fun p =>
        actionFam act (f ∘ₗ jets.adjointDualCoeff U p.1)
          (g ∘ₗ repDualCoeff rep U p.2) φ).sum
```

lemma repGauge_cons [source]

If `F` transforms in `rep`, so do its `κ ::ₘ s`-derived symbols, with the extra derivative traced through `IsInfinitesimalActionOf.repDualCoeff_cons`: the Leibniz splittings where `κ` stays a derivative, minus the convolution where `κ` hits the representation — `act` of the derived Maurer–Cartan form.

```
lemma TransformsIn.repGauge_cons
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B}
    (hF : TransformsIn repGauge rep F)
```

```
    (hact : IsInfinitesimalActionOf jets act rep)
    (U : GJ) (κ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ V) :
    repGauge U (F (κ ::ₘ s) φ) =
      (s.antidiagonal.map fun p =>
        F (κ ::ₘ p.2) (repDualCoeff rep U⁻¹ p.1 φ)).sum
      - (s.antidiagonal.map fun p =>
          (p.1.antidiagonal.map fun q =>
            F p.2 (repDualCoeff rep U⁻¹ q.2
              (φ ∘ₗ act (jets.evalLie (jets.iteratedDeriv q.1
                (jets.maurerCartan U⁻¹ κ)))))).sum).sum
```

lemma repGauge_actionFamConv [source]

The all-orders gauge transformation of the derived action `A_ρ · F` for `F` transforming in `rep`: since `F` transforms homogeneously, only one cross-term convolution through `act` survives — the analogue of `TransformsInAdjoint.repGauge_bracketFamConv` with a matter field in the second slot.

```
lemma TransformsIn.repGauge_actionFamConv
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B}
    (hF : TransformsIn repGauge rep F)
    (hact : IsInfinitesimalActionOf jets act rep)
    (U : GJ) (s : Multiset (Fin 1 ⊕ Fin 3)) (ρ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ V) :
    repGauge U (actionFamConv h.A act ρ F s φ) =
      (s.antidiagonal.map fun p =>
        actionFamConv h.A act ρ F p.2 (repDualCoeff rep U⁻¹ p.1 φ)).sum
      + (s.antidiagonal.map fun p =>
          (p.2.antidiagonal.map fun r =>
            F r.2 (repDualCoeff rep U⁻¹ r.1
              (φ ∘ₗ act (jets.evalLie (jets.iteratedDeriv p.1
                (jets.maurerCartan U⁻¹ ρ)))))).sum).sum
```

lemma covDerivAction [source]

The covariant derivative preserves `TransformsIn`: if `F` transforms in the representation `rep` and `act` is the infinitesimal action underlying `rep`, then `∇_ρ F = [∂_ρ F] + A_ρ · F` transforms in `rep`. The single inhomogeneous convolution of `[∂_{ρ ::ₘ s} F]` cancels the single `act` cross-term convolution of `A_ρ · F` through the coassociativity of the antidiagonal — the matter-field analogue of `TransformsInAdjoint.covDerivAdjoint`.

```
lemma TransformsIn.covDerivAction
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B}
    (hF : TransformsIn repGauge rep F)
    (hact : IsInfinitesimalActionOf jets act rep) (ρ : Fin 1 ⊕ Fin 3) :
    TransformsIn repGauge rep (GaugeAlgebraRealization.covDerivAction h.A act F ρ)
```

lemma covDerivIter [source]

Every iterated covariant derivative preserves `TransformsIn`: if `F` transforms in `rep` and `act` is the infinitesimal action underlying `rep`, then `∇_{l 0} ⋯ ∇_{l (n-1)} F` transforms in `rep` — the recursion of `TransformsIn.covDerivAction` over the tuple of directions.

```
lemma TransformsIn.covDerivIter
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B}
    (hF : TransformsIn repGauge rep F)
    (hact : IsInfinitesimalActionOf jets act rep)
    (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    TransformsIn repGauge rep (GaugeAlgebraRealization.covDerivIter h.A act F n l)
```

def actionConj [source]

The conjugate of an infinitesimal action: the same maps, read on the conjugate module — the generators of the conjugate representation.

```
noncomputable def actionConj (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V) :
    𝔤 →ₗ[ℝ] ConjModule V →ₗ[ℂ] ConjModule V where
```

lemma actionConj_apply [source]

```
@[simp]
lemma actionConj_apply (act : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V) (c : 𝔤) :
    actionConj act c = ConjModule.endConj (act c)
```

def conjJetEquiv [source]

The identification of the jets of a conjugate field with the conjugates of the jets: conjugation is monoidal, and the star of the jet-ring factor absorbs the twist — `conj (g ⊗ u) ↦ star g ⊗ conj u`. This is the equivalence along which `JetComponentSpace.repConj` carries the conjugated representation.

```
noncomputable def conjJetEquiv :
    ConjModule (SpaceTimeAlgebra ⊗[ℂ] V) ≃ₗ[ℂ] SpaceTimeAlgebra ⊗[ℂ] ConjModule V
```

lemma conjJetEquiv_conjEquiv_tmul [source]

```
lemma conjJetEquiv_conjEquiv_tmul (g : SpaceTimeAlgebra) (u : V) :
    conjJetEquiv (conjEquiv (k := ℂ) (M := SpaceTimeAlgebra ⊗[ℂ] V) (g ⊗ₜ[ℂ] u))
      = star g ⊗ₜ[ℂ] conjEquiv (k := ℂ) (M := V) u
```

lemma repConj_conjJetEquiv [source]

The conjugate representation acts through `conjJetEquiv` by the original maps.

```
lemma repConj_conjJetEquiv (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (U : GJ) (w : SpaceTimeAlgebra ⊗[ℂ] V) :
    JetComponentSpace.repConj rep U
        (conjJetEquiv (conjEquiv (k := ℂ) (M := SpaceTimeAlgebra ⊗[ℂ] V) w))
      = conjJetEquiv (conjEquiv (k := ℂ) (M := SpaceTimeAlgebra ⊗[ℂ] V) (rep U w))
```

lemma repCoeff_repConj [source]

The base-point Taylor coefficients of the conjugate representation are the conjugated coefficients: the derivative directions are real, so conjugation passes through `∂_x` and the base-point evaluation untouched.

```
lemma repCoeff_repConj (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    repCoeff (JetComponentSpace.repConj rep) U x
      = ConjModule.endConj (repCoeff rep U x)
```

lemma repCoeff_repConj_zero_eq_id [source]

The base-point triviality of the zeroth Taylor coefficient passes to the conjugate representation.

```
lemma repCoeff_repConj_zero_eq_id {W : GJ}
    (hrep : repCoeff rep W 0 = LinearMap.id) :
    repCoeff (JetComponentSpace.repConj rep) W 0 = LinearMap.id
```

lemma conj [source]

The conjugate of an infinitesimal action underlies the conjugate representation: conjugating the Taylor coefficients preserves both the Maurer–Cartan Leibniz law and the adjoint intertwining, since the gauge-algebra inputs are real.

```
lemma IsInfinitesimalActionOf.conj (h : IsInfinitesimalActionOf jets act rep) :
    IsInfinitesimalActionOf jets (actionConj act) (JetComponentSpace.repConj rep)
```

2.240 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.JetComponent.Basic

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/JetComponent/Basic.lean on GitHub.

abbrev JetComponentSpace [source]

The space of component functions of the matter field `M`: the span of the symbols `∂_s ψ_α` and their conjugates `∂_s ψ̄_α`, where `α` runs over the target space `M.V`. The first factor holds the unconjugated symbols, the second the conjugate ones; in each, `SpaceTimeDerivAlgebraℂ` carries the derivative label `s` and the dual factor the target component `α`.

Only `M.V` enters the space itself; the field's Lorentz and gauge representations and its mass weight enter the structure carried on it below. Taking the whole matter field rather than its value space is what lets that structure be read off `M` instead of being supplied by hand at each use.

```
abbrev JetComponentSpace (M : MatterField jets) : Type
```

def repLorentzGroup [source]

***The Lorentz action on the jet component space.** Under a Lorentz transformation a matter field transforms as `ψ(x) ↦ ρ(Λ) ψ(Λ⁻¹ x)`, so a derivative symbol `∂_s ψ_α` is acted on in *both* of its labels: the derivative multiset `s` by the Lorentz action on covectors, extended to `SpaceTimeDerivAlgebraℂ`, and the target index `α` by the contragredient of `ρ`.*

Unlike the gauge action, this needs no fibrewise-linearity or finite-dimensionality hypothesis: the two labels transform independently, so the action is simply a tensor

product of representations. The conjugate half is the same with ρ replaced by its conjugate, the symbols `∂_s ψ¯_α` transforming by `star` of the spinor matrix.

```
noncomputable def JetComponentSpace.repLorentzGroup (M : MatterField jets) :
    Representation ℂ SL(2,ℂ) (JetComponentSpace M)
```

lemma repLorentzGroup_fst [source]

```
@[simp]
lemma JetComponentSpace.repLorentzGroup_fst (Λ : SL(2,ℂ)) (x : JetComponentSpace M) :
    (JetComponentSpace.repLorentzGroup M Λ x).1
      = (SpaceTimeDerivAlgebraℂ.repLorentzGroup.tprod M.repLorentz.dual) Λ x.1
```

lemma repLorentzGroup_snd [source]

```
@[simp]
lemma JetComponentSpace.repLorentzGroup_snd (Λ : SL(2,ℂ)) (x : JetComponentSpace M) :
    (JetComponentSpace.repLorentzGroup M Λ x).2
      = (SpaceTimeDerivAlgebraℂ.repLorentzGroup.tprod M.repLorentz.conj.dual) Λ x.2
```

lemma repLorentzGroup_fst_tmul [source]

On a pure symbol the Lorentz action is diagonal in the two labels: the derivative label transforms in `SpaceTimeDerivAlgebraℂ`, the target index contragrediently.

```
lemma JetComponentSpace.repLorentzGroup_fst_tmul (Λ : SL(2,ℂ)) (a : SpaceTimeDerivAlgebraℂ)
    (φ : Module.Dual ℂ M.V)
    (y : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule M.V)) :
    (JetComponentSpace.repLorentzGroup M Λ (a ⊗ₜ[ℂ] φ, y)).1
      = SpaceTimeDerivAlgebraℂ.repLorentzGroup Λ a ⊗ₜ[ℂ] (φ ∘ₗ M.repLorentz Λ⁻¹)
```

def jetDeriv [source]

the derivative of components in the jet component space, in the direction `μ`: the shift `∂_s ψ_α ↦ ∂_{s + {μ}} ψ_α` of the derivative label, and likewise on the conjugate components.

This is right multiplication by the degree-one element `∂_μ` on the `SpaceTimeDerivAlgebraℂ` factor, leaving the target index untouched. It uses a basis of the Lorentz covectors — that is what the index `μ` is — but no basis of `V`.

```
noncomputable def JetComponentSpace.jetDeriv (μ : Fin 1 ⊕ Fin 3) :
    JetComponentSpace M →ₗ[ℂ] JetComponentSpace M
```

lemma jetDeriv_fst_tmul [source]

```
@[simp]
lemma JetComponentSpace.jetDeriv_fst_tmul (μ : Fin 1 ⊕ Fin 3)
    (a : SpaceTimeDerivAlgebraℂ) (φ : Module.Dual ℂ M.V)
    (y : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule M.V)) :
    (JetComponentSpace.jetDeriv μ (a ⊗ₜ[ℂ] φ, y)).1
      = (a * SpaceTimeDerivAlgebraℂ.basis ({μ} : Multiset (Fin 1 ⊕ Fin 3))) ⊗ₜ[ℂ] φ
```

lemma jetDeriv_snd_tmul [source]

```
@[simp]
lemma JetComponentSpace.jetDeriv_snd_tmul (μ : Fin 1 ⊕ Fin 3)
    (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ M.V)
    (a : SpaceTimeDerivAlgebraℂ) (φ : Module.Dual ℂ (ConjModule M.V)) :
    (JetComponentSpace.jetDeriv μ (x, a ⊗ₜ[ℂ] φ)).2
      = (a * SpaceTimeDerivAlgebraℂ.basis ({μ} : Multiset (Fin 1 ⊕ Fin 3))) ⊗ₜ[ℂ] φ
```

lemma jetDeriv_comm [source]

***Total derivatives commute.** Mixed partials agree because the derivative labels live in a *symmetric* algebra; no basis of `V` is involved.*

```
lemma JetComponentSpace.jetDeriv_comm (μ ν : Fin 1 ⊕ Fin 3) :
    (JetComponentSpace.jetDeriv (M := M) μ).comp (JetComponentSpace.jetDeriv ν)
      = (JetComponentSpace.jetDeriv (M := M) ν).comp (JetComponentSpace.jetDeriv μ)
```

lemma jetDeriv_eq_ι [source]

The element being multiplied in is the degree-one derivative symbol `∂_μ`, the image of the dual basis covector under `SymmetricAlgebra.ι`.

```
lemma JetComponentSpace.jetDeriv_eq_ι (μ : Fin 1 ⊕ Fin 3) :
    JetComponentSpace.jetDeriv (M := M) μ
      = LinearMap.prodMap
        (TensorProduct.map
          (LinearMap.mulRight ℂ (SymmetricAlgebra.ι ℂ (Module.Dual ℂ Lorentz.ComplexCoVector)
            (Lorentz.ComplexCoVector.basis.dualBasis μ))) LinearMap.id)
        (TensorProduct.map
          (LinearMap.mulRight ℂ (SymmetricAlgebra.ι ℂ (Module.Dual ℂ Lorentz.ComplexCoVector)
            (Lorentz.ComplexCoVector.basis.dualBasis μ))) LinearMap.id)
```

lemma jetDeriv_fst [source]

```
@[simp]
lemma JetComponentSpace.jetDeriv_fst (μ : Fin 1 ⊕ Fin 3) (v : JetComponentSpace M) :
    (JetComponentSpace.jetDeriv μ v).1
      = TensorProduct.map
        (LinearMap.mulRight ℂ (SpaceTimeDerivAlgebraℂ.basis ({μ} : Multiset (Fin 1 ⊕ Fin 3))))
        LinearMap.id v.1
```

lemma jetDeriv_snd [source]

```
@[simp]
lemma JetComponentSpace.jetDeriv_snd (μ : Fin 1 ⊕ Fin 3) (v : JetComponentSpace M) :
    (JetComponentSpace.jetDeriv μ v).2
      = TensorProduct.map
        (LinearMap.mulRight ℂ (SpaceTimeDerivAlgebraℂ.basis ({μ} : Multiset (Fin 1 ⊕ Fin 3))))
        LinearMap.id v.2
```

lemma repLorentzGroup_tprod_mulRight_jetSymbol [source]

The covariance of the derivative-symbol multiplication on one tensor factor of the component space, for an arbitrary representation on the other factor.

```
private lemma repLorentzGroup_tprod_mulRight_jetSymbol {W : Type*} [AddCommGroup W]
    [Module ℂ W] (ρ : Representation ℂ SL(2,ℂ) W) (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3)
    (w : SpaceTimeDerivAlgebraℂ ⊗[ℂ] W) :
    (SpaceTimeDerivAlgebraℂ.repLorentzGroup.tprod ρ) Λ
      (TensorProduct.map
        (LinearMap.mulRight ℂ (SpaceTimeDerivAlgebraℂ.basis ({μ} : Multiset (Fin 1 ⊕ Fin 3))))
        LinearMap.id w) =
    ∑ a, (((Lorentz.SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
      TensorProduct.map
        (LinearMap.mulRight ℂ (SpaceTimeDerivAlgebraℂ.basis ({a} : Multiset (Fin 1 ⊕ Fin 3))))
        LinearMap.id ((SpaceTimeDerivAlgebraℂ.repLorentzGroup.tprod ρ) Λ w)
```

lemma repLorentzGroup_jetDeriv [source]

***The jet derivative is a Lorentz vector on the component space.** Appending `∂_μ` and then acting is acting and then appending the transformed `∂_μ`, which is a combination of the `∂_a`. Both halves of the component space are covered by the same argument: the derivative label lives in the first tensor factor, and what sits in the second factor — `M.repLorentz.dual` or `M.repLorentz.conj.dual` — plays no role.*

```
lemma JetComponentSpace.repLorentzGroup_jetDeriv (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3)
    (v : JetComponentSpace M) :
    JetComponentSpace.repLorentzGroup M Λ (JetComponentSpace.jetDeriv μ v) =
      ∑ a, (((Lorentz.SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
        JetComponentSpace.jetDeriv a (JetComponentSpace.repLorentzGroup M Λ v)
```

def comap [source]

***The component space is contravariant in the target space.** A linear map `f : M.V →ₗ N.V` of target spaces pulls the component functions of the field `N` back to component functions of the field `M`: a component function is a *covector* on the target, so it transposes. The derivative label is untouched, and the conjugate half transposes the conjugate of `f`.*

```
noncomputable def JetComponentSpace.comap (f : M.V →ₗ[ℂ] N.V) :
    JetComponentSpace N →ₗ[ℂ] JetComponentSpace M
```

lemma comap_fst_tmul [source]

```
@[simp]
lemma JetComponentSpace.comap_fst_tmul (f : M.V →ₗ[ℂ] N.V) (a : SpaceTimeDerivAlgebraℂ)
    (φ : Module.Dual ℂ N.V) (y : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule N.V)) :
    (JetComponentSpace.comap f (a ⊗ₜ[ℂ] φ, y)).1 = a ⊗ₜ[ℂ] (φ ∘ₗ f)
```

lemma comap_snd_tmul [source]

```
@[simp]
lemma JetComponentSpace.comap_snd_tmul (f : M.V →ₗ[ℂ] N.V)
    (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ N.V) (a : SpaceTimeDerivAlgebraℂ)
    (φ : Module.Dual ℂ (ConjModule N.V)) :
    (JetComponentSpace.comap f (x, a ⊗ₜ[ℂ] φ)).2 = a ⊗ₜ[ℂ] (φ ∘ₗ ConjModule.map f)
```

lemma comap_id [source]

```
@[simp]
lemma JetComponentSpace.comap_id :
    JetComponentSpace.comap (LinearMap.id : M.V →ₗ[ℂ] M.V) = LinearMap.id
```

lemma comap_comp [source]

Functoriality: pulling back along `g ∘ f` is pulling back along `g` and then along `f`. The order reverses, as it must for a contravariant construction.

```
lemma JetComponentSpace.comap_comp {P : MatterField jets}
    (f : M.V →ₗ[ℂ] N.V) (g : N.V →ₗ[ℂ] P.V) :
    JetComponentSpace.comap (g.comp f)
      = (JetComponentSpace.comap f).comp (JetComponentSpace.comap g)
```

def comapEquiv [source]

A relabelling of the target space relabels the component functions. An isomorphism `e : M.V ≃ₗ N.V` of target spaces identifies the two component spaces, contravariantly: the component functions of the field `N` become those of the field `M`. This is `comap` upgraded to an equivalence, the two directions being mutually inverse by functoriality.

```
noncomputable def JetComponentSpace.comapEquiv (e : M.V ≃ₗ[ℂ] N.V) :
    JetComponentSpace N ≃ₗ[ℂ] JetComponentSpace M
```

lemma comapEquiv_apply [source]

```
@[simp]
lemma JetComponentSpace.comapEquiv_apply (e : M.V ≃ₗ[ℂ] N.V) (x : JetComponentSpace N) :
    JetComponentSpace.comapEquiv e x = JetComponentSpace.comap e.toLinearMap x
```

lemma comap_comp_repLorentzGroup [source]

***An equivariant map of target spaces gives an equivariant pullback.** If `f : M.V →ₗ N.V` intertwines the two Lorentz representations then `comap f` intertwines the induced actions on the component spaces, in the opposite direction. Component functions are covectors, so the unconjugated half transposes `f` against the contragredient action and the conjugate half against its conjugate; both reduce to equivariance of `f` at `Λ⁻¹`.*

```
lemma JetComponentSpace.comap_comp_repLorentzGroup (f : M.V →ₗ[ℂ] N.V)
    (hf : ∀ Λ : SL(2,ℂ), f.comp (M.repLorentz Λ) = (N.repLorentz Λ).comp f)
    (Λ : SL(2,ℂ)) :
    (JetComponentSpace.comap f).comp (JetComponentSpace.repLorentzGroup N Λ)
      = (JetComponentSpace.repLorentzGroup M Λ).comp (JetComponentSpace.comap f)
```

lemma comap_jetDeriv [source]

***The pullback commutes with the jet derivative.** The two act on different tensor factors — the derivative label and the target index — so an inclusion of species is a map of differential algebras.*

```
lemma JetComponentSpace.comap_jetDeriv (f : M.V →ₗ[ℂ] N.V) (μ : Fin 1 ⊕ Fin 3) :
    (JetComponentSpace.comap f).comp (JetComponentSpace.jetDeriv μ)
```

```
    = (JetComponentSpace.jetDeriv μ).comp (JetComponentSpace.comap f)
```

def massWeightScale [source]

The mass-weight scaling on the jet component space of a field of mass weight `w` (twice the mass dimension): the generator `∂_s φ_α` and its conjugate are scaled by `c ^ (w + 2 |s|)`, through the derivative-degree scaling `SpaceTimeDerivAlgebraℂ.gradeScale` on the derivative label.

```
noncomputable def JetComponentSpace.massWeightScale (w : ℕ) (c : ℂ) :
    JetComponentSpace M →ₗ[ℂ] JetComponentSpace M
```

lemma massWeightScale_fst_basis_tmul [source]

On an unconjugated component function `∂_s φ_α` the mass-weight scaling is multiplication by `c ^ (w + 2 |s|)`.

```
lemma JetComponentSpace.massWeightScale_fst_basis_tmul (w : ℕ) (c : ℂ)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ M.V)
    (y : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule M.V)) :
    (JetComponentSpace.massWeightScale w c
        ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, y) : JetComponentSpace M)).1
      = c ^ (w + 2 * Multiset.card s) • (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ)
```

lemma massWeightScale_fst [source]

```
@[simp]
lemma JetComponentSpace.massWeightScale_fst (w : ℕ) (c : ℂ) (v : JetComponentSpace M) :
    (JetComponentSpace.massWeightScale w c v).1
      = c ^ w • TensorProduct.map
          (SpaceTimeDerivAlgebraℂ.gradeScale (c ^ 2)).toLinearMap LinearMap.id v.1
```

lemma massWeightScale_snd [source]

```
@[simp]
lemma JetComponentSpace.massWeightScale_snd (w : ℕ) (c : ℂ) (v : JetComponentSpace M) :
    (JetComponentSpace.massWeightScale w c v).2
      = c ^ w • TensorProduct.map
          (SpaceTimeDerivAlgebraℂ.gradeScale (c ^ 2)).toLinearMap LinearMap.id v.2
```

lemma gradeScale_map_mulRight_basis [source]

The derivative-degree scaling intertwines multiplication by a single derivative symbol up to one factor of the scaling parameter, on either half of the component space.

```
private lemma gradeScale_map_mulRight_basis {W : Type*} [AddCommGroup W] [Module ℂ W]
    (c : ℂ) (μ : Fin 1 ⊕ Fin 3) (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] W) :
    TensorProduct.map (SpaceTimeDerivAlgebraℂ.gradeScale (c ^ 2)).toLinearMap LinearMap.id
      (TensorProduct.map (LinearMap.mulRight ℂ
        (SpaceTimeDerivAlgebraℂ.basis ({μ} : Multiset (Fin 1 ⊕ Fin 3)))) LinearMap.id x)
    = c ^ 2 • TensorProduct.map (LinearMap.mulRight ℂ
        (SpaceTimeDerivAlgebraℂ.basis ({μ} : Multiset (Fin 1 ⊕ Fin 3)))) LinearMap.id
        (TensorProduct.map (SpaceTimeDerivAlgebraℂ.gradeScale (c ^ 2)).toLinearMap
          LinearMap.id x)
```

lemma massWeightScale_jetDeriv [source]

***The total derivative carries mass weight two** on the component space: the scaling intertwines the derivative shift up to a factor `c ^ 2`.*

```
lemma JetComponentSpace.massWeightScale_jetDeriv (w : ℕ) (c : ℂ) (μ : Fin 1 ⊕ Fin 3) :
    (JetComponentSpace.massWeightScale (M := M) w c).comp (JetComponentSpace.jetDeriv μ)
      = c ^ 2 • (JetComponentSpace.jetDeriv μ).comp
          (JetComponentSpace.massWeightScale w c)
```

lemma comap_comp_massWeightScale [source]

***The mass-weight scaling is natural in the target space.** It commutes with every pullback, acting as it does on the derivative label and not on the target index. So the scaling cannot see which part of a target space a component function came from: in `JetComponentSpace (∀ i, V i)`, where a species enters through `comap (LinearMap.proj i)`, every species is scaled by the same weight. This is why the generator space of a multi-species theory, `GaugeFieldData.FermionGenerators`, records the weights on a direct sum, one per species, rather than on a single component space of the product.*

```
lemma JetComponentSpace.comap_comp_massWeightScale (f : M.V →ₗ[ℂ] N.V) (w : ℕ) (c : ℂ) :
    (JetComponentSpace.comap f).comp (JetComponentSpace.massWeightScale w c)
      = (JetComponentSpace.massWeightScale w c).comp (JetComponentSpace.comap f)
```

def prodPiEquiv [source]

A pair of families is the same thing as a family of pairs. This is the last step of the splitting of a component space over a product of target spaces: the two halves split separately into families, and this puts the two families back together index by index. Everything is the identity on underlying data, so every law is `rfl`.

```
def prodPiEquiv {A B : ι → Type*} [∀ i, AddCommGroup (A i)] [∀ i, Module ℂ (A i)]
    [∀ i, AddCommGroup (B i)] [∀ i, Module ℂ (B i)] :
    ((∀ i, A i) × (∀ i, B i)) ≃ₗ[ℂ] ∀ i, (A i × B i) where
```

def fstPiEquiv [source]

***The unconjugated half of the component space of a finite direct sum splits.** The symbols `∂_s ψ_α` of a `(∀ i, E i)`-valued field are the families, over the index, of the symbols of the summands: the dual distributes over the finite product and the derivative label is untouched.*

```
noncomputable def JetComponentSpace.fstPiEquiv :
    (SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (∀ i, E i))
      ≃ₗ[ℂ] ∀ i, SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (E i)
```

lemma fstPiEquiv_tmul [source]

On a pure symbol the splitting restricts the target index to one summand: the component `∂_s ψ_α` of the direct sum in the summand `i` is `∂_s` of the covector `φ` precomposed with the inclusion of that summand.

```
@[simp]
lemma JetComponentSpace.fstPiEquiv_tmul (a : SpaceTimeDerivAlgebraℂ)
```

```
    (φ : Module.Dual ℂ (∀ i, E i)) (i : ι) :
    fstPiEquiv E (a ⊗ₜ[ℂ] φ) i = a ⊗ₜ[ℂ] (φ ∘ₗ LinearMap.single ℂ E i)
```

def sndPiEquiv [source]

***The conjugate half of the component space of a finite direct sum splits**, by the same argument applied to the conjugate modules, using that conjugation commutes with products.*

```
noncomputable def JetComponentSpace.sndPiEquiv :
    (SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule (∀ i, E i)))
      ≃ₗ[ℂ] ∀ i, SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule (E i))
```

lemma sndPiEquiv_tmul [source]

On a pure conjugate symbol the splitting again restricts the target index to one summand, the inclusion being read through the conjugation.

```
@[simp]
lemma JetComponentSpace.sndPiEquiv_tmul (a : SpaceTimeDerivAlgebraℂ)
    (ψ : Module.Dual ℂ (ConjModule (∀ i, E i))) (i : ι) :
    sndPiEquiv E (a ⊗ₜ[ℂ] ψ) i
      = a ⊗ₜ[ℂ] (ψ ∘ₗ ConjModule.map (k := ℂ) (LinearMap.single ℂ E i))
```

lemma fstPiEquiv_symm_single [source]

The splitting sends the family supported on one summand back to the pullback along the projection onto that summand: a component function of the summand `i`, read as a component function of the whole, is `φ ∘ proj i`.

```
lemma JetComponentSpace.fstPiEquiv_symm_single (i : ι)
    (z : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (E i)) :
    (fstPiEquiv E).symm (Pi.single i z)
      = TensorProduct.map LinearMap.id (Module.Dual.transpose (LinearMap.proj i)) z
```

lemma sndPiEquiv_symm_single [source]

The conjugate half of the splitting behaves in the same way, the projection read through the conjugation.

```
lemma JetComponentSpace.sndPiEquiv_symm_single (i : ι)
    (z : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule (E i))) :
    (sndPiEquiv E).symm (Pi.single i z)
      = TensorProduct.map LinearMap.id
          (Module.Dual.transpose (ConjModule.map (k := ℂ) (LinearMap.proj i))) z
```

2.241 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.JetComponent.GaugeAction

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/JetComponent/GaugeAction.lean on GitHub.

def symbolAction [source]

***The action of a coefficient on the symbols.** A coefficient `g ⊗ T` acts by `jetRingAction g` on the derivative label — the Leibniz convolution redistributing derivatives between*

the gauge transformation and the field — and by the transpose T^T on the target index.

A coefficient is allowed to change the value space, so that the symbol action of an endomorphism and the pullback along a map of value spaces are the same construction; at `W = V` this is the action on `Module.End ℂ (SpaceTimeDerivAlgebraℂ ⊗ Module.Dual ℂ V)` that `repDual` uses.

```
noncomputable def symbolAction :
    (SpaceTimeAlgebra ⊗[ℂ] (V →ₗ[ℂ] W)) →ₗ[ℂ]
      ((SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ W) →ₗ[ℂ]
        (SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ V))
```

lemma symbolAction_tmul [source]

```
@[simp]
lemma symbolAction_tmul (g : SpaceTimeAlgebra) (T : V →ₗ[ℂ] W) :
    symbolAction (g ⊗ₜ[ℂ] T)
      = TensorProduct.map (SpaceTimeDerivAlgebraℂ.jetRingAction g) (Module.Dual.transpose T)
```

lemma symbolAction_one_tmul [source]

***A coefficient acts on the undifferentiated symbol through its value at the base point.** On `1 ⊗ φ` — the symbol `ψ_φ` carrying no derivatives — only the constant term of the power-series coefficient survives, so the result is again undifferentiated and the target index is acted on by the transpose of the base-point value.*

```
lemma symbolAction_one_tmul (c : SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V) (φ : Module.Dual ℂ V) :
    symbolAction c ((1 : SpaceTimeDerivAlgebraℂ) ⊗ₜ[ℂ] φ)
      = (1 : SpaceTimeDerivAlgebraℂ) ⊗ₜ[ℂ]
        Module.Dual.transpose (jetEval ∘ₗ TensorProduct.lift
          ((LinearMap.llcomp ℂ V V (SpaceTimeAlgebra ⊗[ℂ] V)).comp
            (TensorProduct.mk ℂ SpaceTimeAlgebra V)) c) φ
```

def repDual [source]

***The gauge action on the symbols.** Given a fibrewise gauge action on the jets of a `V`-valued field, this is the induced (contragredient) action on the derivative symbols `∂_s ψ_α`, which span `SpaceTimeDerivAlgebraℂ ⊗ Module.Dual ℂ V`.*

Multiplicativity is bookkeeping: `coeff_mul_of_smul_comm` makes the coefficient multiplicative, `symbolAction_mul` makes its action an anti-homomorphism, and the inverse flips that back.

```
noncomputable def repDual [Module.Free ℂ V] [Module.Finite ℂ V]
    (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (hlin : ∀ (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V),
      rep U (χ • z) = χ • rep U z) :
    Representation ℂ GJ (SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ V) where
```

lemma repDual_one_tmul [source]

***The undifferentiated symbol transforms by the value of the gauge transformation at the base point.** No derivative of the gauge jet contributes: the symbol `ψ_φ` is*

acted on by the contragredient of `rep U⁻¹` restricted to constant jets and evaluated at the base point.

```
lemma repDual_one_tmul [Module.Free ℂ V] [Module.Finite ℂ V]
    (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (hlin : ∀ (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V),
      rep U (χ • z) = χ • rep U z)
    (U : GJ) (φ : Module.Dual ℂ V) :
    repDual rep hlin U ((1 : SpaceTimeDerivAlgebraℂ) ⊗ₜ[ℂ] φ)
      = (1 : SpaceTimeDerivAlgebraℂ) ⊗ₜ[ℂ]
        Module.Dual.transpose (jetEval ∘ₗ (rep U⁻¹).comp jetOfConstant) φ
```

def repJet [source]

***The gauge action on the jet component space.** The induced action of the jets of gauge transformations on the full space of component functions of the matter field `M` — the symbols `∂_s ψ_α` together with their conjugates `∂_s ψ̄_α`.*

*The unconjugated half is `repDual M.repJet`, the contragredient action on the symbols. The conjugate half is the *same* construction applied to `repConj M.repJet`, the action on the jets of the conjugate field; `repConj_smul_comm` supplies the fibrewise-linearity it needs. The conjugate half therefore carries `star` of the gauge matrix, which is the physicists' `ψ̄ ↦ ψ̄ U†`.*

Everything the construction needs is a field of `MatterField`: the jet action `M.repJet`, its fibrewise linearity `M.repJet_smul`, and the freeness and finiteness of `M.V`. Taking the matter field rather than a bare value space is what removes all three from the argument list.

```
noncomputable def repJet (M : MatterField jets) :
    Representation ℂ GJ (JetComponentSpace M)
```

lemma repJet_fst [source]

```
@[simp]
lemma repJet_fst (M : MatterField jets) (U : GJ) (x : JetComponentSpace M) :
    (repJet M U x).1 = repDual M.repJet M.repJet_smul U x.1
```

lemma repJet_snd [source]

```
@[simp]
lemma repJet_snd (M : MatterField jets) (U : GJ) (x : JetComponentSpace M) :
    (repJet M U x).2
      = repDual (repConj M.repJet) (repConj_smul_comm M.repJet_smul) U x.2
```

lemma map_transpose_comp_symbolAction [source]

Pulling back and then acting is acting and then pulling back, on the symbols of a coefficient. Precomposing the symbol action of a coefficient of `W` with the pullback along `f` is the symbol action of the coefficient precomposed with `f`.

```
lemma map_transpose_comp_symbolAction (f : V →ₗ[ℂ] W) (y : SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ W)
    :
    (TensorProduct.map LinearMap.id (Module.Dual.transpose f)).comp (symbolAction y)
      = symbolAction (LinearMap.lTensor SpaceTimeAlgebra (LinearMap.lcomp ℂ W f) y)
```

lemma symbolAction_comp_map_transpose [source]

The companion of `map_transpose_comp_symbolAction` on the other side: postcomposing the symbol action of a coefficient of `V` with the pullback along `f` is the symbol action of the coefficient postcomposed with `f`.

```
lemma symbolAction_comp_map_transpose (f : V →ₗ[ℂ] W) (x : SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V)
    :
    (symbolAction x).comp (TensorProduct.map LinearMap.id (Module.Dual.transpose f))
      = symbolAction (LinearMap.lTensor SpaceTimeAlgebra (LinearMap.llcomp ℂ V V W f) x)
```

lemma comap_comp_repDual [source]

The gauge action on the unconjugated symbols is natural in the value space. A linear map of value spaces intertwining two fibrewise jet actions makes the pullback of symbols equivariant for the two contragredient actions. The whole content is the naturality of the coefficient, `JetComponentSpace.jetCoeff_naturality`, read through the symbol action; the group element is inverted on both sides alike, so no convention is disturbed by it.

```
lemma comap_comp_repDual [Module.Free ℂ V] [Module.Finite ℂ V]
    [Module.Free ℂ W] [Module.Finite ℂ W]
    (repV : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (hV : ∀ (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V), repV U
        (χ • z) = χ • repV U z)
    (repW : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] W))
    (hW : ∀ (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] W), repW U
        (χ • z) = χ • repW U z)
    (f : V →ₗ[ℂ] W)
    (hf : ∀ U : GJ, (LinearMap.lTensor SpaceTimeAlgebra f).comp (repV U)
      = (repW U).comp (LinearMap.lTensor SpaceTimeAlgebra f)) (U : GJ) :
    (TensorProduct.map LinearMap.id (Module.Dual.transpose f)).comp (repDual repW hW U)
      = (repDual repV hV U).comp
        (TensorProduct.map LinearMap.id (Module.Dual.transpose f))
```

lemma comap_comp_repJet [source]

The gauge action on the jet component space is natural in the value space. If `f : M.V →ₗ N.V` intertwines the two fields' actions on the jets, then the pullback of component functions along `f` intertwines the induced actions on the component spaces, in the opposite direction — the pullback of a component function of a `W`-valued field being a component function of a `V`-valued field.

Both halves are covered and every derivative label is carried: the statement is an equality of linear maps on the whole component space, not a statement about undifferentiated symbols. The conjugate half is the unconjugated argument applied to `repConj M.repJet` and `repConj N.repJet`, whose intertwining is `JetComponentSpace.lTensor_comp_repConj`.

```
lemma comap_comp_repJet {M N : MatterField jets} (f : M.V →ₗ[ℂ] N.V)
    (hf : ∀ U : GJ, (LinearMap.lTensor SpaceTimeAlgebra f).comp (M.repJet U)
      = (N.repJet U).comp (LinearMap.lTensor SpaceTimeAlgebra f)) (U : GJ) :
    (comap f).comp (repJet N U) = (repJet M U).comp (comap f)
```

2.242 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.JetComponent.TransformsIn

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/JetComponent/TransformsIn.lean on GitHub.

lemma jetIteratedDeriv_tmul [source]

The iterated formal derivative of a `V`-valued jet acts on the jet-ring factor of a pure tensor: the value factor carries no spacetime dependence.

```
lemma jetIteratedDeriv_tmul (x : Multiset (Fin 1 ⊕ Fin 3)) (f : SpaceTimeAlgebra) (v : V) :
    jetIteratedDeriv x (f ⊗ₜ[ℂ] v) = SpaceTimeAlgebra.iteratedPDeriv x f ⊗ₜ[ℂ] v
```

def jetCoeffAt [source]

The base-point Taylor coefficient at `x` derivatives of a jet of endomorphisms of `V`: differentiate `x` times and evaluate at the base point. It is the `V`-valued jet toolkit applied to the value space `Module.End ℂ V`, and it is what a coefficient in `SpaceTimeAlgebra ⊗ End V` contributes to the derivative symbol `∂_x`.

```
noncomputable def jetCoeffAt (x : Multiset (Fin 1 ⊕ Fin 3)) :
    SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V →ₗ[ℂ] Module.End ℂ V
```

lemma jetCoeffAt_tmul [source]

On a pure coefficient `f ⊗ T` the Taylor coefficient is the base-point Taylor coefficient of `f` times `T`.

```
@[simp]
lemma jetCoeffAt_tmul (x : Multiset (Fin 1 ⊕ Fin 3)) (f : SpaceTimeAlgebra) (T : Module.End ℂ V
    ) :
    jetCoeffAt x (f ⊗ₜ[ℂ] T)
      = constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x f) • T
```

lemma jetCoeffAt_apply [source]

The Taylor coefficient of a jet of endomorphisms, evaluated at a vector, is the Taylor coefficient of the `V`-valued jet obtained by feeding that vector to the coefficient.

```
lemma jetCoeffAt_apply (x : Multiset (Fin 1 ⊕ Fin 3))
    (c : SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V) (v : V) :
    jetCoeffAt x c v = jetEval (jetIteratedDeriv x (TensorProduct.lift
      ((LinearMap.llcomp ℂ V V (SpaceTimeAlgebra ⊗[ℂ] V)).comp
        (TensorProduct.mk ℂ SpaceTimeAlgebra V)) c v))
```

lemma jetCoeffAt_jetCoeff [source]

The Taylor coefficients of the gauge coefficient are the Taylor coefficients of the representation: `jetCoeff rep U` reproduces `rep U` on constant jets, and both sides of this identity read off the same derivative of that.

```
lemma jetCoeffAt_jetCoeff [Module.Free ℂ V] [Module.Finite ℂ V]
    (rep : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V)) (U : G)
    (x : Multiset (Fin 1 ⊕ Fin 3)) :
    jetCoeffAt x (jetCoeff rep U) = GaugeAlgebraRealization.repCoeff rep U x
```

lemma dualMap_add_apply [source]

The transpose is additive in the endomorphism.

```
private lemma dualMap_add_apply (A B : Module.End ℂ V) (φ : Module.Dual ℂ V) :
    (A + B).dualMap φ = A.dualMap φ + B.dualMap φ
```

lemma dualMap_smul_apply [source]

The transpose is homogeneous in the endomorphism.

```
private lemma dualMap_smul_apply (c : ℂ) (T : Module.End ℂ V) (φ : Module.Dual ℂ V) :
    (c • T).dualMap φ = c • T.dualMap φ
```

lemma sum_tmul_right [source]

A multiset sum in the derivative label distributes out of a pure symbol.

```
private lemma sum_tmul_right (m : Multiset SpaceTimeDerivAlgebraℂ) (w : Module.Dual ℂ V) :
    m.sum ⊗ₜ[ℂ] w = (m.map fun a => a ⊗ₜ[ℂ] w).sum
```

lemma symbolAction_basis_tmul [source]

A coefficient acts on the derivative symbol `∂_s ψ_φ` by the all-orders Leibniz convolution of its base-point Taylor coefficients: each splitting `s = s₁ + s₂` of the derivative multiset contributes the Taylor coefficient at `s₁` acting on the target index of the lower symbol `∂_{s₂} ψ_φ`.

This is `SpaceTimeDerivAlgebraℂ.jetRingAction_basis_multiset` in the derivative label, together with the identification of the scalars it produces as `jetCoeffAt`; both sides are additive in the coefficient, so it suffices to check it on a pure tensor.

```
lemma symbolAction_basis_tmul (c : SpaceTimeAlgebra ⊗[ℂ] Module.End ℂ V)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    symbolAction c (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ)
      = (s.antidiagonal.map fun p =>
          SpaceTimeDerivAlgebraℂ.basis p.2 ⊗ₜ[ℂ] (jetCoeffAt p.1 c).dualMap φ).sum
```

lemma repDual_basis_tmul [source]

The transformation law of the derivative symbol `∂_s ψ_φ` under the jet gauge group: the all-orders Leibniz convolution of the dual representation coefficients `GaugeAlgebraRealization.repDualC` against lower symbols, with no inhomogeneous term. This is the identity the `LocalGaugeData.TransformsIn` obligations of a matter field rest on.

Nothing here is special to the unconjugated half of the component space: the conjugate half is this lemma at `repConj rep`.

```
lemma repDual_basis_tmul [Module.Free ℂ V] [Module.Finite ℂ V]
    (rep : Representation ℂ G (SpaceTimeAlgebra ⊗[ℂ] V))
    (hlin : ∀ (U : G) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V),
      rep U (χ • z) = χ • rep U z)
    (U : G) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    repDual rep hlin U (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ)
      = (s.antidiagonal.map fun p =>
          SpaceTimeDerivAlgebraℂ.basis p.2 ⊗ₜ[ℂ]
```

```
      GaugeAlgebraRealization.repDualCoeff rep U⁻¹ p.1 φ).sum
```

2.243 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.LorentzCovariantDeriv

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/LorentzCovariantDeriv.lean on GitHub.

lemma sum_derivConv_sum_fam [source]

A scalar combination of convolutions against the gauge field is linear in the right-hand families.

```
lemma sum_derivConv_sum_fam {ι κ ι' : Type} [Fintype ι] [Fintype κ] [Fintype ι']
    (f : ι → Multiset (Fin 1 ⊕ Fin 3) → B) (coef : ι → κ → ℂ) (c : ι' → ℂ)
    (g : ι' → κ → Multiset (Fin 1 ⊕ Fin 3) → B) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    ∑ j, ∑ k, coef j k • derivConv (f j) (fun y => ∑ i, c i • g i k y) s =
      ∑ i, c i • ∑ j, ∑ k, coef j k • derivConv (f j) (g i k) s
```

lemma repLorentz_eq_lorentzMix [source]

A Lorentz law in the tuple form, read on the underlying multisets: the transformed family mixes by `lorentzMix`.

```
lemma repLorentz_eq_lorentzMix (Λ : SL(2,ℂ)) (f g : Multiset (Fin 1 ⊕ Fin 3) → B)
    (hfg : ∀ (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)), repLorentz Λ (f (List.ofFn l)) =
      ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ i, L[Λ] (p i) (l i)) • g (List.ofFn p))
    (x : Multiset (Fin 1 ⊕ Fin 3)) : repLorentz Λ (f x) = lorentzMix Λ g x 0
```

lemma repLorentz_apply_mix [source]

The Lorentz law of the gauge-field symbols, in the multiset form.

```
lemma repLorentz_apply_mix (Λ : SL(2,ℂ))
    (x : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (χ : Module.Dual ℝ 𝔤) :
    repLorentz Λ (h.A x μ χ) = lorentzMix Λ (fun t => ∑ a, L[Λ] a μ • h.A t a χ) x 0
```

lemma isLorentzDerivTransforms_mix [source]

The Lorentz law of a family of derivative symbols, in the multiset form.

```
lemma isLorentzDerivTransforms_mix {rep : Representation ℂ SL(2,ℂ) V}
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B}
    (hF : IsLorentzDerivTransforms repLorentz rep F) (Λ : SL(2,ℂ))
    (x : Multiset (Fin 1 ⊕ Fin 3)) (χ : Module.Dual ℂ V) :
    repLorentz Λ (F x χ) = lorentzMix Λ (fun t => F t (rep.dual Λ χ)) x 0
```

lemma repLorentz_sum_derivConv [source]

The Lorentz law of a scalar combination of convolutions against the gauge field: the direction of the gauge field mixes by its own column, the derivative slots by `lorentzMix`, and the right-hand families are replaced by their transforms.

```
lemma repLorentz_sum_derivConv (Λ : SL(2,ℂ)) (ρ : Fin 1 ⊕ Fin 3)
    {ι κ : Type} [Fintype ι] [Fintype κ] (bg : Module.Basis ι ℝ 𝔤) (coef : ι → κ → ℂ)
    (g g' : κ → Multiset (Fin 1 ⊕ Fin 3) → B)
    (hg : ∀ k y, repLorentz Λ (g k y) = lorentzMix Λ (g' k) y 0)
```

```
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    repLorentz Λ (∑ j, ∑ k, coef j k • derivConv (fun x => h.A x ρ (bg.coord j)) (g k) s) =
      ∑ a, L[Λ] a ρ • lorentzMix Λ
        (fun t => ∑ j, ∑ k, coef j k • derivConv (fun x => h.A x a (bg.coord j)) (g' k) t)
        s 0
```

lemma actionFam_apply_eq_sum [source]

The action of families expanded in bases of the gauge algebra and the value space.

```
lemma actionFam_apply_eq_sum {ι κ : Type} [Fintype ι] [Fintype κ]
    (bg : Module.Basis ι ℝ 𝔤) (bv : Module.Basis κ ℂ V)
    (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) (g : Module.Dual ℂ V →ₗ[ℂ] B)
    (φ : Module.Dual ℂ V) :
    actionFam act f g φ =
      ∑ j, ∑ k, φ (act (bg j) (bv k)) • (f (bg.coord j) * g (bv.coord k))
```

lemma actionFamConv_eq_sum [source]

The derived action family expanded in bases.

```
lemma actionFamConv_eq_sum {ι κ : Type} [Fintype ι] [Fintype κ]
    (bg : Module.Basis ι ℝ 𝔤) (bv : Module.Basis κ ℂ V)
    (ρ : Fin 1 ⊕ Fin 3) (G : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    actionFamConv A act ρ G s φ = ∑ j, ∑ k, φ (act (bg j) (bv k)) •
      derivConv (fun x => A x ρ (bg.coord j)) (fun y => G y (bv.coord k)) s
```

lemma actionFamConv_sum_fam [source]

The derived action family is linear in the matter family.

```
lemma actionFamConv_sum_fam {ι : Type} [Fintype ι] (ρ : Fin 1 ⊕ Fin 3) (c : ι → ℂ)
    (H : ι → Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    actionFamConv A act ρ (fun t => ∑ i, c i • H i t) s φ =
      ∑ i, c i • actionFamConv A act ρ (H i) s φ
```

lemma repLorentz_actionFamConv [source]

The Lorentz law of the derived action family: the derivative slots mix, the direction of the gauge field mixes by its own column, and the value index is carried by the transformed matter family.

```
lemma repLorentz_actionFamConv (Λ : SL(2,ℂ)) (ρ : Fin 1 ⊕ Fin 3)
    (G G' : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (hG : ∀ y χ, repLorentz Λ (G y χ) = lorentzMix Λ (fun t => G' t χ) y 0)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    repLorentz Λ (actionFamConv h.A act ρ G s φ) =
      ∑ a, L[Λ] a ρ • lorentzMix Λ (fun t => actionFamConv h.A act a G' t φ) s 0
```

lemma dual_twist [source]

The twist of the value index past the gauge action: an endomorphism commuting with the gauge action may be moved from the dual basis onto the dual vector.

```
lemma dual_twist {κ : Type} [Fintype κ] (bv : Module.Basis κ ℂ V) (T : V →ₗ[ℂ] V)
    (hT : ∀ (c : 𝔤) (v : V), act c (T v) = T (act c v))
    (ψ : Module.Dual ℂ V →ₗ[ℂ] B) (c : 𝔤) (φ : Module.Dual ℂ V) :
    ∑ k, φ (act c (bv k)) • ψ (T.dualMap (bv.coord k)) =
      ∑ k, (T.dualMap φ) (act c (bv k)) • ψ (bv.coord k)
```

lemma actionFam_comp_dual [source]

The contragredient action may be pulled out of an action of families, provided the gauge action commutes with it on the value space.

```
lemma actionFam_comp_dual (T : V →ₗ[ℂ] V)
    (hT : ∀ (c : 𝔤) (v : V), act c (T v) = T (act c v))
    (f : Module.Dual ℝ 𝔤 →ₗ[ℝ] B) (g : Module.Dual ℂ V →ₗ[ℂ] B)
    (φ : Module.Dual ℂ V) :
    actionFam act f (g ∘ₗ T.dualMap) φ = actionFam act f g (T.dualMap φ)
```

lemma actionFamConv_comp_dual [source]

The contragredient action may be pulled out of a derived action family.

```
lemma actionFamConv_comp_dual (T : V →ₗ[ℂ] V)
    (hT : ∀ (c : 𝔤) (v : V), act c (T v) = T (act c v)) (ρ : Fin 1 ⊕ Fin 3)
    (K : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    actionFamConv A act ρ (fun t => K t ∘ₗ T.dualMap) s φ =
      actionFamConv A act ρ K s (T.dualMap φ)
```

lemma repLorentz_covDerivIter [source]

The Lorentz law of the iterated covariant derivative of a matter family: the ordered covariant slots mix by their own columns and the multiset of plain derivative slots mixes by `lorentzMix`, while the value index transforms contragrediently.

```
lemma repLorentz_covDerivIter {rep : Representation ℂ SL(2,ℂ) V}
    (hcomm : ∀ (c : 𝔤) (Λ : SL(2,ℂ)) (v : V), act c (rep Λ v) = rep Λ (act c v))
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (hF : IsLorentzDerivTransforms repLorentz rep F) (Λ : SL(2,ℂ))
    (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ V) :
    repLorentz Λ (covDerivIter h.A act F n l s φ) = ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
      (∏ i, L[Λ] (p i) (l i)) •
        lorentzMix Λ (fun t => covDerivIter h.A act F n p t (rep.dual Λ φ)) s 0
```

lemma isLorentzCovDerivTransforms_covDerivIter [source]

The iterated covariant derivative of a matter family transforms as the covariant derivatives of a Lorentz-covariant field, given the Lorentz law of the bare symbols, the Lorentz law of the gauge field, and the commutation of the infinitesimal gauge action with the Lorentz action on the value space.

```
lemma isLorentzCovDerivTransforms_covDerivIter {rep : Representation ℂ SL(2,ℂ) V}
    (hcomm : ∀ (c : 𝔤) (Λ : SL(2,ℂ)) (v : V), act c (rep Λ v) = rep Λ (act c v))
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (hF : IsLorentzDerivTransforms repLorentz rep F) :
```

```
    IsLorentzCovDerivTransforms repLorentz rep (fun {n} l => covDerivIter h.A act F n l 0)
```

lemma actionConj_comm_repConj [source]

Conjugation preserves the commutation of the gauge action with the Lorentz action: both are read on the conjugate module through the same underlying maps.

```
lemma actionConj_comm_repConj (rep : Representation ℂ SL(2,ℂ) V)
    (hcomm : ∀ (c : 𝔤) (Λ : SL(2,ℂ)) (v : V), act c (rep Λ v) = rep Λ (act c v))
    (c : 𝔤) (Λ : SL(2,ℂ)) (v : ConjModule V) :
    LocalGaugeData.actionConj act c (rep.conj Λ v) =
      rep.conj Λ (LocalGaugeData.actionConj act c v)
```

lemma isLorentzCovDerivTransforms_covDerivIter_conj [source]

The Lorentz law of the covariant tower of a conjugate family, from the commutation of the gauge action with the Lorentz action of the unconjugated species.

```
lemma isLorentzCovDerivTransforms_covDerivIter_conj {rep : Representation ℂ SL(2,ℂ) V}
    (hcomm : ∀ (c : 𝔤) (Λ : SL(2,ℂ)) (v : V), act c (rep Λ v) = rep Λ (act c v))
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ (ConjModule V) →ₗ[ℂ] B)
    (hF : IsLorentzDerivTransforms repLorentz rep.conj F) :
    IsLorentzCovDerivTransforms repLorentz rep.conj
      (fun {n} l => covDerivIter h.A (LocalGaugeData.actionConj act) F n l 0)
```

2.244 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.MatrixRep.Basic

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/MatrixRep/Basic.lean on GitHub.

structure MatrixRep [source]

***A matrix representation** of the jets of gauge transformations on an internal index `ι`: the jets act by the matrix of jets `mat U`, the gauge algebra by the matrix `act c`, and the jets of the gauge algebra by the matrix of jets `jetAct a`, subject to the derivative identity and the equivariance identity that make `act` the infinitesimal action underlying `mat`.*

```
structure MatrixRep (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) (ι : Type) [Fintype ι] [DecidableEq ι]
    where
  mat : GJ → Matrix ι ι SpaceTimeAlgebra
  mat_one : mat 1 = 1
  mat_mul : ∀ U V, mat (U * V) = mat U * mat V
  act : 𝔤 →ₗ[ℝ] Matrix ι ι ℂ
  jetAct : 𝔤J → Matrix ι ι SpaceTimeAlgebra
  jetAct_ofConstantLie : ∀ c, jetAct (jets.ofConstantLie c) = (act c).map (C : ℂ →
    SpaceTimeAlgebra)
  jetAct_map_cc_foldl : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (a : 𝔤J),
    ((jetAct a).map fun f => constantCoeff (SpaceTimeAlgebra.iteratedPDeriv p f))
      = act (jets.evalLie (jets.iteratedDeriv p a))
  mat_map_pderiv : ∀ (U : GJ) (μ : Fin 1 ⊕ Fin 3),
    (mat U).map (fun f => pderiv μ f) = -(jetAct (jets.maurerCartan U μ) * mat U)
  mat_mul_jetAct : ∀ (U : GJ) (c : 𝔤),
    mat U * jetAct (jets.ofConstantLie c)
      = jetAct (jets.adjointJ U (jets.ofConstantLie c)) * mat U
```

def pderivPi [source]

The entrywise formal derivative on `ι → SpaceTimeAlgebra`, as a `ℂ`-linear map.

```
noncomputable def pderivPi (μ : Fin 1 ⊕ Fin 3) : (ι → SpaceTimeAlgebra) →ₗ[ℂ]
    (ι → SpaceTimeAlgebra) where
```

lemma pderivPi_apply [source]

```
lemma pderivPi_apply (μ : Fin 1 ⊕ Fin 3) (w : ι → SpaceTimeAlgebra) (i : ι) :
    pderivPi μ w i = pderiv μ (w i)
```

def foldPi [source]

The entrywise iterated formal derivative on `ι → SpaceTimeAlgebra`, as a `ℂ`-linear map.

```
noncomputable def foldPi (x : Multiset (Fin 1 ⊕ Fin 3)) :
    (ι → SpaceTimeAlgebra) →ₗ[ℂ] (ι → SpaceTimeAlgebra) where
```

lemma foldPi_apply [source]

```
lemma foldPi_apply (x : Multiset (Fin 1 ⊕ Fin 3)) (w : ι → SpaceTimeAlgebra) (i : ι) :
    foldPi x w i = SpaceTimeAlgebra.iteratedPDeriv x (w i)
```

lemma foldPi_zero [source]

```
lemma foldPi_zero : foldPi (ι := ι) 0 = LinearMap.id
```

lemma pderivPi_comp_foldPi [source]

```
lemma pderivPi_comp_foldPi (μ : Fin 1 ⊕ Fin 3) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    pderivPi (ι := ι) μ ∘ₗ foldPi x = foldPi (μ ::ₘ x)
```

def ccPi [source]

The entrywise base-point evaluation on `ι → SpaceTimeAlgebra`, as a `ℂ`-linear map.

```
noncomputable def ccPi : (ι → SpaceTimeAlgebra) →ₗ[ℂ] (ι → ℂ) where
```

lemma ccPi_apply [source]

```
lemma ccPi_apply (w : ι → SpaceTimeAlgebra) (i : ι) : ccPi w i = constantCoeff (w i)
```

lemma C_mul_eq_smul [source]

A constant jet times a jet is the scalar multiple.

```
lemma C_mul_eq_smul (z : ℂ) (f : SpaceTimeAlgebra) : (C z : SpaceTimeAlgebra) * f = z • f
```

lemma ccPi_foldPi_mulVec [source]

The base-point evaluation of the iterated derivative of a matrix–vector product with constant entries is the matrix–vector product of the base-point coefficients.

```
lemma ccPi_foldPi_mulVec (x : Multiset (Fin 1 ⊕ Fin 3)) (A : Matrix ι ι SpaceTimeAlgebra)
    (v : ι → ℂ) :
    ccPi (foldPi x (A.mulVec fun k => (C (v k) : SpaceTimeAlgebra)))
      = (A.map fun f => constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x f)).mulVec v
```

def jetEquiv [source]

The jets of a `V`-valued field as `S ⊗ (ι → SpaceTimeAlgebra)`, through the identification `e : V ≃ S ⊗ (ι → ℂ)`: the jet ring is absorbed into the internal index.

```
noncomputable def jetEquiv (e : V ≃ₗ[ℂ] S ⊗[ℂ] (ι → ℂ)) :
    SpaceTimeAlgebra ⊗[ℂ] V ≃ₗ[ℂ] S ⊗[ℂ] (ι → SpaceTimeAlgebra)
```

lemma jetEquiv_tmul [source]

```
lemma jetEquiv_tmul (f : SpaceTimeAlgebra) (s : S) (v : ι → ℂ) :
    jetEquiv e (f ⊗ₜ[ℂ] e.symm (s ⊗ₜ[ℂ] v)) = s ⊗ₜ[ℂ] (fun i => v i • f)
```

lemma induction_on [source]

Induction on the jets of a `V`-valued field through the identification `e`.

```
lemma induction_on {P : SpaceTimeAlgebra ⊗[ℂ] V → Prop} (z : SpaceTimeAlgebra ⊗[ℂ] V)
    (tmul : ∀ (f : SpaceTimeAlgebra) (s : S) (v : ι → ℂ), P (f ⊗ₜ[ℂ] e.symm (s ⊗ₜ[ℂ] v)))
    (add : ∀ a b, P a → P b → P (a + b)) : P z
```

lemma jetEquiv_jetDeriv [source]

The identification of jets intertwines the formal derivative with the entrywise derivative.

```
lemma jetEquiv_jetDeriv (μ : Fin 1 ⊕ Fin 3) (z : SpaceTimeAlgebra ⊗[ℂ] V) :
    jetEquiv e (jetDeriv μ z) = LinearMap.lTensor S (pderivPi μ) (jetEquiv e z)
```

lemma jetEquiv_jetIteratedDeriv [source]

The identification of jets intertwines the iterated formal derivative with the entrywise iterated derivative.

```
lemma jetEquiv_jetIteratedDeriv (x : Multiset (Fin 1 ⊕ Fin 3)) (z : SpaceTimeAlgebra ⊗[ℂ] V) :
    jetEquiv e (jetIteratedDeriv x z) = LinearMap.lTensor S (foldPi x) (jetEquiv e z)
```

lemma jetEval_eq [source]

The base-point evaluation of a jet through the identification.

```
lemma jetEval_eq (z : SpaceTimeAlgebra ⊗[ℂ] V) :
    jetEval z = e.symm (LinearMap.lTensor S ccPi (jetEquiv e z))
```

lemma jetEquiv_smul [source]

Multiplication by a scalar jet through the identification.

```
lemma jetEquiv_smul (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V) :
    jetEquiv e (χ • z)
      = LinearMap.lTensor S
          ((LinearMap.lsmul SpaceTimeAlgebra (ι → SpaceTimeAlgebra) χ).restrictScalars ℂ)
          (jetEquiv e z)
```

lemma jetEquiv_jetOfConstant [source]

A jet of a constant through the identification.

```
lemma jetEquiv_jetOfConstant (s : S) (v : ι → ℂ) :
    jetEquiv e (jetOfConstant (e.symm (s ⊗ₜ[ℂ] v))) = s ⊗ₜ[ℂ] (fun i => C (v i))
```

def valEndAlgHom [source]

The endomorphism of the target space $V \simeq S \otimes (\iota \to \mathbb{C})$ defined by a complex matrix on the internal index, as an algebra map.

```
noncomputable def valEndAlgHom : Matrix ι ι ℂ →ₐ[ℂ] Module.End ℂ V
```

def valEnd [source]

The endomorphism of the target space $V \simeq S \otimes (\iota \to \mathbb{C})$ defined by a complex matrix on the internal index, with the Lorentz factor untouched.

```
noncomputable def valEnd (B : Matrix ι ι ℂ) : V →ₗ[ℂ] V
```

lemma valEnd_apply [source]

```
lemma valEnd_apply (B : Matrix ι ι ℂ) (d : V) :
    valEnd e B d = e.symm (LinearMap.lTensor S (Matrix.toLin' B) (e d))
```

lemma valEnd_apply_symm_tmul [source]

```
lemma valEnd_apply_symm_tmul (B : Matrix ι ι ℂ) (s : S) (v : ι → ℂ) :
    valEnd e B (e.symm (s ⊗ₜ[ℂ] v)) = e.symm (s ⊗ₜ[ℂ] (B.mulVec v))
```

lemma valEnd_add [source]

```
lemma valEnd_add (A B : Matrix ι ι ℂ) : valEnd e (A + B) = valEnd e A + valEnd e B
```

lemma valEnd_smul [source]

```
lemma valEnd_smul (z : ℂ) (A : Matrix ι ι ℂ) : valEnd e (z • A) = z • valEnd e A
```

lemma valEnd_zero [source]

```
lemma valEnd_zero : valEnd e (0 : Matrix ι ι ℂ) = 0
```

lemma valEnd_neg [source]

```
lemma valEnd_neg (A : Matrix ι ι ℂ) : valEnd e (-A) = -valEnd e A
```

lemma valEnd_multiset_sum [source]

```
lemma valEnd_multiset_sum (m : Multiset (Matrix ι ι ℂ)) :
    valEnd e m.sum = (m.map (valEnd e)).sum
```

lemma valEnd_mul [source]

```
lemma valEnd_mul (A B : Matrix ι ι ℂ) : valEnd e (A * B) = valEnd e A ∘ₗ valEnd e B
```

lemma valEnd_one [source]

```
lemma valEnd_one : valEnd e (1 : Matrix ι ι ℂ) = LinearMap.id
```

def jetEnd [source]

The endomorphism of `S ⊗ (ι → SpaceTimeAlgebra)` defined by a matrix of jets on the internal index.

```
noncomputable def jetEnd (A : Matrix ι ι SpaceTimeAlgebra) :
    S ⊗[ℂ] (ι → SpaceTimeAlgebra) →ₗ[ℂ] S ⊗[ℂ] (ι → SpaceTimeAlgebra)
```

lemma jetEnd_eq_lTensor [source]

```
lemma jetEnd_eq_lTensor (A : Matrix ι ι SpaceTimeAlgebra) :
    jetEnd S A = LinearMap.lTensor S
      ((Matrix.toLinAlgEquiv' A : Module.End SpaceTimeAlgebra
          (ι → SpaceTimeAlgebra)).restrictScalars ℂ)
```

lemma jetEnd_tmul [source]

```
lemma jetEnd_tmul (A : Matrix ι ι SpaceTimeAlgebra) (s : S) (w : ι → SpaceTimeAlgebra) :
    jetEnd S A (s ⊗ₜ[ℂ] w) = s ⊗ₜ[ℂ] (A.mulVec w)
```

lemma jetEnd_one [source]

```
lemma jetEnd_one : jetEnd S (1 : Matrix ι ι SpaceTimeAlgebra) = LinearMap.id
```

lemma jetEnd_mul [source]

```
lemma jetEnd_mul (A B : Matrix ι ι SpaceTimeAlgebra) : jetEnd S (A * B) =
    jetEnd S A ∘ₗ jetEnd S B
```

def matEnd [source]

The endomorphism of the jets `SpaceTimeAlgebra ⊗ V` of the field defined by a matrix of jets on the internal index, through `jetEquiv`.

```
noncomputable def matEnd
    (A : Matrix ι ι SpaceTimeAlgebra) : SpaceTimeAlgebra ⊗[ℂ] V →ₗ[ℂ] SpaceTimeAlgebra ⊗[ℂ] V
```

lemma matEnd_apply [source]

```
lemma matEnd_apply (A : Matrix ι ι SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V) :
    matEnd e A z = (jetEquiv e).symm (jetEnd S A (jetEquiv e z))
```

lemma matEnd_one [source]

```
lemma matEnd_one : matEnd e (1 : Matrix ι ι SpaceTimeAlgebra) = LinearMap.id
```

lemma matEnd_mul [source]

```
lemma matEnd_mul (A B : Matrix ι ι SpaceTimeAlgebra) :
    matEnd e (A * B) = matEnd e A ∘ₗ matEnd e B
```

lemma matEnd_smul [source]

The matrix endomorphisms are fibrewise: they commute with multiplication by scalar jets.

```
lemma matEnd_smul (A : Matrix ι ι SpaceTimeAlgebra) (χ : SpaceTimeAlgebra)
    (z : SpaceTimeAlgebra ⊗[ℂ] V) :
    matEnd e A (χ • z) = χ • matEnd e A z
```

def repJet [source]

***The jet gauge action** of a matrix representation on the jets of a `V`-valued field: the matrix of jets acts on the internal index by matrix–vector multiplication, with the Lorentz factor untouched.*

```
noncomputable def repJet : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V) where
```

lemma repJet_apply [source]

```
lemma repJet_apply (U : GJ) : R.repJet e U = matEnd e (R.mat U)
```

lemma repJet_smul [source]

***The jet gauge action is fibrewise**: it commutes with multiplication by scalar jets.*

```
lemma repJet_smul (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V) :
    R.repJet e U (χ • z) = χ • R.repJet e U z
```

def repAlgebra [source]

***The action of the gauge algebra** of a matrix representation on the target space `V ≃ S ⊗ (ι → ℂ)`: the action matrix acts on the internal index, real-linearly in the algebra slot and complex-linearly in the value slot.*

```
noncomputable def repAlgebra : 𝔤 →ₗ[ℝ] V →ₗ[ℂ] V where
```

lemma repAlgebra_apply [source]

```
lemma repAlgebra_apply (c : 𝔤) : R.repAlgebra e c = valEnd e (R.act c)
```

lemma repCoeff_eq [source]

***The base-point Taylor coefficients of the jet gauge action** are the endomorphisms of the base-point Taylor coefficients of the matrix of jets.*

```
lemma repCoeff_eq (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    GaugeAlgebraRealization.repCoeff (R.repJet e) U x
      = valEnd e ((R.mat U).map fun f =>
          constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x f))
```

lemma isInfinitesimalActionOf [source]

***The action of the gauge algebra of a matrix representation is the infinitesimal action underlying its jet gauge action**: the base-point Taylor coefficients obey the Maurer–Cartan Leibniz law and intertwine the action with the adjoint transports.*

```
lemma isInfinitesimalActionOf :
    jets.IsInfinitesimalActionOf (R.repAlgebra e) (R.repJet e)
```

def repLorentz [source]

The Lorentz action on the target space `V ≃ S ⊗ (ι → ℂ)`: the given action on the Lorentz factor `S`, transported through `e`.

```
noncomputable def repLorentz (ρ : Representation ℂ SL(2,ℂ) S) :
    Representation ℂ SL(2,ℂ) V
```

def repGlobal [source]

***The global gauge action** of a matrix representation of jets: a global gauge transformation acts by the constant term of the matrix of its constant jet.*

```
noncomputable def repGlobal : Representation ℂ G₀ V where
```

lemma repGlobal_apply [source]

```
lemma repGlobal_apply (g : G₀) :
    R.repGlobal e g = valEnd e
        ((R.mat (jets.ofConstant g)).map (constantCoeff : SpaceTimeAlgebra → ℂ))
```

lemma repGlobal_apply_symm_tmul [source]

The global action on a pure tensor: the constant term of the matrix acts on the internal index.

```
lemma repGlobal_apply_symm_tmul (g : G₀) (s : S) (v : ι → ℂ) :
    R.repGlobal e g (e.symm (s ⊗ₜ[ℂ] v))
      = e.symm (s ⊗ₜ[ℂ]
          ((R.mat (jets.ofConstant g)).map (constantCoeff : SpaceTimeAlgebra → ℂ)).mulVec v)
```

lemma map_C_mulVec_smul [source]

A matrix of constant jets acts on a scalar jet times a constant vector through its constant matrix.

```
lemma map_C_mulVec_smul (B : Matrix ι ι ℂ) (v : ι → ℂ) (χ : SpaceTimeAlgebra) :
    (B.map (C : ℂ → SpaceTimeAlgebra)).mulVec (fun i => v i • χ) = fun i => (B.mulVec v) i • χ
```

lemma repJet_ofConstant [source]

***On jets of constant gauge transformations the jet action is the global action** on the value factor, provided the matrices of constant jets are constant.*

```
lemma repJet_ofConstant
    (hconst : ∀ g, R.mat (jets.ofConstant g)
      = ((R.mat (jets.ofConstant g)).map (constantCoeff : SpaceTimeAlgebra → ℂ)).map
          (C : ℂ → SpaceTimeAlgebra))
    (g : G₀) :
    R.repJet e (jets.ofConstant g) = TensorProduct.map LinearMap.id (R.repGlobal e g)
```

def matterField [source]

***The matter field of a matrix representation**: the target space `V ≃ S ⊗ (ι → ℂ)` with the Lorentz action on `S`, the jet gauge action of the matrix representation, and the given mass weight and declared statistics.*

```
noncomputable def matterField (ρ : Representation ℂ SL(2,ℂ) S) (w : ℕ) (σ : FieldStatistic) :
    MatterField jets where
```

lemma matterField_V [source]

```
@[simp]
lemma matterField_V : (R.matterField e ρ w σ).V = V
```

lemma matterField_repLorentz [source]

```
@[simp]
lemma matterField_repLorentz : (R.matterField e ρ w σ).repLorentz = repLorentz e ρ
```

lemma matterField_repJet [source]

```
@[simp]
lemma matterField_repJet : (R.matterField e ρ w σ).repJet = R.repJet e
```

lemma matterField_repAlgebra [source]

```
@[simp]
lemma matterField_repAlgebra : (R.matterField e ρ w σ).repAlgebra = R.repAlgebra e
```

lemma matterField_massWeight [source]

```
@[simp]
lemma matterField_massWeight : (R.matterField e ρ w σ).massWeight = w
```

lemma matterField_statistic [source]

```
@[simp]
lemma matterField_statistic : (R.matterField e ρ w σ).statistic = σ
```

lemma repLorentz_apply_symm_tmul [source]

```
lemma repLorentz_apply_symm_tmul (Λ : SL(2,ℂ)) (s : S) (v : ι → ℂ) :
    repLorentz e ρ Λ (e.symm (s ⊗ₜ[ℂ] v)) = e.symm (ρ Λ s ⊗ₜ[ℂ] v)
```

lemma repAlgebra_comm_repLorentz [source]

The gauge algebra acts on the internal index and the Lorentz group on the Lorentz factor, so the two actions commute.

```
lemma repAlgebra_comm_repLorentz (c : 𝔤) (Λ : SL(2,ℂ)) (v : V) :
    R.repAlgebra e c (repLorentz e ρ Λ v) = repLorentz e ρ Λ (R.repAlgebra e c v)
```

lemma matterField_gaugeLorentzCompatible [source]

The matter field of a matrix representation satisfies `MatterField.GaugeLorentzCompatible`, for every matrix representation and Lorentz factor: the two actions live on different tensor factors.

```
lemma matterField_gaugeLorentzCompatible :
    (R.matterField e ρ w σ).GaugeLorentzCompatible
```

lemma matterField_pureJetsActTrivially [source]

The matter field of a matrix representation satisfies `MatterField.PureJetsActTrivially` as soon as the matrix of every jet with trivial value has identity constant term. This hypothesis is not a consequence of the axioms of `MatrixRep`, which fix the constant term of `mat` on pure jets only up to a scalar character.

```
lemma matterField_pureJetsActTrivially
    (hmat : ∀ {W : GJ}, jets.eval W = 1 → (R.mat W).map
        (constantCoeff : SpaceTimeAlgebra → ℂ) = 1) :
    (R.matterField e ρ w σ).PureJetsActTrivially
```

2.245 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.MatrixRep.Constructions

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/MatrixRep/Constructions.lean on GitHub.

lemma kronecker_one_map [source]

```
lemma kronecker_one_map [MulZeroOneClass α] [MulZeroOneClass β] (φ : α → β) (h0 : φ 0 = 0)
    (A : Matrix ι₁ ι₁ α) :
    (A ⊗ₖ (1 : Matrix ι₂ ι₂ α)).map φ = (A.map φ) ⊗ₖ (1 : Matrix ι₂ ι₂ β)
```

lemma one_kronecker_map [source]

```
lemma one_kronecker_map [MulZeroOneClass α] [MulZeroOneClass β] (φ : α → β) (h0 : φ 0 = 0)
    (B : Matrix ι₂ ι₂ α) :
    ((1 : Matrix ι₁ ι₁ α) ⊗ₖ B).map φ = (1 : Matrix ι₁ ι₁ β) ⊗ₖ (B.map φ)
```

lemma kronecker_map_of_mul [source]

```
lemma kronecker_map_of_mul [Mul α] [Mul β] (φ : α → β) (hφ : ∀ a b, φ (a * b) = φ a * φ b)
    (A : Matrix ι₁ ι₁ α) (B : Matrix ι₂ ι₂ α) :
    (A ⊗ₖ B).map φ = (A.map φ) ⊗ₖ (B.map φ)
```

lemma neg_kronecker [source]

```
lemma neg_kronecker [Mul α] [HasDistribNeg α] (A : Matrix ι₁ ι₁ α) (B : Matrix ι₂ ι₂ α) :
    (-A) ⊗ₖ B = -(A ⊗ₖ B)
```

lemma kronecker_neg [source]

```
lemma kronecker_neg [Mul α] [HasDistribNeg α] (A : Matrix ι₁ ι₁ α) (B : Matrix ι₂ ι₂ α) :
    A ⊗ₖ (-B) = -(A ⊗ₖ B)
```

def trivial [source]

***The trivial representation**: the one-dimensional singlet, on which every gauge jet acts as the identity and the gauge algebra by zero.*

```
noncomputable def trivial : MatrixRep jets (Fin 1) where
```

def kron [source]

***The Kronecker product** of two matrix representations: the gauge jets act by the Kronecker product of the two matrices of jets, the gauge algebra by the Kronecker sum of the two action matrices.*

```
noncomputable def kron (R₁ : MatrixRep jets ι₁) (R₂ : MatrixRep jets ι₂) :
    MatrixRep jets (ι₁ × ι₂) where
```

lemma foldl_pderiv_star [source]

The iterated formal derivative commutes with conjugation.

```
lemma foldl_pderiv_star (x : Multiset (Fin 1 ⊕ Fin 3)) (f : SpaceTimeAlgebra) :
    x.foldl (fun h ρ => pderiv ρ h) (star f) = star (x.foldl (fun h ρ => pderiv ρ h) f)
```

def conj [source]

***The conjugate representation**: the gauge jets act by the entrywise conjugate matrix of jets, the gauge algebra by the entrywise conjugate action matrix.*

```
noncomputable def conj (R : MatrixRep jets ι) : MatrixRep jets ι where
```

2.246 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.MatrixRep.Factors

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/MatrixRep/Factors.lean on GitHub.

lemma pderiv_chargePow [source]

***The derivative of a power of a unitary jet**: `∂_μ (u ^ n) = n · u ^ n · (u⁻¹ ∂_μ u)`, for every integer `n`.*

```
lemma pderiv_chargePow (n : ℤ) (w : unitary SpaceTimeAlgebra) (μ : Fin 1 ⊕ Fin 3) :
    pderiv μ (chargePow n w)
      = (n : ℂ) •
          (chargePow n w * (star (w : SpaceTimeAlgebra) * pderiv μ (w : SpaceTimeAlgebra)))
```

lemma pderiv_chargePow_u [source]

The derivative of the charge-`n` power of the unitary jet of a gauge jet, in terms of the Maurer–Cartan form: `∂_μ (u ^ n) = -(i n) φJ (ω_μ U) · u ^ n`.

```
lemma pderiv_chargePow_u (n : ℤ) (U : GJ) (μ : Fin 1 ⊕ Fin 3) :
    pderiv μ (chargePow n (F.u U))
      = -(((Complex.I * n) • F.φJ (jets.maurerCartan U μ)) * chargePow n (F.u U))
```

def charge [source]

***The charge twist**: a matrix representation twisted by the charge-`n` power of the unitary jet of the `U(1)` factor. The gauge algebra acts by the original action plus `i n` times the `u(1)` component.*

```
noncomputable def charge (n : ℤ) (R : MatrixRep jets ι) : MatrixRep jets ι where
```

def fund [source]

***The fundamental representation** of an `SU(n)` factor: the gauge jets act by their unitary matrices of jets, the gauge algebra by `i` times its matrix component.*

```
noncomputable def fund : MatrixRep jets n where
```

2.247 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.MatrixRep.Table

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/MatrixRep/Table.lean on GitHub.

inductive SURep [source]

A representation label under `SU(n)`.

```
inductive SURep
  | singlet
  | fund
  | antifund
  deriving DecidableEq, Repr
```

abbrev dim [source]

The dimension of the representation of `SU(n)` a label names.

```
abbrev SURep.dim (n : ℕ) : SURep → ℕ
  | .singlet => 1
  | .fund => n
  | .antifund => n
```

def SURep.rep [source]

The matrix representation of an `SU(n)` factor named by a label.

```
noncomputable def SURep.rep {n : ℕ} (F : SUFactor jets (Fin n)) :
    (r : SURep) → MatrixRep jets (Fin (r.dim n))
  | .singlet => MatrixRep.trivial
  | .fund => F.fund
  | .antifund => F.fund.conj
```

abbrev Charge [source]

The charge a field carries under a factor: an integer under `U(1)`, a representation label under `SU(n)`.

```
abbrev Factor.Charge : Factor jets → Type
  | .U1 _ => ℤ
  | .SU _ => SURep
```

abbrev Charges [source]

***The charge tuple** of a row: one charge per factor, as a nested pair so that a row reads `(1, .fund, .fund)`.*

```
abbrev Charges : Factors jets → Type
  | [] => Unit
  | [f] => f.Charge
  | f :: g :: gs => f.Charge × Charges (g :: gs)
```

abbrev IdxOfDims [source]

***The internal index of a list of dimensions**: the product of the `Fin n` over the list, with `Fin 1` for the empty list and no trailing factor for a one-element list.*

```
abbrev IdxOfDims : List ℕ → Type
  | [] => Fin 1
  | [n] => Fin n
  | n :: m :: ms => Fin n × IdxOfDims (m :: ms)
```

def fintype [source]

The index of a list of dimensions is finite.

```
@[instance_reducible]
def IdxOfDims.fintype : (ds : List ℕ) → Fintype (IdxOfDims ds)
  | [] => inferInstanceAs (Fintype (Fin 1))
  | [n] => inferInstanceAs (Fintype (Fin n))
  | n :: m :: ms =>
    letI
```

def decidableEq [source]

The index of a list of dimensions has decidable equality.

```
@[instance_reducible]
def IdxOfDims.decidableEq : (ds : List ℕ) → DecidableEq (IdxOfDims ds)
```

```
  | [] => inferInstanceAs (DecidableEq (Fin 1))
  | [n] => inferInstanceAs (DecidableEq (Fin n))
  | n :: m :: ms =>
    letI
```

instance Fintype (IdxOfDims ds) [source]

```
instance (ds : List ℕ) : Fintype (IdxOfDims ds)
```

instance DecidableEq (IdxOfDims ds) [source]

```
instance (ds : List ℕ) : DecidableEq (IdxOfDims ds)
```

abbrev SURep.dims [source]

The dimensions an `SU(n)` label contributes to the internal index: none for the singlet, `n` for the fundamental and the antifundamental.

```
abbrev SURep.dims (n : ℕ) : SURep → List ℕ
  | .singlet => []
  | .fund => [n]
  | .antifund => [n]
```

abbrev Charges.dims [source]

***The dimensions of the internal index** of a charge tuple: the sizes of the nontrivial `SU(n)` representations it names, in the order of the factors. `U(1)` factors and singlets contribute nothing, so that a field charged under a single `SU(n)` factor is indexed by `Fin n` alone.*

```
abbrev Charges.dims : (Γ : Factors jets) → Charges Γ → List ℕ
  | [], _ => []
  | [.U1 _], _ => []
  | [.SU (n := n) _], r => SURep.dims n r
  | .U1 _ :: g :: gs, c => Charges.dims (g :: gs) c.2
  | .SU (n := n) _ :: g :: gs, c => SURep.dims n c.1 ++ Charges.dims (g :: gs) c.2
```

abbrev LocalGaugeData.Idx [source]

***The internal index** of a field with the given charges: the product of the nontrivial `SU(n)`-representation indices.*

```
abbrev Idx (Γ : Factors jets) (c : Charges Γ) : Type
```

def kronDims [source]

The Kronecker product with the representation on the remaining dimensions, with the trivial representation dropped when there are none.

```
noncomputable def MatrixRep.kronDims {n : ℕ} (R₁ : MatrixRep jets (Fin n)) :
    (ds : List ℕ) → MatrixRep jets (IdxOfDims ds) → MatrixRep jets (IdxOfDims (n :: ds))
  | [], _ => R₁
  | _ :: _, R₂ => R₁.kron R₂
```

def repDims [source]

The matrix representation of an `SU(n)` factor named by a label, on the index of the dimensions the label contributes.

```
noncomputable def SURep.repDims {n : ℕ} (F : SUFactor jets (Fin n)) :
    (r : SURep) → MatrixRep jets (IdxOfDims (r.dims n))
  | .singlet => MatrixRep.trivial
  | .fund => F.fund
  | .antifund => F.fund.conj
```

def suRep [source]

The `SU(n)` part of the representation named by a charge tuple: the Kronecker product of the nontrivial representations the labels name.

```
noncomputable def Charges.suRep :
    (Γ : Factors jets) → (c : Charges Γ) → MatrixRep jets (IdxOfDims (Charges.dims Γ c))
  | [], _ => MatrixRep.trivial
  | [.U1 _], _ => MatrixRep.trivial
  | [.SU F], r => SURep.repDims F r
  | .U1 _ :: g :: gs, c => Charges.suRep (g :: gs) c.2
  | .SU _ :: g :: gs, (.singlet, c) => Charges.suRep (g :: gs) c
  | .SU F :: g :: gs, (.fund, c) => F.fund.kronDims _ (Charges.suRep (g :: gs) c)
  | .SU F :: g :: gs, (.antifund, c) => F.fund.conj.kronDims _ (Charges.suRep (g :: gs) c)
```

def twist [source]

The `U(1)` part of the representation named by a charge tuple: the charge twists of all the `U(1)` factors, applied to a given representation.

```
noncomputable def Charges.twist {ι : Type} [Fintype ι] [DecidableEq ι] :
    (Γ : Factors jets) → Charges Γ → MatrixRep jets ι → MatrixRep jets ι
  | [], _, R => R
  | [.U1 F], q, R => F.charge q R
  | [.SU _], _, R => R
  | .U1 F :: g :: gs, c, R => F.charge c.1 (Charges.twist (g :: gs) c.2 R)
  | .SU _ :: g :: gs, c, R => Charges.twist (g :: gs) c.2 R
```

def Charges.rep [source]

***The matrix representation named by a charge tuple**: the Kronecker product of the nontrivial `SU(n)` representations the labels name, twisted by the charge powers of the `U(1)` jets.*

```
noncomputable def Charges.rep (Γ : Factors jets) (c : Charges Γ) : MatrixRep jets (Idx Γ c)
```

inductive LorentzLabel [source]

***The Lorentz label of a field**: a left- or right-handed Weyl spinor or a scalar.*

```
inductive LorentzLabel
  | L
  | R
  | scalar
  deriving DecidableEq, Repr
```

def statistic [source]

The statistics a table row declares for a field with the given label, fermionic for a Weyl spinor and bosonic for a scalar. Like `massWeight`, it is a convention of the tables, chosen to match the spin–statistics relation.

```
def statistic : LorentzLabel → FieldStatistic
  | .L => .fermionic
  | .R => .fermionic
  | .scalar => .bosonic
```

abbrev Space [source]

The Lorentz factor of the target space of a field with the given label.

```
abbrev Space : LorentzLabel → Type
  | .L => Fermion.LeftHandedWeyl
  | .R => Fermion.RightHandedWeyl
  | .scalar => ℂ
```

instance Module.Finite ℂ Fermion.LeftHandedWeyl [source]

```
instance : Module.Finite ℂ Fermion.LeftHandedWeyl
```

instance Module.Finite ℂ Fermion.RightHandedWeyl [source]

```
instance : Module.Finite ℂ Fermion.RightHandedWeyl
```

instance instAddCommGroupSpace [source]

```
instance instAddCommGroupSpace : (l : LorentzLabel) → AddCommGroup l.Space
  | .L => inferInstanceAs (AddCommGroup Fermion.LeftHandedWeyl)
  | .R => inferInstanceAs (AddCommGroup Fermion.RightHandedWeyl)
  | .scalar => inferInstanceAs (AddCommGroup ℂ)
```

instance instModuleSpace [source]

```
instance instModuleSpace : (l : LorentzLabel) → Module ℂ l.Space
  | .L => inferInstanceAs (Module ℂ Fermion.LeftHandedWeyl)
  | .R => inferInstanceAs (Module ℂ Fermion.RightHandedWeyl)
  | .scalar => inferInstanceAs (Module ℂ ℂ)
```

instance instFreeSpace [source]

```
instance instFreeSpace : (l : LorentzLabel) → Module.Free ℂ l.Space
  | .L => inferInstanceAs (Module.Free ℂ Fermion.LeftHandedWeyl)
  | .R => inferInstanceAs (Module.Free ℂ Fermion.RightHandedWeyl)
  | .scalar => inferInstanceAs (Module.Free ℂ ℂ)
```

instance instFiniteSpace [source]

```
instance instFiniteSpace : (l : LorentzLabel) → Module.Finite ℂ l.Space
  | .L => inferInstanceAs (Module.Finite ℂ Fermion.LeftHandedWeyl)
```

```
  | .R => inferInstanceAs (Module.Finite ℂ Fermion.RightHandedWeyl)
  | .scalar => inferInstanceAs (Module.Finite ℂ ℂ)
```

def LorentzLabel.rep [source]

The representation of the Lorentz group on the Lorentz factor of a label.

```
noncomputable def rep : (l : LorentzLabel) → Representation ℂ SL(2,ℂ) l.Space
  | .L => Fermion.LeftHandedWeyl.rep
  | .R => Fermion.RightHandedWeyl.rep
  | .scalar => Representation.trivial ℂ SL(2,ℂ) ℂ
```

def LorentzLabel.massWeight [source]

The mass weight of a field with the given label, in the units in which a derivative has weight `2`: `3` for a Weyl spinor, `2` for a scalar.

```
def massWeight : LorentzLabel → ℕ
  | .L => 3
  | .R => 3
  | .scalar => 2
```

abbrev basisIndex [source]

The index of the basis of the Lorentz factor of a label: the two spinor components of a Weyl spinor, one component for a scalar.

```
abbrev basisIndex : LorentzLabel → Type
  | .L => Fin 2
  | .R => Fin 2
  | .scalar => Unit
```

instance instFintypeBasisIndex [source]

```
instance instFintypeBasisIndex : (l : LorentzLabel) → Fintype l.basisIndex
  | .L => inferInstanceAs (Fintype (Fin 2))
  | .R => inferInstanceAs (Fintype (Fin 2))
  | .scalar => inferInstanceAs (Fintype Unit)
```

instance instDecidableEqBasisIndex [source]

```
instance instDecidableEqBasisIndex : (l : LorentzLabel) → DecidableEq l.basisIndex
  | .L => inferInstanceAs (DecidableEq (Fin 2))
  | .R => inferInstanceAs (DecidableEq (Fin 2))
  | .scalar => inferInstanceAs (DecidableEq Unit)
```

def LorentzLabel.basis [source]

The basis of the Lorentz factor of a label: the Weyl basis, or `1` for a scalar.

```
noncomputable def basis : (l : LorentzLabel) → Module.Basis l.basisIndex ℂ l.Space
  | .L => Fermion.LeftHandedWeyl.basis
  | .R => Fermion.RightHandedWeyl.basis
  | .scalar => Module.Basis.singleton Unit ℂ
```

abbrev MatterFieldData [source]

***The data of a matter field**: its Lorentz label and its charge tuple, so that a field reads `(.L, .singlet, .fund, -3)`.*

```
abbrev MatterFieldData (Γ : Factors jets) : Type
```

abbrev lorentz [source]

The Lorentz label of the field.

```
abbrev lorentz : LorentzLabel
```

abbrev charges [source]

The charges of the field.

```
abbrev charges : Charges Γ
```

abbrev MatterFieldData.Idx [source]

The internal index of the field.

```
abbrev Idx : Type
```

abbrev MatterFieldData.rep [source]

The matrix representation named by the charges of the field.

```
noncomputable abbrev rep : MatrixRep jets M.Idx
```

abbrev MatterFieldData.massWeight [source]

The mass weight of the field.

```
abbrev massWeight : ℕ
```

abbrev V [source]

***The target space of the field**: the Lorentz factor of its label tensored with the functions on its internal index.*

```
abbrev V : Type
```

def MatterFieldData.basis [source]

***The basis of the target space**: the basis of the Lorentz factor tensored with the coordinate basis of the internal index.*

```
noncomputable def basis : Module.Basis (M.lorentz.basisIndex × M.Idx) ℂ M.V
```

lemma basis_apply [source]

A basis vector of the target space is a basis vector of the Lorentz factor tensored with a coordinate vector of the internal index.

```
@[simp]
lemma basis_apply (a : M.lorentz.basisIndex) (i : M.Idx) :
    M.basis (a, i) = M.lorentz.basis a ⊗ₜ Pi.single i 1
```

def toMatterFieldOn [source]

***The matter field of a datum on a presented target space**: a target space `V` identified with the Lorentz factor of the label tensored with the internal index of the charges, transforming in the representation the charges name, with the mass weight and statistic its label assigns.*

```
noncomputable def toMatterFieldOn {V : Type} [AddCommGroup V] [Module ℂ V]
    [Module.Free ℂ V] [Module.Finite ℂ V]
    (e : V ≃ₗ[ℂ] M.lorentz.Space ⊗[ℂ] (M.Idx → ℂ)) : MatterField jets
```

def toMatterField [source]

***The matter field of a datum**: the target space is the Lorentz factor of the label tensored with the internal index of the charges.*

```
noncomputable def toMatterField : MatterField jets
```

lemma toMatterFieldOn_V [source]

```
@[simp]
lemma toMatterFieldOn_V : (M.toMatterFieldOn e).V = V
```

lemma toMatterFieldOn_repJet [source]

```
@[simp]
lemma toMatterFieldOn_repJet : (M.toMatterFieldOn e).repJet = M.rep.repJet e
```

lemma toMatterFieldOn_repAlgebra [source]

```
@[simp]
lemma toMatterFieldOn_repAlgebra : (M.toMatterFieldOn e).repAlgebra = M.rep.repAlgebra e
```

lemma toMatterFieldOn_repLorentz [source]

```
@[simp]
lemma toMatterFieldOn_repLorentz :
    (M.toMatterFieldOn e).repLorentz = MatrixRep.repLorentz e M.lorentz.rep
```

lemma toMatterFieldOn_massWeight [source]

```
@[simp]
lemma toMatterFieldOn_massWeight : (M.toMatterFieldOn e).massWeight = M.massWeight
```

lemma toMatterField_massWeight [source]

```
@[simp]
lemma toMatterField_massWeight : M.toMatterField.massWeight = M.massWeight
```

lemma toMatterFieldOn_statistic [source]

The matter field of a datum on a presented target space has the statistics its Lorentz label assigns.

```
@[simp]
lemma toMatterFieldOn_statistic : (M.toMatterFieldOn e).statistic = M.lorentz.statistic
```

lemma toMatterField_statistic [source]

The matter field of a datum has the statistics its Lorentz label assigns.

```
@[simp]
lemma toMatterField_statistic : M.toMatterField.statistic = M.lorentz.statistic
```

lemma toMatterField_V [source]

The target space of the matter field of a datum is the target space of the datum.

```
@[simp]
lemma toMatterField_V : M.toMatterField.V = M.V
```

lemma toMatterFieldOn_gaugeLorentzCompatible [source]

The gauge and Lorentz actions of the matter field of a datum commute.

```
lemma toMatterFieldOn_gaugeLorentzCompatible :
    (M.toMatterFieldOn e).GaugeLorentzCompatible
```

lemma toMatterField_gaugeLorentzCompatible [source]

The gauge and Lorentz actions of the matter field of a datum commute.

```
lemma toMatterField_gaugeLorentzCompatible : M.toMatterField.GaugeLorentzCompatible
```

abbrev FieldData [source]

***The field data of a model**: for each field of the model, its number of generations and its matter field data.*

```
abbrev FieldData (Γ : Factors jets) (Fields : Type) : Type
```

abbrev generations [source]

The number of generations of a field.

```
abbrev generations (f : Fields) : ℕ
```

abbrev data [source]

The matter field data of a field.

```
abbrev data (f : Fields) : MatterFieldData Γ
```

abbrev Fermions [source]

The fermionic fields of the model: those whose label assigns fermionic statistics.

```
abbrev Fermions : Type
```

abbrev Bosons [source]

The bosonic fields of the model: those whose label assigns bosonic statistics.

```
abbrev Bosons : Type
```

abbrev FermionSpecies [source]

The fermionic species of the model: a fermionic field together with a generation.

```
abbrev FermionSpecies : Type
```

abbrev BosonSpecies [source]

The bosonic species of the model: a bosonic field together with a generation.

```
abbrev BosonSpecies : Type
```

def toGaugeFieldData [source]

***The field content of a model**: one matter field per species, the matter field of the species' data.*

```
noncomputable def toGaugeFieldData : GaugeFieldData jets where
```

lemma toGaugeFieldData_FermionSpecies [source]

```
@[simp]
lemma toGaugeFieldData_FermionSpecies :
    D.toGaugeFieldData.FermionSpecies = D.FermionSpecies
```

lemma toGaugeFieldData_fermion [source]

```
@[simp]
lemma toGaugeFieldData_fermion (s : D.FermionSpecies) :
    D.toGaugeFieldData.fermion s = (D.data s.1.1).toMatterField
```

lemma toGaugeFieldData_BosonSpecies [source]

```
@[simp]
lemma toGaugeFieldData_BosonSpecies : D.toGaugeFieldData.BosonSpecies = D.BosonSpecies
```

lemma toGaugeFieldData_boson [source]

```
@[simp]
lemma toGaugeFieldData_boson (s : D.BosonSpecies) :
    D.toGaugeFieldData.boson s = (D.data s.1.1).toMatterField
```

lemma toGaugeFieldData_gaugeLorentzCompatible [source]

The field content of a model satisfies `GaugeFieldData.GaugeLorentzCompatible`: every species is a matrix representation on the internal index tensored with a Lorentz factor.

```
lemma toGaugeFieldData_gaugeLorentzCompatible : D.toGaugeFieldData.GaugeLorentzCompatible
```

2.248 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.Pi

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/Pi.lean on GitHub.

lemma piMap_add_piMap [source]

```
private lemma piMap_add_piMap (f g : ∀ i, W i →ₗ[ℂ] W i) :
    LinearMap.piMap (fun i => f i + g i) = LinearMap.piMap f + LinearMap.piMap g
```

lemma piMap_neg [source]

```
private lemma piMap_neg (f : ∀ i, W i →ₗ[ℂ] W i) :
    LinearMap.piMap (fun i => -f i) = -LinearMap.piMap f
```

lemma piMap_comp [source]

```
private lemma piMap_comp (f g : ∀ i, W i →ₗ[ℂ] W i) :
    (LinearMap.piMap f).comp (LinearMap.piMap g)
      = LinearMap.piMap fun i => (f i).comp (g i)
```

lemma piMap_multiset_sum [source]

```
private lemma piMap_multiset_sum {κ : Type} (S : Multiset κ)
    (f : κ → ∀ i, W i →ₗ[ℂ] W i) :
    LinearMap.piMap (fun i => (S.map fun k => f k i).sum)
      = (S.map fun k => LinearMap.piMap (f k)).sum
```

def repPi [source]

The product of a family of representations, acting index by index. This is the indexed analogue of `Representation.prod`, which Mathlib provides only in the binary case.

```
noncomputable def repPi {W : ι → Type} [∀ i, AddCommGroup (W i)]
    [∀ i, Module ℂ (W i)] (ρ : ∀ i, Representation ℂ GJ (W i)) :
    Representation ℂ GJ (∀ i, W i) where
```

def repJetPi [source]

***The jet gauge action of an indexed direct sum**: the family of actions, read through the identification of the jets of `∀ i, (M i).V` with the family of jets.*

```
noncomputable def repJetPi : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] (∀ i, (M i).V)) where
```

lemma repJetPi_apply [source]

```
lemma repJetPi_apply (U : GJ) (z : SpaceTimeAlgebra ⊗[ℂ] (∀ i, (M i).V)) :
    repJetPi M U z = (jetPiEquiv fun i => (M i).V).symm
      (fun i => (M i).repJet U (jetPiEquiv (fun i => (M i).V) z i))
```

lemma repJetPi_smul [source]

The jet gauge action of an indexed direct sum is fibrewise. It acts index by index, and each summand is fibrewise, so multiplication by a scalar jet passes through the splitting untouched. This is the field `repJet_smul` of `MatterField.pi`, stated separately so that it can be used without fixing a common mass weight.

```
lemma repJetPi_smul (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] (∀ i, (M i).V))
    :
    repJetPi M U (χ • z) = χ • repJetPi M U z
```

lemma lTensor_proj_repJetPi [source]

A summand is a subrepresentation of the jet gauge action of the direct sum. The projection onto the value space of one summand, applied to the jets, intertwines the summed action with that summand's own: the summed action is the family of the actions, and reading off a summand of the jets is the projection on the value factor.

```
lemma lTensor_proj_repJetPi (i : ι) (U : GJ) :
    (LinearMap.lTensor SpaceTimeAlgebra (LinearMap.proj i)).comp (repJetPi M U)
      = ((M i).repJet U).comp (LinearMap.lTensor SpaceTimeAlgebra (LinearMap.proj i))
```

def repAlgebraPi [source]

***The infinitesimal action of an indexed direct sum**: the family of actions, one on each summand.*

```
noncomputable def repAlgebraPi : 𝔤 →ₗ[ℝ] (∀ i, (M i).V) →ₗ[ℂ] (∀ i, (M i).V) where
```

lemma repAlgebraPi_apply [source]

```
@[simp]
lemma repAlgebraPi_apply (c : 𝔤) :
    repAlgebraPi M c = LinearMap.piMap fun i => (M i).repAlgebra c
```

lemma repCoeff_repJetPi [source]

The base-point Taylor coefficients of the summed jet action are the family of the coefficients of the summands: `jetOfConstant`, `jetIteratedDeriv` and `jetEval` all act index by index through `jetPiEquiv`.

```
lemma repCoeff_repJetPi (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    GaugeAlgebraRealization.repCoeff (repJetPi M) U x =
      LinearMap.piMap fun i => GaugeAlgebraRealization.repCoeff (M i).repJet U x
```

lemma isInfinitesimalActionOf_repAlgebraPi [source]

***The index-by-index algebra action generates the index-by-index jet action**: both laws of `IsInfinitesimalActionOf` are the corresponding laws of the summands, read through `repCoeff_repJetPi`, since `piMap` is additive in the family and composes index by index.*

```
lemma isInfinitesimalActionOf_repAlgebraPi :
    jets.IsInfinitesimalActionOf (repAlgebraPi M) (repJetPi M) where
```

def pi [source]

***The direct sum of a finite family of matter fields** with common mass weight `w` and common statistics `σ`, given with proofs that every summand has them. It is one field valued in `∀ i, (M i).V`, of weight `w` and statistics `σ`, with every action acting index by index.*

```
noncomputable def pi (w : ℕ) (_h : ∀ i, (M i).massWeight = w) (σ : FieldStatistic)
    (_hσ : ∀ i, (M i).statistic = σ) : MatterField jets where
```

lemma pi_V [source]

```
lemma pi_V : (pi M w h σ hσ).V = ∀ i, (M i).V
```

lemma pi_repJet [source]

```
@[simp]
lemma pi_repJet : (pi M w h σ hσ).repJet = repJetPi M
```

lemma pi_repAlgebra [source]

```
@[simp]
lemma pi_repAlgebra : (pi M w h σ hσ).repAlgebra = repAlgebraPi M
```

lemma pi_massWeight [source]

```
@[simp]
lemma pi_massWeight : (pi M w h σ hσ).massWeight = w
```

lemma pi_statistic [source]

```
@[simp]
lemma pi_statistic : (pi M w h σ hσ).statistic = σ
```

lemma pi_repLorentz_apply [source]

The Lorentz action of the direct sum is the family of Lorentz actions, acting index by index.

```
lemma pi_repLorentz_apply (Λ : SL(2,ℂ)) (v : ∀ i, (M i).V) (i : ι) :
    (pi M w h σ hσ).repLorentz Λ v i = (M i).repLorentz Λ (v i)
```

def jetComponentSpacePiEquiv [source]

***The component space of a direct sum of matter fields splits.** A component function of `MatterField.pi M w h σ hσ` is exactly a family, one component function per summand. Both halves split by `JetComponentSpace.fstPiEquiv` and `JetComponentSpace.sndPiEquiv`, and the pair of families is reassembled into a family of pairs index by index.*

```
noncomputable def jetComponentSpacePiEquiv :
    JetComponentSpace (pi M w h σ hσ) ≃ₗ[ℂ] ∀ i, JetComponentSpace (M i)
```

lemma jetComponentSpacePiEquiv_apply [source]

```
lemma jetComponentSpacePiEquiv_apply (x : JetComponentSpace (pi M w h σ hσ)) (i : ι) :
    jetComponentSpacePiEquiv M w h σ hσ x i =
      (JetComponentSpace.fstPiEquiv (fun i => (M i).V) x.1 i,
        JetComponentSpace.sndPiEquiv (fun i => (M i).V) x.2 i)
```

lemma jetComponentSpacePiEquiv_symm_single [source]

***The summand of one species is the pullback along the projection onto it.** A component function of the summand `i`, placed in the family and read back as a component function of the direct sum, is that function precomposed with the projection onto the summand. This is what identifies the splitting with the species inclusions of a direct sum of component spaces.*

```
lemma jetComponentSpacePiEquiv_symm_single (i : ι) (x : JetComponentSpace (M i)) :
    (jetComponentSpacePiEquiv M w h σ hσ).symm (Pi.single i x)
      = JetComponentSpace.comap
        (LinearMap.proj (φ := fun i => (M i).V) i : (pi M w h σ hσ).V →ₗ[ℂ] (M i).V) x
```

2.249 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.Prod

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/Prod.lean on GitHub.

lemma prodMap_add_prodMap [source]

```
private lemma prodMap_add_prodMap {V₁ V₂ : Type} [AddCommGroup V₁] [Module ℂ V₁]
    [AddCommGroup V₂] [Module ℂ V₂] (f₁ f₂ : V₁ →ₗ[ℂ] V₁) (g₁ g₂ : V₂ →ₗ[ℂ] V₂) :
    (f₁ + f₂).prodMap (g₁ + g₂) = f₁.prodMap g₁ + f₂.prodMap g₂
```

lemma prodMap_multiset_sum [source]

```
private lemma prodMap_multiset_sum {ι V₁ V₂ : Type} [AddCommGroup V₁] [Module ℂ V₁]
    [AddCommGroup V₂] [Module ℂ V₂] (S : Multiset ι) (f : ι → V₁ →ₗ[ℂ] V₁)
    (g : ι → V₂ →ₗ[ℂ] V₂) :
    ((S.map f).sum).prodMap ((S.map g).sum) = (S.map fun i => (f i).prodMap (g i)).sum
```

def repJetProd [source]

***The jet gauge action of a direct sum**: the two actions, read through the identification of the jets of `M.V × N.V` with the pair of jets.*

```
noncomputable def repJetProd :
    Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] (M.V × N.V)) where
```

lemma repJetProd_apply [source]

```
lemma repJetProd_apply (U : GJ) (z : SpaceTimeAlgebra ⊗[ℂ] (M.V × N.V)) :
    repJetProd M N U z =
      jetProdEquiv.symm (M.repJet U (jetProdEquiv z).1, N.repJet U (jetProdEquiv z).2)
```

def repAlgebraProd [source]

***The infinitesimal action of a direct sum**: the two actions on the two summands.*

```
noncomputable def repAlgebraProd : 𝔤 →ₗ[ℝ] (M.V × N.V) →ₗ[ℂ] (M.V × N.V) where
```

lemma repAlgebraProd_apply [source]

```
@[simp]
lemma repAlgebraProd_apply (c : 𝔤) :
    repAlgebraProd M N c = (M.repAlgebra c).prodMap (N.repAlgebra c)
```

lemma repCoeff_repJetProd [source]

The base-point Taylor coefficients of the summed jet action are the pair of the coefficients of the summands: `jetOfConstant`, `jetIteratedDeriv` and `jetEval` all act componentwise through `jetProdEquiv`.

```
lemma repCoeff_repJetProd (U : GJ) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    GaugeAlgebraRealization.repCoeff (repJetProd M N) U x =
      (GaugeAlgebraRealization.repCoeff M.repJet U x).prodMap
        (GaugeAlgebraRealization.repCoeff N.repJet U x)
```

lemma isInfinitesimalActionOf_repAlgebraProd [source]

***The componentwise algebra action generates the componentwise jet action**: both laws of `IsInfinitesimalActionOf` are the corresponding laws of the summands, read through `repCoeff_repJetProd`, since `prodMap` is additive in both slots at once and composes componentwise.*

```
lemma isInfinitesimalActionOf_repAlgebraProd :
    jets.IsInfinitesimalActionOf (repAlgebraProd M N) (repJetProd M N) where
```

def prod [source]

***The direct sum of two matter fields** of the same mass weight and statistics, one field valued in `M.V × N.V` with every action acting componentwise and with the weight and statistics of `M`.*

```
noncomputable def prod (_h : M.massWeight = N.massWeight) (_hσ : M.statistic = N.statistic) :
    MatterField jets where
```

lemma prod_V [source]

```
@[simp]
lemma prod_V : (prod M N h hσ).V = (M.V × N.V)
```

lemma prod_repJet [source]

```
@[simp]
lemma prod_repJet : (prod M N h hσ).repJet = repJetProd M N
```

lemma prod_repAlgebra [source]

```
@[simp]
lemma prod_repAlgebra : (prod M N h hσ).repAlgebra = repAlgebraProd M N
```

lemma prod_repLorentz [source]

```
@[simp]
lemma prod_repLorentz : (prod M N h hσ).repLorentz = M.repLorentz.prod N.repLorentz
```

lemma prod_massWeight [source]

```
@[simp]
lemma prod_massWeight : (prod M N h hσ).massWeight = M.massWeight
```

lemma prod_massWeight_right [source]

The shared weight, read off the second summand: this is what the hypothesis of `prod` buys — the direct sum has one weight, and it is the weight of either summand.

```
lemma prod_massWeight_right : (prod M N h hσ).massWeight = N.massWeight
```

lemma prod_statistic [source]

```
@[simp]
lemma prod_statistic : (prod M N h hσ).statistic = M.statistic
```

def prodEquiv [source]

***The component space of a direct sum splits.** The component functions of the direct sum `M.prod N h hσ` are those of `M` together with those of `N`: the dual and the conjugate both distribute over the finite product, and the derivative label is untouched. Only the value spaces enter, so the hypotheses `h` and `hσ` are carried along and not used.*

```
noncomputable def _root_.JetComponentSpace.prodEquiv (M N : MatterField jets)
    (h : M.massWeight = N.massWeight) (hσ : M.statistic = N.statistic) :
    JetComponentSpace (M.prod N h hσ) ≃ₗ[ℂ] JetComponentSpace M × JetComponentSpace N
```

2.250 PhyslibAlpha.ClassicalFieldTheory.GaugeTheory.MatterField.TransformsIn

PhyslibAlpha/ClassicalFieldTheory/GaugeTheory/MatterField/TransformsIn.lean on GitHub.

def TransformsIn [source]

*A component family `F`, valued in `B` and indexed by the complex dual of the representation space `V`, *transforms in* the representation `rep` of the jet gauge group — with*

the ambient action `repGauge` on `B` — when each derivative symbol `[∂_s F^φ]` transforms by the Leibniz convolution of the dual representation coefficients against lower symbols, with no inhomogeneous term — the generalization of `TransformsInAdjoint` from the adjoint representation to an arbitrary one, and the form consumed by `AlgebraRealization`.

```
def TransformsIn (repGauge : Representation ℂ GJ B)
    (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B) : Prop
```

lemma repGauge_zero [source]

A matter gauge tensor transforms at the base point through the dual coefficient of the base-point value of the gauge jet alone: the antidiagonal of the empty multiset has a single term.

```
lemma TransformsIn.repGauge_zero (hF : TransformsIn repGauge rep F) (U : GJ)
    (φ : Module.Dual ℂ V) :
    repGauge U (F 0 φ) = F 0 (repDualCoeff rep U⁻¹ 0 φ)
```

lemma repGauge_eq_of_eval_eq_one [source]

Matter gauge tensors whose zeroth representation coefficient is trivial on pure jets are fixed by pure jets: for a family transforming in `rep`, a gauge jet with trivial base-point value acts trivially on the underived symbol, provided the representation's zeroth Taylor coefficient is the identity on such jets.

```
lemma TransformsIn.repGauge_eq_of_eval_eq_one {𝔤 : Type} [LieRing 𝔤] [LieAlgebra ℝ 𝔤]
    {G₀ : Type} [Group G₀] {𝔤J : Type} [LieRing 𝔤J] [LieAlgebra ℝ 𝔤J]
    {jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J} (hF : TransformsIn repGauge rep F)
    (hrep : ∀ {W : GJ}, jets.eval W = 1 → repCoeff rep W 0 = LinearMap.id)
    {U : GJ} (hU : jets.eval U = 1) (φ : Module.Dual ℂ V) :
    repGauge U (F 0 φ) = F 0 φ
```

lemma repCoeff_zero_eq_ofConstant_eval [source]

If pure jets act trivially at the base point, the zeroth Taylor coefficient of a jet is that of the constant jet of its value: every jet is a pure jet times the constant jet of its value, and the zeroth coefficients are multiplicative for a fibrewise representation.

```
lemma repCoeff_zero_eq_ofConstant_eval {𝔤 : Type} [LieRing 𝔤] [LieAlgebra ℝ 𝔤]
    {G₀ : Type} [Group G₀] {𝔤J : Type} [LieRing 𝔤J] [LieAlgebra ℝ 𝔤J]
    (jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J) (rep : Representation ℂ GJ (SpaceTimeAlgebra ⊗[ℂ] V))
    (hlin : ∀ (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V), rep U
        (χ • z) = χ • rep U z)
    (hrep : ∀ {W : GJ}, jets.eval W = 1 → repCoeff rep W 0 = LinearMap.id) (U : GJ) :
    repCoeff rep U 0 = repCoeff rep (jets.ofConstant (jets.eval U)) 0
```

lemma repGauge_zero_eq_ofConstant_eval [source]

If pure jets act trivially at the base point, a matter gauge tensor transforms at the base point as under the constant jet of the value of the gauge jet: the jet action on underived symbols factors through evaluation.

```
lemma TransformsIn.repGauge_zero_eq_ofConstant_eval {𝓖 : Type} [LieRing 𝓖] [LieAlgebra ℝ 𝓖]
    {G₀ : Type} [Group G₀] {𝓖J : Type} [LieRing 𝓖J] [LieAlgebra ℝ 𝓖J]
    {jets : LocalGaugeData G₀ 𝓖 GJ 𝓖J} (hF : TransformsIn repGauge rep F)
    (hlin : ∀ (U : GJ) (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] V), rep U
        (χ • z) = χ • rep U z)
    (hrep : ∀ {W : GJ}, jets.eval W = 1 → repCoeff rep W 0 = LinearMap.id) (U : GJ)
    (φ : Module.Dual ℂ V) :
    repGauge U (F 0 φ) = repGauge (jets.ofConstant (jets.eval U)) (F 0 φ)
```

lemma repGauge_eq_of_mem_truncationKer_zero [source]

Matter gauge tensors are fixed by pure jets: the members of the zeroth truncation kernel are the jets with trivial base-point value, so `repGauge_eq_of_eval_eq_one` applies.

```
lemma TransformsIn.repGauge_eq_of_mem_truncationKer_zero {𝓖 : Type} [LieRing 𝓖] [LieAlgebra ℝ
    𝓖]
    {G₀ : Type} [Group G₀] {𝓖J : Type} [LieRing 𝓖J] [LieAlgebra ℝ 𝓖J]
    {jets : LocalGaugeData G₀ 𝓖 GJ 𝓖J} (hF : TransformsIn repGauge rep F)
    (hrep : ∀ {W : GJ}, jets.eval W = 1 → repCoeff rep W 0 = LinearMap.id)
    (U : jets.truncationKer 0) (φ : Module.Dual ℂ V) :
    repGauge U.1 (F 0 φ) = F 0 φ
```

2.251 PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Criterion

PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Criterion.lean on GitHub.

lemma «lemma@102e1711» [source]

```
private lemma
    isCritical_iff_eulerLagrange_zero_of_contDiff_and_coordinateRegularity_of_hasFiniteAction
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hcoord : Lagrangian.ContDiffCoordDerivInCoordinates L)
    (hbase : HasFiniteAction L f)
    (hlocal :
      ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
        HasCompactlySupportedActionVariationDifference L f η) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

2.252 PhyslibAlpha.ClassicalMechanics.CentralForce.AngularMomentum

PhyslibAlpha/ClassicalMechanics/CentralForce/AngularMomentum.lean on GitHub.

def angularMomentum [source]

The angular momentum `L = q × p` of a particle at position `q` with momentum `p`.

```
noncomputable def angularMomentum (p q : EuclideanSpace ℝ (Fin 3)) : EuclideanSpace ℝ (Fin 3)
```

lemma angularMomentum_apply_zero [source]

The x-component of the angular momentum is `q₁ p₂ - q₂ p₁`.

```
lemma angularMomentum_apply_zero (p q : EuclideanSpace ℝ (Fin 3)) :
    angularMomentum p q 0 = q 1 * p 2 - q 2 * p 1
```

lemma angularMomentum_apply_one [source]

The y-component of the angular momentum is $q_2\ p_0 - q_0\ p_2$.

```
lemma angularMomentum_apply_one (p q : EuclideanSpace ℝ (Fin 3)) :
    angularMomentum p q 1 = q 2 * p 0 - q 0 * p 2
```

lemma angularMomentum_apply_two [source]

The z-component of the angular momentum is $q_0\ p_1 - q_1\ p_0$.

```
lemma angularMomentum_apply_two (p q : EuclideanSpace ℝ (Fin 3)) :
    angularMomentum p q 2 = q 0 * p 1 - q 1 * p 0
```

def angularMomentumSq [source]

The squared magnitude $L^2 = \|q \times p\|^2$ of the angular momentum, as a function on phase space.

```
noncomputable def angularMomentumSq (p q : EuclideanSpace ℝ (Fin 3)) : ℝ
```

def angularMomentumZ [source]

The component L_z of the angular momentum along the z-axis, as a function on phase space.

```
noncomputable def angularMomentumZ (p q : EuclideanSpace ℝ (Fin 3)) : ℝ
```

lemma angularMomentumSq_eq_sum [source]

The squared magnitude of the angular momentum is the sum of the squares of its components.

```
lemma angularMomentumSq_eq_sum (p q : EuclideanSpace ℝ (Fin 3)) :
    angularMomentumSq p q =
      angularMomentum p q 0 ^ 2 + angularMomentum p q 1 ^ 2 + angularMomentum p q 2 ^ 2
```

lemma angularMomentumSq_eq [source]

Lagrange's identity: the squared magnitude of the angular momentum $q \times p$ is $\langle\langle q, q\rangle\rangle\ \langle\langle p, p\rangle\rangle - \langle\langle q, p\rangle\rangle^2$.

```
lemma angularMomentumSq_eq (p q : EuclideanSpace ℝ (Fin 3)) :
    angularMomentumSq p q = ⟪q, q⟫_ℝ * ⟪p, p⟫_ℝ - ⟪q, p⟫_ℝ ^ 2
```

lemma contDiff_angularMomentumSq [source]

The squared magnitude of the angular momentum is a smooth function on phase space.

```
lemma contDiff_angularMomentumSq : ContDiff ℝ ∞ (Function.uncurry angularMomentumSq)
```

lemma contDiff_angularMomentumZ [source]

The z-component of the angular momentum is a smooth function on phase space.

```
lemma contDiff_angularMomentumZ : ContDiff ℝ ∞ (Function.uncurry angularMomentumZ)
```

lemma hasGradientAt_angularMomentumSq_momentum [source]

The momentum gradient of L^2 is $2 \langle\langle q, q\rangle\rangle p - 2 \langle\langle q, p\rangle\rangle q$.

```
lemma hasGradientAt_angularMomentumSq_momentum (p q : EuclideanSpace ℝ (Fin 3)) :
    HasGradientAt (fun p' => angularMomentumSq p' q)
      ((2 * ⟪q, q⟫_ℝ) • p - (2 * ⟪q, p⟫_ℝ) • q) p
```

lemma hasGradientAt_angularMomentumSq_position [source]

The position gradient of L^2 is $2 \langle\langle p, p\rangle\rangle q - 2 \langle\langle q, p\rangle\rangle p$.

```
lemma hasGradientAt_angularMomentumSq_position (p q : EuclideanSpace ℝ (Fin 3)) :
    HasGradientAt (fun q' => angularMomentumSq p q')
      ((2 * ⟪p, p⟫_ℝ) • q - (2 * ⟪q, p⟫_ℝ) • p) q
```

lemma hasGradientAt_angularMomentumZ_momentum [source]

The momentum gradient of L_z is $q_0 e_1 - q_1 e_0$.

```
lemma hasGradientAt_angularMomentumZ_momentum (p q : EuclideanSpace ℝ (Fin 3)) :
    HasGradientAt (fun p' => angularMomentumZ p' q)
      (q 0 • EuclideanSpace.single 1 1 - q 1 • EuclideanSpace.single 0 1) p
```

lemma hasGradientAt_angularMomentumZ_position [source]

The position gradient of L_z is $p_1 e_0 - p_0 e_1$.

```
lemma hasGradientAt_angularMomentumZ_position (p q : EuclideanSpace ℝ (Fin 3)) :
    HasGradientAt (fun q' => angularMomentumZ p q')
      (p 1 • EuclideanSpace.single 0 1 - p 0 • EuclideanSpace.single 1 1) q
```

lemma poissonBracket_angularMomentumSq_angularMomentumZ [source]

The squared magnitude of the angular momentum and its z-component are in involution.

```
lemma poissonBracket_angularMomentumSq_angularMomentumZ (p q : EuclideanSpace ℝ (Fin 3)) :
    poissonBracket angularMomentumSq angularMomentumZ p q = 0
```

lemma poissonBracket_angularMomentumSq_eq_zero_of_hasGradientAt [source]

L^2 Poisson-commutes with any phase-space function whose momentum gradient is parallel to the momentum and whose position gradient is parallel to the position.

```
lemma poissonBracket_angularMomentumSq_eq_zero_of_hasGradientAt
    {H : EuclideanSpace ℝ (Fin 3) → EuclideanSpace ℝ (Fin 3) → ℝ}
    {p q : EuclideanSpace ℝ (Fin 3)} {a c : ℝ}
    (hp : HasGradientAt (fun p' => H p' q) (a • p) p)
    (hq : HasGradientAt (fun q' => H p q') (c • q) q) :
```

```
    poissonBracket angularMomentumSq H p q = 0
```

lemma poissonBracket_angularMomentumZ_eq_zero_of_hasGradientAt [source]

`L_z` Poisson-commutes with any phase-space function whose momentum gradient is parallel to the momentum and whose position gradient is parallel to the position.

```
lemma poissonBracket_angularMomentumZ_eq_zero_of_hasGradientAt
    {H : EuclideanSpace ℝ (Fin 3) → EuclideanSpace ℝ (Fin 3) → ℝ}
    {p q : EuclideanSpace ℝ (Fin 3)} {a c : ℝ}
    (hp : HasGradientAt (fun p' => H p' q) (a • p) p)
    (hq : HasGradientAt (fun q' => H p q') (c • q) q) :
    poissonBracket angularMomentumZ H p q = 0
```

lemma poissonBracket_angularMomentumSq_hamiltonian [source]

Away from the centre of force, the squared magnitude of the angular momentum Poisson-commutes with the Hamiltonian of a central-force system, so is conserved by the motion.

```
lemma poissonBracket_angularMomentumSq_hamiltonian (p : EuclideanSpace ℝ (Fin 3))
    {q : EuclideanSpace ℝ (Fin 3)} (hq : q ≠ 0) :
    poissonBracket angularMomentumSq S.hamiltonian p q = 0
```

lemma poissonBracket_angularMomentumZ_hamiltonian [source]

Away from the centre of force, the z-component of the angular momentum Poisson-commutes with the Hamiltonian of a central-force system, so is conserved by the motion.

```
lemma poissonBracket_angularMomentumZ_hamiltonian (p : EuclideanSpace ℝ (Fin 3))
    {q : EuclideanSpace ℝ (Fin 3)} (hq : q ≠ 0) :
    poissonBracket angularMomentumZ S.hamiltonian p q = 0
```

2.253 PhyslibAlpha.ClassicalMechanics.CentralForce.Basic

PhyslibAlpha/ClassicalMechanics/CentralForce/Basic.lean on GitHub.

structure CentralForce [source]

A particle in three-dimensional space under a conservative central force centred at the origin, specified by its mass `m` and a radial potential `V` smooth at positive radii.

```
structure CentralForce where
  m : ℝ
  V : ℝ → ℝ
  m_pos : 0 < m
  contDiffOn_V : ContDiffOn ℝ ∞ V (Set.Ioi 0)
```

lemma m_ne_zero [source]

The mass of the particle is non-zero.

```
@[simp]
lemma m_ne_zero : S.m ≠ 0
```

def hamiltonian [source]

The Hamiltonian `H(p, q) = ⟪p, p⟫ / (2 m) + V ‖q‖` of a central-force system, as a function of the momentum `p` and the position `q`.

```
noncomputable def hamiltonian (p q : EuclideanSpace ℝ (Fin 3)) : ℝ
```

lemma hasGradientAt_hamiltonian_momentum [source]

The momentum gradient of the Hamiltonian of a central-force system is the velocity `p / m`.

```
lemma hasGradientAt_hamiltonian_momentum (p q : EuclideanSpace ℝ (Fin 3)) :
    HasGradientAt (fun p' => S.hamiltonian p' q) (S.m⁻¹ • p) p
```

lemma hasGradientAt_hamiltonian_position [source]

Away from the centre of force, the position gradient of the Hamiltonian of a central-force system is `V'(‖q‖) q / ‖q‖`, which is parallel to the position.

```
lemma hasGradientAt_hamiltonian_position (p : EuclideanSpace ℝ (Fin 3))
    {q : EuclideanSpace ℝ (Fin 3)} (hq : q ≠ 0) :
    HasGradientAt (fun q' => S.hamiltonian p q') ((deriv S.V ‖q‖ / ‖q‖) • q) q
```

lemma contDiffOn_hamiltonian [source]

The Hamiltonian of a central-force system is smooth away from the centre of force.

```
lemma contDiffOn_hamiltonian :
    ContDiffOn ℝ ∞ (Function.uncurry S.hamiltonian) {x | x.2 ≠ 0}
```

2.254 PhyslibAlpha.ClassicalMechanics.LiouvilleIntegrable.Basic

PhyslibAlpha/ClassicalMechanics/LiouvilleIntegrable/Basic.lean on GitHub.

structure LiouvilleIntegrableOn [source]

A Liouville integrable system for a Hamiltonian `H` on an open dense domain `U` of the `2d`-dimensional phase space `EuclideanSpace ℝ (Fin d) × EuclideanSpace ℝ (Fin d)`, whose points are read as `(p, q)`: `d` smooth conserved quantities, one per degree of freedom, that are pairwise in involution and functionally independent at every point of `U`.

```
structure LiouvilleIntegrableOn {d : ℕ}
    (U : Set (EuclideanSpace ℝ (Fin d) × EuclideanSpace ℝ (Fin d)))
    (H : EuclideanSpace ℝ (Fin d) → EuclideanSpace ℝ (Fin d) → ℝ) where
  isOpen : IsOpen U
  dense : Dense U
  f : Fin d → EuclideanSpace ℝ (Fin d) → EuclideanSpace ℝ (Fin d) → ℝ
  contDiffOn : ∀ i, ContDiffOn ℝ ∞ (Function.uncurry (f i)) U
  exists_hamiltonian : ∃ i, f i = H
  involutive : ∀ i j, ∀ x ∈ U, poissonBracket (f i) (f j) x.1 x.2 = 0
  independent : ∀ x ∈ U, LinearIndependent ℝ
    (fun i => (gradient (fun p' => f i p' x.2) x.1, gradient (fun q' => f i x.1 q') x.2))
```

lemma deriv_comp_eq_zero [source]

Each conserved quantity of a Liouville integrable system has zero time derivative along a solution of Hamilton's equations for `H` which lies in the domain `U` at time `t`.

```
lemma LiouvilleIntegrableOn.deriv_comp_eq_zero {d : ℕ}
    {U : Set (EuclideanSpace ℝ (Fin d) × EuclideanSpace ℝ (Fin d))}
    {H : EuclideanSpace ℝ (Fin d) → EuclideanSpace ℝ (Fin d) → ℝ}
    (LI : LiouvilleIntegrableOn U H)
    (p q : Time → EuclideanSpace ℝ (Fin d)) (t : Time) (i : Fin d)
    (heq : hamiltonEqOp (fun _ => H) p q = 0)
    (hp : DifferentiableAt ℝ p t) (hq : DifferentiableAt ℝ q t) (hU : (p t, q t) ∈ U) :
    ∂ₜ (fun t => LI.f i (p t) (q t)) t = 0
```

def oneDegreeOfFreedom [source]

A Hamiltonian system with one degree of freedom is Liouville integrable, with the Hamiltonian as its single conserved quantity, on any open dense domain on which the Hamiltonian is smooth and has non-vanishing phase-space gradient.

```
def LiouvilleIntegrableOn.oneDegreeOfFreedom
    {U : Set (EuclideanSpace ℝ (Fin 1) × EuclideanSpace ℝ (Fin 1))}
    {H : EuclideanSpace ℝ (Fin 1) → EuclideanSpace ℝ (Fin 1) → ℝ}
    (hU : IsOpen U) (hdense : Dense U) (hH : ContDiffOn ℝ ∞ (Function.uncurry H) U)
    (hgrad : ∀ x ∈ U,
      (gradient (fun p' => H p' x.2) x.1, gradient (fun q' => H x.1 q') x.2) ≠ 0) :
    LiouvilleIntegrableOn U H where
```

2.255 PhyslibAlpha.CondensedMatter.Bloch.Algebraic

PhyslibAlpha/CondensedMatter/Bloch/Algebraic.lean on GitHub.

def translation [source]

Translation of finitely supported configurations: `(T a ψ) m = ψ (m - a)`.

```
noncomputable def translation : Multiplicative Λ →* Module.End K (Λ →₀ M) where
```

lemma translation_single [source]

```
lemma translation_single (a : Multiplicative Λ) (m : Λ) (v : M) :
    translation (K := K) a (Finsupp.single m v)
      = Finsupp.single (m + Multiplicative.toAdd a) v
```

def charFun [source]

A character `χ` of `Λ`, viewed as a function on the multiplicative group of translations.

```
def charFun (χ : AddChar Λ K) : Multiplicative Λ → K
```

def ev [source]

Fourier evaluation at the character `χ`: `ev χ ψ = ∑ m, χ m • ψ m`.

```
noncomputable def ev (χ : AddChar Λ K) : (Λ →₀ M) →ₗ[K] M
```

lemma ev_translation [source]

```
lemma ev_translation (χ : AddChar Λ K) (a : Multiplicative Λ) (ψ : Λ →₀ M) :
    ev χ (translation (K := K) a ψ) = charFun χ a • ev χ ψ
```

lemma ev_single_zero [source]

```
lemma ev_single_zero (χ : AddChar Λ K) (v : M) : ev χ (Finsupp.single 0 v) = v
```

lemma ev_surjective [source]

```
lemma ev_surjective (χ : AddChar Λ K) : Function.Surjective (ev (M := M) χ)
```

lemma sub_single_ev_mem_cobound [source]

Every configuration is congruent, modulo the fiber relations, to its Fourier evaluation placed in the origin cell.

```
lemma sub_single_ev_mem_cobound (χ : AddChar Λ K) (ψ : Λ →₀ M) :
    ψ - Finsupp.single 0 (ev χ ψ)
      ∈ cobound (translation (K := K) (Λ := Λ) (M := M)) (charFun χ)
```

lemma ker_ev [source]

The kernel of Fourier evaluation is exactly the span defining the momentum fiber.

```
lemma ker_ev (χ : AddChar Λ K) :
    LinearMap.ker (ev (M := M) χ)
      = cobound (translation (K := K) (Λ := Λ) (M := M)) (charFun χ)
```

def fiberEquiv [source]

The momentum fiber of the translation action at `χ` is canonically the unit-cell space `M`.

```
noncomputable def fiberEquiv (χ : AddChar Λ K) :
    Fiber (translation (K := K) (Λ := Λ) (M := M)) (charFun χ) ≃ₗ[K] M
```

def symbol [source]

The Bloch matrix of a linear map `D` at the character `χ`: apply `D` to the unit cell at the origin and Fourier-evaluate.

```
noncomputable def symbol (χ : AddChar Λ K) (D : (Λ →₀ M) →ₗ[K] (Λ →₀ M)) :
    M →ₗ[K] M
```

lemma symbol_apply [source]

```
lemma symbol_apply (χ : AddChar Λ K) (D : (Λ →₀ M) →ₗ[K] (Λ →₀ M)) (v : M) :
    symbol χ D v = ev χ (D (Finsupp.single 0 v))
```

lemma ev_apply [source]

***Bloch theorem.** Fourier evaluation intertwines a translation-equivariant linear map with its Bloch matrix.*

```
lemma ev_apply {D : (Λ →₀ M) →ₗ[K] (Λ →₀ M)}
    (hD : IsEquivariant (translation (K := K)) (translation (K := K)) D)
    (χ : AddChar Λ K) (ψ : Λ →₀ M) : ev χ (D ψ) = symbol χ D (ev χ ψ)
```

lemma symbol_comp [source]

The Bloch matrix is multiplicative: only the left factor needs to be equivariant.

```
lemma symbol_comp {D₁ D₂ : (Λ →₀ M) →ₗ[K] (Λ →₀ M)}
    (h₁ : IsEquivariant (translation (K := K)) (translation (K := K)) D₁)
    (χ : AddChar Λ K) : symbol χ (D₁ ∘ₗ D₂) = symbol χ D₁ ∘ₗ symbol χ D₂
```

lemma symbol_add [source]

The Bloch matrix is additive.

```
lemma symbol_add (χ : AddChar Λ K) (D₁ D₂ : (Λ →₀ M) →ₗ[K] (Λ →₀ M)) :
    symbol χ (D₁ + D₂) = symbol χ D₁ + symbol χ D₂
```

lemma isUnit_symbol [source]

If an equivariant map is invertible by an equivariant map, its Bloch matrix is invertible a every momentum.

```
lemma isUnit_symbol {D E : (Λ →₀ M) →ₗ[K] (Λ →₀ M)}
    (hD : IsEquivariant (translation (K := K)) (translation (K := K)) D)
    (hE : IsEquivariant (translation (K := K)) (translation (K := K)) E)
    (h₁ : D ∘ₗ E = LinearMap.id) (h₂ : E ∘ₗ D = LinearMap.id) (χ : AddChar Λ K) :
    IsUnit (symbol χ D)
```

def hop [source]

One hopping term: apply `Hn` in each cell, then translate by `n`.

```
noncomputable def hop (n : Λ) (Hn : M →ₗ[K] M) : (Λ →₀ M) →ₗ[K] (Λ →₀ M)
```

def hopping [source]

A finite-range hopping operator with hopping matrices `H n : M → M`: `(hopping H ψ) m = ∑ n, H n (ψ (m - n))`.

```
noncomputable def hopping (H : Λ →₀ (M →ₗ[K] M)) : (Λ →₀ M) →ₗ[K] (Λ →₀ M)
```

lemma isEquivariant_hop [source]

```
lemma isEquivariant_hop (n : Λ) (Hn : M →ₗ[K] M) :
    IsEquivariant (translation (K := K)) (translation (K := K)) (hop n Hn)
```

lemma isEquivariant_hopping [source]

```
lemma isEquivariant_hopping (H : Λ →₀ (M →ₗ[K] M)) :
    IsEquivariant (translation (K := K)) (translation (K := K)) (hopping H)
```

lemma symbol_hopping [source]

***Bloch Hamiltonian.** The Bloch matrix of a hopping operator is `∑ n, χ n • H n`.*

```
lemma symbol_hopping (χ : AddChar Λ K) (H : Λ →₀ (M →ₗ[K] M)) :
    symbol χ (hopping H) = H.sum fun n Hn => χ n • Hn
```

2.256 PhyslibAlpha.CondensedMatter.Bloch.FiniteBloch

PhyslibAlpha/CondensedMatter/Bloch/FiniteBloch.lean on GitHub.

def hopping [source]

The hopping operator `(B ψ) x = ∑_r A_r ψ (x - r)` on configurations of a finite periodic crystal.

```
def hopping (A : G → Matrix (Fin N) (Fin N) ℂ) :
    (G → Fin N → ℂ) →ₗ[ℂ] (G → Fin N → ℂ) where
```

lemma hopping_apply [source]

```
lemma hopping_apply (A : G → Matrix (Fin N) (Fin N) ℂ) (ψ : G → Fin N → ℂ) (x : G) :
    hopping A ψ x = ∑ r, (A r).mulVec (ψ (x - r))
```

def bloch [source]

The Bloch matrix `b(χ) = ∑_r χ(-r) A_r` of a hopping operator.

```
def bloch (A : G → Matrix (Fin N) (Fin N) ℂ) (χ : AddChar G ℂ) : Matrix (Fin N) (Fin N) ℂ
```

def fourier [source]

The Fourier transform `ψ̂(χ) = ∑_x χ(-x) ψ(x)` of a configuration.

```
def fourier (χ : AddChar G ℂ) (ψ : G → Fin N → ℂ) : Fin N → ℂ
```

lemma fourier_hopping [source]

***Bloch's theorem on a finite crystal.** The Fourier transform of `B ψ` at momentum `χ` is the Bloch matrix applied to the Fourier transform of `ψ`.*

```
lemma fourier_hopping (A : G → Matrix (Fin N) (Fin N) ℂ) (χ : AddChar G ℂ)
    (ψ : G → Fin N → ℂ) : fourier χ (hopping A ψ) = (bloch A χ).mulVec (fourier χ ψ)
```

lemma mem_spectrum_iff_exists [source]

An eigenvalue of an operator on a finite-dimensional space is a point of its spectrum.

```
lemma mem_spectrum_iff_exists {V : Type*} [AddCommGroup V] [Module ℂ V] [FiniteDimensional ℂ V]
    (f : Module.End ℂ V) (μ : ℂ) : μ ∈ spectrum ℂ f ↔ ∃ v, v ≠ 0 ∧ f v = μ • v
```

lemma mem_spectrum_matrix_iff [source]

A complex number is in the spectrum of a matrix exactly when it has an eigenvector.

```
lemma mem_spectrum_matrix_iff (M : Matrix (Fin N) (Fin N) ℂ) (μ : ℂ) :
    μ ∈ spectrum ℂ M ↔ ∃ v : Fin N → ℂ, v ≠ 0 ∧ M.mulVec v = μ • v
```

lemma fourier_smul [source]

```
lemma fourier_smul (χ : AddChar G ℂ) (c : ℂ) (ψ : G → Fin N → ℂ) :
    fourier χ (c • ψ) = c • fourier χ ψ
```

lemma sum_character_smul_fourier [source]

***Fourier inversion** on a finite abelian group.*

```
lemma sum_character_smul_fourier (ψ : G → Fin N → ℂ) (x : G) :
    ∑ χ : AddChar G ℂ, χ x • fourier χ ψ = (Fintype.card G : ℂ) • ψ x
```

lemma eq_zero_of_fourier_eq_zero [source]

A configuration with vanishing Fourier transform vanishes.

```
lemma eq_zero_of_fourier_eq_zero {ψ : G → Fin N → ℂ} (h : ∀ χ, fourier χ ψ = 0) : ψ = 0
```

lemma hopping_planeWave [source]

A plane wave of crystal momentum χ is mapped to a plane wave with the same momentum.

```
lemma hopping_planeWave (A : G → Matrix (Fin N) (Fin N) ℂ) (χ : AddChar G ℂ)
    (v : Fin N → ℂ) :
    hopping A (fun x => χ x • v) = fun x => χ x • (bloch A χ).mulVec v
```

lemma spectrum_hopping [source]

The spectrum of a translation-invariant operator on a finite periodic crystal is the union of the spectra of its Bloch matrices.

```
lemma spectrum_hopping (A : G → Matrix (Fin N) (Fin N) ℂ) :
    spectrum ℂ (hopping A) = ⋃ χ : AddChar G ℂ, spectrum ℂ (bloch A χ)
```

2.257 PhyslibAlpha.CondensedMatter.Bloch.Hypercubic

PhyslibAlpha/CondensedMatter/Bloch/Hypercubic.lean on GitHub.

def hypercubicCone [source]

The cone `0 ≤ k_{d-1} ≤ … ≤ k_0 ≤ 1/2` of coordinates.

```
def hypercubicCone (d : ℕ) : Set (Fin d → ℝ)
```

def hypercubicZone [source]

The irreducible zone of the hypercubic lattice: the momenta with coordinates in the cone.

```
def hypercubicZone (d : ℕ) : Set (BrillouinZone (Fin d → ℤ))
```

lemma signedPerm_mem_symmetries [source]

A signed permutation is a bond-preserving symmetry of the hypercubic lattice.

```
lemma signedPerm_mem_symmetries (π : Equiv.Perm (Fin d)) (ε : Fin d → Bool) :
    (⟨signedPerm π ε, 1, 0⟩ : CellSymmetry (Fin d → ℤ) Unit) ∈
      (hypercubicGraph d).symmetries
```

lemma smul_brillouinPoint_of_linear [source]

The coordinates of the image of a momentum under a symmetry whose linear part is a signed permutation.

```
lemma smul_brillouinPoint_of_linear {F : Type*} (s : CellSymmetry (Fin d → ℤ) F)
    (π : Equiv.Perm (Fin d)) (ε : Fin d → Bool) (h : s.linear = signedPerm π ε)
    (k : Fin d → ℝ) :
    s • brillouinPoint k = brillouinPoint (fun j => (signOf (ε j) : ℝ) * k (π.symm j))
```

lemma smul_brillouinPoint_signedPerm [source]

The coordinates of the image of a momentum under a signed permutation.

```
lemma smul_brillouinPoint_signedPerm (π : Equiv.Perm (Fin d)) (ε : Fin d → Bool) (k : Fin d →
    ℝ) :
    (⟨signedPerm π ε, 1, 0⟩ : CellSymmetry (Fin d → ℤ) Unit) • brillouinPoint k =
      brillouinPoint (fun j => (signOf (ε j) : ℝ) * k (π.symm j))
```

lemma exists_signedPerm_to_cone [source]

Every momentum is carried into the irreducible zone by a signed permutation.

```
lemma exists_signedPerm_to_cone (k : Fin d → ℝ) :
    ∃ (π : Equiv.Perm (Fin d)) (ε : Fin d → Bool) (k' : Fin d → ℝ), k' ∈ hypercubicCone d ∧
      (⟨signedPerm π ε, 1, 0⟩ : CellSymmetry (Fin d → ℤ) Unit) • brillouinPoint k =
        brillouinPoint k'
```

lemma coversOrbits_hypercubic [source]

The hypercubic zone covers the orbits of the point group.

```
lemma coversOrbits_hypercubic :
    CoversOrbits (hypercubicGraph d) (hypercubicZone d)
```

def hypercubicBand [source]

The band of nearest-neighbour hopping with amplitude `t` on the hypercubic lattice, at the momentum with coordinates `k`: `2 t ∑_j cos (2π k_j)`.

```
noncomputable def hypercubicBand (t : ℝ) (k : Fin d → ℝ) : ℝ
```

lemma brillouinPoint_single_sign [source]

```
private lemma brillouinPoint_single_sign (k : Fin d → ℝ) (j : Fin d) (b : ℤ) :
    brillouinPoint k (Multiplicative.ofAdd (Pi.single j b : Fin d → ℤ)) =
      Circle.exp (2 * Real.pi * (k j * b))
```

lemma circle_exp_add_neg [source]

```
private lemma circle_exp_add_neg (x : ℝ) :
    ((Circle.exp x : Circle) : ℂ) + ((Circle.exp (-x) : Circle) : ℂ) =
      ((2 * Real.cos x : ℝ) : ℂ)
```

lemma hoppingSymbol_hypercubic [source]

***The Bloch matrix of nearest-neighbour hopping on the hypercubic lattice** is the `1 × 1` matrix with the entry `2 t ∑_j cos (2π k_j)`.*

```
lemma hoppingSymbol_hypercubic (t : ℝ) (k : Fin d → ℝ) :
    hoppingSymbol (uniformHopping (hypercubicGraph d) (t : ℂ)) (brillouinPoint k) =
      Matrix.of fun _ _ => ((hypercubicBand t k : ℝ) : ℂ)
```

lemma spectrum_const [source]

```
private lemma spectrum_const (c : ℂ) :
    spectrum ℂ (Matrix.of fun (_ : Unit) (_ : Unit) => c) = {c}
```

lemma iUnion_spectrum_hypercubic [source]

***The spectrum of nearest-neighbour hopping on the hypercubic lattice** is the set of values of the band `2 t ∑_j cos (2π k_j)` on the irreducible zone `0 ≤ k_{d-1} ≤ … ≤ k_0 ≤ 1/2`, which is `1 / (2^d d!)` of the Brillouin zone.*

```
lemma iUnion_spectrum_hypercubic (t : ℝ) :
    (⋃ χ : BrillouinZone (Fin d → ℤ),
       spectrum ℂ (hoppingSymbol (uniformHopping (hypercubicGraph d) (t : ℂ)) χ)) =
      {z : ℂ | ∃ k ∈ hypercubicCone d, z = ((hypercubicBand t k : ℝ) : ℂ)}
```

lemma convex_hypercubicCone [source]

```
lemma convex_hypercubicCone : Convex ℝ (hypercubicCone d)
```

lemma continuous_hypercubicBand [source]

```
lemma continuous_hypercubicBand (t : ℝ) : Continuous (hypercubicBand (d := d) t)
```

lemma abs_hypercubicBand_le [source]

```
lemma abs_hypercubicBand_le (t : ℝ) (k : Fin d → ℝ) :
    |hypercubicBand t k| ≤ 2 * d * |t|
```

lemma hypercubicBand_image [source]

The band of nearest-neighbour hopping takes exactly the values in `[-2 d |t|, 2 d |t|]` on the irreducible zone.

```
lemma hypercubicBand_image (t : ℝ) :
    hypercubicBand t '' hypercubicCone d = Set.Icc (-(2 * d * |t|)) (2 * d * |t|)
```

lemma iUnion_spectrum_hypercubic_eq_Icc [source]

The spectrum of nearest-neighbour hopping on the hypercubic lattice is the interval `[-2 d |t|, 2 d |t|]`.

```
lemma iUnion_spectrum_hypercubic_eq_Icc (t : ℝ) :
    (⋃ χ : BrillouinZone (Fin d → ℤ),
        spectrum ℂ (hoppingSymbol (uniformHopping (hypercubicGraph d) (t : ℂ)) χ)) =
      ((↑) : ℝ → ℂ) '' Set.Icc (-(2 * d * |t|)) (2 * d * |t|)
```

lemma littleGroup_gamma [source]

The centre `Γ` of the Brillouin zone is fixed by every symmetry.

```
lemma littleGroup_gamma (d : ℕ) :
    (hypercubicGraph d).littleGroup (brillouinPoint (0 : Fin d → ℝ)) = ⊤
```

lemma littleGroup_corner [source]

***The corner `R = (1/2, …, 1/2)` of the Brillouin zone is fixed by every symmetry**: its little group is the whole group.*

```
lemma littleGroup_corner (d : ℕ) :
    (hypercubicGraph d).littleGroup (brillouinPoint (fun _ : Fin d => (1 / 2 : ℝ))) = ⊤
```

2.258 PhyslibAlpha.CondensedMatter.Bloch.PointSymmetry

PhyslibAlpha/CondensedMatter/Bloch/PointSymmetry.lean on GitHub.

def fibreMatrix [source]

The fibre matrix of a symmetry `s = (A, σ, b)` at the momentum `χ`: the monomial matrix with the entry `χ(b i)` at `(σ i, i)`.

```
noncomputable def fibreMatrix (s : CellSymmetry G F) (χ : BrillouinZone G) :
    Matrix F F ℂ
```

lemma fibreMatrix_apply [source]

```
lemma fibreMatrix_apply (s : CellSymmetry G F) (χ : BrillouinZone G) (j i : F) :
    fibreMatrix s χ j i = if j = s.perm i then (χ (Multiplicative.ofAdd (s.shift i)) : ℂ) else
    0
```

lemma fibreMatrix_mul [source]

***The cocycle identity** for the fibre matrices: `V_{st}(χ) = V_s(χ) V_t(s⁻¹ • χ)`.*

```
lemma fibreMatrix_mul (s t : CellSymmetry G F) (χ : BrillouinZone G) :
    fibreMatrix (s * t) χ = fibreMatrix s χ * fibreMatrix t (s⁻¹ • χ)
```

lemma circle_mul_star [source]

```
private lemma circle_mul_star (z : Circle) : (z : ℂ) * star (z : ℂ) = 1
```

lemma fibreMatrix_mul_conjTranspose [source]

The fibre matrices are unitary.

```
lemma fibreMatrix_mul_conjTranspose (s : CellSymmetry G F) (χ : BrillouinZone G) :
    fibreMatrix s χ * (fibreMatrix s χ)ᴴ = 1
```

lemma fibreMatrix_conjTranspose_mul [source]

The fibre matrices are unitary.

```
lemma fibreMatrix_conjTranspose_mul (s : CellSymmetry G F) (χ : BrillouinZone G) :
    (fibreMatrix s χ)ᴴ * fibreMatrix s χ = 1
```

def hoppingSymbol [source]

The Bloch matrix `b(χ) = ∑_r χ(r) T r` of a finite family of hopping matrices.

```
noncomputable def hoppingSymbol (T : G →₀ Matrix F F ℂ) (χ : BrillouinZone G) : Matrix F F ℂ
```

def IsInvariant [source]

A family of hopping matrices is invariant under a symmetry `(A, σ, b)` when `T (A r + b i - b j) (σ i) (σ j) = T r i j`: the amplitude to hop from `j` to `i` across `r` equals the amplitude between the transformed sites across the transformed displacement.

```
def IsInvariant (T : G →₀ Matrix F F ℂ) (s : CellSymmetry G F) : Prop
```

def IsCovariant [source]

A symbol `b` on the Brillouin zone is covariant under a symmetry when the fibre matrix intertwines it: `V_s(s • χ) b(χ) = b(s • χ) V_s(s • χ)`.

```
def IsCovariant (b : BrillouinZone G → Matrix F F ℂ) (s : CellSymmetry G F) : Prop
```

def entry [source]

The entries of a hopping family at a fixed pair of orbitals, as a finitely supported function of the displacement.

```
noncomputable def entry (T : G →₀ Matrix F F ℂ) (i j : F) : G →₀ ℂ
```

lemma hoppingSymbol_apply [source]

```
lemma hoppingSymbol_apply (T : G →₀ Matrix F F ℂ) (χ : BrillouinZone G) (i j : F) :
    hoppingSymbol T χ i j = (entry T i j).sum fun r m => (χ (Multiplicative.ofAdd r) : ℂ) * m
```

lemma smul_apply_linear [source]

```
lemma smul_apply_linear (s : CellSymmetry G F) (χ : BrillouinZone G) (r : G) :
    (s • χ) (Multiplicative.ofAdd (s.linear r)) = χ (Multiplicative.ofAdd r)
```

lemma isCovariant_of_invariant [source]

Invariant hopping has a covariant Bloch symbol.

```
lemma isCovariant_of_invariant {T : G →₀ Matrix F F ℂ} {s : CellSymmetry G F}
    (hT : IsInvariant T s) : IsCovariant (hoppingSymbol T) s
```

lemma conj_of_covariant [source]

A covariant symbol is conjugated by the fibre matrix between a momentum and its image.

```
lemma conj_of_covariant {b : BrillouinZone G → Matrix F F ℂ} {s : CellSymmetry G F}
    (hb : IsCovariant b s) (χ : BrillouinZone G) :
    b (s • χ) = fibreMatrix s (s • χ) * b χ * (fibreMatrix s (s • χ))ᴴ
```

lemma charpoly_eq_of_covariant [source]

The Bloch matrices at related momenta have the same characteristic polynomial.

```
lemma charpoly_eq_of_covariant {b : BrillouinZone G → Matrix F F ℂ} {s : CellSymmetry G F}
    (hb : IsCovariant b s) (χ : BrillouinZone G) :
    (b (s • χ)).charpoly = (b χ).charpoly
```

lemma eigenvalues_eq_of_covariant [source]

***The bands agree at momenta related by a symmetry.** If the Bloch matrices are Hermitian, their eigenvalues at χ and $s \bullet \chi$ coincide.*

```
lemma eigenvalues_eq_of_covariant {b : BrillouinZone G → Matrix F F ℂ}
    (hH : ∀ χ, (b χ).IsHermitian) {s : CellSymmetry G F} (hb : IsCovariant b s)
    (χ : BrillouinZone G) : (hH (s • χ)).eigenvalues = (hH χ).eigenvalues
```

def bandsOnZone [source]

***The bands are functions on the irreducible zone.** For a symbol that is covariant under all bond-preserving symmetries of a crystal, the eigenvalues of the Bloch matrices depend only on the orbit of the momentum under the point group.*

```
noncomputable def bandsOnZone {g : PeriodicGraph (Crystal.ofUnitCell G F)}
    {b : BrillouinZone G → Matrix F F ℂ} (hH : ∀ χ, (b χ).IsHermitian)
    (hb : ∀ s : g.symmetries, IsCovariant b s) : g.IrreducibleZone → (F → ℝ)
```

lemma commute_of_smul_eq [source]

At a momentum fixed by a symmetry, the Bloch matrix commutes with its fibre matrix.

```
lemma commute_of_smul_eq {b : BrillouinZone G → Matrix F F ℂ} {s : CellSymmetry G F}
    (hb : IsCovariant b s) {χ : BrillouinZone G} (hχ : s • χ = χ) :
    fibreMatrix s χ * b χ = b χ * fibreMatrix s χ
```

lemma le_finrank_eigenspace [source]

***Symmetry-enforced degeneracy.** If `M` commutes with a set of matrices and every nonzero subspace invariant under them has dimension at least `m`, every eigenspace of `M` has dimension at least `m`.*

```
lemma le_finrank_eigenspace (M : Matrix F F ℂ) (S : Set (Matrix F F ℂ))
    (hcomm : ∀ V ∈ S, V * M = M * V) {m : ℕ}
    (hS : ∀ W : Submodule ℂ (F → ℂ), W ≠ ⊥ → (∀ V ∈ S, ∀ w ∈ W, V.mulVec w ∈ W) →
      m ≤ Module.finrank ℂ W) (μ : ℂ) (hμ : ∃ v : F → ℂ, v ≠ 0 ∧ M.mulVec v = μ • v) :
    m ≤ Module.finrank ℂ (Module.End.eigenspace (Matrix.toLin' M) μ)
```

lemma le_finrank_eigenspace_of_littleGroup [source]

***Degeneracy forced by the little group.** For a symbol covariant under the bond-preserving symmetries of a crystal, if every nonzero subspace of `ℂ^F` invariant under the fibre matrices of the little group at `χ` has dimension at least `m`, every eigenspace of the Bloch matrix at `χ` has dimension at least `m`.*

```
lemma le_finrank_eigenspace_of_littleGroup {g : PeriodicGraph (Crystal.ofUnitCell G F)}
    {b : BrillouinZone G → Matrix F F ℂ} (hb : ∀ s : g.symmetries, IsCovariant b s)
    (χ : BrillouinZone G) {m : ℕ}
    (hS : ∀ W : Submodule ℂ (F → ℂ), W ≠ ⊥ →
      (∀ s ∈ g.littleGroup χ, ∀ w ∈ W, (fibreMatrix (s : CellSymmetry G F) χ).mulVec w ∈ W) →
      m ≤ Module.finrank ℂ W) (μ : ℂ) (hμ : ∃ v : F → ℂ, v ≠ 0 ∧ (b χ).mulVec v = μ • v) :
    m ≤ Module.finrank ℂ (Module.End.eigenspace (Matrix.toLin' (b χ)) μ)
```

def uniformHopping [source]

Hopping with the amplitude `t` between any two bonded sites of a periodic graph: the entry `(i, j)` of the matrix at the displacement `r` is `t` when the site `j` of a cell is bonded to the site `i` of the cell displaced by `r`.

```
noncomputable def uniformHopping (g : PeriodicGraph (Crystal.ofUnitCell G F)) (t : ℂ) :
    G →₀ Matrix F F ℂ
```

lemma uniformHopping_apply [source]

```
lemma uniformHopping_apply (g : PeriodicGraph (Crystal.ofUnitCell G F)) (t : ℂ) (r : G) (i j :
    F) :
    uniformHopping g t r i j = if g.Bonded j i r then t else 0
```

lemma isInvariant_uniformHopping [source]

Uniform hopping is invariant under every bond-preserving symmetry.

```
lemma isInvariant_uniformHopping (g : PeriodicGraph (Crystal.ofUnitCell G F)) (t : ℂ)
    {s : CellSymmetry G F} (hs : s ∈ g.symmetries) : IsInvariant (uniformHopping g t) s
```

lemma isCovariant_uniformHopping [source]

Uniform hopping has a covariant Bloch symbol under every bond-preserving symmetry.

```
lemma isCovariant_uniformHopping (g : PeriodicGraph (Crystal.ofUnitCell G F)) (t : ℂ)
    (s : g.symmetries) : IsCovariant (hoppingSymbol (uniformHopping g t)) s
```

lemma isHermitian_hoppingSymbol [source]

If `(T r)^H = T (-r)` for all `r`, the Bloch matrices are Hermitian.

```
lemma isHermitian_hoppingSymbol {T : G →₀ Matrix F F ℂ} (hT : ∀ r, (T r)ᴴ = T (-r))
    (χ : BrillouinZone G) : (hoppingSymbol T χ).IsHermitian
```

lemma bonded_symm [source]

Bonds are symmetric: the bond from `i` to `j` across `d` is the bond from `j` to `i` across `-d`.

```
lemma bonded_symm (g : PeriodicGraph (Crystal.ofUnitCell G F)) (i j : F) (d : G) :
    g.Bonded i j d ↔ g.Bonded j i (-d)
```

lemma isHermitian_hoppingSymbol_uniform [source]

Uniform real hopping gives Hermitian Bloch matrices at every momentum.

```
lemma isHermitian_hoppingSymbol_uniform (g : PeriodicGraph (Crystal.ofUnitCell G F)) (t : ℝ)
    (χ : BrillouinZone G) : (hoppingSymbol (uniformHopping g (t : ℂ)) χ).IsHermitian
```

def CoversOrbits [source]

A set of momenta covers the orbits of the point group when every momentum is carried into it by some element of the point group. An irreducible zone is a set that covers the orbits.

```
def CoversOrbits (g : PeriodicGraph (Crystal.ofUnitCell G F)) (D : Set (BrillouinZone G)) :
    Prop
```

lemma spectrum_eq_range_eigenvalues [source]

The spectrum of a Bloch matrix is the set of its eigenvalues.

```
lemma spectrum_eq_range_eigenvalues {M : Matrix F F ℂ} (hM : M.IsHermitian) :
    spectrum ℂ M = ((↑) : ℝ → ℂ) '' Set.range hM.eigenvalues
```

lemma spectrum_eq_of_covariant [source]

The spectra of the Bloch matrices at related momenta agree.

```
lemma spectrum_eq_of_covariant {b : BrillouinZone G → Matrix F F ℂ}
    (hH : ∀ χ, (b χ).IsHermitian) {s : CellSymmetry G F} (hb : IsCovariant b s)
    (χ : BrillouinZone G) : spectrum ℂ (b (s • χ)) = spectrum ℂ (b χ)
```

lemma iUnion_spectrum_eq_of_covers [source]

***The spectrum over the whole Brillouin zone is the spectrum over an irreducible zone.** If a symbol is covariant under the bond-preserving symmetries, it suffices to take the union of the spectra of its Bloch matrices over a set of momenta that covers the orbits of the point group.*

```
lemma iUnion_spectrum_eq_of_covers {g : PeriodicGraph (Crystal.ofUnitCell G F)}
    {b : BrillouinZone G → Matrix F F ℂ} (hH : ∀ χ, (b χ).IsHermitian)
    (hb : ∀ s : g.symmetries, IsCovariant b s) {D : Set (BrillouinZone G)}
    (hD : CoversOrbits g D) :
    (⋃ χ : BrillouinZone G, spectrum ℂ (b χ)) = ⋃ χ ∈ D, spectrum ℂ (b χ)
```

lemma two_le_finrank_of_no_common_eigenvector [source]

***A criterion for twofold degeneracy.** If the matrices in `S` have no common eigenvector, every nonzero invariant subspace has dimension at least `2`. Combined with `le_finrank_eigenspace_of_littleGroup`, this forces every eigenvalue of the Bloch matrix at a momentum whose little group has no common eigenvector in `ℂ^F` to be at least twofold degenerate.*

```
lemma two_le_finrank_of_no_common_eigenvector (S : Set (Matrix F F ℂ))
    (h : ¬∃ v : F → ℂ, v ≠ 0 ∧ ∀ V ∈ S, ∃ c : ℂ, V.mulVec v = c • v)
    (W : Submodule ℂ (F → ℂ)) (hW : W ≠ ⊥) (hinv : ∀ V ∈ S, ∀ w ∈ W, V.mulVec w ∈ W) :
    2 ≤ Module.finrank ℂ W
```

2.259 PhyslibAlpha.CondensedMatter.Bloch.RegularRep

PhyslibAlpha/CondensedMatter/Bloch/RegularRep.lean on GitHub.

lemma measurePreserving_addRight [source]

Translation of a discrete group preserves the counting measure.

```
lemma measurePreserving_addRight (g : G) :
    MeasurePreserving (fun x : G => x + g) (Measure.count : Measure G) Measure.count
```

def shift [source]

Translation by `a`, as a permutation of the group.

```
def shift (a : G) : Equiv.Perm G where
```

def translations [source]

The translation action of the group on itself, as permutations.

```
def translations : Multiplicative G →* Equiv.Perm G where
```

lemma measurePreserving_translations [source]

```
lemma measurePreserving_translations (g : Multiplicative G) :
    MeasurePreserving (translations g) (Measure.count : Measure G) Measure.count
```

abbrev Space [source]

The Hilbert space `ℓ²(G, ℂ^N)` of square-summable functions on `G` with values in the internal space `ℂ^N`, as `L²` of the counting measure.

```
abbrev Space
```

def translation [source]

***The translation representation** on `ℓ²(G, ℂ^N)`: `(U_g ψ)(x) = ψ(x - g)`.*

```
def translation : Multiplicative G →* unitary (Space G N →L[ℂ] Space G N)
```

abbrev pointwise [source]

Pointwise action of an operator of the internal space on `ℓ²(G, ℂ^N)`.

```
abbrev pointwise (L : EuclideanSpace ℂ (Fin N) →L[ℂ] EuclideanSpace ℂ (Fin N)) :
    Space G N →L[ℂ] Space G N
```

lemma pointwise_apply_ae [source]

```
lemma pointwise_apply_ae (L : EuclideanSpace ℂ (Fin N) →L[ℂ] EuclideanSpace ℂ (Fin N))
    (f : Space G N) : pointwise G N L f =ᵐ[Measure.count] fun x => L (f x)
```

lemma pointwise_mul [source]

```
lemma pointwise_mul (A B : EuclideanSpace ℂ (Fin N) →L[ℂ] EuclideanSpace ℂ (Fin N)) :
    pointwise G N (A * B) = pointwise G N A * pointwise G N B
```

lemma pointwise_one [source]

```
lemma pointwise_one : pointwise G N (1 : EuclideanSpace ℂ (Fin N) →L[ℂ] EuclideanSpace ℂ (Fin N
    )) =
    1
```

lemma pointwise_star [source]

```
lemma pointwise_star (L : EuclideanSpace ℂ (Fin N) →L[ℂ] EuclideanSpace ℂ (Fin N)) :
    pointwise G N (star L) = star (pointwise G N L)
```

def internal [source]

***The internal algebra**: a matrix acts on the internal index of a wavefunction, pointwise.*

```
def internal : Matrix (Fin N) (Fin N) ℂ →⋆ₐ[ℂ] (Space G N →L[ℂ] Space G N) where
```

lemma translation_apply [source]

```
lemma translation_apply (g : Multiplicative G) (f : Space G N) :
    ((translation G N g : unitary (Space G N →L[ℂ] Space G N)) : Space G N →L[ℂ] Space G N) f =
      Lp.compMeasurePreserving (translations g⁻¹) (measurePreserving_translations g⁻¹) f
```

lemma commute_translation_internal [source]

The translations commute with the internal algebra: the matrices act on the internal index and the translations move the site.

```
lemma commute_translation_internal (g : Multiplicative G) (M : Matrix (Fin N) (Fin N) ℂ) :
    Commute ((translation G N g : unitary (Space G N →L[ℂ] Space G N)) :
      Space G N →L[ℂ] Space G N) (internal M)
```

def localized [source]

The state localized at the origin with internal state `e`.

```
def localized (e : EuclideanSpace ℂ (Fin N)) : Space G N
```

lemma count_preimage_ne_top [source]

```
lemma count_preimage_ne_top (g : Multiplicative G) :
    (Measure.count : Measure G) ((translations g⁻¹) ⁻¹' {(0 : G)}) ≠ ⊤
```

lemma translation_localized [source]

Translating a localized state moves it to the translated site.

```
lemma translation_localized (g : Multiplicative G) (e : EuclideanSpace ℂ (Fin N)) :
    ((translation G N g : unitary (Space G N →L[ℂ] Space G N)) : Space G N →L[ℂ] Space G N)
        (localized e) =
      indicatorConstLp 2 ((measurableSet_singleton (0 : G)).preimage
        (measurable_of_countable (translations g⁻¹))) (count_preimage_ne_top g) e
```

lemma inner_localized_translation [source]

The matrix coefficients of a localized state are the Kronecker delta.

```
lemma inner_localized_translation (g : Multiplicative G) (e e' : EuclideanSpace ℂ (Fin N)) :
    ⟪localized (G := G) e, ((translation G N g : unitary (Space G N →L[ℂ] Space G N)) :
      Space G N →L[ℂ] Space G N) (localized e')⟫_ℂ = if g = 1 then ⟪e, e'⟫_ℂ else 0
```

lemma pointwise_localized [source]

```
lemma pointwise_localized (L : EuclideanSpace ℂ (Fin N) →L[ℂ] EuclideanSpace ℂ (Fin N))
    (e : EuclideanSpace ℂ (Fin N)) : pointwise G N L (localized e) = localized (L e)
```

lemma localized_add [source]

```
lemma localized_add (e e' : EuclideanSpace ℂ (Fin N)) :
    localized (G := G) (e + e') = localized e + localized e'
```

lemma localized_smul [source]

```
lemma localized_smul (c : ℂ) (e : EuclideanSpace ℂ (Fin N)) :
    localized (G := G) (c • e) = c • localized e
```

lemma vectorMeasure_localized [source]

***A localized state has a uniform momentum distribution**: its spectral measure for the translations is the Haar measure of the dual group, weighted by the squared norm of the internal state.*

```
lemma vectorMeasure_localized (e : EuclideanSpace ℂ (Fin N)) :
    UnitaryRep.vectorMeasure (translation G N) (localized e) =
      ENNReal.ofReal (‖e‖ ^ 2) • dualHaar (G := G)
```

lemma projection_localized_ne_zero [source]

The spectral projection of a nonempty open set of crystal momenta does not annihilate a localized state of nonzero internal state.

```
lemma projection_localized_ne_zero {U : Set (PontryaginDual (Multiplicative G))}
    (hU : IsOpen U) (hne : U.Nonempty) {e : EuclideanSpace ℂ (Fin N)} (he : e ≠ 0) :
    UnitaryRep.projection (translation G N) U (localized e) ≠ 0
```

lemma hasFullSupport [source]

***The regular representation has full support**: every crystal momentum occurs in every internal state.*

```
lemma hasFullSupport :
    UnitaryRep.HasFullSupport (translation G N) (internal (G := G) (N := N))
```

2.260 PhyslibAlpha.CondensedMatter.Bloch.SiteRepresentation

PhyslibAlpha/CondensedMatter/Bloch/SiteRepresentation.lean on GitHub.

instance MeasurableSpace (CellSite G F) [source]

Every set of sites is measurable.

```
instance : MeasurableSpace (CellSite G F)
```

instance MeasurableSingletonClass (CellSite G F) [source]

```
instance : MeasurableSingletonClass (CellSite G F)
```

instance Countable (CellSite G F) [source]

```
instance [Countable G] [Countable F] : Countable (CellSite G F)
```

lemma measurePreserving_count [source]

A permutation of a countable set preserves the counting measure.

```
lemma measurePreserving_count {X : Type*} [MeasurableSpace X] [MeasurableSingletonClass X]
    [Countable X] (e : Equiv.Perm X) :
    MeasurePreserving e (Measure.count : Measure X) Measure.count
```

abbrev SiteSpace [source]

The square-summable functions on the sites of a crystal built from a unit cell.

```
abbrev SiteSpace (G F : Type*)
```

def siteTranslationPerm [source]

The translations act on the sites by permutations.

```
def siteTranslationPerm (G F : Type*) [AddCommGroup G] :
    Multiplicative G →* Equiv.Perm (CellSite G F)
```

def siteTranslation [source]

***The translation representation** on ℓ^2 of the sites: $(U_a\ \psi)(x) = \psi(-a + x)$.*

```
def siteTranslation (G F : Type*) [AddCommGroup G] [Countable G] [Countable F] :
    Multiplicative G →* unitary (SiteSpace G F →L[ℂ] SiteSpace G F)
```

def siteSymmetryPerm [source]

The bond-preserving symmetries act on the sites by permutations.

```
def siteSymmetryPerm : g.symmetries →* Equiv.Perm (CellSite G F)
```

def siteSymmetry [source]

***The symmetry representation** on ℓ^2 of the sites: $(U_s\ \psi)(x) = \psi(s^{-1} \bullet x)$.*

```
def siteSymmetry : g.symmetries →* unitary (SiteSpace G F →L[ℂ] SiteSpace G F)
```

def latticeAut [source]

The action of a symmetry on the lattice of translations, through its linear part.

```
def latticeAut : g.symmetries →* MulAut (Multiplicative G) where
```

lemma siteSymmetryPerm_translation [source]

A symmetry moves a translated site to the correspondingly translated image: the permutations of the sites satisfy $s \circ \tau_a = \tau_{A\ a} \circ s$.

```
lemma siteSymmetryPerm_translation (s : g.symmetries) (a : Multiplicative G) :
    siteSymmetryPerm g s * siteTranslationPerm G F a =
      siteTranslationPerm G F (latticeAut g s a) * siteSymmetryPerm g s
```

lemma siteTranslation_covariant [source]

The translations and the bond-preserving symmetries form a covariant representation on ℓ^2 of the sites.

```
lemma siteTranslation_covariant :
    UnitaryRep.IsCovariantRep (latticeAut g) (siteTranslation G F) (siteSymmetry g)
```

lemma spectrum_sectorOp_eq_site [source]

Symmetry reduction on a crystal. Let `T` be a bounded operator on ℓ^2 of the sites of a crystal, commuting with the translations and with the bond-preserving symmetries. Its restriction to the sector of total momenta in a set `S` of characters is unitarily equivalent to its restriction to the sector of momenta in the image of `S` under any symmetry, by the symmetry itself. In particular their spectra agree.

```
lemma spectrum_sectorOp_eq_site {T : SiteSpace G F →L[ℂ] SiteSpace G F}
    (hTπ : ∀ a, Commute ((siteTranslation G F a : unitary _) : SiteSpace G F →L[ℂ] SiteSpace G
    F) T)
    (hTρ : ∀ s, Commute ((siteSymmetry g s : unitary _) : SiteSpace G F →L[ℂ] SiteSpace G F) T)
    (s : g.symmetries) {S : Set (PontryaginDual (Multiplicative G))} (hS : MeasurableSet S) :
    spectrum ℂ (UnitaryRep.sectorOp hTπ S) =
      spectrum ℂ (UnitaryRep.sectorOp hTπ (UnitaryRep.dualAct (latticeAut g) s '' S))
```

2.261 PhyslibAlpha.CondensedMatter.Bloch.Spectrum

PhyslibAlpha/CondensedMatter/Bloch/Spectrum.lean on GitHub.

lemma norm_sq_calculus_projection_localized [source]

The norm of a continuous function of the translations applied to the momentum-`W` component of a localized state: the L^2 norm of the function over `W`, for the Haar measure.

```
lemma norm_sq_calculus_projection_localized (g : C(PontryaginDual (Multiplicative G), ℂ))
    {W : Set (PontryaginDual (Multiplicative G))} (hW : MeasurableSet W)
    (e : EuclideanSpace ℂ (Fin N)) :
    ‖UnitaryRep.calculus (translation G N) g
        (UnitaryRep.projection (translation G N) W (localized e))‖ ^ 2 =
      ‖e‖ ^ 2 * ∫ χ in W, ‖g χ‖ ^ 2 ∂(dualHaar (G := G))
```

lemma toEuclideanCLM_single_apply [source]

The matrix unit `E_{ij}` acts on a vector as the `j`-th coordinate carried to the `i`-th.

```
lemma toEuclideanCLM_single_apply (i j : Fin N) (e : EuclideanSpace ℂ (Fin N)) :
    Matrix.toEuclideanCLM (n := Fin N) (𝕜 := ℂ) (Matrix.single i j (1 : ℂ)) e =
      e j • EuclideanSpace.single i (1 : ℂ)
```

lemma internal_single_localized [source]

The internal matrix unit `E_{ij}` acts on a localized state by moving the `j`-th component to the `i`-th.

```
lemma internal_single_localized (i j : Fin N) (e : EuclideanSpace ℂ (Fin N)) :
    internal (G := G) (Matrix.single i j (1 : ℂ)) (localized e) =
```

```
    e j • localized (EuclideanSpace.single i (1 : ℂ))
```

abbrev blochOperator [source]

The translation-invariant operator ∫ f dP of a symbol f on ℓ²(G, ℂ^N).

```
abbrev blochOperator (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ)) :
    Space G N →L[ℂ] Space G N
```

lemma commute_projection_internal [source]

The internal matrix units commute with the spectral projections.

```
lemma commute_projection_internal (W : Set (PontryaginDual (Multiplicative G)))
    (M : Matrix (Fin N) (Fin N) ℂ) :
    Commute (UnitaryRep.projection (translation G N) W) (internal (G := G) M)
```

lemma blochOperator_projection_localized [source]

***Action of a symbol on a spectral component of a localized state**: the operator of a symbol f applied to the momentum-W component of the state localized at the origin with internal state e is the sum over the internal indices i of functions of the translations applied to the momentum-W component of the state localized in the i-th internal state.*

```
lemma blochOperator_projection_localized
    (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (W : Set (PontryaginDual (Multiplicative G))) (e : EuclideanSpace ℂ (Fin N)) :
    blochOperator f (UnitaryRep.projection (translation G N) W (localized e)) =
      ∑ i, UnitaryRep.calculus (translation G N) (∑ j, e j • UnitaryRep.entry f i j)
        (UnitaryRep.projection (translation G N) W
          (localized (EuclideanSpace.single i (1 : ℂ))))
```

lemma entry_combination_apply [source]

The i-th component of a symbol applied to a vector is the combination of its entries.

```
lemma entry_combination_apply
    (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (e : EuclideanSpace ℂ (Fin N)) (i : Fin N) (χ : PontryaginDual (Multiplicative G)) :
    (∑ j, e j • UnitaryRep.entry f i j) χ =
      (Matrix.toEuclideanCLM (n := Fin N) (𝕜 := ℂ) (f χ) e) i
```

lemma sum_norm_entry_combination [source]

```
lemma sum_norm_entry_combination
    (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (e : EuclideanSpace ℂ (Fin N)) (χ : PontryaginDual (Multiplicative G)) :
    ∑ i, ‖(∑ j, e j • UnitaryRep.entry f i j) χ‖ ^ 2 =
      ‖Matrix.toEuclideanCLM (n := Fin N) (𝕜 := ℂ) (f χ) e‖ ^ 2
```

lemma norm_blochOperator_projection_localized_le [source]

***The symbol acts on spectral components of a localized state with the size of the symbol on those momenta**: if the symbol moves the internal state `e` by at most `δ` at all crystal momenta in `W`, then the operator of the symbol moves the momentum-`W` component of the localized state by at most `√N δ` times its norm, up to the norm of `e`.*

```
lemma norm_blochOperator_projection_localized_le
    (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    {W : Set (PontryaginDual (Multiplicative G))} (hW : MeasurableSet W)
    (e : EuclideanSpace ℂ (Fin N)) {δ : ℝ}
    (hδ : ∀ χ ∈ W, ‖Matrix.toEuclideanCLM (n := Fin N) (𝕜 := ℂ) (f χ) e‖ ≤ δ) :
    ‖blochOperator f (UnitaryRep.projection (translation G N) W (localized e))‖ ^ 2 ≤
      N * δ ^ 2 * (dualHaar (G := G)).real W
```

lemma norm_sq_projection_localized [source]

The squared norm of the momentum-`W` component of a localized state.

```
lemma norm_sq_projection_localized {W : Set (PontryaginDual (Multiplicative G))}
    (hW : MeasurableSet W) (e : EuclideanSpace ℂ (Fin N)) :
    ‖UnitaryRep.projection (translation G N) W (localized e)‖ ^ 2 =
      ‖e‖ ^ 2 * (dualHaar (G := G)).real W
```

lemma spectrum_blochOperator [source]

***The spectrum of a translation-invariant operator is the union of its Bloch bands.** For every continuous symbol `b` on the Brillouin zone, the spectrum of the operator `∫ b dP` on `ℓ²(G, ℂ^N)` is the union over crystal momenta `χ` of the spectra of the matrices `b(χ)`.*

```
lemma spectrum_blochOperator
    (b : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ)) :
    spectrum ℂ (blochOperator b) =
      ⋃ χ : PontryaginDual (Multiplicative G), spectrum ℂ (b χ)
```

lemma spectrum_blochOperator_eq_bands [source]

***The band structure**: for a Hermitian symbol, the spectrum of the translation-invariant operator on `ℓ²(G, ℂ^N)` is exactly the set of band values `ε_k(χ)`, the ordered eigenvalues of the Bloch matrices.*

```
lemma spectrum_blochOperator_eq_bands
    (b : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (hb : ∀ χ, (b χ).IsHermitian) :
    spectrum ℂ (blochOperator b) =
      {z | ∃ (χ : PontryaginDual (Multiplicative G)) (k : Fin (Fintype.card (Fin N))),
        z = ((bandOf (hb χ) k : ℝ) : ℂ)}
```

2.262 PhyslibAlpha.CondensedMatter.Crystal.Balls

PhyslibAlpha/CondensedMatter/Crystal/Balls.lean on GitHub.

def ball [source]

The sites within `n` bonds of a site.

```
def ball : ℕ → C.Site → Finset C.Site
  | 0, x => {x}
  | n + 1, x => ball n x ∪ (ball n x).biUnion g.neighbors
```

lemma self_mem_ball [source]

```
lemma self_mem_ball (n : ℕ) (x : C.Site) : x ∈ g.ball n x
```

lemma ball_mono [source]

```
lemma ball_mono {m n : ℕ} (h : m ≤ n) (x : C.Site) : g.ball m x ⊆ g.ball n x
```

lemma mem_ball_add [source]

Following `m` bonds and then `n` bonds stays within `m + n` bonds.

```
lemma mem_ball_add {m n : ℕ} {a b c : C.Site} (hb : b ∈ g.ball n a) (hc : c ∈ g.ball m b) :
    c ∈ g.ball (m + n) a
```

lemma mem_neighbors_smul_iff [source]

A group action preserving the bonds carries the neighbours of a site to the neighbours of its image.

```
lemma mem_neighbors_smul_iff (hH : ∀ (h : H) (x y : C.Site),
    g.toSimpleGraph.Adj (h • x) (h • y) ↔ g.toSimpleGraph.Adj x y) (h : H) (x y : C.Site) :
    h • y ∈ g.neighbors (h • x) ↔ y ∈ g.neighbors x
```

lemma mem_ball_smul_iff [source]

A group action preserving the bonds carries balls to balls.

```
lemma mem_ball_smul_iff (hH : ∀ (h : H) (x y : C.Site),
    g.toSimpleGraph.Adj (h • x) (h • y) ↔ g.toSimpleGraph.Adj x y) (h : H) (n : ℕ)
    (x y : C.Site) : h • y ∈ g.ball n (h • x) ↔ y ∈ g.ball n x
```

2.263 PhyslibAlpha.CondensedMatter.Crystal.Basic

PhyslibAlpha/CondensedMatter/Crystal/Basic.lean on GitHub.

structure Crystal [source]

A perfectly repeating arrangement of sites under the translation group `G`. Every site belongs to one of finitely many families of translated copies.

```
structure Crystal (G : Type u) [AddGroup G] where
  Site : Type v
  [translations : AddAction G Site]
  [free : IsCancelVAdd G Site]
  [finiteSiteOrbits : Finite (AddAction.orbitRel.Quotient G Site)]
```

instance AddAction G (CellSite G F) [source]

Translate a site by changing its cell index and preserving its label.

```
instance {G F : Type*} [AddGroup G] : AddAction G (CellSite G F) where
```

instance IsCancelVAdd G (CellSite G F) [source]

Different cell translations give different copies of a site.

```
instance {G F : Type*} [AddGroup G] : IsCancelVAdd G (CellSite G F) where
```

def ofUnitCell [source]

Construct a crystal by repeating a unit cell containing one site for each label in `F`. The group `G` indexes translated cells. A site of the resulting crystal has address `⟨n, i⟩`, where `n : G` specifies its cell and `i : F` identifies the site within that cell.

```
@[reducible]
def ofUnitCell (G : Type u) (F : Type v) [AddGroup G] [Finite F] : Crystal G where
```

instance Finite C.Site [source]

A finite translation group gives a finite crystal.

```
instance {G : Type*} [AddGroup G] [Finite G] (C : Crystal G) : Finite C.Site
```

2.264 PhyslibAlpha.CondensedMatter.Crystal.Brillouin

PhyslibAlpha/CondensedMatter/Crystal/Brillouin.lean on GitHub.

abbrev BrillouinZone [source]

The Brillouin zone of a crystal with translation group `G`: the crystal momenta, which are the characters of `G`.

```
abbrev BrillouinZone (G : Type*) [AddCommGroup G] [TopologicalSpace G]
```

def dualMap [source]

A lattice automorphism acts on a character by precomposition.

```
noncomputable def dualMap (A : G ≃+ G) (χ : BrillouinZone G) : BrillouinZone G where
```

lemma dualMap_apply [source]

```
lemma dualMap_apply (A : G ≃+ G) (χ : BrillouinZone G) (a : G) :
    dualMap A χ (Multiplicative.ofAdd a) = χ (Multiplicative.ofAdd (A a))
```

lemma dualMap_refl [source]

```
lemma dualMap_refl (χ : BrillouinZone G) : dualMap (AddEquiv.refl G) χ = χ
```

lemma dualMap_trans [source]

```
lemma dualMap_trans (A B : G ≃+ G) (χ : BrillouinZone G) :
    dualMap B (dualMap A χ) = dualMap (B.trans A) χ
```

instance MulAction (CellSymmetry G F) (BrillouinZone G) [source]

The symmetries of a crystal act on the crystal momenta: a symmetry with linear part A sends χ to $\chi \circ A^{-1}$.

```
noncomputable instance : MulAction (CellSymmetry G F) (BrillouinZone G) where
```

lemma smul_apply [source]

```
lemma smul_apply (s : CellSymmetry G F) (χ : BrillouinZone G) (a : G) :
    (s • χ) (Multiplicative.ofAdd a) = χ (Multiplicative.ofAdd (s.linear.symm a))
```

lemma translation_smul_brillouin [source]

Translations act trivially on crystal momenta.

```
lemma translation_smul_brillouin (a : G) (χ : BrillouinZone G) :
    (CellSymmetry.translation a : CellSymmetry G F) • χ = χ
```

def symmetryAction [source]

The bond-preserving symmetries act on the Brillouin zone by permutations.

```
noncomputable def symmetryAction : g.symmetries →* Equiv.Perm (BrillouinZone G)
```

lemma translations_le_ker_symmetryAction [source]

```
lemma translations_le_ker_symmetryAction :
    (CellSymmetry.translations G F).subgroupOf g.symmetries ≤ (symmetryAction g).ker
```

def pointAction [source]

The point group acts on the Brillouin zone.

```
noncomputable def pointAction : g.pointGroup →* Equiv.Perm (BrillouinZone G)
```

instance MulAction g.pointGroup (BrillouinZone G) [source]

```
noncomputable instance : MulAction g.pointGroup (BrillouinZone G)
```

lemma mk_smul [source]

```
lemma mk_smul (s : g.symmetries) (χ : BrillouinZone G) :
    (QuotientGroup.mk s : g.pointGroup) • χ = s • χ
```

def littleGroup [source]

The little group of a momentum: the bond-preserving symmetries that fix it. It contains all translations.

```
noncomputable def littleGroup (χ : BrillouinZone G) : Subgroup g.symmetries
```

def littleCoGroup [source]

The little co-group of a momentum: the elements of the point group that fix it.

```
noncomputable def littleCoGroup (χ : BrillouinZone G) : Subgroup g.pointGroup
```

lemma mem_littleGroup [source]

```
lemma mem_littleGroup (χ : BrillouinZone G) (s : g.symmetries) :
    s ∈ g.littleGroup χ ↔ s • χ = χ
```

lemma translation_mem_littleGroup [source]

```
lemma translation_mem_littleGroup (χ : BrillouinZone G) (a : G) :
    (⟨CellSymmetry.translation a, g.translation_mem_symmetries a⟩ : g.symmetries) ∈
      g.littleGroup χ
```

lemma littleGroup_eq_comap [source]

The little group is the preimage of the little co-group.

```
lemma littleGroup_eq_comap (χ : BrillouinZone G) :
    g.littleGroup χ = (g.littleCoGroup χ).comap (QuotientGroup.mk' _)
```

abbrev IrreducibleZone [source]

The irreducible Brillouin zone: the momenta modulo the point group.

```
abbrev IrreducibleZone
```

lemma irreducibleZone_mk_eq_iff [source]

Momenta in the same orbit of the point group are identified in the irreducible zone.

```
lemma irreducibleZone_mk_eq_iff (χ ψ : BrillouinZone G) :
    (Quotient.mk (MulAction.orbitRel g.pointGroup (BrillouinZone G)) χ : g.IrreducibleZone) =
      Quotient.mk _ ψ ↔ ∃ p : g.pointGroup, p • ψ = χ
```

2.265 PhyslibAlpha.CondensedMatter.Crystal.BrillouinCoordinates

PhyslibAlpha/CondensedMatter/Crystal/BrillouinCoordinates.lean on GitHub.

def brillouinPoint [source]

The momentum with coordinates `k` along the reciprocal basis: the character `r ↦ exp (2πi k · r)` of `ℤ^d`.

```
noncomputable def brillouinPoint (k : Fin d → ℝ) : BrillouinZone (Fin d → ℤ) where
```

lemma brillouinPoint_apply [source]

```
lemma brillouinPoint_apply (k : Fin d → ℝ) (r : Fin d → ℤ) :
    brillouinPoint k (Multiplicative.ofAdd r) =
      Circle.exp (2 * Real.pi * ∑ j, k j * (r j : ℝ))
```

lemma brillouinPoint_single [source]

```
lemma brillouinPoint_single (k : Fin d → ℝ) (j : Fin d) :
    brillouinPoint k (Multiplicative.ofAdd (Pi.single j 1)) = Circle.exp (2 * Real.pi * k j)
```

lemma brillouinPoint_eq_iff [source]

Two sets of coordinates give the same momentum exactly when they differ by integers.

```
lemma brillouinPoint_eq_iff (k k' : Fin d → ℝ) :
    brillouinPoint k = brillouinPoint k' ↔ ∀ j, ∃ z : ℤ, k j - k' j = z
```

lemma brillouinZone_ext [source]

A character of ℤ^d is determined by its values on the basis vectors.

```
lemma brillouinZone_ext {χ ψ : BrillouinZone (Fin d → ℤ)}
    (h : ∀ j, χ (Multiplicative.ofAdd (Pi.single j 1)) = ψ (Multiplicative.ofAdd (Pi.single j
    1))) :
    χ = ψ
```

lemma brillouinPoint_surjective [source]

Every momentum has coordinates.

```
lemma brillouinPoint_surjective : Function.Surjective (brillouinPoint (d := d))
```

lemma apply_eq_sum_single [source]

A lattice automorphism is determined by its values on the basis vectors.

```
lemma apply_eq_sum_single (A : (Fin d → ℤ) ≃+ (Fin d → ℤ)) (r : Fin d → ℤ) (i : Fin d) :
    A r i = ∑ j, r j * A (Pi.single j 1) i
```

lemma smul_brillouinPoint [source]

***A symmetry acts on the coordinates of a momentum** by the transpose of the inverse of the matrix of its linear part.*

```
lemma smul_brillouinPoint {F : Type*} (s : CellSymmetry (Fin d → ℤ) F) (k : Fin d → ℝ) :
    s • brillouinPoint k =
      brillouinPoint (fun j => ∑ i, k i * (s.linear.symm (Pi.single j 1) i : ℝ))
```

def torsionPoint [source]

The momentum with coordinates `m / n`: a point of finite order of the Brillouin zone.

```
noncomputable def torsionPoint (n : ℕ) (m : Fin d → ℤ) : BrillouinZone (Fin d → ℤ)
```

lemma torsionPoint_eq_iff [source]

Momenta of finite order are equal exactly when the numerators agree modulo `n`.

```
lemma torsionPoint_eq_iff {n : ℕ} (hn : 0 < n) (m m' : Fin d → ℤ) :
    torsionPoint n m = torsionPoint n m' ↔ ∀ j, (n : ℤ) ∣ m j - m' j
```

lemma smul_torsionPoint [source]

A symmetry acts on a momentum of finite order by an integer matrix on the numerators.

```
lemma smul_torsionPoint {F : Type*} (n : ℕ) (s : CellSymmetry (Fin d → ℤ) F) (m : Fin d → ℤ) :
    s • torsionPoint n m =
      torsionPoint n (fun j => ∑ i, m i * s.linear.symm (Pi.single j 1) i)
```

lemma mem_littleGroup_torsionPoint_iff [source]

***The little group of a momentum of finite order**, as integer arithmetic: a bond-preserving symmetry fixes the momentum with numerators `m` modulo `n` exactly when the integer vector obtained from the matrix of its linear part is congruent to `m` modulo `n`.*

```
lemma mem_littleGroup_torsionPoint_iff {F : Type*} [Finite F]
    (g : PeriodicGraph (Crystal.ofUnitCell (Fin d → ℤ) F)) {n : ℕ} (hn : 0 < n)
    (m : Fin d → ℤ) (s : g.symmetries) :
    s ∈ g.littleGroup (torsionPoint n m) ↔
      ∀ j, (n : ℤ) ∣ (∑ i, m i * (s : CellSymmetry (Fin d → ℤ) F).linear.symm (Pi.single j 1) i)
     -
        m j
```

2.266 PhyslibAlpha.CondensedMatter.Crystal.Examples.Basic

PhyslibAlpha/CondensedMatter/Crystal/Examples/Basic.lean on GitHub.

def hypercubic [source]

One site per integer-indexed cell in `d` dimensions.

```
@[reducible]
def hypercubic (d : ℕ)
```

def hypercubicDisplacement [source]

Unit orthogonal cell displacements in Cartesian coordinates.

```
noncomputable def hypercubicDisplacement (d : ℕ) :
    (Fin d → ℤ) →+ EuclideanSpace ℝ (Fin d)
```

def hypercubicGeometry [source]

The hypercubic lattice embedded with unit spacing along each Cartesian axis.

```
noncomputable def hypercubicGeometry (d : ℕ) : Geometry (hypercubic d) (EuclideanSpace ℝ (Fin d
    ))
```

def hypercubicBonds [source]

One positive-axis bond for each direction generates all hypercubic connections.

```
def hypercubicBonds (d : ℕ) : List (Bond (Fin d → ℤ) Unit)
```

def hypercubicGraph [source]

The graph connecting each site to its neighboring cells along the coordinate axes.

```
def hypercubicGraph (d : ℕ) : PeriodicGraph (hypercubic d)
```

def chain [source]

The infinite chain with one site per cell.

```
@[reducible]
def chain
```

def chainGeometry [source]

Unit-spaced positions of the infinite chain.

```
noncomputable def chainGeometry
```

def chainGraph [source]

Consecutive sites of the infinite chain are connected.

```
def chainGraph
```

def square [source]

The square lattice with one site per cell.

```
@[reducible]
def square
```

def squareGeometry [source]

Cartesian positions of the unit-spaced square lattice.

```
noncomputable def squareGeometry
```

def squareGraph [source]

The four-neighbor square-lattice graph.

```
def squareGraph
```

def triangular [source]

One site per cell of the triangular lattice.

```
@[reducible]
def triangular
```

def triangularVectors [source]

Unit primitive vectors meeting at an angle of sixty degrees.

```
noncomputable def triangularVectors : Fin 2 → EuclideanSpace ℝ (Fin 2)
```

lemma triangularVectors_independent [source]

The two primitive translations of the triangular lattice are independent.

```
lemma triangularVectors_independent : LinearIndependent ℝ triangularVectors
```

def triangularGeometry [source]

The triangular lattice embedded with unit bond lengths.

```
noncomputable def triangularGeometry : Geometry triangular (EuclideanSpace ℝ (Fin 2))
```

def triangularBonds [source]

Three cell displacements generate the six triangular-lattice neighbors.

```
def triangularBonds : List (Bond (Fin 2 → ℤ) Unit)
```

def triangularGraph [source]

The six-neighbor triangular-lattice graph.

```
def triangularGraph : PeriodicGraph triangular
```

inductive HoneycombLabel [source]

The two sites in a honeycomb unit cell.

```
inductive HoneycombLabel
  | A
  | B
  deriving DecidableEq, Repr
```

instance Fintype HoneycombLabel [source]

```
instance : Fintype HoneycombLabel where
```

def honeycomb [source]

A honeycomb crystal has two sites per integer-indexed cell.

```
@[reducible]
def honeycomb
```

def honeycombVectors [source]

Primitive cell translations for a honeycomb lattice with unit bond lengths.

```
noncomputable def honeycombVectors : Fin 2 → EuclideanSpace ℝ (Fin 2)
```

lemma honeycombVectors_independent [source]

The primitive cell translations of the honeycomb lattice are independent.

```
lemma honeycombVectors_independent : LinearIndependent ℝ honeycombVectors
```

def honeycombBasis [source]

Physical positions of A and B within the reference honeycomb cell.

```
noncomputable def honeycombBasis : HoneycombLabel → EuclideanSpace ℝ (Fin 2)
  | .A => 0
  | .B => !₂[0, 1]
```

lemma honeycomb_positions_injective [source]

The honeycomb reference positions and their translated copies are all distinct.

```
lemma honeycomb_positions_injective : Function.Injective
    (fun x : CellSite (Fin 2 → ℤ) HoneycombLabel =>
      latticeMap honeycombVectors x.cell + honeycombBasis x.label)
```

def honeycombGeometry [source]

The honeycomb crystal embedded with distinct A and B sites and unit bond spacing.

```
noncomputable def honeycombGeometry : Geometry honeycomb (EuclideanSpace ℝ (Fin 2))
```

def honeycombBonds [source]

Each A site connects to B in its own cell and in two adjacent cells.

```
def honeycombBonds : List (Bond (Fin 2 → ℤ) HoneycombLabel)
```

def honeycombGraph [source]

The three-neighbor honeycomb graph.

```
def honeycombGraph : PeriodicGraph honeycomb
```

inductive DiamondLabel [source]

The two atoms in a diamond unit cell.

```
inductive DiamondLabel
  | A
  | B
  deriving DecidableEq, Repr
```

instance Fintype DiamondLabel [source]

```
instance : Fintype DiamondLabel where
```

def diamond [source]

A diamond crystal has two atoms per integer-indexed cell of its face-centered cubic lattice.

```
@[reducible]
def diamond
```

def diamondVectors [source]

Primitive translations of the face-centered cubic lattice, in units of the cubic lattice constant.

```
noncomputable def diamondVectors : Fin 3 → EuclideanSpace ℝ (Fin 3)
```

lemma diamondVectors_independent [source]

The primitive translations of the face-centered cubic lattice are independent.

```
lemma diamondVectors_independent : LinearIndependent ℝ diamondVectors
```

def diamondBasis [source]

Physical positions of the two atoms within the reference diamond cell.

```
noncomputable def diamondBasis : DiamondLabel → EuclideanSpace ℝ (Fin 3)
  | .A => 0
  | .B => !₂[1 / 4, 1 / 4, 1 / 4]
```

lemma diamond_positions_injective [source]

The positions of the two diamond sublattices are distinct, and so are all translated copies.

```
lemma diamond_positions_injective : Function.Injective
    (fun x : CellSite (Fin 3 → ℤ) DiamondLabel =>
      latticeMap diamondVectors x.cell + diamondBasis x.label)
```

def diamondGeometry [source]

The diamond crystal embedded with distinct atoms in three-dimensional space.

```
noncomputable def diamondGeometry : Geometry diamond (EuclideanSpace ℝ (Fin 3))
```

def diamondBonds [source]

Each A atom connects to the B atom of its own cell and of the three cells reached by subtracting one primitive translation: its four tetrahedral neighbors.

```
def diamondBonds : List (Bond (Fin 3 → ℤ) DiamondLabel)
```

def diamondGraph [source]

The four-neighbor tetrahedral diamond graph.

```
def diamondGraph : PeriodicGraph diamond
```

def periodicChain [source]

A finite chain whose cell indices are taken modulo `m`.

```
@[reducible]
def periodicChain (m : ℕ)
```

def periodicChainGeometry [source]

Equally spaced sites on a circle of circumference one.

```
noncomputable def periodicChainGeometry (m : ℕ) [NeZero m] :
    Geometry (periodicChain m) UnitAddCircle
```

def periodicChainBonds [source]

Connect consecutive cells around the periodic chain.

```
def periodicChainBonds (m : ℕ) : List (Bond (ZMod m) Unit)
```

def periodicChainGraph [source]

The undirected graph of consecutive cells modulo `m`.

```
def periodicChainGraph (m : ℕ) : PeriodicGraph (periodicChain m)
```

lemma chain_neighbors_card [source]

Each site of the infinite chain has two neighbors.

```
lemma chain_neighbors_card (x : chain.Site) : (chainGraph.neighbors x).card = 2
```

lemma square_neighbors_card [source]

Each site of the square lattice has four neighbors.

```
lemma square_neighbors_card (x : square.Site) : (squareGraph.neighbors x).card = 4
```

lemma triangular_neighbors_card [source]

Each site of the triangular lattice has six neighbors.

```
lemma triangular_neighbors_card (x : triangular.Site) : (triangularGraph.neighbors x).card = 6
```

lemma honeycomb_neighbors_card [source]

Both honeycomb site labels have three neighbors in every cell.

```
lemma honeycomb_neighbors_card (x : honeycomb.Site) : (honeycombGraph.neighbors x).card = 3
```

lemma diamond_neighbors_card [source]

Both diamond site labels have four neighbors in every cell.

```
lemma diamond_neighbors_card (x : diamond.Site) : (diamondGraph.neighbors x).card = 4
```

lemma hypercubic_bond_length [source]

Every hypercubic graph edge has unit physical length, in any dimension.

```
lemma hypercubic_bond_length (d : ℕ) {x y : (hypercubic d).Site}
    (h : (hypercubicGraph d).toSimpleGraph.Adj x y) :
    (hypercubicGeometry d).distance x y = 1
```

lemma square_bond_length [source]

All square-lattice graph edges have unit physical length.

```
lemma square_bond_length {x y : square.Site} (h : squareGraph.toSimpleGraph.Adj x y) :
    squareGeometry.distance x y = 1
```

lemma triangular_bond_length [source]

All triangular-lattice graph edges have unit physical length.

```
lemma triangular_bond_length {x y : triangular.Site}
    (h : triangularGraph.toSimpleGraph.Adj x y) : triangularGeometry.distance x y = 1
```

lemma honeycomb_bond_length [source]

All honeycomb graph edges have unit physical length.

```
lemma honeycomb_bond_length {x y : honeycomb.Site}
    (h : honeycombGraph.toSimpleGraph.Adj x y) : honeycombGeometry.distance x y = 1
```

2.267 PhyslibAlpha.CondensedMatter.Crystal.Examples.Monoclinic

PhyslibAlpha/CondensedMatter/Crystal/Examples/Monoclinic.lean on GitHub.

def monoclinic [source]

One site per primitive cell of the primitive monoclinic lattice.

```
@[reducible]
def monoclinic
```

def monoclinicVectors [source]

The primitive vectors `(a, 0, 0)`, `(0, b, 0)` and `(c₁, 0, c₃)`.

```
noncomputable def monoclinicVectors (a b c₁ c₃ : ℝ) : Fin 3 → EuclideanSpace ℝ (Fin 3)
```

lemma monoclinicVectors_independent [source]

The primitive vectors are independent for nonzero `a`, `b` and `c₃`.

```
lemma monoclinicVectors_independent {a b c₁ c₃ : ℝ} (ha : a ≠ 0) (hb : b ≠ 0) (hc : c₃ ≠ 0) :
    LinearIndependent ℝ (monoclinicVectors a b c₁ c₃)
```

def monoclinicBonds [source]

The bonds of the first shells for `a = 3`, `b = 5`, `c = (1, 0, 3)`, up to sign.

```
def monoclinicBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def monoclinicGraph [source]

The twenty-neighbour primitive monoclinic graph.

```
def monoclinicGraph : PeriodicGraph monoclinic
```

def baseCentredMonoclinic [source]

One site per primitive cell of the base-centred monoclinic lattice.

```
@[reducible]
def baseCentredMonoclinic
```

def baseCentredMonoclinicVectors [source]

The primitive vectors `(a / 2, -b / 2, 0)`, `(a / 2, b / 2, 0)` and `(c₁, 0, c₃)`.

```
noncomputable def baseCentredMonoclinicVectors (a b c₁ c₃ : ℝ) :
    Fin 3 → EuclideanSpace ℝ (Fin 3)
```

lemma baseCentredMonoclinicVectors_independent [source]

The primitive vectors are independent for nonzero `a`, `b` and `c₃`.

```
lemma baseCentredMonoclinicVectors_independent {a b c₁ c₃ : ℝ} (ha : a ≠ 0) (hb : b ≠ 0)
    (hc : c₃ ≠ 0) : LinearIndependent ℝ (baseCentredMonoclinicVectors a b c₁ c₃)
```

def baseCentredMonoclinicBonds [source]

The bonds of the first shells for `a = 4`, `b = 2`, `c = (1, 0, 3)`, up to sign, in the basis `baseCentredMonoclinicVectors`.

```
def baseCentredMonoclinicBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def baseCentredMonoclinicGraph [source]

The sixteen-neighbour base-centred monoclinic graph.

```
def baseCentredMonoclinicGraph : PeriodicGraph baseCentredMonoclinic
```

def triclinic [source]

One site per primitive cell of the triclinic lattice.

```
@[reducible]
def triclinic
```

def triclinicVectors [source]

The primitive vectors `(a, 0, 0)`, `(b₁, b₂, 0)` and `(c₁, c₂, c₃)`.

```
noncomputable def triclinicVectors (a b₁ b₂ c₁ c₂ c₃ : ℝ) : Fin 3 → EuclideanSpace ℝ (Fin 3)
```

lemma triclinicVectors_independent [source]

The primitive vectors are independent for nonzero `a`, `b₂` and `c₃`.

```
lemma triclinicVectors_independent {a b₁ b₂ c₁ c₂ c₃ : ℝ} (ha : a ≠ 0) (hb : b₂ ≠ 0)
    (hc : c₃ ≠ 0) : LinearIndependent ℝ (triclinicVectors a b₁ b₂ c₁ c₂ c₃)
```

def triclinicBonds [source]

The bonds of the first shells for `(3, 0, 0)`, `(1, 4, 0)`, `(1, 1, 5)`, up to sign.

```
def triclinicBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def triclinicGraph [source]

The sixteen-neighbour triclinic graph.

```
def triclinicGraph : PeriodicGraph triclinic
```

lemma monoclinic_neighbors_card [source]

Each site of the primitive monoclinic lattice has twenty neighbours.

```
lemma monoclinic_neighbors_card (x : monoclinic.Site) :
    (monoclinicGraph.neighbors x).card = 20
```

lemma baseCentredMonoclinic_neighbors_card [source]

Each site of the base-centred monoclinic lattice has sixteen neighbours.

```
lemma baseCentredMonoclinic_neighbors_card (x : baseCentredMonoclinic.Site) :
    (baseCentredMonoclinicGraph.neighbors x).card = 16
```

lemma triclinic_neighbors_card [source]

Each site of the triclinic lattice has sixteen neighbours.

```
lemma triclinic_neighbors_card (x : triclinic.Site) :
    (triclinicGraph.neighbors x).card = 16
```

2.268 PhyslibAlpha.CondensedMatter.Crystal.Examples.Orthorhombic

PhyslibAlpha/CondensedMatter/Crystal/Examples/Orthorhombic.lean on GitHub.

def orthorhombic [source]

One site per primitive cell of the primitive orthorhombic lattice.

```
@[reducible]
def orthorhombic
```

def orthorhombicBonds [source]

The bonds of the first shells for `(a, b, c) = (3, 4, 7)`, up to sign: the three axes, the two in-plane diagonals and twice the shortest axis.

```
def orthorhombicBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def orthorhombicGraph [source]

The twelve-neighbour primitive orthorhombic graph.

```
def orthorhombicGraph : PeriodicGraph orthorhombic
```

def baseCentredOrthorhombic [source]

One site per primitive cell of the base-centred orthorhombic lattice.

```
@[reducible]
def baseCentredOrthorhombic
```

def baseCentredVectors [source]

Primitive vectors from a corner to the centres of the two adjacent `a`–`b` faces of a box with sides `a`, `b`, `c` that share an edge along `x`, and along the third axis.

```
noncomputable def baseCentredVectors (a b c : ℝ) : Fin 3 → EuclideanSpace ℝ (Fin 3)
```

lemma baseCentredVectors_independent [source]

The primitive vectors of the base-centred lattice are independent for positive lengths.

```
lemma baseCentredVectors_independent {a b c : ℝ} (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
    LinearIndependent ℝ (baseCentredVectors a b c)
```

def baseCentredOrthorhombicGeometry [source]

The base-centred orthorhombic lattice embedded with a box of sides `a`, `b`, `c`.

```
noncomputable def baseCentredOrthorhombicGeometry (a b c : ℝ) (ha : 0 < a) (hb : 0 < b)
    (hc : 0 < c) : Geometry baseCentredOrthorhombic (EuclideanSpace ℝ (Fin 3))
```

def baseCentredOrthorhombicBonds [source]

The bonds of the first shells for `(a, b, c) = (3, 6, 5)`, up to sign: the two primitive vectors in the plane, the third axis, their sum `a x` and the two vectors `(3a / 2, ±b / 2, 0)`.

```
def baseCentredOrthorhombicBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def baseCentredOrthorhombicGraph [source]

The twelve-neighbour base-centred orthorhombic graph.

```
def baseCentredOrthorhombicGraph : PeriodicGraph baseCentredOrthorhombic
```

def bodyCentredOrthorhombic [source]

One site per primitive cell of the body-centred orthorhombic lattice.

```
@[reducible]
def bodyCentredOrthorhombic
```

def bodyCentredOrthorhombicBonds [source]

The bonds of the first shells for `(a, b, c) = (2, 7, 5)` in the basis `bccVectors a b c`, up to sign: the four body diagonals, `a x`, `2a x` and `c z`.

```
def bodyCentredOrthorhombicBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def bodyCentredOrthorhombicGraph [source]

The fourteen-neighbour body-centred orthorhombic graph, embedded by `bccGeometry a b c`.

```
def bodyCentredOrthorhombicGraph : PeriodicGraph bodyCentredOrthorhombic
```

def faceCentredOrthorhombic [source]

One site per primitive cell of the face-centred orthorhombic lattice.

```
@[reducible]
def faceCentredOrthorhombic
```

def faceCentredOrthorhombicBonds [source]

The bonds of the first shells for `(a, b, c) = (2, 4, 5)` in the basis `fccVectors a b c`, up to sign: the six face diagonals, `a x` and the two vectors `(3a / 2, ±b / 2, 0)`.

```
def faceCentredOrthorhombicBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def faceCentredOrthorhombicGraph [source]

The eighteen-neighbour face-centred orthorhombic graph, embedded by `fccGeometry a b c`.

```
def faceCentredOrthorhombicGraph : PeriodicGraph faceCentredOrthorhombic
```

lemma orthorhombic_neighbors_card [source]

Each site of the primitive orthorhombic lattice has twelve neighbours.

```
lemma orthorhombic_neighbors_card (x : orthorhombic.Site) :
    (orthorhombicGraph.neighbors x).card = 12
```

lemma baseCentredOrthorhombic_neighbors_card [source]

Each site of the base-centred orthorhombic lattice has twelve neighbours.

```
lemma baseCentredOrthorhombic_neighbors_card (x : baseCentredOrthorhombic.Site) :
    (baseCentredOrthorhombicGraph.neighbors x).card = 12
```

lemma bodyCentredOrthorhombic_neighbors_card [source]

Each site of the body-centred orthorhombic lattice has fourteen neighbours.

```
lemma bodyCentredOrthorhombic_neighbors_card (x : bodyCentredOrthorhombic.Site) :
    (bodyCentredOrthorhombicGraph.neighbors x).card = 14
```

lemma faceCentredOrthorhombic_neighbors_card [source]

Each site of the face-centred orthorhombic lattice has eighteen neighbours.

```
lemma faceCentredOrthorhombic_neighbors_card (x : faceCentredOrthorhombic.Site) :
    (faceCentredOrthorhombicGraph.neighbors x).card = 18
```

2.269 PhyslibAlpha.CondensedMatter.Crystal.Examples.Rhombohedral

PhyslibAlpha/CondensedMatter/Crystal/Examples/Rhombohedral.lean on GitHub.

def rhombohedral [source]

One site per primitive cell of the rhombohedral lattice.

```
@[reducible]
def rhombohedral
```

def rhombohedralVectors [source]

The primitive vectors `(p, q, q)`, `(q, p, q)` and `(q, q, p)` of a rhombohedral lattice.

```
noncomputable def rhombohedralVectors (p q : ℝ) : Fin 3 → EuclideanSpace ℝ (Fin 3)
```

lemma rhombohedralVectors_independent [source]

The primitive vectors are independent when `p ≠ q` and `p + 2 q ≠ 0`.

```
lemma rhombohedralVectors_independent {p q : ℝ} (hpq : p ≠ q) (hp : p + 2 * q ≠ 0) :
    LinearIndependent ℝ (rhombohedralVectors p q)
```

def rhombohedralGeometry [source]

The rhombohedral lattice embedded with the primitive vectors `rhombohedralVectors p q`.

```
noncomputable def rhombohedralGeometry (p q : ℝ) (hpq : p ≠ q) (hp : p + 2 * q ≠ 0) :
    Geometry rhombohedral (EuclideanSpace ℝ (Fin 3))
```

def rhombohedralBonds [source]

The bonds of the first two shells for `-1 / 4 < cos α < 0`, up to sign: the three primitive vectors and their pairwise sums.

```
def rhombohedralBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def rhombohedralGraph [source]

The twelve-neighbour rhombohedral graph.

```
def rhombohedralGraph : PeriodicGraph rhombohedral
```

lemma rhombohedral_neighbors_card [source]

Each site of the rhombohedral lattice has twelve neighbours.

```
lemma rhombohedral_neighbors_card (x : rhombohedral.Site) :
    (rhombohedralGraph.neighbors x).card = 12
```

2.270 PhyslibAlpha.CondensedMatter.Crystal.Examples.TetragonalHexagonal

PhyslibAlpha/CondensedMatter/Crystal/Examples/TetragonalHexagonal.lean on GitHub.

def tetragonal [source]

One site per primitive cell of the primitive tetragonal lattice.

```
@[reducible]
def tetragonal
```

def tetragonalGeometry [source]

The primitive tetragonal lattice embedded with in-plane length `a` and period `c`.

```
noncomputable def tetragonalGeometry (a c : ℝ) (ha : 0 < a) (hc : 0 < c) :
    Geometry tetragonal (EuclideanSpace ℝ (Fin 3))
```

def tetragonalBonds [source]

The bonds of the first three shells for `a < c < a √2`, up to sign: the two in-plane axes, the period, and the two in-plane diagonals.

```
def tetragonalBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def tetragonalGraph [source]

The ten-neighbour primitive tetragonal graph.

```
def tetragonalGraph : PeriodicGraph tetragonal
```

def bodyCentredTetragonal [source]

One site per primitive cell of the body-centred tetragonal lattice.

```
@[reducible]
def bodyCentredTetragonal
```

def bodyCentredTetragonalGeometry [source]

The body-centred tetragonal lattice embedded with in-plane length `a` and period `c`, with the primitive vectors `bccVectors a a c`.

```
noncomputable def bodyCentredTetragonalGeometry (a c : ℝ) (ha : 0 < a) (hc : 0 < c) :
    Geometry bodyCentredTetragonal (EuclideanSpace ℝ (Fin 3))
```

def bodyCentredTetragonalBonds [source]

The bonds of the first three shells for `a √2 < c < a √6`, up to sign: the four body diagonals, the two in-plane axes and the two in-plane diagonals. In the basis `bccVectors a a c` the in-plane axes are `![0, 1, 1]` and `![1, 0, 1]`.

```
def bodyCentredTetragonalBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def bodyCentredTetragonalGraph [source]

The sixteen-neighbour body-centred tetragonal graph.

```
def bodyCentredTetragonalGraph : PeriodicGraph bodyCentredTetragonal
```

def hexagonal [source]

One site per primitive cell of the primitive hexagonal lattice.

```
@[reducible]
def hexagonal
```

def hexagonalGeometry [source]

The primitive hexagonal lattice embedded with in-plane length `a` and period `c`, with the primitive vectors `hcpVectors a c`.

```
noncomputable def hexagonalGeometry (a c : ℝ) (ha : 0 < a) (hc : 0 < c) :
    Geometry hexagonal (EuclideanSpace ℝ (Fin 3))
```

def hexagonalBonds [source]

The bonds of the first shells for `c < a √3`, up to sign: three in the plane, at `120°` from each other, and the period.

```
def hexagonalBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def hexagonalGraph [source]

The eight-neighbour primitive hexagonal graph.

```
def hexagonalGraph : PeriodicGraph hexagonal
```

lemma tetragonal_neighbors_card [source]

Each site of the primitive tetragonal lattice has ten neighbours.

```
lemma tetragonal_neighbors_card (x : tetragonal.Site) :
    (tetragonalGraph.neighbors x).card = 10
```

lemma bodyCentredTetragonal_neighbors_card [source]

Each site of the body-centred tetragonal lattice has sixteen neighbours.

```
lemma bodyCentredTetragonal_neighbors_card (x : bodyCentredTetragonal.Site) :
    (bodyCentredTetragonalGraph.neighbors x).card = 16
```

lemma hexagonal_neighbors_card [source]

Each site of the primitive hexagonal lattice has eight neighbours.

```
lemma hexagonal_neighbors_card (x : hexagonal.Site) :
    (hexagonalGraph.neighbors x).card = 8
```

2.271 PhyslibAlpha.CondensedMatter.Crystal.Examples.ThreeDimensional

PhyslibAlpha/CondensedMatter/Crystal/Examples/ThreeDimensional.lean on GitHub.

def orthorhombicVectors [source]

The primitive vectors of an orthorhombic lattice: `l i` along the `i`-th coordinate axis.

```
noncomputable def orthorhombicVectors {d : ℕ} (l : Fin d → ℝ) : Fin d → EuclideanSpace ℝ (Fin d
    )
```

lemma orthorhombicVectors_independent [source]

The primitive vectors of an orthorhombic lattice are independent when no length vanishes.

```
lemma orthorhombicVectors_independent {d : ℕ} {l : Fin d → ℝ} (hl : ∀ i, 0 < l i) :
    LinearIndependent ℝ (orthorhombicVectors l)
```

def orthorhombicGeometry [source]

The hypercubic crystal embedded with the length `l i` along the `i`-th axis.

```
noncomputable def orthorhombicGeometry {d : ℕ} (l : Fin d → ℝ) (hl : ∀ i, 0 < l i) :
    Geometry (hypercubic d) (EuclideanSpace ℝ (Fin d))
```

def fcc [source]

One site per primitive cell of the face-centred cubic lattice.

```
@[reducible]
def fcc
```

def fccVectors [source]

Primitive vectors from a corner to the centres of the three adjacent faces of a box with sides `a`, `b`, `c`.

```
noncomputable def fccVectors (a b c : ℝ) : Fin 3 → EuclideanSpace ℝ (Fin 3)
```

lemma fccVectors_independent [source]

The primitive vectors of the face-centred lattice are independent for positive lengths.

```
lemma fccVectors_independent {a b c : ℝ} (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
    LinearIndependent ℝ (fccVectors a b c)
```

def fccGeometry [source]

The face-centred cubic lattice embedded with a box of sides `a`, `b`, `c`.

```
noncomputable def fccGeometry (a b c : ℝ) (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
    Geometry fcc (EuclideanSpace ℝ (Fin 3))
```

def fccBonds [source]

The twelve nearest neighbours, up to sign: along the three primitive vectors and along their differences.

```
def fccBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def fccGraph [source]

The twelve-neighbour face-centred cubic graph.

```
def fccGraph : PeriodicGraph fcc
```

def bcc [source]

One site per primitive cell of the body-centred cubic lattice.

```
@[reducible]
def bcc
```

def bccVectors [source]

Primitive vectors from a corner to the centres of three adjacent boxes of sides `a`, `b`, `c`.

```
noncomputable def bccVectors (a b c : ℝ) : Fin 3 → EuclideanSpace ℝ (Fin 3)
```

lemma bccVectors_independent [source]

The primitive vectors of the body-centred lattice are independent for positive lengths.

```
lemma bccVectors_independent {a b c : ℝ} (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
    LinearIndependent ℝ (bccVectors a b c)
```

def bccGeometry [source]

The body-centred cubic lattice embedded with a box of sides `a`, `b`, `c`.

```
noncomputable def bccGeometry (a b c : ℝ) (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
    Geometry bcc (EuclideanSpace ℝ (Fin 3))
```

def bccBonds [source]

The eight nearest neighbours, up to sign: along the three primitive vectors and along their sum, the body diagonal of the box.

```
def bccBonds : List (Bond (Fin 3 → ℤ) Unit)
```

def bccGraph [source]

The eight-neighbour body-centred cubic graph.

```
def bccGraph : PeriodicGraph bcc
```

inductive HcpLabel [source]

The two layers of an hcp crystal: the sites of the second layer are shifted within the plane and raised by half a period.

```
inductive HcpLabel
  | A
  | B
  deriving DecidableEq, Repr
```

instance Fintype HcpLabel [source]

```
instance : Fintype HcpLabel where
```

def hcp [source]

Two sites per primitive cell of the hexagonal close-packed crystal.

```
@[reducible]
def hcp
```

def hcpVectors [source]

Primitive vectors of the hexagonal lattice with in-plane length `a` and period `c`.

```
noncomputable def hcpVectors (a c : ℝ) : Fin 3 → EuclideanSpace ℝ (Fin 3)
```

lemma hcpVectors_independent [source]

The primitive vectors of the hexagonal lattice are independent for positive lengths.

```
lemma hcpVectors_independent {a c : ℝ} (ha : 0 < a) (hc : 0 < c) :
    LinearIndependent ℝ (hcpVectors a c)
```

def hcpBasis [source]

Positions of the two sites in the reference cell.

```
noncomputable def hcpBasis (a c : ℝ) : HcpLabel → EuclideanSpace ℝ (Fin 3)
  | .A => 0
  | .B => ![a / 2, a * Real.sqrt 3 / 6, c / 2]
```

lemma hcp_positions_injective [source]

The two layers occupy distinct positions.

```
lemma hcp_positions_injective {a c : ℝ} (ha : 0 < a) (hc : 0 < c) : Function.Injective
    (fun x : CellSite (Fin 3 → ℤ) HcpLabel =>
      latticeMap (hcpVectors a c) x.cell + hcpBasis a c x.label)
```

def hcpGeometry [source]

The hexagonal close-packed crystal with in-plane length `a` and layer period `c`.

```
noncomputable def hcpGeometry (a c : ℝ) (ha : 0 < a) (hc : 0 < c) :
    Geometry hcp (EuclideanSpace ℝ (Fin 3))
```

def hcpBonds [source]

The nearest neighbours: six in the plane of each layer, and three in each adjacent layer.

```
def hcpBonds : List (Bond (Fin 3 → ℤ) HcpLabel)
```

def hcpGraph [source]

The twelve-neighbour hexagonal close-packed graph.

```
def hcpGraph : PeriodicGraph hcp
```

lemma fcc_neighbors_card [source]

Each site of the face-centred cubic lattice has twelve neighbours.

```
lemma fcc_neighbors_card (x : fcc.Site) : (fccGraph.neighbors x).card = 12
```

lemma bcc_neighbors_card [source]

Each site of the body-centred cubic lattice has eight neighbours.

```
lemma bcc_neighbors_card (x : bcc.Site) : (bccGraph.neighbors x).card = 8
```

lemma hcp_neighbors_card [source]

Both sites of the hexagonal close-packed crystal have twelve neighbours.

```
lemma hcp_neighbors_card (x : hcp.Site) : (hcpGraph.neighbors x).card = 12
```

lemma sqrt_eq_of_eq_sq [source]

```
private lemma sqrt_eq_of_eq_sq {s L : ℝ} (hL : 0 ≤ L) (h : s = L ^ 2) : Real.sqrt s = L
```

lemma fcc_bond_length [source]

In the face-centred cubic lattice with cube side `a`, every edge has length `a √2 / 2`.

```
lemma fcc_bond_length {a : ℝ} (ha : 0 < a) {x y : fcc.Site}
    (h : fccGraph.toSimpleGraph.Adj x y) :
    (fccGeometry a a a ha ha ha).distance x y = a * Real.sqrt 2 / 2
```

lemma bcc_bond_length [source]

In the body-centred cubic lattice with cube side `a`, every edge has length `a √3 / 2`.

```
lemma bcc_bond_length {a : ℝ} (ha : 0 < a) {x y : bcc.Site}
    (h : bccGraph.toSimpleGraph.Adj x y) :
    (bccGeometry a a a ha ha ha).distance x y = a * Real.sqrt 3 / 2
```

lemma hcp_bond_length [source]

In the ideal hexagonal close-packed crystal, with `c ^ 2 = 8 a ^ 2 / 3`, every edge has length `a`.

```
lemma hcp_bond_length {a c : ℝ} (ha : 0 < a) (hc : 0 < c) (hideal : c ^ 2 = 8 * a ^ 2 / 3)
    {x y : hcp.Site} (h : hcpGraph.toSimpleGraph.Adj x y) :
    (hcpGeometry a c ha hc).distance x y = a
```

2.272 PhyslibAlpha.CondensedMatter.Crystal.Folner

PhyslibAlpha/CondensedMatter/Crystal/Folner.lean on GitHub.

def box [source]

The cube `{0, …, m-1}^d` of the lattice `ℤ^d`.

```
def box (d m : ℕ) : Finset (Fin d → ℤ)
```

lemma card_box [source]

```
lemma card_box (d m : ℕ) : (box d m).card = m ^ d
```

lemma image_add_box [source]

```
lemma image_add_box (d m : ℕ) (h : Fin d → ℤ) :
    (box d m).image (· + h) = Fintype.piFinset fun j => Finset.Ico (h j) (m + h j)
```

lemma card_image_add_box [source]

```
lemma card_image_add_box (d m : ℕ) (h : Fin d → ℤ) : ((box d m).image (· + h)).card = m ^ d
```

lemma card_inter_box [source]

The overlap of the cube with its translate.

```
lemma card_inter_box (d m : ℕ) (h : Fin d → ℤ) :
    ((box d m).image (· + h) ∩ box d m).card =
      ∏ j, (Finset.Ico (h j) (m + h j) ∩ Finset.Ico (0 : ℤ) m).card
```

lemma card_Ico_inter [source]

```
lemma card_Ico_inter (m : ℕ) (a : ℤ) (ha : |a| ≤ m) :
    (Finset.Ico a (m + a) ∩ Finset.Ico (0 : ℤ) m).card = m - a.natAbs
```

lemma card_sdiff_add_card_sdiff [source]

The number of points of the cube not shared with its translate.

```
lemma card_sdiff_add_card_sdiff (d m : ℕ) (h : Fin d → ℤ) :
    (((box d m).image (· + h) \ box d m).card + (box d m \ (box d m).image (· + h)).card : ℝ) =
      2 * ((m : ℝ) ^ d - ∏ j, ((Finset.Ico (h j) (m + h j) ∩ Finset.Ico (0 : ℤ) m).card : ℝ))
```

lemma tendsto_box_sdiff [source]

***The cubes are asymptotically invariant under translation**: the fraction of the points of `{0, …, n}^d` that are not shared with a fixed translate tends to zero.*

```
lemma tendsto_box_sdiff (d : ℕ) (h : Fin d → ℤ) :
    Tendsto (fun n : ℕ => (((box d (n + 1)).image (· + h) \ box d (n + 1)).card +
      (box d (n + 1) \ (box d (n + 1)).image (· + h)).card : ℝ) / ((n + 1 : ℕ) : ℝ) ^ d)
      atTop (𝓝 0)
```

def boxes [source]

The cubes `{0, …, n}^d`, as finite sets of the translation group.

```
def boxes (d n : ℕ) : Finset (Multiplicative (Fin d → ℤ))
```

lemma image_mul_boxes [source]

```
lemma image_mul_boxes (d n : ℕ) (h : Multiplicative (Fin d → ℤ)) :
    (boxes d n).image (· * h) =
      ((box d (n + 1)).image (· + Multiplicative.toAdd h)).map
        Multiplicative.ofAdd.toEmbedding
```

lemma isRightFolner_boxes [source]

***The cubes are a Følner family** for the translations of `ℤ^d`, along any ultrafilter on ℕ finer than the filter of large cubes.*

```
lemma isRightFolner_boxes (d : ℕ) (𝒰 : Ultrafilter ℕ) (h𝒰 : (𝒰 : Filter ℕ) ≤ atTop) :
    IsRightFolner (boxes d) 𝒰
```

2.273 PhyslibAlpha.CondensedMatter.Crystal.Geometry

PhyslibAlpha/CondensedMatter/Crystal/Geometry.lean on GitHub.

structure Geometry [source]

An embedding of a crystal into physical space, compatible with translations.

```
structure Geometry {G : Type*} [AddGroup G] (C : Crystal G) (A : Type*)
    [MetricSpace A] where
  [translations : AddAction G A]
  [isometricTranslations : IsIsometricVAdd G A]
  position : C.Site ↪ A
  position_vadd : ∀ (a : G) (x : C.Site), position (a +ᵥ x) = a +ᵥ position x
```

def distance [source]

Physical distance between two sites.

```
noncomputable def distance (R : Geometry C A) (x y : C.Site) : ℝ
```

def metricSpace [source]

The metric on crystal sites inherited from their physical positions.

```
@[instance_reducible]
noncomputable def metricSpace (R : Geometry C A) : MetricSpace C.Site
```

lemma isometry_position [source]

The position embedding preserves the induced physical metric.

```
lemma isometry_position (R : Geometry C A) :
    letI
```

def ofUnitCell [source]

Embed a repeated unit cell by translating its reference positions. The injectivity condition ensures that no two site addresses occupy the same point.

```
def ofUnitCell (G F : Type*) [AddGroup G] [Finite F] (A : Type*)
    [MetricSpace A] [AddAction G A] [IsIsometricVAdd G A]
    (basisPositions : F → A)
    (injective : Function.Injective (fun x : CellSite G F => x.cell +ᵥ basisPositions x.label))
     :
    Geometry (Crystal.ofUnitCell G F) A where
```

def ofDisplacements [source]

Embed a unit cell using physical displacements and reference positions. The translation map specifies the displacement of each cell; the injectivity condition ensures that different sites have different physical positions.

```
def ofDisplacements {G F A : Type*} [AddGroup G] [Finite F] [NormedAddCommGroup A]
    (displacement : G →+ A) (basisPositions : F → A)
    (injective : Function.Injective
      (fun x : CellSite G F => displacement x.cell + basisPositions x.label)) :
    Geometry (Crystal.ofUnitCell G F) A
```

def ofLattice [source]

A one-site unit cell embedded by an injective physical translation map.

```
def ofLattice {G A : Type*} [AddGroup G] [NormedAddCommGroup A]
    (displacement : G →+ A) (injective : Function.Injective displacement) :
    Geometry (Crystal.ofUnitCell G Unit) A
```

lemma distance_of_links [source]

Every translated copy of a bond has its reference-cell length.

```
lemma distance_of_links {G F A : Type*} [AddGroup G] [Finite F] [MetricSpace A]
    (R : Geometry (Crystal.ofUnitCell G F) A) (b : Bond G F)
    {x y : CellSite G F} (h : b.Links x y) :
    R.distance x y = R.distance ⟨0, b.source⟩ ⟨b.displacement, b.target⟩
```

lemma distance_eq_of_adj [source]

A common reference bond length gives the length of every generated graph edge.

```
lemma distance_eq_of_adj {G F A : Type*} [AddGroup G] [Finite F] [MetricSpace A]
    (R : Geometry (Crystal.ofUnitCell G F) A) (bonds : List (Bond G F)) (length : ℝ)
    (h : ∀ b ∈ bonds, R.distance ⟨0, b.source⟩ ⟨b.displacement, b.target⟩ = length)
    {x y : CellSite G F} (hxy : (graphFromBonds bonds).Adj x y) : R.distance x y = length
```

def latticeMap [source]

Convert integer cell coordinates into physical displacements along primitive vectors.

```
noncomputable def latticeMap {d D : ℕ} (vectors : Fin d → EuclideanSpace ℝ (Fin D)) :
    (Fin d → ℤ) →+ EuclideanSpace ℝ (Fin D) where
```

lemma latticeMap_injective [source]

Independent primitive vectors separate the displacements of different cells.

```
lemma latticeMap_injective {d D : ℕ} {vectors : Fin d → EuclideanSpace ℝ (Fin D)}
    (independent : LinearIndependent ℝ vectors) : Function.Injective (latticeMap vectors)
```

2.274 PhyslibAlpha.CondensedMatter.Crystal.Graph

PhyslibAlpha/CondensedMatter/Crystal/Graph.lean on GitHub.

def SimpleGraph.bondCountBetween [source]

Number of connections from sites in Λ to sites in M. For disjoint regions, each bond between them is counted once; with $M = \Lambda$, each internal bond is counted twice.

```
def bondCountBetween (g : SimpleGraph X) [DecidableRel g.Adj] (Λ M : Finset X) : ℕ
```

lemma bondCountBetween_le [source]

A bound on coordination numbers bounds the number of connections from a finite region.

```
lemma bondCountBetween_le (g : SimpleGraph X) [DecidableRel g.Adj] [g.LocallyFinite]
    (Λ M : Finset X) (q : ℕ)
    (hq : ∀ x ∈ Λ, (g.neighborFinset x).card ≤ q) :
    g.bondCountBetween Λ M ≤ q * Λ.card
```

structure PeriodicGraph [source]

A periodic pattern of undirected connections between crystal sites, with finitely many neighbors at each site.

```
structure PeriodicGraph {G : Type*} [AddGroup G] (C : Crystal G) where
  toSimpleGraph : SimpleGraph C.Site
  adj_vadd : ∀ (a : G) (x y : C.Site),
    toSimpleGraph.Adj (a +ᵥ x) (a +ᵥ y) ↔ toSimpleGraph.Adj x y
  [decidableAdj : DecidableRel toSimpleGraph.Adj]
  [locallyFinite : toSimpleGraph.LocallyFinite]
```

def connected [source]

Check whether two sites are connected by a crystal bond.

```
def connected (g : PeriodicGraph C) (x y : C.Site) : Bool
```

def neighbors [source]

The finite set of sites connected to a given site.

```
def neighbors (g : PeriodicGraph C) (x : C.Site) : Finset C.Site
```

def translateIso [source]

A cell translation is an automorphism of the bond graph.

```
def translateIso (g : PeriodicGraph C) (a : G) : g.toSimpleGraph ≃g g.toSimpleGraph where
```

lemma neighbors_vadd [source]

Translating a site translates all of its graph neighbors.

```
lemma neighbors_vadd (g : PeriodicGraph C) (a : G) (x : C.Site) :
    g.neighbors (a +ᵥ x) = (g.neighbors x).map (AddAction.toPerm a).toEmbedding
```

lemma card_neighbors_vadd [source]

The coordination number is constant along translated copies of a site.

```
lemma card_neighbors_vadd (g : PeriodicGraph C) (a : G) (x : C.Site) :
    (g.neighbors (a +ᵥ x)).card = (g.neighbors x).card
```

def PeriodicGraph.bondCountBetween [source]

Count the crystal bonds between finite regions, using its ordinary graph.

```
def bondCountBetween (g : PeriodicGraph C) (Λ M : Finset C.Site) : ℕ
```

structure Bond [source]

A bond from label `source` in a reference cell to label `target` in the cell indexed by `displacement`. Translating both endpoints gives the copies of this bond in other cells.

```
structure Bond (G F : Type*) where
  source : F
  target : F
  displacement : G
  deriving DecidableEq
```

def Links [source]

Site `x` is the source and site `y` is the target of a copy of this bond.

```
def Links (b : Bond G F) (x y : CellSite G F) : Prop
```

instance DecidableRel b.Links [source]

The source and target of a bond can be checked from their site addresses.

```
instance [DecidableEq G] [DecidableEq F] (b : Bond G F) : DecidableRel b.Links
```

def endpoints [source]

The sites reached by following this bond in either direction.

```
def endpoints [DecidableEq G] [DecidableEq F] (b : Bond G F) (x : CellSite G F) :
    Finset (CellSite G F)
```

lemma mem_endpoints [source]

A site reached through this bond is connected in one of its two directions.

```
lemma mem_endpoints [DecidableEq G] [DecidableEq F] (b : Bond G F) (x y : CellSite G F) :
    y ∈ b.endpoints x ↔ b.Links x y ∨ b.Links y x
```

lemma links_vadd [source]

Translating the endpoints gives another copy of the same bond.

```
lemma links_vadd (b : Bond G F) (a : G) (x y : CellSite G F) :
    b.Links (a +ᵥ x) (a +ᵥ y) ↔ b.Links x y
```

def graphFromBonds [source]

The undirected graph generated by translating every supplied bond.

```
def graphFromBonds {G F : Type*} [AddGroup G] (bonds : List (Bond G F)) :
    SimpleGraph (CellSite G F) where
```

instance DecidableRel (graphFromBonds bonds).Adj [source]

Connections generated by a finite bond list can be decided.

```
instance {G F : Type*} [AddGroup G] [DecidableEq G] [DecidableEq F]
    (bonds : List (Bond G F)) : DecidableRel (graphFromBonds bonds).Adj
```

def bondEndpoints [source]

The sites reached from `x` by following any of the listed bonds in either direction.

```
def bondEndpoints {G F : Type*} [AddGroup G] [DecidableEq G] [DecidableEq F]
    (bonds : List (Bond G F)) (x : CellSite G F) : Finset (CellSite G F)
```

lemma mem_bondEndpoints [source]

Every reached site is an endpoint of one of the listed bonds.

```
lemma mem_bondEndpoints {G F : Type*} [AddGroup G] [DecidableEq G] [DecidableEq F]
    (bonds : List (Bond G F)) (x y : CellSite G F) :
    y ∈ bondEndpoints bonds x ↔ ∃ b ∈ bonds, b.Links x y ∨ b.Links y x
```

def neighborsFromBonds [source]

The distinct neighboring sites connected by the listed bonds, excluding `x` itself.

```
def neighborsFromBonds {G F : Type*} [AddGroup G] [DecidableEq G] [DecidableEq F]
    (bonds : List (Bond G F)) (x : CellSite G F) : Finset (CellSite G F)
```

def ofBonds [source]

Construct the periodic graph of a repeated unit cell from its bonds. Each bond connects its source label to its target label at the specified cell displacement, in every cell. Connections are undirected; repeated connections and self-loops do not add neighbors.

```
def PeriodicGraph.ofBonds {G F : Type*} [AddGroup G] [Finite F]
    [DecidableEq G] [DecidableEq F] (bonds : List (Bond G F)) :
    PeriodicGraph (Crystal.ofUnitCell G F) where
```

lemma neighbors_ofBonds [source]

The neighbors in the graph of a bond list are the endpoints of its bonds, without the site itself.

```
lemma PeriodicGraph.neighbors_ofBonds {G F : Type*} [AddGroup G] [Finite F]
    [DecidableEq G] [DecidableEq F] (bonds : List (Bond G F)) (x : CellSite G F) :
    (PeriodicGraph.ofBonds bonds).neighbors x = neighborsFromBonds bonds x
```

lemma card_neighbors_cell [source]

The coordination number of a cell site equals that of its reference-cell copy.

```
lemma PeriodicGraph.card_neighbors_cell {G F : Type*} [AddGroup G] [Finite F]
    (g : PeriodicGraph (Crystal.ofUnitCell G F)) (n : G) (i : F) :
    (g.neighbors ⟨n, i⟩).card = (g.neighbors ⟨0, i⟩).card
```

2.275 PhyslibAlpha.CondensedMatter.Crystal.InvariantState

PhyslibAlpha/CondensedMatter/Crystal/InvariantState.lean on GitHub.

def cubeUltrafilter [source]

An ultrafilter on the sizes of the cubes, finer than the filter of large cubes.

```
def cubeUltrafilter : Ultrafilter ℕ
```

lemma cubeUltrafilter_le [source]

```
lemma cubeUltrafilter_le : (cubeUltrafilter : Filter ℕ) ≤ atTop
```

def translationInvariantLimit [source]

***The translation invariant limit of a state of the crystal `ℤ^d`**: the limit of its averages over the translations by the points of the cubes.*

```
def translationInvariantLimit (φ : AlgState (crystalAlgebra Ω₀ (Fin d → ℤ) F)) :
    AlgState (crystalAlgebra Ω₀ (Fin d → ℤ) F)
```

lemma translationInvariantLimit_invariant [source]

The translation invariant limit of a state is invariant under the translations.

```
lemma translationInvariantLimit_invariant (φ : AlgState (crystalAlgebra Ω₀ (Fin d → ℤ) F))
    (a : Fin d → ℤ) (x : crystalAlgebra Ω₀ (Fin d → ℤ) F) :
    translationInvariantLimit Ω₀ φ (translationAction Ω₀ (Multiplicative.ofAdd a) x) =
      translationInvariantLimit Ω₀ φ x
```

def translationInvariantState [source]

***The translation invariant state of the canonical ensemble** of an interaction on the crystal ℤ^d: the translation average of a thermodynamic limit state of the finite boxes.*

```
def _root_.CondensedMatter.QuasiLocal.Interaction.translationInvariantState [Nonempty Ω₀]
    (I : Interaction (fun _ : CellSite (Fin d → ℤ) F => Ω₀)) (β : ℝ) :
    AlgState (crystalAlgebra Ω₀ (Fin d → ℤ) F)
```

lemma translationInvariantState_invariant [source]

The translation invariant state of an interaction is invariant under the translations.

```
lemma translationInvariantState_invariant [Nonempty Ω₀]
    (I : Interaction (fun _ : CellSite (Fin d → ℤ) F => Ω₀)) (β : ℝ) (a : Fin d → ℤ)
    (x : crystalAlgebra Ω₀ (Fin d → ℤ) F) :
    I.translationInvariantState β (translationAction Ω₀ (Multiplicative.ofAdd a) x) =
      I.translationInvariantState β x
```

2.276 PhyslibAlpha.CondensedMatter.Crystal.InvariantStateUncertainty

PhyslibAlpha/CondensedMatter/Crystal/InvariantStateUncertainty.lean on GitHub.

lemma robertson_det_translationInvariantState [source]

***Robertson's relation for several observables in the translation invariant state** of an interaction on the crystal ℤ^d.*

```
lemma robertson_det_translationInvariantState {ι : Type*} [Fintype ι] [DecidableEq ι]
    (a : ι → selfAdjoint (crystalAlgebra Ω₀ (Fin d → ℤ) F)) :
    |((I.translationInvariantState β).bracketMatrix a).det| ≤
      ((I.translationInvariantState β).covarianceMatrix a).det
```

lemma covariance_translationInvariantState_eq_zero [source]

In the translation invariant state, an observable fixed and one reversed by a translation are uncorrelated.

```
lemma covariance_translationInvariantState_eq_zero :
    (I.translationInvariantState β).covariance A B = 0
```

lemma bracket_translationInvariantState_eq_zero [source]

In the translation invariant state, an observable fixed and one reversed by a translation have a bracket of expectation zero.

```
lemma bracket_translationInvariantState_eq_zero :
    (I.translationInvariantState β).bracket A B = 0
```

2.277 PhyslibAlpha.CondensedMatter.Crystal.ManyBody

PhyslibAlpha/CondensedMatter/Crystal/ManyBody.lean on GitHub.

def manyBodyRep [source]

***The unitary representation of the symmetries on the GNS space** of an invariant state.*

```
def manyBodyRep : H →* unitary (ω.GNS →L[ℂ] ω.GNS)
```

def translationIn [source]

The translations as elements of the symmetry subgroup.

```
def translationIn : Multiplicative G →* H
```

def manyBodyTranslation [source]

The translation representation on the GNS space.

```
def manyBodyTranslation : Multiplicative G →* unitary (ω.GNS →L[ℂ] ω.GNS)
```

lemma manyBodyTranslation_apply [source]

```
lemma manyBodyTranslation_apply (a : Multiplicative G) :
    manyBodyTranslation H ω hω htrans a = manyBodyRep H ω hω (translationIn H htrans a)
```

def linearAut [source]

The action of a symmetry on the lattice of translations, through its linear part.

```
def linearAut : H →* MulAut (Multiplicative G) where
```

lemma manyBodyRep_covariant [source]

The translations and the symmetries are covariant on the GNS space.

```
lemma manyBodyRep_covariant :
    UnitaryRep.IsCovariantRep (linearAut H) (manyBodyTranslation H ω hω htrans)
      (manyBodyRep H ω hω)
```

lemma commute_manyBodyRep [source]

A dynamics commuting with a symmetry is implemented by an operator commuting with the unitary implementing the symmetry.

```
lemma commute_manyBodyRep (s : H) :
    Commute ((manyBodyRep H ω hω s : unitary (ω.GNS →L[ℂ] ω.GNS)) : ω.GNS →L[ℂ] ω.GNS)
      (AlgState.gnsAut τ hτ)
```

lemma commute_manyBodyTranslation [source]

A dynamics commuting with the symmetries commutes with the translation representation.

```
lemma commute_manyBodyTranslation (a : Multiplicative G) :
    Commute ((manyBodyTranslation H ω hω htrans a : unitary (ω.GNS →L[ℂ] ω.GNS)) :
      ω.GNS →L[ℂ] ω.GNS) (AlgState.gnsAut τ hτ)
```

lemma spectrum_sectorOp_eq_manyBody [source]

***Symmetry reduction of a many-body dynamics on an infinite crystal.** Let `ω` be a state of the quasi-local algebra invariant under a group `H` of cell symmetries containing the translations, and `τ` an automorphism preserving `ω` and commuting with `H`: a time evolution. Then the unitary implementing `τ` is reduced by the sectors of total momentum, and its restriction to the sector of the momenta in a set `S` has the same spectrum as its restriction to the sector of the momenta in the image of `S` under any symmetry `s` in `H`.*

```
lemma spectrum_sectorOp_eq_manyBody (s : H)
    {S : Set (PontryaginDual (Multiplicative G))} (hS : MeasurableSet S) :
    spectrum ℂ (UnitaryRep.sectorOp
        (commute_manyBodyTranslation H ω hω htrans τ hτ hcomm) S) =
      spectrum ℂ (UnitaryRep.sectorOp
        (commute_manyBodyTranslation H ω hω htrans τ hτ hcomm)
        (UnitaryRep.dualAct (linearAut H) s '' S))
```

2.278 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Basic

PhyslibAlpha/CondensedMatter/Crystal/PointGroup/Basic.lean on GitHub.

def pointStabilizer [source]

The lattice automorphisms that map bonded displacements to bonded displacements.

```
def pointStabilizer (g : PeriodicGraph (Crystal.ofUnitCell G Unit)) :
    Subgroup (Multiplicative (AddAut G)) where
```

def linearHom [source]

The linear part of a cell symmetry as a homomorphism to the lattice automorphisms.

```
def linearHom (G F : Type*) [AddCommGroup G] : CellSymmetry G F →* Multiplicative (AddAut G)
    where
```

lemma linear_mem_pointStabilizer [source]

Bond-preserving symmetries have linear parts in the point stabilizer.

```
lemma linear_mem_pointStabilizer (g : PeriodicGraph (Crystal.ofUnitCell G Unit))
    {s : CellSymmetry G Unit} (hs : s ∈ g.symmetries) :
    Multiplicative.ofAdd s.linear ∈ g.pointStabilizer
```

def pointHom [source]

The homomorphism from bond-preserving symmetries to the point stabilizer taking a symmetry to its action on the lattice.

```
def pointHom (g : PeriodicGraph (Crystal.ofUnitCell G Unit)) :
    g.symmetries →* g.pointStabilizer
```

lemma pointHom_surjective [source]

```
lemma pointHom_surjective (g : PeriodicGraph (Crystal.ofUnitCell G Unit)) :
    Function.Surjective (pointHom g)
```

lemma ker_pointHom [source]

```
lemma ker_pointHom (g : PeriodicGraph (Crystal.ofUnitCell G Unit)) :
    (pointHom g).ker = (CellSymmetry.translations G Unit).subgroupOf g.symmetries
```

def pointGroupEquiv [source]

The point group of a crystal with one site per cell is the group of lattice automorphisms preserving the bonded displacements.

```
noncomputable def pointGroupEquiv (g : PeriodicGraph (Crystal.ofUnitCell G Unit)) :
    g.pointGroup ≃* g.pointStabilizer
```

def pointLinearHom [source]

***The point group of any crystal acts on its translation lattice** through the linear parts of its symmetries.*

```
noncomputable def pointLinearHom {F : Type*} [Finite F]
    (g : PeriodicGraph (Crystal.ofUnitCell G F)) : g.pointGroup →* Multiplicative (AddAut G)
```

2.279 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Count

PhyslibAlpha/CondensedMatter/Crystal/PointGroup/Count.lean on GitHub.

def act [source]

The linear map of $\mathbb{Z}^n$ taking the basis vectors to the given vectors.

```
def act (v : Fin n → Fin n → ℤ) : (Fin n → ℤ) →ₗ[ℤ] (Fin n → ℤ)
```

lemma act_apply [source]

```
lemma act_apply (v : Fin n → Fin n → ℤ) (d : Fin n → ℤ) : act v d = ∑ j, d j • v j
```

lemma act_single [source]

```
lemma act_single (v : Fin n → Fin n → ℤ) (j : Fin n) : act v (Pi.single j 1) = v j
```

lemma apply_eq_act [source]

A lattice automorphism is determined by the images of the basis vectors.

```
lemma apply_eq_act (A : AddAut (Fin n → ℤ)) (d : Fin n → ℤ) :
    A d = act (fun j => A (Pi.single j 1)) d
```

structure Admissible [source]

A tuple of neighbour displacements is admissible when its linear map takes the set of neighbour displacements into itself, injectively.

```
structure Admissible (N : Finset (Fin n → ℤ)) (v : Fin n → Fin n → ℤ) : Prop where
  mem : ∀ j, v j ∈ N
  mapsTo : ∀ d ∈ N, act v d ∈ N
  injOn : Set.InjOn (act v) N
```

lemma image_eq [source]

An admissible tuple maps the set of neighbour displacements onto itself.

```
lemma Admissible.image_eq {v : Fin n → Fin n → ℤ} (hv : Admissible N v) :
    N.image (act v) = N
```

lemma surjective [source]

An admissible tuple gives a surjective linear map.

```
lemma Admissible.surjective {v : Fin n → Fin n → ℤ} (hv : Admissible N v) :
    Function.Surjective (act v)
```

lemma injective [source]

An admissible tuple gives an injective linear map.

```
lemma Admissible.injective {v : Fin n → Fin n → ℤ} (hv : Admissible N v) :
    Function.Injective (act v)
```

lemma act_mem_iff [source]

A displacement is a neighbour exactly when its image under an admissible tuple is.

```
lemma Admissible.act_mem_iff {v : Fin n → Fin n → ℤ} (hv : Admissible N v) (d : Fin n → ℤ) :
    act v d ∈ N ↔ d ∈ N
```

def equiv [source]

The lattice automorphism of an admissible tuple.

```
noncomputable def Admissible.equiv {v : Fin n → Fin n → ℤ} (hv : Admissible N v) :
    AddAut (Fin n → ℤ)
```

lemma equiv_apply [source]

```
lemma Admissible.equiv_apply {v : Fin n → Fin n → ℤ} (hv : Admissible N v) (d : Fin n → ℤ) :
    hv.equiv hN d = act v d
```

def pointStabilizerEquiv [source]

The point group of a Bravais lattice is in bijection with the admissible tuples of neighbour displacements.

```
noncomputable def pointStabilizerEquiv : g.pointStabilizer ≃ {v // Admissible N v} where
```

lemma card_pointGroup [source]

***The order of the point group** is the number of admissible tuples of neighbour displacements.*

```
lemma card_pointGroup : Nat.card g.pointGroup = Nat.card {v // Admissible N v}
```

2.280 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Cubic

PhyslibAlpha/CondensedMatter/Crystal/PointGroup/Cubic.lean on GitHub.

def bondNeighbors [source]

The neighbour displacements of a bond list on a one-site cell: the bond displacements and their negatives, without zero.

```
def bondNeighbors (bonds : List (Bond (Fin n → ℤ) Unit)) : Finset (Fin n → ℤ)
```

lemma mem_bondNeighbors [source]

```
lemma mem_bondNeighbors (bonds : List (Bond (Fin n → ℤ) Unit)) (d : Fin n → ℤ) :
    d ∈ bondNeighbors bonds ↔ d ≠ 0 ∧ ∃ b ∈ bonds, d = b.displacement ∨ d = -b.displacement
```

lemma bonded_ofBonds_unit [source]

The bonded displacements of a graph generated by bonds on a one-site cell.

```
lemma bonded_ofBonds_unit (bonds : List (Bond (Fin n → ℤ) Unit)) (d : Fin n → ℤ) :
    (PeriodicGraph.ofBonds bonds).Bonded () () d ↔ d ∈ bondNeighbors bonds
```

def admissibleEquiv [source]

The admissible triples can be counted among triples of neighbour displacements.

```
def admissibleEquiv (N : Finset (Fin n → ℤ)) :
    {v // Admissible N v} ≃ {w : Fin n → ↥N // Admissible N (fun j => (w j : Fin n → ℤ))} where
```

instance Decidable (Admissible N v) [source]

```
instance (N : Finset (Fin n → ℤ)) (v : Fin n → Fin n → ℤ) : Decidable (Admissible N v)
```

lemma fcc_card_pointGroup [source]

The point group of the face-centred cubic lattice has order `48`.

```
lemma fcc_card_pointGroup : Nat.card fccGraph.pointGroup = 48
```

lemma bcc_card_pointGroup [source]

The point group of the body-centred cubic lattice has order `48`.

```
lemma bcc_card_pointGroup : Nat.card bccGraph.pointGroup = 48
```

2.281 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Hypercubic

PhyslibAlpha/CondensedMatter/Crystal/PointGroup/Hypercubic.lean on GitHub.

def l1 [source]

The sum of the absolute values of the coordinates.

```
def l1 (r : Fin d → ℤ) : ℤ
```

lemma l1_neg [source]

```
lemma l1_neg (r : Fin d → ℤ) : l1 (-r) = l1 r
```

lemma l1_single [source]

```
lemma l1_single (j : Fin d) : l1 (Pi.single j (1 : ℤ) : Fin d → ℤ) = 1
```

lemma hypercubic_bonded_iff_exists [source]

The bonded displacements of the hypercubic lattice are the vectors `±e_j`.

```
lemma hypercubic_bonded_iff_exists (r : Fin d → ℤ) :
    (hypercubicGraph d).Bonded () () r ↔ ∃ j, r = Pi.single j 1 ∨ r = -Pi.single j 1
```

lemma hypercubic_bonded_iff [source]

Two sites of the hypercubic lattice are bonded exactly when their cells differ by a vector whose coordinates have absolute values summing to one.

```
lemma hypercubic_bonded_iff (r : Fin d → ℤ) :
    (hypercubicGraph d).Bonded () () r ↔ l1 r = 1
```

def signOf [source]

The sign `+1` for `true` and `-1` for `false`.

```
def signOf (b : Bool) : ℤ
```

lemma signOf_mul_self [source]

```
lemma signOf_mul_self (b : Bool) : signOf b * signOf b = 1
```

lemma abs_signOf [source]

```
lemma abs_signOf (b : Bool) : |signOf b| = 1
```

lemma signOf_ne_zero [source]

```
lemma signOf_ne_zero (b : Bool) : signOf b ≠ 0
```

lemma signOf_injective [source]

```
lemma signOf_injective : Function.Injective signOf
```

def signedPerm [source]

The lattice automorphism that permutes the axes by `π` and multiplies the `i`-th coordinate by the sign `ε i`.

```
def signedPerm (π : Equiv.Perm (Fin d)) (ε : Fin d → Bool) : (Fin d → ℤ) ≃+ (Fin d → ℤ) where
```

lemma signedPerm_apply [source]

```
lemma signedPerm_apply (π : Equiv.Perm (Fin d)) (ε : Fin d → Bool) (r : Fin d → ℤ) (i : Fin d)
    :
    signedPerm π ε r i = signOf (ε i) * r (π.symm i)
```

lemma l1_signedPerm [source]

A signed permutation preserves the sum of the absolute values of the coordinates.

```
lemma l1_signedPerm (π : Equiv.Perm (Fin d)) (ε : Fin d → Bool) (r : Fin d → ℤ) :
    l1 (signedPerm π ε r) = l1 r
```

lemma signedPerm_single [source]

```
lemma signedPerm_single (π : Equiv.Perm (Fin d)) (ε : Fin d → Bool) (j : Fin d) :
    signedPerm π ε (Pi.single j 1) = Pi.single (π j) (signOf (ε (π j)))
```

lemma signedPerm_injective [source]

```
lemma signedPerm_injective :
    Function.Injective fun p : Equiv.Perm (Fin d) × (Fin d → Bool) => signedPerm p.1 p.2
```

lemma signedPerm_mem_pointStabilizer [source]

A signed permutation preserves the bonds of the hypercubic lattice.

```
lemma signedPerm_mem_pointStabilizer (π : Equiv.Perm (Fin d)) (ε : Fin d → Bool) :
    Multiplicative.ofAdd (signedPerm π ε) ∈ (hypercubicGraph d).pointStabilizer
```

lemma mem_pointStabilizer_hypercubic_iff [source]

The lattice automorphisms preserving the bonds of the hypercubic lattice are exactly the signed permutations.

```
lemma mem_pointStabilizer_hypercubic_iff (A : Multiplicative (AddAut (Fin d → ℤ))) :
    A ∈ (hypercubicGraph d).pointStabilizer ↔
      ∃ (π : Equiv.Perm (Fin d)) (ε : Fin d → Bool), Multiplicative.toAdd A = signedPerm π ε
```

def pointStabilizerEquivSignedPerm [source]

***The point group of the hypercubic lattice is the group of signed permutations of the axes**: it is in bijection with the pairs of a permutation and a choice of signs.*

```
noncomputable def pointStabilizerEquivSignedPerm :
    (Equiv.Perm (Fin d) × (Fin d → Bool)) ≃ (hypercubicGraph d).pointStabilizer
```

lemma card_pointGroup_hypercubic [source]

The point group of the hypercubic lattice in `d` dimensions has order `2 ^ d * d !`.

```
lemma card_pointGroup_hypercubic (d : ℕ) :
    Nat.card (hypercubicGraph d).pointGroup = 2 ^ d * d.factorial
```

2.282 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Monoclinic

PhyslibAlpha/CondensedMatter/Crystal/PointGroup/Monoclinic.lean on GitHub.

def monoclinicPointTriples [source]

The four point operations of the primitive monoclinic lattice: the identity, the reflection `b ↦ -b`, the rotation by `π` about `b` and the inversion.

```
def monoclinicPointTriples : Finset (Fin 3 → Fin 3 → ℤ)
```

lemma monoclinic_admissible_iff [source]

The point operations of the primitive monoclinic lattice are the four of `monoclinicPointTriples`.

```
lemma monoclinic_admissible_iff (v : Fin 3 → Fin 3 → ℤ) :
    Admissible (bondNeighbors monoclinicBonds) v ↔ v ∈ monoclinicPointTriples
```

def baseCentredMonoclinicPointTriples [source]

The four point operations of the base-centred monoclinic lattice, in the basis `baseCentredMonoclinicVectors`

```
def baseCentredMonoclinicPointTriples : Finset (Fin 3 → Fin 3 → ℤ)
```

lemma baseCentredMonoclinic_admissible_iff [source]

The point operations of the base-centred monoclinic lattice are the four of `baseCentredMonoclinicPointTripl`

```
lemma baseCentredMonoclinic_admissible_iff (v : Fin 3 → Fin 3 → ℤ) :
    Admissible (bondNeighbors baseCentredMonoclinicBonds) v ↔
      v ∈ baseCentredMonoclinicPointTriples
```

def triclinicPointTriples [source]

The two point operations of the triclinic lattice: the identity and the inversion.

```
def triclinicPointTriples : Finset (Fin 3 → Fin 3 → ℤ)
```

lemma triclinic_admissible_iff [source]

The point operations of the triclinic lattice are the identity and the inversion.

```
lemma triclinic_admissible_iff (v : Fin 3 → Fin 3 → ℤ) :
    Admissible (bondNeighbors triclinicBonds) v ↔ v ∈ triclinicPointTriples
```

lemma monoclinic_card_pointGroup [source]

The point group of the primitive monoclinic lattice has order `4`.

```
lemma monoclinic_card_pointGroup : Nat.card monoclinicGraph.pointGroup = 4
```

lemma baseCentredMonoclinic_card_pointGroup [source]

The point group of the base-centred monoclinic lattice has order `4`.

```
lemma baseCentredMonoclinic_card_pointGroup :
    Nat.card baseCentredMonoclinicGraph.pointGroup = 4
```

lemma triclinic_card_pointGroup [source]

The point group of the triclinic lattice has order `2`.

```
lemma triclinic_card_pointGroup : Nat.card triclinicGraph.pointGroup = 2
```

2.283 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Orthorhombic

PhyslibAlpha/CondensedMatter/Crystal/PointGroup/Orthorhombic.lean on GitHub.

def PassesTests [source]

The triple `(a, b, c)` passes the tests `D₀`, `D₁`, `D₂` on its first, first two and three vectors: each test `d` and its image under the triple are neighbours together.

```
def PassesTests (N : Finset (Fin 3 → ℤ)) (D₀ : List ℤ) (D₁ : List (ℤ × ℤ))
    (D₂ : List (Fin 3 → ℤ)) (a b c : Fin 3 → ℤ) : Prop
```

instance Decidable (PassesTests N D₀ D₁ D₂ a b c) [source]

```
instance (N : Finset (Fin 3 → ℤ)) (D₀ : List ℤ) (D₁ : List (ℤ × ℤ)) (D₂ : List (Fin 3 → ℤ))
    (a b c : Fin 3 → ℤ) : Decidable (PassesTests N D₀ D₁ D₂ a b c)
```

lemma admissible_iff_of_tests [source]

The admissible triples are the triples of `S`, if every triple of `S` is admissible and only the triples of `S` pass the tests.

```
lemma admissible_iff_of_tests (hN : ∀ j, (Pi.single j 1 : Fin 3 → ℤ) ∈ N)
    {S : Finset (Fin 3 → Fin 3 → ℤ)} (D₀ : List ℤ) (D₁ : List (ℤ × ℤ))
    (D₂ : List (Fin 3 → ℤ)) (hS : ∀ v ∈ S, Admissible N v)
    (hD : ∀ a ∈ N, ∀ b ∈ N, ∀ c ∈ N, PassesTests N D₀ D₁ D₂ a b c → ![a, b, c] ∈ S)
    (v : Fin 3 → Fin 3 → ℤ) : Admissible N v ↔ v ∈ S
```

lemma card_pointGroup_ofBonds [source]

The order of the point group is the number of admissible triples, once they are known.

```
lemma card_pointGroup_ofBonds (bonds : List (Bond (Fin 3 → ℤ) Unit))
    (hN : ∀ j, (Pi.single j 1 : Fin 3 → ℤ) ∈ bondNeighbors bonds)
    {S : Finset (Fin 3 → Fin 3 → ℤ)} (h : ∀ v, Admissible (bondNeighbors bonds) v ↔ v ∈ S) :
    Nat.card (ofBonds bonds).pointGroup = S.card
```

def orthorhombicPointTriples [source]

The images of the basis vectors under the eight point operations of the primitive orthorhombic lattice, the sign changes of the three basis vectors.

```
def orthorhombicPointTriples : Finset (Fin 3 → Fin 3 → ℤ)
```

lemma orthorhombic_admissible_iff [source]

The point operations of the primitive orthorhombic lattice are the eight of `orthorhombicPointTriples`.

```
lemma orthorhombic_admissible_iff (v : Fin 3 → Fin 3 → ℤ) :
    Admissible (bondNeighbors orthorhombicBonds) v ↔ v ∈ orthorhombicPointTriples
```

def baseCentredOrthorhombicPointTriples [source]

The images of the basis vectors under the eight point operations of the base-centred orthorhombic lattice, in the basis `baseCentredVectors a b c`.

```
def baseCentredOrthorhombicPointTriples : Finset (Fin 3 → Fin 3 → ℤ)
```

lemma baseCentredOrthorhombic_admissible_iff [source]

The point operations of the base-centred orthorhombic lattice are the eight of `baseCentredOrthorhombicPoint`

```
lemma baseCentredOrthorhombic_admissible_iff (v : Fin 3 → Fin 3 → ℤ) :
    Admissible (bondNeighbors baseCentredOrthorhombicBonds) v ↔
      v ∈ baseCentredOrthorhombicPointTriples
```

def bodyCentredOrthorhombicPointTriples [source]

The images of the basis vectors under the eight point operations of the body-centred orthorhombic lattice, in the basis `bccVectors a b c`.

```
def bodyCentredOrthorhombicPointTriples : Finset (Fin 3 → Fin 3 → ℤ)
```

lemma bodyCentredOrthorhombic_admissible_iff [source]

The point operations of the body-centred orthorhombic lattice are the eight of `bodyCentredOrthorhombicPoint`

```
lemma bodyCentredOrthorhombic_admissible_iff (v : Fin 3 → Fin 3 → ℤ) :
    Admissible (bondNeighbors bodyCentredOrthorhombicBonds) v ↔
      v ∈ bodyCentredOrthorhombicPointTriples
```

def faceCentredOrthorhombicPointTriples [source]

The images of the basis vectors under the eight point operations of the face-centred orthorhombic lattice, in the basis `fccVectors a b c`.

```
def faceCentredOrthorhombicPointTriples : Finset (Fin 3 → Fin 3 → ℤ)
```

lemma faceCentredOrthorhombic_admissible_iff [source]

The point operations of the face-centred orthorhombic lattice are the eight of `faceCentredOrthorhombicPoint`

```
lemma faceCentredOrthorhombic_admissible_iff (v : Fin 3 → Fin 3 → ℤ) :
    Admissible (bondNeighbors faceCentredOrthorhombicBonds) v ↔
      v ∈ faceCentredOrthorhombicPointTriples
```

lemma orthorhombic_card_pointGroup [source]

***The point group of the primitive orthorhombic lattice has order* `8`.**

```
lemma orthorhombic_card_pointGroup : Nat.card orthorhombicGraph.pointGroup = 8
```

lemma baseCentredOrthorhombic_card_pointGroup [source]

***The point group of the base-centred orthorhombic lattice has order* `8`.**

```
lemma baseCentredOrthorhombic_card_pointGroup :
    Nat.card baseCentredOrthorhombicGraph.pointGroup = 8
```

lemma bodyCentredOrthorhombic_card_pointGroup [source]

***The point group of the body-centred orthorhombic lattice has order* `8`.**

```
lemma bodyCentredOrthorhombic_card_pointGroup :
    Nat.card bodyCentredOrthorhombicGraph.pointGroup = 8
```

lemma faceCentredOrthorhombic_card_pointGroup [source]

***The point group of the face-centred orthorhombic lattice has order* `8`.**

```
lemma faceCentredOrthorhombic_card_pointGroup :
    Nat.card faceCentredOrthorhombicGraph.pointGroup = 8
```

2.284 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Rhombohedral

PhyslibAlpha/CondensedMatter/Crystal/PointGroup/Rhombohedral.lean on GitHub.

def rhombohedralPointTriples [source]

The images of the basis vectors under the twelve point operations of the rhombohedral lattice: the permutations of the primitive vectors, with or without the inversion.

```
def rhombohedralPointTriples : Finset (Fin 3 → Fin 3 → ℤ)
```

lemma rhombohedral_admissible_iff [source]

The point operations of the rhombohedral lattice are the twelve of `rhombohedralPointTriples`.

```
lemma rhombohedral_admissible_iff (v : Fin 3 → Fin 3 → ℤ) :
    Admissible (bondNeighbors rhombohedralBonds) v ↔ v ∈ rhombohedralPointTriples
```

lemma rhombohedral_card_pointGroup [source]

The point group of the rhombohedral lattice has order `12`.

```
lemma rhombohedral_card_pointGroup : Nat.card rhombohedralGraph.pointGroup = 12
```

2.285 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.Sublattices

PhyslibAlpha/CondensedMatter/Crystal/PointGroup/Sublattices.lean on GitHub.

def bondDisplacements [source]

The displacements `d` such that site `i` of the reference cell is bonded to site `j` of the cell `d`, in the graph of a bond list.

```
def bondDisplacements (bonds : List (Bond G F)) (i j : F) : Finset G
```

lemma bonded_ofBonds_iff [source]

```
lemma bonded_ofBonds_iff (bonds : List (Bond G F)) (i j : F) (d : G) :
    (ofBonds bonds).Bonded i j d ↔ d ∈ bondDisplacements bonds i j
```

lemma mem_symmetries_ofBonds_iff [source]

***A finite criterion for the symmetries of the graph of a bond list**: `(A, σ, b)` is a symmetry exactly when `d ↦ A d + b j - b i` maps `E i j` onto `E (σ i) (σ j)` for all labels.*

```
lemma mem_symmetries_ofBonds_iff (bonds : List (Bond G F)) (s : CellSymmetry G F) :
    s ∈ (ofBonds bonds).symmetries ↔ ∀ i j, (bondDisplacements bonds i j).image
      (fun d => s.linear d + s.shift j - s.shift i) =
        bondDisplacements bonds (s.perm i) (s.perm j)
```

def bondDifferences [source]

The nonzero differences `d₁ - d₂` of two displacements in a common `E i j`: translations between two sites with the same label that are bonded to a common site.

```
def bondDifferences (bonds : List (Bond G F)) : Finset G
```

lemma mem_bondDifferences [source]

```
lemma mem_bondDifferences (bonds : List (Bond G F)) (e : G) :
    e ∈ bondDifferences bonds ↔ e ≠ 0 ∧ ∃ i j, ∃ d₁ ∈ bondDisplacements bonds i j,
      ∃ d₂ ∈ bondDisplacements bonds i j, e = d₁ - d₂
```

lemma linear_mem_bondDifferences [source]

```
lemma linear_mem_bondDifferences {bonds : List (Bond G F)} {s : CellSymmetry G F}
    (hs : s ∈ (ofBonds bonds).symmetries) {e : G} (he : e ∈ bondDifferences bonds) :
    s.linear e ∈ bondDifferences bonds
```

lemma linear_mem_bondDifferences_iff [source]

The linear part of a symmetry preserves the differences of bonds.

```
lemma linear_mem_bondDifferences_iff {bonds : List (Bond G F)} {s : CellSymmetry G F}
    (hs : s ∈ (ofBonds bonds).symmetries) (e : G) :
    s.linear e ∈ bondDifferences bonds ↔ e ∈ bondDifferences bonds
```

def linearToLattice [source]

The linear part of a symmetry, as an element of the point stabilizer of the lattice of differences.

```
def linearToLattice (hΔ : bondDifferences bonds = bondNeighbors bonds') :
    (ofBonds bonds).symmetries →* (ofBonds bonds').pointStabilizer
```

lemma ker_linearToLattice [source]

```
lemma ker_linearToLattice (hΔ : bondDifferences bonds = bondNeighbors bonds')
    (hker : ∀ s ∈ (ofBonds bonds).symmetries, s.linear = AddEquiv.refl _ →
      s ∈ CellSymmetry.translations (Fin n → ℤ) F) :
    (linearToLattice hΔ).ker =
      (CellSymmetry.translations (Fin n → ℤ) F).subgroupOf (ofBonds bonds).symmetries
```

def pointGroupMulEquivOfLift [source]

***The point group of a crystal with several sites per cell is the point group of its lattice of differences**, if only translations have the identity as linear part and every point operation of the lattice lifts to a symmetry.*

```
noncomputable def pointGroupMulEquivOfLift (hΔ : bondDifferences bonds = bondNeighbors bonds')
    (hker : ∀ s ∈ (ofBonds bonds).symmetries, s.linear = AddEquiv.refl _ →
      s ∈ CellSymmetry.translations (Fin n → ℤ) F)
    (hlift : Function.Surjective (linearToLattice hΔ)) :
    (ofBonds bonds).pointGroup ≃* (ofBonds bonds').pointGroup
```

def shiftCandidates [source]

The candidates for the cell offsets of a symmetry, relative to the label i_0.

```
def shiftCandidates (bonds : List (Bond (Fin n → ℤ) F)) (i₀ : F) : Finset (Fin n → ℤ)
```

lemma mem_translations_of_linear_eq [source]

***Only translations have the identity as linear part**, if every label is bonded to `i₀` and this holds for the finitely many candidate offsets.*

```
lemma mem_translations_of_linear_eq (i₀ : F)
    (hconn : ∀ i, i = i₀ ∨ (bondDisplacements bonds i₀ i).Nonempty)
    (h : ∀ σ : Equiv.Perm F, ∀ r ∈ Fintype.piFinset fun _ => shiftCandidates bonds i₀,
      r i₀ = 0 → (∀ i j, (bondDisplacements bonds i j).image (fun d => d + r j - r i) =
        bondDisplacements bonds (σ i) (σ j)) → σ = 1 ∧ r = 0)
    (s : CellSymmetry (Fin n → ℤ) F) (hs : s ∈ (ofBonds bonds).symmetries)
    (hL : s.linear = AddEquiv.refl _) : s ∈ CellSymmetry.translations (Fin n → ℤ) F
```

lemma lift_of_admissible [source]

***Every point operation of the lattice of differences lifts to a symmetry**, if every admissible tuple of neighbours does, with a permutation and offsets `b i ∈ T i`.*

```
lemma lift_of_admissible (hΔ : bondDifferences bonds = bondNeighbors bonds')
    (hN : ∀ j, (Pi.single j 1 : Fin n → ℤ) ∈ bondNeighbors bonds') (T : F → Finset (Fin n → ℤ))
    (h : ∀ w : Fin n → bondNeighbors bonds', Admissible (bondNeighbors bonds') (fun j => w j) →
      ∃ σ : Equiv.Perm F, ∃ b ∈ Fintype.piFinset T, ∀ i j, (bondDisplacements bonds i j).image
        (fun d => act (fun j => w j) d + b j - b i) = bondDisplacements bonds (σ i) (σ j)) :
    Function.Surjective (linearToLattice hΔ)
```

2.286 PhyslibAlpha.CondensedMatter.Crystal.PointGroup.TetragonalHexagonal

PhyslibAlpha/CondensedMatter/Crystal/PointGroup/TetragonalHexagonal.lean on GitHub.

lemma toAdd_pow_apply [source]

```
lemma toAdd_pow_apply {G : Type*} [AddCommGroup G] (A : Multiplicative (AddAut G)) (n : ℕ)
    (x : G) : (Multiplicative.toAdd (A ^ n) : AddAut G) x =
      (Multiplicative.toAdd A : AddAut G)^[n] x
```

lemma pow_eq_one_of_admissible [source]

If the `n`-th power of every admissible triple fixes the basis, then every element of the point stabilizer satisfies `A ^ n = 1`.

```
lemma pow_eq_one_of_admissible (n : ℕ)
    (h : ∀ v, Admissible N v → ∀ j, (act v)^[n] (Pi.single j 1) = Pi.single j 1)
    (A : Multiplicative (AddAut (Fin 3 → ℤ))) (hA : A ∈ g.pointStabilizer) : A ^ n = 1
```

lemma tetragonal_card_pointGroup [source]

The point group of the primitive tetragonal lattice has order `16`.

```
lemma tetragonal_card_pointGroup : Nat.card tetragonalGraph.pointGroup = 16
```

abbrev bctFiber [source]

The admissible triples of the body-centred tetragonal graph whose first vector is `a`. Each is checked on its own: one kernel computation over all `16 ^ 3` triples does not fit in memory.

```
private abbrev bctFiber (a : ↥(PeriodicGraph.bondNeighbors bodyCentredTetragonalBonds)) : ℕ
```

lemma bctFiber_0 [source]

```
private lemma bctFiber_0 : bctFiber ⟨![1, 0, 0], by decide⟩ = 2
```

lemma bctFiber_1 [source]

```
private lemma bctFiber_1 : bctFiber ⟨![0, 1, 0], by decide⟩ = 2
```

lemma bctFiber_2 [source]

```
private lemma bctFiber_2 : bctFiber ⟨![0, 0, 1], by decide⟩ = 2
```

lemma bctFiber_3 [source]

```
private lemma bctFiber_3 : bctFiber ⟨![1, 1, 1], by decide⟩ = 2
```

lemma bctFiber_4 [source]

```
private lemma bctFiber_4 : bctFiber ⟨![0, 1, 1], by decide⟩ = 0
```

lemma bctFiber_5 [source]

```
private lemma bctFiber_5 : bctFiber ⟨![1, 0, 1], by decide⟩ = 0
```

lemma bctFiber_6 [source]

```
private lemma bctFiber_6 : bctFiber ⟨![1, 1, 2], by decide⟩ = 0
```

lemma bctFiber_7 [source]

```
private lemma bctFiber_7 : bctFiber ⟨![-1, 1, 0], by decide⟩ = 0
```

lemma bctFiber_8 [source]

```
private lemma bctFiber_8 : bctFiber ⟨![-1, 0, 0], by decide⟩ = 2
```

lemma bctFiber_9 [source]

```
private lemma bctFiber_9 : bctFiber ⟨![0, -1, 0], by decide⟩ = 2
```

lemma bctFiber_10 [source]

```
private lemma bctFiber_10 : bctFiber ⟨![0, 0, -1], by decide⟩ = 2
```

lemma bctFiber_11 [source]

```
private lemma bctFiber_11 : bctFiber ⟨![-1, -1, -1], by decide⟩ = 2
```

lemma bctFiber_12 [source]

```
private lemma bctFiber_12 : bctFiber ⟨![0, -1, -1], by decide⟩ = 0
```

lemma bctFiber_13 [source]

```
private lemma bctFiber_13 : bctFiber ⟨![-1, 0, -1], by decide⟩ = 0
```

lemma bctFiber_14 [source]

```
private lemma bctFiber_14 : bctFiber ⟨![-1, -1, -2], by decide⟩ = 0
```

lemma bctFiber_15 [source]

```
private lemma bctFiber_15 : bctFiber ⟨![1, -1, 0], by decide⟩ = 0
```

lemma bodyCentredTetragonal_card_pointGroup [source]

The point group of the body-centred tetragonal lattice has order `16`.

```
lemma bodyCentredTetragonal_card_pointGroup :
    Nat.card bodyCentredTetragonalGraph.pointGroup = 16
```

lemma hexagonal_card_pointGroup [source]

The point group of the primitive hexagonal lattice has order `24`.

```
lemma hexagonal_card_pointGroup : Nat.card hexagonalGraph.pointGroup = 24
```

lemma tetragonal_firstShell_card_pointGroup [source]

With the bonds of the first shell only, `±x, ±y, ±z`, a tetragonal lattice has the point group of the simple cubic lattice, of order `48`.

```
lemma tetragonal_firstShell_card_pointGroup :
    Nat.card (PeriodicGraph.ofBonds (G := Fin 3 → ℤ) (F := Unit)
      [⟨(), (), ![1, 0, 0]⟩, ⟨(), (), ![0, 1, 0]⟩, ⟨(), (), ![0, 0, 1]⟩]).pointGroup = 48
```

lemma tetragonal_pow_four [source]

Every element of the point group of the primitive tetragonal lattice satisfies `A ^ 4 = 1`.

```
lemma tetragonal_pow_four (A : Multiplicative (AddAut (Fin 3 → ℤ)))
    (hA : A ∈ tetragonalGraph.pointStabilizer) : A ^ 4 = 1
```

lemma hexagonal_pow_six [source]

Every element of the point group of the primitive hexagonal lattice satisfies `A ^ 6 = 1`.

```
lemma hexagonal_pow_six (A : Multiplicative (AddAut (Fin 3 → ℤ)))
    (hA : A ∈ hexagonalGraph.pointStabilizer) : A ^ 6 = 1
```

2.287 PhyslibAlpha.CondensedMatter.Crystal.QuasiLocal

PhyslibAlpha/CondensedMatter/Crystal/QuasiLocal.lean on GitHub.

abbrev crystalAlgebra [source]

***The quasi-local algebra of an infinite crystal** whose sites all carry the finite configuration space `Ω₀`.*

```
abbrev crystalAlgebra (G F : Type*) [DecidableEq (CellSite G F)]
```

def cellSymmetryAction [source]

***The cell symmetries of a crystal act on its quasi-local algebra** by *-automorphisms.*

```
def cellSymmetryAction : CellSymmetry G F →* (crystalAlgebra Ω₀ G F ≃⋆ₐ[ℂ] crystalAlgebra Ω₀ G
    F)
```

def translationHom [source]

The translations as cell symmetries.

```
def translationHom (G F : Type*) [AddCommGroup G] : Multiplicative G →* CellSymmetry G F where
```

def translationAction [source]

***The translations act on the quasi-local algebra** by *-automorphisms.*

```
def translationAction : Multiplicative G →* (crystalAlgebra Ω₀ G F ≃⋆ₐ[ℂ] crystalAlgebra Ω₀ G F
    )
```

lemma cellSymmetryAction_translation_conj [source]

***Covariance**: a cell symmetry carries the translation by `a` to the translation by `A a`.*

```
lemma cellSymmetryAction_translation_conj (s : CellSymmetry G F) (a : G) :
    cellSymmetryAction Ω₀ s * cellSymmetryAction Ω₀ (CellSymmetry.translation a) *
        (cellSymmetryAction Ω₀ s)⁻¹ =
      cellSymmetryAction Ω₀ (CellSymmetry.translation (s.linear a))
```

2.288 PhyslibAlpha.CondensedMatter.Crystal.Reciprocal

PhyslibAlpha/CondensedMatter/Crystal/Reciprocal.lean on GitHub.

def phase [source]

The phase accumulated by a plane wave of wave vector `k` under the lattice translation `n`.

```
noncomputable def phase (k : EuclideanSpace ℝ (Fin D)) (n : Multiplicative (Fin d → ℤ)) : ℝ
```

def dualCharacter [source]

The character of the translation group `ℤ^d` given by the plane wave of wave vector `k`.

```
noncomputable def dualCharacter (k : EuclideanSpace ℝ (Fin D)) :
    PontryaginDual (Multiplicative (Fin d → ℤ)) where
```

def reciprocalLattice [source]

The reciprocal lattice: wave vectors whose plane wave is trivial on every lattice translation.

```
def reciprocalLattice : AddSubgroup (EuclideanSpace ℝ (Fin D)) where
```

lemma dualCharacter_apply [source]

Evaluating the character of a wave vector gives the exponential of the phase.

```
lemma dualCharacter_apply (k : EuclideanSpace ℝ (Fin D)) (n : Multiplicative (Fin d → ℤ)) :
    dualCharacter a k n = Circle.exp (phase a k n)
```

lemma phase_add [source]

The phase of a lattice translation is additive in the wave vector.

```
lemma phase_add (k l : EuclideanSpace ℝ (Fin D)) (n : Multiplicative (Fin d → ℤ)) :
    phase a (k + l) n = phase a k n + phase a l n
```

lemma dualCharacter_eq_one_iff [source]

A wave vector gives the trivial character exactly when it lies in the reciprocal lattice.

```
lemma dualCharacter_eq_one_iff (k : EuclideanSpace ℝ (Fin D)) :
    dualCharacter a k = 1 ↔ k ∈ reciprocalLattice a
```

lemma dualCharacter_add_of_mem [source]

Wave vectors differing by a reciprocal lattice vector give the same character.

```
lemma dualCharacter_add_of_mem {k l : EuclideanSpace ℝ (Fin D)}
    (hl : l ∈ reciprocalLattice a) : dualCharacter a (k + l) = dualCharacter a k
```

def latticeBasis [source]

The basis of space formed by independent primitive translations.

```
noncomputable def latticeBasis (ha : LinearIndependent ℝ a) :
    Module.Basis (Fin d) ℝ (EuclideanSpace ℝ (Fin d))
```

def reciprocalBasis [source]

The reciprocal basis `b_i`, characterized by `⟪a_j, b_i⟫ = 2π δ_ij`.

```
noncomputable def reciprocalBasis (ha : LinearIndependent ℝ a) (i : Fin d) :
    EuclideanSpace ℝ (Fin d)
```

lemma inner_reciprocalBasis [source]

The reciprocal basis is dual to the primitive translations up to the factor `2π`.

```
lemma inner_reciprocalBasis (ha : LinearIndependent ℝ a) (i j : Fin d) :
    ⟪a j, reciprocalBasis ha i⟫ = if i = j then 2 * Real.pi else 0
```

lemma dualCharacter_surjective [source]

Every character of the translation group is the plane wave of some wave vector.

```
lemma dualCharacter_surjective (ha : LinearIndependent ℝ a) :
    Function.Surjective (dualCharacter a)
```

2.289 PhyslibAlpha.CondensedMatter.Crystal.Symmetry

PhyslibAlpha/CondensedMatter/Crystal/Symmetry.lean on GitHub.

instance Mul (CellSymmetry G F) [source]

```
instance : Mul (CellSymmetry G F) where
```

instance One (CellSymmetry G F) [source]

```
instance : One (CellSymmetry G F)
```

instance Inv (CellSymmetry G F) [source]

```
instance : Inv (CellSymmetry G F) where
```

instance Group (CellSymmetry G F) [source]

```
instance : Group (CellSymmetry G F) where
```

instance SMul (CellSymmetry G F) (CellSite G F) [source]

```
instance : SMul (CellSymmetry G F) (CellSite G F) where
```

lemma smul_def [source]

```
lemma smul_def (s : CellSymmetry G F) (x : CellSite G F) :
    s • x = ⟨s.linear x.cell + s.shift x.label, s.perm x.label⟩
```

instance MulAction (CellSymmetry G F) (CellSite G F) [source]

The site `σ i` of cell `A n + b i` to which a symmetry sends the site `i` of cell `n`.

```
instance : MulAction (CellSymmetry G F) (CellSite G F) where
```

def translation [source]

Translation by `a`: it moves every cell by `a` and fixes the labels.

```
def translation (a : G) : CellSymmetry G F
```

lemma translation_mul [source]

```
lemma translation_mul (a b : G) :
    (translation a : CellSymmetry G F) * translation b = translation (a + b)
```

lemma mul_translation_mul_inv [source]

Conjugating a translation by a symmetry gives the translation by the transformed vector.

```
lemma mul_translation_mul_inv (s : CellSymmetry G F) (a : G) :
    s * translation a * s⁻¹ = translation (s.linear a)
```

def translations [source]

The subgroup of translations.

```
def translations (G F : Type*) [AddCommGroup G] : Subgroup (CellSymmetry G F) where
```

instance (translations G F).Normal [source]

```
instance : (translations G F).Normal where
```

abbrev PointGroup [source]

The abstract point group of a crystal: cell symmetries modulo translations.

```
abbrev PointGroup (G F : Type*) [AddCommGroup G]
```

def Bonded [source]

Site `i` of the reference cell is bonded to site `j` of the cell `d`.

```
def Bonded (g : PeriodicGraph (Crystal.ofUnitCell G F)) (i j : F) (d : G) : Prop
```

lemma adj_iff_bonded [source]

Bonds between any two cells are the translates of bonds to the reference cell.

```
lemma adj_iff_bonded (g : PeriodicGraph (Crystal.ofUnitCell G F)) (n m : G) (i j : F) :
    g.toSimpleGraph.Adj ⟨n, i⟩ ⟨m, j⟩ ↔ g.Bonded i j (m - n)
```

def symmetries [source]

A symmetry preserves the bonds of the graph.

```
def symmetries (g : PeriodicGraph (Crystal.ofUnitCell G F)) : Subgroup (CellSymmetry G F) where
```

lemma mem_symmetries_iff [source]

***A finite criterion for bond-preserving symmetries.** The data `(A, σ, b)` is a symmetry of the crystal graph exactly when site `i` is bonded to site `j` in the cell `d` if and only if site `σ i` is bonded to site `σ j` in the cell `A d + b j - b i`.*

```
lemma mem_symmetries_iff (g : PeriodicGraph (Crystal.ofUnitCell G F)) (s : CellSymmetry G F) :
    s ∈ g.symmetries ↔ ∀ i j d, g.Bonded i j d ↔
      g.Bonded (s.perm i) (s.perm j) (s.linear d + s.shift j - s.shift i)
```

def symmetryIso [source]

A bond-preserving symmetry is an automorphism of the bond graph.

```
def symmetryIso (g : PeriodicGraph (Crystal.ofUnitCell G F)) (s : g.symmetries) :
    g.toSimpleGraph ≃g g.toSimpleGraph where
```

lemma translation_mem_symmetries [source]

Translations are bond-preserving symmetries.

```
lemma translation_mem_symmetries (g : PeriodicGraph (Crystal.ofUnitCell G F)) (a : G) :
    CellSymmetry.translation a ∈ g.symmetries
```

abbrev pointGroup [source]

The point group of a periodic graph: its bond-preserving symmetries modulo translations.

```
abbrev pointGroup (g : PeriodicGraph (Crystal.ofUnitCell G F))
```

lemma exists_symmetry_of_equivariant [source]

***Every automorphism of the bond graph that normalises the translations is a cell symmetry.** If `φ` is a graph automorphism and `A` an automorphism of the translation group with `φ (a +ᵥ x) = A a +ᵥ φ x`, then `φ` acts as a bond-preserving cell symmetry with linear part `A`.*

```
lemma exists_symmetry_of_equivariant (g : PeriodicGraph (Crystal.ofUnitCell G F))
    (φ : g.toSimpleGraph ≃g g.toSimpleGraph) (A : G ≃+ G)
    (hφ : ∀ (a : G) (x : CellSite G F), φ (a +ᵥ x) = A a +ᵥ φ x) :
    ∃ s ∈ g.symmetries, s.linear = A ∧ ∀ x, s • x = φ x
```

2.290 PhyslibAlpha.CondensedMatter.IsingModel.Basic

PhyslibAlpha/CondensedMatter/IsingModel/Basic.lean on GitHub.

structure IsingModel [source]

A classical nearest-neighbor Ising model on the specified undirected graph.

```
structure IsingModel {X : Type*} (graph : SimpleGraph X) where
  J : ℝ
  h : ℝ
```

abbrev Configuration [source]

Up or down spins at every site of the crystal.

```
abbrev Configuration (_I : IsingModel g)
```

def spin [source]

The dimensionless spin value, with `true` denoting up.

```
def spin (b : Bool) : ℝ
```

def pairSpin [source]

The exchange contribution of an ordered pair; nonadjacent sites contribute zero.

```
noncomputable def pairSpin (I : IsingModel g) (σ : I.Configuration) (x y : X) : ℝ
```

def energy [source]

Energy of the spins and internal edges of a finite region. The factor `1/2` counts each undirected graph edge once. Spins outside the region do not contribute.

```
noncomputable def energy (I : IsingModel g) (Λ : Finset X)
    (σ : I.Configuration) : ℝ
```

lemma energy_congr [source]

A region's energy depends only on the spin configuration inside that region.

```
lemma energy_congr (I : IsingModel g) (Λ : Finset X) {σ τ : I.Configuration}
    (hστ : ∀ x ∈ Λ, σ x = τ x) : I.energy Λ σ = I.energy Λ τ
```

lemma pairSpin_comm [source]

The exchange contribution is symmetric in the endpoints of a bond.

```
lemma pairSpin_comm (I : IsingModel g) (σ : I.Configuration) (x y : X) :
    I.pairSpin σ x y = I.pairSpin σ y x
```

lemma energy_union [source]

Joining disjoint regions adds exactly their crossing exchange interactions.

```
lemma energy_union [DecidableEq X] (I : IsingModel g) (Λ M : Finset X)
    (hΛM : Disjoint Λ M) (σ : I.Configuration) :
    I.energy (Λ ∪ M) σ = I.energy Λ σ + I.energy M σ -
```

```
    I.J * ∑ x ∈ Λ, ∑ y ∈ M, I.pairSpin σ x y
```

lemma abs_pairSpin [source]

An ordered bond contributes at most one in absolute spin units.

```
lemma abs_pairSpin (I : IsingModel g) (σ : I.Configuration) (x y : X) :
    |I.pairSpin σ x y| = if g.Adj x y then 1 else 0
```

lemma abs_sum_pairSpin_le [source]

The total exchange spin sum is bounded by the number of connections between the regions.

```
lemma abs_sum_pairSpin_le (I : IsingModel g) (Λ M : Finset X)
    (σ : I.Configuration) :
    |∑ x ∈ Λ, ∑ y ∈ M, I.pairSpin σ x y| ≤ g.bondCountBetween Λ M
```

lemma abs_energy_union_sub_le [source]

Failure of energy additivity is controlled solely by the bonds crossing the boundary.

```
lemma abs_energy_union_sub_le [DecidableEq X] (I : IsingModel g)
    (Λ M : Finset X) (hΛM : Disjoint Λ M) (σ : I.Configuration) :
    |I.energy (Λ ∪ M) σ - I.energy Λ σ - I.energy M σ| ≤
      |I.J| * g.bondCountBetween Λ M
```

lemma abs_energy_le [source]

The absolute energy is bounded by the internal bond count and the number of spins.

```
lemma abs_energy_le (I : IsingModel g) (Λ : Finset X) (σ : I.Configuration) :
    |I.energy Λ σ| ≤ |I.J| / 2 * g.bondCountBetween Λ Λ + |I.h| * Λ.card
```

lemma abs_energy_le_of_degree_le [source]

A coordination bound gives a uniform energy bound proportional to the region's volume.

```
lemma abs_energy_le_of_degree_le [g.LocallyFinite] (I : IsingModel g) (Λ : Finset X)
    (σ : I.Configuration) (q : ℕ) (hq : ∀ x ∈ Λ, (g.neighborFinset x).card ≤ q) :
    |I.energy Λ σ| ≤ (|I.J| * q / 2 + |I.h|) * Λ.card
```

def IsGroundState [source]

A configuration minimizing the energy of the finite region Λ. Only interactions internal to the region enter this definition.

```
def IsGroundState (I : IsingModel g) (Λ : Finset X) (σ : I.Configuration) : Prop
```

def polarized [source]

Uniform spins aligned with the field; at zero field the up orientation is chosen.

```
noncomputable def polarized (I : IsingModel g) : I.Configuration
```

def groundEnergy [source]

The least finite-region energy over all spin configurations.

```
noncomputable def groundEnergy (I : IsingModel g) (Λ : Finset X) : ℝ
```

lemma groundEnergy_eq_of_isGroundState [source]

An energy-minimizing configuration attains the least energy.

```
lemma groundEnergy_eq_of_isGroundState (I : IsingModel g) (Λ : Finset X)
    {σ : I.Configuration} (hσ : I.IsGroundState Λ σ) :
    I.groundEnergy Λ = I.energy Λ σ
```

lemma energy_polarized [source]

Uniform field-aligned polarization has saturated exchange and field contributions.

```
lemma energy_polarized (I : IsingModel g) (Λ : Finset X) :
    I.energy Λ I.polarized =
      -(I.J / 2) * g.bondCountBetween Λ Λ - |I.h| * Λ.card
```

lemma energy_lower_bound [source]

Ferromagnetic energies cannot lie below the saturated exchange and field contributions.

```
lemma energy_lower_bound (I : IsingModel g) (hJ : 0 ≤ I.J) (Λ : Finset X)
    (σ : I.Configuration) :
    -(I.J / 2) * g.bondCountBetween Λ Λ - |I.h| * Λ.card ≤ I.energy Λ σ
```

lemma isGroundState_polarized [source]

Uniform field-aligned polarization is a ground state on every finite ferromagnetic region.

```
lemma isGroundState_polarized (I : IsingModel g) (hJ : 0 ≤ I.J) (Λ : Finset X) :
    I.IsGroundState Λ I.polarized
```

lemma groundEnergy_eq [source]

The exact ferromagnetic minimum counts internal bonds and field-aligned spins.

```
lemma groundEnergy_eq (I : IsingModel g) (hJ : 0 ≤ I.J) (Λ : Finset X) :
    I.groundEnergy Λ = -(I.J / 2) * g.bondCountBetween Λ Λ - |I.h| * Λ.card
```

def groundEnergyDensity [source]

The minimum finite-region energy divided by the number of sites.

```
noncomputable def groundEnergyDensity (I : IsingModel g) (Λ : Finset X) : ℝ
```

lemma groundEnergyDensity_eq [source]

The ferromagnetic minimum per site is determined by average internal coordination.

```
lemma groundEnergyDensity_eq (I : IsingModel g) (hJ : 0 ≤ I.J) (Λ : Finset X)
    (hΛ : Λ.Nonempty) :
    I.groundEnergyDensity Λ =
      -(I.J / 2) * ((g.bondCountBetween Λ Λ : ℝ) / Λ.card) - |I.h|
```

lemma tendsto_groundEnergyDensity [source]

Convergent average internal coordination gives a ferromagnetic ground-state energy density. The statement applies to any sequence of nonempty finite regions on any graph.

```
lemma tendsto_groundEnergyDensity (I : IsingModel g) (hJ : 0 ≤ I.J)
    (Λ : ℕ → Finset X) (hΛ : ∀ n, (Λ n).Nonempty) (z : ℝ)
    (hz : Filter.Tendsto
      (fun n => (g.bondCountBetween (Λ n) (Λ n) : ℝ) / (Λ n).card)
      Filter.atTop (nhds z)) :
    Filter.Tendsto (fun n => I.groundEnergyDensity (Λ n))
      Filter.atTop (nhds (-(I.J / 2) * z - |I.h|))
```

def translate [source]

Transport a configuration along a crystal translation.

```
def translate (I : IsingModel g.toSimpleGraph) (a : G) (σ : I.Configuration) : I.Configuration
```

def translateRegion [source]

Translate a finite region by the existing bijective action on crystal sites.

```
def translateRegion (a : G) (Λ : Finset C.Site) : Finset C.Site
```

lemma pairSpin_translate [source]

Exchange contributions are unchanged by translating both spins and sites.

```
lemma pairSpin_translate (I : IsingModel g.toSimpleGraph) (a : G) (σ : I.Configuration)
    (x y : C.Site) :
    I.pairSpin (I.translate a σ) (a +ᵥ x) (a +ᵥ y) = I.pairSpin σ x y
```

lemma energy_translate [source]

Uniform Ising interactions inherit the crystal's translation symmetry. Translating the region as well as its configuration preserves its energy.

```
lemma energy_translate (I : IsingModel g.toSimpleGraph) (a : G)
    (Λ : Finset C.Site) (σ : I.Configuration) :
    I.energy (translateRegion a Λ) (I.translate a σ) = I.energy Λ σ
```

2.291 PhyslibAlpha.CondensedMatter.IsingModel.Examples

PhyslibAlpha/CondensedMatter/IsingModel/Examples.lean on GitHub.

def hypercubic [source]

Uniform classical Ising interactions on the hypercubic bond graph in arbitrary dimension.

```
def hypercubic (d : ℕ) (J h : ℝ) : IsingModel (Crystal.hypercubicGraph d).toSimpleGraph
```

def square [source]

The square-lattice model, specializing the hypercubic construction to two dimensions.

```
def square (J h : ℝ) : IsingModel Crystal.squareGraph.toSimpleGraph
```

def honeycomb [source]

The honeycomb classical Ising model, using the crystal's existing bonds.

```
def honeycomb (J h : ℝ) : IsingModel Crystal.honeycombGraph.toSimpleGraph
```

def cells [source]

The cells `{0, …, n}^d` of a hypercube of side `n + 1` unit cells.

```
def cells (d n : ℕ) : Finset (Fin d → ℤ)
```

lemma mem_cells [source]

```
lemma mem_cells {d n : ℕ} (c : Fin d → ℤ) :
    c ∈ cells d n ↔ ∀ i, 0 ≤ c i ∧ c i ≤ n
```

lemma card_cells [source]

```
lemma card_cells (d n : ℕ) : (cells d n).card = (n + 1) ^ d
```

def interior [source]

The cells at least `R` unit cells away from every face of the hypercube.

```
def interior (d n R : ℕ) : Finset (Fin d → ℤ)
```

lemma mem_interior [source]

```
lemma mem_interior {d n R : ℕ} (c : Fin d → ℤ) :
    c ∈ interior d n R ↔ ∀ i, (R : ℤ) ≤ c i ∧ c i ≤ n - R
```

lemma card_interior [source]

The interior of a hypercube of side `n + 1` is a hypercube of side `n + 1 - 2 R`.

```
lemma card_interior (d n R : ℕ) : (interior d n R).card = (n + 1 - 2 * R) ^ d
```

lemma interior_subset [source]

```
lemma interior_subset {d n R : ℕ} : interior d n R ⊆ cells d n
```

def region [source]

All sites of the crystal with unit cell labels `F` that lie in the cells `B`.

```
def region (B : Finset (Fin d → ℤ)) : Finset (Crystal.ofUnitCell (Fin d → ℤ) F).Site
```

lemma mem_region [source]

```
lemma mem_region {B : Finset (Fin d → ℤ)} (x : (Crystal.ofUnitCell (Fin d → ℤ) F).Site) :
    x ∈ region B ↔ x.cell ∈ B
```

lemma sum_region [source]

```
lemma sum_region {M : Type*} [AddCommMonoid M] {B : Finset (Fin d → ℤ)}
    (f : (Crystal.ofUnitCell (Fin d → ℤ) F).Site → M) :
    ∑ x ∈ region B, f x = ∑ c ∈ B, ∑ i : F, f ⟨c, i⟩
```

lemma card_region [source]

```
lemma card_region (B : Finset (Fin d → ℤ)) :
    (region (F := F) B).card = B.card * Fintype.card F
```

lemma region_cells_nonempty [source]

Every hypercube contains at least one site if the unit cell does.

```
lemma region_cells_nonempty [Nonempty F] (d n : ℕ) :
    (region (F := F) (cells d n)).Nonempty
```

lemma exists_range [source]

Bonds have a finite range: there is a distance `R`, in units of the cell spacing, beyond which no two sites are connected. This holds because the coordination is finite and every site is a translate of one of finitely many reference sites.

```
lemma exists_range :
    ∃ R : ℕ, ∀ x y : (Crystal.ofUnitCell (Fin d → ℤ) F).Site,
      g.toSimpleGraph.Adj x y → ∀ k, |y.cell k - x.cell k| ≤ R
```

lemma neighbors_subset_region [source]

Sites in the interior of a hypercube have all their neighbors in the sample, once the interior is at least the bond range away from the surface.

```
lemma neighbors_subset_region {n R : ℕ}
    (hR : ∀ x y : (Crystal.ofUnitCell (Fin d → ℤ) F).Site,
      g.toSimpleGraph.Adj x y → ∀ k, |y.cell k - x.cell k| ≤ R)
    {x : (Crystal.ofUnitCell (Fin d → ℤ) F).Site} (hx : x.cell ∈ interior d n R) :
    g.neighbors x ⊆ region (cells d n)
```

def meanCoordination [source]

The mean number of neighbors per site of a crystal with unit cell `F`: the coordination averaged over the sites of one unit cell.

```
noncomputable def meanCoordination : ℝ
```

lemma sum_card_neighbors_region [source]

Summing the coordination over the sites of a set of cells counts every unit-cell coordination once per cell.

```
lemma sum_card_neighbors_region (B : Finset (Fin d → ℤ)) :
    ∑ x ∈ region B, (g.neighbors x).card =
      B.card * ∑ i : F, (g.neighbors ⟨0, i⟩).card
```

lemma bondCountBetween_le [source]

The ordered bonds inside a region number at most the total coordination of its sites.

```
lemma bondCountBetween_le (Λ : Finset (Crystal.ofUnitCell (Fin d → ℤ) F).Site) :
    g.toSimpleGraph.bondCountBetween Λ Λ ≤ ∑ x ∈ Λ, (g.neighbors x).card
```

lemma sum_interior_le_bondCountBetween [source]

Every site in the interior of the hypercube has all its bonds inside the hypercube.

```
lemma sum_interior_le_bondCountBetween {n R : ℕ}
    (hR : ∀ x y : (Crystal.ofUnitCell (Fin d → ℤ) F).Site,
      g.toSimpleGraph.Adj x y → ∀ k, |y.cell k - x.cell k| ≤ R) :
    ∑ x ∈ region (F := F) (interior d n R), (g.neighbors x).card ≤
      g.toSimpleGraph.bondCountBetween (region (cells d n)) (region (cells d n))
```

lemma tendsto_averageCoordination [source]

The average coordination of hypercubes converges to the mean coordination of the crystal: the sites near the surface, whose neighbors are cut off, become a vanishing fraction of the sample.

```
lemma tendsto_averageCoordination :
    Filter.Tendsto
      (fun n : ℕ => (g.toSimpleGraph.bondCountBetween (region (cells d n))
        (region (cells d n)) : ℝ) / (region (F := F) (cells d n)).card)
      Filter.atTop (nhds (meanCoordination g))
```

lemma tendsto_groundEnergyDensity_hypercube [source]

The ground-state energy per spin of a ferromagnetic Ising model on any crystal, in hypercubes of growing size, converges to `-(J / 2) z¯ - |h|`, where `z¯` is the mean coordination. Each spin carries the energy `-J` for half of its bonds; the missing bonds at the surface of the sample do not contribute in the limit. The field may have either sign, and the lattice may have several sites per unit cell.

```
lemma tendsto_groundEnergyDensity_hypercube (I : IsingModel g.toSimpleGraph) (hJ : 0 ≤ I.J) :
    Filter.Tendsto (fun n : ℕ => I.groundEnergyDensity (region (cells d n)))
```

```
      Filter.atTop (nhds (-(I.J / 2) * meanCoordination g - |I.h|))
```

lemma meanCoordination_honeycomb [source]

The honeycomb lattice has three neighbors per site.

```
lemma meanCoordination_honeycomb : Hypercube.meanCoordination Crystal.honeycombGraph = 3
```

lemma tendsto_groundEnergyDensity_honeycomb [source]

The ground-state energy per spin of the ferromagnetic honeycomb Ising model converges to `-3 J / 2 - |h|` as the sample grows.

```
lemma tendsto_groundEnergyDensity_honeycomb (J h : ℝ) (hJ : 0 ≤ J) :
    Filter.Tendsto (fun n : ℕ => (honeycomb J h).groundEnergyDensity
        (Hypercube.region (Hypercube.cells 2 n)))
      Filter.atTop (nhds (-(3 * J / 2) - |h|))
```

lemma meanCoordination_square [source]

The square lattice has four neighbors per site.

```
lemma meanCoordination_square : Hypercube.meanCoordination Crystal.squareGraph = 4
```

lemma tendsto_groundEnergyDensity_square [source]

The ground-state energy per spin of the ferromagnetic square-lattice Ising model converges to `-2 J - |h|` as the sample grows.

```
lemma tendsto_groundEnergyDensity_square (J h : ℝ) (hJ : 0 ≤ J) :
    Filter.Tendsto (fun n : ℕ => (square J h).groundEnergyDensity
        (Hypercube.region (Hypercube.cells 2 n)))
      Filter.atTop (nhds (-(2 * J) - |h|))
```

lemma meanCoordination_of_card [source]

If every site has `q` neighbors, the mean coordination is `q`.

```
lemma meanCoordination_of_card {q : ℕ} (h : ∀ x, (g.neighbors x).card = q) :
    Hypercube.meanCoordination g = q
```

lemma tendsto_groundEnergyDensity_of_card [source]

***The ferromagnetic ground-state energy density of a `q`-regular crystal** converges to `-q J / 2 - |h|`.*

```
lemma tendsto_groundEnergyDensity_of_card {q : ℕ} (h : ∀ x, (g.neighbors x).card = q)
    (I : IsingModel g.toSimpleGraph) (hJ : 0 ≤ I.J) :
    Filter.Tendsto (fun n : ℕ => I.groundEnergyDensity (Hypercube.region (Hypercube.cells d n))
    )
      Filter.atTop (nhds (-(q * I.J / 2) - |I.h|))
```

lemma tendsto_groundEnergyDensity_diamond [source]

The diamond lattice has four neighbors per site.

```
lemma tendsto_groundEnergyDensity_diamond (J h : ℝ) (hJ : 0 ≤ J) :
    Filter.Tendsto (fun n : ℕ => ((J, h) : IsingModel Crystal.diamondGraph.toSimpleGraph
        ).groundEnergyDensity (Hypercube.region (Hypercube.cells 3 n)))
      Filter.atTop (nhds (-(4 * J / 2) - |h|))
```

lemma tendsto_groundEnergyDensity_fcc [source]

The face-centred cubic lattice has twelve neighbors per site.

```
lemma tendsto_groundEnergyDensity_fcc (J h : ℝ) (hJ : 0 ≤ J) :
    Filter.Tendsto (fun n : ℕ => ((J, h) : IsingModel Crystal.fccGraph.toSimpleGraph
        ).groundEnergyDensity (Hypercube.region (Hypercube.cells 3 n)))
      Filter.atTop (nhds (-(12 * J / 2) - |h|))
```

lemma tendsto_groundEnergyDensity_bcc [source]

The body-centred cubic lattice has eight neighbors per site.

```
lemma tendsto_groundEnergyDensity_bcc (J h : ℝ) (hJ : 0 ≤ J) :
    Filter.Tendsto (fun n : ℕ => ((J, h) : IsingModel Crystal.bccGraph.toSimpleGraph
        ).groundEnergyDensity (Hypercube.region (Hypercube.cells 3 n)))
      Filter.atTop (nhds (-(8 * J / 2) - |h|))
```

lemma tendsto_groundEnergyDensity_hcp [source]

The hexagonal close-packed crystal has twelve neighbors per site.

```
lemma tendsto_groundEnergyDensity_hcp (J h : ℝ) (hJ : 0 ≤ J) :
    Filter.Tendsto (fun n : ℕ => ((J, h) : IsingModel Crystal.hcpGraph.toSimpleGraph
        ).groundEnergyDensity (Hypercube.region (Hypercube.cells 3 n)))
      Filter.atTop (nhds (-(12 * J / 2) - |h|))
```

2.292 PhyslibAlpha.CondensedMatter.QuasiLocal.Gibbs

PhyslibAlpha/CondensedMatter/QuasiLocal/Gibbs.lean on GitHub.

def gibbsWeight [source]

The Gibbs weight `e^{-h}` of an energy operator `h`.

```
def gibbsWeight (h : Matrix n n ℂ) : Matrix n n ℂ
```

def gibbsRoot [source]

The square root `e^{-h/2}` of the Gibbs weight.

```
def gibbsRoot (h : Matrix n n ℂ) : Matrix n n ℂ
```

lemma gibbsWeight_eq_root_mul [source]

```
lemma gibbsWeight_eq_root_mul (h : Matrix n n ℂ) : gibbsWeight h = gibbsRoot h * gibbsRoot h
```

lemma gibbsRoot_isHermitian [source]

```
lemma gibbsRoot_isHermitian {h : Matrix n n ℂ} (hh : h.IsHermitian) :
    (gibbsRoot h).IsHermitian
```

lemma gibbsWeight_isHermitian [source]

```
lemma gibbsWeight_isHermitian {h : Matrix n n ℂ} (hh : h.IsHermitian) :
    (gibbsWeight h).IsHermitian
```

def partition [source]

*The **partition function** `tr e^{-h}`.*

```
def partition (h : Matrix n n ℂ) : ℂ
```

lemma partition_eq [source]

```
lemma partition_eq (h : Matrix n n ℂ) (hh : h.IsHermitian) :
    partition h = ((gibbsRoot h)^H * gibbsRoot h).trace
```

lemma gibbsRoot_ne_zero [source]

```
lemma gibbsRoot_ne_zero [Nonempty n] (h : Matrix n n ℂ) : gibbsRoot h ≠ 0
```

lemma partition_pos [source]

The partition function of a Hermitian operator is positive.

```
lemma partition_pos [Nonempty n] {h : Matrix n n ℂ} (hh : h.IsHermitian) : 0 < partition h
```

lemma partition_ne_zero [source]

```
lemma partition_ne_zero [Nonempty n] {h : Matrix n n ℂ} (hh : h.IsHermitian) :
    partition h ≠ 0
```

def gibbsFunctional [source]

The Gibbs functional `a ↦ tr(e^{-h} a) / tr(e^{-h})`.

```
def gibbsFunctional (h : Matrix n n ℂ) : Matrix n n ℂ →ₗ[ℂ] ℂ
```

lemma gibbsFunctional_apply [source]

```
lemma gibbsFunctional_apply (h a : Matrix n n ℂ) :
    gibbsFunctional h a = (partition h)⁻¹ * (gibbsWeight h * a).trace
```

lemma inv_partition_nonneg [source]

The inverse of the partition function is a nonnegative real number.

```
lemma inv_partition_nonneg [Nonempty n] {h : Matrix n n ℂ} (hh : h.IsHermitian) :
    0 ≤ (partition h)⁻¹
```

def gibbsState [source]

***The Gibbs state of a Hermitian operator** is a state of the matrix algebra: normalized, Hermitian and positive.*

```
def gibbsState [Nonempty n] (h : Matrix n n ℂ) (hh : h.IsHermitian) : AlgState (Matrix n n ℂ)
    where
```

def canonicalState [source]

***The canonical ensemble** at inverse temperature `β` for a Hamiltonian `H`.*

```
def canonicalState [Nonempty n] (β : ℝ) (H : Matrix n n ℂ) (hH : H.IsHermitian) :
    AlgState (Matrix n n ℂ)
```

def grandCanonicalOp [source]

The operator whose Gibbs weight is the grand canonical weight: `β (H - μ N)`.

```
def grandCanonicalOp (β μ : ℝ) (H N : Matrix n n ℂ) : Matrix n n ℂ
```

lemma grandCanonicalOp_isHermitian [source]

```
lemma grandCanonicalOp_isHermitian {H N : Matrix n n ℂ} (hH : H.IsHermitian) (hN : N.
    IsHermitian)
    (β μ : ℝ) : (grandCanonicalOp β μ H N).IsHermitian
```

def grandCanonicalState [source]

***The grand canonical ensemble** at inverse temperature `β` and chemical potential `μ`, for a Hamiltonian `H` and a charge `N`.*

```
def grandCanonicalState [Nonempty n] (β μ : ℝ) (H N : Matrix n n ℂ) (hH : H.IsHermitian)
    (hN : N.IsHermitian) : AlgState (Matrix n n ℂ)
```

lemma grandCanonicalOp_zero [source]

At zero chemical potential the grand canonical ensemble is the canonical one.

```
lemma grandCanonicalOp_zero (β : ℝ) (H N : Matrix n n ℂ) :
    grandCanonicalOp β 0 H N = (β : ℂ) • H
```

lemma grandCanonicalWeight_of_commute [source]

***Factorization for a conserved charge**: if `H` and `N` commute, the grand canonical weight is the canonical weight times the fugacity factor `e^{βμ N}`.*

```
lemma grandCanonicalWeight_of_commute (β μ : ℝ) {H N : Matrix n n ℂ} (hc : Commute H N) :
    gibbsWeight (grandCanonicalOp β μ H N) =
      gibbsWeight ((β : ℂ) • H) * NormedSpace.exp (((β * μ : ℝ) : ℂ) • N)
```

lemma gibbsWeight_diagonal [source]

***For a diagonal energy operator the Gibbs weight is the Boltzmann factor** `e^{-E(c)}` of each configuration `c`: the classical Gibbs distribution.*

```
lemma gibbsWeight_diagonal (v : n → ℂ) :
    gibbsWeight (Matrix.diagonal v) = Matrix.diagonal fun c => Complex.exp (-v c)
```

lemma partition_diagonal [source]

The partition function of a diagonal energy operator is the sum of the Boltzmann factors.

```
lemma partition_diagonal (v : n → ℂ) : partition (Matrix.diagonal v) = ∑ c, Complex.exp (-v c)
```

2.293 PhyslibAlpha.CondensedMatter.QuasiLocal.GibbsSymmetry

PhyslibAlpha/CondensedMatter/QuasiLocal/GibbsSymmetry.lean on GitHub.

lemma quantumIso_exp [source]

Moving commutes with the exponential.

```
lemma quantumIso_exp (γ : Γ) (X : Finset S) (a : Alg (fun _ : S => Ω₀) X) :
    quantumIso γ X (NormedSpace.exp a) = NormedSpace.exp (quantumIso γ X a)
```

lemma quantumIso_trace [source]

Moving preserves the trace.

```
lemma quantumIso_trace (γ : Γ) (X : Finset S) (a : Alg (fun _ : S => Ω₀) X) :
    (quantumIso γ X a).trace = a.trace
```

lemma quantumIso_gibbsWeight [source]

```
lemma quantumIso_gibbsWeight (γ : Γ) (X : Finset S) (a : Alg (fun _ : S => Ω₀) X) :
    quantumIso γ X (gibbsWeight a) = gibbsWeight (quantumIso γ X a)
```

lemma quantumIso_partition [source]

```
lemma quantumIso_partition (γ : Γ) (X : Finset S) (a : Alg (fun _ : S => Ω₀) X) :
    partition (quantumIso γ X a) = partition a
```

lemma gibbsFunctional_quantumIso [source]

The Gibbs functional is covariant.

```
lemma gibbsFunctional_quantumIso (γ : Γ) (X : Finset S) (h b : Alg (fun _ : S => Ω₀) X) :
    gibbsFunctional (quantumIso γ X h) (quantumIso γ X b) = gibbsFunctional h b
```

def IsInvariant [source]

*An interaction is *invariant* under `γ` if moving a term gives the term of the moved region.*

```
def Interaction.IsInvariant (I : Interaction (fun _ : S => Ω₀)) (γ : Γ) : Prop
```

def powersetSmul [source]

The bijection between the regions of a box and the regions of the moved box.

```
def powersetSmul (γ : Γ) (Λ : Finset S) : Λ.powerset ≃ (γ • Λ).powerset where
```

lemma hamiltonian_smul [source]

***The Hamiltonian of a moved box**: for an invariant interaction, moving the Hamiltonian of a box gives the Hamiltonian of the moved box.*

```
lemma Interaction.hamiltonian_smul (I : Interaction (fun _ : S => Ω₀)) {γ : Γ}
    (hI : I.IsInvariant γ) (Λ : Finset S) :
    quantumIso γ Λ (I.hamiltonian Λ) = I.hamiltonian (γ • Λ)
```

lemma canonicalState_smul [source]

***The canonical ensemble is covariant**: the expectation of a moved observable in the ensemble of the moved box is the expectation of the observable in the ensemble of the box.*

```
lemma Interaction.canonicalState_smul (I : Interaction (fun _ : S => Ω₀)) {γ : Γ}
    (hI : I.IsInvariant γ) (β : ℝ) (Λ : Finset S) (a : Alg (fun _ : S => Ω₀) Λ) :
    I.canonicalState β (γ • Λ) (quantumIso γ Λ a) = I.canonicalState β Λ a
```

def siteSmul [source]

The bijection between the sites of a box and the sites of the moved box.

```
def siteSmul (γ : Γ) (Λ : Finset S) : ɪΛ ≃ ɪ(γ • Λ) where
```

lemma chargeOp_smul [source]

A uniform charge is moved to the uniform charge of the moved box.

```
lemma chargeOp_smul (q₀ : Ω₀ → ℝ) (γ : Γ) (Λ : Finset S) :
    quantumIso γ Λ (chargeOp (fun _ : S => q₀) Λ) = chargeOp (fun _ : S => q₀) (γ • Λ)
```

lemma grandCanonicalOp_quantumIso [source]

The charge of a moved box, as a statement about the operator whose weight is the grand canonical one.

```
lemma grandCanonicalOp_quantumIso (γ : Γ) (Λ : Finset S) (β μ : ℝ) (H N : Alg (fun _ : S => Ω₀)
     Λ) :
    quantumIso γ Λ (grandCanonicalOp β μ H N) =
      grandCanonicalOp β μ (quantumIso γ Λ H) (quantumIso γ Λ N)
```

lemma grandCanonicalState_smul [source]

***The grand canonical ensemble is covariant**, for an invariant interaction and a uniform charge.*

```
lemma Interaction.grandCanonicalState_smul (I : Interaction (fun _ : S => Ω₀)) {γ : Γ}
    (hI : I.IsInvariant γ) (q₀ : Ω₀ → ℝ) (β μ : ℝ) (Λ : Finset S)
    (a : Alg (fun _ : S => Ω₀) Λ) :
    I.grandCanonicalState (fun _ : S => q₀) β μ (γ • Λ) (quantumIso γ Λ a) =
      I.grandCanonicalState (fun _ : S => q₀) β μ Λ a
```

2.294 PhyslibAlpha.CondensedMatter.QuasiLocal.Instances

PhyslibAlpha/CondensedMatter/QuasiLocal/Instances.lean on GitHub.

def quantumNet [source]

***The quantum net**: the matrices on the configurations of each region, with `M ↦ M ⊗ 1` as inclusion.*

```
def quantumNet : AlgebraNet S where
```

abbrev CAlg [source]

The classical algebra of a region: the complex functions on its configurations.

```
abbrev CAlg (X : Finset S) : Type _
```

def classicalIncl [source]

A function of the values in `X` is a function of the values in a larger region `Y`.

```
def classicalIncl {X Y : Finset S} (h : X ≤ Y) : CAlg Ω X →⋆ₐ[ℂ] CAlg Ω Y where
```

def classicalNet [source]

***The classical net**: the functions on the configurations of each region.*

```
def classicalNet : AlgebraNet S where
```

def diagonalInclusion [source]

The classical algebra of a region as the diagonal operators.

```
def diagonalInclusion (X : Finset S) : CAlg Ω X →⋆ₐ[ℂ] Alg Ω X where
```

lemma diagonalInclusion_incl [source]

***The classical net is a subnet of the quantum net**: the diagonal inclusion commutes with the inclusions of regions.*

```
lemma diagonalInclusion_incl {X Y : Finset S} (h : X ≤ Y) (f : CAlg Ω X) :
    inclMat h (diagonalInclusion Ω X f) = diagonalInclusion Ω Y (classicalIncl Ω h f)
```

2.295 PhyslibAlpha.CondensedMatter.QuasiLocal.Interaction

PhyslibAlpha/CondensedMatter/QuasiLocal/Interaction.lean on GitHub.

structure Interaction [source]

***An interaction**: a Hermitian operator of each finite region.*

```
structure Interaction where
  Φ : ∀ X : Finset S, Alg Ω X
  isHermitian : ∀ X, (Φ X).IsHermitian
```

def hamiltonian [source]

***The Hamiltonian of a finite box**: the sum of the lifts to the box of the terms of the interaction supported inside it.*

```
def Interaction.hamiltonian (I : Interaction Ω) (Λ : Finset S) : Alg Ω Λ
```

lemma hamiltonian_isHermitian [source]

```
lemma Interaction.hamiltonian_isHermitian (I : Interaction Ω) (Λ : Finset S) :
    (I.hamiltonian Λ).IsHermitian
```

lemma inclMat_exp [source]

The lift `M ↦ M ⊗ 1` commutes with the exponential.

```
lemma inclMat_exp {X Y : Finset S} (h : X ≤ Y) (a : Alg Ω X) :
    inclMat h (NormedSpace.exp a) = NormedSpace.exp (inclMat h a)
```

lemma inclMat_gibbsWeight [source]

The lift of a Gibbs weight is the Gibbs weight of the lift.

```
lemma inclMat_gibbsWeight {X Y : Finset S} (h : X ≤ Y) (a : Alg Ω X) :
    inclMat h (gibbsWeight a) = gibbsWeight (inclMat h a)
```

abbrev Charge [source]

A charge: a real number attached to each value of the configuration space of each site, such as the occupation number of a mode.

```
abbrev Charge (Ω : S → Type*)
```

def chargeOp [source]

***The total charge of a box**: the diagonal operator with the value `∑_{x ∈ Λ} q_x(c_x)` on the configuration `c`.*

```
def chargeOp (q : Charge Ω) (Λ : Finset S) : Alg Ω Λ
```

lemma chargeOp_isHermitian [source]

```
lemma chargeOp_isHermitian (q : Charge Ω) (Λ : Finset S) : (chargeOp q Λ).IsHermitian
```

def canonicalState [source]

***The canonical ensemble of a box** at inverse temperature β.*

```
def Interaction.canonicalState (I : Interaction Ω) (β : ℝ) (Λ : Finset S) :
    AlgState (Alg Ω Λ)
```

def grandCanonicalState [source]

***The grand canonical ensemble of a box** at inverse temperature β and chemical potential μ for the charge q.*

```
def Interaction.grandCanonicalState (I : Interaction Ω) (q : Charge Ω) (β μ : ℝ)
    (Λ : Finset S) : AlgState (Alg Ω Λ)
```

def Conserves [source]

The interaction conserves the charge: the Hamiltonian of every box commutes with its total charge.

```
def Interaction.Conserves (I : Interaction Ω) (q : Charge Ω) : Prop
```

lemma grandCanonicalWeight_of_conserves [source]

For a conserved charge the grand canonical weight of a box is its canonical weight times a fugacity.

```
lemma Interaction.grandCanonicalWeight_of_conserves (I : Interaction Ω) {q : Charge Ω}
    (hq : I.Conserves q) (β μ : ℝ) (Λ : Finset S) :
    gibbsWeight (grandCanonicalOp β μ (I.hamiltonian Λ) (chargeOp q Λ)) =
      gibbsWeight ((β : ℂ) • I.hamiltonian Λ) *
        NormedSpace.exp (((β * μ : ℝ) : ℂ) • chargeOp q Λ)
```

2.296 PhyslibAlpha.CondensedMatter.QuasiLocal.InvariantLimit

PhyslibAlpha/CondensedMatter/QuasiLocal/InvariantLimit.lean on GitHub.

def invariantLimit [source]

***The invariant limit of a state of a uniform lattice system**: the Følner limit of its averages over the site symmetries.*

```
def invariantLimit (φ : AlgState (uniformNet (S := S) Ω₀).Quasilocal) (F : ι → Finset Γ)
    (hne : ∀ i, (F i).Nonempty) (𝒰 : Ultrafilter ι) :
    AlgState (uniformNet (S := S) Ω₀).Quasilocal
```

lemma invariantLimit_invariant [source]

The invariant limit of a state is invariant under the site symmetries.

```
lemma invariantLimit_invariant (φ : AlgState (uniformNet (S := S) Ω₀).Quasilocal)
    (F : ι → Finset Γ) (hne : ∀ i, (F i).Nonempty) (𝒰 : Ultrafilter ι)
    (hF : IsRightFolner F 𝒰) (γ : Γ) (a : (uniformNet (S := S) Ω₀).Quasilocal) :
    invariantLimit φ F hne 𝒰 (symAction (Ω₀ := Ω₀) γ a) = invariantLimit φ F hne 𝒰 a
```

lemma exists_invariant_state [source]

***Existence of invariant states**: if the finite sets of site symmetries are asymptotically invariant, every state of the quasi-local algebra has an invariant limit state.*

```
lemma exists_invariant_state (φ : AlgState (uniformNet (S := S) Ω₀).Quasilocal)
    (F : ι → Finset Γ) (hne : ∀ i, (F i).Nonempty) (𝒰 : Ultrafilter ι)
    (hF : IsRightFolner F 𝒰) :
    ∃ ψ : AlgState (uniformNet (S := S) Ω₀).Quasilocal,
      ∀ (γ : Γ) a, ψ (symAction (Ω₀ := Ω₀) γ a) = ψ a
```

def invariantLimitState [source]

***The symmetric thermodynamic limit state of the canonical ensemble** of an interaction on a uniform lattice system.*

```
def Interaction.invariantLimitState (I : Interaction (fun _ : S => Ω₀)) (β : ℝ)
    (𝒰₀ : Ultrafilter (Finset S)) (h𝒰₀ : (𝒰₀ : Filter (Finset S)) ≤ atTop) (F : ι → Finset Γ)
    (hne : ∀ i, (F i).Nonempty) (𝒰 : Ultrafilter ι) :
    AlgState (uniformNet (S := S) Ω₀).Quasilocal
```

lemma invariantLimitState_invariant [source]

The symmetric limit state of an interaction is invariant under the site symmetries.

```
lemma Interaction.invariantLimitState_invariant (I : Interaction (fun _ : S => Ω₀)) (β : ℝ)
    (𝒰₀ : Ultrafilter (Finset S)) (h𝒰₀ : (𝒰₀ : Filter (Finset S)) ≤ atTop) (F : ι → Finset Γ)
    (hne : ∀ i, (F i).Nonempty) (𝒰 : Ultrafilter ι) (hF : IsRightFolner F 𝒰) (γ : Γ)
    (a : (uniformNet (S := S) Ω₀).Quasilocal) :
    I.invariantLimitState β 𝒰₀ h𝒰₀ F hne 𝒰 (symAction (Ω₀ := Ω₀) γ a) =
      I.invariantLimitState β 𝒰₀ h𝒰₀ F hne 𝒰 a
```

2.297 PhyslibAlpha.CondensedMatter.QuasiLocal.LimitState

PhyslibAlpha/CondensedMatter/QuasiLocal/LimitState.lean on GitHub.

lemma matrix_exists_sumSq [source]

In a matrix algebra every `a* a` lies below a multiple of the identity by a sum of squares.

```
lemma matrix_exists_sumSq {n : Type*} [Fintype n] [DecidableEq n] (a : Matrix n n ℂ) :
    ∃ c : ℝ, ∃ p ∈ sumSq (Matrix n n ℂ), star a * a + p = (c : ℂ) • (1 : Matrix n n ℂ)
```

lemma quantumNet_isLocallyBounded [source]

The net of operators is locally bounded.

```
lemma quantumNet_isLocallyBounded : (quantumNet Ω).IsLocallyBounded
```

def canonicalLimitState [source]

***The thermodynamic limit state of the canonical ensemble** along an ultrafilter of boxes: a state of the infinite system.*

```
def Interaction.canonicalLimitState (I : Interaction Ω) (β : ℝ) (𝒰 : Ultrafilter (Finset S))
    (h𝒰 : (𝒰 : Filter (Finset S)) ≤ atTop) : AlgState (quantumNet Ω).Quasilocal
```

def grandCanonicalLimitState [source]

***The thermodynamic limit state of the grand canonical ensemble** along an ultrafilter of boxes.*

```
def Interaction.grandCanonicalLimitState (I : Interaction Ω) (q : Charge Ω) (β μ : ℝ)
    (𝒰 : Ultrafilter (Finset S)) (h𝒰 : (𝒰 : Filter (Finset S)) ≤ atTop) :
    AlgState (quantumNet Ω).Quasilocal
```

lemma canonicalLimitState_of [source]

The limit state of the canonical ensemble on a local observable is the limit of the box expectations.

```
lemma Interaction.canonicalLimitState_of (I : Interaction Ω) (β : ℝ) (𝒰 : Ultrafilter (Finset
    S))
    (h𝒰 : (𝒰 : Filter (Finset S)) ≤ atTop) (X : Finset S) (a : Alg Ω X) :
    Tendsto (localSeq (N := quantumNet Ω) (fun Λ => I.canonicalState β Λ) X a) 𝒰
      (nhds (I.canonicalLimitState β 𝒰 h𝒰 ((quantumNet Ω).of X a)))
```

2.298 PhyslibAlpha.CondensedMatter.QuasiLocal.Local

PhyslibAlpha/CondensedMatter/QuasiLocal/Local.lean on GitHub.

abbrev Cfg [source]

The configurations of a region: a value of the local configuration space at each of its sites.

```
abbrev Cfg (X : Finset S) : Type _
```

abbrev restr [source]

The restriction of a configuration of a region to a subregion.

```
abbrev restr {X Y : Finset S} (h : X ≤ Y) (c : Cfg Ω Y) : Cfg Ω X
```

def Agree [source]

Two configurations of a region agree outside a subregion.

```
def Agree {X Y : Finset S} (_h : X ≤ Y) (c c' : Cfg Ω Y) : Prop
```

instance Decidable (Agree h c c') [source]

```
instance {X Y : Finset S} (h : X ≤ Y) (c c' : Cfg Ω Y) : Decidable (Agree h c c')
```

def glue [source]

Replace the values of `c` inside the subregion `X` by those of `a`.

```
def glue {X Y : Finset S} (_h : X ≤ Y) (c : Cfg Ω Y) (a : Cfg Ω X) : Cfg Ω Y
```

lemma restr_glue [source]

```
lemma restr_glue {X Y : Finset S} (h : X ≤ Y) (c : Cfg Ω Y) (a : Cfg Ω X) :
    restr h (glue h c a) = a
```

lemma agree_glue [source]

```
lemma agree_glue {X Y : Finset S} (h : X ≤ Y) (c : Cfg Ω Y) (a : Cfg Ω X) :
    Agree h c (glue h c a)
```

lemma agree_iff_glue [source]

```
lemma agree_iff_glue {X Y : Finset S} (h : X ≤ Y) {c c' : Cfg Ω Y} :
    Agree h c c' ↔ glue h c (restr h c') = c'
```

lemma agree_refl [source]

```
lemma agree_refl {X Y : Finset S} (h : X ≤ Y) (c : Cfg Ω Y) : Agree h c c
```

lemma symm [source]

```
lemma Agree.symm {X Y : Finset S} {h : X ≤ Y} {c c' : Cfg Ω Y} (hc : Agree h c c') :
    Agree h c' c
```

lemma trans [source]

```
lemma Agree.trans {X Y : Finset S} {h : X ≤ Y} {c c' c'' : Cfg Ω Y} (hc : Agree h c c')
    (hc' : Agree h c' c'') : Agree h c c''
```

lemma eq_of_agree_of_restr_eq [source]

```
lemma eq_of_agree_of_restr_eq {X Y : Finset S} {h : X ≤ Y} {c c' : Cfg Ω Y} (hc : Agree h c c')
    (hr : restr h c = restr h c') : c = c'
```

abbrev Alg [source]

The operators on the sites of a region: matrices indexed by configurations.

```
abbrev Alg (X : Finset S) : Type _
```

def inclMat [source]

The operator `M ⊗ 1` on a larger region: `M` on the sites of `X` and the identity on the other sites.

```
def inclMat {X Y : Finset S} (h : X ≤ Y) (M : Alg Ω X) : Alg Ω Y
```

lemma inclMat_apply [source]

```
lemma inclMat_apply {X Y : Finset S} (h : X ≤ Y) (M : Alg Ω X) (c c' : Cfg Ω Y) :
    inclMat h M c c' = if Agree h c c' then M (restr h c) (restr h c') else 0
```

lemma inclMat_one [source]

```
lemma inclMat_one {X Y : Finset S} (h : X ≤ Y) : inclMat h (1 : Alg Ω X) = 1
```

lemma inclMat_add [source]

```
lemma inclMat_add {X Y : Finset S} (h : X ≤ Y) (M N : Alg Ω X) :
    inclMat h (M + N) = inclMat h M + inclMat h N
```

lemma inclMat_smul [source]

```
lemma inclMat_smul {X Y : Finset S} (h : X ≤ Y) (r : ℂ) (M : Alg Ω X) :
    inclMat h (r • M) = r • inclMat h M
```

lemma inclMat_conjTranspose [source]

```
lemma inclMat_conjTranspose {X Y : Finset S} (h : X ≤ Y) (M : Alg Ω X) :
    inclMat h M^H = (inclMat h M)^H
```

lemma inclMat_mul [source]

```
lemma inclMat_mul {X Y : Finset S} (h : X ≤ Y) (M N : Alg Ω X) :
    inclMat h (M * N) = inclMat h M * inclMat h N
```

lemma inclMat_refl [source]

```
lemma inclMat_refl {X : Finset S} (M : Alg Ω X) : inclMat (le_refl X) M = M
```

lemma agree_trans_iff [source]

```
lemma agree_trans_iff {X Y Z : Finset S} (h₁ : X ≤ Y) (h₂ : Y ≤ Z) (c c' : Cfg Ω Z) :
    Agree (h₁.trans h₂) c c' ↔ Agree h₂ c c' ∧ Agree h₁ (restr h₂ c) (restr h₂ c')
```

lemma inclMat_trans [source]

```
lemma inclMat_trans {X Y Z : Finset S} (h₁ : X ≤ Y) (h₂ : Y ≤ Z) (M : Alg Ω X) :
    inclMat (h₁.trans h₂) M = inclMat h₂ (inclMat h₁ M)
```

def inclStarAlg [source]

*The operator `M ⊗ 1` as a unital *-homomorphism.*

```
def inclStarAlg {X Y : Finset S} (h : X ≤ Y) : Alg Ω X →⋆ₐ[ℂ] Alg Ω Y where
```

2.299 `PhyslibAlpha.CondensedMatter.QuasiLocal.Symmetry`

PhyslibAlpha/CondensedMatter/QuasiLocal/Symmetry.lean on GitHub.

def cfgEquiv [source]

A configuration of the region $\gamma \bullet X$ is a configuration of X: read it at $\gamma \bullet x$.

```
def cfgEquiv (γ : Γ) (X : Finset S) : Cfg (fun _ : S => Ω₀) (γ • X) ≃ Cfg (fun _ : S => Ω₀) X
    where
```

def quantumIso [source]

The operator of $\gamma \bullet X$ that acts as M after moving the configurations back to X.

```
def quantumIso (γ : Γ) (X : Finset S) :
    Alg (fun _ : S => Ω₀) X →⋆ₐ[ℂ] Alg (fun _ : S => Ω₀) (γ • X) where
```

lemma smul_mono [source]

```
lemma smul_mono (γ : Γ) {X Y : Finset S} (h : X ≤ Y) : γ • X ≤ γ • Y
```

lemma restr_cfgEquiv [source]

```
lemma restr_cfgEquiv (γ : Γ) {X Y : Finset S} (h : X ≤ Y)
    (c : Cfg (fun _ : S => Ω₀) (γ • Y)) :
    restr h (cfgEquiv Γ Ω₀ γ Y c) = cfgEquiv Γ Ω₀ γ X (restr (smul_mono γ h) c)
```

lemma agree_cfgEquiv [source]

```
lemma agree_cfgEquiv (γ : Γ) {X Y : Finset S} (h : X ≤ Y)
    (c c' : Cfg (fun _ : S => Ω₀) (γ • Y)) :
    Agree h (cfgEquiv Γ Ω₀ γ Y c) (cfgEquiv Γ Ω₀ γ Y c') ↔ Agree (smul_mono γ h) c c'
```

lemma quantumIso_incl [source]

***Naturality**: moving operators commutes with extending them by the identity.*

```
lemma quantumIso_incl (γ : Γ) {X Y : Finset S} (h : X ≤ Y) (M : Alg (fun _ : S => Ω₀) X) :
    quantumIso γ Y (inclMat h M) = inclMat (smul_mono γ h) (quantumIso γ X M)
```

def cfgCast [source]

Transport of configurations along an equality of regions.

```
def cfgCast {X Y : Finset S} (e : X = Y) (c : Cfg (fun _ : S => Ω₀) Y) :
    Cfg (fun _ : S => Ω₀) X
```

def regionIso [source]

Transport of operators along an equality of regions.

```
def regionIso {X Y : Finset S} (e : X = Y) :
    Alg (fun _ : S => Ω₀) X →⋆ₐ[ℂ] Alg (fun _ : S => Ω₀) Y
```

lemma regionIso_apply [source]

```
lemma regionIso_apply {X Y : Finset S} (e : X = Y) (M : Alg (fun _ : S => Ω₀) X)
    (c c' : Cfg (fun _ : S => Ω₀) Y) :
    regionIso e M c c' = M (cfgCast e c) (cfgCast e c')
```

abbrev uniformNet [source]

The quantum net of a system whose sites all carry the same finite configuration space.

```
abbrev uniformNet (Ω₀ : Type*) [Fintype Ω₀] [DecidableEq Ω₀] : AlgebraNet S
```

lemma of_regionIso [source]

```
lemma of_regionIso {X Y : Finset S} (e : X = Y) (M : Alg (fun _ : S => Ω₀) X) :
    (uniformNet Ω₀).of Y (regionIso e M) = (uniformNet Ω₀).of X M
```

def symAct [source]

***The action of a site symmetry on the quasi-local algebra**: an endomorphism that moves the observables of a region `X` to those of `γ • X`.*

```
def symAct (γ : Γ) :
    (uniformNet (S := S) Ω₀).Quasilocal →⋆ₐ[ℂ] (uniformNet (S := S) Ω₀).Quasilocal
```

lemma symAct_of [source]

```
lemma symAct_of (γ : Γ) (X : Finset S) (M : Alg (fun _ : S => Ω₀) X) :
    symAct (Ω₀ := Ω₀) γ ((uniformNet Ω₀).of X M) = (uniformNet Ω₀).of (γ • X) (quantumIso γ X M
    )
```

lemma quantumIso_mul [source]

```
lemma quantumIso_mul (γ₁ γ₂ : Γ) (X : Finset S) (M : Alg (fun _ : S => Ω₀) X) :
    regionIso (mul_smul γ₁ γ₂ X) (quantumIso (γ₁ * γ₂) X M) =
      quantumIso γ₁ (γ₂ • X) (quantumIso γ₂ X M)
```

lemma quantumIso_one [source]

```
lemma quantumIso_one (X : Finset S) (M : Alg (fun _ : S => Ω₀) X) :
    regionIso (one_smul Γ X) (quantumIso (1 : Γ) X M) = M
```

lemma symAct_mul [source]

The action of site symmetries is multiplicative.

```
lemma symAct_mul (γ₁ γ₂ : Γ) :
    symAct (Ω₀ := Ω₀) (S := S) (γ₁ * γ₂) = (symAct γ₁).comp (symAct γ₂)
```

lemma symAct_one [source]

```
lemma symAct_one : symAct (Ω₀ := Ω₀) (S := S) (1 : Γ) = StarAlgHom.id ℂ _
```

def symEquiv [source]

A group acting on the sites acts on the quasi-local algebra by automorphisms.

```
def symEquiv (γ : Γ) : (uniformNet (S := S) Ω₀).Quasilocal ≃⋆ₐ[ℂ]
    (uniformNet (S := S) Ω₀).Quasilocal
```

def symAction [source]

***The symmetry action** of a group on the sites, as a homomorphism to the automorphisms of the quasi-local algebra.*

```
def symAction : Γ →* ((uniformNet (S := S) Ω₀).Quasilocal ≃⋆ₐ[ℂ]
    (uniformNet (S := S) Ω₀).Quasilocal) where
```

2.300 PhyslibAlpha.CondensedMatter.TightBindingChain.Cube.AxisCommutation

PhyslibAlpha/CondensedMatter/TightBindingChain/Cube/AxisCommutation.lean on GitHub.

lemma hopping_eq_zero_of_degenerate [source]

Without hopping, or with a single site, the hopping operator is zero.

```
lemma hopping_eq_zero_of_degenerate (T : TightBindingChain) (h : T.t = 0 ∨ T.N < 2) :
    T.hopping = 0
```

lemma bracket_openHamiltonian_position_eq_zero_iff [source]

On one chain, the Hamiltonian and the position commute if and only if there is no hopping or only one site.

```
lemma bracket_openHamiltonian_position_eq_zero_iff (T : TightBindingChain) :
    ⁅T.openHamiltonianObservable, T.positionObservable⁆ = 0 ↔ T.t = 0 ∨ T.N < 2
```

lemma axisBracket_maxCurrentCubeVectorState_ne_zero [source]

In the maximal current state of the cube, the expectation of `⁅H, X⁆` is nonzero on each axis.

```
lemma axisBracket_maxCurrentCubeVectorState_ne_zero (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0)
    (hz : Tz.t ≠ 0) (hNx : 2 ≤ Tx.N) (hNy : 2 ≤ Ty.N) (hNz : 2 ≤ Tz.N) (k : Fin 3) :
    Tx.axisBracket Ty Tz (Tx.maxCurrentCubeVectorState Ty Tz) k ≠ 0
```

lemma bracket_cubePairs_pair_ne_zero [source]

The Hamiltonian and the position of one axis of the cube do not commute.

```
lemma bracket_cubePairs_pair_ne_zero (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0) (hz : Tz.t ≠ 0)
    (hNx : 2 ≤ Tx.N) (hNy : 2 ≤ Ty.N) (hNz : 2 ≤ Tz.N) (k : Fin 3) :
    ⁅Tx.cubePairs Ty Tz (0, k), Tx.cubePairs Ty Tz (1, k)⁆ ≠ 0
```

lemma variance_mul_variance_cubePairs_pos [source]

In the maximal current state of the cube, the variances of `H` and `X` of each axis have a positive product.

```
lemma variance_mul_variance_cubePairs_pos (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0) (hz : Tz.t ≠ 0)
    (hNx : 2 ≤ Tx.N) (hNy : 2 ≤ Ty.N) (hNz : 2 ≤ Tz.N) (k : Fin 3) :
    0 < variance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.cubePairs Ty Tz (0, k)) *
      variance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.cubePairs Ty Tz (1, k))
```

lemma bracket_cubePairs_eq_zero_iff [source]

Two distinct observables of the cube commute if and only if they lie on different axes.

```
lemma bracket_cubePairs_eq_zero_iff (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0) (hz : Tz.t ≠ 0)
    (hNx : 2 ≤ Tx.N) (hNy : 2 ≤ Ty.N) (hNz : 2 ≤ Tz.N) {p q : Fin 2 × Fin 3} (h : p ≠ q) :
    ⁅Tx.cubePairs Ty Tz p, Tx.cubePairs Ty Tz q⁆ = 0 ↔ p.2 ≠ q.2
```

2.301 PhyslibAlpha.CondensedMatter.TightBindingChain.Cube.Basic

PhyslibAlpha/CondensedMatter/TightBindingChain/Cube/Basic.lean on GitHub.

abbrev CubeHilbertSpace [source]

The Hilbert space of the cube: states on the sites `(nx, ny, nz)`.

```
abbrev CubeHilbertSpace
```

abbrev CubeOperators [source]

The operators on the Hilbert space of the cube.

```
abbrev CubeOperators
```

abbrev alongX [source]

An observable of the chain `Tx` acting on the `x` axis of the cube.

```
noncomputable abbrev alongX (a : Observable (Tx.HilbertSpace →L[ℂ] Tx.HilbertSpace)) :
    Observable (Tx.CubeOperators Ty Tz)
```

abbrev alongY [source]

An observable of the chain `Ty` acting on the `y` axis of the cube.

```
noncomputable abbrev alongY (b : Observable (Ty.HilbertSpace →L[ℂ] Ty.HilbertSpace)) :
    Observable (Tx.CubeOperators Ty Tz)
```

abbrev alongZ [source]

An observable of the chain `Tz` acting on the `z` axis of the cube.

```
noncomputable abbrev alongZ (c : Observable (Tz.HilbertSpace →L[ℂ] Tz.HilbertSpace)) :
    Observable (Tx.CubeOperators Ty Tz)
```

lemma bracket_eq_zero_of_mul_comm [source]

Commuting observables have a vanishing bracket.

```
private lemma bracket_eq_zero_of_mul_comm {A : Type*} [CStarAlgebra A]
    {a b : Observable A} (h : (a : A) * b = b * a) : ⁅a, b⁆ = 0
```

lemma bracket_alongX_alongY [source]

Observables of the `x` and `y` axes commute.

```
lemma bracket_alongX_alongY (a : Observable (Tx.HilbertSpace →L[ℂ] Tx.HilbertSpace))
    (b : Observable (Ty.HilbertSpace →L[ℂ] Ty.HilbertSpace)) :
    ⁅Tx.alongX Ty Tz a, Tx.alongY Ty Tz b⁆ = 0
```

lemma bracket_alongX_alongZ [source]

Observables of the `x` and `z` axes commute.

```
lemma bracket_alongX_alongZ (a : Observable (Tx.HilbertSpace →L[ℂ] Tx.HilbertSpace))
    (c : Observable (Tz.HilbertSpace →L[ℂ] Tz.HilbertSpace)) :
    ⁅Tx.alongX Ty Tz a, Tx.alongZ Ty Tz c⁆ = 0
```

lemma bracket_alongY_alongZ [source]

Observables of the `y` and `z` axes commute.

```
lemma bracket_alongY_alongZ (b : Observable (Ty.HilbertSpace →L[ℂ] Ty.HilbertSpace))
    (c : Observable (Tz.HilbertSpace →L[ℂ] Tz.HilbertSpace)) :
    ⁅Tx.alongY Ty Tz b, Tx.alongZ Ty Tz c⁆ = 0
```

def maxCurrentCubeState [source]

The maximal current state of the cube, the product of those of the three chains.

```
noncomputable def maxCurrentCubeState : Tx.CubeHilbertSpace Ty Tz
```

lemma norm_maxCurrentCubeState [source]

```
lemma norm_maxCurrentCubeState : ‖Tx.maxCurrentCubeState Ty Tz‖ = 1
```

abbrev maxCurrentCubeVectorState [source]

The maximal current state of the cube as a state on its operators.

```
noncomputable abbrev maxCurrentCubeVectorState : 𝓢[ℂ, Tx.CubeOperators Ty Tz]
```

lemma variance_alongX [source]

Along `x`, the cube sees the statistics of the chain `Tx`.

```
lemma variance_alongX :
    variance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongX Ty Tz a) =
      variance Tx.maxCurrentVectorState a
```

lemma centeredGramDefect_alongX [source]

```
lemma centeredGramDefect_alongX :
    centeredGramDefect (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongX Ty Tz a)
      (Tx.alongX Ty Tz b) = centeredGramDefect Tx.maxCurrentVectorState a b
```

lemma variance_alongY [source]

Along `y`, the cube sees the statistics of the chain `Ty`.

```
lemma variance_alongY :
    variance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongY Ty Tz b') =
      variance Ty.maxCurrentVectorState b'
```

lemma centeredGramDefect_alongY [source]

```
lemma centeredGramDefect_alongY :
    centeredGramDefect (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongY Ty Tz b')
      (Tx.alongY Ty Tz c') = centeredGramDefect Ty.maxCurrentVectorState b' c'
```

lemma variance_alongZ [source]

Along `z`, the cube sees the statistics of the chain `Tz`.

```
lemma variance_alongZ :
    variance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongZ Ty Tz b'') =
      variance Tz.maxCurrentVectorState b''
```

lemma centeredGramDefect_alongZ [source]

```
lemma centeredGramDefect_alongZ :
    centeredGramDefect (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongZ Ty Tz b'')
      (Tx.alongZ Ty Tz c'') = centeredGramDefect Tz.maxCurrentVectorState b'' c''
```

lemma uncertaintyRatio_alongX [source]

The `x` axis of the cube carries the constant `C` of the chain `Tx`.

```
lemma uncertaintyRatio_alongX :
    √(variance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongX Ty Tz Tx.
    openHamiltonianObservable) *
      variance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongX Ty Tz Tx.positionObservable)) /
        |Tx.a * Tx.t * Real.cos (Real.pi / (Tx.N + 1))| = Tx.uncertaintyRatio
```

lemma uncertaintyRatio_alongY [source]

The `y` axis of the cube carries the constant `C` of the chain `Ty`.

```
lemma uncertaintyRatio_alongY :
    √(variance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongY Ty Tz Ty.
    openHamiltonianObservable) *
      variance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongY Ty Tz Ty.positionObservable)) /
        |Ty.a * Ty.t * Real.cos (Real.pi / (Ty.N + 1))| = Ty.uncertaintyRatio
```

lemma uncertaintyRatio_alongZ [source]

The `z` axis of the cube carries the constant `C` of the chain `Tz`.

```
lemma uncertaintyRatio_alongZ :
    √(variance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongZ Ty Tz Tz.
    openHamiltonianObservable) *
      variance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongZ Ty Tz Tz.positionObservable)) /
        |Tz.a * Tz.t * Real.cos (Real.pi / (Tz.N + 1))| = Tz.uncertaintyRatio
```

lemma centeredGramDefect_alongX_eq_zero_iff [source]

Along `x`, the centered Gram defect vanishes, that is the uncertainty relation is an equality, iff the axis has `2` or `3` sites.

```
lemma centeredGramDefect_alongX_eq_zero_iff (ht : Tx.t ≠ 0) (hN : 2 ≤ Tx.N) :
    centeredGramDefect (Tx.maxCurrentCubeVectorState Ty Tz)
      (Tx.alongX Ty Tz Tx.openHamiltonianObservable) (Tx.alongX Ty Tz Tx.positionObservable) =
    0 ↔
      Tx.N = 2 ∨ Tx.N = 3
```

lemma centeredGramDefect_alongY_eq_zero_iff [source]

Along `y`, the uncertainty relation is an equality iff the axis has `2` or `3` sites.

```
lemma centeredGramDefect_alongY_eq_zero_iff (ht : Ty.t ≠ 0) (hN : 2 ≤ Ty.N) :
    centeredGramDefect (Tx.maxCurrentCubeVectorState Ty Tz)
      (Tx.alongY Ty Tz Ty.openHamiltonianObservable) (Tx.alongY Ty Tz Ty.positionObservable) =
    0 ↔
      Ty.N = 2 ∨ Ty.N = 3
```

lemma centeredGramDefect_alongZ_eq_zero_iff [source]

Along `z`, the uncertainty relation is an equality iff the axis has `2` or `3` sites.

```
lemma centeredGramDefect_alongZ_eq_zero_iff (ht : Tz.t ≠ 0) (hN : 2 ≤ Tz.N) :
    centeredGramDefect (Tx.maxCurrentCubeVectorState Ty Tz)
      (Tx.alongZ Ty Tz Tz.openHamiltonianObservable) (Tx.alongZ Ty Tz Tz.positionObservable) =
    0 ↔
      Tz.N = 2 ∨ Tz.N = 3
```

lemma robertson_schrodinger_cube [source]

Robertson–Schrödinger on the cube. From `4 × 4 × 4` on, the uncertainty relation of the maximal current state is strict on the three axes at once: the centered Gram defect of each axis is positive.

```
lemma robertson_schrodinger_cube (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0) (hz : Tz.t ≠ 0)
    (hNx : 4 ≤ Tx.N) (hNy : 4 ≤ Ty.N) (hNz : 4 ≤ Tz.N) :
    0 < centeredGramDefect (Tx.maxCurrentCubeVectorState Ty Tz)
        (Tx.alongX Ty Tz Tx.openHamiltonianObservable) (Tx.alongX Ty Tz Tx.positionObservable)
    ∧
      0 < centeredGramDefect (Tx.maxCurrentCubeVectorState Ty Tz)
        (Tx.alongY Ty Tz Ty.openHamiltonianObservable) (Tx.alongY Ty Tz Ty.positionObservable)
    ∧
      0 < centeredGramDefect (Tx.maxCurrentCubeVectorState Ty Tz)
```

```
    (Tx.alongZ Ty Tz Tz.openHamiltonianObservable) (Tx.alongZ Ty Tz Tz.positionObservable)
```

def conjugatePair [source]

The conjugate pair of a chain: its Hamiltonian and its position.

```
noncomputable def conjugatePair (T : TightBindingChain) :
    Fin 2 → Observable (T.HilbertSpace →L[ℂ] T.HilbertSpace)
```

def cubePairs [source]

The six observables of the cube, the conjugate pair `(H, X)` of each axis: `(j, k)` is the `j`-th member of the pair of the axis `k`.

```
noncomputable def cubePairs : Fin 2 × Fin 3 → Observable (Tx.CubeOperators Ty Tz)
```

def axisBracket [source]

The expectation `ω⟦H, X⟧` of the bracket of the pair of the axis `k`.

```
noncomputable def axisBracket (ω : 𝒮[ℂ, Tx.CubeOperators Ty Tz]) (k : Fin 3) : ℝ
```

lemma cubePairs_x [source]

```
@[simp]
lemma cubePairs_x (j : Fin 2) :
    Tx.cubePairs Ty Tz (j, 0) = Tx.alongX Ty Tz (Tx.conjugatePair j)
```

lemma cubePairs_y [source]

```
@[simp]
lemma cubePairs_y (j : Fin 2) :
    Tx.cubePairs Ty Tz (j, 1) = Tx.alongY Ty Tz (Ty.conjugatePair j)
```

lemma cubePairs_z [source]

```
@[simp]
lemma cubePairs_z (j : Fin 2) :
    Tx.cubePairs Ty Tz (j, 2) = Tx.alongZ Ty Tz (Tz.conjugatePair j)
```

lemma bracket_swap [source]

Swapping a bracket flips its sign.

```
private lemma bracket_swap {A : Type*} [CStarAlgebra A] (a b : Observable A) :
    ⁅b, a⁆ = -⁅a, b⁆
```

lemma bracket_swap_eq_zero [source]

```
private lemma bracket_swap_eq_zero {A : Type*} [CStarAlgebra A] {a b : Observable A}
    (h : ⁅a, b⁆ = 0) : ⁅b, a⁆ = 0
```

lemma bracket_cubePairs_of_ne [source]

Observables of different axes of the cube commute.

```
lemma bracket_cubePairs_of_ne {p q : Fin 2 × Fin 3} (h : p.2 ≠ q.2) :
    ⁅Tx.cubePairs Ty Tz p, Tx.cubePairs Ty Tz q⁆ = 0
```

lemma expectation_zero [source]

```
private lemma expectation_zero {A : Type*} [CStarAlgebra A] [PartialOrder A]
    [StarOrderedRing A] (ω : 𝒮[ℂ, A]) : ω⟨(0 : Observable A)⟩ = 0
```

lemma expectation_neg [source]

```
private lemma expectation_neg {A : Type*} [CStarAlgebra A] [PartialOrder A]
    [StarOrderedRing A] (ω : 𝒮[ℂ, A]) (a : Observable A) : ω⟨-a⟩ = -ω⟨a⟩
```

lemma bracketMatrix_cubePairs [source]

In every state of the cube the bracket matrix `Ω` of the six observables is block diagonal, one antisymmetric `2 × 2` block per axis.

```
lemma bracketMatrix_cubePairs (ω : 𝒮[ℂ, Tx.CubeOperators Ty Tz]) :
    bracketMatrix ω (Tx.cubePairs Ty Tz) =
      blockDiagonal fun k => !![0, Tx.axisBracket Ty Tz ω k; -Tx.axisBracket Ty Tz ω k, 0]
```

lemma robertson_det_cube [source]

***Robertson's relation on the cube.** In every state of the cube the volume `det Σ` of the dispersion of the six observables is at least the product of the squared brackets of the three axes.*

```
lemma robertson_det_cube (ω : 𝒮[ℂ, Tx.CubeOperators Ty Tz]) :
    (Tx.axisBracket Ty Tz ω 0 * Tx.axisBracket Ty Tz ω 1 * Tx.axisBracket Ty Tz ω 2) ^ 2 ≤
      (covarianceMatrix ω (Tx.cubePairs Ty Tz)).det
```

lemma covariance_alongX [source]

```
lemma covariance_alongX :
    covariance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongX Ty Tz a) (Tx.alongX Ty Tz b) =
      covariance Tx.maxCurrentVectorState a b
```

lemma covariance_alongY [source]

```
lemma covariance_alongY :
    covariance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongY Ty Tz b') (Tx.alongY Ty Tz c') =
      covariance Ty.maxCurrentVectorState b' c'
```

lemma covariance_alongZ [source]

```
lemma covariance_alongZ :
    covariance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongZ Ty Tz b'') (Tx.alongZ Ty Tz c'')
     =
      covariance Tz.maxCurrentVectorState b'' c''
```

lemma covariance_alongX_alongY [source]

In the maximal current state of the cube the `x` and `y` axes are uncorrelated.

```
lemma covariance_alongX_alongY :
    covariance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongX Ty Tz a) (Tx.alongY Ty Tz b') =
    0
```

lemma covariance_alongX_alongZ [source]

In the maximal current state of the cube the `x` and `z` axes are uncorrelated.

```
lemma covariance_alongX_alongZ :
    covariance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongX Ty Tz a) (Tx.alongZ Ty Tz b'') =
     0
```

lemma covariance_alongY_alongZ [source]

In the maximal current state of the cube the `y` and `z` axes are uncorrelated.

```
lemma covariance_alongY_alongZ :
    covariance (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.alongY Ty Tz b') (Tx.alongZ Ty Tz b'')
    =
      0
```

lemma covarianceMatrix_maxCurrentCubeState [source]

In the maximal current state of the cube the covariance matrix `Σ` of the six observables is block diagonal, the block of each axis being the covariance matrix of its chain.

```
lemma covarianceMatrix_maxCurrentCubeState :
    covarianceMatrix (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.cubePairs Ty Tz) =
      blockDiagonal ![covarianceMatrix Tx.maxCurrentVectorState Tx.conjugatePair,
        covarianceMatrix Ty.maxCurrentVectorState Ty.conjugatePair,
        covarianceMatrix Tz.maxCurrentVectorState Tz.conjugatePair]
```

lemma det_covarianceMatrix_conjugatePair [source]

In the maximal current state of a chain, `det Σ = C² (a t cos (π / (N + 1)))²` for the pair `(H, X)`.

```
lemma det_covarianceMatrix_conjugatePair (T : TightBindingChain) (ht : T.t ≠ 0) (hN : 2 ≤ T.N)
    :
    (covarianceMatrix T.maxCurrentVectorState T.conjugatePair).det =
      T.uncertaintyRatio ^ 2 * (T.a * T.t * Real.cos (Real.pi / (T.N + 1))) ^ 2
```

lemma sq_axisBracket_maxCurrentCubeState [source]

In the maximal current state of the cube, the bracket of each axis is that of its chain, `± a t cos (π / (N + 1))`.

```
lemma sq_axisBracket_maxCurrentCubeState :
    (Tx.axisBracket Ty Tz (Tx.maxCurrentCubeVectorState Ty Tz) 0 *
        Tx.axisBracket Ty Tz (Tx.maxCurrentCubeVectorState Ty Tz) 1 *
        Tx.axisBracket Ty Tz (Tx.maxCurrentCubeVectorState Ty Tz) 2) ^ 2 =
      (Tx.a * Tx.t * Real.cos (Real.pi / (Tx.N + 1)) *
```

```
      (Ty.a * Ty.t * Real.cos (Real.pi / (Ty.N + 1))) *
      (Tz.a * Tz.t * Real.cos (Real.pi / (Tz.N + 1)))) ^ 2
```

lemma det_covarianceMatrix_maxCurrentCubeState [source]

In the maximal current state of the cube, `det Σ = C(x)² C(y)² C(z)² |det Ω|`: the ratio of the two sides of Robertson's relation is the product of the constants of the three axes.

```
lemma det_covarianceMatrix_maxCurrentCubeState (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0) (hz : Tz.t ≠ 0)
    (hNx : 2 ≤ Tx.N) (hNy : 2 ≤ Ty.N) (hNz : 2 ≤ Tz.N) :
    (covarianceMatrix (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.cubePairs Ty Tz)).det =
      (Tx.uncertaintyRatio * Ty.uncertaintyRatio * Tz.uncertaintyRatio) ^ 2 *
        (Tx.axisBracket Ty Tz (Tx.maxCurrentCubeVectorState Ty Tz) 0 *
          Tx.axisBracket Ty Tz (Tx.maxCurrentCubeVectorState Ty Tz) 1 *
          Tx.axisBracket Ty Tz (Tx.maxCurrentCubeVectorState Ty Tz) 2) ^ 2
```

lemma robertson_det_maxCurrentCubeState_strict [source]

***Robertson's relation is strict in the maximal current state of the cube.** From `4 × 4 × 4` on, `|det Ω| < det Σ` for the six observables of the cube.*

```
lemma robertson_det_maxCurrentCubeState_strict (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0)
    (hz : Tz.t ≠ 0) (hNx : 4 ≤ Tx.N) (hNy : 4 ≤ Ty.N) (hNz : 4 ≤ Tz.N) :
    |(bracketMatrix (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.cubePairs Ty Tz)).det| <
      (covarianceMatrix (Tx.maxCurrentCubeVectorState Ty Tz) (Tx.cubePairs Ty Tz)).det
```

2.302 PhyslibAlpha.CondensedMatter.TightBindingChain.Cube.MandelstamTamm

PhyslibAlpha/CondensedMatter/TightBindingChain/Cube/MandelstamTamm.lean on GitHub.

lemma mandelstamTammRatio_alongX [source]

Along `x`, the maximal current state of the cube has the ratio `1 / C²` of `Tx`.

```
lemma mandelstamTammRatio_alongX :
    (Tx.maxCurrentCubeVectorState Ty Tz)(Tx.alongX Ty Tz Tx.currentObservable) ^ 2 /
      (4 * (variance (Tx.maxCurrentCubeVectorState Ty Tz)
          (Tx.alongX Ty Tz Tx.openHamiltonianObservable) *
        variance (Tx.maxCurrentCubeVectorState Ty Tz)
          (Tx.alongX Ty Tz Tx.positionObservable))) = 1 / Tx.uncertaintyRatio ^ 2
```

lemma mandelstamTammRatio_alongY [source]

Along `y`, the maximal current state of the cube has the ratio `1 / C²` of `Ty`.

```
lemma mandelstamTammRatio_alongY :
    (Tx.maxCurrentCubeVectorState Ty Tz)(Tx.alongY Ty Tz Ty.currentObservable) ^ 2 /
      (4 * (variance (Tx.maxCurrentCubeVectorState Ty Tz)
          (Tx.alongY Ty Tz Ty.openHamiltonianObservable) *
        variance (Tx.maxCurrentCubeVectorState Ty Tz)
          (Tx.alongY Ty Tz Ty.positionObservable))) = 1 / Ty.uncertaintyRatio ^ 2
```

lemma mandelstamTammRatio_alongZ [source]

Along `z`, the maximal current state of the cube has the ratio `1 / C²` of `Tz`.

```
lemma mandelstamTammRatio_alongZ :
    (Tx.maxCurrentCubeVectorState Ty Tz)(Tx.alongZ Ty Tz Tz.currentObservable) ^ 2 /
      (4 * (variance (Tx.maxCurrentCubeVectorState Ty Tz)
          (Tx.alongZ Ty Tz Tz.openHamiltonianObservable) *
        variance (Tx.maxCurrentCubeVectorState Ty Tz)
          (Tx.alongZ Ty Tz Tz.positionObservable))) = 1 / Tz.uncertaintyRatio ^ 2
```

2.303 PhyslibAlpha.CondensedMatter.TightBindingChain.Cube.VolumetricQuantum

PhyslibAlpha/CondensedMatter/TightBindingChain/Cube/VolumetricQuantum.lean on GitHub.

def volQuantum [source]

The volumetric quantum of the cube `Nx × Ny × Nz`: the product of the dimensional quanta of its axes, each axis carrying its own `C` (`uncertaintyRatio_alongX`, `uncertaintyRatio_alongY`, `uncertaintyRatio_alongZ`).

```
noncomputable def volQuantum : ℝ
```

lemma volQuantum_eq_zero_iff [source]

The volumetric quantum vanishes iff some axis has `2` or `3` sites.

```
lemma volQuantum_eq_zero_iff (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0) (hz : Tz.t ≠ 0)
    (hNx : 2 ≤ Tx.N) (hNy : 2 ≤ Ty.N) (hNz : 2 ≤ Tz.N) :
    volQuantum Tx Ty Tz = 0 ↔
      (Tx.N = 2 ∨ Tx.N = 3) ∨ (Ty.N = 2 ∨ Ty.N = 3) ∨ (Tz.N = 2 ∨ Tz.N = 3)
```

lemma volQuantum_pos [source]

From `4 × 4 × 4` on, the volumetric quantum is positive.

```
lemma volQuantum_pos (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0) (hz : Tz.t ≠ 0)
    (hNx : 4 ≤ Tx.N) (hNy : 4 ≤ Ty.N) (hNz : 4 ≤ Tz.N) : 0 < volQuantum Tx Ty Tz
```

lemma volQuantum_lt_volQuantum_x [source]

From `4 × 4 × 4` on, the volumetric quantum grows strictly along `x`.

```
lemma volQuantum_lt_volQuantum_x {Tx' : TightBindingChain} (hx : Tx.t ≠ 0) (hx' : Tx'.t ≠ 0)
    (hy : Ty.t ≠ 0) (hz : Tz.t ≠ 0) (hNx : 4 ≤ Tx.N) (hNy : 4 ≤ Ty.N) (hNz : 4 ≤ Tz.N)
    (h : Tx.N < Tx'.N) : volQuantum Tx Ty Tz < volQuantum Tx' Ty Tz
```

lemma volQuantum_lt_volQuantum_y [source]

From `4 × 4 × 4` on, the volumetric quantum grows strictly along `y`.

```
lemma volQuantum_lt_volQuantum_y {Ty' : TightBindingChain} (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0)
    (hy' : Ty'.t ≠ 0) (hz : Tz.t ≠ 0) (hNx : 4 ≤ Tx.N) (hNy : 4 ≤ Ty.N) (hNz : 4 ≤ Tz.N)
    (h : Ty.N < Ty'.N) : volQuantum Tx Ty Tz < volQuantum Tx Ty' Tz
```

lemma volQuantum_lt_volQuantum_z [source]

From `4 × 4 × 4` on, the volumetric quantum grows strictly along `z`.

```
lemma volQuantum_lt_volQuantum_z {Tz' : TightBindingChain} (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0)
    (hz : Tz.t ≠ 0) (hz' : Tz'.t ≠ 0) (hNx : 4 ≤ Tx.N) (hNy : 4 ≤ Ty.N) (hNz : 4 ≤ Tz.N)
    (h : Tz.N < Tz'.N) : volQuantum Tx Ty Tz < volQuantum Tx Ty Tz'
```

lemma volQuantum_lt [source]

From `4 × 4 × 4` on, the volumetric quantum stays below `(√(π² / 3 - 2) - 1)³`.

```
lemma volQuantum_lt (hx : Tx.t ≠ 0) (hy : Ty.t ≠ 0) (hz : Tz.t ≠ 0)
    (hNx : 4 ≤ Tx.N) (hNy : 4 ≤ Ty.N) (hNz : 4 ≤ Tz.N) :
    volQuantum Tx Ty Tz < (√(Real.pi ^ 2 / 3 - 2) - 1) ^ 3
```

2.304 PhyslibAlpha.CondensedMatter.TightBindingChain.Current.Basic

PhyslibAlpha/CondensedMatter/TightBindingChain/Current/Basic.lean on GitHub.

def current [source]

The current operator `J = i (H X - X H)` of the open chain, the velocity `dX/dt`.

```
noncomputable def current : T.HilbertSpace →ₗ[ℂ] T.HilbertSpace
```

lemma current_hermitian [source]

The current operator is hermitian.

```
lemma current_hermitian (ψ φ : T.HilbertSpace) :
    ⟪T.current ψ, φ⟫_ℂ = ⟪ψ, T.current φ⟫_ℂ
```

lemma inner_current [source]

The matrix elements of `J` are those of `H`, weighted by `i` times the hopping distance.

```
lemma inner_current (m n : Fin T.N) :
    ⟪⟨m|, T.current |n⟩⟫_ℂ =
      Complex.I * ((T.a * ((n : ℕ) - (m : ℕ)) : ℝ) : ℂ) * ⟪⟨m|, T.openHamiltonian |n⟩⟫_ℂ
```

lemma inner_current_eq [source]

The current moves the electron exactly one site, a distance `a`, with amplitude `∓ i a t`; every other matrix element vanishes.

```
lemma inner_current_eq (m n : Fin T.N) :
    ⟪⟨m|, T.current |n⟩⟫_ℂ =
      if (m : ℕ) + 1 = n then -(Complex.I * (T.a * T.t : ℝ))
      else if (n : ℕ) + 1 = m then Complex.I * (T.a * T.t : ℝ) else 0
```

lemma inner_current_self_of_eigen [source]

Stationary states carry no current: `⟨ψ|J|ψ⟩ = 0` whenever `H ψ = E ψ`.

```
lemma inner_current_self_of_eigen {ψ : T.HilbertSpace} {E : ℝ}
    (h : T.openHamiltonian ψ = (E : ℂ) • ψ) : ⟪ψ, T.current ψ⟫_ℂ = 0
```

lemma inner_current_localizedState [source]

A localized electron carries no current: `⟨n|J|n⟩ = 0`.

```
lemma inner_current_localizedState (n : Fin T.N) : ⟪|n⟩, T.current |n⟩⟫_ℂ = 0
```

def hopping [source]

The hopping part `K = H - E₀` of the open Hamiltonian: its kinetic energy.

```
noncomputable def hopping : T.HilbertSpace →ₗ[ℂ] T.HilbertSpace
```

lemma inner_hopping [source]

`K` hops between neighbouring sites only, with amplitude `-t`, and has no diagonal.

```
lemma inner_hopping (m n : Fin T.N) :
    ⟪|m⟩, T.hopping |n⟩⟫_ℂ = if (m : ℕ) + 1 = n ∨ (n : ℕ) + 1 = m then -(T.t : ℂ) else 0
```

lemma inner_apply_eq_sum [source]

An operator in components: `⟨m|A ψ⟩ = ∑ n, ⟨m|A|n⟩ ⟨n|ψ⟩`.

```
lemma inner_apply_eq_sum (A : T.HilbertSpace →ₗ[ℂ] T.HilbertSpace) (m : Fin T.N)
    (ψ : T.HilbertSpace) : ⟪|m⟩, A ψ⟫_ℂ = ∑ n, ⟪|m⟩, A |n⟩⟫_ℂ * ⟪|n⟩, ψ⟫_ℂ
```

lemma ext_localizedState [source]

Two vectors with the same components on every site are equal.

```
lemma ext_localizedState {ψ φ : T.HilbertSpace} (h : ∀ m, ⟪|m⟩, ψ⟫_ℂ = ⟪|m⟩, φ⟫_ℂ) : ψ = φ
```

lemma current_eq_hopping [source]

The current only sees the hopping: `J = i (K X - X K)`.

```
lemma current_eq_hopping :
    T.current = Complex.I • (T.hopping ∘ₗ T.position - T.position ∘ₗ T.hopping)
```

lemma inner_current_hopping [source]

The matrix elements of `J` are those of `K`, weighted by `i` times the hopping distance.

```
lemma inner_current_hopping (m n : Fin T.N) :
    ⟪|m⟩, T.current |n⟩⟫_ℂ =
      Complex.I * ((T.a * ((n : ℕ) - (m : ℕ)) : ℝ) : ℂ) * ⟪|m⟩, T.hopping |n⟩⟫_ℂ
```

lemma current_comp_position_sub [source]

***Position and current close on the kinetic energy.** `J X - X J = i a² K`.*

```
lemma current_comp_position_sub :
    T.current ∘ₗ T.position - T.position ∘ₗ T.current =
      (Complex.I * (T.a : ℂ) ^ 2) • T.hopping
```

lemma inner_hopping_comp_current_sub [source]

On the hopping, `⟨m|(K J - J K)|n⟩ = i a ∑ k, ⟨m|K|k⟩ ⟨k|K|n⟩ (n + m - 2 k)`.

```
lemma inner_hopping_comp_current_sub (m n : Fin T.N) :
    ⟪⟨m|, (T.hopping ∘ₗ T.current - T.current ∘ₗ T.hopping) |n⟩⟫_ℂ =
      ∑ k, Complex.I * T.a * ⟪⟨m|, T.hopping |k⟩⟫_ℂ * ⟪⟨k|, T.hopping |n⟩⟫_ℂ *
        (((n : ℕ) + (m : ℕ) - 2 * (k : ℕ) : ℝ) : ℂ)
```

lemma sum_ite_succ_eq [source]

Counting the left neighbour: `∑ k, [k + 1 = m] = [1 ≤ m]`.

```
private lemma sum_ite_succ_eq (m : Fin T.N) :
    ∑ k : Fin T.N, (if (k : ℕ) + 1 = m then (1 : ℂ) else 0) = if 1 ≤ (m : ℕ) then 1 else 0
```

lemma sum_ite_eq_succ [source]

Counting the right neighbour: `∑ k, [m + 1 = k] = [m + 1 < N]`.

```
private lemma sum_ite_eq_succ (m : Fin T.N) :
    ∑ k : Fin T.N, (if (m : ℕ) + 1 = k then (1 : ℂ) else 0) =
      if (m : ℕ) + 1 < T.N then 1 else 0
```

lemma hopping_term_self [source]

On the diagonal, a term of `inner_hopping_comp_current_sub` counts the neighbours of `m`.

```
private lemma hopping_term_self (m k : Fin T.N) :
    Complex.I * T.a * ⟪⟨m|, T.hopping |k⟩⟫_ℂ * ⟪⟨k|, T.hopping |m⟩⟫_ℂ *
        (((m : ℕ) + (m : ℕ) - 2 * (k : ℕ) : ℝ) : ℂ) =
      2 * Complex.I * T.a * T.t ^ 2 *
        ((if (k : ℕ) + 1 = m then 1 else 0) - (if (m : ℕ) + 1 = k then 1 else 0))
```

lemma hopping_term_ne [source]

Off the diagonal every term of `inner_hopping_comp_current_sub` vanishes: two hops from `m` to `n ≠ m` pass through the midpoint.

```
private lemma hopping_term_ne {m n : Fin T.N} (h : m ≠ n) (k : Fin T.N) :
    Complex.I * T.a * ⟪⟨m|, T.hopping |k⟩⟫_ℂ * ⟪⟨k|, T.hopping |n⟩⟫_ℂ *
      (((n : ℕ) + (m : ℕ) - 2 * (k : ℕ) : ℝ) : ℂ) = 0
```

lemma openHamiltonian_comp_current_sub [source]

The current commutes with `H` as with its hopping part: `H J - J H = K J - J K`.

```
lemma openHamiltonian_comp_current_sub :
    T.openHamiltonian ∘ₗ T.current - T.current ∘ₗ T.openHamiltonian =
      T.hopping ∘ₗ T.current - T.current ∘ₗ T.hopping
```

lemma inner_openHamiltonian_comp_current_sub [source]

***The current changes only at the two ends of the chain.** `H J - J H` is diagonal, with entry `-2 i a t²` on the first site, `2 i a t²` on the last, and `0` inside the chain.*

```
lemma inner_openHamiltonian_comp_current_sub (m n : Fin T.N) :
    ⟪⟨m|, (T.openHamiltonian ∘ₗ T.current - T.current ∘ₗ T.openHamiltonian) |n⟩⟫_ℂ =
      if m = n then 2 * Complex.I * T.a * T.t ^ 2 *
        ((if 1 ≤ (m : ℕ) then 1 else 0) - (if (m : ℕ) + 1 < T.N then 1 else 0)) else 0
```

lemma eq_zero_of_hopping_eq_smul [source]

***The recurrence of the hopping.** A solution of `K ψ = μ ψ` vanishing on the first site vanishes: the row `n` fixes `ψ (n + 1)` from `ψ n` and `ψ (n - 1)`.*

```
lemma eq_zero_of_hopping_eq_smul (ht : T.t ≠ 0) (hN : 0 < T.N) {μ : ℂ} {ψ : T.HilbertSpace}
    (h : T.hopping ψ = μ • ψ) (h0 : ⟪⟨|(⟨0, hN⟩ : Fin T.N)⟩, ψ⟫_ℂ = 0) : ψ = 0
```

lemma eq_zero_of_hopping_eq_smul_of_current_eq_zero [source]

***The hopping and the current share no eigenvector.** For `t ≠ 0` and `N ≥ 2`, `K ψ = μ ψ` and `J ψ = 0` force `ψ = 0`: `X ψ` solves the same recurrence and vanishes on the first site, so `ψ` lives on the first site, from which `K` moves it.*

```
lemma eq_zero_of_hopping_eq_smul_of_current_eq_zero (ht : T.t ≠ 0) (hN : 2 ≤ T.N) {μ : ℂ}
    {ψ : T.HilbertSpace} (hK : T.hopping ψ = μ • ψ) (hJ : T.current ψ = 0) : ψ = 0
```

2.305 PhyslibAlpha.CondensedMatter.TightBindingChain.Current.Eigenstates

PhyslibAlpha/CondensedMatter/TightBindingChain/Current/Eigenstates.lean on GitHub.

def currentEigenstate [source]

The current eigenstate `∑ n, i^n sin ((n + 1) k π / (N + 1)) |n⟩`.

```
noncomputable def currentEigenstate (k : ℕ) : T.HilbertSpace
```

def currentEigenvalue [source]

The current `2 a t cos (k π / (N + 1))` of the eigenstate with quantum number `k`.

```
noncomputable def currentEigenvalue (k : ℕ) : ℝ
```

lemma inner_currentEigenstate [source]

The amplitude of the current eigenstate on the site `m`.

```
lemma inner_currentEigenstate (k : ℕ) (m : Fin T.N) :
    ⟪⟨|m⟩, T.currentEigenstate k⟫_ℂ =
      Complex.I ^ (m : ℕ) * Real.sin (((m : ℕ) + 1) * (k * Real.pi / (T.N + 1)))
```

lemma sum_ite_eq_of_unique [source]

A sum over the sites picking at most one site.

```
lemma sum_ite_eq_of_unique {N : ℕ} (P : Fin N → Prop) [DecidablePred P] (v : ℂ)
    (h : ∀ n n', P n → P n' → n = n') (hv : (∀ n, ¬ P n) → v = 0) :
    ∑ n, (if P n then v else 0) = v
```

lemma current_currentEigenstate [source]

The current eigenstates satisfy `J ψ_k = 2 a t cos (k π / (N + 1)) ψ_k`.

```
lemma current_currentEigenstate (k : ℕ) :
    T.current (T.currentEigenstate k) =
      (T.currentEigenvalue k : ℂ) • T.currentEigenstate k
```

lemma sum_cos_eq_zero [source]

The cosines `cos (2 j k π / (N + 1))` sum to zero over `j < N + 1` for `1 ≤ k ≤ N`.

```
private lemma sum_cos_eq_zero {k : ℕ} (hk : 1 ≤ k) (hkN : k ≤ T.N) :
    ∑ j ∈ Finset.range (T.N + 1), Real.cos (2 * (j * (k * Real.pi / (T.N + 1)))) = 0
```

lemma sum_sin_sq [source]

The squared sines `sin² (j k π / (N + 1))` sum to `(N + 1) / 2` over `j < N + 1`.

```
private lemma sum_sin_sq {k : ℕ} (hk : 1 ≤ k) (hkN : k ≤ T.N) :
    ∑ j ∈ Finset.range (T.N + 1), Real.sin (j * (k * Real.pi / (T.N + 1))) ^ 2 =
      (T.N + 1) / 2
```

lemma norm_sq_currentEigenstate [source]

The current eigenstates have squared norm `(N + 1) / 2` for `1 ≤ k ≤ N`.

```
lemma norm_sq_currentEigenstate {k : ℕ} (hk : 1 ≤ k) (hkN : k ≤ T.N) :
    ‖T.currentEigenstate k‖ ^ 2 = (T.N + 1) / 2
```

2.306 PhyslibAlpha.CondensedMatter.TightBindingChain.Current.MandelstamTamm

PhyslibAlpha/CondensedMatter/TightBindingChain/Current/MandelstamTamm.lean on GitHub.

abbrev currentObservable [source]

The current `J = i (H X - X H)` as an observable.

```
noncomputable abbrev currentObservable
```

lemma coe_bracket_openHamiltonian_position [source]

The bracket of the Hamiltonian and the position is minus half the current: `⁅H, X⁆ = -J / 2`.

```
lemma coe_bracket_openHamiltonian_position :
    (⁅T.openHamiltonianObservable, T.positionObservable⁆ : Observable _) :
      T.HilbertSpace →L[ℂ] T.HilbertSpace) =
```

```
    (-(1 / 2) : ℂ) • (T.currentObservable : T.HilbertSpace →L[ℂ] T.HilbertSpace)
```

lemma expectation_bracket_openHamiltonian_position [source]

The expected bracket is minus half the expected current.

```
lemma expectation_bracket_openHamiltonian_position
    (ω : 𝒮[ℂ, T.HilbertSpace →L[ℂ] T.HilbertSpace]) :
    ω⟦T.openHamiltonianObservable, T.positionObservable⟧ = -(ω⟨T.currentObservable⟩ / 2)
```

lemma sq_expectation_current_le [source]

In every state, `⟨J⟩² ≤ 4 Var H · Var X`.

```
lemma sq_expectation_current_le :
    ω⟨T.currentObservable⟩ ^ 2 ≤
      4 * (variance ω T.openHamiltonianObservable * variance ω T.positionObservable)
```

lemma mandelstam_tamm [source]

***Mandelstam–Tamm.** The position moves no faster than the energy spread allows: `|d⟨X⟩/dt| = |⟨J⟩| ≤ 2 ΔH ΔX`.*

```
lemma mandelstam_tamm :
    |ω⟨T.currentObservable⟩| ≤
      2 * (√(variance ω T.openHamiltonianObservable) * √(variance ω T.positionObservable))
```

lemma cramer_rao [source]

***Quantum Cramér–Rao.** For a displacement generated by `H`, the error-propagation Fisher information `⟨J⟩² / Var X` of the position is at most `4 Var H`, the quantum Fisher information of a pure state.*

```
lemma cramer_rao :
    ω⟨T.currentObservable⟩ ^ 2 / variance ω T.positionObservable ≤
      4 * variance ω T.openHamiltonianObservable
```

def mandelstamTammRatio [source]

The Mandelstam–Tamm ratio `⟨J⟩² / (4 Var H · Var X)`: how close a state comes to both Mandelstam–Tamm and Cramér–Rao.

```
noncomputable def mandelstamTammRatio : ℝ
```

lemma mandelstamTammRatio_le_one [source]

The Mandelstam–Tamm ratio never exceeds one.

```
lemma mandelstamTammRatio_le_one : T.mandelstamTammRatio ω ≤ 1
```

2.307 PhyslibAlpha.CondensedMatter.TightBindingChain.Current.SpeedLimit

PhyslibAlpha/CondensedMatter/TightBindingChain/Current/SpeedLimit.lean on GitHub.

def maxCurrent [source]

The largest current `2 |a t| cos (π / (N + 1))` of the open chain.

```
noncomputable def maxCurrent : ℝ
```

lemma abs_currentEigenvalue_le [source]

Every current eigenvalue is at most `maxCurrent` in absolute value.

```
lemma abs_currentEigenvalue_le {k : ℕ} (hk : 1 ≤ k) (hkN : k ≤ T.N) :
    |T.currentEigenvalue k| ≤ T.maxCurrent
```

def currentBasisVec [source]

The normalized current eigenstates `ψ_{k+1}`, `k < N`.

```
noncomputable def currentBasisVec (k : Fin T.N) : T.HilbertSpace
```

lemma norm_currentBasisVec [source]

```
lemma norm_currentBasisVec (k : Fin T.N) : ‖T.currentBasisVec k‖ = 1
```

lemma orthonormal_currentBasisVec [source]

The normalized current eigenstates are orthonormal.

```
lemma orthonormal_currentBasisVec (ht : T.t ≠ 0) : Orthonormal ℂ T.currentBasisVec
```

def currentBasis [source]

The orthonormal basis of normalized current eigenstates.

```
noncomputable def currentBasis (ht : T.t ≠ 0) : OrthonormalBasis (Fin T.N) ℂ T.HilbertSpace
```

lemma current_currentBasis [source]

The current on the basis of current eigenstates.

```
lemma current_currentBasis (ht : T.t ≠ 0) (k : Fin T.N) :
    T.current (T.currentBasis ht k) =
      (T.currentEigenvalue ((k : ℕ) + 1) : ℂ) • T.currentBasis ht k
```

lemma re_inner_current_eq_sum [source]

The mean current in the basis of current eigenstates: `re ⟨ψ|J|ψ⟩ = ∑ k, λ_k |⟨k|ψ⟩|²`.

```
lemma re_inner_current_eq_sum (ht : T.t ≠ 0) (ψ : T.HilbertSpace) :
    (⟪ψ, T.current ψ⟫_ℂ).re =
      ∑ k : Fin T.N, T.currentEigenvalue ((k : ℕ) + 1) * ‖⟪T.currentBasis ht k, ψ⟫_ℂ‖ ^ 2
```

lemma abs_re_inner_current_le [source]

In every vector, $|re\ ⟪ψ|J|ψ⟫| ≤ maxCurrent\ ‖ψ‖^2$.

```
lemma abs_re_inner_current_le (ht : T.t ≠ 0) (ψ : T.HilbertSpace) :
    |(⟪ψ, T.current ψ⟫_ℂ).re| ≤ T.maxCurrent * ‖ψ‖ ^ 2
```

lemma abs_expectation_current_le [source]

***Speed limit.** In every state, $|⟨J⟩| ≤ maxCurrent$: no state of the open chain moves faster than $2\ |a\ t|\ cos\ (π\ /\ (N + 1))$.*

```
lemma abs_expectation_current_le (ht : T.t ≠ 0)
    (ω : 𝓢[ℂ, T.HilbertSpace →L[ℂ] T.HilbertSpace]) :
    |ω⟨T.currentObservable⟩| ≤ T.maxCurrent
```

lemma expectation_current_maxCurrentState [source]

The maximal current state carries the current $⟨J⟩ = 2\ a\ t\ cos\ (π\ /\ (N + 1))$.

```
lemma expectation_current_maxCurrentState :
    T.maxCurrentVectorState⟨T.currentObservable⟩ =
      2 * (T.a * T.t * Real.cos (Real.pi / (T.N + 1)))
```

lemma abs_expectation_current_maxCurrentState [source]

The maximal current state moves at the speed limit.

```
lemma abs_expectation_current_maxCurrentState :
    |T.maxCurrentVectorState⟨T.currentObservable⟩| = T.maxCurrent
```

2.308 PhyslibAlpha.CondensedMatter.TightBindingChain.Current.TimeReversal

PhyslibAlpha/CondensedMatter/TightBindingChain/Current/TimeReversal.lean on GitHub.

def timeReversal [source]

Time reversal: complex conjugation of the amplitudes.

```
noncomputable def timeReversal (ψ : T.HilbertSpace) : T.HilbertSpace
```

lemma timeReversal_val [source]

```
lemma timeReversal_val (ψ : T.HilbertSpace) (n : Fin T.N) :
    (T.timeReversal ψ).val n = (starRingEnd ℂ) (ψ.val n)
```

lemma timeReversal_sub [source]

```
lemma timeReversal_sub (ψ φ : T.HilbertSpace) :
    T.timeReversal (ψ - φ) = T.timeReversal ψ - T.timeReversal φ
```

lemma timeReversal_smul [source]

```
lemma timeReversal_smul (c : ℂ) (ψ : T.HilbertSpace) :
    T.timeReversal (c • ψ) = (starRingEnd ℂ) c • T.timeReversal ψ
```

lemma timeReversal_real_smul [source]

```
lemma timeReversal_real_smul (r : ℝ) (ψ : T.HilbertSpace) :
    T.timeReversal (r • ψ) = r • T.timeReversal ψ
```

lemma inner_timeReversal [source]

```
lemma inner_timeReversal (ψ φ : T.HilbertSpace) :
    ⟪T.timeReversal ψ, T.timeReversal φ⟫_ℂ = (starRingEnd ℂ) ⟪ψ, φ⟫_ℂ
```

lemma norm_timeReversal [source]

```
lemma norm_timeReversal (ψ : T.HilbertSpace) : ‖T.timeReversal ψ‖ = ‖ψ‖
```

lemma apply_timeReversal [source]

An operator with real matrix elements commutes with time reversal.

```
lemma apply_timeReversal {A : T.HilbertSpace →ₗ[ℂ] T.HilbertSpace}
    (hA : ∀ n m, (starRingEnd ℂ) ⟪|n⟩, A |m⟩⟫_ℂ = ⟪|n⟩, A |m⟩⟫_ℂ) (ψ : T.HilbertSpace) :
    A (T.timeReversal ψ) = T.timeReversal (A ψ)
```

lemma openHamiltonian_timeReversal [source]

```
lemma openHamiltonian_timeReversal (ψ : T.HilbertSpace) :
    T.openHamiltonian (T.timeReversal ψ) = T.timeReversal (T.openHamiltonian ψ)
```

lemma position_timeReversal [source]

```
lemma position_timeReversal (ψ : T.HilbertSpace) :
    T.position (T.timeReversal ψ) = T.timeReversal (T.position ψ)
```

lemma current_timeReversal [source]

Time reversal reverses the current.

```
lemma current_timeReversal (ψ : T.HilbertSpace) :
    T.current (T.timeReversal ψ) = -T.timeReversal (T.current ψ)
```

2.309 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.Basic

PhyslibAlpha/CondensedMatter/TightBindingChain/MaxCurrentState/Basic.lean on GitHub.

lemma currentEigenvalue_le_one [source]

For `t ≥ 0`, the current eigenstate with `k = 1` carries the largest current.

```
lemma currentEigenvalue_le_one (ht : 0 ≤ T.t) {k : ℕ} (hk : 1 ≤ k) (hkN : k ≤ T.N) :
    T.currentEigenvalue k ≤ T.currentEigenvalue 1
```

lemma inner_openHamiltonian_currentEigenstate [source]

The Hamiltonian on the current eigenstates: ⟨m|H ψ_k⟩ = E0 ⟨m|ψ_k⟩ - 2 t sin θ i^(m+1) cos ((m + 1) θ) with θ = k π / (N + 1).

```
lemma inner_openHamiltonian_currentEigenstate (k : ℕ) (m : Fin T.N) :
    ⟪|m⟩, T.openHamiltonian (T.currentEigenstate k)⟫_ℂ =
      T.E0 * ⟪|m⟩, T.currentEigenstate k⟫_ℂ - 2 * T.t * Real.sin (k * Real.pi / (T.N + 1)) *
        Complex.I ^ ((m : ℕ) + 1) * Real.cos (((m : ℕ) + 1) * (k * Real.pi / (T.N + 1)))
```

def maxCurrentState [source]

The maximal current state: the current eigenstate with k = 1, times √(2 / (N + 1)).

```
noncomputable def maxCurrentState : T.HilbertSpace
```

lemma norm_maxCurrentState [source]

The maximal current state has norm one.

```
lemma norm_maxCurrentState : ‖T.maxCurrentState‖ = 1
```

lemma inner_maxCurrentState [source]

The amplitude of the maximal current state on the site m.

```
lemma inner_maxCurrentState (m : Fin T.N) :
    ⟪|m⟩, T.maxCurrentState⟫_ℂ = (√(2 / (T.N + 1)) : ℝ) *
      (Complex.I ^ (m : ℕ) * Real.sin (((m : ℕ) + 1) * (Real.pi / (T.N + 1))))
```

lemma inner_openHamiltonian_maxCurrentState [source]

The Hamiltonian on the maximal current state, site by site.

```
lemma inner_openHamiltonian_maxCurrentState (m : Fin T.N) :
    ⟪|m⟩, T.openHamiltonian T.maxCurrentState⟫_ℂ = T.E0 * ⟪|m⟩, T.maxCurrentState⟫_ℂ -
      (√(2 / (T.N + 1)) : ℝ) * (2 * T.t * Real.sin (Real.pi / (T.N + 1)) *
        Complex.I ^ ((m : ℕ) + 1) * Real.cos (((m : ℕ) + 1) * (Real.pi / (T.N + 1))))
```

lemma current_maxCurrentState [source]

The maximal current state carries the current 2 a t cos (π / (N + 1)).

```
lemma current_maxCurrentState :
    T.current T.maxCurrentState = (T.currentEigenvalue 1 : ℂ) • T.maxCurrentState
```

abbrev maxCurrentVectorState [source]

The maximal current state as a state on the operators of the chain.

```
noncomputable abbrev maxCurrentVectorState :
    𝒮[ℂ, T.HilbertSpace →L[ℂ] T.HilbertSpace]
```

lemma expectation_maxCurrentState [source]

Expectations in the maximal current state are real parts of its diagonal matrix elements.

```
lemma expectation_maxCurrentState (A : Observable (T.HilbertSpace →L[ℂ] T.HilbertSpace)) :
    T.maxCurrentVectorState⟨A⟩ =
      (⟪T.maxCurrentState, (A : T.HilbertSpace →L[ℂ] T.HilbertSpace) T.maxCurrentState⟫_ℂ).re
```

lemma conj_I_pow_mul [source]

conj (iᵐ) iᵐ = 1.

```
private lemma conj_I_pow_mul (m : ℕ) : (starRingEnd ℂ) (Complex.I ^ m) * Complex.I ^ m = 1
```

lemma sum_norm_sq_inner_maxCurrentState [source]

The squared amplitudes of the maximal current state add up to one.

```
private lemma sum_norm_sq_inner_maxCurrentState :
    ∑ m, ‖⟪|m⟩, T.maxCurrentState⟫_ℂ‖ ^ 2 = 1
```

lemma expectation_openHamiltonian_maxCurrentState [source]

The maximal current state sits at the centre of the band, `⟨H⟩ = E0`.

```
lemma expectation_openHamiltonian_maxCurrentState :
    T.maxCurrentVectorState⟨T.openHamiltonianObservable⟩ = T.E0
```

lemma expectation_position_maxCurrentState [source]

The maximal current state sits at the centre of the chain, `⟨X⟩ = a (N - 1) / 2`.

```
lemma expectation_position_maxCurrentState :
    T.maxCurrentVectorState⟨T.positionObservable⟩ = T.a * (T.N - 1) / 2
```

lemma expectation_bracket_maxCurrentState [source]

The bracket of the maximal current state: `⟨⁅H, X⁆⟩ = -a t cos (π / (N + 1))`.

```
lemma expectation_bracket_maxCurrentState :
    T.maxCurrentVectorState⟨⁅T.openHamiltonianObservable, T.positionObservable⁆⟩ =
      -(T.a * T.t * Real.cos (Real.pi / (T.N + 1)))
```

lemma robertson_schrodinger_maxCurrentState [source]

***Energy–position uncertainty of the maximal current state.** `Cov(H, X)² + (a t cos (π / (N + 1)))² ≤ Var H · Var X` in the maximal current state.*

```
lemma robertson_schrodinger_maxCurrentState :
    covariance T.maxCurrentVectorState T.openHamiltonianObservable T.positionObservable ^ 2 +
        (T.a * T.t * Real.cos (Real.pi / (T.N + 1))) ^ 2 ≤
      variance T.maxCurrentVectorState T.openHamiltonianObservable *
        variance T.maxCurrentVectorState T.positionObservable
```

2.310 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.FluctuationAngle

PhyslibAlpha/CondensedMatter/TightBindingChain/MaxCurrentState/FluctuationAngle.lean on GitHub.

def fluctuationAngle [source]

The fluctuation angle: the angle between the energy and position fluctuations in the maximal current state, `arccos (‖ω(δH δX)‖ / (σ_H σ_X))`.

```
noncomputable def fluctuationAngle : ℝ
```

lemma fluctuationAngle_eq [source]

The fluctuation angle: `cos θ = 1 / C`.

```
lemma fluctuationAngle_eq (ht : T.t ≠ 0) (hN : 2 ≤ T.N) :
    T.fluctuationAngle = Real.arccos (1 / T.uncertaintyRatio)
```

lemma fluctuationAngle_eq_zero_iff [source]

The fluctuation angle vanishes exactly for `N = 2` and `N = 3`.

```
lemma fluctuationAngle_eq_zero_iff (ht : T.t ≠ 0) (hN : 2 ≤ T.N) :
    T.fluctuationAngle = 0 ↔ T.N = 2 ∨ T.N = 3
```

lemma fluctuationAngle_pos [source]

From four sites on, the fluctuation angle is open.

```
lemma fluctuationAngle_pos (ht : T.t ≠ 0) (hN : 4 ≤ T.N) : 0 < T.fluctuationAngle
```

lemma arccos_inv_lt [source]

```
private lemma arccos_inv_lt {C C' : ℝ} (hC : 1 ≤ C) (hCC' : C < C') :
    Real.arccos (1 / C) < Real.arccos (1 / C')
```

lemma fluctuationAngle_lt_fluctuationAngle [source]

From four sites on, the fluctuation angle grows strictly with the number of sites.

```
lemma fluctuationAngle_lt_fluctuationAngle {T T' : TightBindingChain} (ht : T.t ≠ 0)
    (ht' : T'.t ≠ 0) (hN : 4 ≤ T.N) (hNN' : T.N < T'.N) :
    T.fluctuationAngle < T'.fluctuationAngle
```

lemma fluctuationAngle_lt [source]

From four sites on, the fluctuation angle stays below `arccos (1 / √(π² / 3 - 2))`.

```
lemma fluctuationAngle_lt (ht : T.t ≠ 0) (hN : 4 ≤ T.N) :
    T.fluctuationAngle < Real.arccos (1 / √(Real.pi ^ 2 / 3 - 2))
```

def dimQuantum [source]

The dimensional quantum of the chain: the excess `C - 1` over saturation.

```
noncomputable def dimQuantum : ℝ
```

lemma dimQuantum_nonneg [source]

```
lemma dimQuantum_nonneg (ht : T.t ≠ 0) (hN : 2 ≤ T.N) : 0 ≤ T.dimQuantum
```

lemma dimQuantum_eq_zero_iff [source]

The dimensional quantum vanishes exactly for `N = 2` and `N = 3`.

```
lemma dimQuantum_eq_zero_iff (ht : T.t ≠ 0) (hN : 2 ≤ T.N) :
    T.dimQuantum = 0 ↔ T.N = 2 ∨ T.N = 3
```

lemma dimQuantum_pos [source]

From four sites on, the dimensional quantum is positive.

```
lemma dimQuantum_pos (ht : T.t ≠ 0) (hN : 4 ≤ T.N) : 0 < T.dimQuantum
```

lemma dimQuantum_lt_dimQuantum [source]

From four sites on, the dimensional quantum grows strictly with the number of sites.

```
lemma dimQuantum_lt_dimQuantum {T T' : TightBindingChain} (ht : T.t ≠ 0) (ht' : T'.t ≠ 0)
    (hN : 4 ≤ T.N) (hNN' : T.N < T'.N) : T.dimQuantum < T'.dimQuantum
```

lemma dimQuantum_lt [source]

From four sites on, the dimensional quantum stays below `√(π² / 3 - 2) - 1`.

```
lemma dimQuantum_lt (ht : T.t ≠ 0) (hN : 4 ≤ T.N) :
    T.dimQuantum < √(Real.pi ^ 2 / 3 - 2) - 1
```

lemma fluctuationAngle_eq_dimQuantum [source]

The dimensional quantum fixes the fluctuation angle: `θ = arccos (1 / (1 + δ))`.

```
lemma fluctuationAngle_eq_dimQuantum (ht : T.t ≠ 0) (hN : 2 ≤ T.N) :
    T.fluctuationAngle = Real.arccos (1 / (1 + T.dimQuantum))
```

2.311 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.LongChainLimit

PhyslibAlpha/CondensedMatter/TightBindingChain/MaxCurrentState/LongChainLimit.lean on GitHub.

def uncertaintyRatioSqForm [source]

The closed form of `C²` as a function of `n = N + 1`, written with `sinc`.

```
noncomputable def uncertaintyRatioSqForm (n : ℝ) : ℝ
```

lemma tendsto_uncertaintyRatioSqForm [source]

The closed form tends to `π² / 3 - 2` as `n → ∞`.

```
lemma tendsto_uncertaintyRatioSqForm :
    Tendsto uncertaintyRatioSqForm atTop (𝓝 (Real.pi ^ 2 / 3 - 2))
```

lemma uncertaintyRatio_sq_eq_uncertaintyRatioSqForm [source]

For `N ≥ 2`, `C²` is the closed form at `n = N + 1`.

```
lemma uncertaintyRatio_sq_eq_uncertaintyRatioSqForm (T : TightBindingChain) (ht : T.t ≠ 0)
    (hN : 2 ≤ T.N) :
    T.uncertaintyRatio ^ 2 = uncertaintyRatioSqForm (T.N + 1)
```

lemma tendsto_uncertaintyRatio [source]

***The long chain limit.** Along any family of chains whose number of sites tends to infinity, the constant `C` of the maximal current state tends to `√(π² / 3 - 2)`.*

```
lemma tendsto_uncertaintyRatio {ι : Type*} {l : Filter ι} (T : ι → TightBindingChain)
    (ht : ∀ i, (T i).t ≠ 0) (hN : Tendsto (fun i => (T i).N) l atTop) :
    Tendsto (fun i => (T i).uncertaintyRatio) l (𝓝 √(Real.pi ^ 2 / 3 - 2))
```

lemma one_lt_sqrt_pi_sq_div_three_sub_two [source]

The limit `√(π² / 3 - 2)` is strictly larger than one, since `π > 3`.

```
lemma one_lt_sqrt_pi_sq_div_three_sub_two : 1 < √(Real.pi ^ 2 / 3 - 2)
```

2.312 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.LongChainMonotonicity

PhyslibAlpha/CondensedMatter/TightBindingChain/MaxCurrentState/LongChainMonotonicity.lean on GitHub.

lemma cos_le_taylor [source]

```
private lemma cos_le_taylor {x : ℝ} (hx : 0 ≤ x) : Real.cos x ≤ 1 - x ^ 2 / 2 + x ^ 4 / 24
```

lemma sin_le_taylor [source]

```
private lemma sin_le_taylor {x : ℝ} (hx : 0 ≤ x) : Real.sin x ≤ x - x ^ 3 / 6 + x ^ 5 / 120
```

def remainder [source]

The remainder of the derivative: a polynomial in `x` and `p = π`, of degree three in `p`.

```
private noncomputable def remainder (x p : ℝ) : ℝ
```

lemma bound_poly_neg [source]

The remainder with π replaced by rational bounds, `3.141592` in the terms of degree one and two in `p` and `3.141593` in the term of degree three, is negative on `0 ≤ x ≤ 1571 / 2500`: all thirteen Bernstein coefficients are negative.

```
private lemma bound_poly_neg {x : ℝ} (hx0 : 0 ≤ x) (hx1 : x ≤ 1571 / 2500) :
    (5 * x ^ 12 - 180 * x ^ 10 + 1880 * x ^ 8 - 7552 * x ^ 6 + 6720 * x ^ 4 + 34560 * x ^ 2 +
        69120) +
      (-384 * x ^ 5 + 7680 * x ^ 3 - 46080 * x) * (3141592 / 1000000) +
      (-160 * x ^ 6 + 2048 * x ^ 4 - 5760 * x ^ 2 - 11520) * (3141592 / 1000000) ^ 2 +
      (160 * x ^ 5 - 2144 * x ^ 3 + 7680 * x) * (3141593 / 1000000) ^ 3 < 0
```

lemma remainder_neg [source]

The remainder `P(x, π)` of the derivative is negative for `0 ≤ x ≤ 1571 / 2500`.

```
private lemma remainder_neg {x p : ℝ} (hx0 : 0 ≤ x) (hx1 : x ≤ 1571 / 2500)
    (hp0 : 3141592 / 1000000 ≤ p) (hp1 : p ≤ 3141593 / 1000000) :
    remainder x p < 0
```

def angleForm [source]

The closed form of `C²` in the angle `x = π / n`.

```
private noncomputable def angleForm (x : ℝ) : ℝ
```

lemma uncertaintyRatioSqForm_eq_angleForm [source]

```
private lemma uncertaintyRatioSqForm_eq_angleForm {n : ℝ} (hn : 2 < n) :
    uncertaintyRatioSqForm n = angleForm (Real.pi / n)
```

lemma hasDerivAt_angleForm [source]

```
private lemma hasDerivAt_angleForm {x : ℝ} (hx : x ≠ 0) (hcos : Real.cos x ≠ 0) :
    HasDerivAt angleForm
      (((-2 * Real.pi ^ 2 / x ^ 3 + 2 * Real.pi / x ^ 2 + 8 / Real.pi) * Real.sin x ^ 2 *
          Real.cos x + 2 * (Real.pi ^ 2 / x ^ 2 - 2 * Real.pi / x - 4 + 8 * x / Real.pi) *
          Real.sin x + 12 / Real.pi * Real.cos x ^ 3) / (3 * Real.cos x ^ 3)) x
```

lemma deriv_angleForm_neg [source]

```
private lemma deriv_angleForm_neg {x : ℝ} (hx0 : 0 < x) (hx5 : x ≤ Real.pi / 5) :
    deriv angleForm x < 0
```

lemma angleForm_strictAntiOn [source]

```
private lemma angleForm_strictAntiOn : StrictAntiOn angleForm (Ioc 0 (Real.pi / 5))
```

lemma uncertaintyRatio_nonneg [source]

```
private lemma uncertaintyRatio_nonneg : 0 ≤ T.uncertaintyRatio
```

lemma uncertaintyRatioSqForm_strictMonoOn [source]

The closed form of `C²` is strictly increasing on `n ≥ 5`, that is, from four sites on.

```
lemma uncertaintyRatioSqForm_strictMonoOn : StrictMonoOn uncertaintyRatioSqForm (Ici 5)
```

lemma uncertaintyRatio_lt_uncertaintyRatio [source]

***Monotonicity.** From four sites on, `C` strictly increases with the number of sites.*

```
lemma uncertaintyRatio_lt_uncertaintyRatio {T T' : TightBindingChain} (ht : T.t ≠ 0)
    (ht' : T'.t ≠ 0) (hN : 4 ≤ T.N) (hNN' : T.N < T'.N) :
    T.uncertaintyRatio < T'.uncertaintyRatio
```

lemma uncertaintyRatio_lt_sqrt_pi_sq_div_three_sub_two [source]

From four sites on, `C` stays strictly below its long chain limit `√(π² / 3 - 2)`.

```
lemma uncertaintyRatio_lt_sqrt_pi_sq_div_three_sub_two (ht : T.t ≠ 0) (hN : 4 ≤ T.N) :
    T.uncertaintyRatio < √(Real.pi ^ 2 / 3 - 2)
```

2.313 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.MandelstamTamm

PhyslibAlpha/CondensedMatter/TightBindingChain/MaxCurrentState/MandelstamTamm.lean on GitHub.

lemma mandelstamTammRatio_maxCurrentState [source]

In the maximal current state the Mandelstam–Tamm ratio is `1 / C²`.

```
lemma mandelstamTammRatio_maxCurrentState :
    T.mandelstamTammRatio T.maxCurrentVectorState = 1 / T.uncertaintyRatio ^ 2
```

lemma mandelstamTammRatio_maxCurrentState_eq_one_iff [source]

Mandelstam–Tamm and Cramér–Rao are attained in the maximal current state iff `N = 2, 3`.

```
lemma mandelstamTammRatio_maxCurrentState_eq_one_iff (ht : T.t ≠ 0) (hN : 2 ≤ T.N) :
    T.mandelstamTammRatio T.maxCurrentVectorState = 1 ↔ T.N = 2 ∨ T.N = 3
```

lemma mandelstamTammRatio_maxCurrentState_lt_one [source]

From four sites on, the maximal current state misses Mandelstam–Tamm and Cramér–Rao.

```
lemma mandelstamTammRatio_maxCurrentState_lt_one (ht : T.t ≠ 0) (hN : 4 ≤ T.N) :
    T.mandelstamTammRatio T.maxCurrentVectorState < 1
```

2.314 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.Saturation

PhyslibAlpha/CondensedMatter/TightBindingChain/MaxCurrentState/Saturation.lean on GitHub.

lemma inner_energyFluctuation_maxCurrentState [source]

The energy fluctuation of the maximal current state, site by site.

```
lemma inner_energyFluctuation_maxCurrentState (m : Fin T.N) :
    ⟪⟨|m⟩, (T.openHamiltonianObservable : T.HilbertSpace →L[ℂ] T.HilbertSpace) T.maxCurrentState
     -
      T.maxCurrentVectorState(T.openHamiltonianObservable) • T.maxCurrentState⟫_ℂ =
      -((√(2 / (T.N + 1)) : ℝ) * (2 * T.t * Real.sin (Real.pi / (T.N + 1)) *
        Complex.I ^ ((m : ℕ) + 1) * Real.cos (((m : ℕ) + 1) * (Real.pi / (T.N + 1)))))
```

lemma inner_positionFluctuation_maxCurrentState [source]

The position fluctuation of the maximal current state, site by site.

```
lemma inner_positionFluctuation_maxCurrentState (m : Fin T.N) :
    ⟪⟨|m⟩, (T.positionObservable : T.HilbertSpace →L[ℂ] T.HilbertSpace) T.maxCurrentState -
      T.maxCurrentVectorState(T.positionObservable) • T.maxCurrentState⟫_ℂ =
      (T.a * ((m : ℕ) - (T.N - 1) / 2) : ℝ) * ⟪⟨|m⟩, T.maxCurrentState⟫_ℂ
```

lemma parallel_of_two_or_three [source]

In the maximal current state, `(2 m - N + 1) cos θ sin ((m + 1) θ) + (N - 1) sin θ cos ((m + 1) θ) = 0 on every site when N = 2, 3, where θ = π / (N + 1)`.

```
private lemma parallel_of_two_or_three (h : T.N = 2 ∨ T.N = 3) (m : Fin T.N) :
    (2 * ((m : ℕ) : ℝ) - T.N + 1) * Real.cos (Real.pi / (T.N + 1)) *
        Real.sin (((m : ℕ) + 1) * (Real.pi / (T.N + 1))) +
      (T.N - 1) * Real.sin (Real.pi / (T.N + 1)) *
        Real.cos (((m : ℕ) + 1) * (Real.pi / (T.N + 1))) = 0
```

lemma centeredGramDefect_maxCurrentState_eq_zero_iff [source]

In the maximal current state of a chain with `N ≥ 2` sites and hopping `t ≠ 0`, the centered Gram defect of `H` and `X` vanishes exactly for `N = 2` and `N = 3`.

```
lemma centeredGramDefect_maxCurrentState_eq_zero_iff (ht : T.t ≠ 0) (hN : 2 ≤ T.N) :
    centeredGramDefect T.maxCurrentVectorState T.openHamiltonianObservable
      T.positionObservable = 0 ↔ T.N = 2 ∨ T.N = 3
```

lemma robertson_schrodinger_maxCurrentState_eq_iff [source]

***Saturation.** In the maximal current state of a chain with `N ≥ 2` sites and hopping `t ≠ 0`, the energy–position uncertainty relation is an equality exactly for `N = 2, 3`.*

```
lemma robertson_schrodinger_maxCurrentState_eq_iff (ht : T.t ≠ 0) (hN : 2 ≤ T.N) :
    covariance T.maxCurrentVectorState T.openHamiltonianObservable T.positionObservable ^ 2 +
        (T.a * T.t * Real.cos (Real.pi / (T.N + 1))) ^ 2 =
      variance T.maxCurrentVectorState T.openHamiltonianObservable *
        variance T.maxCurrentVectorState T.positionObservable ↔ T.N = 2 ∨ T.N = 3
```

lemma robertson_schrodinger_maxCurrentState_lt [source]

From four sites on, the uncertainty relation of the maximal current state is strict.

```
lemma robertson_schrodinger_maxCurrentState_lt (ht : T.t ≠ 0) (hN : 4 ≤ T.N) :
    covariance T.maxCurrentVectorState T.openHamiltonianObservable T.positionObservable ^ 2 +
        (T.a * T.t * Real.cos (Real.pi / (T.N + 1))) ^ 2 <
      variance T.maxCurrentVectorState T.openHamiltonianObservable *
        variance T.maxCurrentVectorState T.positionObservable
```

2.315 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.UncertaintyRatio

PhyslibAlpha/CondensedMatter/TightBindingChain/MaxCurrentState/UncertaintyRatio.lean on GitHub.

lemma covariance_maxCurrentState [source]

The energy and position fluctuations of the maximal current state are uncorrelated.

```
lemma covariance_maxCurrentState :
    covariance T.maxCurrentVectorState T.openHamiltonianObservable T.positionObservable = 0
```

lemma variance_mul_variance_maxCurrentState [source]

The uncertainty identity of the maximal current state: `Var H · Var X = (a t cos (π / (N + 1)))² + defect`.

```
lemma variance_mul_variance_maxCurrentState :
    variance T.maxCurrentVectorState T.openHamiltonianObservable *
        variance T.maxCurrentVectorState T.positionObservable =
      (T.a * T.t * Real.cos (Real.pi / (T.N + 1))) ^ 2 +
        centeredGramDefect T.maxCurrentVectorState T.openHamiltonianObservable
          T.positionObservable
```

def uncertaintyRatio [source]

The uncertainty ratio `C = √(Var H · Var X) / |a t cos (π / (N + 1))|` of the maximal current state: the uncertainty product over its Robertson bound.

```
noncomputable def uncertaintyRatio : ℝ
```

lemma bound_ne_zero [source]

The bound `a t cos (π / (N + 1))` is nonzero for `t ≠ 0` and `N ≥ 2`.

```
lemma bound_ne_zero (ht : T.t ≠ 0) (hN : 2 ≤ T.N) :
    T.a * T.t * Real.cos (Real.pi / (T.N + 1)) ≠ 0
```

lemma one_le_uncertaintyRatio [source]

`C` is at least one.

```
lemma one_le_uncertaintyRatio (ht : T.t ≠ 0) (hN : 2 ≤ T.N) : 1 ≤ T.uncertaintyRatio
```

lemma uncertaintyRatio_eq_one_iff [source]

`C` equals one exactly for `N = 2` and `N = 3`.

```
lemma uncertaintyRatio_eq_one_iff (ht : T.t ≠ 0) (hN : 2 ≤ T.N) :
    T.uncertaintyRatio = 1 ↔ T.N = 2 ∨ T.N = 3
```

lemma one_lt_uncertaintyRatio [source]

From four sites on, `C` is strictly larger than one.

```
lemma one_lt_uncertaintyRatio (ht : T.t ≠ 0) (hN : 4 ≤ T.N) : 1 < T.uncertaintyRatio
```

lemma robertson_schrodinger_uncertaintyRatio [source]

The Robertson–Schrödinger relation through the uncertainty ratio. In the maximal current state, `Var H · Var X = (a t cos (π / (N + 1)))² + defect`, `C` is at least one, and it equals one exactly for `N = 2` and `N = 3`.

```
lemma robertson_schrodinger_uncertaintyRatio (ht : T.t ≠ 0)
    (hN : 2 ≤ T.N) :
    variance T.maxCurrentVectorState T.openHamiltonianObservable *
        variance T.maxCurrentVectorState T.positionObservable =
      (T.a * T.t * Real.cos (Real.pi / (T.N + 1))) ^ 2 +
        centeredGramDefect T.maxCurrentVectorState T.openHamiltonianObservable
          T.positionObservable ∧
      1 ≤ T.uncertaintyRatio ∧ (T.uncertaintyRatio = 1 ↔ T.N = 2 ∨ T.N = 3)
```

2.316 PhyslibAlpha.CondensedMatter.TightBindingChain.MaxCurrentState.Variances

PhyslibAlpha/CondensedMatter/TightBindingChain/MaxCurrentState/Variances.lean on GitHub.

lemma sub_one_mul_sum_mul_pow [source]

Summation by parts against the powers of `z`.

```
private lemma sub_one_mul_sum_mul_pow (g : ℕ → ℂ) (z : ℂ) (m : ℕ) :
    (z - 1) * ∑ j ∈ Finset.range m, g j * z ^ j =
      g m * z ^ m - g 0 - ∑ j ∈ Finset.range m, (g (j + 1) - g j) * z ^ (j + 1)
```

lemma sum_sq_mul_pow [source]

At a root of unity `z ≠ 1` of order `n`, `∑ j < n, (j - n / 2)² z^j = -2 n z / (z - 1)²`.

```
private lemma sum_sq_mul_pow {z : ℂ} {n : ℕ} (hz : z ^ n = 1) (hz1 : z ≠ 1) :
    ∑ j ∈ Finset.range n, ((j : ℂ) - n / 2) ^ 2 * z ^ j = -2 * n * z / (z - 1) ^ 2
```

lemma sum_sq_mul_cos [source]

`∑ j < N + 1, (j - (N + 1) / 2)² cos (2 j θ) = (N + 1) / (2 sin² θ)` for `θ = π / (N + 1)`.

```
private lemma sum_sq_mul_cos :
    ∑ j ∈ Finset.range (T.N + 1), (((j : ℕ) : ℝ) - (T.N + 1) / 2) ^ 2 *
        Real.cos (2 * (j * (Real.pi / (T.N + 1)))) =
      (T.N + 1) / (2 * Real.sin (Real.pi / (T.N + 1)) ^ 2)
```

lemma sum_sub_sq [source]

$\sum j < m, (j - c)^2 = m (m - 1) (2 m - 1) / 6 - c m (m - 1) + c^2 m.$

```
private lemma sum_sub_sq (c : ℝ) (m : ℕ) :
    ∑ j ∈ Finset.range m, ((j : ℝ) - c) ^ 2 =
      m * (m - 1) * (2 * m - 1) / 6 - c * m * (m - 1) + c ^ 2 * m
```

lemma sum_sin_sq_sites [source]

$\sum m, \sin^2 ((m + 1) \theta) = (N + 1) / 2$ *over the sites, for* $\theta = \pi / (N + 1)$.

```
private lemma sum_sin_sq_sites :
    ∑ m : Fin T.N, Real.sin (((m : ℕ) + 1) * (Real.pi / (T.N + 1))) ^ 2 = (T.N + 1) / 2
```

lemma variance_openHamiltonian_maxCurrentState [source]

The energy variance of the maximal current state, $\mathrm{Var}\ H = 4\ t^2\ \sin^2\ \theta\ (N - 1) / (N + 1)$.

```
lemma variance_openHamiltonian_maxCurrentState :
    variance T.maxCurrentVectorState T.openHamiltonianObservable =
      4 * T.t ^ 2 * Real.sin (Real.pi / (T.N + 1)) ^ 2 * (T.N - 1) / (T.N + 1)
```

lemma variance_position_maxCurrentState [source]

The position variance of the maximal current state, $\mathrm{Var}\ X = a^2\ (((N + 1)^2 + 2) / 12 - 1 / (2 \sin^2\ \theta))$.

```
lemma variance_position_maxCurrentState :
    variance T.maxCurrentVectorState T.positionObservable =
      T.a ^ 2 * (((T.N + 1) ^ 2 + 2) / 12 - 1 / (2 * Real.sin (Real.pi / (T.N + 1)) ^ 2))
```

lemma uncertaintyRatio_sq_eq [source]

The closed form of C^2 *in the maximal current state, with* $\theta = \pi / (N + 1)$: $C^2 = 2\ (N - 1) / ((N + 1)\ \cos^2\ \theta) \cdot (((N + 1)^2 + 2) / 6 \cdot \sin^2\ \theta - 1)$.

```
lemma uncertaintyRatio_sq_eq (ht : T.t ≠ 0) (hN : 2 ≤ T.N) :
    T.uncertaintyRatio ^ 2 = 2 * (T.N - 1) / ((T.N + 1) * Real.cos (Real.pi / (T.N + 1)) ^ 2) *
      (((T.N + 1) ^ 2 + 2) / 6 * Real.sin (Real.pi / (T.N + 1)) ^ 2 - 1)
```

2.317 PhyslibAlpha.CondensedMatter.TightBindingChain.Open.LocalizedUncertainty

PhyslibAlpha/CondensedMatter/TightBindingChain/Open/LocalizedUncertainty.lean on GitHub.

lemma flucOf_eq_zero_of_eigen [source]

A unit eigenvector of A *with real eigenvalue has no fluctuation in* A.

```
lemma flucOf_eq_zero_of_eigen {A : T.HilbertSpace →ₗ[ℂ] T.HilbertSpace} {ψ : T.HilbertSpace}
    (hψ : ‖ψ‖ = 1) {c : ℝ} (h : A ψ = (c : ℂ) • ψ) : T.flucOf A ψ = 0
```

lemma fluc_ne_zero_of_defectOf_pos [source]

A positive defect forces fluctuations in both `H` and `X`.

```
lemma fluc_ne_zero_of_defectOf_pos {ψ : T.HilbertSpace} (h : 0 < T.defectOf ψ) :
    T.flucOf T.openHamiltonian ψ ≠ 0 ∧ T.flucOf T.position ψ ≠ 0
```

lemma not_eigen_of_defectOf_pos [source]

A vector with positive defect is an eigenvector of neither `H` nor `X`.

```
lemma not_eigen_of_defectOf_pos {ψ : T.HilbertSpace} (hψ : ‖ψ‖ = 1)
    (h : 0 < T.defectOf ψ) (c : ℝ) :
    T.openHamiltonian ψ ≠ (c : ℂ) • ψ ∧ T.position ψ ≠ (c : ℂ) • ψ
```

lemma norm_localizedState [source]

```
lemma norm_localizedState (n : Fin T.N) : ‖(|n⟩ : T.HilbertSpace)‖ = 1
```

lemma flucOf_position_localizedState [source]

A localized electron has no fluctuation in position.

```
lemma flucOf_position_localizedState (n : Fin T.N) : T.flucOf T.position |n⟩ = 0
```

lemma exists_neighbour [source]

Every site of a chain with two sites has a neighbour.

```
lemma exists_neighbour (hN : 2 ≤ T.N) (n : Fin T.N) :
    ∃ m : Fin T.N, (m : ℕ) + 1 = n ∨ (n : ℕ) + 1 = m
```

lemma flucOf_openHamiltonian_localizedState_ne_zero [source]

***A localized electron fluctuates in energy.** If the chain hops and has two sites, `H |n⟩ - ⟨H⟩ |n⟩ ≠ 0`: its overlap with a neighbour is `-t`.*

```
lemma flucOf_openHamiltonian_localizedState_ne_zero (ht : T.t ≠ 0) (hN : 2 ≤ T.N)
    (n : Fin T.N) : T.flucOf T.openHamiltonian |n⟩ ≠ 0
```

lemma inner_bracket_localizedState [source]

The bracket `⁅H, X⁆` vanishes in a localized electron.

```
lemma inner_bracket_localizedState (n : Fin T.N) :
    ⟪|n⟩, (⁅T.openHamiltonianObservable, T.positionObservable⁆ : Observable _) :
      _ →L[ℂ] _) |n⟩⟫_ℂ = 0
```

lemma robertson_schrodinger_localizedState [source]

***Robertson–Schrödinger is empty in a localized electron.** In `|n⟩`, `Var X = 0` and the centered Gram defect is `0`: both sides of the relation vanish. The energy does not: `Var H > 0` as soon as the chain hops and has two sites.*

```
lemma robertson_schrodinger_localizedState (ht : T.t ≠ 0) (hN : 2 ≤ T.N) (n : Fin T.N) :
    variance (ofVec (T.norm_localizedState n)) T.positionObservable = 0 ∧
      centeredGramDefect (ofVec (T.norm_localizedState n)) T.openHamiltonianObservable
        T.positionObservable = 0 ∧
      0 < variance (ofVec (T.norm_localizedState n)) T.openHamiltonianObservable
```

2.318 PhyslibAlpha.CondensedMatter.TightBindingChain.VelocityBand.Basic

PhyslibAlpha/CondensedMatter/TightBindingChain/VelocityBand/Basic.lean on GitHub.

def meanOf [source]

The mean `re ⟨ψ|A|ψ⟩` of an operator in a vector.

```
noncomputable def meanOf (A : T.HilbertSpace →ₗ[ℂ] T.HilbertSpace) (ψ : T.HilbertSpace) : ℝ
```

def flucOf [source]

The fluctuation `A ψ - ⟨A⟩ ψ` of an operator in a vector.

```
noncomputable def flucOf (A : T.HilbertSpace →ₗ[ℂ] T.HilbertSpace) (ψ : T.HilbertSpace) :
    T.HilbertSpace
```

def defectOf [source]

The centered Gram defect `‖u‖² ‖w‖² - |⟪u, w⟫|²` of `H` and `X` in a vector.

```
noncomputable def defectOf (ψ : T.HilbertSpace) : ℝ
```

lemma continuous_meanOf [source]

```
lemma continuous_meanOf (A : T.HilbertSpace →ₗ[ℂ] T.HilbertSpace) : Continuous (T.meanOf A)
```

lemma continuous_defectOf [source]

```
lemma continuous_defectOf : Continuous T.defectOf
```

lemma defectOf_nonneg [source]

```
lemma defectOf_nonneg (ψ : T.HilbertSpace) : 0 ≤ T.defectOf ψ
```

lemma expectation_ofVec [source]

In a unit vector, the mean is the expectation of the vector state.

```
lemma expectation_ofVec {ψ : T.HilbertSpace} (h : ‖ψ‖ = 1)
    (A : T.HilbertSpace →ₗ[ℂ] T.HilbertSpace) (hA : A.IsSymmetric) :
    ⟪ofVec h⟫⟪T.toObservable A hA⟫ = T.meanOf A ψ
```

lemma centeredGramDefect_ofVec_eq [source]

In a unit vector, `defectOf` is the centered Gram defect of the vector state.

```
lemma centeredGramDefect_ofVec_eq {ψ : T.HilbertSpace} (h : ‖ψ‖ = 1) :
    centeredGramDefect (ofVec h) T.openHamiltonianObservable T.positionObservable =
      T.defectOf ψ
```

def minUncertainty [source]

The unit vectors of minimum uncertainty.

```
noncomputable def minUncertainty : Set T.HilbertSpace
```

lemma isCompact_minUncertainty [source]

```
lemma isCompact_minUncertainty : IsCompact T.minUncertainty
```

lemma localizedState_mem_minUncertainty [source]

A localized electron has minimum uncertainty: its position is sharp.

```
lemma localizedState_mem_minUncertainty (n : Fin T.N) : (|n⟩ : T.HilbertSpace) ∈
    T.minUncertainty
```

def threshold [source]

The largest speed `|⟨J⟩|` of a vector of minimum uncertainty.

```
noncomputable def threshold : ℝ
```

lemma exists_threshold [source]

The threshold is attained by a vector of minimum uncertainty.

```
lemma exists_threshold :
    ∃ ψ ∈ T.minUncertainty, |T.meanOf T.current ψ| = T.threshold
```

lemma le_threshold [source]

```
lemma le_threshold {ψ : T.HilbertSpace} (hψ : ψ ∈ T.minUncertainty) :
    |T.meanOf T.current ψ| ≤ T.threshold
```

lemma threshold_le_maxCurrent [source]

```
lemma threshold_le_maxCurrent (ht : T.t ≠ 0) : T.threshold ≤ T.maxCurrent
```

def velocityBand [source]

The velocity band `(threshold, maxCurrent]` of forced uncertainty.

```
noncomputable def velocityBand : Set ℝ
```

lemma defectOf_pos_of_mem_velocityBand [source]

In the velocity band, the uncertainty defect is forced.

```
lemma defectOf_pos_of_mem_velocityBand {ψ : T.HilbertSpace} (hψ : ‖ψ‖ = 1)
    (hv : |T.meanOf T.current ψ| ∈ T.velocityBand) : 0 < T.defectOf ψ
```

lemma velocityBand_eq_empty [source]

For `N = 2, 3` the maximal current state has minimum uncertainty: the band is empty.

```
lemma velocityBand_eq_empty (ht : T.t ≠ 0) (hN : T.N = 2 ∨ T.N = 3) :
    T.velocityBand = ∅
```

2.319 PhyslibAlpha.CondensedMatter.TightBindingChain.VelocityBand.Width

PhyslibAlpha/CondensedMatter/TightBindingChain/VelocityBand/Width.lean on GitHub.

lemma meanOf_timeReversal [source]

```
lemma meanOf_timeReversal {A : T.HilbertSpace →ₗ[ℂ] T.HilbertSpace}
    (hA : ∀ ψ, A (T.timeReversal ψ) = T.timeReversal (A ψ)) (ψ : T.HilbertSpace) :
    T.meanOf A (T.timeReversal ψ) = T.meanOf A ψ
```

lemma meanOf_current_timeReversal [source]

```
lemma meanOf_current_timeReversal (ψ : T.HilbertSpace) :
    T.meanOf T.current (T.timeReversal ψ) = -T.meanOf T.current ψ
```

lemma flucOf_timeReversal [source]

```
lemma flucOf_timeReversal {A : T.HilbertSpace →ₗ[ℂ] T.HilbertSpace}
    (hA : ∀ ψ, A (T.timeReversal ψ) = T.timeReversal (A ψ)) (ψ : T.HilbertSpace) :
    T.flucOf A (T.timeReversal ψ) = T.timeReversal (T.flucOf A ψ)
```

lemma defectOf_timeReversal [source]

Time reversal keeps the uncertainty defect.

```
lemma defectOf_timeReversal (ψ : T.HilbertSpace) :
    T.defectOf (T.timeReversal ψ) = T.defectOf ψ
```

lemma defectOf_smul [source]

A global phase keeps the uncertainty defect.

```
lemma defectOf_smul {c : ℂ} (hc : ‖c‖ = 1) (ψ : T.HilbertSpace) :
    T.defectOf (c • ψ) = T.defectOf ψ
```

lemma timeReversal_currentEigenstate_one [source]

The time reverse of the longest standing wave is the shortest one.

```
lemma timeReversal_currentEigenstate_one :
    T.timeReversal (T.currentEigenstate 1) = T.currentEigenstate T.N
```

lemma currentBasis_zero [source]

The maximal current state and its time reverse are the extreme current eigenstates.

```
lemma currentBasis_zero (ht : T.t ≠ 0) :
    T.currentBasis ht ⟨0, Nat.pos_of_neZero _⟩ = T.maxCurrentState
```

lemma currentBasis_last [source]

```
lemma currentBasis_last (ht : T.t ≠ 0) :
    T.currentBasis ht ⟨T.N - 1, Nat.sub_lt (Nat.pos_of_neZero _) one_pos⟩ =
      T.timeReversal T.maxCurrentState
```

lemma defectOf_eq_of_meanOf_current_eq [source]

A unit vector moving at the speed limit has the defect of the maximal current state.

```
lemma defectOf_eq_of_meanOf_current_eq (ht : T.t ≠ 0) {ψ : T.HilbertSpace} (hψ : ‖ψ‖ = 1)
    (hv : T.meanOf T.current ψ = T.maxCurrent) :
    T.defectOf ψ = T.defectOf T.maxCurrentState
```

lemma threshold_lt_maxCurrent [source]

From four sites on, the threshold lies strictly below the speed limit.

```
lemma threshold_lt_maxCurrent (ht : T.t ≠ 0) (hN : 4 ≤ T.N) : T.threshold < T.maxCurrent
```

lemma maxCurrent_mem_velocityBand [source]

From four sites on, the velocity band is not empty: it contains the speed limit.

```
lemma maxCurrent_mem_velocityBand (ht : T.t ≠ 0) (hN : 4 ≤ T.N) :
    T.maxCurrent ∈ T.velocityBand
```

2.320 PhyslibAlpha.Electromagnetism.BoxChargeConservation

PhyslibAlpha/Electromagnetism/BoxChargeConservation.lean on GitHub.

lemma div_differentiable_time [source]

```
lemma div_differentiable_time {d} (f : Time → Space d → EuclideanSpace ℝ (Fin d))
    (hf : ContDiff ℝ 2 ↿f) (x : Space d) :
    Differentiable ℝ (fun t => (∇ · f t) x)
```

lemma time_deriv_div_commute [source]

The divergence and the time derivative commute.

```
lemma time_deriv_div_commute {d} (f : Time → Space d → EuclideanSpace ℝ (Fin d))
    (hf : ContDiff ℝ 2 ↿f) (t : Time) (x : Space d) :
    ∂ₜ (fun t => (∇ · f t) x) t = (∇ · fun x => ∂ₜ (fun t => f t x) t) x
```

lemma curl_contDiff [source]

```
lemma curl_contDiff {n : WithTop ℕ∞} (f : Space → EuclideanSpace ℝ (Fin 3))
    (hf : ContDiff ℝ (n + 1) f) : ContDiff ℝ n (∇ × f)
```

lemma div_mk_eq_sum_fderiv [source]

```
lemma div_mk_eq_sum_fderiv {n : ℕ} (f : Space n → EuclideanSpace ℝ (Fin n))
    (hf : Differentiable ℝ f) (x : Fin n → ℝ) :
    (∇ · f) ⟨x⟩ = ∑ i, fderiv ℝ (fun y : Fin n → ℝ => f ⟨y⟩ i) x (Pi.single i 1)
```

lemma integral_div_box [source]

The divergence theorem on a coordinate box in `Space`.

```
lemma integral_div_box {n : ℕ} (f : Space (n + 1) → EuclideanSpace ℝ (Fin (n + 1)))
    (hf : ContDiff ℝ 1 f) (a b : Fin (n + 1) → ℝ) (hle : a ≤ b) :
    ∫ x in Icc a b, (∇ · f) ⟨x⟩ =
      ∑ i, ((∫ y in Icc (a ∘ i.succAbove) (b ∘ i.succAbove), f ⟨i.insertNth (b i) y⟩ i) -
        ∫ y in Icc (a ∘ i.succAbove) (b ∘ i.succAbove), f ⟨i.insertNth (a i) y⟩ i)
```

lemma continuity_of_gauss_ampere [source]

Fields `E`, `B`, `ρ` and `J` obeying Gauss's law for the electric field and Ampère's law obey the continuity equation.

```
lemma continuity_of_gauss_ampere (𝓕 : FreeSpace)
    {E B J : Time → Space → EuclideanSpace ℝ (Fin 3)} {ρ : Time → Space → ℝ}
    (hE : ContDiff ℝ 2 ↿E) (hB : ∀ t, ContDiff ℝ 2 (B t))
    (hGauss : ∀ t x, (∇ · E t) x = ρ t x / 𝓕.ε₀)
    (hAmpere : ∀ t x, (∇ × B t) x = 𝓕.μ₀ • J t x + 𝓕.μ₀ • 𝓕.ε₀ • ∂ₜ (fun t => E t x) t)
    (t : Time) (x : Space) :
    ∂ₜ (fun t => ρ t x) t + (∇ · J t) x = 0
```

lemma magneticField_contDiff_space [source]

```
lemma magneticField_contDiff_space (hV : ContDiff ℝ ∞ V) (t : Time) :
    ContDiff ℝ 2 (V.magneticField 𝓕.c t)
```

theorem continuityEquation [source]

The continuity equation: Maxwell's equations imply local conservation of charge.

```
theorem continuityEquation (t : Time) (x : Space)
    (h : IsExtrema 𝓕 V J₄) (hV : ContDiff ℝ ∞ V) (hJ : ContDiff ℝ ∞ J₄) :
    ∂ₜ (fun t => J₄.chargeDensity 𝓕.c t x) t + (∇ · J₄.currentDensity 𝓕.c t) x = 0
```

def boxFaceCurrent [source]

The electric current at time `t` through the face `x i = s` of the coordinate box `[a, b]`, counted positively in the direction of increasing `x i`.

```
noncomputable def _root_.Electromagnetism.LorentzCurrentDensity.boxFaceCurrent
    (c : SpeedOfLight) (J : LorentzCurrentDensity 3) (t : Time) (a b : Fin 3 → ℝ) (i : Fin 3)
    (s : ℝ) : ℝ
```

def boxOutwardCurrent [source]

The net electric current at time `t` leaving the coordinate box `[a, b]` through its six faces.

```
noncomputable def _root_.Electromagnetism.LorentzCurrentDensity.boxOutwardCurrent
    (c : SpeedOfLight) (J : LorentzCurrentDensity 3) (t : Time) (a b : Fin 3 → ℝ) : ℝ
```

lemma boxOutwardCurrent_eq [source]

The net current leaving the coordinate box `[a, b]` equals minus the rate of change of the charge inside it.

```
lemma boxOutwardCurrent_eq (t : Time) (a b : Fin 3 → ℝ) (hle : a ≤ b)
    (h : IsExtrema 𝓕 V J₄) (hV : ContDiff ℝ ∞ V) (hJ : ContDiff ℝ ∞ J₄) :
    J₄.boxOutwardCurrent 𝓕.c t a b =
      - ∫ x in Icc a b, ∂ₜ (fun t => J₄.chargeDensity 𝓕.c t ⟨x⟩) t
```

lemma boxOutwardCurrent_eq_zero_of_steady [source]

If no charge accumulates inside the coordinate box `[a, b]`, the net current leaving through its six faces is zero. This is motivated by Kirchhoff's current law, with the box enclosing a node of a circuit.

```
lemma boxOutwardCurrent_eq_zero_of_steady (t : Time) (a b : Fin 3 → ℝ) (hle : a ≤ b)
    (h : IsExtrema 𝓕 V J₄) (hV : ContDiff ℝ ∞ V) (hJ : ContDiff ℝ ∞ J₄)
    (hsteady : ∀ x ∈ Icc a b, ∂ₜ (fun t => J₄.chargeDensity 𝓕.c t ⟨x⟩) t = 0) :
    J₄.boxOutwardCurrent 𝓕.c t a b = 0
```

2.321 PhyslibAlpha.Electromagnetism.TemperedDistributional.WireJunction

PhyslibAlpha/Electromagnetism/TemperedDistributional/WireJunction.lean on GitHub.

lemma distTimeDeriv_distSpaceDiv [source]

The divergence and the time derivative of distributions commute.

```
lemma distTimeDeriv_distSpaceDiv {d} (f : (Time × Space d) →d[ℝ] EuclideanSpace ℝ (Fin d)) :
    distTimeDeriv (distSpaceDiv f) = distSpaceDiv (distTimeDeriv f)
```

lemma exists_schwartz_eq_comp_ray [source]

The restriction of a Schwartz function to a line through the origin is a Schwartz function.

```
lemma exists_schwartz_eq_comp_ray {d} (χ : 𝓢(Space d, ℝ)) {w : Space d} (hw : w ≠ 0) :
    ∃ g : 𝓢(ℝ, ℝ), ∀ s : ℝ, g s = χ (s • w)
```

lemma integral_Ioi_fderiv_ray [source]

The fundamental theorem of calculus along the ray in direction `w`.

```
lemma integral_Ioi_fderiv_ray {d} (ψ : 𝓢(Space d, ℝ)) {w : Space d} (hw : w ≠ 0) :
    ∫ s in Ioi (0 : ℝ), fderiv ℝ ψ (s • w) w = - ψ 0
```

lemma integral_Ioi_fderiv_ray_basis [source]

```
lemma integral_Ioi_fderiv_ray_basis {d} (ψ : 𝓢(Space d, ℝ)) {u : EuclideanSpace ℝ (Fin d)}
    (hu : u ≠ 0) :
    ∑ i, (∫ s in Ioi (0 : ℝ), fderiv ℝ ψ (s • basis.repr.symm u) (basis i)) * u i = - ψ 0
```

lemma timeIntegralSchwartz_fderiv_space [source]

Integrating over time commutes with differentiating in space.

```
lemma timeIntegralSchwartz_fderiv_space {d} (η : 𝓢(Time × Space d, ℝ)) (i : Fin d)
    (x : Space d) :
    timeIntegralSchwartz (SchwartzMap.evalCLM ℝ (Time × Space d) ℝ (0, basis i)
      (fderivCLM ℝ (Time × Space d) ℝ η)) x = fderiv ℝ (timeIntegralSchwartz η) x (basis i)
```

lemma exists_integral_time_ne_zero [source]

Some test function has a non-zero time integral at the origin.

```
lemma exists_integral_time_ne_zero {d} :
    ∃ η : 𝓢(Time × Space d, ℝ), ∫ t, η (t, 0) ≠ 0
```

theorem continuityEquation [source]

The continuity equation for distributions: Maxwell's equations imply local conservation of charge.

```
theorem continuityEquation {d} {𝓕 : FreeSpace} (A : TempDistElectromagneticPotential d)
    (J : TempDistLorentzCurrentDensity d) (h : IsExtrema 𝓕 A J) (ε : 𝓢(Time × Space d, ℝ)) :
    distTimeDeriv (J.chargeDensity 𝓕.c) ε + distSpaceDiv (J.currentDensity 𝓕.c) ε = 0
```

def IsWireJunction [source]

The current density of `J` is that of steady currents `I k` flowing away from the origin along thin, straight, semi-infinite wires in the directions `u k`.

```
def IsWireJunction {d} (c : SpeedOfLight) (J : TempDistLorentzCurrentDensity d) {ι : Type}
    [Fintype ι] (u : ι → EuclideanSpace ℝ (Fin d)) (I : ι → ℝ) : Prop
```

lemma distSpaceDiv_currentDensity [source]

The wires of a junction meet at the origin, where they act as a source of strength the total current leaving along them.

```
lemma IsWireJunction.distSpaceDiv_currentDensity {d} {c : SpeedOfLight}
    {J : TempDistLorentzCurrentDensity d} {ι : Type} [Fintype ι] {u : ι → EuclideanSpace ℝ (Fin
     d)}
    {I : ι → ℝ} (hJ : J.IsWireJunction c u I) (hu : ∀ k, u k ≠ 0) (η : 𝓢(Time × Space d, ℝ)) :
    distSpaceDiv (J.currentDensity c) η = (∑ k, I k) * ∫ t, η (t, 0)
```

lemma sum_currents_eq_zero [source]

If Maxwell's equations hold and no charge accumulates, the steady currents leaving a junction of thin wires sum to zero. This is motivated by Kirchhoff's current law, of which it is the special case of a single node.

```
lemma IsWireJunction.sum_currents_eq_zero {d} {𝓕 : FreeSpace} {J :
    TempDistLorentzCurrentDensity d}
    {ι : Type} [Fintype ι] {u : ι → EuclideanSpace ℝ (Fin d)} {I : ι → ℝ}
    (hJ : J.IsWireJunction 𝓕.c u I) (hu : ∀ k, u k ≠ 0) (A : TempDistElectromagneticPotential
    d)
    (h : TempDistElectromagneticPotential.IsExtrema 𝓕 A J)
    (hρ : distTimeDeriv (J.chargeDensity 𝓕.c) = 0) :
    ∑ k, I k = 0
```

2.322 PhyslibAlpha.Mathematics.AlgebraGeneration

PhyslibAlpha/Mathematics/AlgebraGeneration.lean on GitHub.

lemma range_le_of_adjoin_eq_top [source]

If `s` generates `A` and a subalgebra `S` of `B` contains the image of `s`, then it contains the whole range of an algebra map `A →ₐ[R] B`.

```
lemma range_le_of_adjoin_eq_top {s : Set A} (hs : Algebra.adjoin R s = ⊤) (φ : A →ₐ[R] B)
    {S : Subalgebra R B} (h : ∀ x ∈ s, φ x ∈ S) : φ.range ≤ S
```

lemma apply_mem_of_mem_adjoin [source]

An algebra map carrying a generating set into a subalgebra `S` carries the generated subalgebra into `S`. This is what extends the stability of a subalgebra under an endomorphism from its generators to all of its elements.

```
lemma apply_mem_of_mem_adjoin {s : Set A} (φ : A →ₐ[R] B) {S : Subalgebra R B}
    (h : ∀ x ∈ s, φ x ∈ S) {x : A} (hx : x ∈ Algebra.adjoin R s) : φ x ∈ S
```

lemma commute_of_adjoin_eq_top [source]

If `s` generates `A` and `y` commutes with the image of `s` under an algebra map, then `y` commutes with the whole image. This is what extends a commutation relation from generators to a factor of a tensor product.

```
lemma commute_of_adjoin_eq_top {s : Set A} (hs : Algebra.adjoin R s = ⊤) (φ : A →ₐ[R] B)
    {y : B} (h : ∀ x ∈ s, Commute (φ x) y) (a : A) : Commute (φ a) y
```

lemma mem_of_tmul_one_of_one_tmul [source]

A subalgebra of `A ⊗[R] B` containing every `a ⊗ₜ 1` and every `1 ⊗ₜ b` contains everything, since a pure tensor is the product of the two.

```
lemma mem_of_tmul_one_of_one_tmul {S : Subalgebra R (A ⊗[R] B)}
    (hA : ∀ a : A, a ⊗ₜ[R] (1 : B) ∈ S) (hB : ∀ b : B, (1 : A) ⊗ₜ[R] b ∈ S)
    (x : A ⊗[R] B) : x ∈ S
```

lemma eq_top_of_tmul_one_of_one_tmul [source]

A subalgebra of `A ⊗[R] B` containing every `a ⊗ₜ 1` and every `1 ⊗ₜ b` is the whole algebra.

```
lemma eq_top_of_tmul_one_of_one_tmul {S : Subalgebra R (A ⊗[R] B)}
    (hA : ∀ a : A, a ⊗ₜ[R] (1 : B) ∈ S) (hB : ∀ b : B, (1 : A) ⊗ₜ[R] b ∈ S) : S = ⊤
```

lemma includeRight_mul_comm [source]

A commutative right factor is central in the tensor product. Stated as an equation of products at abstract types, so that it can be instantiated on a concrete tensor product without unifying its multiplication instances.

```
lemma includeRight_mul_comm (b : B) (x : A ⊗[R] B) :
    x * includeRight (R := R) (A := A) b = includeRight (R := R) (A := A) b * x
```

lemma mem_adjoin_one_tmul [source]

Extension of scalars on the elements `1 ⊗ₜ a`. If `s` generates `A` over `R` then every `1 ⊗ₜ a` lies in the `S`-subalgebra of `S ⊗[R] A` generated by `1 ⊗ₜ s`.

```
lemma mem_adjoin_one_tmul (hs : Algebra.adjoin R s = ⊤) (a : A) :
    (1 : S) ⊗ₜ[R] a ∈ Algebra.adjoin S ((fun a : A => (1 : S) ⊗ₜ[R] a) '' s)
```

lemma mem_adjoin_one_tmul_of_generates [source]

Every element of a base change lies in the subalgebra generated by the `1 ⊗ₜ s`.

```
lemma mem_adjoin_one_tmul_of_generates (hs : Algebra.adjoin R s = ⊤) (x : S ⊗[R] A) :
    x ∈ Algebra.adjoin S ((fun a : A => (1 : S) ⊗ₜ[R] a) '' s)
```

lemma adjoin_one_tmul_eq_top [source]

Extension of scalars preserves generation. If `s` generates `A` over `R` then the elements `1 ⊗ₜ a`, for `a ∈ s`, generate `S ⊗[R] A` over `S`.

```
lemma adjoin_one_tmul_eq_top (hs : Algebra.adjoin R s = ⊤) :
    Algebra.adjoin S ((fun a : A => (1 : S) ⊗ₜ[R] a) '' s)
      = (⊤ : Subalgebra S (S ⊗[R] A))
```

lemma mul_swap_of_mul_self [source]

Polarization. If every value of a linear map into a ring squares to zero, then any two values anticommute. Applied to a space of fermionic generators, this is what

makes the single relation "every vector squares to zero" carry the anticommutation of distinct generators, including generators of different species, which sit in the same space.

```
lemma LinearMap.mul_swap_of_mul_self {R M B : Type*} [CommSemiring R] [AddCommMonoid M]
    [Module R M] [Ring B] [Algebra R B] (f : M →ₗ[R] B) (h : ∀ v, f v * f v = 0) (v w : M) :
    f v * f w = -(f w * f v)
```

lemma mem_of_lof [source]

If a subalgebra contains the image of every summand, it contains the whole image.

```
lemma mem_of_lof {S : Subalgebra R B} {F : (⨁ i, M i) →ₗ[R] B}
    (h : ∀ i x, F (lof R ι M i x) ∈ S) (v : ⨁ i, M i) : F v ∈ S
```

lemma commute_of_lof_left [source]

If the image of every summand commutes with a fixed element, so does the whole image.

```
lemma commute_of_lof_left {F : (⨁ i, M i) →ₗ[R] B} {b : B}
    (h : ∀ i x, Commute (F (lof R ι M i x)) b) (v : ⨁ i, M i) : Commute (F v) b
```

lemma commute_of_lof [source]

Commutation extends from the summands. If the image of every summand of one direct sum commutes with the image of every summand of another, then the two images commute elementwise.

```
lemma commute_of_lof {F : (⨁ i, M i) →ₗ[R] B} {G : (⨁ j, N j) →ₗ[R] B}
    (h : ∀ i j x y, Commute (F (lof R ι M i x)) (G (lof R κ N j y)))
    (v : ⨁ i, M i) (w : ⨁ j, N j) : Commute (F v) (G w)
```

lemma mul_swap_of_lof [source]

Anticommutation extends from the summands.

```
lemma mul_swap_of_lof {F : (⨁ i, M i) →ₗ[R] B}
    (h : ∀ i j x y, F (lof R ι M i x) * F (lof R ι M j y)
      = -(F (lof R ι M j y) * F (lof R ι M i x)))
    (v w : ⨁ i, M i) : F v * F w = -(F w * F v)
```

lemma mul_self_of_lof [source]

Square-zero extends from the summands, given anticommutation across them. The cross terms of `(x + y) * (x + y)` cancel exactly because the two images anticommute; square-zero on each summand alone would leave them.

```
lemma mul_self_of_lof {F : (⨁ i, M i) →ₗ[R] B}
    (hsq : ∀ i x, F (lof R ι M i x) * F (lof R ι M i x) = 0)
    (hswap : ∀ i j x y, F (lof R ι M i x) * F (lof R ι M j y)
      = -(F (lof R ι M j y) * F (lof R ι M i x)))
    (v : ⨁ i, M i) : F v * F v = 0
```

lemma mul_self_iff_lof [source]

Square-zero on a direct sum is not square-zero summand by summand. It is equivalent to square-zero on each summand together with anticommutation between the images of any two summands, the second condition being vacuous only when there is at most one summand. This is the precise sense in which Fermi statistics for a family of species is more than the Fermi statistics of the individual species.

```
lemma mul_self_iff_lof {F : (⨁ i, M i) →ₗ[R] B} :
    (∀ v, F v * F v = 0) ↔
      ((∀ i x, F (lof R ι M i x) * F (lof R ι M i x) = 0) ∧
        ∀ i j x y, F (lof R ι M i x) * F (lof R ι M j y)
          = -(F (lof R ι M j y) * F (lof R ι M i x)))
```

2.323 PhyslibAlpha.Mathematics.AlgebraRepresentation

PhyslibAlpha/Mathematics/AlgebraRepresentation.lean on GitHub.

lemma tprod_apply_one [source]

The tensor product of two unit-preserving representations preserves the unit.

```
lemma tprod_apply_one {k G A B : Type*} [CommSemiring k] [Monoid G]
    [Ring A] [Algebra k A] [Ring B] [Algebra k B]
    (ρ : Representation k G A) (σ : Representation k G B) (g : G)
    (hρ : ρ g 1 = 1) (hσ : σ g 1 = 1) : (ρ.tprod σ) g 1 = 1
```

lemma tprod_apply_one_tmul [source]

On a pure tensor whose left entry is the unit only the right factor moves.

```
lemma tprod_apply_one_tmul {k G A B : Type*} [CommSemiring k]
    [Monoid G] [Ring A] [Algebra k A] [Ring B] [Algebra k B]
    (ρ : Representation k G A) (σ : Representation k G B) (g : G)
    (hρ : ρ g 1 = 1) (y : B) :
    (ρ.tprod σ) g ((1 : A) ⊗ₜ[k] y) = (1 : A) ⊗ₜ[k] σ g y
```

lemma tprod_apply_tmul_one [source]

On a pure tensor whose right entry is the unit only the left factor moves.

```
lemma tprod_apply_tmul_one {k G A B : Type*} [CommSemiring k]
    [Monoid G] [Ring A] [Algebra k A] [Ring B] [Algebra k B]
    (ρ : Representation k G A) (σ : Representation k G B) (g : G) (x : A)
    (hσ : σ g 1 = 1) :
    (ρ.tprod σ) g (x ⊗ₜ[k] (1 : B)) = ρ g x ⊗ₜ[k] (1 : B)
```

lemma tprod_apply_mul [source]

The tensor product of two multiplicative representations on algebras is multiplicative.

```
lemma tprod_apply_mul {k G A B : Type*} [CommSemiring k] [Monoid G]
    [Ring A] [Algebra k A] [Ring B] [Algebra k B]
    (ρ : Representation k G A) (σ : Representation k G B)
    (hρ : ∀ (g : G) (x y : A), ρ g (x * y) = ρ g x * ρ g y)
    (hσ : ∀ (g : G) (x y : B), σ g (x * y) = σ g x * σ g y)
```

```
    (g : G) (x y : A ⊗[k] B) :
    (ρ.tprod σ) g (x * y) = (ρ.tprod σ) g x * (ρ.tprod σ) g y
```

def Representation.restrictSubalgebra [source]

The restriction of a representation to a subalgebra each group element preserves.

```
noncomputable def restrictSubalgebra {k A G : Type*} [CommSemiring k]
    [Monoid G] [Semiring A] [Algebra k A] (ρ : Representation k G A) (S : Subalgebra k A)
    (hS : ∀ (g : G) {x : A}, x ∈ S → ρ g x ∈ S) : Representation k G S where
```

lemma coe_restrictSubalgebra [source]

```
@[simp]
lemma coe_restrictSubalgebra {k A G : Type*} [CommSemiring k]
    [Monoid G] [Semiring A] [Algebra k A] (ρ : Representation k G A) (S : Subalgebra k A)
    (hS : ∀ (g : G) {x : A}, x ∈ S → ρ g x ∈ S) (g : G) (x : S) :
    (ρ.restrictSubalgebra S hS g x : A) = ρ g (x : A)
```

lemma inclusion_restrictSubalgebra [source]

The restrictions to two nested invariant subalgebras agree along the inclusion of the smaller into the larger.

```
lemma inclusion_restrictSubalgebra {k A G : Type*} [CommSemiring k]
    [Monoid G] [Semiring A] [Algebra k A] (ρ : Representation k G A) {S S' : Subalgebra k A}
    (h : S ≤ S') (hS : ∀ (g : G) {x : A}, x ∈ S → ρ g x ∈ S)
    (hS' : ∀ (g : G) {x : A}, x ∈ S' → ρ g x ∈ S') (g : G) (x : S) :
    Subalgebra.inclusion h (ρ.restrictSubalgebra S hS g x)
      = ρ.restrictSubalgebra S' hS' g (Subalgebra.inclusion h x)
```

def restrictScalars [source]

The restriction of scalars of a representation: a representation on an `S`-module is a representation on the same space as an `R`-module, for `R` acting through `S`.

```
def restrictScalars (R : Type*) {S G V : Type*} [CommSemiring R] [CommSemiring S] [Monoid G]
    [AddCommMonoid V] [Module R V] [Module S V] [LinearMap.CompatibleSMul V V R S]
    (ρ : Representation S G V) : Representation R G V where
```

lemma restrictScalars_apply [source]

```
@[simp]
lemma restrictScalars_apply (R : Type*) {S G V : Type*} [CommSemiring R] [CommSemiring S]
    [Monoid G] [AddCommMonoid V] [Module R V] [Module S V] [LinearMap.CompatibleSMul V V R S]
    (ρ : Representation S G V) (g : G) (x : V) : ρ.restrictScalars R g x = ρ g x
```

lemma apply_one_of_mul [source]

A representation of a group acting by multiplicative maps preserves the unit. The action of `g` is surjective, its inverse being the action of `g⁻¹`, so `ρ g 1` is a left unit on the whole algebra.

```
lemma apply_one_of_mul (g : G) : ρ g 1 = 1
```

def toAlgHom [source]

A representation of a group acting by multiplicative maps acts by algebra endomorphisms.

```
noncomputable def toAlgHom (g : G) : A →ₐ[k] A where
```

lemma toAlgHom_apply [source]

```
@[simp]
lemma toAlgHom_apply (g : G) (x : A) : ρ.toAlgHom hρ g x = ρ g x
```

structure EquivariantAlgHom [source]

An algebra map `A →ₐ[k] B` intertwining two pairs of representations, `ρ₁, σ₁` of the monoid `G₁` and `ρ₂, σ₂` of the monoid `G₂`, with both target representations multiplicative on the whole of `B`. The two pairs are independent: the actions of `G₁` and `G₂` are not assumed to commute, and no relation between them is used. Unit preservation is not a field and does not follow from multiplicativity over a monoid; when the acting monoid is a group and the target is a ring it does (`Representation.apply_one_of_mul`), and the target action is then by algebra endomorphisms.

```
@[ext]
structure EquivariantAlgHom (ρ₁ : Representation k G₁ A) (σ₁ : Representation k G₁ B)
    (ρ₂ : Representation k G₂ A) (σ₂ : Representation k G₂ B) where
  toAlgHom : A →ₐ[k] B
  map_fst : ∀ (g : G₁) (x : A), toAlgHom (ρ₁ g x) = σ₁ g (toAlgHom x)
  map_snd : ∀ (g : G₂) (x : A), toAlgHom (ρ₂ g x) = σ₂ g (toAlgHom x)
  fst_mul : ∀ (g : G₁) (b₁ b₂ : B), σ₁ g (b₁ * b₂) = σ₁ g b₁ * σ₁ g b₂
  snd_mul : ∀ (g : G₂) (b₁ b₂ : B), σ₂ g (b₁ * b₂) = σ₂ g b₁ * σ₂ g b₂
```

def id [source]

The identity map of an algebra carrying two multiplicative representations.

```
def id (h₁ : ∀ (g : G₁) (x y : A), ρ₁ g (x * y) = ρ₁ g x * ρ₁ g y)
    (h₂ : ∀ (g : G₂) (x y : A), ρ₂ g (x * y) = ρ₂ g x * ρ₂ g y) :
    EquivariantAlgHom ρ₁ ρ₁ ρ₂ ρ₂ where
```

lemma id_toAlgHom [source]

```
@[simp]
lemma id_toAlgHom (h₁ : ∀ (g : G₁) (x y : A), ρ₁ g (x * y) = ρ₁ g x * ρ₁ g y)
    (h₂ : ∀ (g : G₂) (x y : A), ρ₂ g (x * y) = ρ₂ g x * ρ₂ g y) :
    (EquivariantAlgHom.id ρ₁ ρ₂ h₁ h₂).toAlgHom = AlgHom.id k A
```

def comp [source]

The precomposition with an algebra map into the source intertwining two representations on its own source with the two source representations; the target and its two actions are unchanged.

```
def comp {A' : Type*} [Semiring A'] [Algebra k A'] {ρ₁' : Representation k G₁ A'}
    {ρ₂' : Representation k G₂ A'} (f : EquivariantAlgHom ρ₁ σ₁ ρ₂ σ₂) (φ : A' →ₐ[k] A)
    (hφ₁ : ∀ (g : G₁) (x : A'), φ (ρ₁' g x) = ρ₁ g (φ x))
```

```
    (hφ₂ : ∀ (g : G₂) (x : A'), φ (ρ₂' g x) = ρ₂ g (φ x)) :
    EquivariantAlgHom ρ₁' σ₁ ρ₂' σ₂ where
```

lemma comp_toAlgHom [source]

```
@[simp]
lemma comp_toAlgHom {A' : Type*} [Semiring A'] [Algebra k A'] {ρ₁' : Representation k G₁ A'}
    {ρ₂' : Representation k G₂ A'} (f : EquivariantAlgHom ρ₁ σ₁ ρ₂ σ₂) (φ : A' →ₐ[k] A)
    (hφ₁ : ∀ (g : G₁) (x : A'), φ (ρ₁' g x) = ρ₁ g (φ x))
    (hφ₂ : ∀ (g : G₂) (x : A'), φ (ρ₂' g x) = ρ₂ g (φ x)) :
    (f.comp φ hφ₁ hφ₂).toAlgHom = f.toAlgHom.comp φ
```

def compEquiv [source]

Precomposition with an algebra equivalence of the sources intertwining the source representations: equivariant maps out of equivalent sources correspond.

```
noncomputable def compEquiv {A' : Type*} [Semiring A'] [Algebra k A']
    {ρ₁' : Representation k G₁ A'} {ρ₂' : Representation k G₂ A'} (e : A' ≃ₐ[k] A)
    (he₁ : ∀ (g : G₁) (x : A'), e (ρ₁' g x) = ρ₁ g (e x))
    (he₂ : ∀ (g : G₂) (x : A'), e (ρ₂' g x) = ρ₂ g (e x)) :
    EquivariantAlgHom ρ₁ σ₁ ρ₂ σ₂ ≃ EquivariantAlgHom ρ₁' σ₁ ρ₂' σ₂ where
```

lemma compEquiv_toAlgHom [source]

```
@[simp]
lemma compEquiv_toAlgHom {A' : Type*} [Semiring A'] [Algebra k A']
    {ρ₁' : Representation k G₁ A'} {ρ₂' : Representation k G₂ A'} (e : A' ≃ₐ[k] A)
    (he₁ : ∀ (g : G₁) (x : A'), e (ρ₁' g x) = ρ₁ g (e x))
    (he₂ : ∀ (g : G₂) (x : A'), e (ρ₂' g x) = ρ₂ g (e x))
    (f : EquivariantAlgHom ρ₁ σ₁ ρ₂ σ₂) :
    (compEquiv e he₁ he₂ f).toAlgHom = f.toAlgHom.comp e.toAlgHom
```

lemma compEquiv_symm_toAlgHom [source]

```
@[simp]
lemma compEquiv_symm_toAlgHom {A' : Type*} [Semiring A'] [Algebra k A']
    {ρ₁' : Representation k G₁ A'} {ρ₂' : Representation k G₂ A'} (e : A' ≃ₐ[k] A)
    (he₁ : ∀ (g : G₁) (x : A'), e (ρ₁' g x) = ρ₁ g (e x))
    (he₂ : ∀ (g : G₂) (x : A'), e (ρ₂' g x) = ρ₂ g (e x))
    (f : EquivariantAlgHom ρ₁' σ₁ ρ₂' σ₂) :
    ((compEquiv e he₁ he₂).symm f).toAlgHom = f.toAlgHom.comp e.symm.toAlgHom
```

def EquivariantAlgHom.restrictSubalgebra [source]

The restriction to a subalgebra of the source preserved by both source representations, along its inclusion; the target and its two actions are unchanged.

```
noncomputable def restrictSubalgebra (f : EquivariantAlgHom ρ₁ σ₁ ρ₂ σ₂) (S : Subalgebra k A)
    (hS₁ : ∀ (g : G₁) {x : A}, x ∈ S → ρ₁ g x ∈ S)
    (hS₂ : ∀ (g : G₂) {x : A}, x ∈ S → ρ₂ g x ∈ S) :
    EquivariantAlgHom (ρ₁.restrictSubalgebra S hS₁) σ₁ (ρ₂.restrictSubalgebra S hS₂) σ₂
```

lemma restrictSubalgebra_toAlgHom_apply [source]

```
@[simp]
lemma restrictSubalgebra_toAlgHom_apply (f : EquivariantAlgHom ρ₁ σ₁ ρ₂ σ₂) (S : Subalgebra k A
    )
    (hS₁ : ∀ (g : G₁) {x : A}, x ∈ S → ρ₁ g x ∈ S)
    (hS₂ : ∀ (g : G₂) {x : A}, x ∈ S → ρ₂ g x ∈ S) (x : S) :
    (f.restrictSubalgebra S hS₁ hS₂).toAlgHom x = f.toAlgHom x
```

def compFst [source]

The precomposition of the first pair of representations with a monoid map; the algebra map is unchanged.

```
def compFst {H : Type*} [Monoid H] (f : EquivariantAlgHom ρ₁ σ₁ ρ₂ σ₂) (φ : H →* G₁) :
    EquivariantAlgHom (ρ₁.comp φ) (σ₁.comp φ) ρ₂ σ₂ where
```

lemma compFst_toAlgHom [source]

```
@[simp]
lemma compFst_toAlgHom {H : Type*} [Monoid H] (f : EquivariantAlgHom ρ₁ σ₁ ρ₂ σ₂) (φ : H →* G₁)
     :
    (f.compFst φ).toAlgHom = f.toAlgHom
```

lemma liftEquiv_baseChange [source]

Base change along `R → S` preserves equivariance: the `S`-algebra map out of `S ⊗[R] A` corresponding to an equivariant `R`-algebra map `f` intertwines the base change of the source action with the target action.

```
lemma liftEquiv_baseChange {R S A B G : Type*} [CommSemiring R] [CommSemiring S] [Algebra R S]
    [Semiring A] [Algebra R A] [Semiring B] [Algebra S B] [Algebra R B] [IsScalarTower R S B]
    [Monoid G] (f : A →ₐ[R] B) (ρ : Representation R G A) (σ : Representation S G B)
    (hf : ∀ (g : G) (x : A), f (ρ g x) = σ g (f x)) (g : G) (x : S ⊗[R] A) :
    AlgHom.liftEquiv R S A B f (LinearMap.baseChange S (ρ g) x)
      = σ g (AlgHom.liftEquiv R S A B f x)
```

def baseChange [source]

The base change of a representation along `R → S`: the same action on the second factor of `S ⊗[R] M`.

```
noncomputable def baseChange {R M G : Type*} [CommSemiring R] (S : Type*) [CommSemiring S]
    [Algebra R S] [AddCommMonoid M] [Module R M] [Monoid G] (ρ : Representation R G M) :
    Representation S G (S ⊗[R] M) where
```

lemma baseChange_tmul [source]

```
@[simp]
lemma baseChange_tmul {R M G : Type*} [CommSemiring R] (S : Type*) [CommSemiring S]
    [Algebra R S] [AddCommMonoid M] [Module R M] [Monoid G] (ρ : Representation R G M) (g : G)
    (s : S) (x : M) : baseChange S ρ g (s ⊗ₜ[R] x) = s ⊗ₜ[R] ρ g x
```

lemma baseChange_naturality [source]

Base change is natural: a linear map intertwining two representations base changes to one intertwining their base changes.

```
lemma baseChange_naturality {R M N G : Type*} [CommSemiring R] (S : Type*) [CommSemiring S]
    [Algebra R S] [AddCommMonoid M] [Module R M] [AddCommMonoid N] [Module R N] [Monoid G]
    {ρ : Representation R G M} {σ : Representation R G N} (f : M →ₗ[R] N)
    (h : ∀ (g : G) (x : M), f (ρ g x) = σ g (f x)) (g : G) (y : S ⊗[R] M) :
    LinearMap.baseChange S f (baseChange S ρ g y)
      = baseChange S σ g (LinearMap.baseChange S f y)
```

lemma eq_baseChange_of_tmul [source]

A representation on a base change acting on the pure tensors through a representation on the second factor is that representation's base change.

```
lemma eq_baseChange_of_tmul {R S A G : Type*} [CommSemiring R] [CommSemiring S] [Algebra R S]
    [AddCommMonoid A] [Module R A] [Monoid G] (ρ : Representation R G A)
    (ρ' : Representation S G (S ⊗[R] A))
    (h : ∀ (g : G) (s : S) (x : A), ρ' g (s ⊗ₜ[R] x) = s ⊗ₜ[R] ρ g x) (g : G) (y : S ⊗[R] A) :
    ρ' g y = LinearMap.baseChange S (ρ g) y
```

def liftEquivBaseChange [source]

For a target over `S`, the equivariant `R`-algebra maps out of `A` are the equivariant `S`-algebra maps out of the base change `S ⊗[R] A`, by `AlgHom.liftEquiv`. The two source actions on the base change are recognised by their values on the pure tensors, and the target keeps its `S`-actions, restricted to `R` on the left-hand side.

```
noncomputable def EquivariantAlgHom.liftEquivBaseChange {R S A B G₁ G₂ : Type*} [CommSemiring R
    ]
    [CommSemiring S] [Algebra R S] [Semiring A] [Algebra R A] [Semiring B] [Algebra S B]
    [Algebra R B] [IsScalarTower R S B] [Monoid G₁] [Monoid G₂]
    {ρ₁ : Representation R G₁ A} {ρ₂ : Representation R G₂ A}
    {ρ₁' : Representation S G₁ (S ⊗[R] A)} {ρ₂' : Representation S G₂ (S ⊗[R] A)}
    {σ₁ : Representation S G₁ B} {σ₂ : Representation S G₂ B}
    (h₁ : ∀ (g : G₁) (s : S) (x : A), ρ₁' g (s ⊗ₜ[R] x) = s ⊗ₜ[R] ρ₁ g x)
    (h₂ : ∀ (g : G₂) (s : S) (x : A), ρ₂' g (s ⊗ₜ[R] x) = s ⊗ₜ[R] ρ₂ g x) :
    EquivariantAlgHom ρ₁ (σ₁.restrictScalars R) ρ₂ (σ₂.restrictScalars R)
      ≃ EquivariantAlgHom ρ₁' σ₁ ρ₂' σ₂ where
```

lemma map_add_smul_one [source]

An algebra map carries an affine combination `x + z • 1` to the same combination of the image, the shift being the image of a scalar.

```
lemma AlgHom.map_add_smul_one {k A B : Type*} [CommSemiring k] [Semiring A] [Algebra k A]
    [Semiring B] [Algebra k B] (f : A →ₐ[k] B) (x : A) (z : k) :
    f (x + z • (1 : A)) = f x + z • (1 : B)
```

2.324 PhyslibAlpha.Mathematics.ExteriorAlgebra

PhyslibAlpha/Mathematics/ExteriorAlgebra.lean on GitHub.

def derivationGen [source]

The generator map of the derivation extending `d`, into the trivial square-zero extension of the exterior algebra: `ι x ↦ (ι x, ι (d x))`.

```
noncomputable def derivationGen :
    M →ₗ[R] TrivSqZeroExt (ExteriorAlgebra R M) (ExteriorAlgebra R M) where
```

lemma derivationGen_fst [source]

```
@[simp]
lemma derivationGen_fst (x : M) : (derivationGen d x).fst = ι R x
```

lemma derivationGen_snd [source]

```
@[simp]
lemma derivationGen_snd (x : M) : (derivationGen d x).snd = ι R (d x)
```

lemma derivationGen_mul_self [source]

The generator map squares to zero: degree-one elements of the exterior algebra anticommute, which is exactly the square-zero condition on the pair.

```
lemma derivationGen_mul_self (x : M) : derivationGen d x * derivationGen d x = 0
```

def derivationHom [source]

The lift of the derivation extending `d` to the trivial square-zero extension of the exterior algebra: the algebra homomorphism `x ↦ (x, derivationOfLinear d x)`.

```
noncomputable def derivationHom :
    ExteriorAlgebra R M →ₐ[R]
      TrivSqZeroExt (ExteriorAlgebra R M) (ExteriorAlgebra R M)
```

lemma derivationHom_ι [source]

```
@[simp]
lemma derivationHom_ι (x : M) : derivationHom d (ι R x) = derivationGen d x
```

lemma derivationHom_fst [source]

The first component of the square-zero lift is the identity.

```
@[simp]
lemma derivationHom_fst (x : ExteriorAlgebra R M) : (derivationHom d x).fst = x
```

def derivationOfLinear [source]

The even derivation of the exterior algebra extending a linear endomorphism `d` of `M`: the map obeying the Leibniz rule, with no Koszul signs, whose value on a generator `ι x` is `ι (d x)`.

```
noncomputable def derivationOfLinear : ExteriorAlgebra R M →ₗ[R] ExteriorAlgebra R M where
```

lemma derivationOfLinear_ι [source]

```
@[simp]
lemma derivationOfLinear_ι (x : M) : derivationOfLinear d (ι R x) = ι R (d x)
```

lemma derivationOfLinear_one [source]

```
@[simp]
lemma derivationOfLinear_one : derivationOfLinear d (1 : ExteriorAlgebra R M) = 0
```

lemma derivationOfLinear_algebraMap [source]

```
@[simp]
lemma derivationOfLinear_algebraMap (r : R) :
    derivationOfLinear d (algebraMap R (ExteriorAlgebra R M) r) = 0
```

lemma derivationOfLinear_mul [source]

The Leibniz rule for the derivation extending `d`, with no Koszul signs: the derivation is even even though the generators are odd.

```
lemma derivationOfLinear_mul (x y : ExteriorAlgebra R M) :
    derivationOfLinear d (x * y)
      = derivationOfLinear d x * y + x * derivationOfLinear d y
```

lemma derivationOfLinear_comm_apply [source]

Derivations extending commuting endomorphisms commute.

```
lemma derivationOfLinear_comm_apply {d₁ d₂ : M →ₗ[R] M} (h : d₁ ∘ₗ d₂ = d₂ ∘ₗ d₁)
    (x : ExteriorAlgebra R M) :
    derivationOfLinear d₁ (derivationOfLinear d₂ x)
      = derivationOfLinear d₂ (derivationOfLinear d₁ x)
```

lemma algHom_derivationOfLinear [source]

An algebra map carries one derivation to the other when the linear map it is determined by on the generators intertwines the two endomorphisms. The hypotheses are stated pointwise so that the lemma can be applied without rewriting inside a large algebra.

```
lemma algHom_derivationOfLinear {N : Type*} [AddCommGroup N] [Module R N]
    (F : ExteriorAlgebra R M →ₐ[R] ExteriorAlgebra R N) {f : M →ₗ[R] N}
    (hF : ∀ x, F (ι R x) = ι R (f x)) {d : M →ₗ[R] M} {d' : N →ₗ[R] N}
    (h : ∀ x, f (d x) = d' (f x)) (y : ExteriorAlgebra R M) :
    F (derivationOfLinear d y) = derivationOfLinear d' (F y)
```

def exteriorAlgebra [source]

The representation on the exterior algebra extending a representation on the generators, by functoriality of `ExteriorAlgebra.map`.

```
noncomputable def exteriorAlgebra (ρ : Representation R G M) :
    Representation R G (ExteriorAlgebra R M) where
```

lemma exteriorAlgebra_apply [source]

```
lemma exteriorAlgebra_apply (ρ : Representation R G M) (g : G)
    (x : ExteriorAlgebra R M) :
    ρ.exteriorAlgebra g x = ExteriorAlgebra.map (ρ g) x
```

lemma exteriorAlgebra_ι [source]

```
@[simp]
lemma exteriorAlgebra_ι (ρ : Representation R G M) (g : G) (x : M) :
    ρ.exteriorAlgebra g (ExteriorAlgebra.ι R x) = ExteriorAlgebra.ι R (ρ g x)
```

lemma exteriorAlgebra_apply_one [source]

```
@[simp]
lemma exteriorAlgebra_apply_one (ρ : Representation R G M) (g : G) :
    ρ.exteriorAlgebra g (1 : ExteriorAlgebra R M) = 1
```

lemma exteriorAlgebra_apply_mul [source]

```
lemma exteriorAlgebra_apply_mul (ρ : Representation R G M) (g : G)
    (x y : ExteriorAlgebra R M) :
    ρ.exteriorAlgebra g (x * y) = ρ.exteriorAlgebra g x * ρ.exteriorAlgebra g y
```

lemma exteriorAlgebra_algebraMap [source]

```
lemma exteriorAlgebra_algebraMap (ρ : Representation R G M) (g : G) (r : R) :
    ρ.exteriorAlgebra g (algebraMap R (ExteriorAlgebra R M) r)
      = algebraMap R (ExteriorAlgebra R M) r
```

lemma exteriorAlgebra_derivationOfLinear [source]

Covariance of the derivation under the representation. If the representation carries each endomorphism of a family into a combination of the others on the generators, then it carries the derivation extending one into the same combination of the derivations extending the others on the whole exterior algebra. This is the shape the statement that the total derivative is a Lorentz vector takes; it is proved by induction rather than by algebra-map extensionality, a derivation not being an algebra map.

```
lemma exteriorAlgebra_derivationOfLinear {G κ : Type*} [Monoid G] [Fintype κ]
    (ρ : Representation R G M) (g : G) (d : κ → M →ₗ[R] M) (μ : κ) (c : κ → R)
    (h : ∀ x, ρ g (d μ x) = ∑ a, c a • d a (ρ g x)) (y : ExteriorAlgebra R M) :
    ρ.exteriorAlgebra g (derivationOfLinear (d μ) y)
      = ∑ a, c a • derivationOfLinear (d a) (ρ.exteriorAlgebra g y)
```

lemma algHom_exteriorAlgebra [source]

An algebra map carries one representation to the other when the linear map it is determined by on the generators intertwines the two representations on them. Unlike the derivation statement this is pure extensionality of algebra maps, each group element acting by one; the hypotheses are stated pointwise so that the lemma can be applied without rewriting inside a large algebra.

```
lemma algHom_exteriorAlgebra {N G : Type*} [AddCommGroup N] [Module R N] [Monoid G]
    (F : ExteriorAlgebra R M →ₐ[R] ExteriorAlgebra R N) {f : M →ₗ[R] N}
    (hF : ∀ x, F (ι R x) = ι R (f x))
    {ρ : Representation R G M} {σ : Representation R G N} (g : G)
    (h : ∀ x, f (ρ g x) = σ g (f x)) (y : ExteriorAlgebra R M) :
    F (ρ.exteriorAlgebra g y) = σ.exteriorAlgebra g (F y)
```

def mapEquiv [source]

Transport of an exterior algebra along a linear equivalence, built from `ExteriorAlgebra.map` so that no `CliffordAlgebra` isometry has to be unfolded.

```
noncomputable def mapEquiv {R M N : Type*} [CommRing R] [AddCommGroup M]
    [Module R M] [AddCommGroup N] [Module R N] (e : M ≃ₗ[R] N) :
    ExteriorAlgebra R M ≃ₐ[R] ExteriorAlgebra R N
```

lemma mapEquiv_apply_ι [source]

```
@[simp]
lemma mapEquiv_apply_ι {R M N : Type*} [CommRing R] [AddCommGroup M]
    [Module R M] [AddCommGroup N] [Module R N] (e : M ≃ₗ[R] N) (x : M) :
    mapEquiv e (ExteriorAlgebra.ι R x) = ExteriorAlgebra.ι R (e x)
```

2.325 PhyslibAlpha.Mathematics.ForMathlib.DataStructures.Matrix.Scalar

PhyslibAlpha/Mathematics/ForMathlib/DataStructures/Matrix/Scalar.lean on GitHub.

lemma star_scalar [source]

The conjugate transpose of a scalar matrix is the scalar matrix of the star.

```
lemma star_scalar [StarRing R] (x : R) : star (scalar n x) = scalar n (star x)
```

lemma map_scalar [source]

An entrywise map fixing zero sends a scalar matrix to a scalar matrix.

```
lemma map_scalar {S : Type*} [CommRing S] (f : R → S) (hf : f 0 = 0) (x : R) :
    (scalar n x).map f = scalar n (f x)
```

lemma scalar_smul [source]

A scalar action passes into a scalar matrix.

```
lemma scalar_smul {M : Type*} [Monoid M] [DistribMulAction M R] (c : M) (x : R) :
    scalar n (c • x) = c • scalar n x
```

lemma scalar_eq_conj [source]

Conjugation by a unitary scalar fixes a scalar matrix.

```
lemma scalar_eq_conj [StarRing R] {u : R} (hu : u * star u = 1) (x : R) :
    scalar n x = scalar n u * scalar n x * star (scalar n u)
```

def scalarSelfAdjoint [source]

A self-adjoint element as a scalar matrix, real-linearly.

```
noncomputable def scalarSelfAdjoint [StarRing R] [Algebra ℝ R] [StarModule ℝ R] :
    ↥(selfAdjoint R) →ₗ[ℝ] Matrix n n R where
```

lemma scalarSelfAdjoint_apply [source]

```
@[simp]
lemma scalarSelfAdjoint_apply [StarRing R] [Algebra ℝ R] [StarModule ℝ R]
    (a : ↥(selfAdjoint R)) : (scalarSelfAdjoint (n := n)) a = scalar n a.1
```

lemma eq_scalar_fin_one [source]

A `1 × 1` matrix is the scalar matrix of its entry.

```
lemma eq_scalar_fin_one (A : Matrix (Fin 1) (Fin 1) R) : A = scalar (Fin 1) (A 0 0)
```

2.326 PhyslibAlpha.Mathematics.ForMathlib.Fin

PhyslibAlpha/Mathematics/ForMathlib/Fin.lean on GitHub.

lemma sum_pi_succ_prod_smul [source]

A sum over tuples of indices, weighted by one scalar factor per slot, split into the first slot and the remaining ones.

```
lemma sum_pi_succ_prod_smul {R M ι : Type*} [CommSemiring R] [AddCommMonoid M] [Module R M]
    [Fintype ι] {n : ℕ} (c : Fin (n + 1) → ι → R) (X : (Fin (n + 1) → ι) → M) :
    ∑ q : Fin (n + 1) → ι, (∏ i, c i (q i)) • X q =
      ∑ b : ι, ∑ p : Fin n → ι, (c 0 b * ∏ i, c i.succ (p i)) • X (Fin.cons b p)
```

lemma sum_filter_weight_succ [source]

Peeling the first slot off a sum over the tuples of a prescribed total weight: the first slot takes its own weight `w b` and the remaining slots make up the rest. Each slot draws its letter from the same alphabet `ι`, `w` gives a letter its weight and the weights of the slots add; `f s` is the factor contributed by slot `s`.

```
lemma sum_filter_weight_succ {R ι : Type*} [CommSemiring R] [Fintype ι] {n : ℕ} (w : ι → ℤ)
    (f : Fin (n + 1) → ι → R) (m : ℤ) :
    ∑ q ∈ Finset.univ.filter (fun q : Fin (n + 1) → ι => (∑ s, w (q s)) = m), ∏ s, f s (q s)
      = ∑ b : ι, f 0 b
          * ∑ q ∈ Finset.univ.filter (fun q : Fin n → ι => (∑ s, w (q s)) = m - w b),
              ∏ s, f s.succ (q s)
```

2.327 PhyslibAlpha.Mathematics.HomogeneousGenerators

PhyslibAlpha/Mathematics/HomogeneousGenerators.lean on GitHub.

def homogeneousSubmodule [source]

The degree-`w` part of `A` for `f`: the elements `x ∈ A` with `f x = monomial w x`.

```
noncomputable def homogeneousSubmodule (A : Subalgebra R B) (f : B →ₐ[R] Polynomial B)
    (w : ℕ) : Submodule R B
```

lemma mem_homogeneousSubmodule_iff [source]

```
lemma mem_homogeneousSubmodule_iff {w : ℕ} {x : B} :
    x ∈ A.homogeneousSubmodule f w ↔ x ∈ A ∧ f x = Polynomial.monomial w x
```

lemma one_le_homogeneousSubmodule_zero [source]

The scalars have degree zero.

```
lemma one_le_homogeneousSubmodule_zero : (1 : Submodule R B) ≤ A.homogeneousSubmodule f 0
```

lemma homogeneousSubmodule_mul_le [source]

Degrees add under multiplication.

```
lemma homogeneousSubmodule_mul_le (m n : ℕ) :
    A.homogeneousSubmodule f m * A.homogeneousSubmodule f n
      ≤ A.homogeneousSubmodule f (m + n)
```

def productsOfDegree [source]

The span of the products `D n₁ * ⋯ * D nₖ` whose degrees `deg n₁ + ⋯ + deg nₖ` add up to `w`. Every order of the factors occurs separately, and the empty product is `1`.

```
noncomputable def productsOfDegree (D : ℕ → Submodule R B) (deg : ℕ → ℕ) (w : ℕ) :
    Submodule R B
```

lemma list_prod_le_productsOfDegree [source]

```
lemma list_prod_le_productsOfDegree (l : List ℕ) :
    (l.map D).prod ≤ productsOfDegree D deg (l.map deg).sum
```

lemma productsOfDegree_mul_le [source]

Concatenating products adds their degrees.

```
lemma productsOfDegree_mul_le (a b : ℕ) :
    productsOfDegree D deg a * productsOfDegree D deg b ≤ productsOfDegree D deg (a + b)
```

lemma le_homogeneousSubmodule [source]

A generator in `D n` has degree `deg n`.

```
lemma le_homogeneousSubmodule (n : ℕ) : D n ≤ A.homogeneousSubmodule f (deg n)
```

lemma list_prod_le_homogeneousSubmodule [source]

A product of generators has the sum of their degrees.

```
lemma list_prod_le_homogeneousSubmodule (l : List ℕ) :
    (l.map D).prod ≤ A.homogeneousSubmodule f (l.map deg).sum
```

lemma productsOfDegree_le_homogeneousSubmodule [source]

The products of generators of total degree `w` lie in the degree-`w` part of `A`.

```
lemma productsOfDegree_le_homogeneousSubmodule (w : ℕ) :
    productsOfDegree D deg w ≤ A.homogeneousSubmodule f w
```

lemma coeff_mem_productsOfDegree [source]

Every coefficient of `f x`, for `x ∈ A`, is a sum of products of generators: the one induction over the generation of `A`.

```
lemma coeff_mem_productsOfDegree {x : B} (hx : x ∈ A) (m : ℕ) :
    (f x).coeff m ∈ productsOfDegree D deg m
```

lemma coeff_mem_homogeneousSubmodule [source]

The coefficient of `X ^ m` in `f x`, for `x ∈ A`, lies in the degree-`m` part of `A`.

```
lemma coeff_mem_homogeneousSubmodule {x : B} (hx : x ∈ A) (m : ℕ) :
    (f x).coeff m ∈ A.homogeneousSubmodule f m
```

lemma homogeneousSubmodule_eq_productsOfDegree [source]

The degree-`w` part of `A` is spanned by the products of generators, in every order, whose degrees add up to `w`.

```
lemma homogeneousSubmodule_eq_productsOfDegree (w : ℕ) :
    A.homogeneousSubmodule f w = productsOfDegree D deg w
```

lemma homogeneousSubmodule_eq_iSup_mul [source]

For positive `w`, the degree-`w` part of `A` is spanned by the products of a generator `D k` of degree at most `w` with the part of the remaining degree. The bound `n ≤ deg n` only makes the range of `k` finite.

```
lemma homogeneousSubmodule_eq_iSup_mul (hle : ∀ n, n ≤ deg n) {w : ℕ} (hw : 0 < w) :
    A.homogeneousSubmodule f w
      = ⨆ k ∈ (Finset.range (w + 1)).filter (fun k => deg k ≤ w),
          D k * A.homogeneousSubmodule f (w - deg k)
```

lemma homogeneousSubmodule_zero_eq_one [source]

If every generator has positive degree, degree zero is the scalar submodule.

```
lemma homogeneousSubmodule_zero_eq_one (hpos : ∀ n, 0 < deg n) :
    A.homogeneousSubmodule f 0 = 1
```

lemma homogeneousSubmodule_eq_sup_iSup_mul [source]

If `n < deg n` for every `n`, the degree-`i` part of `A`, for positive `i`, is spanned by the generators of degree `i` and the products of two parts of positive degrees adding up to `i`. It follows from removing the leftmost generator: a generator of degree `i` leaves degree zero, which is the scalars.

```
lemma homogeneousSubmodule_eq_sup_iSup_mul (hlt : ∀ n, n < deg n) (i : ℕ) (hi : 0 < i) :
    A.homogeneousSubmodule f i
      = (⨆ k ∈ Finset.univ.filter (fun k : Fin i => deg k = i), D k)
        ⊔ (⨆ p ∈ Finset.univ.filter (fun p : Fin i × Fin i => (p.1 : ℕ) + (p.2 : ℕ) = i),
            A.homogeneousSubmodule f p.1 * A.homogeneousSubmodule f p.2)
```

lemma homogeneousSubmodule_eq_bot [source]

If a set of degrees contains `0` and every `deg n` and is closed under addition, a degree outside it has zero part. This proves vanishing only: a degree in the set may still have zero part.

```
lemma homogeneousSubmodule_eq_bot (P : ℕ → Prop) (h0 : P 0) (hP : ∀ n, P (deg n))
    (hadd : ∀ a b, P a → P b → P (a + b)) {w : ℕ} (hw : ¬ P w) :
    A.homogeneousSubmodule f w = ⊥
```

2.328 PhyslibAlpha.Mathematics.InvariantReduction

PhyslibAlpha/Mathematics/InvariantReduction.lean on GitHub.

def IsStableUnder [source]

A submodule carried into itself by every map of the family `σ`.

```
def IsStableUnder (σ : G → B →ₗ[R] B) (V : Submodule R B) : Prop
```

def IsFixedBy [source]

A submodule fixed pointwise by every map of the family `σ`.

```
def IsFixedBy (σ : G → B →ₗ[R] B) (V : Submodule R B) : Prop
```

lemma isStableUnder_iff_map [source]

Stability as an inclusion of images.

```
lemma isStableUnder_iff_map {V : Submodule R B} :
    IsStableUnder σ V ↔ ∀ g, Submodule.map (σ g) V ≤ V
```

lemma isStableUnder [source]

A pointwise-fixed submodule is stable.

```
lemma IsFixedBy.isStableUnder {V : Submodule R B} (hV : IsFixedBy σ V) : IsStableUnder σ V
```

lemma IsFixedBy.sup [source]

A join of two pointwise-fixed submodules is pointwise fixed.

```
lemma IsFixedBy.sup {V V' : Submodule R B} (hV : IsFixedBy σ V) (hV' : IsFixedBy σ V') :
    IsFixedBy σ (V ⊔ V')
```

lemma isStableUnder_bot [source]

The zero submodule is stable.

```
lemma isStableUnder_bot : IsStableUnder σ (⊥ : Submodule R B)
```

lemma IsStableUnder.sup [source]

A join of two stable submodules is stable.

```
lemma IsStableUnder.sup {V V' : Submodule R B} (hV : IsStableUnder σ V)
    (hV' : IsStableUnder σ V') : IsStableUnder σ (V ⊔ V')
```

lemma isStableUnder_iSup [source]

An indexed join of stable submodules is stable. The index is a `Sort`, so this covers the bounded join `⨆ i ∈ s, V i`.

```
lemma isStableUnder_iSup {ι : Sort*} {V : ι → Submodule R B}
    (hV : ∀ i, IsStableUnder σ (V i)) : IsStableUnder σ (⨆ i, V i)
```

lemma isFixedBy_span_singleton [source]

The span of a fixed vector is pointwise fixed.

```
lemma isFixedBy_span_singleton {b : B} (hb : ∀ g, σ g b = b) : IsFixedBy σ (R • b)
```

lemma isFixedBy_iSup [source]

An indexed join of pointwise-fixed submodules is pointwise fixed.

```
lemma isFixedBy_iSup {ι : Sort*} {V : ι → Submodule R B} (hV : ∀ i, IsFixedBy σ (V i)) :
    IsFixedBy σ (⨆ i, V i)
```

lemma isFixedBy_span_range [source]

The span of a family of fixed vectors is pointwise fixed.

```
lemma isFixedBy_span_range {ι : Sort*} {T : ι → B} (hT : ∀ i g, σ g (T i) = T i) :
    IsFixedBy σ (Submodule.span R (Set.range T))
```

lemma isStableUnder_span_range [source]

The span of a family is stable when each map sends each member into the span.

```
lemma isStableUnder_span_range {ι : Sort*} {T : ι → B}
    (hT : ∀ g i, σ g (T i) ∈ Submodule.span R (Set.range T)) :
    IsStableUnder σ (Submodule.span R (Set.range T))
```

lemma isStableUnder_span_range_of_sum [source]

The span of a finite family is stable when each map sends each member to a combination of the family.

```
lemma isStableUnder_span_range_of_sum {ι : Type*} [Fintype ι] {T : ι → B}
    (hT : ∀ g i, ∃ c : ι → R, σ g (T i) = ∑ a, c a • T a) :
    IsStableUnder σ (Submodule.span R (Set.range T))
```

lemma mul [source]

A product of two stable submodules of an algebra is stable under maps respecting multiplication.

```
lemma IsStableUnder.mul {R A : Type*} [CommSemiring R] [Semiring A] [Algebra R A]
    {σ : G → A →ₗ[R] A} (hσ : ∀ g (a b : A), σ g (a * b) = σ g a * σ g b)
    {V V' : Submodule R A} (hV : IsStableUnder σ V) (hV' : IsStableUnder σ V') :
    IsStableUnder σ (V * V')
```

def ReducesInvariantsTo [source]

Every invariant of $V \sqcup S$, for S a σ-stable submodule, lies in $W \sqcup S$.

```
def ReducesInvariantsTo (σ : G → B →ₗ[R] B) (V W : Submodule R B) : Prop
```

lemma reducesInvariantsTo_of_le [source]

A submodule reduces to any submodule containing it.

```
lemma reducesInvariantsTo_of_le {V W : Submodule R B} (hVW : V ≤ W) :
    ReducesInvariantsTo σ V W
```

lemma mono_left [source]

A reduction restricts to a smaller source.

```
lemma ReducesInvariantsTo.mono_left {V V' W : Submodule R B} (hP : ReducesInvariantsTo σ V' W)
    (hV : V ≤ V') : ReducesInvariantsTo σ V W
```

lemma mono_right [source]

A reduction extends to a larger target.

```
lemma ReducesInvariantsTo.mono_right {V W W' : Submodule R B}
    (hP : ReducesInvariantsTo σ V W') (hW : W' ≤ W) : ReducesInvariantsTo σ V W
```

lemma trans [source]

Successive reductions compose.

```
lemma ReducesInvariantsTo.trans {V W W' : Submodule R B} (hP : ReducesInvariantsTo σ V W)
    (hQ : ReducesInvariantsTo σ W W') : ReducesInvariantsTo σ V W'
```

lemma ReducesInvariantsTo.sup [source]

Reductions of `V` and of a stable `V'` to a common stable target combine to a reduction of `V ⊔ V'`.

```
lemma ReducesInvariantsTo.sup {V V' W : Submodule R B} (hP : ReducesInvariantsTo σ V W)
    (hQ : ReducesInvariantsTo σ V' W) (hV' : IsStableUnder σ V') (hW : IsStableUnder σ W) :
    ReducesInvariantsTo σ (V ⊔ V') W
```

lemma biSup [source]

Reductions of stable submodules to a common stable target combine over a finite set.

```
lemma ReducesInvariantsTo.biSup {ι : Type*} [DecidableEq ι] {V : ι → Submodule R B}
    {W : Submodule R B} (hP : ∀ i, ReducesInvariantsTo σ (V i) W)
    (hV : ∀ i, IsStableUnder σ (V i)) (hW : IsStableUnder σ W) (s : Finset ι) :
    ReducesInvariantsTo σ (⨆ i ∈ s, V i) W
```

lemma iSup_of_biSup [source]

A reduction of every join over a finite set of indices is a reduction of the join over the whole index type, which may be infinite: an element of the join lies in the join over finitely many of the summands. No stability is assumed of the summands or of the target; the remainder is stable, as in every reduction.

```
lemma ReducesInvariantsTo.iSup_of_biSup {ι : Type*} {V : ι → Submodule R B} {W : Submodule R B}
    (hP : ∀ s : Finset ι, ReducesInvariantsTo σ (⨆ i ∈ s, V i) W) :
    ReducesInvariantsTo σ (⨆ i, V i) W
```

lemma iSup [source]

Reductions of stable submodules to a common stable target combine over any index type, finite or not: `biSup` combines them over every finite set of indices, and `iSup_of_biSup` passes to the whole join.

```
lemma ReducesInvariantsTo.iSup {ι : Type*} {V : ι → Submodule R B} {W : Submodule R B}
    (hP : ∀ i, ReducesInvariantsTo σ (V i) W) (hV : ∀ i, IsStableUnder σ (V i))
    (hW : IsStableUnder σ W) : ReducesInvariantsTo σ (⨆ i, V i) W
```

lemma comp [source]

A reduction for the subfamily `σ ∘ ι` is a reduction for `σ`: an invariant of `σ` is an invariant of the subfamily, and a `σ`-stable submodule is stable under the subfamily.

```
lemma ReducesInvariantsTo.comp {G' : Type*} (ι : G' → G) {V W : Submodule R B}
    (hP : ReducesInvariantsTo (fun g' => σ (ι g')) V W) : ReducesInvariantsTo σ V W
```

lemma reducesInvariantsTo_bot_of_apply_eq_smul [source]

A submodule on which one map of the family acts by a scalar `μ ≠ 1` reduces to `⊥`. An invariant `v + s` of `V ⊔ S` has `(μ - 1) • v = s - σ g s`, which lies in `S`, and so does `v`. Of the stability of `S` only stability under `σ g` is used.

```
lemma reducesInvariantsTo_bot_of_apply_eq_smul (g : G) {μ : K} (hμ : μ ≠ 1)
    {V : Submodule K B} (hV : ∀ v ∈ V, σ g v = μ • v) : ReducesInvariantsTo σ V ⊥
```

lemma mem_sup_of_quotient [source]

An invariant of `V ⊔ S` lies in `W ⊔ S` when every invariant of the image of `V` in `B ⧸ S` lies in the image of `W`.

```
lemma IsStableUnder.mem_sup_of_quotient (hS : IsStableUnder σ S)
    (hclass : ∀ x ∈ V.map S.mkQ, (∀ g, S.mapQ S (σ g) (hS g) x = x) → x ∈ W.map S.mkQ)
    {x : B} (hx : x ∈ V ⊔ S) (hinv : ∀ g, σ g x = x) : x ∈ W ⊔ S
```

lemma reducesInvariantsTo_of_quotient [source]

`V` reduces to `W` when, for every stable `S`, every invariant of the image of `V` in `B ⧸ S` lies in the image of `W`.

```
lemma reducesInvariantsTo_of_quotient
    (hclass : ∀ S : Submodule R B, ∀ hS : IsStableUnder σ S, ∀ x ∈ V.map S.mkQ,
      (∀ g, S.mapQ S (σ g) (hS g) x = x) → x ∈ W.map S.mkQ) :
    ReducesInvariantsTo σ V W
```

lemma exists_add_of_mem_sup [source]

An invariant of `W ⊔ S`, for `W` pointwise fixed, is an element of `W` plus an invariant element of `S`.

```
lemma IsFixedBy.exists_add_of_mem_sup (hW : IsFixedBy σ W) {x : B} (hx : x ∈ W ⊔ S)
    (hinv : ∀ g, σ g x = x) : ∃ w ∈ W, ∃ y ∈ S, x = w + y ∧ ∀ g, σ g y = y
```

lemma mem_sup_and_forall_eq_self_iff [source]

A reduction of `V` to a pointwise-fixed `W ≤ V` describes the invariants of `V ⊔ S`, for `S` stable, exactly: they are the elements of `W` plus an invariant element of `S`. This is the form in which a classification is stated.

```
lemma ReducesInvariantsTo.mem_sup_and_forall_eq_self_iff (hP : ReducesInvariantsTo σ V W)
    (hWV : W ≤ V) (hW : IsFixedBy σ W) (hS : IsStableUnder σ S) (x : B) :
    (x ∈ V ⊔ S ∧ ∀ g, σ g x = x) ↔ ∃ y ∈ S, (∀ g, σ g y = y) ∧ x - y ∈ W
```

lemma exists_add_of_quotient [source]

Quotient to remainder: when every invariant of the image of `V` in `B ⧸ S` lies in the image of a pointwise-fixed `W`, an invariant of `V ⊔ S` is an element of `W` plus an invariant element of `S`.

```
lemma IsStableUnder.exists_add_of_quotient (hS : IsStableUnder σ S) (hW : IsFixedBy σ W)
    (hclass : ∀ x ∈ V.map S.mkQ, (∀ g, S.mapQ S (σ g) (hS g) x = x) → x ∈ W.map S.mkQ)
    {x : B} (hx : x ∈ V ⊔ S) (hinv : ∀ g, σ g x = x) :
    ∃ w ∈ W, ∃ y ∈ S, x = w + y ∧ ∀ g, σ g y = y
```

lemma exists_smul_add_of_quotient [source]

Quotient to remainder for a single fixed vector `v`: when every invariant of the image of `V` in `B ⧸ S` is a multiple of the class of `v`, an invariant of `V ⊔ S` is a multiple of `v` plus an invariant element of `S`.

```
lemma IsStableUnder.exists_smul_add_of_quotient (hS : IsStableUnder σ S) {v : B}
    (hv : ∀ g, σ g v = v)
    (hclass : ∀ x ∈ V.map S.mkQ, (∀ g, S.mapQ S (σ g) (hS g) x = x) → ∃ c : R, x = c • S.mkQ v)
    {x : B} (hx : x ∈ V ⊔ S) (hinv : ∀ g, σ g x = x) :
    ∃ c : R, ∃ y ∈ S, x = c • v + y ∧ ∀ g, σ g y = y
```

structure InvariantReductionToSpan [source]

A stable submodule `V` together with a fixed vector such that, for every stable `S`, every invariant of `V ⊔ S` is a multiple of that vector plus an element of `S`. The vector need not be nonzero or lie in `V`.

```
structure InvariantReductionToSpan (σ : G → B →ₗ[R] B) (V : Submodule R B) where
  spanningVector : B
  stable : IsStableUnder σ V
  spanningVector_fixed : ∀ g, σ g spanningVector = spanningVector
  reduce : ∀ S : Submodule R B, IsStableUnder σ S → ∀ x ∈ V ⊔ S, (∀ g, σ g x = x) →
    ∃ c : R, ∃ y ∈ S, x = c • spanningVector + y
```

lemma reducesInvariantsTo [source]

The submodule reduces to the span of the spanning vector.

```
lemma reducesInvariantsTo (r : InvariantReductionToSpan σ V) :
    ReducesInvariantsTo σ V (R • r.spanningVector)
```

lemma reducesInvariantsTo_iSup [source]

A family of reductions to spans, over any index type, reduces the join of the submodules to the span of the spanning vectors.

```
lemma reducesInvariantsTo_iSup {κ : Type*} {V : κ → Submodule R B}
    (r : ∀ k, InvariantReductionToSpan σ (V k)) :
    ReducesInvariantsTo σ (⨆ k, V k)
      (Submodule.span R (Set.range fun k => (r k).spanningVector))
```

def ofReducesInvariantsTo [source]

The reduction of a stable submodule to the span of a fixed vector.

```
def ofReducesInvariantsTo (hV : IsStableUnder σ V) (v : B) (hv : ∀ g, σ g v = v)
    (h : ReducesInvariantsTo σ V (R • v)) : InvariantReductionToSpan σ V where
```

def ofFixed [source]

The span of a fixed vector reduces to itself.

```
def ofFixed (b : B) (hb : ∀ g, σ g b = b) :
    InvariantReductionToSpan σ (R • b) where
```

def ofFixedFamily [source]

The span of a nonempty family whose members all equal one fixed vector reduces to the span of that vector.

```
def ofFixedFamily {ι : Type*} [Nonempty ι] {T : ι → B} (b : B)
    (hTb : ∀ i, T i = b) (hb : ∀ g, σ g b = b) :
    InvariantReductionToSpan σ (Submodule.span R (Set.range T))
```

lemma mapQ_mkQ_eq_sum_smul [source]

A map moving each member of a family by a matrix moves their classes modulo a submodule that it preserves by the same matrix.

```
lemma Submodule.mapQ_mkQ_eq_sum_smul {R B ι : Type*} [Ring R] [AddCommGroup B] [Module R B]
    [Fintype ι] (S : Submodule R B) {f : B →ₗ[R] B} (hS : ∀ y ∈ S, f y ∈ S) {T : ι → B}
    {M : Matrix ι ι R} {l : ι} (hf : f (T l) = ∑ a, M a l • T a) :
    S.mapQ S f hS (S.mkQ (T l)) = ∑ a, M a l • S.mkQ (T a)
```

lemma reducesInvariantsTo_of_mulVec_eq [source]

A family moved by matrices whose conjugate transposes are among them reduces to the combinations of any submodule `E` of coefficient vectors containing every vector fixed by all the matrices.

```
lemma reducesInvariantsTo_of_mulVec_eq (T : ι → B) (M : G → Matrix ι ι ℂ)
    (hT : ∀ g l, σ g (T l) = ∑ a, M g a l • T a) (hM : ∀ g, ∃ g', M g' = (M g)ᴴ)
    (E : Submodule ℂ (ι → ℂ)) (hE : ∀ c, (∀ g, M g *ᵥ c = c) → c ∈ E) :
    ReducesInvariantsTo σ (Submodule.span ℂ (Set.range T))
      (E.map (Fintype.linearCombination ℂ T))
```

lemma reducesInvariantsTo_bot_of_mulVec_eq [source]

The form of `reducesInvariantsTo_of_mulVec_eq` when no nonzero coefficient vector is fixed: the span reduces to ⊥.

```
lemma reducesInvariantsTo_bot_of_mulVec_eq (T : ι → B) (M : G → Matrix ι ι ℂ)
    (hT : ∀ g l, σ g (T l) = ∑ a, M g a l • T a) (hM : ∀ g, ∃ g', M g' = (M g)ᴴ)
    (hE : ∀ c, (∀ g, M g *ᵥ c = c) → c = 0) :
    ReducesInvariantsTo σ (Submodule.span ℂ (Set.range T)) ⊥
```

lemma reducesInvariantsTo_span_singleton_of_mulVec_eq [source]

The form of `reducesInvariantsTo_of_mulVec_eq` when every fixed coefficient vector is a multiple of `e`: the span reduces to the span of the combination with coefficients `e`.

```
lemma reducesInvariantsTo_span_singleton_of_mulVec_eq (T : ι → B) (M : G → Matrix ι ι ℂ)
    (hT : ∀ g l, σ g (T l) = ∑ a, M g a l • T a) (hM : ∀ g, ∃ g', M g' = (M g)ᴴ)
    (e : ι → ℂ) (hE : ∀ c, (∀ g, M g *ᵥ c = c) → ∃ z : ℂ, c = z • e) :
    ReducesInvariantsTo σ (Submodule.span ℂ (Set.range T)) (ℂ • ∑ i, e i • T i)
```

2.329 PhyslibAlpha.Mathematics.LeviCivita.Basic

PhyslibAlpha/Mathematics/LeviCivita/Basic.lean on GitHub.

lemma cast_leviCivitaSymbol [source]

The symbol as a determinant over any commutative ring, the integer determinant carried along the ring map from $\mathbb{Z}$.

```
lemma cast_leviCivitaSymbol (g : ι → ι) :
    ((leviCivitaSymbol g : ℤ) : R)
      = (Matrix.of fun i j : ι => if g i = j then (1 : R) else 0).det
```

lemma det_eq_sum_perm_prod [source]

The Leibniz formula with the permutation moving the column index.

```
private lemma det_eq_sum_perm_prod (X : Matrix ι ι R) :
    X.det = ∑ σ : Equiv.Perm ι, ((Equiv.Perm.sign σ : ℤ) : R) * ∏ i, X i (σ i)
```

lemma sum_leviCivitaSymbol_mul_prod [source]

Contracted against the rows of `M` selected by `a`, the symbol gives `det M` times the symbol of `a`.

```
lemma sum_leviCivitaSymbol_mul_prod (M : Matrix ι ι R) (a : ι → ι) :
    ∑ g : ι → ι, ((leviCivitaSymbol g : ℤ) : R) * ∏ i, M (a i) (g i)
      = M.det * ((leviCivitaSymbol a : ℤ) : R)
```

2.330 PhyslibAlpha.Mathematics.LieAlgebraUnit

PhyslibAlpha/Mathematics/LieAlgebraUnit.lean on GitHub.

instance Bracket Unit Unit [source]

```
instance : Bracket Unit Unit
```

instance instLieRingUnit [source]

```
instance instLieRingUnit : LieRing Unit where
```

instance instLieAlgebraUnit [source]

```
instance instLieAlgebraUnit {R : Type*} [CommRing R] : LieAlgebra R Unit where
```

2.331 PhyslibAlpha.Mathematics.LinearCombination

PhyslibAlpha/Mathematics/LinearCombination.lean on GitHub.

lemma map_span_range [source]

The image of the span of a family is the span of the images.

```
lemma Submodule.map_span_range {R B B' : Type*} [Semiring R] [AddCommMonoid B] [Module R B]
    [AddCommMonoid B'] [Module R B'] {ι : Sort*} (f : B →ₗ[R] B') (T : ι → B) :
    (Submodule.span R (Set.range T)).map f = Submodule.span R (Set.range fun i => f (T i))
```

lemma sum_sum_mul_smul [source]

Contracting the components against coefficients moved by a matrix is the original combination of the matrix-moved components.

```
lemma Fintype.sum_sum_mul_smul (M : ι → κ → R) (c : κ → R) (T : ι → B) :
    ∑ a, (∑ d, c d * M a d) • T a = ∑ d, c d • ∑ a, M a d • T a
```

lemma map_sum_smul_of_forall_eq [source]

A linear map that moves each component of a family by a matrix moves a combination of the components by that matrix acting on the coefficients, with the free index first.

```
lemma LinearMap.map_sum_smul_of_forall_eq (φ : B →ₗ[R] B') (T : ι → B) (T' : κ → B')
    (M : κ → ι → R) (hT : ∀ l, φ (T l) = ∑ a, M a l • T' a) (c : ι → R) :
    φ (∑ l, c l • T l) = ∑ a, (∑ l, c l * M a l) • T' a
```

lemma map_sum_smul_eq_sum_mulVec_smul [source]

The matrix form of `LinearMap.map_sum_smul_of_forall_eq`: a linear map moving `T l` to `∑ a, M a l • T' a` moves the combination with coefficients `c` to the combination with coefficients `M *ᵥ c`.

```
lemma LinearMap.map_sum_smul_eq_sum_mulVec_smul (φ : B →ₗ[R] B') (T : ι → B) (T' : κ → B')
    (M : Matrix κ ι R) (hT : ∀ l, φ (T l) = ∑ a, M a l • T' a) (c : ι → R) :
    φ (∑ l, c l • T l) = ∑ a, (M *ᵥ c) a • T' a
```

lemma exists_invariant_coeff_of_adjoint_mem [source]

A vector of the span of `T` fixed by every `φ g` is the combination of coefficients fixed by every `A g`, where `φ g` moves combinations by moving their coefficients with `A g`. The one condition on `A` is that for every `g` some `g'` acts as the adjoint of `g` for the standard inner product on coefficients; neither the `φ g` nor the `A g` need form a representation, and `B` carries no inner product. The components may be dependent, so the coefficients need not be unique: the proof takes the part of any coefficients orthogonal to those contracting to `0`.

```
lemma Fintype.exists_invariant_coeff_of_adjoint_mem {ι G B : Type*} [Fintype ι]
    [AddCommGroup B] [Module ℂ B] (T : ι → B) (φ : G → B →ₗ[ℂ] B)
    (A : G → (ι → ℂ) →ₗ[ℂ] (ι → ℂ))
    (hφ : ∀ (g : G) (c : ι → ℂ), φ g (∑ i, c i • T i) = ∑ i, A g c i • T i)
    (hA : ∀ g : G, ∃ g' : G, ∀ u v : EuclideanSpace ℂ ι,
      ⟪u, WithLp.toLp 2 (A g v.ofLp)⟫_ℂ = ⟪WithLp.toLp 2 (A g' u.ofLp), v⟫_ℂ)
    {x : B} (hx : x ∈ Submodule.span ℂ (Set.range T)) (hinv : ∀ g, φ g x = x) :
    ∃ c : ι → ℂ, x = ∑ i, c i • T i ∧ ∀ g, A g c = c
```

lemma exists_mulVec_eq_of_conjTranspose_mem [source]

The matrix form of `Fintype.exists_invariant_coeff_of_adjoint_mem`: when each `φ g` moves the family by a matrix `M g`, and the conjugate transpose of each `M g` is some `M g'`, a vector of the span fixed by every `φ g` is the combination of a coefficient vector fixed by every `M g`.

```
lemma Fintype.exists_mulVec_eq_of_conjTranspose_mem {ι G B : Type*} [Fintype ι]
    [AddCommGroup B] [Module ℂ B] (T : ι → B) (φ : G → B →ₗ[ℂ] B) (M : G → Matrix ι ι ℂ)
```

```
    (hT : ∀ g l, φ g (T l) = ∑ a, M g a l • T a) (hM : ∀ g, ∃ g', M g' = (M g)ᴴ)
    {x : B} (hx : x ∈ Submodule.span ℂ (Set.range T)) (hinv : ∀ g, φ g x = x) :
    ∃ c : ι → ℂ, x = ∑ i, c i • T i ∧ ∀ g, M g *ᵥ c = c
```

lemma range_eq_span_range_basis [source]

The range of a linear map is the span of the images of a basis.

```
lemma LinearMap.range_eq_span_range_basis {ι R M N : Type*} [Semiring R] [AddCommMonoid M]
    [Module R M] [AddCommMonoid N] [Module R N] (b : Module.Basis ι R M) (f : M →ₗ[R] N) :
    LinearMap.range f = Submodule.span R (Set.range fun i => f (b i))
```

lemma mul_le_of_le_span_range [source]

A product of two submodules, each inside the span of a family, lies in any submodule containing the products of the members of the two families.

```
lemma Submodule.mul_le_of_le_span_range {ι κ R A : Type*} [CommSemiring R] [Semiring A]
    [Algebra R A] {V V' X : Submodule R A} {a : ι → A} {b : κ → A}
    (hV : V ≤ Submodule.span R (Set.range a)) (hV' : V' ≤ Submodule.span R (Set.range b))
    (hX : ∀ i j, a i * b j ∈ X) : V * V' ≤ X
```

lemma mul_mul_le_of_le_span_range [source]

The three-factor form of `Submodule.mul_le_of_le_span_range`.

```
lemma Submodule.mul_mul_le_of_le_span_range {ι κ ν R A : Type*} [CommSemiring R] [Semiring A]
    [Algebra R A] {V V' V'' X : Submodule R A} {a : ι → A} {b : κ → A} {c : ν → A}
    (hV : V ≤ Submodule.span R (Set.range a)) (hV' : V' ≤ Submodule.span R (Set.range b))
    (hV'' : V'' ≤ Submodule.span R (Set.range c)) (hX : ∀ i j k, a i * (b j * c k) ∈ X) :
    V * (V' * V'') ≤ X
```

2.332 PhyslibAlpha.Mathematics.MeasureTheory.Pi

PhyslibAlpha/Mathematics/MeasureTheory/Pi.lean on GitHub.

lemma pi_mono [source]

The finite product measure `Measure.pi μ` is monotone in the family of measures `μ`: if `μ i ≤ ν i` for every `i`, then `Measure.pi μ ≤ Measure.pi ν`.

```
lemma pi_mono {ι : Type*} [Fintype ι] {α : ι → Type*} [∀ i, MeasurableSpace (α i)]
    {μ ν : (i : ι) → Measure (α i)} (h : ∀ i, μ i ≤ ν i) : Measure.pi μ ≤ Measure.pi ν
```

2.333 PhyslibAlpha.Mathematics.Modules.ConjModule

PhyslibAlpha/Mathematics/Modules/ConjModule.lean on GitHub.

instance Ring (ConjModule A) [source]

```
instance : Ring (ConjModule A)
```

instance instAlgebra [source]

The conjugate module of a `k`-algebra is a `k`-algebra: the same ring, with scalars acting through `star`.

```
instance instAlgebra [Algebra k A] : Algebra k (ConjModule A)
```

def conjDualEquiv [source]

The canonical conjugate-linear equivalence between the dual of a module `M` and the dual of its conjugate.

```
def conjDualEquiv : Module.Dual k M ≃ₛₗ[starRingEnd k] Module.Dual k (ConjModule M) where
```

def conj [source]

The conjugate of a representation `ρ` of `G` on `M`: the same maps `ρ g`, acting on `ConjModule M` through `conjEquiv`.

```
def _root_.Representation.conj {G} [Group G] (ρ : Representation k G M) :
    Representation k G (ConjModule M) where
```

lemma conj_apply [source]

```
lemma _root_.Representation.conj_apply {G} [Group G] (ρ : Representation k G M) (g : G)
    (m : ConjModule M) :
    ρ.conj g m = conjEquiv (k := k) (M := M) (ρ g ((conjEquiv (k := k) (M := M)).symm m))
```

lemma conj_trivial_dual_apply [source]

The dual of the conjugate of the trivial representation acts trivially.

```
lemma _root_.Representation.conj_trivial_dual_apply {G : Type*} [Group G] (g : G)
    (φ : Module.Dual k (ConjModule M)) :
    ((Representation.trivial k G M).conj).dual g φ = φ
```

def map [source]

Functoriality of conjugation: a `k`-linear map induces a `k`-linear map of the conjugate modules, given by the same underlying function.

```
def map (f : M →ₗ[k] N) : ConjModule M →ₗ[k] ConjModule N where
```

lemma map_apply [source]

```
@[simp]
lemma map_apply (f : M →ₗ[k] N) (x : ConjModule M) : map f x = f x
```

def prodEquiv [source]

***Conjugation commutes with finite products.** The conjugate of a product is the product of the conjugates, by the identity underlying function: the twisted scalar action is applied componentwise.*

```
def prodEquiv : ConjModule (M × N) ≃ₗ[k] ConjModule M × ConjModule N where
```

lemma prodEquiv_apply [source]

```
@[simp]
lemma prodEquiv_apply (x : ConjModule (M × N)) :
    prodEquiv (k := k) x = (map (LinearMap.fst k M N) x, map (LinearMap.snd k M N) x)
```

def piEquiv [source]

***Conjugation commutes with arbitrary products.** The conjugate of a product of a family of modules is the product of their conjugates, by the identity underlying function: the twisted scalar action is applied componentwise, so no finiteness of the index type is needed.*

```
def piEquiv : ConjModule (∀ i, P i) ≃ₗ[k] ∀ i, ConjModule (P i) where
```

lemma piEquiv_apply [source]

```
@[simp]
lemma piEquiv_apply (x : ConjModule (∀ i, P i)) (i : ι) :
    (piEquiv P : ConjModule (∀ i, P i) ≃ₗ[k] ∀ i, ConjModule (P i)) x i
      = map (k := k) (LinearMap.proj i) x
```

lemma piEquiv_symm_apply [source]

```
lemma piEquiv_symm_apply (x : ∀ i, ConjModule (P i)) (i : ι) :
    map (k := k) (LinearMap.proj i)
        ((piEquiv P : ConjModule (∀ i, P i) ≃ₗ[k] ∀ i, ConjModule (P i)).symm x) = x i
```

instance instFinite [source]

The conjugate module of a finite free module is finite: the conjugated basis `Basis.conj` is indexed by the same type.

```
instance instFinite [Module.Free k M] [Module.Finite k M] :
    Module.Finite k (ConjModule M)
```

def tensorHom [source]

The canonical `k`-linear map `ConjModule M ⊗ ConjModule N → ConjModule (M ⊗ N)`, the identity on pure tensors.

```
noncomputable def tensorHom : ConjModule M ⊗[k] ConjModule N →ₗ[k] ConjModule (M ⊗[k] N)
```

lemma tensorHom_tmul [source]

```
@[simp]
lemma tensorHom_tmul (m : ConjModule M) (n : ConjModule N) :
    tensorHom (k := k) (m ⊗ₜ[k] n)
      = conjEquiv (k := k) (M := M ⊗[k] N)
        ((conjEquiv (k := k) (M := M)).symm m ⊗ₜ[k] (conjEquiv (k := k) (M := N)).symm n)
```

def tensorInv [source]

The inverse map `ConjModule (M ⊗ N) → ConjModule M ⊗ ConjModule N`, again the identity on pure tensors. A `k`-linear map out of `ConjModule X` is the same data as a `k`-linear map into `ConjModule` of the target, which is what `map` and `involution` package here.

```
noncomputable def tensorInv : ConjModule (M ⊗[k] N) →ₗ[k] ConjModule M ⊗[k] ConjModule N
```

def tensorEquiv [source]

***Conjugation is monoidal.** `ConjModule M ⊗ ConjModule N ≃ₗ[k] ConjModule (M ⊗ N)`, the identity on pure tensors. Injectivity comes from `tensorInv` being a left inverse; surjectivity from every element of `M ⊗ N` being a sum of pure tensors.*

```
noncomputable def tensorEquiv :
    ConjModule M ⊗[k] ConjModule N ≃ₗ[k] ConjModule (M ⊗[k] N)
```

lemma tensorEquiv_tmul [source]

```
@[simp]
lemma tensorEquiv_tmul (m : ConjModule M) (n : ConjModule N) :
    tensorEquiv (k := k) (m ⊗ₜ[k] n)
      = conjEquiv (k := k) (M := M ⊗[k] N)
        ((conjEquiv (k := k) (M := M)).symm m ⊗ₜ[k] (conjEquiv (k := k) (M := N)).symm n)
```

lemma tensorEquiv_symm_conjEquiv_tmul [source]

```
@[simp]
lemma tensorEquiv_symm_conjEquiv_tmul (m : M) (n : N) :
    (tensorEquiv (k := k) (M := M) (N := N)).symm
        (conjEquiv (k := k) (M := M ⊗[k] N) (m ⊗ₜ[k] n))
      = conjEquiv (k := k) (M := M) m ⊗ₜ[k] conjEquiv (k := k) (M := N) n
```

def endConj [source]

A linear endomorphism read on the conjugate module: the same underlying map, through the identity `conjEquiv`. Conjugating twists nothing at the level of the additive group, so all structural identities (`comp`, `add`, `neg`, sums) hold definitionally.

```
def endConj {k : Type*} [CommRing k] [StarRing k] {M : Type*}
    [AddCommGroup M] [Module k M] (f : M →ₗ[k] M) :
    ConjModule M →ₗ[k] ConjModule M where
```

lemma endConj_apply [source]

```
@[simp]
lemma endConj_apply {k : Type*} [CommRing k] [StarRing k] {M : Type*}
    [AddCommGroup M] [Module k M] (f : M →ₗ[k] M) (v : ConjModule M) :
    ConjModule.endConj f v =
      conjEquiv (k := k) (M := M) (f ((conjEquiv (k := k) (M := M)).symm v))
```

lemma endConj_id [source]

```
lemma endConj_id {k : Type*} [CommRing k] [StarRing k] {M : Type*}
    [AddCommGroup M] [Module k M] :
    ConjModule.endConj (LinearMap.id : M →ₗ[k] M) = LinearMap.id
```

lemma endConj_comp [source]

```
lemma endConj_comp {k : Type*} [CommRing k] [StarRing k] {M : Type*}
    [AddCommGroup M] [Module k M] (f g : M →ₗ[k] M) :
    ConjModule.endConj (f ∘ₗ g) = ConjModule.endConj f ∘ₗ ConjModule.endConj g
```

lemma endConj_add [source]

```
lemma endConj_add {k : Type*} [CommRing k] [StarRing k] {M : Type*}
    [AddCommGroup M] [Module k M] (f g : M →ₗ[k] M) :
    ConjModule.endConj (f + g) = ConjModule.endConj f + ConjModule.endConj g
```

lemma endConj_neg [source]

```
lemma endConj_neg {k : Type*} [CommRing k] [StarRing k] {M : Type*}
    [AddCommGroup M] [Module k M] (f : M →ₗ[k] M) :
    ConjModule.endConj (-f) = -ConjModule.endConj f
```

lemma endConj_multiset_sum [source]

```
lemma endConj_multiset_sum {k : Type*} [CommRing k] [StarRing k]
    {M : Type*} [AddCommGroup M] [Module k M] (S : Multiset (M →ₗ[k] M)) :
    ConjModule.endConj S.sum = (S.map ConjModule.endConj).sum
```

lemma endConj_real_smul [source]

Conjugation of endomorphisms commutes with real scalars: the star on the conjugated complex scalar is invisible on the reals.

```
lemma endConj_real_smul {M : Type*} [AddCommGroup M] [Module ℂ M]
    (r : ℝ) (f : M →ₗ[ℂ] M) :
    ConjModule.endConj (r • f) = r • ConjModule.endConj f
```

2.334 PhyslibAlpha.Mathematics.MultisetAntidiagonal

PhyslibAlpha/Mathematics/MultisetAntidiagonal.lean on GitHub.

lemma sum_antidiagonal_assoc [source]

Coassociativity of antidiagonal sums: summing over `s = u + v` and then `u = x + y` is summing over `s = x + t` and then `t = y + v`.

```
lemma sum_antidiagonal_assoc {ι M : Type*} [AddCommMonoid M]
    (s : Multiset ι) (h : Multiset ι → Multiset ι → Multiset ι → M) :
    (s.antidiagonal.map fun p =>
      (p.1.antidiagonal.map fun q => h q.1 q.2 p.2).sum).sum =
    (s.antidiagonal.map fun p =>
      (p.2.antidiagonal.map fun q => h p.1 q.1 q.2).sum).sum
```

lemma sum_antidiagonal_exchange [source]

The exchange law of doubly-split antidiagonal sums: splitting `s = u + v` and then `u = x + y`, `v = z + w` is, with the middle parts exchanged, splitting `s = u' + v'` and then `u' = x + z`, `v' = y + w`.

```
lemma sum_antidiagonal_exchange {ι M : Type*} [AddCommMonoid M]
    (s : Multiset ι) (h : Multiset ι → Multiset ι → Multiset ι → Multiset ι → M) :
    (s.antidiagonal.map fun p =>
      (p.1.antidiagonal.map fun q =>
        (p.2.antidiagonal.map fun r => h q.1 q.2 r.1 r.2).sum).sum).sum =
    (s.antidiagonal.map fun p =>
      (p.1.antidiagonal.map fun q =>
        (p.2.antidiagonal.map fun r => h q.1 r.1 q.2 r.2).sum).sum).sum
```

lemma sum_linearMap_apply [source]

A multiset sum of linear maps, applied: the sum of the applications.

```
lemma sum_linearMap_apply {R M N : Type*} [Semiring R] [AddCommMonoid M]
    [AddCommMonoid N] [Module R M] [Module R N] (S : Multiset (M →ₗ[R] N)) (x : M) :
    S.sum x = (S.map fun f => f x).sum
```

lemma tmul_sum [source]

A pure tensor against a multiset sum distributes over the sum.

```
lemma tmul_sum {R M N : Type*} [CommSemiring R] [AddCommMonoid M]
    [AddCommMonoid N] [Module R M] [Module R N] (m : M) (S : Multiset N) :
    m ⊗ₜ[R] S.sum = (S.map fun n => m ⊗ₜ[R] n).sum
```

lemma sum_antidiagonal_swap [source]

Antidiagonal sums are symmetric under swapping the two parts.

```
lemma sum_antidiagonal_swap {ι M : Type*} [AddCommMonoid M]
    (s : Multiset ι) (h : Multiset ι → Multiset ι → M) :
    (s.antidiagonal.map fun p => h p.1 p.2).sum =
    (s.antidiagonal.map fun p => h p.2 p.1).sum
```

lemma sum_map_neg'' [source]

A multiset sum of negations is the negation of the sum.

```
lemma sum_map_neg'' {ι M : Type*} [AddCommGroup M]
    (s : Multiset ι) (f : ι → M) :
    (s.map fun i => -f i).sum = -(s.map f).sum
```

lemma sum_map_finsetSum [source]

The exchange of a finite sum with a multiset sum.

```
lemma sum_map_finsetSum {α β M : Type*} [AddCommMonoid M]
    (m : Multiset α) (t : Finset β) (f : β → α → M) :
    (m.map fun a => ∑ b ∈ t, f b a).sum = ∑ b ∈ t, (m.map (f b)).sum
```

lemma fst_le_of_mem_antidiagonal [source]

The first part of a splitting of `s` is a sub-multiset of `s`.

```
lemma fst_le_of_mem_antidiagonal {ι : Type*} {s : Multiset ι} {p : Multiset ι × Multiset ι}
    (hp : p ∈ s.antidiagonal) : p.1 ≤ s
```

lemma snd_le_of_mem_antidiagonal [source]

The second part of a splitting of `s` is a sub-multiset of `s`.

```
lemma snd_le_of_mem_antidiagonal {ι : Type*} {s : Multiset ι} {p : Multiset ι × Multiset ι}
    (hp : p ∈ s.antidiagonal) : p.2 ≤ s
```

lemma sum_antidiagonal_eq_of_fst_ne_zero [source]

A sum over the antidiagonal of a family vanishing off `p.1 = 0` collapses to the single term at `(0, s)`.

```
lemma sum_antidiagonal_eq_of_fst_ne_zero {ι M : Type*} [AddCommMonoid M]
    (s : Multiset ι) (F : Multiset ι × Multiset ι → M)
    (hF : ∀ p ∈ s.antidiagonal, p.1 ≠ 0 → F p = 0) :
    (s.antidiagonal.map F).sum = F (0, s)
```

lemma sum_antidiagonal_eq_of_snd_ne_zero [source]

A sum over the antidiagonal of a family vanishing off `p.2 = 0` collapses to the single term at `(s, 0)`.

```
lemma sum_antidiagonal_eq_of_snd_ne_zero {ι M : Type*} [AddCommMonoid M]
    (s : Multiset ι) (F : Multiset ι × Multiset ι → M)
    (hF : ∀ p ∈ s.antidiagonal, p.2 ≠ 0 → F p = 0) :
    (s.antidiagonal.map F).sum = F (s, 0)
```

lemma sum_antidiagonal_middle_exchange [source]

The exchange of the second and third slot in a nested antidiagonal sum.

```
lemma sum_antidiagonal_middle_exchange {ι M : Type*} [AddCommMonoid M]
    (s : Multiset ι) (h : Multiset ι → Multiset ι → Multiset ι → M) :
    (s.antidiagonal.map fun p =>
        (p.1.antidiagonal.map fun q => h q.1 q.2 p.2).sum).sum
      = (s.antidiagonal.map fun p =>
        (p.1.antidiagonal.map fun q => h q.1 p.2 q.2).sum).sum
```

2.335 PhyslibAlpha.Mathematics.MvPolynomialTranslation

PhyslibAlpha/Mathematics/MvPolynomialTranslation.lean on GitHub.

lemma eval_aeval [source]

Evaluation of a substitution of a multivariate polynomial: substitution followed by evaluation is evaluation at the evaluated substituents.

```
lemma eval_aeval (x : I → R) (g : I → MvPolynomial I R) (p : MvPolynomial I R) :
    eval x (aeval g p) = eval (fun i => eval x (g i)) p
```

lemma aeval_update_zero_eq_of_forall_aeval_add_eq [source]

A polynomial invariant under all translations of a fixed variable is unchanged by setting that variable to zero.

```
lemma aeval_update_zero_eq_of_forall_aeval_add_eq (Q : MvPolynomial I R) (j : I)
    (hQ : ∀ r : R, aeval (fun i => X i + C (if i = j then r else 0)) Q = Q) :
    aeval (fun i => if i = j then 0 else X i) Q = Q
```

lemma notMem_vars_of_forall_aeval_add_eq [source]

A polynomial invariant under all translations of a fixed variable does not involve that variable.

```
lemma notMem_vars_of_forall_aeval_add_eq (Q : MvPolynomial I R) (j : I)
    (hQ : ∀ r : R, aeval (fun i => X i + C (if i = j then r else 0)) Q = Q) :
    j ∉ Q.vars
```

lemma mem_adjoin_range_X_sub_X_of_forall_aeval_add_eq [source]

A multivariate polynomial over an infinite integral domain that is invariant under simultaneously translating, for every fiber of an idempotent map π on the variables, all the variables in that fiber by a common constant, is a polynomial in the differences `X i - X (π i)`.

```
lemma mem_adjoin_range_X_sub_X_of_forall_aeval_add_eq (π : I → I)
    (hπ : ∀ i, π (π i) = π i) (P : MvPolynomial I R)
    (hP : ∀ (i₀ : I) (r : R),
      aeval (fun i => X i + C (if π i = π i₀ then r else 0)) P = P) :
    P ∈ Algebra.adjoin R (Set.range fun i => (X i - X (π i) : MvPolynomial I R))
```

2.336 PhyslibAlpha.Mathematics.PolynomialEval

PhyslibAlpha/Mathematics/PolynomialEval.lean on GitHub.

lemma eq_zero_of_forall_eval_algebraMap_eq_zero [source]

A polynomial with coefficients in an algebra over an infinite field vanishes as soon as it vanishes at the image of every scalar. Linear functionals separate the coefficients, and over an infinite field a polynomial is determined by its values.

```
lemma eq_zero_of_forall_eval_algebraMap_eq_zero {k A : Type*} [Field k] [Infinite k]
    [Ring A] [Algebra k A] {p : Polynomial A}
    (h : ∀ c : k, p.eval (algebraMap k A c) = 0) : p = 0
```

lemma ext_of_forall_eval_algebraMap [source]

Two polynomials with coefficients in an algebra over an infinite field are equal as soon as they agree at the image of every scalar.

```
lemma ext_of_forall_eval_algebraMap {k A : Type*} [Field k] [Infinite k]
    [Ring A] [Algebra k A] {p q : Polynomial A}
    (h : ∀ c : k, p.eval (algebraMap k A c) = q.eval (algebraMap k A c)) : p = q
```

def mapCoeffs [source]

The polynomial obtained by applying a function to every coefficient. Unlike `Polynomial.map` this needs no multiplicativity, so it applies to derivations.

The argument is a bare function rather than a linear map: on an algebra built as a tensor product the module structure coming from the algebra and the one coming from the tensor product are equal but not syntactically so, and bundling would force the caller to reconcile them. The properties needed are taken as hypotheses instead.

```
noncomputable def mapCoeffs {A : Type*} [Semiring A] (f : A → A) (p : Polynomial A) :
    Polynomial A
```

lemma coeff_mapCoeffs [source]

```
lemma coeff_mapCoeffs {A : Type*} [Semiring A] {f : A → A} (hf0 : f 0 = 0)
    (p : Polynomial A) (n : ℕ) : (mapCoeffs f p).coeff n = f (p.coeff n)
```

lemma mapCoeffs_zero [source]

```
@[simp]
lemma mapCoeffs_zero {A : Type*} [Semiring A] (f : A → A) : mapCoeffs f 0 = 0
```

lemma mapCoeffs_monomial [source]

```
lemma mapCoeffs_monomial {A : Type*} [Semiring A] {f : A → A} (hf0 : f 0 = 0)
    (n : ℕ) (a : A) :
    mapCoeffs f (Polynomial.monomial n a) = Polynomial.monomial n (f a)
```

lemma mapCoeffs_add [source]

```
lemma mapCoeffs_add {A : Type*} [Semiring A] {f : A → A} (hf0 : f 0 = 0)
    (hadd : ∀ a b : A, f (a + b) = f a + f b) (p q : Polynomial A) :
    mapCoeffs f (p + q) = mapCoeffs f p + mapCoeffs f q
```

lemma eval_algebraMap_mapAlgHom [source]

Evaluation at a scalar commutes with pushing a polynomial along an algebra map: an algebra map fixes the scalars.

```
lemma eval_algebraMap_mapAlgHom {k A B : Type*} [CommSemiring k] [Semiring A] [Semiring B]
    [Algebra k A] [Algebra k B] (f : A →ₐ[k] B) (p : Polynomial A) (c : k) :
    (Polynomial.mapAlgHom f p).eval (algebraMap k B c) = f (p.eval (algebraMap k A c))
```

lemma eval_algebraMap_mapCoeffs [source]

Evaluation at a scalar commutes with applying a linear map to the coefficients: the powers of the scalar are central, so they pass through the linear map.

```
lemma eval_algebraMap_mapCoeffs {k A : Type*} [Field k] [Ring A] [Algebra k A]
    (f : A →ₗ[k] A) (p : Polynomial A) (c : k) :
    (mapCoeffs f p).eval (algebraMap k A c) = f (p.eval (algebraMap k A c))
```

lemma eval_one_mapAlgHom [source]

Evaluation at one commutes with pushing a polynomial along an algebra map.

```
lemma eval_one_mapAlgHom {k A B : Type*} [CommSemiring k] [Semiring A] [Semiring B]
    [Algebra k A] [Algebra k B] (f : A →ₐ[k] B) (p : Polynomial A) :
    (Polynomial.mapAlgHom f p).eval 1 = f (p.eval 1)
```

lemma mapCoeffs_mul_of_leibniz [source]

A map satisfying the Leibniz rule satisfies it coefficientwise on polynomials. Applied to a total derivative this is the Leibniz rule for the mass-weight polynomial.

```
lemma mapCoeffs_mul_of_leibniz {A : Type*} [Ring A] {D : A → A} (hD0 : D 0 = 0)
    (hDadd : ∀ a b : A, D (a + b) = D a + D b)
    (hD : ∀ a b : A, D (a * b) = D a * b + a * D b) (p q : Polynomial A) :
    mapCoeffs D (p * q) = mapCoeffs D p * q + p * mapCoeffs D q
```

2.337 PhyslibAlpha.Mathematics.RepresentationDual

PhyslibAlpha/Mathematics/RepresentationDual.lean on GitHub.

lemma dual_apply_dualBasis [source]

The components of a dual representation on a dual basis: if $\rho\ g^{-1}$ has matrix M in the basis b (columns indexing the argument), then $\rho.\mathrm{dual}\ g$ acts on the dual basis by the rows of M.

```
lemma dual_apply_dualBasis {k G V ι : Type*} [CommRing k]
    [Group G] [AddCommGroup V] [Module k V] [Fintype ι] [DecidableEq ι]
    (ρ : Representation k G V) (b : Module.Basis ι k V) (g : G) (i : ι)
    (M : Matrix ι ι k) (hM : ∀ j, ρ g⁻¹ (b j) = ∑ l, M l j • b l) :
    ρ.dual g (b.dualBasis i) = ∑ j, M i j • b.dualBasis j
```

2.338 PhyslibAlpha.Mathematics.RepresentationProdMap

PhyslibAlpha/Mathematics/RepresentationProdMap.lean on GitHub.

def prodMap [source]

The componentwise representation of a product group on a product space.

```
noncomputable def Representation.prodMap {k G₁ G₂ V₁ V₂ : Type*} [CommSemiring k]
    [Monoid G₁] [Monoid G₂] [AddCommMonoid V₁] [Module k V₁] [AddCommMonoid V₂] [Module k V₂]
    (ρ₁ : Representation k G₁ V₁) (ρ₂ : Representation k G₂ V₂) :
    Representation k (G₁ × G₂) (V₁ × V₂) where
```

lemma prodMap_apply [source]

```
@[simp]
lemma Representation.prodMap_apply {k G₁ G₂ V₁ V₂ : Type*} [CommSemiring k]
    [Monoid G₁] [Monoid G₂] [AddCommMonoid V₁] [Module k V₁] [AddCommMonoid V₂] [Module k V₂]
    (ρ₁ : Representation k G₁ V₁) (ρ₂ : Representation k G₂ V₂) (p : G₁ × G₂) (v : V₁ × V₂) :
    Representation.prodMap ρ₁ ρ₂ p v = (ρ₁ p.1 v.1, ρ₂ p.2 v.2)
```

2.339 `PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.Basic`

PhyslibAlpha/Mathematics/RepresentationTheory/Sl2/Basic.lean on GitHub.

lemma Φ_leibniz [source]

The action map satisfies the Leibniz rule, since Φ preserves brackets.

```
lemma Φ_leibniz [CharZero K] (t : IsSl2Triple h e f) (x y : ↥(t.toLieSubalgebra (R := K)))
    (v : StandardSl2Module K n) :
    Φ (n := n) t x (Φ t y v) = Φ t ⁅x, y⁆ v + Φ t y (Φ t x v)
```

lemma standardSl2ModuleLieModule [source]

Standard module `V(n)`, module structure, continued.

```
lemma standardSl2ModuleLieModule [CharZero K] (t : IsSl2Triple h e f) :
    letI
```

lemma h_mem_subalgebra [source]

```
lemma h_mem_subalgebra (t : IsSl2Triple h e f) : h ∈ t.toLieSubalgebra (R := K)
```

lemma e_mem_subalgebra [source]

```
lemma e_mem_subalgebra (t : IsSl2Triple h e f) : e ∈ t.toLieSubalgebra (R := K)
```

lemma f_mem_subalgebra [source]

```
lemma f_mem_subalgebra (t : IsSl2Triple h e f) : f ∈ t.toLieSubalgebra (R := K)
```

def hSub [source]

The subalgebra element corresponding to `h`, as used by the theorems below.

```
def hSub (t : IsSl2Triple h e f) : ↥(t.toLieSubalgebra (R := K))
```

def eSub [source]

The subalgebra element corresponding to `e`, as used by the theorems below.

```
def eSub (t : IsSl2Triple h e f) : ↥(t.toLieSubalgebra (R := K))
```

def fSub [source]

The subalgebra element corresponding to `f`, as used by the theorems below.

```
def fSub (t : IsSl2Triple h e f) : ↥(t.toLieSubalgebra (R := K))
```

lemma hSub_eq [source]

```
lemma hSub_eq (t : IsSl2Triple h e f) :
    hSub (K := K) t = subalgebraBasis (K := K) t 2
```

lemma eSub_eq [source]

```
lemma eSub_eq (t : IsSl2Triple h e f) :
    eSub (K := K) t = subalgebraBasis (K := K) t 0
```

lemma fSub_eq [source]

```
lemma fSub_eq (t : IsSl2Triple h e f) :
    fSub (K := K) t = subalgebraBasis (K := K) t 1
```

lemma standardSl2ModuleLieRingModule_h_apply [source]

```
lemma standardSl2ModuleLieRingModule_h_apply (t : IsSl2Triple h e f) (k : Fin (n + 1)) :
    letI
```

lemma standardSl2ModuleLieRingModule_e_apply_zero [source]

```
lemma standardSl2ModuleLieRingModule_e_apply_zero (t : IsSl2Triple h e f) :
    letI
```

lemma standardSl2ModuleLieRingModule_e_apply_pos [source]

```
lemma standardSl2ModuleLieRingModule_e_apply_pos (t : IsSl2Triple h e f)
    (k : Fin (n + 1)) (hk : 0 < (k : ℕ)) :
    letI
```

lemma standardSl2ModuleLieRingModule_f_apply_lt [source]

```
lemma standardSl2ModuleLieRingModule_f_apply_lt (t : IsSl2Triple h e f)
    (k : Fin (n + 1)) (hk : (k : ℕ) < n) :
    letI
```

lemma standardSl2ModuleLieRingModule_f_apply_last [source]

```
lemma standardSl2ModuleLieRingModule_f_apply_last (t : IsSl2Triple h e f) :
    letI
```

lemma isSl2Triple_sub [source]

`hSub t`, `eSub t`, `fSub t` themselves form an `IsSl2Triple` within the subalgebra `↥(t.toLieSubalgebra K)` (as opposed to `t` itself, whose `h, e, f` live in the ambient `L`). Useful for restricting sl2-representation-theory results (e.g. `IsSl2Triple.exists_hasPrimitiveVectorWith`) to a submodule that is invariant under the subalgebra generated by `t` but not necessarily under all of `L`.

```
lemma isSl2Triple_sub (t : IsSl2Triple h e f) :
    IsSl2Triple (hSub (K := K) t) (eSub (K := K) t) (fSub (K := K) t) where
```

2.340 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.BivariateBasis

PhyslibAlpha/Mathematics/RepresentationTheory/Sl2/BivariateBasis.lean on GitHub.

lemma toEnd_ambient_eq [source]

Under the tautological action, `LieModule.toEnd` is the original endomorphism.

```
lemma toEnd_ambient_eq (x : Module.End K V) :
    LieModule.toEnd K (Module.End K V) V x = x
```

lemma toEnd_ambient_pow [source]

Powers of the tautological action are powers of the endomorphism.

```
lemma toEnd_ambient_pow (x : Module.End K V) (k : ℕ) :
    LieModule.toEnd K (Module.End K V) V x ^ k = x ^ k
```

lemma commute_of_lie_eq_zero [source]

Commuting endomorphisms have vanishing associative-ring bracket, and conversely.

```
lemma commute_of_lie_eq_zero {a b : Module.End K V} (h : ⁅a, b⁆ = 0) : Commute a b
```

lemma apply_pow_comm [source]

A power of `f` commutes past an endomorphism commuting with `f`.

```
lemma apply_pow_comm {f y : Module.End K V} (hy : Commute f y) (a : ℕ) (v : V) :
    y ((f ^ a) v) = (f ^ a) (y v)
```

lemma pow_apply_eq_zero [source]

Powers of an endomorphism vanish beyond the first vanishing power.

```
lemma pow_apply_eq_zero {f : Module.End K V} {v : V} {k : ℕ} (h : (f ^ k) v = 0) {a : ℕ}
    (ha : k ≤ a) : (f ^ a) v = 0
```

lemma map_mem_span_of_range [source]

A linear map sending a spanning family into its span preserves the span.

```
lemma map_mem_span_of_range {ι : Type*} (g : ι → V) (y : Module.End K V)
    (hy : ∀ i, y (g i) ∈ Submodule.span K (Set.range g)) :
    ∀ v ∈ Submodule.span K (Set.range g), y v ∈ Submodule.span K (Set.range g)
```

lemma prim_h [source]

A primitive vector is a `h`-eigenvector, in plain-operator form.

```
lemma prim_h (P : t.HasPrimitiveVectorWith m (n : K)) : h m = (n : K) • m
```

lemma prim_e [source]

A primitive vector is killed by `e`, in plain-operator form.

```
lemma prim_e (P : t.HasPrimitiveVectorWith m (n : K)) : e m = 0
```

lemma prim_h_pow [source]

The `h`-weights along a lowering string, in plain-operator form.

```
lemma prim_h_pow (P : t.HasPrimitiveVectorWith m (n : K)) (k : ℕ) :
    h ((f ^ k) m) = ((n : K) - 2 * k) • (f ^ k) m
```

lemma prim_e_pow_succ [source]

The raising operator along a lowering string, in plain-operator form.

```
lemma prim_e_pow_succ (P : t.HasPrimitiveVectorWith m (n : K)) (k : ℕ) :
    e ((f ^ (k + 1)) m) = (((k : K) + 1) * ((n : K) - k)) • (f ^ k) m
```

lemma prim_f_pow_zero [source]

A lowering string closes after `n + 1` steps, in plain-operator form.

```
lemma prim_f_pow_zero [CharZero K] [FiniteDimensional K V]
    (P : t.HasPrimitiveVectorWith m (n : K)) : (f ^ (n + 1)) m = 0
```

lemma prim_f_pow_ne_zero [source]

The first `n + 1` lowering steps of a primitive vector are nonzero.

```
lemma prim_f_pow_ne_zero [CharZero K] (P : t.HasPrimitiveVectorWith m (n : K)) {k : ℕ}
    (hk : k ≤ n) : (f ^ k) m ≠ 0
```

lemma eq_zero_of_sub_eq_mul [source]

A difference of two numbers in `[0, n]` that is a multiple of `n + 1` vanishes.

```
lemma eq_zero_of_sub_eq_mul {n a a' d : ℤ} (ha : 0 ≤ a ∧ a ≤ n) (ha' : 0 ≤ a' ∧ a' ≤ n)
    (h : a' - a = (n + 1) * d) : d = 0
```

lemma int_weight_inj [source]

The integer weights `(n1 - 2a) + (n1 + 1)(n2 - 2b)` separate grid points.

```
lemma int_weight_inj {n1 n2 a a' b b' : ℕ} (ha : a ≤ n1) (ha' : a' ≤ n1)
    (h : (n1 : ℤ) - 2 * a + ((n1 : ℤ) + 1) * ((n2 : ℤ) - 2 * b) =
      (n1 : ℤ) - 2 * a' + ((n1 : ℤ) + 1) * ((n2 : ℤ) - 2 * b')) : a = a' ∧ b = b'
```

lemma weight_pair_injective [source]

The combined weights `h1 + (n1 + 1) h2` separate the points of the grid.

```
lemma weight_pair_injective [CharZero K] {n1 n2 : ℕ} :
    Function.Injective (fun p : Fin (n1 + 1) × Fin (n2 + 1) =>
      ((n1 : K) - 2 * (p.1 : ℕ)) + ((n1 : K) + 1) * ((n2 : K) - 2 * (p.2 : ℕ)))
```

structure IsBivariateGrid [source]

A family `g` indexed by the grid `Fin (n1 + 1) × Fin (n2 + 1)` realises the standard `V(n1) ⊗ V(n2)` action of two commuting `sl₂`-triples: `g (a, b)` has weights `(n1 - 2a, n2 - 2b)`, the lowering operators shift one index, and the raising operators shift it back with the standard coefficients.

```
structure IsBivariateGrid (h1 e1 f1 h2 e2 f2 : Module.End K V) (n1 n2 : ℕ)
    (g : Fin (n1 + 1) × Fin (n2 + 1) → V) : Prop where
  h1_apply (a : Fin (n1 + 1)) (b : Fin (n2 + 1)) :
    h1 (g (a, b)) = ((n1 : K) - 2 * (a : ℕ)) • g (a, b)
  h2_apply (a : Fin (n1 + 1)) (b : Fin (n2 + 1)) :
    h2 (g (a, b)) = ((n2 : K) - 2 * (b : ℕ)) • g (a, b)
  e1_zero (b : Fin (n2 + 1)) : e1 (g (0, b)) = 0
  e1_pos (a : Fin (n1 + 1)) (_ : 0 < (a : ℕ)) (b : Fin (n2 + 1)) : e1 (g (a, b)) =
    (((a : ℕ) : K) * ((n1 : K) - ((a : ℕ) : K) + 1)) • g (⟨(a : ℕ) - 1, by omega⟩, b)
  f1_lt (a : Fin (n1 + 1)) (_ : (a : ℕ) < n1) (b : Fin (n2 + 1)) :
    f1 (g (a, b)) = g (⟨(a : ℕ) + 1, by omega⟩, b)
  f1_last (b : Fin (n2 + 1)) : f1 (g (Fin.last n1, b)) = 0
  e2_zero (a : Fin (n1 + 1)) : e2 (g (a, 0)) = 0
  e2_pos (a : Fin (n1 + 1)) (b : Fin (n2 + 1)) (_ : 0 < (b : ℕ)) : e2 (g (a, b)) =
    (((b : ℕ) : K) * ((n2 : K) - ((b : ℕ) : K) + 1)) • g (a, ⟨(b : ℕ) - 1, by omega⟩)
  f2_lt (a : Fin (n1 + 1)) (b : Fin (n2 + 1)) (_ : (b : ℕ) < n2) :
    f2 (g (a, b)) = g (a, ⟨(b : ℕ) + 1, by omega⟩)
  f2_last (a : Fin (n1 + 1)) : f2 (g (a, Fin.last n2)) = 0
```

def JointlyIrreducible [source]

No proper nonzero subspace is invariant under all six operators of two `sl₂`-triples.

```
def JointlyIrreducible (h1 e1 f1 h2 e2 f2 : Module.End K V) : Prop
```

def gridVec [source]

The vector `f1 ^ a (f2 ^ b m)` of the bivariate lowering grid.

```
def gridVec (f1 f2 : Module.End K V) (m : V) (a b : ℕ) : V
```

def gridFun [source]

The bivariate lowering grid, indexed by `Fin (n1 + 1) × Fin (n2 + 1)`.

```
def gridFun (f1 f2 : Module.End K V) (m : V) (n1 n2 : ℕ) :
    Fin (n1 + 1) × Fin (n2 + 1) → V
```

def gridSpan [source]

The span of the bivariate lowering grid.

```
def gridSpan (f1 f2 : Module.End K V) (m : V) (n1 n2 : ℕ) : Submodule K V
```

lemma grid_primitive1 [source]

Each lowering of the second triple is still primitive for the first triple.

```
lemma grid_primitive1 {b : ℕ} (hb : b ≤ n2) :
    t1.HasPrimitiveVectorWith ((f2 ^ b) m0) (n1 : K)
```

lemma grid_ne [source]

The grid vectors inside the rectangle are nonzero.

```
lemma grid_ne {a b : ℕ} (ha : a ≤ n1) (hb : b ≤ n2) : gridVec f1 f2 m0 a b ≠ 0
```

lemma grid_h1 [source]

The first Cartan weight of a grid vector.

```
lemma grid_h1 {b : ℕ} (a : ℕ) (hb : b ≤ n2) :
    h1 (gridVec f1 f2 m0 a b) = ((n1 : K) - 2 * a) • gridVec f1 f2 m0 a b
```

lemma grid_h2 [source]

The second Cartan weight of a grid vector.

```
lemma grid_h2 (a b : ℕ) :
    h2 (gridVec f1 f2 m0 a b) = ((n2 : K) - 2 * b) • gridVec f1 f2 m0 a b
```

lemma grid_e1_zero [source]

The first raising operator kills the top of each column.

```
lemma grid_e1_zero {b : ℕ} (hb : b ≤ n2) : e1 (gridVec f1 f2 m0 0 b) = 0
```

lemma grid_e1_succ [source]

The first raising operator on a grid vector.

```
lemma grid_e1_succ {b : ℕ} (a : ℕ) (hb : b ≤ n2) :
    e1 (gridVec f1 f2 m0 (a + 1) b) =
      (((a : K) + 1) * ((n1 : K) - a)) • gridVec f1 f2 m0 a b
```

lemma grid_f1 [source]

The first lowering operator shifts the first index.

```
lemma grid_f1 (a b : ℕ) : f1 (gridVec f1 f2 m0 a b) = gridVec f1 f2 m0 (a + 1) b
```

lemma grid_f1_last [source]

The first lowering operator kills the end of each column.

```
lemma grid_f1_last [FiniteDimensional K V] {b : ℕ} (hb : b ≤ n2) :
    f1 (gridVec f1 f2 m0 n1 b) = 0
```

lemma grid_e2_zero [source]

The second raising operator kills the top of each row.

```
lemma grid_e2_zero (a : ℕ) : e2 (gridVec f1 f2 m0 a 0) = 0
```

lemma grid_e2_succ [source]

The second raising operator on a grid vector.

```
lemma grid_e2_succ (a b : ℕ) :
    e2 (gridVec f1 f2 m0 a (b + 1)) =
      (((b : K) + 1) * ((n2 : K) - b)) • gridVec f1 f2 m0 a b
```

lemma grid_f2 [source]

The second lowering operator shifts the second index.

```
lemma grid_f2 (a b : ℕ) : f2 (gridVec f1 f2 m0 a b) = gridVec f1 f2 m0 a (b + 1)
```

lemma grid_f2_last [source]

The second lowering operator kills the end of each row.

```
lemma grid_f2_last [FiniteDimensional K V] (a : ℕ) : f2 (gridVec f1 f2 m0 a n2) = 0
```

lemma grid_eq_zero [source]

Grid vectors outside the rectangle vanish.

```
lemma grid_eq_zero [FiniteDimensional K V] {a b : ℕ} (h : n1 < a ∨ n2 < b) :
    gridVec f1 f2 m0 a b = 0
```

lemma grid_mem_gridSpan [source]

Every grid vector lies in the span of the finite grid.

```
lemma grid_mem_gridSpan [FiniteDimensional K V] (a b : ℕ) :
    gridVec f1 f2 m0 a b ∈ gridSpan f1 f2 m0 n1 n2
```

lemma gridSpan_stable_h1 [source]

The span of the grid is stable under `h1`.

```
lemma gridSpan_stable_h1 : ∀ v ∈ gridSpan f1 f2 m0 n1 n2, h1 v ∈ gridSpan f1 f2 m0 n1 n2
```

lemma gridSpan_stable_h2 [source]

The span of the grid is stable under `h2`.

```
lemma gridSpan_stable_h2 : ∀ v ∈ gridSpan f1 f2 m0 n1 n2, h2 v ∈ gridSpan f1 f2 m0 n1 n2
```

lemma gridSpan_stable_f1 [source]

The span of the grid is stable under `f1`.

```
lemma gridSpan_stable_f1 : ∀ v ∈ gridSpan f1 f2 m0 n1 n2, f1 v ∈ gridSpan f1 f2 m0 n1 n2
```

lemma gridSpan_stable_f2 [source]

The span of the grid is stable under `f2`.

```
lemma gridSpan_stable_f2 : ∀ v ∈ gridSpan f1 f2 m0 n1 n2, f2 v ∈ gridSpan f1 f2 m0 n1 n2
```

lemma gridSpan_stable_e1 [source]

The span of the grid is stable under `e1`.

```
lemma gridSpan_stable_e1 : ∀ v ∈ gridSpan f1 f2 m0 n1 n2, e1 v ∈ gridSpan f1 f2 m0 n1 n2
```

lemma gridSpan_stable_e2 [source]

The span of the grid is stable under `e2`.

```
lemma gridSpan_stable_e2 : ∀ v ∈ gridSpan f1 f2 m0 n1 n2, e2 v ∈ gridSpan f1 f2 m0 n1 n2
```

lemma gridSpan_stable [source]

The span of the grid is stable under all six generators.

```
lemma gridSpan_stable {x : Module.End K V} (hx : x ∈ ({h1, e1, f1, h2, e2, f2} : Set _)) :
    ∀ v ∈ gridSpan f1 f2 m0 n1 n2, x v ∈ gridSpan f1 f2 m0 n1 n2
```

lemma gridSpan_eq_top [source]

Joint irreducibility forces the grid to span.

```
lemma gridSpan_eq_top (hirr : JointlyIrreducible h1 e1 f1 h2 e2 f2) :
    gridSpan f1 f2 m0 n1 n2 = ⊤
```

lemma top_le_span_gridFun [source]

The grid spans, in the form needed to build a basis.

```
lemma top_le_span_gridFun (hirr : JointlyIrreducible h1 e1 f1 h2 e2 f2) :
    ⊤ ≤ Submodule.span K (Set.range (gridFun f1 f2 m0 n1 n2))
```

lemma grid_hasEigenvector [source]

The combined Cartan operator has the grid vectors as eigenvectors.

```
lemma grid_hasEigenvector (p : Fin (n1 + 1) × Fin (n2 + 1)) :
    (h1 + ((n1 : K) + 1) • h2).HasEigenvector
      (((n1 : K) - 2 * (p.1 : ℕ)) + ((n1 : K) + 1) * ((n2 : K) - 2 * (p.2 : ℕ)))
      (gridFun f1 f2 m0 n1 n2 p)
```

lemma grid_linearIndependent [source]

The bivariate grid is linearly independent.

```
lemma grid_linearIndependent : LinearIndependent K (gridFun f1 f2 m0 n1 n2)
```

lemma grid_e1_pos [source]

The first raising operator on a grid vector, for a positive index.

```
lemma grid_e1_pos (a : Fin (n1 + 1)) (ha : 0 < (a : ℕ)) {b : ℕ} (hb : b ≤ n2) :
    e1 (gridVec f1 f2 m0 a b) = (((a : ℕ) : K) * ((n1 : K) - ((a : ℕ) : K) + 1)) •
      gridVec f1 f2 m0 ((a : ℕ) - 1) b
```

lemma grid_e2_pos [source]

The second raising operator on a grid vector, for a positive index.

```
lemma grid_e2_pos (a : ℕ) (b : Fin (n2 + 1)) (hb : 0 < (b : ℕ)) :
    e2 (gridVec f1 f2 m0 a b) = (((b : ℕ) : K) * ((n2 : K) - ((b : ℕ) : K) + 1)) •
      gridVec f1 f2 m0 a ((b : ℕ) - 1)
```

lemma gridFun_isBivariateGrid [source]

The grid realises the standard bivariate action.

```
lemma gridFun_isBivariateGrid :
    IsBivariateGrid h1 e1 f1 h2 e2 f2 n1 n2 (gridFun f1 f2 m0 n1 n2) where
```

def primSpace [source]

The first-triple primitive vectors of weight `n`: vectors killed by `e` with `h`-weight `n`.

```
def primSpace (h e : Module.End K V) (n : K) : Submodule K V
```

lemma mem_primSpace [source]

Membership in `primSpace`, in plain-operator form.

```
lemma mem_primSpace {h e : Module.End K V} {n : K} {v : V} :
    v ∈ primSpace h e n ↔ e v = 0 ∧ h v = n • v
```

lemma primSpace_map_mem [source]

An endomorphism commuting with `e` and `h` preserves `primSpace`.

```
lemma primSpace_map_mem {h e y : Module.End K V} {n : K} {v : V} (hey : Commute e y)
    (hhy : Commute h y) (hv : v ∈ primSpace h e n) : y v ∈ primSpace h e n
```

lemma hasPrimitiveVectorWith_of_mem_primSpace [source]

A vector of `primSpace` that is nonzero is primitive.

```
lemma hasPrimitiveVectorWith_of_mem_primSpace {h e f : Module.End K V} {t : IsSl2Triple h e f}
    {n : K} {w : V} (hw : w ∈ primSpace h e n) (hne : w ≠ 0) : t.HasPrimitiveVectorWith w n
```

lemma commute_of_hcomm [source]

Commuting triples give commuting operators.

```
lemma commute_of_hcomm {h1 e1 f1 h2 e2 f2 : Module.End K V}
    (hcomm : ∀ x ∈ ({h1, e1, f1} : Set (Module.End K V)),
      ∀ y ∈ ({h2, e2, f2} : Set (Module.End K V)), ⁅x, y⁆ = 0) :
    ∀ x ∈ ({h1, e1, f1} : Set (Module.End K V)),
      ∀ y ∈ ({h2, e2, f2} : Set (Module.End K V)), Commute x y
```

lemma nontrivial_of_isSl2Triple [source]

The representation space of a nonzero $\mathfrak{sl}_2$-triple is nontrivial.

```
lemma nontrivial_of_isSl2Triple {h e f : Module.End K V} (t : IsSl2Triple h e f) :
    Nontrivial V
```

lemma lie_mem_of_generators [source]

A subspace stable under the generators is stable under the generated subalgebra.

```
lemma lie_mem_of_generators {h e f : Module.End K V} (t : IsSl2Triple h e f) (W : Submodule K V
    )
    (hh : ∀ v ∈ W, h v ∈ W) (he : ∀ v ∈ W, e v ∈ W) (hf : ∀ v ∈ W, f v ∈ W)
    (x : ↥(t.toLieSubalgebra (R := K))) {v : V} (hv : v ∈ W) : ⁅(x : Module.End K V), v⁆ ∈ W
```

lemma hasPrimitiveVectorWith_of_sub [source]

A primitive vector of the induced triple on a Lie submodule is primitive in V.

```
lemma hasPrimitiveVectorWith_of_sub {h e f : Module.End K V} {t : IsSl2Triple h e f}
    {N : LieSubmodule K ↥(t.toLieSubalgebra (R := K)) V} {w : N} {n : K}
    (P : (isSl2Triple_sub (K := K) t).HasPrimitiveVectorWith w n) :
    t.HasPrimitiveVectorWith (w : V) n
```

lemma exists_primitive_in_invariant [source]

A nonzero subspace stable under the generators contains a primitive vector.

```
lemma exists_primitive_in_invariant {h e f : Module.End K V} (t : IsSl2Triple h e f)
    (W : Submodule K V) (hW : W ≠ ⊥) (hh : ∀ v ∈ W, h v ∈ W) (he : ∀ v ∈ W, e v ∈ W)
    (hf : ∀ v ∈ W, f v ∈ W) : ∃ (n : ℕ) (w : V), w ∈ W ∧ t.HasPrimitiveVectorWith w (n : K)
```

lemma primSpace_stable [source]

The primitive space of the first triple is stable under every operator of the second.

```
lemma primSpace_stable {n : K} {y : Module.End K V} {v : V}
    (hc : ∀ x ∈ ({h1, e1, f1} : Set (Module.End K V)),
      ∀ y ∈ ({h2, e2, f2} : Set (Module.End K V)), Commute x y)
    (hy : y ∈ ({h2, e2, f2} : Set (Module.End K V))) (hv : v ∈ primSpace h1 e1 n) :
    y v ∈ primSpace h1 e1 n
```

lemma exists_joint_primitive [source]

Two commuting triples have a common primitive vector. The primitive-weight space for the first triple is invariant under the second, so a primitive vector for the second can be chosen inside it.

```
lemma exists_joint_primitive : ∃ (n1 n2 : ℕ) (m0 : V), m0 ≠ 0 ∧
    t1.HasPrimitiveVectorWith m0 (n1 : K) ∧ t2.HasPrimitiveVectorWith m0 (n2 : K)
```

lemma exists_bivariate_basis [source]

For a jointly irreducible pair of commuting triples, the vectors `f1 ^ a (f2 ^ b m0)` form a basis. The basis realises the standard bivariate action, including the endpoints of each string.

```
lemma exists_bivariate_basis :
    ∃ (n1 n2 : ℕ) (B : Module.Basis (Fin (n1 + 1) × Fin (n2 + 1)) K V),
      IsBivariateGrid h1 e1 f1 h2 e2 f2 n1 n2 B
```

2.341 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.Classification

PhyslibAlpha/Mathematics/RepresentationTheory/Sl2/Classification.lean on GitHub.

def stringVecs [source]

The vectors `m, fm, …, f^n m` of the primitive-vector string, indexed by `Fin (n+1)`.

```
def stringVecs (f : L) (m : M) (n : ℕ) (k : Fin (n + 1)) : M
```

lemma string_hasEigenvector [source]

Each vector in a primitive lowering string is a nonzero eigenvector of `h`.

```
lemma string_hasEigenvector [CharZero K] {t : IsSl2Triple h e f}
    (P : t.HasPrimitiveVectorWith m (n : K)) (k : Fin (n + 1)) :
    (LieModule.toEnd K L M h).HasEigenvector ((n : K) - 2 * (k : ℕ))
      (stringVecs (K := K) f m n k)
```

lemma string_weight_injective [source]

The weights `n - 2k` of a lowering string are pairwise distinct in characteristic zero.

```
lemma string_weight_injective [CharZero K] {n : ℕ} :
    Function.Injective (fun k : Fin (n + 1) => (n : K) - 2 * (k : ℕ))
```

lemma string_linearIndependent [source]

The lowering string is linearly independent because its weights are distinct.

```
lemma string_linearIndependent [CharZero K] {t : IsSl2Triple h e f}
    (P : t.HasPrimitiveVectorWith m (n : K)) :
    LinearIndependent K (stringVecs (K := K) f m n)
```

def stringSpan [source]

The span of the primitive-vector string, as a plain `K`-submodule.

```
def stringSpan (f : L) (m : M) (n : ℕ) : Submodule K M
```

lemma stringPow_mem_span [source]

Every power of the lowering operator lies in the truncated string span.

```
lemma stringPow_mem_span [CharZero K] [FiniteDimensional K M] {t : IsSl2Triple h e f}
    (P : t.HasPrimitiveVectorWith m (n : K)) (k : ℕ) :
    (LieModule.toEnd K L M f ^ k) m ∈ stringSpan (K := K) f m n
```

lemma lie_e_string_mem [source]

The raising operator preserves the span of a primitive lowering string.

```
lemma lie_e_string_mem [CharZero K] [FiniteDimensional K M] {t : IsSl2Triple h e f}
    (P : t.HasPrimitiveVectorWith m (n : K)) (k : Fin (n + 1)) :
    ⁅e, stringVecs (K := K) f m n k⁆ ∈ stringSpan (K := K) f m n
```

lemma lie_f_string_mem [source]

The raising operator preserves the string span on `f`-powers.

```
lemma lie_f_string_mem [CharZero K] [FiniteDimensional K M] {t : IsSl2Triple h e f}
    (P : t.HasPrimitiveVectorWith m (n : K)) (k : Fin (n + 1)) :
    ⁅f, stringVecs (K := K) f m n k⁆ ∈ stringSpan (K := K) f m n
```

lemma lie_h_string_mem [source]

The Cartan operator preserves the string span.

```
lemma lie_h_string_mem {t : IsSl2Triple h e f}
    (P : t.HasPrimitiveVectorWith m (n : K)) (k : Fin (n + 1)) :
    ⁅h, stringVecs (K := K) f m n k⁆ ∈ stringSpan (K := K) f m n
```

lemma lie_generator_string_mem [source]

Each generator of the triple preserves the lowering-string span.

```
lemma lie_generator_string_mem [CharZero K] [FiniteDimensional K M] {t : IsSl2Triple h e f}
    (P : t.HasPrimitiveVectorWith m (n : K))
    (x : ↥(t.toLieSubalgebra (R := K))) (k : Fin (n + 1)) :
    ⁅x, stringVecs (K := K) f m n k⁆ ∈ stringSpan (K := K) f m n
```

lemma exists_stringLieSubmodule [source]

The lowering string spans a submodule for the Lie subalgebra generated by `t`. The finite-dimensional hypothesis supplies the relation that closes the string after `n + 1` lowering steps.

```
lemma exists_stringLieSubmodule [CharZero K] [FiniteDimensional K M] {t : IsSl2Triple h e f}
    (P : t.HasPrimitiveVectorWith m (n : K)) :
    ∃ N : LieSubmodule K ↥(t.toLieSubalgebra (R := K)) M,
      (N : Submodule K M) = stringSpan f m n
```

lemma stringSpan_eq_top [source]

Irreducibility makes the nonzero lowering-string submodule equal to the whole module.

```
lemma stringSpan_eq_top {t : IsSl2Triple h e f}
    [LieModule.IsIrreducible K ↥(t.toLieSubalgebra (R := K)) M]
    (P : t.HasPrimitiveVectorWith m (n : K)) (N : LieSubmodule K ↥(t.toLieSubalgebra (R := K))
    M)
    (hN : (N : Submodule K M) = stringSpan f m n) : N = ⊤
```

lemma string_isBasis [source]

In an irreducible module, the lowering string is independent and spans.

```
lemma string_isBasis [CharZero K] [FiniteDimensional K M] {t : IsSl2Triple h e f}
    [LieModule.IsIrreducible K ↥(t.toLieSubalgebra (R := K)) M]
    (P : t.HasPrimitiveVectorWith m (n : K)) :
    LinearIndependent K (stringVecs (K := K) f m n) ∧
      Submodule.span K (Set.range (stringVecs (K := K) f m n)) = ⊤
```

lemma finrank_eq_succ [source]

Basis theorem, dimension count.

```
lemma finrank_eq_succ [CharZero K] [FiniteDimensional K M] {t : IsSl2Triple h e f}
    [LieModule.IsIrreducible K ↥(t.toLieSubalgebra (R := K)) M]
    (P : t.HasPrimitiveVectorWith m (n : K)) :
    Module.finrank K M = n + 1
```

def stringBasis [source]

The basis of an irreducible module obtained by lowering a primitive vector.

```
noncomputable def stringBasis [CharZero K] [FiniteDimensional K M]
    {t : IsSl2Triple h e f}
    [LieModule.IsIrreducible K ↥(t.toLieSubalgebra (R := K)) M]
    (P : t.HasPrimitiveVectorWith m (n : K)) : Module.Basis (Fin (n + 1)) K M
```

def stringEquiv [source]

The linear equivalence sending the lowering-string basis to the coordinate basis.

```
noncomputable def stringEquiv [CharZero K] [FiniteDimensional K M]
    {t : IsSl2Triple h e f}
    [LieModule.IsIrreducible K ↥(t.toLieSubalgebra (R := K)) M]
    (P : t.HasPrimitiveVectorWith m (n : K)) : M ≃ₗ[K] StandardSl2Module K n
```

lemma stringEquiv_apply [source]

The linear equivalence sends each lowering-string vector to the corresponding coordinate vector.

```
lemma stringEquiv_apply [CharZero K] [FiniteDimensional K M]
    {t : IsSl2Triple h e f}
    [LieModule.IsIrreducible K ↥(t.toLieSubalgebra (R := K)) M]
    (P : t.HasPrimitiveVectorWith m (n : K)) (i : Fin (n + 1)) :
    stringEquiv P (stringVecs (K := K) f m n i) = StandardSl2Module.v i
```

lemma stringEquiv_e_zero [source]

The raising operators agree at the primitive vector.

```
lemma stringEquiv_e_zero :
    letI
```

lemma stringEquiv_e_succ [source]

The raising operators agree at the string vector with index `j + 1`.

```
lemma stringEquiv_e_succ (j : ℕ) (hj : j < n) :
    letI
```

lemma stringEquiv_e_pos [source]

The raising operators agree at every positive string index.

```
lemma stringEquiv_e_pos (i : Fin (n + 1)) (hi : 0 < (i : ℕ)) :
    letI
```

lemma equiv_comm_of_basis [source]

A linear equivalence intertwines two endomorphisms if it does so on a basis.

```
lemma equiv_comm_of_basis {ι V : Type*} [AddCommGroup V] [Module K V]
    (b : Module.Basis ι K M) (Ψ : M ≃ₗ[K] V) (A : M →ₗ[K] M) (B : V →ₗ[K] V)
    (hb : ∀ i, Ψ (A (b i)) = B (Ψ (b i))) (w : M) : Ψ (A w) = B (Ψ w)
```

lemma stringEquiv_lie_of_basis [source]

The coordinate equivalence intertwines `x` once it does so on the string basis.

```
lemma stringEquiv_lie_of_basis (x : ↥(t.toLieSubalgebra (R := K)))
    (hx : letI := standardSl2ModuleLieRingModule (K := K) (n := n) t
      ∀ i, stringEquiv P ⁅x, stringVecs (K := K) f m n i⁆ =
        ⁅x, StandardSl2Module.v (K := K) (n := n) i⁆) (w : M) :
    letI
```

lemma stringEquiv_e [source]

The coordinate equivalence intertwines the raising operator.

```
lemma stringEquiv_e (w : M) :
    letI
```

lemma stringEquiv_f_last [source]

The lowering operators agree at the last string vector.

```
lemma stringEquiv_f_last :
    letI
```

lemma stringEquiv_f_lt [source]

The lowering operators agree before the last string vector.

```
lemma stringEquiv_f_lt (i : Fin (n + 1)) (hi : (i : ℕ) < n) :
    letI
```

lemma stringEquiv_f [source]

The coordinate equivalence intertwines the lowering operator.

```
lemma stringEquiv_f (w : M) :
    letI
```

lemma stringEquiv_h_basis [source]

The Cartan operators agree on every string vector.

```
lemma stringEquiv_h_basis (i : Fin (n + 1)) :
    letI
```

lemma stringEquiv_h [source]

The coordinate equivalence intertwines the Cartan operator.

```
lemma stringEquiv_h (w : M) :
    letI
```

lemma exists_decomp [source]

Every element of the generated subalgebra is a combination of `e`, `f`, `h`.

```
lemma exists_decomp (x : ↥(t.toLieSubalgebra (R := K))) :
    ∃ c₁ c₂ c₃ : K, x = c₁ • eSub (K := K) t + c₂ • fSub (K := K) t + c₃ • hSub (K := K) t
```

lemma stringEquiv_lie [source]

The coordinate equivalence intertwines every element of the generated subalgebra.

```
lemma stringEquiv_lie (x : ↥(t.toLieSubalgebra (R := K))) (w : M) :
    letI
```

lemma classification [source]

A finite-dimensional irreducible module for the generated $\mathfrak{sl}_2$ subalgebra is equivalent to a standard highest-weight module.

```
lemma classification [IsAlgClosed K] [CharZero K] [FiniteDimensional K M] (t : IsSl2Triple h e
    f)
    [LieModule.IsIrreducible K ↥(t.toLieSubalgebra (R := K)) M] :
    ∃ n : ℕ, letI
```

lemma classification_n_eq_finrank_sub_one [source]

Uniqueness of the highest weight: the `n` produced by `classification` is determined by `M` alone, namely `n = finrank K M - 1`.

Unlike `classification`, irreducibility is not actually needed here: only the bare existence of some `K`-linear equivalence `M ≃ₗ[K] V(n)` (forgetting the Lie-module structure `hn` provides) is used, via a dimension count.

```
lemma classification_n_eq_finrank_sub_one [CharZero K]
    {t : IsSl2Triple h e f} {n : ℕ}
    (hn : letI := standardSl2ModuleLieRingModule (K := K) (n := n) t
      Nonempty (M ≃ₗ[K, ↥(t.toLieSubalgebra (R := K))] StandardSl2Module K n)) :
    n = Module.finrank K M - 1
```

2.342 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.ClebschGordan

PhyslibAlpha/Mathematics/RepresentationTheory/Sl2/ClebschGordan.lean on GitHub.

lemma sum_arithmetic_progression [source]

The arithmetic progression underlying the Clebsch–Gordan dimension count.

```
lemma sum_arithmetic_progression (m d : ℕ) :
    ∑ k ∈ Finset.range (m + 1), (d + 2 * k + 1) = (m + 1) * (d + m + 1)
```

lemma clebsch_gordan_finrank_aux [source]

Reindex the dimension sum when `m ≤ n`.

```
lemma clebsch_gordan_finrank_aux (m n : ℕ) (h : m ≤ n) :
    (m + 1) * (n + 1) = ∑ k ∈ Finset.range (m + 1), (m + n - 2 * k + 1)
```

lemma clebsch_gordan_finrank [source]

Clebsch–Gordan decomposition, dimension form: the numerical shadow `(m+1)(n+1) = ∑_{k=0}^{min(m,n)} (m+n-2k+1)` of the module isomorphism `V(m) ⊗ V(n) ≅ ⊕_k V(m+n-2k)`.

```
lemma clebsch_gordan_finrank (m n : ℕ) :
    (m + 1) * (n + 1) = ∑ k ∈ Finset.range (min m n + 1), (m + n - 2 * k + 1)
```

lemma clebsch_gordan [source]

Clebsch–Gordan decomposition, vector-space form: `V(m) ⊗ V(n)` is linearly isomorphic to `⊕_{k=0}^{min(m,n)} V(m+n-2k)`.

The equivalence follows only from equal dimensions. It does not identify the diagonal sl_2 action with the action on the direct sum or provide Clebsch–Gordan coefficients.

```
lemma clebsch_gordan (m n : ℕ) :
    Nonempty (StandardSl2Module K m ⊗[K] StandardSl2Module K n ≃ₗ[K]
      ⨁ _k : Fin (min m n + 1), StandardSl2Module K (m + n - 2 * (_k : ℕ)))
```

2.343 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.CommutingActions

PhyslibAlpha/Mathematics/RepresentationTheory/Sl2/CommutingActions.lean on GitHub.

lemma finrank_standardSl2Module [source]

The standard module `V(k)` has dimension `k + 1`.

```
lemma finrank_standardSl2Module (k : ℕ) : Module.finrank K (StandardSl2Module K k) = k + 1
```

lemma commuting_classification_finrank [source]

Dimension consequence of `commuting_classification`: `finrank K V = (m+1)(n+1)`.

```
lemma commuting_classification_finrank :
    ∃ m n : ℕ, Module.finrank K V = (m + 1) * (n + 1)
```

lemma commuting_classification [source]

Classification of jointly irreducible commuting $\mathfrak{sl}_2$-actions. Two $\mathfrak{sl}_2$-triples of operators on a finite-dimensional space `V`, commuting with each other and with no proper subspace invariant under both, force `V` to be linearly isomorphic to a tensor product `V(m) ⊗ V(n)` of standard modules.

The equivalence is obtained from a dimension count. An equivalence intertwining the two actions remains to be constructed.

```
lemma commuting_classification :
    ∃ m n : ℕ, Nonempty (V ≃ₗ[K] StandardSl2Module K m ⊗[K] StandardSl2Module K n)
```

2.344 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.Construction

PhyslibAlpha/Mathematics/RepresentationTheory/Sl2/Construction.lean on GitHub.

instance AddCommGroup (StandardSl2Module K n) [source]

```
instance : AddCommGroup (StandardSl2Module K n)
```

instance Module K (StandardSl2Module K n) [source]

```
instance : Module K (StandardSl2Module K n)
```

def v [source]

The standard basis vector `v_k` of `StandardSl2Module K n`.

```
def StandardSl2Module.v (k : Fin (n + 1)) : StandardSl2Module K n
```

lemma generators_eigenvectors [source]

The three generators are eigenvectors of the adjoint action of `h`.

```
lemma generators_eigenvectors (t : IsSl2Triple h e f) (i : Fin 3) :
    (LieModule.toEnd K L L h).HasEigenvector (![(2 : K), -2, 0] i) (![e, f, h] i)
```

lemma linearIndependent_efh [source]

The generators of an $\mathfrak{sl}_2$ triple are linearly independent.

```
lemma linearIndependent_efh (t : IsSl2Triple h e f) :
    LinearIndependent K ![e, f, h]
```

lemma span_efh_eq [source]

```
lemma span_efh_eq (t : IsSl2Triple h e f) :
    Submodule.span K (Set.range ![e, f, h]) = (t.toLieSubalgebra (R := K)).toSubmodule
```

def efhSub [source]

The generators `(e,f,h)`, as elements of the subalgebra subtype.

```
def efhSub (t : IsSl2Triple h e f) : Fin 3 → ↥(t.toLieSubalgebra (R := K))
```

lemma linearIndependent_efhSub [source]

The generators `(e,f,h)` are linearly independent in the subalgebra.

```
lemma linearIndependent_efhSub (t : IsSl2Triple h e f) :
    LinearIndependent K (efhSub (K := K) t)
```

lemma efhSub_span_top [source]

The generators `(e,f,h)` span the subalgebra.

```
lemma efhSub_span_top (t : IsSl2Triple h e f) :
    ⊤ ≤ Submodule.span K (Set.range (efhSub (K := K) t))
```

def subalgebraBasis [source]

A basis of the $\mathfrak{sl}_2$-subalgebra generated by `t`, given by `(e,f,h)`.

```
def subalgebraBasis (t : IsSl2Triple h e f) :
    Basis (Fin 3) K ↥(t.toLieSubalgebra (R := K))
```

lemma subalgebraBasis_apply [source]

```
lemma subalgebraBasis_apply (t : IsSl2Triple h e f) (i : Fin 3) :
    (subalgebraBasis (K := K) t i : L) = ![e, f, h] i
```

def ρE [source]

*`e` acting on `StandardSl2Module K n`: `ρE (v k) = (k(n-k+1)) • v (k-1)`, `ρE (v 0) = 0`. As a per-component formula (component `k` of the output reads from input component `k+1`, scaled by the coefficient attached to *that* input index): zero at `k = n` (nothing shifts down into the top component), else `((k+1)(n-k)) • w (k+1)`.*

```
def ρE : Module.End K (StandardSl2Module K n) where
```

def ρF [source]

f acting on `StandardSl2Module K n`: `ρF (v k) = v (k+1)`, `ρF (v n) = 0`.

```
def ρF : Module.End K (StandardSl2Module K n) where
```

def ρH [source]

h acting on `StandardSl2Module K n`: `ρH (v k) = (n - 2k) • v k`.

```
def ρH : Module.End K (StandardSl2Module K n) where
```

lemma ρH_v [source]

```
lemma ρH_v (k : Fin (n + 1)) :
    (ρH : Module.End K (StandardSl2Module K n)) (StandardSl2Module.v k)
      = ((n : K) - 2 * (k : ℕ)) • StandardSl2Module.v k
```

lemma ρF_v_lt [source]

```
lemma ρF_v_lt (k : Fin (n + 1)) (hk : (k : ℕ) < n) :
    (ρF : Module.End K (StandardSl2Module K n)) (StandardSl2Module.v k)
      = StandardSl2Module.v ⟨(k : ℕ) + 1, by omega⟩
```

lemma ρF_v_last [source]

```
lemma ρF_v_last :
    (ρF : Module.End K (StandardSl2Module K n)) (StandardSl2Module.v (Fin.last n)) = 0
```

lemma ρE_v_pos [source]

```
lemma ρE_v_pos (k : Fin (n + 1)) (hk : 0 < (k : ℕ)) :
    (ρE : Module.End K (StandardSl2Module K n)) (StandardSl2Module.v k)
      = ((k : ℕ) * (n - (k : ℕ) + 1) : K) • StandardSl2Module.v ⟨(k : ℕ) - 1, by omega⟩
```

lemma ρE_v_zero [source]

```
lemma ρE_v_zero :
    (ρE : Module.End K (StandardSl2Module K n)) (StandardSl2Module.v ⟨0, Nat.succ_pos n⟩) = 0
```

lemma cast_pred [source]

Casting a predecessor index into `K` subtracts one.

```
lemma cast_pred (j : Fin (n + 1)) (hj0 : (j : ℕ) ≠ 0) :
    (((j : ℕ) - 1 : ℕ) : K) = (j : K) - 1
```

lemma pred_succ_index [source]

The successor of the predecessor index is the index itself.

```
lemma pred_succ_index (j : Fin (n + 1)) (hj0 : (j : ℕ) ≠ 0) :
    (⟨(j : ℕ) - 1 + 1, by omega⟩ : Fin (n + 1)) = j
```

lemma rel_ef_last_zero [source]

The $\mathfrak{sl}_2$ relations hold for `(ρH,ρE,ρF)` on `StandardSl2Module K n`: `[ρE,ρF]=ρH`, `[ρH,ρE]=2ρE`, `[ρH,ρF]=-2ρF`. Checked componentwise on an arbitrary vector `w`: each composite operator, evaluated at output index `j`, only ever reads `w j` itself (the `E/F` shifts cancel), so the identity reduces to the coefficient computation already recorded in the `ρ_v` lemmas' docstrings.

```
lemma rel_ef_last_zero (w : StandardSl2Module K n) (j : Fin (n + 1))
    (hjn : (j : ℕ) = n) (hj0 : (j : ℕ) = 0) :
    (((ρE * ρF - ρF * ρE : Module.End K (StandardSl2Module K n)) w) j) =
      (ρH : Module.End K (StandardSl2Module K n)) w j
```

lemma rel_ef_last_pos [source]

```
lemma rel_ef_last_pos (w : StandardSl2Module K n) (j : Fin (n + 1))
    (hjn : (j : ℕ) = n) (hj0 : (j : ℕ) ≠ 0) :
    (((ρE * ρF - ρF * ρE : Module.End K (StandardSl2Module K n)) w) j) =
      (ρH : Module.End K (StandardSl2Module K n)) w j
```

lemma rel_ef_first [source]

```
lemma rel_ef_first (w : StandardSl2Module K n) (j : Fin (n + 1))
    (hjn : (j : ℕ) ≠ n) (hj0 : (j : ℕ) = 0) :
    (((ρE * ρF - ρF * ρE : Module.End K (StandardSl2Module K n)) w) j) =
      (ρH : Module.End K (StandardSl2Module K n)) w j
```

lemma rel_ef_middle [source]

```
lemma rel_ef_middle (w : StandardSl2Module K n) (j : Fin (n + 1))
    (hjn : (j : ℕ) ≠ n) (hj0 : (j : ℕ) ≠ 0) :
    (((ρE * ρF - ρF * ρE : Module.End K (StandardSl2Module K n)) w) j) =
      (ρH : Module.End K (StandardSl2Module K n)) w j
```

lemma rel_ef [source]

The standard raising and lowering operators have commutator `ρH`.

```
lemma rel_ef :
    ⁅(ρE : Module.End K (StandardSl2Module K n)), (ρF : Module.End K (StandardSl2Module K n))⁆
      = ρH
```

lemma rel_he [source]

```
lemma rel_he :
    ⁅(ρH : Module.End K (StandardSl2Module K n)), (ρE : Module.End K (StandardSl2Module K n))⁆
      = (2 : K) • ρE
```

lemma rel_hf [source]

```
lemma rel_hf :
    ⁅(ρH : Module.End K (StandardSl2Module K n)), (ρF : Module.End K (StandardSl2Module K n))⁆
      = (-2 : K) • ρF
```

lemma endLieSkew [source]

Antisymmetry of the bracket on `Module.End K (StandardSl2Module K n)`, proved directly via the associative-ring commutator rather than via the generic `lie_skew` (which does not syntactically match brackets here, since Mathlib derives a `LieRing/Bracket` structure on an endomorphism ring through more than one route).

```
lemma endLieSkew (a b : Module.End K (StandardSl2Module K n)) : ⁅a, b⁆ = -⁅b, a⁆
```

lemma endAddLie [source]

Bilinearity of the bracket on `Module.End K (StandardSl2Module K n)`, proved directly via the associative-ring commutator for the same reason as `endLieSkew`.

```
lemma endAddLie (a b c : Module.End K (StandardSl2Module K n)) :
    ⁅a + b, c⁆ = ⁅a, c⁆ + ⁅b, c⁆
```

lemma endLieAdd [source]

```
lemma endLieAdd (a b c : Module.End K (StandardSl2Module K n)) :
    ⁅a, b + c⁆ = ⁅a, b⁆ + ⁅a, c⁆
```

lemma endSmulLie [source]

```
lemma endSmulLie (r : K) (a b : Module.End K (StandardSl2Module K n)) :
    ⁅r • a, b⁆ = r • ⁅a, b⁆
```

lemma endLieSmul [source]

```
lemma endLieSmul (r : K) (a b : Module.End K (StandardSl2Module K n)) :
    ⁅a, r • b⁆ = r • ⁅a, b⁆
```

def Φ [source]

The Lie-algebra homomorphism from the $\mathfrak{sl}_2$-subalgebra generated by `t` into `Module.End K (StandardSl2Module K n)`, sending `e ↦ ρE`, `f ↦ ρF`, `h ↦ ρH`. Built by transporting `![ρE,ρF,ρH]` along the basis `subalgebraBasis t`; the fact that it respects brackets (`Φ_lie`, below) is checked on the generators via `rel_ef/rel_he/rel_hf` and then extended to the whole subalgebra by bilinearity.

```
def Φ (t : IsSl2Triple h e f) :
    𝔰(t.toLieSubalgebra (R := K)) →ₗ⁅K⁆ Module.End K (StandardSl2Module K n)
```

lemma Φ_apply [source]

```
lemma Φ_apply (t : IsSl2Triple h e f) (i : Fin 3) :
    Φ t (subalgebraBasis t i) = ![(ρE : Module.End K (StandardSl2Module K n)), ρF, ρH] i
```

lemma lie_subalgebraBasis [source]

Transports a bracket identity among the generators `(e,f,h)` down to the corresponding identity among the basis elements `subalgebraBasis t 0,1,2` of the subalgebra.

```
lemma lie_subalgebraBasis (t : IsSl2Triple h e f) (i j k : Fin 3) (c : K)
    (hc : ⁅![e, f, h] i, ![e, f, h] j⁆ = c • ![e, f, h] k) :
    ⁅subalgebraBasis (K := K) t i, subalgebraBasis (K := K) t j⁆
      = c • subalgebraBasis (K := K) t k
```

lemma Φ_bracket_refl [source]

```
lemma Φ_bracket_refl (t : IsSl2Triple h e f)
    (x : ↥(t.toLieSubalgebra (R := K))) :
    Φ (K := K) (n := n) t ⁅x, x⁆ = ⁅Φ (K := K) (n := n) t x, Φ (K := K) (n := n) t x⁆
```

lemma Φ_bracket_swap [source]

```
lemma Φ_bracket_swap (t : IsSl2Triple h e f)
    (x y : ↥(t.toLieSubalgebra (R := K)))
    (hxy : Φ (K := K) (n := n) t ⁅x, y⁆ = ⁅Φ (K := K) (n := n) t x, Φ (K := K) (n := n) t y⁆) :
    Φ (K := K) (n := n) t ⁅y, x⁆ = ⁅Φ (K := K) (n := n) t y, Φ (K := K) (n := n) t x⁆
```

lemma Φ_bracket_ef [source]

```
lemma Φ_bracket_ef (t : IsSl2Triple h e f) :
    Φ (K := K) (n := n) t
      ⁅subalgebraBasis (K := K) t 0, subalgebraBasis (K := K) t 1⁆ =
        ⁅Φ (K := K) (n := n) t (subalgebraBasis (K := K) t 0),
          Φ (K := K) (n := n) t (subalgebraBasis (K := K) t 1)⁆
```

lemma Φ_bracket_he [source]

```
lemma Φ_bracket_he (t : IsSl2Triple h e f) :
    Φ (K := K) (n := n) t
      ⁅subalgebraBasis (K := K) t 2, subalgebraBasis (K := K) t 0⁆ =
        ⁅Φ (K := K) (n := n) t (subalgebraBasis (K := K) t 2),
          Φ (K := K) (n := n) t (subalgebraBasis (K := K) t 0)⁆
```

lemma Φ_bracket_hf [source]

```
lemma Φ_bracket_hf (t : IsSl2Triple h e f) :
    Φ (K := K) (n := n) t
      ⁅subalgebraBasis (K := K) t 2, subalgebraBasis (K := K) t 1⁆ =
        ⁅Φ (K := K) (n := n) t (subalgebraBasis (K := K) t 2),
          Φ (K := K) (n := n) t (subalgebraBasis (K := K) t 1)⁆
```

lemma Φ_bracket_basis [source]

The action map preserves brackets on pairs of the three generators.

```
lemma Φ_bracket_basis (t : IsSl2Triple h e f) (i j : Fin 3) :
    Φ (K := K) (n := n) t
      ⁅subalgebraBasis (K := K) t i, subalgebraBasis (K := K) t j⁆ =
        ⁅Φ (K := K) (n := n) t (subalgebraBasis (K := K) t i),
          Φ (K := K) (n := n) t (subalgebraBasis (K := K) t j)⁆
```

lemma Φ_lie [source]

The action map preserves brackets because it does so on pairs of basis elements.

```
lemma Φ_lie (t : IsSl2Triple h e f)
    (x y : ↥(t.toLieSubalgebra (R := K))) :
    Φ (K := K) (n := n) t ⁅x, y⁆
      =
      ⁅(Φ (K := K) (n := n) t x : Module.End K (StandardSl2Module K n)),
        (Φ (K := K) (n := n) t y : Module.End K (StandardSl2Module K n))⁆
```

2.345 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.Schur

PhyslibAlpha/Mathematics/RepresentationTheory/Sl2/Schur.lean on GitHub.

structure IsIntertwiner [source]

A linear map between standard modules is an intertwiner if it commutes with the raising, lowering and weight operators.

```
structure IsIntertwiner (φ : StandardSl2Module K a →ₗ[K] StandardSl2Module K b) : Prop where
  comm_ρE : φ ∘ₗ ρE = ρE ∘ₗ φ
  comm_ρF : φ ∘ₗ ρF = ρF ∘ₗ φ
  comm_ρH : φ ∘ₗ ρH = ρH ∘ₗ φ
```

lemma apply_ρE [source]

```
lemma apply_ρE (hφ : IsIntertwiner φ) (x : StandardSl2Module K a) : φ (ρE x) = ρE (φ x)
```

lemma apply_ρF [source]

```
lemma apply_ρF (hφ : IsIntertwiner φ) (x : StandardSl2Module K a) : φ (ρF x) = ρF (φ x)
```

lemma apply_ρH [source]

```
lemma apply_ρH (hφ : IsIntertwiner φ) (x : StandardSl2Module K a) : φ (ρH x) = ρH (φ x)
```

lemma apply_ρF_pow [source]

```
lemma apply_ρF_pow (hφ : IsIntertwiner φ) (l : ℕ) (x : StandardSl2Module K a) :
    φ ((ρF ^ l) x) = (ρF ^ l) (φ x)
```

lemma id [source]

The identity is an intertwiner.

```
lemma id : IsIntertwiner (LinearMap.id : StandardSl2Module K n →ₗ[K] StandardSl2Module K n)
```

lemma smul [source]

Scalar multiples of intertwiners are intertwiners.

```
lemma smul (hφ : IsIntertwiner φ) (c : K) : IsIntertwiner (c • φ) where
```

lemma add [source]

Sums of intertwiners are intertwiners.

```
lemma add (hφ : IsIntertwiner φ) (hψ : IsIntertwiner ψ) : IsIntertwiner (φ + ψ) where
```

lemma ρF_pow_vNat_zero [source]

The lowering string of `v_0` runs through the coordinate vectors: `f^l v_0 = v_l`.

```
lemma ρF_pow_vNat_zero (l : ℕ) :
    ((ρF : Module.End K (StandardSl2Module K n)) ^ l) (vNat n 0) = vNat n l
```

lemma linearMap_ext_vNat [source]

Two linear maps out of a standard module agree if they agree on all `v_l`.

```
lemma StandardSl2Module.linearMap_ext_vNat {M : Type*} [AddCommGroup M]
    [Module K M] {α β : StandardSl2Module K a →ₗ[K] M} (h : ∀ l, α (vNat a l) = β (vNat a l)) :
    α = β
```

lemma eq_smul_vNat_zero_of_ρE_eq_zero [source]

A vector of `V(n)` killed by the raising operator is a multiple of `v_0`.

```
lemma eq_smul_vNat_zero_of_ρE_eq_zero [CharZero K] {w : StandardSl2Module K n}
    (hw : (ρE : Module.End K (StandardSl2Module K n)) w = 0) :
    w = w ⟨0, Nat.succ_pos n⟩ • vNat n 0
```

lemma apply_vNat_zero [source]

The image of `v_0` under an intertwiner is a multiple of `v_0`.

```
lemma apply_vNat_zero (hφ : IsIntertwiner φ) :
    φ (vNat a 0) = φ (vNat a 0) ⟨0, Nat.succ_pos b⟩ • vNat b 0
```

lemma apply_vNat [source]

An intertwiner sends every coordinate vector `v_l` to the same multiple `c • v_l`, where `c` is the `v_0`-coordinate of the image of `v_0`.

```
lemma apply_vNat (hφ : IsIntertwiner φ) (l : ℕ) :
    φ (vNat a l) = φ (vNat a 0) ⟨0, Nat.succ_pos b⟩ • vNat b l
```

lemma coeff_eq_zero_of_ne [source]

The scalar of an intertwiner vanishes unless the two highest weights agree.

```
lemma coeff_eq_zero_of_ne (hφ : IsIntertwiner φ) (hab : a ≠ b) :
    φ (vNat a 0) ⟨0, Nat.succ_pos b⟩ = 0
```

lemma eq_zero_of_ne [source]

***Schur's lemma**, first half: an intertwiner `V(a) → V(b)` with `a ≠ b` is zero.*

```
lemma eq_zero_of_ne (hφ : IsIntertwiner φ) (hab : a ≠ b) : φ = 0
```

lemma eq_smul_id [source]

***Schur's lemma**, second half: an intertwiner `V(n) → V(n)` is a scalar multiple of the identity.*

```
lemma eq_smul_id {φ : StandardSl2Module K n →ₗ[K] StandardSl2Module K n} (hφ : IsIntertwiner φ)
    :
    φ = φ (vNat n 0) ⟨0, Nat.succ_pos n⟩ • LinearMap.id
```

2.346 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.TensorDecomposition

PhyslibAlpha/Mathematics/RepresentationTheory/Sl2/TensorDecomposition.lean on GitHub.

def stringVec [source]

The `l`-th vector `f^l w_k` of the lowering string of `hwVec m n k`.

```
def stringVec (m n k l : ℕ) : StandardSl2Module K m ⊗[K] StandardSl2Module K n
```

lemma tensorF_stringVec [source]

```
lemma tensorF_stringVec (k l : ℕ) :
    tensorF (K := K) m n (stringVec m n k l) = stringVec m n k (l + 1)
```

lemma tensorH_stringVec [source]

```
lemma tensorH_stringVec (l : ℕ) :
    tensorH (K := K) m n (stringVec m n k l) =
      (((m + n - 2 * k : ℕ) : K) - 2 * l) • stringVec m n k l
```

lemma tensorE_stringVec_succ [source]

```
lemma tensorE_stringVec_succ (l : ℕ) :
    tensorE (K := K) m n (stringVec m n k (l + 1)) =
      (((l : K) + 1) * (((m + n - 2 * k : ℕ) : K) - l)) • stringVec m n k l
```

lemma stringVec_ne_zero [source]

```
lemma stringVec_ne_zero {l : ℕ} (hl : l ≤ m + n - 2 * k) :
    stringVec (K := K) m n k l ≠ 0
```

lemma stringVec_eq_zero [source]

```
lemma stringVec_eq_zero {l : ℕ} (hl : m + n - 2 * k < l) :
    stringVec (K := K) m n k l = 0
```

lemma tensorE_stringVec_zero [source]

```
lemma tensorE_stringVec_zero : tensorE (K := K) m n (stringVec m n k 0) = 0
```

def separator [source]

The operator `M (2EF + 2FE + H²) + H` with `M = 2(m + n) + 1`. The bracket is a multiple of the Casimir element, which distinguishes the strings; the weight `H` distinguishes the vectors within a string.

```
def separator (m n : ℕ) : Module.End K (StandardSl2Module K m ⊗[K] StandardSl2Module K n)
```

def sepVal [source]

The eigenvalue of `separator` on `stringVec m n k l`.

```
def sepVal (m n k l : ℕ) : K
```

lemma separator_stringVec [source]

```
lemma separator_stringVec {k : ℕ} (hmn : 0 < m + n) (hkm : k ≤ m) (hkn : k ≤ n) (l : ℕ) :
    separator (K := K) m n (stringVec m n k l) =
      sepVal (K := K) m n k l • stringVec m n k l
```

abbrev StringIndex [source]

Index set of the lowering strings: `k ≤ min m n` and `l ≤ m + n - 2k`.

```
abbrev StringIndex (m n : ℕ) : Type
```

instance Nonempty (StringIndex m n) [source]

```
instance : Nonempty (StringIndex m n)
```

lemma le_left [source]

```
lemma StringIndex.le_left (i : StringIndex m n) : (i.1 : ℕ) ≤ m
```

lemma le_right [source]

```
lemma StringIndex.le_right (i : StringIndex m n) : (i.1 : ℕ) ≤ n
```

lemma sepVal_int_injective [source]

Integer core of `sepVal_injective`: with `M = 2s + 1`, the value `M N (N + 2) + N - 2l` determines `N` and `l`, provided `N ≤ s` and `l ≤ N`.

```
lemma sepVal_int_injective {s N N' l l' : ℕ} (hN : N ≤ s) (hN' : N' ≤ s) (hl : l ≤ N)
    (hl' : l' ≤ N')
    (h : (2 * s + 1 : ℤ) * (N * (N + 2)) + N - 2 * l =
      (2 * s + 1 : ℤ) * (N' * (N' + 2)) + N' - 2 * l') : N = N' ∧ l = l'
```

lemma sepVal_injective [source]

The eigenvalues of `separator` on the lowering strings are pairwise distinct.

```
lemma sepVal_injective :
    Function.Injective (fun i : StringIndex m n => sepVal (K := K) m n i.1 i.2)
```

lemma stringVec_linearIndependent [source]

The lowering strings of the highest-weight vectors are linearly independent.

```
lemma stringVec_linearIndependent (hmn : 0 < m + n) :
    LinearIndependent K (fun i : StringIndex m n => stringVec (K := K) m n i.1 i.2)
```

lemma card_stringIndex [source]

```
lemma card_stringIndex : Fintype.card (StringIndex m n) = (m + 1) * (n + 1)
```

lemma finrank_standardSl2Module_tensor [source]

```
lemma finrank_standardSl2Module_tensor :
    Module.finrank K (StandardSl2Module K m ⊗[K] StandardSl2Module K n) = (m + 1) * (n + 1)
```

def tensorStringBasis [source]

The basis of `V(m) ⊗ V(n)` formed by the lowering strings of the highest-weight vectors.

```
def tensorStringBasis (hmn : 0 < m + n) :
    Module.Basis (StringIndex m n) K (StandardSl2Module K m ⊗[K] StandardSl2Module K n)
```

lemma tensorStringBasis_apply [source]

```
lemma tensorStringBasis_apply (hmn : 0 < m + n) (i : StringIndex m n) :
    tensorStringBasis (K := K) hmn i = stringVec m n i.1 i.2
```

abbrev CGSummand [source]

The summand `V(m + n - 2k)` of the Clebsch–Gordan decomposition.

```
abbrev CGSummand (K : Type*) (m n : ℕ) (k : Fin (min m n + 1)) : Type _
```

abbrev CGSum [source]

The direct sum `⨁_k V(m + n - 2k)` of the Clebsch–Gordan decomposition.

```
abbrev CGSum (K : Type*) [Field K] (m n : ℕ) : Type _
```

def sumVec [source]

The coordinate vector `v_l` of the `k`-th summand of `CGSum`.

```
def sumVec (m n : ℕ) (k : Fin (min m n + 1)) (l : ℕ) : CGSum K m n
```

def sumEnd [source]

The endomorphism of `CGSum` acting by `A k` on the `k`-th summand.

```
def sumEnd (A : ∀ k, Module.End K (CGSummand K m n k)) : Module.End K (CGSum K m n)
```

lemma sumEnd_lof [source]

```
lemma sumEnd_lof (A : ∀ k, Module.End K (CGSummand K m n k)) (k : Fin (min m n + 1))
    (x : CGSummand K m n k) :
    sumEnd A (DirectSum.lof K _ (CGSummand K m n) k x) =
      DirectSum.lof K _ (CGSummand K m n) k (A k x)
```

lemma sumVec_linearIndependent [source]

```
lemma sumVec_linearIndependent :
    LinearIndependent K (fun i : StringIndex m n => sumVec (K := K) m n i.1 i.2)
```

def sumBasis [source]

The basis of `CGSum` formed by the coordinate vectors of the summands.

```
def sumBasis (m n : ℕ) : Module.Basis (StringIndex m n) K (CGSum K m n)
```

lemma sumBasis_apply [source]

```
lemma sumBasis_apply (i : StringIndex m n) : sumBasis (K := K) m n i = sumVec m n i.1 i.2
```

def cgEquiv [source]

The Clebsch–Gordan equivalence `V(m) ⊗ V(n) ≃ ⨁_k V(m + n - 2k)`, sending the `l`-th vector of the `k`-th lowering string to the coordinate vector `v_l` of the `k`-th summand.

```
def cgEquiv (hmn : 0 < m + n) :
    (StandardSl2Module K m ⊗[K] StandardSl2Module K n) ≃ₗ[K] CGSum K m n
```

lemma cgEquiv_stringVec [source]

`cgEquiv` on the lowering strings, for every `l : ℕ`.

```
lemma cgEquiv_stringVec (hmn : 0 < m + n) (k : Fin (min m n + 1)) (l : ℕ) :
    cgEquiv (K := K) hmn (stringVec m n k l) = sumVec m n k l
```

lemma cgEquiv_tensorF [source]

`cgEquiv` intertwines the diagonal lowering operator with `ρF` on each summand.

```
lemma cgEquiv_tensorF (hmn : 0 < m + n) (x : StandardSl2Module K m ⊗[K] StandardSl2Module K n)
    :
    cgEquiv hmn (tensorF m n x) = sumEnd (fun _ => ρF) (cgEquiv hmn x)
```

lemma cgEquiv_tensorH [source]

cgEquiv intertwines the diagonal weight operator with ρH on each summand.

```
lemma cgEquiv_tensorH (hmn : 0 < m + n) (x : StandardSl2Module K m ⊗[K] StandardSl2Module K n)
    :
    cgEquiv hmn (tensorH m n x) = sumEnd (fun _ => ρH) (cgEquiv hmn x)
```

lemma cgEquiv_tensorE [source]

cgEquiv intertwines the diagonal raising operator with ρE on each summand.

```
lemma cgEquiv_tensorE (hmn : 0 < m + n) (x : StandardSl2Module K m ⊗[K] StandardSl2Module K n)
    :
    cgEquiv hmn (tensorE m n x) = sumEnd (fun _ => ρE) (cgEquiv hmn x)
```

2.347 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.TensorHighestWeight

PhyslibAlpha/Mathematics/RepresentationTheory/Sl2/TensorHighestWeight.lean on GitHub.

def diagEnd [source]

The diagonal action A ⊗ 1 + 1 ⊗ B of two endomorphisms on M ⊗ N.

```
def diagEnd (A : Module.End K M) (B : Module.End K N) : Module.End K (M ⊗[K] N)
```

lemma diagEnd_tmul [source]

```
lemma diagEnd_tmul (A : Module.End K M) (B : Module.End K N) (x : M) (y : N) :
    diagEnd A B (x ⊗ₜ y) = A x ⊗ₜ y + x ⊗ₜ B y
```

lemma lie_diagEnd [source]

The diagonal action respects commutators.

```
lemma lie_diagEnd (A A' : Module.End K M) (B B' : Module.End K N) :
    ⁅diagEnd A B, diagEnd A' B'⁆ = diagEnd ⁅A, A'⁆ ⁅B, B'⁆
```

lemma diagEnd_smul [source]

```
lemma diagEnd_smul (c : K) (A : Module.End K M) (B : Module.End K N) :
    diagEnd (c • A) (c • B) = c • diagEnd A B
```

lemma diagEnd_add [source]

The diagonal action is additive in both endomorphisms.

```
lemma diagEnd_add (A A' : Module.End K M) (B B' : Module.End K N) :
    diagEnd (A + A') (B + B') = diagEnd A B + diagEnd A' B'
```

lemma diagEnd_sub [source]

The diagonal action respects differences.

```
lemma diagEnd_sub (A A' : Module.End K M) (B B' : Module.End K N) :
    diagEnd (A - A') (B - B') = diagEnd A B - diagEnd A' B'
```

def vNat [source]

The coordinate vector `v_i` of `StandardSl2Module K n`, indexed by a natural number; it is zero for `i > n`.

```
def StandardSl2Module.vNat (n i : ℕ) : StandardSl2Module K n
```

lemma vNat_apply [source]

```
lemma vNat_apply (i : ℕ) (k : Fin (n + 1)) :
    vNat (K := K) n i k = if (k : ℕ) = i then 1 else 0
```

lemma vNat_eq_v [source]

```
lemma vNat_eq_v {i : ℕ} (hi : i ≤ n) : vNat (K := K) n i = v (i, by omega)
```

lemma vNat_of_lt [source]

```
lemma vNat_of_lt {i : ℕ} (hi : n < i) : vNat (K := K) n i = 0
```

lemma ρH_vNat [source]

```
lemma ρH_vNat (i : ℕ) :
    (ρH : Module.End K (StandardSl2Module K n)) (vNat n i) = ((n : K) - 2 * i) • vNat n i
```

lemma ρF_vNat [source]

```
lemma ρF_vNat (i : ℕ) :
    (ρF : Module.End K (StandardSl2Module K n)) (vNat n i) = vNat n (i + 1)
```

lemma ρE_vNat_zero [source]

```
lemma ρE_vNat_zero :
    (ρE : Module.End K (StandardSl2Module K n)) (vNat n 0) = 0
```

lemma ρE_vNat_succ [source]

```
lemma ρE_vNat_succ [CharZero K] (i : ℕ) :
    (ρE : Module.End K (StandardSl2Module K n)) (vNat n (i + 1)) =
      (((i : K) + 1) * ((n : K) - i)) • vNat n i
```

def tensorE [source]

The diagonal raising operator on `V(m) ⊗ V(n)`.

```
def tensorE (m n : ℕ) : Module.End K (StandardSl2Module K m ⊗[K] StandardSl2Module K n)
```

def tensorF [source]

The diagonal lowering operator on `V(m) ⊗ V(n)`.

```
def tensorF (m n : ℕ) : Module.End K (StandardSl2Module K m ⊗[K] StandardSl2Module K n)
```

def tensorH [source]

The diagonal weight operator on `V(m) ⊗ V(n)`.

```
def tensorH (m n : ℕ) : Module.End K (StandardSl2Module K m ⊗[K] StandardSl2Module K n)
```

def tensorCoord [source]

The coefficient of `v_i ⊗ v_j` in an element of `V(m) ⊗ V(n)`.

```
def tensorCoord (m n : ℕ) (i : Fin (m + 1)) (j : Fin (n + 1)) :
    StandardSl2Module K m ⊗[K] StandardSl2Module K n →ₗ[K] K
```

lemma tensorCoord_tmul [source]

```
lemma tensorCoord_tmul (i : Fin (m + 1)) (j : Fin (n + 1)) (x : StandardSl2Module K m)
    (y : StandardSl2Module K n) : tensorCoord m n i j (x ⊗ₜ y) = x i * y j
```

lemma tensorH_tmul_vNat [source]

The diagonal weight operator on a product of coordinate vectors.

```
lemma tensorH_tmul_vNat (a b : ℕ) :
    tensorH (K := K) m n (vNat m a ⊗ₜ vNat n b) =
      ((m : K) + n - 2 * (a + b)) • (vNat m a ⊗ₜ vNat n b)
```

lemma isSl2Triple_tensor [source]

The diagonal operators on `V(m) ⊗ V(n)` form an `sl₂` triple when `m + n > 0`.

```
lemma isSl2Triple_tensor [CharZero K] (hmn : 0 < m + n) :
    IsSl2Triple (tensorH (K := K) m n) (tensorE m n) (tensorF m n) where
```

def cgCoeff [source]

The coefficient of `v_a ⊗ v_{k-a}` in the highest-weight vector of weight `m + n - 2k` of `V(m) ⊗ V(n)`: `(-1)^a (k choose a) (n-k+a)! (m-a)!`.

```
def cgCoeff (m n k a : ℕ) : K
```

lemma cgCoeff_rec [source]

The two-term recursion of the coefficients that makes the raising operator vanish.

```
lemma cgCoeff_rec {j a : ℕ} (hjm : j + 1 ≤ m) (hjn : j + 1 ≤ n) (ha : a ≤ j) :
    cgCoeff (K := K) m n (j + 1) (a + 1) * (((a : K) + 1) * ((m : K) - a)) +
      cgCoeff m n (j + 1) a * ((((j - a : ℕ) : K) + 1) * ((n : K) - ((j - a : ℕ) : K))) = 0
```

def hwVec [source]

The highest-weight vector of weight `m + n - 2k` in `V(m) ⊗ V(n)`.

```
def hwVec (m n k : ℕ) : StandardSl2Module K m ⊗[K] StandardSl2Module K n
```

lemma tensorH_hwVec [source]

`hwVec m n k` has weight `m + n - 2k`.

```
lemma tensorH_hwVec (k : ℕ) :
    tensorH (K := K) m n (hwVec m n k) = ((m : K) + n - 2 * k) • hwVec m n k
```

lemma tensorE_hwVec [source]

`hwVec m n k` is killed by the diagonal raising operator when `k ≤ min m n`.

```
lemma tensorE_hwVec [CharZero K] {k : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) :
    tensorE (K := K) m n (hwVec m n k) = 0
```

lemma hwVec_ne_zero [source]

`hwVec m n k` is nonzero for `k ≤ n`: its `v_0 ⊗ v_k` coefficient is `(n-k)! m!`.

```
lemma hwVec_ne_zero [CharZero K] {k : ℕ} (hkn : k ≤ n) :
    hwVec (K := K) m n k ≠ 0
```

lemma hwVec_hasPrimitiveVectorWith [source]

`hwVec m n k` is a primitive vector of weight `m + n - 2k` for the diagonal action.

```
lemma hwVec_hasPrimitiveVectorWith [CharZero K] {k : ℕ} (hmn : 0 < m + n) (hkm : k ≤ m)
    (hkn : k ≤ n) :
    (isSl2Triple_tensor (K := K) hmn).HasPrimitiveVectorWith (hwVec (K := K) m n k)
      ((m + n - 2 * k : ℕ) : K) where
```

lemma tensorF_pow_hwVec_ne_zero [source]

The first `m + n - 2k + 1` members of the lowering string of `hwVec m n k` are nonzero.

```
lemma tensorF_pow_hwVec_ne_zero [CharZero K] {k l : ℕ} (hmn : 0 < m + n) (hkm : k ≤ m)
    (hkn : k ≤ n) (hl : l ≤ m + n - 2 * k) :
    (tensorF (K := K) m n ^ l) (hwVec m n k) ≠ 0
```

lemma tensorF_pow_hwVec_eq_zero [source]

The lowering string of `hwVec m n k` ends after `m + n - 2k + 1` steps.

```
lemma tensorF_pow_hwVec_eq_zero [CharZero K] {k : ℕ} (hmn : 0 < m + n) (hkm : k ≤ m)
    (hkn : k ≤ n) :
    (tensorF (K := K) m n ^ (m + n - 2 * k + 1)) (hwVec m n k) = 0
```

2.348 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.TensorIntertwiner

PhyslibAlpha/Mathematics/RepresentationTheory/Sl2/TensorIntertwiner.lean on GitHub.

structure IsTensorIntertwiner [source]

A linear map `V(m) ⊗ V(n) → V(p)` is a tensor intertwiner if it carries the diagonal raising, lowering and weight operators on `V(m) ⊗ V(n)` to those on `V(p)`.

```
structure IsTensorIntertwiner
    (φ : StandardSl2Module K m ⊗[K] StandardSl2Module K n →ₗ[K] StandardSl2Module K p) : Prop
    where
  comm_E : φ ∘ₗ tensorE m n = ρE ∘ₗ φ
  comm_F : φ ∘ₗ tensorF m n = ρF ∘ₗ φ
  comm_H : φ ∘ₗ tensorH m n = ρH ∘ₗ φ
```

lemma smul [source]

Scalar multiples of tensor intertwiners are tensor intertwiners.

```
lemma smul (hφ : IsTensorIntertwiner φ) (c : K) : IsTensorIntertwiner (c • φ) where
```

lemma add [source]

Sums of tensor intertwiners are tensor intertwiners.

```
lemma add (hφ : IsTensorIntertwiner φ) (hψ : IsTensorIntertwiner ψ) :
    IsTensorIntertwiner (φ + ψ) where
```

lemma component_sumEnd [source]

The component of a direct sum commutes with a summand-wise endomorphism.

```
lemma component_sumEnd (A : ∀ j, Module.End K (CGSummand K m n j)) (j : Fin (min m n + 1))
    (y : CGSum K m n) :
    DirectSum.component K _ (CGSummand K m n) j (sumEnd A y) =
      A j (DirectSum.component K _ (CGSummand K m n) j y)
```

def cgProj [source]

The projection of `V(m) ⊗ V(n)` onto the summand `V(m + n - 2j)` of the Clebsch–Gordan decomposition.

```
def cgProj (hmn : 0 < m + n) (j : Fin (min m n + 1)) :
    StandardSl2Module K m ⊗[K] StandardSl2Module K n →ₗ[K]
      StandardSl2Module K (m + n - 2 * (j : ℕ))
```

lemma cgProj_apply [source]

```
lemma cgProj_apply (hmn : 0 < m + n) (j : Fin (min m n + 1))
    (x : StandardSl2Module K m ⊗[K] StandardSl2Module K n) :
    cgProj hmn j x = DirectSum.component K _ (CGSummand K m n) j (cgEquiv hmn x)
```

lemma cgProj_isTensorIntertwiner [source]

The projections are tensor intertwiners.

```
lemma cgProj_isTensorIntertwiner (hmn : 0 < m + n) (j : Fin (min m n + 1)) :
    IsTensorIntertwiner (cgProj (K := K) hmn j) where
```

lemma cgProj_cgEquiv_symm_lof [source]

The projection sends the preimage of the highest-weight vector of the `j`-th summand to that vector; in particular it is not zero.

```
lemma cgProj_cgEquiv_symm_lof (hmn : 0 < m + n) (j : Fin (min m n + 1))
    (z : CGSummand K m n j) :
    cgProj hmn j ((cgEquiv hmn).symm (DirectSum.lof K _ (CGSummand K m n) j z)) = z
```

lemma cgProj_ne_zero [source]

```
lemma cgProj_ne_zero (hmn : 0 < m + n) (j : Fin (min m n + 1)) :
    cgProj (K := K) hmn j ≠ 0
```

lemma cgEquiv_symm_sumEnd_E [source]

```
lemma cgEquiv_symm_sumEnd_E (y : CGSum K m n) :
    (cgEquiv hmn).symm (sumEnd (fun _ => ρE) y) = tensorE m n ((cgEquiv hmn).symm y)
```

lemma cgEquiv_symm_sumEnd_F [source]

```
lemma cgEquiv_symm_sumEnd_F (y : CGSum K m n) :
    (cgEquiv hmn).symm (sumEnd (fun _ => ρF) y) = tensorF m n ((cgEquiv hmn).symm y)
```

lemma cgEquiv_symm_sumEnd_H [source]

```
lemma cgEquiv_symm_sumEnd_H (y : CGSum K m n) :
    (cgEquiv hmn).symm (sumEnd (fun _ => ρH) y) = tensorH m n ((cgEquiv hmn).symm y)
```

def cgRestrict [source]

The restriction of a linear map on `V(m) ⊗ V(n)` to the `j`-th summand of the Clebsch–Gordan decomposition.

```
def cgRestrict
    (φ : StandardSl2Module K m ⊗[K] StandardSl2Module K n →ₗ[K] StandardSl2Module K p)
    (j : Fin (min m n + 1)) : CGSummand K m n j →ₗ[K] StandardSl2Module K p
```

lemma cgRestrict_isIntertwiner [source]

The restriction of a tensor intertwiner to a summand is an intertwiner.

```
lemma cgRestrict_isIntertwiner (hφ : IsTensorIntertwiner φ) (j : Fin (min m n + 1)) :
    IsIntertwiner (cgRestrict hmn φ j) where
```

lemma eq_of_cgRestrict_eq [source]

A linear map on `V(m) ⊗ V(n)` is determined by its restrictions to the summands.

```
lemma eq_of_cgRestrict_eq (hmn : 0 < m + n) {M : Type*} [AddCommGroup M] [Module K M]
    {α β : StandardSl2Module K m ⊗[K] StandardSl2Module K n →ₗ[K] M}
    (h : ∀ j, α ∘ₗ (cgEquiv hmn).symm.toLinearMap ∘ₗ DirectSum.lof K _ (CGSummand K m n) j =
      β ∘ₗ (cgEquiv hmn).symm.toLinearMap ∘ₗ DirectSum.lof K _ (CGSummand K m n) j) :
    α = β
```

lemma eq_zero_of_forall_ne [source]

***Selection rule.** If `p` is not of the form `m + n - 2j` with `j ≤ min m n`, every tensor intertwiner `V(m) ⊗ V(n) → V(p)` is zero.*

```
lemma eq_zero_of_forall_ne (hmn : 0 < m + n)
    {φ : StandardSl2Module K m ⊗[K] StandardSl2Module K n →ₗ[K] StandardSl2Module K p}
    (hφ : IsTensorIntertwiner φ) (hp : ∀ j : Fin (min m n + 1), m + n - 2 * (j : ℕ) ≠ p) :
    φ = 0
```

lemma exists_eq_smul_cgProj [source]

Every tensor intertwiner onto the summand `V(m + n - 2j)` is a scalar multiple of the projection `cgProj`.

```
lemma exists_eq_smul_cgProj (hmn : 0 < m + n) {j : Fin (min m n + 1)}
    {φ : StandardSl2Module K m ⊗[K] StandardSl2Module K n →ₗ[K]
      StandardSl2Module K (m + n - 2 * (j : ℕ))}
    (hφ : IsTensorIntertwiner φ) : ∃ c : K, φ = c • cgProj hmn j
```

2.349 PhyslibAlpha.Mathematics.RepresentationTheory.Sl2.Weights

PhyslibAlpha/Mathematics/RepresentationTheory/Sl2/Weights.lean on GitHub.

def weightSpace [source]

The eigenspace of the Cartan operator `h` with eigenvalue `n`.

```
def weightSpace (_t : IsSl2Triple h e f) (n : K) : Submodule K M
```

lemma e_mapsTo_weightSpace [source]

The raising operator maps weight `n` into weight `n + 2`.

```
lemma e_mapsTo_weightSpace (t : IsSl2Triple h e f) (n : K) :
    ∀ v ∈ weightSpace (M := M) t n, LieModule.toEnd K L M e v ∈ weightSpace t (n + 2)
```

lemma f_mapsTo_weightSpace [source]

`f` lowers the weight by `2`. Dual to `e_mapsTo_weightSpace`.

```
lemma f_mapsTo_weightSpace (t : IsSl2Triple h e f) (n : K) :
    ∀ v ∈ weightSpace (M := M) t n, LieModule.toEnd K L M f v ∈ weightSpace t (n - 2)
```

lemma repr_apply_diag [source]

If a basis diagonalises `T`, then `T` multiplies the `i`-th coordinate by `μ i`.

```
lemma repr_apply_diag {ι : Type*} (b : Module.Basis ι K M) (T : Module.End K M)
    (μ : ι → K) (hT : ∀ j, T (b j) = μ j • b j) (i : ι) (w : M) :
    b.repr (T w) i = μ i * b.repr w i
```

lemma string_coord_eq_zero [source]

A weight vector has vanishing string coordinates at every other weight.

```
lemma string_coord_eq_zero {n : K} {w : M} (hw : w ∈ weightSpace (M := M) t n)
    (i : Fin (N + 1)) (hi : (N : K) - 2 * (i : ℕ) ≠ n) : (stringBasis P).repr w i = 0
```

lemma weightSpace_eq_bot_of_ne [source]

The weight space is trivial at a value that is not a weight of the string.

```
lemma weightSpace_eq_bot_of_ne {n : K} (hn : ∀ i : Fin (N + 1), (N : K) - 2 * (i : ℕ) ≠ n) :
    weightSpace (M := M) t n = ⊥
```

lemma weightSpace_finrank_le_one_of_eq [source]

The weight space of a weight of the string has dimension at most one.

```
lemma weightSpace_finrank_le_one_of_eq {n : K} (i₀ : Fin (N + 1))
    (hi₀ : (N : K) - 2 * (i₀ : ℕ) = n) : Module.finrank K (weightSpace (M := M) t n) ≤ 1
```

lemma weightSpace_finrank_le_one [source]

Every Cartan weight space of a finite-dimensional irreducible $\mathfrak{sl}_2$ module has dimension at most one. The lowering-string basis has pairwise distinct weights.

```
lemma weightSpace_finrank_le_one [IsAlgClosed K] [CharZero K] [FiniteDimensional K M]
    (t : IsSl2Triple h e f) [LieModule.IsIrreducible K ↥(t.toLieSubalgebra (R := K)) M]
    (n : K) : Module.finrank K (weightSpace (M := M) t n) ≤ 1
```

lemma lie_e_eq_zero_of_top_weight [source]

A vector at a weight with empty weight space two steps above is killed by e.

```
lemma lie_e_eq_zero_of_top_weight {n : K} {v : M} (t : IsSl2Triple h e f)
    (hv : v ∈ weightSpace (M := M) t n) (hn : weightSpace (M := M) t (n + 2) = ⊥) :
    ⁅e, v⁆ = 0
```

lemma finrank_of_top_weight [source]

A top weight p of a finite-dimensional irreducible module fixes the dimension.

```
lemma finrank_of_top_weight [CharZero K] [FiniteDimensional K M]
    (t : IsSl2Triple h e f) [LieModule.IsIrreducible K ↥(t.toLieSubalgebra (R := K)) M]
    {p : ℕ} (hp : weightSpace (M := M) t (p : K) ≠ ⊥)
    (hp' : weightSpace (M := M) t ((p : K) + 2) = ⊥) : Module.finrank K M = p + 1
```

lemma weightSpace_highest_unique [source]

A nonzero weight space with no weight space two steps above it determines the highest weight of a finite-dimensional irreducible module.

```
lemma weightSpace_highest_unique [CharZero K] [FiniteDimensional K M]
    (t : IsSl2Triple h e f) [LieModule.IsIrreducible K ↥(t.toLieSubalgebra (R := K)) M]
    {m n : ℕ} (hm : weightSpace (M := M) t (m : K) ≠ ⊥)
    (hm' : weightSpace (M := M) t ((m : K) + 2) = ⊥)
```

```
    (hn : weightSpace (M := M) t (n : K) ≠ ⊥)
    (hn' : weightSpace (M := M) t ((n : K) + 2) = ⊥) : m = n
```

2.350 PhyslibAlpha.Mathematics.SpecialFunctions.Laguerre

PhyslibAlpha/Mathematics/SpecialFunctions/Laguerre.lean on GitHub.

def laguerreCoeff [source]

The coefficient `(-1)^i C(N + α, N - i) / i!` of `x^i` in `L_N^(α)`.

```
def laguerreCoeff (N α i : ℕ) : ℝ
```

def laguerre [source]

The generalised Laguerre polynomial `L_N^(α)(x) = Σ_i (-1)^i C(N + α, N - i) x^i / i!`.

```
def laguerre (N α : ℕ) : ℝ[X]
```

lemma eval_laguerre [source]

```
lemma eval_laguerre (x : ℝ) :
    (laguerre N α).eval x = ∑ i ∈ Finset.range (N + 1), laguerreCoeff N α i * x ^ i
```

lemma eval_laguerre_zero [source]

```
lemma eval_laguerre_zero : (laguerre N α).eval 0 = ((N + α).choose N : ℝ)
```

lemma eval_derivative_laguerre [source]

```
lemma eval_derivative_laguerre (x : ℝ) :
    (derivative (laguerre N α)).eval x =
      ∑ i ∈ Finset.range (N + 1), laguerreCoeff N α i * ((i : ℝ) * x ^ (i - 1))
```

lemma eval_derivative_derivative_laguerre [source]

```
lemma eval_derivative_derivative_laguerre (x : ℝ) :
    (derivative (derivative (laguerre N α))).eval x =
      ∑ i ∈ Finset.range (N + 1),
        laguerreCoeff N α i * ((i : ℝ) * ((i - 1 : ℕ) : ℝ) * x ^ (i - 1 - 1))
```

lemma laguerreCoeff_succ_mul [source]

The two-term recursion of the coefficients.

```
lemma laguerreCoeff_succ_mul {i : ℕ} (hi : i < N) :
    laguerreCoeff N α (i + 1) * (((i : ℝ) + 1) * ((i : ℝ) + 1 + α)) =
      -((N : ℝ) - i) * laguerreCoeff N α i
```

lemma laguerre_ode [source]

***Laguerre's differential equation** `x L'' + (α + 1 - x) L' + N L = 0`.*

```
lemma laguerre_ode (x : ℝ) :
    x * (derivative (derivative (laguerre N α))).eval x +
      ((α : ℝ) + 1 - x) * (derivative (laguerre N α)).eval x +
        (N : ℝ) * (laguerre N α).eval x = 0
```

lemma sum_choose_mul_choose_neg [source]

`Σ_i (-1)^i C(N + α, N - i) C(q + i, i) = C(N + α - q - 1, N)`, the Chu–Vandermonde identity with a negative upper index.

```
lemma sum_choose_mul_choose_neg (q : ℕ) :
    ∑ i ∈ Finset.range (N + 1), (-1 : ℤ) ^ i * ((N + α).choose (N - i) : ℤ) * ((q + i).choose i
    ) =
      Ring.choose ((N + α : ℤ) - q - 1) N
```

lemma sum_laguerreCoeff_mul_factorial [source]

***Moments.** `Σ_i c_i (q + i)! = q! C(N + α - q - 1, N)`, the integral of `x^q e^(-x) L_N^(α)` with the integer binomial coefficient `Ring.choose`.*

```
lemma sum_laguerreCoeff_mul_factorial (q : ℕ) :
    ∑ i ∈ Finset.range (N + 1), laguerreCoeff N α i * ((q + i).factorial : ℝ) =
      (q.factorial : ℝ) * ((Ring.choose ((N + α : ℤ) - q - 1) N : ℤ) : ℝ)
```

lemma ringChoose_natCast_of_lt [source]

```
lemma ringChoose_natCast_of_lt {t : ℕ} (h : t < N) : Ring.choose (t : ℤ) N = 0
```

lemma ringChoose_neg_one [source]

```
lemma ringChoose_neg_one : Ring.choose (-1 : ℤ) N = (-1) ^ N
```

lemma ringChoose_neg_two [source]

```
lemma ringChoose_neg_two : Ring.choose (-2 : ℤ) N = (-1) ^ N * (N + 1)
```

lemma integrableOn_pow_mul_exp_neg [source]

```
lemma integrableOn_pow_mul_exp_neg (q : ℕ) :
    IntegrableOn (fun x : ℝ => x ^ q * Real.exp (-x)) (Ioi 0)
```

lemma integral_pow_mul_exp_neg [source]

`∫₀^∞ x^q e^(-x) dx = q!`.

```
lemma integral_pow_mul_exp_neg (q : ℕ) :
    ∫ x in Ioi (0 : ℝ), x ^ q * Real.exp (-x) = q.factorial
```

lemma integral_pow_mul_exp_neg_mul_laguerre_sq [source]

∫₀^∞ x^e e^(-x) L(x)² dx = Σ_j c_j Σ_i c_i (e + j + i)!.

```
lemma integral_pow_mul_exp_neg_mul_laguerre_sq (e : ℕ) :
    ∫ x in Ioi (0 : ℝ), x ^ e * Real.exp (-x) * ((laguerre N α).eval x) ^ 2 =
      ∑ j ∈ Finset.range (N + 1), laguerreCoeff N α j *
        ∑ i ∈ Finset.range (N + 1), laguerreCoeff N α i * ((e + j + i).factorial : ℝ)
```

lemma integral_pow_mul_exp_neg_mul_laguerre_sq_eq [source]

The integrals through the moments.

```
lemma integral_pow_mul_exp_neg_mul_laguerre_sq_eq (e : ℕ) :
    ∫ x in Ioi (0 : ℝ), x ^ e * Real.exp (-x) * ((laguerre N α).eval x) ^ 2 =
      ∑ j ∈ Finset.range (N + 1), laguerreCoeff N α j * (((e + j).factorial : ℝ) *
        ((Ring.choose ((N + α : ℤ) - (e + j : ℕ) - 1) N : ℤ) : ℝ))
```

lemma neg_one_pow_mul_self [source]

```
lemma neg_one_pow_mul_self (n : ℕ) : (-1 : ℝ) ^ n * (-1) ^ n = 1
```

lemma integral_laguerre_sq_alpha [source]

∫₀^∞ x^α e^(-x) L_N^(α)(x)² dx = (N + α)!/N!.

```
lemma integral_laguerre_sq_alpha :
    ∫ x in Ioi (0 : ℝ), x ^ α * Real.exp (-x) * ((laguerre N α).eval x) ^ 2 =
      ((N + α).factorial : ℝ) / N.factorial
```

lemma integral_laguerre_sq_alpha_add_one [source]

∫₀^∞ x^(α+1) e^(-x) L_N^(α)(x)² dx = (N + α)!/N! · (2N + α + 1).

```
lemma integral_laguerre_sq_alpha_add_one :
    ∫ x in Ioi (0 : ℝ), x ^ (α + 1) * Real.exp (-x) * ((laguerre N α).eval x) ^ 2 =
      ((N + α).factorial : ℝ) / N.factorial * (2 * N + α + 1)
```

lemma integral_laguerre_sq_alpha_sub_one [source]

∫₀^∞ x^(α-1) e^(-x) L_N^(α)(x)² dx = (N + α)!/(N! α).

```
lemma integral_laguerre_sq_alpha_sub_one (hα : 1 ≤ α) :
    ∫ x in Ioi (0 : ℝ), x ^ (α - 1) * Real.exp (-x) * ((laguerre N α).eval x) ^ 2 =
      ((N + α).factorial : ℝ) / (N.factorial * α)
```

lemma integral_laguerre_sq_alpha_sub_two [source]

∫₀^∞ x^(α-2) e^(-x) L_N^(α)(x)² dx = (N + α)! (2N + α + 1)/(N! (α - 1) α (α + 1)).

```
lemma integral_laguerre_sq_alpha_sub_two (hα : 2 ≤ α) :
    ∫ x in Ioi (0 : ℝ), x ^ (α - 2) * Real.exp (-x) * ((laguerre N α).eval x) ^ 2 =
      ((N + α).factorial : ℝ) * (2 * N + α + 1) /
        (N.factorial * (((α : ℝ) - 1) * α * (α + 1)))
```

2.351 PhyslibAlpha.Mathematics.SpecialFunctions.Walsh

PhyslibAlpha/Mathematics/SpecialFunctions/Walsh.lean on GitHub.

def walshSign [source]

The sign `(-1) ^ (j * k)` for `j k : Fin 2`, as a complex number.

```
def walshSign (j k : Fin 2) : ℂ
```

lemma walshSign_zero_left [source]

The sign `walshSign 0 k` is `1`.

```
@[simp]
lemma walshSign_zero_left (k : Fin 2) : walshSign 0 k = 1
```

lemma walshSign_zero_right [source]

The sign `walshSign j 0` is `1`.

```
@[simp]
lemma walshSign_zero_right (j : Fin 2) : walshSign j 0 = 1
```

lemma walshSign_one_one [source]

The sign `walshSign 1 1` is `-1`.

```
@[simp]
lemma walshSign_one_one : walshSign 1 1 = -1
```

lemma walshSign_comm [source]

The sign `(-1) ^ (j * k)` is symmetric in `j` and `k`.

```
lemma walshSign_comm (j k : Fin 2) : walshSign j k = walshSign k j
```

lemma walshSign_mul_self [source]

A sign squares to one.

```
lemma walshSign_mul_self (j k : Fin 2) : walshSign j k * walshSign j k = 1
```

lemma conj_walshSign [source]

A sign equals its complex conjugate.

```
lemma conj_walshSign (j k : Fin 2) : starRingEnd ℂ (walshSign j k) = walshSign j k
```

lemma norm_walshSign [source]

A sign has unit norm.

```
lemma norm_walshSign (j k : Fin 2) : ‖walshSign j k‖ = 1
```

lemma sum_walshSign_mul_walshSign [source]

Orthogonality of the signs on `Fin 2`.

```
lemma sum_walshSign_mul_walshSign (a b : Fin 2) :
    ∑ k, walshSign a k * walshSign b k = if a = b then 2 else 0
```

def walsh [source]

The Walsh character `∏ i, (-1) ^ (x i * z i)` on `Fin n → Fin 2`.

```
def walsh (x z : Fin n → Fin 2) : ℂ
```

lemma walsh_eq [source]

The Walsh character is the product of the signs of the components.

```
lemma walsh_eq (x z : Fin n → Fin 2) : walsh x z = ∏ i, walshSign (x i) (z i)
```

lemma walsh_comm [source]

The Walsh character `walsh x z` is symmetric in `x` and `z`.

```
lemma walsh_comm (x z : Fin n → Fin 2) : walsh x z = walsh z x
```

lemma walsh_mul_self [source]

A Walsh character squares to one.

```
lemma walsh_mul_self (x z : Fin n → Fin 2) : walsh x z * walsh x z = 1
```

lemma conj_walsh [source]

A Walsh character equals its complex conjugate.

```
lemma conj_walsh (x z : Fin n → Fin 2) : starRingEnd ℂ (walsh x z) = walsh x z
```

lemma norm_walsh [source]

A Walsh character has unit norm.

```
lemma norm_walsh (x z : Fin n → Fin 2) : ‖walsh x z‖ = 1
```

lemma sum_walsh_mul_walsh [source]

Orthogonality of the Walsh characters.

```
lemma sum_walsh_mul_walsh (x y : Fin n → Fin 2) :
    ∑ z, walsh x z * walsh y z = if x = y then 2 ^ n else 0
```

def walshVec [source]

The rescaled Walsh vector `z ↦ (√(2 ^ n))⁻¹ * walsh x z` in `EuclideanSpace ℂ (Fin n → Fin 2)`.

```
def walshVec (x : Fin n → Fin 2) : EuclideanSpace ℂ (Fin n → Fin 2)
```

lemma walshVec_apply [source]

The component `z` of `walshVec x` is `(√(2 ^ n))⁻¹ * walsh x z`.

```
@[simp]
lemma walshVec_apply (x z : Fin n → Fin 2) :
    walshVec x z = (Real.sqrt (2 ^ n))⁻¹ * walsh x z
```

lemma orthonormal_walshVec [source]

The rescaled Walsh vectors are orthonormal.

```
lemma orthonormal_walshVec : Orthonormal ℂ (walshVec (n := n))
```

def walshBasis [source]

The orthonormal basis of `EuclideanSpace ℂ (Fin n → Fin 2)` formed by the rescaled Walsh vectors `walshVec x`.

```
def walshBasis (n : ℕ) :
    OrthonormalBasis (Fin n → Fin 2) ℂ (EuclideanSpace ℂ (Fin n → Fin 2))
```

lemma coe_walshBasis [source]

The vectors of `walshBasis n` are the rescaled Walsh vectors.

```
@[simp]
lemma coe_walshBasis (n : ℕ) : ⇑(walshBasis n) = walshVec
```

lemma sum_norm_sq_sum_mul_inv_sqrt_mul_walsh [source]

Parseval's identity for the rescaled Walsh vectors. The vector `x ↦ ∑ z, β z * (√(2 ^ n))⁻¹ * walsh x z` has the same squared norm as `β`.

```
lemma sum_norm_sq_sum_mul_inv_sqrt_mul_walsh (n : ℕ) (β : (Fin n → Fin 2) → ℂ) :
    ∑ x : Fin n → Fin 2, ‖∑ z, β z * ((Real.sqrt (2 ^ n))⁻¹ * walsh x z)‖ ^ 2 =
      ∑ z, ‖β z‖ ^ 2
```

2.352 PhyslibAlpha.Mathematics.SubalgebraRestriction

PhyslibAlpha/Mathematics/SubalgebraRestriction.lean on GitHub.

abbrev polyRange [source]

The polynomials with coefficients in a subalgebra, as a subalgebra of the polynomials with coefficients in the ambient algebra.

```
noncomputable abbrev polyRange (S : Subalgebra R A) : Subalgebra R (Polynomial A)
```

lemma monomial_mem_polyRange [source]

A monomial with a coefficient in `S` has coefficients in `S`.

```
lemma monomial_mem_polyRange {S : Subalgebra R A} {n : ℕ} {y : A} (hy : y ∈ S) :
    Polynomial.monomial n y ∈ S.polyRange
```

lemma mem_range_mapAlgHom_of_adjoin [source]

If an algebra map into the polynomials sends every generator of an adjoined subalgebra into the polynomials over `S`, it sends the whole subalgebra there: the condition cuts out a subalgebra, and the generators lie in it.

```
lemma mem_range_mapAlgHom_of_adjoin {G : Set A} {f : A →ₐ[R] Polynomial A}
    (S : Subalgebra R A) (hgen : ∀ y ∈ G, f y ∈ S.polyRange)
    {x : A} (hx : x ∈ Algebra.adjoin R G) : f x ∈ S.polyRange
```

lemma mapAlgHom_val_injective [source]

Polynomials over a subalgebra inject into polynomials over the ambient algebra.

```
lemma mapAlgHom_val_injective (S : Subalgebra R A) :
    Function.Injective (Polynomial.mapAlgHom S.val)
```

def polyRestrict [source]

An algebra map into the polynomials, restricted to a subalgebra it carries into the polynomials over that subalgebra.

```
noncomputable def polyRestrict (S : Subalgebra R A) (f : A →ₐ[R] Polynomial A)
    (hf : ∀ x : S, f (x : A) ∈ S.polyRange) : S →ₐ[R] Polynomial S
```

lemma mapAlgHom_polyRestrict [source]

```
@[simp]
lemma mapAlgHom_polyRestrict {S : Subalgebra R A} {f : A →ₐ[R] Polynomial A}
    (hf : ∀ x : S, f (x : A) ∈ S.polyRange) (x : S) :
    Polynomial.mapAlgHom S.val (S.polyRestrict f hf x) = f (x : A)
```

lemma polyRestrict_eq_monomial_iff [source]

An eigenvalue equation for the restricted map is the ambient eigenvalue equation.

```
lemma polyRestrict_eq_monomial_iff {S : Subalgebra R A} {f : A →ₐ[R] Polynomial A}
    (hf : ∀ x : S, f (x : A) ∈ S.polyRange) {n : ℕ} (x : S) :
    S.polyRestrict f hf x = Polynomial.monomial n x
      ↔ f (x : A) = Polynomial.monomial n (x : A)
```

2.353 PhyslibAlpha.Mathematics.SymmetricAlgebra

PhyslibAlpha/Mathematics/SymmetricAlgebra.lean on GitHub.

def map [source]

The algebra homomorphism between symmetric algebras induced by a linear map of the underlying modules.

```
def map (f : M →ₗ[R] N) : SymmetricAlgebra R M →ₐ[R] SymmetricAlgebra R N
```

lemma map_apply_ι [source]

```
@[simp]
lemma map_apply_ι (f : M →ₗ[R] N) (x : M) : map f (ι R M x) = ι R N (f x)
```

lemma map_id [source]

```
@[simp]
lemma map_id : map (LinearMap.id : M →ₗ[R] M) = AlgHom.id R (SymmetricAlgebra R M)
```

lemma map_comp_map [source]

```
lemma map_comp_map (f : N →ₗ[R] P) (g : M →ₗ[R] N) :
    (map f).comp (map g) = map (f ∘ₗ g)
```

def congr [source]

The algebra equivalence between symmetric algebras induced by a linear equivalence of the underlying modules.

```
def congr (e : M ≃ₗ[R] N) : SymmetricAlgebra R M ≃ₐ[R] SymmetricAlgebra R N
```

lemma congr_apply_ι [source]

```
@[simp]
lemma congr_apply_ι (e : M ≃ₗ[R] N) (x : M) : congr e (ι R M x) = ι R N (e x)
```

lemma adjoin_range_ι [source]

The symmetric algebra is generated, as an `R`-algebra, by the degree-one elements. Only a commutative semiring of scalars and a module are needed, so that the statement applies to a module whose additive structure is not presented as a group, such as a direct sum.

```
@[simp]
lemma adjoin_range_ι :
    Algebra.adjoin R (Set.range (ι R M)) = (⊤ : Subalgebra R (SymmetricAlgebra R M))
```

def prodToTensor [source]

The forward half of `prodEquiv`: a generator `(m, n)` is sent to `ι m ⊗ 1 + 1 ⊗ ι n`.

```
noncomputable def prodToTensor :
    SymmetricAlgebra R (M × N) →ₐ[R] SymmetricAlgebra R M ⊗[R] SymmetricAlgebra R N
```

lemma prodToTensor_ι [source]

```
@[simp]
lemma prodToTensor_ι (x : M × N) :
    prodToTensor (ι R (M × N) x)
      = ι R M x.1 ⊗ₜ[R] 1 + (1 : SymmetricAlgebra R M) ⊗ₜ[R] ι R N x.2
```

def tensorToProd [source]

The backward half of `prodEquiv`: the two inclusions of the factors, multiplied together.

```
noncomputable def tensorToProd :
    SymmetricAlgebra R M ⊗[R] SymmetricAlgebra R N →ₐ[R] SymmetricAlgebra R (M × N)
```

lemma tensorToProd_tmul [source]

```
@[simp]
lemma tensorToProd_tmul (a : SymmetricAlgebra R M) (b : SymmetricAlgebra R N) :
    tensorToProd (a ⊗ₜ[R] b) = map (LinearMap.inl R M N) a * map (LinearMap.inr R M N) b
```

def prodEquiv [source]

***The symmetric algebra of a direct sum is the tensor product of the symmetric algebras.** Two bosonic fields taken together are one field valued in the direct sum of their target spaces; their generators commute, so the ordinary tensor product suffices — no grading is needed, in contrast to the exterior-algebra analogue.*

```
noncomputable def prodEquiv :
    SymmetricAlgebra R (M × N) ≃ₐ[R] SymmetricAlgebra R M ⊗[R] SymmetricAlgebra R N
```

lemma prodEquiv_ι [source]

```
@[simp]
lemma prodEquiv_ι (x : M × N) :
    prodEquiv (ι R (M × N) x)
      = ι R M x.1 ⊗ₜ[R] 1 + (1 : SymmetricAlgebra R M) ⊗ₜ[R] ι R N x.2
```

def derivationHom [source]

The lift of the derivation extending `d` to the trivial square-zero extension of the symmetric algebra: the algebra homomorphism `x ↦ (x, derivationOfLinear d x)`.

```
noncomputable def derivationHom :
    SymmetricAlgebra R M →ₐ[R]
      TrivSqZeroExt (SymmetricAlgebra R M) (SymmetricAlgebra R M)
```

lemma derivationHom_ι [source]

```
@[simp]
lemma derivationHom_ι (x : M) :
    derivationHom d (ι R M x) = (ι R M x, ι R M (d x))
```

lemma derivationHom_fst [source]

The first component of the square-zero lift is the identity.

```
@[simp]
lemma derivationHom_fst (x : SymmetricAlgebra R M) : (derivationHom d x).fst = x
```

def derivationOfLinear [source]

***The derivation of the symmetric algebra extending a linear endomorphism** `d` of `M`: the map obeying the Leibniz rule whose value on a generator `ι x` is `ι (d x)`.*

```
noncomputable def derivationOfLinear : SymmetricAlgebra R M →ₗ[R] SymmetricAlgebra R M where
```

lemma derivationOfLinear_ι [source]

```
@[simp]
lemma derivationOfLinear_ι (x : M) :
    derivationOfLinear d (ι R M x) = ι R M (d x)
```

lemma derivationOfLinear_one [source]

```
@[simp]
lemma derivationOfLinear_one : derivationOfLinear d (1 : SymmetricAlgebra R M) = 0
```

lemma derivationOfLinear_algebraMap [source]

```
@[simp]
lemma derivationOfLinear_algebraMap (r : R) :
    derivationOfLinear d (algebraMap R (SymmetricAlgebra R M) r) = 0
```

lemma derivationOfLinear_mul [source]

The Leibniz rule for the derivation extending `d`.

```
lemma derivationOfLinear_mul (x y : SymmetricAlgebra R M) :
    derivationOfLinear d (x * y)
      = derivationOfLinear d x * y + x * derivationOfLinear d y
```

lemma derivationOfLinear_comm_apply [source]

Derivations extending commuting endomorphisms commute.

```
lemma derivationOfLinear_comm_apply {d₁ d₂ : M →ₗ[R] M} (h : d₁ ∘ₗ d₂ = d₂ ∘ₗ d₁)
    (x : SymmetricAlgebra R M) :
    derivationOfLinear d₁ (derivationOfLinear d₂ x)
      = derivationOfLinear d₂ (derivationOfLinear d₁ x)
```

lemma algHom_derivationOfLinear [source]

An algebra map carries one derivation to the other when the linear map it is determined by on the generators intertwines the two endomorphisms. The hypotheses are stated pointwise so that the lemma can be applied without rewriting inside a large algebra.

```
lemma algHom_derivationOfLinear
    (F : SymmetricAlgebra R M →ₐ[R] SymmetricAlgebra R N) {f : M →ₗ[R] N}
    (hF : ∀ x, F (ι R M x) = ι R N (f x)) {d : M →ₗ[R] M} {d' : N →ₗ[R] N}
    (h : ∀ x, f (d x) = d' (f x)) (y : SymmetricAlgebra R M) :
    F (derivationOfLinear d y) = derivationOfLinear d' (F y)
```

def symmetricAlgebra [source]

The representation on the symmetric algebra extending a representation on the generators: the substitution homomorphism determined by the action on the generators. It is built from `SymmetricAlgebra.lift` rather than from `SymmetricAlgebra.map`, so that it is available over a commutative semiring and an additive commutative monoid — the generality the generator spaces of a multi-species field theory, which are direct sums, are elaborated at.

```
noncomputable def symmetricAlgebra (ρ : Representation R G M) :
    Representation R G (SymmetricAlgebra R M) where
```

def symmetricAlgebraAlgHom [source]

The action of a group element, as an algebra homomorphism.

```
noncomputable def symmetricAlgebraAlgHom (ρ : Representation R G M) (g : G) :
    SymmetricAlgebra R M →ₐ[R] SymmetricAlgebra R M
```

lemma symmetricAlgebra_apply [source]

```
lemma symmetricAlgebra_apply (ρ : Representation R G M) (g : G)
    (x : SymmetricAlgebra R M) :
    ρ.symmetricAlgebra g x = ρ.symmetricAlgebraAlgHom g x
```

lemma symmetricAlgebra_ι [source]

```
@[simp]
lemma symmetricAlgebra_ι (ρ : Representation R G M) (g : G) (x : M) :
    ρ.symmetricAlgebra g (SymmetricAlgebra.ι R M x)
      = SymmetricAlgebra.ι R M (ρ g x)
```

lemma symmetricAlgebra_apply_one [source]

```
@[simp]
lemma symmetricAlgebra_apply_one (ρ : Representation R G M) (g : G) :
    ρ.symmetricAlgebra g (1 : SymmetricAlgebra R M) = 1
```

lemma symmetricAlgebra_apply_mul [source]

```
lemma symmetricAlgebra_apply_mul (ρ : Representation R G M) (g : G)
    (x y : SymmetricAlgebra R M) :
    ρ.symmetricAlgebra g (x * y)
      = ρ.symmetricAlgebra g x * ρ.symmetricAlgebra g y
```

lemma symmetricAlgebra_algebraMap [source]

```
lemma symmetricAlgebra_algebraMap (ρ : Representation R G M) (g : G) (r : R) :
    ρ.symmetricAlgebra g (algebraMap R (SymmetricAlgebra R M) r)
      = algebraMap R (SymmetricAlgebra R M) r
```

lemma symmetricAlgebra_derivationOfLinear [source]

Covariance of the derivation under the representation. If the representation carries each endomorphism of a family into a combination of the others on the generators, then it carries the derivation extending one into the same combination of the derivations extending the others on the whole exterior algebra. This is the shape the statement that the total derivative is a Lorentz vector takes; it is proved by induction rather than by algebra-map extensionality, a derivation not being an algebra map.

```
lemma symmetricAlgebra_derivationOfLinear {G κ : Type*} [Monoid G] [Fintype κ]
    (ρ : Representation R G M) (g : G) (d : κ → M →ₗ[R] M) (μ : κ) (c : κ → R)
    (h : ∀ x, ρ g (d μ x) = ∑ a, c a • d a (ρ g x)) (y : SymmetricAlgebra R M) :
    ρ.symmetricAlgebra g (derivationOfLinear (d μ) y)
      = ∑ a, c a • derivationOfLinear (d a) (ρ.symmetricAlgebra g y)
```

lemma algHom_symmetricAlgebra [source]

An algebra map carries one representation to the other when the linear map it is determined by on the generators intertwines the two representations on them. Unlike the derivation statement this is pure extensionality of algebra maps, each group element acting by one; the hypotheses are stated pointwise so that the lemma can be applied without rewriting inside a large algebra.

```
lemma algHom_symmetricAlgebra {N G : Type*} [AddCommMonoid N] [Module R N] [Monoid G]
    (F : SymmetricAlgebra R M →ₐ[R] SymmetricAlgebra R N) {f : M →ₗ[R] N}
    (hF : ∀ x, F (ι R M x) = ι R N (f x))
    {ρ : Representation R G M} {σ : Representation R G N} (g : G)
    (h : ∀ x, f (ρ g x) = σ g (f x)) (y : SymmetricAlgebra R M) :
    F (ρ.symmetricAlgebra g y) = σ.symmetricAlgebra g (F y)
```

2.354 PhyslibAlpha.Mathematics.TauCeti.Analysis.Bochner.DiscreteGroup

PhyslibAlpha/Mathematics/TauCeti/Analysis/Bochner/DiscreteGroup.lean on GitHub.

lemma isPositiveDefiniteSub_iff_exists_pontryaginMeasureTransform_eq [source]

***Bochner's theorem on a discrete abelian group.** A function on a discrete abelian group is positive definite if and only if it is the Fourier–Stieltjes transform of a finite inner regular measure on the Pontryagin dual.*

```
lemma isPositiveDefiniteSub_iff_exists_pontryaginMeasureTransform_eq (φ : G → ℂ) :
    IsPositiveDefiniteSub φ ↔
      ∃ μ : FiniteMeasure (PontryaginDual (Multiplicative G)),
        μ.toMeasure.InnerRegular ∧ μ.pontryaginMeasureTransform = φ
```

2.355 PhyslibAlpha.Mathematics.TauCeti.Analysis.Bochner.LocallyCompactGroup

PhyslibAlpha/Mathematics/TauCeti/Analysis/Bochner/LocallyCompactGroup.lean on GitHub.

lemma exists_pontryaginMeasureTransform_eq_of_continuousAt [source]

***Bochner's theorem on a locally compact abelian group**, existence half. A positive-definite function that is continuous at `0` is the Fourier–Stieltjes transform of a finite inner regular measure on the Pontryagin dual.*

```
lemma IsPositiveDefiniteSub.exists_pontryaginMeasureTransform_eq_of_continuousAt {φ : G → ℂ}
    (hφ : IsPositiveDefiniteSub φ) (hφ₀ : ContinuousAt φ 0) :
    ∃ μ : FiniteMeasure (PontryaginDual (Multiplicative G)),
      μ.toMeasure.InnerRegular ∧ μ.pontryaginMeasureTransform = φ
```

lemma «lemma@69465c7e» [source]

***Unique inner regular form of Bochner's theorem.** A positive-definite function continuous at `0` has a unique finite inner regular representing measure on the Pontryagin dual. No countability or metrizability assumption is needed.*

```
lemma
    IsPositiveDefiniteSub.
    existsUnique_innerRegular_pontryaginMeasureTransform_eq_of_continuousAt
    {φ : G → ℂ} (hφ : IsPositiveDefiniteSub φ) (hφ₀ : ContinuousAt φ 0) :
    ∃! μ : FiniteMeasure (PontryaginDual (Multiplicative G)),
      μ.toMeasure.InnerRegular ∧ μ.pontryaginMeasureTransform = φ
```

lemma «lemma@c96371b0» [source]

***Bochner's characterization without countability assumptions.** A function on a locally compact abelian group is continuous and positive definite if and only if it is the Fourier–Stieltjes transform of a unique finite inner regular measure on the Pontryagin dual.*

```
lemma

    continuous_and_isPositiveDefiniteSub_iff_existsUnique_innerRegular_pontryaginMeasureTransform_eq

    (φ : G → ℂ) :
    (Continuous φ ∧ IsPositiveDefiniteSub φ) ↔
      ∃! μ : FiniteMeasure (PontryaginDual (Multiplicative G)),
        μ.toMeasure.InnerRegular ∧ μ.pontryaginMeasureTransform = φ
```

lemma continuous_and_isPositiveDefiniteSub_iff_existsUnique_pontryaginMeasureTransform_eq [source]

***Bochner's characterization on a locally compact abelian group with Polish dual.** A function is continuous and positive definite if and only if it is the Fourier–Stieltjes transform of a unique finite positive Borel measure on the dual group. The dual is Polish when the group is second countable.*

```
lemma continuous_and_isPositiveDefiniteSub_iff_existsUnique_pontryaginMeasureTransform_eq
    [PolishSpace (PontryaginDual (Multiplicative G))] (φ : G → ℂ) :
    (Continuous φ ∧ IsPositiveDefiniteSub φ) ↔
      ∃! μ : FiniteMeasure (PontryaginDual (Multiplicative G)),
        μ.pontryaginMeasureTransform = φ
```

2.356 PhyslibAlpha.Mathematics.TauCeti.Analysis.CStarAlgebra.CharacterSpaceMeasure

PhyslibAlpha/Mathematics/TauCeti/Analysis/CStarAlgebra/CharacterSpaceMeasure.lean on GitHub.

lemma nonneg_apply_symm_realToRCLike [source]

Pulled back along the Gelfand transform, a functional nonnegative on `star a * a` is nonnegative on nonnegative real functions: such a function is `star h * h` for `h = √g`.

```
private lemma nonneg_apply_symm_realToRCLike (hf : ∀ a, 0 ≤ f (star a * a))
    {g : C(characterSpace ℂ A, ℝ)} (hg : 0 ≤ g) :
    0 ≤ f ((gelfandStarTransform A).symm (g.realToRCLike ℂ))
```

lemma apply_symm_realToRCLike_eq_re [source]

Pulled back along the Gelfand transform, a functional nonnegative on `star a * a` is real on real functions.

```
private lemma apply_symm_realToRCLike_eq_re (hf : ∀ a, 0 ≤ f (star a * a))
    (g : C(characterSpace ℂ A, ℝ)) :
    f ((gelfandStarTransform A).symm (g.realToRCLike ℂ)) =
      (f ((gelfandStarTransform A).symm (g.realToRCLike ℂ))).re
```

def realFunctional [source]

The positive linear functional on `C_c(Δ, ℝ) = C(Δ, ℝ)` induced by `f` through the Gelfand transform.

```
private def realFunctional (hf : ∀ a, 0 ≤ f (star a * a)) :
    C_c(characterSpace ℂ A, ℝ) →ₚ[ℝ] ℝ where
```

lemma realFunctional_apply [source]

```
private lemma realFunctional_apply (hf : ∀ a, 0 ≤ f (star a * a))
    (g : C_c(characterSpace ℂ A, ℝ)) :
    realFunctional f hf g =
      (f ((gelfandStarTransform A).symm (g.toContinuousMap.realToRCLike ℂ))).re
```

lemma exists_isFiniteMeasure_integral_characterSpace_eq [source]

***Positive functionals on a commutative C⋆-algebra are measures on its character space.** A linear functional on a unital commutative C⋆-algebra that is nonnegative on every `star a * a` is integration against a finite inner regular positive measure `μ` on the character space: `f a = ∫ ω, ω a ∂μ`.*

```
lemma LinearMap.exists_isFiniteMeasure_integral_characterSpace_eq (f : A →ₗ[ℂ] ℂ)
    (hf : ∀ a, 0 ≤ f (star a * a)) :
    ∃ μ : Measure (characterSpace ℂ A), IsFiniteMeasure μ ∧ μ.InnerRegular ∧
      ∀ a, f a = ∫ ω, ω a ∂μ
```

lemma integral_apply_star_mul_self [source]

For a measure `μ` on the character space of a unital C⋆-algebra, integrating `ω ↦ ω (star a * a) = |ω a|²` gives the squared `L²` norm of `ω ↦ ω a`. Hence if `μ` represents a

functional `f`, as in `LinearMap.exists_isFiniteMeasure_integral_characterSpace_eq`, then `f (star a * a) = ∫ |ω a|² dμ(ω)`.

```
lemma WeakDual.CharacterSpace.integral_apply_star_mul_self {E : Type*} [CStarAlgebra E]
    [MeasurableSpace (characterSpace ℂ E)] (μ : Measure (characterSpace ℂ E)) (a : E) :
    ∫ ω, ω (star a * a) ∂μ = ((∫ ω, ‖ω a‖ ^ 2 ∂μ : ℝ) : ℂ)
```

lemma inner_star_mul_self_apply [source]

For a star algebra `B` of operators, the positive vector functional of `ξ` takes the value `‖a ξ‖²` at `star a * a`.

```
private lemma inner_star_mul_self_apply (ξ : H) (a : B) :
    ⟪ξ, ((star a * a : B) : H →L[ℂ] H) ξ⟫_ℂ = ((‖(a : H →L[ℂ] H) ξ‖ ^ 2 : ℝ) : ℂ)
```

lemma integral_norm_sq_eq_norm_apply_sq [source]

For a closed star algebra `B` of operators, if a measure `μ` on the character space represents the positive vector functional of `ξ`, then `∫ |ω a|² dμ(ω) = ‖a ξ‖²` for every `a ∈ B`.

```
lemma StarSubalgebra.integral_norm_sq_eq_norm_apply_sq {μ : Measure (characterSpace ℂ B)}
    {ξ : H} (hμ : ∀ a : B, ⟪ξ, (a : H →L[ℂ] H) ξ⟫_ℂ = ∫ ω, ω a ∂μ) (a : B) :
    ∫ ω, ‖ω a‖ ^ 2 ∂μ = ‖(a : H →L[ℂ] H) ξ‖ ^ 2
```

lemma exists_isFiniteMeasure_integral_characterSpace_eq_inner [source]

***Positive vector functionals are measures on the character space.** For a closed commutative star algebra `B` of operators on a complex Hilbert space and a vector `ξ`, the positive vector functional `a ↦ ⟪ξ, a ξ⟫` is integration against a finite inner regular positive measure on the character space of `B`.*

```
lemma StarSubalgebra.exists_isFiniteMeasure_integral_characterSpace_eq_inner (ξ : H) :
    ∃ μ : Measure (characterSpace ℂ B), IsFiniteMeasure μ ∧ μ.InnerRegular ∧
      ∀ a : B, ⟪ξ, (a : H →L[ℂ] H) ξ⟫_ℂ = ∫ ω, ω a ∂μ
```

2.357 PhyslibAlpha.Mathematics.TauCeti.Analysis.CStarAlgebra.UnitaryCharacter

PhyslibAlpha/Mathematics/TauCeti/Analysis/CStarAlgebra/UnitaryCharacter.lean on GitHub.

lemma existsUnique_pontryaginDual_of_unitary_translate [source]

A character nonzero on a commuting operator with norm-continuous unitary translates induces a unique continuous circle-valued character. It describes every translate in the subalgebra, not just the translates of the witnessing operator.

```
lemma _root_.MonoidHom.existsUnique_pontryaginDual_of_unitary_translate
    (ρ : G →* B) (hρ : ∀ g, ρ g ∈ unitary B)
    (A : StarSubalgebra ℂ B) [CompleteSpace A]
    (a : A) (hcomm : ∀ g, Commute (ρ g) (a : B))
    (hmem : ∀ g, ρ g * (a : B) ∈ A)
    (hcont : Continuous fun g => ρ g * (a : B))
    (ω : characterSpace ℂ A) (ha : ω a ≠ 0) :
    ∃! χ : PontryaginDual G,
      ∀ (g : G) (b : A) (hb : ρ g * (b : B) ∈ A),
```

```
        ω ⟨ρ g * (b : B), hb⟩ = (χ g : ℂ) * ω b
```

2.358 PhyslibAlpha.Mathematics.TauCeti.Analysis.Fourier.Pontryagin.Continuity

PhyslibAlpha/Mathematics/TauCeti/Analysis/Fourier/Pontryagin/Continuity.lean on GitHub.

lemma continuous_pontryaginMeasureTransform [source]

The transform of a tight finite measure on the Pontryagin dual is continuous. Tightness is automatic for finite Radon measures and is the regularity needed when the dual is not metrizable.

```
lemma _root_.MeasureTheory.FiniteMeasure.continuous_pontryaginMeasureTransform
    (μ : FiniteMeasure (PontryaginDual (Multiplicative G)))
    (hμ : IsTightMeasureSet {μ.toMeasure}) :
    Continuous μ.pontryaginMeasureTransform
```

2.359 PhyslibAlpha.Mathematics.TauCeti.Analysis.Fourier.Pontryagin.DualEval

PhyslibAlpha/Mathematics/TauCeti/Analysis/Fourier/Pontryagin/DualEval.lean on GitHub.

lemma continuous_coe_eval_const [source]

Complex-valued evaluation of a Pontryagin character at a fixed monoid element is continuous.

```
@[fun_prop]
lemma continuous_coe_eval_const (g : G) :
    Continuous (fun χ : _root_.PontryaginDual G => (χ g : ℂ))
```

lemma integrable_coe_eval [source]

Complex-valued evaluation of a Pontryagin character is integrable against a finite measure.

```
lemma integrable_coe_eval
    {μ : Measure (_root_.PontryaginDual G)} [IsFiniteMeasure μ] (g : G) :
    Integrable (fun χ : _root_.PontryaginDual G => (χ g : ℂ)) μ
```

2.360 PhyslibAlpha.Mathematics.TauCeti.Analysis.Fourier.Pontryagin.Measure

PhyslibAlpha/Mathematics/TauCeti/Analysis/Fourier/Pontryagin/Measure.lean on GitHub.

def pontryaginMeasureTransform [source]

The Fourier–Stieltjes transform of a finite measure on the Pontryagin dual of an additive group.

```
noncomputable def _root_.MeasureTheory.FiniteMeasure.pontryaginMeasureTransform
    (μ : FiniteMeasure (PontryaginDual (Multiplicative G))) (g : G) : ℂ
```

lemma pontryaginMeasureTransform_apply [source]

The transform evaluated at a group element is the integral of character evaluations.

```
lemma _root_.MeasureTheory.FiniteMeasure.pontryaginMeasureTransform_apply
    (μ : FiniteMeasure (PontryaginDual (Multiplicative G))) (g : G) :
    μ.pontryaginMeasureTransform g =
      ∫ χ, (χ (Multiplicative.ofAdd g) : ℂ) ∂μ.toMeasure
```

lemma pontryaginMeasureTransform_zero [source]

At the identity, the transform records the total mass of the measure.

```
@[simp]
lemma _root_.MeasureTheory.FiniteMeasure.pontryaginMeasureTransform_zero
    (μ : FiniteMeasure (PontryaginDual (Multiplicative G))) :
    μ.pontryaginMeasureTransform 0 = (μ.toMeasure.real Set.univ : ℂ)
```

lemma pontryaginMeasureTransform_zero_measure [source]

The transform of the zero measure vanishes.

```
@[simp]
lemma _root_.MeasureTheory.FiniteMeasure.pontryaginMeasureTransform_zero_measure :
    (0 : FiniteMeasure (PontryaginDual (Multiplicative G))).pontryaginMeasureTransform = 0
```

lemma pontryaginMeasureTransform_smul [source]

The transform commutes with nonnegative scalar multiplication of finite measures.

```
@[simp]
lemma _root_.MeasureTheory.FiniteMeasure.pontryaginMeasureTransform_smul
    (c : NNReal) (μ : FiniteMeasure (PontryaginDual (Multiplicative G))) :
    (c • μ).pontryaginMeasureTransform = c • μ.pontryaginMeasureTransform
```

lemma pontryaginMeasureTransform_add [source]

The transform commutes with addition of finite measures.

```
@[simp]
lemma _root_.MeasureTheory.FiniteMeasure.pontryaginMeasureTransform_add
    (μ ν : FiniteMeasure (PontryaginDual (Multiplicative G))) :
    (μ + ν).pontryaginMeasureTransform =
      μ.pontryaginMeasureTransform + ν.pontryaginMeasureTransform
```

lemma pontryaginMeasureTransform_dirac [source]

The transform of a point mass is its character.

```
@[simp]
lemma _root_.MeasureTheory.FiniteMeasure.pontryaginMeasureTransform_dirac
    (χ : PontryaginDual (Multiplicative G)) (g : G) :
    FiniteMeasure.pontryaginMeasureTransform
      (⟨Measure.dirac χ, inferInstance⟩ : FiniteMeasure
        (PontryaginDual (Multiplicative G))) g =
      (χ (Multiplicative.ofAdd g) : ℂ)
```

2.361 PhyslibAlpha.Mathematics.TauCeti.Analysis.Fourier.Pontryagin.StronglyContinuous

PhyslibAlpha/Mathematics/TauCeti/Analysis/Fourier/Pontryagin/StronglyContinuous.lean on GitHub.

abbrev algebra [source]

The closed star algebra generated by the integrated operators, for a Haar measure.

```
private abbrev algebra : StarSubalgebra ℂ (H →L[ℂ] H)
```

instance IsMulCommutative (algebra π hcont hπ) [source]

```
private instance : IsMulCommutative (algebra π hcont hπ)
```

instance CommCStarAlgebra (algebra π hcont hπ) [source]

```
private noncomputable instance : CommCStarAlgebra (algebra π hcont hπ) where
```

abbrev op [source]

An integrated operator, as an element of the integrated algebra.

```
private abbrev op (f : G →₁[Measure.addHaar] ℂ) : algebra π hcont hπ
```

abbrev translate [source]

A translated integrated operator `π(g) π(f)`, as an element of the integrated algebra.

```
private abbrev translate (f : G →₁[Measure.addHaar] ℂ) (g : G) : algebra π hcont hπ
```

lemma translate_eq [source]

```
private lemma translate_eq (f : G →₁[Measure.addHaar] ℂ) (g : G) :
    translate π hcont hπ f g =
      op π hcont hπ (Lp.compMeasurePreserving (fun x ↦ -g + x)
        (measurePreserving_add_left Measure.addHaar (-g)) f)
```

lemma mem_algebra [source]

```
private lemma mem_algebra (f : G →₁[Measure.addHaar] ℂ) :
    π.integratedOperatorL1 hcont hπ.exists_norm_le Measure.addHaar f ∈ algebra π hcont hπ
```

lemma op_eq [source]

```
private lemma op_eq (f : G →₁[Measure.addHaar] ℂ) :
    op π hcont hπ f = ⟨_, mem_algebra π hcont hπ f⟩
```

abbrev support [source]

The characters of the integrated algebra that do not annihilate every integrated operator.

```
private abbrev support : Set (characterSpace ℂ (algebra π hcont hπ))
```

def dual [source]

The continuous group character attached to a character of the integrated algebra, and `1` for a character annihilating every integrated operator.

```
private def dual (ω : characterSpace ℂ (algebra π hcont hπ)) :
    PontryaginDual (Multiplicative G)
```

lemma apply_translate [source]

***The translation equation.** Every character of the integrated algebra multiplies a translated integrated operator by its group character.*

```
private lemma apply_translate (ω : characterSpace ℂ (algebra π hcont hπ)) (g : G)
    (f : G →₁[Measure.addHaar] ℂ) :
    ω (translate π hcont hπ f g) = (dual π hcont hπ ω (.ofAdd g) : ℂ) * ω (op π hcont hπ f)
```

lemma continuousOn_dual [source]

The group character of a character of the integrated algebra depends continuously on it, on the open set of characters that do not annihilate every integrated operator.

```
private lemma continuousOn_dual : ContinuousOn (dual π hcont hπ) (support π hcont hπ)
```

lemma ae_mem_support [source]

A vector-state measure gives zero mass to characters annihilating all integrated operators.

```
private lemma ae_mem_support [MeasurableSpace (characterSpace ℂ (algebra π hcont hπ))]
    [BorelSpace (characterSpace ℂ (algebra π hcont hπ))]
    {ν : Measure (characterSpace ℂ (algebra π hcont hπ))} [IsFiniteMeasure ν] {ξ : H}
    (hrep : ∀ a : algebra π hcont hπ, ⟪ξ, (a : H →L[ℂ] H) ξ⟫_ℂ = ∫ ω, ω a ∂ν) :
    ∀ᵐ ω ∂ν, ω ∈ support π hcont hπ
```

lemma norm_integral_mul_apply_le [source]

If `ν` represents the positive vector functional of `ξ`, then integrating `ω ↦ ω a` against a weight bounded by `1` costs at most `δ ν(A) + ‖a ξ‖² / δ`, for every `δ > 0`. This is the bound of an `L¹` norm by an `L²` norm, through `‖z‖ ≤ δ + ‖z‖² / δ`.

```
private lemma norm_integral_mul_apply_le [MeasurableSpace (characterSpace ℂ (algebra π hcont h
    π))]
    [BorelSpace (characterSpace ℂ (algebra π hcont hπ))]
    {ν : Measure (characterSpace ℂ (algebra π hcont hπ))} [IsFiniteMeasure ν] {ξ : H}
    (hrep : ∀ a : algebra π hcont hπ, ⟪ξ, (a : H →L[ℂ] H) ξ⟫_ℂ = ∫ ω, ω a ∂ν)
    {w : characterSpace ℂ (algebra π hcont hπ) → ℂ}
    (hw₁ : ∀ ω, ‖w ω‖ ≤ 1) (a : algebra π hcont hπ) {δ : ℝ} (hδ : 0 < δ) :
    ‖∫ ω, w ω * ω a ∂ν‖ ≤ δ * ν.real Set.univ + ‖(a : H →L[ℂ] H) ξ‖ ^ 2 / δ
```

lemma measurable_dual [source]

The group character of a character of the integrated algebra depends measurably on it.

```
private lemma measurable_dual [MeasurableSpace (characterSpace ℂ (algebra π hcont hπ))]
    [BorelSpace (characterSpace ℂ (algebra π hcont hπ))] : Measurable (dual π hcont hπ)
```

lemma inner_eq_integral_dual [source]

***The matrix coefficient is the integral of the group characters.** If `ν` represents the positive vector functional of `ξ` on the integrated algebra, then `⟪ξ, π(g) ξ⟫ = ∫ χ_ω(g) dν(ω)`.*

```
private lemma inner_eq_integral_dual [MeasurableSpace (characterSpace ℂ (algebra π hcont hπ))]
    [BorelSpace (characterSpace ℂ (algebra π hcont hπ))]
    {ν : Measure (characterSpace ℂ (algebra π hcont hπ))} [IsFiniteMeasure ν] {ξ : H}
    (hrep : ∀ a : algebra π hcont hπ, ⟪ξ, (a : H →L[ℂ] H) ξ⟫_ℂ = ∫ ω, ω a ∂ν) (g : G) :
    ⟪ξ, π (.ofAdd g) ξ⟫_ℂ = ∫ ω, (dual π hcont hπ ω (.ofAdd g) : ℂ) ∂ν
```

lemma exists_pontryaginMeasureTransform_eq_inner [source]

***Spectral theorem for strongly continuous unitary representations**, cyclic form. For a strongly continuous unitary representation `π` of a locally compact abelian group `G` on a Hilbert space `H` and a vector `ξ`, the matrix coefficient `g ↦ ⟪ξ, π(g) ξ⟫` is the Fourier–Stieltjes transform of a finite inner regular measure on the Pontryagin dual of `G`.*

```
lemma ContRepresentation.exists_pontryaginMeasureTransform_eq_inner
    (π : ContRepresentation ℂ (Multiplicative G) H)
    (hcont : ∀ v, Continuous fun g : G ↦ π (.ofAdd g) v)
    (hπ : ContRepresentation.IsUnitary π) (ξ : H) :
    ∃ μ : FiniteMeasure (PontryaginDual (Multiplicative G)),
      μ.toMeasure.InnerRegular ∧ ∀ g, μ.pontryaginMeasureTransform g = ⟪ξ, π (.ofAdd g) ξ⟫_ℂ
```

2.362 PhyslibAlpha.Mathematics.TauCeti.Analysis.Fourier.Pontryagin.Uniqueness

PhyslibAlpha/Mathematics/TauCeti/Analysis/Fourier/Pontryagin/Uniqueness.lean on GitHub.

def evalBoundedContinuous [source]

Evaluation at `g`, as a bounded continuous complex-valued function on the Pontryagin dual. Its values lie on the unit circle, so its norm is bounded by one.

```
def evalBoundedContinuous (g : G) :
    _root_.PontryaginDual (Multiplicative G) →ᵇ ℂ
```

lemma evalBoundedContinuous_apply [source]

Evaluation of `evalBoundedContinuous` at a character.

```
@[simp]
lemma evalBoundedContinuous_apply (g : G)
    (χ : _root_.PontryaginDual (Multiplicative G)) :
    evalBoundedContinuous g χ = (χ (Multiplicative.ofAdd g) : ℂ)
```

lemma evalBoundedContinuous_zero [source]

Evaluation at zero is the constant function one.

```
@[simp]
lemma evalBoundedContinuous_zero :
    evalBoundedContinuous (0 : G) = 1
```

lemma evalBoundedContinuous_add [source]

Evaluation turns addition in the original group into pointwise multiplication.

```
@[simp]
lemma evalBoundedContinuous_add (g h : G) :
    evalBoundedContinuous (g + h) =
      evalBoundedContinuous g * evalBoundedContinuous h
```

lemma evalBoundedContinuous_neg [source]

Evaluation at the negative of a group element is the pointwise star of evaluation there.

```
@[simp]
lemma evalBoundedContinuous_neg (g : G) :
    evalBoundedContinuous (-g) = star (evalBoundedContinuous g)
```

def evalMonoidHom [source]

Evaluation as a monoid homomorphism from the multiplicative copy of the original group.

```
def evalMonoidHom : Multiplicative G →*
    (_root_.PontryaginDual (Multiplicative G) →ᵇ ℂ) where
```

lemma evalMonoidHom_apply [source]

Evaluation of `evalMonoidHom` at a character.

```
@[simp]
lemma evalMonoidHom_apply (g : Multiplicative G)
    (χ : _root_.PontryaginDual (Multiplicative G)) :
    evalMonoidHom g χ = (χ g : ℂ)
```

def evalAlgHom [source]

The algebra homomorphism sending a formal finite linear combination of group elements to the corresponding finite linear combination of evaluation characters.

```
def evalAlgHom : AddMonoidAlgebra ℂ G →ₐ[ℂ]
    (_root_.PontryaginDual (Multiplicative G) →ᵇ ℂ)
```

lemma evalAlgHom_apply [source]

Evaluation of `evalAlgHom` is the corresponding finite linear combination of characters.

```
@[simp]
lemma evalAlgHom_apply (a : AddMonoidAlgebra ℂ G)
    (χ : _root_.PontryaginDual (Multiplicative G)) :
    evalAlgHom a χ = a.coeff.sum fun g c => c * (χ (Multiplicative.ofAdd g) : ℂ)
```

lemma star_mem_range_evalAlgHom [source]

The range of `evalAlgHom` is closed under pointwise complex conjugation.

```
lemma star_mem_range_evalAlgHom
    {f : _root_.PontryaginDual (Multiplicative G) →ᵇ ℂ}
    (hf : f ∈ evalAlgHom.range) : star f ∈ evalAlgHom.range
```

def evalPoly [source]

The star subalgebra of bounded continuous functions generated by evaluation characters.

```
def evalPoly : StarSubalgebra ℂ
    (_root_.PontryaginDual (Multiplicative G) →ᵇ ℂ) where
```

lemma evalPoly_toSubalgebra [source]

The underlying subalgebra of `evalPoly` is the range of `evalAlgHom`.

```
lemma evalPoly_toSubalgebra :
    (evalPoly (G := G)).toSubalgebra = evalAlgHom.range
```

lemma mem_evalPoly [source]

Membership in `evalPoly` means being a finite linear combination of evaluation characters.

```
lemma mem_evalPoly (f : _root_.PontryaginDual (Multiplicative G) →ᵇ ℂ) :
    f ∈ evalPoly ↔
      ∃ a : AddMonoidAlgebra ℂ G,
        f = a.coeff.sum fun g c => c • evalBoundedContinuous g
```

lemma evalBoundedContinuous_mem_evalPoly [source]

Every evaluation character belongs to `evalPoly`.

```
lemma evalBoundedContinuous_mem_evalPoly (g : G) :
    evalBoundedContinuous g ∈ evalPoly
```

lemma separatesPoints_evalPoly [source]

The evaluation-character algebra separates points of the Pontryagin dual.

```
lemma separatesPoints_evalPoly :
    ((evalPoly : StarSubalgebra ℂ
      (_root_.PontryaginDual (Multiplicative G) →ᵇ ℂ)).map
        (BoundedContinuousFunction.toContinuousMapStarₐ ℂ)).SeparatesPoints
```

lemma integral_eq_of_forall_pontryaginMeasureTransform_eq [source]

Two finite measures on the dual with the same Fourier–Stieltjes transform integrate every finite linear combination of evaluation characters equally.

```
lemma _root_.MeasureTheory.FiniteMeasure.integral_eq_of_forall_pontryaginMeasureTransform_eq
    {P Q : FiniteMeasure (_root_.PontryaginDual (Multiplicative G))}
    (h : ∀ g, P.pontryaginMeasureTransform g = Q.pontryaginMeasureTransform g)
    {f : _root_.PontryaginDual (Multiplicative G) →ᵇ ℂ} (hf : f ∈ _root_.PontryaginDual.
    evalPoly) :
    ∫ χ, f χ ∂P = ∫ χ, f χ ∂Q
```

lemma ext_of_forall_pontryaginMeasureTransform_eq [source]

***Uniqueness of Fourier–Stieltjes measures on a Polish Pontryagin dual.** Two finite measures on the dual are equal when their transforms agree on every element of the original group.*

```
lemma _root_.MeasureTheory.FiniteMeasure.ext_of_forall_pontryaginMeasureTransform_eq
    {P Q : FiniteMeasure (_root_.PontryaginDual (Multiplicative G))}
    (h : ∀ g, P.pontryaginMeasureTransform g = Q.pontryaginMeasureTransform g) :
    P = Q
```

lemma pontryaginMeasureTransform_injective [source]

The Fourier–Stieltjes transform is injective on finite measures on a Polish Pontryagin dual.

```
lemma _root_.MeasureTheory.FiniteMeasure.pontryaginMeasureTransform_injective :
    Function.Injective
      (fun P : FiniteMeasure (_root_.PontryaginDual (Multiplicative G)) =>
        P.pontryaginMeasureTransform)
```

lemma «lemma@add06f0d» [source]

***Uniqueness of inner regular Fourier–Stieltjes measures.** On the Pontryagin dual of a locally compact abelian group, two finite inner regular measures are equal when their transforms agree on every element of the group. No countability or metrizability is assumed.*

```
lemma
    _root_.MeasureTheory.FiniteMeasure.
    ext_of_forall_pontryaginMeasureTransform_eq_of_innerRegular
    {P Q : FiniteMeasure (_root_.PontryaginDual (Multiplicative G))}
    [P.toMeasure.InnerRegular] [Q.toMeasure.InnerRegular]
    (h : ∀ g, P.pontryaginMeasureTransform g = Q.pontryaginMeasureTransform g) :
    P = Q
```

lemma pontryaginMeasureTransform_injOn_innerRegular [source]

The Fourier–Stieltjes transform is injective on finite inner regular measures on the Pontryagin dual of a locally compact abelian group.

```
lemma _root_.MeasureTheory.FiniteMeasure.pontryaginMeasureTransform_injOn_innerRegular :
    Set.InjOn
      (fun P : FiniteMeasure (_root_.PontryaginDual (Multiplicative G)) =>
```

```
      P.pontryaginMeasureTransform)
    {P | P.toMeasure.InnerRegular}
```

2.363 PhyslibAlpha.Mathematics.TauCeti.Analysis.InnerProductSpace.BasisSpan

PhyslibAlpha/Mathematics/TauCeti/Analysis/InnerProductSpace/BasisSpan.lean on GitHub.

def spanIndices [source]

The subspace spanned by the orthonormal basis vectors `b i` for indices `i ∈ s`.

```
noncomputable def spanIndices (b : OrthonormalBasis ι 𝕜 E) (s : Set ι) :
    Submodule 𝕜 E
```

lemma spanIndices_eq_span [source]

The selected subspace is the span of its basis vectors. Use this identity to apply `Submodule.span`'s induction or universal property.

```
lemma spanIndices_eq_span (b : OrthonormalBasis ι 𝕜 E) (s : Set ι) :
    b.spanIndices s = Submodule.span 𝕜 (b '' s)
```

lemma spanIndices_mono [source]

Selecting more indices spans more.

```
lemma spanIndices_mono (b : OrthonormalBasis ι 𝕜 E) {s t : Set ι} (h : s ⊆ t) :
    b.spanIndices s ≤ b.spanIndices t
```

lemma mem_spanIndices_of_mem [source]

A selected basis vector lies in the span of its index set.

```
lemma mem_spanIndices_of_mem (b : OrthonormalBasis ι 𝕜 E) {s : Set ι} {i : ι}
    (hi : i ∈ s) : b i ∈ b.spanIndices s
```

lemma repr_eq_zero_of_mem_spanIndices [source]

A vector in the span of a selected subfamily has zero coordinate at any index outside the selection.

```
lemma repr_eq_zero_of_mem_spanIndices (b : OrthonormalBasis ι 𝕜 E)
    {s : Set ι} {x : E} (hx : x ∈ b.spanIndices s) {i : ι} (hi : i ∉ s) :
    b.repr x i = 0
```

lemma mem_spanIndices_iff [source]

Membership in the span of a selected subfamily is exactly the vanishing of the coordinates outside the selection.

```
@[simp]
lemma mem_spanIndices_iff (b : OrthonormalBasis ι 𝕜 E)
    {s : Set ι} {x : E} :
    x ∈ b.spanIndices s ↔ ∀ i ∉ s, b.repr x i = 0
```

lemma finrank_spanIndices [source]

The span of a selected subfamily has dimension the number of selected indices.

```
lemma finrank_spanIndices (b : OrthonormalBasis ι 𝕜 E) (s : Finset ι) :
    finrank 𝕜 (b.spanIndices ↑s) = s.card
```

lemma finrank_spanIndices_set [source]

The span of a selected set of basis vectors has dimension equal to the number of selected indices.

```
lemma finrank_spanIndices_set (b : OrthonormalBasis ι 𝕜 E) (s : Set ι) :
    finrank 𝕜 (b.spanIndices s) = s.ncard
```

lemma orthogonal_spanIndices [source]

The orthogonal complement of the span of a selected subfamily is the span of the complementary subfamily.

```
lemma orthogonal_spanIndices (b : OrthonormalBasis ι 𝕜 E) (s : Set ι) :
    (b.spanIndices s)ᗮ = b.spanIndices sᶜ
```

2.364 PhyslibAlpha.Mathematics.TauCeti.Analysis.InnerProductSpace.CourantFischer

PhyslibAlpha/Mathematics/TauCeti/Analysis/InnerProductSpace/CourantFischer.lean on GitHub.

lemma sum_sq_norm_repr_eq_sq_norm [source]

Parseval: in an orthonormal basis the squared norms of the coordinates sum to the squared norm. Thin wrapper around `OrthonormalBasis.sum_sq_norm_inner_right`.

```
private lemma sum_sq_norm_repr_eq_sq_norm (b : OrthonormalBasis (Fin n) 𝕜 E) (x : E) :
    ∑ i : Fin n, ‖b.repr x i‖ ^ 2 = ‖x‖ ^ 2
```

lemma re_inner_apply_self_eq_sum_of_eigenbasis [source]

Diagonalization of the quadratic form in any orthonormal eigenbasis: if `S (w i) = μ i • w i` for all `i`, then `re ⟪S x, x⟫ = ∑ i, μ i * ‖w.repr x i‖ ^ 2`.

```
lemma re_inner_apply_self_eq_sum_of_eigenbasis
    (hS : S.IsSymmetric) (w : OrthonormalBasis (Fin n) 𝕜 E) {μ : Fin n → ℝ}
    (hw : ∀ i, S (w i) = (μ i : 𝕜) • w i) (x : E) :
    RCLike.re ⟪S x, x⟫_𝕜 = ∑ i : Fin n, μ i * ‖w.repr x i‖ ^ 2
```

lemma le_re_inner_of_eigenbasis [source]

On the span of the eigenvectors selected by `s`, the quadratic form is bounded below by any lower bound on the selected values (general-eigenbasis version of `le_re_inner_apply_self_of_mem_spanI`

```
lemma le_re_inner_of_eigenbasis
    (hS : S.IsSymmetric) (w : OrthonormalBasis (Fin n) 𝕜 E) {μ : Fin n → ℝ}
    (hw : ∀ i, S (w i) = (μ i : 𝕜) • w i) {s : Set (Fin n)} {c : ℝ}
    (hc : ∀ i ∈ s, c ≤ μ i) {x : E} (hx : x ∈ w.spanIndices s) :
    c * ‖x‖ ^ 2 ≤ RCLike.re ⟪S x, x⟫_𝕜
```

lemma re_inner_le_of_eigenbasis [source]

Dual of `le_re_inner_of_eigenbasis`.

```
lemma re_inner_le_of_eigenbasis
    (hS : S.IsSymmetric) (w : OrthonormalBasis (Fin n) 𝕜 E) {μ : Fin n → ℝ}
    (hw : ∀ i, S (w i) = (μ i : 𝕜) • w i) {s : Set (Fin n)} {c : ℝ}
    (hc : ∀ i ∈ s, μ i ≤ c) {x : E} (hx : x ∈ w.spanIndices s) :
    RCLike.re ⟪S x, x⟫_𝕜 ≤ c * ‖x‖ ^ 2
```

lemma re_inner_apply_self_eq_sum_eigenvalues_mul_sq [source]

The quadratic form `re ⟪T x, x⟫` of a symmetric operator `T` expressed in its eigenbasis: it is the eigenvalue-weighted sum of the squared norms of the coordinates of `x`. This is the diagonalization of the quadratic form. (For symmetric `T` the inner product `⟪T x, x⟫` is real, so no information is lost by taking the real part.)

```
lemma re_inner_apply_self_eq_sum_eigenvalues_mul_sq
    (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n) (x : E) :
    RCLike.re ⟪T x, x⟫_𝕜
      = ∑ i : Fin n, hT.eigenvalues hn i * ‖(hT.eigenvectorBasis hn).repr x i‖ ^ 2
```

lemma re_inner_apply_self_le_of_mem_spanIndices [source]

On the span of a selected subfamily of the eigenbasis, the quadratic form is bounded by any bound on the selected eigenvalues: if `x ∈ b.spanIndices s` (so its coordinates vanish off `s`) and every selected eigenvalue satisfies $\lambda_i \le c$, then `re ⟪T x, x⟫` $\le c\,\|x\|^2$.

```
lemma re_inner_apply_self_le_of_mem_spanIndices
    (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n) {s : Set (Fin n)} {c : ℝ}
    (hc : ∀ i ∈ s, hT.eigenvalues hn i ≤ c)
    {x : E} (hx : x ∈ (hT.eigenvectorBasis hn).spanIndices s) :
    RCLike.re ⟪T x, x⟫_𝕜 ≤ c * ‖x‖ ^ 2
```

lemma le_re_inner_apply_self_of_mem_spanIndices [source]

Dual of `re_inner_apply_self_le_of_mem_spanIndices`: if `x ∈ b.spanIndices s` and every selected eigenvalue satisfies $c \le \lambda_i$, then $c\,\|x\|^2 \le$ `re ⟪T x, x⟫`.

```
lemma le_re_inner_apply_self_of_mem_spanIndices
    (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n) {s : Set (Fin n)} {c : ℝ}
    (hc : ∀ i ∈ s, c ≤ hT.eigenvalues hn i)
    {x : E} (hx : x ∈ (hT.eigenvectorBasis hn).spanIndices s) :
    c * ‖x‖ ^ 2 ≤ RCLike.re ⟪T x, x⟫_𝕜
```

lemma exists_unit_vector_re_inner_le_eigenvalue [source]

***Courant–Fischer, upper direction.** On any subspace `V` of dimension `k + 1` there is a unit vector `x` with `re ⟪T x, x⟫` $\le \lambda_k(T)$, where λ is the decreasing enumeration `LinearMap.IsSymmetric.eigenvalues` of the eigenvalues of the symmetric operator `T`.*

```
lemma exists_unit_vector_re_inner_le_eigenvalue
    (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n) (k : Fin n)
    (V : Submodule 𝕜 E) (hV : finrank 𝕜 V = (k : ℕ) + 1) :
    ∃ x ∈ V, ‖x‖ = 1 ∧ RCLike.re ⟪T x, x⟫_𝕜 ≤ hT.eigenvalues hn k
```

lemma exists_submodule_forall_unit_eigenvalue_le_re_inner [source]

***Courant–Fischer, lower direction.** There is a subspace `V` of dimension `k + 1` on which every unit vector `x` satisfies `λₖ(T) ≤ re ⟪T x, x⟫`, where `λ` is the decreasing enumeration `LinearMap.IsSymmetric.eigenvalues` of the eigenvalues of the symmetric operator `T`.*

Witness: `V = span {bᵢ : i ≤ k}`; on it the quadratic form is bounded below by `λₖ` since all involved eigenvalues are `≥ λₖ`.

```
lemma exists_submodule_forall_unit_eigenvalue_le_re_inner
    (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n) (k : Fin n) :
    ∃ V : Submodule 𝕜 E, finrank 𝕜 V = (k : ℕ) + 1 ∧
      ∀ x ∈ V, ‖x‖ = 1 → hT.eigenvalues hn k ≤ RCLike.re ⟪T x, x⟫_𝕜
```

lemma eigenvalues_eq_iSup_iInf_re_inner [source]

***Courant–Fischer min-max equality.** The `k`-th (decreasingly sorted) eigenvalue of a symmetric operator on a finite-dimensional inner product space over `𝕜 = ℝ, ℂ` is the supremum, over the subspaces `V` of dimension `k + 1`, of the infimum of the Rayleigh quotient `re ⟪T x, x⟫` over the unit vectors of `V`.*

*Horn & Johnson, *Matrix Analysis* 2nd ed., Theorem 4.2.6.*

```
lemma eigenvalues_eq_iSup_iInf_re_inner
    (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n) (k : Fin n) :
    hT.eigenvalues hn k =
      ⨆ V : {V : Submodule 𝕜 E // finrank 𝕜 V = (k : ℕ) + 1},
        ⨅ x : {x : E // x ∈ (V : Submodule 𝕜 E) ∧ ‖x‖ = 1},
          RCLike.re ⟪T (x : E), (x : E)⟫_𝕜
```

lemma eigenvalues_eq_of_eigenbasis [source]

***Sorted-eigenvalue uniqueness.** If an orthonormal basis `w` diagonalizes the symmetric operator `S` with an *antitone* value list `μ`, then `μ` is the sorted eigenvalue list: `hS.eigenvalues hn = μ`. (Courant–Fischer: both lists satisfy the same minimax characterization.)*

```
lemma eigenvalues_eq_of_eigenbasis
    (hS : S.IsSymmetric) (hn : finrank 𝕜 E = n) (w : OrthonormalBasis (Fin n) 𝕜 E)
    {μ : Fin n → ℝ} (hμ : Antitone μ) (hw : ∀ i, S (w i) = (μ i : 𝕜) • w i) :
    hS.eigenvalues hn = μ
```

lemma eigenvalue_mono [source]

***Loewner monotonicity of the sorted eigenvalues.** If the quadratic form of `T` is dominated by that of `S`, then so is every sorted eigenvalue.*

```
lemma eigenvalue_mono
    (hT : T.IsSymmetric) (hS : S.IsSymmetric) (hn : finrank 𝕜 E = n)
    (h : ∀ x, RCLike.re ⟪T x, x⟫_𝕜 ≤ RCLike.re ⟪S x, x⟫_𝕜) (k : Fin n) :
    hT.eigenvalues hn k ≤ hS.eigenvalues hn k
```

lemma map_mem_spanIndices [source]

The span of a selected subfamily of the eigenbasis of a symmetric operator is invariant under the operator.

```
lemma map_mem_spanIndices (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n)
    (s : Set (Fin n)) {x : E}
    (hx : x ∈ (hT.eigenvectorBasis hn).spanIndices s) :
    T x ∈ (hT.eigenvectorBasis hn).spanIndices s
```

lemma eigenvalues_sub_le [source]

One-sided Weyl bound: `λₖ(S) − λₖ(T) ≤ ‖S − T‖op`. This is the core estimate; Weyl's inequality follows by symmetry.

We take a witness subspace `V` of dimension `k + 1` on which `λₖ(S) ≤ re ⟪S x, x⟫` (lower direction for `S`), then a unit vector `x ∈ V` with `re ⟪T x, x⟫ ≤ λₖ(T)` (upper direction for `T`). The difference is controlled by Cauchy–Schwarz.

```
private lemma eigenvalues_sub_le
    (hT : T.IsSymmetric) (hS : S.IsSymmetric) (hn : finrank 𝕜 E = n)
    {ε : ℝ} (hε : ∀ x : E, ‖(S - T) x‖ ≤ ε * ‖x‖) (k : Fin n) :
    hS.eigenvalues hn k - hT.eigenvalues hn k ≤ ε
```

lemma abs_eigenvalue_sub_eigenvalue_le [source]

***Weyl's inequality** for symmetric operators on a finite-dimensional inner product space over `𝕜 = ℝ, ℂ`: the `k`-th (decreasingly sorted) eigenvalues of `T` and `S` differ by at most the operator norm of `T − S`.*

At `LinearMap` level there is no operator norm, so the bound is supplied as the pointwise hypothesis `∀ x, ‖(T − S) x‖ ≤ ε * ‖x‖`; see `LinearMap.IsSymmetric.abs_eigenvalue_sub_eigenvalue_le_n` for the continuous-linear-map operator-norm form.

*Horn & Johnson, *Matrix Analysis* 2nd ed., Theorem 4.3.1; Bhatia, *Matrix Analysis*, Corollary III.2.6.*

```
lemma abs_eigenvalue_sub_eigenvalue_le
    (hT : T.IsSymmetric) (hS : S.IsSymmetric) (hn : finrank 𝕜 E = n)
    {ε : ℝ} (hε : ∀ x : E, ‖(T - S) x‖ ≤ ε * ‖x‖) (k : Fin n) :
    |hT.eigenvalues hn k - hS.eigenvalues hn k| ≤ ε
```

lemma abs_eigenvalue_sub_eigenvalue_le_norm [source]

***Weyl's inequality**, operator-norm form, for symmetric (equivalently, self-adjoint) continuous linear maps on a finite-dimensional inner product space: the `k`-th sorted eigenvalues of `T` and `S` differ by at most `‖T − S‖`. The symmetry hypotheses are stated on the underlying linear maps so that the signature does not require the adjoint star structure (whose instance needs `CompleteSpace E`, which `FiniteDimensional` deliberately does not register as an instance).*

The bound is expressed directly in the norm of a continuous linear map.

```
lemma abs_eigenvalue_sub_eigenvalue_le_norm
    {T S : E →L[𝕜] E}
    (hT : LinearMap.IsSymmetric (T : E →ₗ[𝕜] E))
    (hS : LinearMap.IsSymmetric (S : E →ₗ[𝕜] E))
    (hn : finrank 𝕜 E = n) (k : Fin n) :
```

```
    |hT.eigenvalues hn k - hS.eigenvalues hn k| ≤ ‖T - S‖
```

lemma abs_eigenvalues_sub_le_opNorm [source]

Weyl's inequality for linear maps, using the operator norm of the corresponding continuous linear map. Unlike the preceding theorem, the symmetry hypotheses and eigenvalue enumeration are directly attached to `LinearMap`s.

```
lemma abs_eigenvalues_sub_le_opNorm
    {T S : E →ₗ[𝕜] E} (hT : T.IsSymmetric) (hS : S.IsSymmetric)
    (hn : finrank 𝕜 E = n) (k : Fin n) :
    |hT.eigenvalues hn k - hS.eigenvalues hn k|
      ≤ ‖LinearMap.toContinuousLinearMap (T - S)‖
```

lemma eigenvalues_congr [source]

Sorted eigenvalues are congruent along an operator equality (the eigenvalue enumeration depends only on the operator, not on the symmetry proof).

```
lemma eigenvalues_congr {S₁ S₂ : E →ₗ[𝕜] E} (h : S₁ = S₂)
    (hS₁ : S₁.IsSymmetric) (hS₂ : S₂.IsSymmetric) (hn : finrank 𝕜 E = n) :
    hS₁.eigenvalues hn = hS₂.eigenvalues hn
```

lemma eigenvalues_cast [source]

The eigenvalue enumeration does not depend on which witness of the dimension indexes it: two witnesses `finrank 𝕜 E = m` and `finrank 𝕜 E = n` enumerate the same eigenvalues, read across the induced `Fin m ≃ Fin n`.

Both spellings occur in practice. A matrix over `Fin n` has `Matrix.IsHermitian.eigenvalues₀` indexed by `Fin (Fintype.card (Fin n))`, while the operator theory it is transported to indexes by `Fin n`; `Fintype.card (Fin n) = n` is a theorem and not definitional, so the two index types are genuinely different.

```
lemma eigenvalues_cast {T : E →ₗ[𝕜] E} (hT : T.IsSymmetric) {m : ℕ}
    (hm : finrank 𝕜 E = m) (hn : finrank 𝕜 E = n) (hmn : m = n) (i : Fin m) :
    hT.eigenvalues hm i = hT.eigenvalues hn (Fin.cast hmn i)
```

lemma exists_unit_vector_re_inner_le_eigenvalue_of_le [source]

***Courant–Fischer, upper direction, for large subspaces.** On any subspace `W` of dimension at least `k + 1` there is a unit vector `x` with `re ⟪T x, x⟫ ≤ λₖ(T)`.*

```
lemma exists_unit_vector_re_inner_le_eigenvalue_of_le
    (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n) (k : Fin n)
    (W : Submodule 𝕜 E) (hW : (k : ℕ) + 1 ≤ finrank 𝕜 W) :
    ∃ x ∈ W, ‖x‖ = 1 ∧ RCLike.re ⟪T x, x⟫_𝕜 ≤ hT.eigenvalues hn k
```

lemma eigenvalue_compression_le [source]

***Upper Cauchy interlacing.** If the quadratic form of the symmetric operator `A` on `V` agrees with that of `T`, then `μₖ(A) ≤ λₖ(T)` for every `k < m`.*

```
lemma eigenvalue_compression_le
    (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n)
```

```
    (hA : A.IsSymmetric) (hm : finrank 𝕜 V = m)
    (hcomp : ∀ y : V, RCLike.re ⟪A y, y⟫_𝕜 = RCLike.re ⟪T y, y⟫_𝕜)
    (k : Fin m) (hk : (k : ℕ) < n) :
    hA.eigenvalues hm k ≤ hT.eigenvalues hn ⟨k, hk⟩
```

lemma eigenvalue_le_eigenvalue_compression [source]

***Lower Cauchy interlacing.** If the quadratic form of the symmetric operator `A` on `V` agrees with that of `T`, then `λ_{k + (n - m)}(T) ≤ μₖ(A)` for every `k < m`.*

```
lemma eigenvalue_le_eigenvalue_compression
    (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n)
    (hA : A.IsSymmetric) (hm : finrank 𝕜 V = m)
    (hcomp : ∀ y : V, RCLike.re ⟪A y, y⟫_𝕜 = RCLike.re ⟪T y, y⟫_𝕜)
    (k : Fin m) (hmn : m ≤ n) :
    hT.eigenvalues hn ⟨k + (n - m), by omega⟩ ≤ hA.eigenvalues hm k
```

lemma eigenvalue_ascending_le_ritz [source]

***Hylleraas–Undheim–MacDonald.** In ascending order, the `k`-th lowest Ritz value is an upper bound for the `k`-th lowest exact eigenvalue: `λ↑ₖ(T) ≤ μ↑ₖ(S)` for every `k < m`.*

```
lemma eigenvalue_ascending_le_ritz
    (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n)
    (hA : A.IsSymmetric) (hm : finrank 𝕜 V = m)
    (hcomp : ∀ y : V, RCLike.re ⟪A y, y⟫_𝕜 = RCLike.re ⟪T y, y⟫_𝕜)
    (hmn : m ≤ n) (k : ℕ) (hk : k < m) :
    hT.eigenvalues hn ⟨n - 1 - k, by omega⟩ ≤ hA.eigenvalues hm ⟨m - 1 - k, by omega⟩
```

lemma eigenvalue_min_le_ritz_min [source]

***Variational ground-state bound.** The lowest Ritz value bounds the lowest exact eigenvalue from above.*

```
lemma eigenvalue_min_le_ritz_min
    (hT : T.IsSymmetric) (hn : finrank 𝕜 E = n)
    (hA : A.IsSymmetric) (hm : finrank 𝕜 V = m)
    (hcomp : ∀ y : V, RCLike.re ⟪A y, y⟫_𝕜 = RCLike.re ⟪T y, y⟫_𝕜)
    (hmn : m ≤ n) (hm0 : 0 < m) :
    hT.eigenvalues hn ⟨n - 1, by omega⟩ ≤ hA.eigenvalues hm ⟨m - 1, by omega⟩
```

def compression [source]

The compression `P_V ∘ T ∘ ι_V` of `T` to the subspace `V` (the Ritz operator).

```
noncomputable def compression (_hT : T.IsSymmetric) (V : Submodule 𝕜 E) : V →ₗ[𝕜] V
```

lemma inner_compression_left [source]

The compression has the inner products of `T` against vectors of `V`.

```
lemma inner_compression_left (hT : T.IsSymmetric) (V : Submodule 𝕜 E) (y z : V) :
    ⟪hT.compression V y, z⟫_𝕜 = ⟪T y, z⟫_𝕜
```

lemma compression_isSymmetric [source]

The compression of a symmetric operator is symmetric.

```
lemma compression_isSymmetric (hT : T.IsSymmetric) (V : Submodule 𝕜 E) :
    (hT.compression V).IsSymmetric
```

lemma re_inner_compression [source]

The compression has the quadratic form of `T` on `V`, as required by the interlacing lemmas.

```
lemma re_inner_compression (hT : T.IsSymmetric) (V : Submodule 𝕜 E) (y : V) :
    RCLike.re ⟪hT.compression V y, y⟫_𝕜 = RCLike.re ⟪T y, y⟫_𝕜
```

2.365 PhyslibAlpha.Mathematics.TauCeti.Analysis.InnerProductSpace.LinearCombination

PhyslibAlpha/Mathematics/TauCeti/Analysis/InnerProductSpace/LinearCombination.lean on GitHub.

lemma norm_linearCombination_eq_of_inner_eq [source]

Families with equal pairwise inner products give linear combinations of equal norm for the same finitely supported coefficients.

```
lemma norm_linearCombination_eq_of_inner_eq (f : α →₀ 𝕜) {φ : α → E} {ψ : α → F}
    (h : ∀ a b, ⟪φ a, φ b⟫_𝕜 = ⟪ψ a, ψ b⟫_𝕜) :
    ‖linearCombination 𝕜 φ f‖ = ‖linearCombination 𝕜 ψ f‖
```

2.366 PhyslibAlpha.Mathematics.TauCeti.Analysis.Matrix.PosSemidef

PhyslibAlpha/Mathematics/TauCeti/Analysis/Matrix/PosSemidef.lean on GitHub.

lemma finTwo_posSemidef [source]

The `2 × 2` principal submatrix at two indices is positive semidefinite.

```
private lemma finTwo_posSemidef {K : α → α → 𝕜}
    (hK : Matrix.PosSemidef K) (a b : α) :
    (Matrix.of fun i j : Fin 2 => K (![a, b] i) (![a, b] j)).PosSemidef
```

lemma normSq_le [source]

Scalar Cauchy–Schwarz for an `RCLike`-valued positive-semidefinite matrix.

```
lemma normSq_le {K : α → α → 𝕜}
    (hK : Matrix.PosSemidef K) (a b : α) :
    RCLike.normSq (K a b) ≤ RCLike.re (K a a) * RCLike.re (K b b)
```

lemma eq_zero_of_apply_self_eq_zero_left [source]

A zero diagonal entry forces the corresponding row entry to vanish.

```
lemma eq_zero_of_apply_self_eq_zero_left {K : α → α → 𝕜}
    (hK : Matrix.PosSemidef K) {a b : α} (ha : K a a = 0) : K a b = 0
```

lemma eq_zero_of_apply_self_eq_zero_right [source]

A zero diagonal entry forces the corresponding column entry to vanish.

```
lemma eq_zero_of_apply_self_eq_zero_right {K : α → α → 𝕜}
    (hK : Matrix.PosSemidef K) {a b : α} (hb : K b b = 0) : K a b = 0
```

lemma norm_le_one_of_apply_self_eq_one [source]

If two diagonal entries are `1`, then the corresponding off-diagonal entry has norm at most `1`.

```
lemma norm_le_one_of_apply_self_eq_one {K : α → α → 𝕜}
    (hK : Matrix.PosSemidef K) {a b : α} (ha : K a a = 1) (hb : K b b = 1) :
    ‖K a b‖ ≤ 1
```

lemma norm_le_of_norm_apply_self_le [source]

***The diagonal of a positive-semidefinite matrix bounds all of its entries.** Cauchy–Schwarz turns a uniform bound on the (nonnegative real) diagonal entries into the same bound everywhere.*

```
lemma norm_le_of_norm_apply_self_le {K : α → α → 𝕜} {C : ℝ}
    (hK : Matrix.PosSemidef K) (hdiag : ∀ i, ‖K i i‖ ≤ C) (a b : α) : ‖K a b‖ ≤ C
```

lemma apply_one_le_apply_zero_of_sq_le_mul [source]

The numerical core of the Hankel estimate: a bounded sequence of reals with `0 ≤ b 0` whose squares satisfy the Cauchy–Schwarz inequality `b n ^ 2 ≤ b 0 * b (n + n)` cannot increase at the first step. If it did, the ratio `r = b 1 / b 0` would exceed `1` and the doubling inequality would push `b (2 ^ k)` past `b 0 * r ^ (2 ^ k)`, which is unbounded.

```
private lemma apply_one_le_apply_zero_of_sq_le_mul {b : ℕ → ℝ} {D : ℝ} (hb0 : 0 ≤ b 0)
    (hsq : ∀ n, b n ^ 2 ≤ b 0 * b (n + n)) (hbd : ∀ n, b n ≤ D) : b 1 ≤ b 0
```

lemma sub_nonneg_of_posSemidef_hankel [source]

***A positive-semidefinite Hankel sequence bounded above does not increase at the first step.** If `(m, n) ↦ a (m + n)` is positive semidefinite and the real parts of `a` are bounded above by `D` — the entries are automatically real — then `a 1 ≤ a 0` in the `RCLike` order. Cauchy–Schwarz alone gives `a n ^ 2 ≤ a 0 * a (2 n)`, and the bound rules out a ratio `a 1 / a 0` greater than `1`.*

```
lemma sub_nonneg_of_posSemidef_hankel {a : ℕ → 𝕜} {D : ℝ}
    (ha : Matrix.PosSemidef fun m n : ℕ => a (m + n)) (hbd : ∀ n, RCLike.re (a n) ≤ D) :
    0 ≤ a 0 - a 1
```

2.367 PhyslibAlpha.Mathematics.TauCeti.Analysis.Normed.Operator.Dense

PhyslibAlpha/Mathematics/TauCeti/Analysis/Normed/Operator/Dense.lean on GitHub.

lemma norm_sub_apply_le_of_norm_le [source]

Comparing two continuous linear maps of norm at most `K` at `x` costs at most `2 K ‖x - y‖` more than comparing them at `y`. This is the shared estimate of the two density arguments below: `y` is chosen in the dense subset, close to the arbitrary vector `x`.

```
lemma norm_sub_apply_le_of_norm_le {T S : X →L[𝕜] Y} {K : ℝ}
    (hT : ‖T‖ ≤ K) (hS : ‖S‖ ≤ K)
    (x y : X) : ‖T x - S x‖ ≤ ‖T y - S y‖ + 2 * K * ‖x - y‖
```

lemma tendsto_apply_of_dense [source]

A uniformly bounded family of continuous linear maps that converges pointwise on a dense subset converges pointwise everywhere.

```
lemma tendsto_apply_of_dense {ι : Type*} {l : Filter ι} {D : Set X}
    (hD : Dense D) {f : ι → X →L[𝕜] Y} {g : X →L[𝕜] Y} {C : ℝ}
    (hbound : ∀ᶠ i in l, ‖f i‖ ≤ C)
    (htendsto : ∀ x ∈ D, Tendsto (fun i => f i x) l (nhds (g x))) (x : X) :
    Tendsto (fun i => f i x) l (nhds (g x))
```

lemma uniformCauchySeqOn_apply_of_dense [source]

A uniformly bounded family of continuous linear maps that is uniformly Cauchy on a parameter set at every vector in a dense subset is uniformly Cauchy there at every vector.

```
lemma uniformCauchySeqOn_apply_of_dense {ι Z : Type*} {p : Filter ι}
    {D : Set X}
    (hD : Dense D) {f : ι → Z → X →L[𝕜] Y} {s : Set Z} {C : ℝ}
    (hbound : ∀ᶠ i in p, ∀ z ∈ s, ‖f i z‖ ≤ C)
    (hcauchy : ∀ x ∈ D, UniformCauchySeqOn (fun i z => f i z x) p s) (x : X) :
    UniformCauchySeqOn (fun i z => f i z x) p s
```

2.368 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.AddGroup

PhyslibAlpha/Mathematics/TauCeti/Analysis/PositiveDefinite/AddGroup.lean on GitHub.

instance AddCommGroup (WithNegStar G) [source]

```
instance : AddCommGroup (WithNegStar G)
```

instance Star (WithNegStar G) [source]

```
instance : Star (WithNegStar G)
```

lemma star_eq_neg [source]

```
@[simp]
lemma star_eq_neg (a : WithNegStar G) : star a = -a
```

instance StarAddMonoid (WithNegStar G) [source]

```
instance : StarAddMonoid (WithNegStar G) where
```

def toNegStar [source]

The identity additive equivalence from `G` to its negation-involution copy.

```
def toNegStar : G ≃+ WithNegStar G
```

def ofNegStar [source]

The identity additive equivalence from the negation-involution copy of `G` back to `G`.

```
def ofNegStar : WithNegStar G ≃+ G
```

lemma ofNegStar_toNegStar [source]

```
@[simp]
lemma ofNegStar_toNegStar (a : G) : ofNegStar (toNegStar a) = a
```

lemma toNegStar_ofNegStar [source]

```
@[simp]
lemma toNegStar_ofNegStar (a : WithNegStar G) : toNegStar (ofNegStar a) = a
```

instance SeminormedAddCommGroup (WithNegStar G) [source]

```
instance : SeminormedAddCommGroup (WithNegStar G)
```

def IsPositiveDefiniteSub [source]

*A function `F : G → ℂ` on an additive commutative group is **positive definite** when, for every finite family of scalars `c : Fin n → ℂ` and points `v : Fin n → G`, the Hermitian form `∑_{i,j} c i · conj (c j) * F (v i - v j)` is a nonnegative real number (using the order on ℂ for which `0 ≤ z` means `z` is real and nonnegative).*

This is the classical translation-invariant condition, the one Bochner's theorem is stated in. It is `TauCeti.IsPositiveDefinite` for the negation involution; see `TauCeti.isPositiveDefiniteSub_iff_isPositiveDefinite`.

```
def IsPositiveDefiniteSub (F : G → ℂ) : Prop
```

lemma isPositiveDefiniteSub_iff_forall_sum_nonneg [source]

The defining finite-family characterization, for building a positive-definite function directly from the quadratic-form condition.

```
lemma isPositiveDefiniteSub_iff_forall_sum_nonneg :
    IsPositiveDefiniteSub F ↔ ∀ (n : ℕ) (c : Fin n → ℂ) (v : Fin n → G),
      0 ≤ ∑ i, ∑ j, c i * conj (c j) * F (v i - v j)
```

lemma isPositiveDefiniteSub_iff_isPositiveDefinite [source]

Subtraction-form positive definiteness is the generic involutive predicate for the negation involution, read on the type synonym `TauCeti.WithNegStar`.

```
lemma isPositiveDefiniteSub_iff_isPositiveDefinite :
    IsPositiveDefiniteSub F ↔
      IsPositiveDefinite fun a : WithNegStar G => F (WithNegStar.ofNegStar a)
```

lemma isPositiveDefinite [source]

The generic involutive predicate attached to a subtraction-form positive-definite function. This is the workhorse for transporting the generic API.

```
lemma isPositiveDefinite (hF : IsPositiveDefiniteSub F) :
    IsPositiveDefinite fun a : WithNegStar G => F (WithNegStar.ofNegStar a)
```

lemma sum_nonneg [source]

Positive-definiteness holds for an arbitrary finite index type, not just `Fin n`.

```
lemma sum_nonneg (hF : IsPositiveDefiniteSub F) {ι : Type*} [Fintype ι] (c : ι → ℂ) (v : ι → G)
    :
    0 ≤ ∑ i, ∑ j, c i * conj (c j) * F (v i - v j)
```

lemma posSemidef [source]

A subtraction-form positive-definite function gives the translation-invariant positive-definite kernel `K(a, b) = F (a - b)`.

```
lemma posSemidef (hF : IsPositiveDefiniteSub F) :
    Matrix.PosSemidef fun a b : G => F (a - b)
```

lemma of_posSemidef [source]

A function whose translation-invariant kernel `K(a, b) = F (a - b)` is positive definite is positive definite.

```
lemma of_posSemidef (hK : Matrix.PosSemidef fun a b : G => F (a - b)) :
    IsPositiveDefiniteSub F
```

lemma isPositiveDefiniteSub_iff_posSemidef [source]

The PD-function ↔ PD-kernel equivalence in its classical translation-invariant form: `F` is positive definite if and only if the kernel `K(a, b) = F (a - b)` is positive definite.

```
lemma isPositiveDefiniteSub_iff_posSemidef :
    IsPositiveDefiniteSub F ↔ Matrix.PosSemidef fun a b : G => F (a - b)
```

lemma isPositiveDefinite_iff_isPositiveDefiniteSub [source]

On a group whose own involution is negation, the generic involutive predicate and the subtraction-form predicate agree.

```
lemma isPositiveDefinite_iff_isPositiveDefiniteSub [StarAddMonoid G]
    (hstar : ∀ a : G, star a = -a) : IsPositiveDefinite F ↔ IsPositiveDefiniteSub F
```

lemma map_zero_nonneg [source]

The value of a positive-definite function at `0` is real and nonnegative.

```
lemma map_zero_nonneg (hF : IsPositiveDefiniteSub F) : 0 ≤ F 0
```

lemma map_zero_im [source]

The value of a positive-definite function at `0` has zero imaginary part.

```
@[simp]
lemma map_zero_im (hF : IsPositiveDefiniteSub F) : (F 0).im = 0
```

lemma map_zero_re_nonneg [source]

The real part of the value of a positive-definite function at `0` is nonnegative.

```
lemma map_zero_re_nonneg (hF : IsPositiveDefiniteSub F) : 0 ≤ (F 0).re
```

lemma map_zero_eq_ofReal_re [source]

The value at the origin of a positive-definite function is the real number `(F 0).re`, viewed as a complex number.

```
lemma map_zero_eq_ofReal_re (hF : IsPositiveDefiniteSub F) : F 0 = ((F 0).re : ℂ)
```

lemma map_zero_re_pos_of_ne_zero [source]

If a positive-definite function is nonzero at the origin, then the real part of that value is strictly positive.

```
lemma map_zero_re_pos_of_ne_zero (hF : IsPositiveDefiniteSub F) (h0 : F 0 ≠ 0) :
    0 < (F 0).re
```

lemma conj_symm [source]

A positive-definite function is conjugate symmetric: `conj (F (b - a)) = F (a - b)`.

```
@[simp]
lemma conj_symm (hF : IsPositiveDefiniteSub F) (a b : G) : conj (F (b - a)) = F (a - b)
```

lemma map_neg [source]

A positive-definite function satisfies `F (-a) = conj (F a)`.

```
lemma map_neg (hF : IsPositiveDefiniteSub F) (a : G) : F (-a) = conj (F a)
```

lemma normSq_le [source]

The Cauchy–Schwarz inequality for a positive-definite function: the squared norm of any value is bounded by the square of the value at the origin.

```
lemma normSq_le (hF : IsPositiveDefiniteSub F) (a b : G) :
    Complex.normSq (F (a - b)) ≤ (F 0).re * (F 0).re
```

lemma norm_apply_le_map_zero_re [source]

A positive-definite function is bounded by its value at the origin.

```
lemma norm_apply_le_map_zero_re (hF : IsPositiveDefiniteSub F) (a : G) : ‖F a‖ ≤ (F 0).re
```

lemma apply_eq_zero_of_map_zero_re_eq_zero [source]

A positive-definite function with `(F 0).re = 0` vanishes identically.

```
lemma apply_eq_zero_of_map_zero_re_eq_zero (hF : IsPositiveDefiniteSub F) (h0 : (F 0).re = 0)
    (a : G) : F a = 0
```

lemma add [source]

Positive-definite functions are closed under addition.

```
lemma add (hF : IsPositiveDefiniteSub F) (hH : IsPositiveDefiniteSub H) :
    IsPositiveDefiniteSub fun x => F x + H x
```

lemma const_mul [source]

Positive-definite functions are closed under multiplication by a nonnegative complex scalar.

```
lemma const_mul {k : ℂ} (hk : 0 ≤ k) (hF : IsPositiveDefiniteSub F) :
    IsPositiveDefiniteSub fun x => k * F x
```

lemma real_smul [source]

Positive-definite functions are closed under multiplication by a nonnegative real scalar.

```
lemma real_smul {r : ℝ} (hr : 0 ≤ r) (hF : IsPositiveDefiniteSub F) :
    IsPositiveDefiniteSub fun x => r • F x
```

lemma mul [source]

Positive-definite functions are closed under pointwise multiplication (Schur product).

```
lemma mul (hF : IsPositiveDefiniteSub F) (hH : IsPositiveDefiniteSub H) :
    IsPositiveDefiniteSub fun x => F x * H x
```

lemma sum [source]

Positive-definite functions are closed under finite sums.

```
lemma sum {ι : Type*} {s : Finset ι} {F : ι → G → ℂ}
    (hF : ∀ i ∈ s, IsPositiveDefiniteSub (F i)) :
    IsPositiveDefiniteSub fun x => ∑ i ∈ s, F i x
```

lemma prod [source]

Positive-definite functions are closed under finite products (Schur products).

```
lemma prod {ι : Type*} {s : Finset ι} {F : ι → G → ℂ}
    (hF : ∀ i ∈ s, IsPositiveDefiniteSub (F i)) :
    IsPositiveDefiniteSub fun x => ∏ i ∈ s, F i x
```

lemma comp_addMonoidHom [source]

Positive definiteness is preserved by precomposition with an additive homomorphism; no compatibility with an involution is needed, since an additive homomorphism automatically commutes with negation.

```
lemma comp_addMonoidHom {N : Type*} [AddCommGroup N] (hF : IsPositiveDefiniteSub F)
    (φ : N →+ G) : IsPositiveDefiniteSub fun x => F (φ x)
```

lemma comp_smul [source]

Positive definiteness is preserved by rescaling the argument.

```
lemma comp_smul {R : Type*} [DistribSMul R G] (hF : IsPositiveDefiniteSub F) (r : R) :
    IsPositiveDefiniteSub fun x => F (r • x)
```

lemma comp_neg [source]

Positive definiteness is preserved by negating the argument.

```
lemma comp_neg (hF : IsPositiveDefiniteSub F) : IsPositiveDefiniteSub fun x => F (-x)
```

lemma of_tendsto [source]

Positive definiteness is preserved under pointwise limits along a nontrivial filter.

```
lemma of_tendsto {ι : Type*} {l : Filter ι} [NeBot l] {F : ι → G → ℂ} {H : G → ℂ}
    (hF : ∀ᶠ i in l, IsPositiveDefiniteSub (F i))
    (hlim : ∀ x : G, Tendsto (fun i => F i x) l (𝓝 (H x))) :
    IsPositiveDefiniteSub H
```

lemma normalize [source]

Multiplying a positive-definite function by the reciprocal of its real value at the origin preserves positive definiteness.

```
lemma normalize (hF : IsPositiveDefiniteSub F) :
    IsPositiveDefiniteSub fun x => (((F 0).re)⁻¹ : ℂ) * F x
```

lemma normalize_apply_zero [source]

The normalized function has value `1` at the origin.

```
@[simp]
lemma normalize_apply_zero (hF : IsPositiveDefiniteSub F) (h0 : F 0 ≠ 0) :
    (((F 0).re)⁻¹ : ℂ) * F 0 = 1
```

lemma norm_normalize_apply_le_one [source]

A normalized positive-definite function is bounded by `1`.

```
lemma norm_normalize_apply_le_one (hF : IsPositiveDefiniteSub F) (a : G) :
    ‖(((F 0).re)⁻¹ : ℂ) * F a‖ ≤ 1
```

lemma uniformContinuous_of_continuousAt_zero [source]

A positive-definite function on a seminormed additive commutative group is uniformly continuous as soon as it is continuous at the origin.

```
lemma uniformContinuous_of_continuousAt_zero (hF : IsPositiveDefiniteSub F)
    (hcont : ContinuousAt F 0) : UniformContinuous F
```

lemma continuous_of_continuousAt_zero [source]

A positive-definite function on a seminormed additive commutative group is continuous as soon as it is continuous at the origin.

```
lemma continuous_of_continuousAt_zero (hF : IsPositiveDefiniteSub F)
    (hcont : ContinuousAt F 0) : Continuous F
```

lemma isPositiveDefiniteSub_const [source]

A nonnegative real constant is a positive-definite function.

```
lemma isPositiveDefiniteSub_const {k : ℂ} (hk : 0 ≤ k) :
    IsPositiveDefiniteSub (fun _ : G => k)
```

lemma isPositiveDefiniteSub_zero [source]

The zero function is positive definite.

```
lemma isPositiveDefiniteSub_zero : IsPositiveDefiniteSub (fun _ : G => (0 : ℂ))
```

2.369 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.AdditiveCharacter

PhyslibAlpha/Mathematics/TauCeti/Analysis/PositiveDefinite/AdditiveCharacter.lean on GitHub.

lemma isPositiveDefiniteSub [source]

Precomposing a Pontryagin character with `Multiplicative.ofAdd` gives a continuous function in subtraction-positive-definite form.

```
lemma isPositiveDefiniteSub
    (χ : PontryaginDual (Multiplicative G)) :
    Continuous (fun g : G => (χ (Multiplicative.ofAdd g) : ℂ)) ∧
      IsPositiveDefiniteSub
        (fun g : G => (χ (Multiplicative.ofAdd g) : ℂ))
```

2.370 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Basic

PhyslibAlpha/Mathematics/TauCeti/Analysis/PositiveDefinite/Basic.lean on GitHub.

lemma sum_nonneg [source]

Positive-definiteness holds for an arbitrary finite index type, not just `Fin n`: for every finite family of scalars `c : ι → ℂ` and points `v : ι → M`, the Hermitian form `∑_{i,j} c i · conj (c j) · F (v i + star (v j))` is a nonnegative real number.

```
lemma sum_nonneg (hF : IsPositiveDefinite F) {ι : Type*} [Fintype ι] (c : ι → ℂ) (v : ι → M) :
    0 ≤ ∑ i, ∑ j, c i * conj (c j) * F (v i + star (v j))
```

lemma quadForm_two_nonneg [source]

The `2 × 2` Hermitian sub-form of a positive-definite function at the points `a, b` with coefficients `c₀, c₁` is nonnegative.

```
lemma quadForm_two_nonneg (hF : IsPositiveDefinite F) (a b : M) (c₀ c₁ : ℂ) :
    0 ≤ c₀ * conj c₀ * F (a + star a) + c₀ * conj c₁ * F (a + star b)
      + c₁ * conj c₀ * F (b + star a) + c₁ * conj c₁ * F (b + star b)
```

lemma add_star_self_nonneg [source]

A positive-definite function takes a real, nonnegative value at every "norm point" `a + star a`.

```
lemma add_star_self_nonneg (hF : IsPositiveDefinite F) (a : M) : 0 ≤ F (a + star a)
```

lemma add_star_self_im [source]

The value of a positive-definite function at a "norm point" `a + star a` has zero imaginary part.

```
lemma add_star_self_im (hF : IsPositiveDefinite F) (a : M) : (F (a + star a)).im = 0
```

lemma add_star_self_re_nonneg [source]

The value of a positive-definite function at a "norm point" `a + star a` has nonnegative real part.

```
lemma add_star_self_re_nonneg (hF : IsPositiveDefinite F) (a : M) : 0 ≤ (F (a + star a)).re
```

lemma map_zero_nonneg [source]

The value of a positive-definite function at `0` is real and nonnegative.

```
lemma map_zero_nonneg (hF : IsPositiveDefinite F) : 0 ≤ F 0
```

lemma map_zero_im [source]

The value of a positive-definite function at `0` has zero imaginary part.

```
@[simp]
lemma map_zero_im (hF : IsPositiveDefinite F) : (F 0).im = 0
```

lemma map_zero_re_nonneg [source]

The real part of the value of a positive-definite function at `0` is nonnegative.

```
lemma map_zero_re_nonneg (hF : IsPositiveDefinite F) : 0 ≤ (F 0).re
```

lemma map_zero_eq_ofReal_re [source]

The value at the origin of a positive-definite function is equal to the real number `(F 0).re`, viewed as a complex number.

```
lemma map_zero_eq_ofReal_re (hF : IsPositiveDefinite F) : F 0 = ((F 0).re : ℂ)
```

lemma map_zero_re_pos_of_ne_zero [source]

If a positive-definite function is nonzero at the origin, then the real part of that value is strictly positive.

```
lemma map_zero_re_pos_of_ne_zero (hF : IsPositiveDefinite F) (h0 : F 0 ≠ 0) :
    0 < (F 0).re
```

lemma conj_symm [source]

A positive-definite function is conjugate symmetric in the involution: `conj (F (b + star a)) = F (a + star b)`.

```
@[simp]
lemma conj_symm (hF : IsPositiveDefinite F) (a b : M) :
    conj (F (b + star a)) = F (a + star b)
```

lemma posSemidef [source]

A positive-definite function `F` induces the positive-definite kernel `K(a, b) = F(a + b⋆)`. This is the forward half of the function ↔ kernel correspondence.

```
lemma posSemidef (hF : IsPositiveDefinite F) :
    Matrix.PosSemidef (fun a b => F (a + star b))
```

lemma of_posSemidef [source]

If the kernel `K(a, b) = F(a + b⋆)` is positive definite, then so is the function `F`. This is the reverse half of the function ↔ kernel correspondence.

```
lemma of_posSemidef
    (hK : Matrix.PosSemidef (fun a b => F (a + star b))) : IsPositiveDefinite F
```

lemma normSq_le [source]

The Cauchy–Schwarz inequality for a positive-definite function: the squared norm of an off-diagonal value is bounded by the product of the two diagonal values.

```
lemma normSq_le (hF : IsPositiveDefinite F) (a b : M) :
    Complex.normSq (F (a + star b))
      ≤ (F (a + star a)).re * (F (b + star b)).re
```

lemma norm_apply_le_map_zero_re_of_add_star_eq_zero [source]

If `a + star a = 0`, then a positive-definite function is bounded at `a` by its value at zero.

```
lemma norm_apply_le_map_zero_re_of_add_star_eq_zero (hF : IsPositiveDefinite F)
    (a : M) (ha : a + star a = 0) : ‖F a‖ ≤ (F 0).re
```

lemma apply_eq_zero_of_map_zero_re_eq_zero [source]

A positive-definite function with `(F 0).re = 0` vanishes identically.

No hypothesis on the point is required, and none on the ambient structure beyond an involutive additive monoid (`AddMonoid` and `StarAddMonoid`).

```
lemma apply_eq_zero_of_map_zero_re_eq_zero (hF : IsPositiveDefinite F)
    (h0 : (F 0).re = 0) (a : M) : F a = 0
```

lemma norm_apply_le_map_zero_re_of_star_eq_neg [source]

If the involution negates the point `a`, then a positive-definite function is bounded at `a` by its value at zero. In particular this applies on an additive group whose involution is negation.

```
lemma norm_apply_le_map_zero_re_of_star_eq_neg (hH : IsPositiveDefinite H)
    (a : N) (hstar_a : star a = -a) : ‖H a‖ ≤ (H 0).re
```

lemma add [source]

Positive-definite functions are closed under addition.

```
lemma add (hF : IsPositiveDefinite F) (hG : IsPositiveDefinite G) :
    IsPositiveDefinite (fun x => F x + G x)
```

lemma const_mul [source]

Positive-definite functions are closed under multiplication by a nonnegative complex scalar.

```
lemma const_mul {k : ℂ} (hk : 0 ≤ k) (hF : IsPositiveDefinite F) :
    IsPositiveDefinite (fun x => k * F x)
```

lemma mul [source]

Positive-definite functions are closed under pointwise multiplication (Schur product).

```
lemma mul (hF : IsPositiveDefinite F) (hG : IsPositiveDefinite G) :
    IsPositiveDefinite (fun x => F x * G x)
```

lemma isPositiveDefinite_const [source]

A nonnegative real constant is a positive-definite function.

```
lemma isPositiveDefinite_const {k : ℂ} (hk : 0 ≤ k) :
    IsPositiveDefinite (fun _ : M => k)
```

lemma isPositiveDefinite_zero [source]

The zero function is positive definite.

```
lemma isPositiveDefinite_zero : IsPositiveDefinite (fun _ : M => (0 : ℂ))
```

lemma sum [source]

Positive-definite functions are closed under finite sums.

```
lemma sum {ι : Type*} {s : Finset ι} {F : ι → M → ℂ} (hF : ∀ i ∈ s, IsPositiveDefinite (F i)) :
    IsPositiveDefinite (fun x => ∑ i ∈ s, F i x)
```

lemma prod [source]

Positive-definite functions are closed under finite products (Schur product).

```
lemma prod {ι : Type*} {s : Finset ι} {F : ι → M → ℂ} (hF : ∀ i ∈ s, IsPositiveDefinite (F i))
    :
    IsPositiveDefinite (fun x => ∏ i ∈ s, F i x)
```

2.371 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Continuity

PhyslibAlpha/Mathematics/TauCeti/Analysis/PositiveDefinite/Continuity.lean on GitHub.

lemma gram_three_add_star_expand_of_values [source]

```
private lemma gram_three_add_star_expand_of_values
    {x y : E} {lam : ℂ} {c : Fin 3 → ℂ} {p : Fin 3 → E}
    (hc0 : c 0 = 1) (hc1 : c 1 = -1) (hc2 : c 2 = lam)
    (hp0 : p 0 = x) (hp1 : p 1 = y) (hp2 : p 2 = 0) (hx : x + star x = 0) (hy : y + star y = 0)
    (hyx : F (y + star x) = conj (F (x + star y)))
    (hnx : F (star x) = conj (F x)) (hny : F (0 + star y) = conj (F y)) : (∑ i : Fin 3, ∑ j :
    Fin 3,
      c i * conj (c j) * F (p i + star (p j)))
      = F 0 - F (x + star y) + conj lam * F x - conj (F (x + star y)) + F 0
        - conj lam * F y + lam * conj (F x) - lam * conj (F y)
        + lam * conj lam * F 0
```

lemma gram_three_add_star_expand [source]

```
private lemma gram_three_add_star_expand {x y : E} (hx : x + star x = 0) (hy : y + star y = 0)
    (hyx : F (y + star x) = conj (F (x + star y)))
    (hnx : F (star x) = conj (F x)) (hny : F (0 + star y) = conj (F y)) {lam : ℂ} :
    (∑ i : Fin 3, ∑ j : Fin 3,
      ![1, -1, lam] i * conj (![1, -1, lam] j) *
        F (![x, y, 0] i + star (![x, y, 0] j)))
      = F 0 - F (x + star y) + conj lam * F x - conj (F (x + star y)) + F 0
        - conj lam * F y + lam * conj (F x) - lam * conj (F y)
        + lam * conj lam * F 0
```

lemma gram_three_sub_re_complex_algebra [source]

```
private lemma gram_three_sub_re_complex_algebra {r : ℝ} {z d lam : ℂ}
    (hlam : lam = -d / (r : ℂ)) (hr : 0 < r) :
    ((r : ℂ) - z + conj lam * d - conj z + (r : ℂ) + lam * conj d
        + lam * conj lam * (r : ℂ)).re
      = 2 * r - 2 * z.re - Complex.normSq d / r
```

lemma gram_three_add_star_re_algebra [source]

```
private lemma gram_three_add_star_re_algebra (hF : IsPositiveDefinite F)
    {x y : E} (hx : x + star x = 0) (hy : y + star y = 0)
    (hyx : F (y + star x) = conj (F (x + star y)))
    (hnx : F (star x) = conj (F x)) (hny : F (0 + star y) = conj (F y)) {C d lam : ℂ}
    (hC : C = F 0) (hd : d = F x - F y) (hlam : lam = -d / C) (hCpos : 0 < (F 0).re) :
    (∑ i : Fin 3, ∑ j : Fin 3,
      ![1, -1, lam] i * conj (![1, -1, lam] j) *
        F (![x, y, 0] i + star (![x, y, 0] j))).re
      = 2 * (F 0).re - 2 * (F (x + star y)).re - Complex.normSq d / (F 0).re
```

lemma gram_three_add_star_re_eq [source]

```
private lemma gram_three_add_star_re_eq (hF : IsPositiveDefinite F)
    {x y : E} (hx : x + star x = 0) (hy : y + star y = 0) {C d lam : ℂ}
    (hC : C = F 0) (hd : d = F x - F y) (hlam : lam = -d / C) (hCpos : 0 < (F 0).re) :
    (∑ i : Fin 3, ∑ j : Fin 3,
      ![1, -1, lam] i * conj (![1, -1, lam] j) *
        F (![x, y, 0] i + star (![x, y, 0] j))).re
      = 2 * (F 0).re - 2 * (F (x + star y)).re - Complex.normSq d / (F 0).re
```

lemma norm_sub_sq_le_two_mul_map_zero_re_mul_re_sub_of_add_star_eq_zero [source]

The local monoid-level real-part form of the standard positive-definite continuity estimate.

```
lemma norm_sub_sq_le_two_mul_map_zero_re_mul_re_sub_of_add_star_eq_zero
    (hF : IsPositiveDefinite F) {x y : E} (hx : x + star x = 0) (hy : y + star y = 0) :
    ‖F x - F y‖ ^ 2
      ≤ 2 * (F 0).re * ((F 0).re - (F (x + star y)).re)
```

lemma norm_sub_sq_le_two_mul_map_zero_re_mul_norm_sub_of_add_star_eq_zero [source]

The local monoid-level norm-valued form of the standard positive-definite continuity estimate.

```
lemma norm_sub_sq_le_two_mul_map_zero_re_mul_norm_sub_of_add_star_eq_zero
    (hF : IsPositiveDefinite F) {x y : E} (hx : x + star x = 0) (hy : y + star y = 0) :
    ‖F x - F y‖ ^ 2 ≤ 2 * (F 0).re * ‖F (x + star y) - F 0‖
```

lemma norm_sub_sq_le_two_mul_map_zero_re_mul_re_sub_of_star_eq_neg [source]

The local real-part form of the standard positive-definite continuity estimate.

```
lemma norm_sub_sq_le_two_mul_map_zero_re_mul_re_sub_of_star_eq_neg (hF : IsPositiveDefinite F)
    {x y : E} (hx : star x = -x) (hy : star y = -y) :
    ‖F x - F y‖ ^ 2
      ≤ 2 * (F 0).re * ((F 0).re - (F (x - y)).re)
```

lemma norm_sub_sq_le_two_mul_map_zero_re_mul_re_sub_of_forall_star_eq_neg [source]

The real-part form of the standard positive-definite continuity estimate under a globally negating involution.

```
lemma norm_sub_sq_le_two_mul_map_zero_re_mul_re_sub_of_forall_star_eq_neg
    (hF : IsPositiveDefinite F) (hstar : ∀ x : E, star x = -x) (x y : E) :
    ‖F x - F y‖ ^ 2
      ≤ 2 * (F 0).re * ((F 0).re - (F (x - y)).re)
```

lemma norm_sub_sq_le_two_mul_map_zero_re_mul_norm_sub_of_star_eq_neg [source]

The local norm-valued form of the standard positive-definite continuity estimate.

```
lemma norm_sub_sq_le_two_mul_map_zero_re_mul_norm_sub_of_star_eq_neg (hF : IsPositiveDefinite F
    )
    {x y : E} (hx : star x = -x) (hy : star y = -y) :
    ‖F x - F y‖ ^ 2 ≤ 2 * (F 0).re * ‖F (x - y) - F 0‖
```

lemma norm_sub_sq_le_two_mul_map_zero_re_mul_norm_sub_of_forall_star_eq_neg [source]

The norm-valued form of the standard positive-definite continuity estimate under a globally negating involution.

```
lemma norm_sub_sq_le_two_mul_map_zero_re_mul_norm_sub_of_forall_star_eq_neg
    (hF : IsPositiveDefinite F) (hstar : ∀ x : E, star x = -x) (x y : E) :
    ‖F x - F y‖ ^ 2 ≤ 2 * (F 0).re * ‖F (x - y) - F 0‖
```

lemma uniformContinuous_of_continuousAt_zero_of_forall_star_eq_neg [source]

A positive-definite function on a seminormed additive group with the negation involution is uniformly continuous as soon as it is continuous at the origin.

```
lemma uniformContinuous_of_continuousAt_zero_of_forall_star_eq_neg (hF : IsPositiveDefinite F)
    (hstar : ∀ x : E, star x = -x) (hcont : ContinuousAt F 0) :
    UniformContinuous F
```

2.372 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Function.Closure

PhyslibAlpha/Mathematics/TauCeti/Analysis/PositiveDefinite/Function/Closure.lean on GitHub.

lemma real_smul [source]

Positive-definite functions are closed under multiplication by a nonnegative real scalar.

```
lemma real_smul {r : ℝ} {F : M → ℂ} (hr : 0 ≤ r) (hF : IsPositiveDefinite F) :
    IsPositiveDefinite (fun x => r • F x)
```

lemma sum_smul [source]

Finite nonnegative complex-weighted sums of positive-definite functions are positive definite. This is the finite-mixture form used when the weights are already complex scalars with nonnegative real value.

```
lemma sum_smul {ι : Type*} {s : Finset ι} {w : ι → ℂ} {F : ι → M → ℂ}
    (hw : ∀ i ∈ s, 0 ≤ w i) (hF : ∀ i ∈ s, IsPositiveDefinite (F i)) :
    IsPositiveDefinite (fun x => ∑ i ∈ s, w i • F i x)
```

lemma sum_const_mul [source]

Finite nonnegative complex-weighted sums, written using multiplication rather than scalar notation.

```
lemma sum_const_mul {ι : Type*} {s : Finset ι} {w : ι → ℂ} {F : ι → M → ℂ}
    (hw : ∀ i ∈ s, 0 ≤ w i) (hF : ∀ i ∈ s, IsPositiveDefinite (F i)) :
    IsPositiveDefinite (fun x => ∑ i ∈ s, w i * F i x)
```

lemma sum_real_smul [source]

Finite nonnegative real-weighted sums of positive-definite functions are positive definite.

```
lemma sum_real_smul {ι : Type*} {s : Finset ι} {w : ι → ℝ} {F : ι → M → ℂ}
    (hw : ∀ i ∈ s, 0 ≤ w i) (hF : ∀ i ∈ s, IsPositiveDefinite (F i)) :
    IsPositiveDefinite (fun x => ∑ i ∈ s, w i • F i x)
```

lemma sum_real_const_mul [source]

Finite nonnegative real-weighted sums, written with the real weight coerced to $\mathbb{C}$ and multiplied in the codomain.

```
lemma sum_real_const_mul {ι : Type*} {s : Finset ι} {w : ι → ℝ} {F : ι → M → ℂ}
    (hw : ∀ i ∈ s, 0 ≤ w i) (hF : ∀ i ∈ s, IsPositiveDefinite (F i)) :
    IsPositiveDefinite (fun x => ∑ i ∈ s, (w i : ℂ) * F i x)
```

lemma pow [source]

Schur powers of a positive-definite function are positive definite.

```
lemma pow {F : M → ℂ} (hF : IsPositiveDefinite F) (n : ℕ) :
    IsPositiveDefinite (fun x => F x ^ n)
```

lemma sum_smul_apply_of_apply_eq_one [source]

If each summand in a finite complex-weighted sum is normalized at a point, then the sum's value at that point is the sum of the weights.

```
lemma sum_smul_apply_of_apply_eq_one {ι : Type*} {s : Finset ι} {w : ι → ℂ}
    {F : ι → N → ℂ} {x : N} (hFx : ∀ i ∈ s, F i x = 1) : (∑ i ∈ s, w i • F i x) = ∑ i ∈ s, w i
```

lemma sum_smul_apply_eq_one [source]

A finite complex-weighted sum of functions normalized at a point is normalized at that point when the weights sum to 1.

```
lemma sum_smul_apply_eq_one {ι : Type*} {s : Finset ι} {w : ι → ℂ}
    {F : ι → N → ℂ} {x : N} (hFx : ∀ i ∈ s, F i x = 1) (hw_sum : ∑ i ∈ s, w i = 1) :
    (fun y => ∑ i ∈ s, w i • F i y) x = 1
```

lemma sum_const_mul_apply_of_apply_eq_one [source]

If each summand in a finite complex-weighted sum is normalized at a point, then the sum's multiplication-form value at that point is the sum of the weights.

```
lemma sum_const_mul_apply_of_apply_eq_one {ι : Type*} {s : Finset ι} {w : ι → ℂ}
    {F : ι → N → ℂ} {x : N} (hFx : ∀ i ∈ s, F i x = 1) : (∑ i ∈ s, w i * F i x) = ∑ i ∈ s, w i
```

lemma sum_const_mul_apply_eq_one [source]

A finite complex-weighted sum, in multiplication form, of functions normalized at a point is normalized at that point when the weights sum to 1.

```
lemma sum_const_mul_apply_eq_one {ι : Type*} {s : Finset ι} {w : ι → ℂ}
    {F : ι → N → ℂ} {x : N} (hFx : ∀ i ∈ s, F i x = 1) (hw_sum : ∑ i ∈ s, w i = 1) :
    (fun y => ∑ i ∈ s, w i * F i y) x = 1
```

lemma sum_real_smul_apply_of_apply_eq_one [source]

If each summand in a finite real-weighted sum is normalized at a point, then the sum's value at that point is the complex coercion of the sum of the real weights.

```
lemma sum_real_smul_apply_of_apply_eq_one {ι : Type*} {s : Finset ι} {w : ι → ℝ}
    {F : ι → N → ℂ} {x : N} (hFx : ∀ i ∈ s, F i x = 1) :
    (∑ i ∈ s, w i • F i x) = ((∑ i ∈ s, w i : ℝ) : ℂ)
```

lemma sum_real_smul_apply_eq_one [source]

A finite real-weighted sum of functions normalized at a point is normalized at that point when the weights sum to 1.

```
lemma sum_real_smul_apply_eq_one {ι : Type*} {s : Finset ι} {w : ι → ℝ}
    {F : ι → N → ℂ} {x : N} (hFx : ∀ i ∈ s, F i x = 1) (hw_sum : ∑ i ∈ s, w i = 1) :
    (fun y => ∑ i ∈ s, w i • F i y) x = 1
```

lemma sum_real_const_mul_apply_of_apply_eq_one [source]

If each summand in a finite real-weighted sum is normalized at a point, then the sum's multiplication-form value at that point is the complex coercion of the sum of the real weights.

```
lemma sum_real_const_mul_apply_of_apply_eq_one {ι : Type*} {s : Finset ι}
    {w : ι → ℝ} {F : ι → N → ℂ} {x : N} (hFx : ∀ i ∈ s, F i x = 1) :
    (∑ i ∈ s, (w i : ℂ) * F i x) = ((∑ i ∈ s, w i : ℝ) : ℂ)
```

lemma sum_real_const_mul_apply_eq_one [source]

A finite real-weighted sum, in multiplication form, of functions normalized at a point is normalized at that point when the weights sum to 1.

```
lemma sum_real_const_mul_apply_eq_one {ι : Type*} {s : Finset ι} {w : ι → ℝ}
    {F : ι → N → ℂ} {x : N} (hFx : ∀ i ∈ s, F i x = 1) (hw_sum : ∑ i ∈ s, w i = 1) :
    (fun y => ∑ i ∈ s, (w i : ℂ) * F i y) x = 1
```

lemma sum_smul_apply_zero_of_apply_zero_eq_one [source]

If each summand in a finite complex-weighted sum is normalized at the origin, then the sum's value at the origin is the sum of the weights.

```
lemma sum_smul_apply_zero_of_apply_zero_eq_one {ι : Type*} {s : Finset ι}
    {w : ι → ℂ} {F : ι → N → ℂ} (hF0 : ∀ i ∈ s, F i 0 = 1) :
    (∑ i ∈ s, w i • F i 0) = ∑ i ∈ s, w i
```

lemma sum_smul_apply_zero_eq_one [source]

A finite complex-weighted sum of functions normalized at the origin is normalized when the weights sum to 1.

```
lemma sum_smul_apply_zero_eq_one {ι : Type*} {s : Finset ι} {w : ι → ℂ}
    {F : ι → N → ℂ} (hF0 : ∀ i ∈ s, F i 0 = 1) (hw_sum : ∑ i ∈ s, w i = 1) :
    (fun x => ∑ i ∈ s, w i • F i x) 0 = 1
```

lemma sum_const_mul_apply_zero_of_apply_zero_eq_one [source]

If each summand in a finite complex-weighted sum is normalized at the origin, then the sum's multiplication-form value at the origin is the sum of the weights.

```
lemma sum_const_mul_apply_zero_of_apply_zero_eq_one {ι : Type*} {s : Finset ι}
    {w : ι → ℂ} {F : ι → N → ℂ} (hF0 : ∀ i ∈ s, F i 0 = 1) :
    (∑ i ∈ s, w i * F i 0) = ∑ i ∈ s, w i
```

lemma sum_const_mul_apply_zero_eq_one [source]

A finite complex-weighted sum, in multiplication form, of functions normalized at the origin is normalized when the weights sum to 1.

```
lemma sum_const_mul_apply_zero_eq_one {ι : Type*} {s : Finset ι} {w : ι → ℂ}
    {F : ι → N → ℂ} (hF0 : ∀ i ∈ s, F i 0 = 1) (hw_sum : ∑ i ∈ s, w i = 1) :
    (fun x => ∑ i ∈ s, w i * F i x) 0 = 1
```

lemma sum_real_smul_apply_zero_of_apply_zero_eq_one [source]

If each summand in a finite real-weighted sum is normalized at the origin, then the sum's value at the origin is the complex coercion of the sum of the real weights.

```
lemma sum_real_smul_apply_zero_of_apply_zero_eq_one {ι : Type*} {s : Finset ι}
    {w : ι → ℝ} {F : ι → N → ℂ} (hF0 : ∀ i ∈ s, F i 0 = 1) :
    (∑ i ∈ s, w i • F i 0) = ((∑ i ∈ s, w i : ℝ) : ℂ)
```

lemma sum_real_smul_apply_zero_eq_one [source]

A finite real-weighted sum of functions normalized at the origin is normalized when the weights sum to `1`.

```
lemma sum_real_smul_apply_zero_eq_one {ι : Type*} {s : Finset ι} {w : ι → ℝ}
    {F : ι → N → ℂ} (hF0 : ∀ i ∈ s, F i 0 = 1) (hw_sum : ∑ i ∈ s, w i = 1) :
    (fun x => ∑ i ∈ s, w i • F i x) 0 = 1
```

lemma sum_real_const_mul_apply_zero_of_apply_zero_eq_one [source]

If each summand in a finite real-weighted sum is normalized at the origin, then the sum's multiplication-form value at the origin is the complex coercion of the sum of the real weights.

```
lemma sum_real_const_mul_apply_zero_of_apply_zero_eq_one {ι : Type*} {s : Finset ι}
    {w : ι → ℝ} {F : ι → N → ℂ} (hF0 : ∀ i ∈ s, F i 0 = 1) :
    (∑ i ∈ s, (w i : ℂ) * F i 0) = ((∑ i ∈ s, w i : ℝ) : ℂ)
```

lemma sum_real_const_mul_apply_zero_eq_one [source]

A finite real-weighted sum, in multiplication form, of functions normalized at the origin is normalized when the weights sum to `1`.

```
lemma sum_real_const_mul_apply_zero_eq_one {ι : Type*} {s : Finset ι} {w : ι → ℝ}
    {F : ι → N → ℂ} (hF0 : ∀ i ∈ s, F i 0 = 1) (hw_sum : ∑ i ∈ s, w i = 1) :
    (fun x => ∑ i ∈ s, (w i : ℂ) * F i x) 0 = 1
```

2.373 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Function.GNS

PhyslibAlpha/Mathematics/TauCeti/Analysis/PositiveDefinite/Function/GNS.lean on GitHub.

abbrev gnsSpace [source]

The canonical Hilbert space of the translation-invariant kernel `K(a,b) = F(a-b)`.

```
abbrev gnsSpace
```

def gnsVector [source]

The vector in the GNS space corresponding to a group element.

```
def gnsVector (a : G) : hF.gnsSpace
```

lemma inner_gnsVector [source]

The inner product of two GNS vectors is the original positive-definite kernel.

```
@[simp]
lemma inner_gnsVector (a b : G) : ⟪hF.gnsVector a, hF.gnsVector b⟫_ℂ = F (a - b)
```

lemma norm_gnsVector_sub_sq [source]

Squared distances between GNS vectors are controlled by the real part of the function.

```
@[simp]
lemma norm_gnsVector_sub_sq (a b : G) :
    ‖hF.gnsVector a - hF.gnsVector b‖ ^ 2 =
      2 * ((F 0).re - (F (a - b)).re)
```

lemma gnsVector_dense [source]

The GNS vectors span a dense subspace.

```
lemma gnsVector_dense :
    (Submodule.span ℂ (Set.range hF.gnsVector)).topologicalClosure = ⊤
```

lemma translated_gnsVector_dense [source]

```
private lemma translated_gnsVector_dense (g : G) :
    (Submodule.span ℂ (Set.range fun a : G => hF.gnsVector (g + a))).topologicalClosure = ⊤
```

def gnsTranslation [source]

Translation by `g` is a unitary operator on the canonical GNS Hilbert space. Its action on the dense family of kernel vectors is `v(a) ↦ v(g+a)`.

```
def gnsTranslation (g : G) : hF.gnsSpace ≃ₗᵢ[ℂ] hF.gnsSpace
```

lemma gnsTranslation_gnsVector [source]

Translation acts on the canonical GNS vectors by addition.

```
@[simp]
lemma gnsTranslation_gnsVector (g a : G) :
    hF.gnsTranslation g (hF.gnsVector a) = hF.gnsVector (g + a)
```

lemma gnsTranslation_zero [source]

The translation at zero is the identity operator.

```
@[simp]
lemma gnsTranslation_zero : hF.gnsTranslation 0 = LinearIsometryEquiv.refl ℂ hF.gnsSpace
```

lemma gnsTranslation_add [source]

GNS translations compose according to the group law.

```
@[simp]
lemma gnsTranslation_add (g k : G) :
    hF.gnsTranslation (g + k) = (hF.gnsTranslation k).trans (hF.gnsTranslation g)
```

def gnsRepresentation [source]

The GNS representation as a homomorphism from the multiplicative copy of `G` to the unitary operators on its canonical Hilbert space.

```
def gnsRepresentation : Multiplicative G →* (hF.gnsSpace ≃ₗᵢ[ℂ] hF.gnsSpace)
```

lemma gnsRepresentation_ofAdd [source]

The representation operator at `g` is translation by `g`.

```
@[simp]
lemma gnsRepresentation_ofAdd (g : G) :
    hF.gnsRepresentation (Multiplicative.ofAdd g) = hF.gnsTranslation g
```

lemma gnsRepresentation_gnsVector [source]

The representation translates each GNS vector.

```
lemma gnsRepresentation_gnsVector (g a : G) :
    hF.gnsRepresentation (Multiplicative.ofAdd g) (hF.gnsVector a) =
      hF.gnsVector (g + a)
```

lemma eq_inner_gnsRepresentation [source]

A positive-definite function is a matrix coefficient of its canonical unitary representation.

```
lemma eq_inner_gnsRepresentation (g : G) :
    F g = ⟪hF.gnsRepresentation (Multiplicative.ofAdd g) (hF.gnsVector 0),
      hF.gnsVector 0⟫_ℂ
```

lemma continuous_gnsVector [source]

Continuity of a positive-definite function at zero makes its canonical feature map continuous. This is the regularity needed for a strongly continuous representation.

```
lemma continuous_gnsVector [TopologicalSpace G] [IsTopologicalAddGroup G]
    (hcont : ContinuousAt F 0) : Continuous hF.gnsVector
```

lemma continuous_gnsTranslation_gnsVector [source]

```
private lemma continuous_gnsTranslation_gnsVector [TopologicalSpace G] [IsTopologicalAddGroup G
    ]
    (hcont : ContinuousAt F 0) (a : G) :
    Continuous (fun g : G => hF.gnsTranslation g (hF.gnsVector a))
```

lemma continuous_gnsTranslation_apply [source]

The GNS translation representation is strongly continuous: every vector has a continuous orbit.

```
lemma continuous_gnsTranslation_apply [TopologicalSpace G] [IsTopologicalAddGroup G]
    (hcont : ContinuousAt F 0) (x : hF.gnsSpace) :
    Continuous (fun g : G => hF.gnsTranslation g x)
```

lemma continuous_gnsRepresentation_apply [source]

The canonical unitary representation has continuous orbits.

```
lemma continuous_gnsRepresentation_apply [TopologicalSpace G] [IsTopologicalAddGroup G]
    (hcont : ContinuousAt F 0) (x : hF.gnsSpace) :
    Continuous (fun g : Multiplicative G => hF.gnsRepresentation g x)
```

2.374 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Function.Kernel

PhyslibAlpha/Mathematics/TauCeti/Analysis/PositiveDefinite/Function/Kernel.lean on GitHub.

lemma isPositiveDefinite_iff_posSemidef [source]

A function `F` on an involutive additive monoid is positive definite if and only if the two-variable kernel `K(a, b) = F(a + b⋆)` is positive definite.

```
lemma isPositiveDefinite_iff_posSemidef :
    IsPositiveDefinite F ↔ Matrix.PosSemidef (fun a b => F (a + star b))
```

lemma posSemidef_sub [source]

Under the negation involution `a⋆ = -a`, a positive-definite function `F` gives the translation-invariant positive-definite kernel `K(a, b) = F(a - b)`. This is the form in which positive definiteness is usually stated on groups such as $\mathbb{R}^d$ or a real inner-product space.

```
lemma posSemidef_sub (hstar : ∀ a : G, star a = -a) (hF : IsPositiveDefinite F) :
    Matrix.PosSemidef (fun a b => F (a - b))
```

lemma isPositiveDefinite_iff_posSemidef_sub [source]

Under the negation involution `a⋆ = -a`, `F` is positive definite if and only if the translation-invariant kernel `K(a, b) = F(a - b)` is positive definite.

```
lemma isPositiveDefinite_iff_posSemidef_sub (hstar : ∀ a : G, star a = -a) :
    IsPositiveDefinite F ↔ Matrix.PosSemidef (fun a b => F (a - b))
```

2.375 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Kernel.Kolmogorov

PhyslibAlpha/Mathematics/TauCeti/Analysis/PositiveDefinite/Kernel/Kolmogorov.lean on GitHub.

def positiveDefiniteKernelOperator [source]

A scalar kernel regarded as an operator-valued kernel on the one-dimensional Hilbert space $\mathbb{k}$: the entry at `(a, b)` is left multiplication by `K a b`.

```
noncomputable def positiveDefiniteKernelOperator (K : α → α → 𝕜) :
    Matrix α α (𝕜 →L[𝕜] 𝕜)
```

lemma positiveDefiniteKernelOperator_apply [source]

```
@[simp]
lemma positiveDefiniteKernelOperator_apply (K : α → α → 𝕜) (a b : α) (z : 𝕜) :
    positiveDefiniteKernelOperator K a b z = K a b * z
```

lemma posSemidef_positiveDefiniteKernelOperator [source]

The operator-valued kernel obtained from a scalar positive-definite kernel is positive semidefinite. This is the bridge needed by Mathlib's `RKHS.OfKernel` construction.

```
lemma posSemidef_positiveDefiniteKernelOperator (hK : Matrix.PosSemidef K) :
    (positiveDefiniteKernelOperator K).PosSemidef
```

abbrev KolmogorovSpace [source]

The canonical Hilbert space in the Kolmogorov decomposition of `K`.

This is an abbreviation because Mathlib's `RKHS.OfKernel` currently has to be an abbreviation in order for its normed-group and inner-product instances to reduce.

```
noncomputable abbrev KolmogorovSpace (hK : Matrix.PosSemidef K)
```

def kolmogorovFeature [source]

The canonical feature map into the Kolmogorov space. It sends `a` to the reproducing-kernel vector at `a`, evaluated on the scalar `1`.

```
noncomputable def kolmogorovFeature (hK : Matrix.PosSemidef K) (a : α) :
    KolmogorovSpace hK
```

lemma inner_kolmogorovFeature [source]

***Kolmogorov identity.** Inner products of the canonical feature vectors recover the original kernel.*

```
@[simp]
lemma inner_kolmogorovFeature (hK : Matrix.PosSemidef K) (a b : α) :
    ⟪kolmogorovFeature hK a, kolmogorovFeature hK b⟫_𝕜 = K a b
```

lemma norm_kolmogorovFeature_sq [source]

The squared norm of a canonical feature vector is the real part of the corresponding diagonal kernel value.

```
@[simp]
lemma norm_kolmogorovFeature_sq (hK : Matrix.PosSemidef K) (a : α) :
    ‖kolmogorovFeature hK a‖ ^ 2 = RCLike.re (K a a)
```

lemma norm_kolmogorovFeature_sub_sq [source]

The squared distance between two canonical feature vectors, expressed entirely in terms of the original kernel.

```
@[simp]
lemma norm_kolmogorovFeature_sub_sq (hK : Matrix.PosSemidef K) (a b : α) :
    ‖kolmogorovFeature hK a - kolmogorovFeature hK b‖ ^ 2 =
      RCLike.re (K a a) - 2 * RCLike.re (K a b) + RCLike.re (K b b)
```

lemma kolmogorovFeature_dense [source]

The canonical feature vectors have dense linear span in the Kolmogorov space. Thus the construction is minimal: it contains no orthogonal summand invisible to the kernel.

```
lemma kolmogorovFeature_dense (hK : Matrix.PosSemidef K) :
    (Submodule.span 𝕜 (Set.range (kolmogorovFeature hK))).topologicalClosure = ⊤
```

lemma denseRange_featureCombination [source]

```
private lemma denseRange_featureCombination (hK : Matrix.PosSemidef K) :
    DenseRange (Finsupp.linearCombination 𝕜 (kolmogorovFeature hK))
```

def kolmogorovIsometry [source]

The unique linear isometry from the canonical Kolmogorov space into any Hilbert-space realization of `K`. It sends each canonical feature vector to the corresponding vector in the given realization.

```
noncomputable def kolmogorovIsometry {E : Type w} [NormedAddCommGroup E]
    [InnerProductSpace 𝕜 E] [CompleteSpace E] (hK : Matrix.PosSemidef K) (φ : α → E)
    (hφ : ∀ a b, ⟪φ a, φ b⟫_𝕜 = K a b) : KolmogorovSpace hK →ₗᵢ[𝕜] E
```

lemma kolmogorovIsometry_apply [source]

The universal comparison isometry sends canonical feature vectors to the vectors of the given realization.

```
@[simp]
lemma kolmogorovIsometry_apply {E : Type w} [NormedAddCommGroup E]
    [InnerProductSpace 𝕜 E] [CompleteSpace E] (hK : Matrix.PosSemidef K) (φ : α → E)
    (hφ : ∀ a b, ⟪φ a, φ b⟫_𝕜 = K a b) (a : α) :
    kolmogorovIsometry hK φ hφ (kolmogorovFeature hK a) = φ a
```

lemma kolmogorovIsometry_unique [source]

A linear isometry out of the canonical Kolmogorov space is uniquely determined by its values on the canonical feature vectors.

```
lemma kolmogorovIsometry_unique {E : Type w} [NormedAddCommGroup E]
    [InnerProductSpace 𝕜 E] [CompleteSpace E] (hK : Matrix.PosSemidef K) (φ : α → E)
    (hφ : ∀ a b, ⟪φ a, φ b⟫_𝕜 = K a b) (T : KolmogorovSpace hK →ₗᵢ[𝕜] E)
    (hT : ∀ a, T (kolmogorovFeature hK a) = φ a) : T = kolmogorovIsometry hK φ hφ
```

lemma kolmogorovIsometry_surjective [source]

If the vectors in a realization of `K` have dense linear span, the universal comparison isometry is surjective.

```
lemma kolmogorovIsometry_surjective {E : Type w} [NormedAddCommGroup E]
    [InnerProductSpace 𝕜 E] [CompleteSpace E] (hK : Matrix.PosSemidef K) (φ : α → E)
    (hφ : ∀ a b, ⟪φ a, φ b⟫_𝕜 = K a b)
    (hφdense : (Submodule.span 𝕜 (Set.range φ)).topologicalClosure = ⊤) :
    Function.Surjective (kolmogorovIsometry hK φ hφ)
```

def kolmogorovEquiv [source]

The canonical Kolmogorov space is linearly isometric to every other minimal realization of the same kernel.

```
noncomputable def kolmogorovEquiv {E : Type w} [NormedAddCommGroup E]
    [InnerProductSpace 𝕜 E] [CompleteSpace E] (hK : Matrix.PosSemidef K) (φ : α → E)
    (hφ : ∀ a b, ⟪φ a, φ b⟫_𝕜 = K a b)
    (hφdense : (Submodule.span 𝕜 (Set.range φ)).topologicalClosure = ⊤) :
    KolmogorovSpace hK ≃ₗᵢ[𝕜] E
```

lemma kolmogorovEquiv_apply [source]

The equivalence with another minimal realization sends canonical feature vectors to the vectors of that realization.

```
@[simp]
lemma kolmogorovEquiv_apply {E : Type w} [NormedAddCommGroup E]
    [InnerProductSpace 𝕜 E] [CompleteSpace E] (hK : Matrix.PosSemidef K) (φ : α → E)
    (hφ : ∀ a b, ⟪φ a, φ b⟫_𝕜 = K a b)
    (hφdense : (Submodule.span 𝕜 (Set.range φ)).topologicalClosure = ⊤) (a : α) :
    kolmogorovEquiv hK φ hφ hφdense (kolmogorovFeature hK a) = φ a
```

2.376 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Limits

PhyslibAlpha/Mathematics/TauCeti/Analysis/PositiveDefinite/Limits.lean on GitHub.

lemma of_tendsto [source]

Positive-definiteness is preserved under pointwise limits along a nontrivial filter. The hypothesis on positive-definiteness is eventual, so this applies equally to nets that are eventually positive definite.

```
lemma of_tendsto {ι : Type*} {l : Filter ι} [NeBot l] {F : ι → M → ℂ} {G : M → ℂ}
    (hF : ∀ᶠ i in l, IsPositiveDefinite (F i))
    (hlim : ∀ x : M, Tendsto (fun i => F i x) l (𝓝 (G x))) :
    IsPositiveDefinite G
```

lemma of_forall_tendsto [source]

A pointwise limit of a family of positive-definite functions is positive definite. This is the non-eventual form of `TauCeti.IsPositiveDefinite.of_tendsto`.

```
lemma of_forall_tendsto {ι : Type*} {l : Filter ι} [NeBot l] {F : ι → M → ℂ} {G : M → ℂ}
```

```
    (hF : ∀ i, IsPositiveDefinite (F i)) (hlim : ∀ x : M, Tendsto (fun i => F i x) l (𝓝 (G x))
    ) :
    IsPositiveDefinite G
```

lemma of_seq_tendsto [source]

Sequential pointwise limits of eventually positive-definite functions are positive definite.

```
lemma of_seq_tendsto {F : ℕ → M → ℂ} {G : M → ℂ} (hF : ∀ᶠ n in atTop, IsPositiveDefinite (F n))
    (hlim : ∀ x : M, Tendsto (fun n => F n x) atTop (𝓝 (G x))) :
    IsPositiveDefinite G
```

lemma of_tendstoLocallyUniformly [source]

A locally uniform limit of eventually positive-definite functions is positive definite.

Unlike continuity, positive-definiteness itself only needs pointwise convergence; local uniform convergence is converted to pointwise convergence before applying `TauCeti.IsPositiveDefinite.of_`

```
lemma of_tendstoLocallyUniformly {ι : Type*} {l : Filter ι} [NeBot l] {F : ι → M → ℂ} {G : M →
    ℂ}
    (hF : ∀ᶠ i in l, IsPositiveDefinite (F i)) (hlim : TendstoLocallyUniformly F G l) :
    IsPositiveDefinite G
```

2.377 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.Normalize

PhyslibAlpha/Mathematics/TauCeti/Analysis/PositiveDefinite/Normalize.lean on GitHub.

lemma normalizeScalar_nonneg [source]

The normalizing scalar `((F 0).re)⁻¹`, viewed as a complex number, is nonnegative.

```
private lemma normalizeScalar_nonneg (hF : IsPositiveDefinite F) :
    0 ≤ (((F 0).re)⁻¹ : ℂ)
```

lemma normalize [source]

Multiplying a positive-definite function by the reciprocal of its real value at the origin preserves positive-definiteness. If `F 0 = 0` this gives the zero scaling; the separate `normalize_apply_zero` lemma below records the useful nonzero case.

```
lemma normalize (hF : IsPositiveDefinite F) :
    IsPositiveDefinite (fun x => (((F 0).re)⁻¹ : ℂ) * F x)
```

lemma normalize_apply_zero [source]

The normalized positive-definite function has value `1` at the origin.

```
@[simp]
lemma normalize_apply_zero (hF : IsPositiveDefinite F) (h0 : F 0 ≠ 0) :
    (((F 0).re)⁻¹ : ℂ) * F 0 = 1
```

lemma norm_normalize_apply_le_one_of_add_star_eq_zero [source]

At points satisfying `a + star a = 0`, the normalized positive-definite function is bounded by `1`. When `F 0 = 0` the normalizing scalar is `0`, so the bound holds trivially.

```
lemma norm_normalize_apply_le_one_of_add_star_eq_zero (hF : IsPositiveDefinite F)
    (a : M) (ha : a + star a = 0) :
    ‖(((F 0).re)⁻¹ : ℂ) * F a‖ ≤ 1
```

lemma norm_normalize_apply_le_one_of_star_eq_neg [source]

Under the negation involution, the normalized positive-definite function is bounded by `1` at every point.

```
lemma norm_normalize_apply_le_one_of_star_eq_neg (hH : IsPositiveDefinite H)
    (a : G) (hstar_a : star a = -a) :
    ‖(((H 0).re)⁻¹ : ℂ) * H a‖ ≤ 1
```

lemma norm_normalize_apply_le_one_of_forall_star_eq_neg [source]

If the involution is negation everywhere, the normalized positive-definite function is uniformly bounded by `1`.

```
lemma norm_normalize_apply_le_one_of_forall_star_eq_neg (hH : IsPositiveDefinite H)
    (hstar : ∀ a : G, star a = -a) (a : G) :
    ‖(((H 0).re)⁻¹ : ℂ) * H a‖ ≤ 1
```

2.378 PhyslibAlpha.Mathematics.TauCeti.Analysis.PositiveDefinite.PontryaginMeasure

PhyslibAlpha/Mathematics/TauCeti/Analysis/PositiveDefinite/PontryaginMeasure.lean on GitHub.

lemma isPositiveDefiniteSub_pontryaginMeasureTransform [source]

The transform of a finite positive measure on the dual is positive definite.

```
lemma _root_.MeasureTheory.FiniteMeasure.isPositiveDefiniteSub_pontryaginMeasureTransform
    (μ : FiniteMeasure (PontryaginDual (Multiplicative G))) :
    IsPositiveDefiniteSub μ.pontryaginMeasureTransform
```

lemma norm_pontryaginMeasureTransform_le [source]

The absolute value of a measure transform is bounded by the measure's total mass.

```
lemma _root_.MeasureTheory.FiniteMeasure.norm_pontryaginMeasureTransform_le
    (μ : FiniteMeasure (PontryaginDual (Multiplicative G))) (g : G) :
    ‖μ.pontryaginMeasureTransform g‖ ≤ μ.toMeasure.real Set.univ
```

2.379 PhyslibAlpha.Mathematics.TauCeti.LinearAlgebra.Dimension.Submodule

PhyslibAlpha/Mathematics/TauCeti/LinearAlgebra/Dimension/Submodule.lean on GitHub.

lemma inf_ne_bot_of_finrank_lt [source]

Two subspaces whose dimensions sum to more than the dimension of the ambient space have nontrivial intersection.

```
lemma inf_ne_bot_of_finrank_lt [FiniteDimensional 𝕜 E] {V W : Submodule 𝕜 E}
    (h : finrank 𝕜 E < finrank 𝕜 V + finrank 𝕜 W) : V ⊓ W ≠ ⊥
```

2.380 PhyslibAlpha.Mathematics.TauCeti.LinearAlgebra.Matrix.PosSemidef

PhyslibAlpha/Mathematics/TauCeti/LinearAlgebra/Matrix/PosSemidef.lean on GitHub.

lemma posSemidef_of_support_posSemidef [source]

```
private lemma posSemidef_of_support_posSemidef {R : Type u}
    [Ring R] [PartialOrder R] [StarRing R] (K : α → α → R)
    (hHerm : (Matrix.of fun a b => K a b).IsHermitian) (hgram : ∀ x : α →₀ R,
      (Matrix.of fun i j : x.support => K (i : α) (j : α)).PosSemidef) :
    (Matrix.of fun a b => K a b).PosSemidef
```

lemma posSemidef_rankOne [source]

The rank-one matrix `(a, b) ↦ star (g a) · g b` is positive semidefinite for an arbitrary index type. Such matrices are elementary building blocks for positive-semidefinite matrices; taking `g ≡ 1` gives the constant matrix `1`.

```
lemma posSemidef_rankOne {R : Type u}
    [Ring R] [PartialOrder R] [StarRing R] [StarOrderedRing R] (g : α → R) :
    Matrix.PosSemidef (fun a b => star (g a) * g b)
```

lemma posSemidef_const_one [source]

The constant matrix with value `1` is positive semidefinite.

```
lemma posSemidef_const_one {R : Type u}
    [Ring R] [PartialOrder R] [StarRing R] [StarOrderedRing R] :
    Matrix.PosSemidef (fun _ _ : α => (1 : R))
```

lemma posSemidef_const_of_nonneg [source]

A nonnegative constant gives a positive-semidefinite constant matrix.

```
lemma posSemidef_const_of_nonneg {R : Type u}
    [Ring R] [PartialOrder R] [StarRing R] [StarOrderedRing R]
    {c : R} (hc : 0 ≤ c) : Matrix.PosSemidef (fun _ _ : α => c)
```

lemma posSemidef_iff_finite_sum [source]

The quadratic-form characterization of an arbitrary-index positive-semidefinite matrix: conjugate symmetry and nonnegativity on every finite family, allowing repeated indices.

```
lemma posSemidef_iff_finite_sum {R : Type u} [Ring R] [PartialOrder R] [StarRing R]
    {K : α → α → R} :
    Matrix.PosSemidef K ↔
      (∀ a b, star (K a b) = K b a) ∧
        ∀ {ι : Type*} [Fintype ι] (v : ι → α) (x : ι → R),
          0 ≤ ∑ i, ∑ j, star (x i) * K (v i) (v j) * x j
```

lemma posSemidef_schur_finset_prod [source]

Finite pointwise Schur products of positive-semidefinite matrices are positive semidefinite.

```
lemma posSemidef_schur_finset_prod {ι : Type w} {s : Finset ι}
    {K : ι → α → α → 𝕜} (hK : ∀ i ∈ s, Matrix.PosSemidef (K i)) :
    Matrix.PosSemidef (fun a b => ∏ i ∈ s, K i a b)
```

lemma posSemidef_prod_union [source]

For weights `w` in `[0, 1]` and finite sets `L i`, the matrix `(i, j) ↦ ∏_{a ∈ L i ∪ L j} w a` is positive semidefinite.

```
lemma posSemidef_prod_union {ι : Type w} [DecidableEq α] (L : ι → Finset α)
    {w : α → ℝ} (hw₀ : ∀ a, 0 ≤ w a) (hw₁ : ∀ a, w a ≤ 1) :
    Matrix.PosSemidef (fun i j => ∏ a ∈ L i ∪ L j, w a)
```

lemma hadamard_pow [source]

Schur powers of a positive-semidefinite matrix are positive semidefinite.

```
lemma Matrix.PosSemidef.hadamard_pow {α 𝕜 : Type*} [RCLike 𝕜] {K : α → α → 𝕜}
    (hK : Matrix.PosSemidef K) (n : ℕ) :
    Matrix.PosSemidef (fun a b => K a b ^ n)
```

2.381 PhyslibAlpha.Mathematics.TauCeti.MeasureTheory.Function.StronglyMeasurable.InnerRegul

PhyslibAlpha/Mathematics/TauCeti/MeasureTheory/Function/StronglyMeasurable/InnerRegular.lean on GitHub.

lemma aestronglyMeasurable_of_measure_ne_top [source]

For a measure that is inner regular with respect to compact sets on sets of finite measure, a function continuous on a measurable set `s` of finite measure is a.e. strongly measurable for the restriction of the measure to `s`.

```
lemma ContinuousOn.aestronglyMeasurable_of_measure_ne_top {f : α → β} {s : Set α}
    (hf : ContinuousOn f s) (hs : MeasurableSet s) (hμs : μ s ≠ ∞) :
    AEStronglyMeasurable f (μ.restrict s)
```

lemma aestronglyMeasurable_of_sigmaFinite [source]

A continuous function is a.e. strongly measurable for a σ-finite measure that is inner regular with respect to compact sets on sets of finite measure. Compare `Continuous.aestronglyMeasurable`, which asks for second countability of the source or the target instead.

```
lemma Continuous.aestronglyMeasurable_of_sigmaFinite [SigmaFinite μ] {f : α → β}
    (hf : Continuous f) : AEStronglyMeasurable f μ
```

lemma aestronglyMeasurable_smul [source]

The product of an a.e. finitely strongly measurable function `f`, for instance an integrable one, and a continuous function `g` is a.e. strongly measurable, for a measure

that is inner regular with respect to compact sets on sets of finite measure. No σ-finiteness of the measure and no second countability is needed: `f` vanishes outside a σ-finite set, on which `g` is a.e. strongly measurable by `Continuous.aestronglyMeasurable_of_sigmaFinite`.

```
lemma MeasureTheory.AEFinStronglyMeasurable.aestronglyMeasurable_smul {𝕜 : Type*}
    [TopologicalSpace 𝕜] [T2Space 𝕜] [Zero 𝕜] [Zero β] [SMulWithZero 𝕜 β] [ContinuousSMul 𝕜 β]
    {f : α → 𝕜} (hf : AEFinStronglyMeasurable f μ) {g : α → β} (hg : Continuous g) :
    AEStronglyMeasurable (fun x ↦ f x • g x) μ
```

2.382 PhyslibAlpha.Mathematics.TauCeti.MeasureTheory.Integral.PeakFunction

PhyslibAlpha/Mathematics/TauCeti/MeasureTheory/Integral/PeakFunction.lean on GitHub.

lemma norm_integral_smul_sub_le [source]

***The approximate-identity estimate.** If an `L¹` class `f` has unit integral and vanishes almost everywhere outside `U`, and `φ` stays within `ε` of `y` on `U`, then the integral of `φ` against `f` is within `ε * ‖f‖` of `y`. The integrand `f • φ` is assumed almost everywhere strongly measurable.*

```
lemma norm_integral_smul_sub_le {f : α →₁[μ] 𝕜} (hf : ∫ x, f x ∂μ = 1) {U : Set α}
    (hfU : ∀ᵐ x ∂μ, x ∉ U → f x = 0) {φ : α → E}
    (hφ : AEStronglyMeasurable (fun x ↦ f x • φ x) μ) {y : E}
    {ε : ℝ} (hφU : ∀ x ∈ U, ‖φ x - y‖ ≤ ε) :
    ‖(∫ x, f x • φ x ∂μ) - y‖ ≤ ε * ‖f‖
```

lemma tendsto_integral_smul_of_tendsto [source]

***`L¹` peak functions integrate a function to its limit at the peak.** Let `f i` be `L¹` classes which eventually have unit integral and `L¹` norm at most `C`, and which concentrate at `x₀`: for every neighbourhood `U` of `x₀`, eventually `f i` vanishes almost everywhere outside `U`. Then `∫ x, f i x • φ x ∂μ` tends to `a` for every `φ` tending to `a` at `x₀` whose products `f i • φ` are eventually almost everywhere strongly measurable.*

```
lemma tendsto_integral_smul_of_tendsto {ι : Type*} {l : Filter ι} {f : ι → α →₁[μ] 𝕜}
    {C : ℝ} {x₀ : α} [TopologicalSpace α] (hf : ∀ᶠ i in l, ∫ x, f i x ∂μ = 1)
    (hfC : ∀ᶠ i in l, ‖f i‖ ≤ C) (hfx₀ : ∀ U ∈ 𝓝 x₀, ∀ᶠ i in l, ∀ᵐ x ∂μ, x ∉ U → f i x = 0)
    {φ : α → E} (hφ : ∀ᶠ i in l, AEStronglyMeasurable (fun x ↦ f i x • φ x) μ) {a : E}
    (hφa : Tendsto φ (𝓝 x₀) (𝓝 a)) :
    Tendsto (fun i ↦ ∫ x, f i x • φ x ∂μ) l (𝓝 a)
```

lemma exists_integral_eq_one_norm_eq_one [source]

***`L¹` peak functions exist.** For a measure positive on nonempty open sets and finite on some neighbourhood of each point, every neighbourhood `U` of a point carries an `L¹` class of unit integral and unit `L¹` norm vanishing almost everywhere outside `U`: the indicator of a neighbourhood of finite measure inside `U`, normalized by that measure.*

```
lemma exists_integral_eq_one_norm_eq_one [TopologicalSpace α] [OpensMeasurableSpace α]
    [μ.IsOpenPosMeasure] [IsLocallyFiniteMeasure μ] {x₀ : α} {U : Set α} (hU : U ∈ 𝓝 x₀) :
    ∃ f : α →₁[μ] 𝕜, ∫ x, f x ∂μ = 1 ∧ ‖f‖ = 1 ∧ ∀ᵐ x ∂μ, x ∉ U → f x = 0
```

2.383 PhyslibAlpha.Mathematics.TauCeti.MeasureTheory.Measure.FiniteMeasureExt

PhyslibAlpha/Mathematics/TauCeti/MeasureTheory/Measure/FiniteMeasureExt.lean on GitHub.

lemma ext_of_forall_mem_submonoid_integral_eq_of_polish [source]

***A separating submonoid of test functions determines a finite measure.** Two finite Borel measures on a Polish space that give the same integral to every member of a submonoid `S` of the real bounded continuous functions are equal, provided the members of `S` separate points.*

```
lemma ext_of_forall_mem_submonoid_integral_eq_of_polish [PolishSpace E] [BorelSpace E]
    {P P' : Measure E} [IsFiniteMeasure P] [IsFiniteMeasure P'] {S : Submonoid (E →ᵇ ℝ)}
    (hS : ∀ {x y : E}, x ≠ y → ∃ g ∈ S, g x ≠ g y)
    (heq : ∀ g ∈ S, ∫ x, g x ∂P = ∫ x, g x ∂P') : P = P'
```

lemma abs_integral_mulExpNegMulSq_comp_sub_le_of_isTightMeasureSet_of_borel [source]

```
private lemma abs_integral_mulExpNegMulSq_comp_sub_le_of_isTightMeasureSet_of_borel
    [BorelSpace E]
    (hP : IsTightMeasureSet {P}) (hP' : IsTightMeasureSet {P'}) (f : E →ᵇ ℝ)
    {A : Subalgebra ℝ (E →ᵇ ℝ)} (hA : (A.map (toContinuousMapₐ ℝ)).SeparatesPoints)
    (heq : ∀ g ∈ A, ∫ x, (g : E → ℝ) x ∂P = ∫ x, (g : E → ℝ) x ∂P') {ε : ℝ} (hε : 0 < ε) :
    |∫ x, mulExpNegMulSq ε (f x) ∂P - ∫ x, mulExpNegMulSq ε (f x) ∂P'| ≤ 6 * √ε
```

lemma abs_integral_mulExpNegMulSq_comp_sub_le_of_isTightMeasureSet [source]

If two tight finite measures on a Hausdorff space integrate every member of a point-separating real subalgebra `A` of `E →ᵇ ℝ` equally, then their integrals of `mulExpNegMulSq ε ∘ f` differ by at most `6 √ε`. This is the tight form of Mathlib's `dist_integral_mulExpNegMulSq_comp_le`, which assumes a complete second-countable pseudo-metric space instead.

```
lemma abs_integral_mulExpNegMulSq_comp_sub_le_of_isTightMeasureSet
    (hP : IsTightMeasureSet {P}) (hP' : IsTightMeasureSet {P'}) (f : E →ᵇ ℝ)
    {A : Subalgebra ℝ (E →ᵇ ℝ)} (hA : (A.map (toContinuousMapₐ ℝ)).SeparatesPoints)
    (heq : ∀ g ∈ A, ∫ x, (g : E → ℝ) x ∂P = ∫ x, (g : E → ℝ) x ∂P') {ε : ℝ} (hε : 0 < ε) :
    |∫ x, mulExpNegMulSq ε (f x) ∂P - ∫ x, mulExpNegMulSq ε (f x) ∂P'| ≤ 6 * √ε
```

lemma tendsto_integral_mulExpNegMulSq_comp_of_opens [source]

```
private lemma tendsto_integral_mulExpNegMulSq_comp_of_opens (f : E →ᵇ ℝ) :
    Tendsto (fun ε => ∫ x, mulExpNegMulSq ε (f x) ∂P) (𝓝[>] 0) (𝓝 (∫ x, f x ∂P))
```

lemma integral_eq_of_forall_mem_subalgebra_integral_eq_of_isTightMeasureSet [source]

***A separating subalgebra determines the integrals of tight measures.** Two tight finite measures on a Hausdorff space that integrate every member of a point-separating star subalgebra `A` of `E →ᵇ 𝕜` equally give the same integral to every real bounded continuous function.*

```
lemma integral_eq_of_forall_mem_subalgebra_integral_eq_of_isTightMeasureSet {𝕜 : Type*} [RCLike
    𝕜]
    (hP : IsTightMeasureSet {P}) (hP' : IsTightMeasureSet {P'})
    {A : StarSubalgebra 𝕜 (E →ᵇ 𝕜)} (hA : (A.map (toContinuousMapStarₐ 𝕜)).SeparatesPoints)
    (heq : ∀ g ∈ A, ∫ x, (g : E → 𝕜) x ∂P = ∫ x, (g : E → 𝕜) x ∂P') (f : E →ᵇ ℝ) :
    ∫ x, f x ∂P = ∫ x, f x ∂P'
```

lemma ext_of_forall_mem_subalgebra_integral_eq_of_innerRegular [source]

***A separating subalgebra determines a finite inner regular measure.** On a locally compact Hausdorff space, two finite inner regular (equivalently, for finite Borel measures there, regular) measures that integrate every member of a point-separating star subalgebra `A` of `E →ᵇ 𝕜` equally are equal. Unlike `MeasureTheory.ext_of_forall_mem_subalgebra_integral_e` no metrizability or countability is assumed.*

```
lemma ext_of_forall_mem_subalgebra_integral_eq_of_innerRegular {𝕜 : Type*} [RCLike 𝕜]
    [BorelSpace E] [LocallyCompactSpace E] [P.InnerRegular] [P'.InnerRegular]
    {A : StarSubalgebra 𝕜 (E →ᵇ 𝕜)} (hA : (A.map (toContinuousMapStarₐ 𝕜)).SeparatesPoints)
    (heq : ∀ g ∈ A, ∫ x, (g : E → 𝕜) x ∂P = ∫ x, (g : E → 𝕜) x ∂P') : P = P'
```

2.384 PhyslibAlpha.Mathematics.TauCeti.MeasureTheory.Measure.Regular

PhyslibAlpha/Mathematics/TauCeti/MeasureTheory/Measure/Regular.lean on GitHub.

lemma innerRegular_map_of_continuousOn [source]

A measurable map continuous on a measurable set of full measure preserves inner regularity. No separation, countability or finiteness assumption is required.

```
lemma innerRegular_map_of_continuousOn {μ : Measure α} [μ.InnerRegular]
    {f : α → β} {s : Set α} (hf : Measurable f) (hs : MeasurableSet s)
    (hcont : ContinuousOn f s) (hmem : ∀ᵐ x ∂μ, x ∈ s) :
    (Measure.map f μ).InnerRegular
```

2.385 PhyslibAlpha.Mathematics.TauCeti.Order.Filter.SmallSets

PhyslibAlpha/Mathematics/TauCeti/Order/Filter/SmallSets.lean on GitHub.

lemma comap_val_smallSets_neBot [source]

***A filter indexes a nontrivial net by its own members.** A net indexed by the members of a filter `l` converges along this filter, the members read through `Filter.smallSets`, so that `∀ᶠ s in _, p s` says that `p` holds for every small enough member of `l`; it is not the bottom filter, every member of `l` being an index below itself.*

```
lemma comap_val_smallSets_neBot {α : Type*} (l : Filter α) :
    NeBot (comap (Subtype.val : {s : Set α // s ∈ l} → Set α) l.smallSets)
```

2.386 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Integrated.Algebra

PhyslibAlpha/Mathematics/TauCeti/RepresentationTheory/Continuous/Integrated/Algebra.lean on GitHub.

def integratedAlgebra [source]

The closed unital star algebra generated by the integrated operators of `π`.

```
noncomputable def _root_.ContRepresentation.integratedAlgebra :
    StarSubalgebra ℂ (H →L[ℂ] H)
```

lemma integratedAlgebra_def [source]

The integrated algebra is the closure of the star algebra generated by the integrated form.

```
lemma _root_.ContRepresentation.integratedAlgebra_def :
    π.integratedAlgebra hcont hbdd μ =
      (StarAlgebra.adjoin ℂ
        (Set.range (π.integratedOperatorL1 hcont hbdd μ))).topologicalClosure
```

lemma integratedOperatorL1_mem_integratedAlgebra [source]

Every integrated operator belongs to the integrated algebra.

```
@[simp]
lemma _root_.ContRepresentation.integratedOperatorL1_mem_integratedAlgebra
    (f : G →₁[μ] ℂ) :
    π.integratedOperatorL1 hcont hbdd μ f ∈ π.integratedAlgebra hcont hbdd μ
```

def integratedOperatorL1ToAlgebra [source]

The integrated form with values in its closed star algebra.

```
noncomputable def _root_.ContRepresentation.integratedOperatorL1ToAlgebra :
    (G →₁[μ] ℂ) →L[ℂ] π.integratedAlgebra hcont hbdd μ
```

lemma coe_integratedOperatorL1ToAlgebra [source]

Coercing the algebra-valued integrated form recovers the original integrated operator.

```
@[simp]
lemma _root_.ContRepresentation.coe_integratedOperatorL1ToAlgebra (f : G →₁[μ] ℂ) :
    (π.integratedOperatorL1ToAlgebra hcont hbdd μ f : H →L[ℂ] H) =
      π.integratedOperatorL1 hcont hbdd μ f
```

lemma star_integratedOperatorL1ToAlgebra [source]

The algebra-valued integrated form of a unitary representation takes the involuted weight to the star of the integrated operator.

```
@[simp]
lemma _root_.ContRepresentation.star_integratedOperatorL1ToAlgebra
    [MeasurableNeg G] [μ.IsNegInvariant] (hπ : ContRepresentation.IsUnitary π)
    (f : G →₁[μ] ℂ) :
    star (π.integratedOperatorL1ToAlgebra hcont hbdd μ f) =
      π.integratedOperatorL1ToAlgebra hcont hbdd μ
        (star (Lp.compMeasurePreserving Neg.neg (Measure.measurePreserving_neg μ) f))
```

lemma isClosed_integratedAlgebra [source]

The integrated algebra is closed in the operator norm.

```
@[instance]
lemma _root_.ContRepresentation.isClosed_integratedAlgebra :
    IsClosed (π.integratedAlgebra hcont hbdd μ : Set (H →L[ℂ] H))
```

lemma integratedAlgebra_le_iff [source]

A closed star subalgebra contains the integrated algebra exactly when it contains every integrated operator.

```
lemma _root_.ContRepresentation.integratedAlgebra_le_iff (B : StarSubalgebra ℂ (H →L[ℂ] H))
    (hB : IsClosed (B : Set (H →L[ℂ] H))) :
    π.integratedAlgebra hcont hbdd μ ≤ B ↔
      ∀ f : G →₁[μ] ℂ, π.integratedOperatorL1 hcont hbdd μ f ∈ B
```

lemma comp_integratedOperatorL1_mem_integratedAlgebra [source]

A group translate of an integrated operator still belongs to the integrated algebra.

```
@[simp]
lemma _root_.ContRepresentation.comp_integratedOperatorL1_mem_integratedAlgebra
    (g : G) (f : G →₁[μ] ℂ) :
    π (.ofAdd g) ∘L π.integratedOperatorL1 hcont hbdd μ f ∈
      π.integratedAlgebra hcont hbdd μ
```

def translatedIntegratedOperatorL1 [source]

Translated integrated operators, as elements of the integrated algebra.

```
noncomputable def _root_.ContRepresentation.translatedIntegratedOperatorL1
    (f : G →₁[μ] ℂ) (g : G) : π.integratedAlgebra hcont hbdd μ
```

lemma coe_translatedIntegratedOperatorL1 [source]

The underlying operator of a translated integrated operator is `π(g) π(f)`.

```
@[simp]
lemma _root_.ContRepresentation.coe_translatedIntegratedOperatorL1
    (f : G →₁[μ] ℂ) (g : G) :
    (π.translatedIntegratedOperatorL1 hcont hbdd μ f g : H →L[ℂ] H) =
      π (.ofAdd g) ∘L π.integratedOperatorL1 hcont hbdd μ f
```

lemma translatedIntegratedOperatorL1_eq [source]

A translated integrated operator is the algebra-valued integrated form of the translated weight.

```
lemma _root_.ContRepresentation.translatedIntegratedOperatorL1_eq
    (f : G →₁[μ] ℂ) (g : G) :
    π.translatedIntegratedOperatorL1 hcont hbdd μ f g =
      π.integratedOperatorL1ToAlgebra hcont hbdd μ
        (Lp.compMeasurePreserving (fun x ↦ -g + x) (measurePreserving_add_left μ (-g)) f)
```

lemma translatedIntegratedOperatorL1_zero [source]

Translation by zero recovers the algebra-valued integrated form.

```
@[simp]
lemma _root_.ContRepresentation.translatedIntegratedOperatorL1_zero (f : G →₁[μ] ℂ) :
    π.translatedIntegratedOperatorL1 hcont hbdd μ f 0 =
      π.integratedOperatorL1ToAlgebra hcont hbdd μ f
```

lemma continuous_translatedIntegratedOperatorL1 [source]

Translated integrated operators depend continuously on the group element in the norm of the integrated algebra.

```
lemma _root_.ContRepresentation.continuous_translatedIntegratedOperatorL1
    [IsTopologicalAddGroup G] [IsLocallyFiniteMeasure μ] (f : G →₁[μ] ℂ) :
    Continuous (π.translatedIntegratedOperatorL1 hcont hbdd μ f)
```

lemma integratedOperatorL1ToAlgebra_convolution [source]

The algebra-valued integrated form takes convolution of weights to multiplication.

```
lemma _root_.ContRepresentation.integratedOperatorL1ToAlgebra_convolution {f₁ f₂ : G → ℂ}
    (hf₁ : Integrable f₁ μ) (hf₂ : Integrable f₂ μ) :
    π.integratedOperatorL1ToAlgebra hcont hbdd μ
        ((hf₁.integrable_convolution (ContinuousLinearMap.mul ℂ ℂ) hf₂).toL1 _) =
      π.integratedOperatorL1ToAlgebra hcont hbdd μ (hf₁.toL1 f₁) *
        π.integratedOperatorL1ToAlgebra hcont hbdd μ (hf₂.toL1 f₂)
```

lemma translatedIntegratedOperatorL1_convolution [source]

Translating the integrated form of a convolution translates the first factor in its product.

```
lemma _root_.ContRepresentation.translatedIntegratedOperatorL1_convolution
    [μ.IsAddLeftInvariant] {f₁ f₂ : G → ℂ} (hf₁ : Integrable f₁ μ) (hf₂ : Integrable f₂ μ)
    (g : G) :
    π.translatedIntegratedOperatorL1 hcont hbdd μ
        ((hf₁.integrable_convolution (ContinuousLinearMap.mul ℂ ℂ) hf₂).toL1 _) g =
      π.translatedIntegratedOperatorL1 hcont hbdd μ (hf₁.toL1 f₁) g *
        π.integratedOperatorL1ToAlgebra hcont hbdd μ (hf₂.toL1 f₂)
```

lemma isMulCommutative_integratedAlgebra [source]

For a unitary abelian-group representation, the closed algebra of integrated operators is commutative.

```
lemma _root_.ContRepresentation.isMulCommutative_integratedAlgebra
    (hπ : ContRepresentation.IsUnitary π) :
    IsMulCommutative (π.integratedAlgebra hcont hbdd μ)
```

2.387 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Integrated.Basic

PhyslibAlpha/Mathematics/TauCeti/RepresentationTheory/Continuous/Integrated/Basic.lean on GitHub.

lemma integrable_smul_apply [source]

The integrand of the integrated form is integrable.

```
private lemma integrable_smul_apply (hcont : ∀ v, Continuous fun g : G ↦ π (.ofAdd g) v)
    {C : ℝ} (hC : ∀ g, ‖π g‖ ≤ C) {f : G → 𝕜} (hf : Integrable f μ) (v : E) :
    Integrable (fun g ↦ f g • π (.ofAdd g) v) μ
```

lemma norm_integral_smul_apply_le [source]

The integral of the integrand of the integrated form is bounded by the `L¹` norm of the weight.

```
private lemma norm_integral_smul_apply_le {C : ℝ} (hC : ∀ g, ‖π g‖ ≤ C) {f : G → 𝕜}
    (hf : Integrable f μ) (v : E) :
    ‖∫ g, f g • π (.ofAdd g) v ∂μ‖ ≤ C * (∫ g, ‖f g‖ ∂μ) * ‖v‖
```

def integratedOperatorL1 [source]

*The **integrated form** of a strongly continuous, uniformly bounded representation `π` of an additive group `G` (written multiplicatively as `Multiplicative G`): the bounded operator `π(f) = ∫ g, f g • π g ∂μ` attached to an integrable weight `f`, defined pointwise by the Bochner integral `π(f) v = ∫ g, f g • π g v ∂μ` of the continuous orbit `g ↦ π g v`.*

```
noncomputable def _root_.ContRepresentation.integratedOperatorL1
    (hcont : ∀ v, Continuous fun g : G ↦ π (.ofAdd g) v)
    (hbdd : ∃ C, ∀ g, ‖π g‖ ≤ C) (μ : Measure G) [μ.InnerRegularCompactLTTop] :
    (G →₁[μ] 𝕜) →L[𝕜] E →L[𝕜] E
```

lemma integratedOperatorL1_apply [source]

The integrated form acts on a vector by integrating its orbit against the weight.

```
lemma _root_.ContRepresentation.integratedOperatorL1_apply (f : G →₁[μ] 𝕜) (v : E) :
    π.integratedOperatorL1 hcont hbdd μ f v = ∫ g, f g • π (.ofAdd g) v ∂μ
```

lemma integratedOperatorL1_toL1 [source]

The integrated form of the class of an integrable function.

```
@[simp]
lemma _root_.ContRepresentation.integratedOperatorL1_toL1 {f : G → 𝕜} (hf : Integrable f μ)
    (v : E) :
    π.integratedOperatorL1 hcont hbdd μ (hf.toL1 f) v = ∫ g, f g • π (.ofAdd g) v ∂μ
```

lemma norm_integratedOperatorL1_le [source]

The integrated form is bounded by the uniform bound of the representation times the `L¹` norm of the weight.

```
lemma _root_.ContRepresentation.norm_integratedOperatorL1_le {C : ℝ} (hC : ∀ g, ‖π g‖ ≤ C)
    (f : G →₁[μ] 𝕜) :
    ‖π.integratedOperatorL1 hcont hbdd μ f‖ ≤ C * ‖f‖
```

lemma comp_integratedOperatorL1 [source]

Translating the weight on the left by `h` amounts to composing the integrated form with `π h`.

```
lemma _root_.ContRepresentation.comp_integratedOperatorL1 [CompleteSpace E] [MeasurableAdd G]
    [μ.IsAddLeftInvariant] (h : G) (f : G →₁[μ] 𝕜) :
    π (.ofAdd h) ∘L π.integratedOperatorL1 hcont hbdd μ f =
```

```
    π.integratedOperatorL1 hcont hbdd μ
      (Lp.compMeasurePreserving (fun g ↦ -h + g) (measurePreserving_add_left μ (-h)) f)
```

lemma continuous_comp_integratedOperatorL1 [source]

Although `π` is only strongly continuous, `h ↦ π h ∘L π(f)` is continuous in the operator norm: by `comp_integratedOperatorL1` it is the integrated form of the left translates of `f`, which depend continuously on `h` in `L¹`.

```
lemma _root_.ContRepresentation.continuous_comp_integratedOperatorL1 [CompleteSpace E]
    [IsTopologicalAddGroup G] [MeasurableAdd G] [μ.IsAddLeftInvariant] [IsLocallyFiniteMeasure
    μ]
    (f : G →₁[μ] 𝕜) :
    Continuous fun h : G ↦ π (.ofAdd h) ∘L π.integratedOperatorL1 hcont hbdd μ f
```

lemma commute_integratedOperatorL1 [source]

Every action operator of a representation of an abelian group commutes with its integrated operators. No invariance hypothesis on the measure is needed.

```
lemma _root_.ContRepresentation.commute_integratedOperatorL1
    (g : Multiplicative G) (f : G →₁[μ] 𝕜) :
    Commute (π g) (π.integratedOperatorL1 hcont hbdd μ f)
```

lemma integratedOperatorL1_commute [source]

Integrated operators of an abelian-group representation commute for any measure.

```
lemma _root_.ContRepresentation.integratedOperatorL1_commute (f₁ f₂ : G →₁[μ] 𝕜) :
    Commute (π.integratedOperatorL1 hcont hbdd μ f₁)
      (π.integratedOperatorL1 hcont hbdd μ f₂)
```

lemma integratedOperatorL1_convolution [source]

The integrated form turns convolution of weights into composition of operators: `π(f₁ ⋆ f₂) = π(f₁) ∘L π(f₂)`.

```
lemma _root_.ContRepresentation.integratedOperatorL1_convolution {f₁ f₂ : G → 𝕜}
    (hf₁ : Integrable f₁ μ) (hf₂ : Integrable f₂ μ) :
    π.integratedOperatorL1 hcont hbdd μ
        ((hf₁.integrable_convolution (ContinuousLinearMap.mul 𝕜 𝕜) hf₂).toL1 _) =
      π.integratedOperatorL1 hcont hbdd μ (hf₁.toL1 f₁) ∘L
        π.integratedOperatorL1 hcont hbdd μ (hf₂.toL1 f₂)
```

lemma inner_integratedOperatorL1_apply [source]

Matrix coefficients of the integrated form are the integrals of the matrix coefficients of the representation against the weight.

```
lemma _root_.ContRepresentation.inner_integratedOperatorL1_apply (f : G →₁[μ] 𝕜) (w v : E) :
    ⟪w, π.integratedOperatorL1 hcont hbdd μ f v⟫_𝕜 = ∫ g, f g * ⟪w, π (.ofAdd g) v⟫_𝕜 ∂μ
```

lemma adjoint_integratedOperatorL1 [source]

The adjoint of the integrated form of a unitary representation is the integrated form of the involuted weight `g ↦ conj (f (-g))`.

```
lemma _root_.ContRepresentation.adjoint_integratedOperatorL1 [MeasurableNeg G]
    [μ.IsNegInvariant] (hπ : ContRepresentation.IsUnitary π) (f : G →₁[μ] 𝕜) :
    ContinuousLinearMap.adjoint (π.integratedOperatorL1 hcont hbdd μ f) =
      π.integratedOperatorL1 hcont hbdd μ
        (star (Lp.compMeasurePreserving Neg.neg (Measure.measurePreserving_neg μ) f))
```

lemma isStarNormal_integratedOperatorL1 [source]

The integrated operators of a unitary abelian-group representation are normal.

```
lemma _root_.ContRepresentation.isStarNormal_integratedOperatorL1
    (hπ : ContRepresentation.IsUnitary π) (f : G →₁[μ] 𝕜) :
    IsStarNormal (π.integratedOperatorL1 hcont hbdd μ f)
```

lemma tendsto_integratedOperatorL1_apply [source]

***Approximate identities for the integrated form.** If the weights `f i` eventually have unit integral and `L¹` norm at most `C`, and concentrate at `0` (for every neighbourhood `U` of `0`, eventually `f i` vanishes almost everywhere outside `U`), then `π(f i)` tends strongly to the identity: `π(f i) v → v` for every `v`.*

```
lemma _root_.ContRepresentation.tendsto_integratedOperatorL1_apply {ι : Type*} {l : Filter ι}
    {f : ι → G →₁[μ] 𝕜} {C : ℝ} (hf : ∀ᶠ i in l, ∫ g, f i g ∂μ = 1)
    (hfC : ∀ᶠ i in l, ‖f i‖ ≤ C)
    (hf₀ : ∀ U ∈ 𝒩 (0 : G), ∀ᶠ i in l, ∀ᵐ g ∂μ, g ∉ U → f i g = 0) (v : E) :
    Tendsto (fun i ↦ π.integratedOperatorL1 hcont hbdd μ (f i) v) l (𝒩 v)
```

lemma mem_closure_range_integratedOperatorL1_apply [source]

***The integrated form is nondegenerate.** For a measure positive on nonempty open sets and finite on some neighbourhood of each point, such as a Haar measure on a locally compact group, every vector `v` is a limit of vectors `π(f) v`.*

```
lemma _root_.ContRepresentation.mem_closure_range_integratedOperatorL1_apply
    [μ.IsOpenPosMeasure] [IsLocallyFiniteMeasure μ] (v : E) :
    v ∈ closure (Set.range fun f : G →₁[μ] 𝕜 ↦ π.integratedOperatorL1 hcont hbdd μ f v)
```

2.388 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Pontryagin.Basic

PhyslibAlpha/Mathematics/TauCeti/RepresentationTheory/Continuous/Pontryagin/Basic.lean on GitHub.

lemma existsUnique_pontryaginDual_of_integratedOperatorL1 [source]

A nonvanishing character of an algebra containing the integrated form of a strongly continuous unitary representation detects a unique continuous group character. Its value is the multiplier on every integrated operator, independently of the witnessing weight.

```
lemma _root_.ContRepresentation.existsUnique_pontryaginDual_of_integratedOperatorL1
    (π : ContRepresentation ℂ (Multiplicative G) H)
    {hcont : ∀ v, Continuous fun g : G => π (.ofAdd g) v}
```

```
    (hbdd : ∃ C, ∀ g, ‖π g‖ ≤ C)
    (hπ : ContRepresentation.IsUnitary π)
    (A : StarSubalgebra ℂ (H →L[ℂ] H)) [CompleteSpace A] :
    ∀ (hA : ∀ f : G →₁[μ] ℂ, π.integratedOperatorL1 hcont hbdd μ f ∈ A)
    (ω : characterSpace ℂ A),
    (∃ f : G →₁[μ] ℂ, ω ⟨π.integratedOperatorL1 hcont hbdd μ f, hA f⟩ ≠ 0) →
    ∃! χ : PontryaginDual (Multiplicative G), ∀ (g : G) (f : G →₁[μ] ℂ),
      ω ⟨π.integratedOperatorL1 hcont hbdd μ
          (Lp.compMeasurePreserving (fun t => -g + t)
            (measurePreserving_add_left μ (-g)) f), hA _⟩ =
        (χ (.ofAdd g) : ℂ) * ω ⟨π.integratedOperatorL1 hcont hbdd μ f, hA f⟩
```

2.389 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Pontryagin.Continuit

PhyslibAlpha/Mathematics/TauCeti/RepresentationTheory/Continuous/Pontryagin/Continuity.lean on GitHub.

def integratedCharacterSet [source]

The characters of an algebra containing the integrated form which do not annihilate every integrated operator. When π is unitary, these are exactly the algebra characters from which the representation detects a point of the Pontryagin dual.

```
def integratedCharacterSet : Set (characterSpace ℂ A)
```

lemma mem_integratedCharacterSet_iff [source]

Membership in the integrated-character set means nonvanishing on some integrated operator.

```
@[simp]
lemma mem_integratedCharacterSet_iff (ω : characterSpace ℂ A) :
    ω ∈ π.integratedCharacterSet hcont hbdd A hA ↔
      ∃ f : G →₁[μ] ℂ, ω ⟨π.integratedOperatorL1 hcont hbdd μ f, hA f⟩ ≠ 0
```

lemma isOpen_integratedCharacterSet [source]

The integrated-character set is open in the character space.

```
lemma isOpen_integratedCharacterSet :
    IsOpen (π.integratedCharacterSet hcont hbdd A hA)
```

def integratedCharacterToPontryaginDual [source]

The continuous group character detected by an algebra character that does not annihilate the integrated form.

```
noncomputable def integratedCharacterToPontryaginDual
    (ω : π.integratedCharacterSet hcont hbdd A hA) : PontryaginDual (Multiplicative G)
```

lemma integratedCharacterToPontryaginDual_spec [source]

The defining multiplier identity for the group character detected by an integrated character.

```
lemma integratedCharacterToPontryaginDual_spec
    (ω : π.integratedCharacterSet hcont hbdd A hA) (g : G) (f : G →₁[μ] ℂ) :
    ω.1
        ⟨π.integratedOperatorL1 hcont hbdd μ
          (Lp.compMeasurePreserving (fun t => -g + t)
            (measurePreserving_add_left μ (-g)) f), hA _⟩ =
      (π.integratedCharacterToPontryaginDual hcont hbdd A hA hπ ω (.ofAdd g) : ℂ) *
        ω.1 ⟨π.integratedOperatorL1 hcont hbdd μ f, hA f⟩
```

lemma coe_integratedCharacterToPontryaginDual_apply_eq_div [source]

Local quotient formula for the group character detected by an integrated character.

```
lemma coe_integratedCharacterToPontryaginDual_apply_eq_div
    (ω : π.integratedCharacterSet hcont hbdd A hA) (g : G) (f : G →₁[μ] ℂ)
    (hf : ω.1 ⟨π.integratedOperatorL1 hcont hbdd μ f, hA f⟩ ≠ 0) :
    (π.integratedCharacterToPontryaginDual hcont hbdd A hA hπ ω (.ofAdd g) : ℂ) =
      ω.1 ⟨π.integratedOperatorL1 hcont hbdd μ
        (Lp.compMeasurePreserving (fun t => -g + t)
          (measurePreserving_add_left μ (-g)) f), hA _⟩ /
        ω.1 ⟨π.integratedOperatorL1 hcont hbdd μ f, hA f⟩
```

lemma continuous_integratedCharacterToPontryaginDual [source]

The group character detected by a nonvanishing integrated character depends continuously on the algebra character.

```
lemma continuous_integratedCharacterToPontryaginDual :
    Continuous (π.integratedCharacterToPontryaginDual hcont hbdd A hA hπ)
```

2.390 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Subrepresentation

PhyslibAlpha/Mathematics/TauCeti/RepresentationTheory/Continuous/Subrepresentation.lean on GitHub.

def subrepresentation [source]

The restriction of a continuous representation to an invariant submodule. This is the continuous counterpart of `Representation.subrepresentation`.

```
def subrepresentation (π : ContRepresentation R G V) (W : Submodule R V)
    (hW : ∀ g, ∀ v ∈ W, π g v ∈ W) : ContRepresentation R G W
```

lemma coe_subrepresentation_apply [source]

The restricted action is the ambient action, read on the underlying vectors.

```
@[simp]
lemma coe_subrepresentation_apply (g : G) (v : W) :
    ((subrepresentation π W hW g v : W) : V) = π g (v : V)
```

def subrepresentationInclusion [source]

The inclusion of a subrepresentation into its ambient continuous representation, packaged as a continuous intertwiner.

```
noncomputable def subrepresentationInclusion
    (π : ContRepresentation R G V) (σ : Subrepresentation π.toRepresentation) :
    ContIntertwiningMap
      (subrepresentation π σ.toSubmodule
        (fun g _ hv ↦ σ.apply_mem_toSubmodule g hv)) π
```

lemma subrepresentationInclusion_apply [source]

The subrepresentation inclusion sends a vector to the same vector in the ambient space.

```
@[simp]
lemma subrepresentationInclusion_apply (π : ContRepresentation R G V)
    (σ : Subrepresentation π.toRepresentation) (v : σ.toSubmodule) :
    π.subrepresentationInclusion σ v = (v : V)
```

lemma mem_invariants_subrepresentation [source]

A vector of an invariant submodule is invariant for the restricted representation exactly when it is invariant for the ambient one: the restricted action is the ambient action.

```
lemma mem_invariants_subrepresentation {x : W} :
    x ∈ (subrepresentation π W hW).invariants ↔ (x : V) ∈ π.invariants
```

lemma toRepresentation_subrepresentation [source]

The underlying representation of a restricted continuous representation is the restriction of the underlying representation.

```
@[simp]
lemma toRepresentation_subrepresentation :
    (subrepresentation π W hW).toRepresentation
      = π.toRepresentation.subrepresentation W fun g _ hv => hW g _ hv
```

lemma toRepresentation_subrepresentation_toSubmodule [source]

Restricting `π` to the submodule a subrepresentation `σ` of `π.toRepresentation` carries has `σ.toRepresentation` as its underlying representation: both restrict the ambient action to the same submodule.

```
lemma _root_.Subrepresentation.toRepresentation_subrepresentation_toSubmodule
    (σ : Subrepresentation π.toRepresentation) :
    (subrepresentation π σ.toSubmodule
      (fun g _ hv ↦ σ.apply_mem_toSubmodule g hv)).toRepresentation = σ.toRepresentation
```

def quotient [source]

A continuous representation descended to the quotient by an invariant submodule, which carries the quotient topology. This is the continuous counterpart of `Representation.quotient`.

```
def quotient (π : ContRepresentation R G V) (W : Submodule R V)
    (hW : ∀ g, ∀ v ∈ W, π g v ∈ W) : ContRepresentation R G (V ⧸ W)
```

lemma quotient_apply_mk [source]

The descended action on the class of `v` is the class of the ambient action on `v`.

```
@[simp]
lemma quotient_apply_mk (g : G) (v : V) :
    quotient π W hW g (Submodule.Quotient.mk v) = Submodule.Quotient.mk (π g v)
```

lemma toRepresentation_quotient [source]

The underlying representation of a descended continuous representation is the descended underlying representation.

```
@[simp]
lemma toRepresentation_quotient :
    (quotient π W hW).toRepresentation
      = π.toRepresentation.quotient W fun g _ hv => hW g _ hv
```

lemma continuous_subrepresentation [source]

Restricting a continuous representation to an invariant submodule preserves continuity: from continuity of `g ↦ π g` as a map into the continuous linear endomorphisms of `V`, the restricted action `g ↦ subrepresentation π W hW g` is continuous into those of `W`. This supplies the continuity argument that `matrixCoeff` and the rest of the continuous-representation API take explicitly, so a subrepresentation can be used wherever a continuous representation is expected.

```
lemma continuous_subrepresentation (hπ : Continuous π) :
    Continuous (subrepresentation π W hW)
```

2.391 PhyslibAlpha.Mathematics.TauCeti.RepresentationTheory.Continuous.Unitary.Basic

PhyslibAlpha/Mathematics/TauCeti/RepresentationTheory/Continuous/Unitary/Basic.lean on GitHub.

lemma self_inv_apply [source]

Applying a continuous group representation at an element and then at its inverse cancels.

This restates `Representation.self_inv_apply` for the continuous action coercion, which `simp` does not otherwise reach; it is an implementation detail of the inverse-transfer proofs below.

```
private lemma self_inv_apply (π : ContRepresentation R G V) (g : G) (v : V) :
    π g (π g⁻¹ v) = v
```

def IsUnitary [source]

A continuous representation is unitary when every action operator preserves the inner product.

```
def IsUnitary (π : ContRepresentation 𝕜 G V) : Prop
```

lemma isUnitary_iff_norm_map [source]

A representation is unitary exactly when every action map preserves norms.

```
lemma isUnitary_iff_norm_map (π : ContRepresentation 𝕜 G V) :
    IsUnitary π ↔ ∀ g v, ‖π g v‖ = ‖v‖
```

lemma isUnitary_iff_isometry [source]

A representation is unitary exactly when every action map is an isometry.

```
lemma isUnitary_iff_isometry (π : ContRepresentation 𝕜 G V) :
    IsUnitary π ↔ ∀ g, Isometry (π g)
```

lemma inner_map_map [source]

A unitary representation preserves the inner product.

```
@[simp]
lemma inner_map_map (hπ : IsUnitary π) (g : G) (v w : V) :
    ⟪π g v, π g w⟫_𝕜 = ⟪v, w⟫_𝕜
```

lemma norm_map [source]

Every action map of a unitary representation preserves norms.

```
@[simp]
lemma norm_map (hπ : IsUnitary π) (g : G) (v : V) :
    ‖π g v‖ = ‖v‖
```

lemma norm_le_one [source]

Every action operator of a unitary representation has operator norm at most `1`.

```
lemma norm_le_one (hπ : IsUnitary π) (g : G) : ‖π g‖ ≤ 1
```

lemma exists_norm_le [source]

The action operators of a unitary representation are uniformly bounded, in the form taken by the integrated-form API.

```
lemma exists_norm_le (hπ : IsUnitary π) : ∃ C, ∀ g, ‖π g‖ ≤ C
```

lemma isometry [source]

Every action map of a unitary representation is an isometry.

```
lemma isometry (hπ : IsUnitary π) (g : G) : Isometry (π g)
```

lemma injective [source]

Every action map of a unitary representation is injective.

```
lemma injective (hπ : IsUnitary π) (g : G) : Function.Injective (π g)
```

lemma restrict [source]

Restriction along a monoid homomorphism preserves unitarity.

```
lemma restrict {H : Type*} [Monoid H] (hπ : IsUnitary π) (φ : H →* G) :
    IsUnitary (π.restrict φ)
```

lemma subrepresentation [source]

The restriction of a unitary representation to an invariant submodule is unitary.

```
lemma subrepresentation (hπ : IsUnitary π) {W : Submodule 𝕜 V}
    (hW : ∀ g, ∀ v ∈ W, π g v ∈ W) : IsUnitary (subrepresentation π W hW)
```

lemma norm_inner_map_le [source]

A matrix coefficient of a unitary representation is bounded by the product of the vector norms.

```
lemma norm_inner_map_le (hπ : IsUnitary π) (g : G) (v w : V) :
    ‖⟪π g v, w⟫_𝕜‖ ≤ ‖v‖ * ‖w‖
```

lemma adjoint_comp_self [source]

The adjoint of a unitary action operator is a left inverse.

```
lemma adjoint_comp_self (hπ : IsUnitary π) (g : G) :
    (ContinuousLinearMap.adjoint (π g)).comp (π g) = 1
```

lemma isUnitary_trivial [source]

The trivial continuous representation is unitary.

```
lemma isUnitary_trivial :
    IsUnitary (ContRepresentation.trivial 𝕜 G V)
```

lemma isUnitary_iff_mem_unitary [source]

For a group representation, inner-product preservation is equivalent to every action operator belonging to Mathlib's unitary submonoid.

```
lemma isUnitary_iff_mem_unitary (π : ContRepresentation 𝕜 G V) :
    IsUnitary π ↔ ∀ g, π g ∈ unitary (V →L[𝕜] V)
```

lemma mem_unitary [source]

Every action operator of a unitary group representation belongs to Mathlib's `unitary` submonoid.

```
lemma mem_unitary (hπ : IsUnitary π) (g : G) :
    π g ∈ unitary (V →L[𝕜] V)
```

lemma self_comp_adjoint [source]

The adjoint of a unitary action operator is a right inverse.

```
lemma self_comp_adjoint (hπ : IsUnitary π) (g : G) :
    (π g).comp (ContinuousLinearMap.adjoint (π g)) = 1
```

lemma inner_map_left [source]

Moving a unitary action from the first inner-product argument to the second replaces the group element by its inverse.

```
lemma inner_map_left (hπ : IsUnitary π) (g : G) (v w : V) :
    ⟪π g v, w⟫_𝕜 = ⟪v, π g⁻¹ w⟫_𝕜
```

lemma inner_map_right [source]

Moving a unitary action from the second inner-product argument to the first replaces the group element by its inverse.

```
lemma inner_map_right (hπ : IsUnitary π) (g : G) (v w : V) :
    ⟪v, π g w⟫_𝕜 = ⟪π g⁻¹ v, w⟫_𝕜
```

lemma adjoint_eq_inv [source]

The adjoint of a unitary action operator is the action of the inverse group element.

```
@[simp]
lemma adjoint_eq_inv (hπ : IsUnitary π) (g : G) :
    ContinuousLinearMap.adjoint (π g) = π g⁻¹
```

2.392 PhyslibAlpha.Mathematics.TauCeti.Topology.Algebra.ContinuousMonoidHom.Polish

PhyslibAlpha/Mathematics/TauCeti/Topology/Algebra/ContinuousMonoidHom/Polish.lean on GitHub.

instance instPolishSpace [source]

Continuous homomorphisms from a second-countable locally compact monoid to a Polish topological monoid form a Polish space in the compact-open topology.

```
instance _root_.ContinuousMonoidHom.instPolishSpace {A B : Type*}
    [Monoid A] [TopologicalSpace A] [LocallyCompactSpace A] [SecondCountableTopology A]
    [Monoid B] [TopologicalSpace B] [ContinuousMul B] [PolishSpace B] :
    PolishSpace (A →ₜ* B)
```

2.393 PhyslibAlpha.Mathematics.TauCeti.Topology.Algebra.PontryaginDual

PhyslibAlpha/Mathematics/TauCeti/Topology/Algebra/PontryaginDual.lean on GitHub.

instance instPolishSpacePontryaginDual [source]

The Pontryagin dual of a second-countable locally compact monoid is Polish.

```
instance instPolishSpacePontryaginDual {A : Type*} [Monoid A] [TopologicalSpace A]
    [LocallyCompactSpace A] [SecondCountableTopology A] : PolishSpace (PontryaginDual A)
```

def circleEquivPontryaginDualInt [source]

The unit circle is the Pontryagin dual of $\mathbb{Z}$: the point `z` corresponds to the character $n \mapsto z^n$, and a character corresponds to its value at `1`.

```
def circleEquivPontryaginDualInt : Circle ≃ₜ* PontryaginDual (Multiplicative ℤ) where
```

lemma circleEquivPontryaginDualInt_apply_apply [source]

The character attached to `z` sends `n` to z^n.

```
@[simp]
lemma circleEquivPontryaginDualInt_apply_apply (z : Circle) (n : Multiplicative ℤ) :
    circleEquivPontryaginDualInt z n = z ^ n.toAdd
```

lemma circleEquivPontryaginDualInt_symm_apply [source]

The point of the circle attached to a character of $\mathbb{Z}$ is its value at `1`.

```
@[simp]
lemma circleEquivPontryaginDualInt_symm_apply (χ : PontryaginDual (Multiplicative ℤ)) :
    circleEquivPontryaginDualInt.symm χ = χ (Multiplicative.ofAdd 1)
```

2.394 PhyslibAlpha.Mathematics.TensorProductComm

PhyslibAlpha/Mathematics/TensorProductComm.lean on GitHub.

lemma lTensor_map_id_comm [source]

An endomorphism of the second tensor factor commutes with one of the first.

```
lemma lTensor_map_id_comm {k : Type} [CommSemiring k] {W X : Type} [AddCommMonoid W]
    [Module k W] [AddCommMonoid X] [Module k X] (f : X →ₗ[k] X) (g : W →ₗ[k] W)
    (t : W ⊗[k] X) :
    (LinearMap.lTensor W f) (TensorProduct.map g LinearMap.id t) =
      TensorProduct.map g LinearMap.id ((LinearMap.lTensor W f) t)
```

lemma congr_assoc_map_id_comm [source]

Reassociating and recombining the last two tensor factors commutes with an endomorphism of the first.

```
lemma congr_assoc_map_id_comm {k : Type} [CommSemiring k] {W X Y Z : Type} [AddCommMonoid W]
    [Module k W] [AddCommMonoid X] [Module k X] [AddCommMonoid Y] [Module k Y]
    [AddCommMonoid Z] [Module k Z] (E : X ⊗[k] Y ≃ₗ[k] Z) (g : W →ₗ[k] W)
    (t : (W ⊗[k] X) ⊗[k] Y) :
    (TensorProduct.congr (LinearEquiv.refl k W) E) (TensorProduct.assoc k W X Y
        (TensorProduct.map (TensorProduct.map g LinearMap.id) LinearMap.id t)) =
      TensorProduct.map g LinearMap.id
        ((TensorProduct.congr (LinearEquiv.refl k W) E) (TensorProduct.assoc k W X Y t))
```

2.395 PhyslibAlpha.Particles.QED.Basic

PhyslibAlpha/Particles/QED/Basic.lean on GitHub.

def indexList [source]

The canonical sorted list of a multiset of spacetime directions, sorted through `Fin 1 ⊕ Fin 3 ≃ Fin 4`.

```
noncomputable def indexList (s : Multiset (Fin 1 ⊕ Fin 3)) : List (Fin 1 ⊕ Fin 3)
```

lemma indexList_zero [source]

```
@[simp]
lemma indexList_zero : indexList 0 = []
```

lemma indexList_singleton [source]

```
@[simp]
lemma indexList_singleton (μ : Fin 1 ⊕ Fin 3) : indexList {μ} = [μ]
```

lemma mem_indexList [source]

```
lemma mem_indexList {t : Multiset (Fin 1 ⊕ Fin 3)} {a : Fin 1 ⊕ Fin 3} :
    a ∈ indexList t ↔ a ∈ t
```

lemma indexList_length [source]

```
lemma indexList_length (t : Multiset (Fin 1 ⊕ Fin 3)) :
    (indexList t).length = Multiset.card t
```

def classRep [source]

The canonical representative of a nonempty multiset of spacetime directions: the head of its canonical sorted list.

```
noncomputable def classRep (t : Multiset (Fin 1 ⊕ Fin 3)) : Fin 1 ⊕ Fin 3
```

lemma classRep_mem [source]

```
lemma classRep_mem {t : Multiset (Fin 1 ⊕ Fin 3)} (ht : t ≠ 0) : classRep t ∈ t
```

def derivSum [source]

The Lorentz transport of a family indexed by derivative multisets along a list of derivative directions: each direction in the list is summed against a row of Λ^{-1}, and the chosen directions accumulate in the multiset argument of the continuation `k`.

```
noncomputable def derivSum {M : Type*} [AddCommMonoid M] [Module ℝ M]
    (Λ : LorentzGroup 3) :
    List (Fin 1 ⊕ Fin 3) → (Multiset (Fin 1 ⊕ Fin 3) → M) → M
  | [], k => k 0
  | σ :: l, k => ∑ τ, (Λ⁻¹).1 τ σ • derivSum Λ l fun t => k (t + {τ})
```

lemma derivSum_nil [source]

```
@[simp]
lemma derivSum_nil {M : Type*} [AddCommMonoid M] [Module ℝ M]
    (Λ : LorentzGroup 3) (k : Multiset (Fin 1 ⊕ Fin 3) → M) :
    derivSum Λ [] k = k 0
```

lemma derivSum_cons [source]

```
@[simp]
lemma derivSum_cons {M : Type*} [AddCommMonoid M] [Module ℝ M]
    (Λ : LorentzGroup 3) (σ : Fin 1 ⊕ Fin 3) (l : List (Fin 1 ⊕ Fin 3))
    (k : Multiset (Fin 1 ⊕ Fin 3) → M) :
    derivSum Λ (σ :: l) k =
      ∑ τ, (Λ⁻¹).1 τ σ • derivSum Λ l fun t => k (t + {τ})
```

inductive Photon.JetGenerators [source]

The jet coordinates of the electromagnetic potential: the symbol `∂_s A_μ`, the `s`-th derivative of the `μ`-th covariant component.

```
inductive JetGenerators where
  | dA (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) : JetGenerators
  deriving DecidableEq
```

def Photon.JetGenerators.massWeight [source]

The mass weight (twice the mass dimension) of a photon jet coordinate: the potential has mass dimension one and each derivative adds one.

```
def JetGenerators.massWeight : JetGenerators → ℕ
  | .dA s _ => 2 + 2 * s.card
```

def indexClass [source]

The symmetrized-index class of a photon jet coordinate: under a gauge transformation `∂_s A_μ` shifts by `∂_s ∂_μ χ`, which depends only on the multiset `s + {μ}`. Coordinates in a common class shift together.

```
def JetGenerators.indexClass : JetGenerators → Multiset (Fin 1 ⊕ Fin 3)
  | .dA s μ => s + {μ}
```

def classProj [source]

The canonical jet coordinate of a symmetrized-index class: the coordinate whose Lorentz index is the canonical representative of the class.

```
noncomputable def JetGenerators.classProj (j : JetGenerators) : JetGenerators
```

abbrev Photon.JetAlgebra [source]

The jet algebra of the photon: real polynomials in the jet coordinates.

```
abbrev JetAlgebra : Type
```

def coord [source]

The jet coordinate `∂_s A_μ` as an element of the jet algebra.

```
noncomputable def coord (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) : JetAlgebra
```

abbrev JetAlgebra.GaugeJet [source]

A photon gauge jet: the family `s ↦ ∂_s χ` of symmetrised derivatives of a gauge function at the base point. This is all the photon jet algebra sees of a gauge transformation.

```
abbrev GaugeJet : Type
```

def Photon.JetAlgebra.gaugeAction [source]

The gauge action on the photon jet algebra: the algebra map determined by `∂_s A_μ ↦ ∂_s A_μ + ∂_s ∂_μ χ`.

```
noncomputable def gaugeAction (c : GaugeJet) : JetAlgebra →ₐ[ℝ] JetAlgebra
```

lemma gaugeAction_coord [source]

```
@[simp]
lemma gaugeAction_coord (c : GaugeJet) (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    gaugeAction c (coord s μ) = coord s μ + MvPolynomial.C (c (s + {μ}))
```

lemma gaugeAction_C [source]

```
lemma gaugeAction_C (c : GaugeJet) (r : ℝ) :
    gaugeAction c (MvPolynomial.C r) = MvPolynomial.C r
```

def derivMultiset [source]

The iterated partial derivative `∂_s f` along a multiset `s` of spacetime directions, taken in the canonical order obtained by sorting `s`.

```
noncomputable def derivMultiset (s : Multiset (Fin 1 ⊕ Fin 3)) (f : SpaceTime 3 → ℝ) :
    SpaceTime 3 → ℝ
```

lemma derivMultiset_zero [source]

```
@[simp]
lemma derivMultiset_zero (f : SpaceTime 3 → ℝ) : derivMultiset 0 f = f
```

lemma derivMultiset_singleton [source]

```
@[simp]
lemma derivMultiset_singleton (μ : Fin 1 ⊕ Fin 3) (f : SpaceTime 3 → ℝ) :
    derivMultiset {μ} f = ∂_ μ f
```

def coPotential [source]

The covariant components `A_μ = η_{μμ} A^μ` of an electromagnetic potential.

```
noncomputable def coPotential (A : ElectromagneticPotential 3) (μ : Fin 1 ⊕ Fin 3) :
    SpaceTime 3 → ℝ
```

def evalPotential [source]

The evaluation of the photon jet algebra at an electromagnetic potential `A`: the algebra map sending the formal jet coordinate `∂_s A_μ` to the honest function `∂_s A_μ` on spacetime.

```
noncomputable def evalPotential (A : ElectromagneticPotential 3) :
    JetAlgebra →ₐ[ℝ] (SpaceTime 3 → ℝ)
```

lemma evalPotential_coord [source]

```
@[simp]
lemma evalPotential_coord (A : ElectromagneticPotential 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (μ : Fin 1 ⊕ Fin 3) :
    evalPotential A (coord s μ) = derivMultiset s (coPotential A μ)
```

def Photon.JetAlgebra.lorentzAction [source]

The Lorentz action on the photon jet algebra: the algebra map transporting every lower index of `∂_s A_μ` with `Λ⁻¹`.

```
noncomputable def lorentzAction (Λ : LorentzGroup 3) : JetAlgebra →ₐ[ℝ] JetAlgebra
```

lemma lorentzAction_coord [source]

```
@[simp]
lemma lorentzAction_coord (Λ : LorentzGroup 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (μ : Fin 1 ⊕ Fin 3) :
    lorentzAction Λ (coord s μ) =
      derivSum Λ (indexList s) fun t => ∑ ν, (Λ⁻¹).1 ν μ • coord t ν
```

lemma lorentzAction_coord_zero [source]

```
lemma lorentzAction_coord_zero (Λ : LorentzGroup 3) (μ : Fin 1 ⊕ Fin 3) :
    lorentzAction Λ (coord 0 μ) = ∑ ν, (Λ⁻¹).1 ν μ • coord 0 ν
```

lemma lorentzAction_coord_singleton [source]

```
lemma lorentzAction_coord_singleton (Λ : LorentzGroup 3) (σ μ : Fin 1 ⊕ Fin 3) :
    lorentzAction Λ (coord {σ} μ) =
      ∑ τ, ∑ ν, ((Λ⁻¹).1 τ σ * (Λ⁻¹).1 ν μ) • coord {τ} ν
```

def Photon.JetAlgebra.massScale [source]

The mass-weight scaling on the photon jet algebra: the algebra map multiplying each jet coordinate by `c` to the power of its mass weight.

```
noncomputable def massScale (c : ℝ) : JetAlgebra →ₐ[ℝ] JetAlgebra
```

lemma massScale_coord [source]

```
@[simp]
lemma massScale_coord (c : ℝ) (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    massScale c (coord s μ) = c ^ (2 + 2 * Multiset.card s) • coord s μ
```

def Photon.JetAlgebra.jetDeriv [source]

The formal total spacetime derivative on the photon jet algebra in the direction `ρ`: the derivation appending the derivative index, `∂_s A_μ ↦ ∂_{s + {ρ}} A_μ`.

```
noncomputable def jetDeriv (ρ : Fin 1 ⊕ Fin 3) : JetAlgebra →ₗ[ℝ] JetAlgebra
```

lemma jetDeriv_coord [source]

```
@[simp]
lemma jetDeriv_coord (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (μ : Fin 1 ⊕ Fin 3) :
    jetDeriv ρ (coord s μ) = coord (s + {ρ}) μ
```

lemma Photon.JetAlgebra.jetDeriv_one [source]

```
@[simp]
lemma jetDeriv_one (ρ : Fin 1 ⊕ Fin 3) : jetDeriv ρ (1 : JetAlgebra) = 0
```

lemma Photon.JetAlgebra.jetDeriv_mul [source]

The total derivative is a derivation on the photon jet algebra.

```
lemma jetDeriv_mul (ρ : Fin 1 ⊕ Fin 3) (x y : JetAlgebra) :
    jetDeriv ρ (x * y) = jetDeriv ρ x * y + x * jetDeriv ρ y
```

lemma jetDeriv_baseChange_mul [source]

The Leibniz rule for the complexified total derivative.

```
lemma jetDeriv_baseChange_mul (ρ : Fin 1 ⊕ Fin 3) (x y : ℂ ⊗[ℝ] JetAlgebra) :
    LinearMap.baseChange ℂ (jetDeriv ρ) (x * y) =
      LinearMap.baseChange ℂ (jetDeriv ρ) x * y +
        x * LinearMap.baseChange ℂ (jetDeriv ρ) y
```

lemma sum_map_antidiagonal_ite [source]

Summing an indicator supported on the splittings `(0, t)` over the antidiagonal of `t` picks out `f t`: the splitting `(0, t)` occurs exactly once in `Multiset.antidiagonal t`.

```
lemma sum_map_antidiagonal_ite {M : Type*} [AddCommMonoid M]
    (t : Multiset (Fin 1 ⊕ Fin 3)) (f : Multiset (Fin 1 ⊕ Fin 3) → M) :
    ((t.antidiagonal).map fun p => if p.1 = 0 then f p.2 else 0).sum = f t
```

def phaseAct [source]

The Leibniz convolution of a phase family against a module-valued family of jets, over the antidiagonal of the derivative multiset: the formal expansion `∂_s (u · f) = ∑_{x + y = s} (∂_x u) (∂_y f)`, with the multiplicities of `Multiset.antidiagonal` supplying the multinomial weights.

```
noncomputable def phaseAct {M : Type*} [AddCommMonoid M] [Module ℂ M]
    (u : Multiset (Fin 1 ⊕ Fin 3) → ℂ) (f : Multiset (Fin 1 ⊕ Fin 3) → M) :
    Multiset (Fin 1 ⊕ Fin 3) → M
```

lemma phaseAct_zero_arg [source]

```
@[simp]
lemma phaseAct_zero_arg : phaseAct u f 0 = u 0 • f 0
```

lemma phaseAct_eq_sum [source]

The convolution as a literal antidiagonal sum of products, for scalar-valued families.

```
lemma phaseAct_eq_sum (s : Multiset (Fin 1 ⊕ Fin 3)) :
    phaseAct u v s = (s.antidiagonal.map fun p => u p.1 * v p.2).sum
```

lemma phaseAct_add_singleton [source]

The Leibniz rule of the convolution: differentiating a convolution differentiates one factor at a time.

```
lemma phaseAct_add_singleton (a : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    phaseAct u f (s + {a}) =
      phaseAct u (fun t => f (t + {a})) s +
        phaseAct (fun t => u (t + {a})) f s
```

lemma phaseAct_add_left [source]

```
lemma phaseAct_add_left (s : Multiset (Fin 1 ⊕ Fin 3)) :
    phaseAct (fun t => u₁ t + u₂ t) f s = phaseAct u₁ f s + phaseAct u₂ f s
```

lemma phaseAct_add_right [source]

```
lemma phaseAct_add_right (s : Multiset (Fin 1 ⊕ Fin 3)) :
    phaseAct u (fun t => f t + g t) s = phaseAct u f s + phaseAct u g s
```

lemma phaseAct_smul_left [source]

```
lemma phaseAct_smul_left (c : ℂ) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    phaseAct (fun t => c * u t) f s = c • phaseAct u f s
```

lemma phaseAct_smul_right [source]

```
lemma phaseAct_smul_right (c : ℂ) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    phaseAct u (fun t => c • f t) s = c • phaseAct u f s
```

lemma phaseAct_assoc [source]

Associativity of the convolution: acting by `u` after `v` is acting by the convolution `u ⋆ v`.

```
lemma phaseAct_assoc (s : Multiset (Fin 1 ⊕ Fin 3)) :
    phaseAct u (phaseAct v f) s = phaseAct (phaseAct u v) f s
```

lemma phaseAct_comm [source]

Commutativity of the scalar convolution.

```
lemma phaseAct_comm (s : Multiset (Fin 1 ⊕ Fin 3)) :
    phaseAct u v s = phaseAct v u s
```

lemma map_phaseAct [source]

A linear map passes through the convolution.

```
lemma map_phaseAct {N : Type*} [AddCommMonoid N] [Module ℂ N] (L : M →ₗ[ℂ] N)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    L (phaseAct u f s) = phaseAct u (fun t => L (f t)) s
```

lemma phaseAct_indicator [source]

The convolution against the indicator of the empty multiset is the identity: the splitting `(0, t)` occurs exactly once in the antidiagonal.

```
lemma phaseAct_indicator (s : Multiset (Fin 1 ⊕ Fin 3)) :
    phaseAct (fun t => if t = 0 then 1 else 0) f s = f s
```

lemma star_phaseAct [source]

The star of a convolution is the convolution of the stars.

```
lemma star_phaseAct (s : Multiset (Fin 1 ⊕ Fin 3)) :
    star (phaseAct u v s) =
      phaseAct (fun t => star (u t)) (fun t => star (v t)) s
```

structure QED.GaugeJet [source]

The jets of a `U(1)_em` gauge transformation with coupling `e`: the derivative jets `χjet s = ∂_s χ` of the real gauge function and `phase s = ∂_s (exp (I e χ))` of its unitary phase at the base point, subject to the two identities every honest gauge function satisfies: the phase has unit norm at the base point, and its derivatives obey the formal Leibniz expansion of `∂_μ (exp (I e χ)) = I e (∂_μ χ) exp (I e χ)`.

```
structure GaugeJet (e : ℝ) where
  χjet : Multiset (Fin 1 ⊕ Fin 3) → ℝ
  phase : Multiset (Fin 1 ⊕ Fin 3) → ℂ
  phase_zero_unitary : phase 0 * star (phase 0) = 1
  phase_deriv : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3),
    phase (s + {μ}) = Complex.I * e *
      ((s.antidiagonal.map fun p => (χjet (p.1 + {μ}) : ℂ) * phase p.2).sum)
```

lemma star_phase_zero_unitary [source]

```
lemma star_phase_zero_unitary : star (g.phase 0) * g.phase 0 = 1
```

lemma phase_singleton [source]

The first derivative of the phase: the `s = 0` case of the Leibniz identity, `∂_μ u = I e (∂_μ χ) u` at the base point.

```
lemma phase_singleton (μ : Fin 1 ⊕ Fin 3) :
    g.phase {μ} = Complex.I * e * (g.χjet {μ} * g.phase 0)
```

lemma star_phase_singleton [source]

The first derivative of the conjugate phase, `∂_μ ū = -I e (∂_μ χ) ū` at the base point.

```
lemma star_phase_singleton (μ : Fin 1 ⊕ Fin 3) :
    star (g.phase {μ}) = -(Complex.I * e * (g.χjet {μ} * star (g.phase 0)))
```

def trivial [source]

The trivial gauge jet: the jets of the constant gauge function `χ = 0`.

```
noncomputable def trivial (e : ℝ) : GaugeJet e where
```

lemma ext [source]

```
lemma ext {g₁ g₂ : GaugeJet e} (h1 : g₁.χjet = g₂.χjet)
    (h2 : g₁.phase = g₂.phase) : g₁ = g₂
```

instance Mul (GaugeJet e) [source]

The composite of two gauge jets: the gauge functions add and the phases convolve by the Leibniz rule.

```
noncomputable instance : Mul (GaugeJet e) where
```

lemma mul_χjet [source]

```
@[simp]
lemma mul_χjet (g₁ g₂ : GaugeJet e) : (g₁ * g₂).χjet = g₁.χjet + g₂.χjet
```

lemma mul_phase [source]

```
@[simp]
lemma mul_phase (g₁ g₂ : GaugeJet e) :
    (g₁ * g₂).phase = phaseAct g₁.phase g₂.phase
```

instance One (GaugeJet e) [source]

```
noncomputable instance : One (GaugeJet e)
```

lemma one_χjet [source]

```
@[simp]
lemma one_χjet : (1 : GaugeJet e).χjet = 0
```

lemma one_phase [source]

```
@[simp]
lemma one_phase :
    (1 : GaugeJet e).phase = fun s => if s = 0 then (1 : ℂ) else 0
```

instance CommMonoid (GaugeJet e) [source]

***The gauge jets form a commutative monoid**: the gauge symmetry data of QED composes associatively, with the trivial gauge jet as the unit.*

```
noncomputable instance : CommMonoid (GaugeJet e) where
```

inductive Electron.JetGenerators [source]

The jet coordinates of the Dirac electron: the symbols `∂_s ψ_α` and `∂_s ψ̄_α`, the `s`-th derivatives of the Dirac components and their conjugates. The electron has electric charge `-1`; its conjugate has charge `+1`.

```
inductive JetGenerators where
  | dψ (s : Multiset (Fin 1 ⊕ Fin 3)) (α : Fin 2 ⊕ Fin 2) : JetGenerators
  | dbarψ (s : Multiset (Fin 1 ⊕ Fin 3)) (α : Fin 2 ⊕ Fin 2) : JetGenerators
  deriving DecidableEq
```

def Electron.JetGenerators.massWeight [source]

The mass weight (twice the mass dimension) of an electron jet coordinate: a fermion has mass dimension `3/2` and each derivative adds one.

```
def JetGenerators.massWeight : JetGenerators → ℕ
  | .dψ s _ => 3 + 2 * s.card
  | .dbarψ s _ => 3 + 2 * s.card
```

abbrev JetComponentSpace [source]

The jet component space of the electron: the free complex module on the jet coordinates.

```
abbrev JetComponentSpace : Type
```

abbrev Electron.JetAlgebra [source]

The jet algebra of the electron: the exterior algebra on the free module over the jet coordinates. The exterior product implements the fermionic anticommutativity of the electron field.

```
abbrev JetAlgebra : Type
```

def ofGenerator [source]

The jet coordinate `∂_s ψ_α` or `∂_s ψ¯_α` as an element of the jet algebra.

```
noncomputable def ofGenerator (j : JetGenerators) : JetAlgebra
```

def gaugeActionGenerator [source]

The gauge action on a single electron jet coordinate: the Leibniz expansion of `∂_s (ū ψ_α)` and `∂_s (u ψ¯_α)` over the splittings of `s`.

```
noncomputable def gaugeActionGenerator {e : ℝ} (g : GaugeJet e) :
    JetGenerators → JetComponentSpace
  | .dψ t α => (t.antidiagonal.map fun p =>
      Finsupp.single (JetGenerators.dψ p.2 α) (star (g.phase p.1))).sum
  | .dbarψ t α => (t.antidiagonal.map fun p =>
      Finsupp.single (JetGenerators.dbarψ p.2 α) (g.phase p.1)).sum
```

def gaugeActionCS [source]

The gauge action on the jet component space: the linear extension of the Leibniz expansion on the jet coordinates.

```
noncomputable def gaugeActionCS {e : ℝ} (g : GaugeJet e) :
    JetComponentSpace →ₗ[ℂ] JetComponentSpace
```

lemma gaugeActionCS_single [source]

```
@[simp]
lemma gaugeActionCS_single {e : ℝ} (g : GaugeJet e) (j : JetGenerators) :
    gaugeActionCS g (Finsupp.single j 1) = gaugeActionGenerator g j
```

def Electron.JetAlgebra.gaugeAction [source]

The gauge action on the electron jet algebra: the algebra map induced by the Leibniz expansion on the jet coordinates.

```
noncomputable def gaugeAction {e : ℝ} (g : GaugeJet e) :
    JetAlgebra →ₐ[ℂ] JetAlgebra
```

lemma gaugeAction_ofGenerator_dψ [source]

```
lemma gaugeAction_ofGenerator_dψ {e : ℝ} (g : GaugeJet e)
    (t : Multiset (Fin 1 ⊕ Fin 3)) (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (ofGenerator (.dψ t α)) =
      (t.antidiagonal.map fun p =>
        star (g.phase p.1) • ofGenerator (.dψ p.2 α)).sum
```

lemma gaugeAction_ofGenerator_dbarψ [source]

```
lemma gaugeAction_ofGenerator_dbarψ {e : ℝ} (g : GaugeJet e)
    (t : Multiset (Fin 1 ⊕ Fin 3)) (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (ofGenerator (.dbarψ t α)) =
      (t.antidiagonal.map fun p =>
        g.phase p.1 • ofGenerator (.dbarψ p.2 α)).sum
```

lemma antidiagonal_singleton [source]

```
lemma antidiagonal_singleton (μ : Fin 1 ⊕ Fin 3) :
    ({μ} : Multiset (Fin 1 ⊕ Fin 3)).antidiagonal = {(0, {μ}), ({μ}, 0)}
```

lemma gaugeAction_ofGenerator_dψ_zero [source]

```
@[simp]
lemma gaugeAction_ofGenerator_dψ_zero {e : ℝ} (g : GaugeJet e) (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (ofGenerator (.dψ 0 α)) =
      star (g.phase 0) • ofGenerator (.dψ 0 α)
```

lemma gaugeAction_ofGenerator_dbarψ_zero [source]

```
@[simp]
lemma gaugeAction_ofGenerator_dbarψ_zero {e : ℝ} (g : GaugeJet e) (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (ofGenerator (.dbarψ 0 α)) =
      g.phase 0 • ofGenerator (.dbarψ 0 α)
```

lemma gaugeAction_ofGenerator_dψ_singleton [source]

```
lemma gaugeAction_ofGenerator_dψ_singleton {e : ℝ} (g : GaugeJet e)
    (μ : Fin 1 ⊕ Fin 3) (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (ofGenerator (.dψ {μ} α)) =
      star (g.phase 0) • ofGenerator (.dψ {μ} α) +
        star (g.phase {μ}) • ofGenerator (.dψ 0 α)
```

lemma gaugeAction_ofGenerator_dbarψ_singleton [source]

```
lemma gaugeAction_ofGenerator_dbarψ_singleton {e : ℝ} (g : GaugeJet e)
    (μ : Fin 1 ⊕ Fin 3) (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (ofGenerator (.dbarψ {μ} α)) =
      g.phase 0 • ofGenerator (.dbarψ {μ} α) +
        g.phase {μ} • ofGenerator (.dbarψ 0 α)
```

def spinorRep [source]

The Dirac spinor representation of `SL(2,ℂ)` in the chiral basis: the two Weyl components transform in the two conjugate-dual fundamental representations, `S(M) = ((M, 0), (0, (M†)⁻¹))`, the assignment being fixed by the conventions of `Lorentz.SL2C.toLorentzGroup`.

```
noncomputable def spinorRep (M : SL(2,ℂ)) :
    Matrix (Fin 2 ⊕ Fin 2) (Fin 2 ⊕ Fin 2) ℂ
```

def lorentzActionGenerator [source]

The Lorentz action on a single electron jet coordinate: the spinor index is rotated by the spinor representation (its conjugate for `∂_s ψ̄`) and the derivative indices are transported with `Λ(M)⁻¹`.

```
noncomputable def lorentzActionGenerator (M : SL(2,ℂ)) :
    JetGenerators → JetComponentSpace
  | .dψ t α => derivSum (Lorentz.SL2C.toLorentzGroup M) (indexList t) fun t' =>
      ∑ β, spinorRep M α β • Finsupp.single (JetGenerators.dψ t' β) 1
```

```
  | .dbarψ t α => derivSum (Lorentz.SL2C.toLorentzGroup M) (indexList t) fun t' =>
      ∑ β, star (spinorRep M α β) • Finsupp.single (JetGenerators.dbarψ t' β) 1
```

def lorentzActionCS [source]

The Lorentz action on the jet component space.

```
noncomputable def lorentzActionCS (M : SL(2,ℂ)) :
    JetComponentSpace →ₗ[ℂ] JetComponentSpace
```

lemma lorentzActionCS_single [source]

```
@[simp]
lemma lorentzActionCS_single (M : SL(2,ℂ)) (j : JetGenerators) :
    lorentzActionCS M (Finsupp.single j 1) = lorentzActionGenerator M j
```

def Electron.JetAlgebra.lorentzAction [source]

The Lorentz action on the electron jet algebra: the algebra map induced by the action on the jet coordinates.

```
noncomputable def lorentzAction (M : SL(2,ℂ)) : JetAlgebra →ₐ[ℂ] JetAlgebra
```

lemma lorentzAction_ofGenerator_dψ_zero [source]

```
@[simp]
lemma lorentzAction_ofGenerator_dψ_zero (M : SL(2,ℂ)) (α : Fin 2 ⊕ Fin 2) :
    lorentzAction M (ofGenerator (.dψ 0 α)) =
      ∑ β, spinorRep M α β • ofGenerator (.dψ 0 β)
```

lemma lorentzAction_ofGenerator_dbarψ_zero [source]

```
@[simp]
lemma lorentzAction_ofGenerator_dbarψ_zero (M : SL(2,ℂ)) (α : Fin 2 ⊕ Fin 2) :
    lorentzAction M (ofGenerator (.dbarψ 0 α)) =
      ∑ β, star (spinorRep M α β) • ofGenerator (.dbarψ 0 β)
```

lemma lorentzAction_ofGenerator_dψ_singleton [source]

```
lemma lorentzAction_ofGenerator_dψ_singleton (M : SL(2,ℂ)) (σ : Fin 1 ⊕ Fin 3)
    (α : Fin 2 ⊕ Fin 2) :
    lorentzAction M (ofGenerator (.dψ {σ} α)) =
      ∑ τ, ∑ β, ((((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 τ σ : ℝ) • spinorRep M α β) •
        ofGenerator (.dψ {τ} β)
```

lemma lorentzAction_ofGenerator_dbarψ_singleton [source]

```
lemma lorentzAction_ofGenerator_dbarψ_singleton (M : SL(2,ℂ)) (σ : Fin 1 ⊕ Fin 3)
    (α : Fin 2 ⊕ Fin 2) :
    lorentzAction M (ofGenerator (.dbarψ {σ} α)) =
      ∑ τ, ∑ β, ((((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 τ σ : ℝ) •
          star (spinorRep M α β)) • ofGenerator (.dbarψ {τ} β)
```

def massScaleCS [source]

The mass-weight scaling on the jet component space: the diagonal map multiplying each jet coordinate by `c` to the power of its mass weight.

```
noncomputable def massScaleCS (c : ℝ) : JetComponentSpace →ₗ[ℂ] JetComponentSpace
```

def Electron.JetAlgebra.massScale [source]

The mass-weight scaling on the electron jet algebra.

```
noncomputable def massScale (c : ℝ) : JetAlgebra →ₐ[ℂ] JetAlgebra
```

lemma massScale_ofGenerator [source]

```
@[simp]
lemma massScale_ofGenerator (c : ℝ) (j : JetGenerators) :
    massScale c (ofGenerator j) = (c : ℂ) ^ j.massWeight • ofGenerator j
```

def shift [source]

The jet coordinate with one further derivative in the direction `ρ`.

```
def JetGenerators.shift (ρ : Fin 1 ⊕ Fin 3) : JetGenerators → JetGenerators
  | .dψ s α => .dψ (s + {ρ}) α
  | .dbarψ s α => .dbarψ (s + {ρ}) α
```

def jetDerivCS [source]

The total derivative on the jet component space: the shift of the derivative multi-index.

```
noncomputable def jetDerivCS (ρ : Fin 1 ⊕ Fin 3) :
    JetComponentSpace →ₗ[ℂ] JetComponentSpace
```

lemma jetDerivCS_single [source]

```
@[simp]
lemma jetDerivCS_single (ρ : Fin 1 ⊕ Fin 3) (j : JetGenerators) :
    jetDerivCS ρ (Finsupp.single j 1) =
      Finsupp.single (JetGenerators.shift ρ j) 1
```

def jetDerivGen [source]

The generator map of the total derivative into the trivial square-zero extension of the jet algebra: `ι x ↦ (ι x, ι (∂_ρ x))`.

```
noncomputable def jetDerivGen (ρ : Fin 1 ⊕ Fin 3) :
    JetComponentSpace →ₗ[ℂ] TrivSqZeroExt JetAlgebra JetAlgebra where
```

lemma jetDerivGen_fst [source]

```
@[simp]
lemma jetDerivGen_fst (ρ : Fin 1 ⊕ Fin 3) (x : JetComponentSpace) :
```

```
    (jetDerivGen ρ x).fst = ExteriorAlgebra.ι ℂ x
```

lemma jetDerivGen_snd [source]

```
@[simp]
lemma jetDerivGen_snd (ρ : Fin 1 ⊕ Fin 3) (x : JetComponentSpace) :
    (jetDerivGen ρ x).snd = ExteriorAlgebra.ι ℂ (jetDerivCS ρ x)
```

lemma jetDerivGen_mul_self [source]

The generator map squares to zero: degree-one elements of the exterior algebra anticommute.

```
lemma jetDerivGen_mul_self (ρ : Fin 1 ⊕ Fin 3) (x : JetComponentSpace) :
    jetDerivGen ρ x * jetDerivGen ρ x = 0
```

def jetDerivHom [source]

The lift of the total derivative to the trivial square-zero extension of the jet algebra: the algebra homomorphism `x ↦ (x, ∂_ρ x)`.

```
noncomputable def jetDerivHom (ρ : Fin 1 ⊕ Fin 3) :
    JetAlgebra →ₐ[ℂ] TrivSqZeroExt JetAlgebra JetAlgebra
```

lemma jetDerivHom_ι [source]

```
@[simp]
lemma jetDerivHom_ι (ρ : Fin 1 ⊕ Fin 3) (x : JetComponentSpace) :
    jetDerivHom ρ (ExteriorAlgebra.ι ℂ x) = jetDerivGen ρ x
```

lemma jetDerivHom_fst [source]

The first component of the square-zero lift is the identity.

```
@[simp]
lemma jetDerivHom_fst (ρ : Fin 1 ⊕ Fin 3) (x : JetAlgebra) :
    (jetDerivHom ρ x).fst = x
```

def Electron.JetAlgebra.jetDeriv [source]

The formal total spacetime derivative on the electron jet algebra in the direction `ρ`: the even derivation extending the shift `∂_s ψ_α ↦ ∂_{s + {ρ}} ψ_α` of the jet coordinates.

```
noncomputable def jetDeriv (ρ : Fin 1 ⊕ Fin 3) : JetAlgebra →ₗ[ℂ] JetAlgebra where
```

lemma jetDeriv_apply [source]

```
lemma jetDeriv_apply (ρ : Fin 1 ⊕ Fin 3) (x : JetAlgebra) :
    jetDeriv ρ x = (jetDerivHom ρ x).snd
```

lemma jetDeriv_ι [source]

```
@[simp]
lemma jetDeriv_ι (ρ : Fin 1 ⊕ Fin 3) (x : JetComponentSpace) :
    jetDeriv ρ (ExteriorAlgebra.ι ℂ x) =
      ExteriorAlgebra.ι ℂ (jetDerivCS ρ x)
```

lemma jetDeriv_ofGenerator [source]

The total derivative appends the derivative index to each jet coordinate.

```
@[simp]
lemma jetDeriv_ofGenerator (ρ : Fin 1 ⊕ Fin 3) (j : JetGenerators) :
    jetDeriv ρ (ofGenerator j) = ofGenerator (JetGenerators.shift ρ j)
```

lemma Electron.JetAlgebra.jetDeriv_one [source]

```
@[simp]
lemma jetDeriv_one (ρ : Fin 1 ⊕ Fin 3) : jetDeriv ρ (1 : JetAlgebra) = 0
```

lemma Electron.JetAlgebra.jetDeriv_mul [source]

The total derivative is an even derivation: the Leibniz rule holds on the electron jet algebra with no Koszul signs.

```
lemma jetDeriv_mul (ρ : Fin 1 ⊕ Fin 3) (x y : JetAlgebra) :
    jetDeriv ρ (x * y) = jetDeriv ρ x * y + x * jetDeriv ρ y
```

def QED.JetAlgebra [source]

The jet algebra of quantum electrodynamics: the tensor product of the complexified photon jet algebra with the electron jet algebra.

This is a `def` rather than an `abbrev`, and its algebraic structure is fixed by the single `Ring` and `Algebra` instances below, so that every algebraic class projects from one root. On the bare tensor product `One`, `Mul`, `Zero`, `Add`, `SMul` and `Module` are instead supplied by standalone `TensorProduct.*` instances rather than as projections of the semiring; those are definitionally the projections, but not syntactically, so a generic lemma whose type argument is not pinned by an explicit argument (such as `one_pow`) cannot be unified against a goal. Rooting the structure here keeps the generic algebraic lemmas usable.

```
def JetAlgebra : Type
```

instance Ring JetAlgebra [source]

```
noncomputable instance : Ring JetAlgebra
```

instance Algebra ℂ JetAlgebra [source]

```
noncomputable instance : Algebra ℂ JetAlgebra
```

def tmul [source]

A pure tensor, as an element of the jet algebra.

*Writing `a ⊗ₜ[ℂ] b` builds an element of the *underlying* tensor product, which is only definitionally an element of `JetAlgebra`. A goal mixing such a term with the jet algebra's own operations is then not type-correct at `instances` transparency, and no rewrite can fire on it. This constructor keeps pure tensors typed at `JetAlgebra`.*

```
noncomputable def tmul (a : ℂ ⊗[ℝ] Photon.JetAlgebra) (b : Electron.JetAlgebra) :
    JetAlgebra
```

lemma tmul_eq [source]

`tmul` is the pure tensor of the underlying tensor product; use this to move between the jet algebra and lemmas stated for the tensor product.

```
lemma tmul_eq (a : ℂ ⊗[ℝ] Photon.JetAlgebra) (b : Electron.JetAlgebra) :
    a ⊗ⱼ b = a ⊗ₜ[ℂ] b
```

lemma one_eq_tmul [source]

```
lemma one_eq_tmul : (1 : JetAlgebra) = (1 : ℂ ⊗[ℝ] Photon.JetAlgebra) ⊗ⱼ 1
```

lemma tmul_mul_tmul [source]

Multiplication of pure tensors. `Algebra.TensorProduct.tmul_mul_tmul` does not rewrite here, even though it is definitionally the same statement.

```
@[simp]
lemma tmul_mul_tmul (a₁ a₂ : ℂ ⊗[ℝ] Photon.JetAlgebra)
    (b₁ b₂ : Electron.JetAlgebra) :
    (a₁ ⊗ⱼ b₁) * (a₂ ⊗ⱼ b₂) = (a₁ * a₂) ⊗ⱼ (b₁ * b₂)
```

lemma zero_tmul [source]

```
@[simp]
lemma zero_tmul (b : Electron.JetAlgebra) :
    (0 : ℂ ⊗[ℝ] Photon.JetAlgebra) ⊗ⱼ b = 0
```

lemma tmul_zero [source]

```
@[simp]
lemma tmul_zero (a : ℂ ⊗[ℝ] Photon.JetAlgebra) :
    a ⊗ⱼ (0 : Electron.JetAlgebra) = 0
```

lemma add_tmul [source]

```
@[simp]
lemma add_tmul (a₁ a₂ : ℂ ⊗[ℝ] Photon.JetAlgebra) (b : Electron.JetAlgebra) :
    (a₁ + a₂) ⊗ⱼ b = a₁ ⊗ⱼ b + a₂ ⊗ⱼ b
```

lemma tmul_add [source]

```
@[simp]
lemma tmul_add (a : ℂ ⊗[ℝ] Photon.JetAlgebra) (b₁ b₂ : Electron.JetAlgebra) :
    a ⊗ⱼ (b₁ + b₂) = a ⊗ⱼ b₁ + a ⊗ⱼ b₂
```

lemma sub_tmul [source]

```
@[simp]
lemma sub_tmul (a₁ a₂ : ℂ ⊗[ℝ] Photon.JetAlgebra) (b : Electron.JetAlgebra) :
    (a₁ - a₂) ⊗ⱼ b = a₁ ⊗ⱼ b - a₂ ⊗ⱼ b
```

lemma tmul_sub [source]

```
@[simp]
lemma tmul_sub (a : ℂ ⊗[ℝ] Photon.JetAlgebra) (b₁ b₂ : Electron.JetAlgebra) :
    a ⊗ⱼ (b₁ - b₂) = a ⊗ⱼ b₁ - a ⊗ⱼ b₂
```

lemma neg_tmul [source]

```
@[simp]
lemma neg_tmul (a : ℂ ⊗[ℝ] Photon.JetAlgebra) (b : Electron.JetAlgebra) :
    (-a) ⊗ⱼ b = -(a ⊗ⱼ b)
```

lemma tmul_neg [source]

```
@[simp]
lemma tmul_neg (a : ℂ ⊗[ℝ] Photon.JetAlgebra) (b : Electron.JetAlgebra) :
    a ⊗ⱼ (-b) = -(a ⊗ⱼ b)
```

lemma tmul_sum [source]

```
lemma tmul_sum {ι : Type*} (a : ℂ ⊗[ℝ] Photon.JetAlgebra) (s : Finset ι)
    (f : ι → Electron.JetAlgebra) : a ⊗ⱼ (∑ i ∈ s, f i) = ∑ i ∈ s, a ⊗ⱼ f i
```

lemma sum_tmul [source]

```
lemma sum_tmul {ι : Type*} (s : Finset ι) (f : ι → ℂ ⊗[ℝ] Photon.JetAlgebra)
    (b : Electron.JetAlgebra) : (∑ i ∈ s, f i) ⊗ⱼ b = ∑ i ∈ s, f i ⊗ⱼ b
```

lemma tmul_smul [source]

```
@[simp]
lemma tmul_smul (r : ℂ) (a : ℂ ⊗[ℝ] Photon.JetAlgebra) (b : Electron.JetAlgebra) :
    a ⊗ⱼ (r • b) = r • (a ⊗ⱼ b)
```

lemma smul_tmul' [source]

```
@[simp]
lemma smul_tmul' (r : ℂ) (a : ℂ ⊗[ℝ] Photon.JetAlgebra) (b : Electron.JetAlgebra) :
    (r • a) ⊗ⱼ b = r • (a ⊗ⱼ b)
```

lemma real_smul_tmul [source]

An $\mathbb{R}$-scalar on the photon factor is a $\mathbb{C}$-scalar of the jet algebra.

```
lemma real_smul_tmul (r : ℝ) (a : ℂ ⊗[ℝ] Photon.JetAlgebra)
    (b : Electron.JetAlgebra) :
    (r • a) ⊗ⱼ b = (r : ℂ) • (a ⊗ⱼ b)
```

lemma tmul_C_eq_smul_one [source]

A constant of the photon factor is a scalar of the jet algebra.

```
lemma tmul_C_eq_smul_one (r : ℝ) :
    ((1 : ℂ) ⊗ₜ[ℝ] (MvPolynomial.C r : Photon.JetAlgebra)) ⊗ⱼ
        (1 : Electron.JetAlgebra) = (r : ℂ) • (1 : JetAlgebra)
```

lemma induction_on [source]

Induction on the jet algebra, stated for `JetAlgebra` itself. Using `TensorProduct.induction_on` directly leaves the sum and the pure tensors in the goals carrying the tensor product's structure rather than the jet algebra's, which makes those goals unrewritable.

```
@[elab_as_elim]
lemma induction_on {motive : JetAlgebra → Prop} (x : JetAlgebra)
    (tmul : ∀ (a : ℂ ⊗[ℝ] Photon.JetAlgebra) (b : Electron.JetAlgebra),
      motive (a ⊗ⱼ b))
    (add : ∀ x y : JetAlgebra, motive x → motive y → motive (x + y)) : motive x
```

abbrev inclA [source]

The photon factor included into the QED jet algebra.

```
noncomputable abbrev inclA : (ℂ ⊗[ℝ] Photon.JetAlgebra) →ₐ[ℂ] JetAlgebra
```

abbrev inclE [source]

The electron factor included into the QED jet algebra.

```
noncomputable abbrev inclE : Electron.JetAlgebra →ₐ[ℂ] JetAlgebra
```

lemma inclA_apply [source]

```
lemma inclA_apply (a : ℂ ⊗[ℝ] Photon.JetAlgebra) : inclA a = a ⊗ⱼ 1
```

lemma inclE_apply [source]

```
lemma inclE_apply (b : Electron.JetAlgebra) : inclE b = 1 ⊗ⱼ b
```

def gaugeActionPhoton [source]

The gauge action on the complexified photon jet algebra: the complexification of the affine action $\partial_s A_\mu \mapsto \partial_s A_\mu + \partial_s \partial_\mu \chi$.

```
noncomputable def gaugeActionPhoton (c : Photon.JetAlgebra.GaugeJet) :
    (ℂ ⊗[ℝ] Photon.JetAlgebra) →ₐ[ℂ] ℂ ⊗[ℝ] Photon.JetAlgebra
```

lemma gaugeActionPhoton_tmul [source]

```
@[simp]
lemma gaugeActionPhoton_tmul (c : Photon.JetAlgebra.GaugeJet) (x : ℂ)
    (p : Photon.JetAlgebra) :
    gaugeActionPhoton c (x ⊗ₜ[ℝ] p) = x ⊗ₜ[ℝ] Photon.JetAlgebra.gaugeAction c p
```

def QED.JetAlgebra.gaugeAction [source]

The gauge action on the QED jet algebra: the tensor product of the affine action on the photon factor with the Leibniz phase rotation on the electron factor.

```
noncomputable def gaugeAction {e : ℝ} (g : GaugeJet e) :
    JetAlgebra →ₐ[ℂ] JetAlgebra
```

lemma gaugeAction_tmul [source]

```
lemma gaugeAction_tmul {e : ℝ} (g : GaugeJet e) (a : ℂ ⊗[ℝ] Photon.JetAlgebra)
    (b : Electron.JetAlgebra) :
    gaugeAction g (a ⊗ⱼ b) =
      gaugeActionPhoton g.χjet a ⊗ⱼ Electron.JetAlgebra.gaugeAction g b
```

def lorentzActionPhoton [source]

The Lorentz action on the complexified photon jet algebra.

```
noncomputable def lorentzActionPhoton (Λ : LorentzGroup 3) :
    (ℂ ⊗[ℝ] Photon.JetAlgebra) →ₐ[ℂ] ℂ ⊗[ℝ] Photon.JetAlgebra
```

lemma lorentzActionPhoton_tmul [source]

```
@[simp]
lemma lorentzActionPhoton_tmul (Λ : LorentzGroup 3) (x : ℂ) (p : Photon.JetAlgebra) :
    lorentzActionPhoton Λ (x ⊗ₜ[ℝ] p) = x ⊗ₜ[ℝ] Photon.JetAlgebra.lorentzAction Λ p
```

def QED.JetAlgebra.lorentzAction [source]

The Lorentz action on the QED jet algebra: the tensor product of the photon action through the covering map with the electron spinor action.

```
noncomputable def lorentzAction (M : SL(2,ℂ)) : JetAlgebra →ₐ[ℂ] JetAlgebra
```

lemma lorentzAction_tmul [source]

```
lemma lorentzAction_tmul (M : SL(2,ℂ)) (a : ℂ ⊗[ℝ] Photon.JetAlgebra)
    (b : Electron.JetAlgebra) :
    lorentzAction M (a ⊗ⱼ b) =
      lorentzActionPhoton (Lorentz.SL2C.toLorentzGroup M) a ⊗ⱼ
        Electron.JetAlgebra.lorentzAction M b
```

def massScalePhoton [source]

The mass-weight scaling on the complexified photon jet algebra.

```
noncomputable def massScalePhoton (c : ℝ) :
    (ℂ ⊗[ℝ] Photon.JetAlgebra) →ₐ[ℂ] ℂ ⊗[ℝ] Photon.JetAlgebra
```

lemma massScalePhoton_tmul [source]

```
@[simp]
lemma massScalePhoton_tmul (c : ℝ) (x : ℂ) (p : Photon.JetAlgebra) :
    massScalePhoton c (x ⊗ₜ[ℝ] p) = x ⊗ₜ[ℝ] Photon.JetAlgebra.massScale c p
```

def QED.JetAlgebra.massScale [source]

The mass-weight scaling on the QED jet algebra: the algebra map multiplying each jet coordinate by `c` to the power of its mass weight, i.e. `c` squared to the power of its mass dimension.

```
noncomputable def massScale (c : ℝ) : JetAlgebra →ₐ[ℂ] JetAlgebra
```

lemma massScale_tmul [source]

```
lemma massScale_tmul (c : ℝ) (a : ℂ ⊗[ℝ] Photon.JetAlgebra)
    (b : Electron.JetAlgebra) :
    massScale c (a ⊗ⱼ b) =
      massScalePhoton c a ⊗ⱼ Electron.JetAlgebra.massScale c b
```

def QED.JetAlgebra.jetDeriv [source]

The formal total spacetime derivative on the QED jet algebra in the direction `ρ`: the Leibniz extension of the total derivatives of the photon and electron factors.

```
noncomputable def jetDeriv (ρ : Fin 1 ⊕ Fin 3) : JetAlgebra →ₗ[ℂ] JetAlgebra
```

lemma jetDeriv_tmul [source]

```
lemma jetDeriv_tmul (ρ : Fin 1 ⊕ Fin 3) (a : ℂ ⊗[ℝ] Photon.JetAlgebra)
    (b : Electron.JetAlgebra) :
    jetDeriv ρ (a ⊗ⱼ b) =
      (LinearMap.baseChange ℂ (Photon.JetAlgebra.jetDeriv ρ) a) ⊗ⱼ b +
        a ⊗ⱼ Electron.JetAlgebra.jetDeriv ρ b
```

lemma QED.JetAlgebra.jetDeriv_one [source]

```
@[simp]
lemma jetDeriv_one (ρ : Fin 1 ⊕ Fin 3) : jetDeriv ρ (1 : JetAlgebra) = 0
```

lemma QED.JetAlgebra.jetDeriv_mul [source]

The total derivative is an even derivation on the QED jet algebra: the Leibniz rule holds with no Koszul signs.

```
lemma jetDeriv_mul (ρ : Fin 1 ⊕ Fin 3) (x y : JetAlgebra) :
    jetDeriv ρ (x * y) = jetDeriv ρ x * y + x * jetDeriv ρ y
```

2.396 PhyslibAlpha.Particles.QED.CurrentCoupling

PhyslibAlpha/Particles/QED/CurrentCoupling.lean on GitHub.

lemma A_mul_diracCurrent [source]

The photon potential times the Dirac current, written through the fermion bilinears.

```
lemma A_mul_diracCurrent (μ : Fin 1 ⊕ Fin 3) :
    A 0 μ * diracCurrent μ =
      ∑ α, ∑ β, kineticGamma μ α β • (A 0 μ * (barψ 0 α * ψ 0 β))
```

lemma diracKineticTerm_eq_free_add_current [source]

***Minimal coupling**: the Dirac kinetic term with coupling `e` is the free Dirac kinetic term plus the current coupling `- e J^μ A_μ`. All of the interaction of QED is the photon contracted with the Dirac current, the jet-algebra counterpart of the current coupling of `Physlib.Electromagnetism.Dynamics`.*

```
lemma diracKineticTerm_eq_free_add_current (e : ℝ) :
    diracKineticTerm e = diracKineticTerm 0 +
      (-e : ℂ) • ∑ μ, A 0 μ * diracCurrent μ
```

lemma gaugeAction_diracCurrent [source]

***The Dirac current is gauge invariant**: the electron and its conjugate carry opposite charges, so the phases cancel by unitarity. This is what makes `J^μ` a physically meaningful source for the photon.*

```
@[simp]
lemma gaugeAction_diracCurrent {e : ℝ} (g : GaugeJet e) (μ : Fin 1 ⊕ Fin 3) :
    gaugeAction g (diracCurrent μ) = diracCurrent μ
```

lemma current_conservation [source]

***Noether's identity for the Dirac current**: the divergence of the current is a combination of the Dirac-equation elements, `i ∂_μ J^μ = ψ̄ · (Dirac eq) + (adjoint Dirac eq) · ψ`. On solutions of the Dirac equations the current is conserved, `∂_μ J^μ = 0` — for every coupling `e` and mass `m`: the gauge interaction and the mass drop out of the divergence identically.*

```
lemma current_conservation (e m : ℝ) :
    Complex.I • ∑ μ, jetDeriv μ (diracCurrent μ) =
      ∑ α, barψ 0 α * diracEquation e m α +
        ∑ β, diracAdjEquation e m β * ψ 0 β
```

2.397 PhyslibAlpha.Particles.QED.Evaluation

PhyslibAlpha/Particles/QED/Evaluation.lean on GitHub.

lemma deriv_coPotential [source]

The derivative of a covariant component. Differentiability is needed to move the constant `η_{νν}` through the derivative.

```
lemma deriv_coPotential (A : ElectromagneticPotential 3) (hA : Differentiable ℝ A)
    (μ ν : Fin 1 ⊕ Fin 3) (x : SpaceTime 3) :
    ∂_ μ (coPotential A ν) x = η ν ν * ∂_ μ A x ν
```

lemma evalPotential_fieldStrength_zero_apply [source]

```
lemma evalPotential_fieldStrength_zero_apply (A : ElectromagneticPotential 3)
    (hA : Differentiable ℝ A) (μ ν : Fin 1 ⊕ Fin 3) (x : SpaceTime 3) :
    evalPotential A (fieldStrength 0 μ ν) x = η ν ν * ∂_ μ A x ν - η μ μ * ∂_ ν A x μ
```

lemma evalPotential_fieldStrength_zero [source]

The formal field strength evaluates to the field strength of the potential with both indices lowered, `F_{μν} = η_{μμ} η_{νν} F^{μν}`.

```
lemma evalPotential_fieldStrength_zero (A : ElectromagneticPotential 3)
    (hA : Differentiable ℝ A) (μ ν : Fin 1 ⊕ Fin 3) (x : SpaceTime 3) :
    evalPotential A (fieldStrength 0 μ ν) x =
      η μ μ * η ν ν * toScalar {A.toFieldStrength x | [μ] [ν]}ᵀ
```

lemma evalPotential_maxwellTerm [source]

***The formal Maxwell term is the Maxwell Lagrangian.** Evaluated on any differentiable electromagnetic potential, the gauge-invariant jet polynomial `F_{μν} F^{μν}` is `-4 μ₀` times the kinetic term `- 1/(4 μ₀) F_{μν} F^{μν}` of `Physlib.Electromagnetism`.*

```
lemma evalPotential_maxwellTerm (ℱ : FreeSpace) (A : ElectromagneticPotential 3)
    (hA : Differentiable ℝ A) (x : SpaceTime 3) :
    evalPotential A maxwellTerm x = -(4 * ℱ.μ₀) * A.kineticTerm ℱ x
```

lemma electricField_eq_evalPotential_fieldStrength [source]

The electric field is (the speed of light times) the evaluated time–space components of the formal field strength: `E_i = c ∂_0 A_i - c ∂_i A_0` with lowered indices.

```
lemma electricField_eq_evalPotential_fieldStrength (c : SpeedOfLight)
    (A : ElectromagneticPotential 3) (hA : Differentiable ℝ A) (t : Time)
    (x : Space) (i : Fin 3) :
    A.electricField c t x i =
      c * evalPotential A (fieldStrength 0 (Sum.inl 0) (Sum.inr i))
        ((toTimeAndSpace c).symm (t, x))
```

lemma magneticField_eq_evalPotential_fieldStrength [source]

The magnetic field is the evaluated space–space components of the formal field strength, `B_i = - F_{(i+1)(i+2)}` with lowered indices.

```
lemma magneticField_eq_evalPotential_fieldStrength (c : SpeedOfLight)
    (A : ElectromagneticPotential 3) (hA : Differentiable ℝ A) (t : Time)
    (x : Space) (i : Fin 3) :
```

```
    A.magneticField c t x i =
      - evalPotential A (fieldStrength 0 (Sum.inr (i + 1)) (Sum.inr (i + 2)))
        ((toTimeAndSpace c).symm (t, x))
```

lemma evalPotential_neg_quarter_maxwellTerm [source]

The Maxwell part - `1/4 F_{μν} F^{μν}` of the QED Lagrangian evaluates to `μ₀` times the electromagnetic kinetic term of `Physlib.Electromagnetism.Dynamics`: the two Lagrangians agree up to the choice of units absorbed into the field normalisation.

```
lemma evalPotential_neg_quarter_maxwellTerm (𝓕 : FreeSpace)
    (A : ElectromagneticPotential 3) (hA : Differentiable ℝ A) (x : SpaceTime 3) :
    evalPotential A ((-(1 : ℝ)/4) • maxwellTerm) x = 𝓕.μ₀ * A.kineticTerm 𝓕 x
```

lemma contDiff_coPotential [source]

```
lemma contDiff_coPotential {A : ElectromagneticPotential 3} (hA : ContDiff ℝ 2 A)
    (ν : Fin 1 ⊕ Fin 3) : ContDiff ℝ 2 (coPotential A ν)
```

lemma derivMultiset_pair [source]

The iterated derivative along a pair of directions, in either order: for a `C²` function the canonical sorted order is immaterial by Clairaut's theorem.

```
lemma derivMultiset_pair (a b : Fin 1 ⊕ Fin 3) (f : SpaceTime 3 → ℝ)
    (hf : ContDiff ℝ 2 f) :
    derivMultiset {a, b} f = ∂_ a (∂_ b f)
```

lemma deriv_sub_eq [source]

```
lemma deriv_sub_eq {f g : SpaceTime 3 → ℝ} (lam : Fin 1 ⊕ Fin 3)
    (hf : Differentiable ℝ f) (hg : Differentiable ℝ g) :
    ∂_ lam (f - g) = ∂_ lam f - ∂_ lam g
```

lemma evalPotential_fieldStrength_singleton [source]

Evaluation intertwines the first-order jet with the spacetime derivative: the evaluated `∂_lam F_{μν}` is the derivative of the evaluated `F_{μν}`.

```
lemma evalPotential_fieldStrength_singleton (A : ElectromagneticPotential 3)
    (hA : ContDiff ℝ 2 A) (lam μ ν : Fin 1 ⊕ Fin 3) :
    evalPotential A (fieldStrength {lam} μ ν) =
      ∂_ lam (evalPotential A (fieldStrength 0 μ ν))
```

lemma evalPotential_maxwell_homogeneous [source]

***The homogeneous Maxwell equations** in covariant form, `∂_lam F_{μν} + ∂_μ F_{ν lam} + ∂_ν F_{lam μ} = 0`, as the evaluation of the formal Bianchi identity of `PhyslibAlpha.Particles.QED.FieldStrength`: Faraday's law and the absence of magnetic monopoles are its time–space–space and space–space–space components.*

```
lemma evalPotential_maxwell_homogeneous (A : ElectromagneticPotential 3)
    (hA : ContDiff ℝ 2 A) (lam μ ν : Fin 1 ⊕ Fin 3) :
    ∂_ lam (evalPotential A (fieldStrength 0 μ ν)) +
```

```
    ∂_ μ (evalPotential A (fieldStrength 0 ν lam)) +
    ∂_ ν (evalPotential A (fieldStrength 0 lam μ)) = 0
```

lemma deriv_const_mul_apply [source]

```
lemma deriv_const_mul_apply (c : ℝ) {f : SpaceTime 3 → ℝ} (ρ : Fin 1 ⊕ Fin 3)
    (hf : Differentiable ℝ f) (x : SpaceTime 3) :
    ∂_ ρ (fun y => c * f y) x = c * ∂_ ρ f x
```

lemma isExtrema_iff_evalPotential_maxwellOperator [source]

***The action principle through the jet algebra**: an electromagnetic potential extremises the Maxwell action with source `J` exactly when the evaluated formal Maxwell operator equals `μ₀ J` — the inhomogeneous Maxwell equations `∂_μ F^{μν} = μ₀ J^ν`.*

```
lemma isExtrema_iff_evalPotential_maxwellOperator (ℱ : FreeSpace)
    (A : ElectromagneticPotential 3) (hA : ContDiff ℝ ∞ A)
    (J : LorentzCurrentDensity 3) (hJ : ContDiff ℝ ∞ J) :
    ElectromagneticPotential.IsExtrema ℱ A J ↔
      ∀ x ν, evalPotential A (maxwellOperator ν) x = ℱ.μ₀ * J x ν
```

lemma evalPotential_fieldStrength_lorentzAction [source]

***Compatibility of the formal and concrete Lorentz actions**: the evaluation of the field strength on `Λ • A` at `x` is the evaluation of its formal Lorentz transform on `A` at `Λ⁻¹ • x`, matching the equivariance `Physlib.Electromagnetism.Kinematics.FieldStrength.toFieldStrengt` on the concrete side.*

```
lemma evalPotential_fieldStrength_lorentzAction (Λ : LorentzGroup 3)
    (A : ElectromagneticPotential 3) (hA : Differentiable ℝ A)
    (μ ν : Fin 1 ⊕ Fin 3) (x : SpaceTime 3) :
    evalPotential (Λ • A) (fieldStrength 0 μ ν) x =
      evalPotential A (lorentzAction Λ (fieldStrength 0 μ ν)) (Λ⁻¹ • x)
```

lemma differentiable_gaugeTransform [source]

```
lemma differentiable_gaugeTransform {A : ElectromagneticPotential 3} {χ : SpaceTime 3 → ℝ}
    (hA : Differentiable ℝ A) (hχ : ContDiff ℝ 2 χ) :
    Differentiable ℝ (ElectromagneticPotential.gaugeTransform χ A)
```

lemma evalPotential_fieldStrength_gaugeTransform [source]

The evaluated field strength is invariant under the concrete gauge transformation `A ↦ A + ∂χ`, matching the formal gauge invariance `Physlib.Particles.QED.GaugeInvariance.Photon.JetAlgebra`

```
lemma evalPotential_fieldStrength_gaugeTransform (A : ElectromagneticPotential 3)
    (χ : SpaceTime 3 → ℝ) (hA : Differentiable ℝ A) (hχ : ContDiff ℝ 2 χ)
    (μ ν : Fin 1 ⊕ Fin 3) (x : SpaceTime 3) :
    evalPotential (ElectromagneticPotential.gaugeTransform χ A) (fieldStrength 0 μ ν) x =
      evalPotential A (fieldStrength 0 μ ν) x
```

lemma evalPotential_maxwellTerm_gaugeTransform [source]

The Maxwell Lagrangian is gauge invariant, as read off from the jet algebra.

```
lemma evalPotential_maxwellTerm_gaugeTransform (A : ElectromagneticPotential 3)
    (χ : SpaceTime 3 → ℝ) (hA : Differentiable ℝ A) (hχ : ContDiff ℝ 2 χ)
    (x : SpaceTime 3) :
    evalPotential (ElectromagneticPotential.gaugeTransform χ A) maxwellTerm x =
      evalPotential A maxwellTerm x
```

2.398 `PhyslibAlpha.Particles.QED.FermionStatistics`

PhyslibAlpha/Particles/QED/FermionStatistics.lean on GitHub.

lemma ofGenerator_mul_self [source]

```
@[simp]
lemma ofGenerator_mul_self (j : JetGenerators) :
    ofGenerator j * ofGenerator j = 0
```

lemma ofGenerator_anticommute [source]

The jet coordinates of the electron anticommute: the electron is a fermion.

```
lemma ofGenerator_anticommute (i j : JetGenerators) :
    ofGenerator i * ofGenerator j = -(ofGenerator j * ofGenerator i)
```

lemma ψ_mul_ψ_anticomm [source]

The electron jet coordinates anticommute.

```
lemma ψ_mul_ψ_anticomm (s t : Multiset (Fin 1 ⊕ Fin 3)) (α β : Fin 2 ⊕ Fin 2) :
    ψ s α * ψ t β = -(ψ t β * ψ s α)
```

lemma ψ_mul_barψ_anticomm [source]

The electron and conjugate-electron jet coordinates anticommute.

```
lemma ψ_mul_barψ_anticomm (s t : Multiset (Fin 1 ⊕ Fin 3)) (α β : Fin 2 ⊕ Fin 2) :
    ψ s α * barψ t β = -(barψ t β * ψ s α)
```

lemma barψ_mul_barψ_anticomm [source]

The conjugate-electron jet coordinates anticommute.

```
lemma barψ_mul_barψ_anticomm (s t : Multiset (Fin 1 ⊕ Fin 3)) (α β : Fin 2 ⊕ Fin 2) :
    barψ s α * barψ t β = -(barψ t β * barψ s α)
```

lemma ψ_mul_self [source]

Pauli exclusion: an electron jet coordinate squares to zero.

```
@[simp]
lemma ψ_mul_self (s : Multiset (Fin 1 ⊕ Fin 3)) (α : Fin 2 ⊕ Fin 2) :
    ψ s α * ψ s α = 0
```

lemma barψ_mul_self [source]

Pauli exclusion: a conjugate electron jet coordinate squares to zero.

```
@[simp]
lemma barψ_mul_self (s : Multiset (Fin 1 ⊕ Fin 3)) (α : Fin 2 ⊕ Fin 2) :
    barψ s α * barψ s α = 0
```

lemma A_mul_A_comm [source]

The photon jet coordinates commute among themselves: the photon is a boson.

```
lemma A_mul_A_comm (s t : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    A s μ * A t ν = A t ν * A s μ
```

lemma A_mul_ψ_comm [source]

The photon jet coordinates commute with the electron jet coordinates.

```
lemma A_mul_ψ_comm (s t : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (α : Fin 2 ⊕ Fin 2) :
    A s μ * ψ t α = ψ t α * A s μ
```

lemma A_mul_barψ_comm [source]

The photon jet coordinates commute with the conjugate electron jet coordinates.

```
lemma A_mul_barψ_comm (s t : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (α : Fin 2 ⊕ Fin 2) :
    A s μ * barψ t α = barψ t α * A s μ
```

2.399 PhyslibAlpha.Particles.QED.Fields

PhyslibAlpha/Particles/QED/Fields.lean on GitHub.

def Photon.JetAlgebra.fieldStrength [source]

The formal field strength `∂_s F_{μν} = ∂_s ∂_μ A_ν - ∂_s ∂_ν A_μ`.

```
noncomputable def fieldStrength (s : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    JetAlgebra
```

def Photon.JetAlgebra.maxwellTerm [source]

The formal Maxwell term `F_{μν} F^{μν}`, both indices raised with the (diagonal) Minkowski metric.

```
noncomputable def maxwellTerm : JetAlgebra
```

def maxwellOperator [source]

The Maxwell operator `∂_μ F^{μν}`: the divergence of the field strength with raised indices. Its vanishing is the vacuum Maxwell equation; its evaluation on an honest potential is the Euler–Lagrange gradient of the Maxwell action — see `PhyslibAlpha.Particles.QED.Evaluatio`

```
noncomputable def maxwellOperator (ν : Fin 1 ⊕ Fin 3) : JetAlgebra
```

def A [source]

The photon jet coordinate `∂_s A_μ` in the QED jet algebra.

```
noncomputable def A (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    JetAlgebra
```

def ψ [source]

The electron jet coordinate `∂_s ψ_α` in the QED jet algebra.

```
noncomputable def ψ (s : Multiset (Fin 1 ⊕ Fin 3)) (α : Fin 2 ⊕ Fin 2) :
    JetAlgebra
```

def barψ [source]

The conjugate electron jet coordinate `∂_s ψ¯_α` in the QED jet algebra.

```
noncomputable def barψ (s : Multiset (Fin 1 ⊕ Fin 3)) (α : Fin 2 ⊕ Fin 2) :
    JetAlgebra
```

def QED.JetAlgebra.fieldStrength [source]

The formal field strength `∂_s F_{μν}` in the QED jet algebra.

```
noncomputable def fieldStrength (s : Multiset (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) : JetAlgebra
```

def QED.JetAlgebra.maxwellTerm [source]

The formal Maxwell term `F_{μν} F^{μν}` in the QED jet algebra. Its evaluation on an honest electromagnetic potential is the Maxwell Lagrangian; see `PhyslibAlpha.Particles.QED.Evaluation`.

```
noncomputable def maxwellTerm : JetAlgebra
```

def gammaMatrix [source]

The Dirac γ matrices in the chiral (Weyl) representation: `γ^μ = ((0, σ^μ), (σ¯^μ, 0))`, acting on the Dirac index `Fin 2 ⊕ Fin 2` whose summands are the left- and right-handed Weyl components.

```
noncomputable def gammaMatrix (μ : Fin 1 ⊕ Fin 3) :
    Matrix (Fin 2 ⊕ Fin 2) (Fin 2 ⊕ Fin 2) ℂ
```

def kineticGamma [source]

The contraction matrices `γ⁰ γ^μ = ((σ¯^μ, 0), (0, σ^μ))` of the Dirac kinetic term `i ψ† (γ⁰ γ^μ) D_μ ψ`; see `Physlib.Particles.QED.GammaMatrices.kineticGamma_eq_gammaMatrix_mul`.

```
noncomputable def kineticGamma (μ : Fin 1 ⊕ Fin 3) :
    Matrix (Fin 2 ⊕ Fin 2) (Fin 2 ⊕ Fin 2) ℂ
```

def covDψ [source]

The covariant derivative jet `(D_μ ψ)_α = ∂_μ ψ_α + i e A_μ ψ_α` of the electron.

```
noncomputable def covDψ (e : ℝ) (μ : Fin 1 ⊕ Fin 3) (α : Fin 2 ⊕ Fin 2) :
    JetAlgebra
```

def covDbarψ [source]

The covariant derivative jet `(D_μ ψ¯)_α = ∂_μ ψ¯_α - i e A_μ ψ¯_α` of the conjugate electron.

```
noncomputable def covDbarψ (e : ℝ) (μ : Fin 1 ⊕ Fin 3) (α : Fin 2 ⊕ Fin 2) :
    JetAlgebra
```

2.400 PhyslibAlpha.Particles.QED.FieldStrength

PhyslibAlpha/Particles/QED/FieldStrength.lean on GitHub.

lemma Photon.JetAlgebra.fieldStrength_antisymm [source]

```
lemma fieldStrength_antisymm (s : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    fieldStrength s μ ν = -fieldStrength s ν μ
```

lemma Photon.JetAlgebra.fieldStrength_self [source]

```
@[simp]
lemma fieldStrength_self (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    fieldStrength s μ μ = 0
```

lemma Photon.JetAlgebra.fieldStrength_bianchi [source]

***The Bianchi identity** `∂_lam F_{μν} + ∂_μ F_{ν lam} + ∂_ν F_{lam μ} = 0` in the photon jet algebra: the homogeneous Maxwell equations hold exactly, for every derivative order `s`.*

```
lemma fieldStrength_bianchi (s : Multiset (Fin 1 ⊕ Fin 3))
    (lam μ ν : Fin 1 ⊕ Fin 3) :
    fieldStrength (s + {lam}) μ ν + fieldStrength (s + {μ}) ν lam +
      fieldStrength (s + {ν}) lam μ = 0
```

lemma fieldStrength_eq_sub [source]

The field strength is the antisymmetrised derivative of the potential, `∂_s F_{μν} = ∂_s ∂_μ A_ν - ∂_s ∂_ν A_μ`.

```
lemma fieldStrength_eq_sub (s : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    fieldStrength s μ ν = A (s + {μ}) ν - A (s + {ν}) μ
```

lemma QED.JetAlgebra.fieldStrength_antisymm [source]

```
lemma fieldStrength_antisymm (s : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    fieldStrength s μ ν = -fieldStrength s ν μ
```

lemma QED.JetAlgebra.fieldStrength_self [source]

```
@[simp]
lemma fieldStrength_self (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    fieldStrength s μ μ = 0
```

lemma QED.JetAlgebra.fieldStrength_bianchi [source]

***The Bianchi identity** in the QED jet algebra.*

```
lemma fieldStrength_bianchi (s : Multiset (Fin 1 ⊕ Fin 3))
    (lam μ ν : Fin 1 ⊕ Fin 3) :
    fieldStrength (s + {lam}) μ ν + fieldStrength (s + {μ}) ν lam +
      fieldStrength (s + {ν}) lam μ = 0
```

lemma fieldStrength_singleton_eq_jetDeriv [source]

The first-order field-strength jet is the total derivative of the zeroth-order one, in the photon jet algebra.

```
lemma _root_.QED.Photon.JetAlgebra.fieldStrength_singleton_eq_jetDeriv
    (ρ μ ν : Fin 1 ⊕ Fin 3) :
    Photon.JetAlgebra.fieldStrength {ρ} μ ν =
      Photon.JetAlgebra.jetDeriv ρ (Photon.JetAlgebra.fieldStrength 0 μ ν)
```

lemma jetDeriv_A [source]

The total derivative appends the derivative index to the photon jet coordinate.

```
@[simp]
lemma jetDeriv_A (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (μ : Fin 1 ⊕ Fin 3) :
    jetDeriv ρ (A s μ) = A (s + {ρ}) μ
```

lemma jetDeriv_ψ [source]

The total derivative appends the derivative index to the electron jet coordinate.

```
@[simp]
lemma jetDeriv_ψ (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (α : Fin 2 ⊕ Fin 2) :
    jetDeriv ρ (ψ s α) = ψ (s + {ρ}) α
```

lemma jetDeriv_barψ [source]

```
@[simp]
lemma jetDeriv_barψ (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (α : Fin 2 ⊕ Fin 2) :
    jetDeriv ρ (barψ s α) = barψ (s + {ρ}) α
```

2.401 PhyslibAlpha.Particles.QED.GammaMatrices

PhyslibAlpha/Particles/QED/GammaMatrices.lean on GitHub.

lemma pauliMatrix_mul_smul_add_swap [source]

```
lemma pauliMatrix_mul_smul_add_swap (μ ν : Fin 1 ⊕ Fin 3) :
    σ μ * (η ν ν • σ ν) + σ ν * (η μ μ • σ μ) =
      (2 * η μ ν) • (1 : Matrix (Fin 2) (Fin 2) ℂ)
```

lemma smul_pauliMatrix_mul_add_swap [source]

```
lemma smul_pauliMatrix_mul_add_swap (μ ν : Fin 1 ⊕ Fin 3) :
    (η μ μ • σ μ) * σ ν + (η ν ν • σ ν) * σ μ =
      (2 * η μ ν) • (1 : Matrix (Fin 2) (Fin 2) ℂ)
```

lemma gammaMatrix_mul_add_swap [source]

***The Clifford algebra relation** of the Dirac γ matrices: `γ^μ γ^ν + γ^ν γ^μ = 2 η^{μν} 1`. This is the algebraic identity that makes the Dirac operator a square root of the wave operator, and hence the Dirac equation relativistic.*

```
lemma gammaMatrix_mul_add_swap (μ ν : Fin 1 ⊕ Fin 3) :
    gammaMatrix μ * gammaMatrix ν + gammaMatrix ν * gammaMatrix μ =
      (2 * η μ ν) • (1 : Matrix (Fin 2 ⊕ Fin 2) (Fin 2 ⊕ Fin 2) ℂ)
```

lemma gammaMatrix_sq [source]

The square of a γ matrix: `(γ^μ)² = η^{μμ} 1` (no sum).

```
lemma gammaMatrix_sq (μ : Fin 1 ⊕ Fin 3) :
    gammaMatrix μ * gammaMatrix μ =
      (η μ μ) • (1 : Matrix (Fin 2 ⊕ Fin 2) (Fin 2 ⊕ Fin 2) ℂ)
```

lemma gammaMatrix_anticommute [source]

The γ matrices of distinct indices anticommute.

```
lemma gammaMatrix_anticommute {μ ν : Fin 1 ⊕ Fin 3} (h : μ ≠ ν) :
    gammaMatrix μ * gammaMatrix ν = -(gammaMatrix ν * gammaMatrix μ)
```

lemma gammaMatrix_inl_zero [source]

`γ⁰` in the chiral representation is the block off-diagonal identity.

```
lemma gammaMatrix_inl_zero :
    gammaMatrix (Sum.inl 0) = Matrix.fromBlocks 0 1 1 0
```

lemma kineticGamma_eq_gammaMatrix_mul [source]

The contraction matrices of the kinetic term are `γ⁰ γ^μ`.

```
lemma kineticGamma_eq_gammaMatrix_mul (μ : Fin 1 ⊕ Fin 3) :
    kineticGamma μ = gammaMatrix (Sum.inl 0) * gammaMatrix μ
```

lemma kineticGamma_conjTranspose [source]

The contraction matrices γ⁰ γ^μ of the kinetic term are self-adjoint; this is what makes the Dirac kinetic term hermitian up to a total derivative.

```
lemma kineticGamma_conjTranspose (μ : Fin 1 ⊕ Fin 3) :
    (kineticGamma μ)ᴴ = kineticGamma μ
```

lemma gammaMatrix_zero_conjTranspose [source]

γ⁰ is self-adjoint.

```
lemma gammaMatrix_zero_conjTranspose :
    (gammaMatrix (Sum.inl 0))ᴴ = gammaMatrix (Sum.inl 0)
```

lemma gammaMatrix_conjTranspose [source]

The hermiticity relation of the γ matrices, (γ^μ)† = γ⁰ γ^μ γ⁰.

```
lemma gammaMatrix_conjTranspose (μ : Fin 1 ⊕ Fin 3) :
    (gammaMatrix μ)ᴴ =
      gammaMatrix (Sum.inl 0) * gammaMatrix μ * gammaMatrix (Sum.inl 0)
```

2.402 PhyslibAlpha.Particles.QED.GaugeInvariance

PhyslibAlpha/Particles/QED/GaugeInvariance.lean on GitHub.

lemma gaugeAction_comp [source]

Photon gauge jets compose by addition: the gauge transformations form a group acting on the photon jet algebra.

```
lemma gaugeAction_comp (c₁ c₂ : GaugeJet) :
    (gaugeAction c₁).comp (gaugeAction c₂) = gaugeAction (c₁ + c₂)
```

lemma gaugeAction_zero [source]

```
@[simp]
lemma gaugeAction_zero : gaugeAction 0 = AlgHom.id ℝ JetAlgebra
```

lemma Photon.JetAlgebra.gaugeAction_fieldStrength [source]

```
@[simp]
lemma gaugeAction_fieldStrength (c : GaugeJet) (s : Multiset (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) :
    gaugeAction c (fieldStrength s μ ν) = fieldStrength s μ ν
```

lemma Photon.JetAlgebra.gaugeAction_maxwellTerm [source]

```
@[simp]
lemma gaugeAction_maxwellTerm (c : GaugeJet) : gaugeAction c maxwellTerm = maxwellTerm
```

lemma Electron.JetAlgebra.gaugeAction_trivial [source]

The trivial gauge jet acts trivially on the electron jet algebra: the gauge action is unital.

```
lemma gaugeAction_trivial (e : ℝ) :
    gaugeAction (GaugeJet.trivial e) = AlgHom.id ℂ JetAlgebra
```

lemma QED.JetAlgebra.gaugeAction_trivial [source]

The trivial gauge jet acts trivially on the QED jet algebra: the gauge action is unital.

```
lemma gaugeAction_trivial (e : ℝ) :
    gaugeAction (GaugeJet.trivial e) = AlgHom.id ℂ JetAlgebra
```

lemma Electron.JetAlgebra.gaugeAction_mul [source]

The electron gauge actions compose through the monoid of gauge jets.

```
lemma _root_.QED.Electron.JetAlgebra.gaugeAction_mul {e : ℝ} (g₁ g₂ : GaugeJet e) :
    (Electron.JetAlgebra.gaugeAction g₁).comp (Electron.JetAlgebra.gaugeAction g₂) =
      Electron.JetAlgebra.gaugeAction (g₁ * g₂)
```

lemma gaugeActionPhoton_comp [source]

The complexified photon gauge actions compose by addition of the gauge jets.

```
lemma gaugeActionPhoton_comp (c₁ c₂ : Photon.JetAlgebra.GaugeJet) :
    (gaugeActionPhoton c₁).comp (gaugeActionPhoton c₂) =
      gaugeActionPhoton (c₁ + c₂)
```

lemma QED.JetAlgebra.gaugeAction_mul [source]

***The gauge actions compose through the monoid of gauge jets**: the QED gauge action is a monoid action on the jet algebra.*

```
lemma gaugeAction_mul {e : ℝ} (g₁ g₂ : GaugeJet e) :
    (gaugeAction g₁).comp (gaugeAction g₂) = gaugeAction (g₁ * g₂)
```

lemma gaugeAction_mul_apply [source]

```
lemma gaugeAction_mul_apply {e : ℝ} (g₁ g₂ : GaugeJet e) (x : JetAlgebra) :
    gaugeAction (g₁ * g₂) x = gaugeAction g₁ (gaugeAction g₂ x)
```

lemma gaugeAction_A [source]

The gauge action on the photon jet coordinate: the affine shift $\partial_s A_\mu \mapsto \partial_s A_\mu + \partial_s \partial_\mu \chi$.

```
lemma gaugeAction_A {e : ℝ} (g : GaugeJet e) (s : Multiset (Fin 1 ⊕ Fin 3))
    (μ : Fin 1 ⊕ Fin 3) :
    gaugeAction g (A s μ) = A s μ + (g.χjet (s + {μ}) : ℂ) • 1
```

lemma gaugeAction_ψ_zero [source]

```
@[simp]
lemma gaugeAction_ψ_zero {e : ℝ} (g : GaugeJet e) (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (ψ 0 α) = star (g.phase 0) • ψ 0 α
```

lemma gaugeAction_barψ_zero [source]

```
@[simp]
lemma gaugeAction_barψ_zero {e : ℝ} (g : GaugeJet e) (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (barψ 0 α) = g.phase 0 • barψ 0 α
```

lemma gaugeAction_ψ_singleton [source]

```
lemma gaugeAction_ψ_singleton {e : ℝ} (g : GaugeJet e) (μ : Fin 1 ⊕ Fin 3)
    (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (ψ {μ} α) =
      star (g.phase 0) • ψ {μ} α + star (g.phase {μ}) • ψ 0 α
```

lemma gaugeAction_barψ_singleton [source]

```
lemma gaugeAction_barψ_singleton {e : ℝ} (g : GaugeJet e) (μ : Fin 1 ⊕ Fin 3)
    (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (barψ {μ} α) =
      g.phase 0 • barψ {μ} α + g.phase {μ} • barψ 0 α
```

lemma QED.JetAlgebra.gaugeAction_fieldStrength [source]

```
@[simp]
lemma gaugeAction_fieldStrength {e : ℝ} (g : GaugeJet e)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    gaugeAction g (fieldStrength s μ ν) = fieldStrength s μ ν
```

lemma QED.JetAlgebra.gaugeAction_maxwellTerm [source]

```
@[simp]
lemma gaugeAction_maxwellTerm {e : ℝ} (g : GaugeJet e) :
    gaugeAction g maxwellTerm = maxwellTerm
```

lemma gaugeAction_covDψ [source]

***Gauge covariance of the covariant derivative**: `D_μ ψ` rotates by the conjugate phase, exactly like `ψ` itself. The shift of the photon coordinate cancels the derivative of the phase.*

```
lemma gaugeAction_covDψ {e : ℝ} (g : GaugeJet e) (μ : Fin 1 ⊕ Fin 3)
    (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (covDψ e μ α) = star (g.phase 0) • covDψ e μ α
```

lemma gaugeAction_covDbarψ [source]

Gauge covariance of the conjugate covariant derivative: `D_μ ψ⁻` rotates by the phase, exactly like `ψ⁻` itself.

```
lemma gaugeAction_covDbarψ {e : ℝ} (g : GaugeJet e) (μ : Fin 1 ⊕ Fin 3)
    (α : Fin 2 ⊕ Fin 2) :
    gaugeAction g (covDbarψ e μ α) = g.phase 0 • covDbarψ e μ α
```

lemma gaugeAction_mul_phase_cancel [source]

A product of a factor rotating by the phase and a factor rotating by the conjugate phase is gauge invariant: the phases cancel by unitarity. This is the reason every charge-neutral fermion bilinear of QED is gauge invariant.

```
lemma gaugeAction_mul_phase_cancel {e : ℝ} (g : GaugeJet e) {x y : JetAlgebra}
    (hx : gaugeAction g x = g.phase 0 • x)
    (hy : gaugeAction g y = star (g.phase 0) • y) :
    gaugeAction g (x * y) = x * y
```

lemma gaugeAction_diracKineticTerm [source]

```
@[simp]
lemma gaugeAction_diracKineticTerm {e : ℝ} (g : GaugeJet e) :
    gaugeAction g (diracKineticTerm e) = diracKineticTerm e
```

lemma gaugeAction_diracKineticTermBar [source]

```
@[simp]
lemma gaugeAction_diracKineticTermBar {e : ℝ} (g : GaugeJet e) :
    gaugeAction g (diracKineticTermBar e) = diracKineticTermBar e
```

lemma gaugeAction_electronMassTerm [source]

```
@[simp]
lemma gaugeAction_electronMassTerm {e : ℝ} (g : GaugeJet e) :
    gaugeAction g electronMassTerm = electronMassTerm
```

lemma gaugeAction_lagrangian [source]

***Gauge invariance of the QED Lagrangian.** The Maxwell term is invariant by the symmetry of the photon shift in its derivative indices, the kinetic term by the covariance of the covariant derivative, and the mass term by the unitarity of the phase.*

```
lemma gaugeAction_lagrangian {e : ℝ} (g : GaugeJet e) (m : ℝ) :
    gaugeAction g (lagrangian e m) = lagrangian e m
```

2.403 PhyslibAlpha.Particles.QED.JetCompleteness

PhyslibAlpha/Particles/QED/JetCompleteness.lean on GitHub.

lemma indexClass_dA [source]

```
lemma indexClass_dA (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    (JetGenerators.dA s μ).indexClass = s + {μ}
```

lemma indexClass_ne_zero [source]

```
lemma indexClass_ne_zero (j : JetGenerators) : j.indexClass ≠ 0
```

lemma indexClass_classProj [source]

Erasing the class representative and putting it back as the Lorentz index preserves the class.

```
lemma indexClass_classProj (j : JetGenerators) :
    j.classProj.indexClass = j.indexClass
```

lemma classProj_idem [source]

The class projection is idempotent.

```
lemma classProj_idem (j : JetGenerators) : j.classProj.classProj = j.classProj
```

lemma classProj_eq_classProj_iff [source]

Two jet coordinates have the same class projection exactly when they lie in the same symmetrized-index class.

```
lemma classProj_eq_classProj_iff (j j' : JetGenerators) :
    j.classProj = j'.classProj ↔ j.indexClass = j'.indexClass
```

lemma coord_sub_classProj [source]

A jet coordinate minus the canonical coordinate of its class is a derivative of the field strength.

```
lemma coord_sub_classProj (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (h : (JetGenerators.dA s μ).classProj ≠ JetGenerators.dA s μ) :
    (X (JetGenerators.dA s μ) : JetAlgebra) - X ((JetGenerators.dA s μ).classProj) =
      fieldStrength (s.erase (classRep (s + {μ}))) (classRep (s + {μ})) μ
```

lemma fieldStrength_mem_range [source]

Every field-strength jet lies in the range of the field-strength family.

```
lemma fieldStrength_mem_range (t : Multiset (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    fieldStrength t μ ν ∈ Set.range (fun p :
        Multiset (Fin 1 ⊕ Fin 3) × (Fin 1 ⊕ Fin 3) × (Fin 1 ⊕ Fin 3) =>
        fieldStrength p.1 p.2.1 p.2.2)
```

lemma gaugeInvariant_iff_mem_adjoin_fieldStrength [source]

***Completeness of the field strength for gauge invariance**: an element of the photon jet algebra is invariant under every gauge transformation if and only if it is a polynomial in the derivatives `∂_s F_{μν}` of the field strength. The field strength does not just provide *some* gauge invariants — it generates *all* of them.*

```
lemma gaugeInvariant_iff_mem_adjoin_fieldStrength (x : JetAlgebra) :
    (∀ c : GaugeJet, gaugeAction c x = x) ↔
      x ∈ Algebra.adjoin ℝ (Set.range fun p :
```

```
Multiset (Fin 1 ⊕ Fin 3) × (Fin 1 ⊕ Fin 3) × (Fin 1 ⊕ Fin 3) =>
fieldStrength p.1 p.2.1 p.2.2)
```

2.404 `PhyslibAlpha.Particles.QED.Lagrangian`

`PhyslibAlpha/Particles/QED/Lagrangian.lean` on GitHub.

`def diracKineticTerm` [source]

The Dirac kinetic term `i ψ¯ γ^μ D_μ ψ = i ψ†_α (γ⁰ γ^μ)_{αβ} (D_μ ψ)_β` of the electron.

```
noncomputable def diracKineticTerm (e : ℝ) : JetAlgebra
```

`def diracKineticTermBar` [source]

The conjugate Dirac kinetic term `-i (D_μ ψ¯) γ⁰ γ^μ ψ = -i (D_μ ψ¯)_α (γ⁰ γ^μ)_{αβ} ψ_β`; the hermitian form of the kinetic term is the average of `diracKineticTerm` and this term.

```
noncomputable def diracKineticTermBar (e : ℝ) : JetAlgebra
```

`def electronMassTerm` [source]

The Dirac mass term `ψ¯ ψ = ψ†_α (γ⁰)_{αβ} ψ_β` of the electron. This is the dimension-three term available because the electron is a Dirac fermion: its two Weyl components have the same electric charge, so the bilinear pairing them against the conjugate components is charge neutral.

```
noncomputable def electronMassTerm : JetAlgebra
```

`def diracCurrent` [source]

The Dirac current `J^μ = ψ¯ γ^μ ψ = ψ†_α (γ⁰ γ^μ)_{αβ} ψ_β` of the electron: the Noether current of the `U(1)_em` phase symmetry. Its coupling `- e J^μ A_μ` to the photon is the entire interaction of QED — this is the jet-algebra counterpart of the current coupling of `Physlib.Electromagnetism.Dynamics`; see `PhyslibAlpha.Particles.QED.CurrentCoupling`.

```
noncomputable def diracCurrent (μ : Fin 1 ⊕ Fin 3) : JetAlgebra
```

`def diracEquation` [source]

The Dirac-equation element `γ⁰ (i γ^μ D_μ - m) ψ`, row `α`: its vanishing is the interacting Dirac equation.

```
noncomputable def diracEquation (e m : ℝ) (α : Fin 2 ⊕ Fin 2) : JetAlgebra
```

`def diracAdjEquation` [source]

The adjoint Dirac-equation element `i (D_μ ψ¯) γ⁰ γ^μ + m ψ¯ γ⁰`, column `β`: its vanishing is the interacting adjoint Dirac equation.

```
noncomputable def diracAdjEquation (e m : ℝ) (β : Fin 2 ⊕ Fin 2) : JetAlgebra
```

def qedMaxwellEquation [source]

The QED Maxwell-equation element `∂_μ F^{μν} - e J^ν`: its vanishing is the inhomogeneous Maxwell equation sourced by the Dirac current.

```
noncomputable def qedMaxwellEquation (e : ℝ) (ν : Fin 1 ⊕ Fin 3) : JetAlgebra
```

def lagrangian [source]

The QED Lagrangian `L = - 1/4 F_{μν} F^{μν} + i ψ̄ γ^μ D_μ ψ - m ψ̄ ψ` with electric coupling `e` and electron mass `m`, as an element of the QED jet algebra. Evaluated on an honest electromagnetic potential, the first term is the Maxwell Lagrangian of `Physlib.Electromagnetism`; see `PhyslibAlpha.Particles.QED.Evaluation`.

```
noncomputable def lagrangian (e m : ℝ) : JetAlgebra
```

2.405 PhyslibAlpha.Particles.QED.LorentzInvariance

PhyslibAlpha/Particles/QED/LorentzInvariance.lean on GitHub.

lemma sum_eta_inv_inv [source]

The defining property of the Lorentz group in index form: contracting two rows of `Λ⁻¹` with the Minkowski metric reproduces the metric.

```
lemma sum_eta_inv_inv (Λ : LorentzGroup 3) (τ τ' : Fin 1 ⊕ Fin 3) :
    ∑ μ, η μ μ * ((Λ⁻¹).1 τ μ * (Λ⁻¹).1 τ' μ) = η τ τ'
```

lemma pauliSelfAdjoint'_coe [source]

The covariant Pauli matrices are `σ̄^μ = η_{μμ} σ^μ` (no sum).

```
lemma pauliSelfAdjoint'_coe (μ : Fin 1 ⊕ Fin 3) :
    (PauliMatrix.pauliSelfAdjoint' μ).1 = η μ μ • σ μ
```

lemma kineticGamma_eq_fromBlocks_pauliSelfAdjoint' [source]

The kinetic matrices through the covariant Pauli matrices: `γ⁰ γ^μ = ((σ̄^μ, 0), (0, η_{μμ} σ̄^μ))`.

```
lemma kineticGamma_eq_fromBlocks_pauliSelfAdjoint' (μ : Fin 1 ⊕ Fin 3) :
    JetAlgebra.kineticGamma μ =
      Matrix.fromBlocks (PauliMatrix.pauliSelfAdjoint' μ).1 0 0
        (η μ μ • (PauliMatrix.pauliSelfAdjoint' μ).1)
```

lemma sl2c_conj_pauliSelfAdjoint' [source]

Conjugating a covariant Pauli matrix by `N : SL(2,ℂ)` transforms its index by the image of `N` in the Lorentz group; this is the defining property of the covering map.

```
lemma sl2c_conj_pauliSelfAdjoint' (N : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) :
    N.1 * (PauliMatrix.pauliSelfAdjoint' μ).1 * N.1^H =
      ∑ ν, (Lorentz.SL2C.toLorentzGroup N).1 ν μ •
        (PauliMatrix.pauliSelfAdjoint' ν).1
```

lemma sum_fromBlocks [source]

A block matrix summed over the diagonal blocks.

```
lemma sum_fromBlocks {ι : Type*} (s : Finset ι)
    (A : ι → Matrix (Fin 2) (Fin 2) ℂ) (D : ι → Matrix (Fin 2) (Fin 2) ℂ) :
    ∑ i ∈ s, Matrix.fromBlocks (A i) 0 0 (D i) =
      Matrix.fromBlocks (∑ i ∈ s, A i) 0 0 (∑ i ∈ s, D i)
```

lemma sum_lorentz_inv_conjTranspose_pauli_conj [source]

```
lemma sum_lorentz_inv_conjTranspose_pauli_conj (M : SL(2,ℂ)) (τ : Fin 1 ⊕ Fin 3) :
    ∑ μ, ((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 τ μ •
        (M.1ᴴ * (PauliMatrix.pauliSelfAdjoint' μ).1 * M.1) =
      (PauliMatrix.pauliSelfAdjoint' τ).1
```

lemma sum_lorentz_inv_eta_pauli_conj [source]

The right Weyl block: transporting the index of η_{μμ} σ̄^μ with Λ(M)⁻¹ cancels the conjugation by (M⁻¹)†, through Λ⁻¹ η (Λ⁻¹)ᵀ = η.

```
lemma sum_lorentz_inv_eta_pauli_conj (M : SL(2,ℂ)) (τ : Fin 1 ⊕ Fin 3) :
    ∑ μ, (((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 τ μ * η μ μ) •
        ((M⁻¹).1 * (PauliMatrix.pauliSelfAdjoint' μ).1 * ((M⁻¹).1)ᴴ) =
      η τ τ • (PauliMatrix.pauliSelfAdjoint' τ).1
```

lemma sum_lorentz_inv_spinorRep_kineticGamma [source]

The matrix form of the contraction identity: transporting the vector index of γ⁰ γ^μ with Λ(M)⁻¹ cancels the conjugation by the spinor representation.

```
lemma sum_lorentz_inv_spinorRep_kineticGamma (M : SL(2,ℂ)) (τ : Fin 1 ⊕ Fin 3) :
    ∑ μ, ((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 τ μ •
        ((Electron.JetAlgebra.spinorRep M)ᴴ * JetAlgebra.kineticGamma μ *
          Electron.JetAlgebra.spinorRep M) =
      JetAlgebra.kineticGamma τ
```

lemma sum_kineticGamma_contraction [source]

***The contraction identity of the Dirac kinetic term**: the index form of ∑_μ (Λ⁻¹)_{τμ} S(M)† (γ⁰ γ^μ) S(M) = γ⁰ γ^τ. This is the identity that makes i ψ̄ γ^μ D_μ ψ a Lorentz scalar.*

```
lemma sum_kineticGamma_contraction (M : SL(2,ℂ)) (τ : Fin 1 ⊕ Fin 3)
    (α' β' : Fin 2 ⊕ Fin 2) :
    ∑ μ, ∑ α, ∑ β, JetAlgebra.kineticGamma μ α β *
        (star (Electron.JetAlgebra.spinorRep M α α') *
          (((((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 τ μ : ℝ) : ℂ) *
            Electron.JetAlgebra.spinorRep M β β')) =
      JetAlgebra.kineticGamma τ α' β'
```

lemma spinorRep_conjTranspose_gammaZero_spinorRep [source]

The spinor representation preserves γ⁰: S(M)† γ⁰ S(M) = γ⁰. This is the identity that makes the Dirac mass term m ψ̄ ψ a Lorentz scalar.

```
lemma spinorRep_conjTranspose_gammaZero_spinorRep (M : SL(2,ℂ)) :
    (Electron.JetAlgebra.spinorRep M)^H * JetAlgebra.gammaMatrix (Sum.inl 0) *
      Electron.JetAlgebra.spinorRep M = JetAlgebra.gammaMatrix (Sum.inl 0)
```

lemma sum_gammaZero_contraction [source]

The index form of $S(M)\dagger\ \gamma^0\ S(M) = \gamma^0$.

```
lemma sum_gammaZero_contraction (M : SL(2,ℂ)) (α' β' : Fin 2 ⊕ Fin 2) :
    ∑ α, ∑ β, JetAlgebra.gammaMatrix (Sum.inl 0) α β *
        (star (Electron.JetAlgebra.spinorRep M α α') *
          Electron.JetAlgebra.spinorRep M β β') =
      JetAlgebra.gammaMatrix (Sum.inl 0) α' β'
```

lemma lorentzAction_A_zero [source]

The photon jet coordinate transforms as a covector.

```
lemma lorentzAction_A_zero (M : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) :
    lorentzAction M (A 0 μ) =
      ∑ ν, ((((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 ν μ : ℝ) : ℂ) • A 0 ν
```

lemma lorentzAction_A_singleton [source]

The first-order photon jet coordinate transforms as a two-tensor.

```
lemma lorentzAction_A_singleton (M : SL(2,ℂ)) (ρ μ : Fin 1 ⊕ Fin 3) :
    lorentzAction M (A {ρ} μ) =
      ∑ τ, ∑ ν, ((((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 τ ρ *
          ((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 ν μ : ℝ) : ℂ) • A {τ} ν
```

lemma lorentzAction_ψ_zero [source]

The electron jet coordinate transforms in the spinor representation.

```
lemma lorentzAction_ψ_zero (M : SL(2,ℂ)) (α : Fin 2 ⊕ Fin 2) :
    lorentzAction M (ψ 0 α) =
      ∑ β, Electron.JetAlgebra.spinorRep M α β • ψ 0 β
```

lemma lorentzAction_barψ_zero [source]

The conjugate electron jet coordinate transforms in the conjugate spinor representation.

```
lemma lorentzAction_barψ_zero (M : SL(2,ℂ)) (α : Fin 2 ⊕ Fin 2) :
    lorentzAction M (barψ 0 α) =
      ∑ β, star (Electron.JetAlgebra.spinorRep M α β) • barψ 0 β
```

lemma lorentzAction_ψ_singleton [source]

The first-order electron jet coordinate transforms as a spinor with a covector derivative index.

```
lemma lorentzAction_ψ_singleton (M : SL(2,ℂ)) (ρ : Fin 1 ⊕ Fin 3)
    (α : Fin 2 ⊕ Fin 2) :
    lorentzAction M (ψ {ρ} α) =
      ∑ τ, ∑ β, (((((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 τ ρ : ℝ) : ℂ) *
          Electron.JetAlgebra.spinorRep M α β) • ψ {τ} β
```

lemma lorentzAction_barψ_singleton [source]

```
lemma lorentzAction_barψ_singleton (M : SL(2,ℂ)) (ρ : Fin 1 ⊕ Fin 3)
    (α : Fin 2 ⊕ Fin 2) :
    lorentzAction M (barψ {ρ} α) =
      ∑ τ, ∑ β, (((((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 τ ρ : ℝ) : ℂ) *
          star (Electron.JetAlgebra.spinorRep M α β)) • barψ {τ} β
```

lemma lorentzAction_fieldStrength_zero [source]

The formal field strength transforms as an antisymmetric two-tensor.

```
lemma lorentzAction_fieldStrength_zero (Λ : LorentzGroup 3) (μ ν : Fin 1 ⊕ Fin 3) :
    lorentzAction Λ (fieldStrength 0 μ ν) =
      ∑ a, ∑ b, ((Λ⁻¹).1 a μ * (Λ⁻¹).1 b ν) • fieldStrength 0 a b
```

lemma Photon.JetAlgebra.lorentzAction_maxwellTerm [source]

***Lorentz invariance of the Maxwell term** in the photon jet algebra: the two metric contractions absorb the four transformation matrices through $\Lambda^{-1}\ \eta\ (\Lambda^{-1})^T = \eta$.*

```
lemma lorentzAction_maxwellTerm (Λ : LorentzGroup 3) :
    lorentzAction Λ maxwellTerm = maxwellTerm
```

lemma QED.JetAlgebra.lorentzAction_maxwellTerm [source]

***Lorentz invariance of the Maxwell term** in the QED jet algebra, inherited from the photon jet algebra.*

```
lemma lorentzAction_maxwellTerm (M : SL(2,ℂ)) :
    lorentzAction M maxwellTerm = maxwellTerm
```

lemma lorentzAction_covDψ [source]

The covariant derivative transforms exactly like the first-order jet coordinate: as a spinor with a covector derivative index.

```
lemma lorentzAction_covDψ (M : SL(2,ℂ)) (e : ℝ) (μ : Fin 1 ⊕ Fin 3)
    (β : Fin 2 ⊕ Fin 2) :
    lorentzAction M (covDψ e μ β) =
      ∑ τ, ∑ β', (((((Lorentz.SL2C.toLorentzGroup M)⁻¹).1 τ μ : ℝ) : ℂ) *
        Electron.JetAlgebra.spinorRep M β β') • covDψ e τ β'
```

lemma lorentzAction_electronMassTerm [source]

***Lorentz invariance of the Dirac mass term**: the spinor phases of the electron and its conjugate cancel through $S(M)\dagger\ \gamma^0\ S(M) = \gamma^0$.*

```
lemma lorentzAction_electronMassTerm (M : SL(2,ℂ)) :
    lorentzAction M electronMassTerm = electronMassTerm
```

lemma lorentzAction_diracKineticTerm [source]

***Lorentz invariance of the Dirac kinetic term**: the transformation of the two spinor slots and the derivative slot cancels through the contraction identity of the matrices `γ⁰` `γ^μ`.*

```
lemma lorentzAction_diracKineticTerm (M : SL(2,ℂ)) (e : ℝ) :
    lorentzAction M (diracKineticTerm e) = diracKineticTerm e
```

lemma lorentzAction_lagrangian [source]

***Lorentz invariance of the QED Lagrangian.** The Maxwell term is invariant through `Λ⁻¹ η (Λ⁻¹)^T = η`, the kinetic term through the contraction identity of `γ⁰ γ^μ` under the spinor representation, and the mass term through `S(M)† γ⁰ S(M) = γ⁰`.*

```
lemma lorentzAction_lagrangian (M : SL(2,ℂ)) (e m : ℝ) :
    lorentzAction M (lagrangian e m) = lagrangian e m
```

2.406 PhyslibAlpha.Particles.QED.MassDimension

PhyslibAlpha/Particles/QED/MassDimension.lean on GitHub.

lemma Photon.JetAlgebra.massScale_fieldStrength_zero [source]

The photon-level field strength has mass dimension two.

```
lemma massScale_fieldStrength_zero (c : ℝ) (μ ν : Fin 1 ⊕ Fin 3) :
    massScale c (fieldStrength 0 μ ν) = c ^ 4 • fieldStrength 0 μ ν
```

lemma Photon.JetAlgebra.massScale_maxwellTerm [source]

The photon-level Maxwell term has mass dimension four.

```
lemma massScale_maxwellTerm (c : ℝ) :
    massScale c maxwellTerm = c ^ 8 • maxwellTerm
```

lemma massScale_A [source]

```
lemma massScale_A (c : ℝ) (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    massScale c (A s μ) = (c : ℂ) ^ (2 + 2 * Multiset.card s) • A s μ
```

lemma massScale_ψ [source]

```
lemma massScale_ψ (c : ℝ) (s : Multiset (Fin 1 ⊕ Fin 3)) (α : Fin 2 ⊕ Fin 2) :
    massScale c (ψ s α) = (c : ℂ) ^ (3 + 2 * Multiset.card s) • ψ s α
```

lemma massScale_barψ [source]

```
lemma massScale_barψ (c : ℝ) (s : Multiset (Fin 1 ⊕ Fin 3)) (α : Fin 2 ⊕ Fin 2) :
    massScale c (barψ s α) = (c : ℂ) ^ (3 + 2 * Multiset.card s) • barψ s α
```

lemma massScale_A_zero [source]

The photon jet coordinate has mass dimension one.

```
lemma massScale_A_zero (c : ℝ) (μ : Fin 1 ⊕ Fin 3) :
    massScale c (A 0 μ) = (c : ℂ) ^ 2 • A 0 μ
```

lemma massScale_ψ_zero [source]

The electron jet coordinate has mass dimension `3/2`.

```
lemma massScale_ψ_zero (c : ℝ) (α : Fin 2 ⊕ Fin 2) :
    massScale c (ψ 0 α) = (c : ℂ) ^ 3 • ψ 0 α
```

lemma massScale_barψ_zero [source]

```
lemma massScale_barψ_zero (c : ℝ) (α : Fin 2 ⊕ Fin 2) :
    massScale c (barψ 0 α) = (c : ℂ) ^ 3 • barψ 0 α
```

lemma massScale_ψ_singleton [source]

```
lemma massScale_ψ_singleton (c : ℝ) (μ : Fin 1 ⊕ Fin 3) (α : Fin 2 ⊕ Fin 2) :
    massScale c (ψ {μ} α) = (c : ℂ) ^ 5 • ψ {μ} α
```

lemma massScale_barψ_singleton [source]

```
lemma massScale_barψ_singleton (c : ℝ) (μ : Fin 1 ⊕ Fin 3) (α : Fin 2 ⊕ Fin 2) :
    massScale c (barψ {μ} α) = (c : ℂ) ^ 5 • barψ {μ} α
```

lemma QED.JetAlgebra.massScale_fieldStrength_zero [source]

The field strength has mass dimension two.

```
lemma massScale_fieldStrength_zero (c : ℝ) (μ ν : Fin 1 ⊕ Fin 3) :
    massScale c (fieldStrength 0 μ ν) = (c : ℂ) ^ 4 • fieldStrength 0 μ ν
```

lemma massScale_covDψ [source]

***The covariant derivative is homogeneous**, of the same weight as the plain derivative: the electric coupling `e` is dimensionless. This is the power-counting statement behind the renormalizability of QED.*

```
lemma massScale_covDψ (c : ℝ) (e : ℝ) (μ : Fin 1 ⊕ Fin 3) (α : Fin 2 ⊕ Fin 2) :
    massScale c (covDψ e μ α) = (c : ℂ) ^ 5 • covDψ e μ α
```

lemma massScale_covDbarψ [source]

```
lemma massScale_covDbarψ (c : ℝ) (e : ℝ) (μ : Fin 1 ⊕ Fin 3) (α : Fin 2 ⊕ Fin 2) :
    massScale c (covDbarψ e μ α) = (c : ℂ) ^ 5 • covDbarψ e μ α
```

lemma QED.JetAlgebra.massScale_maxwellTerm [source]

The Maxwell term has mass dimension four.

```
lemma massScale_maxwellTerm (c : ℝ) :
    massScale c maxwellTerm = (c : ℂ) ^ 8 • maxwellTerm
```

lemma massScale_diracKineticTerm [source]

The Dirac kinetic term has mass dimension four.

```
lemma massScale_diracKineticTerm (c : ℝ) (e : ℝ) :
    massScale c (diracKineticTerm e) = (c : ℂ) ^ 8 • diracKineticTerm e
```

lemma massScale_diracKineticTermBar [source]

The conjugate Dirac kinetic term has mass dimension four.

```
lemma massScale_diracKineticTermBar (c : ℝ) (e : ℝ) :
    massScale c (diracKineticTermBar e) = (c : ℂ) ^ 8 • diracKineticTermBar e
```

lemma massScale_electronMassTerm [source]

The Dirac mass term has mass dimension three.

```
lemma massScale_electronMassTerm (c : ℝ) :
    massScale c electronMassTerm = (c : ℂ) ^ 6 • electronMassTerm
```

lemma massScale_lagrangian [source]

***The QED Lagrangian has mass dimension four.** Rescaling all fields by their mass weights takes `L(e, c² m)` to `c⁸ L(e, m)`: the coupling `e` is dimensionless and the electron mass is a coefficient of dimension one, so every term of the Lagrangian is renormalizable.*

```
lemma massScale_lagrangian (c : ℝ) (e m : ℝ) :
    massScale c (lagrangian e (c ^ 2 * m)) = (c : ℂ) ^ 8 • lagrangian e m
```

2.407 PhyslibAlpha.Particles.StandardModel.Basic

PhyslibAlpha/Particles/StandardModel/Basic.lean on GitHub.

abbrev gauge [source]

The gauge group `SU(3) × SU(2) × U(1)`, as a list of factors.

```
abbrev gauge : List FactorSpec
```

abbrev gaugeData [source]

The local gauge data of the Standard Model gauge group: jets of `SU(3) × SU(2) × U(1)` gauge transformations with their Lie algebra of jets.

```
noncomputable abbrev gaugeData
```

abbrev factors [source]

The factors of the gauge group, as what the rows are charged under.

```
noncomputable abbrev factors : Factors gaugeData
```

abbrev quarkDoublet [source]

The quark doublet `q ~ (3, 2)_{1}`.

```
abbrev quarkDoublet : MatterFieldData factors
```

abbrev leptonDoublet [source]

The lepton doublet `l ~ (1, 2)_{-3}`.

```
abbrev leptonDoublet : MatterFieldData factors
```

abbrev upSinglet [source]

The up-type quark singlet `u ~ (3, 1)_{4}`.

```
abbrev upSinglet : MatterFieldData factors
```

abbrev downSinglet [source]

The down-type quark singlet `d ~ (3, 1)_{-2}`.

```
abbrev downSinglet : MatterFieldData factors
```

abbrev leptonSinglet [source]

The charged-lepton singlet `e ~ (1, 1)_{-6}`.

```
abbrev leptonSinglet : MatterFieldData factors
```

abbrev higgs [source]

The Higgs `H ~ (1, 2)_{3}`.

```
abbrev higgs : MatterFieldData factors
```

inductive Fields [source]

The fields of the Standard Model.

```
inductive Fields
  | q
  | l
  | u
  | d
  | e
  | H
  deriving DecidableEq, Repr
```

instance Fintype Fields [source]

```
instance : Fintype Fields
```

def table [source]

***The Standard Model table**: each field with its number of generations, three for the fermions and one for the Higgs.*

```
def table : FieldData factors Fields
  | .q => (3, quarkDoublet)
  | .l => (3, leptonDoublet)
  | .u => (3, upSinglet)
  | .d => (3, downSinglet)
  | .e => (3, leptonSinglet)
  | .H => (1, higgs)
```

def fieldData [source]

***The field content of the Standard Model**: the fifteen fermionic species (five fields in three generations) and the Higgs, as matter fields of `gaugeData`.*

```
noncomputable def fieldData : GaugeFieldData gaugeData
```

lemma card_fermionSpecies [source]

The Standard Model has fifteen fermionic species.

```
lemma card_fermionSpecies : Fintype.card fieldData.FermionSpecies = 15
```

lemma card_bosonSpecies [source]

The Standard Model has one bosonic species.

```
lemma card_bosonSpecies : Fintype.card fieldData.BosonSpecies = 1
```

2.408 PhyslibAlpha.Particles.StandardModel.CovRealization.Basic

PhyslibAlpha/Particles/StandardModel/CovRealization/Basic.lean on GitHub.

structure CovAlgebraRealization [source]

The algebra `B`, with a gauge action, a Lorentz action and a mass-weight grading, is a covariant Standard Model when it receives an algebra map from the covariant jet

algebra of the Standard Model which is equivariant for both actions and compatible with the grading. The covariant fields of the Standard Model then sit inside `B` as the images of the covariant jet algebra's own, and every law they satisfy there is the covariant jet algebra's own law pushed along the map. It is the covariant counterpart of `AlgebraRealization`: where that asks for an equivariant algebra map out of `JetAlgebra`, on which the whole jet gauge group acts, this asks for one out of `CovJetAlgebra`, on which only the global gauge group acts. The last two fields are not consequences of the first four: an equivariant map forces the two actions to be multiplicative only on its image, whereas the sector structures demand them multiplicative on the whole of `B`.

```
structure CovAlgebraRealization (B : Type) [Ring B] [Algebra ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B)
    (repLorentz : Representation ℂ SL(2,ℂ) B)
    (massWeightPoly : B →ₐ[ℂ] Polynomial B) where
  toAlgHom : CovJetAlgebra →ₐ[ℂ] B
  map_repGauge : ∀ (g : GaugeGroupI) (x : CovJetAlgebra),
    toAlgHom (CovJetAlgebra.repGaugeGroupI g x) = repGauge g (toAlgHom x)
  map_repLorentz : ∀ (Λ : SL(2,ℂ)) (x : CovJetAlgebra),
    toAlgHom (CovJetAlgebra.repLorentzGroup Λ x) = repLorentz Λ (toAlgHom x)
  map_massWeight : ∀ x : CovJetAlgebra, massWeightPoly (toAlgHom x)
    = Polynomial.mapAlgHom toAlgHom (CovJetAlgebra.massWeightPoly x)
  repGauge_mul : ∀ (g : GaugeGroupI) (b₁ b₂ : B),
    repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂
  repLorentz_mul : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
    repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂
```

`def id` [source]

The covariant jet algebra of the Standard Model is a covariant Standard Model: it is one along the identity algebra map.

```
noncomputable def id : CovAlgebraRealization CovJetAlgebra CovJetAlgebra.repGaugeGroupI
    CovJetAlgebra.repLorentzGroup CovJetAlgebra.massWeightPoly where
```

`def covF` [source]

The covariant derivatives `∇_l F_{μν}` of the field strength inside `B`.

```
noncomputable def covF {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B
```

`def covH` [source]

The covariant derivatives `∇_l H` of the Higgs field inside `B`.

```
noncomputable def covH {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ HiggsVec →ₗ[ℂ] B
```

`def covBarH` [source]

The covariant derivatives `∇_l H⁻` of the conjugate Higgs field inside `B`.

```
noncomputable def covBarH {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule HiggsVec) →ₗ[ℂ] B
```

def covD [source]

The covariant derivatives of the `i`-th generation down-type quark singlet inside `B`.

```
noncomputable def covD (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ DownSinglet →ₗ[ℂ] B
```

def covBarD [source]

The covariant derivatives of the `i`-th generation conjugate down-type quark singlet inside `B`.

```
noncomputable def covBarD (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule DownSinglet) →ₗ[ℂ] B
```

def covU [source]

The covariant derivatives of the `i`-th generation up-type quark singlet inside `B`.

```
noncomputable def covU (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ UpSinglet →ₗ[ℂ] B
```

def covBarU [source]

The covariant derivatives of the `i`-th generation conjugate up-type quark singlet inside `B`.

```
noncomputable def covBarU (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule UpSinglet) →ₗ[ℂ] B
```

def covQ [source]

The covariant derivatives of the `i`-th generation quark doublet inside `B`.

```
noncomputable def covQ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ QuarkDoublet →ₗ[ℂ] B
```

def covBarQ [source]

The covariant derivatives of the `i`-th generation conjugate quark doublet inside `B`.

```
noncomputable def covBarQ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule QuarkDoublet) →ₗ[ℂ] B
```

def covL [source]

The covariant derivatives of the `i`-th generation lepton doublet inside `B`.

```
noncomputable def covL (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ LeptonDoubletV →ₗ[ℂ] B
```

def covBarL [source]

The covariant derivatives of the `i`-th generation conjugate lepton doublet inside `B`.

```
noncomputable def covBarL (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule LeptonDoubletV) →ₗ[ℂ] B
```

def covE [source]

The covariant derivatives of the `i`-th generation charged-lepton singlet inside `B`.

```
noncomputable def covE (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ LeptonSinglet →ₗ[ℂ] B
```

def covBarE [source]

The covariant derivatives of the `i`-th generation conjugate charged-lepton singlet inside `B`.

```
noncomputable def covBarE (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule LeptonSinglet) →ₗ[ℂ] B
```

lemma map_repGauge_eq [source]

A gauge transformation law transports along the defining map: the map is equivariant for the global gauge group.

```
lemma map_repGauge_eq {g : GaugeGroupI} {x y : CovJetAlgebra}
    (hxy : CovJetAlgebra.repGaugeGroupI g x = y) :
    repGauge g (k.toAlgHom x) = k.toAlgHom y
```

lemma map_anticomm [source]

An anticommutation law transports along the defining map: the map preserves products and negation.

```
lemma map_anticomm {x y : CovJetAlgebra} (hxy : x * y = -(y * x)) :
    k.toAlgHom x * k.toAlgHom y = -(k.toAlgHom y * k.toAlgHom x)
```

lemma map_neg_eq [source]

An antisymmetry transports along the defining map: the map preserves negation.

```
lemma map_neg_eq {x y : CovJetAlgebra} (hxy : x = -y) :
    k.toAlgHom x = -k.toAlgHom y
```

lemma map_massWeight_monomial [source]

A mass-weight eigenvalue equation transports along the defining map: the map carries the grading of the covariant jet algebra to that of `B`.

```
lemma map_massWeight_monomial {n : ℕ} {x : CovJetAlgebra}
    (hx : CovJetAlgebra.massWeightPoly x = Polynomial.monomial n x) :
    massWeightPoly (k.toAlgHom x) = Polynomial.monomial n (k.toAlgHom x)
```

lemma map_lorentz [source]

A Lorentz transformation law of a covariant tower transports along the defining map: the slot mixing is a finite sum of scalar multiples, and the map is linear and equivariant.

```
lemma map_lorentz {V : Type} [AddCommGroup V] [Module ℂ V]
    {rep : Representation ℂ SL(2,ℂ) V}
    {G : {n : ℕ} → (Fin n → (Fin 1 ⊕ Fin 3)) → Module.Dual ℂ V →ₗ[ℂ] CovJetAlgebra}
    (hG : IsLorentzCovDerivTransforms CovJetAlgebra.repLorentzGroup rep G) :
    IsLorentzCovDerivTransforms repLorentz rep
    (fun {_n} l => k.toAlgHom.toLinearMap ∘ₗ G l)
```

def isHiggsSector [source]

The Higgs sector of a covariant Standard Model: the defining map restricted to the covariant jet algebra of the Higgs field.

```
noncomputable def isHiggsSector :
    HiggsAlgebraCovRealization B repGauge repLorentz massWeightPoly where
```

lemma isHiggsSector_covH [source]

The Higgs towers of the Higgs sector of a covariant Standard Model are its own.

```
@[simp]
lemma isHiggsSector_covH (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    k.isHiggsSector.covH n l = k.covH l
```

lemma isHiggsSector_covBarH [source]

The conjugate Higgs towers of the Higgs sector of a covariant Standard Model are its own.

```
@[simp]
lemma isHiggsSector_covBarH (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    k.isHiggsSector.covBarH n l = k.covBarH l
```

lemma isGaugeSector [source]

The gauge sector of a covariant Standard Model.

```
lemma isGaugeSector : IsGaugeSector B repGauge k.repGauge_mul repLorentz k.repLorentz_mul
    (fun {_n} l μ ν => k.covF l μ ν) massWeightPoly where
```

lemma covF_comp_perm_of_map_lie [source]

At a dual vector vanishing on every bracket, such as the hypercharge coordinate, the field-strength tower is symmetric in its derivative slots, at every order: the law of the covariant jet algebra, where those covariant derivatives are ordinary ones, pushed along the defining map. It is not a law of `IsGaugeSector`, and the components along the non-abelian factors have no such symmetry.

```
lemma covF_comp_perm_of_map_lie {φ : Module.Dual ℝ GaugeAlgebra}
    (hφ : ∀ x y : GaugeAlgebra, φ ⁅x, y⁆ = 0) {n : ℕ} (σ : Equiv.Perm (Fin n))
```

```
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    k.covF (l ∘ σ) μ ν φ = k.covF l μ ν φ
```

lemma isFermionSector [source]

The fermion sector of a covariant Standard Model.

```
lemma isFermionSector : IsFermionSector B repGauge k.repGauge_mul repLorentz k.repLorentz_mul
    (fun {_n} i l => k.covD i l) (fun {_n} i l => k.covBarD i l)
    (fun {_n} i l => k.covU i l) (fun {_n} i l => k.covBarU i l)
    (fun {_n} i l => k.covQ i l) (fun {_n} i l => k.covBarQ i l)
    (fun {_n} i l => k.covL i l) (fun {_n} i l => k.covBarL i l)
    (fun {_n} i l => k.covE i l) (fun {_n} i l => k.covBarE i l) massWeightPoly where
```

lemma repGauge_one [source]

The gauge action fixes the unit of the algebra: it is multiplicative, and every element of the group is invertible.

```
lemma repGauge_one (g : GaugeGroupI) : repGauge g (1 : B) = 1
```

lemma repLorentz_one [source]

The Lorentz action fixes the unit of the algebra.

```
lemma repLorentz_one (Λ : SL(2,ℂ)) : repLorentz Λ (1 : B) = 1
```

lemma F_comm_H [source]

The cross-sector commutation rule `F_comm_H` of a covariant Standard Model.

```
lemma F_comm_H {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) :
    Commute (k.covF l μ ν ψ) (k.covH l' φ)
```

lemma F_comm_barH [source]

The cross-sector commutation rule `F_comm_barH` of a covariant Standard Model.

```
lemma F_comm_barH {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    Commute (k.covF l μ ν ψ) (k.covBarH l' φ)
```

lemma F_comm_d [source]

The cross-sector commutation rule `F_comm_d` of a covariant Standard Model.

```
lemma F_comm_d {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ DownSinglet) :
    Commute (k.covF l μ ν ψ) (k.covD i l' φ)
```

lemma F_comm_bard [source]

The cross-sector commutation rule `F_comm_bard` of a covariant Standard Model.

```
lemma F_comm_bard {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    Commute (k.covF l μ ν ψ) (k.covBarD i l' φ)
```

lemma F_comm_u [source]

The cross-sector commutation rule `F_comm_u` of a covariant Standard Model.

```
lemma F_comm_u {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ UpSinglet) :
    Commute (k.covF l μ ν ψ) (k.covU i l' φ)
```

lemma F_comm_baru [source]

The cross-sector commutation rule `F_comm_baru` of a covariant Standard Model.

```
lemma F_comm_baru {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    Commute (k.covF l μ ν ψ) (k.covBarU i l' φ)
```

lemma F_comm_Q [source]

The cross-sector commutation rule `F_comm_Q` of a covariant Standard Model.

```
lemma F_comm_Q {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ QuarkDoublet) :
    Commute (k.covF l μ ν ψ) (k.covQ i l' φ)
```

lemma F_comm_barQ [source]

The cross-sector commutation rule `F_comm_barQ` of a covariant Standard Model.

```
lemma F_comm_barQ {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    Commute (k.covF l μ ν ψ) (k.covBarQ i l' φ)
```

lemma F_comm_L [source]

The cross-sector commutation rule `F_comm_L` of a covariant Standard Model.

```
lemma F_comm_L {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ LeptonDoubletV) :
    Commute (k.covF l μ ν ψ) (k.covL i l' φ)
```

lemma F_comm_barL [source]

The cross-sector commutation rule `F_comm_barL` of a covariant Standard Model.

```
lemma F_comm_barL {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    Commute (k.covF l μ ν ψ) (k.covBarL i l' φ)
```

lemma F_comm_e [source]

The cross-sector commutation rule `F_comm_e` of a covariant Standard Model.

```
lemma F_comm_e {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ LeptonSinglet) :
    Commute (k.covF l μ ν ψ) (k.covE i l' φ)
```

lemma F_comm_bare [source]

The cross-sector commutation rule `F_comm_bare` of a covariant Standard Model.

```
lemma F_comm_bare {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    Commute (k.covF l μ ν ψ) (k.covBarE i l' φ)
```

lemma H_comm_d [source]

The cross-sector commutation rule `H_comm_d` of a covariant Standard Model.

```
lemma H_comm_d {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ DownSinglet) :
    Commute (k.covH l φ) (k.covD i l' φ')
```

lemma H_comm_bard [source]

The cross-sector commutation rule `H_comm_bard` of a covariant Standard Model.

```
lemma H_comm_bard {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule DownSinglet)) :
    Commute (k.covH l φ) (k.covBarD i l' φ')
```

lemma H_comm_u [source]

The cross-sector commutation rule `H_comm_u` of a covariant Standard Model.

```
lemma H_comm_u {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ UpSinglet) :
    Commute (k.covH l φ) (k.covU i l' φ')
```

lemma H_comm_baru [source]

The cross-sector commutation rule `H_comm_baru` of a covariant Standard Model.

```
lemma H_comm_baru {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule UpSinglet)) :
    Commute (k.covH l φ) (k.covBarU i l' φ')
```

lemma H_comm_Q [source]

The cross-sector commutation rule `H_comm_Q` of a covariant Standard Model.

```
lemma H_comm_Q {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ QuarkDoublet) :
    Commute (k.covH l φ) (k.covQ i l' φ')
```

lemma H_comm_barQ [source]

The cross-sector commutation rule `H_comm_barQ` of a covariant Standard Model.

```
lemma H_comm_barQ {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    Commute (k.covH l φ) (k.covBarQ i l' φ')
```

lemma H_comm_L [source]

The cross-sector commutation rule `H_comm_L` of a covariant Standard Model.

```
lemma H_comm_L {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ LeptonDoubletV) :
    Commute (k.covH l φ) (k.covL i l' φ')
```

lemma H_comm_barL [source]

The cross-sector commutation rule `H_comm_barL` of a covariant Standard Model.

```
lemma H_comm_barL {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    Commute (k.covH l φ) (k.covBarL i l' φ')
```

lemma H_comm_e [source]

The cross-sector commutation rule `H_comm_e` of a covariant Standard Model.

```
lemma H_comm_e {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ LeptonSinglet) :
    Commute (k.covH l φ) (k.covE i l' φ')
```

lemma H_comm_bare [source]

The cross-sector commutation rule `H_comm_bare` of a covariant Standard Model.

```
lemma H_comm_bare {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    Commute (k.covH l φ) (k.covBarE i l' φ')
```

lemma barH_comm_d [source]

The cross-sector commutation rule `barH_comm_d` of a covariant Standard Model.

```
lemma barH_comm_d {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ DownSinglet) :
    Commute (k.covBarH l φ) (k.covD i l' φ')
```

lemma barH_comm_bard [source]

The cross-sector commutation rule `barH_comm_bard` of a covariant Standard Model.

```
lemma barH_comm_bard {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule DownSinglet)) :
    Commute (k.covBarH l φ) (k.covBarD i l' φ')
```

lemma barH_comm_u [source]

The cross-sector commutation rule `barH_comm_u` of a covariant Standard Model.

```
lemma barH_comm_u {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ UpSinglet) :
    Commute (k.covBarH l φ) (k.covU i l' φ')
```

lemma barH_comm_baru [source]

The cross-sector commutation rule `barH_comm_baru` of a covariant Standard Model.

```
lemma barH_comm_baru {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule UpSinglet)) :
    Commute (k.covBarH l φ) (k.covBarU i l' φ')
```

lemma barH_comm_Q [source]

The cross-sector commutation rule `barH_comm_Q` of a covariant Standard Model.

```
lemma barH_comm_Q {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ QuarkDoublet) :
    Commute (k.covBarH l φ) (k.covQ i l' φ')
```

lemma barH_comm_barQ [source]

The cross-sector commutation rule `barH_comm_barQ` of a covariant Standard Model.

```
lemma barH_comm_barQ {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    Commute (k.covBarH l φ) (k.covBarQ i l' φ')
```

lemma barH_comm_L [source]

The cross-sector commutation rule `barH_comm_L` of a covariant Standard Model.

```
lemma barH_comm_L {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ LeptonDoubletV) :
    Commute (k.covBarH l φ) (k.covL i l' φ')
```

lemma barH_comm_barL [source]

The cross-sector commutation rule `barH_comm_barL` of a covariant Standard Model.

```
lemma barH_comm_barL {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    Commute (k.covBarH l φ) (k.covBarL i l' φ')
```

lemma barH_comm_e [source]

The cross-sector commutation rule `barH_comm_e` of a covariant Standard Model.

```
lemma barH_comm_e {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ LeptonSinglet) :
    Commute (k.covBarH l φ) (k.covE i l' φ')
```

lemma barH_comm_bare [source]

The cross-sector commutation rule `barH_comm_bare` of a covariant Standard Model.

```
lemma barH_comm_bare {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    Commute (k.covBarH l φ) (k.covBarE i l' φ')
```

def fieldAlgebra [source]

The algebra generated by all the covariant fields of a covariant Standard Model: the covariant-derivative towers of the field strength, of the Higgs and its conjugate, and of the three families of each fermion species with their conjugates.

```
def fieldAlgebra : Subalgebra ℂ B
```

lemma F_commute_mem_fieldAlgebra [source]

```
lemma F_commute_mem_fieldAlgebra {n : ℕ} {l : Fin n → Fin 1 ⊕ Fin 3} {μ ν : Fin 1 ⊕ Fin 3}
    (φ : Module.Dual ℝ GaugeAlgebra) (x : B) (hx : x ∈ k.fieldAlgebra) :
    k.covF l μ ν φ * x = x * k.covF l μ ν φ
```

lemma H_commute_mem_fieldAlgebra [source]

```
lemma H_commute_mem_fieldAlgebra {n : ℕ} {l : Fin n → Fin 1 ⊕ Fin 3}
    (φ : Module.Dual ℂ HiggsVec) (x : B) (hx : x ∈ k.fieldAlgebra) :
    k.covH l φ * x = x * k.covH l φ
```

lemma barH_commute_mem_fieldAlgebra [source]

```
lemma barH_commute_mem_fieldAlgebra {n : ℕ} {l : Fin n → Fin 1 ⊕ Fin 3}
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (x : B) (hx : x ∈ k.fieldAlgebra) :
    k.covBarH l φ * x = x * k.covBarH l φ
```

abbrev mapAdj [source]

Composing an adjoint-indexed family with an algebra map.

```
noncomputable abbrev mapAdj (Φ : B →ₐ[ℂ] B')
    (f : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B) : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B'
```

lemma map_real_smul [source]

An algebra map over $\mathbb{C}$ is real-linear.

```
lemma map_real_smul (Φ : B →ₐ[ℂ] B') (r : ℝ) (b : B) : Φ (r • b) = r • Φ b
```

lemma actionFam_map [source]

The action pairing commutes with an algebra map: it is a finite double sum of scalar multiples of products of components.

```
lemma actionFam_map (Φ : B →ₐ[ℂ] B') (act : GaugeAlgebra →ₗ[ℝ] V →ₗ[ℂ] V)
    (f : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B) (g : Module.Dual ℂ V →ₗ[ℂ] B)
    (φ : Module.Dual ℂ V) :
    Φ (actionFam act f g φ) = actionFam act (mapAdj Φ f) (Φ.toLinearMap ∘ₗ g) φ
```

lemma bracketFam_map [source]

The bracket of two adjoint families commutes with an algebra map.

```
lemma bracketFam_map (Φ : B →ₐ[ℂ] B') (f g : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B)
    (φ : Module.Dual ℝ GaugeAlgebra) :
    Φ (bracketFam f g φ) = bracketFam (mapAdj Φ f) (mapAdj Φ g) φ
```

lemma actionFam_map' [source]

The action pairing of two families related by an algebra map is the pairing of the images.

```
lemma actionFam_map' (Φ : B →ₐ[ℂ] B')
    {f : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B} {f' : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B'}
    (hf : ∀ ψ, Φ (f ψ) = f' ψ)
    {g : Module.Dual ℂ V →ₗ[ℂ] B} {g' : Module.Dual ℂ V →ₗ[ℂ] B'} (hg : ∀ χ, Φ (g χ) = g' χ)
    (φ : Module.Dual ℂ V) :
    Φ (actionFam act f g φ) = actionFam act f' g' φ
```

lemma bracketFam_map' [source]

The bracket of two adjoint families related by an algebra map is the bracket of the images.

```
lemma bracketFam_map' (Φ : B →ₐ[ℂ] B')
    {f : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B} {f' : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B'}
    (hf : ∀ ψ, Φ (f ψ) = f' ψ)
    {g : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B} {g' : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B'}
    (hg : ∀ ψ, Φ (g ψ) = g' ψ) (φ : Module.Dual ℝ GaugeAlgebra) :
    Φ (bracketFam f g φ) = bracketFam f' g' φ
```

lemma actionFamConv_map' [source]

The derived action family commutes with an algebra map.

```
lemma actionFamConv_map' (Φ : B →ₐ[ℂ] B') (hA : ∀ p ρ ψ, Φ (A p ρ ψ) = A' p ρ ψ)
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B}
    {F' : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B'}
    (hF : ∀ s χ, Φ (F s χ) = F' s χ) (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ V) :
    Φ (actionFamConv A act ρ F s φ) = actionFamConv A' act ρ F' s φ
```

lemma bracketFamConv_map' [source]

The derived bracket family commutes with an algebra map.

```
lemma bracketFamConv_map' (Φ : B →ₐ[ℂ] B') (hA : ∀ p ρ ψ, Φ (A p ρ ψ) = A' p ρ ψ)
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B}
    {F' : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B'}
    (hF : ∀ s χ, Φ (F s χ) = F' s χ) (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℝ GaugeAlgebra) :
    Φ (bracketFamConv A ρ F s φ) = bracketFamConv A' ρ F' s φ
```

lemma commutatorFam_map' [source]

The derived commutator family commutes with an algebra map.

```
lemma commutatorFam_map' (Φ : B →ₐ[ℂ] B') (hA : ∀ p ρ ψ, Φ (A p ρ ψ) = A' p ρ ψ)
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ GaugeAlgebra) :
    Φ (commutatorFam A μ ν s φ) = commutatorFam A' μ ν s φ
```

lemma fieldStrength_map' [source]

The field strength commutes with an algebra map.

```
lemma fieldStrength_map' (Φ : B →ₐ[ℂ] B') (hA : ∀ p ρ ψ, Φ (A p ρ ψ) = A' p ρ ψ)
    (μ ν : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ GaugeAlgebra) :
```

```
    Φ (fieldStrength A μ ν s φ) = fieldStrength A' μ ν s φ
```

lemma covDerivAction_map' [source]

The covariant derivative of a matter family commutes with an algebra map.

```
lemma covDerivAction_map' (Φ : B →ₐ[ℂ] B') (hA : ∀ p ρ ψ, Φ (A p ρ ψ) = A' p ρ ψ)
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B}
    {F' : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B'}
    (hF : ∀ s χ, Φ (F s χ) = F' s χ) (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ V) :
    Φ (covDerivAction A act F ρ s φ) = covDerivAction A' act F' ρ s φ
```

lemma covDerivAdjoint_map' [source]

The covariant derivative of an adjoint family commutes with an algebra map.

```
lemma covDerivAdjoint_map' (Φ : B →ₐ[ℂ] B') (hA : ∀ p ρ ψ, Φ (A p ρ ψ) = A' p ρ ψ)
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B}
    {F' : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B'}
    (hF : ∀ s χ, Φ (F s χ) = F' s χ) (ρ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℝ GaugeAlgebra) :
    Φ (covDerivAdjoint A F ρ s φ) = covDerivAdjoint A' F' ρ s φ
```

lemma covDerivIter_map' [source]

The iterated covariant derivative of a matter family commutes with an algebra map.

```
lemma covDerivIter_map' (Φ : B →ₐ[ℂ] B') (hA : ∀ p ρ ψ, Φ (A p ρ ψ) = A' p ρ ψ)
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B}
    {F' : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B'}
    (hF : ∀ s χ, Φ (F s χ) = F' s χ) :
    ∀ (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ V),
      Φ (covDerivIter A act F n l s φ) = covDerivIter A' act F' n l s φ
  | 0, _, s, φ => hF s φ
  | n + 1, l, s, φ =>
    covDerivAction_map' Φ hA
    (fun s' χ => covDerivIter_map' Φ hA hF n (fun i => l i.succ) s' χ) (l 0) s φ
```

lemma iteratedCovDerivAdjoint_map' [source]

The iterated covariant derivative of an adjoint family commutes with an algebra map.

```
lemma iteratedCovDerivAdjoint_map' (Φ : B →ₐ[ℂ] B') (hA : ∀ p ρ ψ, Φ (A p ρ ψ) = A' p ρ ψ)
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B}
    {F' : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B'}
    (hF : ∀ s χ, Φ (F s χ) = F' s χ) :
    ∀ (l : List (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℝ GaugeAlgebra),
      Φ (iteratedCovDerivAdjoint A l F s φ) = iteratedCovDerivAdjoint A' l F' s φ
  | [], s, φ => hF s φ
  | ρ :: l, s, φ =>
    covDerivAdjoint_map' Φ hA
    (fun s' χ => iteratedCovDerivAdjoint_map' Φ hA hF l s' χ) ρ s φ
```

def toCovAlgebraRealization [source]

A Standard Model is a covariant Standard Model, for the global gauge group: the defining algebra map out of the jet algebra, restricted to the covariant jet algebra.

```
noncomputable def toCovAlgebraRealization :
    CovAlgebraRealization B (repGlobal repJet) repLorentz massWeightPoly where
```

lemma toAlgHom_id_A [source]

The defining map of a Standard Model carries the jet algebra's gauge-field symbols to its own: both are the jet algebra's, one of them pushed forward.

```
lemma toAlgHom_id_A (p : Multiset (Fin 1 ⊕ Fin 3)) (ρ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) :
    h.toAlgHom (AlgebraRealization.id.A p ρ ψ) = h.A p ρ ψ
```

lemma toCovAlgebraRealization_covF [source]

The field-strength tower of the covariant Standard Model carried by a Standard Model is its own field-strength tower.

```
@[simp]
lemma toCovAlgebraRealization_covF {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) :
    h.toCovAlgebraRealization.covF l μ ν = h.covF l μ ν
```

lemma toCovAlgebraRealization_covH [source]

The higgs tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covH {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covH l = h.covDerivH l
```

lemma toCovAlgebraRealization_covBarH [source]

The conjugate higgs tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covBarH {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covBarH l = h.covDerivBarH l
```

lemma toCovAlgebraRealization_covD [source]

The down-type quark tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covD (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covD i l = h.covDerivD i l
```

lemma toCovAlgebraRealization_covBarD [source]

The conjugate down-type quark tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covBarD (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covBarD i l = h.covDerivBarD i l
```

lemma toCovAlgebraRealization_covU [source]

The up-type quark tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covU (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covU i l = h.covDerivU i l
```

lemma toCovAlgebraRealization_covBarU [source]

The conjugate up-type quark tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covBarU (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covBarU i l = h.covDerivBarU i l
```

lemma toCovAlgebraRealization_covQ [source]

The quark doublet tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covQ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covQ i l = h.covDerivQ i l
```

lemma toCovAlgebraRealization_covBarQ [source]

The conjugate quark doublet tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covBarQ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covBarQ i l = h.covDerivBarQ i l
```

lemma toCovAlgebraRealization_covL [source]

The lepton doublet tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covL (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covL i l = h.covDerivL i l
```

lemma toCovAlgebraRealization_covBarL [source]

The conjugate lepton doublet tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covBarL (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covBarL i l = h.covDerivBarL i l
```

lemma toCovAlgebraRealization_covE [source]

The charged-lepton singlet tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covE (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covE i l = h.covDerivE i l
```

lemma toCovAlgebraRealization_covBarE [source]

The conjugate charged-lepton singlet tower of the covariant Standard Model carried by a Standard Model is its own.

```
@[simp]
lemma toCovAlgebraRealization_covBarE (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    h.toCovAlgebraRealization.covBarE i l = h.covDerivBarE i l
```

2.409 PhyslibAlpha.Particles.StandardModel.CovRealization.FermionGaugeSector.Basic

PhyslibAlpha/Particles/StandardModel/CovRealization/FermionGaugeSector/Basic.lean on GitHub.

lemma commute_of_mem_gaugeAlgebra_of_mem_fermionAlgebra [source]

The gauge algebra and the fermion algebra commute element-wise: every generator of the gauge algebra commutes with every generator of the fermion algebra by the structure fields `h.F_comm_d`, …, `h.F_comm_bare`, and commutation extends from generators to the algebras they generate.

```
lemma commute_of_mem_gaugeAlgebra_of_mem_fermionAlgebra {x y : B}
    (hx : x ∈ h.isGaugeSector.gaugeAlgebra) (hy : y ∈ h.isFermionSector.fermionAlgebra) :
    Commute x y
```

lemma fermionMassWeight_mul_gaugeMassWeight_le [source]

The fermion sector's mass-weight submodules and the gauge sector's mass-weight submodules commute past each other, in the order needed by `sectorMassWeight_pair_le`.

```
lemma fermionMassWeight_mul_gaugeMassWeight_le (a b : ℕ) :
    h.isFermionSector.massWeightSubmodule a * h.isGaugeSector.massWeightSubmodule b
      ≤ h.isGaugeSector.massWeightSubmodule b * h.isFermionSector.massWeightSubmodule a
```

lemma sectorMassWeight_gauge_fermion_le [source]

***The mixed gauge-fermion sector decomposition.** The weight-`w` piece of the `{gauge, fermion}` sector lies in the join, over the splittings of `w` into two non-zero parts, of the products of the gauge and fermion sectors' own mass-weight submodules.*

```
lemma sectorMassWeight_gauge_fermion_le (w : ℕ) :
    h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.fermion} w
      ≤ ⨆ (p : ℕ × ℕ) (_ : p.1 + p.2 = w) (_ : p.1 ≠ 0) (_ : p.2 ≠ 0),
        h.isGaugeSector.massWeightSubmodule p.1 * h.isFermionSector.massWeightSubmodule p.2
```

lemma sectorMassWeight_gauge_fermion_eq_bot_of_lt_seven [source]

***The mixed gauge-fermion sector vanishes below weight `7`.** A non-zero gauge weight is at least `4` and a non-zero fermion weight is at least `3`, so no splitting of a weight below `7` into two non-zero parts can supply both.*

```
lemma sectorMassWeight_gauge_fermion_eq_bot_of_lt_seven {w : ℕ} (hw : w < 7) :
    h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.fermion} w = ⊥
```

lemma sectorMassWeight_gauge_fermion_eight [source]

***The mixed gauge-fermion sector vanishes at weight `8`.** The only splittings of `8` into two non-zero parts with a non-zero gauge weight and a non-zero fermion weight would need the gauge part to be `4` or `6` (with fermion part `4` or `2`), but the fermion sector vanishes at both `4` and `2`.*

```
lemma sectorMassWeight_gauge_fermion_eight :
    h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.fermion} 8 = ⊥
```

lemma sectorMassWeight_gauge_fermion_seven [source]

***The mixed gauge-fermion sector at weight `7`** is exactly the product of the underived field-strength submodule with the underived fermion submodule: the only splitting of `7` into a non-zero gauge weight and a non-zero fermion weight that survives is `4 + 3`.*

```
lemma sectorMassWeight_gauge_fermion_seven :
    h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.fermion} 7
      = h.isGaugeSector.derivSubmodule 0 * h.isFermionSector.derivSubmodule 0
```

2.410 PhyslibAlpha.Particles.StandardModel.CovRealization.FermionGaugeSector.MassWeight

PhyslibAlpha/Particles/StandardModel/CovRealization/FermionGaugeSector/MassWeight.lean on GitHub.

lemma gaugeFermion_le_centreEigenspace [source]

A field-strength derivative submodule against a fermion one is of half-integer spin: `+1` times `-1`.

```
private lemma gaugeFermion_le_centreEigenspace (a b : ℕ) :
    h.isGaugeSector.derivSubmodule a * h.isFermionSector.derivSubmodule b
      ≤ centreEigenspace repLorentz (-1)
```

lemma mem_of_lorentz_invariant_sectorMassWeight_gauge_fermion_seven_sup [source]

Mass weight seven carries no Lorentz invariant modulo a Lorentz-stable submodule: a Lorentz invariant of `sectorMassWeight {gauge, fermion} 7 ⊔ S` lies in `S`. The weight is the underived field strength against the underived fermion towers, of half-integer spin.

```
lemma mem_of_lorentz_invariant_sectorMassWeight_gauge_fermion_seven_sup (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.fermion} 7 ⊔ S)
    (hL : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_invariant_sectorMassWeight_gauge_fermion_lt_nine_sup [source]

Below mass weight nine the gauge-fermion sector carries no Lorentz invariant: a Lorentz invariant of `sectorMassWeight {gauge, fermion} w ⊔ S` for `w < 9` lies in `S`. Weights below seven and weight eight are trivial submodules, and weight seven is section B.

```
lemma mem_of_invariant_sectorMassWeight_gauge_fermion_lt_nine_sup (w : ℕ) (hw : w < 9)
    (S : Submodule ℂ B) (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.fermion} w ⊔ S)
    (hL : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_sectorMassWeight_gauge_fermion_lt_nine_sup_and_gauge_lorentz_invariant_iff [source]

The classification below mass weight nine as an equivalence, in the shape of the gauge- and Yukawa-sector statements: an element of `sectorMassWeight {gauge, fermion} w ⊔ S` for `w < 9` is fixed by both groups exactly when it is itself an element of `S` fixed by both groups. Gauge stability of `S` is not needed, and neither is gauge invariance of `x`: the forward direction is the boost-weight parity argument, which uses the Lorentz group alone.

```
lemma mem_sectorMassWeight_gauge_fermion_lt_nine_sup_and_gauge_lorentz_invariant_iff
    (w : ℕ) (hw : w < 9) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.fermion} w ⊔ S
        ∧ (∀ g : GaugeGroupI, repGauge g x = x) ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x = y
```

lemma mem_sectorMassWeight_gauge_fermion_lt_nine_sup_and_gauge_lorentz_invariant_iff_mem [source]

The same classification without the existential: below mass weight nine an element of `sectorMassWeight {gauge, fermion} w ⊔ S` fixed by both groups is an element of `S` fixed by both groups, and conversely.

```
lemma mem_sectorMassWeight_gauge_fermion_lt_nine_sup_and_gauge_lorentz_invariant_iff_mem
    (w : ℕ) (hw : w < 9) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.fermion} w ⊔ S
        ∧ (∀ g : GaugeGroupI, repGauge g x = x) ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ (x ∈ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
          ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
```

2.411 PhyslibAlpha.Particles.StandardModel.CovRealization.GaugeHiggsSector.Basic

PhyslibAlpha/Particles/StandardModel/CovRealization/GaugeHiggsSector/Basic.lean on GitHub.

lemma commute_of_mem_gaugeAlgebra_of_mem_higgsAlgebra [source]

The gauge algebra and the Higgs algebra commute element-wise.

```
lemma commute_of_mem_gaugeAlgebra_of_mem_higgsAlgebra {x y : B}
    (hx : x ∈ h.isGaugeSector.gaugeAlgebra) (hy : y ∈ h.isHiggsSector.higgsAlgebra) :
    Commute x y
```

lemma higgsMassWeight_mul_gaugeMassWeight_le [source]

The Higgs and gauge mass-weight submodules commute past each other.

```
lemma higgsMassWeight_mul_gaugeMassWeight_le (a b : ℕ) :
    h.isHiggsSector.massWeightSubmodule a * h.isGaugeSector.massWeightSubmodule b
      ≤ h.isGaugeSector.massWeightSubmodule b * h.isHiggsSector.massWeightSubmodule a
```

lemma sectorMassWeight_gauge_higgs_le [source]

The mixed gauge-Higgs sector decomposition.

```
lemma sectorMassWeight_gauge_higgs_le (w : ℕ) :
    h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} w
      ≤ ⨆ (p : ℕ × ℕ) (_ : p.1 + p.2 = w) (_ : p.1 ≠ 0) (_ : p.2 ≠ 0),
        h.isGaugeSector.massWeightSubmodule p.1 * h.isHiggsSector.massWeightSubmodule p.2
```

lemma sectorMassWeight_gauge_higgs_eq_bot_of_lt_six [source]

The mixed gauge-Higgs sector vanishes below weight `6`.

```
lemma sectorMassWeight_gauge_higgs_eq_bot_of_lt_six {w : ℕ} (hw : w < 6) :
    h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} w = ⊥
```

lemma sectorMassWeight_gauge_higgs_seven [source]

The mixed gauge-Higgs sector vanishes at weight `7`: both a gauge weight and a Higgs weight are even, so they cannot sum to an odd number.

```
lemma sectorMassWeight_gauge_higgs_seven :
    h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} 7 = ⊥
```

lemma sectorMassWeight_gauge_higgs_six [source]

***The mixed gauge-Higgs sector at weight `6`** is exactly the product of the underived field strength with the underived Higgs: the only surviving splitting is `4 + 2`.*

```
lemma sectorMassWeight_gauge_higgs_six :
    h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} 6
      = h.isGaugeSector.derivSubmodule 0 * h.isHiggsSector.derivSubmodule 0
```

lemma mul_le_sectorMassWeight_gauge_higgs [source]

A product of a gauge-weight piece and a Higgs-weight piece lands in the mixed sector of the total weight.

```
lemma mul_le_sectorMassWeight_gauge_higgs {a b w : ℕ} {X Y : Submodule ℂ B}
    (ha : a ≠ 0) (hb : b ≠ 0) (hab : a + b = w)
    (hX : X ≤ h.isGaugeSector.massWeightSubmodule a)
    (hY : Y ≤ h.isHiggsSector.massWeightSubmodule b) :
    X * Y ≤ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} w
```

lemma sectorMassWeight_gauge_higgs_eight [source]

***The mixed gauge-Higgs sector at weight `8`.** The surviving splittings are `4 + 4` and `6 + 2`, giving the field strength against the weight-four Higgs terms and the once-derived field strength against the underived Higgs.*

```
lemma sectorMassWeight_gauge_higgs_eight :
    h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} 8
      = h.isGaugeSector.derivSubmodule 0 * h.isHiggsSector.derivSubmodule 1
        ⊔ h.isGaugeSector.derivSubmodule 0 * h.isHiggsSector.derivSubmodule 0
          * h.isHiggsSector.derivSubmodule 0
        ⊔ h.isGaugeSector.derivSubmodule 1 * h.isHiggsSector.derivSubmodule 0
```

2.412 PhyslibAlpha.Particles.StandardModel.CovRealization.GaugeHiggsSector.MassWeight

PhyslibAlpha/Particles/StandardModel/CovRealization/GaugeHiggsSector/MassWeight.lean on GitHub.

lemma mul_fixed [source]

Multiplying a Lorentz tensor family by a Lorentz-inert element gives a family of the same rank: the element rides through the transformation law untouched.

```
lemma IsLorentzCovariant.mul_fixed {n : ℕ}
    (hmul : ∀ (Λ : SL(2,ℂ)) (x y : B), repLorentz Λ (x * y) = repLorentz Λ x * repLorentz Λ y)
    {T : (Fin n → Fin 1 ⊕ Fin 3) → B} (hT : IsLorentzCovariant n B repLorentz (ofComponents T))
    {y : B} (hy : ∀ g : SL(2,ℂ), repLorentz g y = y) :
    IsLorentzCovariant n B repLorentz (ofComponents fun d => T d * y)
```

lemma ofComponents_mul_right [source]

The map of a family multiplied on the right by an element is the map of the family, multiplied by that element.

```
lemma ofComponents_mul_right {n : ℕ} (T : (Fin n → Fin 1 ⊕ Fin 3) → B) (y : B)
    (t : ℂT(fun _ : Fin n => Color.up)) :
    ofComponents (fun d => T d * y) t = ofComponents T t * y
```

lemma mul_vector [source]

A rank-two family against a Lorentz vector is a rank-three family: the two covector indices of the first factor and the single index of the second make three.

```
lemma IsLorentzCovariant.mul_vector
    (hmul : ∀ (Λ : SL(2,ℂ)) (x y : B), repLorentz Λ (x * y) = repLorentz Λ x * repLorentz Λ y)
    {T : (Fin 2 → Fin 1 ⊕ Fin 3) → B} (hT : IsLorentzCovariant 2 B repLorentz (ofComponents T))
```

```
    {U : (Fin 1 ⊕ Fin 3) → B}
    (hU : ∀ (g : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3), repLorentz g (U μ)
      = ∑ ν : Fin 1 ⊕ Fin 3, (((SL2C.toLorentzGroup g).1 ν μ : ℝ) : ℂ) • U ν) :
    IsLorentzCovariant 3 B repLorentz
      (ofComponents fun d : Fin 3 → Fin 1 ⊕ Fin 3 => T ![d 0, d 1] * U (d 2))
```

lemma mem_of_lorentz_invariant_iSup_rankTwo_span [source]

A Lorentz invariant of a join, over an arbitrary index type, of the spans of bi-Lorentz families with vanishing metric traces lies in the stable submodule it is taken modulo. Each span reduces to the line through its metric contraction, which is zero; the reductions combine over every finite set of indices, and so over the whole join.

```
lemma mem_of_lorentz_invariant_iSup_rankTwo_span {ι : Type}
    {T : ι → (Fin 2 → Fin 1 ⊕ Fin 3) → B}
    (hT : ∀ i, IsLorentzCovariant 2 B repLorentz (ofComponents (T i)))
    (hzero : ∀ i, ofComponents (T i) RankTwo.metric = 0) (S : Submodule ℂ B)
    (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ (⨆ i, Submodule.span ℂ (Set.range (T i))) ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_lorentz_invariant_iSup_rankThree_span [source]

A Lorentz invariant of a join, over an arbitrary index type, of the spans of triple Lorentz families lies in the stable submodule it is taken modulo: three covector indices admit no invariant contraction at all, so each span reduces to ⊥, and the reductions combine over every finite set of indices, and so over the whole join.

```
lemma mem_of_lorentz_invariant_iSup_rankThree_span {ι : Type}
    {T : ι → (Fin 3 → Fin 1 ⊕ Fin 3) → B}
    (hT : ∀ i, IsLorentzCovariant 3 B repLorentz (ofComponents (T i)))
    (S : Submodule ℂ B) (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ (⨆ i, Submodule.span ℂ (Set.range (T i))) ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma repLorentz_eq_self_of_mem_mul [source]

A product of two pointwise Lorentz-inert submodules is pointwise Lorentz inert.

```
lemma repLorentz_eq_self_of_mem_mul
    (hmul : ∀ (Λ : SL(2,ℂ)) (x y : B), repLorentz Λ (x * y) = repLorentz Λ x * repLorentz Λ y)
    {V W : Submodule ℂ B} (hV : ∀ (g : SL(2,ℂ)), ∀ y ∈ V, repLorentz g y = y)
    (hW : ∀ (g : SL(2,ℂ)), ∀ y ∈ W, repLorentz g y = y) (g : SL(2,ℂ)) :
    ∀ y ∈ V * W, repLorentz g y = y
```

lemma repLorentz_mem_mul_of_stable [source]

A product of two Lorentz-stable submodules is Lorentz stable.

```
lemma repLorentz_mem_mul_of_stable
    (hmul : ∀ (Λ : SL(2,ℂ)) (x y : B), repLorentz Λ (x * y) = repLorentz Λ x * repLorentz Λ y)
    {V W : Submodule ℂ B} (hV : ∀ (g : SL(2,ℂ)), ∀ y ∈ V, repLorentz g y ∈ V)
    (hW : ∀ (g : SL(2,ℂ)), ∀ y ∈ W, repLorentz g y ∈ W) (g : SL(2,ℂ)) :
    ∀ y ∈ V * W, repLorentz g y ∈ V * W
```

lemma repLorentz_mem_of_fixed [source]

A pointwise Lorentz-inert submodule is Lorentz stable.

```
lemma repLorentz_mem_of_fixed {V : Submodule ℂ B}
    (hV : ∀ (g : SL(2,ℂ)), ∀ y ∈ V, repLorentz g y = y) (g : SL(2,ℂ)) :
    ∀ y ∈ V, repLorentz g y ∈ V
```

lemma repLorentz_mem_sup_of_stable [source]

A join of two Lorentz-stable submodules is Lorentz stable.

```
lemma repLorentz_mem_sup_of_stable {V W : Submodule ℂ B}
    (hV : ∀ (g : SL(2,ℂ)), ∀ y ∈ V, repLorentz g y ∈ V)
    (hW : ∀ (g : SL(2,ℂ)), ∀ y ∈ W, repLorentz g y ∈ W) (g : SL(2,ℂ)) :
    ∀ y ∈ V ⊔ W, repLorentz g y ∈ V ⊔ W
```

lemma repLorentz_eq_self_of_mem_higgs_derivSubmodule_zero [source]

The underived Higgs and conjugate-Higgs symbols are Lorentz scalars with no derivative slot to rotate, so every element of the weight-two Higgs submodule is fixed by the whole Lorentz group.

```
lemma repLorentz_eq_self_of_mem_higgs_derivSubmodule_zero (g : SL(2,ℂ)) {y : B}
    (hy : y ∈ h.isHiggsSector.derivSubmodule 0) : repLorentz g y = y
```

lemma mem_of_lorentz_invariant_derivSubmodule_zero_mul_fixed_sup [source]

An underived field strength against Lorentz-inert material carries no Lorentz invariant modulo a Lorentz-stable submodule.

```
lemma mem_of_lorentz_invariant_derivSubmodule_zero_mul_fixed_sup (C : Submodule ℂ B)
    (hC : ∀ (g : SL(2,ℂ)), ∀ y ∈ C, repLorentz g y = y) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.isGaugeSector.derivSubmodule 0 * C ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_lorentz_invariant_derivSubmodule_one_mul_fixed_sup [source]

A once-derived field strength against Lorentz-inert material carries no Lorentz invariant modulo a Lorentz-stable submodule.

```
lemma mem_of_lorentz_invariant_derivSubmodule_one_mul_fixed_sup (C : Submodule ℂ B)
    (hC : ∀ (g : SL(2,ℂ)), ∀ y ∈ C, repLorentz g y = y) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.isGaugeSector.derivSubmodule 1 * C ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_lorentz_invariant_derivSubmodule_zero_mul_higgs_one_sup [source]

An underived field strength against a once-derived Higgs carries no Lorentz invariant modulo a Lorentz-stable submodule: three covector indices admit no contraction.

```
lemma mem_of_lorentz_invariant_derivSubmodule_zero_mul_higgs_one_sup (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.isGaugeSector.derivSubmodule 0 * h.isHiggsSector.derivSubmodule 1 ⊔ S)
```

```
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_lorentz_invariant_sectorMassWeight_gauge_higgs_six_sup [source]

Mass weight six carries no Lorentz invariant modulo a Lorentz-stable submodule: a Lorentz invariant of `sectorMassWeight {gauge, higgs} 6 ⊔ S` lies in `S`. The weight is the underived field strength against the underived Higgs, whose two covector indices are contracted only by the metric, and that trace vanishes.

```
lemma mem_of_lorentz_invariant_sectorMassWeight_gauge_higgs_six_sup (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} 6 ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_lorentz_invariant_sectorMassWeight_gauge_higgs_eight_sup [source]

Mass weight eight carries no Lorentz invariant modulo a Lorentz-stable submodule: a Lorentz invariant of `sectorMassWeight {gauge, higgs} 8 ⊔ S` lies in `S`. The three products making up the weight are peeled off one at a time, the two carrying three covector indices by the absence of any contraction and the one carrying two by the vanishing of the metric trace.

```
lemma mem_of_lorentz_invariant_sectorMassWeight_gauge_higgs_eight_sup (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} 8 ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_invariant_sectorMassWeight_gauge_higgs_lt_nine_sup [source]

Below mass weight nine the gauge-Higgs sector carries no Lorentz invariant: a Lorentz invariant of `sectorMassWeight {gauge, higgs} w ⊔ S` for `w < 9` lies in `S`. Weights below six and weight seven are trivial submodules, and weights six and eight are the two index counts.

```
lemma mem_of_invariant_sectorMassWeight_gauge_higgs_lt_nine_sup (w : ℕ) (hw : w < 9)
    (S : Submodule ℂ B) (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} w ⊔ S)
    (hL : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_sectorMassWeight_gauge_higgs_lt_nine_sup_and_gauge_lorentz_invariant_iff [source]

The classification below mass weight nine as an equivalence, in the shape of the gauge- and Yukawa-sector statements: an element of `sectorMassWeight {gauge, higgs} w ⊔ S` for `w < 9` is fixed by both groups exactly when it is itself an element of `S` fixed by both groups. Gauge stability of `S` is not needed, and neither is gauge invariance of `x`: the forward direction is the index count, which uses the Lorentz group alone.

```
lemma mem_sectorMassWeight_gauge_higgs_lt_nine_sup_and_gauge_lorentz_invariant_iff
    (w : ℕ) (hw : w < 9) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} w ⊔ S
        ∧ (∀ g : GaugeGroupI, repGauge g x = x) ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
```

```
        ∧ x = y
```

lemma mem_sectorMassWeight_gauge_higgs_lt_nine_sup_and_gauge_lorentz_invariant_iff_mem [source]

The same classification without the existential: below mass weight nine an element of `sectorMassWeight {gauge, higgs} w ⊔ S` fixed by both groups is an element of `S` fixed by both groups, and conversely.

```
lemma mem_sectorMassWeight_gauge_higgs_lt_nine_sup_and_gauge_lorentz_invariant_iff_mem
    (w : ℕ) (hw : w < 9) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} w ⊔ S
        ∧ (∀ g : GaugeGroupI, repGauge g x = x) ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ (x ∈ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
          ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
```

2.413 PhyslibAlpha.Particles.StandardModel.CovRealization.Generators

PhyslibAlpha/Particles/StandardModel/CovRealization/Generators.lean on GitHub.

inductive Generators [source]

The abstract index of a single covariant generator of the field algebra: one of the covariant-derivative towers of the field strength, of the Higgs and its conjugate, or of the three families of each fermion species and their conjugates, applied to a member of the dual basis of its value space. Only basis indices are stored, so for a fixed tower length the generators of a given mass weight form a finite type. The evaluation in `B` is `generatorVal`.

```
inductive Generators where
  | H : (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 → Generators
  | barH : (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 → Generators
  | F : (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) →
      (Fin 8 ⊕ Fin 3 ⊕ Fin 1) → Generators
  | d : Fin 3 → (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 × Fin 3 → Generators
  | bard : Fin 3 → (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 × Fin 3 → Generators
  | u : Fin 3 → (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 × Fin 3 → Generators
  | baru : Fin 3 → (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 × Fin 3 → Generators
  | Q : Fin 3 → (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 × Fin 3 × Fin 2 → Generators
  | barQ : Fin 3 → (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 × Fin 3 × Fin 2 → Generators
  | L : Fin 3 → (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Generators
  | barL : Fin 3 → (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Generators
  | e : Fin 3 → (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 → Generators
  | bare : Fin 3 → (n : ℕ) → (Fin n → Fin 1 ⊕ Fin 3) → Fin 2 → Generators
```

def IsFermionic [source]

Whether a generator is one of the twenty fermion towers, the barred ones included.

```
def IsFermionic : Generators → Prop
  | .H _ _ _ => False
  | .barH _ _ _ => False
  | .F _ _ _ _ _ => False
  | .d _ _ _ _ => True
  | .bard _ _ _ _ => True
  | .u _ _ _ _ => True
```

```
  | .baru _ _ _ _ => True
  | .Q _ _ _ _ => True
  | .barQ _ _ _ _ => True
  | .L _ _ _ _ => True
  | .barL _ _ _ _ => True
  | .e _ _ _ _ => True
  | .bare _ _ _ _ => True
```

def IsHiggs [source]

The Higgs generators: the covariant towers of the Higgs field and of its conjugate.

```
def IsHiggs : Generators → Prop
  | .H _ _ _ => True
  | .barH _ _ _ => True
  | _ => False
```

def IsGaugeField [source]

Whether a generator is a gauge-field symbol `∇_l F_{μν}^φ`.

```
def IsGaugeField : Generators → Prop
  | .F _ _ _ _ _ => True
  | _ => False
```

def toNumDerivatives [source]

The number of derivatives for a given generator.

```
def toNumDerivatives : Generators → ℕ
  | .H n _ _ => n
  | .barH n _ _ => n
  | .F n _ _ _ _ => n
  | .d _ n _ _ => n
  | .bard _ n _ _ => n
  | .u _ n _ _ => n
  | .baru _ n _ _ => n
  | .Q _ n _ _ => n
  | .barQ _ n _ _ => n
  | .L _ n _ _ => n
  | .barL _ n _ _ => n
  | .e _ n _ _ => n
  | .bare _ n _ _ => n
```

inductive GeneratorClass [source]

The three classes of covariant generator.

```
inductive GeneratorClass where
  | gauge : GeneratorClass
  | higgs : GeneratorClass
  | fermion : GeneratorClass
```

instance instFintypeGeneratorClass [source]

The three classes exhaust `GeneratorClass`. The instance is written out rather than derived: the `Fintype` deriving handler builds the enumerating `Finset` with a membership that resolves through `SetLike`, which is not type-correct here.

```
instance instFintypeGeneratorClass : Fintype GeneratorClass where
```

def kind [source]

The class of a covariant generator.

```
def Generators.kind : Generators → GeneratorClass
  | .F _ _ _ _ _ => .gauge
  | .H _ _ _ => .higgs
  | .barH _ _ _ => .higgs
  | _ => .fermion
```

lemma isGaugeField_iff_kind [source]

```
@[simp]
lemma Generators.isGaugeField_iff_kind (g : Generators) :
    g.IsGaugeField ↔ g.kind = .gauge
```

lemma isHiggs_iff_kind [source]

```
@[simp]
lemma Generators.isHiggs_iff_kind (g : Generators) : g.IsHiggs ↔ g.kind = .higgs
```

lemma isFermionic_iff_kind [source]

```
@[simp]
lemma Generators.isFermionic_iff_kind (g : Generators) :
    g.IsFermionic ↔ g.kind = .fermion
```

def weight [source]

The mass weight (twice the mass dimension) of a covariant generator.

```
def Generators.weight : Generators → ℕ
  | .H n _ _ => 2 * (1 + n)
  | .barH n _ _ => 2 * (1 + n)
  | .F n _ _ _ _ => 2 * (2 + n)
  | .d _ n _ _ => 3 + 2 * n
  | .bard _ n _ _ => 3 + 2 * n
  | .u _ n _ _ => 3 + 2 * n
  | .baru _ n _ _ => 3 + 2 * n
  | .Q _ n _ _ => 3 + 2 * n
  | .barQ _ n _ _ => 3 + 2 * n
  | .L _ n _ _ => 3 + 2 * n
  | .barL _ n _ _ => 3 + 2 * n
  | .e _ n _ _ => 3 + 2 * n
  | .bare _ n _ _ => 3 + 2 * n
```

lemma four_le_weight_of_gauge [source]

A gauge-class generator — a field-strength tower symbol — carries mass weight at least four.

```
lemma Generators.four_le_weight_of_gauge {g : Generators} (hg : g.kind = GeneratorClass.gauge)
    :
    4 ≤ g.weight
```

lemma two_le_weight_of_higgs [source]

A Higgs-class generator — a Higgs or conjugate-Higgs tower symbol — carries mass weight at least two.

```
lemma Generators.two_le_weight_of_higgs {g : Generators} (hg : g.kind = GeneratorClass.higgs) :
    2 ≤ g.weight
```

lemma three_le_weight_of_fermion [source]

A fermion-class generator — any of the ten families of fermion tower symbols — carries mass weight at least three.

```
lemma Generators.three_le_weight_of_fermion {g : Generators}
    (hg : g.kind = GeneratorClass.fermion) : 3 ≤ g.weight
```

def wordClasses [source]

The classes realised by a word in the covariant generators.

```
def wordClasses (gl : List Generators) : Finset GeneratorClass
```

lemma wordClasses_nil [source]

```
@[simp]
lemma wordClasses_nil : wordClasses [] = ∅
```

lemma wordClasses_append [source]

Concatenating words unions the classes they realise.

```
lemma wordClasses_append (gl gl' : List Generators) :
    wordClasses (gl ++ gl') = wordClasses gl ∪ wordClasses gl'
```

lemma wordClasses_cons [source]

Prepending a generator inserts its class.

```
lemma wordClasses_cons (a : Generators) (gl : List Generators) :
    wordClasses (a :: gl) = insert a.kind (wordClasses gl)
```

def classWeight [source]

The total mass weight carried by the generators of a given class in a word.

```
def classWeight (c : GeneratorClass) (gl : List Generators) : ℕ
```

lemma classWeight_cons_of_eq [source]

```
lemma classWeight_cons_of_eq {c : GeneratorClass} {g : Generators} (hg : g.kind = c)
    (t : List Generators) :
    classWeight c (g :: t) = g.weight + classWeight c t
```

lemma classWeight_cons_of_ne [source]

```
lemma classWeight_cons_of_ne {c : GeneratorClass} {g : Generators} (hg : g.kind ≠ c)
    (t : List Generators) : classWeight c (g :: t) = classWeight c t
```

lemma weight_pos [source]

Every generator carries a non-zero mass weight.

```
lemma Generators.weight_pos (g : Generators) : 0 < g.weight
```

lemma classWeight_ne_zero [source]

A class realised by a word carries a non-zero part of its weight.

```
lemma classWeight_ne_zero {c : GeneratorClass} {gl : List Generators}
    (hc : c ∈ wordClasses gl) : classWeight c gl ≠ 0
```

lemma classWeight_add [source]

Over a word realising only two classes, the two class weights add up to the total weight.

```
lemma classWeight_add {c₁ c₂ : GeneratorClass} (hne : c₁ ≠ c₂) {gl : List Generators}
    (hgl : ∀ g ∈ gl, g.kind = c₁ ∨ g.kind = c₂) :
    classWeight c₁ gl + classWeight c₂ gl = (gl.map Generators.weight).sum
```

lemma le_classWeight_of_mem [source]

A class realised by a word carries at least the minimum weight of that class: the generator witnessing the realisation already contributes that much, and the remaining generators of the class only add more.

```
lemma le_classWeight_of_mem {c : GeneratorClass} {gl : List Generators} {m : ℕ}
    (hc : c ∈ wordClasses gl) (hm : ∀ g : Generators, g.kind = c → m ≤ g.weight) :
    m ≤ classWeight c gl
```

lemma classWeight_add_three [source]

The three class weights exhaust the total weight of a word: every generator has exactly one of the three kinds.

```
lemma classWeight_add_three (gl : List Generators) :
    classWeight GeneratorClass.gauge gl + classWeight GeneratorClass.higgs gl
      + classWeight GeneratorClass.fermion gl = (gl.map Generators.weight).sum
```

def generatorVal [source]

The value in `B` of a covariant generator: the corresponding covariant tower applied to the indicated dual basis vector of its value space.

```
noncomputable def generatorVal
    (h : CovAlgebraRealization B repGauge repLorentz massWeightPoly) : Generators → B
  | .H _ l j => h.covH l (HiggsVec.orthonormBasis.toBasis.coord j)
  | .barH _ l j => h.covBarH l ((Basis.conj HiggsVec.orthonormBasis.toBasis).coord j)
  | .F _ l μ ν j => h.covF l μ ν (GaugeAlgebra.stdBasis.coord j)
  | .d i _ l j => h.covD i l (DownSinglet.basis.coord j)
  | .bard i _ l j => h.covBarD i l ((Basis.conj DownSinglet.basis).coord j)
  | .u i _ l j => h.covU i l (UpSinglet.basis.coord j)
  | .baru i _ l j => h.covBarU i l ((Basis.conj UpSinglet.basis).coord j)
  | .Q i _ l j => h.covQ i l (QuarkDoublet.basis.coord j)
  | .barQ i _ l j => h.covBarQ i l ((Basis.conj QuarkDoublet.basis).coord j)
  | .L i _ l j => h.covL i l (LeptonDoubletV.basis.coord j)
  | .barL i _ l j => h.covBarL i l ((Basis.conj LeptonDoubletV.basis).coord j)
  | .e i _ l j => h.covE i l (LeptonSinglet.basis.coord j)
  | .bare i _ l j => h.covBarE i l ((Basis.conj LeptonSinglet.basis).coord j)
```

lemma massWeightPoly_generatorVal [source]

Every covariant generator is a `massWeightPoly`-eigenvector of its weight.

```
lemma massWeightPoly_generatorVal (g : Generators) :
    massWeightPoly (h.generatorVal g) = Polynomial.monomial g.weight (h.generatorVal g)
```

lemma generatorVal_mem_fieldAlgebra [source]

```
lemma generatorVal_mem_fieldAlgebra (g : Generators) :
    h.generatorVal g ∈ h.fieldAlgebra
```

lemma fieldAlgebra_le_adjoin_range [source]

Expanding every dual vector in the dual basis of its value space: the field algebra is already generated by the countable family of basis generators.

```
lemma fieldAlgebra_le_adjoin_range :
    h.fieldAlgebra ≤ Algebra.adjoin ℂ (Set.range h.generatorVal)
```

lemma fieldAlgebra_eq_adjoin_range [source]

The field algebra is generated by the covariant basis generators.

```
lemma fieldAlgebra_eq_adjoin_range :
    h.fieldAlgebra = Algebra.adjoin ℂ (Set.range h.generatorVal)
```

lemma exists_list_map_eq [source]

A list of generator values is the list of values of a list of generators.

```
lemma exists_list_map_eq (l₀ : List B) :
    (∀ y ∈ l₀, y ∈ Set.range h.generatorVal) →
    ∃ gl : List Generators, gl.map h.generatorVal = l₀
```

lemma massWeightPoly_generatorVal_list_prod [source]

A word in the covariant generators is a `massWeightPoly`-eigenvector whose weight is the sum of the weights of its factors.

```
lemma massWeightPoly_generatorVal_list_prod (gl : List Generators) :
    massWeightPoly ((gl.map h.generatorVal).prod) =
      Polynomial.monomial ((gl.map Generators.weight).sum) ((gl.map h.generatorVal).prod)
```

lemma commute_F_generatorVal [source]

The field-strength symbols commute with the value of every covariant generator.

```
lemma commute_F_generatorVal {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (g : Generators) :
    Commute (h.covF l μ ν ψ) (h.generatorVal g)
```

lemma commute_H_generatorVal [source]

The Higgs symbols commute with the value of every covariant generator.

```
lemma commute_H_generatorVal {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (g : Generators) :
    Commute (h.covH l φ) (h.generatorVal g)
```

lemma commute_barH_generatorVal [source]

The conjugate-Higgs symbols commute with the value of every covariant generator.

```
lemma commute_barH_generatorVal {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (g : Generators) :
    Commute (h.covBarH l φ) (h.generatorVal g)
```

lemma commute_generatorVal_of_even [source]

A covariant generator of even mass weight is bosonic: its value commutes with the value of every covariant generator.

```
lemma commute_generatorVal_of_even {g : Generators} (hg : g.weight % 2 = 0)
    (g' : Generators) : Commute (h.generatorVal g) (h.generatorVal g')
```

lemma generatorVal_anticomm_of_odd_of_odd [source]

Fermionic generator values anticommute: the values of two covariant generators of odd mass weight exchange with a sign.

```
lemma generatorVal_anticomm_of_odd_of_odd {g g' : Generators}
    (hg : g.weight % 2 = 1) (hg' : g'.weight % 2 = 1) :
    h.generatorVal g * h.generatorVal g'
      = -(h.generatorVal g' * h.generatorVal g)
```

lemma generatorVal_mul_generatorVal [source]

Two covariant generators exchange up to the sign determined by their mass weights: the weight is odd exactly on the fermionic generators, so the sign is `-1` precisely when both generators are fermionic.

```
lemma generatorVal_mul_generatorVal (g g' : Generators) :
    h.generatorVal g * h.generatorVal g' =
      ((-1 : ℂ) ^ (g.weight * g'.weight)) • (h.generatorVal g' * h.generatorVal g)
```

lemma generatorVal_mul_list_prod [source]

A generator value moves past a word of generators up to the sign of the product of the weights.

```
lemma generatorVal_mul_list_prod (g : Generators) (gl : List Generators) :
    h.generatorVal g * (gl.map h.generatorVal).prod =
      ((-1 : ℂ) ^ (g.weight * (gl.map Generators.weight).sum)) •
        ((gl.map h.generatorVal).prod * h.generatorVal g)
```

lemma list_prod_mul_list_prod [source]

Two words of covariant generators exchange up to the sign of the product of their total weights.

```
lemma list_prod_mul_list_prod (gl gl' : List Generators) :
    (gl.map h.generatorVal).prod * (gl'.map h.generatorVal).prod =
      ((-1 : ℂ) ^ ((gl.map Generators.weight).sum * (gl'.map Generators.weight).sum)) •
        ((gl'.map h.generatorVal).prod * (gl.map h.generatorVal).prod)
```

2.414 PhyslibAlpha.Particles.StandardModel.CovRealization.MassWeight

PhyslibAlpha/Particles/StandardModel/CovRealization/MassWeight.lean on GitHub.

def massWeightSubmodule [source]

All elements of the field algebra of mass weight exactly `n`: the intersection of the algebra generated by the covariant fields with the part on which `massWeightPoly` is the monomial `X ^ n`.

```
noncomputable def massWeightSubmodule
    (h : CovAlgebraRealization B repGauge repLorentz massWeightPoly) (n : ℕ) : Submodule ℂ B
```

lemma massWeightPoly_of_mem_massWeightSubmodule [source]

```
lemma massWeightPoly_of_mem_massWeightSubmodule {n : ℕ} {x : B}
    (hx : x ∈ h.massWeightSubmodule n) :
    massWeightPoly x = Polynomial.monomial n x
```

lemma mem_fieldAlgebra_of_mem_massWeightSubmodule [source]

```
lemma mem_fieldAlgebra_of_mem_massWeightSubmodule {n : ℕ} {x : B}
    (hx : x ∈ h.massWeightSubmodule n) : x ∈ h.fieldAlgebra
```

lemma mem_massWeightSubmodule_of [source]

Membership in a mass-weight submodule: an element of the field algebra which `massWeightPoly` sends to the monomial of that weight.

```
lemma mem_massWeightSubmodule_of {n : ℕ} {x : B} (hmem : x ∈ h.fieldAlgebra)
    (hpoly : massWeightPoly x = Polynomial.monomial n x) :
    x ∈ h.massWeightSubmodule n
```

lemma list_prod_mem_massWeightSubmodule [source]

A word in the covariant generators lies in the mass-weight submodule of its total weight.

```
lemma list_prod_mem_massWeightSubmodule {w : ℕ} {gl : List Generators}
    (hw : (gl.map Generators.weight).sum = w) :
    (gl.map h.generatorVal).prod ∈ h.massWeightSubmodule w
```

lemma coeff_massWeightPoly_mem_span [source]

Reading off the `X ^ w` coefficient of `massWeightPoly` sends the field algebra into the span of the words of total weight `w` — the projection onto the weight-`w` component, with no independence argument needed.

```
lemma coeff_massWeightPoly_mem_span (w : ℕ) {x : B}
    (hx : x ∈ h.fieldAlgebra) :
    (massWeightPoly x).coeff w ∈ Submodule.span ℂ
      {y | ∃ gl : List Generators,
        (gl.map Generators.weight).sum = w ∧ (gl.map h.generatorVal).prod = y}
```

lemma massWeightSubmodule_eq_span [source]

***The weight grading of the field algebra.** The submodule of elements of the field algebra of mass weight `w` is exactly the span of the words in the covariant basis generators of total weight `w`.*

```
lemma massWeightSubmodule_eq_span (w : ℕ) :
    h.massWeightSubmodule w = Submodule.span ℂ
      {x | ∃ gl : List Generators,
        (gl.map Generators.weight).sum = w ∧ (gl.map h.generatorVal).prod = x}
```

lemma mul_eq_smul_mul_of_mem_massWeightSubmodule [source]

Weight-homogeneous elements of the field algebra supercommute: elements of the mass-weight submodules of weights `w` and `w'` exchange up to the sign `(-1) ^ (w * w')` — fermion parity is the parity of the mass weight.

```
lemma mul_eq_smul_mul_of_mem_massWeightSubmodule {w w' : ℕ} {x y : B}
    (hx : x ∈ h.massWeightSubmodule w) (hy : y ∈ h.massWeightSubmodule w') :
    x * y = ((-1 : ℂ) ^ (w * w')) • (y * x)
```

lemma exists_smul_mul_comm_of_mem_massWeightSubmodule [source]

Weight-homogeneous elements of the field algebra commute up to a scalar.

```
lemma exists_smul_mul_comm_of_mem_massWeightSubmodule {w w' : ℕ} {x y : B}
    (hx : x ∈ h.massWeightSubmodule w) (hy : y ∈ h.massWeightSubmodule w') :
    ∃ c : ℂ, x * y = c • (y * x)
```

lemma span_mul_comm_of_mem_massWeightSubmodule [source]

Reordering a product of two weight-homogeneous elements does not change its span.

```
lemma span_mul_comm_of_mem_massWeightSubmodule {w w' : ℕ} {x y : B}
    (hx : x ∈ h.massWeightSubmodule w) (hy : y ∈ h.massWeightSubmodule w') :
    ℂ • (x * y) = ℂ • (y * x)
```

lemma mul_mem_massWeightSubmodule [source]

The mass-weight submodules are multiplicative: weights add under multiplication.

```
lemma mul_mem_massWeightSubmodule {w w2 : ℕ} {x y : B}
    (hx : x ∈ h.massWeightSubmodule w) (hy : y ∈ h.massWeightSubmodule w2) :
    x * y ∈ h.massWeightSubmodule (w + w2)
```

lemma H_mem_massWeightSubmodule [source]

Any Higgs tower symbol lies in the mass-weight submodule of its weight.

```
lemma H_mem_massWeightSubmodule {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) : h.covH l φ ∈ h.massWeightSubmodule (2 * (1 + n))
```

lemma barH_mem_massWeightSubmodule [source]

Any conjugate-Higgs tower symbol lies in the mass-weight submodule of its weight.

```
lemma barH_mem_massWeightSubmodule {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    h.covBarH l φ ∈ h.massWeightSubmodule (2 * (1 + n))
```

lemma F_mem_massWeightSubmodule [source]

Any field-strength tower symbol lies in the mass-weight submodule of its weight.

```
lemma F_mem_massWeightSubmodule {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ GaugeAlgebra) :
    h.covF l μ ν φ ∈ h.massWeightSubmodule (2 * (2 + n))
```

lemma d_mem_massWeightSubmodule [source]

Any `d` tower symbol lies in the mass-weight submodule of its weight.

```
lemma d_mem_massWeightSubmodule (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ DownSinglet) : h.covD i l φ ∈ h.massWeightSubmodule (3 + 2 * n)
```

lemma d_supercommute_mem_massWeightSubmodule [source]

A `d` tower symbol supercommutes with any weight-homogeneous element: its own weight `3 + 2 * n` is odd, so moving it past an element of weight `w` costs `(-1) ^ w`.

```
lemma d_supercommute_mem_massWeightSubmodule (i : Fin 3) {n : ℕ} {l : Fin n → Fin 1 ⊕ Fin 3}
    (φ : Module.Dual ℂ DownSinglet) {w : ℕ} (x : B) (hx : x ∈ h.massWeightSubmodule w) :
    h.covD i l φ * x = ((-1 : ℂ) ^ w) • (x * h.covD i l φ)
```

lemma bard_mem_massWeightSubmodule [source]

Any `bard` tower symbol lies in the mass-weight submodule of its weight.

```
lemma bard_mem_massWeightSubmodule (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) : h.covBarD i l φ ∈ h.massWeightSubmodule (3 +
     2 *
      n)
```

lemma u_mem_massWeightSubmodule [source]

Any `u` tower symbol lies in the mass-weight submodule of its weight.

```
lemma u_mem_massWeightSubmodule (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ UpSinglet) : h.covU i l φ ∈ h.massWeightSubmodule (3 + 2 * n)
```

lemma baru_mem_massWeightSubmodule [source]

Any `baru` tower symbol lies in the mass-weight submodule of its weight.

```
lemma baru_mem_massWeightSubmodule (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    h.covBarU i l φ ∈ h.massWeightSubmodule (3 + 2 * n)
```

lemma Q_mem_massWeightSubmodule [source]

Any `Q` tower symbol lies in the mass-weight submodule of its weight.

```
lemma Q_mem_massWeightSubmodule (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ QuarkDoublet) : h.covQ i l φ ∈ h.massWeightSubmodule (3 + 2 * n)
```

lemma barQ_mem_massWeightSubmodule [source]

Any `barQ` tower symbol lies in the mass-weight submodule of its weight.

```
lemma barQ_mem_massWeightSubmodule (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) : h.covBarQ i l φ ∈ h.massWeightSubmodule (3
    + 2 *
      n)
```

lemma L_mem_massWeightSubmodule [source]

Any `L` tower symbol lies in the mass-weight submodule of its weight.

```
lemma L_mem_massWeightSubmodule (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ LeptonDoubletV) : h.covL i l φ ∈ h.massWeightSubmodule (3 + 2 * n)
```

lemma barL_mem_massWeightSubmodule [source]

Any `barL` tower symbol lies in the mass-weight submodule of its weight.

```
lemma barL_mem_massWeightSubmodule (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) : h.covBarL i l φ ∈ h.massWeightSubmodule
    (3 +
      2 * n)
```

lemma e_mem_massWeightSubmodule [source]

Any `e` tower symbol lies in the mass-weight submodule of its weight.

```
lemma e_mem_massWeightSubmodule (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ LeptonSinglet) : h.covE i l φ ∈ h.massWeightSubmodule (3 + 2 * n)
```

lemma bare_mem_massWeightSubmodule [source]

Any `bare` tower symbol lies in the mass-weight submodule of its weight.

```
lemma bare_mem_massWeightSubmodule (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) : h.covBarE i l φ ∈ h.massWeightSubmodule (3
     +
      2 * n)
```

lemma repGauge_generatorVal_mem [source]

The gauge action carries a covariant generator into the mass-weight submodule of its weight.

```
lemma repGauge_generatorVal_mem (g : GaugeGroupI) (a : Generators) :
    repGauge g (h.generatorVal a) ∈ h.massWeightSubmodule a.weight
```

lemma repLorentz_generatorVal_mem [source]

The Lorentz action carries a covariant generator into the mass-weight submodule of its weight.

```
lemma repLorentz_generatorVal_mem (Λ : SL(2,ℂ)) (a : Generators) :
    repLorentz Λ (h.generatorVal a) ∈ h.massWeightSubmodule a.weight
```

lemma repGauge_mem_massWeightSubmodule [source]

The action `repGauge` preserves the mass-weight submodules.

```
lemma repGauge_mem_massWeightSubmodule {w : ℕ} {x : B} (g : GaugeGroupI)
    (hx : x ∈ h.massWeightSubmodule w) : repGauge g x ∈ h.massWeightSubmodule w
```

lemma repLorentz_mem_massWeightSubmodule [source]

The action `repLorentz` preserves the mass-weight submodules.

```
lemma repLorentz_mem_massWeightSubmodule {w : ℕ} {x : B} (Λ : SL(2,ℂ))
    (hx : x ∈ h.massWeightSubmodule w) : repLorentz Λ x ∈ h.massWeightSubmodule w
```

lemma eq_zero_of_sum_massWeightSubmodule [source]

Components of pairwise distinct mass weights are independent: a vanishing sum of weight-homogeneous elements has vanishing terms.

```
lemma eq_zero_of_sum_massWeightSubmodule {n : ℕ} {w : Fin n → ℕ}
    (hw : Function.Injective w) {f : Fin n → B}
    (hf : ∀ i, f i ∈ h.massWeightSubmodule (w i)) (hsum : ∑ i, f i = 0) :
    ∀ i, f i = 0
```

lemma invariant_of_eq_sum_massWeightSubmodule [source]

An invariant element decomposes into invariant weight components: if a gauge- and Lorentz-invariant `x` is the sum of components of pairwise distinct mass weights, every component is itself gauge and Lorentz invariant.

```
lemma invariant_of_eq_sum_massWeightSubmodule {n : ℕ} {x : B} {w : Fin n → ℕ}
    (hw : Function.Injective w) (f : Fin n → B) (hf : x = ∑ i, f i)
    (hx : ∀ i, f i ∈ h.massWeightSubmodule (w i))
    (hgauge : ∀ g, repGauge g x = x) (hlorentz : ∀ Λ, repLorentz Λ x = x) :
    ∀ i, (∀ g, repGauge g (f i) = f i) ∧ ∀ Λ, repLorentz Λ (f i) = f i
```

lemma invaraint_of_eq_sum_massWeightSubmodule [source]

The reduction of questions on invariants to invariants within mass weight submodules.

```
lemma invaraint_of_eq_sum_massWeightSubmodule {n : ℕ} {x : B}
    (f : Fin n → B) (hf : x = ∑ i, f i) (hx : ∀ i, f i ∈ h.massWeightSubmodule i.val)
    (hgauge : ∀ g, repGauge g x = x) (hlorentz : ∀ Λ, repLorentz Λ x = x) :
    ∀ i, (∀ g, repGauge g (f i) = f i) ∧ ∀ Λ, repLorentz Λ (f i) = f i
```

2.415 PhyslibAlpha.Particles.StandardModel.CovRealization.MassWeight.Filtration

PhyslibAlpha/Particles/StandardModel/CovRealization/MassWeight/Filtration.lean on GitHub.

def massWeightSubmoduleLE [source]

The elements of the field algebra of mass weight at most `w`: the join of the mass-weight submodules of weight `0` through `w`. This is where a Lagrangian lives, a sum of terms of every mass dimension up to a cut-off rather than of a single one.

```
noncomputable def massWeightSubmoduleLE (w : ℕ) : Submodule ℂ B
```

lemma massWeightSubmodule_le_massWeightSubmoduleLE [source]

Each graded piece of weight at most `w` sits inside the filtration at `w`.

```
lemma massWeightSubmodule_le_massWeightSubmoduleLE {k w : ℕ} (hk : k ≤ w) :
    h.massWeightSubmodule k ≤ h.massWeightSubmoduleLE w
```

lemma mem_massWeightSubmoduleLE [source]

An element of a graded piece of weight at most `w` lies in the filtration at `w`.

```
lemma mem_massWeightSubmoduleLE {k w : ℕ} (hk : k ≤ w) {x : B}
    (hx : x ∈ h.massWeightSubmodule k) : x ∈ h.massWeightSubmoduleLE w
```

lemma massWeightSubmoduleLE_le [source]

A submodule containing every graded piece of weight at most `w` contains the filtration at `w`: the join is taken over exactly those pieces.

```
lemma massWeightSubmoduleLE_le {w : ℕ} {V : Submodule ℂ B}
    (hV : ∀ k ≤ w, h.massWeightSubmodule k ≤ V) : h.massWeightSubmoduleLE w ≤ V
```

lemma massWeightSubmoduleLE_mono [source]

The filtration grows with the bound.

```
lemma massWeightSubmoduleLE_mono {w w' : ℕ} (hw : w ≤ w') :
    h.massWeightSubmoduleLE w ≤ h.massWeightSubmoduleLE w'
```

lemma massWeightSubmoduleLE_eq_iSup [source]

The filtration as a join over a finite index type, which is the form in which the reduction of a join consumes it.

```
lemma massWeightSubmoduleLE_eq_iSup (w : ℕ) :
    h.massWeightSubmoduleLE w = ⨆ k : Fin (w + 1), h.massWeightSubmodule (k : ℕ)
```

lemma isStableUnder_massWeightSubmoduleLE [source]

The filtration is carried into itself by both groups: each graded piece is, and a join of stable submodules is stable.

```
lemma isStableUnder_massWeightSubmoduleLE (w : ℕ) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.massWeightSubmoduleLE w)
```

lemma massWeightSubmodule_zero [source]

The weight-zero part of the field algebra is the scalars. Every covariant generator carries positive mass weight, so the only word of total weight zero is the empty one, whose value is the unit. This is the constant term of the Lagrangian, of mass dimension zero — the cosmological term.

```
lemma massWeightSubmodule_zero : h.massWeightSubmodule 0 = 1
```

lemma isFixedBy_massWeightSubmodule_zero [source]

The scalars are fixed by both groups: each acts by an algebra map, so each fixes the unit. This is what makes the constant term an invariant, and it is what the backward direction of the classification needs of the weight-zero part of the span.

```
lemma isFixedBy_massWeightSubmodule_zero :
    IsFixedBy (gaugeLorentzMaps repGauge repLorentz) (h.massWeightSubmodule 0)
```

def standardModelSpanLE [source]

The gauge- and Lorentz-invariant content of the Standard Model up to mass weight `w`: the weight-zero part of the field algebra, which is the constant term, joined with the Standard-Model span of every weight up to `w`.

```
noncomputable def standardModelSpanLE (w : ℕ) : Submodule ℂ B
```

lemma massWeightSubmodule_zero_le_standardModelSpanLE [source]

The constant term lies in the span of the filtration, at every bound.

```
lemma massWeightSubmodule_zero_le_standardModelSpanLE (w : ℕ) :
    h.massWeightSubmodule 0 ≤ h.standardModelSpanLE w
```

lemma standardModelSpan_le_standardModelSpanLE [source]

The graded span of a weight at most `w` lies in the span of the filtration at `w`.

```
lemma standardModelSpan_le_standardModelSpanLE {k w : ℕ} (hk : k ≤ w) :
    h.standardModelSpan k ≤ h.standardModelSpanLE w
```

lemma standardModelSpanLE_le_massWeightSubmoduleLE [source]

The span of the filtration at `w` has mass weight at most `w`: the constant term has weight zero and each graded span has its own weight.

```
lemma standardModelSpanLE_le_massWeightSubmoduleLE (w : ℕ) :
    h.standardModelSpanLE w ≤ h.massWeightSubmoduleLE w
```

lemma isFixedBy_standardModelSpanLE [source]

The span of the filtration is fixed pointwise by the gauge and Lorentz groups together. At positive weight this is the fixedness of the graded spans; at weight zero it is the fixedness of the unit, and that is the only new content of the filtration.

```
lemma isFixedBy_standardModelSpanLE (w : ℕ) :
    IsFixedBy (gaugeLorentzMaps repGauge repLorentz) (h.standardModelSpanLE w)
```

lemma repGauge_of_mem_standardModelSpanLE [source]

Every element of the span of the filtration is a gauge invariant.

```
lemma repGauge_of_mem_standardModelSpanLE (w : ℕ) (g : GaugeGroupI) {y : B}
    (hy : y ∈ h.standardModelSpanLE w) : repGauge g y = y
```

lemma repLorentz_of_mem_standardModelSpanLE [source]

Every element of the span of the filtration is a Lorentz invariant.

```
lemma repLorentz_of_mem_standardModelSpanLE (w : ℕ) (Λ : SL(2,ℂ)) {y : B}
    (hy : y ∈ h.standardModelSpanLE w) : repLorentz Λ y = y
```

lemma standardModelSpanLE_eight [source]

The span of the filtration at bound eight in reduced form. Of the nine graded spans only two are non-trivial, the Higgs mass term at weight four and the dimension-four Lagrangian at weight eight, so what survives up to mass dimension four is the constant term, the Higgs mass term, and the Lagrangian.

```
lemma standardModelSpanLE_eight :
    h.standardModelSpanLE 8 = (1 : Submodule ℂ B) ⊔ (h.isHiggsSector.dotSpan 0 0
      ⊔ (h.isGaugeSector.lorentzContractionEightSpan
            ⊔ h.isHiggsSector.lorentzContractionEightSpan
          ⊔ (h.isFermionSector.kineticSpan ⊔ h.yukawaSpan)))
```

lemma reducesInvariantsTo_massWeightSubmoduleLE [source]

The filtration reduces to its span, at every bound up to eight. The filtration is a join of the graded pieces, each of them stable under both groups, and `ReducesInvariantsTo` is closed under joins in its source: the weights are taken one at a time, each in turn joining the error term of the others. At positive weight the graded reduction of `Invariants.lean` is used; at weight zero a submodule reduces to itself, the constant term being carried in the span.

```
lemma reducesInvariantsTo_massWeightSubmoduleLE {w : ℕ} (hw : w ≤ 8) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.massWeightSubmoduleLE w)
      (h.standardModelSpanLE w)
```

lemma mem_massWeightSubmoduleLE_sup_and_gauge_lorentz_invariant_iff [source]

The classification of the Standard Model up to mass weight `w ≤ 8` as an equivalence, in the shape every sector uses: an element of `massWeightSubmoduleLE w ⊔ S`, with `S` stable under both groups, is fixed by both groups exactly when it is a combination of the constant term and the Standard-Model terms of weight at most `w`, up to a remainder in `S` fixed by both groups.

```
lemma mem_massWeightSubmoduleLE_sup_and_gauge_lorentz_invariant_iff (w : ℕ) (hw : w ≤ 8)
    (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmoduleLE w ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ h.standardModelSpanLE w
```

lemma exists_mem_standardModelSpanLE_of_gauge_and_lorentz_invariant [source]

The gauge and Lorentz invariants of mass weight at most `w`, for `w ≤ 8`, modulo a submodule `S` stable under both groups: such an invariant is a combination of the constant term and the Standard-Model terms of weight at most `w`, plus a remainder in `S`, and the remainder is fixed by both groups as well, being the difference of two invariants.

```
lemma exists_mem_standardModelSpanLE_of_gauge_and_lorentz_invariant (w : ℕ) (hw : w ≤ 8)
    (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
```

```
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.massWeightSubmoduleLE w ⊔ S)
    (hG : ∀ g : GaugeGroupI, repGauge g x = x)
    (hL : ∀ g : SL(2,ℂ), repLorentz g x = x) :
    ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
      ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
      ∧ x - y ∈ h.standardModelSpanLE w
```

lemma mem_massWeightSubmoduleLE_sup_and_gauge_lorentz_invariant_iff_mem [source]

The same classification without the existential: at every bound up to eight an element of `massWeightSubmoduleLE w ⊔ S` fixed by both groups is an element of the span of the filtration joined with `S` fixed by both groups, and conversely.

```
lemma mem_massWeightSubmoduleLE_sup_and_gauge_lorentz_invariant_iff_mem (w : ℕ)
    (hw : w ≤ 8) (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmoduleLE w ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ (x ∈ h.standardModelSpanLE w ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
          ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
```

lemma mem_massWeightSubmoduleLE_eight_sup_and_gauge_lorentz_invariant_iff [source]

The classification at bound eight, that is at mass dimension at most four, as an equivalence: an element of `massWeightSubmoduleLE 8 ⊔ S`, with `S` stable under both groups, is fixed by both groups exactly when it lies in the span of the filtration up to a remainder in `S` fixed by both groups.

```
lemma mem_massWeightSubmoduleLE_eight_sup_and_gauge_lorentz_invariant_iff
    (S : Submodule ℂ B) (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmoduleLE 8 ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ h.standardModelSpanLE 8
```

lemma mem_massWeightSubmoduleLE_eight_sup_and_gauge_lorentz_invariant_iff_mem [source]

The same at bound eight without the existential.

```
lemma mem_massWeightSubmoduleLE_eight_sup_and_gauge_lorentz_invariant_iff_mem
    (S : Submodule ℂ B) (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmoduleLE 8 ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ (x ∈ h.standardModelSpanLE 8 ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
          ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
```

lemma mem_massWeightSubmoduleLE_eight_sup_and_gauge_lorentz_invariant_iff_lagrangian [source]

The invariant content of the Standard Model up to mass dimension four. An element of `massWeightSubmoduleLE 8 ⊔ S`, for `S` a submodule stable under both groups,

is fixed by the gauge group and the Lorentz group exactly when it is a combination of the constant term, of mass dimension zero, the Higgs mass term `H† H`, of mass dimension two (`HiggsAlgebraCovRealization.dotSpan`), and the dimension-four span — the four Lorentz contractions of the three `F·F` trace families and of the twice-derived hypercharge field strength, among them the gauge kinetic and theta terms (`IsGaugeSector.lorentzContractionEightSpan`), the Higgs kinetic term, its quartic potential and its two box terms (`HiggsAlgebraCovRealization.lorentzContractionEightSpan`), the kinetic terms of the ten fermion species over the nine family pairs (`IsFermionSector.kineticSpan`), and the six Yukawa couplings over the nine family pairs (`yukawaSpan`) — up to a remainder in `S` fixed by both groups, and nothing else. The generators span; they are not shown to be independent, and no quotient by total derivatives is taken.

```
lemma mem_massWeightSubmoduleLE_eight_sup_and_gauge_lorentz_invariant_iff_lagrangian
    (S : Submodule ℂ B) (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmoduleLE 8 ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ (1 : Submodule ℂ B) ⊔ (h.isHiggsSector.dotSpan 0 0
              ⊔ (h.isGaugeSector.lorentzContractionEightSpan
                    ⊔ h.isHiggsSector.lorentzContractionEightSpan
                ⊔ (h.isFermionSector.kineticSpan ⊔ h.yukawaSpan)))
```

2.416 `PhyslibAlpha.Particles.StandardModel.CovRealization.MassWeight.Invariants`

`PhyslibAlpha/Particles/StandardModel/CovRealization/MassWeight/Invariants.lean` on GitHub.

`def standardModelSpan` [source]

The gauge- and Lorentz-invariant content of the Standard Model at mass weight `w`: the join of the surviving spans of the eight sectors. At weight eight it is the gauge sector's four Lorentz contractions of each of its four families, two of which already span for each family (`IsGaugeSector.lorentzContractionEightSpan_eq`), and the two of the twice-derived hypercharge field strength vanish, leaving the two contractions of each of the three trace families (`isGaugeSector_lorentzContractionEightSpan_eq_traceFamilies`): the kinetic-shape and theta-shape contractions of the three gauge groups. Together with these come the Higgs sector's two box terms, kinetic term and quartic potential, the fermion sector's ten kinetic terms over the nine family pairs, and the six Yukawa couplings over the nine family pairs. Below weight eight only the Higgs sector survives, and only at weight four, where it contributes the Higgs mass term.

```
noncomputable def standardModelSpan (w : ℕ) : Submodule ℂ B
```

`lemma standardModelSpan_eight` [source]

At mass weight eight the span is the gauge, Higgs, fermion and Yukawa spans together.

```
lemma standardModelSpan_eight :
    h.standardModelSpan 8 = h.isGaugeSector.lorentzContractionEightSpan
      ⊔ h.isHiggsSector.lorentzContractionEightSpan
    ⊔ (h.isFermionSector.kineticSpan ⊔ h.yukawaSpan)
```

lemma standardModelSpan_four [source]

At mass weight four the span is the line through the Higgs mass term `H† H`, the one invariant of the Standard Model below mass dimension four.

```
lemma standardModelSpan_four : h.standardModelSpan 4 = h.isHiggsSector.dotSpan 0 0
```

lemma standardModelSpan_eq_bot [source]

At every mass weight other than four and eight the span is trivial: apart from the Higgs mass term there is no Standard-Model term below mass dimension four.

```
lemma standardModelSpan_eq_bot {w : ℕ} (hw : w ≠ 8) (hw4 : w ≠ 4) :
    h.standardModelSpan w = ⊥
```

lemma hyperchargeDeriv_comp_swap_zero_one [source]

The twice-derived hypercharge field strengths are symmetric in their two derivative slots.

```
lemma hyperchargeDeriv_comp_swap_zero_one (d : IsGaugeSector.EightIdx) :
    h.isGaugeSector.hyperchargeDeriv (d ∘ Equiv.swap 0 1) = h.isGaugeSector.hyperchargeDeriv d
```

lemma hyperchargeDeriv_contractionTensor_one [source]

`η^{μρ} η^{νσ} ∇_μ ∇_ν B_{ρσ} = 0`: a symmetric pair of derivatives contracted against the antisymmetric pair of field-strength indices.

```
lemma hyperchargeDeriv_contractionTensor_one :
    ofComponents h.isGaugeSector.hyperchargeDeriv (RankFour.contractionTensor 1) = 0
```

lemma hyperchargeDeriv_contractionTensor_three [source]

`ε^{μνρσ} ∇_μ ∇_ν B_{ρσ} = 0`: a symmetric pair of derivatives contracted against the antisymmetric symbol.

```
lemma hyperchargeDeriv_contractionTensor_three :
    ofComponents h.isGaugeSector.hyperchargeDeriv (RankFour.contractionTensor 3) = 0
```

lemma isGaugeSector_lorentzContractionEightSpan_eq_traceFamilies [source]

The gauge contribution at mass weight eight is spanned by the Lorentz contractions of the three trace families alone: for each of `gluonTrace`, `wTrace` and `hyperchargeTrace`, the `(0,2)(1,3)` metric contraction `η^{μρ} η^{νσ} F_{μν} F_{ρσ}` and the Levi-Civita contraction `ε^{μνρσ} F_{μν} F_{ρσ}`, as plain component sums. The two twice-derived hypercharge generators of `IsGaugeSector.lorentzContractionEightSpan_eq` vanish. The six span; they are not shown to be independent or nonzero.

```
lemma isGaugeSector_lorentzContractionEightSpan_eq_traceFamilies :
    h.isGaugeSector.lorentzContractionEightSpan
      = Submodule.span ℂ {ofComponents h.isGaugeSector.gluonTrace (contractionTensor 1),
          ofComponents h.isGaugeSector.gluonTrace (contractionTensor 3)}
        ⊔ (Submodule.span ℂ {ofComponents h.isGaugeSector.wTrace (contractionTensor 1),
            ofComponents h.isGaugeSector.wTrace (contractionTensor 3)}
```

```
        ⊔ Submodule.span ℂ {ofComponents h.isGaugeSector.hyperchargeTrace (contractionTensor
  1),
          ofComponents h.isGaugeSector.hyperchargeTrace (contractionTensor 3)})
```

lemma isGaugeSector_massWeightSubmodule_le [source]

At a non-zero weight the gauge sector's mass-weight submodule sits inside the covariant model's, being the `{gauge}` piece of the sector decomposition there.

```
lemma isGaugeSector_massWeightSubmodule_le {w : ℕ} (hw : w ≠ 0) :
    h.isGaugeSector.massWeightSubmodule w ≤ h.massWeightSubmodule w
```

lemma isHiggsSector_massWeightSubmodule_le [source]

At a non-zero weight the Higgs sector's mass-weight submodule sits inside the covariant model's.

```
lemma isHiggsSector_massWeightSubmodule_le {w : ℕ} (hw : w ≠ 0) :
    h.isHiggsSector.massWeightSubmodule w ≤ h.massWeightSubmodule w
```

lemma isFermionSector_massWeightSubmodule_le [source]

At a non-zero weight the fermion sector's mass-weight submodule sits inside the covariant model's.

```
lemma isFermionSector_massWeightSubmodule_le {w : ℕ} (hw : w ≠ 0) :
    h.isFermionSector.massWeightSubmodule w ≤ h.massWeightSubmodule w
```

lemma standardModelSpan_le_massWeightSubmodule [source]

The span at weight `w` has mass weight `w`: each of its contributions is a combination of words of that weight.

```
lemma standardModelSpan_le_massWeightSubmodule (w : ℕ) :
    h.standardModelSpan w ≤ h.massWeightSubmodule w
```

lemma isFixedBy_standardModelSpan [source]

The span at weight `w` is fixed pointwise by the gauge and Lorentz groups together: every one of its contributions is a span of invariants. This is the easy direction of the classification, and it is also what supplies the stability the reduction asks of its target.

```
lemma isFixedBy_standardModelSpan (w : ℕ) :
    IsFixedBy (gaugeLorentzMaps repGauge repLorentz) (h.standardModelSpan w)
```

lemma repGauge_of_mem_standardModelSpan [source]

Every element of the span at weight `w` is a gauge invariant.

```
lemma repGauge_of_mem_standardModelSpan (w : ℕ) (g : GaugeGroupI) {y : B}
    (hy : y ∈ h.standardModelSpan w) : repGauge g y = y
```

lemma repLorentz_of_mem_standardModelSpan [source]

Every element of the span at weight `w` is a Lorentz invariant.

```
lemma repLorentz_of_mem_standardModelSpan (w : ℕ) (Λ : SL(2,ℂ)) {y : B}
    (hy : y ∈ h.standardModelSpan w) : repLorentz Λ y = y
```

lemma reducesInvariantsTo_sectorMassWeight_empty [source]

The empty sector reduces to the span: away from weight zero it is trivial, its only word being the empty one.

```
lemma reducesInvariantsTo_sectorMassWeight_empty {w : ℕ} (hw : w ≠ 0) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.sectorMassWeight ∅ w)
      (h.standardModelSpan w)
```

lemma reducesInvariantsTo_sectorMassWeight_gauge [source]

The gauge sector reduces to the span: at weight eight to the four Lorentz contractions of its four families, two of each spanning, below it to nothing at all.

```
lemma reducesInvariantsTo_sectorMassWeight_gauge {w : ℕ} (hw0 : 0 < w) (hw : w ≤ 8) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz)
      (h.sectorMassWeight {GeneratorClass.gauge} w) (h.standardModelSpan w)
```

lemma reducesInvariantsTo_sectorMassWeight_higgs [source]

The Higgs sector reduces to the span: at weight eight to the two box terms, the kinetic term and the quartic potential, at weight four to the Higgs mass term, and elsewhere to nothing.

```
lemma reducesInvariantsTo_sectorMassWeight_higgs {w : ℕ} (hw0 : 0 < w) (hw : w ≤ 8) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz)
      (h.sectorMassWeight {GeneratorClass.higgs} w) (h.standardModelSpan w)
```

lemma reducesInvariantsTo_sectorMassWeight_fermion [source]

The fermion sector reduces to the span: at weight eight to the ten kinetic terms over the nine family pairs, below it to nothing — there is no Dirac mass term.

```
lemma reducesInvariantsTo_sectorMassWeight_fermion {w : ℕ} (hw0 : 0 < w) (hw : w ≤ 8) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz)
      (h.sectorMassWeight {GeneratorClass.fermion} w) (h.standardModelSpan w)
```

lemma reducesInvariantsTo_sectorMassWeight_higgs_fermion [source]

The Yukawa sector reduces to the span: at weight eight to the six Yukawa couplings over the nine family pairs, below it to nothing.

```
lemma reducesInvariantsTo_sectorMassWeight_higgs_fermion {w : ℕ} (hw : w ≤ 8) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz)
      (h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} w)
      (h.standardModelSpan w)
```

lemma reducesInvariantsTo_sectorMassWeight_gauge_higgs [source]

The gauge-Higgs sector reduces to nothing: it carries no Lorentz invariant below weight nine.

```
lemma reducesInvariantsTo_sectorMassWeight_gauge_higgs {w : ℕ} (hw : w ≤ 8) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz)
      (h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs} w)
      (h.standardModelSpan w)
```

lemma reducesInvariantsTo_sectorMassWeight_gauge_fermion [source]

The gauge-fermion sector reduces to nothing: it carries no Lorentz invariant below weight nine.

```
lemma reducesInvariantsTo_sectorMassWeight_gauge_fermion {w : ℕ} (hw : w ≤ 8) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz)
      (h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.fermion} w)
      (h.standardModelSpan w)
```

lemma reducesInvariantsTo_sectorMassWeight_mixed [source]

The mixed sector reduces to nothing: it is trivial below weight nine.

```
lemma reducesInvariantsTo_sectorMassWeight_mixed {w : ℕ} (hw : w ≤ 8) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz)
      (h.sectorMassWeight
        {GeneratorClass.gauge, GeneratorClass.higgs, GeneratorClass.fermion} w)
      (h.standardModelSpan w)
```

lemma isStableUnder_sectorMassWeight [source]

Every weight part of every sector is carried into itself by both groups: the stability the join of the reductions asks of its summands.

```
lemma isStableUnder_sectorMassWeight (T : Finset GeneratorClass) (w : ℕ) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.sectorMassWeight T w)
```

lemma reducesInvariantsTo_sectorMassWeight [source]

Every sector reduces to the span, at every weight from one to eight. The three constructors of `GeneratorClass` give eight class sets, and section C treats each.

```
lemma reducesInvariantsTo_sectorMassWeight {w : ℕ} (hw0 : 0 < w) (hw : w ≤ 8)
    (T : Finset GeneratorClass) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.sectorMassWeight T w)
      (h.standardModelSpan w)
```

lemma reducesInvariantsTo_massWeightSubmodule [source]

The whole weight-`w` submodule reduces to the span, for `w` from one to eight. The mass-weight submodule is the join of the eight sectors' weight-`w` parts, each of them stable under both groups, and `ReducesInvariantsTo` is closed under joins in its source: the sectors are taken one at a time, each in turn joining the error term of the others. No independence of the sectors is used, and none is available.

```
lemma reducesInvariantsTo_massWeightSubmodule {w : ℕ} (hw0 : 0 < w) (hw : w ≤ 8) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.massWeightSubmodule w)
      (h.standardModelSpan w)
```

lemma mem_massWeightSubmodule_sup_and_gauge_lorentz_invariant_iff [source]

The classification of the Standard Model at mass dimension at most four as an equivalence, in the shape every sector uses: an element of `massWeightSubmodule w ⊔ S` for `0 < w ≤ 8`, with `S` stable under both groups, is fixed by both groups exactly when it is a combination of the Standard-Model terms of weight `w` up to a remainder in `S` fixed by both groups. Forwards this is the reduction of section D; backwards it uses that the span is made of invariants of weight `w`, section B. The weight-four Higgs mass term is what makes the span, rather than the bare equation `x = y`, the right form of the statement.

```
lemma mem_massWeightSubmodule_sup_and_gauge_lorentz_invariant_iff (w : ℕ) (hw0 : 0 < w)
    (hw : w ≤ 8) (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule w ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ h.standardModelSpan w
```

lemma exists_mem_standardModelSpan_of_gauge_and_lorentz_invariant [source]

The gauge and Lorentz invariants of mass weight `w` for `0 < w ≤ 8`, modulo a submodule `S` stable under both groups: such an invariant is a combination of the Standard-Model terms of that weight plus a remainder in `S`, and the remainder is fixed by both groups as well, being the difference of two invariants.

```
lemma exists_mem_standardModelSpan_of_gauge_and_lorentz_invariant (w : ℕ)
    (hw0 : 0 < w) (hw : w ≤ 8) (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.massWeightSubmodule w ⊔ S)
    (hG : ∀ g : GaugeGroupI, repGauge g x = x)
    (hL : ∀ g : SL(2,ℂ), repLorentz g x = x) :
    ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
      ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
      ∧ x - y ∈ h.standardModelSpan w
```

lemma mem_massWeightSubmodule_sup_and_gauge_lorentz_invariant_iff_mem [source]

The same classification without the existential: at every weight from one to eight an element of `massWeightSubmodule w ⊔ S` fixed by both groups is an element of the Standard-Model span joined with `S` fixed by both groups, and conversely.

```
lemma mem_massWeightSubmodule_sup_and_gauge_lorentz_invariant_iff_mem (w : ℕ)
    (hw0 : 0 < w) (hw : w ≤ 8) (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule w ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
```

```
    ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
  ↔ (x ∈ h.standardModelSpan w ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
    ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
```

lemma `mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff_lagrangian` [source]

The invariant content of the Standard Model at mass dimension four. An element of `massWeightSubmodule 8 ⊔ S`, for `S` a submodule stable under both groups, is fixed by the gauge group and the Lorentz group exactly when it is a combination of the four Lorentz contractions of the three `F·F` trace families and of the twice-derived hypercharge field strength (`IsGaugeSector.lorentzContractionEightSpan`), two of each spanning by `IsGaugeSector.lorentzContractionEightSpan_eq`: the kinetic-shape and theta-shape contractions of the three gauge groups and two contractions of the twice-derived hypercharge field strength, which vanish (`mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_`
the Higgs kinetic term, its quartic potential and its two box terms (`HiggsAlgebraCovRealization.lorentzCon`
the kinetic terms of the ten fermion species over the nine family pairs (`IsFermionSector.kineticSpan`),
and the six Yukawa couplings over the nine family pairs (`yukawaSpan`), up to a remainder in `S` fixed by both groups — and nothing else. The generators span; they are not shown to be independent, and no quotient by total derivatives is taken.

```
lemma mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff_lagrangian
    (S : Submodule ℂ B) (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule 8 ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ h.isGaugeSector.lorentzContractionEightSpan
                ⊔ h.isHiggsSector.lorentzContractionEightSpan
              ⊔ (h.isFermionSector.kineticSpan ⊔ h.yukawaSpan)
```

lemma `mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff_lagrangian_traceFamilies` [source]

The invariant content of the Standard Model at mass dimension four with the gauge contribution written through the Lorentz contractions of the three trace families alone: the `(0,2)(1,3)` metric contraction and the Levi-Civita contraction of each of `gluonTrace`, `wTrace` and `hyperchargeTrace` (`isGaugeSector_lorentzContractionEightSpan_eq_traceFamilie`
The Higgs, fermion and Yukawa contributions are as in `mem_massWeightSubmodule_eight_sup_and_gauge_lo`
The generators span; they are not shown to be independent or nonzero.

```
lemma
    mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff_lagrangian_traceFamilies
    (S : Submodule ℂ B) (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule 8 ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ (Submodule.span ℂ
                {ofComponents h.isGaugeSector.gluonTrace (contractionTensor 1),
                  ofComponents h.isGaugeSector.gluonTrace (contractionTensor 3)}
              ⊔ (Submodule.span ℂ {ofComponents h.isGaugeSector.wTrace (contractionTensor 1),
                  ofComponents h.isGaugeSector.wTrace (contractionTensor 3)}
                ⊔ Submodule.span ℂ
```

```
                {ofComponents h.isGaugeSector.hyperchargeTrace (contractionTensor 1),
                  ofComponents h.isGaugeSector.hyperchargeTrace (contractionTensor 3)}))
              ⊔ h.isHiggsSector.lorentzContractionEightSpan
            ⊔ (h.isFermionSector.kineticSpan ⊔ h.yukawaSpan)
```

lemma mem_of_gauge_and_lorentz_invariant_massWeightSubmodule_sup [source]

Below mass dimension two there is nothing at all, and at mass dimension two only the Higgs mass term: at every weight from one to seven other than four an element of `massWeightSubmodule w ⊔ S` fixed by both groups already lies in `S`.

```
lemma mem_of_gauge_and_lorentz_invariant_massWeightSubmodule_sup (w : ℕ) (hw0 : 0 < w)
    (hw : w < 8) (hw4 : w ≠ 4) (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.massWeightSubmodule w ⊔ S)
    (hG : ∀ g : GaugeGroupI, repGauge g x = x)
    (hL : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_massWeightSubmodule_four_sup_and_gauge_lorentz_invariant_iff_higgsMass [source]

At mass dimension two the only invariant of the Standard Model is the Higgs mass term `H† H`: an element of `massWeightSubmodule 4 ⊔ S` fixed by both groups is a multiple of it up to a remainder in `S` fixed by both groups.

```
lemma mem_massWeightSubmodule_four_sup_and_gauge_lorentz_invariant_iff_higgsMass
    (S : Submodule ℂ B) (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule 4 ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ h.isHiggsSector.dotSpan 0 0
```

2.417 PhyslibAlpha.Particles.StandardModel.CovRealization.MixedSector.Basic

PhyslibAlpha/Particles/StandardModel/CovRealization/MixedSector/Basic.lean on GitHub.

lemma sectorMassWeight_mixed_eq_bot_of_lt_nine [source]

The mixed sector vanishes below weight nine: a word realising all three classes carries gauge weight at least four, Higgs weight at least two and fermion weight at least three, for a total of at least nine — so no such word exists at a lower weight, and the sector's span there is trivial.

```
lemma sectorMassWeight_mixed_eq_bot_of_lt_nine {w : ℕ} (hw : w < 9) :
    h.sectorMassWeight
      {GeneratorClass.gauge, GeneratorClass.higgs, GeneratorClass.fermion} w = ⊥
```

lemma sectorMassWeight_mixed_eight [source]

The weight-eight mixed sector vanishes: the mass weight of a dimension-four Standard-Model term is at most eight, and the mixed sector is trivial there — no dimension-four term mixes gauge, Higgs and fermion fields together.

```
lemma sectorMassWeight_mixed_eight :
    h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs, GeneratorClass.fermion} 8
      = ⊥
```

lemma mem_of_invariant_sectorMassWeight_mixed_lt_nine_sup [source]

Below mass weight nine the mixed sector adds nothing to a submodule `S`: an element of `sectorMassWeight {gauge, higgs, fermion} w ⊔ S` for `w < 9` already lies in `S`. The sector is trivial there, so no invariance is asked of `x` and no stability of `S`.

```
lemma mem_of_invariant_sectorMassWeight_mixed_lt_nine_sup (w : ℕ) (hw : w < 9)
    (S : Submodule ℂ B) {x : B}
    (hx : x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs,
      GeneratorClass.fermion} w ⊔ S) : x ∈ S
```

lemma mem_sectorMassWeight_mixed_lt_nine_sup_and_gauge_lorentz_invariant_iff [source]

The classification below mass weight nine as an equivalence, in the shape of the gauge- and Yukawa-sector statements: an element of `sectorMassWeight {gauge, higgs, fermion} w ⊔ S` for `w < 9` is fixed by both groups exactly when it is itself an element of `S` fixed by both groups. Neither stability hypothesis on `S` is needed, the forward direction being the vanishing of the sector.

```
lemma mem_sectorMassWeight_mixed_lt_nine_sup_and_gauge_lorentz_invariant_iff (w : ℕ)
    (hw : w < 9) (S : Submodule ℂ B) (x : B) :
    (x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs,
          GeneratorClass.fermion} w ⊔ S
        ∧ (∀ g : GaugeGroupI, repGauge g x = x) ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x = y
```

lemma mem_sectorMassWeight_mixed_lt_nine_sup_and_gauge_lorentz_invariant_iff_mem [source]

The same classification without the existential: below mass weight nine an element of `sectorMassWeight {gauge, higgs, fermion} w ⊔ S` fixed by both groups is an element of `S` fixed by both groups, and conversely.

```
lemma mem_sectorMassWeight_mixed_lt_nine_sup_and_gauge_lorentz_invariant_iff_mem (w : ℕ)
    (hw : w < 9) (S : Submodule ℂ B) (x : B) :
    (x ∈ h.sectorMassWeight {GeneratorClass.gauge, GeneratorClass.higgs,
          GeneratorClass.fermion} w ⊔ S
        ∧ (∀ g : GaugeGroupI, repGauge g x = x) ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ (x ∈ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
          ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
```

2.418 PhyslibAlpha.Particles.StandardModel.CovRealization.Sectors

PhyslibAlpha/Particles/StandardModel/CovRealization/Sectors.lean on GitHub.

def sectorSubmodule [source]

The span of the words in the covariant generators realising exactly the classes `S`.

```
def sectorSubmodule (S : Finset GeneratorClass) : Submodule ℂ B
```

lemma mul_mem_sectorSubmodule [source]

Multiplication carries the class spans of `S` and `T` into that of `S ∪ T`.

```
lemma mul_mem_sectorSubmodule {S T : Finset GeneratorClass} {x y : B}
    (hx : x ∈ h.sectorSubmodule S) (hy : y ∈ h.sectorSubmodule T) :
    x * y ∈ h.sectorSubmodule (S ∪ T)
```

def sector [source]

The sector realising exactly the classes `S`: the span of the words whose generators realise `S`. It is a non-unital subalgebra — closed under multiplication since `S ∪ S = S`, but containing no non-zero scalar, since the empty word realises no class.

```
def sector (S : Finset GeneratorClass) : NonUnitalSubalgebra ℂ B
```

lemma mem_sector [source]

```
@[simp]
lemma mem_sector {S : Finset GeneratorClass} {x : B} :
    x ∈ h.sector S ↔ x ∈ h.sectorSubmodule S
```

lemma list_prod_mem_sector [source]

A word lies in the sector of the classes it realises.

```
lemma list_prod_mem_sector (gl : List Generators) :
    (gl.map h.generatorVal).prod ∈ h.sector (wordClasses gl)
```

lemma mul_mem_sector [source]

Multiplication carries the sectors of `S` and `T` into the sector of `S ∪ T`.

```
lemma mul_mem_sector {S T : Finset GeneratorClass} {x y : B}
    (hx : x ∈ h.sector S) (hy : y ∈ h.sector T) : x * y ∈ h.sector (S ∪ T)
```

lemma mem_fieldAlgebra_of_mem_sector [source]

Every sector sits inside the field algebra.

```
lemma mem_fieldAlgebra_of_mem_sector {S : Finset GeneratorClass} {x : B}
    (hx : x ∈ h.sector S) : x ∈ h.fieldAlgebra
```

lemma fieldAlgebra_eq_iSup_sector [source]

***The sectors exhaust the field algebra**: every element of the field algebra is a sum of elements of the sectors, since every word realises exactly one class set. The unit is supplied by `sector ∅`, the sector of the empty word, so the join is the whole of `fieldAlgebra` — read as a non-unital subalgebra, the two sides having otherwise different types.*

```
lemma fieldAlgebra_eq_iSup_sector :
    h.fieldAlgebra.toNonUnitalSubalgebra = ⨆ S : Finset GeneratorClass, h.sector S
```

lemma H_mem_sector [source]

Any Higgs tower symbol lies in the Higgs sector.

```
lemma H_mem_sector {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) : h.covH l φ ∈ h.sector {GeneratorClass.higgs}
```

lemma barH_mem_sector [source]

Any conjugate-Higgs tower symbol lies in the Higgs sector.

```
lemma barH_mem_sector {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    h.covBarH l φ ∈ h.sector {GeneratorClass.higgs}
```

lemma F_mem_sector [source]

Any field-strength tower symbol lies in the gauge sector.

```
lemma F_mem_sector {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) : h.covF l μ ν φ ∈ h.sector {GeneratorClass.gauge}
```

lemma d_mem_sector [source]

Any `d` tower symbol lies in the fermion sector.

```
lemma d_mem_sector (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ DownSinglet) : h.covD i l φ ∈ h.sector {GeneratorClass.fermion}
```

lemma bard_mem_sector [source]

Any `bard` tower symbol lies in the fermion sector.

```
lemma bard_mem_sector (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) : h.covBarD i l φ ∈ h.sector
      {GeneratorClass.fermion}
```

lemma u_mem_sector [source]

Any `u` tower symbol lies in the fermion sector.

```
lemma u_mem_sector (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ UpSinglet) : h.covU i l φ ∈ h.sector {GeneratorClass.fermion}
```

lemma baru_mem_sector [source]

Any `baru` tower symbol lies in the fermion sector.

```
lemma baru_mem_sector (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    h.covBarU i l φ ∈ h.sector {GeneratorClass.fermion}
```

lemma Q_mem_sector [source]

Any `Q` tower symbol lies in the fermion sector.

```
lemma Q_mem_sector (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ QuarkDoublet) : h.covQ i l φ ∈ h.sector {GeneratorClass.fermion}
```

lemma barQ_mem_sector [source]

Any `barQ` tower symbol lies in the fermion sector.

```
lemma barQ_mem_sector (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) : h.covBarQ i l φ ∈ h.sector
      {GeneratorClass.fermion}
```

lemma L_mem_sector [source]

Any `L` tower symbol lies in the fermion sector.

```
lemma L_mem_sector (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ LeptonDoubletV) : h.covL i l φ ∈ h.sector {GeneratorClass.fermion}
```

lemma barL_mem_sector [source]

Any `barL` tower symbol lies in the fermion sector.

```
lemma barL_mem_sector (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) : h.covBarL i l φ ∈ h.sector
      {GeneratorClass.fermion}
```

lemma e_mem_sector [source]

Any `e` tower symbol lies in the fermion sector.

```
lemma e_mem_sector (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ LeptonSinglet) : h.covE i l φ ∈ h.sector {GeneratorClass.fermion}
```

lemma bare_mem_sector [source]

Any `bare` tower symbol lies in the fermion sector.

```
lemma bare_mem_sector (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) : h.covBarE i l φ ∈ h.sector
      {GeneratorClass.fermion}
```

lemma repGauge_generatorVal_mem_sector [source]

The gauge action carries a covariant generator into the sector of its class.

```
lemma repGauge_generatorVal_mem_sector (g : GaugeGroupI) (a : Generators) :
    repGauge g (h.generatorVal a) ∈ h.sector {a.kind}
```

lemma repLorentz_generatorVal_mem_sector [source]

The Lorentz action carries a covariant generator into the sector of its class.

```
lemma repLorentz_generatorVal_mem_sector (Λ : SL(2,ℂ)) (a : Generators) :
    repLorentz Λ (h.generatorVal a) ∈ h.sector {a.kind}
```

lemma repGauge_mem_sector [source]

The action `repGauge` preserves every sector.

```
lemma repGauge_mem_sector {S : Finset GeneratorClass} {x : B} (g : GaugeGroupI)
    (hx : x ∈ h.sector S) : repGauge g x ∈ h.sector S
```

lemma repLorentz_mem_sector [source]

The action `repLorentz` preserves every sector.

```
lemma repLorentz_mem_sector {S : Finset GeneratorClass} {x : B} (Λ : SL(2,ℂ))
    (hx : x ∈ h.sector S) : repLorentz Λ x ∈ h.sector S
```

def sectorMassWeight [source]

The span of the words realising exactly the classes `S` of total mass weight `w`.

```
def sectorMassWeight (S : Finset GeneratorClass) (w : ℕ) : Submodule ℂ B
```

lemma sectorMassWeight_le_sectorSubmodule [source]

```
lemma sectorMassWeight_le_sectorSubmodule (S : Finset GeneratorClass) (w : ℕ) :
    h.sectorMassWeight S w ≤ h.sectorSubmodule S
```

lemma sectorMassWeight_le_massWeightSubmodule [source]

```
lemma sectorMassWeight_le_massWeightSubmodule (S : Finset GeneratorClass) (w : ℕ) :
    h.sectorMassWeight S w ≤ h.massWeightSubmodule w
```

lemma list_prod_mem_sectorMassWeight [source]

A word lies in the weight part of its sector given by its total weight.

```
lemma list_prod_mem_sectorMassWeight (gl : List Generators) :
    (gl.map h.generatorVal).prod
      ∈ h.sectorMassWeight (wordClasses gl) ((gl.map Generators.weight).sum)
```

lemma mul_mem_sectorMassWeight [source]

Multiplication carries the weight-`w` part of the sector of `S` and the weight-`w'` part of the sector of `T` into the weight-`w + w'` part of the sector of `S ∪ T`.

```
lemma mul_mem_sectorMassWeight {S T : Finset GeneratorClass} {w w' : ℕ} {x y : B}
    (hx : x ∈ h.sectorMassWeight S w) (hy : y ∈ h.sectorMassWeight T w') :
    x * y ∈ h.sectorMassWeight (S ∪ T) (w + w')
```

lemma coeff_massWeightPoly_mem_sectorMassWeight [source]

Reading off the `X ^ w` coefficient of `massWeightPoly` sends the sector of `S` into its weight-`w` part — the projection onto the weight-`w` component, with no independence argument needed.

```
lemma coeff_massWeightPoly_mem_sectorMassWeight {S : Finset GeneratorClass} {x : B}
    (hx : x ∈ h.sector S) (w : ℕ) :
    (massWeightPoly x).coeff w ∈ h.sectorMassWeight S w
```

lemma sectorMassWeight_eq_inf [source]

The weight-`w` part of the sector of `S` is exactly the intersection of the sector with the mass-weight submodule.

```
lemma sectorMassWeight_eq_inf (S : Finset GeneratorClass) (w : ℕ) :
    h.sectorMassWeight S w = h.sectorSubmodule S ⊓ h.massWeightSubmodule w
```

lemma massWeightSubmodule_eq_iSup_sectorMassWeight [source]

***The decomposition of the mass-weight submodule into sectors**: the weight-`w` component of the field algebra is the join over the class sets `S` of the weight-`w` parts of the sectors, since every word realises exactly one class set. The empty class set contributes the scalars, at weight zero only.*

```
lemma massWeightSubmodule_eq_iSup_sectorMassWeight (w : ℕ) :
    h.massWeightSubmodule w = ⨆ S : Finset GeneratorClass, h.sectorMassWeight S w
```

lemma repGauge_mem_sectorMassWeight [source]

The action `repGauge` preserves the weight parts of every sector.

```
lemma repGauge_mem_sectorMassWeight {S : Finset GeneratorClass} {w : ℕ} {x : B}
    (g : GaugeGroupI) (hx : x ∈ h.sectorMassWeight S w) :
    repGauge g x ∈ h.sectorMassWeight S w
```

lemma repLorentz_mem_sectorMassWeight [source]

The action `repLorentz` preserves the weight parts of every sector.

```
lemma repLorentz_mem_sectorMassWeight {S : Finset GeneratorClass} {w : ℕ} {x : B}
    (Λ : SL(2,ℂ)) (hx : x ∈ h.sectorMassWeight S w) :
    repLorentz Λ x ∈ h.sectorMassWeight S w
```

lemma sectorMassWeight_empty_of_ne_zero [source]

At a non-zero weight the `∅` sector has no weight part: its only word is the empty word, of weight zero.

```
lemma sectorMassWeight_empty_of_ne_zero {w : ℕ} (hw : w ≠ 0) :
    h.sectorMassWeight ∅ w = ⊥
```

lemma higgsAlgebra_le_sup_sectorSubmodule [source]

The algebra generated by the Higgs towers decomposes into the `{higgs}` sector and the scalar `∅` sector.

```
lemma higgsAlgebra_le_sup_sectorSubmodule :
    Subalgebra.toSubmodule h.isHiggsSector.higgsAlgebra
      ≤ h.sectorSubmodule {GeneratorClass.higgs} ⊔ h.sectorSubmodule ∅
```

lemma sectorMassWeight_higgs_le [source]

The weight-`w` part of the `{higgs}` sector lies in the Higgs-sector mass-weight submodule: its words are products of Higgs towers of total weight `w`.

```
lemma sectorMassWeight_higgs_le (w : ℕ) :
    h.sectorMassWeight {GeneratorClass.higgs} w
      ≤ h.isHiggsSector.massWeightSubmodule w
```

lemma sectorMassWeight_higgs_eq [source]

***The Higgs-sector mass-weight submodules are the weight parts of the `{higgs}` sector**, at any non-zero weight. (At weight zero the Higgs-sector submodule also contains the scalars, which the sector decomposition files under the `∅` sector.)*

```
lemma sectorMassWeight_higgs_eq {w : ℕ} (hw : w ≠ 0) :
    h.sectorMassWeight {GeneratorClass.higgs} w
      = h.isHiggsSector.massWeightSubmodule w
```

lemma gaugeAlgebra_le_sup_sectorSubmodule [source]

The algebra generated by the field-strength towers decomposes into the `{gauge}` sector and the scalar `∅` sector.

```
lemma gaugeAlgebra_le_sup_sectorSubmodule :
    Subalgebra.toSubmodule h.isGaugeSector.gaugeAlgebra
      ≤ h.sectorSubmodule {GeneratorClass.gauge} ⊔ h.sectorSubmodule ∅
```

lemma sectorMassWeight_gauge_le [source]

The weight-`w` part of the `{gauge}` sector lies in the gauge sector's mass-weight submodule.

```
lemma sectorMassWeight_gauge_le (w : ℕ) :
    h.sectorMassWeight {GeneratorClass.gauge} w
      ≤ h.isGaugeSector.massWeightSubmodule w
```

lemma sectorMassWeight_gauge_eq [source]

***The gauge sector's mass-weight submodules are the weight parts of the `{gauge}` sector**, at any non-zero weight. (At weight zero the sector's submodule also contains the scalars, which the sector decomposition files under the `∅` sector.)*

```
lemma sectorMassWeight_gauge_eq {w : ℕ} (hw : w ≠ 0) :
    h.sectorMassWeight {GeneratorClass.gauge} w
      = h.isGaugeSector.massWeightSubmodule w
```

lemma fermionAlgebra_le_sup_sectorSubmodule [source]

The algebra generated by the fermion towers decomposes into the `{fermion}` sector and the scalar `∅` sector.

```
lemma fermionAlgebra_le_sup_sectorSubmodule :
    Subalgebra.toSubmodule h.isFermionSector.fermionAlgebra
      ≤ h.sectorSubmodule {GeneratorClass.fermion} ⊔ h.sectorSubmodule ∅
```

lemma sectorMassWeight_fermion_le [source]

The weight-`w` part of the `{fermion}` sector lies in the fermion sector's mass-weight submodule.

```
lemma sectorMassWeight_fermion_le (w : ℕ) :
    h.sectorMassWeight {GeneratorClass.fermion} w
      ≤ h.isFermionSector.massWeightSubmodule w
```

lemma sectorMassWeight_fermion_eq [source]

***The fermion sector's mass-weight submodules are the weight parts of the `{fermion}` sector**, at any non-zero weight. (At weight zero the sector's submodule also contains the scalars, which the sector decomposition files under the `∅` sector.)*

```
lemma sectorMassWeight_fermion_eq {w : ℕ} (hw : w ≠ 0) :
    h.sectorMassWeight {GeneratorClass.fermion} w
      = h.isFermionSector.massWeightSubmodule w
```

lemma generatorVal_mem_of_kind [source]

A single generator's value lies in any family of submodules dominating its own class's sector.

```
lemma generatorVal_mem_of_kind {c : GeneratorClass} {M : ℕ → Submodule ℂ B}
    (hM : ∀ w, h.sectorMassWeight {c} w ≤ M w) {g : Generators} (hg : g.kind = c) :
    h.generatorVal g ∈ M g.weight
```

lemma list_prod_mem_mul_of_forall_kind [source]

***The two-class word decomposition.** A word all of whose generators lie in one of two classes is a product of an element of weight `classWeight c₁` from the first class's family and an element of weight `classWeight c₂` from the second.*

```
lemma list_prod_mem_mul_of_forall_kind {c₁ c₂ : GeneratorClass} (hne : c₁ ≠ c₂)
    {M₁ M₂ : ℕ → Submodule ℂ B}
    (hM₁ : ∀ w, h.sectorMassWeight {c₁} w ≤ M₁ w)
    (hM₂ : ∀ w, h.sectorMassWeight {c₂} w ≤ M₂ w)
    (hone₁ : (1 : Submodule ℂ B) ≤ M₁ 0) (hone₂ : (1 : Submodule ℂ B) ≤ M₂ 0)
    (hmul₁ : ∀ a b, M₁ a * M₁ b ≤ M₁ (a + b))
    (hmul₂ : ∀ a b, M₂ a * M₂ b ≤ M₂ (a + b))
    (hcomm : ∀ a b, M₂ a * M₁ b ≤ M₁ b * M₂ a)
    (gl : List Generators) (hgl : ∀ g ∈ gl, g.kind = c₁ ∨ g.kind = c₂) :
    (gl.map h.generatorVal).prod
      ∈ M₁ (classWeight c₁ gl) * M₂ (classWeight c₂ gl)
```

lemma sectorMassWeight_pair_le [source]

***The two-class sector decomposition.** The weight-`w` piece of the sector of two classes is contained in the join, over the splittings of `w` into two non-zero parts, of the products of the two classes' mass-weight submodules.*

```
lemma sectorMassWeight_pair_le {c₁ c₂ : GeneratorClass} (hne : c₁ ≠ c₂)
    {M₁ M₂ : ℕ → Submodule ℂ B}
    (hM₁ : ∀ w, h.sectorMassWeight {c₁} w ≤ M₁ w)
    (hM₂ : ∀ w, h.sectorMassWeight {c₂} w ≤ M₂ w)
    (hone₁ : (1 : Submodule ℂ B) ≤ M₁ 0) (hone₂ : (1 : Submodule ℂ B) ≤ M₂ 0)
    (hmul₁ : ∀ a b, M₁ a * M₁ b ≤ M₁ (a + b))
    (hmul₂ : ∀ a b, M₂ a * M₂ b ≤ M₂ (a + b))
    (hcomm : ∀ a b, M₂ a * M₁ b ≤ M₁ b * M₂ a) (w : ℕ) :
    h.sectorMassWeight {c₁, c₂} w
      ≤ ⨆ (p : ℕ × ℕ) (_ : p.1 + p.2 = w) (_ : p.1 ≠ 0) (_ : p.2 ≠ 0), M₁ p.1 * M₂ p.2
```

lemma sector_invariant_of_iSupIndep [source]

An element of the weight-`w` submodule fixed by both actions is a sum of weight-`w` sector pieces, each of them fixed by both actions, provided the weight parts of the sectors are independent. Independence is what turns the two decompositions `x = ∑ s, f s` and `x = ∑ s, repGauge g (f s)` into an equality piece by piece; without it the pieces are not determined by their sum.

```
lemma sector_invariant_of_iSupIndep {w : ℕ}
    (hind : iSupIndep fun S : Finset GeneratorClass => h.sectorMassWeight S w)
    (x : B) (x_gauge_invariant : ∀ g, repGauge g x = x)
    (x_lorentz_invariant : ∀ g, repLorentz g x = x)
    (x_mass_dim : x ∈ h.massWeightSubmodule w) :
    ∃ f : Finset GeneratorClass → B,
      x = ∑ s, f s ∧ (∀ s, f s ∈ h.sectorMassWeight s w ∧
      (∀ g, repGauge g (f s) = (f s)) ∧ (∀ g, repLorentz g (f s) = (f s)))
```

2.419 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.Basic

PhyslibAlpha/Particles/StandardModel/CovRealization/YukawaSector/Basic.lean on GitHub.

lemma commute_of_mem_higgsAlgebra_of_mem_fermionAlgebra [source]

***The Higgs sector is bosonic**: every element of the algebra generated by the Higgs towers commutes with every element of the algebra generated by the fermion towers.*

```
lemma commute_of_mem_higgsAlgebra_of_mem_fermionAlgebra {x y : B}
    (hx : x ∈ h.isHiggsSector.higgsAlgebra) (hy : y ∈ h.isFermionSector.fermionAlgebra) :
    Commute x y
```

lemma fermionMassWeight_mul_higgsMassWeight_le [source]

The fermion-sector and Higgs-sector mass-weight submodules commute past each other, in the order needed by `sectorMassWeight_pair_le`.

```
lemma fermionMassWeight_mul_higgsMassWeight_le (a b : ℕ) :
    h.isFermionSector.massWeightSubmodule a * h.isHiggsSector.massWeightSubmodule b
      ≤ h.isHiggsSector.massWeightSubmodule b * h.isFermionSector.massWeightSubmodule a
```

lemma sectorMassWeight_higgs_fermion_le [source]

***The Yukawa-sector decomposition**: the weight-w piece of the `{higgs, fermion} sector is contained in the join, over the splittings of w` into two non-zero parts, of the products of the Higgs-sector and fermion-sector mass-weight submodules.*

```
lemma sectorMassWeight_higgs_fermion_le (w : ℕ) :
    h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} w
      ≤ ⨆ (p : ℕ × ℕ) (_ : p.1 + p.2 = w) (_ : p.1 ≠ 0) (_ : p.2 ≠ 0),
        h.isHiggsSector.massWeightSubmodule p.1 * h.isFermionSector.massWeightSubmodule p.2
```

lemma sectorMassWeight_higgs_fermion_eq_bot_of_lt_five [source]

***Below weight five, the Yukawa sector vanishes**: no splitting of a total weight under five into two non-zero parts survives — the Higgs part is either odd (hence zero) or equal to two, forcing the fermion part to be one or two (hence also zero).*

```
lemma sectorMassWeight_higgs_fermion_eq_bot_of_lt_five {w : ℕ} (hw : w < 5) :
    h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} w = ⊥
```

lemma sectorMassWeight_higgs_fermion_six [source]

***The Yukawa sector vanishes at weight six**: every splitting of six into two non-zero parts has either an odd Higgs part or a fermion part of weight two or four, all of which vanish.*

```
lemma sectorMassWeight_higgs_fermion_six :
    h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 6 = ⊥
```

lemma mul_le_sectorMassWeight_higgs_fermion [source]

A product of a Higgs-weight piece and a fermion-weight piece lands in the Yukawa sector of the total weight.

```
lemma mul_le_sectorMassWeight_higgs_fermion {a b w : ℕ} {X Y : Submodule ℂ B}
    (ha : a ≠ 0) (hb : b ≠ 0) (hab : a + b = w)
    (hX : X ≤ h.isHiggsSector.massWeightSubmodule a)
    (hY : Y ≤ h.isFermionSector.massWeightSubmodule b) :
    X * Y ≤ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} w
```

lemma sectorMassWeight_higgs_fermion_five [source]

***The weight-five Yukawa sector**: the only surviving splitting is the Higgs field itself (weight two) against the underived fermion towers (weight three).*

```
lemma sectorMassWeight_higgs_fermion_five :
    h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 5
      = h.isHiggsSector.derivSubmodule 0 * h.isFermionSector.derivSubmodule 0
```

lemma sectorMassWeight_higgs_fermion_seven [source]

***The weight-seven Yukawa sector**: the surviving splittings pair the Higgs field (weight two) with the once-derived fermion towers (weight five), or the once-derived Higgs field (weight four) with the underived fermion towers (weight three).*

```
lemma sectorMassWeight_higgs_fermion_seven :
    h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 7
      = h.isHiggsSector.derivSubmodule 0 * h.isFermionSector.derivSubmodule 1
        ⊔ h.isHiggsSector.derivSubmodule 1 * h.isFermionSector.derivSubmodule 0
        ⊔ h.isHiggsSector.derivSubmodule 0 * h.isHiggsSector.derivSubmodule 0
          * h.isFermionSector.derivSubmodule 0
```

lemma sectorMassWeight_higgs_fermion_eight [source]

***The weight-eight Yukawa sector**: the only surviving splitting pairs the Higgs field (weight two) with the product of two underived fermion towers (weight six) — this is the sector of the Yukawa term `H ψ ψ` itself.*

```
lemma sectorMassWeight_higgs_fermion_eight :
    h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
      = h.isHiggsSector.derivSubmodule 0
        * (h.isFermionSector.derivSubmodule 0 * h.isFermionSector.derivSubmodule 0)
```

2.420 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.Families.BarHiggs

PhyslibAlpha/Particles/StandardModel/CovRealization/YukawaSector/Families/BarHiggs.lean on GitHub.

def barDownBlock [source]

The components of the conjugate down-type Yukawa block `barH bard Q`: a conjugate Higgs symbol, a conjugate down-singlet symbol and a quark-doublet symbol, none carrying derivatives.

```
noncomputable def barDownBlock (f f' : Fin 3) (i sbd : Fin 2) (cbd : Fin 3) (sQ : Fin 2)
    (cQ : Fin 3) (wQ : Fin 2) : B
```

lemma isSU3FundamentalAntiFundamental_barDownBlock [source]

The two colour indices of the conjugate down-type block carry one fundamental and one anti-fundamental `su(3)` index, the conjugate down singlet supplying the fundamental one.

```
lemma isSU3FundamentalAntiFundamental_barDownBlock (f f' : Fin 3) (i sbd sQ wQ : Fin 2) :
    IsSU3FundamentalAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 3 => h.barDownBlock f f' i sbd (l 0) sQ (l 1) wQ)
```

lemma isSU2FundamentalAntiFundamental_barDownBlock [source]

The two isospin indices of the conjugate down-type block carry one fundamental and one anti-fundamental `su(2)` index, the conjugate Higgs symbol supplying the fundamental one.

```
lemma isSU2FundamentalAntiFundamental_barDownBlock (f f' : Fin 3) (sbd : Fin 2) (cbd : Fin 3)
    (sQ : Fin 2) (cQ : Fin 3) :
    IsSU2FundamentalAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 2 => h.barDownBlock f f' (l 0) sbd cbd sQ cQ (l 1))
```

lemma isBiDualLeftWeyl_barDownBlock [source]

The two spinor indices of the conjugate down-type block are both dual left-handed.

```
lemma isBiDualLeftWeyl_barDownBlock (f f' : Fin 3) (i : Fin 2) (cbd cQ : Fin 3)
    (wQ : Fin 2) :
    IsBiDualLeftWeyl B repLorentz
      (ofPairComponents fun a b => h.barDownBlock f f' i a cbd b cQ wQ)
```

lemma repGauge_u1_barDownBlock [source]

A hypercharge transformation fixes every component of the conjugate down-type block, the three hypercharges `3`, `-2` and `-1` cancelling.

```
lemma repGauge_u1_barDownBlock (t : unitary ℂ) (f f' : Fin 3) (i sbd : Fin 2)
    (cbd : Fin 3) (sQ : Fin 2) (cQ : Fin 3) (wQ : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.barDownBlock f f' i sbd cbd sQ cQ wQ)
      = h.barDownBlock f f' i sbd cbd sQ cQ wQ
```

def barDownBlockColour [source]

The colour contraction of the conjugate down-type block.

```
noncomputable def barDownBlockColour (f f' : Fin 3) (i sbd sQ wQ : Fin 2) : B
```

lemma barDownBlockColour_eq [source]

The colour contraction of the conjugate down-type block written out.

```
lemma barDownBlockColour_eq (f f' : Fin 3) (i sbd sQ wQ : Fin 2) :
    h.barDownBlockColour f f' i sbd sQ wQ
      = ∑ a : Fin 3, h.barDownBlock f f' i sbd a sQ a wQ
```

lemma isSU2FundamentalAntiFundamental_barDownBlockColour [source]

The colour contraction of the conjugate down-type block still carries one fundamental and one anti-fundamental isospin index.

```
lemma isSU2FundamentalAntiFundamental_barDownBlockColour (f f' : Fin 3) (sbd sQ : Fin 2) :
    IsSU2FundamentalAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 2 => h.barDownBlockColour f f' (l 0) sbd sQ (l 1))
```

def barDownBlockIsospin [source]

The isospin contraction of the colour-contracted conjugate down-type block.

```
noncomputable def barDownBlockIsospin (f f' : Fin 3) (sbd sQ : Fin 2) : B
```

lemma barDownBlockIsospin_eq [source]

The doubly contracted conjugate down-type block written out.

```
lemma barDownBlockIsospin_eq (f f' : Fin 3) (sbd sQ : Fin 2) :
    h.barDownBlockIsospin f f' sbd sQ
      = ∑ p : Fin 2 × Fin 3, h.barDownBlock f f' p.1 sbd p.2 sQ p.2 p.1
```

lemma isBiDualLeftWeyl_barDownBlockIsospin [source]

The doubly contracted conjugate down-type block carries two dual left-handed Weyl indices.

```
lemma isBiDualLeftWeyl_barDownBlockIsospin (f f' : Fin 3) :
    IsBiDualLeftWeyl B repLorentz (ofPairComponents (h.barDownBlockIsospin f f'))
```

def barDownYukawa [source]

The conjugate down-type Yukawa term of the family pair `(f, f')`. It is the image of the metric `εL'` under the map with the contracted block as its components.

```
noncomputable def barDownYukawa (f f' : Fin 3) : B
```

lemma repGauge_su3_barDownBlockColour [source]

The colour contraction of the conjugate down-type block is fixed by the colour factor.

```
lemma repGauge_su3_barDownBlockColour (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (i sbd sQ wQ : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.barDownBlockColour f f' i sbd sQ wQ)
      = h.barDownBlockColour f f' i sbd sQ wQ
```

lemma repGauge_su3_barDownBlockIsospin [source]

The doubly contracted conjugate down-type block is fixed by the colour factor.

```
lemma repGauge_su3_barDownBlockIsospin (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (sbd sQ : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.barDownBlockIsospin f f' sbd sQ)
      = h.barDownBlockIsospin f f' sbd sQ
```

lemma repGauge_su2_barDownBlockIsospin [source]

The doubly contracted conjugate down-type block is fixed by the isospin factor.

```
lemma repGauge_su2_barDownBlockIsospin (V : specialUnitaryGroup (Fin 2) ℂ) (f f' : Fin 3)
    (sbd sQ : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.barDownBlockIsospin f f' sbd sQ)
      = h.barDownBlockIsospin f f' sbd sQ
```

lemma repGauge_u1_barDownBlockIsospin [source]

The doubly contracted conjugate down-type block is fixed by the hypercharge factor.

```
lemma repGauge_u1_barDownBlockIsospin (t : unitary ℂ) (f f' : Fin 3) (sbd sQ : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.barDownBlockIsospin f f' sbd sQ)
      = h.barDownBlockIsospin f f' sbd sQ
```

lemma repGauge_barDownYukawa [source]

The conjugate down-type Yukawa term is gauge invariant.

```
lemma repGauge_barDownYukawa (f f' : Fin 3) (g : GaugeGroupI) :
    repGauge g (h.barDownYukawa f f') = h.barDownYukawa f f'
```

lemma repLorentz_barDownYukawa [source]

The conjugate down-type Yukawa term is Lorentz invariant.

```
lemma repLorentz_barDownYukawa (f f' : Fin 3) (Λ : SL(2,ℂ)) :
    repLorentz Λ (h.barDownYukawa f f') = h.barDownYukawa f f'
```

lemma barDownBlock_mem_sectorMassWeight [source]

Every component of the conjugate down-type block sits at mass weight eight in the Yukawa sector.

```
lemma barDownBlock_mem_sectorMassWeight (f f' : Fin 3) (i sbd : Fin 2) (cbd : Fin 3)
    (sQ : Fin 2) (cQ : Fin 3) (wQ : Fin 2) :
    h.barDownBlock f f' i sbd cbd sQ cQ wQ
      ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma barDownYukawa_mem_sectorMassWeight [source]

The conjugate down-type Yukawa term sits at mass weight eight in the Yukawa sector.

```
lemma barDownYukawa_mem_sectorMassWeight (f f' : Fin 3) :
    h.barDownYukawa f f'
      ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

def barUpBlock [source]

The components of the conjugate up-type Yukawa block `barH u barQ`.

```
noncomputable def barUpBlock (f f' : Fin 3) (i su : Fin 2) (cu : Fin 3) (sbQ : Fin 2)
    (cbQ : Fin 3) (wbQ : Fin 2) : B
```

lemma isSU3FundamentalAntiFundamental_barUpBlock [source]

The two colour indices of the conjugate up-type block carry one fundamental and one anti-fundamental `su(3)` index, the conjugate quark doublet supplying the fundamental one.

```
lemma isSU3FundamentalAntiFundamental_barUpBlock (f f' : Fin 3) (i su sbQ wbQ : Fin 2) :
    IsSU3FundamentalAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 3 => h.barUpBlock f f' i su (l 1) sbQ (l 0) wbQ)
```

lemma isSU2BiFundamental_barUpBlock [source]

The two isospin indices of the conjugate up-type block are both fundamental: the conjugate Higgs symbol and the conjugate quark doublet both carry the fundamental of `su(2)`.

```
lemma isSU2BiFundamental_barUpBlock (f f' : Fin 3) (su : Fin 2) (cu : Fin 3)
    (sbQ : Fin 2) (cbQ : Fin 3) :
    IsSU2BiFundamental B repGauge
      (fun l : Fin 2 → Fin 2 => h.barUpBlock f f' (l 0) su cu sbQ cbQ (l 1))
```

lemma isBiDualRightWeyl_barUpBlock [source]

The two spinor indices of the conjugate up-type block are both dual right-handed.

```
lemma isBiDualRightWeyl_barUpBlock (f f' : Fin 3) (i : Fin 2) (cu cbQ : Fin 3)
    (wbQ : Fin 2) :
    IsBiDualRightWeyl B repLorentz
      (ofPairComponents fun a b => h.barUpBlock f f' i a cu b cbQ wbQ)
```

lemma repGauge_u1_barUpBlock [source]

A hypercharge transformation fixes every component of the conjugate up-type block, the three hypercharges `3`, `-4` and `1` cancelling.

```
lemma repGauge_u1_barUpBlock (t : unitary ℂ) (f f' : Fin 3) (i su : Fin 2) (cu : Fin 3)
    (sbQ : Fin 2) (cbQ : Fin 3) (wbQ : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.barUpBlock f f' i su cu sbQ cbQ wbQ)
      = h.barUpBlock f f' i su cu sbQ cbQ wbQ
```

def barUpBlockColour [source]

The colour contraction of the conjugate up-type block.

```
noncomputable def barUpBlockColour (f f' : Fin 3) (i su sbQ wbQ : Fin 2) : B
```

lemma barUpBlockColour_eq [source]

The colour contraction of the conjugate up-type block written out.

```
lemma barUpBlockColour_eq (f f' : Fin 3) (i su sbQ wbQ : Fin 2) :
    h.barUpBlockColour f f' i su sbQ wbQ
      = ∑ a : Fin 3, h.barUpBlock f f' i su a sbQ a wbQ
```

lemma isSU2BiFundamental_barUpBlockColour [source]

The colour contraction of the conjugate up-type block still carries two fundamental isospin indices.

```
lemma isSU2BiFundamental_barUpBlockColour (f f' : Fin 3) (su sbQ : Fin 2) :
    IsSU2BiFundamental B repGauge
      (fun l : Fin 2 → Fin 2 => h.barUpBlockColour f f' (l 0) su sbQ (l 1))
```

def barUpBlockIsospin [source]

The isospin contraction of the colour-contracted conjugate up-type block, by the antisymmetric symbol.

```
noncomputable def barUpBlockIsospin (f f' : Fin 3) (su sbQ : Fin 2) : B
```

lemma barUpBlockIsospin_eq [source]

The doubly contracted conjugate up-type block written out.

```
lemma barUpBlockIsospin_eq (f f' : Fin 3) (su sbQ : Fin 2) :
    h.barUpBlockIsospin f f' su sbQ = (∑ a : Fin 3, h.barUpBlock f f' 0 su a sbQ a 1)
      - ∑ a : Fin 3, h.barUpBlock f f' 1 su a sbQ a 0
```

lemma isBiDualRightWeyl_barUpBlockIsospin [source]

The doubly contracted conjugate up-type block carries two dual right-handed Weyl indices.

```
lemma isBiDualRightWeyl_barUpBlockIsospin (f f' : Fin 3) :
    IsBiDualRightWeyl B repLorentz (ofPairComponents (h.barUpBlockIsospin f f'))
```

def barUpYukawa [source]

The conjugate up-type Yukawa term of the family pair `(f, f')`. It is the image of the metric `εR'` under the map with the contracted block as its components.

```
noncomputable def barUpYukawa (f f' : Fin 3) : B
```

lemma repGauge_su3_barUpBlockColour [source]

The colour contraction of the conjugate up-type block is fixed by the colour factor.

```
lemma repGauge_su3_barUpBlockColour (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (i su sbQ wbQ : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.barUpBlockColour f f' i su sbQ wbQ)
      = h.barUpBlockColour f f' i su sbQ wbQ
```

lemma repGauge_su3_barUpBlockIsospin [source]

The doubly contracted conjugate up-type block is fixed by the colour factor.

```
lemma repGauge_su3_barUpBlockIsospin (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (su sbQ : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.barUpBlockIsospin f f' su sbQ)
      = h.barUpBlockIsospin f f' su sbQ
```

lemma repGauge_su2_barUpBlockIsospin [source]

The doubly contracted conjugate up-type block is fixed by the isospin factor.

```
lemma repGauge_su2_barUpBlockIsospin (V : specialUnitaryGroup (Fin 2) ℂ) (f f' : Fin 3)
    (su sbQ : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.barUpBlockIsospin f f' su sbQ)
      = h.barUpBlockIsospin f f' su sbQ
```

lemma repGauge_u1_barUpBlockIsospin [source]

The doubly contracted conjugate up-type block is fixed by the hypercharge factor.

```
lemma repGauge_u1_barUpBlockIsospin (t : unitary ℂ) (f f' : Fin 3) (su sbQ : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.barUpBlockIsospin f f' su sbQ)
      = h.barUpBlockIsospin f f' su sbQ
```

lemma repGauge_barUpYukawa [source]

The conjugate up-type Yukawa term is gauge invariant.

```
lemma repGauge_barUpYukawa (f f' : Fin 3) (g : GaugeGroupI) :
    repGauge g (h.barUpYukawa f f') = h.barUpYukawa f f'
```

lemma repLorentz_barUpYukawa [source]

The conjugate up-type Yukawa term is Lorentz invariant.

```
lemma repLorentz_barUpYukawa (f f' : Fin 3) (Λ : SL(2,ℂ)) :
    repLorentz Λ (h.barUpYukawa f f') = h.barUpYukawa f f'
```

lemma barUpBlock_mem_sectorMassWeight [source]

Every component of the conjugate up-type block sits at mass weight eight in the Yukawa sector.

```
lemma barUpBlock_mem_sectorMassWeight (f f' : Fin 3) (i su : Fin 2) (cu : Fin 3)
    (sbQ : Fin 2) (cbQ : Fin 3) (wbQ : Fin 2) :
    h.barUpBlock f f' i su cu sbQ cbQ wbQ
      ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma barUpYukawa_mem_sectorMassWeight [source]

The conjugate up-type Yukawa term sits at mass weight eight in the Yukawa sector.

```
lemma barUpYukawa_mem_sectorMassWeight (f f' : Fin 3) :
    h.barUpYukawa f f'
      ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

def barLeptonBlock [source]

The components of the conjugate charged-lepton Yukawa block `barH L bare`.

```
noncomputable def barLeptonBlock (f f' : Fin 3) (i sL wL sbe : Fin 2) : B
```

lemma repGauge_su3_barLeptonBlock [source]

The conjugate lepton block is colour invariant outright.

```
lemma repGauge_su3_barLeptonBlock (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (i sL wL sbe : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.barLeptonBlock f f' i sL wL sbe)
      = h.barLeptonBlock f f' i sL wL sbe
```

lemma isSU2FundamentalAntiFundamental_barLeptonBlock [source]

The two isospin indices of the conjugate lepton block carry one fundamental and one anti-fundamental `su(2)` index, the conjugate Higgs symbol supplying the fundamental one.

```
lemma isSU2FundamentalAntiFundamental_barLeptonBlock (f f' : Fin 3) (sL sbe : Fin 2) :
    IsSU2FundamentalAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 2 => h.barLeptonBlock f f' (l 0) sL (l 1) sbe)
```

lemma isBiDualLeftWeyl_barLeptonBlock [source]

The two spinor indices of the conjugate lepton block are both dual left-handed.

```
lemma isBiDualLeftWeyl_barLeptonBlock (f f' : Fin 3) (i wL : Fin 2) :
    IsBiDualLeftWeyl B repLorentz
      (ofPairComponents fun a b => h.barLeptonBlock f f' i a wL b)
```

lemma repGauge_u1_barLeptonBlock [source]

A hypercharge transformation fixes every component of the conjugate lepton block, the three hypercharges `3`, `3` and `-6` cancelling.

```
lemma repGauge_u1_barLeptonBlock (t : unitary ℂ) (f f' : Fin 3) (i sL wL sbe : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.barLeptonBlock f f' i sL wL sbe)
      = h.barLeptonBlock f f' i sL wL sbe
```

def barLeptonBlockIsospin [source]

The isospin contraction of the conjugate lepton block.

```
noncomputable def barLeptonBlockIsospin (f f' : Fin 3) (sL sbe : Fin 2) : B
```

lemma barLeptonBlockIsospin_eq [source]

The isospin contraction of the conjugate lepton block written out.

```
lemma barLeptonBlockIsospin_eq (f f' : Fin 3) (sL sbe : Fin 2) :
    h.barLeptonBlockIsospin f f' sL sbe
      = ∑ w : Fin 2, h.barLeptonBlock f f' w sL w sbe
```

lemma isBiDualLeftWeyl_barLeptonBlockIsospin [source]

The contracted conjugate lepton block carries two dual left-handed Weyl indices.

```
lemma isBiDualLeftWeyl_barLeptonBlockIsospin (f f' : Fin 3) :
    IsBiDualLeftWeyl B repLorentz (ofPairComponents (h.barLeptonBlockIsospin f f'))
```

def barLeptonYukawa [source]

The conjugate charged-lepton Yukawa term of the family pair `(f, f')`. It is the image of the metric `εL'` under the map with the contracted block as its components.

```
noncomputable def barLeptonYukawa (f f' : Fin 3) : B
```

lemma repGauge_su3_barLeptonBlockIsospin [source]

The contracted conjugate lepton block is fixed by the colour factor.

```
lemma repGauge_su3_barLeptonBlockIsospin (U : specialUnitaryGroup (Fin 3) ℂ)
    (f f' : Fin 3) (sL sbe : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.barLeptonBlockIsospin f f' sL sbe)
      = h.barLeptonBlockIsospin f f' sL sbe
```

lemma repGauge_su2_barLeptonBlockIsospin [source]

The contracted conjugate lepton block is fixed by the isospin factor.

```
lemma repGauge_su2_barLeptonBlockIsospin (V : specialUnitaryGroup (Fin 2) ℂ)
    (f f' : Fin 3) (sL sbe : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.barLeptonBlockIsospin f f' sL sbe)
      = h.barLeptonBlockIsospin f f' sL sbe
```

lemma repGauge_u1_barLeptonBlockIsospin [source]

The contracted conjugate lepton block is fixed by the hypercharge factor.

```
lemma repGauge_u1_barLeptonBlockIsospin (t : unitary ℂ) (f f' : Fin 3) (sL sbe : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.barLeptonBlockIsospin f f' sL sbe)
      = h.barLeptonBlockIsospin f f' sL sbe
```

lemma repGauge_barLeptonYukawa [source]

The conjugate charged-lepton Yukawa term is gauge invariant.

```
lemma repGauge_barLeptonYukawa (f f' : Fin 3) (g : GaugeGroupI) :
    repGauge g (h.barLeptonYukawa f f') = h.barLeptonYukawa f f'
```

lemma repLorentz_barLeptonYukawa [source]

The conjugate charged-lepton Yukawa term is Lorentz invariant.

```
lemma repLorentz_barLeptonYukawa (f f' : Fin 3) (Λ : SL(2,ℂ)) :
    repLorentz Λ (h.barLeptonYukawa f f') = h.barLeptonYukawa f f'
```

lemma barLeptonBlock_mem_sectorMassWeight [source]

Every component of the conjugate lepton block sits at mass weight eight in the Yukawa sector.

```
lemma barLeptonBlock_mem_sectorMassWeight (f f' : Fin 3) (i sL wL sbe : Fin 2) :
    h.barLeptonBlock f f' i sL wL sbe
      ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma barLeptonYukawa_mem_sectorMassWeight [source]

The conjugate charged-lepton Yukawa term sits at mass weight eight in the Yukawa sector.

```
lemma barLeptonYukawa_mem_sectorMassWeight (f f' : Fin 3) :
    h.barLeptonYukawa f f'
      ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

def barDownYukawaSpan [source]

The span of the conjugate down-type Yukawa terms over the nine family pairs.

```
noncomputable def barDownYukawaSpan : Submodule ℂ B
```

def barUpYukawaSpan [source]

The span of the conjugate up-type Yukawa terms over the nine family pairs.

```
noncomputable def barUpYukawaSpan : Submodule ℂ B
```

def barLeptonYukawaSpan [source]

The span of the conjugate charged-lepton Yukawa terms over the nine family pairs.

```
noncomputable def barLeptonYukawaSpan : Submodule ℂ B
```

lemma barDownYukawaSpan_le_sectorMassWeight [source]

The conjugate down-type Yukawa span sits at mass weight eight in the Yukawa sector.

```
lemma barDownYukawaSpan_le_sectorMassWeight :
    h.barDownYukawaSpan
      ≤ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma barDownYukawaSpan_le_invariants [source]

The conjugate down-type Yukawa span is a space of gauge invariants.

```
lemma barDownYukawaSpan_le_invariants : h.barDownYukawaSpan ≤ repGauge.invariants
```

lemma barDownYukawaSpan_le_lorentzInvariants [source]

The conjugate down-type Yukawa span is a space of Lorentz invariants.

```
lemma barDownYukawaSpan_le_lorentzInvariants :
    h.barDownYukawaSpan ≤ repLorentz.invariants
```

lemma barUpYukawaSpan_le_sectorMassWeight [source]

The conjugate up-type Yukawa span sits at mass weight eight in the Yukawa sector.

```
lemma barUpYukawaSpan_le_sectorMassWeight :
    h.barUpYukawaSpan
      ≤ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma barUpYukawaSpan_le_invariants [source]

The conjugate up-type Yukawa span is a space of gauge invariants.

```
lemma barUpYukawaSpan_le_invariants : h.barUpYukawaSpan ≤ repGauge.invariants
```

lemma barUpYukawaSpan_le_lorentzInvariants [source]

The conjugate up-type Yukawa span is a space of Lorentz invariants.

```
lemma barUpYukawaSpan_le_lorentzInvariants : h.barUpYukawaSpan ≤ repLorentz.invariants
```

lemma barLeptonYukawaSpan_le_sectorMassWeight [source]

The conjugate charged-lepton Yukawa span sits at mass weight eight in the Yukawa sector.

```
lemma barLeptonYukawaSpan_le_sectorMassWeight :
    h.barLeptonYukawaSpan
      ≤ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma barLeptonYukawaSpan_le_invariants [source]

The conjugate charged-lepton Yukawa span is a space of gauge invariants.

```
lemma barLeptonYukawaSpan_le_invariants : h.barLeptonYukawaSpan ≤ repGauge.invariants
```

lemma barLeptonYukawaSpan_le_lorentzInvariants [source]

The conjugate charged-lepton Yukawa span is a space of Lorentz invariants.

```
lemma barLeptonYukawaSpan_le_lorentzInvariants :
    h.barLeptonYukawaSpan ≤ repLorentz.invariants
```

def yukawaSpan [source]

The Yukawa span of the Standard Model at mass weight eight: the join of the six couplings, each over the nine family pairs.

```
noncomputable def yukawaSpan : Submodule ℂ B
```

lemma yukawaSpan_le_inf [source]

The Yukawa span lies inside the gauge- and Lorentz-invariants of the Yukawa sector at mass weight eight. This is the easy direction of the classification: every Yukawa term is an invariant of the right mass weight, so the eventual classification is an equivalence and not merely a one-way inclusion.

```
lemma yukawaSpan_le_inf :
    h.yukawaSpan ≤ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
        ⊓ repGauge.invariants ⊓ repLorentz.invariants
```

2.421 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.Families.Higgs

PhyslibAlpha/Particles/StandardModel/CovRealization/YukawaSector/Families/Higgs.lean on GitHub.

def downBlock [source]

The components of the down-type Yukawa block `H d barQ`: a Higgs symbol, a down-singlet symbol and a conjugate quark-doublet symbol, none carrying derivatives, multiplied in the order in which the block of `sectorMassWeightEightGaugeWeight_piece_zero` multiplies them.

```
noncomputable def downBlock (f f' : Fin 3) (i sd : Fin 2) (cd : Fin 3) (sq : Fin 2)
    (cq : Fin 3) (wq : Fin 2) : B
```

lemma isSU3FundamentalAntiFundamental_downBlock [source]

The two colour indices of the down-type block carry one fundamental and one anti-fundamental `su(3)` index: the conjugate quark doublet supplies the fundamental index, so it goes in the first slot, and the down singlet the anti-fundamental one.

```
lemma isSU3FundamentalAntiFundamental_downBlock (f f' : Fin 3) (i sd sq wq : Fin 2) :
    IsSU3FundamentalAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 3 => h.downBlock f f' i sd (l 1) sq (l 0) wq)
```

lemma isSU2FundamentalAntiFundamental_downBlock [source]

The two isospin indices of the down-type block carry one fundamental and one anti-fundamental `su(2)` index: the conjugate quark doublet supplies the fundamental index and the Higgs symbol the anti-fundamental one, so the Higgs index goes in the second slot.

```
lemma isSU2FundamentalAntiFundamental_downBlock (f f' : Fin 3) (sd : Fin 2) (cd : Fin 3)
    (sq : Fin 2) (cq : Fin 3) :
    IsSU2FundamentalAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 2 => h.downBlock f f' (l 1) sd cd sq cq (l 0))
```

lemma isBiDualRightWeyl_downBlock [source]

The two spinor indices of the down-type block are both dual right-handed: the down singlet and the conjugate quark doublet are both right-handed, and the Higgs symbol without derivatives is a Lorentz scalar.

```
lemma isBiDualRightWeyl_downBlock (f f' : Fin 3) (i : Fin 2) (cd cq : Fin 3) (wq : Fin 2) :
    IsBiDualRightWeyl B repLorentz
      (ofPairComponents fun a b => h.downBlock f f' i a cd b cq wq)
```

lemma repGauge_u1_downBlock [source]

A hypercharge transformation fixes every component of the down-type block, the three hypercharges `-3`, `2` and `1` cancelling.

```
lemma repGauge_u1_downBlock (t : unitary ℂ) (f f' : Fin 3) (i sd : Fin 2) (cd : Fin 3)
    (sq : Fin 2) (cq : Fin 3) (wq : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.downBlock f f' i sd cd sq cq wq)
      = h.downBlock f f' i sd cd sq cq wq
```

def downBlockColour [source]

The colour contraction of the down-type block: the Kronecker delta joining the fundamental colour index of the conjugate quark doublet to the anti-fundamental one of the down singlet.

```
noncomputable def downBlockColour (f f' : Fin 3) (i sd sq wq : Fin 2) : B
```

lemma downBlockColour_eq [source]

The colour contraction written out: the sum of the three components with equal colour indices.

```
lemma downBlockColour_eq (f f' : Fin 3) (i sd sq wq : Fin 2) :
    h.downBlockColour f f' i sd sq wq
      = ∑ a : Fin 3, h.downBlock f f' i sd a sq a wq
```

lemma isSU2FundamentalAntiFundamental_downBlockColour [source]

The colour contraction of the down-type block still carries one fundamental and one anti-fundamental isospin index, the colour sum being an isospin spectator.

```
lemma isSU2FundamentalAntiFundamental_downBlockColour (f f' : Fin 3) (sd sq : Fin 2) :
    IsSU2FundamentalAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 2 => h.downBlockColour f f' (l 1) sd sq (l 0))
```

def downBlockIsospin [source]

The isospin contraction of the colour-contracted down-type block: the Kronecker delta joining the fundamental isospin index of the conjugate quark doublet to the anti-fundamental one of the Higgs.

```
noncomputable def downBlockIsospin (f f' : Fin 3) (sd sq : Fin 2) : B
```

lemma downBlockIsospin_eq [source]

The doubly contracted block written out as a single sum over the isospin and colour indices it identifies.

```
lemma downBlockIsospin_eq (f f' : Fin 3) (sd sq : Fin 2) :
    h.downBlockIsospin f f' sd sq
      = ∑ p : Fin 2 × Fin 3, h.downBlock f f' p.1 sd p.2 sq p.2 p.1
```

lemma isBiDualRightWeyl_downBlockIsospin [source]

The doubly contracted block carries two dual right-handed Weyl indices, the colour and isospin sums being Lorentz spectators.

```
lemma isBiDualRightWeyl_downBlockIsospin (f f' : Fin 3) :
    IsBiDualRightWeyl B repLorentz (ofPairComponents (h.downBlockIsospin f f'))
```

def downYukawa [source]

The down-type Yukawa term of the family pair `(f, f')`: the down-singlet symbol of family `f` against the conjugate quark doublet of family `f'` and a Higgs symbol, with the colour indices joined by the Kronecker delta, the isospin indices by the Kronecker delta, and the two right-handed spinor indices by the antisymmetric symbol. It is the image of the metric `εR'` under the map with the contracted block as its components.

```
noncomputable def downYukawa (f f' : Fin 3) : B
```

lemma repGauge_su3_downBlockColour [source]

The colour contraction of the down-type block is fixed by the colour factor.

```
lemma repGauge_su3_downBlockColour (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (i sd sq wq : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.downBlockColour f f' i sd sq wq)
      = h.downBlockColour f f' i sd sq wq
```

lemma repGauge_su3_downBlockIsospin [source]

The doubly contracted down-type block is fixed by the colour factor.

```
lemma repGauge_su3_downBlockIsospin (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (sd sq : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.downBlockIsospin f f' sd sq)
      = h.downBlockIsospin f f' sd sq
```

lemma repGauge_su2_downBlockIsospin [source]

The doubly contracted down-type block is fixed by the isospin factor.

```
lemma repGauge_su2_downBlockIsospin (V : specialUnitaryGroup (Fin 2) ℂ) (f f' : Fin 3)
    (sd sq : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.downBlockIsospin f f' sd sq)
      = h.downBlockIsospin f f' sd sq
```

lemma repGauge_u1_downBlockIsospin [source]

The doubly contracted down-type block is fixed by the hypercharge factor, already component by component.

```
lemma repGauge_u1_downBlockIsospin (t : unitary ℂ) (f f' : Fin 3) (sd sq : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.downBlockIsospin f f' sd sq)
      = h.downBlockIsospin f f' sd sq
```

lemma repGauge_downYukawa [source]

The down-type Yukawa term is gauge invariant: the colour indices are joined by the Kronecker delta, the isospin indices by the Kronecker delta, and the three hypercharges cancel.

```
lemma repGauge_downYukawa (f f' : Fin 3) (g : GaugeGroupI) :
    repGauge g (h.downYukawa f f') = h.downYukawa f f'
```

lemma repLorentz_downYukawa [source]

The down-type Yukawa term is Lorentz invariant, the two right-handed spinor indices being joined by the antisymmetric symbol.

```
lemma repLorentz_downYukawa (f f' : Fin 3) (Λ : SL(2,ℂ)) :
    repLorentz Λ (h.downYukawa f f') = h.downYukawa f f'
```

lemma downBlock_mem_sectorMassWeight [source]

Every component of the down-type block sits at mass weight eight in the Yukawa sector.

```
lemma downBlock_mem_sectorMassWeight (f f' : Fin 3) (i sd : Fin 2) (cd : Fin 3)
    (sq : Fin 2) (cq : Fin 3) (wq : Fin 2) :
    h.downBlock f f' i sd cd sq cq wq
      ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma downYukawa_mem_sectorMassWeight [source]

The down-type Yukawa term sits at mass weight eight in the Yukawa sector.

```
lemma downYukawa_mem_sectorMassWeight (f f' : Fin 3) :
    h.downYukawa f f' ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

def upBlock [source]

The components of the up-type Yukawa block `H baru Q`: a Higgs symbol, a conjugate up-singlet symbol and a quark-doublet symbol, none carrying derivatives.

```
noncomputable def upBlock (f f' : Fin 3) (i su : Fin 2) (cu : Fin 3) (sQ : Fin 2)
    (cQ : Fin 3) (wQ : Fin 2) : B
```

lemma isSU3FundamentalAntiFundamental_upBlock [source]

The two colour indices of the up-type block carry one fundamental and one anti-fundamental `su(3)` index, the conjugate up singlet supplying the fundamental one.

```
lemma isSU3FundamentalAntiFundamental_upBlock (f f' : Fin 3) (i su sQ wQ : Fin 2) :
    IsSU3FundamentalAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 3 => h.upBlock f f' i su (l 0) sQ (l 1) wQ)
```

lemma isSU2BiAntiFundamental_upBlock [source]

The two isospin indices of the up-type block are both anti-fundamental: the Higgs symbol and the quark doublet both carry the anti-fundamental of `su(2)`.

```
lemma isSU2BiAntiFundamental_upBlock (f f' : Fin 3) (su : Fin 2) (cu : Fin 3) (sQ : Fin 2)
    (cQ : Fin 3) :
    IsSU2BiAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 2 => h.upBlock f f' (l 0) su cu sQ cQ (l 1))
```

lemma isBiDualLeftWeyl_upBlock [source]

The two spinor indices of the up-type block are both dual left-handed.

```
lemma isBiDualLeftWeyl_upBlock (f f' : Fin 3) (i : Fin 2) (cu cQ : Fin 3) (wQ : Fin 2) :
    IsBiDualLeftWeyl B repLorentz
      (ofPairComponents fun a b => h.upBlock f f' i a cu b cQ wQ)
```

lemma repGauge_u1_upBlock [source]

A hypercharge transformation fixes every component of the up-type block, the three hypercharges `-3`, `4` and `-1` cancelling.

```
lemma repGauge_u1_upBlock (t : unitary ℂ) (f f' : Fin 3) (i su : Fin 2) (cu : Fin 3)
    (sQ : Fin 2) (cQ : Fin 3) (wQ : Fin 2) :
```

```
    repGauge ((1, 1, t) : GaugeGroupI) (h.upBlock f f' i su cu sQ cQ wQ)
      = h.upBlock f f' i su cu sQ cQ wQ
```

def upBlockColour [source]

The colour contraction of the up-type block.

```
noncomputable def upBlockColour (f f' : Fin 3) (i su sQ wQ : Fin 2) : B
```

lemma upBlockColour_eq [source]

The colour contraction of the up-type block written out.

```
lemma upBlockColour_eq (f f' : Fin 3) (i su sQ wQ : Fin 2) :
    h.upBlockColour f f' i su sQ wQ = ∑ a : Fin 3, h.upBlock f f' i su a sQ a wQ
```

lemma isSU2BiAntiFundamental_upBlockColour [source]

The colour contraction of the up-type block still carries two anti-fundamental isospin indices.

```
lemma isSU2BiAntiFundamental_upBlockColour (f f' : Fin 3) (su sQ : Fin 2) :
    IsSU2BiAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 2 => h.upBlockColour f f' (l 0) su sQ (l 1))
```

def upBlockIsospin [source]

The isospin contraction of the colour-contracted up-type block, by the antisymmetric symbol: two anti-fundamental isospin indices admit no trace.

```
noncomputable def upBlockIsospin (f f' : Fin 3) (su sQ : Fin 2) : B
```

lemma upBlockIsospin_eq [source]

The doubly contracted up-type block written out.

```
lemma upBlockIsospin_eq (f f' : Fin 3) (su sQ : Fin 2) :
    h.upBlockIsospin f f' su sQ = (∑ a : Fin 3, h.upBlock f f' 0 su a sQ a 1)
      - ∑ a : Fin 3, h.upBlock f f' 1 su a sQ a 0
```

lemma isBiDualLeftWeyl_upBlockIsospin [source]

The doubly contracted up-type block carries two dual left-handed Weyl indices.

```
lemma isBiDualLeftWeyl_upBlockIsospin (f f' : Fin 3) :
    IsBiDualLeftWeyl B repLorentz (ofPairComponents (h.upBlockIsospin f f'))
```

def upYukawa [source]

The up-type Yukawa term of the family pair `(f, f')`: the colour indices are joined by the Kronecker delta, the isospin indices by the antisymmetric symbol, and the two left-handed spinor indices by the antisymmetric symbol. It is the image of the metric `εL'` under the map with the contracted block as its components.

```
noncomputable def upYukawa (f f' : Fin 3) : B
```

lemma repGauge_su3_upBlockColour [source]

The colour contraction of the up-type block is fixed by the colour factor.

```
lemma repGauge_su3_upBlockColour (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (i su sQ wQ : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.upBlockColour f f' i su sQ wQ)
      = h.upBlockColour f f' i su sQ wQ
```

lemma repGauge_su3_upBlockIsospin [source]

The doubly contracted up-type block is fixed by the colour factor.

```
lemma repGauge_su3_upBlockIsospin (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (su sQ : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.upBlockIsospin f f' su sQ)
      = h.upBlockIsospin f f' su sQ
```

lemma repGauge_su2_upBlockIsospin [source]

The doubly contracted up-type block is fixed by the isospin factor.

```
lemma repGauge_su2_upBlockIsospin (V : specialUnitaryGroup (Fin 2) ℂ) (f f' : Fin 3)
    (su sQ : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.upBlockIsospin f f' su sQ)
      = h.upBlockIsospin f f' su sQ
```

lemma repGauge_u1_upBlockIsospin [source]

The doubly contracted up-type block is fixed by the hypercharge factor.

```
lemma repGauge_u1_upBlockIsospin (t : unitary ℂ) (f f' : Fin 3) (su sQ : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.upBlockIsospin f f' su sQ)
      = h.upBlockIsospin f f' su sQ
```

lemma repGauge_upYukawa [source]

The up-type Yukawa term is gauge invariant.

```
lemma repGauge_upYukawa (f f' : Fin 3) (g : GaugeGroupI) :
    repGauge g (h.upYukawa f f') = h.upYukawa f f'
```

lemma repLorentz_upYukawa [source]

The up-type Yukawa term is Lorentz invariant.

```
lemma repLorentz_upYukawa (f f' : Fin 3) (Λ : SL(2,ℂ)) :
    repLorentz Λ (h.upYukawa f f') = h.upYukawa f f'
```

lemma upBlock_mem_sectorMassWeight [source]

Every component of the up-type block sits at mass weight eight in the Yukawa sector.

```
lemma upBlock_mem_sectorMassWeight (f f' : Fin 3) (i su : Fin 2) (cu : Fin 3)
    (sQ : Fin 2) (cQ : Fin 3) (wQ : Fin 2) :
    h.upBlock f f' i su cu sQ cQ wQ
      ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma upYukawa_mem_sectorMassWeight [source]

The up-type Yukawa term sits at mass weight eight in the Yukawa sector.

```
lemma upYukawa_mem_sectorMassWeight (f f' : Fin 3) :
    h.upYukawa f f' ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

def leptonBlock [source]

The components of the charged-lepton Yukawa block `H barL e`: a Higgs symbol, a conjugate lepton-doublet symbol and a lepton-singlet symbol, none carrying derivatives.

```
noncomputable def leptonBlock (f f' : Fin 3) (i sL wL se : Fin 2) : B
```

lemma repGauge_su3_leptonBlock [source]

The lepton block is colour invariant outright: none of its three symbols carries a colour index. This is what stands in for the colour classification of the quark blocks.

```
lemma repGauge_su3_leptonBlock (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (i sL wL se : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.leptonBlock f f' i sL wL se)
      = h.leptonBlock f f' i sL wL se
```

lemma isSU2FundamentalAntiFundamental_leptonBlock [source]

The two isospin indices of the lepton block carry one fundamental and one anti-fundamental `su(2)` index, the conjugate lepton doublet supplying the fundamental one.

```
lemma isSU2FundamentalAntiFundamental_leptonBlock (f f' : Fin 3) (sL se : Fin 2) :
    IsSU2FundamentalAntiFundamental B repGauge
      (fun l : Fin 2 → Fin 2 => h.leptonBlock f f' (l 1) sL (l 0) se)
```

lemma isBiDualRightWeyl_leptonBlock [source]

The two spinor indices of the lepton block are both dual right-handed.

```
lemma isBiDualRightWeyl_leptonBlock (f f' : Fin 3) (i wL : Fin 2) :
    IsBiDualRightWeyl B repLorentz
      (ofPairComponents fun a b => h.leptonBlock f f' i a wL b)
```

lemma repGauge_u1_leptonBlock [source]

A hypercharge transformation fixes every component of the lepton block, the three hypercharges `-3`, `-3` and `6` cancelling.

```
lemma repGauge_u1_leptonBlock (t : unitary ℂ) (f f' : Fin 3) (i sL wL se : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.leptonBlock f f' i sL wL se)
      = h.leptonBlock f f' i sL wL se
```

def leptonBlockIsospin [source]

The isospin contraction of the lepton block.

```
noncomputable def leptonBlockIsospin (f f' : Fin 3) (sL se : Fin 2) : B
```

lemma leptonBlockIsospin_eq [source]

The isospin contraction of the lepton block written out.

```
lemma leptonBlockIsospin_eq (f f' : Fin 3) (sL se : Fin 2) :
    h.leptonBlockIsospin f f' sL se = ∑ w : Fin 2, h.leptonBlock f f' w sL w se
```

lemma isBiDualRightWeyl_leptonBlockIsospin [source]

The contracted lepton block carries two dual right-handed Weyl indices.

```
lemma isBiDualRightWeyl_leptonBlockIsospin (f f' : Fin 3) :
    IsBiDualRightWeyl B repLorentz (ofPairComponents (h.leptonBlockIsospin f f'))
```

def leptonYukawa [source]

The charged-lepton Yukawa term of the family pair `(f, f')`: the isospin indices are joined by the Kronecker delta and the two right-handed spinor indices by the antisymmetric symbol, colour playing no part. It is the image of the metric `εR'` under the map with the contracted block as its components.

```
noncomputable def leptonYukawa (f f' : Fin 3) : B
```

lemma repGauge_su3_leptonBlockIsospin [source]

The contracted lepton block is fixed by the colour factor.

```
lemma repGauge_su3_leptonBlockIsospin (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (sL se : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.leptonBlockIsospin f f' sL se)
      = h.leptonBlockIsospin f f' sL se
```

lemma repGauge_su2_leptonBlockIsospin [source]

The contracted lepton block is fixed by the isospin factor.

```
lemma repGauge_su2_leptonBlockIsospin (V : specialUnitaryGroup (Fin 2) ℂ) (f f' : Fin 3)
    (sL se : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.leptonBlockIsospin f f' sL se)
      = h.leptonBlockIsospin f f' sL se
```

lemma repGauge_u1_leptonBlockIsospin [source]

The contracted lepton block is fixed by the hypercharge factor.

```
lemma repGauge_u1_leptonBlockIsospin (t : unitary ℂ) (f f' : Fin 3) (sL se : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.leptonBlockIsospin f f' sL se)
      = h.leptonBlockIsospin f f' sL se
```

lemma repGauge_leptonYukawa [source]

The charged-lepton Yukawa term is gauge invariant.

```
lemma repGauge_leptonYukawa (f f' : Fin 3) (g : GaugeGroupI) :
    repGauge g (h.leptonYukawa f f') = h.leptonYukawa f f'
```

lemma repLorentz_leptonYukawa [source]

The charged-lepton Yukawa term is Lorentz invariant.

```
lemma repLorentz_leptonYukawa (f f' : Fin 3) (Λ : SL(2,ℂ)) :
    repLorentz Λ (h.leptonYukawa f f') = h.leptonYukawa f f'
```

lemma leptonBlock_mem_sectorMassWeight [source]

Every component of the lepton block sits at mass weight eight in the Yukawa sector.

```
lemma leptonBlock_mem_sectorMassWeight (f f' : Fin 3) (i sL wL se : Fin 2) :
    h.leptonBlock f f' i sL wL se
      ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma leptonYukawa_mem_sectorMassWeight [source]

The charged-lepton Yukawa term sits at mass weight eight in the Yukawa sector.

```
lemma leptonYukawa_mem_sectorMassWeight (f f' : Fin 3) :
    h.leptonYukawa f f'
      ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

def downYukawaSpan [source]

The span of the down-type Yukawa terms over the nine family pairs: the down-type Yukawa coupling with an arbitrary 3×3 matrix.

```
noncomputable def downYukawaSpan : Submodule ℂ B
```

def upYukawaSpan [source]

The span of the up-type Yukawa terms over the nine family pairs.

```
noncomputable def upYukawaSpan : Submodule ℂ B
```

def leptonYukawaSpan [source]

The span of the charged-lepton Yukawa terms over the nine family pairs.

```
noncomputable def leptonYukawaSpan : Submodule ℂ B
```

lemma downYukawaSpan_le_sectorMassWeight [source]

The down-type Yukawa span sits at mass weight eight in the Yukawa sector.

```
lemma downYukawaSpan_le_sectorMassWeight :
    h.downYukawaSpan ≤ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma downYukawaSpan_le_invariants [source]

The down-type Yukawa span is a space of gauge invariants.

```
lemma downYukawaSpan_le_invariants : h.downYukawaSpan ≤ repGauge.invariants
```

lemma downYukawaSpan_le_lorentzInvariants [source]

The down-type Yukawa span is a space of Lorentz invariants.

```
lemma downYukawaSpan_le_lorentzInvariants : h.downYukawaSpan ≤ repLorentz.invariants
```

lemma upYukawaSpan_le_sectorMassWeight [source]

The up-type Yukawa span sits at mass weight eight in the Yukawa sector.

```
lemma upYukawaSpan_le_sectorMassWeight :
    h.upYukawaSpan ≤ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma upYukawaSpan_le_invariants [source]

The up-type Yukawa span is a space of gauge invariants.

```
lemma upYukawaSpan_le_invariants : h.upYukawaSpan ≤ repGauge.invariants
```

lemma upYukawaSpan_le_lorentzInvariants [source]

The up-type Yukawa span is a space of Lorentz invariants.

```
lemma upYukawaSpan_le_lorentzInvariants : h.upYukawaSpan ≤ repLorentz.invariants
```

lemma leptonYukawaSpan_le_sectorMassWeight [source]

The charged-lepton Yukawa span sits at mass weight eight in the Yukawa sector.

```
lemma leptonYukawaSpan_le_sectorMassWeight :
    h.leptonYukawaSpan ≤ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8
```

lemma leptonYukawaSpan_le_invariants [source]

The charged-lepton Yukawa span is a space of gauge invariants.

```
lemma leptonYukawaSpan_le_invariants : h.leptonYukawaSpan ≤ repGauge.invariants
```

lemma leptonYukawaSpan_le_lorentzInvariants [source]

The charged-lepton Yukawa span is a space of Lorentz invariants.

```
lemma leptonYukawaSpan_le_lorentzInvariants : h.leptonYukawaSpan ≤ repLorentz.invariants
```

2.422 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.Families.Symbols

PhyslibAlpha/Particles/StandardModel/CovRealization/YukawaSector/Families/Symbols.lean on GitHub.

lemma repGauge_mul_fixed_left [source]

A triple product whose first factor is fixed and whose second and third move by given coefficients moves by the product of those coefficients.

```
lemma repGauge_mul_fixed_left (g : GaugeGroupI) {ι κ : Type} [Fintype ι] [Fintype κ]
    {A : B} {X : ι → B} {Y : κ → B} {x₀ : ι} {y₀ : κ} {cX : ι → ℂ} {cY : κ → ℂ}
    (hA : repGauge g A = A) (hX : repGauge g (X x₀) = ∑ x, cX x • X x)
    (hY : repGauge g (Y y₀) = ∑ y, cY y • Y y) :
    repGauge g (A * (X x₀ * Y y₀)) = ∑ x, ∑ y, (cX x * cY y) • (A * (X x * Y y))
```

lemma repGauge_mul_fixed_mid [source]

A triple product whose second factor is fixed.

```
lemma repGauge_mul_fixed_mid (g : GaugeGroupI) {ι κ : Type} [Fintype ι] [Fintype κ]
    {A : ι → B} {X : B} {Y : κ → B} {a₀ : ι} {y₀ : κ} {cA : ι → ℂ} {cY : κ → ℂ}
    (hA : repGauge g (A a₀) = ∑ a, cA a • A a) (hX : repGauge g X = X)
    (hY : repGauge g (Y y₀) = ∑ y, cY y • Y y) :
    repGauge g (A a₀ * (X * Y y₀)) = ∑ a, ∑ y, (cA a * cY y) • (A a * (X * Y y))
```

lemma repGauge_mul_fixed_right [source]

A triple product whose third factor is fixed.

```
lemma repGauge_mul_fixed_right (g : GaugeGroupI) {ι κ : Type} [Fintype ι] [Fintype κ]
    {A : ι → B} {X : κ → B} {Y : B} {a₀ : ι} {x₀ : κ} {cA : ι → ℂ} {cX : κ → ℂ}
    (hA : repGauge g (A a₀) = ∑ a, cA a • A a)
    (hX : repGauge g (X x₀) = ∑ x, cX x • X x) (hY : repGauge g Y = Y) :
    repGauge g (A a₀ * (X x₀ * Y)) = ∑ a, ∑ x, (cA a * cX x) • (A a * (X x * Y))
```

lemma repLorentz_mul_fixed_left [source]

The Lorentz analogue of `repGauge_mul_fixed_left`, the Higgs factor being inert.

```
lemma repLorentz_mul_fixed_left (Λ : SL(2,ℂ)) {ι κ : Type} [Fintype ι] [Fintype κ]
    {A : B} {X : ι → B} {Y : κ → B} {x₀ : ι} {y₀ : κ} {cX : ι → ℂ} {cY : κ → ℂ}
    (hA : repLorentz Λ A = A) (hX : repLorentz Λ (X x₀) = ∑ x, cX x • X x)
    (hY : repLorentz Λ (Y y₀) = ∑ y, cY y • Y y) :
    repLorentz Λ (A * (X x₀ * Y y₀)) = ∑ x, ∑ y, (cX x * cY y) • (A * (X x * Y y))
```

lemma repGauge_su3_higgs [source]

A colour transformation fixes a Higgs symbol.

```
lemma repGauge_su3_higgs (U : specialUnitaryGroup (Fin 3) ℂ) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (i : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isHiggsSector.higgs l i)
      = h.isHiggsSector.higgs l i
```

lemma repGauge_su2_higgs [source]

An isospin transformation moves the isospin index of a Higgs symbol by the conjugate matrix, the index being anti-fundamental.

```
lemma repGauge_su2_higgs (V : specialUnitaryGroup (Fin 2) ℂ) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (i : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isHiggsSector.higgs l i)
      = ∑ a, conj (V.1 a i) • h.isHiggsSector.higgs l a
```

lemma repGauge_u1_higgs [source]

A hypercharge transformation scales a Higgs symbol by the cube of the conjugate scalar, the Higgs carrying `6Y = -3`.

```
lemma repGauge_u1_higgs (t : unitary ℂ) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (i : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isHiggsSector.higgs l i)
      = (star (t : ℂ)) ^ 3 • h.isHiggsSector.higgs l i
```

lemma repGauge_su3_barHiggs [source]

A colour transformation fixes a conjugate Higgs symbol.

```
lemma repGauge_su3_barHiggs (U : specialUnitaryGroup (Fin 3) ℂ) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (i : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isHiggsSector.barHiggs l i)
      = h.isHiggsSector.barHiggs l i
```

lemma repGauge_su2_barHiggs [source]

An isospin transformation moves the isospin index of a conjugate Higgs symbol by the matrix itself, the index being fundamental.

```
lemma repGauge_su2_barHiggs (V : specialUnitaryGroup (Fin 2) ℂ) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (i : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isHiggsSector.barHiggs l i)
      = ∑ a, V.1 a i • h.isHiggsSector.barHiggs l a
```

lemma repGauge_u1_barHiggs [source]

A hypercharge transformation scales a conjugate Higgs symbol by the cube of the scalar, the conjugate Higgs carrying `6Y = 3`.

```
lemma repGauge_u1_barHiggs (t : unitary ℂ) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (i : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isHiggsSector.barHiggs l i)
      = (t : ℂ) ^ 3 • h.isHiggsSector.barHiggs l i
```

lemma repLorentz_higgs_zero [source]

A Higgs symbol carrying no derivatives is Lorentz invariant.

```
lemma repLorentz_higgs_zero (Λ : SL(2,ℂ)) (l : Fin 0 → Fin 1 ⊕ Fin 3) (i : Fin 2) :
    repLorentz Λ (h.isHiggsSector.higgs l i) = h.isHiggsSector.higgs l i
```

lemma repLorentz_barHiggs_zero [source]

A conjugate Higgs symbol carrying no derivatives is Lorentz invariant.

```
lemma repLorentz_barHiggs_zero (Λ : SL(2,ℂ)) (l : Fin 0 → Fin 1 ⊕ Fin 3) (i : Fin 2) :
    repLorentz Λ (h.isHiggsSector.barHiggs l i) = h.isHiggsSector.barHiggs l i
```

lemma repGauge_su3_d [source]

A colour transformation moves the colour index of a down-singlet symbol by the conjugate matrix, the index being anti-fundamental.

```
lemma repGauge_su3_d (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isFermionSector.dComponent f l (s, c))
      = ∑ a, conj (U.1 a c) • h.isFermionSector.dComponent f l (s, a)
```

lemma repGauge_su2_d [source]

An isospin transformation fixes a down-singlet symbol, which carries no isospin.

```
lemma repGauge_su2_d (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isFermionSector.dComponent f l j)
      = h.isFermionSector.dComponent f l j
```

lemma repGauge_u1_d [source]

A hypercharge transformation scales a down-singlet symbol by the square of the scalar, the down singlet carrying `6Y = 2`.

```
lemma repGauge_u1_d (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isFermionSector.dComponent f l j)
      = (t : ℂ) ^ 2 • h.isFermionSector.dComponent f l j
```

lemma repGauge_su3_bard [source]

A colour transformation moves the colour index of a conjugate down-singlet symbol by the matrix itself, the index being fundamental.

```
lemma repGauge_su3_bard (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isFermionSector.bardComponent f l (s, c))
      = ∑ a, U.1 a c • h.isFermionSector.bardComponent f l (s, a)
```

lemma repGauge_su2_bard [source]

An isospin transformation fixes a conjugate down-singlet symbol.

```
lemma repGauge_su2_bard (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isFermionSector.bardComponent f l j)
      = h.isFermionSector.bardComponent f l j
```

lemma repGauge_u1_bard [source]

A hypercharge transformation scales a conjugate down-singlet symbol by the square of the conjugate scalar, the conjugate down singlet carrying `6Y = -2`.

```
lemma repGauge_u1_bard (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isFermionSector.bardComponent f l j)
      = (star (t : ℂ)) ^ 2 • h.isFermionSector.bardComponent f l j
```

lemma repGauge_su3_u [source]

A colour transformation moves the colour index of an up-singlet symbol by the conjugate matrix, the index being anti-fundamental.

```
lemma repGauge_su3_u (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isFermionSector.uComponent f l (s, c))
      = ∑ a, conj (U.1 a c) • h.isFermionSector.uComponent f l (s, a)
```

lemma repGauge_su2_u [source]

An isospin transformation fixes an up-singlet symbol.

```
lemma repGauge_su2_u (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isFermionSector.uComponent f l j)
      = h.isFermionSector.uComponent f l j
```

lemma repGauge_u1_u [source]

A hypercharge transformation scales an up-singlet symbol by the fourth power of the conjugate scalar, the up singlet carrying `6Y = -4`.

```
lemma repGauge_u1_u (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isFermionSector.uComponent f l j)
      = (star (t : ℂ)) ^ 4 • h.isFermionSector.uComponent f l j
```

lemma repGauge_su3_baru [source]

A colour transformation moves the colour index of a conjugate up-singlet symbol by the matrix itself, the index being fundamental.

```
lemma repGauge_su3_baru (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isFermionSector.baruComponent f l (s, c))
```

```
    = ∑ a, U.1 a c • h.isFermionSector.baruComponent f l (s, a)
```

lemma repGauge_su2_baru [source]

An isospin transformation fixes a conjugate up-singlet symbol.

```
lemma repGauge_su2_baru (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isFermionSector.baruComponent f l j)
      = h.isFermionSector.baruComponent f l j
```

lemma repGauge_u1_baru [source]

A hypercharge transformation scales a conjugate up-singlet symbol by the fourth power of the scalar, the conjugate up singlet carrying `6Y = 4`.

```
lemma repGauge_u1_baru (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isFermionSector.baruComponent f l j)
      = (t : ℂ) ^ 4 • h.isFermionSector.baruComponent f l j
```

lemma repGauge_su3_Q [source]

A colour transformation moves the colour index of a quark-doublet symbol by the conjugate matrix, the index being anti-fundamental.

```
lemma repGauge_su3_Q (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) (w : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isFermionSector.QComponent f l (s, c, w))
      = ∑ a, conj (U.1 a c) • h.isFermionSector.QComponent f l (s, a, w)
```

lemma repGauge_su2_Q [source]

An isospin transformation moves the isospin index of a quark-doublet symbol by the conjugate matrix, the index being anti-fundamental.

```
lemma repGauge_su2_Q (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) (w : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isFermionSector.QComponent f l (s, c, w))
      = ∑ a, conj (V.1 a w) • h.isFermionSector.QComponent f l (s, c, a)
```

lemma repGauge_u1_Q [source]

A hypercharge transformation scales a quark-doublet symbol by the conjugate scalar, the quark doublet carrying `6Y = -1`.

```
lemma repGauge_u1_Q (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (s : Fin 2) (c : Fin 3) (w : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isFermionSector.QComponent f l (s, c, w))
      = star (t : ℂ) • h.isFermionSector.QComponent f l (s, c, w)
```

lemma repGauge_su3_barQ [source]

A colour transformation moves the colour index of a conjugate quark-doublet symbol by the matrix itself, the index being fundamental.

```
lemma repGauge_su3_barQ (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) (w : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isFermionSector.barQComponent f l (s, c, w))
      = ∑ a, U.1 a c • h.isFermionSector.barQComponent f l (s, a, w)
```

lemma repGauge_su2_barQ [source]

An isospin transformation moves the isospin index of a conjugate quark-doublet symbol by the matrix itself, the index being fundamental.

```
lemma repGauge_su2_barQ (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) (w : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isFermionSector.barQComponent f l (s, c, w))
      = ∑ a, V.1 a w • h.isFermionSector.barQComponent f l (s, c, a)
```

lemma repGauge_u1_barQ [source]

A hypercharge transformation scales a conjugate quark-doublet symbol by the scalar, the conjugate quark doublet carrying `6Y = 1`.

```
lemma repGauge_u1_barQ (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (s : Fin 2) (c : Fin 3) (w : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isFermionSector.barQComponent f l (s, c, w))
      = (t : ℂ) • h.isFermionSector.barQComponent f l (s, c, w)
```

lemma repGauge_su3_L [source]

A colour transformation fixes a lepton-doublet symbol, which carries no colour.

```
lemma repGauge_su3_L (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isFermionSector.LComponent f l j)
      = h.isFermionSector.LComponent f l j
```

lemma repGauge_su2_L [source]

An isospin transformation moves the isospin index of a lepton-doublet symbol by the conjugate matrix, the index being anti-fundamental.

```
lemma repGauge_su2_L (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s w : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isFermionSector.LComponent f l (s, w))
      = ∑ a, conj (V.1 a w) • h.isFermionSector.LComponent f l (s, a)
```

lemma repGauge_u1_L [source]

A hypercharge transformation scales a lepton-doublet symbol by the cube of the scalar, the lepton doublet carrying `6Y = 3`.

```
lemma repGauge_u1_L (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isFermionSector.LComponent f l j)
```

```
    = (t : ℂ) ^ 3 • h.isFermionSector.LComponent f l j
```

lemma repGauge_su3_barL [source]

A colour transformation fixes a conjugate lepton-doublet symbol, which carries no colour.

```
lemma repGauge_su3_barL (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isFermionSector.barLComponent f l j)
      = h.isFermionSector.barLComponent f l j
```

lemma repGauge_su2_barL [source]

An isospin transformation moves the isospin index of a conjugate lepton-doublet symbol by the matrix itself, the index being fundamental.

```
lemma repGauge_su2_barL (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s w : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isFermionSector.barLComponent f l (s, w))
      = ∑ a, V.1 a w • h.isFermionSector.barLComponent f l (s, a)
```

lemma repGauge_u1_barL [source]

A hypercharge transformation scales a conjugate lepton-doublet symbol by the cube of the conjugate scalar, the conjugate lepton doublet carrying `6Y = -3`.

```
lemma repGauge_u1_barL (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (s w : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isFermionSector.barLComponent f l (s, w))
      = (star (t : ℂ)) ^ 3 • h.isFermionSector.barLComponent f l (s, w)
```

lemma repGauge_su3_e [source]

A colour transformation fixes a lepton-singlet symbol.

```
lemma repGauge_su3_e (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isFermionSector.eComponent f l s)
      = h.isFermionSector.eComponent f l s
```

lemma repGauge_su2_e [source]

An isospin transformation fixes a lepton-singlet symbol.

```
lemma repGauge_su2_e (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isFermionSector.eComponent f l s)
      = h.isFermionSector.eComponent f l s
```

lemma repGauge_u1_e [source]

A hypercharge transformation scales a lepton-singlet symbol by the sixth power of the scalar, the lepton singlet carrying `6Y = 6`.

```
lemma repGauge_u1_e (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (s : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isFermionSector.eComponent f l s)
      = (t : ℂ) ^ 6 • h.isFermionSector.eComponent f l s
```

lemma repGauge_su3_bare [source]

A colour transformation fixes a conjugate lepton-singlet symbol.

```
lemma repGauge_su3_bare (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.isFermionSector.bareComponent f l s)
      = h.isFermionSector.bareComponent f l s
```

lemma repGauge_su2_bare [source]

An isospin transformation fixes a conjugate lepton-singlet symbol.

```
lemma repGauge_su2_bare (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.isFermionSector.bareComponent f l s)
      = h.isFermionSector.bareComponent f l s
```

lemma repGauge_u1_bare [source]

A hypercharge transformation scales a conjugate lepton-singlet symbol by the sixth power of the conjugate scalar, the conjugate lepton singlet carrying `6Y = -6`.

```
lemma repGauge_u1_bare (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (s : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.isFermionSector.bareComponent f l s)
      = (star (t : ℂ)) ^ 6 • h.isFermionSector.bareComponent f l s
```

lemma higgs_mem_derivSubmodule [source]

A Higgs symbol lies in the Higgs derivative submodule.

```
lemma higgs_mem_derivSubmodule {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (i : Fin 2) :
    h.isHiggsSector.higgs l i ∈ h.isHiggsSector.derivSubmodule n
```

lemma barHiggs_mem_derivSubmodule [source]

A conjugate Higgs symbol lies in the Higgs derivative submodule.

```
lemma barHiggs_mem_derivSubmodule {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (i : Fin 2) :
    h.isHiggsSector.barHiggs l i ∈ h.isHiggsSector.derivSubmodule n
```

lemma range_d_le_derivSubmodule [source]

The range of a down-singlet symbol map lies in the fermion derivative submodule.

```
lemma range_d_le_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covD f l) ≤ h.isFermionSector.derivSubmodule n
```

lemma range_bard_le_derivSubmodule [source]

The range of a conjugate down-singlet symbol map lies in the fermion derivative submodule.

```
lemma range_bard_le_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covBarD f l) ≤ h.isFermionSector.derivSubmodule n
```

lemma range_u_le_derivSubmodule [source]

The range of an up-singlet symbol map lies in the fermion derivative submodule.

```
lemma range_u_le_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covU f l) ≤ h.isFermionSector.derivSubmodule n
```

lemma range_baru_le_derivSubmodule [source]

The range of a conjugate up-singlet symbol map lies in the fermion derivative submodule.

```
lemma range_baru_le_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covBarU f l) ≤ h.isFermionSector.derivSubmodule n
```

lemma range_Q_le_derivSubmodule [source]

The range of a quark-doublet symbol map lies in the fermion derivative submodule.

```
lemma range_Q_le_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covQ f l) ≤ h.isFermionSector.derivSubmodule n
```

lemma range_barQ_le_derivSubmodule [source]

The range of a conjugate quark-doublet symbol map lies in the fermion derivative submodule.

```
lemma range_barQ_le_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covBarQ f l) ≤ h.isFermionSector.derivSubmodule n
```

lemma range_L_le_derivSubmodule [source]

The range of a lepton-doublet symbol map lies in the fermion derivative submodule.

```
lemma range_L_le_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covL f l) ≤ h.isFermionSector.derivSubmodule n
```

lemma range_barL_le_derivSubmodule [source]

The range of a conjugate lepton-doublet symbol map lies in the fermion derivative submodule.

```
lemma range_barL_le_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covBarL f l) ≤ h.isFermionSector.derivSubmodule n
```

lemma range_e_le_derivSubmodule [source]

The range of a lepton-singlet symbol map lies in the fermion derivative submodule.

```
lemma range_e_le_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covE f l) ≤ h.isFermionSector.derivSubmodule n
```

lemma range_bare_le_derivSubmodule [source]

The range of a conjugate lepton-singlet symbol map lies in the fermion derivative submodule.

```
lemma range_bare_le_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covBarE f l) ≤ h.isFermionSector.derivSubmodule n
```

lemma dComponent_mem_derivSubmodule [source]

A `d` component lies in the fermion derivative submodule.

```
lemma dComponent_mem_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    h.isFermionSector.dComponent f l j ∈ h.isFermionSector.derivSubmodule n
```

lemma bardComponent_mem_derivSubmodule [source]

A `bard` component lies in the fermion derivative submodule.

```
lemma bardComponent_mem_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    h.isFermionSector.bardComponent f l j ∈ h.isFermionSector.derivSubmodule n
```

lemma uComponent_mem_derivSubmodule [source]

A `u` component lies in the fermion derivative submodule.

```
lemma uComponent_mem_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    h.isFermionSector.uComponent f l j ∈ h.isFermionSector.derivSubmodule n
```

lemma baruComponent_mem_derivSubmodule [source]

A `baru` component lies in the fermion derivative submodule.

```
lemma baruComponent_mem_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    h.isFermionSector.baruComponent f l j ∈ h.isFermionSector.derivSubmodule n
```

lemma QComponent_mem_derivSubmodule [source]

A `Q` component lies in the fermion derivative submodule.

```
lemma QComponent_mem_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3 × Fin 2) :
    h.isFermionSector.QComponent f l j ∈ h.isFermionSector.derivSubmodule n
```

lemma barQComponent_mem_derivSubmodule [source]

A `barQ` component lies in the fermion derivative submodule.

```
lemma barQComponent_mem_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3 × Fin 2) :
    h.isFermionSector.barQComponent f l j ∈ h.isFermionSector.derivSubmodule n
```

lemma LComponent_mem_derivSubmodule [source]

A `L` component lies in the fermion derivative submodule.

```
lemma LComponent_mem_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 2) :
    h.isFermionSector.LComponent f l j ∈ h.isFermionSector.derivSubmodule n
```

lemma barLComponent_mem_derivSubmodule [source]

A `barL` component lies in the fermion derivative submodule.

```
lemma barLComponent_mem_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 2) :
    h.isFermionSector.barLComponent f l j ∈ h.isFermionSector.derivSubmodule n
```

lemma eComponent_mem_derivSubmodule [source]

A `e` component lies in the fermion derivative submodule.

```
lemma eComponent_mem_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2) :
    h.isFermionSector.eComponent f l j ∈ h.isFermionSector.derivSubmodule n
```

lemma bareComponent_mem_derivSubmodule [source]

A `bare` component lies in the fermion derivative submodule.

```
lemma bareComponent_mem_derivSubmodule (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2) :
    h.isFermionSector.bareComponent f l j ∈ h.isFermionSector.derivSubmodule n
```

lemma mul_comm_of_le_derivSubmodule [source]

Two submodules of fermion derivative submodules commute, the fermion symbols anticommuting and a submodule being closed under negation.

```
lemma mul_comm_of_le_derivSubmodule {n m : ℕ} {V W : Submodule ℂ B}
    (hV : V ≤ h.isFermionSector.derivSubmodule n)
    (hW : W ≤ h.isFermionSector.derivSubmodule m) : V * W = W * V
```

lemma mul_mul_piece_swap [source]

Swapping the two fermion factors of a block leaves every weight piece unchanged: the two fermion factors commute as submodules, so the two orderings of the block are the same submodule. This is what makes the six transposed blocks of the mass-weight-eight decomposition a rewrite rather than six fresh classifications.

```
lemma mul_mul_piece_swap {VH VX VY : Submodule ℂ B} {n m : ℕ}
    (dH : GaugeWeightDecomposition repGauge VH)
    (dX : GaugeWeightDecomposition repGauge VX)
    (dY : GaugeWeightDecomposition repGauge VY)
    (hX : VX ≤ h.isFermionSector.derivSubmodule n)
    (hY : VY ≤ h.isFermionSector.derivSubmodule m) (w : GaugeWeight) :
    (GaugeWeightDecomposition.mul (d := dH)
        (d' := GaugeWeightDecomposition.mul (d := dX) (d' := dY))).piece w
      = (GaugeWeightDecomposition.mul (d := dH)
        (d' := GaugeWeightDecomposition.mul (d := dY) (d' := dX))).piece w
```

lemma mul_mul_swap_eq_neg [source]

Swapping the two fermion factors of a block negates it.

```
lemma mul_mul_swap_eq_neg {n m : ℕ} (a : B) {x y : B}
    (hx : x ∈ h.isFermionSector.derivSubmodule n)
    (hy : y ∈ h.isFermionSector.derivSubmodule m) :
    a * (y * x) = -(a * (x * y))
```

2.423 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.GaugeWeight

PhyslibAlpha/Particles/StandardModel/CovRealization/YukawaSector/GaugeWeight.lean on GitHub.

def sectorMassWeightEightGaugeWeight [source]

The gauge weight decomposition of the Yukawa sector at mass weight eight, transported along `sectorMassWeight_higgs_fermion_eight` from the product of the Higgs derivative submodule with two copies of the underived fermion towers.

```
@[implicit_reducible]
noncomputable def sectorMassWeightEightGaugeWeight :
    GaugeWeightDecomposition repGauge
      (h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8)
```

lemma piece_mul_sup_mul [source]

If the first fermion factor of a triple product is a join `VA ⊔ VB`, its weight-`w` piece splits along the join.

```
lemma piece_mul_sup_mul {VX VA VB VZ : Submodule ℂ B}
    (dX : GaugeWeightDecomposition repGauge VX) (dA : GaugeWeightDecomposition repGauge VA)
    (dB : GaugeWeightDecomposition repGauge VB) (dZ : GaugeWeightDecomposition repGauge VZ)
    (w : GaugeWeight) :
    (GaugeWeightDecomposition.mul (d := dX)
        (d' := GaugeWeightDecomposition.mul
          (d := GaugeWeightDecomposition.sup (d := dA) (d' := dB)) (d' := dZ))).piece w
      = (GaugeWeightDecomposition.mul (d := dX)
          (d' := GaugeWeightDecomposition.mul (d := dA) (d' := dZ))).piece w
        ⊔ (GaugeWeightDecomposition.mul (d := dX)
          (d' := GaugeWeightDecomposition.mul (d := dB) (d' := dZ))).piece w
```

lemma piece_mul_mul_sup [source]

If the second fermion factor of a triple product is a join `VA ⊔ VB`, its weight-`w` piece splits along the join.

```
lemma piece_mul_mul_sup {VX VY VA VB : Submodule ℂ B}
    (dX : GaugeWeightDecomposition repGauge VX) (dY : GaugeWeightDecomposition repGauge VY)
    (dA : GaugeWeightDecomposition repGauge VA) (dB : GaugeWeightDecomposition repGauge VB)
    (w : GaugeWeight) :
    (GaugeWeightDecomposition.mul (d := dX)
        (d' := GaugeWeightDecomposition.mul (d := dY)
          (d' := GaugeWeightDecomposition.sup (d := dA) (d' := dB)))).piece w
      = (GaugeWeightDecomposition.mul (d := dX)
            (d' := GaugeWeightDecomposition.mul (d := dY) (d' := dA))).piece w
        ⊔ (GaugeWeightDecomposition.mul (d := dX)
            (d' := GaugeWeightDecomposition.mul (d := dY) (d' := dB))).piece w
```

lemma mul_supp_hypercharge [source]

Two decompositions with constant hypercharge have a product of constant hypercharge, the sum of the two: the support of a product is the pointwise sum of the supports, and hypercharge is the fourth coordinate of a gauge weight.

```
lemma mul_supp_hypercharge {V V' : Submodule ℂ B}
    {dV : GaugeWeightDecomposition repGauge V} {dV' : GaugeWeightDecomposition repGauge V'}
    {hc hc' : ℤ} (hV : ∀ w ∈ dV.supp, w.2.2.2 = hc) (hV' : ∀ w ∈ dV'.supp, w.2.2.2 = hc') :
    ∀ w ∈ (GaugeWeightDecomposition.mul (d := dV) (d' := dV')).supp, w.2.2.2 = hc + hc'
```

lemma higgsSubmoduleGaugeWeight_hc [source]

The Higgs symbols carry hypercharge `-3`, independent of isospin and of the number of derivatives: the two weights in the support are `(0, 0, ∓1, -3)`.

```
lemma higgsSubmoduleGaugeWeight_hc (n : ℕ) :
    ∀ w ∈ (h.isHiggsSector.higgsSubmoduleGaugeWeight n).supp, w.2.2.2 = -3
```

lemma barHiggsSubmoduleGaugeWeight_hc [source]

The conjugate-Higgs symbols carry hypercharge `3`, independent of isospin and of the number of derivatives: the two weights in the support are `(0, 0, ±1, 3)`.

```
lemma barHiggsSubmoduleGaugeWeight_hc (n : ℕ) :
    ∀ w ∈ (h.isHiggsSector.barHiggsSubmoduleGaugeWeight n).supp, w.2.2.2 = 3
```

lemma mul_speciesGaugeWeight_mul_piece_zero_neg_three [source]

The hypercharge sieve against a Higgs symbol. If `dV` has constant hypercharge `-3`, as the Higgs symbols do, then of the hundred species pairings in a product of two fermion towers only the six whose hypercharges sum to `+3` survive at gauge weight zero: `d barQ`, `baru Q` and `barL e`, each in both orders. The other ninety-four pairings leave a nonzero hypercharge behind and so contribute nothing.

```
lemma mul_speciesGaugeWeight_mul_piece_zero_neg_three {V : Submodule ℂ B}
    {dV : GaugeWeightDecomposition repGauge V} (hV : ∀ w ∈ dV.supp, w.2.2.2 = -3)
    {n m : ℕ} (f f' : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3)
    (l' : Fin m → Fin 1 ⊕ Fin 3) :
    (GaugeWeightDecomposition.mul (d := dV)
```

```
        (d' := GaugeWeightDecomposition.mul
          (d := h.isFermionSector.speciesGaugeWeight f l)
          (d' := h.isFermionSector.speciesGaugeWeight f' l'))).piece 0
      = (GaugeWeightDecomposition.mul (d := dV)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightD f l)
              (d' := h.isFermionSector.rangeGaugeWeightBarQ f' l'))).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := dV)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightBarQ f l)
              (d' := h.isFermionSector.rangeGaugeWeightD f' l'))).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := dV)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightBarU f l)
              (d' := h.isFermionSector.rangeGaugeWeightQ f' l'))).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := dV)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightQ f l)
              (d' := h.isFermionSector.rangeGaugeWeightBarU f' l'))).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := dV)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightBarL f l)
              (d' := h.isFermionSector.rangeGaugeWeightE f' l'))).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := dV)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightE f l)
              (d' := h.isFermionSector.rangeGaugeWeightBarL f' l'))).piece 0
```

lemma mul_speciesGaugeWeight_mul_piece_zero_pos_three [source]

The hypercharge sieve against a conjugate-Higgs symbol. If `dV` has constant hypercharge `3`, as the conjugate-Higgs symbols do, then of the hundred species pairings only the six whose hypercharges sum to `-3` survive at gauge weight zero: `bard Q`, `u barQ` and `L bare`, each in both orders.

```
lemma mul_speciesGaugeWeight_mul_piece_zero_pos_three {V : Submodule ℂ B}
    {dV : GaugeWeightDecomposition repGauge V} (hV : ∀ w ∈ dV.supp, w.2.2.2 = 3)
    {n m : ℕ} (f f' : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3)
    (l' : Fin m → Fin 1 ⊕ Fin 3) :
    (GaugeWeightDecomposition.mul (d := dV)
        (d' := GaugeWeightDecomposition.mul
          (d := h.isFermionSector.speciesGaugeWeight f l)
          (d' := h.isFermionSector.speciesGaugeWeight f' l'))).piece 0
      = (GaugeWeightDecomposition.mul (d := dV)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightBarD f l)
              (d' := h.isFermionSector.rangeGaugeWeightQ f' l'))).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := dV)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightQ f l)
              (d' := h.isFermionSector.rangeGaugeWeightBarD f' l'))).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := dV)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightU f l)
              (d' := h.isFermionSector.rangeGaugeWeightBarQ f' l'))).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := dV)
            (d' := GaugeWeightDecomposition.mul
```

```
          (d := h.isFermionSector.rangeGaugeWeightBarQ f l)
          (d' := h.isFermionSector.rangeGaugeWeightU f' l'))).piece 0
    ⊔ (GaugeWeightDecomposition.mul (d := dV)
        (d' := GaugeWeightDecomposition.mul
          (d := h.isFermionSector.rangeGaugeWeightL f l)
          (d' := h.isFermionSector.rangeGaugeWeightBarE f' l'))).piece 0
    ⊔ (GaugeWeightDecomposition.mul (d := dV)
        (d' := GaugeWeightDecomposition.mul
          (d := h.isFermionSector.rangeGaugeWeightBarE f l)
          (d' := h.isFermionSector.rangeGaugeWeightL f' l'))).piece 0
```

lemma sectorMassWeightEightGaugeWeight_piece_zero [source]

The weight-zero piece of the Yukawa sector at mass weight eight: the join, over pairs of families, of the twelve blocks that hypercharge allows. Of the two hundred ways of choosing one Higgs symbol and two fermion species, only these twelve have vanishing total hypercharge; the remaining one hundred and eighty-eight are killed by `mul_speciesGaugeWeight_mul_piece_zero_neg_three` *and* `mul_speciesGaugeWeight_mul_piece_zero_pos_thr`

```
lemma sectorMassWeightEightGaugeWeight_piece_zero :
    h.sectorMassWeightEightGaugeWeight.piece 0
      = ⨆ (f : Fin 3) (f' : Fin 3),
        (GaugeWeightDecomposition.mul
            (d := h.isHiggsSector.higgsSubmoduleGaugeWeight 0)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightD f ![])
              (d' := h.isFermionSector.rangeGaugeWeightBarQ f' ![]))).piece 0
      ⊔ (GaugeWeightDecomposition.mul
            (d := h.isHiggsSector.higgsSubmoduleGaugeWeight 0)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightBarQ f ![])
              (d' := h.isFermionSector.rangeGaugeWeightD f' ![]))).piece 0
      ⊔ (GaugeWeightDecomposition.mul
            (d := h.isHiggsSector.higgsSubmoduleGaugeWeight 0)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightBarU f ![])
              (d' := h.isFermionSector.rangeGaugeWeightQ f' ![]))).piece 0
      ⊔ (GaugeWeightDecomposition.mul
            (d := h.isHiggsSector.higgsSubmoduleGaugeWeight 0)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightQ f ![])
              (d' := h.isFermionSector.rangeGaugeWeightBarU f' ![]))).piece 0
      ⊔ (GaugeWeightDecomposition.mul
            (d := h.isHiggsSector.higgsSubmoduleGaugeWeight 0)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightBarL f ![])
              (d' := h.isFermionSector.rangeGaugeWeightE f' ![]))).piece 0
      ⊔ (GaugeWeightDecomposition.mul
            (d := h.isHiggsSector.higgsSubmoduleGaugeWeight 0)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightE f ![])
              (d' := h.isFermionSector.rangeGaugeWeightBarL f' ![]))).piece 0
      ⊔ (GaugeWeightDecomposition.mul
            (d := h.isHiggsSector.barHiggsSubmoduleGaugeWeight 0)
            (d' := GaugeWeightDecomposition.mul
              (d := h.isFermionSector.rangeGaugeWeightBarD f ![])
              (d' := h.isFermionSector.rangeGaugeWeightQ f' ![]))).piece 0
```

```
      ⊔ (GaugeWeightDecomposition.mul
          (d := h.isHiggsSector.barHiggsSubmoduleGaugeWeight 0)
```

lemma mem_sectorMassWeightEight_piece_zero_sup_of_invariant [source]

A gauge-invariant element of the Yukawa sector at mass weight eight joined with a torus-stable `S` lies in the weight-zero piece joined with `S`, so the twelve blocks of `sectorMassWeightEightGaugeWeight_piece_zero` are all that a Yukawa invariant can be built from.

```
lemma mem_sectorMassWeightEight_piece_zero_sup_of_invariant {S : Submodule ℂ B}
    (hS : ∀ (i : Fin 4) (y : B), y ∈ S → repGauge (gaugeTorusGen i) y ∈ S) {x : B}
    (hx : x ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8 ⊔ S)
    (hinv : ∀ g : GaugeGroupI, repGauge g x = x) :
    x ∈ h.sectorMassWeightEightGaugeWeight.piece 0 ⊔ S
```

2.424 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.MassDimEight

PhyslibAlpha/Particles/StandardModel/CovRealization/YukawaSector/MassDimEight.lean on GitHub.

lemma higgsSubmodule_zero_le [source]

The Higgs submodule without derivatives lies in the span of the Higgs components.

```
lemma higgsSubmodule_zero_le :
    h.isHiggsSector.higgsSubmodule 0
      ≤ Submodule.span ℂ (Set.range (h.isHiggsSector.higgs ![]))
```

lemma barHiggsSubmodule_zero_le [source]

The conjugate Higgs submodule without derivatives lies in the span of the conjugate Higgs components.

```
lemma barHiggsSubmodule_zero_le :
    h.isHiggsSector.barHiggsSubmodule 0
      ≤ Submodule.span ℂ (Set.range (h.isHiggsSector.barHiggs ![]))
```

lemma range_d_eq [source]

The range of the down-singlet symbol map is the span of its components.

```
lemma range_d_eq (f : Fin 3) :
    LinearMap.range (h.covD f (![] : Fin 0 → Fin 1 ⊕ Fin 3))
      = Submodule.span ℂ (Set.range (h.isFermionSector.dComponent f ![]))
```

lemma range_bard_eq [source]

The range of the conjugate down-singlet symbol map is the span of its components.

```
lemma range_bard_eq (f : Fin 3) :
    LinearMap.range (h.covBarD f (![] : Fin 0 → Fin 1 ⊕ Fin 3))
      = Submodule.span ℂ (Set.range (h.isFermionSector.bardComponent f ![]))
```

lemma range_u_eq [source]

The range of the up-singlet symbol map is the span of its components.

```
lemma range_u_eq (f : Fin 3) :
    LinearMap.range (h.covU f (![] : Fin 0 → Fin 1 ⊕ Fin 3))
      = Submodule.span ℂ (Set.range (h.isFermionSector.uComponent f ![]))
```

lemma range_baru_eq [source]

The range of the conjugate up-singlet symbol map is the span of its components.

```
lemma range_baru_eq (f : Fin 3) :
    LinearMap.range (h.covBarU f (![] : Fin 0 → Fin 1 ⊕ Fin 3))
      = Submodule.span ℂ (Set.range (h.isFermionSector.baruComponent f ![]))
```

lemma range_Q_eq [source]

The range of the quark-doublet symbol map is the span of its components.

```
lemma range_Q_eq (f : Fin 3) :
    LinearMap.range (h.covQ f (![] : Fin 0 → Fin 1 ⊕ Fin 3))
      = Submodule.span ℂ (Set.range (h.isFermionSector.QComponent f ![]))
```

lemma range_barQ_eq [source]

The range of the conjugate quark-doublet symbol map is the span of its components.

```
lemma range_barQ_eq (f : Fin 3) :
    LinearMap.range (h.covBarQ f (![] : Fin 0 → Fin 1 ⊕ Fin 3))
      = Submodule.span ℂ (Set.range (h.isFermionSector.barQComponent f ![]))
```

lemma range_L_eq [source]

The range of the lepton-doublet symbol map is the span of its components.

```
lemma range_L_eq (f : Fin 3) :
    LinearMap.range (h.covL f (![] : Fin 0 → Fin 1 ⊕ Fin 3))
      = Submodule.span ℂ (Set.range (h.isFermionSector.LComponent f ![]))
```

lemma range_barL_eq [source]

The range of the conjugate lepton-doublet symbol map is the span of its components.

```
lemma range_barL_eq (f : Fin 3) :
    LinearMap.range (h.covBarL f (![] : Fin 0 → Fin 1 ⊕ Fin 3))
      = Submodule.span ℂ (Set.range (h.isFermionSector.barLComponent f ![]))
```

lemma range_e_eq [source]

The range of the lepton-singlet symbol map is the span of its components.

```
lemma range_e_eq (f : Fin 3) :
    LinearMap.range (h.covE f (![] : Fin 0 → Fin 1 ⊕ Fin 3))
      = Submodule.span ℂ (Set.range (h.isFermionSector.eComponent f ![]))
```

lemma range_bare_eq [source]

The range of the conjugate lepton-singlet symbol map is the span of its components.

```
lemma range_bare_eq (f : Fin 3) :
    LinearMap.range (h.covBarE f (![] : Fin 0 → Fin 1 ⊕ Fin 3))
      = Submodule.span ℂ (Set.range (h.isFermionSector.bareComponent f ![]))
```

lemma isStableUnder_higgsSubmodule_zero [source]

The Higgs submodule without derivatives is carried into itself by both groups.

```
lemma isStableUnder_higgsSubmodule_zero :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (h.isHiggsSector.higgsSubmodule 0)
```

lemma isStableUnder_barHiggsSubmodule_zero [source]

The conjugate Higgs submodule without derivatives is carried into itself by both groups.

```
lemma isStableUnder_barHiggsSubmodule_zero :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (h.isHiggsSector.barHiggsSubmodule 0)
```

lemma isStableUnder_range_d [source]

The range of the down-singlet symbol map is carried into itself by both groups.

```
lemma isStableUnder_range_d (f : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (LinearMap.range (h.covD f (![] : Fin 0 → Fin 1 ⊕ Fin 3)))
```

lemma isStableUnder_range_bard [source]

The range of the conjugate down-singlet symbol map is carried into itself by both groups.

```
lemma isStableUnder_range_bard (f : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (LinearMap.range (h.covBarD f (![] : Fin 0 → Fin 1 ⊕ Fin 3)))
```

lemma isStableUnder_range_u [source]

The range of the up-singlet symbol map is carried into itself by both groups.

```
lemma isStableUnder_range_u (f : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (LinearMap.range (h.covU f (![] : Fin 0 → Fin 1 ⊕ Fin 3)))
```

lemma isStableUnder_range_baru [source]

The range of the conjugate up-singlet symbol map is carried into itself by both groups.

```
lemma isStableUnder_range_baru (f : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (LinearMap.range (h.covBarU f (![] : Fin 0 → Fin 1 ⊕ Fin 3)))
```

lemma isStableUnder_range_Q [source]

The range of the quark-doublet symbol map is carried into itself by both groups.

```
lemma isStableUnder_range_Q (f : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (LinearMap.range (h.covQ f (![] : Fin 0 → Fin 1 ⊕ Fin 3)))
```

lemma isStableUnder_range_barQ [source]

The range of the conjugate quark-doublet symbol map is carried into itself by both groups.

```
lemma isStableUnder_range_barQ (f : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (LinearMap.range (h.covBarQ f (![] : Fin 0 → Fin 1 ⊕ Fin 3)))
```

lemma isStableUnder_range_L [source]

The range of the lepton-doublet symbol map is carried into itself by both groups.

```
lemma isStableUnder_range_L (f : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (LinearMap.range (h.covL f (![] : Fin 0 → Fin 1 ⊕ Fin 3)))
```

lemma isStableUnder_range_barL [source]

The range of the conjugate lepton-doublet symbol map is carried into itself by both groups.

```
lemma isStableUnder_range_barL (f : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (LinearMap.range (h.covBarL f (![] : Fin 0 → Fin 1 ⊕ Fin 3)))
```

lemma isStableUnder_range_e [source]

The range of the lepton-singlet symbol map is carried into itself by both groups.

```
lemma isStableUnder_range_e (f : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (LinearMap.range (h.covE f (![] : Fin 0 → Fin 1 ⊕ Fin 3)))
```

lemma isStableUnder_range_bare [source]

The range of the conjugate lepton-singlet symbol map is carried into itself by both groups.

```
lemma isStableUnder_range_bare (f : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (LinearMap.range (h.covBarE f (![] : Fin 0 → Fin 1 ⊕ Fin 3)))
```

def downBlockSubmodule [source]

The submodule of the down-type block `H d barQ` of a family pair.

```
noncomputable def downBlockSubmodule (f f' : Fin 3) : Submodule ℂ B
```

def upBlockSubmodule [source]

The submodule of the up-type block `H baru Q` of a family pair.

```
noncomputable def upBlockSubmodule (f f' : Fin 3) : Submodule ℂ B
```

def leptonBlockSubmodule [source]

The submodule of the charged-lepton block `H barL e` of a family pair.

```
noncomputable def leptonBlockSubmodule (f f' : Fin 3) : Submodule ℂ B
```

def barDownBlockSubmodule [source]

The submodule of the conjugate down-type block `barH bard Q` of a family pair.

```
noncomputable def barDownBlockSubmodule (f f' : Fin 3) : Submodule ℂ B
```

def barUpBlockSubmodule [source]

The submodule of the conjugate up-type block `barH u barQ` of a family pair.

```
noncomputable def barUpBlockSubmodule (f f' : Fin 3) : Submodule ℂ B
```

def barLeptonBlockSubmodule [source]

The submodule of the conjugate charged-lepton block `barH L bare` of a family pair.

```
noncomputable def barLeptonBlockSubmodule (f f' : Fin 3) : Submodule ℂ B
```

lemma isStableUnder_downBlockSubmodule [source]

The down-type block submodule is carried into itself by both groups.

```
lemma isStableUnder_downBlockSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.downBlockSubmodule f f')
```

lemma isStableUnder_upBlockSubmodule [source]

The up-type block submodule is carried into itself by both groups.

```
lemma isStableUnder_upBlockSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.upBlockSubmodule f f')
```

lemma isStableUnder_leptonBlockSubmodule [source]

The charged-lepton block submodule is carried into itself by both groups.

```
lemma isStableUnder_leptonBlockSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.leptonBlockSubmodule f f')
```

lemma isStableUnder_barDownBlockSubmodule [source]

The conjugate down-type block submodule is carried into itself by both groups.

```
lemma isStableUnder_barDownBlockSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.barDownBlockSubmodule f f')
```

lemma isStableUnder_barUpBlockSubmodule [source]

The conjugate up-type block submodule is carried into itself by both groups.

```
lemma isStableUnder_barUpBlockSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.barUpBlockSubmodule f f')
```

lemma isStableUnder_barLeptonBlockSubmodule [source]

The conjugate charged-lepton block submodule is carried into itself by both groups.

```
lemma isStableUnder_barLeptonBlockSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.barLeptonBlockSubmodule f f')
```

def blockSubmodule [source]

The join of the six block submodules over the nine family pairs: what the weight-zero piece of the Yukawa sector at mass weight eight is contained in.

```
noncomputable def blockSubmodule : Submodule ℂ B
```

lemma sectorMassWeightEightGaugeWeight_piece_zero_le [source]

The weight-zero piece of the Yukawa sector at mass weight eight lies in the join of the six block submodules over the nine family pairs. The twelve blocks of `sectorMassWeightEightGaugeWeight_pi` *become six because the two fermion factors of a block commute, so the transposed block of* `(f, f')` *is the block of* `(f', f)`*; and the weight refinement inside a block is dropped, hypercharge having already done its work and colour, isospin and Lorentz being what decide the rest.*

```
lemma sectorMassWeightEightGaugeWeight_piece_zero_le :
    h.sectorMassWeightEightGaugeWeight.piece 0 ≤ h.blockSubmodule
```

lemma reducesInvariantsTo_downYukawa [source]

The down-type block reduces to the down-type Yukawa term.

```
lemma reducesInvariantsTo_downYukawa (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.downBlockSubmodule f f')
      (ℂ • h.downYukawa f f')
```

lemma reducesInvariantsTo_upYukawa [source]

The up-type block reduces to the up-type Yukawa term. Isospin is contracted by the antisymmetric symbol here, the Higgs symbol and the quark doublet both carrying the anti-fundamental.

```
lemma reducesInvariantsTo_upYukawa (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.upBlockSubmodule f f')
      (ℂ • h.upYukawa f f')
```

lemma reducesInvariantsTo_leptonYukawa [source]

The charged-lepton block reduces to the charged-lepton Yukawa term. Its colour stage is the trivial one: the three symbols carry no colour index between them, so the block is fixed by the colour factor and the stage reduces it to itself.

```
lemma reducesInvariantsTo_leptonYukawa (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.leptonBlockSubmodule f f')
      (ℂ • h.leptonYukawa f f')
```

lemma reducesInvariantsTo_barDownYukawa [source]

The conjugate down-type block reduces to the conjugate down-type Yukawa term.

```
lemma reducesInvariantsTo_barDownYukawa (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.barDownBlockSubmodule f f')
      (ℂ • h.barDownYukawa f f')
```

lemma reducesInvariantsTo_barUpYukawa [source]

The conjugate up-type block reduces to the conjugate up-type Yukawa term.

```
lemma reducesInvariantsTo_barUpYukawa (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.barUpBlockSubmodule f f')
      (ℂ • h.barUpYukawa f f')
```

lemma reducesInvariantsTo_barLeptonYukawa [source]

The conjugate charged-lepton block reduces to the conjugate charged-lepton Yukawa term, again with the trivial colour stage.

```
lemma reducesInvariantsTo_barLeptonYukawa (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.barLeptonBlockSubmodule f f')
      (ℂ • h.barLeptonYukawa f f')
```

lemma isFixedBy_yukawaSpan [source]

The Yukawa span is fixed pointwise by both groups, `yukawaSpan_le_inf` placing it inside both spaces of invariants.

```
lemma isFixedBy_yukawaSpan : IsFixedBy (gaugeLorentzMaps repGauge repLorentz) h.yukawaSpan
```

lemma span_downYukawa_le_yukawaSpan [source]

The line through a down-type Yukawa term lies in the Yukawa span.

```
lemma span_downYukawa_le_yukawaSpan (f f' : Fin 3) :
    ℂ • h.downYukawa f f' ≤ h.yukawaSpan
```

lemma span_upYukawa_le_yukawaSpan [source]

The line through an up-type Yukawa term lies in the Yukawa span.

```
lemma span_upYukawa_le_yukawaSpan (f f' : Fin 3) :
    ℂ • h.upYukawa f f' ≤ h.yukawaSpan
```

lemma span_leptonYukawa_le_yukawaSpan [source]

The line through a charged-lepton Yukawa term lies in the Yukawa span.

```
lemma span_leptonYukawa_le_yukawaSpan (f f' : Fin 3) :
    ℂ • h.leptonYukawa f f' ≤ h.yukawaSpan
```

lemma span_barDownYukawa_le_yukawaSpan [source]

The line through a conjugate down-type Yukawa term lies in the Yukawa span.

```
lemma span_barDownYukawa_le_yukawaSpan (f f' : Fin 3) :
    ℂ • h.barDownYukawa f f' ≤ h.yukawaSpan
```

lemma span_barUpYukawa_le_yukawaSpan [source]

The line through a conjugate up-type Yukawa term lies in the Yukawa span.

```
lemma span_barUpYukawa_le_yukawaSpan (f f' : Fin 3) :
    ℂ • h.barUpYukawa f f' ≤ h.yukawaSpan
```

lemma span_barLeptonYukawa_le_yukawaSpan [source]

The line through a conjugate charged-lepton Yukawa term lies in the Yukawa span.

```
lemma span_barLeptonYukawa_le_yukawaSpan (f f' : Fin 3) :
    ℂ • h.barLeptonYukawa f f' ≤ h.yukawaSpan
```

lemma reducesInvariantsTo_yukawaSpan [source]

The join of the six block submodules over the nine family pairs reduces to the Yukawa span: the fifty-four blocks are taken one at a time, each in turn joining the error term of the others, which is what their stability is for.

```
lemma reducesInvariantsTo_yukawaSpan :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) h.blockSubmodule h.yukawaSpan
```

lemma reducesInvariantsTo_sectorMassWeight_higgs_fermion_eight [source]

The Yukawa sector at mass weight eight reduces, for the gauge and Lorentz groups together, to the Yukawa span. Hypercharge puts an invariant in the weight-zero piece, the piece lies in the six block submodules, and colour, isospin and Lorentz reduce each block to its Yukawa term.

```
lemma reducesInvariantsTo_sectorMassWeight_higgs_fermion_eight :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz)
      (h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8) h.yukawaSpan
```

lemma mem_sectorMassWeight_higgs_fermion_eight_sup_and_gauge_lorentz_invariant_iff [source]

The classification of the Yukawa sector at mass weight eight as an equivalence: an element of the sector joined with a submodule `S` stable under both groups is fixed by both groups exactly when it is a combination of the six Yukawa couplings over the nine family pairs up to a remainder in `S` fixed by both groups.

```
lemma mem_sectorMassWeight_higgs_fermion_eight_sup_and_gauge_lorentz_invariant_iff
    (S : Submodule ℂ B) (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 8 ⊔ S
        ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ h.yukawaSpan
```

2.425 PhyslibAlpha.Particles.StandardModel.CovRealization.YukawaSector.MassDimLTEight

PhyslibAlpha/Particles/StandardModel/CovRealization/YukawaSector/MassDimLTEight.lean on GitHub.

lemma higgsFermion_le_centreEigenspace [source]

A Higgs derivative submodule against a fermion one is of half-integer spin: `+1` times `-1`.

```
private lemma higgsFermion_le_centreEigenspace (a b : ℕ) :
    h.isHiggsSector.derivSubmodule a * h.isFermionSector.derivSubmodule b
      ≤ centreEigenspace repLorentz (-1)
```

lemma higgsSqFermion_le_centreEigenspace [source]

Two Higgs derivative submodules against a fermion one is of half-integer spin: `+1` times `+1` times `-1`.

```
private lemma higgsSqFermion_le_centreEigenspace (a b c : ℕ) :
    h.isHiggsSector.derivSubmodule a * h.isHiggsSector.derivSubmodule b
        * h.isFermionSector.derivSubmodule c
      ≤ centreEigenspace repLorentz (-1)
```

lemma mem_of_lorentz_invariant_sectorMassWeight_higgs_fermion_five_sup [source]

Mass weight five carries no Lorentz invariant modulo a Lorentz-stable submodule: a Lorentz invariant of `sectorMassWeight {higgs, fermion} 5 ⊔ S` lies in `S`. The weight is one Higgs field against the underived fermion towers, of half-integer spin.

```
lemma mem_of_lorentz_invariant_sectorMassWeight_higgs_fermion_five_sup (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 5 ⊔ S)
    (hL : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_lorentz_invariant_sectorMassWeight_higgs_fermion_seven_sup [source]

Mass weight seven carries no Lorentz invariant modulo a Lorentz-stable submodule: a Lorentz invariant of `sectorMassWeight {higgs, fermion} 7 ⊔ S` lies in `S`. Each of the three products making up the weight has a single fermion factor, so each is of half-integer spin and so is their join.

```
lemma mem_of_lorentz_invariant_sectorMassWeight_higgs_fermion_seven_sup
    (S : Submodule ℂ B) (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} 7 ⊔ S)
    (hL : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_lorentz_invariant_sectorMassWeight_higgs_fermion_lt_eight_sup [source]

Below mass weight eight the Yukawa sector carries no Lorentz invariant: a Lorentz invariant of `sectorMassWeight {higgs, fermion} w ⊔ S` for `w < 8` lies in `S`. Weights below five and weight six are trivial submodules, and weights five and seven are section B.

```
lemma mem_of_lorentz_invariant_sectorMassWeight_higgs_fermion_lt_eight_sup (w : ℕ)
    (hw : w < 8) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} w ⊔ S)
    (hL : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_sectorMassWeight_higgs_fermion_lt_eight_sup_and_gauge_lorentz_invariant_iff [source]

The classification below mass weight eight as an equivalence, in the shape of the gauge-sector statement `mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff`: an element of `sectorMassWeight {higgs, fermion} w ⊔ S` for `w < 8` is fixed by both groups exactly when it is itself an element of `S` fixed by both groups. Gauge stability of `S` is not needed, and neither is gauge invariance of `x`: the forward direction is the spin parity argument, which uses the Lorentz group alone.

```
lemma mem_sectorMassWeight_higgs_fermion_lt_eight_sup_and_gauge_lorentz_invariant_iff
    (w : ℕ) (hw : w < 8) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} w ⊔ S
        ∧ (∀ g : GaugeGroupI, repGauge g x = x) ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x = y
```

lemma mem_sectorMassWeight_higgs_fermion_lt_eight_sup_and_gauge_lorentz_invariant_iff_mem [source]

The same classification without the existential: below mass weight eight an element of `sectorMassWeight {higgs, fermion} w ⊔ S` fixed by both groups is an element of `S` fixed by both groups, and conversely.

```
lemma mem_sectorMassWeight_higgs_fermion_lt_eight_sup_and_gauge_lorentz_invariant_iff_mem
    (w : ℕ) (hw : w < 8) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.sectorMassWeight {GeneratorClass.higgs, GeneratorClass.fermion} w ⊔ S
        ∧ (∀ g : GaugeGroupI, repGauge g x = x) ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ (x ∈ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
          ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
```

2.426 `PhyslibAlpha.Particles.StandardModel.Fermions.DownSinglet.Basic`

PhyslibAlpha/Particles/StandardModel/Fermions/DownSinglet/Basic.lean on GitHub.

`def basis` [source]

A basis on the down singlets.

```
noncomputable def basis : Module.Basis (Fin 2 × Fin 3) ℂ DownSinglet
```

`instance Module.Finite ℂ DownSinglet` [source]

```
instance : Module.Finite ℂ DownSinglet
```

`instance Module.Free ℂ DownSinglet` [source]

```
instance : Module.Free ℂ DownSinglet
```

`def jetValLinEquiv` [source]

Absorbs the jet ring into the colour index: a jet of a down-type singlet is the same thing as a right-handed Weyl spinor tensored with a `SpaceTimeAlgebra`-valued colour vector,

`SpaceTimeAlgebra ⊗[ℂ] DownSinglet ≃ RightHandedWeyl ⊗[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 3)`.

```
noncomputable def jetValLinEquiv :
    SpaceTimeAlgebra ⊗[ℂ] DownSinglet ≃ₗ[ℂ]
      Fermion.RightHandedWeyl ⊗[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 3)
```

`def repJetGaugeGroupI` [source]

The `(3, 1)_{-2}` action of the jet gauge group on the jet space of the down-type singlet. Through `jetValLinEquiv` the colour matrix of the gauge jet, carrying the `-2` hypercharge phase `(star u) ^ 2`, acts `SpaceTimeAlgebra`-linearly on the colour factor by matrix-vector multiplication, while the Weyl factor is untouched.

Both monoid laws come from bundled algebra maps — `Matrix.toLpLinAlgEquiv` and `Module.End.lTensorAlgHom` are morphisms of algebras — so only the multiplicativity of the colour-times-hypercharge matrix itself is checked. Note `Matrix.toLpLinAlgEquiv 2` is the same map as the `Matrix.toEuclideanLin` used by `repGaugeGroupI`, which

is an abbreviation for `Matrix.toLpLin 2 2`, taken at the `CommRing` generality that `SpaceTimeAlgebra` needs.

```
noncomputable def repJetGaugeGroupI :
    Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] DownSinglet) where
```

lemma jetValLinEquiv_smul [source]

The identification of the jets of the down-type singlet intertwines multiplication by a scalar jet with the `SpaceTimeAlgebra`-scalar action on the colour coordinates.

```
lemma jetValLinEquiv_smul (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] DownSinglet) :
    jetValLinEquiv (χ • z)
      = Module.End.lTensorAlgHom ℂ (EuclideanSpace SpaceTimeAlgebra (Fin 3))
          Fermion.RightHandedWeyl
          ((LinearMap.lsmul SpaceTimeAlgebra
              (EuclideanSpace SpaceTimeAlgebra (Fin 3)) χ).restrictScalars ℂ)
          (jetValLinEquiv z)
```

lemma repJetGaugeGroupI_smul [source]

***The jet gauge action on the jets of the down-type singlet is fibrewise**: it commutes with multiplication by scalar jets, acting on the values of the field over the identity on spacetime.*

```
lemma repJetGaugeGroupI_smul (U : JetGaugeGroupI) (χ : SpaceTimeAlgebra)
    (z : SpaceTimeAlgebra ⊗[ℂ] DownSinglet) :
    repJetGaugeGroupI U (χ • z) = χ • repJetGaugeGroupI U z
```

lemma repJetGaugeGroupI_ofConstant [source]

On jets of constant gauge transformations the jet action reduces to the global gauge action on the fibre: the `(3, 1)_{-2}` action on the down-singlet factor, and the trivial action on the jet ring.

```
lemma repJetGaugeGroupI_ofConstant (g : GaugeGroupI) :
    repJetGaugeGroupI (JetGaugeGroupI.ofConstant g) =
      TensorProduct.map LinearMap.id (repGaugeGroupI g)
```

lemma basis_eq_mk [source]

The down-singlet basis vector as an explicit spinor–colour tensor.

```
lemma basis_eq_mk (k : Fin 2) (c : Fin 3) : basis (k, c) =
    ⟨Fermion.RightHandedWeyl.basis k ⊗ₜ[ℂ] EuclideanSpace.basisFun (Fin 3) ℂ c⟩
```

lemma repLorentzGroup_apply_basis [source]

The Lorentz action on the down-singlet basis: the colour index is inert and the spinor index transforms by the entrywise conjugate matrix.

```
lemma repLorentzGroup_apply_basis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3) :
    repLorentzGroup Λ (basis j) = ∑ β, star (Λ.1 β j.1) • basis (β, j.2)
```

lemma repLorentzGroup_dual_dualBasis [source]

The down-singlet coordinate functionals transform contragrediently, by the entrywise conjugate of the inverse matrix.

```
lemma repLorentzGroup_dual_dualBasis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3) :
    repLorentzGroup.dual Λ (basis.dualBasis j) =
      ∑ β, star ((Λ⁻¹).1 j.1 β) • basis.dualBasis (β, j.2)
```

lemma repLorentzGroup_conj_apply_basis [source]

The Lorentz action on the conjugate down-singlet basis: the coefficients are the conjugates of those of the down-singlet action, that is, the matrix itself.

```
lemma repLorentzGroup_conj_apply_basis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3) :
    repLorentzGroup.conj Λ ((Basis.conj basis) j)
      = ∑ β, Λ.1 β j.1 • (Basis.conj basis) (β, j.2)
```

lemma repLorentzGroup_conj_dual_dualBasis [source]

The conjugate down-singlet coordinate functionals transform by the inverse matrix.

```
lemma repLorentzGroup_conj_dual_dualBasis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3) :
    repLorentzGroup.conj.dual Λ ((Basis.conj basis).dualBasis j) =
      ∑ β, (Λ⁻¹).1 j.1 β • (Basis.conj basis).dualBasis (β, j.2)
```

lemma repLorentzGroup_neg_one [source]

***The centre of `SL(2,ℂ)` acts on the down-singlet space by `-1`**: the value space carries a single Weyl-spinor index, and `-1` is not the identity on a half-integer spin.*

```
lemma repLorentzGroup_neg_one : repLorentzGroup (-1) = -LinearMap.id
```

lemma repLorentzGroup_conj_neg_one [source]

The centre acts on the conjugate down-singlet space by `-1` as well: conjugation does not move a real sign.

```
lemma repLorentzGroup_conj_neg_one : repLorentzGroup.conj (-1) = -LinearMap.id
```

lemma repGaugeGroupI_apply_basis [source]

The gauge action on the down-singlet basis: the spinor index is inert and the colour index transforms by the `SU(3)` matrix, scaled by the hypercharge factor.

```
lemma repGaugeGroupI_apply_basis (g : GaugeGroupI) (j : Fin 2 × Fin 3) :
    repGaugeGroupI g (basis j) =
      ∑ c, (star g.toU1.1 ^ 2 * g.toSU3.1 c j.2) • basis (j.1, c)
```

lemma repGaugeGroupI_dual_dualBasis [source]

The down-singlet coordinate functionals carry the contragredient gauge action: the hypercharge and `SU(3)` factors of the inverse group element, transposed.

```
lemma repGaugeGroupI_dual_dualBasis (g : GaugeGroupI) (j : Fin 2 × Fin 3) :
    repGaugeGroupI.dual g (basis.dualBasis j) =
      ∑ c, (star (g⁻¹).toU1.1 ^ 2 * (g⁻¹).toSU3.1 j.2 c) • basis.dualBasis (j.1, c)
```

lemma repGaugeGroupI_conj_apply_basis [source]

The gauge action on the conjugate down-singlet basis: the coefficients of the down-singlet action, conjugated.

```
lemma repGaugeGroupI_conj_apply_basis (g : GaugeGroupI) (j : Fin 2 × Fin 3) :
    repGaugeGroupI.conj g ((Basis.conj basis) j) =
      ∑ c, star (star g.toU1.1 ^ 2 * g.toSU3.1 c j.2) • (Basis.conj basis) (j.1, c)
```

lemma repGaugeGroupI_conj_dual_dualBasis [source]

The conjugate down-singlet coordinate functionals carry the conjugate of the contragredient gauge action.

```
lemma repGaugeGroupI_conj_dual_dualBasis (g : GaugeGroupI) (j : Fin 2 × Fin 3) :
    repGaugeGroupI.conj.dual g ((Basis.conj basis).dualBasis j) =
      ∑ c, star (star (g⁻¹).toU1.1 ^ 2 * (g⁻¹).toSU3.1 j.2 c) •
        (Basis.conj basis).dualBasis (j.1, c)
```

def valueGaugeWeight [source]

The gauge weight of the down-singlet basis: the colour weights and hypercharge `-2`.

```
def DownSinglet.valueGaugeWeight (j : Fin 2 × Fin 3) : GaugeWeight
```

lemma repGaugeGroupI_gaugeTorusGen_basis [source]

The gauge torus acts diagonally on the basis of `DownSinglet`, with the weights `DownSinglet.valueGaugeWeight`.

```
lemma DownSinglet.repGaugeGroupI_gaugeTorusGen_basis (i : Fin 4) (j : Fin 2 × Fin 3) :
    DownSinglet.repGaugeGroupI (gaugeTorusGen i) (DownSinglet.basis j)
      = ((expI : ℂ) ^ GaugeWeight.coord (DownSinglet.valueGaugeWeight j) i) •
        DownSinglet.basis j
```

lemma repGaugeGroupI_dual_gaugeTorusGen_coord [source]

The dual action of the gauge torus on the coordinate functionals of `DownSinglet`: the weights are negated.

```
lemma DownSinglet.repGaugeGroupI_dual_gaugeTorusGen_coord (i : Fin 4) (j : Fin 2 × Fin 3) :
    DownSinglet.repGaugeGroupI.dual (gaugeTorusGen i) (DownSinglet.basis.coord j)
      = ((expI : ℂ) ^ (-(GaugeWeight.coord (DownSinglet.valueGaugeWeight j) i))) •
        DownSinglet.basis.coord j
```

lemma repGaugeGroupI_conj_dual_gaugeTorusGen_coord [source]

The dual of the conjugate action of the gauge torus on the coordinate functionals of the conjugate of `DownSinglet`: the two negations cancel and the weights are those of the value space.

```
lemma DownSinglet.repGaugeGroupI_conj_dual_gaugeTorusGen_coord (i : Fin 4) (j : Fin 2 × Fin 3)
    :
    DownSinglet.repGaugeGroupI.conj.dual (gaugeTorusGen i)
      (((Basis.conj DownSinglet.basis)).coord j)
      = ((expI : ℂ) ^ GaugeWeight.coord (DownSinglet.valueGaugeWeight j) i) •
        ((Basis.conj DownSinglet.basis)).coord j
```

lemma downSinglet_repLorentzGroup_boostAxis_two_basis [source]

The down-singlet basis diagonalises the z-boost: the colour index is inert, so the weight is the Weyl weight of the spinor index.

```
lemma downSinglet_repLorentzGroup_boostAxis_two_basis (t : ℝ) (ht : t ≠ 0)
    (j : Fin 2 × Fin 3) :
    DownSinglet.repLorentzGroup (SL2C.boostAxis 2 t ht) (DownSinglet.basis j)
      = ((t : ℝ) : ℂ) ^ (weylWeight j.1) • DownSinglet.basis j
```

2.427 PhyslibAlpha.Particles.StandardModel.Fermions.DownSinglet.GaugeAlgebraAction

PhyslibAlpha/Particles/StandardModel/Fermions/DownSinglet/GaugeAlgebraAction.lean on GitHub.

def colourEnd [source]

The endomorphism of the down singlet defined by a 3 × 3 complex matrix acting on the colour index, with the Weyl factor untouched.

```
noncomputable def colourEnd (A : Matrix (Fin 3) (Fin 3) ℂ) :
    DownSinglet →ₗ[ℂ] DownSinglet
```

lemma colourEnd_apply_mk [source]

```
lemma colourEnd_apply_mk (A : Matrix (Fin 3) (Fin 3) ℂ) (v : DownSinglet) :
    colourEnd A v
      = valLinEquiv.symm
          (Module.End.lTensorAlgHom ℂ (EuclideanSpace ℂ (Fin 3)) Fermion.RightHandedWeyl
            (Matrix.toLpLinAlgEquiv 2 A) (valLinEquiv v))
```

lemma colourEnd_add [source]

```
lemma colourEnd_add (A B : Matrix (Fin 3) (Fin 3) ℂ) :
    colourEnd (A + B) = colourEnd A + colourEnd B
```

lemma colourEnd_smul [source]

```
lemma colourEnd_smul (z : ℂ) (A : Matrix (Fin 3) (Fin 3) ℂ) :
    colourEnd (z • A) = z • colourEnd A
```

lemma colourEnd_zero [source]

```
lemma colourEnd_zero : colourEnd 0 = 0
```

lemma colourEnd_neg [source]

```
lemma colourEnd_neg (A : Matrix (Fin 3) (Fin 3) ℂ) : colourEnd (-A) = -colourEnd A
```

lemma colourEnd_multiset_sum [source]

```
lemma colourEnd_multiset_sum (m : Multiset (Matrix (Fin 3) (Fin 3) ℂ)) :
    colourEnd m.sum = (m.map colourEnd).sum
```

lemma colourEnd_mul [source]

The colour endomorphisms compose through matrix multiplication.

```
lemma colourEnd_mul (A B : Matrix (Fin 3) (Fin 3) ℂ) :
    colourEnd (A * B) = colourEnd A ∘ₗ colourEnd B
```

def actionMatrix [source]

The matrix of the infinitesimal `(3, 1)_{-2}` action of a gauge algebra element on the colour index: `i` times the colour part, shifted by `i` times `-2` the hypercharge.

```
noncomputable def actionMatrix (c : GaugeAlgebra) : Matrix (Fin 3) (Fin 3) ℂ
```

def gaugeAlgebraAction [source]

***The infinitesimal action of the gauge algebra on the down-type singlet**: the derivative of the `(3, 1)_{-2}` action of the gauge group, real-linear in the algebra slot and complex-linear in the value slot — the form consumed by the covariant derivative `GaugeAlgebraRealization.covDerivIter` and by `LocalGaugeData.IsInfinitesimalActionOf`.*

```
noncomputable def gaugeAlgebraAction :
    GaugeAlgebra →ₗ[ℝ] DownSinglet →ₗ[ℂ] DownSinglet where
```

lemma foldl_pderiv_sub [source]

The iterated formal derivative of a difference.

```
private lemma foldl_pderiv_sub (x : Multiset (Fin 1 ⊕ Fin 3)) (f g : SpaceTimeAlgebra) :
    SpaceTimeAlgebra.iteratedPDeriv x (f - g)
      = SpaceTimeAlgebra.iteratedPDeriv x f - SpaceTimeAlgebra.iteratedPDeriv x g
```

def jetActionMatrix [source]

The jet-valued matrix of the infinitesimal `(3, 1)_{-2}` action of a jet of gauge algebra elements: the jet analogue of `actionMatrix`.

```
noncomputable def jetActionMatrix (a : JetGaugeAlgebra) : Matrix (Fin 3) (Fin 3)
    SpaceTimeAlgebra
```

lemma jetActionMatrix_map_cc_foldl [source]

The base-point Taylor coefficients of the jet action matrix are the action matrices of the base-point Taylor coefficients.

```
lemma jetActionMatrix_map_cc_foldl (p : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    ((jetActionMatrix a).map fun f =>
        constantCoeff (SpaceTimeAlgebra.iteratedPDeriv p f))
      = actionMatrix (JetGaugeAlgebra.eval (JetGaugeAlgebra.iteratedDeriv p a))
```

def downMatrix [source]

The `SpaceTimeAlgebra`-valued colour matrix of the jet gauge action on the down singlet: the colour matrix of the gauge jet carrying the `-2` hypercharge phase.

```
noncomputable def downMatrix (U : JetGaugeGroupI) : Matrix (Fin 3) (Fin 3) SpaceTimeAlgebra
```

lemma repJetGaugeGroupI_eq_downMatrix [source]

```
lemma repJetGaugeGroupI_eq_downMatrix (U : JetGaugeGroupI)
    (z : SpaceTimeAlgebra ⊗[ℂ] DownSinglet) :
    repJetGaugeGroupI U z
      = jetValLinEquiv.symm
          (Module.End.lTensorAlgHom ℂ (EuclideanSpace SpaceTimeAlgebra (Fin 3))
            Fermion.RightHandedWeyl
            ((Matrix.toLpLinAlgEquiv 2 (downMatrix U)).restrictScalars ℂ)
            (jetValLinEquiv z))
```

def pderivColour [source]

The entrywise formal derivative on the colour coordinates, as a $\mathbb{C}$-linear map.

```
private noncomputable def pderivColour (μ : Fin 1 ⊕ Fin 3) :
    EuclideanSpace SpaceTimeAlgebra (Fin 3) →ₗ[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 3) where
```

def foldColour [source]

The entrywise iterated formal derivative on the colour coordinates.

```
private noncomputable def foldColour (x : Multiset (Fin 1 ⊕ Fin 3)) :
    EuclideanSpace SpaceTimeAlgebra (Fin 3) →ₗ[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 3) where
```

def ccColour [source]

The entrywise base-point evaluation on the colour coordinates.

```
private noncomputable def ccColour :
    EuclideanSpace SpaceTimeAlgebra (Fin 3) →ₗ[ℂ] EuclideanSpace ℂ (Fin 3) where
```

lemma pderivColour_comp_foldColour [source]

```
private lemma pderivColour_comp_foldColour (μ : Fin 1 ⊕ Fin 3)
    (x : Multiset (Fin 1 ⊕ Fin 3)) :
    pderivColour μ ∘ₗ foldColour x = foldColour (μ ::ₘ x)
```

lemma jetValLinEquiv_jetDeriv [source]

The identification of down-singlet jets intertwines the formal derivative with the entrywise derivative on the colour coordinates.

```
private lemma jetValLinEquiv_jetDeriv (μ : Fin 1 ⊕ Fin 3)
    (z : SpaceTimeAlgebra ⊗[ℂ] DownSinglet) :
    jetValLinEquiv (jetDeriv μ z)
      = (TensorProduct.map LinearMap.id (pderivColour μ)) (jetValLinEquiv z)
```

lemma jetValLinEquiv_jetIteratedDeriv [source]

The identification of down-singlet jets intertwines the iterated formal derivative with the entrywise iterated derivative on the colour coordinates.

```
private lemma jetValLinEquiv_jetIteratedDeriv (x : Multiset (Fin 1 ⊕ Fin 3))
    (z : SpaceTimeAlgebra ⊗[ℂ] DownSinglet) :
    jetValLinEquiv (jetIteratedDeriv x z)
      = (TensorProduct.map LinearMap.id (foldColour x)) (jetValLinEquiv z)
```

lemma valLinEquiv_jetEval [source]

The base-point evaluation of a down-singlet jet through the colour coordinates.

```
private lemma valLinEquiv_jetEval (z : SpaceTimeAlgebra ⊗[ℂ] DownSinglet) :
    valLinEquiv (jetEval z)
      = (TensorProduct.map LinearMap.id ccColour) (jetValLinEquiv z)
```

lemma downMatrix_map_pderiv [source]

***The derivative identity** for the colour matrix of the jet gauge action: the formal derivative of the colour matrix is minus the jet action matrix of the Maurer–Cartan form times the colour matrix.*

```
lemma downMatrix_map_pderiv (U : JetGaugeGroupI) (μ : Fin 1 ⊕ Fin 3) :
    (downMatrix U).map (fun f => pderiv μ f)
      = -(jetActionMatrix (maurerCartanForm U μ) * downMatrix U)
```

lemma downMatrix_mul_jetActionMatrix [source]

***The equivariance identity** for the colour matrix of the jet gauge action: the colour matrix intertwines the constant jet action matrix with its adjoint transform.*

```
lemma downMatrix_mul_jetActionMatrix (U : JetGaugeGroupI) (c : GaugeAlgebra) :
    downMatrix U * jetActionMatrix (JetGaugeAlgebra.ofConstant c)
      = jetActionMatrix (JetGaugeAlgebra.adjointMap U (JetGaugeAlgebra.ofConstant c))
        * downMatrix U
```

lemma repCoeff_eq [source]

***The base-point Taylor coefficients of the jet gauge action** on the down-type singlet are the colour endomorphisms of the base-point Taylor coefficients of the colour matrix.*

```
lemma repCoeff_eq (U : JetGaugeGroupI) (x : Multiset (Fin 1 ⊕ Fin 3)) :
```

```
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U x
      = colourEnd ((downMatrix U).map fun f =>
          constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x f))
```

lemma colourEnd_one [source]

The colour endomorphism of the identity matrix is the identity.

```
lemma colourEnd_one : colourEnd 1 = LinearMap.id
```

lemma repCoeff_zero_of_eval_eq_one [source]

At the base point, a gauge jet with trivial value acts trivially: the zeroth Taylor coefficient of the jet gauge action is the identity.

```
lemma repCoeff_zero_of_eval_eq_one {U : JetGaugeGroupI} (hU : U.eval = 1) :
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U 0 = LinearMap.id
```

lemma isInfinitesimalActionOf [source]

***The `(3, 1)_{-2}` action of the gauge algebra is the infinitesimal action underlying the jet gauge action on the down-type singlet**: its base-point Taylor coefficients obey the Maurer–Cartan Leibniz law and intertwine the action with the adjoint transports.*

```
lemma isInfinitesimalActionOf :
    localGaugeData.IsInfinitesimalActionOf gaugeAlgebraAction repJetGaugeGroupI
```

lemma gaugeAlgebraAction_comm_repLorentzGroup [source]

The infinitesimal gauge action on the down-type singlet acts on the colour factor, the Lorentz action on the Weyl factor, so the two commute.

```
lemma gaugeAlgebraAction_comm_repLorentzGroup (c : GaugeAlgebra) (Λ : SL(2,ℂ))
    (v : DownSinglet) :
    DownSinglet.gaugeAlgebraAction c (DownSinglet.repLorentzGroup Λ v) =
      DownSinglet.repLorentzGroup Λ (DownSinglet.gaugeAlgebraAction c v)
```

2.428 PhyslibAlpha.Particles.StandardModel.Fermions.JetAlgebra.Basic

PhyslibAlpha/Particles/StandardModel/Fermions/JetAlgebra/Basic.lean on GitHub.

abbrev FermionSpace [source]

The total target space of the Standard Model fermions: the direct sum of three generations each of the lepton doublet, the charged-lepton singlet, the quark doublet, and the up- and down-type quark singlets. The three generations of a species sit together, so that a species can be split off the jet algebra as a single exterior factor.

```
abbrev FermionSpace : Type
```

def generations [source]

The matter field of three generations of one fermion species: the direct sum of three copies of it, which all carry the same mass weight and the same declared statistic.

```
noncomputable def generations (M : MatterField localGaugeData) : MatterField localGaugeData
```

def fermionMatterField [source]

***The fermionic matter field of the Standard Model**: the direct sum of three generations of each of the five species, in the order in which `FermionSpace` lists them. Its value space is `FermionSpace` by definition, and every summand carries mass weight three and is declared fermionic, which is what lets the direct sums be formed.*

```
noncomputable def fermionMatterField : MatterField localGaugeData
```

lemma fermionMatterField_V [source]

```
lemma fermionMatterField_V : fermionMatterField.V = FermionSpace
```

def leptonDoubletProj [source]

The projection onto the `i`-th generation of the lepton doublet.

```
noncomputable def leptonDoubletProj (i : Fin 3) : FermionSpace →ₗ[ℂ] LeptonDoubletV
```

def leptonSingletProj [source]

The projection onto the `i`-th generation of the charged-lepton singlet.

```
noncomputable def leptonSingletProj (i : Fin 3) : FermionSpace →ₗ[ℂ] LeptonSinglet
```

def quarkDoubletProj [source]

The projection onto the `i`-th generation of the quark doublet.

```
noncomputable def quarkDoubletProj (i : Fin 3) : FermionSpace →ₗ[ℂ] QuarkDoublet
```

def upSingletProj [source]

The projection onto the `i`-th generation of the up-type quark singlet.

```
noncomputable def upSingletProj (i : Fin 3) : FermionSpace →ₗ[ℂ] UpSinglet
```

def downSingletProj [source]

The projection onto the `i`-th generation of the down-type quark singlet.

```
noncomputable def downSingletProj (i : Fin 3) : FermionSpace →ₗ[ℂ] DownSinglet
```

def leptonDoubletIncl [source]

The inclusion of the `i`-th generation lepton doublet as a summand.

```
noncomputable def leptonDoubletIncl (i : Fin 3) : LeptonDoubletV →ₗ[ℂ] FermionSpace
```

def leptonSingletIncl [source]

The inclusion of the `i`-th generation charged-lepton singlet as a summand.

```
noncomputable def leptonSingletIncl (i : Fin 3) : LeptonSinglet →ₗ[ℂ] FermionSpace
```

def quarkDoubletIncl [source]

The inclusion of the `i`-th generation quark doublet as a summand.

```
noncomputable def quarkDoubletIncl (i : Fin 3) : QuarkDoublet →ₗ[ℂ] FermionSpace
```

def upSingletIncl [source]

The inclusion of the `i`-th generation up-type quark singlet as a summand.

```
noncomputable def upSingletIncl (i : Fin 3) : UpSinglet →ₗ[ℂ] FermionSpace
```

def downSingletIncl [source]

The inclusion of the `i`-th generation down-type quark singlet as a summand.

```
noncomputable def downSingletIncl (i : Fin 3) : DownSinglet →ₗ[ℂ] FermionSpace
```

lemma leptonDoubletProj_comp_leptonDoubletIncl [source]

```
@[simp]
lemma leptonDoubletProj_comp_leptonDoubletIncl (i : Fin 3) :
    (leptonDoubletProj i).comp (leptonDoubletIncl i) = LinearMap.id
```

lemma leptonSingletProj_comp_leptonSingletIncl [source]

```
@[simp]
lemma leptonSingletProj_comp_leptonSingletIncl (i : Fin 3) :
    (leptonSingletProj i).comp (leptonSingletIncl i) = LinearMap.id
```

lemma quarkDoubletProj_comp_quarkDoubletIncl [source]

```
@[simp]
lemma quarkDoubletProj_comp_quarkDoubletIncl (i : Fin 3) :
    (quarkDoubletProj i).comp (quarkDoubletIncl i) = LinearMap.id
```

lemma upSingletProj_comp_upSingletIncl [source]

```
@[simp]
lemma upSingletProj_comp_upSingletIncl (i : Fin 3) :
    (upSingletProj i).comp (upSingletIncl i) = LinearMap.id
```

lemma downSingletProj_comp_downSingletIncl [source]

```
@[simp]
lemma downSingletProj_comp_downSingletIncl (i : Fin 3) :
    (downSingletProj i).comp (downSingletIncl i) = LinearMap.id
```

def pi [source]

The pointwise representation on a finite power of the representation space.

```
noncomputable def _root_.Representation.pi {k G V : Type*} (ι : Type*) [CommSemiring k]
    [Monoid G] [AddCommMonoid V] [Module k V] (ρ : Representation k G V) :
    Representation k G (ι → V) where
```

def FermionSpace.repLorentzGroup [source]

The Lorentz action on the total fermionic target space: each species and generation transforms in its own Lorentz representation.

```
noncomputable def repLorentzGroup : Representation ℂ SL(2,ℂ) FermionSpace
```

def FermionSpace.repGaugeGroupI [source]

The global gauge action on the total fermionic target space: each species and generation transforms in its own representation of the gauge group.

```
noncomputable def repGaugeGroupI : Representation ℂ GaugeGroupI FermionSpace
```

def jetEquiv [source]

The jets of the total fermionic field as the product of the jets of the species and generations. The identification is `SpaceTimeAlgebra`-linear.

```
noncomputable def jetEquiv :
    SpaceTimeAlgebra ⊗[ℂ] FermionSpace ≃ₗ[SpaceTimeAlgebra]
      (Fin 3 → SpaceTimeAlgebra ⊗[ℂ] LeptonDoubletV) ×
        ((Fin 3 → SpaceTimeAlgebra ⊗[ℂ] LeptonSinglet) ×
          ((Fin 3 → SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet) ×
            ((Fin 3 → SpaceTimeAlgebra ⊗[ℂ] UpSinglet) ×
              (Fin 3 → SpaceTimeAlgebra ⊗[ℂ] DownSinglet))))
```

def jetActionMap [source]

The map through which a jet of gauge transformations acts on the jets of the total fermionic field: the species actions, factor by factor.

```
noncomputable def jetActionMap (U : JetGaugeGroupI) :
    ((Fin 3 → SpaceTimeAlgebra ⊗[ℂ] LeptonDoubletV) ×
      ((Fin 3 → SpaceTimeAlgebra ⊗[ℂ] LeptonSinglet) ×
        ((Fin 3 → SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet) ×
          ((Fin 3 → SpaceTimeAlgebra ⊗[ℂ] UpSinglet) ×
            (Fin 3 → SpaceTimeAlgebra ⊗[ℂ] DownSinglet))))) →ₗ[ℂ]
    ((Fin 3 → SpaceTimeAlgebra ⊗[ℂ] LeptonDoubletV) ×
      ((Fin 3 → SpaceTimeAlgebra ⊗[ℂ] LeptonSinglet) ×
        ((Fin 3 → SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet) ×
          ((Fin 3 → SpaceTimeAlgebra ⊗[ℂ] UpSinglet) ×
            (Fin 3 → SpaceTimeAlgebra ⊗[ℂ] DownSinglet)))))
```

lemma piMap_id [source]

The pointwise lift of the identity maps is the identity.

```
lemma _root_.LinearMap.piMap_id {R ι : Type*} {φ : ι → Type*} [Semiring R]
    [∀ i, AddCommMonoid (φ i)] [∀ i, Module R (φ i)] :
    LinearMap.piMap (fun i => (LinearMap.id : φ i →ₗ[R] φ i)) = LinearMap.id
```

lemma piMap_comp_piMap [source]

The pointwise lift of compositions is the composition of the pointwise lifts.

```
lemma _root_.LinearMap.piMap_comp_piMap {R ι : Type*} {φ ψ ω : ι → Type*} [Semiring R]
    [∀ i, AddCommMonoid (φ i)] [∀ i, Module R (φ i)]
    [∀ i, AddCommMonoid (ψ i)] [∀ i, Module R (ψ i)]
    [∀ i, AddCommMonoid (ω i)] [∀ i, Module R (ω i)]
    (f : ∀ i, ψ i →ₗ[R] ω i) (g : ∀ i, φ i →ₗ[R] ψ i) :
    (LinearMap.piMap f).comp (LinearMap.piMap g)
      = LinearMap.piMap fun i => (f i).comp (g i)
```

lemma jetActionMap_one [source]

The map of jets of the identity is the identity.

```
lemma jetActionMap_one : jetActionMap 1 = LinearMap.id
```

lemma jetActionMap_mul [source]

The map of jets of a product is the composition of the maps of jets.

```
lemma jetActionMap_mul (U V : JetGaugeGroupI) :
    jetActionMap (U * V) = (jetActionMap U).comp (jetActionMap V)
```

def FermionSpace.repJetGaugeGroupI [source]

***The jet gauge action on the jets of the total fermionic field**: the species actions, transported through the splitting of the jets.*

```
noncomputable def repJetGaugeGroupI :
    Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] FermionSpace) where
```

lemma repJetGaugeGroupI_smul [source]

***The jet gauge action on the jets of the total fermionic field is fibrewise**: it commutes with multiplication by scalar jets, because the splitting of the jets is `SpaceTimeAlgebra`-linear and each species action is fibrewise.*

```
lemma repJetGaugeGroupI_smul (U : JetGaugeGroupI) (χ : SpaceTimeAlgebra)
    (z : SpaceTimeAlgebra ⊗[ℂ] FermionSpace) :
    repJetGaugeGroupI U (χ • z) = χ • repJetGaugeGroupI U z
```

abbrev FermionJetAlgebra [source]

***The jet algebra of the Standard Model fermions**: the exterior product of the fermionic algebras of the five species, realized as the fermionic algebra of their direct sum. Its generators are the component functions `∂_s ψ_φ` and `∂_s ψ̄_φ` of every*

species and generation, and any two of them anticommute — within a species and across species alike.

This is the fermionic factor of the full Standard Model jet algebra; the gauge and Higgs factors are bosonic and commute with it.

```
abbrev FermionJetAlgebra : Type
```

def ofLeptonDoublet [source]

The component functions of the `i`-th generation lepton doublet inside the Standard Model jet algebra.

```
noncomputable def ofLeptonDoublet (i : Fin 3) :
    Module.Dual ℂ LeptonDoubletV →ₗ[ℂ] FermionJetAlgebra
```

def ofLeptonSinglet [source]

The component functions of the `i`-th generation charged-lepton singlet.

```
noncomputable def ofLeptonSinglet (i : Fin 3) :
    Module.Dual ℂ LeptonSinglet →ₗ[ℂ] FermionJetAlgebra
```

def ofQuarkDoublet [source]

The component functions of the `i`-th generation quark doublet.

```
noncomputable def ofQuarkDoublet (i : Fin 3) :
    Module.Dual ℂ QuarkDoublet →ₗ[ℂ] FermionJetAlgebra
```

def ofUpSinglet [source]

The component functions of the `i`-th generation up-type quark singlet.

```
noncomputable def ofUpSinglet (i : Fin 3) :
    Module.Dual ℂ UpSinglet →ₗ[ℂ] FermionJetAlgebra
```

def ofDownSinglet [source]

The component functions of the `i`-th generation down-type quark singlet.

```
noncomputable def ofDownSinglet (i : Fin 3) :
    Module.Dual ℂ DownSinglet →ₗ[ℂ] FermionJetAlgebra
```

def ofConjLeptonDoublet [source]

The conjugate component functions of the `i`-th generation lepton doublet.

```
noncomputable def ofConjLeptonDoublet (i : Fin 3) :
    Module.Dual ℂ (ConjModule LeptonDoubletV) →ₗ[ℂ] FermionJetAlgebra
```

def ofConjLeptonSinglet [source]

The conjugate component functions of the `i`-th generation charged-lepton singlet.

```
noncomputable def ofConjLeptonSinglet (i : Fin 3) :
    Module.Dual ℂ (ConjModule LeptonSinglet) →ₗ[ℂ] FermionJetAlgebra
```

def ofConjQuarkDoublet [source]

The conjugate component functions of the `i`-th generation quark doublet.

```
noncomputable def ofConjQuarkDoublet (i : Fin 3) :
    Module.Dual ℂ (ConjModule QuarkDoublet) →ₗ[ℂ] FermionJetAlgebra
```

def ofConjUpSinglet [source]

The conjugate component functions of the `i`-th generation up-type quark singlet.

```
noncomputable def ofConjUpSinglet (i : Fin 3) :
    Module.Dual ℂ (ConjModule UpSinglet) →ₗ[ℂ] FermionJetAlgebra
```

def ofConjDownSinglet [source]

The conjugate component functions of the `i`-th generation down-type quark singlet.

```
noncomputable def ofConjDownSinglet (i : Fin 3) :
    Module.Dual ℂ (ConjModule DownSinglet) →ₗ[ℂ] FermionJetAlgebra
```

def exteriorProductLeptonDoublet [source]

The fermionic jet algebra as the exterior product of the three-generation lepton-doublet algebra with the algebra of the remaining four species.

```
noncomputable def exteriorProductLeptonDoublet :
    FermionJetAlgebra ≃ₐ[ℂ] (FermionicAlgebra.evenOdd (generations LeptonDoubletV.matterField)
    ᵍ⊗[ℂ]
      FermionicAlgebra.evenOdd
          ((generations LeptonSinglet.matterField).prod
          ((generations QuarkDoublet.matterField).prod
          ((generations UpSinglet.matterField).prod
          (generations DownSinglet.matterField) rfl rfl) rfl rfl) rfl rfl))
```

def exteriorProductLeptonSinglet [source]

The charged-lepton singlets split off the remaining three species.

```
noncomputable def exteriorProductLeptonSinglet :
    FermionicAlgebra
        ((generations LeptonSinglet.matterField).prod
        ((generations QuarkDoublet.matterField).prod
        ((generations UpSinglet.matterField).prod (generations DownSinglet.matterField) rfl rfl
    )
        rfl rfl) rfl rfl) ≃ₐ[ℂ]
      (FermionicAlgebra.evenOdd (generations LeptonSinglet.matterField) ᵍ⊗[ℂ]
        FermionicAlgebra.evenOdd
            ((generations QuarkDoublet.matterField).prod
            ((generations UpSinglet.matterField).prod
            (generations DownSinglet.matterField) rfl rfl) rfl rfl))
```

def exteriorProductQuarkDoublet [source]

The quark doublets split off the two quark singlets.

```
noncomputable def exteriorProductQuarkDoublet :
    FermionicAlgebra
        ((generations QuarkDoublet.matterField).prod
        ((generations UpSinglet.matterField).prod
        (generations DownSinglet.matterField) rfl rfl) rfl rfl) ≃ₐ[ℂ]
      (FermionicAlgebra.evenOdd (generations QuarkDoublet.matterField) ᵍ⊗[ℂ]
        FermionicAlgebra.evenOdd
            ((generations UpSinglet.matterField).prod (generations DownSinglet.matterField)
              rfl rfl))
```

def exteriorProductUpSinglet [source]

The two quark singlets as an exterior product.

```
noncomputable def exteriorProductUpSinglet :
    FermionicAlgebra
        ((generations UpSinglet.matterField).prod (generations DownSinglet.matterField)
          rfl rfl) ≃ₐ[ℂ]
      (FermionicAlgebra.evenOdd (generations UpSinglet.matterField) ᵍ⊗[ℂ]
        FermionicAlgebra.evenOdd (generations DownSinglet.matterField))
```

def FermionJetAlgebra.repLorentzGroup [source]

The Lorentz action on the fermionic jet algebra of the Standard Model.

```
noncomputable def repLorentzGroup : Representation ℂ SL(2,ℂ) FermionJetAlgebra
```

def FermionJetAlgebra.repJetGaugeGroupI [source]

The jet gauge action on the fermionic jet algebra of the Standard Model.

```
noncomputable def repJetGaugeGroupI : Representation ℂ JetGaugeGroupI FermionJetAlgebra
```

def FermionJetAlgebra.repGaugeGroupI [source]

The global gauge action on the fermionic jet algebra of the Standard Model.

```
noncomputable def repGaugeGroupI : Representation ℂ GaugeGroupI FermionJetAlgebra
```

def massWeightScale [source]

The mass-dimension scaling on the fermionic jet algebra: every Standard Model fermion has mass dimension `3/2`, that is mass weight three, and each derivative adds mass weight two.

```
noncomputable def massWeightScale (c : ℂ) : FermionJetAlgebra →ₐ[ℂ] FermionJetAlgebra
```

2.429 PhyslibAlpha.Particles.StandardModel.Fermions.JetAlgebra.Species

PhyslibAlpha/Particles/StandardModel/Fermions/JetAlgebra/Species.lean on GitHub.

lemma lTensor_comp_jetOfConstant [source]

Including a constant into value-space jets is natural in the value space.

```
lemma lTensor_comp_jetOfConstant (p : V →ₗ[ℂ] W) :
    (LinearMap.lTensor SpaceTimeAlgebra p).comp jetOfConstant = jetOfConstant.comp p
```

lemma lTensor_comp_jetDeriv [source]

The formal derivative on value-space jets is natural in the value space: it touches only the jet factor.

```
lemma lTensor_comp_jetDeriv (p : V →ₗ[ℂ] W) (μ : Fin 1 ⊕ Fin 3) :
    (LinearMap.lTensor SpaceTimeAlgebra p).comp (jetDeriv μ)
      = (jetDeriv μ).comp (LinearMap.lTensor SpaceTimeAlgebra p)
```

lemma lTensor_comp_jetIteratedDeriv [source]

The iterated formal derivative on value-space jets is natural in the value space.

```
lemma lTensor_comp_jetIteratedDeriv (p : V →ₗ[ℂ] W) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (LinearMap.lTensor SpaceTimeAlgebra p).comp (jetIteratedDeriv s)
      = (jetIteratedDeriv s).comp (LinearMap.lTensor SpaceTimeAlgebra p)
```

lemma jetEval_comp_lTensor [source]

Evaluation of a value-space jet at the base point is natural in the value space.

```
lemma jetEval_comp_lTensor (p : V →ₗ[ℂ] W) :
    (jetEval (V := W)).comp (LinearMap.lTensor SpaceTimeAlgebra p) = p.comp jetEval
```

lemma repCoeff_comp [source]

The base-point Taylor coefficients of two jet gauge actions are intertwined by any map of value spaces intertwining the actions themselves: `repCoeff` is built from `jetOfConstant`, `jetIteratedDeriv` and `jetEval`, and each of those is natural.

```
lemma repCoeff_comp {repV : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] V)}
    {repW : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] W)} (p : V →ₗ[ℂ] W)
    (hp : ∀ U : JetGaugeGroupI, (LinearMap.lTensor SpaceTimeAlgebra p).comp (repV U)
      = (repW U).comp (LinearMap.lTensor SpaceTimeAlgebra p))
    (U : JetGaugeGroupI) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    p.comp (GaugeAlgebraRealization.repCoeff repV U s)
      = (GaugeAlgebraRealization.repCoeff repW U s).comp p
```

lemma repDualCoeff_comp [source]

The transposed form of `repCoeff_comp`: the dual coefficients, which act on the component-function index, are intertwined the other way round.

```
lemma repDualCoeff_comp {repV : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] V)}
    {repW : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] W)} (p : V →ₗ[ℂ] W)
    (hp : ∀ U : JetGaugeGroupI, (LinearMap.lTensor SpaceTimeAlgebra p).comp (repV U)
      = (repW U).comp (LinearMap.lTensor SpaceTimeAlgebra p))
    (U : JetGaugeGroupI) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (GaugeAlgebraRealization.repDualCoeff repV U s).comp (Module.Dual.transpose p)
```

```
    = (Module.Dual.transpose p).comp (GaugeAlgebraRealization.repDualCoeff repW U s)
```

lemma jetEquiv_leptonDoublet [source]

The lepton-doublet slot of the splitting of the jets is the projection on the value factor.

```
lemma jetEquiv_leptonDoublet (i : Fin 3) (z : SpaceTimeAlgebra ⊗[ℂ] FermionSpace) :
    (jetEquiv z).1 i = LinearMap.lTensor SpaceTimeAlgebra (leptonDoubletProj i) z
```

lemma jetEquiv_leptonSinglet [source]

The charged-lepton-singlet slot of the splitting of the jets is the projection on the value factor.

```
lemma jetEquiv_leptonSinglet (i : Fin 3) (z : SpaceTimeAlgebra ⊗[ℂ] FermionSpace) :
    (jetEquiv z).2.1 i = LinearMap.lTensor SpaceTimeAlgebra (leptonSingletProj i) z
```

lemma jetEquiv_quarkDoublet [source]

The quark-doublet slot of the splitting of the jets is the projection on the value factor.

```
lemma jetEquiv_quarkDoublet (i : Fin 3) (z : SpaceTimeAlgebra ⊗[ℂ] FermionSpace) :
    (jetEquiv z).2.2.1 i = LinearMap.lTensor SpaceTimeAlgebra (quarkDoubletProj i) z
```

lemma jetEquiv_upSinglet [source]

The up-type-quark-singlet slot of the splitting of the jets is the projection on the value factor.

```
lemma jetEquiv_upSinglet (i : Fin 3) (z : SpaceTimeAlgebra ⊗[ℂ] FermionSpace) :
    (jetEquiv z).2.2.2.1 i = LinearMap.lTensor SpaceTimeAlgebra (upSingletProj i) z
```

lemma jetEquiv_downSinglet [source]

The down-type-quark-singlet slot of the splitting of the jets is the projection on the value factor.

```
lemma jetEquiv_downSinglet (i : Fin 3) (z : SpaceTimeAlgebra ⊗[ℂ] FermionSpace) :
    (jetEquiv z).2.2.2.2 i = LinearMap.lTensor SpaceTimeAlgebra (downSingletProj i) z
```

lemma lTensor_leptonDoubletProj_repJetGaugeGroupI [source]

The lepton-doublet projection intertwines the total jet gauge action with the lepton doublet's own.

```
lemma lTensor_leptonDoubletProj_repJetGaugeGroupI (i : Fin 3) (U : JetGaugeGroupI) :
    (LinearMap.lTensor SpaceTimeAlgebra (leptonDoubletProj i)).comp (repJetGaugeGroupI U)
      = (LeptonDoubletV.repJetGaugeGroupI U).comp
        (LinearMap.lTensor SpaceTimeAlgebra (leptonDoubletProj i))
```

lemma lTensor_leptonSingletProj_repJetGaugeGroupI [source]

The charged-lepton-singlet projection intertwines the total jet gauge action with the charged-lepton singlet's own.

```
lemma lTensor_leptonSingletProj_repJetGaugeGroupI (i : Fin 3) (U : JetGaugeGroupI) :
    (LinearMap.lTensor SpaceTimeAlgebra (leptonSingletProj i)).comp (repJetGaugeGroupI U)
      = (LeptonSinglet.repJetGaugeGroupI U).comp
        (LinearMap.lTensor SpaceTimeAlgebra (leptonSingletProj i))
```

lemma lTensor_quarkDoubletProj_repJetGaugeGroupI [source]

The quark-doublet projection intertwines the total jet gauge action with the quark doublet's own.

```
lemma lTensor_quarkDoubletProj_repJetGaugeGroupI (i : Fin 3) (U : JetGaugeGroupI) :
    (LinearMap.lTensor SpaceTimeAlgebra (quarkDoubletProj i)).comp (repJetGaugeGroupI U)
      = (QuarkDoublet.repJetGaugeGroupI U).comp
        (LinearMap.lTensor SpaceTimeAlgebra (quarkDoubletProj i))
```

lemma lTensor_upSingletProj_repJetGaugeGroupI [source]

The up-type-quark-singlet projection intertwines the total jet gauge action with the up-type quark singlet's own.

```
lemma lTensor_upSingletProj_repJetGaugeGroupI (i : Fin 3) (U : JetGaugeGroupI) :
    (LinearMap.lTensor SpaceTimeAlgebra (upSingletProj i)).comp (repJetGaugeGroupI U)
      = (UpSinglet.repJetGaugeGroupI U).comp
        (LinearMap.lTensor SpaceTimeAlgebra (upSingletProj i))
```

lemma lTensor_downSingletProj_repJetGaugeGroupI [source]

The down-type-quark-singlet projection intertwines the total jet gauge action with the down-type quark singlet's own.

```
lemma lTensor_downSingletProj_repJetGaugeGroupI (i : Fin 3) (U : JetGaugeGroupI) :
    (LinearMap.lTensor SpaceTimeAlgebra (downSingletProj i)).comp (repJetGaugeGroupI U)
      = (DownSinglet.repJetGaugeGroupI U).comp
        (LinearMap.lTensor SpaceTimeAlgebra (downSingletProj i))
```

lemma leptonDoubletProj_comp_repCoeff [source]

The lepton-doublet projection intertwines the base-point Taylor coefficients of the total jet gauge action with those of the lepton doublet's own.

```
lemma leptonDoubletProj_comp_repCoeff (i : Fin 3) (U : JetGaugeGroupI)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (leptonDoubletProj i).comp (GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U s)
      = (GaugeAlgebraRealization.repCoeff LeptonDoubletV.repJetGaugeGroupI U s).comp
          (leptonDoubletProj i)
```

lemma leptonSingletProj_comp_repCoeff [source]

The charged-lepton-singlet projection intertwines the base-point Taylor coefficients of the total jet gauge action with those of the charged-lepton singlet's own.

```
lemma leptonSingletProj_comp_repCoeff (i : Fin 3) (U : JetGaugeGroupI)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (leptonSingletProj i).comp (GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U s)
```

```
    = (GaugeAlgebraRealization.repCoeff LeptonSinglet.repJetGaugeGroupI U s).comp
        (leptonSingletProj i)
```

lemma quarkDoubletProj_comp_repCoeff [source]

The quark-doublet projection intertwines the base-point Taylor coefficients of the total jet gauge action with those of the quark doublet's own.

```
lemma quarkDoubletProj_comp_repCoeff (i : Fin 3) (U : JetGaugeGroupI)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (quarkDoubletProj i).comp (GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U s)
      = (GaugeAlgebraRealization.repCoeff QuarkDoublet.repJetGaugeGroupI U s).comp
          (quarkDoubletProj i)
```

lemma upSingletProj_comp_repCoeff [source]

The up-type-quark-singlet projection intertwines the base-point Taylor coefficients of the total jet gauge action with those of the up-type quark singlet's own.

```
lemma upSingletProj_comp_repCoeff (i : Fin 3) (U : JetGaugeGroupI)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (upSingletProj i).comp (GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U s)
      = (GaugeAlgebraRealization.repCoeff UpSinglet.repJetGaugeGroupI U s).comp
          (upSingletProj i)
```

lemma downSingletProj_comp_repCoeff [source]

The down-type-quark-singlet projection intertwines the base-point Taylor coefficients of the total jet gauge action with those of the down-type quark singlet's own.

```
lemma downSingletProj_comp_repCoeff (i : Fin 3) (U : JetGaugeGroupI)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (downSingletProj i).comp (GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U s)
      = (GaugeAlgebraRealization.repCoeff DownSinglet.repJetGaugeGroupI U s).comp
          (downSingletProj i)
```

lemma leptonDoubletProj_comp_repLorentzGroup [source]

The lepton-doublet projection intertwines the total Lorentz action with the lepton doublet's own.

```
lemma leptonDoubletProj_comp_repLorentzGroup (i : Fin 3) (Λ : SL(2,ℂ)) :
    (leptonDoubletProj i).comp (repLorentzGroup Λ)
      = (LeptonDoubletV.repLorentzGroup Λ).comp (leptonDoubletProj i)
```

lemma leptonSingletProj_comp_repLorentzGroup [source]

The charged-lepton-singlet projection intertwines the total Lorentz action with the charged-lepton singlet's own.

```
lemma leptonSingletProj_comp_repLorentzGroup (i : Fin 3) (Λ : SL(2,ℂ)) :
    (leptonSingletProj i).comp (repLorentzGroup Λ)
      = (LeptonSinglet.repLorentzGroup Λ).comp (leptonSingletProj i)
```

lemma quarkDoubletProj_comp_repLorentzGroup [source]

The quark-doublet projection intertwines the total Lorentz action with the quark doublet's own.

```
lemma quarkDoubletProj_comp_repLorentzGroup (i : Fin 3) (Λ : SL(2,ℂ)) :
    (quarkDoubletProj i).comp (repLorentzGroup Λ)
      = (QuarkDoublet.repLorentzGroup Λ).comp (quarkDoubletProj i)
```

lemma upSingletProj_comp_repLorentzGroup [source]

The up-type-quark-singlet projection intertwines the total Lorentz action with the up-type quark singlet's own.

```
lemma upSingletProj_comp_repLorentzGroup (i : Fin 3) (Λ : SL(2,ℂ)) :
    (upSingletProj i).comp (repLorentzGroup Λ)
      = (UpSinglet.repLorentzGroup Λ).comp (upSingletProj i)
```

lemma downSingletProj_comp_repLorentzGroup [source]

The down-type-quark-singlet projection intertwines the total Lorentz action with the down-type quark singlet's own.

```
lemma downSingletProj_comp_repLorentzGroup (i : Fin 3) (Λ : SL(2,ℂ)) :
    (downSingletProj i).comp (repLorentzGroup Λ)
      = (DownSinglet.repLorentzGroup Λ).comp (downSingletProj i)
```

lemma sum_incl_proj [source]

The identity on the total fermionic target space is the sum, over the five species and the three generations, of the inclusion after the projection.

```
lemma sum_incl_proj (v : FermionSpace) :
    (∑ i, leptonDoubletIncl i (leptonDoubletProj i v))
      + (∑ i, leptonSingletIncl i (leptonSingletProj i v))
      + (∑ i, quarkDoubletIncl i (quarkDoubletProj i v))
      + (∑ i, upSingletIncl i (upSingletProj i v))
      + (∑ i, downSingletIncl i (downSingletProj i v)) = v
```

lemma sum_conj_incl_proj [source]

The conjugate form of `sum_incl_proj`: conjugation leaves the underlying additive group and the underlying maps alone.

```
lemma sum_conj_incl_proj (v : ConjModule FermionSpace) :
    (∑ i, ConjModule.map (leptonDoubletIncl i)
        (ConjModule.map (leptonDoubletProj i) v))
      + (∑ i, ConjModule.map (leptonSingletIncl i)
          (ConjModule.map (leptonSingletProj i) v))
      + (∑ i, ConjModule.map (quarkDoubletIncl i)
          (ConjModule.map (quarkDoubletProj i) v))
      + (∑ i, ConjModule.map (upSingletIncl i)
          (ConjModule.map (upSingletProj i) v))
      + (∑ i, ConjModule.map (downSingletIncl i)
          (ConjModule.map (downSingletProj i) v)) = v
```

lemma dual_eq_sum [source]

Every covector on the total fermionic target space is the sum, over the five species and the three generations, of its restriction to that species and generation pulled back along the corresponding projection.

```
lemma dual_eq_sum (φ : Module.Dual ℂ FermionSpace) :
    φ = (∑ i, Module.Dual.transpose (leptonDoubletProj i)
          (Module.Dual.transpose (leptonDoubletIncl i) φ))
      + (∑ i, Module.Dual.transpose (leptonSingletProj i)
          (Module.Dual.transpose (leptonSingletIncl i) φ))
      + (∑ i, Module.Dual.transpose (quarkDoubletProj i)
          (Module.Dual.transpose (quarkDoubletIncl i) φ))
      + (∑ i, Module.Dual.transpose (upSingletProj i)
          (Module.Dual.transpose (upSingletIncl i) φ))
      + (∑ i, Module.Dual.transpose (downSingletProj i)
          (Module.Dual.transpose (downSingletIncl i) φ))
```

def speciesDual [source]

The covectors on the total fermionic target space that are pulled back from a single species and generation.

```
def speciesDual : Set (Module.Dual ℂ FermionSpace)
```

lemma span_speciesDual_eq_top [source]

The covectors pulled back from the individual species and generations span every covector on the total fermionic target space. This is what lets an adjoin taken over the species covectors reach an adjoin taken over all of them.

```
lemma span_speciesDual_eq_top : Submodule.span ℂ speciesDual = ⊤
```

lemma conjDual_eq_sum [source]

The conjugate form of `dual_eq_sum`: every covector on the conjugate of the total fermionic target space is the sum of the covectors pulled back from the species and generations.

```
lemma conjDual_eq_sum (φ : Module.Dual ℂ (ConjModule FermionSpace)) :
    φ = (∑ i, Module.Dual.transpose (ConjModule.map (leptonDoubletProj i))
          (Module.Dual.transpose (ConjModule.map (leptonDoubletIncl i)) φ))
      + (∑ i, Module.Dual.transpose (ConjModule.map (leptonSingletProj i))
          (Module.Dual.transpose (ConjModule.map (leptonSingletIncl i)) φ))
      + (∑ i, Module.Dual.transpose (ConjModule.map (quarkDoubletProj i))
          (Module.Dual.transpose (ConjModule.map (quarkDoubletIncl i)) φ))
      + (∑ i, Module.Dual.transpose (ConjModule.map (upSingletProj i))
          (Module.Dual.transpose (ConjModule.map (upSingletIncl i)) φ))
      + (∑ i, Module.Dual.transpose (ConjModule.map (downSingletProj i))
          (Module.Dual.transpose (ConjModule.map (downSingletIncl i)) φ))
```

def speciesConjDual [source]

The covectors on the conjugate of the total fermionic target space that are pulled back from a single species and generation.

```
def speciesConjDual : Set (Module.Dual ℂ (ConjModule FermionSpace))
```

lemma span_speciesConjDual_eq_top [source]

The conjugate covectors pulled back from the individual species and generations span every covector on the conjugate of the total fermionic target space.

```
lemma span_speciesConjDual_eq_top : Submodule.span ℂ speciesConjDual = ⊤
```

2.430 PhyslibAlpha.Particles.StandardModel.Fermions.LeptonDoublet.Basic

PhyslibAlpha/Particles/StandardModel/Fermions/LeptonDoublet/Basic.lean on GitHub.

abbrev LeptonDoubletV [source]

The target vector space of one Standard Model lepton doublet: the target space of the datum `StandardModel.Model.leptonDoublet`, a left-handed Weyl spinor tensored with the weak coordinates. It carries the `(1, 2)_{-3}` representation of the gauge group.

```
abbrev LeptonDoubletV : Type
```

def basis [source]

A basis on the lepton doublets: the Weyl basis tensored with the coordinate basis of the weak index.

```
noncomputable def basis : Module.Basis (Fin 2 × Fin 2) ℂ LeptonDoubletV
```

lemma basis_apply [source]

The lepton-doublet basis vector as an explicit spinor–weak tensor.

```
lemma basis_apply (k j : Fin 2) :
    basis (k, j) = Fermion.LeftHandedWeyl.basis k ⊗ₜ[ℂ] Pi.single j 1
```

def repLorentzGroup [source]

The left-handed Lorentz representation on lepton doublets: the Lorentz action the general theory derives from the datum.

```
noncomputable def repLorentzGroup : Representation ℂ (SL(2,ℂ)) LeptonDoubletV
```

lemma repLorentzGroup_tmul [source]

The Lorentz action on a pure spinor–weak tensor: the left-handed action on the Weyl factor, the weak index untouched.

```
lemma repLorentzGroup_tmul (Λ : SL(2,ℂ)) (s : Fermion.LeftHandedWeyl) (v : Fin 2 → ℂ) :
    repLorentzGroup Λ (s ⊗ₜ v) = Fermion.LeftHandedWeyl.rep Λ s ⊗ₜ v
```

def doubletMatrix [source]

The `SpaceTimeAlgebra`-valued weak matrix of the jet gauge action on the lepton doublet: the matrix of jets by which a gauge jet acts on the datum.

```
noncomputable def doubletMatrix (U : JetGaugeGroupI) : Matrix (Fin 2) (Fin 2) SpaceTimeAlgebra
```

lemma doubletMatrix_eq [source]

The weak matrix of a gauge jet is its `SU(2)` matrix carrying the `-3` hypercharge phase `(star u) ^ 3`.

```
lemma doubletMatrix_eq (U : JetGaugeGroupI) :
    doubletMatrix U = ((star ((U.2.2 : unitary SpaceTimeAlgebra) : SpaceTimeAlgebra)) ^ 3) •
      ((U.2.1 : specialUnitaryGroup (Fin 2) SpaceTimeAlgebra) : Matrix (Fin 2)
          (Fin 2) SpaceTimeAlgebra)
```

lemma doubletMatrix_ofConstant_map_constantCoeff [source]

The constant term of the weak matrix of a constant gauge jet: the `SU(2)` matrix of the gauge transformation carrying its `-3` hypercharge phase.

```
lemma doubletMatrix_ofConstant_map_constantCoeff (g : GaugeGroupI) :
    (doubletMatrix (JetGaugeGroupI.ofConstant g)).map (constantCoeff : SpaceTimeAlgebra → ℂ)
      = (star g.toU1.1 ^ 3) • g.toSU2.1
```

lemma doubletMatrix_ofConstant [source]

The weak matrix of a constant gauge jet is constant.

```
lemma doubletMatrix_ofConstant (g : GaugeGroupI) :
    doubletMatrix (JetGaugeGroupI.ofConstant g)
      = ((doubletMatrix (JetGaugeGroupI.ofConstant g)).map
          (constantCoeff : SpaceTimeAlgebra → ℂ)).map
          (MvPowerSeries.C : ℂ → SpaceTimeAlgebra)
```

def repGaugeGroupI [source]

The `(1, 2)_{-3}` action of the unquotiented Standard Model gauge group: the global action the general theory derives from the datum, the constant term of the jet action.

```
noncomputable def repGaugeGroupI : Representation ℂ GaugeGroupI LeptonDoubletV
```

lemma repGaugeGroupI_tmul [source]

The gauge action on a pure spinor–weak tensor: the `SU(2)` matrix, scaled by the hypercharge factor, acts on the weak index.

```
lemma repGaugeGroupI_tmul (g : GaugeGroupI) (v : Fermion.LeftHandedWeyl) (w : Fin 2 → ℂ) :
    repGaugeGroupI g (v ⊗ₜ w) = v ⊗ₜ ((star g.toU1.1 ^ 3) • g.toSU2.1).mulVec w
```

lemma repGaugeGroupI_apply_basis [source]

The gauge action on the lepton-doublet basis: the spinor index is inert and the weak index transforms by the `SU(2)` matrix, scaled by the hypercharge factor.

```
lemma repGaugeGroupI_apply_basis (g : GaugeGroupI) (j : Fin 2 × Fin 2) :
    repGaugeGroupI g (basis j) =
      ∑ w, (star g.toU1.1 ^ 3 * g.toSU2.1 w j.2) • basis (j.1, w)
```

lemma repGaugeGroupI_tmul_basis_eq_sum [source]

Expands the lepton-doublet gauge action on a tensor-product basis vector.

```
lemma repGaugeGroupI_tmul_basis_eq_sum (g : GaugeGroupI) (k j : Fin 2) :
    repGaugeGroupI g (basis (k, j)) =
      ∑ j' : Fin 2, (star g.toU1.1 ^ 3 * g.toSU2.1 j' j) • basis (k, j')
```

lemma repGaugeGroupI_eq_iff_mul_eq [source]

Two gauge elements induce the same action exactly when their weak-basis coefficients agree.

```
lemma repGaugeGroupI_eq_iff_mul_eq {g₁ g₂ : GaugeGroupI} :
    repGaugeGroupI g₁ = repGaugeGroupI g₂ ↔ ∀ j j',
    star g₁.toU1.1 ^ 3 * g₁.toSU2.1 j' j =
      star g₂.toU1.1 ^ 3 * g₂.toSU2.1 j' j
```

lemma mem_repGaugeGroupI_ker_iff_eq [source]

Characterizes the full-group elements acting trivially on the lepton doublet.

```
lemma mem_repGaugeGroupI_ker_iff_eq {g : GaugeGroupI} :
    g ∈ repGaugeGroupI.ker ↔ ∃ a : ℂ, g.toSU2.1 = a • 1 ∧
      a * star g.toU1.1 ^ 3 = 1
```

lemma gaugeGroup_subgroup_ℤ₆_le_ker_repGaugeGroupI [source]

The central $\mathbb{Z}_6$ subgroup acts trivially on `(1, 2)_{-3}`.

```
lemma gaugeGroup_subgroup_ℤ₆_le_ker_repGaugeGroupI :
    GaugeGroupQuot.subgroup .ℤ₆ ≤ repGaugeGroupI.ker
```

lemma gaugeGroup_subgroup_le_ker_repGaugeGroupI [source]

Every supported quotient subgroup acts trivially on the lepton doublet.

```
lemma gaugeGroup_subgroup_le_ker_repGaugeGroupI (Q : GaugeGroupQuot) :
    Q.subgroup ≤ repGaugeGroupI.ker
```

def repGaugeGroup [source]

The `(1, 2)_{-3}` representation for every supported global form of the Standard Model gauge group.

```
noncomputable def repGaugeGroup : (Q : GaugeGroupQuot) →
    Representation ℂ (GaugeGroup Q) LeptonDoubletV
  | .I => repGaugeGroupI
  | .ℤ₆ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₆)
  | .ℤ₂ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₂)
```

```
  | .ℤ₃ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₃)
```

def repJetGaugeGroupI [source]

The `(1, 2)_{-3}` action of the jet gauge group on the jet space of the lepton doublet: the jet action the general theory derives from the datum.

```
noncomputable def repJetGaugeGroupI :
    Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] LeptonDoubletV)
```

lemma repJetGaugeGroupI_smul [source]

***The jet gauge action on the jets of the lepton doublet is fibrewise**: it commutes with multiplication by scalar jets.*

```
lemma repJetGaugeGroupI_smul (U : JetGaugeGroupI) (χ : SpaceTimeAlgebra)
    (z : SpaceTimeAlgebra ⊗[ℂ] LeptonDoubletV) :
    repJetGaugeGroupI U (χ • z) = χ • repJetGaugeGroupI U z
```

lemma repJetGaugeGroupI_ofConstant [source]

On jets of constant gauge transformations the jet action reduces to the global gauge action on the fibre: the `(1, 2)_{-3}` action on the lepton-doublet factor, and the trivial action on the jet ring.

```
lemma repJetGaugeGroupI_ofConstant (g : GaugeGroupI) :
    repJetGaugeGroupI (JetGaugeGroupI.ofConstant g) =
      TensorProduct.map LinearMap.id (repGaugeGroupI g)
```

lemma repLorentzGroup_apply_basis [source]

The Lorentz action on the lepton-doublet basis: the weak index is inert and the spinor index transforms by the matrix itself.

```
lemma repLorentzGroup_apply_basis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 2) :
    repLorentzGroup Λ (basis j) = ∑ β, Λ.1 β j.1 • basis (β, j.2)
```

lemma repLorentzGroup_dual_dualBasis [source]

The lepton-doublet coordinate functionals transform contragrediently, by the inverse matrix.

```
lemma repLorentzGroup_dual_dualBasis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 2) :
    repLorentzGroup.dual Λ (basis.dualBasis j) =
      ∑ β, (Λ⁻¹).1 j.1 β • basis.dualBasis (β, j.2)
```

lemma repLorentzGroup_conj_apply_basis [source]

The Lorentz action on the conjugate lepton-doublet basis: the coefficients are the conjugates of those of the lepton-doublet action.

```
lemma repLorentzGroup_conj_apply_basis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 2) :
    repLorentzGroup.conj Λ ((Basis.conj basis) j)
      = ∑ β, star (Λ.1 β j.1) • (Basis.conj basis) (β, j.2)
```

lemma repLorentzGroup_conj_dual_dualBasis [source]

The conjugate lepton-doublet coordinate functionals transform by the entrywise conjugate of the inverse matrix.

```
lemma repLorentzGroup_conj_dual_dualBasis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 2) :
    repLorentzGroup.conj.dual Λ ((Basis.conj basis).dualBasis j) =
      ∑ β, star ((Λ⁻¹).1 j.1 β) • (Basis.conj basis).dualBasis (β, j.2)
```

lemma repLorentzGroup_neg_one [source]

***The centre of `SL(2,ℂ)` acts on the lepton-doublet space by `-1`**: the value space carries a single Weyl-spinor index, and `-1` is not the identity on a half-integer spin.*

```
lemma repLorentzGroup_neg_one : repLorentzGroup (-1) = -LinearMap.id
```

lemma repLorentzGroup_conj_neg_one [source]

The centre acts on the conjugate lepton-doublet space by `-1` as well: conjugation does not move a real sign.

```
lemma repLorentzGroup_conj_neg_one : repLorentzGroup.conj (-1) = -LinearMap.id
```

lemma repGaugeGroupI_dual_dualBasis [source]

The lepton-doublet coordinate functionals carry the contragredient gauge action: the hypercharge and `SU(2)` factors of the inverse group element, transposed.

```
lemma repGaugeGroupI_dual_dualBasis (g : GaugeGroupI) (j : Fin 2 × Fin 2) :
    repGaugeGroupI.dual g (basis.dualBasis j) =
      ∑ w, (star (g⁻¹).toU1.1 ^ 3 * (g⁻¹).toSU2.1 j.2 w) • basis.dualBasis (j.1, w)
```

lemma repGaugeGroupI_conj_apply_basis [source]

The gauge action on the conjugate lepton-doublet basis: the coefficients of the lepton-doublet action, conjugated.

```
lemma repGaugeGroupI_conj_apply_basis (g : GaugeGroupI) (j : Fin 2 × Fin 2) :
    repGaugeGroupI.conj g ((Basis.conj basis) j) =
      ∑ w, star (star g.toU1.1 ^ 3 * g.toSU2.1 w j.2) • (Basis.conj basis) (j.1, w)
```

lemma repGaugeGroupI_conj_dual_dualBasis [source]

The conjugate lepton-doublet coordinate functionals carry the conjugate of the contragredient gauge action.

```
lemma repGaugeGroupI_conj_dual_dualBasis (g : GaugeGroupI) (j : Fin 2 × Fin 2) :
    repGaugeGroupI.conj.dual g ((Basis.conj basis).dualBasis j) =
      ∑ w, star (star (g⁻¹).toU1.1 ^ 3 * (g⁻¹).toSU2.1 j.2 w) •
        (Basis.conj basis).dualBasis (j.1, w)
```

def valueGaugeWeight [source]

The gauge weight of the lepton-doublet basis: the isospin weight and hypercharge `-3`.

```
def LeptonDoubletV.valueGaugeWeight (j : Fin 2 × Fin 2) : GaugeWeight
```

lemma repGaugeGroupI_gaugeTorusGen_basis [source]

The gauge torus acts diagonally on the basis of `LeptonDoubletV`, with the weights `LeptonDoubletV.valueGaugeWeight`.

```
lemma LeptonDoubletV.repGaugeGroupI_gaugeTorusGen_basis (i : Fin 4) (j : Fin 2 × Fin 2) :
    LeptonDoubletV.repGaugeGroupI (gaugeTorusGen i) (LeptonDoubletV.basis j)
      = ((expI : ℂ) ^ GaugeWeight.coord (LeptonDoubletV.valueGaugeWeight j) i) •
        LeptonDoubletV.basis j
```

lemma repGaugeGroupI_dual_gaugeTorusGen_coord [source]

The dual action of the gauge torus on the coordinate functionals of `LeptonDoubletV`: the weights are negated.

```
lemma LeptonDoubletV.repGaugeGroupI_dual_gaugeTorusGen_coord (i : Fin 4) (j : Fin 2 × Fin 2) :
    LeptonDoubletV.repGaugeGroupI.dual (gaugeTorusGen i) (LeptonDoubletV.basis.coord j)
      = ((expI : ℂ) ^ (-(GaugeWeight.coord (LeptonDoubletV.valueGaugeWeight j) i))) •
        LeptonDoubletV.basis.coord j
```

lemma repGaugeGroupI_conj_dual_gaugeTorusGen_coord [source]

The dual of the conjugate action of the gauge torus on the coordinate functionals of the conjugate of `LeptonDoubletV`: the two negations cancel and the weights are those of the value space.

```
lemma LeptonDoubletV.repGaugeGroupI_conj_dual_gaugeTorusGen_coord (i : Fin 4) (j : Fin 2 × Fin
    2) :
    LeptonDoubletV.repGaugeGroupI.conj.dual (gaugeTorusGen i)
      (((Basis.conj LeptonDoubletV.basis)).coord j)
      = ((expI : ℂ) ^ GaugeWeight.coord (LeptonDoubletV.valueGaugeWeight j) i) •
        ((Basis.conj LeptonDoubletV.basis)).coord j
```

lemma leptonDoublet_repLorentzGroup_boostAxis_two_basis [source]

The lepton-doublet basis diagonalises the `z`-boost: the isospin index is inert.

```
lemma leptonDoublet_repLorentzGroup_boostAxis_two_basis (t : ℝ) (ht : t ≠ 0)
    (j : Fin 2 × Fin 2) :
    LeptonDoubletV.repLorentzGroup (SL2C.boostAxis 2 t ht) (LeptonDoubletV.basis j)
      = ((t : ℝ) : ℂ) ^ (weylWeight j.1) • LeptonDoubletV.basis j
```

2.431 PhyslibAlpha.Particles.StandardModel.Fermions.LeptonDoublet.GaugeAlgebraAction

PhyslibAlpha/Particles/StandardModel/Fermions/LeptonDoublet/GaugeAlgebraAction.lean on GitHub.

def weakEnd [source]

The endomorphism of the lepton doublet defined by a `2 × 2` complex matrix acting on the weak index, with the Weyl factor untouched.

```
noncomputable def weakEnd (A : Matrix (Fin 2) (Fin 2) ℂ) :
    LeptonDoubletV →ₗ[ℂ] LeptonDoubletV
```

lemma weakEnd_tmul [source]

The weak endomorphism on a pure spinor–weak tensor: the matrix acts on the weak coordinates.

```
lemma weakEnd_tmul (A : Matrix (Fin 2) (Fin 2) ℂ) (s : Fermion.LeftHandedWeyl)
    (v : Fin 2 → ℂ) :
    weakEnd A (s ⊗ₜ v) = s ⊗ₜ A.mulVec v
```

lemma weakEnd_add [source]

```
lemma weakEnd_add (A B : Matrix (Fin 2) (Fin 2) ℂ) :
    weakEnd (A + B) = weakEnd A + weakEnd B
```

lemma weakEnd_smul [source]

```
lemma weakEnd_smul (z : ℂ) (A : Matrix (Fin 2) (Fin 2) ℂ) :
    weakEnd (z • A) = z • weakEnd A
```

lemma weakEnd_zero [source]

```
lemma weakEnd_zero : weakEnd 0 = 0
```

lemma weakEnd_neg [source]

```
lemma weakEnd_neg (A : Matrix (Fin 2) (Fin 2) ℂ) : weakEnd (-A) = -weakEnd A
```

lemma weakEnd_multiset_sum [source]

```
lemma weakEnd_multiset_sum (m : Multiset (Matrix (Fin 2) (Fin 2) ℂ)) :
    weakEnd m.sum = (m.map weakEnd).sum
```

lemma weakEnd_mul [source]

The weak endomorphisms compose through matrix multiplication.

```
lemma weakEnd_mul (A B : Matrix (Fin 2) (Fin 2) ℂ) :
    weakEnd (A * B) = weakEnd A ∘ₗ weakEnd B
```

lemma weakEnd_one [source]

The weak endomorphism of the identity matrix is the identity.

```
lemma weakEnd_one : weakEnd 1 = LinearMap.id
```

def actionMatrix [source]

The matrix of the infinitesimal `(1, 2)_{-3}` action of a gauge algebra element on the weak index: the action matrix of the datum.

```
noncomputable def actionMatrix (c : GaugeAlgebra) : Matrix (Fin 2) (Fin 2) ℂ
```

lemma actionMatrix_eq [source]

The action matrix is `i` times the weak part, shifted by `i` times `-3` the hypercharge.

```
lemma actionMatrix_eq (c : GaugeAlgebra) :
    actionMatrix c = Complex.I • (c.toSU2Matrix - ((3 : ℂ) • c.toU1Value) • 1)
```

def gaugeAlgebraAction [source]

***The infinitesimal action of the gauge algebra on the lepton doublet**: the derivative of the `(1, 2)_{-3}` action of the gauge group, real-linear in the algebra slot and complex-linear in the value slot — the form consumed by the covariant derivative `GaugeAlgebraRealization.covDerivIter` and by `LocalGaugeData.IsInfinitesimalActionOf`. It is the action the general theory derives from the datum.*

```
noncomputable def gaugeAlgebraAction :
    GaugeAlgebra →ₗ[ℝ] LeptonDoubletV →ₗ[ℂ] LeptonDoubletV
```

lemma gaugeAlgebraAction_apply [source]

The gauge algebra acts by the weak endomorphism of its action matrix.

```
lemma gaugeAlgebraAction_apply (c : GaugeAlgebra) :
    gaugeAlgebraAction c = weakEnd (actionMatrix c)
```

def jetActionMatrix [source]

The jet-valued matrix of the infinitesimal `(1, 2)_{-3}` action of a jet of gauge algebra elements: the jet action matrix of the datum.

```
noncomputable def jetActionMatrix (a : JetGaugeAlgebra) : Matrix (Fin 2) (Fin 2)
    SpaceTimeAlgebra
```

lemma jetActionMatrix_eq [source]

The jet action matrix is `i` times the weak part, shifted by `i` times `-3` the hypercharge.

```
lemma jetActionMatrix_eq (a : JetGaugeAlgebra) :
    jetActionMatrix a = Complex.I • (a.toSU2Matrix - ((3 : ℂ) • a.toU1Value) • 1)
```

lemma jetActionMatrix_map_cc_foldl [source]

The base-point Taylor coefficients of the jet action matrix are the action matrices of the base-point Taylor coefficients.

```
lemma jetActionMatrix_map_cc_foldl (p : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    ((jetActionMatrix a).map fun f =>
        constantCoeff (SpaceTimeAlgebra.iteratedPDeriv p f))
```

```
      = actionMatrix (JetGaugeAlgebra.eval (JetGaugeAlgebra.iteratedDeriv p a))
```

lemma repCoeff_eq [source]

***The base-point Taylor coefficients of the jet gauge action** on the lepton doublet are the weak endomorphisms of the base-point Taylor coefficients of the weak matrix.*

```
lemma repCoeff_eq (U : JetGaugeGroupI) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U x
      = weakEnd ((doubletMatrix U).map fun f =>
          constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x f))
```

lemma repCoeff_zero_of_eval_eq_one [source]

At the base point, a gauge jet with trivial value acts trivially: the zeroth Taylor coefficient of the jet gauge action is the identity.

```
lemma repCoeff_zero_of_eval_eq_one {U : JetGaugeGroupI} (hU : U.eval = 1) :
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U 0 = LinearMap.id
```

lemma isInfinitesimalActionOf [source]

***The `(1, 2)_{-3}` action of the gauge algebra is the infinitesimal action underlying the jet gauge action on the lepton doublet**: its base-point Taylor coefficients obey the Maurer–Cartan Leibniz law and intertwine the action with the adjoint transports. This is the generic statement for the datum's matrix representation.*

```
lemma isInfinitesimalActionOf :
    localGaugeData.IsInfinitesimalActionOf gaugeAlgebraAction repJetGaugeGroupI
```

lemma gaugeAlgebraAction_comm_repLorentzGroup [source]

The infinitesimal gauge action on the lepton doublet acts on the weak factor, the Lorentz action on the Weyl factor, so the two commute.

```
lemma gaugeAlgebraAction_comm_repLorentzGroup (c : GaugeAlgebra) (Λ : SL(2,ℂ))
    (v : LeptonDoubletV) :
    LeptonDoubletV.gaugeAlgebraAction c (LeptonDoubletV.repLorentzGroup Λ v) =
      LeptonDoubletV.repLorentzGroup Λ (LeptonDoubletV.gaugeAlgebraAction c v)
```

2.432 PhyslibAlpha.Particles.StandardModel.Fermions.LeptonSinglet.Basic

PhyslibAlpha/Particles/StandardModel/Fermions/LeptonSinglet/Basic.lean on GitHub.

def basis [source]

A basis on the charged-lepton singlets.

```
noncomputable def basis : Module.Basis (Fin 2) ℂ LeptonSinglet
```

instance Module.Finite ℂ LeptonSinglet [source]

```
instance : Module.Finite ℂ LeptonSinglet
```

instance Module.Free ℂ LeptonSinglet [source]

```
instance : Module.Free ℂ LeptonSinglet
```

lemma repLorentzGroup_apply_basis [source]

The Lorentz action on the lepton-singlet basis: the right-handed Weyl action by the entrywise conjugate matrix.

```
lemma repLorentzGroup_apply_basis (Λ : SL(2,ℂ)) (α : Fin 2) :
    repLorentzGroup Λ (basis α) = ∑ β, star (Λ.1 β α) • basis β
```

lemma repLorentzGroup_dual_dualBasis [source]

The lepton jet coordinates transform contragrediently, by the entrywise conjugate of the inverse matrix.

```
lemma repLorentzGroup_dual_dualBasis (Λ : SL(2,ℂ)) (α : Fin 2) :
    repLorentzGroup.dual Λ (basis.dualBasis α) =
      ∑ β, star ((Λ⁻¹).1 α β) • basis.dualBasis β
```

lemma repLorentzGroup_conj_apply_basis [source]

The Lorentz action on the conjugate lepton basis: the coefficients are the conjugates of those of the lepton action, that is, the matrix itself.

```
lemma repLorentzGroup_conj_apply_basis (Λ : SL(2,ℂ)) (α : Fin 2) :
    repLorentzGroup.conj Λ ((Basis.conj basis) α) = ∑ β, Λ.1 β α • (Basis.conj basis) β
```

lemma repLorentzGroup_conj_dual_dualBasis [source]

The conjugate lepton jet coordinates transform by the inverse matrix.

```
lemma repLorentzGroup_conj_dual_dualBasis (Λ : SL(2,ℂ)) (α : Fin 2) :
    repLorentzGroup.conj.dual Λ ((Basis.conj basis).dualBasis α) =
      ∑ β, (Λ⁻¹).1 α β • (Basis.conj basis).dualBasis β
```

lemma repLorentzGroup_neg_one [source]

***The centre of `SL(2,ℂ)` acts on the charged-lepton-singlet space by `-1`**: the value space carries a single Weyl-spinor index, and `-1` is not the identity on a half-integer spin.*

```
lemma repLorentzGroup_neg_one : repLorentzGroup (-1) = -LinearMap.id
```

lemma repLorentzGroup_conj_neg_one [source]

The centre acts on the conjugate charged-lepton-singlet space by `-1` as well: conjugation does not move a real sign.

```
lemma repLorentzGroup_conj_neg_one : repLorentzGroup.conj (-1) = -LinearMap.id
```

lemma repGaugeGroupI_apply_basis [source]

The gauge action on the lepton-singlet basis: multiplication by the hypercharge scalar, the colour and weak factors acting trivially.

```
lemma repGaugeGroupI_apply_basis (g : GaugeGroupI) (α : Fin 2) :
    repGaugeGroupI g (basis α) = (star g.toU1.1 ^ 6 : ℂ) • basis α
```

lemma repGaugeGroupI_dual_dualBasis [source]

The lepton-singlet coordinate functionals carry the contragredient gauge action: the hypercharge scalar of the inverse group element.

```
lemma repGaugeGroupI_dual_dualBasis (g : GaugeGroupI) (α : Fin 2) :
    repGaugeGroupI.dual g (basis.dualBasis α) =
      (star (g⁻¹).toU1.1 ^ 6 : ℂ) • basis.dualBasis α
```

lemma repGaugeGroupI_conj_apply_basis [source]

The gauge action on the conjugate lepton-singlet basis: the hypercharge scalar, conjugated.

```
lemma repGaugeGroupI_conj_apply_basis (g : GaugeGroupI) (α : Fin 2) :
    repGaugeGroupI.conj g ((Basis.conj basis) α) =
      star (star g.toU1.1 ^ 6 : ℂ) • (Basis.conj basis) α
```

lemma repGaugeGroupI_conj_dual_dualBasis [source]

The conjugate lepton-singlet coordinate functionals carry the conjugate of the contragredient gauge action.

```
lemma repGaugeGroupI_conj_dual_dualBasis (g : GaugeGroupI) (α : Fin 2) :
    repGaugeGroupI.conj.dual g ((Basis.conj basis).dualBasis α) =
      star (star (g⁻¹).toU1.1 ^ 6 : ℂ) • (Basis.conj basis).dualBasis α
```

def repJetGaugeGroupI [source]

The `(1, 1)_{-6}` action of the jet gauge group on the jet space of the charged-lepton singlet: multiplication of the jet-ring factor by the hypercharge power series `(star u) ^ 6`.

```
noncomputable def repJetGaugeGroupI :
    Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] LeptonSinglet) where
```

lemma repJetGaugeGroupI_tmul [source]

The jet gauge action on a pure tensor of the jet space of the charged-lepton singlet.

```
lemma repJetGaugeGroupI_tmul (U : JetGaugeGroupI) (f : SpaceTimeAlgebra) (ψ : LeptonSinglet) :
    repJetGaugeGroupI U (f ⊗ₜ[ℂ] ψ)
      = ((star ((U.2.2 : unitary SpaceTimeAlgebra) : SpaceTimeAlgebra)) ^ 6 * f) ⊗ₜ[ℂ] ψ
```

lemma repJetGaugeGroupI_smul [source]

***The jet gauge action on the jets of the charged-lepton singlet is fibrewise**: it commutes with multiplication by scalar jets.*

```
lemma repJetGaugeGroupI_smul (U : JetGaugeGroupI) (χ : SpaceTimeAlgebra)
    (z : SpaceTimeAlgebra ⊗[ℂ] LeptonSinglet) :
    repJetGaugeGroupI U (χ • z) = χ • repJetGaugeGroupI U z
```

lemma repJetGaugeGroupI_ofConstant [source]

On jets of constant gauge transformations the jet action reduces to the global gauge action on the fibre: the `(1, 1)_{-6}` action on the lepton-singlet factor, and the trivial action on the jet ring.

```
lemma repJetGaugeGroupI_ofConstant (g : GaugeGroupI) :
    repJetGaugeGroupI (JetGaugeGroupI.ofConstant g) =
      TensorProduct.map LinearMap.id (repGaugeGroupI g)
```

def valueGaugeWeight [source]

The gauge weight of the lepton-singlet basis: hypercharge `-6`.

```
def LeptonSinglet.valueGaugeWeight (_ : Fin 2) : GaugeWeight
```

lemma repGaugeGroupI_gaugeTorusGen_basis [source]

The gauge torus acts diagonally on the basis of `LeptonSinglet`, with the weights `LeptonSinglet.valueGaugeWeight`.

```
lemma LeptonSinglet.repGaugeGroupI_gaugeTorusGen_basis (i : Fin 4) (j : Fin 2) :
    LeptonSinglet.repGaugeGroupI (gaugeTorusGen i) (LeptonSinglet.basis j)
      = ((expI : ℂ) ^ GaugeWeight.coord (LeptonSinglet.valueGaugeWeight j) i) •
        LeptonSinglet.basis j
```

lemma repGaugeGroupI_dual_gaugeTorusGen_coord [source]

The dual action of the gauge torus on the coordinate functionals of `LeptonSinglet`: the weights are negated.

```
lemma LeptonSinglet.repGaugeGroupI_dual_gaugeTorusGen_coord (i : Fin 4) (j : Fin 2) :
    LeptonSinglet.repGaugeGroupI.dual (gaugeTorusGen i) (LeptonSinglet.basis.coord j)
      = ((expI : ℂ) ^ (-(GaugeWeight.coord (LeptonSinglet.valueGaugeWeight j) i))) •
        LeptonSinglet.basis.coord j
```

lemma repGaugeGroupI_conj_dual_gaugeTorusGen_coord [source]

The dual of the conjugate action of the gauge torus on the coordinate functionals of the conjugate of `LeptonSinglet`: the two negations cancel and the weights are those of the value space.

```
lemma LeptonSinglet.repGaugeGroupI_conj_dual_gaugeTorusGen_coord (i : Fin 4) (j : Fin 2) :
    LeptonSinglet.repGaugeGroupI.conj.dual (gaugeTorusGen i)
      (((Basis.conj LeptonSinglet.basis)).coord j)
      = ((expI : ℂ) ^ GaugeWeight.coord (LeptonSinglet.valueGaugeWeight j) i) •
```

```
        ((Basis.conj LeptonSinglet.basis)).coord j
```

lemma leptonSinglet_repLorentzGroup_boostAxis_two_basis [source]

The charged-lepton-singlet basis diagonalises the `z`-boost.

```
lemma leptonSinglet_repLorentzGroup_boostAxis_two_basis (t : ℝ) (ht : t ≠ 0) (j : Fin 2) :
    LeptonSinglet.repLorentzGroup (SL2C.boostAxis 2 t ht) (LeptonSinglet.basis j)
      = ((t : ℝ) : ℂ) ^ (weylWeight j) • LeptonSinglet.basis j
```

2.433 PhyslibAlpha.Particles.StandardModel.Fermions.LeptonSinglet.GaugeAlgebraAction

PhyslibAlpha/Particles/StandardModel/Fermions/LeptonSinglet/GaugeAlgebraAction.lean on GitHub.

def gaugeAlgebraAction [source]

***The infinitesimal action of the gauge algebra on the charged-lepton singlet**: the derivative of the `(1, 1)_{-6}` action of the gauge group — scalar multiplication by `i` times `-6` times the `u(1)` value, real-linear in the algebra slot and complex-linear in the value slot — the form consumed by `LocalGaugeData.IsInfinitesimalActionOf`.*

```
noncomputable def gaugeAlgebraAction :
    GaugeAlgebra →ₗ[ℝ] LeptonSinglet →ₗ[ℂ] LeptonSinglet where
```

lemma gaugeAlgebraAction_apply [source]

The gauge-algebra action on the charged-lepton singlet is scalar multiplication by `i` times `-6` times the `u(1)` value.

```
lemma gaugeAlgebraAction_apply (c : GaugeAlgebra) :
    gaugeAlgebraAction c
      = (Complex.I * (-(6 : ℂ) * c.toU1Value))
        • (LinearMap.id : LeptonSinglet →ₗ[ℂ] LeptonSinglet)
```

def jetPhase [source]

The `SpaceTimeAlgebra`-valued hypercharge phase of the jet gauge action on the charged-lepton singlet: the `-6` hypercharge power series `(star u) ^ 6` of the gauge jet.

```
noncomputable def jetPhase (U : JetGaugeGroupI) : SpaceTimeAlgebra
```

lemma jetPhase_eq [source]

The hypercharge phase, unfolded.

```
lemma jetPhase_eq (U : JetGaugeGroupI) :
    jetPhase U = (star ((U.2.2 : unitary SpaceTimeAlgebra) : SpaceTimeAlgebra)) ^ 6
```

lemma repJetGaugeGroupI_eq_jetPhase [source]

The jet gauge action on the charged-lepton singlet is multiplication of the jet-ring factor by the hypercharge phase.

```
lemma repJetGaugeGroupI_eq_jetPhase (U : JetGaugeGroupI) :
    repJetGaugeGroupI U
      = LinearMap.rTensor LeptonSinglet (LinearMap.mulLeft ℂ (jetPhase U))
```

lemma jetIteratedDeriv_tmul [source]

The iterated formal derivative of a jet of charged-lepton singlets acts on the jet-ring factor of a pure tensor.

```
private lemma jetIteratedDeriv_tmul (x : Multiset (Fin 1 ⊕ Fin 3)) (f : SpaceTimeAlgebra)
    (ψ : LeptonSinglet) :
    jetIteratedDeriv x (f ⊗ₜ[ℂ] ψ)
      = SpaceTimeAlgebra.iteratedPDeriv x f ⊗ₜ[ℂ] ψ
```

lemma smul_id_comp [source]

Scalar multiples of the identity compose through multiplication.

```
private lemma smul_id_comp (a b : ℂ) :
    (a • (LinearMap.id : LeptonSinglet →ₗ[ℂ] LeptonSinglet))
        ∘ₗ (b • (LinearMap.id : LeptonSinglet →ₗ[ℂ] LeptonSinglet))
      = (a * b) • (LinearMap.id : LeptonSinglet →ₗ[ℂ] LeptonSinglet)
```

lemma sum_map_smul_id [source]

A multiset sum of scalar multiples of the identity is the scalar multiple by the sum.

```
private lemma sum_map_smul_id {α : Type*} (m : Multiset α) (z : α → ℂ) :
    (m.map fun p => z p • (LinearMap.id : LeptonSinglet →ₗ[ℂ] LeptonSinglet)).sum
      = (m.map z).sum • (LinearMap.id : LeptonSinglet →ₗ[ℂ] LeptonSinglet)
```

lemma repCoeff_eq [source]

***The base-point Taylor coefficients of the jet gauge action** on the charged-lepton singlet are scalar: multiplication by the base-point Taylor coefficients of the hypercharge phase.*

```
lemma repCoeff_eq (U : JetGaugeGroupI) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U x
      = constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x (jetPhase U))
        • (LinearMap.id : LeptonSinglet →ₗ[ℂ] LeptonSinglet)
```

lemma repCoeff_zero_of_eval_eq_one [source]

At the base point, a gauge jet with trivial value acts trivially: the zeroth Taylor coefficient of the jet gauge action is the identity.

```
lemma repCoeff_zero_of_eval_eq_one {U : JetGaugeGroupI} (hU : U.eval = 1) :
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U 0 = LinearMap.id
```

lemma jetPhase_pderiv [source]

***The derivative identity** for the hypercharge phase of the jet gauge action: the formal derivative of the phase is minus `i` times `-6` times the `u(1)` value of the Maurer–Cartan form, times the phase.*

```
lemma jetPhase_pderiv (U : JetGaugeGroupI) (μ : Fin 1 ⊕ Fin 3) :
    pderiv μ (jetPhase U)
      = -(((Complex.I * (-(6 : ℂ))) • (maurerCartanForm U μ).toU1Value)
          * jetPhase U)
```

lemma isInfinitesimalActionOf [source]

***The `(1, 1)_{-6}` action of the gauge algebra is the infinitesimal action underlying the jet gauge action on the charged-lepton singlet**: its base-point Taylor coefficients obey the Maurer–Cartan Leibniz law and intertwine the action with the adjoint transports.*

```
lemma isInfinitesimalActionOf :
    localGaugeData.IsInfinitesimalActionOf gaugeAlgebraAction repJetGaugeGroupI
```

lemma gaugeAlgebraAction_comm_repLorentzGroup [source]

The infinitesimal gauge action on the charged-lepton singlet is a scalar, so it commutes with the Lorentz action.

```
lemma gaugeAlgebraAction_comm_repLorentzGroup (c : GaugeAlgebra) (Λ : SL(2,ℂ))
    (v : LeptonSinglet) :
    LeptonSinglet.gaugeAlgebraAction c (LeptonSinglet.repLorentzGroup Λ v) =
      LeptonSinglet.repLorentzGroup Λ (LeptonSinglet.gaugeAlgebraAction c v)
```

2.434 PhyslibAlpha.Particles.StandardModel.Fermions.MatterField

PhyslibAlpha/Particles/StandardModel/Fermions/MatterField.lean on GitHub.

def LeptonDoubletV.matterField [source]

The lepton doublet as a matter field of `StandardModel.localGaugeData`, in the `(1, 2)_{-3}` representation with its left-handed Lorentz action: the matter field the general theory derives from the table's datum `StandardModel.Model.leptonDoublet`.

```
noncomputable def matterField : MatterField localGaugeData
```

lemma LeptonDoubletV.matterField_V [source]

```
@[simp]
lemma matterField_V : matterField.V = LeptonDoubletV
```

lemma LeptonDoubletV.matterField_repLorentz [source]

```
@[simp]
lemma matterField_repLorentz : matterField.repLorentz = repLorentzGroup
```

lemma LeptonDoubletV.matterField_repJet [source]

```
@[simp]
lemma matterField_repJet : matterField.repJet = repJetGaugeGroupI
```

lemma LeptonDoubletV.matterField_repAlgebra [source]

```
@[simp]
lemma matterField_repAlgebra : matterField.repAlgebra = gaugeAlgebraAction
```

lemma LeptonDoubletV.matterField_massWeight [source]

```
@[simp]
lemma matterField_massWeight : matterField.massWeight = 3
```

lemma LeptonDoubletV.matterField_pureJetsActTrivially [source]

Pure gauge jets act trivially on the lepton doublet at the base point.

```
lemma matterField_pureJetsActTrivially : matterField.PureJetsActTrivially
```

lemma LeptonDoubletV.matterField_gaugeLorentzCompatible [source]

The gauge and Lorentz actions on the lepton doublet commute.

```
lemma matterField_gaugeLorentzCompatible : matterField.GaugeLorentzCompatible
```

def LeptonSinglet.matterField [source]

The charged-lepton singlet as a matter field of `StandardModel.localGaugeData`, in the `(1, 1)_{-6}` representation with its right-handed Lorentz action.

```
noncomputable def matterField : MatterField localGaugeData where
```

lemma LeptonSinglet.matterField_V [source]

```
@[simp]
lemma matterField_V : matterField.V = LeptonSinglet
```

lemma LeptonSinglet.matterField_repLorentz [source]

```
@[simp]
lemma matterField_repLorentz : matterField.repLorentz = repLorentzGroup
```

lemma LeptonSinglet.matterField_repJet [source]

```
@[simp]
lemma matterField_repJet : matterField.repJet = repJetGaugeGroupI
```

lemma LeptonSinglet.matterField_repAlgebra [source]

```
@[simp]
lemma matterField_repAlgebra : matterField.repAlgebra = gaugeAlgebraAction
```

lemma LeptonSinglet.matterField_massWeight [source]

```
@[simp]
lemma matterField_massWeight : matterField.massWeight = 3
```

lemma LeptonSinglet.matterField_pureJetsActTrivially [source]

Pure gauge jets act trivially on the charged-lepton singlet at the base point.

```
lemma matterField_pureJetsActTrivially : matterField.PureJetsActTrivially
```

lemma LeptonSinglet.matterField_gaugeLorentzCompatible [source]

The gauge and Lorentz actions on the charged-lepton singlet commute.

```
lemma matterField_gaugeLorentzCompatible : matterField.GaugeLorentzCompatible
```

def QuarkDoublet.matterField [source]

The quark doublet as a matter field of `StandardModel.localGaugeData`, in the `(3, 2)_{1}` representation with its left-handed Lorentz action.

```
noncomputable def matterField : MatterField localGaugeData where
```

lemma QuarkDoublet.matterField_V [source]

```
@[simp]
lemma matterField_V : matterField.V = QuarkDoublet
```

lemma QuarkDoublet.matterField_repLorentz [source]

```
@[simp]
lemma matterField_repLorentz : matterField.repLorentz = repLorentzGroup
```

lemma QuarkDoublet.matterField_repJet [source]

```
@[simp]
lemma matterField_repJet : matterField.repJet = repJetGaugeGroupI
```

lemma QuarkDoublet.matterField_repAlgebra [source]

```
@[simp]
lemma matterField_repAlgebra : matterField.repAlgebra = gaugeAlgebraAction
```

lemma QuarkDoublet.matterField_massWeight [source]

```
@[simp]
lemma matterField_massWeight : matterField.massWeight = 3
```

lemma QuarkDoublet.matterField_pureJetsActTrivially [source]

Pure gauge jets act trivially on the quark doublet at the base point.

```
lemma matterField_pureJetsActTrivially : matterField.PureJetsActTrivially
```

lemma QuarkDoublet.matterField_gaugeLorentzCompatible [source]

The gauge and Lorentz actions on the quark doublet commute.

```
lemma matterField_gaugeLorentzCompatible : matterField.GaugeLorentzCompatible
```

def UpSinglet.matterField [source]

The up-type quark singlet as a matter field of `StandardModel.localGaugeData`, in the `(3, 1)_{4}` representation with its right-handed Lorentz action.

```
noncomputable def matterField : MatterField localGaugeData where
```

lemma UpSinglet.matterField_V [source]

```
@[simp]
lemma matterField_V : matterField.V = UpSinglet
```

lemma UpSinglet.matterField_repLorentz [source]

```
@[simp]
lemma matterField_repLorentz : matterField.repLorentz = repLorentzGroup
```

lemma UpSinglet.matterField_repJet [source]

```
@[simp]
lemma matterField_repJet : matterField.repJet = repJetGaugeGroupI
```

lemma UpSinglet.matterField_repAlgebra [source]

```
@[simp]
lemma matterField_repAlgebra : matterField.repAlgebra = gaugeAlgebraAction
```

lemma UpSinglet.matterField_massWeight [source]

```
@[simp]
lemma matterField_massWeight : matterField.massWeight = 3
```

lemma UpSingletmatterField_pureJetsActTrivially [source]

Pure gauge jets act trivially on the up-type singlet at the base point.

```
lemma matterField_pureJetsActTrivially : matterField.PureJetsActTrivially
```

lemma UpSingletmatterField_gaugeLorentzCompatible [source]

The gauge and Lorentz actions on the up-type singlet commute.

```
lemma matterField_gaugeLorentzCompatible : matterField.GaugeLorentzCompatible
```

def DownSingletmatterField [source]

The down-type quark singlet as a matter field of `StandardModel.localGaugeData`, in the `(3, 1)_{-2}` representation with its right-handed Lorentz action.

```
noncomputable def matterField : MatterField localGaugeData where
```

lemma DownSingletmatterField_V [source]

```
@[simp]
lemma matterField_V : matterField.V = DownSinglet
```

lemma DownSingletmatterField_repLorentz [source]

```
@[simp]
lemma matterField_repLorentz : matterField.repLorentz = repLorentzGroup
```

lemma DownSingletmatterField_repJet [source]

```
@[simp]
lemma matterField_repJet : matterField.repJet = repJetGaugeGroupI
```

lemma DownSingletmatterField_repAlgebra [source]

```
@[simp]
lemma matterField_repAlgebra : matterField.repAlgebra = gaugeAlgebraAction
```

lemma DownSingletmatterField_massWeight [source]

```
@[simp]
lemma matterField_massWeight : matterField.massWeight = 3
```

lemma DownSingletmatterField_pureJetsActTrivially [source]

Pure gauge jets act trivially on the down-type singlet at the base point.

```
lemma matterField_pureJetsActTrivially : matterField.PureJetsActTrivially
```

lemma DownSinglet.matterField_gaugeLorentzCompatible [source]

The gauge and Lorentz actions on the down-type singlet commute.

```
lemma matterField_gaugeLorentzCompatible : matterField.GaugeLorentzCompatible
```

2.435 PhyslibAlpha.Particles.StandardModel.Fermions.QuarkDoublet.Basic

PhyslibAlpha/Particles/StandardModel/Fermions/QuarkDoublet/Basic.lean on GitHub.

def basis [source]

A basis on the quark doublets.

```
noncomputable def basis : Module.Basis (Fin 2 × Fin 3 × Fin 2) ℂ QuarkDoublet
```

instance Module.Finite ℂ QuarkDoublet [source]

```
instance : Module.Finite ℂ QuarkDoublet
```

instance Module.Free ℂ QuarkDoublet [source]

```
instance : Module.Free ℂ QuarkDoublet
```

def colourWeakEquiv [source]

The colour and weak factors of the quark doublet combined into a single Euclidean factor over `Fin 3 × Fin 2`.

```
noncomputable def colourWeakEquiv :
    EuclideanSpace ℂ (Fin 3) ⊗[ℂ] EuclideanSpace ℂ (Fin 2) ≃ₗ[ℂ] (Fin 3 × Fin 2 → ℂ)
```

lemma colourWeakEquiv_tmul [source]

```
@[simp]
lemma colourWeakEquiv_tmul (c : EuclideanSpace ℂ (Fin 3)) (w : EuclideanSpace ℂ (Fin 2))
    (p : Fin 3 × Fin 2) :
    colourWeakEquiv (c ⊗ₜ[ℂ] w) p = c.ofLp p.1 * w.ofLp p.2
```

def jetValLinEquiv [source]

Absorbs the jet ring into the combined colour–weak index: a jet of a quark doublet is the same thing as a left-handed Weyl spinor tensored with a `SpaceTimeAlgebra`-valued colour–weak vector,

`SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet ≃ LeftHandedWeyl ⊗[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 3 × Fin 2)`.

```
noncomputable def jetValLinEquiv :
    SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet ≃ₗ[ℂ]
      Fermion.LeftHandedWeyl ⊗[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 3 × Fin 2)
```

def jetGaugeMatrix [source]

The matrix of jets through which a jet of gauge transformations acts on the combined colour–weak index of the quark doublet: the Kronecker product of the `SU(3)` and `SU(2)` power-series matrices, scaled by the hypercharge power series `u`.

```
noncomputable def jetGaugeMatrix (U : JetGaugeGroupI) :
    Matrix (Fin 3 × Fin 2) (Fin 3 × Fin 2) SpaceTimeAlgebra
```

lemma jetGaugeMatrix_one [source]

```
lemma jetGaugeMatrix_one : jetGaugeMatrix 1 = 1
```

lemma jetGaugeMatrix_mul [source]

```
lemma jetGaugeMatrix_mul (U₁ U₂ : JetGaugeGroupI) :
    jetGaugeMatrix (U₁ * U₂) = jetGaugeMatrix U₁ * jetGaugeMatrix U₂
```

def repJetGaugeGroupI [source]

The `(3, 2)_{1}` action of the jet gauge group on the jet space of the quark doublet. Through `jetValLinEquiv` the Kronecker matrix of the gauge jet, carrying the hypercharge phase `u`, acts `SpaceTimeAlgebra`-linearly on the combined colour–weak factor by matrix-vector multiplication, while the Weyl factor is untouched.

```
noncomputable def repJetGaugeGroupI :
    Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet) where
```

lemma jetGaugeMatrix_ofConstant [source]

The entries of the gauge matrix of a jet of a constant gauge transformation are the constant power series with the global gauge coefficients.

```
lemma jetGaugeMatrix_ofConstant (g : GaugeGroupI) (p q : Fin 3 × Fin 2) :
    jetGaugeMatrix (JetGaugeGroupI.ofConstant g) p q =
      MvPowerSeries.C ((g.toU1.1 : ℂ) * (g.toSU3.1 p.1 q.1 * g.toSU2.1 p.2 q.2))
```

lemma jetValLinEquiv_smul [source]

The identification of the jets of the quark doublet intertwines multiplication by a scalar jet with the `SpaceTimeAlgebra`-scalar action on the colour–weak coordinates.

```
lemma jetValLinEquiv_smul (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet) :
    jetValLinEquiv (χ • z)
      = Module.End.lTensorAlgHom ℂ (EuclideanSpace SpaceTimeAlgebra (Fin 3 × Fin 2))
          Fermion.LeftHandedWeyl
          ((LinearMap.lsmul SpaceTimeAlgebra
            (EuclideanSpace SpaceTimeAlgebra (Fin 3 × Fin 2)) χ).restrictScalars ℂ)
          (jetValLinEquiv z)
```

lemma repJetGaugeGroupI_smul [source]

***The jet gauge action on the jets of the quark doublet is fibrewise**: it commutes with multiplication by scalar jets.*

```
lemma repJetGaugeGroupI_smul (U : JetGaugeGroupI) (χ : SpaceTimeAlgebra)
    (z : SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet) :
    repJetGaugeGroupI U (χ • z) = χ • repJetGaugeGroupI U z
```

lemma repJetGaugeGroupI_ofConstant [source]

On jets of constant gauge transformations the jet action reduces to the global gauge action on the fibre: the `(3, 2)_{1}` action on the quark-doublet factor, and the trivial action on the jet ring.

```
lemma repJetGaugeGroupI_ofConstant (g : GaugeGroupI) :
    repJetGaugeGroupI (JetGaugeGroupI.ofConstant g) =
      TensorProduct.map LinearMap.id (repGaugeGroupI g)
```

lemma basis_eq_mk [source]

The quark-doublet basis vector as an explicit spinor–colour–weak tensor.

```
lemma basis_eq_mk (k : Fin 2) (c : Fin 3) (w : Fin 2) : basis (k, c, w) =
    ⟨Fermion.LeftHandedWeyl.basis k ⊗ₜ[ℂ] EuclideanSpace.basisFun (Fin 3) ℂ c
      ⊗ₜ[ℂ] EuclideanSpace.basisFun (Fin 2) ℂ w⟩
```

lemma repLorentzGroup_apply_basis [source]

The Lorentz action on the quark-doublet basis: the colour and weak indices are inert and the spinor index transforms by the matrix itself.

```
lemma repLorentzGroup_apply_basis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3 × Fin 2) :
    repLorentzGroup Λ (basis j) = ∑ β, Λ.1 β j.1 • basis (β, j.2.1, j.2.2)
```

lemma repLorentzGroup_dual_dualBasis [source]

The quark-doublet coordinate functionals transform contragrediently, by the inverse matrix.

```
lemma repLorentzGroup_dual_dualBasis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3 × Fin 2) :
    repLorentzGroup.dual Λ (basis.dualBasis j) =
      ∑ β, (Λ⁻¹).1 j.1 β • basis.dualBasis (β, j.2.1, j.2.2)
```

lemma repLorentzGroup_conj_apply_basis [source]

The Lorentz action on the conjugate quark-doublet basis: the coefficients are the conjugates of those of the quark-doublet action.

```
lemma repLorentzGroup_conj_apply_basis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3 × Fin 2) :
    repLorentzGroup.conj Λ ((Basis.conj basis) j) =
      ∑ β, star (Λ.1 β j.1) • (Basis.conj basis) (β, j.2.1, j.2.2)
```

lemma repLorentzGroup_conj_dual_dualBasis [source]

The conjugate quark-doublet coordinate functionals transform by the entrywise conjugate of the inverse matrix.

```
lemma repLorentzGroup_conj_dual_dualBasis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3 × Fin 2) :
    repLorentzGroup.conj.dual Λ ((Basis.conj basis).dualBasis j) =
      ∑ β, star ((Λ⁻¹).1 j.1 β) • (Basis.conj basis).dualBasis (β, j.2.1, j.2.2)
```

lemma repLorentzGroup_neg_one [source]

***The centre of `SL(2,ℂ)` acts on the quark-doublet space by `-1`**: the value space carries a single Weyl-spinor index, and `-1` is not the identity on a half-integer spin.*

```
lemma repLorentzGroup_neg_one : repLorentzGroup (-1) = -LinearMap.id
```

lemma repLorentzGroup_conj_neg_one [source]

The centre acts on the conjugate quark-doublet space by `-1` as well: conjugation does not move a real sign.

```
lemma repLorentzGroup_conj_neg_one : repLorentzGroup.conj (-1) = -LinearMap.id
```

lemma repGaugeGroupI_apply_basis [source]

The gauge action on the quark-doublet basis: the spinor index is inert, the colour index transforms by the `SU(3)` matrix and the weak index by the `SU(2)` matrix, scaled by the hypercharge factor.

```
lemma repGaugeGroupI_apply_basis (g : GaugeGroupI) (j : Fin 2 × Fin 3 × Fin 2) :
    repGaugeGroupI g (basis j) =
      ∑ c, ∑ w, (g.toU1.1 * g.toSU3.1 c j.2.1 * g.toSU2.1 w j.2.2) • basis (j.1, c, w)
```

lemma repGaugeGroupI_dual_dualBasis [source]

The quark-doublet coordinate functionals carry the contragredient gauge action: the hypercharge, `SU(3)` and `SU(2)` factors of the inverse group element, transposed.

```
lemma repGaugeGroupI_dual_dualBasis (g : GaugeGroupI) (j : Fin 2 × Fin 3 × Fin 2) :
    repGaugeGroupI.dual g (basis.dualBasis j) =
      ∑ c, ∑ w, ((g⁻¹).toU1.1 * (g⁻¹).toSU3.1 j.2.1 c * (g⁻¹).toSU2.1 j.2.2 w) •
        basis.dualBasis (j.1, c, w)
```

lemma repGaugeGroupI_conj_apply_basis [source]

The gauge action on the conjugate quark-doublet basis: the coefficients of the quark-doublet action, conjugated.

```
lemma repGaugeGroupI_conj_apply_basis (g : GaugeGroupI) (j : Fin 2 × Fin 3 × Fin 2) :
    repGaugeGroupI.conj g ((Basis.conj basis) j) =
      ∑ c, ∑ w, star (g.toU1.1 * g.toSU3.1 c j.2.1 * g.toSU2.1 w j.2.2) •
        (Basis.conj basis) (j.1, c, w)
```

lemma repGaugeGroupI_conj_dual_dualBasis [source]

The conjugate quark-doublet coordinate functionals carry the conjugate of the contragredient gauge action.

```
lemma repGaugeGroupI_conj_dual_dualBasis (g : GaugeGroupI) (j : Fin 2 × Fin 3 × Fin 2) :
    repGaugeGroupI.conj.dual g ((Basis.conj basis).dualBasis j) =
      ∑ c, ∑ w, star ((g⁻¹).toU1.1 * (g⁻¹).toSU3.1 j.2.1 c * (g⁻¹).toSU2.1 j.2.2 w) •
        (Basis.conj basis).dualBasis (j.1, c, w)
```

def valueGaugeWeight [source]

The gauge weight of the quark-doublet basis: the colour and isospin weights and hypercharge `1`.

```
def QuarkDoublet.valueGaugeWeight (j : Fin 2 × Fin 3 × Fin 2) : GaugeWeight
```

lemma repGaugeGroupI_gaugeTorusGen_basis [source]

The gauge torus acts diagonally on the basis of `QuarkDoublet`, with the weights `QuarkDoublet.valueGaugeWeight`.

```
lemma QuarkDoublet.repGaugeGroupI_gaugeTorusGen_basis (i : Fin 4) (j : Fin 2 × Fin 3 × Fin 2) :
    QuarkDoublet.repGaugeGroupI (gaugeTorusGen i) (QuarkDoublet.basis j)
      = ((expI : ℂ) ^ GaugeWeight.coord (QuarkDoublet.valueGaugeWeight j) i) •
        QuarkDoublet.basis j
```

lemma repGaugeGroupI_dual_gaugeTorusGen_coord [source]

The dual action of the gauge torus on the coordinate functionals of `QuarkDoublet`: the weights are negated.

```
lemma QuarkDoublet.repGaugeGroupI_dual_gaugeTorusGen_coord (i : Fin 4)
    (j : Fin 2 × Fin 3 × Fin 2) :
    QuarkDoublet.repGaugeGroupI.dual (gaugeTorusGen i) (QuarkDoublet.basis.coord j)
      = ((expI : ℂ) ^ (-(GaugeWeight.coord (QuarkDoublet.valueGaugeWeight j) i))) •
        QuarkDoublet.basis.coord j
```

lemma repGaugeGroupI_conj_dual_gaugeTorusGen_coord [source]

The dual of the conjugate action of the gauge torus on the coordinate functionals of the conjugate of `QuarkDoublet`: the two negations cancel and the weights are those of the value space.

```
lemma QuarkDoublet.repGaugeGroupI_conj_dual_gaugeTorusGen_coord (i : Fin 4)
    (j : Fin 2 × Fin 3 × Fin 2) :
    QuarkDoublet.repGaugeGroupI.conj.dual (gaugeTorusGen i)
      (((Basis.conj QuarkDoublet.basis)).coord j)
      = ((expI : ℂ) ^ GaugeWeight.coord (QuarkDoublet.valueGaugeWeight j) i) •
        ((Basis.conj QuarkDoublet.basis)).coord j
```

lemma quarkDoublet_repLorentzGroup_boostAxis_two_basis [source]

The quark-doublet basis diagonalises the `z`-boost: the colour and isospin indices are inert.

```
lemma quarkDoublet_repLorentzGroup_boostAxis_two_basis (t : ℝ) (ht : t ≠ 0)
    (j : Fin 2 × Fin 3 × Fin 2) :
    QuarkDoublet.repLorentzGroup (SL2C.boostAxis 2 t ht) (QuarkDoublet.basis j)
```

```
      = ((t : ℝ) : ℂ) ^ (weylWeight j.1) • QuarkDoublet.basis j
```

2.436 PhyslibAlpha.Particles.StandardModel.Fermions.QuarkDoublet.GaugeAlgebraAction

PhyslibAlpha/Particles/StandardModel/Fermions/QuarkDoublet/GaugeAlgebraAction.lean on GitHub.

def colourWeakValLinEquiv [source]

The identification of the quark doublet with a left-handed Weyl spinor tensored with a colour–weak vector over the combined index `Fin 3 × Fin 2`: the ℂ-level analogue of `jetValLinEquiv`.

```
noncomputable def colourWeakValLinEquiv :
    QuarkDoublet ≃ₗ[ℂ] Fermion.LeftHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 3 × Fin 2)
```

def colourWeakEnd [source]

The endomorphism of the quark doublet defined by a complex matrix over the combined colour–weak index, with the Weyl factor untouched.

```
noncomputable def colourWeakEnd (A : Matrix (Fin 3 × Fin 2) (Fin 3 × Fin 2) ℂ) :
    QuarkDoublet →ₗ[ℂ] QuarkDoublet
```

lemma colourWeakEnd_apply_mk [source]

```
lemma colourWeakEnd_apply_mk (A : Matrix (Fin 3 × Fin 2) (Fin 3 × Fin 2) ℂ)
    (v : QuarkDoublet) :
    colourWeakEnd A v
      = colourWeakValLinEquiv.symm
          (Module.End.lTensorAlgHom ℂ (EuclideanSpace ℂ (Fin 3 × Fin 2))
            Fermion.LeftHandedWeyl
            (Matrix.toLpLinAlgEquiv 2 A) (colourWeakValLinEquiv v))
```

lemma colourWeakEnd_add [source]

```
lemma colourWeakEnd_add (A B : Matrix (Fin 3 × Fin 2) (Fin 3 × Fin 2) ℂ) :
    colourWeakEnd (A + B) = colourWeakEnd A + colourWeakEnd B
```

lemma colourWeakEnd_smul [source]

```
lemma colourWeakEnd_smul (z : ℂ) (A : Matrix (Fin 3 × Fin 2) (Fin 3 × Fin 2) ℂ) :
    colourWeakEnd (z • A) = z • colourWeakEnd A
```

lemma colourWeakEnd_zero [source]

```
lemma colourWeakEnd_zero : colourWeakEnd 0 = 0
```

lemma colourWeakEnd_neg [source]

```
lemma colourWeakEnd_neg (A : Matrix (Fin 3 × Fin 2) (Fin 3 × Fin 2) ℂ) :
    colourWeakEnd (-A) = -colourWeakEnd A
```

lemma colourWeakEnd_multiset_sum [source]

```
lemma colourWeakEnd_multiset_sum
    (m : Multiset (Matrix (Fin 3 × Fin 2) (Fin 3 × Fin 2) ℂ)) :
    colourWeakEnd m.sum = (m.map colourWeakEnd).sum
```

lemma colourWeakEnd_mul [source]

The colour–weak endomorphisms compose through matrix multiplication.

```
lemma colourWeakEnd_mul (A B : Matrix (Fin 3 × Fin 2) (Fin 3 × Fin 2) ℂ) :
    colourWeakEnd (A * B) = colourWeakEnd A ∘ₗ colourWeakEnd B
```

def actionMatrix [source]

The matrix of the infinitesimal `(3, 2)_{1}` action of a gauge algebra element on the combined colour–weak index: `i` times the Kronecker sum of the colour and weak parts, shifted by `i` times the hypercharge.

```
noncomputable def actionMatrix (c : GaugeAlgebra) :
    Matrix (Fin 3 × Fin 2) (Fin 3 × Fin 2) ℂ
```

def gaugeAlgebraAction [source]

***The infinitesimal action of the gauge algebra on the quark doublet**: the derivative of the `(3, 2)_{1}` action of the gauge group, real-linear in the algebra slot and complex-linear in the value slot — the form consumed by the covariant derivative `GaugeAlgebraRealization.covDerivIter` and by `LocalGaugeData.IsInfinitesimalActionOf`.*

```
noncomputable def gaugeAlgebraAction :
    GaugeAlgebra →ₗ[ℝ] QuarkDoublet →ₗ[ℂ] QuarkDoublet where
```

def jetActionMatrix [source]

The jet-valued matrix of the infinitesimal `(3, 2)_{1}` action of a jet of gauge algebra elements: the jet analogue of `actionMatrix`.

```
noncomputable def jetActionMatrix (a : JetGaugeAlgebra) :
    Matrix (Fin 3 × Fin 2) (Fin 3 × Fin 2) SpaceTimeAlgebra
```

lemma jetActionMatrix_map_cc_foldl [source]

The base-point Taylor coefficients of the jet action matrix are the action matrices of the base-point Taylor coefficients.

```
lemma jetActionMatrix_map_cc_foldl (p : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    ((jetActionMatrix a).map fun f =>
        constantCoeff (SpaceTimeAlgebra.iteratedPDeriv p f))
      = actionMatrix (JetGaugeAlgebra.eval (JetGaugeAlgebra.iteratedDeriv p a))
```

lemma repJetGaugeGroupI_eq_jetGaugeMatrix [source]

```
lemma repJetGaugeGroupI_eq_jetGaugeMatrix (U : JetGaugeGroupI)
    (z : SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet) :
```

```
    repJetGaugeGroupI U z
      = jetValLinEquiv.symm
          (Module.End.lTensorAlgHom ℂ (EuclideanSpace SpaceTimeAlgebra (Fin 3 × Fin 2))
            Fermion.LeftHandedWeyl
            ((Matrix.toLpLinAlgEquiv 2 (jetGaugeMatrix U)).restrictScalars ℂ)
            (jetValLinEquiv z))
```

def pderivColourWeak [source]

The entrywise formal derivative on the colour–weak coordinates, as a $\mathbb{C}$-linear map.

```
private noncomputable def pderivColourWeak (μ : Fin 1 ⊕ Fin 3) :
    EuclideanSpace SpaceTimeAlgebra (Fin 3 × Fin 2) →ₗ[ℂ]
      EuclideanSpace SpaceTimeAlgebra (Fin 3 × Fin 2) where
```

def foldColourWeak [source]

The entrywise iterated formal derivative on the colour–weak coordinates.

```
private noncomputable def foldColourWeak (x : Multiset (Fin 1 ⊕ Fin 3)) :
    EuclideanSpace SpaceTimeAlgebra (Fin 3 × Fin 2) →ₗ[ℂ]
      EuclideanSpace SpaceTimeAlgebra (Fin 3 × Fin 2) where
```

def ccColourWeak [source]

The entrywise base-point evaluation on the colour–weak coordinates.

```
private noncomputable def ccColourWeak :
    EuclideanSpace SpaceTimeAlgebra (Fin 3 × Fin 2) →ₗ[ℂ] EuclideanSpace ℂ (Fin 3 × Fin 2)
    where
```

lemma pderivColourWeak_comp_foldColourWeak [source]

```
private lemma pderivColourWeak_comp_foldColourWeak (μ : Fin 1 ⊕ Fin 3)
    (x : Multiset (Fin 1 ⊕ Fin 3)) :
    pderivColourWeak μ ∘ₗ foldColourWeak x = foldColourWeak (μ ::ₘ x)
```

lemma jetValLinEquiv_jetDeriv [source]

The identification of quark-doublet jets intertwines the formal derivative with the entrywise derivative on the colour–weak coordinates.

```
private lemma jetValLinEquiv_jetDeriv (μ : Fin 1 ⊕ Fin 3)
    (z : SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet) :
    jetValLinEquiv (jetDeriv μ z)
      = (TensorProduct.map LinearMap.id (pderivColourWeak μ)) (jetValLinEquiv z)
```

lemma jetValLinEquiv_jetIteratedDeriv [source]

The identification of quark-doublet jets intertwines the iterated formal derivative with the entrywise iterated derivative on the colour–weak coordinates.

```
private lemma jetValLinEquiv_jetIteratedDeriv (x : Multiset (Fin 1 ⊕ Fin 3))
    (z : SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet) :
    jetValLinEquiv (jetIteratedDeriv x z)
```

```
      = (TensorProduct.map LinearMap.id (foldColourWeak x)) (jetValLinEquiv z)
```

lemma colourWeakValLinEquiv_jetEval [source]

The base-point evaluation of a quark-doublet jet through the colour–weak coordinates.

```
private lemma colourWeakValLinEquiv_jetEval (z : SpaceTimeAlgebra ⊗[ℂ] QuarkDoublet) :
    colourWeakValLinEquiv (jetEval z)
      = (TensorProduct.map LinearMap.id ccColourWeak) (jetValLinEquiv z)
```

lemma jetGaugeMatrix_map_pderiv [source]

***The derivative identity** for the colour–weak matrix of the jet gauge action: the formal derivative of the colour–weak matrix is minus the jet action matrix of the Maurer–Cartan form times the colour–weak matrix.*

```
lemma jetGaugeMatrix_map_pderiv (U : JetGaugeGroupI) (μ : Fin 1 ⊕ Fin 3) :
    (jetGaugeMatrix U).map (fun f => pderiv μ f)
      = -(jetActionMatrix (maurerCartanForm U μ) * jetGaugeMatrix U)
```

lemma jetGaugeMatrix_mul_jetActionMatrix [source]

***The equivariance identity** for the colour–weak matrix of the jet gauge action: the colour–weak matrix intertwines the constant jet action matrix with its adjoint transform.*

```
lemma jetGaugeMatrix_mul_jetActionMatrix (U : JetGaugeGroupI) (c : GaugeAlgebra) :
    jetGaugeMatrix U * jetActionMatrix (JetGaugeAlgebra.ofConstant c)
      = jetActionMatrix (JetGaugeAlgebra.adjointMap U (JetGaugeAlgebra.ofConstant c))
        * jetGaugeMatrix U
```

lemma repCoeff_eq [source]

***The base-point Taylor coefficients of the jet gauge action** on the quark doublet are the colour–weak endomorphisms of the base-point Taylor coefficients of the colour–weak matrix.*

```
lemma repCoeff_eq (U : JetGaugeGroupI) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U x
      = colourWeakEnd ((jetGaugeMatrix U).map fun f =>
          constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x f))
```

lemma colourWeakEnd_one [source]

The colour–weak endomorphism of the identity matrix is the identity.

```
lemma colourWeakEnd_one : colourWeakEnd 1 = LinearMap.id
```

lemma repCoeff_zero_of_eval_eq_one [source]

At the base point, a gauge jet with trivial value acts trivially: the zeroth Taylor coefficient of the jet gauge action is the identity.

```
lemma repCoeff_zero_of_eval_eq_one {U : JetGaugeGroupI} (hU : U.eval = 1) :
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U 0 = LinearMap.id
```

lemma isInfinitesimalActionOf [source]

***The `(3, 2)_{1}` action of the gauge algebra is the infinitesimal action underlying the jet gauge action on the quark doublet**: its base-point Taylor coefficients obey the Maurer–Cartan Leibniz law and intertwine the action with the adjoint transports.*

```
lemma isInfinitesimalActionOf :
    localGaugeData.IsInfinitesimalActionOf gaugeAlgebraAction repJetGaugeGroupI
```

lemma gaugeAlgebraAction_comm_repLorentzGroup [source]

The infinitesimal gauge action on the quark doublet acts on the combined colour–weak factor, the Lorentz action on the Weyl factor, so the two commute.

```
lemma gaugeAlgebraAction_comm_repLorentzGroup (c : GaugeAlgebra) (Λ : SL(2,ℂ))
    (v : QuarkDoublet) :
    QuarkDoublet.gaugeAlgebraAction c (QuarkDoublet.repLorentzGroup Λ v) =
      QuarkDoublet.repLorentzGroup Λ (QuarkDoublet.gaugeAlgebraAction c v)
```

2.437 PhyslibAlpha.Particles.StandardModel.Fermions.UpSinglet.Basic

PhyslibAlpha/Particles/StandardModel/Fermions/UpSinglet/Basic.lean on GitHub.

def basis [source]

A basis on the up singlets.

```
noncomputable def basis : Module.Basis (Fin 2 × Fin 3) ℂ UpSinglet
```

instance Module.Finite ℂ UpSinglet [source]

```
instance : Module.Finite ℂ UpSinglet
```

instance Module.Free ℂ UpSinglet [source]

```
instance : Module.Free ℂ UpSinglet
```

def jetValLinEquiv [source]

Absorbs the jet ring into the colour index: a jet of an up-type singlet is the same thing as a right-handed Weyl spinor tensored with a `SpaceTimeAlgebra`-valued colour vector,

`SpaceTimeAlgebra ⊗[ℂ] UpSinglet ≃ RightHandedWeyl ⊗[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 3)`.

```
noncomputable def jetValLinEquiv :
    SpaceTimeAlgebra ⊗[ℂ] UpSinglet ≃ₗ[ℂ]
      Fermion.RightHandedWeyl ⊗[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 3)
```

def repJetGaugeGroupI [source]

The `(3, 1)_{4}` action of the jet gauge group on the jet space of the up-type singlet. Through `jetValLinEquiv` the colour matrix of the gauge jet, carrying the `4` hypercharge phase `u ^ 4`, acts `SpaceTimeAlgebra`-linearly on the colour factor by matrix-vector multiplication, while the Weyl factor is untouched.

```
noncomputable def repJetGaugeGroupI :
    Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] UpSinglet) where
```

lemma jetValLinEquiv_smul [source]

The identification of the jets of the up-type singlet intertwines multiplication by a scalar jet with the `SpaceTimeAlgebra`-scalar action on the colour coordinates.

```
lemma jetValLinEquiv_smul (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] UpSinglet) :
    jetValLinEquiv (χ • z)
      = Module.End.lTensorAlgHom ℂ (EuclideanSpace SpaceTimeAlgebra (Fin 3))
          Fermion.RightHandedWeyl
          ((LinearMap.lsmul SpaceTimeAlgebra
              (EuclideanSpace SpaceTimeAlgebra (Fin 3)) χ).restrictScalars ℂ)
          (jetValLinEquiv z)
```

lemma repJetGaugeGroupI_smul [source]

***The jet gauge action on the jets of the up-type singlet is fibrewise**: it commutes with multiplication by scalar jets.*

```
lemma repJetGaugeGroupI_smul (U : JetGaugeGroupI) (χ : SpaceTimeAlgebra)
    (z : SpaceTimeAlgebra ⊗[ℂ] UpSinglet) :
    repJetGaugeGroupI U (χ • z) = χ • repJetGaugeGroupI U z
```

lemma repJetGaugeGroupI_ofConstant [source]

On jets of constant gauge transformations the jet action reduces to the global gauge action on the fibre: the `(3, 1)_{4}` action on the up-singlet factor, and the trivial action on the jet ring.

```
lemma repJetGaugeGroupI_ofConstant (g : GaugeGroupI) :
    repJetGaugeGroupI (JetGaugeGroupI.ofConstant g) =
      TensorProduct.map LinearMap.id (repGaugeGroupI g)
```

lemma basis_eq_mk [source]

The up-singlet basis vector as an explicit spinor–colour tensor.

```
lemma basis_eq_mk (k : Fin 2) (c : Fin 3) : basis (k, c) =
    ⟨Fermion.RightHandedWeyl.basis k ⊗ₜ[ℂ] EuclideanSpace.basisFun (Fin 3) ℂ c⟩
```

lemma repLorentzGroup_apply_basis [source]

The Lorentz action on the up-singlet basis: the colour index is inert and the spinor index transforms by the entrywise conjugate matrix.

```
lemma repLorentzGroup_apply_basis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3) :
    repLorentzGroup Λ (basis j) = ∑ β, star (Λ.1 β j.1) • basis (β, j.2)
```

lemma repLorentzGroup_dual_dualBasis [source]

The up-singlet coordinate functionals transform contragrediently, by the entrywise conjugate of the inverse matrix.

```
lemma repLorentzGroup_dual_dualBasis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3) :
    repLorentzGroup.dual Λ (basis.dualBasis j) =
      ∑ β, star ((Λ⁻¹).1 j.1 β) • basis.dualBasis (β, j.2)
```

lemma repLorentzGroup_conj_apply_basis [source]

The Lorentz action on the conjugate up-singlet basis: the coefficients are the conjugates of those of the up-singlet action, that is, the matrix itself.

```
lemma repLorentzGroup_conj_apply_basis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3) :
    repLorentzGroup.conj Λ ((Basis.conj basis) j)
      = ∑ β, Λ.1 β j.1 • (Basis.conj basis) (β, j.2)
```

lemma repLorentzGroup_conj_dual_dualBasis [source]

The conjugate up-singlet coordinate functionals transform by the inverse matrix.

```
lemma repLorentzGroup_conj_dual_dualBasis (Λ : SL(2,ℂ)) (j : Fin 2 × Fin 3) :
    repLorentzGroup.conj.dual Λ ((Basis.conj basis).dualBasis j) =
      ∑ β, (Λ⁻¹).1 j.1 β • (Basis.conj basis).dualBasis (β, j.2)
```

lemma repLorentzGroup_neg_one [source]

***The centre of `SL(2,ℂ)` acts on the up-singlet space by `-1`**: the value space carries a single Weyl-spinor index, and `-1` is not the identity on a half-integer spin.*

```
lemma repLorentzGroup_neg_one : repLorentzGroup (-1) = -LinearMap.id
```

lemma repLorentzGroup_conj_neg_one [source]

The centre acts on the conjugate up-singlet space by `-1` as well: conjugation does not move a real sign.

```
lemma repLorentzGroup_conj_neg_one : repLorentzGroup.conj (-1) = -LinearMap.id
```

lemma repGaugeGroupI_apply_basis [source]

The gauge action on the up-singlet basis: the spinor index is inert and the colour index transforms by the `SU(3)` matrix, scaled by the hypercharge factor.

```
lemma repGaugeGroupI_apply_basis (g : GaugeGroupI) (j : Fin 2 × Fin 3) :
    repGaugeGroupI g (basis j) =
      ∑ c, (g.toU1.1 ^ 4 * g.toSU3.1 c j.2) • basis (j.1, c)
```

lemma repGaugeGroupI_dual_dualBasis [source]

The up-singlet coordinate functionals carry the contragredient gauge action: the hypercharge and `SU(3)` factors of the inverse group element, transposed.

```
lemma repGaugeGroupI_dual_dualBasis (g : GaugeGroupI) (j : Fin 2 × Fin 3) :
    repGaugeGroupI.dual g (basis.dualBasis j) =
      ∑ c, ((g⁻¹).toU1.1 ^ 4 * (g⁻¹).toSU3.1 j.2 c) • basis.dualBasis (j.1, c)
```

lemma repGaugeGroupI_conj_apply_basis [source]

The gauge action on the conjugate up-singlet basis: the coefficients of the up-singlet action, conjugated.

```
lemma repGaugeGroupI_conj_apply_basis (g : GaugeGroupI) (j : Fin 2 × Fin 3) :
    repGaugeGroupI.conj g ((Basis.conj basis) j) =
      ∑ c, star (g.toU1.1 ^ 4 * g.toSU3.1 c j.2) • (Basis.conj basis) (j.1, c)
```

lemma repGaugeGroupI_conj_dual_dualBasis [source]

The conjugate up-singlet coordinate functionals carry the conjugate of the contragredient gauge action.

```
lemma repGaugeGroupI_conj_dual_dualBasis (g : GaugeGroupI) (j : Fin 2 × Fin 3) :
    repGaugeGroupI.conj.dual g ((Basis.conj basis).dualBasis j) =
      ∑ c, star ((g⁻¹).toU1.1 ^ 4 * (g⁻¹).toSU3.1 j.2 c) •
        (Basis.conj basis).dualBasis (j.1, c)
```

def valueGaugeWeight [source]

The gauge weight of the up-singlet basis: the colour weights and hypercharge `4`.

```
def UpSinglet.valueGaugeWeight (j : Fin 2 × Fin 3) : GaugeWeight
```

lemma repGaugeGroupI_gaugeTorusGen_basis [source]

The gauge torus acts diagonally on the basis of `UpSinglet`, with the weights `UpSinglet.valueGaugeWeight`.

```
lemma UpSinglet.repGaugeGroupI_gaugeTorusGen_basis (i : Fin 4) (j : Fin 2 × Fin 3) :
    UpSinglet.repGaugeGroupI (gaugeTorusGen i) (UpSinglet.basis j)
      = ((expI : ℂ) ^ GaugeWeight.coord (UpSinglet.valueGaugeWeight j) i) •
        UpSinglet.basis j
```

lemma repGaugeGroupI_dual_gaugeTorusGen_coord [source]

The dual action of the gauge torus on the coordinate functionals of `UpSinglet`: the weights are negated.

```
lemma UpSinglet.repGaugeGroupI_dual_gaugeTorusGen_coord (i : Fin 4) (j : Fin 2 × Fin 3) :
    UpSinglet.repGaugeGroupI.dual (gaugeTorusGen i) (UpSinglet.basis.coord j)
      = ((expI : ℂ) ^ (-(GaugeWeight.coord (UpSinglet.valueGaugeWeight j) i))) •
        UpSinglet.basis.coord j
```

lemma repGaugeGroupI_conj_dual_gaugeTorusGen_coord [source]

The dual of the conjugate action of the gauge torus on the coordinate functionals of the conjugate of `UpSinglet`: the two negations cancel and the weights are those of the value space.

```
lemma UpSinglet.repGaugeGroupI_conj_dual_gaugeTorusGen_coord (i : Fin 4) (j : Fin 2 × Fin 3) :
    UpSinglet.repGaugeGroupI.conj.dual (gaugeTorusGen i) (((Basis.conj UpSinglet.basis)).coord
    j)
      = ((expI : ℂ) ^ GaugeWeight.coord (UpSinglet.valueGaugeWeight j) i) •
        ((Basis.conj UpSinglet.basis)).coord j
```

lemma upSinglet_repLorentzGroup_boostAxis_two_basis [source]

The up-singlet basis diagonalises the z-boost.

```
lemma upSinglet_repLorentzGroup_boostAxis_two_basis (t : ℝ) (ht : t ≠ 0)
    (j : Fin 2 × Fin 3) :
    UpSinglet.repLorentzGroup (SL2C.boostAxis 2 t ht) (UpSinglet.basis j)
      = ((t : ℝ) : ℂ) ^ (weylWeight j.1) • UpSinglet.basis j
```

2.438 PhyslibAlpha.Particles.StandardModel.Fermions.UpSinglet.GaugeAlgebraAction

PhyslibAlpha/Particles/StandardModel/Fermions/UpSinglet/GaugeAlgebraAction.lean on GitHub.

def colourEnd [source]

The endomorphism of the up singlet defined by a `3 × 3` complex matrix acting on the colour index, with the Weyl factor untouched.

```
noncomputable def colourEnd (A : Matrix (Fin 3) (Fin 3) ℂ) :
    UpSinglet →ₗ[ℂ] UpSinglet
```

lemma colourEnd_apply_mk [source]

```
lemma colourEnd_apply_mk (A : Matrix (Fin 3) (Fin 3) ℂ) (v : UpSinglet) :
    colourEnd A v
      = valLinEquiv.symm
          (Module.End.lTensorAlgHom ℂ (EuclideanSpace ℂ (Fin 3)) Fermion.RightHandedWeyl
            (Matrix.toLpLinAlgEquiv 2 A) (valLinEquiv v))
```

lemma colourEnd_add [source]

```
lemma colourEnd_add (A B : Matrix (Fin 3) (Fin 3) ℂ) :
    colourEnd (A + B) = colourEnd A + colourEnd B
```

lemma colourEnd_smul [source]

```
lemma colourEnd_smul (z : ℂ) (A : Matrix (Fin 3) (Fin 3) ℂ) :
    colourEnd (z • A) = z • colourEnd A
```

lemma colourEnd_zero [source]

```
lemma colourEnd_zero : colourEnd 0 = 0
```

lemma colourEnd_neg [source]

```
lemma colourEnd_neg (A : Matrix (Fin 3) (Fin 3) ℂ) : colourEnd (-A) = -colourEnd A
```

lemma colourEnd_multiset_sum [source]

```
lemma colourEnd_multiset_sum (m : Multiset (Matrix (Fin 3) (Fin 3) ℂ)) :
    colourEnd m.sum = (m.map colourEnd).sum
```

lemma colourEnd_mul [source]

The colour endomorphisms compose through matrix multiplication.

```
lemma colourEnd_mul (A B : Matrix (Fin 3) (Fin 3) ℂ) :
    colourEnd (A * B) = colourEnd A ∘ₗ colourEnd B
```

def actionMatrix [source]

The matrix of the infinitesimal `(3, 1)_{4}` action of a gauge algebra element on the colour index: `i` times the colour part, shifted by `i` times `4` the hypercharge.

```
noncomputable def actionMatrix (c : GaugeAlgebra) : Matrix (Fin 3) (Fin 3) ℂ
```

def gaugeAlgebraAction [source]

***The infinitesimal action of the gauge algebra on the up-type singlet**: the derivative of the `(3, 1)_{4}` action of the gauge group, real-linear in the algebra slot and complex-linear in the value slot — the form consumed by the covariant derivative `GaugeAlgebraRealization.covDerivIter` and by `LocalGaugeData.IsInfinitesimalActionOf`.*

```
noncomputable def gaugeAlgebraAction :
    GaugeAlgebra →ₗ[ℝ] UpSinglet →ₗ[ℂ] UpSinglet where
```

def jetActionMatrix [source]

The jet-valued matrix of the infinitesimal `(3, 1)_{4}` action of a jet of gauge algebra elements: the jet analogue of `actionMatrix`.

```
noncomputable def jetActionMatrix (a : JetGaugeAlgebra) : Matrix (Fin 3) (Fin 3)
    SpaceTimeAlgebra
```

lemma jetActionMatrix_map_cc_foldl [source]

The base-point Taylor coefficients of the jet action matrix are the action matrices of the base-point Taylor coefficients.

```
lemma jetActionMatrix_map_cc_foldl (p : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    ((jetActionMatrix a).map fun f =>
        constantCoeff (SpaceTimeAlgebra.iteratedPDeriv p f))
      = actionMatrix (JetGaugeAlgebra.eval (JetGaugeAlgebra.iteratedDeriv p a))
```

def upMatrix [source]

The `SpaceTimeAlgebra`-valued colour matrix of the jet gauge action on the up singlet: the colour matrix of the gauge jet carrying the `4` hypercharge phase.

```
noncomputable def upMatrix (U : JetGaugeGroupI) : Matrix (Fin 3) (Fin 3) SpaceTimeAlgebra
```

lemma repJetGaugeGroupI_eq_upMatrix [source]

```
lemma repJetGaugeGroupI_eq_upMatrix (U : JetGaugeGroupI)
    (z : SpaceTimeAlgebra ⊗[ℂ] UpSinglet) :
    repJetGaugeGroupI U z
      = jetValLinEquiv.symm
          (Module.End.lTensorAlgHom ℂ (EuclideanSpace SpaceTimeAlgebra (Fin 3))
            Fermion.RightHandedWeyl
            ((Matrix.toLpLinAlgEquiv 2 (upMatrix U)).restrictScalars ℂ)
            (jetValLinEquiv z))
```

def pderivColour [source]

The entrywise formal derivative on the colour coordinates, as a $\mathbb{C}$-linear map.

```
private noncomputable def pderivColour (μ : Fin 1 ⊕ Fin 3) :
    EuclideanSpace SpaceTimeAlgebra (Fin 3) →ₗ[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 3) where
```

def foldColour [source]

The entrywise iterated formal derivative on the colour coordinates.

```
private noncomputable def foldColour (x : Multiset (Fin 1 ⊕ Fin 3)) :
    EuclideanSpace SpaceTimeAlgebra (Fin 3) →ₗ[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 3) where
```

def ccColour [source]

The entrywise base-point evaluation on the colour coordinates.

```
private noncomputable def ccColour :
    EuclideanSpace SpaceTimeAlgebra (Fin 3) →ₗ[ℂ] EuclideanSpace ℂ (Fin 3) where
```

lemma pderivColour_comp_foldColour [source]

```
private lemma pderivColour_comp_foldColour (μ : Fin 1 ⊕ Fin 3)
    (x : Multiset (Fin 1 ⊕ Fin 3)) :
    pderivColour μ ∘ₗ foldColour x = foldColour (μ ::ₘ x)
```

lemma jetValLinEquiv_jetDeriv [source]

The identification of up-singlet jets intertwines the formal derivative with the entrywise derivative on the colour coordinates.

```
private lemma jetValLinEquiv_jetDeriv (μ : Fin 1 ⊕ Fin 3)
    (z : SpaceTimeAlgebra ⊗[ℂ] UpSinglet) :
    jetValLinEquiv (jetDeriv μ z)
      = (TensorProduct.map LinearMap.id (pderivColour μ)) (jetValLinEquiv z)
```

lemma jetValLinEquiv_jetIteratedDeriv [source]

The identification of up-singlet jets intertwines the iterated formal derivative with the entrywise iterated derivative on the colour coordinates.

```
private lemma jetValLinEquiv_jetIteratedDeriv (x : Multiset (Fin 1 ⊕ Fin 3))
    (z : SpaceTimeAlgebra ⊗[ℂ] UpSinglet) :
    jetValLinEquiv (jetIteratedDeriv x z)
      = (TensorProduct.map LinearMap.id (foldColour x)) (jetValLinEquiv z)
```

lemma valLinEquiv_jetEval [source]

The base-point evaluation of an up-singlet jet through the colour coordinates.

```
private lemma valLinEquiv_jetEval (z : SpaceTimeAlgebra ⊗[ℂ] UpSinglet) :
    valLinEquiv (jetEval z)
      = (TensorProduct.map LinearMap.id ccColour) (jetValLinEquiv z)
```

lemma upMatrix_map_pderiv [source]

***The derivative identity** for the colour matrix of the jet gauge action: the formal derivative of the colour matrix is minus the jet action matrix of the Maurer–Cartan form times the colour matrix.*

```
lemma upMatrix_map_pderiv (U : JetGaugeGroupI) (μ : Fin 1 ⊕ Fin 3) :
    (upMatrix U).map (fun f => pderiv μ f)
      = -(jetActionMatrix (maurerCartanForm U μ) * upMatrix U)
```

lemma upMatrix_mul_jetActionMatrix [source]

***The equivariance identity** for the colour matrix of the jet gauge action: the colour matrix intertwines the constant jet action matrix with its adjoint transform.*

```
lemma upMatrix_mul_jetActionMatrix (U : JetGaugeGroupI) (c : GaugeAlgebra) :
    upMatrix U * jetActionMatrix (JetGaugeAlgebra.ofConstant c)
      = jetActionMatrix (JetGaugeAlgebra.adjointMap U (JetGaugeAlgebra.ofConstant c))
        * upMatrix U
```

lemma repCoeff_eq [source]

***The base-point Taylor coefficients of the jet gauge action** on the up-type singlet are the colour endomorphisms of the base-point Taylor coefficients of the colour matrix.*

```
lemma repCoeff_eq (U : JetGaugeGroupI) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U x
      = colourEnd ((upMatrix U).map fun f =>
          constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x f))
```

lemma colourEnd_one [source]

The colour endomorphism of the identity matrix is the identity.

```
lemma colourEnd_one : colourEnd 1 = LinearMap.id
```

lemma repCoeff_zero_of_eval_eq_one [source]

At the base point, a gauge jet with trivial value acts trivially: the zeroth Taylor coefficient of the jet gauge action is the identity.

```
lemma repCoeff_zero_of_eval_eq_one {U : JetGaugeGroupI} (hU : U.eval = 1) :
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U 0 = LinearMap.id
```

lemma isInfinitesimalActionOf [source]

***The `(3, 1)_{4}` action of the gauge algebra is the infinitesimal action underlying the jet gauge action on the up-type singlet**: its base-point Taylor coefficients obey the Maurer–Cartan Leibniz law and intertwine the action with the adjoint transports.*

```
lemma isInfinitesimalActionOf :
    localGaugeData.IsInfinitesimalActionOf gaugeAlgebraAction repJetGaugeGroupI
```

lemma gaugeAlgebraAction_comm_repLorentzGroup [source]

The infinitesimal gauge action on the up-type singlet acts on the colour factor, the Lorentz action on the Weyl factor, so the two commute.

```
lemma gaugeAlgebraAction_comm_repLorentzGroup (c : GaugeAlgebra) (Λ : SL(2,ℂ))
    (v : UpSinglet) :
    UpSinglet.gaugeAlgebraAction c (UpSinglet.repLorentzGroup Λ v) =
      UpSinglet.repLorentzGroup Λ (UpSinglet.gaugeAlgebraAction c v)
```

2.439 PhyslibAlpha.Particles.StandardModel.FieldData

PhyslibAlpha/Particles/StandardModel/FieldData.lean on GitHub.

inductive FermionType [source]

The fifteen fermion species of the Standard Model: each of the five fermion types in each of the three generations, the generation `i : Fin 3` carried by the constructor. Two generations of one type share a representation package but are distinct species.

```
inductive FermionType where
  | leptonDoublet (i : Fin 3) : FermionType
  | leptonSinglet (i : Fin 3) : FermionType
  | quarkDoublet (i : Fin 3) : FermionType
  | upSinglet (i : Fin 3) : FermionType
  | downSinglet (i : Fin 3) : FermionType
```

def matterField [source]

The matter field of a fermion species, one of the five existing adapters. It is the same in every generation.

```
noncomputable def matterField : FermionType → MatterField localGaugeData
  | .leptonDoublet _ => LeptonDoubletV.matterField
  | .leptonSinglet _ => LeptonSinglet.matterField
  | .quarkDoublet _ => QuarkDoublet.matterField
  | .upSinglet _ => UpSinglet.matterField
  | .downSinglet _ => DownSinglet.matterField
```

lemma matterField_leptonDoublet [source]

```
@[simp]
lemma matterField_leptonDoublet (i : Fin 3) :
    matterField (.leptonDoublet i) = LeptonDoubletV.matterField
```

lemma matterField_leptonSinglet [source]

```
@[simp]
lemma matterField_leptonSinglet (i : Fin 3) :
    matterField (.leptonSinglet i) = LeptonSinglet.matterField
```

lemma matterField_quarkDoublet [source]

```
@[simp]
lemma matterField_quarkDoublet (i : Fin 3) :
    matterField (.quarkDoublet i) = QuarkDoublet.matterField
```

lemma matterField_upSinglet [source]

```
@[simp]
lemma matterField_upSinglet (i : Fin 3) :
    matterField (.upSinglet i) = UpSinglet.matterField
```

lemma matterField_downSinglet [source]

```
@[simp]
lemma matterField_downSinglet (i : Fin 3) :
    matterField (.downSinglet i) = DownSinglet.matterField
```

lemma matterField_massWeight [source]

Every Standard Model fermion carries mass weight three, in every generation.

```
@[simp]
lemma matterField_massWeight (t : FermionType) : (matterField t).massWeight = 3
```

lemma matterField_statistic [source]

Every Standard Model fermion is declared fermionic, in every generation.

```
@[simp]
lemma matterField_statistic (t : FermionType) : (matterField t).statistic = .fermionic
```

lemma matterField_pureJetsActTrivially [source]

Pure gauge jets act trivially on every Standard Model fermion at the base point.

```
lemma matterField_pureJetsActTrivially (t : FermionType) :
    (matterField t).PureJetsActTrivially
```

lemma matterField_gaugeLorentzCompatible [source]

The gauge and Lorentz actions on every Standard Model fermion commute.

```
lemma matterField_gaugeLorentzCompatible (t : FermionType) :
    (matterField t).GaugeLorentzCompatible
```

def fieldData [source]

The field data of the Standard Model: three generations of each of the five fermion types and one Higgs multiplet, over the gauge context `StandardModel.localGaugeData`.

```
noncomputable def fieldData : GaugeFieldData localGaugeData where
```

lemma fieldData_fermionSpecies [source]

```
@[simp]
lemma fieldData_fermionSpecies : fieldData.FermionSpecies = FermionType
```

lemma fieldData_fermion [source]

A fermionic species is the multiplet of its type, whichever generation it is in.

```
@[simp]
lemma fieldData_fermion (t : FermionType) : fieldData.fermion t = t.matterField
```

lemma fieldData_bosonSpecies [source]

```
@[simp]
lemma fieldData_bosonSpecies : fieldData.BosonSpecies = Unit
```

lemma fieldData_boson [source]

The one bosonic species is the Higgs multiplet.

```
@[simp]
lemma fieldData_boson (j : fieldData.BosonSpecies) :
    fieldData.boson j = HiggsVec.matterField
```

lemma card_fieldData_fermionSpecies [source]

Fifteen fermionic multiplets: each of the five types in each of the three generations, exactly once.

```
lemma card_fieldData_fermionSpecies : Nat.card fieldData.FermionSpecies = 15
```

lemma card_fieldData_bosonSpecies [source]

Exactly one bosonic multiplet, the Higgs.

```
lemma card_fieldData_bosonSpecies : Nat.card fieldData.BosonSpecies = 1
```

lemma fieldData_fermion_pureJetsActTrivially [source]

Pure gauge jets act trivially on every fermionic species of the Standard Model.

```
lemma fieldData_fermion_pureJetsActTrivially (t : FermionType) :
    (fieldData.fermion t).PureJetsActTrivially
```

lemma fieldData_boson_pureJetsActTrivially [source]

Pure gauge jets act trivially on the bosonic species of the Standard Model.

```
lemma fieldData_boson_pureJetsActTrivially (j : fieldData.BosonSpecies) :
    (fieldData.boson j).PureJetsActTrivially
```

lemma fieldData_fermion_gaugeLorentzCompatible [source]

The gauge and Lorentz actions commute on every fermionic species of the Standard Model.

```
lemma fieldData_fermion_gaugeLorentzCompatible (t : FermionType) :
    (fieldData.fermion t).GaugeLorentzCompatible
```

lemma fieldData_boson_gaugeLorentzCompatible [source]

The gauge and Lorentz actions commute on the bosonic species of the Standard Model.

```
lemma fieldData_boson_gaugeLorentzCompatible (j : fieldData.BosonSpecies) :
    (fieldData.boson j).GaugeLorentzCompatible
```

lemma fieldData_pureJetsActTrivially [source]

Pure gauge jets act trivially on every species of the Standard Model.

```
lemma fieldData_pureJetsActTrivially : fieldData.PureJetsActTrivially
```

lemma fieldData_gaugeLorentzCompatible [source]

The gauge and Lorentz actions commute on every species of the Standard Model.

```
lemma fieldData_gaugeLorentzCompatible : fieldData.GaugeLorentzCompatible
```

lemma fieldData_fermion_massWeight [source]

Every fermionic species of the datum carries mass weight three.

```
@[simp]
lemma fieldData_fermion_massWeight (j : fieldData.FermionSpecies) :
    (fieldData.fermion j).massWeight = 3
```

lemma fieldData_boson_massWeight [source]

The Higgs multiplet carries mass weight two.

```
@[simp]
lemma fieldData_boson_massWeight (j : fieldData.BosonSpecies) :
    (fieldData.boson j).massWeight = 2
```

lemma fieldData_massWeightScaleFermion_inclFermion_basis_tmul [source]

A fermionic component function `∂_s ψ_α` scales by `c ^ (3 + 2 |s|)`, whichever species it belongs to.

```
lemma fieldData_massWeightScaleFermion_inclFermion_basis_tmul (c : ℂ)
    (j : fieldData.FermionSpecies) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (fieldData.FermionValue j)) :
    fieldData.massWeightScaleFermion c (fieldData.inclFermion j
        ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) :
          JetComponentSpace (fieldData.fermion j)))
      = c ^ (3 + 2 * Multiset.card s) • fieldData.inclFermion j
        ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) :
          JetComponentSpace (fieldData.fermion j))
```

lemma fieldData_massWeightScaleBoson_inclBoson_basis_tmul [source]

A Higgs component function `∂_s H_α` scales by `c ^ (2 + 2 |s|)`.

```
lemma fieldData_massWeightScaleBoson_inclBoson_basis_tmul (c : ℂ)
    (j : fieldData.BosonSpecies) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (fieldData.BosonValue j)) :
    fieldData.massWeightScaleBoson c (fieldData.inclBoson j
        ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) :
          JetComponentSpace (fieldData.boson j)))
      = c ^ (2 + 2 * Multiset.card s) • fieldData.inclBoson j
        ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) :
          JetComponentSpace (fieldData.boson j))
```

2.440 PhyslibAlpha.Particles.StandardModel.GaugeAlgebra.Basic

PhyslibAlpha/Particles/StandardModel/GaugeAlgebra/Basic.lean on GitHub.

abbrev GaugeAlgebra [source]

The gauge algebra of the Standard Model: the Lie algebra of `GaugeGroupI`, with one factor per gauge group factor — traceless self-adjoint `3 × 3` and `2 × 2` complex matrices and a self-adjoint (i.e. real) scalar. This is the constant-coefficient analogue of `JetGaugeAlgebra`, and the value at the base point of the jets it contains.

```
abbrev GaugeAlgebra
```

instance Module.Finite ℝ (selfAdjoint ℂ) [source]

The self-adjoint scalars form a finite-dimensional real vector space, through the identification with the corresponding submodule.

```
instance : Module.Finite ℝ (selfAdjoint ℂ)
```

instance Module.Finite ℝ GaugeAlgebra [source]

```
instance : Module.Finite ℝ GaugeAlgebra
```

def toSU3Matrix [source]

The `su(3)`-factor component of an element of the gauge algebra.

```
def toSU3Matrix (a : GaugeAlgebra) : Matrix (Fin 3) (Fin 3) ℂ
```

def toSU2Matrix [source]

The `su(2)`-factor component of an element of the gauge algebra.

```
def toSU2Matrix (a : GaugeAlgebra) : Matrix (Fin 2) (Fin 2) ℂ
```

def toU1Value [source]

The `u(1)`-factor component of an element of the gauge algebra.

```
def toU1Value (a : GaugeAlgebra) : ℂ
```

lemma ext_of_matrix [source]

```
@[ext]
lemma ext_of_matrix {a b : GaugeAlgebra} (h1 : a.toSU3Matrix = b.toSU3Matrix)
    (h2 : a.toSU2Matrix = b.toSU2Matrix) (h3 : a.toU1Value = b.toU1Value) : a = b
```

def ofMatrixProd [source]

The element of the gauge algebra constructed from a triple of matrices satisfying the relevant hermiticity and tracelessness conditions.

```
def ofMatrixProd (A : Matrix (Fin 3) (Fin 3) ℂ ×
    Matrix (Fin 2) (Fin 2) ℂ × ℂ) (hA : star A.1 = A.1 ∧ A.1.trace = 0)
    (hB : star A.2.1 = A.2.1 ∧ A.2.1.trace = 0) (hC : star A.2.2 = A.2.2) : GaugeAlgebra
```

lemma ofMatrixProd_toSU3Matrix [source]

```
@[simp]
lemma ofMatrixProd_toSU3Matrix (A : Matrix (Fin 3) (Fin 3) ℂ ×
    Matrix (Fin 2) (Fin 2) ℂ × ℂ) (hA : star A.1 = A.1 ∧ A.1.trace = 0)
    (hB : star A.2.1 = A.2.1 ∧ A.2.1.trace = 0) (hC : star A.2.2 = A.2.2) :
    (ofMatrixProd A hA hB hC).toSU3Matrix = A.1
```

lemma ofMatrixProd_toSU2Matrix [source]

```
@[simp]
lemma ofMatrixProd_toSU2Matrix (A : Matrix (Fin 3) (Fin 3) ℂ ×
    Matrix (Fin 2) (Fin 2) ℂ × ℂ) (hA : star A.1 = A.1 ∧ A.1.trace = 0)
    (hB : star A.2.1 = A.2.1 ∧ A.2.1.trace = 0) (hC : star A.2.2 = A.2.2) :
    (ofMatrixProd A hA hB hC).toSU2Matrix = A.2.1
```

lemma ofMatrixProd_toU1Value [source]

```
@[simp]
lemma ofMatrixProd_toU1Value (A : Matrix (Fin 3) (Fin 3) ℂ ×
    Matrix (Fin 2) (Fin 2) ℂ × ℂ) (hA : star A.1 = A.1 ∧ A.1.trace = 0)
    (hB : star A.2.1 = A.2.1 ∧ A.2.1.trace = 0) (hC : star A.2.2 = A.2.2) :
    (ofMatrixProd A hA hB hC).toU1Value = A.2.2
```

lemma add_toSU3Matrix [source]

```
@[simp]
lemma add_toSU3Matrix (a b : GaugeAlgebra) :
    (a + b).toSU3Matrix = a.toSU3Matrix + b.toSU3Matrix
```

lemma add_toSU2Matrix [source]

```
@[simp]
lemma add_toSU2Matrix (a b : GaugeAlgebra) :
    (a + b).toSU2Matrix = a.toSU2Matrix + b.toSU2Matrix
```

lemma add_toU1Value [source]

```
@[simp]
lemma add_toU1Value (a b : GaugeAlgebra) :
    (a + b).toU1Value = a.toU1Value + b.toU1Value
```

lemma zero_toSU3Matrix [source]

```
@[simp]
lemma zero_toSU3Matrix : (0 : GaugeAlgebra).toSU3Matrix = 0
```

lemma zero_toSU2Matrix [source]

```
@[simp]
lemma zero_toSU2Matrix : (0 : GaugeAlgebra).toSU2Matrix = 0
```

lemma zero_toU1Value [source]

```
@[simp]
lemma zero_toU1Value : (0 : GaugeAlgebra).toU1Value = 0
```

lemma smul_toSU3Matrix [source]

```
@[simp]
lemma smul_toSU3Matrix (r : ℝ) (a : GaugeAlgebra) :
    (r • a).toSU3Matrix = r • a.toSU3Matrix
```

lemma smul_toSU2Matrix [source]

```
@[simp]
lemma smul_toSU2Matrix (r : ℝ) (a : GaugeAlgebra) :
    (r • a).toSU2Matrix = r • a.toSU2Matrix
```

lemma smul_toU1Value [source]

```
@[simp]
lemma smul_toU1Value (r : ℝ) (a : GaugeAlgebra) :
    (r • a).toU1Value = r • a.toU1Value
```

instance Bracket GaugeAlgebra GaugeAlgebra [source]

The bracket on the gauge algebra: `I` times the matrix commutator on the `su(3)` and `su(2)` factors, and zero on the (commutative) `u(1)` factor. The factor of `I` is what makes the bracket of two hermitian matrices hermitian again; it is also why the bracket is only ℝ-bilinear, not ℂ-bilinear.

```
noncomputable instance : Bracket GaugeAlgebra GaugeAlgebra where
```

lemma bracket_toSU3Matrix [source]

```
@[simp]
lemma bracket_toSU3Matrix (a b : GaugeAlgebra) :
    ⁅a, b⁆.toSU3Matrix =
      Complex.I • (a.toSU3Matrix * b.toSU3Matrix - b.toSU3Matrix * a.toSU3Matrix)
```

lemma bracket_toSU2Matrix [source]

```
@[simp]
lemma bracket_toSU2Matrix (a b : GaugeAlgebra) :
    ⁅a, b⁆.toSU2Matrix =
      Complex.I • (a.toSU2Matrix * b.toSU2Matrix - b.toSU2Matrix * a.toSU2Matrix)
```

lemma bracket_toU1Value [source]

```
@[simp]
lemma bracket_toU1Value (a b : GaugeAlgebra) :
    ⁅a, b⁆.toU1Value = 0
```

instance LieRing GaugeAlgebra [source]

```
noncomputable instance : LieRing GaugeAlgebra where
```

instance LieAlgebra ℝ GaugeAlgebra [source]

```
noncomputable instance : LieAlgebra ℝ GaugeAlgebra where
```

lemma conj_mem [source]

The conjugate of a hermitian traceless matrix by a unitary matrix is hermitian and traceless.

```
lemma conj_mem {n : ℕ} {U A : Matrix (Fin n) (Fin n) ℂ}
    (hU : U ∈ Matrix.unitaryGroup (Fin n) ℂ) (hA : star A = A) (htr : A.trace = 0) :
    star (U * A * star U) = U * A * star U ∧ (U * A * star U).trace = 0
```

def adjointMap [source]

The linear map by which one gauge group element acts on the gauge algebra in the adjoint action: conjugation by the corresponding unitary on the `su(3)` and `su(2)` factors, and the identity on the commutative `u(1)` factor.

```
noncomputable def adjointMap (g : GaugeGroupI) : GaugeAlgebra →ₗ[ℝ] GaugeAlgebra where
```

lemma adjointMap_toSU3Matrix [source]

```
@[simp]
lemma adjointMap_toSU3Matrix (g : GaugeGroupI) (a : GaugeAlgebra) :
    (adjointMap g a).toSU3Matrix = g.toSU3.1 * a.toSU3Matrix * star g.toSU3.1
```

lemma adjointMap_toSU2Matrix [source]

```
@[simp]
lemma adjointMap_toSU2Matrix (g : GaugeGroupI) (a : GaugeAlgebra) :
    (adjointMap g a).toSU2Matrix = g.toSU2.1 * a.toSU2Matrix * star g.toSU2.1
```

lemma adjointMap_toU1Value [source]

```
@[simp]
lemma adjointMap_toU1Value (g : GaugeGroupI) (a : GaugeAlgebra) :
    (adjointMap g a).toU1Value = a.toU1Value
```

def adjoint [source]

***The adjoint action of the global gauge group on its gauge algebra**: conjugation by the corresponding unitary on the `su(3)` and `su(2)` factors, and the trivial action on the commutative `u(1)` factor.*

```
noncomputable def adjoint : Representation ℝ GaugeGroupI GaugeAlgebra where
```

lemma adjoint_toSU3Matrix [source]

```
@[simp]
lemma adjoint_toSU3Matrix (g : GaugeGroupI) (a : GaugeAlgebra) :
    (adjoint g a).toSU3Matrix = g.toSU3.1 * a.toSU3Matrix * star g.toSU3.1
```

lemma adjoint_toSU2Matrix [source]

```
@[simp]
lemma adjoint_toSU2Matrix (g : GaugeGroupI) (a : GaugeAlgebra) :
    (adjoint g a).toSU2Matrix = g.toSU2.1 * a.toSU2Matrix * star g.toSU2.1
```

lemma adjoint_toU1Value [source]

```
@[simp]
lemma adjoint_toU1Value (g : GaugeGroupI) (a : GaugeAlgebra) :
    (adjoint g a).toU1Value = a.toU1Value
```

2.441 PhyslibAlpha.Particles.StandardModel.GaugeAlgebra.Basis

PhyslibAlpha/Particles/StandardModel/GaugeAlgebra/Basis.lean on GitHub.

def planeEmbed [source]

The embedding of `2 × 2` matrices into the `3 × 3` matrices supported on the plane of two coordinate directions: the entries of `A` land at the row and column indices `p 0` and `p 1`, every other entry vanishing.

```
def planeEmbed (p : Fin 2 → Fin 3) (A : Matrix (Fin 2) (Fin 2) ℂ) :
    Matrix (Fin 3) (Fin 3) ℂ
```

def gellMannMatrix [source]

The Gell-Mann matrices: the standard basis of the traceless hermitian `3 × 3` matrices. The first seven are the Pauli matrices `σ1`, `σ2`, `σ3` embedded along the three coordinate planes; the eighth is the normalised traceless diagonal matrix.

```
def gellMannMatrix : Fin 8 → Matrix (Fin 3) (Fin 3) ℂ
  | 0 => planeEmbed ![0, 1] σ1
  | 1 => planeEmbed ![0, 1] σ2
  | 2 => planeEmbed ![0, 1] σ3
  | 3 => planeEmbed ![0, 2] σ1
  | 4 => planeEmbed ![0, 2] σ2
  | 5 => planeEmbed ![1, 2] σ1
  | 6 => planeEmbed ![1, 2] σ2
  | 7 => (((Real.sqrt 3 : ℝ) : ℂ))⁻¹ • !![1, 0, 0; 0, 1, 0; 0, 0, -2]
```

lemma gellMannMatrix_zero [source]

```
lemma gellMannMatrix_zero : gellMannMatrix 0 = !![0, 1, 0; 1, 0, 0; 0, 0, 0]
```

lemma gellMannMatrix_one [source]

```
lemma gellMannMatrix_one :
    gellMannMatrix 1 = !![0, -Complex.I, 0; Complex.I, 0, 0; 0, 0, 0]
```

lemma gellMannMatrix_two [source]

```
lemma gellMannMatrix_two : gellMannMatrix 2 = !![1, 0, 0; 0, -1, 0; 0, 0, 0]
```

lemma gellMannMatrix_three [source]

```
lemma gellMannMatrix_three : gellMannMatrix 3 = !![0, 0, 1; 0, 0, 0; 1, 0, 0]
```

lemma gellMannMatrix_four [source]

```
lemma gellMannMatrix_four :
    gellMannMatrix 4 = !![0, 0, -Complex.I; 0, 0, 0; Complex.I, 0, 0]
```

lemma gellMannMatrix_five [source]

```
lemma gellMannMatrix_five : gellMannMatrix 5 = !![0, 0, 0; 0, 0, 1; 0, 1, 0]
```

lemma gellMannMatrix_six [source]

```
lemma gellMannMatrix_six :
    gellMannMatrix 6 = !![0, 0, 0; 0, 0, -Complex.I; 0, Complex.I, 0]
```

lemma gellMannMatrix_seven [source]

```
lemma gellMannMatrix_seven :
    gellMannMatrix 7 = (((Real.sqrt 3 : ℝ) : ℂ))⁻¹ • !![1, 0, 0; 0, 1, 0; 0, 0, -2]
```

lemma gellMannMatrix_selfAdjoint [source]

The Gell-Mann matrices are hermitian.

```
lemma gellMannMatrix_selfAdjoint (k : Fin 8) :
    star (gellMannMatrix k) = gellMannMatrix k
```

lemma gellMannMatrix_trace [source]

The Gell-Mann matrices are traceless.

```
lemma gellMannMatrix_trace (k : Fin 8) : (gellMannMatrix k).trace = 0
```

lemma sum_smul_gellMannMatrix [source]

A combination of the Gell-Mann matrices, entry by entry.

```
lemma sum_smul_gellMannMatrix (g : Fin 8 → ℝ) :
    ∑ k, g k • gellMannMatrix k =
      !![((g 2 + (Real.sqrt 3)⁻¹ * g 7 : ℝ) : ℂ),
          ((g 0 : ℝ) : ℂ) - ((g 1 : ℝ) : ℂ) * Complex.I,
          ((g 3 : ℝ) : ℂ) - ((g 4 : ℝ) : ℂ) * Complex.I;
        ((g 0 : ℝ) : ℂ) + ((g 1 : ℝ) : ℂ) * Complex.I,
          ((-g 2 + (Real.sqrt 3)⁻¹ * g 7 : ℝ) : ℂ),
          ((g 5 : ℝ) : ℂ) - ((g 6 : ℝ) : ℂ) * Complex.I;
        ((g 3 : ℝ) : ℂ) + ((g 4 : ℝ) : ℂ) * Complex.I,
          ((g 5 : ℝ) : ℂ) + ((g 6 : ℝ) : ℂ) * Complex.I,
          ((-2 * (Real.sqrt 3)⁻¹ * g 7 : ℝ) : ℂ)]
```

lemma sum_smul_pauliMatrix_inr [source]

A combination of the three Pauli matrices `σ1`, `σ2`, `σ3`, entry by entry.

```
lemma sum_smul_pauliMatrix_inr (g : Fin 3 → ℝ) :
    ∑ i, g i • pauliMatrix (Sum.inr i) =
      !![((g 2 : ℝ) : ℂ), ((g 0 : ℝ) : ℂ) - ((g 1 : ℝ) : ℂ) * Complex.I;
        ((g 0 : ℝ) : ℂ) + ((g 1 : ℝ) : ℂ) * Complex.I, ((-g 2 : ℝ) : ℂ)]
```

lemma pauliMatrix_inr_star [source]

The Pauli matrices `σ1`, `σ2`, `σ3` are hermitian, phrased through `star`.

```
lemma pauliMatrix_inr_star (i : Fin 3) :
    star (pauliMatrix (Sum.inr i)) = pauliMatrix (Sum.inr i)
```

lemma pauliMatrix_inr_trace [source]

The Pauli matrices `σ1`, `σ2`, `σ3` are traceless.

```
lemma pauliMatrix_inr_trace (i : Fin 3) : (pauliMatrix (Sum.inr i)).trace = 0
```

lemma entry_symm_of_star_eq [source]

The entries of a hermitian matrix are conjugate-symmetric.

```
lemma entry_symm_of_star_eq {n : ℕ} {M : Matrix (Fin n) (Fin n) ℂ} (hself : star M = M)
    (i j : Fin n) : M j i = (starRingEnd ℂ) (M i j)
```

lemma diag_re_of_star_eq [source]

The diagonal entries of a hermitian matrix are real.

```
lemma diag_re_of_star_eq {n : ℕ} {M : Matrix (Fin n) (Fin n) ℂ} (hself : star M = M)
    (i : Fin n) : M i i = ((M i i).re : ℂ)
```

def gellMannCoeff [source]

The coordinates of a matrix in the Gell-Mann basis, read off from its entries.

```
def gellMannCoeff (M : Matrix (Fin 3) (Fin 3) ℂ) : Fin 8 → ℝ
  | 0 => (M 0 1).re
  | 1 => -(M 0 1).im
  | 2 => ((M 0 0).re - (M 1 1).re) / 2
  | 3 => (M 0 2).re
  | 4 => -(M 0 2).im
  | 5 => (M 1 2).re
  | 6 => -(M 1 2).im
  | 7 => Real.sqrt 3 / 2 * ((M 0 0).re + (M 1 1).re)
```

def pauliCoeff [source]

The coordinates of a matrix in the Pauli basis `σ1`, `σ2`, `σ3`, read off from its entries.

```
def pauliCoeff (M : Matrix (Fin 2) (Fin 2) ℂ) : Fin 3 → ℝ
  | 0 => (M 0 1).re
  | 1 => -(M 0 1).im
  | 2 => (M 0 0).re
```

lemma eq_sum_gellMannCoeff_smul [source]

A traceless hermitian `3 × 3` matrix is the combination of the Gell-Mann matrices with its `gellMannCoeff` coordinates.

```
lemma eq_sum_gellMannCoeff_smul {M : Matrix (Fin 3) (Fin 3) ℂ}
    (hself : star M = M) (htr : M.trace = 0) :
    M = ∑ k, gellMannCoeff M k • gellMannMatrix k
```

lemma eq_sum_pauliCoeff_smul [source]

A traceless hermitian `2 × 2` matrix is the combination of the Pauli matrices `σ1`, `σ2`, `σ3` with its `pauliCoeff` coordinates.

```
lemma eq_sum_pauliCoeff_smul {M : Matrix (Fin 2) (Fin 2) ℂ}
    (hself : star M = M) (htr : M.trace = 0) :
    M = ∑ i, pauliCoeff M i • pauliMatrix (Sum.inr i)
```

lemma gellMannCoeff_eq_trace [source]

The Gell-Mann coordinates of a traceless hermitian matrix are its trace pairings with the Gell-Mann matrices.

```
lemma gellMannCoeff_eq_trace {M : Matrix (Fin 3) (Fin 3) ℂ}
    (hself : star M = M) (htr : M.trace = 0) (k : Fin 8) :
    gellMannCoeff M k = 2⁻¹ * (Matrix.trace (gellMannMatrix k * M)).re
```

lemma pauliCoeff_eq_trace [source]

The Pauli coordinates of a traceless hermitian matrix are its trace pairings with the Pauli matrices `σ1`, `σ2`, `σ3`.

```
lemma pauliCoeff_eq_trace {M : Matrix (Fin 2) (Fin 2) ℂ}
    (hself : star M = M) (htr : M.trace = 0) (i : Fin 3) :
    pauliCoeff M i = 2⁻¹ * (Matrix.trace (pauliMatrix (Sum.inr i) * M)).re
```

def gellMannSU3 [source]

The Gell-Mann matrices as elements of the `su(3)` factor of the gauge algebra.

```
def gellMannSU3 (k : Fin 8) :
    ↥(selfAdjoint.submodule ℝ (Matrix (Fin 3) (Fin 3) ℂ) ⊓
      LinearMap.ker (Matrix.traceLinearMap (Fin 3) ℝ ℂ))
```

lemma coe_gellMannSU3 [source]

```
@[simp]
lemma coe_gellMannSU3 (k : Fin 8) :
    (gellMannSU3 k : Matrix (Fin 3) (Fin 3) ℂ) = gellMannMatrix k
```

lemma gellMannSU3_linearIndependent [source]

The Gell-Mann matrices are linearly independent.

```
lemma gellMannSU3_linearIndependent : LinearIndependent ℝ gellMannSU3
```

lemma gellMannSU3_span [source]

The Gell-Mann matrices span the `su(3)` factor.

```
lemma gellMannSU3_span : ⊤ ≤ Submodule.span ℝ (Set.range gellMannSU3)
```

def su3Basis [source]

The Gell-Mann basis of the `su(3)` factor of the gauge algebra.

```
def su3Basis : Basis (Fin 8) ℝ
    ↥(selfAdjoint.submodule ℝ (Matrix (Fin 3) (Fin 3) ℂ) ⊓
      LinearMap.ker (Matrix.traceLinearMap (Fin 3) ℝ ℂ))
```

lemma su3Basis_apply [source]

```
@[simp]
lemma su3Basis_apply (k : Fin 8) : su3Basis k = gellMannSU3 k
```

def pauliSU2 [source]

The Pauli matrices `σ1`, `σ2`, `σ3` as elements of the `su(2)` factor of the gauge algebra.

```
def pauliSU2 (i : Fin 3) :
    ↥(selfAdjoint.submodule ℝ (Matrix (Fin 2) (Fin 2) ℂ) ⊓
      LinearMap.ker (Matrix.traceLinearMap (Fin 2) ℝ ℂ))
```

lemma coe_pauliSU2 [source]

```
@[simp]
lemma coe_pauliSU2 (i : Fin 3) :
    (pauliSU2 i : Matrix (Fin 2) (Fin 2) ℂ) = pauliMatrix (Sum.inr i)
```

lemma pauliSU2_linearIndependent [source]

The Pauli matrices `σ1`, `σ2`, `σ3` are linearly independent.

```
lemma pauliSU2_linearIndependent : LinearIndependent ℝ pauliSU2
```

lemma pauliSU2_span [source]

The Pauli matrices `σ1`, `σ2`, `σ3` span the `su(2)` factor.

```
lemma pauliSU2_span : ⊤ ≤ Submodule.span ℝ (Set.range pauliSU2)
```

def su2Basis [source]

The Pauli basis of the `su(2)` factor of the gauge algebra.

```
def su2Basis : Basis (Fin 3) ℝ
    ↥(selfAdjoint.submodule ℝ (Matrix (Fin 2) (Fin 2) ℂ) ⊓
      LinearMap.ker (Matrix.traceLinearMap (Fin 2) ℝ ℂ))
```

lemma su2Basis_apply [source]

```
@[simp]
lemma su2Basis_apply (i : Fin 3) : su2Basis i = pauliSU2 i
```

def u1One [source]

The unit `1` as the single basis element of the `u(1)` factor of the gauge algebra.

```
def u1One (_ : Fin 1) : selfAdjoint ℂ
```

lemma coe_u1One [source]

```
@[simp]
lemma coe_u1One (i : Fin 1) : (u1One i : ℂ) = 1
```

lemma u1One_linearIndependent [source]

The unit is linearly independent.

```
lemma u1One_linearIndependent : LinearIndependent ℝ u1One
```

lemma u1One_span [source]

The unit spans the `u(1)` factor.

```
lemma u1One_span : ⊤ ≤ Submodule.span ℝ (Set.range u1One)
```

def u1Basis [source]

The basis of the `u(1)` factor of the gauge algebra.

```
def u1Basis : Basis (Fin 1) ℝ (selfAdjoint ℂ)
```

lemma u1Basis_apply [source]

```
@[simp]
lemma u1Basis_apply (i : Fin 1) : u1Basis i = 1
```

def stdBasis [source]

The standard basis of the gauge algebra, indexed by `Fin 8 ⊕ Fin 3 ⊕ Fin 1`: the eight Gell-Mann matrices on the `su(3)` factor, the three Pauli matrices `σ1`, `σ2`, `σ3` on the `su(2)` factor, and `1` on the `u(1)` factor.

```
def stdBasis : Basis (Fin 8 ⊕ Fin 3 ⊕ Fin 1) ℝ GaugeAlgebra
```

lemma stdBasis_inl_toSU3Matrix [source]

```
@[simp]
lemma stdBasis_inl_toSU3Matrix (k : Fin 8) :
    (stdBasis (Sum.inl k)).toSU3Matrix = gellMannMatrix k
```

lemma stdBasis_inl_toSU2Matrix [source]

```
@[simp]
lemma stdBasis_inl_toSU2Matrix (k : Fin 8) :
    (stdBasis (Sum.inl k)).toSU2Matrix = 0
```

lemma stdBasis_inl_toU1Value [source]

```
@[simp]
lemma stdBasis_inl_toU1Value (k : Fin 8) :
    (stdBasis (Sum.inl k)).toU1Value = 0
```

lemma stdBasis_inr_inl_toSU3Matrix [source]

```
@[simp]
lemma stdBasis_inr_inl_toSU3Matrix (i : Fin 3) :
    (stdBasis (Sum.inr (Sum.inl i))).toSU3Matrix = 0
```

lemma stdBasis_inr_inl_toSU2Matrix [source]

```
@[simp]
lemma stdBasis_inr_inl_toSU2Matrix (i : Fin 3) :
    (stdBasis (Sum.inr (Sum.inl i))).toSU2Matrix = pauliMatrix (Sum.inr i)
```

lemma stdBasis_inr_inl_toU1Value [source]

```
@[simp]
lemma stdBasis_inr_inl_toU1Value (i : Fin 3) :
    (stdBasis (Sum.inr (Sum.inl i))).toU1Value = 0
```

lemma stdBasis_inr_inr_toSU3Matrix [source]

```
@[simp]
lemma stdBasis_inr_inr_toSU3Matrix (i : Fin 1) :
    (stdBasis (Sum.inr (Sum.inr i))).toSU3Matrix = 0
```

lemma stdBasis_inr_inr_toSU2Matrix [source]

```
@[simp]
lemma stdBasis_inr_inr_toSU2Matrix (i : Fin 1) :
    (stdBasis (Sum.inr (Sum.inr i))).toSU2Matrix = 0
```

lemma stdBasis_inr_inr_toU1Value [source]

```
@[simp]
lemma stdBasis_inr_inr_toU1Value (i : Fin 1) :
    (stdBasis (Sum.inr (Sum.inr i))).toU1Value = 1
```

lemma stdBasis_coord_inr_inr_lie [source]

The hypercharge coordinate vanishes on every bracket: the `u(1)` factor is abelian, so a bracket has no `u(1)` component.

```
lemma stdBasis_coord_inr_inr_lie (i : Fin 1) (x y : GaugeAlgebra) :
    stdBasis.coord (Sum.inr (Sum.inr i)) ⁅x, y⁆ = 0
```

lemma toSU3Matrix_sum [source]

```
lemma toSU3Matrix_sum {ι : Type*} (s : Finset ι) (f : ι → GaugeAlgebra) :
    (∑ x ∈ s, f x).toSU3Matrix = ∑ x ∈ s, (f x).toSU3Matrix
```

lemma toSU2Matrix_sum [source]

```
lemma toSU2Matrix_sum {ι : Type*} (s : Finset ι) (f : ι → GaugeAlgebra) :
    (∑ x ∈ s, f x).toSU2Matrix = ∑ x ∈ s, (f x).toSU2Matrix
```

lemma toU1Value_sum [source]

```
lemma toU1Value_sum {ι : Type*} (s : Finset ι) (f : ι → GaugeAlgebra) :
    (∑ x ∈ s, f x).toU1Value = ∑ x ∈ s, (f x).toU1Value
```

def adjointMatrix [source]

The matrix of the adjoint action of a gauge group element in the standard basis: block diagonal, with the `su(3)` and `su(2)` blocks the trace pairings `2⁻¹ * (trace (T a * g T b g⁻¹)).re` of the basis elements with the conjugated basis elements, `1` on the `u(1)` entry, and `0` between different factors.

```
noncomputable def adjointMatrix (g : GaugeGroupI) :
    Matrix (Fin 8 ⊕ Fin 3 ⊕ Fin 1) (Fin 8 ⊕ Fin 3 ⊕ Fin 1) ℝ
```

lemma adjointMatrix_inl_inl [source]

```
@[simp]
lemma adjointMatrix_inl_inl (g : GaugeGroupI) (a b : Fin 8) :
    adjointMatrix g (Sum.inl a) (Sum.inl b)
      = 2⁻¹ * (Matrix.trace (gellMannMatrix a *
          (g.toSU3.1 * gellMannMatrix b * star g.toSU3.1))).re
```

lemma adjointMatrix_inl_inr [source]

```
@[simp]
lemma adjointMatrix_inl_inr (g : GaugeGroupI) (a : Fin 8) (x : Fin 3 ⊕ Fin 1) :
    adjointMatrix g (Sum.inl a) (Sum.inr x) = 0
```

lemma adjointMatrix_inr_inl [source]

```
@[simp]
lemma adjointMatrix_inr_inl (g : GaugeGroupI) (x : Fin 3 ⊕ Fin 1) (b : Fin 8) :
    adjointMatrix g (Sum.inr x) (Sum.inl b) = 0
```

lemma adjointMatrix_inr_inl_inr_inl [source]

```
@[simp]
lemma adjointMatrix_inr_inl_inr_inl (g : GaugeGroupI) (i j : Fin 3) :
    adjointMatrix g (Sum.inr (Sum.inl i)) (Sum.inr (Sum.inl j))
      = 2⁻¹ * (Matrix.trace (pauliMatrix (Sum.inr i) *
          (g.toSU2.1 * pauliMatrix (Sum.inr j) * star g.toSU2.1))).re
```

lemma adjointMatrix_inr_inl_inr_inr [source]

```
@[simp]
lemma adjointMatrix_inr_inl_inr_inr (g : GaugeGroupI) (i : Fin 3) (u : Fin 1) :
    adjointMatrix g (Sum.inr (Sum.inl i)) (Sum.inr (Sum.inr u)) = 0
```

lemma adjointMatrix_inr_inr_inr_inl [source]

```
@[simp]
lemma adjointMatrix_inr_inr_inr_inl (g : GaugeGroupI) (u : Fin 1) (j : Fin 3) :
    adjointMatrix g (Sum.inr (Sum.inr u)) (Sum.inr (Sum.inl j)) = 0
```

lemma adjointMatrix_inr_inr_inr_inr [source]

```
@[simp]
lemma adjointMatrix_inr_inr_inr_inr (g : GaugeGroupI) (u v : Fin 1) :
    adjointMatrix g (Sum.inr (Sum.inr u)) (Sum.inr (Sum.inr v)) = 1
```

lemma adjoint_stdBasis [source]

The adjoint action of the gauge group acts on the standard basis through `adjointMatrix`.

```
lemma adjoint_stdBasis (g : GaugeGroupI) (b : Fin 8 ⊕ Fin 3 ⊕ Fin 1) :
    adjoint g (stdBasis b) = ∑ a, adjointMatrix g a b • stdBasis a
```

lemma toMatrix_adjoint [source]

The matrix of the adjoint action in the standard basis is `adjointMatrix`.

```
lemma toMatrix_adjoint (g : GaugeGroupI) :
    LinearMap.toMatrix stdBasis stdBasis (adjoint g) = adjointMatrix g
```

lemma adjointMatrix_mulVec_repr [source]

The action of `adjointMatrix` on coordinates in the standard basis corresponds to the adjoint action of the gauge group on the gauge algebra.

```
lemma adjointMatrix_mulVec_repr (g : GaugeGroupI) (a : GaugeAlgebra) :
    (adjointMatrix g).mulVec (stdBasis.repr a) = ⇑(stdBasis.repr (adjoint g a))
```

lemma adjoint_dualMap_coord [source]

The dual adjoint action on the dual standard basis: the coordinate functions transform through the rows of `adjointMatrix`.

```
lemma adjoint_dualMap_coord (g : GaugeGroupI) (a : Fin 8 ⊕ Fin 3 ⊕ Fin 1) :
    (adjoint g).dualMap (stdBasis.coord a)
      = ∑ b, adjointMatrix g a b • stdBasis.coord b
```

lemma adjointMatrix_mul [source]

The matrix of the adjoint action turns a product in the gauge group into the product of the corresponding matrices.

```
lemma adjointMatrix_mul (g h : GaugeGroupI) :
    GaugeAlgebra.adjointMatrix (g * h)
      = GaugeAlgebra.adjointMatrix g * GaugeAlgebra.adjointMatrix h
```

lemma adjointMatrix_one [source]

The matrix of the adjoint action of the identity is the identity matrix.

```
lemma adjointMatrix_one : GaugeAlgebra.adjointMatrix (1 : GaugeGroupI) = 1
```

lemma gaugeGroup_mul_star_self [source]

The star of a gauge group element is its inverse.

```
lemma gaugeGroup_mul_star_self (g : GaugeGroupI) : g * star g = 1
```

lemma adjointMatrix_star [source]

The matrix of the adjoint action of the star of a gauge group element is the transpose of the matrix of the adjoint action, since the trace pairing is symmetric under moving the conjugation from one argument to the other.

```
lemma adjointMatrix_star (g : GaugeGroupI) :
    GaugeAlgebra.adjointMatrix (star g) = (GaugeAlgebra.adjointMatrix g)ᵀ
```

lemma adjointMatrix_mul_transpose [source]

The matrix of the adjoint action is orthogonal.

```
lemma adjointMatrix_mul_transpose (g : GaugeGroupI) :
    GaugeAlgebra.adjointMatrix g * (GaugeAlgebra.adjointMatrix g)ᵀ = 1
```

lemma sum_adjointMatrix_inl_row_mul [source]

The rows of the `su(3)` block of the adjoint matrix are orthonormal. The matrix is block diagonal, so orthogonality of the whole matrix restricts to each block.

```
lemma sum_adjointMatrix_inl_row_mul (g : GaugeGroupI) (c d : Fin 8) :
    ∑ a : Fin 8, adjointMatrix g (Sum.inl c) (Sum.inl a) *
      adjointMatrix g (Sum.inl d) (Sum.inl a) = if c = d then 1 else 0
```

lemma sum_adjointMatrix_inr_inl_row_mul [source]

The rows of the `su(2)` block of the adjoint matrix are orthonormal.

```
lemma sum_adjointMatrix_inr_inl_row_mul (g : GaugeGroupI) (c d : Fin 3) :
    ∑ a : Fin 3, adjointMatrix g (Sum.inr (Sum.inl c)) (Sum.inr (Sum.inl a)) *
      adjointMatrix g (Sum.inr (Sum.inl d)) (Sum.inr (Sum.inl a))
      = if c = d then 1 else 0
```

lemma adjointMatrix_inv_apply [source]

The matrix of the adjoint action of the inverse of a gauge group element is the transpose of the matrix of the adjoint action.

```
lemma adjointMatrix_inv_apply (g : GaugeGroupI) (a b : Fin 8 ⊕ Fin 3 ⊕ Fin 1) :
    adjointMatrix g⁻¹ a b = adjointMatrix g b a
```

2.442 PhyslibAlpha.Particles.StandardModel.GaugeAlgebra.JetGaugeAlgebra

PhyslibAlpha/Particles/StandardModel/GaugeAlgebra/JetGaugeAlgebra.lean on GitHub.

abbrev JetGaugeAlgebra [source]

The jet gauge algebra: the Lie-algebra analogue of `JetGaugeGroupI`, with one factor per gauge group factor — traceless self-adjoint `3 × 3` and `2 × 2` matrices and a self-adjoint scalar, all with coefficients in the ring `SpaceTimeAlgebra` of formal power series in the spacetime coordinates. The Maurer–Cartan forms of the jet gauge group are valued here, hermiticity being `star_maurerCartanSU3` and its companions.

```
abbrev JetGaugeAlgebra
```

def toSU3Matrix [source]

The `su(3)`-factor component of an element of the jet gauge algebra.

```
def toSU3Matrix (a : JetGaugeAlgebra) : Matrix (Fin 3) (Fin 3) SpaceTimeAlgebra
```

def toSU2Matrix [source]

The `su(2)`-factor component of an element of the jet gauge algebra.

```
def toSU2Matrix (a : JetGaugeAlgebra) : Matrix (Fin 2) (Fin 2) SpaceTimeAlgebra
```

def toU1Value [source]

The `u(1)`-factor component of an element of the jet gauge algebra.

```
def toU1Value (a : JetGaugeAlgebra) :  SpaceTimeAlgebra
```

def toVal [source]

The underlying matrix value of an element of the jet gauge algebra, as a product of matrices.

```
def toVal (a : JetGaugeAlgebra) :
    Matrix (Fin 3) (Fin 3) SpaceTimeAlgebra × Matrix (Fin 2)
        (Fin 2) SpaceTimeAlgebra × SpaceTimeAlgebra
```

lemma toVal_fst [source]

```
@[simp]
lemma toVal_fst (a : JetGaugeAlgebra) : a.toVal.1 = a.toSU3Matrix
```

lemma toVal_snd_fst [source]

```
@[simp]
lemma toVal_snd_fst (a : JetGaugeAlgebra) : a.toVal.2.1 = a.toSU2Matrix
```

lemma toVal_snd_snd [source]

```
@[simp]
lemma toVal_snd_snd (a : JetGaugeAlgebra) : a.toVal.2.2 = a.toU1Value
```

lemma ext_of_matrix [source]

```
@[ext]
lemma ext_of_matrix {a b : JetGaugeAlgebra} (h1 : a.toSU3Matrix = b.toSU3Matrix)
    (h2 : a.toSU2Matrix = b.toSU2Matrix) (h3 : a.toU1Value = b.toU1Value) : a = b
```

def ofMatrixProd [source]

A jet of the gauge algebra from a triple of matrices of jets: the `SU(3)` and `SU(2)` components self-adjoint and traceless, the `U(1)` component self-adjoint.

```
def ofMatrixProd (A : Matrix (Fin 3) (Fin 3) SpaceTimeAlgebra ×
    Matrix (Fin 2) (Fin 2) SpaceTimeAlgebra × SpaceTimeAlgebra)
        (hA : star A.1 = A.1 ∧ A.1.trace = 0)
    (hB : star A.2.1 = A.2.1 ∧ A.2.1.trace = 0) (hC : star A.2.2 = A.2.2) : JetGaugeAlgebra
```

lemma ofMatrixProd_toSU3Matrix [source]

```
@[simp]
lemma ofMatrixProd_toSU3Matrix (A : Matrix (Fin 3) (Fin 3) SpaceTimeAlgebra ×
    Matrix (Fin 2) (Fin 2) SpaceTimeAlgebra × SpaceTimeAlgebra)
        (hA : star A.1 = A.1 ∧ A.1.trace = 0)
    (hB : star A.2.1 = A.2.1 ∧ A.2.1.trace = 0) (hC : star A.2.2 = A.2.2) :
    (ofMatrixProd A hA hB hC).toSU3Matrix = A.1
```

lemma ofMatrixProd_toSU2Matrix [source]

```
@[simp]
lemma ofMatrixProd_toSU2Matrix (A : Matrix (Fin 3) (Fin 3) SpaceTimeAlgebra ×
    Matrix (Fin 2) (Fin 2) SpaceTimeAlgebra × SpaceTimeAlgebra)
        (hA : star A.1 = A.1 ∧ A.1.trace = 0)
    (hB : star A.2.1 = A.2.1 ∧ A.2.1.trace = 0) (hC : star A.2.2 = A.2.2) :
    (ofMatrixProd A hA hB hC).toSU2Matrix = A.2.1
```

lemma ofMatrixProd_toU1Value [source]

```
@[simp]
lemma ofMatrixProd_toU1Value (A : Matrix (Fin 3) (Fin 3) SpaceTimeAlgebra ×
    Matrix (Fin 2) (Fin 2) SpaceTimeAlgebra × SpaceTimeAlgebra)
        (hA : star A.1 = A.1 ∧ A.1.trace = 0)
    (hB : star A.2.1 = A.2.1 ∧ A.2.1.trace = 0) (hC : star A.2.2 = A.2.2) :
    (ofMatrixProd A hA hB hC).toU1Value = A.2.2
```

lemma add_toSU3Matrix [source]

```
@[simp]
lemma add_toSU3Matrix (a b : JetGaugeAlgebra) :
    (a + b).toSU3Matrix = a.toSU3Matrix + b.toSU3Matrix
```

lemma add_toSU2Matrix [source]

```
@[simp]
lemma add_toSU2Matrix (a b : JetGaugeAlgebra) :
    (a + b).toSU2Matrix = a.toSU2Matrix + b.toSU2Matrix
```

lemma add_toU1Value [source]

```
@[simp]
lemma add_toU1Value (a b : JetGaugeAlgebra) :
    (a + b).toU1Value = a.toU1Value + b.toU1Value
```

lemma toSU3Matrix_sum [source]

```
lemma toSU3Matrix_sum {ι : Type*} (t : Finset ι) (f : ι → JetGaugeAlgebra) :
    (∑ i ∈ t, f i).toSU3Matrix = ∑ i ∈ t, (f i).toSU3Matrix
```

lemma toSU2Matrix_sum [source]

```
lemma toSU2Matrix_sum {ι : Type*} (t : Finset ι) (f : ι → JetGaugeAlgebra) :
    (∑ i ∈ t, f i).toSU2Matrix = ∑ i ∈ t, (f i).toSU2Matrix
```

lemma toU1Value_sum [source]

```
lemma toU1Value_sum {ι : Type*} (t : Finset ι) (f : ι → JetGaugeAlgebra) :
    (∑ i ∈ t, f i).toU1Value = ∑ i ∈ t, (f i).toU1Value
```

lemma zero_toSU3Matrix [source]

```
@[simp]
lemma zero_toSU3Matrix : (0 : JetGaugeAlgebra).toSU3Matrix = 0
```

lemma zero_toSU2Matrix [source]

```
@[simp]
lemma zero_toSU2Matrix : (0 : JetGaugeAlgebra).toSU2Matrix = 0
```

lemma zero_toU1Value [source]

```
@[simp]
lemma zero_toU1Value : (0 : JetGaugeAlgebra).toU1Value = 0
```

lemma smul_toSU3Matrix [source]

```
@[simp]
lemma smul_toSU3Matrix (r : ℝ) (a : JetGaugeAlgebra) :
    (r • a).toSU3Matrix = r • a.toSU3Matrix
```

lemma smul_toSU2Matrix [source]

```
@[simp]
lemma smul_toSU2Matrix (r : ℝ) (a : JetGaugeAlgebra) :
    (r • a).toSU2Matrix = r • a.toSU2Matrix
```

lemma smul_toU1Value [source]

```
@[simp]
lemma smul_toU1Value (r : ℝ) (a : JetGaugeAlgebra) :
    (r • a).toU1Value = r • a.toU1Value
```

lemma sub_toSU3Matrix [source]

```
@[simp]
lemma sub_toSU3Matrix (a b : JetGaugeAlgebra) :
    (a - b).toSU3Matrix = a.toSU3Matrix - b.toSU3Matrix
```

lemma sub_toSU2Matrix [source]

```
@[simp]
lemma sub_toSU2Matrix (a b : JetGaugeAlgebra) :
    (a - b).toSU2Matrix = a.toSU2Matrix - b.toSU2Matrix
```

lemma sub_toU1Value [source]

```
@[simp]
lemma sub_toU1Value (a b : JetGaugeAlgebra) :
    (a - b).toU1Value = a.toU1Value - b.toU1Value
```

instance Bracket JetGaugeAlgebra JetGaugeAlgebra [source]

The bracket on the jet gauge algebra: `I` times the matrix commutator on the `su(3)` and `su(2)` factors, and zero on the (commutative) `u(1)` factor. The factor of `I` is what makes the bracket of two hermitian matrices hermitian again; it is also why the bracket is only ℝ-bilinear, not ℂ-bilinear.

```
noncomputable instance : Bracket JetGaugeAlgebra JetGaugeAlgebra where
```

lemma bracket_toSU3Matrix [source]

```
@[simp]
lemma bracket_toSU3Matrix (a b : JetGaugeAlgebra) :
    ⁅a, b⁆.toSU3Matrix =
      Complex.I • (a.toSU3Matrix * b.toSU3Matrix - b.toSU3Matrix * a.toSU3Matrix)
```

lemma bracket_toSU2Matrix [source]

```
@[simp]
lemma bracket_toSU2Matrix (a b : JetGaugeAlgebra) :
    ⁅a, b⁆.toSU2Matrix =
      Complex.I • (a.toSU2Matrix * b.toSU2Matrix - b.toSU2Matrix * a.toSU2Matrix)
```

lemma bracket_toU1Value [source]

```
@[simp]
lemma bracket_toU1Value (a b : JetGaugeAlgebra) :
    ⁅a, b⁆.toU1Value = 0
```

instance LieRing JetGaugeAlgebra [source]

```
noncomputable instance : LieRing JetGaugeAlgebra where
```

instance LieAlgebra ℝ JetGaugeAlgebra [source]

```
noncomputable instance : LieAlgebra ℝ JetGaugeAlgebra where
```

def deriv [source]

The formal derivative in the direction μ on the jet gauge algebra, acting entrywise on each factor. It preserves hermiticity since `star` commutes with `pderiv`, and tracelessness since the trace of the entrywise derivative is the derivative of the trace.

```
noncomputable def deriv (μ : Fin 1 ⊕ Fin 3) : JetGaugeAlgebra →ₗ[ℝ] JetGaugeAlgebra where
```

lemma deriv_toSU3Matrix [source]

```
@[simp]
lemma deriv_toSU3Matrix (μ : Fin 1 ⊕ Fin 3) (a : JetGaugeAlgebra) :
    (deriv μ a).toSU3Matrix = a.toSU3Matrix.map (pderiv μ)
```

lemma deriv_toSU2Matrix [source]

```
@[simp]
lemma deriv_toSU2Matrix (μ : Fin 1 ⊕ Fin 3) (a : JetGaugeAlgebra) :
    (deriv μ a).toSU2Matrix = a.toSU2Matrix.map (pderiv μ)
```

lemma deriv_toU1Value [source]

```
@[simp]
lemma deriv_toU1Value (μ : Fin 1 ⊕ Fin 3) (a : JetGaugeAlgebra) :
```

```
    (deriv μ a).toU1Value = pderiv μ a.toU1Value
```

def coord [source]

Multiplication by the spacetime coordinate `x_μ`, entrywise on each factor. It preserves hermiticity since the coordinates are self-adjoint, and tracelessness since it is a scalar.

```
noncomputable def coord (μ : Fin 1 ⊕ Fin 3) : JetGaugeAlgebra →ₗ[ℝ] JetGaugeAlgebra where
```

lemma coord_toSU3Matrix [source]

```
@[simp]
lemma coord_toSU3Matrix (μ : Fin 1 ⊕ Fin 3) (a : JetGaugeAlgebra) :
    (coord μ a).toSU3Matrix = (X μ : SpaceTimeAlgebra) • a.toSU3Matrix
```

lemma coord_toSU2Matrix [source]

```
@[simp]
lemma coord_toSU2Matrix (μ : Fin 1 ⊕ Fin 3) (a : JetGaugeAlgebra) :
    (coord μ a).toSU2Matrix = (X μ : SpaceTimeAlgebra) • a.toSU2Matrix
```

lemma coord_toU1Value [source]

```
@[simp]
lemma coord_toU1Value (μ : Fin 1 ⊕ Fin 3) (a : JetGaugeAlgebra) :
    (coord μ a).toU1Value = (X μ : SpaceTimeAlgebra) * a.toU1Value
```

lemma deriv_coord [source]

The Leibniz rule for a coordinate: `∂_μ (x_ν a) = x_ν ∂_μ a + δ_{μν} a`.

```
lemma deriv_coord (μ ν : Fin 1 ⊕ Fin 3) (a : JetGaugeAlgebra) :
    deriv μ (coord ν a) = coord ν (deriv μ a) + if μ = ν then a else 0
```

lemma coord_lie [source]

The coordinates are central for the bracket.

```
lemma coord_lie (μ : Fin 1 ⊕ Fin 3) (a b : JetGaugeAlgebra) :
    ⁅coord μ a, b⁆ = coord μ ⁅a, b⁆
```

lemma deriv_comm [source]

Formal derivatives on the jet gauge algebra commute.

```
lemma deriv_comm (μ ν : Fin 1 ⊕ Fin 3) (a : JetGaugeAlgebra) :
    deriv μ (deriv ν a) = deriv ν (deriv μ a)
```

lemma deriv_bracket [source]

The derivative is a derivation of the bracket: the Leibniz rule `deriv μ ⁅x, y⁆ = ⁅deriv μ x, y⁆ + ⁅x, deriv μ y⁆`.

```
lemma deriv_bracket (μ : Fin 1 ⊕ Fin 3) (x y : JetGaugeAlgebra) :
    deriv μ ⁅x, y⁆ = ⁅deriv μ x, y⁆ + ⁅x, deriv μ y⁆
```

instance RightCommutative (fun (D : JetGaugeAlgebra →ₗ[ℝ] JetGaugeAlgebra) (μ :… [source]

Post-composition with `deriv` is right-commutative, since formal derivatives commute (`deriv_comm`). This is what allows iterated derivatives to be indexed by a `Multiset` of directions.

```
instance : RightCommutative
    (fun (D : JetGaugeAlgebra →ₗ[ℝ] JetGaugeAlgebra) (μ : Fin 1 ⊕ Fin 3) => D.comp (deriv μ))
    where
```

def iteratedDeriv [source]

The iterated formal derivative on the jet gauge algebra, in the (unordered, since derivatives commute) directions given by the multiset `μs`.

```
noncomputable def iteratedDeriv (μs : Multiset (Fin 1 ⊕ Fin 3)) :
    JetGaugeAlgebra →ₗ[ℝ] JetGaugeAlgebra
```

lemma iteratedDeriv_zero [source]

```
@[simp]
lemma iteratedDeriv_zero : iteratedDeriv 0 = LinearMap.id
```

lemma iteratedDeriv_cons [source]

```
lemma iteratedDeriv_cons (μ : Fin 1 ⊕ Fin 3) (μs : Multiset (Fin 1 ⊕ Fin 3)) :
    iteratedDeriv (μ ::ₘ μs) = (deriv μ).comp (iteratedDeriv μs)
```

lemma iteratedDeriv_add [source]

The iterated derivative is additive in the multiset of directions: deriving along `s + t` is deriving along `t` and then along `s`.

```
lemma iteratedDeriv_add (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    iteratedDeriv (s + t) = (iteratedDeriv s).comp (iteratedDeriv t)
```

lemma iteratedDeriv_singleton [source]

```
@[simp]
lemma iteratedDeriv_singleton (μ : Fin 1 ⊕ Fin 3) :
    iteratedDeriv ({μ} : Multiset (Fin 1 ⊕ Fin 3)) = deriv μ
```

lemma iteratedDeriv_toSU3Matrix [source]

```
lemma iteratedDeriv_toSU3Matrix (s : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    (iteratedDeriv s a).toSU3Matrix =
      a.toSU3Matrix.map fun f => SpaceTimeAlgebra.iteratedPDeriv s f
```

lemma iteratedDeriv_toSU2Matrix

[source]

```
lemma iteratedDeriv_toSU2Matrix (s : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    (iteratedDeriv s a).toSU2Matrix =
      a.toSU2Matrix.map fun f => SpaceTimeAlgebra.iteratedPDeriv s f
```

lemma iteratedDeriv_toU1Value

[source]

```
lemma iteratedDeriv_toU1Value (s : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    (iteratedDeriv s a).toU1Value = SpaceTimeAlgebra.iteratedPDeriv s a.toU1Value
```

def taylorCoeff

[source]

The Taylor coefficient of an element of the jet gauge algebra at the monomial given by the multiset `r` of spacetime directions, taken entrywise, as an $\mathbb{R}$-linear map to the constant gauge algebra `GaugeAlgebra`.

For `r ≠ 0` this is only linear: the coefficient of a product is a convolution of coefficients, so it does not respect the bracket. The zeroth coefficient does; see `eval` for that morphism of Lie algebras.

```
noncomputable def taylorCoeff (r : Multiset (Fin 1 ⊕ Fin 3)) :
    JetGaugeAlgebra →ₗ[ℝ] GaugeAlgebra where
```

lemma taylorCoeff_toSU3Matrix

[source]

```
@[simp]
lemma taylorCoeff_toSU3Matrix (r : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    (taylorCoeff r a).toSU3Matrix = a.toSU3Matrix.map (coeff r.toFinsupp)
```

lemma taylorCoeff_toSU2Matrix

[source]

```
@[simp]
lemma taylorCoeff_toSU2Matrix (r : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    (taylorCoeff r a).toSU2Matrix = a.toSU2Matrix.map (coeff r.toFinsupp)
```

lemma taylorCoeff_toU1Value

[source]

```
@[simp]
lemma taylorCoeff_toU1Value (r : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    (taylorCoeff r a).toU1Value = coeff r.toFinsupp a.toU1Value
```

lemma taylorCoeff_zero_bracket

[source]

The zeroth Taylor coefficient respects the bracket, since the constant coefficient of a product of jets is the product of the constant coefficients.

```
lemma taylorCoeff_zero_bracket (a b : JetGaugeAlgebra) :
    taylorCoeff 0 ⁅a, b⁆ = ⁅taylorCoeff 0 a, taylorCoeff 0 b⁆
```

def eval

[source]

Evaluation of the jet gauge algebra at the base point: the zeroth Taylor coefficient, as a morphism of Lie algebras.

```
noncomputable def eval : JetGaugeAlgebra →ₗ[ℝ] GaugeAlgebra
```

def ofConstant [source]

The inclusion of the constant gauge algebra into the jet gauge algebra: the jets with no spacetime dependence, given entrywise by the constant power series. This is a section of `eval`.

```
noncomputable def ofConstant : GaugeAlgebra →ₗ[ℝ] JetGaugeAlgebra where
```

lemma ofConstant_toSU3Matrix [source]

```
@[simp]
lemma ofConstant_toSU3Matrix (a : GaugeAlgebra) :
    (ofConstant a).toSU3Matrix = a.toSU3Matrix.map (C : ℂ → SpaceTimeAlgebra)
```

lemma ofConstant_toSU2Matrix [source]

```
@[simp]
lemma ofConstant_toSU2Matrix (a : GaugeAlgebra) :
    (ofConstant a).toSU2Matrix = a.toSU2Matrix.map (C : ℂ → SpaceTimeAlgebra)
```

lemma ofConstant_toU1Value [source]

```
@[simp]
lemma ofConstant_toU1Value (a : GaugeAlgebra) :
    (ofConstant a).toU1Value = C a.toU1Value
```

lemma eval_apply [source]

```
lemma eval_apply (a : JetGaugeAlgebra) : eval a = taylorCoeff 0 a
```

lemma eval_ofConstant [source]

```
@[simp]
lemma eval_ofConstant (a : GaugeAlgebra) : eval (ofConstant a) = a
```

lemma eval_toSU3Matrix_apply [source]

```
lemma eval_toSU3Matrix_apply (a : JetGaugeAlgebra) (i j : Fin 3) :
    (eval a).toSU3Matrix i j = constantCoeff (a.toSU3Matrix i j)
```

lemma eval_toSU2Matrix_apply [source]

```
lemma eval_toSU2Matrix_apply (a : JetGaugeAlgebra) (i j : Fin 2) :
    (eval a).toSU2Matrix i j = constantCoeff (a.toSU2Matrix i j)
```

lemma eval_toU1Value_eq [source]

```
lemma eval_toU1Value_eq (a : JetGaugeAlgebra) :
    (eval a).toU1Value = constantCoeff a.toU1Value
```

lemma eval_coord [source]

A coordinate multiple vanishes at the base point.

```
lemma eval_coord (μ : Fin 1 ⊕ Fin 3) (a : JetGaugeAlgebra) : eval (coord μ a) = 0
```

lemma eval_iteratedDeriv_toSU3Matrix [source]

The `su(3)` component of the base-point Taylor coefficients.

```
lemma eval_iteratedDeriv_toSU3Matrix (x : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    (eval (iteratedDeriv x a)).toSU3Matrix
      = a.toSU3Matrix.map fun f => constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x f)
```

lemma eval_iteratedDeriv_toSU2Matrix [source]

The `su(2)` component of the base-point Taylor coefficients.

```
lemma eval_iteratedDeriv_toSU2Matrix (x : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    (eval (iteratedDeriv x a)).toSU2Matrix
      = a.toSU2Matrix.map fun f => constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x f)
```

lemma eval_iteratedDeriv_toU1Value [source]

The `u(1)` component of the base-point Taylor coefficients.

```
lemma eval_iteratedDeriv_toU1Value (x : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    (eval (iteratedDeriv x a)).toU1Value
      = constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x a.toU1Value)
```

lemma ext_of_eval_iteratedDeriv [source]

Taylor determinacy: a jet gauge algebra element is determined by the base-point values of its iterated derivatives.

```
lemma ext_of_eval_iteratedDeriv {x y : JetGaugeAlgebra}
    (h : ∀ s, eval (iteratedDeriv s x) = eval (iteratedDeriv s y)) : x = y
```

def adjointMap [source]

The adjoint action of an element `U` of the jet gauge group on the jet gauge algebra, acting on the `su(3)` and `su(2)` factors by `a ↦ U a U⁻¹`, with `U⁻¹ = star U` by unitarity, and trivially on the `u(1)` factor since `SpaceTimeAlgebra` is commutative. Hermiticity is preserved since `star (U a (star U)) = U (star a) (star U)`, and tracelessness since the trace is invariant under conjugation.

```
noncomputable def adjointMap (U : JetGaugeGroupI) : JetGaugeAlgebra →ₗ[ℝ] JetGaugeAlgebra where
```

lemma adjointMap_toSU3Matrix [source]

```
@[simp]
lemma adjointMap_toSU3Matrix (U : JetGaugeGroupI) (a : JetGaugeAlgebra) :
    (adjointMap U a).toSU3Matrix = U.1.1 * a.toSU3Matrix * star U.1.1
```

lemma adjointMap_toSU2Matrix [source]

```
@[simp]
lemma adjointMap_toSU2Matrix (U : JetGaugeGroupI) (a : JetGaugeAlgebra) :
    (adjointMap U a).toSU2Matrix = U.2.1.1 * a.toSU2Matrix * star U.2.1.1
```

lemma adjointMap_toU1Value [source]

```
@[simp]
lemma adjointMap_toU1Value (U : JetGaugeGroupI) (a : JetGaugeAlgebra) :
    (adjointMap U a).toU1Value = a.toU1Value
```

def adjoint [source]

The adjoint representation of the jet gauge group on the jet gauge algebra, `U ↦ (a ↦ U a U⁻¹)` factorwise.

```
noncomputable def adjoint : Representation ℝ JetGaugeGroupI JetGaugeAlgebra where
```

lemma eval_adjointMap [source]

At the base point the adjoint action of a gauge jet is the adjoint action of its value: the constant coefficient of `U x U†` is `U₀ x₀ U₀†`.

```
lemma eval_adjointMap (U : JetGaugeGroupI) (x : JetGaugeAlgebra) :
    eval (adjointMap U x) = GaugeAlgebra.adjoint U.eval (eval x)
```

lemma ofConstant_lie [source]

The constant inclusion is a morphism of Lie algebras: constants bracket to constants.

```
lemma ofConstant_lie (a b : GaugeAlgebra) :
    ofConstant ⁅a, b⁆ = ⁅ofConstant a, ofConstant b⁆
```

lemma adjointMap_lie [source]

The adjoint action preserves the bracket: conjugation is an automorphism of the Lie algebra, using unitarity to cancel the inner `U† U` factors.

```
lemma adjointMap_lie (U : JetGaugeGroupI) (x y : JetGaugeAlgebra) :
    adjointMap U ⁅x, y⁆ = ⁅adjointMap U x, adjointMap U y⁆
```

2.443 PhyslibAlpha.Particles.StandardModel.GaugeAlgebra.RootDecomposition

PhyslibAlpha/Particles/StandardModel/GaugeAlgebra/RootDecomposition.lean on GitHub.

lemma star_expI_zpow [source]

Conjugation inverts a power of `expI`.

```
lemma star_expI_zpow (z : ℤ) : star ((expI : ℂ) ^ z) = (expI : ℂ) ^ (-z)
```

def stdCoeff [source]

coords

```
noncomputable def stdCoeff (x : GaugeAlgebra) : Fin 8 ⊕ Fin 3 ⊕ Fin 1 → ℝ
  | Sum.inl k => gellMannCoeff x.toSU3Matrix k
  | Sum.inr (Sum.inl i) => pauliCoeff x.toSU2Matrix i
  | Sum.inr (Sum.inr _) => (x.toU1Value).re
```

lemma eq_sum_stdCoeff [source]

```
lemma eq_sum_stdCoeff (x : GaugeAlgebra) : x = ∑ y, stdCoeff x y • stdBasis y
```

lemma stdBasis_coord_apply [source]

The coordinate functionals of `stdBasis` read off a gauge-algebra element's matrix entries.

```
lemma stdBasis_coord_apply (y : GaugeAlgebra) (a : Fin 8 ⊕ Fin 3 ⊕ Fin 1) :
    stdBasis.coord a y = stdCoeff y a
```

def torusSU3Diag [source]

su3 diagonals of inverse torus gens

```
noncomputable def torusSU3Diag : Fin 4 → Fin 3 → ℂ
```

def torusSU2Diag [source]

su2

```
noncomputable def torusSU2Diag : Fin 4 → Fin 2 → ℂ
```

lemma toSU3_inv_gaugeTorusGen [source]

```
lemma toSU3_inv_gaugeTorusGen (i : Fin 4) :
    ((GaugeGroupI.toSU3 (gaugeTorusGen i)⁻¹ : specialUnitaryGroup (Fin 3) ℂ) :
      Matrix (Fin 3) (Fin 3) ℂ) = Matrix.diagonal (torusSU3Diag i)
```

lemma toSU2_inv_gaugeTorusGen [source]

```
lemma toSU2_inv_gaugeTorusGen (i : Fin 4) :
    ((GaugeGroupI.toSU2 (gaugeTorusGen i)⁻¹ : specialUnitaryGroup (Fin 2) ℂ) :
      Matrix (Fin 2) (Fin 2) ℂ) = Matrix.diagonal (torusSU2Diag i)
```

lemma adjointMap_toSU3Matrix_apply_diagonal [source]

```
lemma adjointMap_toSU3Matrix_apply_diagonal {g : GaugeGroupI} {d : Fin 3 → ℂ}
    (hg : ((GaugeGroupI.toSU3 g : specialUnitaryGroup (Fin 3) ℂ) :
      Matrix (Fin 3) (Fin 3) ℂ) = Matrix.diagonal d)
    (x : GaugeAlgebra) (j k : Fin 3) :
    (adjointMap g x).toSU3Matrix j k = d j * star (d k) * x.toSU3Matrix j k
```

lemma adjointMap_toSU2Matrix_apply_diagonal [source]

```
lemma adjointMap_toSU2Matrix_apply_diagonal {g : GaugeGroupI} {d : Fin 2 → ℂ}
    (hg : ((GaugeGroupI.toSU2 g : specialUnitaryGroup (Fin 2) ℂ) :
      Matrix (Fin 2) (Fin 2) ℂ) = Matrix.diagonal d)
    (x : GaugeAlgebra) (j k : Fin 2) :
    (adjointMap g x).toSU2Matrix j k = d j * star (d k) * x.toSU2Matrix j k
```

lemma torusSU3Diag_mul_star [source]

```
lemma torusSU3Diag_mul_star (i : Fin 4) (j : Fin 3) :
    torusSU3Diag i j * star (torusSU3Diag i j) = 1
```

lemma torusSU2Diag_mul_star [source]

```
lemma torusSU2Diag_mul_star (i : Fin 4) (j : Fin 2) :
    torusSU2Diag i j * star (torusSU2Diag i j) = 1
```

lemma dualMap_pair_of_entry [source]

```
lemma dualMap_pair_of_entry {g : GaugeGroupI} {e : GaugeAlgebra → ℂ}
    {φ₁ φ₂ : Module.Dual ℝ GaugeAlgebra} {z : ℂ}
    (h1 : ∀ x, φ₁ x = (e x).re) (h2 : ∀ x, φ₂ x = -(e x).im)
    (he : ∀ x, e (adjointMap g x) = star z * e x) :
    (adjointMap g).dualMap φ₁ = z.re • φ₁ - z.im • φ₂ ∧
      (adjointMap g).dualMap φ₂ = z.im • φ₁ + z.re • φ₂
```

def rootIdx [source]

The four root directions of the adjoint.

```
def rootIdx : Fin 4 → (Fin 8 ⊕ Fin 3 ⊕ Fin 1) × (Fin 8 ⊕ Fin 3 ⊕ Fin 1)
  | 0 => (Sum.inl 0, Sum.inl 1)
  | 1 => (Sum.inl 3, Sum.inl 4)
  | 2 => (Sum.inl 5, Sum.inl 6)
  | 3 => (Sum.inr (Sum.inl 0), Sum.inr (Sum.inl 1))
```

def rootWeight [source]

The gauge weight of each root direction.

```
def rootWeight : Fin 4 → GaugeWeight
  | 0 => (2, -1, 0, 0)
  | 1 => (1, 1, 0, 0)
  | 2 => (-1, 2, 0, 0)
  | 3 => (0, 0, 2, 0)
```

def rootEntry [source]

The matrix entry scaled by the torus along each root direction.

```
def rootEntry : Fin 4 → GaugeAlgebra → ℂ
  | 0, x => x.toSU3Matrix 0 1
  | 1, x => x.toSU3Matrix 0 2
  | 2, x => x.toSU3Matrix 1 2
  | 3, x => x.toSU2Matrix 0 1
```

abbrev su3CartanId [source]

The Gell-Mann indices of the two Cartan directions of `su(3)`.

```
abbrev su3CartanId : Fin 2 → Fin 8
  | 0 => 2
  | 1 => 7
```

abbrev su2CartanId [source]

The Pauli index of the Cartan direction of `su(2)`.

```
abbrev su2CartanId : Fin 3
```

def cartanIdx [source]

The four weight-zero directions: the two `su(3)` Cartan generators, the `su(2)` Cartan generator and the `u(1)` generator.

```
def cartanIdx : Fin 4 → (Fin 8 ⊕ Fin 3 ⊕ Fin 1)
  | 0 => Sum.inl (su3CartanId 0)
  | 1 => Sum.inl (su3CartanId 1)
  | 2 => Sum.inr (Sum.inl su2CartanId)
  | 3 => Sum.inr (Sum.inr 0)
```

lemma coord_rootIdx_fst [source]

```
lemma coord_rootIdx_fst (r : Fin 4) (x : GaugeAlgebra) :
    stdBasis.coord (rootIdx r).1 x = (rootEntry r x).re
```

lemma coord_rootIdx_snd [source]

```
lemma coord_rootIdx_snd (r : Fin 4) (x : GaugeAlgebra) :
    stdBasis.coord (rootIdx r).2 x = -(rootEntry r x).im
```

lemma rootEntry_adjointMap [source]

```
lemma rootEntry_adjointMap (r : Fin 4) (i : Fin 4) (x : GaugeAlgebra) :
    rootEntry r (adjointMap (gaugeTorusGen i)⁻¹ x)
      = star ((expI : ℂ) ^ GaugeWeight.coord (rootWeight r) i) * rootEntry r x
```

lemma dualMap_coord_cartanIdx [source]

```
lemma dualMap_coord_cartanIdx (c : Fin 4) (i : Fin 4) :
    (adjointMap (gaugeTorusGen i)⁻¹).dualMap (stdBasis.coord (cartanIdx c))
      = stdBasis.coord (cartanIdx c)
```

lemma real_smul_eq_complex_smul [source]

A real scalar acts on a complex algebra through its complex image.

```
lemma real_smul_eq_complex_smul (r : ℝ) (b : B) : r • b = ((r : ℂ)) • b
```

def IsCoadjointSymbol [source]

A coadjoint symbol map: a real-linear map from the dual of the gauge algebra into a complex algebra which intertwines the gauge action with the dual adjoint action. The field strength of the gauge sector is one such map.

```
def IsCoadjointSymbol (rep : Representation ℂ GaugeGroupI B)
    (F : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B) : Prop
```

abbrev AdjIdx [source]

The index type of the adjoint weight vectors: four positive roots, four negative roots and four Cartan directions.

```
abbrev AdjIdx : Type
```

def adjWeight [source]

The gauge weight carried by each adjoint weight vector.

```
def adjWeight : AdjIdx → GaugeWeight
  | Sum.inl r => rootWeight r
  | Sum.inr (Sum.inl r) => -(rootWeight r)
  | Sum.inr (Sum.inr _) => 0
```

def adjVec [source]

The weight vectors of the adjoint in the image of a coadjoint symbol map: for each root the two combinations of its paired coordinate symbols, and for each Cartan direction the symbol itself.

```
noncomputable def adjVec (F : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B) : AdjIdx → B
  | Sum.inl r => F (stdBasis.coord (rootIdx r).1)
      + Complex.I • F (stdBasis.coord (rootIdx r).2)
  | Sum.inr (Sum.inl r) => F (stdBasis.coord (rootIdx r).1)
      - Complex.I • F (stdBasis.coord (rootIdx r).2)
  | Sum.inr (Sum.inr c) => F (stdBasis.coord (cartanIdx c))
```

lemma adjWeight_eq_zero_iff [source]

A weight vector carries zero gauge weight exactly when it is a Cartan direction; the eight root directions all carry a nonzero weight.

```
lemma adjWeight_eq_zero_iff (k : AdjIdx) :
    adjWeight k = 0 ↔ ∃ c : Fin 4, k = Sum.inr (Sum.inr c)
```

lemma exists_rootIdx_or_cartanIdx [source]

The root and Cartan directions exhaust the standard basis: every standard index is one of the two members of a root pair, or a Cartan index.

```
lemma exists_rootIdx_or_cartanIdx (a : Fin 8 ⊕ Fin 3 ⊕ Fin 1) :
    (∃ r : Fin 4, a = (rootIdx r).1) ∨ (∃ r : Fin 4, a = (rootIdx r).2)
      ∨ ∃ c : Fin 4, a = cartanIdx c
```

lemma rep_pair_add [source]

The positive combination of a rotating pair of symbols is scaled by the rotation.

```
lemma rep_pair_add (hF : IsCoadjointSymbol rep F) (g : GaugeGroupI)
    (φ₁ φ₂ : Module.Dual ℝ GaugeAlgebra) (z : ℂ)
    (h1 : (adjointMap g⁻¹).dualMap φ₁ = z.re • φ₁ - z.im • φ₂)
    (h2 : (adjointMap g⁻¹).dualMap φ₂ = z.im • φ₁ + z.re • φ₂) :
    rep g (F φ₁ + Complex.I • F φ₂) = z • (F φ₁ + Complex.I • F φ₂)
```

lemma rep_pair_sub [source]

The negative combination of a rotating pair of symbols is scaled by the conjugate rotation.

```
lemma rep_pair_sub (hF : IsCoadjointSymbol rep F) (g : GaugeGroupI)
    (φ₁ φ₂ : Module.Dual ℝ GaugeAlgebra) (z : ℂ)
    (h1 : (adjointMap g⁻¹).dualMap φ₁ = z.re • φ₁ - z.im • φ₂)
    (h2 : (adjointMap g⁻¹).dualMap φ₂ = z.im • φ₁ + z.re • φ₂) :
    rep g (F φ₁ - Complex.I • F φ₂)
      = (starRingEnd ℂ z) • (F φ₁ - Complex.I • F φ₂)
```

lemma rep_fixed [source]

A symbol at a fixed coordinate functional is itself fixed.

```
lemma rep_fixed (hF : IsCoadjointSymbol rep F) (g : GaugeGroupI)
    (φ : Module.Dual ℝ GaugeAlgebra) (h1 : (adjointMap g⁻¹).dualMap φ = φ) :
    rep g (F φ) = F φ
```

lemma rep_adjVec [source]

Each adjoint weight vector is a simultaneous eigenvector of the four torus generators, at the character of its weight.

```
lemma rep_adjVec (hF : IsCoadjointSymbol rep F) (k : AdjIdx) (i : Fin 4) :
    rep (gaugeTorusGen i) (adjVec F k)
      = ((expI : ℂ) ^ GaugeWeight.coord (adjWeight k) i) • adjVec F k
```

lemma symbol_rootIdx_fst [source]

The first symbol of a root pair, recovered from the two weight vectors.

```
lemma symbol_rootIdx_fst (F : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B) (r : Fin 4) :
    F (stdBasis.coord (rootIdx r).1)
      = (2 : ℂ)⁻¹ • (adjVec F (Sum.inl r) + adjVec F (Sum.inr (Sum.inl r)))
```

lemma symbol_rootIdx_snd [source]

The second symbol of a root pair, recovered from the two weight vectors.

```
lemma symbol_rootIdx_snd (F : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B) (r : Fin 4) :
    F (stdBasis.coord (rootIdx r).2)
      = (-(Complex.I / 2)) • (adjVec F (Sum.inl r) - adjVec F (Sum.inr (Sum.inl r)))
```

lemma symbol_mem_iSup [source]

Every standard coordinate symbol lies in the join of the twelve weight lines.

```
lemma symbol_mem_iSup (F : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B)
    (a : Fin 8 ⊕ Fin 3 ⊕ Fin 1) :
    F (stdBasis.coord a) ∈ ⨆ k : AdjIdx, Submodule.span ℂ {adjVec F k}
```

lemma span_range_eq_iSup [source]

The span of the symbols is the join of the twelve weight lines.

```
lemma span_range_eq_iSup (F : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B) :
    Submodule.span ℂ (Set.range F) = ⨆ k : AdjIdx, Submodule.span ℂ {adjVec F k}
```

def adjointDecomposition [source]

The root decomposition read as a gauge weight decomposition: the span of the symbols of a coadjoint map, joined out of the twelve root and Cartan lines.

```
@[implicit_reducible]
noncomputable def adjointDecomposition (hmul : IsMulRep rep)
    (hF : IsCoadjointSymbol rep F) :
    GaugeWeightDecomposition rep (Submodule.span ℂ (Set.range F))
```

lemma adjointDecomposition_supp [source]

The gauge weights of the adjoint: the six `su(3)` roots, the two `su(2)` roots and the zero weight of the Cartan and `u(1)` directions. Every one of them has vanishing hypercharge.

```
lemma adjointDecomposition_supp (hmul : IsMulRep rep) (hF : IsCoadjointSymbol rep F) :
    (adjointDecomposition hmul hF).supp
      = {((2, -1, 0, 0) : GaugeWeight), (1, 1, 0, 0), (-1, 2, 0, 0), (0, 0, 2, 0),
        (-2, 1, 0, 0), (-1, -1, 0, 0), (1, -2, 0, 0), (0, 0, -2, 0), (0, 0, 0, 0)}
```

lemma adjointDecomposition_piece [source]

The pieces of the decomposition: the weight-`w` piece is the join of the weight lines whose weight is `w`.

```
lemma adjointDecomposition_piece (hmul : IsMulRep rep) (hF : IsCoadjointSymbol rep F)
    (w : GaugeWeight) :
    (adjointDecomposition hmul hF).piece w
      = ⨆ k : AdjIdx, if w = adjWeight k then Submodule.span ℂ {adjVec F k} else ⊥
```

lemma adjointDecomposition_piece_rootWeight [source]

The piece at a root weight is the line of that root alone.

```
lemma adjointDecomposition_piece_rootWeight (hmul : IsMulRep rep)
    (hF : IsCoadjointSymbol rep F) (r : Fin 4) :
    (adjointDecomposition hmul hF).piece (rootWeight r)
      = Submodule.span ℂ {adjVec F (Sum.inl r)}
```

lemma adjointDecomposition_piece_neg_rootWeight [source]

The piece at the opposite of a root weight is the line of the opposite root.

```
lemma adjointDecomposition_piece_neg_rootWeight (hmul : IsMulRep rep)
    (hF : IsCoadjointSymbol rep F) (r : Fin 4) :
    (adjointDecomposition hmul hF).piece (-(rootWeight r))
      = Submodule.span ℂ {adjVec F (Sum.inr (Sum.inl r))}
```

lemma adjointDecomposition_piece_zero [source]

The weight-zero piece is the span of the four Cartan symbols: the two `su(3)` Cartan generators, the `su(2)` Cartan generator and the `u(1)` generator.

```
lemma adjointDecomposition_piece_zero (hmul : IsMulRep rep)
    (hF : IsCoadjointSymbol rep F) :
    (adjointDecomposition hmul hF).piece 0
      = ⨆ c : Fin 4, Submodule.span ℂ {F (stdBasis.coord (cartanIdx c))}
```

lemma adjointDecomposition_piece_eq_bot [source]

Every weight outside the nine is absent from the adjoint.

```
lemma adjointDecomposition_piece_eq_bot (hmul : IsMulRep rep)
    (hF : IsCoadjointSymbol rep F) {w : GaugeWeight}
    (hw : w ∉ (adjointDecomposition hmul hF).supp) :
    (adjointDecomposition hmul hF).piece w = ⊥
```

abbrev su3WeightIdx [source]

The index type of the `su(3)` weight coordinates: three positive roots, three negative roots and two Cartan directions.

```
abbrev su3WeightIdx : Type
```

def su3RootPair [source]

The pairs of Gell-Mann indices making up the three root directions of `su(3)`.

```
def su3RootPair : Fin 3 → Fin 8 × Fin 8
  | 0 => (0, 1)
  | 1 => (3, 4)
  | 2 => (5, 6)
```

lemma rootIdx_castSucc [source]

The `su(3)` root pairs are the first three root pairs of the whole gauge algebra.

```
lemma rootIdx_castSucc (r : Fin 3) :
    rootIdx r.castSucc = (Sum.inl (su3RootPair r).1, Sum.inl (su3RootPair r).2)
```

lemma cartanIdx_castSucc [source]

The `su(3)` Cartan indices are the first two Cartan indices of the whole gauge algebra.

```
lemma cartanIdx_castSucc (c : Fin 2) :
    cartanIdx c.castSucc.castSucc = Sum.inl (su3CartanId c)
```

def su3WeightCoeff [source]

The `su(3)` weight coordinates in Gell-Mann coordinates: `x₁ ± i x₂` on each root pair, and the Cartan coordinates themselves.

```
noncomputable def su3WeightCoeff : su3WeightIdx → Fin 8 → ℂ
  | Sum.inl r, a => (if a = (su3RootPair r).1 then 1 else 0)
      + Complex.I * (if a = (su3RootPair r).2 then 1 else 0)
  | Sum.inr (Sum.inl r), a => (if a = (su3RootPair r).1 then 1 else 0)
      - Complex.I * (if a = (su3RootPair r).2 then 1 else 0)
  | Sum.inr (Sum.inr c), a => if a = su3CartanId c then 1 else 0
```

abbrev su2WeightIdx [source]

The index type of the `su(2)` weight coordinates: the positive root, the negative root and the Cartan direction.

```
abbrev su2WeightIdx : Type
```

def su2RootPair [source]

The pair of Pauli indices making up the root direction of `su(2)`.

```
def su2RootPair : Fin 3 × Fin 3
```

lemma rootIdx_three [source]

The `su(2)` root pair is the fourth root pair of the whole gauge algebra.

```
lemma rootIdx_three :
    rootIdx 3 = (Sum.inr (Sum.inl su2RootPair.1), Sum.inr (Sum.inl su2RootPair.2))
```

lemma cartanIdx_two [source]

The `su(2)` Cartan index is the third Cartan index of the whole gauge algebra.

```
lemma cartanIdx_two : cartanIdx 2 = Sum.inr (Sum.inl su2CartanId)
```

def su2WeightCoeff [source]

The `su(2)` weight coordinates in Pauli coordinates: `x₁ ± i x₂` on the root pair, and the Cartan coordinate itself.

```
noncomputable def su2WeightCoeff : su2WeightIdx → Fin 3 → ℂ
  | Sum.inl _, a => (if a = su2RootPair.1 then 1 else 0)
      + Complex.I * (if a = su2RootPair.2 then 1 else 0)
  | Sum.inr (Sum.inl _), a => (if a = su2RootPair.1 then 1 else 0)
      - Complex.I * (if a = su2RootPair.2 then 1 else 0)
  | Sum.inr (Sum.inr _), a => if a = su2CartanId then 1 else 0
```

2.444 PhyslibAlpha.Particles.StandardModel.GaugeGroup.AdjointMatrix

PhyslibAlpha/Particles/StandardModel/GaugeGroup/AdjointMatrix.lean on GitHub.

def su2AdjointMatrix [source]

The adjoint matrix of an element of `SU(2)`: the trace pairing of the Pauli basis of `su(2)` with the Pauli basis conjugated by that element.

```
noncomputable def su2AdjointMatrix (U : specialUnitaryGroup (Fin 2) ℂ) :
    Matrix (Fin 3) (Fin 3) ℝ
```

lemma su2AdjointMatrix_apply [source]

The entries of the adjoint matrix.

```
@[simp]
lemma su2AdjointMatrix_apply (U : specialUnitaryGroup (Fin 2) ℂ) (i j : Fin 3) :
    su2AdjointMatrix U i j
      = 2⁻¹ * (Matrix.trace (pauliMatrix (Sum.inr i) *
          (U.1 * pauliMatrix (Sum.inr j) * star U.1))).re
```

lemma sum_su2AdjointMatrix_row_mul [source]

The rows of the adjoint matrix are orthonormal.

```
lemma sum_su2AdjointMatrix_row_mul (U : specialUnitaryGroup (Fin 2) ℂ) (c d : Fin 3) :
    ∑ a : Fin 3, su2AdjointMatrix U c a * su2AdjointMatrix U d a
      = if c = d then 1 else 0
```

lemma su2AdjointMatrix_inv [source]

The adjoint matrix of the inverse is the transpose.

```
lemma su2AdjointMatrix_inv (U : specialUnitaryGroup (Fin 2) ℂ) (a b : Fin 3) :
    su2AdjointMatrix U⁻¹ a b = su2AdjointMatrix U b a
```

def su3AdjointMatrix [source]

The adjoint matrix of `U ∈ SU(3)`: the trace pairing of the Gell-Mann basis with the Gell-Mann basis conjugated by `U`.

```
noncomputable def su3AdjointMatrix (U : specialUnitaryGroup (Fin 3) ℂ) :
    Matrix (Fin 8) (Fin 8) ℝ
```

lemma su3AdjointMatrix_apply [source]

```
@[simp]
lemma su3AdjointMatrix_apply (U : specialUnitaryGroup (Fin 3) ℂ) (i j : Fin 8) :
    su3AdjointMatrix U i j
      = 2⁻¹ * (Matrix.trace (gellMannMatrix i * (U.1 * gellMannMatrix j * star U.1))).re
```

lemma sum_su3AdjointMatrix_row_mul [source]

The rows of the adjoint matrix are orthonormal.

```
lemma sum_su3AdjointMatrix_row_mul (U : specialUnitaryGroup (Fin 3) ℂ) (c d : Fin 8) :
    ∑ a : Fin 8, su3AdjointMatrix U c a * su3AdjointMatrix U d a
      = if c = d then 1 else 0
```

lemma su3AdjointMatrix_inv [source]

The adjoint matrix of the inverse is the transpose.

```
lemma su3AdjointMatrix_inv (U : specialUnitaryGroup (Fin 3) ℂ) (a b : Fin 8) :
    su3AdjointMatrix U⁻¹ a b = su3AdjointMatrix U b a
```

2.445 PhyslibAlpha.Particles.StandardModel.GaugeGroup.Basic

PhyslibAlpha/Particles/StandardModel/GaugeGroup/Basic.lean on GitHub.

def cubeRootOfUnity [source]

The primitive cube root of unity `ω = exp (2 π i / 3)`.

```
noncomputable def cubeRootOfUnity : ℂ
```

lemma cubeRootOfUnity_isPrimitiveRoot [source]

`ω` is a primitive cube root of unity.

```
lemma cubeRootOfUnity_isPrimitiveRoot : IsPrimitiveRoot cubeRootOfUnity 3
```

lemma cubeRootOfUnity_ne_zero [source]

`ω` is nonzero, being a value of the complex exponential.

```
lemma cubeRootOfUnity_ne_zero : cubeRootOfUnity ≠ 0
```

lemma cubeRootOfUnity_pow_mod [source]

Powers of `ω` only see the exponent modulo three.

```
lemma cubeRootOfUnity_pow_mod (m : ℕ) :
    cubeRootOfUnity ^ (m % 3) = cubeRootOfUnity ^ m
```

lemma cubeRootOfUnity_mul_star [source]

`ω` has modulus one.

```
lemma cubeRootOfUnity_mul_star : cubeRootOfUnity * star cubeRootOfUnity = 1
```

2.446 PhyslibAlpha.Particles.StandardModel.GaugeGroup.GaugeWeight

PhyslibAlpha/Particles/StandardModel/GaugeGroup/GaugeWeight.lean on GitHub.

def expI [source]

The unitary scalar `exp i`, a point of the unit circle of infinite order.

```
noncomputable def expI : unitary ℂ
```

lemma expI_zpow_injective [source]

The powers of `exp i` are pairwise distinct, by the irrationality of `π`.

```
lemma expI_zpow_injective : Function.Injective fun n : ℤ => ((expI : ℂ) ^ n)
```

lemma expI_ne_zero [source]

`exp i` is nonzero.

```
lemma expI_ne_zero : ((expI : ℂ)) ≠ 0
```

lemma expI_inv_eq_star [source]

The inverse of `exp i` is its star.

```
lemma expI_inv_eq_star : ((expI : ℂ))⁻¹ = star (expI : ℂ)
```

lemma expI_mul_conj [source]

`exp i` times its conjugate is one.

```
lemma expI_mul_conj : (expI : ℂ) * (starRingEnd ℂ) (expI : ℂ) = 1
```

lemma conj_mul_expI [source]

The conjugate of `exp i` times `exp i` is one.

```
lemma conj_mul_expI : (starRingEnd ℂ) (expI : ℂ) * (expI : ℂ) = 1
```

lemma mem_specialUnitaryGroup_diagonal [source]

A diagonal matrix whose entries are unit scalars with product one lies in the special unitary group.

```
lemma _root_.Matrix.mem_specialUnitaryGroup_diagonal {n : Type*} [Fintype n] [DecidableEq n]
    (d : n → ℂ) (hd : ∀ i, d i * star (d i) = 1) (hdet : ∏ i, d i = 1) :
    Matrix.diagonal d ∈ Matrix.specialUnitaryGroup n ℂ
```

def su3ExpIOne [source]

The first colour torus generator, `diag (exp i, exp (-i), 1)`.

```
noncomputable def su3ExpIOne : specialUnitaryGroup (Fin 3) ℂ
```

def su3ExpITwo [source]

The second colour torus generator, `diag (1, exp i, exp (-i))`.

```
noncomputable def su3ExpITwo : specialUnitaryGroup (Fin 3) ℂ
```

def su2ExpI [source]

The `SU(2)` torus element, `diag (exp i, exp (-i))`.

```
noncomputable def su2ExpI : specialUnitaryGroup (Fin 2) ℂ
```

lemma su2ExpI_coe [source]

The underlying matrix of the `SU(2)` torus element.

```
lemma su2ExpI_coe :
    (su2ExpI : specialUnitaryGroup (Fin 2) ℂ).1
      = !![(expI : ℂ), 0; 0, star (expI : ℂ)]
```

lemma su2ExpI_inv_coe [source]

The inverse torus element is `diag (exp (-i), exp i)`, so on a doublet the two components are scaled by the reciprocal characters.

```
lemma su2ExpI_inv_coe :
    (su2ExpI⁻¹ : specialUnitaryGroup (Fin 2) ℂ).1
      = !![star (expI : ℂ), 0; 0, (expI : ℂ)]
```

def gaugeTorusGen [source]

The four commuting generators of the maximal torus of the gauge group.

```
noncomputable def gaugeTorusGen : Fin 4 → GaugeGroupI
```

abbrev GaugeWeight [source]

*A **gauge weight**, the four exponents `(colour₁, colour₂, isospin, hypercharge)` recording how a vector scales under `gaugeTorusGen`.*

```
abbrev GaugeWeight : Type
```

def coord [source]

The exponent of a gauge weight against the `i`-th torus generator.

```
def GaugeWeight.coord (w : GaugeWeight) : Fin 4 → ℤ
```

lemma coord_add [source]

Weights add coordinatewise. With `zero_coord` this says `coord` is additive, which is what makes gauge weights add under multiplication.

```
lemma GaugeWeight.coord_add (w w' : GaugeWeight) (i : Fin 4) :
    (w + w').coord i = w.coord i + w'.coord i
```

lemma coord_injective [source]

A gauge weight is determined by its four exponents. This is what lets a weight be recovered from the characters by which the torus acts; see `piece_eq_inf`.

```
lemma GaugeWeight.coord_injective : Function.Injective GaugeWeight.coord
```

abbrev IsMulRep [source]

A representation acts by algebra maps, respecting multiplication. This is the hypothesis under which charges are additive.

```
abbrev IsMulRep (rep : Representation ℂ GaugeGroupI B) : Prop
```

lemma map_one [source]

A representation that respects multiplication respects the unit, since `g` is invertible and so `rep g 1` is cancellable.

```
lemma IsMulRep.map_one {rep : Representation ℂ GaugeGroupI B} (hmul : IsMulRep rep)
    (g : GaugeGroupI) : rep g 1 = 1
```

lemma starRingEnd_expI_pow [source]

```
lemma starRingEnd_expI_pow (n : ℕ) :
    ((starRingEnd ℂ) (expI : ℂ)) ^ n = ((expI : ℂ) ^ n)⁻¹
```

lemma starRingEnd_expI_zpow [source]

```
lemma starRingEnd_expI_zpow (z : ℤ) :
    (starRingEnd ℂ) ((expI : ℂ) ^ z) = (expI : ℂ) ^ (-z)
```

lemma expI_zpow_ne_zero [source]

```
lemma expI_zpow_ne_zero (z : ℤ) : ((expI : ℂ) ^ z) ≠ 0
```

def colourWeight [source]

The colour weights of the fundamental of `SU(3)` against the two colour torus generators.

```
def colourWeight (c : Fin 3) : ℤ × ℤ
```

def isoWeight [source]

The isospin weight of the fundamental of `SU(2)` against the isospin torus generator.

```
def isoWeight (s : Fin 2) : ℤ
```

lemma dual_gaugeTorusGen_coord [source]

```
lemma dual_gaugeTorusGen_coord (ρ : Representation ℂ GaugeGroupI V)
    (b : Module.Basis ι ℂ V) (g : GaugeGroupI) (w : ι → ℤ)
    (hb : ∀ j, ρ g (b j) = ((expI : ℂ) ^ w j) • b j) (j : ι) :
    ρ.dual g (b.coord j) = ((expI : ℂ) ^ (-(w j))) • b.coord j
```

lemma conj_gaugeTorusGen_basis [source]

```
lemma conj_gaugeTorusGen_basis (ρ : Representation ℂ GaugeGroupI V)
    (b : Module.Basis ι ℂ V) (g : GaugeGroupI) (w : ι → ℤ)
    (hb : ∀ j, ρ g (b j) = ((expI : ℂ) ^ w j) • b j) (j : ι) :
    ρ.conj g (Basis.conj b j)
      = ((expI : ℂ) ^ (-(w j))) • Basis.conj b j
```

class GaugeWeightDecomposition [source]

*A **gauge weight decomposition** of a submodule `V`, a finitely supported family of subspaces of pure gauge weight whose supremum is `V`. Purity is recorded against the four commuting torus generators simultaneously.*

```
class GaugeWeightDecomposition (rep : Representation ℂ GaugeGroupI B)
    (V : Submodule ℂ B) where
  piece : GaugeWeight → Submodule ℂ B
  supp : Finset GaugeWeight
  rep_mul : IsMulRep rep
  piece_le : ∀ w, ∀ x, x ∈ piece w → ∀ i,
    rep (gaugeTorusGen i) x = ((expI : ℂ) ^ w.coord i) • x
  piece_eq_bot : ∀ w ∉ supp, piece w = ⊥
  iSup_piece : (⨆ w, piece w) = V
```

lemma piece_le_eigenspace [source]

The weight-`w` piece lies in the eigenspace of the `i`-th torus generator at the eigenvalue `(exp i) ^ (w.coord i)`. This is `piece_le` phrased as an inequality of submodules.

```
lemma piece_le_eigenspace (d : GaugeWeightDecomposition rep V) (w : GaugeWeight) (i : Fin 4) :
    d.piece w ≤ Module.End.eigenspace (rep (gaugeTorusGen i)) ((expI : ℂ) ^ w.coord i)
```

lemma piece_eq_zero_of_not_mem_supp [source]

A weight outside the support has vanishing piece. This is the `piece_eq_bot` field, in the form a `simp` set can use to discard the absent weights of a computed product.

```
lemma piece_eq_zero_of_not_mem_supp (d : GaugeWeightDecomposition rep V) (w : GaugeWeight)
    (hw : w ∉ d.supp) : d.piece w = ⊥
```

lemma piece_le_self [source]

A weight piece lies inside the submodule it decomposes.

```
lemma piece_le_self (d : GaugeWeightDecomposition rep V) (w : GaugeWeight) :
    d.piece w ≤ V
```

def copy [source]

Transport a decomposition along an equality of submodules.

```
@[implicit_reducible]
def copy (d : GaugeWeightDecomposition rep V) (W : Submodule ℂ B) (hW : W = V) :
    GaugeWeightDecomposition rep W where
```

lemma copy_piece [source]

```
@[simp]
lemma copy_piece (d : GaugeWeightDecomposition rep V) (W : Submodule ℂ B) (hW : W = V) :
    (copy d W hW).piece = d.piece
```

instance sup [source]

The join of two gauge weight decompositions, decomposing `V ⊔ V'`. Pieces and supports combine one weight at a time.

```
@[implicit_reducible]
noncomputable instance sup [d : GaugeWeightDecomposition rep V]
    [d' : GaugeWeightDecomposition rep V'] : GaugeWeightDecomposition rep (V ⊔ V') where
```

lemma sup_piece [source]

```
@[simp]
lemma sup_piece [GaugeWeightDecomposition rep V] [GaugeWeightDecomposition rep V']
    (w : GaugeWeight) : piece rep (V ⊔ V') w = piece rep V w ⊔ piece rep V' w
```

def bot [source]

The zero submodule carries the empty decomposition.

```
@[implicit_reducible]
def bot (hmul : IsMulRep rep) :
    GaugeWeightDecomposition rep (⊥ : Submodule ℂ B) where
```

lemma bot_piece [source]

```
@[simp]
lemma bot_piece (hmul : IsMulRep rep)
    (w : GaugeWeight) : (bot hmul).piece w = ⊥
```

lemma bot_supp [source]

```
@[simp]
lemma bot_supp (hmul : IsMulRep rep) :
    (bot hmul).supp = ∅
```

def spanSingleton [source]

The span of a single simultaneous eigenvector of the gauge torus, as a decomposition concentrated in its one weight. This is the base case from which the decompositions of spans of weight vectors are assembled by `iSup` and `sup`.

```
@[implicit_reducible]
noncomputable def spanSingleton (hmul : IsMulRep rep) (x : B) (w : GaugeWeight)
    (hx : ∀ i, rep (gaugeTorusGen i) x = ((expI : ℂ) ^ w.coord i) • x) :
    GaugeWeightDecomposition rep (Submodule.span ℂ {x}) where
```

lemma spanSingleton_piece [source]

```
@[simp]
lemma spanSingleton_piece (hmul : IsMulRep rep) (x : B) (w : GaugeWeight)
    (hx : ∀ i, rep (gaugeTorusGen i) x = ((expI : ℂ) ^ w.coord i) • x)
    (w' : GaugeWeight) :
    (spanSingleton hmul x w hx).piece w'
      = if w' = w then Submodule.span ℂ {x} else ⊥
```

def iSup [source]

An indexed join of decompositions. A family of decompositions indexed by a finite type decomposes the join, its pieces joined and its supports united one weight at a time. This is the arbitrary-arity form of `sup`.

```
@[implicit_reducible]
noncomputable def iSup {ι : Type*} [Fintype ι] {V : ι → Submodule ℂ B}
    (hmul : IsMulRep rep)
    (d : (a : ι) → GaugeWeightDecomposition rep (V a)) :
    GaugeWeightDecomposition rep (⨆ a, V a) where
```

lemma piece_iSup [source]

```
@[simp]
lemma piece_iSup {ι : Type*} [Fintype ι] {V : ι → Submodule ℂ B}
    (hmul : IsMulRep rep)
    (d : (a : ι) → GaugeWeightDecomposition rep (V a)) (w : GaugeWeight) :
    (iSup hmul d).piece w = ⨆ a, (d a).piece w
```

lemma iSup_supp [source]

The support of an indexed join is the union of the supports.

```
lemma iSup_supp {ι : Type*} [Fintype ι] {V : ι → Submodule ℂ B}
    (hmul : IsMulRep rep)
    (d : (a : ι) → GaugeWeightDecomposition rep (V a)) :
    (iSup hmul d).supp = Finset.univ.biUnion fun a => (d a).supp
```

def iSupProp [source]

A join over a proposition. `⨆ _ : p, V` is `V` when `p` holds and `⊥` otherwise, so it is decomposed by the given decomposition or by `bot`. The argument is a function of the proof, so the decomposition of `V` may itself depend on `p`.

```
@[implicit_reducible]
noncomputable def iSupProp {p : Prop} [Decidable p]
    (hmul : IsMulRep rep)
    (d : p → GaugeWeightDecomposition rep V) :
    GaugeWeightDecomposition rep (⨆ _ : p, V)
```

instance mul [source]

The product of two gauge weight decompositions, decomposing `V * V'`.

```
@[implicit_reducible]
noncomputable instance mul [d : GaugeWeightDecomposition rep V]
    [d' : GaugeWeightDecomposition rep V'] :
    GaugeWeightDecomposition rep (V * V') where
```

lemma mul_supp [source]

The support of a product is the pointwise sum of the supports.

```
lemma mul_supp [GaugeWeightDecomposition rep V] [GaugeWeightDecomposition rep V'] :
    supp rep (V * V') = supp rep V + supp rep V'
```

lemma mul_piece [source]

Weights add under multiplication. The weight-`w` piece of a product is spanned by the products of pieces whose weights sum to `w`.

```
lemma mul_piece [GaugeWeightDecomposition rep V] [GaugeWeightDecomposition rep V']
    (w : GaugeWeight) :
    piece rep (V * V') w
      = ⨆ w₁, ⨆ w₂, ⨆ _ : w₁ + w₂ = w, piece rep V w₁ * piece rep V' w₂
```

lemma mul_piece_eq_sub [source]

The product formula with the second weight eliminated against the support of the left factor, the right factor being read at the complement `w - w₁`.

```
lemma mul_piece_eq_sub [d : GaugeWeightDecomposition rep V]
    [d' : GaugeWeightDecomposition rep V'] (w : GaugeWeight) :
    piece rep (V * V') w = ⨆ w₁ ∈ supp rep V, piece rep V w₁ * piece rep V' (w - w₁)
```

lemma mul_piece_eq_sub' [source]

The mirror of `mul_piece_eq_sub`, joining over the weights of the right factor.

```
lemma mul_piece_eq_sub' [d : GaugeWeightDecomposition rep V]
    [d' : GaugeWeightDecomposition rep V'] (w : GaugeWeight) :
    piece rep (V * V') w = ⨆ w₂ ∈ supp rep V', piece rep V (w - w₂) * piece rep V' w₂
```

lemma mem_iSup_of_eigenvector [source]

The one-generator refinement step. A vector in the span of a family graded along a single operator, and an eigenvector of that operator at exponent `n`, lies in the span of just those pieces at exponent `n`.

```
lemma mem_iSup_of_eigenvector {ι : Type*} {T : Module.End ℂ B} {p : ι → Submodule ℂ B}
    {f : ι → ℤ} (hp : ∀ j, p j ≤ Module.End.eigenspace T ((expI : ℂ) ^ f j))
    {x : B} (hx : x ∈ ⨆ j, p j) {n : ℤ} (hT : T x = ((expI : ℂ) ^ n) • x) :
    x ∈ ⨆ j, ⨆ _ : f j = n, p j
```

lemma mem_iSup_of_forall_eigenvector [source]

The many-generator refinement. The same for a finite family of operators. A vector in the span of the family and an eigenvector of every operator lies in the span of just those pieces whose exponents match throughout.

```
lemma mem_iSup_of_forall_eigenvector {ι κ : Type*} [Fintype κ] [DecidableEq κ]
    {T : κ → Module.End ℂ B} {p : ι → Submodule ℂ B} {f : ι → κ → ℤ}
    (hp : ∀ j k, p j ≤ Module.End.eigenspace (T k) ((expI : ℂ) ^ f j k))
    {x : B} (hx : x ∈ ⨆ j, p j) {n : κ → ℤ}
    (hT : ∀ k, T k x = ((expI : ℂ) ^ n k) • x) :
    x ∈ ⨆ j, ⨆ _ : f j = n, p j
```

lemma piece_eq_inf [source]

The pieces are canonical. The weight-`w` piece is exactly the part of `V` on which the four torus generators act by the weight-`w` characters. See `piece_congr`.

```
lemma piece_eq_inf (d : GaugeWeightDecomposition rep V) (w : GaugeWeight) :
    d.piece w
      = V ⊓ ⨅ i, Module.End.eigenspace (rep (gaugeTorusGen i)) ((expI : ℂ) ^ w.coord i)
```

lemma piece_congr [source]

The pieces depend only on the submodule. Two decompositions of equal submodules have the same pieces, so a computation of `piece` may be carried along any equality of submodules.

```
lemma piece_congr {W : Submodule ℂ B} [d : GaugeWeightDecomposition rep V]
    [d' : GaugeWeightDecomposition rep W] (hVW : V = W) (w : GaugeWeight) :
    d.piece w = d'.piece w
```

lemma mem_zero_of_invariant [source]

A gauge-invariant element sits in the zero-weight piece. Only invariance under the four torus generators is used. The converse is false; see the warning in section F.

```
lemma mem_zero_of_invariant (d : GaugeWeightDecomposition rep V) {x : B} (hx : x ∈ V)
    (hV : ∀ g : GaugeGroupI, rep g x = x) : x ∈ d.piece 0
```

lemma reducesInvariantsTo_piece_zero [source]

The torus sieve as a reduction for the four torus generators: modulo a torus-stable submodule, their invariants in `V` lie in the weight-zero piece. Every other weight in the support is scaled by some generator by `(exp i) ^ n` with `n ≠ 0`, a scalar other than `1`, so its piece reduces to `⊥` by `reducesInvariantsTo_bot_of_apply_eq_smul`; each piece is carried into itself by the torus, and `ReducesInvariantsTo.iSup` joins the pieces.

```
lemma reducesInvariantsTo_piece_zero (dV : GaugeWeightDecomposition rep V) :
    ReducesInvariantsTo (fun i => rep (gaugeTorusGen i)) V (dV.piece 0)
```

lemma mem_piece_zero_sup_of_invariant [source]

An element of `V ⊔ S` fixed by the four torus generators, for `S` closed under them, lies in the weight-zero piece joined with `S`. This is `reducesInvariantsTo_piece_zero` unfolded, the form used to sieve a sector with `S` the part already understood. Only the torus is used, so gauge invariance of the element and gauge stability of `S` are more than is needed.

```
lemma mem_piece_zero_sup_of_invariant {S : Submodule ℂ B}
    (dV : GaugeWeightDecomposition rep V)
    (hS : ∀ (i : Fin 4) (y : B), y ∈ S → rep (gaugeTorusGen i) y ∈ S)
    {x : B} (hx : x ∈ V ⊔ S) (hinv : ∀ i : Fin 4, rep (gaugeTorusGen i) x = x) :
    x ∈ dV.piece 0 ⊔ S
```

2.447 PhyslibAlpha.Particles.StandardModel.GaugeGroup.Invariants.Adjoint

PhyslibAlpha/Particles/StandardModel/GaugeGroup/Invariants/Adjoint.lean on GitHub.

def su3LabelFun [source]

The labels of the Gell-Mann matrices of `su(3)` as generalized Gell-Mann labels: the symmetric and antisymmetric matrices of the planes `(0, 1)`, `(0, 2)` and `(1, 2)`, and the two diagonal matrices.

```
def su3LabelFun : Fin 8 → GellMann.Index 3
  | 0 => .inl ⟨(0, 1), by decide⟩
  | 1 => .inr (.inl ⟨(0, 1), by decide⟩)
  | 2 => .inr (.inr 0)
  | 3 => .inl ⟨(0, 2), by decide⟩
  | 4 => .inr (.inl ⟨(0, 2), by decide⟩)
  | 5 => .inl ⟨(1, 2), by decide⟩
  | 6 => .inr (.inl ⟨(1, 2), by decide⟩)
  | 7 => .inr (.inr 1)
```

def su3Label [source]

The relabelling of the Gell-Mann matrices of `su(3)` by generalized Gell-Mann labels.

```
noncomputable def su3Label : Fin 8 ≃ GellMann.Index 3
```

def su2LabelFun [source]

The labels of the Pauli matrices of `su(2)` as generalized Gell-Mann labels.

```
def su2LabelFun : Fin 3 → GellMann.Index 2
  | 0 => .inl ⟨(0, 1), by decide⟩
  | 1 => .inr (.inl ⟨(0, 1), by decide⟩)
  | 2 => .inr (.inr 0)
```

def su2Label [source]

The relabelling of the Pauli matrices of `su(2)` by generalized Gell-Mann labels.

```
noncomputable def su2Label : Fin 3 ≃ GellMann.Index 2
```

lemma su3Label_apply [source]

```
@[simp]
lemma su3Label_apply (a : Fin 8) : su3Label a = su3LabelFun a
```

lemma su2Label_apply [source]

```
@[simp]
lemma su2Label_apply (i : Fin 3) : su2Label i = su2LabelFun i
```

lemma gellMannMatrix_eq_matrix [source]

The Gell-Mann matrices of `su(3)` are the generalized Gell-Mann matrices.

```
lemma gellMannMatrix_eq_matrix (a : Fin 8) :
    gellMannMatrix a = GellMann.matrix (su3Label a)
```

lemma pauliMatrix_inr_eq_matrix [source]

The Pauli matrices are the generalized Gell-Mann matrices of `su(2)`.

```
lemma pauliMatrix_inr_eq_matrix (i : Fin 3) :
    pauliMatrix (Sum.inr i) = GellMann.matrix (su2Label i)
```

lemma ofReal_half_re_trace [source]

A complex number fixed by conjugation is the image of its real part.

```
lemma ofReal_half_re_trace {z : ℂ} (hz : star (z / 2) = z / 2) :
    ((2⁻¹ * z.re : ℝ) : ℂ) = z / 2
```

lemma su3AdjointMatrix_eq_adjMatrix [source]

The adjoint matrix of `SU(3)` is the matrix of the adjoint action in the Gell-Mann basis.

```
lemma su3AdjointMatrix_eq_adjMatrix (U : SU 3) (a b : Fin 8) :
    ((su3AdjointMatrix U a b : ℝ) : ℂ) = adjMatrix U (su3Label a) (su3Label b)
```

lemma su2AdjointMatrix_eq_adjMatrix [source]

The adjoint matrix of `SU(2)` is the matrix of the adjoint action in the Pauli basis.

```
lemma su2AdjointMatrix_eq_adjMatrix (U : SU 2) (i j : Fin 3) :
    ((su2AdjointMatrix U i j : ℝ) : ℂ) = adjMatrix U (su2Label i) (su2Label j)
```

lemma adjMap_smul_of_relabel_law [source]

A linear map moving a family with one adjoint label by the Standard Model adjoint matrix of `U` intertwines the map of the relabelled family with the action of `U`.

```
lemma adjMap_smul_of_relabel_law (T : ι → B) {f : B →ₗ[ℂ] B} {U : SU N}
    (hf : ∀ l, f (T l) = ∑ a, ((M U a l : ℝ) : ℂ) • T a) (t : SuT[N, .adj]) :
    f (adjMap (T ∘ e.symm) t) = adjMap (T ∘ e.symm) (U • t)
```

lemma adjPairMap_smul_of_relabel_law [source]

A linear map moving a family with two adjoint labels by the Standard Model adjoint matrix of `U` on each label intertwines the map of the relabelled family with the action of `U`.

```
lemma adjPairMap_smul_of_relabel_law (T : (Fin 2 → ι) → B) {f : B →ₗ[ℂ] B} {U : SU N}
    (hf : ∀ l, f (T l) = ∑ a : Fin 2 → ι, (∏ i : Fin 2, ((M U (a i) (l i) : ℝ) : ℂ)) • T a)
    (t : SuT[N, .adj, .adj]) :
    f (adjPairMap (fun n => T (e.symm ∘ n)) t) = adjPairMap (fun n => T (e.symm ∘ n)) (U • t)
```

lemma adjPairMap_relabel_two_smul_adjUnitTensor [source]

The map of the relabelled family sends twice the unit tensor to the trace contraction.

```
lemma adjPairMap_relabel_two_smul_adjUnitTensor (T : (Fin 2 → ι) → B) :
    adjPairMap (fun n => T (e.symm ∘ n)) ((2 : ℂ) • adjUnitTensor N) = ∑ a : ι, T ![a, a]
```

def IsSU3AdjointMat [source]

The linear map `f` moves the components of `T` as `U ∈ SU(3)` moves a tensor with one adjoint index: one factor of the adjoint matrix, the summed index first.

```
def IsSU3AdjointMat (U : SU 3) (f : B →ₗ[ℂ] B) (T : Fin 8 → B) : Prop
```

abbrev IsSU3Adjoint [source]

A family `T^a` with one `su(3)` adjoint index: the map of the relabelled family out of the tensors of `SU(3)` is equivariant for the colour factor of the gauge group.

```
abbrev IsSU3Adjoint (B : Type*) [AddCommGroup B] [Module ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B) (T : Fin 8 → B) : Prop
```

def IsSU3BiAdjointMat [source]

The linear map `f` moves the components of `T` as `U ∈ SU(3)` moves a tensor with two adjoint indices: one factor of the adjoint matrix per index, the summed index first.

```
def IsSU3BiAdjointMat (U : SU 3) (f : B →ₗ[ℂ] B) (T : (Fin 2 → Fin 8) → B) : Prop
```

abbrev IsSU3BiAdjoint [source]

A family `T^{ab}` with two `su(3)` adjoint indices: the map of the relabelled family out of the tensors of `SU(3)` is equivariant for the colour factor of the gauge group.

```
abbrev IsSU3BiAdjoint (B : Type*) [AddCommGroup B] [Module ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B) (T : (Fin 2 → Fin 8) → B) : Prop
```

lemma IsSU3Adjoint.of_law [source]

A family obeying the law for every colour rotation is such a family.

```
lemma of_law (hT : ∀ U : SU 3, IsSU3AdjointMat U (repGauge (U, 1, 1)) T) :
    IsSU3Adjoint B repGauge T
```

lemma IsSU3Adjoint.isStableUnder_span [source]

The span of the components is stable under the colour factor.

```
lemma isStableUnder_span (hT : IsSU3Adjoint B repGauge T) :
    IsStableUnder (fun U : SU 3 => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T))
```

lemma IsSU3Adjoint.reducesInvariantsTo_bot [source]

A colour invariant of the span of the components joined with a stable submodule lies in that submodule: an adjoint index contributes nothing to the invariants.

```
lemma reducesInvariantsTo_bot (hT : IsSU3Adjoint B repGauge T) :
    ReducesInvariantsTo (fun U : SU 3 => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T)) ⊥
```

lemma IsSU3BiAdjoint.of_law [source]

A family obeying the law for every colour rotation is such a family.

```
lemma of_law (hT : ∀ U : SU 3, IsSU3BiAdjointMat U (repGauge (U, 1, 1)) T) :
    IsSU3BiAdjoint B repGauge T
```

def IsSU3BiAdjoint.traceContraction [source]

The trace contraction `∑ a, T ![a, a]` of the two adjoint indices.

```
def traceContraction (T : (Fin 2 → Fin 8) → B) : B
```

lemma IsSU3BiAdjoint.traceContraction_mem_span [source]

```
lemma traceContraction_mem_span (T : (Fin 2 → Fin 8) → B) :
    traceContraction T ∈ Submodule.span ℂ (Set.range T)
```

lemma IsSU3BiAdjoint.map_traceContraction [source]

Any map moving the components by the adjoint matrix of an element of `SU(3)` fixes the trace contraction.

```
lemma map_traceContraction {U : SU 3} {f : B →ₗ[ℂ] B} (hf : IsSU3BiAdjointMat U f T) :
    f (traceContraction T) = traceContraction T
```

lemma IsSU3BiAdjoint.range_adjPairMap [source]

The range of the map of the relabelled family is the span of the components.

```
lemma range_adjPairMap (T : (Fin 2 → Fin 8) → B) :
    LinearMap.range (adjPairMap fun n => T (su3Label.symm ∘ n))
      = Submodule.span ℂ (Set.range T)
```

lemma IsSU3BiAdjoint.isStableUnder_span [source]

The span of the components is stable under the colour factor.

```
lemma isStableUnder_span (hT : IsSU3BiAdjoint B repGauge T) :
    IsStableUnder (fun U : SU 3 => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T))
```

lemma IsSU3BiAdjoint.reducesInvariantsTo_span_traceContraction [source]

The colour invariants of the span of the components reduce to the line through the trace contraction.

```
lemma reducesInvariantsTo_span_traceContraction (hT : IsSU3BiAdjoint B repGauge T) :
    ReducesInvariantsTo (fun U : SU 3 => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T)) (ℂ • traceContraction T)
```

def IsSU2AdjointMat [source]

The linear map `f` moves the components of `T` as `U ∈ SU(2)` moves a tensor with one adjoint index: one factor of the adjoint matrix, the summed index first.

```
def IsSU2AdjointMat (U : SU 2) (f : B →ₗ[ℂ] B) (T : Fin 3 → B) : Prop
```

abbrev IsSU2Adjoint [source]

A family `T^a` with one `su(2)` adjoint index: the map of the relabelled family out of the tensors of `SU(2)` is equivariant for the isospin factor of the gauge group.

```
abbrev IsSU2Adjoint (B : Type*) [AddCommGroup B] [Module ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B) (T : Fin 3 → B) : Prop
```

def IsSU2BiAdjointMat [source]

The linear map `f` moves the components of `T` as `U ∈ SU(2)` moves a tensor with two adjoint indices: one factor of the adjoint matrix per index, the summed index first.

```
def IsSU2BiAdjointMat (U : SU 2) (f : B →ₗ[ℂ] B) (T : (Fin 2 → Fin 3) → B) : Prop
```

abbrev IsSU2BiAdjoint [source]

A family `T^{ab}` with two `su(2)` adjoint indices: the map of the relabelled family out of the tensors of `SU(2)` is equivariant for the isospin factor of the gauge group.

```
abbrev IsSU2BiAdjoint (B : Type*) [AddCommGroup B] [Module ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B) (T : (Fin 2 → Fin 3) → B) : Prop
```

lemma IsSU2Adjoint.of_law [source]

A family obeying the law for every isospin rotation is such a family.

```
lemma of_law (hT : ∀ U : SU 2, IsSU2AdjointMat U (repGauge (1, U, 1)) T) :
    IsSU2Adjoint B repGauge T
```

lemma IsSU2Adjoint.isStableUnder_span [source]

The span of the components is stable under the isospin factor.

```
lemma isStableUnder_span (hT : IsSU2Adjoint B repGauge T) :
    IsStableUnder (fun U : SU 2 => repGauge ((1, U, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T))
```

lemma IsSU2Adjoint.reducesInvariantsTo_bot [source]

A isospin invariant of the span of the components joined with a stable submodule lies in that submodule: an adjoint index contributes nothing to the invariants.

```
lemma reducesInvariantsTo_bot (hT : IsSU2Adjoint B repGauge T) :
    ReducesInvariantsTo (fun U : SU 2 => repGauge ((1, U, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T)) ⊥
```

lemma IsSU2BiAdjoint.of_law [source]

A family obeying the law for every isospin rotation is such a family.

```
lemma of_law (hT : ∀ U : SU 2, IsSU2BiAdjointMat U (repGauge (1, U, 1)) T) :
    IsSU2BiAdjoint B repGauge T
```

def IsSU2BiAdjoint.traceContraction [source]

The trace contraction `∑ a, T ![a, a]` of the two adjoint indices.

```
def traceContraction (T : (Fin 2 → Fin 3) → B) : B
```

lemma IsSU2BiAdjoint.traceContraction_mem_span [source]

```
lemma traceContraction_mem_span (T : (Fin 2 → Fin 3) → B) :
    traceContraction T ∈ Submodule.span ℂ (Set.range T)
```

lemma IsSU2BiAdjoint.map_traceContraction [source]

Any map moving the components by the adjoint matrix of an element of `SU(2)` fixes the trace contraction.

```
lemma map_traceContraction {U : SU 2} {f : B →ₗ[ℂ] B} (hf : IsSU2BiAdjointMat U f T) :
    f (traceContraction T) = traceContraction T
```

lemma IsSU2BiAdjoint.range_adjPairMap [source]

The range of the map of the relabelled family is the span of the components.

```
lemma range_adjPairMap (T : (Fin 2 → Fin 3) → B) :
    LinearMap.range (adjPairMap fun n => T (su2Label.symm ∘ n))
      = Submodule.span ℂ (Set.range T)
```

lemma IsSU2BiAdjoint.isStableUnder_span [source]

The span of the components is stable under the isospin factor.

```
lemma isStableUnder_span (hT : IsSU2BiAdjoint B repGauge T) :
    IsStableUnder (fun U : SU 2 => repGauge ((1, U, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T))
```

lemma IsSU2BiAdjoint.reducesInvariantsTo_span_traceContraction [source]

The isospin invariants of the span of the components reduce to the line through the trace contraction.

```
lemma reducesInvariantsTo_span_traceContraction (hT : IsSU2BiAdjoint B repGauge T) :
    ReducesInvariantsTo (fun U : SU 2 => repGauge ((1, U, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T)) (ℂ • traceContraction T)
```

2.448 PhyslibAlpha.Particles.StandardModel.GaugeGroup.Invariants.Basic

PhyslibAlpha/Particles/StandardModel/GaugeGroup/Invariants/Basic.lean on GitHub.

def ofSU3 [source]

The colour factor `SU(3)` of the gauge group.

```
def GaugeGroupI.ofSU3 : SU 3 →* GaugeGroupI
```

def ofSU2 [source]

The isospin factor `SU(2)` of the gauge group.

```
def GaugeGroupI.ofSU2 : SU 2 →* GaugeGroupI
```

lemma ofSU3_apply [source]

```
@[simp]
lemma GaugeGroupI.ofSU3_apply (U : SU 3) : GaugeGroupI.ofSU3 U = (U, 1, 1)
```

lemma ofSU2_apply [source]

```
@[simp]
lemma GaugeGroupI.ofSU2_apply (V : SU 2) : GaugeGroupI.ofSU2 V = (1, V, 1)
```

lemma sum_pi_two [source]

A sum over pairs of indices is a double sum.

```
lemma sum_pi_two {n : ℕ} {M : Type*} [AddCommMonoid M] (F : (Fin 2 → Fin n) → M) :
    ∑ d : Fin 2 → Fin n, F d = ∑ x : Fin n, ∑ y : Fin n, F ![x, y]
```

def IsSU3FundamentalAntiFundamentalMat [source]

The linear map `f` moves the components of `T` as `U ∈ SU(3)` moves a tensor with one fundamental and one anti-fundamental index: a factor of `U` for the first index and a factor of `conj U` for the second, the summed index first.

```
def IsSU3FundamentalAntiFundamentalMat (U : SU 3) (f : B →ₗ[ℂ] B) (T : (Fin 2 → Fin 3) → B) :
    Prop
```

abbrev IsSU3FundamentalAntiFundamental [source]

A family `T^a{}_b` with one fundamental and one anti-fundamental colour index: its map out of the tensors of `SU(3)` is equivariant for the colour factor of the gauge group.

```
abbrev IsSU3FundamentalAntiFundamental (B : Type*) [AddCommGroup B] [Module ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B) (T : (Fin 2 → Fin 3) → B) : Prop
```

def IsSU2FundamentalAntiFundamentalMat [source]

The linear map `f` moves the components of `T` as `V ∈ SU(2)` moves a tensor with one fundamental and one anti-fundamental index.

```
def IsSU2FundamentalAntiFundamentalMat (V : SU 2) (f : B →ₗ[ℂ] B) (T : (Fin 2 → Fin 2) → B) :
    Prop
```

abbrev IsSU2FundamentalAntiFundamental [source]

A family `T^a{}_b` with one fundamental and one anti-fundamental isospin index: its map out of the tensors of `SU(2)` is equivariant for the isospin factor of the gauge group.

```
abbrev IsSU2FundamentalAntiFundamental (B : Type*) [AddCommGroup B] [Module ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B) (T : (Fin 2 → Fin 2) → B) : Prop
```

lemma IsSU3FundamentalAntiFundamental.of_law [source]

A family obeying the law for every colour rotation is such a family.

```
lemma of_law (hT : ∀ U : SU 3, IsSU3FundamentalAntiFundamentalMat U (repGauge (U, 1, 1)) T) :
    IsSU3FundamentalAntiFundamental B repGauge T
```

lemma IsSU3FundamentalAntiFundamental.sum [source]

A finite sum of such families is such a family.

```
lemma sum {ι : Type} [Fintype ι] {T : ι → (Fin 2 → Fin 3) → B}
    (hT : ∀ i, IsSU3FundamentalAntiFundamental B repGauge (T i)) :
    IsSU3FundamentalAntiFundamental B repGauge (fun l => ∑ i, T i l)
```

def IsSU3FundamentalAntiFundamental.deltaContraction [source]

The delta contraction `∑ a, T ![a, a]`.

```
def deltaContraction (T : (Fin 2 → Fin 3) → B) : B
```

lemma IsSU3FundamentalAntiFundamental.deltaContraction_mem_span [source]

```
lemma deltaContraction_mem_span (T : (Fin 2 → Fin 3) → B) :
    deltaContraction T ∈ Submodule.span ℂ (Set.range T)
```

lemma IsSU3FundamentalAntiFundamental.map_deltaContraction [source]

Any map moving the components by an element of `SU(3)` fixes the delta contraction.

```
lemma map_deltaContraction {U : SU 3} {f : B →ₗ[ℂ] B}
    (hf : IsSU3FundamentalAntiFundamentalMat U f T) :
    f (deltaContraction T) = deltaContraction T
```

lemma IsSU3FundamentalAntiFundamental.repGauge_deltaContraction [source]

The delta contraction is colour invariant.

```
lemma repGauge_deltaContraction (hT : IsSU3FundamentalAntiFundamental B repGauge T)
    (U : SU 3) : repGauge (U, 1, 1) (deltaContraction T) = deltaContraction T
```

def IsSU3FundamentalAntiFundamental.invariantReductionToSpan [source]

The colour invariants of the component span reduce to the span of the delta contraction.

```
noncomputable def invariantReductionToSpan (hT : IsSU3FundamentalAntiFundamental B repGauge T)
    :
    InvariantReductionToSpan (fun U : SU 3 => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T))
```

lemma IsSU2FundamentalAntiFundamental.of_law [source]

A family obeying the law for every isospin rotation is such a family.

```
lemma of_law (hT : ∀ V : SU 2, IsSU2FundamentalAntiFundamentalMat V (repGauge (1, V, 1)) T) :
    IsSU2FundamentalAntiFundamental B repGauge T
```

lemma IsSU2FundamentalAntiFundamental.sum [source]

A finite sum of such families is such a family.

```
lemma sum {ι : Type} [Fintype ι] {T : ι → (Fin 2 → Fin 2) → B}
    (hT : ∀ i, IsSU2FundamentalAntiFundamental B repGauge (T i)) :
    IsSU2FundamentalAntiFundamental B repGauge (fun l => ∑ i, T i l)
```

def IsSU2FundamentalAntiFundamental.deltaContraction [source]

The delta contraction `T ![0, 0] + T ![1, 1]`.

```
def deltaContraction (T : (Fin 2 → Fin 2) → B) : B
```

lemma deltaContraction_eq_sum [source]

```
lemma deltaContraction_eq_sum (T : (Fin 2 → Fin 2) → B) :
    deltaContraction T = ∑ a : Fin 2, T ![a, a]
```

lemma IsSU2FundamentalAntiFundamental.deltaContraction_mem_span [source]

```
lemma deltaContraction_mem_span (T : (Fin 2 → Fin 2) → B) :
    deltaContraction T ∈ Submodule.span ℂ (Set.range T)
```

lemma IsSU2FundamentalAntiFundamental.map_deltaContraction [source]

Any map moving the components by an element of `SU(2)` fixes the delta contraction.

```
lemma map_deltaContraction {V : SU 2} {f : B →ₗ[ℂ] B}
    (hf : IsSU2FundamentalAntiFundamentalMat V f T) :
    f (deltaContraction T) = deltaContraction T
```

lemma IsSU2FundamentalAntiFundamental.repGauge_deltaContraction [source]

The delta contraction is isospin invariant.

```
lemma repGauge_deltaContraction (hT : IsSU2FundamentalAntiFundamental B repGauge T)
    (V : SU 2) : repGauge (1, V, 1) (deltaContraction T) = deltaContraction T
```

def IsSU2FundamentalAntiFundamental.invariantReductionToSpan [source]

The isospin invariants of the component span reduce to the span of the delta contraction.

```
noncomputable def invariantReductionToSpan (hT : IsSU2FundamentalAntiFundamental B repGauge T)
    :
    InvariantReductionToSpan (fun V : SU 2 => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T))
```

lemma reducesInvariantsTo_span_deltaContraction [source]

The isospin invariants of the component span reduce to the line through the delta contraction.

```
lemma reducesInvariantsTo_span_deltaContraction
    (hT : IsSU2FundamentalAntiFundamental B repGauge T) :
    ReducesInvariantsTo (fun V : SU 2 => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T)) (ℂ • deltaContraction T)
```

def IsSU2BiFundamentalMat [source]

The linear map `f` moves the components of `T` as `V ∈ SU(2)` moves a tensor with two fundamental indices: one factor of `V` per index.

```
def IsSU2BiFundamentalMat (V : SU 2) (f : B →ₗ[ℂ] B) (T : (Fin 2 → Fin 2) → B) : Prop
```

def IsSU2BiAntiFundamentalMat [source]

The linear map `f` moves the components of `T` as `V ∈ SU(2)` moves a tensor with two anti-fundamental indices: one factor of `conj V` per index.

```
def IsSU2BiAntiFundamentalMat (V : SU 2) (f : B →ₗ[ℂ] B) (T : (Fin 2 → Fin 2) → B) : Prop
```

abbrev IsSU2BiFundamental [source]

A family `T^{ab}` with two fundamental isospin indices: its map out of the tensors of `SU(2)` is equivariant for the isospin factor of the gauge group.

```
abbrev IsSU2BiFundamental (B : Type*) [AddCommGroup B] [Module ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B) (T : (Fin 2 → Fin 2) → B) : Prop
```

abbrev IsSU2BiAntiFundamental [source]

A family `T_{ab}` with two anti-fundamental isospin indices: its map out of the tensors of `SU(2)` is equivariant for the isospin factor of the gauge group.

```
abbrev IsSU2BiAntiFundamental (B : Type*) [AddCommGroup B] [Module ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B) (T : (Fin 2 → Fin 2) → B) : Prop
```

lemma IsSU2BiFundamental.of_law [source]

A family obeying the law for every isospin rotation is such a family.

```
lemma of_law (hT : ∀ V : SU 2, IsSU2BiFundamentalMat V (repGauge (1, V, 1)) T) :
    IsSU2BiFundamental B repGauge T
```

lemma IsSU2BiFundamental.sum [source]

A finite sum of such families is such a family.

```
lemma sum {ι : Type} [Fintype ι] {T : ι → (Fin 2 → Fin 2) → B}
    (hT : ∀ i, IsSU2BiFundamental B repGauge (T i)) :
    IsSU2BiFundamental B repGauge (fun l => ∑ i, T i l)
```

def epsilonContraction [source]

The epsilon contraction `T ![0, 1] - T ![1, 0]`.

```
def epsilonContraction (T : (Fin 2 → Fin 2) → B) : B
```

lemma epsilonContraction_mem_span [source]

```
lemma epsilonContraction_mem_span (T : (Fin 2 → Fin 2) → B) :
    epsilonContraction T ∈ Submodule.span ℂ (Set.range T)
```

lemma IsSU2BiFundamental.map_epsilonContraction [source]

Any map moving the components by an element of SU(2) fixes the epsilon contraction.

```
lemma map_epsilonContraction {V : SU 2} {f : B →ₗ[ℂ] B} (hf : IsSU2BiFundamentalMat V f T) :
    f (epsilonContraction T) = epsilonContraction T
```

lemma IsSU2BiFundamental.repGauge_epsilonContraction [source]

The epsilon contraction is isospin invariant.

```
lemma repGauge_epsilonContraction (hT : IsSU2BiFundamental B repGauge T) (V : SU 2) :
    repGauge (1, V, 1) (epsilonContraction T) = epsilonContraction T
```

def IsSU2BiFundamental.invariantReductionToSpan [source]

The isospin invariants of the component span reduce to the span of the epsilon contraction.

```
noncomputable def invariantReductionToSpan (hT : IsSU2BiFundamental B repGauge T) :
    InvariantReductionToSpan (fun V : SU 2 => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T))
```

lemma IsSU2BiAntiFundamental.of_law [source]

A family obeying the law for every isospin rotation is such a family.

```
lemma of_law (hT : ∀ V : SU 2, IsSU2BiAntiFundamentalMat V (repGauge (1, V, 1)) T) :
    IsSU2BiAntiFundamental B repGauge T
```

lemma IsSU2BiAntiFundamental.sum [source]

A finite sum of such families is such a family.

```
lemma sum {ι : Type} [Fintype ι] {T : ι → (Fin 2 → Fin 2) → B}
    (hT : ∀ i, IsSU2BiAntiFundamental B repGauge (T i)) :
    IsSU2BiAntiFundamental B repGauge (fun l => ∑ i, T i l)
```

lemma IsSU2BiAntiFundamental.map_epsilonContraction [source]

Any map moving the components by an element of SU(2) in the anti-fundamental fixes the epsilon contraction.

```
lemma map_epsilonContraction {V : SU 2} {f : B →ₗ[ℂ] B} (hf : IsSU2BiAntiFundamentalMat V f T) :

    f (IsSU2BiFundamental.epsilonContraction T) = IsSU2BiFundamental.epsilonContraction T
```

lemma IsSU2BiAntiFundamental.repGauge_epsilonContraction [source]

The epsilon contraction is isospin invariant.

```
lemma repGauge_epsilonContraction (hT : IsSU2BiAntiFundamental B repGauge T) (V : SU 2) :
    repGauge (1, V, 1) (IsSU2BiFundamental.epsilonContraction T)
      = IsSU2BiFundamental.epsilonContraction T
```

def IsSU2BiAntiFundamental.invariantReductionToSpan [source]

The isospin invariants of the component span reduce to the span of the epsilon contraction.

```
noncomputable def invariantReductionToSpan (hT : IsSU2BiAntiFundamental B repGauge T) :
    InvariantReductionToSpan (fun V : SU 2 => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T))
```

def IsSU2QuadFundamentalMat [source]

The linear map `f` moves the components of `T` as `V ∈ SU(2)` moves a tensor with four fundamental indices: one factor of `V` per index.

```
def IsSU2QuadFundamentalMat (V : SU 2) (f : B →ₗ[ℂ] B) (T : (Fin 4 → Fin 2) → B) : Prop
```

abbrev IsSU2QuadFundamental [source]

A family `T^{abcd}` with four fundamental isospin indices: its map out of the tensors of `SU(2)` is equivariant for the isospin factor of the gauge group.

```
abbrev IsSU2QuadFundamental (B : Type*) [AddCommGroup B] [Module ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B) (T : (Fin 4 → Fin 2) → B) : Prop
```

lemma IsSU2QuadFundamental.of_law [source]

A family obeying the law for every isospin rotation is such a family.

```
lemma of_law (hT : ∀ V : SU 2, IsSU2QuadFundamentalMat V (repGauge (1, V, 1)) T) :
    IsSU2QuadFundamental B repGauge T
```

lemma sum_pi_four [source]

A sum over families of four fundamental indices is a fourfold sum.

```
lemma sum_pi_four {M : Type*} [AddCommMonoid M] (F : (Fin 4 → Fin 2) → M) :
    ∑ d : Fin 4 → Fin 2, F d
      = ∑ x : Fin 2, ∑ y : Fin 2, ∑ z : Fin 2, ∑ w : Fin 2, F ![x, y, z, w]
```

def epsilonContraction₁₂ [source]

The contraction pairing the first index with the second and the third with the fourth.

```
def epsilonContraction₁₂ (T : (Fin 4 → Fin 2) → B) : B
```

def epsilonContraction₁₃ [source]

The contraction pairing the first index with the third and the second with the fourth.

```
def epsilonContraction₁₃ (T : (Fin 4 → Fin 2) → B) : B
```

lemma reducesInvariantsTo_span_epsilonContractions [source]

The isospin invariants of the component span reduce to the span of the two epsilon contractions.

```
lemma reducesInvariantsTo_span_epsilonContractions (hT : IsSU2QuadFundamental B repGauge T) :
    ReducesInvariantsTo (fun V : SU 2 => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range T))
      (Submodule.span ℂ {epsilonContraction₁₂ T, epsilonContraction₁₃ T})
```

2.449 PhyslibAlpha.Particles.StandardModel.GaugeGroup.Invariants.IsU1BiAdjoint

PhyslibAlpha/Particles/StandardModel/GaugeGroup/Invariants/IsU1BiAdjoint.lean on GitHub.

def u1AdjointMatrix [source]

The adjoint matrix of an element of `U(1)`: the one by one matrix `1`, the `u(1)` factor being abelian and so acting trivially on its own algebra.

```
def u1AdjointMatrix (_u : unitary ℂ) : Matrix (Fin 1) (Fin 1) ℝ
```

lemma u1AdjointMatrix_apply [source]

The single entry of the adjoint matrix of an element of `U(1)` is `1`.

```
@[simp]
lemma u1AdjointMatrix_apply (u : unitary ℂ) (i j : Fin 1) :
    u1AdjointMatrix u i j = 1
```

def IsU1BiAdjointMat [source]

The linear map `f` moves the components of `T` as `u ∈ U(1)` moves a tensor with two adjoint indices: one factor of `u1AdjointMatrix u` per index, with the summed index in the row slot.

```
def IsU1BiAdjointMat {B : Type*} [AddCommMonoid B] [Module ℂ B]
    (u : unitary ℂ) (f : B →ₗ[ℂ] B)
    (T : (Fin 2 → Fin 1) → B) : Prop
```

lemma isU1BiAdjointMat_iff [source]

The `u(1)` transformation law says exactly that the map fixes every component: the adjoint matrix is `1`, and there is a single family of two `u(1)` indices to sum over.

```
lemma isU1BiAdjointMat_iff {B : Type*} [AddCommMonoid B] [Module ℂ B]
    (u : unitary ℂ) (f : B →ₗ[ℂ] B) (T : (Fin 2 → Fin 1) → B) :
    IsU1BiAdjointMat u f T ↔ ∀ l : Fin 2 → Fin 1, f (T l) = T l
```

lemma map_T [source]

A linear map obeying the `u(1)` transformation law fixes every component.

```
lemma IsU1BiAdjointMat.map_T {B : Type*} [AddCommMonoid B] [Module ℂ B] {u : unitary ℂ}
    {f : B →ₗ[ℂ] B} {T : (Fin 2 → Fin 1) → B} (hf : IsU1BiAdjointMat u f T)
    (l : Fin 2 → Fin 1) : f (T l) = T l
```

structure IsU1BiAdjoint [source]

A family `T` of elements of `B`, indexed by two `u(1)` adjoint indices, transforms as a tensor `T^{a b}` under the hypercharge factor of the gauge group. Nothing is asked of the colour and isospin factors.

```
structure IsU1BiAdjoint (B : Type*) [AddCommMonoid B] [Module ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B)
    (T : (Fin 2 → Fin 1) → B) : Prop where
  repGauge_T : ∀ g : unitary ℂ, IsU1BiAdjointMat g (repGauge (1, 1, g)) T
```

def traceContraction [source]

The trace contraction: the Kronecker contraction of the two `u(1)` indices, which is the one component of the family.

```
def traceContraction (T : (Fin 2 → Fin 1) → B) : B
```

lemma map_traceContraction [source]

Any map obeying the `u(1)` law fixes the trace contraction.

```
lemma map_traceContraction {u : unitary ℂ} {f : B →ₗ[ℂ] B} (hf : IsU1BiAdjointMat u f T) :
    f (traceContraction T) = traceContraction T
```

2.450 PhyslibAlpha.Particles.StandardModel.GaugeGroup.JetGaugeGroup.Basic

PhyslibAlpha/Particles/StandardModel/GaugeGroup/JetGaugeGroup/Basic.lean on GitHub.

abbrev JetGaugeGroupI [source]

The group of formal infinite-order jets, at a spacetime point, of local gauge transformations of the Standard Model: the `R`-points of the gauge group for `R` the ring `SpaceTimeAlgebra` of formal power series in the spacetime coordinates.

Since gauge transformations multiply pointwise, jets multiply as power series and the group structure is that of the matrix groups over `SpaceTimeAlgebra`. The unitarity and determinant constraints hold as power-series identities, i.e. at every jet order.

Evaluation at the base point recovers `GaugeGroupI`; see `JetGaugeGroupI.eval`.

```
abbrev JetGaugeGroupI : Type
```

def toVal [source]

The underlying matrix value of an element of `JetGaugeGroupI`.

```
def toVal (U : JetGaugeGroupI) : Matrix (Fin 3) (Fin 3) SpaceTimeAlgebra × Matrix (Fin 2)
    (Fin 2) SpaceTimeAlgebra × SpaceTimeAlgebra
```

lemma mapMatrix_constantCoeff_star [source]

Entrywise evaluation at the base point commutes with the conjugate transpose.

```
lemma mapMatrix_constantCoeff_star {n : Type} [Fintype n] [DecidableEq n]
    (A : Matrix n n SpaceTimeAlgebra) :
    (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix (star A) =
      star ((constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix A)
```

def evalSU [source]

Evaluation of a jet of a special-unitary gauge transformation at the base point: the entrywise constant coefficient.

```
noncomputable def evalSU (n : Type) [Fintype n] [DecidableEq n] :
    specialUnitaryGroup n SpaceTimeAlgebra →* specialUnitaryGroup n ℂ where
```

def evalU1 [source]

Evaluation of a jet of a `U(1)` gauge transformation at the base point: the constant coefficient.

```
noncomputable def evalU1 : unitary SpaceTimeAlgebra →* unitary ℂ where
```

def eval [source]

Evaluation of a jet of a gauge transformation at the base point, projecting the jet gauge group onto the gauge group `GaugeGroupI` by taking zeroth-order parts on each factor.

```
noncomputable def eval : JetGaugeGroupI →* GaugeGroupI
```

def deriv [source]

The derivative of an element of `JetGaugeGroupI` returning a product of matrices.

```
noncomputable def deriv (μ : Fin 1 ⊕ Fin 3) (U : JetGaugeGroupI) :
    Matrix (Fin 3) (Fin 3) SpaceTimeAlgebra × Matrix (Fin 2)
        (Fin 2) SpaceTimeAlgebra × SpaceTimeAlgebra
```

lemma deriv_mul [source]

```
lemma deriv_mul (μ : Fin 1 ⊕ Fin 3) (U V : JetGaugeGroupI) :
    deriv μ (U * V) = deriv μ U * V.toVal + U.toVal * deriv μ V
```

lemma deriv_one [source]

```
@[simp]
lemma deriv_one (μ : Fin 1 ⊕ Fin 3) : deriv μ (1 : JetGaugeGroupI) = 0
```

lemma star_deriv

[source]

```
lemma star_deriv (μ : Fin 1 ⊕ Fin 3) (U : JetGaugeGroupI) :
    star (deriv μ U) = deriv μ (star U)
```

lemma deriv_mul_inv_toVal_SU3_traceless

[source]

```
lemma deriv_mul_inv_toVal_SU3_traceless (μ : Fin 1 ⊕ Fin 3) (U : JetGaugeGroupI) :
    (Complex.I • (deriv μ U * (U⁻¹).toVal)).1.trace = 0
```

lemma deriv_mul_inv_toVal_SU2_traceless

[source]

```
lemma deriv_mul_inv_toVal_SU2_traceless (μ : Fin 1 ⊕ Fin 3) (U : JetGaugeGroupI) :
    (Complex.I • (deriv μ U * (U⁻¹).toVal)).2.1.trace = 0
```

lemma star_deriv_mul_inv_toVal_SU3

[source]

```
lemma star_deriv_mul_inv_toVal_SU3 (μ : Fin 1 ⊕ Fin 3) (U : JetGaugeGroupI) :
    star ((Complex.I • (deriv μ U * (U⁻¹).toVal)).1) =
    (Complex.I • (deriv μ U * (U⁻¹).toVal)).1
```

lemma star_deriv_mul_inv_toVal_SU2

[source]

```
lemma star_deriv_mul_inv_toVal_SU2 (μ : Fin 1 ⊕ Fin 3) (U : JetGaugeGroupI) :
    star ((Complex.I • (deriv μ U * (U⁻¹).toVal)).2.1) =
    (Complex.I • (deriv μ U * (U⁻¹).toVal)).2.1
```

lemma star_deriv_mul_inv_toVal_U1

[source]

```
lemma star_deriv_mul_inv_toVal_U1 (μ : Fin 1 ⊕ Fin 3) (U : JetGaugeGroupI) :
    star ((Complex.I • (deriv μ U * (U⁻¹).toVal)).2.2) =
    (Complex.I • (deriv μ U * (U⁻¹).toVal)).2.2
```

lemma mapMatrix_C_star

[source]

Entrywise inclusion of constants commutes with the conjugate transpose.

```
lemma mapMatrix_C_star {n : Type} [Fintype n] [DecidableEq n] (A : Matrix n n ℂ) :
    (C : ℂ →+* SpaceTimeAlgebra).mapMatrix (star A) = star
        ((C : ℂ →+* SpaceTimeAlgebra).mapMatrix A)
```

def ofConstantSU

[source]

The jet of a constant special-unitary gauge transformation: the entrywise inclusion of constants.

```
noncomputable def ofConstantSU (n : Type) [Fintype n] [DecidableEq n] :
    specialUnitaryGroup n ℂ →* specialUnitaryGroup n SpaceTimeAlgebra where
```

def ofConstantU1

[source]

The jet of a constant `U(1)` gauge transformation: the inclusion of constants.

```
noncomputable def ofConstantU1 : unitary ℂ →* unitary SpaceTimeAlgebra where
```

def ofConstant [source]

The embedding of the gauge group into the jet gauge group as the jets of constant (global) gauge transformations.

```
noncomputable def ofConstant : GaugeGroupI →* JetGaugeGroupI
```

lemma eval_ofConstant [source]

Evaluating the jet of a constant gauge transformation at the base point recovers the gauge transformation: `ofConstant` is a section of `eval`.

```
@[simp]
lemma eval_ofConstant (g : GaugeGroupI) : eval (ofConstant g) = g
```

lemma deriv_ofConstant [source]

```
@[simp]
lemma deriv_ofConstant (μ : Fin 1 ⊕ Fin 3) (U₀ : GaugeGroupI) :
    deriv μ (JetGaugeGroupI.ofConstant U₀) = 0
```

2.451 PhyslibAlpha.Particles.StandardModel.GaugeGroup.JetGaugeGroup.Truncation

PhyslibAlpha/Particles/StandardModel/GaugeGroup/JetGaugeGroup/Truncation.lean on GitHub.

def truncation [source]

The `n`-th truncation of a jet of a gauge transformation: componentwise, all Taylor coefficients of total degree greater than `n` are set to zero.

```
noncomputable def truncation (n : ℕ) (U : JetGaugeGroupI) :
    Matrix (Fin 3) (Fin 3) SpaceTimeAlgebra × Matrix (Fin 2)
        (Fin 2) SpaceTimeAlgebra × SpaceTimeAlgebra
```

lemma truncation_one [source]

Truncation of the identity jet is the identity value triple.

```
@[simp]
lemma truncation_one (n : ℕ) : truncation n (1 : JetGaugeGroupI) = 1
```

lemma coeff_maurerCartanForm_eq_zero_of_mem_truncationKer [source]

The base-point Taylor data of the Maurer–Cartan form of a jet trivial to order `n`, read as power-series coefficients of a scalar component `f` of the jet gauge algebra through a scalar `ψ` of the gauge algebra computing evaluated iterated derivatives: they vanish below degree `n`.

```
lemma coeff_maurerCartanForm_eq_zero_of_mem_truncationKer (ψ : GaugeAlgebra → ℂ)
    (hψ : ψ 0 = 0) (f : JetGaugeAlgebra → SpaceTimeAlgebra)
    (hf : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra),
```

```
    ψ (JetGaugeAlgebra.eval (JetGaugeAlgebra.iteratedDeriv s a)) =
      constantCoeff (SpaceTimeAlgebra.iteratedPDeriv s (f a)))
  {U : JetGaugeGroupI} {n : ℕ} (hU : U ∈ localGaugeData.truncationKer n) (ρ : Fin 1 ⊕ Fin 3)
  {m : (Fin 1 ⊕ Fin 3) →₀ ℕ} (hm : Finsupp.degree m < n) :
  coeff m (f (maurerCartanForm U ρ)) = 0
```

lemma coeff_maurerCartanForm_toSU3Matrix_eq_zero_of_mem_truncationKer [source]

The `su(3)` entries of `coeff_maurerCartanForm_eq_zero_of_mem_truncationKer`.

```
lemma coeff_maurerCartanForm_toSU3Matrix_eq_zero_of_mem_truncationKer {U : JetGaugeGroupI}
  {n : ℕ} (hU : U ∈ localGaugeData.truncationKer n) (ρ : Fin 1 ⊕ Fin 3)
  {m : (Fin 1 ⊕ Fin 3) →₀ ℕ} (hm : Finsupp.degree m < n) (i j : Fin 3) :
  coeff m ((maurerCartanForm U ρ).toSU3Matrix i j) = 0
```

lemma coeff_maurerCartanForm_toSU2Matrix_eq_zero_of_mem_truncationKer [source]

The `su(2)` entries of `coeff_maurerCartanForm_eq_zero_of_mem_truncationKer`.

```
lemma coeff_maurerCartanForm_toSU2Matrix_eq_zero_of_mem_truncationKer {U : JetGaugeGroupI}
  {n : ℕ} (hU : U ∈ localGaugeData.truncationKer n) (ρ : Fin 1 ⊕ Fin 3)
  {m : (Fin 1 ⊕ Fin 3) →₀ ℕ} (hm : Finsupp.degree m < n) (i j : Fin 2) :
  coeff m ((maurerCartanForm U ρ).toSU2Matrix i j) = 0
```

lemma coeff_maurerCartanForm_toU1Value_eq_zero_of_mem_truncationKer [source]

The `u(1)` value of `coeff_maurerCartanForm_eq_zero_of_mem_truncationKer`.

```
lemma coeff_maurerCartanForm_toU1Value_eq_zero_of_mem_truncationKer {U : JetGaugeGroupI}
  {n : ℕ} (hU : U ∈ localGaugeData.truncationKer n) (ρ : Fin 1 ⊕ Fin 3)
  {m : (Fin 1 ⊕ Fin 3) →₀ ℕ} (hm : Finsupp.degree m < n) :
  coeff m (maurerCartanForm U ρ).toU1Value = 0
```

lemma truncation_eq_one_of_mem_truncationKer [source]

Maurer–Cartan triangularity for the Standard Model: a jet trivial to order `n` in the sense of the Maurer–Cartan filtration truncates to the identity at order `n`. On each factor the radial relation `∂_ρ U = -i ω_ρ(U) U` and the Euler vanishing principle propagate the vanishing of the Maurer–Cartan coefficients below degree `n` to the vanishing of the coefficients of `U` in nonzero degree up to `n`.

```
lemma truncation_eq_one_of_mem_truncationKer {U : JetGaugeGroupI} {n : ℕ}
  (hU : U ∈ localGaugeData.truncationKer n) : truncation n U = 1
```

2.452 PhyslibAlpha.Particles.StandardModel.GaugeGroup.LocalGaugeData

PhyslibAlpha/Particles/StandardModel/GaugeGroup/LocalGaugeData.lean on GitHub.

def localGaugeData [source]

The Standard Model gauge group as local gauge data, for the jet gauge group `JetGaugeGroupI` and its Lie algebra `JetGaugeAlgebra` over the global group `GaugeGroupI` and gauge algebra `GaugeAlgebra`. It is the gauge data `StandardModel.Model.gaugeData` that the model table assembles from the factor list `[.SU 3, .SU 2, .U1]` by `LocalGaugeData.ofFactors`;

the carriers of that assembly are `JetGaugeGroupI` and `JetGaugeAlgebra` on the nose, and each data field is definitionally the existing Standard Model construction (the rules of section B are all `rfl`).

```
noncomputable def localGaugeData :
    LocalGaugeData GaugeGroupI GaugeAlgebra JetGaugeGroupI JetGaugeAlgebra
```

instance instFreeLocalGaugeData [source]

The Standard Model package is free. It is the local gauge data of the factors `SU(3)`, `SU(2)` and `U(1)`, each free, and freeness passes to products: `LocalGaugeData.instFreeOfFactors`. The symmetrized Maurer–Cartan data are therefore free coordinates on its pure jets, by `LocalGaugeData.symmetrizedMaurerCartanCoeff_bijective`.

```
instance instFreeLocalGaugeData : localGaugeData.Free
```

lemma localGaugeData_eval [source]

```
@[simp]
lemma localGaugeData_eval : localGaugeData.eval = JetGaugeGroupI.eval
```

lemma localGaugeData_ofConstant [source]

```
@[simp]
lemma localGaugeData_ofConstant : localGaugeData.ofConstant = JetGaugeGroupI.ofConstant
```

lemma localGaugeData_evalLie [source]

```
@[simp]
lemma localGaugeData_evalLie : localGaugeData.evalLie = JetGaugeAlgebra.eval
```

lemma localGaugeData_ofConstantLie [source]

```
@[simp]
lemma localGaugeData_ofConstantLie :
    localGaugeData.ofConstantLie = JetGaugeAlgebra.ofConstant
```

lemma localGaugeData_adjoint [source]

```
@[simp]
lemma localGaugeData_adjoint : localGaugeData.adjoint = GaugeAlgebra.adjoint
```

lemma localGaugeData_deriv [source]

```
@[simp]
lemma localGaugeData_deriv (μ : Fin 1 ⊕ Fin 3) :
    localGaugeData.deriv μ = JetGaugeAlgebra.deriv μ
```

lemma localGaugeData_iteratedDeriv [source]

The generic iterated derivative is the Standard Model iterated derivative, both being the same fold of `JetGaugeAlgebra.deriv` over the multiset of directions.

```
@[simp]
lemma localGaugeData_iteratedDeriv (s : Multiset (Fin 1 ⊕ Fin 3)) :
    localGaugeData.iteratedDeriv s = JetGaugeAlgebra.iteratedDeriv s
```

lemma localGaugeData_mulCoord [source]

```
@[simp]
lemma localGaugeData_mulCoord (μ : Fin 1 ⊕ Fin 3) :
    localGaugeData.mulCoord μ = JetGaugeAlgebra.coord μ
```

lemma localGaugeData_adjointJ [source]

```
@[simp]
lemma localGaugeData_adjointJ : localGaugeData.adjointJ = JetGaugeAlgebra.adjoint
```

lemma localGaugeData_maurerCartan [source]

```
@[simp]
lemma localGaugeData_maurerCartan : localGaugeData.maurerCartan = maurerCartanForm
```

lemma localGaugeData_adjointCoeff_apply [source]

The adjoint Taylor coefficients of the package, written out in Standard Model terms.

```
@[simp]
lemma localGaugeData_adjointCoeff_apply (U : JetGaugeGroupI) (x : Multiset (Fin 1 ⊕ Fin 3))
    (a : GaugeAlgebra) :
    localGaugeData.adjointCoeff U x a =
      JetGaugeAlgebra.eval (JetGaugeAlgebra.iteratedDeriv x
        (JetGaugeAlgebra.adjointMap U (JetGaugeAlgebra.ofConstant a)))
```

lemma localGaugeData_adjointCoeff_toSU3Matrix [source]

The `su(3)` component of the adjoint Taylor coefficients.

```
lemma localGaugeData_adjointCoeff_toSU3Matrix (U : JetGaugeGroupI)
    (p : Multiset (Fin 1 ⊕ Fin 3)) (b : GaugeAlgebra) :
    (localGaugeData.adjointCoeff U p b).toSU3Matrix
      = ((U.1.1 * b.toSU3Matrix.map (MvPowerSeries.C : ℂ → SpaceTimeAlgebra) *
          star U.1.1).map fun f =>
          MvPowerSeries.constantCoeff (SpaceTimeAlgebra.iteratedPDeriv p f))
```

lemma localGaugeData_adjointCoeff_toSU2Matrix [source]

The `su(2)` component of the adjoint Taylor coefficients.

```
lemma localGaugeData_adjointCoeff_toSU2Matrix (U : JetGaugeGroupI)
    (p : Multiset (Fin 1 ⊕ Fin 3)) (b : GaugeAlgebra) :
    (localGaugeData.adjointCoeff U p b).toSU2Matrix
      = ((U.2.1.1 * b.toSU2Matrix.map (MvPowerSeries.C : ℂ → SpaceTimeAlgebra) * star U.2.1.1).
    map
         fun f => MvPowerSeries.constantCoeff (SpaceTimeAlgebra.iteratedPDeriv p f))
```

lemma localGaugeData_adjointCoeff_toU1Value [source]

The `u(1)` component of the adjoint Taylor coefficients.

```
lemma localGaugeData_adjointCoeff_toU1Value (U : JetGaugeGroupI)
    (p : Multiset (Fin 1 ⊕ Fin 3)) (b : GaugeAlgebra) :
    (localGaugeData.adjointCoeff U p b).toU1Value
      = MvPowerSeries.constantCoeff
          (SpaceTimeAlgebra.iteratedPDeriv p (MvPowerSeries.C b.toU1Value))
```

instance instFaithfulLocalGaugeData [source]

The Standard Model package is faithful: an element of the jet gauge algebra is determined by the base-point values of its iterated derivatives, and a jet whose Maurer–Cartan form vanishes is the constant jet of its value. Both are statements about power series, proved from the matrix definitions in `JetGaugeAlgebra.ext_of_eval_iteratedDeriv` and `maurerCartanForm_eq_zero_iff_ofConstant`. Unlike the package itself this is a property of it and not a choice, so it is an instance.

```
instance instFaithfulLocalGaugeData : localGaugeData.Faithful
```

2.453 PhyslibAlpha.Particles.StandardModel.GaugeGroup.MaurerCartan.Basic

PhyslibAlpha/Particles/StandardModel/GaugeGroup/MaurerCartan/Basic.lean on GitHub.

def maurerCartanForm [source]

The Maurer–Cartan form `ω_μ(U) := i (∂_μ U) U⁻¹` of the jet gauge group, valued in the jet gauge algebra.

```
noncomputable def maurerCartanForm (U : JetGaugeGroupI) (μ : Fin 1 ⊕ Fin 3) : JetGaugeAlgebra
```

lemma maurerCartanForm_toSU3Matrix [source]

```
@[simp]
lemma maurerCartanForm_toSU3Matrix (U : JetGaugeGroupI) (μ : Fin 1 ⊕ Fin 3) :
    (maurerCartanForm U μ).toSU3Matrix =
      Complex.I • (U.1.1.map (pderiv μ) * star U.1.1)
```

lemma maurerCartanForm_toSU2Matrix [source]

```
@[simp]
lemma maurerCartanForm_toSU2Matrix (U : JetGaugeGroupI) (μ : Fin 1 ⊕ Fin 3) :
    (maurerCartanForm U μ).toSU2Matrix =
      Complex.I • (U.2.1.1.map (pderiv μ) * star U.2.1.1)
```

lemma maurerCartanForm_toU1Value [source]

```
@[simp]
lemma maurerCartanForm_toU1Value (U : JetGaugeGroupI) (μ : Fin 1 ⊕ Fin 3) :
    (maurerCartanForm U μ).toU1Value =
      Complex.I • (pderiv μ U.2.2.1 * star U.2.2.1)
```

lemma maurerCartanForm_one [source]

```
@[simp]
lemma maurerCartanForm_one : maurerCartanForm (1 : JetGaugeGroupI) = 0
```

lemma maurerCartanForm_ofConstant [source]

```
lemma maurerCartanForm_ofConstant (U₀ : GaugeGroupI) :
    maurerCartanForm (JetGaugeGroupI.ofConstant U₀) = 0
```

lemma maurerCartanForm_cocycle [source]

```
lemma maurerCartanForm_cocycle (U V : JetGaugeGroupI) (μ : Fin 1 ⊕ Fin 3) :
    maurerCartanForm (U * V) μ = maurerCartanForm U μ + adjoint U (maurerCartanForm V μ)
```

lemma deriv_zero_of_maurerCartanForm_zero [source]

```
lemma deriv_zero_of_maurerCartanForm_zero (U : JetGaugeGroupI) (h : maurerCartanForm U = 0) :
    ∀ μ, U.deriv μ = 0
```

lemma maurerCartanForm_eq_zero_iff_ofConstant [source]

```
lemma maurerCartanForm_eq_zero_iff_ofConstant (U : JetGaugeGroupI) :
    maurerCartanForm U = 0 ↔ ∃ c, U = JetGaugeGroupI.ofConstant c
```

lemma maurerCartanForm_structure [source]

The structural (Maurer–Cartan) equation, basis-independently: the Maurer–Cartan form is flat,

`∂_μ ω_ν − ∂_ν ω_μ + ⁅ω_μ, ω_ν⁆ = 0`.

In components with respect to a basis of the jet gauge algebra this is `∂_μ ω^a_ν − ∂_ν ω^a_μ = ∑_{b c} f^a_{b c} · ω^b_μ · ω^c_ν`. On each matrix factor the second-derivative terms cancel by symmetry of mixed partials, the derivative of `A†` is rewritten through the differentiated unitarity relation, and the surviving first-order terms form the commutator; on the abelian `U(1)` factor the commutator is absent and only the symmetry of mixed partials remains.

```
lemma maurerCartanForm_structure (U : JetGaugeGroupI) (μ ν : Fin 1 ⊕ Fin 3) :
    deriv μ (maurerCartanForm U ν) - deriv ν (maurerCartanForm U μ) +
      ⁅maurerCartanForm U μ, maurerCartanForm U ν⁆ = 0
```

lemma deriv_ofConstant [source]

The constant inclusion has vanishing formal derivative: constants have no spacetime dependence.

```
@[simp]
lemma JetGaugeAlgebra.deriv_ofConstant (μ : Fin 1 ⊕ Fin 3) (a : GaugeAlgebra) :
    deriv μ (ofConstant a) = 0
```

lemma deriv_adjointMap [source]

The formal derivative intertwines the adjoint action through the Maurer–Cartan form: `∂_μ (Ad_U x) = Ad_U (∂_μ x) - ⁅ω_μ(U), Ad_U x⁆`. On the matrix factors this is the Leibniz rule with the derivative of `U†` rewritten through the differentiated unitarity relation; on the abelian `u(1)` factor the adjoint action is trivial and the bracket is absent.

```
lemma deriv_adjointMap (U : JetGaugeGroupI) (μ : Fin 1 ⊕ Fin 3) (x : JetGaugeAlgebra) :
    deriv μ (adjointMap U x) =
      adjointMap U (deriv μ x) - ⁅maurerCartanForm U μ, adjointMap U x⁆
```

2.454 PhyslibAlpha.Particles.StandardModel.GaugeGroup.SU2Conjugation

PhyslibAlpha/Particles/StandardModel/GaugeGroup/SU2Conjugation.lean on GitHub.

def su2Epsilon [source]

The antisymmetric symbol `ε_{ab}` on two `su(2)` fundamental indices: the Levi-Civita symbol of `Fin 2`, normalized by `ε 0 1 = 1`.

```
def su2Epsilon (a b : Fin 2) : ℂ
```

lemma sum_su2Epsilon_mul [source]

Every element of `SU(2)` fixes the antisymmetric symbol: contracted against two rows of `U` it gives `det U` times itself (`sum_leviCivitaSymbol_mul_prod`), and `det U = 1`.

```
lemma sum_su2Epsilon_mul (U : specialUnitaryGroup (Fin 2) ℂ) (b c : Fin 2) :
    ∑ x : Fin 2, ∑ y : Fin 2, su2Epsilon x y * (U.1 b x * U.1 c y) = su2Epsilon b c
```

lemma su2_star_eq_adjugate [source]

The conjugate transpose of an `SU(2)` matrix is its adjugate.

```
lemma su2_star_eq_adjugate (U : specialUnitaryGroup (Fin 2) ℂ) :
    star U.1 = Matrix.adjugate U.1
```

lemma su2_conj_apply [source]

The conjugate of an entry of an `SU(2)` matrix is the transposed entry of its adjugate.

```
lemma su2_conj_apply (U : specialUnitaryGroup (Fin 2) ℂ) (i j : Fin 2) :
    conj (U.1 i j) = Matrix.adjugate U.1 j i
```

2.455 PhyslibAlpha.Particles.StandardModel.HiggsBoson.GaugeAlgebraAction

PhyslibAlpha/Particles/StandardModel/HiggsBoson/GaugeAlgebraAction.lean on GitHub.

def weakEnd [source]

The endomorphism of the Higgs doublet defined by a `2 × 2` complex matrix acting on the weak index.

```
noncomputable def weakEnd (A : Matrix (Fin 2) (Fin 2) ℂ) :
    HiggsVec →ₗ[ℂ] HiggsVec
```

lemma weakEnd_apply [source]

```
lemma weakEnd_apply (A : Matrix (Fin 2) (Fin 2) ℂ) (v : HiggsVec) :
    weakEnd A v = Matrix.toLpLinAlgEquiv 2 A v
```

lemma weakEnd_add [source]

```
lemma weakEnd_add (A B : Matrix (Fin 2) (Fin 2) ℂ) :
    weakEnd (A + B) = weakEnd A + weakEnd B
```

lemma weakEnd_smul [source]

```
lemma weakEnd_smul (z : ℂ) (A : Matrix (Fin 2) (Fin 2) ℂ) :
    weakEnd (z • A) = z • weakEnd A
```

lemma weakEnd_zero [source]

```
lemma weakEnd_zero : weakEnd 0 = 0
```

lemma weakEnd_neg [source]

```
lemma weakEnd_neg (A : Matrix (Fin 2) (Fin 2) ℂ) : weakEnd (-A) = -weakEnd A
```

lemma weakEnd_multiset_sum [source]

```
lemma weakEnd_multiset_sum (m : Multiset (Matrix (Fin 2) (Fin 2) ℂ)) :
    weakEnd m.sum = (m.map weakEnd).sum
```

lemma weakEnd_mul [source]

The weak endomorphisms compose through matrix multiplication.

```
lemma weakEnd_mul (A B : Matrix (Fin 2) (Fin 2) ℂ) :
    weakEnd (A * B) = weakEnd A ∘ₗ weakEnd B
```

def actionMatrix [source]

The matrix of the infinitesimal `(1, 2)_{3}` action of a gauge algebra element on the weak index: `i` times the weak part, shifted by `i` times `3` the hypercharge.

```
noncomputable def actionMatrix (c : GaugeAlgebra) : Matrix (Fin 2) (Fin 2) ℂ
```

def gaugeAlgebraAction [source]

***The infinitesimal action of the gauge algebra on the Higgs doublet**: the derivative of the `(1, 2)_{3}` action of the gauge group, real-linear in the algebra slot and complex-linear in the value slot — the form consumed by the covariant derivative `GaugeAlgebraRealization.covDerivIter` and by `LocalGaugeData.IsInfinitesimalActionOf`.*

```
noncomputable def gaugeAlgebraAction :
    GaugeAlgebra →ₗ[ℝ] HiggsVec →ₗ[ℂ] HiggsVec where
```

def jetActionMatrix [source]

The jet-valued matrix of the infinitesimal `(1, 2)_{3}` action of a jet of gauge algebra elements: the jet analogue of `actionMatrix`.

```
noncomputable def jetActionMatrix (a : JetGaugeAlgebra) : Matrix (Fin 2) (Fin 2)
    SpaceTimeAlgebra
```

lemma jetActionMatrix_map_cc_foldl [source]

The base-point Taylor coefficients of the jet action matrix are the action matrices of the base-point Taylor coefficients.

```
lemma jetActionMatrix_map_cc_foldl (p : Multiset (Fin 1 ⊕ Fin 3)) (a : JetGaugeAlgebra) :
    ((jetActionMatrix a).map fun f =>
        constantCoeff (SpaceTimeAlgebra.iteratedPDeriv p f))
      = actionMatrix (JetGaugeAlgebra.eval (JetGaugeAlgebra.iteratedDeriv p a))
```

def pderivWeak [source]

The entrywise formal derivative on the weak coordinates, as a $\mathbb{C}$-linear map.

```
private noncomputable def pderivWeak (μ : Fin 1 ⊕ Fin 3) :
    EuclideanSpace SpaceTimeAlgebra (Fin 2) →ₗ[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 2) where
```

def foldWeak [source]

The entrywise iterated formal derivative on the weak coordinates.

```
private noncomputable def foldWeak (x : Multiset (Fin 1 ⊕ Fin 3)) :
    EuclideanSpace SpaceTimeAlgebra (Fin 2) →ₗ[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 2) where
```

def ccWeak [source]

The entrywise base-point evaluation on the weak coordinates.

```
private noncomputable def ccWeak :
    EuclideanSpace SpaceTimeAlgebra (Fin 2) →ₗ[ℂ] EuclideanSpace ℂ (Fin 2) where
```

lemma pderivWeak_comp_foldWeak [source]

```
private lemma pderivWeak_comp_foldWeak (μ : Fin 1 ⊕ Fin 3)
    (x : Multiset (Fin 1 ⊕ Fin 3)) :
    pderivWeak μ ∘ₗ foldWeak x = foldWeak (μ ::ₘ x)
```

lemma jetValLinEquiv_jetDeriv [source]

The identification of Higgs-doublet jets intertwines the formal derivative with the entrywise derivative on the weak coordinates.

```
private lemma jetValLinEquiv_jetDeriv (μ : Fin 1 ⊕ Fin 3)
    (z : SpaceTimeAlgebra ⊗[ℂ] HiggsVec) :
    jetValLinEquiv (jetDeriv μ z)
      = pderivWeak μ (jetValLinEquiv z)
```

lemma jetValLinEquiv_jetIteratedDeriv [source]

The identification of Higgs-doublet jets intertwines the iterated formal derivative with the entrywise iterated derivative on the weak coordinates.

```
private lemma jetValLinEquiv_jetIteratedDeriv (x : Multiset (Fin 1 ⊕ Fin 3))
    (z : SpaceTimeAlgebra ⊗[ℂ] HiggsVec) :
    jetValLinEquiv (jetIteratedDeriv x z)
      = foldWeak x (jetValLinEquiv z)
```

lemma jetEval_eq [source]

The base-point evaluation of a Higgs-doublet jet through the weak coordinates.

```
private lemma jetEval_eq (z : SpaceTimeAlgebra ⊗[ℂ] HiggsVec) :
    jetEval z = ccWeak (jetValLinEquiv z)
```

lemma jetGaugeMatrix_map_pderiv [source]

***The derivative identity** for the weak matrix of the jet gauge action: the formal derivative of the weak matrix is minus the jet action matrix of the Maurer–Cartan form times the weak matrix.*

```
lemma jetGaugeMatrix_map_pderiv (U : JetGaugeGroupI) (μ : Fin 1 ⊕ Fin 3) :
    (jetGaugeMatrix U).map (fun f => pderiv μ f)
      = -(jetActionMatrix (maurerCartanForm U μ) * jetGaugeMatrix U)
```

lemma jetGaugeMatrix_mul_jetActionMatrix [source]

***The equivariance identity** for the weak matrix of the jet gauge action: the weak matrix intertwines the constant jet action matrix with its adjoint transform.*

```
lemma jetGaugeMatrix_mul_jetActionMatrix (U : JetGaugeGroupI) (c : GaugeAlgebra) :
    jetGaugeMatrix U * jetActionMatrix (JetGaugeAlgebra.ofConstant c)
      = jetActionMatrix (JetGaugeAlgebra.adjointMap U (JetGaugeAlgebra.ofConstant c))
        * jetGaugeMatrix U
```

lemma repCoeff_eq [source]

***The base-point Taylor coefficients of the jet gauge action** on the Higgs doublet are the weak endomorphisms of the base-point Taylor coefficients of the weak matrix.*

```
lemma repCoeff_eq (U : JetGaugeGroupI) (x : Multiset (Fin 1 ⊕ Fin 3)) :
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U x
      = weakEnd ((jetGaugeMatrix U).map fun f =>
          constantCoeff (SpaceTimeAlgebra.iteratedPDeriv x f))
```

lemma weakEnd_one [source]

The weak endomorphism of the identity matrix is the identity.

```
lemma weakEnd_one : weakEnd 1 = LinearMap.id
```

lemma repCoeff_zero_of_eval_eq_one [source]

At the base point, a gauge jet with trivial value acts trivially: the zeroth Taylor coefficient of the jet gauge action is the identity.

```
lemma repCoeff_zero_of_eval_eq_one {U : JetGaugeGroupI} (hU : U.eval = 1) :
    GaugeAlgebraRealization.repCoeff repJetGaugeGroupI U 0 = LinearMap.id
```

lemma isInfinitesimalActionOf [source]

***The `(1, 2)_{3}` action of the gauge algebra is the infinitesimal action underlying the jet gauge action on the Higgs doublet**: its base-point Taylor coefficients obey the Maurer–Cartan Leibniz law and intertwine the action with the adjoint transports.*

```
lemma isInfinitesimalActionOf :
    localGaugeData.IsInfinitesimalActionOf gaugeAlgebraAction repJetGaugeGroupI
```

lemma gaugeAlgebraAction_comm_repLorentz [source]

The infinitesimal gauge action on the Higgs commutes with the Lorentz action, which is trivial.

```
lemma gaugeAlgebraAction_comm_repLorentz (c : GaugeAlgebra) (Λ : SL(2,ℂ)) (v : HiggsVec) :
    HiggsVec.gaugeAlgebraAction c ((Representation.trivial ℂ SL(2,ℂ) HiggsVec) Λ v) =
      (Representation.trivial ℂ SL(2,ℂ) HiggsVec) Λ (HiggsVec.gaugeAlgebraAction c v)
```

2.456 PhyslibAlpha.Particles.StandardModel.HiggsBoson.JetAlgebra.Algebra

PhyslibAlpha/Particles/StandardModel/HiggsBoson/JetAlgebra/Algebra.lean on GitHub.

abbrev HiggsJetAlgebra [source]

***The jet algebra of the Higgs field**: the bosonic algebra of the `HiggsVec`-valued Higgs field. Its generators are the component functions `∂_s H_α` and `∂_s H¯_α`, and they commute — the Higgs is a boson.*

```
abbrev HiggsJetAlgebra : Type
```

def ofHiggs [source]

The component functions of the Higgs field inside its jet algebra.

```
noncomputable def ofHiggs : Module.Dual ℂ HiggsVec →ₗ[ℂ] HiggsJetAlgebra
```

def ofConjHiggs [source]

The conjugate component functions of the Higgs field inside its jet algebra.

```
noncomputable def ofConjHiggs :
    Module.Dual ℂ (ConjModule HiggsVec) →ₗ[ℂ] HiggsJetAlgebra
```

def repLorentzGroup [source]

The Lorentz action on the jet algebra of the Higgs field: the Higgs is a Lorentz scalar, so the Lorentz group acts on the component functions only through their derivative labels.

```
noncomputable def repLorentzGroup : Representation ℂ SL(2,ℂ) HiggsJetAlgebra
```

def repJetGaugeGroupI [source]

The jet gauge action on the jet algebra of the Higgs field, lifted from the fibrewise action on its jets.

```
noncomputable def repJetGaugeGroupI : Representation ℂ JetGaugeGroupI HiggsJetAlgebra
```

def repGaugeGroupI [source]

The action of the constant — global — gauge transformations on the jet algebra of the Higgs field.

```
noncomputable def repGaugeGroupI : Representation ℂ GaugeGroupI HiggsJetAlgebra
```

def massWeightScale [source]

The mass-dimension scaling on the jet algebra of the Higgs field: the Higgs has mass dimension one, that is mass weight two, and each derivative adds mass weight two.

```
noncomputable def massWeightScale (c : ℂ) : HiggsJetAlgebra →ₐ[ℂ] HiggsJetAlgebra
```

lemma massWeightScale_ofHiggs [source]

The Higgs field carries mass weight two — mass dimension one.

```
@[simp]
lemma massWeightScale_ofHiggs (c : ℂ) (φ : Module.Dual ℂ HiggsVec) :
    massWeightScale c (ofHiggs φ) = c ^ 2 • ofHiggs φ
```

lemma massWeightScale_jetDeriv [source]

A derivative of the Higgs field adds mass weight two.

```
lemma massWeightScale_jetDeriv (c : ℂ) (μ : Fin 1 ⊕ Fin 3) (x : HiggsJetAlgebra) :
    massWeightScale c (BosonicAlgebra.jetDeriv μ x)
      = c ^ 2 • BosonicAlgebra.jetDeriv μ (massWeightScale c x)
```

2.457 PhyslibAlpha.Particles.StandardModel.HiggsBoson.JetAlgebra.Basic

PhyslibAlpha/Particles/StandardModel/HiggsBoson/JetAlgebra/Basic.lean on GitHub.

def jetValLinEquiv [source]

Absorbs the jet ring into the weak index: a jet of the Higgs field is the same thing as a `SpaceTimeAlgebra`-valued weak doublet,

`SpaceTimeAlgebra ⊗[ℂ] HiggsVec ≃ EuclideanSpace SpaceTimeAlgebra (Fin 2).`

```
noncomputable def jetValLinEquiv :
    SpaceTimeAlgebra ⊗[ℂ] HiggsVec ≃ₗ[ℂ] EuclideanSpace SpaceTimeAlgebra (Fin 2)
```

lemma jetValLinEquiv_tmul [source]

```
lemma jetValLinEquiv_tmul (f : SpaceTimeAlgebra) (v : HiggsVec) :
    jetValLinEquiv (f ⊗ₜ[ℂ] v) = WithLp.toLp 2 fun i => v.ofLp i • f
```

lemma jetValLinEquiv_smul [source]

The identification of the jets of the Higgs field is `SpaceTimeAlgebra`-linear: multiplying a jet by a scalar jet multiplies each of its weak components.

```
lemma jetValLinEquiv_smul (χ : SpaceTimeAlgebra) (z : SpaceTimeAlgebra ⊗[ℂ] HiggsVec) :
    jetValLinEquiv (χ • z) = χ • jetValLinEquiv z
```

lemma jetValLinEquiv_symm_smul [source]

```
lemma jetValLinEquiv_symm_smul (χ : SpaceTimeAlgebra)
    (y : EuclideanSpace SpaceTimeAlgebra (Fin 2)) :
    jetValLinEquiv.symm (χ • y) = χ • jetValLinEquiv.symm y
```

def jetGaugeMatrix [source]

The matrix of jets through which a jet of gauge transformations acts on the Higgs doublet: the `SU(2)` power-series matrix scaled by the hypercharge power series `u ^ 3`.

```
noncomputable def jetGaugeMatrix (U : JetGaugeGroupI) : Matrix (Fin 2) (Fin 2) SpaceTimeAlgebra
```

lemma jetGaugeMatrix_one [source]

```
lemma jetGaugeMatrix_one : jetGaugeMatrix 1 = 1
```

lemma jetGaugeMatrix_mul [source]

```
lemma jetGaugeMatrix_mul (U₁ U₂ : JetGaugeGroupI) :
    jetGaugeMatrix (U₁ * U₂) = jetGaugeMatrix U₁ * jetGaugeMatrix U₂
```

def repJetGaugeGroupI [source]

The `2_{3}` action of the jet gauge group on the jets of the Higgs field. Through `jetValLinEquiv` the weak matrix of the gauge jet, carrying the `3` hypercharge phase `u ^ 3`, acts `SpaceTimeAlgebra`-linearly by matrix-vector multiplication.

```
noncomputable def repJetGaugeGroupI :
    Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] HiggsVec) where
```

lemma repJetGaugeGroupI_apply [source]

```
lemma repJetGaugeGroupI_apply (U : JetGaugeGroupI) (z : SpaceTimeAlgebra ⊗[ℂ] HiggsVec) :
    repJetGaugeGroupI U z =
      jetValLinEquiv.symm
        (Matrix.toLpLinAlgEquiv 2 (jetGaugeMatrix U) (jetValLinEquiv z))
```

lemma repJetGaugeGroupI_smul [source]

***The jet gauge action on the jets of the Higgs field is fibrewise**: it commutes with multiplication by scalar jets, acting on the values of the field over the identity on spacetime. This is the hypothesis under which the action lifts to the bosonic algebra.*

```
lemma repJetGaugeGroupI_smul (U : JetGaugeGroupI) (χ : SpaceTimeAlgebra)
    (z : SpaceTimeAlgebra ⊗[ℂ] HiggsVec) :
    repJetGaugeGroupI U (χ • z) = χ • repJetGaugeGroupI U z
```

lemma repJetGaugeGroupI_ofConstant [source]

On jets of constant gauge transformations the jet action reduces to the global gauge action on the fibre: the action `HiggsVec.repGaugeGroupI` on the Higgs factor, and the trivial action on the jet ring.

```
lemma repJetGaugeGroupI_ofConstant (g : GaugeGroupI) :
    repJetGaugeGroupI (JetGaugeGroupI.ofConstant g) =
      TensorProduct.map LinearMap.id (repGaugeGroupI g)
```

2.458 PhyslibAlpha.Particles.StandardModel.HiggsBoson.MatterField

PhyslibAlpha/Particles/StandardModel/HiggsBoson/MatterField.lean on GitHub.

def matterField [source]

The Higgs field as a matter field of `StandardModel.localGaugeData`, valued in `HiggsVec`, in the `2_{3}` representation of the gauge group, a Lorentz scalar, of mass weight two, declared bosonic.

```
noncomputable def matterField : MatterField localGaugeData where
```

lemma matterField_V [source]

```
@[simp]
lemma matterField_V : matterField.V = HiggsVec
```

lemma matterField_repLorentz [source]

```
@[simp]
lemma matterField_repLorentz :
    matterField.repLorentz = Representation.trivial ℂ SL(2,ℂ) HiggsVec
```

lemma matterField_repJet [source]

```
@[simp]
lemma matterField_repJet : matterField.repJet = repJetGaugeGroupI
```

lemma matterField_repAlgebra [source]

```
@[simp]
lemma matterField_repAlgebra : matterField.repAlgebra = gaugeAlgebraAction
```

lemma matterField_massWeight [source]

```
@[simp]
lemma matterField_massWeight : matterField.massWeight = 2
```

lemma matterField_pureJetsActTrivially [source]

Pure gauge jets act trivially on the Higgs at the base point.

```
lemma matterField_pureJetsActTrivially : matterField.PureJetsActTrivially
```

lemma matterField_gaugeLorentzCompatible [source]

The gauge action on the Higgs commutes with its (trivial) Lorentz action.

```
lemma matterField_gaugeLorentzCompatible : matterField.GaugeLorentzCompatible
```

2.459 PhyslibAlpha.Particles.StandardModel.InvariantReduction

PhyslibAlpha/Particles/StandardModel/InvariantReduction.lean on GitHub.

def gaugeLorentzMaps [source]

The gauge and Lorentz groups read as a single family of linear maps, indexed by their disjoint union.

```
def gaugeLorentzMaps : GaugeGroupI ⊕ SL(2,ℂ) → B →ₗ[ℂ] B
```

lemma isStableUnder_gaugeLorentzMaps_iff [source]

A submodule stable under both groups is stable under the combined family, and conversely.

```
lemma isStableUnder_gaugeLorentzMaps_iff {V : Submodule ℂ B} :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) V
      ↔ (∀ g : GaugeGroupI, ∀ y ∈ V, repGauge g y ∈ V)
        ∧ ∀ Λ : SL(2,ℂ), ∀ y ∈ V, repLorentz Λ y ∈ V
```

lemma forall_gaugeLorentzMaps_eq_self_iff [source]

An element fixed by both groups is fixed by the combined family, and conversely.

```
lemma forall_gaugeLorentzMaps_eq_self_iff {x : B} :
    (∀ p, gaugeLorentzMaps repGauge repLorentz p x = x)
      ↔ (∀ g : GaugeGroupI, repGauge g x = x) ∧ ∀ Λ : SL(2,ℂ), repLorentz Λ x = x
```

lemma isFixedBy_gaugeLorentzMaps_iff [source]

A submodule fixed pointwise by both groups is fixed pointwise by the combined family, and conversely.

```
lemma isFixedBy_gaugeLorentzMaps_iff {V : Submodule ℂ B} :
    IsFixedBy (gaugeLorentzMaps repGauge repLorentz) V
      ↔ (∀ g : GaugeGroupI, ∀ y ∈ V, repGauge g y = y)
        ∧ ∀ Λ : SL(2,ℂ), ∀ y ∈ V, repLorentz Λ y = y
```

lemma mem_sup_and_gauge_lorentz_invariant_iff [source]

The classification endpoint `ReducesInvariantsTo.mem_sup_and_forall_eq_self_iff` for the gauge and Lorentz groups together, with the two invariances and the two stabilities stated separately.

```
lemma ReducesInvariantsTo.mem_sup_and_gauge_lorentz_invariant_iff {V W S : Submodule ℂ B}
    (hP : ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) V W) (hWV : W ≤ V)
    (hW : IsFixedBy (gaugeLorentzMaps repGauge repLorentz) W)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ Λ : SL(2,ℂ), ∀ y ∈ S, repLorentz Λ y ∈ S) (x : B) :
    (x ∈ V ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x) ∧ ∀ Λ : SL(2,ℂ), repLorentz Λ x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ Λ : SL(2,ℂ), repLorentz Λ y = y) ∧ x - y ∈ W
```

lemma ofGauge [source]

A gauge reduction is a reduction for the gauge and Lorentz groups together.

```
lemma ReducesInvariantsTo.ofGauge {V W : Submodule ℂ B}
    (hP : ReducesInvariantsTo (fun g : GaugeGroupI => repGauge g) V W) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) V W
```

lemma ofSU3 [source]

A reduction for the colour factor is a reduction for the gauge and Lorentz groups together.

```
lemma ReducesInvariantsTo.ofSU3 {V W : Submodule ℂ B}
    (hP : ReducesInvariantsTo (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge (U, 1, 1)) V W
    ) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) V W
```

lemma ofSU2 [source]

A reduction for the isospin factor is a reduction for the gauge and Lorentz groups together.

```
lemma ReducesInvariantsTo.ofSU2 {V W : Submodule ℂ B}
    (hP : ReducesInvariantsTo (fun U : specialUnitaryGroup (Fin 2) ℂ => repGauge (1, U, 1)) V W
    ) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) V W
```

lemma ofLorentz [source]

A Lorentz reduction is a reduction for the gauge and Lorentz groups together.

```
lemma ReducesInvariantsTo.ofLorentz {V W : Submodule ℂ B}
    (hP : ReducesInvariantsTo (fun Λ : SL(2,ℂ) => repLorentz Λ) V W) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) V W
```

lemma gaugeLorentzMaps_mul [source]

The two groups read as one family of maps respect multiplication, each of the two representations doing so.

```
lemma gaugeLorentzMaps_mul
    (hG : ∀ (g : GaugeGroupI) (b₁ b₂ : B),
      repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂)
    (hL : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
      repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂)
    (p : GaugeGroupI ⊕ SL(2,ℂ)) (a b : B) :
    gaugeLorentzMaps repGauge repLorentz p (a * b)
      = gaugeLorentzMaps repGauge repLorentz p a
        * gaugeLorentzMaps repGauge repLorentz p b
```

lemma su3_inv_apply [source]

The entries of the inverse of an `SU(3)` element are the conjugated transposed entries, the inverse of a unitary matrix being its conjugate transpose.

```
lemma su3_inv_apply (U : specialUnitaryGroup (Fin 3) ℂ) (a b : Fin 3) :
    (U⁻¹).1 a b = conj (U.1 b a)
```

lemma su2_inv_apply [source]

The entries of the inverse of an `SU(2)` element are the conjugated transposed entries.

```
lemma su2_inv_apply (U : specialUnitaryGroup (Fin 2) ℂ) (a b : Fin 2) :
    (U⁻¹).1 a b = conj (U.1 b a)
```

lemma unitary_inv_coe [source]

The inverse of a unitary scalar is its conjugate.

```
lemma unitary_inv_coe (t : unitary ℂ) : ((t⁻¹ : unitary ℂ) : ℂ) = star (t : ℂ)
```

lemma inv_su3Elt [source]

The inverse of a colour gauge transformation is the colour transformation of the inverse.

```
lemma inv_su3Elt (U : specialUnitaryGroup (Fin 3) ℂ) :
    ((U, 1, 1) : GaugeGroupI)⁻¹ = ((U⁻¹, 1, 1) : GaugeGroupI)
```

lemma inv_su2Elt [source]

The inverse of an isospin gauge transformation is the isospin transformation of the inverse.

```
lemma inv_su2Elt (V : specialUnitaryGroup (Fin 2) ℂ) :
    ((1, V, 1) : GaugeGroupI)⁻¹ = ((1, V⁻¹, 1) : GaugeGroupI)
```

lemma inv_u1Elt [source]

The inverse of a hypercharge gauge transformation is the hypercharge transformation of the inverse.

```
lemma inv_u1Elt (t : unitary ℂ) :
    ((1, 1, t) : GaugeGroupI)⁻¹ = ((1, 1, t⁻¹) : GaugeGroupI)
```

lemma forall_repGauge_eq_self [source]

A gauge transformation is the product of its colour, isospin and hypercharge parts, so an element fixed by each of the three factors separately is gauge invariant.

```
lemma forall_repGauge_eq_self {B : Type*} [AddCommGroup B] [Module ℂ B]
    {rep : Representation ℂ GaugeGroupI B} {x : B}
    (h3 : ∀ U : specialUnitaryGroup (Fin 3) ℂ, rep (U, 1, 1) x = x)
    (h2 : ∀ V : specialUnitaryGroup (Fin 2) ℂ, rep (1, V, 1) x = x)
    (h1 : ∀ t : unitary ℂ, rep (1, 1, t) x = x) (g : GaugeGroupI) : rep g x = x
```

lemma isStableUnder_range_repGauge [source]

The range of a symbol map is carried into itself by the gauge group: the symbol is equivariant, so a gauge transformation only moves the dual vector it is evaluated at.

```
lemma isStableUnder_range_repGauge {M : Type} [AddCommGroup M] [Module ℂ M]
    {repGauge : Representation ℂ GaugeGroupI B} {ρ : Representation ℂ GaugeGroupI M}
    {F : Module.Dual ℂ M →ₗ[ℂ] B} (hF : ∀ g φ, repGauge g (F φ) = F (ρ.dual g φ)) :
    ∀ g : GaugeGroupI, ∀ y ∈ LinearMap.range F, repGauge g y ∈ LinearMap.range F
```

lemma isStableUnder_range_repLorentz [source]

The range of an underived symbol map is carried into itself by the Lorentz group: with no derivative slots to mix, the transformation law moves the dual vector alone.

```
lemma isStableUnder_range_repLorentz {M : Type} [AddCommGroup M] [Module ℂ M]
    {repLorentz : Representation ℂ SL(2,ℂ) B} {ρ : Representation ℂ SL(2,ℂ) M}
    {F : {n : ℕ} → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ M →ₗ[ℂ] B}
    (hF : IsLorentzCovDerivTransforms repLorentz ρ F) (Λ : SL(2,ℂ)) :
    ∀ y ∈ LinearMap.range (F (![] : Fin 0 → Fin 1 ⊕ Fin 3)),
      repLorentz Λ y ∈ LinearMap.range (F (![] : Fin 0 → Fin 1 ⊕ Fin 3))
```

2.460 PhyslibAlpha.Particles.StandardModel.IsFermionSector.Basic

PhyslibAlpha/Particles/StandardModel/IsFermionSector/Basic.lean on GitHub.

structure IsFermionSector [source]

The ten fermion families and their covariant derivatives as a sector of the algebra `B`: gauge transformation through the dual of the species representations (conjugate for the barred fields), the Lorentz transformation of the towers, and the mass weights `3 + 2 * n`.

```
structure IsFermionSector (B : Type) [Ring B] [Algebra ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B)
    (repGauge_mul : ∀ (g : GaugeGroupI) (b₁ b₂ : B),
      repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂)
    (repLorentz : Representation ℂ SL(2,ℂ) B)
    (repLorentz_mul : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
      repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂)
    (d : {n : ℕ} → Fin 3 → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ DownSinglet →ₗ[ℂ] B)
    (bard : {n : ℕ} → Fin 3 → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ (ConjModule DownSinglet)
    →ₗ[ℂ]
      B)
    (u : {n : ℕ} → Fin 3 → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ UpSinglet →ₗ[ℂ] B)
    (baru : {n : ℕ} → Fin 3 → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ (ConjModule UpSinglet)
    →ₗ[ℂ]
      B)
    (Q : {n : ℕ} → Fin 3 → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ QuarkDoublet →ₗ[ℂ] B)
    (barQ : {n : ℕ} → Fin 3 → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ (ConjModule QuarkDoublet)
      →ₗ[ℂ] B)
    (L : {n : ℕ} → Fin 3 → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ LeptonDoubletV →ₗ[ℂ] B)
    (barL : {n : ℕ} → Fin 3 → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ (ConjModule
    LeptonDoubletV)
      →ₗ[ℂ] B)
    (e : {n : ℕ} → Fin 3 → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ LeptonSinglet →ₗ[ℂ] B)
    (bare : {n : ℕ} → Fin 3 → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ (ConjModule LeptonSinglet
    )
      →ₗ[ℂ] B)
    (massWeightPoly : B →ₐ[ℂ] Polynomial B) : Prop where
  repGauge_d : ∀ (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
      (l : Fin n → Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ DownSinglet),
    repGauge g (d i l φ) = d i l (DownSinglet.repGaugeGroupI.dual g φ)
  repGauge_bard : ∀ (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
      (l : Fin n → Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ (ConjModule DownSinglet)),
    repGauge g (bard i l φ) = bard i l (DownSinglet.repGaugeGroupI.conj.dual g φ)
  repGauge_u : ∀ (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
      (l : Fin n → Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ UpSinglet),
    repGauge g (u i l φ) = u i l (UpSinglet.repGaugeGroupI.dual g φ)
  repGauge_baru : ∀ (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
      (l : Fin n → Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ (ConjModule UpSinglet)),
    repGauge g (baru i l φ) = baru i l (UpSinglet.repGaugeGroupI.conj.dual g φ)
  repGauge_Q : ∀ (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
      (l : Fin n → Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ QuarkDoublet),
    repGauge g (Q i l φ) = Q i l (QuarkDoublet.repGaugeGroupI.dual g φ)
  repGauge_barQ : ∀ (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
      (l : Fin n → Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ (ConjModule QuarkDoublet)),
    repGauge g (barQ i l φ) = barQ i l (QuarkDoublet.repGaugeGroupI.conj.dual g φ)
  repGauge_L : ∀ (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
      (l : Fin n → Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ LeptonDoubletV),
    repGauge g (L i l φ) = L i l (LeptonDoubletV.repGaugeGroupI.dual g φ)
  repGauge_barL : ∀ (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
      (l : Fin n → Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)),
    repGauge g (barL i l φ) = barL i l (LeptonDoubletV.repGaugeGroupI.conj.dual g φ)
  repGauge_e : ∀ (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
      (l : Fin n → Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ LeptonSinglet),
    repGauge g (e i l φ) = e i l (LeptonSinglet.repGaugeGroupI.dual g φ)
  repGauge_bare : ∀ (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
      (l : Fin n → Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ (ConjModule LeptonSinglet)),
    repGauge g (bare i l φ) = bare i l (LeptonSinglet.repGaugeGroupI.conj.dual g φ)
```

```
  repLorentz_d : ∀ i, IsLorentzCovDerivTransforms repLorentz
    DownSinglet.repLorentzGroup (d i)
  repLorentz_bard : ∀ i, IsLorentzCovDerivTransforms repLorentz
    DownSinglet.repLorentzGroup.conj (bard i)
  repLorentz_u : ∀ i, IsLorentzCovDerivTransforms repLorentz
    UpSinglet.repLorentzGroup (u i)
  repLorentz_baru : ∀ i, IsLorentzCovDerivTransforms repLorentz
```

def fermionAlgebra [source]

The algebra generated by the ten fermion families and all their covariant derivatives.

```
@[nolint unusedArguments]
def fermionAlgebra (h : IsFermionSector B repGauge hrepGauge_mul repLorentz hrepLorentz_mul
    d bard u baru Q barQ L barL e bare massWeightPoly) : Subalgebra ℂ B
```

def derivSubmodule [source]

The submodule of `B` generated by the fermion symbols carrying exactly `n` covariant derivatives: the join, over the families and derivative slots, of the ranges of the ten species' symbol maps.

```
@[nolint unusedArguments]
noncomputable def derivSubmodule
  (h : IsFermionSector B repGauge hrepGauge_mul repLorentz hrepLorentz_mul
      d bard u baru Q barQ L barL e bare massWeightPoly) (n : ℕ) : Submodule ℂ B
```

lemma derivSubmodule_eq_span [source]

The derivative submodule as the span of the fermion symbol values.

```
lemma derivSubmodule_eq_span (n : ℕ) :
    h.derivSubmodule n = Submodule.span ℂ
      (⋃ (i : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3),
        Set.range (d i l) ∪ Set.range (bard i l) ∪
        Set.range (u i l) ∪ Set.range (baru i l) ∪
        Set.range (Q i l) ∪ Set.range (barQ i l) ∪
        Set.range (L i l) ∪ Set.range (barL i l) ∪
        Set.range (e i l) ∪ Set.range (bare i l))
```

lemma anticomm_of_mem_derivSubmodule [source]

Any two elements of the fermion derivative submodules anticommute: the pairwise anticommutation of the symbols extends bilinearly to the spans.

```
lemma anticomm_of_mem_derivSubmodule {n m : ℕ} {x y : B}
    (hx : x ∈ h.derivSubmodule n) (hy : y ∈ h.derivSubmodule m) :
    x * y = -(y * x)
```

lemma derivSubmodule_mul_comm [source]

The fermion derivative submodules commute with one another as submodules: the sign from anticommutation is absorbed by the span.

```
lemma derivSubmodule_mul_comm (n m : ℕ) :
    h.derivSubmodule n * h.derivSubmodule m = h.derivSubmodule m * h.derivSubmodule n
```

2.461 PhyslibAlpha.Particles.StandardModel.IsFermionSector.Components

PhyslibAlpha/Particles/StandardModel/IsFermionSector/Components.lean on GitHub.

def dComponent [source]

The component `∇_l d_i` of the down-singlet symbol against the basis vector `j` of `DownSinglet`.

```
@[nolint unusedArguments]
noncomputable def dComponent (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) : B
```

def bardComponent [source]

The component `∇_l bard_i` of the conjugate down-singlet symbol against the basis vector `j` of `ConjModule DownSinglet`.

```
@[nolint unusedArguments]
noncomputable def bardComponent (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) : B
```

def uComponent [source]

The component `∇_l u_i` of the up-singlet symbol against the basis vector `j` of `UpSinglet`.

```
@[nolint unusedArguments]
noncomputable def uComponent (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) : B
```

def baruComponent [source]

The component `∇_l baru_i` of the conjugate up-singlet symbol against the basis vector `j` of `ConjModule UpSinglet`.

```
@[nolint unusedArguments]
noncomputable def baruComponent (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) : B
```

def QComponent [source]

The component `∇_l Q_i` of the quark-doublet symbol against the basis vector `j` of `QuarkDoublet`.

```
@[nolint unusedArguments]
noncomputable def QComponent (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
```

```
    (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3 × Fin 2) : B
```

def barQComponent [source]

The component `∇_l barQ_i` of the conjugate quark-doublet symbol against the basis vector `j` of `ConjModule QuarkDoublet`.

```
@[nolint unusedArguments]
noncomputable def barQComponent (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3 × Fin 2) : B
```

def LComponent [source]

The component `∇_l L_i` of the lepton-doublet symbol against the basis vector `j` of `LeptonDoubletV`.

```
@[nolint unusedArguments]
noncomputable def LComponent (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) : B
```

def barLComponent [source]

The component `∇_l barL_i` of the conjugate lepton-doublet symbol against the basis vector `j` of `ConjModule LeptonDoubletV`.

```
@[nolint unusedArguments]
noncomputable def barLComponent (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) : B
```

def eComponent [source]

The component `∇_l e_i` of the lepton-singlet symbol against the basis vector `j` of `LeptonSinglet`.

```
@[nolint unusedArguments]
noncomputable def eComponent (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2) : B
```

def bareComponent [source]

The component `∇_l bare_i` of the conjugate lepton-singlet symbol against the basis vector `j` of `ConjModule LeptonSinglet`.

```
@[nolint unusedArguments]
noncomputable def bareComponent (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    (i : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2) : B
```

lemma rep_dComponent [source]

The gauge transformation of a down-singlet component: the colour index mixes by the transposed `SU(3)` matrix of `g⁻¹`, scaled by the conjugate hypercharge factor.

```
lemma rep_dComponent (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge g (h.dComponent i l j) =
      ∑ c, (star (g⁻¹).toU1.1 ^ 2 * (g⁻¹).toSU3.1 j.2 c) • h.dComponent i l (j.1, c)
```

lemma rep_bardComponent [source]

The gauge transformation of a conjugate down-singlet component: the coefficients of the down-singlet law, conjugated.

```
lemma rep_bardComponent (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge g (h.bardComponent i l j) =
      ∑ c, star (star (g⁻¹).toU1.1 ^ 2 * (g⁻¹).toSU3.1 j.2 c) •
        h.bardComponent i l (j.1, c)
```

lemma rep_uComponent [source]

The gauge transformation of an up-singlet component: the colour index mixes by the transposed `SU(3)` matrix of `g⁻¹`, scaled by the hypercharge factor.

```
lemma rep_uComponent (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge g (h.uComponent i l j) =
      ∑ c, ((g⁻¹).toU1.1 ^ 4 * (g⁻¹).toSU3.1 j.2 c) • h.uComponent i l (j.1, c)
```

lemma rep_baruComponent [source]

The gauge transformation of a conjugate up-singlet component: the coefficients of the up-singlet law, conjugated.

```
lemma rep_baruComponent (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge g (h.baruComponent i l j) =
      ∑ c, star ((g⁻¹).toU1.1 ^ 4 * (g⁻¹).toSU3.1 j.2 c) •
        h.baruComponent i l (j.1, c)
```

lemma rep_QComponent [source]

The gauge transformation of a quark-doublet component: the colour index mixes by the transposed `SU(3)` matrix of `g⁻¹` and the isospin index by the transposed `SU(2)` matrix, scaled by the hypercharge factor.

```
lemma rep_QComponent (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3 × Fin 2) :
    repGauge g (h.QComponent i l j) =
      ∑ c, ∑ w, ((g⁻¹).toU1.1 * (g⁻¹).toSU3.1 j.2.1 c * (g⁻¹).toSU2.1 j.2.2 w) •
        h.QComponent i l (j.1, c, w)
```

lemma rep_barQComponent [source]

The gauge transformation of a conjugate quark-doublet component: the coefficients of the quark-doublet law, conjugated.

```
lemma rep_barQComponent (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3 × Fin 2) :
    repGauge g (h.barQComponent i l j) =
      ∑ c, ∑ w, star ((g⁻¹).toU1.1 * (g⁻¹).toSU3.1 j.2.1 c * (g⁻¹).toSU2.1 j.2.2 w) •
        h.barQComponent i l (j.1, c, w)
```

lemma rep_LComponent [source]

The gauge transformation of a lepton-doublet component: the isospin index mixes by the transposed `SU(2)` matrix of `g⁻¹`, scaled by the conjugate hypercharge factor.

```
lemma rep_LComponent (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) :
    repGauge g (h.LComponent i l j) =
      ∑ w, (star (g⁻¹).toU1.1 ^ 3 * (g⁻¹).toSU2.1 j.2 w) • h.LComponent i l (j.1, w)
```

lemma rep_barLComponent [source]

The gauge transformation of a conjugate lepton-doublet component: the coefficients of the lepton-doublet law, conjugated.

```
lemma rep_barLComponent (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) :
    repGauge g (h.barLComponent i l j) =
      ∑ w, star (star (g⁻¹).toU1.1 ^ 3 * (g⁻¹).toSU2.1 j.2 w) •
        h.barLComponent i l (j.1, w)
```

lemma rep_eComponent [source]

The gauge transformation of a lepton-singlet component: colour and isospin act trivially, so the sum over components collapses to the conjugate hypercharge scalar.

```
lemma rep_eComponent (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2) :
    repGauge g (h.eComponent i l j) =
      (star (g⁻¹).toU1.1 ^ 6 : ℂ) • h.eComponent i l j
```

lemma rep_bareComponent [source]

The gauge transformation of a conjugate lepton-singlet component: the scalar of the lepton-singlet law, conjugated.

```
lemma rep_bareComponent (g : GaugeGroupI) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2) :
    repGauge g (h.bareComponent i l j) =
      star (star (g⁻¹).toU1.1 ^ 6 : ℂ) • h.bareComponent i l j
```

lemma univ_derivIndex_zero [source]

At zero covariant derivatives the assignments of derivative directions form a one-element type, so the Lorentz law of a tower has a single term.

```
lemma univ_derivIndex_zero (l : Fin 0 → Fin 1 ⊕ Fin 3) :
    (Finset.univ : Finset (Fin 0 → Fin 1 ⊕ Fin 3)) = {l}
```

lemma repLorentz_dComponent [source]

The Lorentz transformation of a down-singlet component carrying no derivatives: the right-handed spinor index transforms by the entrywise conjugate of the inverse matrix.

```
lemma repLorentz_dComponent (Λ : SL(2,ℂ)) (i : Fin 3) (l : Fin 0 → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repLorentz Λ (h.dComponent i l j) =
      ∑ β, star ((Λ⁻¹).1 j.1 β) • h.dComponent i l (β, j.2)
```

lemma repLorentz_bardComponent [source]

The Lorentz transformation of a conjugate down-singlet component carrying no derivatives: the coefficients of the down-singlet law, conjugated.

```
lemma repLorentz_bardComponent (Λ : SL(2,ℂ)) (i : Fin 3) (l : Fin 0 → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repLorentz Λ (h.bardComponent i l j) =
      ∑ β, (Λ⁻¹).1 j.1 β • h.bardComponent i l (β, j.2)
```

lemma repLorentz_uComponent [source]

The Lorentz transformation of an up-singlet component carrying no derivatives: the right-handed spinor index transforms by the entrywise conjugate of the inverse matrix.

```
lemma repLorentz_uComponent (Λ : SL(2,ℂ)) (i : Fin 3) (l : Fin 0 → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repLorentz Λ (h.uComponent i l j) =
      ∑ β, star ((Λ⁻¹).1 j.1 β) • h.uComponent i l (β, j.2)
```

lemma repLorentz_baruComponent [source]

The Lorentz transformation of a conjugate up-singlet component carrying no derivatives: the coefficients of the up-singlet law, conjugated.

```
lemma repLorentz_baruComponent (Λ : SL(2,ℂ)) (i : Fin 3) (l : Fin 0 → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repLorentz Λ (h.baruComponent i l j) =
      ∑ β, (Λ⁻¹).1 j.1 β • h.baruComponent i l (β, j.2)
```

lemma repLorentz_QComponent [source]

The Lorentz transformation of a quark-doublet component carrying no derivatives: the left-handed spinor index transforms by the inverse matrix.

```
lemma repLorentz_QComponent (Λ : SL(2,ℂ)) (i : Fin 3) (l : Fin 0 → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3 × Fin 2) :
```

```
    repLorentz Λ (h.QComponent i l j) =
      ∑ β, (Λ⁻¹).1 j.1 β • h.QComponent i l (β, j.2.1, j.2.2)
```

lemma repLorentz_barQComponent [source]

The Lorentz transformation of a conjugate quark-doublet component carrying no derivatives: the coefficients of the quark-doublet law, conjugated.

```
lemma repLorentz_barQComponent (Λ : SL(2,ℂ)) (i : Fin 3) (l : Fin 0 → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3 × Fin 2) :
    repLorentz Λ (h.barQComponent i l j) =
      ∑ β, star ((Λ⁻¹).1 j.1 β) • h.barQComponent i l (β, j.2.1, j.2.2)
```

lemma repLorentz_LComponent [source]

The Lorentz transformation of a lepton-doublet component carrying no derivatives: the left-handed spinor index transforms by the inverse matrix.

```
lemma repLorentz_LComponent (Λ : SL(2,ℂ)) (i : Fin 3) (l : Fin 0 → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 2) :
    repLorentz Λ (h.LComponent i l j) =
      ∑ β, (Λ⁻¹).1 j.1 β • h.LComponent i l (β, j.2)
```

lemma repLorentz_barLComponent [source]

The Lorentz transformation of a conjugate lepton-doublet component carrying no derivatives: the coefficients of the lepton-doublet law, conjugated.

```
lemma repLorentz_barLComponent (Λ : SL(2,ℂ)) (i : Fin 3) (l : Fin 0 → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 2) :
    repLorentz Λ (h.barLComponent i l j) =
      ∑ β, star ((Λ⁻¹).1 j.1 β) • h.barLComponent i l (β, j.2)
```

lemma repLorentz_eComponent [source]

The Lorentz transformation of a lepton-singlet component carrying no derivatives: the right-handed spinor index transforms by the entrywise conjugate of the inverse matrix.

```
lemma repLorentz_eComponent (Λ : SL(2,ℂ)) (i : Fin 3) (l : Fin 0 → Fin 1 ⊕ Fin 3)
    (j : Fin 2) :
    repLorentz Λ (h.eComponent i l j) =
      ∑ β, star ((Λ⁻¹).1 j β) • h.eComponent i l β
```

lemma repLorentz_bareComponent [source]

The Lorentz transformation of a conjugate lepton-singlet component carrying no derivatives: the coefficients of the lepton-singlet law, conjugated.

```
lemma repLorentz_bareComponent (Λ : SL(2,ℂ)) (i : Fin 3) (l : Fin 0 → Fin 1 ⊕ Fin 3)
    (j : Fin 2) :
    repLorentz Λ (h.bareComponent i l j) =
      ∑ β, (Λ⁻¹).1 j β • h.bareComponent i l β
```

lemma expI_inv_coe [source]

The inverse of the unitary `exp i` is its conjugate.

```
lemma _root_.StandardModel.expI_inv_coe : ((expI⁻¹ : unitary ℂ) : ℂ) = star (expI : ℂ)
```

lemma su3ExpIOne_inv_coe [source]

The inverse of the first colour torus generator, `diag (exp (-i), exp i, 1)`.

```
lemma _root_.StandardModel.su3ExpIOne_inv_coe :
    (su3ExpIOne⁻¹ : specialUnitaryGroup (Fin 3) ℂ).1
      = Matrix.diagonal ![star (expI : ℂ), (expI : ℂ), 1]
```

lemma su3ExpITwo_inv_coe [source]

The inverse of the second colour torus generator, `diag (1, exp (-i), exp i)`.

```
lemma _root_.StandardModel.su3ExpITwo_inv_coe :
    (su3ExpITwo⁻¹ : specialUnitaryGroup (Fin 3) ℂ).1
      = Matrix.diagonal ![1, star (expI : ℂ), (expI : ℂ)]
```

lemma repGauge_gaugeTorusGen_dComponent [source]

The `d` components carry the negative of the down-singlet weight.

```
lemma repGauge_gaugeTorusGen_dComponent (t : Fin 4) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge (gaugeTorusGen t) (h.dComponent i l j)
      = ((expI : ℂ) ^ GaugeWeight.coord (-(DownSinglet.valueGaugeWeight j)) t) •
        h.dComponent i l j
```

lemma repGauge_gaugeTorusGen_bardComponent [source]

The `bard` components carry the down-singlet weight itself.

```
lemma repGauge_gaugeTorusGen_bardComponent (t : Fin 4) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge (gaugeTorusGen t) (h.bardComponent i l j)
      = ((expI : ℂ) ^ GaugeWeight.coord (DownSinglet.valueGaugeWeight j) t) •
        h.bardComponent i l j
```

lemma repGauge_gaugeTorusGen_uComponent [source]

The `u` components carry the negative of the up-singlet weight.

```
lemma repGauge_gaugeTorusGen_uComponent (t : Fin 4) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge (gaugeTorusGen t) (h.uComponent i l j)
      = ((expI : ℂ) ^ GaugeWeight.coord (-(UpSinglet.valueGaugeWeight j)) t) •
        h.uComponent i l j
```

lemma repGauge_gaugeTorusGen_baruComponent [source]

The `baru` components carry the up-singlet weight itself.

```
lemma repGauge_gaugeTorusGen_baruComponent (t : Fin 4) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge (gaugeTorusGen t) (h.baruComponent i l j)
      = ((expI : ℂ) ^ GaugeWeight.coord (UpSinglet.valueGaugeWeight j) t) •
        h.baruComponent i l j
```

lemma repGauge_gaugeTorusGen_QComponent [source]

The `Q` components carry the negative of the quark-doublet weight.

```
lemma repGauge_gaugeTorusGen_QComponent (t : Fin 4) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3 × Fin 2) :
    repGauge (gaugeTorusGen t) (h.QComponent i l j)
      = ((expI : ℂ) ^ GaugeWeight.coord (-(QuarkDoublet.valueGaugeWeight j)) t) •
        h.QComponent i l j
```

lemma repGauge_gaugeTorusGen_barQComponent [source]

The `barQ` components carry the quark-doublet weight itself.

```
lemma repGauge_gaugeTorusGen_barQComponent (t : Fin 4) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3 × Fin 2) :
    repGauge (gaugeTorusGen t) (h.barQComponent i l j)
      = ((expI : ℂ) ^ GaugeWeight.coord (QuarkDoublet.valueGaugeWeight j) t) •
        h.barQComponent i l j
```

lemma repGauge_gaugeTorusGen_LComponent [source]

The `L` components carry the negative of the lepton-doublet weight.

```
lemma repGauge_gaugeTorusGen_LComponent (t : Fin 4) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) :
    repGauge (gaugeTorusGen t) (h.LComponent i l j)
      = ((expI : ℂ) ^ GaugeWeight.coord (-(LeptonDoubletV.valueGaugeWeight j)) t) •
        h.LComponent i l j
```

lemma repGauge_gaugeTorusGen_barLComponent [source]

The `barL` components carry the lepton-doublet weight itself.

```
lemma repGauge_gaugeTorusGen_barLComponent (t : Fin 4) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) :
    repGauge (gaugeTorusGen t) (h.barLComponent i l j)
      = ((expI : ℂ) ^ GaugeWeight.coord (LeptonDoubletV.valueGaugeWeight j) t) •
        h.barLComponent i l j
```

lemma repGauge_gaugeTorusGen_eComponent [source]

The `e` components carry the negative of the lepton-singlet weight.

```
lemma repGauge_gaugeTorusGen_eComponent (t : Fin 4) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2) :
    repGauge (gaugeTorusGen t) (h.eComponent i l j)
      = ((expI : ℂ) ^ GaugeWeight.coord (-(LeptonSinglet.valueGaugeWeight j)) t) •
        h.eComponent i l j
```

lemma repGauge_gaugeTorusGen_bareComponent [source]

The `bare` components carry the lepton-singlet weight itself.

```
lemma repGauge_gaugeTorusGen_bareComponent (t : Fin 4) (i : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2) :
    repGauge (gaugeTorusGen t) (h.bareComponent i l j)
      = ((expI : ℂ) ^ GaugeWeight.coord (LeptonSinglet.valueGaugeWeight j) t) •
        h.bareComponent i l j
```

2.462 `PhyslibAlpha.Particles.StandardModel.IsFermionSector.Deriv.Centre`

PhyslibAlpha/Particles/StandardModel/IsFermionSector/Deriv/Centre.lean on GitHub.

lemma range_d_le_centreEigenspace [source]

The `d` symbols carry the sign `-1`.

```
lemma range_d_le_centreEigenspace (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (d f l) ≤ centreEigenspace repLorentz (-1)
```

lemma range_bard_le_centreEigenspace [source]

The `bard` symbols carry the sign `-1`.

```
lemma range_bard_le_centreEigenspace (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (bard f l) ≤ centreEigenspace repLorentz (-1)
```

lemma range_u_le_centreEigenspace [source]

The `u` symbols carry the sign `-1`.

```
lemma range_u_le_centreEigenspace (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (u f l) ≤ centreEigenspace repLorentz (-1)
```

lemma range_baru_le_centreEigenspace [source]

The `baru` symbols carry the sign `-1`.

```
lemma range_baru_le_centreEigenspace (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (baru f l) ≤ centreEigenspace repLorentz (-1)
```

lemma range_Q_le_centreEigenspace [source]

The `Q` symbols carry the sign `-1`.

```
lemma range_Q_le_centreEigenspace (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (Q f l) ≤ centreEigenspace repLorentz (-1)
```

lemma range_barQ_le_centreEigenspace [source]

The `barQ` symbols carry the sign `-1`.

```
lemma range_barQ_le_centreEigenspace (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (barQ f l) ≤ centreEigenspace repLorentz (-1)
```

lemma range_L_le_centreEigenspace [source]

The `L` symbols carry the sign `-1`.

```
lemma range_L_le_centreEigenspace (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (L f l) ≤ centreEigenspace repLorentz (-1)
```

lemma range_barL_le_centreEigenspace [source]

The `barL` symbols carry the sign `-1`.

```
lemma range_barL_le_centreEigenspace (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (barL f l) ≤ centreEigenspace repLorentz (-1)
```

lemma range_e_le_centreEigenspace [source]

The `e` symbols carry the sign `-1`.

```
lemma range_e_le_centreEigenspace (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (e f l) ≤ centreEigenspace repLorentz (-1)
```

lemma range_bare_le_centreEigenspace [source]

The `bare` symbols carry the sign `-1`.

```
lemma range_bare_le_centreEigenspace (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (bare f l) ≤ centreEigenspace repLorentz (-1)
```

lemma derivSubmodule_le_centreEigenspace [source]

***The centre of `SL(2,ℂ)` acts on the fermion derivative submodules by `-1`**, for any number of covariant derivatives: every fermion symbol carries one Weyl-spinor value index, and the derivative slots are inert at the centre.*

```
lemma derivSubmodule_le_centreEigenspace (n : ℕ) :
    h.derivSubmodule n ≤ centreEigenspace repLorentz (-1)
```

2.463 PhyslibAlpha.Particles.StandardModel.IsFermionSector.Deriv.GaugeWeight

PhyslibAlpha/Particles/StandardModel/IsFermionSector/Deriv/GaugeWeight.lean on GitHub.

lemma range_eq_iSup_span [source]

```
lemma range_eq_iSup_span {M : Type} [AddCommGroup M] [Module ℂ M]
    (b : Module.Basis ι ℂ V) (f : Module.Dual ℂ V →ₗ[ℂ] M) :
    LinearMap.range f = ⨆ j, Submodule.span ℂ {f (b.coord j)}
```

lemma repGauge_gaugeTorusGen_d [source]

The gauge torus acts diagonally on the `d` symbol components.

```
lemma repGauge_gaugeTorusGen_d (i : Fin 4) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge (gaugeTorusGen i) (d f l ((DownSinglet.basis).coord j))
      = ((expI : ℂ) ^ GaugeWeight.coord (-(DownSinglet.valueGaugeWeight j)) i) • d f l
```

```
    ((DownSinglet.basis).coord j)
```

lemma repGauge_gaugeTorusGen_bard [source]

The gauge torus acts diagonally on the `bard` symbol components.

```
lemma repGauge_gaugeTorusGen_bard (i : Fin 4) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge (gaugeTorusGen i) (bard f l (((Basis.conj DownSinglet.basis)).coord j))
      = ((expI : ℂ) ^ GaugeWeight.coord (DownSinglet.valueGaugeWeight j) i) • bard f l
        (((Basis.conj DownSinglet.basis)).coord j)
```

lemma repGauge_gaugeTorusGen_u [source]

The gauge torus acts diagonally on the `u` symbol components.

```
lemma repGauge_gaugeTorusGen_u (i : Fin 4) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge (gaugeTorusGen i) (u f l ((UpSinglet.basis).coord j))
      = ((expI : ℂ) ^ GaugeWeight.coord (-(UpSinglet.valueGaugeWeight j)) i) • u f l
        ((UpSinglet.basis).coord j)
```

lemma repGauge_gaugeTorusGen_baru [source]

The gauge torus acts diagonally on the `baru` symbol components.

```
lemma repGauge_gaugeTorusGen_baru (i : Fin 4) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge (gaugeTorusGen i) (baru f l (((Basis.conj UpSinglet.basis)).coord j))
      = ((expI : ℂ) ^ GaugeWeight.coord (UpSinglet.valueGaugeWeight j) i) • baru f l
        (((Basis.conj UpSinglet.basis)).coord j)
```

lemma repGauge_gaugeTorusGen_Q [source]

The gauge torus acts diagonally on the `Q` symbol components.

```
lemma repGauge_gaugeTorusGen_Q (i : Fin 4) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3 × Fin 2) :
    repGauge (gaugeTorusGen i) (Q f l ((QuarkDoublet.basis).coord j))
      = ((expI : ℂ) ^ GaugeWeight.coord (-(QuarkDoublet.valueGaugeWeight j)) i) • Q f l
        ((QuarkDoublet.basis).coord j)
```

lemma repGauge_gaugeTorusGen_barQ [source]

The gauge torus acts diagonally on the `barQ` symbol components.

```
lemma repGauge_gaugeTorusGen_barQ (i : Fin 4) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3 × Fin 2) :
    repGauge (gaugeTorusGen i) (barQ f l (((Basis.conj QuarkDoublet.basis)).coord j))
      = ((expI : ℂ) ^ GaugeWeight.coord (QuarkDoublet.valueGaugeWeight j) i) • barQ f l
        (((Basis.conj QuarkDoublet.basis)).coord j)
```

lemma repGauge_gaugeTorusGen_L [source]

The gauge torus acts diagonally on the `L` symbol components.

```
lemma repGauge_gaugeTorusGen_L (i : Fin 4) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) :
    repGauge (gaugeTorusGen i) (L f l ((LeptonDoubletV.basis).coord j))
      = ((expI : ℂ) ^ GaugeWeight.coord (-(LeptonDoubletV.valueGaugeWeight j)) i) • L f l
        ((LeptonDoubletV.basis).coord j)
```

lemma repGauge_gaugeTorusGen_barL [source]

The gauge torus acts diagonally on the `barL` symbol components.

```
lemma repGauge_gaugeTorusGen_barL (i : Fin 4) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) :
    repGauge (gaugeTorusGen i) (barL f l (((Basis.conj LeptonDoubletV.basis)).coord j))
      = ((expI : ℂ) ^ GaugeWeight.coord (LeptonDoubletV.valueGaugeWeight j) i) • barL f l
        (((Basis.conj LeptonDoubletV.basis)).coord j)
```

lemma repGauge_gaugeTorusGen_e [source]

The gauge torus acts diagonally on the `e` symbol components.

```
lemma repGauge_gaugeTorusGen_e (i : Fin 4) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2) :
    repGauge (gaugeTorusGen i) (e f l ((LeptonSinglet.basis).coord j))
      = ((expI : ℂ) ^ GaugeWeight.coord (-(LeptonSinglet.valueGaugeWeight j)) i) • e f l
        ((LeptonSinglet.basis).coord j)
```

lemma repGauge_gaugeTorusGen_bare [source]

The gauge torus acts diagonally on the `bare` symbol components.

```
lemma repGauge_gaugeTorusGen_bare (i : Fin 4) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2) :
    repGauge (gaugeTorusGen i) (bare f l (((Basis.conj LeptonSinglet.basis)).coord j))
      = ((expI : ℂ) ^ GaugeWeight.coord (LeptonSinglet.valueGaugeWeight j) i) • bare f l
        (((Basis.conj LeptonSinglet.basis)).coord j)
```

def rangeGaugeWeightD [source]

The gauge weight decomposition of the range of the `d` symbols.

```
@[implicit_reducible]
noncomputable def rangeGaugeWeightD (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge (LinearMap.range (d f l))
```

def rangeGaugeWeightBarD [source]

The gauge weight decomposition of the range of the `bard` symbols.

```
@[implicit_reducible]
noncomputable def rangeGaugeWeightBarD (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge (LinearMap.range (bard f l))
```

def rangeGaugeWeightU [source]

The gauge weight decomposition of the range of the `u` symbols.

```
@[implicit_reducible]
noncomputable def rangeGaugeWeightU (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge (LinearMap.range (u f l))
```

def rangeGaugeWeightBarU [source]

The gauge weight decomposition of the range of the `baru` symbols.

```
@[implicit_reducible]
noncomputable def rangeGaugeWeightBarU (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge (LinearMap.range (baru f l))
```

def rangeGaugeWeightQ [source]

The gauge weight decomposition of the range of the `Q` symbols.

```
@[implicit_reducible]
noncomputable def rangeGaugeWeightQ (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge (LinearMap.range (Q f l))
```

def rangeGaugeWeightBarQ [source]

The gauge weight decomposition of the range of the `barQ` symbols.

```
@[implicit_reducible]
noncomputable def rangeGaugeWeightBarQ (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge (LinearMap.range (barQ f l))
```

def rangeGaugeWeightL [source]

The gauge weight decomposition of the range of the `L` symbols.

```
@[implicit_reducible]
noncomputable def rangeGaugeWeightL (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge (LinearMap.range (L f l))
```

def rangeGaugeWeightBarL [source]

The gauge weight decomposition of the range of the `barL` symbols.

```
@[implicit_reducible]
noncomputable def rangeGaugeWeightBarL (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge (LinearMap.range (barL f l))
```

def rangeGaugeWeightE [source]

The gauge weight decomposition of the range of the `e` symbols.

```
@[implicit_reducible]
noncomputable def rangeGaugeWeightE (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge (LinearMap.range (e f l))
```

def rangeGaugeWeightBarE [source]

The gauge weight decomposition of the range of the `bare` symbols.

```
@[implicit_reducible]
noncomputable def rangeGaugeWeightBarE (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge (LinearMap.range (bare f l))
```

instance derivSubmoduleGaugeWeight [source]

***The gauge weight decomposition of the fermion derivative submodules**, for any number of covariant derivatives: the join, over families, derivative slots and the ten species, of the spans of the symbol components, each of pure gauge weight.*

This is an instance: its statement mentions `h`, so unification against the goal recovers the sector and with it all the implicit data of `IsFermionSector`. The `rangeGaugeWeight*` decompositions above cannot be instances for exactly that reason — their statements name only the symbol maps, leaving the rest of the structure's parameters undetermined.

```
@[implicit_reducible]
noncomputable instance derivSubmoduleGaugeWeight (n : ℕ) :
    GaugeWeightDecomposition repGauge (h.derivSubmodule n)
```

def fermionGaugeWeights [source]

The gauge weights carried by the fermion symbols: for each species the image of its value weights, negated for the unbarred species (the symbols pair with the dual of the value space) and taken as they are for the barred ones.

```
def fermionGaugeWeights : Finset GaugeWeight
```

lemma rangeGaugeWeightD_supp [source]

The support of the `d` range decomposition.

```
lemma rangeGaugeWeightD_supp (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeightD f l).supp
      = Finset.univ.image (fun j : Fin 2 × Fin 3 => -(DownSinglet.valueGaugeWeight j))
```

lemma rangeGaugeWeightBarD_supp [source]

The support of the `bard` range decomposition.

```
lemma rangeGaugeWeightBarD_supp (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeightBarD f l).supp
      = Finset.univ.image (fun j : Fin 2 × Fin 3 => DownSinglet.valueGaugeWeight j)
```

lemma rangeGaugeWeightU_supp [source]

The support of the `u` range decomposition.

```
lemma rangeGaugeWeightU_supp (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeightU f l).supp
      = Finset.univ.image (fun j : Fin 2 × Fin 3 => -(UpSinglet.valueGaugeWeight j))
```

lemma rangeGaugeWeightBarU_supp [source]

The support of the `baru` range decomposition.

```
lemma rangeGaugeWeightBarU_supp (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeightBarU f l).supp
      = Finset.univ.image (fun j : Fin 2 × Fin 3 => UpSinglet.valueGaugeWeight j)
```

lemma rangeGaugeWeightQ_supp [source]

The support of the `Q` range decomposition.

```
lemma rangeGaugeWeightQ_supp (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeightQ f l).supp
      = Finset.univ.image (fun j : Fin 2 × Fin 3 × Fin 2 => -(QuarkDoublet.valueGaugeWeight j))
```

lemma rangeGaugeWeightBarQ_supp [source]

The support of the `barQ` range decomposition.

```
lemma rangeGaugeWeightBarQ_supp (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeightBarQ f l).supp
      = Finset.univ.image (fun j : Fin 2 × Fin 3 × Fin 2 => QuarkDoublet.valueGaugeWeight j)
```

lemma rangeGaugeWeightL_supp [source]

The support of the `L` range decomposition.

```
lemma rangeGaugeWeightL_supp (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeightL f l).supp
      = Finset.univ.image (fun j : Fin 2 × Fin 2 => -(LeptonDoubletV.valueGaugeWeight j))
```

lemma rangeGaugeWeightBarL_supp [source]

The support of the `barL` range decomposition.

```
lemma rangeGaugeWeightBarL_supp (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeightBarL f l).supp
      = Finset.univ.image (fun j : Fin 2 × Fin 2 => LeptonDoubletV.valueGaugeWeight j)
```

lemma rangeGaugeWeightE_supp [source]

The support of the `e` range decomposition.

```
lemma rangeGaugeWeightE_supp (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeightE f l).supp
      = Finset.univ.image (fun j : Fin 2 => -(LeptonSinglet.valueGaugeWeight j))
```

lemma rangeGaugeWeightBarE_supp [source]

The support of the `bare` range decomposition.

```
lemma rangeGaugeWeightBarE_supp (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeightBarE f l).supp
      = Finset.univ.image (fun j : Fin 2 => LeptonSinglet.valueGaugeWeight j)
```

lemma derivSubmoduleGaugeWeight_supp [source]

***The support of the gauge weight decomposition of the fermion derivative submodules**: the gauge weights of the ten species, independent of the number of covariant derivatives.*

```
lemma derivSubmoduleGaugeWeight_supp (n : ℕ) :
    (h.derivSubmoduleGaugeWeight n).supp = fermionGaugeWeights
```

2.464 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.Basic

PhyslibAlpha/Particles/StandardModel/IsFermionSector/MassWeight/Basic.lean on GitHub.

def massWeightSubmodule [source]

All elements of the fermion algebra of mass weight exactly `w`: the intersection of the algebra generated by the fermion towers with the part on which `massWeightPoly` is the monomial `X ^ w`.

```
noncomputable def massWeightSubmodule
  (h : IsFermionSector B repGauge hrepGauge_mul repLorentz hrepLorentz_mul
      d bard u baru Q barQ L barL e bare massWeightPoly) (w : ℕ) :
    Submodule ℂ B
```

lemma massWeightPoly_of_mem_massWeightSubmodule [source]

```
lemma massWeightPoly_of_mem_massWeightSubmodule {w : ℕ} {x : B}
    (hx : x ∈ h.massWeightSubmodule w) :
    massWeightPoly x = Polynomial.monomial w x
```

lemma mem_fermionAlgebra_of_mem_massWeightSubmodule [source]

```
lemma mem_fermionAlgebra_of_mem_massWeightSubmodule {w : ℕ} {x : B}
    (hx : x ∈ h.massWeightSubmodule w) : x ∈ h.fermionAlgebra
```

lemma one_le_massWeightSubmodule_zero [source]

```
lemma one_le_massWeightSubmodule_zero : (1 : Submodule ℂ B) ≤ h.massWeightSubmodule 0
```

lemma massWeightSubmodule_mul_le [source]

```
lemma massWeightSubmodule_mul_le (m n : ℕ) :
    h.massWeightSubmodule m * h.massWeightSubmodule n ≤ h.massWeightSubmodule (m + n)
```

lemma fermionAlgebra_eq_adjoin_derivSubmodule [source]

The fermion algebra is generated by the fermion towers of every derivative order.

```
lemma fermionAlgebra_eq_adjoin_derivSubmodule :
    h.fermionAlgebra = Algebra.adjoin ℂ (⋃ n, (h.derivSubmodule n : Set B))
```

lemma massWeightPoly_of_mem_derivSubmodule [source]

*massWeightPoly is the monomial X ^ (3 + 2 * n) on the fermion towers with n covariant derivatives.*

```
lemma massWeightPoly_of_mem_derivSubmodule (n : ℕ) :
    ∀ x ∈ h.derivSubmodule n, massWeightPoly x = Polynomial.monomial (3 + 2 * n) x
```

lemma derivSubmodule_le_massWeightSubmodule [source]

*A fermion tower with n covariant derivatives has mass weight 3 + 2 * n.*

```
lemma derivSubmodule_le_massWeightSubmodule (n : ℕ) :
    h.derivSubmodule n ≤ h.massWeightSubmodule (3 + 2 * n)
```

lemma massWeightSubmodule_zero_eq [source]

Weight zero is the scalars: every fermion tower has positive weight.

```
lemma massWeightSubmodule_zero_eq : h.massWeightSubmodule 0 = 1
```

lemma massWeightSubmodule_eq [source]

The weight recursion: an element of positive mass weight i is a sum of single fermion towers of weight i and of products of two elements of lower positive weights summing to i.

```
lemma massWeightSubmodule_eq (i : ℕ) (hi : 0 < i) :
    h.massWeightSubmodule i
      = (⨆ k ∈ Finset.univ.filter (fun k : Fin i => 3 + 2 * (k : ℕ) = i),
          h.derivSubmodule (k : ℕ))
        ⊔ (⨆ p ∈ Finset.univ.filter (fun p : Fin i × Fin i => (p.1 : ℕ) + (p.2 : ℕ) = i),
            h.massWeightSubmodule (p.1 : ℕ) * h.massWeightSubmodule (p.2 : ℕ))
```

lemma massWeightSubmodule_eq_iSup_mul [source]

*Removing the leftmost fermion tower: an element of positive weight w is a sum of products of a tower $\nabla^n \psi$ of weight 3 + 2 * n ≤ w with an element of the remaining weight.*

```
lemma massWeightSubmodule_eq_iSup_mul (w : ℕ) (hw : 0 < w) :
    h.massWeightSubmodule w
      = ⨆ n ∈ (Finset.range (w + 1)).filter (fun n => 3 + 2 * n ≤ w),
```

```
        h.derivSubmodule n * h.massWeightSubmodule (w - (3 + 2 * n))
```

lemma massWeightSubmodule_eq_bot [source]

The weights `1`, `2` and `4` are empty: the tower weights `3`, `5`, `7`, … and their sums only reach `0`, `3` and every weight from `5` on.

```
lemma massWeightSubmodule_eq_bot {w : ℕ} (hw : ¬ (w = 0 ∨ w = 3 ∨ 5 ≤ w)) :
    h.massWeightSubmodule w = ⊥
```

lemma massWeightSubmodule_one_eq [source]

There is nothing of weight one.

```
lemma massWeightSubmodule_one_eq : h.massWeightSubmodule 1 = ⊥
```

lemma massWeightSubmodule_two_eq [source]

There is nothing of weight two.

```
lemma massWeightSubmodule_two_eq : h.massWeightSubmodule 2 = ⊥
```

lemma massWeightSubmodule_three_eq [source]

Weight three is the underived fermion towers: `ψ` leaves weight zero.

```
lemma massWeightSubmodule_three_eq :
    h.massWeightSubmodule 3 = h.derivSubmodule 0
```

lemma massWeightSubmodule_four_eq [source]

There is nothing of weight four.

```
lemma massWeightSubmodule_four_eq : h.massWeightSubmodule 4 = ⊥
```

lemma massWeightSubmodule_five_eq [source]

Weight five is the once-derived fermion towers: `ψ` would leave weight two, which is empty, and `∇ψ` leaves weight zero.

```
lemma massWeightSubmodule_five_eq :
    h.massWeightSubmodule 5 = h.derivSubmodule 1
```

lemma massWeightSubmodule_six_eq [source]

Weight six is the products of two underived fermion towers: `ψ` leaves weight three, which is `ψ`, and `∇ψ` would leave weight one, which is empty.

```
lemma massWeightSubmodule_six_eq :
    h.massWeightSubmodule 6 = h.derivSubmodule 0 * h.derivSubmodule 0
```

lemma massWeightSubmodule_seven_eq [source]

Weight seven is the twice-derived fermion towers: ψ and $\nabla\psi$ would leave weights four and two, which are empty, and $\nabla\nabla\psi$ leaves weight zero.

```
lemma massWeightSubmodule_seven_eq :
    h.massWeightSubmodule 7 = h.derivSubmodule 2
```

lemma massWeightSubmodule_eight_eq [source]

Weight eight is the products of an underived and a once-derived fermion tower, the kinetic-term sector: ψ leaves $\nabla\psi$, $\nabla\psi$ leaves ψ, and $\nabla\nabla\psi$ would leave weight one, which is empty. The two orders agree by `derivSubmodule_mul_comm`.

```
lemma massWeightSubmodule_eight_eq :
    h.massWeightSubmodule 8 = h.derivSubmodule 0 * h.derivSubmodule 1
```

2.465 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.GaugeWeight

PhyslibAlpha/Particles/StandardModel/IsFermionSector/MassWeight/GaugeWeight.lean on GitHub.

def massWeightSubmoduleGaugeWeightOne [source]

Weight one is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightOne :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 1)
```

def massWeightSubmoduleGaugeWeightTwo [source]

Weight two is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightTwo :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 2)
```

def massWeightSubmoduleGaugeWeightThree [source]

Weight three is the underived fermion towers.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightThree :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 3)
```

def massWeightSubmoduleGaugeWeightFour [source]

Weight four is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightFour :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 4)
```

def massWeightSubmoduleGaugeWeightFive [source]

Weight five is the once-derived fermion towers.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightFive :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 5)
```

def massWeightSubmoduleGaugeWeightSix [source]

Weight six is the products of two underived fermion towers.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightSix :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 6)
```

def massWeightSubmoduleGaugeWeightSeven [source]

Weight seven is the twice-derived fermion towers.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightSeven :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 7)
```

def massWeightSubmoduleGaugeWeightEight [source]

Weight eight is the kinetic sector: an underived tower against a once-derived one.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightEight :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 8)
```

lemma zero_not_mem_fermionGaugeWeights [source]

Every gauge weight carried by a fermion symbol has nonzero hypercharge: each of the ten species has a fixed nonzero hypercharge, independent of colour, isospin and family, and the barred species carry the negative of the unbarred one. So the zero weight never occurs.

```
lemma zero_not_mem_fermionGaugeWeights : (0 : GaugeWeight) ∉ fermionGaugeWeights
```

lemma derivSubmoduleGaugeWeight_piece_zero [source]

The weight-zero piece of the fermion derivative submodules is trivial: unlike the gauge sector, no single fermion symbol is a gauge singlet, since every one of the ten species carries a fixed nonzero hypercharge. A gauge-invariant combination needs at least two fermion insertions, which is why it is the mass weights six and eight, the products of two towers, that carry the interesting weight-zero content.

```
lemma derivSubmoduleGaugeWeight_piece_zero (n : ℕ) :
    (h.derivSubmoduleGaugeWeight n).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightOne_piece_zero [source]

The weight-zero piece at mass weight 1: the submodule itself is trivial.

```
lemma massWeightSubmoduleGaugeWeightOne_piece_zero :
    (h.massWeightSubmoduleGaugeWeightOne).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightTwo_piece_zero [source]

The weight-zero piece at mass weight 2: the submodule itself is trivial.

```
lemma massWeightSubmoduleGaugeWeightTwo_piece_zero :
    (h.massWeightSubmoduleGaugeWeightTwo).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightThree_piece_zero [source]

The weight-zero piece at mass weight three: the underived fermion towers carry no gauge singlet, since every fermion symbol has nonzero hypercharge.

```
lemma massWeightSubmoduleGaugeWeightThree_piece_zero :
    (h.massWeightSubmoduleGaugeWeightThree).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightFour_piece_zero [source]

The weight-zero piece at mass weight 4: the submodule itself is trivial.

```
lemma massWeightSubmoduleGaugeWeightFour_piece_zero :
    (h.massWeightSubmoduleGaugeWeightFour).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightFive_piece_zero [source]

The weight-zero piece at mass weight five: the once-derived fermion towers carry no gauge singlet, since every fermion symbol has nonzero hypercharge.

```
lemma massWeightSubmoduleGaugeWeightFive_piece_zero :
    (h.massWeightSubmoduleGaugeWeightFive).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightSeven_piece_zero [source]

The weight-zero piece at mass weight seven: the twice-derived fermion towers carry no gauge singlet, since every fermion symbol has nonzero hypercharge.

```
lemma massWeightSubmoduleGaugeWeightSeven_piece_zero :
    (h.massWeightSubmoduleGaugeWeightSeven).piece 0 = ⊥
```

lemma piece_sup_mul [source]

If the left factor of a product decomposition is a join `VA ⊔ VB`, its weight-`w` piece splits along the join.

```
lemma piece_sup_mul {VA VB VC : Submodule ℂ B}
    (dA : GaugeWeightDecomposition repGauge VA) (dB : GaugeWeightDecomposition repGauge VB)
    (dC : GaugeWeightDecomposition repGauge VC) (w : GaugeWeight) :
    (GaugeWeightDecomposition.mul (d := GaugeWeightDecomposition.sup (d := dA) (d' := dB))
        (d' := dC)).piece w
```

```
      = (GaugeWeightDecomposition.mul (d := dA) (d' := dC)).piece w
        ⊔ (GaugeWeightDecomposition.mul (d := dB) (d' := dC)).piece w
```

lemma piece_mul_sup [source]

If the right factor of a product decomposition is a join `VA ⊔ VB`, its weight-`w` piece splits along the join.

```
lemma piece_mul_sup {VA VB VC : Submodule ℂ B}
    (dC : GaugeWeightDecomposition repGauge VC) (dA : GaugeWeightDecomposition repGauge VA)
    (dB : GaugeWeightDecomposition repGauge VB) (w : GaugeWeight) :
    (GaugeWeightDecomposition.mul (d := dC)
        (d' := GaugeWeightDecomposition.sup (d := dA) (d' := dB))).piece w
      = (GaugeWeightDecomposition.mul (d := dC) (d' := dA)).piece w
        ⊔ (GaugeWeightDecomposition.mul (d := dC) (d' := dB)).piece w
```

lemma mul_piece_zero_eq_bot_of_hypercharge [source]

Two decompositions with constant, non-cancelling hypercharge across their whole supports have a trivial product at weight zero: a weight from one can never cancel a weight from the other.

```
lemma mul_piece_zero_eq_bot_of_hypercharge {V V' : Submodule ℂ B}
    {dV : GaugeWeightDecomposition repGauge V} {dV' : GaugeWeightDecomposition repGauge V'}
    {hc hc' : ℤ} (hV : ∀ w ∈ dV.supp, w.2.2.2 = hc) (hV' : ∀ w ∈ dV'.supp, w.2.2.2 = hc')
    (hne : hc + hc' ≠ 0) :
    (GaugeWeightDecomposition.mul (d := dV) (d' := dV')).piece 0 = ⊥
```

lemma rangeGaugeWeightD_hc [source]

The `d` symbols carry hypercharge `2` (the negative of the down-singlet's `-2`), independent of colour and family.

```
lemma rangeGaugeWeightD_hc {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    ∀ w ∈ (h.rangeGaugeWeightD f l).supp, w.2.2.2 = 2
```

lemma rangeGaugeWeightBarD_hc [source]

The `bard` symbols carry hypercharge `-2`, independent of colour and family.

```
lemma rangeGaugeWeightBarD_hc {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    ∀ w ∈ (h.rangeGaugeWeightBarD f l).supp, w.2.2.2 = -2
```

lemma rangeGaugeWeightU_hc [source]

The `u` symbols carry hypercharge `-4`, independent of colour and family.

```
lemma rangeGaugeWeightU_hc {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    ∀ w ∈ (h.rangeGaugeWeightU f l).supp, w.2.2.2 = -4
```

lemma rangeGaugeWeightBarU_hc [source]

The `baru` symbols carry hypercharge `4`, independent of colour and family.

```
lemma rangeGaugeWeightBarU_hc {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    ∀ w ∈ (h.rangeGaugeWeightBarU f l).supp, w.2.2.2 = 4
```

lemma rangeGaugeWeightQ_hc [source]

The `Q` symbols carry hypercharge `-1`, independent of colour, isospin and family.

```
lemma rangeGaugeWeightQ_hc {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    ∀ w ∈ (h.rangeGaugeWeightQ f l).supp, w.2.2.2 = -1
```

lemma rangeGaugeWeightBarQ_hc [source]

The `barQ` symbols carry hypercharge `1`, independent of colour, isospin and family.

```
lemma rangeGaugeWeightBarQ_hc {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    ∀ w ∈ (h.rangeGaugeWeightBarQ f l).supp, w.2.2.2 = 1
```

lemma rangeGaugeWeightL_hc [source]

The `L` symbols carry hypercharge `3`, independent of isospin and family.

```
lemma rangeGaugeWeightL_hc {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    ∀ w ∈ (h.rangeGaugeWeightL f l).supp, w.2.2.2 = 3
```

lemma rangeGaugeWeightBarL_hc [source]

The `barL` symbols carry hypercharge `-3`, independent of isospin and family.

```
lemma rangeGaugeWeightBarL_hc {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    ∀ w ∈ (h.rangeGaugeWeightBarL f l).supp, w.2.2.2 = -3
```

lemma rangeGaugeWeightE_hc [source]

The `e` symbols carry hypercharge `6`, independent of family.

```
lemma rangeGaugeWeightE_hc {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    ∀ w ∈ (h.rangeGaugeWeightE f l).supp, w.2.2.2 = 6
```

lemma rangeGaugeWeightBarE_hc [source]

The `bare` symbols carry hypercharge `-6`, independent of family.

```
lemma rangeGaugeWeightBarE_hc {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    ∀ w ∈ (h.rangeGaugeWeightBarE f l).supp, w.2.2.2 = -6
```

def speciesGaugeWeight [source]

The gauge weight decomposition of one family's full set of symbols at fixed derivative slots, matching the recipe of `derivSubmodule` itself: the join of the ten species' ranges.

```
@[implicit_reducible]
noncomputable def speciesGaugeWeight (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge
      (LinearMap.range (d f l) ⊔ LinearMap.range (bard f l) ⊔
```

```
      LinearMap.range (u f l) ⊔ LinearMap.range (baru f l) ⊔
      LinearMap.range (Q f l) ⊔ LinearMap.range (barQ f l) ⊔
      LinearMap.range (L f l) ⊔ LinearMap.range (barL f l) ⊔
      LinearMap.range (e f l) ⊔ LinearMap.range (bare f l))
```

lemma speciesGaugeWeight_mul_piece_zero [source]

The weight-zero piece of the product of two families' full symbol sets collapses to the ten conjugate pairings: every other combination of species has hypercharges that cannot cancel.

```
lemma speciesGaugeWeight_mul_piece_zero {n m : ℕ} (f f' : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3)
    (l' : Fin m → Fin 1 ⊕ Fin 3) :
    (GaugeWeightDecomposition.mul (d := h.speciesGaugeWeight f l)
        (d' := h.speciesGaugeWeight f' l')).piece 0
      =
      (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightD f l)
            (d' := h.rangeGaugeWeightBarD f' l')).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarD f l)
            (d' := h.rangeGaugeWeightD f' l')).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightU f l)
            (d' := h.rangeGaugeWeightBarU f' l')).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarU f l)
            (d' := h.rangeGaugeWeightU f' l')).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightQ f l)
            (d' := h.rangeGaugeWeightBarQ f' l')).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarQ f l)
            (d' := h.rangeGaugeWeightQ f' l')).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightL f l)
            (d' := h.rangeGaugeWeightBarL f' l')).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarL f l)
            (d' := h.rangeGaugeWeightL f' l')).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightE f l)
            (d' := h.rangeGaugeWeightBarE f' l')).piece 0
        ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarE f l)
            (d' := h.rangeGaugeWeightE f' l')).piece 0
```

lemma iSup_fin_zero_eq [source]

The zero-index derivative slot collapses a supremum over it to its value: there is nothing to derive with respect to.

```
lemma iSup_fin_zero_eq {α : Type} [CompleteLattice α] (F : (Fin 0 → Fin 1 ⊕ Fin 3) → α) :
    ⨆ l, F l = F ![]
```

lemma derivSubmodule_zero_eq [source]

The underived derivative submodule as a join over families alone, the trivial derivative slot dropped.

```
lemma derivSubmodule_zero_eq :
    h.derivSubmodule 0 = ⨆ (f : Fin 3),
      (LinearMap.range (d f ![]) ⊔
        LinearMap.range (bard f ![]) ⊔
        LinearMap.range (u f ![]) ⊔
        LinearMap.range (baru f ![]) ⊔
```

```
      LinearMap.range (Q f ![]) ⊔
      LinearMap.range (barQ f ![]) ⊔
      LinearMap.range (L f ![]) ⊔
      LinearMap.range (barL f ![]) ⊔
      LinearMap.range (e f ![]) ⊔
      LinearMap.range (bare f ![]))
```

lemma massWeightSubmoduleGaugeWeightSix_piece_zero [source]

The weight-zero piece at mass weight six, written out in the mass terms themselves: the join, over pairs of families, of the ten ways to pair each species with its own conjugate. Every other pairing of species has hypercharges that cannot cancel, by `speciesGaugeWeight_mul_piece_zero`.

```
lemma massWeightSubmoduleGaugeWeightSix_piece_zero :
    (h.massWeightSubmoduleGaugeWeightSix).piece 0
      = ⨆ (f : Fin 3) (f' : Fin 3),
        (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightD f ![])
              (d' := h.rangeGaugeWeightBarD f' ![])).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarD f ![])
              (d' := h.rangeGaugeWeightD f' ![])).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightU f ![])
              (d' := h.rangeGaugeWeightBarU f' ![])).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarU f ![])
              (d' := h.rangeGaugeWeightU f' ![])).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightQ f ![])
              (d' := h.rangeGaugeWeightBarQ f' ![])).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarQ f ![])
              (d' := h.rangeGaugeWeightQ f' ![])).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightL f ![])
              (d' := h.rangeGaugeWeightBarL f' ![])).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarL f ![])
              (d' := h.rangeGaugeWeightL f' ![])).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightE f ![])
              (d' := h.rangeGaugeWeightBarE f' ![])).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarE f ![])
              (d' := h.rangeGaugeWeightE f' ![])).piece 0
```

lemma massWeightSubmoduleGaugeWeightEight_piece_zero [source]

The weight-zero piece at mass weight eight, written out in the kinetic terms themselves: the join, over pairs of families and a once-derived slot, of the ten ways to pair each species with its own conjugate. Every other pairing of species has hypercharges that cannot cancel, by `speciesGaugeWeight_mul_piece_zero`.

```
lemma massWeightSubmoduleGaugeWeightEight_piece_zero :
    (h.massWeightSubmoduleGaugeWeightEight).piece 0
      = ⨆ (f : Fin 3) (f' : Fin 3) (l' : Fin 1 → Fin 1 ⊕ Fin 3),
        (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightD f ![])
              (d' := h.rangeGaugeWeightBarD f' l')).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarD f ![])
              (d' := h.rangeGaugeWeightD f' l')).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightU f ![])
              (d' := h.rangeGaugeWeightBarU f' l')).piece 0
          ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarU f ![])
              (d' := h.rangeGaugeWeightU f' l')).piece 0
```

```
    ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightQ f ![])
        (d' := h.rangeGaugeWeightBarQ f' l')).piece 0
    ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarQ f ![])
        (d' := h.rangeGaugeWeightQ f' l')).piece 0
    ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightL f ![])
        (d' := h.rangeGaugeWeightBarL f' l')).piece 0
    ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarL f ![])
        (d' := h.rangeGaugeWeightL f' l')).piece 0
    ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightE f ![])
        (d' := h.rangeGaugeWeightBarE f' l')).piece 0
    ⊔ (GaugeWeightDecomposition.mul (d := h.rangeGaugeWeightBarE f ![])
        (d' := h.rangeGaugeWeightE f' l')).piece 0
```

lemma mem_of_invariant_of_mem_derivSubmodule_sup [source]

A gauge-invariant element of `h.derivSubmodule n ⊔ S`, for any submodule `S` closed under the gauge action, already lies in `S`. The fermionic part carries no gauge singlet, since each of the ten species has a fixed nonzero hypercharge, so it cannot survive; what is left is the part in `S`. Compare `derivSubmoduleGaugeWeight_piece_zero`.

```
lemma mem_of_invariant_of_mem_derivSubmodule_sup {n : ℕ} {S : Submodule ℂ B}
    (hS : ∀ (g : GaugeGroupI) (y : B), y ∈ S → repGauge g y ∈ S)
    {x : B} (hx : x ∈ h.derivSubmodule n ⊔ S)
    (hinv : ∀ g : GaugeGroupI, repGauge g x = x) : x ∈ S
```

lemma mem_of_mem_massWeightSubmoduleOne_sup [source]

Mass weight one contributes nothing to a join: the submodule is trivial, so no invariance hypothesis is needed.

```
lemma mem_of_mem_massWeightSubmoduleOne_sup {S : Submodule ℂ B}
    {x : B} (hx : x ∈ h.massWeightSubmodule 1 ⊔ S) : x ∈ S
```

lemma mem_of_mem_massWeightSubmoduleTwo_sup [source]

Mass weight two contributes nothing to a join: the submodule is trivial, so no invariance hypothesis is needed.

```
lemma mem_of_mem_massWeightSubmoduleTwo_sup {S : Submodule ℂ B}
    {x : B} (hx : x ∈ h.massWeightSubmodule 2 ⊔ S) : x ∈ S
```

lemma mem_of_mem_massWeightSubmoduleFour_sup [source]

Mass weight four contributes nothing to a join: the submodule is trivial, so no invariance hypothesis is needed.

```
lemma mem_of_mem_massWeightSubmoduleFour_sup {S : Submodule ℂ B}
    {x : B} (hx : x ∈ h.massWeightSubmodule 4 ⊔ S) : x ∈ S
```

lemma mem_of_invariant_of_mem_massWeightSubmoduleThree_sup [source]

A gauge-invariant element of `h.massWeightSubmodule 3 ⊔ S`, for `S` closed under the gauge action, lies in `S`: mass weight three is the underived fermion towers, which carry no gauge singlet.

```
lemma mem_of_invariant_of_mem_massWeightSubmoduleThree_sup {S : Submodule ℂ B}
    (hS : ∀ (g : GaugeGroupI) (y : B), y ∈ S → repGauge g y ∈ S)
    {x : B} (hx : x ∈ h.massWeightSubmodule 3 ⊔ S)
    (hinv : ∀ g : GaugeGroupI, repGauge g x = x) : x ∈ S
```

lemma mem_of_invariant_of_mem_massWeightSubmoduleFive_sup [source]

A gauge-invariant element of `h.massWeightSubmodule 5 ⊔ S`, for `S` closed under the gauge action, lies in `S`: mass weight five is the once-derived fermion towers, which carry no gauge singlet.

```
lemma mem_of_invariant_of_mem_massWeightSubmoduleFive_sup {S : Submodule ℂ B}
    (hS : ∀ (g : GaugeGroupI) (y : B), y ∈ S → repGauge g y ∈ S)
    {x : B} (hx : x ∈ h.massWeightSubmodule 5 ⊔ S)
    (hinv : ∀ g : GaugeGroupI, repGauge g x = x) : x ∈ S
```

lemma mem_of_invariant_of_mem_massWeightSubmoduleSeven_sup [source]

A gauge-invariant element of `h.massWeightSubmodule 7 ⊔ S`, for `S` closed under the gauge action, lies in `S`: mass weight seven is the twice-derived fermion towers, which carry no gauge singlet.

```
lemma mem_of_invariant_of_mem_massWeightSubmoduleSeven_sup {S : Submodule ℂ B}
    (hS : ∀ (g : GaugeGroupI) (y : B), y ∈ S → repGauge g y ∈ S)
    {x : B} (hx : x ∈ h.massWeightSubmodule 7 ⊔ S)
    (hinv : ∀ g : GaugeGroupI, repGauge g x = x) : x ∈ S
```

2.466 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.KineticFamilies

PhyslibAlpha/Particles/StandardModel/IsFermionSector/MassWeight/KineticFamilies.lean on GitHub.

lemma sum_deriv_one [source]

A sum over the assignments of one derivative direction is a single sum.

```
lemma sum_deriv_one {M : Type*} [AddCommMonoid M] (f : (Fin 1 → Fin 1 ⊕ Fin 3) → M) :
    ∑ p : Fin 1 → Fin 1 ⊕ Fin 3, f p = ∑ x : Fin 1 ⊕ Fin 3, f ![x]
```

lemma repLorentz_symbol_deriv_one [source]

The Lorentz transformation of a symbol with one covariant derivative: the derivative slot moves by the columns of the Lorentz matrix and the value index by the contragredient action.

```
lemma repLorentz_symbol_deriv_one {V : Type} [AddCommGroup V] [Module ℂ V]
    {rep : Representation ℂ SL(2,ℂ) V}
    {X : {n : ℕ} → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B}
    (hX : IsLorentzCovDerivTransforms repLorentz rep X) (Λ : SL(2,ℂ))
    (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ V) :
    repLorentz Λ (X ![μ] φ)
      = ∑ ν, (((SL2C.toLorentzGroup Λ).1 ν μ : ℝ) : ℂ) • X ![ν] (rep.dual Λ φ)
```

lemma repLorentz_component_deriv_one [source]

The Lorentz transformation of a component of a once-derived symbol, when the coordinate functionals `c` are permuted by the contragredient action with coefficients `m`. This is the once-derived form of the laws of `Components`.

```
lemma repLorentz_component_deriv_one {V : Type} [AddCommGroup V] [Module ℂ V]
    {rep : Representation ℂ SL(2,ℂ) V}
    {X : {n : ℕ} → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B}
    (hX : IsLorentzCovDerivTransforms repLorentz rep X)
    {c : Fin 2 → Module.Dual ℂ V} {m : SL(2,ℂ) → Fin 2 → Fin 2 → ℂ}
    (hc : ∀ (Λ : SL(2,ℂ)) (a : Fin 2), rep.dual Λ (c a) = ∑ β, m Λ a β • c β)
    (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) (a : Fin 2) :
    repLorentz Λ (X ![μ] (c a))
      = ∑ ν, ∑ β, ((((SL2C.toLorentzGroup Λ).1 ν μ : ℝ) : ℂ) * m Λ a β) • X ![ν] (c β)
```

lemma isVectorDualLeftWeyl_leftComp [source]

Every undotted component family carries a four-vector index and a dual undotted spinor index, at one covariant derivative.

```
lemma isVectorDualLeftWeyl_leftComp (i : LeftIdx) :
    IsVectorDualLeftWeyl repLorentz (fun μ => h.leftComp ![μ] i)
```

lemma isVectorDualRightWeyl_rightComp [source]

Every dotted component family carries a four-vector index and a dual dotted spinor index, at one covariant derivative.

```
lemma isVectorDualRightWeyl_rightComp (i : RightIdx) :
    IsVectorDualRightWeyl repLorentz (fun μ => h.rightComp ![μ] i)
```

lemma repGauge_su3_d [source]

A colour transformation moves the colour index of a down-singlet symbol by the conjugate matrix, the index being anti-fundamental.

```
lemma repGauge_su3_d (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.dComponent f l (s, c))
      = ∑ a, conj (U.1 a c) • h.dComponent f l (s, a)
```

lemma repGauge_su2_d [source]

An isospin transformation fixes a down-singlet symbol, which carries no isospin.

```
lemma repGauge_su2_d (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.dComponent f l j)
      = h.dComponent f l j
```

lemma repGauge_u1_d [source]

A hypercharge transformation scales a down-singlet symbol by the square of the scalar, the down singlet carrying `6Y = 2`.

```
lemma repGauge_u1_d (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.dComponent f l j)
      = (t : ℂ) ^ 2 • h.dComponent f l j
```

lemma repGauge_su3_bard [source]

A colour transformation moves the colour index of a conjugate down-singlet symbol by the matrix itself, the index being fundamental.

```
lemma repGauge_su3_bard (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.bardComponent f l (s, c))
      = ∑ a, U.1 a c • h.bardComponent f l (s, a)
```

lemma repGauge_su2_bard [source]

An isospin transformation fixes a conjugate down-singlet symbol.

```
lemma repGauge_su2_bard (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.bardComponent f l j)
      = h.bardComponent f l j
```

lemma repGauge_u1_bard [source]

A hypercharge transformation scales a conjugate down-singlet symbol by the square of the conjugate scalar, the conjugate down singlet carrying `6Y = -2`.

```
lemma repGauge_u1_bard (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.bardComponent f l j)
      = (star (t : ℂ)) ^ 2 • h.bardComponent f l j
```

lemma repGauge_su3_u [source]

A colour transformation moves the colour index of an up-singlet symbol by the conjugate matrix, the index being anti-fundamental.

```
lemma repGauge_su3_u (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.uComponent f l (s, c))
      = ∑ a, conj (U.1 a c) • h.uComponent f l (s, a)
```

lemma repGauge_su2_u [source]

An isospin transformation fixes an up-singlet symbol.

```
lemma repGauge_su2_u (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.uComponent f l j)
      = h.uComponent f l j
```

lemma repGauge_u1_u [source]

A hypercharge transformation scales an up-singlet symbol by the fourth power of the conjugate scalar, the up singlet carrying `6Y = -4`.

```
lemma repGauge_u1_u (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.uComponent f l j)
      = (star (t : ℂ)) ^ 4 • h.uComponent f l j
```

lemma repGauge_su3_baru [source]

A colour transformation moves the colour index of a conjugate up-singlet symbol by the matrix itself, the index being fundamental.

```
lemma repGauge_su3_baru (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.baruComponent f l (s, c))
      = ∑ a, U.1 a c • h.baruComponent f l (s, a)
```

lemma repGauge_su2_baru [source]

An isospin transformation fixes a conjugate up-singlet symbol.

```
lemma repGauge_su2_baru (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 3) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.baruComponent f l j)
      = h.baruComponent f l j
```

lemma repGauge_u1_baru [source]

A hypercharge transformation scales a conjugate up-singlet symbol by the fourth power of the scalar, the conjugate up singlet carrying `6Y = 4`.

```
lemma repGauge_u1_baru (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 3) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.baruComponent f l j)
      = (t : ℂ) ^ 4 • h.baruComponent f l j
```

lemma repGauge_su3_Q [source]

A colour transformation moves the colour index of a quark-doublet symbol by the conjugate matrix, the index being anti-fundamental.

```
lemma repGauge_su3_Q (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) (w : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.QComponent f l (s, c, w))
      = ∑ a, conj (U.1 a c) • h.QComponent f l (s, a, w)
```

lemma repGauge_su2_Q [source]

An isospin transformation moves the isospin index of a quark-doublet symbol by the conjugate matrix, the index being anti-fundamental.

```
lemma repGauge_su2_Q (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) (w : Fin 2) :
```

```
    repGauge ((1, V, 1) : GaugeGroupI) (h.QComponent f l (s, c, w))
      = ∑ a, conj (V.1 a w) • h.QComponent f l (s, c, a)
```

lemma repGauge_u1_Q [source]

A hypercharge transformation scales a quark-doublet symbol by the conjugate scalar, the quark doublet carrying `6Y = -1`.

```
lemma repGauge_u1_Q (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (s : Fin 2) (c : Fin 3) (w : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.QComponent f l (s, c, w))
      = star (t : ℂ) • h.QComponent f l (s, c, w)
```

lemma repGauge_su3_barQ [source]

A colour transformation moves the colour index of a conjugate quark-doublet symbol by the matrix itself, the index being fundamental.

```
lemma repGauge_su3_barQ (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) (w : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.barQComponent f l (s, c, w))
      = ∑ a, U.1 a c • h.barQComponent f l (s, a, w)
```

lemma repGauge_su2_barQ [source]

An isospin transformation moves the isospin index of a conjugate quark-doublet symbol by the matrix itself, the index being fundamental.

```
lemma repGauge_su2_barQ (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) (c : Fin 3) (w : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.barQComponent f l (s, c, w))
      = ∑ a, V.1 a w • h.barQComponent f l (s, c, a)
```

lemma repGauge_u1_barQ [source]

A hypercharge transformation scales a conjugate quark-doublet symbol by the scalar, the conjugate quark doublet carrying `6Y = 1`.

```
lemma repGauge_u1_barQ (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (s : Fin 2) (c : Fin 3) (w : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.barQComponent f l (s, c, w))
      = (t : ℂ) • h.barQComponent f l (s, c, w)
```

lemma repGauge_su3_L [source]

A colour transformation fixes a lepton-doublet symbol, which carries no colour.

```
lemma repGauge_su3_L (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.LComponent f l j)
      = h.LComponent f l j
```

lemma repGauge_su2_L [source]

An isospin transformation moves the isospin index of a lepton-doublet symbol by the conjugate matrix, the index being anti-fundamental.

```
lemma repGauge_su2_L (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s w : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.LComponent f l (s, w))
      = ∑ a, conj (V.1 a w) • h.LComponent f l (s, a)
```

lemma repGauge_u1_L [source]

A hypercharge transformation scales a lepton-doublet symbol by the cube of the scalar, the lepton doublet carrying `6Y = 3`.

```
lemma repGauge_u1_L (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (j : Fin 2 × Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.LComponent f l j)
      = (t : ℂ) ^ 3 • h.LComponent f l j
```

lemma repGauge_su3_barL [source]

A colour transformation fixes a conjugate lepton-doublet symbol, which carries no colour.

```
lemma repGauge_su3_barL (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (j : Fin 2 × Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.barLComponent f l j)
      = h.barLComponent f l j
```

lemma repGauge_su2_barL [source]

An isospin transformation moves the isospin index of a conjugate lepton-doublet symbol by the matrix itself, the index being fundamental.

```
lemma repGauge_su2_barL (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s w : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.barLComponent f l (s, w))
      = ∑ a, V.1 a w • h.barLComponent f l (s, a)
```

lemma repGauge_u1_barL [source]

A hypercharge transformation scales a conjugate lepton-doublet symbol by the cube of the conjugate scalar, the conjugate lepton doublet carrying `6Y = -3`.

```
lemma repGauge_u1_barL (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (s w : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.barLComponent f l (s, w))
      = (star (t : ℂ)) ^ 3 • h.barLComponent f l (s, w)
```

lemma repGauge_su3_e [source]

A colour transformation fixes a lepton-singlet symbol.

```
lemma repGauge_su3_e (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) :
```

```
    repGauge ((U, 1, 1) : GaugeGroupI) (h.eComponent f l s)
      = h.eComponent f l s
```

lemma repGauge_su2_e [source]

An isospin transformation fixes a lepton-singlet symbol.

```
lemma repGauge_su2_e (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.eComponent f l s)
      = h.eComponent f l s
```

lemma repGauge_u1_e [source]

A hypercharge transformation scales a lepton-singlet symbol by the sixth power of the scalar, the lepton singlet carrying `6Y = 6`.

```
lemma repGauge_u1_e (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (s : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.eComponent f l s)
      = (t : ℂ) ^ 6 • h.eComponent f l s
```

lemma repGauge_su3_bare [source]

A colour transformation fixes a conjugate lepton-singlet symbol.

```
lemma repGauge_su3_bare (U : specialUnitaryGroup (Fin 3) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.bareComponent f l s)
      = h.bareComponent f l s
```

lemma repGauge_su2_bare [source]

An isospin transformation fixes a conjugate lepton-singlet symbol.

```
lemma repGauge_su2_bare (V : specialUnitaryGroup (Fin 2) ℂ) (f : Fin 3) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (s : Fin 2) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.bareComponent f l s)
      = h.bareComponent f l s
```

lemma repGauge_u1_bare [source]

A hypercharge transformation scales a conjugate lepton-singlet symbol by the sixth power of the conjugate scalar, the conjugate lepton singlet carrying `6Y = -6`.

```
lemma repGauge_u1_bare (t : unitary ℂ) (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (s : Fin 2) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.bareComponent f l s)
      = (star (t : ℂ)) ^ 6 • h.bareComponent f l s
```

lemma isSU3FundamentalAntiFundamental_mul [source]

A product of a component with a fundamental colour index and one with an anti-fundamental colour index carries one colour index of each kind.

```
lemma isSU3FundamentalAntiFundamental_mul
    (hmul : ∀ (g : GaugeGroupI) (b₁ b₂ : B),
      repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂)
    {A C : Fin 3 → B}
    (hA : ∀ (U : specialUnitaryGroup (Fin 3) ℂ) (c : Fin 3),
      repGauge ((U, 1, 1) : GaugeGroupI) (A c) = ∑ a, U.1 a c • A a)
    (hC : ∀ (U : specialUnitaryGroup (Fin 3) ℂ) (c : Fin 3),
      repGauge ((U, 1, 1) : GaugeGroupI) (C c) = ∑ a, conj (U.1 a c) • C a) :
    IsSU3FundamentalAntiFundamental B repGauge (fun l : Fin 2 → Fin 3 => A (l 0) * C (l 1))
```

lemma isSU3FundamentalAntiFundamental_mul_swap [source]

The same with the two factors exchanged, the anti-fundamental one first.

```
lemma isSU3FundamentalAntiFundamental_mul_swap
    (hmul : ∀ (g : GaugeGroupI) (b₁ b₂ : B),
      repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂)
    {A C : Fin 3 → B}
    (hA : ∀ (U : specialUnitaryGroup (Fin 3) ℂ) (c : Fin 3),
      repGauge ((U, 1, 1) : GaugeGroupI) (A c) = ∑ a, conj (U.1 a c) • A a)
    (hC : ∀ (U : specialUnitaryGroup (Fin 3) ℂ) (c : Fin 3),
      repGauge ((U, 1, 1) : GaugeGroupI) (C c) = ∑ a, U.1 a c • C a) :
    IsSU3FundamentalAntiFundamental B repGauge (fun l : Fin 2 → Fin 3 => A (l 1) * C (l 0))
```

lemma isSU2FundamentalAntiFundamental_mul [source]

A product of a component with a fundamental isospin index and one with an anti-fundamental isospin index carries one isospin index of each kind.

```
lemma isSU2FundamentalAntiFundamental_mul
    (hmul : ∀ (g : GaugeGroupI) (b₁ b₂ : B),
      repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂)
    {A C : Fin 2 → B}
    (hA : ∀ (V : specialUnitaryGroup (Fin 2) ℂ) (w : Fin 2),
      repGauge ((1, V, 1) : GaugeGroupI) (A w) = ∑ a, V.1 a w • A a)
    (hC : ∀ (V : specialUnitaryGroup (Fin 2) ℂ) (w : Fin 2),
      repGauge ((1, V, 1) : GaugeGroupI) (C w) = ∑ a, conj (V.1 a w) • C a) :
    IsSU2FundamentalAntiFundamental B repGauge (fun l : Fin 2 → Fin 2 => A (l 0) * C (l 1))
```

lemma isSU2FundamentalAntiFundamental_mul_swap [source]

The same with the two factors exchanged, the anti-fundamental one first.

```
lemma isSU2FundamentalAntiFundamental_mul_swap
    (hmul : ∀ (g : GaugeGroupI) (b₁ b₂ : B),
      repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂)
    {A C : Fin 2 → B}
    (hA : ∀ (V : specialUnitaryGroup (Fin 2) ℂ) (w : Fin 2),
      repGauge ((1, V, 1) : GaugeGroupI) (A w) = ∑ a, conj (V.1 a w) • A a)
    (hC : ∀ (V : specialUnitaryGroup (Fin 2) ℂ) (w : Fin 2),
      repGauge ((1, V, 1) : GaugeGroupI) (C w) = ∑ a, V.1 a w • C a) :
    IsSU2FundamentalAntiFundamental B repGauge (fun l : Fin 2 → Fin 2 => A (l 1) * C (l 0))
```

lemma repGauge_mul_fixed [source]

A product of two components that a gauge transformation fixes is fixed by it: the form in which a block whose symbols carry no colour, or no isospin, supplies the corresponding stage.

```
lemma repGauge_mul_fixed
    (hmul : ∀ (g : GaugeGroupI) (b₁ b₂ : B),
      repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂)
    {g : GaugeGroupI} {a c : B} (ha : repGauge g a = a) (hc : repGauge g c = c) :
    repGauge g (a * c) = a * c
```

lemma repGauge_mul_smul_fixed [source]

A product of two components that a gauge transformation scales by reciprocal scalars is fixed by it: the form in which the hypercharges of a species and its conjugate cancel.

```
lemma repGauge_mul_smul_fixed
    (hmul : ∀ (g : GaugeGroupI) (b₁ b₂ : B),
      repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂)
    {g : GaugeGroupI} {z z' : ℂ} {a c : B} (hz : z * z' = 1)
    (ha : repGauge g a = z • a) (hc : repGauge g c = z' • c) :
    repGauge g (a * c) = a * c
```

lemma isVectorDualLeftRightWeyl_sum [source]

A finite sum of families carrying one four-vector index and a pair of dual opposite-chirality Weyl indices is such a family again: the colour and isospin contractions are Lorentz spectators.

```
lemma isVectorDualLeftRightWeyl_sum {ι : Type} [Fintype ι]
    {T : ι → (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 → B}
    (hT : ∀ i, IsVectorDualLeftRightWeyl B repLorentz (ofDualVectorComponents (T i))) :
    IsVectorDualLeftRightWeyl B repLorentz (ofDualVectorComponents fun p => ∑ i, T i p)
```

lemma unitary_mul_star_coe [source]

A unitary scalar times its conjugate is one, in the order the hypercharge cancellation of a species against its conjugate needs.

```
lemma unitary_mul_star_coe (t : unitary ℂ) : (t : ℂ) * star (t : ℂ) = 1
```

lemma unitary_star_mul_coe [source]

The conjugate of a unitary scalar times itself is one.

```
lemma unitary_star_mul_coe (t : unitary ℂ) : star (t : ℂ) * (t : ℂ) = 1
```

structure KineticBlock [source]

One kinetic block of the fermion sector, together with the three classifications its indices admit. The block is indexed by the derivative direction, the pair of spinor indices in the order `(undotted, dotted)`, the pair of colour indices and the pair of isospin indices, each pair in the order `(fundamental, anti-fundamental)`.

```
structure KineticBlock where
  blk : (Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Fin 3 → Fin 3 → Fin 2 → Fin 2 → B
  colourStep : ∀ (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2) (w w' : Fin 2),
    InvariantReductionToSpan (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge (U, 1, 1))
      (Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 3 => blk q l (n 0) (n 1) w w'))
  colourStep_mem : ∀ q l w w', (colourStep q l w w').spanningVector
    ∈ Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 3 => blk q l (n 0) (n 1) w w')
  isospinStep : ∀ (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2),
    InvariantReductionToSpan (fun V : specialUnitaryGroup (Fin 2) ℂ => repGauge (1, V, 1))
      (Submodule.span ℂ
        (Set.range fun n : Fin 2 → Fin 2 => (colourStep q l (n 0) (n 1)).spanningVector))
  isospinStep_mem : ∀ q l, (isospinStep q l).spanningVector
    ∈ Submodule.span ℂ
      (Set.range fun n : Fin 2 → Fin 2 => (colourStep q l (n 0) (n 1)).spanningVector)
  lorentzStep : InvariantReductionToSpan (fun Λ : SL(2,ℂ) => repLorentz Λ)
    (Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
      (isospinStep p.1 p.2).spanningVector))
  lorentzStep_mem : lorentzStep.spanningVector
    ∈ Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
      (isospinStep p.1 p.2).spanningVector)
  hyper : ∀ (t : unitary ℂ) q l c c' w w',
    repGauge ((1, 1, t) : GaugeGroupI) (blk q l c c' w w') = blk q l c c' w w'
```

def kineticTerm [source]

The kinetic term of a block: the conjugate Pauli contraction of its doubly contracted form, which is ψ̄ σ̄^μ ∂_μ ψ with the colour and isospin indices already joined.

```
noncomputable def kineticTerm : B
```

def blockSpan [source]

The join, over the derivative, spinor and isospin indices, of the colour spans of the block: what the block submodule is reduced from.

```
noncomputable def blockSpan : Submodule ℂ B
```

lemma reducesInvariantsTo [source]

The three stages in sequence: the block span reduces to the span of the kinetic term.

```
lemma reducesInvariantsTo :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) K.blockSpan (ℂ • K.kineticTerm)
```

lemma kineticTerm_mem [source]

The kinetic term lies in any submodule containing every component of the block: each contraction lies in the span of the objects of the stage before it.

```
lemma kineticTerm_mem {V : Submodule ℂ B}
    (hV : ∀ q l c c' w w', K.blk q l c c' w w' ∈ V) : K.kineticTerm ∈ V
```

lemma repGauge_su3_kineticTerm [source]

The kinetic term is fixed by the colour factor: the colour contractions are, and every later stage stays inside their span.

```
lemma repGauge_su3_kineticTerm (U : specialUnitaryGroup (Fin 3) ℂ) :
    repGauge ((U, 1, 1) : GaugeGroupI) K.kineticTerm = K.kineticTerm
```

lemma repGauge_su2_kineticTerm [source]

The kinetic term is fixed by the isospin factor.

```
lemma repGauge_su2_kineticTerm (V : specialUnitaryGroup (Fin 2) ℂ) :
    repGauge ((1, V, 1) : GaugeGroupI) K.kineticTerm = K.kineticTerm
```

lemma repGauge_u1_kineticTerm [source]

The kinetic term is fixed by the hypercharge factor, the hypercharges of a species and its conjugate cancelling on every component of the block.

```
lemma repGauge_u1_kineticTerm (t : unitary ℂ) :
    repGauge ((1, 1, t) : GaugeGroupI) K.kineticTerm = K.kineticTerm
```

lemma repGauge_kineticTerm [source]

The kinetic term is gauge invariant: a gauge transformation is the product of its colour, isospin and hypercharge parts, and each fixes it.

```
lemma repGauge_kineticTerm (g : GaugeGroupI) : repGauge g K.kineticTerm = K.kineticTerm
```

lemma repLorentz_kineticTerm [source]

The kinetic term is Lorentz invariant, being the conjugate Pauli contraction of a vector dual left-right Weyl family.

```
lemma repLorentz_kineticTerm (Λ : SL(2,ℂ)) :
    repLorentz Λ K.kineticTerm = K.kineticTerm
```

2.467 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.KineticTerms

PhyslibAlpha/Particles/StandardModel/IsFermionSector/MassWeight/KineticTerms.lean on GitHub.

def dbardBlk [source]

The components of the block `d ∂ bard`: an underived down-singlet symbol against a once-derived conjugate down-singlet symbol.

```
noncomputable def dbardBlk (f f' : Fin 3) :
    (Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Fin 3 → Fin 3 → Fin 2 → Fin 2 → B
```

lemma isSU3FundamentalAntiFundamental_dbardBlk [source]

The two colour indices of the `d ∂ bard` block are one fundamental and one anti-fundamental, the barred symbol supplying the fundamental one.

```
lemma isSU3FundamentalAntiFundamental_dbardBlk (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (w w' : Fin 2) :
    IsSU3FundamentalAntiFundamental B repGauge
      (fun n : Fin 2 → Fin 3 => h.dbardBlk f f' q l (n 0) (n 1) w w')
```

lemma repGauge_su2_dbardBlk [source]

An isospin transformation fixes the `d ∂ bard` block, neither symbol carrying isospin.

```
lemma repGauge_su2_dbardBlk (V : specialUnitaryGroup (Fin 2) ℂ) (f f' : Fin 3)
    (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.dbardBlk f f' q l c c' w w')
      = h.dbardBlk f f' q l c c' w w'
```

lemma repGauge_u1_dbardBlk [source]

A hypercharge transformation fixes the `d ∂ bard` block, the hypercharges of a species and its conjugate cancelling.

```
lemma repGauge_u1_dbardBlk (t : unitary ℂ) (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.dbardBlk f f' q l c c' w w')
      = h.dbardBlk f f' q l c c' w w'
```

def dbardColourStep [source]

The colour stage of the `d ∂ bard` block.

```
noncomputable def dbardColourStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    InvariantReductionToSpan
      (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ
        (Set.range fun n : Fin 2 → Fin 3 => h.dbardBlk f f' q l (n 0) (n 1) w w'))
```

lemma dbardColourStep_contraction [source]

The colour contraction of the `d ∂ bard` block, written out.

```
lemma dbardColourStep_contraction (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    (h.dbardColourStep f f' q l w w').spanningVector
      = ∑ a : Fin 3, h.dbardBlk f f' q l a a w w'
```

def dbardIsospinStep [source]

The isospin stage of the `d ∂ bard` block.

```
noncomputable def dbardIsospinStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) :
    InvariantReductionToSpan
      (fun V : specialUnitaryGroup (Fin 2) ℂ => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 2 =>
        (h.dbardColourStep f f' q l (n 0) (n 1)).spanningVector))
```

lemma isVectorDualLeftRightWeyl_dbard [source]

The doubly contracted `d ∂ bard` block carries one four-vector index and a pair of dual opposite-chirality Weyl indices.

```
lemma isVectorDualLeftRightWeyl_dbard (f f' : Fin 3) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.dbardIsospinStep f f' p.1 p.2).spanningVector)
```

def dbardLorentzStep [source]

The Lorentz stage of the `d ∂ bard` block.

```
noncomputable def dbardLorentzStep (f f' : Fin 3) :
    InvariantReductionToSpan (fun Λ : SL(2,ℂ) => repLorentz Λ)
      (Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.dbardIsospinStep f f' p.1 p.2).spanningVector))
```

def dbardKineticBlock [source]

The `d ∂ bard` block as a kinetic block.

```
noncomputable def dbardKineticBlock (f f' : Fin 3) : KineticBlock repGauge repLorentz where
```

def barddBlk [source]

The components of the block `bard ∂ d`: an underived conjugate down-singlet symbol against a once-derived down-singlet symbol.

```
noncomputable def barddBlk (f f' : Fin 3) :
    (Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Fin 3 → Fin 3 → Fin 2 → Fin 2 → B
```

lemma isSU3FundamentalAntiFundamental_barddBlk [source]

The two colour indices of the `bard ∂ d` block are one fundamental and one anti-fundamental, the barred symbol supplying the fundamental one.

```
lemma isSU3FundamentalAntiFundamental_barddBlk (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (w w' : Fin 2) :
    IsSU3FundamentalAntiFundamental B repGauge
      (fun n : Fin 2 → Fin 3 => h.barddBlk f f' q l (n 0) (n 1) w w')
```

lemma repGauge_su2_barddBlk [source]

An isospin transformation fixes the `bard ∂ d` block, neither symbol carrying isospin.

```
lemma repGauge_su2_barddBlk (V : specialUnitaryGroup (Fin 2) ℂ) (f f' : Fin 3)
    (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.barddBlk f f' q l c c' w w')
      = h.barddBlk f f' q l c c' w w'
```

lemma repGauge_u1_barddBlk [source]

A hypercharge transformation fixes the `bard ∂ d` block, the hypercharges of a species and its conjugate cancelling.

```
lemma repGauge_u1_barddBlk (t : unitary ℂ) (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.barddBlk f f' q l c c' w w')
      = h.barddBlk f f' q l c c' w w'
```

def barddColourStep [source]

The colour stage of the `bard ∂ d` block.

```
noncomputable def barddColourStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    InvariantReductionToSpan
      (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ
        (Set.range fun n : Fin 2 → Fin 3 => h.barddBlk f f' q l (n 0) (n 1) w w'))
```

lemma barddColourStep_contraction [source]

The colour contraction of the `bard ∂ d` block, written out.

```
lemma barddColourStep_contraction (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    (h.barddColourStep f f' q l w w').spanningVector
      = ∑ a : Fin 3, h.barddBlk f f' q l a a w w'
```

def barddIsospinStep [source]

The isospin stage of the `bard ∂ d` block.

```
noncomputable def barddIsospinStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) :
    InvariantReductionToSpan
      (fun V : specialUnitaryGroup (Fin 2) ℂ => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 2 =>
        (h.barddColourStep f f' q l (n 0) (n 1)).spanningVector))
```

lemma isVectorDualLeftRightWeyl_bardd [source]

The doubly contracted `bard ∂ d` block carries one four-vector index and a pair of dual opposite-chirality Weyl indices.

```
lemma isVectorDualLeftRightWeyl_bardd (f f' : Fin 3) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.barddIsospinStep f f' p.1 p.2).spanningVector)
```

def barddLorentzStep [source]

The Lorentz stage of the `bard ∂ d` block.

```
noncomputable def barddLorentzStep (f f' : Fin 3) :
    InvariantReductionToSpan (fun Λ : SL(2,ℂ) => repLorentz Λ)
      (Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
```

```
      (h.barddIsospinStep f f' p.1 p.2).spanningVector))
```

def barddKineticBlock [source]

The `bard ∂ d` block as a kinetic block.

```
noncomputable def barddKineticBlock (f f' : Fin 3) : KineticBlock repGauge repLorentz where
```

def ubaruBlk [source]

The components of the block `u ∂ baru`: an underived up-singlet symbol against a once-derived conjugate up-singlet symbol.

```
noncomputable def ubaruBlk (f f' : Fin 3) :
    (Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Fin 3 → Fin 3 → Fin 2 → Fin 2 → B
```

lemma isSU3FundamentalAntiFundamental_ubaruBlk [source]

The two colour indices of the `u ∂ baru` block are one fundamental and one anti-fundamental, the barred symbol supplying the fundamental one.

```
lemma isSU3FundamentalAntiFundamental_ubaruBlk (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (w w' : Fin 2) :
    IsSU3FundamentalAntiFundamental B repGauge
      (fun n : Fin 2 → Fin 3 => h.ubaruBlk f f' q l (n 0) (n 1) w w')
```

lemma repGauge_su2_ubaruBlk [source]

An isospin transformation fixes the `u ∂ baru` block, neither symbol carrying isospin.

```
lemma repGauge_su2_ubaruBlk (V : specialUnitaryGroup (Fin 2) ℂ) (f f' : Fin 3)
    (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.ubaruBlk f f' q l c c' w w')
      = h.ubaruBlk f f' q l c c' w w'
```

lemma repGauge_u1_ubaruBlk [source]

A hypercharge transformation fixes the `u ∂ baru` block, the hypercharges of a species and its conjugate cancelling.

```
lemma repGauge_u1_ubaruBlk (t : unitary ℂ) (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.ubaruBlk f f' q l c c' w w')
      = h.ubaruBlk f f' q l c c' w w'
```

def ubaruColourStep [source]

The colour stage of the `u ∂ baru` block.

```
noncomputable def ubaruColourStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    InvariantReductionToSpan
      (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ
        (Set.range fun n : Fin 2 → Fin 3 => h.ubaruBlk f f' q l (n 0) (n 1) w w'))
```

lemma ubaruColourStep_contraction [source]

The colour contraction of the `u ∂ baru` block, written out.

```
lemma ubaruColourStep_contraction (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    (h.ubaruColourStep f f' q l w w').spanningVector
      = ∑ a : Fin 3, h.ubaruBlk f f' q l a a w w'
```

def ubaruIsospinStep [source]

The isospin stage of the `u ∂ baru` block.

```
noncomputable def ubaruIsospinStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) :
    InvariantReductionToSpan
      (fun V : specialUnitaryGroup (Fin 2) ℂ => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 2 =>
        (h.ubaruColourStep f f' q l (n 0) (n 1)).spanningVector))
```

lemma isVectorDualLeftRightWeyl_ubaru [source]

The doubly contracted `u ∂ baru` block carries one four-vector index and a pair of dual opposite-chirality Weyl indices.

```
lemma isVectorDualLeftRightWeyl_ubaru (f f' : Fin 3) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.ubaruIsospinStep f f' p.1 p.2).spanningVector)
```

def ubaruLorentzStep [source]

The Lorentz stage of the `u ∂ baru` block.

```
noncomputable def ubaruLorentzStep (f f' : Fin 3) :
    InvariantReductionToSpan (fun Λ : SL(2,ℂ) => repLorentz Λ)
      (Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.ubaruIsospinStep f f' p.1 p.2).spanningVector))
```

def ubaruKineticBlock [source]

The `u ∂ baru` block as a kinetic block.

```
noncomputable def ubaruKineticBlock (f f' : Fin 3) : KineticBlock repGauge repLorentz where
```

def baruuBlk [source]

The components of the block `baru ∂ u`: an underived conjugate up-singlet symbol against a once-derived up-singlet symbol.

```
noncomputable def baruuBlk (f f' : Fin 3) :
    (Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Fin 3 → Fin 3 → Fin 2 → Fin 2 → B
```

lemma isSU3FundamentalAntiFundamental_baruuBlk [source]

The two colour indices of the `baru ∂ u` block are one fundamental and one anti-fundamental, the barred symbol supplying the fundamental one.

```
lemma isSU3FundamentalAntiFundamental_baruuBlk (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (w w' : Fin 2) :
    IsSU3FundamentalAntiFundamental B repGauge
      (fun n : Fin 2 → Fin 3 => h.baruuBlk f f' q l (n 0) (n 1) w w')
```

lemma repGauge_su2_baruuBlk [source]

An isospin transformation fixes the `baru ∂ u` block, neither symbol carrying isospin.

```
lemma repGauge_su2_baruuBlk (V : specialUnitaryGroup (Fin 2) ℂ) (f f' : Fin 3)
    (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.baruuBlk f f' q l c c' w w')
      = h.baruuBlk f f' q l c c' w w'
```

lemma repGauge_u1_baruuBlk [source]

A hypercharge transformation fixes the `baru ∂ u` block, the hypercharges of a species and its conjugate cancelling.

```
lemma repGauge_u1_baruuBlk (t : unitary ℂ) (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.baruuBlk f f' q l c c' w w')
      = h.baruuBlk f f' q l c c' w w'
```

def baruuColourStep [source]

The colour stage of the `baru ∂ u` block.

```
noncomputable def baruuColourStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    InvariantReductionToSpan
      (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ
        (Set.range fun n : Fin 2 → Fin 3 => h.baruuBlk f f' q l (n 0) (n 1) w w'))
```

lemma baruuColourStep_contraction [source]

The colour contraction of the `baru ∂ u` block, written out.

```
lemma baruuColourStep_contraction (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    (h.baruuColourStep f f' q l w w').spanningVector
      = ∑ a : Fin 3, h.baruuBlk f f' q l a a w w'
```

def baruuIsospinStep [source]

The isospin stage of the `baru ∂ u` block.

```
noncomputable def baruuIsospinStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) :
    InvariantReductionToSpan
```

```
      (fun V : specialUnitaryGroup (Fin 2) ℂ => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 2 =>
        (h.baruuColourStep f f' q l (n 0) (n 1)).spanningVector))
```

lemma isVectorDualLeftRightWeyl_baruu [source]

The doubly contracted `baru ∂ u` block carries one four-vector index and a pair of dual opposite-chirality Weyl indices.

```
lemma isVectorDualLeftRightWeyl_baruu (f f' : Fin 3) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.baruuIsospinStep f f' p.1 p.2).spanningVector)
```

def baruuLorentzStep [source]

The Lorentz stage of the `baru ∂ u` block.

```
noncomputable def baruuLorentzStep (f f' : Fin 3) :
    InvariantReductionToSpan (fun Λ : SL(2,ℂ) => repLorentz Λ)
      (Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.baruuIsospinStep f f' p.1 p.2).spanningVector))
```

def baruuKineticBlock [source]

The `baru ∂ u` block as a kinetic block.

```
noncomputable def baruuKineticBlock (f f' : Fin 3) : KineticBlock repGauge repLorentz where
```

def QbarQBlk [source]

The components of the block `Q ∂ barQ`: an underived quark-doublet symbol against a once-derived conjugate quark-doublet symbol.

```
noncomputable def QbarQBlk (f f' : Fin 3) :
    (Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Fin 3 → Fin 3 → Fin 2 → Fin 2 → B
```

lemma isSU3FundamentalAntiFundamental_QbarQBlk [source]

The two colour indices of the `Q ∂ barQ` block are one fundamental and one anti-fundamental, the barred symbol supplying the fundamental one.

```
lemma isSU3FundamentalAntiFundamental_QbarQBlk (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (w w' : Fin 2) :
    IsSU3FundamentalAntiFundamental B repGauge
      (fun n : Fin 2 → Fin 3 => h.QbarQBlk f f' q l (n 0) (n 1) w w')
```

lemma isSU2FundamentalAntiFundamental_QbarQBlk [source]

The two isospin indices of the `Q ∂ barQ` block are one fundamental and one anti-fundamental, the barred symbol supplying the fundamental one.

```
lemma isSU2FundamentalAntiFundamental_QbarQBlk (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' : Fin 3) :
    IsSU2FundamentalAntiFundamental B repGauge
```

```
      (fun n : Fin 2 → Fin 2 => h.QbarQBlk f f' q l c c' (n 0) (n 1))
```

lemma repGauge_u1_QbarQBlk [source]

A hypercharge transformation fixes the `Q ∂ barQ` block, the hypercharges of a species and its conjugate cancelling.

```
lemma repGauge_u1_QbarQBlk (t : unitary ℂ) (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.QbarQBlk f f' q l c c' w w')
      = h.QbarQBlk f f' q l c c' w w'
```

def QbarQColourStep [source]

The colour stage of the `Q ∂ barQ` block.

```
noncomputable def QbarQColourStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    InvariantReductionToSpan
      (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ
        (Set.range fun n : Fin 2 → Fin 3 => h.QbarQBlk f f' q l (n 0) (n 1) w w'))
```

lemma QbarQColourStep_contraction [source]

The colour contraction of the `Q ∂ barQ` block, written out.

```
lemma QbarQColourStep_contraction (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    (h.QbarQColourStep f f' q l w w').spanningVector
      = ∑ a : Fin 3, h.QbarQBlk f f' q l a a w w'
```

def QbarQIsospinStep [source]

The isospin stage of the `Q ∂ barQ` block.

```
noncomputable def QbarQIsospinStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) :
    InvariantReductionToSpan
      (fun V : specialUnitaryGroup (Fin 2) ℂ => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 2 =>
        (h.QbarQColourStep f f' q l (n 0) (n 1)).spanningVector))
```

lemma isVectorDualLeftRightWeyl_QbarQ [source]

The doubly contracted `Q ∂ barQ` block carries one four-vector index and a pair of dual opposite-chirality Weyl indices.

```
lemma isVectorDualLeftRightWeyl_QbarQ (f f' : Fin 3) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.QbarQIsospinStep f f' p.1 p.2).spanningVector)
```

def QbarQLorentzStep [source]

The Lorentz stage of the `Q ∂ barQ` block.

```
noncomputable def QbarQLorentzStep (f f' : Fin 3) :
    InvariantReductionToSpan (fun Λ : SL(2,ℂ) => repLorentz Λ)
      (Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.QbarQIsospinStep f f' p.1 p.2).spanningVector))
```

def QbarQKineticBlock [source]

The `Q ∂ barQ` block as a kinetic block.

```
noncomputable def QbarQKineticBlock (f f' : Fin 3) : KineticBlock repGauge repLorentz where
```

def barQQBlk [source]

The components of the block `barQ ∂ Q`: an underived conjugate quark-doublet symbol against a once-derived quark-doublet symbol.

```
noncomputable def barQQBlk (f f' : Fin 3) :
    (Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Fin 3 → Fin 3 → Fin 2 → Fin 2 → B
```

lemma isSU3FundamentalAntiFundamental_barQQBlk [source]

The two colour indices of the `barQ ∂ Q` block are one fundamental and one anti-fundamental, the barred symbol supplying the fundamental one.

```
lemma isSU3FundamentalAntiFundamental_barQQBlk (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (w w' : Fin 2) :
    IsSU3FundamentalAntiFundamental B repGauge
      (fun n : Fin 2 → Fin 3 => h.barQQBlk f f' q l (n 0) (n 1) w w')
```

lemma isSU2FundamentalAntiFundamental_barQQBlk [source]

The two isospin indices of the `barQ ∂ Q` block are one fundamental and one anti-fundamental, the barred symbol supplying the fundamental one.

```
lemma isSU2FundamentalAntiFundamental_barQQBlk (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' : Fin 3) :
    IsSU2FundamentalAntiFundamental B repGauge
      (fun n : Fin 2 → Fin 2 => h.barQQBlk f f' q l c c' (n 0) (n 1))
```

lemma repGauge_u1_barQQBlk [source]

A hypercharge transformation fixes the `barQ ∂ Q` block, the hypercharges of a species and its conjugate cancelling.

```
lemma repGauge_u1_barQQBlk (t : unitary ℂ) (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.barQQBlk f f' q l c c' w w')
      = h.barQQBlk f f' q l c c' w w'
```

def barQQColourStep [source]

The colour stage of the `barQ ∂ Q` block.

```
noncomputable def barQQColourStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    InvariantReductionToSpan
      (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ
        (Set.range fun n : Fin 2 → Fin 3 => h.barQQBlk f f' q l (n 0) (n 1) w w'))
```

lemma barQQColourStep_contraction [source]

The colour contraction of the `barQ ∂ Q` block, written out.

```
lemma barQQColourStep_contraction (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    (h.barQQColourStep f f' q l w w').spanningVector
      = ∑ a : Fin 3, h.barQQBlk f f' q l a a w w'
```

def barQQIsospinStep [source]

The isospin stage of the `barQ ∂ Q` block.

```
noncomputable def barQQIsospinStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) :
    InvariantReductionToSpan
      (fun V : specialUnitaryGroup (Fin 2) ℂ => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 2 =>
        (h.barQQColourStep f f' q l (n 0) (n 1)).spanningVector))
```

lemma isVectorDualLeftRightWeyl_barQQ [source]

The doubly contracted `barQ ∂ Q` block carries one four-vector index and a pair of dual opposite-chirality Weyl indices.

```
lemma isVectorDualLeftRightWeyl_barQQ (f f' : Fin 3) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.barQQIsospinStep f f' p.1 p.2).spanningVector)
```

def barQQLorentzStep [source]

The Lorentz stage of the `barQ ∂ Q` block.

```
noncomputable def barQQLorentzStep (f f' : Fin 3) :
    InvariantReductionToSpan (fun Λ : SL(2,ℂ) => repLorentz Λ)
      (Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.barQQIsospinStep f f' p.1 p.2).spanningVector))
```

def barQQKineticBlock [source]

The `barQ ∂ Q` block as a kinetic block.

```
noncomputable def barQQKineticBlock (f f' : Fin 3) : KineticBlock repGauge repLorentz where
```

def LbarLBlk [source]

The components of the block `L ∂ barL`: an underived lepton-doublet symbol against a once-derived conjugate lepton-doublet symbol.

```
noncomputable def LbarLBlk (f f' : Fin 3) :
    (Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Fin 3 → Fin 3 → Fin 2 → Fin 2 → B
```

lemma repGauge_su3_LbarLBlk [source]

A colour transformation fixes the `L ∂ barL` block, neither symbol carrying colour.

```
lemma repGauge_su3_LbarLBlk (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.LbarLBlk f f' q l c c' w w')
      = h.LbarLBlk f f' q l c c' w w'
```

lemma isSU2FundamentalAntiFundamental_LbarLBlk [source]

The two isospin indices of the `L ∂ barL` block are one fundamental and one anti-fundamental, the barred symbol supplying the fundamental one.

```
lemma isSU2FundamentalAntiFundamental_LbarLBlk (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' : Fin 3) :
    IsSU2FundamentalAntiFundamental B repGauge
      (fun n : Fin 2 → Fin 2 => h.LbarLBlk f f' q l c c' (n 0) (n 1))
```

lemma repGauge_u1_LbarLBlk [source]

A hypercharge transformation fixes the `L ∂ barL` block, the hypercharges of a species and its conjugate cancelling.

```
lemma repGauge_u1_LbarLBlk (t : unitary ℂ) (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.LbarLBlk f f' q l c c' w w')
      = h.LbarLBlk f f' q l c c' w w'
```

def LbarLColourStep [source]

The colour stage of the `L ∂ barL` block.

```
noncomputable def LbarLColourStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    InvariantReductionToSpan
      (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ
        (Set.range fun n : Fin 2 → Fin 3 => h.LbarLBlk f f' q l (n 0) (n 1) w w'))
```

lemma LbarLColourStep_contraction [source]

The colour contraction of the `L ∂ barL` block, written out.

```
lemma LbarLColourStep_contraction (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    (h.LbarLColourStep f f' q l w w').spanningVector
      = h.LbarLBlk f f' q l 0 0 w w'
```

def LbarLIsospinStep [source]

The isospin stage of the `L ∂ barL` block.

```
noncomputable def LbarLIsospinStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) :
    InvariantReductionToSpan
      (fun V : specialUnitaryGroup (Fin 2) ℂ => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 2 =>
        (h.LbarLColourStep f f' q l (n 0) (n 1)).spanningVector))
```

lemma isVectorDualLeftRightWeyl_LbarL [source]

The doubly contracted `L ∂ barL` block carries one four-vector index and a pair of dual opposite-chirality Weyl indices.

```
lemma isVectorDualLeftRightWeyl_LbarL (f f' : Fin 3) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.LbarLIsospinStep f f' p.1 p.2).spanningVector)
```

def LbarLLorentzStep [source]

The Lorentz stage of the `L ∂ barL` block.

```
noncomputable def LbarLLorentzStep (f f' : Fin 3) :
    InvariantReductionToSpan (fun Λ : SL(2,ℂ) => repLorentz Λ)
      (Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.LbarLIsospinStep f f' p.1 p.2).spanningVector))
```

def LbarLKineticBlock [source]

The `L ∂ barL` block as a kinetic block.

```
noncomputable def LbarLKineticBlock (f f' : Fin 3) : KineticBlock repGauge repLorentz where
```

def barLLBlk [source]

The components of the block `barL ∂ L`: an underived conjugate lepton-doublet symbol against a once-derived lepton-doublet symbol.

```
noncomputable def barLLBlk (f f' : Fin 3) :
    (Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Fin 3 → Fin 3 → Fin 2 → Fin 2 → B
```

lemma repGauge_su3_barLLBlk [source]

A colour transformation fixes the `barL ∂ L` block, neither symbol carrying colour.

```
lemma repGauge_su3_barLLBlk (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.barLLBlk f f' q l c c' w w')
      = h.barLLBlk f f' q l c c' w w'
```

lemma isSU2FundamentalAntiFundamental_barLLBlk [source]

The two isospin indices of the `barL ∂ L` block are one fundamental and one anti-fundamental, the barred symbol supplying the fundamental one.

```
lemma isSU2FundamentalAntiFundamental_barLLBlk (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' : Fin 3) :
    IsSU2FundamentalAntiFundamental B repGauge
      (fun n : Fin 2 → Fin 2 => h.barLLBlk f f' q l c c' (n 0) (n 1))
```

lemma repGauge_u1_barLLBlk [source]

A hypercharge transformation fixes the `barL ∂ L` block, the hypercharges of a species and its conjugate cancelling.

```
lemma repGauge_u1_barLLBlk (t : unitary ℂ) (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.barLLBlk f f' q l c c' w w')
      = h.barLLBlk f f' q l c c' w w'
```

def barLLColourStep [source]

The colour stage of the `barL ∂ L` block.

```
noncomputable def barLLColourStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    InvariantReductionToSpan
      (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ
        (Set.range fun n : Fin 2 → Fin 3 => h.barLLBlk f f' q l (n 0) (n 1) w w'))
```

lemma barLLColourStep_contraction [source]

The colour contraction of the `barL ∂ L` block, written out.

```
lemma barLLColourStep_contraction (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    (h.barLLColourStep f f' q l w w').spanningVector
      = h.barLLBlk f f' q l 0 0 w w'
```

def barLLIsospinStep [source]

The isospin stage of the `barL ∂ L` block.

```
noncomputable def barLLIsospinStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) :
    InvariantReductionToSpan
      (fun V : specialUnitaryGroup (Fin 2) ℂ => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 2 =>
        (h.barLLColourStep f f' q l (n 0) (n 1)).spanningVector))
```

lemma isVectorDualLeftRightWeyl_barLL [source]

The doubly contracted `barL ∂ L` block carries one four-vector index and a pair of dual opposite-chirality Weyl indices.

```
lemma isVectorDualLeftRightWeyl_barLL (f f' : Fin 3) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.barLLIsospinStep f f' p.1 p.2).spanningVector)
```

def barLLLorentzStep [source]

The Lorentz stage of the `barL ∂ L` block.

```
noncomputable def barLLLorentzStep (f f' : Fin 3) :
    InvariantReductionToSpan (fun Λ : SL(2,ℂ) => repLorentz Λ)
      (Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.barLLIsospinStep f f' p.1 p.2).spanningVector))
```

def barLLKineticBlock [source]

The `barL ∂ L` block as a kinetic block.

```
noncomputable def barLLKineticBlock (f f' : Fin 3) : KineticBlock repGauge repLorentz where
```

def ebareBlk [source]

The components of the block `e ∂ bare`: an underived lepton-singlet symbol against a once-derived conjugate lepton-singlet symbol.

```
noncomputable def ebareBlk (f f' : Fin 3) :
    (Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Fin 3 → Fin 3 → Fin 2 → Fin 2 → B
```

lemma repGauge_su3_ebareBlk [source]

A colour transformation fixes the `e ∂ bare` block, neither symbol carrying colour.

```
lemma repGauge_su3_ebareBlk (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.ebareBlk f f' q l c c' w w')
      = h.ebareBlk f f' q l c c' w w'
```

lemma repGauge_su2_ebareBlk [source]

An isospin transformation fixes the `e ∂ bare` block, neither symbol carrying isospin.

```
lemma repGauge_su2_ebareBlk (V : specialUnitaryGroup (Fin 2) ℂ) (f f' : Fin 3)
    (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.ebareBlk f f' q l c c' w w')
      = h.ebareBlk f f' q l c c' w w'
```

lemma repGauge_u1_ebareBlk [source]

A hypercharge transformation fixes the `e ∂ bare` block, the hypercharges of a species and its conjugate cancelling.

```
lemma repGauge_u1_ebareBlk (t : unitary ℂ) (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.ebareBlk f f' q l c c' w w')
```

```
    = h.ebareBlk f f' q l c c' w w'
```

def ebareColourStep [source]

The colour stage of the `e ∂ bare` block.

```
noncomputable def ebareColourStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    InvariantReductionToSpan
      (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ
        (Set.range fun n : Fin 2 → Fin 3 => h.ebareBlk f f' q l (n 0) (n 1) w w'))
```

lemma ebareColourStep_contraction [source]

The colour contraction of the `e ∂ bare` block, written out.

```
lemma ebareColourStep_contraction (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    (h.ebareColourStep f f' q l w w').spanningVector
      = h.ebareBlk f f' q l 0 0 w w'
```

def ebareIsospinStep [source]

The isospin stage of the `e ∂ bare` block.

```
noncomputable def ebareIsospinStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) :
    InvariantReductionToSpan
      (fun V : specialUnitaryGroup (Fin 2) ℂ => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 2 =>
        (h.ebareColourStep f f' q l (n 0) (n 1)).spanningVector))
```

lemma isVectorDualLeftRightWeyl_ebare [source]

The doubly contracted `e ∂ bare` block carries one four-vector index and a pair of dual opposite-chirality Weyl indices.

```
lemma isVectorDualLeftRightWeyl_ebare (f f' : Fin 3) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.ebareIsospinStep f f' p.1 p.2).spanningVector)
```

def ebareLorentzStep [source]

The Lorentz stage of the `e ∂ bare` block.

```
noncomputable def ebareLorentzStep (f f' : Fin 3) :
    InvariantReductionToSpan (fun Λ : SL(2,ℂ) => repLorentz Λ)
      (Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.ebareIsospinStep f f' p.1 p.2).spanningVector))
```

def ebareKineticBlock [source]

The `e ∂ bare` block as a kinetic block.

```
noncomputable def ebareKineticBlock (f f' : Fin 3) : KineticBlock repGauge repLorentz where
```

def bareeBlk [source]

The components of the block `bare ∂ e`: an underived conjugate lepton-singlet symbol against a once-derived lepton-singlet symbol.

```
noncomputable def bareeBlk (f f' : Fin 3) :
    (Fin 1 ⊕ Fin 3) → Fin 2 × Fin 2 → Fin 3 → Fin 3 → Fin 2 → Fin 2 → B
```

lemma repGauge_su3_bareeBlk [source]

A colour transformation fixes the `bare ∂ e` block, neither symbol carrying colour.

```
lemma repGauge_su3_bareeBlk (U : specialUnitaryGroup (Fin 3) ℂ) (f f' : Fin 3)
    (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((U, 1, 1) : GaugeGroupI) (h.bareeBlk f f' q l c c' w w')
      = h.bareeBlk f f' q l c c' w w'
```

lemma repGauge_su2_bareeBlk [source]

An isospin transformation fixes the `bare ∂ e` block, neither symbol carrying isospin.

```
lemma repGauge_su2_bareeBlk (V : specialUnitaryGroup (Fin 2) ℂ) (f f' : Fin 3)
    (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, V, 1) : GaugeGroupI) (h.bareeBlk f f' q l c c' w w')
      = h.bareeBlk f f' q l c c' w w'
```

lemma repGauge_u1_bareeBlk [source]

A hypercharge transformation fixes the `bare ∂ e` block, the hypercharges of a species and its conjugate cancelling.

```
lemma repGauge_u1_bareeBlk (t : unitary ℂ) (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) (c c' w w' : _) :
    repGauge ((1, 1, t) : GaugeGroupI) (h.bareeBlk f f' q l c c' w w')
      = h.bareeBlk f f' q l c c' w w'
```

def bareeColourStep [source]

The colour stage of the `bare ∂ e` block.

```
noncomputable def bareeColourStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    InvariantReductionToSpan
      (fun U : specialUnitaryGroup (Fin 3) ℂ => repGauge ((U, 1, 1) : GaugeGroupI))
      (Submodule.span ℂ
        (Set.range fun n : Fin 2 → Fin 3 => h.bareeBlk f f' q l (n 0) (n 1) w w'))
```

lemma bareeColourStep_contraction [source]

The colour contraction of the `bare ∂ e` block, written out.

```
lemma bareeColourStep_contraction (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2)
    (w w' : Fin 2) :
    (h.bareeColourStep f f' q l w w').spanningVector
      = h.bareeBlk f f' q l 0 0 w w'
```

def bareeIsospinStep [source]

The isospin stage of the `bare ∂ e` block.

```
noncomputable def bareeIsospinStep (f f' : Fin 3) (q : Fin 1 ⊕ Fin 3)
    (l : Fin 2 × Fin 2) :
    InvariantReductionToSpan
      (fun V : specialUnitaryGroup (Fin 2) ℂ => repGauge ((1, V, 1) : GaugeGroupI))
      (Submodule.span ℂ (Set.range fun n : Fin 2 → Fin 2 =>
        (h.bareeColourStep f f' q l (n 0) (n 1)).spanningVector))
```

lemma isVectorDualLeftRightWeyl_baree [source]

The doubly contracted `bare ∂ e` block carries one four-vector index and a pair of dual opposite-chirality Weyl indices.

```
lemma isVectorDualLeftRightWeyl_baree (f f' : Fin 3) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.bareeIsospinStep f f' p.1 p.2).spanningVector)
```

def bareeLorentzStep [source]

The Lorentz stage of the `bare ∂ e` block.

```
noncomputable def bareeLorentzStep (f f' : Fin 3) :
    InvariantReductionToSpan (fun Λ : SL(2,ℂ) => repLorentz Λ)
      (Submodule.span ℂ (Set.range fun p : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 =>
        (h.bareeIsospinStep f f' p.1 p.2).spanningVector))
```

def bareeKineticBlock [source]

The `bare ∂ e` block as a kinetic block.

```
noncomputable def bareeKineticBlock (f f' : Fin 3) : KineticBlock repGauge repLorentz where
```

2.468 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.MassDimEight

PhyslibAlpha/Particles/StandardModel/IsFermionSector/MassWeight/MassDimEight.lean on GitHub.

lemma isStableUnder_iSup_range_deriv_one [source]

The join, over the derivative direction, of the ranges of a once-derived symbol map is carried into itself by the Lorentz group: a Lorentz transformation mixes the derivative slot into the other directions and moves the covector, and both stay inside the join.

```
lemma isStableUnder_iSup_range_deriv_one {M : Type} [AddCommGroup M] [Module ℂ M]
    {ρ : Representation ℂ SL(2,ℂ) M}
    {F : {n : ℕ} → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ M →ₗ[ℂ] B}
```

```
    (hF : IsLorentzCovDerivTransforms repLorentz ρ F) (Λ : SL(2,ℂ)) :
    ∀ y ∈ (⨆ μ : Fin 1 ⊕ Fin 3, LinearMap.range (F ![μ])),
      repLorentz Λ y ∈ ⨆ μ : Fin 1 ⊕ Fin 3, LinearMap.range (F ![μ])
```

lemma isStableUnder_range_underived [source]

The range of an underived symbol map is carried into itself by both groups.

```
lemma isStableUnder_range_underived {M : Type} [AddCommGroup M] [Module ℂ M]
    {ρG : Representation ℂ GaugeGroupI M} {ρL : Representation ℂ SL(2,ℂ) M}
    {F : {n : ℕ} → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ M →ₗ[ℂ] B}
    (hG : ∀ (g : GaugeGroupI) (φ : Module.Dual ℂ M),
      repGauge g (F (![] : Fin 0 → Fin 1 ⊕ Fin 3) φ)
        = F (![] : Fin 0 → Fin 1 ⊕ Fin 3) (ρG.dual g φ))
    (hL : IsLorentzCovDerivTransforms repLorentz ρL F) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (LinearMap.range (F (![] : Fin 0 → Fin 1 ⊕ Fin 3)))
```

lemma isStableUnder_iSup_range_derived [source]

The join, over the derivative direction, of the ranges of a once-derived symbol map is carried into itself by both groups.

```
lemma isStableUnder_iSup_range_derived {M : Type} [AddCommGroup M] [Module ℂ M]
    {ρG : Representation ℂ GaugeGroupI M} {ρL : Representation ℂ SL(2,ℂ) M}
    {F : {n : ℕ} → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ M →ₗ[ℂ] B}
    (hG : ∀ (g : GaugeGroupI) (μ : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℂ M),
      repGauge g (F ![μ] φ) = F ![μ] (ρG.dual g φ))
    (hL : IsLorentzCovDerivTransforms repLorentz ρL F) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz)
      (⨆ μ : Fin 1 ⊕ Fin 3, LinearMap.range (F ![μ]))
```

def dbardPairSubmodule [source]

The submodule of the `d ∂ bard` block of a generation pair: an underived down-singlet range against the once-derived conjugate down-singlet ranges, joined over the derivative direction.

```
@[nolint unusedArguments]
noncomputable def dbardPairSubmodule (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) (f f' : Fin 3) :
    Submodule ℂ B
```

lemma isStableUnder_dbardPairSubmodule [source]

The `d ∂ bard` block submodule is carried into itself by both groups.

```
lemma isStableUnder_dbardPairSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.dbardPairSubmodule f f')
```

lemma dbardPairSubmodule_le_blockSpan [source]

The `d ∂ bard` block submodule lies in the span of the block's components.

```
lemma dbardPairSubmodule_le_blockSpan (f f' : Fin 3) :
    h.dbardPairSubmodule f f' ≤ (h.dbardKineticBlock f f').blockSpan
```

lemma reducesInvariantsTo_dbard [source]

The `d ∂ bard` block reduces to its kinetic term.

```
lemma reducesInvariantsTo_dbard (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.dbardPairSubmodule f f')
      (ℂ • (h.dbardKineticBlock f f').kineticTerm)
```

def barddPairSubmodule [source]

The submodule of the `bard ∂ d` block of a generation pair: an underived conjugate down-singlet range against the once-derived down-singlet ranges, joined over the derivative direction.

```
@[nolint unusedArguments]
noncomputable def barddPairSubmodule (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) (f f' : Fin 3) :
    Submodule ℂ B
```

lemma isStableUnder_barddPairSubmodule [source]

The `bard ∂ d` block submodule is carried into itself by both groups.

```
lemma isStableUnder_barddPairSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.barddPairSubmodule f f')
```

lemma barddPairSubmodule_le_blockSpan [source]

The `bard ∂ d` block submodule lies in the span of the block's components.

```
lemma barddPairSubmodule_le_blockSpan (f f' : Fin 3) :
    h.barddPairSubmodule f f' ≤ (h.barddKineticBlock f f').blockSpan
```

lemma reducesInvariantsTo_bardd [source]

The `bard ∂ d` block reduces to its kinetic term.

```
lemma reducesInvariantsTo_bardd (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.barddPairSubmodule f f')
      (ℂ • (h.barddKineticBlock f f').kineticTerm)
```

def ubaruPairSubmodule [source]

The submodule of the `u ∂ baru` block of a generation pair: an underived up-singlet range against the once-derived conjugate up-singlet ranges, joined over the derivative direction.

```
@[nolint unusedArguments]
noncomputable def ubaruPairSubmodule (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) (f f' : Fin 3) :
    Submodule ℂ B
```

lemma isStableUnder_ubaruPairSubmodule [source]

The `u ∂ baru` block submodule is carried into itself by both groups.

```
lemma isStableUnder_ubaruPairSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.ubaruPairSubmodule f f')
```

lemma ubaruPairSubmodule_le_blockSpan [source]

The `u ∂ baru` block submodule lies in the span of the block's components.

```
lemma ubaruPairSubmodule_le_blockSpan (f f' : Fin 3) :
    h.ubaruPairSubmodule f f' ≤ (h.ubaruKineticBlock f f').blockSpan
```

lemma reducesInvariantsTo_ubaru [source]

The `u ∂ baru` block reduces to its kinetic term.

```
lemma reducesInvariantsTo_ubaru (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.ubaruPairSubmodule f f')
      (ℂ • (h.ubaruKineticBlock f f').kineticTerm)
```

def baruuPairSubmodule [source]

The submodule of the `baru ∂ u` block of a generation pair: an underived conjugate up-singlet range against the once-derived up-singlet ranges, joined over the derivative direction.

```
@[nolint unusedArguments]
noncomputable def baruuPairSubmodule (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) (f f' : Fin 3) :
    Submodule ℂ B
```

lemma isStableUnder_baruuPairSubmodule [source]

The `baru ∂ u` block submodule is carried into itself by both groups.

```
lemma isStableUnder_baruuPairSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.baruuPairSubmodule f f')
```

lemma baruuPairSubmodule_le_blockSpan [source]

The `baru ∂ u` block submodule lies in the span of the block's components.

```
lemma baruuPairSubmodule_le_blockSpan (f f' : Fin 3) :
    h.baruuPairSubmodule f f' ≤ (h.baruuKineticBlock f f').blockSpan
```

lemma reducesInvariantsTo_baruu [source]

The `baru ∂ u` block reduces to its kinetic term.

```
lemma reducesInvariantsTo_baruu (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.baruuPairSubmodule f f')
      (ℂ • (h.baruuKineticBlock f f').kineticTerm)
```

def QbarQPairSubmodule [source]

The submodule of the `Q ∂ barQ` block of a generation pair: an underived quark-doublet range against the once-derived conjugate quark-doublet ranges, joined over the derivative direction.

```
@[nolint unusedArguments]
noncomputable def QbarQPairSubmodule (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) (f f' : Fin 3) :
    Submodule ℂ B
```

lemma isStableUnder_QbarQPairSubmodule [source]

The `Q ∂ barQ` block submodule is carried into itself by both groups.

```
lemma isStableUnder_QbarQPairSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.QbarQPairSubmodule f f')
```

lemma QbarQPairSubmodule_le_blockSpan [source]

The `Q ∂ barQ` block submodule lies in the span of the block's components.

```
lemma QbarQPairSubmodule_le_blockSpan (f f' : Fin 3) :
    h.QbarQPairSubmodule f f' ≤ (h.QbarQKineticBlock f f').blockSpan
```

lemma reducesInvariantsTo_QbarQ [source]

The `Q ∂ barQ` block reduces to its kinetic term.

```
lemma reducesInvariantsTo_QbarQ (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.QbarQPairSubmodule f f')
      (ℂ • (h.QbarQKineticBlock f f').kineticTerm)
```

def barQQPairSubmodule [source]

The submodule of the `barQ ∂ Q` block of a generation pair: an underived conjugate quark-doublet range against the once-derived quark-doublet ranges, joined over the derivative direction.

```
@[nolint unusedArguments]
noncomputable def barQQPairSubmodule (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) (f f' : Fin 3) :
    Submodule ℂ B
```

lemma isStableUnder_barQQPairSubmodule [source]

The `barQ ∂ Q` block submodule is carried into itself by both groups.

```
lemma isStableUnder_barQQPairSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.barQQPairSubmodule f f')
```

lemma barQQPairSubmodule_le_blockSpan [source]

The `barQ ∂ Q` block submodule lies in the span of the block's components.

```
lemma barQQPairSubmodule_le_blockSpan (f f' : Fin 3) :
    h.barQQPairSubmodule f f' ≤ (h.barQQKineticBlock f f').blockSpan
```

lemma reducesInvariantsTo_barQQ [source]

The `barQ ∂ Q` block reduces to its kinetic term.

```
lemma reducesInvariantsTo_barQQ (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.barQQPairSubmodule f f')
      (ℂ • (h.barQQKineticBlock f f').kineticTerm)
```

def LbarLPairSubmodule [source]

The submodule of the `L ∂ barL` block of a generation pair: an underived lepton-doublet range against the once-derived conjugate lepton-doublet ranges, joined over the derivative direction.

```
@[nolint unusedArguments]
noncomputable def LbarLPairSubmodule (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) (f f' : Fin 3) :
    Submodule ℂ B
```

lemma isStableUnder_LbarLPairSubmodule [source]

The `L ∂ barL` block submodule is carried into itself by both groups.

```
lemma isStableUnder_LbarLPairSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.LbarLPairSubmodule f f')
```

lemma LbarLPairSubmodule_le_blockSpan [source]

The `L ∂ barL` block submodule lies in the span of the block's components.

```
lemma LbarLPairSubmodule_le_blockSpan (f f' : Fin 3) :
    h.LbarLPairSubmodule f f' ≤ (h.LbarLKineticBlock f f').blockSpan
```

lemma reducesInvariantsTo_LbarL [source]

The `L ∂ barL` block reduces to its kinetic term.

```
lemma reducesInvariantsTo_LbarL (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.LbarLPairSubmodule f f')
      (ℂ • (h.LbarLKineticBlock f f').kineticTerm)
```

def barLLPairSubmodule [source]

The submodule of the `barL ∂ L` block of a generation pair: an underived conjugate lepton-doublet range against the once-derived lepton-doublet ranges, joined over the derivative direction.

```
@[nolint unusedArguments]
noncomputable def barLLPairSubmodule (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) (f f' : Fin 3) :
    Submodule ℂ B
```

lemma isStableUnder_barLLPairSubmodule [source]

The `barL ∂ L` block submodule is carried into itself by both groups.

```
lemma isStableUnder_barLLPairSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.barLLPairSubmodule f f')
```

lemma barLLPairSubmodule_le_blockSpan [source]

The `barL ∂ L` block submodule lies in the span of the block's components.

```
lemma barLLPairSubmodule_le_blockSpan (f f' : Fin 3) :
    h.barLLPairSubmodule f f' ≤ (h.barLLKineticBlock f f').blockSpan
```

lemma reducesInvariantsTo_barLL [source]

The `barL ∂ L` block reduces to its kinetic term.

```
lemma reducesInvariantsTo_barLL (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.barLLPairSubmodule f f')
      (ℂ • (h.barLLKineticBlock f f').kineticTerm)
```

def ebarePairSubmodule [source]

The submodule of the `e ∂ bare` block of a generation pair: an underived lepton-singlet range against the once-derived conjugate lepton-singlet ranges, joined over the derivative direction.

```
@[nolint unusedArguments]
noncomputable def ebarePairSubmodule (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) (f f' : Fin 3) :
    Submodule ℂ B
```

lemma isStableUnder_ebarePairSubmodule [source]

The `e ∂ bare` block submodule is carried into itself by both groups.

```
lemma isStableUnder_ebarePairSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.ebarePairSubmodule f f')
```

lemma ebarePairSubmodule_le_blockSpan [source]

The `e ∂ bare` block submodule lies in the span of the block's components.

```
lemma ebarePairSubmodule_le_blockSpan (f f' : Fin 3) :
    h.ebarePairSubmodule f f' ≤ (h.ebareKineticBlock f f').blockSpan
```

lemma reducesInvariantsTo_ebare [source]

The `e ∂ bare` block reduces to its kinetic term.

```
lemma reducesInvariantsTo_ebare (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.ebarePairSubmodule f f')
      (ℂ • (h.ebareKineticBlock f f').kineticTerm)
```

def bareePairSubmodule [source]

The submodule of the `bare ∂ e` block of a generation pair: an underived conjugate lepton-singlet range against the once-derived lepton-singlet ranges, joined over the derivative direction.

```
@[nolint unusedArguments]
noncomputable def bareePairSubmodule (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) (f f' : Fin 3) :
    Submodule ℂ B
```

lemma isStableUnder_bareePairSubmodule [source]

The `bare ∂ e` block submodule is carried into itself by both groups.

```
lemma isStableUnder_bareePairSubmodule (f f' : Fin 3) :
    IsStableUnder (gaugeLorentzMaps repGauge repLorentz) (h.bareePairSubmodule f f')
```

lemma bareePairSubmodule_le_blockSpan [source]

The `bare ∂ e` block submodule lies in the span of the block's components.

```
lemma bareePairSubmodule_le_blockSpan (f f' : Fin 3) :
    h.bareePairSubmodule f f' ≤ (h.bareeKineticBlock f f').blockSpan
```

lemma reducesInvariantsTo_baree [source]

The `bare ∂ e` block reduces to its kinetic term.

```
lemma reducesInvariantsTo_baree (f f' : Fin 3) :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.bareePairSubmodule f f')
      (ℂ • (h.bareeKineticBlock f f').kineticTerm)
```

lemma etaExpand_deriv_one [source]

A family of one derivative direction is the tuple of its own entry.

```
lemma etaExpand_deriv_one (l : Fin 1 → Fin 1 ⊕ Fin 3) : ![l 0] = l
```

lemma range_d_le_derivSubmodule [source]

The range of the down-singlet symbols lies in the derivative submodule of its slots.

```
lemma range_d_le_derivSubmodule {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (d f l) ≤ h.derivSubmodule n
```

lemma range_bard_le_derivSubmodule [source]

The range of the conjugate down-singlet symbols lies in the derivative submodule of its slots.

```
lemma range_bard_le_derivSubmodule {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (bard f l) ≤ h.derivSubmodule n
```

lemma range_u_le_derivSubmodule [source]

The range of the up-singlet symbols lies in the derivative submodule of its slots.

```
lemma range_u_le_derivSubmodule {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (u f l) ≤ h.derivSubmodule n
```

lemma range_baru_le_derivSubmodule [source]

The range of the conjugate up-singlet symbols lies in the derivative submodule of its slots.

```
lemma range_baru_le_derivSubmodule {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (baru f l) ≤ h.derivSubmodule n
```

lemma range_Q_le_derivSubmodule [source]

The range of the quark-doublet symbols lies in the derivative submodule of its slots.

```
lemma range_Q_le_derivSubmodule {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (Q f l) ≤ h.derivSubmodule n
```

lemma range_barQ_le_derivSubmodule [source]

The range of the conjugate quark-doublet symbols lies in the derivative submodule of its slots.

```
lemma range_barQ_le_derivSubmodule {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (barQ f l) ≤ h.derivSubmodule n
```

lemma range_L_le_derivSubmodule [source]

The range of the lepton-doublet symbols lies in the derivative submodule of its slots.

```
lemma range_L_le_derivSubmodule {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (L f l) ≤ h.derivSubmodule n
```

lemma range_barL_le_derivSubmodule [source]

The range of the conjugate lepton-doublet symbols lies in the derivative submodule of its slots.

```
lemma range_barL_le_derivSubmodule {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (barL f l) ≤ h.derivSubmodule n
```

lemma range_e_le_derivSubmodule [source]

The range of the lepton-singlet symbols lies in the derivative submodule of its slots.

```
lemma range_e_le_derivSubmodule {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (e f l) ≤ h.derivSubmodule n
```

lemma range_bare_le_derivSubmodule [source]

The range of the conjugate lepton-singlet symbols lies in the derivative submodule of its slots.

```
lemma range_bare_le_derivSubmodule {n : ℕ} (f : Fin 3) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (bare f l) ≤ h.derivSubmodule n
```

lemma dbardKineticTerm_mem_massWeightSubmodule [source]

The kinetic term of the `d ∂ bard` block has mass weight eight: every component of the block is an underived tower against a once-derived one.

```
lemma dbardKineticTerm_mem_massWeightSubmodule (f f' : Fin 3) :
    (h.dbardKineticBlock f f').kineticTerm ∈ h.massWeightSubmodule 8
```

lemma barddKineticTerm_mem_massWeightSubmodule [source]

The kinetic term of the `bard ∂ d` block has mass weight eight: every component of the block is an underived tower against a once-derived one.

```
lemma barddKineticTerm_mem_massWeightSubmodule (f f' : Fin 3) :
    (h.barddKineticBlock f f').kineticTerm ∈ h.massWeightSubmodule 8
```

lemma ubaruKineticTerm_mem_massWeightSubmodule [source]

The kinetic term of the `u ∂ baru` block has mass weight eight: every component of the block is an underived tower against a once-derived one.

```
lemma ubaruKineticTerm_mem_massWeightSubmodule (f f' : Fin 3) :
    (h.ubaruKineticBlock f f').kineticTerm ∈ h.massWeightSubmodule 8
```

lemma baruuKineticTerm_mem_massWeightSubmodule [source]

The kinetic term of the `baru ∂ u` block has mass weight eight: every component of the block is an underived tower against a once-derived one.

```
lemma baruuKineticTerm_mem_massWeightSubmodule (f f' : Fin 3) :
    (h.baruuKineticBlock f f').kineticTerm ∈ h.massWeightSubmodule 8
```

lemma QbarQKineticTerm_mem_massWeightSubmodule [source]

The kinetic term of the `Q ∂ barQ` block has mass weight eight: every component of the block is an underived tower against a once-derived one.

```
lemma QbarQKineticTerm_mem_massWeightSubmodule (f f' : Fin 3) :
    (h.QbarQKineticBlock f f').kineticTerm ∈ h.massWeightSubmodule 8
```

lemma barQQKineticTerm_mem_massWeightSubmodule [source]

The kinetic term of the `barQ ∂ Q` block has mass weight eight: every component of the block is an underived tower against a once-derived one.

```
lemma barQQKineticTerm_mem_massWeightSubmodule (f f' : Fin 3) :
    (h.barQQKineticBlock f f').kineticTerm ∈ h.massWeightSubmodule 8
```

lemma LbarLKineticTerm_mem_massWeightSubmodule [source]

The kinetic term of the `L ∂ barL` block has mass weight eight: every component of the block is an underived tower against a once-derived one.

```
lemma LbarLKineticTerm_mem_massWeightSubmodule (f f' : Fin 3) :
    (h.LbarLKineticBlock f f').kineticTerm ∈ h.massWeightSubmodule 8
```

lemma barLLKineticTerm_mem_massWeightSubmodule [source]

The kinetic term of the `barL ∂ L` block has mass weight eight: every component of the block is an underived tower against a once-derived one.

```
lemma barLLKineticTerm_mem_massWeightSubmodule (f f' : Fin 3) :
    (h.barLLKineticBlock f f').kineticTerm ∈ h.massWeightSubmodule 8
```

lemma ebareKineticTerm_mem_massWeightSubmodule [source]

The kinetic term of the `e ∂ bare` block has mass weight eight: every component of the block is an underived tower against a once-derived one.

```
lemma ebareKineticTerm_mem_massWeightSubmodule (f f' : Fin 3) :
    (h.ebareKineticBlock f f').kineticTerm ∈ h.massWeightSubmodule 8
```

lemma bareeKineticTerm_mem_massWeightSubmodule [source]

The kinetic term of the `bare ∂ e` block has mass weight eight: every component of the block is an underived tower against a once-derived one.

```
lemma bareeKineticTerm_mem_massWeightSubmodule (f f' : Fin 3) :
    (h.bareeKineticBlock f f').kineticTerm ∈ h.massWeightSubmodule 8
```

def kineticSpan [source]

The kinetic span of the fermion sector at mass weight eight: the join of the ten conjugate pairings, each over the nine pairs of generations.

```
noncomputable def kineticSpan (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
    hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) : Submodule ℂ B
```

lemma span_dbard_le_kineticSpan [source]

The line through the `dbard` kinetic term lies in the kinetic span.

```
lemma span_dbard_le_kineticSpan (f f' : Fin 3) :
    ℂ • (h.dbardKineticBlock f f').kineticTerm ≤ h.kineticSpan
```

lemma span_bardd_le_kineticSpan [source]

The line through the `bardd` kinetic term lies in the kinetic span.

```
lemma span_bardd_le_kineticSpan (f f' : Fin 3) :
    ℂ • (h.barddKineticBlock f f').kineticTerm ≤ h.kineticSpan
```

lemma span_ubaru_le_kineticSpan [source]

The line through the `ubaru` kinetic term lies in the kinetic span.

```
lemma span_ubaru_le_kineticSpan (f f' : Fin 3) :
    ℂ • (h.ubaruKineticBlock f f').kineticTerm ≤ h.kineticSpan
```

lemma span_baruu_le_kineticSpan [source]

The line through the `baruu` kinetic term lies in the kinetic span.

```
lemma span_baruu_le_kineticSpan (f f' : Fin 3) :
    ℂ • (h.baruuKineticBlock f f').kineticTerm ≤ h.kineticSpan
```

lemma span_QbarQ_le_kineticSpan [source]

The line through the `QbarQ` kinetic term lies in the kinetic span.

```
lemma span_QbarQ_le_kineticSpan (f f' : Fin 3) :
    ℂ • (h.QbarQKineticBlock f f').kineticTerm ≤ h.kineticSpan
```

lemma span_barQQ_le_kineticSpan [source]

The line through the `barQQ` kinetic term lies in the kinetic span.

```
lemma span_barQQ_le_kineticSpan (f f' : Fin 3) :
    ℂ • (h.barQQKineticBlock f f').kineticTerm ≤ h.kineticSpan
```

lemma span_LbarL_le_kineticSpan [source]

The line through the `LbarL` kinetic term lies in the kinetic span.

```
lemma span_LbarL_le_kineticSpan (f f' : Fin 3) :
    ℂ • (h.LbarLKineticBlock f f').kineticTerm ≤ h.kineticSpan
```

lemma span_barLL_le_kineticSpan [source]

The line through the `barLL` kinetic term lies in the kinetic span.

```
lemma span_barLL_le_kineticSpan (f f' : Fin 3) :
    ℂ • (h.barLLKineticBlock f f').kineticTerm ≤ h.kineticSpan
```

lemma span_ebare_le_kineticSpan [source]

The line through the `ebare` kinetic term lies in the kinetic span.

```
lemma span_ebare_le_kineticSpan (f f' : Fin 3) :
    ℂ • (h.ebareKineticBlock f f').kineticTerm ≤ h.kineticSpan
```

lemma span_baree_le_kineticSpan [source]

The line through the `baree` kinetic term lies in the kinetic span.

```
lemma span_baree_le_kineticSpan (f f' : Fin 3) :
    ℂ • (h.bareeKineticBlock f f').kineticTerm ≤ h.kineticSpan
```

lemma isFixedBy_kineticSpan [source]

The kinetic span is fixed pointwise by both groups, each of its generators being a gauge and Lorentz invariant.

```
lemma isFixedBy_kineticSpan :
    IsFixedBy (gaugeLorentzMaps repGauge repLorentz) h.kineticSpan
```

lemma kineticSpan_le_massWeightSubmodule [source]

The kinetic span lies at mass weight eight.

```
lemma kineticSpan_le_massWeightSubmodule : h.kineticSpan ≤ h.massWeightSubmodule 8
```

lemma kineticSpan_le_invariants [source]

The kinetic span is a space of gauge invariants.

```
lemma kineticSpan_le_invariants : h.kineticSpan ≤ repGauge.invariants
```

lemma kineticSpan_le_lorentzInvariants [source]

The kinetic span is a space of Lorentz invariants.

```
lemma kineticSpan_le_lorentzInvariants : h.kineticSpan ≤ repLorentz.invariants
```

def kineticBlockSubmodule [source]

The join of the ten block submodules over the nine pairs of generations.

```
noncomputable def kineticBlockSubmodule (h : IsFermionSector B repGauge hrepGauge_mul
      repLorentz hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) :
    Submodule ℂ B
```

lemma massWeightSubmoduleGaugeWeightEight_piece_zero_le [source]

The weight-zero piece at mass weight eight lies in the join of the ten block submodules: each of the ten conjugate pairings of `massWeightSubmoduleGaugeWeightEight_piece_zero` is an underived range against a once-derived one, and the derivative direction is joined over.

```
lemma massWeightSubmoduleGaugeWeightEight_piece_zero_le :
    (h.massWeightSubmoduleGaugeWeightEight).piece 0 ≤ h.kineticBlockSubmodule
```

lemma reducesInvariantsTo_kineticBlockSubmodule [source]

The join of the ten block submodules reduces to the kinetic span.

```
lemma reducesInvariantsTo_kineticBlockSubmodule :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) h.kineticBlockSubmodule
      h.kineticSpan
```

lemma reducesInvariantsTo_kineticSpan [source]

The fermion sector at mass weight eight reduces, for the gauge and Lorentz groups together, to the kinetic span: the torus puts an invariant in the weight-zero piece, which lies in the ten block submodules, and those reduce to the kinetic terms.

```
lemma reducesInvariantsTo_kineticSpan :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.massWeightSubmodule 8)
      h.kineticSpan
```

lemma mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff [source]

The classification of mass weight eight as an equivalence, in the shape of the sibling sectors: an element of `massWeightSubmodule 8 ⊔ S` is fixed by both groups exactly when it is a combination of the kinetic terms up to a remainder in `S` fixed by both groups.

```
lemma mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff
    (S : Submodule ℂ B) (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule 8 ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ h.kineticSpan
```

2.469 PhyslibAlpha.Particles.StandardModel.IsFermionSector.MassWeight.MassDimLTEight

PhyslibAlpha/Particles/StandardModel/IsFermionSector/MassWeight/MassDimLTEight.lean on GitHub.

def IsDualLeftWeyl [source]

A two-element family `X` of elements of `B` carries a dual undotted spinor index: it transforms by the contragredient of the left-handed Weyl representation.

```
def IsDualLeftWeyl (repLorentz : Representation ℂ SL(2,ℂ) B) (X : Fin 2 → B) : Prop
```

def IsDualRightWeyl [source]

A two-element family `Y` of elements of `B` carries a dual dotted spinor index: it transforms by the contragredient of the right-handed Weyl representation, which is the conjugate of the undotted law.

```
def IsDualRightWeyl (repLorentz : Representation ℂ SL(2,ℂ) B) (Y : Fin 2 → B) : Prop
```

def IsVectorDualLeftWeyl [source]

A family `X` carrying one four-vector index and one dual undotted spinor index: the once-derived form of `IsDualLeftWeyl`, the derivative slot moving by the columns of the Lorentz matrix. Only the value index of a fermion symbol is dualised, so the derivative slot keeps the plain Lorentz law.

```
def IsVectorDualLeftWeyl (repLorentz : Representation ℂ SL(2,ℂ) B)
    (X : (Fin 1 ⊕ Fin 3) → Fin 2 → B) : Prop
```

def IsVectorDualRightWeyl [source]

A family `Y` carrying one four-vector index and one dual dotted spinor index: the once-derived form of `IsDualRightWeyl`.

```
def IsVectorDualRightWeyl (repLorentz : Representation ℂ SL(2,ℂ) B)
    (Y : (Fin 1 ⊕ Fin 3) → Fin 2 → B) : Prop
```

lemma isDualLeftRightWeyl_mul [source]

An undotted family times a dotted one is a dual left-right Weyl family.

```
lemma isDualLeftRightWeyl_mul {X Y : Fin 2 → B} (hX : IsDualLeftWeyl repLorentz X)
    (hY : IsDualRightWeyl repLorentz Y) :
    IsDualLeftRightWeyl B repLorentz
      (ofPairComponents (k := .downL) (k' := .downR) fun a b => X a * Y b)
```

lemma isDualLeftRightWeyl_mul_swap [source]

A dotted family times an undotted one is a dual left-right Weyl family, the two spinor slots exchanged.

```
lemma isDualLeftRightWeyl_mul_swap {X Y : Fin 2 → B} (hX : IsDualRightWeyl repLorentz X)
    (hY : IsDualLeftWeyl repLorentz Y) :
    IsDualLeftRightWeyl B repLorentz
      (ofPairComponents (k := .downL) (k' := .downR) fun a b => X b * Y a)
```

lemma isVectorDualLeftRightWeyl_mul [source]

An undotted family times a once-derived dotted one is a vector dual left-right Weyl family: the derivative supplies the four-vector index.

```
lemma isVectorDualLeftRightWeyl_mul {X : Fin 2 → B} {Y : (Fin 1 ⊕ Fin 3) → Fin 2 → B}
    (hX : IsDualLeftWeyl repLorentz X) (hY : IsVectorDualRightWeyl repLorentz Y) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p => X p.2.1 * Y p.1 p.2.2)
```

lemma isVectorDualLeftRightWeyl_mul_swap [source]

A dotted family times a once-derived undotted one is a vector dual left-right Weyl family, the two spinor slots exchanged.

```
lemma isVectorDualLeftRightWeyl_mul_swap {X : Fin 2 → B} {Y : (Fin 1 ⊕ Fin 3) → Fin 2 → B}
    (hX : IsDualRightWeyl repLorentz X) (hY : IsVectorDualLeftWeyl repLorentz Y) :
    IsVectorDualLeftRightWeyl B repLorentz
      (ofDualVectorComponents fun p => X p.2.2 * Y p.1 p.2.1)
```

lemma mem_of_lorentz_invariant_biSup_dualLeftRightWeyl_span [source]

A finite join of the spans of dual left-right Weyl families carries no Lorentz invariant modulo a Lorentz-stable submodule: such a family has no invariant, so each span reduces to `⊥`, and `ReducesInvariantsTo.biSup` combines the reductions. A Lorentz invariant of the join together with `S` lies in `S`.

```
lemma mem_of_lorentz_invariant_biSup_dualLeftRightWeyl_span {ι : Type} [DecidableEq ι]
    {T : ι → Fin 2 × Fin 2 → B} (hT : ∀ i, IsDualLeftRightWeyl B repLorentz
      (ofPairComponents (k := .downL) (k' := .downR) fun a b => T i (a, b)))
    (S : Submodule ℂ B) (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (s : Finset ι)
    {x : B} (hx : x ∈ (⨆ i ∈ s, ⨆ l, ℂ • T i l) ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_lorentz_invariant_iSup_dualLeftRightWeyl_span [source]

The version of `mem_of_lorentz_invariant_biSup_dualLeftRightWeyl_span` joining over a whole finite index type.

```
lemma mem_of_lorentz_invariant_iSup_dualLeftRightWeyl_span {ι : Type} [Fintype ι]
    [DecidableEq ι] {T : ι → Fin 2 × Fin 2 → B}
    (hT : ∀ i, IsDualLeftRightWeyl B repLorentz
      (ofPairComponents (k := .downL) (k' := .downR) fun a b => T i (a, b))) (S : Submodule ℂ B
    )
    (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ (⨆ i, ⨆ l, ℂ • T i l) ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

inductive LeftIdx [source]

An index for the components of the five undotted fermion species: the generation together with the colour and isospin the species carries.

```
inductive LeftIdx
  | bard (f : Fin 3) (c : Fin 3)
  | baru (f : Fin 3) (c : Fin 3)
  | Q (f : Fin 3) (c : Fin 3) (s : Fin 2)
  | L (f : Fin 3) (s : Fin 2)
  | bare (f : Fin 3)
  deriving DecidableEq, Fintype
```

inductive RightIdx [source]

An index for the components of the five dotted fermion species: the generation together with the colour and isospin the species carries.

```
inductive RightIdx
  | d (f : Fin 3) (c : Fin 3)
  | u (f : Fin 3) (c : Fin 3)
  | barQ (f : Fin 3) (c : Fin 3) (s : Fin 2)
  | barL (f : Fin 3) (s : Fin 2)
  | e (f : Fin 3)
  deriving DecidableEq, Fintype
```

def leftComp [source]

The components of the five undotted species at the derivative slots `l`, presented as a two-element family in the spinor index.

```
noncomputable def leftComp (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) : LeftIdx → Fin 2 → B
```

```
  | .bard f c => fun a => h.bardComponent f l (a, c)
  | .baru f c => fun a => h.baruComponent f l (a, c)
  | .Q f c s => fun a => h.QComponent f l (a, c, s)
  | .L f s => fun a => h.LComponent f l (a, s)
  | .bare f => fun a => h.bareComponent f l a
```

def rightComp [source]

The components of the five dotted species at the derivative slots `l`, presented as a two-element family in the spinor index.

```
noncomputable def rightComp (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) : RightIdx → Fin 2 → B
  | .d f c => fun a => h.dComponent f l (a, c)
  | .u f c => fun a => h.uComponent f l (a, c)
  | .barQ f c s => fun a => h.barQComponent f l (a, c, s)
  | .barL f s => fun a => h.barLComponent f l (a, s)
  | .e f => fun a => h.eComponent f l a
```

lemma isDualLeftWeyl_leftComp [source]

Every undotted component family carries a dual undotted spinor index, at zero covariant derivatives.

```
lemma isDualLeftWeyl_leftComp (i : LeftIdx) :
    IsDualLeftWeyl repLorentz (h.leftComp ![] i)
```

lemma isDualRightWeyl_rightComp [source]

Every dotted component family carries a dual dotted spinor index, at zero covariant derivatives.

```
lemma isDualRightWeyl_rightComp (i : RightIdx) :
    IsDualRightWeyl repLorentz (h.rightComp ![] i)
```

def leftSpan [source]

The join of the lines through the components of the five undotted species.

```
noncomputable def leftSpan (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) : Submodule ℂ B
```

def rightSpan [source]

The join of the lines through the components of the five dotted species.

```
noncomputable def rightSpan (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly)
    {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) : Submodule ℂ B
```

lemma dualBasis_apply [source]

A basis vector of the dual basis is the matching coordinate functional.

```
lemma dualBasis_apply {ι M : Type} [AddCommGroup M] [Module ℂ M] [Fintype ι]
    [DecidableEq ι] (b : Module.Basis ι ℂ M) (j : ι) : b.dualBasis j = b.coord j
```

lemma range_bard_le_leftSpan [source]

The range of the `bard` symbols lies in the undotted span.

```
lemma range_bard_le_leftSpan (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (bard f l) ≤ h.leftSpan l
```

lemma range_baru_le_leftSpan [source]

The range of the `baru` symbols lies in the undotted span.

```
lemma range_baru_le_leftSpan (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (baru f l) ≤ h.leftSpan l
```

lemma range_Q_le_leftSpan [source]

The range of the `Q` symbols lies in the undotted span.

```
lemma range_Q_le_leftSpan (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (Q f l) ≤ h.leftSpan l
```

lemma range_L_le_leftSpan [source]

The range of the `L` symbols lies in the undotted span.

```
lemma range_L_le_leftSpan (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (L f l) ≤ h.leftSpan l
```

lemma range_bare_le_leftSpan [source]

The range of the `bare` symbols lies in the undotted span.

```
lemma range_bare_le_leftSpan (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (bare f l) ≤ h.leftSpan l
```

lemma range_d_le_rightSpan [source]

The range of the `d` symbols lies in the dotted span.

```
lemma range_d_le_rightSpan (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (d f l) ≤ h.rightSpan l
```

lemma range_u_le_rightSpan [source]

The range of the `u` symbols lies in the dotted span.

```
lemma range_u_le_rightSpan (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (u f l) ≤ h.rightSpan l
```

lemma range_barQ_le_rightSpan [source]

The range of the `barQ` symbols lies in the dotted span.

```
lemma range_barQ_le_rightSpan (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (barQ f l) ≤ h.rightSpan l
```

lemma range_barL_le_rightSpan [source]

The range of the `barL` symbols lies in the dotted span.

```
lemma range_barL_le_rightSpan (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (barL f l) ≤ h.rightSpan l
```

lemma range_e_le_rightSpan [source]

The range of the `e` symbols lies in the dotted span.

```
lemma range_e_le_rightSpan (f : Fin 3) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (e f l) ≤ h.rightSpan l
```

lemma mul_le_iSup_span_pair [source]

If `V` lies in the join of the lines through an undotted family and `W` in the join for a dotted one, the product lies in the join of the spans of the pair families.

```
lemma mul_le_iSup_span_pair {ι κ : Type} {X : ι → Fin 2 → B} {Y : κ → Fin 2 → B}
    {V W : Submodule ℂ B} (hV : V ≤ ⨆ (i : ι) (a : Fin 2), ℂ • X i a)
    (hW : W ≤ ⨆ (j : κ) (a : Fin 2), ℂ • Y j a) :
    V * W ≤ ⨆ (p : ι × κ) (l : Fin 2 × Fin 2), ℂ • (X p.1 l.1 * Y p.2 l.2)
```

lemma mul_le_iSup_span_pair_swap [source]

The mirror of `mul_le_iSup_span_pair` with the two spinor slots exchanged, for a product whose left factor is the dotted one.

```
lemma mul_le_iSup_span_pair_swap {ι κ : Type} {X : ι → Fin 2 → B} {Y : κ → Fin 2 → B}
    {V W : Submodule ℂ B} (hV : V ≤ ⨆ (i : ι) (a : Fin 2), ℂ • X i a)
    (hW : W ≤ ⨆ (j : κ) (a : Fin 2), ℂ • Y j a) :
    V * W ≤ ⨆ (p : ι × κ) (l : Fin 2 × Fin 2), ℂ • (X p.1 l.2 * Y p.2 l.1)
```

def sixFamily [source]

The mass-weight six families: a product of two underived components of opposite chirality, in either order.

```
noncomputable def sixFamily (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) :
    (LeftIdx × RightIdx) ⊕ (RightIdx × LeftIdx) → Fin 2 × Fin 2 → B
  | .inl p => fun l => h.leftComp ![] p.1 l.1 * h.rightComp ![] p.2 l.2
  | .inr p => fun l => h.rightComp ![] p.1 l.2 * h.leftComp ![] p.2 l.1
```

def sixSpan [source]

The join of the spans of the mass-weight six families.

```
noncomputable def sixSpan (h : IsFermionSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul d bard u baru Q barQ L barL e bare massWeightPoly) :
    Submodule ℂ B
```

lemma isDualLeftRightWeyl_sixFamily [source]

Each mass-weight six family carries one dual undotted and one dual dotted spinor index.

```
lemma isDualLeftRightWeyl_sixFamily
    (i : (LeftIdx × RightIdx) ⊕ (RightIdx × LeftIdx)) :
    IsDualLeftRightWeyl B repLorentz
      (ofPairComponents (k := .downL) (k' := .downR) fun a b => h.sixFamily i (a, b))
```

lemma mul_le_sixSpan_left [source]

An undotted range times a dotted one lies in the mass-weight six span.

```
lemma mul_le_sixSpan_left {V W : Submodule ℂ B} (hV : V ≤ h.leftSpan ![])
    (hW : W ≤ h.rightSpan ![]) : V * W ≤ h.sixSpan
```

lemma mul_le_sixSpan_right [source]

A dotted range times an undotted one lies in the mass-weight six span.

```
lemma mul_le_sixSpan_right {V W : Submodule ℂ B} (hV : V ≤ h.rightSpan ![])
    (hW : W ≤ h.leftSpan ![]) : V * W ≤ h.sixSpan
```

lemma massWeightSubmoduleGaugeWeightSix_piece_zero_le [source]

The weight-zero piece at mass weight six lies in the span of the mass-weight six families: each of the ten conjugate pairings is a product of an undotted range with a dotted one, in one order or the other.

```
lemma massWeightSubmoduleGaugeWeightSix_piece_zero_le :
    (h.massWeightSubmoduleGaugeWeightSix).piece 0 ≤ h.sixSpan
```

lemma mem_of_invariant_of_mem_massWeightSubmoduleSix_sup [source]

Mass weight six carries no invariant modulo a stable submodule: there is no Dirac mass term. A gauge- and Lorentz-invariant element of `massWeightSubmodule 6` ⊔ `S` lies in `S`, gauge invariance cutting the hundred pairings of two fermion symbols down to the ten conjugate ones and Lorentz invariance killing each of those, its two spinor indices being of opposite chirality.

```
lemma mem_of_invariant_of_mem_massWeightSubmoduleSix_sup {S : Submodule ℂ B}
    (hS : ∀ (g : GaugeGroupI) (y : B), y ∈ S → repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.massWeightSubmodule 6 ⊔ S)
    (hG : ∀ g : GaugeGroupI, repGauge g x = x)
    (hL : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_invariant_massWeightSubmodule_lt_eight_sup [source]

Between mass weight zero and mass weight eight the fermion sector carries no invariant: an element of `massWeightSubmodule w ⊔ S` for `0 < w < 8` fixed by both groups lies in `S`. Weights one, two and four are trivial, weights three, five and seven carry no gauge singlet, and weight six is the missing Dirac mass term.

```
lemma mem_of_invariant_massWeightSubmodule_lt_eight_sup (w : ℕ) (hw0 : 0 < w)
    (hw : w < 8) (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.massWeightSubmodule w ⊔ S)
    (hG : ∀ g : GaugeGroupI, repGauge g x = x)
    (hL : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff [source]

The classification below mass weight eight as an equivalence, in the shape of `mem_massWeightSubmodule_eigh` an element of `massWeightSubmodule w ⊔ S` for `0 < w < 8` is fixed by both groups exactly when it is itself an element of `S` fixed by both groups. The span of invariants that the weight eight statement leaves over is here the trivial one, so `x - y` lies in it exactly when `x = y`.

```
lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff (w : ℕ)
    (hw0 : 0 < w) (hw : w < 8) (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule w ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x = y
```

lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff_mem [source]

The same classification without the existential: below mass weight eight an element of `massWeightSubmodule w ⊔ S` fixed by both groups is an element of `S` fixed by both groups, and conversely.

```
lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff_mem (w : ℕ)
    (hw0 : 0 < w) (hw : w < 8) (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule w ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ (x ∈ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
          ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
```

2.470 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.Basic

PhyslibAlpha/Particles/StandardModel/IsGaugeSector/Basic.lean on GitHub.

structure IsGaugeSector [source]

The field strength and its covariant derivatives as a sector of the algebra `B`: gauge transformation through the adjoint action, the Lorentz transformation of the towers with two explicit covector indices, and the mass weights `2 * (2 + n)`.

```
structure IsGaugeSector (B : Type) [Ring B] [Algebra ℂ B]
    (repGauge : Representation ℂ GaugeGroupI B)
    (repGauge_mul : ∀ (g : GaugeGroupI) (b₁ b₂ : B),
      repGauge g (b₁ * b₂) = repGauge g b₁ * repGauge g b₂)
    (repLorentz : Representation ℂ SL(2,ℂ) B)
    (repLorentz_mul : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
      repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂)
    (F : {n : ℕ} → (Fin n → Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) →
      Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B)
    (massWeightPoly : B →ₐ[ℂ] Polynomial B) : Prop where
  repGauge_F : ∀ (g : GaugeGroupI) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
      (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ GaugeAlgebra),
    repGauge g (F l μ ν φ) = F l μ ν ((GaugeAlgebra.adjointMap g⁻¹).dualMap φ)
  repLorentz_F : ∀ (Λ : SL(2,ℂ)) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3))
      (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ GaugeAlgebra),
    repLorentz Λ (F l μ ν φ) =
      ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
        (∏ i, (((SL2C.toLorentzGroup Λ).1 (p i) (l i) : ℝ) : ℂ)) •
      ∑ a, (((SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
      ∑ b, (((SL2C.toLorentzGroup Λ).1 b ν : ℝ) : ℂ) • F p a b φ
  massWeight_F : ∀ {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3) φ,
    massWeightPoly (F l μ ν φ) = Polynomial.monomial (2 * (2 + n)) (F l μ ν φ)
  F_comm_F : ∀ {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
      (ψ : Module.Dual ℝ GaugeAlgebra) (l' : Fin m → Fin 1 ⊕ Fin 3)
      (μ' ν' : Fin 1 ⊕ Fin 3) (ψ' : Module.Dual ℝ GaugeAlgebra),
    Commute (F l μ ν ψ) (F l' μ' ν' ψ')
  F_antisymm : ∀ {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
      (φ : Module.Dual ℝ GaugeAlgebra),
    F l ν μ φ = - F l μ ν φ
```

def gaugeAlgebra [source]

The algebra generated by the field strength and all its covariant derivatives.

```
@[nolint unusedArguments]
def gaugeAlgebra (_h : IsGaugeSector B repGauge hrepGauge_mul repLorentz hrepLorentz_mul
    F massWeightPoly) : Subalgebra ℂ B
```

def gluonField [source]

The gluon contribution to the field strength (and its covariant derivatives): the field-strength symbol evaluated on the coordinate dual to the `a`-th Gell-Mann direction of the standard basis of the gauge algebra, i.e. the `su(3)` factor.

```
@[nolint unusedArguments]
noncomputable def gluonField (h : IsGaugeSector B repGauge hrepGauge_mul repLorentz
    hrepLorentz_mul
      F massWeightPoly) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (a : Fin 8) : B
```

def wField [source]

The `W`-boson contribution to the field strength (and its covariant derivatives): the field-strength symbol evaluated on the coordinate dual to the `i`-th Pauli direction of the standard basis of the gauge algebra, i.e. the `su(2)` factor.

```
@[nolint unusedArguments]
noncomputable def wField (h : IsGaugeSector B repGauge hrepGauge_mul repLorentz hrepLorentz_mul
      F massWeightPoly) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (i : Fin 3) : B
```

def hyperchargeField [source]

The hypercharge contribution to the field strength (and its covariant derivatives), i.e. the `B`-boson contribution: the field-strength symbol evaluated on the coordinate dual to the single basis direction of the `u(1)` factor of the standard basis of the gauge algebra.

```
@[nolint unusedArguments]
noncomputable def hyperchargeField (h : IsGaugeSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul F massWeightPoly) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) : B
```

def derivSubmodule [source]

The submodule of `B` generated by the field-strength symbols carrying exactly `n` covariant derivatives: the join, over the derivative slots and the two covector indices, of the ranges of the symbol maps.

```
@[nolint unusedArguments]
def derivSubmodule (h : IsGaugeSector B repGauge hrepGauge_mul repLorentz hrepLorentz_mul
    F massWeightPoly) (n : ℕ) : Submodule ℂ B
```

lemma commute_of_mem_derivSubmodule [source]

The gauge sector is bosonic: any element of a derivative-`n` submodule commutes with any element of a derivative-`m` submodule. This extends `F_comm_F` from generators to the submodules that they span.

```
lemma commute_of_mem_derivSubmodule {n m : ℕ} {x y : B}
    (hx : x ∈ h.derivSubmodule n) (hy : y ∈ h.derivSubmodule m) : Commute x y
```

lemma derivSubmodule_mul_comm [source]

The derivative-`n` and derivative-`m` submodules commute with one another as submodules of `B`, since every pair of their elements commute.

```
lemma derivSubmodule_mul_comm (n m : ℕ) :
    h.derivSubmodule n * h.derivSubmodule m = h.derivSubmodule m * h.derivSubmodule n
```

lemma derivSubmodule_map_repGauge_le [source]

The image of a derivative-`n` submodule under a gauge transformation lies inside the same submodule: each generator `F l μ ν φ` is sent by `repGauge_F` to another generator `F l μ ν φ'` with the same derivative slots and covector indices.

```
lemma derivSubmodule_map_repGauge_le (n : ℕ) (g : GaugeGroupI) :
    (h.derivSubmodule n).map (repGauge g) ≤ h.derivSubmodule n
```

lemma derivSubmodule_map_repGauge [source]

The derivative-`n` submodule is invariant, as a set, under the gauge group.

```
lemma derivSubmodule_map_repGauge (n : ℕ) (g : GaugeGroupI) :
    (h.derivSubmodule n).map (repGauge g) = h.derivSubmodule n
```

lemma derivSubmodule_map_repLorentz_le [source]

The image of a derivative-`n` submodule under a Lorentz transformation lies inside the same submodule: `repLorentz_F` expands each generator into a finite linear combination of generators with the same number `n` of derivative slots.

```
lemma derivSubmodule_map_repLorentz_le (n : ℕ) (Λ : SL(2,ℂ)) :
    (h.derivSubmodule n).map (repLorentz Λ) ≤ h.derivSubmodule n
```

lemma derivSubmodule_map_repLorentz [source]

The derivative-`n` submodule is invariant, as a set, under the Lorentz group.

```
lemma derivSubmodule_map_repLorentz (n : ℕ) (Λ : SL(2,ℂ)) :
    (h.derivSubmodule n).map (repLorentz Λ) = h.derivSubmodule n
```

2.471 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.Deriv.Centre

PhyslibAlpha/Particles/StandardModel/IsGaugeSector/Deriv/Centre.lean on GitHub.

lemma repLorentz_neg_one_F [source]

A field-strength symbol is fixed by the centre of `SL(2,ℂ)`: all of its indices mix by the Lorentz matrix, which is the identity there.

```
lemma repLorentz_neg_one_F {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) : repLorentz (-1) (F l μ ν φ) = F l μ ν φ
```

lemma span_range_F_le_centreEigenspace [source]

The span of a field-strength symbol family carries the sign `+1`.

```
lemma span_range_F_le_centreEigenspace {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) :
    Submodule.span ℂ (Set.range (F l μ ν)) ≤ centreEigenspace repLorentz 1
```

lemma derivSubmodule_le_centreEigenspace [source]

***The centre of `SL(2,ℂ)` acts on the field-strength derivative submodules by `+1`**, for any number of covariant derivatives: the field strength is of integer spin and every one of its indices is inert at the centre.*

```
lemma derivSubmodule_le_centreEigenspace (n : ℕ) :
    h.derivSubmodule n ≤ centreEigenspace repLorentz 1
```

2.472 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.Deriv.GaugeWeight

PhyslibAlpha/Particles/StandardModel/IsGaugeSector/Deriv/GaugeWeight.lean on GitHub.

lemma biUnion_univ_const [source]

A join over a nonempty index of a constant family of supports is that support.

```
lemma biUnion_univ_const {ι : Type*} [Fintype ι] [Nonempty ι] (t : Finset GaugeWeight) :
    (Finset.univ : Finset ι).biUnion (fun _ => t) = t
```

lemma real_smul_eq [source]

```
lemma real_smul_eq (r : ℝ) (b : B) : r • b = ((r : ℂ)) • b
```

lemma repGauge_pair_add [source]

```
lemma repGauge_pair_add (g : GaugeGroupI) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) (φ₁ φ₂ : Module.Dual ℝ GaugeAlgebra) (z : ℂ)
    (h1 : (GaugeAlgebra.adjointMap g⁻¹).dualMap φ₁ = z.re • φ₁ - z.im • φ₂)
    (h2 : (GaugeAlgebra.adjointMap g⁻¹).dualMap φ₂ = z.im • φ₁ + z.re • φ₂) :
    repGauge g (F l μ ν φ₁ + Complex.I • F l μ ν φ₂)
      = z • (F l μ ν φ₁ + Complex.I • F l μ ν φ₂)
```

lemma repGauge_pair_sub [source]

```
lemma repGauge_pair_sub (g : GaugeGroupI) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) (φ₁ φ₂ : Module.Dual ℝ GaugeAlgebra) (z : ℂ)
    (h1 : (GaugeAlgebra.adjointMap g⁻¹).dualMap φ₁ = z.re • φ₁ - z.im • φ₂)
    (h2 : (GaugeAlgebra.adjointMap g⁻¹).dualMap φ₂ = z.im • φ₁ + z.re • φ₂) :
    repGauge g (F l μ ν φ₁ - Complex.I • F l μ ν φ₂)
      = (starRingEnd ℂ z) • (F l μ ν φ₁ - Complex.I • F l μ ν φ₂)
```

lemma repGauge_fixed [source]

```
lemma repGauge_fixed (g : GaugeGroupI) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ GaugeAlgebra)
    (h1 : (GaugeAlgebra.adjointMap g⁻¹).dualMap φ = φ) :
    repGauge g (F l μ ν φ) = F l μ ν φ
```

def adjVec [source]

The weight vectors of the adjoint: for each root the two complex combinations of the paired coordinate symbols, and for each Cartan direction the symbol itself.

```
@[nolint unusedArguments]
noncomputable def adjVec (h : IsGaugeSector B repGauge hrepGauge_mul repLorentz
      hrepLorentz_mul F massWeightPoly) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) : Fin 4 ⊕ Fin 4 ⊕ Fin 4 → B
  | Sum.inl r => F l μ ν (stdBasis.coord (rootIdx r).1)
      + Complex.I • F l μ ν (stdBasis.coord (rootIdx r).2)
  | Sum.inr (Sum.inl r) => F l μ ν (stdBasis.coord (rootIdx r).1)
      - Complex.I • F l μ ν (stdBasis.coord (rootIdx r).2)
  | Sum.inr (Sum.inr c) => F l μ ν (stdBasis.coord (cartanIdx c))
```

def adjWeight [source]

The gauge weight of each adjoint weight vector.

```
def adjWeight : Fin 4 ⊕ Fin 4 ⊕ Fin 4 → GaugeWeight
  | Sum.inl r => GaugeAlgebra.rootWeight r
  | Sum.inr (Sum.inl r) => -(GaugeAlgebra.rootWeight r)
  | Sum.inr (Sum.inr _) => 0
```

lemma repGauge_adjVec [source]

Each adjoint weight vector is a simultaneous eigenvector of the gauge torus.

```
lemma repGauge_adjVec {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (k : Fin 4 ⊕ Fin 4 ⊕ Fin 4) (i : Fin 4) :
    repGauge (gaugeTorusGen i) (h.adjVec l μ ν k)
      = ((expI : ℂ) ^ GaugeWeight.coord (adjWeight k) i) • h.adjVec l μ ν k
```

lemma F_coord_rootIdx_fst [source]

The first symbol of a root pair, recovered from the two weight vectors.

```
lemma F_coord_rootIdx_fst {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (r : Fin 4) :
    F l μ ν (stdBasis.coord (rootIdx r).1)
      = (2 : ℂ)⁻¹ • (h.adjVec l μ ν (Sum.inl r)
        + h.adjVec l μ ν (Sum.inr (Sum.inl r)))
```

lemma F_coord_rootIdx_snd [source]

The second symbol of a root pair, recovered from the two weight vectors.

```
lemma F_coord_rootIdx_snd {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (r : Fin 4) :
    F l μ ν (stdBasis.coord (rootIdx r).2)
      = (-(Complex.I / 2)) • (h.adjVec l μ ν (Sum.inl r)
        - h.adjVec l μ ν (Sum.inr (Sum.inl r)))
```

lemma F_coord_mem_iSup [source]

Every standard coordinate symbol lies in the join of the weight-vector lines.

```
lemma F_coord_mem_iSup {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (a : Fin 8 ⊕ Fin 3 ⊕ Fin 1) :
    F l μ ν (stdBasis.coord a)
      ∈ ⨆ k, Submodule.span ℂ {h.adjVec l μ ν k}
```

lemma span_range_eq_iSup [source]

The span of the field-strength symbols is the join of the twelve weight lines.

```
lemma span_range_eq_iSup {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3) :
    Submodule.span ℂ (Set.range (F l μ ν))
      = ⨆ k, Submodule.span ℂ {h.adjVec l μ ν k}
```

def rangeGaugeWeight [source]

The gauge weight decomposition of the span of one field-strength symbol map.

```
@[implicit_reducible]
noncomputable def rangeGaugeWeight {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) :
    GaugeWeightDecomposition repGauge (Submodule.span ℂ (Set.range (F l μ ν)))
```

instance derivSubmoduleGaugeWeight [source]

***The gauge weight decomposition of the gauge derivative submodules**, for any number of covariant derivatives.*

```
@[implicit_reducible]
noncomputable instance derivSubmoduleGaugeWeight (n : ℕ) :
    GaugeWeightDecomposition repGauge (h.derivSubmodule n)
```

lemma rangeGaugeWeight_supp [source]

The support of the decomposition of one symbol map: the image of `adjWeight`.

```
lemma rangeGaugeWeight_supp {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeight l μ ν).supp
      = Finset.univ.biUnion fun k : Fin 4 ⊕ Fin 4 ⊕ Fin 4 =>
        ({adjWeight k} : Finset GaugeWeight)
```

lemma derivSubmoduleGaugeWeight_supp [source]

***The gauge weights occurring in the gauge derivative submodules**: the six `su(3)` roots, the two `su(2)` roots and the zero weight carried by the Cartan and `u(1)` directions. The weights do not depend on the number of covariant derivatives, and every one of them has vanishing hypercharge.*

```
lemma derivSubmoduleGaugeWeight_supp (n : ℕ) :
    (h.derivSubmoduleGaugeWeight n).supp
      = {((2, -1, 0, 0) : GaugeWeight), (1, 1, 0, 0), (-1, 2, 0, 0), (0, 0, 2, 0),
        (-2, 1, 0, 0), (-1, -1, 0, 0), (1, -2, 0, 0), (0, 0, -2, 0), (0, 0, 0, 0)}
```

lemma derivSubmoduleGaugeWeight_piece [source]

***The pieces of the gauge weight decomposition.** The weight-`w` piece is the join, over the derivative slots and the two covector indices, of the lines spanned by those weight vectors whose weight is `w`.*

```
lemma derivSubmoduleGaugeWeight_piece (n : ℕ) (w : GaugeWeight) :
    (h.derivSubmoduleGaugeWeight n).piece w
      = ⨆ (l : Fin n → Fin 1 ⊕ Fin 3) (μ : Fin 1 ⊕ Fin 3) (ν : Fin 1 ⊕ Fin 3)
        (k : Fin 4 ⊕ Fin 4 ⊕ Fin 4),
        (if w = adjWeight k then ℂ • h.adjVec l μ ν k else ⊥)
```

lemma derivSubmoduleGaugeWeight_piece_rootWeight [source]

The piece at a root weight: the `+` combination for that root alone.

```
lemma derivSubmoduleGaugeWeight_piece_rootWeight (n : ℕ) (r : Fin 4) :
    (h.derivSubmoduleGaugeWeight n).piece (GaugeAlgebra.rootWeight r)
      = ⨆ (l : Fin n → Fin 1 ⊕ Fin 3) (μ : Fin 1 ⊕ Fin 3) (ν : Fin 1 ⊕ Fin 3),
        ℂ • h.adjVec l μ ν (Sum.inl r)
```

lemma derivSubmoduleGaugeWeight_piece_neg_rootWeight [source]

The piece at the opposite of a root weight: the - combination for that root.

```
lemma derivSubmoduleGaugeWeight_piece_neg_rootWeight (n : ℕ) (r : Fin 4) :
    (h.derivSubmoduleGaugeWeight n).piece (-(GaugeAlgebra.rootWeight r))
      = ⨆ (l : Fin n → Fin 1 ⊕ Fin 3) (μ : Fin 1 ⊕ Fin 3) (ν : Fin 1 ⊕ Fin 3),
        ℂ • h.adjVec l μ ν (Sum.inr (Sum.inl r))
```

lemma derivSubmoduleGaugeWeight_piece_zero' [source]

The weight-zero piece: the two `su(3)` Cartan generators, the `su(2)` Cartan generator and the `u(1)` generator, the only directions the torus fixes.

```
lemma derivSubmoduleGaugeWeight_piece_zero' (n : ℕ) :
    (h.derivSubmoduleGaugeWeight n).piece 0
      = ⨆ (l : Fin n → Fin 1 ⊕ Fin 3) (μ : Fin 1 ⊕ Fin 3) (ν : Fin 1 ⊕ Fin 3)
          (c : Fin 4),
        ℂ • h.adjVec l μ ν (Sum.inr (Sum.inr c))
```

lemma derivSubmoduleGaugeWeight_piece_eq_bot [source]

Every other weight has a trivial piece.

```
lemma derivSubmoduleGaugeWeight_piece_eq_bot (n : ℕ) {w : GaugeWeight}
    (hw : w ∉ (h.derivSubmoduleGaugeWeight n).supp) :
    (h.derivSubmoduleGaugeWeight n).piece w = ⊥
```

2.473 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.MassWeight.Basic

PhyslibAlpha/Particles/StandardModel/IsGaugeSector/MassWeight/Basic.lean on GitHub.

def massWeightSubmodule [source]

All elements of the gauge algebra of mass weight exactly `w`: the intersection of the algebra generated by the field-strength towers with the part on which `massWeightPoly` is the monomial `X ^ w`.

```
noncomputable def massWeightSubmodule
  (h : IsGaugeSector B repGauge hrepGauge_mul repLorentz hrepLorentz_mul
      F massWeightPoly) (w : ℕ) :
    Submodule ℂ B
```

lemma massWeightPoly_of_mem_massWeightSubmodule [source]

```
lemma massWeightPoly_of_mem_massWeightSubmodule {w : ℕ} {x : B}
    (hx : x ∈ h.massWeightSubmodule w) :
    massWeightPoly x = Polynomial.monomial w x
```

lemma mem_gaugeAlgebra_of_mem_massWeightSubmodule [source]

```
lemma mem_gaugeAlgebra_of_mem_massWeightSubmodule {w : ℕ} {x : B}
    (hx : x ∈ h.massWeightSubmodule w) : x ∈ h.gaugeAlgebra
```

lemma one_le_massWeightSubmodule_zero [source]

```
lemma one_le_massWeightSubmodule_zero : (1 : Submodule ℂ B) ≤ h.massWeightSubmodule 0
```

lemma massWeightSubmodule_mul_le [source]

```
lemma massWeightSubmodule_mul_le (m n : ℕ) :
    h.massWeightSubmodule m * h.massWeightSubmodule n ≤ h.massWeightSubmodule (m + n)
```

lemma gaugeAlgebra_eq_adjoin_derivSubmodule [source]

The gauge algebra is generated by the field-strength towers of every derivative order.

```
lemma gaugeAlgebra_eq_adjoin_derivSubmodule :
    h.gaugeAlgebra = Algebra.adjoin ℂ (⋃ n, (h.derivSubmodule n : Set B))
```

lemma massWeightPoly_of_mem_derivSubmodule [source]

`massWeightPoly` is the monomial `X ^ (2 * (2 + n))` on the field-strength towers with `n` covariant derivatives.

```
lemma massWeightPoly_of_mem_derivSubmodule (n : ℕ) :
    ∀ x ∈ h.derivSubmodule n, massWeightPoly x = Polynomial.monomial (2 * (2 + n)) x
```

lemma derivSubmodule_le_massWeightSubmodule [source]

A field-strength tower with `n` covariant derivatives has mass weight `2 * (2 + n)`.

```
lemma derivSubmodule_le_massWeightSubmodule (n : ℕ) :
    h.derivSubmodule n ≤ h.massWeightSubmodule (2 * (2 + n))
```

lemma massWeightSubmodule_zero_eq [source]

Weight zero is the scalars: every field-strength tower has positive weight.

```
lemma massWeightSubmodule_zero_eq : h.massWeightSubmodule 0 = 1
```

lemma massWeightSubmodule_eq [source]

The weight recursion: an element of positive mass weight `i` is a sum of single field-strength towers of weight `i` and of products of two elements of lower positive weights summing to `i`.

```
lemma massWeightSubmodule_eq (i : ℕ) (hi : 0 < i) :
    h.massWeightSubmodule i
      = (⨆ k ∈ Finset.univ.filter (fun k : Fin i => 2 * (2 + (k : ℕ)) = i),
          h.derivSubmodule (k : ℕ))
        ⊔ (⨆ p ∈ Finset.univ.filter (fun p : Fin i × Fin i => (p.1 : ℕ) + (p.2 : ℕ) = i),
            h.massWeightSubmodule (p.1 : ℕ) * h.massWeightSubmodule (p.2 : ℕ))
```

lemma massWeightSubmodule_eq_iSup_mul [source]

Removing the leftmost field-strength tower: an element of positive weight `w` is a sum of products of a tower $\nabla^n F$ of weight `2 * (2 + n) ≤ w` with an element of the remaining weight.

```
lemma massWeightSubmodule_eq_iSup_mul (w : ℕ) (hw : 0 < w) :
    h.massWeightSubmodule w
      = ⨆ n ∈ (Finset.range (w + 1)).filter (fun n => 2 * (2 + n) ≤ w),
          h.derivSubmodule n * h.massWeightSubmodule (w - 2 * (2 + n))
```

lemma massWeightSubmodule_eq_bot [source]

The weights `1`, `2`, `3`, `5` and `7` are empty: the tower weights `4, 6, 8, …` and their sums only reach `0` and the even weights from `4` on.

```
lemma massWeightSubmodule_eq_bot {w : ℕ} (hw : ¬ (w = 0 ∨ (4 ≤ w ∧ w % 2 = 0))) :
    h.massWeightSubmodule w = ⊥
```

lemma massWeightSubmodule_one_eq [source]

There is nothing of weight one.

```
lemma massWeightSubmodule_one_eq : h.massWeightSubmodule 1 = ⊥
```

lemma massWeightSubmodule_two_eq [source]

There is nothing of weight two.

```
lemma massWeightSubmodule_two_eq : h.massWeightSubmodule 2 = ⊥
```

lemma massWeightSubmodule_three_eq [source]

There is nothing of weight three.

```
lemma massWeightSubmodule_three_eq : h.massWeightSubmodule 3 = ⊥
```

lemma massWeightSubmodule_four_eq [source]

Weight four is the underived field-strength towers: `F` leaves weight zero.

```
lemma massWeightSubmodule_four_eq :
    h.massWeightSubmodule 4 = h.derivSubmodule 0
```

lemma massWeightSubmodule_five_eq [source]

There is nothing of weight five.

```
lemma massWeightSubmodule_five_eq : h.massWeightSubmodule 5 = ⊥
```

lemma massWeightSubmodule_six_eq [source]

Weight six is the once-derived field-strength towers: `F` would leave weight two, which is empty, and `∇F` leaves weight zero.

```
lemma massWeightSubmodule_six_eq :
    h.massWeightSubmodule 6 = h.derivSubmodule 1
```

lemma massWeightSubmodule_seven_eq [source]

There is nothing of weight seven.

```
lemma massWeightSubmodule_seven_eq : h.massWeightSubmodule 7 = ⊥
```

lemma massWeightSubmodule_eight_eq [source]

Weight eight is the twice-derived field-strength towers together with the products of two underived ones, the ∇∇F and F · F sectors: F leaves weight four, which is F, ∇F would leave weight two, which is empty, and ∇∇F leaves weight zero.

```
lemma massWeightSubmodule_eight_eq :
    h.massWeightSubmodule 8 = h.derivSubmodule 2 ⊔ h.derivSubmodule 0 * h.derivSubmodule 0
```

2.474 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.MassWeight.GaugeWeight

PhyslibAlpha/Particles/StandardModel/IsGaugeSector/MassWeight/GaugeWeight.lean on GitHub.

def massWeightSubmoduleGaugeWeightOne [source]

Weight one is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightOne :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 1)
```

def massWeightSubmoduleGaugeWeightTwo [source]

Weight two is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightTwo :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 2)
```

def massWeightSubmoduleGaugeWeightThree [source]

Weight three is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightThree :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 3)
```

def massWeightSubmoduleGaugeWeightFour [source]

Weight four is the underived field strength.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightFour :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 4)
```

def massWeightSubmoduleGaugeWeightFive [source]

Weight five is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightFive :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 5)
```

def massWeightSubmoduleGaugeWeightSix [source]

Weight six is the once-derived field strength.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightSix :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 6)
```

def massWeightSubmoduleGaugeWeightSeven [source]

Weight seven is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightSeven :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 7)
```

def massWeightSubmoduleGaugeWeightEight [source]

Weight eight is the twice-derived field strength together with the products of two underived ones.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightEight :
    GaugeWeightDecomposition repGauge (h.massWeightSubmodule 8)
```

lemma rangeGaugeWeight_piece_zero [source]

The weight-zero piece of one symbol map's decomposition: the Cartan and `u(1)` directions, the only ones the torus fixes.

```
lemma rangeGaugeWeight_piece_zero {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) :
    (h.rangeGaugeWeight l μ ν).piece 0
      = ⨆ c : Fin 4, ℂ • F l μ ν (stdBasis.coord (cartanIdx c))
```

lemma derivSubmoduleGaugeWeight_piece_zero [source]

***The weight-zero piece of the gauge derivative submodules**: the spans of the field-strength symbols evaluated on the four weight-zero directions of the adjoint — the two `su(3)` Cartan generators, the `su(2)` Cartan generator and the `u(1)` generator. The four are distinct, so the join carries no duplicates.*

```
lemma derivSubmoduleGaugeWeight_piece_zero (n : ℕ) :
    (h.derivSubmoduleGaugeWeight n).piece 0
      = ⨆ (l : Fin n → Fin 1 ⊕ Fin 3) (μ : Fin 1 ⊕ Fin 3) (ν : Fin 1 ⊕ Fin 3)
          (c : Fin 4),
        ℂ • F l μ ν (stdBasis.coord (cartanIdx c))
```

lemma massWeightSubmoduleGaugeWeightOne_piece_zero [source]

The weight-zero piece at mass weight 1: the submodule itself is trivial.

```
lemma massWeightSubmoduleGaugeWeightOne_piece_zero :
    (h.massWeightSubmoduleGaugeWeightOne).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightTwo_piece_zero [source]

The weight-zero piece at mass weight 2: the submodule itself is trivial.

```
lemma massWeightSubmoduleGaugeWeightTwo_piece_zero :
    (h.massWeightSubmoduleGaugeWeightTwo).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightThree_piece_zero [source]

The weight-zero piece at mass weight 3: the submodule itself is trivial.

```
lemma massWeightSubmoduleGaugeWeightThree_piece_zero :
    (h.massWeightSubmoduleGaugeWeightThree).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightFive_piece_zero [source]

The weight-zero piece at mass weight 5: the submodule itself is trivial.

```
lemma massWeightSubmoduleGaugeWeightFive_piece_zero :
    (h.massWeightSubmoduleGaugeWeightFive).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightSeven_piece_zero [source]

The weight-zero piece at mass weight 7: the submodule itself is trivial.

```
lemma massWeightSubmoduleGaugeWeightSeven_piece_zero :
    (h.massWeightSubmoduleGaugeWeightSeven).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightFour_piece_zero [source]

The weight-zero piece at mass weight four: the undifferentiated field strength on the four fixed directions of the adjoint.

```
lemma massWeightSubmoduleGaugeWeightFour_piece_zero :
    (h.massWeightSubmoduleGaugeWeightFour).piece 0
      = ⨆ (μ : Fin 1 ⊕ Fin 3) (ν : Fin 1 ⊕ Fin 3) (c : Fin 4),
        ℂ • F ![] μ ν (stdBasis.coord (cartanIdx c))
```

lemma massWeightSubmoduleGaugeWeightSix_piece_zero [source]

The weight-zero piece at mass weight six: the once-differentiated field strength on the four fixed directions of the adjoint.

```
lemma massWeightSubmoduleGaugeWeightSix_piece_zero :
    (h.massWeightSubmoduleGaugeWeightSix).piece 0
      = ⨆ (l : Fin 1 → Fin 1 ⊕ Fin 3) (μ : Fin 1 ⊕ Fin 3) (ν : Fin 1 ⊕ Fin 3)
          (c : Fin 4),
        ℂ • F l μ ν (stdBasis.coord (cartanIdx c))
```

lemma piece_mul_comm [source]

Any two weight pieces of a gauge derivative submodule commute: the gauge sector is bosonic, and every piece sits inside the derivative submodule.

```
lemma piece_mul_comm (n : ℕ) (w w' : GaugeWeight) :
    (h.derivSubmoduleGaugeWeight n).piece w * (h.derivSubmoduleGaugeWeight n).piece w'
      = (h.derivSubmoduleGaugeWeight n).piece w' * (h.derivSubmoduleGaugeWeight n).piece w
```

def rootRaisingSpan [source]

The span of the underived raising vectors along the `r`-th root direction.

```
noncomputable def rootRaisingSpan (r : Fin 4) : Submodule ℂ B
```

def rootLoweringSpan [source]

The span of the underived lowering vectors along the `r`-th root direction.

```
noncomputable def rootLoweringSpan (r : Fin 4) : Submodule ℂ B
```

def cartanSpan [source]

The span of the underived weight-zero vectors: the two `su(3)` Cartan directions, the `su(2)` Cartan direction and the `u(1)` direction.

```
noncomputable def cartanSpan : Submodule ℂ B
```

def gluonRootPart [source]

The gluon contribution to the weight-zero piece: the three products pairing an `su(3)` raising vector against the matching lowering vector.

```
noncomputable def gluonRootPart : Submodule ℂ B
```

def isospinRootPart [source]

The isospin contribution to the weight-zero piece: the single product pairing the `su(2)` raising vector against the matching lowering vector.

```
noncomputable def isospinRootPart : Submodule ℂ B
```

def neutralCartanPart [source]

The neutral contribution to the weight-zero piece: the products of the weight-zero directions with themselves, namely the two `su(3)` Cartan directions, the `su(2)` Cartan direction and hypercharge.

```
noncomputable def neutralCartanPart : Submodule ℂ B
```

lemma massWeightSubmoduleGaugeWeightEight_piece_zero [source]

The weight-zero piece at mass weight eight, split into the contributions of the three gauge group factors: the twice-differentiated field strength on the four fixed directions of the adjoint, joined with the gluon, isospin and neutral parts.

```
lemma massWeightSubmoduleGaugeWeightEight_piece_zero :
    (h.massWeightSubmoduleGaugeWeightEight).piece 0
      = (⨆ (l : Fin 2 → Fin 1 ⊕ Fin 3) (μ : Fin 1 ⊕ Fin 3) (ν : Fin 1 ⊕ Fin 3)
          (c : Fin 4),
          ℂ • F l μ ν (stdBasis.coord (cartanIdx c)))
        ⊔ (h.gluonRootPart ⊔ (h.isospinRootPart ⊔ h.neutralCartanPart))
```

2.475 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.MassWeight.MassDimEight

PhyslibAlpha/Particles/StandardModel/IsGaugeSector/MassWeight/MassDimEight.lean on GitHub.

lemma iSup_span_singleton_le [source]

A span lies in a submodule as soon as its generators do.

```
lemma iSup_span_singleton_le {ι : Sort*} (T : ι → B) {V : Submodule ℂ B}
    (hV : ∀ i, T i ∈ V) : (⨆ i, ℂ • T i) ≤ V
```

lemma mem_iSup_span₃ [source]

A generator of a family with three indices lies in its span.

```
lemma mem_iSup_span₃ {α β γ : Sort*} (T : α → β → γ → B) (a : α) (b : β) (c : γ) :
    T a b c ∈ ⨆ (a) (b) (c), ℂ • T a b c
```

lemma span_stable_of_map_eq_sum [source]

A linear map moving each generator to a combination of the generators carries the span into itself; the transformation laws of this file all have this shape.

```
lemma span_stable_of_map_eq_sum {ι : Type} [Fintype ι] (T : ι → B) (f : B →ₗ[ℂ] B)
    {c : ι → ι → ℂ} (hf : ∀ l, f (T l) = ∑ a, c a l • T a) :
    ∀ y ∈ Submodule.span ℂ (Set.range T), f y ∈ Submodule.span ℂ (Set.range T)
```

lemma map_eq_self_of_mem_iSup_span [source]

A linear map fixing each generator fixes the span pointwise.

```
lemma map_eq_self_of_mem_iSup_span {ι : Sort*} (T : ι → B) (f : B →ₗ[ℂ] B)
    (hf : ∀ i, f (T i) = T i) : ∀ y ∈ ⨆ i, ℂ • T i, f y = y
```

lemma span_singleton_mul_span_singleton_le [source]

The product of two lines lies in a submodule as soon as the product of the two generators does.

```
lemma span_singleton_mul_span_singleton_le {a b : B} {V : Submodule ℂ B} (hab : a * b ∈ V) :
    (ℂ • a) * (ℂ • b) ≤ V
```

lemma iSup_span_mul_iSup_span_le [source]

The product of two spans lies in a submodule as soon as the products of their generators do.

```
lemma iSup_span_mul_iSup_span_le {α β γ α' β' γ' : Sort*} (T : α → β → γ → B)
    (T' : α' → β' → γ' → B) {V : Submodule ℂ B} (hV : ∀ a b c a' b' c', T a b c * T' a' b' c'
    ∈ V) :
    (⨆ (a) (b) (c), ℂ • T a b c) * (⨆ (a) (b) (c), ℂ • T' a b c) ≤ V
```

lemma sum_mul_sum_eq_sum_pi_two [source]

A product of two combinations is a combination indexed by pairs.

```
lemma sum_mul_sum_eq_sum_pi_two {k : ℕ} (c₀ c₁ : Fin k → ℂ) (X Y : Fin k → B) :
    (∑ a, c₀ a • X a) * ∑ b, c₁ b • Y b
      = ∑ d : Fin 2 → Fin k, (c₀ (d 0) * c₁ (d 1)) • (X (d 0) * Y (d 1))
```

lemma map_mul_fixed_eq_sum [source]

A multiplicative map moving a family by a matrix and fixing a vector moves the products of the family with that vector by the same matrix.

```
lemma map_mul_fixed_eq_sum {k : ℕ} {f : B →ₗ[ℂ] B} (hf : ∀ x y, f (x * y) = f x * f y)
    {c : Fin k → Fin k → ℂ} {T : Fin k → B} (hT : ∀ l, f (T l) = ∑ a, c a l • T a) {v : B}
    (hv : f v = v) (l : Fin k) : f (T l * v) = ∑ a, c a l • (T a * v)
```

lemma map_fixed_mul_eq_sum [source]

The mirror of `map_mul_fixed_eq_sum` with the fixed vector on the left.

```
lemma map_fixed_mul_eq_sum {k : ℕ} {f : B →ₗ[ℂ] B} (hf : ∀ x y, f (x * y) = f x * f y)
    {c : Fin k → Fin k → ℂ} {T : Fin k → B} (hT : ∀ l, f (T l) = ∑ a, c a l • T a) {v : B}
    (hv : f v = v) (l : Fin k) : f (v * T l) = ∑ a, c a l • (v * T a)
```

lemma repGauge_F_coord [source]

The field-strength symbol evaluated on a standard-basis coordinate transforms under the gauge group through the column of `adjointMatrix` indexed by that coordinate.

```
lemma repGauge_F_coord (g : GaugeGroupI) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) (c : Fin 8 ⊕ Fin 3 ⊕ Fin 1) :
    repGauge g (F l μ ν (GaugeAlgebra.stdBasis.coord c))
      = ∑ b, ((GaugeAlgebra.adjointMatrix g b c : ℝ) : ℂ) •
          F l μ ν (GaugeAlgebra.stdBasis.coord b)
```

lemma repGauge_gluonField [source]

The gluon field strength transforms in the adjoint representation of the `su(3)` factor of the gauge group.

```
lemma repGauge_gluonField (g : GaugeGroupI) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) (c : Fin 8) :
    repGauge g (h.gluonField l μ ν c)
      = ∑ a : Fin 8, ((su3AdjointMatrix (GaugeGroupI.toSU3 g) a c : ℝ) : ℂ) •
          h.gluonField l μ ν a
```

lemma repGauge_wField [source]

The `W`-boson field strength transforms in the adjoint representation of the `su(2)` factor of the gauge group.

```
lemma repGauge_wField (g : GaugeGroupI) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) (c : Fin 3) :
    repGauge g (h.wField l μ ν c)
      = ∑ i : Fin 3, ((su2AdjointMatrix (GaugeGroupI.toSU2 g) i c : ℝ) : ℂ) •
          h.wField l μ ν i
```

lemma repGauge_hyperchargeField [source]

The hypercharge field strength is gauge invariant: the adjoint action of the gauge group on the `u(1)` factor of the gauge algebra is trivial.

```
lemma repGauge_hyperchargeField (g : GaugeGroupI) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) :
    repGauge g (h.hyperchargeField l μ ν) = h.hyperchargeField l μ ν
```

lemma repGauge_su3_wField [source]

The `W`-boson field strengths are fixed by the colour factor of the gauge group: the `su(2)` block of the adjoint matrix reads the isospin factor alone.

```
lemma repGauge_su3_wField (U : specialUnitaryGroup (Fin 3) ℂ) {n : ℕ}
    (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3) (i : Fin 3) :
    repGauge (U, 1, 1) (h.wField l μ ν i) = h.wField l μ ν i
```

lemma isSU3Adjoint_gluonField [source]

The gluon field strengths at fixed derivative slots and covector indices form a family of one `su(3)` adjoint index.

```
lemma isSU3Adjoint_gluonField {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3) :
    IsSU3Adjoint B repGauge (h.gluonField l μ ν)
```

lemma isSU2Adjoint_wField [source]

The `W`-boson field strengths at fixed derivative slots and covector indices form a family of one `su(2)` adjoint index.

```
lemma isSU2Adjoint_wField {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3) :
    IsSU2Adjoint B repGauge (h.wField l μ ν)
```

abbrev EightIdx [source]

The index of a product of two underived field strengths: the two covector indices of the first factor followed by the two of the second, read as one family of four four-vector indices so that the Lorentz classification applies to it.

```
abbrev EightIdx : Type
```

def gluonPair [source]

The product of two underived gluon field strengths at the covector indices `p`, indexed by the two `su(3)` adjoint indices it carries.

```
noncomputable def gluonPair (p : EightIdx) (a : Fin 2 → Fin 8) : B
```

def wPair [source]

The product of two underived `W`-boson field strengths at the covector indices `p`, indexed by the two `su(2)` adjoint indices it carries.

```
noncomputable def wPair (p : EightIdx) (a : Fin 2 → Fin 3) : B
```

def hyperchargePair [source]

The product of two underived hypercharge field strengths at the covector indices `p`, indexed by the two `u(1)` adjoint indices it carries.

```
noncomputable def hyperchargePair (p : EightIdx) (_ : Fin 2 → Fin 1) : B
```

def hyperchargeDeriv [source]

The twice-derived hypercharge field strength at the derivative slots `d 0`, `d 1` and the covector indices `d 2`, `d 3`.

```
noncomputable def hyperchargeDeriv (d : EightIdx) : B
```

lemma isSU3BiAdjointMat_gluonPair [source]

A gauge transformation moves a gluon pair as the `SU(3)` factor of that gauge group element moves a tensor with two `su(3)` adjoint indices.

```
lemma isSU3BiAdjointMat_gluonPair (p : EightIdx) (g : GaugeGroupI) :
    IsSU3BiAdjointMat (GaugeGroupI.toSU3 g) (repGauge g) (h.gluonPair p)
```

lemma isSU3BiAdjoint_gluonPair [source]

A gluon pair is a bi-adjoint `su(3)` tensor.

```
lemma isSU3BiAdjoint_gluonPair (p : EightIdx) : IsSU3BiAdjoint B repGauge (h.gluonPair p)
```

lemma isSU2BiAdjointMat_wPair [source]

A gauge transformation moves a `W`-boson pair as the `SU(2)` factor of that gauge group element moves a tensor with two `su(2)` adjoint indices.

```
lemma isSU2BiAdjointMat_wPair (p : EightIdx) (g : GaugeGroupI) :
    IsSU2BiAdjointMat (GaugeGroupI.toSU2 g) (repGauge g) (h.wPair p)
```

lemma isSU2BiAdjoint_wPair [source]

A `W`-boson pair is a bi-adjoint `su(2)` tensor.

```
lemma isSU2BiAdjoint_wPair (p : EightIdx) : IsSU2BiAdjoint B repGauge (h.wPair p)
```

lemma isU1BiAdjointMat_hyperchargePair [source]

A gauge transformation fixes a hypercharge pair, which is the `u(1)` bi-adjoint law.

```
lemma isU1BiAdjointMat_hyperchargePair (p : EightIdx) (g : GaugeGroupI) :
    IsU1BiAdjointMat (GaugeGroupI.toU1 g) (repGauge g) (h.hyperchargePair p)
```

lemma isU1BiAdjoint_hyperchargePair [source]

A hypercharge pair is a bi-adjoint `u(1)` tensor.

```
lemma isU1BiAdjoint_hyperchargePair (p : EightIdx) :
    IsU1BiAdjoint B repGauge (h.hyperchargePair p)
```

def gluonTrace [source]

The gluon trace contraction at the covector indices `p`: the Kronecker contraction of the two colour indices of the gluon pair.

```
noncomputable def gluonTrace (p : EightIdx) : B
```

def wTrace [source]

The `W`-boson trace contraction at the covector indices `p`.

```
noncomputable def wTrace (p : EightIdx) : B
```

def hyperchargeTrace [source]

The hypercharge trace contraction at the covector indices `p`.

```
noncomputable def hyperchargeTrace (p : EightIdx) : B
```

lemma gluonTrace_eq [source]

The gluon trace contraction is the kinetic pairing of two gluon field strengths.

```
lemma gluonTrace_eq (p : EightIdx) :
    h.gluonTrace p
      = ∑ a : Fin 8, h.gluonField ![] (p 0) (p 1) a * h.gluonField ![] (p 2) (p 3) a
```

lemma wTrace_eq [source]

The `W`-boson trace contraction is the kinetic pairing of two `W`-boson field strengths.

```
lemma wTrace_eq (p : EightIdx) :
    h.wTrace p = ∑ i : Fin 3, h.wField ![] (p 0) (p 1) i * h.wField ![] (p 2) (p 3) i
```

lemma hyperchargeTrace_eq [source]

The hypercharge trace contraction is the product of the two hypercharge field strengths, the `u(1)` factor being one dimensional.

```
lemma hyperchargeTrace_eq (p : EightIdx) :
    h.hyperchargeTrace p
      = h.hyperchargeField ![] (p 0) (p 1) * h.hyperchargeField ![] (p 2) (p 3)
```

lemma repGauge_gluonTrace [source]

The gluon trace contraction is fixed by the whole gauge group.

```
lemma repGauge_gluonTrace (g : GaugeGroupI) (p : EightIdx) :
    repGauge g (h.gluonTrace p) = h.gluonTrace p
```

lemma repGauge_wTrace [source]

The `W`-boson trace contraction is fixed by the whole gauge group.

```
lemma repGauge_wTrace (g : GaugeGroupI) (p : EightIdx) :
    repGauge g (h.wTrace p) = h.wTrace p
```

lemma repGauge_hyperchargeTrace [source]

The hypercharge trace contraction is fixed by the whole gauge group.

```
lemma repGauge_hyperchargeTrace (g : GaugeGroupI) (p : EightIdx) :
    repGauge g (h.hyperchargeTrace p) = h.hyperchargeTrace p
```

lemma repGauge_hyperchargeDeriv [source]

The twice-derived hypercharge field strength is fixed by the whole gauge group.

```
lemma repGauge_hyperchargeDeriv (g : GaugeGroupI) (d : EightIdx) :
    repGauge g (h.hyperchargeDeriv d) = h.hyperchargeDeriv d
```

lemma F_mem_derivSubmodule [source]

Every field-strength symbol lies in the derivative submodule of its own number of covariant derivatives.

```
lemma F_mem_derivSubmodule {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) : F l μ ν φ ∈ h.derivSubmodule n
```

lemma F_mem_massWeightSubmodule [source]

A field-strength symbol with `n` covariant derivatives has mass weight `2 * (2 + n)`.

```
lemma F_mem_massWeightSubmodule {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) : F l μ ν φ ∈ h.massWeightSubmodule (2 * (2 + n))
```

lemma F_mul_F_mem_massWeightSubmodule_eight [source]

A product of two underived field-strength symbols has mass weight eight.

```
lemma F_mul_F_mem_massWeightSubmodule_eight (μ ν μ' ν' : Fin 1 ⊕ Fin 3)
    (φ φ' : Module.Dual ℝ GaugeAlgebra) :
    F ![] μ ν φ * F ![] μ' ν' φ' ∈ h.massWeightSubmodule 8
```

lemma gluonTrace_mem_massWeightSubmodule [source]

The gluon trace contraction has mass weight eight.

```
lemma gluonTrace_mem_massWeightSubmodule (p : EightIdx) :
    h.gluonTrace p ∈ h.massWeightSubmodule 8
```

lemma wTrace_mem_massWeightSubmodule [source]

The `W`-boson trace contraction has mass weight eight.

```
lemma wTrace_mem_massWeightSubmodule (p : EightIdx) : h.wTrace p ∈ h.massWeightSubmodule 8
```

lemma hyperchargeTrace_mem_massWeightSubmodule [source]

The hypercharge trace contraction has mass weight eight.

```
lemma hyperchargeTrace_mem_massWeightSubmodule (p : EightIdx) :
    h.hyperchargeTrace p ∈ h.massWeightSubmodule 8
```

lemma hyperchargeDeriv_mem_massWeightSubmodule [source]

The twice-derived hypercharge field strength has mass weight `2 * (2 + 2)`, eight.

```
lemma hyperchargeDeriv_mem_massWeightSubmodule (d : EightIdx) :
    h.hyperchargeDeriv d ∈ h.massWeightSubmodule 8
```

def su3AdjIdx [source]

The `su(3)` adjoint weight indices read as weight indices of the whole gauge algebra: the three colour roots and the two colour Cartan directions.

```
def su3AdjIdx : GaugeAlgebra.su3WeightIdx → Fin 4 ⊕ Fin 4 ⊕ Fin 4
  | Sum.inl r => Sum.inl r.castSucc
  | Sum.inr (Sum.inl r) => Sum.inr (Sum.inl r.castSucc)
  | Sum.inr (Sum.inr c) => Sum.inr (Sum.inr c.castSucc.castSucc)
```

def su2AdjIdx [source]

The `su(2)` adjoint weight indices read as weight indices of the whole gauge algebra: the isospin root and the isospin Cartan direction.

```
def su2AdjIdx : GaugeAlgebra.su2WeightIdx → Fin 4 ⊕ Fin 4 ⊕ Fin 4
  | Sum.inl _ => Sum.inl 3
  | Sum.inr (Sum.inl _) => Sum.inr (Sum.inl 3)
  | Sum.inr (Sum.inr _) => Sum.inr (Sum.inr 2)
```

lemma sum_su3WeightCoeff_smul_gluonField [source]

A weight vector of the colour part of the adjoint is the matching combination of gluon field strengths.

```
lemma sum_su3WeightCoeff_smul_gluonField {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) (k : GaugeAlgebra.su3WeightIdx) :
    ∑ a : Fin 8, GaugeAlgebra.su3WeightCoeff k a • h.gluonField l μ ν a
      = h.adjVec l μ ν (su3AdjIdx k)
```

lemma sum_su2WeightCoeff_smul_wField [source]

A weight vector of the isospin part of the adjoint is the matching combination of W-boson field strengths.

```
lemma sum_su2WeightCoeff_smul_wField {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (μ ν : Fin 1 ⊕ Fin 3) (k : GaugeAlgebra.su2WeightIdx) :
    ∑ i : Fin 3, GaugeAlgebra.su2WeightCoeff k i • h.wField l μ ν i
      = h.adjVec l μ ν (su2AdjIdx k)
```

lemma adjVec_mul_adjVec_mem_gluonPair_span [source]

A product of two colour weight vectors of the adjoint lies in the span of the matching gluon pair family.

```
lemma adjVec_mul_adjVec_mem_gluonPair_span (p : EightIdx)
    (k₀ k₁ : GaugeAlgebra.su3WeightIdx) :
    h.adjVec ![] (p 0) (p 1) (su3AdjIdx k₀) * h.adjVec ![] (p 2) (p 3) (su3AdjIdx k₁)
      ∈ Submodule.span ℂ (Set.range (h.gluonPair p))
```

lemma adjVec_mul_adjVec_mem_wPair_span [source]

A product of two isospin weight vectors of the adjoint lies in the span of the matching W-boson pair family.

```
lemma adjVec_mul_adjVec_mem_wPair_span (p : EightIdx) (k₀ k₁ : GaugeAlgebra.su2WeightIdx) :
    h.adjVec ![] (p 0) (p 1) (su2AdjIdx k₀) * h.adjVec ![] (p 2) (p 3) (su2AdjIdx k₁)
      ∈ Submodule.span ℂ (Set.range (h.wPair p))
```

def gluonPairSpan [source]

The join, over all covector indices, of the spans of the gluon pair families.

```
noncomputable def gluonPairSpan : Submodule ℂ B
```

def wPairSpan [source]

The join, over all covector indices, of the spans of the W-boson pair families.

```
noncomputable def wPairSpan : Submodule ℂ B
```

lemma gluonRootPart_le_gluonPairSpan [source]

The gluon root part of the zero-weight piece, the three products of a colour raising vector against the matching lowering vector, lies in the gluon pair spans.

```
lemma gluonRootPart_le_gluonPairSpan : h.gluonRootPart ≤ h.gluonPairSpan
```

lemma isospinRootPart_le_wPairSpan [source]

The isospin root part of the zero-weight piece, the product of the isospin raising vector against the lowering vector, lies in the W-boson pair spans.

```
lemma isospinRootPart_le_wPairSpan : h.isospinRootPart ≤ h.wPairSpan
```

def hyperchargeTraceSpan [source]

The span of the hypercharge trace contractions.

```
noncomputable def hyperchargeTraceSpan : Submodule ℂ B
```

def hyperchargeDerivSpan [source]

The span of the twice-derived hypercharge field strengths.

```
noncomputable def hyperchargeDerivSpan : Submodule ℂ B
```

def neutralVec [source]

The two neutral underived directions that pair with a colour index in the mixed neutral products: the isospin Cartan direction and hypercharge.

```
noncomputable def neutralVec (μ ν : Fin 1 ⊕ Fin 3) : Fin 2 → B
  | 0 => h.wField ![] μ ν GaugeAlgebra.su2CartanId
  | 1 => h.hyperchargeField ![] μ ν
```

lemma repGauge_su3_neutralVec [source]

The neutral directions are fixed by the colour factor of the gauge group.

```
lemma repGauge_su3_neutralVec (U : specialUnitaryGroup (Fin 3) ℂ) (μ ν : Fin 1 ⊕ Fin 3)
    (j : Fin 2) : repGauge (U, 1, 1) (h.neutralVec μ ν j) = h.neutralVec μ ν j
```

abbrev DerivIdx [source]

The index of a twice-derived symbol: the two derivative slots and the two covector indices.

```
abbrev DerivIdx : Type
```

abbrev MixIdx [source]

The index of a mixed neutral product: the two covector indices of the colour factor, the two of the neutral factor, and which of the two neutral directions it is.

```
abbrev MixIdx : Type
```

abbrev ColourIdx [source]

The index of a family carrying one unpaired `su(3)` adjoint index at mass weight eight: a twice-derived gluon tower, or an underived gluon field strength against a neutral underived factor on either side.

```
abbrev ColourIdx : Type
```

def colourFamily [source]

The families carrying one unpaired `su(3)` adjoint index.

```
noncomputable def colourFamily : ColourIdx → Fin 8 → B
  | Sum.inl p => h.gluonField p.1 p.2.1 p.2.2
  | Sum.inr (Sum.inl q) =>
      fun a => h.gluonField ![] q.1 q.2.1 a * h.neutralVec q.2.2.1 q.2.2.2.1 q.2.2.2.2
  | Sum.inr (Sum.inr q) =>
      fun a => h.neutralVec q.2.2.1 q.2.2.2.1 q.2.2.2.2 * h.gluonField ![] q.1 q.2.1 a
```

lemma isSU3Adjoint_colourFamily [source]

Each colour family is an `su(3)` adjoint family: the colour factor moves the gluon index and fixes the neutral factor.

```
lemma isSU3Adjoint_colourFamily : ∀ i : ColourIdx, IsSU3Adjoint B repGauge (h.colourFamily i)
  | Sum.inl p => h.isSU3Adjoint_gluonField p.1 p.2.1 p.2.2
  | Sum.inr (Sum.inl q) => IsSU3Adjoint.of_law fun U => map_mul_fixed_eq_sum (hrepGauge_mul _)
      (h.repGauge_gluonField (U, 1, 1) ![] q.1 q.2.1) (h.repGauge_su3_neutralVec U _ _ _)
  | Sum.inr (Sum.inr q) => IsSU3Adjoint.of_law fun U => map_fixed_mul_eq_sum (hrepGauge_mul _)
      (h.repGauge_gluonField (U, 1, 1) ![] q.1 q.2.1) (h.repGauge_su3_neutralVec U _ _ _)
```

abbrev IsospinIdx [source]

The index of a family carrying one unpaired `su(2)` adjoint index at mass weight eight: a twice-derived `W`-boson tower, or an underived `W`-boson field strength against an underived hypercharge field strength on either side.

```
abbrev IsospinIdx : Type
```

def isospinFamily [source]

The families carrying one unpaired `su(2)` adjoint index.

```
noncomputable def isospinFamily : IsospinIdx → Fin 3 → B
  | Sum.inl p => h.wField p.1 p.2.1 p.2.2
  | Sum.inr (Sum.inl q) => fun i => h.wField ![] (q 0) (q 1) i * h.hyperchargeField ![] (q 2) (
    q 3)
  | Sum.inr (Sum.inr q) => fun i => h.hyperchargeField ![] (q 2) (q 3) * h.wField ![] (q 0) (q
    1) i
```

lemma isSU2Adjoint_isospinFamily [source]

Each isospin family is an `su(2)` adjoint family: the isospin factor moves the `W`-boson index and fixes hypercharge.

```
lemma isSU2Adjoint_isospinFamily : ∀ i : IsospinIdx, IsSU2Adjoint B repGauge (h.isospinFamily i
    )
  | Sum.inl p => h.isSU2Adjoint_wField p.1 p.2.1 p.2.2
  | Sum.inr (Sum.inl q) => IsSU2Adjoint.of_law fun U => map_mul_fixed_eq_sum (hrepGauge_mul _)
      (h.repGauge_wField (1, U, 1) ![] (q 0) (q 1)) (h.repGauge_hyperchargeField _ _ _ _)
  | Sum.inr (Sum.inr q) => IsSU2Adjoint.of_law fun U => map_fixed_mul_eq_sum (hrepGauge_mul _)
      (h.repGauge_wField (1, U, 1) ![] (q 0) (q 1)) (h.repGauge_hyperchargeField _ _ _ _)
```

lemma repGauge_su3_isospinFamily [source]

The isospin families are fixed by the colour factor, every one of their factors being.

```
lemma repGauge_su3_isospinFamily (U : specialUnitaryGroup (Fin 3) ℂ) :
    ∀ (i : IsospinIdx) (a : Fin 3),
      repGauge (U, 1, 1) (h.isospinFamily i a) = h.isospinFamily i a
  | Sum.inl p, a => h.repGauge_su3_wField U p.1 p.2.1 p.2.2 a
  | Sum.inr (Sum.inl q), a => by
      simp only [isospinFamily, hrepGauge_mul, h.repGauge_su3_wField, h.
    repGauge_hyperchargeField]
  | Sum.inr (Sum.inr q), a => by
      simp only [isospinFamily, hrepGauge_mul, h.repGauge_su3_wField, h.
    repGauge_hyperchargeField]
```

def unpairedColourSpan [source]

The join of the spans of the colour families.

```
noncomputable def unpairedColourSpan : Submodule ℂ B
```

def unpairedIsospinSpan [source]

The join of the spans of the isospin families.

```
noncomputable def unpairedIsospinSpan : Submodule ℂ B
```

lemma colourFamily_mem [source]

A component of a colour family lies in the join of the colour spans.

```
lemma colourFamily_mem (i : ColourIdx) (a : Fin 8) : h.colourFamily i a ∈ h.unpairedColourSpan
```

lemma isospinFamily_mem [source]

A component of an isospin family lies in the join of the isospin spans.

```
lemma isospinFamily_mem (i : IsospinIdx) (a : Fin 3) :
    h.isospinFamily i a ∈ h.unpairedIsospinSpan
```

lemma repGauge_su3_of_mem_unpairedIsospinSpan [source]

The join of the isospin families is fixed pointwise by the colour factor.

```
lemma repGauge_su3_of_mem_unpairedIsospinSpan (U : specialUnitaryGroup (Fin 3) ℂ) :
    ∀ y ∈ h.unpairedIsospinSpan, repGauge (U, 1, 1) y = y
```

def colourCartanSpan [source]

The colour Cartan directions of the underived tower: the two weight-zero directions of the `su(3)` factor.

```
noncomputable def colourCartanSpan : Submodule ℂ B
```

def neutralSpan [source]

The neutral directions of the underived tower: the isospin Cartan direction and hypercharge.

```
noncomputable def neutralSpan : Submodule ℂ B
```

lemma adjVec_colourCartan [source]

A colour Cartan weight vector is the gluon field strength on the matching Cartan direction of `su(3)`.

```
lemma adjVec_colourCartan {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (c : Fin 2) :
    h.adjVec l μ ν (Sum.inr (Sum.inr c.castSucc.castSucc))
      = h.gluonField l μ ν (GaugeAlgebra.su3CartanId c)
```

lemma cartanSpan_le [source]

The weight-zero directions of the adjoint are the colour Cartan directions and the neutral directions.

```
lemma cartanSpan_le : h.cartanSpan ≤ h.colourCartanSpan ⊔ h.neutralSpan
```

lemma colourCartanSpan_mul_colourCartanSpan_le [source]

A product of two colour Cartan directions is a component of a gluon pair family.

```
lemma colourCartanSpan_mul_colourCartanSpan_le :
    h.colourCartanSpan * h.colourCartanSpan ≤ h.gluonPairSpan
```

lemma colourCartanSpan_mul_neutralSpan_le [source]

A colour Cartan direction against a neutral direction is a component of a colour family.

```
lemma colourCartanSpan_mul_neutralSpan_le :
    h.colourCartanSpan * h.neutralSpan ≤ h.unpairedColourSpan
```

lemma neutralSpan_mul_colourCartanSpan_le [source]

A neutral direction against a colour Cartan direction is a component of a colour family.

```
lemma neutralSpan_mul_colourCartanSpan_le :
    h.neutralSpan * h.colourCartanSpan ≤ h.unpairedColourSpan
```

lemma neutralSpan_mul_neutralSpan_le [source]

A product of two neutral directions: isospin against isospin is a component of a W-boson pair family, hypercharge against hypercharge is a hypercharge trace contraction, and the two mixed products are components of isospin families.

```
lemma neutralSpan_mul_neutralSpan_le :
    h.neutralSpan * h.neutralSpan
      ≤ h.unpairedIsospinSpan ⊔ (h.wPairSpan ⊔ h.hyperchargeTraceSpan)
```

lemma neutralCartanPart_le [source]

The neutral part of the zero-weight piece: the products pairing a factor with itself are components of the pair families or hypercharge trace contractions, and the mixed products carry an unpaired non-abelian index.

```
lemma neutralCartanPart_le :
    h.neutralCartanPart ≤ (h.unpairedColourSpan ⊔ h.unpairedIsospinSpan)
      ⊔ (h.gluonPairSpan ⊔ (h.wPairSpan ⊔ h.hyperchargeTraceSpan))
```

lemma derivCartanSpan_le [source]

The twice-derived symbols on the weight-zero directions: on a colour or isospin Cartan direction a component of an adjoint family, on hypercharge a twice-derived hypercharge field strength.

```
lemma derivCartanSpan_le :
    (⨆ (l : Fin 2 → Fin 1 ⊕ Fin 3) (μ : Fin 1 ⊕ Fin 3) (ν : Fin 1 ⊕ Fin 3) (c : Fin 4),
        ℂ • F l μ ν (GaugeAlgebra.stdBasis.coord (GaugeAlgebra.cartanIdx c)))
      ≤ (h.unpairedColourSpan ⊔ h.unpairedIsospinSpan) ⊔ h.hyperchargeDerivSpan
```

lemma massWeightSubmoduleGaugeWeightEight_piece_zero_le [source]

The zero-weight piece of mass weight eight is bounded by the joins of the unpaired families together with the pair spans, the hypercharge trace contractions and the twice-derived hypercharge field strengths.

```
lemma massWeightSubmoduleGaugeWeightEight_piece_zero_le :
    (h.massWeightSubmoduleGaugeWeightEight).piece 0
      ≤ (h.unpairedColourSpan ⊔ h.unpairedIsospinSpan)
        ⊔ (h.gluonPairSpan
          ⊔ (h.wPairSpan ⊔ (h.hyperchargeTraceSpan ⊔ h.hyperchargeDerivSpan)))
```

lemma reducesInvariantsTo_unpaired [source]

The joins of the unpaired families reduce to ⊥ for the gauge group: the colour families for the colour factor, with the colour-fixed isospin join kept in the target, and then the isospin families for the isospin factor.

```
lemma reducesInvariantsTo_unpaired :
    ReducesInvariantsTo (fun g : GaugeGroupI => repGauge g)
      (h.unpairedColourSpan ⊔ h.unpairedIsospinSpan) ⊥
```

def gluonTraceSpan [source]

The span of the gluon trace contractions.

```
noncomputable def gluonTraceSpan : Submodule ℂ B
```

def wTraceSpan [source]

The span of the W-boson trace contractions.

```
noncomputable def wTraceSpan : Submodule ℂ B
```

def traceContractionEightSpan [source]

The span of the three underived trace contractions, over all covector indices: the gauge invariants of mass weight eight that the bi-adjoint classification produces.

```
noncomputable def traceContractionEightSpan : Submodule ℂ B
```

lemma isStableUnder_gluonPairSpan [source]

The gluon pair spans are stable under the gauge group.

```
lemma isStableUnder_gluonPairSpan :
    IsStableUnder (fun g : GaugeGroupI => repGauge g) h.gluonPairSpan
```

lemma isStableUnder_wPairSpan [source]

The W-boson pair spans are stable under the gauge group.

```
lemma isStableUnder_wPairSpan :
    IsStableUnder (fun g : GaugeGroupI => repGauge g) h.wPairSpan
```

lemma repGauge_of_mem_hyperchargeTraceSpan [source]

The hypercharge trace contractions are fixed pointwise by the gauge group.

```
lemma repGauge_of_mem_hyperchargeTraceSpan (g : GaugeGroupI) :
    ∀ y ∈ h.hyperchargeTraceSpan, repGauge g y = y
```

lemma repGauge_of_mem_hyperchargeDerivSpan [source]

The twice-derived hypercharge field strengths are fixed pointwise by the gauge group.

```
lemma repGauge_of_mem_hyperchargeDerivSpan (g : GaugeGroupI) :
    ∀ y ∈ h.hyperchargeDerivSpan, repGauge g y = y
```

lemma traceContractionEightSpan_sup_hyperchargeDerivSpan_le [source]

The gauge span is a space of gauge invariants of mass weight eight: each generator is fixed by the gauge group and has mass weight eight by section C. This is the converse of the classification.

```
lemma traceContractionEightSpan_sup_hyperchargeDerivSpan_le :
    h.traceContractionEightSpan ⊔ h.hyperchargeDerivSpan
      ≤ h.massWeightSubmodule 8 ⊓ repGauge.invariants
```

lemma isFixedBy_traceContractionEightSpan_sup [source]

The gauge span is fixed pointwise by the gauge group.

```
lemma isFixedBy_traceContractionEightSpan_sup :
    IsFixedBy (fun g : GaugeGroupI => repGauge g)
      (h.traceContractionEightSpan ⊔ h.hyperchargeDerivSpan)
```

lemma reducesInvariantsTo_gluonPairSpan [source]

The gluon pair spans reduce, for the gauge group, to the gluon trace contractions: each gluon pair family is bi-adjoint for the colour factor.

```
lemma reducesInvariantsTo_gluonPairSpan :
    ReducesInvariantsTo (fun g : GaugeGroupI => repGauge g) h.gluonPairSpan h.gluonTraceSpan
```

lemma reducesInvariantsTo_wPairSpan [source]

The `W`-boson pair spans reduce, for the gauge group, to the `W`-boson trace contractions: each `W`-boson pair family is bi-adjoint for the isospin factor.

```
lemma reducesInvariantsTo_wPairSpan :
    ReducesInvariantsTo (fun g : GaugeGroupI => repGauge g) h.wPairSpan h.wTraceSpan
```

lemma reducesInvariantsTo_traceContractionEightSpan_sup [source]

Mass weight eight reduces, for the gauge group, to the three underived trace contractions and the twice-derived hypercharge field strengths. The torus puts a gauge invariant in the zero-weight piece; of the parts bounding it, the unpaired joins reduce to $\bot$, the pair spans to their trace contractions, and the hypercharge spans are already fixed.

```
lemma reducesInvariantsTo_traceContractionEightSpan_sup :
    ReducesInvariantsTo (fun g : GaugeGroupI => repGauge g) (h.massWeightSubmodule 8)
      (h.traceContractionEightSpan ⊔ h.hyperchargeDerivSpan)
```

lemma repLorentz_F_underived [source]

The Lorentz transformation of an underived field-strength symbol: the general law of `IsGaugeSector` at no covariant derivatives, where the sum over the derivative slots is a single term, written with the two covector rotations gathered into one coefficient.

```
lemma repLorentz_F_underived (Λ : SL(2,ℂ)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) :
    repLorentz Λ (F ![] μ ν φ)
      = ∑ a : Fin 1 ⊕ Fin 3, ∑ b : Fin 1 ⊕ Fin 3,
        ((((SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) *
          (((SL2C.toLorentzGroup Λ).1 b ν : ℝ) : ℂ)) • F ![] a b φ
```

lemma sum_mul_sum_eq_sum_pi_four [source]

A product of two double combinations is a combination indexed by quadruples.

```
lemma sum_mul_sum_eq_sum_pi_four (c c' : (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → ℂ)
    (X Y : (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → B) :
    (∑ a, ∑ b, c a b • X a b) * (∑ x, ∑ y, c' x y • Y x y)
      = ∑ d : Fin 4 → Fin 1 ⊕ Fin 3,
        (c (d 0) (d 1) * c' (d 2) (d 3)) • (X (d 0) (d 1) * Y (d 2) (d 3))
```

lemma isLorentzCovariant_F_mul [source]

A product of two underived field-strength symbols, viewed as a family indexed by the four covector indices it carries, is a quadruple Lorentz tensor.

```
lemma isLorentzCovariant_F_mul (φ ψ : Module.Dual ℝ GaugeAlgebra) :
    IsLorentzCovariant 4 B repLorentz (ofComponents
      fun d : Fin 4 → Fin 1 ⊕ Fin 3 => F ![] (d 0) (d 1) φ * F ![] (d 2) (d 3) ψ)
```

lemma isLorentzCovariant_sum [source]

A finite sum of quadruple Lorentz tensors is a quadruple Lorentz tensor: the transformation law is linear in the family.

```
lemma isLorentzCovariant_sum {ι : Type} [Fintype ι] {T : ι → (Fin 4 → Fin 1 ⊕ Fin 3) → B}
    (hT : ∀ i, IsLorentzCovariant 4 B repLorentz (ofComponents (T i))) :
    IsLorentzCovariant 4 B repLorentz (ofComponents fun d => ∑ i, T i d)
```

lemma isLorentzCovariant_of_eq_sum [source]

A family of four four-vector indices whose members are sums of products of two underived field-strength symbols is a quadruple Lorentz tensor.

```
lemma isLorentzCovariant_of_eq_sum {ι : Type} [Fintype ι] {T : EightIdx → B}
    (φ : ι → Module.Dual ℝ GaugeAlgebra)
    (hT : ∀ d, T d = ∑ i, F ![] (d 0) (d 1) (φ i) * F ![] (d 2) (d 3) (φ i)) :
    IsLorentzCovariant 4 B repLorentz (ofComponents T)
```

lemma isLorentzCovariant_gluonTrace [source]

The gluon trace contractions, read as a family of four four-vector indices, form a quadruple Lorentz tensor: a sum over the colour index of products of two underived field-strength symbols.

```
lemma isLorentzCovariant_gluonTrace :
    IsLorentzCovariant 4 B repLorentz (ofComponents h.gluonTrace)
```

lemma isLorentzCovariant_wTrace [source]

The `W`-boson trace contractions form a quadruple Lorentz tensor.

```
lemma isLorentzCovariant_wTrace : IsLorentzCovariant 4 B repLorentz (ofComponents h.wTrace)
```

lemma isLorentzCovariant_hyperchargeTrace [source]

The hypercharge trace contractions form a quadruple Lorentz tensor.

```
lemma isLorentzCovariant_hyperchargeTrace :
    IsLorentzCovariant 4 B repLorentz (ofComponents h.hyperchargeTrace)
```

lemma isLorentzCovariant_hyperchargeDeriv [source]

The twice-derived hypercharge field strengths, read as a family of four four-vector indices, form a quadruple Lorentz tensor: the two derivative slots and the two covector indices all rotate. This is the second shape of mass weight eight.

```
lemma isLorentzCovariant_hyperchargeDeriv :
    IsLorentzCovariant 4 B repLorentz (ofComponents h.hyperchargeDeriv)
```

lemma gluonTrace_comp_swap_two_three [source]

The gluon trace contractions are antisymmetric in the last two covector indices, by `F_antisymm` in the second factor.

```
lemma gluonTrace_comp_swap_two_three (d : EightIdx) :
    h.gluonTrace (d ∘ Equiv.swap 2 3) = -h.gluonTrace d
```

lemma wTrace_comp_swap_two_three [source]

The `W`-boson trace contractions are antisymmetric in the last two covector indices.

```
lemma wTrace_comp_swap_two_three (d : EightIdx) :
    h.wTrace (d ∘ Equiv.swap 2 3) = -h.wTrace d
```

lemma hyperchargeTrace_comp_swap_two_three [source]

The hypercharge trace contractions are antisymmetric in the last two covector indices.

```
lemma hyperchargeTrace_comp_swap_two_three (d : EightIdx) :
    h.hyperchargeTrace (d ∘ Equiv.swap 2 3) = -h.hyperchargeTrace d
```

lemma hyperchargeDeriv_comp_swap_two_three [source]

The twice-derived hypercharge field strengths are antisymmetric in their two covector indices, the last two slots, by `F_antisymm` at two covariant derivatives. Nothing is assumed about the two derivative slots.

```
lemma hyperchargeDeriv_comp_swap_two_three (d : EightIdx) :
    h.hyperchargeDeriv (d ∘ Equiv.swap 2 3) = -h.hyperchargeDeriv d
```

def quadContractionSpan [source]

The span of the four Lorentz contractions of a quadruple Lorentz tensor: the images of the three metric pairings and of the Levi-Civita symbol.

```
noncomputable def quadContractionSpan (T : (Fin 4 → Fin 1 ⊕ Fin 3) → B) : Submodule ℂ B
```

lemma quadContractionSpan_le_span [source]

The span of the four Lorentz contractions of a quadruple Lorentz family lies in the span of its components: each contraction is a combination of components with constant coefficients.

```
lemma quadContractionSpan_le_span {T : (Fin 4 → Fin 1 ⊕ Fin 3) → B} :
    quadContractionSpan T ≤ Submodule.span ℂ (Set.range T)
```

lemma quadContractionSpan_le_lorentzInvariants [source]

The span of the four Lorentz contractions of a quadruple Lorentz family is a space of Lorentz invariants, the four contractions being invariant by `RankFour`.

```
lemma quadContractionSpan_le_lorentzInvariants {T : (Fin 4 → Fin 1 ⊕ Fin 3) → B}
    (hT : IsLorentzCovariant 4 B repLorentz (ofComponents T)) :
    quadContractionSpan T ≤ repLorentz.invariants
```

lemma quadContractionSpan_eq_of_antisymm [source]

For a quadruple family antisymmetric in its last two slots the four Lorentz contractions reduce to two, the `(0,2)(1,3)` metric contraction and the Levi-Civita one: the `(0,1)(2,3)` metric contraction vanishes and the `(0,3)(1,2)` one is minus the `(0,2)(1,3)` one.

```
lemma quadContractionSpan_eq_of_antisymm {T : (Fin 4 → Fin 1 ⊕ Fin 3) → B}
    (hT : ∀ d, T (d ∘ Equiv.swap 2 3) = -T d) :
    quadContractionSpan T = Submodule.span ℂ {ofComponents T (RankFour.contractionTensor 1),
      ofComponents T (RankFour.contractionTensor 3)}
```

def lorentzContractionEightSpan [source]

The span of the four Lorentz contractions of each of the three underived trace-contraction families and of the twice-derived hypercharge family: the gauge and Lorentz invariants of mass weight eight that the two classifications together produce.

```
noncomputable def lorentzContractionEightSpan : Submodule ℂ B
```

lemma lorentzContractionEightSpan_le_traceContractionEightSpan_sup [source]

The Lorentz contraction span sits inside the gauge span: each of its four blocks is spanned by the four contractions of a quadruple Lorentz family whose components generate the matching block of the gauge span.

```
lemma lorentzContractionEightSpan_le_traceContractionEightSpan_sup :
    h.lorentzContractionEightSpan ≤ h.traceContractionEightSpan ⊔ h.hyperchargeDerivSpan
```

lemma lorentzContractionEightSpan_le_invariants [source]

The Lorentz contraction span is a space of gauge invariants: it lies in the gauge span, whose generators the gauge group fixes.

```
lemma lorentzContractionEightSpan_le_invariants :
    h.lorentzContractionEightSpan ≤ repGauge.invariants
```

lemma lorentzContractionEightSpan_eq [source]

The Lorentz contraction span is spanned by eight of its sixteen generators: for each family the `(0,2)(1,3)` metric contraction and the Levi-Civita contraction. All four families are antisymmetric in their last two slots by `F_antisymm`, which removes the other two contractions of each.

```
lemma lorentzContractionEightSpan_eq :
    h.lorentzContractionEightSpan
      = Submodule.span ℂ {ofComponents h.gluonTrace (contractionTensor 1),
          ofComponents h.gluonTrace (contractionTensor 3)}
        ⊔ (Submodule.span ℂ {ofComponents h.wTrace (contractionTensor 1),
            ofComponents h.wTrace (contractionTensor 3)}
          ⊔ (Submodule.span ℂ {ofComponents h.hyperchargeTrace (contractionTensor 1),
              ofComponents h.hyperchargeTrace (contractionTensor 3)}
            ⊔ Submodule.span ℂ {ofComponents h.hyperchargeDeriv (contractionTensor 1),
              ofComponents h.hyperchargeDeriv (contractionTensor 3)}))
```

lemma lorentzContractionEightSpan_le_massWeightSubmodule [source]

The Lorentz contraction span lies in the mass-weight eight submodule.

```
lemma lorentzContractionEightSpan_le_massWeightSubmodule :
    h.lorentzContractionEightSpan ≤ h.massWeightSubmodule 8
```

lemma lorentzContractionEightSpan_le_lorentzInvariants [source]

The Lorentz contraction span is a space of Lorentz invariants.

```
lemma lorentzContractionEightSpan_le_lorentzInvariants :
    h.lorentzContractionEightSpan ≤ repLorentz.invariants
```

lemma isFixedBy_lorentzContractionEightSpan [source]

The Lorentz contraction span is fixed pointwise by both groups.

```
lemma isFixedBy_lorentzContractionEightSpan :
    IsFixedBy (gaugeLorentzMaps repGauge repLorentz) h.lorentzContractionEightSpan
```

lemma reducesInvariantsTo_lorentzContractionEightSpan [source]

The gauge sector at mass weight eight reduces, for the gauge and Lorentz groups together, to the four Lorentz contractions of each of the four families. The gauge group leaves the trace contractions and the twice-derived hypercharge field strengths; each of the four spans is spanned by a quadruple Lorentz tensor and reduces, for the Lorentz group, to the span of its four contractions. Two of the four suffice for each family, by `lorentzContractionEightSpan_eq`.

```
lemma reducesInvariantsTo_lorentzContractionEightSpan :
    ReducesInvariantsTo (gaugeLorentzMaps repGauge repLorentz) (h.massWeightSubmodule 8)
      h.lorentzContractionEightSpan
```

lemma mem_massWeightSubmodule_eight_sup_and_invariant_iff [source]

The gauge classification of mass weight eight as an equivalence: an element of `massWeightSubmodule 8 ⊔ S` is gauge invariant exactly when it is a combination of the three underived trace contractions and the twice-derived hypercharge field strengths up to a gauge-invariant remainder in `S`.

```
lemma mem_massWeightSubmodule_eight_sup_and_invariant_iff (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule 8 ⊔ S ∧ ∀ g : GaugeGroupI, repGauge g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ x - y ∈ h.traceContractionEightSpan ⊔ h.hyperchargeDerivSpan
```

lemma mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff [source]

The gauge and Lorentz classification of mass weight eight as an equivalence: an element of `massWeightSubmodule 8 ⊔ S` is fixed by both groups exactly when it is a combination of the four Lorentz contractions of the four families of section H up to a remainder in `S` fixed by both groups. The version with two contractions of each family is `mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff_reduced`.

```
lemma mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff
    (S : Submodule ℂ B) (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule 8 ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ h.lorentzContractionEightSpan
```

lemma mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff_reduced [source]

The same equivalence with the eight generators of `lorentzContractionEightSpan_eq`: for each of the three underived trace families its `(0,2)(1,3)` metric contraction, of the shape `η^{μρ} η^{νσ} F_{μν} F_{ρσ}`, and its Levi-Civita contraction, of the shape `ε^{μνρσ} F_{μν} F_{ρσ}`, and for the twice-derived hypercharge field strengths `η^{μρ} η^{νσ} ∇_μ ∇_ν B_{ρσ}` and `ε^{μνρσ} ∇_μ ∇_ν B_{ρσ}`. The eight span; they are not shown to be independent or nonzero.

```
lemma mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff_reduced
    (S : Submodule ℂ B) (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, repGauge g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule 8 ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ Submodule.span ℂ {ofComponents h.gluonTrace (contractionTensor 1),
              ofComponents h.gluonTrace (contractionTensor 3)}
            ⊔ (Submodule.span ℂ {ofComponents h.wTrace (contractionTensor 1),
                ofComponents h.wTrace (contractionTensor 3)}
              ⊔ (Submodule.span ℂ {ofComponents h.hyperchargeTrace (contractionTensor 1),
                  ofComponents h.hyperchargeTrace (contractionTensor 3)}
                ⊔ Submodule.span ℂ {ofComponents h.hyperchargeDeriv (contractionTensor 1),
                  ofComponents h.hyperchargeDeriv (contractionTensor 3)}))
```

2.476 PhyslibAlpha.Particles.StandardModel.IsGaugeSector.MassWeight.MassDimLTEight

PhyslibAlpha/Particles/StandardModel/IsGaugeSector/MassWeight/MassDimLTEight.lean on GitHub.

lemma F_mem_iSup_span_coord [source]

A field-strength symbol lies in the span of the twelve symbols evaluated on the coordinates of the standard basis of the gauge algebra.

```
lemma F_mem_iSup_span_coord {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) :
    F l μ ν φ ∈ ⨆ c : Fin 8 ⊕ Fin 3 ⊕ Fin 1,
      ℂ • F l μ ν (GaugeAlgebra.stdBasis.coord c)
```

lemma sum_cov_one [source]

A sum over families of one covector index is a single sum.

```
lemma sum_cov_one {M : Type*} [AddCommMonoid M] (f : (Fin 1 → Fin 1 ⊕ Fin 3) → M) :
    ∑ d : Fin 1 → Fin 1 ⊕ Fin 3, f d = ∑ x : Fin 1 ⊕ Fin 3, f ![x]
```

lemma sum_cov_three [source]

A sum over families of three covector indices is a triple sum.

```
lemma sum_cov_three {M : Type*} [AddCommMonoid M] (f : (Fin 3 → Fin 1 ⊕ Fin 3) → M) :
    ∑ d : Fin 3 → Fin 1 ⊕ Fin 3, f d
      = ∑ x : Fin 1 ⊕ Fin 3, ∑ y : Fin 1 ⊕ Fin 3, ∑ z : Fin 1 ⊕ Fin 3, f ![x, y, z]
```

lemma isLorentzCovariant_F_underived [source]

An underived field-strength symbol, viewed as a family indexed by its two covector indices, is a bi-Lorentz tensor.

```
lemma isLorentzCovariant_F_underived (φ : Module.Dual ℝ GaugeAlgebra) :
    IsLorentzCovariant 2 B repLorentz
      (ofComponents fun d : Fin 2 → Fin 1 ⊕ Fin 3 => F ![] (d 0) (d 1) φ)
```

lemma isLorentzCovariant_F_deriv_one [source]

A once-derived field-strength symbol, viewed as a family indexed by its derivative slot and its two covector indices, is a triple Lorentz tensor.

```
lemma isLorentzCovariant_F_deriv_one (φ : Module.Dual ℝ GaugeAlgebra) :
    IsLorentzCovariant 3 B repLorentz
      (ofComponents fun d : Fin 3 → Fin 1 ⊕ Fin 3 => F ![d 0] (d 1) (d 2) φ)
```

lemma ofComponents_metric_eq_zero_of_antisymm [source]

The metric trace of a bi-Lorentz family antisymmetric in its two indices vanishes: the metric is diagonal, and the diagonal components of such a family are zero.

```
lemma ofComponents_metric_eq_zero_of_antisymm {T : (Fin 2 → Fin 1 ⊕ Fin 3) → B}
    (hswap : ∀ x y : Fin 1 ⊕ Fin 3, T ![y, x] = - T ![x, y]) :
    ofComponents T RankTwo.metric = 0
```

lemma mem_of_lorentz_invariant_biSup_rankTwo_span [source]

A finite join of the spans of bi-Lorentz families with vanishing metric traces carries no Lorentz invariant modulo a Lorentz-stable submodule: a Lorentz invariant of the join together with `S` lies in `S`. Each span reduces to the line through its metric contraction, which is zero, and `ReducesInvariantsTo.biSup` combines the reductions.

```
lemma mem_of_lorentz_invariant_biSup_rankTwo_span {ι : Type} [DecidableEq ι]
    {T : ι → (Fin 2 → Fin 1 ⊕ Fin 3) → B}
    (hT : ∀ i, IsLorentzCovariant 2 B repLorentz (ofComponents (T i)))
    (hzero : ∀ i, ofComponents (T i) RankTwo.metric = 0) (S : Submodule ℂ B)
    (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (s : Finset ι) {x : B}
    (hx : x ∈ (⨆ i ∈ s, Submodule.span ℂ (Set.range (T i))) ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_lorentz_invariant_biSup_rankThree_span [source]

A finite join of the spans of triple Lorentz families carries no Lorentz invariant modulo a Lorentz-stable submodule: three covector indices carry no invariant contraction at all, so each span reduces to ⊥, and `ReducesInvariantsTo.biSup` combines the reductions.

```
lemma mem_of_lorentz_invariant_biSup_rankThree_span {ι : Type} [DecidableEq ι]
    {T : ι → (Fin 3 → Fin 1 ⊕ Fin 3) → B}
    (hT : ∀ i, IsLorentzCovariant 3 B repLorentz (ofComponents (T i)))
    (S : Submodule ℂ B) (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (s : Finset ι)
    {x : B} (hx : x ∈ (⨆ i ∈ s, Submodule.span ℂ (Set.range (T i))) ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma iSup_eq_biSup_univ [source]

A join over a finite index type is the join over its universal finite set.

```
lemma iSup_eq_biSup_univ {ι : Type} [Fintype ι] (f : ι → Submodule ℂ B) :
    ⨆ i, f i = ⨆ i ∈ (Finset.univ : Finset ι), f i
```

lemma ofComponents_F_underived_metric_eq_zero [source]

The metric trace of the underived field-strength symbols at a fixed direction of the gauge algebra vanishes, the symbol being antisymmetric in its two covector indices.

```
lemma ofComponents_F_underived_metric_eq_zero (φ : Module.Dual ℝ GaugeAlgebra) :
    ofComponents (fun d : Fin 2 → Fin 1 ⊕ Fin 3 => F ![] (d 0) (d 1) φ) RankTwo.metric = 0
```

lemma derivSubmodule_zero_le_iSup_span [source]

The underived field strengths lie in the join, over the twelve directions of the standard basis of the gauge algebra, of the spans of the bi-Lorentz families they form.

```
lemma derivSubmodule_zero_le_iSup_span :
    h.derivSubmodule 0 ≤ ⨆ c : Fin 8 ⊕ Fin 3 ⊕ Fin 1,
      Submodule.span ℂ (Set.range fun d : Fin 2 → Fin 1 ⊕ Fin 3 =>
        F ![] (d 0) (d 1) (GaugeAlgebra.stdBasis.coord c))
```

lemma mem_of_lorentz_invariant_massWeightSubmodule_four_sup [source]

Mass weight four carries no Lorentz invariant modulo a Lorentz-stable submodule: a Lorentz invariant of `massWeightSubmodule 4 ⊔ S` lies in `S`.

```
lemma mem_of_lorentz_invariant_massWeightSubmodule_four_sup (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.massWeightSubmodule 4 ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma etaExpand_cov_one [source]

A family of one covector index is the tuple of its own entry.

```
lemma etaExpand_cov_one (l : Fin 1 → Fin 1 ⊕ Fin 3) : ![l 0] = l
```

lemma derivSubmodule_one_le_iSup_span [source]

The once-derived field strengths lie in the join, over the twelve directions of the standard basis of the gauge algebra, of the spans of the triple Lorentz families they form.

```
lemma derivSubmodule_one_le_iSup_span :
    h.derivSubmodule 1 ≤ ⨆ c : Fin 8 ⊕ Fin 3 ⊕ Fin 1,
      Submodule.span ℂ (Set.range fun d : Fin 3 → Fin 1 ⊕ Fin 3 =>
        F ![d 0] (d 1) (d 2) (GaugeAlgebra.stdBasis.coord c))
```

lemma mem_of_lorentz_invariant_massWeightSubmodule_six_sup [source]

Mass weight six carries no Lorentz invariant modulo a Lorentz-stable submodule: a Lorentz invariant of `massWeightSubmodule 6 ⊔ S` lies in `S`.

```
lemma mem_of_lorentz_invariant_massWeightSubmodule_six_sup (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.massWeightSubmodule 6 ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_of_lorentz_invariant_massWeightSubmodule_lt_eight_sup [source]

Between mass weight zero and mass weight eight there is no Lorentz invariant: a Lorentz invariant of `massWeightSubmodule w ⊔ S` for `0 < w < 8` lies in `S`. The five odd or small weights are trivial submodules, and weights four and six are sections F and G.

```
lemma mem_of_lorentz_invariant_massWeightSubmodule_lt_eight_sup (w : ℕ) (hw0 : 0 < w)
    (hw : w < 8) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ h.massWeightSubmodule w ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff [source]

The classification below mass weight eight as an equivalence, in the shape of `mem_massWeightSubmodule_eigh` an element of `massWeightSubmodule w ⊔ S` for `0 < w < 8` is fixed by both groups exactly when it is itself an element of `S` fixed by both groups. The span of invariants that

the weight eight statement leaves over is here the trivial one, so `x - y` lies in it exactly when `x = y`. Gauge stability of `S` is not needed, and neither is gauge invariance of `x`: the forward direction is `mem_of_lorentz_invariant_massWeightSubmodule_lt_eight_sup`, which uses the Lorentz group alone.

```
lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff (w : ℕ)
    (hw0 : 0 < w) (hw : w < 8) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule w ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, repGauge g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x = y
```

lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff_mem [source]

The same classification without the existential: below mass weight eight an element of `massWeightSubmodule w ⊔ S` fixed by both groups is an element of `S` fixed by both groups, and conversely.

```
lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff_mem (w : ℕ)
    (hw0 : 0 < w) (hw : w < 8) (S : Submodule ℂ B)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule w ⊔ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ (x ∈ S ∧ (∀ g : GaugeGroupI, repGauge g x = x)
          ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
```

2.477 PhyslibAlpha.Particles.StandardModel.JetAlgebra.AlgebraRealization

PhyslibAlpha/Particles/StandardModel/JetAlgebra/AlgebraRealization.lean on GitHub.

lemma algebraRealization_fieldAlgebra_eq_top [source]

The fields of the Standard Model generate its jet algebra: the field algebra of the instance is the whole of `JetAlgebra`.

```
lemma algebraRealization_fieldAlgebra_eq_top : AlgebraRealization.id.fieldAlgebra = ⊤
```

lemma massWeightSubmodule_eq_ker [source]

When the field algebra is everything the graded piece of mass weight `n` is the weight-`n` eigenspace of `massWeightPoly` on the whole algebra: the intersection with the field algebra in the definition of `massWeightSubmodule` cuts nothing away.

```
lemma massWeightSubmodule_eq_ker (htop : h.fieldAlgebra = ⊤) (n : ℕ) :
    h.massWeightSubmodule n
      = LinearMap.ker (massWeightPoly.toLinearMap
        - (Polynomial.monomial n : B →ₗ[B] Polynomial B).restrictScalars ℂ)
```

lemma massWeightSubmoduleLE_eq_iSup_ker [source]

When the field algebra is everything the filtration by mass weight at most `w` is the join of the eigenspaces of `massWeightPoly` of weight `0` through `w`, taken over the whole algebra.

```
lemma massWeightSubmoduleLE_eq_iSup_ker (htop : h.fieldAlgebra = ⊤) (w : ℕ) :
    h.massWeightSubmoduleLE w
      = ⨆ k ∈ Finset.range (w + 1), LinearMap.ker (massWeightPoly.toLinearMap
          - (Polynomial.monomial k : B →ₗ[B] Polynomial B).restrictScalars ℂ)
```

def massWeightSubmodule [source]

The weight-`n` piece of the jet algebra, defined on the algebra itself: the eigenspace on which `massWeightPoly` is the monomial `X ^ n`. Nothing about `AlgebraRealization` enters the definition; that it agrees with the instance's graded piece is `algebraRealization_massWeightSubmodul`

```
noncomputable def massWeightSubmodule (n : ℕ) : Submodule ℂ JetAlgebra where
```

lemma mem_massWeightSubmodule [source]

Membership of the weight-`n` piece is the eigenvalue equation.

```
@[simp]
lemma mem_massWeightSubmodule {n : ℕ} {x : JetAlgebra} :
    x ∈ massWeightSubmodule n ↔ massWeightPoly x = Polynomial.monomial n x
```

def massWeightSubmoduleLE [source]

The mass-weight filtration of the jet algebra, defined on the algebra itself: the join of the weight pieces of weight at most `w`. An element lies in it exactly when it is a sum of eigenvectors of `massWeightPoly` of weight at most `w`.

```
noncomputable def massWeightSubmoduleLE (w : ℕ) : Submodule ℂ JetAlgebra
```

lemma algebraRealization_massWeightSubmodule [source]

The graded piece defined on the jet algebra is the graded piece of the instance: the two differ only by the intersection with the field algebra, which is everything.

```
lemma algebraRealization_massWeightSubmodule (n : ℕ) :
    AlgebraRealization.id.massWeightSubmodule n = massWeightSubmodule n
```

lemma mem_submodule_congr [source]

Membership of two equal submodules of the jet algebra agree. It is stated at variable endpoints so that the substitution never abstracts a pattern out of a goal mentioning the jet algebra.

```
private lemma mem_submodule_congr {S T : Submodule ℂ JetAlgebra} (h : S = T)
    (x : JetAlgebra) : x ∈ S ↔ x ∈ T
```

lemma algebraRealization_massWeightSubmoduleLE [source]

The filtration defined on the jet algebra is the filtration of the instance.

```
lemma algebraRealization_massWeightSubmoduleLE (w : ℕ) :
    AlgebraRealization.id.massWeightSubmoduleLE w = massWeightSubmoduleLE w
```

lemma mem_massWeightSubmoduleLE_eight_and_invariant_iff_lagrangian [source]

The invariant content of the Standard Model up to mass dimension four, on the jet algebra of the Standard Model itself, with nothing set aside. An element of mass weight at most eight is fixed by the jet gauge group and by the Lorentz group exactly when it is a combination of the constant term, the Higgs mass term `H† H`, and the dimension-four span: the Lorentz contractions of the gauge sector, among them the gauge kinetic and theta terms (`IsGaugeSector.lorentzContractionEightSpan`), the Higgs kinetic term, quartic potential and box terms (`HiggsAlgebraCovRealization.lorentzContractionEightSpan`), the fermion kinetic terms (`IsFermionSector.kineticSpan`) and the Yukawa couplings (`yukawaSpan`) — and nothing else.

There are no other hypotheses. The mass-weight condition is a condition on `x` alone: by `mem_massWeightSubmodule` it says that `x` is a sum of eigenvectors of `massWeightPoly` of weight at most eight, with no demand that `x` lie in any subalgebra.

```
lemma mem_massWeightSubmoduleLE_eight_and_invariant_iff_lagrangian (x : JetAlgebra) :
    (x ∈ massWeightSubmoduleLE 8
        ∧ (∀ U : JetGaugeGroupI, repJetGaugeGroupI U x = x)
        ∧ ∀ Λ : SL(2,ℂ), repLorentzGroup Λ x = x)
      ↔ x ∈ 1
        ∐ (AlgebraRealization.id.toCovAlgebraRealization.isHiggsSector.dotSpan 0 0
          ∐ (AlgebraRealization.id.toCovAlgebraRealization.isGaugeSector.
    lorentzContractionEightSpan
           ∐ AlgebraRealization.id.toCovAlgebraRealization.isHiggsSector.
    lorentzContractionEightSpan
            ∐ (AlgebraRealization.id.toCovAlgebraRealization.isFermionSector.kineticSpan
              ∐ AlgebraRealization.id.toCovAlgebraRealization.yukawaSpan)))
```

lemma mem_massWeightSubmoduleLE_eight_sup_and_invariant_iff_lagrangian [source]

The same classification as `mem_massWeightSubmoduleLE_eight_and_invariant_iff_lagrangian`, with a submodule `S` set aside — the operators of mass dimension above four, say. An element of `massWeightSubmoduleLE 8 ∐ S` is fixed by the jet gauge group and the Lorentz group exactly when it is the Standard Model Lagrangian, the Higgs mass term and a constant, up to a remainder in `S` fixed by both groups.

The hypothesis `hScov` does not disappear when the field algebra becomes everything: the covariant subalgebra `covAlgebra` remains a proper subalgebra, because a covariant element is fixed by the pure gauge jets whereas the gauge potential picks up the Maurer–Cartan shift. A set-aside `S` therefore still has to be written in the covariant towers, which is the case of interest — higher-dimension operators are built from covariant derivatives and the field strength.

```
lemma mem_massWeightSubmoduleLE_eight_sup_and_invariant_iff_lagrangian
    (S : Submodule ℂ JetAlgebra)
    (hS : ∀ U : JetGaugeGroupI, ∀ y ∈ S, repJetGaugeGroupI U y ∈ S)
    (hSL : ∀ Λ : SL(2,ℂ), ∀ y ∈ S, repLorentzGroup Λ y ∈ S)
    (hScov : S ≤ AlgebraRealization.id.covAlgebra.toSubmodule) (x : JetAlgebra) :
    (x ∈ massWeightSubmoduleLE 8 ∐ S
        ∧ (∀ U : JetGaugeGroupI, repJetGaugeGroupI U x = x)
        ∧ ∀ Λ : SL(2,ℂ), repLorentzGroup Λ x = x)
      ↔ ∃ y ∈ S, (∀ U : JetGaugeGroupI, repJetGaugeGroupI U y = y)
        ∧ (∀ Λ : SL(2,ℂ), repLorentzGroup Λ y = y)
        ∧ x - y ∈ 1
        ∐ (AlgebraRealization.id.toCovAlgebraRealization.isHiggsSector.dotSpan 0 0
```

```
      ⊔ (AlgebraRealization.id.toCovAlgebraRealization.isGaugeSector.
  lorentzContractionEightSpan
       ⊔ AlgebraRealization.id.toCovAlgebraRealization.isHiggsSector.
  lorentzContractionEightSpan
       ⊔ (AlgebraRealization.id.toCovAlgebraRealization.isFermionSector.kineticSpan
         ⊔ AlgebraRealization.id.toCovAlgebraRealization.yukawaSpan)))
```

2.478 PhyslibAlpha.Particles.StandardModel.JetAlgebra.Basic

PhyslibAlpha/Particles/StandardModel/JetAlgebra/Basic.lean on GitHub.

abbrev JetAlgebra [source]

The jet algebra of the Standard Model: the local field algebra of the Standard Model field datum. A Standard Model Lagrangian is an element of this algebra.

```
abbrev JetAlgebra : Type
```

def includeFermion [source]

The inclusion of the fermionic sector.

```
noncomputable def includeFermion : FermionJetAlgebra →ₐ[ℂ] JetAlgebra
```

def includeHiggs [source]

The inclusion of the Higgs sector.

```
noncomputable def includeHiggs : HiggsJetAlgebra →ₐ[ℂ] JetAlgebra
```

def includeGauge [source]

The inclusion of the gauge sector. The Standard Model gauge bosons are the generic ones at `GaugeAlgebra`, so this is the connection inclusion of the datum itself.

```
noncomputable def includeGauge :
    (ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) →ₐ[ℂ] JetAlgebra
```

lemma includeGauge_apply [source]

```
lemma includeGauge_apply (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    includeGauge y = ((1 : fieldData.MatterAlgebra) ⊗ₜ[ℂ] y : JetAlgebra)
```

lemma includeFermion_apply_equiv [source]

The fermionic sector inclusion factors through the sector equivalence and the generic fermionic factor inclusion.

```
lemma includeFermion_apply_equiv (a : FermionJetAlgebra) :
    includeFermion a = fieldData.includeFermion (fermionAlgebraEquiv a)
```

lemma includeHiggs_apply_equiv [source]

The Higgs sector inclusion factors through the sector equivalence and the generic bosonic factor inclusion.

```
lemma includeHiggs_apply_equiv (h : HiggsJetAlgebra) :
    includeHiggs h = fieldData.includeBoson (higgsAlgebraEquiv h)
```

lemma includeGauge_eq_includeConnection [source]

The gauge sector inclusion is the generic connection factor inclusion.

```
lemma includeGauge_eq_includeConnection :
    includeGauge = fieldData.includeConnection
```

lemma includeFermion_ι [source]

A degree-one element of the fermionic sector, included, is a total fermionic generator of the field datum, read through the fermionic generator identification.

```
lemma includeFermion_ι (v : JetComponentSpace fermionMatterField) :
    includeFermion (ExteriorAlgebra.ι ℂ v)
      = fieldData.ιFermionTotal (fermionGeneratorsEquiv.symm v)
```

lemma includeHiggs_ι [source]

A degree-one element of the Higgs sector, included, is the generator of the one bosonic species of the field datum.

```
lemma includeHiggs_ι (v : JetComponentSpace HiggsVec.matterField) :
    includeHiggs (SymmetricAlgebra.ι ℂ (JetComponentSpace HiggsVec.matterField) v)
      = fieldData.ιBoson () v
```

lemma includeGauge_one_tmul_ι [source]

A real degree-one element of the gauge sector, included, is a connection generator of the field datum: the connection factors of the two presentations are the same type, and the complexification is the scalar one.

```
lemma includeGauge_one_tmul_ι (v : GaugeBoson.JetComponentSpace GaugeAlgebra) :
    includeGauge ((1 : ℂ) ⊗ₜ[ℝ]
        SymmetricAlgebra.ι ℝ (GaugeBoson.JetComponentSpace GaugeAlgebra) v)
      = fieldData.ιConnection v
```

lemma tensor_includeRight_comm [source]

The right factor of a tensor product with a commutative right factor is central: the abstract statement, proved by tensor induction at abstract types so that it can be instantiated on the jet algebra without rewriting inside it.

```
private lemma tensor_includeRight_comm {A B : Type*} [Ring A] [Algebra ℂ A]
    [CommRing B] [Algebra ℂ B] (y : B) (x : A ⊗[ℂ] B) :
    x * Algebra.TensorProduct.includeRight (R := ℂ) (A := A) y
      = Algebra.TensorProduct.includeRight (R := ℂ) (A := A) y * x
```

lemma includeGauge_commute [source]

The image of the gauge sector is central: gauge-boson symbols commute with everything, as bosons must.

```
lemma includeGauge_commute (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) (x : JetAlgebra)
    :
    x * includeGauge y = includeGauge y * x
```

2.479 PhyslibAlpha.Particles.StandardModel.JetAlgebra.CovJetAlgebra.Basic

PhyslibAlpha/Particles/StandardModel/JetAlgebra/CovJetAlgebra/Basic.lean on GitHub.

lemma covF_mem_covAlgebra [source]

The field-strength tower lies in the covariant subalgebra.

```
lemma covF_mem_covAlgebra {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) : h.covF l μ ν φ ∈ h.covAlgebra
```

lemma covDerivH_mem_covAlgebra [source]

The Higgs tower lies in the covariant subalgebra.

```
lemma covDerivH_mem_covAlgebra {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ HiggsVec) : h.covDerivH l φ ∈ h.covAlgebra
```

lemma covDerivBarH_mem_covAlgebra [source]

The conjugate Higgs tower lies in the covariant subalgebra.

```
lemma covDerivBarH_mem_covAlgebra {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) : h.covDerivBarH l φ ∈ h.covAlgebra
```

lemma covDerivD_mem_covAlgebra [source]

The down-type quark tower lies in the covariant subalgebra.

```
lemma covDerivD_mem_covAlgebra (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) : h.covDerivD i l φ ∈ h.covAlgebra
```

lemma covDerivBarD_mem_covAlgebra [source]

The conjugate down-type quark tower lies in the covariant subalgebra.

```
lemma covDerivBarD_mem_covAlgebra (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) : h.covDerivBarD i l φ ∈ h.covAlgebra
```

lemma covDerivU_mem_covAlgebra [source]

The up-type quark tower lies in the covariant subalgebra.

```
lemma covDerivU_mem_covAlgebra (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet) : h.covDerivU i l φ ∈ h.covAlgebra
```

lemma covDerivBarU_mem_covAlgebra [source]

The conjugate up-type quark tower lies in the covariant subalgebra.

```
lemma covDerivBarU_mem_covAlgebra (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) : h.covDerivBarU i l φ ∈ h.covAlgebra
```

lemma covDerivQ_mem_covAlgebra [source]

The quark doublet tower lies in the covariant subalgebra.

```
lemma covDerivQ_mem_covAlgebra (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet) : h.covDerivQ i l φ ∈ h.covAlgebra
```

lemma covDerivBarQ_mem_covAlgebra [source]

The conjugate quark doublet tower lies in the covariant subalgebra.

```
lemma covDerivBarQ_mem_covAlgebra (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) : h.covDerivBarQ i l φ ∈ h.covAlgebra
```

lemma covDerivL_mem_covAlgebra [source]

The lepton doublet tower lies in the covariant subalgebra.

```
lemma covDerivL_mem_covAlgebra (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonDoubletV) : h.covDerivL i l φ ∈ h.covAlgebra
```

lemma covDerivBarL_mem_covAlgebra [source]

The conjugate lepton doublet tower lies in the covariant subalgebra.

```
lemma covDerivBarL_mem_covAlgebra (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) : h.covDerivBarL i l φ ∈ h.covAlgebra
```

lemma covDerivE_mem_covAlgebra [source]

The charged-lepton singlet tower lies in the covariant subalgebra.

```
lemma covDerivE_mem_covAlgebra (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonSinglet) : h.covDerivE i l φ ∈ h.covAlgebra
```

lemma covDerivBarE_mem_covAlgebra [source]

The conjugate charged-lepton singlet tower lies in the covariant subalgebra.

```
lemma covDerivBarE_mem_covAlgebra (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) : h.covDerivBarE i l φ ∈ h.covAlgebra
```

lemma covGenerators_induction [source]

A property that holds of every covariant tower holds of every covariant generator: the case analysis of the generating set, done once.

```
lemma covGenerators_induction {P : B → Prop}
    (hF : ∀ {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
      (φ : Module.Dual ℝ GaugeAlgebra), P (h.covF l μ ν φ))
    (hH : ∀ {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec),
      P (h.covDerivH l φ))
    (hBarH : ∀ {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec)),
      P (h.covDerivBarH l φ))
    (hD : ∀ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ DownSinglet), P (h.covDerivD i l φ))
    (hBarD : ∀ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule DownSinglet)), P (h.covDerivBarD i l φ))
    (hU : ∀ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ UpSinglet), P (h.covDerivU i l φ))
    (hBarU : ∀ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule UpSinglet)), P (h.covDerivBarU i l φ))
    (hQ : ∀ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ QuarkDoublet), P (h.covDerivQ i l φ))
    (hBarQ : ∀ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule QuarkDoublet)), P (h.covDerivBarQ i l φ))
    (hL : ∀ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ LeptonDoubletV), P (h.covDerivL i l φ))
    (hBarL : ∀ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)), P (h.covDerivBarL i l φ))
    (hE : ∀ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ LeptonSinglet), P (h.covDerivE i l φ))
    (hBarE : ∀ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule LeptonSinglet)), P (h.covDerivBarE i l φ)) :
    ∀ x ∈ h.covGenerators, P x
```

lemma mapsTo_covAlgebra [source]

A unital multiplicative endomorphism of the algebra that carries the covariant generators into the covariant subalgebra carries the whole subalgebra into itself.

```
lemma mapsTo_covAlgebra {f : B →ₗ[ℂ] B} (hone : f 1 = 1)
    (hmul : ∀ b₁ b₂ : B, f (b₁ * b₂) = f b₁ * f b₂)
    (hgen : ∀ x ∈ h.covGenerators, f x ∈ h.covAlgebra) {x : B} (hx : x ∈ h.covAlgebra) :
    f x ∈ h.covAlgebra
```

lemma sum_smul_mem_covAlgebra [source]

A Lorentz slot-mixing sum of covariant towers lies in the covariant subalgebra.

```
lemma sum_smul_mem_covAlgebra {n : ℕ} {c : (Fin n → (Fin 1 ⊕ Fin 3)) → ℂ}
    {G : (Fin n → (Fin 1 ⊕ Fin 3)) → B} (hG : ∀ p, G p ∈ h.covAlgebra) :
    ∑ p : Fin n → (Fin 1 ⊕ Fin 3), c p • G p ∈ h.covAlgebra
```

lemma repGlobal_one [source]

The global gauge action fixes the unit of the algebra.

```
lemma repGlobal_one (g : GaugeGroupI) : repGlobal repJet g (1 : B) = 1
```

lemma repGlobal_mem_covAlgebra [source]

The global gauge action preserves the covariant subalgebra: it carries each covariant tower to a tower of the same shape at a rotated value index, and it is multiplicative.

```
lemma repGlobal_mem_covAlgebra (g : GaugeGroupI) {x : B} (hx : x ∈ h.covAlgebra) :
    repGlobal repJet g x ∈ h.covAlgebra
```

lemma repLorentz_one [source]

The Lorentz action fixes the unit of the algebra.

```
lemma repLorentz_one (Λ : SL(2,ℂ)) : repLorentz Λ (1 : B) = 1
```

lemma repLorentz_mem_covAlgebra [source]

The Lorentz action preserves the covariant subalgebra: it carries each covariant tower to a slot-mixing sum of towers of the same shape, and it is multiplicative.

```
lemma repLorentz_mem_covAlgebra (Λ : SL(2,ℂ)) {x : B} (hx : x ∈ h.covAlgebra) :
    repLorentz Λ x ∈ h.covAlgebra
```

lemma monomial_mem_range [source]

A monomial with a coefficient in the covariant subalgebra is the image of a monomial over the covariant subalgebra.

```
private lemma monomial_mem_range {n : ℕ} {y : B} (hy : y ∈ h.covAlgebra) :
    Polynomial.monomial n y ∈ (Polynomial.mapAlgHom h.covAlgebra.val).range
```

lemma massWeightPoly_mem_range_mapAlgHom [source]

The mass-weight polynomial carries the covariant subalgebra into the polynomials with coefficients in it: each covariant tower is an eigenvector whose eigenvector is the tower itself, and the mass-weight polynomial is an algebra map.

```
lemma massWeightPoly_mem_range_mapAlgHom {x : B} (hx : x ∈ h.covAlgebra) :
    massWeightPoly x ∈ (Polynomial.mapAlgHom h.covAlgebra.val).range
```

def covRepGauge [source]

The global gauge action on the covariant subalgebra: the ambient global gauge action, which section B shows preserves it.

```
noncomputable def covRepGauge : Representation ℂ GaugeGroupI ↥h.covAlgebra
```

lemma coe_covRepGauge [source]

```
@[simp]
lemma coe_covRepGauge (g : GaugeGroupI) (x : ↥h.covAlgebra) :
    (h.covRepGauge g x : B) = repGlobal repJet g (x : B)
```

def covRepLorentz [source]

The Lorentz action on the covariant subalgebra: the ambient Lorentz action, which section B shows preserves it.

```
noncomputable def covRepLorentz : Representation ℂ SL(2,ℂ) ↥h.covAlgebra
```

lemma coe_covRepLorentz [source]

```
@[simp]
lemma coe_covRepLorentz (Λ : SL(2,ℂ)) (x : ↥h.covAlgebra) :
    (h.covRepLorentz Λ x : B) = repLorentz Λ (x : B)
```

lemma mapAlgHom_val_injective [source]

Polynomials over the covariant subalgebra inject into polynomials over the algebra.

```
lemma mapAlgHom_val_injective :
    Function.Injective (Polynomial.mapAlgHom h.covAlgebra.val)
```

def covMassWeightPoly [source]

The mass-weight polynomial of the covariant subalgebra: the ambient mass-weight polynomial, whose value on the subalgebra has all of its coefficients in the subalgebra by section B.

```
noncomputable def covMassWeightPoly : ↥h.covAlgebra →ₐ[ℂ] Polynomial ↥h.covAlgebra
```

lemma mapAlgHom_covMassWeightPoly [source]

```
@[simp]
lemma mapAlgHom_covMassWeightPoly (x : ↥h.covAlgebra) :
    Polynomial.mapAlgHom h.covAlgebra.val (h.covMassWeightPoly x) = massWeightPoly (x : B)
```

lemma covMassWeightPoly_eq_monomial_iff [source]

A mass-weight eigenvalue equation in the covariant subalgebra is the ambient one.

```
lemma covMassWeightPoly_eq_monomial_iff {n : ℕ} (x : ↥h.covAlgebra) :
    h.covMassWeightPoly x = Polynomial.monomial n x
      ↔ massWeightPoly (x : B) = Polynomial.monomial n (x : B)
```

def corestrict [source]

A family of symbols whose values lie in the covariant subalgebra, read as a family valued in that subalgebra.

```
noncomputable def corestrict {V : Type} [AddCommGroup V] [Module ℂ V]
    (G : Module.Dual ℂ V →ₗ[ℂ] B) (hG : ∀ φ, G φ ∈ h.covAlgebra) :
    Module.Dual ℂ V →ₗ[ℂ] ↥h.covAlgebra where
```

lemma coe_corestrict [source]

```
@[simp]
lemma coe_corestrict {V : Type} [AddCommGroup V] [Module ℂ V]
    (G : Module.Dual ℂ V →ₗ[ℂ] B) (hG : ∀ φ, G φ ∈ h.covAlgebra) (φ : Module.Dual ℂ V) :
    (h.corestrict G hG φ : B) = G φ
```

def corestrictReal [source]

The real-linear analogue of `corestrict`, for the gauge-algebra valued family.

```
noncomputable def corestrictReal (G : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B)
    (hG : ∀ φ, G φ ∈ h.covAlgebra) :
    Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] ↥h.covAlgebra where
```

lemma coe_corestrictReal [source]

```
@[simp]
lemma coe_corestrictReal (G : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B)
    (hG : ∀ φ, G φ ∈ h.covAlgebra) (φ : Module.Dual ℝ GaugeAlgebra) :
    (h.corestrictReal G hG φ : B) = G φ
```

abbrev CovJetAlgebra [source]

The covariant jet algebra of the Standard Model: the covariant field algebra of the jet algebra. Its elements are exactly the polynomial expressions in the covariant derivatives of the field strength and of the twelve matter towers.

```
abbrev CovJetAlgebra : Type
```

instance Ring CovJetAlgebra [source]

The covariant jet algebra is a ring: it is a subalgebra of the jet algebra. The instance is given explicitly because the generic `Subalgebra.toRing` is not found by instance search at this concrete algebra.

```
noncomputable instance : Ring CovJetAlgebra
```

instance Algebra ℂ CovJetAlgebra [source]

The covariant jet algebra is a complex algebra, stated at the ring instance just given.

```
noncomputable instance : Algebra ℂ CovJetAlgebra
```

abbrev repGaugeGroupI [source]

The action of the global gauge group on the covariant jet algebra.

```
noncomputable abbrev repGaugeGroupI : Representation ℂ GaugeGroupI CovJetAlgebra
```

abbrev repLorentzGroup [source]

The action of the Lorentz group on the covariant jet algebra.

```
noncomputable abbrev repLorentzGroup : Representation ℂ SL(2,ℂ) CovJetAlgebra
```

abbrev massWeightPoly [source]

The mass-weight polynomial of the covariant jet algebra.

```
noncomputable abbrev massWeightPoly : CovJetAlgebra →ₐ[ℂ] Polynomial CovJetAlgebra
```

def fieldStrength [source]

In the covariant jet algebra, the tower of covariant derivatives of the field strength.

```
noncomputable def fieldStrength {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] CovJetAlgebra
```

lemma coe_fieldStrength [source]

```
@[simp]
lemma coe_fieldStrength {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) :
    (fieldStrength l μ ν φ : JetAlgebra) = AlgebraRealization.id.covF l μ ν φ
```

def higgsField [source]

In the covariant jet algebra, the tower of covariant derivatives of the Higgs field.

```
noncomputable def higgsField {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ HiggsVec →ₗ[ℂ] CovJetAlgebra
```

lemma coe_higgsField [source]

```
@[simp]
lemma coe_higgsField {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ HiggsVec) :
    (higgsField l φ : JetAlgebra) = AlgebraRealization.id.covDerivH l φ
```

def conjHiggsField [source]

In the covariant jet algebra, the tower of covariant derivatives of the conjugate Higgs field.

```
noncomputable def conjHiggsField {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule HiggsVec) →ₗ[ℂ] CovJetAlgebra
```

lemma coe_conjHiggsField [source]

```
@[simp]
lemma coe_conjHiggsField {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    (conjHiggsField l φ : JetAlgebra) = AlgebraRealization.id.covDerivBarH l φ
```

def downSingletField [source]

In the covariant jet algebra, the tower of covariant derivatives of the down-type quark singlet of the `i`-th generation.

```
noncomputable def downSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ DownSinglet →ₗ[ℂ] CovJetAlgebra
```

lemma coe_downSingletField [source]

```
@[simp]
lemma coe_downSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) :
    (downSingletField i l φ : JetAlgebra) = AlgebraRealization.id.covDerivD i l φ
```

def conjDownSingletField [source]

In the covariant jet algebra, the tower of covariant derivatives of the conjugate down-type quark singlet of the `i`-th generation.

```
noncomputable def conjDownSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule DownSinglet) →ₗ[ℂ] CovJetAlgebra
```

lemma coe_conjDownSingletField [source]

```
@[simp]
lemma coe_conjDownSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    (conjDownSingletField i l φ : JetAlgebra) = AlgebraRealization.id.covDerivBarD i l φ
```

def upSingletField [source]

In the covariant jet algebra, the tower of covariant derivatives of the up-type quark singlet of the `i`-th generation.

```
noncomputable def upSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ UpSinglet →ₗ[ℂ] CovJetAlgebra
```

lemma coe_upSingletField [source]

```
@[simp]
lemma coe_upSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet) :
    (upSingletField i l φ : JetAlgebra) = AlgebraRealization.id.covDerivU i l φ
```

def conjUpSingletField [source]

In the covariant jet algebra, the tower of covariant derivatives of the conjugate up-type quark singlet of the `i`-th generation.

```
noncomputable def conjUpSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule UpSinglet) →ₗ[ℂ] CovJetAlgebra
```

lemma coe_conjUpSingletField [source]

```
@[simp]
lemma coe_conjUpSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    (conjUpSingletField i l φ : JetAlgebra) = AlgebraRealization.id.covDerivBarU i l φ
```

def quarkDoubletField [source]

In the covariant jet algebra, the tower of covariant derivatives of the quark doublet of the `i`-th generation.

```
noncomputable def quarkDoubletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ QuarkDoublet →ₗ[ℂ] CovJetAlgebra
```

lemma coe_quarkDoubletField [source]

```
@[simp]
lemma coe_quarkDoubletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet) :
    (quarkDoubletField i l φ : JetAlgebra) = AlgebraRealization.id.covDerivQ i l φ
```

def conjQuarkDoubletField [source]

In the covariant jet algebra, the tower of covariant derivatives of the conjugate quark doublet of the `i`-th generation.

```
noncomputable def conjQuarkDoubletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule QuarkDoublet) →ₗ[ℂ] CovJetAlgebra
```

lemma coe_conjQuarkDoubletField [source]

```
@[simp]
lemma coe_conjQuarkDoubletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    (conjQuarkDoubletField i l φ : JetAlgebra) = AlgebraRealization.id.covDerivBarQ i l φ
```

def leptonDoubletField [source]

In the covariant jet algebra, the tower of covariant derivatives of the lepton doublet of the `i`-th generation.

```
noncomputable def leptonDoubletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ LeptonDoubletV →ₗ[ℂ] CovJetAlgebra
```

lemma coe_leptonDoubletField [source]

```
lemma coe_leptonDoubletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonDoubletV) :
    (leptonDoubletField i l φ : JetAlgebra) = AlgebraRealization.id.covDerivL i l φ
```

def conjLeptonDoubletField [source]

In the covariant jet algebra, the tower of covariant derivatives of the conjugate lepton doublet of the `i`-th generation.

```
noncomputable def conjLeptonDoubletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule LeptonDoubletV) →ₗ[ℂ] CovJetAlgebra
```

lemma coe_conjLeptonDoubletField [source]

```
lemma coe_conjLeptonDoubletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    (conjLeptonDoubletField i l φ : JetAlgebra) = AlgebraRealization.id.covDerivBarL i l φ
```

def leptonSingletField [source]

In the covariant jet algebra, the tower of covariant derivatives of the charged-lepton singlet of the `i`-th generation.

```
noncomputable def leptonSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ LeptonSinglet →ₗ[ℂ] CovJetAlgebra
```

lemma coe_leptonSingletField [source]

```
@[simp]
lemma coe_leptonSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonSinglet) :
    (leptonSingletField i l φ : JetAlgebra) = AlgebraRealization.id.covDerivE i l φ
```

def conjLeptonSingletField [source]

In the covariant jet algebra, the tower of covariant derivatives of the conjugate charged- lepton singlet of the `i`-th generation.

```
noncomputable def conjLeptonSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule LeptonSinglet) →ₗ[ℂ] CovJetAlgebra
```

lemma coe_conjLeptonSingletField [source]

```
@[simp]
lemma coe_conjLeptonSingletField (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    (conjLeptonSingletField i l φ : JetAlgebra) = AlgebraRealization.id.covDerivBarE i l φ
```

lemma repGaugeGroupI_mul [source]

The global gauge action on the covariant jet algebra is multiplicative.

```
lemma repGaugeGroupI_mul (g : GaugeGroupI) (x y : CovJetAlgebra) :
    repGaugeGroupI g (x * y) = repGaugeGroupI g x * repGaugeGroupI g y
```

lemma repLorentzGroup_mul [source]

The Lorentz action on the covariant jet algebra is multiplicative.

```
lemma repLorentzGroup_mul (Λ : SL(2,ℂ)) (x y : CovJetAlgebra) :
    repLorentzGroup Λ (x * y) = repLorentzGroup Λ x * repLorentzGroup Λ y
```

lemma massWeightPoly_eq_monomial [source]

A mass-weight eigenvalue equation of the jet algebra, for an element of the covariant jet algebra, is a mass-weight eigenvalue equation there.

```
lemma massWeightPoly_eq_monomial {n : ℕ} {x : CovJetAlgebra}
    (hx : JetAlgebra.massWeightPoly (x : JetAlgebra)
      = Polynomial.monomial n (x : JetAlgebra)) :
    massWeightPoly x = Polynomial.monomial n x
```

lemma isLorentzCovDerivTransforms_of [source]

A Lorentz law of the jet algebra, for a family valued in the covariant jet algebra, is a Lorentz law there: the coercion is additive and commutes with scalar multiplication.

```
lemma isLorentzCovDerivTransforms_of {V : Type} [AddCommGroup V] [Module ℂ V]
    {rep : Representation ℂ SL(2,ℂ) V}
    {G : {n : ℕ} → (Fin n → (Fin 1 ⊕ Fin 3)) → Module.Dual ℂ V →ₗ[ℂ] CovJetAlgebra}
    (hG : ∀ (Λ : SL(2,ℂ)) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V),
      JetAlgebra.repLorentzGroup Λ ((G l φ : CovJetAlgebra) : JetAlgebra) =
        ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
          (∏ i, (((SL2C.toLorentzGroup Λ).1 (p i) (l i) : ℝ) : ℂ)) •
            ((G p (rep.dual Λ φ) : CovJetAlgebra) : JetAlgebra)) :
    IsLorentzCovDerivTransforms repLorentzGroup rep G
```

2.480 PhyslibAlpha.Particles.StandardModel.JetAlgebra.CovJetAlgebra.Higgs

PhyslibAlpha/Particles/StandardModel/JetAlgebra/CovJetAlgebra/Higgs.lean on GitHub.

def higgsGenerators [source]

The covariant towers of the Higgs field and of its conjugate, as a subset of the covariant jet algebra.

```
noncomputable def higgsGenerators : Set CovJetAlgebra
```

def higgsSubalgebra [source]

The subalgebra of the covariant jet algebra generated by the Higgs towers.

```
noncomputable def higgsSubalgebra : Subalgebra ℂ CovJetAlgebra
```

lemma higgsField_mem_higgsSubalgebra [source]

The Higgs tower lies in the Higgs subalgebra.

```
lemma higgsField_mem_higgsSubalgebra {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ HiggsVec) : higgsField l φ ∈ higgsSubalgebra
```

lemma conjHiggsField_mem_higgsSubalgebra [source]

The conjugate Higgs tower lies in the Higgs subalgebra.

```
lemma conjHiggsField_mem_higgsSubalgebra {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) : conjHiggsField l φ ∈ higgsSubalgebra
```

lemma mem_higgsSubalgebra_of_eq [source]

Membership of the Higgs subalgebra transported along an equation, stated at variable endpoints so that the substitution never abstracts a pattern out of a goal mentioning the covariant jet algebra.

```
private lemma mem_higgsSubalgebra_of_eq {x y : CovJetAlgebra} (h : x = y)
    (hx : x ∈ higgsSubalgebra) : y ∈ higgsSubalgebra
```

lemma mem_polyRange_of_eq [source]

The same, for the polynomials over the Higgs subalgebra.

```
private lemma mem_polyRange_of_eq {p q : Polynomial CovJetAlgebra} (h : p = q)
    (hp : p ∈ higgsSubalgebra.polyRange) : q ∈ higgsSubalgebra.polyRange
```

lemma higgsGenerators_induction [source]

A property that holds of both Higgs towers holds of every Higgs generator.

```
lemma higgsGenerators_induction {P : CovJetAlgebra → Prop}
    (hH : ∀ {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec),
      P (higgsField l φ))
    (hBarH : ∀ {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ (ConjModule HiggsVec)), P (conjHiggsField l φ)) :
    ∀ x ∈ higgsGenerators, P x
```

lemma mapsTo_higgsSubalgebra [source]

A unital multiplicative endomorphism carrying the Higgs towers into the Higgs subalgebra carries the whole subalgebra into itself.

```
lemma mapsTo_higgsSubalgebra {f : CovJetAlgebra →ₗ[ℂ] CovJetAlgebra} (hone : f 1 = 1)
    (hmul : ∀ b₁ b₂ : CovJetAlgebra, f (b₁ * b₂) = f b₁ * f b₂)
    (hgen : ∀ x ∈ higgsGenerators, f x ∈ higgsSubalgebra)
    {x : CovJetAlgebra} (hx : x ∈ higgsSubalgebra) : f x ∈ higgsSubalgebra
```

lemma sum_smul_mem_higgsSubalgebra [source]

A Lorentz slot-mixing sum of Higgs towers lies in the Higgs subalgebra.

```
lemma sum_smul_mem_higgsSubalgebra {n : ℕ} {c : (Fin n → (Fin 1 ⊕ Fin 3)) → ℂ}
    {G : (Fin n → (Fin 1 ⊕ Fin 3)) → CovJetAlgebra} (hG : ∀ p, G p ∈ higgsSubalgebra) :
    ∑ p : Fin n → (Fin 1 ⊕ Fin 3), c p • G p ∈ higgsSubalgebra
```

lemma repGaugeGroupI_mem_higgsSubalgebra [source]

The global gauge action preserves the Higgs subalgebra.

```
lemma repGaugeGroupI_mem_higgsSubalgebra (g : GaugeGroupI) {x : CovJetAlgebra}
    (hx : x ∈ higgsSubalgebra) : repGaugeGroupI g x ∈ higgsSubalgebra
```

lemma repLorentzGroup_mem_higgsSubalgebra [source]

The Lorentz action preserves the Higgs subalgebra.

```
lemma repLorentzGroup_mem_higgsSubalgebra (Λ : SL(2,ℂ)) {x : CovJetAlgebra}
    (hx : x ∈ higgsSubalgebra) : repLorentzGroup Λ x ∈ higgsSubalgebra
```

lemma massWeightPoly_mem_polyRange [source]

The mass-weight polynomial carries the Higgs subalgebra into the polynomials with coefficients in it: each Higgs tower is an eigenvector whose eigenvector is the tower itself, and the mass-weight polynomial is an algebra map.

```
lemma massWeightPoly_mem_polyRange (x : higgsSubalgebra) :
    massWeightPoly (x : CovJetAlgebra) ∈ higgsSubalgebra.polyRange
```

abbrev CovHiggsJetAlgebra [source]

The covariant jet algebra of the Higgs field: the subalgebra of the covariant jet algebra of the Standard Model generated by the covariant towers of the Higgs field and of its conjugate. Its elements are exactly the polynomial expressions in `∇_l H` and `∇_l H⁻`.

```
abbrev CovHiggsJetAlgebra : Type
```

instance Ring CovHiggsJetAlgebra [source]

The covariant jet algebra of the Higgs field is a ring. The instance is given explicitly because the generic `Subalgebra.toRing` is not found by instance search at this concrete algebra.

```
noncomputable instance : Ring CovHiggsJetAlgebra
```

instance Algebra ℂ CovHiggsJetAlgebra [source]

The covariant jet algebra of the Higgs field is a complex algebra, stated at the ring instance just given.

```
noncomputable instance : Algebra ℂ CovHiggsJetAlgebra
```

def repGaugeGroupI [source]

The action of the global gauge group on the covariant jet algebra of the Higgs field.

```
noncomputable def repGaugeGroupI : Representation ℂ GaugeGroupI CovHiggsJetAlgebra
```

lemma coe_repGaugeGroupI [source]

```
@[simp]
lemma coe_repGaugeGroupI (g : GaugeGroupI) (x : CovHiggsJetAlgebra) :
    (repGaugeGroupI g x : CovJetAlgebra) = CovJetAlgebra.repGaugeGroupI g (x : CovJetAlgebra)
```

def repLorentzGroup [source]

The action of the Lorentz group on the covariant jet algebra of the Higgs field.

```
noncomputable def repLorentzGroup : Representation ℂ SL(2,ℂ) CovHiggsJetAlgebra
```

lemma coe_repLorentzGroup [source]

```
@[simp]
lemma coe_repLorentzGroup (Λ : SL(2,ℂ)) (x : CovHiggsJetAlgebra) :
    (repLorentzGroup Λ x : CovJetAlgebra)
      = CovJetAlgebra.repLorentzGroup Λ (x : CovJetAlgebra)
```

def massWeightPoly [source]

The mass-weight polynomial of the covariant jet algebra of the Higgs field.

```
noncomputable def massWeightPoly :
    CovHiggsJetAlgebra →ₐ[ℂ] Polynomial CovHiggsJetAlgebra
```

lemma massWeightPoly_eq_monomial [source]

A mass-weight eigenvalue equation in the covariant jet algebra of the Higgs field is the covariant jet algebra's own.

```
lemma massWeightPoly_eq_monomial {n : ℕ} {x : CovHiggsJetAlgebra}
    (hx : CovJetAlgebra.massWeightPoly (x : CovJetAlgebra)
      = Polynomial.monomial n (x : CovJetAlgebra)) :
    massWeightPoly x = Polynomial.monomial n x
```

def higgsField [source]

In the covariant jet algebra of the Higgs field, the tower of covariant derivatives of the Higgs field.

```
noncomputable def higgsField {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ HiggsVec →ₗ[ℂ] CovHiggsJetAlgebra where
```

lemma coe_higgsField [source]

```
@[simp]
lemma coe_higgsField {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) :
    (higgsField l φ : CovJetAlgebra) = CovJetAlgebra.higgsField l φ
```

def conjHiggsField [source]

In the covariant jet algebra of the Higgs field, the tower of covariant derivatives of the conjugate Higgs field.

```
noncomputable def conjHiggsField {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule HiggsVec) →ₗ[ℂ] CovHiggsJetAlgebra where
```

lemma coe_conjHiggsField [source]

```
@[simp]
lemma coe_conjHiggsField {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    (conjHiggsField l φ : CovJetAlgebra) = CovJetAlgebra.conjHiggsField l φ
```

lemma repGaugeGroupI_mul [source]

The global gauge action on the covariant jet algebra of the Higgs field is multiplicative.

```
lemma repGaugeGroupI_mul (g : GaugeGroupI) (x y : CovHiggsJetAlgebra) :
    repGaugeGroupI g (x * y) = repGaugeGroupI g x * repGaugeGroupI g y
```

lemma repLorentzGroup_mul [source]

The Lorentz action on the covariant jet algebra of the Higgs field is multiplicative.

```
lemma repLorentzGroup_mul (Λ : SL(2,ℂ)) (x y : CovHiggsJetAlgebra) :
    repLorentzGroup Λ (x * y) = repLorentzGroup Λ x * repLorentzGroup Λ y
```

lemma isLorentzCovDerivTransforms_of [source]

A Lorentz law of the covariant jet algebra, for a family valued in the covariant jet algebra of the Higgs field, is a Lorentz law there.

```
lemma isLorentzCovDerivTransforms_of {V : Type} [AddCommGroup V] [Module ℂ V]
    {rep : Representation ℂ SL(2,ℂ) V}
    {G : {n : ℕ} → (Fin n → (Fin 1 ⊕ Fin 3)) → Module.Dual ℂ V →ₗ[ℂ] CovHiggsJetAlgebra}
    (hG : ∀ (Λ : SL(2,ℂ)) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V),
      CovJetAlgebra.repLorentzGroup Λ ((G l φ : CovHiggsJetAlgebra) : CovJetAlgebra) =
        ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
          (∏ i, (((SL2C.toLorentzGroup Λ).1 (p i) (l i) : ℝ) : ℂ)) •
            ((G p (rep.dual Λ φ) : CovHiggsJetAlgebra) : CovJetAlgebra)) :
    IsLorentzCovDerivTransforms repLorentzGroup rep G
```

lemma repGaugeGroupI_higgsField [source]

The Higgs tower is equivariant for the global gauge group.

```
lemma repGaugeGroupI_higgsField (g : GaugeGroupI) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ HiggsVec) :
    repGaugeGroupI g (higgsField l φ) = higgsField l (HiggsVec.repGaugeGroupI.dual g φ)
```

lemma repGaugeGroupI_conjHiggsField [source]

The conjugate Higgs tower is equivariant for the global gauge group.

```
lemma repGaugeGroupI_conjHiggsField (g : GaugeGroupI) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    repGaugeGroupI g (conjHiggsField l φ)
      = conjHiggsField l (HiggsVec.repGaugeGroupI.conj.dual g φ)
```

lemma commute_higgsField_higgsField [source]

Two Higgs towers commute.

```
lemma commute_higgsField_higgsField {n m : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (l' : Fin m → (Fin 1 ⊕ Fin 3)) (φ ψ : Module.Dual ℂ HiggsVec) :
    Commute (higgsField l φ) (higgsField l' ψ)
```

lemma commute_higgsField_conjHiggsField [source]

A Higgs tower commutes with a conjugate Higgs tower.

```
lemma commute_higgsField_conjHiggsField {n m : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (l' : Fin m → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec)
    (ψ : Module.Dual ℂ (ConjModule HiggsVec)) :
    Commute (higgsField l φ) (conjHiggsField l' ψ)
```

lemma commute_conjHiggsField_conjHiggsField [source]

Two conjugate Higgs towers commute.

```
lemma commute_conjHiggsField_conjHiggsField {n m : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (l' : Fin m → (Fin 1 ⊕ Fin 3)) (φ ψ : Module.Dual ℂ (ConjModule HiggsVec)) :
    Commute (conjHiggsField l φ) (conjHiggsField l' ψ)
```

lemma massWeightPoly_higgsField [source]

The mass weight of the Higgs tower is `2 * (1 + n)`.

```
lemma massWeightPoly_higgsField {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ HiggsVec) :
    massWeightPoly (higgsField l φ)
      = Polynomial.monomial (2 * (1 + n)) (higgsField l φ)
```

lemma massWeightPoly_conjHiggsField [source]

The mass weight of the conjugate Higgs tower is `2 * (1 + n)`.

```
lemma massWeightPoly_conjHiggsField {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    massWeightPoly (conjHiggsField l φ)
      = Polynomial.monomial (2 * (1 + n)) (conjHiggsField l φ)
```

lemma isLorentzCovDerivTransforms_higgsField [source]

The Higgs tower transforms as the covariant derivatives of a Lorentz scalar.

```
lemma isLorentzCovDerivTransforms_higgsField :
    IsLorentzCovDerivTransforms repLorentzGroup
      (Representation.trivial ℂ SL(2,ℂ) HiggsVec) (fun {_n} l => higgsField l)
```

lemma isLorentzCovDerivTransforms_conjHiggsField [source]

The conjugate Higgs tower transforms as the covariant derivatives of a Lorentz scalar.

```
lemma isLorentzCovDerivTransforms_conjHiggsField :
    IsLorentzCovDerivTransforms repLorentzGroup
      (Representation.trivial ℂ SL(2,ℂ) HiggsVec).conj (fun {_n} l => conjHiggsField l)
```

2.481 PhyslibAlpha.Particles.StandardModel.JetAlgebra.CovJetAlgebra.Sectors

PhyslibAlpha/Particles/StandardModel/JetAlgebra/CovJetAlgebra/Sectors.lean on GitHub.

def isHiggsSector [source]

The Higgs sector of the covariant jet algebra: its Higgs towers are the covariant jet algebra of the Higgs field, included.

```
noncomputable def isHiggsSector :
    HiggsAlgebraCovRealization CovJetAlgebra repGaugeGroupI repLorentzGroup massWeightPoly
    where
```

lemma isGaugeSector [source]

The gauge sector of the covariant jet algebra.

```
lemma isGaugeSector : IsGaugeSector CovJetAlgebra repGaugeGroupI repGaugeGroupI_mul
    repLorentzGroup repLorentzGroup_mul (fun {_n} l μ ν => fieldStrength l μ ν)
    massWeightPoly where
```

lemma isFermionSector [source]

The fermion sector of the covariant jet algebra.

```
lemma isFermionSector : IsFermionSector CovJetAlgebra repGaugeGroupI repGaugeGroupI_mul
    repLorentzGroup repLorentzGroup_mul
    (fun {_n} i l => downSingletField i l) (fun {_n} i l => conjDownSingletField i l)
    (fun {_n} i l => upSingletField i l) (fun {_n} i l => conjUpSingletField i l)
    (fun {_n} i l => quarkDoubletField i l) (fun {_n} i l => conjQuarkDoubletField i l)
    (fun {_n} i l => leptonDoubletField i l) (fun {_n} i l => conjLeptonDoubletField i l)
    (fun {_n} i l => leptonSingletField i l) (fun {_n} i l => conjLeptonSingletField i l)
    massWeightPoly where
```

lemma F_comm_H [source]

The cross-sector commutation rule `F_comm_H` in the covariant jet algebra.

```
lemma F_comm_H {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) :
    Commute (fieldStrength l μ ν ψ) (higgsField l' φ)
```

lemma F_comm_barH [source]

The cross-sector commutation rule `F_comm_barH` in the covariant jet algebra.

```
lemma F_comm_barH {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    Commute (fieldStrength l μ ν ψ) (conjHiggsField l' φ)
```

lemma F_comm_d [source]

The cross-sector commutation rule `F_comm_d` in the covariant jet algebra.

```
lemma F_comm_d {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ DownSinglet) :
    Commute (fieldStrength l μ ν ψ) (downSingletField i l' φ)
```

lemma F_comm_bard [source]

The cross-sector commutation rule `F_comm_bard` in the covariant jet algebra.

```
lemma F_comm_bard {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    Commute (fieldStrength l μ ν ψ) (conjDownSingletField i l' φ)
```

lemma F_comm_u [source]

The cross-sector commutation rule `F_comm_u` in the covariant jet algebra.

```
lemma F_comm_u {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ UpSinglet) :
    Commute (fieldStrength l μ ν ψ) (upSingletField i l' φ)
```

lemma F_comm_baru [source]

The cross-sector commutation rule `F_comm_baru` in the covariant jet algebra.

```
lemma F_comm_baru {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    Commute (fieldStrength l μ ν ψ) (conjUpSingletField i l' φ)
```

lemma F_comm_Q [source]

The cross-sector commutation rule `F_comm_Q` in the covariant jet algebra.

```
lemma F_comm_Q {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ QuarkDoublet) :
    Commute (fieldStrength l μ ν ψ) (quarkDoubletField i l' φ)
```

lemma F_comm_barQ [source]

The cross-sector commutation rule `F_comm_barQ` in the covariant jet algebra.

```
lemma F_comm_barQ {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    Commute (fieldStrength l μ ν ψ) (conjQuarkDoubletField i l' φ)
```

lemma F_comm_L [source]

The cross-sector commutation rule `F_comm_L` in the covariant jet algebra.

```
lemma F_comm_L {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ LeptonDoubletV) :
    Commute (fieldStrength l μ ν ψ) (leptonDoubletField i l' φ)
```

lemma F_comm_barL [source]

The cross-sector commutation rule `F_comm_barL` in the covariant jet algebra.

```
lemma F_comm_barL {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    Commute (fieldStrength l μ ν ψ) (conjLeptonDoubletField i l' φ)
```

lemma F_comm_e [source]

The cross-sector commutation rule `F_comm_e` in the covariant jet algebra.

```
lemma F_comm_e {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ LeptonSinglet) :
    Commute (fieldStrength l μ ν ψ) (leptonSingletField i l' φ)
```

lemma F_comm_bare [source]

The cross-sector commutation rule `F_comm_bare` in the covariant jet algebra.

```
lemma F_comm_bare {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    Commute (fieldStrength l μ ν ψ) (conjLeptonSingletField i l' φ)
```

lemma H_comm_d [source]

The cross-sector commutation rule `H_comm_d` in the covariant jet algebra.

```
lemma H_comm_d {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ DownSinglet) :
    Commute (higgsField l φ) (downSingletField i l' φ')
```

lemma H_comm_bard [source]

The cross-sector commutation rule `H_comm_bard` in the covariant jet algebra.

```
lemma H_comm_bard {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule DownSinglet)) :
    Commute (higgsField l φ) (conjDownSingletField i l' φ')
```

lemma H_comm_u [source]

The cross-sector commutation rule `H_comm_u` in the covariant jet algebra.

```
lemma H_comm_u {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ UpSinglet) :
    Commute (higgsField l φ) (upSingletField i l' φ')
```

lemma H_comm_baru [source]

The cross-sector commutation rule `H_comm_baru` in the covariant jet algebra.

```
lemma H_comm_baru {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule UpSinglet)) :
    Commute (higgsField l φ) (conjUpSingletField i l' φ')
```

lemma H_comm_Q [source]

The cross-sector commutation rule `H_comm_Q` in the covariant jet algebra.

```
lemma H_comm_Q {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ QuarkDoublet) :
    Commute (higgsField l φ) (quarkDoubletField i l' φ')
```

lemma H_comm_barQ [source]

The cross-sector commutation rule `H_comm_barQ` in the covariant jet algebra.

```
lemma H_comm_barQ {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    Commute (higgsField l φ) (conjQuarkDoubletField i l' φ')
```

lemma H_comm_L [source]

The cross-sector commutation rule `H_comm_L` in the covariant jet algebra.

```
lemma H_comm_L {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ LeptonDoubletV) :
    Commute (higgsField l φ) (leptonDoubletField i l' φ')
```

lemma H_comm_barL [source]

The cross-sector commutation rule `H_comm_barL` in the covariant jet algebra.

```
lemma H_comm_barL {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    Commute (higgsField l φ) (conjLeptonDoubletField i l' φ')
```

lemma H_comm_e [source]

The cross-sector commutation rule `H_comm_e` in the covariant jet algebra.

```
lemma H_comm_e {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ LeptonSinglet) :
    Commute (higgsField l φ) (leptonSingletField i l' φ')
```

lemma H_comm_bare [source]

The cross-sector commutation rule `H_comm_bare` in the covariant jet algebra.

```
lemma H_comm_bare {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ HiggsVec) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    Commute (higgsField l φ) (conjLeptonSingletField i l' φ')
```

lemma barH_comm_d [source]

The cross-sector commutation rule `barH_comm_d` in the covariant jet algebra.

```
lemma barH_comm_d {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ DownSinglet) :
    Commute (conjHiggsField l φ) (downSingletField i l' φ')
```

lemma barH_comm_bard [source]

The cross-sector commutation rule `barH_comm_bard` in the covariant jet algebra.

```
lemma barH_comm_bard {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule DownSinglet)) :
    Commute (conjHiggsField l φ) (conjDownSingletField i l' φ')
```

lemma barH_comm_u [source]

The cross-sector commutation rule `barH_comm_u` in the covariant jet algebra.

```
lemma barH_comm_u {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ UpSinglet) :
    Commute (conjHiggsField l φ) (upSingletField i l' φ')
```

lemma barH_comm_baru [source]

The cross-sector commutation rule `barH_comm_baru` in the covariant jet algebra.

```
lemma barH_comm_baru {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule UpSinglet)) :
    Commute (conjHiggsField l φ) (conjUpSingletField i l' φ')
```

lemma barH_comm_Q [source]

The cross-sector commutation rule `barH_comm_Q` in the covariant jet algebra.

```
lemma barH_comm_Q {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ QuarkDoublet) :
    Commute (conjHiggsField l φ) (quarkDoubletField i l' φ')
```

lemma barH_comm_barQ [source]

The cross-sector commutation rule `barH_comm_barQ` in the covariant jet algebra.

```
lemma barH_comm_barQ {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    Commute (conjHiggsField l φ) (conjQuarkDoubletField i l' φ')
```

lemma barH_comm_L [source]

The cross-sector commutation rule `barH_comm_L` in the covariant jet algebra.

```
lemma barH_comm_L {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ LeptonDoubletV) :
    Commute (conjHiggsField l φ) (leptonDoubletField i l' φ')
```

lemma barH_comm_barL [source]

The cross-sector commutation rule `barH_comm_barL` in the covariant jet algebra.

```
lemma barH_comm_barL {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    Commute (conjHiggsField l φ) (conjLeptonDoubletField i l' φ')
```

lemma barH_comm_e [source]

The cross-sector commutation rule `barH_comm_e` in the covariant jet algebra.

```
lemma barH_comm_e {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ LeptonSinglet) :
    Commute (conjHiggsField l φ) (leptonSingletField i l' φ')
```

lemma barH_comm_bare [source]

The cross-sector commutation rule `barH_comm_bare` in the covariant jet algebra.

```
lemma barH_comm_bare {n m : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) (i : Fin 3) (l' : Fin m → Fin 1 ⊕ Fin 3)
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    Commute (conjHiggsField l φ) (conjLeptonSingletField i l' φ')
```

2.482 PhyslibAlpha.Particles.StandardModel.JetAlgebra.FieldAlgebra

PhyslibAlpha/Particles/StandardModel/JetAlgebra/FieldAlgebra.lean on GitHub.

def generators [source]

The derivative symbols of every field of the Standard Model: the gauge field, the Higgs and its conjugate, and the three generations of each of the five fermion species with their conjugates. The set is written in exactly the shape of the body of `AlgebraRealization.fieldAlgebra`, so that the field algebra of the eventual `AlgebraRealization` instance on the jet algebra is this set adjoined.

```
noncomputable def generators : Set JetAlgebra
```

lemma gaugeField_mem_generators [source]

The gauge-field symbols are generators.

```
lemma gaugeField_mem_generators (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) : gaugeField s μ φ ∈ generators
```

lemma higgsField_mem_generators [source]

The Higgs symbols are generators.

```
lemma higgsField_mem_generators (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ HiggsVec) : higgsField s φ ∈ generators
```

lemma conjHiggsField_mem_generators [source]

The conjugate Higgs symbols are generators.

```
lemma conjHiggsField_mem_generators (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) : conjHiggsField s φ ∈ generators
```

lemma downSingletField_mem_generators [source]

The symbols of the `i`-th generation down-type quark singlet are generators.

```
lemma downSingletField_mem_generators (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) : downSingletField i s φ ∈ generators
```

lemma conjDownSingletField_mem_generators [source]

The conjugate symbols of the `i`-th generation down-type quark singlet are generators.

```
lemma conjDownSingletField_mem_generators (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) : conjDownSingletField i s φ ∈ generators
```

lemma upSingletField_mem_generators [source]

The symbols of the `i`-th generation up-type quark singlet are generators.

```
lemma upSingletField_mem_generators (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet) : upSingletField i s φ ∈ generators
```

lemma conjUpSingletField_mem_generators [source]

The conjugate symbols of the `i`-th generation up-type quark singlet are generators.

```
lemma conjUpSingletField_mem_generators (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) : conjUpSingletField i s φ ∈ generators
```

lemma quarkDoubletField_mem_generators [source]

The symbols of the `i`-th generation quark doublet are generators.

```
lemma quarkDoubletField_mem_generators (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet) : quarkDoubletField i s φ ∈ generators
```

lemma conjQuarkDoubletField_mem_generators [source]

The conjugate symbols of the `i`-th generation quark doublet are generators.

```
lemma conjQuarkDoubletField_mem_generators (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) : conjQuarkDoubletField i s φ ∈ generators
```

lemma leptonDoubletField_mem_generators [source]

The symbols of the `i`-th generation lepton doublet are generators.

```
lemma leptonDoubletField_mem_generators (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonDoubletV) : leptonDoubletField i s φ ∈ generators
```

lemma conjLeptonDoubletField_mem_generators [source]

The conjugate symbols of the `i`-th generation lepton doublet are generators.

```
lemma conjLeptonDoubletField_mem_generators (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) : conjLeptonDoubletField i s φ ∈ generators
```

lemma leptonSingletField_mem_generators [source]

The symbols of the `i`-th generation charged-lepton singlet are generators.

```
lemma leptonSingletField_mem_generators (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonSinglet) : leptonSingletField i s φ ∈ generators
```

lemma conjLeptonSingletField_mem_generators [source]

The conjugate symbols of the `i`-th generation charged-lepton singlet are generators.

```
lemma conjLeptonSingletField_mem_generators (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) : conjLeptonSingletField i s φ ∈ generators
```

lemma mem_of_adjoin_eq_top [source]

If a set generates an algebra, then every image of that algebra under an algebra map lies in any subalgebra of the target containing the image of the generating set. This is the step that transports each sector's own generation theorem into the jet algebra.

```
private lemma mem_of_adjoin_eq_top {R A B : Type*} [CommSemiring R] [Semiring A]
    [Algebra R A] [Semiring B] [Algebra R B] (f : A →ₐ[R] B) {T : Set A}
    (hT : Algebra.adjoin R T = ⊤) {C : Subalgebra R B} (hfT : f '' T ⊆ (C : Set B))
    (x : A) : f x ∈ C
```

lemma mem_adjoin_generators_of_eq [source]

Membership in the generated algebra transported along an equation. It is stated at variable endpoints so that the substitution never has to abstract a pattern out of a goal mentioning the jet algebra.

```
private lemma mem_adjoin_generators_of_eq {x y : JetAlgebra} (h : x = y)
    (hx : x ∈ Algebra.adjoin ℂ generators) : y ∈ Algebra.adjoin ℂ generators
```

lemma higgsField_eq_includeHiggs [source]

A Higgs symbol is the Higgs sector's own derivative symbol, included.

```
lemma higgsField_eq_includeHiggs (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ HiggsVec) :
    higgsField s φ
      = includeHiggs (BosonicAlgebra.iteratedJetDeriv s (BosonicAlgebra.ofField φ))
```

lemma conjHiggsField_eq_includeHiggs [source]

A conjugate Higgs symbol is the Higgs sector's own conjugate derivative symbol, included.

```
lemma conjHiggsField_eq_includeHiggs (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    conjHiggsField s φ
      = includeHiggs (BosonicAlgebra.iteratedJetDeriv s
          (BosonicAlgebra.ofConjField φ))
```

lemma includeHiggs_mem_adjoin_generators [source]

Every element of the Higgs sector lies in the algebra generated by the symbols: the Higgs jet algebra is generated by the Higgs field, its conjugate and their derivatives, and those are exactly the two Higgs families.

```
lemma includeHiggs_mem_adjoin_generators (h : HiggsJetAlgebra) :
    includeHiggs h ∈ Algebra.adjoin ℂ generators
```

lemma fermionSymbol_eq_includeFermion [source]

A fermionic symbol is the fermionic sector's own derivative symbol, included.

```
lemma fermionSymbol_eq_includeFermion (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ FermionSpace) :
    fermionSymbol s φ
      = includeFermion (FermionicAlgebra.iteratedJetDeriv s
          (FermionicAlgebra.ofField φ))
```

lemma conjFermionSymbol_eq_includeFermion [source]

A conjugate fermionic symbol is the fermionic sector's own conjugate derivative symbol, included.

```
lemma conjFermionSymbol_eq_includeFermion (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule FermionSpace)) :
    conjFermionSymbol s φ
```

```
    = includeFermion (FermionicAlgebra.iteratedJetDeriv s
        (FermionicAlgebra.ofConjField φ))
```

lemma fermionSymbol_mem_adjoin_generators [source]

Every fermionic symbol lies in the generated algebra. The families give the symbols of the covectors pulled back from a single species and generation; those span every covector on the total fermionic target space, and the symbol map is linear.

```
lemma fermionSymbol_mem_adjoin_generators (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ FermionSpace) :
    fermionSymbol s φ ∈ Algebra.adjoin ℂ generators
```

lemma conjFermionSymbol_mem_adjoin_generators [source]

Every conjugate fermionic symbol lies in the generated algebra, by the conjugate form of the spanning argument.

```
lemma conjFermionSymbol_mem_adjoin_generators (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule FermionSpace)) :
    conjFermionSymbol s φ ∈ Algebra.adjoin ℂ generators
```

lemma includeFermion_mem_adjoin_generators [source]

Every element of the fermionic sector lies in the algebra generated by the symbols.

```
lemma includeFermion_mem_adjoin_generators (f : FermionJetAlgebra) :
    includeFermion f ∈ Algebra.adjoin ℂ generators
```

lemma iteratedD_complexJetDeriv_tmul [source]

The iterated derivative of the complexification acts on a pure tensor through the gauge sector's own iterated derivative.

```
lemma iteratedD_complexJetDeriv_tmul (s : Multiset (Fin 1 ⊕ Fin 3)) (z : ℂ)
    (x : (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    Lorentz.iteratedD (LocalGaugeFieldAlgebra.complexJetDeriv GaugeAlgebra)
        LocalGaugeFieldAlgebra.complexJetDeriv_comm s (z ⊗ₜ[ℝ] x)
      = z ⊗ₜ[ℝ] (LocalGaugeFieldAlgebra.iteratedJetDeriv GaugeAlgebra) s x
```

def includeGaugeReal [source]

The real gauge-boson jet algebra inside the jet algebra of the Standard Model: the inclusion of the gauge sector precomposed with the inclusion of the real part of the complexification. It is a map of $\mathbb{R}$-algebras, which is the level at which the gauge sector's generation theorem is stated.

```
noncomputable def includeGaugeReal : (LocalGaugeFieldAlgebra GaugeAlgebra) →ₐ[ℝ] JetAlgebra
```

lemma includeGaugeReal_apply [source]

The real gauge inclusion is the gauge inclusion of the pure tensor with complex part one.

```
@[simp]
lemma includeGaugeReal_apply (x : (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    includeGaugeReal x = includeGauge ((1 : ℂ) ⊗ₜ[ℝ] x)
```

lemma gaugeField_eq_includeGaugeReal [source]

A gauge-field symbol is the gauge sector's own derivative symbol, included through the real part of the complexification.

```
lemma gaugeField_eq_includeGaugeReal (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) :
    gaugeField s μ φ
      = includeGaugeReal ((LocalGaugeFieldAlgebra.iteratedJetDeriv GaugeAlgebra) s
          ((LocalGaugeFieldAlgebra.ofA GaugeAlgebra) μ φ))
```

lemma includeGaugeReal_mem_adjoin_generators [source]

Every element of the real gauge sector lies in the algebra generated by the symbols: the gauge-boson jet algebra is generated over ℝ by the derivative symbols of the gauge field, and those are the gauge family.

```
lemma includeGaugeReal_mem_adjoin_generators (x : (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    includeGaugeReal x ∈ Algebra.adjoin ℂ generators
```

lemma includeGauge_mem_adjoin_generators [source]

Every element of the complexified gauge sector lies in the algebra generated by the symbols: a pure tensor splits as a complex scalar times the image of its real part.

```
lemma includeGauge_mem_adjoin_generators (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    includeGauge y ∈ Algebra.adjoin ℂ generators
```

lemma includeFermionFactor_mem_adjoin_generators [source]

Every element of the fermionic factor of the carrier lies in the generated algebra.

```
lemma includeFermionFactor_mem_adjoin_generators
    (a : ExteriorAlgebra ℂ fieldData.FermionGenerators) :
    fieldData.includeFermion a ∈ Algebra.adjoin ℂ generators
```

lemma includeBosonFactor_mem_adjoin_generators [source]

Every element of the bosonic factor of the carrier lies in the generated algebra.

```
lemma includeBosonFactor_mem_adjoin_generators
    (b : SymmetricAlgebra ℂ fieldData.BosonGenerators) :
    fieldData.includeBoson b ∈ Algebra.adjoin ℂ generators
```

lemma tensor_tmul_tmul [source]

A triple pure tensor is the product of the three factors placed in their own slots: the abstract statement, proved at abstract types so that it can be instantiated on the jet algebra without rewriting inside it.

```
private lemma tensor_tmul_tmul {A B C : Type*} [Ring A] [Algebra ℂ A] [Ring B]
    [Algebra ℂ B] [Ring C] [Algebra ℂ C] (a : A) (b : B) (c : C) :
    ((a ⊗ₜ[ℂ] (1 : B)) ⊗ₜ[ℂ] (1 : C)) * (((1 : A) ⊗ₜ[ℂ] b) ⊗ₜ[ℂ] (1 : C))
        * (((1 : A) ⊗ₜ[ℂ] (1 : B)) ⊗ₜ[ℂ] c)
      = (a ⊗ₜ[ℂ] b) ⊗ₜ[ℂ] c
```

lemma includeFermionFactor_mul_includeBosonFactor_mul_includeConnection [source]

A pure tensor of the jet algebra is the product of the three factor inclusions applied to its factors. The three inclusions are unfolded through the abstract `GaugeFieldData` rules, which is what keeps the computation out of the carrier's own instances.

```
lemma includeFermionFactor_mul_includeBosonFactor_mul_includeConnection
    (a : ExteriorAlgebra ℂ fieldData.FermionGenerators)
    (b : SymmetricAlgebra ℂ fieldData.BosonGenerators)
    (c : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    fieldData.includeFermion a * fieldData.includeBoson b * fieldData.includeConnection c
      = ((a ⊗ₜ[ℂ] b) ⊗ₜ[ℂ] c : JetAlgebra)
```

lemma tmul_mem_adjoin_generators [source]

Every pure tensor of the jet algebra lies in the algebra generated by the symbols.

```
lemma tmul_mem_adjoin_generators (w : fieldData.MatterAlgebra)
    (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    (w ⊗ₜ[ℂ] y : JetAlgebra) ∈ Algebra.adjoin ℂ generators
```

lemma adjoin_generators_eq_top [source]

The fields of the Standard Model generate its jet algebra. As a $\mathbb{C}$-algebra, `JetAlgebra` is adjoined by the derivative symbols of the gauge field, the Higgs and its conjugate, and the three generations of each of the five fermion species with their conjugates.

Physically: every element of the algebra in which a Standard Model Lagrangian lives is a polynomial in the fields and their spacetime derivatives — nothing else is available to write down. Formally it is the statement that the field algebra of the eventual `AlgebraRealization` instance on the jet algebra is the whole of it.

```
lemma adjoin_generators_eq_top :
    Algebra.adjoin ℂ generators = (⊤ : Subalgebra ℂ JetAlgebra)
```

2.483 PhyslibAlpha.Particles.StandardModel.JetAlgebra.GaugeAction

PhyslibAlpha/Particles/StandardModel/JetAlgebra/GaugeAction.lean on GitHub.

abbrev repJetGaugeGroupIFermion [source]

The jet gauge action on the fermionic factor: the exterior-algebra functor applied to the species-wise action on the fermionic generator space.

```
noncomputable abbrev repJetGaugeGroupIFermion :
    Representation ℂ JetGaugeGroupI (ExteriorAlgebra ℂ fieldData.FermionGenerators)
```

abbrev repJetGaugeGroupIBoson [source]

The jet gauge action on the bosonic factor: the symmetric-algebra functor applied to the species-wise action on the bosonic generator space.

```
noncomputable abbrev repJetGaugeGroupIBoson :
    Representation ℂ JetGaugeGroupI (SymmetricAlgebra ℂ fieldData.BosonGenerators)
```

def repJetGaugeGroupI [source]

The jet gauge action on the jet algebra of the Standard Model. Matter is acted on species by species from `fieldData`; the connection factor carries the generic affine `LocalGaugeFieldAlgebra.complexRepJet` action of the Standard Model local gauge data.

```
noncomputable def repJetGaugeGroupI : Representation ℂ JetGaugeGroupI JetAlgebra
```

lemma repJetGaugeGroupI_tmul [source]

```
@[simp]
lemma repJetGaugeGroupI_tmul (U : JetGaugeGroupI)
    (w : ExteriorAlgebra ℂ fieldData.FermionGenerators ⊗[ℂ]
      SymmetricAlgebra ℂ fieldData.BosonGenerators)
    (g : ℂ ⊗[ℝ] _root_.LocalGaugeFieldAlgebra GaugeAlgebra) :
    repJetGaugeGroupI U (w ⊗ₜ[ℂ] g)
      = ((repJetGaugeGroupIFermion.tprod repJetGaugeGroupIBoson) U w)
          ⊗ₜ[ℂ] (_root_.LocalGaugeFieldAlgebra.complexRepJet localGaugeData U g)
```

lemma repJetGaugeGroupI_apply_mul [source]

The jet gauge action on the jet algebra is multiplicative: a jet of gauge transformations acts on a Lagrangian term factor by factor.

```
lemma repJetGaugeGroupI_apply_mul (U : JetGaugeGroupI) (x y : JetAlgebra) :
    repJetGaugeGroupI U (x * y) = repJetGaugeGroupI U x * repJetGaugeGroupI U y
```

lemma complexRepJetGaugeGroupI_apply_one [source]

The jet gauge action on the complexified gauge sector fixes the unit.

```
lemma complexRepJetGaugeGroupI_apply_one (U : JetGaugeGroupI) :
    (_root_.LocalGaugeFieldAlgebra.complexRepJet localGaugeData) U
      (1 : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) = 1
```

lemma repJetGaugeGroupI_matter_one [source]

The matter factor of the jet gauge action fixes the unit. The proof instantiates the abstract `Representation.tprod_apply_one`, so that the unit of the matter factor is never unfolded: its two free algebras are quotients by congruences.

```
lemma repJetGaugeGroupI_matter_one (U : JetGaugeGroupI) :
    (repJetGaugeGroupIFermion.tprod repJetGaugeGroupIBoson) U
        (1 : fieldData.MatterAlgebra) = 1
```

lemma repJetGaugeGroupI_includeConnection [source]

The jet gauge action restricts to the generic connection factor, where it is the generic affine action of the Standard Model local gauge data.

```
lemma repJetGaugeGroupI_includeConnection (U : JetGaugeGroupI)
    (y : ℂ ⊗[ℝ] _root_.LocalGaugeFieldAlgebra GaugeAlgebra) :
    repJetGaugeGroupI U (fieldData.includeConnection y)
      = fieldData.includeConnection
          (_root_.LocalGaugeFieldAlgebra.complexRepJet localGaugeData U y)
```

lemma repJetGaugeGroupI_includeFermionFactor [source]

The jet gauge action restricts to the fermionic factor, where it is the exterior-algebra functor applied to the species-wise action of the datum. The factor inclusions are unfolded through the generic `GaugeFieldData` rules rather than by `rfl`: at the Standard Model datum the definitional unfolding of the units has to see through the unexposed ring congruence of the symmetric algebra.

```
lemma repJetGaugeGroupI_includeFermionFactor (U : JetGaugeGroupI)
    (a : ExteriorAlgebra ℂ fieldData.FermionGenerators) :
    repJetGaugeGroupI U (fieldData.includeFermion a)
      = fieldData.includeFermion (repJetGaugeGroupIFermion U a)
```

lemma repJetGaugeGroupI_includeBosonFactor [source]

The jet gauge action restricts to the bosonic factor, where it is the symmetric-algebra functor applied to the species-wise action of the datum.

```
lemma repJetGaugeGroupI_includeBosonFactor (U : JetGaugeGroupI)
    (b : SymmetricAlgebra ℂ fieldData.BosonGenerators) :
    repJetGaugeGroupI U (fieldData.includeBoson b)
      = fieldData.includeBoson (repJetGaugeGroupIBoson U b)
```

lemma repJetGaugeGroupI_includeGauge [source]

The jet gauge action restricts to the gauge sector's own action. The gauge sector inclusion is the connection inclusion of the datum, the Standard Model gauge bosons being the generic ones at `GaugeAlgebra`.

```
lemma repJetGaugeGroupI_includeGauge (U : JetGaugeGroupI)
    (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    repJetGaugeGroupI U (includeGauge y)
      = includeGauge (_root_.LocalGaugeFieldAlgebra.complexRepJet localGaugeData U y)
```

lemma repJetGaugeGroupI_includeFermion [source]

The jet gauge action restricts to the fermionic sector's own action.

```
lemma repJetGaugeGroupI_includeFermion (U : JetGaugeGroupI) (f : FermionJetAlgebra) :
    repJetGaugeGroupI U (includeFermion f)
      = includeFermion (FermionJetAlgebra.repJetGaugeGroupI U f)
```

lemma repJetGaugeGroupI_includeHiggs [source]

The jet gauge action restricts to the Higgs sector's own action.

```
lemma repJetGaugeGroupI_includeHiggs (U : JetGaugeGroupI) (h : HiggsJetAlgebra) :
    repJetGaugeGroupI U (includeHiggs h)
      = includeHiggs (HiggsJetAlgebra.repJetGaugeGroupI U h)
```

2.484 PhyslibAlpha.Particles.StandardModel.JetAlgebra.Generators

PhyslibAlpha/Particles/StandardModel/JetAlgebra/Generators.lean on GitHub.

def higgsField [source]

The derivative symbols ∂_s H_φ of the Higgs field inside the jet algebra of the Standard Model.

```
noncomputable def higgsField (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ HiggsVec →ₗ[ℂ] JetAlgebra
```

def conjHiggsField [source]

The derivative symbols ∂_s H̄_φ of the conjugate Higgs field inside the jet algebra of the Standard Model.

```
noncomputable def conjHiggsField (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule HiggsVec) →ₗ[ℂ] JetAlgebra
```

lemma higgsField_apply [source]

A Higgs symbol is a single generator of the Higgs sector, included: the derivative label s sits in the derivative factor of the unconjugated half of the component space.

```
lemma higgsField_apply (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) :
    higgsField s φ = includeHiggs (SymmetricAlgebra.ι ℂ _
      ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace HiggsVec.matterField))
```

lemma conjHiggsField_apply [source]

A conjugate Higgs symbol is a single generator of the Higgs sector, included: the derivative label s sits in the derivative factor of the conjugate half of the component space.

```
lemma conjHiggsField_apply (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    conjHiggsField s φ = includeHiggs (SymmetricAlgebra.ι ℂ _
      ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace HiggsVec.matterField))
```

def fermionSymbol [source]

The derivative symbols ∂_s ψ_φ of a covector φ on the total fermionic target space, inside the jet algebra of the Standard Model.

```
noncomputable def fermionSymbol (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ FermionSpace →ₗ[ℂ] JetAlgebra
```

def conjFermionSymbol [source]

The conjugate derivative symbols `∂_s ψ̄_φ` of a covector `φ` on the conjugate of the total fermionic target space.

```
noncomputable def conjFermionSymbol (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule FermionSpace) →ₗ[ℂ] JetAlgebra
```

lemma fermionSymbol_apply [source]

A fermionic symbol is a single generator of the fermionic sector, included: the derivative label `s` sits in the derivative factor of the unconjugated half of the component space.

```
lemma fermionSymbol_apply (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ FermionSpace) :
    fermionSymbol s φ = includeFermion (ExteriorAlgebra.ι ℂ
      ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace fermionMatterField))
```

lemma conjFermionSymbol_apply [source]

A conjugate fermionic symbol is a single generator of the fermionic sector, included: the derivative label `s` sits in the derivative factor of the conjugate half of the component space.

```
lemma conjFermionSymbol_apply (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule FermionSpace)) :
    conjFermionSymbol s φ = includeFermion (ExteriorAlgebra.ι ℂ
      ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace fermionMatterField))
```

def leptonDoubletField [source]

The derivative symbols `∂_s ψ_φ` of the `i`-th generation lepton doublet inside the jet algebra of the Standard Model.

```
noncomputable def leptonDoubletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ LeptonDoubletV →ₗ[ℂ] JetAlgebra
```

def conjLeptonDoubletField [source]

The conjugate derivative symbols `∂_s ψ̄_φ` of the `i`-th generation lepton doublet inside the jet algebra of the Standard Model.

```
noncomputable def conjLeptonDoubletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule LeptonDoubletV) →ₗ[ℂ] JetAlgebra
```

def leptonSingletField [source]

The derivative symbols `∂_s ψ_φ` of the `i`-th generation charged-lepton singlet inside the jet algebra of the Standard Model.

```
noncomputable def leptonSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ LeptonSinglet →ₗ[ℂ] JetAlgebra
```

def conjLeptonSingletField [source]

The conjugate derivative symbols `∂_s ψ̄_φ` of the `i`-th generation charged-lepton singlet inside the jet algebra of the Standard Model.

```
noncomputable def conjLeptonSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule LeptonSinglet) →ₗ[ℂ] JetAlgebra
```

def quarkDoubletField [source]

The derivative symbols `∂_s ψ_φ` of the `i`-th generation quark doublet inside the jet algebra of the Standard Model.

```
noncomputable def quarkDoubletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ QuarkDoublet →ₗ[ℂ] JetAlgebra
```

def conjQuarkDoubletField [source]

The conjugate derivative symbols `∂_s ψ̄_φ` of the `i`-th generation quark doublet inside the jet algebra of the Standard Model.

```
noncomputable def conjQuarkDoubletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule QuarkDoublet) →ₗ[ℂ] JetAlgebra
```

def upSingletField [source]

The derivative symbols `∂_s ψ_φ` of the `i`-th generation up-type quark singlet inside the jet algebra of the Standard Model.

```
noncomputable def upSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ UpSinglet →ₗ[ℂ] JetAlgebra
```

def conjUpSingletField [source]

The conjugate derivative symbols `∂_s ψ̄_φ` of the `i`-th generation up-type quark singlet inside the jet algebra of the Standard Model.

```
noncomputable def conjUpSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule UpSinglet) →ₗ[ℂ] JetAlgebra
```

def downSingletField [source]

The derivative symbols `∂_s ψ_φ` of the `i`-th generation down-type quark singlet inside the jet algebra of the Standard Model.

```
noncomputable def downSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ DownSinglet →ₗ[ℂ] JetAlgebra
```

def conjDownSingletField [source]

The conjugate derivative symbols `∂_s ψ̄_φ` of the `i`-th generation down-type quark singlet inside the jet algebra of the Standard Model.

```
noncomputable def conjDownSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule DownSinglet) →ₗ[ℂ] JetAlgebra
```

lemma leptonDoubletField_eq_fermionSymbol [source]

The symbols of the `i`-th generation lepton doublet are the total fermionic symbols of the covectors pulled back along the projection onto that species and generation.

```
lemma leptonDoubletField_eq_fermionSymbol (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonDoubletV) :
    leptonDoubletField i s φ
      = fermionSymbol s (Module.Dual.transpose (FermionSpace.leptonDoubletProj i) φ)
```

lemma conjLeptonDoubletField_eq_conjFermionSymbol [source]

The conjugate symbols of the `i`-th generation lepton doublet are the total conjugate fermionic symbols of the covectors pulled back along the projection.

```
lemma conjLeptonDoubletField_eq_conjFermionSymbol (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    conjLeptonDoubletField i s φ = conjFermionSymbol s
      (Module.Dual.transpose (ConjModule.map (FermionSpace.leptonDoubletProj i)) φ)
```

lemma leptonDoubletField_apply [source]

A symbol of the `i`-th generation lepton doublet is a single generator of the fermionic sector, included.

```
lemma leptonDoubletField_apply (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonDoubletV) :
    leptonDoubletField i s φ = includeFermion (ExteriorAlgebra.ι ℂ
      (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (FermionSpace.leptonDoubletProj i) φ, 0))
```

lemma conjLeptonDoubletField_apply [source]

A conjugate symbol of the `i`-th generation lepton doublet is a single generator of the fermionic sector, included.

```
lemma conjLeptonDoubletField_apply (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    conjLeptonDoubletField i s φ = includeFermion (ExteriorAlgebra.ι ℂ
      (0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (ConjModule.map (FermionSpace.leptonDoubletProj i)) φ))
```

lemma leptonSingletField_eq_fermionSymbol [source]

The symbols of the `i`-th generation charged-lepton singlet are the total fermionic symbols of the covectors pulled back along the projection onto that species and generation.

```
lemma leptonSingletField_eq_fermionSymbol (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonSinglet) :
    leptonSingletField i s φ
      = fermionSymbol s (Module.Dual.transpose (FermionSpace.leptonSingletProj i) φ)
```

lemma conjLeptonSingletField_eq_conjFermionSymbol [source]

The conjugate symbols of the `i`-th generation charged-lepton singlet are the total conjugate fermionic symbols of the covectors pulled back along the projection.

```
lemma conjLeptonSingletField_eq_conjFermionSymbol (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    conjLeptonSingletField i s φ = conjFermionSymbol s
      (Module.Dual.transpose (ConjModule.map (FermionSpace.leptonSingletProj i)) φ)
```

lemma leptonSingletField_apply [source]

A symbol of the `i`-th generation charged-lepton singlet is a single generator of the fermionic sector, included.

```
lemma leptonSingletField_apply (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonSinglet) :
    leptonSingletField i s φ = includeFermion (ExteriorAlgebra.ι ℂ
      (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (FermionSpace.leptonSingletProj i) φ, 0))
```

lemma conjLeptonSingletField_apply [source]

A conjugate symbol of the `i`-th generation charged-lepton singlet is a single generator of the fermionic sector, included.

```
lemma conjLeptonSingletField_apply (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    conjLeptonSingletField i s φ = includeFermion (ExteriorAlgebra.ι ℂ
      (0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (ConjModule.map (FermionSpace.leptonSingletProj i)) φ))
```

lemma quarkDoubletField_eq_fermionSymbol [source]

The symbols of the `i`-th generation quark doublet are the total fermionic symbols of the covectors pulled back along the projection onto that species and generation.

```
lemma quarkDoubletField_eq_fermionSymbol (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet) :
    quarkDoubletField i s φ
      = fermionSymbol s (Module.Dual.transpose (FermionSpace.quarkDoubletProj i) φ)
```

lemma conjQuarkDoubletField_eq_conjFermionSymbol [source]

The conjugate symbols of the `i`-th generation quark doublet are the total conjugate fermionic symbols of the covectors pulled back along the projection.

```
lemma conjQuarkDoubletField_eq_conjFermionSymbol (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    conjQuarkDoubletField i s φ = conjFermionSymbol s
      (Module.Dual.transpose (ConjModule.map (FermionSpace.quarkDoubletProj i)) φ)
```

lemma quarkDoubletField_apply [source]

A symbol of the `i`-th generation quark doublet is a single generator of the fermionic sector, included.

```
lemma quarkDoubletField_apply (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet) :
    quarkDoubletField i s φ = includeFermion (ExteriorAlgebra.ι ℂ
      (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (FermionSpace.quarkDoubletProj i) φ, 0))
```

lemma conjQuarkDoubletField_apply [source]

A conjugate symbol of the `i`-th generation quark doublet is a single generator of the fermionic sector, included.

```
lemma conjQuarkDoubletField_apply (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    conjQuarkDoubletField i s φ = includeFermion (ExteriorAlgebra.ι ℂ
      (0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (ConjModule.map (FermionSpace.quarkDoubletProj i)) φ))
```

lemma upSingletField_eq_fermionSymbol [source]

The symbols of the `i`-th generation up-type quark singlet are the total fermionic symbols of the covectors pulled back along the projection onto that species and generation.

```
lemma upSingletField_eq_fermionSymbol (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet) :
    upSingletField i s φ
      = fermionSymbol s (Module.Dual.transpose (FermionSpace.upSingletProj i) φ)
```

lemma conjUpSingletField_eq_conjFermionSymbol [source]

The conjugate symbols of the `i`-th generation up-type quark singlet are the total conjugate fermionic symbols of the covectors pulled back along the projection.

```
lemma conjUpSingletField_eq_conjFermionSymbol (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    conjUpSingletField i s φ = conjFermionSymbol s
      (Module.Dual.transpose (ConjModule.map (FermionSpace.upSingletProj i)) φ)
```

lemma upSingletField_apply [source]

A symbol of the `i`-th generation up-type quark singlet is a single generator of the fermionic sector, included.

```
lemma upSingletField_apply (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet) :
    upSingletField i s φ = includeFermion (ExteriorAlgebra.ι ℂ
      (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (FermionSpace.upSingletProj i) φ, 0))
```

lemma conjUpSingletField_apply [source]

A conjugate symbol of the `i`-th generation up-type quark singlet is a single generator of the fermionic sector, included.

```
lemma conjUpSingletField_apply (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    conjUpSingletField i s φ = includeFermion (ExteriorAlgebra.ι ℂ
      (0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (ConjModule.map (FermionSpace.upSingletProj i)) φ))
```

lemma downSingletField_eq_fermionSymbol [source]

The symbols of the `i`-th generation down-type quark singlet are the total fermionic symbols of the covectors pulled back along the projection onto that species and generation.

```
lemma downSingletField_eq_fermionSymbol (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) :
    downSingletField i s φ
      = fermionSymbol s (Module.Dual.transpose (FermionSpace.downSingletProj i) φ)
```

lemma conjDownSingletField_eq_conjFermionSymbol [source]

The conjugate symbols of the `i`-th generation down-type quark singlet are the total conjugate fermionic symbols of the covectors pulled back along the projection.

```
lemma conjDownSingletField_eq_conjFermionSymbol (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    conjDownSingletField i s φ = conjFermionSymbol s
      (Module.Dual.transpose (ConjModule.map (FermionSpace.downSingletProj i)) φ)
```

lemma downSingletField_apply [source]

A symbol of the `i`-th generation down-type quark singlet is a single generator of the fermionic sector, included.

```
lemma downSingletField_apply (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) :
    downSingletField i s φ = includeFermion (ExteriorAlgebra.ι ℂ
      (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (FermionSpace.downSingletProj i) φ, 0))
```

lemma conjDownSingletField_apply [source]

A conjugate symbol of the `i`-th generation down-type quark singlet is a single generator of the fermionic sector, included.

```
lemma conjDownSingletField_apply (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    conjDownSingletField i s φ = includeFermion (ExteriorAlgebra.ι ℂ
      (0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (ConjModule.map (FermionSpace.downSingletProj i)) φ))
```

lemma gaugeField_commute [source]

The gauge-field symbols commute with everything in the jet algebra: they lie in the gauge tensor factor, which is central.

```
lemma gaugeField_commute (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (x : JetAlgebra) :
    Commute (gaugeField s μ ψ) x
```

def MemHiggsSector [source]

An element of the jet algebra lies in the Higgs sector when it is the image of the Higgs jet algebra under the Higgs inclusion.

```
def MemHiggsSector (x : JetAlgebra) : Prop
```

def MemFermionSector [source]

An element of the jet algebra lies in the fermionic sector when it is the image of the fermionic jet algebra under the fermionic inclusion.

```
def MemFermionSector (x : JetAlgebra) : Prop
```

lemma memHiggsSector_higgsField [source]

The Higgs symbols lie in the Higgs sector.

```
lemma memHiggsSector_higgsField (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ HiggsVec) : MemHiggsSector (higgsField s φ)
```

lemma memHiggsSector_conjHiggsField [source]

The conjugate Higgs symbols lie in the Higgs sector.

```
lemma memHiggsSector_conjHiggsField (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) : MemHiggsSector (conjHiggsField s φ)
```

lemma commute [source]

Two elements of the Higgs sector commute: the Higgs jet algebra is a symmetric algebra, hence commutative, and the inclusion is an algebra map.

```
lemma MemHiggsSector.commute {x y : JetAlgebra} (hx : MemHiggsSector x)
    (hy : MemHiggsSector y) : Commute x y
```

lemma tensor_left_comm_right [source]

The right factor of a tensor product commutes with the left: the abstract statement, proved at abstract types so that it can be instantiated on the jet algebra without rewriting inside it.

```
private lemma tensor_left_comm_right {A B C : Type*} [Ring A] [Algebra ℂ A]
    [Ring B] [Algebra ℂ B] [Ring C] [Algebra ℂ C] (a : A) (b : B) :
    (((1 : A) ⊗ₜ[ℂ] b) ⊗ₜ[ℂ] (1 : C)) * ((a ⊗ₜ[ℂ] (1 : B)) ⊗ₜ[ℂ] (1 : C))
      = ((a ⊗ₜ[ℂ] (1 : B)) ⊗ₜ[ℂ] (1 : C)) * (((1 : A) ⊗ₜ[ℂ] b) ⊗ₜ[ℂ] (1 : C))
```

lemma commute_of_memFermionSector [source]

An element of the Higgs sector commutes with an element of the fermionic sector: they sit in different factors of the tensor product, so their product is the same pure tensor in either order. This is the statement that the Higgs is a boson — it carries no statistics against the fermions.

```
lemma MemHiggsSector.commute_of_memFermionSector {x y : JetAlgebra}
    (hx : MemHiggsSector x) (hy : MemFermionSector y) : Commute x y
```

def IsFermionGenerator [source]

An element of the jet algebra is a fermionic generator when it is the image, under the fermionic inclusion, of a degree-one element of the fermionic jet algebra. Every one of the ten fermion families consists of such elements, by the reductions of section B.3.

```
def IsFermionGenerator (x : JetAlgebra) : Prop
```

lemma memFermionSector [source]

A fermionic generator lies in the fermionic sector.

```
lemma IsFermionGenerator.memFermionSector {x : JetAlgebra} (hx : IsFermionGenerator x) :
    MemFermionSector x
```

lemma anticomm [source]

Two fermionic generators anticommute: they are degree-one elements of an exterior algebra, pushed through an algebra map. This single fact is the Fermi statistics of every matter symbol of the Standard Model, across species and generations alike.

```
lemma IsFermionGenerator.anticomm {x y : JetAlgebra} (hx : IsFermionGenerator x)
    (hy : IsFermionGenerator y) : x * y = -(y * x)
```

lemma isFermionGenerator_leptonDoubletField [source]

The symbols of the `i`-th generation lepton doublet are fermionic generators.

```
lemma isFermionGenerator_leptonDoubletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonDoubletV) : IsFermionGenerator (leptonDoubletField i s φ)
```

lemma isFermionGenerator_conjLeptonDoubletField [source]

The conjugate symbols of the `i`-th generation lepton doublet are fermionic generators.

```
lemma isFermionGenerator_conjLeptonDoubletField (i : Fin 3)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    IsFermionGenerator (conjLeptonDoubletField i s φ)
```

lemma isFermionGenerator_leptonSingletField [source]

The symbols of the `i`-th generation charged-lepton singlet are fermionic generators.

```
lemma isFermionGenerator_leptonSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonSinglet) : IsFermionGenerator (leptonSingletField i s φ)
```

lemma isFermionGenerator_conjLeptonSingletField [source]

The conjugate symbols of the `i`-th generation charged-lepton singlet are fermionic generators.

```
lemma isFermionGenerator_conjLeptonSingletField (i : Fin 3)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    IsFermionGenerator (conjLeptonSingletField i s φ)
```

lemma isFermionGenerator_quarkDoubletField [source]

The symbols of the `i`-th generation quark doublet are fermionic generators.

```
lemma isFermionGenerator_quarkDoubletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet) : IsFermionGenerator (quarkDoubletField i s φ)
```

lemma isFermionGenerator_conjQuarkDoubletField [source]

The conjugate symbols of the `i`-th generation quark doublet are fermionic generators.

```
lemma isFermionGenerator_conjQuarkDoubletField (i : Fin 3)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    IsFermionGenerator (conjQuarkDoubletField i s φ)
```

lemma isFermionGenerator_upSingletField [source]

The symbols of the `i`-th generation up-type quark singlet are fermionic generators.

```
lemma isFermionGenerator_upSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet) : IsFermionGenerator (upSingletField i s φ)
```

lemma isFermionGenerator_conjUpSingletField [source]

The conjugate symbols of the `i`-th generation up-type quark singlet are fermionic generators.

```
lemma isFermionGenerator_conjUpSingletField (i : Fin 3)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    IsFermionGenerator (conjUpSingletField i s φ)
```

lemma isFermionGenerator_downSingletField [source]

The symbols of the `i`-th generation down-type quark singlet are fermionic generators.

```
lemma isFermionGenerator_downSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) : IsFermionGenerator (downSingletField i s φ)
```

lemma isFermionGenerator_conjDownSingletField [source]

The conjugate symbols of the `i`-th generation down-type quark singlet are fermionic generators.

```
lemma isFermionGenerator_conjDownSingletField (i : Fin 3)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    IsFermionGenerator (conjDownSingletField i s φ)
```

lemma leptonDoubletField_eq_ιFermion [source]

The symbols `∂_s ψ_φ` of the `i`-th generation lepton doublet are the generators of that species in the field datum.

```
lemma leptonDoubletField_eq_ιFermion (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonDoubletV) :
    leptonDoubletField i s φ
      = fieldData.ιFermion (.leptonDoublet i)
          ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) :
            JetComponentSpace (fieldData.fermion (.leptonDoublet i)))
```

lemma conjLeptonDoubletField_eq_ιFermion [source]

The conjugate symbols `∂_s ψ̄_φ` of the `i`-th generation lepton doublet are the conjugate generators of that species in the field datum.

```
lemma conjLeptonDoubletField_eq_ιFermion (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    conjLeptonDoubletField i s φ
      = fieldData.ιFermion (.leptonDoublet i)
          ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) :
            JetComponentSpace (fieldData.fermion (.leptonDoublet i)))
```

lemma leptonSingletField_eq_ιFermion [source]

The symbols `∂_s ψ_φ` of the `i`-th generation charged-lepton singlet are the generators of that species in the field datum.

```
@[simp]
lemma leptonSingletField_eq_ιFermion (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonSinglet) :
    leptonSingletField i s φ
      = fieldData.ιFermion (.leptonSinglet i)
          ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) :
            JetComponentSpace (fieldData.fermion (.leptonSinglet i)))
```

lemma conjLeptonSingletField_eq_ιFermion [source]

The conjugate symbols `∂_s ψ̄_φ` of the `i`-th generation charged-lepton singlet are the conjugate generators of that species in the field datum.

```
@[simp]
lemma conjLeptonSingletField_eq_ιFermion (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    conjLeptonSingletField i s φ
      = fieldData.ιFermion (.leptonSinglet i)
```

```
          ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) :
            JetComponentSpace (fieldData.fermion (.leptonSinglet i)))
```

lemma quarkDoubletField_eq_ιFermion [source]

The symbols `∂_s ψ_φ` of the `i`-th generation quark doublet are the generators of that species in the field datum.

```
@[simp]
lemma quarkDoubletField_eq_ιFermion (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet) :
    quarkDoubletField i s φ
      = fieldData.ιFermion (.quarkDoublet i)
          ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) :
            JetComponentSpace (fieldData.fermion (.quarkDoublet i)))
```

lemma conjQuarkDoubletField_eq_ιFermion [source]

The conjugate symbols `∂_s ψ̄_φ` of the `i`-th generation quark doublet are the conjugate generators of that species in the field datum.

```
@[simp]
lemma conjQuarkDoubletField_eq_ιFermion (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    conjQuarkDoubletField i s φ
      = fieldData.ιFermion (.quarkDoublet i)
          ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) :
            JetComponentSpace (fieldData.fermion (.quarkDoublet i)))
```

lemma upSingletField_eq_ιFermion [source]

The symbols `∂_s ψ_φ` of the `i`-th generation up-type quark singlet are the generators of that species in the field datum.

```
@[simp]
lemma upSingletField_eq_ιFermion (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet) :
    upSingletField i s φ
      = fieldData.ιFermion (.upSinglet i)
          ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) :
            JetComponentSpace (fieldData.fermion (.upSinglet i)))
```

lemma conjUpSingletField_eq_ιFermion [source]

The conjugate symbols `∂_s ψ̄_φ` of the `i`-th generation up-type quark singlet are the conjugate generators of that species in the field datum.

```
@[simp]
lemma conjUpSingletField_eq_ιFermion (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    conjUpSingletField i s φ
      = fieldData.ιFermion (.upSinglet i)
          ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) :
            JetComponentSpace (fieldData.fermion (.upSinglet i)))
```

lemma downSingletField_eq_ιFermion [source]

The symbols `∂_s ψ_φ` of the `i`-th generation down-type quark singlet are the generators of that species in the field datum.

```
@[simp]
lemma downSingletField_eq_ιFermion (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) :
    downSingletField i s φ
      = fieldData.ιFermion (.downSinglet i)
          ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) :
            JetComponentSpace (fieldData.fermion (.downSinglet i)))
```

lemma conjDownSingletField_eq_ιFermion [source]

The conjugate symbols `∂_s ψ̄_φ` of the `i`-th generation down-type quark singlet are the conjugate generators of that species in the field datum.

```
@[simp]
lemma conjDownSingletField_eq_ιFermion (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    conjDownSingletField i s φ
      = fieldData.ιFermion (.downSinglet i)
          ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) :
            JetComponentSpace (fieldData.fermion (.downSinglet i)))
```

lemma higgsField_eq_ιBoson [source]

The Higgs symbols `∂_s H_φ` are the generators of the one bosonic species.

```
@[simp]
lemma higgsField_eq_ιBoson (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) :
    higgsField s φ
      = fieldData.ιBoson ()
          ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace HiggsVec.matterField
    )
```

lemma conjHiggsField_eq_ιBoson [source]

The conjugate Higgs symbols `∂_s H̄_φ` are the conjugate generators of the one bosonic species.

```
@[simp]
lemma conjHiggsField_eq_ιBoson (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    conjHiggsField s φ
      = fieldData.ιBoson ()
          ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace HiggsVec.matterField
    )
```

2.485 PhyslibAlpha.Particles.StandardModel.JetAlgebra.Invariants

PhyslibAlpha/Particles/StandardModel/JetAlgebra/Invariants.lean on GitHub.

def gaugeField [source]

The gauge-field derivative symbols of the jet algebra of the Standard Model: the gauge sector's symbols, included into the full algebra. It is written as a composite of two existing linear maps rather than as an anonymous constructor, so that its real linearity is inherited rather than proved by rewriting inside the jet algebra.

```
noncomputable def gaugeField (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] JetAlgebra
```

lemma gaugeField_apply [source]

```
@[simp]
lemma gaugeField_apply (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) :
    gaugeField s μ φ = includeGauge ((LocalGaugeFieldAlgebra.gaugeField GaugeAlgebra) s μ φ)
```

lemma gaugeField_eq_ιConnection [source]

The gauge-field symbols `∂_s A_μ^φ` are the connection generators of the field datum: the derivative label, the spacetime index and the adjoint covector are unchanged, and the real generator enters the complexification with the scalar one.

```
lemma gaugeField_eq_ιConnection (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) :
    gaugeField s μ φ
      = fieldData.ιConnection (SpaceTimeDerivAlgebraℝ.basisMultiset s ⊗ₜ[ℝ]
          GaugeBoson.componentDual GaugeAlgebra
            (Lorentz.CoVector.basis.dualBasis μ) φ)
```

lemma includeGauge_mem_center [source]

The gauge sector lands in the centre of the jet algebra.

```
lemma includeGauge_mem_center (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    includeGauge y ∈ Subring.center JetAlgebra
```

def gaugeRealization [source]

The jet algebra of the Standard Model realizes the gauge-boson jet algebra through the central inclusion of the gauge sector, equivariantly for the jet gauge group and the Lorentz group.

```
noncomputable def gaugeRealization :
    GaugeAlgebraRealization localGaugeData JetAlgebra repJetGaugeGroupI repLorentzGroup where
```

lemma gaugeRealization_A [source]

```
lemma gaugeRealization_A : gaugeRealization.A = gaugeField
```

lemma invariant_mem_adjoin_fieldStrength [source]

***The classification of gauge invariants of the jet algebra of the Standard Model**: a gauge-invariant element of the subalgebra generated by the gauge-field symbols `∂_s A_μ^φ` and a set `S` of elements fixed by the pure jets — such as the covariant towers*

of the fermion and Higgs fields — is a polynomial in the covariant derivatives of the field strength and the elements of `S`.

This is the covariance reduction of the Standard Model jet algebra: gauge invariance eliminates the bare gauge-field symbols in favour of the field strength, its covariant derivatives, and the matter content `S`.

```
lemma invariant_mem_adjoin_fieldStrength (S : Set JetAlgebra)
    (hS : ∀ y ∈ S, ∀ U : localGaugeData.truncationKer 0, repJetGaugeGroupI U.1 y = y)
    {x : JetAlgebra}
    (hx : x ∈ Algebra.adjoin ℂ ({b : JetAlgebra |
      ∃ (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
        (φ : Module.Dual ℝ GaugeAlgebra),
      b = gaugeField p μ φ} ∪ S))
    (hinv : ∀ U : JetGaugeGroupI, repJetGaugeGroupI U x = x) :
    x ∈ Algebra.adjoin ℂ ({b : JetAlgebra |
      ∃ (l : List (Fin 1 ⊕ Fin 3)) (ν lam : Fin 1 ⊕ Fin 3)
        (φ : Module.Dual ℝ GaugeAlgebra),
      b = GaugeAlgebraRealization.iteratedCovDerivAdjoint gaugeField l
        (GaugeAlgebraRealization.fieldStrength gaugeField ν lam) 0 φ} ∪ S)
```

2.486 `PhyslibAlpha.Particles.StandardModel.JetAlgebra.JetDeriv`

`PhyslibAlpha/Particles/StandardModel/JetAlgebra/JetDeriv.lean` on GitHub.

`abbrev jetDerivFermionFactor` [source]

The total derivative on the fermionic factor: the even derivation of the exterior algebra extending the derivative shift on the fermionic generator space of the datum.

```
noncomputable abbrev jetDerivFermionFactor (μ : Fin 1 ⊕ Fin 3) :
    ExteriorAlgebra ℂ fieldData.FermionGenerators →ₗ[ℂ]
      ExteriorAlgebra ℂ fieldData.FermionGenerators
```

`abbrev jetDerivBosonFactor` [source]

The total derivative on the bosonic factor: the derivation of the symmetric algebra extending the derivative shift on the bosonic generator space of the datum.

```
noncomputable abbrev jetDerivBosonFactor (μ : Fin 1 ⊕ Fin 3) :
    SymmetricAlgebra ℂ fieldData.BosonGenerators →ₗ[ℂ]
      SymmetricAlgebra ℂ fieldData.BosonGenerators
```

`def jetDeriv` [source]

***The formal total derivative on the jet algebra of the Standard Model**: the sum of the total derivatives of the three factors, each acting on its own factor.*

```
noncomputable def jetDeriv (μ : Fin 1 ⊕ Fin 3) : JetAlgebra →ₗ[ℂ] JetAlgebra
```

`lemma jetDeriv_tmul` [source]

```
lemma jetDeriv_tmul (μ : Fin 1 ⊕ Fin 3) (f : ExteriorAlgebra ℂ fieldData.FermionGenerators)
    (h : SymmetricAlgebra ℂ fieldData.BosonGenerators)
    (g : ℂ ⊗[ℝ] _root_.LocalGaugeFieldAlgebra GaugeAlgebra) :
    jetDeriv μ ((f ⊗ₜ[ℂ] h) ⊗ₜ[ℂ] g)
```

```
      = ((jetDerivFermionFactor μ f) ⊗ₜ[ℂ] h) ⊗ₜ[ℂ] g
        + (f ⊗ₜ[ℂ] (jetDerivBosonFactor μ h)) ⊗ₜ[ℂ] g
        + (f ⊗ₜ[ℂ] h) ⊗ₜ[ℂ] (_root_.LocalGaugeFieldAlgebra.complexJetDeriv GaugeAlgebra μ g)
```

lemma deriv₃_tmul_left [source]

The three-factor derivation on a pure tensor, with the second and third derivatives annihilating their entries. Like the assemblies of sections B–D it is proved at abstract types and instantiated, so that the unit of a factor is never unfolded: the free algebras are quotients by congruences.

```
private lemma deriv₃_tmul_left {A B C : Type*} [AddCommGroup A] [Module ℂ A]
    [AddCommGroup B] [Module ℂ B] [AddCommGroup C] [Module ℂ C]
    (D : A →ₗ[ℂ] A) (E : B →ₗ[ℂ] B) (F : C →ₗ[ℂ] C) {b : B} {c : C}
    (hE : E b = 0) (hF : F c = 0) (a : A) :
    (TensorProduct.map (TensorProduct.map D (LinearMap.id (M := B)))
          (LinearMap.id (M := C))
        + TensorProduct.map (TensorProduct.map (LinearMap.id (M := A)) E)
          (LinearMap.id (M := C))
        + TensorProduct.map (LinearMap.id (M := A ⊗[ℂ] B)) F) ((a ⊗ₜ[ℂ] b) ⊗ₜ[ℂ] c)
      = (D a ⊗ₜ[ℂ] b) ⊗ₜ[ℂ] c
```

lemma deriv₃_tmul_mid [source]

The same with only the middle derivative surviving.

```
private lemma deriv₃_tmul_mid {A B C : Type*} [AddCommGroup A] [Module ℂ A]
    [AddCommGroup B] [Module ℂ B] [AddCommGroup C] [Module ℂ C]
    (D : A →ₗ[ℂ] A) (E : B →ₗ[ℂ] B) (F : C →ₗ[ℂ] C) {a : A} {c : C}
    (hD : D a = 0) (hF : F c = 0) (b : B) :
    (TensorProduct.map (TensorProduct.map D (LinearMap.id (M := B)))
          (LinearMap.id (M := C))
        + TensorProduct.map (TensorProduct.map (LinearMap.id (M := A)) E)
          (LinearMap.id (M := C))
        + TensorProduct.map (LinearMap.id (M := A ⊗[ℂ] B)) F) ((a ⊗ₜ[ℂ] b) ⊗ₜ[ℂ] c)
      = (a ⊗ₜ[ℂ] E b) ⊗ₜ[ℂ] c
```

lemma deriv₃_tmul_right [source]

The same with only the third derivative surviving.

```
private lemma deriv₃_tmul_right {A B C : Type*} [AddCommGroup A] [Module ℂ A]
    [AddCommGroup B] [Module ℂ B] [AddCommGroup C] [Module ℂ C]
    (D : A →ₗ[ℂ] A) (E : B →ₗ[ℂ] B) (F : C →ₗ[ℂ] C) {a : A} {b : B}
    (hD : D a = 0) (hE : E b = 0) (c : C) :
    (TensorProduct.map (TensorProduct.map D (LinearMap.id (M := B)))
          (LinearMap.id (M := C))
        + TensorProduct.map (TensorProduct.map (LinearMap.id (M := A)) E)
          (LinearMap.id (M := C))
        + TensorProduct.map (LinearMap.id (M := A ⊗[ℂ] B)) F) ((a ⊗ₜ[ℂ] b) ⊗ₜ[ℂ] c)
      = (a ⊗ₜ[ℂ] b) ⊗ₜ[ℂ] F c
```

lemma map_comp_map_left [source]

Composition of two factorwise maps on the left factor, at abstract types.

```
private lemma map_comp_map_left {A B : Type*} [AddCommGroup A] [Module ℂ A]
    [AddCommGroup B] [Module ℂ B] (D D' : A →ₗ[ℂ] A) :
    (TensorProduct.map D (LinearMap.id (M := B))).comp
        (TensorProduct.map D' (LinearMap.id (M := B)))
      = TensorProduct.map (D.comp D') (LinearMap.id (M := B))
```

lemma map_comp_map_right [source]

Composition of two factorwise maps on the right factor, at abstract types.

```
private lemma map_comp_map_right {A B : Type*} [AddCommGroup A] [Module ℂ A]
    [AddCommGroup B] [Module ℂ B] (D D' : B →ₗ[ℂ] B) :
    (TensorProduct.map (LinearMap.id (M := A)) D).comp
        (TensorProduct.map (LinearMap.id (M := A)) D')
      = TensorProduct.map (LinearMap.id (M := A)) (D.comp D')
```

lemma map_left_comm_map_right [source]

Maps on opposite factors commute, at abstract types.

```
private lemma map_left_comm_map_right {A B : Type*} [AddCommGroup A] [Module ℂ A]
    [AddCommGroup B] [Module ℂ B] (D : A →ₗ[ℂ] A) (D' : B →ₗ[ℂ] B) :
    (TensorProduct.map D (LinearMap.id (M := B))).comp
        (TensorProduct.map (LinearMap.id (M := A)) D')
      = (TensorProduct.map (LinearMap.id (M := A)) D').comp
        (TensorProduct.map D (LinearMap.id (M := B)))
```

lemma iteratedD_map [source]

A linear map intertwining two commuting families intertwines their iterates. Proved at abstract types and instantiated in term mode, so that the induction never runs inside the jet algebra.

```
private lemma iteratedD_map {A B : Type} [Ring A] [Algebra ℂ A] [Ring B] [Algebra ℂ B]
    (D : (Fin 1 ⊕ Fin 3) → A →ₗ[ℂ] A)
    (hD : ∀ μ ν, (D μ).comp (D ν) = (D ν).comp (D μ))
    (E : (Fin 1 ⊕ Fin 3) → B →ₗ[ℂ] B)
    (hE : ∀ μ ν, (E μ).comp (E ν) = (E ν).comp (E μ))
    (φ : A →ₗ[ℂ] B) (hφ : ∀ μ x, φ (D μ x) = E μ (φ x))
    (s : Multiset (Fin 1 ⊕ Fin 3)) (x : A) :
    φ (Lorentz.iteratedD D hD s x) = Lorentz.iteratedD E hE s (φ x)
```

lemma complexJetDeriv_one [source]

The connection factor's derivative annihilates the unit of the complexified gauge-boson jet algebra.

```
private lemma complexJetDeriv_one (μ : Fin 1 ⊕ Fin 3) :
    _root_.LocalGaugeFieldAlgebra.complexJetDeriv GaugeAlgebra μ
        (1 : ℂ ⊗[ℝ] _root_.LocalGaugeFieldAlgebra GaugeAlgebra) = 0
```

lemma jetDeriv_includeFermionFactor [source]

The derivative acts on the fermionic factor through that factor's own derivation. The factor inclusions are unfolded through the generic `GaugeFieldData` rules rather than by `rfl`: at the Standard Model datum the definitional unfolding of the units has to see through the unexposed ring congruence of the symmetric algebra.

```
lemma jetDeriv_includeFermionFactor (μ : Fin 1 ⊕ Fin 3)
    (f : ExteriorAlgebra ℂ fieldData.FermionGenerators) :
    jetDeriv μ (fieldData.includeFermion f)
      = fieldData.includeFermion (jetDerivFermionFactor μ f)
```

lemma jetDeriv_includeBosonFactor [source]

The derivative acts on the bosonic factor through that factor's own derivation.

```
lemma jetDeriv_includeBosonFactor (μ : Fin 1 ⊕ Fin 3)
    (h : SymmetricAlgebra ℂ fieldData.BosonGenerators) :
    jetDeriv μ (fieldData.includeBoson h)
      = fieldData.includeBoson (jetDerivBosonFactor μ h)
```

lemma jetDeriv_includeConnection [source]

The derivative acts on the connection factor through the generic gauge-boson derivative.

```
lemma jetDeriv_includeConnection (μ : Fin 1 ⊕ Fin 3)
    (y : ℂ ⊗[ℝ] _root_.LocalGaugeFieldAlgebra GaugeAlgebra) :
    jetDeriv μ (fieldData.includeConnection y)
      = fieldData.includeConnection
          (_root_.LocalGaugeFieldAlgebra.complexJetDeriv GaugeAlgebra μ y)
```

lemma jetDeriv_includeFermion [source]

The derivative acts on the fermionic sector through the fermionic sector's own derivative.

```
lemma jetDeriv_includeFermion (μ : Fin 1 ⊕ Fin 3) (f : FermionJetAlgebra) :
    jetDeriv μ (includeFermion f) = includeFermion (FermionicAlgebra.jetDeriv μ f)
```

lemma jetDeriv_includeHiggs [source]

The derivative acts on the Higgs sector through the Higgs sector's own derivative.

```
lemma jetDeriv_includeHiggs (μ : Fin 1 ⊕ Fin 3) (h : HiggsJetAlgebra) :
    jetDeriv μ (includeHiggs h) = includeHiggs (BosonicAlgebra.jetDeriv μ h)
```

lemma jetDeriv_includeGauge [source]

The derivative acts on the gauge sector through the gauge sector's own derivative. The gauge sector inclusion is the connection inclusion of the datum, the Standard Model gauge bosons being the generic ones at `GaugeAlgebra`.

```
lemma jetDeriv_includeGauge (μ : Fin 1 ⊕ Fin 3)
    (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    jetDeriv μ (includeGauge y)
      = includeGauge (_root_.LocalGaugeFieldAlgebra.complexJetDeriv GaugeAlgebra μ y)
```

lemma map_derivation_left [source]

A derivation of the left factor extends to a derivation of the tensor product.

```
lemma _root_.TensorProduct.map_derivation_left {A B : Type*} [Ring A] [Algebra ℂ A]
    [Ring B] [Algebra ℂ B] (D : A →ₗ[ℂ] A)
    (hD : ∀ x y, D (x * y) = D x * y + x * D y) (x y : A ⊗[ℂ] B) :
    TensorProduct.map D LinearMap.id (x * y)
      = TensorProduct.map D LinearMap.id x * y
        + x * TensorProduct.map D LinearMap.id y
```

lemma map_derivation_right [source]

A derivation of the right factor extends to a derivation of the tensor product.

```
lemma _root_.TensorProduct.map_derivation_right {A B : Type*} [Ring A] [Algebra ℂ A]
    [Ring B] [Algebra ℂ B] (D : B →ₗ[ℂ] B)
    (hD : ∀ x y, D (x * y) = D x * y + x * D y) (x y : A ⊗[ℂ] B) :
    TensorProduct.map LinearMap.id D (x * y)
      = TensorProduct.map LinearMap.id D x * y
        + x * TensorProduct.map LinearMap.id D y
```

lemma add₃_derivation [source]

The sum of three derivations is a derivation: the purely additive assembly, stated abstractly so it can be instantiated without rewriting inside a large type.

```
private lemma add₃_derivation {R : Type*} [NonUnitalNonAssocRing R]
    {D₁ D₂ D₃ : R → R} {x y : R}
    (h₁ : D₁ (x * y) = D₁ x * y + x * D₁ y)
    (h₂ : D₂ (x * y) = D₂ x * y + x * D₂ y)
    (h₃ : D₃ (x * y) = D₃ x * y + x * D₃ y) :
    D₁ (x * y) + D₂ (x * y) + D₃ (x * y)
      = (D₁ x + D₂ x + D₃ x) * y + x * (D₁ y + D₂ y + D₃ y)
```

lemma jetDeriv_mul [source]

***The Leibniz rule** for the total derivative on the jet algebra.*

```
lemma jetDeriv_mul (μ : Fin 1 ⊕ Fin 3) (x y : JetAlgebra) :
    jetDeriv μ (x * y) = jetDeriv μ x * y + x * jetDeriv μ y
```

lemma add₃_comp_comm [source]

The sum of three maps pairwise commuting with the sum of three others commutes with it: the purely additive assembly, stated abstractly so it can be instantiated without rewriting inside a large type.

```
private lemma add₃_comp_comm {M : Type*} [AddCommMonoid M] [Module ℂ M]
    {A₁ A₂ A₃ B₁ B₂ B₃ : M →ₗ[ℂ] M}
    (h11 : A₁.comp B₁ = B₁.comp A₁) (h12 : A₁.comp B₂ = B₂.comp A₁)
    (h13 : A₁.comp B₃ = B₃.comp A₁) (h21 : A₂.comp B₁ = B₁.comp A₂)
    (h22 : A₂.comp B₂ = B₂.comp A₂) (h23 : A₂.comp B₃ = B₃.comp A₂)
    (h31 : A₃.comp B₁ = B₁.comp A₃) (h32 : A₃.comp B₂ = B₂.comp A₃)
    (h33 : A₃.comp B₃ = B₃.comp A₃) :
    (A₁ + A₂ + A₃).comp (B₁ + B₂ + B₃) = (B₁ + B₂ + B₃).comp (A₁ + A₂ + A₃)
```

lemma jetDeriv_comm [source]

The total derivatives on the jet algebra commute.

```
lemma jetDeriv_comm (μ ν : Fin 1 ⊕ Fin 3) :
    (jetDeriv μ).comp (jetDeriv ν) = (jetDeriv ν).comp (jetDeriv μ)
```

lemma iteratedD_includeFermion [source]

The iterated total derivative acts on the fermionic sector through the fermionic sector's own iterated derivative.

```
lemma iteratedD_includeFermion (s : Multiset (Fin 1 ⊕ Fin 3)) (f : FermionJetAlgebra) :
    Lorentz.iteratedD jetDeriv jetDeriv_comm s (includeFermion f)
      = includeFermion (FermionicAlgebra.iteratedJetDeriv s f)
```

lemma iteratedD_includeHiggs [source]

The iterated total derivative acts on the Higgs sector through the Higgs sector's own iterated derivative.

```
lemma iteratedD_includeHiggs (s : Multiset (Fin 1 ⊕ Fin 3)) (h : HiggsJetAlgebra) :
    Lorentz.iteratedD jetDeriv jetDeriv_comm s (includeHiggs h)
      = includeHiggs (BosonicAlgebra.iteratedJetDeriv s h)
```

lemma iteratedD_includeGauge [source]

The iterated total derivative acts on the gauge sector through the gauge sector's own iterated derivative. Like its two siblings this instantiates the abstract `iteratedD_map` rather than running the induction inside the jet algebra.

```
lemma iteratedD_includeGauge (s : Multiset (Fin 1 ⊕ Fin 3))
    (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    Lorentz.iteratedD jetDeriv jetDeriv_comm s (includeGauge y)
      = includeGauge (Lorentz.iteratedD
          (_root_.LocalGaugeFieldAlgebra.complexJetDeriv GaugeAlgebra)
          _root_.LocalGaugeFieldAlgebra.complexJetDeriv_comm s y)
```

2.487 PhyslibAlpha.Particles.StandardModel.JetAlgebra.LorentzAction

PhyslibAlpha/Particles/StandardModel/JetAlgebra/LorentzAction.lean on GitHub.

abbrev repLorentzGroupFermion [source]

The Lorentz action on the fermionic factor: the exterior-algebra functor applied to the species-wise Lorentz action on the fermionic generator space.

```
noncomputable abbrev repLorentzGroupFermion :
    Representation ℂ SL(2,ℂ) (ExteriorAlgebra ℂ fieldData.FermionGenerators)
```

abbrev repLorentzGroupBoson [source]

The Lorentz action on the bosonic factor.

```
noncomputable abbrev repLorentzGroupBoson :
    Representation ℂ SL(2,ℂ) (SymmetricAlgebra ℂ fieldData.BosonGenerators)
```

def repLorentzGroup [source]

The Lorentz action on the jet algebra of the Standard Model: the three factors transform independently.

```
noncomputable def repLorentzGroup : Representation ℂ SL(2,ℂ) JetAlgebra
```

lemma repLorentzGroup_tmul [source]

```
@[simp]
lemma repLorentzGroup_tmul (Λ : SL(2,ℂ))
    (w : ExteriorAlgebra ℂ fieldData.FermionGenerators ⊗[ℂ]
      SymmetricAlgebra ℂ fieldData.BosonGenerators)
    (g : ℂ ⊗[ℝ] _root_.LocalGaugeFieldAlgebra GaugeAlgebra) :
    repLorentzGroup Λ (w ⊗ₜ[ℂ] g)
      = ((repLorentzGroupFermion.tprod repLorentzGroupBoson) Λ w)
          ⊗ₜ[ℂ] (_root_.LocalGaugeFieldAlgebra.complexRepLorentzGroup GaugeAlgebra Λ g)
```

lemma repLorentzGroup_apply_mul [source]

The Lorentz action on the jet algebra is multiplicative.

```
lemma repLorentzGroup_apply_mul (Λ : SL(2,ℂ)) (x y : JetAlgebra) :
    repLorentzGroup Λ (x * y) = repLorentzGroup Λ x * repLorentzGroup Λ y
```

lemma complexRepLorentzGroup_apply_one [source]

The Lorentz action on the complexified gauge sector fixes the unit.

```
lemma complexRepLorentzGroup_apply_one (Λ : SL(2,ℂ)) :
    (_root_.LocalGaugeFieldAlgebra.complexRepLorentzGroup GaugeAlgebra) Λ
      (1 : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) = 1
```

lemma repLorentzGroup_matter_one [source]

The matter factor of the Lorentz action fixes the unit, by the same abstract instantiation as in the gauge action.

```
lemma repLorentzGroup_matter_one (Λ : SL(2,ℂ)) :
    (repLorentzGroupFermion.tprod repLorentzGroupBoson) Λ
        (1 : fieldData.MatterAlgebra) = 1
```

lemma repLorentzGroup_includeConnection [source]

The Lorentz action restricts to the generic connection factor.

```
lemma repLorentzGroup_includeConnection (Λ : SL(2,ℂ))
    (y : ℂ ⊗[ℝ] _root_.LocalGaugeFieldAlgebra GaugeAlgebra) :
    repLorentzGroup Λ (fieldData.includeConnection y)
      = fieldData.includeConnection
          (_root_.LocalGaugeFieldAlgebra.complexRepLorentzGroup GaugeAlgebra Λ y)
```

lemma repLorentzGroup_includeFermionFactor [source]

The Lorentz action restricts to the fermionic factor, where it is the exterior-algebra functor applied to the species-wise action of the datum.

```
lemma repLorentzGroup_includeFermionFactor (Λ : SL(2,ℂ))
    (a : ExteriorAlgebra ℂ fieldData.FermionGenerators) :
    repLorentzGroup Λ (fieldData.includeFermion a)
      = fieldData.includeFermion (repLorentzGroupFermion Λ a)
```

lemma repLorentzGroup_includeBosonFactor [source]

The Lorentz action restricts to the bosonic factor, where it is the symmetric-algebra functor applied to the species-wise action of the datum.

```
lemma repLorentzGroup_includeBosonFactor (Λ : SL(2,ℂ))
    (b : SymmetricAlgebra ℂ fieldData.BosonGenerators) :
    repLorentzGroup Λ (fieldData.includeBoson b)
      = fieldData.includeBoson (repLorentzGroupBoson Λ b)
```

lemma repLorentzGroup_includeGauge [source]

The Lorentz action restricts to the gauge sector's own action. The gauge sector inclusion is the connection inclusion of the datum, the Standard Model gauge bosons being the generic ones at `GaugeAlgebra`.

```
lemma repLorentzGroup_includeGauge (Λ : SL(2,ℂ))
    (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    repLorentzGroup Λ (includeGauge y)
      = includeGauge (_root_.LocalGaugeFieldAlgebra.complexRepLorentzGroup GaugeAlgebra Λ y)
```

lemma repLorentzGroup_includeFermion [source]

The Lorentz action restricts to the fermionic sector's own action.

```
lemma repLorentzGroup_includeFermion (Λ : SL(2,ℂ)) (f : FermionJetAlgebra) :
    repLorentzGroup Λ (includeFermion f)
      = includeFermion (FermionJetAlgebra.repLorentzGroup Λ f)
```

lemma repLorentzGroup_includeHiggs [source]

The Lorentz action restricts to the Higgs sector's own action.

```
lemma repLorentzGroup_includeHiggs (Λ : SL(2,ℂ)) (h : HiggsJetAlgebra) :
    repLorentzGroup Λ (includeHiggs h)
      = includeHiggs (HiggsJetAlgebra.repLorentzGroup Λ h)
```

lemma tprod_deriv_sum [source]

A factorwise sum of Lorentz-vector derivatives on a tensor product is a Lorentz vector: the abstract two-factor assembly, proved by tensor induction at abstract types so that it can be instantiated on the jet algebra without rewriting inside it.

```
private lemma tprod_deriv_sum {M N : Type} [AddCommGroup M] [Module ℂ M]
    [AddCommGroup N] [Module ℂ N]
    (ρ : Representation ℂ SL(2,ℂ) M) (σ : Representation ℂ SL(2,ℂ) N)
```

```
    (D : (Fin 1 ⊕ Fin 3) → M →ₗ[ℂ] M) (E : (Fin 1 ⊕ Fin 3) → N →ₗ[ℂ] N)
    (c : (Fin 1 ⊕ Fin 3) → (Fin 1 ⊕ Fin 3) → ℂ) (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3)
    (hD : ∀ ν x, ρ Λ (D ν x) = ∑ a, c a ν • D a (ρ Λ x))
    (hE : ∀ ν x, σ Λ (E ν x) = ∑ a, c a ν • E a (σ Λ x)) (x : M ⊗[ℂ] N) :
    (ρ.tprod σ) Λ
        ((TensorProduct.map (D μ) (LinearMap.id (M := N))
          + TensorProduct.map (LinearMap.id (M := M)) (E μ)) x)
      = ∑ a, c a μ •
          (TensorProduct.map (D a) (LinearMap.id (M := N))
            + TensorProduct.map (LinearMap.id (M := M)) (E a))
            ((ρ.tprod σ) Λ x)
```

lemma repLorentzGroup_jetDeriv [source]

The total derivative on the jet algebra is a Lorentz vector.

```
lemma repLorentzGroup_jetDeriv (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) (x : JetAlgebra) :
    repLorentzGroup Λ (jetDeriv μ x) =
      ∑ a, (((Lorentz.SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
        jetDeriv a (repLorentzGroup Λ x)
```

instance instIsLorentzDeriv [source]

The total derivatives on the jet algebra form a Lorentz derivative.

```
instance instIsLorentzDeriv : Lorentz.IsLorentzDeriv repLorentzGroup jetDeriv where
```

2.488 PhyslibAlpha.Particles.StandardModel.JetAlgebra.MassWeightPoly

PhyslibAlpha/Particles/StandardModel/JetAlgebra/MassWeightPoly.lean on GitHub.

lemma commute_monomial [source]

Two monomials with commuting coefficients commute: the variable is central, so the two products are the same monomial.

```
lemma commute_monomial {C : Type*} [Semiring C] {a b : C} (h : Commute a b)
    (n m : ℕ) : Commute (Polynomial.monomial n a) (Polynomial.monomial m b)
```

lemma commute_mapAlgHom [source]

Polynomials pushed forward along two algebra maps with commuting images commute: every polynomial is a sum of monomials, and monomials with commuting coefficients commute.

```
lemma commute_mapAlgHom {A B C : Type*} [Semiring A] [Algebra ℂ A] [Semiring B]
    [Algebra ℂ B] [Semiring C] [Algebra ℂ C] (f : A →ₐ[ℂ] C) (g : B →ₐ[ℂ] C)
    (h : ∀ (a : A) (b : B), Commute (f a) (g b)) (p : Polynomial A) (q : Polynomial B) :
    Commute (Polynomial.mapAlgHom f p) (Polynomial.mapAlgHom g q)
```

lemma commute_mapAlgHom_includeGauge [source]

Every polynomial over the jet algebra commutes with a polynomial whose coefficients lie in the gauge sector: the gauge sector is central, so the commutation holds coefficient by coefficient.

```
lemma commute_mapAlgHom_includeGauge (p : Polynomial JetAlgebra)
    (q : Polynomial (ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra))) :
    Commute p (Polynomial.mapAlgHom includeGauge q)
```

def fermionMassWeightPoly [source]

The fermionic mass-weight grading, transported into the full jet algebra. The fermionic symbols have mass dimension `3/2`, hence mass weight three.

```
noncomputable def fermionMassWeightPoly :
    FermionJetAlgebra →ₐ[ℂ] Polynomial JetAlgebra
```

def higgsMassWeightPoly [source]

The Higgs mass-weight grading, transported into the full jet algebra. The Higgs symbols have mass dimension one, hence mass weight two.

```
noncomputable def higgsMassWeightPoly : HiggsJetAlgebra →ₐ[ℂ] Polynomial JetAlgebra
```

def gaugeMassWeightPoly [source]

The gauge-boson mass-weight grading, transported into the full jet algebra. The gauge symbols have mass dimension one, hence mass weight two.

```
noncomputable def gaugeMassWeightPoly :
    (ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) →ₐ[ℂ] Polynomial JetAlgebra
```

def fermionFactorMassWeightPoly [source]

The fermionic grading read on the fermionic factor of the carrier: the fermionic sector grading, precomposed with the sector equivalence. The sector helper above keeps its own domain, the fermionic sector algebra; this is the map the carrier's factor needs.

```
noncomputable def fermionFactorMassWeightPoly :
    ExteriorAlgebra ℂ fieldData.FermionGenerators →ₐ[ℂ] Polynomial JetAlgebra
```

def bosonFactorMassWeightPoly [source]

The Higgs grading read on the bosonic factor of the carrier.

```
noncomputable def bosonFactorMassWeightPoly :
    SymmetricAlgebra ℂ fieldData.BosonGenerators →ₐ[ℂ] Polynomial JetAlgebra
```

lemma fermionFactorMassWeightPoly_apply [source]

```
lemma fermionFactorMassWeightPoly_apply (a : ExteriorAlgebra ℂ fieldData.FermionGenerators) :
    fermionFactorMassWeightPoly a
      = Polynomial.mapAlgHom includeFermion
        (FermionicAlgebra.massWeightPoly 3 (fermionAlgebraEquiv.symm a))
```

lemma bosonFactorMassWeightPoly_apply [source]

```
lemma bosonFactorMassWeightPoly_apply (b : SymmetricAlgebra ℂ fieldData.BosonGenerators) :
    bosonFactorMassWeightPoly b
      = Polynomial.mapAlgHom includeHiggs
        (BosonicAlgebra.massWeightPoly 2 (higgsAlgebraEquiv.symm b))
```

def matterMassWeightPoly [source]

The mass-weight grading on the matter factor of the jet algebra: the fermionic and Higgs gradings, lifted over the tensor product of the two matter factors of the carrier. The side condition is that the two images commute, which they do because the Higgs sector commutes with the fermionic sector.

```
noncomputable def matterMassWeightPoly :
    fieldData.MatterAlgebra →ₐ[ℂ] Polynomial JetAlgebra
```

def massWeightPoly [source]

The mass-weight polynomial on the jet algebra of the Standard Model: the ℂ-algebra map sending a generator of mass weight `n` to `X ^ n` times itself, so that the coefficient of `X ^ n` in `massWeightPoly a` is the part of `a` of mass weight `n`. It is the three sector gradings lifted over the tensor product, the side condition for the outer lift being the centrality of the gauge sector.

```
noncomputable def massWeightPoly : JetAlgebra →ₐ[ℂ] Polynomial JetAlgebra
```

lemma massWeightPoly_tmul [source]

On a pure tensor the grading is the product of the matter and gauge gradings.

```
lemma massWeightPoly_tmul (x : fieldData.MatterAlgebra)
    (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    massWeightPoly (x ⊗ₜ[ℂ] y) = matterMassWeightPoly x * gaugeMassWeightPoly y
```

lemma matterMassWeightPoly_tmul [source]

On a pure tensor the matter grading is the product of the fermionic and bosonic factor gradings.

```
lemma matterMassWeightPoly_tmul (a : ExteriorAlgebra ℂ fieldData.FermionGenerators)
    (b : SymmetricAlgebra ℂ fieldData.BosonGenerators) :
    matterMassWeightPoly (a ⊗ₜ[ℂ] b)
      = fermionFactorMassWeightPoly a * bosonFactorMassWeightPoly b
```

lemma massWeightPoly_includeFermionFactor [source]

On the fermionic factor the grading is that factor's own grading. Like every step below it is written as an equation chain, which never abstracts a pattern out of a goal mentioning the jet algebra.

```
lemma massWeightPoly_includeFermionFactor
    (a : ExteriorAlgebra ℂ fieldData.FermionGenerators) :
    massWeightPoly (fieldData.includeFermion a) = fermionFactorMassWeightPoly a
```

lemma massWeightPoly_includeBosonFactor [source]

On the bosonic factor the grading is that factor's own grading.

```
lemma massWeightPoly_includeBosonFactor
    (b : SymmetricAlgebra ℂ fieldData.BosonGenerators) :
    massWeightPoly (fieldData.includeBoson b) = bosonFactorMassWeightPoly b
```

lemma massWeightPoly_includeConnection [source]

On the connection factor the grading is the generic gauge-boson grading.

```
lemma massWeightPoly_includeConnection (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    massWeightPoly (fieldData.includeConnection y) = gaugeMassWeightPoly y
```

lemma massWeightPoly_includeFermion [source]

On the fermionic sector the grading is the fermionic sector's own grading, pushed forward along the fermionic inclusion.

```
lemma massWeightPoly_includeFermion (a : FermionJetAlgebra) :
    massWeightPoly (includeFermion a)
      = Polynomial.mapAlgHom includeFermion (FermionicAlgebra.massWeightPoly 3 a)
```

lemma massWeightPoly_includeHiggs [source]

On the Higgs sector the grading is the Higgs sector's own grading, pushed forward along the Higgs inclusion.

```
lemma massWeightPoly_includeHiggs (h : HiggsJetAlgebra) :
    massWeightPoly (includeHiggs h)
      = Polynomial.mapAlgHom includeHiggs (BosonicAlgebra.massWeightPoly 2 h)
```

lemma massWeightPoly_includeGauge [source]

On the gauge sector the grading is the gauge sector's own grading, pushed forward along the gauge inclusion. The gauge sector inclusion is the connection inclusion of the datum, so there is nothing to transport here.

```
lemma massWeightPoly_includeGauge (y : ℂ ⊗[ℝ] (LocalGaugeFieldAlgebra GaugeAlgebra)) :
    massWeightPoly (includeGauge y)
      = Polynomial.mapAlgHom includeGauge (LocalGaugeFieldAlgebra.complexMassWeightPoly y)
```

lemma monomial_two_mul_one_add [source]

The mass-weight exponent of a symbol of mass dimension one, in the two forms the statements below use: `AlgebraRealization` asks for `2 * (1 + |s|)`, and each sector grading produces `2 + 2 * |s|`.

```
private lemma monomial_two_mul_one_add (n : ℕ) (x : JetAlgebra) :
    Polynomial.monomial (2 + 2 * n) x = Polynomial.monomial (2 * (1 + n)) x
```

lemma massWeightPoly_higgsField [source]

The Higgs symbol `∂_s H_φ` is a monomial eigenvector of mass weight `2 * (1 + |s|)`: the Higgs field has mass dimension one and each derivative adds one more.

```
lemma massWeightPoly_higgsField (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ HiggsVec) :
    massWeightPoly (higgsField s φ)
      = Polynomial.monomial (2 * (1 + Multiset.card s)) (higgsField s φ)
```

lemma massWeightPoly_conjHiggsField [source]

The conjugate Higgs symbol `∂_s H⁻_φ` is a monomial eigenvector of the same mass weight `2 * (1 + |s|)` as the symbol it conjugates.

```
lemma massWeightPoly_conjHiggsField (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    massWeightPoly (conjHiggsField s φ)
      = Polynomial.monomial (2 * (1 + Multiset.card s)) (conjHiggsField s φ)
```

lemma massWeightPoly_gaugeField [source]

The gauge-field symbol `∂_s A_μ^φ` is a monomial eigenvector of mass weight `2 * (1 + |s|)`: the gauge field has mass dimension one and each derivative adds one more.

```
lemma massWeightPoly_gaugeField (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) :
    massWeightPoly (gaugeField s μ φ)
      = Polynomial.monomial (2 * (1 + Multiset.card s)) (gaugeField s μ φ)
```

lemma massWeightPoly_fermionSymbol [source]

A fermionic symbol `∂_s ψ_φ` on the total fermionic target space is a monomial eigenvector of mass weight `3 + 2 |s|`: a fermion has mass dimension `3/2` and each derivative adds one more.

```
lemma massWeightPoly_fermionSymbol (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ FermionSpace) :
    massWeightPoly (fermionSymbol s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (fermionSymbol s φ)
```

lemma massWeightPoly_conjFermionSymbol [source]

A conjugate fermionic symbol `∂_s ψ⁻_φ` on the total fermionic target space is a monomial eigenvector of the same mass weight `3 + 2 |s|` as the symbol it conjugates.

```
lemma massWeightPoly_conjFermionSymbol (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule FermionSpace)) :
    massWeightPoly (conjFermionSymbol s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (conjFermionSymbol s φ)
```

lemma massWeightPoly_leptonDoubletField [source]

The symbol `∂_s ψ_φ` of the `i`-th generation lepton doublet is a monomial eigenvector of mass weight `3 + 2 |s|`.

```
lemma massWeightPoly_leptonDoubletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonDoubletV) :
    massWeightPoly (leptonDoubletField i s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (leptonDoubletField i s φ)
```

lemma massWeightPoly_conjLeptonDoubletField [source]

The conjugate symbol `∂_s ψ̄_φ` of the `i`-th generation lepton doublet is a monomial eigenvector of mass weight `3 + 2 |s|`.

```
lemma massWeightPoly_conjLeptonDoubletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    massWeightPoly (conjLeptonDoubletField i s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (conjLeptonDoubletField i s φ)
```

lemma massWeightPoly_leptonSingletField [source]

The symbol `∂_s ψ_φ` of the `i`-th generation charged-lepton singlet is a monomial eigenvector of mass weight `3 + 2 |s|`.

```
lemma massWeightPoly_leptonSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonSinglet) :
    massWeightPoly (leptonSingletField i s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (leptonSingletField i s φ)
```

lemma massWeightPoly_conjLeptonSingletField [source]

The conjugate symbol `∂_s ψ̄_φ` of the `i`-th generation charged-lepton singlet is a monomial eigenvector of mass weight `3 + 2 |s|`.

```
lemma massWeightPoly_conjLeptonSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    massWeightPoly (conjLeptonSingletField i s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (conjLeptonSingletField i s φ)
```

lemma massWeightPoly_quarkDoubletField [source]

The symbol `∂_s ψ_φ` of the `i`-th generation quark doublet is a monomial eigenvector of mass weight `3 + 2 |s|`.

```
lemma massWeightPoly_quarkDoubletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet) :
    massWeightPoly (quarkDoubletField i s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (quarkDoubletField i s φ)
```

lemma massWeightPoly_conjQuarkDoubletField [source]

The conjugate symbol `∂_s ψ̄_φ` of the `i`-th generation quark doublet is a monomial eigenvector of mass weight `3 + 2 |s|`.

```
lemma massWeightPoly_conjQuarkDoubletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    massWeightPoly (conjQuarkDoubletField i s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (conjQuarkDoubletField i s φ)
```

lemma massWeightPoly_upSingletField [source]

The symbol `∂_s ψ_φ` of the `i`-th generation up-type quark singlet is a monomial eigenvector of mass weight `3 + 2 |s|`.

```
lemma massWeightPoly_upSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet) :
    massWeightPoly (upSingletField i s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (upSingletField i s φ)
```

lemma massWeightPoly_conjUpSingletField [source]

The conjugate symbol `∂_s ψ̄_φ` of the `i`-th generation up-type quark singlet is a monomial eigenvector of mass weight `3 + 2 |s|`.

```
lemma massWeightPoly_conjUpSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    massWeightPoly (conjUpSingletField i s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (conjUpSingletField i s φ)
```

lemma massWeightPoly_downSingletField [source]

The symbol `∂_s ψ_φ` of the `i`-th generation down-type quark singlet is a monomial eigenvector of mass weight `3 + 2 |s|`.

```
lemma massWeightPoly_downSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) :
    massWeightPoly (downSingletField i s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (downSingletField i s φ)
```

lemma massWeightPoly_conjDownSingletField [source]

The conjugate symbol `∂_s ψ̄_φ` of the `i`-th generation down-type quark singlet is a monomial eigenvector of mass weight `3 + 2 |s|`.

```
lemma massWeightPoly_conjDownSingletField (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    massWeightPoly (conjDownSingletField i s φ)
      = Polynomial.monomial (3 + 2 * Multiset.card s) (conjDownSingletField i s φ)
```

2.489 PhyslibAlpha.Particles.StandardModel.JetAlgebra.Realization

PhyslibAlpha/Particles/StandardModel/JetAlgebra/Realization.lean on GitHub.

def id [source]

The jet algebra of the Standard Model is a Standard Model: it is one along the identity algebra map.

```
noncomputable def id : AlgebraRealization JetAlgebra JetAlgebra.repJetGaugeGroupI
    JetAlgebra.repLorentzGroup JetAlgebra.massWeightPoly where
```

2.490 PhyslibAlpha.Particles.StandardModel.JetAlgebra.SectorEquiv.Basic

PhyslibAlpha/Particles/StandardModel/JetAlgebra/SectorEquiv/Basic.lean on GitHub.

def fermionSpaceEquiv [source]

The total fermionic target space is the fermionic module of the datum. The nested product of five three-generation blocks that `FermionSpace` is, rearranged into a dependent function on the fifteen species. It is a relabelling: every component of a value on one side is a component of the corresponding value on the other.

```
noncomputable def fermionSpaceEquiv :
    fermionMatterField.V ≃ₗ[ℂ] (fieldData.fermionMatterField 3 fieldData_fermion_massWeight).V
    where
```

def fermionProj [source]

The projection of the total fermionic target space onto the value space of a species of the datum: the relabelling followed by the projection of the module.

```
noncomputable def fermionProj (t : fieldData.FermionSpecies) :
    fermionMatterField.V →ₗ[ℂ] (fieldData.fermion t).V
```

lemma fermionProj_eq [source]

```
lemma fermionProj_eq (t : fieldData.FermionSpecies) :
    fermionProj t = (fieldData.projFermionField 3 fieldData_fermion_massWeight t).comp
      fermionSpaceEquiv.toLinearMap
```

lemma fermionProj_leptonDoublet [source]

```
@[simp]
lemma fermionProj_leptonDoublet (i : Fin 3) :
    fermionProj (.leptonDoublet i) = FermionSpace.leptonDoubletProj i
```

lemma fermionProj_leptonSinglet [source]

```
@[simp]
lemma fermionProj_leptonSinglet (i : Fin 3) :
    fermionProj (.leptonSinglet i) = FermionSpace.leptonSingletProj i
```

lemma fermionProj_quarkDoublet [source]

```
@[simp]
lemma fermionProj_quarkDoublet (i : Fin 3) :
    fermionProj (.quarkDoublet i) = FermionSpace.quarkDoubletProj i
```

lemma fermionProj_upSinglet [source]

```
@[simp]
lemma fermionProj_upSinglet (i : Fin 3) :
    fermionProj (.upSinglet i) = FermionSpace.upSingletProj i
```

lemma fermionProj_downSinglet [source]

```
@[simp]
lemma fermionProj_downSinglet (i : Fin 3) :
```

```
    fermionProj (.downSinglet i) = FermionSpace.downSingletProj i
```

def higgsModuleEquiv [source]

The Higgs multiplet is the bosonic module of the datum. There is one bosonic species, so the module of bosonic values is the constant family on it.

```
noncomputable def higgsModuleEquiv :
    HiggsVec.matterField.V ≃ₗ[ℂ] (fieldData.bosonMatterField 2 fieldData_boson_massWeight).V
    where
```

lemma projBosonValue_comp_higgsModuleEquiv [source]

Reading off the one species undoes the relabelling.

```
lemma projBosonValue_comp_higgsModuleEquiv (j : fieldData.BosonSpecies) :
    (fieldData.projBosonField 2 fieldData_boson_massWeight j).comp
        higgsModuleEquiv.toLinearMap = LinearMap.id
```

lemma comap_projBosonValue_comp_comap_higgsModuleEquiv [source]

The pullback along the relabelling undoes the pullback along the one projection.

```
lemma comap_projBosonValue_comp_comap_higgsModuleEquiv (j : fieldData.BosonSpecies) :
    (JetComponentSpace.comap higgsModuleEquiv.toLinearMap).comp
        (JetComponentSpace.comap (fieldData.projBosonField 2 fieldData_boson_massWeight j))
      = LinearMap.id
```

def fermionGeneratorsEquiv [source]

The fermionic generator space of the datum is the component space of the total fermionic target space. The generic presentation of the direct sum as the component space of the fermionic module, followed by the relabelling of the target space.

```
noncomputable def fermionGeneratorsEquiv :
    fieldData.FermionGenerators ≃ₗ[ℂ] JetComponentSpace fermionMatterField
```

lemma fermionGeneratorsEquiv_inclFermion [source]

A species sits inside the fermionic generators as the pullback along the projection onto that species.

```
@[simp]
lemma fermionGeneratorsEquiv_inclFermion (t : fieldData.FermionSpecies)
    (x : JetComponentSpace (fieldData.fermion t)) :
    fermionGeneratorsEquiv (fieldData.inclFermion t x)
      = JetComponentSpace.comap (fermionProj t) x
```

lemma fermionGeneratorsEquiv_symm_comap [source]

```
@[simp]
lemma fermionGeneratorsEquiv_symm_comap (t : fieldData.FermionSpecies)
    (x : JetComponentSpace (fieldData.fermion t)) :
    fermionGeneratorsEquiv.symm (JetComponentSpace.comap (fermionProj t) x)
```

```
    = fieldData.inclFermion t x
```

lemma comap_snd_of_zero [source]

The second half of a pullback of an unconjugated symbol vanishes.

```
lemma _root_.JetComponentSpace.comap_snd_of_zero {G₀ : Type} [Group G₀] {𝔤 : Type} [LieRing 𝔤]
    [LieAlgebra ℝ 𝔤]
    {GJ : Type} [Group GJ] {𝔤J : Type} [LieRing 𝔤J] [LieAlgebra ℝ 𝔤J]
    {jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J} {M N : MatterField jets}
    (f : M.V →ₗ[ℂ] N.V) (x : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ N.V) :
    (JetComponentSpace.comap f ((x, 0) : JetComponentSpace N)).2 = 0
```

lemma comap_fst_of_zero [source]

The first half of a pullback of a conjugate symbol vanishes.

```
lemma _root_.JetComponentSpace.comap_fst_of_zero {G₀ : Type} [Group G₀] {𝔤 : Type} [LieRing 𝔤]
    [LieAlgebra ℝ 𝔤]
    {GJ : Type} [Group GJ] {𝔤J : Type} [LieRing 𝔤J] [LieAlgebra ℝ 𝔤J]
    {jets : LocalGaugeData G₀ 𝔤 GJ 𝔤J} {M N : MatterField jets}
    (f : M.V →ₗ[ℂ] N.V) (y : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule N.V)) :
    (JetComponentSpace.comap f ((0, y) : JetComponentSpace N)).1 = 0
```

lemma fermionGeneratorsEquiv_symm_basis_tmul [source]

The unconjugated symbol $\partial_s\ \psi_\varphi$ of a species, read on the total target space.

```
lemma fermionGeneratorsEquiv_symm_basis_tmul (t : fieldData.FermionSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (fieldData.FermionValue t)) :
    fermionGeneratorsEquiv.symm ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (fermionProj t) φ, 0) : JetComponentSpace fermionMatterField)
      = fieldData.inclFermion t ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) :
          JetComponentSpace (fieldData.fermion t))
```

lemma fermionGeneratorsEquiv_symm_basis_tmul_conj [source]

The conjugate symbol $\partial_s\ \bar{\psi}_\varphi$ of a species, read on the total target space.

```
lemma fermionGeneratorsEquiv_symm_basis_tmul_conj (t : fieldData.FermionSpecies)
    (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule (fieldData.FermionValue t))) :
    fermionGeneratorsEquiv.symm ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ]
        Module.Dual.transpose (ConjModule.map (fermionProj t)) φ) :
          JetComponentSpace fermionMatterField)
      = fieldData.inclFermion t ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) :
          JetComponentSpace (fieldData.fermion t))
```

def bosonGeneratorsEquiv [source]

The bosonic generator space of the datum is the component space of the Higgs. There is one bosonic species, so the direct sum has one summand and the relabelling of the target space is the identification of a one-element function space with its value.

```
noncomputable def bosonGeneratorsEquiv :
    fieldData.BosonGenerators ≃ₗ[ℂ] JetComponentSpace HiggsVec.matterField
```

lemma bosonGeneratorsEquiv_inclBoson [source]

```
lemma bosonGeneratorsEquiv_inclBoson (j : fieldData.BosonSpecies)
    (y : JetComponentSpace (fieldData.boson j)) :
    bosonGeneratorsEquiv (fieldData.inclBoson j y) = y
```

lemma bosonGeneratorsEquiv_symm_apply [source]

```
@[simp]
lemma bosonGeneratorsEquiv_symm_apply (y : JetComponentSpace HiggsVec.matterField) :
    bosonGeneratorsEquiv.symm y = fieldData.inclBoson () y
```

def fermionAlgebraEquiv [source]

The fermionic factor: one exterior algebra, over the two presentations of the same generator space.

```
noncomputable def fermionAlgebraEquiv :
    FermionJetAlgebra ≃ₐ[ℂ] ExteriorAlgebra ℂ fieldData.FermionGenerators
```

def higgsAlgebraEquiv [source]

The Higgs factor: one symmetric algebra, over the two presentations of the same generator space.

```
noncomputable def higgsAlgebraEquiv :
    HiggsJetAlgebra ≃ₐ[ℂ] SymmetricAlgebra ℂ fieldData.BosonGenerators
```

lemma fermionAlgebraEquiv_ι [source]

```
@[simp]
lemma fermionAlgebraEquiv_ι (v : JetComponentSpace fermionMatterField) :
    fermionAlgebraEquiv (ExteriorAlgebra.ι ℂ v)
      = ExteriorAlgebra.ι ℂ (fermionGeneratorsEquiv.symm v)
```

lemma higgsAlgebraEquiv_ι [source]

```
@[simp]
lemma higgsAlgebraEquiv_ι (v : JetComponentSpace HiggsVec.matterField) :
    higgsAlgebraEquiv (SymmetricAlgebra.ι ℂ (JetComponentSpace HiggsVec.matterField) v)
      = SymmetricAlgebra.ι ℂ fieldData.BosonGenerators (bosonGeneratorsEquiv.symm v)
```

lemma includeFermion_ι [source]

A degree-one element of the fermionic factor, included, is a total fermionic generator.

```
lemma includeFermion_ι (w : fieldData.FermionGenerators) :
    fieldData.includeFermion (ExteriorAlgebra.ι ℂ w) = fieldData.ιFermionTotal w
```

lemma includeBoson_ι [source]

A degree-one element of the bosonic factor, included, is a total bosonic generator.

```
lemma includeBoson_ι (w : fieldData.BosonGenerators) :
    fieldData.includeBoson (SymmetricAlgebra.ι ℂ fieldData.BosonGenerators w)
      = fieldData.ιBosonTotal w
```

lemma includeConnection_one_tmul_ι [source]

A real degree-one element of the connection factor, included, is a connection generator.

```
lemma includeConnection_one_tmul_ι (v : GaugeBoson.JetComponentSpace GaugeAlgebra) :
    fieldData.includeConnection ((1 : ℂ) ⊗ₜ[ℝ]
        SymmetricAlgebra.ι ℝ (GaugeBoson.JetComponentSpace GaugeAlgebra) v)
      = fieldData.ιConnection v
```

lemma ιFermionTotal_inclFermion [source]

The total fermionic generator of a species summand is that species' generator.

```
lemma ιFermionTotal_inclFermion (t : fieldData.FermionSpecies)
    (x : JetComponentSpace (fieldData.fermion t)) :
    fieldData.ιFermionTotal (fieldData.inclFermion t x) = fieldData.ιFermion t x
```

lemma ιBosonTotal_inclBoson [source]

The total bosonic generator of a species summand is that species' generator.

```
lemma ιBosonTotal_inclBoson (j : fieldData.BosonSpecies)
    (y : JetComponentSpace (fieldData.boson j)) :
    fieldData.ιBosonTotal (fieldData.inclBoson j y) = fieldData.ιBoson j y
```

lemma fermionGeneratorsEquiv_jetDerivFermion [source]

The fermionic generator identification intertwines the derivative shift of the datum with the derivative shift on the component space of the total target space.

```
lemma fermionGeneratorsEquiv_jetDerivFermion (μ : Fin 1 ⊕ Fin 3)
    (w : fieldData.FermionGenerators) :
    fermionGeneratorsEquiv (fieldData.jetDerivFermion μ w)
      = JetComponentSpace.jetDeriv μ (fermionGeneratorsEquiv w)
```

lemma fermionGeneratorsEquiv_symm_jetDeriv [source]

The inverse form of `fermionGeneratorsEquiv_jetDerivFermion`.

```
lemma fermionGeneratorsEquiv_symm_jetDeriv (μ : Fin 1 ⊕ Fin 3)
    (v : JetComponentSpace fermionMatterField) :
    fermionGeneratorsEquiv.symm (JetComponentSpace.jetDeriv μ v)
      = fieldData.jetDerivFermion μ (fermionGeneratorsEquiv.symm v)
```

lemma bosonGeneratorsEquiv_jetDerivBoson [source]

The bosonic generator identification intertwines the two derivative shifts.

```
lemma bosonGeneratorsEquiv_jetDerivBoson (μ : Fin 1 ⊕ Fin 3)
    (w : fieldData.BosonGenerators) :
    bosonGeneratorsEquiv (fieldData.jetDerivBoson μ w)
      = JetComponentSpace.jetDeriv μ (bosonGeneratorsEquiv w)
```

lemma bosonGeneratorsEquiv_symm_jetDeriv [source]

The inverse form of `bosonGeneratorsEquiv_jetDerivBoson`.

```
lemma bosonGeneratorsEquiv_symm_jetDeriv (μ : Fin 1 ⊕ Fin 3)
    (v : JetComponentSpace HiggsVec.matterField) :
    bosonGeneratorsEquiv.symm (JetComponentSpace.jetDeriv μ v)
      = fieldData.jetDerivBoson μ (bosonGeneratorsEquiv.symm v)
```

lemma fermionicAlgebra_jetDeriv_eq [source]

The total derivative on the fermionic algebra of a species is the general even derivation of its exterior algebra extending the derivative shift. The two constructions are the same lift into the trivial square-zero extension, written once in the matter sector and once in general; the identification lets the general theory apply to the fermionic factor of the migrated carrier, whose generator space is a direct sum of component spaces rather than a single one.

```
private lemma fermionicAlgebra_jetDeriv_eq (M : MatterField localGaugeData)
    (μ : Fin 1 ⊕ Fin 3) :
    FermionicAlgebra.jetDeriv (M := M) μ
      = ExteriorAlgebra.derivationOfLinear (JetComponentSpace.jetDeriv μ)
```

lemma fermionAlgebraEquiv_jetDeriv [source]

The fermionic sector equivalence is a map of differential algebras: the exterior derivation extending the derivative shift on the total target space goes to the exterior derivation extending the derivative shift of the datum.

```
lemma fermionAlgebraEquiv_jetDeriv (μ : Fin 1 ⊕ Fin 3) (f : FermionJetAlgebra) :
    fermionAlgebraEquiv (FermionicAlgebra.jetDeriv μ f)
      = ExteriorAlgebra.derivationOfLinear (fieldData.jetDerivFermion μ)
          (fermionAlgebraEquiv f)
```

lemma higgsAlgebraEquiv_jetDeriv [source]

The Higgs sector equivalence is a map of differential algebras.

```
lemma higgsAlgebraEquiv_jetDeriv (μ : Fin 1 ⊕ Fin 3) (h : HiggsJetAlgebra) :
    higgsAlgebraEquiv (BosonicAlgebra.jetDeriv μ h)
      = SymmetricAlgebra.derivationOfLinear (fieldData.jetDerivBoson μ)
          (higgsAlgebraEquiv h)
```

2.491 PhyslibAlpha.Particles.StandardModel.JetAlgebra.SectorEquiv.Structure

PhyslibAlpha/Particles/StandardModel/JetAlgebra/SectorEquiv/Structure.lean on GitHub.

lemma fermionProj_comp_repLorentzGroup [source]

The projection onto a species intertwines the total Lorentz action on the fermionic target space with that species' own.

```
lemma fermionProj_comp_repLorentzGroup (t : fieldData.FermionSpecies) (Λ : SL(2,ℂ)) :
    (fermionProj t).comp (FermionSpace.repLorentzGroup Λ)
      = ((fieldData.fermion t).repLorentz Λ).comp (fermionProj t)
```

lemma fermionSpaceEquiv_comp_repLorentzGroup [source]

The relabelling of the fermionic target space is Lorentz-equivariant. Both actions are species-diagonal, so this is the previous lemma read one species at a time.

```
lemma fermionSpaceEquiv_comp_repLorentzGroup (Λ : SL(2,ℂ)) :
    fermionSpaceEquiv.toLinearMap.comp (FermionSpace.repLorentzGroup Λ)
      = (fieldData.repLorentzFermionModule Λ).comp fermionSpaceEquiv.toLinearMap
```

lemma higgsModuleEquiv_comp_repLorentzGroup [source]

The relabelling of the Higgs is Lorentz-equivariant. The Higgs is a Lorentz scalar, so both sides are the identity; the content is that the one bosonic species of the datum carries exactly the trivial Higgs representation.

```
lemma higgsModuleEquiv_comp_repLorentzGroup (Λ : SL(2,ℂ)) :
    higgsModuleEquiv.toLinearMap.comp (Representation.trivial ℂ SL(2,ℂ) HiggsVec Λ)
      = (fieldData.repLorentzBosonModule Λ).comp higgsModuleEquiv.toLinearMap
```

lemma lTensor_fermionProj_repJetGaugeGroupI [source]

The projection onto a species intertwines the total jet gauge action on the jets of the fermionic target space with that species' own.

```
lemma lTensor_fermionProj_repJetGaugeGroupI (t : fieldData.FermionSpecies)
    (U : JetGaugeGroupI) :
    (LinearMap.lTensor SpaceTimeAlgebra (fermionProj t)).comp (fermionMatterField.repJet U)
      = ((fieldData.fermion t).repJet U).comp
        (LinearMap.lTensor SpaceTimeAlgebra (fermionProj t))
```

lemma lTensor_fermionSpaceEquiv_repJetGaugeGroupI [source]

The relabelling of the fermionic target space is equivariant for the jet gauge action. Both actions are species-diagonal, so this is the previous lemma read one species at a time, the two being compared through the splitting of the jets of a product.

```
lemma lTensor_fermionSpaceEquiv_repJetGaugeGroupI (U : JetGaugeGroupI) :
    (LinearMap.lTensor SpaceTimeAlgebra fermionSpaceEquiv.toLinearMap).comp
        (fermionMatterField.repJet U)
      = ((fieldData.fermionMatterField 3 fieldData_fermion_massWeight).repJet U).comp
        (LinearMap.lTensor SpaceTimeAlgebra fermionSpaceEquiv.toLinearMap)
```

lemma lTensor_higgsModuleEquiv_repJetGaugeGroupI [source]

The relabelling of the Higgs is equivariant for the jet gauge action. There is one bosonic species, and reading it off undoes the relabelling, so both sides are the Higgs

action itself; the content is that the single bosonic species of the datum carries exactly the Higgs representation.

```
lemma lTensor_higgsModuleEquiv_repJetGaugeGroupI (U : JetGaugeGroupI) :
    (LinearMap.lTensor SpaceTimeAlgebra higgsModuleEquiv.toLinearMap).comp
        (HiggsVec.matterField.repJet U)
      = ((fieldData.bosonMatterField 2 fieldData_boson_massWeight).repJet U).comp
        (LinearMap.lTensor SpaceTimeAlgebra higgsModuleEquiv.toLinearMap)
```

lemma fermionGeneratorsEquiv_repLorentzFermion [source]

The fermionic generator identification intertwines the Lorentz actions: the species-diagonal action of the datum with the Standard Model action on the component space of the total fermionic target space.

```
lemma fermionGeneratorsEquiv_repLorentzFermion (Λ : SL(2,ℂ))
    (v : fieldData.FermionGenerators) :
    fermionGeneratorsEquiv (fieldData.repLorentzFermion Λ v)
      = JetComponentSpace.repLorentzGroup fermionMatterField Λ
        (fermionGeneratorsEquiv v)
```

lemma fermionGeneratorsEquiv_comp_repLorentzFermion [source]

The composed form of `fermionGeneratorsEquiv_repLorentzFermion`.

```
lemma fermionGeneratorsEquiv_comp_repLorentzFermion (Λ : SL(2,ℂ)) :
    fermionGeneratorsEquiv.toLinearMap.comp (fieldData.repLorentzFermion Λ)
      = (JetComponentSpace.repLorentzGroup fermionMatterField Λ).comp
        fermionGeneratorsEquiv.toLinearMap
```

lemma bosonGeneratorsEquiv_repLorentzBoson [source]

The bosonic generator identification intertwines the Lorentz actions. The Higgs is a Lorentz scalar, so both sides act only on the derivative labels; the content is that the single bosonic species of the datum carries exactly the trivial Higgs representation of the Standard Model.

```
lemma bosonGeneratorsEquiv_repLorentzBoson (Λ : SL(2,ℂ))
    (w : fieldData.BosonGenerators) :
    bosonGeneratorsEquiv (fieldData.repLorentzBoson Λ w)
      = JetComponentSpace.repLorentzGroup HiggsVec.matterField Λ
        (bosonGeneratorsEquiv w)
```

lemma bosonGeneratorsEquiv_comp_repLorentzBoson [source]

The composed form of `bosonGeneratorsEquiv_repLorentzBoson`.

```
lemma bosonGeneratorsEquiv_comp_repLorentzBoson (Λ : SL(2,ℂ)) :
    bosonGeneratorsEquiv.toLinearMap.comp (fieldData.repLorentzBoson Λ)
      = (JetComponentSpace.repLorentzGroup HiggsVec.matterField
          Λ).comp bosonGeneratorsEquiv.toLinearMap
```

lemma fermionGeneratorsEquiv_symm_repLorentzGroup [source]

The inverse form of `fermionGeneratorsEquiv_repLorentzFermion`.

```
lemma fermionGeneratorsEquiv_symm_repLorentzGroup (Λ : SL(2,ℂ))
    (w : JetComponentSpace fermionMatterField) :
    fermionGeneratorsEquiv.symm
        (JetComponentSpace.repLorentzGroup fermionMatterField Λ w)
      = fieldData.repLorentzFermion Λ (fermionGeneratorsEquiv.symm w)
```

lemma bosonGeneratorsEquiv_symm_repLorentzGroup [source]

The inverse form of `bosonGeneratorsEquiv_repLorentzBoson`.

```
lemma bosonGeneratorsEquiv_symm_repLorentzGroup (Λ : SL(2,ℂ))
    (w : JetComponentSpace HiggsVec.matterField) :
    bosonGeneratorsEquiv.symm (JetComponentSpace.repLorentzGroup HiggsVec.matterField Λ w)
      = fieldData.repLorentzBoson Λ (bosonGeneratorsEquiv.symm w)
```

lemma fermionGeneratorsEquiv_repJetFermion [source]

The fermionic generator identification intertwines the jet gauge actions: the species-diagonal action of the datum with the Standard Model action on the component space of the total fermionic target space. No common mass weight is used.

```
lemma fermionGeneratorsEquiv_repJetFermion (U : JetGaugeGroupI)
    (v : fieldData.FermionGenerators) :
    fermionGeneratorsEquiv (fieldData.repJetFermion U v)
      = JetComponentSpace.repJet fermionMatterField U (fermionGeneratorsEquiv v)
```

lemma fermionGeneratorsEquiv_symm_repJet [source]

The inverse form of `fermionGeneratorsEquiv_repJetFermion`.

```
lemma fermionGeneratorsEquiv_symm_repJet (U : JetGaugeGroupI)
    (w : JetComponentSpace fermionMatterField) :
    fermionGeneratorsEquiv.symm (JetComponentSpace.repJet fermionMatterField U w)
      = fieldData.repJetFermion U (fermionGeneratorsEquiv.symm w)
```

lemma bosonGeneratorsEquiv_repJetBoson [source]

The bosonic generator identification intertwines the jet gauge actions, at the one Higgs species.

```
lemma bosonGeneratorsEquiv_repJetBoson (U : JetGaugeGroupI)
    (v : fieldData.BosonGenerators) :
    bosonGeneratorsEquiv (fieldData.repJetBoson U v)
      = JetComponentSpace.repJet HiggsVec.matterField U (bosonGeneratorsEquiv v)
```

lemma bosonGeneratorsEquiv_symm_repJet [source]

The inverse form of `bosonGeneratorsEquiv_repJetBoson`.

```
lemma bosonGeneratorsEquiv_symm_repJet (U : JetGaugeGroupI)
    (w : JetComponentSpace HiggsVec.matterField) :
    bosonGeneratorsEquiv.symm (JetComponentSpace.repJet HiggsVec.matterField U w)
      = fieldData.repJetBoson U (bosonGeneratorsEquiv.symm w)
```

lemma fermionGeneratorsEquiv_massWeightScaleFermion [source]

The fermionic generator identification intertwines the mass-weight scalings: the per-species scaling of the datum, which carries the weight three of every Standard Model fermion, with the single weight-three scaling of the component space of the total target space. That the two agree is exactly the statement that all fifteen species have the same weight; the relabelling of the target space costs nothing, the scaling being natural in the value space.

```
lemma fermionGeneratorsEquiv_massWeightScaleFermion (c : ℂ)
    (v : fieldData.FermionGenerators) :
    fermionGeneratorsEquiv (fieldData.massWeightScaleFermion c v)
      = JetComponentSpace.massWeightScale 3 c (fermionGeneratorsEquiv v)
```

lemma fermionGeneratorsEquiv_comp_massWeightScaleFermion [source]

The composed form of `fermionGeneratorsEquiv_massWeightScaleFermion`.

```
lemma fermionGeneratorsEquiv_comp_massWeightScaleFermion (c : ℂ) :
    fermionGeneratorsEquiv.toLinearMap.comp (fieldData.massWeightScaleFermion c)
      = (JetComponentSpace.massWeightScale 3 c).comp
        fermionGeneratorsEquiv.toLinearMap
```

lemma bosonGeneratorsEquiv_massWeightScaleBoson [source]

The bosonic generator identification intertwines the mass-weight scalings, at the Higgs weight two.

```
lemma bosonGeneratorsEquiv_massWeightScaleBoson (c : ℂ)
    (w : fieldData.BosonGenerators) :
    bosonGeneratorsEquiv (fieldData.massWeightScaleBoson c w)
      = JetComponentSpace.massWeightScale 2 c (bosonGeneratorsEquiv w)
```

lemma bosonGeneratorsEquiv_comp_massWeightScaleBoson [source]

The composed form of `bosonGeneratorsEquiv_massWeightScaleBoson`.

```
lemma bosonGeneratorsEquiv_comp_massWeightScaleBoson (c : ℂ) :
    bosonGeneratorsEquiv.toLinearMap.comp (fieldData.massWeightScaleBoson c)
      = (JetComponentSpace.massWeightScale 2 c).comp bosonGeneratorsEquiv.toLinearMap
```

lemma fermionAlgebraEquiv_repJetGaugeGroupI [source]

The fermionic sector equivalence intertwines the jet gauge actions.

```
lemma fermionAlgebraEquiv_repJetGaugeGroupI (U : JetGaugeGroupI) (f : FermionJetAlgebra) :
    fermionAlgebraEquiv (FermionJetAlgebra.repJetGaugeGroupI U f)
      = fieldData.repJetFermion.exteriorAlgebra U (fermionAlgebraEquiv f)
```

lemma higgsAlgebraEquiv_repJetGaugeGroupI [source]

The Higgs sector equivalence intertwines the jet gauge actions.

```
lemma higgsAlgebraEquiv_repJetGaugeGroupI (U : JetGaugeGroupI) (h : HiggsJetAlgebra) :
    higgsAlgebraEquiv (HiggsJetAlgebra.repJetGaugeGroupI U h)
      = fieldData.repJetBoson.symmetricAlgebra U (higgsAlgebraEquiv h)
```

lemma fermionAlgebraEquiv_repLorentzGroup [source]

The fermionic sector equivalence intertwines the Lorentz actions.

```
lemma fermionAlgebraEquiv_repLorentzGroup (Λ : SL(2,ℂ)) (f : FermionJetAlgebra) :
    fermionAlgebraEquiv (FermionJetAlgebra.repLorentzGroup Λ f)
      = fieldData.repLorentzFermion.exteriorAlgebra Λ (fermionAlgebraEquiv f)
```

lemma higgsAlgebraEquiv_repLorentzGroup [source]

The Higgs sector equivalence intertwines the Lorentz actions.

```
lemma higgsAlgebraEquiv_repLorentzGroup (Λ : SL(2,ℂ)) (h : HiggsJetAlgebra) :
    higgsAlgebraEquiv (HiggsJetAlgebra.repLorentzGroup Λ h)
      = fieldData.repLorentzBoson.symmetricAlgebra Λ (higgsAlgebraEquiv h)
```

2.492 PhyslibAlpha.Particles.StandardModel.JetAlgebra.TransformsIn

PhyslibAlpha/Particles/StandardModel/JetAlgebra/TransformsIn.lean on GitHub.

lemma transformsIn_higgsField [source]

The Higgs symbols transform in the jet gauge representation carried by the jets of the Higgs field.

```
lemma transformsIn_higgsField :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI HiggsVec.repJetGaugeGroupI
      higgsField
```

lemma transformsIn_conjHiggsField [source]

The conjugate Higgs symbols transform in the conjugate of the jet gauge representation carried by the jets of the Higgs field.

```
lemma transformsIn_conjHiggsField :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI
      (JetComponentSpace.repConj HiggsVec.repJetGaugeGroupI)
      conjHiggsField
```

lemma transformsIn_species [source]

The jet gauge transformation law of a fermion species: a family of symbols obtained from the total fermionic symbols by pulling covectors back along a projection intertwining the two jet gauge actions transforms in the species' own representation.

```
private lemma transformsIn_species {W : Type} [AddCommGroup W] [Module ℂ W]
    (repW : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] W)) (p : FermionSpace →ₗ[ℂ]
    W)
    (hp : ∀ U : JetGaugeGroupI, (LinearMap.lTensor SpaceTimeAlgebra p).comp
        (FermionSpace.repJetGaugeGroupI U)
      = (repW U).comp (LinearMap.lTensor SpaceTimeAlgebra p))
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ W →ₗ[ℂ] JetAlgebra}
    (hF : ∀ s φ, F s φ = fermionSymbol s (Module.Dual.transpose p φ)) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI repW F
```

lemma repDualCoeff_repConj_transpose [source]

The base-point Taylor coefficients of two conjugate jet gauge actions are intertwined, on the component-function index, by the conjugate of any map of value spaces intertwining the unconjugated coefficients: conjugation changes neither the underlying maps nor the real directions in which the coefficients are taken.

```
private lemma repDualCoeff_repConj_transpose {V W : Type} [AddCommGroup V] [Module ℂ V]
    [AddCommGroup W] [Module ℂ W]
    {repV : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] V)}
    {repW : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] W)} (p : V →ₗ[ℂ] W)
    (hp : ∀ (U : JetGaugeGroupI) (s : Multiset (Fin 1 ⊕ Fin 3)),
      p.comp (GaugeAlgebraRealization.repCoeff repV U s)
        = (GaugeAlgebraRealization.repCoeff repW U s).comp p)
    (U : JetGaugeGroupI) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule W)) :
    GaugeAlgebraRealization.repDualCoeff (JetComponentSpace.repConj repV) U s
        (Module.Dual.transpose (ConjModule.map p) φ)
      = Module.Dual.transpose (ConjModule.map p)
          (GaugeAlgebraRealization.repDualCoeff (JetComponentSpace.repConj repW) U s φ)
```

lemma transformsIn_conjSpecies [source]

The jet gauge transformation law of the conjugate symbols of a fermion species: the law of the species itself, read on the conjugate representations.

```
private lemma transformsIn_conjSpecies {W : Type} [AddCommGroup W] [Module ℂ W]
    (repW : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] W)) (p : FermionSpace →ₗ[ℂ]
    W)
    (hp : ∀ U : JetGaugeGroupI, (LinearMap.lTensor SpaceTimeAlgebra p).comp
        (FermionSpace.repJetGaugeGroupI U)
      = (repW U).comp (LinearMap.lTensor SpaceTimeAlgebra p))
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ (ConjModule W) →ₗ[ℂ] JetAlgebra}
    (hF : ∀ s φ, F s φ = conjFermionSymbol s
      (Module.Dual.transpose (ConjModule.map p) φ)) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI
      (JetComponentSpace.repConj repW) F
```

lemma transformsIn_downSingletField [source]

The symbols of the `i`-th generation down-type quark singlet transform in the jet gauge representation carried by the jets of that species.

```
lemma transformsIn_downSingletField (i : Fin 3) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI DownSinglet.
    repJetGaugeGroupI
      (downSingletField i)
```

lemma transformsIn_conjDownSingletField [source]

The conjugate symbols of the `i`-th generation down-type quark singlet transform in the conjugate of the jet gauge representation carried by the jets of that species.

```
lemma transformsIn_conjDownSingletField (i : Fin 3) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI
      (JetComponentSpace.repConj DownSinglet.repJetGaugeGroupI) (conjDownSingletField i)
```

lemma transformsIn_upSingletField [source]

The symbols of the `i`-th generation up-type quark singlet transform in the jet gauge representation carried by the jets of that species.

```
lemma transformsIn_upSingletField (i : Fin 3) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI UpSinglet.repJetGaugeGroupI
      (upSingletField i)
```

lemma transformsIn_conjUpSingletField [source]

The conjugate symbols of the `i`-th generation up-type quark singlet transform in the conjugate of the jet gauge representation carried by the jets of that species.

```
lemma transformsIn_conjUpSingletField (i : Fin 3) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI
      (JetComponentSpace.repConj UpSinglet.repJetGaugeGroupI) (conjUpSingletField i)
```

lemma transformsIn_quarkDoubletField [source]

The symbols of the `i`-th generation quark doublet transform in the jet gauge representation carried by the jets of that species.

```
lemma transformsIn_quarkDoubletField (i : Fin 3) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI QuarkDoublet.
    repJetGaugeGroupI
      (quarkDoubletField i)
```

lemma transformsIn_conjQuarkDoubletField [source]

The conjugate symbols of the `i`-th generation quark doublet transform in the conjugate of the jet gauge representation carried by the jets of that species.

```
lemma transformsIn_conjQuarkDoubletField (i : Fin 3) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI
      (JetComponentSpace.repConj QuarkDoublet.repJetGaugeGroupI) (conjQuarkDoubletField i)
```

lemma transformsIn_leptonDoubletField [source]

The symbols of the `i`-th generation lepton doublet transform in the jet gauge representation carried by the jets of that species.

```
lemma transformsIn_leptonDoubletField (i : Fin 3) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI LeptonDoubletV.
    repJetGaugeGroupI
      (leptonDoubletField i)
```

lemma transformsIn_conjLeptonDoubletField [source]

The conjugate symbols of the `i`-th generation lepton doublet transform in the conjugate of the jet gauge representation carried by the jets of that species.

```
lemma transformsIn_conjLeptonDoubletField (i : Fin 3) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI
      (JetComponentSpace.repConj LeptonDoubletV.repJetGaugeGroupI) (conjLeptonDoubletField i)
```

lemma transformsIn_leptonSingletField [source]

The symbols of the `i`-th generation charged-lepton singlet transform in the jet gauge representation carried by the jets of that species.

```
lemma transformsIn_leptonSingletField (i : Fin 3) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI LeptonSinglet.
    repJetGaugeGroupI
      (leptonSingletField i)
```

lemma transformsIn_conjLeptonSingletField [source]

The conjugate symbols of the `i`-th generation charged-lepton singlet transform in the conjugate of the jet gauge representation carried by the jets of that species.

```
lemma transformsIn_conjLeptonSingletField (i : Fin 3) :
    LocalGaugeData.TransformsIn (B := JetAlgebra) repJetGaugeGroupI
      (JetComponentSpace.repConj LeptonSinglet.repJetGaugeGroupI) (conjLeptonSingletField i)
```

lemma isLorentzDerivTransforms_higgsField [source]

The Higgs symbols transform as the derivative symbols of a Lorentz scalar.

```
lemma isLorentzDerivTransforms_higgsField :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
      (Representation.trivial ℂ SL(2,ℂ) HiggsVec) higgsField
```

lemma isLorentzDerivTransforms_conjHiggsField [source]

The conjugate Higgs symbols transform as the derivative symbols of the conjugate of a Lorentz scalar.

```
lemma isLorentzDerivTransforms_conjHiggsField :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
      (Representation.trivial ℂ SL(2,ℂ) HiggsVec).conj conjHiggsField
```

lemma isLorentzDerivTransforms_species [source]

The Lorentz transformation law of a fermion species: a family of symbols obtained from the total fermionic symbols by pulling covectors back along a projection whose contragredient is species-diagonal transforms in the species' own Weyl representation.

```
private lemma isLorentzDerivTransforms_species {W : Type} [AddCommGroup W] [Module ℂ W]
    (repW : Representation ℂ SL(2,ℂ) W) (p : FermionSpace →ₗ[ℂ] W)
    (hdual : ∀ (Λ : SL(2,ℂ)) (φ : Module.Dual ℂ W),
      FermionSpace.repLorentzGroup.dual Λ (Module.Dual.transpose p φ)
        = Module.Dual.transpose p (repW.dual Λ φ))
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ W →ₗ[ℂ] JetAlgebra}
    (hF : ∀ s φ, F s φ = fermionSymbol s (Module.Dual.transpose p φ)) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup repW F
```

lemma isLorentzDerivTransforms_conjSpecies [source]

The Lorentz transformation law of the conjugate symbols of a fermion species: the law of the species itself, read on the conjugate representations.

```
private lemma isLorentzDerivTransforms_conjSpecies {W : Type} [AddCommGroup W]
    [Module ℂ W] (repW : Representation ℂ SL(2,ℂ) W) (p : FermionSpace →ₗ[ℂ] W)
    (hdual : ∀ (Λ : SL(2,ℂ)) (φ : Module.Dual ℂ (ConjModule W)),
      FermionSpace.repLorentzGroup.conj.dual Λ
          (Module.Dual.transpose (ConjModule.map p) φ)
        = Module.Dual.transpose (ConjModule.map p) (repW.conj.dual Λ φ))
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ (ConjModule W) →ₗ[ℂ] JetAlgebra}
    (hF : ∀ s φ, F s φ = conjFermionSymbol s
      (Module.Dual.transpose (ConjModule.map p) φ)) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup repW.conj F
```

lemma isLorentzDerivTransforms_downSingletField [source]

The symbols of the `i`-th generation down-type quark singlet transform as the derivative symbols of a Weyl spinor in that species' Lorentz representation.

```
lemma isLorentzDerivTransforms_downSingletField (i : Fin 3) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
      DownSinglet.repLorentzGroup (downSingletField i)
```

lemma isLorentzDerivTransforms_conjDownSingletField [source]

The conjugate symbols of the `i`-th generation down-type quark singlet transform as the derivative symbols of the conjugate Weyl spinor.

```
lemma isLorentzDerivTransforms_conjDownSingletField (i : Fin 3) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
      DownSinglet.repLorentzGroup.conj (conjDownSingletField i)
```

lemma isLorentzDerivTransforms_upSingletField [source]

The symbols of the `i`-th generation up-type quark singlet transform as the derivative symbols of a Weyl spinor in that species' Lorentz representation.

```
lemma isLorentzDerivTransforms_upSingletField (i : Fin 3) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
      UpSinglet.repLorentzGroup (upSingletField i)
```

lemma isLorentzDerivTransforms_conjUpSingletField [source]

The conjugate symbols of the `i`-th generation up-type quark singlet transform as the derivative symbols of the conjugate Weyl spinor.

```
lemma isLorentzDerivTransforms_conjUpSingletField (i : Fin 3) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
      UpSinglet.repLorentzGroup.conj (conjUpSingletField i)
```

lemma isLorentzDerivTransforms_quarkDoubletField [source]

The symbols of the `i`-th generation quark doublet transform as the derivative symbols of a Weyl spinor in that species' Lorentz representation.

```
lemma isLorentzDerivTransforms_quarkDoubletField (i : Fin 3) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
```

```
    QuarkDoublet.repLorentzGroup (quarkDoubletField i)
```

lemma isLorentzDerivTransforms_conjQuarkDoubletField [source]

The conjugate symbols of the `i`-th generation quark doublet transform as the derivative symbols of the conjugate Weyl spinor.

```
lemma isLorentzDerivTransforms_conjQuarkDoubletField (i : Fin 3) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
      QuarkDoublet.repLorentzGroup.conj (conjQuarkDoubletField i)
```

lemma isLorentzDerivTransforms_leptonDoubletField [source]

The symbols of the `i`-th generation lepton doublet transform as the derivative symbols of a Weyl spinor in that species' Lorentz representation.

```
lemma isLorentzDerivTransforms_leptonDoubletField (i : Fin 3) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
      LeptonDoubletV.repLorentzGroup (leptonDoubletField i)
```

lemma isLorentzDerivTransforms_conjLeptonDoubletField [source]

The conjugate symbols of the `i`-th generation lepton doublet transform as the derivative symbols of the conjugate Weyl spinor.

```
lemma isLorentzDerivTransforms_conjLeptonDoubletField (i : Fin 3) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
      LeptonDoubletV.repLorentzGroup.conj (conjLeptonDoubletField i)
```

lemma isLorentzDerivTransforms_leptonSingletField [source]

The symbols of the `i`-th generation charged-lepton singlet transform as the derivative symbols of a Weyl spinor in that species' Lorentz representation.

```
lemma isLorentzDerivTransforms_leptonSingletField (i : Fin 3) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
      LeptonSinglet.repLorentzGroup (leptonSingletField i)
```

lemma isLorentzDerivTransforms_conjLeptonSingletField [source]

The conjugate symbols of the `i`-th generation charged-lepton singlet transform as the derivative symbols of the conjugate Weyl spinor.

```
lemma isLorentzDerivTransforms_conjLeptonSingletField (i : Fin 3) :
    IsLorentzDerivTransforms (A := JetAlgebra) repLorentzGroup
      LeptonSinglet.repLorentzGroup.conj (conjLeptonSingletField i)
```

2.493 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.Basic

PhyslibAlpha/Particles/StandardModel/Matter/Bosonic/Basic.lean on GitHub.

abbrev BosonicAlgebra [source]

The bosonic algebra of the matter field `M`: the symmetric algebra on the space of component functions `∂_s φ_α` and `∂_s φ⁻_α`. The symmetric product is the product of bosonic fields, its commutativity the Bose statistics.

```
abbrev BosonicAlgebra (M : MatterField jets) : Type
```

lemma adjoin_ι_eq_top [source]

***The bosonic algebra is generated by the component functions.** Every element is a polynomial in the degree-one elements — the symbols `∂_s φ_α` and `∂_s φ⁻_α` themselves. This is the algebraic form of "every Lagrangian term is a polynomial in the component functions".*

```
lemma adjoin_ι_eq_top :
    Algebra.adjoin ℂ (Set.range (SymmetricAlgebra.ι ℂ (JetComponentSpace M))) = ⊤
```

lemma ι_mul_ι_comm [source]

Two component functions commute: Bose statistics.

```
lemma ι_mul_ι_comm (x y : JetComponentSpace M) :
    (SymmetricAlgebra.ι ℂ _ x * SymmetricAlgebra.ι ℂ _ y : BosonicAlgebra M)
      = SymmetricAlgebra.ι ℂ _ y * SymmetricAlgebra.ι ℂ _ x
```

def ofField [source]

***The component function `ψ_φ` of the matter field** along the covector `φ` on `V`: the undifferentiated generator, sitting at the empty derivative label in the unconjugated half of the component space.*

```
noncomputable def ofField : Module.Dual ℂ M.V →ₗ[ℂ] BosonicAlgebra M
```

def ofConjField [source]

***The component function `ψ⁻_φ` of the conjugate matter field** along the covector `φ` on `ConjModule M.V`: the undifferentiated generator in the conjugate half of the component space.*

```
noncomputable def ofConjField : Module.Dual ℂ (ConjModule M.V) →ₗ[ℂ] BosonicAlgebra M
```

lemma ofField_apply [source]

```
lemma ofField_apply (φ : Module.Dual ℂ M.V) :
    ofField φ = SymmetricAlgebra.ι ℂ _
      (((1 : SpaceTimeDerivAlgebraℂ) ⊗ₜ[ℂ] φ, 0) : JetComponentSpace M)
```

lemma ofConjField_apply [source]

```
lemma ofConjField_apply (φ : Module.Dual ℂ (ConjModule M.V)) :
    ofConjField φ = SymmetricAlgebra.ι ℂ _
      ((0, (1 : SpaceTimeDerivAlgebraℂ) ⊗ₜ[ℂ] φ) : JetComponentSpace M)
```

def comap [source]

***The bosonic algebra is contravariant in the target space.** A linear map `f : V →ₗ[ℂ] W` induces an algebra homomorphism `BosonicAlgebra N →ₐ[ℂ] BosonicAlgebra M` by pulling back component functions. Applied to a *projection* out of a multi-species target space, this is the inclusion of one species' algebra into the whole.*

```
noncomputable def comap (f : M.V →ₗ[ℂ] N.V) : BosonicAlgebra N →ₐ[ℂ] BosonicAlgebra M
```

lemma comap_ι [source]

```
@[simp]
lemma comap_ι (f : M.V →ₗ[ℂ] N.V) (x : JetComponentSpace N) :
    comap f (SymmetricAlgebra.ι ℂ _ x)
      = SymmetricAlgebra.ι ℂ _ (JetComponentSpace.comap f x)
```

lemma comap_id [source]

```
@[simp]
lemma comap_id : comap (LinearMap.id : M.V →ₗ[ℂ] M.V) = AlgHom.id ℂ (BosonicAlgebra M)
```

lemma comap_comp [source]

Functoriality: the order reverses, as it must for a contravariant construction.

```
lemma comap_comp {P : MatterField jets} (f : M.V →ₗ[ℂ] N.V) (g : N.V →ₗ[ℂ] P.V) :
    comap (g.comp f) = (comap f).comp (comap g)
```

lemma comap_ofField [source]

The inclusion sends a component function of the species to the corresponding component function of the whole.

```
@[simp]
lemma comap_ofField (f : M.V →ₗ[ℂ] N.V) (φ : Module.Dual ℂ N.V) :
    comap f (ofField φ) = ofField (φ ∘ₗ f)
```

lemma comap_ofConjField [source]

The inclusion sends a conjugate component function of the species to the corresponding conjugate component function of the whole.

```
@[simp]
lemma comap_ofConjField (f : M.V →ₗ[ℂ] N.V) (φ : Module.Dual ℂ (ConjModule N.V)) :
    comap f (ofConjField φ) = ofConjField (φ ∘ₗ ConjModule.map f)
```

2.494 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.GaugeAction

PhyslibAlpha/Particles/StandardModel/Matter/Bosonic/GaugeAction.lean on GitHub.

def repJetGaugeGroupI [source]

***The jet gauge action on the bosonic algebra** of the matter field `M`: the symmetric-algebra functor applied to the gauge action on the jet component space. The fibrewise action on the jets and its fibrewise-linearity are fields of `M`.*

```
noncomputable def repJetGaugeGroupI :
    Representation ℂ GJ (BosonicAlgebra M) where
```

lemma repJetGaugeGroupI_apply [source]

```
lemma repJetGaugeGroupI_apply
    (U : GJ) (x : BosonicAlgebra M) :
    repJetGaugeGroupI M U x =
      SymmetricAlgebra.map (JetComponentSpace.repJet M U) x
```

lemma repJetGaugeGroupI_apply_one [source]

```
@[simp]
lemma repJetGaugeGroupI_apply_one
    (U : GJ) :
    repJetGaugeGroupI M U (1 : BosonicAlgebra M) = 1
```

lemma repJetGaugeGroupI_apply_mul [source]

```
lemma repJetGaugeGroupI_apply_mul
    (U : GJ) (x y : BosonicAlgebra M) :
    repJetGaugeGroupI M U (x * y) =
      repJetGaugeGroupI M U x * repJetGaugeGroupI M U y
```

lemma repJetGaugeGroupI_ι [source]

On a component function the jet gauge action is the action on the component space.

```
@[simp]
lemma repJetGaugeGroupI_ι
    (U : GJ) (v : JetComponentSpace M) :
    repJetGaugeGroupI M U (SymmetricAlgebra.ι ℂ _ v) =
      SymmetricAlgebra.ι ℂ _ (JetComponentSpace.repJet M U v)
```

def repJetGaugeGroupIAlgHom [source]

The jet gauge action as an algebra homomorphism: a gauge transformation acts on a Lagrangian term factor by factor.

```
noncomputable def repJetGaugeGroupIAlgHom
    (U : GJ) : BosonicAlgebra M →ₐ[ℂ] BosonicAlgebra M where
```

lemma repJetGaugeGroupI_ofField [source]

***`ofField` is gauge equivariant.** The undifferentiated component functions transform by the contragredient of the value of the gauge transformation at the base point; no derivative of the gauge jet contributes.*

```
lemma repJetGaugeGroupI_ofField
    (U : GJ) (φ : Module.Dual ℂ M.V) :
```

```
    repJetGaugeGroupI M U (ofField φ) =
      ofField (Module.Dual.transpose (jetEval ∘ₗ (M.repJet U⁻¹).comp jetOfConstant) φ)
```

lemma repJetGaugeGroupI_ofConjField [source]

***`ofConjField` is gauge equivariant**, for the conjugate action `JetComponentSpace.repConj rep` on the jets of the conjugate field — which is the physicists' `φ⁻ ↦ φ⁻ U†`.*

```
lemma repJetGaugeGroupI_ofConjField
    (U : GJ) (φ : Module.Dual ℂ (ConjModule M.V)) :
    repJetGaugeGroupI M U (ofConjField φ) =
      ofConjField (Module.Dual.transpose
        (jetEval ∘ₗ (JetComponentSpace.repConj M.repJet U⁻¹).comp jetOfConstant) φ)
```

def repGaugeGroupI [source]

The action of the constant — that is, global — gauge transformations on the bosonic algebra, obtained by including a gauge transformation as a constant gauge jet.

```
noncomputable def repGaugeGroupI :
    Representation ℂ G₀ (BosonicAlgebra M)
```

lemma repGaugeGroupI_apply [source]

```
lemma repGaugeGroupI_apply
    (g : G₀) (x : BosonicAlgebra M) :
    repGaugeGroupI M g x =
      repJetGaugeGroupI M (jets.ofConstant g) x
```

lemma repGaugeGroupI_apply_one [source]

```
@[simp]
lemma repGaugeGroupI_apply_one
    (g : G₀) :
    repGaugeGroupI M g (1 : BosonicAlgebra M) = 1
```

lemma repGaugeGroupI_apply_mul [source]

```
lemma repGaugeGroupI_apply_mul
    (g : G₀) (x y : BosonicAlgebra M) :
    repGaugeGroupI M g (x * y) =
      repGaugeGroupI M g x * repGaugeGroupI M g y
```

lemma repGaugeGroupI_ofField [source]

A constant gauge transformation acts on the undifferentiated field by the contragredient of its value — which for a constant jet is the transformation itself.

```
lemma repGaugeGroupI_ofField
    (g : G₀) (φ : Module.Dual ℂ M.V) :
    repGaugeGroupI M g (ofField φ) =
      ofField (Module.Dual.transpose
        (jetEval ∘ₗ (M.repJet (jets.ofConstant g⁻¹)).comp jetOfConstant) φ)
```

lemma repGaugeGroupI_ofConjField [source]

A constant gauge transformation acts on the undifferentiated conjugate field by the conjugate contragredient of its value.

```
lemma repGaugeGroupI_ofConjField
    (g : G₀) (φ : Module.Dual ℂ (ConjModule M.V)) :
    repGaugeGroupI M g (ofConjField φ) =
      ofConjField (Module.Dual.transpose
        (jetEval ∘ₗ (JetComponentSpace.repConj M.repJet (jets.ofConstant g⁻¹)).comp
          jetOfConstant) φ)
```

2.495 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.JetDeriv

PhyslibAlpha/Particles/StandardModel/Matter/Bosonic/JetDeriv.lean on GitHub.

def jetDeriv [source]

The formal total spacetime derivative on the bosonic algebra of a `V`-valued matter field in the direction `μ`: the derivation extending the shift `∂_s φ_α ↦ ∂_{s + {μ}} φ_α` of the component functions.

```
noncomputable def jetDeriv (μ : Fin 1 ⊕ Fin 3) :
    BosonicAlgebra M →ₗ[ℂ] BosonicAlgebra M
```

lemma jetDeriv_ι [source]

On a component function the total derivative is the shift of the derivative label.

```
@[simp]
lemma jetDeriv_ι (μ : Fin 1 ⊕ Fin 3) (x : JetComponentSpace M) :
    jetDeriv μ (SymmetricAlgebra.ι ℂ _ x) =
      SymmetricAlgebra.ι ℂ _ (JetComponentSpace.jetDeriv μ x)
```

lemma jetDeriv_one [source]

```
@[simp]
lemma jetDeriv_one (μ : Fin 1 ⊕ Fin 3) : jetDeriv (M := M) μ (1 : BosonicAlgebra M) = 0
```

lemma jetDeriv_algebraMap [source]

```
@[simp]
lemma jetDeriv_algebraMap (μ : Fin 1 ⊕ Fin 3) (r : ℂ) :
    jetDeriv (M := M) μ (algebraMap ℂ (BosonicAlgebra M) r) = 0
```

lemma jetDeriv_mul [source]

The total derivative is a derivation: the Leibniz rule holds on the bosonic algebra.

```
lemma jetDeriv_mul (μ : Fin 1 ⊕ Fin 3) (x y : BosonicAlgebra M) :
    jetDeriv μ (x * y) = jetDeriv μ x * y + x * jetDeriv μ y
```

lemma jetDeriv_comm_apply [source]

***Mixed partials agree.** The derivative labels live in a *symmetric* algebra, so the total derivatives in different directions commute.*

```
lemma jetDeriv_comm_apply (μ ν : Fin 1 ⊕ Fin 3) (x : BosonicAlgebra M) :
    jetDeriv μ (jetDeriv ν x) = jetDeriv ν (jetDeriv μ x)
```

lemma jetDeriv_comm [source]

```
lemma jetDeriv_comm (μ ν : Fin 1 ⊕ Fin 3) :
    (jetDeriv (M := M) μ).comp (jetDeriv ν) = (jetDeriv (M := M) ν).comp (jetDeriv μ)
```

def iteratedJetDeriv [source]

The iterated total derivative `∂_s = ∂_{ν₁} ⋯ ∂_{νₙ}` along a multiset `s` of directions. It is well defined on a multiset — i.e. independent of the order in which the directions are listed — because the directional derivatives commute.

```
noncomputable def iteratedJetDeriv (s : Multiset (Fin 1 ⊕ Fin 3)) :
    BosonicAlgebra M →ₗ[ℂ] BosonicAlgebra M
```

lemma iteratedJetDeriv_zero [source]

```
@[simp]
lemma iteratedJetDeriv_zero :
    iteratedJetDeriv (0 : Multiset (Fin 1 ⊕ Fin 3))
      = LinearMap.id (R := ℂ) (M := BosonicAlgebra M)
```

lemma iteratedJetDeriv_cons [source]

```
lemma iteratedJetDeriv_cons (μ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    iteratedJetDeriv (M := M) (μ ::ₘ s) = (jetDeriv μ).comp (iteratedJetDeriv s)
```

lemma iteratedJetDeriv_cons' [source]

The companion of `iteratedJetDeriv_cons`, peeling the extra derivative on the inside.

```
lemma iteratedJetDeriv_cons' (μ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    iteratedJetDeriv (M := M) (μ ::ₘ s) = (iteratedJetDeriv s).comp (jetDeriv μ)
```

lemma iteratedJetDeriv_singleton [source]

```
@[simp]
lemma iteratedJetDeriv_singleton (μ : Fin 1 ⊕ Fin 3) :
    iteratedJetDeriv (M := M) {μ} = jetDeriv μ
```

lemma iteratedJetDeriv_add [source]

The iterated derivative is additive in the multiset of directions: differentiating along `s + t` is differentiating along `t` and then along `s`.

```
lemma iteratedJetDeriv_add (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    iteratedJetDeriv (M := M) (s + t)
      = (iteratedJetDeriv s).comp (iteratedJetDeriv t)
```

lemma iteratedJetDeriv_mul [source]

***The all-orders Leibniz rule.** The iterated derivative of a product distributes over the antidiagonal of the multiset of directions: each way of splitting the derivatives between the two factors contributes one term.*

```
lemma iteratedJetDeriv_mul (s : Multiset (Fin 1 ⊕ Fin 3)) (x y : BosonicAlgebra M) :
    iteratedJetDeriv s (x * y) =
      (s.antidiagonal.map fun p =>
        iteratedJetDeriv p.1 x * iteratedJetDeriv p.2 y).sum
```

lemma iteratedJetDeriv_one_of_ne_zero [source]

A nonempty iterated derivative kills the constants.

```
lemma iteratedJetDeriv_one_of_ne_zero {s : Multiset (Fin 1 ⊕ Fin 3)} (hs : s ≠ 0) :
    iteratedJetDeriv (M := M) s (1 : BosonicAlgebra M) = 0
```

lemma iteratedJetDeriv_ι [source]

***On a component function the iterated derivative is the derivative symbol `∂_s`.** Both halves of the component space — the field and its conjugate — are multiplied by the degree-`|s|` element `∂_s` of `SpaceTimeDerivAlgebraℂ` in their derivative-label factor, with the target index untouched.*

```
lemma iteratedJetDeriv_ι (s : Multiset (Fin 1 ⊕ Fin 3)) (x : JetComponentSpace M) :
    iteratedJetDeriv s (SymmetricAlgebra.ι ℂ _ x) =
      SymmetricAlgebra.ι ℂ _
        (TensorProduct.map (LinearMap.mulRight ℂ (SpaceTimeDerivAlgebraℂ.basis s))
            LinearMap.id x.1,
          TensorProduct.map (LinearMap.mulRight ℂ (SpaceTimeDerivAlgebraℂ.basis s))
            LinearMap.id x.2)
```

lemma iteratedJetDeriv_ofField [source]

The iterated derivative of the field is the generator carrying the derivative symbol `∂_s`: applying `∂_s` to `ψ_φ` writes the label `s` into the derivative factor.

```
@[simp]
lemma iteratedJetDeriv_ofField (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ M.V) :
    iteratedJetDeriv s (ofField φ) =
      SymmetricAlgebra.ι ℂ _
        ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace M)
```

lemma iteratedJetDeriv_ofConjField [source]

The iterated derivative of the conjugate field is the conjugate generator carrying the derivative symbol `∂_s`.

```
@[simp]
lemma iteratedJetDeriv_ofConjField (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule M.V)) :
    iteratedJetDeriv s (ofConjField φ) =
      SymmetricAlgebra.ι ℂ _
        ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace M)
```

lemma adjoin_iteratedJetDeriv_eq_top [source]

***The bosonic algebra is generated by the field, its conjugate, and their derivatives.** As a `ℂ`-algebra, `BosonicAlgebra M` is the algebra adjoined by the iterated total derivatives `∂_s ψ_φ` and `∂_s ψ̄_φ` of the undifferentiated component functions. Physically: every Lagrangian term for a `V`-valued bosonic matter field is a polynomial in the field, its conjugate, and their spacetime derivatives — nothing else is available to write down.*

```
lemma adjoin_iteratedJetDeriv_eq_top :
    Algebra.adjoin ℂ
      (⋃ s : Multiset (Fin 1 ⊕ Fin 3),
        Set.range (fun φ : Module.Dual ℂ M.V => iteratedJetDeriv s (ofField φ)) ∪
          Set.range (fun φ : Module.Dual ℂ (ConjModule M.V) =>
            iteratedJetDeriv s (ofConjField φ)))
      = (⊤ : Subalgebra ℂ (BosonicAlgebra M))
```

lemma comap_jetDeriv [source]

***The inclusion of a species is a map of differential algebras.** Pulling back along a map of target spaces commutes with the total derivative: the two act on different labels of a component function.*

```
lemma comap_jetDeriv (f : M.V →ₗ[ℂ] N.V) (μ : Fin 1 ⊕ Fin 3) (x : BosonicAlgebra N) :
    comap f (jetDeriv μ x) = jetDeriv μ (comap f x)
```

lemma comap_iteratedJetDeriv [source]

The inclusion of a species commutes with the iterated total derivative.

```
lemma comap_iteratedJetDeriv (f : M.V →ₗ[ℂ] N.V) (s : Multiset (Fin 1 ⊕ Fin 3))
    (x : BosonicAlgebra N) :
    comap f (iteratedJetDeriv s x) = iteratedJetDeriv s (comap f x)
```

2.496 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.LorentzAction

PhyslibAlpha/Particles/StandardModel/Matter/Bosonic/LorentzAction.lean on GitHub.

def repLorentzGroup [source]

***The Lorentz action on the bosonic algebra** of the matter field `M`, induced from a representation `M.repLorentz` of `SL(2,ℂ)` on `V`: the symmetric-algebra functor applied to the Lorentz action on the jet component space.*

```
noncomputable def repLorentzGroup :
    Representation ℂ SL(2,ℂ) (BosonicAlgebra M) where
```

lemma repLorentzGroup_apply [source]

```
lemma repLorentzGroup_apply (Λ : SL(2,ℂ))
    (x : BosonicAlgebra M) :
    repLorentzGroup M Λ x =
      SymmetricAlgebra.map (JetComponentSpace.repLorentzGroup M Λ) x
```

lemma repLorentzGroup_apply_one [source]

```
@[simp]
lemma repLorentzGroup_apply_one (Λ : SL(2,ℂ)) :
    repLorentzGroup M Λ (1 : BosonicAlgebra M) = 1
```

lemma repLorentzGroup_apply_mul [source]

```
lemma repLorentzGroup_apply_mul (Λ : SL(2,ℂ))
    (x y : BosonicAlgebra M) :
    repLorentzGroup M Λ (x * y)
      = repLorentzGroup M Λ x * repLorentzGroup M Λ y
```

lemma repLorentzGroup_ι [source]

On a component function the Lorentz action is the action on the component space.

```
@[simp]
lemma repLorentzGroup_ι (Λ : SL(2,ℂ))
    (v : JetComponentSpace M) :
    repLorentzGroup M Λ (SymmetricAlgebra.ι ℂ _ v) =
      SymmetricAlgebra.ι ℂ _ (JetComponentSpace.repLorentzGroup M Λ v)
```

def repLorentzGroupAlgHom [source]

The Lorentz action as an algebra homomorphism: it preserves the symmetric product, so a Lorentz transformation acts on a Lagrangian term factor by factor.

```
noncomputable def repLorentzGroupAlgHom (Λ : SL(2,ℂ)) :
    BosonicAlgebra M →ₐ[ℂ] BosonicAlgebra M where
```

lemma repLorentzGroup_ofField [source]

***`ofField` is `SL(2,ℂ)`-equivariant.** The undifferentiated component functions carry the contragredient of the representation on the target space, and no derivative labels are generated: `ofField` intertwines `M.repLorentz.dual` with the action on the bosonic algebra.*

```
@[simp]
lemma repLorentzGroup_ofField (Λ : SL(2,ℂ))
    (φ : Module.Dual ℂ M.V) :
    repLorentzGroup M Λ (ofField φ) = ofField (M.repLorentz.dual Λ φ)
```

lemma repLorentzGroup_ofConjField [source]

***`ofConjField` is `SL(2,ℂ)`-equivariant**, for the conjugate of the representation on the target space: the conjugate component functions transform by `star` of the spinor matrix.*

```
@[simp]
lemma repLorentzGroup_ofConjField (Λ : SL(2,ℂ))
    (φ : Module.Dual ℂ (ConjModule M.V)) :
    repLorentzGroup M Λ (ofConjField φ) = ofConjField (M.repLorentz.conj.dual Λ φ)
```

lemma repLorentzGroup_jetDeriv [source]

***The total derivative on the bosonic algebra is a Lorentz vector.** The four derivations `∂_μ` transform into each other by the columns of the Lorentz matrix of `Λ`, exactly as the covector index `μ` should.*

```
lemma repLorentzGroup_jetDeriv (Λ : SL(2,ℂ))
    (μ : Fin 1 ⊕ Fin 3) (x : BosonicAlgebra M) :
    repLorentzGroup M Λ (jetDeriv μ x) =
      ∑ a, (((Lorentz.SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
        jetDeriv a (repLorentzGroup M Λ x)
```

instance instIsLorentzDeriv [source]

The total derivatives on the bosonic algebra form a Lorentz derivative, giving access to the boost-weight machinery of `PhyslibAlpha.Relativity.IsLorentzDeriv`.

```
instance instIsLorentzDeriv :
    Lorentz.IsLorentzDeriv (repLorentzGroup M) (jetDeriv (M := M)) where
```

2.497 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.MassDim

PhyslibAlpha/Particles/StandardModel/Matter/Bosonic/MassDim.lean on GitHub.

def massWeightScale [source]

***The mass-weight scaling on the bosonic algebra** of a field of mass weight `w`: the algebra endomorphism scaling the generator `∂_s φ_α` by `c ^ (w + 2 |s|)`, the functorial lift of the scaling on the jet component space.*

```
noncomputable def massWeightScale (w : ℕ) (c : ℂ) : BosonicAlgebra M →ₐ[ℂ] BosonicAlgebra M
```

lemma massWeightScale_ι [source]

```
@[simp]
lemma massWeightScale_ι (w : ℕ) (c : ℂ) (x : JetComponentSpace M) :
    massWeightScale w c (SymmetricAlgebra.ι ℂ _ x)
      = SymmetricAlgebra.ι ℂ _ (JetComponentSpace.massWeightScale w c x)
```

lemma massWeightScale_ofField [source]

The undifferentiated field carries its own mass weight.

```
@[simp]
lemma massWeightScale_ofField (w : ℕ) (c : ℂ) (φ : Module.Dual ℂ M.V) :
    massWeightScale w c (ofField φ) = c ^ w • ofField φ
```

lemma massWeightScale_ofConjField [source]

The undifferentiated conjugate field carries the same mass weight as the field.

```
@[simp]
lemma massWeightScale_ofConjField (w : ℕ) (c : ℂ) (φ : Module.Dual ℂ (ConjModule M.V)) :
    massWeightScale w c (ofConjField φ) = c ^ w • ofConjField φ
```

lemma massWeightScale_jetDeriv [source]

***A total derivative adds mass weight two**: the scaling intertwines the total derivative up to a factor `c ^ 2`.*

```
lemma massWeightScale_jetDeriv (w : ℕ) (c : ℂ) (μ : Fin 1 ⊕ Fin 3) (x : BosonicAlgebra M) :
    massWeightScale w c (jetDeriv μ x) = c ^ 2 • jetDeriv μ (massWeightScale w c x)
```

lemma massWeightScale_iteratedJetDeriv [source]

The iterated derivative `∂_s` adds mass weight `2 |s|`.

```
lemma massWeightScale_iteratedJetDeriv (w : ℕ) (c : ℂ) (s : Multiset (Fin 1 ⊕ Fin 3))
    (x : BosonicAlgebra M) :
    massWeightScale w c (iteratedJetDeriv s x)
      = c ^ (2 * Multiset.card s) • iteratedJetDeriv s (massWeightScale w c x)
```

2.498 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.MassWeightPoly

PhyslibAlpha/Particles/StandardModel/Matter/Bosonic/MassWeightPoly.lean on GitHub.

def monomial$_l$ [source]

The monomial map into polynomials over the bosonic algebra, as a map of `ℂ`-modules rather than of `BosonicAlgebra M`-modules.

```
noncomputable def monomial_l (n : ℕ) :
    BosonicAlgebra M →ₗ[ℂ] Polynomial (BosonicAlgebra M)
```

lemma monomial$_l$_apply [source]

```
@[simp]
lemma monomial_l_apply (n : ℕ) (x : BosonicAlgebra M) :
    monomial_l n x = Polynomial.monomial n x
```

def halfPoly [source]

One half of the mass-weight polynomial on the jet component space, for a field of mass weight `w`: the linear map sending the symbol `∂_s φ` to `X ^ (w + 2 |s|)` times its image under `k`. The two halves of the component space differ only in the inclusion `k` of the symbols into the bosonic algebra, so both are instances of this map.

```
noncomputable def halfPoly {W : Type} [AddCommGroup W] [Module ℂ W] (w : ℕ)
    (k : SpaceTimeDerivAlgebraℂ ⊗[ℂ] W →ₗ[ℂ] BosonicAlgebra M) :
    SpaceTimeDerivAlgebraℂ ⊗[ℂ] W →ₗ[ℂ] Polynomial (BosonicAlgebra M)
```

lemma halfPoly_basis_tmul [source]

On the symbol `∂_s φ` the half mass-weight polynomial is the monomial of degree `w + 2 |s|`: the field contributes `w` and each derivative two.

```
lemma halfPoly_basis_tmul {W : Type} [AddCommGroup W] [Module ℂ W] (w : ℕ)
    (k : SpaceTimeDerivAlgebraℂ ⊗[ℂ] W →ₗ[ℂ] BosonicAlgebra M)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (x : W) :
    halfPoly w k (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] x) =
      Polynomial.monomial (w + 2 * Multiset.card s)
        (k (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] x))
```

def ιFst [source]

The inclusion of the unconjugated symbols into the bosonic algebra.

```
noncomputable def ιFst : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ M.V →ₗ[ℂ] BosonicAlgebra M
```

def ιSnd [source]

The inclusion of the conjugate symbols into the bosonic algebra.

```
noncomputable def ιSnd :
    SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule M.V) →ₗ[ℂ] BosonicAlgebra M
```

def jetComponentPoly [source]

The mass-weight polynomial of a component function of a field of mass weight `w`: the sum of the two half maps, one for the field and one for its conjugate.

```
noncomputable def jetComponentPoly (w : ℕ) :
    JetComponentSpace M →ₗ[ℂ] Polynomial (BosonicAlgebra M)
```

lemma jetComponentPoly_apply [source]

```
lemma jetComponentPoly_apply (w : ℕ) (x : JetComponentSpace M) :
    jetComponentPoly w x = halfPoly w ιFst x.1 + halfPoly w ιSnd x.2
```

lemma jetComponentPoly_inl [source]

On an unconjugated derivative monomial the component map is a monomial eigenvector.

```
@[simp]
lemma jetComponentPoly_inl (w : ℕ) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ M.V) :
    jetComponentPoly w ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace M) =
      Polynomial.monomial (w + 2 * Multiset.card s)
        (SymmetricAlgebra.ι ℂ (JetComponentSpace M)
          ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace M))
```

lemma jetComponentPoly_inr [source]

On a conjugate derivative monomial the component map is a monomial eigenvector.

```
@[simp]
lemma jetComponentPoly_inr (w : ℕ) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule M.V)) :
    jetComponentPoly w ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace M) =
      Polynomial.monomial (w + 2 * Multiset.card s)
        (SymmetricAlgebra.ι ℂ (JetComponentSpace M)
          ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace M))
```

def massWeightPoly [source]

The mass-weight polynomial on the bosonic algebra of a field of mass weight `w`: the `ℂ`-algebra map sending a generator of mass weight `n` to `X ^ n` times itself. It is `BosonicAlgebra.massWeightScale` with the scalar replaced by the formal variable `X`, and needs no side condition because `Polynomial (BosonicAlgebra M)` is commutative.

```
noncomputable def massWeightPoly (w : ℕ) :
    BosonicAlgebra M →ₐ[ℂ] Polynomial (BosonicAlgebra M)
```

lemma massWeightPoly_ι [source]

On a component function the mass-weight polynomial is the component-function map.

```
@[simp]
lemma massWeightPoly_ι (w : ℕ) (x : JetComponentSpace M) :
    massWeightPoly w (SymmetricAlgebra.ι ℂ (JetComponentSpace M) x) =
      jetComponentPoly w x
```

lemma massWeightPoly_iteratedJetDeriv_ofField [source]

The generator `∂_s ψ_φ` is a monomial eigenvector of mass weight `w + 2 |s|`: the field carries its own mass weight and each derivative adds two.

```
lemma massWeightPoly_iteratedJetDeriv_ofField (w : ℕ) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ M.V) :
    massWeightPoly w (iteratedJetDeriv s (ofField φ)) =
      Polynomial.monomial (w + 2 * Multiset.card s) (iteratedJetDeriv s (ofField φ))
```

lemma massWeightPoly_iteratedJetDeriv_ofConjField [source]

The conjugate generator `∂_s ψ̄_φ` is a monomial eigenvector of the same mass weight `w + 2 |s|` as the generator it conjugates.

```
lemma massWeightPoly_iteratedJetDeriv_ofConjField (w : ℕ) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule M.V)) :
    massWeightPoly w (iteratedJetDeriv s (ofConjField φ)) =
      Polynomial.monomial (w + 2 * Multiset.card s)
        (iteratedJetDeriv s (ofConjField φ))
```

lemma massWeightPoly_ofField [source]

The undifferentiated field has mass weight `w`.

```
lemma massWeightPoly_ofField (w : ℕ) (φ : Module.Dual ℂ M.V) :
    massWeightPoly w (ofField φ) = Polynomial.monomial w (ofField φ)
```

lemma massWeightPoly_ofConjField [source]

The undifferentiated conjugate field has mass weight `w`.

```
lemma massWeightPoly_ofConjField (w : ℕ) (φ : Module.Dual ℂ (ConjModule M.V)) :
    massWeightPoly w (ofConjField φ) = Polynomial.monomial w (ofConjField φ)
```

lemma halfPoly_eval_one [source]

Setting the formal variable to one collapses a half mass-weight polynomial back to the symbol it graded. The derivative monomials span, so it is enough to check this on the multiset basis.

```
lemma halfPoly_eval_one {W : Type} [AddCommGroup W] [Module ℂ W] (w : ℕ)
    (k : SpaceTimeDerivAlgebraℂ ⊗[ℂ] W →ₗ[ℂ] BosonicAlgebra M)
    (y : SpaceTimeDerivAlgebraℂ ⊗[ℂ] W) : (halfPoly w k y).eval 1 = k y
```

lemma jetComponentPoly_eval_one [source]

Setting the formal variable to one recovers the component function.

```
lemma jetComponentPoly_eval_one (w : ℕ) (x : JetComponentSpace M) :
    (jetComponentPoly w x).eval 1 = SymmetricAlgebra.ι ℂ (JetComponentSpace M) x
```

lemma massWeightPoly_eval_one [source]

Setting the formal variable to one recovers the original element: the mass-weight pieces of an element sum back to it.

```
lemma massWeightPoly_eval_one (w : ℕ) (a : BosonicAlgebra M) :
    (massWeightPoly w a).eval 1 = a
```

lemma massWeightPoly_injective [source]

The mass-weight polynomial is injective: an element is recovered from its graded pieces. It is not surjective, since a monomial of the wrong degree is not the grading of anything.

```
lemma massWeightPoly_injective (w : ℕ) :
    Function.Injective (massWeightPoly (M := M) w)
```

2.499 PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.Prod

PhyslibAlpha/Particles/StandardModel/Matter/Bosonic/Prod.lean on GitHub.

def prodEquiv [source]

***The bosonic algebra of a direct sum is the tensor product of the bosonic algebras.** Two bosonic matter fields taken together are one field valued in the direct sum of their target spaces, and its bosonic algebra is the tensor product of theirs. The ordinary —*

rather than the graded — tensor product is correct here: bosonic generators commute across species just as they do within one.

```
noncomputable def BosonicAlgebra.prodEquiv (M N : MatterField jets)
    (h : M.massWeight = N.massWeight) (hσ : M.statistic = N.statistic) :
    BosonicAlgebra (M.prod N h hσ) ≃ₐ[ℂ] BosonicAlgebra M ⊗[ℂ] BosonicAlgebra N
```

2.500 `PhyslibAlpha.Particles.StandardModel.Matter.Bosonic.TransformsIn`

PhyslibAlpha/Particles/StandardModel/Matter/Bosonic/TransformsIn.lean on GitHub.

lemma `sum_inl` [source]

A multiset sum in the unconjugated half of the component space passes through the inclusion of the generators.

```
private lemma sum_inl (m : Multiset (SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ M.V)) :
    SymmetricAlgebra.ι ℂ _ ((m.sum, 0) : JetComponentSpace M)
      = (m.map fun a =>
          SymmetricAlgebra.ι ℂ _ ((a, 0) : JetComponentSpace M)).sum
```

lemma `sum_inr` [source]

A multiset sum in the conjugate half of the component space passes through the inclusion of the generators.

```
private lemma sum_inr (m : Multiset (SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule M.V)
    )) :
    SymmetricAlgebra.ι ℂ _ ((0, m.sum) : JetComponentSpace M)
      = (m.map fun a =>
          SymmetricAlgebra.ι ℂ _ ((0, a) : JetComponentSpace M)).sum
```

lemma `repJetGaugeGroupI_iteratedJetDeriv_ofField` [source]

The transformation law of the derivative generators of a matter field: a jet of gauge transformations mixes `∂_s ψ_φ` with the lower generators, each splitting `s = s₁ + s₂` of the derivative multiset contributing the base-point Taylor coefficient of the gauge jet at `s₁` acting on the target index of `∂_{s₂} ψ_φ`. There is no inhomogeneous term: unlike a gauge field, a matter field transforms linearly.

```
lemma repJetGaugeGroupI_iteratedJetDeriv_ofField
    (U : GJ) (φ : Module.Dual ℂ M.V) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    repJetGaugeGroupI M U (iteratedJetDeriv s (ofField φ)) =
      (s.antidiagonal.map fun p =>
        iteratedJetDeriv p.2
          (ofField (GaugeAlgebraRealization.repDualCoeff M.repJet U⁻¹ p.1 φ))).sum
```

lemma `transformsIn_iteratedJetDeriv_ofField` [source]

The derivative generators of a matter field transform in the representation `rep` carried by its jets, in the sense demanded by `LocalGaugeData.TransformsIn`.

```
lemma transformsIn_iteratedJetDeriv_ofField :
    LocalGaugeData.TransformsIn (repJetGaugeGroupI M) M.repJet
      fun s => (iteratedJetDeriv s).comp (ofField (M := M))
```

lemma repJetGaugeGroupI_iteratedJetDeriv_ofConjField [source]

The transformation law of the derivative generators of the conjugate matter field. It is the law of the field itself for the conjugate action `JetComponentSpace.repConj M.repJet` on the jets of the conjugate field — the physicists' `ψ⁻ ↦ ψ⁻ U†` and its derivatives.

```
lemma repJetGaugeGroupI_iteratedJetDeriv_ofConjField
    (U : GJ) (φ : Module.Dual ℂ (ConjModule M.V))
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    repJetGaugeGroupI M U (iteratedJetDeriv s (ofConjField φ)) =
      (s.antidiagonal.map fun p =>
        iteratedJetDeriv p.2
          (ofConjField
            (GaugeAlgebraRealization.repDualCoeff (JetComponentSpace.repConj M.repJet) U⁻¹ p.1
              φ))).sum
```

lemma transformsIn_iteratedJetDeriv_ofConjField [source]

The derivative generators of the conjugate matter field transform in the conjugate representation `JetComponentSpace.repConj M.repJet`, in the sense demanded by `LocalGaugeData.TransformsIn`.

```
lemma transformsIn_iteratedJetDeriv_ofConjField :
    LocalGaugeData.TransformsIn (repJetGaugeGroupI M) (JetComponentSpace.repConj M.repJet)
      fun s => (iteratedJetDeriv s).comp (ofConjField (M := M))
```

2.501 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.Basic

PhyslibAlpha/Particles/StandardModel/Matter/Fermionic/Basic.lean on GitHub.

abbrev FermionicAlgebra [source]

The fermionic algebra of the matter field `M`: the exterior algebra on the space of component functions `∂_s ψ_α` and `∂_s ψ⁻_α`. The exterior product is the product of fermionic fields, its anticommutativity the Fermi statistics.

```
abbrev FermionicAlgebra (M : MatterField jets) : Type
```

lemma adjoin_ι_eq_top [source]

***The fermionic algebra is generated by the component functions.** Every element is a polynomial in the degree-one elements — the symbols `∂_s ψ_α` and `∂_s ψ⁻_α` themselves. This is the algebraic form of "every Lagrangian term is a polynomial in the component functions".*

```
lemma adjoin_ι_eq_top :
    Algebra.adjoin ℂ (Set.range (ExteriorAlgebra.ι ℂ (M := JetComponentSpace M))) = ⊤
```

lemma ι_sq_zero [source]

A component function squares to zero: no fermionic field appears twice.

```
lemma ι_sq_zero (x : JetComponentSpace M) :
    ExteriorAlgebra.ι ℂ x * ExteriorAlgebra.ι ℂ x = (0 : FermionicAlgebra M)
```

lemma ι_mul_ι_swap [source]

Two component functions anticommute.

```
lemma ι_mul_ι_swap (x y : JetComponentSpace M) :
    (ExteriorAlgebra.ι ℂ x * ExteriorAlgebra.ι ℂ y : FermionicAlgebra M)
      = - (ExteriorAlgebra.ι ℂ y * ExteriorAlgebra.ι ℂ x)
```

def ofField [source]

***The component function `ψ_φ` of the matter field** along the covector `φ` on `V`: the undifferentiated generator, sitting at the empty derivative label in the unconjugated half of the component space.*

```
noncomputable def ofField : Module.Dual ℂ M.V →ₗ[ℂ] FermionicAlgebra M
```

def ofConjField [source]

***The component function `ψ̄_φ` of the conjugate matter field** along the covector `φ` on `ConjModule M.V`: the undifferentiated generator in the conjugate half of the component space.*

```
noncomputable def ofConjField : Module.Dual ℂ (ConjModule M.V) →ₗ[ℂ] FermionicAlgebra M
```

lemma ofField_apply [source]

```
lemma ofField_apply (φ : Module.Dual ℂ M.V) :
    ofField φ = ExteriorAlgebra.ι ℂ
      (((1 : SpaceTimeDerivAlgebraℂ) ⊗ₜ[ℂ] φ, 0) : JetComponentSpace M)
```

lemma ofConjField_apply [source]

```
lemma ofConjField_apply (φ : Module.Dual ℂ (ConjModule M.V)) :
    ofConjField φ = ExteriorAlgebra.ι ℂ
      ((0, (1 : SpaceTimeDerivAlgebraℂ) ⊗ₜ[ℂ] φ) : JetComponentSpace M)
```

def comap [source]

***The fermionic algebra is contravariant in the target space.** A linear map `f : V →ₗ[ℂ] W` induces an algebra homomorphism `FermionicAlgebra N →ₐ[ℂ] FermionicAlgebra M` by pulling back component functions. Applied to a *projection* out of a multi-species target space, this is the inclusion of one species' algebra into the whole.*

```
noncomputable def comap (f : M.V →ₗ[ℂ] N.V) : FermionicAlgebra N →ₐ[ℂ] FermionicAlgebra M
```

lemma comap_ι [source]

```
@[simp]
lemma comap_ι (f : M.V →ₗ[ℂ] N.V) (x : JetComponentSpace N) :
    comap f (ExteriorAlgebra.ι ℂ x)
      = ExteriorAlgebra.ι ℂ (JetComponentSpace.comap f x)
```

lemma comap_id [source]

```
@[simp]
lemma comap_id : comap (LinearMap.id : M.V →ₗ[ℂ] M.V) = AlgHom.id ℂ (FermionicAlgebra M)
```

lemma comap_comp [source]

Functoriality: the order reverses, as it must for a contravariant construction.

```
lemma comap_comp {P : MatterField jets} (f : M.V →ₗ[ℂ] N.V) (g : N.V →ₗ[ℂ] P.V) :
    comap (g.comp f) = (comap f).comp (comap g)
```

lemma comap_ofField [source]

The inclusion sends a component function of the species to the corresponding component function of the whole.

```
@[simp]
lemma comap_ofField (f : M.V →ₗ[ℂ] N.V) (φ : Module.Dual ℂ N.V) :
    comap f (ofField φ) = ofField (φ ∘ₗ f)
```

lemma comap_ofConjField [source]

The inclusion sends a conjugate component function of the species to the corresponding conjugate component function of the whole.

```
@[simp]
lemma comap_ofConjField (f : M.V →ₗ[ℂ] N.V) (φ : Module.Dual ℂ (ConjModule N.V)) :
    comap f (ofConjField φ) = ofConjField (φ ∘ₗ ConjModule.map f)
```

2.502 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.GaugeAction

PhyslibAlpha/Particles/StandardModel/Matter/Fermionic/GaugeAction.lean on GitHub.

def repJetGaugeGroupI [source]

***The jet gauge action on the fermionic algebra** of the matter field `M`: the exterior-algebra functor applied to the gauge action on the jet component space. The fibrewise action on the jets and its fibrewise-linearity are fields of `M`.*

```
noncomputable def repJetGaugeGroupI :
    Representation ℂ GJ (FermionicAlgebra M) where
```

lemma repJetGaugeGroupI_apply [source]

```
lemma repJetGaugeGroupI_apply
    (U : GJ) (x : FermionicAlgebra M) :
    repJetGaugeGroupI M U x =
      ExteriorAlgebra.map (JetComponentSpace.repJet M U) x
```

lemma repJetGaugeGroupI_apply_one [source]

```
@[simp]
lemma repJetGaugeGroupI_apply_one
```

```
    (U : GJ) :
    repJetGaugeGroupI M U (1 : FermionicAlgebra M) = 1
```

lemma repJetGaugeGroupI_apply_mul [source]

```
lemma repJetGaugeGroupI_apply_mul
    (U : GJ) (x y : FermionicAlgebra M) :
    repJetGaugeGroupI M U (x * y) =
      repJetGaugeGroupI M U x * repJetGaugeGroupI M U y
```

lemma repJetGaugeGroupI_ι [source]

On a component function the jet gauge action is the action on the component space.

```
@[simp]
lemma repJetGaugeGroupI_ι
    (U : GJ) (v : JetComponentSpace M) :
    repJetGaugeGroupI M U (ExteriorAlgebra.ι ℂ v) =
      ExteriorAlgebra.ι ℂ (JetComponentSpace.repJet M U v)
```

def repJetGaugeGroupIAlgHom [source]

The jet gauge action as an algebra homomorphism: a gauge transformation acts on a Lagrangian term factor by factor.

```
noncomputable def repJetGaugeGroupIAlgHom
    (U : GJ) : FermionicAlgebra M →ₐ[ℂ] FermionicAlgebra M where
```

lemma repJetGaugeGroupI_ofField [source]

***`ofField` is gauge equivariant.** The undifferentiated component functions transform by the contragredient of the value of the gauge transformation at the base point; no derivative of the gauge jet contributes.*

```
lemma repJetGaugeGroupI_ofField
    (U : GJ) (φ : Module.Dual ℂ M.V) :
    repJetGaugeGroupI M U (ofField φ) =
      ofField (Module.Dual.transpose (jetEval ∘ₗ (M.repJet U⁻¹).comp jetOfConstant) φ)
```

lemma repJetGaugeGroupI_ofConjField [source]

***`ofConjField` is gauge equivariant**, for the conjugate action `JetComponentSpace.repConj rep` on the jets of the conjugate field — which is the physicists' `ψ̄ ↦ ψ̄ U†`.*

```
lemma repJetGaugeGroupI_ofConjField
    (U : GJ) (φ : Module.Dual ℂ (ConjModule M.V)) :
    repJetGaugeGroupI M U (ofConjField φ) =
      ofConjField (Module.Dual.transpose
        (jetEval ∘ₗ (JetComponentSpace.repConj M.repJet U⁻¹).comp jetOfConstant) φ)
```

def repGaugeGroupI [source]

The action of the constant — that is, global — gauge transformations on the fermionic algebra, obtained by including a gauge transformation as a constant gauge jet.

```
noncomputable def repGaugeGroupI :
    Representation ℂ G₀ (FermionicAlgebra M)
```

lemma repGaugeGroupI_apply [source]

```
lemma repGaugeGroupI_apply
    (g : G₀) (x : FermionicAlgebra M) :
    repGaugeGroupI M g x =
      repJetGaugeGroupI M (jets.ofConstant g) x
```

lemma repGaugeGroupI_apply_one [source]

```
@[simp]
lemma repGaugeGroupI_apply_one
    (g : G₀) :
    repGaugeGroupI M g (1 : FermionicAlgebra M) = 1
```

lemma repGaugeGroupI_apply_mul [source]

```
lemma repGaugeGroupI_apply_mul
    (g : G₀) (x y : FermionicAlgebra M) :
    repGaugeGroupI M g (x * y) =
      repGaugeGroupI M g x * repGaugeGroupI M g y
```

lemma repGaugeGroupI_ofField [source]

A constant gauge transformation acts on the undifferentiated field by the contragredient of its value — which for a constant jet is the transformation itself.

```
lemma repGaugeGroupI_ofField
    (g : G₀) (φ : Module.Dual ℂ M.V) :
    repGaugeGroupI M g (ofField φ) =
      ofField (Module.Dual.transpose
        (jetEval ∘ₗ (M.repJet (jets.ofConstant g⁻¹)).comp jetOfConstant) φ)
```

lemma repGaugeGroupI_ofConjField [source]

A constant gauge transformation acts on the undifferentiated conjugate field by the conjugate contragredient of its value.

```
lemma repGaugeGroupI_ofConjField
    (g : G₀) (φ : Module.Dual ℂ (ConjModule M.V)) :
    repGaugeGroupI M g (ofConjField φ) =
      ofConjField (Module.Dual.transpose
        (jetEval ∘ₗ (JetComponentSpace.repConj M.repJet (jets.ofConstant g⁻¹)).comp
          jetOfConstant) φ)
```

2.503 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.JetDeriv

PhyslibAlpha/Particles/StandardModel/Matter/Fermionic/JetDeriv.lean on GitHub.

def jetDerivGen [source]

The generator map of the total derivative into the trivial square-zero extension of the fermionic algebra: `ι x ↦ (ι x, ι (∂_μ x))`.

```
noncomputable def jetDerivGen (μ : Fin 1 ⊕ Fin 3) :
    JetComponentSpace M →ₗ[ℂ] TrivSqZeroExt (FermionicAlgebra M) (FermionicAlgebra M) where
```

lemma jetDerivGen_fst [source]

```
@[simp]
lemma jetDerivGen_fst (μ : Fin 1 ⊕ Fin 3) (x : JetComponentSpace M) :
    (jetDerivGen μ x).fst = ExteriorAlgebra.ι ℂ x
```

lemma jetDerivGen_snd [source]

```
@[simp]
lemma jetDerivGen_snd (μ : Fin 1 ⊕ Fin 3) (x : JetComponentSpace M) :
    (jetDerivGen μ x).snd = ExteriorAlgebra.ι ℂ (JetComponentSpace.jetDeriv μ x)
```

lemma jetDerivGen_mul_self [source]

The generator map squares to zero: degree-one elements of the exterior algebra anticommute.

```
lemma jetDerivGen_mul_self (μ : Fin 1 ⊕ Fin 3) (x : JetComponentSpace M) :
    jetDerivGen μ x * jetDerivGen μ x = 0
```

def jetDerivHom [source]

The lift of the total derivative to the trivial square-zero extension of the fermionic algebra: the algebra homomorphism `x ↦ (x, ∂_μ x)`.

```
noncomputable def jetDerivHom (μ : Fin 1 ⊕ Fin 3) :
    FermionicAlgebra M →ₐ[ℂ] TrivSqZeroExt (FermionicAlgebra M) (FermionicAlgebra M)
```

lemma jetDerivHom_ι [source]

```
@[simp]
lemma jetDerivHom_ι (μ : Fin 1 ⊕ Fin 3) (x : JetComponentSpace M) :
    jetDerivHom μ (ExteriorAlgebra.ι ℂ x) = jetDerivGen μ x
```

lemma jetDerivHom_fst [source]

The first component of the square-zero lift is the identity.

```
@[simp]
lemma jetDerivHom_fst (μ : Fin 1 ⊕ Fin 3) (x : FermionicAlgebra M) :
    (jetDerivHom μ x).fst = x
```

def jetDeriv [source]

The formal total spacetime derivative on the fermionic algebra of a `V`-valued matter field in the direction `μ`: the even derivation extending the shift `∂_s ψ_α ↦ ∂_{s + {μ}} ψ_α` of the component functions.

```
noncomputable def jetDeriv (μ : Fin 1 ⊕ Fin 3) :
    FermionicAlgebra M →ₗ[ℂ] FermionicAlgebra M where
```

lemma jetDeriv_apply [source]

```
lemma jetDeriv_apply (μ : Fin 1 ⊕ Fin 3) (x : FermionicAlgebra M) :
    jetDeriv μ x = (jetDerivHom μ x).snd
```

lemma jetDeriv_ι [source]

On a component function the total derivative is the shift of the derivative label.

```
@[simp]
lemma jetDeriv_ι (μ : Fin 1 ⊕ Fin 3) (x : JetComponentSpace M) :
    jetDeriv μ (ExteriorAlgebra.ι ℂ x) =
      ExteriorAlgebra.ι ℂ (JetComponentSpace.jetDeriv μ x)
```

lemma jetDeriv_one [source]

```
@[simp]
lemma jetDeriv_one (μ : Fin 1 ⊕ Fin 3) : jetDeriv (M := M) μ (1 : FermionicAlgebra M) = 0
```

lemma jetDeriv_algebraMap [source]

```
@[simp]
lemma jetDeriv_algebraMap (μ : Fin 1 ⊕ Fin 3) (r : ℂ) :
    jetDeriv (M := M) μ (algebraMap ℂ (FermionicAlgebra M) r) = 0
```

lemma jetDeriv_mul [source]

The total derivative is an even derivation: the Leibniz rule holds on the fermionic algebra with no Koszul signs.

```
lemma jetDeriv_mul (μ : Fin 1 ⊕ Fin 3) (x y : FermionicAlgebra M) :
    jetDeriv μ (x * y) = jetDeriv μ x * y + x * jetDeriv μ y
```

lemma jetDeriv_comm_apply [source]

***Mixed partials agree.** The derivative labels live in a *symmetric* algebra, so the total derivatives in different directions commute.*

```
lemma jetDeriv_comm_apply (μ ν : Fin 1 ⊕ Fin 3) (x : FermionicAlgebra M) :
    jetDeriv μ (jetDeriv ν x) = jetDeriv ν (jetDeriv μ x)
```

lemma jetDeriv_comm [source]

```
lemma jetDeriv_comm (μ ν : Fin 1 ⊕ Fin 3) :
    (jetDeriv (M := M) μ).comp (jetDeriv ν) = (jetDeriv (M := M) ν).comp (jetDeriv μ)
```

def iteratedJetDeriv [source]

The iterated total derivative `∂_s = ∂_{ν₁} ⋯ ∂_{νₙ}` along a multiset `s` of directions. It is well defined on a multiset — i.e. independent of the order in which the directions are listed — because the directional derivatives commute.

```
noncomputable def iteratedJetDeriv (s : Multiset (Fin 1 ⊕ Fin 3)) :
    FermionicAlgebra M →ₗ[ℂ] FermionicAlgebra M
```

lemma iteratedJetDeriv_zero [source]

```
@[simp]
lemma iteratedJetDeriv_zero :
    iteratedJetDeriv (0 : Multiset (Fin 1 ⊕ Fin 3))
      = LinearMap.id (R := ℂ) (M := FermionicAlgebra M)
```

lemma iteratedJetDeriv_cons [source]

```
lemma iteratedJetDeriv_cons (μ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    iteratedJetDeriv (M := M) (μ ::ₘ s) = (jetDeriv μ).comp (iteratedJetDeriv s)
```

lemma iteratedJetDeriv_cons' [source]

The companion of `iteratedJetDeriv_cons`, peeling the extra derivative on the inside.

```
lemma iteratedJetDeriv_cons' (μ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    iteratedJetDeriv (M := M) (μ ::ₘ s) = (iteratedJetDeriv s).comp (jetDeriv μ)
```

lemma iteratedJetDeriv_singleton [source]

```
@[simp]
lemma iteratedJetDeriv_singleton (μ : Fin 1 ⊕ Fin 3) :
    iteratedJetDeriv (M := M) {μ} = jetDeriv μ
```

lemma iteratedJetDeriv_add [source]

The iterated derivative is additive in the multiset of directions: differentiating along `s + t` is differentiating along `t` and then along `s`.

```
lemma iteratedJetDeriv_add (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    iteratedJetDeriv (M := M) (s + t)
      = (iteratedJetDeriv s).comp (iteratedJetDeriv t)
```

lemma iteratedJetDeriv_mul [source]

***The all-orders Leibniz rule.** The iterated derivative of a product distributes over the antidiagonal of the multiset of directions: each way of splitting the derivatives between the two factors contributes one term.*

```
lemma iteratedJetDeriv_mul (s : Multiset (Fin 1 ⊕ Fin 3)) (x y : FermionicAlgebra M) :
    iteratedJetDeriv s (x * y) =
      (s.antidiagonal.map fun p =>
        iteratedJetDeriv p.1 x * iteratedJetDeriv p.2 y).sum
```

lemma iteratedJetDeriv_one_of_ne_zero [source]

A nonempty iterated derivative kills the constants.

```
lemma iteratedJetDeriv_one_of_ne_zero {s : Multiset (Fin 1 ⊕ Fin 3)} (hs : s ≠ 0) :
    iteratedJetDeriv (M := M) s (1 : FermionicAlgebra M) = 0
```

lemma iteratedJetDeriv_ι [source]

***On a component function the iterated derivative is the derivative symbol `∂_s`.** Both halves of the component space — the field and its conjugate — are multiplied by the degree-`|s|` element `∂_s` of `SpaceTimeDerivAlgebraℂ` in their derivative-label factor, with the target index untouched.*

```
lemma iteratedJetDeriv_ι (s : Multiset (Fin 1 ⊕ Fin 3)) (x : JetComponentSpace M) :
    iteratedJetDeriv s (ExteriorAlgebra.ι ℂ x) =
      ExteriorAlgebra.ι ℂ
        (TensorProduct.map (LinearMap.mulRight ℂ (SpaceTimeDerivAlgebraℂ.basis s))
            LinearMap.id x.1,
          TensorProduct.map (LinearMap.mulRight ℂ (SpaceTimeDerivAlgebraℂ.basis s))
            LinearMap.id x.2)
```

lemma iteratedJetDeriv_ofField [source]

The iterated derivative of the field is the generator carrying the derivative symbol `∂_s`: applying `∂_s` to `ψ_φ` writes the label `s` into the derivative factor.

```
@[simp]
lemma iteratedJetDeriv_ofField (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ M.V) :
    iteratedJetDeriv s (ofField φ) =
      ExteriorAlgebra.ι ℂ ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace M)
```

lemma iteratedJetDeriv_ofConjField [source]

The iterated derivative of the conjugate field is the conjugate generator carrying the derivative symbol `∂_s`.

```
@[simp]
lemma iteratedJetDeriv_ofConjField (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule M.V)) :
    iteratedJetDeriv s (ofConjField φ) =
      ExteriorAlgebra.ι ℂ ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace M)
```

lemma adjoin_iteratedJetDeriv_eq_top [source]

***The fermionic algebra is generated by the field, its conjugate, and their derivatives.** As a `ℂ`-algebra, `FermionicAlgebra M` is the algebra adjoined by the iterated total derivatives `∂_s ψ_φ` and `∂_s ψ̄_φ` of the undifferentiated component functions. Physically: every Lagrangian term for the matter field `M` is a polynomial in the field, its conjugate, and their spacetime derivatives — nothing else is available to write down.*

This sharpens `adjoin_ι_eq_top`, which only says the algebra is generated by the component functions; here the component functions are themselves produced from the two inclusions `ofField` and `ofConjField` by differentiating.

```
lemma adjoin_iteratedJetDeriv_eq_top :
    Algebra.adjoin ℂ
      (⋃ s : Multiset (Fin 1 ⊕ Fin 3),
        Set.range (fun φ : Module.Dual ℂ M.V => iteratedJetDeriv s (ofField φ)) ∪
          Set.range (fun φ : Module.Dual ℂ (ConjModule M.V) =>
            iteratedJetDeriv s (ofConjField φ)))
      = (⊤ : Subalgebra ℂ (FermionicAlgebra M))
```

lemma comap_jetDeriv [source]

***The inclusion of a species is a map of differential algebras.** Pulling back along a map of target spaces commutes with the total derivative: the two act on different labels of a component function.*

```
lemma comap_jetDeriv (f : M.V →ₗ[ℂ] N.V) (μ : Fin 1 ⊕ Fin 3) (x : FermionicAlgebra N) :
    comap f (jetDeriv μ x) = jetDeriv μ (comap f x)
```

lemma comap_iteratedJetDeriv [source]

The inclusion of a species commutes with the iterated total derivative.

```
lemma comap_iteratedJetDeriv (f : M.V →ₗ[ℂ] N.V) (s : Multiset (Fin 1 ⊕ Fin 3))
    (x : FermionicAlgebra N) :
    comap f (iteratedJetDeriv s x) = iteratedJetDeriv s (comap f x)
```

2.504 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.LorentzAction

PhyslibAlpha/Particles/StandardModel/Matter/Fermionic/LorentzAction.lean on GitHub.

def repLorentzGroup [source]

***The Lorentz action on the fermionic algebra** of the matter field `M`, induced from a representation `M.repLorentz` of `SL(2,ℂ)` on `V`: the exterior-algebra functor applied to the Lorentz action on the jet component space.*

```
noncomputable def repLorentzGroup :
    Representation ℂ SL(2,ℂ) (FermionicAlgebra M) where
```

lemma repLorentzGroup_apply [source]

```
lemma repLorentzGroup_apply (Λ : SL(2,ℂ))
    (x : FermionicAlgebra M) :
    repLorentzGroup M Λ x =
      ExteriorAlgebra.map (JetComponentSpace.repLorentzGroup M Λ) x
```

lemma repLorentzGroup_apply_one [source]

```
@[simp]
lemma repLorentzGroup_apply_one (Λ : SL(2,ℂ)) :
    repLorentzGroup M Λ (1 : FermionicAlgebra M) = 1
```

lemma repLorentzGroup_apply_mul [source]

```
lemma repLorentzGroup_apply_mul (Λ : SL(2,ℂ))
    (x y : FermionicAlgebra M) :
    repLorentzGroup M Λ (x * y)
      = repLorentzGroup M Λ x * repLorentzGroup M Λ y
```

lemma repLorentzGroup_ι [source]

On a component function the Lorentz action is the action on the component space.

```
@[simp]
lemma repLorentzGroup_ι (Λ : SL(2,ℂ))
    (v : JetComponentSpace M) :
    repLorentzGroup M Λ (ExteriorAlgebra.ι ℂ v) =
      ExteriorAlgebra.ι ℂ (JetComponentSpace.repLorentzGroup M Λ v)
```

def repLorentzGroupAlgHom [source]

The Lorentz action as an algebra homomorphism: it preserves the exterior product, so a Lorentz transformation acts on a Lagrangian term factor by factor.

```
noncomputable def repLorentzGroupAlgHom (Λ : SL(2,ℂ)) :
    FermionicAlgebra M →ₐ[ℂ] FermionicAlgebra M where
```

lemma repLorentzGroup_ofField [source]

***`ofField` is `SL(2,ℂ)`-equivariant.** The undifferentiated component functions carry the contragredient of the representation on the target space, and no derivative labels are generated: `ofField` intertwines `M.repLorentz.dual` with the action on the fermionic algebra.*

```
@[simp]
lemma repLorentzGroup_ofField (Λ : SL(2,ℂ))
    (φ : Module.Dual ℂ M.V) :
    repLorentzGroup M Λ (ofField φ) = ofField (M.repLorentz.dual Λ φ)
```

lemma repLorentzGroup_ofConjField [source]

***`ofConjField` is `SL(2,ℂ)`-equivariant**, for the conjugate of the representation on the target space: the conjugate component functions transform by `star` of the spinor matrix.*

```
@[simp]
lemma repLorentzGroup_ofConjField (Λ : SL(2,ℂ))
    (φ : Module.Dual ℂ (ConjModule M.V)) :
    repLorentzGroup M Λ (ofConjField φ) = ofConjField (M.repLorentz.conj.dual Λ φ)
```

lemma repLorentzGroup_jetDeriv [source]

***The total derivative on the fermionic algebra is a Lorentz vector.** The four derivations `∂_μ` transform into each other by the columns of the Lorentz matrix of `Λ`, exactly as the covector index `μ` should.*

```
lemma repLorentzGroup_jetDeriv (Λ : SL(2,ℂ))
    (μ : Fin 1 ⊕ Fin 3) (x : FermionicAlgebra M) :
    repLorentzGroup M Λ (jetDeriv μ x) =
      ∑ a, (((Lorentz.SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
        jetDeriv a (repLorentzGroup M Λ x)
```

instance instIsLorentzDeriv [source]

The total derivatives on the fermionic algebra form a Lorentz derivative, giving access to the boost-weight machinery of `PhyslibAlpha.Relativity.IsLorentzDeriv`.

```
instance instIsLorentzDeriv :
    Lorentz.IsLorentzDeriv (repLorentzGroup M) (jetDeriv (M := M)) where
```

2.505 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.MassDim

PhyslibAlpha/Particles/StandardModel/Matter/Fermionic/MassDim.lean on GitHub.

def massWeightScale [source]

***The mass-weight scaling on the fermionic algebra** of a field of mass weight `w`: the algebra endomorphism scaling the generator `∂_s ψ_α` by `c ^ (w + 2 |s|)`, the functorial lift of the scaling on the jet component space.*

```
noncomputable def massWeightScale (w : ℕ) (c : ℂ) :
    FermionicAlgebra M →ₐ[ℂ] FermionicAlgebra M
```

lemma massWeightScale_ι [source]

```
@[simp]
lemma massWeightScale_ι (w : ℕ) (c : ℂ) (x : JetComponentSpace M) :
    massWeightScale w c (ExteriorAlgebra.ι ℂ x)
      = ExteriorAlgebra.ι ℂ (JetComponentSpace.massWeightScale w c x)
```

lemma massWeightScale_ofField [source]

The undifferentiated field carries its own mass weight.

```
@[simp]
lemma massWeightScale_ofField (w : ℕ) (c : ℂ) (φ : Module.Dual ℂ M.V) :
    massWeightScale w c (ofField φ) = c ^ w • ofField φ
```

lemma massWeightScale_ofConjField [source]

The undifferentiated conjugate field carries the same mass weight as the field.

```
@[simp]
lemma massWeightScale_ofConjField (w : ℕ) (c : ℂ) (φ : Module.Dual ℂ (ConjModule M.V)) :
    massWeightScale w c (ofConjField φ) = c ^ w • ofConjField φ
```

lemma massWeightScale_jetDeriv [source]

***A total derivative adds mass weight two**: the scaling intertwines the total derivative up to a factor `c ^ 2`.*

```
lemma massWeightScale_jetDeriv (w : ℕ) (c : ℂ) (μ : Fin 1 ⊕ Fin 3)
    (x : FermionicAlgebra M) :
    massWeightScale w c (jetDeriv μ x) = c ^ 2 • jetDeriv μ (massWeightScale w c x)
```

lemma massWeightScale_iteratedJetDeriv [source]

The iterated derivative `∂_s` adds mass weight `2 |s|`.

```
lemma massWeightScale_iteratedJetDeriv (w : ℕ) (c : ℂ) (s : Multiset (Fin 1 ⊕ Fin 3))
    (x : FermionicAlgebra M) :
    massWeightScale w c (iteratedJetDeriv s x)
      = c ^ (2 * Multiset.card s) • iteratedJetDeriv s (massWeightScale w c x)
```

2.506 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.MassWeightPoly

PhyslibAlpha/Particles/StandardModel/Matter/Fermionic/MassWeightPoly.lean on GitHub.

def monomial$_l$ [source]

The monomial map into polynomials over the fermionic algebra, as a map of `ℂ`-modules rather than of `FermionicAlgebra M`-modules.

```
noncomputable def monomialₗ (n : ℕ) :
    FermionicAlgebra M →ₗ[ℂ] Polynomial (FermionicAlgebra M)
```

lemma monomial$_l$_apply [source]

```
@[simp]
lemma monomialₗ_apply (n : ℕ) (x : FermionicAlgebra M) :
    monomialₗ n x = Polynomial.monomial n x
```

def halfPoly [source]

One half of the mass-weight polynomial on the jet component space, for a field of mass weight `w`: the linear map sending the symbol `∂_s ψ` to `X ^ (w + 2 |s|)` times its image under `k`. The two halves of the component space differ only in the inclusion `k` of the symbols into the fermionic algebra, so both are instances of this map.

```
noncomputable def halfPoly {W : Type} [AddCommGroup W] [Module ℂ W] (w : ℕ)
    (k : SpaceTimeDerivAlgebraℂ ⊗[ℂ] W →ₗ[ℂ] FermionicAlgebra M) :
    SpaceTimeDerivAlgebraℂ ⊗[ℂ] W →ₗ[ℂ] Polynomial (FermionicAlgebra M)
```

lemma halfPoly_basis_tmul [source]

On the symbol `∂_s ψ` the half mass-weight polynomial is the monomial of degree `w + 2 |s|`: the field contributes `w` and each derivative two.

```
lemma halfPoly_basis_tmul {W : Type} [AddCommGroup W] [Module ℂ W] (w : ℕ)
    (k : SpaceTimeDerivAlgebraℂ ⊗[ℂ] W →ₗ[ℂ] FermionicAlgebra M)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (x : W) :
    halfPoly w k (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] x) =
      Polynomial.monomial (w + 2 * Multiset.card s)
```

```
        (k (SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] x))
```

def ιFst [source]

The inclusion of the unconjugated symbols into the fermionic algebra.

```
noncomputable def ιFst : SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ M.V →ₗ[ℂ] FermionicAlgebra M
```

def ιSnd [source]

The inclusion of the conjugate symbols into the fermionic algebra.

```
noncomputable def ιSnd :
    SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule M.V) →ₗ[ℂ] FermionicAlgebra M
```

def jetComponentPoly [source]

The mass-weight polynomial of a component function of a field of mass weight `w`: the sum of the two half maps, one for the field and one for its conjugate.

```
noncomputable def jetComponentPoly (w : ℕ) :
    JetComponentSpace M →ₗ[ℂ] Polynomial (FermionicAlgebra M)
```

lemma jetComponentPoly_apply [source]

```
lemma jetComponentPoly_apply (w : ℕ) (x : JetComponentSpace M) :
    jetComponentPoly w x = halfPoly w ιFst x.1 + halfPoly w ιSnd x.2
```

lemma jetComponentPoly_inl [source]

On an unconjugated derivative monomial the component map is a monomial eigenvector.

```
@[simp]
lemma jetComponentPoly_inl (w : ℕ) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ M.V) :
    jetComponentPoly w ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace M) =
      Polynomial.monomial (w + 2 * Multiset.card s)
        (ExteriorAlgebra.ι ℂ
          ((SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ, 0) : JetComponentSpace M))
```

lemma jetComponentPoly_inr [source]

On a conjugate derivative monomial the component map is a monomial eigenvector.

```
@[simp]
lemma jetComponentPoly_inr (w : ℕ) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule M.V)) :
    jetComponentPoly w ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace M) =
      Polynomial.monomial (w + 2 * Multiset.card s)
        (ExteriorAlgebra.ι ℂ
          ((0, SpaceTimeDerivAlgebraℂ.basis s ⊗ₜ[ℂ] φ) : JetComponentSpace M))
```

lemma basisTmul_span_top [source]

The derivative monomials span a tensor product with the derivative algebra: this is the spanning set on which the square-zero condition is checked.

```
private lemma basisTmul_span_top {W : Type} [AddCommGroup W] [Module ℂ W] :
    Submodule.span ℂ (Set.range fun p : Multiset (Fin 1 ⊕ Fin 3) × W =>
      SpaceTimeDerivAlgebraℂ.basis p.1 ⊗ₜ[ℂ] p.2) = ⊤
```

def generators [source]

The set of derivative monomials in the jet component space: the unconjugated symbols `∂_s ψ_φ` together with the conjugate symbols `∂_s ψ̄_φ`.

```
def generators (M : MatterField jets) : Set (JetComponentSpace M)
```

lemma span_generators_eq_top [source]

The derivative monomials span the jet component space: every component function is the sum of its two halves, and each half is spanned by derivative monomials.

```
lemma span_generators_eq_top : Submodule.span ℂ (generators M) = ⊤
```

lemma exists_jetComponentPoly_eq_monomial [source]

Every derivative monomial is a monomial eigenvector of the component map: this is the only property of the component map that the square-zero argument uses.

```
lemma exists_jetComponentPoly_eq_monomial (w : ℕ) {v : JetComponentSpace M}
    (hv : v ∈ generators M) :
    ∃ n : ℕ, jetComponentPoly w v = Polynomial.monomial n (ExteriorAlgebra.ι ℂ v)
```

lemma jetComponentPoly_mul_self [source]

The component map squares to zero, as the universal property of the exterior algebra demands. The symmetrised square is a bilinear form, so it is enough to check that it vanishes on the derivative monomials, where it is a monomial multiple of `ExteriorAlgebra.ι_add_mul_swap`; halving then gives the square itself.

```
lemma jetComponentPoly_mul_self (w : ℕ) (v : JetComponentSpace M) :
    jetComponentPoly w v * jetComponentPoly w v = 0
```

def massWeightPoly [source]

The mass-weight polynomial on the fermionic algebra of a field of mass weight `w`: the `ℂ`-algebra map sending a generator of mass weight `n` to `X ^ n` times itself. It is `FermionicAlgebra.massWeightScale` with the scalar replaced by the formal variable `X`.

```
noncomputable def massWeightPoly (w : ℕ) :
    FermionicAlgebra M →ₐ[ℂ] Polynomial (FermionicAlgebra M)
```

lemma massWeightPoly_ι [source]

On a component function the mass-weight polynomial is the component-function map.

```
@[simp]
lemma massWeightPoly_ι (w : ℕ) (x : JetComponentSpace M) :
    massWeightPoly w (ExteriorAlgebra.ι ℂ x) = jetComponentPoly w x
```

lemma massWeightPoly_iteratedJetDeriv_ofField [source]

The generator `∂_s ψ_φ` is a monomial eigenvector of mass weight `w + 2 |s|`: the field carries its own mass weight and each derivative adds two.

```
lemma massWeightPoly_iteratedJetDeriv_ofField (w : ℕ) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ M.V) :
    massWeightPoly w (iteratedJetDeriv s (ofField φ)) =
      Polynomial.monomial (w + 2 * Multiset.card s) (iteratedJetDeriv s (ofField φ))
```

lemma massWeightPoly_iteratedJetDeriv_ofConjField [source]

The conjugate generator `∂_s ψ̄_φ` is a monomial eigenvector of the same mass weight `w + 2 |s|` as the generator it conjugates.

```
lemma massWeightPoly_iteratedJetDeriv_ofConjField (w : ℕ) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule M.V)) :
    massWeightPoly w (iteratedJetDeriv s (ofConjField φ)) =
      Polynomial.monomial (w + 2 * Multiset.card s)
        (iteratedJetDeriv s (ofConjField φ))
```

lemma massWeightPoly_ofField [source]

The undifferentiated field has mass weight `w`.

```
lemma massWeightPoly_ofField (w : ℕ) (φ : Module.Dual ℂ M.V) :
    massWeightPoly w (ofField φ) = Polynomial.monomial w (ofField φ)
```

lemma massWeightPoly_ofConjField [source]

The undifferentiated conjugate field has mass weight `w`.

```
lemma massWeightPoly_ofConjField (w : ℕ) (φ : Module.Dual ℂ (ConjModule M.V)) :
    massWeightPoly w (ofConjField φ) = Polynomial.monomial w (ofConjField φ)
```

lemma halfPoly_eval_one [source]

Setting the formal variable to one collapses a half mass-weight polynomial back to the symbol it graded. The derivative monomials span, so it is enough to check this on the multiset basis.

```
lemma halfPoly_eval_one {W : Type} [AddCommGroup W] [Module ℂ W] (w : ℕ)
    (k : SpaceTimeDerivAlgebra ℂ ⊗[ℂ] W →ₗ[ℂ] FermionicAlgebra M)
    (y : SpaceTimeDerivAlgebra ℂ ⊗[ℂ] W) : (halfPoly w k y).eval 1 = k y
```

lemma jetComponentPoly_eval_one [source]

Setting the formal variable to one recovers the component function.

```
lemma jetComponentPoly_eval_one (w : ℕ) (x : JetComponentSpace M) :
    (jetComponentPoly w x).eval 1 = ExteriorAlgebra.ι ℂ x
```

lemma massWeightPoly_eval_one [source]

Setting the formal variable to one recovers the original element: the mass-weight pieces of an element sum back to it.

```
lemma massWeightPoly_eval_one (w : ℕ) (a : FermionicAlgebra M) :
    (massWeightPoly w a).eval 1 = a
```

lemma massWeightPoly_injective [source]

The mass-weight polynomial is injective: an element is recovered from its graded pieces. It is not surjective, since a monomial of the wrong degree is not the grading of anything.

```
lemma massWeightPoly_injective (w : ℕ) :
    Function.Injective (massWeightPoly (M := M) w)
```

2.507 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.Prod

PhyslibAlpha/Particles/StandardModel/Matter/Fermionic/Prod.lean on GitHub.

abbrev evenOdd [source]

***The Fermi-parity grading** of the fermionic algebra: the `ZMod 2` grading of the exterior algebra by the number of component functions in a monomial. An even element commutes with everything; two odd elements anticommute.*

```
abbrev FermionicAlgebra.evenOdd (M : MatterField jets) :
    ZMod 2 → Submodule ℂ (FermionicAlgebra M)
```

def prodEquiv [source]

***The fermionic algebra of a direct sum is the exterior product of the fermionic algebras.** Two matter fields taken together are one field valued in the direct sum of their target spaces, and its fermionic algebra is the graded tensor product of theirs.*

*The tensor product must be the *graded* one `ᵍ⊗`: an ordinary `⊗[ℂ]` would make a generator of the first field commute with a generator of the second, whereas fermionic generators anticommute across species just as they do within one.*

```
noncomputable def FermionicAlgebra.prodEquiv (M N : MatterField jets)
    (h : M.massWeight = N.massWeight) (hσ : M.statistic = N.statistic) :
    FermionicAlgebra (M.prod N h hσ) ≃ₐ[ℂ]
      (FermionicAlgebra.evenOdd M ᵍ⊗[ℂ] FermionicAlgebra.evenOdd N)
```

2.508 PhyslibAlpha.Particles.StandardModel.Matter.Fermionic.TransformsIn

PhyslibAlpha/Particles/StandardModel/Matter/Fermionic/TransformsIn.lean on GitHub.

lemma sum_inl [source]

A multiset sum in the unconjugated half of the component space passes through the inclusion of the generators.

```
private lemma sum_inl (m : Multiset (SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ M.V)) :
    ExteriorAlgebra.ι ℂ ((m.sum, 0) : JetComponentSpace M)
      = (m.map fun a => ExteriorAlgebra.ι ℂ ((a, 0) : JetComponentSpace M)).sum
```

lemma sum_inr [source]

A multiset sum in the conjugate half of the component space passes through the inclusion of the generators.

```
private lemma sum_inr (m : Multiset (SpaceTimeDerivAlgebraℂ ⊗[ℂ] Module.Dual ℂ (ConjModule M.V)
    )) :
    ExteriorAlgebra.ι ℂ ((0, m.sum) : JetComponentSpace M)
      = (m.map fun a => ExteriorAlgebra.ι ℂ ((0, a) : JetComponentSpace M)).sum
```

lemma repJetGaugeGroupI_iteratedJetDeriv_ofField [source]

The transformation law of the derivative generators of a matter field: a jet of gauge transformations mixes `∂_s ψ_φ` with the lower generators, each splitting `s = s₁ + s₂` of the derivative multiset contributing the base-point Taylor coefficient of the gauge jet at `s₁` acting on the target index of `∂_{s₂} ψ_φ`. There is no inhomogeneous term: unlike a gauge field, a matter field transforms linearly.

```
lemma repJetGaugeGroupI_iteratedJetDeriv_ofField
    (U : GJ) (φ : Module.Dual ℂ M.V) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    repJetGaugeGroupI M U (iteratedJetDeriv s (ofField φ)) =
      (s.antidiagonal.map fun p =>
        iteratedJetDeriv p.2
          (ofField (GaugeAlgebraRealization.repDualCoeff M.repJet U⁻¹ p.1 φ))).sum
```

lemma transformsIn_iteratedJetDeriv_ofField [source]

The derivative generators of a matter field transform in the representation `rep` carried by its jets, in the sense demanded by `LocalGaugeData.TransformsIn`.

```
lemma transformsIn_iteratedJetDeriv_ofField :
    LocalGaugeData.TransformsIn (repJetGaugeGroupI M) M.repJet
      fun s => (iteratedJetDeriv s).comp (ofField (M := M))
```

lemma repJetGaugeGroupI_iteratedJetDeriv_ofConjField [source]

The transformation law of the derivative generators of the conjugate matter field. It is the law of the field itself for the conjugate action `JetComponentSpace.repConj M.repJet` on the jets of the conjugate field — the physicists' `ψ⁻ ↦ ψ⁻ U†` and its derivatives.

```
lemma repJetGaugeGroupI_iteratedJetDeriv_ofConjField
    (U : GJ) (φ : Module.Dual ℂ (ConjModule M.V))
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    repJetGaugeGroupI M U (iteratedJetDeriv s (ofConjField φ)) =
      (s.antidiagonal.map fun p =>
        iteratedJetDeriv p.2
```

```
        (ofConjField
          (GaugeAlgebraRealization.repDualCoeff (JetComponentSpace.repConj M.repJet) U⁻¹ p.1
            φ))).sum
```

lemma transformsIn_iteratedJetDeriv_ofConjField [source]

The derivative generators of the conjugate matter field transform in the conjugate representation `JetComponentSpace.repConj M.repJet`, in the sense demanded by `LocalGaugeData.TransformsIn`.

```
lemma transformsIn_iteratedJetDeriv_ofConjField :
    LocalGaugeData.TransformsIn (repJetGaugeGroupI M) (JetComponentSpace.repConj M.repJet)
      fun s => (iteratedJetDeriv s).comp (ofConjField (M := M))
```

2.509 PhyslibAlpha.Particles.StandardModel.Model.Consistency

PhyslibAlpha/Particles/StandardModel/Model/Consistency.lean on GitHub.

lemma localGaugeData_eq [source]

The local gauge data assembled from the factors of the table is the existing local gauge data of the Standard Model.

```
lemma localGaugeData_eq : StandardModel.localGaugeData = gaugeData
```

lemma quarkDoublet_rep_mat [source]

The quark doublet acts on its colour–weak index by the existing matrix `u · (U₃ ⊗ U₂)` of the quark doublet.

```
lemma quarkDoublet_rep_mat (U : JetGaugeGroupI) :
    quarkDoublet.rep.mat U = QuarkDoublet.jetGaugeMatrix U
```

2.510 PhyslibAlpha.Particles.StandardModel.Model.LeptonDoublet

PhyslibAlpha/Particles/StandardModel/Model/LeptonDoublet.lean on GitHub.

lemma leptonDoublet_toMatterField_eq [source]

***The matter field of the datum is the matter field of the species**: the table's description and the species' description of the lepton doublet are one definition.*

```
lemma leptonDoublet_toMatterField_eq :
    leptonDoublet.toMatterField = LeptonDoubletV.matterField
```

2.511 PhyslibAlpha.Particles.StandardModel.Realization.Basic

PhyslibAlpha/Particles/StandardModel/Realization/Basic.lean on GitHub.

structure AlgebraRealization [source]

The algebra `B`, with a jet gauge action, a Lorentz action and a mass-weight grading, is a Standard Model when it receives an algebra map from the jet algebra of the Standard Model which is equivariant for both actions and compatible with the grading. The

fields of the Standard Model then sit inside `B` as the images of the jet algebra's own, and every law they satisfy there is the jet algebra's own law pushed along the map.

The last two fields are not consequences of the first four: an equivariant map forces the two actions to be multiplicative only on its image, whereas the covariant derivative of a matter field needs them multiplicative on the whole of `B`.

```
structure AlgebraRealization (B : Type) [Ring B] [Algebra ℂ B]
    (repJet : Representation ℂ JetGaugeGroupI B)
    (repLorentz : Representation ℂ SL(2,ℂ) B)
    (massWeightPoly : B →ₐ[ℂ] Polynomial B) where
  toAlgHom : JetAlgebra →ₐ[ℂ] B
  map_repJet : ∀ (U : JetGaugeGroupI) (x : JetAlgebra),
    toAlgHom (JetAlgebra.repJetGaugeGroupI U x) = repJet U (toAlgHom x)
  map_repLorentz : ∀ (Λ : SL(2,ℂ)) (x : JetAlgebra),
    toAlgHom (JetAlgebra.repLorentzGroup Λ x) = repLorentz Λ (toAlgHom x)
  map_massWeight : ∀ x : JetAlgebra, massWeightPoly (toAlgHom x)
    = Polynomial.mapAlgHom toAlgHom (JetAlgebra.massWeightPoly x)
  repJet_mul : ∀ (U : JetGaugeGroupI) (b₁ b₂ : B),
    repJet U (b₁ * b₂) = repJet U b₁ * repJet U b₂
  repLorentz_mul : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
    repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂
```

`def A` [source]

The derivative symbols `∂_s A_μ^ψ` of the gauge field inside `B`.

```
noncomputable def A (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B
```

`def H` [source]

The derivative symbols `∂_s H_φ` of the Higgs field inside `B`.

```
noncomputable def H (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ HiggsVec →ₗ[ℂ] B
```

`def barH` [source]

The derivative symbols `∂_s H̄_φ` of the conjugate Higgs field inside `B`.

```
noncomputable def barH (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule HiggsVec) →ₗ[ℂ] B
```

`def d` [source]

The derivative symbols of the `i`-th generation down-type quark singlet inside `B`.

```
noncomputable def d (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ DownSinglet →ₗ[ℂ] B
```

`def bard` [source]

The derivative symbols of the `i`-th generation conjugate down-type quark singlet inside `B`.

```
noncomputable def bard (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule DownSinglet) →ₗ[ℂ] B
```

def u [source]

The derivative symbols of the `i`-th generation up-type quark singlet inside `B`.

```
noncomputable def u (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ UpSinglet →ₗ[ℂ] B
```

def baru [source]

The derivative symbols of the `i`-th generation conjugate up-type quark singlet inside `B`.

```
noncomputable def baru (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule UpSinglet) →ₗ[ℂ] B
```

def Q [source]

The derivative symbols of the `i`-th generation quark doublet inside `B`.

```
noncomputable def Q (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ QuarkDoublet →ₗ[ℂ] B
```

def barQ [source]

The derivative symbols of the `i`-th generation conjugate quark doublet inside `B`.

```
noncomputable def barQ (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule QuarkDoublet) →ₗ[ℂ] B
```

def L [source]

The derivative symbols of the `i`-th generation lepton doublet inside `B`.

```
noncomputable def L (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ LeptonDoubletV →ₗ[ℂ] B
```

def barL [source]

The derivative symbols of the `i`-th generation conjugate lepton doublet inside `B`.

```
noncomputable def barL (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule LeptonDoubletV) →ₗ[ℂ] B
```

def e [source]

The derivative symbols of the `i`-th generation charged-lepton singlet inside `B`.

```
noncomputable def e (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ LeptonSinglet →ₗ[ℂ] B
```

def bare [source]

The derivative symbols of the `i`-th generation conjugate charged-lepton singlet inside `B`.

```
noncomputable def bare (i : Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule LeptonSinglet) →ₗ[ℂ] B
```

lemma map_family_repJet [source]

A jet gauge transformation law transports along the defining map: the convolution is a multiset sum, and the map is additive and equivariant.

```
private lemma map_family_repJet {V : Type} [AddCommGroup V] [Module ℂ V]
    {rep : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] V)}
    {G : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] JetAlgebra}
    (hG : LocalGaugeData.TransformsIn (B := JetAlgebra) JetAlgebra.repJetGaugeGroupI rep G) :
    LocalGaugeData.TransformsIn repJet rep fun s => h.toAlgHom.toLinearMap ∘ₗ G s
```

lemma map_family_repLorentz [source]

A Lorentz transformation law transports along the defining map: the slot mixing is a finite sum of scalar multiples, and the map is linear and equivariant.

```
private lemma map_family_repLorentz {V : Type} [AddCommGroup V] [Module ℂ V]
    {rep : Representation ℂ SL(2,ℂ) V}
    {G : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] JetAlgebra}
    (hG : IsLorentzDerivTransforms (A := JetAlgebra) JetAlgebra.repLorentzGroup rep G) :
    IsLorentzDerivTransforms repLorentz rep fun s => h.toAlgHom.toLinearMap ∘ₗ G s
```

def gaugeRealization [source]

The gauge-boson part of a Standard Model: the realization of the gauge-boson jet algebra in `B` through the gauge sector of the jet algebra.

```
noncomputable def gaugeRealization :
    GaugeAlgebraRealization localGaugeData B repJet repLorentz where
```

lemma gaugeRealization_A [source]

```
lemma gaugeRealization_A : h.gaugeRealization.A = h.A
```

lemma repJet_H [source]

The law `repJet_H` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_H : LocalGaugeData.TransformsIn repJet HiggsVec.repJetGaugeGroupI h.H
```

lemma repJet_barH [source]

The law `repJet_barH` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_barH :
    LocalGaugeData.TransformsIn repJet (JetComponentSpace.repConj HiggsVec.repJetGaugeGroupI)
      h.barH
```

lemma repJet_d [source]

The law `repJet_d` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_d : ∀ i, LocalGaugeData.TransformsIn repJet DownSinglet.repJetGaugeGroupI (h.d i)
```

lemma repJet_bard [source]

The law `repJet_bard` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_bard : ∀ i,
    LocalGaugeData.TransformsIn repJet (JetComponentSpace.repConj DownSinglet.repJetGaugeGroupI
    )
      (h.bard i)
```

lemma repJet_u [source]

The law `repJet_u` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_u : ∀ i, LocalGaugeData.TransformsIn repJet UpSinglet.repJetGaugeGroupI (h.u i)
```

lemma repJet_baru [source]

The law `repJet_baru` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_baru : ∀ i,
    LocalGaugeData.TransformsIn repJet (JetComponentSpace.repConj UpSinglet.repJetGaugeGroupI)
      (h.baru i)
```

lemma repJet_Q [source]

The law `repJet_Q` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_Q : ∀ i, LocalGaugeData.TransformsIn repJet QuarkDoublet.repJetGaugeGroupI (h.Q i)
```

lemma repJet_barQ [source]

The law `repJet_barQ` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_barQ : ∀ i,
    LocalGaugeData.TransformsIn repJet (JetComponentSpace.repConj QuarkDoublet.
    repJetGaugeGroupI)
      (h.barQ i)
```

lemma repJet_L [source]

The law `repJet_L` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_L : ∀ i, LocalGaugeData.TransformsIn repJet LeptonDoubletV.repJetGaugeGroupI (h.L
    i)
```

lemma repJet_barL [source]

The law `repJet_barL` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_barL : ∀ i,
    LocalGaugeData.TransformsIn repJet (JetComponentSpace.repConj LeptonDoubletV.
    repJetGaugeGroupI)
      (h.barL i)
```

lemma repJet_e [source]

The law `repJet_e` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_e : ∀ i, LocalGaugeData.TransformsIn repJet LeptonSinglet.repJetGaugeGroupI (h.e i
    )
```

lemma repJet_bare [source]

The law `repJet_bare` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repJet_bare : ∀ i,
    LocalGaugeData.TransformsIn repJet (JetComponentSpace.repConj LeptonSinglet.
    repJetGaugeGroupI)
      (h.bare i)
```

lemma repLorentz_H [source]

The law `repLorentz_H` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_H : IsLorentzDerivTransforms repLorentz
    (Representation.trivial ℂ SL(2,ℂ) HiggsVec) h.H
```

lemma repLorentz_barH [source]

The law `repLorentz_barH` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_barH : IsLorentzDerivTransforms repLorentz
    (Representation.trivial ℂ SL(2,ℂ) HiggsVec).conj h.barH
```

lemma repLorentz_d [source]

The law `repLorentz_d` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_d : ∀ i, IsLorentzDerivTransforms repLorentz
    DownSinglet.repLorentzGroup (h.d i)
```

lemma repLorentz_bard [source]

The law `repLorentz_bard` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_bard : ∀ i, IsLorentzDerivTransforms repLorentz
    DownSinglet.repLorentzGroup.conj (h.bard i)
```

lemma repLorentz_u [source]

The law `repLorentz_u` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_u : ∀ i, IsLorentzDerivTransforms repLorentz
    UpSinglet.repLorentzGroup (h.u i)
```

lemma repLorentz_baru [source]

The law `repLorentz_baru` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_baru : ∀ i, IsLorentzDerivTransforms repLorentz
    UpSinglet.repLorentzGroup.conj (h.baru i)
```

lemma repLorentz_Q [source]

The law `repLorentz_Q` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_Q : ∀ i, IsLorentzDerivTransforms repLorentz
    QuarkDoublet.repLorentzGroup (h.Q i)
```

lemma repLorentz_barQ [source]

The law `repLorentz_barQ` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_barQ : ∀ i, IsLorentzDerivTransforms repLorentz
    QuarkDoublet.repLorentzGroup.conj (h.barQ i)
```

lemma repLorentz_L [source]

The law `repLorentz_L` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_L : ∀ i, IsLorentzDerivTransforms repLorentz
    LeptonDoubletV.repLorentzGroup (h.L i)
```

lemma repLorentz_barL [source]

The law `repLorentz_barL` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_barL : ∀ i, IsLorentzDerivTransforms repLorentz
    LeptonDoubletV.repLorentzGroup.conj (h.barL i)
```

lemma repLorentz_e [source]

The law `repLorentz_e` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_e : ∀ i, IsLorentzDerivTransforms repLorentz
    LeptonSinglet.repLorentzGroup (h.e i)
```

lemma repLorentz_bare [source]

The law `repLorentz_bare` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma repLorentz_bare : ∀ i, IsLorentzDerivTransforms repLorentz
    LeptonSinglet.repLorentzGroup.conj (h.bare i)
```

2.512 PhyslibAlpha.Particles.StandardModel.Realization.Commutations

PhyslibAlpha/Particles/StandardModel/Realization/Commutations.lean on GitHub.

lemma map_anticomm [source]

An anticommutation transports along the defining map: the map preserves products and negation.

```
private lemma map_anticomm {x y : JetAlgebra} (hxy : x * y = -(y * x)) :
    h.toAlgHom x * h.toAlgHom y = -(h.toAlgHom y * h.toAlgHom x)
```

lemma A_comm_A [source]

The law `A_comm_A` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_A : ∀ (s s' : Multiset (Fin 1 ⊕ Fin 3)) (μ μ' : Fin 1 ⊕ Fin 3)
    (ψ ψ' : Module.Dual ℝ GaugeAlgebra), Commute (h.A s μ ψ) (h.A s' μ' ψ')
```

lemma A_comm_H [source]

The law `A_comm_H` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_H : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ
    HiggsVec),
    Commute (h.A s μ ψ) (h.H s' φ)
```

lemma A_comm_barH [source]

The law `A_comm_barH` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_barH : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ
      (ConjModule HiggsVec)),
    Commute (h.A s μ ψ) (h.barH s' φ)
```

lemma A_comm_d [source]

The law `A_comm_d` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_d : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ DownSinglet),
    Commute (h.A s μ ψ) (h.d i s' φ)
```

lemma A_comm_bard [source]

The law `A_comm_bard` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_bard : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.
    Dual ℂ
      (ConjModule DownSinglet)),
    Commute (h.A s μ ψ) (h.bard i s' φ)
```

lemma A_comm_u [source]

The law `A_comm_u` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_u : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ UpSinglet),
    Commute (h.A s μ ψ) (h.u i s' φ)
```

lemma A_comm_baru [source]

The law `A_comm_baru` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_baru : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.
    Dual ℂ
      (ConjModule UpSinglet)),
    Commute (h.A s μ ψ) (h.baru i s' φ)
```

lemma A_comm_Q [source]

The law `A_comm_Q` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_Q : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ QuarkDoublet),
    Commute (h.A s μ ψ) (h.Q i s' φ)
```

lemma A_comm_barQ [source]

The law `A_comm_barQ` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_barQ : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.
    Dual ℂ
      (ConjModule QuarkDoublet)),
    Commute (h.A s μ ψ) (h.barQ i s' φ)
```

lemma A_comm_L [source]

The law `A_comm_L` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_L : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ LeptonDoubletV),
    Commute (h.A s μ ψ) (h.L i s' φ)
```

lemma A_comm_barL [source]

The law `A_comm_barL` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_barL : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.
    Dual ℂ
      (ConjModule LeptonDoubletV)),
    Commute (h.A s μ ψ) (h.barL i s' φ)
```

lemma A_comm_e [source]

The law `A_comm_e` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_e : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3))
      (φ : Module.Dual ℂ LeptonSinglet),
    Commute (h.A s μ ψ) (h.e i s' φ)
```

lemma A_comm_bare [source]

The law `A_comm_bare` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma A_comm_bare : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.
    Dual ℂ
      (ConjModule LeptonSinglet)),
    Commute (h.A s μ ψ) (h.bare i s' φ)
```

lemma H_comm_H [source]

The Higgs is bosonic: two Higgs symbols commute.

```
lemma H_comm_H : ∀ (s s' : Multiset (Fin 1 ⊕ Fin 3)) (φ φ' : Module.Dual ℂ HiggsVec),
    Commute (h.H s φ) (h.H s' φ')
```

lemma H_comm_barH [source]

A Higgs symbol commutes with a conjugate Higgs symbol.

```
lemma H_comm_barH : ∀ (s s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec)
    (φ' : Module.Dual ℂ (ConjModule HiggsVec)),
    Commute (h.H s φ) (h.barH s' φ')
```

lemma barH_comm_barH [source]

Two conjugate Higgs symbols commute.

```
lemma barH_comm_barH : ∀ (s s' : Multiset (Fin 1 ⊕ Fin 3)) (φ φ' : Module.Dual ℂ
    (ConjModule HiggsVec)),
    Commute (h.barH s φ) (h.barH s' φ')
```

lemma H_comm_d [source]

The Higgs symbols commute with the down-type quark symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma H_comm_d : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) (i : Fin 3)
    (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ DownSinglet),
    Commute (h.H s φ) (h.d i s' φ')
```

lemma H_comm_bard [source]

The Higgs symbols commute with the conjugate down-type quark symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma H_comm_bard : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) (i : Fin 3)
    (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ (ConjModule DownSinglet)),
    Commute (h.H s φ) (h.bard i s' φ')
```

lemma H_comm_u [source]

The Higgs symbols commute with the up-type quark symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma H_comm_u : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) (i : Fin 3)
    (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ UpSinglet),
```

```
    Commute (h.H s φ) (h.u i s' φ')
```

lemma H_comm_baru [source]

The Higgs symbols commute with the conjugate up-type quark symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma H_comm_baru : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) (i : Fin 3)
    (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ (ConjModule UpSinglet)),
    Commute (h.H s φ) (h.baru i s' φ')
```

lemma H_comm_Q [source]

The Higgs symbols commute with the quark doublet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma H_comm_Q : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) (i : Fin 3)
    (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ QuarkDoublet),
    Commute (h.H s φ) (h.Q i s' φ')
```

lemma H_comm_barQ [source]

The Higgs symbols commute with the conjugate quark doublet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma H_comm_barQ : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) (i : Fin 3)
    (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)),
    Commute (h.H s φ) (h.barQ i s' φ')
```

lemma H_comm_L [source]

The Higgs symbols commute with the lepton doublet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma H_comm_L : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) (i : Fin 3)
    (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ LeptonDoubletV),
    Commute (h.H s φ) (h.L i s' φ')
```

lemma H_comm_barL [source]

The Higgs symbols commute with the conjugate lepton doublet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma H_comm_barL : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) (i : Fin 3)
    (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)),
    Commute (h.H s φ) (h.barL i s' φ')
```

lemma H_comm_e [source]

The Higgs symbols commute with the lepton singlet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma H_comm_e : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) (i : Fin 3)
    (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ LeptonSinglet),
```

```
    Commute (h.H s φ) (h.e i s' φ')
```

lemma H_comm_bare [source]

The Higgs symbols commute with the conjugate lepton singlet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma H_comm_bare : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec) (i : Fin 3)
    (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)),
    Commute (h.H s φ) (h.bare i s' φ')
```

lemma barH_comm_d [source]

The conjugate Higgs symbols commute with the down-type quark symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma barH_comm_d : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ DownSinglet),
    Commute (h.barH s φ) (h.d i s' φ')
```

lemma barH_comm_bard [source]

The conjugate Higgs symbols commute with the conjugate down-type quark symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma barH_comm_bard : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec
    ))
    (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ (ConjModule DownSinglet)),
    Commute (h.barH s φ) (h.bard i s' φ')
```

lemma barH_comm_u [source]

The conjugate Higgs symbols commute with the up-type quark symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma barH_comm_u : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ UpSinglet),
    Commute (h.barH s φ) (h.u i s' φ')
```

lemma barH_comm_baru [source]

The conjugate Higgs symbols commute with the conjugate up-type quark symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma barH_comm_baru : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec
    ))
    (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ (ConjModule UpSinglet)),
    Commute (h.barH s φ) (h.baru i s' φ')
```

lemma barH_comm_Q [source]

The conjugate Higgs symbols commute with the quark doublet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma barH_comm_Q : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ QuarkDoublet),
    Commute (h.barH s φ) (h.Q i s' φ')
```

lemma barH_comm_barQ [source]

The conjugate Higgs symbols commute with the conjugate quark doublet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma barH_comm_barQ : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec
    ))
    (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)),
    Commute (h.barH s φ) (h.barQ i s' φ')
```

lemma barH_comm_L [source]

The conjugate Higgs symbols commute with the lepton doublet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma barH_comm_L : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ LeptonDoubletV),
    Commute (h.barH s φ) (h.L i s' φ')
```

lemma barH_comm_barL [source]

The conjugate Higgs symbols commute with the conjugate lepton doublet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma barH_comm_barL : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec
    ))
    (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)
    ),
    Commute (h.barH s φ) (h.barL i s' φ')
```

lemma barH_comm_e [source]

The conjugate Higgs symbols commute with the lepton singlet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma barH_comm_e : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ LeptonSinglet),
    Commute (h.barH s φ) (h.e i s' φ')
```

lemma barH_comm_bare [source]

The conjugate Higgs symbols commute with the conjugate lepton singlet symbols: the Higgs is a boson, so it carries no statistics against the fermions.

```
lemma barH_comm_bare : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec
    ))
    (i : Fin 3) (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ (ConjModule LeptonSinglet))
    ,
    Commute (h.barH s φ) (h.bare i s' φ')
```

lemma d_anticomm_d [source]

The down-type quark symbols anticommute among themselves.

```
lemma d_anticomm_d : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ φ' : Module.Dual ℂ DownSinglet),
    h.d i s φ * h.d j s' φ' = -(h.d j s' φ' * h.d i s φ)
```

lemma d_anticomm_bard [source]

The down-type quark symbols anticommute with the conjugate down-type quark symbols.

```
lemma d_anticomm_bard : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) (φ' : Module.Dual ℂ (ConjModule DownSinglet)),
    h.d i s φ * h.bard j s' φ' = -(h.bard j s' φ' * h.d i s φ)
```

lemma d_anticomm_u [source]

The down-type quark symbols anticommute with the up-type quark symbols.

```
lemma d_anticomm_u : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet)
    (φ' : Module.Dual ℂ UpSinglet),
    h.d i s φ * h.u j s' φ' = -(h.u j s' φ' * h.d i s φ)
```

lemma d_anticomm_baru [source]

The down-type quark symbols anticommute with the conjugate up-type quark symbols.

```
lemma d_anticomm_baru : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) (φ' : Module.Dual ℂ (ConjModule UpSinglet)),
    h.d i s φ * h.baru j s' φ' = -(h.baru j s' φ' * h.d i s φ)
```

lemma d_anticomm_Q [source]

The down-type quark symbols anticommute with the quark doublet symbols.

```
lemma d_anticomm_Q : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet)
    (φ' : Module.Dual ℂ QuarkDoublet),
    h.d i s φ * h.Q j s' φ' = -(h.Q j s' φ' * h.d i s φ)
```

lemma d_anticomm_barQ [source]

The down-type quark symbols anticommute with the conjugate quark doublet symbols.

```
lemma d_anticomm_barQ : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)),
    h.d i s φ * h.barQ j s' φ' = -(h.barQ j s' φ' * h.d i s φ)
```

lemma d_anticomm_L [source]

The down-type quark symbols anticommute with the lepton doublet symbols.

```
lemma d_anticomm_L : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet)
    (φ' : Module.Dual ℂ LeptonDoubletV),
    h.d i s φ * h.L j s' φ' = -(h.L j s' φ' * h.d i s φ)
```

lemma d_anticomm_barL [source]

The down-type quark symbols anticommute with the conjugate lepton doublet symbols.

```
lemma d_anticomm_barL : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)),
    h.d i s φ * h.barL j s' φ' = -(h.barL j s' φ' * h.d i s φ)
```

lemma d_anticomm_e [source]

The down-type quark symbols anticommute with the lepton singlet symbols.

```
lemma d_anticomm_e : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet)
    (φ' : Module.Dual ℂ LeptonSinglet),
    h.d i s φ * h.e j s' φ' = -(h.e j s' φ' * h.d i s φ)
```

lemma d_anticomm_bare [source]

The down-type quark symbols anticommute with the conjugate lepton singlet symbols.

```
lemma d_anticomm_bare : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ DownSinglet) (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)),
    h.d i s φ * h.bare j s' φ' = -(h.bare j s' φ' * h.d i s φ)
```

lemma bard_anticomm_bard [source]

The conjugate down-type quark symbols anticommute among themselves.

```
lemma bard_anticomm_bard : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ φ' : Module.Dual ℂ (ConjModule DownSinglet)),
    h.bard i s φ * h.bard j s' φ' = -(h.bard j s' φ' * h.bard i s φ)
```

lemma bard_anticomm_u [source]

The conjugate down-type quark symbols anticommute with the up-type quark symbols.

```
lemma bard_anticomm_u : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) (φ' : Module.Dual ℂ UpSinglet),
    h.bard i s φ * h.u j s' φ' = -(h.u j s' φ' * h.bard i s φ)
```

lemma bard_anticomm_baru [source]

The conjugate down-type quark symbols anticommute with the conjugate up-type quark symbols.

```
lemma bard_anticomm_baru : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) (φ' : Module.Dual ℂ (ConjModule UpSinglet)),
    h.bard i s φ * h.baru j s' φ' = -(h.baru j s' φ' * h.bard i s φ)
```

lemma bard_anticomm_Q [source]

The conjugate down-type quark symbols anticommute with the quark doublet symbols.

```
lemma bard_anticomm_Q : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) (φ' : Module.Dual ℂ QuarkDoublet),
    h.bard i s φ * h.Q j s' φ' = -(h.Q j s' φ' * h.bard i s φ)
```

lemma bard_anticomm_barQ [source]

The conjugate down-type quark symbols anticommute with the conjugate quark doublet symbols.

```
lemma bard_anticomm_barQ : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) (φ' : Module.Dual ℂ (ConjModule QuarkDoublet))
    ,
    h.bard i s φ * h.barQ j s' φ' = -(h.barQ j s' φ' * h.bard i s φ)
```

lemma bard_anticomm_L [source]

The conjugate down-type quark symbols anticommute with the lepton doublet symbols.

```
lemma bard_anticomm_L : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) (φ' : Module.Dual ℂ LeptonDoubletV),
    h.bard i s φ * h.L j s' φ' = -(h.L j s' φ' * h.bard i s φ)
```

lemma bard_anticomm_barL [source]

The conjugate down-type quark symbols anticommute with the conjugate lepton doublet symbols.

```
lemma bard_anticomm_barL : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV
    )),
    h.bard i s φ * h.barL j s' φ' = -(h.barL j s' φ' * h.bard i s φ)
```

lemma bard_anticomm_e [source]

The conjugate down-type quark symbols anticommute with the lepton singlet symbols.

```
lemma bard_anticomm_e : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) (φ' : Module.Dual ℂ LeptonSinglet),
    h.bard i s φ * h.e j s' φ' = -(h.e j s' φ' * h.bard i s φ)
```

lemma bard_anticomm_bare [source]

The conjugate down-type quark symbols anticommute with the conjugate lepton singlet symbols.

```
lemma bard_anticomm_bare : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)
    ),
    h.bard i s φ * h.bare j s' φ' = -(h.bare j s' φ' * h.bard i s φ)
```

lemma u_anticomm_u [source]

The up-type quark symbols anticommute among themselves.

```
lemma u_anticomm_u : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ φ' : Module.Dual ℂ UpSinglet),
    h.u i s φ * h.u j s' φ' = -(h.u j s' φ' * h.u i s φ)
```

lemma u_anticomm_baru [source]

The up-type quark symbols anticommute with the conjugate up-type quark symbols.

```
lemma u_anticomm_baru : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet)
    (φ' : Module.Dual ℂ (ConjModule UpSinglet)),
    h.u i s φ * h.baru j s' φ' = -(h.baru j s' φ' * h.u i s φ)
```

lemma u_anticomm_Q [source]

The up-type quark symbols anticommute with the quark doublet symbols.

```
lemma u_anticomm_Q : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ
    UpSinglet)
    (φ' : Module.Dual ℂ QuarkDoublet),
    h.u i s φ * h.Q j s' φ' = -(h.Q j s' φ' * h.u i s φ)
```

lemma u_anticomm_barQ [source]

The up-type quark symbols anticommute with the conjugate quark doublet symbols.

```
lemma u_anticomm_barQ : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet)
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)),
    h.u i s φ * h.barQ j s' φ' = -(h.barQ j s' φ' * h.u i s φ)
```

lemma u_anticomm_L [source]

The up-type quark symbols anticommute with the lepton doublet symbols.

```
lemma u_anticomm_L : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ
    UpSinglet)
    (φ' : Module.Dual ℂ LeptonDoubletV),
    h.u i s φ * h.L j s' φ' = -(h.L j s' φ' * h.u i s φ)
```

lemma u_anticomm_barL [source]

The up-type quark symbols anticommute with the conjugate lepton doublet symbols.

```
lemma u_anticomm_barL : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet)
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)),
    h.u i s φ * h.barL j s' φ' = -(h.barL j s' φ' * h.u i s φ)
```

lemma u_anticomm_e [source]

The up-type quark symbols anticommute with the lepton singlet symbols.

```
lemma u_anticomm_e : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ
    UpSinglet)
    (φ' : Module.Dual ℂ LeptonSinglet),
    h.u i s φ * h.e j s' φ' = -(h.e j s' φ' * h.u i s φ)
```

lemma u_anticomm_bare [source]

The up-type quark symbols anticommute with the conjugate lepton singlet symbols.

```
lemma u_anticomm_bare : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ UpSinglet)
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)),
    h.u i s φ * h.bare j s' φ' = -(h.bare j s' φ' * h.u i s φ)
```

lemma baru_anticomm_baru [source]

The conjugate up-type quark symbols anticommute among themselves.

```
lemma baru_anticomm_baru : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ φ' : Module.Dual ℂ (ConjModule UpSinglet)),
    h.baru i s φ * h.baru j s' φ' = -(h.baru j s' φ' * h.baru i s φ)
```

lemma baru_anticomm_Q [source]

The conjugate up-type quark symbols anticommute with the quark doublet symbols.

```
lemma baru_anticomm_Q : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) (φ' : Module.Dual ℂ QuarkDoublet),
    h.baru i s φ * h.Q j s' φ' = -(h.Q j s' φ' * h.baru i s φ)
```

lemma baru_anticomm_barQ [source]

The conjugate up-type quark symbols anticommute with the conjugate quark doublet symbols.

```
lemma baru_anticomm_barQ : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)),
    h.baru i s φ * h.barQ j s' φ' = -(h.barQ j s' φ' * h.baru i s φ)
```

lemma baru_anticomm_L [source]

The conjugate up-type quark symbols anticommute with the lepton doublet symbols.

```
lemma baru_anticomm_L : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) (φ' : Module.Dual ℂ LeptonDoubletV),
    h.baru i s φ * h.L j s' φ' = -(h.L j s' φ' * h.baru i s φ)
```

lemma baru_anticomm_barL [source]

The conjugate up-type quark symbols anticommute with the conjugate lepton doublet symbols.

```
lemma baru_anticomm_barL : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV))
    ,
    h.baru i s φ * h.barL j s' φ' = -(h.barL j s' φ' * h.baru i s φ)
```

lemma baru_anticomm_e [source]

The conjugate up-type quark symbols anticommute with the lepton singlet symbols.

```
lemma baru_anticomm_e : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) (φ' : Module.Dual ℂ LeptonSinglet),
    h.baru i s φ * h.e j s' φ' = -(h.e j s' φ' * h.baru i s φ)
```

lemma baru_anticomm_bare [source]

The conjugate up-type quark symbols anticommute with the conjugate lepton singlet symbols.

```
lemma baru_anticomm_bare : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)),
    h.baru i s φ * h.bare j s' φ' = -(h.bare j s' φ' * h.baru i s φ)
```

lemma Q_anticomm_Q [source]

The quark doublet symbols anticommute among themselves.

```
lemma Q_anticomm_Q : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ φ' : Module.Dual ℂ QuarkDoublet),
    h.Q i s φ * h.Q j s' φ' = -(h.Q j s' φ' * h.Q i s φ)
```

lemma Q_anticomm_barQ [source]

The quark doublet symbols anticommute with the conjugate quark doublet symbols.

```
lemma Q_anticomm_barQ : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet) (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)),
    h.Q i s φ * h.barQ j s' φ' = -(h.barQ j s' φ' * h.Q i s φ)
```

lemma Q_anticomm_L [source]

The quark doublet symbols anticommute with the lepton doublet symbols.

```
lemma Q_anticomm_L : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet)
    (φ' : Module.Dual ℂ LeptonDoubletV),
    h.Q i s φ * h.L j s' φ' = -(h.L j s' φ' * h.Q i s φ)
```

lemma Q_anticomm_barL [source]

The quark doublet symbols anticommute with the conjugate lepton doublet symbols.

```
lemma Q_anticomm_barL : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet) (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)),
    h.Q i s φ * h.barL j s' φ' = -(h.barL j s' φ' * h.Q i s φ)
```

lemma Q_anticomm_e [source]

The quark doublet symbols anticommute with the lepton singlet symbols.

```
lemma Q_anticomm_e : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet)
    (φ' : Module.Dual ℂ LeptonSinglet),
    h.Q i s φ * h.e j s' φ' = -(h.e j s' φ' * h.Q i s φ)
```

lemma Q_anticomm_bare [source]

The quark doublet symbols anticommute with the conjugate lepton singlet symbols.

```
lemma Q_anticomm_bare : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ QuarkDoublet) (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)),
    h.Q i s φ * h.bare j s' φ' = -(h.bare j s' φ' * h.Q i s φ)
```

lemma barQ_anticomm_barQ [source]

The conjugate quark doublet symbols anticommute among themselves.

```
lemma barQ_anticomm_barQ : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ φ' : Module.Dual ℂ (ConjModule QuarkDoublet)),
    h.barQ i s φ * h.barQ j s' φ' = -(h.barQ j s' φ' * h.barQ i s φ)
```

lemma barQ_anticomm_L [source]

The conjugate quark doublet symbols anticommute with the lepton doublet symbols.

```
lemma barQ_anticomm_L : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) (φ' : Module.Dual ℂ LeptonDoubletV),
    h.barQ i s φ * h.L j s' φ' = -(h.L j s' φ' * h.barQ i s φ)
```

lemma barQ_anticomm_barL [source]

The conjugate quark doublet symbols anticommute with the conjugate lepton doublet symbols.

```
lemma barQ_anticomm_barL : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) (φ' : Module.Dual ℂ (ConjModule
    LeptonDoubletV)),
    h.barQ i s φ * h.barL j s' φ' = -(h.barL j s' φ' * h.barQ i s φ)
```

lemma barQ_anticomm_e [source]

The conjugate quark doublet symbols anticommute with the lepton singlet symbols.

```
lemma barQ_anticomm_e : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) (φ' : Module.Dual ℂ LeptonSinglet),
    h.barQ i s φ * h.e j s' φ' = -(h.e j s' φ' * h.barQ i s φ)
```

lemma barQ_anticomm_bare [source]

The conjugate quark doublet symbols anticommute with the conjugate lepton singlet symbols.

```
lemma barQ_anticomm_bare : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) (φ' : Module.Dual ℂ (ConjModule LeptonSinglet
    )),
    h.barQ i s φ * h.bare j s' φ' = -(h.bare j s' φ' * h.barQ i s φ)
```

lemma L_anticomm_L [source]

The lepton doublet symbols anticommute among themselves.

```
lemma L_anticomm_L : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ φ' : Module.Dual ℂ LeptonDoubletV),
    h.L i s φ * h.L j s' φ' = -(h.L j s' φ' * h.L i s φ)
```

lemma L_anticomm_barL [source]

The lepton doublet symbols anticommute with the conjugate lepton doublet symbols.

```
lemma L_anticomm_barL : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonDoubletV) (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)),
    h.L i s φ * h.barL j s' φ' = -(h.barL j s' φ' * h.L i s φ)
```

lemma L_anticomm_e [source]

The lepton doublet symbols anticommute with the lepton singlet symbols.

```
lemma L_anticomm_e : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonDoubletV)
    (φ' : Module.Dual ℂ LeptonSinglet),
    h.L i s φ * h.e j s' φ' = -(h.e j s' φ' * h.L i s φ)
```

lemma L_anticomm_bare [source]

The lepton doublet symbols anticommute with the conjugate lepton singlet symbols.

```
lemma L_anticomm_bare : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonDoubletV) (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)),
    h.L i s φ * h.bare j s' φ' = -(h.bare j s' φ' * h.L i s φ)
```

lemma barL_anticomm_barL [source]

The conjugate lepton doublet symbols anticommute among themselves.

```
lemma barL_anticomm_barL : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)),
    h.barL i s φ * h.barL j s' φ' = -(h.barL j s' φ' * h.barL i s φ)
```

lemma barL_anticomm_e [source]

The conjugate lepton doublet symbols anticommute with the lepton singlet symbols.

```
lemma barL_anticomm_e : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) (φ' : Module.Dual ℂ LeptonSinglet),
    h.barL i s φ * h.e j s' φ' = -(h.e j s' φ' * h.barL i s φ)
```

lemma barL_anticomm_bare [source]

The conjugate lepton doublet symbols anticommute with the conjugate lepton singlet symbols.

```
lemma barL_anticomm_bare : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) (φ' : Module.Dual ℂ (ConjModule
    LeptonSinglet))
      ,
    h.barL i s φ * h.bare j s' φ' = -(h.bare j s' φ' * h.barL i s φ)
```

lemma e_anticomm_e [source]

The lepton singlet symbols anticommute among themselves.

```
lemma e_anticomm_e : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ φ' : Module.Dual ℂ LeptonSinglet),
    h.e i s φ * h.e j s' φ' = -(h.e j s' φ' * h.e i s φ)
```

lemma e_anticomm_bare [source]

The lepton singlet symbols anticommute with the conjugate lepton singlet symbols.

```
lemma e_anticomm_bare : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ LeptonSinglet) (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)),
    h.e i s φ * h.bare j s' φ' = -(h.bare j s' φ' * h.e i s φ)
```

lemma bare_anticomm_bare [source]

The conjugate lepton singlet symbols anticommute among themselves.

```
lemma bare_anticomm_bare : ∀ (i j : Fin 3) (s s' : Multiset (Fin 1 ⊕ Fin 3))
    (φ φ' : Module.Dual ℂ (ConjModule LeptonSinglet)),
    h.bare i s φ * h.bare j s' φ' = -(h.bare j s' φ' * h.bare i s φ)
```

2.513 PhyslibAlpha.Particles.StandardModel.Realization.CovariantDeriv

PhyslibAlpha/Particles/StandardModel/Realization/CovariantDeriv.lean on GitHub.

def lorentzMixStep [source]

One peeling step of the Lorentz mixing: the direction `a` is removed from the multiset index of the family and put back as every direction `b`, weighted by `L[Λ] b a`.

```
noncomputable def lorentzMixStep (Λ : SL(2,ℂ)) (a : Fin 1 ⊕ Fin 3)
    (G : Multiset (Fin 1 ⊕ Fin 3) → M) : Multiset (Fin 1 ⊕ Fin 3) → M
```

instance LeftCommutative (lorentzMixStep (M [source]

Peeling two directions commutes, so the mixing is well defined on a multiset.

```
instance (Λ : SL(2,ℂ)) : LeftCommutative (lorentzMixStep (M := M) Λ) where
```

def lorentzMix [source]

The Lorentz mixing of a multiset-indexed family along a multiset `s` of directions: every direction of `s` is peeled and replaced by all directions, weighted by the corresponding column of the Lorentz matrix.

```
noncomputable def lorentzMix (Λ : SL(2,ℂ)) (G : Multiset (Fin 1 ⊕ Fin 3) → M)
    (s : Multiset (Fin 1 ⊕ Fin 3)) : Multiset (Fin 1 ⊕ Fin 3) → M
```

lemma lorentzMix_zero [source]

```
@[simp]
lemma lorentzMix_zero : lorentzMix Λ G 0 = G
```

lemma lorentzMix_cons_apply [source]

Mixing along `a ::ₘ s` peels `a` after mixing along `s`.

```
lemma lorentzMix_cons_apply (a : Fin 1 ⊕ Fin 3) (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ G (a ::ₘ s) t = ∑ b, L[Λ] b a • lorentzMix Λ G s (b ::ₘ t)
```

lemma lorentzMix_apply_add [source]

Evaluating a mixed family away from the empty multiset is mixing the translated family at the empty multiset.

```
lemma lorentzMix_apply_add (s : Multiset (Fin 1 ⊕ Fin 3)) :
    ∀ (G : Multiset (Fin 1 ⊕ Fin 3) → M) (t : Multiset (Fin 1 ⊕ Fin 3)),
      lorentzMix Λ G s t = lorentzMix Λ (fun r => G (r + t)) s 0
```

lemma lorentzMix_cons_zero [source]

Peeling at the empty multiset: the peeled direction is pushed into the family.

```
lemma lorentzMix_cons_zero (a : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ G (a ::ₘ s) 0 = ∑ b, L[Λ] b a • lorentzMix Λ (fun t => G (b ::ₘ t)) s 0
```

lemma lorentzMix_map [source]

The mixing operator commutes with any linear map applied to the values.

```
lemma lorentzMix_map (Φ : M →ₗ[ℂ] N) (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    Φ (lorentzMix Λ G s t) = lorentzMix Λ (fun r => Φ (G r)) s t
```

lemma lorentzMix_smul_fam [source]

The mixing operator is homogeneous in the family.

```
lemma lorentzMix_smul_fam (c : ℂ) (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ (fun r => c • G r) s t = c • lorentzMix Λ G s t
```

lemma lorentzMix_add_fam [source]

The mixing operator is additive in the family.

```
lemma lorentzMix_add_fam (G₁ G₂ : Multiset (Fin 1 ⊕ Fin 3) → M) (s t : Multiset (Fin 1 ⊕ Fin 3)
    ) :
    lorentzMix Λ (fun r => G₁ r + G₂ r) s t = lorentzMix Λ G₁ s t + lorentzMix Λ G₂ s t
```

lemma lorentzMix_sum_fam [source]

The mixing operator commutes with finite sums of families.

```
lemma lorentzMix_sum_fam {ι : Type*} [Fintype ι] (H : ι → Multiset (Fin 1 ⊕ Fin 3) → M)
    (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ (fun r => ∑ i, H i r) s t = ∑ i, lorentzMix Λ (H i) s t
```

lemma sum_fin_succ_prod_smul [source]

A sum over tuples of directions, with one Lorentz matrix factor per slot, split into its first slot and the remaining ones.

```
lemma sum_fin_succ_prod_smul {n : ℕ} (l : Fin (n + 1) → (Fin 1 ⊕ Fin 3))
    (X : (Fin (n + 1) → (Fin 1 ⊕ Fin 3)) → M) :
    ∑ q : Fin (n + 1) → (Fin 1 ⊕ Fin 3), (∏ i, L[Λ] (q i) (l i)) • X q =
      ∑ b, ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
        (L[Λ] b (l 0) * ∏ i, L[Λ] (p i) (l i.succ)) • X (Fin.cons b p)
```

lemma lorentzMix_ofFn [source]

The mixing operator agrees with the tuple form of the Lorentz law: along an ordered tuple of directions it is the sum over all tuples with one Lorentz matrix factor per slot.

```
lemma lorentzMix_ofFn {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (t : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ G (List.ofFn l) t =
      ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
        (∏ i, L[Λ] (p i) (l i)) • G ((List.ofFn p : List (Fin 1 ⊕ Fin 3)) + t)
```

lemma lorentzMix_neg_fam [source]

The mixing operator commutes with negation of the family.

```
lemma lorentzMix_neg_fam (G : Multiset (Fin 1 ⊕ Fin 3) → M) (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ (fun r => -G r) s t = -lorentzMix Λ G s t
```

lemma lorentzMix_sub_fam [source]

The mixing operator is additive in the family, in subtracted form.

```
lemma lorentzMix_sub_fam (G₁ G₂ : Multiset (Fin 1 ⊕ Fin 3) → M) (s t : Multiset (Fin 1 ⊕ Fin 3)
    ) :
    lorentzMix Λ (fun r => G₁ r - G₂ r) s t = lorentzMix Λ G₁ s t - lorentzMix Λ G₂ s t
```

lemma multiset_sum_map_sum [source]

A finite sum inside a multiset sum may be taken outside.

```
lemma multiset_sum_map_sum {α ι : Type*} [Fintype ι] (m : Multiset α) (F : ι → α → B) :
    (m.map fun x => ∑ i, F i x).sum = ∑ i, (m.map (F i)).sum
```

def derivConv [source]

The Leibniz convolution of two families of derivative symbols: the sum over the splittings of the multiset of the products of the two symbols.

```
noncomputable def derivConv (f g : Multiset (Fin 1 ⊕ Fin 3) → B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) : B
```

lemma derivConv_cons [source]

One derivative peeled off a convolution lands on one factor or the other.

```
lemma derivConv_cons (f g : Multiset (Fin 1 ⊕ Fin 3) → B) (a : Fin 1 ⊕ Fin 3)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    derivConv f g (a ::ₘ s) =
      derivConv f (fun r => g (a ::ₘ r)) s + derivConv (fun r => f (a ::ₘ r)) g s
```

lemma derivConv_sum_right [source]

The convolution is linear in its right-hand family.

```
lemma derivConv_sum_right {ι : Type*} [Fintype ι] (f : Multiset (Fin 1 ⊕ Fin 3) → B)
    (c : ι → ℂ) (g : ι → Multiset (Fin 1 ⊕ Fin 3) → B) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    derivConv f (fun r => ∑ i, c i • g i r) s = ∑ i, c i • derivConv f (g i) s
```

lemma derivConv_sum_left [source]

The convolution is linear in its left-hand family.

```
lemma derivConv_sum_left {ι : Type*} [Fintype ι] (g : Multiset (Fin 1 ⊕ Fin 3) → B)
    (c : ι → ℂ) (f : ι → Multiset (Fin 1 ⊕ Fin 3) → B) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    derivConv (fun r => ∑ i, c i • f i r) g s = ∑ i, c i • derivConv (f i) g s
```

lemma lorentzMix_derivConv [source]

The Lorentz mixing operator is a morphism for the Leibniz convolution: mixing the two factors separately and convolving is the same as convolving and then mixing.

```
lemma lorentzMix_derivConv (Λ : SL(2,ℂ)) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    ∀ (f g : Multiset (Fin 1 ⊕ Fin 3) → B),
      derivConv (fun x => lorentzMix Λ f x 0) (fun y => lorentzMix Λ g y 0) s =
        lorentzMix Λ (derivConv f g) s 0
```

lemma repLorentz_derivConv [source]

The Lorentz law of a Leibniz convolution: the mixing operator is a morphism for the convolution, so a convolution of two families with Lorentz laws has one too.

```
lemma repLorentz_derivConv
    (hmul : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B), repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz
    Λ b₂)
    (Λ : SL(2,ℂ)) (f f' g g' : Multiset (Fin 1 ⊕ Fin 3) → B)
    (hf : ∀ x, repLorentz Λ (f x) = lorentzMix Λ f' x 0)
    (hg : ∀ y, repLorentz Λ (g y) = lorentzMix Λ g' y 0) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    repLorentz Λ (derivConv f g s) = lorentzMix Λ (derivConv f' g') s 0
```

lemma repLorentz_tower [source]

The Lorentz law of a covariant tower. The tower `T` is built by the step `hstep` out of a correction `C`, and so is the tower `T'` it transforms into; the seed of `T` transforms into the seed of `T'` with the value index twisted by `τ` (`hzero`); and the correction is Lorentz covariant (`hC`), linear in the family it corrects (`hClin`), and lets the twist through (`hCτ`). Then the covariant slots mix by their own columns, the plain slots by `lorentzMix`, and the value index by `τ`.

```
lemma repLorentz_tower {K : Type} [Field K] {W : Type} [AddCommGroup W] [Module K W]
    [Module K B] [SMulCommClass K ℂ B] (Λ : SL(2,ℂ))
    (T T' : (n : ℕ) → (Fin n → (Fin 1 ⊕ Fin 3)) → Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B)
    (C : (Fin 1 ⊕ Fin 3) → (Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B) →
      Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B)
    (τ : W →ₗ[K] W)
    (hstep : ∀ (n : ℕ) (l : Fin (n + 1) → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3)),
      T (n + 1) l s = T n (fun i => l i.succ) (l 0 ::ₘ s) + C (l 0) (T n fun i => l i.succ) s)
    (hstep' : ∀ (n : ℕ) (l : Fin (n + 1) → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3)),
      T' (n + 1) l s = T' n (fun i => l i.succ) (l 0 ::ₘ s) + C (l 0) (T' n fun i => l i.succ)
    s)
    (hzero : ∀ (l : Fin 0 → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : W),
      repLorentz Λ (T 0 l s φ) = lorentzMix Λ (fun t => T' 0 l t (τ φ)) s 0)
    (hC : ∀ (ρ : Fin 1 ⊕ Fin 3) (G G' : Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B),
      (∀ y χ, repLorentz Λ (G y χ) = lorentzMix Λ (fun t => G' t χ) y 0) →
      ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : W), repLorentz Λ (C ρ G s φ) =
        ∑ a, L[Λ] a ρ • lorentzMix Λ (fun t => C a G' t φ) s 0)
    (hClin : ∀ (ρ : Fin 1 ⊕ Fin 3) {ι : Type} [Fintype ι] (c : ι → ℂ)
      (G : ι → Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B) (s : Multiset (Fin 1 ⊕ Fin 3))
      (φ : W), C ρ (fun t => ∑ i, c i • G i t) s φ = ∑ i, c i • C ρ (G i) s φ)
    (hCτ : ∀ (ρ : Fin 1 ⊕ Fin 3) (G : Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B)
      (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : W),
      C ρ (fun t => G t ∘ₗ τ) s φ = C ρ G s (τ φ)) :
    ∀ (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : W),
      repLorentz Λ (T n l s φ) =
        ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
          (∏ i, L[Λ] (p i) (l i)) • lorentzMix Λ (fun t => T' n p t (τ φ)) s 0
```

lemma sum_comm₃ [source]

Rotating a triple sum so that the innermost index comes first.

```
lemma sum_comm₃ {α β γ M : Type*} [Fintype α] [Fintype β] [Fintype γ] [AddCommMonoid M]
    (X : α → β → γ → M) : (∑ a, ∑ b, ∑ c, X a b c) = ∑ c, ∑ a, ∑ b, X a b c
```

lemma sum_derivConv_sum_fam [source]

A scalar combination of convolutions against the gauge field is linear in the right-hand families.

```
lemma sum_derivConv_sum_fam {ι κ ι' : Type} [Fintype ι] [Fintype κ] [Fintype ι']
    (f : ι → Multiset (Fin 1 ⊕ Fin 3) → B) (coef : ι → κ → ℂ) (c : ι' → ℂ)
    (g : ι' → κ → Multiset (Fin 1 ⊕ Fin 3) → B) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    ∑ j, ∑ k, coef j k • derivConv (f j) (fun y => ∑ i, c i • g i k y) s =
      ∑ i, c i • ∑ j, ∑ k, coef j k • derivConv (f j) (g i k) s
```

lemma repLorentz_eq_lorentzMix [source]

A Lorentz law in the tuple form, read on the underlying multisets: the transformed family mixes by `lorentzMix`.

```
lemma repLorentz_eq_lorentzMix (Λ : SL(2,ℂ)) (f g : Multiset (Fin 1 ⊕ Fin 3) → B)
    (hfg : ∀ (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)), repLorentz Λ (f (List.ofFn l)) =
      ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ i, L[Λ] (p i) (l i)) • g (List.ofFn p))
    (x : Multiset (Fin 1 ⊕ Fin 3)) : repLorentz Λ (f x) = lorentzMix Λ g x 0
```

lemma repLorentz_apply_mix [source]

The Lorentz law of the gauge-field symbols, in the multiset form.

```
lemma repLorentz_apply_mix (Λ : SL(2,ℂ))
    (x : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (χ : Module.Dual ℝ GaugeAlgebra) :
    repLorentz Λ (h.A x μ χ) = lorentzMix Λ (fun t => ∑ a, L[Λ] a μ • h.A t a χ) x 0
```

lemma isLorentzDerivTransforms_mix [source]

The Lorentz law of a family of derivative symbols, in the multiset form.

```
lemma isLorentzDerivTransforms_mix {rep : Representation ℂ SL(2,ℂ) V}
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B}
    (hF : IsLorentzDerivTransforms repLorentz rep F) (Λ : SL(2,ℂ))
    (x : Multiset (Fin 1 ⊕ Fin 3)) (χ : Module.Dual ℂ V) :
    repLorentz Λ (F x χ) = lorentzMix Λ (fun t => F t (rep.dual Λ χ)) x 0
```

lemma repLorentz_sum_derivConv [source]

The Lorentz law of a scalar combination of convolutions against the gauge field: the direction of the gauge field mixes by its own column, the derivative slots by `lorentzMix`, and the right-hand families are replaced by their transforms.

```
lemma repLorentz_sum_derivConv (Λ : SL(2,ℂ)) (ρ : Fin 1 ⊕ Fin 3)
    {ι κ : Type} [Fintype ι] [Fintype κ] (bg : Module.Basis ι ℝ GaugeAlgebra) (coef : ι → κ →
    ℂ)
    (g g' : κ → Multiset (Fin 1 ⊕ Fin 3) → B)
    (hg : ∀ k y, repLorentz Λ (g k y) = lorentzMix Λ (g' k) y 0) (s : Multiset (Fin 1 ⊕ Fin 3))
     :
    repLorentz Λ (∑ j, ∑ k, coef j k • derivConv (fun x => h.A x ρ (bg.coord j)) (g k) s) =
      ∑ a, L[Λ] a ρ • lorentzMix Λ
        (fun t => ∑ j, ∑ k, coef j k • derivConv (fun x => h.A x a (bg.coord j)) (g' k) t) s 0
```

lemma actionFam_apply_eq_sum [source]

The action of families expanded in bases of the gauge algebra and the value space.

```
lemma actionFam_apply_eq_sum {ι κ : Type} [Fintype ι] [Fintype κ]
    (bg : Module.Basis ι ℝ GaugeAlgebra) (bv : Module.Basis κ ℂ V)
    (f : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B) (g : Module.Dual ℂ V →ₗ[ℂ] B)
    (φ : Module.Dual ℂ V) :
    actionFam act f g φ =
      ∑ j, ∑ k, φ (act (bg j) (bv k)) • (f (bg.coord j) * g (bv.coord k))
```

lemma actionFamConv_eq_sum [source]

The derived action family expanded in bases.

```
lemma actionFamConv_eq_sum {ι κ : Type} [Fintype ι] [Fintype κ]
    (bg : Module.Basis ι ℝ GaugeAlgebra) (bv : Module.Basis κ ℂ V)
    (ρ : Fin 1 ⊕ Fin 3) (G : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    actionFamConv A act ρ G s φ = ∑ j, ∑ k, φ (act (bg j) (bv k)) •
      derivConv (fun x => A x ρ (bg.coord j)) (fun y => G y (bv.coord k)) s
```

lemma actionFamConv_sum_fam [source]

The derived action family is linear in the matter family.

```
lemma actionFamConv_sum_fam {ι : Type} [Fintype ι] (ρ : Fin 1 ⊕ Fin 3) (c : ι → ℂ)
    (H : ι → Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    actionFamConv A act ρ (fun t => ∑ i, c i • H i t) s φ =
      ∑ i, c i • actionFamConv A act ρ (H i) s φ
```

lemma repLorentz_actionFamConv [source]

The Lorentz law of the derived action family: the derivative slots mix, the direction of the gauge field mixes by its own column, and the value index is carried by the transformed matter family.

```
lemma repLorentz_actionFamConv (Λ : SL(2,ℂ)) (ρ : Fin 1 ⊕ Fin 3)
    (G G' : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (hG : ∀ y χ, repLorentz Λ (G y χ) = lorentzMix Λ (fun t => G' t χ) y 0)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    repLorentz Λ (actionFamConv h.A act ρ G s φ) =
      ∑ a, L[Λ] a ρ • lorentzMix Λ (fun t => actionFamConv h.A act a G' t φ) s 0
```

lemma dual_twist [source]

The twist of the value index past the gauge action: an endomorphism commuting with the gauge action may be moved from the dual basis onto the dual vector.

```
lemma dual_twist {κ : Type} [Fintype κ] (bv : Module.Basis κ ℂ V) (T : V →ₗ[ℂ] V)
    (hT : ∀ (c : GaugeAlgebra) (v : V), act c (T v) = T (act c v))
    (ψ : Module.Dual ℂ V →ₗ[ℂ] B) (c : GaugeAlgebra) (φ : Module.Dual ℂ V) :
    ∑ k, φ (act c (bv k)) • ψ (T.dualMap (bv.coord k)) =
      ∑ k, (T.dualMap φ) (act c (bv k)) • ψ (bv.coord k)
```

lemma actionFam_comp_dual [source]

The contragredient action may be pulled out of an action of families, provided the gauge action commutes with it on the value space.

```
lemma actionFam_comp_dual (T : V →ₗ[ℂ] V)
    (hT : ∀ (c : GaugeAlgebra) (v : V), act c (T v) = T (act c v))
    (f : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B) (g : Module.Dual ℂ V →ₗ[ℂ] B)
    (φ : Module.Dual ℂ V) :
    actionFam act f (g ∘ₗ T.dualMap) φ = actionFam act f g (T.dualMap φ)
```

lemma actionFamConv_comp_dual [source]

The contragredient action may be pulled out of a derived action family.

```
lemma actionFamConv_comp_dual (T : V →ₗ[ℂ] V)
    (hT : ∀ (c : GaugeAlgebra) (v : V), act c (T v) = T (act c v)) (ρ : Fin 1 ⊕ Fin 3)
    (K : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    actionFamConv A act ρ (fun t => K t ∘ₗ T.dualMap) s φ =
      actionFamConv A act ρ K s (T.dualMap φ)
```

lemma repLorentz_covDerivIter [source]

The Lorentz law of the iterated covariant derivative of a matter family: the ordered covariant slots mix by their own columns and the multiset of plain derivative slots mixes by `lorentzMix`, while the value index transforms contragrediently.

```
lemma repLorentz_covDerivIter {rep : Representation ℂ SL(2,ℂ) V}
    (hcomm : ∀ (c : GaugeAlgebra) (Λ : SL(2,ℂ)) (v : V), act c (rep Λ v) = rep Λ (act c v))
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (hF : IsLorentzDerivTransforms repLorentz rep F) (Λ : SL(2,ℂ))
    (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V)
    :
    repLorentz Λ (covDerivIter h.A act F n l s φ) = ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
      (∏ i, L[Λ] (p i) (l i)) •
        lorentzMix Λ (fun t => covDerivIter h.A act F n p t (rep.dual Λ φ)) s 0
```

lemma isLorentzCovDerivTransforms_covDerivIter [source]

The iterated covariant derivative of a matter family transforms as the covariant derivatives of a Lorentz-covariant field, given the Lorentz law of the bare symbols, the Lorentz law of the gauge field, and the commutation of the infinitesimal gauge action with the Lorentz action on the value space.

```
lemma isLorentzCovDerivTransforms_covDerivIter {rep : Representation ℂ SL(2,ℂ) V}
    (hcomm : ∀ (c : GaugeAlgebra) (Λ : SL(2,ℂ)) (v : V), act c (rep Λ v) = rep Λ (act c v))
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B)
    (hF : IsLorentzDerivTransforms repLorentz rep F) :
    IsLorentzCovDerivTransforms repLorentz rep (fun {n} l => covDerivIter h.A act F n l 0)
```

lemma actionConj_comm_repConj [source]

Conjugation preserves the commutation of the gauge action with the Lorentz action: both are read on the conjugate module through the same underlying maps.

```
lemma actionConj_comm_repConj (rep : Representation ℂ SL(2,ℂ) V)
    (hcomm : ∀ (c : GaugeAlgebra) (Λ : SL(2,ℂ)) (v : V), act c (rep Λ v) = rep Λ (act c v))
    (c : GaugeAlgebra) (Λ : SL(2,ℂ)) (v : ConjModule V) :
    LocalGaugeData.actionConj act c (rep.conj Λ v) =
      rep.conj Λ (LocalGaugeData.actionConj act c v)
```

lemma isLorentzCovDerivTransforms_covDerivIter_conj [source]

The Lorentz law of the covariant tower of a conjugate family, from the commutation of the gauge action with the Lorentz action of the unconjugated species.

```
lemma isLorentzCovDerivTransforms_covDerivIter_conj {rep : Representation ℂ SL(2,ℂ) V}
    (hcomm : ∀ (c : GaugeAlgebra) (Λ : SL(2,ℂ)) (v : V), act c (rep Λ v) = rep Λ (act c v))
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ (ConjModule V) →ₗ[ℂ] B)
    (hF : IsLorentzDerivTransforms repLorentz rep.conj F) :
    IsLorentzCovDerivTransforms repLorentz rep.conj
      (fun {n} l => covDerivIter h.A (LocalGaugeData.actionConj act) F n l 0)
```

lemma bracketFamConv_eq_sum [source]

The derived bracket family expanded in a basis of the gauge algebra.

```
lemma bracketFamConv_eq_sum (ρ : Fin 1 ⊕ Fin 3)
    (G : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ GaugeAlgebra) :
    bracketFamConv A ρ G s φ =
      ∑ j, ∑ k, ((φ [Module.Free.chooseBasis ℝ GaugeAlgebra j,
            Module.Free.chooseBasis ℝ GaugeAlgebra k] : ℝ) : ℂ) •
        derivConv (fun x => A x ρ ((Module.Free.chooseBasis ℝ GaugeAlgebra).coord j))
          (fun y => G y ((Module.Free.chooseBasis ℝ GaugeAlgebra).coord k)) s
```

lemma bracketFamConv_sum_fam [source]

The derived bracket family is linear in the second family.

```
lemma bracketFamConv_sum_fam {ι : Type} [Fintype ι] (ρ : Fin 1 ⊕ Fin 3) (c : ι → ℂ)
    (H : ι → Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ GaugeAlgebra) :
    bracketFamConv A ρ (fun t => ∑ i, c i • H i t) s φ =
      ∑ i, c i • bracketFamConv A ρ (H i) s φ
```

lemma repLorentz_bracketFamConv [source]

The Lorentz law of the derived bracket family.

```
lemma repLorentz_bracketFamConv (Λ : SL(2,ℂ)) (ρ : Fin 1 ⊕ Fin 3)
    (G G' : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B)
    (hG : ∀ y χ, repLorentz Λ (G y χ) = lorentzMix Λ (fun t => G' t χ) y 0)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ GaugeAlgebra) :
    repLorentz Λ (bracketFamConv h.A ρ G s φ) =
      ∑ a, L[Λ] a ρ • lorentzMix Λ (fun t => bracketFamConv h.A a G' t φ) s 0
```

lemma repLorentz_iteratedCovDerivAdjoint [source]

The Lorentz law of the iterated covariant derivative in the adjoint: the covariant slots mix by their own columns and the seed family is replaced by its transform.

```
lemma repLorentz_iteratedCovDerivAdjoint (Λ : SL(2,ℂ))
    (F F' : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B)
    (hF : ∀ x χ, repLorentz Λ (F x χ) = lorentzMix Λ (fun t => F' t χ) x 0)
    (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (x : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℝ GaugeAlgebra) :
    repLorentz Λ (iteratedCovDerivAdjoint h.A (List.ofFn l) F x φ) = ∑ p : Fin n → (Fin 1 ⊕ Fin
     3),
      (∏ i, L[Λ] (p i) (l i)) •
        lorentzMix Λ (fun t => iteratedCovDerivAdjoint h.A (List.ofFn p) F' t φ) x 0
```

lemma iteratedCovDerivAdjoint_sum_fam [source]

The iterated covariant derivative in the adjoint is linear in the seed family.

```
lemma iteratedCovDerivAdjoint_sum_fam {ι : Type} [Fintype ι] (c : ι → ℂ)
    (H : ι → Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B) :
    ∀ (l : List (Fin 1 ⊕ Fin 3)) (x : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ GaugeAlgebra
    ),
      iteratedCovDerivAdjoint A l (fun t => ∑ i, c i • H i t) x φ =
        ∑ i, c i • iteratedCovDerivAdjoint A l (H i) x φ
  | [], x, φ => by simp only [iteratedCovDerivAdjoint, LinearMap.sum_apply, LinearMap.
    smul_apply]
  | ρ :: l, x, φ => by
      have hfam : iteratedCovDerivAdjoint A l (fun t => ∑ i, c i • H i t) =
          fun t => ∑ i, c i • iteratedCovDerivAdjoint A l (H i) t
```

lemma iteratedCovDerivAdjoint_neg_fam [source]

The iterated covariant derivative is odd in the family it differentiates: the case of a one-element index in `iteratedCovDerivAdjoint_sum_fam`.

```
lemma iteratedCovDerivAdjoint_neg_fam
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B)
    (l : List (Fin 1 ⊕ Fin 3)) (x : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ GaugeAlgebra)
    :
    iteratedCovDerivAdjoint A l (fun t => - F t) x φ = - iteratedCovDerivAdjoint A l F x φ
```

lemma repLorentz_fieldStrength_mix [source]

The Lorentz law of the field strength: both covector indices mix by their columns, and the derivative slots mix by `lorentzMix`.

```
lemma repLorentz_fieldStrength_mix (Λ : SL(2,ℂ)) (μ ν : Fin 1 ⊕ Fin 3)
    (x : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ GaugeAlgebra) :
    repLorentz Λ (fieldStrength h.A μ ν x φ) =
      lorentzMix Λ (fun t => ∑ a, L[Λ] a μ • ∑ b, L[Λ] b ν • fieldStrength h.A a b t φ) x 0
```

lemma repLorentz_iteratedCovDerivAdjoint_fieldStrength [source]

The Lorentz law of the covariant tower of the field strength: the covariant slots mix by their own columns and the two covector indices of the field strength mix by theirs.

```
lemma repLorentz_iteratedCovDerivAdjoint_fieldStrength (Λ : SL(2,ℂ)) (n : ℕ)
```

```
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) :
    repLorentz Λ (iteratedCovDerivAdjoint h.A (List.ofFn l) (fieldStrength h.A μ ν) 0 φ) =
      ∑ p : Fin n → (Fin 1 ⊕ Fin 3), (∏ i, L[Λ] (p i) (l i)) • ∑ a, L[Λ] a μ • ∑ b, L[Λ] b ν •
        iteratedCovDerivAdjoint h.A (List.ofFn p) (fieldStrength h.A a b) 0 φ
```

lemma symbol_mem_adjoin [source]

A bare matter symbol is a polynomial in the gauge-field symbols and the covariant tower of its family.

```
lemma symbol_mem_adjoin {X : Set B} (hA : ∀ s μ ψ, A s μ ψ ∈ X)
    (hF : ∀ (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, covDerivIter A act F n l 0 φ ∈ X)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) : F s φ ∈ Algebra.adjoin ℂ X
```

lemma covDerivIter_mem_adjoin [source]

A symbol of a covariant tower is a polynomial in the gauge-field symbols and the bare symbols of its family.

```
lemma covDerivIter_mem_adjoin {X : Set B} (hA : ∀ s μ ψ, A s μ ψ ∈ X)
    (hF : ∀ s φ, F s φ ∈ X) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    covDerivIter A act F n l 0 φ ∈ Algebra.adjoin ℂ X
```

lemma commute_covDerivIter [source]

A symbol of a covariant tower commutes with the gauge-field symbols, as soon as the bare symbols of its family do.

```
lemma commute_covDerivIter
    (hAA : ∀ (s s' : Multiset (Fin 1 ⊕ Fin 3)) (μ μ' : Fin 1 ⊕ Fin 3)
      (ψ ψ' : Module.Dual ℝ GaugeAlgebra), Commute (A s μ ψ) (A s' μ' ψ'))
    (hAF : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (ψ : Module.Dual ℝ GaugeAlgebra
    )
      (s' : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V), Commute (A s μ ψ) (F s' φ))
    (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V)
    (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (ψ : Module.Dual ℝ GaugeAlgebra) :
    Commute (covDerivIter A act F n l 0 φ) (A p μ ψ)
```

lemma repDualCoeff_zero_of_mem_truncationKer_zero [source]

A pure gauge jet acts trivially through the dual base-point coefficient of a representation whose zeroth Taylor coefficient is the identity on pure jets.

```
lemma repDualCoeff_zero_of_mem_truncationKer_zero
    {rep : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] V)}
    (hrep : ∀ {W : JetGaugeGroupI}, localGaugeData.eval W = 1 → repCoeff rep W 0 = LinearMap.id
    )
    (U : localGaugeData.truncationKer 0) (φ : Module.Dual ℂ V) :
    repDualCoeff rep U.1⁻¹ 0 φ = φ
```

def symbols [source]

The generators of the field algebra: every derivative symbol of the gauge field, of the Higgs and its conjugate, and of the three generations of each fermion species and their conjugates.

```
def symbols : Set B
```

def fieldAlgebra [source]

The algebra generated by all the fields of the Standard Model and their derivative symbols: the gauge field, the Higgs and its conjugate, and the three families of each fermion species with their conjugates.

```
def fieldAlgebra : Subalgebra ℂ B
```

def covDerivH [source]

The iterated covariant derivative of the Higgs field.

```
noncomputable def covDerivH {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ HiggsVec →ₗ[ℂ] B
```

def covDerivBarH [source]

The iterated covariant derivative of the conjugate Higgs field.

```
noncomputable def covDerivBarH {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule HiggsVec) →ₗ[ℂ] B
```

def covDerivD [source]

The iterated covariant derivative of the down-type quarks.

```
noncomputable def covDerivD (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ DownSinglet →ₗ[ℂ] B
```

def covDerivBarD [source]

The iterated covariant derivative of the conjugate down-type quarks.

```
noncomputable def covDerivBarD (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule DownSinglet) →ₗ[ℂ] B
```

def covDerivU [source]

The iterated covariant derivative of the up-type quarks.

```
noncomputable def covDerivU (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ UpSinglet →ₗ[ℂ] B
```

def covDerivBarU [source]

The iterated covariant derivative of the conjugate up-type quarks.

```
noncomputable def covDerivBarU (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule UpSinglet) →ₗ[ℂ] B
```

def covDerivQ [source]

The iterated covariant derivative of the quark doublets.

```
noncomputable def covDerivQ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ QuarkDoublet →ₗ[ℂ] B
```

def covDerivBarQ [source]

The iterated covariant derivative of the conjugate quark doublets.

```
noncomputable def covDerivBarQ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule QuarkDoublet) →ₗ[ℂ] B
```

def covDerivL [source]

The iterated covariant derivative of the lepton doublets.

```
noncomputable def covDerivL (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ LeptonDoubletV →ₗ[ℂ] B
```

def covDerivBarL [source]

The iterated covariant derivative of the conjugate lepton doublets.

```
noncomputable def covDerivBarL (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule LeptonDoubletV) →ₗ[ℂ] B
```

def covDerivE [source]

The iterated covariant derivative of the lepton singlets.

```
noncomputable def covDerivE (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ LeptonSinglet →ₗ[ℂ] B
```

def covDerivBarE [source]

The iterated covariant derivative of the conjugate lepton singlets.

```
noncomputable def covDerivBarE (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule LeptonSinglet) →ₗ[ℂ] B
```

def covDerivFieldStrength [source]

The iterated covariant derivative ∇_{l₁} ⋯ ∇_{lₙ} F_{μν} of the field strength of the gauge field, along an ordered list of directions.

```
noncomputable def covDerivFieldStrength (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B
```

def matterTowers [source]

The covariant towers of the twelve matter families: every symbol of every tower.

```
def matterTowers : Set B
```

lemma matterTowers_induction [source]

A property of every symbol of every matter tower is proved tower by tower.

```
lemma matterTowers_induction (P : B → Prop) {b : B} (hb : b ∈ h.matterTowers)
    (hH : ∀ (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivH l φ))
    (hbarH : ∀ (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivBarH l φ))
    (hd : ∀ (i : Fin 3) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivD i l φ))
    (hbard : ∀ (i : Fin 3) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivBarD i l φ))
    (hu : ∀ (i : Fin 3) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivU i l φ))
    (hbaru : ∀ (i : Fin 3) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivBarU i l φ))
    (hQ : ∀ (i : Fin 3) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivQ i l φ))
    (hbarQ : ∀ (i : Fin 3) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivBarQ i l φ))
    (hL : ∀ (i : Fin 3) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivL i l φ))
    (hbarL : ∀ (i : Fin 3) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivBarL i l φ))
    (he : ∀ (i : Fin 3) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivE i l φ))
    (hbare : ∀ (i : Fin 3) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) φ, P (h.covDerivBarE i l φ)) :
    P b
```

lemma symbols_subset_adjoin_matterTowers [source]

Every bare symbol is a polynomial in the gauge-field symbols and the matter towers.

```
lemma symbols_subset_adjoin_matterTowers :
    h.symbols ⊆ Algebra.adjoin ℂ
      ((⋃ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3), Set.range (h.A s μ)) ∪
        h.matterTowers)
```

lemma fieldAlgebra_eq_covDeriv [source]

The covariant towers generate the field algebra: replacing the plain derivative symbols of every matter field by their covariant derivative towers does not change the generated algebra; only the gauge-field symbols remain plain.

```
lemma fieldAlgebra_eq_covDeriv :
    h.fieldAlgebra = Algebra.adjoin ℂ
    ((⋃ (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3), Set.range (h.A s μ)) ∪
      (⋃ (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)),
        Set.range (h.covDerivH l) ∪ Set.range (h.covDerivBarH l)) ∪
      (⋃ (i : Fin 3) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)),
        Set.range (h.covDerivD i l) ∪ Set.range (h.covDerivBarD i l) ∪
        Set.range (h.covDerivU i l) ∪ Set.range (h.covDerivBarU i l) ∪
        Set.range (h.covDerivQ i l) ∪ Set.range (h.covDerivBarQ i l) ∪
        Set.range (h.covDerivL i l) ∪ Set.range (h.covDerivBarL i l) ∪
        Set.range (h.covDerivE i l) ∪ Set.range (h.covDerivBarE i l)))
```

lemma repJet_covDerivIter [source]

The covariant tower of a matter family transforms through the base point of a gauge jet alone, given the gauge law of the family and the infinitesimal action of its species.

```
lemma repJet_covDerivIter {V : Type} [AddCommGroup V] [Module ℂ V] [FiniteDimensional ℂ V]
    {rep : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] V)}
        {act : GaugeAlgebra →ₗ[ℝ] V →ₗ[ℂ] V}
    {F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B} (hF : TransformsIn repJet rep F)
    (hact : localGaugeData.IsInfinitesimalActionOf act rep) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI) (φ : Module.Dual ℂ V) :
```

```
    repJet U (covDerivIter h.A act F n l 0 φ) =
      covDerivIter h.A act F n l 0 (repDualCoeff rep U⁻¹ 0 φ)
```

lemma repJet_covDerivH [source]

The Higgs tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivH {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI)
    (φ : Module.Dual ℂ HiggsVec) : repJet U (h.covDerivH l φ) =
      h.covDerivH l (repDualCoeff HiggsVec.repJetGaugeGroupI U⁻¹ 0 φ)
```

lemma repJet_covDerivBarH [source]

The conjugate Higgs tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivBarH {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI)
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) : repJet U (h.covDerivBarH l φ) =
      h.covDerivBarH l (repDualCoeff (repConj HiggsVec.repJetGaugeGroupI) U⁻¹ 0 φ)
```

lemma repJet_covDerivD [source]

The down-type quark tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivD (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI)
    (φ : Module.Dual ℂ DownSinglet) : repJet U (h.covDerivD i l φ) =
      h.covDerivD i l (repDualCoeff DownSinglet.repJetGaugeGroupI U⁻¹ 0 φ)
```

lemma repJet_covDerivBarD [source]

The conjugate down-type quark tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivBarD (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI
    )
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) : repJet U (h.covDerivBarD i l φ) =
      h.covDerivBarD i l (repDualCoeff (repConj DownSinglet.repJetGaugeGroupI) U⁻¹ 0 φ)
```

lemma repJet_covDerivU [source]

The up-type quark tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivU (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI)
    (φ : Module.Dual ℂ UpSinglet) : repJet U (h.covDerivU i l φ) =
      h.covDerivU i l (repDualCoeff UpSinglet.repJetGaugeGroupI U⁻¹ 0 φ)
```

lemma repJet_covDerivBarU [source]

The conjugate up-type quark tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivBarU (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI
    )
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) : repJet U (h.covDerivBarU i l φ) =
      h.covDerivBarU i l (repDualCoeff (repConj UpSinglet.repJetGaugeGroupI) U⁻¹ 0 φ)
```

lemma repJet_covDerivQ [source]

The quark doublet tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivQ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI)
    (φ : Module.Dual ℂ QuarkDoublet) : repJet U (h.covDerivQ i l φ) =
      h.covDerivQ i l (repDualCoeff QuarkDoublet.repJetGaugeGroupI U⁻¹ 0 φ)
```

lemma repJet_covDerivBarQ [source]

The conjugate quark doublet tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivBarQ (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI
    )
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) : repJet U (h.covDerivBarQ i l φ) =
      h.covDerivBarQ i l (repDualCoeff (repConj QuarkDoublet.repJetGaugeGroupI) U⁻¹ 0 φ)
```

lemma repJet_covDerivL [source]

The lepton doublet tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivL (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI)
    (φ : Module.Dual ℂ LeptonDoubletV) : repJet U (h.covDerivL i l φ) =
      h.covDerivL i l (repDualCoeff LeptonDoubletV.repJetGaugeGroupI U⁻¹ 0 φ)
```

lemma repJet_covDerivBarL [source]

The conjugate lepton doublet tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivBarL (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI
    )
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) : repJet U (h.covDerivBarL i l φ) =
      h.covDerivBarL i l (repDualCoeff (repConj LeptonDoubletV.repJetGaugeGroupI) U⁻¹ 0 φ)
```

lemma repJet_covDerivE [source]

The lepton singlet tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivE (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI)
    (φ : Module.Dual ℂ LeptonSinglet) : repJet U (h.covDerivE i l φ) =
      h.covDerivE i l (repDualCoeff LeptonSinglet.repJetGaugeGroupI U⁻¹ 0 φ)
```

lemma repJet_covDerivBarE [source]

The conjugate lepton singlet tower transforms through the base point of a gauge jet.

```
lemma repJet_covDerivBarE (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : JetGaugeGroupI
    )
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) : repJet U (h.covDerivBarE i l φ) =
      h.covDerivBarE i l (repDualCoeff (repConj LeptonSinglet.repJetGaugeGroupI) U⁻¹ 0 φ)
```

lemma repJet_covDerivH_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the Higgs tower.

```
lemma repJet_covDerivH_of_mem_truncationKer_zero {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (U : localGaugeData.truncationKer 0) (φ : Module.Dual ℂ HiggsVec) :
    repJet U.1 (h.covDerivH l φ) = h.covDerivH l φ
```

lemma repJet_covDerivBarH_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the conjugate Higgs tower.

```
lemma repJet_covDerivBarH_of_mem_truncationKer_zero {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (U : localGaugeData.truncationKer 0) (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    repJet U.1 (h.covDerivBarH l φ) = h.covDerivBarH l φ
```

lemma repJet_covDerivD_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the down-type quark tower.

```
lemma repJet_covDerivD_of_mem_truncationKer_zero (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin
    3))
    (U : localGaugeData.truncationKer 0) (φ : Module.Dual ℂ DownSinglet) :
    repJet U.1 (h.covDerivD i l φ) = h.covDerivD i l φ
```

lemma repJet_covDerivBarD_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the conjugate down-type quark tower.

```
lemma repJet_covDerivBarD_of_mem_truncationKer_zero (i : Fin 3) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : localGaugeData.truncationKer 0)
    (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    repJet U.1 (h.covDerivBarD i l φ) = h.covDerivBarD i l φ
```

lemma repJet_covDerivU_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the up-type quark tower.

```
lemma repJet_covDerivU_of_mem_truncationKer_zero (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin
    3))
    (U : localGaugeData.truncationKer 0) (φ : Module.Dual ℂ UpSinglet) :
    repJet U.1 (h.covDerivU i l φ) = h.covDerivU i l φ
```

lemma repJet_covDerivBarU_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the conjugate up-type quark tower.

```
lemma repJet_covDerivBarU_of_mem_truncationKer_zero (i : Fin 3) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : localGaugeData.truncationKer 0)
    (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    repJet U.1 (h.covDerivBarU i l φ) = h.covDerivBarU i l φ
```

lemma repJet_covDerivQ_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the quark doublet tower.

```
lemma repJet_covDerivQ_of_mem_truncationKer_zero (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin
    3))
    (U : localGaugeData.truncationKer 0) (φ : Module.Dual ℂ QuarkDoublet) :
```

```
    repJet U.1 (h.covDerivQ i l φ) = h.covDerivQ i l φ
```

lemma repJet_covDerivBarQ_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the conjugate quark doublet tower.

```
lemma repJet_covDerivBarQ_of_mem_truncationKer_zero (i : Fin 3) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : localGaugeData.truncationKer 0)
    (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    repJet U.1 (h.covDerivBarQ i l φ) = h.covDerivBarQ i l φ
```

lemma repJet_covDerivL_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the lepton doublet tower.

```
lemma repJet_covDerivL_of_mem_truncationKer_zero (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin
    3))
    (U : localGaugeData.truncationKer 0) (φ : Module.Dual ℂ LeptonDoubletV) :
    repJet U.1 (h.covDerivL i l φ) = h.covDerivL i l φ
```

lemma repJet_covDerivBarL_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the conjugate lepton doublet tower.

```
lemma repJet_covDerivBarL_of_mem_truncationKer_zero (i : Fin 3) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : localGaugeData.truncationKer 0)
    (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    repJet U.1 (h.covDerivBarL i l φ) = h.covDerivBarL i l φ
```

lemma repJet_covDerivE_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the lepton singlet tower.

```
lemma repJet_covDerivE_of_mem_truncationKer_zero (i : Fin 3) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin
    3))
    (U : localGaugeData.truncationKer 0) (φ : Module.Dual ℂ LeptonSinglet) :
    repJet U.1 (h.covDerivE i l φ) = h.covDerivE i l φ
```

lemma repJet_covDerivBarE_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the conjugate lepton singlet tower.

```
lemma repJet_covDerivBarE_of_mem_truncationKer_zero (i : Fin 3) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3)) (U : localGaugeData.truncationKer 0)
    (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    repJet U.1 (h.covDerivBarE i l φ) = h.covDerivBarE i l φ
```

lemma covDerivFieldStrength_swap [source]

The covariant tower of the field strength is antisymmetric in its two covector indices: the field strength itself is, the gauge-field symbols commuting, and the iterated covariant derivative is odd in the family it differentiates.

```
lemma covDerivFieldStrength_swap (l : List (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (φ : Module.Dual ℝ GaugeAlgebra) :
```

```
    h.covDerivFieldStrength l ν μ φ = - h.covDerivFieldStrength l μ ν φ
```

lemma repJet_covDerivFieldStrength [source]

The covariant tower of the field strength transforms through the base point of a gauge jet: no derivative of the gauge transformation enters.

```
lemma repJet_covDerivFieldStrength (U : JetGaugeGroupI) (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ GaugeAlgebra) :
    repJet U (h.covDerivFieldStrength l μ ν φ) =
      h.covDerivFieldStrength l μ ν (localGaugeData.adjointDualCoeff U⁻¹ 0 φ)
```

lemma repJet_covDerivFieldStrength_of_mem_truncationKer_zero [source]

A pure gauge jet fixes the covariant tower of the field strength.

```
lemma repJet_covDerivFieldStrength_of_mem_truncationKer_zero
    (U : localGaugeData.truncationKer 0) (l : List (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ GaugeAlgebra) :
    repJet U.1 (h.covDerivFieldStrength l μ ν φ) = h.covDerivFieldStrength l μ ν φ
```

lemma repLorentz_covDerivH [source]

The Higgs tower transforms as a Lorentz scalar.

```
lemma repLorentz_covDerivH : IsLorentzCovDerivTransforms repLorentz
    (Representation.trivial ℂ SL(2,ℂ) HiggsVec) (fun {_n} l => h.covDerivH l)
```

lemma repLorentz_covDerivBarH [source]

The conjugate Higgs tower transforms as a Lorentz scalar.

```
lemma repLorentz_covDerivBarH : IsLorentzCovDerivTransforms repLorentz
    (Representation.trivial ℂ SL(2,ℂ) HiggsVec).conj (fun {_n} l => h.covDerivBarH l)
```

lemma repLorentz_covDerivD [source]

The down-type quark tower transforms as a right-handed Weyl spinor.

```
lemma repLorentz_covDerivD (i : Fin 3) : IsLorentzCovDerivTransforms repLorentz
    DownSinglet.repLorentzGroup (fun {_n} l => h.covDerivD i l)
```

lemma repLorentz_covDerivBarD [source]

The conjugate down-type quark tower transforms in the conjugate Weyl representation.

```
lemma repLorentz_covDerivBarD (i : Fin 3) : IsLorentzCovDerivTransforms repLorentz
    DownSinglet.repLorentzGroup.conj (fun {_n} l => h.covDerivBarD i l)
```

lemma repLorentz_covDerivU [source]

The up-type quark tower transforms as a right-handed Weyl spinor.

```
lemma repLorentz_covDerivU (i : Fin 3) : IsLorentzCovDerivTransforms repLorentz
    UpSinglet.repLorentzGroup (fun {_n} l => h.covDerivU i l)
```

lemma repLorentz_covDerivBarU [source]

The conjugate up-type quark tower transforms in the conjugate Weyl representation.

```
lemma repLorentz_covDerivBarU (i : Fin 3) : IsLorentzCovDerivTransforms repLorentz
    UpSinglet.repLorentzGroup.conj (fun {_n} l => h.covDerivBarU i l)
```

lemma repLorentz_covDerivQ [source]

The quark doublet tower transforms as a left-handed Weyl spinor.

```
lemma repLorentz_covDerivQ (i : Fin 3) : IsLorentzCovDerivTransforms repLorentz
    QuarkDoublet.repLorentzGroup (fun {_n} l => h.covDerivQ i l)
```

lemma repLorentz_covDerivBarQ [source]

The conjugate quark doublet tower transforms in the conjugate Weyl representation.

```
lemma repLorentz_covDerivBarQ (i : Fin 3) : IsLorentzCovDerivTransforms repLorentz
    QuarkDoublet.repLorentzGroup.conj (fun {_n} l => h.covDerivBarQ i l)
```

lemma repLorentz_covDerivL [source]

The lepton doublet tower transforms as a left-handed Weyl spinor.

```
lemma repLorentz_covDerivL (i : Fin 3) : IsLorentzCovDerivTransforms repLorentz
    LeptonDoubletV.repLorentzGroup (fun {_n} l => h.covDerivL i l)
```

lemma repLorentz_covDerivBarL [source]

The conjugate lepton doublet tower transforms in the conjugate Weyl representation.

```
lemma repLorentz_covDerivBarL (i : Fin 3) : IsLorentzCovDerivTransforms repLorentz
    LeptonDoubletV.repLorentzGroup.conj (fun {_n} l => h.covDerivBarL i l)
```

lemma repLorentz_covDerivE [source]

The lepton singlet tower transforms as a right-handed Weyl spinor.

```
lemma repLorentz_covDerivE (i : Fin 3) : IsLorentzCovDerivTransforms repLorentz
    LeptonSinglet.repLorentzGroup (fun {_n} l => h.covDerivE i l)
```

lemma repLorentz_covDerivBarE [source]

The conjugate lepton singlet tower transforms in the conjugate Weyl representation.

```
lemma repLorentz_covDerivBarE (i : Fin 3) : IsLorentzCovDerivTransforms repLorentz
    LeptonSinglet.repLorentzGroup.conj (fun {_n} l => h.covDerivBarE i l)
```

2.514 PhyslibAlpha.Particles.StandardModel.Realization.CovFieldAlgebra.Basic

PhyslibAlpha/Particles/StandardModel/Realization/CovFieldAlgebra/Basic.lean on GitHub.

def covF [source]

The covariant derivatives of the field strength in the ordered-tuple indexing used by the covariant form of the theory.

```
noncomputable def covF (h : AlgebraRealization B repJet repLorentz massWeightPoly) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B
```

lemma covF_swap [source]

The field-strength tower is antisymmetric in its two covector indices: the ordered-tuple indexing of `covDerivFieldStrength_swap`.

```
lemma covF_swap (h : AlgebraRealization B repJet repLorentz massWeightPoly) {n : ℕ}
    (l : Fin n → (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ GaugeAlgebra) :
    h.covF l ν μ φ = - h.covF l μ ν φ
```

lemma covF_comp_perm_of_map_lie [source]

At a dual vector vanishing on every bracket, such as the hypercharge coordinate, the field-strength tower is symmetric in its derivative slots, at every order: there the covariant derivatives are the ordinary ones, which commute. The components along the non-abelian factors have no such symmetry.

```
lemma covF_comp_perm_of_map_lie (h : AlgebraRealization B repJet repLorentz massWeightPoly)
    {φ : Module.Dual ℝ GaugeAlgebra} (hφ : ∀ x y : GaugeAlgebra, φ ⁅x, y⁆ = 0) {n : ℕ}
    (σ : Equiv.Perm (Fin n)) (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3) :
    h.covF (l ∘ σ) μ ν φ = h.covF l μ ν φ
```

def covGenerators [source]

The covariant generators of the Standard Model: the field-strength tower, the Higgs towers and their conjugates, and the ten fermion towers and their conjugates.

```
def covGenerators (h : AlgebraRealization B repJet repLorentz massWeightPoly) : Set B
```

def covAlgebra [source]

The covariant subalgebra: the algebra generated by the covariant towers. This is the `CovAlgebraRealization.fieldAlgebra` of the covariant form of the theory.

```
def covAlgebra (h : AlgebraRealization B repJet repLorentz massWeightPoly) : Subalgebra ℂ B
```

def covGeneratorsList [source]

The covariant generating set, with the field-strength tower indexed by lists rather than by ordered tuples.

```
def covGeneratorsList (h : AlgebraRealization B repJet repLorentz massWeightPoly) : Set B
```

lemma covGenerators_eq_covGeneratorsList [source]

The two indexings of the covariant generating set agree.

```
lemma covGenerators_eq_covGeneratorsList : h.covGenerators = h.covGeneratorsList
```

lemma invariant_mem_adjoin_covDeriv [source]

The classification of gauge invariants of the field algebra: a `repJet`-invariant element of the field algebra lies in the covariant subalgebra — it is a polynomial in the covariant derivatives of the field strength and the covariant derivatives of the matter fields.

```
lemma invariant_mem_adjoin_covDeriv {x : B}
    (hx : x ∈ h.fieldAlgebra)
    (hinv : ∀ U : JetGaugeGroupI, repJet U x = x) :
    x ∈ h.covAlgebra
```

2.515 PhyslibAlpha.Particles.StandardModel.Realization.CovStandardModel

PhyslibAlpha/Particles/StandardModel/Realization/CovStandardModel.lean on GitHub.

def repGlobal [source]

The action of the global gauge group on the algebra: the jet action restricted along the inclusion of the constant jets.

```
noncomputable def repGlobal (repJet : Representation ℂ JetGaugeGroupI B) :
    Representation ℂ GaugeGroupI B
```

lemma repGlobal_apply [source]

The global gauge action is the jet action at the corresponding constant jet.

```
@[simp]
lemma repGlobal_apply (repJet : Representation ℂ JetGaugeGroupI B) (g : GaugeGroupI)
    (b : B) : repGlobal repJet g b = repJet (JetGaugeGroupI.ofConstant g) b
```

lemma repGlobal_mul [source]

The global gauge action is multiplicative: it is the jet action at a constant jet, and the jet action is an algebra map.

```
lemma repGlobal_mul (g : GaugeGroupI) (b₁ b₂ : B) :
    repGlobal repJet g (b₁ * b₂) = repGlobal repJet g b₁ * repGlobal repJet g b₂
```

structure Species [source]

A matter species of the Standard Model, as the laws of this file consume it: the gauge-algebra action on its value space, its bare family of derivative symbols, the mass weight `c` of its undifferentiated symbols, and the two facts that the family commutes with the gauge-field symbols and has mass weight `c + 2 * |t|` at the derivative multiset `t`.

```
structure Species (V : Type) [AddCommGroup V] [Module ℂ V] where
  act : GaugeAlgebra →ₗ[ℝ] V →ₗ[ℂ] V
  F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] B
  c : ℕ
  A_comm : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (ψ : Module.Dual ℝ GaugeAlgebra
    )
    (t : Multiset (Fin 1 ⊕ Fin 3)) (χ : Module.Dual ℂ V), Commute (h.A p μ ψ) (F t χ)
  massWeight : ∀ (t : Multiset (Fin 1 ⊕ Fin 3)) (χ : Module.Dual ℂ V),
    massWeightPoly (F t χ) = Polynomial.monomial (c + 2 * Multiset.card t) (F t χ)
```

def tower [source]

The covariant tower of the species, in the ordered-tuple indexing of the covariant form of the theory.

```
noncomputable def tower {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) : Module.Dual ℂ V →ₗ[ℂ] B
```

lemma comm_A [source]

The tower commutes with the gauge-field symbols: it is a polynomial in symbols that do.

```
lemma comm_A {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V)
    (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (ψ : Module.Dual ℝ GaugeAlgebra) :
    Commute (S.tower l φ) (h.A p μ ψ)
```

def speciesH [source]

The Higgs field, of mass weight 2.

```
noncomputable def speciesH : h.Species HiggsVec
```

def speciesBarH [source]

The conjugate Higgs field, of mass weight 2.

```
noncomputable def speciesBarH : h.Species (ConjModule HiggsVec)
```

def speciesD [source]

The down-type quarks of generation `i`, of mass weight 3.

```
noncomputable def speciesD (i : Fin 3) : h.Species DownSinglet
```

def speciesBarD [source]

The conjugate down-type quarks of generation `i`, of mass weight 3.

```
noncomputable def speciesBarD (i : Fin 3) : h.Species (ConjModule DownSinglet)
```

def speciesU [source]

The up-type quarks of generation `i`, of mass weight `3`.

```
noncomputable def speciesU (i : Fin 3) : h.Species UpSinglet
```

def speciesBarU [source]

The conjugate up-type quarks of generation `i`, of mass weight `3`.

```
noncomputable def speciesBarU (i : Fin 3) : h.Species (ConjModule UpSinglet)
```

def speciesQ [source]

The quark doublets of generation `i`, of mass weight `3`.

```
noncomputable def speciesQ (i : Fin 3) : h.Species QuarkDoublet
```

def speciesBarQ [source]

The conjugate quark doublets of generation `i`, of mass weight `3`.

```
noncomputable def speciesBarQ (i : Fin 3) : h.Species (ConjModule QuarkDoublet)
```

def speciesL [source]

The lepton doublets of generation `i`, of mass weight `3`.

```
noncomputable def speciesL (i : Fin 3) : h.Species LeptonDoubletV
```

def speciesBarL [source]

The conjugate lepton doublets of generation `i`, of mass weight `3`.

```
noncomputable def speciesBarL (i : Fin 3) : h.Species (ConjModule LeptonDoubletV)
```

def speciesE [source]

The lepton singlets of generation `i`, of mass weight `3`.

```
noncomputable def speciesE (i : Fin 3) : h.Species LeptonSinglet
```

def speciesBarE [source]

The conjugate lepton singlets of generation `i`, of mass weight `3`.

```
noncomputable def speciesBarE (i : Fin 3) : h.Species (ConjModule LeptonSinglet)
```

lemma matterTowers_induction_species [source]

A property of every symbol of every matter tower is proved species by species: each matter tower is the tower of one of the twelve species.

```
lemma matterTowers_induction_species (P : B → Prop) {b : B} (hb : b ∈ h.matterTowers)
    (hS : ∀ {V : Type} [AddCommGroup V] [Module ℂ V] [FiniteDimensional ℂ V] (S : h.Species V)
      (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V), P (S.tower l φ)) : P b
```

lemma covGenerators_cases [source]

A property of every covariant generator is proved for the field-strength tower and for the matter towers.

```
lemma covGenerators_cases (P : B → Prop) {b : B} (hb : b ∈ h.covGenerators)
    (hF : ∀ (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
      (φ : Module.Dual ℝ GaugeAlgebra), P (h.covF l μ ν φ))
    (hM : ∀ b ∈ h.matterTowers, P b) : P b
```

lemma repJet_algebraMap [source]

Gauge jets fix the scalars: the action is multiplicative, hence unital, and complex-linear.

```
lemma repJet_algebraMap (U : JetGaugeGroupI) (c : ℂ) :
    repJet U (algebraMap ℂ B c) = algebraMap ℂ B c
```

lemma repJet_eq_of_mem_covGenerators_of_mem_truncationKer_zero [source]

Pure gauge jets fix the covariant generators: the thirteen cases are section L of `AlgebraRealization.CovariantDeriv`.

```
lemma repJet_eq_of_mem_covGenerators_of_mem_truncationKer_zero
    (U : localGaugeData.truncationKer 0) {x : B} (hx : x ∈ h.covGenerators) :
    repJet U.1 x = x
```

lemma repJet_eq_of_mem_covAlgebra_of_mem_truncationKer_zero [source]

Pure gauge jets fix the covariant algebra pointwise: they fix its generators, and the jet action is an algebra map.

```
lemma repJet_eq_of_mem_covAlgebra_of_mem_truncationKer_zero
    (U : localGaugeData.truncationKer 0) {x : B} (hx : x ∈ h.covAlgebra) :
    repJet U.1 x = x
```

lemma forall_repJet_eq_iff [source]

The reduction of jet gauge invariance to global gauge invariance: an element of the field algebra is invariant under the whole jet gauge group exactly when it lies in the covariant algebra and is invariant under the global gauge group.

```
lemma forall_repJet_eq_iff {x : B} (hx : x ∈ h.fieldAlgebra) :
    (∀ U : JetGaugeGroupI, repJet U x = x) ↔
      x ∈ h.covAlgebra ∧ ∀ g : GaugeGroupI, repGlobal repJet g x = x
```

lemma forall_repJet_and_repLorentz_eq_iff [source]

The reduction theorem in the form used for Lagrangians: on the field algebra, invariance under the jet gauge group and the Lorentz group is membership of the covariant algebra with invariance under the global gauge group and the Lorentz group.

```
lemma forall_repJet_and_repLorentz_eq_iff {x : B} (hx : x ∈ h.fieldAlgebra) :
    ((∀ U : JetGaugeGroupI, repJet U x = x) ∧ ∀ Λ : SL(2,ℂ), repLorentz Λ x = x) ↔
      (x ∈ h.covAlgebra ∧ (∀ g : GaugeGroupI, repGlobal repJet g x = x) ∧
        ∀ Λ : SL(2,ℂ), repLorentz Λ x = x)
```

lemma repCoeff_zero_ofConstant [source]

The zeroth Taylor coefficient of a jet representation at a constant jet is the underlying action of the global gauge group.

```
lemma repCoeff_zero_ofConstant {V : Type} [AddCommGroup V] [Module ℂ V]
    {rep : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] V)}
    {repG : Representation ℂ GaugeGroupI V} {g : GaugeGroupI}
    (hg : rep (JetGaugeGroupI.ofConstant g) = TensorProduct.map LinearMap.id (repG g)) :
    GaugeAlgebraRealization.repCoeff rep (JetGaugeGroupI.ofConstant g) 0 = repG g
```

lemma repGlobal_of_repJet [source]

A tower that transforms through the base-point dual coefficient of a gauge jet is equivariant for the global gauge group, in the contragredient of the global representation.

```
lemma repGlobal_of_repJet {V : Type} [AddCommGroup V] [Module ℂ V]
    {rep : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] V)}
    {repG : Representation ℂ GaugeGroupI V} {T : Module.Dual ℂ V →ₗ[ℂ] B}
    (hT : ∀ (U : JetGaugeGroupI) (φ : Module.Dual ℂ V),
      repJet U (T φ) = T (GaugeAlgebraRealization.repDualCoeff rep U⁻¹ 0 φ))
    (hg : ∀ g : GaugeGroupI,
      rep (JetGaugeGroupI.ofConstant g) = TensorProduct.map LinearMap.id (repG g))
    (g : GaugeGroupI) (φ : Module.Dual ℂ V) :
    repGlobal repJet g (T φ) = T (repG.dual g φ)
```

lemma repGlobal_of_repJet_conj [source]

The conjugate form of `repGlobal_of_repJet`: a tower transforming through the dual coefficient of the conjugate representation is equivariant in its conjugate contragredient.

```
lemma repGlobal_of_repJet_conj {V : Type} [AddCommGroup V] [Module ℂ V]
    {rep : Representation ℂ JetGaugeGroupI (SpaceTimeAlgebra ⊗[ℂ] V)}
    {repG : Representation ℂ GaugeGroupI V} {T : Module.Dual ℂ (ConjModule V) →ₗ[ℂ] B}
    (hT : ∀ (U : JetGaugeGroupI) (φ : Module.Dual ℂ (ConjModule V)), repJet U (T φ) =
      T (GaugeAlgebraRealization.repDualCoeff (JetComponentSpace.repConj rep) U⁻¹ 0 φ))
    (hg : ∀ g : GaugeGroupI,
      rep (JetGaugeGroupI.ofConstant g) = TensorProduct.map LinearMap.id (repG g))
    (g : GaugeGroupI) (φ : Module.Dual ℂ (ConjModule V)) :
    repGlobal repJet g (T φ) = T (repG.conj.dual g φ)
```

lemma adjointDualCoeff_zero_ofConstant_inv [source]

At an inverse constant jet the dual adjoint coefficient is the contragredient adjoint action of the global gauge group.

```
lemma localGaugeData.adjointDualCoeff_zero_ofConstant_inv (g : GaugeGroupI) :
    localGaugeData.adjointDualCoeff (JetGaugeGroupI.ofConstant g)⁻¹ 0 =
      (GaugeAlgebra.adjointMap g⁻¹).dualMap
```

lemma repGlobal_covF [source]

The field-strength tower is equivariant for the global gauge group, in the contragredient adjoint representation.

```
lemma repGlobal_covF (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ GaugeAlgebra) :
    repGlobal repJet g (h.covF l μ ν φ) =
      h.covF l μ ν ((GaugeAlgebra.adjointMap g⁻¹).dualMap φ)
```

lemma repGlobal_covDerivH [source]

```
lemma repGlobal_covDerivH (φ : Module.Dual ℂ HiggsVec) :
    repGlobal repJet g (h.covDerivH l φ) = h.covDerivH l (HiggsVec.repGaugeGroupI.dual g φ)
```

lemma repGlobal_covDerivBarH [source]

```
lemma repGlobal_covDerivBarH (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    repGlobal repJet g (h.covDerivBarH l φ) =
      h.covDerivBarH l (HiggsVec.repGaugeGroupI.conj.dual g φ)
```

lemma repGlobal_covDerivD [source]

```
lemma repGlobal_covDerivD (φ : Module.Dual ℂ DownSinglet) :
    repGlobal repJet g (h.covDerivD i l φ) =
      h.covDerivD i l (DownSinglet.repGaugeGroupI.dual g φ)
```

lemma repGlobal_covDerivBarD [source]

```
lemma repGlobal_covDerivBarD (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    repGlobal repJet g (h.covDerivBarD i l φ) =
      h.covDerivBarD i l (DownSinglet.repGaugeGroupI.conj.dual g φ)
```

lemma repGlobal_covDerivU [source]

```
lemma repGlobal_covDerivU (φ : Module.Dual ℂ UpSinglet) :
    repGlobal repJet g (h.covDerivU i l φ) = h.covDerivU i l (UpSinglet.repGaugeGroupI.dual g
    φ)
```

lemma repGlobal_covDerivBarU [source]

```
lemma repGlobal_covDerivBarU (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    repGlobal repJet g (h.covDerivBarU i l φ) =
      h.covDerivBarU i l (UpSinglet.repGaugeGroupI.conj.dual g φ)
```

lemma repGlobal_covDerivQ [source]

```
lemma repGlobal_covDerivQ (φ : Module.Dual ℂ QuarkDoublet) :
    repGlobal repJet g (h.covDerivQ i l φ) =
      h.covDerivQ i l (QuarkDoublet.repGaugeGroupI.dual g φ)
```

lemma repGlobal_covDerivBarQ [source]

```
lemma repGlobal_covDerivBarQ (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    repGlobal repJet g (h.covDerivBarQ i l φ) =
      h.covDerivBarQ i l (QuarkDoublet.repGaugeGroupI.conj.dual g φ)
```

lemma repGlobal_covDerivL [source]

```
lemma repGlobal_covDerivL (φ : Module.Dual ℂ LeptonDoubletV) :
    repGlobal repJet g (h.covDerivL i l φ) =
      h.covDerivL i l (LeptonDoubletV.repGaugeGroupI.dual g φ)
```

lemma repGlobal_covDerivBarL [source]

```
lemma repGlobal_covDerivBarL (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    repGlobal repJet g (h.covDerivBarL i l φ) =
      h.covDerivBarL i l (LeptonDoubletV.repGaugeGroupI.conj.dual g φ)
```

lemma repGlobal_covDerivE [source]

```
lemma repGlobal_covDerivE (φ : Module.Dual ℂ LeptonSinglet) :
    repGlobal repJet g (h.covDerivE i l φ) =
      h.covDerivE i l (LeptonSinglet.repGaugeGroupI.dual g φ)
```

lemma repGlobal_covDerivBarE [source]

```
lemma repGlobal_covDerivBarE (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    repGlobal repJet g (h.covDerivBarE i l φ) =
      h.covDerivBarE i l (LeptonSinglet.repGaugeGroupI.conj.dual g φ)
```

lemma covF_mem_adjoin_gaugeSymbols [source]

The covariant derivatives of the field strength are polynomials in the gauge-field symbols.

```
lemma covF_mem_adjoin_gaugeSymbols {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ GaugeAlgebra) :
    h.covF l μ ν φ ∈ Algebra.adjoin ℂ {b : B | ∃ (s : Multiset (Fin 1 ⊕ Fin 3))
      (ρ : Fin 1 ⊕ Fin 3) (ψ : Module.Dual ℝ GaugeAlgebra), b = h.A s ρ ψ}
```

lemma covF_comm_of_comm_A [source]

The field-strength tower commutes with anything the gauge-field symbols commute with.

```
lemma covF_comm_of_comm_A {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) {y : B}
```

```
    (hy : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (ρ : Fin 1 ⊕ Fin 3) (ψ' : Module.Dual ℝ GaugeAlgebra
    ),
      Commute (h.A p ρ ψ') y) : Commute (h.covF l μ ν ψ) y
```

lemma commute_gaugeSymbol_of_mem_covGenerators [source]

Every covariant generator commutes with every gauge-field symbol: the covariant towers are polynomials in the gauge-field and matter symbols, and the gauge field is bosonic.

```
lemma commute_gaugeSymbol_of_mem_covGenerators (p : Multiset (Fin 1 ⊕ Fin 3))
    (ρ : Fin 1 ⊕ Fin 3) (ψ : Module.Dual ℝ GaugeAlgebra) {y : B}
    (hy : y ∈ h.covGenerators) : Commute y (h.A p ρ ψ)
```

lemma covF_commute_of_mem_covAlgebra [source]

The field-strength tower is central in the covariant algebra: it commutes with every covariant generator, and commutation extends to the algebra they generate.

```
lemma covF_commute_of_mem_covAlgebra {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (ψ : Module.Dual ℝ GaugeAlgebra) {x : B}
    (hx : x ∈ h.covAlgebra) : Commute (h.covF l μ ν ψ) x
```

lemma actionFam_apply_mem_submodule [source]

The action pairing of two families lands in any submodule containing the products of their components: expanded in bases it is a finite sum of scalar multiples of them.

```
lemma actionFam_apply_mem_submodule {V : Type} [AddCommGroup V] [Module ℂ V]
    [FiniteDimensional ℂ V] {act : GaugeAlgebra →ₗ[ℝ] V →ₗ[ℂ] V} {M : Submodule ℂ B}
    {f : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B} {g : Module.Dual ℂ V →ₗ[ℂ] B}
    (hfg : ∀ ψ χ, f ψ * g χ ∈ M) (φ : Module.Dual ℂ V) :
    GaugeAlgebraRealization.actionFam act f g φ ∈ M
```

lemma bracketFam_apply_mem_submodule [source]

The bracket pairing of two adjoint families lands in any submodule containing the products of their components: it is the sum of the structure constants against them.

```
lemma bracketFam_apply_mem_submodule {M : Submodule ℂ B}
    {f g : Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B}
    (hfg : ∀ ψ χ, f ψ * g χ ∈ M) (φ : Module.Dual ℝ GaugeAlgebra) :
    GaugeAlgebraRealization.bracketFam f g φ ∈ M
```

def massWeightEigenspace [source]

The weight-`w` part of the algebra: the elements on which the mass-weight algebra map is the monomial `X ^ w`.

```
noncomputable def massWeightEigenspace (massWeightPoly : B →ₐ[ℂ] Polynomial B) (w : ℕ) :
    Submodule ℂ B
```

lemma mem_massWeightEigenspace_iff [source]

Membership of the weight-`w` part is the eigenvector equation itself.

```
lemma mem_massWeightEigenspace_iff {w : ℕ} {b : B} :
    b ∈ massWeightEigenspace massWeightPoly w ↔
      massWeightPoly b = Polynomial.monomial w b
```

lemma mul_mem_massWeightEigenspace [source]

The mass weight is additive on products: `massWeightPoly` is an algebra map and monomials multiply by adding their degrees.

```
lemma mul_mem_massWeightEigenspace {w w' : ℕ} {b b' : B}
    (hb : b ∈ massWeightEigenspace massWeightPoly w)
    (hb' : b' ∈ massWeightEigenspace massWeightPoly w') :
    b * b' ∈ massWeightEigenspace massWeightPoly (w + w')
```

lemma A_mem_massWeightEigenspace [source]

A gauge-field symbol with `|p|` derivatives has mass weight `2 * (1 + |p|)`.

```
lemma A_mem_massWeightEigenspace (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) :
    h.A p μ ψ ∈ massWeightEigenspace massWeightPoly (2 * (1 + Multiset.card p))
```

lemma covDerivIter_mem_massWeightEigenspace [source]

The mass weight of a matter covariant tower: the `n`-fold covariant derivative of the family at the derivative multiset `s` has weight `c + 2 * n + 2 * |s|`. Each covariant derivative costs two units, whether it lands on the derivative index or brings down a gauge-field factor.

```
lemma covDerivIter_mem_massWeightEigenspace {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    GaugeAlgebraRealization.covDerivIter h.A S.act S.F n l s φ ∈
      massWeightEigenspace massWeightPoly (S.c + 2 * n + 2 * Multiset.card s)
```

lemma massWeight_tower [source]

The mass weight of the tower of a species is `c + 2 * n`: the form in which the sector structures ask for it.

```
lemma massWeight_tower {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    massWeightPoly (S.tower l φ) = Polynomial.monomial (S.c + 2 * n) (S.tower l φ)
```

lemma fieldStrength_mem_massWeightEigenspace [source]

The mass weight of the bare field strength: two gauge-field symbols, or one with an extra derivative, in either case weight `4 + 2 * |s|`.

```
lemma fieldStrength_mem_massWeightEigenspace (μ ν : Fin 1 ⊕ Fin 3)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℝ GaugeAlgebra) :
    GaugeAlgebraRealization.fieldStrength h.A μ ν s φ ∈
      massWeightEigenspace massWeightPoly (4 + 2 * Multiset.card s)
```

lemma iteratedCovDerivAdjoint_mem_massWeightEigenspace [source]

The mass weight of an adjoint covariant tower: the adjoint analogue of `Species.covDerivIter_mem_massWei` with the bracket pairing in place of the action pairing.

```
lemma iteratedCovDerivAdjoint_mem_massWeightEigenspace (c : ℕ)
    (G : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual ℝ GaugeAlgebra →ₗ[ℝ] B)
    (hG : ∀ (t : Multiset (Fin 1 ⊕ Fin 3)) (χ : Module.Dual ℝ GaugeAlgebra),
      G t χ ∈ massWeightEigenspace massWeightPoly (c + 2 * Multiset.card t))
    (l : List (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℝ GaugeAlgebra) :
    GaugeAlgebraRealization.iteratedCovDerivAdjoint h.A l G s φ ∈
      massWeightEigenspace massWeightPoly (c + 2 * l.length + 2 * Multiset.card s)
```

lemma massWeight_covF [source]

The mass weight of the field-strength tower is `2 * (2 + n)`: mass dimension `2 + n`.

```
lemma massWeight_covF (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ GaugeAlgebra) :
    massWeightPoly (h.covF l μ ν φ) = Polynomial.monomial (2 * (2 + n)) (h.covF l μ ν φ)
```

lemma massWeight_covDerivH [source]

```
lemma massWeight_covDerivH (φ : Module.Dual ℂ HiggsVec) :
    massWeightPoly (h.covDerivH l φ) = Polynomial.monomial (2 * (1 + n)) (h.covDerivH l φ)
```

lemma massWeight_covDerivBarH [source]

```
lemma massWeight_covDerivBarH (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    massWeightPoly (h.covDerivBarH l φ) =
      Polynomial.monomial (2 * (1 + n)) (h.covDerivBarH l φ)
```

lemma massWeight_covDerivD [source]

```
lemma massWeight_covDerivD (φ : Module.Dual ℂ DownSinglet) :
    massWeightPoly (h.covDerivD i l φ) = Polynomial.monomial (3 + 2 * n) (h.covDerivD i l φ)
```

lemma massWeight_covDerivBarD [source]

```
lemma massWeight_covDerivBarD (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    massWeightPoly (h.covDerivBarD i l φ) =
      Polynomial.monomial (3 + 2 * n) (h.covDerivBarD i l φ)
```

lemma massWeight_covDerivU [source]

```
lemma massWeight_covDerivU (φ : Module.Dual ℂ UpSinglet) :
    massWeightPoly (h.covDerivU i l φ) = Polynomial.monomial (3 + 2 * n) (h.covDerivU i l φ)
```

lemma massWeight_covDerivBarU [source]

```
lemma massWeight_covDerivBarU (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    massWeightPoly (h.covDerivBarU i l φ) =
      Polynomial.monomial (3 + 2 * n) (h.covDerivBarU i l φ)
```

lemma massWeight_covDerivQ [source]

```
lemma massWeight_covDerivQ (φ : Module.Dual ℂ QuarkDoublet) :
    massWeightPoly (h.covDerivQ i l φ) = Polynomial.monomial (3 + 2 * n) (h.covDerivQ i l φ)
```

lemma massWeight_covDerivBarQ [source]

```
lemma massWeight_covDerivBarQ (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    massWeightPoly (h.covDerivBarQ i l φ) =
      Polynomial.monomial (3 + 2 * n) (h.covDerivBarQ i l φ)
```

lemma massWeight_covDerivL [source]

```
lemma massWeight_covDerivL (φ : Module.Dual ℂ LeptonDoubletV) :
    massWeightPoly (h.covDerivL i l φ) = Polynomial.monomial (3 + 2 * n) (h.covDerivL i l φ)
```

lemma massWeight_covDerivBarL [source]

```
lemma massWeight_covDerivBarL (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    massWeightPoly (h.covDerivBarL i l φ) =
      Polynomial.monomial (3 + 2 * n) (h.covDerivBarL i l φ)
```

lemma massWeight_covDerivE [source]

```
lemma massWeight_covDerivE (φ : Module.Dual ℂ LeptonSinglet) :
    massWeightPoly (h.covDerivE i l φ) = Polynomial.monomial (3 + 2 * n) (h.covDerivE i l φ)
```

lemma massWeight_covDerivBarE [source]

```
lemma massWeight_covDerivBarE (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    massWeightPoly (h.covDerivBarE i l φ) =
      Polynomial.monomial (3 + 2 * n) (h.covDerivBarE i l φ)
```

def anticommuteSubmodule [source]

The elements of the algebra anticommuting with a fixed element. It is a submodule, which is what lets the anticommutation of a tower be checked term by term.

```
def anticommuteSubmodule (x : B) : Submodule ℂ B where
```

lemma mem_anticommuteSubmodule_iff [source]

Membership of the anticommutant is the anticommutation relation itself.

```
lemma mem_anticommuteSubmodule_iff {x y : B} :
    y ∈ anticommuteSubmodule x ↔ x * y = -(y * x)
```

lemma anticomm_symm [source]

Anticommutation is symmetric in its two arguments.

```
lemma anticomm_symm {a b : B} (hab : a * b = -(b * a)) : b * a = -(a * b)
```

lemma mul_mem_anticommuteSubmodule [source]

Multiplying an anticommuting element on the left by a commuting one keeps it anticommuting.

```
lemma mul_mem_anticommuteSubmodule {x a b : B} (ha : Commute x a)
    (hb : b ∈ anticommuteSubmodule x) : a * b ∈ anticommuteSubmodule x
```

lemma commute_tower [source]

Anything commuting with every gauge-field symbol and with every bare symbol of the species commutes with its tower.

```
lemma commute_tower {y : B}
    (hyA : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (ψ : Module.Dual ℝ GaugeAlgebra
    ),
      Commute (h.A p μ ψ) y)
    (hyF : ∀ (t : Multiset (Fin 1 ⊕ Fin 3)) (χ : Module.Dual ℂ V), Commute (S.F t χ) y)
    {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) : Commute (S.tower l φ) y
```

lemma commute_tower_tower [source]

The towers of two species whose bare families commute with each other commute. The hypothesis is stated in the shape of the bare laws `H_comm_d`.

```
lemma commute_tower_tower (T : h.Species W)
    (hST : ∀ (t : Multiset (Fin 1 ⊕ Fin 3)) (χ : Module.Dual ℂ V) (t' : Multiset (Fin 1 ⊕ Fin
    3))
      (χ' : Module.Dual ℂ W), Commute (S.F t χ) (T.F t' χ'))
    {n m : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V)
    (l' : Fin m → (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ W) :
    Commute (S.tower l φ) (T.tower l' φ')
```

lemma anticomm_tower [source]

Anything commuting with every gauge-field symbol and anticommuting with every bare symbol of the species anticommutes with its tower: each term of the tower is a product of gauge-field symbols with a single bare symbol.

```
lemma anticomm_tower {x : B}
    (hxA : ∀ (p : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3) (ψ : Module.Dual ℝ GaugeAlgebra
    ),
      Commute x (h.A p μ ψ))
    (hxF : ∀ (t : Multiset (Fin 1 ⊕ Fin 3)) (χ : Module.Dual ℂ V), x * S.F t χ = -(S.F t χ * x)
    )
    {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    x * S.tower l φ = -(S.tower l φ * x)
```

lemma anticomm_tower_tower [source]

The towers of two species whose bare families anticommute with each other anticommute. The hypothesis is stated in the shape of the bare laws `d_anticomm_bard`.

```
lemma anticomm_tower_tower (T : h.Species W)
    (hST : ∀ (t t' : Multiset (Fin 1 ⊕ Fin 3)) (χ : Module.Dual ℂ V) (χ' : Module.Dual ℂ W),
      S.F t χ * T.F t' χ' = -(T.F t' χ' * S.F t χ))
```

```
    {n m : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V)
    (l' : Fin m → (Fin 1 ⊕ Fin 3)) (φ' : Module.Dual ℂ W) :
    S.tower l φ * T.tower l' φ' = -(T.tower l' φ' * S.tower l φ)
```

lemma covF_comm_tower [source]

The field-strength tower commutes with the tower of every species.

```
lemma covF_comm_tower {k n : ℕ} (l : Fin k → (Fin 1 ⊕ Fin 3)) (μ ν : Fin 1 ⊕ Fin 3)
    (ψ : Module.Dual ℝ GaugeAlgebra) (l' : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ V) :
    Commute (h.covF l μ ν ψ) (S.tower l' φ)
```

lemma covF_comm_covF [source]

Two field-strength towers commute: both are polynomials in the gauge-field symbols, and the gauge field is bosonic.

```
lemma covF_comm_covF {n m : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3))
    (l' : Fin m → (Fin 1 ⊕ Fin 3)) (μ ν μ' ν' : Fin 1 ⊕ Fin 3)
    (ψ ψ' : Module.Dual ℝ GaugeAlgebra) :
    Commute (h.covF l μ ν ψ) (h.covF l' μ' ν' ψ')
```

lemma covF_comm_covH [source]

```
lemma covF_comm_covH (φ : Module.Dual ℂ HiggsVec) :
    Commute (h.covF l μ ν ψ) (h.covDerivH l' φ)
```

lemma covF_comm_covBarH [source]

```
lemma covF_comm_covBarH (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    Commute (h.covF l μ ν ψ) (h.covDerivBarH l' φ)
```

lemma covF_comm_covD [source]

```
lemma covF_comm_covD (φ : Module.Dual ℂ DownSinglet) :
    Commute (h.covF l μ ν ψ) (h.covDerivD i l' φ)
```

lemma covF_comm_covBarD [source]

```
lemma covF_comm_covBarD (φ : Module.Dual ℂ (ConjModule DownSinglet)) :
    Commute (h.covF l μ ν ψ) (h.covDerivBarD i l' φ)
```

lemma covF_comm_covU [source]

```
lemma covF_comm_covU (φ : Module.Dual ℂ UpSinglet) :
    Commute (h.covF l μ ν ψ) (h.covDerivU i l' φ)
```

lemma covF_comm_covBarU [source]

```
lemma covF_comm_covBarU (φ : Module.Dual ℂ (ConjModule UpSinglet)) :
    Commute (h.covF l μ ν ψ) (h.covDerivBarU i l' φ)
```

lemma covF_comm_covQ [source]

```
lemma covF_comm_covQ (φ : Module.Dual ℂ QuarkDoublet) :
    Commute (h.covF l μ ν ψ) (h.covDerivQ i l' φ)
```

lemma covF_comm_covBarQ [source]

```
lemma covF_comm_covBarQ (φ : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    Commute (h.covF l μ ν ψ) (h.covDerivBarQ i l' φ)
```

lemma covF_comm_covL [source]

```
lemma covF_comm_covL (φ : Module.Dual ℂ LeptonDoubletV) :
    Commute (h.covF l μ ν ψ) (h.covDerivL i l' φ)
```

lemma covF_comm_covBarL [source]

```
lemma covF_comm_covBarL (φ : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    Commute (h.covF l μ ν ψ) (h.covDerivBarL i l' φ)
```

lemma covF_comm_covE [source]

```
lemma covF_comm_covE (φ : Module.Dual ℂ LeptonSinglet) :
    Commute (h.covF l μ ν ψ) (h.covDerivE i l' φ)
```

lemma covF_comm_covBarE [source]

```
lemma covF_comm_covBarE (φ : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    Commute (h.covF l μ ν ψ) (h.covDerivBarE i l' φ)
```

lemma covH_comm_covH [source]

```
lemma covH_comm_covH (φ φ' : Module.Dual ℂ HiggsVec) :
    Commute (h.covDerivH l φ) (h.covDerivH l' φ')
```

lemma covH_comm_covBarH [source]

```
lemma covH_comm_covBarH (φ : Module.Dual ℂ HiggsVec) (φ' : Module.Dual ℂ (ConjModule HiggsVec))
     :
    Commute (h.covDerivH l φ) (h.covDerivBarH l' φ')
```

lemma covBarH_comm_covBarH [source]

```
lemma covBarH_comm_covBarH (φ φ' : Module.Dual ℂ (ConjModule HiggsVec)) :
    Commute (h.covDerivBarH l φ) (h.covDerivBarH l' φ')
```

lemma covH_comm_covD [source]

```
lemma covH_comm_covD (φ : Module.Dual ℂ HiggsVec) (φ' : Module.Dual ℂ DownSinglet) :
    Commute (h.covDerivH l φ) (h.covDerivD i l' φ')
```

lemma covH_comm_covBarD [source]

```
lemma covH_comm_covBarD (φ : Module.Dual ℂ HiggsVec) (φ' : Module.Dual ℂ (ConjModule
    DownSinglet)) :
    Commute (h.covDerivH l φ) (h.covDerivBarD i l' φ')
```

lemma covH_comm_covU [source]

```
lemma covH_comm_covU (φ : Module.Dual ℂ HiggsVec) (φ' : Module.Dual ℂ UpSinglet) :
    Commute (h.covDerivH l φ) (h.covDerivU i l' φ')
```

lemma covH_comm_covBarU [source]

```
lemma covH_comm_covBarU (φ : Module.Dual ℂ HiggsVec) (φ' : Module.Dual ℂ (ConjModule UpSinglet)
    ) :
    Commute (h.covDerivH l φ) (h.covDerivBarU i l' φ')
```

lemma covH_comm_covQ [source]

```
lemma covH_comm_covQ (φ : Module.Dual ℂ HiggsVec) (φ' : Module.Dual ℂ QuarkDoublet) :
    Commute (h.covDerivH l φ) (h.covDerivQ i l' φ')
```

lemma covH_comm_covBarQ [source]

```
lemma covH_comm_covBarQ (φ : Module.Dual ℂ HiggsVec)
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    Commute (h.covDerivH l φ) (h.covDerivBarQ i l' φ')
```

lemma covH_comm_covL [source]

```
lemma covH_comm_covL (φ : Module.Dual ℂ HiggsVec) (φ' : Module.Dual ℂ LeptonDoubletV) :
    Commute (h.covDerivH l φ) (h.covDerivL i l' φ')
```

lemma covH_comm_covBarL [source]

```
lemma covH_comm_covBarL (φ : Module.Dual ℂ HiggsVec)
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    Commute (h.covDerivH l φ) (h.covDerivBarL i l' φ')
```

lemma covH_comm_covE [source]

```
lemma covH_comm_covE (φ : Module.Dual ℂ HiggsVec) (φ' : Module.Dual ℂ LeptonSinglet) :
    Commute (h.covDerivH l φ) (h.covDerivE i l' φ')
```

lemma covH_comm_covBarE [source]

```
lemma covH_comm_covBarE (φ : Module.Dual ℂ HiggsVec)
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    Commute (h.covDerivH l φ) (h.covDerivBarE i l' φ')
```

lemma covBarH_comm_covD [source]

```
lemma covBarH_comm_covD (φ : Module.Dual ℂ (ConjModule HiggsVec)) (φ' : Module.Dual ℂ
    DownSinglet) :
    Commute (h.covDerivBarH l φ) (h.covDerivD i l' φ')
```

lemma covBarH_comm_covBarD [source]

```
lemma covBarH_comm_covBarD (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (φ' : Module.Dual ℂ (ConjModule DownSinglet)) :
    Commute (h.covDerivBarH l φ) (h.covDerivBarD i l' φ')
```

lemma covBarH_comm_covU [source]

```
lemma covBarH_comm_covU (φ : Module.Dual ℂ (ConjModule HiggsVec)) (φ' : Module.Dual ℂ UpSinglet
    ) :
    Commute (h.covDerivBarH l φ) (h.covDerivU i l' φ')
```

lemma covBarH_comm_covBarU [source]

```
lemma covBarH_comm_covBarU (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (φ' : Module.Dual ℂ (ConjModule UpSinglet)) :
    Commute (h.covDerivBarH l φ) (h.covDerivBarU i l' φ')
```

lemma covBarH_comm_covQ [source]

```
lemma covBarH_comm_covQ (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (φ' : Module.Dual ℂ QuarkDoublet) :
    Commute (h.covDerivBarH l φ) (h.covDerivQ i l' φ')
```

lemma covBarH_comm_covBarQ [source]

```
lemma covBarH_comm_covBarQ (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    Commute (h.covDerivBarH l φ) (h.covDerivBarQ i l' φ')
```

lemma covBarH_comm_covL [source]

```
lemma covBarH_comm_covL (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (φ' : Module.Dual ℂ LeptonDoubletV) :
    Commute (h.covDerivBarH l φ) (h.covDerivL i l' φ')
```

lemma covBarH_comm_covBarL [source]

```
lemma covBarH_comm_covBarL (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    Commute (h.covDerivBarH l φ) (h.covDerivBarL i l' φ')
```

lemma covBarH_comm_covE [source]

```
lemma covBarH_comm_covE (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (φ' : Module.Dual ℂ LeptonSinglet) :
```

```
    Commute (h.covDerivBarH l φ) (h.covDerivE i l' φ')
```

lemma covBarH_comm_covBarE [source]

```
lemma covBarH_comm_covBarE (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    Commute (h.covDerivBarH l φ) (h.covDerivBarE i l' φ')
```

lemma covD_anticomm_covD [source]

```
lemma covD_anticomm_covD (φ φ' : Module.Dual ℂ DownSinglet) :
    h.covDerivD i l φ * h.covDerivD j l' φ' = -(h.covDerivD j l' φ' * h.covDerivD i l φ)
```

lemma covD_anticomm_covBarD [source]

```
lemma covD_anticomm_covBarD (φ : Module.Dual ℂ DownSinglet)
    (φ' : Module.Dual ℂ (ConjModule DownSinglet)) :
    h.covDerivD i l φ * h.covDerivBarD j l' φ' = -(h.covDerivBarD j l' φ' * h.covDerivD i l φ)
```

lemma covD_anticomm_covU [source]

```
lemma covD_anticomm_covU (φ : Module.Dual ℂ DownSinglet) (φ' : Module.Dual ℂ UpSinglet) :
    h.covDerivD i l φ * h.covDerivU j l' φ' = -(h.covDerivU j l' φ' * h.covDerivD i l φ)
```

lemma covD_anticomm_covBarU [source]

```
lemma covD_anticomm_covBarU (φ : Module.Dual ℂ DownSinglet)
    (φ' : Module.Dual ℂ (ConjModule UpSinglet)) :
    h.covDerivD i l φ * h.covDerivBarU j l' φ' = -(h.covDerivBarU j l' φ' * h.covDerivD i l φ)
```

lemma covD_anticomm_covQ [source]

```
lemma covD_anticomm_covQ (φ : Module.Dual ℂ DownSinglet) (φ' : Module.Dual ℂ QuarkDoublet) :
    h.covDerivD i l φ * h.covDerivQ j l' φ' = -(h.covDerivQ j l' φ' * h.covDerivD i l φ)
```

lemma covD_anticomm_covBarQ [source]

```
lemma covD_anticomm_covBarQ (φ : Module.Dual ℂ DownSinglet)
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    h.covDerivD i l φ * h.covDerivBarQ j l' φ' = -(h.covDerivBarQ j l' φ' * h.covDerivD i l φ)
```

lemma covD_anticomm_covL [source]

```
lemma covD_anticomm_covL (φ : Module.Dual ℂ DownSinglet) (φ' : Module.Dual ℂ LeptonDoubletV) :
    h.covDerivD i l φ * h.covDerivL j l' φ' = -(h.covDerivL j l' φ' * h.covDerivD i l φ)
```

lemma covD_anticomm_covBarL [source]

```
lemma covD_anticomm_covBarL (φ : Module.Dual ℂ DownSinglet)
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    h.covDerivD i l φ * h.covDerivBarL j l' φ' = -(h.covDerivBarL j l' φ' * h.covDerivD i l φ)
```

lemma covD_anticomm_covE [source]

```
lemma covD_anticomm_covE (φ : Module.Dual ℂ DownSinglet) (φ' : Module.Dual ℂ LeptonSinglet) :
    h.covDerivD i l φ * h.covDerivE j l' φ' = -(h.covDerivE j l' φ' * h.covDerivD i l φ)
```

lemma covD_anticomm_covBarE [source]

```
lemma covD_anticomm_covBarE (φ : Module.Dual ℂ DownSinglet)
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    h.covDerivD i l φ * h.covDerivBarE j l' φ' = -(h.covDerivBarE j l' φ' * h.covDerivD i l φ)
```

lemma covBarD_anticomm_covBarD [source]

```
lemma covBarD_anticomm_covBarD (φ φ' : Module.Dual ℂ (ConjModule DownSinglet)) :
    h.covDerivBarD i l φ * h.covDerivBarD j l' φ' =
      -(h.covDerivBarD j l' φ' * h.covDerivBarD i l φ)
```

lemma covBarD_anticomm_covU [source]

```
lemma covBarD_anticomm_covU (φ : Module.Dual ℂ (ConjModule DownSinglet))
    (φ' : Module.Dual ℂ UpSinglet) :
    h.covDerivBarD i l φ * h.covDerivU j l' φ' = -(h.covDerivU j l' φ' * h.covDerivBarD i l φ)
```

lemma covBarD_anticomm_covBarU [source]

```
lemma covBarD_anticomm_covBarU (φ : Module.Dual ℂ (ConjModule DownSinglet))
    (φ' : Module.Dual ℂ (ConjModule UpSinglet)) :
    h.covDerivBarD i l φ * h.covDerivBarU j l' φ' =
      -(h.covDerivBarU j l' φ' * h.covDerivBarD i l φ)
```

lemma covBarD_anticomm_covQ [source]

```
lemma covBarD_anticomm_covQ (φ : Module.Dual ℂ (ConjModule DownSinglet))
    (φ' : Module.Dual ℂ QuarkDoublet) :
    h.covDerivBarD i l φ * h.covDerivQ j l' φ' = -(h.covDerivQ j l' φ' * h.covDerivBarD i l φ)
```

lemma covBarD_anticomm_covBarQ [source]

```
lemma covBarD_anticomm_covBarQ (φ : Module.Dual ℂ (ConjModule DownSinglet))
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    h.covDerivBarD i l φ * h.covDerivBarQ j l' φ' =
      -(h.covDerivBarQ j l' φ' * h.covDerivBarD i l φ)
```

lemma covBarD_anticomm_covL [source]

```
lemma covBarD_anticomm_covL (φ : Module.Dual ℂ (ConjModule DownSinglet))
    (φ' : Module.Dual ℂ LeptonDoubletV) :
    h.covDerivBarD i l φ * h.covDerivL j l' φ' = -(h.covDerivL j l' φ' * h.covDerivBarD i l φ)
```

lemma covBarD_anticomm_covBarL [source]

```
lemma covBarD_anticomm_covBarL (φ : Module.Dual ℂ (ConjModule DownSinglet))
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    h.covDerivBarD i l φ * h.covDerivBarL j l' φ' =
      -(h.covDerivBarL j l' φ' * h.covDerivBarD i l φ)
```

lemma covBarD_anticomm_covE [source]

```
lemma covBarD_anticomm_covE (φ : Module.Dual ℂ (ConjModule DownSinglet))
    (φ' : Module.Dual ℂ LeptonSinglet) :
    h.covDerivBarD i l φ * h.covDerivE j l' φ' = -(h.covDerivE j l' φ' * h.covDerivBarD i l φ)
```

lemma covBarD_anticomm_covBarE [source]

```
lemma covBarD_anticomm_covBarE (φ : Module.Dual ℂ (ConjModule DownSinglet))
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    h.covDerivBarD i l φ * h.covDerivBarE j l' φ' =
      -(h.covDerivBarE j l' φ' * h.covDerivBarD i l φ)
```

lemma covU_anticomm_covU [source]

```
lemma covU_anticomm_covU (φ φ' : Module.Dual ℂ UpSinglet) :
    h.covDerivU i l φ * h.covDerivU j l' φ' = -(h.covDerivU j l' φ' * h.covDerivU i l φ)
```

lemma covU_anticomm_covBarU [source]

```
lemma covU_anticomm_covBarU (φ : Module.Dual ℂ UpSinglet)
    (φ' : Module.Dual ℂ (ConjModule UpSinglet)) :
    h.covDerivU i l φ * h.covDerivBarU j l' φ' = -(h.covDerivBarU j l' φ' * h.covDerivU i l φ)
```

lemma covU_anticomm_covQ [source]

```
lemma covU_anticomm_covQ (φ : Module.Dual ℂ UpSinglet) (φ' : Module.Dual ℂ QuarkDoublet) :
    h.covDerivU i l φ * h.covDerivQ j l' φ' = -(h.covDerivQ j l' φ' * h.covDerivU i l φ)
```

lemma covU_anticomm_covBarQ [source]

```
lemma covU_anticomm_covBarQ (φ : Module.Dual ℂ UpSinglet)
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    h.covDerivU i l φ * h.covDerivBarQ j l' φ' = -(h.covDerivBarQ j l' φ' * h.covDerivU i l φ)
```

lemma covU_anticomm_covL [source]

```
lemma covU_anticomm_covL (φ : Module.Dual ℂ UpSinglet) (φ' : Module.Dual ℂ LeptonDoubletV) :
    h.covDerivU i l φ * h.covDerivL j l' φ' = -(h.covDerivL j l' φ' * h.covDerivU i l φ)
```

lemma covU_anticomm_covBarL [source]

```
lemma covU_anticomm_covBarL (φ : Module.Dual ℂ UpSinglet)
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    h.covDerivU i l φ * h.covDerivBarL j l' φ' = -(h.covDerivBarL j l' φ' * h.covDerivU i l φ)
```

lemma covU_anticomm_covE [source]

```
lemma covU_anticomm_covE (φ : Module.Dual ℂ UpSinglet) (φ' : Module.Dual ℂ LeptonSinglet) :
    h.covDerivU i l φ * h.covDerivE j l' φ' = -(h.covDerivE j l' φ' * h.covDerivU i l φ)
```

lemma covU_anticomm_covBarE [source]

```
lemma covU_anticomm_covBarE (φ : Module.Dual ℂ UpSinglet)
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    h.covDerivU i l φ * h.covDerivBarE j l' φ' = -(h.covDerivBarE j l' φ' * h.covDerivU i l φ)
```

lemma covBarU_anticomm_covBarU [source]

```
lemma covBarU_anticomm_covBarU (φ φ' : Module.Dual ℂ (ConjModule UpSinglet)) :
    h.covDerivBarU i l φ * h.covDerivBarU j l' φ' =
      -(h.covDerivBarU j l' φ' * h.covDerivBarU i l φ)
```

lemma covBarU_anticomm_covQ [source]

```
lemma covBarU_anticomm_covQ (φ : Module.Dual ℂ (ConjModule UpSinglet))
    (φ' : Module.Dual ℂ QuarkDoublet) :
    h.covDerivBarU i l φ * h.covDerivQ j l' φ' = -(h.covDerivQ j l' φ' * h.covDerivBarU i l φ)
```

lemma covBarU_anticomm_covBarQ [source]

```
lemma covBarU_anticomm_covBarQ (φ : Module.Dual ℂ (ConjModule UpSinglet))
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    h.covDerivBarU i l φ * h.covDerivBarQ j l' φ' =
      -(h.covDerivBarQ j l' φ' * h.covDerivBarU i l φ)
```

lemma covBarU_anticomm_covL [source]

```
lemma covBarU_anticomm_covL (φ : Module.Dual ℂ (ConjModule UpSinglet))
    (φ' : Module.Dual ℂ LeptonDoubletV) :
    h.covDerivBarU i l φ * h.covDerivL j l' φ' = -(h.covDerivL j l' φ' * h.covDerivBarU i l φ)
```

lemma covBarU_anticomm_covBarL [source]

```
lemma covBarU_anticomm_covBarL (φ : Module.Dual ℂ (ConjModule UpSinglet))
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    h.covDerivBarU i l φ * h.covDerivBarL j l' φ' =
      -(h.covDerivBarL j l' φ' * h.covDerivBarU i l φ)
```

lemma covBarU_anticomm_covE [source]

```
lemma covBarU_anticomm_covE (φ : Module.Dual ℂ (ConjModule UpSinglet))
    (φ' : Module.Dual ℂ LeptonSinglet) :
    h.covDerivBarU i l φ * h.covDerivE j l' φ' = -(h.covDerivE j l' φ' * h.covDerivBarU i l φ)
```

lemma covBarU_anticomm_covBarE [source]

```
lemma covBarU_anticomm_covBarE (φ : Module.Dual ℂ (ConjModule UpSinglet))
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    h.covDerivBarU i l φ * h.covDerivBarE j l' φ' =
      -(h.covDerivBarE j l' φ' * h.covDerivBarU i l φ)
```

lemma covQ_anticomm_covQ [source]

```
lemma covQ_anticomm_covQ (φ φ' : Module.Dual ℂ QuarkDoublet) :
    h.covDerivQ i l φ * h.covDerivQ j l' φ' = -(h.covDerivQ j l' φ' * h.covDerivQ i l φ)
```

lemma covQ_anticomm_covBarQ [source]

```
lemma covQ_anticomm_covBarQ (φ : Module.Dual ℂ QuarkDoublet)
    (φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    h.covDerivQ i l φ * h.covDerivBarQ j l' φ' = -(h.covDerivBarQ j l' φ' * h.covDerivQ i l φ)
```

lemma covQ_anticomm_covL [source]

```
lemma covQ_anticomm_covL (φ : Module.Dual ℂ QuarkDoublet) (φ' : Module.Dual ℂ LeptonDoubletV) :
    h.covDerivQ i l φ * h.covDerivL j l' φ' = -(h.covDerivL j l' φ' * h.covDerivQ i l φ)
```

lemma covQ_anticomm_covBarL [source]

```
lemma covQ_anticomm_covBarL (φ : Module.Dual ℂ QuarkDoublet)
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    h.covDerivQ i l φ * h.covDerivBarL j l' φ' = -(h.covDerivBarL j l' φ' * h.covDerivQ i l φ)
```

lemma covQ_anticomm_covE [source]

```
lemma covQ_anticomm_covE (φ : Module.Dual ℂ QuarkDoublet) (φ' : Module.Dual ℂ LeptonSinglet) :
    h.covDerivQ i l φ * h.covDerivE j l' φ' = -(h.covDerivE j l' φ' * h.covDerivQ i l φ)
```

lemma covQ_anticomm_covBarE [source]

```
lemma covQ_anticomm_covBarE (φ : Module.Dual ℂ QuarkDoublet)
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    h.covDerivQ i l φ * h.covDerivBarE j l' φ' = -(h.covDerivBarE j l' φ' * h.covDerivQ i l φ)
```

lemma covBarQ_anticomm_covBarQ [source]

```
lemma covBarQ_anticomm_covBarQ (φ φ' : Module.Dual ℂ (ConjModule QuarkDoublet)) :
    h.covDerivBarQ i l φ * h.covDerivBarQ j l' φ' =
      -(h.covDerivBarQ j l' φ' * h.covDerivBarQ i l φ)
```

lemma covBarQ_anticomm_covL [source]

```
lemma covBarQ_anticomm_covL (φ : Module.Dual ℂ (ConjModule QuarkDoublet))
    (φ' : Module.Dual ℂ LeptonDoubletV) :
    h.covDerivBarQ i l φ * h.covDerivL j l' φ' = -(h.covDerivL j l' φ' * h.covDerivBarQ i l φ)
```

lemma covBarQ_anticomm_covBarL [source]

```
lemma covBarQ_anticomm_covBarL (φ : Module.Dual ℂ (ConjModule QuarkDoublet))
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    h.covDerivBarQ i l φ * h.covDerivBarL j l' φ' =
      -(h.covDerivBarL j l' φ' * h.covDerivBarQ i l φ)
```

lemma covBarQ_anticomm_covE [source]

```
lemma covBarQ_anticomm_covE (φ : Module.Dual ℂ (ConjModule QuarkDoublet))
    (φ' : Module.Dual ℂ LeptonSinglet) :
    h.covDerivBarQ i l φ * h.covDerivE j l' φ' = -(h.covDerivE j l' φ' * h.covDerivBarQ i l φ)
```

lemma covBarQ_anticomm_covBarE [source]

```
lemma covBarQ_anticomm_covBarE (φ : Module.Dual ℂ (ConjModule QuarkDoublet))
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    h.covDerivBarQ i l φ * h.covDerivBarE j l' φ' =
      -(h.covDerivBarE j l' φ' * h.covDerivBarQ i l φ)
```

lemma covL_anticomm_covL [source]

```
lemma covL_anticomm_covL (φ φ' : Module.Dual ℂ LeptonDoubletV) :
    h.covDerivL i l φ * h.covDerivL j l' φ' = -(h.covDerivL j l' φ' * h.covDerivL i l φ)
```

lemma covL_anticomm_covBarL [source]

```
lemma covL_anticomm_covBarL (φ : Module.Dual ℂ LeptonDoubletV)
    (φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    h.covDerivL i l φ * h.covDerivBarL j l' φ' = -(h.covDerivBarL j l' φ' * h.covDerivL i l φ)
```

lemma covL_anticomm_covE [source]

```
lemma covL_anticomm_covE (φ : Module.Dual ℂ LeptonDoubletV) (φ' : Module.Dual ℂ LeptonSinglet)
    :
    h.covDerivL i l φ * h.covDerivE j l' φ' = -(h.covDerivE j l' φ' * h.covDerivL i l φ)
```

lemma covL_anticomm_covBarE [source]

```
lemma covL_anticomm_covBarE (φ : Module.Dual ℂ LeptonDoubletV)
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    h.covDerivL i l φ * h.covDerivBarE j l' φ' = -(h.covDerivBarE j l' φ' * h.covDerivL i l φ)
```

lemma covBarL_anticomm_covBarL [source]

```
lemma covBarL_anticomm_covBarL (φ φ' : Module.Dual ℂ (ConjModule LeptonDoubletV)) :
    h.covDerivBarL i l φ * h.covDerivBarL j l' φ' =
      -(h.covDerivBarL j l' φ' * h.covDerivBarL i l φ)
```

lemma covBarL_anticomm_covE [source]

```
lemma covBarL_anticomm_covE (φ : Module.Dual ℂ (ConjModule LeptonDoubletV))
    (φ' : Module.Dual ℂ LeptonSinglet) :
    h.covDerivBarL i l φ * h.covDerivE j l' φ' = -(h.covDerivE j l' φ' * h.covDerivBarL i l φ)
```

lemma covBarL_anticomm_covBarE [source]

```
lemma covBarL_anticomm_covBarE (φ : Module.Dual ℂ (ConjModule LeptonDoubletV))
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    h.covDerivBarL i l φ * h.covDerivBarE j l' φ' =
      -(h.covDerivBarE j l' φ' * h.covDerivBarL i l φ)
```

lemma covE_anticomm_covE [source]

```
lemma covE_anticomm_covE (φ φ' : Module.Dual ℂ LeptonSinglet) :
    h.covDerivE i l φ * h.covDerivE j l' φ' = -(h.covDerivE j l' φ' * h.covDerivE i l φ)
```

lemma covE_anticomm_covBarE [source]

```
lemma covE_anticomm_covBarE (φ : Module.Dual ℂ LeptonSinglet)
    (φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    h.covDerivE i l φ * h.covDerivBarE j l' φ' = -(h.covDerivBarE j l' φ' * h.covDerivE i l φ)
```

lemma covBarE_anticomm_covBarE [source]

```
lemma covBarE_anticomm_covBarE (φ φ' : Module.Dual ℂ (ConjModule LeptonSinglet)) :
    h.covDerivBarE i l φ * h.covDerivBarE j l' φ' =
      -(h.covDerivBarE j l' φ' * h.covDerivBarE i l φ)
```

lemma repLorentz_covF [source]

The Lorentz law of the covariant field-strength tower: the covariant derivative slots mix by their own columns of the Lorentz matrix, and the two covector indices of the field strength mix by theirs.

```
lemma repLorentz_covF (Λ : SL(2,ℂ)) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3))
    (μ ν : Fin 1 ⊕ Fin 3) (φ : Module.Dual ℝ GaugeAlgebra) :
    repLorentz Λ (h.covF l μ ν φ) =
      ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
        (∏ i, (((SL2C.toLorentzGroup Λ).1 (p i) (l i) : ℝ) : ℂ)) •
      ∑ a, (((SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) •
      ∑ b, (((SL2C.toLorentzGroup Λ).1 b ν : ℝ) : ℂ) • h.covF p a b φ
```

2.516 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.Basic

PhyslibAlpha/Particles/StandardModel/Realization/HiggsCov/Basic.lean on GitHub.

structure HiggsAlgebraCovRealization [source]

The algebra `B`, with a gauge action, a Lorentz action and a mass-weight grading, carries a Higgs sector when it receives an algebra map from the covariant jet algebra of the Higgs field which is equivariant for both actions and compatible with the

grading. The covariant towers `∇_l H` and `∇_l H⁻` then sit inside `B` as the images of that algebra's own, and every law they satisfy there is its law pushed along the map.

It is the Higgs-sector counterpart of `CovAlgebraRealization`, and stands to `CovHiggsJetAlgebra` as that does to `CovJetAlgebra`.

The last two fields are not consequences of the first three: an equivariant map forces the two actions to be multiplicative only on its image, whereas the sector needs them multiplicative on the whole of `B`.

```
structure HiggsAlgebraCovRealization (B : Type) [Ring B] [Algebra ℂ B]
    (rep : Representation ℂ GaugeGroupI B)
    (repLorentz : Representation ℂ SL(2,ℂ) B)
    (massWeightPoly : B →ₐ[ℂ] Polynomial B) where
  toAlgHom : CovHiggsJetAlgebra →ₐ[ℂ] B
  map_rep : ∀ (g : GaugeGroupI) (x : CovHiggsJetAlgebra),
    toAlgHom (CovHiggsJetAlgebra.repGaugeGroupI g x) = rep g (toAlgHom x)
  map_repLorentz : ∀ (Λ : SL(2,ℂ)) (x : CovHiggsJetAlgebra),
    toAlgHom (CovHiggsJetAlgebra.repLorentzGroup Λ x) = repLorentz Λ (toAlgHom x)
  map_massWeight : ∀ x : CovHiggsJetAlgebra, massWeightPoly (toAlgHom x)
    = Polynomial.mapAlgHom toAlgHom (CovHiggsJetAlgebra.massWeightPoly x)
  rep_mul : ∀ (g : GaugeGroupI) (b₁ b₂ : B), rep g (b₁ * b₂) = rep g b₁ * rep g b₂
  repLorentz_mul : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
    repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂
```

def covH [source]

The covariant derivatives `∇_l H` of the Higgs field inside `B`.

```
noncomputable def covH (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ HiggsVec →ₗ[ℂ] B
```

def covBarH [source]

The covariant derivatives `∇_l H⁻` of the conjugate Higgs field inside `B`.

```
noncomputable def covBarH (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    Module.Dual ℂ (ConjModule HiggsVec) →ₗ[ℂ] B
```

lemma map_rep_eq [source]

A gauge law of the covariant jet algebra of the Higgs field transports along the defining map.

```
lemma map_rep_eq {g : GaugeGroupI} {x y : CovHiggsJetAlgebra}
    (hxy : CovHiggsJetAlgebra.repGaugeGroupI g x = y) :
    rep g (h.toAlgHom x) = h.toAlgHom y
```

lemma H_equivariant [source]

The Higgs symbol carries the dual of the gauge representation on `HiggsVec`: the `SU(2)` index transforms contragrediently, and the hypercharge character by `u⁻³`.

```
lemma H_equivariant (g : GaugeGroupI) (φ : Module.Dual ℂ HiggsVec) (n : ℕ)
    (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    rep g (h.covH n l φ) = h.covH n l (HiggsVec.repGaugeGroupI.dual g φ)
```

lemma barH_equivariant [source]

The conjugate Higgs symbol carries the conjugate-dual of the gauge representation: the physicists' `H^† ↦ H^† g^†`.

```
lemma barH_equivariant (g : GaugeGroupI) (φ : Module.Dual ℂ (ConjModule HiggsVec)) (n : ℕ)
    (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    rep g (h.covBarH n l φ) = h.covBarH n l (HiggsVec.repGaugeGroupI.conj.dual g φ)
```

lemma H_comm_H [source]

The Higgs is bosonic: two Higgs symbols commute.

```
lemma H_comm_H (φ ψ : Module.Dual ℂ HiggsVec) (n1 n2 : ℕ)
    (l1 : Fin n1 → (Fin 1 ⊕ Fin 3)) (l2 : Fin n2 → (Fin 1 ⊕ Fin 3)) :
    Commute (h.covH n1 l1 φ) (h.covH n2 l2 ψ)
```

lemma H_comm_barH [source]

A Higgs symbol commutes with a conjugate Higgs symbol.

```
lemma H_comm_barH (φ : Module.Dual ℂ HiggsVec) (ψ : Module.Dual ℂ (ConjModule HiggsVec))
    (n1 n2 : ℕ) (l1 : Fin n1 → (Fin 1 ⊕ Fin 3)) (l2 : Fin n2 → (Fin 1 ⊕ Fin 3)) :
    Commute (h.covH n1 l1 φ) (h.covBarH n2 l2 ψ)
```

lemma barH_comm_barH [source]

Two conjugate Higgs symbols commute.

```
lemma barH_comm_barH (φ ψ : Module.Dual ℂ (ConjModule HiggsVec)) (n1 n2 : ℕ)
    (l1 : Fin n1 → (Fin 1 ⊕ Fin 3)) (l2 : Fin n2 → (Fin 1 ⊕ Fin 3)) :
    Commute (h.covBarH n1 l1 φ) (h.covBarH n2 l2 ψ)
```

lemma map_massWeight_monomial [source]

A mass-weight eigenvalue equation transports along the defining map.

```
lemma map_massWeight_monomial {n : ℕ} {x : CovHiggsJetAlgebra}
    (hx : CovHiggsJetAlgebra.massWeightPoly x = Polynomial.monomial n x) :
    massWeightPoly (h.toAlgHom x) = Polynomial.monomial n (h.toAlgHom x)
```

lemma H_massWeight [source]

The mass weight of the Higgs tower is `2 * (1 + n)`.

```
lemma H_massWeight (φ : Module.Dual ℂ HiggsVec) (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    massWeightPoly (h.covH n l φ) = Polynomial.monomial (2 * (1 + n)) (h.covH n l φ)
```

lemma barH_massWeight [source]

The mass weight of the conjugate Higgs tower is `2 * (1 + n)`.

```
lemma barH_massWeight (φ : Module.Dual ℂ (ConjModule HiggsVec)) (n : ℕ)
    (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    massWeightPoly (h.covBarH n l φ)
      = Polynomial.monomial (2 * (1 + n)) (h.covBarH n l φ)
```

lemma map_lorentz [source]

A Lorentz law of the covariant jet algebra of the Higgs field transports along the defining map.

```
lemma map_lorentz {V : Type} [AddCommGroup V] [Module ℂ V]
    {repV : Representation ℂ SL(2,ℂ) V}
    {G : {n : ℕ} → (Fin n → (Fin 1 ⊕ Fin 3)) → Module.Dual ℂ V →ₗ[ℂ] CovHiggsJetAlgebra}
    (hG : IsLorentzCovDerivTransforms CovHiggsJetAlgebra.repLorentzGroup repV G) :
    IsLorentzCovDerivTransforms repLorentz repV
      (fun {_n} l => h.toAlgHom.toLinearMap ∘ₗ G l)
```

lemma repLorentz_H [source]

The Higgs tower transforms under the Lorentz group as the covariant derivatives of a Lorentz scalar: each derivative slot mixes by the Lorentz matrix, and the value index is inert.

```
lemma repLorentz_H : IsLorentzCovDerivTransforms repLorentz
    (Representation.trivial ℂ SL(2,ℂ) HiggsVec) (fun {n} => h.covH n)
```

lemma repLorentz_barH [source]

The conjugate Higgs tower transforms as the covariant derivatives of a Lorentz scalar, through the conjugate of the trivial representation.

```
lemma repLorentz_barH : IsLorentzCovDerivTransforms repLorentz
    (Representation.trivial ℂ SL(2,ℂ) HiggsVec).conj (fun {n} => h.covBarH n)
```

lemma repLorentz_H_apply [source]

The pointwise form of `repLorentz_H`: the Lorentz action rotates the derivative indices of a Higgs symbol, and the value index is inert.

```
lemma repLorentz_H_apply (g : SL(2,ℂ)) (φ : Module.Dual ℂ HiggsVec) (n : ℕ)
    (l : Fin n → Fin 1 ⊕ Fin 3) :
    repLorentz g (h.covH n l φ) = ∑ (a : Fin n → Fin 1 ⊕ Fin 3),
      (∏ (i : Fin n), (((SL2C.toLorentzGroup g).1 (a i) (l i) : ℝ) : ℂ)) • h.covH n a φ
```

lemma repLorentz_barH_apply [source]

The pointwise form of `repLorentz_barH`.

```
lemma repLorentz_barH_apply (g : SL(2,ℂ)) (φ : Module.Dual ℂ (ConjModule HiggsVec))
    (n : ℕ) (l : Fin n → Fin 1 ⊕ Fin 3) :
    repLorentz g (h.covBarH n l φ) = ∑ (a : Fin n → Fin 1 ⊕ Fin 3),
      (∏ (i : Fin n), (((SL2C.toLorentzGroup g).1 (a i) (l i) : ℝ) : ℂ)) • h.covBarH n a φ
```

def higgsAlgebra [source]

The subalgebra of `B` generated by the Higgs, its conjugate and all their derivatives.

```
def higgsAlgebra (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly) :
    Subalgebra ℂ B
```

lemma covH_mem_higgsAlgebra [source]

A Higgs symbol lies in the Higgs algebra.

```
lemma covH_mem_higgsAlgebra {n : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ HiggsVec) : h.covH n d φ ∈ h.higgsAlgebra
```

lemma covBarH_mem_higgsAlgebra [source]

A conjugate Higgs symbol lies in the Higgs algebra.

```
lemma covBarH_mem_higgsAlgebra {n : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) : h.covBarH n d φ ∈ h.higgsAlgebra
```

lemma higgsAlgebra_induction [source]

Induction on the Higgs algebra: a property of elements of `B` which holds on every Higgs and conjugate Higgs symbol and on every scalar, and is closed under sums and products of elements of the algebra, holds on the whole algebra.

```
lemma higgsAlgebra_induction {P : B → Prop}
    (hH : ∀ (n : ℕ) (d : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ HiggsVec),
      P (h.covH n d φ))
    (hbarH : ∀ (n : ℕ) (d : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual ℂ (ConjModule HiggsVec)),
      P (h.covBarH n d φ))
    (halg : ∀ r : ℂ, P (algebraMap ℂ B r))
    (hadd : ∀ x y, x ∈ h.higgsAlgebra → y ∈ h.higgsAlgebra → P x → P y → P (x + y))
    (hmul : ∀ x y, x ∈ h.higgsAlgebra → y ∈ h.higgsAlgebra → P x → P y → P (x * y))
    {x : B} (hx : x ∈ h.higgsAlgebra) : P x
```

lemma commute_of_mem_higgsAlgebra [source]

Any two elements of the Higgs algebra commute.

```
lemma commute_of_mem_higgsAlgebra {x y : B} (hx : x ∈ h.higgsAlgebra)
    (hy : y ∈ h.higgsAlgebra) : Commute x y
```

lemma mul_comm_of_le_higgsAlgebra [source]

Two submodules of the Higgs algebra commute.

```
lemma mul_comm_of_le_higgsAlgebra {M N : Submodule ℂ B}
    (hM : M ≤ Subalgebra.toSubmodule h.higgsAlgebra)
    (hN : N ≤ Subalgebra.toSubmodule h.higgsAlgebra) : M * N = N * M
```

def higgs [source]

The component `∇_d H^i` in the algebra.

```
noncomputable def higgs (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly)
    {n : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3)) (i : Fin 2) : B
```

def barHiggs [source]

The component `∇_d H¯^i` in the algebra.

```
noncomputable def barHiggs (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly)
    {n : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3)) (i : Fin 2) : B
```

lemma rep_higgsComponent [source]

The gauge group mixes the components of `∇_d H` by the matrix `u⁻³ g⁻¹` of the hypercharge and `SU(2)` parts of `g⁻¹`.

```
lemma rep_higgsComponent (g : GaugeGroupI) {n : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3)) (i : Fin 2) :
    rep g (h.higgs d i) =
      ∑ j, (((g⁻¹).toU1 : ℂ) ^ 3 * (g⁻¹).toSU2.1 i j) • h.higgs d j
```

lemma rep_barHiggsComponent [source]

The gauge group mixes the components of `∇_d H⁻` by the conjugate matrix.

```
lemma rep_barHiggsComponent (g : GaugeGroupI) {n : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3))
    (i : Fin 2) :
    rep g (h.barHiggs d i) =
      ∑ j, (starRingEnd ℂ (((g⁻¹).toU1 : ℂ) ^ 3 * (g⁻¹).toSU2.1 i j)) • h.barHiggs d j
```

def higgsSubmodule [source]

The submodule of `B` generated by the Higgs symbols carrying `n` derivatives: the join, over the Lorentz indices `d`, of the ranges of the symbol maps `H n d`. Its elements are the terms linear in `∇_d H` — of mass dimension `1 + n`.

```
noncomputable def higgsSubmodule
    (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly) (n : ℕ) :
    Submodule ℂ B
```

def barHiggsSubmodule [source]

The submodule of `B` generated by the conjugate Higgs symbols carrying `n` derivatives: the join, over the Lorentz indices `d`, of the ranges of the symbol maps `barH n d`.

```
noncomputable def barHiggsSubmodule
    (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly) (n : ℕ) :
    Submodule ℂ B
```

lemma higgsSubmodule_eq_iSup_span [source]

The Higgs submodule is spanned by the components `∇_d H^j`.

```
lemma higgsSubmodule_eq_iSup_span (n : ℕ) :
    h.higgsSubmodule n = ⨆ (d : Fin n → (Fin 1 ⊕ Fin 3)) (j : Fin 2), ℂ • h.higgs d j
```

lemma barHiggsSubmodule_eq_iSup_span [source]

The conjugate Higgs submodule is spanned by the components `∇_d H⁻^j`.

```
lemma barHiggsSubmodule_eq_iSup_span (n : ℕ) :
    h.barHiggsSubmodule n
      = ⨆ (d : Fin n → (Fin 1 ⊕ Fin 3)) (j : Fin 2), ℂ • h.barHiggs d j
```

lemma higgsSubmodule_le_higgsAlgebra [source]

The Higgs submodule lies in the Higgs algebra.

```
lemma higgsSubmodule_le_higgsAlgebra (n : ℕ) :
    h.higgsSubmodule n ≤ Subalgebra.toSubmodule h.higgsAlgebra
```

lemma barHiggsSubmodule_le_higgsAlgebra [source]

The conjugate Higgs submodule lies in the Higgs algebra.

```
lemma barHiggsSubmodule_le_higgsAlgebra (n : ℕ) :
    h.barHiggsSubmodule n ≤ Subalgebra.toSubmodule h.higgsAlgebra
```

lemma barHiggsSubmodule_comm_higgsSubmodule [source]

The conjugate Higgs and Higgs submodules commute.

```
@[simp]
lemma barHiggsSubmodule_comm_higgsSubmodule (n1 n2 : ℕ) :
    (h.barHiggsSubmodule n1) * (h.higgsSubmodule n2)
    =  (h.higgsSubmodule n2)  * (h.barHiggsSubmodule n1)
```

lemma higgsSubmodule_comm_higgsSubmodule [source]

Two Higgs submodules commute.

```
lemma higgsSubmodule_comm_higgsSubmodule (n1 n2 : ℕ) :
    (h.higgsSubmodule n1) * (h.higgsSubmodule n2)
    =  (h.higgsSubmodule n2)  * (h.higgsSubmodule n1)
```

lemma barHiggsSubmodule_comm_barHiggsSubmodule [source]

Two conjugate Higgs submodules commute.

```
lemma barHiggsSubmodule_comm_barHiggsSubmodule (n1 n2 : ℕ) :
    (h.barHiggsSubmodule n1) * (h.barHiggsSubmodule n2)
    =  (h.barHiggsSubmodule n2)  * (h.barHiggsSubmodule n1)
```

lemma barHiggs_higgs_left_comm [source]

A conjugate Higgs submodule commutes past a Higgs submodule standing in front of a third factor.

```
lemma barHiggs_higgs_left_comm (n1 n2 : ℕ) (C : Submodule ℂ B) :
    h.barHiggsSubmodule n1 * (h.higgsSubmodule n2 * C)
      = h.higgsSubmodule n2 * (h.barHiggsSubmodule n1 * C)
```

lemma rep_gaugeTorusGen_higgs [source]

The torus generators act on `∇_d H^j` by the character of weight `(0, 0, -isoWeight j, -3)`.

```
lemma rep_gaugeTorusGen_higgs (i : Fin 4) {n : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3)) (j : Fin 2) :
    rep (gaugeTorusGen i) (h.higgs d j)
      = ((expI : ℂ) ^ GaugeWeight.coord (0, 0, -isoWeight j, -3) i) • h.higgs d j
```

lemma rep_gaugeTorusGen_barHiggs [source]

The torus generators act on `∇_d H⁻^j` by the character of weight `(0, 0, isoWeight j, 3)`.

```
lemma rep_gaugeTorusGen_barHiggs (i : Fin 4) {n : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3))
    (j : Fin 2) :
    rep (gaugeTorusGen i) (h.barHiggs d j)
      = ((expI : ℂ) ^ GaugeWeight.coord (0, 0, isoWeight j, 3) i) • h.barHiggs d j
```

lemma iSup_span_le_eigenspace [source]

The span of a family of eigenvectors of a torus generator lies in its eigenspace.

```
lemma iSup_span_le_eigenspace {ι : Type} (i : Fin 4) (y : ι → B) (w : GaugeWeight)
    (hy : ∀ d, rep (gaugeTorusGen i) (y d) = ((expI : ℂ) ^ w.coord i) • y d) :
    (⨆ d, ℂ • y d) ≤ Module.End.eigenspace (rep (gaugeTorusGen i)) ((expI : ℂ) ^ w.coord i)
```

def doubletGaugeWeight [source]

The gauge weight decomposition of the span of a two-component family `x d j` of torus eigenvectors, the components `x d 0` of weight `w₀` and `x d 1` of weight `w₁ ≠ w₀`: the weight-`w₀` piece is the span of the `x d 0` and the weight-`w₁` piece that of the `x d 1`.

```
@[implicit_reducible]
noncomputable def doubletGaugeWeight {ι : Type} (hmul : IsMulRep rep) (x : ι → Fin 2 → B)
    (w₀ w₁ : GaugeWeight) (hw : w₀ ≠ w₁)
    (hx₀ : ∀ (i : Fin 4) (d : ι),
      rep (gaugeTorusGen i) (x d 0) = ((expI : ℂ) ^ w₀.coord i) • x d 0)
    (hx₁ : ∀ (i : Fin 4) (d : ι),
      rep (gaugeTorusGen i) (x d 1) = ((expI : ℂ) ^ w₁.coord i) • x d 1) :
    GaugeWeightDecomposition rep (⨆ (d : ι) (j : Fin 2), ℂ • x d j) where
```

instance higgsSubmoduleGaugeWeight [source]

The gauge weight decomposition of the Higgs submodule: `∇_d H⁰` spans the piece of weight `(0, 0, -1, -3)` and `∇_d H¹` that of weight `(0, 0, 1, -3)`.

```
noncomputable instance higgsSubmoduleGaugeWeight (n : ℕ) :
    GaugeWeightDecomposition rep (h.higgsSubmodule n)
```

instance barHiggsSubmoduleGaugeWeight [source]

The gauge weight decomposition of the conjugate Higgs submodule: `∇_d H⁻⁰` spans the piece of weight `(0, 0, 1, 3)` and `∇_d H⁻¹` that of weight `(0, 0, -1, 3)`.

```
noncomputable instance barHiggsSubmoduleGaugeWeight (n : ℕ) :
    GaugeWeightDecomposition rep (h.barHiggsSubmodule n)
```

def dotGaugeHiggs [source]

The gauge-invariant pairing `∇_{d1} H^j ∇_{d2} H⁻^j`, summed over the isospin index.

```
noncomputable def dotGaugeHiggs (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly
    )
    {n1 n2 : ℕ} (d1 : Fin n1 → (Fin 1 ⊕ Fin 3)) (d2 : Fin n2 → (Fin 1 ⊕ Fin 3)) :
    B
```

lemma rep_dotGaugeHiggs_invariant [source]

The pairing is gauge invariant: the matrix acting on `H` is unitary, and `H⁻` transforms by its conjugate.

```
lemma rep_dotGaugeHiggs_invariant {n1 n2 : ℕ} (g : GaugeGroupI) (d1 : Fin n1 → (Fin 1 ⊕ Fin 3))
    (d2 : Fin n2 → (Fin 1 ⊕ Fin 3)) :
    rep g (h.dotGaugeHiggs d1 d2) = h.dotGaugeHiggs d1 d2
```

lemma repLorentz_higgs [source]

The Lorentz action rotates the derivative indices of a Higgs component.

```
lemma repLorentz_higgs {n : ℕ} (g : SL(2,ℂ)) (d : Fin n → Fin 1 ⊕ Fin 3) (k : Fin 2) :
    repLorentz g (h.higgs d k) = ∑ a : Fin n → Fin 1 ⊕ Fin 3,
      (∏ j, (((SL2C.toLorentzGroup g).1 (a j) (d j) : ℝ) : ℂ)) • h.higgs a k
```

lemma repLorentz_barHiggs [source]

The Lorentz action rotates the derivative indices of a conjugate Higgs component.

```
lemma repLorentz_barHiggs {n : ℕ} (g : SL(2,ℂ)) (d : Fin n → Fin 1 ⊕ Fin 3) (k : Fin 2) :
    repLorentz g (h.barHiggs d k) = ∑ a : Fin n → Fin 1 ⊕ Fin 3,
      (∏ j, (((SL2C.toLorentzGroup g).1 (a j) (d j) : ℝ) : ℂ)) • h.barHiggs a k
```

lemma repLorentz_dotGaugeHiggs [source]

The Lorentz action on the Higgs inner product: the two factors' derivative indices rotate independently, and the inner product itself is a Lorentz scalar.

```
lemma repLorentz_dotGaugeHiggs {m n : ℕ} (g : SL(2,ℂ))
    (d₁ : Fin m → Fin 1 ⊕ Fin 3) (d₂ : Fin n → Fin 1 ⊕ Fin 3) :
    repLorentz g (h.dotGaugeHiggs d₁ d₂) =
      ∑ a₁ : Fin m → Fin 1 ⊕ Fin 3, ∑ a₂ : Fin n → Fin 1 ⊕ Fin 3,
        ((∏ j, (((SL2C.toLorentzGroup g).1 (a₁ j) (d₁ j) : ℝ) : ℂ)) *
          (∏ j, (((SL2C.toLorentzGroup g).1 (a₂ j) (d₂ j) : ℝ) : ℂ))) •
          h.dotGaugeHiggs a₁ a₂
```

def massWeightSubmodule [source]

All terms built from the Higgs symbols and their derivatives which have mass weight exactly `n`: the intersection of the algebra generated by every `∇_d H` and `∇_d H⁻` with the part on which `massWeightPoly` is the monomial `X ^ n`.

```
noncomputable def massWeightSubmodule
    (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly) (n : ℕ) :
    Submodule ℂ B
```

lemma mem_massWeightSubmodule_iff [source]

An element has mass weight `n` when it lies in the Higgs algebra and `massWeightPoly` scales it by `X ^ n`.

```
lemma mem_massWeightSubmodule_iff {n : ℕ} {x : B} :
    x ∈ h.massWeightSubmodule n
      ↔ x ∈ h.higgsAlgebra ∧ massWeightPoly x = Polynomial.monomial n x
```

lemma massWeightPoly_of_mem_massWeightSubmodule [source]

The mass weight of an element of the weight-`n` submodule.

```
lemma massWeightPoly_of_mem_massWeightSubmodule {n : ℕ} {x : B}
    (hx : x ∈ h.massWeightSubmodule n) : massWeightPoly x = Polynomial.monomial n x
```

lemma mem_higgsAlgebra_of_mem_massWeightSubmodule [source]

The weight-`n` submodule lies in the Higgs algebra.

```
lemma mem_higgsAlgebra_of_mem_massWeightSubmodule {n : ℕ} {x : B}
    (hx : x ∈ h.massWeightSubmodule n) : x ∈ h.higgsAlgebra
```

lemma massWeightSubmodule_mul_comm [source]

Two mass weight submodules commute.

```
lemma massWeightSubmodule_mul_comm (n m : ℕ) :
    h.massWeightSubmodule n * h.massWeightSubmodule m
    = h.massWeightSubmodule m * h.massWeightSubmodule n
```

lemma one_le_massWeightSubmodule_zero [source]

The scalars have mass weight zero.

```
lemma one_le_massWeightSubmodule_zero : (1 : Submodule ℂ B) ≤ h.massWeightSubmodule 0
```

lemma massWeightSubmodule_mul_le [source]

Mass weights add under multiplication.

```
lemma massWeightSubmodule_mul_le (m n : ℕ) :
    h.massWeightSubmodule m * h.massWeightSubmodule n ≤ h.massWeightSubmodule (m + n)
```

lemma massWeightSubmodule_higgsSubmodule_le [source]

The Higgs symbols with `n` derivatives have mass weight `2 * (1 + n)`.

```
lemma massWeightSubmodule_higgsSubmodule_le (n : ℕ) :
    h.higgsSubmodule n ≤ h.massWeightSubmodule (2 * (1 + n))
```

lemma massWeightSubmodule_barHiggsSubmodule_le [source]

The conjugate Higgs symbols with `n` derivatives have mass weight `2 * (1 + n)`.

```
lemma massWeightSubmodule_barHiggsSubmodule_le (n : ℕ) :
    h.barHiggsSubmodule n ≤ h.massWeightSubmodule (2 * (1 + n))
```

lemma higgsAlgebra_eq_adjoin [source]

The Higgs algebra is generated by the Higgs and conjugate Higgs towers of every derivative order.

```
lemma higgsAlgebra_eq_adjoin :
    h.higgsAlgebra = Algebra.adjoin ℂ
      (⋃ n, ((h.higgsSubmodule n ⊔ h.barHiggsSubmodule n : Submodule ℂ B) : Set B))
```

lemma massWeightPoly_of_mem_higgsSubmodule_sup [source]

*massWeightPoly is the monomial X ^ (2 * (1 + n)) on the Higgs and conjugate Higgs towers with n derivatives.*

```
lemma massWeightPoly_of_mem_higgsSubmodule_sup (n : ℕ) :
    ∀ x ∈ h.higgsSubmodule n ⊔ h.barHiggsSubmodule n,
      massWeightPoly x = Polynomial.monomial (2 * (1 + n)) x
```

lemma massWeightSubmodule_zero_eq [source]

Weight zero is the scalars: every Higgs tower has positive weight.

```
lemma massWeightSubmodule_zero_eq : h.massWeightSubmodule 0 = 1
```

lemma massWeightSubmodule_eq [source]

The weight recursion: a term of positive mass weight i is a sum of symbols of weight i and of products of two terms of lower positive weight adding up to i.

```
lemma massWeightSubmodule_eq (i : ℕ) (hi : 0 < i) :
    h.massWeightSubmodule i
      = (⨆ k ∈ Finset.univ.filter (fun k : Fin i => 2 * (1 + (k : ℕ)) = i),
          h.higgsSubmodule (k : ℕ) ⊔ h.barHiggsSubmodule (k : ℕ))
        ⊔ (⨆ p ∈ Finset.univ.filter (fun p : Fin i × Fin i => (p.1 : ℕ) + (p.2 : ℕ) = i),
            h.massWeightSubmodule (p.1 : ℕ) * h.massWeightSubmodule (p.2 : ℕ))
```

lemma massWeightSubmodule_eq_iSup_mul [source]

*Removing the leftmost Higgs tower: a term of positive weight w is a sum of products of a tower $\nabla^n H$ or $\nabla^n H^-$, of weight 2 * (1 + n) ≤ w, with a term of the remaining weight.*

```
lemma massWeightSubmodule_eq_iSup_mul (w : ℕ) (hw : 0 < w) :
    h.massWeightSubmodule w
      = ⨆ n ∈ (Finset.range (w + 1)).filter (fun n => 2 * (1 + n) ≤ w),
          (h.higgsSubmodule n ⊔ h.barHiggsSubmodule n) * h.massWeightSubmodule (w - 2 * (1 + n))
    )
```

lemma massWeightSubmodule_odd_eq_bot [source]

The odd mass weight submodules are trivial: every Higgs tower has even weight, and weights add under products.

```
lemma massWeightSubmodule_odd_eq_bot (n : ℕ) (hn : Odd n) :
    h.massWeightSubmodule n = ⊥
```

def massWeightSubmoduleGaugeWeight [source]

The gauge weight decomposition of the mass weight submodules. By recursion on the weight through `massWeightSubmodule_eq`: a term of weight `i` is either a symbol of that weight — decomposed by `higgsSubmoduleGaugeWeight` and `barHiggsSubmoduleGaugeWeight` — or a product of two terms of lower positive weight, decomposed by `mul` from the decompositions supplied by the recursion.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeight :
    (i : ℕ) → 0 < i → GaugeWeightDecomposition rep (h.massWeightSubmodule i)
```

instance massWeightSubmoduleGaugeWeightOfNeZero [source]

The `NeZero` form of `massWeightSubmoduleGaugeWeight`.

```
noncomputable instance massWeightSubmoduleGaugeWeightOfNeZero (i : ℕ) [NeZero i] :
    GaugeWeightDecomposition rep (h.massWeightSubmodule i)
```

lemma massWeightSubmodule_two_eq [source]

Weight two: the underived Higgs and conjugate Higgs.

```
lemma massWeightSubmodule_two_eq :
    h.massWeightSubmodule 2 = h.higgsSubmodule 0 ⊔ h.barHiggsSubmodule 0
```

lemma massWeightSubmodule_four_eq [source]

Weight four: the once-derived symbols and the products of two underived ones. The leftmost tower is underived, leaving weight two, or once-derived, leaving weight zero.

```
lemma massWeightSubmodule_four_eq :
    h.massWeightSubmodule 4 = h.higgsSubmodule 1 ⊔ h.barHiggsSubmodule 1 ⊔
    h.higgsSubmodule 0 * h.higgsSubmodule 0 ⊔ h.higgsSubmodule 0 *
    h.barHiggsSubmodule 0 ⊔ h.barHiggsSubmodule 0 * h.barHiggsSubmodule 0
```

lemma massWeightSubmodule_six_eq [source]

Weight six: the twice-derived symbols, a once-derived symbol against an underived one, and the products of three underived ones. The leftmost tower leaves weight four, two or zero.

```
lemma massWeightSubmodule_six_eq : h.massWeightSubmodule 6 =
```

lemma massWeightSubmodule_eight_eq [source]

Weight eight: the derivative terms with up to three derivatives, and the products of four underived symbols. The leftmost tower leaves weight six, four, two or zero.

```
lemma massWeightSubmodule_eight_eq :
    h.massWeightSubmodule 8 =
```

def gaugeInvariantOfMassDim [source]

The gauge invariants of a given mass weight.

```
noncomputable def gaugeInvariantOfMassDim (M : ℕ) : Submodule ℂ B
```

2.517 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.DerivSubmodule.Basic

PhyslibAlpha/Particles/StandardModel/Realization/HiggsCov/DerivSubmodule/Basic.lean on GitHub.

def derivSubmodule [source]

The submodule of `B` generated by the Higgs symbols and their conjugates carrying `n` derivatives.

```
noncomputable def derivSubmodule (h : HiggsAlgebraCovRealization B rep repLorentz
    massWeightPoly)
    (n : ℕ) : Submodule ℂ B
```

lemma derivSubmodule_le_ker_H [source]

Every element of a derivative submodule commutes with a fixed Higgs symbol.

```
lemma derivSubmodule_le_ker_H {n k : ℕ} (d : Fin k → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ HiggsVec) :
    h.derivSubmodule n
      ≤ LinearMap.ker (LinearMap.mulLeft ℂ (h.covH k d φ)
        - LinearMap.mulRight ℂ (h.covH k d φ))
```

lemma derivSubmodule_le_ker_barH [source]

Every element of a derivative submodule commutes with a fixed conjugate-Higgs symbol.

```
lemma derivSubmodule_le_ker_barH {n k : ℕ} (d : Fin k → (Fin 1 ⊕ Fin 3))
    (φ : Module.Dual ℂ (ConjModule HiggsVec)) :
    h.derivSubmodule n
      ≤ LinearMap.ker (LinearMap.mulLeft ℂ (h.covBarH k d φ)
        - LinearMap.mulRight ℂ (h.covBarH k d φ))
```

lemma commute_of_mem_derivSubmodule [source]

Any element of a derivative submodule commutes with any element of any derivative submodule: the Higgs is bosonic.

```
lemma commute_of_mem_derivSubmodule {n m : ℕ} {x y : B}
    (hx : x ∈ h.derivSubmodule n) (hy : y ∈ h.derivSubmodule m) : Commute x y
```

lemma derivSubmodule_mul_comm [source]

Derivative submodules commute with one another at the level of submodule products.

```
lemma derivSubmodule_mul_comm (n m : ℕ) :
    h.derivSubmodule n * h.derivSubmodule m = h.derivSubmodule m * h.derivSubmodule n
```

lemma derivSubmodule_map_rep_le [source]

The derivative submodules are closed under the gauge action: each symbol is carried to a symbol with the same number of derivatives.

```
lemma derivSubmodule_map_rep_le (n : ℕ) (g : GaugeGroupI) :
    (h.derivSubmodule n).map (rep g) ≤ h.derivSubmodule n
```

lemma derivSubmodule_map_rep [source]

The derivative submodules are closed under the gauge action.

```
lemma derivSubmodule_map_rep (n : ℕ) (g : GaugeGroupI) :
    (h.derivSubmodule n).map (rep g) = h.derivSubmodule n
```

lemma derivSubmodule_map_repLorentz_le [source]

The derivative submodules are closed under the Lorentz action: the Lorentz group only mixes the derivative indices within a fixed number of derivatives.

```
lemma derivSubmodule_map_repLorentz_le (n : ℕ) (Λ : SL(2,ℂ)) :
    (h.derivSubmodule n).map (repLorentz Λ) ≤ h.derivSubmodule n
```

lemma derivSubmodule_map_repLorentz [source]

The derivative submodules are closed under the Lorentz action.

```
lemma derivSubmodule_map_repLorentz (n : ℕ) (Λ : SL(2,ℂ)) :
    (h.derivSubmodule n).map (repLorentz Λ) = h.derivSubmodule n
```

2.518 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.DerivSubmodule.Centre

PhyslibAlpha/Particles/StandardModel/Realization/HiggsCov/DerivSubmodule/Centre.lean on GitHub.

lemma range_covH_le_centreEigenspace [source]

The Higgs symbols carry the sign +1: their value space is a Lorentz scalar.

```
lemma range_covH_le_centreEigenspace (n : ℕ) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covH n l) ≤ centreEigenspace repLorentz 1
```

lemma range_covBarH_le_centreEigenspace [source]

The conjugate Higgs symbols carry the sign +1, for the same reason.

```
lemma range_covBarH_le_centreEigenspace (n : ℕ) (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (h.covBarH n l) ≤ centreEigenspace repLorentz 1
```

lemma derivSubmodule_le_centreEigenspace [source]

***The centre of `SL(2,ℂ)` acts on the Higgs derivative submodules by `+1`**, for any number of covariant derivatives: the Higgs is a Lorentz scalar and the derivative slots are inert at the centre.*

```
lemma derivSubmodule_le_centreEigenspace (n : ℕ) :
    h.derivSubmodule n ≤ centreEigenspace repLorentz 1
```

2.519 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.DerivSubmodule.GaugeWeight

PhyslibAlpha/Particles/StandardModel/Realization/HiggsCov/DerivSubmodule/GaugeWeight.lean on GitHub.

instance derivSubmoduleGaugeWeight [source]

***The gauge weight decomposition of the Higgs derivative submodules**: the join of the decompositions of the Higgs and conjugate-Higgs submodules, whose weights are `(0, 0, ∓1, -3)` and `(0, 0, ±1, 3)` respectively.*

This is an instance: its statement mentions `h`, so unification against the goal recovers the sector and with it the rest of the structure's implicit data.

```
@[implicit_reducible]
noncomputable instance derivSubmoduleGaugeWeight (n : ℕ) :
    GaugeWeightDecomposition rep (h.derivSubmodule n)
```

lemma derivSubmoduleGaugeWeight_supp [source]

The gauge weights occurring in the Higgs derivative submodules: the two Higgs weights `(0, 0, ∓1, -3)` and the two conjugate-Higgs weights `(0, 0, ±1, 3)`.

```
lemma derivSubmoduleGaugeWeight_supp (n : ℕ) :
    (h.derivSubmoduleGaugeWeight n).supp
      = {((0, 0, -1, -3) : GaugeWeight), (0, 0, 1, -3), (0, 0, 1, 3), (0, 0, -1, 3)}
```

2.520 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.MassWeight.Basic

PhyslibAlpha/Particles/StandardModel/Realization/HiggsCov/MassWeight/Basic.lean on GitHub.

lemma derivSubmodule_le_massWeightSubmodule [source]

The derivative submodule sits in the mass-weight submodule of weight `2 * (1 + n)`.

```
lemma derivSubmodule_le_massWeightSubmodule (n : ℕ) :
    h.derivSubmodule n ≤ h.massWeightSubmodule (2 * (1 + n))
```

lemma massWeightSubmodule_eq_derivSubmodule [source]

The weight recursion, with the single-symbol part written as a derivative submodule.

```
lemma massWeightSubmodule_eq_derivSubmodule (i : ℕ) (hi : 0 < i) :
    h.massWeightSubmodule i
      = (⨆ k ∈ Finset.univ.filter (fun k : Fin i => 2 * (1 + (k : ℕ)) = i),
          h.derivSubmodule (k : ℕ))
        ⊔ (⨆ p ∈ Finset.univ.filter (fun p : Fin i × Fin i => (p.1 : ℕ) + (p.2 : ℕ) = i),
            h.massWeightSubmodule (p.1 : ℕ) * h.massWeightSubmodule (p.2 : ℕ))
```

lemma massWeightSubmodule_eq_iSup_derivSubmodule_mul [source]

Removing the leftmost Higgs tower, written with the derivative submodules: a term of positive weight `w` is a sum of products of a tower `derivSubmodule n`, of weight `2 * (1 + n) ≤ w`, with a term of the remaining weight.

```
lemma massWeightSubmodule_eq_iSup_derivSubmodule_mul (w : ℕ) (hw : 0 < w) :
    h.massWeightSubmodule w
      = ⨆ n ∈ (Finset.range (w + 1)).filter (fun n => 2 * (1 + n) ≤ w),
          h.derivSubmodule n * h.massWeightSubmodule (w - 2 * (1 + n))
```

lemma massWeightSubmodule_two_eq_deriv [source]

Weight two is the underived Higgs symbols.

```
lemma massWeightSubmodule_two_eq_deriv :
    h.massWeightSubmodule 2 = h.derivSubmodule 0
```

lemma massWeightSubmodule_four_eq_deriv [source]

Weight four: the leftmost tower is underived, leaving weight two, which is an underived tower, or once-derived, leaving weight zero.

```
lemma massWeightSubmodule_four_eq_deriv :
    h.massWeightSubmodule 4
      = h.derivSubmodule 1 ⊔ h.derivSubmodule 0 * h.derivSubmodule 0
```

lemma massWeightSubmodule_six_eq_deriv [source]

Weight six: the leftmost tower leaves weight four, two or zero. The product of an underived tower with a once-derived one occurs in both orders, which agree by `derivSubmodule_mul_comm`.

```
lemma massWeightSubmodule_six_eq_deriv :
    h.massWeightSubmodule 6
      = h.derivSubmodule 2 ⊔ h.derivSubmodule 1 * h.derivSubmodule 0
        ⊔ h.derivSubmodule 0 * h.derivSubmodule 0 * h.derivSubmodule 0
```

lemma massWeightSubmodule_eight_eq_deriv [source]

Weight eight: the leftmost tower leaves weight six, four, two or zero. Products that differ only in the order of commuting towers agree by `derivSubmodule_mul_comm`.

```
lemma massWeightSubmodule_eight_eq_deriv :
    h.massWeightSubmodule 8
      = h.derivSubmodule 3 ⊔ h.derivSubmodule 2 * h.derivSubmodule 0
        ⊔ h.derivSubmodule 1 * h.derivSubmodule 1
        ⊔ h.derivSubmodule 1 * h.derivSubmodule 0 * h.derivSubmodule 0
        ⊔ h.derivSubmodule 0 * h.derivSubmodule 0 * h.derivSubmodule 0
          * h.derivSubmodule 0
```

2.521 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.MassWeight.GaugeWeight

PhyslibAlpha/Particles/StandardModel/Realization/HiggsCov/MassWeight/GaugeWeight.lean on GitHub.

def massWeightSubmoduleGaugeWeightOdd [source]

The odd mass-weight submodules are trivial, so they carry the empty decomposition.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightOdd (n : ℕ) (hn : Odd n) :
    GaugeWeightDecomposition rep (h.massWeightSubmodule n)
```

def massWeightSubmoduleGaugeWeightOne [source]

Weight one is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightOne :
    GaugeWeightDecomposition rep (h.massWeightSubmodule 1)
```

def massWeightSubmoduleGaugeWeightTwo [source]

Weight two is the underived Higgs tower.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightTwo :
    GaugeWeightDecomposition rep (h.massWeightSubmodule 2)
```

def massWeightSubmoduleGaugeWeightThree [source]

Weight three is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightThree :
    GaugeWeightDecomposition rep (h.massWeightSubmodule 3)
```

def massWeightSubmoduleGaugeWeightFour [source]

Weight four is the once-derived tower together with the products of two underived ones.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightFour :
    GaugeWeightDecomposition rep (h.massWeightSubmodule 4)
```

def massWeightSubmoduleGaugeWeightFive [source]

Weight five is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightFive :
    GaugeWeightDecomposition rep (h.massWeightSubmodule 5)
```

def massWeightSubmoduleGaugeWeightSix [source]

Weight six is the twice-derived tower, the once-derived tower against an underived one, and the products of three underived ones.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightSix :
    GaugeWeightDecomposition rep (h.massWeightSubmodule 6)
```

def massWeightSubmoduleGaugeWeightSeven [source]

Weight seven is trivial.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightSeven :
    GaugeWeightDecomposition rep (h.massWeightSubmodule 7)
```

def massWeightSubmoduleGaugeWeightEight [source]

Weight eight: the thrice-derived tower, the two ways of splitting the derivatives over two towers, the once-derived tower against two underived ones, and the products of four underived ones.

```
@[implicit_reducible]
noncomputable def massWeightSubmoduleGaugeWeightEight :
    GaugeWeightDecomposition rep (h.massWeightSubmodule 8)
```

lemma derivSubmoduleGaugeWeight_piece_eq [source]

The weight-`w` piece of a derivative submodule, as the join of the Higgs and conjugate-Higgs pieces.

```
lemma derivSubmoduleGaugeWeight_piece_eq (n : ℕ) (w : GaugeWeight) :
    (h.derivSubmoduleGaugeWeight n).piece w
      = (if w = ((0, 0, -1, -3) : GaugeWeight) then
          ⨆ d : Fin n → (Fin 1 ⊕ Fin 3), ℂ • h.higgs d 0
        else if w = ((0, 0, 1, -3) : GaugeWeight) then
          ⨆ d : Fin n → (Fin 1 ⊕ Fin 3), ℂ • h.higgs d 1
        else ⊥)
        ⊔ (if w = ((0, 0, 1, 3) : GaugeWeight) then
            ⨆ d : Fin n → (Fin 1 ⊕ Fin 3), ℂ • h.barHiggs d 0
          else if w = ((0, 0, -1, 3) : GaugeWeight) then
            ⨆ d : Fin n → (Fin 1 ⊕ Fin 3), ℂ • h.barHiggs d 1
          else ⊥)
```

lemma derivSubmoduleGaugeWeight_piece_higgs_zero [source]

The piece at the weight of the upper Higgs component.

```
lemma derivSubmoduleGaugeWeight_piece_higgs_zero (n : ℕ) :
    (h.derivSubmoduleGaugeWeight n).piece (0, 0, -1, -3)
      = ⨆ d : Fin n → (Fin 1 ⊕ Fin 3), ℂ • h.higgs d 0
```

lemma derivSubmoduleGaugeWeight_piece_higgs_one [source]

The piece at the weight of the lower Higgs component.

```
lemma derivSubmoduleGaugeWeight_piece_higgs_one (n : ℕ) :
    (h.derivSubmoduleGaugeWeight n).piece (0, 0, 1, -3)
      = ⨆ d : Fin n → (Fin 1 ⊕ Fin 3), ℂ • h.higgs d 1
```

lemma derivSubmoduleGaugeWeight_piece_barHiggs_zero [source]

The piece at the weight of the upper conjugate-Higgs component.

```
lemma derivSubmoduleGaugeWeight_piece_barHiggs_zero (n : ℕ) :
    (h.derivSubmoduleGaugeWeight n).piece (0, 0, 1, 3)
      = ⨆ d : Fin n → (Fin 1 ⊕ Fin 3), ℂ • h.barHiggs d 0
```

lemma derivSubmoduleGaugeWeight_piece_barHiggs_one [source]

The piece at the weight of the lower conjugate-Higgs component.

```
lemma derivSubmoduleGaugeWeight_piece_barHiggs_one (n : ℕ) :
    (h.derivSubmoduleGaugeWeight n).piece (0, 0, -1, 3)
      = ⨆ d : Fin n → (Fin 1 ⊕ Fin 3), ℂ • h.barHiggs d 1
```

lemma derivSubmoduleGaugeWeight_piece_zero [source]

A derivative submodule has no weight-zero content: every Higgs symbol carries hypercharge.

```
lemma derivSubmoduleGaugeWeight_piece_zero (n : ℕ) :
    (h.derivSubmoduleGaugeWeight n).piece 0 = ⊥
```

def higgsBarHiggsSpan [source]

The span of the isospin-diagonal pairings of a Higgs symbol carrying `n` derivatives with a conjugate-Higgs symbol carrying `m` derivatives, at isospin component `i`.

```
noncomputable def higgsBarHiggsSpan (h : HiggsAlgebraCovRealization B rep repLorentz
    massWeightPoly)
    (n m : ℕ) (i : Fin 2) : Submodule ℂ B
```

def quarticSpan [source]

The span of the underived quartic monomial pairing the isospin components `i` and `j`.

```
noncomputable def quarticSpan (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly)
    (i j : Fin 2) : Submodule ℂ B
```

lemma derivSubmodule_mul_piece_zero [source]

The weight-zero piece of a product of two derivative submodules: the isospin-diagonal pairings, taken in both orders of the two towers.

```
lemma derivSubmodule_mul_piece_zero (n m : ℕ) :
    GaugeWeightDecomposition.piece rep (h.derivSubmodule n * h.derivSubmodule m) 0
      = h.higgsBarHiggsSpan n m 0 ⊔ h.higgsBarHiggsSpan n m 1
        ⊔ h.higgsBarHiggsSpan m n 0 ⊔ h.higgsBarHiggsSpan m n 1
```

lemma derivSubmodule_mul_mul_piece_zero [source]

A product of three derivative submodules has no weight-zero content: the hypercharge of three Higgs generators is an odd multiple of three.

```
lemma derivSubmodule_mul_mul_piece_zero (n m k : ℕ) :
    GaugeWeightDecomposition.piece rep
      (h.derivSubmodule n * h.derivSubmodule m * h.derivSubmodule k) 0 = ⊥
```

lemma derivSubmodule_zero_pow_four_piece_zero [source]

The weight-zero piece of the product of four underived derivative submodules: the three quartic monomials, the ones pairing two Higgs symbols against two conjugate ones with matching isospin.

```
lemma derivSubmodule_zero_pow_four_piece_zero :
    GaugeWeightDecomposition.piece rep
      (h.derivSubmodule 0 * h.derivSubmodule 0 * h.derivSubmodule 0
        * h.derivSubmodule 0) 0
      = h.quarticSpan 0 0 ⊔ h.quarticSpan 0 1 ⊔ h.quarticSpan 1 1
```

lemma massWeightSubmoduleGaugeWeightOdd_piece_zero [source]

The weight-zero piece at an odd mass weight: the submodule itself is trivial.

```
lemma massWeightSubmoduleGaugeWeightOdd_piece_zero (n : ℕ) (hn : Odd n) :
    (h.massWeightSubmoduleGaugeWeightOdd n hn).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightTwo_piece_zero [source]

The weight-zero piece at mass weight two: a single Higgs symbol carries hypercharge.

```
lemma massWeightSubmoduleGaugeWeightTwo_piece_zero :
    (h.massWeightSubmoduleGaugeWeightTwo).piece 0 = ⊥
```

lemma massWeightSubmoduleGaugeWeightFour_piece_zero [source]

The weight-zero piece at mass weight four: the underived isospin-diagonal pairings.

```
lemma massWeightSubmoduleGaugeWeightFour_piece_zero :
    (h.massWeightSubmoduleGaugeWeightFour).piece 0
      = h.higgsBarHiggsSpan 0 0 0 ⊔ h.higgsBarHiggsSpan 0 0 1
```

lemma massWeightSubmoduleGaugeWeightSix_piece_zero [source]

The weight-zero piece at mass weight six: the isospin-diagonal pairings carrying one derivative, on either of the two factors.

```
lemma massWeightSubmoduleGaugeWeightSix_piece_zero :
    (h.massWeightSubmoduleGaugeWeightSix).piece 0
      = h.higgsBarHiggsSpan 1 0 0 ⊔ h.higgsBarHiggsSpan 1 0 1
        ⊔ h.higgsBarHiggsSpan 0 1 0 ⊔ h.higgsBarHiggsSpan 0 1 1
```

lemma massWeightSubmoduleGaugeWeightEight_piece_zero [source]

The weight-zero piece at mass weight eight: the isospin-diagonal pairings carrying two derivatives on one factor or one on each, together with the three quartic monomials.

```
lemma massWeightSubmoduleGaugeWeightEight_piece_zero :
    (h.massWeightSubmoduleGaugeWeightEight).piece 0
      = h.higgsBarHiggsSpan 2 0 0 ⊔ h.higgsBarHiggsSpan 2 0 1
        ⊔ h.higgsBarHiggsSpan 0 2 0 ⊔ h.higgsBarHiggsSpan 0 2 1
        ⊔ (h.higgsBarHiggsSpan 1 1 0 ⊔ h.higgsBarHiggsSpan 1 1 1)
        ⊔ (h.quarticSpan 0 0 ⊔ h.quarticSpan 0 1 ⊔ h.quarticSpan 1 1)
```

lemma eq_zero_of_invariant_massWeightSubmodule_odd [source]

A gauge-invariant term of odd mass weight vanishes.

```
lemma eq_zero_of_invariant_massWeightSubmodule_odd (n : ℕ) (hn : Odd n) {x : B}
    (hx : x ∈ h.massWeightSubmodule n) : x = 0
```

lemma eq_zero_of_invariant_massWeightSubmodule_two [source]

A gauge-invariant term of mass weight two vanishes: a single Higgs symbol carries hypercharge, so nothing at that weight is neutral.

```
lemma eq_zero_of_invariant_massWeightSubmodule_two {x : B}
    (hx : x ∈ h.massWeightSubmodule 2) (hg : ∀ g : GaugeGroupI, rep g x = x) : x = 0
```

lemma mem_of_invariant_massWeightSubmodule_four [source]

A gauge-invariant term of mass weight four is a combination of the two underived isospin-diagonal pairings.

```
lemma mem_of_invariant_massWeightSubmodule_four {x : B}
    (hx : x ∈ h.massWeightSubmodule 4) (hg : ∀ g : GaugeGroupI, rep g x = x) :
    x ∈ h.higgsBarHiggsSpan 0 0 0 ⊔ h.higgsBarHiggsSpan 0 0 1
```

lemma mem_of_invariant_massWeightSubmodule_six [source]

A gauge-invariant term of mass weight six is a combination of the isospin-diagonal pairings carrying one derivative, on either factor.

```
lemma mem_of_invariant_massWeightSubmodule_six {x : B}
    (hx : x ∈ h.massWeightSubmodule 6) (hg : ∀ g : GaugeGroupI, rep g x = x) :
    x ∈ h.higgsBarHiggsSpan 1 0 0 ⊔ h.higgsBarHiggsSpan 1 0 1
      ⊔ h.higgsBarHiggsSpan 0 1 0 ⊔ h.higgsBarHiggsSpan 0 1 1
```

lemma mem_of_invariant_massWeightSubmodule_eight [source]

A gauge-invariant term of mass weight eight is a combination of the isospin-diagonal pairings carrying two derivatives and of the three quartic monomials.

```
lemma mem_of_invariant_massWeightSubmodule_eight {x : B}
    (hx : x ∈ h.massWeightSubmodule 8) (hg : ∀ g : GaugeGroupI, rep g x = x) :
    x ∈ h.higgsBarHiggsSpan 2 0 0 ⊔ h.higgsBarHiggsSpan 2 0 1
      ⊔ h.higgsBarHiggsSpan 0 2 0 ⊔ h.higgsBarHiggsSpan 0 2 1
      ⊔ (h.higgsBarHiggsSpan 1 1 0 ⊔ h.higgsBarHiggsSpan 1 1 1)
```

```
    ⊔ (h.quarticSpan 0 0 ⊔ h.quarticSpan 0 1 ⊔ h.quarticSpan 1 1)
```

lemma su2_inv_apply [source]

The entries of the inverse of an `SU(2)` element are the conjugated transposed entries, the inverse of a unitary matrix being its conjugate transpose.

```
lemma su2_inv_apply (V : specialUnitaryGroup (Fin 2) ℂ) (a b : Fin 2) :
    (V⁻¹).1 a b = conj (V.1 b a)
```

lemma rep_su2_higgs [source]

An isospin transformation moves the isospin index of a Higgs symbol by the conjugate matrix: the index of a Higgs symbol is anti-fundamental.

```
lemma rep_su2_higgs (V : specialUnitaryGroup (Fin 2) ℂ) {n : ℕ}
    (d : Fin n → (Fin 1 ⊕ Fin 3)) (i : Fin 2) :
    rep ((1, V, 1) : GaugeGroupI) (h.higgs d i)
      = ∑ a, conj (V.1 a i) • h.higgs d a
```

lemma rep_su2_barHiggs [source]

An isospin transformation moves the isospin index of a conjugate Higgs symbol by the matrix itself: the index of a conjugate Higgs symbol is fundamental.

```
lemma rep_su2_barHiggs (V : specialUnitaryGroup (Fin 2) ℂ) {n : ℕ}
    (d : Fin n → (Fin 1 ⊕ Fin 3)) (i : Fin 2) :
    rep ((1, V, 1) : GaugeGroupI) (h.barHiggs d i)
      = ∑ a, V.1 a i • h.barHiggs d a
```

lemma rep_mul_pair [source]

A product of two symbols each moving by given coefficients moves by the product of those coefficients.

```
lemma rep_mul_pair (g : GaugeGroupI) {ι κ : Type} [Fintype ι] [Fintype κ]
    {X : ι → B} {Y : κ → B} {x₀ : ι} {y₀ : κ} {cX : ι → ℂ} {cY : κ → ℂ}
    (hX : rep g (X x₀) = ∑ x, cX x • X x) (hY : rep g (Y y₀) = ∑ y, cY y • Y y) :
    rep g (X x₀ * Y y₀) = ∑ x, ∑ y, (cX x * cY y) • (X x * Y y)
```

lemma higgs_mul_barHiggs_comm [source]

A Higgs symbol commutes with a conjugate Higgs symbol, in the components.

```
lemma higgs_mul_barHiggs_comm {n m : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3))
    (d' : Fin m → (Fin 1 ⊕ Fin 3)) (i j : Fin 2) :
    h.higgs d i * h.barHiggs d' j = h.barHiggs d' j * h.higgs d i
```

lemma higgs_mul_higgs_comm [source]

Two Higgs symbols commute, in the components.

```
lemma higgs_mul_higgs_comm {n m : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3))
    (d' : Fin m → (Fin 1 ⊕ Fin 3)) (i j : Fin 2) :
    h.higgs d i * h.higgs d' j = h.higgs d' j * h.higgs d i
```

lemma barHiggs_mul_barHiggs_comm [source]

Two conjugate Higgs symbols commute, in the components.

```
lemma barHiggs_mul_barHiggs_comm {n m : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3))
    (d' : Fin m → (Fin 1 ⊕ Fin 3)) (i j : Fin 2) :
    h.barHiggs d i * h.barHiggs d' j = h.barHiggs d' j * h.barHiggs d i
```

def isoFamily [source]

The isospin family of a Higgs tower carrying `n` derivatives against a conjugate tower carrying `m`: the conjugate symbol supplies the fundamental index and so goes in the first slot, the Higgs symbol the anti-fundamental one.

```
noncomputable def isoFamily (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly)
    {n m : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3))
    (d' : Fin m → (Fin 1 ⊕ Fin 3)) : (Fin 2 → Fin 2) → B
```

lemma isSU2FundamentalAntiFundamental_isoFamily [source]

The isospin family carries one fundamental and one anti-fundamental isospin index.

```
lemma isSU2FundamentalAntiFundamental_isoFamily {n m : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3))
    (d' : Fin m → (Fin 1 ⊕ Fin 3)) : IsSU2FundamentalAntiFundamental B rep (h.isoFamily d d')
```

lemma deltaContraction_isoFamily [source]

The delta contraction of the isospin family, which spans its isospin invariants, is the isospin contraction: the Higgs mass term of the two towers.

```
lemma deltaContraction_isoFamily {n m : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3))
    (d' : Fin m → (Fin 1 ⊕ Fin 3)) :
    IsSU2FundamentalAntiFundamental.deltaContraction (h.isoFamily d d') = h.dotGaugeHiggs d d'
```

def dotSpan [source]

The span of the isospin contractions of a Higgs tower carrying `n` derivatives against a conjugate tower carrying `m`: the gauge invariants the isospin classification leaves at those two derivative orders.

```
noncomputable def dotSpan (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly)
    (n m : ℕ) : Submodule ℂ B
```

lemma isoFamily_span_stable [source]

The span of the isospin family is stable under the whole gauge group: each factor of a component goes to a combination of the factors of components.

```
lemma isoFamily_span_stable {n m : ℕ} (d : Fin n → (Fin 1 ⊕ Fin 3))
    (d' : Fin m → (Fin 1 ⊕ Fin 3)) (g : GaugeGroupI) {y : B}
    (hy : y ∈ Submodule.span ℂ (Set.range (h.isoFamily d d'))) :
    rep g y ∈ Submodule.span ℂ (Set.range (h.isoFamily d d'))
```

lemma higgsBarHiggsSpan_le_isoFamily_span [source]

The isospin-diagonal pairing spans of section C sit inside the span of the isospin family: a diagonal pairing is one of the four components, the two factors commuting.

```
lemma higgsBarHiggsSpan_le_isoFamily_span (n m : ℕ) :
    h.higgsBarHiggsSpan n m 0 ⊔ h.higgsBarHiggsSpan n m 1
      ≤ ⨆ (d : Fin n → (Fin 1 ⊕ Fin 3)) (d' : Fin m → (Fin 1 ⊕ Fin 3)),
        Submodule.span ℂ (Set.range (h.isoFamily d d'))
```

def tildeHiggs [source]

The re-index of an underived Higgs symbol by the antisymmetric symbol, `H~⁰ = H¹` and `H~¹ = -H⁰`. `SU(2)` is pseudo-real, so this turns the anti-fundamental index of a Higgs symbol into a fundamental one, which is what the quartic family needs.

```
noncomputable def tildeHiggs (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly)
  (i : Fin 2) : B
```

lemma rep_su2_tildeHiggs [source]

The re-indexed Higgs symbol carries a fundamental isospin index: the four entry identities of `GaugeGroup.SU2Conjugation` remove every complex conjugate.

```
lemma rep_su2_tildeHiggs (V : specialUnitaryGroup (Fin 2) ℂ) (i : Fin 2) :
    rep ((1, V, 1) : GaugeGroupI) (h.tildeHiggs i)
      = ∑ a, V.1 a i • h.tildeHiggs a
```

def quadFamily [source]

The quartic isospin family: two conjugate Higgs symbols against two re-indexed Higgs symbols, each of the four carrying a fundamental isospin index.

```
noncomputable def quadFamily (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly) :
    (Fin 4 → Fin 2) → B
```

lemma isSU2QuadFundamental_quadFamily [source]

The quartic family carries four fundamental isospin indices.

```
lemma isSU2QuadFundamental_quadFamily : IsSU2QuadFundamental B rep h.quadFamily
```

lemma barHiggs_higgs_left_comm_zero [source]

Moving a Higgs symbol past a conjugate one, at no derivatives and inside a product: the normalisation used to compare quartic monomials.

```
private lemma barHiggs_higgs_left_comm_zero (i j : Fin 2) (y : B) :
    h.barHiggs (![] : Fin 0 → (Fin 1 ⊕ Fin 3)) i * (h.higgs ![] j * y)
      = h.higgs ![] j * (h.barHiggs ![] i * y)
```

lemma barHiggs_mul_higgs_comm_zero [source]

Moving a Higgs symbol past a conjugate one, at no derivatives.

```
private lemma barHiggs_mul_higgs_comm_zero (i j : Fin 2) :
    h.barHiggs (![] : Fin 0 → (Fin 1 ⊕ Fin 3)) i * h.higgs ![] j
      = h.higgs ![] j * h.barHiggs ![] i
```

lemma higgs_left_comm_zero [source]

Sorting two Higgs symbols inside a product.

```
private lemma higgs_left_comm_zero (y : B) :
    h.higgs (![] : Fin 0 → (Fin 1 ⊕ Fin 3)) 1 * (h.higgs ![] 0 * y)
      = h.higgs ![] 0 * (h.higgs ![] 1 * y)
```

lemma epsilonContraction₁₂_quadFamily [source]

The first epsilon contraction of the quartic family is the square of the isospin contraction: pairing the first conjugate symbol with the first Higgs symbol, and the second with the second, is pairing each H^* with an H.

```
lemma epsilonContraction₁₂_quadFamily :
    IsSU2QuadFundamental.epsilonContraction₁₂ h.quadFamily
      = h.dotGaugeHiggs ![] ![] * h.dotGaugeHiggs ![] ![]
```

lemma epsilonContraction₁₃_quadFamily [source]

The second epsilon contraction of the quartic family vanishes: it pairs the two conjugate symbols with each other and the two Higgs symbols with each other, and an antisymmetric contraction of two commuting factors is zero.

```
lemma epsilonContraction₁₃_quadFamily :
    IsSU2QuadFundamental.epsilonContraction₁₃ h.quadFamily = 0
```

lemma quarticSpan_le_quadFamily_span [source]

The three quartic monomials of section C lie in the span of the components of the quartic family, each being one of those components up to a sign.

```
lemma quarticSpan_le_quadFamily_span :
    h.quarticSpan 0 0 ⊔ h.quarticSpan 0 1 ⊔ h.quarticSpan 1 1
      ≤ Submodule.span ℂ (Set.range h.quadFamily)
```

def isoSpan [source]

The span of the components of all the isospin families of a Higgs tower carrying n derivatives against a conjugate tower carrying m. This is the gauge-stable enlargement of the pairing span of section C.

```
noncomputable def isoSpan (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly)
    (n m : ℕ) : Submodule ℂ B
```

lemma isStableUnder_isoSpan [source]

The isospin span is stable under the gauge group.

```
lemma isStableUnder_isoSpan (n m : ℕ) :
    IsStableUnder (fun g : GaugeGroupI => rep g) (h.isoSpan n m)
```

lemma isFixedBy_dotSpan [source]

The span of the isospin contractions is fixed pointwise by the gauge group.

```
lemma isFixedBy_dotSpan (n m : ℕ) :
    IsFixedBy (fun g : GaugeGroupI => rep g) (h.dotSpan n m)
```

lemma higgsBarHiggsSpan_le_isoSpan' [source]

Each isospin-diagonal pairing span of section C sits inside the isospin span.

```
lemma higgsBarHiggsSpan_le_isoSpan' (n m : ℕ) (i : Fin 2) :
    h.higgsBarHiggsSpan n m i ≤ h.isoSpan n m
```

lemma higgsBarHiggsSpan_le_isoSpan [source]

The pairing span of section C sits inside the isospin span.

```
lemma higgsBarHiggsSpan_le_isoSpan (n m : ℕ) :
    h.higgsBarHiggsSpan n m 0 ⊔ h.higgsBarHiggsSpan n m 1 ≤ h.isoSpan n m
```

lemma reducesInvariantsTo_isoSpan [source]

The isospin span reduces, for the gauge group, to the span of the isospin contractions: each family's span reduces to the line through its delta contraction, which is the isospin contraction of the two towers.

```
lemma reducesInvariantsTo_isoSpan (n m : ℕ) :
    ReducesInvariantsTo (fun g : GaugeGroupI => rep g) (h.isoSpan n m) (h.dotSpan n m)
```

lemma reducesInvariantsTo_massWeightSubmodule_two [source]

Mass weight two reduces to ⊥ for the gauge group: a single Higgs symbol carries hypercharge, so the weight-zero piece is trivial.

```
lemma reducesInvariantsTo_massWeightSubmodule_two :
    ReducesInvariantsTo (fun g : GaugeGroupI => rep g) (h.massWeightSubmodule 2) ⊥
```

lemma reducesInvariantsTo_massWeightSubmodule_four [source]

Mass weight four reduces, for the gauge group, to the underived isospin contraction, the Higgs mass term.

```
lemma reducesInvariantsTo_massWeightSubmodule_four :
    ReducesInvariantsTo (fun g : GaugeGroupI => rep g) (h.massWeightSubmodule 4)
      (h.dotSpan 0 0)
```

lemma reducesInvariantsTo_massWeightSubmodule_six [source]

Mass weight six reduces, for the gauge group, to the isospin contractions carrying one derivative, on either factor.

```
lemma reducesInvariantsTo_massWeightSubmodule_six :
    ReducesInvariantsTo (fun g : GaugeGroupI => rep g) (h.massWeightSubmodule 6)
      (h.dotSpan 1 0 ⊔ h.dotSpan 0 1)
```

lemma reducesInvariantsTo_massWeightSubmodule_eight [source]

Mass weight eight reduces, for the gauge group, to the isospin contractions carrying two derivatives and the square of the underived one, the quartic potential.

```
lemma reducesInvariantsTo_massWeightSubmodule_eight :
    ReducesInvariantsTo (fun g : GaugeGroupI => rep g) (h.massWeightSubmodule 8)
      (h.dotSpan 2 0 ⊔ h.dotSpan 0 2 ⊔ h.dotSpan 1 1
        ⊔ ℂ • (h.dotGaugeHiggs ![] ![] * h.dotGaugeHiggs ![] ![]))
```

lemma mem_dotSpan_of_invariant_massWeightSubmodule_four [source]

A gauge-invariant term of mass weight four is a multiple of the underived isospin contraction.

```
lemma mem_dotSpan_of_invariant_massWeightSubmodule_four {x : B}
    (hx : x ∈ h.massWeightSubmodule 4) (hg : ∀ g : GaugeGroupI, rep g x = x) :
    x ∈ h.dotSpan 0 0
```

lemma mem_dotSpan_of_invariant_massWeightSubmodule_six [source]

A gauge-invariant term of mass weight six is a combination of the isospin contractions carrying one derivative, on either factor.

```
lemma mem_dotSpan_of_invariant_massWeightSubmodule_six {x : B}
    (hx : x ∈ h.massWeightSubmodule 6) (hg : ∀ g : GaugeGroupI, rep g x = x) :
    x ∈ h.dotSpan 1 0 ⊔ h.dotSpan 0 1
```

lemma mem_dotSpan_of_invariant_massWeightSubmodule_eight [source]

A gauge-invariant term of mass weight eight is a combination of the isospin contractions carrying two derivatives and of the square of the underived contraction.

```
lemma mem_dotSpan_of_invariant_massWeightSubmodule_eight {x : B}
    (hx : x ∈ h.massWeightSubmodule 8) (hg : ∀ g : GaugeGroupI, rep g x = x) :
    x ∈ h.dotSpan 2 0 ⊔ h.dotSpan 0 2 ⊔ h.dotSpan 1 1
      ⊔ ℂ • (h.dotGaugeHiggs ![] ![] * h.dotGaugeHiggs ![] ![])
```

lemma dotGaugeHiggs_mem_massWeightSubmodule [source]

An isospin contraction has the mass weight of its two towers together.

```
lemma dotGaugeHiggs_mem_massWeightSubmodule {n1 n2 : ℕ} (d1 : Fin n1 → (Fin 1 ⊕ Fin 3))
    (d2 : Fin n2 → (Fin 1 ⊕ Fin 3)) :
    h.dotGaugeHiggs d1 d2 ∈ h.massWeightSubmodule (2 * (1 + n1) + 2 * (1 + n2))
```

lemma gaugeInvariantOfMassDim_four_eq_dotSpan [source]

The gauge invariants of mass weight four: the underived isospin contraction.

```
lemma gaugeInvariantOfMassDim_four_eq_dotSpan :
    h.gaugeInvariantOfMassDim 4 = h.dotSpan 0 0
```

lemma gaugeInvariantOfMassDim_six_eq_dotSpan [source]

The gauge invariants of mass weight six: the isospin contractions with one derivative on either factor.

```
lemma gaugeInvariantOfMassDim_six_eq_dotSpan :
    h.gaugeInvariantOfMassDim 6 = h.dotSpan 1 0 ⊔ h.dotSpan 0 1
```

lemma gaugeInvariantOfMassDim_eight_eq_dotSpan [source]

The gauge invariants of mass weight eight: the isospin contractions with two derivatives distributed over the two factors, together with the square of the underived contraction — the quartic potential.

```
lemma gaugeInvariantOfMassDim_eight_eq_dotSpan :
    h.gaugeInvariantOfMassDim 8 = h.dotSpan 2 0 ⊔ h.dotSpan 0 2 ⊔ h.dotSpan 1 1
      ⊔ ℂ • (h.dotGaugeHiggs ![] ![] * h.dotGaugeHiggs ![] ![])
```

2.522 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.MassWeight.MassDimEight

PhyslibAlpha/Particles/StandardModel/Realization/HiggsCov/MassWeight/MassDimEight.lean on GitHub.

lemma sum_cov_one [source]

A sum over families of one covector index is a single sum.

```
lemma sum_cov_one {M : Type*} [AddCommMonoid M] (f : (Fin 1 → Fin 1 ⊕ Fin 3) → M) :
    ∑ d : Fin 1 → Fin 1 ⊕ Fin 3, f d = ∑ x : Fin 1 ⊕ Fin 3, f ![x]
```

lemma etaExpand_cov_one [source]

A family of one covector index is the tuple of its own entry.

```
lemma etaExpand_cov_one (l : Fin 1 → Fin 1 ⊕ Fin 3) : ![l 0] = l
```

lemma isLorentzCovariant_rankTwo_dotGaugeHiggs_left [source]

Both derivatives on the Higgs tower: a bi-Lorentz tensor in the two derivative slots.

```
lemma isLorentzCovariant_rankTwo_dotGaugeHiggs_left :
    IsLorentzCovariant 2 B repLorentz
      (ofComponents fun d : Fin 2 → Fin 1 ⊕ Fin 3 => h.dotGaugeHiggs d ![])
```

lemma isLorentzCovariant_rankTwo_dotGaugeHiggs_right [source]

Both derivatives on the conjugate tower: a bi-Lorentz tensor in the same way.

```
lemma isLorentzCovariant_rankTwo_dotGaugeHiggs_right :
    IsLorentzCovariant 2 B repLorentz
      (ofComponents fun d : Fin 2 → Fin 1 ⊕ Fin 3 => h.dotGaugeHiggs ![] d)
```

lemma isLorentzCovariant_rankTwo_dotGaugeHiggs_mixed [source]

One derivative on each tower: the family whose metric contraction is the kinetic term.

```
lemma isLorentzCovariant_rankTwo_dotGaugeHiggs_mixed :
    IsLorentzCovariant 2 B repLorentz
      (ofComponents fun d : Fin 2 → Fin 1 ⊕ Fin 3 => h.dotGaugeHiggs ![d 0] ![d 1])
```

lemma dotSpan_two_zero_eq [source]

The span of the isospin contractions with both derivatives on the Higgs tower is the span of the components of the corresponding bi-Lorentz tensor.

```
lemma dotSpan_two_zero_eq :
    h.dotSpan 2 0
      = Submodule.span ℂ (Set.range fun d : Fin 2 → Fin 1 ⊕ Fin 3 => h.dotGaugeHiggs d ![])
```

lemma dotSpan_zero_two_eq [source]

The span of the isospin contractions with both derivatives on the conjugate tower is the span of the components of the corresponding bi-Lorentz tensor.

```
lemma dotSpan_zero_two_eq :
    h.dotSpan 0 2
      = Submodule.span ℂ (Set.range fun d : Fin 2 → Fin 1 ⊕ Fin 3 => h.dotGaugeHiggs ![] d)
```

lemma dotSpan_one_one_eq [source]

The span of the isospin contractions with one derivative on each tower is the span of the components of the mixed bi-Lorentz tensor.

```
lemma dotSpan_one_one_eq :
    h.dotSpan 1 1
      = Submodule.span ℂ
        (Set.range fun d : Fin 2 → Fin 1 ⊕ Fin 3 => h.dotGaugeHiggs ![d 0] ![d 1])
```

lemma rep_ofComponents_metric [source]

The metric trace of a family of gauge invariants is a gauge invariant.

```
lemma rep_ofComponents_metric {T : (Fin 2 → Fin 1 ⊕ Fin 3) → B}
    (hTG : ∀ (g : GaugeGroupI) (d : Fin 2 → Fin 1 ⊕ Fin 3), rep g (T d) = T d)
    (g : GaugeGroupI) :
    rep g (ofComponents T RankTwo.metric) = ofComponents T RankTwo.metric
```

def lorentzContractionEightSpan [source]

The gauge and Lorentz invariants of the Higgs sector at mass weight eight: the two box terms $\Box H^\dagger H$ and $H^\dagger \Box H$, the kinetic term $\partial^\mu H^\dagger \partial_\mu H$, and the quartic potential $(H^\dagger H)^2$.

```
noncomputable def lorentzContractionEightSpan
    (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly) :
    Submodule ℂ B
```

lemma invariant_dotGaugeHiggs_sq [source]

The square of the underived isospin contraction is fixed by both groups: it is a product of two invariants and both representations are multiplicative.

```
lemma invariant_dotGaugeHiggs_sq :
    (∀ g : SL(2,ℂ), repLorentz g (h.dotGaugeHiggs (![] : Fin 0 → Fin 1 ⊕ Fin 3) ![]
        * h.dotGaugeHiggs ![] ![])
      = h.dotGaugeHiggs ![] ![] * h.dotGaugeHiggs ![] ![])
    ∧ ∀ g : GaugeGroupI, rep g (h.dotGaugeHiggs (![] : Fin 0 → Fin 1 ⊕ Fin 3) ![]
        * h.dotGaugeHiggs ![] ![])
      = h.dotGaugeHiggs ![] ![] * h.dotGaugeHiggs ![] ![]
```

lemma ofComponents_metric_mem_massWeightSubmodule [source]

The metric trace of a family of elements of mass weight eight has mass weight eight.

```
lemma ofComponents_metric_mem_massWeightSubmodule {T : (Fin 2 → Fin 1 ⊕ Fin 3) → B}
    (hT : ∀ d, T d ∈ h.massWeightSubmodule 8) :
    ofComponents T RankTwo.metric ∈ h.massWeightSubmodule 8
```

lemma lorentzContractionEightSpan_le_massWeightSubmodule [source]

The weight-eight span lies in the mass-weight submodule of weight eight.

```
lemma lorentzContractionEightSpan_le_massWeightSubmodule :
    h.lorentzContractionEightSpan ≤ h.massWeightSubmodule 8
```

lemma rep_of_mem_lorentzContractionEightSpan [source]

Every element of the weight-eight span is a gauge invariant.

```
lemma rep_of_mem_lorentzContractionEightSpan (g : GaugeGroupI) {y : B}
    (hy : y ∈ h.lorentzContractionEightSpan) : rep g y = y
```

lemma repLorentz_of_mem_lorentzContractionEightSpan [source]

Every element of the weight-eight span is a Lorentz invariant.

```
lemma repLorentz_of_mem_lorentzContractionEightSpan (g : SL(2,ℂ)) {y : B}
    (hy : y ∈ h.lorentzContractionEightSpan) : repLorentz g y = y
```

lemma isFixedBy_lorentzContractionEightSpan [source]

The weight-eight span is fixed pointwise by both groups.

```
lemma isFixedBy_lorentzContractionEightSpan :
    IsFixedBy (gaugeLorentzMaps rep repLorentz) h.lorentzContractionEightSpan
```

lemma reducesInvariantsTo_lorentzContractionEightSpan [source]

The Higgs sector at mass weight eight reduces, for the gauge and Lorentz groups together, to the two box terms, the kinetic term and the quartic potential. The gauge group leaves the isospin contractions with two derivatives and the square of the underived one; the Lorentz group contracts the two derivative slots of each family with the metric, and the square, which carries no index, is fixed.

```
lemma reducesInvariantsTo_lorentzContractionEightSpan :
    ReducesInvariantsTo (gaugeLorentzMaps rep repLorentz) (h.massWeightSubmodule 8)
      h.lorentzContractionEightSpan
```

lemma mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff [source]

The gauge and Lorentz classification of mass weight eight as an equivalence: an element of `massWeightSubmodule 8 ⊔ S` is fixed by both groups exactly when it is a combination of the two box terms, the kinetic term and the quartic potential, up to a remainder in `S` fixed by both groups.

```
lemma mem_massWeightSubmodule_eight_sup_and_gauge_lorentz_invariant_iff
    (S : Submodule ℂ B) (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, rep g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule 8 ⊔ S ∧ (∀ g : GaugeGroupI, rep g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, rep g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ h.lorentzContractionEightSpan
```

2.523 PhyslibAlpha.Particles.StandardModel.Realization.HiggsCov.MassWeight.MassDimLTEight

PhyslibAlpha/Particles/StandardModel/Realization/HiggsCov/MassWeight/MassDimLTEight.lean on GitHub.

lemma sum_cov_zero [source]

A sum over families of no covector indices is its single term.

```
lemma sum_cov_zero {M : Type*} [AddCommMonoid M] (f : (Fin 0 → Fin 1 ⊕ Fin 3) → M) :
    ∑ d : Fin 0 → Fin 1 ⊕ Fin 3, f d = f ![]
```

lemma isLorentzCovariant_rankOne_dotGaugeHiggs_left [source]

The isospin contraction of a once-derived Higgs tower against an underived conjugate tower, read as a family indexed by its derivative slot, is a Lorentz vector.

```
lemma isLorentzCovariant_rankOne_dotGaugeHiggs_left :
    IsLorentzCovariant 1 B repLorentz
      (ofComponents fun d : Fin 1 → Fin 1 ⊕ Fin 3 => h.dotGaugeHiggs d ![])
```

lemma isLorentzCovariant_rankOne_dotGaugeHiggs_right [source]

The isospin contraction of an underived Higgs tower against a once-derived conjugate tower is a Lorentz vector in the same way.

```
lemma isLorentzCovariant_rankOne_dotGaugeHiggs_right :
    IsLorentzCovariant 1 B repLorentz
      (ofComponents fun d : Fin 1 → Fin 1 ⊕ Fin 3 => h.dotGaugeHiggs ![] d)
```

lemma dotSpan_one_zero_eq [source]

The span of the isospin contractions with one derivative on the Higgs tower is the span of the components of the corresponding Lorentz vector.

```
lemma dotSpan_one_zero_eq :
    h.dotSpan 1 0
      = Submodule.span ℂ (Set.range fun d : Fin 1 → Fin 1 ⊕ Fin 3 => h.dotGaugeHiggs d ![])
```

lemma dotSpan_zero_one_eq [source]

The span of the isospin contractions with one derivative on the conjugate tower is the span of the components of the corresponding Lorentz vector.

```
lemma dotSpan_zero_one_eq :
    h.dotSpan 0 1
      = Submodule.span ℂ (Set.range fun d : Fin 1 → Fin 1 ⊕ Fin 3 => h.dotGaugeHiggs ![] d)
```

lemma repLorentz_dotGaugeHiggs_zero [source]

The underived isospin contraction is a Lorentz scalar.

```
lemma repLorentz_dotGaugeHiggs_zero (g : SL(2,ℂ)) :
    repLorentz g (h.dotGaugeHiggs (![] : Fin 0 → Fin 1 ⊕ Fin 3) ![])
      = h.dotGaugeHiggs ![] ![]
```

lemma dotSpan_zero_zero_eq [source]

The span of the underived isospin contractions is the line through the mass term.

```
lemma dotSpan_zero_zero_eq :
    h.dotSpan 0 0 = ℂ • h.dotGaugeHiggs (![] : Fin 0 → Fin 1 ⊕ Fin 3) ![]
```

lemma repLorentz_of_mem_dotSpan_zero_zero [source]

Every element of the line through the underived isospin contraction is a Lorentz invariant.

```
lemma repLorentz_of_mem_dotSpan_zero_zero (g : SL(2,ℂ)) {y : B} (hy : y ∈ h.dotSpan 0 0) :
    repLorentz g y = y
```

lemma rep_of_mem_dotSpan [source]

Every element of a span of isospin contractions is a gauge invariant.

```
lemma rep_of_mem_dotSpan {n m : ℕ} (g : GaugeGroupI) {y : B} (hy : y ∈ h.dotSpan n m) :
    rep g y = y
```

lemma dotSpan_zero_zero_le_massWeightSubmodule [source]

The line through the underived isospin contraction lies in mass weight four.

```
lemma dotSpan_zero_zero_le_massWeightSubmodule :
    h.dotSpan 0 0 ≤ h.massWeightSubmodule 4
```

lemma reducesInvariantsTo_dotSpan_one_zero_sup_zero_one [source]

The once-derived isospin contractions reduce to ⊥ for the Lorentz group: each span is spanned by a Lorentz vector, and a single covector index carries no invariant.

```
lemma reducesInvariantsTo_dotSpan_one_zero_sup_zero_one :
    ReducesInvariantsTo (fun g : SL(2,ℂ) => repLorentz g) (h.dotSpan 1 0 ⊔ h.dotSpan 0 1) ⊥
```

def lorentzContractionLTEightSpan [source]

The gauge and Lorentz invariants of the Higgs sector at mass weight `w` for `0 < w < 8`: the line through the Higgs mass term at weight four, and nothing at any other weight.

```
noncomputable def lorentzContractionLTEightSpan
    (h : HiggsAlgebraCovRealization B rep repLorentz massWeightPoly)
    (w : ℕ) : Submodule ℂ B
```

lemma lorentzContractionLTEightSpan_le_massWeightSubmodule [source]

The surviving span at weight `w` lies in the mass-weight submodule of weight `w`.

```
lemma lorentzContractionLTEightSpan_le_massWeightSubmodule (w : ℕ) :
    h.lorentzContractionLTEightSpan w ≤ h.massWeightSubmodule w
```

lemma rep_of_mem_lorentzContractionLTEightSpan [source]

Every element of the surviving span is a gauge invariant.

```
lemma rep_of_mem_lorentzContractionLTEightSpan (w : ℕ) (g : GaugeGroupI) {y : B}
    (hy : y ∈ h.lorentzContractionLTEightSpan w) : rep g y = y
```

lemma repLorentz_of_mem_lorentzContractionLTEightSpan [source]

Every element of the surviving span is a Lorentz invariant.

```
lemma repLorentz_of_mem_lorentzContractionLTEightSpan (w : ℕ) (g : SL(2,ℂ)) {y : B}
    (hy : y ∈ h.lorentzContractionLTEightSpan w) : repLorentz g y = y
```

lemma isFixedBy_lorentzContractionLTEightSpan [source]

The surviving span below weight eight is fixed pointwise by both groups.

```
lemma isFixedBy_lorentzContractionLTEightSpan (w : ℕ) :
    IsFixedBy (gaugeLorentzMaps rep repLorentz) (h.lorentzContractionLTEightSpan w)
```

lemma reducesInvariantsTo_lorentzContractionLTEightSpan [source]

Below mass weight eight the Higgs sector reduces, for the gauge and Lorentz groups together, to the surviving span. The four odd weights are trivial submodules, weight two dies on hypercharge, weight four leaves the Higgs mass term, which the Lorentz group fixes, and at weight six the gauge group leaves the once-derived contractions and the Lorentz group nothing.

```
lemma reducesInvariantsTo_lorentzContractionLTEightSpan {w : ℕ} (hw0 : 0 < w) (hw : w < 8) :
    ReducesInvariantsTo (gaugeLorentzMaps rep repLorentz) (h.massWeightSubmodule w)
      (h.lorentzContractionLTEightSpan w)
```

lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff [source]

The classification below mass weight eight as an equivalence, in the shape of `mem_massWeightSubmodule_eigh` an element of `massWeightSubmodule w ⊔ S` for `0 < w < 8` is fixed by both groups exactly when it is an element of the surviving span up to a remainder in `S` fixed by both groups. That span is the line through the Higgs mass term at weight four and trivial elsewhere, so at every weight but four this says `x = y`, as in the gauge and Yukawa sectors.

```
lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff (w : ℕ)
    (hw0 : 0 < w) (hw : w < 8) (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, rep g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule w ⊔ S ∧ (∀ g : GaugeGroupI, rep g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ y ∈ S, (∀ g : GaugeGroupI, rep g y = y)
          ∧ (∀ g : SL(2,ℂ), repLorentz g y = y)
          ∧ x - y ∈ h.lorentzContractionLTEightSpan w
```

lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff_mem [source]

The same classification without the existential: below mass weight eight an element of `massWeightSubmodule w ⊔ S` fixed by both groups is an element of the surviving span joined with `S` fixed by both groups, and conversely.

```
lemma mem_massWeightSubmodule_lt_eight_sup_and_gauge_lorentz_invariant_iff_mem (w : ℕ)
    (hw0 : 0 < w) (hw : w < 8) (S : Submodule ℂ B)
    (hS : ∀ g : GaugeGroupI, ∀ y ∈ S, rep g y ∈ S)
    (hSL : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) (x : B) :
    (x ∈ h.massWeightSubmodule w ⊔ S ∧ (∀ g : GaugeGroupI, rep g x = x)
        ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ (x ∈ h.lorentzContractionLTEightSpan w ⊔ S ∧ (∀ g : GaugeGroupI, rep g x = x)
          ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
```

2.524 PhyslibAlpha.Particles.StandardModel.Realization.MassWeight.Basic

PhyslibAlpha/Particles/StandardModel/Realization/MassWeight/Basic.lean on GitHub.

lemma map_massWeight_monomial [source]

A monomial mass-weight eigenvalue transports along the defining map: the map carries the mass-weight polynomial to the mass-weight polynomial, and a monomial to a monomial.

```
private lemma map_massWeight_monomial {x : JetAlgebra} {n : ℕ}
    (hx : JetAlgebra.massWeightPoly x = Polynomial.monomial n x) :
    massWeightPoly (h.toAlgHom x) = Polynomial.monomial n (h.toAlgHom x)
```

lemma massWeight_H [source]

The law `massWeight_H` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_H : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.H s φ) = Polynomial.monomial (2 * (1 + Multiset.card s)) (h.H s φ)
```

lemma massWeight_barH [source]

The law `massWeight_barH` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_barH : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.barH s φ) = Polynomial.monomial (2 * (1 + Multiset.card s)) (h.barH s φ)
```

lemma massWeight_A [source]

The law `massWeight_A` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_A : ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) μ φ,
    massWeightPoly (h.A s μ φ) = Polynomial.monomial (2 * (1 + Multiset.card s)) (h.A s μ φ)
```

lemma massWeight_d [source]

The law `massWeight_d` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_d : ∀ i (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.d i s φ) = Polynomial.monomial (3 + 2 * Multiset.card s) (h.d i s φ)
```

lemma massWeight_bard [source]

The law `massWeight_bard` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_bard : ∀ i (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.bard i s φ) = Polynomial.monomial (3 + 2 * Multiset.card s) (h.bard i s
    φ)
```

lemma massWeight_u [source]

The law `massWeight_u` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_u : ∀ i (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.u i s φ) = Polynomial.monomial (3 + 2 * Multiset.card s) (h.u i s φ)
```

lemma massWeight_baru [source]

The law `massWeight_baru` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_baru : ∀ i (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.baru i s φ) = Polynomial.monomial (3 + 2 * Multiset.card s) (h.baru i s
    φ)
```

lemma massWeight_Q [source]

The law `massWeight_Q` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_Q : ∀ i (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.Q i s φ) = Polynomial.monomial (3 + 2 * Multiset.card s) (h.Q i s φ)
```

lemma massWeight_barQ [source]

The law `massWeight_barQ` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_barQ : ∀ i (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.barQ i s φ) = Polynomial.monomial (3 + 2 * Multiset.card s) (h.barQ i s
    φ)
```

lemma massWeight_L [source]

The law `massWeight_L` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_L : ∀ i (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.L i s φ) = Polynomial.monomial (3 + 2 * Multiset.card s) (h.L i s φ)
```

lemma massWeight_barL [source]

The law `massWeight_barL` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_barL : ∀ i (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.barL i s φ) = Polynomial.monomial (3 + 2 * Multiset.card s) (h.barL i s
    φ)
```

lemma massWeight_e [source]

The law `massWeight_e` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_e : ∀ i (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.e i s φ) = Polynomial.monomial (3 + 2 * Multiset.card s) (h.e i s φ)
```

lemma massWeight_bare [source]

The law `massWeight_bare` of a Standard Model, obtained from the corresponding law of the jet algebra by pushing it along the defining algebra map.

```
lemma massWeight_bare : ∀ i (s : Multiset (Fin 1 ⊕ Fin 3)) φ,
    massWeightPoly (h.bare i s φ) = Polynomial.monomial (3 + 2 * Multiset.card s) (h.bare i s
    φ)
```

2.525 PhyslibAlpha.Particles.StandardModel.Realization.MassWeight.Filtration

PhyslibAlpha/Particles/StandardModel/Realization/MassWeight/Filtration.lean on GitHub.

def massWeightSubmodule [source]

All elements of the field algebra of mass weight exactly `n`: the intersection of the algebra generated by the bare symbols with the part on which `massWeightPoly` is the monomial `X ^ n`. This is the jet-form counterpart of `CovAlgebraRealization.massWeightSubmodule`.

```
noncomputable def massWeightSubmodule
    (h : AlgebraRealization B repJet repLorentz massWeightPoly) (n : ℕ) : Submodule ℂ B
```

lemma massWeightPoly_of_mem_massWeightSubmodule [source]

On the mass-weight submodule of weight `n` the map `massWeightPoly` is the monomial `X ^ n`, which is what the kernel condition says.

```
lemma massWeightPoly_of_mem_massWeightSubmodule {n : ℕ} {x : B}
    (hx : x ∈ h.massWeightSubmodule n) :
    massWeightPoly x = Polynomial.monomial n x
```

lemma massWeightSubmodule_le_fieldAlgebra [source]

Each graded piece lies in the field algebra.

```
lemma massWeightSubmodule_le_fieldAlgebra (w : ℕ) :
    h.massWeightSubmodule w ≤ h.fieldAlgebra.toSubmodule
```

def massWeightSubmoduleLE [source]

The elements of the field algebra of mass weight at most `w`: the join of the mass-weight submodules of weight `0` through `w`. This is where a Lagrangian lives, a sum of terms of every mass dimension up to a cut-off rather than of a single one.

```
noncomputable def massWeightSubmoduleLE (w : ℕ) : Submodule ℂ B
```

lemma massWeightSubmodule_le_massWeightSubmoduleLE [source]

Each graded piece of weight at most `w` sits inside the filtration at `w`.

```
lemma massWeightSubmodule_le_massWeightSubmoduleLE {k w : ℕ} (hk : k ≤ w) :
    h.massWeightSubmodule k ≤ h.massWeightSubmoduleLE w
```

lemma mem_massWeightSubmoduleLE [source]

An element of a graded piece of weight at most `w` lies in the filtration at `w`.

```
lemma mem_massWeightSubmoduleLE {k w : ℕ} (hk : k ≤ w) {x : B}
    (hx : x ∈ h.massWeightSubmodule k) : x ∈ h.massWeightSubmoduleLE w
```

lemma massWeightSubmoduleLE_le [source]

A submodule containing every graded piece of weight at most `w` contains the filtration at `w`: the join is taken over exactly those pieces.

```
lemma massWeightSubmoduleLE_le {w : ℕ} {V : Submodule ℂ B}
    (hV : ∀ k ≤ w, h.massWeightSubmodule k ≤ V) : h.massWeightSubmoduleLE w ≤ V
```

lemma massWeightSubmoduleLE_mono [source]

The filtration grows with the bound.

```
lemma massWeightSubmoduleLE_mono {w w' : ℕ} (hw : w ≤ w') :
    h.massWeightSubmoduleLE w ≤ h.massWeightSubmoduleLE w'
```

lemma massWeightSubmoduleLE_le_fieldAlgebra [source]

The filtration lies in the field algebra, being a join of pieces that do.

```
lemma massWeightSubmoduleLE_le_fieldAlgebra (w : ℕ) :
    h.massWeightSubmoduleLE w ≤ h.fieldAlgebra.toSubmodule
```

lemma covAlgebra_le_fieldAlgebra [source]

The covariant subalgebra sits inside the field algebra: the field-strength tower is a polynomial in the gauge-field symbols, and the matter towers generate the same algebra as the bare matter symbols.

```
lemma covAlgebra_le_fieldAlgebra : h.covAlgebra ≤ h.fieldAlgebra
```

lemma covAlgebra_toSubmodule_le_fieldAlgebra [source]

The covariant subalgebra as a submodule sits inside the field algebra.

```
lemma covAlgebra_toSubmodule_le_fieldAlgebra :
    h.covAlgebra.toSubmodule ≤ h.fieldAlgebra.toSubmodule
```

lemma toCovAlgebraRealization_fieldAlgebra [source]

The field algebra of the covariant form of the theory is the covariant subalgebra: the two are generated by the same set of covariant towers.

```
lemma toCovAlgebraRealization_fieldAlgebra :
    h.toCovAlgebraRealization.fieldAlgebra = h.covAlgebra
```

lemma covMassWeightSubmodule_eq [source]

A covariant weight piece is a weight piece that happens to be covariant: the two submodules cut the same kernel of `massWeightPoly` out of the two algebras, and one algebra sits inside the other.

```
lemma covMassWeightSubmodule_eq (w : ℕ) :
    h.toCovAlgebraRealization.massWeightSubmodule w
      = h.massWeightSubmodule w ⊓ h.covAlgebra.toSubmodule
```

lemma covMassWeightSubmodule_le [source]

A covariant weight piece lies in the weight piece of the field algebra.

```
lemma covMassWeightSubmodule_le (w : ℕ) :
    h.toCovAlgebraRealization.massWeightSubmodule w ≤ h.massWeightSubmodule w
```

lemma covMassWeightSubmoduleLE_le [source]

The covariant filtration lies in the filtration of the field algebra.

```
lemma covMassWeightSubmoduleLE_le (w : ℕ) :
    h.toCovAlgebraRealization.massWeightSubmoduleLE w ≤ h.massWeightSubmoduleLE w
```

lemma covMassWeightSubmoduleLE_le_covAlgebra [source]

The covariant filtration lies in the covariant subalgebra.

```
lemma covMassWeightSubmoduleLE_le_covAlgebra (w : ℕ) :
    h.toCovAlgebraRealization.massWeightSubmoduleLE w ≤ h.covAlgebra.toSubmodule
```

lemma eq_sum_coeff_massWeightPoly [source]

The weight decomposition of an element of the filtration is canonical: an element of mass weight at most `w` is the sum of the coefficients of `X ^ 0, …, X ^ w` in its image under `massWeightPoly`. Both sides are linear in the element, so it is enough to check it on a single graded piece, where `massWeightPoly` is a monomial.

```
lemma eq_sum_coeff_massWeightPoly {w : ℕ} {x : B} (hx : x ∈ h.massWeightSubmoduleLE w) :
    x = ∑ k ∈ Finset.range (w + 1), (massWeightPoly x).coeff k
```

lemma mem_covMassWeightSubmoduleLE [source]

A covariant element of the filtration lies in the covariant filtration. Its weight components are the coefficients of `massWeightPoly`, and reading off a coefficient does not leave the covariant algebra: on the covariant algebra the coefficient of `X ^ k` lands in the span of the covariant words of weight `k`.

```
lemma mem_covMassWeightSubmoduleLE {w : ℕ} {x : B} (hx : x ∈ h.massWeightSubmoduleLE w)
    (hcov : x ∈ h.covAlgebra) : x ∈ h.toCovAlgebraRealization.massWeightSubmoduleLE w
```

lemma covStandardModelSpanLE_le_covAlgebra [source]

The span of the covariant filtration lies in the covariant subalgebra.

```
lemma covStandardModelSpanLE_le_covAlgebra (w : ℕ) :
    h.toCovAlgebraRealization.standardModelSpanLE w ≤ h.covAlgebra.toSubmodule
```

lemma forall_repJet_of_mem_covStandardModelSpanLE [source]

Every element of the span of the covariant filtration is invariant under the whole jet gauge group, not just the global one: it is covariant and globally invariant, which is what the reduction theorem asks for.

```
lemma forall_repJet_of_mem_covStandardModelSpanLE {w : ℕ} {y : B}
    (hy : y ∈ h.toCovAlgebraRealization.standardModelSpanLE w) (U : JetGaugeGroupI) :
    repJet U y = y
```

lemma mem_massWeightSubmoduleLE_eight_sup_and_invariant_iff [source]

The classification of the Standard Model up to mass dimension four in its jet form: an element of `massWeightSubmoduleLE 8 ⊔ S`, for `S` a covariant submodule stable under the jet gauge group and the Lorentz group, is fixed by both groups exactly when it lies in the span of the covariant filtration up to a remainder in `S` fixed by both groups.

The hypothesis `hScov` has no counterpart in the covariant statement. It is what repairs a decomposition `x = v + s`: the reduction puts `x` in the covariant algebra, and `v` is then covariant only because `s` is. Jet-gauge stability of `S` gives global stability for free, `repGlobal` being `repJet` at a constant jet.

```
lemma mem_massWeightSubmoduleLE_eight_sup_and_invariant_iff (S : Submodule ℂ B)
    (hS : ∀ U : JetGaugeGroupI, ∀ y ∈ S, repJet U y ∈ S)
    (hSL : ∀ Λ : SL(2,ℂ), ∀ y ∈ S, repLorentz Λ y ∈ S)
    (hScov : S ≤ h.covAlgebra.toSubmodule) (x : B) :
    (x ∈ h.massWeightSubmoduleLE 8 ⊔ S ∧ (∀ U : JetGaugeGroupI, repJet U x = x)
        ∧ ∀ Λ : SL(2,ℂ), repLorentz Λ x = x)
      ↔ ∃ y ∈ S, (∀ U : JetGaugeGroupI, repJet U y = y)
          ∧ (∀ Λ : SL(2,ℂ), repLorentz Λ y = y)
          ∧ x - y ∈ h.toCovAlgebraRealization.standardModelSpanLE 8
```

lemma mem_massWeightSubmoduleLE_eight_sup_and_invariant_iff_lagrangian [source]

The invariant content of the Standard Model up to mass dimension four, in its jet form. An element of `massWeightSubmoduleLE 8 ⊔ S`, for `S` a covariant submodule stable under both groups, is fixed by the jet gauge group and the Lorentz group exactly when it is a combination of the constant term, of mass dimension zero, the Higgs mass term `H† H`, of mass dimension two (`HiggsAlgebraCovRealization.dotSpan`), and the dimension-four span — the four Lorentz contractions of the three `F·F` trace families and of the twice-derived hypercharge field strength, among them the gauge kinetic and theta terms (`IsGaugeSector.lorentzContractionEightSpan`), the Higgs kinetic term, its quartic potential and its two box terms (`HiggsAlgebraCovRealization.lorentzContractionEightSpan`), the kinetic terms of the ten fermion species over the nine family pairs (`IsFermionSector.kineticSpan`), and the six Yukawa couplings over the nine family pairs (`yukawaSpan`) — up to a remainder in `S` fixed by both groups, and nothing else.

```
lemma mem_massWeightSubmoduleLE_eight_sup_and_invariant_iff_lagrangian (S : Submodule ℂ B)
    (hS : ∀ U : JetGaugeGroupI, ∀ y ∈ S, repJet U y ∈ S)
    (hSL : ∀ Λ : SL(2,ℂ), ∀ y ∈ S, repLorentz Λ y ∈ S)
    (hScov : S ≤ h.covAlgebra.toSubmodule) (x : B) :
```

```
    (x ∈ h.massWeightSubmoduleLE 8 ⊔ S ∧ (∀ U : JetGaugeGroupI, repJet U x = x)
        ∧ ∀ Λ : SL(2,ℂ), repLorentz Λ x = x)
      ↔ ∃ y ∈ S, (∀ U : JetGaugeGroupI, repJet U y = y)
          ∧ (∀ Λ : SL(2,ℂ), repLorentz Λ y = y)
          ∧ x - y ∈ (1 : Submodule ℂ B) ⊔ (h.toCovAlgebraRealization.isHiggsSector.dotSpan 0 0
              ⊔ (h.toCovAlgebraRealization.isGaugeSector.lorentzContractionEightSpan
                    ⊔ h.toCovAlgebraRealization.isHiggsSector.lorentzContractionEightSpan
                ⊔ (h.toCovAlgebraRealization.isFermionSector.kineticSpan
                  ⊔ h.toCovAlgebraRealization.yukawaSpan)))
```

lemma mem_massWeightSubmoduleLE_eight_and_invariant_iff [source]

The invariants of the filtration, with nothing set aside: an element of mass weight at most eight is fixed by the jet gauge group and the Lorentz group exactly when it lies in the span of the covariant filtration. This is `mem_massWeightSubmoduleLE_eight_sup_and_invariant_iff` at `S = ⊥`, where the stability and covariance hypotheses hold vacuously.

```
lemma mem_massWeightSubmoduleLE_eight_and_invariant_iff (x : B) :
    (x ∈ h.massWeightSubmoduleLE 8 ∧ (∀ U : JetGaugeGroupI, repJet U x = x)
        ∧ ∀ Λ : SL(2,ℂ), repLorentz Λ x = x)
      ↔ x ∈ h.toCovAlgebraRealization.standardModelSpanLE 8
```

lemma mem_massWeightSubmoduleLE_eight_and_invariant_iff_lagrangian [source]

The invariant content of the Standard Model up to mass dimension four, with nothing set aside: an element of mass weight at most eight is fixed by the jet gauge group and the Lorentz group exactly when it is a combination of the constant term, the Higgs mass term `H† H`, and the Standard-Model Lagrangian of mass dimension four, and nothing else.

```
lemma mem_massWeightSubmoduleLE_eight_and_invariant_iff_lagrangian (x : B) :
    (x ∈ h.massWeightSubmoduleLE 8 ∧ (∀ U : JetGaugeGroupI, repJet U x = x)
        ∧ ∀ Λ : SL(2,ℂ), repLorentz Λ x = x)
      ↔ x ∈ (1 : Submodule ℂ B) ⊔ (h.toCovAlgebraRealization.isHiggsSector.dotSpan 0 0
          ⊔ (h.toCovAlgebraRealization.isGaugeSector.lorentzContractionEightSpan
                ⊔ h.toCovAlgebraRealization.isHiggsSector.lorentzContractionEightSpan
            ⊔ (h.toCovAlgebraRealization.isFermionSector.kineticSpan
              ⊔ h.toCovAlgebraRealization.yukawaSpan)))
```

2.526 PhyslibAlpha.ProbabilisticTheory.CStarAlgebra.GNSAutomorphism

PhyslibAlpha/ProbabilisticTheory/CStarAlgebra/GNSAutomorphism.lean on GitHub.

def preGNSAut [source]

The map `[a] ↦ [σ a]` on the pre-Hilbert space of a state.

```
def preGNSAut : ω.toPositiveLinearMap.PreGNS →ₗ[ℂ] ω.toPositiveLinearMap.PreGNS
```

lemma ofPreGNS_preGNSAut [source]

```
lemma ofPreGNS_preGNSAut (x : ω.toPositiveLinearMap.PreGNS) :
    ω.toPositiveLinearMap.ofPreGNS (preGNSAut ω σ x) = σ (ω.toPositiveLinearMap.ofPreGNS x)
```

lemma inner_preGNSAut [source]

A symmetry preserving the state preserves the GNS inner product.

```
lemma inner_preGNSAut (hσ : ∀ a, ω (σ a) = ω a) (x y : ω.toPositiveLinearMap.PreGNS) :
    ⟪preGNSAut ω σ x, preGNSAut ω σ y⟫_ℂ = ⟪x, y⟫_ℂ
```

lemma norm_preGNSAut [source]

```
lemma norm_preGNSAut (hσ : ∀ a, ω (σ a) = ω a) (x : ω.toPositiveLinearMap.PreGNS) :
    ‖preGNSAut ω σ x‖ = ‖x‖
```

def preGNSAutCLM [source]

The symmetry as a bounded operator on the pre-Hilbert space.

```
def preGNSAutCLM (hσ : ∀ a, ω (σ a) = ω a) :
    ω.toPositiveLinearMap.PreGNS →L[ℂ] ω.toPositiveLinearMap.PreGNS
```

def gnsAut [source]

The unitary operator on the GNS space implementing a symmetry that preserves the state.

```
def gnsAut (hσ : ∀ a, ω (σ a) = ω a) : ω.GNS →L[ℂ] ω.GNS
```

lemma gnsAut_coe [source]

```
lemma gnsAut_coe (hσ : ∀ a, ω (σ a) = ω a) (x : ω.toPositiveLinearMap.PreGNS) :
    gnsAut hσ (x : ω.GNS) = ((preGNSAut ω σ x : ω.toPositiveLinearMap.PreGNS) : ω.GNS)
```

lemma inner_gnsAut [source]

The implementing operator preserves the inner product of the GNS space.

```
lemma inner_gnsAut (hσ : ∀ a, ω (σ a) = ω a) (x y : ω.GNS) :
    ⟪gnsAut hσ x, gnsAut hσ y⟫_ℂ = ⟪x, y⟫_ℂ
```

lemma gnsAut_mul [source]

```
lemma gnsAut_mul (g h : Multiplicative G) :
    gnsAut (hα (g * h)) = gnsAut (hα g) * gnsAut (hα h)
```

lemma gnsAut_one [source]

```
lemma gnsAut_one : gnsAut (hα 1) = 1
```

lemma star_gnsAut_mul_gnsAut [source]

```
lemma star_gnsAut_mul_gnsAut (g : Multiplicative G) :
    star (gnsAut (hα g)) * gnsAut (hα g) = 1
```

lemma star_gnsAut [source]

```
lemma star_gnsAut (g : Multiplicative G) : star (gnsAut (hα g)) = gnsAut (hα g⁻¹)
```

def gnsUnitary [source]

***Translations of an invariant state are unitarily implemented**: the unitary operator on the GNS space implementing the symmetry `α g`.*

```
def gnsUnitary (g : Multiplicative G) : unitary (ω.GNS →L[ℂ] ω.GNS)
```

def gnsUnitaryRep [source]

The unitary representation of a symmetry group on the GNS space of an invariant state.

```
def gnsUnitaryRep : Multiplicative G →* unitary (ω.GNS →L[ℂ] ω.GNS) where
```

lemma gnsUnitary_cyclicVector [source]

The implementing unitary fixes the cyclic vector.

```
lemma gnsUnitary_cyclicVector (g : Multiplicative G) :
    ((gnsUnitary hα g : unitary (ω.GNS →L[ℂ] ω.GNS)) : ω.GNS →L[ℂ] ω.GNS) ω.gnsCyclicVector =
      ω.gnsCyclicVector
```

lemma gnsUnitary_gnsRep_cyclicVector [source]

The implementing unitary carries `π(a) Ω` to `π(α_g a) Ω`.

```
lemma gnsUnitary_gnsRep_cyclicVector (g : Multiplicative G) (a : A) :
    ((gnsUnitary hα g : unitary (ω.GNS →L[ℂ] ω.GNS)) : ω.GNS →L[ℂ] ω.GNS)
        (ω.gnsRep a ω.gnsCyclicVector) = ω.gnsRep (α g a) ω.gnsCyclicVector
```

lemma gnsUnitary_comp_gnsRep [source]

The implementing unitaries intertwine the GNS representation with the symmetry: `U_g π(a) = π(α_g a) U_g`.

```
lemma gnsUnitary_comp_gnsRep (g : Multiplicative G) (a : A) :
    ((gnsUnitary hα g : unitary (ω.GNS →L[ℂ] ω.GNS)) : ω.GNS →L[ℂ] ω.GNS) * ω.gnsRep a =
      ω.gnsRep (α g a) * ((gnsUnitary hα g : unitary (ω.GNS →L[ℂ] ω.GNS)) : ω.GNS →L[ℂ] ω.GNS)
```

lemma inner_gnsRep_cyclicVector_gnsUnitary [source]

The matrix coefficients of the GNS vectors `π(a) Ω` are the correlation functions of the state: `⟪π(a) Ω, U_g π(a) Ω⟫ = ω(a⋆ α_g(a))`.

```
lemma inner_gnsRep_cyclicVector_gnsUnitary (g : Multiplicative G) (a : A) :
    ⟪ω.gnsRep a ω.gnsCyclicVector,
      ((gnsUnitary hα g : unitary (ω.GNS →L[ℂ] ω.GNS)) : ω.GNS →L[ℂ] ω.GNS)
        (ω.gnsRep a ω.gnsCyclicVector)⟫_ℂ = ω (star a * α g a)
```

2.527 PhyslibAlpha.ProbabilisticTheory.CStarAlgebra.GNSSpectral

PhyslibAlpha/ProbabilisticTheory/CStarAlgebra/GNSSpectral.lean on GitHub.

def gnsStructureFactor [source]

The momentum distribution of the excitation `π(a) Ω` created by the observable `a` from the invariant state: the distribution of the vector `π(a) Ω` over the characters of the symmetry group.

```
def gnsStructureFactor (a : A) : Measure (PontryaginDual (Multiplicative G))
```

instance IsFiniteMeasure (gnsStructureFactor hα a) [source]

```
instance (a : A) : IsFiniteMeasure (gnsStructureFactor hα a)
```

lemma integral_gnsStructureFactor [source]

***The Fourier transform of the momentum distribution is the correlation function**: `∫ χ(g) dμ_a(χ) = ω(a⋆ α_g(a))`.*

```
lemma integral_gnsStructureFactor (a : A) (g : Multiplicative G) :
    ∫ χ, ((χ g : Circle) : ℂ) ∂gnsStructureFactor hα a = ω (star a * α g a)
```

lemma gnsStructureFactor_univ [source]

The total momentum weight of the excitation `π(a) Ω` is `ω(a⋆ a)`.

```
lemma gnsStructureFactor_univ (a : A) :
    (gnsStructureFactor hα a).real Set.univ = (ω (star a * a)).re
```

lemma gnsCyclicVector_mem_sector [source]

The cyclic vector of an invariant state has momentum zero: its spectral measure is carried by the trivial character.

```
lemma gnsCyclicVector_mem_sector :
    ω.gnsCyclicVector ∈ ProbabilisticTheory.Representation.sector (toEnd (gnsUnitaryRep hα))
      (fun g => (((1 : PontryaginDual (Multiplicative G)) g : Circle) : ℂ))
```

lemma projection_one_gnsCyclicVector [source]

***The vacuum has momentum zero.** For a countable symmetry group the spectral projection of the trivial character fixes the cyclic vector.*

```
lemma projection_one_gnsCyclicVector [Countable G] :
    projection (gnsUnitaryRep hα) {(1 : PontryaginDual (Multiplicative G))} ω.gnsCyclicVector =
      ω.gnsCyclicVector
```

2.528 PhyslibAlpha.ProbabilisticTheory.CStarAlgebra.Uncertainty

PhyslibAlpha/ProbabilisticTheory/CStarAlgebra/Uncertainty.lean on GitHub.

def normalizer [source]

A matrix `T` with `T A T^H = 1` for a positive definite `A`: the inverse square roots of the eigenvalues of `A` times the adjoint of its eigenvector unitary.

```
noncomputable def PosDef.normalizer {A : Matrix n n ℂ} (hA : A.PosDef) : Matrix n n ℂ
```

lemma normalizer_mul_mul_conjTranspose [source]

```
lemma PosDef.normalizer_mul_mul_conjTranspose {A : Matrix n n ℂ} (hA : A.PosDef) :
    hA.normalizer * A * hA.normalizer^H = 1
```

lemma abs_eigenvalues_le_one [source]

The eigenvalues of a hermitian `K` with `1 - K` and `1 + K` positive semidefinite lie in `[-1, 1]`.

```
lemma IsHermitian.abs_eigenvalues_le_one {K : Matrix n n ℂ} (hK : K.IsHermitian)
    (hm : (1 - K).PosSemidef) (hp : (1 + K).PosSemidef) (i : n) : |hK.eigenvalues i| ≤ 1
```

lemma norm_det_le_re_det [source]

If `A` is positive definite, `B` hermitian and `A - B`, `A + B` positive semidefinite, then `‖det B‖ ≤ re (det A)`: conjugating by the normalizer of `A` puts `B` between `-1` and `1`.

```
lemma PosDef.norm_det_le_re_det {A B : Matrix n n ℂ} (hA : A.PosDef) (hB : B.IsHermitian)
    (h₁ : (A - B).PosSemidef) (h₂ : (A + B).PosSemidef) : ‖B.det‖ ≤ (A.det).re
```

lemma map_ofReal_add_smul_one [source]

```
lemma map_ofReal_add_smul_one (S : Matrix n n ℝ) (ε : ℝ) :
    (S + ε • 1).map ((↑) : ℝ → ℂ) = S.map ((↑) : ℝ → ℂ) + (ε : ℂ) • 1
```

lemma abs_det_le_det_of_posSemidef [source]

For real `S` and `W`, if `S - iW` and `S + iW` are positive semidefinite then `|det W| ≤ det S`.

```
lemma abs_det_le_det_of_posSemidef {S W : Matrix n n ℝ}
    (h₁ : (S.map ((↑) : ℝ → ℂ) - Complex.I • W.map ((↑) : ℝ → ℂ)).PosSemidef)
    (h₂ : (S.map ((↑) : ℝ → ℂ) + Complex.I • W.map ((↑) : ℝ → ℂ)).PosSemidef) :
    |W.det| ≤ S.det
```

lemma mul_self_eq_one [source]

A hermitian `K` with eigenvalues in `[-1, 1]` and `‖det K‖ = 1` squares to the identity.

```
lemma IsHermitian.mul_self_eq_one {K : Matrix n n ℂ} (hK : K.IsHermitian)
    (hle : ∀ i, |hK.eigenvalues i| ≤ 1) (h1 : ‖K.det‖ = 1) : K * K = 1
```

lemma inv_eq_conjTranspose_normalizer_mul [source]

The normalizer of a positive definite `A` is invertible, and `A⁻¹ = T^H T`.

```
lemma PosDef.inv_eq_conjTranspose_normalizer_mul {A : Matrix n n ℂ} (hA : A.PosDef) :
    A⁻¹ = hA.normalizer^H * hA.normalizer
```

lemma add_mul_inv_mul_sub_eq_zero [source]

***The equality case.** If `A` is positive definite, `B` hermitian, `A ∓ B` positive semidefinite and `‖det B‖ = re (det A)`, then `(A + B) A⁻¹ (A − B) = 0`: every eigenvalue of the normalized `B` is `±1`.*

```
lemma PosDef.add_mul_inv_mul_sub_eq_zero {A B : Matrix n n ℂ} (hA : A.PosDef)
    (hB : B.IsHermitian) (h₁ : (A - B).PosSemidef) (h₂ : (A + B).PosSemidef)
    (heq : ‖B.det‖ = (A.det).re) : (A + B) * A⁻¹ * (A - B) = 0
```

lemma card_le_finrank_ker_add_finrank_ker [source]

At equality the kernels of `A + B` and `A − B` together have dimension at least `n`: every vector is `A⁻¹ (A − B) x / 2 + A⁻¹ (A + B) x / 2`.

```
lemma PosDef.card_le_finrank_ker_add_finrank_ker {A B : Matrix n n ℂ} (hA : A.PosDef)
    (hB : B.IsHermitian) (h₁ : (A - B).PosSemidef) (h₂ : (A + B).PosSemidef)
    (heq : ‖B.det‖ = (A.det).re) :
    Fintype.card n ≤ Module.finrank ℂ (LinearMap.ker (A + B).mulVecLin) +
      Module.finrank ℂ (LinearMap.ker (A - B).mulVecLin)
```

lemma finrank_ker_transpose [source]

`G` and G^T have kernels of the same dimension.

```
lemma finrank_ker_transpose (G : Matrix n n ℂ) :
    Module.finrank ℂ (LinearMap.ker G^T.mulVecLin) =
      Module.finrank ℂ (LinearMap.ker G.mulVecLin)
```

lemma card_le_two_mul_finrank_ker [source]

***Equality in Robertson's determinant relation.** Let `G = S + iW` be positive semidefinite with G^T `= S − iW`, for real `S` and `W`. If `|det W| = det S ≠ 0`, the kernel of `G` has at least half the dimension.*

```
lemma card_le_two_mul_finrank_ker {G : Matrix n n ℂ} {S W : Matrix n n ℝ} (hG : G.PosSemidef)
    (hGS : G = S.map ((↑) : ℝ → ℂ) + Complex.I • W.map ((↑) : ℝ → ℂ))
    (hGT : G^T = S.map ((↑) : ℝ → ℂ) - Complex.I • W.map ((↑) : ℝ → ℂ)) (hW : W.det ≠ 0)
    (heq : |W.det| = S.det) :
    Fintype.card n ≤ 2 * Module.finrank ℂ (LinearMap.ker G.mulVecLin)
```

def covarianceMatrix [source]

The covariance matrix `Σ_jk = cov(a_j, a_k)` of a family of observables.

```
noncomputable def covarianceMatrix (ω : 𝒮[ℂ, A]) (a : ι → Observable A) : Matrix ι ι ℝ
```

def bracketMatrix [source]

The matrix `Ω_jk = ω⟦a_j, a_k⟧` of expectations of brackets of a family of observables.

```
noncomputable def bracketMatrix (ω : 𝒮[ℂ, A]) (a : ι → Observable A) : Matrix ι ι ℝ
```

def centeredGNSVector [source]

The GNS vectors of the centered observables.

```
noncomputable def centeredGNSVector (ω : 𝒮[ℂ, A]) (a : Observable A)
```

lemma gram_centeredGNSVector [source]

$\Sigma + i\Omega$ is the Gram matrix of the GNS vectors of the centered observables.

```
lemma gram_centeredGNSVector (ω : 𝒮[ℂ, A]) (a : ι → Observable A) :
    gram ℂ (fun j => centeredGNSVector ω (a j)) =
      (covarianceMatrix ω a).map ((↑) : ℝ → ℂ) +
        Complex.I • (bracketMatrix ω a).map ((↑) : ℝ → ℂ)
```

lemma transpose_gram_centeredGNSVector [source]

$\Sigma - i\Omega$ is the transpose of the Gram matrix.

```
lemma transpose_gram_centeredGNSVector (ω : 𝒮[ℂ, A]) (a : ι → Observable A) :
    (gram ℂ (fun j => centeredGNSVector ω (a j)))ᵀ =
      (covarianceMatrix ω a).map ((↑) : ℝ → ℂ) -
        Complex.I • (bracketMatrix ω a).map ((↑) : ℝ → ℂ)
```

lemma robertson_det [source]

***Robertson's uncertainty relation for several observables**: for every state and every finite family of observables, $|\det \Omega| \le \det \Sigma$.*

```
lemma robertson_det [Fintype ι] [DecidableEq ι] (ω : 𝒮[ℂ, A]) (a : ι → Observable A) :
    |(bracketMatrix ω a).det| ≤ (covarianceMatrix ω a).det
```

2.529 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.JointSpectral.Basic

PhyslibAlpha/ProbabilisticTheory/HilbertSpace/JointSpectral/Basic.lean on GitHub.

structure CommutingStarFamily [source]

A commuting family of bounded operators closed under adjoints.

```
structure CommutingStarFamily where
  carrier : Set (H →L[ℂ] H)
  commute : ∀ a ∈ carrier, ∀ b ∈ carrier, Commute a b
  star_mem : ∀ a ∈ carrier, star a ∈ carrier
```

lemma isStarNormal_of_mem [source]

```
lemma isStarNormal_of_mem {a : H →L[ℂ] H} (ha : a ∈ F.carrier) : IsStarNormal a
```

instance isMulCommutative_adjoin [source]

```
instance isMulCommutative_adjoin : IsMulCommutative (StarAlgebra.adjoin ℂ F.carrier)
```

def algebra [source]

The closed ⋆-subalgebra of the bounded operators generated by a commuting ⋆-family.

```
def algebra : StarSubalgebra ℂ (H →L[ℂ] H)
```

lemma carrier_subset_algebra [source]

```
lemma carrier_subset_algebra : F.carrier ⊆ F.algebra
```

instance isClosed_algebra [source]

```
instance isClosed_algebra : IsClosed (F.algebra : Set (H →L[ℂ] H))
```

instance isMulCommutative_algebra [source]

```
instance isMulCommutative_algebra : IsMulCommutative F.algebra
```

instance commCStarAlgebraAlgebra [source]

The algebra generated by a commuting ⋆-family is a commutative C⋆-algebra.

```
instance commCStarAlgebraAlgebra : CommCStarAlgebra F.algebra where
```

lemma commute_of_forall_mem_carrier [source]

```
lemma commute_of_forall_mem_carrier {T : H →L[ℂ] H} (hT : ∀ a ∈ F.carrier, Commute a T)
    {b : H →L[ℂ] H} (hb : b ∈ F.algebra) : Commute b T
```

instance MeasurableSpace (characterSpace ℂ F.algebra) [source]

```
instance : MeasurableSpace (characterSpace ℂ F.algebra)
```

instance BorelSpace (characterSpace ℂ F.algebra) [source]

```
instance : BorelSpace (characterSpace ℂ F.algebra)
```

def starHom [source]

The inclusion of the algebra as a representation of the continuous functions on its character space, by Gelfand duality.

```
def starHom : C(characterSpace ℂ F.algebra, ℂ) →⋆ₐ[ℂ] (H →L[ℂ] H)
```

lemma starHom_gelfandStarTransform [source]

```
lemma starHom_gelfandStarTransform (a : F.algebra) :
    F.starHom (gelfandStarTransform F.algebra a) = a
```

lemma starHom_mem_algebra [source]

```
lemma starHom_mem_algebra (f : C(characterSpace ℂ F.algebra, ℂ)) : F.starHom f ∈ F.algebra
```

def spectralMeasure [source]

The joint spectral measure of a commuting ⋆-family, on the character space of the C⋆-algebra it generates.

```
def spectralMeasure : WOTSpectralMeasure (characterSpace ℂ F.algebra) H
```

lemma boundedIntegral_gelfandStarTransform [source]

Every operator of the algebra generated by the family is the integral of its Gelfand transform against the joint spectral measure.

```
lemma boundedIntegral_gelfandStarTransform [Nonempty (characterSpace ℂ F.algebra)]
    (a : F.algebra) :
    ContinuousLinearMapWOT.toCLM
      (WOTSpectralMeasure.boundedIntegral F.spectralMeasure (gelfandStarTransform F.algebra a)
        (gelfandStarTransform F.algebra a).continuous.measurable
        (boundedIntegral_bound_of_continuous _)) = a
```

lemma spectralMeasure_commute [source]

The joint spectral measure commutes with every bounded operator commuting with the family.

```
lemma spectralMeasure_commute {T : H →L[ℂ] H} (hT : ∀ a ∈ F.carrier, Commute a T) (S : Set _) :
    F.spectralMeasure S * ContinuousLinearMapWOT.ofCLM T =
      ContinuousLinearMapWOT.ofCLM T * F.spectralMeasure S
```

2.530 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.OrthonormalMeasurement

PhyslibAlpha/ProbabilisticTheory/HilbertSpace/OrthonormalMeasurement.lean on GitHub.

lemma inner_rankOne_self_apply [source]

⟪φ, |v⟩⟨v| φ⟫ = ‖⟪v, φ⟫‖ ^ 2.

```
lemma inner_rankOne_self_apply (v φ : H) :
    ⟪φ, rankOne ℂ v v φ⟫_ℂ = (‖⟪v, φ⟫_ℂ‖ ^ 2 : ℝ)
```

lemma inner_sum_rankOne_self_apply [source]

⟪φ, ∑ y, |b y⟩⟨b y| φ⟫ = ∑ y, ‖⟪b y, φ⟫‖ ^ 2.

```
lemma inner_sum_rankOne_self_apply (b : X → H) (φ : H) :
    ⟪φ, (∑ y, rankOne ℂ (b y) (b y)) φ⟫_ℂ = (∑ y, ‖⟪b y, φ⟫_ℂ‖ ^ 2 : ℝ)
```

lemma isPositive_one_sub_sum_rankOne [source]

By Bessel's inequality, `1 - ∑ y, |b y⟩⟨b y|` is positive for an orthonormal family `b`.

```
lemma isPositive_one_sub_sum_rankOne {b : X → H} (hb : Orthonormal ℂ b) :
    (1 - ∑ y, rankOne ℂ (b y) (b y)).IsPositive
```

def rankOneObservable [source]

The rank-one observable `|v⟩⟨v|`.

```
noncomputable def rankOneObservable (v : H) : Observable (H →L[ℂ] H)
```

lemma coe_rankOneObservable [source]

```
@[simp]
lemma coe_rankOneObservable (v : H) :
    (rankOneObservable v : H →L[ℂ] H) = rankOne ℂ v v
```

lemma rankOneObservable_nonneg [source]

The rank-one observable `|v⟩⟨v|` is positive.

```
lemma rankOneObservable_nonneg (v : H) : 0 ≤ rankOneObservable v
```

def orthonormalObservable [source]

The observable of the outcome `x` in the measurement in the family `b`: `|b x⟩⟨b x|`, plus the remainder `1 - ∑ y, |b y⟩⟨b y|` when `x = x₀`.

```
noncomputable def orthonormalObservable (b : X → H) (x₀ x : X) : Observable (H →L[ℂ] H)
```

lemma coe_orthonormalObservable [source]

The operator underlying the observable of the outcome `x`.

```
lemma coe_orthonormalObservable (b : X → H) (x₀ x : X) :
    (orthonormalObservable b x₀ x : H →L[ℂ] H) =
      rankOne ℂ (b x) (b x) + if x = x₀ then 1 - ∑ y, rankOne ℂ (b y) (b y) else 0
```

lemma sum_orthonormalObservable [source]

The outcome observables add up to `1`.

```
lemma sum_orthonormalObservable (b : X → H) (x₀ : X) :
    ∑ x, orthonormalObservable b x₀ x = 1
```

lemma orthonormalObservable_nonneg [source]

The outcome observables of an orthonormal family are positive.

```
lemma orthonormalObservable_nonneg {b : X → H} (hb : Orthonormal ℂ b) (x₀ x : X) :
    0 ≤ orthonormalObservable b x₀ x
```

def orthonormalEffect [source]

The outcome effects of the measurement in an orthonormal family.

```
noncomputable def orthonormalEffect {b : X → H} (hb : Orthonormal ℂ b) (x₀ x : X) :
    Effect (Observable (H →L[ℂ] H))
```

lemma coe_orthonormalEffect [source]

```
@[simp]
lemma coe_orthonormalEffect {b : X → H} (hb : Orthonormal ℂ b) (x₀ x : X) :
    (orthonormalEffect hb x₀ x : Observable (H →L[ℂ] H)) = orthonormalObservable b x₀ x
```

def ofOrthonormal [source]

The measurement in the orthonormal family `b`: the outcome `x` has effect `|b x)(b x|`, and the outcome `x₀` also registers the orthogonal complement of the span of the family.

```
noncomputable def ofOrthonormal {b : X → H} (hb : Orthonormal ℂ b) (x₀ : X) :
    Measurement X (Observable (H →L[ℂ] H))
```

lemma coe_ofOrthonormal_singleton [source]

The outcome `x` of the measurement in an orthonormal family has the observable `orthonormalObservable b x₀ x`.

```
lemma coe_ofOrthonormal_singleton {b : X → H} (hb : Orthonormal ℂ b) (x₀ x : X)
    (hx : MeasurableSet {x}) :
    (ofOrthonormal hb x₀ {x} hx : Observable (H →L[ℂ] H)) = orthonormalObservable b x₀ x
```

lemma expectation_ofOrthonormal_singleton [source]

The Born rule for the measurement in an orthonormal family: on a unit vector `φ`, the outcome `x` has probability `‖⟪b x, φ⟫‖ ^ 2`, plus `1 - ∑ y, ‖⟪b y, φ⟫‖ ^ 2` when `x = x₀`.

```
lemma expectation_ofOrthonormal_singleton [MeasurableSingletonClass X] {b : X → H}
    (hb : Orthonormal ℂ b) (x₀ : X) {φ : H} (hφ : ‖φ‖ = 1) (x : X) :
    (ofVec hφ)(ofOrthonormal hb x₀ {x} (measurableSet_singleton x)) =
      ‖⟪b x, φ⟫_ℂ‖ ^ 2 + if x = x₀ then 1 - ∑ y, ‖⟪b y, φ⟫_ℂ‖ ^ 2 else 0
```

2.531 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.State.Vector

PhyslibAlpha/ProbabilisticTheory/HilbertSpace/State/Vector.lean on GitHub.

lemma ofVec_smul [source]

Vector states do not see a global phase: `ofVec (c • ψ) = ofVec ψ` for `‖c‖ = 1`.

```
lemma UnitalPositiveLinearMap.ofVec_smul {ψ : H} (h : ‖ψ‖ = 1) {c : 𝕜} (hc : ‖c‖ = 1)
    (h' : ‖c • ψ‖ = 1) :
    UnitalPositiveLinearMap.ofVec (𝕜 := 𝕜) h' = UnitalPositiveLinearMap.ofVec (𝕜 := 𝕜) h
```

lemma norm_inner_lt_one_of_ofVec_ne [source]

Unit vectors with different vector states are not parallel: ‖⟪ψ, φ⟫‖ < 1.

```
lemma UnitalPositiveLinearMap.norm_inner_lt_one_of_ofVec_ne {ψ φ : H} (hψ : ‖ψ‖ = 1)
    (hφ : ‖φ‖ = 1) (h : ofVec (𝕜 := 𝕜) hψ ≠ ofVec (𝕜 := 𝕜) hφ) : ‖⟪ψ, φ⟫_𝕜‖ < 1
```

2.532 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.State.VectorUncertainty

PhyslibAlpha/ProbabilisticTheory/HilbertSpace/State/VectorUncertainty.lean on GitHub.

def fluctuation [source]

The fluctuation vector `a ψ - ⟨a⟩ ψ` of an observable in a vector state.

```
noncomputable def fluctuation (h : ‖ψ‖ = 1) (a : Observable (H →L[ℂ] H)) : H
```

lemma gram_fluctuation [source]

In a vector state, `Σ + iΩ` is the Gram matrix of the fluctuation vectors.

```
lemma gram_fluctuation (h : ‖ψ‖ = 1) (a : ι → Observable (H →L[ℂ] H)) :
    gram ℂ (fun j => fluctuation h (a j)) =
      (covarianceMatrix (ofVec h) a).map ((↑) : ℝ → ℂ) +
        Complex.I • (bracketMatrix (ofVec h) a).map ((↑) : ℝ → ℂ)
```

lemma transpose_gram_fluctuation [source]

In a vector state, `Σ - iΩ` is the transpose of the Gram matrix of the fluctuation vectors.

```
lemma transpose_gram_fluctuation (h : ‖ψ‖ = 1) (a : ι → Observable (H →L[ℂ] H)) :
    (gram ℂ (fun j => fluctuation h (a j)))ᵀ =
      (covarianceMatrix (ofVec h) a).map ((↑) : ℝ → ℂ) -
        Complex.I • (bracketMatrix (ofVec h) a).map ((↑) : ℝ → ℂ)
```

lemma gram_mulVec_eq_zero_iff [source]

The kernel of a Gram matrix is the space of linear relations among the vectors.

```
lemma gram_mulVec_eq_zero_iff [Fintype ι] (v : ι → H) (c : ι → ℂ) :
    gram ℂ v *ᵥ c = 0 ↔ ∑ j, c j • v j = 0
```

lemma card_le_two_mul_finrank_ker_gram_fluctuation [source]

***Equality in Robertson's relation for several observables.** In a vector state with `|det Ω| = det Σ ≠ 0`, the fluctuation vectors of the family satisfy linear relations `∑ j, c j • (a_j ψ - ⟨a_j⟩ ψ) = 0` forming a space of at least half the size of the family.*

```
lemma card_le_two_mul_finrank_ker_gram_fluctuation [Fintype ι] [DecidableEq ι] (h : ‖ψ‖ = 1)
    (a : ι → Observable (H →L[ℂ] H)) (hΩ : (bracketMatrix (ofVec h) a).det ≠ 0)
    (heq : |(bracketMatrix (ofVec h) a).det| = (covarianceMatrix (ofVec h) a).det) :
    Fintype.card ι ≤
      2 * Module.finrank ℂ (LinearMap.ker (gram ℂ fun j => fluctuation h (a j)).mulVecLin)
```

2.533 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.Basic

PhyslibAlpha/ProbabilisticTheory/HilbertSpace/Unbounded/Basic.lean on GitHub.

instance HasUniqueDiv (H →WOT[ℂ] H) [source]

```
@[nolint unusedArguments]
instance (H : Type*) [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteSpace H] :
    HasUniqueDiv (H →WOT[ℂ] H) where
```

2.534 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.BoundedIntegralCommute

PhyslibAlpha/ProbabilisticTheory/HilbertSpace/Unbounded/BoundedIntegralCommute.lean on GitHub.

lemma scalarMeasure_inner_projection' [source]

```
lemma scalarMeasure_inner_projection' (x y : H) (S : Set α) :
    ⟪y, μS S x⟫_ℂ = ⟪μS S y, μS S x⟫_ℂ
```

lemma scalarMeasure_variation_le_diagonal_add' [source]

```
lemma scalarMeasure_variation_le_diagonal_add' (x y : H) :
    (μS.scalarMeasure x y).variation ≤ μS.diagonalMeasure x + μS.diagonalMeasure y
```

lemma scalarMeasure_isFiniteVariation' [source]

The matrix-coefficient measures of a weak spectral measure have finite variation.

```
lemma scalarMeasure_isFiniteVariation' (x y : H) :
    IsFiniteMeasure (μS.scalarMeasure x y).variation
```

lemma boundedIntegral_commute [source]

An operator commuting with every spectral projection commutes with every bounded integral against the spectral measure.

```
lemma boundedIntegral_commute [Nonempty α] {T : H →L[ℂ] H}
    (hT : ∀ S, μS S * ContinuousLinearMapWOT.ofCLM T = ContinuousLinearMapWOT.ofCLM T * μS S)
    {f : α → ℂ} (hf : Measurable f) (hbf : ∃ C, ∀ z, ‖f z‖ ≤ C) :
    ContinuousLinearMapWOT.toCLM (boundedIntegral μS f hf hbf) * T =
      T * ContinuousLinearMapWOT.toCLM (boundedIntegral μS f hf hbf)
```

2.535 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.ContinuousCalculus

PhyslibAlpha/ProbabilisticTheory/HilbertSpace/Unbounded/ContinuousCalculus.lean on GitHub.

def integralCLM [source]

The bounded integral of a continuous function, as a bounded operator.

```
def integralCLM (f : C(X, ℂ)) : H →L[ℂ] H
```

lemma bound_of_continuous [source]

```
lemma bound_of_continuous (f : C(X, ℂ)) : ∃ C, ∀ z, ‖f z‖ ≤ C
```

lemma integralCLM_eq_of_forall_eq [source]

The integral of a measurable bounded function that agrees with a continuous function.

```
lemma integralCLM_eq_of_forall_eq (f : C(X, ℂ)) {F : X → ℂ} (hF : Measurable F)
    (hbF : ∃ C, ∀ z, ‖F z‖ ≤ C) (h : ∀ z, f z = F z) :
    P.integralCLM f = ContinuousLinearMapWOT.toCLM (P.boundedIntegral F hF hbF)
```

lemma integralCLM_add [source]

```
lemma integralCLM_add (f g : C(X, ℂ)) :
    P.integralCLM (f + g) = P.integralCLM f + P.integralCLM g
```

lemma integralCLM_mul [source]

```
lemma integralCLM_mul (f g : C(X, ℂ)) :
    P.integralCLM (f * g) = P.integralCLM f * P.integralCLM g
```

lemma integralCLM_const [source]

```
lemma integralCLM_const (c : ℂ) : P.integralCLM (ContinuousMap.const X c) = c • 1
```

lemma integralCLM_star [source]

```
lemma integralCLM_star (f : C(X, ℂ)) : P.integralCLM (star f) = star (P.integralCLM f)
```

def continuousCalculus [source]

***The continuous functional calculus of a spectral measure**: `f ↦ ∫ f dP` is a unital ⋆-homomorphism from the continuous functions to the bounded operators.*

```
def continuousCalculus : C(X, ℂ) →⋆ₐ[ℂ] (H →L[ℂ] H) where
```

2.536 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.StarHomSpec.Commutant

PhyslibAlpha/ProbabilisticTheory/HilbertSpace/Unbounded/StarHomSpec/Commutant.lean on GitHub.

def starHomCommutant [source]

The commutant of the range of `Φ`, a ⋆-subalgebra of the bounded operators.

```
def starHomCommutant : StarSubalgebra ℂ (H →L[ℂ] H)
```

lemma mem_starHomCommutant_iff [source]

```
lemma mem_starHomCommutant_iff {T : H →L[ℂ] H} :
    T ∈ starHomCommutant Φ ↔ ∀ f : C(X, ℂ), Commute (Φ f) T
```

instance starHomCommutant_isClosed [source]

```
instance starHomCommutant_isClosed : IsClosed (starHomCommutant Φ : Set (H →L[ℂ] H))
```

lemma starHomSpectralOperator_commute_of_commute [source]

***The spectral operators commute with the commutant**: every bounded operator commuting with the range of `Φ` commutes with each spectral operator of `Φ`. A bounded operator commuting with the range is a combination of four unitary operators in the C⋆-algebra of such operators, and the spectral operators commute with unitaries commuting with the range.*

```
lemma starHomSpectralOperator_commute_of_commute {T : H →L[ℂ] H}
    (hT : ∀ f : C(X, ℂ), Commute (Φ f) T) (S : Set X) :
    starHomSpectralOperator Φ S * T = T * starHomSpectralOperator Φ S
```

lemma starHomSpectralMeasure_commute_of_commute [source]

***The spectral measure commutes with the commutant**: every bounded operator commuting with the range of `Φ` commutes with each spectral projection of `Φ`.*

```
lemma starHomSpectralMeasure_commute_of_commute {T : H →L[ℂ] H}
    (hT : ∀ f : C(X, ℂ), Commute (Φ f) T) (S : Set X) :
    starHomSpectralMeasure Φ S * ContinuousLinearMapWOT.ofCLM T =
      ContinuousLinearMapWOT.ofCLM T * starHomSpectralMeasure Φ S
```

2.537 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.StarHomSpec.Functional

PhyslibAlpha/ProbabilisticTheory/HilbertSpace/Unbounded/StarHomSpec/Functional.lean on GitHub.

def reCompact [source]

The real part of a continuous complex function on a compact space, as a compactly supported continuous real function.

```
def reCompact (f : C(X, ℂ)) : CompactlySupportedContinuousMap X ℝ
```

def imCompact [source]

The imaginary part of a continuous complex function on a compact space, as a compactly supported continuous real function.

```
def imCompact (f : C(X, ℂ)) : CompactlySupportedContinuousMap X ℝ
```

lemma realToComplex_reCompact_add_I_smul [source]

```
lemma realToComplex_reCompact_add_I_smul (f : C(X, ℂ)) :
    realToComplex (reCompact f) + Complex.I • realToComplex (imCompact f) = f
```

lemma starHomSpectralMeasure_diagonalMeasure [source]

The diagonal measure `S ↦ ‖E(S) x‖²` of the spectral measure of `Φ` is the Riesz measure of the vector state of `x`.

```
lemma starHomSpectralMeasure_diagonalMeasure (x : H) :
    (starHomSpectralMeasure Φ).diagonalMeasure x = starHomScalarMeasure Φ x
```

lemma integral_starHomScalarMeasure_real [source]

The integral of a real continuous function against the Riesz measure of the vector state of `x` is the expectation of its operator.

```
lemma integral_starHomScalarMeasure_real (u : CompactlySupportedContinuousMap X ℝ) (x : H) :
    ∫ z, (u z : ℂ) ∂starHomScalarMeasure Φ x = ⟪x, starHomRealOperator Φ u x⟫_ℂ
```

lemma integral_starHomScalarMeasure [source]

The vector state of `x` in the representation `Φ` is integration against its Riesz measure: `⟪x, Φ f x⟫ = ∫ f dμ_x` for every continuous `f`.

```
lemma integral_starHomScalarMeasure (f : C(X, ℂ)) (x : H) :
    ∫ z, f z ∂starHomScalarMeasure Φ x = ⟪x, Φ f x⟫_ℂ
```

lemma boundedIntegral_bound_of_continuous [source]

```
lemma boundedIntegral_bound_of_continuous (f : C(X, ℂ)) : ∃ C, ∀ z, ‖f z‖ ≤ C
```

lemma starHomSpectralMeasure_boundedIntegral_real [source]

The bounded integral of a real continuous function against the spectral measure of `Φ` is `Φ` of that function.

```
lemma starHomSpectralMeasure_boundedIntegral_real (u : CompactlySupportedContinuousMap X ℝ) :
    ContinuousLinearMapWOT.toCLM
      (WOTSpectralMeasure.boundedIntegral (starHomSpectralMeasure Φ)
        (fun z => (u z : ℂ)) (by fun_prop) ⟨‖u.toContinuousMap‖, fun z => by
          simpa using u.toContinuousMap.norm_coe_le_norm z⟩) = starHomRealOperator Φ u
```

lemma starHomSpectralMeasure_boundedIntegral [source]

***The spectral measure of `Φ` reconstructs `Φ`**: the bounded integral of a continuous function `f` against `starHomSpectralMeasure Φ` is `Φ f`.*

```
lemma starHomSpectralMeasure_boundedIntegral (f : C(X, ℂ)) :
    ContinuousLinearMapWOT.toCLM
      (WOTSpectralMeasure.boundedIntegral (starHomSpectralMeasure Φ) f f.continuous.measurable
        (boundedIntegral_bound_of_continuous f)) = Φ f
```

lemma starHomSpectralMeasure_boundedIntegral_map [source]

***Change of spectral variable**: integrating a bounded measurable function `g` against the pushforward of the spectral measure of `Φ` along a measurable map `f` is integrating `g ∘ f` against the spectral measure itself.*

```
lemma starHomSpectralMeasure_boundedIntegral_map {β : Type*} [MeasurableSpace β] [Nonempty β]
    {f : X → β} (hf : Measurable f) {g : β → ℂ} (hg : Measurable g)
    (hbg : ∃ C, ∀ b, ‖g b‖ ≤ C) :
```

```
    WOTSpectralMeasure.boundedIntegral ((starHomSpectralMeasure Φ).map f hf) g hg hbg =
      WOTSpectralMeasure.boundedIntegral (starHomSpectralMeasure Φ) (g ∘ f) (hg.comp hf)
        (by obtain ⟨C, hC⟩ := hbg; exact ⟨C, fun z => hC (f z)⟩)
```

2.538 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.StarHomSpec.SesquilinearForm

PhyslibAlpha/ProbabilisticTheory/HilbertSpace/Unbounded/StarHomSpec/SesquilinearForm.lean on GitHub.

def realToComplex [source]

Regard a real-valued continuous function on the spectrum of a normal operator as a complex valued one. Keeping this map explicit prevents the real/complex scalar changes in the Riesz construction from being hidden in coercions.

```
noncomputable def realToComplex
    (f : CompactlySupportedContinuousMap (X) ℝ) :
    C(X, ℂ)
```

lemma realToComplex_apply [source]

```
@[simp]
lemma realToComplex_apply
    (f : CompactlySupportedContinuousMap (X) ℝ) (z : X) :
    realToComplex f z = f z
```

lemma realToComplex_add [source]

```
lemma realToComplex_add
    (f g : CompactlySupportedContinuousMap (X) ℝ) :
    realToComplex (f + g) =
      realToComplex f + realToComplex g
```

lemma realToComplex_smul [source]

```
lemma realToComplex_smul (r : ℝ)
    (f : CompactlySupportedContinuousMap (X) ℝ) :
    realToComplex (r • f) =
      (r : ℂ) • realToComplex f
```

def starHomRealOperator [source]

The operator obtained by applying the complex continuous functional calculus to a real test function.

```
noncomputable def starHomRealOperator
    (f : CompactlySupportedContinuousMap (X) ℝ) : H →L[ℂ] H
```

lemma starHomRealOperator_isSelfAdjoint [source]

```
lemma starHomRealOperator_isSelfAdjoint
    (f : CompactlySupportedContinuousMap (X) ℝ) :
    IsSelfAdjoint (starHomRealOperator Φ f)
```

lemma starHomRealOperator_nonneg [source]

```
lemma starHomRealOperator_nonneg
    (f : CompactlySupportedContinuousMap (X) ℝ)
    (hf : ∀ z, 0 ≤ f z) :
    ContinuousLinearMap.IsPositive (starHomRealOperator Φ f)
```

def starHomScalarFunctional [source]

The vector state of the continuous functional calculus, restricted to real test functions. The positivity proof is the square-root argument above; this is the exact input required by Riesz–Markov.

```
noncomputable def starHomScalarFunctional (x : H) :
    CompactPositiveFunctional (X := X) where
```

lemma starHomScalarFunctional_apply [source]

```
lemma starHomScalarFunctional_apply (x : H)
    (f : CompactlySupportedContinuousMap (X) ℝ) :
    (starHomScalarFunctional Φ x).functional f =
      RCLike.re ⟪x, starHomRealOperator Φ f x⟫_ℂ
```

def starHomScalarMeasure [source]

The scalar spectral measure supplied by Riesz–Markov for the vector state at `x`.

```
noncomputable def starHomScalarMeasure (x : H) : Measure (X)
```

instance starHomScalarMeasure_regular [source]

```
instance starHomScalarMeasure_regular (x : H) : (starHomScalarMeasure Φ x).Regular
```

instance starHomScalarMeasure_isFinite [source]

```
instance starHomScalarMeasure_isFinite (x : H) : IsFiniteMeasure (starHomScalarMeasure Φ x)
```

lemma starHomScalarMeasure_integral [source]

```
lemma starHomScalarMeasure_integral (x : H)
    (f : CompactlySupportedContinuousMap (X) ℝ) :
    ∫ z, f z ∂starHomScalarMeasure Φ x =
      RCLike.re ⟪x, starHomRealOperator Φ f x⟫_ℂ
```

lemma starHomScalarMeasure_integral_complex [source]

```
lemma starHomScalarMeasure_integral_complex (x : H)
    (f : CompactlySupportedContinuousMap (X) ℝ) :
    ((∫ z, f z ∂starHomScalarMeasure Φ x : ℝ) : ℂ) =
      ⟪starHomRealOperator Φ f x, x⟫_ℂ
```

def polarizedStarHomRealIntegral [source]

The integral of a real function against the polarized measure.

```
noncomputable def polarizedStarHomRealIntegral
    (f : CompactlySupportedContinuousMap (X) ℝ) (x y : H) : ℂ
```

lemma polarizedStarHomRealIntegral_eq_inner [source]

```
lemma polarizedStarHomRealIntegral_eq_inner
    (f : CompactlySupportedContinuousMap (X) ℝ) (x y : H) :
    polarizedStarHomRealIntegral Φ f x y =
      ⟪y, starHomRealOperator Φ f x⟫_ℂ
```

lemma starHomScalarMeasure_parallelogram [source]

```
lemma starHomScalarMeasure_parallelogram (x y : H) :
    starHomScalarMeasure Φ (x + y) + starHomScalarMeasure Φ (x - y) =
      (starHomScalarMeasure Φ x + starHomScalarMeasure Φ x) +
        (starHomScalarMeasure Φ y + starHomScalarMeasure Φ y)
```

lemma starHomScalarMeasure_neg [source]

```
lemma starHomScalarMeasure_neg (x : H) :
    starHomScalarMeasure Φ (-x) = starHomScalarMeasure Φ x
```

lemma starHomScalarMeasure_I_smul [source]

```
lemma starHomScalarMeasure_I_smul (x : H) :
    starHomScalarMeasure Φ (Complex.I • x) = starHomScalarMeasure Φ x
```

def polarizedStarHomScalarMeasure [source]

The complex measure $\langle\langle y, E(\cdot)\, x\rangle\rangle$, obtained by polarizing the four measures $\langle\langle v, E(\cdot)\, v\rangle\rangle$.

```
noncomputable def polarizedStarHomScalarMeasure (x y : H) : ComplexMeasure (X)
```

lemma polarizedStarHomScalarMeasure_isFiniteMeasure [source]

```
lemma polarizedStarHomScalarMeasure_isFiniteMeasure (x y : H) :
    IsFiniteMeasure
      (polarizedStarHomScalarMeasure Φ x y).variation
```

lemma polarizedStarHomScalarMeasure_complexIntegral [source]

```
lemma polarizedStarHomScalarMeasure_complexIntegral
    (f : CompactlySupportedContinuousMap (X) ℝ) (x y : H) :
    ∫ᵛ z, (f z : ℂ) ∂[ContinuousLinearMap.lsmul ℝ ℂ (E := ℂ);
      polarizedStarHomScalarMeasure Φ x y] =
      polarizedStarHomRealIntegral Φ f x y
```

lemma polarizedStarHomScalarMeasure_complexIntegral_eq_inner [source]

```
lemma polarizedStarHomScalarMeasure_complexIntegral_eq_inner
    (f : CompactlySupportedContinuousMap (X) ℝ) (x y : H) :
    ∫ᵛ z, (f z : ℂ) ∂[ContinuousLinearMap.lsmul ℝ ℂ (E := ℂ);
      polarizedStarHomScalarMeasure Φ x y] =
      ⟪y, starHomRealOperator Φ f x⟫_ℂ
```

lemma polarizedStarHomScalarMeasure_apply [source]

```
lemma polarizedStarHomScalarMeasure_apply {x y : H}
    {S : Set (X)} (hS : MeasurableSet S) :
    polarizedStarHomScalarMeasure Φ x y S =
      ((1 / 4 : ℝ) *
          ((starHomScalarMeasure Φ (x + y)).real S -
            (starHomScalarMeasure Φ (x - y)).real S) : ℂ) +
        Complex.I * ((1 / 4 : ℝ) *
          ((starHomScalarMeasure Φ (x + Complex.I • y)).real S -
            (starHomScalarMeasure Φ (x - Complex.I • y)).real S) : ℂ)
```

lemma starHomScalarMeasure_real_parallelogram [source]

The real-valued version of the parallelogram law, at a fixed (not necessarily measurable) set `S`. This is the algebraic engine behind the sesquilinearity proofs below.

```
lemma starHomScalarMeasure_real_parallelogram (a b : H) (S : Set (X)) :
    (starHomScalarMeasure Φ (a + b)).real S + (starHomScalarMeasure Φ (a - b)).real S =
      2 * (starHomScalarMeasure Φ a).real S + 2 * (starHomScalarMeasure Φ b).real S
```

lemma starHomScalarMeasure_real_neg [source]

```
lemma starHomScalarMeasure_real_neg (a : H) (S : Set (X)) :
    (starHomScalarMeasure Φ (-a)).real S = (starHomScalarMeasure Φ a).real S
```

lemma starHomScalarMeasure_real_I_smul [source]

```
lemma starHomScalarMeasure_real_I_smul (a : H) (S : Set (X)) :
    (starHomScalarMeasure Φ (Complex.I • a)).real S = (starHomScalarMeasure Φ a).real S
```

lemma starHomScalarMeasure_real_nonneg [source]

```
lemma starHomScalarMeasure_real_nonneg (a : H) (S : Set (X)) :
    0 ≤ (starHomScalarMeasure Φ a).real S
```

lemma starHomScalarMeasure_real_univ [source]

The Riesz measure for the vector state at `x` has total mass `‖x‖ ^ 2`: the vector-state counterpart of `Φ 1 = 1`.

```
lemma starHomScalarMeasure_real_univ (x : H) :
    (starHomScalarMeasure Φ x).real Set.univ = ‖x‖ ^ 2
```

lemma starHomScalarMeasure_real_le_univ [source]

```
lemma starHomScalarMeasure_real_le_univ (a : H) (S : Set (X)) :
    (starHomScalarMeasure Φ a).real S ≤ ‖a‖ ^ 2
```

lemma starHomScalarMeasure_real_polar_symm [source]

The real polarization `Q(a + b) - Q(a - b)` of the diagonal measures is symmetric in `a` and `b`.

```
lemma starHomScalarMeasure_real_polar_symm (a b : H) (S : Set (X)) :
    (starHomScalarMeasure Φ (a + b)).real S - (starHomScalarMeasure Φ (a - b)).real S =
      (starHomScalarMeasure Φ (b + a)).real S - (starHomScalarMeasure Φ (b - a)).real S
```

lemma starHomScalarMeasure_real_polar_add_left [source]

First-argument additivity of the real polarization `Q(a+b) - Q(a-b)`, established by pure algebra from the parallelogram law (`starHomScalarMeasure_real_parallelogram`); no continuity is needed for this step, matching the classical Jordan–von Neumann polarization argument (`Mathlib.Analysis.InnerProductSpace.OfNorm`).

```
lemma starHomScalarMeasure_real_polar_add_left (x y z : H) (S : Set (X)) :
    (starHomScalarMeasure Φ (x + y + z)).real S - (starHomScalarMeasure Φ (x + y - z)).real S =
      ((starHomScalarMeasure Φ (x + z)).real S - (starHomScalarMeasure Φ (x - z)).real S) +
        ((starHomScalarMeasure Φ (y + z)).real S - (starHomScalarMeasure Φ (y - z)).real S)
```

lemma starHomScalarMeasure_real_polar_add_right [source]

Second-argument additivity of the real polarization, obtained from first-argument additivity (`starHomScalarMeasure_real_polar_add_left`) via the symmetry `starHomScalarMeasure_real_polar`

```
lemma starHomScalarMeasure_real_polar_add_right (a b c : H) (S : Set (X)) :
    (starHomScalarMeasure Φ (a + (b + c))).real S -
        (starHomScalarMeasure Φ (a - (b + c))).real S =
      ((starHomScalarMeasure Φ (a + b)).real S - (starHomScalarMeasure Φ (a - b)).real S) +
        ((starHomScalarMeasure Φ (a + c)).real S - (starHomScalarMeasure Φ (a - c)).real S)
```

lemma polarizedStarHomScalarMeasure_add_right [source]

Additivity of the polarized measure in its second (right) vector argument, for a fixed measurable set `S`. This is the sesquilinearity step promised in the module docstring: it follows from the parallelogram law by pure algebra, with no continuity or Riesz-representation machinery required.

```
lemma polarizedStarHomScalarMeasure_add_right (x y₁ y₂ : H) {S : Set (X)}
    (hS : MeasurableSet S) :
    polarizedStarHomScalarMeasure Φ x (y₁ + y₂) S =
      polarizedStarHomScalarMeasure Φ x y₁ S +
        polarizedStarHomScalarMeasure Φ x y₂ S
```

lemma polarizedStarHomScalarMeasure_add_left [source]

Additivity of the polarized measure in its first (left) vector argument, for a fixed measurable set `S`.

```
lemma polarizedStarHomScalarMeasure_add_left (x₁ x₂ y : H) {S : Set (X)}
    (hS : MeasurableSet S) :
    polarizedStarHomScalarMeasure Φ (x₁ + x₂) y S =
      polarizedStarHomScalarMeasure Φ x₁ y S +
        polarizedStarHomScalarMeasure Φ x₂ y S
```

lemma polarizedStarHomScalarMeasure_I_smul_right [source]

Multiplying the second vector by `i` multiplies the polarized measure by `-i`.

```
lemma polarizedStarHomScalarMeasure_I_smul_right (x y : H) {S : Set (X)}
    (hS : MeasurableSet S) :
    polarizedStarHomScalarMeasure Φ x (Complex.I • y) S =
      -Complex.I * polarizedStarHomScalarMeasure Φ x y S
```

lemma polarizedStarHomScalarMeasure_I_smul_left [source]

Multiplying the first vector by `i` multiplies the polarized measure by `i`.

```
lemma polarizedStarHomScalarMeasure_I_smul_left (x y : H) {S : Set (X)}
    (hS : MeasurableSet S) :
    polarizedStarHomScalarMeasure Φ (Complex.I • x) y S =
      Complex.I * polarizedStarHomScalarMeasure Φ x y S
```

lemma polarizedStarHomScalarMeasure_norm_le [source]

A crude but sufficient bound on the polarized measure, from the parallelogram law and `starHomScalarMeasure_real_le_univ`. This is enough to see `(x, y) ↦ polarizedStarHomScalarMeasure Φ x y S` is jointly bounded, the input a Riesz-representation argument needs.

```
lemma polarizedStarHomScalarMeasure_norm_le (x y : H) {S : Set (X)}
    (hS : MeasurableSet S) :
    ‖polarizedStarHomScalarMeasure Φ x y S‖ ≤ ‖x‖ ^ 2 + ‖y‖ ^ 2
```

lemma polarizedStarHomScalarMeasure_real_smul_right [source]

The polarized measure is real homogeneous in the second vector.

```
lemma polarizedStarHomScalarMeasure_real_smul_right (x y : H) (r : ℝ) {S : Set (X)}
    (hS : MeasurableSet S) :
    polarizedStarHomScalarMeasure Φ x (r • y) S =
      (r : ℂ) * polarizedStarHomScalarMeasure Φ x y S
```

lemma polarizedStarHomScalarMeasure_real_smul_left [source]

Real-scalar homogeneity in the left (first) argument, the counterpart of `polarizedStarHomScalarMeasure_rea` needed for conjugate-linearity in `x` (real scalars are self-conjugate, so the same statement serves both roles).

```
lemma polarizedStarHomScalarMeasure_real_smul_left (x y : H) (r : ℝ) {S : Set (X)}
    (hS : MeasurableSet S) :
    polarizedStarHomScalarMeasure Φ (r • x) y S =
      (r : ℂ) * polarizedStarHomScalarMeasure Φ x y S
```

lemma polarizedStarHomScalarMeasure_smul_left [source]

Full $\mathbb{C}$-linearity of the polarized measure in its first (left) argument.

```
lemma polarizedStarHomScalarMeasure_smul_left (c : ℂ) (x y : H) {S : Set (X)}
    (hS : MeasurableSet S) :
    polarizedStarHomScalarMeasure Φ (c • x) y S =
      c * polarizedStarHomScalarMeasure Φ x y S
```

lemma polarizedStarHomScalarMeasure_smul_right [source]

Full conjugate-$\mathbb{C}$-linearity of the polarized measure in its second (right) argument.

```
lemma polarizedStarHomScalarMeasure_smul_right (c : ℂ) (x y : H) {S : Set (X)}
    (hS : MeasurableSet S) :
    polarizedStarHomScalarMeasure Φ x (c • y) S =
      (starRingEnd ℂ c) * polarizedStarHomScalarMeasure Φ x y S
```

lemma polarizedStarHomScalarMeasure_zero_left [source]

```
lemma polarizedStarHomScalarMeasure_zero_left (y : H) {S : Set (X)}
    (hS : MeasurableSet S) : polarizedStarHomScalarMeasure Φ 0 y S = 0
```

lemma polarizedStarHomScalarMeasure_zero_right [source]

```
lemma polarizedStarHomScalarMeasure_zero_right (x : H) {S : Set (X)}
    (hS : MeasurableSet S) : polarizedStarHomScalarMeasure Φ x 0 S = 0
```

lemma polarizedStarHomScalarMeasure_norm_le_mul [source]

The polarized measure is bounded by $\|x\|\ \|y\|$.

```
lemma polarizedStarHomScalarMeasure_norm_le_mul (x y : H) {S : Set (X)}
    (hS : MeasurableSet S) :
    ‖polarizedStarHomScalarMeasure Φ x y S‖ ≤ 2 * ‖x‖ * ‖y‖
```

def starHomSesquilinearFormAux [source]

The algebraic (not-yet-continuous) sesquilinear form representing `S`, conjugate-linear in the first argument and linear in the second — the convention Mathlib's `InnerProductSpace.continuousLi` expects.

```
noncomputable def starHomSesquilinearFormAux (S : Set (X)) (hS : MeasurableSet S) :
    H →ₛₗ[starRingEnd ℂ] H →ₗ[ℂ] ℂ
```

lemma starHomSesquilinearFormAux_apply [source]

```
lemma starHomSesquilinearFormAux_apply (S : Set (X)) (hS : MeasurableSet S) (x y : H) :
    starHomSesquilinearFormAux Φ S hS x y =
      starRingEnd ℂ (polarizedStarHomScalarMeasure Φ x y S)
```

def starHomSesquilinearForm [source]

The continuous sesquilinear form, in the shape `InnerProductSpace.continuousLinearMapOfBilin` consumes.

```
noncomputable def starHomSesquilinearForm (S : Set (X)) (hS : MeasurableSet S) :
    H →L⋆[ℂ] H →L[ℂ] ℂ
```

lemma starHomSesquilinearForm_apply [source]

```
@[simp]
lemma starHomSesquilinearForm_apply (S : Set (X)) (hS : MeasurableSet S) (x y : H) :
    starHomSesquilinearForm Φ S hS x y =
      starRingEnd ℂ (polarizedStarHomScalarMeasure Φ x y S)
```

2.539 PhyslibAlpha.ProbabilisticTheory.HilbertSpace.Unbounded.StarHomSpec.SpectralMeasure

PhyslibAlpha/ProbabilisticTheory/HilbertSpace/Unbounded/StarHomSpec/SpectralMeasure.lean on GitHub.

def starHomSpectralOperatorAux [source]

The Riesz-represented operator at the measurable set `S`.

```
noncomputable def starHomSpectralOperatorAux (S : Set (X)) (hS : MeasurableSet S) :
    H →L[ℂ] H
```

lemma starHomSpectralOperatorAux_inner [source]

```
lemma starHomSpectralOperatorAux_inner (S : Set (X)) (hS : MeasurableSet S) (x y : H) :
    ⟪y, starHomSpectralOperatorAux Φ S hS x⟫_ℂ =
      polarizedStarHomScalarMeasure Φ x y S
```

def starHomSpectralOperator [source]

The Riesz-represented operator on every set, `0` off the measurable sets so that this directly matches `MeasureTheory.VectorMeasure`'s `not_measurable'` convention.

```
noncomputable def starHomSpectralOperator (S : Set (X)) : H →L[ℂ] H
```

lemma starHomSpectralOperator_inner [source]

```
lemma starHomSpectralOperator_inner (S : Set (X)) (x y : H) :
    ⟪y, starHomSpectralOperator Φ S x⟫_ℂ = polarizedStarHomScalarMeasure Φ x y S
```

lemma starHomSpectralOperator_apply_eq_zero_of_not_measurableSet [source]

```
lemma starHomSpectralOperator_apply_eq_zero_of_not_measurableSet {S : Set (X)}
    (hS : ¬MeasurableSet S) : starHomSpectralOperator Φ S = 0
```

lemma starHomScalarMeasure_real_eq_negI_smul [source]

The `Complex.I`-inversion companion of `starHomScalarMeasure_real_I_smul`: rewrites a Riesz measure at `v` as the Riesz measure at `(-I) • v`. This is the algebraic engine behind Hermitian symmetry of the polarized measure below.

```
lemma starHomScalarMeasure_real_eq_negI_smul (v : H) (S : Set (X)) :
    (starHomScalarMeasure Φ v).real S = (starHomScalarMeasure Φ ((-Complex.I) • v)).real S
```

lemma polarizedStarHomScalarMeasure_conj_symm [source]

Hermitian symmetry of the polarized measure: `B(y,x,S) = conj (B(x,y,S))`, matching the convention that `B` is exactly `⟪y, starHomRealOperator Φ f x⟫` in the limit and inner products are conjugate-symmetric. This is the input `starHomSpectralOperator_isSelfAdjoint` needs.

```
lemma polarizedStarHomScalarMeasure_conj_symm (x y : H) (S : Set (X)) :
    polarizedStarHomScalarMeasure Φ y x S =
      starRingEnd ℂ (polarizedStarHomScalarMeasure Φ x y S)
```

lemma starHomSpectralOperator_isSelfAdjoint [source]

`starHomSpectralOperator Φ S` is self-adjoint, for every `S` (measurable or not — the non-measurable case is trivial since the operator is `0`). This follows from `polarizedStarHomScalarMeasure_conj_symm` via `LinearMap.IsSymmetric`.

```
lemma starHomSpectralOperator_isSelfAdjoint (S : Set (X)) :
    IsSelfAdjoint (starHomSpectralOperator Φ S)
```

lemma polarizedStarHomScalarMeasure_univ [source]

The polarized measure of the whole spectrum is `⟪y, x⟫`.

```
lemma polarizedStarHomScalarMeasure_univ (x y : H) :
    polarizedStarHomScalarMeasure Φ x y Set.univ = ⟪y, x⟫_ℂ
```

lemma starHomSpectralOperator_univ_apply [source]

```
lemma starHomSpectralOperator_univ_apply (x : H) :
    starHomSpectralOperator Φ Set.univ x = x
```

lemma starHomSpectralOperator_univ [source]

```
lemma starHomSpectralOperator_univ : starHomSpectralOperator Φ Set.univ = 1
```

lemma starHomSpectralOperator_inner_self [source]

On the diagonal, `⟪x, E(S) x⟫` is the diagonal measure of `S`.

```
lemma starHomSpectralOperator_inner_self {S : Set (X)} (hS : MeasurableSet S) (x : H) :
    ⟪x, starHomSpectralOperator Φ S x⟫_ℂ = ((starHomScalarMeasure Φ x).real S : ℂ)
```

lemma starHomSpectralOperator_reApplyInnerSelf [source]

`reApplyInnerSelf` version of `starHomSpectralOperator_inner_self`, in the exact shape `ContinuousLinearMap.IsPositive` consumes.

```
lemma starHomSpectralOperator_reApplyInnerSelf {S : Set (X)} (hS : MeasurableSet S)
    (x : H) :
    (starHomSpectralOperator Φ S).reApplyInnerSelf x = (starHomScalarMeasure Φ x).real S
```

lemma starHomSpectralOperator_isPositive [source]

starHomSpectralOperator Φ S is a positive operator, for measurable S.

```
lemma starHomSpectralOperator_isPositive {S : Set (X)} (hS : MeasurableSet S) :
    ContinuousLinearMap.IsPositive (starHomSpectralOperator Φ S)
```

lemma starHomSpectralOperator_nonneg [source]

```
lemma starHomSpectralOperator_nonneg {S : Set (X)} (hS : MeasurableSet S) :
    0 ≤ starHomSpectralOperator Φ S
```

lemma starHomSpectralOperator_mono [source]

starHomSpectralOperator Φ S is monotone in S, for measurable sets, in the Loewner order on H →L[ℂ] H.

```
lemma starHomSpectralOperator_mono {S T : Set (X)} (hS : MeasurableSet S)
    (hT : MeasurableSet T) (hST : S ⊆ T) :
    starHomSpectralOperator Φ S ≤ starHomSpectralOperator Φ T
```

lemma starHomSpectralOperator_le_one [source]

```
lemma starHomSpectralOperator_le_one {S : Set (X)} (hS : MeasurableSet S) :
    starHomSpectralOperator Φ S ≤ 1
```

lemma starHomSpectralOperator_empty [source]

```
lemma starHomSpectralOperator_empty : starHomSpectralOperator Φ ∅ = 0
```

lemma starHomSpectralOperator_add_of_disjoint [source]

Finite additivity of starHomSpectralOperator on disjoint measurable sets.

```
lemma starHomSpectralOperator_add_of_disjoint {S T : Set (X)} (hS : MeasurableSet S)
    (hT : MeasurableSet T) (hdisj : Disjoint S T) :
    starHomSpectralOperator Φ (S ∪ T) =
      starHomSpectralOperator Φ S + starHomSpectralOperator Φ T
```

def starHomSpectralMeasureFun [source]

The operator at S, valued in the weak-operator-topology type.

```
noncomputable def starHomSpectralMeasureFun (S : Set (X)) : H →WOT[ℂ] H
```

lemma starHomSpectralMeasureFun_inner [source]

```
@[simp]
lemma starHomSpectralMeasureFun_inner (S : Set (X)) (x y : H) :
    ⟪y, starHomSpectralMeasureFun Φ S x⟫_ℂ = polarizedStarHomScalarMeasure Φ x y S
```

def starHomSpectralVectorMeasure [source]

The operators `E(S)`, as a vector measure in the weak operator topology.

```
noncomputable def starHomSpectralVectorMeasure :
    VectorMeasure (X) (H →WOT[ℂ] H) where
```

lemma starHomSpectralVectorMeasure_apply [source]

```
@[simp]
lemma starHomSpectralVectorMeasure_apply (S : Set (X)) :
    starHomSpectralVectorMeasure Φ S = starHomSpectralMeasureFun Φ S
```

lemma starHomSpectralOperatorAux_nonneg [source]

```
lemma starHomSpectralOperatorAux_nonneg {S : Set (X)} (hS : MeasurableSet S) :
    0 ≤ starHomSpectralOperatorAux Φ S hS
```

lemma starHomSpectralOperatorAux_le_one [source]

```
lemma starHomSpectralOperatorAux_le_one {S : Set (X)} (hS : MeasurableSet S) :
    starHomSpectralOperatorAux Φ S hS ≤ 1
```

lemma starHomSpectralOperatorAux_mono [source]

```
lemma starHomSpectralOperatorAux_mono {S T : Set (X)} (hS : MeasurableSet S)
    (hT : MeasurableSet T) (hST : S ⊆ T) :
    starHomSpectralOperatorAux Φ S hS ≤ starHomSpectralOperatorAux Φ T hT
```

lemma starHomRealOperator_reApplyInnerSelf [source]

```
lemma starHomRealOperator_reApplyInnerSelf
    (f : CompactlySupportedContinuousMap (X) ℝ) (x : H) :
    (starHomRealOperator Φ f).reApplyInnerSelf x =
      ∫ z, f z ∂starHomScalarMeasure Φ x
```

lemma starHomSpectralOperator_le_cfcRealOperator_of_compact_subset_open [source]

```
lemma starHomSpectralOperator_le_cfcRealOperator_of_compact_subset_open
    {K V : Set (X)} (hK : IsCompact K) (_hV : IsOpen V) (_hKV : K ⊆ V)
    (f : CompactlySupportedContinuousMap (X) ℝ)
    (hfK : Set.EqOn f 1 K) (_hfV : tsupport f ⊆ V) (hf : ∀ z, f z ∈ Set.Icc 0 1) :
    starHomSpectralOperator Φ K ≤ starHomRealOperator Φ f
```

lemma starHomRealOperator_le_cfcSpectralOperator_of_support_subse [source]

```
lemma starHomRealOperator_le_cfcSpectralOperator_of_support_subse
    {V : Set (X)} (hV : IsOpen V)
    (f : CompactlySupportedContinuousMap (X) ℝ)
    (hfV : tsupport f ⊆ V) (hf : ∀ z, f z ∈ Set.Icc 0 1) :
    starHomRealOperator Φ f ≤ starHomSpectralOperator Φ V
```

lemma starHomRealOperator_mul_self [source]

```
lemma starHomRealOperator_mul_self
    (f : CompactlySupportedContinuousMap (X) ℝ) :
    starHomRealOperator Φ (f * f) = starHomRealOperator Φ f * starHomRealOperator Φ f
```

lemma starHomRealOperator_sub_mul_self [source]

```
lemma starHomRealOperator_sub_mul_self
    (f : CompactlySupportedContinuousMap (X) ℝ) :
    starHomRealOperator Φ (f - f * f) =
      starHomRealOperator Φ f - starHomRealOperator Φ f * starHomRealOperator Φ f
```

lemma starHomSpectralOperator_isIdempotent_of_isCompact [source]

```
lemma starHomSpectralOperator_isIdempotent_of_isCompact
    {K : Set (X)} (hK : IsCompact K) :
    IsIdempotentElem (starHomSpectralOperator Φ K)
```

lemma starHomSpectralOperatorAux_isIdempotentElem [source]

***`E(S)` is idempotent.** For compact `S` this follows from a Urysohn function between `S` and an open neighbourhood, using `0 ≤ A ≤ 1 ⟹ ‖A x‖² ≤ re ⟪x, A x⟫`; inner regularity extends it to measurable `S`.*

```
lemma starHomSpectralOperatorAux_isIdempotentElem (S : Set (X)) (hS : MeasurableSet S) :
    starHomSpectralOperatorAux Φ S hS * starHomSpectralOperatorAux Φ S hS =
      starHomSpectralOperatorAux Φ S hS
```

lemma starHomSpectralOperator_isIdempotentElem [source]

Idempotency of `starHomSpectralOperator` on every set — `0` (hence trivially idempotent) off the measurable sets, and the order/regularity theorem above on measurable sets.

```
lemma starHomSpectralOperator_isIdempotentElem (S : Set (X)) :
    IsIdempotentElem (starHomSpectralOperator Φ S)
```

lemma starHomSpectralOperator_isStarProjection [source]

`starHomSpectralOperator Φ S` is a star-projection, for every `S`; both self-adjointness and idempotency are proved.

```
lemma starHomSpectralOperator_isStarProjection (S : Set (X)) :
    IsStarProjection (starHomSpectralOperator Φ S)
```

def starHomSpectralMeasure [source]

***The weak-operator spectral measure of a bounded normal operator.** Assembled from the Riesz-represented, weakly σ-additive family `starHomSpectralOperator` (Steps B–C, fully proved) and the star-projection property (Step D), including the order/regularity proof of idempotency.*

```
noncomputable def starHomSpectralMeasure : QuantumMechanics.WOTSpectralMeasure (X) H where
```

lemma starHomSpectralMeasure_apply [source]

```
@[simp]
lemma starHomSpectralMeasure_apply (S : Set (X)) :
    starHomSpectralMeasure Φ S = starHomSpectralMeasureFun Φ S
```

lemma starHomSpectralMeasure_scalarMeasure [source]

The scalar measure attached to `starHomSpectralMeasure` is exactly the polarized Riesz measure it was built from.

```
lemma starHomSpectralMeasure_scalarMeasure (x y : H) :
    (starHomSpectralMeasure Φ).scalarMeasure x y = polarizedStarHomScalarMeasure Φ x y
```

lemma starHomSpectralMeasure_reconstruction [source]

***The spectral measure reconstructs the functional calculus**: integrating a continuous function `f` gives `f(U)`.*

```
lemma starHomSpectralMeasure_reconstruction
    (f : CompactlySupportedContinuousMap (X) ℝ) (x y : H) :
    (starHomSpectralMeasure Φ).complexWeakIntegral (fun z => (f z : ℂ)) x y =
      ⟪y, starHomRealOperator Φ f x⟫_ℂ
```

lemma starHomRealOperator_commute [source]

An operator commuting with the range of `Φ` commutes with `Φ` at every real test function.

```
lemma starHomRealOperator_commute {T : H →L[ℂ] H} (hT : ∀ f : C(X, ℂ), Commute (Φ f) T)
    (f : CompactlySupportedContinuousMap (X) ℝ) :
    Commute (starHomRealOperator Φ f) T
```

lemma starHomScalarMeasure_eq_of_commute_isometry [source]

If `T` is an isometry (`star T * T = 1`) commuting with `U` and `star U`, the vector-state Riesz measure at `T v` agrees with the one at `v`: an isometric intertwiner of `U` leaves the diagonal spectral measure unchanged.

```
lemma starHomScalarMeasure_eq_of_commute_isometry
    {T : H →L[ℂ] H} (hT : ∀ f : C(X, ℂ), Commute (Φ f) T)
    (hTiso : star T * T = 1) (v : H) :
    starHomScalarMeasure Φ (T v) = starHomScalarMeasure Φ v
```

lemma polarizedStarHomScalarMeasure_eq_of_commute_unitary [source]

If `T` is unitary and commutes with `U` and `star U`, the polarized Riesz measure intertwines: testing at `(T x, y)` matches testing at `(x, star T y)`. This is the sesquilinear form of the commutation identity, and is exactly what `starHomSpectralOperator_inner` needs to see commutation of each spectral projection with `T`.

```
lemma polarizedStarHomScalarMeasure_eq_of_commute_unitary
    {T : H →L[ℂ] H} (hT : ∀ f : C(X, ℂ), Commute (Φ f) T)
    (hTunit : T ∈ unitary (H →L[ℂ] H)) (x y : H) :
    polarizedStarHomScalarMeasure Φ (T x) y =
      polarizedStarHomScalarMeasure Φ x (star T y)
```

lemma starHomSpectralOperator_commute_of_commute_unitary [source]

*Every spectral operator `starHomSpectralOperator Φ S` commutes with any unitary intertwiner of `U` (i.e. any unitary commuting with both `U` and `star U`). Holds for *every* `S`, measurable or not, since `starHomSpectralOperator_inner` itself needs no measurability hypothesis.*

```
lemma starHomSpectralOperator_commute_of_commute_unitary
    {T : H →L[ℂ] H} (hT : ∀ f : C(X, ℂ), Commute (Φ f) T)
    (hTunit : T ∈ unitary (H →L[ℂ] H)) (S : Set (X)) :
    starHomSpectralOperator Φ S * T = T * starHomSpectralOperator Φ S
```

lemma starHomSpectralMeasure_commute_of_commute_unitary [source]

The weak-operator spectral measure `starHomSpectralMeasure Φ` commutes with any unitary intertwiner of `U`, at every set (measurable or not).

```
lemma starHomSpectralMeasure_commute_of_commute_unitary
    {T : H →L[ℂ] H} (hT : ∀ f : C(X, ℂ), Commute (Φ f) T)
    (hTunit : T ∈ unitary (H →L[ℂ] H)) (S : Set (X)) :
    starHomSpectralMeasure Φ S * ContinuousLinearMapWOT.ofCLM T =
      ContinuousLinearMapWOT.ofCLM T * starHomSpectralMeasure Φ S
```

2.540 PhyslibAlpha.ProbabilisticTheory.QuasiLocal.AlgebraNet

PhyslibAlpha/ProbabilisticTheory/QuasiLocal/AlgebraNet.lean on GitHub.

structure AlgebraNet [source]

*A net of unital complex *-algebras over the finite regions of a set of sites: the algebra of each region, and the inclusion of the algebra of a region in that of a larger region.*

```
structure AlgebraNet (S : Type*) where
  A : Finset S → Type*
  [ring : ∀ X, Ring (A X)]
  [algebra : ∀ X, Algebra ℂ (A X)]
  [starRing : ∀ X, StarRing (A X)]
  [starModule : ∀ X, StarModule ℂ (A X)]
  incl : ∀ {X Y : Finset S}, X ≤ Y → A X →⋆ₐ[ℂ] A Y
  incl_refl : ∀ (X : Finset S) (a : A X), incl (le_refl X) a = a
  incl_trans : ∀ {X Y Z : Finset S} (h₁ : X ≤ Y) (h₂ : Y ≤ Z) (a : A X),
    incl (h₁.trans h₂) a = incl h₂ (incl h₁ a)
```

instance DirectedSystem N.A (fun X Y (h : X ≤ Y) => N.incl h) [source]

```
instance : DirectedSystem N.A (fun X Y (h : X ≤ Y) => N.incl h) where
```

abbrev Quasilocal [source]

***The quasi-local algebra** of a net: the direct limit of the algebras of the finite regions.*

```
abbrev Quasilocal : Type _
```

def of [source]

*The embedding of the algebra of a region in the quasi-local algebra, as a unital *-homomorphism.*

```
noncomputable def of (X : Finset S) : N.A X →⋆ₐ[ℂ] N.Quasilocal where
```

lemma of_incl [source]

The embeddings of the local algebras are compatible with the inclusions.

```
lemma of_incl {X Y : Finset S} (h : X ≤ Y) (a : N.A X) : N.of Y (N.incl h a) = N.of X a
```

lemma exists_of [source]

Every element of the quasi-local algebra is an observable of some finite region.

```
lemma exists_of (x : N.Quasilocal) : ∃ (X : Finset S) (a : N.A X), N.of X a = x
```

def moveHom [source]

*A map of the quasi-local algebra induced by maps of the local algebras that move regions: a monotone map `ρ` of regions and, for each region `X`, a unital *-homomorphism of its algebra into that of `ρ X`, natural with respect to the inclusions.*

```
noncomputable def moveHom (ρ : Finset S → Finset S) (hρ : Monotone ρ)
    (φ : ∀ X, N.A X →⋆ₐ[ℂ] N.A (ρ X))
    (hφ : ∀ {X Y : Finset S} (h : X ≤ Y) (a : N.A X), φ Y (N.incl h a) = N.incl (hρ h) (φ X a))
     :
    N.Quasilocal →⋆ₐ[ℂ] N.Quasilocal where
```

lemma moveHom_of [source]

```
lemma moveHom_of (ρ : Finset S → Finset S) (hρ : Monotone ρ) (φ : ∀ X, N.A X →⋆ₐ[ℂ] N.A (ρ X))
    (hφ : ∀ {X Y : Finset S} (h : X ≤ Y) (a : N.A X), φ Y (N.incl h a) = N.incl (hρ h) (φ X a))
    (X : Finset S) (a : N.A X) : N.moveHom ρ hρ φ hφ (N.of X a) = N.of (ρ X) (φ X a)
```

2.541 PhyslibAlpha.ProbabilisticTheory.QuasiLocal.Limit

PhyslibAlpha/ProbabilisticTheory/QuasiLocal/Limit.lean on GitHub.

def sumSq [source]

The sums of squares ∑ s_i s_i of a *-algebra.*

```
def sumSq (A : Type*) [Ring A] [StarRing A] : AddSubmonoid A
```

lemma normSq_le [source]

***The Cauchy–Schwarz inequality for states**: |ω(a)|² ≤ ω(a* a).*

```
lemma _root_.ProbabilisticTheory.AlgState.normSq_le (ω : AlgState A) (a : A) :
    ((‖ω a‖ ^ 2 : ℝ) : ℂ) ≤ ω (star a * a)
```

def IsLocallyBounded [source]

***Local boundedness**: for every local observable a some multiple c · 1 of the identity exceeds a* a by a sum of squares.*

```
def AlgebraNet.IsLocallyBounded : Prop
```

lemma nonneg_of_mem_sumSq [source]

A state is nonnegative on sums of squares.

```
lemma _root_.ProbabilisticTheory.AlgState.nonneg_of_mem_sumSq
    {A : Type*} [Ring A] [Algebra ℂ A] [StarRing A] [StarModule ℂ A] (ω : AlgState A)
    {p : A} (hp : p ∈ sumSq A) : 0 ≤ ω p
```

lemma norm_sq_le [source]

The expectation of a locally bounded observable in any larger region is bounded.

```
lemma AlgebraNet.norm_sq_le (hb : N.IsLocallyBounded) {X Λ : Finset S} (h : X ≤ Λ)
    (ω : AlgState (N.A Λ)) (a : N.A X) :
    ∃ c : ℝ, ‖ω (N.incl h a)‖ ^ 2 ≤ c ∧ ∀ Λ' (h' : X ≤ Λ') (ω' : AlgState (N.A Λ')),
      ‖ω' (N.incl h' a)‖ ^ 2 ≤ c
```

def localSeq [source]

The expectation of a local observable a of the region X in the region Λ, zero if Λ does not contain X.

```
def localSeq (X : Finset S) (a : N.A X) (Λ : Finset S) : ℂ
```

lemma localSeq_eq [source]

```
lemma localSeq_eq (X : Finset S) (a : N.A X) {Λ : Finset S} (h : X ≤ Λ) :
    localSeq ω X a Λ = ω Λ (N.incl h a)
```

lemma exists_tendsto_localSeq [source]

The expectations of a local observable have a limit along a fine ultrafilter.

```
lemma exists_tendsto_localSeq (hb : N.IsLocallyBounded) (h𝒰 : (𝒰 : Filter (Finset S)) ≤ atTop)
    (X : Finset S) (a : N.A X) : ∃ z, Tendsto (localSeq ω X a) 𝒰 (𝒩 z)
```

def limitValue [source]

The limit along the ultrafilter of the expectations of a local observable.

```
def limitValue (X : Finset S) (a : N.A X) : ℂ
```

lemma tendsto_limitValue [source]

```
lemma tendsto_limitValue (X : Finset S) (a : N.A X) :
    Tendsto (localSeq ω X a) 𝒰 (𝒩 (limitValue ω hb h𝒰 X a))
```

def limitFamily [source]

***The limit of a net of local states along an ultrafilter** is a compatible family of local states.*

```
def limitFamily : LocalStateFamily N where
```

def limitState [source]

***The limit state** of a net of local states along an ultrafilter: a state of the quasi-local algebra.*

```
def limitState : AlgState N.Quasilocal
```

lemma limitState_of [source]

The limit state on a local observable is the limit of the local expectations.

```
lemma limitState_of (X : Finset S) (a : N.A X) :
    Tendsto (fun Λ => localSeq ω X a Λ) 𝒰 (𝒩 (limitState ω hb h𝒰 (N.of X a)))
```

2.542 PhyslibAlpha.ProbabilisticTheory.QuasiLocal.State

PhyslibAlpha/ProbabilisticTheory/QuasiLocal/State.lean on GitHub.

structure LocalStateFamily [source]

A compatible family of local states of a net of algebras.

```
structure LocalStateFamily where
  state : ∀ X : Finset S, N.A X →ₗ[ℂ] ℂ
  compat : ∀ {X Y : Finset S} (h : X ≤ Y) (a : N.A X), state Y (N.incl h a) = state X a
  map_one : ∀ X, state X 1 = 1
  map_star : ∀ X (a : N.A X), state X (star a) = star (state X a)
  nonneg : ∀ X (a : N.A X), 0 ≤ state X (star a * a)
```

def lift [source]

The linear functional on the quasi-local algebra defined by a compatible family.

```
def LocalStateFamily.lift (ω : LocalStateFamily N) : N.Quasilocal →ₗ[ℂ] ℂ
```

lemma lift_of [source]

```
lemma LocalStateFamily.lift_of (ω : LocalStateFamily N) (X : Finset S) (a : N.A X) :
    ω.lift (N.of X a) = ω.state X a
```

def toAlgState [source]

***The state of the quasi-local algebra** defined by a compatible family.*

```
def LocalStateFamily.toAlgState (ω : LocalStateFamily N) : AlgState N.Quasilocal where
```

lemma toAlgState_of [source]

The state of a compatible family on a local observable is the local expectation.

```
lemma LocalStateFamily.toAlgState_of (ω : LocalStateFamily N) (X : Finset S) (a : N.A X) :
    ω.toAlgState (N.of X a) = ω.state X a
```

2.543 PhyslibAlpha.ProbabilisticTheory.Representation.AbelianSpectral

PhyslibAlpha/ProbabilisticTheory/Representation/AbelianSpectral.lean on GitHub.

def family [source]

The operators of a unitary representation of an abelian group, as a commuting ⋆-family.

```
def family : CommutingStarFamily H where
```

lemma mem_family [source]

```
lemma mem_family (g : Multiplicative G) :
    ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) ∈ (family π).carrier
```

def elem [source]

The element of the algebra generated by the representation given by `π g`.

```
def elem (g : Multiplicative G) : (family π).algebra
```

lemma elem_mul [source]

```
lemma elem_mul (g h : Multiplicative G) : elem π (g * h) = elem π g * elem π h
```

lemma elem_one [source]

```
lemma elem_one : elem π 1 = 1
```

lemma star_elem [source]

```
lemma star_elem (g : Multiplicative G) : star (elem π g) = elem π g⁻¹
```

def charValue [source]

The value at `π g` of a character of the algebra generated by the representation.

```
def charValue (ψ : characterSpace ℂ (family π).algebra) (g : Multiplicative G) : ℂ
```

lemma norm_charValue [source]

```
lemma norm_charValue (ψ : characterSpace ℂ (family π).algebra) (g : Multiplicative G) :
    ‖charValue π ψ g‖ = 1
```

lemma charValue_mul [source]

```
lemma charValue_mul (ψ : characterSpace ℂ (family π).algebra) (g h : Multiplicative G) :
    charValue π ψ (g * h) = charValue π ψ g * charValue π ψ h
```

lemma charValue_one [source]

```
lemma charValue_one (ψ : characterSpace ℂ (family π).algebra) : charValue π ψ 1 = 1
```

lemma continuous_charValue [source]

```
lemma continuous_charValue (g : Multiplicative G) :
    Continuous fun ψ : characterSpace ℂ (family π).algebra => charValue π ψ g
```

def dual [source]

A character of the algebra generated by the representation restricts to a unitary character of the group.

```
def dual (ψ : characterSpace ℂ (family π).algebra) : PontryaginDual (Multiplicative G) where
```

lemma coe_dual [source]

```
@[simp]
lemma coe_dual (ψ : characterSpace ℂ (family π).algebra) (g : Multiplicative G) :
    ((dual π ψ g : Circle) : ℂ) = ψ (elem π g)
```

lemma continuous_dual [source]

```
lemma continuous_dual : Continuous (dual π)
```

def spectralMeasure [source]

The projection-valued measure of a unitary representation of an abelian group, on the Pontryagin dual of the group.

```
def spectralMeasure : WOTSpectralMeasure (PontryaginDual (Multiplicative G)) H
```

lemma boundedIntegral_character [source]

***Spectral theorem for unitary representations of abelian groups**: every operator `π(g)` of the representation is the integral of the character `χ ↦ χ(g)` against the projection-valued measure on the Pontryagin dual.*

```
lemma boundedIntegral_character (g : Multiplicative G) :
    ContinuousLinearMapWOT.toCLM
      (WOTSpectralMeasure.boundedIntegral (spectralMeasure π)
        (fun χ : PontryaginDual (Multiplicative G) => ((χ g : Circle) : ℂ))
        (TauCeti.PontryaginDual.continuous_coe_eval_const g).measurable
        ⟨1, fun χ => by simp⟩) = ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H)
```

2.544 PhyslibAlpha.ProbabilisticTheory.Representation.Bands

PhyslibAlpha/ProbabilisticTheory/Representation/Bands.lean on GitHub.

def bandOf [source]

The `k`-th largest eigenvalue of a Hermitian matrix: its `k`-th band.

```
def bandOf {N : ℕ} {M : Matrix (Fin N) (Fin N) ℂ} (hM : M.IsHermitian)
    (k : Fin (Fintype.card (Fin N))) : ℝ
```

lemma abs_bandOf_sub_le [source]

***Weyl's inequality for bands**: the `k`-th band of two Hermitian matrices differ by at most the operator norm of the difference.*

```
lemma abs_bandOf_sub_le {N : ℕ} {M S : Matrix (Fin N) (Fin N) ℂ} (hM : M.IsHermitian)
    (hS : S.IsHermitian) (k : Fin (Fintype.card (Fin N))) :
    |bandOf hM k - bandOf hS k| ≤
      ‖LinearMap.toContinuousLinearMap (Matrix.toEuclideanLin (M - S))‖
```

lemma continuous_bandOf [source]

***Bands are continuous**: the `k`-th band of a continuous family of Hermitian matrices is a continuous function.*

```
lemma continuous_bandOf {X : Type*} [TopologicalSpace X] {N : ℕ}
    {b : X → Matrix (Fin N) (Fin N) ℂ} (hb : ∀ x, (b x).IsHermitian) (hc : Continuous b)
    (k : Fin (Fintype.card (Fin N))) : Continuous fun x => bandOf (hb x) k
```

lemma isSelfAdjoint_blochHom [source]

The operator of a Hermitian symbol is self-adjoint.

```
lemma isSelfAdjoint_blochHom
    (b : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (hb : ∀ χ, (b χ).IsHermitian) : IsSelfAdjoint (blochHom π ι hι b)
```

lemma exists_band_eq_of_mem_spectrum [source]

***Every point of the spectrum of the operator of a Hermitian symbol is a band value**: it is the `k`-th band `ε_k(χ)` of some crystal momentum `χ`.*

```
lemma exists_band_eq_of_mem_spectrum
    (b : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (hb : ∀ χ, (b χ).IsHermitian) {z : ℂ} (hz : z ∈ spectrum ℂ (blochHom π ι hι b)) :
    ∃ (χ : PontryaginDual (Multiplicative G)) (k : Fin (Fintype.card (Fin N))),
      z = ((bandOf (hb χ) k : ℝ) : ℂ)
```

2.545 PhyslibAlpha.ProbabilisticTheory.Representation.Commutant

PhyslibAlpha/ProbabilisticTheory/Representation/Commutant.lean on GitHub.

def projection [source]

The spectral projection of a set of characters, as a bounded operator.

```
def projection (S : Set (PontryaginDual (Multiplicative G))) : H →L[ℂ] H
```

lemma projection_isStarProjection [source]

```
lemma projection_isStarProjection (S : Set (PontryaginDual (Multiplicative G))) :
    IsStarProjection (projection π S)
```

lemma projection_eq_zero_of_not_measurable [source]

```
lemma projection_eq_zero_of_not_measurable {S : Set (PontryaginDual (Multiplicative G))}
    (hS : ¬ MeasurableSet S) : projection π S = 0
```

lemma projection_commute [source]

***The commutant is decomposable**: every bounded operator commuting with the representation commutes with every spectral projection.*

```
lemma projection_commute {T : H →L[ℂ] H}
    (hT : ∀ g, Commute ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) T)
    (S : Set (PontryaginDual (Multiplicative G))) : Commute (projection π S) T
```

lemma commute_starProjection_of_mapsTo [source]

An operator mapping a closed subspace and its orthogonal complement into themselves commutes with the orthogonal projection onto the subspace.

```
lemma commute_starProjection_of_mapsTo (W : Submodule ℂ H) [W.HasOrthogonalProjection]
    {T : H →L[ℂ] H} (hW : ∀ v ∈ W, T v ∈ W) (hWo : ∀ v ∈ W^⊥, T v ∈ W^⊥) :
    Commute (W.starProjection : H →L[ℂ] H) T
```

lemma inner_unitary_left [source]

```
lemma inner_unitary_left (g : Multiplicative G) (x w : H) :
    ⟪((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) x, w⟫_ℂ =
      ⟪x, ((π g⁻¹ : unitary (H →L[ℂ] H)) : H →L[ℂ] H) w⟫_ℂ
```

lemma mapsTo_projection_of_invariant [source]

A closed subspace invariant under the representation is invariant under every spectral projection.

```
lemma mapsTo_projection_of_invariant (W : Submodule ℂ H) [W.HasOrthogonalProjection]
    (hW : ∀ g, ∀ v ∈ W, ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) v ∈ W)
    (S : Set (PontryaginDual (Multiplicative G))) {v : H} (hv : v ∈ W) :
    projection π S v ∈ W
```

lemma mapsTo_of_projection_invariant [source]

A closed subspace invariant under every spectral projection is invariant under the representation.

```
lemma mapsTo_of_projection_invariant (W : Submodule ℂ H) [W.HasOrthogonalProjection]
    (hW : ∀ (S : Set (PontryaginDual (Multiplicative G))), ∀ v ∈ W, projection π S v ∈ W)
    (g : Multiplicative G) {v : H} (hv : v ∈ W) :
    ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) v ∈ W
```

def toEnd [source]

The linear action of a unitary representation on its Hilbert space.

```
def toEnd : Multiplicative G →* Module.End ℂ H where
```

lemma toEnd_apply [source]

```
lemma toEnd_apply (g : Multiplicative G) (v : H) :
    toEnd π g v = ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) v
```

lemma character_mul_projection_singleton [source]

The operator $\pi(g)$ acts on the spectral projection of the single character χ as the scalar $\chi(g)$.

```
lemma character_mul_projection_singleton (χ : PontryaginDual (Multiplicative G))
    (g : Multiplicative G) :
    ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) * projection π {χ} =
      ((χ g : Circle) : ℂ) • projection π {χ}
```

lemma ae_eq_of_mem_sector [source]

A vector of momentum χ is carried by the single character χ: its spectral measure is concentrated at χ.

```
lemma ae_eq_of_mem_sector (χ : PontryaginDual (Multiplicative G)) {v : H}
    (hv : v ∈ ProbabilisticTheory.Representation.sector (toEnd π) (fun g => ((χ g : Circle) :
    ℂ))) :
    ∀ᵐ χ' ∂vectorMeasure π v, χ' = χ
```

lemma projection_singleton_eq_self_of_mem_sector [source]

A vector of momentum χ lies in the range of the spectral projection of χ.

```
lemma projection_singleton_eq_self_of_mem_sector (χ : PontryaginDual (Multiplicative G)) {v : H
    }
    (hv : v ∈ ProbabilisticTheory.Representation.sector (toEnd π) (fun g => ((χ g : Circle) :
    ℂ))) :
    projection π {χ} v = v
```

lemma range_projection_singleton [source]

***The spectral projection of a single character is the projection onto the sector of definite momentum**: its range is the subspace of vectors on which the group acts through the character.*

```
lemma range_projection_singleton (χ : PontryaginDual (Multiplicative G)) :
    LinearMap.range (projection π {χ} : H →L[ℂ] H).toLinearMap =
      ProbabilisticTheory.Representation.sector (toEnd π) (fun g => ((χ g : Circle) : ℂ))
```

2.546 PhyslibAlpha.ProbabilisticTheory.Representation.Floquet

PhyslibAlpha/ProbabilisticTheory/Representation/Floquet.lean on GitHub.

instance Nonempty (PontryaginDual (Multiplicative G)) [source]

```
instance : Nonempty (PontryaginDual (Multiplicative G))
```

def calculus [source]

***The functional calculus of a unitary representation**: `f ↦ ∫ f dP` is a unital ⋆-homomorphism from the continuous functions on the Pontryagin dual to the bounded operators.*

```
def calculus : C(PontryaginDual (Multiplicative G), ℂ) →⋆ₐ[ℂ] (H →L[ℂ] H)
```

lemma calculus_character [source]

The functional calculus sends the character `χ ↦ χ(g)` to the operator `π(g)`.

```
lemma calculus_character (g : Multiplicative G) :
    calculus π (fun χ => ((χ g : Circle) : ℂ),
      (TauCeti.PontryaginDual.continuous_coe_eval_const (G := Multiplicative G) g)) =
      ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H)
```

lemma calculus_commute [source]

An operator commuting with the representation commutes with its functional calculus.

```
lemma calculus_commute {T : H →L[ℂ] H}
    (hT : ∀ g, Commute ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) T)
    (f : C(PontryaginDual (Multiplicative G), ℂ)) : Commute (calculus π f) T
```

def entry [source]

The `(i, j)` entry of a continuous matrix-valued function.

```
def entry (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ)) (i j : Fin N) :
    C(PontryaginDual (Multiplicative G), ℂ)
```

lemma calculus_commute_ι [source]

```
lemma calculus_commute_ι (f : C(PontryaginDual (Multiplicative G), ℂ))
    (M : Matrix (Fin N) (Fin N) ℂ) : calculus π f * ι M = ι M * calculus π f
```

def blochOp [source]

The operator `∑_{ij} (∫ f_{ij} dP) ι(E_{ij})` of a continuous matrix-valued function.

```
def blochOp (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ)) : H →L[ℂ] H
```

lemma entry_add [source]

```
lemma entry_add (f g : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (i j : Fin N) : entry (f + g) i j = entry f i j + entry g i j
```

lemma entry_smul [source]

```
lemma entry_smul (c : ℂ) (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (i j : Fin N) : entry (c • f) i j = c • entry f i j
```

lemma entry_zero [source]

```
lemma entry_zero (i j : Fin N) :
    entry (0 : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ)) i j = 0
```

lemma entry_mul [source]

```
lemma entry_mul (f g : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (i j : Fin N) : entry (f * g) i j = ∑ k, entry f i k * entry g k j
```

lemma entry_star [source]

```
lemma entry_star (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (i j : Fin N) : entry (star f) i j = star (entry f j i)
```

lemma entry_one [source]

```
lemma entry_one (i j : Fin N) :
    entry (1 : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ)) i j =
      if i = j then 1 else 0
```

lemma blochOp_add [source]

```
lemma blochOp_add (f g : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ)) :
    blochOp π ι (f + g) = blochOp π ι f + blochOp π ι g
```

lemma blochOp_smul [source]

```
lemma blochOp_smul (c : ℂ)
    (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ)) :
    blochOp π ι (c • f) = c • blochOp π ι f
```

lemma blochOp_one [source]

```
lemma blochOp_one : blochOp π ι (1 : C(PontryaginDual (Multiplicative G),
    Matrix (Fin N) (Fin N) ℂ)) = 1
```

lemma blochOp_mul [source]

```
lemma blochOp_mul (f g : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ)) :
    blochOp π ι (f * g) = blochOp π ι f * blochOp π ι g
```

lemma blochOp_star [source]

```
lemma blochOp_star (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ)) :
    blochOp π ι (star f) = star (blochOp π ι f)
```

def blochHom [source]

***Bloch–Floquet**: the map `f ↦ ∑_{ij} (∫ f_{ij} dP) ι(E_{ij})` is a unital ⋆-homomorphism from the continuous matrix-valued functions on the Brillouin zone to the bounded operators.*

```
def blochHom : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ) →⋆ₐ[ℂ]
    (H →L[ℂ] H) where
```

lemma spectrum_blochHom_subset [source]

***The spectrum of the operator of a symbol is contained in the union of the spectra of the symbol's values**: a translation-invariant operator has no spectrum outside its Bloch bands.*

```
lemma spectrum_blochHom_subset
    (b : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ)) :
    spectrum ℂ (blochHom π ι hι b) ⊆
      ⋃ χ : PontryaginDual (Multiplicative G), spectrum ℂ (b χ)
```

def HasFullSupport [source]

*A representation has **full support** relative to the internal space when the spectral projection of every nonempty open set of crystal momenta is nonzero on every internal state: every crystal momentum is present in every orbital.*

```
def HasFullSupport : Prop
```

lemma compress_blochOp [source]

Compressing the image of a matrix-valued function to the `(i, j)` orbitals isolates the `(i, j)` entry.

```
lemma compress_blochOp (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (i j : Fin N) :
    ι (Matrix.single i i 1) * blochOp π ι f * ι (Matrix.single j i 1) =
      calculus π (entry f i j) * ι (Matrix.single i i 1)
```

lemma entry_eq_zero_of_blochOp_eq_zero [source]

An entry of the image vanishes only if the entry function vanishes: this is the content of full support.

```
lemma entry_eq_zero_of_blochOp_eq_zero (hfull : HasFullSupport π ι)
    (f : C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ))
    (hf : blochOp π ι f = 0) : f = 0
```

lemma blochHom_injective [source]

Under full support the map `f ↦ ∫ f dP` is injective.

```
lemma blochHom_injective (hfull : HasFullSupport π ι) :
    Function.Injective (blochHom π ι hι)
```

2.547 PhyslibAlpha.ProbabilisticTheory.Representation.Hopping

PhyslibAlpha/ProbabilisticTheory/Representation/Hopping.lean on GitHub.

def hopping [source]

A translation-invariant hopping operator `∑_r π(r) ι(A_r)` with finitely many matrices `A_r`, the amplitude of hopping by the lattice vector `r` between orbitals.

```
def hopping (A : G →₀ Matrix (Fin N) (Fin N) ℂ) : H →L[ℂ] H
```

def blochMatrix [source]

The Bloch matrix `b(χ) = ∑_r χ(r) A_r` of a hopping operator, a continuous matrix-valued function on the Brillouin zone.

```
def blochMatrix (A : G →₀ Matrix (Fin N) (Fin N) ℂ) :
    C(PontryaginDual (Multiplicative G), Matrix (Fin N) (Fin N) ℂ) where
```

lemma blochMatrix_apply [source]

```
lemma blochMatrix_apply (A : G →₀ Matrix (Fin N) (Fin N) ℂ)
    (χ : PontryaginDual (Multiplicative G)) :
    blochMatrix A χ = A.sum fun r M => ((χ (Multiplicative.ofAdd r) : Circle) : ℂ) • M
```

lemma blochHom_blochMatrix [source]

The hopping operator is the integral of its Bloch matrix.

```
lemma blochHom_blochMatrix (A : G →₀ Matrix (Fin N) (Fin N) ℂ) :
    blochHom π ι hι (blochMatrix A) = hopping π ι A
```

lemma spectrum_hopping_subset [source]

The spectrum of a finite-range translation-invariant operator is contained in the union of the spectra of its Bloch matrices.

```
lemma spectrum_hopping_subset (A : G →₀ Matrix (Fin N) (Fin N) ℂ) :
    spectrum ℂ (hopping π ι A) ⊆
      ⋃ χ : PontryaginDual (Multiplicative G), spectrum ℂ (blochMatrix A χ)
```

def symmetrize [source]

The Hermitian completion of a family of hopping matrices: the amplitude for hopping by `r` is `A r` plus the adjoint of the amplitude for hopping back by `-r`.

```
def symmetrize (A : G →₀ Matrix (Fin N) (Fin N) ℂ) : G →₀ Matrix (Fin N) (Fin N) ℂ
```

lemma symmetrize_apply [source]

```
lemma symmetrize_apply (A : G →₀ Matrix (Fin N) (Fin N) ℂ) (r : G) :
    symmetrize A r = A r + (A (-r))ᴴ
```

lemma conjTranspose_symmetrize [source]

A symmetrized family of hopping matrices satisfies `(A r)ᴴ = A (-r)`.

```
lemma conjTranspose_symmetrize (A : G →₀ Matrix (Fin N) (Fin N) ℂ) (r : G) :
    (symmetrize A r)ᴴ = symmetrize A (-r)
```

lemma isHermitian_blochMatrix [source]

***Hermitian hopping gives Hermitian Bloch matrices.** If `(A r)ᴴ = A (-r)` for every `r`, then the Bloch matrix `∑_r χ(r) A_r` is Hermitian at every crystal momentum `χ`.*

```
lemma isHermitian_blochMatrix (A : G →₀ Matrix (Fin N) (Fin N) ℂ)
    (hA : ∀ r, (A r)ᴴ = A (-r)) (χ : PontryaginDual (Multiplicative G)) :
    (blochMatrix A χ).IsHermitian
```

2.548 PhyslibAlpha.ProbabilisticTheory.Representation.Koopman

PhyslibAlpha/ProbabilisticTheory/Representation/Koopman.lean on GitHub.

def koopmanIsometry [source]

The composition operator `f ↦ f ∘ T_g⁻¹` on `L²(μ)` as a linear isometry.

```
def koopmanIsometry (g : Γ) : Lp E 2 μ →ₗᵢ[ℂ] Lp E 2 μ
```

lemma compMeasurePreserving_congr [source]

```
lemma compMeasurePreserving_congr {f f' : X → X} (hf : MeasurePreserving f μ μ)
    (hf' : MeasurePreserving f' μ μ) (e : f = f') (g : Lp E 2 μ) :
    Lp.compMeasurePreserving f hf g = Lp.compMeasurePreserving f' hf' g
```

lemma koopmanIsometry_apply [source]

```
lemma koopmanIsometry_apply (g : Γ) (f : Lp E 2 μ) :
    koopmanIsometry μ T hT g f = Lp.compMeasurePreserving (T g⁻¹) (hT g⁻¹) f
```

lemma koopmanIsometry_mul [source]

```
lemma koopmanIsometry_mul (g h : Γ) (f : Lp E 2 μ) :
    koopmanIsometry μ T hT (g * h) f = koopmanIsometry μ T hT g (koopmanIsometry μ T hT h f)
```

lemma koopmanIsometry_one [source]

```
lemma koopmanIsometry_one (f : Lp E 2 μ) : koopmanIsometry μ T hT 1 f = f
```

def koopmanEquiv [source]

The Koopman operator as a linear isometric equivalence.

```
def koopmanEquiv (g : Γ) : Lp E 2 μ ≃ₗᵢ[ℂ] Lp E 2 μ where
```

lemma koopmanEquiv_one [source]

```
lemma koopmanEquiv_one : koopmanEquiv (E := E) μ T hT 1 = 1
```

lemma koopmanEquiv_mul [source]

```
lemma koopmanEquiv_mul (g h : Γ) :
    koopmanEquiv (E := E) μ T hT (g * h) =
      koopmanEquiv (E := E) μ T hT g * koopmanEquiv (E := E) μ T hT h
```

def koopman [source]

***The Koopman representation**: a group acting on a measure space by measure-preserving bijections acts unitarily on `L²(μ)` by `(U_g f)(x) = f(g⁻¹ x)`.*

```
def koopman : Γ →* unitary (Lp E 2 μ →L[ℂ] Lp E 2 μ) where
```

lemma koopman_apply [source]

The Koopman operator acts by composition with the inverse of the group action.

```
lemma koopman_apply (g : Γ) (f : Lp E 2 μ) :
    ((koopman μ T hT g : unitary (Lp E 2 μ →L[ℂ] Lp E 2 μ)) : Lp E 2 μ →L[ℂ] Lp E 2 μ) f =
      Lp.compMeasurePreserving (T g⁻¹) (hT g⁻¹) f
```

lemma inner_koopman [source]

***The correlation function of an observable**: the matrix coefficient of the Koopman representation is the covariance $\int \langle f(x), f(g^{-1}\,x)\rangle\, d\mu(x)$ of f with its translate.*

```
lemma inner_koopman (g : Γ) (f : Lp E 2 μ) :
    ⟪f, ((koopman μ T hT g : unitary (Lp E 2 μ →L[ℂ] Lp E 2 μ)) : Lp E 2 μ →L[ℂ] Lp E 2 μ) f⟫_
    ℂ =
      ∫ x, ⟪f x, f (T g⁻¹ x)⟫_ℂ ∂μ
```

lemma range_projection_one_koopman [source]

***Invariant observables of a classical system have zero momentum**: the spectral projection of the trivial character of the Koopman representation projects onto the translation-invariant functions in $L^2(\mu)$.*

```
lemma range_projection_one_koopman :
    LinearMap.range (projection (koopman (E := E) μ T hT)
        {(1 : PontryaginDual (Multiplicative G))} : Lp E 2 μ →L[ℂ] Lp E 2 μ).toLinearMap =
      ProbabilisticTheory.Representation.sector (toEnd (koopman (E := E) μ T hT)) (fun _ =>
        (1 : ℂ))
```

2.549 PhyslibAlpha.ProbabilisticTheory.Representation.MomentumSectors

PhyslibAlpha/ProbabilisticTheory/Representation/MomentumSectors.lean on GitHub.

def sector [source]

The momentum sector of the character χ: vectors on which g acts as the scalar $\chi\ g$.

```
def sector (β : G →* Module.End K V) (χ : G → K) : Submodule K V where
```

lemma mem_sector [source]

```
lemma mem_sector {β : G →* Module.End K V} {χ : G → K} {v : V} :
    v ∈ sector β χ ↔ ∀ g, β g v = χ g • v
```

def cobound [source]

The submodule $\mathrm{span}\ \{\beta\ g\ v - \chi\ g \bullet v\}$ whose quotient is the momentum fiber. It is the "coboundary" submodule of the twisted action.

```
def cobound (β : G →* Module.End K V) (χ : G → K) : Submodule K V
```

lemma sub_smul_mem_cobound [source]

```
lemma sub_smul_mem_cobound (β : G →* Module.End K V) (χ : G → K) (g : G) (v : V) :
    β g v - χ g • v ∈ cobound β χ
```

abbrev Fiber [source]

The momentum fiber of the character χ: V modulo the vectors $\beta\ g\ v - \chi\ g \bullet v$.

```
abbrev Fiber (β : G →* Module.End K V) (χ : G → K) : Type _
```

def mk [source]

The canonical projection `V → V(χ)`.

```
def Fiber.mk (β : G →* Module.End K V) (χ : G → K) : V →ₗ[K] Fiber β χ
```

lemma mk_action [source]

In the fiber, `g` acts as the scalar `χ g`.

```
lemma Fiber.mk_action (β : G →* Module.End K V) (χ : G → K) (g : G) (v : V) :
    Fiber.mk β χ (β g v) = χ g • Fiber.mk β χ v
```

def IsEquivariant [source]

A linear map intertwining two actions of the same monoid.

```
def IsEquivariant (β : G →* Module.End K V) (β' : G →* Module.End K W)
    (T : V →ₗ[K] W) : Prop
```

lemma mapsTo_sector [source]

Every equivariant linear map preserves each momentum sector.

```
lemma IsEquivariant.mapsTo_sector (hT : IsEquivariant β β' T) {v : V}
    (hv : v ∈ sector β χ) : T v ∈ sector β' χ
```

def sectorMap [source]

The restriction of an equivariant map to a momentum sector.

```
def IsEquivariant.sectorMap (hT : IsEquivariant β β' T) (χ : G → K) :
    sector β χ →ₗ[K] sector β' χ
```

lemma coe_sectorMap [source]

```
@[simp]
lemma IsEquivariant.coe_sectorMap (hT : IsEquivariant β β' T) (v : sector β χ) :
    (hT.sectorMap χ v : W) = T v
```

lemma cobound_le_comap [source]

An equivariant map sends the span defining the fiber of `V` into the span defining the fiber of `W`.

```
lemma IsEquivariant.cobound_le_comap (hT : IsEquivariant β β' T) (χ : G → K) :
    cobound β χ ≤ (cobound β' χ).comap T
```

def fiberMap [source]

Every equivariant linear map induces a well-defined linear map on each momentum fiber.

```
def IsEquivariant.fiberMap (hT : IsEquivariant β β' T) (χ : G → K) :
    Fiber β χ →ₗ[K] Fiber β' χ
```

lemma fiberMap_mk [source]

```
@[simp]
lemma IsEquivariant.fiberMap_mk (hT : IsEquivariant β β' T) (χ : G → K) (v : V) :
    hT.fiberMap χ (Fiber.mk β χ v) = Fiber.mk β' χ (T v)
```

lemma fiberMap_comp_mk [source]

The fiber map is the unique map making the square with the projections commute.

```
lemma IsEquivariant.fiberMap_comp_mk (hT : IsEquivariant β β' T) (χ : G → K) :
    (hT.fiberMap χ).comp (Fiber.mk β χ) = (Fiber.mk β' χ).comp T
```

lemma isEquivariant_id [source]

```
lemma isEquivariant_id (β : G →* Module.End K V) : IsEquivariant β β LinearMap.id
```

lemma comp [source]

```
lemma IsEquivariant.comp (hS : IsEquivariant β' β'' S) (hT : IsEquivariant β β' T) :
    IsEquivariant β β'' (S ∘ₗ T)
```

lemma fiberMap_id [source]

```
lemma fiberMap_id (χ : G → K) :
    (isEquivariant_id β).fiberMap χ = LinearMap.id
```

lemma fiberMap_comp [source]

```
lemma IsEquivariant.fiberMap_comp (hS : IsEquivariant β' β'' S) (hT : IsEquivariant β β' T)
    (χ : G → K) :
    (hS.comp hT).fiberMap χ = (hS.fiberMap χ) ∘ₗ (hT.fiberMap χ)
```

lemma mapsTo_sector_self [source]

Equivariant maps commute with the dynamics they generate: if `D` is an equivariant endomorphism, it preserves every sector and descends to every fiber. This is the form used for translation-invariant evolution.

```
lemma IsEquivariant.mapsTo_sector_self {D : V →ₗ[K] V} (hD : IsEquivariant β β D) {v : V}
    (hv : v ∈ sector β χ) : D v ∈ sector β χ
```

def sectorToFiber [source]

The canonical map from the sector of χ to the fiber of χ.

```
def sectorToFiber (β : G →* Module.End K V) (χ : G → K) :
    sector β χ →ₗ[K] Fiber β χ
```

lemma fiberMap_comp_sectorToFiber [source]

The canonical map from sector to fiber is natural for equivariant maps.

```
lemma IsEquivariant.fiberMap_comp_sectorToFiber (hT : IsEquivariant β β' T) (χ : G → K) :
    (hT.fiberMap χ).comp (sectorToFiber β χ)
      = (sectorToFiber β' χ).comp (hT.sectorMap χ)
```

2.550 PhyslibAlpha.ProbabilisticTheory.Representation.Orthogonality

PhyslibAlpha/ProbabilisticTheory/Representation/Orthogonality.lean on GitHub.

def rationalCircle [source]

The injection of $\mathbb{Q}/\mathbb{Z}$ into the circle, $q \mapsto e^{2\pi i q}$.

```
def rationalCircle : AddCircle (1 : ℚ) →+ Additive Circle
```

lemma rationalCircle_eq_zero [source]

```
lemma rationalCircle_eq_zero {x : AddCircle (1 : ℚ)} (h : rationalCircle x = 0) : x = 0
```

lemma exists_character_apply_ne_one [source]

Characters separate the points of a discrete abelian group.

```
lemma exists_character_apply_ne_one [TopologicalSpace G] [DiscreteTopology G] {g : G}
    (hg : g ≠ 0) : ∃ χ : PontryaginDual (Multiplicative G), χ (Multiplicative.ofAdd g) ≠ 1
```

def dualHaar [source]

The normalized Haar measure of the compact dual group.

```
def dualHaar : Measure (PontryaginDual (Multiplicative G))
```

instance (dualHaar (G [source]

```
instance : (dualHaar (G := G)).IsHaarMeasure
```

instance IsProbabilityMeasure (dualHaar (G [source]

```
instance : IsProbabilityMeasure (dualHaar (G := G))
```

lemma integral_character_dualHaar [source]

***Orthogonality of characters**: the evaluation characters are orthonormal for the Haar probability measure of the dual group.*

```
lemma integral_character_dualHaar (g : G) :
    ∫ χ, ((χ (Multiplicative.ofAdd g) : Circle) : ℂ) ∂(dualHaar (G := G)) =
      if g = 0 then 1 else 0
```

2.551 PhyslibAlpha.ProbabilisticTheory.Representation.PointCovariance

PhyslibAlpha/ProbabilisticTheory/Representation/PointCovariance.lean on GitHub.

def autCont [source]

The automorphism `φ p` of the translation group as a continuous homomorphism.

```
def autCont (p : P) : Multiplicative G →ₜ* Multiplicative G where
```

def dualAct [source]

The action of `p` on a character `χ`: `(p • χ)(g) = χ(φ(p⁻¹) g)`.

```
def dualAct (p : P) : PontryaginDual (Multiplicative G) →ₜ* PontryaginDual (Multiplicative G)
```

lemma dualAct_apply [source]

```
lemma dualAct_apply (p : P) (χ : PontryaginDual (Multiplicative G)) (g : Multiplicative G) :
    dualAct φ p χ g = χ (φ p⁻¹ g)
```

lemma dualAct_mul [source]

```
lemma dualAct_mul (p q : P) (χ : PontryaginDual (Multiplicative G)) :
    dualAct φ (p * q) χ = dualAct φ p (dualAct φ q χ)
```

lemma dualAct_one [source]

```
lemma dualAct_one (χ : PontryaginDual (Multiplicative G)) : dualAct φ 1 χ = χ
```

def dualActEquiv [source]

The action of `p` on the characters is a bijection.

```
def dualActEquiv (p : P) :
    PontryaginDual (Multiplicative G) ≃ PontryaginDual (Multiplicative G) where
```

def IsCovariantRep [source]

The representation `ρ` of `P` is covariant with respect to the translations `π` and the action `φ`: `ρ(p) π(g) = π(φ(p) g) ρ(p)`.

```
def IsCovariantRep : Prop
```

lemma measurable_dualAct [source]

```
lemma measurable_dualAct (p : P) : Measurable (dualAct φ p)
```

lemma isCovariantRep_comp [source]

A representation of a group `Q` acting on the translations, restricted to the translations, is covariant when the group relation `s t(a) = t(φ(s) a) s` holds.

```
lemma isCovariantRep_comp {Q : Type*} [Group Q] (φ : Q →* MulAut (Multiplicative G))
    (U : Q →* unitary (H →L[ℂ] H)) (τ : Multiplicative G →* Q)
    (h : ∀ (s : Q) (a : Multiplicative G), s * τ a = τ (φ s a) * s) :
    IsCovariantRep φ (U.comp τ) U
```

lemma inner_rho_translation [source]

The matrix coefficient of the translated vector.

```
lemma inner_rho_translation (hcov : IsCovariantRep φ π ρ) (p : P) (g : Multiplicative G)
    (ξ : H) :
    ⟪((ρ p : unitary (H →L[ℂ] H)) : H →L[ℂ] H) ξ,
        ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) (((ρ p : unitary (H →L[ℂ] H)) : H →L[ℂ] H) ξ)
    ⟫_ℂ
      = ⟪ξ, ((π (φ p⁻¹ g) : unitary (H →L[ℂ] H)) : H →L[ℂ] H) ξ⟫_ℂ
```

lemma vectorMeasure_rho [source]

***The spectral measure of `ρ(p) ξ` is the pushforward of that of `ξ`** under the action of `p` on the characters.*

```
lemma vectorMeasure_rho (hcov : IsCovariantRep φ π ρ) (p : P) (ξ : H) :
    vectorMeasure π (((ρ p : unitary (H →L[ℂ] H)) : H →L[ℂ] H) ξ) =
      Measure.map (dualAct φ p) (vectorMeasure π ξ)
```

lemma norm_sq_projection [source]

```
lemma norm_sq_projection (S : Set (PontryaginDual (Multiplicative G))) (hS : MeasurableSet S)
    (η : H) : ‖projection π S η‖ ^ 2 = (vectorMeasure π η S).toReal
```

lemma inner_projection_self [source]

```
lemma inner_projection_self (S : Set (PontryaginDual (Multiplicative G))) (η : H) :
    ⟪projection π S η, η⟫_ℂ = ((‖projection π S η‖ ^ 2 : ℝ) : ℂ)
```

lemma rho_mul_projection [source]

***The spectral projections are covariant**: `ρ(p) P(S) = P(p • S) ρ(p)` for every measurable set `S` of characters. The unitary `ρ(p)` carries the sector of momenta in `S` onto the sector of momenta in `p • S`.*

```
lemma rho_mul_projection (hcov : IsCovariantRep φ π ρ) (p : P)
    {S : Set (PontryaginDual (Multiplicative G))} (hS : MeasurableSet S) :
    ((ρ p : unitary (H →L[ℂ] H)) : H →L[ℂ] H) * projection π S =
      projection π (dualAct φ p '' S) * ((ρ p : unitary (H →L[ℂ] H)) : H →L[ℂ] H)
```

2.552 PhyslibAlpha.ProbabilisticTheory.Representation.SectorReduction

PhyslibAlpha/ProbabilisticTheory/Representation/SectorReduction.lean on GitHub.

def sector [source]

The sector of a set of characters: the vectors of total momentum in the set, the range of the spectral projection.

```
def sector (S : Set (PontryaginDual (Multiplicative G))) : Submodule ℂ H
```

lemma mem_sector_iff [source]

```
lemma mem_sector_iff (S : Set (PontryaginDual (Multiplicative G))) (x : H) :
    x ∈ sector π S ↔ ∃ y, projection π S y = x
```

lemma dualAct_image_image [source]

```
lemma dualAct_image_image (p : P) (S : Set (PontryaginDual (Multiplicative G))) :
    dualAct φ p⁻¹ '' (dualAct φ p '' S) = S
```

lemma measurableSet_dualAct_image [source]

```
lemma measurableSet_dualAct_image (p : P) {S : Set (PontryaginDual (Multiplicative G))}
    (hS : MeasurableSet S) : MeasurableSet (dualAct φ p '' S)
```

lemma rho_inv_rho [source]

```
lemma rho_inv_rho (p : P) (x : H) : (ℛ p⁻¹) ((ℛ p) x) = x
```

lemma rho_rho_inv [source]

```
lemma rho_rho_inv (p : P) (x : H) : (ℛ p) ((ℛ p⁻¹) x) = x
```

lemma norm_rho [source]

```
lemma norm_rho (p : P) (x : H) : ‖(ℛ p) x‖ = ‖x‖
```

lemma rho_projection_apply [source]

```
lemma rho_projection_apply (p : P) {S : Set (PontryaginDual (Multiplicative G))}
    (hS : MeasurableSet S) (y : H) :
    (ℛ p) (projection π S y) = projection π (dualAct φ p '' S) ((ℛ p) y)
```

lemma rho_mem_sector [source]

```
lemma rho_mem_sector (p : P) {S : Set (PontryaginDual (Multiplicative G))}
    (hS : MeasurableSet S) {x : H} (hx : x ∈ sector π S) :
    (ℛ p) x ∈ sector π (dualAct φ p '' S)
```

lemma rho_inv_mem_sector [source]

```
lemma rho_inv_mem_sector (p : P) {S : Set (PontryaginDual (Multiplicative G))}
    (hS : MeasurableSet S) {y : H} (hy : y ∈ sector π (dualAct φ p '' S)) :
    (ℛ p⁻¹) y ∈ sector π S
```

def sectorEquiv [source]

***The equivalence of sectors**: `ρ(p)` is an isometric equivalence of the sector of `S` onto the sector of `p • S`.*

```
def sectorEquiv (p : P) {S : Set (PontryaginDual (Multiplicative G))} (hS : MeasurableSet S) :
    sector π S ≃ₗᵢ[ℂ] sector π (dualAct φ p '' S) where
```

lemma coe_sectorEquiv [source]

```
lemma coe_sectorEquiv (p : P) {S : Set (PontryaginDual (Multiplicative G))}
    (hS : MeasurableSet S) (x : sector π S) :
    ((sectorEquiv hcov p hS x : sector π (dualAct φ p '' S)) : H) = (ℛ p) x.1
```

def sectorOp [source]

An operator commuting with the translations restricts to each sector.

```
def sectorOp (hT : ∀ g, Commute ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) T)
    (S : Set (PontryaginDual (Multiplicative G))) : Module.End ℂ (sector π S)
```

lemma coe_sectorOp [source]

```
lemma coe_sectorOp (hT : ∀ g, Commute ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) T)
    (S : Set (PontryaginDual (Multiplicative G))) (x : sector π S) :
    ((sectorOp hT S x : sector π S) : H) = T x.1
```

lemma sectorEquiv_sectorOp [source]

The equivalence of sectors intertwines the restrictions of an invariant operator.

```
lemma sectorEquiv_sectorOp (p : P) {S : Set (PontryaginDual (Multiplicative G))}
    (hS : MeasurableSet S) (x : sector π S) :
    sectorEquiv hcov p hS (sectorOp hTπ S x) = sectorOp hTπ (dualAct φ p '' S)
      (sectorEquiv hcov p hS x)
```

lemma spectrum_sectorOp_eq [source]

***Symmetry reduction.** The spectrum of an operator invariant under the translations and the covariant point group, restricted to the sector of the characters in `S`, is its spectrum on the sector of the characters in `p • S`.*

```
lemma spectrum_sectorOp_eq (p : P) {S : Set (PontryaginDual (Multiplicative G))}
    (hS : MeasurableSet S) :
    spectrum ℂ (sectorOp hTπ S) = spectrum ℂ (sectorOp hTπ (dualAct φ p '' S))
```

2.553 PhyslibAlpha.ProbabilisticTheory.Representation.StructureFactor

PhyslibAlpha/ProbabilisticTheory/Representation/StructureFactor.lean on GitHub.

def correlation [source]

The correlation function `g ↦ ω(a⋆ α_g(a))` of an observable `a`.

```
def correlation (a : B) (g : G) : ℂ
```

lemma isPositiveDefiniteSub_correlation [source]

The correlation function of an invariant positive functional is positive definite.

```
lemma isPositiveDefiniteSub_correlation (hpos : ∀ b : B, 0 ≤ ω (star b * b))
    (hinv : ∀ g b, ω (α g b) = ω b) (a : B) :
    TauCeti.IsPositiveDefiniteSub (correlation ω α a)
```

lemma exists_measure [source]

***The structure factor exists.** The correlation function of an observable in an invariant positive functional is the Fourier transform of a finite positive measure on the momenta: `ω(a⋆ α_g(a)) = ∫ χ(g) dμ_a(χ)`.*

```
lemma exists_measure (hpos : ∀ b : B, 0 ≤ ω (star b * b)) (hinv : ∀ g b, ω (α g b) = ω b)
    (a : B) :
    ∃ μ : Measure (PontryaginDual (Multiplicative G)), IsFiniteMeasure μ ∧ μ.InnerRegular ∧
      ∀ g : G, ∫ χ, ((χ (Multiplicative.ofAdd g) : Circle) : ℂ) ∂μ = correlation ω α a g
```

lemma integral_one_eq [source]

The total mass of the structure factor is the expectation of `a⋆ a`.

```
lemma integral_one_eq {μ : Measure (PontryaginDual (Multiplicative G))}
    (a : B) (h : ∀ g : G, ∫ χ, ((χ (Multiplicative.ofAdd g) : Circle) : ℂ) ∂μ =
      correlation ω α a g) :
    (μ.real univ : ℂ) = ω (star a * a)
```

lemma eq_of_transform_eq [source]

***The structure factor is unique.** For a countable group a finite measure on the momenta is determined by its Fourier transform.*

```
lemma eq_of_transform_eq {μ ν : Measure (PontryaginDual (Multiplicative G))}
    [IsFiniteMeasure μ] [IsFiniteMeasure ν]
    (h : ∀ g : G, ∫ χ, ((χ (Multiplicative.ofAdd g) : Circle) : ℂ) ∂μ =
      ∫ χ, ((χ (Multiplicative.ofAdd g) : Circle) : ℂ) ∂ν) : μ = ν
```

lemma isPositiveDefiniteSub_iff_exists_measure [source]

***Positive semidefinite translation-invariant couplings are Fourier transforms of positive measures.** A coupling `J(r)` depending only on the displacement `r` is positive semidefinite, in the sense that every finite quadratic form `∑ c_i c¯_j J(r_i - r_j)` is nonnegative, if and only if it is the Fourier transform of a finite positive measure on the momenta, whose density is the dispersion of the coupling.*

```
lemma isPositiveDefiniteSub_iff_exists_measure (J : G → ℂ) :
    TauCeti.IsPositiveDefiniteSub J ↔
      ∃ μ : Measure (PontryaginDual (Multiplicative G)), IsFiniteMeasure μ ∧ μ.InnerRegular ∧
        ∀ r : G, J r = ∫ χ, ((χ (Multiplicative.ofAdd r) : Circle) : ℂ) ∂μ
```

2.554 PhyslibAlpha.ProbabilisticTheory.Representation.VectorMeasure

PhyslibAlpha/ProbabilisticTheory/Representation/VectorMeasure.lean on GitHub.

def vectorMeasure [source]

The spectral measure `S ↦ ‖P(S) ξ‖²` of a vector `ξ`, a finite measure on the Pontryagin dual: the distribution of `ξ` over the characters of the group.

```
def vectorMeasure (ξ : H) : Measure (PontryaginDual (Multiplicative G))
```

instance IsFiniteMeasure (vectorMeasure π ξ) [source]

```
instance (ξ : H) : IsFiniteMeasure (vectorMeasure π ξ)
```

lemma vectorMeasure_apply [source]

```
lemma vectorMeasure_apply (ξ : H) {S : Set (PontryaginDual (Multiplicative G))}
    (hS : MeasurableSet S) :
    vectorMeasure π ξ S = ENNReal.ofReal (‖spectralMeasure π S ξ‖ ^ 2)
```

lemma vectorMeasure_univ [source]

The total mass of the spectral measure of a vector is its squared norm.

```
lemma vectorMeasure_univ (ξ : H) : vectorMeasure π ξ univ = ENNReal.ofReal (‖ξ‖ ^ 2)
```

lemma vectorMeasure_eq_map [source]

The vector measure is the pushforward of the Riesz measure of the vector state.

```
lemma vectorMeasure_eq_map (ξ : H) :
    vectorMeasure π ξ = Measure.map (dual π) (starHomScalarMeasure (family π).starHom ξ)
```

lemma inner_eq_integral_vectorMeasure [source]

***The spectral measure of a vector represents its matrix coefficients**: ⟪ξ, π(g) ξ⟫ = ∫ χ(g) dμ_ξ(χ).*

```
lemma inner_eq_integral_vectorMeasure (ξ : H) (g : Multiplicative G) :
    ⟪ξ, ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) ξ⟫_ℂ =
      ∫ χ, ((χ g : Circle) : ℂ) ∂vectorMeasure π ξ
```

lemma eq_vectorMeasure_of_transform_eq [source]

***Uniqueness of the spectral measure of a vector.** For a countable group, a finite measure on the Pontryagin dual whose Fourier–Stieltjes transform is g ↦ ⟪ξ, π(g) ξ⟫ is the spectral measure of ξ.*

```
lemma eq_vectorMeasure_of_transform_eq (ξ : H) (μ : Measure (PontryaginDual (Multiplicative G))
    )
    [IsFiniteMeasure μ]
    (h : ∀ g : Multiplicative G, ∫ χ, ((χ g : Circle) : ℂ) ∂μ =
      ⟪ξ, ((π g : unitary (H →L[ℂ] H)) : H →L[ℂ] H) ξ⟫_ℂ) :
    μ = vectorMeasure π ξ
```

2.555 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.ComplexExtension

PhyslibAlpha/ProbabilisticTheory/StarAlgebra/ComplexExtension.lean on GitHub.

def complexExtension [source]

The complex-linear extension of an expectation value on self-adjoint operators to all operators: ⟨a⟩ = ⟨ℜ a⟩ + i ⟨ℑ a⟩.

```
noncomputable def complexExtension (φ : selfAdjoint A →ₗ[ℝ] ℝ) : A →ₗ[ℂ] ℂ where
```

lemma complexExtension_apply [source]

```
lemma complexExtension_apply (φ : selfAdjoint A →ₗ[ℝ] ℝ) (a : A) :
    complexExtension φ a = φ (ℜ a) + Complex.I * φ (ℑ a)
```

lemma complexExtension_coe [source]

On self-adjoint operators the extension is the original expectation value.

```
lemma complexExtension_coe (φ : selfAdjoint A →ₗ[ℝ] ℝ) (a : selfAdjoint A) :
    complexExtension φ (a : A) = φ a
```

lemma complexExtension_star [source]

The expectation value of the adjoint is the complex conjugate of the expectation value.

```
lemma complexExtension_star (φ : selfAdjoint A →ₗ[ℝ] ℝ) (a : A) :
    complexExtension φ (star a) = starRingEnd ℂ (complexExtension φ a)
```

lemma realPart_linearMap [source]

A complex-linear map commuting with the adjoint maps the real part of an operator to the real part of its image.

```
lemma realPart_linearMap (f : C →ₗ[ℂ] D) (hf : ∀ a, f (star a) = star (f a)) (a : C) :
    ((ℜ (f a) : selfAdjoint D) : D) = f (ℜ a : C)
```

lemma imaginaryPart_linearMap [source]

A complex-linear map commuting with the adjoint maps the imaginary part of an operator to the imaginary part of its image.

```
lemma imaginaryPart_linearMap (f : C →ₗ[ℂ] D) (hf : ∀ a, f (star a) = star (f a)) (a : C) :
    ((ℑ (f a) : selfAdjoint D) : D) = f (ℑ a : C)
```

2.556 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.CovarianceForm

PhyslibAlpha/ProbabilisticTheory/StarAlgebra/CovarianceForm.lean on GitHub.

def fluctuationMap [source]

The fluctuation `a - ω(a)` of an element, as a linear map.

```
def fluctuationMap : A →ₗ[ℂ] A where
```

lemma fluctuationMap_apply [source]

```
lemma fluctuationMap_apply (a : A) : ω.fluctuationMap a = a - ω a • 1
```

def covarianceForm [source]

The covariance form `C(a, b) = ω(a⋆ b) - conj (ω a) ω(b)`, the inner product of the fluctuations in the pre-Hilbert space of the state.

```
noncomputable def covarianceForm (a b : A) : ℂ
```

lemma covarianceForm_apply [source]

```
lemma covarianceForm_apply (a b : A) :
    ω.covarianceForm a b = ω (star a * b) - star (ω a) * ω b
```

lemma covarianceForm_smul_left [source]

```
lemma covarianceForm_smul_left (c : ℂ) (a b : A) :
    ω.covarianceForm (c • a) b = star c * ω.covarianceForm a b
```

lemma covarianceForm_smul_right [source]

```
lemma covarianceForm_smul_right (c : ℂ) (a b : A) :
    ω.covarianceForm a (c • b) = c * ω.covarianceForm a b
```

lemma covarianceForm_sum_left [source]

```
lemma covarianceForm_sum_left {ι : Type*} (s : Finset ι) (a : ι → A) (b : A) :
    ω.covarianceForm (∑ i ∈ s, a i) b = ∑ i ∈ s, ω.covarianceForm (a i) b
```

lemma covarianceForm_sum_right [source]

```
lemma covarianceForm_sum_right {ι : Type*} (s : Finset ι) (a : A) (b : ι → A) :
    ω.covarianceForm a (∑ i ∈ s, b i) = ∑ i ∈ s, ω.covarianceForm a (b i)
```

lemma covarianceForm_selfAdjoint [source]

For observables, `C(a, b) = Cov(a, b) + i ω⁅a, b⁆`.

```
lemma covarianceForm_selfAdjoint (a b : selfAdjoint A) :
    ω.covarianceForm a b = ω.covariance a b + Complex.I * ω.bracket a b
```

lemma covarianceForm_map [source]

***A symmetry of the state preserves the covariance form**: `C(σ a, σ b) = C(a, b)`.*

```
lemma covarianceForm_map (a b : A) : ω.covarianceForm (σ a) (σ b) = ω.covarianceForm a b
```

lemma covarianceForm_eq_zero_of_smul [source]

Elements multiplied by `χ` and `χ'` under a symmetry, with `conj χ · χ' ≠ 1`, have covariance zero.

```
lemma covarianceForm_eq_zero_of_smul {a b : A} {χ χ' : ℂ} (ha : σ a = χ • a) (hb : σ b = χ' • b
    )
    (h : star χ * χ' ≠ 1) : ω.covarianceForm a b = 0
```

lemma covarianceForm_eq_zero_of_neg [source]

An even and an odd element have covariance zero.

```
lemma covarianceForm_eq_zero_of_neg {a b : A} (ha : σ a = a) (hb : σ b = -b) :
    ω.covarianceForm a b = 0
```

lemma covarianceForm_eq_zero_of_character [source]

Elements transforming with different unitary characters have covariance zero. *If `α g a = χ g • a` and `α g b = χ' g • b` with `|χ g| = 1` and `χ ≠ χ'`, then `C(a, b) = 0`.*

```
lemma covarianceForm_eq_zero_of_character {a b : A} {χ χ' : Γ → ℂ} (hχ : ∀ g, ‖χ g‖ = 1)
    (ha : ∀ g, α g a = χ g • a) (hb : ∀ g, α g b = χ' g • b) (hne : χ ≠ χ') :
    ω.covarianceForm a b = 0
```

def covarianceMatrixForm [source]

The matrix `M_ik = C(a_i, b_k)` of covariances of two families.

```
noncomputable def covarianceMatrixForm (a : ι → A) (b : κ → A) : Matrix ι κ ℂ
```

lemma posSemidef_covarianceMatrixForm [source]

***The matrix of covariances of a family is positive semidefinite**, being a Gram matrix.*

```
lemma posSemidef_covarianceMatrixForm (a : ι → A) : (ω.covarianceMatrixForm a a).PosSemidef
```

lemma conjTranspose_mul_covarianceMatrixForm_mul [source]

***A symmetry of the state preserves the matrix of covariances**: if `σ` acts on the two families by the matrices `R` and `S`, then `R^H M S = M`.*

```
lemma conjTranspose_mul_covarianceMatrixForm_mul (σ : A ≃⋆ₐ[ℂ] A) (hω : ∀ x, ω (σ x) = ω x)
    {a : ι → A} {b : κ → A} {R : Matrix ι ι ℂ} {S : Matrix κ κ ℂ}
    (ha : ∀ i, σ (a i) = ∑ j, R j i • a j) (hb : ∀ k, σ (b k) = ∑ l, S l k • b l) :
    R^H * ω.covarianceMatrixForm a b * S = ω.covarianceMatrixForm a b
```

lemma covarianceMatrixForm_isIntertwining [source]

If `α` acts on the families `a` and `b` by `ρ` and `π`, with `ρ` unitary, then the matrix of covariances intertwines `π` with `ρ`.

```
lemma covarianceMatrixForm_isIntertwining {a : ι → A} {b : κ → A}
    (hρ : ∀ g, LinearMap.toMatrix' (ρ g) ∈ unitaryGroup ι ℂ)
    (ha : ∀ g i, α g (a i) = ∑ j, LinearMap.toMatrix' (ρ g) j i • a j)
    (hb : ∀ g k, α g (b k) = ∑ l, LinearMap.toMatrix' (π g) l k • b l) (g : Γ) :
    toLin' (ω.covarianceMatrixForm a b) ∘ₗ π g =
      ρ g ∘ₗ toLin' (ω.covarianceMatrixForm a b)
```

lemma covarianceMatrixForm_eq_zero_of_irreducible [source]

***Schur's lemma for covariances.** If a group of symmetries of the state acts on the family `a` by the irreducible unitary representation `ρ` and on the family `b` by the irreducible representation `π`, and `π` and `ρ` are not equivalent, then `C(a_i, b_k) = 0` for all `i, k`.*

```
lemma covarianceMatrixForm_eq_zero_of_irreducible [ρ.IsIrreducible] [π.IsIrreducible]
    [IsEmpty (π.Equiv ρ)] {a : ι → A} {b : κ → A}
    (hρ : ∀ g, LinearMap.toMatrix' (ρ g) ∈ unitaryGroup ι ℂ)
    (ha : ∀ g i, α g (a i) = ∑ j, LinearMap.toMatrix' (ρ g) j i • a j)
    (hb : ∀ g k, α g (b k) = ∑ l, LinearMap.toMatrix' (π g) l k • b l) :
    ω.covarianceMatrixForm a b = 0
```

2.557 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.CovarianceFormUnitalMap

PhyslibAlpha/ProbabilisticTheory/StarAlgebra/CovarianceFormUnitalMap.lean on GitHub.

lemma apply_nonneg [source]

On a star-ordered ring a state is nonnegative on every nonnegative element.

```
lemma apply_nonneg {a : A} (ha : 0 ≤ a) : 0 ≤ ω a
```

lemma star_dotProduct_covarianceMatrixForm_mulVec [source]

The quadratic form of the matrix of covariances at `c` is the covariance form of $\sum c_j a_j$ with itself.

```
lemma star_dotProduct_covarianceMatrixForm_mulVec {ι : Type*} [Fintype ι] (ω : AlgState A)
    (a : ι → A) (c : ι → ℂ) :
    star c ⬝ᵥ (ω.covarianceMatrixForm a a *ᵥ c) =
      ω.covarianceForm (∑ j, c j • a j) (∑ j, c j • a j)
```

lemma covarianceForm_sub_covarianceForm_map [source]

Pulling the observable through `Φ` and pushing the state differ by the value of `ω` on `Φ(X⋆ X) - Φ(X)⋆ Φ(X)`.

```
lemma covarianceForm_sub_covarianceForm_map (hω' : ∀ x, ω' x = ω (Φ x)) (X : A) :
    ω'.covarianceForm X X - ω.covarianceForm (Φ X) (Φ X) =
      ω (Φ (star X * X) - star (Φ X) * Φ X)
```

lemma covarianceForm_map_le [source]

***A Schwarz inequality for `Φ` bounds the covariance form**: if `Φ(X)⋆ Φ(X) ≤ Φ(X⋆ X)`, then `C_ω(Φ X, Φ X) ≤ C_ω'(X, X)`.*

```
lemma covarianceForm_map_le (hω' : ∀ x, ω' x = ω (Φ x)) {X : A}
    (h : star (Φ X) * Φ X ≤ Φ (star X * X)) :
    ω.covarianceForm (Φ X) (Φ X) ≤ ω'.covarianceForm X X
```

lemma covarianceMatrixForm_map_le [source]

Maps satisfying the Schwarz inequality do not decrease the matrix of covariances. *The hypothesis holds for completely positive unital maps.*

```
lemma covarianceMatrixForm_map_le {ι : Type*} [Fintype ι] (hω' : ∀ x, ω' x = ω (Φ x))
    (hΦ : ∀ X, star (Φ X) * Φ X ≤ Φ (star X * X)) (a : ι → A) (c : ι → ℂ) :
    star c ⬝ᵥ (ω.covarianceMatrixForm (Φ ∘ a) (Φ ∘ a) *ᵥ c) ≤
      star c ⬝ᵥ (ω'.covarianceMatrixForm a a *ᵥ c)
```

lemma sq_sum_le_sum_sq [source]

With positive Q_k summing to 1, $(\sum \lambda_k\ Q_k)^2 \le \sum \lambda_k^2\ Q_k$.

```
lemma sq_sum_le_sum_sq {ι : Type*} [Fintype ι] (Q : ι → Matrix m m ℂ)
    (hQ : ∀ k, (Q k).PosSemidef) (h1 : ∑ k, Q k = 1) (c : ι → ℝ) :
    (∑ k, (c k : ℂ) • Q k) * ∑ k, (c k : ℂ) • Q k ≤ ∑ k, ((c k : ℂ) * c k) • Q k
```

lemma kadison [source]

***Kadison's inequality**: for a positive unital map and Hermitian A, $\Phi(A)^2 \le \Phi(A^2)$.*

```
lemma kadison (Φ : Matrix n n ℂ →ₚ ₗ[ℂ] Matrix m m ℂ) {A : Matrix n n ℂ}
    (hA : A.IsHermitian) : (Φ A)^H * Φ A ≤ Φ (A^H * A)
```

lemma covarianceForm_map_le_of_positive [source]

Positive unital maps do not decrease the covariance form of Hermitian matrices.

```
lemma covarianceForm_map_le_of_positive (hω' : ∀ x, ω' x = ω (Φ x)) {X : Matrix n n ℂ}
    (hX : X.IsHermitian) : ω.covarianceForm (Φ X) (Φ X) ≤ ω'.covarianceForm X X
```

lemma covarianceMatrixForm_map_le_of_positive [source]

***Positive unital maps do not decrease the matrix of covariances** of Hermitian matrices at real coefficients.*

```
lemma covarianceMatrixForm_map_le_of_positive {ι : Type*} [Fintype ι]
    (hω' : ∀ x, ω' x = ω (Φ x)) {a : ι → Matrix n n ℂ} (ha : ∀ i, (a i).IsHermitian)
    (c : ι → ℝ) :
    star (fun i => (c i : ℂ)) ⬝ᵥ (ω.covarianceMatrixForm (Φ ∘ a) (Φ ∘ a) *ᵥ fun i => (c i : ℂ))
     ≤
      star (fun i => (c i : ℂ)) ⬝ᵥ (ω'.covarianceMatrixForm a a *ᵥ fun i => (c i : ℂ))
```

def stateZero [source]

The state $X \mapsto X_{00}$ of 2×2 matrices, the vector state of $|0\rangle$.

```
def stateZero : ProbabilisticTheory.AlgState (Matrix (Fin 2) (Fin 2) ℂ) where
```

lemma stateZero_apply [source]

```
@[simp]
lemma stateZero_apply (X : Matrix (Fin 2) (Fin 2) ℂ) : stateZero X = X 0 0
```

def transposeMap [source]

The transpose of 2×2 matrices.

```
noncomputable def transposeMap : Matrix (Fin 2) (Fin 2) ℂ →ₚ₁[ℂ] Matrix (Fin 2) (Fin 2) ℂ
```

lemma transposeMap_apply [source]

```
@[simp]
lemma transposeMap_apply (X : Matrix (Fin 2) (Fin 2) ℂ) : transposeMap X = X^T
```

lemma stateZero_transposeMap [source]

The state $X \mapsto X_{00}$ is unchanged by the transpose.

```
lemma stateZero_transposeMap (X : Matrix (Fin 2) (Fin 2) ℂ) :
    stateZero X = stateZero (transposeMap X)
```

lemma transposeMap_gap [source]

***The transpose is positive and unital, but the bound fails for E_{01}**: for positive maps it holds only on Hermitian matrices.*

```
lemma transposeMap_gap :
    ¬ stateZero.covarianceForm (transposeMap (single 0 1 1)) (transposeMap (single 0 1 1)) ≤
      stateZero.covarianceForm (single 0 1 1) (single 0 1 1)
```

2.558 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.GNS

PhyslibAlpha/ProbabilisticTheory/StarAlgebra/GNS.lean on GitHub.

structure AlgState [source]

*A state on a unital complex *-algebra: a normalized, Hermitian, positive linear functional.*

```
structure AlgState (A : Type*) [Ring A] [Algebra ℂ A] [StarRing A] [StarModule ℂ A] where
  toLinearMap : A →ₗ[ℂ] ℂ
  map_one' : toLinearMap 1 = 1
  map_star' : ∀ a, toLinearMap (star a) = star (toLinearMap a)
  nonneg' : ∀ a, 0 ≤ toLinearMap (star a * a)
```

instance FunLike (AlgState A) A ℂ [source]

```
instance : FunLike (AlgState A) A ℂ where
```

lemma map_one [source]

```
lemma map_one : ω 1 = 1
```

lemma map_star [source]

```
lemma map_star (a : A) : ω (star a) = star (ω a)
```

lemma nonneg [source]

```
lemma nonneg (a : A) : 0 ≤ ω (star a * a)
```

lemma map_add [source]

```
lemma map_add (a b : A) : ω (a + b) = ω a + ω b
```

lemma map_smul [source]

```
lemma map_smul (c : ℂ) (a : A) : ω (c • a) = c * ω a
```

def PreGNS [source]

The algebra with the inner product induced by a state, before completion.

```
@[nolint unusedArguments]
def PreGNS (_ω : AlgState A)
```

instance AddCommGroup ω.PreGNS [source]

```
instance : AddCommGroup ω.PreGNS
```

instance Module ℂ ω.PreGNS [source]

```
instance : Module ℂ ω.PreGNS
```

def toPre [source]

The identification of the algebra with its pre-Hilbert space.

```
def toPre : A ≃ₗ[ℂ] ω.PreGNS
```

def ofPre [source]

The identification of the pre-Hilbert space with the algebra.

```
def ofPre : ω.PreGNS ≃ₗ[ℂ] A
```

abbrev core [source]

The inner product `⟪a, b⟫ = ω (a* b)`.

```
noncomputable abbrev core : PreInnerProductSpace.Core ℂ ω.PreGNS where
```

instance SeminormedAddCommGroup ω.PreGNS [source]

```
noncomputable instance : SeminormedAddCommGroup ω.PreGNS
```

instance InnerProductSpace ℂ ω.PreGNS [source]

```
noncomputable instance : InnerProductSpace ℂ ω.PreGNS
```

lemma inner_pre [source]

```
lemma inner_pre (a b : ω.PreGNS) : ⟪a, b⟫_ℂ = ω (star (ω.ofPre a) * ω.ofPre b)
```

abbrev GNS [source]

*The **GNS Hilbert space** of a state: the completion of the algebra for the inner product `⟪a, b⟫ = ω (a* b)`.*

```
abbrev GNS : Type _
```

def preAut [source]

The map `[a] ↦ [σ a]` on the pre-Hilbert space.

```
def preAut : ω.PreGNS →ₗ[ℂ] ω.PreGNS
```

lemma ofPre_preAut [source]

```
lemma ofPre_preAut (x : ω.PreGNS) : ω.ofPre (preAut ω σ x) = σ (ω.ofPre x)
```

lemma inner_preAut [source]

```
lemma inner_preAut (hσ : ∀ a, ω (σ a) = ω a) (x y : ω.PreGNS) :
    ⟪preAut ω σ x, preAut ω σ y⟫_ℂ = ⟪x, y⟫_ℂ
```

lemma norm_preAut [source]

```
lemma norm_preAut (hσ : ∀ a, ω (σ a) = ω a) (x : ω.PreGNS) : ‖preAut ω σ x‖ = ‖x‖
```

def preAutCLM [source]

The automorphism as a bounded operator on the pre-Hilbert space.

```
def preAutCLM (hσ : ∀ a, ω (σ a) = ω a) : ω.PreGNS →L[ℂ] ω.PreGNS
```

def gnsAut [source]

The operator on the GNS space implementing a state preserving automorphism.

```
def gnsAut (hσ : ∀ a, ω (σ a) = ω a) : ω.GNS →L[ℂ] ω.GNS
```

lemma gnsAut_coe [source]

```
lemma gnsAut_coe (hσ : ∀ a, ω (σ a) = ω a) (x : ω.PreGNS) :
    gnsAut σ hσ (x : ω.GNS) = ((preAut ω σ x : ω.PreGNS) : ω.GNS)
```

lemma inner_gnsAut [source]

```
lemma inner_gnsAut (hσ : ∀ a, ω (σ a) = ω a) (x y : ω.GNS) :
    ⟪gnsAut σ hσ x, gnsAut σ hσ y⟫_ℂ = ⟪x, y⟫_ℂ
```

lemma gnsAut_mul [source]

```
lemma gnsAut_mul (g h : Γ) :
    gnsAut (α (g * h)) (hα (g * h)) = gnsAut (α g) (hα g) * gnsAut (α h) (hα h)
```

lemma gnsAut_one [source]

```
lemma gnsAut_one : gnsAut (α 1) (hα 1) = 1
```

lemma star_gnsAut_mul_gnsAut [source]

```
lemma star_gnsAut_mul_gnsAut (g : Γ) :
    star (gnsAut (α g) (hα g)) * gnsAut (α g) (hα g) = 1
```

lemma star_gnsAut [source]

```
lemma star_gnsAut (g : Γ) : star (gnsAut (α g) (hα g)) = gnsAut (α g⁻¹) (hα g⁻¹)
```

def gnsUnitary [source]

***The unitary operator on the GNS space implementing a symmetry** of the state.*

```
def gnsUnitary (g : Γ) : unitary (ω.GNS →L[ℂ] ω.GNS)
```

def gnsRep [source]

The unitary representation of a group of symmetries on the GNS space of an invariant state.

```
def gnsRep : Γ →* unitary (ω.GNS →L[ℂ] ω.GNS) where
```

def cyclicVector [source]

The cyclic vector: the class of the unit.

```
def cyclicVector : ω.GNS
```

lemma gnsAut_comp [source]

The operator implementing a composite of automorphisms is the composite of the operators.

```
lemma gnsAut_comp :
    gnsAut (σ * τ) (fun a => by rw [StarAlgEquiv.mul_apply, hσ, hτ]) =
      gnsAut σ hσ * gnsAut τ hτ
```

lemma gnsAut_congr [source]

Equal automorphisms are implemented by equal operators.

```
lemma gnsAut_congr {σ τ : A ≃⋆ₐ[ℂ] A} (h : σ = τ) (hσ : ∀ a, ω (σ a) = ω a)
    (hτ : ∀ a, ω (τ a) = ω a) : gnsAut σ hσ = gnsAut τ hτ
```

lemma gnsAut_commute [source]

Commuting automorphisms are implemented by commuting operators.

```
lemma gnsAut_commute {σ τ : A ≃⋆ₐ[ℂ] A} (hσ : ∀ a, ω (σ a) = ω a) (hτ : ∀ a, ω (τ a) = ω a)
    (h : σ * τ = τ * σ) : Commute (gnsAut σ hσ) (gnsAut τ hτ)
```

2.559 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.Invariant

PhyslibAlpha/ProbabilisticTheory/StarAlgebra/Invariant.lean on GitHub.

def IsBounded [source]

*An observable `a` is **bounded** if some multiple `c · 1` of the identity exceeds `a* a` by a sum of squares.*

```
def IsBounded (a : A) : Prop
```

lemma norm_sq_le_of_isBounded [source]

***Every state is bounded on a bounded observable**, by the same constant.*

```
lemma AlgState.norm_sq_le_of_isBounded {a : A} {c : ℝ} {p : A} (hp : p ∈ sumSq A)
    (h : star a * a + p = (c : ℂ) • (1 : A)) (ω : AlgState A) : ‖ω a‖ ^ 2 ≤ c
```

lemma sumSq_map [source]

A star-algebra equivalence maps sums of squares to sums of squares.

```
lemma sumSq_map {B : Type*} [Ring B] [Algebra ℂ B] [StarRing B]
    (σ : A ≃⋆ₐ[ℂ] B) {p : A} (hp : p ∈ sumSq A) : σ p ∈ sumSq B
```

lemma map [source]

A star-algebra automorphism preserves boundedness.

```
lemma IsBounded.map {a : A} (ha : IsBounded a) (σ : A ≃⋆ₐ[ℂ] A) : IsBounded (σ a)
```

lemma exists_tendsto_state [source]

The values of a bounded net of states on an observable have a limit along an ultrafilter.

```
lemma exists_tendsto_state (ψ : ι → AlgState A) (𝒰 : Ultrafilter ι)
    (hb : ∀ a : A, ∃ c : ℝ, ∀ i, ‖ψ i a‖ ≤ c) (a : A) :
    ∃ z, Tendsto (fun i => ψ i a) 𝒰 (𝒩 z)
```

def limit [source]

***The limit of a bounded net of states along an ultrafilter** is a state.*

```
def AlgState.limit (ψ : ι → AlgState A) (𝒰 : Ultrafilter ι)
    (hb : ∀ a : A, ∃ c : ℝ, ∀ i, ‖ψ i a‖ ≤ c) : AlgState A where
```

lemma tendsto_limit [source]

```
lemma AlgState.tendsto_limit (ψ : ι → AlgState A) (𝒰 : Ultrafilter ι)
    (hb : ∀ a : A, ∃ c : ℝ, ∀ i, ‖ψ i a‖ ≤ c) (a : A) :
    Tendsto (fun i => ψ i a) 𝒰 (𝒩 (AlgState.limit ψ 𝒰 hb a))
```

lemma starAlgEquiv_smul [source]

```
lemma starAlgEquiv_smul (σ : A ≃⋆ₐ[ℂ] A) (c : ℂ) (a : A) : σ (c • a) = c • σ a
```

def averageMap [source]

The linear functional `a ↦ |F|⁻¹ ∑_{g ∈ F} φ (α g a)`.

```
def averageMap (φ : AlgState A) (α : Γ → (A ≃⋆ₐ[ℂ] A)) (F : Finset Γ) : A →ₗ[ℂ] ℂ where
```

lemma averageMap_apply [source]

```
lemma averageMap_apply (φ : AlgState A) (α : Γ → (A ≃⋆ₐ[ℂ] A)) (F : Finset Γ) (a : A) :
    averageMap φ α F a = ((F.card : ℂ))⁻¹ * ∑ g ∈ F, φ (α g a)
```

def average [source]

***The average of a state over a finite set of automorphisms** is a state.*

```
def AlgState.average (φ : AlgState A) (α : Γ → (A ≃⋆ₐ[ℂ] A)) (F : Finset Γ)
    (hF : F.Nonempty) : AlgState A where
```

lemma average_apply [source]

```
lemma AlgState.average_apply (φ : AlgState A) (α : Γ → (A ≃⋆ₐ[ℂ] A)) (F : Finset Γ)
    (hF : F.Nonempty) (a : A) :
    φ.average α F hF a = ((F.card : ℂ))⁻¹ * ∑ g ∈ F, φ (α g a)
```

def compMap [source]

The linear functional `a ↦ φ (σ a)`.

```
def compMap (φ : AlgState A) (σ : A ≃⋆ₐ[ℂ] A) : A →ₗ[ℂ] ℂ where
```

lemma compMap_apply [source]

```
lemma compMap_apply (φ : AlgState A) (σ : A ≃⋆ₐ[ℂ] A) (a : A) : compMap φ σ a = φ (σ a)
```

def comp [source]

***The composition of a state with an automorphism** is a state.*

```
def AlgState.comp (φ : AlgState A) (σ : A ≃⋆ₐ[ℂ] A) : AlgState A where
```

lemma comp_apply [source]

```
lemma AlgState.comp_apply (φ : AlgState A) (σ : A ≃⋆ₐ[ℂ] A) (a : A) : φ.comp σ a = φ (σ a)
```

lemma norm_le_of_isBounded [source]

The bound on a state on a bounded observable, as a norm bound.

```
lemma AlgState.norm_le_of_isBounded {a : A} {c : ℝ} {p : A} (hp : p ∈ sumSq A)
    (h : star a * a + p = (c : ℂ) • (1 : A)) (ω : AlgState A) : ‖ω a‖ ≤ c + 1
```

lemma norm_sum_sub_le [source]

A difference of two sums of a bounded function is bounded by the size of the symmetric difference of the index sets.

```
lemma norm_sum_sub_le (f : Γ → ℂ) (C : ℝ) (hf : ∀ k, ‖f k‖ ≤ C) (S T : Finset Γ) :
    ‖∑ k ∈ S, f k - ∑ k ∈ T, f k‖ ≤ ((S \ T).card + (T \ S).card) * C
```

def IsRightFolner [source]

*The finite sets `F i` are **asymptotically invariant under right translation** along an ultrafilter (a Følner condition): `|F h Δ F| / |F| → 0` for every `h`.*

```
def IsRightFolner (F : ι → Finset Γ) (𝒰 : Ultrafilter ι) : Prop
```

lemma average_bounded [source]

The averaged states of a state over the sets `F i` are bounded on every observable.

```
lemma average_bounded (φ : AlgState A) (hb : ∀ a : A, IsBounded a) (a : A) :
    ∃ c : ℝ, ∀ i, ‖(φ.average (fun g => α g) (F i) (hne i)) a‖ ≤ c
```

def invariantLimit [source]

***The Følner limit of a state**: the ultrafilter limit of its averages over the sets `F i`.*

```
def AlgState.invariantLimit (φ : AlgState A) (hb : ∀ a : A, IsBounded a) (𝒰 : Ultrafilter ι) :
    AlgState A
```

lemma invariantLimit_invariant [source]

***The Følner limit of a state is invariant.** When the sets `F i` are asymptotically invariant under right translation along $\mathcal{U}$, every limit of the averages of a state over them is invariant under the group.*

```
lemma AlgState.invariantLimit_invariant (φ : AlgState A) (hb : ∀ a : A, IsBounded a)
    (𝒰 : Ultrafilter ι) (hF : IsRightFolner F 𝒰) (h : Γ) (a : A) :
    AlgState.invariantLimit α F hne φ hb 𝒰 (α h a) = AlgState.invariantLimit α F hne φ hb 𝒰 a
```

lemma sumSq_map_hom [source]

A star-algebra homomorphism maps sums of squares to sums of squares.

```
lemma sumSq_map_hom {B : Type*} [Ring B] [Algebra ℂ B] [StarRing B]
    (σ : A →⋆ₐ[ℂ] B) {p : A} (hp : p ∈ sumSq A) : σ p ∈ sumSq B
```

lemma isBounded_of_isLocallyBounded [source]

Every observable of the quasi-local algebra of a locally bounded net is bounded.

```
lemma AlgebraNet.isBounded_of_isLocallyBounded (hb : N.IsLocallyBounded) (x : N.Quasilocal) :
    IsBounded x
```

2.560 PhyslibAlpha.ProbabilisticTheory.StarAlgebra.StateUncertainty

PhyslibAlpha/ProbabilisticTheory/StarAlgebra/StateUncertainty.lean on GitHub.

lemma apply_selfAdjoint [source]

A self-adjoint element has a real expectation.

```
lemma apply_selfAdjoint (a : selfAdjoint A) : ω (a : A) = ((ω (a : A)).re : ℂ)
```

def fluctuation [source]

The fluctuation `a - ω(a)` of an observable.

```
def fluctuation (a : selfAdjoint A) : A
```

lemma star_fluctuation [source]

```
lemma star_fluctuation (a : selfAdjoint A) : star (ω.fluctuation a) = ω.fluctuation a
```

def covariance [source]

The covariance `Re ω((a - ω(a)) (b - ω(b)))` of two observables.

```
noncomputable def covariance (a b : selfAdjoint A) : ℝ
```

def variance [source]

The variance of an observable.

```
noncomputable def variance (a : selfAdjoint A) : ℝ
```

def bracket [source]

The expectation of the bracket of two observables.

```
noncomputable def bracket (a b : selfAdjoint A) : ℝ
```

lemma apply_fluctuation_mul [source]

ω((a - ω(a)) (b - ω(b))) = ω(a b) - ω(a) ω(b).

```
lemma apply_fluctuation_mul (a b : selfAdjoint A) :
    ω (ω.fluctuation a * ω.fluctuation b) = ω ((a : A) * b) - ω (a : A) * ω (b : A)
```

lemma bracket_eq_im [source]

The bracket is the imaginary part of ω((a - ω(a)) (b - ω(b))).

```
lemma bracket_eq_im (a b : selfAdjoint A) :
    ω.bracket a b = (ω (ω.fluctuation a * ω.fluctuation b)).im
```

def covarianceMatrix [source]

The covariance matrix Σ_jk = Cov(a_j, a_k) of a family of observables.

```
noncomputable def covarianceMatrix (a : ι → selfAdjoint A) : Matrix ι ι ℝ
```

def bracketMatrix [source]

The matrix Ω_jk = ω(⁅a_j, a_k⁆) of expectations of brackets.

```
noncomputable def bracketMatrix (a : ι → selfAdjoint A) : Matrix ι ι ℝ
```

def fluctuationVector [source]

The fluctuation of an observable as a vector of the pre-Hilbert space of the state.

```
def fluctuationVector (a : selfAdjoint A) : ω.PreGNS
```

lemma inner_fluctuationVector [source]

```
lemma inner_fluctuationVector (a b : selfAdjoint A) :
    ⟪ω.fluctuationVector a, ω.fluctuationVector b⟫_ℂ =
      ω (ω.fluctuation a * ω.fluctuation b)
```

lemma gram_fluctuation [source]

Σ + iΩ is the Gram matrix of the fluctuations.

```
lemma gram_fluctuation (a : ι → selfAdjoint A) :
    gram ℂ (fun j => ω.fluctuationVector (a j)) =
      (ω.covarianceMatrix a).map ((↑) : ℝ → ℂ) +
        Complex.I • (ω.bracketMatrix a).map ((↑) : ℝ → ℂ)
```

lemma transpose_gram_fluctuation [source]

`Σ - iΩ` is the transpose of the Gram matrix.

```
lemma transpose_gram_fluctuation (a : ι → selfAdjoint A) :
    (gram ℂ (fun j => ω.fluctuationVector (a j)))^T =
      (ω.covarianceMatrix a).map ((↑) : ℝ → ℂ) -
        Complex.I • (ω.bracketMatrix a).map ((↑) : ℝ → ℂ)
```

lemma robertson_schrodinger [source]

***The Robertson–Schrödinger relation** for a state on a *-algebra: `Cov(a, b)² + ω([a, b])² ≤ Var a · Var b`.*

```
lemma robertson_schrodinger (a b : selfAdjoint A) :
    ω.covariance a b ^ 2 + ω.bracket a b ^ 2 ≤ ω.variance a * ω.variance b
```

lemma robertson_det [source]

***Robertson's uncertainty relation for several observables** for a state on a *-algebra: for every finite family of observables, `|det Ω| ≤ det Σ`.*

```
lemma robertson_det [Fintype ι] [DecidableEq ι] (a : ι → selfAdjoint A) :
    |(ω.bracketMatrix a).det| ≤ (ω.covarianceMatrix a).det
```

lemma apply_eq_zero_of_symm [source]

```
lemma apply_eq_zero_of_symm : ω (b : A) = 0
```

lemma apply_mul_eq_zero_of_symm [source]

```
lemma apply_mul_eq_zero_of_symm : ω ((a : A) * b) = 0
```

lemma covariance_eq_zero_of_symm [source]

In a symmetric state, an even and an odd observable have covariance zero.

```
lemma covariance_eq_zero_of_symm : ω.covariance a b = 0
```

lemma bracket_eq_zero_of_symm [source]

In a symmetric state, an even and an odd observable have a bracket of expectation zero.

```
lemma bracket_eq_zero_of_symm : ω.bracket a b = 0
```

2.561 PhyslibAlpha.ProbabilisticTheory.State.Metric

PhyslibAlpha/ProbabilisticTheory/State/Metric.lean on GitHub.

lemma apply_effectEquiv [source]

A state's value at a doubled, re-centered effect (`Effect.effectEquiv`) is twice its value at the effect, minus one.

```
lemma apply_effectEquiv (ψ : 𝒮[ℝ, E]) (e : Effect E) :
    ψ ((Effect.effectEquiv e : E)) = 2 * ψ (e : E) - 1
```

2.562 PhyslibAlpha.QuantumMechanics.AngularMomentum.Addition

PhyslibAlpha/QuantumMechanics/AngularMomentum/Addition.lean on GitHub.

def tensor [source]

The coupled angular momentum `J = J₁ ⊗ 1 + 1 ⊗ J₂` on `V ⊗ W`.

```
def tensor (ρ₁ : AngularMomentumRepresentation V) (ρ₂ : AngularMomentumRepresentation W) :
    AngularMomentumRepresentation (V ⊗[ℂ] W) where
```

lemma tensor_Jp [source]

```
lemma tensor_Jp (ρ₁ : AngularMomentumRepresentation V) (ρ₂ : AngularMomentumRepresentation W) :
    (tensor ρ₁ ρ₂).Jp = diagEnd ρ₁.Jp ρ₂.Jp
```

lemma tensor_Jm [source]

```
lemma tensor_Jm (ρ₁ : AngularMomentumRepresentation V) (ρ₂ : AngularMomentumRepresentation W) :
    (tensor ρ₁ ρ₂).Jm = diagEnd ρ₁.Jm ρ₂.Jm
```

lemma ρE_mul_ρF [source]

`EF = FE + H` on `V(n)`, the associative form of `[E, F] = H`.

```
lemma ρE_mul_ρF :
    (ρE : Module.End ℂ (StandardSl2Module ℂ n)) * ρF = ρF * ρE + ρH
```

lemma ρH_mul_ρE [source]

`HE = EH + 2E` on `V(n)`, the associative form of `[H, E] = 2E`.

```
lemma ρH_mul_ρE :
    (ρH : Module.End ℂ (StandardSl2Module ℂ n)) * ρE = ρE * ρH + (2 : ℂ) • ρE
```

lemma ρH_mul_ρF [source]

`HF = FH - 2F` on `V(n)`, the associative form of `[H, F] = -2F`.

```
lemma ρH_mul_ρF :
    (ρH : Module.End ℂ (StandardSl2Module ℂ n)) * ρF = ρF * ρH + (-2 : ℂ) • ρF
```

def stdJx [source]

`Jx = (E + F)/2` on `V(n)`.

```
def stdJx : Module.End ℂ (StandardSl2Module ℂ n)
```

def stdJy [source]

Jy = -i(E - F)/2 on V(n).

```
def stdJy : Module.End ℂ (StandardSl2Module ℂ n)
```

def stdJz [source]

Jz = H/2 on V(n).

```
def stdJz : Module.End ℂ (StandardSl2Module ℂ n)
```

lemma stdJx_stdJy [source]

```
lemma stdJx_stdJy : ⁅stdJx n, stdJy n⁆ = Complex.I • stdJz n
```

lemma stdJy_stdJz [source]

```
lemma stdJy_stdJz : ⁅stdJy n, stdJz n⁆ = Complex.I • stdJx n
```

lemma stdJz_stdJx [source]

```
lemma stdJz_stdJx : ⁅stdJz n, stdJx n⁆ = Complex.I • stdJy n
```

def standard [source]

The spin-n/2 representation on V(n): Jz = H/2, Jx = (E + F)/2, Jy = -i(E - F)/2, so that J+ = E and J- = F.

```
def standard : AngularMomentumRepresentation (StandardSl2Module ℂ n) where
```

lemma standard_Jp [source]

```
lemma standard_Jp : (standard n).Jp = ρE
```

lemma standard_Jm [source]

```
lemma standard_Jm : (standard n).Jm = ρF
```

lemma standard_Jz [source]

```
lemma standard_Jz : (standard n).Jz = (1 / 2 : ℂ) • ρH
```

abbrev coupled [source]

The coupled representation of spins m/2 and n/2 on V(m) ⊗ V(n).

```
abbrev coupled (m n : ℕ) :
    AngularMomentumRepresentation (StandardSl2Module ℂ m ⊗[ℂ] StandardSl2Module ℂ n)
```

lemma coupled_Jz [source]

```
lemma coupled_Jz : (coupled m n).Jz = (1 / 2 : ℂ) • tensorH m n
```

lemma coupled_Jp [source]

```
lemma coupled_Jp : (coupled m n).Jp = tensorE m n
```

lemma coupled_Jm [source]

```
lemma coupled_Jm : (coupled m n).Jm = tensorF m n
```

lemma Jz_coupled_stringVec [source]

`Jz` acts on the `l`-th vector of the `k`-th string by `J - l`, `J = (m + n - 2k)/2`.

```
lemma Jz_coupled_stringVec (l : ℕ) :
    (coupled m n).Jz (stringVec m n k l) =
      (((m + n - 2 * k : ℕ) : ℂ) / 2 - l) • stringVec m n k l
```

lemma Jsq_coupled_stringVec [source]

The total `J²` acts on the `k`-th string by `J(J + 1)`, `J = (m + n - 2k)/2`.

```
lemma Jsq_coupled_stringVec (l : ℕ) :
    (coupled m n).Jsq (stringVec m n k l) =
      (((m + n - 2 * k : ℕ) : ℂ) / 2 * (((m + n - 2 * k : ℕ) : ℂ) / 2 + 1)) •
        stringVec m n k l
```

lemma sumEnd_smul [source]

```
lemma sumEnd_smul (c : ℂ) (A : ∀ k, Module.End ℂ (CGSummand ℂ m n k)) (y : CGSum ℂ m n) :
    sumEnd (fun k => c • A k) y = c • sumEnd A y
```

lemma cgEquiv_Jz_coupled [source]

`cgEquiv` intertwines the total `Jz` with `Jz` of the spin-`J` summands.

```
lemma cgEquiv_Jz_coupled (hmn : 0 < m + n)
    (x : StandardSl2Module ℂ m ⊗[ℂ] StandardSl2Module ℂ n) :
    cgEquiv hmn ((coupled m n).Jz x) = sumEnd (fun _ => (standard _).Jz) (cgEquiv hmn x)
```

lemma cgEquiv_Jp_coupled [source]

`cgEquiv` intertwines the total `J+` with `J+` of the spin-`J` summands.

```
lemma cgEquiv_Jp_coupled (hmn : 0 < m + n)
    (x : StandardSl2Module ℂ m ⊗[ℂ] StandardSl2Module ℂ n) :
    cgEquiv hmn ((coupled m n).Jp x) = sumEnd (fun _ => (standard _).Jp) (cgEquiv hmn x)
```

lemma cgEquiv_Jm_coupled [source]

`cgEquiv` intertwines the total `J-` with `J-` of the spin-`J` summands.

```
lemma cgEquiv_Jm_coupled (hmn : 0 < m + n)
    (x : StandardSl2Module ℂ m ⊗[ℂ] StandardSl2Module ℂ n) :
    cgEquiv hmn ((coupled m n).Jm x) = sumEnd (fun _ => (standard _).Jm) (cgEquiv hmn x)
```

2.563 PhyslibAlpha.QuantumMechanics.AngularMomentum.Basic

PhyslibAlpha/QuantumMechanics/AngularMomentum/Basic.lean on GitHub.

structure AngularMomentumRepresentation [source]

Angular momentum representation. Three operators `Jx, Jy, Jz` on a state space `V`, satisfying the `so(3)` commutation relations `[Jx,Jy] = iJz` and cyclic permutations. (The physical requirement that they be self-adjoint on a Hilbert space is out of scope for this algebraic model.)

```
structure AngularMomentumRepresentation where
  Jx : Module.End ℂ V
  Jy : Module.End ℂ V
  Jz : Module.End ℂ V
  comm_xy : ⁅Jx, Jy⁆ = Complex.I • Jz
  comm_yz : ⁅Jy, Jz⁆ = Complex.I • Jx
  comm_zx : ⁅Jz, Jx⁆ = Complex.I • Jy
```

def Jp [source]

Ladder operators. `J± := Jx ± iJy`.

```
def Jp (ρ : AngularMomentumRepresentation V) : Module.End ℂ V
```

def Jm [source]

```
@[inherit_doc Jp]
def Jm (ρ : AngularMomentumRepresentation V) : Module.End ℂ V
```

lemma comm_z_p [source]

Commutation relations for the ladder operators: `[Jz, J±] = ±J±` and `[J+, J-] = 2Jz`.

```
lemma comm_z_p (ρ : AngularMomentumRepresentation V) : ⁅ρ.Jz, ρ.Jp⁆ = ρ.Jp
```

lemma comm_z_m [source]

```
lemma comm_z_m (ρ : AngularMomentumRepresentation V) : ⁅ρ.Jz, ρ.Jm⁆ = -ρ.Jm
```

lemma comm_p_m [source]

```
lemma comm_p_m (ρ : AngularMomentumRepresentation V) :
    ⁅ρ.Jp, ρ.Jm⁆ = (2 : ℂ) • ρ.Jz
```

lemma isSl2Triple [source]

Angular momentum generates an $\mathfrak{sl}_2$-triple, via `h := 2 • Jz`, `e := J+`, `f := J-` (the doubling of `Jz` reconciles the physicists' normalization `[Jz,J±]=±J±` with the $\mathfrak{sl}_2$-triple normalization `[h,e]=2e`).

The hypothesis `ρ.Jz ≠ 0` is necessary: the all-zero representation (`Jx = Jy = Jz = 0`) satisfies every field of `AngularMomentumRepresentation` but gives `h = 2 • Jz = 0`, which `IsSl2Triple` explicitly excludes (`h_ne_zero`) — the trivial, spin-0 representation does not generate an honest copy of $\mathfrak{sl}_2$ inside `End(V)`.

```
lemma isSl2Triple (ρ : AngularMomentumRepresentation V) (hz : ρ.Jz ≠ 0) :
    IsSl2Triple ((2 : ℂ) • ρ.Jz) ρ.Jp ρ.Jm where
```

def IsIrreducible [source]

Irreducibility of an angular momentum representation: no proper subspace is invariant under `Jx`, `Jy`, and `Jz` simultaneously.

```
def IsIrreducible (ρ : AngularMomentumRepresentation V) : Prop
```

lemma Jx_mem_sl2 [source]

Irreducibility transfers along the bridge: `ρ.IsIrreducible` agrees with irreducibility of the induced module for the $\mathfrak{sl}_2$-subalgebra generated by `(2Jz, J+, J-)` (via the automatic `LieSubalgebra.lieRingModule/lieModule` instances restricting the tautological `Module.End ℂ V`-action on `V`).

```
lemma Jx_mem_sl2 (ρ : AngularMomentumRepresentation V) (hz : ρ.Jz ≠ 0) :
    ρ.Jx ∈ (ρ.isSl2Triple hz).toLieSubalgebra ℂ
```

lemma Jy_mem_sl2 [source]

```
lemma Jy_mem_sl2 (ρ : AngularMomentumRepresentation V) (hz : ρ.Jz ≠ 0) :
    ρ.Jy ∈ (ρ.isSl2Triple hz).toLieSubalgebra ℂ
```

lemma Jz_mem_sl2 [source]

```
lemma Jz_mem_sl2 (ρ : AngularMomentumRepresentation V) (hz : ρ.Jz ≠ 0) :
    ρ.Jz ∈ (ρ.isSl2Triple hz).toLieSubalgebra ℂ
```

lemma Jp_mem_of_invariant [source]

```
lemma Jp_mem_of_invariant (ρ : AngularMomentumRepresentation V)
    (W : Submodule ℂ V) (hx : ∀ v ∈ W, ρ.Jx v ∈ W)
    (hy : ∀ v ∈ W, ρ.Jy v ∈ W) {v : V} (hv : v ∈ W) : ρ.Jp v ∈ W
```

lemma Jm_mem_of_invariant [source]

```
lemma Jm_mem_of_invariant (ρ : AngularMomentumRepresentation V)
    (W : Submodule ℂ V) (hx : ∀ v ∈ W, ρ.Jx v ∈ W)
    (hy : ∀ v ∈ W, ρ.Jy v ∈ W) {v : V} (hv : v ∈ W) : ρ.Jm v ∈ W
```

lemma sl2_invariant_of_angular_invariant [source]

```
lemma sl2_invariant_of_angular_invariant (ρ : AngularMomentumRepresentation V)
    (hz : ρ.Jz ≠ 0) (W : Submodule ℂ V)
    (hx : ∀ v ∈ W, ρ.Jx v ∈ W) (hy : ∀ v ∈ W, ρ.Jy v ∈ W)
    (hzW : ∀ v ∈ W, ρ.Jz v ∈ W) {x : ↥((ρ.isSl2Triple hz).toLieSubalgebra ℂ)}
    {v : V} (hv : v ∈ W) : ⁅x, v⁆ ∈ W
```

lemma nontrivial_of_Jz_ne_zero [source]

A nonzero `Jz` forces the representation space to be nontrivial.

```
lemma nontrivial_of_Jz_ne_zero (ρ : AngularMomentumRepresentation V) (hz : ρ.Jz ≠ 0) :
    Nontrivial V
```

lemma lieSubmodule_stable [source]

Lie submodules are stable under every operator in the generated subalgebra.

```
lemma lieSubmodule_stable (ρ : AngularMomentumRepresentation V) (hz : ρ.Jz ≠ 0)
    (N : LieSubmodule ℂ ↥((ρ.isSl2Triple hz).toLieSubalgebra ℂ) V) {x : Module.End ℂ V}
    (hx : x ∈ (ρ.isSl2Triple hz).toLieSubalgebra ℂ) : ∀ v ∈ (N : Submodule ℂ V), x v ∈ N
```

lemma angular_irreducible_implies_sl2 [source]

```
lemma angular_irreducible_implies_sl2 (ρ : AngularMomentumRepresentation V)
    (hz : ρ.Jz ≠ 0) (hirr : ρ.IsIrreducible) :
    LieModule.IsIrreducible ℂ ↥((ρ.isSl2Triple hz).toLieSubalgebra ℂ) V
```

lemma sl2_irreducible_implies_angular [source]

```
lemma sl2_irreducible_implies_angular (ρ : AngularMomentumRepresentation V)
    (hz : ρ.Jz ≠ 0)
    (hirr : LieModule.IsIrreducible ℂ ↥((ρ.isSl2Triple hz).toLieSubalgebra ℂ) V) :
    ρ.IsIrreducible
```

lemma isIrreducible_iff [source]

Invariance under `Jx`, `Jy`, `Jz` is equivalent to irreducibility for the generated $\mathfrak{sl}_2$ Lie subalgebra.

```
lemma isIrreducible_iff (ρ : AngularMomentumRepresentation V) (hz : ρ.Jz ≠ 0) :
    ρ.IsIrreducible ↔ LieModule.IsIrreducible ℂ (↥((ρ.isSl2Triple hz).toLieSubalgebra ℂ)) V
```

lemma spin_classification [source]

A finite-dimensional irreducible angular momentum representation is isomorphic to a standard $\mathfrak{sl}_2$ module for the subalgebra generated by `(2Jz, Jp, Jm)`.

```
lemma spin_classification [FiniteDimensional ℂ V] (ρ : AngularMomentumRepresentation V)
    (hz : ρ.Jz ≠ 0) (hirr : ρ.IsIrreducible) :
    ∃ n : ℕ, letI
```

lemma Jp_mul_Jm [source]

*J+J- = Jx²+Jy²+Jz: the standard ladder-operator identity, a purely algebraic consequence of the commutation relations (no representation theory needed). Used to reduce Jsq_eq_smul to a statement about Jp * Jm, via J² = Jp*Jm + Jz*Jz - Jz (equivalently Jm*Jp + Jz*Jz + Jz).*

```
lemma Jp_mul_Jm (ρ : AngularMomentumRepresentation V) :
    ρ.Jp * ρ.Jm = ρ.Jx * ρ.Jx + ρ.Jy * ρ.Jy + ρ.Jz
```

lemma Jsq_eq_Jp_mul_Jm [source]

J² = J+J- + Jz² - Jz: a purely algebraic consequence of the commutation relations (no representation theory needed), used to reduce Jsq_eq_smul to a statement about the ladder operators.

```
lemma Jsq_eq_Jp_mul_Jm (ρ : AngularMomentumRepresentation V) :
    ρ.Jx * ρ.Jx + ρ.Jy * ρ.Jy + ρ.Jz * ρ.Jz =
      ρ.Jp * ρ.Jm + ρ.Jz * ρ.Jz - ρ.Jz
```

lemma leibniz_end [source]

*Product Leibniz rule for the associative-ring bracket on Module.End ℂ V: ⁅a*x, z⁆ = a*⁅x,z⁆ + ⁅a,z⁆*x.*

```
lemma leibniz_end (a x z : Module.End ℂ V) :
    (⁅a * x, z⁆ : Module.End ℂ V) = a * ⁅x, z⁆ + ⁅a, z⁆ * x
```

lemma mul_eq_of_lie_eq [source]

*Unfolds the associative-ring bracket relation ⁅a, b⁆ = c into a * b = b * a + c.*

```
lemma mul_eq_of_lie_eq {a b c : Module.End ℂ V} (h : (⁅a, b⁆ : Module.End ℂ V) = c) :
    a * b = b * a + c
```

lemma Jsq_comm_p [source]

J² commutes with J+: a step towards showing J² is central, hence (on an irreducible module) acts as a scalar.

```
lemma Jsq_comm_p (ρ : AngularMomentumRepresentation V) :
    (⁅ρ.Jx * ρ.Jx + ρ.Jy * ρ.Jy + ρ.Jz * ρ.Jz, ρ.Jp⁆ : Module.End ℂ V) = 0
```

lemma Jsq_comm_m [source]

J² commutes with J-: a step towards showing J² is central, hence (on an irreducible module) acts as a scalar.

```
lemma Jsq_comm_m (ρ : AngularMomentumRepresentation V) :
    (⁅ρ.Jx * ρ.Jx + ρ.Jy * ρ.Jy + ρ.Jz * ρ.Jz, ρ.Jm⁆ : Module.End ℂ V) = 0
```

lemma Jsq_comm_z [source]

J² commutes with Jz: derived from Jsq_comm_p/Jsq_comm_m via the Jacobi identity, since 2•Jz = ⁅Jp,Jm⁆.

```
lemma Jsq_comm_z (ρ : AngularMomentumRepresentation V) :
    ⁅ρ.Jx * ρ.Jx + ρ.Jy * ρ.Jy + ρ.Jz * ρ.Jz, ρ.Jz⁆ : Module.End ℂ V) = 0
```

def Jsq [source]

The quadratic angular momentum operator `Jx² + Jy² + Jz²`.

```
def Jsq (ρ : AngularMomentumRepresentation V) : Module.End ℂ V
```

lemma JpJm_on_primitive [source]

On a primitive vector of weight `n`, `Jp Jm` acts by `n`.

```
lemma JpJm_on_primitive (ρ : AngularMomentumRepresentation V) (hz : ρ.Jz ≠ 0)
    {n : ℕ} {v : V} (P : (ρ.isSl2Triple hz).HasPrimitiveVectorWith v (n : ℂ)) :
    ρ.Jp (ρ.Jm v) = (n : ℂ) • v
```

lemma Jz_on_primitive [source]

The Cartan operator acts by half the highest weight on a primitive vector.

```
lemma Jz_on_primitive (ρ : AngularMomentumRepresentation V) (hz : ρ.Jz ≠ 0)
    {n : ℕ} {v : V} (P : (ρ.isSl2Triple hz).HasPrimitiveVectorWith v (n : ℂ)) :
    ρ.Jz v = ((n : ℂ) / 2) • v
```

lemma Jsq_on_primitive [source]

The quadratic operator has the expected eigenvalue on a primitive vector.

```
lemma Jsq_on_primitive (ρ : AngularMomentumRepresentation V) (hz : ρ.Jz ≠ 0)
    {n : ℕ} {v : V} (P : (ρ.isSl2Triple hz).HasPrimitiveVectorWith v (n : ℂ)) :
    ρ.Jsq v = ((n : ℂ) / 2 * ((n : ℂ) / 2 + 1)) • v
```

lemma ker_scalar_stable [source]

An eigenspace of an endomorphism is stable under commuting endomorphisms.

```
lemma ker_scalar_stable (C A : Module.End ℂ V) (c : ℂ)
    (hCA : ⁅C, A⁆ : Module.End ℂ V) = 0)
    {v : V} (hv : v ∈ LinearMap.ker (C - c • (1 : Module.End ℂ V))) :
    A v ∈ LinearMap.ker (C - c • (1 : Module.End ℂ V))
```

lemma Jx_apply_eq [source]

`Jx` is half the sum of the ladder operators.

```
lemma Jx_apply_eq (ρ : AngularMomentumRepresentation V) (v : V) :
    ρ.Jx v = ((1 : ℂ) / 2) • (ρ.Jp v + ρ.Jm v)
```

lemma Jy_apply_eq [source]

`Jy` is `-i/2` times the difference of the ladder operators.

```
lemma Jy_apply_eq (ρ : AngularMomentumRepresentation V) (v : V) :
    ρ.Jy v = (-(Complex.I / 2)) • (ρ.Jp v - ρ.Jm v)
```

lemma Jsq_ker_stable_x [source]

The `J²` eigenspace is stable under `Jx`.

```
lemma Jsq_ker_stable_x (ρ : AngularMomentumRepresentation V) (c : ℂ)
    {v : V} (hv : v ∈ LinearMap.ker (ρ.Jsq - c • (1 : Module.End ℂ V))) :
    ρ.Jx v ∈ LinearMap.ker (ρ.Jsq - c • (1 : Module.End ℂ V))
```

lemma Jsq_ker_stable_y [source]

The `J²` eigenspace is stable under `Jy`.

```
lemma Jsq_ker_stable_y (ρ : AngularMomentumRepresentation V) (c : ℂ)
    {v : V} (hv : v ∈ LinearMap.ker (ρ.Jsq - c • (1 : Module.End ℂ V))) :
    ρ.Jy v ∈ LinearMap.ker (ρ.Jsq - c • (1 : Module.End ℂ V))
```

lemma Jsq_ker_eq_top [source]

A nonzero vector in a `J²` eigenspace forces that eigenspace to be everything.

```
lemma Jsq_ker_eq_top (ρ : AngularMomentumRepresentation V)
    (hirr : ρ.IsIrreducible) (c : ℂ) {v : V} (hv : v ≠ 0)
    (hmem : v ∈ LinearMap.ker (ρ.Jsq - c • (1 : Module.End ℂ V))) :
    LinearMap.ker (ρ.Jsq - c • (1 : Module.End ℂ V)) = ⊤
```

lemma Jsq_eq_scalar_of_eigenvector [source]

A nonzero `J²` eigenvector forces `J²` to be that scalar on an irreducible module.

```
lemma Jsq_eq_scalar_of_eigenvector (ρ : AngularMomentumRepresentation V)
    (hirr : ρ.IsIrreducible) (c : ℂ) {v : V} (hv : v ≠ 0)
    (hCv : ρ.Jsq v = c • v) : ρ.Jsq = c • (1 : Module.End ℂ V)
```

lemma Jsq_eq_zero_of_Jz_eq_zero [source]

If `Jz = 0`, the commutation relations force all three angular momentum operators to vanish.

```
lemma Jsq_eq_zero_of_Jz_eq_zero (ρ : AngularMomentumRepresentation V)
    (hz : ρ.Jz = 0) : ρ.Jsq = 0
```

lemma Jsq_eq_smul [source]

On an irreducible finite-dimensional representation, `J²` acts by `j(j+1)` for some `j = n/2`.

```
lemma Jsq_eq_smul [FiniteDimensional ℂ V] (ρ : AngularMomentumRepresentation V)
    (hirr : ρ.IsIrreducible) :
    ∃ n : ℕ, ρ.Jsq = ((n : ℂ) / 2 * ((n : ℂ) / 2 + 1)) • (1 : Module.End ℂ V)
```

def magneticBasisVec [source]

Magnetic-weight ladder vectors: given a primitive vector `v₀` of weight `n` for the triple `(2Jz, J+, J-)`, this defines `(J-)^k v₀` for `k = 0,…,n`. A basis theorem and reindexing by `m = n/2 - k` remain to be added.

```
def magneticBasisVec (ρ : AngularMomentumRepresentation V) (hz : ρ.Jz ≠ 0) {n : ℕ} {v₀ : V}
    (_ : (ρ.isSl2Triple hz).HasPrimitiveVectorWith v₀ (n : ℂ)) (k : Fin (n + 1)) : V
```

2.564 PhyslibAlpha.QuantumMechanics.AngularMomentum.ClebschGordanCoefficients

PhyslibAlpha/QuantumMechanics/AngularMomentum/ClebschGordanCoefficients.lean on GitHub.

def tensorForm [source]

The contravariant form on `V(m) ⊗ V(n)`: `⟨x ⊗ y, x' ⊗ y'⟩ = ⟨x, x'⟩ ⟨y, y'⟩`.

```
def tensorForm : LinearMap.BilinForm ℝ (StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n)
```

lemma tensorForm_tmul [source]

```
lemma tensorForm_tmul (x x' : StandardSl2Module ℝ m) (y y' : StandardSl2Module ℝ n) :
    tensorForm m n (x ⊗ₜ y) (x' ⊗ₜ y') = invForm ℝ m x x' * invForm ℝ n y y'
```

lemma tensorBilin_ext [source]

Two bilinear forms on `V(m) ⊗ V(n)` agree if they agree on pure tensors.

```
lemma tensorBilin_ext
    {B B' : LinearMap.BilinForm ℝ (StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n)}
    (h : ∀ x y x' y', B (x ⊗ₜ y) (x' ⊗ₜ y') = B' (x ⊗ₜ y) (x' ⊗ₜ y')) : B = B'
```

lemma invForm_ρE_apply [source]

```
lemma invForm_ρE_apply (x y : StandardSl2Module ℝ n) :
    invForm ℝ n (ρE x) y = invForm ℝ n x (ρF y)
```

lemma invForm_ρH_apply [source]

```
lemma invForm_ρH_apply (x y : StandardSl2Module ℝ n) :
    invForm ℝ n (ρH x) y = invForm ℝ n x (ρH y)
```

lemma tensorForm_comm [source]

The form on the tensor product is symmetric.

```
lemma tensorForm_comm (x y : StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n) :
    tensorForm m n x y = tensorForm m n y x
```

lemma tensorForm_tensorE [source]

The diagonal raising and lowering operators are adjoint: `⟨E x, y⟩ = ⟨x, F y⟩`.

```
lemma tensorForm_tensorE (x y : StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n) :
    tensorForm m n (tensorE m n x) y = tensorForm m n x (tensorF m n y)
```

lemma tensorForm_tensorF [source]

⟨F x, y⟩ = ⟨x, E y⟩.

```
lemma tensorForm_tensorF (x y : StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n) :
    tensorForm m n (tensorF m n x) y = tensorForm m n x (tensorE m n y)
```

lemma tensorForm_tensorH [source]

The diagonal weight operator is symmetric.

```
lemma tensorForm_tensorH (x y : StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n) :
    tensorForm m n (tensorH m n x) y = tensorForm m n x (tensorH m n y)
```

lemma tensorForm_hwVec_of_ne [source]

Highest-weight vectors of different weight are orthogonal.

```
lemma tensorForm_hwVec_of_ne {k k' : ℕ} (hk : k ≠ k') :
    tensorForm m n (hwVec m n k) (hwVec m n k') = 0
```

lemma stringVec_zero' [source]

```
lemma stringVec_zero' (k : ℕ) : stringVec (K := ℝ) m n k 0 = hwVec m n k
```

lemma tensorForm_stringVec [source]

The inner products of the lowering strings: ⟨f^a w_k, f^b w_{k'}⟩ = δ_{kk'} δ_{ab} · normSq (m + n - 2k) a · ⟨w_k, w_k⟩.

```
lemma tensorForm_stringVec (hmn : 0 < m + n) {k k' : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hk'm : k' ≤ m) (hk'n : k' ≤ n) (a b : ℕ) :
    tensorForm m n (stringVec m n k a) (stringVec m n k' b) =
      if k = k' ∧ a = b then
        (normSq (m + n - 2 * k) a : ℝ) * tensorForm m n (hwVec m n k) (hwVec m n k)
      else 0
```

def hwNormSq [source]

The squared norm ⟨w_k, w_k⟩ of the highest-weight vector w_k.

```
def hwNormSq (k : ℕ) : ℝ
```

lemma hwNormSq_eq [source]

⟨w_k, w_k⟩ = Σ_a cgCoeff(a)² · normSq m a · normSq n (k - a).

```
lemma hwNormSq_eq {k : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) :
    hwNormSq m n k = ∑ a ∈ Finset.range (k + 1),
      cgCoeff (K := ℝ) m n k a ^ 2 * (normSq m a : ℝ) * (normSq n (k - a) : ℝ)
```

lemma cgCoeff_zero [source]

```
lemma cgCoeff_zero (k : ℕ) :
    cgCoeff (K := ℝ) m n k 0 = ((n - k).factorial * m.factorial : ℕ)
```

lemma hwNormSq_pos [source]

⟨w_k, w_k⟩ > 0.

```
lemma hwNormSq_pos {k : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) : 0 < hwNormSq m n k
```

def coupledVec [source]

The normalised coupled state |J, M⟩ with J = (m + n)/2 - k, M = J - l.

```
def coupledVec (k l : ℕ) : StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n
```

lemma tensorForm_coupledVec [source]

The coupled states are orthonormal.

```
lemma tensorForm_coupledVec (hmn : 0 < m + n) {k k' l l' : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hk'm : k' ≤ m) (hk'n : k' ≤ n) (hl : l ≤ m + n - 2 * k) :
    tensorForm m n (coupledVec m n k l) (coupledVec m n k' l') =
      if k = k' ∧ l = l' then 1 else 0
```

lemma sqrt_coupled_succ [source]

```
lemma sqrt_coupled_succ {k l : ℕ} :
    Real.sqrt ((normSq (m + n - 2 * k) (l + 1) : ℝ) * hwNormSq m n k) =
      Real.sqrt ((normSq (m + n - 2 * k) l : ℝ) * hwNormSq m n k) *
        Real.sqrt (((l : ℝ) + 1) * (((m + n - 2 * k : ℕ) : ℝ) - l))
```

lemma tensorF_coupledVec [source]

J₋ |J, M⟩ = √((J + M)(J - M + 1)) |J, M - 1⟩.

```
lemma tensorF_coupledVec {k l : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (hl : l < m + n - 2 * k) :
    tensorF m n (coupledVec m n k l) =
      Real.sqrt (((l : ℝ) + 1) * (((m + n - 2 * k : ℕ) : ℝ) - l)) • coupledVec m n k (l + 1)
```

lemma tensorE_coupledVec_succ [source]

J₊ |J, M⟩ = √((J - M)(J + M + 1)) |J, M + 1⟩.

```
lemma tensorE_coupledVec_succ (hmn : 0 < m + n) {k l : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hl : l < m + n - 2 * k) :
    tensorE m n (coupledVec m n k (l + 1)) =
      Real.sqrt (((l : ℝ) + 1) * (((m + n - 2 * k : ℕ) : ℝ) - l)) • coupledVec m n k l
```

def cgCoefficient [source]

The Clebsch–Gordan coefficient ⟨j₁ m₁ j₂ m₂ | J M⟩ = ⟨u_a ⊗ u_b, |J, M⟩ with j₁ = m/2, m₁ = j₁ - a, j₂ = n/2, m₂ = j₂ - b, J = (m + n)/2 - k and M = J - l.

```
def cgCoefficient (k l a b : ℕ) : ℝ
```

lemma tensorH_normVec_tmul [source]

```
lemma tensorH_normVec_tmul (a b : ℕ) :
    tensorH m n (normVec m a ⊗ₜ normVec n b) =
      (((m : ℝ) - 2 * a) + ((n : ℝ) - 2 * b)) • (normVec m a ⊗ₜ normVec n b)
```

lemma tensorH_coupledVec [source]

```
lemma tensorH_coupledVec (hmn : 0 < m + n) {k : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (l : ℕ) :
    tensorH m n (coupledVec m n k l) =
      (((m + n - 2 * k : ℕ) : ℝ) - 2 * l) • coupledVec m n k l
```

lemma cgCoefficient_eq_zero_of_ne [source]

***Weight rule.** ⟨j₁ m₁ j₂ m₂ | J M⟩ = 0 unless m₁ + m₂ = M, i.e. a + b = k + l.*

```
lemma cgCoefficient_eq_zero_of_ne (hmn : 0 < m + n) {k l a b : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (h : a + b ≠ k + l) : cgCoefficient m n k l a b = 0
```

lemma invForm_normVec_vNat [source]

```
lemma invForm_normVec_vNat {a : ℕ} (ha : a ≤ n) (c : ℕ) :
    invForm ℝ n (normVec n a) (vNat n c) = if a = c then Real.sqrt (normSq n a) else 0
```

lemma tensorForm_normVec_hwVec [source]

```
lemma tensorForm_normVec_hwVec {k a : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (ha : a ≤ k) :
    tensorForm m n (normVec m a ⊗ₜ normVec n (k - a)) (hwVec m n k) =
      cgCoeff m n k a * Real.sqrt (normSq m a) * Real.sqrt (normSq n (k - a))
```

lemma cgCoefficient_top [source]

The coefficients of the top state |J, J⟩: ⟨j₁ m₁ j₂ m₂ | J J⟩ = cgCoeff(a) · √(normSq m a) · √(normSq n (k - a)) / √(w_k, w_k).

```
lemma cgCoefficient_top {k a : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (ha : a ≤ k) :
    cgCoefficient m n k 0 a (k - a) =
      cgCoeff m n k a * Real.sqrt (normSq m a) * Real.sqrt (normSq n (k - a)) /
        Real.sqrt (hwNormSq m n k)
```

lemma cgCoefficient_top_pos [source]

***Condon–Shortley phase convention.** ⟨j₁ j₁ j₂ (J - j₁) | J J⟩ > 0.*

```
lemma cgCoefficient_top_pos {k : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) :
    0 < cgCoefficient m n k 0 0 k
```

lemma eq_sum_normVec [source]

Every vector of `V(n)` is `Σ_a ⟨u_a, x⟩ u_a`.

```
lemma eq_sum_normVec (x : StandardSl2Module ℝ n) :
    x = ∑ a ∈ Finset.range (n + 1), invForm ℝ n (normVec n a) x • normVec n a
```

lemma eq_sum_normVec_tmul [source]

Every vector of `V(m) ⊗ V(n)` is `Σ_{a b} ⟨u_a ⊗ u_b, x⟩ u_a ⊗ u_b`.

```
lemma eq_sum_normVec_tmul (x : StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n) :
    x = ∑ a ∈ Finset.range (m + 1), ∑ b ∈ Finset.range (n + 1),
      tensorForm m n (normVec m a ⊗ₜ normVec n b) x • (normVec m a ⊗ₜ normVec n b)
```

lemma coupledVec_eq_sum [source]

`|J, M⟩ = Σ_{m₁ m₂} ⟨j₁ m₁ j₂ m₂ | J M⟩ |j₁ m₁⟩ ⊗ |j₂ m₂⟩`.

```
lemma coupledVec_eq_sum (k l : ℕ) :
    coupledVec m n k l = ∑ a ∈ Finset.range (m + 1), ∑ b ∈ Finset.range (n + 1),
      cgCoefficient m n k l a b • (normVec m a ⊗ₜ normVec n b)
```

lemma sum_cgCoefficient_mul [source]

***Orthogonality relation.** `Σ_{m₁ m₂} ⟨j₁ m₁ j₂ m₂ | J M⟩ ⟨j₁ m₁ j₂ m₂ | J' M'⟩ = δ_{JJ'} δ_{MM'}`.*

```
lemma sum_cgCoefficient_mul (hmn : 0 < m + n) {k k' l l' : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hk'm : k' ≤ m) (hk'n : k' ≤ n) (hl : l ≤ m + n - 2 * k) :
    ∑ a ∈ Finset.range (m + 1), ∑ b ∈ Finset.range (n + 1),
      cgCoefficient m n k l a b * cgCoefficient m n k' l' a b =
        if k = k' ∧ l = l' then 1 else 0
```

lemma hwNormSq_zero [source]

```
lemma hwNormSq_zero : hwNormSq m n 0 = ((n.factorial * m.factorial : ℕ) : ℝ) ^ 2
```

lemma cgCoefficient_stretched [source]

`⟨j₁ j₁ j₂ j₂ | j₁+j₂, j₁+j₂⟩ = 1`.

```
lemma cgCoefficient_stretched : cgCoefficient m n 0 0 0 0 = 1
```

lemma hwNormSq_one_one [source]

```
lemma hwNormSq_one_one : hwNormSq 1 1 1 = 2
```

lemma cgCoefficient_singlet_up [source]

Singlet: `⟨½ ½ ½ -½ | 0 0⟩ = 1/√2`.

```
lemma cgCoefficient_singlet_up : cgCoefficient 1 1 1 0 0 1 = 1 / Real.sqrt 2
```

lemma cgCoefficient_singlet_down [source]

Singlet: ⟨½ -½ ½ ½ | 0 0⟩ = -1/√2.

```
lemma cgCoefficient_singlet_down : cgCoefficient 1 1 1 0 1 0 = -1 / Real.sqrt 2
```

lemma coupledVec_one_one_zero_one [source]

```
lemma coupledVec_one_one_zero_one :
    coupledVec 1 1 0 1 = (1 / Real.sqrt 2) • tensorF 1 1 (coupledVec 1 1 0 0)
```

lemma ρE_normVec_one_zero [source]

```
lemma ρE_normVec_one_zero : (ρE : Module.End ℝ (StandardSl2Module ℝ 1)) (normVec 1 0) = 0
```

lemma ρE_normVec_one_one [source]

```
lemma ρE_normVec_one_one :
    (ρE : Module.End ℝ (StandardSl2Module ℝ 1)) (normVec 1 1) = normVec 1 0
```

lemma cgCoefficient_triplet_zero_up [source]

Triplet: ⟨½ ½ ½ -½ | 1 0⟩ = 1/√2.

```
lemma cgCoefficient_triplet_zero_up : cgCoefficient 1 1 0 1 0 1 = 1 / Real.sqrt 2
```

lemma cgCoefficient_triplet_zero_down [source]

Triplet: ⟨½ -½ ½ ½ | 1 0⟩ = 1/√2.

```
lemma cgCoefficient_triplet_zero_down : cgCoefficient 1 1 0 1 1 0 = 1 / Real.sqrt 2
```

2.565 PhyslibAlpha.QuantumMechanics.AngularMomentum.ClebschGordanFormula

PhyslibAlpha/QuantumMechanics/AngularMomentum/ClebschGordanFormula.lean on GitHub.

lemma tensorForm_coupledVec_stringVec [source]

⟨|J, M⟩, f^{l'} w_{k'}⟩ = δ_{kk'} δ_{ll'} √(normSq N l · ⟨w_k, w_k⟩).

```
lemma tensorForm_coupledVec_stringVec (hmn : 0 < m + n) {k k' l l' : ℕ} (hkm : k ≤ m)
    (hkn : k ≤ n) (hk'm : k' ≤ m) (hk'n : k' ≤ n) (hl : l ≤ m + n - 2 * k) :
    tensorForm m n (coupledVec m n k l) (stringVec m n k' l') =
      if k = k' ∧ l = l' then
        Real.sqrt ((normSq (m + n - 2 * k) l : ℝ) * hwNormSq m n k)
      else 0
```

lemma eq_sum_coupledVec [source]

***Completeness.** Every vector of V(m) ⊗ V(n) is Σ_{J M} ⟨|J, M⟩, x⟩ |J, M⟩.*

```
lemma eq_sum_coupledVec (hmn : 0 < m + n) (x : StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n
    ) :
    x = ∑ k ∈ Finset.range (min m n + 1), ∑ l ∈ Finset.range (m + n - 2 * k + 1),
```

```
    tensorForm m n (coupledVec m n k l) x • coupledVec m n k l
```

lemma sum_cgCoefficient_mul_cgCoefficient [source]

***Second orthogonality relation.** Σ_{J M} ⟨j₁ m₁ j₂ m₂ | J M⟩ ⟨j₁ m₁' j₂ m₂' | J M⟩ = δ_{m₁ m₁'} δ_{m₂ m₂'}.*

```
lemma sum_cgCoefficient_mul_cgCoefficient (hmn : 0 < m + n) {a b a' b' : ℕ} (ha : a ≤ m)
    (hb : b ≤ n) (ha' : a' ≤ m) (hb' : b' ≤ n) :
    ∑ k ∈ Finset.range (min m n + 1), ∑ l ∈ Finset.range (m + n - 2 * k + 1),
      cgCoefficient m n k l a b * cgCoefficient m n k l a' b' =
        if a = a' ∧ b = b' then 1 else 0
```

def factorialSum [source]

S(k, p, s) = Σ_{i + j = k} C(k, i) (p + i)! (s + j)!.

```
def factorialSum (k p s : ℕ) : ℕ
```

lemma factorialSum_succ [source]

Pascal's rule for `S`.

```
lemma factorialSum_succ (k p s : ℕ) :
    factorialSum (k + 1) p s = factorialSum k p (s + 1) + factorialSum k (p + 1) s
```

lemma factorialSum_mul [source]

S(k, p, s) · (p + s + 1)! = (p + k + s + 1)! p! s!.

```
lemma factorialSum_mul (k p s : ℕ) :
    factorialSum k p s * (p + s + 1).factorial =
      (p + k + s + 1).factorial * p.factorial * s.factorial
```

lemma sum_choose_mul_factorial [source]

The factorial identity in the form used for ⟨w_k, w_k⟩: Σ_a C(k, a) (p + a)! (s + k - a)! · (p + s + 1)! = (p + k + s + 1)! p! s!.

```
lemma sum_choose_mul_factorial (k p s : ℕ) :
    (∑ a ∈ Finset.range (k + 1), k.choose a * ((p + a).factorial * (s + (k - a)).factorial)) *
      (p + s + 1).factorial = (p + k + s + 1).factorial * p.factorial * s.factorial
```

def hwNormNum [source]

The numerator k! m! n! (m + n - k + 1)! (n - k)! (m - k)! of ⟨w_k, w_k⟩.

```
def hwNormNum (m n k : ℕ) : ℕ
```

lemma cgCoeff_sq_mul_normSq [source]

```
lemma cgCoeff_sq_mul_normSq {k a : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (ha : a ≤ k) :
    cgCoeff (K := ℝ) m n k a ^ 2 * (normSq m a : ℝ) * (normSq n (k - a) : ℝ) =
      ((k.choose a * ((n - k + a).factorial * (m - a).factorial) *
```

```
      (k.factorial * m.factorial * n.factorial) : ℕ) : ℝ)
```

lemma hwNormSq_mul_factorial [source]

⟨w_k, w_k⟩ (m + n - 2k + 1)! = k! m! n! (m + n - k + 1)! (n - k)! (m - k)!.

```
lemma hwNormSq_mul_factorial {k : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) :
    hwNormSq m n k * ((m + n - 2 * k + 1).factorial : ℝ) = hwNormNum m n k
```

lemma hwNormSq_closed [source]

***The closed norm.** ⟨w_k, w_k⟩ = k! m! n! (m + n - k + 1)! (n - k)! (m - k)! / (m + n - 2k + 1)!.*

```
lemma hwNormSq_closed {k : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) :
    hwNormSq m n k = hwNormNum m n k / ((m + n - 2 * k + 1).factorial : ℝ)
```

lemma ρF_pow_vNat [source]

```
lemma ρF_pow_vNat (a l : ℕ) :
    ((ρF : Module.End ℝ (StandardSl2Module ℝ n)) ^ l) (vNat n a) = vNat n (a + l)
```

lemma tensorF_pow_tmul_vNat [source]

The binomial theorem for the lowering operator: f^l (v_a ⊗ v_b) = Σ_i C(l, i) v_{a+i} ⊗ v_{b+l-i}.

```
lemma tensorF_pow_tmul_vNat (a b l : ℕ) :
    (tensorF (K := ℝ) m n ^ l) (vNat m a ⊗ₜ vNat n b) =
      ∑ i ∈ Finset.range (l + 1), (l.choose i : ℝ) • (vNat m (a + i) ⊗ₜ vNat n (b + (l - i)))
```

def cgSum [source]

The alternating sum Σ_{a ≤ min(k, α)} (-1)^a C(k, a) (n - k + a)! (m - a)! C(l, α - a).

```
def cgSum (m n k l α : ℕ) : ℝ
```

lemma sum_choose_mul_ite [source]

```
lemma sum_choose_mul_ite {l a α : ℕ} (X : ℝ) :
    ∑ i ∈ Finset.range (l + 1), (l.choose i : ℝ) * (if α = a + i then X else 0) =
      if a ≤ α then (l.choose (α - a) : ℝ) * X else 0
```

lemma tensorForm_normVec_tmul_vNat [source]

```
lemma tensorForm_normVec_tmul_vNat {α β a i k l : ℕ} (hα : α ≤ m) (hβ : β ≤ n)
    (h : α + β = k + l) (hak : a ≤ k) (hil : i ≤ l) :
    tensorForm m n (normVec m α ⊗ₜ normVec n β) (vNat m (a + i) ⊗ₜ vNat n (k - a + (l - i))) =
      if α = a + i then Real.sqrt (normSq m α) * Real.sqrt (normSq n β) else 0
```

lemma tensorForm_normVec_stringVec [source]

⟨u_α ⊗ u_β, f^l w_k⟩ = cgSum · √(normSq m α) √(normSq n β) for α + β = k + l.

```
lemma tensorForm_normVec_stringVec {k l α β : ℕ} (hα : α ≤ m) (hβ : β ≤ n)
    (h : α + β = k + l) :
    tensorForm m n (normVec m α ⊗ₜ normVec n β) (stringVec m n k l) =
      cgSum m n k l α * Real.sqrt (normSq m α) * Real.sqrt (normSq n β)
```

lemma cgCoefficient_eq_cgSum [source]

The Clebsch–Gordan coefficient as one finite sum, with the norm of `w_k` still unevaluated.

```
lemma cgCoefficient_eq_cgSum {k l α β : ℕ} (hα : α ≤ m) (hβ : β ≤ n) (h : α + β = k + l) :
    cgCoefficient m n k l α β =
      (Real.sqrt ((normSq (m + n - 2 * k) l : ℝ) * hwNormSq m n k))⁻¹ *
        (cgSum m n k l α * Real.sqrt (normSq m α) * Real.sqrt (normSq n β))
```

lemma cgCoefficient_closed [source]

***Closed formula for the Clebsch–Gordan coefficients.** For `α + β = k + l`, with `N = m + n - 2k`, `⟨j₁ m₁ j₂ m₂ | J M⟩ = cgSum · √(normSq m α · normSq n β · (N + 1)! / (normSq N l · hwNormNum))`. For `α + β ≠ k + l` the coefficient vanishes by `cgCoefficient_eq_zero_of_ne`*

```
lemma cgCoefficient_closed {k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hl : l ≤ m + n - 2 * k) (hα : α ≤ m) (hβ : β ≤ n) (h : α + β = k + l) :
    cgCoefficient m n k l α β = cgSum m n k l α *
      Real.sqrt ((normSq m α : ℝ) * normSq n β * (m + n - 2 * k + 1).factorial /
        ((normSq (m + n - 2 * k) l : ℝ) * hwNormNum m n k))
```

def zchoose [source]

`C(n, j)` for `j ≥ 0` and `0` for `j < 0`.

```
def zchoose (n : ℕ) (j : ℤ) : ℤ
```

lemma zchoose_natCast [source]

```
lemma zchoose_natCast (n t : ℕ) : zchoose n t = n.choose t
```

lemma zchoose_of_neg [source]

```
lemma zchoose_of_neg {n : ℕ} {j : ℤ} (h : j < 0) : zchoose n j = 0
```

lemma zchoose_succ [source]

Pascal's rule `C(n + 1, j) = C(n, j) + C(n, j - 1)` for all integers `j`.

```
lemma zchoose_succ (n : ℕ) (j : ℤ) : zchoose (n + 1) j = zchoose n j + zchoose n (j - 1)
```

lemma sum_range_choose_succ_mul [source]

Pascal's rule inside a binomial sum.

```
lemma sum_range_choose_succ_mul (f : ℕ → ℤ) (k : ℕ) :
    ∑ a ∈ Finset.range (k + 1 + 1), ((k + 1).choose a : ℤ) * f a =
      ∑ a ∈ Finset.range (k + 1), (k.choose a : ℤ) * f a +
        ∑ a ∈ Finset.range (k + 1), (k.choose a : ℤ) * f (a + 1)
```

def racahL [source]

`L(k, α, s) = Σ_a (-1)^a C(k, a) C(r + k - a, α - a) C(p + a, s + a)`: the sum `cgSum` after dividing out factorials.

```
def racahL (r p k : ℕ) (α s : ℤ) : ℤ
```

def racahR [source]

`R(k, α, s) = Σ_z (-1)^z C(k, z) C(r, α - z) C(p, s + z)`: Racah's sum after dividing out factorials.

```
def racahR (r p k : ℕ) (α s : ℤ) : ℤ
```

lemma racahL_succ [source]

```
lemma racahL_succ (r p k : ℕ) (α s : ℤ) :
    racahL r p (k + 1) α s = racahL r p k α s - racahL r p k (α - 1) (s + 1)
```

lemma racahR_succ [source]

```
lemma racahR_succ (r p k : ℕ) (α s : ℤ) :
    racahR r p (k + 1) α s = racahR r p k α s - racahR r p k (α - 1) (s + 1)
```

lemma racahL_eq_racahR [source]

***The transformation.** `L = R`, by induction on `k`: both satisfy `X(k + 1, α, s) = X(k, α, s) - X(k, α - 1, s + 1)` and agree for `k = 0`.*

```
lemma racahL_eq_racahR (r p k : ℕ) (α s : ℤ) : racahL r p k α s = racahR r p k α s
```

def racahSum [source]

Racah's alternating sum `Σ_z (-1)^z / (z! (k - z)! (α - z)! (n - β - z)! (m - k - α + z)! (β - k + z)!)` over the `z` for which all six arguments are nonnegative. In spin notation the arguments are `z`, `j₁ + j₂ - J - z`, `j₁ - m₁ - z`, `j₂ + m₂ - z`, `J - j₂ + m₁ + z` and `J - j₁ - m₂ + z`.

```
def racahSum (m n k α β : ℕ) : ℝ
```

lemma cgSum_eq_racahL [source]

```
lemma cgSum_eq_racahL {k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (hα : α ≤ m) (hβ : β ≤ n)
    (h : α + β = k + l) :
    cgSum m n k l α = (l.factorial : ℝ) * (m - α).factorial * (n - β).factorial *
```

```
    (racahL (m - k) (n - k) k α ((l : ℤ) - α) : ℝ)
```

lemma racahR_eq_racahSum [source]

```
lemma racahR_eq_racahSum {k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (h : α + β = k + l) :
    (racahR (m - k) (n - k) k α ((l : ℤ) - α) : ℝ) =
      (k.factorial : ℝ) * (m - k).factorial * (n - k).factorial * racahSum m n k α β
```

lemma cgSum_eq_racahSum [source]

cgSum = l! (m - α)! (n - β)! · k! (m - k)! (n - k)! · racahSum.

```
lemma cgSum_eq_racahSum {k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (hα : α ≤ m) (hβ : β ≤ n)
    (h : α + β = k + l) :
    cgSum m n k l α = (l.factorial : ℝ) * (m - α).factorial * (n - β).factorial *
      ((k.factorial : ℝ) * (m - k).factorial * (n - k).factorial) * racahSum m n k α β
```

lemma cgCoefficient_racah [source]

***Racah's formula.** For α + β = k + l, with j₁ = m/2, m₁ = j₁ - α, j₂ = n/2, m₂ = j₂ - β, J = (m + n)/2 - k, M = J - l, `⟨j₁ m₁ j₂ m₂ | J M⟩ = √((2J + 1) (J + j₁ - j₂)! (J - j₁ + j₂)! (j₁ + j₂ - J)! / (j₁ + j₂ + J + 1)!) · √((J + M)! (J - M)! (j₁ - m₁)! (j₁ + m₁)! (j₂ - m₂)! (j₂ + m₂)!) · racahSum`.*

```
lemma cgCoefficient_racah {k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hl : l ≤ m + n - 2 * k) (hα : α ≤ m) (hβ : β ≤ n) (h : α + β = k + l) :
    cgCoefficient m n k l α β =
      Real.sqrt (((m + n - 2 * k + 1) * (m - k).factorial * (n - k).factorial * k.factorial :
    ℕ) /
        ((m + n - k + 1).factorial : ℝ)) *
      Real.sqrt (((m + n - 2 * k - l).factorial * l.factorial * α.factorial * (m - α).factorial
     *
        β.factorial * (n - β).factorial : ℕ) : ℝ) * racahSum m n k α β
```

2.566 PhyslibAlpha.QuantumMechanics.AngularMomentum.CouplingSchemes

PhyslibAlpha/QuantumMechanics/AngularMomentum/CouplingSchemes.lean on GitHub.

def embed [source]

The isometric embedding V(m + n - 2k) → V(m) ⊗ V(n) onto the k-th irreducible component, u_l ↦ |J, J - l⟩.

```
def embed : StandardSl2Module ℝ (m + n - 2 * (k : ℕ)) →ₗ[ℝ]
    StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n
```

lemma cgEquiv_symm_sumVec [source]

```
lemma cgEquiv_symm_sumVec (l : ℕ) :
    (cgEquiv (K := ℝ) hmn).symm (DirectSum.lof ℝ _ (CGSummand ℝ m n) k (vNat _ l)) =
      stringVec m n k l
```

lemma embed_vNat [source]

```
lemma embed_vNat (l : ℕ) :
    embed hmn k (vNat _ l) = (Real.sqrt (hwNormSq m n k))⁻¹ • stringVec m n k l
```

lemma embed_normVec [source]

```
lemma embed_normVec (l : ℕ) : embed hmn k (normVec _ l) = coupledVec m n k l
```

lemma tensorF_embed [source]

```
lemma tensorF_embed (x : StandardSl2Module ℝ (m + n - 2 * (k : ℕ))) :
    tensorF m n (embed hmn k x) = embed hmn k (ρF x)
```

lemma tensorE_embed [source]

```
lemma tensorE_embed (x : StandardSl2Module ℝ (m + n - 2 * (k : ℕ))) :
    tensorE m n (embed hmn k x) = embed hmn k (ρE x)
```

lemma tensorH_embed [source]

```
lemma tensorH_embed (x : StandardSl2Module ℝ (m + n - 2 * (k : ℕ))) :
    tensorH m n (embed hmn k x) = embed hmn k (ρH x)
```

lemma tensorForm_embed [source]

The embedding is an isometry.

```
lemma tensorForm_embed (x y : StandardSl2Module ℝ (m + n - 2 * (k : ℕ))) :
    tensorForm m n (embed hmn k x) (embed hmn k y) = invForm ℝ _ x y
```

lemma tensorForm_embed_of_ne [source]

Different components are orthogonal.

```
lemma tensorForm_embed_of_ne {k' : Fin (min m n + 1)} (hkk : k ≠ k')
    (x : StandardSl2Module ℝ (m + n - 2 * (k : ℕ)))
    (y : StandardSl2Module ℝ (m + n - 2 * (k' : ℕ))) :
    tensorForm m n (embed hmn k x) (embed hmn k' y) = 0
```

abbrev Triple [source]

`V(a) ⊗ V(b) ⊗ V(c)`, bracketed as `(V(a) ⊗ V(b)) ⊗ V(c)`.

```
abbrev Triple : Type
```

def tripleForm [source]

The contravariant form on the triple product.

```
def tripleForm : LinearMap.BilinForm ℝ (Triple a b c)
```

def tripleE [source]

The total raising operator on the triple product.

```
def tripleE : Module.End ℝ (Triple a b c)
```

def tripleF [source]

The total lowering operator on the triple product.

```
def tripleF : Module.End ℝ (Triple a b c)
```

def tripleH [source]

The total weight operator on the triple product.

```
def tripleH : Module.End ℝ (Triple a b c)
```

lemma tripleForm_tmul [source]

```
lemma tripleForm_tmul (x x' : StandardSl2Module ℝ a ⊗[ℝ] StandardSl2Module ℝ b)
    (y y' : StandardSl2Module ℝ c) :
    tripleForm a b c (x ⊗ₜ y) (x' ⊗ₜ y') = tensorForm a b x x' * invForm ℝ c y y'
```

lemma tripleBilin_ext [source]

```
lemma tripleBilin_ext {B B' : LinearMap.BilinForm ℝ (Triple a b c)}
    (h : ∀ x y z x' y' z', B ((x ⊗ₜ y) ⊗ₜ z) ((x' ⊗ₜ y') ⊗ₜ z') =
      B' ((x ⊗ₜ y) ⊗ₜ z) ((x' ⊗ₜ y') ⊗ₜ z')) : B = B'
```

lemma invForm_ρF_apply [source]

```
lemma invForm_ρF_apply {n : ℕ} (x y : StandardSl2Module ℝ n) :
    invForm ℝ n (ρF x) y = invForm ℝ n x (ρE y)
```

lemma tripleForm_tripleF [source]

```
lemma tripleForm_tripleF (u v : Triple a b c) :
    tripleForm a b c (tripleF a b c u) v = tripleForm a b c u (tripleE a b c v)
```

lemma tripleForm_tripleE [source]

```
lemma tripleForm_tripleE (u v : Triple a b c) :
    tripleForm a b c (tripleE a b c u) v = tripleForm a b c u (tripleF a b c v)
```

lemma tripleForm_tripleH [source]

```
lemma tripleForm_tripleH (u v : Triple a b c) :
    tripleForm a b c (tripleH a b c u) v = tripleForm a b c u (tripleH a b c v)
```

lemma tripleForm_assoc_symm [source]

```
lemma tripleForm_assoc_symm (x x' : StandardSl2Module ℝ a)
    (w w' : StandardSl2Module ℝ b ⊗[ℝ] StandardSl2Module ℝ c) :
    tripleForm a b c ((TensorProduct.assoc ℝ _ _ _).symm (x ⊗ₜ w))
      ((TensorProduct.assoc ℝ _ _ _).symm (x' ⊗ₜ w')) = invForm ℝ a x x' * tensorForm b c w w'
```

lemma tripleF_assoc_symm [source]

```
lemma tripleF_assoc_symm (x : StandardSl2Module ℝ a)
    (w : StandardSl2Module ℝ b ⊗[ℝ] StandardSl2Module ℝ c) :
    tripleF a b c ((TensorProduct.assoc ℝ _ _ _).symm (x ⊗ₜ w)) =
      (TensorProduct.assoc ℝ _ _ _).symm (ρF x ⊗ₜ w + x ⊗ₜ tensorF b c w)
```

lemma tripleE_assoc_symm [source]

```
lemma tripleE_assoc_symm (x : StandardSl2Module ℝ a)
    (w : StandardSl2Module ℝ b ⊗[ℝ] StandardSl2Module ℝ c) :
    tripleE a b c ((TensorProduct.assoc ℝ _ _ _).symm (x ⊗ₜ w)) =
      (TensorProduct.assoc ℝ _ _ _).symm (ρE x ⊗ₜ w + x ⊗ₜ tensorE b c w)
```

lemma tripleH_assoc_symm [source]

```
lemma tripleH_assoc_symm (x : StandardSl2Module ℝ a)
    (w : StandardSl2Module ℝ b ⊗[ℝ] StandardSl2Module ℝ c) :
    tripleH a b c ((TensorProduct.assoc ℝ _ _ _).symm (x ⊗ₜ w)) =
      (TensorProduct.assoc ℝ _ _ _).symm (ρH x ⊗ₜ w + x ⊗ₜ tensorH b c w)
```

lemma tripleForm_comm [source]

```
lemma tripleForm_comm (u v : Triple a b c) : tripleForm a b c u v = tripleForm a b c v u
```

def tensorCas [source]

2 J² = E F + F E + H²/2 on V(m) ⊗ V(n).

```
def tensorCas : Module.End ℝ (StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n)
```

lemma tensorCas_stringVec [source]

2 J² = N(N + 2)/2 = 2 J(J + 1) on the k-th string, N = m + n - 2k.

```
lemma tensorCas_stringVec {k : ℕ} (hmn : 0 < m + n) (hkm : k ≤ m) (hkn : k ≤ n) (l : ℕ) :
    tensorCas m n (stringVec m n k l) =
      (((m + n - 2 * k : ℕ) : ℝ) * (((m + n - 2 * k : ℕ) : ℝ) + 2) / 2) • stringVec m n k l
```

lemma tensorCas_coupledVec [source]

```
lemma tensorCas_coupledVec {k : ℕ} (hmn : 0 < m + n) (hkm : k ≤ m) (hkn : k ≤ n) (l : ℕ) :
    tensorCas m n (coupledVec m n k l) =
      (((m + n - 2 * k : ℕ) : ℝ) * (((m + n - 2 * k : ℕ) : ℝ) + 2) / 2) • coupledVec m n k l
```

def tripleCas [source]

2 J² on the triple product.

```
def tripleCas : Module.End ℝ (Triple a b c)
```

lemma tripleForm_tripleCas [source]

```
lemma tripleForm_tripleCas {a b c : ℕ} (u v : Triple a b c) :
    tripleForm a b c (tripleCas a b c u) v = tripleForm a b c u (tripleCas a b c v)
```

def stateA [source]

|(j₁ j₂) j₁₂, j₃; J, M⟩ : j₁ = a/2, j₂ = b/2, j₃ = c/2, j₁₂ = (a + b)/2 - k₁, J = j₁₂ + j₃ - k₂, M = J - l.

```
def stateA (hab : 0 < a + b) (k₁ : Fin (min a b + 1)) (k₂ l : ℕ) : Triple a b c
```

def assocEmbed [source]

V(a) ⊗ V(b + c - 2k₃) → V(a) ⊗ V(b) ⊗ V(c), through the k₃-th component of V(b) ⊗ V(c).

```
def assocEmbed (hbc : 0 < b + c) (k₃ : Fin (min b c + 1)) :
    StandardSl2Module ℝ a ⊗[ℝ] StandardSl2Module ℝ (b + c - 2 * (k₃ : ℕ)) →ₗ[ℝ] Triple a b c
```

def stateB [source]

|j₁, (j₂ j₃) j₂₃; J, M⟩ : j₂₃ = (b + c)/2 - k₃, J = j₁ + j₂₃ - k₄, M = J - l.

```
def stateB (hbc : 0 < b + c) (k₃ : Fin (min b c + 1)) (k₄ l : ℕ) : Triple a b c
```

lemma tripleF_rTensor_embed [source]

```
lemma tripleF_rTensor_embed
    (u : StandardSl2Module ℝ (a + b - 2 * (k₁ : ℕ)) ⊗[ℝ] StandardSl2Module ℝ c) :
    tripleF a b c ((embed hab k₁).rTensor _ u) = (embed hab k₁).rTensor _ (tensorF _ c u)
```

lemma tripleE_rTensor_embed [source]

```
lemma tripleE_rTensor_embed
    (u : StandardSl2Module ℝ (a + b - 2 * (k₁ : ℕ)) ⊗[ℝ] StandardSl2Module ℝ c) :
    tripleE a b c ((embed hab k₁).rTensor _ u) = (embed hab k₁).rTensor _ (tensorE _ c u)
```

lemma tripleH_rTensor_embed [source]

```
lemma tripleH_rTensor_embed
    (u : StandardSl2Module ℝ (a + b - 2 * (k₁ : ℕ)) ⊗[ℝ] StandardSl2Module ℝ c) :
    tripleH a b c ((embed hab k₁).rTensor _ u) = (embed hab k₁).rTensor _ (tensorH _ c u)
```

lemma tripleCas_rTensor_embed [source]

```
lemma tripleCas_rTensor_embed
    (u : StandardSl2Module ℝ (a + b - 2 * (k₁ : ℕ)) ⊗[ℝ] StandardSl2Module ℝ c) :
    tripleCas a b c ((embed hab k₁).rTensor _ u) = (embed hab k₁).rTensor _ (tensorCas _ c u)
```

lemma tripleForm_rTensor_embed [source]

```
lemma tripleForm_rTensor_embed
    (u u' : StandardSl2Module ℝ (a + b - 2 * (k₁ : ℕ)) ⊗[ℝ] StandardSl2Module ℝ c) :
    tripleForm a b c ((embed hab k₁).rTensor _ u) ((embed hab k₁).rTensor _ u') =
      tensorForm _ c u u'
```

lemma tripleForm_rTensor_embed_of_ne [source]

```
lemma tripleForm_rTensor_embed_of_ne {k₁' : Fin (min a b + 1)} (hk : k₁ ≠ k₁')
    (u : StandardSl2Module ℝ (a + b - 2 * (k₁ : ℕ)) ⊗[ℝ] StandardSl2Module ℝ c)
    (u' : StandardSl2Module ℝ (a + b - 2 * (k₁' : ℕ)) ⊗[ℝ] StandardSl2Module ℝ c) :
    tripleForm a b c ((embed hab k₁).rTensor _ u) ((embed hab k₁').rTensor _ u') = 0
```

lemma tripleF_assocEmbed [source]

```
lemma tripleF_assocEmbed
    (w : StandardSl2Module ℝ a ⊗[ℝ] StandardSl2Module ℝ (b + c - 2 * (k₃ : ℕ))) :
    tripleF a b c (assocEmbed hbc k₃ w) = assocEmbed hbc k₃ (tensorF a _ w)
```

lemma tripleE_assocEmbed [source]

```
lemma tripleE_assocEmbed
    (w : StandardSl2Module ℝ a ⊗[ℝ] StandardSl2Module ℝ (b + c - 2 * (k₃ : ℕ))) :
    tripleE a b c (assocEmbed hbc k₃ w) = assocEmbed hbc k₃ (tensorE a _ w)
```

lemma tripleH_assocEmbed [source]

```
lemma tripleH_assocEmbed
    (w : StandardSl2Module ℝ a ⊗[ℝ] StandardSl2Module ℝ (b + c - 2 * (k₃ : ℕ))) :
    tripleH a b c (assocEmbed hbc k₃ w) = assocEmbed hbc k₃ (tensorH a _ w)
```

lemma tripleCas_assocEmbed [source]

```
lemma tripleCas_assocEmbed
    (w : StandardSl2Module ℝ a ⊗[ℝ] StandardSl2Module ℝ (b + c - 2 * (k₃ : ℕ))) :
    tripleCas a b c (assocEmbed hbc k₃ w) = assocEmbed hbc k₃ (tensorCas a _ w)
```

lemma tripleForm_assocEmbed [source]

```
lemma tripleForm_assocEmbed
    (w w' : StandardSl2Module ℝ a ⊗[ℝ] StandardSl2Module ℝ (b + c - 2 * (k₃ : ℕ))) :
    tripleForm a b c (assocEmbed hbc k₃ w) (assocEmbed hbc k₃ w') = tensorForm a _ w w'
```

lemma tripleForm_assocEmbed_of_ne [source]

```
lemma tripleForm_assocEmbed_of_ne {k₃' : Fin (min b c + 1)} (hk : k₃ ≠ k₃')
    (w : StandardSl2Module ℝ a ⊗[ℝ] StandardSl2Module ℝ (b + c - 2 * (k₃ : ℕ)))
    (w' : StandardSl2Module ℝ a ⊗[ℝ] StandardSl2Module ℝ (b + c - 2 * (k₃' : ℕ))) :
    tripleForm a b c (assocEmbed hbc k₃ w) (assocEmbed hbc k₃' w') = 0
```

2.567 PhyslibAlpha.QuantumMechanics.AngularMomentum.MultipoleSumRule

PhyslibAlpha/QuantumMechanics/AngularMomentum/MultipoleSumRule.lean on GitHub.

lemma wigner3j_cycle [source]

(j₁ j₂ J; m₁ m₂ -M) = (j₂ J j₁; m₂ -M m₁) in the parameters of ClebschGordanCoefficients: the new triple is (n, m + n - 2k) with k' = n - k, l' = m - α, α' = β and β' = m + n - 2k - l.

```
lemma wigner3j_cycle {k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (hl : l ≤ m + n - 2 * k)
    (hα : α ≤ m) (hβ : β ≤ n) :
    wigner3j m n k l α β =
      wigner3j n (m + n - 2 * k) (n - k) (m - α) β (m + n - 2 * k - l)
```

lemma sum_sq_cgCoefficient_initial [source]

***Sum rule for multipole transitions.** With j₁ = m/2 = J_i, j₂ = n/2 = λ, J = (m + n)/2 - k = J_f and m₁ = j₁ - α = M_i, Σ_{μ M_f} ⟨J_i M_i λ μ | J_f M_f⟩² = (2J_f + 1) / (2J_i + 1).*

```
lemma sum_sq_cgCoefficient_initial (hmn : 0 < m + n) {k α : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hα : α ≤ m) :
    ∑ b ∈ Finset.range (n + 1), ∑ l ∈ Finset.range (m + n - 2 * k + 1),
      cgCoefficient m n k l α b ^ 2 = ((m + n - 2 * k + 1 : ℕ) : ℝ) / ((m + 1 : ℕ) : ℝ)
```

2.568 PhyslibAlpha.QuantumMechanics.AngularMomentum.NormalizedBasis

PhyslibAlpha/QuantumMechanics/AngularMomentum/NormalizedBasis.lean on GitHub.

def normSq [source]

The squared norm ⟨v_a, v_a⟩ = a! · n!/(n - a)! of the basis vector v_a of V(n).

```
def normSq (n a : ℕ) : ℕ
```

lemma normSq_succ [source]

```
lemma normSq_succ (n a : ℕ) : normSq n (a + 1) = normSq n a * ((a + 1) * (n - a))
```

lemma normSq_pos [source]

```
lemma normSq_pos {n a : ℕ} (ha : a ≤ n) : 0 < normSq n a
```

def invForm [source]

The contravariant form on `V(n)`: diagonal in the basis `v_a` with entries `normSq n a`.

```
def invForm : StandardSl2Module K n →ₗ[K] StandardSl2Module K n →ₗ[K] K
```

lemma invForm_v [source]

```
lemma invForm_v (i j : Fin (n + 1)) :
    invForm K n (v i) (v j) = if i = j then (normSq n i : K) else 0
```

lemma invForm_vNat [source]

```
lemma invForm_vNat (a b : ℕ) :
    invForm K n (vNat n a) (vNat n b) = if a = b ∧ a ≤ n then (normSq n a : K) else 0
```

lemma bilin_ext_vNat [source]

Two bilinear forms on `V(n)` agree if they agree on all pairs `v_a, v_b`.

```
lemma bilin_ext_vNat {B B' : StandardSl2Module K n →ₗ[K] StandardSl2Module K n →ₗ[K] K}
    (h : ∀ a b, B (vNat n a) (vNat n b) = B' (vNat n a) (vNat n b)) : B = B'
```

lemma invForm_comp_ρE [source]

The raising and lowering operators are adjoint: `⟪e x, y⟫ = ⟪x, f y⟫`.

```
lemma invForm_comp_ρE : (invForm K n).comp ρE = (invForm K n).compl₂ ρF
```

lemma invForm_comp_ρH [source]

The weight operator is symmetric: `⟪h x, y⟫ = ⟪x, h y⟫`.

```
lemma invForm_comp_ρH : (invForm K n).comp ρH = (invForm K n).compl₂ ρH
```

lemma invForm_comm [source]

The form is symmetric.

```
lemma invForm_comm (x y : StandardSl2Module K n) : invForm K n x y = invForm K n y x
```

def normVec [source]

The normalised basis vector `u_a = v_a / √(normSq n a)` of `V(n)` over ℝ; for `a ≤ n` it is the state `|j, m⟩` with `j = n/2`, `m = j - a`.

```
def normVec (n a : ℕ) : StandardSl2Module ℝ n
```

lemma sqrt_normSq_succ [source]

```
lemma sqrt_normSq_succ (n a : ℕ) :
    Real.sqrt (normSq n (a + 1)) =
      Real.sqrt (normSq n a) * Real.sqrt (((a : ℝ) + 1) * ((n : ℝ) - a))
```

lemma invForm_normVec [source]

Orthonormality: ⟪u_a, u_b⟫ = δ_ab for a, b ≤ n.

```
lemma invForm_normVec {a b : ℕ} (ha : a ≤ n) (hb : b ≤ n) :
    invForm ℝ n (normVec n a) (normVec n b) = if a = b then 1 else 0
```

lemma ρH_normVec [source]

The weight operator on the normalised basis: h u_a = (n - 2a) u_a, i.e. Jz |j, m⟩ = m |j, m⟩.

```
lemma ρH_normVec (a : ℕ) :
    (ρH : Module.End ℝ (StandardSl2Module ℝ n)) (normVec n a) =
      ((n : ℝ) - 2 * a) • normVec n a
```

lemma ρF_normVec [source]

The lowering operator on the normalised basis: f u_a = √((a+1)(n-a)) u_{a+1} for a < n, i.e. J₋ |j, m⟩ = √((j + m)(j - m + 1)) |j, m - 1⟩.

```
lemma ρF_normVec {a : ℕ} (ha : a < n) :
    (ρF : Module.End ℝ (StandardSl2Module ℝ n)) (normVec n a) =
      Real.sqrt (((a : ℝ) + 1) * ((n : ℝ) - a)) • normVec n (a + 1)
```

lemma ρE_normVec_succ [source]

The raising operator on the normalised basis: e u_{a+1} = √((a+1)(n-a)) u_a for a < n, i.e. J₊ |j, m⟩ = √((j - m)(j + m + 1)) |j, m + 1⟩.

```
lemma ρE_normVec_succ {a : ℕ} (ha : a < n) :
    (ρE : Module.End ℝ (StandardSl2Module ℝ n)) (normVec n (a + 1)) =
      Real.sqrt (((a : ℝ) + 1) * ((n : ℝ) - a)) • normVec n a
```

2.569 PhyslibAlpha.QuantumMechanics.AngularMomentum.ProjectionTheorem

PhyslibAlpha/QuantumMechanics/AngularMomentum/ProjectionTheorem.lean on GitHub.

lemma IsTensorIntertwiner.exists_eq_smul_of_ne_zero [source]

Two tensor intertwiners into the same V(p) are proportional, provided the second one is nonzero.

```
lemma IsTensorIntertwiner.exists_eq_smul_of_ne_zero (hmn : 0 < m + n)
    {φ ψ : StandardSl2Module K m ⊗[K] StandardSl2Module K n →ₗ[K] StandardSl2Module K p}
    (hφ : IsTensorIntertwiner φ) (hψ : IsTensorIntertwiner ψ) (hψ0 : ψ ≠ 0) :
    ∃ c : K, φ = c • ψ
```

lemma IsTensorOperator.exists_eq_smul_of_ne_zero [source]

Two tensor operators of rank k from V(m) to V(p) are proportional, provided the second one is nonzero.

```
lemma IsTensorOperator.exists_eq_smul_of_ne_zero (hkm : 0 < k + m)
    {T S : StandardSl2Module K k →ₗ[K] StandardSl2Module K m →ₗ[K] StandardSl2Module K p}
    (hT : IsTensorOperator T) (hS : IsTensorOperator S) (hS0 : S ≠ 0) :
```

```
    ∃ c : K, T = c • S
```

def sl2Operator [source]

The sl_2 action on $V(n)$ as a tensor operator of rank 2: $v_0 \mapsto e$, $v_1 \mapsto -h$, $v_2 \mapsto -2f$. These are the spherical components of the angular momentum, up to normalisation.

```
def sl2Operator : StandardSl2Module K 2 →ₗ[K] Module.End K (StandardSl2Module K n)
```

lemma sl2Operator_vNat_zero [source]

```
lemma sl2Operator_vNat_zero : sl2Operator (K := K) n (vNat 2 0) = ρE
```

lemma sl2Operator_vNat_one [source]

```
lemma sl2Operator_vNat_one : sl2Operator (K := K) n (vNat 2 1) = (-1 : K) • ρH
```

lemma sl2Operator_vNat_two [source]

```
lemma sl2Operator_vNat_two : sl2Operator (K := K) n (vNat 2 2) = (-2 : K) • ρF
```

lemma sl2Operator_vNat_of_lt [source]

```
lemma sl2Operator_vNat_of_lt {l : ℕ} (hl : 2 < l) : sl2Operator (K := K) n (vNat 2 l) = 0
```

lemma ρE_mul_ρF' [source]

```
lemma ρE_mul_ρF' : (ρE : Module.End K (StandardSl2Module K n)) * ρF = ρF * ρE + ρH
```

lemma ρH_mul_ρE' [source]

```
lemma ρH_mul_ρE' : (ρH : Module.End K (StandardSl2Module K n)) * ρE = ρE * ρH + (2 : K) • ρE
```

lemma ρH_mul_ρF' [source]

```
lemma ρH_mul_ρF' : (ρH : Module.End K (StandardSl2Module K n)) * ρF = ρF * ρH - (2 : K) • ρF
```

lemma ρE_vNat_two_one [source]

```
lemma ρE_vNat_two_one : (ρE : Module.End K (StandardSl2Module K 2)) (vNat 2 1) =
    (2 : K) • vNat 2 0
```

lemma ρE_vNat_two_two [source]

```
lemma ρE_vNat_two_two : (ρE : Module.End K (StandardSl2Module K 2)) (vNat 2 2) =
    (2 : K) • vNat 2 1
```

lemma ρH_vNat_two_zero [source]

```
lemma ρH_vNat_two_zero : (ρH : Module.End K (StandardSl2Module K 2)) (vNat 2 0) =
    (2 : K) • vNat 2 0
```

lemma ρH_vNat_two_one [source]

```
lemma ρH_vNat_two_one : (ρH : Module.End K (StandardSl2Module K 2)) (vNat 2 1) = 0
```

lemma ρH_vNat_two_two [source]

```
lemma ρH_vNat_two_two : (ρH : Module.End K (StandardSl2Module K 2)) (vNat 2 2) =
    (-2 : K) • vNat 2 2
```

lemma comm_of_vNat [source]

A commutation relation `[Y, A x] = A (X x)` for a linear `A` holds for all `x` once it holds on the coordinate vectors.

```
lemma comm_of_vNat {X : Module.End K (StandardSl2Module K k)}
    {Y : Module.End K (StandardSl2Module K n)}
    {A : StandardSl2Module K k →ₗ[K] Module.End K (StandardSl2Module K n)}
    (h : ∀ l, Y * A (vNat k l) - A (vNat k l) * Y = A (X (vNat k l)))
    (x : StandardSl2Module K k) :
    Y ∘ₗ A x - A x ∘ₗ Y = A (X x)
```

lemma isTensorOperator_sl2Operator [source]

The $\mathfrak{sl}_2$ action is a tensor operator of rank 2 on every `V(n)`.

```
lemma isTensorOperator_sl2Operator : IsTensorOperator (sl2Operator (K := K) n) where
```

lemma sl2Operator_ne_zero [source]

On `V(n)` with `n > 0` the $\mathfrak{sl}_2$ action is a nonzero tensor operator.

```
lemma sl2Operator_ne_zero (hn : 0 < n) : sl2Operator (K := K) n ≠ 0
```

lemma exists_eq_smul_sl2Operator [source]

***Projection theorem.** Every vector operator (tensor operator of rank 2) on the multiplet `V(n)` is a scalar multiple of the angular momentum `sl2Operator n`.*

```
lemma IsTensorOperator.exists_eq_smul_sl2Operator
    {T : StandardSl2Module K 2 →ₗ[K] StandardSl2Module K n →ₗ[K] StandardSl2Module K n}
    (hT : IsTensorOperator T) : ∃ c : K, T = c • sl2Operator n
```

2.570 PhyslibAlpha.QuantumMechanics.AngularMomentum.SelectionRules

PhyslibAlpha/QuantumMechanics/AngularMomentum/SelectionRules.lean on GitHub.

def TriangleRule [source]

The triangle rule in doubled units: `p ≤ k + m`, `k ≤ m + p`, `m ≤ k + p` and `k + m + p` even. For `k = 2λ`, `m = 2I₁`, `p = 2I_f` this is `|I₁ - I_f| ≤ λ ≤ I₁ + I_f` with `I₁ + I_f + λ` an integer.

```
def TriangleRule (k m p : ℕ) : Prop
```

instance Decidable (TriangleRule k m p) [source]

```
instance (k m p : ℕ) : Decidable (TriangleRule k m p)
```

lemma exists_eq_iff_triangleRule [source]

`p` occurs in the Clebsch–Gordan decomposition of `V(k) ⊗ V(m)` exactly when the triangle rule holds.

```
lemma exists_eq_iff_triangleRule {k m p : ℕ} :
    (∃ j : Fin (min k m + 1), k + m - 2 * (j : ℕ) = p) ↔ TriangleRule k m p
```

lemma triangleRule [source]

***Triangle rule.** A nonzero tensor operator of rank `k` from `V(m)` to `V(p)` satisfies `TriangleRule k m p`.*

```
lemma IsTensorOperator.triangleRule {K : Type*} [Field K] [CharZero K] {k m p : ℕ}
    (hkm : 0 < k + m)
    {T : StandardSl2Module K k →ₗ[K] StandardSl2Module K m →ₗ[K] StandardSl2Module K p}
    (hT : IsTensorOperator T) (hT0 : T ≠ 0) : TriangleRule k m p
```

lemma parity_eq_of_ne_zero [source]

***Parity rule.** If a linear map between states of definite parities `π₁` and `π_f` has parity `π_T`, in the sense `π_f • T = π_T • π₁ • T`, and is nonzero, then `π_f = π_T * π₁`.*

```
lemma parity_eq_of_ne_zero {K V W : Type*} [Field K] [AddCommGroup V] [Module K V]
    [AddCommGroup W] [Module K W] {T : V →ₗ[K] W} {πi πf πT : K}
    (h : πf • T = πT • πi • T) (hT : T ≠ 0) : πf = πT * πi
```

inductive MultipoleType [source]

Electric and magnetic multipoles.

```
inductive MultipoleType
  | electric
  | magnetic
  deriving DecidableEq
```

def parity [source]

The parity of a multipole operator of order `λ`: `(-1)^λ` for `Eλ`, `(-1)^(λ+1)` for `Mλ`.

```
def MultipoleType.parity : MultipoleType → ℕ → ℤ
  | .electric, l => (-1) ^ l
  | .magnetic, l => (-1) ^ (l + 1)
```

def IsAllowedMultipole [source]

A multipole transition of type `t` and order `l` between a state of spin I_i and parity π_i and a state of spin `I_f` and parity `π_f` (spins in doubled units `twoIi = 2I_i`, `twoIf = 2I_f`) is allowed if `l ≥ 1`, the triangle rule holds and the parities match.

```
def IsAllowedMultipole (twoIi twoIf : ℕ) (πi πf : ℤ) (t : MultipoleType) (l : ℕ) : Prop
```

instance Decidable (IsAllowedMultipole twoIi twoIf πi πf t l) [source]

```
instance (twoIi twoIf : ℕ) (πi πf : ℤ) (t : MultipoleType) (l : ℕ) :
    Decidable (IsAllowedMultipole twoIi twoIf πi πf t l)
```

lemma hf178_primary_branches_allowed [source]

The three branches depopulating the 16^+ isomer are allowed: `E3` to 13^-, `M4` to 12^-, `E5` to 11^-.

```
lemma hf178_primary_branches_allowed :
    IsAllowedMultipole 32 26 1 (-1) .electric 3 ∧ IsAllowedMultipole 32 24 1 (-1) .magnetic 4 ∧
      IsAllowedMultipole 32 22 1 (-1) .electric 5
```

lemma hf178_E3_lowest [source]

`E3` is the lowest allowed multipole for $16^+ \to 13^-$.

```
lemma hf178_E3_lowest : ∀ l < 3, ¬ IsAllowedMultipole 32 26 1 (-1) .electric l ∧
    ¬ IsAllowedMultipole 32 26 1 (-1) .magnetic l
```

lemma hf178_M4_lowest [source]

`M4` is the lowest allowed multipole for $16^+ \to 12^-$; `E4` is forbidden by parity.

```
lemma hf178_M4_lowest : (∀ l < 4, ¬ IsAllowedMultipole 32 24 1 (-1) .electric l ∧
    ¬ IsAllowedMultipole 32 24 1 (-1) .magnetic l) ∧
    ¬ IsAllowedMultipole 32 24 1 (-1) .electric 4
```

lemma hf178_E5_lowest [source]

`E5` is the lowest allowed multipole for $16^+ \to 11^-$.

```
lemma hf178_E5_lowest : (∀ l < 5, ¬ IsAllowedMultipole 32 22 1 (-1) .electric l ∧
    ¬ IsAllowedMultipole 32 22 1 (-1) .magnetic l) ∧
    ¬ IsAllowedMultipole 32 22 1 (-1) .magnetic 5
```

lemma hf178_E1_8minus [source]

The $8^- \to 8^+$ decay of the `4 s` isomer is `E1`, and `M1` is forbidden by parity.

```
lemma hf178_E1_8minus : IsAllowedMultipole 16 16 (-1) 1 .electric 1 ∧
    ¬ IsAllowedMultipole 16 16 (-1) 1 .magnetic 1
```

2.571 PhyslibAlpha.QuantumMechanics.AngularMomentum.SixJ

PhyslibAlpha/QuantumMechanics/AngularMomentum/SixJ.lean on GitHub.

lemma tripleH_stateA [source]

```
lemma tripleH_stateA (l : ℕ) :
    tripleH a b c (stateA hab k₁ k₂ l) =
      (((a + b - 2 * (k₁ : ℕ) + c - 2 * k₂ : ℕ) : ℝ) - 2 * l) • stateA hab k₁ k₂ l
```

lemma tripleCas_stateA [source]

```
lemma tripleCas_stateA (l : ℕ) :
    tripleCas a b c (stateA hab k₁ k₂ l) =
      (((a + b - 2 * (k₁ : ℕ) + c - 2 * k₂ : ℕ) : ℝ) *
        (((a + b - 2 * (k₁ : ℕ) + c - 2 * k₂ : ℕ) : ℝ) + 2) / 2) • stateA hab k₁ k₂ l
```

lemma tripleF_stateA [source]

```
lemma tripleF_stateA {l : ℕ} (hl : l < a + b - 2 * (k₁ : ℕ) + c - 2 * k₂) :
    tripleF a b c (stateA hab k₁ k₂ l) =
      Real.sqrt (((l : ℝ) + 1) * (((a + b - 2 * (k₁ : ℕ) + c - 2 * k₂ : ℕ) : ℝ) - l)) •
        stateA hab k₁ k₂ (l + 1)
```

lemma tripleH_stateB [source]

```
lemma tripleH_stateB (l : ℕ) :
    tripleH a b c (stateB hbc k₃ k₄ l) =
      (((a + (b + c - 2 * (k₃ : ℕ)) - 2 * k₄ : ℕ) : ℝ) - 2 * l) • stateB hbc k₃ k₄ l
```

lemma tripleCas_stateB [source]

```
lemma tripleCas_stateB (l : ℕ) :
    tripleCas a b c (stateB hbc k₃ k₄ l) =
      (((a + (b + c - 2 * (k₃ : ℕ)) - 2 * k₄ : ℕ) : ℝ) *
        (((a + (b + c - 2 * (k₃ : ℕ)) - 2 * k₄ : ℕ) : ℝ) + 2) / 2) • stateB hbc k₃ k₄ l
```

lemma tripleE_stateB_succ [source]

```
lemma tripleE_stateB_succ {l : ℕ} (hl : l < a + (b + c - 2 * (k₃ : ℕ)) - 2 * k₄) :
    tripleE a b c (stateB hbc k₃ k₄ (l + 1)) =
      Real.sqrt (((l : ℝ) + 1) * (((a + (b + c - 2 * (k₃ : ℕ)) - 2 * k₄ : ℕ) : ℝ) - l)) •
        stateB hbc k₃ k₄ l
```

lemma tripleForm_stateB_self [source]

```
lemma tripleForm_stateB_self {l : ℕ} (hl : l ≤ a + (b + c - 2 * (k₃ : ℕ)) - 2 * k₄) :
    tripleForm a b c (stateB hbc k₃ k₄ l) (stateB hbc k₃ k₄ l) = 1
```

lemma tripleForm_stateB_of_ne [source]

```
lemma tripleForm_stateB_of_ne {k₃' : Fin (min b c + 1)} (hk : k₃ ≠ k₃') (k₄' l l' : ℕ) :
    tripleForm a b c (stateB hbc k₃ k₄ l) (stateB hbc k₃' k₄' l') = 0
```

lemma eq_sum_of_orthonormal [source]

An orthonormal family of `finrank` many vectors expands every vector.

```
lemma eq_sum_of_orthonormal {ι M : Type*} [Fintype ι] [DecidableEq ι] [Nonempty ι] [
    AddCommGroup M]
    [Module ℝ M] (B : LinearMap.BilinForm ℝ M) (v : ι → M)
    (hv : ∀ i j, B (v i) (v j) = if i = j then 1 else 0)
    (hcard : Fintype.card ι = Module.finrank ℝ M) (x : M) :
    x = ∑ i, B (v i) x • v i
```

abbrev IndexA [source]

The index set `(k₁, k₂, l)` of the first coupling scheme.

```
abbrev IndexA : Type
```

instance Nonempty (IndexA a b c) [source]

```
instance : Nonempty (IndexA a b c)
```

def vecA [source]

The states of the first coupling scheme, as a family indexed by `IndexA`.

```
def vecA (hab : 0 < a + b) (i : IndexA a b c) : Triple a b c
```

lemma card_indexA [source]

```
lemma card_indexA : Fintype.card (IndexA a b c) = Module.finrank ℝ (Triple a b c)
```

lemma tripleForm_vecA [source]

```
lemma tripleForm_vecA (hab : 0 < a + b) (hc : 0 < c) (i j : IndexA a b c) :
    tripleForm a b c (vecA hab i) (vecA hab j) = if i = j then 1 else 0
```

lemma eq_sum_vecA [source]

***Completeness.** The states `|(j₁ j₂) j₁₂, j₃; J, M⟩` form an orthonormal basis.*

```
lemma eq_sum_vecA (hab : 0 < a + b) (hc : 0 < c) (x : Triple a b c) :
    x = ∑ i, tripleForm a b c (vecA hab i) x • vecA hab i
```

lemma nat_eq_of_mul_add_two_eq [source]

```
lemma nat_eq_of_mul_add_two_eq {p q : ℕ} (h : p * (p + 2) = q * (q + 2)) : p = q
```

lemma tripleForm_stateA_stateB_eq_zero [source]

States of different total `J` or different `M` are orthogonal.

```
lemma tripleForm_stateA_stateB_eq_zero (hab : 0 < a + b) (hbc : 0 < b + c) (ha : 0 < a)
    (hc : 0 < c) {k₁ : Fin (min a b + 1)} {k₃ : Fin (min b c + 1)} {k₂ k₄ l' l : ℕ}
    (h₁ : k₂ ≤ a + b - 2 * (k₁ : ℕ)) (h₂ : k₂ ≤ c) (h₃ : k₄ ≤ a)
    (h₄ : k₄ ≤ b + c - 2 * (k₃ : ℕ)) (h : ¬((k₁ : ℕ) + k₂ = k₃ + k₄ ∧ l' = l)) :
    tripleForm a b c (stateA hab k₁ k₂ l') (stateB hbc k₃ k₄ l) = 0
```

abbrev ValidA [source]

`k₁` gives an admissible intermediate `j₁₂` for the total `J = (a + b + c)/2 - K`.

```
abbrev ValidA (K k₁ : ℕ) : Prop
```

abbrev ValidB [source]

`k₃` gives an admissible intermediate `j₂₃` for the total `J = (a + b + c)/2 - K`.

```
abbrev ValidB (K k₃ : ℕ) : Prop
```

def recoupling [source]

The recoupling coefficient `⟨(j₁ j₂) j₁₂, j₃; J M | j₁, (j₂ j₃) j₂₃; J M⟩` with `J = (a + b + c)/2 - K`, `M = J - l`, `j₁₂ = (a + b)/2 - k₁` and `j₂₃ = (b + c)/2 - k₃`.

```
def recoupling (hab : 0 < a + b) (hbc : 0 < b + c) (K l : ℕ) (k₁ : Fin (min a b + 1))
    (k₃ : Fin (min b c + 1)) : ℝ
```

lemma recoupling_succ [source]

```
lemma recoupling_succ (hab : 0 < a + b) (hbc : 0 < b + c) (ha : 0 < a) {K l : ℕ}
    {k₁ : Fin (min a b + 1)} {k₃ : Fin (min b c + 1)} (hA : ValidA a b c K k₁)
    (hB : ValidB a b c K k₃) (hl : l < a + b + c - 2 * K) :
    recoupling hab hbc K (l + 1) k₁ k₃ = recoupling hab hbc K l k₁ k₃
```

lemma recoupling_eq_recoupling_zero [source]

The recoupling coefficients do not depend on `M`.

```
lemma recoupling_eq_recoupling_zero (hab : 0 < a + b) (hbc : 0 < b + c) (ha : 0 < a)
    {K l : ℕ} {k₁ : Fin (min a b + 1)} {k₃ : Fin (min b c + 1)} (hA : ValidA a b c K k₁)
    (hB : ValidB a b c K k₃) (hl : l ≤ a + b + c - 2 * K) :
    recoupling hab hbc K l k₁ k₃ = recoupling hab hbc K 0 k₁ k₃
```

lemma sum_recoupling_mul_recoupling [source]

***Orthogonality of the recoupling coefficients.** For fixed `J` and `M`, `Σ_{j₁₂} ⟨(j₁ j₂) j₁₂, j₃; J M | j₁, (j₂ j₃) j₂₃; J M⟩ ⟨(j₁ j₂) j₁₂, j₃; J M | j₁, (j₂ j₃) j₂₃'; J M⟩ = δ_{j₂₃ j₂₃'}`.*

```
lemma sum_recoupling_mul_recoupling (hab : 0 < a + b) (hbc : 0 < b + c) (ha : 0 < a)
    (hc : 0 < c) {K l : ℕ} (hl : l ≤ a + b + c - 2 * K) {k₃ k₃' : Fin (min b c + 1)}
    (hB : ValidB a b c K k₃) (hB' : ValidB a b c K k₃') :
```

```
    ∑ k₁ : Fin (min a b + 1), (if ValidA a b c K k₁ then
      recoupling hab hbc K l k₁ k₃ * recoupling hab hbc K l k₁ k₃' else 0) =
        if k₃ = k₃' then 1 else 0
```

def sixJ [source]

*The **6j symbol** {j₁ j₂ j₁₂; j₃ J j₂₃}, with j₁ = a/2, j₂ = b/2, j₃ = c/2, j₁₂ = (a + b)/2 - k₁, j₂₃ = (b + c)/2 - k₃ and J = (a + b + c)/2 - K, defined through ((j₁ j₂) j₁₂, j₃; J | j₁, (j₂ j₃) j₂₃; J) = (-1)^(j₁ + j₂ + j₃ + J) √((2j₁₂ + 1)(2j₂₃ + 1)) {6j} (j₁ + j₂ + j₃ + J = a + b + c - K), and zero for inadmissible arguments.*

```
def sixJ (hab : 0 < a + b) (hbc : 0 < b + c) (K : ℕ) (k₁ : Fin (min a b + 1))
    (k₃ : Fin (min b c + 1)) : ℝ
```

lemma recoupling_eq_sixJ [source]

The recoupling coefficients in terms of the 6j symbols, for every M.

```
lemma recoupling_eq_sixJ (hab : 0 < a + b) (hbc : 0 < b + c) (ha : 0 < a) {K l : ℕ}
    {k₁ : Fin (min a b + 1)} {k₃ : Fin (min b c + 1)} (hA : ValidA a b c K k₁)
    (hB : ValidB a b c K k₃) (hl : l ≤ a + b + c - 2 * K) :
    recoupling hab hbc K l k₁ k₃ = (-1) ^ (a + b + c - K) *
      Real.sqrt (((a + b - 2 * (k₁ : ℕ) + 1 : ℕ) : ℝ) * ((b + c - 2 * (k₃ : ℕ) + 1 : ℕ) : ℝ)) *
        sixJ hab hbc K k₁ k₃
```

lemma mul_mul_div_mul_div_sqrt [source]

```
lemma mul_mul_div_mul_div_sqrt {s U U' p₁ p₃ p₃' : ℝ} (hs : s * s = 1) (h₁ : 0 < p₁) :
    p₁ * p₃ * (s * U / Real.sqrt (p₁ * p₃)) * (s * U' / Real.sqrt (p₁ * p₃')) =
      Real.sqrt p₃ / Real.sqrt p₃' * (U * U')
```

lemma sum_sixJ_mul_sixJ [source]

*Orthogonality of the 6j symbols. ** Σ_{j₁₂} (2j₁₂ + 1)(2j₂₃ + 1) {j₁ j₂ j₁₂; j₃ J j₂₃} {j₁ j₂ j₁₂; j₃ J j₂₃'} = δ_{j₂₃ j₂₃'}.*

```
lemma sum_sixJ_mul_sixJ (hab : 0 < a + b) (hbc : 0 < b + c) (ha : 0 < a) (hc : 0 < c)
    {K : ℕ} {k₃ k₃' : Fin (min b c + 1)} (hB : ValidB a b c K k₃)
    (hB' : ValidB a b c K k₃') :
    ∑ k₁ : Fin (min a b + 1), ((a + b - 2 * (k₁ : ℕ) + 1 : ℕ) : ℝ) *
      ((b + c - 2 * (k₃ : ℕ) + 1 : ℕ) : ℝ) * sixJ hab hbc K k₁ k₃ * sixJ hab hbc K k₁ k₃' =
        if k₃ = k₃' then 1 else 0
```

2.572 PhyslibAlpha.QuantumMechanics.AngularMomentum.SixJSymmetry

PhyslibAlpha/QuantumMechanics/AngularMomentum/SixJSymmetry.lean on GitHub.

abbrev Adm3 [source]

Admissibility of (j₁ j₂ j₃; m₁ m₂ m₃) in doubled units: the projections add up to zero, |mᵢ| ≤ jᵢ, jᵢ - mᵢ is even and the triangle condition holds with even perimeter.

```
abbrev Adm3 (j₁ j₂ j₃ : ℕ) (m₁ m₂ m₃ : ℤ) : Prop
```

def reggeJ [source]

The Regge square of (j₁ j₂ j₃; m₁ m₂ m₃) in doubled units.

```
def reggeJ (j₁ j₂ j₃ : ℕ) (m₁ m₂ m₃ : ℤ) : Fin 3 → Fin 3 → ℕ
```

def threeJ [source]

*The **3j symbol** (j₁ j₂ j₃; m₁ m₂ m₃) with jᵢ and mᵢ given in doubled units, zero for inadmissible arguments.*

```
def threeJ (j₁ j₂ j₃ : ℕ) (m₁ m₂ m₃ : ℤ) : ℝ
```

lemma threeJ_of_not [source]

```
lemma threeJ_of_not (h : ¬Adm3 j₁ j₂ j₃ m₁ m₂ m₃) : threeJ j₁ j₂ j₃ m₁ m₂ m₃ = 0
```

lemma adm3_of_threeJ_ne_zero [source]

```
lemma adm3_of_threeJ_ne_zero (h : threeJ j₁ j₂ j₃ m₁ m₂ m₃ ≠ 0) : Adm3 j₁ j₂ j₃ m₁ m₂ m₃
```

lemma isMagic_reggeJ [source]

```
lemma isMagic_reggeJ (h : Adm3 j₁ j₂ j₃ m₁ m₂ m₃) : IsMagic (reggeJ j₁ j₂ j₃ m₁ m₂ m₃)
```

lemma reggeJ_row_sum [source]

```
lemma reggeJ_row_sum (h : Adm3 j₁ j₂ j₃ m₁ m₂ m₃) :
    reggeJ j₁ j₂ j₃ m₁ m₂ m₃ 0 0 + reggeJ j₁ j₂ j₃ m₁ m₂ m₃ 0 1 +
      reggeJ j₁ j₂ j₃ m₁ m₂ m₃ 0 2 = (j₁ + j₂ + j₃) / 2
```

lemma threeJ_swap [source]

Exchange of the first two columns: (j₂ j₁ j₃; m₂ m₁ m₃) = (-1)^(j₁+j₂+j₃) (j₁ j₂ j₃; m₁ m₂ m₃).

```
lemma threeJ_swap (j₁ j₂ j₃ : ℕ) (m₁ m₂ m₃ : ℤ) :
    threeJ j₂ j₁ j₃ m₂ m₁ m₃ = (-1) ^ ((j₁ + j₂ + j₃) / 2) * threeJ j₁ j₂ j₃ m₁ m₂ m₃
```

lemma threeJ_cycle [source]

Cyclic permutation of the columns: (j₂ j₃ j₁; m₂ m₃ m₁) = (j₁ j₂ j₃; m₁ m₂ m₃).

```
lemma threeJ_cycle (j₁ j₂ j₃ : ℕ) (m₁ m₂ m₃ : ℤ) :
    threeJ j₂ j₃ j₁ m₂ m₃ m₁ = threeJ j₁ j₂ j₃ m₁ m₂ m₃
```

lemma threeJ_neg [source]

Reversal of the projections: (j₁ j₂ j₃; -m₁ -m₂ -m₃) = (-1)^(j₁+j₂+j₃) (j₁ j₂ j₃; m₁ m₂ m₃).

```
lemma threeJ_neg (j₁ j₂ j₃ : ℕ) (m₁ m₂ m₃ : ℤ) :
    threeJ j₁ j₂ j₃ (-m₁) (-m₂) (-m₃) = (-1) ^ ((j₁ + j₂ + j₃) / 2) * threeJ j₁ j₂ j₃ m₁ m₂ m₃
```

lemma wigner3j_eq_threeJ [source]

The 3j symbols of `Wigner3j` in terms of `threeJ`.

```
lemma wigner3j_eq_threeJ {m n k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hl : l ≤ m + n - 2 * k) (hα : α ≤ m) (hβ : β ≤ n) :
    wigner3j m n k l α β = threeJ m n (m + n - 2 * k) ((m : ℤ) - 2 * α) ((n : ℤ) - 2 * β)
      (-(((m + n - 2 * k : ℕ) : ℤ) - 2 * l))
```

def sixJTerm [source]

The summand of `sixJSym`: `(-1)^(Σ (jᵢ - mᵢ)) (j₁ j₂ j₃; -m₁ -m₂ -m₃) (j₁ j₅ j₆; m₁ -m₅ m₆) (j₄ j₂ j₆; m₄ m₂ -m₆) (j₄ j₅ j₃; -m₄ m₅ m₃)`.

```
def sixJTerm (j₁ j₂ j₃ j₄ j₅ j₆ : ℕ) (m : Fin 6 → ℤ) : ℝ
```

def sixJBox [source]

The projections `|mᵢ| ≤ jᵢ`.

```
def sixJBox (j₁ j₂ j₃ j₄ j₅ j₆ : ℕ) : Finset (Fin 6 → ℤ)
```

lemma mem_sixJBox [source]

```
lemma mem_sixJBox {j₁ j₂ j₃ j₄ j₅ j₆ : ℕ} {m : Fin 6 → ℤ} :
    m ∈ sixJBox j₁ j₂ j₃ j₄ j₅ j₆ ↔ (-(j₁ : ℤ) ≤ m 0 ∧ m 0 ≤ j₁) ∧
      (-(j₂ : ℤ) ≤ m 1 ∧ m 1 ≤ j₂) ∧ (-(j₃ : ℤ) ≤ m 2 ∧ m 2 ≤ j₃) ∧
        (-(j₄ : ℤ) ≤ m 3 ∧ m 3 ≤ j₄) ∧ (-(j₅ : ℤ) ≤ m 4 ∧ m 4 ≤ j₅) ∧
          (-(j₆ : ℤ) ≤ m 5 ∧ m 5 ≤ j₆)
```

def sixJSym [source]

*The **6j symbol** `{j₁ j₂ j₃; j₄ j₅ j₆}` (doubled units) as a sum over the projections of four 3j symbols.*

```
def sixJSym (j₁ j₂ j₃ j₄ j₅ j₆ : ℕ) : ℝ
```

lemma neg_one_zpow_mul_pow_four [source]

```
lemma neg_one_zpow_mul_pow_four {S S' : ℤ} {p q r s : ℕ}
    (h : (S' + p + q + r + s) % 2 = S % 2) :
    (-1 : ℝ) ^ S' * ((-1) ^ p * (-1) ^ q * (-1) ^ r * (-1) ^ s) = (-1) ^ S
```

lemma prod_four_eq_zero_or [source]

```
lemma prod_four_eq_zero_or {F₁ F₂ F₃ F₄ : ℝ} {P : Prop}
    (h : F₁ ≠ 0 → F₂ ≠ 0 → F₃ ≠ 0 → F₄ ≠ 0 → P) : F₁ * F₂ * F₃ * F₄ = 0 ∨ P
```

lemma term_congr_1324 [source]

```
lemma term_congr_1324 {S S' : ℤ} {p q r s : ℕ} {F₁ F₂ F₃ F₄ : ℝ}
    (h : F₁ ≠ 0 → F₂ ≠ 0 → F₃ ≠ 0 → F₄ ≠ 0 → (S' + p + q + r + s) % 2 = S % 2) :
    (-1 : ℝ) ^ S * (F₁ * F₂ * F₃ * F₄) =
      (-1) ^ S' * ((-1) ^ p * F₁ * ((-1) ^ q * F₃) * ((-1) ^ r * F₂) * ((-1) ^ s * F₄))
```

lemma term_congr_1243 [source]

```
lemma term_congr_1243 {S S' : ℤ} {p q r s : ℕ} {F₁ F₂ F₃ F₄ : ℝ}
    (h : F₁ ≠ 0 → F₂ ≠ 0 → F₃ ≠ 0 → F₄ ≠ 0 → (S' + p + q + r + s) % 2 = S % 2) :
    (-1 : ℝ) ^ S * (F₁ * F₂ * F₃ * F₄) =
      (-1) ^ S' * ((-1) ^ p * F₁ * ((-1) ^ q * F₂) * ((-1) ^ r * F₄) * ((-1) ^ s * F₃))
```

lemma term_congr_4321 [source]

```
lemma term_congr_4321 {S S' : ℤ} {F₁ F₂ F₃ F₄ : ℝ}
    (h : F₁ ≠ 0 → F₂ ≠ 0 → F₃ ≠ 0 → F₄ ≠ 0 → S' % 2 = S % 2) :
    (-1 : ℝ) ^ S * (F₁ * F₂ * F₃ * F₄) = (-1) ^ S' * (F₄ * F₃ * F₂ * F₁)
```

lemma threeJ_swap₂₃ [source]

Exchange of the last two columns: (j₁ j₃ j₂; m₁ m₃ m₂) = (-1)^(j₁+j₂+j₃) (j₁ j₂ j₃; m₁ m₂ m₃).

```
lemma threeJ_swap₂₃ (j₁ j₂ j₃ : ℕ) (m₁ m₂ m₃ : ℤ) :
    threeJ j₁ j₃ j₂ m₁ m₃ m₂ = (-1) ^ ((j₁ + j₂ + j₃) / 2) * threeJ j₁ j₂ j₃ m₁ m₂ m₃
```

lemma threeJ_negCycle [source]

(j₂ j₃ j₁; -m₂ m₃ -m₁) = (-1)^(j₁+j₂+j₃) (j₁ j₂ j₃; m₁ m₂ -m₃).

```
lemma threeJ_negCycle (j₁ j₂ j₃ : ℕ) (x y z : ℤ) :
    threeJ j₂ j₃ j₁ (-y) z (-x) = (-1) ^ ((j₁ + j₂ + j₃) / 2) * threeJ j₁ j₂ j₃ x y (-z)
```

lemma sixJSym_swap₁₂ [source]

Exchange of the first two columns: {j₂ j₁ j₃; j₅ j₄ j₆} = {j₁ j₂ j₃; j₄ j₅ j₆}.

```
lemma sixJSym_swap₁₂ : sixJSym j₂ j₁ j₃ j₅ j₄ j₆ = sixJSym j₁ j₂ j₃ j₄ j₅ j₆
```

lemma sixJSym_swap₂₃ [source]

Exchange of the last two columns: {j₁ j₃ j₂; j₄ j₆ j₅} = {j₁ j₂ j₃; j₄ j₅ j₆}.

```
lemma sixJSym_swap₂₃ : sixJSym j₁ j₃ j₂ j₄ j₆ j₅ = sixJSym j₁ j₂ j₃ j₄ j₅ j₆
```

lemma sixJSym_flip₁₂ [source]

Exchange of upper and lower arguments in the first two columns: {j₄ j₅ j₃; j₁ j₂ j₆} = {j₁ j₂ j₃; j₄ j₅ j₆}.

```
lemma sixJSym_flip₁₂ : sixJSym j₄ j₅ j₃ j₁ j₂ j₆ = sixJSym j₁ j₂ j₃ j₄ j₅ j₆
```

def sixJSymEdge [source]

The 6j symbol as a function of the edge labels of the tetrahedron whose vertices are the four triads 0 = (j₁ j₂ j₃), 1 = (j₁ j₅ j₆), 2 = (j₄ j₂ j₆), 3 = (j₄ j₅ j₃).

```
def sixJSymEdge (j : Sym2 (Fin 4) → ℕ) : ℝ
```

lemma sixJSymEdge_comp_map_swap₁₂ [source]

```
lemma sixJSymEdge_comp_map_swap₁₂ (j : Sym2 (Fin 4) → ℕ) :
    sixJSymEdge (j ∘ Sym2.map (Equiv.swap 1 2)) = sixJSymEdge j
```

lemma sixJSymEdge_comp_map_swap₂₃ [source]

```
lemma sixJSymEdge_comp_map_swap₂₃ (j : Sym2 (Fin 4) → ℕ) :
    sixJSymEdge (j ∘ Sym2.map (Equiv.swap 2 3)) = sixJSymEdge j
```

lemma sixJSymEdge_comp_map_flip [source]

```
lemma sixJSymEdge_comp_map_flip (j : Sym2 (Fin 4) → ℕ) :
    sixJSymEdge (j ∘ Sym2.map (Equiv.swap 0 3 * Equiv.swap 1 2 : Equiv.Perm (Fin 4))) =
      sixJSymEdge j
```

lemma sixJSymEdge_comp_map_mul [source]

```
lemma sixJSymEdge_comp_map_mul {σ τ : Equiv.Perm (Fin 4)}
    (hσ : ∀ j, sixJSymEdge (j ∘ Sym2.map σ) = sixJSymEdge j)
    (hτ : ∀ j, sixJSymEdge (j ∘ Sym2.map τ) = sixJSymEdge j) (j : Sym2 (Fin 4) → ℕ) :
    sixJSymEdge (j ∘ Sym2.map (σ * τ : Equiv.Perm (Fin 4))) = sixJSymEdge j
```

lemma sixJSymEdge_comp_map [source]

***Tetrahedral symmetry.** The 6j symbol is invariant under every permutation of the four triads.*

```
lemma sixJSymEdge_comp_map (σ : Equiv.Perm (Fin 4)) (j : Sym2 (Fin 4) → ℕ) :
    sixJSymEdge (j ∘ Sym2.map σ) = sixJSymEdge j
```

lemma sym2_map_perm_injective [source]

The permutations of the triads act faithfully on the edges, so the symmetries of `sixJSymEdge_comp_map` are 24 distinct permutations of the arguments.

```
lemma sym2_map_perm_injective :
    Function.Injective (fun σ : Equiv.Perm (Fin 4) => Sym2.map σ)
```

lemma link_congr [source]

```
lemma link_congr {S : ℤ} {p₁ p₂ p₃ p₄ q₁ q₂ q₃ q₄ r : ℕ} {A B C T₁ T₂ T₃ T₄ : ℝ}
    (hA : 0 ≤ A) (hB : 0 ≤ B)
    (h : T₁ ≠ 0 → T₂ ≠ 0 → T₃ ≠ 0 → T₄ ≠ 0 →
      (S + p₁ + p₂ + p₃ + p₄ + q₁ + q₂ + q₃ + q₄ + r) % 2 = 0) :
    (-1) ^ p₂ * Real.sqrt B * T₂ * ((-1) ^ p₁ * Real.sqrt A * T₁ *
        ((-1) ^ p₄ * Real.sqrt B * T₄ * ((-1) ^ p₃ * Real.sqrt C * T₃))) =
      Real.sqrt (A * C) * B * (-1) ^ r * ((-1) ^ S *
        ((-1) ^ q₁ * T₁ * ((-1) ^ q₄ * T₄) * ((-1) ^ q₃ * T₃) * ((-1) ^ q₂ * T₂)))
```

lemma tripleForm_normVec_stateB [source]

```
lemma tripleForm_normVec_stateB (hbc : 0 < b + c) (k₃ : Fin (min b c + 1)) (k₄ l : ℕ)
    {α : ℕ} (hα : α ≤ a) (β γ : ℕ) :
    tripleForm a b c ((normVec a α ⊗ₜ normVec b β) ⊗ₜ normVec c γ) (stateB hbc k₃ k₄ l) =
      ∑ ν ∈ Finset.range (b + c - 2 * (k₃ : ℕ) + 1),
        cgCoefficient a (b + c - 2 * (k₃ : ℕ)) k₄ l α ν * cgCoefficient b c k₃ ν β γ
```

lemma recoupling_eq_sum [source]

The recoupling coefficient as a sum over products of four Clebsch–Gordan coefficients.

```
lemma recoupling_eq_sum (hab : 0 < a + b) (hbc : 0 < b + c) (K l : ℕ) (k₁ : Fin (min a b + 1))
    (k₃ : Fin (min b c + 1)) :
    recoupling hab hbc K l k₁ k₃ =
      ∑ μ ∈ Finset.range (a + b - 2 * (k₁ : ℕ) + 1), ∑ γ ∈ Finset.range (c + 1),
        cgCoefficient (a + b - 2 * (k₁ : ℕ)) c (K - k₁) l μ γ *
          ∑ α ∈ Finset.range (a + 1), ∑ β ∈ Finset.range (b + 1),
            cgCoefficient a b k₁ μ α β *
              ∑ ν ∈ Finset.range (b + c - 2 * (k₃ : ℕ) + 1),
                cgCoefficient a (b + c - 2 * (k₃ : ℕ)) (K - k₃) l α ν *
                  cgCoefficient b c k₃ ν β γ
```

lemma cg_four_eq [source]

One summand of `recoupling_eq_sum` is a summand of `sixJSym`.

```
lemma cg_four_eq {a b c e f g k₁ k₂ k₃ k₄ l μ γ α β ν : ℕ}
    (hab : 0 < a + b) (hbc : 0 < b + c) (hec : 0 < e + c) (haf : 0 < a + f)
    (he : e = a + b - 2 * k₁) (hf : f = b + c - 2 * k₃) (hg₂ : g = e + c - 2 * k₂)
    (hg₄ : g = a + f - 2 * k₄) (h₁a : k₁ ≤ a) (h₁b : k₁ ≤ b) (h₂e : k₂ ≤ e) (h₂c : k₂ ≤ c)
    (h₃b : k₃ ≤ b) (h₃c : k₃ ≤ c) (h₄a : k₄ ≤ a) (h₄f : k₄ ≤ f) (hl : l ≤ g) (hμ : μ ≤ e)
    (hγ : γ ≤ c) (hα : α ≤ a) (hβ : β ≤ b) (hν : ν ≤ f) :
    cgCoefficient e c k₂ l μ γ * (cgCoefficient a b k₁ μ α β *
        (cgCoefficient a f k₄ l α ν * cgCoefficient b c k₃ ν β γ)) =
      Real.sqrt (((e + 1 : ℕ) : ℝ) * ((f + 1 : ℕ) : ℝ)) * ((g + 1 : ℕ) : ℝ) *
        (-1) ^ (a + b + c - (k₁ + k₂)) *
          sixJTerm a b e c g f ![(a : ℤ) - 2 * α, (b : ℤ) - 2 * β, -((e : ℤ) - 2 * μ),
            (c : ℤ) - 2 * γ, (g : ℤ) - 2 * l, (f : ℤ) - 2 * ν]
```

lemma sum_sixJTerm_eq [source]

The sum of `sixJTerm` over the six projections, written through `jᵢ - 2·index`.

```
lemma sum_sixJTerm_eq (a b c e f g : ℕ) :
    ∑ x ∈ Finset.range (g + 1) ×ˢ (Finset.range (e + 1) ×ˢ (Finset.range (c + 1) ×ˢ
        (Finset.range (a + 1) ×ˢ (Finset.range (b + 1) ×ˢ Finset.range (f + 1))))),
      sixJTerm a b e c g f ![(a : ℤ) - 2 * x.2.2.2.1, (b : ℤ) - 2 * x.2.2.2.2.1,
        -((e : ℤ) - 2 * x.2.1), (c : ℤ) - 2 * x.2.2.1, (g : ℤ) - 2 * x.1,
        (f : ℤ) - 2 * x.2.2.2.2.2] = sixJSym a b e c g f
```

lemma sixJ_eq_sixJSym [source]

***The 6j symbols of `SixJ` are the symmetric sums over four 3j symbols.** With `j₁₂ = (a + b)/2 - k₁`, `j₂₃ = (b + c)/2 - k₃` and `J = (a + b + c)/2 - K`, `{j₁ j₂ j₁₂; j₃ J j₂₃}` of `sixJ` equals `sixJSym`.*

```
lemma sixJ_eq_sixJSym (hab : 0 < a + b) (hbc : 0 < b + c) (ha : 0 < a) (hc : 0 < c) {K : ℕ}
    {k₁ : Fin (min a b + 1)} {k₃ : Fin (min b c + 1)} (hA : ValidA a b c K k₁)
    (hB : ValidB a b c K k₃) :
    sixJ hab hbc K k₁ k₃ =
      sixJSym a b (a + b - 2 * (k₁ : ℕ)) c (a + b + c - 2 * K) (b + c - 2 * (k₃ : ℕ))
```

2.573 PhyslibAlpha.QuantumMechanics.AngularMomentum.SpinOrbit

PhyslibAlpha/QuantumMechanics/AngularMomentum/SpinOrbit.lean on GitHub.

def dot [source]

The scalar product `L · S = Lx ⊗ Sx + Ly ⊗ Sy + Lz ⊗ Sz` on `V ⊗ W`.

```
def dot (ρ₁ : AngularMomentumRepresentation V) (ρ₂ : AngularMomentumRepresentation W) :
    Module.End ℂ (V ⊗[ℂ] W)
```

lemma Jsq_tensor [source]

`J² = L² + S² + 2 L · S`.

```
lemma Jsq_tensor (ρ₁ : AngularMomentumRepresentation V) (ρ₂ : AngularMomentumRepresentation W)
    :
    (tensor ρ₁ ρ₂).Jsq = ρ₁.Jsq.rTensor W + ρ₂.Jsq.lTensor V + (2 : ℂ) • dot ρ₁ ρ₂
```

lemma Jsq_standard [source]

`J² = j(j + 1)` on the spin-`j` representation `V(2j)`.

```
lemma Jsq_standard (n : ℕ) :
    (standard n).Jsq =
      ((n : ℂ) / 2 * ((n : ℂ) / 2 + 1)) • (1 : Module.End ℂ (StandardSl2Module ℂ n))
```

def spinOrbitValue [source]

The eigenvalue `(J(J + 1) - l(l + 1) - s(s + 1)) / 2` of `L · S` for `l = m/2`, `s = n/2` and `J = (m + n)/2 - k`.

```
def spinOrbitValue (m n k : ℕ) : ℂ
```

lemma dot_coupled_stringVec [source]

***Spin–orbit eigenvalues.** Every vector of the `k`-th lowering string of `V(m) ⊗ V(n)`, a state of total angular momentum `J = (m + n)/2 - k`, is an eigenvector of `L · S` with eigenvalue `(J(J + 1) - l(l + 1) - s(s + 1)) / 2`.*

```
lemma dot_coupled_stringVec {k : ℕ} (hmn : 0 < m + n) (hkm : k ≤ m) (hkn : k ≤ n) (l : ℕ) :
    dot (standard m) (standard n) (stringVec m n k l) =
      spinOrbitValue m n k • stringVec m n k l
```

lemma dot_coupled_stringVec_doublet_upper [source]

***Fine-structure doublet, upper level.** For spin `1/2` and `J = l + 1/2`, `L · S = l/2` `(l = m/2)`.*

```
lemma dot_coupled_stringVec_doublet_upper (l' : ℕ) :
    dot (standard m) (standard 1) (stringVec m 1 0 l') =
      ((m : ℂ) / 4) • stringVec m 1 0 l'
```

lemma dot_coupled_stringVec_doublet_lower [source]

***Fine-structure doublet, lower level.** For spin `1/2` and `J = l - 1/2`, `L · S = -(l + 1)/2` (`l = m/2`, `m ≥ 1`).*

```
lemma dot_coupled_stringVec_doublet_lower (hm : 1 ≤ m) (l' : ℕ) :
    dot (standard m) (standard 1) (stringVec m 1 1 l') =
      (-((m : ℂ) + 2) / 4) • stringVec m 1 1 l'
```

lemma spinOrbitValue_doublet_sub [source]

The doublet splitting `l/2 + (l + 1)/2 = (2l + 1)/2`.

```
lemma spinOrbitValue_doublet_sub (hm : 1 ≤ m) :
    spinOrbitValue m 1 0 - spinOrbitValue m 1 1 = ((m : ℂ) + 1) / 2
```

lemma spinOrbitValue_sub_succ [source]

***Landé's interval rule.** The levels `J` and `J - 1` of a multiplet differ in `L · S` by `J`, `J = (m + n)/2 - k`.*

```
lemma spinOrbitValue_sub_succ {k : ℕ} (hkm : k + 1 ≤ m) (hkn : k + 1 ≤ n) :
    spinOrbitValue m n k - spinOrbitValue m n (k + 1) = ((m + n - 2 * k : ℕ) : ℂ) / 2
```

2.574 PhyslibAlpha.QuantumMechanics.AngularMomentum.Wigner3j

PhyslibAlpha/QuantumMechanics/AngularMomentum/Wigner3j.lean on GitHub.

def invFact [source]

`1/u!` for `u ≥ 0` and `0` for `u < 0`.

```
def invFact (u : ℤ) : ℝ
```

lemma invFact_natCast [source]

```
lemma invFact_natCast (t : ℕ) : invFact t = ((t.factorial : ℝ))⁻¹
```

lemma invFact_eq [source]

```
lemma invFact_eq {u : ℤ} {t : ℕ} (h : u = t) : invFact u = ((t.factorial : ℝ))⁻¹
```

lemma invFact_of_neg [source]

```
lemma invFact_of_neg {u : ℤ} (h : u < 0) : invFact u = 0
```

lemma nonneg_of_invFact_ne_zero [source]

```
lemma nonneg_of_invFact_ne_zero {u : ℤ} (h : invFact u ≠ 0) : 0 ≤ u
```

lemma neg_one_zpow_congr [source]

```
lemma neg_one_zpow_congr {p q : ℤ} (h : p % 2 = q % 2) : (-1 : ℝ) ^ p = (-1) ^ q
```

def sixTerm [source]

The summand of `sixSum`.

```
def sixTerm (a₁ a₂ a₃ b₁ b₂ b₃ z : ℤ) : ℝ
```

def sixSum [source]

`Σ_z (-1)^z / ((z - a₁)! (z - a₂)! (z - a₃)! (b₁ - z)! (b₂ - z)! (b₃ - z)!)`, over the `z` for which all six arguments are nonnegative.

```
def sixSum (a₁ a₂ a₃ b₁ b₂ b₃ : ℤ) : ℝ
```

lemma sixSum_swap_a12 [source]

```
lemma sixSum_swap_a12 (a₁ a₂ a₃ b₁ b₂ b₃ : ℤ) :
    sixSum a₁ a₂ a₃ b₁ b₂ b₃ = sixSum a₂ a₁ a₃ b₁ b₂ b₃
```

lemma sixSum_swap_a23 [source]

```
lemma sixSum_swap_a23 (a₁ a₂ a₃ b₁ b₂ b₃ : ℤ) :
    sixSum a₁ a₂ a₃ b₁ b₂ b₃ = sixSum a₁ a₃ a₂ b₁ b₂ b₃
```

lemma sixSum_swap_b12 [source]

```
lemma sixSum_swap_b12 (a₁ a₂ a₃ b₁ b₂ b₃ : ℤ) :
    sixSum a₁ a₂ a₃ b₁ b₂ b₃ = sixSum a₁ a₂ a₃ b₂ b₁ b₃
```

lemma sixSum_swap_b23 [source]

```
lemma sixSum_swap_b23 (a₁ a₂ a₃ b₁ b₂ b₃ : ℤ) :
    sixSum a₁ a₂ a₃ b₁ b₂ b₃ = sixSum a₁ a₂ a₃ b₁ b₃ b₂
```

lemma sixSum_shift [source]

Shifting all arguments by `c` multiplies the sum by `(-1)^c`.

```
lemma sixSum_shift (a₁ a₂ a₃ b₁ b₂ b₃ c : ℤ) :
    sixSum (a₁ + c) (a₂ + c) (a₃ + c) (b₁ + c) (b₂ + c) (b₃ + c) =
      (-1) ^ c * sixSum a₁ a₂ a₃ b₁ b₂ b₃
```

lemma sixSum_neg [source]

The reflection `z ↦ -z`.

```
lemma sixSum_neg (a₁ a₂ a₃ b₁ b₂ b₃ : ℤ) :
    sixSum a₁ a₂ a₃ b₁ b₂ b₃ = sixSum (-b₁) (-b₂) (-b₃) (-a₁) (-a₂) (-a₃)
```

def IsMagic [source]

All rows and columns of `R` have the same sum as the first row.

```
def IsMagic (R : Fin 3 → Fin 3 → ℕ) : Prop
```

def threeJR [source]

The 3j symbol of a Regge square `R`, by Racah's formula.

```
def threeJR (R : Fin 3 → Fin 3 → ℕ) : ℝ
```

lemma neg_one_pow_mul_mul_zpow [source]

```
lemma neg_one_pow_mul_mul_zpow {n n' q : ℕ} {s : ℤ} (X Y : ℝ)
    (h : ((n' : ℤ) + s) % 2 = ((n : ℤ) + q) % 2) :
    (-1 : ℝ) ^ n' * X * ((-1) ^ s * Y) = (-1) ^ q * ((-1) ^ n * X * Y)
```

lemma neg_one_pow_mul_mul_zpow' [source]

```
lemma neg_one_pow_mul_mul_zpow' {n n' : ℕ} {s : ℤ} (X Y : ℝ)
    (h : ((n' : ℤ) + s) % 2 = (n : ℤ) % 2) :
    (-1 : ℝ) ^ n' * X * ((-1) ^ s * Y) = (-1) ^ n * X * Y
```

lemma threeJR_transpose [source]

***Regge symmetry.** Transposing the square leaves the 3j symbol unchanged.*

```
lemma threeJR_transpose {R : Fin 3 → Fin 3 → ℕ} (hR : IsMagic R) :
    threeJR (fun i j => R j i) = threeJR R
```

lemma threeJR_swap_columns [source]

Swapping the first two columns multiplies the 3j symbol by `(-1)^J`.

```
lemma threeJR_swap_columns {R : Fin 3 → Fin 3 → ℕ} (hR : IsMagic R) :
    threeJR (fun i => ![R i 1, R i 0, R i 2]) = (-1) ^ (R 0 0 + R 0 1 + R 0 2) * threeJR R
```

lemma threeJR_cycle_columns [source]

Permuting the columns cyclically leaves the 3j symbol unchanged.

```
lemma threeJR_cycle_columns {R : Fin 3 → Fin 3 → ℕ} (hR : IsMagic R) :
    threeJR (fun i => ![R i 1, R i 2, R i 0]) = threeJR R
```

lemma threeJR_swap_rows [source]

Swapping the last two rows, `m ↦ -m`, multiplies the 3j symbol by `(-1)^J`.

```
lemma threeJR_swap_rows {R : Fin 3 → Fin 3 → ℕ} (hR : IsMagic R) :
    threeJR ![R 0, R 2, R 1] = (-1) ^ (R 0 0 + R 0 1 + R 0 2) * threeJR R
```

lemma isMagic_iff [source]

```
lemma isMagic_iff {R : Fin 3 → Fin 3 → ℕ} :
    IsMagic R ↔ (∀ i, ∑ j, R i j = ∑ j, R 0 j) ∧ ∀ j, ∑ i, R i j = ∑ j, R 0 j
```

lemma isMagic_perm [source]

```
lemma isMagic_perm {R : Fin 3 → Fin 3 → ℕ} (hR : IsMagic R) (σ τ : Equiv.Perm (Fin 3)) :
    IsMagic (fun i j => R (σ i) (τ j))
```

lemma isMagic_transpose [source]

```
lemma isMagic_transpose {R : Fin 3 → Fin 3 → ℕ} (hR : IsMagic R) :
    IsMagic (fun i j => R j i)
```

lemma isMagic_perm_sum [source]

Under a permutation of rows and columns the sum `J` of a magic square is unchanged.

```
lemma isMagic_perm_sum {R : Fin 3 → Fin 3 → ℕ} (hR : IsMagic R) (σ τ : Equiv.Perm (Fin 3)) :
    R (σ 0) (τ 0) + R (σ 0) (τ 1) + R (σ 0) (τ 2) = R 0 0 + R 0 1 + R 0 2
```

lemma isMagic_transpose_sum [source]

```
lemma isMagic_transpose_sum {R : Fin 3 → Fin 3 → ℕ} (hR : IsMagic R) :
    R 0 0 + R 1 0 + R 2 0 = R 0 0 + R 0 1 + R 0 2
```

lemma threeJR_perm_columns [source]

A permutation `τ` of the columns multiplies the 3j symbol by `sign(τ)^J`.

```
lemma threeJR_perm_columns (τ : Equiv.Perm (Fin 3)) {R : Fin 3 → Fin 3 → ℕ} (hR : IsMagic R) :
    threeJR (fun i j => R i (τ j)) =
      ((Equiv.Perm.sign τ : ℤ) : ℝ) ^ (R 0 0 + R 0 1 + R 0 2) * threeJR R
```

lemma threeJR_perm [source]

***Symmetries of the 3j symbol.** Permuting the rows by `σ` and the columns by `τ` multiplies the 3j symbol by `(sign(σ) sign(τ))^J`.*

```
lemma threeJR_perm (σ τ : Equiv.Perm (Fin 3)) {R : Fin 3 → Fin 3 → ℕ} (hR : IsMagic R) :
    threeJR (fun i j => R (σ i) (τ j)) =
      ((Equiv.Perm.sign σ * Equiv.Perm.sign τ : ℤˣ) : ℝ) ^ (R 0 0 + R 0 1 + R 0 2) *
        threeJR R
```

lemma perm_prodMap_injective [source]

Distinct pairs of row and column permutations move the entries of a square differently.

```
lemma perm_prodMap_injective :
    Function.Injective (fun p : Equiv.Perm (Fin 3) × Equiv.Perm (Fin 3) =>
      Prod.map (p.1 : Fin 3 → Fin 3) p.2)
```

lemma prodMap_ne_swap [source]

Transposition is not a permutation of rows and columns.

```
lemma prodMap_ne_swap (σ τ : Equiv.Perm (Fin 3)) :
    Prod.map (σ : Fin 3 → Fin 3) τ ≠ Prod.swap
```

def regge [source]

The Regge square of `(j₁ j₂ J; m₁ m₂ -M)` in the parameters of `ClebschGordanCoefficients`: `m = 2j₁`, `n = 2j₂`, `J = (m + n)/2 - k`, `M = J - l`, `m₁ = j₁ - α`, `m₂ = j₂ - β`.

```
def regge (m n k l α β : ℕ) : Fin 3 → Fin 3 → ℕ
```

lemma isMagic_regge [source]

```
lemma isMagic_regge {k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (hl : l ≤ m + n - 2 * k)
    (hα : α ≤ m) (hβ : β ≤ n) (h : α + β = k + l) : IsMagic (regge m n k l α β)
```

lemma sixSum_regge_eq_racahSum [source]

Racah's sum `racahSum` of `ClebschGordanFormula` is the six-factorial sum of the square.

```
lemma sixSum_regge_eq_racahSum {k α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (hβ : β ≤ n) :
    sixSum 0 ((α : ℤ) - (m - k : ℕ)) (((n - β : ℕ) : ℤ) - (n - k : ℕ)) k α (n - β : ℕ) =
      racahSum m n k α β
```

lemma cgCoefficient_eq_threeJR [source]

***The 3j symbols and the Clebsch–Gordan coefficients.** `(j₁ m₁ j₂ m₂ | J M) = (-1)^(j₁ - j₂ + M) √(2J + 1) (j₁ j₂ J; m₁ m₂ -M)`, where `j₁ - j₂ + M = m - k - l ≡ m + k + l (mod 2)`.*

```
lemma cgCoefficient_eq_threeJR {k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hl : l ≤ m + n - 2 * k) (hα : α ≤ m) (hβ : β ≤ n) (h : α + β = k + l) :
    cgCoefficient m n k l α β = (-1) ^ (m + k + l) *
      Real.sqrt ((m + n - 2 * k + 1 : ℕ) : ℝ) * threeJR (regge m n k l α β)
```

def wigner3j [source]

The 3j symbol `(j₁ j₂ J; m₁ m₂ -M)` in the parameters of `ClebschGordanCoefficients`, zero unless `m₁ + m₂ = M`.

```
def wigner3j (m n k l α β : ℕ) : ℝ
```

lemma cgCoefficient_eq_wigner3j [source]

```
lemma cgCoefficient_eq_wigner3j (hmn : 0 < m + n) {k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hl : l ≤ m + n - 2 * k) (hα : α ≤ m) (hβ : β ≤ n) :
    cgCoefficient m n k l α β = (-1) ^ (m + k + l) *
      Real.sqrt ((m + n - 2 * k + 1 : ℕ) : ℝ) * wigner3j m n k l α β
```

lemma sum_wigner3j_mul_wigner3j [source]

***Orthogonality.** `Σ_{m₁ m₂} (j₁ j₂ J; m₁ m₂ -M) (j₁ j₂ J'; m₁ m₂ -M') = δ_{JJ'} δ_{MM'} / (2J + 1)`.*

```
lemma sum_wigner3j_mul_wigner3j (hmn : 0 < m + n) {k k' l l' : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hk'm : k' ≤ m) (hk'n : k' ≤ n) (hl : l ≤ m + n - 2 * k) (hl' : l' ≤ m + n - 2 * k') :
    ∑ a ∈ Finset.range (m + 1), ∑ b ∈ Finset.range (n + 1),
      wigner3j m n k l a b * wigner3j m n k' l' a b =
        if k = k' ∧ l = l' then 1 / ((m + n - 2 * k + 1 : ℕ) : ℝ) else 0
```

lemma sum_wigner3j_mul_wigner3j_JM [source]

***Second orthogonality relation.** `Σ_{J M} (2J + 1) (j₁ j₂ J; m₁ m₂ -M) (j₁ j₂ J; m₁' m₂' -M) = δ_{m₁ m₁'} δ_{m₂ m₂'}`.*

```
lemma sum_wigner3j_mul_wigner3j_JM (hmn : 0 < m + n) {a b a' b' : ℕ} (ha : a ≤ m) (hb : b ≤ n)
    (ha' : a' ≤ m) (hb' : b' ≤ n) :
    ∑ k ∈ Finset.range (min m n + 1), ∑ l ∈ Finset.range (m + n - 2 * k + 1),
      ((m + n - 2 * k + 1 : ℕ) : ℝ) * wigner3j m n k l a b * wigner3j m n k l a' b' =
        if a = a' ∧ b = b' then 1 else 0
```

2.575 PhyslibAlpha.QuantumMechanics.AngularMomentum.WignerEckart

PhyslibAlpha/QuantumMechanics/AngularMomentum/WignerEckart.lean on GitHub.

structure IsTensorOperator [source]

A tensor operator of rank `k` from `V(m)` to `V(p)`: a linear map `T : V(k) → Hom(V(m), V(p))` with `[X, T x] = T (X x)` for `X = e, f, h`.

```
structure IsTensorOperator
    (T : StandardSl2Module K k →ₗ[K] StandardSl2Module K m →ₗ[K] StandardSl2Module K p) :
    Prop where
  comm_E : ∀ x, ρE ∘ₗ T x - T x ∘ₗ ρE = T (ρE x)
  comm_F : ∀ x, ρF ∘ₗ T x - T x ∘ₗ ρF = T (ρF x)
  comm_H : ∀ x, ρH ∘ₗ T x - T x ∘ₗ ρH = T (ρH x)
```

lemma apply_ρE [source]

Leibniz form of the commutation relation for the raising operator.

```
lemma apply_ρE (hT : IsTensorOperator T) (x : StandardSl2Module K k)
    (y : StandardSl2Module K m) : ρE (T x y) = T (ρE x) y + T x (ρE y)
```

lemma apply_ρF [source]

Leibniz form of the commutation relation for the lowering operator.

```
lemma apply_ρF (hT : IsTensorOperator T) (x : StandardSl2Module K k)
    (y : StandardSl2Module K m) : ρF (T x y) = T (ρF x) y + T x (ρF y)
```

lemma apply_ρH [source]

Leibniz form of the commutation relation for the weight operator.

```
lemma apply_ρH (hT : IsTensorOperator T) (x : StandardSl2Module K k)
    (y : StandardSl2Module K m) : ρH (T x y) = T (ρH x) y + T x (ρH y)
```

lemma lift_isTensorIntertwiner [source]

The map `x ⊗ y ↦ T x y` of a tensor operator is a tensor intertwiner.

```
lemma lift_isTensorIntertwiner (hT : IsTensorOperator T) :
    IsTensorIntertwiner (TensorProduct.lift T) where
```

lemma eq_zero_of_forall_ne [source]

***Triangle and parity rule.** If `p` is not of the form `k + m - 2j` with `j ≤ min k m`, every tensor operator of rank `k` from `V(m)` to `V(p)` is zero.*

```
lemma eq_zero_of_forall_ne (hkm : 0 < k + m)
    {T : StandardSl2Module K k →ₗ[K] StandardSl2Module K m →ₗ[K] StandardSl2Module K p}
    (hT : IsTensorOperator T) (hp : ∀ j : Fin (min k m + 1), k + m - 2 * (j : ℕ) ≠ p) :
    T = 0
```

lemma exists_eq_smul_cgProj [source]

***Wigner–Eckart theorem.** A tensor operator of rank `k` from `V(m)` to `V(k + m - 2j)` factorises as `T x y = c • cgProj (x ⊗ y)`: a single reduced matrix element `c` times the Clebsch–Gordan projection.*

```
lemma exists_eq_smul_cgProj (hkm : 0 < k + m) {j : Fin (min k m + 1)}
    {T : StandardSl2Module K k →ₗ[K] StandardSl2Module K m →ₗ[K]
      StandardSl2Module K (k + m - 2 * (j : ℕ))}
    (hT : IsTensorOperator T) :
    ∃ c : K, ∀ x y, T x y = c • cgProj hkm j (x ⊗ₜ y)
```

lemma exists_coord_eq [source]

***Wigner–Eckart theorem**, matrix-element form: every matrix element `⟨v_s | T v_q | v_r⟩` is the reduced matrix element `c` times the Clebsch–Gordan coefficient, the `v_s`-coordinate of `cgProj (v_q ⊗ v_r)`.*

```
lemma exists_coord_eq (hkm : 0 < k + m) {j : Fin (min k m + 1)}
    {T : StandardSl2Module K k →ₗ[K] StandardSl2Module K m →ₗ[K]
      StandardSl2Module K (k + m - 2 * (j : ℕ))}
    (hT : IsTensorOperator T) :
    ∃ c : K, ∀ (q r : ℕ) (s : Fin (k + m - 2 * (j : ℕ) + 1)),
      T (vNat k q) (vNat m r) s = c * cgProj hkm j (vNat k q ⊗ₜ vNat m r) s
```

lemma apply_vNat_eq_zero_of_weight_ne [source]

***Weight selection rule.** The matrix element `⟨v_s | T v_q | v_r⟩` vanishes unless the weights add up, `(k - 2q) + (m - 2r) = p - 2s`. In physics notation this is `μ + m_j = m_{j'}`.*

```
lemma apply_vNat_eq_zero_of_weight_ne
    {T : StandardSl2Module K k →ₗ[K] StandardSl2Module K m →ₗ[K] StandardSl2Module K p}
    (hT : IsTensorOperator T) (q r : ℕ) (s : Fin (p + 1))
    (hne : (k : ℤ) - 2 * q + ((m : ℤ) - 2 * r) ≠ (p : ℤ) - 2 * (s : ℕ)) :
    T (vNat k q) (vNat m r) s = 0
```

2.576 PhyslibAlpha.QuantumMechanics.AngularMomentum.WignerEckartIrreducible

PhyslibAlpha/QuantumMechanics/AngularMomentum/WignerEckartIrreducible.lean on GitHub.

lemma standard_bracket_eSub [source]

```
lemma standard_bracket_eSub (t : IsSl2Triple h e f) (w : StandardSl2Module K n) :
    letI
```

lemma standard_bracket_fSub [source]

```
lemma standard_bracket_fSub (t : IsSl2Triple h e f) (w : StandardSl2Module K n) :
    letI
```

lemma standard_bracket_hSub [source]

```
lemma standard_bracket_hSub (t : IsSl2Triple h e f) (w : StandardSl2Module K n) :
    letI
```

structure IsStandardEquiv [source]

A linear equivalence `V ≃ V(n)` carrying `J₊`, `J₋` and `2Jz` to `e`, `f` and `h`.

```
structure IsStandardEquiv (ρ : AngularMomentumRepresentation V) {n : ℕ}
    (ψ : V ≃ₗ[ℂ] StandardSl2Module ℂ n) : Prop where
  map_Jp : ∀ v, ψ (ρ.Jp v) = ρE (ψ v)
  map_Jm : ∀ v, ψ (ρ.Jm v) = ρF (ψ v)
  map_Jz : ∀ v, ψ (((2 : ℂ) • ρ.Jz) v) = ρH (ψ v)
```

lemma exists_isStandardEquiv [source]

A finite-dimensional irreducible representation with `Jz ≠ 0` is isomorphic to a standard module by a standard equivalence.

```
lemma exists_isStandardEquiv [FiniteDimensional ℂ V] (ρ : AngularMomentumRepresentation V)
    (hz : ρ.Jz ≠ 0) (hirr : ρ.IsIrreducible) :
    ∃ (n : ℕ) (ψ : V ≃ₗ[ℂ] StandardSl2Module ℂ n), ρ.IsStandardEquiv ψ
```

lemma pos [source]

A standard equivalence for a representation with `Jz ≠ 0` lands in some `V(n)` with `n > 0`.

```
lemma IsStandardEquiv.pos {ρ : AngularMomentumRepresentation V} {n : ℕ}
    {ψ : V ≃ₗ[ℂ] StandardSl2Module ℂ n} (hψ : ρ.IsStandardEquiv ψ) (hz : ρ.Jz ≠ 0) : 0 < n
```

lemma symm_ρE [source]

```
lemma IsStandardEquiv.symm_ρE {ρ : AngularMomentumRepresentation V} {n : ℕ}
    {ψ : V ≃ₗ[ℂ] StandardSl2Module ℂ n} (hψ : ρ.IsStandardEquiv ψ) (y : StandardSl2Module ℂ n) :

    ψ.symm (ρE y) = ρ.Jp (ψ.symm y)
```

lemma symm_ρF [source]

```
lemma IsStandardEquiv.symm_ρF {ρ : AngularMomentumRepresentation V} {n : ℕ}
    {ψ : V ≃ₗ[ℂ] StandardSl2Module ℂ n} (hψ : ρ.IsStandardEquiv ψ) (y : StandardSl2Module ℂ n) :

    ψ.symm (ρF y) = ρ.Jm (ψ.symm y)
```

lemma symm_ρH [source]

```
lemma IsStandardEquiv.symm_ρH {ρ : AngularMomentumRepresentation V} {n : ℕ}
    {ψ : V ≃ₗ[ℂ] StandardSl2Module ℂ n} (hψ : ρ.IsStandardEquiv ψ) (y : StandardSl2Module ℂ n) :

    ψ.symm (ρH y) = ((2 : ℂ) • ρ.Jz) (ψ.symm y)
```

structure IsTensorOperatorOf [source]

A tensor operator of rank `k` from `ρ₁` on `V` to `ρ₂` on `W`: a linear map `T : V(k) → Hom(V, W)` with `[J₊, T x] = T (e x)`, `[J₋, T x] = T (f x)` and `[2Jz, T x] = T (h x)`.

```
structure IsTensorOperatorOf (ρ₁ : AngularMomentumRepresentation V)
    (ρ₂ : AngularMomentumRepresentation W) (T : StandardSl2Module ℂ k →ₗ[ℂ] V →ₗ[ℂ] W) :
    Prop where
  comm_p : ∀ x, ρ₂.Jp ∘ₗ T x - T x ∘ₗ ρ₁.Jp = T (ρE x)
  comm_m : ∀ x, ρ₂.Jm ∘ₗ T x - T x ∘ₗ ρ₁.Jm = T (ρF x)
  comm_z : ∀ x, ((2 : ℂ) • ρ₂.Jz) ∘ₗ T x - T x ∘ₗ ((2 : ℂ) • ρ₁.Jz) = T (ρH x)
```

def conjStd [source]

Conjugation of an operator-valued map by linear equivalences `V ≃ V(n₁)` and `W ≃ V(n₂)`.

```
def conjStd {n₁ n₂ : ℕ} (ψ₁ : V ≃ₗ[ℂ] StandardSl2Module ℂ n₁)
    (ψ₂ : W ≃ₗ[ℂ] StandardSl2Module ℂ n₂) (T : StandardSl2Module ℂ k →ₗ[ℂ] V →ₗ[ℂ] W) :
    StandardSl2Module ℂ k →ₗ[ℂ] StandardSl2Module ℂ n₁ →ₗ[ℂ] StandardSl2Module ℂ n₂
```

lemma conjStd_apply [source]

```
lemma conjStd_apply {n₁ n₂ : ℕ} (ψ₁ : V ≃ₗ[ℂ] StandardSl2Module ℂ n₁)
    (ψ₂ : W ≃ₗ[ℂ] StandardSl2Module ℂ n₂) (T : StandardSl2Module ℂ k →ₗ[ℂ] V →ₗ[ℂ] W)
    (x : StandardSl2Module ℂ k) (y : StandardSl2Module ℂ n₁) :
    conjStd ψ₁ ψ₂ T x y = ψ₂ (T x (ψ₁.symm y))
```

lemma conjStd_injective [source]

```
lemma conjStd_injective {n₁ n₂ : ℕ} (ψ₁ : V ≃ₗ[ℂ] StandardSl2Module ℂ n₁)
    (ψ₂ : W ≃ₗ[ℂ] StandardSl2Module ℂ n₂) {T S : StandardSl2Module ℂ k →ₗ[ℂ] V →ₗ[ℂ] W}
    (h : conjStd ψ₁ ψ₂ T = conjStd ψ₁ ψ₂ S) : T = S
```

lemma isTensorOperator_conjStd [source]

Conjugation by standard equivalences turns a tensor operator between representations into a tensor operator between standard modules.

```
lemma IsTensorOperatorOf.isTensorOperator_conjStd {ρ₁ : AngularMomentumRepresentation V}
    {ρ₂ : AngularMomentumRepresentation W} {T : StandardSl2Module ℂ k →ₗ[ℂ] V →ₗ[ℂ] W}
    (hT : IsTensorOperatorOf ρ₁ ρ₂ T) {n₁ n₂ : ℕ} {ψ₁ : V ≃ₗ[ℂ] StandardSl2Module ℂ n₁}
    {ψ₂ : W ≃ₗ[ℂ] StandardSl2Module ℂ n₂} (h₁ : ρ₁.IsStandardEquiv ψ₁)
    (h₂ : ρ₂.IsStandardEquiv ψ₂) : IsTensorOperator (conjStd ψ₁ ψ₂ T) where
```

lemma conjStd_zero [source]

```
lemma conjStd_zero {n₁ n₂ : ℕ} (ψ₁ : V ≃ₗ[ℂ] StandardSl2Module ℂ n₁)
    (ψ₂ : W ≃ₗ[ℂ] StandardSl2Module ℂ n₂) :
    conjStd ψ₁ ψ₂ (0 : StandardSl2Module ℂ k →ₗ[ℂ] V →ₗ[ℂ] W) = 0
```

lemma triangleRule [source]

***Triangle rule** for irreducible representations: a nonzero tensor operator of rank `k` from `ρ₁ ≅ V(n₁)` to `ρ₂ ≅ V(n₂)` satisfies `TriangleRule k n₁ n₂`, i.e. `|j₁ - j₂| ≤ κ ≤ j₁ + j₂` for `k = 2κ`, `n₁ = 2j₁`, `n₂ = 2j₂`.*

```
lemma triangleRule (h₁z : ρ₁.Jz ≠ 0) {T : StandardSl2Module ℂ k →ₗ[ℂ] V →ₗ[ℂ] W}
    (hT : IsTensorOperatorOf ρ₁ ρ₂ T) (hT0 : T ≠ 0) {n₁ n₂ : ℕ}
    {ψ₁ : V ≃ₗ[ℂ] StandardSl2Module ℂ n₁} {ψ₂ : W ≃ₗ[ℂ] StandardSl2Module ℂ n₂}
    (hψ₁ : ρ₁.IsStandardEquiv ψ₁) (hψ₂ : ρ₂.IsStandardEquiv ψ₂) : TriangleRule k n₁ n₂
```

lemma exists_eq_smul_of_ne_zero [source]

***Wigner–Eckart theorem** for irreducible representations: two tensor operators of the same rank between the same irreducible representations are proportional, provided the second is nonzero.*

```
lemma exists_eq_smul_of_ne_zero (h₁z : ρ₁.Jz ≠ 0) (h₁ : ρ₁.IsIrreducible) (h₂z : ρ₂.Jz ≠ 0)
    (h₂ : ρ₂.IsIrreducible) {T S : StandardSl2Module ℂ k →ₗ[ℂ] V →ₗ[ℂ] W}
    (hT : IsTensorOperatorOf ρ₁ ρ₂ T) (hS : IsTensorOperatorOf ρ₁ ρ₂ S) (hS0 : S ≠ 0) :
    ∃ c : ℂ, T = c • S
```

2.577 PhyslibAlpha.QuantumMechanics.AngularMomentum.WignerEckartNormalized

PhyslibAlpha/QuantumMechanics/AngularMomentum/WignerEckartNormalized.lean on GitHub.

lemma neg_one_pow_congr_nat [source]

```
lemma neg_one_pow_congr_nat {p q : ℕ} (h : p % 2 = q % 2) : (-1 : ℝ) ^ p = (-1) ^ q
```

lemma wigner3j_swap [source]

(j₂ j₁ J; m₂ m₁ -M) = (-1)^(j₁ + j₂ + J) (j₁ j₂ J; m₁ m₂ -M).

```
lemma wigner3j_swap {k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n) (hl : l ≤ m + n - 2 * k)
    (hα : α ≤ m) (hβ : β ≤ n) :
    wigner3j n m k l β α = (-1) ^ (m + n - k) * wigner3j m n k l α β
```

lemma cgCoefficient_swap [source]

***Exchange symmetry.** ⟨j₁ m₁ j₂ m₂ | J M⟩ = (-1)^(j₁ + j₂ - J) ⟨j₂ m₂ j₁ m₁ | J M⟩.*

```
lemma cgCoefficient_swap (hmn : 0 < m + n) {k l α β : ℕ} (hkm : k ≤ m) (hkn : k ≤ n)
    (hl : l ≤ m + n - 2 * k) (hα : α ≤ m) (hβ : β ≤ n) :
    cgCoefficient m n k l α β = (-1) ^ k * cgCoefficient n m k l β α
```

lemma stringVec_eq_smul_embed [source]

```
lemma stringVec_eq_smul_embed (hmn : 0 < m + n) (j : Fin (min m n + 1)) (l : ℕ) :
    stringVec m n j l = Real.sqrt (hwNormSq m n j) • embed hmn j (vNat _ l)
```

lemma invForm_cgProj [source]

⟨u, cgProj x⟩ = ⟨w_j, w_j)^(-1/2) ⟨embed u, x⟩.

```
lemma invForm_cgProj (hmn : 0 < m + n) (j : Fin (min m n + 1))
    (y : StandardSl2Module ℝ (m + n - 2 * (j : ℕ)))
    (x : StandardSl2Module ℝ m ⊗[ℝ] StandardSl2Module ℝ n) :
    invForm ℝ _ y (cgProj hmn j x) =
      (Real.sqrt (hwNormSq m n j))⁻¹ * tensorForm m n (embed hmn j y) x
```

lemma exists_normVec_eq [source]

***Wigner–Eckart theorem with Clebsch–Gordan coefficients.** With κ = k/2, μ = κ - q, J = m/2, M = J - r, J' = (k + m)/2 - j and M' = J' - s, there is a reduced matrix element R with ⟨J' M' | T^κ_μ | J M⟩ = R · ⟨κ μ J M | J' M'⟩ for all q, r, s.*

```
lemma exists_normVec_eq (hT : IsTensorOperator T) :
    ∃ R : ℝ, ∀ q r s : ℕ, invForm ℝ _ (normVec _ s) (T (normVec k q) (normVec m r)) =
      R * cgCoefficient k m j s q r
```

lemma sum_sq_normVec [source]

***The total strength out of a substate.** If ⟨J' M' | T^κ_μ | J M⟩ = R · ⟨κ μ J M | J' M'⟩, then Σ_{μ M'} |⟨J' M' | T^κ_μ | J M⟩|² = R² (2J' + 1) / (2J + 1) for every M.*

```
lemma sum_sq_normVec (R : ℝ)
    (hR : ∀ q r s : ℕ, invForm ℝ _ (normVec _ s) (T (normVec k q) (normVec m r)) =
      R * cgCoefficient k m j s q r) {r : ℕ} (hr : r ≤ m) :
    ∑ q ∈ Finset.range (k + 1), ∑ s ∈ Finset.range (k + m - 2 * (j : ℕ) + 1),
      (invForm ℝ _ (normVec _ s) (T (normVec k q) (normVec m r))) ^ 2 =
        R ^ 2 * (((k + m - 2 * (j : ℕ) + 1 : ℕ) : ℝ) / ((m + 1 : ℕ) : ℝ))
```

2.578 PhyslibAlpha.QuantumMechanics.Blackbody.RayleighJeans

PhyslibAlpha/QuantumMechanics/Blackbody/RayleighJeans.lean on GitHub.

def quantizedOscillator [source]

The quantized oscillator with level spacing `ε`: the microstates `n : ℕ` have energy `n ε` and are counted with unit weight.

```
noncomputable def quantizedOscillator (ε : ℝ) : CanonicalEnsemble ℕ where
```

lemma exp_neg_lt_one [source]

For `x > 0`, the Boltzmann factor `e^{-x}` has norm below one.

```
lemma exp_neg_lt_one {x : ℝ} (hx : 0 < x) : ‖exp (-x)‖ < 1
```

lemma exp_neg_mul_nat [source]

The Boltzmann factor of the level `n` is the `n`-th power of that of the first level.

```
lemma exp_neg_mul_nat (b ε : ℝ) (n : ℕ) : exp (-b * (n * ε)) = exp (-(b * ε)) ^ n
```

lemma hasSum_exp_neg_levels [source]

The partition sum of the levels: `∑ e^{-β n ε} = (1 - e^{-β ε})⁻¹`.

```
lemma hasSum_exp_neg_levels {b ε : ℝ} (hb : 0 < b * ε) :
    HasSum (fun n : ℕ => exp (-b * (n * ε))) (1 - exp (-(b * ε)))⁻¹
```

lemma hasSum_energy_levels [source]

The energy sum of the levels: `∑ n ε e^{-β n ε} = ε e^{-β ε} / (1 - e^{-β ε})²`.

```
lemma hasSum_energy_levels {b ε : ℝ} (hb : 0 < b * ε) :
    HasSum (fun n : ℕ => n * ε * exp (-b * (n * ε)))
      (ε * (exp (-(b * ε)) / (1 - exp (-(b * ε))) ^ 2))
```

lemma quantizedOscillator_meanEnergy [source]

The mean energy of the quantized oscillator is Planck's `ε / (e^{β ε} - 1)`.

```
lemma quantizedOscillator_meanEnergy {ε : ℝ} (hε : 0 < ε) {T : Temperature}
    (hT : 0 < (T : ℝ)) :
    (quantizedOscillator ε).meanEnergy T = ε / (exp (T.β * ε) - 1)
```

def classicalOscillator [source]

The classical oscillator: energies `E > 0` with Lebesgue measure.

```
noncomputable def classicalOscillator : CanonicalEnsemble ℝ where
```

lemma classicalOscillator_meanEnergy [source]

Equipartition: the mean energy of the classical oscillator is `kB T`.

```
lemma classicalOscillator_meanEnergy {T : Temperature} (hT : 0 < (T : ℝ)) :
    classicalOscillator.meanEnergy T = kB * T
```

lemma quantizedOscillator_meanEnergy_lt [source]

The quantized mean energy lies strictly below the classical one, `kB T`.

```
lemma quantizedOscillator_meanEnergy_lt {ε : ℝ} (hε : 0 < ε) {T : Temperature}
    (hT : 0 < (T : ℝ)) :
    (quantizedOscillator ε).meanEnergy T < classicalOscillator.meanEnergy T
```

lemma spectralRadiance_eq_meanEnergy [source]

Planck's law is `2 ν² / c²` times the mean energy of the oscillator of spacing `h ν`.

```
lemma spectralRadiance_eq_meanEnergy (B : BlackBody) (c : SpeedOfLight) {ν : Frequency}
    (hν : 0 < ν.val) (hT : 0 < (B.T : ℝ)) :
    B.spectralRadiance c ν =
      2 * (ν.val : ℝ) ^ 2 / (c : ℝ) ^ 2
        * (quantizedOscillator (h * (ν.val : ℝ))).meanEnergy B.T
```

def rayleighJeans [source]

The Rayleigh–Jeans spectral radiance `2 ν² kB T / c²` of a black body `B`.

```
noncomputable def rayleighJeans (B : BlackBody) (c : SpeedOfLight) (ν : Frequency) : ℝ
```

lemma rayleighJeans_eq_meanEnergy [source]

The Rayleigh–Jeans law is `2 ν² / c²` times the mean energy of the classical oscillator.

```
lemma rayleighJeans_eq_meanEnergy (B : BlackBody) (c : SpeedOfLight) (ν : Frequency)
    (hT : 0 < (B.T : ℝ)) :
    B.rayleighJeans c ν
      = 2 * (ν.val : ℝ) ^ 2 / (c : ℝ) ^ 2 * classicalOscillator.meanEnergy B.T
```

lemma spectralRadiance_lt_rayleighJeans [source]

Planck's law lies strictly below the Rayleigh–Jeans law at every positive frequency and every positive temperature.

```
lemma spectralRadiance_lt_rayleighJeans (B : BlackBody) (c : SpeedOfLight) {ν : Frequency}
    (hν : 0 < ν.val) (hT : 0 < (B.T : ℝ)) :
    B.spectralRadiance c ν < B.rayleighJeans c ν
```

2.579 PhyslibAlpha.QuantumMechanics.Foundations.OntologicalModel

PhyslibAlpha/QuantumMechanics/Foundations/OntologicalModel.lean on GitHub.

structure OntologicalModel [source]

An ontological model (Harrigan–Spekkens) of the quantum system `ℋ[d]`, with ontic state space `Λ`.

Preparations are indexed by unit vectors rather than by states. This is more general: a model whose preparations depend only on the vector state `ofVec hψ` is a special case, while here two unit vectors differing by a global phase, which have the same vector state and hence the same measurement statistics, may be prepared with different measures. Only pure-state preparations are modelled, as only these are needed for the PBR theorem. Measurements have finitely many outcomes, and response functions take values in `ℝ≥0∞` so that they can be integrated with the Lebesgue integral `∫⁻`.

```
structure OntologicalModel (d : Type*) [Fintype d] [DecidableEq d] (Λ : Type*)
    [MeasurableSpace Λ] where
  prep (ψ : ℋ[d]) (hψ : ‖ψ‖ = 1) : Measure Λ
  isProbabilityMeasure_prep (ψ : ℋ[d]) (hψ : ‖ψ‖ = 1) : IsProbabilityMeasure (prep ψ hψ)
  resp {X : Type} [Fintype X] [MeasurableSpace X] [MeasurableSingletonClass X] :
    Measurement X (Observable (ℋ[d] →L[ℂ] ℋ[d])) → X → Λ → ℝ≥0∞
  measurable_resp {X : Type} [Fintype X] [MeasurableSpace X] [MeasurableSingletonClass X]
    (M : Measurement X (Observable (ℋ[d] →L[ℂ] ℋ[d]))) (x : X) : Measurable (resp M x)
  sum_resp {X : Type} [Fintype X] [MeasurableSpace X] [MeasurableSingletonClass X]
    (M : Measurement X (Observable (ℋ[d] →L[ℂ] ℋ[d]))) (l : Λ) : ∑ x, resp M x l = 1
  lintegral_resp {X : Type} [Fintype X] [MeasurableSpace X] [MeasurableSingletonClass X]
    (M : Measurement X (Observable (ℋ[d] →L[ℂ] ℋ[d]))) (ψ : ℋ[d]) (hψ : ‖ψ‖ = 1) (x : X) :
    ∫⁻ l, resp M x l ∂prep ψ hψ =
      ENNReal.ofReal ((ofVec hψ)(M {x} (measurableSet_singleton x)))
```

def IsPsiOntic [source]

An ontological model is ψ-ontic if the preparation measures of any two unit vectors with different vector states are mutually singular, so that the ontic state determines the quantum state. Unit vectors differing only by a global phase have the same vector state, and may share ontic states.

```
def IsPsiOntic (M : OntologicalModel d Λ) : Prop
```

def PreparationIndependent [source]

The preparation-independence assumption of Pusey, Barrett and Rudolph for the model `M`, relative to a family of models `Mn n` of `n` copies of the system (on `ℋ[Fin n → d]`, with ontic states `Λn n`). It asks that `n` independently prepared systems have independent ontic states which jointly determine the ontic state of the `n` copies: there are measurable maps `ι n : (Fin n → Λ) → Λn n` such that preparing the product vector `piProdVec ψ` gives the ontic state `ι n l`, where the `l i` are independent and distributed according to `M.prep (ψ i)`.

The ontic space `Λn n` need not be the Cartesian product `Fin n → Λ`; taking `Λn n = Fin n → Λ` and `ι n = id` gives the stronger Cartesian Product Assumption (in Leifer's terminology), see `PreparationIndependent.of_prep_piProdVec`.

Since preparations are indexed by vectors, two families `ψ`, `ψ'` with the same product vector (for example `ψ 0 ⊗ ψ 1 = (i • ψ 0) ⊗ (-i • ψ 1)`) must give the same pushed-forward product measure, and at `n = 0` the preparation `(Mn 0).prep` of the trivial product vector must be a Dirac measure. The first condition is automatic

when `M.prep` depends only on the ray of the vector, and the second only concerns the trivial system `n = 0`, which the PBR theorem does not use; so the standard PBR setting is covered by choosing one representative vector per ray.

```
def PreparationIndependent (M : OntologicalModel d Λ) {Λn : ℕ → Type*}
    [∀ n, MeasurableSpace (Λn n)] (Mn : (n : ℕ) → OntologicalModel (Fin n → d) (Λn n)) : Prop
```

lemma of_prep_piProdVec [source]

The Cartesian-product form of preparation independence: if the `n`-copy models have ontic space `Fin n → Λ` and prepare the product vector `piProdVec ψ` with the product of the measures `M.prep (ψ i)`, then `M` is preparation independent (with `ι n = id`).

```
lemma PreparationIndependent.of_prep_piProdVec {M : OntologicalModel d Λ}
    {Mn : (n : ℕ) → OntologicalModel (Fin n → d) (Fin n → Λ)}
    (h : ∀ n (ψ : Fin n → ℋ[d]) (hψ : ∀ i, ‖ψ i‖ = 1),
      (Mn n).prep (piProdVec ψ) (norm_piProdVec_eq_one hψ) =
        Measure.pi fun i => M.prep (ψ i) (hψ i)) :
    M.PreparationIndependent Mn
```

2.580 PhyslibAlpha.QuantumMechanics.Foundations.PBR.Phases

PhyslibAlpha/QuantumMechanics/Foundations/PBR/Phases.lean on GitHub.

lemma exists_two_mul_pow_le [source]

For `0 < c` and `0 < s`, some number of copies `n` gives `2 c^n ≤ (c + s)^n`.

```
lemma exists_two_mul_pow_le {c s : ℝ} (hc : 0 < c) (hs : 0 < s) :
    ∃ n : ℕ, 2 * c ^ n ≤ (c + s) ^ n
```

lemma exists_norm_pow_sub_pow_eq [source]

If `0 ≤ s ≤ c` and `2 c^n ≤ (c + s)^n`, some angle `α` gives `‖(c + e^{iα} s)^n - c^n‖ = c^n` (intermediate value theorem on `[0, π]`).

```
lemma exists_norm_pow_sub_pow_eq {c s : ℝ} {n : ℕ} (hs : 0 ≤ s) (hsc : s ≤ c)
    (hn : 2 * c ^ n ≤ (c + s) ^ n) :
    ∃ α : ℝ, ‖((c : ℂ) + Complex.exp (α * Complex.I) * s) ^ n - (c : ℂ) ^ n‖ = c ^ n
```

lemma prod_phase_mul_prod_amp [source]

The phase product `∏ i, ![1, e] (z i)` times the amplitude product `∏ i, ![c, s] (z i)` is the amplitude product `∏ i, ![c, e * s] (z i)`.

```
lemma prod_phase_mul_prod_amp {n : ℕ} (e c s : ℂ) (z : Fin n → Fin 2) :
    (∏ i, ![1, e] (z i)) * ∏ i, ![c, s] (z i) = ∏ i, ![c, e * s] (z i)
```

lemma norm_prod_phase [source]

The phase product `∏ i, ![1, e] (z i)` has unit norm when `e` does.

```
lemma norm_prod_phase {n : ℕ} {e : ℂ} (he : ‖e‖ = 1) (z : Fin n → Fin 2) :
    ‖∏ i, ![1, e] (z i)‖ = 1
```

lemma sum_phases_mul_amp [source]

The phases `ω 0 = -((c + e s)^n - c^n) / c^n` and `ω z = ∏ i, ![1, e] (z i)` for `z ≠ 0` cancel the amplitude products.

```
lemma sum_phases_mul_amp {n : ℕ} (c s e : ℂ) (hc : c ^ n ≠ 0) :
    ∑ z : Fin n → Fin 2, (if z = 0 then -(((c + e * s) ^ n - c ^ n) / c ^ n)
      else ∏ i, ![1, e] (z i)) * ∏ i, ![c, s] (z i) = 0
```

lemma exists_phases [source]

Unit-modulus phases `ω z` with `∑ z, ω z * ∏ i, ![c, s] (z i) = 0`, for `0 ≤ s ≤ c`, `0 < c` and `2 c^n ≤ (c + s)^n`.

```
lemma exists_phases {c s : ℝ} {n : ℕ} (hc : 0 < c) (hs : 0 ≤ s) (hsc : s ≤ c)
    (hn : 2 * c ^ n ≤ (c + s) ^ n) :
    ∃ ω : (Fin n → Fin 2) → ℂ,
      (∀ z, ‖ω z‖ = 1) ∧ ∑ z, ω z * ∏ i, ![(c : ℂ), (s : ℂ)] (z i) = 0
```

2.581 PhyslibAlpha.QuantumMechanics.HilbertSpaces.FiniteTarget.Operators

PhyslibAlpha/QuantumMechanics/HilbertSpaces/FiniteTarget/Operators.lean on GitHub.

instance IsScalarTower ℝ (ℋ[d] →L[ℂ] ℋ[d]) (ℋ[d] →L[ℂ] ℋ[d]) [source]

```
instance : IsScalarTower ℝ (ℋ[d] →L[ℂ] ℋ[d]) (ℋ[d] →L[ℂ] ℋ[d])
```

instance SMulCommClass ℝ (ℋ[d] →L[ℂ] ℋ[d]) (ℋ[d] →L[ℂ] ℋ[d]) [source]

```
instance : SMulCommClass ℝ (ℋ[d] →L[ℂ] ℋ[d]) (ℋ[d] →L[ℂ] ℋ[d])
```

instance SelfAdjointDecompose (ℋ[d] →L[ℂ] ℋ[d]) [source]

```
instance : SelfAdjointDecompose (ℋ[d] →L[ℂ] ℋ[d])
```

2.582 PhyslibAlpha.QuantumMechanics.HilbertSpaces.FiniteTarget.Product

PhyslibAlpha/QuantumMechanics/HilbertSpaces/FiniteTarget/Product.lean on GitHub.

lemma val_apply_basisFun [source]

The amplitude `⟨a|A|a'⟩` of an operator is the `a` component of `A |a'⟩`.

```
lemma val_apply_basisFun (A : ℋ[α] →ₗ[ℂ] ℋ[α]) (a a' : α) :
    (A (basisFun α a')).val a = ⟪basisFun α a, A (basisFun α a')⟫_ℂ
```

lemma val_apply [source]

An operator in components: `(A ψ) a = ∑ a', ⟨a|A|a'⟩ ψ a'`.

```
lemma val_apply (A : ℋ[α] →ₗ[ℂ] ℋ[α]) (ψ : ℋ[α]) (a : α) :
    (A ψ).val a = ∑ a', (A (basisFun α a')).val a * ψ.val a'
```

def onFst [source]

The operator `A` acting on the first coordinate: `(A Ψ) (a, b) = ∑ a', ⟨a|A|a'⟩ Ψ (a', b)`.

```
noncomputable def onFst (A : ℋ[α] →ₗ[ℂ] ℋ[α]) : ℋ[α × β] →ₗ[ℂ] ℋ[α × β] where
```

def onSnd [source]

The operator `B` acting on the second coordinate: `(B Ψ) (a, b) = ∑ b', ⟨b|B|b'⟩ Ψ (a, b')`.

```
noncomputable def onSnd (B : ℋ[β] →ₗ[ℂ] ℋ[β]) : ℋ[α × β] →ₗ[ℂ] ℋ[α × β] where
```

lemma onFst_val [source]

```
lemma onFst_val (A : ℋ[α] →ₗ[ℂ] ℋ[α]) (Ψ : ℋ[α × β]) (p : α × β) :
    (onFst A Ψ).val p = ∑ a, (A (basisFun α a)).val p.1 * Ψ.val (a, p.2)
```

lemma onSnd_val [source]

```
lemma onSnd_val (B : ℋ[β] →ₗ[ℂ] ℋ[β]) (Ψ : ℋ[α × β]) (p : α × β) :
    (onSnd B Ψ).val p = ∑ b, (B (basisFun β b)).val p.2 * Ψ.val (p.1, b)
```

lemma onFst_comp [source]

Acting twice on the first coordinate composes the operators.

```
lemma onFst_comp (A A' : ℋ[α] →ₗ[ℂ] ℋ[α]) :
    onFst (β := β) (A ∘ₗ A') = onFst A ∘ₗ onFst A'
```

lemma onSnd_comp [source]

Acting twice on the second coordinate composes the operators.

```
lemma onSnd_comp (B B' : ℋ[β] →ₗ[ℂ] ℋ[β]) :
    onSnd (α := α) (B ∘ₗ B') = onSnd B ∘ₗ onSnd B'
```

lemma onFst_comp_onSnd [source]

Operators on different coordinates commute.

```
lemma onFst_comp_onSnd (A : ℋ[α] →ₗ[ℂ] ℋ[α]) (B : ℋ[β] →ₗ[ℂ] ℋ[β]) :
    onFst A ∘ₗ onSnd B = onSnd B ∘ₗ onFst A
```

lemma inner_eq_sum [source]

The inner product of `ℋ[α × β]` in components.

```
lemma inner_eq_sum (Ψ Φ : ℋ[α × β]) :
    ⟪Ψ, Φ⟫_ℂ = ∑ p, (starRingEnd ℂ) (Ψ.val p) * Φ.val p
```

lemma conj_val_apply_basisFun [source]

Hermitian amplitudes: `conj ⟨a'|A|a⟩ = ⟨a|A|a'⟩`.

```
lemma conj_val_apply_basisFun {A : ℋ[α] →ₗ[ℂ] ℋ[α]} (hA : A.IsSymmetric) (a a' : α) :
    (starRingEnd ℂ) ((A (basisFun α a)).val a') = (A (basisFun α a')).val a
```

lemma onFst_isSymmetric [source]

A hermitian operator on the first coordinate is hermitian.

```
lemma onFst_isSymmetric {A : ℋ[α] →ₗ[ℂ] ℋ[α]} (hA : A.IsSymmetric) :
    (onFst (β := β) A).IsSymmetric
```

lemma onSnd_isSymmetric [source]

A hermitian operator on the second coordinate is hermitian.

```
lemma onSnd_isSymmetric {B : ℋ[β] →ₗ[ℂ] ℋ[β]} (hB : B.IsSymmetric) :
    (onSnd (α := α) B).IsSymmetric
```

2.583 PhyslibAlpha.QuantumMechanics.HilbertSpaces.FiniteTarget.ProductState

PhyslibAlpha/QuantumMechanics/HilbertSpaces/FiniteTarget/ProductState.lean on GitHub.

def prodVec [source]

The product vector `(ψ ⊗ φ) (a, b) = ψ a φ b`.

```
noncomputable def prodVec (ψ : ℋ[α]) (φ : ℋ[β]) : ℋ[α × β]
```

lemma prodVec_val [source]

```
lemma prodVec_val (ψ : ℋ[α]) (φ : ℋ[β]) (p : α × β) :
    (prodVec ψ φ).val p = ψ.val p.1 * φ.val p.2
```

lemma inner_prodVec [source]

`⟪ψ ⊗ φ, ψ' ⊗ φ'⟫ = ⟪ψ, ψ'⟫ ⟪φ, φ'⟫`.

```
lemma inner_prodVec (ψ ψ' : ℋ[α]) (φ φ' : ℋ[β]) :
    ⟪prodVec ψ φ, prodVec ψ' φ'⟫_ℂ = ⟪ψ, ψ'⟫_ℂ * ⟪φ, φ'⟫_ℂ
```

lemma norm_prodVec [source]

`‖ψ ⊗ φ‖ = ‖ψ‖ ‖φ‖`.

```
lemma norm_prodVec (ψ : ℋ[α]) (φ : ℋ[β]) : ‖prodVec ψ φ‖ = ‖ψ‖ * ‖φ‖
```

lemma norm_prodVec_eq_one [source]

The product of two unit vectors is a unit vector.

```
lemma norm_prodVec_eq_one {ψ : ℋ[α]} {φ : ℋ[β]} (hψ : ‖ψ‖ = 1) (hφ : ‖φ‖ = 1) :
    ‖prodVec ψ φ‖ = 1
```

lemma prodVec_sub_left [source]

```
lemma prodVec_sub_left (ψ ψ' : ℋ[α]) (φ : ℋ[β]) :
    prodVec (ψ - ψ') φ = prodVec ψ φ - prodVec ψ' φ
```

lemma prodVec_sub_right [source]

```
lemma prodVec_sub_right (ψ : ℋ[α]) (φ φ' : ℋ[β]) :
    prodVec ψ (φ - φ') = prodVec ψ φ - prodVec ψ φ'
```

lemma prodVec_smul_left [source]

```
lemma prodVec_smul_left (r : ℝ) (ψ : ℋ[α]) (φ : ℋ[β]) :
    prodVec (r • ψ) φ = r • prodVec ψ φ
```

lemma prodVec_smul_right [source]

```
lemma prodVec_smul_right (r : ℝ) (ψ : ℋ[α]) (φ : ℋ[β]) :
    prodVec ψ (r • φ) = r • prodVec ψ φ
```

lemma onFst_prodVec [source]

(A ⊗ 1) (ψ ⊗ φ) = A ψ ⊗ φ.

```
lemma onFst_prodVec (A : ℋ[α] →ₗ[ℂ] ℋ[α]) (ψ : ℋ[α]) (φ : ℋ[β]) :
    onFst A (prodVec ψ φ) = prodVec (A ψ) φ
```

lemma onSnd_prodVec [source]

(1 ⊗ B) (ψ ⊗ φ) = ψ ⊗ B φ.

```
lemma onSnd_prodVec (B : ℋ[β] →ₗ[ℂ] ℋ[β]) (ψ : ℋ[α]) (φ : ℋ[β]) :
    onSnd B (prodVec ψ φ) = prodVec ψ (B φ)
```

def onFstObservable [source]

An observable of the first coordinate as an observable of ℋ[α × β].

```
noncomputable def onFstObservable (a : Observable (ℋ[α] →L[ℂ] ℋ[α])) :
    Observable (ℋ[α × β] →L[ℂ] ℋ[α × β])
```

def onSndObservable [source]

An observable of the second coordinate as an observable of ℋ[α × β].

```
noncomputable def onSndObservable (b : Observable (ℋ[β] →L[ℂ] ℋ[β])) :
    Observable (ℋ[α × β] →L[ℂ] ℋ[α × β])
```

lemma expectation_onFstObservable [source]

The mean of the first coordinate is that of ψ.

```
lemma expectation_onFstObservable (a : Observable (ℋ[α] →L[ℂ] ℋ[α])) :
    ⟨ofVec (norm_prodVec_eq_one hψ hφ)|onFstObservable a|⟩ = ⟨ofVec hψ|a|⟩
```

lemma fluctuation_onFstObservable [source]

The fluctuation of the first coordinate is that of ψ, tensored with φ.

```
lemma fluctuation_onFstObservable (a : Observable (ℋ[α] →L[ℂ] ℋ[α])) :
    (onFstObservable (β := β) a : ℋ[α × β] →L[ℂ] ℋ[α × β]) (prodVec ψ φ) -
        (ofVec (norm_prodVec_eq_one hψ hφ))⟪onFstObservable a⟫ • prodVec ψ φ =
      prodVec ((a : ℋ[α] →L[ℂ] ℋ[α]) ψ - (ofVec hψ)⟪a⟫ • ψ) φ
```

lemma variance_onFstObservable [source]

```
lemma variance_onFstObservable (a : Observable (ℋ[α] →L[ℂ] ℋ[α])) :
    variance (ofVec (norm_prodVec_eq_one hψ hφ)) (onFstObservable a) = variance (ofVec hψ) a
```

lemma covariance_onFstObservable [source]

```
lemma covariance_onFstObservable (a b : Observable (ℋ[α] →L[ℂ] ℋ[α])) :
    covariance (ofVec (norm_prodVec_eq_one hψ hφ)) (onFstObservable a) (onFstObservable b) =
      covariance (ofVec hψ) a b
```

lemma centeredGramDefect_onFstObservable [source]

```
lemma centeredGramDefect_onFstObservable (a b : Observable (ℋ[α] →L[ℂ] ℋ[α])) :
    centeredGramDefect (ofVec (norm_prodVec_eq_one hψ hφ)) (onFstObservable a)
      (onFstObservable b) = centeredGramDefect (ofVec hψ) a b
```

lemma expectation_onSndObservable [source]

The mean of the second coordinate is that of φ.

```
lemma expectation_onSndObservable (b : Observable (ℋ[β] →L[ℂ] ℋ[β])) :
    (ofVec (norm_prodVec_eq_one hψ hφ))⟪onSndObservable b⟫ = (ofVec hφ)⟪b⟫
```

lemma fluctuation_onSndObservable [source]

The fluctuation of the second coordinate is ψ tensored with that of φ.

```
lemma fluctuation_onSndObservable (b : Observable (ℋ[β] →L[ℂ] ℋ[β])) :
    (onSndObservable (α := α) b : ℋ[α × β] →L[ℂ] ℋ[α × β]) (prodVec ψ φ) -
        (ofVec (norm_prodVec_eq_one hψ hφ))⟪onSndObservable b⟫ • prodVec ψ φ =
      prodVec ψ ((b : ℋ[β] →L[ℂ] ℋ[β]) φ - (ofVec hφ)⟪b⟫ • φ)
```

lemma variance_onSndObservable [source]

```
lemma variance_onSndObservable (b : Observable (ℋ[β] →L[ℂ] ℋ[β])) :
    variance (ofVec (norm_prodVec_eq_one hψ hφ)) (onSndObservable b) = variance (ofVec hφ) b
```

lemma covariance_onSndObservable [source]

```
lemma covariance_onSndObservable (a b : Observable (ℋ[β] →L[ℂ] ℋ[β])) :
    covariance (ofVec (norm_prodVec_eq_one hψ hφ)) (onSndObservable a) (onSndObservable b) =
      covariance (ofVec hφ) a b
```

lemma centeredGramDefect_onSndObservable [source]

```
lemma centeredGramDefect_onSndObservable (a b : Observable (ℋ[β] →L[ℂ] ℋ[β])) :
    centeredGramDefect (ofVec (norm_prodVec_eq_one hψ hφ)) (onSndObservable a)
      (onSndObservable b) = centeredGramDefect (ofVec hφ) a b
```

lemma inner_fluctuation [source]

The fluctuation vector of an observable is orthogonal to the state.

```
lemma inner_fluctuation {γ : Type*} [Fintype γ] [DecidableEq γ] {χ : ℋ[γ]} (hχ : ‖χ‖ = 1)
    (c : Observable (ℋ[γ] →L[ℂ] ℋ[γ])) :
    ⟪χ, (c : ℋ[γ] →L[ℂ] ℋ[γ]) χ - (ofVec hχ)⟨c⟩ • χ⟫_ℂ = 0
```

lemma covariance_onFstObservable_onSndObservable [source]

In a product state, an observable of the first coordinate and one of the second are uncorrelated.

```
lemma covariance_onFstObservable_onSndObservable (a : Observable (ℋ[α] →L[ℂ] ℋ[α]))
    (b : Observable (ℋ[β] →L[ℂ] ℋ[β])) :
    covariance (ofVec (norm_prodVec_eq_one hψ hφ)) (onFstObservable a) (onSndObservable b) = 0
```

def piProdVec [source]

The product vector `(⊗ i, v i) k = ∏ i, v i (k i)` of a finite family of vectors, describing independently prepared copies of a system.

```
noncomputable def piProdVec (v : ι → ℋ[α]) : ℋ[ι → α]
```

lemma piProdVec_val [source]

The components of a product vector are the products of the components.

```
lemma piProdVec_val (v : ι → ℋ[α]) (k : ι → α) :
    (piProdVec v).val k = ∏ i, (v i).val (k i)
```

lemma inner_piProdVec [source]

$\langle\langle \otimes v, \otimes w\rangle\rangle = \prod i, \langle\langle v\ i, w\ i\rangle\rangle$.

```
lemma inner_piProdVec (v w : ι → ℋ[α]) :
    ⟪piProdVec v, piProdVec w⟫_ℂ = ∏ i, ⟪v i, w i⟫_ℂ
```

lemma norm_piProdVec [source]

$\|\otimes v\| = \prod i, \|v\ i\|$.

```
lemma norm_piProdVec (v : ι → ℋ[α]) : ‖piProdVec v‖ = ∏ i, ‖v i‖
```

lemma norm_piProdVec_eq_one [source]

The product of unit vectors is a unit vector.

```
lemma norm_piProdVec_eq_one {v : ι → ℋ[α]} (hv : ∀ i, ‖v i‖ = 1) : ‖piProdVec v‖ = 1
```

2.584 PhyslibAlpha.QuantumMechanics.Hydrogen.Degeneracy

PhyslibAlpha/QuantumMechanics/Hydrogen/Degeneracy.lean on GitHub.

def polyLaplacian [source]

The Laplacian `Σᵢ ∂ᵢ² P` of a polynomial in `d` variables.

```
def polyLaplacian (d : ℕ) : MvPolynomial (Fin d) ℝ →ₗ[ℝ] MvPolynomial (Fin d) ℝ
```

def harmonicSubmodule [source]

The homogeneous harmonic polynomials of degree `l` in `d` variables.

```
def harmonicSubmodule (d l : ℕ) : Submodule ℝ (MvPolynomial (Fin d) ℝ)
```

instance FiniteDimensional ℝ (harmonicSubmodule d l) [source]

```
instance (d l : ℕ) : FiniteDimensional ℝ (harmonicSubmodule d l)
```

lemma finrank_homogeneousSubmodule [source]

The homogeneous polynomials of degree `l` in `d ≥ 1` variables form a space of dimension `C(l + d - 1, d - 1)`: their monomials correspond to multisets of size `l` in `d` letters.

```
lemma finrank_homogeneousSubmodule {d : ℕ} (hd : 1 ≤ d) (l : ℕ) :
    Module.finrank ℝ (homogeneousSubmodule (Fin d) ℝ l) = (l + d - 1).choose (d - 1)
```

lemma polyLaplacian_apply [source]

```
lemma polyLaplacian_apply {d : ℕ} (P : MvPolynomial (Fin d) ℝ) :
    polyLaplacian d P = ∑ i, pderiv i (pderiv i P)
```

lemma polyLaplacian_isHomogeneous [source]

The Laplacian lowers the degree of a homogeneous polynomial by two.

```
lemma polyLaplacian_isHomogeneous {d l : ℕ} {P : MvPolynomial (Fin d) ℝ}
    (hP : P.IsHomogeneous (l + 2)) : (polyLaplacian d P).IsHomogeneous l
```

lemma pderiv_eq_zero_of_isHomogeneous_zero [source]

A homogeneous polynomial of degree `0` is constant, so its derivatives vanish.

```
lemma pderiv_eq_zero_of_isHomogeneous_zero {d : ℕ} {Q : MvPolynomial (Fin d) ℝ}
    (hQ : Q.IsHomogeneous 0) (i : Fin d) : pderiv i Q = 0
```

lemma polyLaplacian_eq_zero_of_lt_two [source]

The Laplacian vanishes on homogeneous polynomials of degree `0` and `1`.

```
lemma polyLaplacian_eq_zero_of_lt_two {d l : ℕ} {P : MvPolynomial (Fin d) ℝ}
    (hP : P.IsHomogeneous l) (hl : l < 2) : polyLaplacian d P = 0
```

def fischerWeight [source]

The weight $m! = \prod_j m_j!$ of a monomial exponent m.

```
noncomputable def fischerWeight (m : Fin d →₀ ℕ) : ℝ
```

lemma fischerWeight_pos [source]

```
lemma fischerWeight_pos (m : Fin d →₀ ℕ) : 0 < fischerWeight m
```

lemma fischerWeight_add_single [source]

```
lemma fischerWeight_add_single (m : Fin d →₀ ℕ) (i : Fin d) :
    fischerWeight (m + Finsupp.single i 1) = fischerWeight m * ((m i : ℝ) + 1)
```

def fischer [source]

The Fischer inner product $\sum_m m!\ P_m\ Q_m$ of two polynomials.

```
noncomputable def fischer (P Q : MvPolynomial (Fin d) ℝ) : ℝ
```

lemma fischer_eq_sum [source]

```
lemma fischer_eq_sum {d : ℕ} {P Q : MvPolynomial (Fin d) ℝ} {S : Finset (Fin d →₀ ℕ)}
    (hS : P.support ⊆ S) :
    fischer P Q = ∑ m ∈ S, fischerWeight m * P.coeff m * Q.coeff m
```

lemma fischer_zero_left [source]

```
lemma fischer_zero_left (Q : MvPolynomial (Fin d) ℝ) : fischer 0 Q = 0
```

lemma fischer_add_left [source]

```
lemma fischer_add_left (P P' Q : MvPolynomial (Fin d) ℝ) :
    fischer (P + P') Q = fischer P Q + fischer P' Q
```

lemma fischer_sum_left [source]

```
lemma fischer_sum_left {ι : Type*} (s : Finset ι) (f : ι → MvPolynomial (Fin d) ℝ)
    (Q : MvPolynomial (Fin d) ℝ) : fischer (∑ i ∈ s, f i) Q = ∑ i ∈ s, fischer (f i) Q
```

lemma fischer_sum_right [source]

```
lemma fischer_sum_right {ι : Type*} (s : Finset ι) (P : MvPolynomial (Fin d) ℝ)
    (f : ι → MvPolynomial (Fin d) ℝ) : fischer P (∑ i ∈ s, f i) = ∑ i ∈ s, fischer P (f i)
```

lemma fischer_zero_right [source]

```
lemma fischer_zero_right (P : MvPolynomial (Fin d) ℝ) : fischer P 0 = 0
```

lemma fischer_mul_X [source]

Multiplication by X_i is adjoint to ∂_i for the Fischer inner product.

```
lemma fischer_mul_X (P Q : MvPolynomial (Fin d) ℝ) (i : Fin d) :
    fischer (P * X i) Q = fischer P (pderiv i Q)
```

lemma fischer_self_pos [source]

The Fischer inner product is positive definite.

```
lemma fischer_self_pos {P : MvPolynomial (Fin d) ℝ} (hP : P ≠ 0) : 0 < fischer P P
```

def rSq [source]

$r^2 = \Sigma_i X_i^2$.

```
def rSq (d : ℕ) : MvPolynomial (Fin d) ℝ
```

lemma rSq_isHomogeneous [source]

```
lemma rSq_isHomogeneous (d : ℕ) : (rSq d).IsHomogeneous 2
```

lemma rSq_ne_zero [source]

```
lemma rSq_ne_zero {d : ℕ} (hd : 1 ≤ d) : rSq d ≠ 0
```

lemma eq_zero_of_polyLaplacian_eq_zero_of_eq_rSq_mul [source]

A harmonic polynomial divisible by r^2 vanishes: $\langle r^2\ P,\ r^2\ P\rangle = \langle P,\ \Delta(r^2\ P)\rangle = 0$.

```
lemma eq_zero_of_polyLaplacian_eq_zero_of_eq_rSq_mul {d : ℕ} {P Q : MvPolynomial (Fin d) ℝ}
    (hQ : polyLaplacian d Q = 0) (hQP : Q = rSq d * P) : Q = 0
```

lemma finrank_harmonicSubmodule_add_finrank [source]

dim Harm_{l+2} + dim Hom_l = dim Hom_{l+2}: the Laplacian Hom_{l+2} → Hom_l is onto.

```
lemma finrank_harmonicSubmodule_add_finrank {d : ℕ} (hd : 1 ≤ d) (l : ℕ) :
    Module.finrank ℝ (harmonicSubmodule d (l + 2)) +
        Module.finrank ℝ (homogeneousSubmodule (Fin d) ℝ l) =
      Module.finrank ℝ (homogeneousSubmodule (Fin d) ℝ (l + 2))
```

lemma finrank_harmonicSubmodule [source]

The homogeneous harmonic polynomials of degree l in $d \geq 2$ variables form a space of dimension $C(l + d - 1, d - 1) - C(l + d - 3, d - 1)$.

```
lemma finrank_harmonicSubmodule {d : ℕ} (hd : 2 ≤ d) (l : ℕ) :
    Module.finrank ℝ (harmonicSubmodule d l) =
      (l + d - 1).choose (d - 1) - (l + d - 3).choose (d - 1)
```

lemma finrank_harmonicSubmodule_three [source]

In three variables the homogeneous harmonic polynomials of degree `l` form a space of dimension `2l + 1`.

```
lemma finrank_harmonicSubmodule_three (l : ℕ) :
    Module.finrank ℝ (harmonicSubmodule 3 l) = 2 * l + 1
```

lemma sum_two_mul_add_one [source]

`Σ_{l ≤ n} (2l + 1) = (n + 1)²`.

```
lemma sum_two_mul_add_one (n : ℕ) : ∑ l ∈ Finset.range (n + 1), (2 * l + 1) = (n + 1) ^ 2
```

def toFun [source]

A polynomial in `d` variables as a function on `Space d`.

```
def toFun {d : ℕ} (P : MvPolynomial (Fin d) ℝ) (y : Space d) : ℝ
```

lemma toFun_C [source]

```
@[simp]
lemma toFun_C {d : ℕ} (a : ℝ) : toFun (C a : MvPolynomial (Fin d) ℝ) = fun _ => a
```

lemma toFun_add [source]

```
@[simp]
lemma toFun_add {d : ℕ} (P Q : MvPolynomial (Fin d) ℝ) : toFun (P + Q) = toFun P + toFun Q
```

lemma toFun_mul_X [source]

```
lemma toFun_mul_X {d : ℕ} (P : MvPolynomial (Fin d) ℝ) (j : Fin d) :
    toFun (P * X j) = fun y => toFun P y * y j
```

lemma contDiff_toFun_infty [source]

Polynomials are smooth.

```
lemma contDiff_toFun_infty {d : ℕ} (P : MvPolynomial (Fin d) ℝ) : ContDiff ℝ ∞ (toFun P)
```

lemma contDiff_toFun [source]

Polynomials are twice continuously differentiable.

```
lemma contDiff_toFun {d : ℕ} (P : MvPolynomial (Fin d) ℝ) : ContDiff ℝ 2 (toFun P)
```

lemma differentiable_toFun [source]

```
lemma differentiable_toFun {d : ℕ} (P : MvPolynomial (Fin d) ℝ) : Differentiable ℝ (toFun P)
```

lemma deriv_toFun [source]

The partial derivative of a polynomial function is the polynomial partial derivative.

```
lemma deriv_toFun {d : ℕ} (i : Fin d) (P : MvPolynomial (Fin d) ℝ) :
    ∂[i] (toFun P) = toFun (pderiv i P)
```

lemma euler_toFun [source]

Euler's relation `Σᵢ yᵢ ∂ᵢP = l P` for a homogeneous polynomial of degree `l`.

```
lemma euler_toFun {d l : ℕ} {P : MvPolynomial (Fin d) ℝ} (hP : P.IsHomogeneous l)
    (y : Space d) : ∑ i, y i * ∂[i] (toFun P) y = l * toFun P y
```

lemma laplacian_toFun [source]

The Laplacian of a polynomial function is the polynomial Laplacian.

```
lemma laplacian_toFun {d : ℕ} (P : MvPolynomial (Fin d) ℝ) (y : Space d) :
    Δ (toFun P) y = toFun (polyLaplacian d P) y
```

lemma boundState_eigen_of_mem_harmonicSubmodule [source]

A homogeneous harmonic polynomial of degree `l` gives the bound state `boundState H N l P` of energy `E_{N+l}`, away from the nucleus.

```
lemma boundState_eigen_of_mem_harmonicSubmodule (H : QuantumMechanics.HydrogenAtom d)
    (hd : 2 ≤ d) (hk : 0 < H.k) (N l : ℕ) {P : MvPolynomial (Fin d) ℝ}
    (hP : P ∈ harmonicSubmodule d l) {x : Space d} (hx : x ≠ 0) :
    -((Constants.ℏ : ℝ) ^ 2 / (2 * H.m)) * Δ (boundState H N l (toFun P)) x +
        H.potential x * boundState H N l (toFun P) x =
      H.energyLevel (N + l) * boundState H N l (toFun P) x
```

lemma toFun_smul_of_isHomogeneous [source]

A homogeneous polynomial of degree `l` scales as `P(t y) = t^l P(y)`.

```
lemma toFun_smul_of_isHomogeneous {d l : ℕ} {P : MvPolynomial (Fin d) ℝ}
    (hP : P.IsHomogeneous l) (t : ℝ) (y : Space d) : toFun P (t • y) = t ^ l * toFun P y
```

lemma eq_zero_of_sum_pow_mul [source]

If `Σ_l c_l t^l f_l(t) = 0` for all `t > 0`, with continuous `f_l` and `f_l(0) ≠ 0`, then all `c_l` vanish: compare the lowest powers of `t` as `t → 0⁺`.

```
lemma eq_zero_of_sum_pow_mul {n : ℕ} (c : Fin (n + 1) → ℝ) (f : Fin (n + 1) → ℝ → ℝ)
    (hf : ∀ l, Continuous (f l)) (hf0 : ∀ l, f l 0 ≠ 0)
    (h : ∀ t, 0 < t → ∑ l, c l * t ^ (l : ℕ) * f l t = 0) : c = 0
```

def boundStateMap [source]

The bound states of the level `n`: a family `(P_l)_{l ≤ n}` of homogeneous harmonic polynomials gives `Σ_l boundState H (n - l) l P_l`.

```
def boundStateMap (H : QuantumMechanics.HydrogenAtom d) (n : ℕ) :
    ((l : Fin (n + 1)) → harmonicSubmodule d l) →ₗ[ℝ] (Space d → ℝ) where
```

lemma boundStateMap_injective [source]

The bound states of one level, for different `l` and linearly independent polynomials, are linearly independent. Along a ray `y = t • ω` the summand of degree `l` is `e^(-κ * t * ‖ω‖)` times `t^l` `P_l(ω)` times a Laguerre polynomial in `t` that does not vanish at `t = 0`.

```
lemma boundStateMap_injective (H : QuantumMechanics.HydrogenAtom d) (n : ℕ) :
    Function.Injective (boundStateMap H n)
```

lemma finrank_range_boundStateMap [source]

The bound states of the level `n` span a space of dimension `Σ_{l ≤ n} dim Harm_l`.

```
lemma finrank_range_boundStateMap (H : QuantumMechanics.HydrogenAtom d) (n : ℕ) :
    Module.finrank ℝ (LinearMap.range (boundStateMap H n)) =
      ∑ l : Fin (n + 1), Module.finrank ℝ (harmonicSubmodule d l)
```

lemma finrank_range_boundStateMap_three [source]

In three dimensions the level `n` carries `(n + 1)²` linearly independent bound states.

```
lemma finrank_range_boundStateMap_three (H : QuantumMechanics.HydrogenAtom 3)
    (n : ℕ) : Module.finrank ℝ (LinearMap.range (boundStateMap H n)) = (n + 1) ^ 2
```

2.585 PhyslibAlpha.QuantumMechanics.Hydrogen.Eigenfunctions

PhyslibAlpha/QuantumMechanics/Hydrogen/Eigenfunctions.lean on GitHub.

lemma isOpen_normSq_pos [source]

```
lemma isOpen_normSq_pos : IsOpen {y : Space d | 0 < ‖y‖ ^ 2}
```

lemma differentiableAt_normSq [source]

```
lemma differentiableAt_normSq (x : Space d) : DifferentiableAt ℝ (fun y : Space d => ‖y‖ ^ 2) x
```

lemma deriv_comp_normSq [source]

`∂ᵢ G(‖y‖²) = G'(‖y‖²) 2 yᵢ.`

```
lemma deriv_comp_normSq {G : ℝ → ℝ} {g : ℝ} {x : Space d} (hG : HasDerivAt G g (‖x‖ ^ 2))
    (i : Fin d) : ∂[i] (fun y : Space d => G (‖y‖ ^ 2)) x = g * (2 * x i)
```

lemma differentiableAt_comp_normSq [source]

```
lemma differentiableAt_comp_normSq {G : ℝ → ℝ} {g : ℝ} {x : Space d}
    (hG : HasDerivAt G g (‖x‖ ^ 2)) : DifferentiableAt ℝ (fun y : Space d => G (‖y‖ ^ 2)) x
```

lemma deriv_add_apply

[source]

```
lemma deriv_add_apply {A B : Space d → ℝ} {x : Space d} (hA : DifferentiableAt ℝ A x)
    (hB : DifferentiableAt ℝ B x) (i : Fin d) :
    ∂[i] (fun y => A y + B y) x = ∂[i] A x + ∂[i] B x
```

lemma deriv_mul_apply

[source]

```
lemma deriv_mul_apply {A B : Space d → ℝ} {x : Space d} (hA : DifferentiableAt ℝ A x)
    (hB : DifferentiableAt ℝ B x) (i : Fin d) :
    ∂[i] (fun y => A y * B y) x = A x * ∂[i] B x + B x * ∂[i] A x
```

lemma laplacian_radial_mul

[source]

***The Laplacian of `G(‖y‖²) P(y)`** away from the origin: `Δ(G P) = G ΔP + 4 G' Σᵢ yᵢ ∂ᵢP + (2d G' + 4‖y‖² G'') P`.*

```
lemma laplacian_radial_mul {G G' G'' : ℝ → ℝ} (hG : ∀ s, 0 < s → HasDerivAt G (G' s) s)
    (hG' : ∀ s, 0 < s → HasDerivAt G' (G'' s) s) {P : Space d → ℝ} (hP : ContDiff ℝ 2 P)
    {x : Space d} (hx : x ≠ 0) :
    Δ (fun y => G (‖y‖ ^ 2) * P y) x = G (‖x‖ ^ 2) * Δ P x +
      4 * G' (‖x‖ ^ 2) * ∑ i, x i * ∂[i] P x +
        (2 * d * G' (‖x‖ ^ 2) + 4 * ‖x‖ ^ 2 * G'' (‖x‖ ^ 2)) * P x
```

lemma hasDerivAt_comp_sqrt

[source]

```
lemma hasDerivAt_comp_sqrt {f f' : ℝ → ℝ} (hf : ∀ r, 0 < r → HasDerivAt f (f' r) r) {s : ℝ}
    (hs : 0 < s) :
    HasDerivAt (fun s => f (Real.sqrt s)) (f' (Real.sqrt s) / (2 * Real.sqrt s)) s
```

lemma hasDerivAt_comp_sqrt'

[source]

```
lemma hasDerivAt_comp_sqrt' {f' f'' : ℝ → ℝ} (hf : ∀ r, 0 < r → HasDerivAt f' (f'' r) r) {s :
    ℝ}
    (hs : 0 < s) : HasDerivAt (fun s => f' (Real.sqrt s) / (2 * Real.sqrt s))
      ((f'' (Real.sqrt s) - f' (Real.sqrt s) / Real.sqrt s) / (4 * s)) s
```

lemma laplacian_radial_harmonic

[source]

The Laplacian of `f(‖y‖) P(y)` for a homogeneous harmonic polynomial `P` of degree `l`: `Δ(f P) = (f'' + (d - 1 + 2l) f'/r) P`.

```
lemma laplacian_radial_harmonic {f f' f'' : ℝ → ℝ} (hf : ∀ r, 0 < r → HasDerivAt f (f' r) r)
    (hf' : ∀ r, 0 < r → HasDerivAt f' (f'' r) r) {P : Space d → ℝ} (hP : ContDiff ℝ 2 P) {l :
    ℕ}
    (hE : ∀ y : Space d, ∑ i, y i * ∂[i] P y = l * P y) (hΔ : ∀ y, Δ P y = 0) {x : Space d}
    (hx : x ≠ 0) :
    Δ (fun y => f ‖y‖ * P y) x =
      (f'' ‖x‖ + ((d : ℝ) - 1 + 2 * l) * f' ‖x‖ / ‖x‖) * P x
```

def laguerreScaled

[source]

The polynomial `L_N^(α)(2κr)` in `r`.

```
def laguerreScaled (κ : ℝ) (N α : ℕ) : ℝ[X]
```

def radialFactor [source]

The radial factor `e^(-κr) L_N^(α)(2κr)` of a bound state.

```
def radialFactor (κ : ℝ) (N α : ℕ) (r : ℝ) : ℝ
```

def radialFactor' [source]

The first derivative of `radialFactor`.

```
def radialFactor' (κ : ℝ) (N α : ℕ) (r : ℝ) : ℝ
```

def radialFactor'' [source]

The second derivative of `radialFactor`.

```
def radialFactor'' (κ : ℝ) (N α : ℕ) (r : ℝ) : ℝ
```

lemma hasDerivAt_radialFactor [source]

```
lemma hasDerivAt_radialFactor (κ : ℝ) (N α : ℕ) (r : ℝ) :
    HasDerivAt (radialFactor κ N α) (radialFactor' κ N α r) r
```

lemma hasDerivAt_radialFactor' [source]

```
lemma hasDerivAt_radialFactor' (κ : ℝ) (N α : ℕ) (r : ℝ) :
    HasDerivAt (radialFactor' κ N α) (radialFactor'' κ N α r) r
```

lemma radialFactor_ode [source]

The radial equation of the bound state: `r f'' + (α + 1) f' + (2κ(N + (α + 1)/2) - κ² r) f = 0`.

```
lemma radialFactor_ode (κ : ℝ) (N α : ℕ) (r : ℝ) :
    r * radialFactor'' κ N α r + ((α : ℝ) + 1) * radialFactor' κ N α r +
      (2 * κ * ((N : ℝ) + ((α : ℝ) + 1) / 2) - κ ^ 2 * r) * radialFactor κ N α r = 0
```

def boundKappa [source]

The decay constant `κ = 1/(a L_n)` of the level `n`, with `L_n = n + (d - 1)/2`.

```
def boundKappa (H : HydrogenAtom d) (n : ℕ) : ℝ
```

def boundState [source]

The bound state `ψ(y) = e^(-κ r) L_N^(2l+d-2)(2κ r) P(y)`, `r = ‖y‖`, of the level `n = N + l`, built from a homogeneous harmonic polynomial `P` of degree `l`.

```
def boundState (H : HydrogenAtom d) (N l : ℕ) (P : Space d → ℝ) (y : Space d) : ℝ
```

lemma boundState_eigen [source]

***Bound states of the hydrogen atom in `d ≥ 2` dimensions.** Let `P` be a homogeneous (`Σᵢ yᵢ ∂ᵢP = l P`) harmonic (`ΔP = 0`) polynomial of degree `l`, for instance `r^l Y_lm` in three dimensions. Then away from the nucleus `-ħ²/(2m) Δψ - (k/r) ψ = E_{N+l} ψ` for `ψ = boundState H N l P`, where `E_n` is the Bohr level `HydrogenAtom.energyLevel n = -Ry/(n + (d - 1)/2)²`.*

```
lemma boundState_eigen (H : HydrogenAtom d) (hd : 2 ≤ d) (hk : 0 < H.k) (N l : ℕ)
    {P : Space d → ℝ} (hP : ContDiff ℝ 2 P) (hE : ∀ y : Space d, ∑ i, y i * ∂[i] P y = l * P y)
    (hΔ : ∀ y, Δ P y = 0) {x : Space d} (hx : x ≠ 0) :
    -((ℏ : ℝ) ^ 2 / (2 * H.m)) * Δ (boundState H N l P) x + H.potential x * boundState H N l P
    x =
      H.energyLevel (N + l) * boundState H N l P x
```

lemma boundState_eigen_s [source]

The constant polynomial is homogeneous of degree `0` and harmonic: the `s` states.

```
lemma boundState_eigen_s (H : HydrogenAtom d) (hd : 2 ≤ d) (hk : 0 < H.k) (N : ℕ)
    {x : Space d} (hx : x ≠ 0) :
    -((ℏ : ℝ) ^ 2 / (2 * H.m)) * Δ (boundState H N 0 fun _ => 1) x +
        H.potential x * boundState H N 0 (fun _ => 1) x =
      H.energyLevel N * boundState H N 0 (fun _ => 1) x
```

lemma boundState_eigen_p [source]

A coordinate `yⱼ` is homogeneous of degree `1` and harmonic: the `p` states.

```
lemma boundState_eigen_p (H : HydrogenAtom d) (hd : 2 ≤ d) (hk : 0 < H.k) (N : ℕ) (j : Fin d)
    {x : Space d} (hx : x ≠ 0) :
    -((ℏ : ℝ) ^ 2 / (2 * H.m)) * Δ (boundState H N 1 fun y => y j) x +
        H.potential x * boundState H N 1 (fun y => y j) x =
      H.energyLevel (N + 1) * boundState H N 1 (fun y => y j) x
```

2.586 PhyslibAlpha.QuantumMechanics.Hydrogen.FineStructure

PhyslibAlpha/QuantumMechanics/Hydrogen/FineStructure.lean on GitHub.

def fineStructureConstant [source]

The fine-structure constant `α = k/(ℏ c)` of the atom.

```
def fineStructureConstant : ℝ
```

def relativisticShift [source]

The relativistic kinetic correction `-⟨p⁴⟩/(8 m³ c²) = -⟨(E - V)²⟩/(2 m c²)` in the state `(ν, l)`, written with `p² = 2m(E - V)`, `V = -k/r` and the radial expectation values `⟨1/r⟩ = ⟨1/ρ⟩/a`, `⟨1/r²⟩ = ⟨1/ρ²⟩/a²`.

```
def relativisticShift (ν l : ℕ) : ℝ
```

def spinOrbitShift [source]

The spin–orbit correction `k/(2 m² c²) ⟨1/r³⟩ ⟨L · S⟩`, where `⟨L · S⟩ = ħ² ls`.

```
def spinOrbitShift (ν l : ℕ) (ls : ℝ) : ℝ
```

def darwinShift [source]

The Darwin correction `π k ħ²/(2 m² c²) |ψ(0)|²`, which is non-zero only for `s` states; there `|ψ(0)|² = |R(0)|²/(4π)` with `R` normalised in physical units.

```
def darwinShift (ν l : ℕ) : ℝ
```

def fineStructureShift [source]

The total first-order fine-structure shift of the level `(ν, l)` with `⟨L · S⟩ = ħ² ls`.

```
def fineStructureShift (ν l : ℕ) (ls : ℝ) : ℝ
```

lemma levelIndex_sub_one [source]

```
lemma levelIndex_sub_one (hν : 1 ≤ ν) : HydrogenAtom.levelIndex 3 (ν - 1) = ν
```

lemma ls_upper [source]

```
lemma ls_upper {ls : ℝ} (hls : (ls : ℂ) = Sl2.spinOrbitValue (2 * l) 1 0) : ls = l / 2
```

lemma ls_lower [source]

```
lemma ls_lower (hl1 : 1 ≤ l) {ls : ℝ} (hls : (ls : ℂ) = Sl2.spinOrbitValue (2 * l) 1 1) :
    ls = -((l : ℝ) + 1) / 2
```

lemma fineStructureShift_upper [source]

***Fine structure, `j = l + 1/2`.** With `⟨L · S⟩/ħ²` the eigenvalue of `L · S` on the `j = l + 1/2` string (`Sl2.dot_coupled_stringVec`), the three corrections add up to `E_ν α²/ν² (ν/(j + 1/2) - 3/4)`. For `l = 0` the spin–orbit term vanishes and the Darwin term takes its place.*

```
lemma fineStructureShift_upper (hk : 0 < H.k) (hl : l < ν) {ls : ℝ}
    (hls : (ls : ℂ) = Sl2.spinOrbitValue (2 * l) 1 0) :
    fineStructureShift H c ν l ls = H.energyLevel (ν - 1) * fineStructureConstant H c ^ 2 /
      (ν : ℝ) ^ 2 * ((ν : ℝ) / ((l + 1 / 2 : ℝ) + 1 / 2) - 3 / 4)
```

lemma fineStructureShift_lower [source]

***Fine structure, `j = l - 1/2`.** For `l ≥ 1` the three corrections add up to `E_ν α²/ν² (ν/(j + 1/2) - 3/4)` with `j = l - 1/2`.*

```
lemma fineStructureShift_lower (hk : 0 < H.k) (hl : l < ν) (hl1 : 1 ≤ l)
    {ls : ℝ} (hls : (ls : ℂ) = Sl2.spinOrbitValue (2 * l) 1 1) :
    fineStructureShift H c ν l ls = H.energyLevel (ν - 1) * fineStructureConstant H c ^ 2 /
      (ν : ℝ) ^ 2 * ((ν : ℝ) / ((l - 1 / 2 : ℝ) + 1 / 2) - 3 / 4)
```

lemma fineStructureShift_upper_eq_lower [source]

***The fine-structure shift depends on `j` only.** The levels `(ν, l, j = l + 1/2)` and `(ν, l + 1, j = l + 1/2)` are shifted by the same amount; for `ν = 2`, `l = 0` this is the degeneracy of `2s_{1/2}` and `2p_{1/2}` that the Lamb shift lifts.*

```
lemma fineStructureShift_upper_eq_lower (hk : 0 < H.k) (hl : l + 1 < ν)
    {ls ls' : ℝ} (hls : (ls : ℂ) = Sl2.spinOrbitValue (2 * l) 1 0)
    (hls' : (ls' : ℂ) = Sl2.spinOrbitValue (2 * (l + 1)) 1 1) :
    fineStructureShift H c ν l ls = fineStructureShift H c ν (l + 1) ls'
```

2.587 PhyslibAlpha.QuantumMechanics.Hydrogen.PauliSpectrum

PhyslibAlpha/QuantumMechanics/Hydrogen/PauliSpectrum.lean on GitHub.

structure PauliSystem [source]

Pauli's abstract hydrogen system for a hydrogen atom `Hy` in three dimensions: angular momentum `L`, Laplace-Runge-Lenz vector `A` and Hamiltonian `H` on a complex vector space `V`, satisfying the commutation relations of the hydrogen operators (with `ε → 0`).

```
structure PauliSystem (Hy : HydrogenAtom 3) where
  L : Fin 3 → Module.End ℂ V
  A : Fin 3 → Module.End ℂ V
  H : Module.End ℂ V
  comm_L_L : ∀ i, ⁅L i, L (i + 1)⁆ = (Complex.I * (ℏ : ℝ)) • L (i + 2)
  comm_L_A_self : ∀ i, ⁅L i, A i⁆ = 0
  comm_L_A_succ : ∀ i, ⁅L i, A (i + 1)⁆ = (Complex.I * (ℏ : ℝ)) • A (i + 2)
  comm_L_A_pred : ∀ i, ⁅L (i + 1), A i⁆ = -((Complex.I * (ℏ : ℝ)) • A (i + 2))
  comm_A_A : ∀ i, ⁅A i, A (i + 1)⁆ = (-2 * Complex.I * (ℏ : ℝ) * Hy.m) • (H * L (i + 2))
  comm_H_L : ∀ i, ⁅H, L i⁆ = 0
  comm_H_A : ∀ i, ⁅H, A i⁆ = 0
  A_sq : ∑ i, A i * A i =
    (2 * Hy.m : ℂ) • (H * (∑ i, L i * L i + ((ℏ : ℝ) ^ 2 : ℂ) • 1)) +
      ((Hy.m : ℂ) ^ 2 * (Hy.k : ℂ) ^ 2) • 1
  L_dot_A : ∑ i, L i * A i = 0
```

def kappa [source]

The rescaling `κ = √(-2mE)` of the Laplace-Runge-Lenz vector at energy `E`.

```
def kappa (Hy : HydrogenAtom 3) (E : ℝ) : ℂ
```

lemma kappa_sq [source]

`κ² = -2mE` for `E < 0`.

```
lemma kappa_sq {E : ℝ} (hE : E < 0) : kappa Hy E ^ 2 = -2 * Hy.m * E
```

lemma kappa_ne_zero [source]

`κ ≠ 0` for `E < 0`.

```
lemma kappa_ne_zero {E : ℝ} (hE : E < 0) : kappa Hy E ≠ 0
```

lemma ne_zero_of_sq_eq [source]

A square root κ of -2mE is nonzero for E < 0.

```
lemma ne_zero_of_sq_eq {E : ℝ} (hE : E < 0) {κ : ℂ} (hκ2 : κ ^ 2 = -2 * Hy.m * E) : κ ≠ 0
```

def J [source]

The components Jᵢ = (Lᵢ + Aᵢ / κ) / (2ħ).

```
def J (P : PauliSystem V Hy) (κ : ℂ) (i : Fin 3) : Module.End ℂ V
```

lemma comm_J [source]

⁅Jᵢ, Jᵢ₊₁⁆ = i Jᵢ₊₂ on a space where H = E < 0, for κ² = -2mE.

```
lemma comm_J (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0) {κ : ℂ}
    (hκ2 : κ ^ 2 = -2 * Hy.m * E) (hH : P.H = (E : ℂ) • 1) (i : Fin 3) :
    ⁅P.J κ i, P.J κ (i + 1)⁆ = Complex.I • P.J κ (i + 2)
```

def rep [source]

The angular momentum representation J = (L + A / κ) / (2ħ) for κ² = -2mE.

```
def rep (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0) {κ : ℂ} (hκ2 : κ ^ 2 = -2 * Hy.m * E)
    (hH : P.H = (E : ℂ) • 1) : Sl2.AngularMomentumRepresentation V where
```

lemma rep_Jsq [source]

J² = (-ħ² - m k² / (2E)) / (4ħ²) on a space where H = E < 0, for κ² = -2mE.

```
lemma rep_Jsq (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0) {κ : ℂ}
    (hκ2 : κ ^ 2 = -2 * Hy.m * E) (hH : P.H = (E : ℂ) • 1) :
    (P.rep hE hκ2 hH).Jsq = ((2 * ((ħ : ℝ) : ℂ))⁻¹ ^ 2 *
      (-((ħ : ℝ) : ℂ) ^ 2 - Hy.m * Hy.k ^ 2 / (2 * E))) • 1
```

def plusRep [source]

The angular momentum representation J₊ = (L + K) / (2ħ), with K = A / κ.

```
def plusRep (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0) (hH : P.H = (E : ℂ) • 1) :
    Sl2.AngularMomentumRepresentation V
```

def minusRep [source]

The angular momentum representation J₋ = (L - K) / (2ħ), with K = A / κ.

```
def minusRep (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0) (hH : P.H = (E : ℂ) • 1) :
    Sl2.AngularMomentumRepresentation V
```

lemma plusRep_Jsq [source]

J₊² = (-ħ² - m k² / (2E)) / (4ħ²) on a space where H = E < 0.

```
lemma plusRep_Jsq (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0) (hH : P.H = (E : ℂ) • 1) :
    (P.plusRep hE hH).Jsq = ((2 * ((ℏ : ℝ) : ℂ))⁻¹ ^ 2 *
      (-((ℏ : ℝ) : ℂ) ^ 2 - Hy.m * Hy.k ^ 2 / (2 * E))) • 1
```

lemma minusRep_Jsq [source]

$J_-^2 = J_+^2$: both are $(-\hbar^2 - m\,k^2 / (2E)) / (4\hbar^2)$ on a space where $H = E < 0$.

```
lemma minusRep_Jsq (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0) (hH : P.H = (E : ℂ) • 1) :
    (P.minusRep hE hH).Jsq = ((2 * ((ℏ : ℝ) : ℂ))⁻¹ ^ 2 *
      (-((ℏ : ℝ) : ℂ) ^ 2 - Hy.m * Hy.k ^ 2 / (2 * E))) • 1
```

lemma comm_J_neg_succ [source]

$[J(\kappa)_i, J(-\kappa)_{i+1}] = 0$ on a space where $H = E < 0$, for $\kappa^2 = -2mE$.

```
lemma comm_J_neg_succ (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0) {κ : ℂ}
    (hκ2 : κ ^ 2 = -2 * Hy.m * E) (hH : P.H = (E : ℂ) • 1) (i : Fin 3) :
    ⁅P.J κ i, P.J (-κ) (i + 1)⁆ = 0
```

lemma comm_J_neg [source]

$[J(\kappa)_i, J(-\kappa)_j] = 0$ on a space where $H = E < 0$, for $\kappa^2 = -2mE$.

```
lemma comm_J_neg (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0) {κ : ℂ}
    (hκ2 : κ ^ 2 = -2 * Hy.m * E) (hH : P.H = (E : ℂ) • 1) (i j : Fin 3) :
    ⁅P.J κ i, P.J (-κ) j⁆ = 0
```

lemma plusRep_comm_minusRep [source]

The $\mathfrak{sl}_2$ triples of J_+ and J_- commute.

```
lemma plusRep_comm_minusRep (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0)
    (hH : P.H = (E : ℂ) • 1) :
    ∀ x ∈ ({(2 : ℂ) • (P.plusRep hE hH).Jz, (P.plusRep hE hH).Jp, (P.plusRep hE hH).Jm} :
        Set (Module.End ℂ V)),
      ∀ y ∈ ({(2 : ℂ) • (P.minusRep hE hH).Jz, (P.minusRep hE hH).Jp,
          (P.minusRep hE hH).Jm} : Set (Module.End ℂ V)), ⁅x, y⁆ = 0
```

lemma exists_Jsq_eigenvector [source]

On a nonzero finite-dimensional space J^2 has an eigenvalue $j(j+1)$ with $2j \in \mathbb{N}$.

```
lemma _root_.PhyslibAlpha.Sl2.AngularMomentumRepresentation.exists_Jsq_eigenvector
    [FiniteDimensional ℂ V] [Nontrivial V] (ρ : Sl2.AngularMomentumRepresentation V) :
    ∃ (n : ℕ) (v : V), v ≠ 0 ∧ ρ.Jsq v = ((n : ℂ) / 2 * ((n : ℂ) / 2 + 1)) • v
```

lemma eq_energyLevel_of_casimir [source]

If the Casimir $(-\hbar^2 - m\,k^2 / (2E)) / (4\hbar^2)$ equals $j(j+1)$ with $n = 2j$, then E is the Bohr level $E = -m\,k^2 / (2\,\hbar^2\,(n+1)^2)$.

```
lemma eq_energyLevel_of_casimir {E : ℝ} (hE : E < 0) {n : ℕ}
    (hc : (2 * ((ℏ : ℝ) : ℂ))⁻¹ ^ 2 * (-((ℏ : ℝ) : ℂ) ^ 2 - Hy.m * Hy.k ^ 2 / (2 * E)) =
```

```
    (n : ℂ) / 2 * ((n : ℂ) / 2 + 1)) :
  E = Hy.energyLevel n
```

lemma exists_energyLevel_of_H_eq [source]

Pauli's theorem on a nonzero finite-dimensional space on which `H = E < 0`.

```
lemma exists_energyLevel_of_H_eq [FiniteDimensional ℂ V] [Nontrivial V]
    (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0) (hH : P.H = (E : ℂ) • 1) :
    ∃ n : ℕ, E = Hy.energyLevel n
```

lemma mapsTo_eigenspace [source]

Operators commuting with `H` preserve its eigenspaces.

```
lemma mapsTo_eigenspace (P : PauliSystem V Hy) {T : Module.End ℂ V} (hT : ⁅P.H, T⁆ = 0)
    (μ : ℂ) : Set.MapsTo T (P.H.eigenspace μ) (P.H.eigenspace μ)
```

def restrictEigenspace [source]

The restriction of a Pauli system to an eigenspace of `H`.

```
def restrictEigenspace (P : PauliSystem V Hy) (μ : ℂ) :
    PauliSystem (P.H.eigenspace μ) Hy where
```

theorem exists_energyLevel [source]

***Pauli's theorem.** Every nonzero finite-dimensional eigenspace of the Hamiltonian with negative eigenvalue `E` has `E` equal to a Bohr level `-m k² / (2 ℏ² n²)`.*

```
theorem exists_energyLevel (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0)
    [FiniteDimensional ℂ (P.H.eigenspace (E : ℂ))] (hne : P.H.eigenspace (E : ℂ) ≠ ⊥) :
    ∃ n : ℕ, E = Hy.energyLevel n
```

lemma exists_energyLevel_le_finrank_of_H_eq [source]

On a nonzero finite-dimensional space on which `H = E < 0`, `E` is a Bohr level `E = -m k² / (2 ℏ² (n + 1)²)` and the space has dimension at least `(n + 1)²`.

```
lemma exists_energyLevel_le_finrank_of_H_eq [FiniteDimensional ℂ V] [Nontrivial V]
    (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0) (hH : P.H = (E : ℂ) • 1) :
    ∃ n : ℕ, E = Hy.energyLevel n ∧ (n + 1) ^ 2 ≤ Module.finrank ℂ V
```

lemma exists_energyLevel_le_finrank [source]

***Degeneracy of the Bohr levels.** A nonzero finite-dimensional eigenspace of the Hamiltonian with eigenvalue `E < 0` has `E = -m k² / (2 ℏ² (n + 1)²)` and dimension at least `(n + 1)²`.*

```
lemma exists_energyLevel_le_finrank (P : PauliSystem V Hy) {E : ℝ} (hE : E < 0)
    [FiniteDimensional ℂ (P.H.eigenspace (E : ℂ))] (hne : P.H.eigenspace (E : ℂ) ≠ ⊥) :
    ∃ n : ℕ, E = Hy.energyLevel n ∧
      (n + 1) ^ 2 ≤ Module.finrank ℂ (P.H.eigenspace (E : ℂ))
```

2.588 PhyslibAlpha.QuantumMechanics.Hydrogen.RadialCompleteness

PhyslibAlpha/QuantumMechanics/Hydrogen/RadialCompleteness.lean on GitHub.

def radialPot [source]

The radial potential term `q(ρ) = l(l+1)/ρ² - 2/ρ + κ²`, so that the radial equation reads `u'' = q u`.

```
def radialPot (κ : ℝ) (l : ℕ) (ρ : ℝ) : ℝ
```

def regCoeff [source]

The coefficients of the regular solution: `c_0 = 1` and `c_{k+1} = (2κ(k + l + 1) - 2) / ((k + 1)(k + 2l + 2)) c_k`.

```
def regCoeff (κ : ℝ) (l : ℕ) : ℕ → ℝ
  | 0 => 1
  | k + 1 => (2 * κ * (k + l + 1) - 2) / (((k : ℝ) + 1) * (k + 2 * l + 2)) * regCoeff κ l k
```

lemma regCoeff_zero [source]

```
@[simp]
lemma regCoeff_zero : regCoeff κ l 0 = 1
```

lemma radialRec_regCoeff [source]

```
lemma radialRec_regCoeff : RadialRec κ l (regCoeff κ l)
```

lemma entireCoeff_regCoeff [source]

```
lemma entireCoeff_regCoeff : EntireCoeff (regCoeff κ l)
```

lemma radial_ode_of_radialRec [source]

***Converse of `recursion_of_radial_ode`.** If the coefficients satisfy the recursion, the ansatz solves the radial equation on `(0, ∞)`.*

```
lemma radial_ode_of_radialRec {c : ℕ → ℝ} (hc : EntireCoeff c) (hrec : RadialRec κ l c) {ρ : ℝ}
    (hρ : 0 < ρ) :
    HasDerivAt (radialSeriesD κ l c) (radialPot κ l ρ * radialSeries κ l c ρ) ρ
```

lemma regSol_near_zero [source]

Near the origin the regular solution is comparable to `ρ^(l+1)`.

```
lemma regSol_near_zero : ∃ δ > 0, ∃ a > 0, ∃ b > 0, ∀ ρ ∈ Ioc 0 δ,
    a * ρ ^ (l + 1) ≤ radialSeries κ l (regCoeff κ l) ρ ∧
      radialSeries κ l (regCoeff κ l) ρ ≤ b * ρ ^ (l + 1)
```

lemma exists_const_of_hasDerivAt_zero [source]

A function with vanishing derivative on an open preconnected set is constant there.

```
lemma exists_const_of_hasDerivAt_zero {f : ℝ → ℝ} {s : Set ℝ} (hs : IsOpen s)
    (hs' : IsPreconnected s) (h : ∀ x ∈ s, HasDerivAt f 0 x) : ∃ a, ∀ x ∈ s, f x = a
```

lemma wronskian_const [source]

The Wronskian of two solutions of `u'' = q u` on `(0, ∞)` is constant.

```
lemma wronskian_const {q u u' v v' : ℝ → ℝ}
    (hu : ∀ ρ, 0 < ρ → HasDerivAt u (u' ρ) ρ) (hu' : ∀ ρ, 0 < ρ → HasDerivAt u' (q ρ * u ρ) ρ)
    (hv : ∀ ρ, 0 < ρ → HasDerivAt v (v' ρ) ρ) (hv' : ∀ ρ, 0 < ρ → HasDerivAt v' (q ρ * v ρ) ρ)
    :
    ∃ W : ℝ, ∀ ρ, 0 < ρ → u ρ * v' ρ - u' ρ * v ρ = W
```

lemma wronskian_nonpos_of_tendsto [source]

One sign of the Wronskian: if it were positive, `u/u₁ - K ρ^(-(2l+1))` would be antitone near `0`, so `u(ρ) ≥ K a - |M| b ρ` for small `ρ`, contradicting `u → 0`.

```
lemma wronskian_nonpos_of_tendsto {u u' : ℝ → ℝ}
    (hu : ∀ ρ, 0 < ρ → HasDerivAt u (u' ρ) ρ)
    (hu' : ∀ ρ, 0 < ρ → HasDerivAt u' (radialPot κ l ρ * u ρ) ρ)
    (h0 : Tendsto u (𝓝[>] 0) (𝓝 0)) {ρ₀ : ℝ} (hρ₀ : 0 < ρ₀) :
    u ρ₀ * radialSeriesD κ l (regCoeff κ l) ρ₀ -
      u' ρ₀ * radialSeries κ l (regCoeff κ l) ρ₀ ≤ 0
```

lemma wronskian_eq_zero_of_tendsto [source]

A solution tending to `0` at the origin has vanishing Wronskian with the regular solution.

```
lemma wronskian_eq_zero_of_tendsto {u u' : ℝ → ℝ}
    (hu : ∀ ρ, 0 < ρ → HasDerivAt u (u' ρ) ρ)
    (hu' : ∀ ρ, 0 < ρ → HasDerivAt u' (radialPot κ l ρ * u ρ) ρ)
    (h0 : Tendsto u (𝓝[>] 0) (𝓝 0)) {ρ : ℝ} (hρ : 0 < ρ) :
    u ρ * radialSeriesD κ l (regCoeff κ l) ρ - u' ρ * radialSeries κ l (regCoeff κ l) ρ = 0
```

lemma exists_eqOn_near_zero_of_tendsto [source]

A solution tending to `0` at the origin is a multiple of the regular solution near `0`.

```
lemma exists_eqOn_near_zero_of_tendsto {u u' : ℝ → ℝ}
    (hu : ∀ ρ, 0 < ρ → HasDerivAt u (u' ρ) ρ)
    (hu' : ∀ ρ, 0 < ρ → HasDerivAt u' (radialPot κ l ρ * u ρ) ρ)
    (h0 : Tendsto u (𝓝[>] 0) (𝓝 0)) :
    ∃ δ > 0, ∃ A : ℝ, ∀ ρ ∈ Ioo 0 δ, u ρ = A * radialSeries κ l (regCoeff κ l) ρ
```

lemma eqOn_zero_of_eqOn_Ioo [source]

A solution of `w'' = q w` with continuous `q` that vanishes on `(0, δ)` vanishes on `(0, ∞)`.

```
lemma eqOn_zero_of_eqOn_Ioo {q w w' : ℝ → ℝ} (hq : ContinuousOn q (Ioi 0))
    (hw : ∀ ρ, 0 < ρ → HasDerivAt w (w' ρ) ρ) (hw' : ∀ ρ, 0 < ρ → HasDerivAt w' (q ρ * w ρ) ρ)
    {δ : ℝ} (hδ : 0 < δ) (h : ∀ ρ ∈ Ioo 0 δ, w ρ = 0) :
    ∀ ρ, 0 < ρ → w ρ = 0
```

lemma radial_regular_solution [source]

***Completeness of the regular radial solutions.** Every solution of the radial equation `u'' = (l(l+1)/ρ² - 2/ρ + κ²) u` on `(0, ∞)` with `u(ρ) → 0` as `ρ → 0⁺` is of the form `ρ^(l+1) e^(-κρ) Σ_k c_k ρ^k` with an everywhere convergent series whose coefficients satisfy the recursion; if `u` is not identically zero on `(0, ∞)`, then `c_0 ≠ 0`.*

```
lemma radial_regular_solution {u : ℝ → ℝ}
    (hd : ∀ ρ, 0 < ρ → DifferentiableAt ℝ u ρ ∧ DifferentiableAt ℝ (deriv u) ρ)
    (hode : ∀ ρ, 0 < ρ → deriv (deriv u) ρ =
      ((l : ℝ) * (l + 1) / ρ ^ 2 - 2 / ρ + κ ^ 2) * u ρ)
    (h0 : Tendsto u (𝓝[>] 0) (𝓝 0)) :
    ∃ c : ℕ → ℝ, EntireCoeff c ∧ RadialRec κ l c ∧ (∀ ρ, 0 < ρ → u ρ = radialSeries κ l c ρ) ∧
      ((∃ ρ, 0 < ρ ∧ u ρ ≠ 0) → c 0 ≠ 0)
```

lemma radial_quantization_of_tendsto [source]

***Quantisation of the hydrogen levels without ansatz.** A non-zero square integrable solution of the radial equation on `(0, ∞)` with energy `-κ²`, `κ > 0`, and `u(ρ) → 0` as `ρ → 0⁺` exists only for `κ = 1/ν` with a natural number `ν > l`, and is then a multiple of `u_{νl}`.*

```
lemma radial_quantization_of_tendsto (hκ : 0 < κ) {u : ℝ → ℝ}
    (hd : ∀ ρ, 0 < ρ → DifferentiableAt ℝ u ρ ∧ DifferentiableAt ℝ (deriv u) ρ)
    (hode : ∀ ρ, 0 < ρ → deriv (deriv u) ρ =
      ((l : ℝ) * (l + 1) / ρ ^ 2 - 2 / ρ + κ ^ 2) * u ρ)
    (h0 : Tendsto u (𝓝[>] 0) (𝓝 0)) (hint : IntegrableOn (fun ρ => u ρ ^ 2) (Ioi 0))
    (hne : ∃ ρ, 0 < ρ ∧ u ρ ≠ 0) :
    ∃ ν : ℕ, l < ν ∧ κ = 1 / (ν : ℝ) ∧ ∃ C : ℝ, ∀ ρ, 0 < ρ → u ρ = C * radialU ν l ρ
```

2.589 PhyslibAlpha.QuantumMechanics.Hydrogen.RadialFunctions

PhyslibAlpha/QuantumMechanics/Hydrogen/RadialFunctions.lean on GitHub.

lemma hasDerivAt_eval_mul_exp [source]

```
lemma hasDerivAt_eval_mul_exp (Q : ℝ[X]) (β ρ : ℝ) :
    HasDerivAt (fun ρ => Q.eval ρ * Real.exp (-(β * ρ)))
      ((derivative Q - C β * Q).eval ρ * Real.exp (-(β * ρ))) ρ
```

lemma deriv_eval_mul_exp [source]

```
lemma deriv_eval_mul_exp (Q : ℝ[X]) (β : ℝ) :
    deriv (fun ρ => Q.eval ρ * Real.exp (-(β * ρ))) =
      fun ρ => (derivative Q - C β * Q).eval ρ * Real.exp (-(β * ρ))
```

def radialPoly [source]

The polynomial `ρ^(l+1) L_{ν-l-1}^(2l+1)(2ρ/ν)`.

```
def radialPoly : ℝ[X]
```

def radialU [source]

The reduced radial function `u_{νl}(ρ) = ρ^(l+1) e^(-ρ/ν) L_{ν-l-1}^(2l+1)(2ρ/ν)` of the hydrogen atom, in units of the Bohr radius and not normalised; `ν` is the principal quantum number. The radial wave function is `R_{νl}(ρ) = u_{νl}(ρ)/ρ`.

```
def radialU (ρ : ℝ) : ℝ
```

lemma eval_radialPoly [source]

```
lemma eval_radialPoly (ρ : ℝ) :
    (radialPoly ν l).eval ρ =
      ρ ^ (l + 1) * (laguerre (ν - l - 1) (2 * l + 1)).eval (2 / (ν : ℝ) * ρ)
```

lemma eval_derivative_radialPoly [source]

```
lemma eval_derivative_radialPoly (ρ : ℝ) :
    (derivative (radialPoly ν l)).eval ρ =
      ((l : ℝ) + 1) * ρ ^ l * (laguerre (ν - l - 1) (2 * l + 1)).eval (2 / (ν : ℝ) * ρ) +
        2 / (ν : ℝ) * ρ ^ (l + 1) *
          (derivative (laguerre (ν - l - 1) (2 * l + 1))).eval (2 / (ν : ℝ) * ρ)
```

lemma eval_derivative_derivative_radialPoly [source]

```
lemma eval_derivative_derivative_radialPoly (ρ : ℝ) :
    (derivative (derivative (radialPoly ν l))).eval ρ =
      ((l : ℝ) + 1) * l * ρ ^ (l - 1) *
          (laguerre (ν - l - 1) (2 * l + 1)).eval (2 / (ν : ℝ) * ρ) +
        2 * ((l : ℝ) + 1) * (2 / (ν : ℝ)) * ρ ^ l *
          (derivative (laguerre (ν - l - 1) (2 * l + 1))).eval (2 / (ν : ℝ) * ρ) +
        (2 / (ν : ℝ)) ^ 2 * ρ ^ (l + 1) *
          (derivative (derivative (laguerre (ν - l - 1) (2 * l + 1)))).eval
            (2 / (ν : ℝ) * ρ)
```

lemma radialPoly_identity [source]

The polynomial identity behind the radial equation: `ρ² (Q'' - 2Q'/ν + Q/ν²) = (l(l+1) - 2ρ + ρ²/ν²) Q` for `Q = radialPoly ν l`.

```
lemma radialPoly_identity (hl : l < ν) (ρ : ℝ) :
    ρ ^ 2 * ((derivative (derivative (radialPoly ν l))).eval ρ -
        2 * (1 / (ν : ℝ)) * (derivative (radialPoly ν l)).eval ρ +
          (1 / (ν : ℝ)) ^ 2 * (radialPoly ν l).eval ρ) =
      ((l : ℝ) * (l + 1) - 2 * ρ + (1 / (ν : ℝ)) ^ 2 * ρ ^ 2) * (radialPoly ν l).eval ρ
```

lemma deriv_deriv_radialU [source]

```
lemma deriv_deriv_radialU (ρ : ℝ) :
    deriv (deriv (radialU ν l)) ρ =
      ((derivative (derivative (radialPoly ν l))).eval ρ -
        2 * (1 / (ν : ℝ)) * (derivative (radialPoly ν l)).eval ρ +
          (1 / (ν : ℝ)) ^ 2 * (radialPoly ν l).eval ρ) * Real.exp (-(1 / (ν : ℝ) * ρ))
```

lemma radialU_ode [source]

***The radial Schrödinger equation of hydrogen.** In units of the Bohr radius and the Rydberg energy, `u_{νl}'' = (l(l+1)/ρ² - 2/ρ + 1/ν²) u_{νl}`, i.e. `u_{νl}` is an eigenfunction of `-d²/dρ² + l(l+1)/ρ² - 2/ρ` with eigenvalue `-1/ν²`.*

```
lemma radialU_ode (hl : l < ν) {ρ : ℝ} (hρ : ρ ≠ 0) :
    deriv (deriv (radialU ν l)) ρ =
      ((l : ℝ) * (l + 1) / ρ ^ 2 - 2 / ρ + 1 / (ν : ℝ) ^ 2) * radialU ν l ρ
```

lemma integral_pow_mul_exp_mul_laguerre_sq_comp [source]

```
lemma integral_pow_mul_exp_mul_laguerre_sq_comp (e N α : ℕ) {b : ℝ} (hb : 0 < b) :
    ∫ ρ in Ioi (0 : ℝ), ρ ^ e * Real.exp (-(b * ρ)) * ((laguerre N α).eval (b * ρ)) ^ 2 =
      b⁻¹ * (b⁻¹ ^ e *
        ∫ x in Ioi (0 : ℝ), x ^ e * Real.exp (-x) * ((laguerre N α).eval x) ^ 2)
```

lemma radialU_sq_div_pow [source]

```
lemma radialU_sq_div_pow {k : ℕ} (hk : k ≤ 2 * l + 2) {ρ : ℝ} (hρ : 0 < ρ) :
    radialU ν l ρ ^ 2 / ρ ^ k = ρ ^ (2 * l + 2 - k) * Real.exp (-(2 / (ν : ℝ) * ρ)) *
      ((laguerre (ν - l - 1) (2 * l + 1)).eval (2 / (ν : ℝ) * ρ)) ^ 2
```

def radialMoment [source]

The radial moment `∫₀^∞ u_{νl}(ρ)² ρ^(-k) dρ`. For `k = 0` this is the squared norm.

```
def radialMoment (ν l k : ℕ) : ℝ
```

def radialMean [source]

The expectation value `⟨ρ^(-k)⟩` in the state `u_{νl}`.

```
def radialMean (ν l k : ℕ) : ℝ
```

lemma radialMoment_eq [source]

```
lemma radialMoment_eq (hl : l < ν) {k : ℕ} (hk : k ≤ 2 * l + 2) :
    radialMoment ν l k = ((ν : ℝ) / 2) * (((ν : ℝ) / 2) ^ (2 * l + 2 - k) *
      ∫ x in Ioi (0 : ℝ), x ^ (2 * l + 2 - k) * Real.exp (-x) *
        ((laguerre (ν - l - 1) (2 * l + 1)).eval x) ^ 2)
```

lemma radialMoment_zero [source]

The squared norm `∫ u_{νl}² dρ = (ν/2)^(2l+3) (ν+l)!/(ν-l-1)! · 2ν`.

```
lemma radialMoment_zero (hl : l < ν) :
    radialMoment ν l 0 = ((ν : ℝ) / 2) ^ (2 * l + 3) *
      (((ν + l).factorial : ℝ) / (ν - l - 1).factorial * (2 * ν))
```

lemma radialMoment_one [source]

```
lemma radialMoment_one (hl : l < ν) :
    radialMoment ν l 1 = ((ν : ℝ) / 2) ^ (2 * l + 2) *
```

```
      (((ν + l).factorial : ℝ) / (ν - l - 1).factorial)
```

lemma radialMoment_two [source]

```
lemma radialMoment_two (hl : l < ν) :
    radialMoment ν l 2 = ((ν : ℝ) / 2) ^ (2 * l + 1) *
      (((ν + l).factorial : ℝ) / ((ν - l - 1).factorial * (2 * l + 1)))
```

lemma radialMoment_three [source]

```
lemma radialMoment_three (hl : l < ν) (hl1 : 1 ≤ l) :
    radialMoment ν l 3 = ((ν : ℝ) / 2) ^ (2 * l) *
      (((ν + l).factorial : ℝ) * (2 * ν) /
        ((ν - l - 1).factorial * (2 * l * (2 * l + 1) * (2 * l + 2))))
```

lemma radialMean_one [source]

⟨1/ρ⟩ = 1/ν², the virial theorem for hydrogen.

```
lemma radialMean_one (hl : l < ν) : radialMean ν l 1 = 1 / (ν : ℝ) ^ 2
```

lemma radialMean_two [source]

⟨1/ρ²⟩ = 1/(ν³ (l + 1/2)).

```
lemma radialMean_two (hl : l < ν) :
    radialMean ν l 2 = 1 / ((ν : ℝ) ^ 3 * (l + 1 / 2))
```

lemma radialMean_three [source]

⟨1/ρ³⟩ = 1/(ν³ l (l + 1/2) (l + 1)) for l ≥ 1.

```
lemma radialMean_three (hl : l < ν) (hl1 : 1 ≤ l) :
    radialMean ν l 3 = 1 / ((ν : ℝ) ^ 3 * (l * (l + 1 / 2) * (l + 1)))
```

def radialR [source]

The radial wave function R_{νl}(ρ) = u_{νl}(ρ)/ρ = ρ^l e^(-ρ/ν) L_{ν-l-1}^(2l+1)(2ρ/ν).

```
def radialR (ν l : ℕ) (ρ : ℝ) : ℝ
```

lemma radialU_eq_mul_radialR [source]

```
lemma radialU_eq_mul_radialR (ρ : ℝ) : radialU ν l ρ = ρ * radialR ν l ρ
```

lemma radialR_zero_sq_div [source]

For an s state the normalised density at the nucleus is |R_{ν0}(0)|²/‖u‖² = 4/ν³.

```
lemma radialR_zero_sq_div (hν : 0 < ν) :
    radialR ν 0 0 ^ 2 / radialMoment ν 0 0 = 4 / (ν : ℝ) ^ 3
```

def bohrRadius [source]

The Bohr radius `a = ħ²/(m k)` of a hydrogen atom.

```
def bohrRadius (H : HydrogenAtom d) : ℝ
```

lemma bohrRadius_pos [source]

```
lemma bohrRadius_pos (H : HydrogenAtom d) (hk : 0 < H.k) : 0 < bohrRadius H
```

def radialFun [source]

The reduced radial function `u_{νl}(r/a)` in physical units.

```
def radialFun (H : HydrogenAtom d) (ν l : ℕ) (r : ℝ) : ℝ
```

lemma radialFun_schrodinger [source]

***The radial Schrödinger equation of hydrogen in physical units.** For `d = 3` and an attractive potential, `u_{νl}(r/a)` solves `-ħ²/(2m) u'' + (ħ² l(l+1)/(2m r²) - k/r) u = E u` with `E = -Ry/ν²`, the Bohr level `H.energyLevel (ν - 1)` of physlib.*

```
lemma radialFun_schrodinger (H : HydrogenAtom 3) (hk : 0 < H.k) (hl : l < ν)
    {r : ℝ} (hr : r ≠ 0) :
    -((ℏ : ℝ) ^ 2 / (2 * H.m)) * deriv (deriv (radialFun H ν l)) r +
        ((ℏ : ℝ) ^ 2 * ((l : ℝ) * (l + 1)) / (2 * H.m * r ^ 2) - H.k / r) * radialFun H ν l r =
      H.energyLevel (ν - 1) * radialFun H ν l r
```

2.590 PhyslibAlpha.QuantumMechanics.Hydrogen.RadialQuantization

PhyslibAlpha/QuantumMechanics/Hydrogen/RadialQuantization.lean on GitHub.

def EntireCoeff [source]

The coefficients `c` define a power series that converges absolutely on all of ℝ.

```
def EntireCoeff (c : ℕ → ℝ) : Prop
```

def psum [source]

The power series `Σ c_k x^k`.

```
def psum (c : ℕ → ℝ) (x : ℝ) : ℝ
```

def dcoeff [source]

The coefficients of the derivative, `(k + 1) c_{k+1}`.

```
def dcoeff (c : ℕ → ℝ) (k : ℕ) : ℝ
```

lemma succ_mul_pow_le_pow_succ [source]

```
lemma succ_mul_pow_le_pow_succ {r : ℝ} (hr : 0 ≤ r) (k : ℕ) :
    ((k : ℝ) + 1) * r ^ k ≤ (1 + r) ^ (k + 1)
```

lemma summable [source]

```
lemma EntireCoeff.summable (hc : EntireCoeff c) (x : ℝ) : Summable fun k => c k * x ^ k
```

lemma derivCoeff [source]

```
lemma EntireCoeff.derivCoeff (hc : EntireCoeff c) : EntireCoeff (dcoeff c)
```

lemma shift [source]

```
lemma EntireCoeff.shift (hc : EntireCoeff c) : EntireCoeff fun k => c (k + 1)
```

lemma psum_eq_add [source]

```
lemma psum_eq_add (hc : EntireCoeff c) (x : ℝ) :
    psum c x = c 0 + x * psum (fun k => c (k + 1)) x
```

lemma psum_zero [source]

```
lemma psum_zero (c : ℕ → ℝ) : psum c 0 = c 0
```

lemma hasDerivAt_psum [source]

Termwise differentiation of an everywhere convergent power series.

```
lemma hasDerivAt_psum (hc : EntireCoeff c) (x : ℝ) :
    HasDerivAt (psum c) (psum (dcoeff c) x) x
```

lemma coeff_eq_zero_of_psum [source]

A power series that vanishes on `(0, ∞)` has vanishing coefficients.

```
lemma coeff_eq_zero_of_psum (n : ℕ) : ∀ c : ℕ → ℝ, EntireCoeff c →
    (∀ x : ℝ, 0 < x → psum c x = 0) → c n = 0
```

lemma unshift [source]

```
lemma EntireCoeff.unshift (h : EntireCoeff fun k => c (k + 1)) : EntireCoeff c
```

lemma add [source]

```
lemma EntireCoeff.add {c' : ℕ → ℝ} (hc : EntireCoeff c) (hc' : EntireCoeff c') :
    EntireCoeff fun k => c k + c' k
```

lemma const_mul [source]

```
lemma EntireCoeff.const_mul (hc : EntireCoeff c) (a : ℝ) : EntireCoeff fun k => a * c k
```

lemma psum_add [source]

```
lemma psum_add {c' : ℕ → ℝ} (hc : EntireCoeff c) (hc' : EntireCoeff c') (x : ℝ) :
    psum (fun k => c k + c' k) x = psum c x + psum c' x
```

lemma psum_const_mul [source]

```
lemma psum_const_mul (c : ℕ → ℝ) (a x : ℝ) : psum (fun k => a * c k) x = a * psum c x
```

lemma dcoeff_eq [source]

```
lemma dcoeff_eq (c : ℕ → ℝ) : (fun k : ℕ => ((k + 1 : ℕ) : ℝ) * c (k + 1)) = dcoeff c
```

lemma natMul [source]

```
lemma EntireCoeff.natMul (hc : EntireCoeff c) : EntireCoeff fun k => (k : ℝ) * c k
```

lemma mul_psum_dcoeff [source]

x Σ_k (k + 1) c_{k+1} x^k = Σ_k k c_k x^k.

```
lemma mul_psum_dcoeff (hc : EntireCoeff c) (x : ℝ) :
    x * psum (dcoeff c) x = psum (fun k => (k : ℝ) * c k) x
```

def radialSeries [source]

The ansatz u(ρ) = ρ^(l+1) e^(-κρ) Σ_k c_k ρ^k for a bound state of energy -κ².

```
def radialSeries (κ : ℝ) (l : ℕ) (c : ℕ → ℝ) (ρ : ℝ) : ℝ
```

lemma hasDerivAt_exp_neg_mul [source]

```
lemma hasDerivAt_exp_neg_mul (κ ρ : ℝ) :
    HasDerivAt (fun ρ => Real.exp (-(κ * ρ))) (Real.exp (-(κ * ρ)) * (-κ)) ρ
```

def radialSeriesD [source]

The first derivative of `radialSeries`.

```
def radialSeriesD (κ : ℝ) (l : ℕ) (c : ℕ → ℝ) (ρ : ℝ) : ℝ
```

lemma hasDerivAt_radialSeries [source]

```
lemma hasDerivAt_radialSeries (hc : EntireCoeff c) (ρ : ℝ) :
    HasDerivAt (radialSeries κ l c) (radialSeriesD κ l c ρ) ρ
```

lemma hasDerivAt_radialSeriesD [source]

```
lemma hasDerivAt_radialSeriesD (hc : EntireCoeff c) (ρ : ℝ) :
    HasDerivAt (radialSeriesD κ l c) (Real.exp (-(κ * ρ)) *
      (-κ * ((((l : ℝ) + 1) * ρ ^ l - κ * ρ ^ (l + 1)) * psum c ρ +
        ρ ^ (l + 1) * psum (dcoeff c) ρ) +
```

```
      ((((l : ℝ) + 1) * (l * ρ ^ (l - 1)) - κ * (((l : ℝ) + 1) * ρ ^ l)) * psum c ρ +
        (((l : ℝ) + 1) * ρ ^ l - κ * ρ ^ (l + 1)) * psum (dcoeff c) ρ +
          ((((l : ℝ) + 1) * ρ ^ l) * psum (dcoeff c) ρ +
            ρ ^ (l + 1) * psum (dcoeff (dcoeff c)) ρ)))) ρ
```

lemma radial_identity [source]

The radial operator applied to the ansatz, in terms of the series `W`, `W'`, `W''`.

```
lemma radial_identity (κ ρ W W₁ W₂ E : ℝ) (l : ℕ) (hρ : ρ ≠ 0) :
    E * (-κ * ((((l : ℝ) + 1) * ρ ^ l - κ * ρ ^ (l + 1)) * W + ρ ^ (l + 1) * W₁) +
      ((((l : ℝ) + 1) * (l * ρ ^ (l - 1)) - κ * (((l : ℝ) + 1) * ρ ^ l)) * W +
        (((l : ℝ) + 1) * ρ ^ l - κ * ρ ^ (l + 1)) * W₁ +
          ((((l : ℝ) + 1) * ρ ^ l) * W₁ + ρ ^ (l + 1) * W₂))) -
      ((l : ℝ) * (l + 1) / ρ ^ 2 - 2 / ρ + κ ^ 2) * (ρ ^ (l + 1) * E * W) =
    ρ ^ l * E * (ρ * W₂ + (2 * l + 2 - 2 * κ * ρ) * W₁ + (2 - 2 * κ * (l + 1)) * W)
```

lemma recursion_of_radial_ode [source]

***The coefficient recursion.** If the ansatz solves the radial equation with energy `-κ²`, the coefficients satisfy `(k + 1)(k + 2l + 2) c_{k+1} = (2κ(k + l + 1) - 2) c_k`.*

```
lemma recursion_of_radial_ode (hc : EntireCoeff c)
    (hode : ∀ ρ : ℝ, 0 < ρ → deriv (deriv (radialSeries κ l c)) ρ =
      ((l : ℝ) * (l + 1) / ρ ^ 2 - 2 / ρ + κ ^ 2) * radialSeries κ l c ρ) (k : ℕ) :
    ((k : ℝ) + 1) * (k + 2 * l + 2) * c (k + 1) = (2 * κ * (k + l + 1) - 2) * c k
```

abbrev RadialRec [source]

The coefficient recursion `(k + 1)(k + 2l + 2) c_{k+1} = (2κ(k + l + 1) - 2) c_k`.

```
abbrev RadialRec (κ : ℝ) (l : ℕ) (c : ℕ → ℝ) : Prop
```

lemma coeff_of_terminating [source]

If `κ (N + l + 1) = 1`, the recursion terminates after `c_N` and reproduces the coefficients of `L_N^(2l+1)(2κρ)`.

```
lemma coeff_of_terminating (hrec : RadialRec κ l c) {N : ℕ} (hN : κ * (N + l + 1) = 1) :
    (∀ k, k ≤ N → c k = c 0 / laguerreCoeff N (2 * l + 1) 0 * laguerreCoeff N (2 * l + 1) k *
      (2 * κ) ^ k) ∧ ∀ k, N < k → c k = 0
```

lemma radialSeries_eq_of_terminating [source]

In the terminating case the ansatz is a multiple of the hydrogen radial function.

```
lemma radialSeries_eq_of_terminating (hrec : RadialRec κ l c) {N : ℕ}
    (hN : κ * (N + l + 1) = 1) (ρ : ℝ) :
    radialSeries κ l c ρ =
      c 0 / laguerreCoeff N (2 * l + 1) 0 * radialU (N + l + 1) l ρ
```

lemma tendsto_pow_mul_exp_neg_mul [source]

```
lemma tendsto_pow_mul_exp_neg_mul (hκ : 0 < κ) (k : ℕ) :
    Tendsto (fun ρ : ℝ => ρ ^ k * Real.exp (-(κ * ρ))) atTop (𝓝 0)
```

lemma coeff_ne_zero [source]

```
lemma coeff_ne_zero (hrec : RadialRec κ l c) (hnt : ∀ N : ℕ, κ * (N + l + 1) ≠ 1)
    (h0 : c 0 ≠ 0) :
    ∀ k, c k ≠ 0
```

lemma not_integrableOn_of_nonterminating [source]

***Non-terminating series grow like `e^(3κρ/2)`.** Then `u = ρ^(l+1) e^(-κρ) Σ c_k ρ^k` is not square integrable.*

```
lemma not_integrableOn_of_nonterminating (hκ : 0 < κ) (hc : EntireCoeff c)
    (hrec : RadialRec κ l c) (hnt : ∀ N : ℕ, κ * (N + l + 1) ≠ 1) (h0 : c 0 ≠ 0) :
    ¬IntegrableOn (fun ρ => radialSeries κ l c ρ ^ 2) (Ioi 0)
```

lemma radial_quantization [source]

***Quantisation of the hydrogen levels.** Let `u(ρ) = ρ^(l+1) e^(-κρ) Σ_k c_k ρ^k` with an everywhere convergent power series, `c_0 ≠ 0` and `κ > 0`, solve the radial equation `u'' = (l(l+1)/ρ² - 2/ρ + κ²) u` on `(0, ∞)`, i.e. be an eigenfunction with energy `-κ²` in Rydberg units. If `u` is square integrable, then `κ = 1/ν` for a natural number `ν > l`, so the energy is the Bohr level `-1/ν²`, and `u` is a multiple of the radial function `u_{νl}`.*

```
lemma radial_quantization (hκ : 0 < κ) (hc : EntireCoeff c) (h0 : c 0 ≠ 0)
    (hode : ∀ ρ : ℝ, 0 < ρ → deriv (deriv (radialSeries κ l c)) ρ =
      ((l : ℝ) * (l + 1) / ρ ^ 2 - 2 / ρ + κ ^ 2) * radialSeries κ l c ρ)
    (hint : IntegrableOn (fun ρ => radialSeries κ l c ρ ^ 2) (Ioi 0)) :
    ∃ ν : ℕ, l < ν ∧ κ = 1 / (ν : ℝ) ∧
      ∃ C : ℝ, ∀ ρ, radialSeries κ l c ρ = C * radialU ν l ρ
```

2.591 PhyslibAlpha.Relativity.Fermions.Dirac.MinimalDimension

PhyslibAlpha/Relativity/Fermions/Dirac/MinimalDimension.lean on GitHub.

def IsCliffordFamily [source]

A family of `n × n` matrices with `γ^μ γ^ν + γ^ν γ^μ = 2 η^{μν} 1`.

```
def IsCliffordFamily {n : ℕ} (γ : Fin 1 ⊕ Fin 3 → Matrix (Fin n) (Fin n) ℂ) : Prop
```

lemma mul_self [source]

```
lemma mul_self (μ : Fin 1 ⊕ Fin 3) : γ μ * γ μ = (minkowskiMatrix μ μ : ℂ) • 1
```

lemma mul_comm_of_ne [source]

```
lemma mul_comm_of_ne {μ ν : Fin 1 ⊕ Fin 3} (h : μ ≠ ν) : γ ν * γ μ = -(γ μ * γ ν)
```

lemma det_ne_zero [source]

Each member squares to `± 1`, so it is invertible.

```
lemma det_ne_zero (μ : Fin 1 ⊕ Fin 3) : (γ μ).det ≠ 0
```

lemma even [source]

A Clifford family has even size.

```
lemma even : Even n
```

def involutions [source]

γ^0, i γ^1, i γ^2, i γ^3: four involutions that anticommute.

```
noncomputable def involutions (γ : Fin 1 ⊕ Fin 3 → Matrix (Fin n) (Fin n) ℂ)
    (μ : Fin 1 ⊕ Fin 3) : Matrix (Fin n) (Fin n) ℂ
```

lemma involutions_anticomm [source]

```
lemma involutions_anticomm (μ ν : Fin 1 ⊕ Fin 3) :
    involutions γ μ * involutions γ ν + involutions γ ν * involutions γ μ =
      (Sum.elim (fun _ => (1 : ℂ)) (fun _ => I) μ * Sum.elim (fun _ => (1 : ℂ)) (fun _ => I) ν)
     •
       (γ μ * γ ν + γ ν * γ μ)
```

lemma ne_two [source]

***There are no 2 × 2 Clifford families**: they would give four anticommuting 2 × 2 involutions.*

```
lemma ne_two : n ≠ 2
```

lemma four_le [source]

A nonempty Clifford family has size at least four.

```
lemma four_le (hn : n ≠ 0) : 4 ≤ n
```

lemma isCliffordFamily_gamma [source]

The Dirac gamma matrices, written as 4 × 4 matrices, are a Clifford family.

```
lemma isCliffordFamily_gamma : IsCliffordFamily fun μ => endEquivMatrix (gamma μ)
```

lemma isLeast_size_isCliffordFamily [source]

***The gamma matrices need four dimensions (Dirac, 1928).** The smallest size of a nonempty Clifford family is four.*

```
lemma isLeast_size_isCliffordFamily :
    IsLeast {n : ℕ | n ≠ 0 ∧ ∃ γ : Fin 1 ⊕ Fin 3 → Matrix (Fin n) (Fin n) ℂ,
      IsCliffordFamily γ} 4
```

2.592 PhyslibAlpha.Relativity.Fermions.Weyl.BoostWeight

PhyslibAlpha/Relativity/Fermions/Weyl/BoostWeight.lean on GitHub.

def weylWeight [source]

The boost weight of a Weyl-spinor index. Along the z-axis the SL(2,ℂ) boost is the diagonal matrix diag (t, t⁻¹), so the first spinor component carries weight +1 and the second weight -1; a Weyl spinor is a half-vector.

```
def weylWeight (k : Fin 2) : ℤ
```

lemma rightHandedWeyl_rep_boostAxis_two_basis [source]

The right-handed Weyl basis diagonalises the z-boost, with weights ±1.

```
lemma rightHandedWeyl_rep_boostAxis_two_basis (t : ℝ) (ht : t ≠ 0) (k : Fin 2) :
    Fermion.RightHandedWeyl.rep (SL2C.boostAxis 2 t ht) (Fermion.RightHandedWeyl.basis k)
      = ((t : ℝ) : ℂ) ^ (weylWeight k) • Fermion.RightHandedWeyl.basis k
```

lemma leftHandedWeyl_rep_boostAxis_two_basis [source]

The left-handed Weyl basis diagonalises the z-boost, with weights ±1.

```
lemma leftHandedWeyl_rep_boostAxis_two_basis (t : ℝ) (ht : t ≠ 0) (k : Fin 2) :
    Fermion.LeftHandedWeyl.rep (SL2C.boostAxis 2 t ht) (Fermion.LeftHandedWeyl.basis k)
      = ((t : ℝ) : ℂ) ^ (weylWeight k) • Fermion.LeftHandedWeyl.basis k
```

2.593 PhyslibAlpha.Relativity.Fermions.Weyl.Metric

PhyslibAlpha/Relativity/Fermions/Weyl/Metric.lean on GitHub.

lemma metricVal_rep [source]

The metric ε^{aa} is invariant under the action of SL(2,ℂ).

```
lemma LeftHandedWeyl.metricVal_rep (M : SL(2,ℂ)) :
    TensorProduct.map (LeftHandedWeyl.rep M) (LeftHandedWeyl.rep M) LeftHandedWeyl.metricVal =
      LeftHandedWeyl.metricVal
```

def epsilon [source]

The antisymmetric symplectic form ε = !![0, 1; -1, 0], the Weyl metric Fermion.metricRaw, as an element of SL(2,ℂ).

```
def epsilon : SL(2,ℂ)
```

lemma epsilon_coe [source]

The matrix underlying epsilon.

```
lemma epsilon_coe : (epsilon : Matrix (Fin 2) (Fin 2) ℂ) = !![0, 1; -1, 0]
```

lemma epsilon_coe_metricRaw [source]

The matrix underlying epsilon is the Weyl metric Fermion.metricRaw.

```
lemma epsilon_coe_metricRaw : (epsilon : Matrix (Fin 2) (Fin 2) ℂ) = Fermion.metricRaw
```

lemma star_epsilon_apply [source]

The entries of ε are real.

```
lemma star_epsilon_apply (l k : Fin 2) : star (epsilon.1 l k) = epsilon.1 l k
```

lemma epsilon_mul_inv [source]

$\varepsilon\ g^{-1} = g^T\ \varepsilon$: `Fermion.metricRaw_comm` at g^{-1}.

```
lemma epsilon_mul_inv (g : SL(2,ℂ)) : epsilon.1 * g.1⁻¹ = g.1^T * epsilon.1
```

lemma transpose_mul_epsilon_mul [source]

ε is the invariant symplectic form of `SL(2,ℂ)`: $g^T\ \varepsilon\ g = \varepsilon$.

```
lemma transpose_mul_epsilon_mul (g : SL(2,ℂ)) : g.1^T * epsilon.1 * g.1 = epsilon.1
```

lemma sum_epsilon_mul_inv_transpose [source]

Single-index form: a factor of $(g^{-1})^T$ moved across ε becomes a factor of g on the other slot.

```
lemma sum_epsilon_mul_inv_transpose (g : SL(2,ℂ)) (l a : Fin 2) :
    ∑ k : Fin 2, epsilon.1 l k * (g.1⁻¹)^T a k = ∑ b : Fin 2, g.1 b l * epsilon.1 b a
```

lemma sum_epsilon_mul_inv_conjTranspose [source]

The conjugate single-index form: a factor of $(g^{-1})^H$ moved across ε becomes a factor of the conjugate of g on the other slot.

```
lemma sum_epsilon_mul_inv_conjTranspose (g : SL(2,ℂ)) (l a : Fin 2) :
    ∑ k : Fin 2, epsilon.1 l k * (g.1⁻¹)^H a k
      = ∑ b : Fin 2, star (g.1 b l) * epsilon.1 b a
```

2.594 PhyslibAlpha.Relativity.IsLorentzDeriv

PhyslibAlpha/Relativity/IsLorentzDeriv.lean on GitHub.

lemma trivial_dual_apply [source]

The dual of the trivial representation acts trivially.

```
lemma _root_.Representation.trivial_dual_apply {k G V : Type*} [CommSemiring k]
    [Group G] [AddCommMonoid V] [Module k V] (g : G) (φ : Module.Dual k V) :
    (Representation.trivial k G V).dual g φ = φ
```

def iteratedD [source]

The iterated operator $D_s = D_{\nu_1} \cdots D_{\nu_n}$ of a pairwise-commuting family of endomorphisms along a multiset s of indices. Commutativity is what makes the operator well-defined on a multiset, i.e. independent of any ordering of s.

```
def iteratedD {ι : Type*} (D : ι → A →ₗ[ℂ] A)
    (hD : ∀ i j, (D i).comp (D j) = (D j).comp (D i)) (s : Multiset ι) : A →ₗ[ℂ] A
```

lemma iteratedD_zero [source]

```
lemma iteratedD_zero {ι : Type*} (D : ι → A →ₗ[ℂ] A)
    (hD : ∀ i j, (D i).comp (D j) = (D j).comp (D i)) :
    iteratedD D hD (0 : Multiset ι) = LinearMap.id
```

lemma iteratedD_cons [source]

```
lemma iteratedD_cons {ι : Type*} (D : ι → A →ₗ[ℂ] A)
    (hD : ∀ i j, (D i).comp (D j) = (D j).comp (D i)) (κ : ι) (s : Multiset ι) :
    iteratedD D hD (κ ::ₘ s) = (D κ).comp (iteratedD D hD s)
```

lemma iteratedD_singleton [source]

The iterated operator of a singleton is the operator itself.

```
lemma iteratedD_singleton {ι : Type*} (D : ι → A →ₗ[ℂ] A)
    (hD : ∀ i j, (D i).comp (D j) = (D j).comp (D i)) (κ : ι) :
    iteratedD D hD {κ} = D κ
```

lemma iteratedD_add [source]

The iterated operator is additive in the multiset of directions: applying along `s + t` is applying along `t` and then along `s`.

```
lemma iteratedD_add {ι : Type*} (D : ι → A →ₗ[ℂ] A)
    (hD : ∀ i j, (D i).comp (D j) = (D j).comp (D i)) (s t : Multiset ι) :
    iteratedD D hD (s + t) = (iteratedD D hD s).comp (iteratedD D hD t)
```

lemma iteratedD_cons' [source]

The companion of `iteratedD_cons`, peeling the new operator on the inside: for a commuting family the extra operator may equally be applied first.

```
lemma iteratedD_cons' {ι : Type*} (D : ι → A →ₗ[ℂ] A)
    (hD : ∀ i j, (D i).comp (D j) = (D j).comp (D i)) (κ : ι) (s : Multiset ι) :
    iteratedD D hD (κ ::ₘ s) = (iteratedD D hD s).comp (D κ)
```

lemma iteratedD_mul [source]

```
lemma iteratedD_mul (D : (Fin 1 ⊕ Fin 3) → A →ₗ[ℂ] A)
    (D_comm : ∀ μ ν, (D μ).comp (D ν) = (D ν).comp (D μ))
    (D_mul : ∀ (μ : Fin 1 ⊕ Fin 3) (b₁ b₂ : A),
      D μ (b₁ * b₂) = D μ b₁ * b₂ + b₁ * D μ b₂)
    (s : Multiset (Fin 1 ⊕ Fin 3)) (b₁ b₂ : A) :
    Lorentz.iteratedD D D_comm s (b₁ * b₂) =
      (s.antidiagonal.map fun p =>
        Lorentz.iteratedD D D_comm p.1 b₁ * Lorentz.iteratedD D D_comm p.2 b₂).sum
```

class IsLorentzDeriv [source]

*A family of operators indexed by the spacetime directions is a **Lorentz derivative** when the representation of `SL(2,ℂ)` intertwines it through the columns of the Lorentz*

matrix. The class needs only the module structure, so it applies uniformly to any representation space.

```
class IsLorentzDeriv {M : Type} [AddCommMonoid M] [Module ℂ M]
    (rep : Representation ℂ SL(2,ℂ) M) (D : (Fin 1 ⊕ Fin 3) → M →ₗ[ℂ] M) where
  rep_deriv {Λ μ x} : rep Λ (D μ x) =
    ∑ a, (((SL2C.toLorentzGroup Λ).1 a μ : ℝ) : ℂ) • D a (rep Λ x)
```

def IsLorentzDerivTransforms [source]

*A family of derivative symbols `F : s ↦ [∂_s ψ^φ]`, indexed by the dual of a value space `V` carrying a representation of `SL(2,ℂ)`, **transforms as the derivative symbols of a Lorentz-covariant field**: each ordered symbol mixes into all tuples of directions by the per-slot columns of the Lorentz matrix, while the value index transforms by the contragredient action `rep.dual` on the dual of `V`. This is the general form of the Lorentz law `GaugeAlgebraRealization.lorentz_apply`, for a field valued in an arbitrary Lorentz representation — the trivial representation for scalars, the Weyl representations for fermions, and their conjugates for the barred fields. At `n = 0` it reduces to the homogeneous law `Λ • F₀^φ = F₀^{Λ^{-T} φ}`.*

```
def IsLorentzDerivTransforms {k V : Type*} [CommRing k] [AddCommGroup V] [Module k V]
    [Module k A]
    (repLorentz : Representation ℂ SL(2,ℂ) A) (rep : Representation k SL(2,ℂ) V)
    (F : Multiset (Fin 1 ⊕ Fin 3) → Module.Dual k V →ₗ[k] A) : Prop
```

def IsLorentzCovDerivTransforms [source]

*A family of *covariant*-derivative symbols, indexed by ordered tuples of directions (covariant derivatives do not commute) and by the dual of a Lorentz representation `V`, **transforms as the covariant derivatives of a Lorentz-covariant field**: each derivative slot mixes by the columns of the Lorentz matrix, while the value index transforms by the contragredient action `rep.dual` on the dual of `V` — the ordered-tuple analogue of `IsLorentzDerivTransforms`.*

```
def IsLorentzCovDerivTransforms {k V : Type*} [CommRing k] [AddCommGroup V]
    [Module k V] [Module k A] (repLorentz : Representation ℂ SL(2,ℂ) A)
    (rep : Representation k SL(2,ℂ) V)
    (F : {n : ℕ} → (Fin n → (Fin 1 ⊕ Fin 3)) → Module.Dual k V →ₗ[k] A) : Prop
```

lemma neg_one_apply [source]

***At the centre the derivative slots do not mix.** The element `-1` of `SL(2,ℂ)` covers the identity Lorentz transformation, so the sum over tuples collapses to the single term `p = l` and only the value index moves.*

```
lemma IsLorentzCovDerivTransforms.neg_one_apply {k V : Type*} [CommRing k] [AddCommGroup V]
    [Module k V] [Module k A] {repLorentz : Representation ℂ SL(2,ℂ) A}
    {rep : Representation k SL(2,ℂ) V}
    {F : {n : ℕ} → (Fin n → (Fin 1 ⊕ Fin 3)) → Module.Dual k V →ₗ[k] A}
    (hF : IsLorentzCovDerivTransforms repLorentz rep F)
    {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (φ : Module.Dual k V) :
    repLorentz (-1) (F l φ) = F l (rep.dual (-1) φ)
```

lemma algebraMap_real_complex [source]

The scalar action of a real parameter, in the form the weight condition presents it.

```
private lemma algebraMap_real_complex (t : ℝ) : (algebraMap ℝ ℂ) t = ((t : ℝ) : ℂ)
```

lemma rep_iteratedD_ofFn [source]

***The Lorentz transformation of iterated derivatives**: for a Lorentz derivative the ordered derivative symbol `D_{l 0} ⋯ D_{l (n-1)} x` mixes into all tuples of directions, with one Lorentz matrix factor per slot.*

```
lemma rep_iteratedD_ofFn [IsLorentzDeriv rep D]
    (D_comm : ∀ μ ν, (D μ).comp (D ν) = (D ν).comp (D μ))
    (Λ : SL(2,ℂ)) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (x : A) :
    rep Λ (iteratedD D D_comm (List.ofFn l) x) =
      ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
        (∏ i, (((SL2C.toLorentzGroup Λ).1 (p i) (l i) : ℝ) : ℂ)) •
          iteratedD D D_comm (List.ofFn p) (rep Λ x)
```

2.595 PhyslibAlpha.Relativity.LightConeDeriv

PhyslibAlpha/Relativity/LightConeDeriv.lean on GitHub.

def lightConeCoeff [source]

The four light-cone directions along the `i`-th axis, written as coefficient vectors on the coordinate directions: `D₀ - Dᵢ`, `D₀ + Dᵢ`, and the two transverse directions.

```
def lightConeCoeff (i : Fin 3) (κ : Fin 4) (μ : Fin 1 ⊕ Fin 3) : ℂ
```

def lightConeWeight [source]

The boost weight carried by each light-cone direction: `+2` for `D₀ - Dᵢ`, `-2` for `D₀ + Dᵢ`, and `0` for the two transverse directions.

```
def lightConeWeight (κ : Fin 4) : ℤ
```

lemma sum_boostAxis_lightConeCoeff [source]

***The light-cone directions are eigenvectors of the boost.** Along the `i`-th axis `D₀ - Dᵢ` is scaled by `t²`, `D₀ + Dᵢ` by `t⁻²`, and the two transverse directions are fixed.*

```
lemma sum_boostAxis_lightConeCoeff (i : Fin 3) (κ : Fin 4) (ν : Fin 1 ⊕ Fin 3)
    {t : ℝ} (ht : t ≠ 0) :
    ∑ μ : Fin 1 ⊕ Fin 3,
        (((SL2C.toLorentzGroup (SL2C.boostAxis i t ht)).1 ν μ : ℝ) : ℂ) * lightConeCoeff i κ μ
      = ((t : ℝ) : ℂ) ^ (lightConeWeight κ) * lightConeCoeff i κ ν
```

def lightConeCoeffInv [source]

The coordinate directions written back in the light-cone basis: `D₀` and `Dᵢ` are the half-sum and half-difference of `D₀ ∓ Dᵢ`, and the transverse directions are themselves.

```
noncomputable def lightConeCoeffInv (i : Fin 3) (μ : Fin 1 ⊕ Fin 3) (κ : Fin 4) : ℂ
```

lemma lightConeCoeffInv_two_eq_zero [source]

The inverse coefficient toward the first transverse direction vanishes off it.

```
lemma lightConeCoeffInv_two_eq_zero (i : Fin 3) {μ : Fin 1 ⊕ Fin 3}
    (hμ : μ ≠ Sum.inr (i + 1)) : lightConeCoeffInv i μ 2 = 0
```

lemma lightConeCoeffInv_three_eq_zero [source]

The inverse coefficient toward the second transverse direction vanishes off it.

```
lemma lightConeCoeffInv_three_eq_zero (i : Fin 3) {μ : Fin 1 ⊕ Fin 3}
    (hμ : μ ≠ Sum.inr (i + 2)) : lightConeCoeffInv i μ 3 = 0
```

lemma sum_lightConeCoeffInv_mul [source]

The light-cone basis is a basis: the two coefficient matrices are inverse.

```
lemma sum_lightConeCoeffInv_mul (i : Fin 3) (μ ν : Fin 1 ⊕ Fin 3) :
    ∑ κ : Fin 4, lightConeCoeffInv i μ κ * lightConeCoeff i κ ν = if μ = ν then 1 else 0
```

lemma sum_prod_lightConeCoeff [source]

The scalar behind `lightConeDeriv_mem`: the boost acts on a light-cone multi-index slot by slot, so the product of the per-slot eigenvalues factors out.

```
lemma sum_prod_lightConeCoeff (i : Fin 3) {n : ℕ} (c : Fin n → Fin 4)
    (a : Fin n → Fin 1 ⊕ Fin 3) {t : ℝ} (ht : t ≠ 0) :
    ∑ d : Fin n → Fin 1 ⊕ Fin 3, (∏ j, lightConeCoeff i (c j) (d j)) *
        (∏ j, (((SL2C.toLorentzGroup (SL2C.boostAxis i t ht)).1 (a j) (d j) : ℝ) : ℂ))
      = ((t : ℝ) : ℂ) ^ (∑ j, lightConeWeight (c j)) * ∏ j, lightConeCoeff i (c j) (a j)
```

lemma sum_prod_lightConeCoeffInv [source]

The two coefficient matrices are inverse slot by slot, hence inverse on multi-indices; this is `sum_prod_lightConeCoeff` with the two factors the other way round.

```
lemma sum_prod_lightConeCoeffInv (i : Fin 3) {n : ℕ} (d e : Fin n → Fin 1 ⊕ Fin 3) :
    ∑ c : Fin n → Fin 4, (∏ j, lightConeCoeffInv i (d j) (c j)) *
        (∏ j, lightConeCoeff i (c j) (e j)) = if d = e then 1 else 0
```

2.596 PhyslibAlpha.Relativity.LorentzGroup.Basic

PhyslibAlpha/Relativity/LorentzGroup/Basic.lean on GitHub.

lemma sum_minkowskiMatrixZ_mul [source]

The defining relation $\Lambda\,\eta\,\Lambda^T = \eta$ over $\mathbb{C}$, read on the entry `(a, b)` and with the integer Minkowski matrix in place of `minkowskiMatrix`. This is the form the invariant classifications contract against, one metric pairing per pair of slots.

```
lemma sum_minkowskiMatrixZ_mul (Λ : LorentzGroup d) (a b : Fin 1 ⊕ Fin d) :
    ∑ x : Fin 1 ⊕ Fin d, ∑ y : Fin 1 ⊕ Fin d, ((minkowskiMatrixZ x y : ℤ) : ℂ)
        * (((Λ.1 a x : ℝ) : ℂ) * ((Λ.1 b y : ℝ) : ℂ))
      = ((minkowskiMatrixZ a b : ℤ) : ℂ)
```

2.597 PhyslibAlpha.Relativity.LorentzGroup.Boosts.Axis

PhyslibAlpha/Relativity/LorentzGroup/Boosts/Axis.lean on GitHub.

lemma star_boostAxis_apply [source]

Hermiticity read on the entries: conjugating an entry of an axis boost transposes it.

```
lemma star_boostAxis_apply (i : Fin 3) (t : ℝ) (ht : t ≠ 0) (β α : Fin 2) :
    star ((boostAxis i t ht).1 β α) = (boostAxis i t ht).1 α β
```

2.598 PhyslibAlpha.Relativity.LorentzGroup.FermionicParity

PhyslibAlpha/Relativity/LorentzGroup/FermionicParity.lean on GitHub.

def fermionicParity [source]

Fermionic parity: the nontrivial element `-1` of the kernel of the two-to-one homomorphism `SL(2, ℂ) → LorentzGroup 3`, that is the rotation by `2π`. It acts trivially on tensors and by `-1` on spinors.

```
def fermionicParity : SL(2, ℂ)
```

lemma toSelfAdjointMap_fermionicParity [source]

Fermionic parity acts trivially on self-adjoint matrices: conjugation by `-1` is the identity.

```
lemma toSelfAdjointMap_fermionicParity :
    Lorentz.SL2C.toSelfAdjointMap fermionicParity = LinearMap.id
```

lemma toLorentzGroup_fermionicParity [source]

Fermionic parity projects to the identity Lorentz transformation: it is invisible on tensors.

```
lemma toLorentzGroup_fermionicParity :
    Lorentz.SL2C.toLorentzGroup fermionicParity = 1
```

lemma fermionicParity_sq [source]

```
@[simp]
lemma fermionicParity_sq : fermionicParity ^ 2 = 1
```

lemma fermionicParity_ne_one [source]

Fermionic parity is not the identity of `SL(2, ℂ)`: the covering is genuinely two-to-one.

```
lemma fermionicParity_ne_one : fermionicParity ≠ 1
```

lemma complexCoVector_SL2CRep_fermionicParity [source]

Fermionic parity acts trivially on complex covariant Lorentz vectors, since it projects to the identity Lorentz transformation.

```
lemma complexCoVector_SL2CRep_fermionicParity :
    Lorentz.ComplexCoVector.SL2CRep fermionicParity = LinearMap.id
```

lemma rightHandedWeyl_rep_fermionicParity [source]

Fermionic parity acts by -1 on right-handed Weyl spinors.

```
lemma rightHandedWeyl_rep_fermionicParity :
    Fermion.RightHandedWeyl.rep fermionicParity = -LinearMap.id
```

2.599 PhyslibAlpha.Relativity.LorentzGroup.Invariants.AdjointClosed

PhyslibAlpha/Relativity/LorentzGroup/Invariants/AdjointClosed.lean on GitHub.

lemma toMatrix_rep_upL [source]

The matrix of g on the left-handed Weyl colour is g.

```
lemma toMatrix_rep_upL (g : SL(2,ℂ)) :
    LinearMap.toMatrix (complexLorentzTensor.basis .upL) (complexLorentzTensor.basis .upL)
      (complexLorentzTensor.rep .upL g) = g.1
```

lemma toMatrix_rep_downL [source]

The matrix of g on the dual left-handed Weyl colour is $(g^{-1})^T$.

```
lemma toMatrix_rep_downL (g : SL(2,ℂ)) :
    LinearMap.toMatrix (complexLorentzTensor.basis .downL) (complexLorentzTensor.basis .downL)
      (complexLorentzTensor.rep .downL g) = (g.1⁻¹)ᵀ
```

lemma toMatrix_rep_upR [source]

The matrix of g on the right-handed Weyl colour is the entrywise conjugate of g.

```
lemma toMatrix_rep_upR (g : SL(2,ℂ)) :
    LinearMap.toMatrix (complexLorentzTensor.basis .upR) (complexLorentzTensor.basis .upR)
      (complexLorentzTensor.rep .upR g) = g.1.map star
```

lemma toMatrix_rep_downR [source]

The matrix of g on the dual right-handed Weyl colour is $(g^{-1})^H$.

```
lemma toMatrix_rep_downR (g : SL(2,ℂ)) :
    LinearMap.toMatrix (complexLorentzTensor.basis .downR) (complexLorentzTensor.basis .downR)
      (complexLorentzTensor.rep .downR g) = (g.1⁻¹)ᴴ
```

lemma toMatrix_rep_up_apply [source]

The matrix of g on the contravariant vector colour is the Lorentz matrix of g.

```
lemma toMatrix_rep_up_apply (g : SL(2,ℂ)) (μ ν : Fin 1 ⊕ Fin 3) :
    LinearMap.toMatrix (complexLorentzTensor.basis .up) (complexLorentzTensor.basis .up)
      (complexLorentzTensor.rep .up g) μ ν
      = (((SL2C.toLorentzGroup g).1 μ ν : ℝ) : ℂ)
```

lemma toMatrix_rep_down_apply [source]

The matrix of `g` on the covariant vector colour is the transpose of the inverse of the Lorentz matrix of `g`.

```
lemma toMatrix_rep_down_apply (g : SL(2,ℂ)) (μ ν : Fin 1 ⊕ Fin 3) :
    LinearMap.toMatrix (complexLorentzTensor.basis .down) (complexLorentzTensor.basis .down)
      (complexLorentzTensor.rep .down g) μ ν
      = (((SL2C.toLorentzGroup g)⁻¹.1 ν μ : ℝ) : ℂ)
```

lemma inv_dagger [source]

The inverse of `g†` is the dagger of the inverse of `g`.

```
lemma inv_dagger (g : SL(2,ℂ)) : (dagger g)⁻¹ = dagger g⁻¹
```

def IsDaggerCompatible [source]

A colour is dagger compatible when, in its basis, the matrix of `g†` is the conjugate transpose of the matrix of `g`.

```
def IsDaggerCompatible (k : complexLorentzTensor.Color) : Prop
```

lemma isDaggerCompatible_of_weyl [source]

The four Weyl colours are dagger compatible.

```
lemma isDaggerCompatible_of_weyl {k : complexLorentzTensor.Color}
    (hk : k = .upL ∨ k = .downL ∨ k = .upR ∨ k = .downR) : IsDaggerCompatible k
```

lemma isDaggerCompatible_up [source]

The contravariant vector colour is dagger compatible: the Lorentz matrix of `g†` is the transpose of that of `g`, and it is real.

```
lemma isDaggerCompatible_up : IsDaggerCompatible .up
```

lemma isDaggerCompatible_down [source]

The covariant vector colour is dagger compatible: the inverse Lorentz matrix of `g†` is the transpose of that of `g`, and it is real.

```
lemma isDaggerCompatible_down : IsDaggerCompatible .down
```

lemma isDaggerCompatible [source]

Every colour of complex Lorentz tensors is dagger compatible.

```
lemma isDaggerCompatible (k : complexLorentzTensor.Color) : IsDaggerCompatible k
```

lemma isAdjointClosed_of_isDaggerCompatible [source]

Colours that are all dagger compatible are closed under adjoints, with `g' = g†`.

```
lemma isAdjointClosed_of_isDaggerCompatible {n : ℕ} {c : Fin n → complexLorentzTensor.Color}
    (hc : ∀ i, IsDaggerCompatible (c i)) : complexLorentzTensor.IsAdjointClosed c
```

lemma isAdjointClosed [source]

Every list of colours of complex Lorentz tensors is closed under adjoints, with `g' = g†`.

```
lemma complexLorentzTensor.isAdjointClosed {n : ℕ} (c : Fin n → complexLorentzTensor.Color) :
    complexLorentzTensor.IsAdjointClosed c
```

2.600 PhyslibAlpha.Relativity.LorentzGroup.Invariants.Basic

PhyslibAlpha/Relativity/LorentzGroup/Invariants/Basic.lean on GitHub.

def actMat [source]

The action on coefficient functions of a matrix moving the components: `(actMat M c) a = ∑_d c_d M_{a d}`, with `a` free and `d` summed.

```
def actMat (M : ι → ι → ℂ) (c : ι → ℂ) (a : ι) : ℂ
```

lemma sum_mul_actMat [source]

A covector `P` that the transposed matrix reproduces up to a scalar `k` reads off a component of the coefficients that `actMat M` scales by `k`.

```
lemma sum_mul_actMat (M : ι → ι → ℂ) (P c : ι → ℂ) (k : ℂ)
    (hP : ∀ d, ∑ a, P a * M a d = k * P d) :
    ∑ a, P a * actMat M c a = k * ∑ a, P a * c a
```

lemma sum_mul_eq_zero_of_actMat_eq [source]

A coefficient function fixed by `actMat M` has no component along a covector that the transposed matrix scales by an eigenvalue other than `1`.

```
lemma sum_mul_eq_zero_of_actMat_eq (M : ι → ι → ℂ) {P c : ι → ℂ} (hc : actMat M c = c) {k : ℂ}
    (hP : ∀ d, ∑ a, P a * M a d = k * P d) (hk : k ≠ 1) :
    ∑ a, P a * c a = 0
```

lemma two_zpow_ne_one [source]

The boost with parameter `2` distinguishes every nonzero weight: `2 ^ w ≠ 1` for `w ≠ 0`.

```
lemma two_zpow_ne_one {w : ℤ} (hw : w ≠ 0) : ((2 : ℝ) : ℂ) ^ w ≠ 1
```

def dagger [source]

The conjugate transpose `g†` of an element of `SL(2,ℂ)`, again in `SL(2,ℂ)`.

```
def dagger (g : SL(2,ℂ)) : SL(2,ℂ)
```

lemma toLorentzGroup_dagger [source]

The Lorentz matrix of `g†` is the transpose of that of `g`, so these matrices are closed under transposition.

```
lemma toLorentzGroup_dagger (g : SL(2,ℂ)) :
    (SL2C.toLorentzGroup (dagger g)).1 = (SL2C.toLorentzGroup g).1^T
```

def act [source]

The action of a real `4 × 4` matrix on coefficient tensors, one factor per slot: `(act Λ c) a = ∑_d c_d Λ_{a₀ d₀} ⋯`, with `a` free and `d` summed.

```
def act (Λ : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ)
    (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ) (a : Fin n → Fin 1 ⊕ Fin 3) : ℂ
```

lemma act_eq_actMat [source]

The action on coefficient tensors is that of the matrix of products, one factor per slot.

```
lemma act_eq_actMat (Λ : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ)
    (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ) :
    act Λ c = actMat (fun a d => ∏ i, ((Λ (a i) (d i) : ℝ) : ℂ)) c
```

def IsInvariantCoeff [source]

A coefficient tensor fixed by `act` of the Lorentz matrix of every `g : SL(2,ℂ)`.

```
def IsInvariantCoeff (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ) : Prop
```

def lightConeComponent [source]

A light-cone component of a coefficient tensor along axis `i`: the multi-index `κ` picks one light-cone direction per slot and `c` is contracted against that choice.

```
def lightConeComponent (i : Fin 3) (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ) (κ : Fin n → Fin 4) : ℂ
```

lemma lightConeComponent_act [source]

A light-cone multi-index that `Λ` reproduces up to a scalar `k` has its light-cone component scaled by `k`. The hypothesis is the eigenvector equation for the covector $\prod_s$ `lightConeCoeff i (κ s) (·)` under the transposed action, which is the form the light-cone directions of an axis satisfy for the transformations diagonal in that basis: the boost along the axis, with `k` a power of its parameter, and the half turn about it, with `k` the product of the signs of the slots.

```
lemma lightConeComponent_act (i : Fin 3) (Λ : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ)
    (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ) (κ : Fin n → Fin 4) (k : ℂ)
    (hΛ : ∀ d : Fin n → Fin 1 ⊕ Fin 3,
      ∑ a : Fin n → Fin 1 ⊕ Fin 3, (∏ s, lightConeCoeff i (κ s) (a s))
          * ∏ s, ((Λ (a s) (d s) : ℝ) : ℂ)
        = k * ∏ s, lightConeCoeff i (κ s) (d s)) :
    lightConeComponent i (act Λ c) κ = k * lightConeComponent i c κ
```

lemma toLorentzGroup_boostAxis_symm [source]

The Lorentz matrix of a boost is symmetric.

```
lemma toLorentzGroup_boostAxis_symm (i : Fin 3) {t : ℝ} (ht : t ≠ 0) (a b : Fin 1 ⊕ Fin 3) :
    (SL2C.toLorentzGroup (SL2C.boostAxis i t ht)).1 a b
      = (SL2C.toLorentzGroup (SL2C.boostAxis i t ht)).1 b a
```

lemma lightConeComponent_eq_zero [source]

An invariant coefficient tensor has no light-cone component of nonzero weight: the boost at `t = 2` would rescale such a component by a factor other than `1`.

```
lemma IsInvariantCoeff.lightConeComponent_eq_zero {c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ}
    (hc : IsInvariantCoeff c) (i : Fin 3) {κ : Fin n → Fin 4}
    (hκ : ∑ s, lightConeWeight (κ s) ≠ 0) :
    lightConeComponent i c κ = 0
```

lemma eq_sum_lightConeComponent [source]

A coefficient tensor is recovered from its light-cone components.

```
lemma eq_sum_lightConeComponent (i : Fin 3) (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ)
    (d : Fin n → Fin 1 ⊕ Fin 3) :
    c d = ∑ κ, (∏ s, lightConeCoeffInv i (d s) (κ s)) * lightConeComponent i c κ
```

lemma act_halfTurn [source]

The half turn about the axis `k` multiplies the coefficient at `a` by the product of the signs `halfTurnSign k` of the slots of `a`.

```
lemma act_halfTurn (k : Fin 3) (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ) (a : Fin n → Fin 1 ⊕ Fin 3) :
    act (SL2C.toLorentzGroup (SL2C.halfTurn k)).1 c a
      = ((∏ s, halfTurnSign k (a s) : ℤ) : ℂ) * c a
```

lemma prod_halfTurnSign_eq_one_or [source]

The sign a half turn attaches to a coefficient is `1` or `-1`, being a product of such signs.

```
lemma prod_halfTurnSign_eq_one_or (k : Fin 3) (d : Fin n → Fin 1 ⊕ Fin 3) :
    ∏ s, halfTurnSign k (d s) = 1 ∨ ∏ s, halfTurnSign k (d s) = -1
```

lemma eq_zero_of_prod_halfTurnSign_ne_one [source]

An invariant coefficient tensor vanishes at every index vector that some half turn negates.

```
lemma IsInvariantCoeff.eq_zero_of_prod_halfTurnSign_ne_one {c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ}
    (hc : IsInvariantCoeff c) {k : Fin 3} {d : Fin n → Fin 1 ⊕ Fin 3}
    (hd : ∏ s, halfTurnSign k (d s) ≠ 1) : c d = 0
```

def cycIdx [source]

The relabelling `cycDir`, which fixes time and sends `x → y → z → x`, applied in every slot.

```
def cycIdx (d : Fin n → Fin 1 ⊕ Fin 3) : Fin n → Fin 1 ⊕ Fin 3
```

lemma cycIdx_cycIdx_cycIdx [source]

Cycling the axes three times is the identity.

```
lemma cycIdx_cycIdx_cycIdx (d : Fin n → Fin 1 ⊕ Fin 3) : cycIdx (cycIdx (cycIdx d)) = d
```

lemma act_rotationCycle [source]

The cyclic rotation permutes coefficients: the new coefficient at `a` is the old one at `a` cycled back.

```
lemma act_rotationCycle (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ) (a : Fin n → Fin 1 ⊕ Fin 3) :
    act (SL2C.toLorentzGroup SL2C.rotationCycle).1 c a = c (cycIdx (cycIdx a))
```

lemma apply_cycIdx [source]

An invariant coefficient tensor is constant on the orbits of the cyclic rotation.

```
lemma IsInvariantCoeff.apply_cycIdx {c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ} (hc : IsInvariantCoeff c)
    (d : Fin n → Fin 1 ⊕ Fin 3) : c (cycIdx d) = c d
```

2.601 PhyslibAlpha.Relativity.LorentzGroup.Invariants.Centre

PhyslibAlpha/Relativity/LorentzGroup/Invariants/Centre.lean on GitHub.

def centreEigenspace [source]

The subspace on which the centre of `SL(2,ℂ)` acts by the scalar `ε`: an integer spin sits at `ε = 1` and a half-integer spin at `ε = -1`.

```
def centreEigenspace (repLorentz : Representation ℂ SL(2,ℂ) V) (ε : ℂ) : Submodule ℂ V
```

lemma mem_centreEigenspace [source]

Membership of `centreEigenspace` unfolded: the centre scales the vector by `ε`.

```
lemma mem_centreEigenspace {repLorentz : Representation ℂ SL(2,ℂ) V} {ε : ℂ} {x : V} :
    x ∈ centreEigenspace repLorentz ε ↔ repLorentz (-1) x = ε • x
```

lemma dual_neg_one_eq_smul_id [source]

Dualising a representation preserves the sign at the centre: `-1` is its own inverse, so the contragredient action of the centre is the transpose of a scalar.

```
lemma dual_neg_one_eq_smul_id {rep : Representation ℂ SL(2,ℂ) V} {ε : ℂ}
    (hrep : rep (-1) = ε • LinearMap.id) : rep.dual (-1) = ε • LinearMap.id
```

lemma range_le_centreEigenspace [source]

The span of one symbol family carries the sign of its value space.

```
lemma range_le_centreEigenspace {repLorentz : Representation ℂ SL(2,ℂ) A}
    {rep : Representation ℂ SL(2,ℂ) V}
    {F : {n : ℕ} → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] A}
    (hF : IsLorentzCovDerivTransforms repLorentz rep F) {ε : ℂ}
    (hrep : rep (-1) = ε • LinearMap.id) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (F l) ≤ centreEigenspace repLorentz ε
```

lemma range_le_centreEigenspace_neg_one [source]

The half-integer case of `range_le_centreEigenspace`, in the form the Weyl value spaces state their sign.

```
lemma range_le_centreEigenspace_neg_one {repLorentz : Representation ℂ SL(2,ℂ) A}
    {rep : Representation ℂ SL(2,ℂ) V}
    {F : {n : ℕ} → (Fin n → Fin 1 ⊕ Fin 3) → Module.Dual ℂ V →ₗ[ℂ] A}
    (hF : IsLorentzCovDerivTransforms repLorentz rep F)
    (hrep : rep (-1) = -LinearMap.id) {n : ℕ} (l : Fin n → Fin 1 ⊕ Fin 3) :
    LinearMap.range (F l) ≤ centreEigenspace repLorentz (-1)
```

lemma mul_le_centreEigenspace [source]

The sign of a product is the product of the signs.

```
lemma mul_le_centreEigenspace {repLorentz : Representation ℂ SL(2,ℂ) A}
    (hmul : ∀ (Λ : SL(2,ℂ)) (x y : A), repLorentz Λ (x * y) = repLorentz Λ x * repLorentz Λ y)
    {W₁ W₂ : Submodule ℂ A} {ε₁ ε₂ : ℂ} (h₁ : W₁ ≤ centreEigenspace repLorentz ε₁)
    (h₂ : W₂ ≤ centreEigenspace repLorentz ε₂) :
    W₁ * W₂ ≤ centreEigenspace repLorentz (ε₁ * ε₂)
```

lemma mem_of_invariant_of_mem_sup_centreEigenspace_neg_one [source]

A subspace of half-integer spin carries no Lorentz invariant beyond a Lorentz-stable submodule `S`: an invariant of the join with `S` already lies in `S`. The centre acts on the subspace by `-1 ≠ 1`, so it reduces to `⊥`.

```
lemma mem_of_invariant_of_mem_sup_centreEigenspace_neg_one
    {repLorentz : Representation ℂ SL(2,ℂ) A} {W : Submodule ℂ A}
    (hW : W ≤ centreEigenspace repLorentz (-1)) (S : Submodule ℂ A)
    (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : A} (hx : x ∈ W ⊔ S)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x ∈ S
```

2.602 PhyslibAlpha.Relativity.LorentzGroup.Invariants.IsBiLeftWeyl

PhyslibAlpha/Relativity/LorentzGroup/Invariants/IsBiLeftWeyl.lean on GitHub.

abbrev IsBiLeftWeyl [source]

A family with two left-handed Weyl indices `T^{α₁ α₂}`: an equivariant linear map out of `ℂT[.upL, .upL]`.

```
abbrev IsBiLeftWeyl (B : Type*) [AddCommMonoid B] [Module ℂ B]
    (repLorentz : Representation ℂ SL(2,ℂ) B) (f : ℂT[.upL, .upL] →ₗ[ℂ] B) : Prop
```

abbrev IsBiRightWeyl [source]

A family with two right-handed Weyl indices `T^{α˙₁ α˙₂}`: an equivariant linear map out of `ℂT[.upR, .upR]`.

```
abbrev IsBiRightWeyl (B : Type*) [AddCommMonoid B] [Module ℂ B]
    (repLorentz : Representation ℂ SL(2,ℂ) B) (f : ℂT[.upR, .upR] →ₗ[ℂ] B) : Prop
```

abbrev IsBiDualLeftWeyl [source]

A family with two dual left-handed Weyl indices `T_{α₁ α₂}`: an equivariant linear map out of `ℂT[.downL, .downL]`.

```
abbrev IsBiDualLeftWeyl (B : Type*) [AddCommMonoid B] [Module ℂ B]
    (repLorentz : Representation ℂ SL(2,ℂ) B) (f : ℂT[.downL, .downL] →ₗ[ℂ] B) : Prop
```

abbrev IsBiDualRightWeyl [source]

A family with two dual right-handed Weyl indices `T_{α˙₁ α˙₂}`: an equivariant linear map out of `ℂT[.downR, .downR]`.

```
abbrev IsBiDualRightWeyl (B : Type*) [AddCommMonoid B] [Module ℂ B]
    (repLorentz : Representation ℂ SL(2,ℂ) B) (f : ℂT[.downR, .downR] →ₗ[ℂ] B) : Prop
```

lemma basis_repr_smul_pair [source]

The components of `g • t` for a tensor with two indices: the matrix of `g` in the colour of each index acts on that index.

```
lemma basis_repr_smul_pair (g : SL(2,ℂ)) (t : ℂT[k, k']) (a : basisIdx k)
    (b : basisIdx k') :
    (Tensor.basis ![k, k']).repr (g • t)
        ((piFinTwoEquiv fun j : Fin 2 => basisIdx (![k, k'] j)).symm (a, b))
      = ∑ x, ∑ y, LinearMap.toMatrix (complexLorentzTensor.basis k)
          (complexLorentzTensor.basis k) (complexLorentzTensor.rep k g) a x *
        LinearMap.toMatrix (complexLorentzTensor.basis k') (complexLorentzTensor.basis k')
          (complexLorentzTensor.rep k' g) b y *
        (Tensor.basis ![k, k']).repr t
          ((piFinTwoEquiv fun j : Fin 2 => basisIdx (![k, k'] j)).symm (x, y))
```

lemma basis_repr_pair_eq_neg_swap_of_invariant [source]

The components of an invariant tensor with two Weyl indices of the same colour are antisymmetric. The inverse boost along `z` scales the two diagonal components by `4` and `4⁻¹`, and the inverse half turn about `x` sends the mixed component `(0, 1)` to minus `(1, 0)`; for the dual colours the inverse cancels against the inverse in the matrix of the colour.

```
lemma basis_repr_pair_eq_neg_swap_of_invariant
    (hk : k = .upL ∨ k = .downL ∨ k = .upR ∨ k = .downR) {t : ℂT[k, k]}
    (ht : ∀ g : SL(2,ℂ), g • t = t) (a b : basisIdx k) :
    (Tensor.basis ![k, k]).repr t
        ((piFinTwoEquiv fun j : Fin 2 => basisIdx (![k, k] j)).symm (a, b))
      = - (Tensor.basis ![k, k]).repr t
        ((piFinTwoEquiv fun j : Fin 2 => basisIdx (![k, k] j)).symm (b, a))
```

lemma exists_eq_smul_metricTensor_of_invariant [source]

The invariant tensors with two Weyl indices of the same colour are the multiples of the metric of that colour: both have antisymmetric components, and the `(0, 1)` component of the metric does not vanish.

```
lemma exists_eq_smul_metricTensor_of_invariant
    (hk : k = .upL ∨ k = .downL ∨ k = .upR ∨ k = .downR) (t : ℂT[k, k])
    (ht : ∀ g : SL(2,ℂ), g • t = t) : ∃ a : ℂ, t = a • metricTensor k
```

def invariantReductionToMetricTensor [source]

For a Weyl colour `k` and an equivariant map `f` out of `ℂT[k, k]`, the Lorentz invariants of the range of `f` reduce to the span of the image `f (metricTensor k)` of the metric.

```
noncomputable def invariantReductionToMetricTensor
    (hk : k = .upL ∨ k = .downL ∨ k = .upR ∨ k = .downR)
    (hf : complexLorentzTensor.IsEquivariant ![k, k] repLorentz f) :
    InvariantReductionToSpan (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f)
```

lemma exists_smul_map_metricTensor_add_of_invariant [source]

For a Weyl colour `k` and an equivariant map `f` out of `ℂT[k, k]`, every Lorentz invariant of `LinearMap.range f ⊔ S`, for `S` a Lorentz-stable submodule, is a multiple of the image `f (metricTensor k)` of the metric plus an element of `S`.

```
lemma exists_smul_map_metricTensor_add_of_invariant
    (hk : k = .upL ∨ k = .downL ∨ k = .upR ∨ k = .downR)
    (hf : complexLorentzTensor.IsEquivariant ![k, k] repLorentz f) (S : Submodule ℂ B)
    (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ LinearMap.range f ⊔ S) (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) :
    ∃ a : ℂ, ∃ y ∈ S, x = a • f (metricTensor k) + y
```

def IsBiLeftWeyl.invariantReductionToSpan [source]

The Lorentz invariants of the range of a family with two left-handed Weyl indices reduce to the span of the image of `εL`.

```
noncomputable def IsBiLeftWeyl.invariantReductionToSpan {B : Type*} [AddCommGroup B]
    [Module ℂ B] {repLorentz : Representation ℂ SL(2,ℂ) B} {f : ℂT[.upL, .upL] →ₗ[ℂ] B}
    (hf : IsBiLeftWeyl B repLorentz f) :
    InvariantReductionToSpan (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f)
```

def IsBiRightWeyl.invariantReductionToSpan [source]

The Lorentz invariants of the range of a family with two right-handed Weyl indices reduce to the span of the image of `εR`.

```
noncomputable def IsBiRightWeyl.invariantReductionToSpan {B : Type*} [AddCommGroup B]
    [Module ℂ B] {repLorentz : Representation ℂ SL(2,ℂ) B} {f : ℂT[.upR, .upR] →ₗ[ℂ] B}
    (hf : IsBiRightWeyl B repLorentz f) :
    InvariantReductionToSpan (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f)
```

def IsBiDualLeftWeyl.invariantReductionToSpan [source]

The Lorentz invariants of the range of a family with two dual left-handed Weyl indices reduce to the span of the image of `εL'`.

```
noncomputable def IsBiDualLeftWeyl.invariantReductionToSpan {B : Type*} [AddCommGroup B]
    [Module ℂ B] {repLorentz : Representation ℂ SL(2,ℂ) B} {f : ℂT[.downL, .downL] →ₗ[ℂ] B}
    (hf : IsBiDualLeftWeyl B repLorentz f) :
    InvariantReductionToSpan (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f)
```

def IsBiDualRightWeyl.invariantReductionToSpan [source]

The Lorentz invariants of the range of a family with two dual right-handed Weyl indices reduce to the span of the image of `εR'`.

```
noncomputable def IsBiDualRightWeyl.invariantReductionToSpan {B : Type*} [AddCommGroup B]
    [Module ℂ B] {repLorentz : Representation ℂ SL(2,ℂ) B} {f : ℂT[.downR, .downR] →ₗ[ℂ] B}
    (hf : IsBiDualRightWeyl B repLorentz f) :
    InvariantReductionToSpan (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f)
```

def ofPairComponents [source]

The linear map out of `ℂT[k, k']` sending the basis tensor `e_a ⊗ e_b` to `T a b`.

```
noncomputable def ofPairComponents (T : basisIdx k → basisIdx k' → B) :
    ℂT[k, k'] →ₗ[ℂ] B
```

lemma range_ofPairComponents [source]

The range of `ofPairComponents T` is the span of the vectors `T a b`.

```
lemma range_ofPairComponents (T : basisIdx k → basisIdx k' → B) :
    LinearMap.range (ofPairComponents T)
      = Submodule.span ℂ (Set.range fun m : basisIdx k × basisIdx k' => T m.1 m.2)
```

lemma ofPairComponents_mem [source]

The image of `ofPairComponents T` lies in every submodule containing the vectors `T a b`.

```
lemma ofPairComponents_mem (T : basisIdx k → basisIdx k' → B) {M : Submodule ℂ B}
    (hT : ∀ a b, T a b ∈ M) (t : ℂT[k, k']) : ofPairComponents T t ∈ M
```

lemma map_ofPairComponents [source]

A linear map applied after `ofPairComponents T` is `ofPairComponents` of its values on the components.

```
lemma map_ofPairComponents {B' : Type*} [AddCommGroup B'] [Module ℂ B'] (σ : B →ₗ[ℂ] B')
    (T : basisIdx k → basisIdx k' → B) (t : ℂT[k, k']) :
    σ (ofPairComponents T t) = ofPairComponents (fun a b => σ (T a b)) t
```

lemma ofPairComponents_sum [source]

`ofPairComponents` of a sum of families is the sum of the maps.

```
lemma ofPairComponents_sum {ι : Type*} (s : Finset ι)
    (T : ι → basisIdx k → basisIdx k' → B) :
    ofPairComponents (fun a b => ∑ i ∈ s, T i a b) = ∑ i ∈ s, ofPairComponents (T i)
```

lemma ofPairComponents_sub [source]

`ofPairComponents` of a difference of families is the difference of the maps.

```
lemma ofPairComponents_sub (T T' : basisIdx k → basisIdx k' → B) :
    ofPairComponents (fun a b => T a b - T' a b) = ofPairComponents T - ofPairComponents T'
```

lemma isEquivariant_ofPairComponents [source]

`ofPairComponents T` is equivariant when each index of `T a b` is moved by the matrix of `g` in its colour, the summed index first in each factor.

```
lemma isEquivariant_ofPairComponents {repLorentz : Representation ℂ SL(2,ℂ) B}
    (T : basisIdx k → basisIdx k' → B)
    (hT : ∀ (g : SL(2,ℂ)) a b, repLorentz g (T a b)
      = ∑ x, ∑ y, (LinearMap.toMatrix (complexLorentzTensor.basis k)
          (complexLorentzTensor.basis k) (complexLorentzTensor.rep k g) x a *
        LinearMap.toMatrix (complexLorentzTensor.basis k') (complexLorentzTensor.basis k')
          (complexLorentzTensor.rep k' g) y b) • T x y) :
    complexLorentzTensor.IsEquivariant ![k, k'] repLorentz (ofPairComponents T)
```

2.603 PhyslibAlpha.Relativity.LorentzGroup.Invariants.IsLeftRightWeyl

PhyslibAlpha/Relativity/LorentzGroup/Invariants/IsLeftRightWeyl.lean on GitHub.

abbrev IsLeftRightWeyl [source]

A family with a left- and a right-handed Weyl index `T^{α α'}`: an equivariant linear map out of `ℂT[.upL, .upR]`.

```
abbrev IsLeftRightWeyl (B : Type*) [AddCommMonoid B] [Module ℂ B]
    (repLorentz : Representation ℂ SL(2,ℂ) B) (f : ℂT[.upL, .upR] →ₗ[ℂ] B) : Prop
```

abbrev IsDualLeftRightWeyl [source]

A family with a dual left- and a dual right-handed Weyl index `T_{α α'}`: an equivariant linear map out of `ℂT[.downL, .downR]`.

```
abbrev IsDualLeftRightWeyl (B : Type*) [AddCommMonoid B] [Module ℂ B]
    (repLorentz : Representation ℂ SL(2,ℂ) B) (f : ℂT[.downL, .downR] →ₗ[ℂ] B) : Prop
```

lemma eq_zero_of_invariant_leftRight [source]

An invariant tensor with a left- and a right-handed Weyl index, or with their duals, is zero. The inverse boost along `z` scales the two diagonal components by `4⁻¹` and `4`, and the inverse half turn about `z` negates the two mixed ones; for the dual colours the inverse cancels against the inverse in the matrix of the colour.

```
lemma eq_zero_of_invariant_leftRight {k k' : complexLorentzTensor.Color}
    (hk : (k = .upL ∧ k' = .upR) ∨ (k = .downL ∧ k' = .downR)) {t : ℂT[k, k']}
```

```
    (ht : ∀ g : SL(2,ℂ), g • t = t) : t = 0
```

lemma IsLeftRightWeyl.reducesInvariantsTo_bot [source]

The invariants of the range of a left-right family reduce to ⊥: a Lorentz invariant of `LinearMap.range f` ⊔ `S`, for `S` a Lorentz-stable submodule, lies in `S`.

```
lemma IsLeftRightWeyl.reducesInvariantsTo_bot {f : ℂT[.upL, .upR] →ₗ[ℂ] B}
    (hf : IsLeftRightWeyl B repLorentz f) :
    ReducesInvariantsTo (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f) ⊥
```

lemma IsLeftRightWeyl.eq_zero_of_invariant [source]

Every Lorentz invariant in the range of a left-right family is zero.

```
lemma IsLeftRightWeyl.eq_zero_of_invariant {f : ℂT[.upL, .upR] →ₗ[ℂ] B}
    (hf : IsLeftRightWeyl B repLorentz f) {x : B} (hx : x ∈ LinearMap.range f)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x = 0
```

lemma IsDualLeftRightWeyl.reducesInvariantsTo_bot [source]

The invariants of the range of a dual left-right family reduce to ⊥: a Lorentz invariant of `LinearMap.range f` ⊔ `S`, for `S` a Lorentz-stable submodule, lies in `S`.

```
lemma IsDualLeftRightWeyl.reducesInvariantsTo_bot {f : ℂT[.downL, .downR] →ₗ[ℂ] B}
    (hf : IsDualLeftRightWeyl B repLorentz f) :
    ReducesInvariantsTo (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f) ⊥
```

lemma IsDualLeftRightWeyl.eq_zero_of_invariant [source]

Every Lorentz invariant in the range of a dual left-right family is zero.

```
lemma IsDualLeftRightWeyl.eq_zero_of_invariant {f : ℂT[.downL, .downR] →ₗ[ℂ] B}
    (hf : IsDualLeftRightWeyl B repLorentz f) {x : B} (hx : x ∈ LinearMap.range f)
    (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x = 0
```

2.604 PhyslibAlpha.Relativity.LorentzGroup.Invariants.IsVectorLeftRightWeyl

PhyslibAlpha/Relativity/LorentzGroup/Invariants/IsVectorLeftRightWeyl.lean on GitHub.

abbrev IsVectorLeftRightWeyl [source]

A family with a four-vector index and a left- and a right-handed Weyl index `T^{μ α α'}`: an equivariant linear map out of `ℂT[.up, .upL, .upR]`.

```
abbrev IsVectorLeftRightWeyl (B : Type*) [AddCommMonoid B] [Module ℂ B]
    (repLorentz : Representation ℂ SL(2,ℂ) B) (f : ℂT[.up, .upL, .upR] →ₗ[ℂ] B) : Prop
```

lemma repLorentz_map_indexBasis [source]

The components of an equivariant map are moved as `T^{μ α α'}`, the summed index first in each factor.

```
lemma repLorentz_map_indexBasis (g : SL(2,ℂ)) (d : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2) :
    repLorentz g (f (indexBasis d)) = ∑ e : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2,
      ((((SL2C.toLorentzGroup g).1 e.1 d.1 : ℝ) : ℂ)
        * (g.1 e.2.1 d.2.1 * star (g.1 e.2.2 d.2.2))) • f (indexBasis e)
```

lemma repLorentz_sum_smul [source]

Moving a contraction of the Weyl pair at vector index `μ`: the vector index moves by the Lorentz matrix and the coefficients of the Weyl pair by the matrix of `g` and its complex conjugate.

```
lemma repLorentz_sum_smul (g : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) (c : Fin 2 × Fin 2 → ℂ) :
    repLorentz g (∑ a : Fin 2 × Fin 2, c a • f (indexBasis (μ, a)))
      = ∑ ν : Fin 1 ⊕ Fin 3, (((SL2C.toLorentzGroup g).1 ν μ : ℝ) : ℂ)
        • ∑ q : Fin 2 × Fin 2, (∑ d : Fin 2 × Fin 2, c d * (g.1 q.1 d.1 * star (g.1 q.2 d.2)))
          • f (indexBasis (ν, q))
```

def vectorPair [source]

The family of two four-vector indices obtained by contracting the Weyl pair of the components of `f` against the covariant Pauli matrices.

```
noncomputable def vectorPair (f : 𝕋T[.up, .upL, .upR] →ₗ[ℂ] B) :
    (Fin 2 → Fin 1 ⊕ Fin 3) → B
```

lemma isLorentzCovariant_vectorPair [source]

The reduced family is a rank-two Lorentz family: the intertwining identity `SL2C.sum_pauliLower_mul_sl2c` carries the Weyl pair into a second vector index.

```
lemma isLorentzCovariant_vectorPair :
    IsLorentzCovariant 2 B repLorentz (ofComponents (vectorPair f))
```

lemma map_indexBasis_eq_sum_vectorPair [source]

The reduction is invertible: by the Fierz completeness relation each component of `f` is recovered from the reduced family.

```
lemma map_indexBasis_eq_sum_vectorPair (f : 𝕋T[.up, .upL, .upR] →ₗ[ℂ] B)
    (μ : Fin 1 ⊕ Fin 3) (b : Fin 2 × Fin 2) :
    f (indexBasis (μ, b)) = ∑ ρ : Fin 1 ⊕ Fin 3,
      ((2 : ℂ)⁻¹ * PauliMatrix.pauliLower ρ b.2 b.1) • vectorPair f ![μ, ρ]
```

lemma span_range_vectorPair [source]

The reduction does not change the span: the reduced family spans the range of `f`.

```
lemma span_range_vectorPair (f : 𝕋T[.up, .upL, .upR] →ₗ[ℂ] B) :
    Submodule.span ℂ (Set.range (vectorPair f)) = LinearMap.range f
```

lemma ofComponents_vectorPair_metric [source]

The image of the metric under the map of the reduced family is the image `f σ^^^` of the Pauli tensor: the two lowerings of the vector index cancel, so no sign and no scalar appear.

```
lemma ofComponents_vectorPair_metric (f : ℂT[.up, .upL, .upR] →ₗ[ℂ] B) :
    ofComponents (vectorPair f) RankTwo.metric = f σ^^^
```

lemma repLorentz_map_pauliMatrix [source]

The image `f σ^^^` of the Pauli tensor is Lorentz invariant.

```
lemma repLorentz_map_pauliMatrix (g : SL(2,ℂ)) : repLorentz g (f σ^^^) = f σ^^^
```

lemma exists_smul_map_pauliMatrix_add_of_invariant [source]

Every Lorentz invariant of `LinearMap.range f ⊔ S`, for `S` a Lorentz-stable submodule, is a multiple of the image `f σ^^^` of the Pauli tensor plus an element of `S`: the shape of the fermion kinetic term.

```
lemma exists_smul_map_pauliMatrix_add_of_invariant (S : Submodule ℂ B)
    (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ LinearMap.range f ⊔ S) (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) :
    ∃ a : ℂ, ∃ y ∈ S, x = a • f σ^^^ + y
```

def IsVectorLeftRightWeyl.invariantReductionToSpan [source]

The Lorentz invariants of the range of `f` reduce to the span of the image `f σ^^^` of the Pauli tensor.

```
noncomputable def invariantReductionToSpan :
    InvariantReductionToSpan (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f) where
```

def ofVectorComponents [source]

The linear map out of `ℂT[.up, .upL, .upR]` sending `e_μ ⊗ e_α ⊗ e_α'` to `T (μ, α, α')`.

```
noncomputable def ofVectorComponents (T : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 → B) :
    ℂT[.up, .upL, .upR] →ₗ[ℂ] B
```

lemma ofVectorComponents_indexBasis [source]

```
@[simp]
lemma ofVectorComponents_indexBasis (T : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 → B)
    (d : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2) : ofVectorComponents T (indexBasis d) = T d
```

lemma range_ofVectorComponents [source]

The range of `ofVectorComponents T` is the span of the vectors `T d`.

```
lemma range_ofVectorComponents (T : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 → B) :
    LinearMap.range (ofVectorComponents T) = Submodule.span ℂ (Set.range T)
```

lemma isVectorLeftRightWeyl_ofVectorComponents [source]

`ofVectorComponents T` is equivariant when `T (μ, α, α')` is moved as `T^{μ α α'}`: the vector index by the Lorentz matrix, the left index by the matrix of `g` and the right index by its complex conjugate, the summed index first in each factor.

```
lemma isVectorLeftRightWeyl_ofVectorComponents {repLorentz : Representation ℂ SL(2,ℂ) B}
    (T : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 → B)
    (hT : ∀ (g : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2),
      repLorentz g (T (μ, l)) = ∑ (ν : Fin 1 ⊕ Fin 3), ∑ (a : Fin 2 × Fin 2),
        ((((SL2C.toLorentzGroup g).1 ν μ : ℝ) : ℂ)
          * (g.1 a.1 l.1 * star (g.1 a.2 l.2))) • T (ν, a)) :
    IsVectorLeftRightWeyl B repLorentz (ofVectorComponents T)
```

abbrev IsVectorDualLeftRightWeyl [source]

A family with a four-vector index and a dual right- and a dual left-handed Weyl index `T^μ{}_{α' α}`: an equivariant linear map out of `ℂT[.up, .downR, .downL]`.

```
abbrev IsVectorDualLeftRightWeyl (B : Type*) [AddCommMonoid B] [Module ℂ B]
    (repLorentz : Representation ℂ SL(2,ℂ) B) (f : ℂT[.up, .downR, .downL] →ₗ[ℂ] B) : Prop
```

def dualWeylMap [source]

Dualising the two Weyl indices of a tensor with a four-vector index and a left-right Weyl pair, the dual indices listed in the order of `σ^__`.

```
noncomputable def dualWeylMap : ℂT[.up, .upL, .upR] →ₗ[ℂ] ℂT[.up, .downR, .downL]
```

lemma dualWeylMap_equivariant [source]

Dualising the Weyl indices commutes with the action of `SL(2,ℂ)`.

```
lemma dualWeylMap_equivariant (g : SL(2,ℂ)) (t : ℂT[.up, .upL, .upR]) :
    dualWeylMap (g • t) = g • dualWeylMap t
```

lemma dualWeylMap_pauliMatrix [source]

Dualising the Weyl indices of `σ^^^` gives `σ^__`.

```
lemma dualWeylMap_pauliMatrix : dualWeylMap σ^^^ = σ^__
```

lemma dualWeylMap_surjective [source]

Dualising the Weyl indices is surjective: the dualisations are inverted by raising the indices again, and the relabelling by its inverse.

```
lemma dualWeylMap_surjective : Function.Surjective dualWeylMap
```

lemma isVectorLeftRightWeyl_comp_dualWeylMap [source]

Composing with the dualisation of the Weyl indices gives a family with a four-vector index and a left-right Weyl pair.

```
lemma isVectorLeftRightWeyl_comp_dualWeylMap :
    IsVectorLeftRightWeyl B repLorentz (f ∘ₗ dualWeylMap) where
```

lemma range_comp_dualWeylMap [source]

Composing with the dualisation of the Weyl indices does not change the range.

```
lemma range_comp_dualWeylMap : LinearMap.range (f ∘ₗ dualWeylMap) = LinearMap.range f
```

lemma exists_smul_map_pauliContrDown_add_of_invariant [source]

Every Lorentz invariant of `LinearMap.range f ⊔ S`, for `S` a Lorentz-stable submodule, is a multiple of the image `f σ^__` of the Pauli tensor with dual Weyl indices plus an element of `S`: the kinetic term of a Weyl fermion.

```
lemma exists_smul_map_pauliContrDown_add_of_invariant (S : Submodule ℂ B)
    (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ LinearMap.range f ⊔ S) (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) :
    ∃ a : ℂ, ∃ y ∈ S, x = a • f σ^__ + y
```

def IsVectorDualLeftRightWeyl.invariantReductionToSpan [source]

The Lorentz invariants of the range of `f` reduce to the span of the image `f σ^__` of the Pauli tensor with dual Weyl indices.

```
noncomputable def invariantReductionToSpan :
    InvariantReductionToSpan (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f) where
```

def dualIndexEquiv [source]

The component indices of a tensor of `ℂT[.up, .downR, .downL]`, as a four-vector index, a dual left-handed and a dual right-handed Weyl index.

```
def dualIndexEquiv : ComponentIdx (S := complexLorentzTensor) ![.up, .downR, .downL] ≃
    (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 where
```

def ofDualVectorComponents [source]

The linear map out of `ℂT[.up, .downR, .downL]` sending `e_μ ⊗ e_α' ⊗ e_α` to `T (μ, α, α')`.

```
noncomputable def ofDualVectorComponents (T : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 → B) :
    ℂT[.up, .downR, .downL] →ₗ[ℂ] B
```

lemma range_ofDualVectorComponents [source]

The range of `ofDualVectorComponents T` is the span of the vectors `T d`.

```
lemma range_ofDualVectorComponents (T : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 → B) :
    LinearMap.range (ofDualVectorComponents T) = Submodule.span ℂ (Set.range T)
```

lemma ofDualVectorComponents_mem_span [source]

The image of `ofDualVectorComponents T` lies in the span of the vectors `T d`.

```
lemma ofDualVectorComponents_mem_span (T : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 → B)
    (t : ℂT[.up, .downR, .downL]) :
    ofDualVectorComponents T t ∈ Submodule.span ℂ (Set.range T)
```

lemma ofDualVectorComponents_sum [source]

`ofDualVectorComponents` of a sum of families is the sum of the maps.

```
lemma ofDualVectorComponents_sum {ι : Type*} (s : Finset ι)
    (T : ι → (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 → B) :
    ofDualVectorComponents (fun d => ∑ i ∈ s, T i d) = ∑ i ∈ s, ofDualVectorComponents (T i)
```

lemma isVectorDualLeftRightWeyl_ofDualVectorComponents [source]

`ofDualVectorComponents T` is equivariant when `T (μ, α, α')` is moved as `T^μ{}_{α α'}`: the vector index by the Lorentz matrix, the dual left index by $(g^{-1})^T$ and the dual right index by $(g^{-1})^H$, the summed index first in each factor.

```
lemma isVectorDualLeftRightWeyl_ofDualVectorComponents
    {repLorentz : Representation ℂ SL(2,ℂ) B} (T : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 → B)
    (hT : ∀ (g : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) (l : Fin 2 × Fin 2),
      repLorentz g (T (μ, l)) = ∑ (ν : Fin 1 ⊕ Fin 3), ∑ (a : Fin 2 × Fin 2),
        ((((SL2C.toLorentzGroup g).1 ν μ : ℝ) : ℂ)
          * ((g.1⁻¹)^T a.1 l.1 * (g.1⁻¹)^H a.2 l.2)) • T (ν, a)) :
    IsVectorDualLeftRightWeyl B repLorentz (ofDualVectorComponents T)
```

def invariantReductionToComponentSpan [source]

For a family whose map is equivariant, the Lorentz invariants of the span of the family reduce to the span of the image `ofDualVectorComponents T σ^__` of the Pauli tensor with dual Weyl indices.

```
noncomputable def IsVectorDualLeftRightWeyl.invariantReductionToComponentSpan
    {repLorentz : Representation ℂ SL(2,ℂ) B} {T : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2 → B}
    (hT : IsVectorDualLeftRightWeyl B repLorentz (ofDualVectorComponents T)) :
    InvariantReductionToSpan (fun g : SL(2,ℂ) => repLorentz g)
      (Submodule.span ℂ (Set.range T)) where
```

2.605 PhyslibAlpha.Relativity.LorentzGroup.Invariants.LightCone

PhyslibAlpha/Relativity/LorentzGroup/Invariants/LightCone.lean on GitHub.

def lightConeCoeffZ [source]

The four light-cone directions of axis `i`, as integers.

```
def lightConeCoeffZ (i : Fin 3) (κ : Fin 4) (μ : Fin 1 ⊕ Fin 3) : ℤ
```

lemma coe_lightConeCoeffZ [source]

The integer copy casts to `lightConeCoeff`.

```
lemma coe_lightConeCoeffZ (i : Fin 3) (κ : Fin 4) (μ : Fin 1 ⊕ Fin 3) :
    ((lightConeCoeffZ i κ μ : ℤ) : ℂ) = lightConeCoeff i κ μ
```

def lightConeCoeffInvZ [source]

Twice the coordinate directions in the light-cone basis, the `2` clearing the halves.

```
def lightConeCoeffInvZ (i : Fin 3) (μ : Fin 1 ⊕ Fin 3) (κ : Fin 4) : ℤ
```

lemma coe_lightConeCoeffInvZ_eq_two_mul [source]

The integer copy is exactly twice `lightConeCoeffInv`.

```
lemma coe_lightConeCoeffInvZ_eq_two_mul (i : Fin 3) (μ : Fin 1 ⊕ Fin 3) (κ : Fin 4) :
    ((lightConeCoeffInvZ i μ κ : ℤ) : ℂ) = 2 * lightConeCoeffInv i μ κ
```

def InBoostPlane [source]

The boost plane of axis `i`: time and the axis, the two directions the boost moves.

```
def InBoostPlane (i : Fin 3) (μ : Fin 1 ⊕ Fin 3) : Prop
```

instance Decidable (InBoostPlane i μ) [source]

```
instance (i : Fin 3) (μ : Fin 1 ⊕ Fin 3) : Decidable (InBoostPlane i μ)
```

lemma lightConeCoeffInvZ_eq_zero_of_inBoostPlane [source]

A direction in the boost plane has no transverse light-cone components.

```
lemma lightConeCoeffInvZ_eq_zero_of_inBoostPlane {i : Fin 3} {μ : Fin 1 ⊕ Fin 3}
    (hμ : InBoostPlane i μ) {κ : Fin 4} (hκ : κ = 2 ∨ κ = 3) :
    lightConeCoeffInvZ i μ κ = 0
```

lemma lightConeCoeffInvZ_eq_zero_of_not_inBoostPlane [source]

A transverse direction has no light-cone components in the boost plane.

```
lemma lightConeCoeffInvZ_eq_zero_of_not_inBoostPlane {i : Fin 3} {μ : Fin 1 ⊕ Fin 3}
    (hμ : ¬InBoostPlane i μ) {κ : Fin 4} (hκ : κ = 0 ∨ κ = 1) :
    lightConeCoeffInvZ i μ κ = 0
```

def slotZ [source]

One slot's factor: twice the coefficient of `κ` in `μ`, times that of `ν` in `κ`.

```
def slotZ (i : Fin 3) (κ : Fin 4) (μ ν : Fin 1 ⊕ Fin 3) : ℤ
```

def transitionZ [source]

Two to the number of slots times the entry, at `d` and `e`, of the map keeping the light-cone components of total weight `m` along axis `i`; the factor is the one

`lightConeCoeffInvZ` carries, one per slot. A slot of `d` in the boost plane takes weight `2` or `-2`, leaving `m - 2` or `m + 2`; a transverse slot takes weight `0` and leaves `m`.

```
def transitionZ (i : Fin 3) : {n : ℕ} → (d e : Fin n → Fin 1 ⊕ Fin 3) → ℤ → ℤ
  | 0, _, _, m => if m = 0 then 1 else 0
  | _ + 1, d, e, m =>
    if InBoostPlane i (d 0) then
      slotZ i 0 (d 0) (e 0) * transitionZ i (Fin.tail d) (Fin.tail e) (m - 2)
        + slotZ i 1 (d 0) (e 0) * transitionZ i (Fin.tail d) (Fin.tail e) (m + 2)
    else (slotZ i 2 (d 0) (e 0) + slotZ i 3 (d 0) (e 0))
      * transitionZ i (Fin.tail d) (Fin.tail e) m
```

lemma transitionZ_succ [source]

One slot of the transition, with the case split of the recursion resolved: the first slot runs over all four light-cone directions, each taking its own weight out of `m`. The two directions of the boost plane drop out of a transverse slot and the two transverse ones drop out of a slot in the boost plane, which is what the two branches of `transitionZ` record.

```
lemma transitionZ_succ (i : Fin 3) {n : ℕ} (d e : Fin (n + 1) → Fin 1 ⊕ Fin 3) (m : ℤ) :
    transitionZ i d e m
      = ∑ κ : Fin 4, slotZ i κ (d 0) (e 0)
          * transitionZ i (Fin.tail d) (Fin.tail e) (m - lightConeWeight κ)
```

lemma transitionZ_eq_sum [source]

The recursion unfolded, as a sum over the multi-indices of total weight `m`.

```
lemma transitionZ_eq_sum (i : Fin 3) :
    ∀ {n : ℕ} (d e : Fin n → Fin 1 ⊕ Fin 3) (m : ℤ),
    transitionZ i d e m
      = ∑ κ ∈ Finset.univ.filter
          (fun κ : Fin n → Fin 4 => (∑ s, lightConeWeight (κ s)) = m),
        ∏ s, slotZ i (κ s) (d s) (e s)
  | 0, d, e, m => by
    rw [Finset.sum_filter, Fintype.sum_unique]
    simp [transitionZ, eq_comm]
  | n + 1, d, e, m => by
    rw [transitionZ_succ,
      Physlib.Fin.sum_filter_weight_succ lightConeWeight fun s κ => slotZ i κ (d s) (e s)]
    refine Finset.sum_congr rfl fun κ₀ _ => ?_
    rw [transitionZ_eq_sum i (Fin.tail d) (Fin.tail e)]
    rfl
```

2.606 PhyslibAlpha.Relativity.LorentzGroup.Invariants.LorentzCovariance

PhyslibAlpha/Relativity/LorentzGroup/Invariants/LorentzCovariance.lean on GitHub.

abbrev IsLorentzCovariant [source]

A family with `n` four-vector indices `T^{μ₁ ⋯ μₙ}` in a representation `repLorentz` of `SL(2,ℂ)`: an equivariant linear map out of the complex Lorentz tensors with `n` contravariant indices.

```
abbrev IsLorentzCovariant (n : ℕ) (B : Type*) [AddCommMonoid B] [Module ℂ B]
```

```
    (repLorentz : Representation ℂ SL(2,ℂ) B)
    (f : ℂT(fun _ : Fin n => Color.up) →ₗ[ℂ] B) : Prop
```

lemma sum_pi_fin_two [source]

A sum over families of two four-vector indices is a double sum.

```
lemma sum_pi_fin_two {M : Type*} [AddCommMonoid M] (f : (Fin 2 → Fin 1 ⊕ Fin 3) → M) :
    ∑ d : Fin 2 → Fin 1 ⊕ Fin 3, f d
      = ∑ x : Fin 1 ⊕ Fin 3, ∑ y : Fin 1 ⊕ Fin 3, f ![x, y]
```

def vectorIdx [source]

The component indices of a tensor with `n` contravariant indices, which are labelled by `Fin 1 ⊕ Fin 3` in each slot.

```
def vectorIdx (n : ℕ) :
    ComponentIdx (S := complexLorentzTensor) (fun _ : Fin n => Color.up)
      ≃ (Fin n → Fin 1 ⊕ Fin 3)
```

lemma vectorIdx_symm_apply [source]

```
lemma vectorIdx_symm_apply (d : Fin n → Fin 1 ⊕ Fin 3) (i : Fin n) :
    (vectorIdx n).symm d i = d i
```

def coeffEquiv [source]

The components of a tensor with `n` contravariant indices, as a coefficient tensor.

```
noncomputable def coeffEquiv (n : ℕ) :
    ℂT(fun _ : Fin n => Color.up) ≃ₗ[ℂ] ((Fin n → Fin 1 ⊕ Fin 3) → ℂ)
```

lemma coeffEquiv_apply [source]

```
lemma coeffEquiv_apply (t : ℂT(fun _ : Fin n => Color.up)) (d : Fin n → Fin 1 ⊕ Fin 3) :
    coeffEquiv n t d = (Tensor.basis _).repr t ((vectorIdx n).symm d)
```

lemma coeffEquiv_symm_apply [source]

The tensor with coefficient tensor `c` is the combination of the basis tensors with those coefficients.

```
lemma coeffEquiv_symm_apply (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ) :
    (coeffEquiv n).symm c = ∑ d, c d • Tensor.basis _ ((vectorIdx n).symm d)
```

lemma permT_coeffEquiv_symm [source]

Permuting the slots of a tensor by `σ` permutes its coefficient tensor: slot `i` of `permT σ h t` holds slot `σ i` of `t`, so the new coefficient at `d` is the old one at `d ∘ σ.symm`.

```
lemma permT_coeffEquiv_symm (σ : Equiv.Perm (Fin n))
    (h : IsReindexing (fun _ : Fin n => Color.up) (fun _ => Color.up) σ)
    (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ) :
    permT σ h ((coeffEquiv n).symm c) = (coeffEquiv n).symm fun d => c (d ∘ σ.symm)
```

lemma coeffEquiv_smul [source]

The action of `g` on tensors is the action `act` of its Lorentz matrix on coefficient tensors.

```
lemma coeffEquiv_smul (g : SL(2,ℂ)) (t : ℂT(fun _ : Fin n => Color.up)) :
    coeffEquiv n (g • t) = act (SL2C.toLorentzGroup g).1 (coeffEquiv n t)
```

lemma invariant_iff_isInvariantCoeff [source]

A tensor is invariant exactly when its coefficient tensor is an invariant coefficient tensor.

```
lemma invariant_iff_isInvariantCoeff (t : ℂT(fun _ : Fin n => Color.up)) :
    (∀ g : SL(2,ℂ), g • t = t) ↔ IsInvariantCoeff (coeffEquiv n t)
```

lemma reducesInvariantsTo_span [source]

When the invariant coefficient tensors lie in the span of the coefficient tensors `K j`, the invariants of the range of `f` reduce to the span of the images of the tensors with those coefficients.

```
lemma reducesInvariantsTo_span (hf : IsLorentzCovariant n B repLorentz f) {ι : Type*}
    (K : ι → (Fin n → Fin 1 ⊕ Fin 3) → ℂ)
    (hK : ∀ c, IsInvariantCoeff c → c ∈ Submodule.span ℂ (Set.range K)) :
    ReducesInvariantsTo (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f)
      (Submodule.span ℂ (Set.range fun j => f ((coeffEquiv n).symm (K j))))
```

lemma reducesInvariantsTo_bot_of_isInvariantCoeff [source]

When the only invariant coefficient tensor is zero, the range of `f` reduces to `⊥`.

```
lemma reducesInvariantsTo_bot_of_isInvariantCoeff (hf : IsLorentzCovariant n B repLorentz f)
    (hK : ∀ c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ, IsInvariantCoeff c → c = 0) :
    ReducesInvariantsTo (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f) ⊥
```

lemma eq_zero_of_isInvariantCoeff [source]

When the only invariant coefficient tensor is zero, so is every Lorentz invariant in the range of `f`.

```
lemma eq_zero_of_isInvariantCoeff (hf : IsLorentzCovariant n B repLorentz f)
    (hK : ∀ c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ, IsInvariantCoeff c → c = 0) {x : B}
    (hx : x ∈ LinearMap.range f) (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x = 0
```

def ofComponents [source]

The linear map out of the tensors with `n` contravariant indices sending the basis tensor with components `d` to `T d`.

```
noncomputable def ofComponents (T : (Fin n → Fin 1 ⊕ Fin 3) → B) :
    ℂT(fun _ : Fin n => Color.up) →ₗ[ℂ] B
```

lemma ofComponents_coeffEquiv_symm [source]

`ofComponents T` contracts the coefficient tensor of its argument with `T`.

```
lemma ofComponents_coeffEquiv_symm (T : (Fin n → Fin 1 ⊕ Fin 3) → B)
    (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ) :
    ofComponents T ((coeffEquiv n).symm c) = ∑ d, c d • T d
```

lemma range_ofComponents [source]

The range of `ofComponents T` is the span of the vectors `T d`.

```
lemma range_ofComponents (T : (Fin n → Fin 1 ⊕ Fin 3) → B) :
    LinearMap.range (ofComponents T) = Submodule.span ℂ (Set.range T)
```

lemma ofComponents_sum [source]

`ofComponents` of a sum of families is the sum of the maps.

```
lemma ofComponents_sum {ι : Type*} (s : Finset ι) (T : ι → (Fin n → Fin 1 ⊕ Fin 3) → B) :
    ofComponents (fun d => ∑ i ∈ s, T i d) = ∑ i ∈ s, ofComponents (T i)
```

lemma comp_ofComponents [source]

A linear map applied after `ofComponents T` is `ofComponents` of its values on the components.

```
lemma comp_ofComponents {B' : Type*} [AddCommGroup B'] [Module ℂ B'] (σ : B →ₗ[ℂ] B')
    (T : (Fin n → Fin 1 ⊕ Fin 3) → B) :
    σ ∘ₗ ofComponents T = ofComponents fun d => σ (T d)
```

lemma ofComponents_comp_permT [source]

Precomposing `ofComponents T` with the slot permutation `permT σ` reads the family at the permuted index vectors `d ∘ σ`.

```
lemma ofComponents_comp_permT (σ : Fin n → Fin n)
    (h : IsReindexing (fun _ : Fin n => Color.up) (fun _ => Color.up) σ)
    (T : (Fin n → Fin 1 ⊕ Fin 3) → B) :
    ofComponents T ∘ₗ permT σ h = ofComponents fun d => T (d ∘ σ)
```

lemma ofComponents_comp_permT_eq_neg [source]

A family antisymmetric under reading its index vectors through `σ` gives a map antisymmetric under `permT σ`.

```
lemma ofComponents_comp_permT_eq_neg (σ : Fin n → Fin n)
    (h : IsReindexing (fun _ : Fin n => Color.up) (fun _ => Color.up) σ)
    {T : (Fin n → Fin 1 ⊕ Fin 3) → B} (hT : ∀ d, T (d ∘ σ) = -T d) :
    ofComponents T ∘ₗ permT σ h = -ofComponents T
```

lemma ofComponents_comp_permT_eq_self [source]

A family symmetric under reading its index vectors through `σ` gives a map invariant under `permT σ`.

```
lemma ofComponents_comp_permT_eq_self (σ : Fin n → Fin n)
    (h : IsReindexing (fun _ : Fin n => Color.up) (fun _ => Color.up) σ)
    {T : (Fin n → Fin 1 ⊕ Fin 3) → B} (hT : ∀ d, T (d ∘ σ) = T d) :
    ofComponents T ∘ₗ permT σ h = ofComponents T
```

lemma isLorentzCovariant_ofComponents_iff [source]

The map of a family is equivariant exactly when the family obeys the transformation law of the components of a tensor with `n` four-vector indices.

```
lemma isLorentzCovariant_ofComponents_iff {repLorentz : Representation ℂ SL(2,ℂ) B}
    (T : (Fin n → Fin 1 ⊕ Fin 3) → B) :
    IsLorentzCovariant n B repLorentz (ofComponents T) ↔
      ∀ (g : SL(2,ℂ)) l, repLorentz g (T l) = ∑ a : Fin n → Fin 1 ⊕ Fin 3,
        (∏ i, (((SL2C.toLorentzGroup g).1 (a i) (l i) : ℝ) : ℂ)) • T a
```

2.607 PhyslibAlpha.Relativity.LorentzGroup.Invariants.RankFour

PhyslibAlpha/Relativity/LorentzGroup/Invariants/RankFour.lean on GitHub.

def contractionCoeff [source]

The four coefficient tensors: the three metric pairings, then the Levi-Civita symbol.

```
def contractionCoeff : Fin 4 → (Fin 4 → Fin 1 ⊕ Fin 3) → ℤ
```

def contractionTensor [source]

The four invariant tensors with four contravariant indices: `η_{μν} η_{ρσ}`, `η_{μρ} η_{νσ}`, `η_{μσ} η_{νρ}` and `ε_{μνρσ}`.

```
noncomputable def contractionTensor (i : Fin 4) : 𝕋T(fun _ : Fin 4 => Color.up)
```

lemma ofComponents_contractionTensor [source]

`ofComponents T` sends each invariant tensor to the contraction of `T` with its coefficients.

```
lemma ofComponents_contractionTensor (T : (Fin 4 → Fin 1 ⊕ Fin 3) → B) (i : Fin 4) :
    ofComponents T (contractionTensor i) = ∑ d, ((contractionCoeff i d : ℤ) : ℂ) • T d
```

lemma sum_pi_four [source]

Bookkeeping: a sum over index vectors is a fourfold sum over directions.

```
lemma sum_pi_four {M : Type*} [AddCommMonoid M] (F : (Fin 4 → Fin 1 ⊕ Fin 3) → M) :
    ∑ d : Fin 4 → Fin 1 ⊕ Fin 3, F d
      = ∑ x : Fin 1 ⊕ Fin 3, ∑ y : Fin 1 ⊕ Fin 3, ∑ z : Fin 1 ⊕ Fin 3,
        ∑ w : Fin 1 ⊕ Fin 3, F ![x, y, z, w]
```

lemma act_outerPair [source]

The pairing of slots `(0,1)` and `(2,3)` is fixed: two copies of `LorentzGroup.sum_minkowskiMatrixZ_mul`.

```
lemma act_outerPair (Λ : LorentzGroup 3) (a : Fin 4 → Fin 1 ⊕ Fin 3) :
    act Λ.1 (fun d => ((minkowskiMatrixZ (d 0) (d 1) * minkowskiMatrixZ (d 2) (d 3) : ℤ) : ℂ))
    a
      = ((minkowskiMatrixZ (a 0) (a 1) * minkowskiMatrixZ (a 2) (a 3) : ℤ) : ℂ)
```

lemma act_outerPair_comp [source]

The same for the slots permuted by `σ`, by renaming the summation variable.

```
lemma act_outerPair_comp (σ : Equiv.Perm (Fin 4)) (Λ : LorentzGroup 3)
    (a : Fin 4 → Fin 1 ⊕ Fin 3) :
    act Λ.1 (fun d => ((minkowskiMatrixZ (d (σ 0)) (d (σ 1))
        * minkowskiMatrixZ (d (σ 2)) (d (σ 3)) : ℤ) : ℂ)) a
      = ((minkowskiMatrixZ (a (σ 0)) (a (σ 1))
        * minkowskiMatrixZ (a (σ 2)) (a (σ 3)) : ℤ) : ℂ)
```

lemma act_leviCivitaSymbol [source]

The symbol is fixed by a Lorentz matrix of determinant `1`; in general it picks up `det Λ`, by `sum_leviCivitaSymbol_mul_prod`.

```
lemma act_leviCivitaSymbol (Λ : LorentzGroup 3) (hΛ : Λ.1.det = 1) (a : Fin 4 → Fin 1 ⊕ Fin 3)
    :
    act Λ.1 (fun d => ((leviCivitaSymbol fun μ => d (finSumFinEquiv μ) : ℤ) : ℂ)) a
      = ((leviCivitaSymbol fun μ => a (finSumFinEquiv μ) : ℤ) : ℂ)
```

lemma isInvariantCoeff_contractionCoeff [source]

The four coefficient tensors are invariant.

```
lemma isInvariantCoeff_contractionCoeff (i : Fin 4) :
    IsInvariantCoeff fun d => ((contractionCoeff i d : ℤ) : ℂ)
```

lemma contractionTensor_invariant [source]

Each of the four tensors is Lorentz invariant, its coefficient tensor being invariant.

```
lemma contractionTensor_invariant (i : Fin 4) (g : SL(2,ℂ)) :
    g • contractionTensor i = contractionTensor i
```

lemma isReindexing_swap_two_three [source]

Exchanging the last two of four contravariant slots is a reindexing.

```
lemma isReindexing_swap_two_three :
    Tensor.IsReindexing (fun _ : Fin 4 => Color.up) (fun _ => Color.up)
      ⇑(Equiv.swap (2 : Fin 4) 3)
```

lemma permT_swap_two_three_contractionTensor_zero [source]

`η_{μν} η_{ρσ}` is unchanged by exchanging its last two slots.

```
lemma permT_swap_two_three_contractionTensor_zero :
    Tensor.permT _ isReindexing_swap_two_three (contractionTensor 0) = contractionTensor 0
```

lemma permT_swap_two_three_contractionTensor_one [source]

Exchanging the last two slots of η_{μρ} η_{νσ} gives η_{μσ} η_{νρ}.

```
lemma permT_swap_two_three_contractionTensor_one :
    Tensor.permT _ isReindexing_swap_two_three (contractionTensor 1) = contractionTensor 2
```

lemma apply_contractionTensor_zero_of_antisymm [source]

A map antisymmetric in the last two slots sends η_{μν} η_{ρσ} to zero.

```
lemma apply_contractionTensor_zero_of_antisymm
    (hf : f ∘ₗ Tensor.permT _ isReindexing_swap_two_three = -f) :
    f (contractionTensor 0) = 0
```

lemma apply_contractionTensor_two_of_antisymm [source]

A map antisymmetric in the last two slots sends η_{μσ} η_{νρ} to minus the image of η_{μρ} η_{νσ}.

```
lemma apply_contractionTensor_two_of_antisymm
    (hf : f ∘ₗ Tensor.permT _ isReindexing_swap_two_three = -f) :
    f (contractionTensor 2) = -f (contractionTensor 1)
```

lemma span_range_contractionTensor_of_antisymm [source]

For a map antisymmetric in the last two slots, the images of the four invariant tensors span the same as the images of η_{μρ} η_{νσ} and ε_{μνρσ}.

```
lemma span_range_contractionTensor_of_antisymm
    (hf : f ∘ₗ Tensor.permT _ isReindexing_swap_two_three = -f) :
    Submodule.span ℂ (Set.range fun i => f (contractionTensor i))
      = Submodule.span ℂ {f (contractionTensor 1), f (contractionTensor 3)}
```

lemma isReindexing_swap_zero_one [source]

Exchanging the first two of four contravariant slots is a reindexing.

```
lemma isReindexing_swap_zero_one :
    Tensor.IsReindexing (fun _ : Fin 4 => Color.up) (fun _ => Color.up)
      ⇑(Equiv.swap (0 : Fin 4) 1)
```

lemma permT_swap_zero_one_contractionTensor_one [source]

Exchanging the first two slots of η_{μρ} η_{νσ} gives η_{μσ} η_{νρ}.

```
lemma permT_swap_zero_one_contractionTensor_one :
    Tensor.permT _ isReindexing_swap_zero_one (contractionTensor 1) = contractionTensor 2
```

lemma permT_swap_zero_one_contractionTensor_three [source]

Exchanging the first two slots negates ε_{μνρσ}.

```
lemma permT_swap_zero_one_contractionTensor_three :
    Tensor.permT _ isReindexing_swap_zero_one (contractionTensor 3) = -contractionTensor 3
```

lemma apply_contractionTensor_one_of_symm_of_antisymm [source]

A map symmetric in the first two slots and antisymmetric in the last two sends `η_{μρ} η_{νσ}` to zero.

```
lemma apply_contractionTensor_one_of_symm_of_antisymm
    (hs : f ∘ₗ Tensor.permT _ isReindexing_swap_zero_one = f)
    (ha : f ∘ₗ Tensor.permT _ isReindexing_swap_two_three = -f) :
    f (contractionTensor 1) = 0
```

lemma apply_contractionTensor_three_of_symm [source]

A map symmetric in the first two slots sends `ε_{μνρσ}` to zero.

```
lemma apply_contractionTensor_three_of_symm
    (hs : f ∘ₗ Tensor.permT _ isReindexing_swap_zero_one = f) :
    f (contractionTensor 3) = 0
```

def IsHalfTurnFixed [source]

All three half turns fix the coefficient at `d`, that is the sign product is `1` for each axis. Equivalently, and not used below, all four directions occur an even number of times among the slots, or all four an odd number.

```
def IsHalfTurnFixed (d : Fin 4 → Fin 1 ⊕ Fin 3) : Prop
```

instance DecidablePred IsHalfTurnFixed [source]

```
instance : DecidablePred IsHalfTurnFixed
```

lemma eq_zero_of_not_isHalfTurnFixed [source]

An invariant coefficient tensor vanishes off the half-turn fixed index vectors.

```
lemma eq_zero_of_not_isHalfTurnFixed {c : (Fin 4 → Fin 1 ⊕ Fin 3) → ℂ}
    (hc : IsInvariantCoeff c) {d : Fin 4 → Fin 1 ⊕ Fin 3} (hd : ¬IsHalfTurnFixed d) :
    c d = 0
```

lemma sixteen_mul_eq_sum_transitionZ [source]

An invariant coefficient tensor is its own weight-zero projection.

```
lemma sixteen_mul_eq_sum_transitionZ {c : (Fin 4 → Fin 1 ⊕ Fin 3) → ℂ}
    (hc : IsInvariantCoeff c) (i : Fin 3) (d : Fin 4 → Fin 1 ⊕ Fin 3) :
    16 * c d = ∑ e, ((transitionZ i d e 0 : ℤ) : ℂ) * c e
```

def orbitRep [source]

One index vector from each of the `22` orbits, checked by the two lemmas below.

```
def orbitRep : Fin 22 → Fin 4 → Fin 1 ⊕ Fin 3
```

def orbit [source]

The `k`-th representative and its two cyclings; for `tttt` the three coincide.

```
def orbit (k : Fin 22) : Finset (Fin 4 → Fin 1 ⊕ Fin 3)
```

lemma isHalfTurnFixed_iff_exists_mem_orbit [source]

The vectors in one of the `22` orbits are exactly the half-turn fixed ones, a finite check.

```
lemma isHalfTurnFixed_iff_exists_mem_orbit :
    ∀ d, IsHalfTurnFixed d ↔ ∃ k, d ∈ orbit k
```

lemma disjoint_orbit [source]

Different orbits share no index vector, a finite check.

```
lemma disjoint_orbit : ∀ k l : Fin 22, k ≠ l → Disjoint (orbit k) (orbit l)
```

lemma eq_of_mem_orbit [source]

An index vector lies in at most one orbit.

```
lemma eq_of_mem_orbit {k l : Fin 22} {d : Fin 4 → Fin 1 ⊕ Fin 3}
    (hk : d ∈ orbit k) (hl : d ∈ orbit l) : k = l
```

lemma eq_orbitRep_of_mem_orbit [source]

A tensor unchanged by cycling takes, on an orbit, its value at the representative.

```
lemma eq_orbitRep_of_mem_orbit {c : (Fin 4 → Fin 1 ⊕ Fin 3) → ℂ}
    (hc : ∀ d, c (cycIdx d) = c d) {k : Fin 22} {d : Fin 4 → Fin 1 ⊕ Fin 3}
    (h : d ∈ orbit k) : c d = c (orbitRep k)
```

def ofOrbitCoord [source]

The coefficient tensor with orbit coordinates `b`, and `0` off the orbits.

```
noncomputable def ofOrbitCoord (b : Fin 22 → ℂ) (d : Fin 4 → Fin 1 ⊕ Fin 3) : ℂ
```

lemma eq_ofOrbitCoord [source]

An invariant coefficient tensor is rebuilt from its `22` orbit coordinates.

```
lemma eq_ofOrbitCoord {c : (Fin 4 → Fin 1 ⊕ Fin 3) → ℂ} (hc : IsInvariantCoeff c) :
    c = ofOrbitCoord fun k => c (orbitRep k)
```

lemma sum_mul_ofOrbitCoord [source]

Contracting against such a tensor collects the `256` index vectors into the `22` orbits.

```
lemma sum_mul_ofOrbitCoord (f : (Fin 4 → Fin 1 ⊕ Fin 3) → ℂ) (b : Fin 22 → ℂ) :
    ∑ e, f e * ofOrbitCoord b e = ∑ l, (∑ e ∈ orbit l, f e) * b l
```

def orbitMatrix [source]

Forty-eight times the weight-zero projection along `z` on the orbit coordinates, the three cyclings summed and not averaged, so `48 = 3 * 16`. Meaningful only through `orbitMatrix_apply`.

```
def orbitMatrix : Matrix (Fin 22) (Fin 22) ℤ
```

instance decidableForallEntries [source]

Decidability for a matrix against a function; search misses the `Matrix` synonym.

```
private instance decidableForallEntries {n : ℕ} (f : Matrix (Fin n) (Fin n) ℤ)
    (g : Fin n → Fin n → ℤ) : Decidable (∀ k l, f k l = g k l)
```

instance decidableForallEntries' [source]

The same for two matrices, which is the shape of `certificate`.

```
private instance decidableForallEntries' {n : ℕ} (f g : Matrix (Fin n) (Fin n) ℤ) :
    Decidable (∀ k l, f k l = g k l)
```

lemma orbitMatrix_apply [source]

Each entry sums the `z`-axis transitions from orbit `k` into orbit `l`, a finite check.

```
lemma orbitMatrix_apply : ∀ k l : Fin 22,
    orbitMatrix k l = ∑ e ∈ orbit l, (transitionZ 2 (orbitRep k) e 0
      + transitionZ 2 (cycIdx (orbitRep k)) e 0
      + transitionZ 2 (cycIdx (cycIdx (orbitRep k))) e 0)
```

lemma orbitMatrix_mulVec [source]

The orbit coordinates of an invariant coefficient tensor satisfy `M b = 48 b`.

```
lemma orbitMatrix_mulVec {c : (Fin 4 → Fin 1 ⊕ Fin 3) → ℂ}
    (hc : IsInvariantCoeff c) :
    orbitMatrix.map (Int.cast : ℤ → ℂ) *ᵥ (fun k => c (orbitRep k))
      = (48 : ℂ) • fun k => c (orbitRep k)
```

def contractionOrbit [source]

The orbit coordinates of the `i`-th coefficient tensor.

```
def contractionOrbit (i : Fin 4) (k : Fin 22) : ℤ
```

def contractionWeight [source]

Four rows of `22` integers paired with `contractionOrbit` to build `projector`. Found by computation and characterised by `contractionWeight_mul_contractionOrbit`; unrelated to boost weight.

```
def contractionWeight : Fin 4 → Fin 22 → ℤ
```

lemma contractionWeight_mul_contractionOrbit [source]

The weight rows and orbit coordinates pair to `24 δᵢⱼ`, a finite check.

```
lemma contractionWeight_mul_contractionOrbit : ∀ i j : Fin 4,
    ∑ k, contractionWeight i k * contractionOrbit j k = if i = j then 24 else 0
```

def projector [source]

Four rank-one matrices, so it sends any vector to a combination of the `v i`.

```
def projector : Matrix (Fin 22) (Fin 22) ℤ
```

lemma certificate [source]

An identity between `22 × 22` integer matrices, a finite check over `484` entries.

```
lemma certificate :
    orbitMatrix * (orbitMatrix - 32 • 1) * (orbitMatrix - 16 • 1)
      * (orbitMatrix * orbitMatrix - 44 • orbitMatrix + 192 • 1) = 393216 • projector
```

lemma projector_mulVec [source]

The integer projector matrix acts on invariant orbit coordinates by multiplication by `24`. The `24` is the normalization of `projector`: each factor of the certificate acts on such a vector as a scalar, `M` by `48`, `M - z` by `48 - z` and the quadratic factor by `48 ^ 2 - 44 * 48 + 192 = 384`, so the left-hand side scales it by `48 * 32 * 16 * 384 = 9437184`, and dividing by the `393216` on the right leaves `24`.

```
lemma projector_mulVec {c : (Fin 4 → Fin 1 ⊕ Fin 3) → ℂ} (hc : IsInvariantCoeff c) :
    projector.map (Int.cast : ℤ → ℂ) *ᵥ (fun k => c (orbitRep k))
      = (24 : ℂ) • fun k => c (orbitRep k)
```

lemma orbitCoord_eq [source]

`24` times an orbit coordinate of an invariant tensor, read entry by entry off the previous lemma: `projector` is built from the columns `contractionOrbit i` and the rows `contractionWeight i`.

```
lemma orbitCoord_eq {c : (Fin 4 → Fin 1 ⊕ Fin 3) → ℂ} (hc : IsInvariantCoeff c)
    (k : Fin 22) :
    24 * c (orbitRep k)
      = ∑ i, (contractionOrbit i k : ℂ) * ∑ l, (contractionWeight i l : ℂ) * c (orbitRep l)
```

lemma exists_eq_sum [source]

An invariant coefficient tensor is a combination of the four.

```
lemma exists_eq_sum {c : (Fin 4 → Fin 1 ⊕ Fin 3) → ℂ} (hc : IsInvariantCoeff c) :
    ∃ a : Fin 4 → ℂ, c = fun d => ∑ i, a i * ((contractionCoeff i d : ℤ) : ℂ)
```

lemma reducesInvariantsTo_span_contractionTensor [source]

The invariants of the range of `f` reduce to the span of the images of the four invariant tensors.

```
lemma reducesInvariantsTo_span_contractionTensor (hf : IsLorentzCovariant 4 B repLorentz f) :
    ReducesInvariantsTo (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f)
      (Submodule.span ℂ (Set.range fun i => f (contractionTensor i)))
```

lemma mem_range_sup_invariant_iff [source]

A vector of `LinearMap.range f ⊔ S`, for `S` a Lorentz-stable submodule, is invariant exactly when it is a combination of the images of the four invariant tensors plus an invariant of `S`. `hS` is used only left to right.

```
lemma mem_range_sup_invariant_iff (hf : IsLorentzCovariant 4 B repLorentz f) (x : B)
    (S : Submodule ℂ B) (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) :
    (x ∈ LinearMap.range f ⊔ S ∧ ∀ g : SL(2,ℂ), repLorentz g x = x)
      ↔ ∃ a : Fin 4 → ℂ, ∃ y ∈ S, x = ∑ i, a i • f (contractionTensor i) + y
        ∧ ∀ g : SL(2,ℂ), repLorentz g y = y
```

2.608 PhyslibAlpha.Relativity.LorentzGroup.Invariants.RankOne

PhyslibAlpha/Relativity/LorentzGroup/Invariants/RankOne.lean on GitHub.

def Transverse [source]

A direction transverse to the boost along axis `i`: neither time nor the axis.

```
def Transverse (i : Fin 3) (μ : Fin 1 ⊕ Fin 3) : Prop
```

instance Decidable (Transverse i μ) [source]

```
instance (i : Fin 3) (μ : Fin 1 ⊕ Fin 3) : Decidable (Transverse i μ)
```

lemma prod_lightConeCoeffInv_eq_zero [source]

A non-transverse direction has no weight-zero light-cone coefficient.

```
lemma prod_lightConeCoeffInv_eq_zero {i : Fin 3} {d : Fin 1 → Fin 1 ⊕ Fin 3}
    (hd : ¬ Transverse i (d 0)) {κ : Fin 1 → Fin 4}
    (hκ : ∑ s, lightConeWeight (κ s) = 0) :
    ∏ s, lightConeCoeffInv i (d s) (κ s) = 0
```

lemma eq_zero_of_not_transverse [source]

An invariant coefficient tensor vanishes off the two directions transverse to the axis.

```
lemma eq_zero_of_not_transverse {c : (Fin 1 → Fin 1 ⊕ Fin 3) → ℂ} (hc : IsInvariantCoeff c)
    (i : Fin 3) {d : Fin 1 → Fin 1 ⊕ Fin 3} (hd : ¬ Transverse i (d 0)) : c d = 0
```

lemma not_transverse_all [source]

No direction is transverse to all three axes at once, a finite check.

```
lemma not_transverse_all (μ : Fin 1 ⊕ Fin 3) :
    ¬(Transverse 0 μ ∧ Transverse 1 μ ∧ Transverse 2 μ)
```

lemma eq_zero_of_isInvariantCoeff [source]

An invariant coefficient tensor is zero.

```
lemma eq_zero_of_isInvariantCoeff {c : (Fin 1 → Fin 1 ⊕ Fin 3) → ℂ}
    (hc : IsInvariantCoeff c) : c = 0
```

lemma eq_zero_of_invariant [source]

Every Lorentz invariant in the range of `f` is zero: one index carry no invariant contraction.

```
lemma eq_zero_of_invariant (hf : IsLorentzCovariant 1 B repLorentz f) {x : B}
    (hx : x ∈ LinearMap.range f) (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x = 0
```

lemma reducesInvariantsTo_bot [source]

The range of `f` reduces to `⊥`: a Lorentz invariant of `LinearMap.range f ⊔ S`, for `S` a Lorentz-stable submodule, lies in `S`.

```
lemma reducesInvariantsTo_bot (hf : IsLorentzCovariant 1 B repLorentz f) :
    ReducesInvariantsTo (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f) ⊥
```

2.609 PhyslibAlpha.Relativity.LorentzGroup.Invariants.RankThree

PhyslibAlpha/Relativity/LorentzGroup/Invariants/RankThree.lean on GitHub.

def lightConeSign [source]

The sign the half turn about an axis gives each light-cone direction of that axis: `1` on the two of weight `±2`, `-1` on the two transverse ones.

```
def lightConeSign (κ : Fin 4) : ℤ
```

lemma sum_halfTurn_lightConeCoeff [source]

The half turn about the axis `i` acts on each light-cone direction along that axis by its sign.

```
lemma sum_halfTurn_lightConeCoeff (i : Fin 3) (κ : Fin 4) (ν : Fin 1 ⊕ Fin 3) :
    ∑ μ : Fin 1 ⊕ Fin 3, lightConeCoeff i κ μ *
        (((SL2C.toLorentzGroup (SL2C.halfTurn i)).1 ν μ : ℝ) : ℂ)
      = ((lightConeSign κ : ℤ) : ℂ) * lightConeCoeff i κ ν
```

lemma sum_prod_halfTurn_lightConeCoeff [source]

The scalar behind the action of the half turn on a light-cone multi-index: the half turn acts slot by slot, so the product of the per-slot signs factors out.

```
lemma sum_prod_halfTurn_lightConeCoeff (i : Fin 3) {n : ℕ} (c : Fin n → Fin 4)
    (a : Fin n → Fin 1 ⊕ Fin 3) :
    ∑ d : Fin n → Fin 1 ⊕ Fin 3, (∏ j, lightConeCoeff i (c j) (d j)) *
        (∏ j, (((SL2C.toLorentzGroup (SL2C.halfTurn i)).1 (a j) (d j) : ℝ) : ℂ))
      = ((∏ j, lightConeSign (c j) : ℤ) : ℂ) * ∏ j, lightConeCoeff i (c j) (a j)
```

lemma prod_lightConeSign_of_sum_lightConeWeight_eq_zero [source]

A light-cone multi-index of three slots and total boost weight zero has an odd number of transverse slots, so the half turn acts on it by -1.

```
lemma prod_lightConeSign_of_sum_lightConeWeight_eq_zero (c : Fin 3 → Fin 4)
    (hc : (∑ j, lightConeWeight (c j)) = 0) : ∏ j, lightConeSign (c j) = -1
```

lemma lightConeComponent_act_halfTurn [source]

The half turn multiplies a light-cone component by the product of the signs of its slots.

```
lemma lightConeComponent_act_halfTurn {n : ℕ} (i : Fin 3)
    (c : (Fin n → Fin 1 ⊕ Fin 3) → ℂ) (κ : Fin n → Fin 4) :
    lightConeComponent i (act (SL2C.toLorentzGroup (SL2C.halfTurn i)).1 c) κ
      = ((∏ s, lightConeSign (κ s) : ℤ) : ℂ) * lightConeComponent i c κ
```

lemma lightConeComponent_eq_zero_of_weight_zero [source]

An invariant coefficient tensor has no weight-zero light-cone component either, the half turn negating those.

```
lemma lightConeComponent_eq_zero_of_weight_zero {c : (Fin 3 → Fin 1 ⊕ Fin 3) → ℂ}
    (hc : IsInvariantCoeff c) (i : Fin 3) {κ : Fin 3 → Fin 4}
    (hκ : ∑ s, lightConeWeight (κ s) = 0) : lightConeComponent i c κ = 0
```

lemma eq_zero_of_isInvariantCoeff [source]

An invariant coefficient tensor is zero: no light-cone component of it survives.

```
lemma eq_zero_of_isInvariantCoeff {c : (Fin 3 → Fin 1 ⊕ Fin 3) → ℂ}
    (hc : IsInvariantCoeff c) : c = 0
```

lemma eq_zero_of_invariant [source]

Every Lorentz invariant in the range of `f` is zero: three indices carry no invariant contraction.

```
lemma eq_zero_of_invariant (hf : IsLorentzCovariant 3 B repLorentz f) {x : B}
    (hx : x ∈ LinearMap.range f) (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) : x = 0
```

lemma reducesInvariantsTo_bot [source]

The range of `f` reduces to `⊥`: a Lorentz invariant of `LinearMap.range f ⊔ S`, for `S` a Lorentz-stable submodule, lies in `S`.

```
lemma reducesInvariantsTo_bot (hf : IsLorentzCovariant 3 B repLorentz f) :
    ReducesInvariantsTo (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f) ⊥
```

2.610 PhyslibAlpha.Relativity.LorentzGroup.Invariants.RankTwo

PhyslibAlpha/Relativity/LorentzGroup/Invariants/RankTwo.lean on GitHub.

def metric [source]

The metric `η` on two contravariant indices, with its colours relabelled from `![.up, .up]` to `fun _ => .up`.

```
noncomputable def metric : ℂT(fun _ : Fin 2 => Color.up)
```

lemma metric_invariant [source]

The metric is Lorentz invariant.

```
lemma metric_invariant (g : SL(2,ℂ)) : g • metric = metric
```

lemma coeffEquiv_symm_minkowskiMatrixZ [source]

The coefficient tensor of the metric is the Minkowski metric.

```
lemma coeffEquiv_symm_minkowskiMatrixZ :
    (coeffEquiv 2).symm (fun d => ((minkowskiMatrixZ (d 0) (d 1) : ℤ) : ℂ)) = metric
```

lemma ofComponents_metric [source]

`ofComponents T` sends the metric to the contraction `η_{μν} T^{μν}`.

```
lemma ofComponents_metric (T : (Fin 2 → Fin 1 ⊕ Fin 3) → B) :
    ofComponents T metric
      = ∑ d : Fin 2 → Fin 1 ⊕ Fin 3, ((minkowskiMatrixZ (d 0) (d 1) : ℤ) : ℂ) • T d
```

lemma exists_halfTurnSign_mul_ne_one [source]

Two distinct directions are told apart by the half turn about some axis: it keeps one and negates the other, a finite check.

```
lemma exists_halfTurnSign_mul_ne_one :
    ∀ μ ν : Fin 1 ⊕ Fin 3, μ ≠ ν → ∃ k, halfTurnSign k μ * halfTurnSign k ν ≠ 1
```

lemma eq_zero_of_ne [source]

An invariant coefficient tensor has no off-diagonal coefficients: for `μ ≠ ν` some half turn multiplies `c_{μν}` by `-1`.

```
lemma eq_zero_of_ne {c : (Fin 2 → Fin 1 ⊕ Fin 3) → ℂ} (hc : IsInvariantCoeff c)
    {d : Fin 2 → Fin 1 ⊕ Fin 3} (hd : d 0 ≠ d 1) : c d = 0
```

lemma apply_inr_inr_eq [source]

The three spatial diagonal coefficients of an invariant coefficient tensor agree: the cyclic rotation carries `c_xx` to `c_yy` to `c_zz`.

```
lemma apply_inr_inr_eq {c : (Fin 2 → Fin 1 ⊕ Fin 3) → ℂ} (hc : IsInvariantCoeff c)
    (j : Fin 3) : c ![Sum.inr j, Sum.inr j] = c ![Sum.inr 2, Sum.inr 2]
```

lemma apply_inl_inl_add_apply_inr_inr [source]

The time and spatial diagonal coefficients of an invariant coefficient tensor are opposite. The boost along `z` scales the light-cone component along `D₀ - D_z` in both slots by `t⁴`, so that component, `c_tt - c_tz - c_zt + c_zz`, vanishes, and the mixed terms are `0`.

```
lemma apply_inl_inl_add_apply_inr_inr {c : (Fin 2 → Fin 1 ⊕ Fin 3) → ℂ}
    (hc : IsInvariantCoeff c) : c ![Sum.inl 0, Sum.inl 0] + c ![Sum.inr 2, Sum.inr 2] = 0
```

lemma eq_smul_minkowskiMatrixZ [source]

An invariant coefficient tensor is `c_tt` times the Minkowski metric.

```
lemma eq_smul_minkowskiMatrixZ {c : (Fin 2 → Fin 1 ⊕ Fin 3) → ℂ} (hc : IsInvariantCoeff c)
    (d : Fin 2 → Fin 1 ⊕ Fin 3) :
    c d = c ![Sum.inl 0, Sum.inl 0] * ((minkowskiMatrixZ (d 0) (d 1) : ℤ) : ℂ)
```

lemma reducesInvariantsTo_span_metric [source]

The invariants of the range of `f` reduce to the line through the image `f metric` of the metric: a Lorentz invariant of `LinearMap.range f ⊔ S`, for `S` a Lorentz-stable submodule, is a multiple of `f metric` plus an element of `S`.

```
lemma reducesInvariantsTo_span_metric (hf : IsLorentzCovariant 2 B repLorentz f) :
    ReducesInvariantsTo (fun g : SL(2,ℂ) => repLorentz g) (LinearMap.range f)
      (ℂ • f metric)
```

lemma exists_smul_map_metric_add_of_invariant [source]

Every Lorentz invariant of `LinearMap.range f ⊔ S`, for `S` a Lorentz-stable submodule, is a multiple of the image `f metric` of the metric plus an element of `S`.

```
lemma exists_smul_map_metric_add_of_invariant (hf : IsLorentzCovariant 2 B repLorentz f)
    (S : Submodule ℂ B) (hS : ∀ g : SL(2,ℂ), ∀ y ∈ S, repLorentz g y ∈ S) {x : B}
    (hx : x ∈ LinearMap.range f ⊔ S) (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) :
    ∃ a : ℂ, ∃ y ∈ S, x = a • f metric + y
```

lemma exists_smul_map_metric_of_invariant [source]

Every Lorentz invariant in the range of `f` is a multiple of the image `f metric` of the metric.

```
lemma exists_smul_map_metric_of_invariant (hf : IsLorentzCovariant 2 B repLorentz f) {x : B}
    (hx : x ∈ LinearMap.range f) (hinv : ∀ g : SL(2,ℂ), repLorentz g x = x) :
    ∃ a : ℂ, x = a • f metric
```

2.611 PhyslibAlpha.Relativity.LorentzMix

PhyslibAlpha/Relativity/LorentzMix.lean on GitHub.

def lorentzMixStep [source]

One peeling step of the Lorentz mixing: the direction `a` is removed from the multiset index of the family and put back as every direction `b`, weighted by `Λ_{b a}`.

```
noncomputable def lorentzMixStep (Λ : SL(2,ℂ)) (a : Fin 1 ⊕ Fin 3)
    (G : Multiset (Fin 1 ⊕ Fin 3) → M) : Multiset (Fin 1 ⊕ Fin 3) → M
```

instance LeftCommutative (lorentzMixStep (M [source]

Peeling two directions commutes, so the mixing is well defined on a multiset.

```
instance (Λ : SL(2,ℂ)) : LeftCommutative (lorentzMixStep (M := M) Λ) where
```

def lorentzMix [source]

The Lorentz mixing of a multiset-indexed family along a multiset `s` of directions: every direction of `s` is peeled and replaced by all directions, weighted by the corresponding column of the Lorentz matrix.

```
noncomputable def lorentzMix (Λ : SL(2,ℂ)) (G : Multiset (Fin 1 ⊕ Fin 3) → M)
    (s : Multiset (Fin 1 ⊕ Fin 3)) : Multiset (Fin 1 ⊕ Fin 3) → M
```

lemma lorentzMix_zero [source]

```
@[simp]
lemma lorentzMix_zero : lorentzMix Λ G 0 = G
```

lemma lorentzMix_cons_apply [source]

Mixing along `a ::ₘ s` peels `a` after mixing along `s`.

```
lemma lorentzMix_cons_apply (a : Fin 1 ⊕ Fin 3) (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ G (a ::ₘ s) t = ∑ b, L[Λ] b a • lorentzMix Λ G s (b ::ₘ t)
```

lemma lorentzMix_apply_add [source]

Evaluating a mixed family away from the empty multiset is mixing the translated family at the empty multiset.

```
lemma lorentzMix_apply_add (s : Multiset (Fin 1 ⊕ Fin 3)) :
    ∀ (G : Multiset (Fin 1 ⊕ Fin 3) → M) (t : Multiset (Fin 1 ⊕ Fin 3)),
      lorentzMix Λ G s t = lorentzMix Λ (fun r => G (r + t)) s 0
```

lemma lorentzMix_cons_zero [source]

Peeling at the empty multiset: the peeled direction is pushed into the family.

```
lemma lorentzMix_cons_zero (a : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ G (a ::ₘ s) 0 = ∑ b, L[Λ] b a • lorentzMix Λ (fun t => G (b ::ₘ t)) s 0
```

lemma lorentzMix_map [source]

The mixing operator commutes with any linear map applied to the values.

```
lemma lorentzMix_map (Φ : M →ₗ[ℂ] N) (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    Φ (lorentzMix Λ G s t) = lorentzMix Λ (fun r => Φ (G r)) s t
```

lemma lorentzMix_smul_fam [source]

The mixing operator is homogeneous in the family.

```
lemma lorentzMix_smul_fam (c : ℂ) (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ (fun r => c • G r) s t = c • lorentzMix Λ G s t
```

lemma lorentzMix_add_fam [source]

The mixing operator is additive in the family.

```
lemma lorentzMix_add_fam (G₁ G₂ : Multiset (Fin 1 ⊕ Fin 3) → M)
    (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ (fun r => G₁ r + G₂ r) s t = lorentzMix Λ G₁ s t + lorentzMix Λ G₂ s t
```

lemma lorentzMix_sum_fam [source]

The mixing operator commutes with finite sums of families.

```
lemma lorentzMix_sum_fam {ι : Type*} [Fintype ι] (H : ι → Multiset (Fin 1 ⊕ Fin 3) → M)
    (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ (fun r => ∑ i, H i r) s t = ∑ i, lorentzMix Λ (H i) s t
```

lemma lorentzMix_ofFn [source]

The mixing operator agrees with the tuple form of the Lorentz law: along an ordered tuple of directions it is the sum over all tuples with one Lorentz matrix factor per slot.

```
lemma lorentzMix_ofFn {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) (t : Multiset (Fin 1 ⊕ Fin 3)) :
    lorentzMix Λ G (List.ofFn l) t =
      ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
        (∏ i, L[Λ] (p i) (l i)) • G ((List.ofFn p : List (Fin 1 ⊕ Fin 3)) + t)
```

def derivConv [source]

The Leibniz convolution of two families of derivative symbols: the sum over the splittings of the multiset of the products of the two symbols.

```
noncomputable def derivConv (f g : Multiset (Fin 1 ⊕ Fin 3) → B)
    (s : Multiset (Fin 1 ⊕ Fin 3)) : B
```

lemma derivConv_cons [source]

One derivative peeled off a convolution lands on one factor or the other.

```
lemma derivConv_cons (f g : Multiset (Fin 1 ⊕ Fin 3) → B) (a : Fin 1 ⊕ Fin 3)
    (s : Multiset (Fin 1 ⊕ Fin 3)) :
    derivConv f g (a ::ₘ s) =
      derivConv f (fun r => g (a ::ₘ r)) s + derivConv (fun r => f (a ::ₘ r)) g s
```

lemma derivConv_sum_right [source]

The convolution is linear in its right-hand family.

```
lemma derivConv_sum_right {ι : Type*} [Fintype ι] (f : Multiset (Fin 1 ⊕ Fin 3) → B)
    (c : ι → ℂ) (g : ι → Multiset (Fin 1 ⊕ Fin 3) → B) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    derivConv f (fun r => ∑ i, c i • g i r) s = ∑ i, c i • derivConv f (g i) s
```

lemma derivConv_sum_left [source]

The convolution is linear in its left-hand family.

```
lemma derivConv_sum_left {ι : Type*} [Fintype ι] (g : Multiset (Fin 1 ⊕ Fin 3) → B)
    (c : ι → ℂ) (f : ι → Multiset (Fin 1 ⊕ Fin 3) → B) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    derivConv (fun r => ∑ i, c i • f i r) g s = ∑ i, c i • derivConv (f i) g s
```

lemma lorentzMix_derivConv [source]

The Lorentz mixing operator is a morphism for the Leibniz convolution: mixing the two factors separately and convolving is the same as convolving and then mixing.

```
lemma lorentzMix_derivConv (Λ : SL(2,ℂ)) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    ∀ (f g : Multiset (Fin 1 ⊕ Fin 3) → B),
      derivConv (fun x => lorentzMix Λ f x 0) (fun y => lorentzMix Λ g y 0) s =
        lorentzMix Λ (derivConv f g) s 0
```

lemma repLorentz_derivConv [source]

The Lorentz law of a Leibniz convolution: the mixing operator is a morphism for the convolution, so a convolution of two families with Lorentz laws has one too.

```
lemma repLorentz_derivConv {repLorentz : Representation ℂ SL(2,ℂ) B}
    (hmul : ∀ (Λ : SL(2,ℂ)) (b₁ b₂ : B),
      repLorentz Λ (b₁ * b₂) = repLorentz Λ b₁ * repLorentz Λ b₂)
    (Λ : SL(2,ℂ)) (f f' g g' : Multiset (Fin 1 ⊕ Fin 3) → B)
    (hf : ∀ x, repLorentz Λ (f x) = lorentzMix Λ f' x 0)
    (hg : ∀ y, repLorentz Λ (g y) = lorentzMix Λ g' y 0) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    repLorentz Λ (derivConv f g s) = lorentzMix Λ (derivConv f' g') s 0
```

lemma repLorentz_tower [source]

The Lorentz law of a covariant tower `T` built by the step `hstep` from a correction `C`, transforming into a tower `T'` built by the same step: the seed of `T` transforms into the seed of `T'` with the value index twisted by `τ` (`hzero`), and the correction is Lorentz covariant (`hC`), linear in the family it corrects (`hClin`) and lets the twist through (`hCτ`). Then the covariant slots mix by their own columns of the Lorentz matrix, the plain slots by `lorentzMix`, and the value index by `τ`.

```
lemma repLorentz_tower {K : Type} [Field K] {W : Type} [AddCommGroup W] [Module K W]
    [Module K B] [SMulCommClass K ℂ B] (Λ : SL(2,ℂ))
    (T T' : (n : ℕ) → (Fin n → (Fin 1 ⊕ Fin 3)) → Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B)
    (C : (Fin 1 ⊕ Fin 3) → (Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B) →
      Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B)
    (τ : W →ₗ[K] W)
    (hstep : ∀ (n : ℕ) (l : Fin (n + 1) → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3)),
      T (n + 1) l s = T n (fun i => l i.succ) (l 0 ::ₘ s) + C (l 0) (T n fun i => l i.succ) s)
    (hstep' : ∀ (n : ℕ) (l : Fin (n + 1) → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3)),
      T' (n + 1) l s = T' n (fun i => l i.succ) (l 0 ::ₘ s) + C (l 0) (T' n fun i => l i.succ)
    s)
```

```
    (hzero : ∀ (l : Fin 0 → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : W),
      repLorentz Λ (T 0 l s φ) = lorentzMix Λ (fun t => T' 0 l t (τ φ)) s 0)
    (hC : ∀ (ρ : Fin 1 ⊕ Fin 3) (G G' : Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B),
      (∀ y χ, repLorentz Λ (G y χ) = lorentzMix Λ (fun t => G' t χ) y 0) →
      ∀ (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : W), repLorentz Λ (C ρ G s φ) =
        ∑ a, L[Λ] a ρ • lorentzMix Λ (fun t => C a G' t φ) s 0)
    (hClin : ∀ (ρ : Fin 1 ⊕ Fin 3) {ι : Type} [Fintype ι] (c : ι → ℂ)
      (G : ι → Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B) (s : Multiset (Fin 1 ⊕ Fin 3))
      (φ : W), C ρ (fun t => ∑ i, c i • G i t) s φ = ∑ i, c i • C ρ (G i) s φ)
    (hCτ : ∀ (ρ : Fin 1 ⊕ Fin 3) (G : Multiset (Fin 1 ⊕ Fin 3) → W →ₗ[K] B)
      (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : W),
      C ρ (fun t => G t ∘ₗ τ) s φ = C ρ G s (τ φ)) :
    ∀ (n : ℕ) (l : Fin n → (Fin 1 ⊕ Fin 3)) (s : Multiset (Fin 1 ⊕ Fin 3)) (φ : W),
      repLorentz Λ (T n l s φ) =
        ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
          (∏ i, L[Λ] (p i) (l i)) • lorentzMix Λ (fun t => T' n p t (τ φ)) s 0
```

2.612 PhyslibAlpha.Relativity.MinkowskiMatrix

PhyslibAlpha/Relativity/MinkowskiMatrix.lean on GitHub.

def minkowskiMatrixZ [source]

The Minkowski matrix over the integers, `diag(1, -1, -1, ...)` in ℤ. Its cast to ℝ is `minkowskiMatrix`, by `minkowskiMatrixZ.cast_apply`.

```
def minkowskiMatrixZ {d : ℕ} : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℤ
```

lemma as_diagonal [source]

The integer Minkowski matrix as a diagonal matrix.

```
lemma as_diagonal : @minkowskiMatrixZ d = diagonal (Sum.elim 1 (-1))
```

lemma inl_0_inl_0 [source]

The time-time component of the integer Minkowski matrix is `1`.

```
@[simp]
lemma inl_0_inl_0 : @minkowskiMatrixZ d (Sum.inl 0) (Sum.inl 0) = 1
```

lemma inr_i_inr_i [source]

The space diagonal components of the integer Minkowski matrix are `-1`.

```
@[simp]
lemma inr_i_inr_i (i : Fin d) : @minkowskiMatrixZ d (Sum.inr i) (Sum.inr i) = -1
```

lemma off_diag_zero [source]

The off-diagonal components of the integer Minkowski matrix vanish.

```
@[simp]
lemma off_diag_zero {μ ν : Fin 1 ⊕ Fin d} (h : μ ≠ ν) : @minkowskiMatrixZ d μ ν = 0
```

lemma comm [source]

The integer Minkowski matrix is symmetric, being diagonal.

```
lemma comm (μ ν : Fin 1 ⊕ Fin d) : @minkowskiMatrixZ d μ ν = minkowskiMatrixZ ν μ
```

lemma cast_apply [source]

The integer Minkowski matrix casts to the Minkowski matrix, entry by entry.

```
lemma cast_apply (μ ν : Fin 1 ⊕ Fin d) :
    ((minkowskiMatrixZ μ ν : ℤ) : ℝ) = minkowskiMatrix μ ν
```

2.613 PhyslibAlpha.Relativity.PauliMatrices.Anticommuting

PhyslibAlpha/Relativity/PauliMatrices/Anticommuting.lean on GitHub.

def IsAnticommuting [source]

A family of `2 × 2` matrices with `τ_a τ_b + τ_b τ_a = 2 δ_ab 1`.

```
def IsAnticommuting {ι : Type*} [DecidableEq ι] (τ : ι → Matrix (Fin 2) (Fin 2) ℂ) : Prop
```

lemma mul_self [source]

```
lemma mul_self (a : ι) : τ a * τ a = 1
```

lemma mul_comm_of_ne [source]

```
lemma mul_comm_of_ne {a b : ι} (h : a ≠ b) : τ b * τ a = -(τ a * τ b)
```

lemma trace_mul [source]

The trace pairs the members orthogonally: `tr (τ_a τ_b) = 2 δ_ab`.

```
lemma trace_mul (a b : ι) : (τ a * τ b).trace = if a = b then 2 else 0
```

lemma trace_eq_zero [source]

In a family with at least two members, every member is traceless.

```
lemma trace_eq_zero {a b : ι} (hab : b ≠ a) : (τ a).trace = 0
```

lemma linearIndependent [source]

In a family with at least two members, the members together with `1` are linearly independent.

```
lemma linearIndependent [Fintype ι] [Nontrivial ι] :
    LinearIndependent ℂ (fun o : Option ι => o.elim (1 : Matrix (Fin 2) (Fin 2) ℂ) τ)
```

lemma card_le_three [source]

An anticommuting family of 2 × 2 matrices has at most three members.

```
lemma card_le_three [Fintype ι] : Fintype.card ι ≤ 3
```

lemma isAnticommuting_pauliMatrix [source]

The Pauli matrices σ1, σ2, σ3 are an anticommuting family.

```
lemma isAnticommuting_pauliMatrix : IsAnticommuting fun i : Fin 3 => pauliMatrix (Sum.inr i)
```

lemma isGreatest_card_isAnticommuting [source]

***The Pauli matrices are a maximal anticommuting family.** The largest anticommuting family of 2 × 2 matrices has three members.*

```
lemma isGreatest_card_isAnticommuting :
    IsGreatest {n : ℕ | ∃ τ : Fin n → Matrix (Fin 2) (Fin 2) ℂ, IsAnticommuting τ} 3
```

2.614 PhyslibAlpha.Relativity.PauliMatrices.SelfAdjoint

PhyslibAlpha/Relativity/PauliMatrices/SelfAdjoint.lean on GitHub.

def pauliLower [source]

The Pauli matrices with the vector index lowered by the Minkowski metric, σ_μ = η_{μμ} σ^μ, as plain matrices: the underlying matrices of pauliSelfAdjoint'.

```
def pauliLower (μ : Fin 1 ⊕ Fin 3) : Matrix (Fin 2) (Fin 2) ℂ
```

lemma pauliBasis'_coe [source]

The covariant Pauli matrices are the underlying matrices of the basis pauliBasis'.

```
lemma pauliBasis'_coe (μ : Fin 1 ⊕ Fin 3) : (pauliBasis' μ).1 = pauliLower μ
```

lemma pauliLower_eq_smul [source]

Lowering the vector index multiplies by the diagonal entry of the metric.

```
lemma pauliLower_eq_smul (μ : Fin 1 ⊕ Fin 3) :
    pauliLower μ = ((minkowskiMatrixZ μ μ : ℤ) : ℂ) • pauliMatrix μ
```

lemma sum_pauliLower_mul_pauliLower [source]

The Fierz completeness relation: the covariant Pauli matrices span the 2 × 2 matrices, with the trace pairing as the duality and normalisation 2.

```
lemma sum_pauliLower_mul_pauliLower (α α' β β' : Fin 2) :
    ∑ ρ : Fin 1 ⊕ Fin 3, pauliLower ρ β' β * pauliLower ρ α α'
      = 2 * ((if α = β then 1 else 0) * (if α' = β' then 1 else 0))
```

2.615 PhyslibAlpha.Relativity.PauliMatrices.ToTensor

PhyslibAlpha/Relativity/PauliMatrices/ToTensor.lean on GitHub.

def indexBasis [source]

The basis of `ℂT[.up, .upL, .upR]` indexed by `(Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2` through `indexEquiv`: `indexBasis (μ, α, β) = e_μ ⊗ e_α ⊗ e_β`.

```
noncomputable def indexBasis :
    Basis ((Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2) ℂ ℂT[.up, .upL, .upR]
```

lemma indexBasis_apply [source]

```
lemma indexBasis_apply (d : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2) :
    indexBasis d = Tensor.basis ![Color.up, Color.upL, Color.upR] (indexEquiv.symm d)
```

lemma toTensor_eq_sum_indexBasis [source]

The Pauli tensor as a sum over its components, `σ^^^ = ∑ σ^μ_{α β} e_μ ⊗ e_α ⊗ e_β`.

```
lemma toTensor_eq_sum_indexBasis : σ^^^ = ∑ d : (Fin 1 ⊕ Fin 3) × Fin 2 × Fin 2,
    pauliMatrix d.1 d.2.1 d.2.2 • indexBasis d
```

2.616 PhyslibAlpha.Relativity.SL2C.AxisRotations

PhyslibAlpha/Relativity/SL2C/AxisRotations.lean on GitHub.

def cycDir [source]

The cyclic permutation of Lorentz direction labels: time is fixed and the spatial directions rotate as `x → y → z → x`.

```
def cycDir : Fin 1 ⊕ Fin 3 → Fin 1 ⊕ Fin 3
```

lemma cycDir_comp_two [source]

Composing the cyclic permutation with a two-slot index vector rotates both entries.

```
lemma cycDir_comp_two (μ ν : Fin 1 ⊕ Fin 3) :
    (fun j => cycDir (![μ, ν] j)) = ![cycDir μ, cycDir ν]
```

lemma cycDir_comp_one [source]

Composing the cyclic permutation with a one-slot index vector rotates its entry.

```
lemma cycDir_comp_one (μ : Fin 1 ⊕ Fin 3) :
    (fun j => cycDir (![μ] j)) = ![cycDir μ]
```

lemma cycDir_comp_nil [source]

Composing the cyclic permutation with the empty index vector is the empty vector.

```
lemma cycDir_comp_nil : (fun j : Fin 0 => cycDir (![] j)) = ![]
```

lemma cycDir_cycDir_cycDir [source]

The cyclic permutation of Lorentz direction labels has order three.

```
lemma cycDir_cycDir_cycDir (μ : Fin 1 ⊕ Fin 3) : cycDir (cycDir (cycDir μ)) = μ
```

lemma cycDir_injective [source]

The cyclic permutation of Lorentz direction labels is injective: applying it twice more returns the original label.

```
lemma cycDir_injective : Function.Injective cycDir
```

lemma cycDir_orbit_distinct [source]

An index not fixed by the rotation has three distinct rotations.

```
lemma cycDir_orbit_distinct {ι : Type*} (d : ι → Fin 1 ⊕ Fin 3)
    (hd : (fun s => cycDir (d s)) ≠ d) :
    (fun s => cycDir (cycDir (d s))) ≠ d
      ∧ (fun s => cycDir (cycDir (d s))) ≠ (fun s => cycDir (d s))
```

def rotationCycle [source]

The cyclic rotation `x → y → z → x` in `SL(2,ℂ)`, realized as the rotation by `2π/3` about the diagonal spatial axis.

```
noncomputable def rotationCycle : SL(2,ℂ)
```

lemma toLorentzGroup_rotationCycle_apply [source]

The Lorentz matrix of `rotationCycle`: it is the permutation matrix associated with `cycDir`.

```
lemma toLorentzGroup_rotationCycle_apply (a b : Fin 1 ⊕ Fin 3) :
    (toLorentzGroup rotationCycle).1 a b = if a = cycDir b then 1 else 0
```

def halfTurnSign [source]

The sign the half turn about the axis `k` gives a direction: `1` on time and on the axis, `-1` on the two transverse directions.

```
def halfTurnSign (k : Fin 3) (μ : Fin 1 ⊕ Fin 3) : ℤ
```

def halfTurn [source]

The half turn about the axis `k`, the rotation by `π` about it, as the element `-i σ_k` of `SL(2,ℂ)`. The other lift `i σ_k` is `-halfTurn k`.

```
noncomputable def halfTurn : Fin 3 → SL(2,ℂ)
  | 0 => ⟨!![0, -Complex.I; -Complex.I, 0], by
      rw [Matrix.det_fin_two_of]
      simp [Complex.I_mul_I]⟩
  | 1 => ⟨!![0, -1; 1, 0], by
      rw [Matrix.det_fin_two_of]
```

```
      simp⟩
  | 2 => ⟨!![-Complex.I, 0; 0, Complex.I], by
      rw [Matrix.det_fin_two_of]
      simp [Complex.I_mul_I]⟩
```

lemma toLorentzGroup_halfTurn_apply [source]

The Lorentz matrix of the half turn about the axis `k` is diagonal, with the signs `halfTurnSign k`: it fixes time and the axis and negates the two transverse directions.

```
lemma toLorentzGroup_halfTurn_apply (k : Fin 3) (a b : Fin 1 ⊕ Fin 3) :
    (toLorentzGroup (halfTurn k)).1 a b = if a = b then (halfTurnSign k a : ℝ) else 0
```

lemma toLorentzGroup_halfTurn_symm [source]

The Lorentz matrix of the half turn is diagonal, hence symmetric.

```
lemma toLorentzGroup_halfTurn_symm (k : Fin 3) (a b : Fin 1 ⊕ Fin 3) :
    (toLorentzGroup (halfTurn k)).1 a b = (toLorentzGroup (halfTurn k)).1 b a
```

2.617 PhyslibAlpha.Relativity.SL2C.Basic

PhyslibAlpha/Relativity/SL2C/Basic.lean on GitHub.

abbrev conjHom [source]

Entrywise complex conjugation as a monoid endomorphism of `SL(2,ℂ)`: Mathlib's `SpecialLinearGroup.map` along `starRingEnd ℂ`.

```
abbrev conjHom : SL(2,ℂ) →* SL(2,ℂ)
```

lemma conjHom_coe [source]

```
lemma conjHom_coe (g : SL(2,ℂ)) : (conjHom g).1 = g.1.map star
```

lemma conjHom_involutive [source]

Conjugation is an involution, hence surjective.

```
lemma conjHom_involutive : Function.Involutive conjHom
```

lemma conjHom_inv_conjTranspose [source]

Conjugating the group argument undoes the conjugation of the entries: the inverse conjugate transpose at `conjHom g` is the inverse transpose at `g`.

```
lemma conjHom_inv_conjTranspose (g : SL(2,ℂ)) : (((conjHom g).1)⁻¹)ᴴ = (g.1⁻¹)ᵀ
```

lemma toLorentzGroup_neg_one [source]

***The centre of `SL(2, ℂ)` covers the identity Lorentz transformation.** The covering map sandwiches, `A ↦ M A Mᴴ`, so the two signs of `-1` cancel. Together with `_root_.map_one` this says that the covering map is two-to-one.*

```
lemma toLorentzGroup_neg_one : toLorentzGroup (-1) = 1
```

lemma sum_pauliLower_mul_sl2c [source]

The intertwining identity `toSelfAdjointMap_basis` read entrywise: sandwiching the covariant Pauli matrix `σ_μ` between `g` and g^H mixes the covariant Pauli matrices by the column `μ` of the Lorentz matrix of `g`, the summed Lorentz index first.

```
lemma sum_pauliLower_mul_sl2c (g : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) (β β' : Fin 2) :
    ∑ p : Fin 2 × Fin 2, PauliMatrix.pauliLower μ p.1 p.2 * (g.1 β p.1 * star (g.1 β' p.2))
      = ∑ ν : Fin 1 ⊕ Fin 3, (((toLorentzGroup g).1 ν μ : ℝ) : ℂ)
        * PauliMatrix.pauliLower ν β β'
```

lemma toLorentzGroup_conjTranspose [source]

The covering map intertwines conjugate transposition with matrix transposition: `L(M†) = L(M)`^{T}.

```
lemma toLorentzGroup_conjTranspose {M N : SL(2,ℂ)} (hN : N.1 = M.1^H) :
    (toLorentzGroup N).1 = (toLorentzGroup M).1^T
```

2.618 PhyslibAlpha.Relativity.Tensors.ComplexTensor.Vector.Basic

PhyslibAlpha/Relativity/Tensors/ComplexTensor/Vector/Basic.lean on GitHub.

lemma SL2CRep_dual_dualBasis [source]

The dual of the complex covector representation on the dual basis: dual covectors transform contravariantly, by the columns of the (complexified) Lorentz matrix. The complex analogue of `Lorentz.CoVector.sl2Rep_dual_dualBasis`.

```
lemma ComplexCoVector.SL2CRep_dual_dualBasis (Λ : SL(2,ℂ))
    (μ : Fin 1 ⊕ Fin 3) :
    Lorentz.ComplexCoVector.SL2CRep.dual Λ (Lorentz.ComplexCoVector.basis.dualBasis μ) =
      ∑ j, (((Lorentz.SL2C.toLorentzGroup Λ).1 j μ : ℝ) : ℂ) •
        Lorentz.ComplexCoVector.basis.dualBasis j
```

2.619 PhyslibAlpha.Relativity.Tensors.Equivariant

PhyslibAlpha/Relativity/Tensors/Equivariant.lean on GitHub.

structure IsEquivariant [source]

A linear map `f` from the tensors of the species `S` with index colors `c` to a representation `ρ` of the group of the species, which is equivariant: it intertwines the action of the group on tensors with `ρ`.

```
structure IsEquivariant (S : TensorSpecies k C G V basisIdx rep b) {n : ℕ} (c : Fin n → C)
    {B : Type*} [AddCommMonoid B] [Module k B] (ρ : Representation k G B)
    (f : S.Tensor c →ₗ[k] B) : Prop where
  equivariant : ∀ (g : G) (t : S.Tensor c), f (g • t) = ρ g (f t)
```

lemma smul_basis_eq_sum [source]

The action of `g` on a basis tensor: the coefficient of `e_ψ` in `g • e_φ` is the product over the indices of the matrix entries of `g` in the color of that index.

```
lemma smul_basis_eq_sum {n : ℕ} (c : Fin n → C) (g : G) (φ : ComponentIdx (S := S) c) :
    g • Tensor.basis (S := S) c φ
      = ∑ ψ : ComponentIdx (S := S) c,
        (∏ i, LinearMap.toMatrix (b (c i)) (b (c i)) (rep (c i) g) (ψ i) (φ i)) •
        Tensor.basis (S := S) c ψ
```

lemma basis_repr_smul [source]

The components of `g • t`: the matrix of products of the matrix entries of `g`, one factor per index, applied to the components of `t`.

```
lemma basis_repr_smul {n : ℕ} (c : Fin n → C) (g : G) (t : S.Tensor c)
    (φ : ComponentIdx (S := S) c) :
    (Tensor.basis c).repr (g • t) φ
      = ∑ ψ, (∏ i, LinearMap.toMatrix (b (c i)) (b (c i)) (rep (c i) g) (φ i) (ψ i)) *
        (Tensor.basis c).repr t ψ
```

lemma isStableUnder_range [source]

The range of an equivariant map is stable under the group.

```
lemma isStableUnder_range : IsStableUnder (fun g : G => ρ g) (LinearMap.range f)
```

lemma rep_map_of_invariant [source]

The image of an invariant tensor under an equivariant map is invariant.

```
lemma rep_map_of_invariant {t : S.Tensor c} (ht : ∀ g : G, g • t = t) (g : G) :
    ρ g (f t) = f t
```

lemma sum [source]

A sum of equivariant maps is equivariant.

```
lemma sum {ι : Type*} (s : Finset ι) {F : ι → S.Tensor c →ₗ[k] B}
    (hF : ∀ i ∈ s, S.IsEquivariant c ρ (F i)) : S.IsEquivariant c ρ (∑ i ∈ s, F i) where
```

lemma comp [source]

Composing with a linear map that intertwines `ρ` with `ρ'` keeps a map equivariant.

```
lemma comp {B' : Type*} [AddCommGroup B'] [Module k B'] {ρ' : Representation k G B'}
    (σ : B →ₗ[k] B') (hσ : ∀ (g : G) (y : B), σ (ρ g y) = ρ' g (σ y)) :
    S.IsEquivariant c ρ' (σ ∘ₗ f) where
```

lemma sub [source]

A difference of equivariant maps is equivariant.

```
lemma sub {f' : S.Tensor c →ₗ[k] B} (hf' : S.IsEquivariant c ρ f') :
    S.IsEquivariant c ρ (f - f') where
```

def IsAdjointClosed [source]

The colors `c` of a complex species are closed under adjoints when for every `g` some `g'` acts on each of them by the conjugate transpose of the matrix of `g`. For a unitary group `g' = g⁻¹` and orthonormal bases; for `SL(2,ℂ)` on the Weyl colors, `g' = g†`.

```
@[nolint unusedArguments]
def IsAdjointClosed (S : TensorSpecies ℂ C G V basisIdx rep b) {n : ℕ} (c : Fin n → C) :
    Prop
```

lemma exists_invariant_eq_of_mem_range [source]

An invariant in the range of `f` is the image of an invariant tensor, when the colors are closed under adjoints: the adjoint of the action of `g` on the coefficients is that of `g'`.

```
lemma exists_invariant_eq_of_mem_range (hc : S.IsAdjointClosed c) {x : B}
    (hx : x ∈ LinearMap.range f) (hinv : ∀ g : G, ρ g x = x) :
    ∃ t : S.Tensor c, (∀ g : G, g • t = t) ∧ f t = x
```

lemma exists_invariant_add_of_mem_sup [source]

An invariant of `LinearMap.range f ⊔ W`, for `W` a stable submodule, is the image of an invariant tensor plus an element of `W`.

```
lemma exists_invariant_add_of_mem_sup (hc : S.IsAdjointClosed c) (W : Submodule ℂ B)
    (hW : ∀ g : G, ∀ y ∈ W, ρ g y ∈ W) {x : B} (hx : x ∈ LinearMap.range f ⊔ W)
    (hinv : ∀ g : G, ρ g x = x) :
    ∃ t : S.Tensor c, (∀ g : G, g • t = t) ∧ ∃ y ∈ W, x = f t + y
```

lemma reducesInvariantsTo_map [source]

When every invariant tensor of `S.Tensor c` lies in `I`, the invariants of the range of `f` reduce to the image of `I`.

```
lemma reducesInvariantsTo_map (hc : S.IsAdjointClosed c) (I : Submodule ℂ (S.Tensor c))
    (hI : ∀ t : S.Tensor c, (∀ g : G, g • t = t) → t ∈ I) :
    ReducesInvariantsTo (fun g : G => ρ g) (LinearMap.range f) (I.map f)
```

lemma reducesInvariantsTo_bot [source]

When the only invariant tensor of `S.Tensor c` is zero, the range of `f` reduces to `⊥`.

```
lemma reducesInvariantsTo_bot (hc : S.IsAdjointClosed c)
    (hI : ∀ t : S.Tensor c, (∀ g : G, g • t = t) → t = 0) :
    ReducesInvariantsTo (fun g : G => ρ g) (LinearMap.range f) ⊥
```

lemma eq_zero_of_invariant [source]

When the only invariant tensor of `S.Tensor c` is zero, so is every invariant in the range of `f`.

```
lemma eq_zero_of_invariant (hc : S.IsAdjointClosed c)
    (hI : ∀ t : S.Tensor c, (∀ g : G, g • t = t) → t = 0) {x : B}
    (hx : x ∈ LinearMap.range f) (hinv : ∀ g : G, ρ g x = x) : x = 0
```

def invariantReductionToSpan [source]

When the invariant tensors of `S.Tensor c` are the multiples of one tensor `t₀`, the invariants of the range of `f` reduce to the span of `f t₀`.

```
noncomputable def invariantReductionToSpan (hc : S.IsAdjointClosed c) (t₀ : S.Tensor c)
    (ht₀ : ∀ g : G, g • t₀ = t₀)
    (hclass : ∀ t : S.Tensor c, (∀ g : G, g • t = t) → ∃ a : ℂ, t = a • t₀) :
    InvariantReductionToSpan (fun g : G => ρ g) (LinearMap.range f) where
```

def invariantReductionToSpanOfEq [source]

The reduction of `invariantReductionToSpan`, stated for a submodule `V` equal to the range of `f` and with the spanning vector given by any expression `v` for `f t₀`.

```
noncomputable def invariantReductionToSpanOfEq (hc : S.IsAdjointClosed c) (t₀ : S.Tensor c)
    (ht₀ : ∀ g : G, g • t₀ = t₀)
    (hclass : ∀ t : S.Tensor c, (∀ g : G, g • t = t) → ∃ a : ℂ, t = a • t₀)
    {V : Submodule ℂ B} (hV : LinearMap.range f = V) (v : B) (hv : f t₀ = v) :
    InvariantReductionToSpan (fun g : G => ρ g) V where
```

lemma isEquivariant_constr [source]

The linear map with prescribed values `T ψ` on the basis tensors is equivariant when the values are moved by `ρ` as the basis tensors are moved by the group.

```
lemma isEquivariant_constr {n : ℕ} {c : Fin n → C} {B : Type*} [AddCommGroup B] [Module k B]
    {ρ : Representation k G B} (T : ComponentIdx (S := S) c → B)
    (hT : ∀ (g : G) φ, ρ g (T φ)
      = ∑ ψ, (∏ i, LinearMap.toMatrix (b (c i)) (b (c i)) (rep (c i) g) (ψ i) (φ i)) • T ψ) :
    S.IsEquivariant c ρ ((Tensor.basis c).constr k T) where
```

def familyMap [source]

The linear map out of `S.Tensor c` sending the basis tensor with components `e.symm l` to `T l`, for a relabelling `e` of the component indices by `ι`.

```
noncomputable def familyMap (e : ComponentIdx (S := S) c ≃ ι) (T : ι → B) :
    S.Tensor c →[k] B
```

lemma familyMap_basis [source]

```
@[simp]
lemma familyMap_basis (e : ComponentIdx (S := S) c ≃ ι) (T : ι → B) (l : ι) :
    familyMap e T (Tensor.basis c (e.symm l)) = T l
```

lemma range_familyMap [source]

The range of `familyMap e T` is the span of the vectors `T l`.

```
lemma range_familyMap (e : ComponentIdx (S := S) c ≃ ι) (T : ι → B) :
    LinearMap.range (familyMap e T) = Submodule.span k (Set.range T)
```

lemma familyMap_mem_span [source]

The image of `familyMap e T` lies in the span of the vectors `T l`.

```
lemma familyMap_mem_span (e : ComponentIdx (S := S) c ≃ ι) (T : ι → B) (t : S.Tensor c) :
    familyMap e T t ∈ Submodule.span k (Set.range T)
```

lemma familyMap_sum [source]

`familyMap` of a sum of families is the sum of the maps.

```
lemma familyMap_sum (e : ComponentIdx (S := S) c ≃ ι) {α : Type*} (s : Finset α)
    (T : α → ι → B) :
    familyMap e (fun l => ∑ i ∈ s, T i l) = ∑ i ∈ s, familyMap e (T i)
```

lemma familyMap_sub [source]

`familyMap` of a difference of families is the difference of the maps.

```
lemma familyMap_sub (e : ComponentIdx (S := S) c ≃ ι) (T T' : ι → B) :
    familyMap e (fun l => T l - T' l) = familyMap e T - familyMap e T'
```

lemma comp_familyMap [source]

A linear map applied after `familyMap e T` is `familyMap` of its values on the components.

```
lemma comp_familyMap (e : ComponentIdx (S := S) c ≃ ι) {B' : Type*} [AddCommGroup B']
    [Module k B'] (σ : B →ₗ[k] B') (T : ι → B) :
    σ ∘ₗ familyMap e T = familyMap e fun l => σ (T l)
```

lemma familyMap_smul_of_law [source]

A linear map moving a family as `g` moves the basis tensors intertwines the map of the family with the action of `g`.

```
lemma familyMap_smul_of_law (e : ComponentIdx (S := S) c ≃ ι) [Fintype ι] (T : ι → B)
    {σ : B →ₗ[k] B} (g : G)
    (hσ : ∀ l : ι, σ (T l) = ∑ a, (∏ i, LinearMap.toMatrix (b (c i)) (b (c i)) (rep (c i) g)
      (e.symm a i) (e.symm l i)) • T a) (t : S.Tensor c) :
    σ (familyMap e T t) = familyMap e T (g • t)
```

lemma isEquivariant_familyMap_iff [source]

The map of a family is equivariant exactly when the family is moved as the basis tensors are: one matrix entry of `g` per index, the summed index first in each factor.

```
lemma isEquivariant_familyMap_iff (e : ComponentIdx (S := S) c ≃ ι) [Fintype ι]
    {ρ : Representation k G B} (T : ι → B) :
    S.IsEquivariant c ρ (familyMap e T) ↔ ∀ (g : G) (l : ι), ρ g (T l)
      = ∑ a, (∏ i, LinearMap.toMatrix (b (c i)) (b (c i)) (rep (c i) g)
        (e.symm a i) (e.symm l i)) • T a
```

2.620 PhyslibAlpha.Relativity.Tensors.RealTensor.CoVector.Representation

PhyslibAlpha/Relativity/Tensors/RealTensor/CoVector/Representation.lean on GitHub.

def sl2Rep [source]

The representation of `SL(2,ℂ)` on real Lorentz covectors, obtained from the representation of the Lorentz group through the covering map `SL(2,ℂ) →* LorentzGroup 3`.

```
noncomputable def sl2Rep : Representation ℝ SL(2,ℂ) Lorentz.CoVector
```

lemma sl2Rep_dual_dualBasis [source]

The dual of the covector representation on the dual basis: dual covectors transform contravariantly, by the columns of the Lorentz matrix.

```
lemma sl2Rep_dual_dualBasis (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) :
    Lorentz.CoVector.sl2Rep.dual Λ (Lorentz.CoVector.basis.dualBasis μ) =
      ∑ j, (Lorentz.SL2C.toLorentzGroup Λ).1 j μ •
        Lorentz.CoVector.basis.dualBasis j
```

2.621 PhyslibAlpha.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Basic

PhyslibAlpha/SpaceAndTime/SpaceTime/SpaceTimeAlgebra/Basic.lean on GitHub.

def starConjEquiv [source]

***The real structure of the jet ring.** Coefficientwise conjugation is a `ℂ`-linear equivalence from the conjugate module of the jet ring back to the jet ring itself. It is honestly `ℂ`-linear, not merely semilinear, because the conjugate-linearity of `star` cancels against the twisted scalar action of `ConjModule`.*

This is what identifies the jets of a conjugate field with the conjugates of the jets: `ConjModule (SpaceTimeAlgebra ⊗[ℂ] V)` and `SpaceTimeAlgebra ⊗[ℂ] ConjModule V` differ exactly by this equivalence on the jet-ring factor.

```
noncomputable def starConjEquiv : ConjModule SpaceTimeAlgebra ≃ₗ[ℂ] SpaceTimeAlgebra
```

lemma starConjEquiv_apply [source]

```
@[simp]
lemma starConjEquiv_apply (f : ConjModule SpaceTimeAlgebra) :
    starConjEquiv f = star ((conjEquiv (k := ℂ) (M := SpaceTimeAlgebra)).symm f)
```

lemma starConjEquiv_symm_apply [source]

```
@[simp]
lemma starConjEquiv_symm_apply (f : SpaceTimeAlgebra) :
    starConjEquiv.symm f = conjEquiv (k := ℂ) (M := SpaceTimeAlgebra) (star f)
```

lemma coeff_single_one_mul [source]

The first-order Leibniz rule: the degree-one Taylor coefficient, in the direction μ, of a product of jets. This is the coefficient-level statement that the first jet of a product is given by the product rule.

```
lemma coeff_single_one_mul (μ : Fin 1 ⊕ Fin 3) (f g : SpaceTimeAlgebra) :
    coeff (Finsupp.single μ 1) (f * g) =
      coeff (Finsupp.single μ 1) f * constantCoeff g +
        constantCoeff f * coeff (Finsupp.single μ 1) g
```

def constantCoeff$_l$ [source]

The constant-coefficient evaluation of a jet, as a $\mathbb{C}$-linear map.

```
noncomputable def constantCoeffₗ : SpaceTimeAlgebra →ₗ[ℂ] ℂ where
```

lemma constantCoeff$_l$_apply [source]

```
@[simp]
lemma constantCoeffₗ_apply (f : SpaceTimeAlgebra) : constantCoeffₗ f = constantCoeff f
```

lemma iteratedPDeriv_pderiv [source]

Iterated formal derivatives commute with a single formal partial derivative.

```
lemma iteratedPDeriv_pderiv (s : Multiset (Fin 1 ⊕ Fin 3)) (μ : Fin 1 ⊕ Fin 3)
    (f : SpaceTimeAlgebra) :
    iteratedPDeriv s (pderiv μ f) = pderiv μ (iteratedPDeriv s f)
```

lemma iteratedPDeriv_smul [source]

Iterated formal derivatives preserve complex scalar multiplication.

```
lemma iteratedPDeriv_smul (s : Multiset (Fin 1 ⊕ Fin 3)) (c : ℂ)
    (f : SpaceTimeAlgebra) :
    iteratedPDeriv s (c • f) = c • iteratedPDeriv s f
```

lemma iteratedPDeriv_neg [source]

Iterated formal derivatives commute with negation.

```
lemma iteratedPDeriv_neg (s : Multiset (Fin 1 ⊕ Fin 3)) (f : SpaceTimeAlgebra) :
    iteratedPDeriv s (-f) = -iteratedPDeriv s f
```

lemma iteratedPDeriv_zero_apply [source]

The iterated formal derivative of the zero jet vanishes.

```
@[simp]
lemma iteratedPDeriv_zero_apply (s : Multiset (Fin 1 ⊕ Fin 3)) :
    iteratedPDeriv s (0 : SpaceTimeAlgebra) = 0
```

lemma iteratedPDeriv_sum [source]

The iterated formal derivative of a finite sum.

```
lemma iteratedPDeriv_sum {κ : Type*} (s : Multiset (Fin 1 ⊕ Fin 3)) (t : Finset κ)
    (f : κ → SpaceTimeAlgebra) :
    iteratedPDeriv s (∑ k ∈ t, f k)
      = ∑ k ∈ t, iteratedPDeriv s (f k)
```

lemma iteratedPDeriv_C_of_ne_zero [source]

The iterated derivative of a constant jet vanishes for a nonempty multiset of directions.

```
lemma iteratedPDeriv_C_of_ne_zero {s : Multiset (Fin 1 ⊕ Fin 3)} (hs : s ≠ 0) (c : ℂ) :
    iteratedPDeriv s (C c : SpaceTimeAlgebra) = 0
```

def truncation [source]

The n-th truncation of a jet: the Taylor coefficients of total degree greater than n are set to zero.

```
noncomputable def truncation (n : ℕ) (f : SpaceTimeAlgebra) : SpaceTimeAlgebra
```

lemma coeff_truncation_of_le [source]

```
@[simp]
lemma coeff_truncation_of_le {n : ℕ} {m : (Fin 1 ⊕ Fin 3) →₀ ℕ}
    (h : Finsupp.degree m ≤ n) (f : SpaceTimeAlgebra) :
    coeff m (truncation n f) = coeff m f
```

lemma coeff_truncation_of_gt [source]

```
@[simp]
lemma coeff_truncation_of_gt {n : ℕ} {m : (Fin 1 ⊕ Fin 3) →₀ ℕ}
    (h : n < Finsupp.degree m) (f : SpaceTimeAlgebra) :
    coeff m (truncation n f) = 0
```

lemma truncation_add [source]

```
lemma truncation_add (n : ℕ) (f g : SpaceTimeAlgebra) :
    truncation n (f + g) = truncation n f + truncation n g
```

lemma truncation_zero [source]

```
@[simp]
lemma truncation_zero (n : ℕ) : truncation n (0 : SpaceTimeAlgebra) = 0
```

lemma truncation_one [source]

Truncation fixes the identity: a constant series has its only nonzero Taylor coefficient in degree zero, which every truncation keeps.

```
@[simp]
lemma truncation_one (n : ℕ) : truncation n (1 : SpaceTimeAlgebra) = 1
```

lemma truncation_eq_one_of_coeff [source]

A power series with value `1` and no coefficients in nonzero degree up to `n` truncates to `1`.

```
lemma truncation_eq_one_of_coeff {n : ℕ} {f : SpaceTimeAlgebra} (h0 : constantCoeff f = 1)
    (hf : ∀ p : (Fin 1 ⊕ Fin 3) →₀ ℕ, p ≠ 0 → Finsupp.degree p ≤ n → coeff p f = 0) :
    SpaceTimeAlgebra.truncation n f = SpaceTimeAlgebra.truncation n (1 : SpaceTimeAlgebra)
```

lemma coeff_X_smul [source]

The Taylor coefficients of a jet multiplied by a formal coordinate: the coefficient shifts down by one in that direction.

```
lemma coeff_X_smul (ρ : Fin 1 ⊕ Fin 3) (f : SpaceTimeAlgebra) (p : (Fin 1 ⊕ Fin 3) →₀ ℕ) :
    coeff p ((X ρ : SpaceTimeAlgebra) • f) =
      if Finsupp.single ρ 1 ≤ p then coeff (p - Finsupp.single ρ 1) f else 0
```

lemma coeff_sum_X_smul_pderiv [source]

The Euler (radial) operator acts on Taylor coefficients as multiplication by the total degree.

```
lemma coeff_sum_X_smul_pderiv (f : SpaceTimeAlgebra) (p : (Fin 1 ⊕ Fin 3) →₀ ℕ) :
    coeff p (∑ ρ, (X ρ : SpaceTimeAlgebra) • pderiv ρ f) =
      ((Finsupp.degree p : ℕ) : ℂ) * coeff p f
```

lemma eq_zero_of_sum_X_smul_pderiv_eq_zero [source]

The scalar vanishing principle for the Euler operator: a jet vanishing at the base point that is killed by the Euler operator is zero.

```
lemma eq_zero_of_sum_X_smul_pderiv_eq_zero {f : SpaceTimeAlgebra} (h0 : constantCoeff f = 0)
    (hf : ∑ ρ, (X ρ : SpaceTimeAlgebra) • pderiv ρ f = 0) : f = 0
```

lemma coeff_mul_eq_zero_of_lt [source]

A product with a factor whose coefficients vanish below degree `n` has coefficients vanishing below degree `n`.

```
lemma coeff_mul_eq_zero_of_lt {n : ℕ} {w : SpaceTimeAlgebra}
    (hw : ∀ q : (Fin 1 ⊕ Fin 3) →₀ ℕ, Finsupp.degree q < n → coeff q w = 0) (v :
    SpaceTimeAlgebra)
    {q : (Fin 1 ⊕ Fin 3) →₀ ℕ} (hq : Finsupp.degree q < n) : coeff q (w * v) = 0
```

lemma coeff_eq_zero_of_coeff_pderiv_eq_zero [source]

The Euler vanishing principle: a power series all of whose first derivatives have coefficients vanishing below degree `n` has vanishing coefficients in every nonzero degree up to `n`, since `∑_ρ x_ρ ∂_ρ f` has the coefficient of `f` at `p` scaled by the degree of `p`.

```
lemma coeff_eq_zero_of_coeff_pderiv_eq_zero {n : ℕ} {f : SpaceTimeAlgebra}
    (hf : ∀ (ρ : Fin 1 ⊕ Fin 3) (q : (Fin 1 ⊕ Fin 3) →₀ ℕ), Finsupp.degree q < n →
      coeff q (pderiv ρ f) = 0)
    {p : (Fin 1 ⊕ Fin 3) →₀ ℕ} (hp : p ≠ 0) (hpn : Finsupp.degree p ≤ n) : coeff p f = 0
```

lemma coeff_eq_zero_of_pderiv_eq_mul [source]

A power series satisfying a radial relation `∂_ρ f = x_ρ f`, with the `x_ρ` vanishing below degree `n`, has no coefficients in nonzero degree up to `n`.

```
lemma coeff_eq_zero_of_pderiv_eq_mul {n : ℕ} {f : SpaceTimeAlgebra}
    {x : (Fin 1 ⊕ Fin 3) → SpaceTimeAlgebra}
    (hd : ∀ ρ, pderiv ρ f = x ρ * f)
    (hx : ∀ (ρ : Fin 1 ⊕ Fin 3) (q : (Fin 1 ⊕ Fin 3) →₀ ℕ), Finsupp.degree q < n →
      coeff q (x ρ) = 0)
    {p : (Fin 1 ⊕ Fin 3) →₀ ℕ} (hp : p ≠ 0) (hpn : Finsupp.degree p ≤ n) : coeff p f = 0
```

lemma C_real_smul [source]

```
lemma C_real_smul (r : ℝ) (x : ℂ) :
    (MvPowerSeries.C (r • x) : SpaceTimeAlgebra) = r • MvPowerSeries.C x
```

lemma constantCoeff_real_smul [source]

The constant coefficient commutes with real scalars.

```
lemma constantCoeff_real_smul (r : ℝ) (f : SpaceTimeAlgebra) :
    MvPowerSeries.constantCoeff (r • f) = r • MvPowerSeries.constantCoeff f
```

lemma pderiv_real_smul [source]

The formal derivatives commute with real scalars.

```
lemma pderiv_real_smul (μ : Fin 1 ⊕ Fin 3) (r : ℝ) (f : SpaceTimeAlgebra) :
    MvPowerSeries.pderiv μ (r • f) = r • MvPowerSeries.pderiv μ f
```

2.622 PhyslibAlpha.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Jacobi

PhyslibAlpha/SpaceAndTime/SpaceTime/SpaceTimeAlgebra/Jacobi.lean on GitHub.

lemma pderiv_finset_prod [source]

The Leibniz rule for a finite product.

```
lemma pderiv_finset_prod {ι : Type*} [DecidableEq ι] (μ : Fin 1 ⊕ Fin 3) (s : Finset ι)
    (f : ι → SpaceTimeAlgebra) :
    pderiv μ (∏ i ∈ s, f i) = ∑ i ∈ s, (∏ j ∈ s.erase i, f j) * pderiv μ (f i)
```

lemma jacobi [source]

***Jacobi's formula**: the derivative of a determinant is the trace of the derivative against the adjugate.*

```
lemma jacobi {κ : Type} [Fintype κ] [DecidableEq κ] (M : Matrix κ κ SpaceTimeAlgebra)
    (μ : Fin 1 ⊕ Fin 3) :
    pderiv μ M.det = (M.map (pderiv μ) * M.adjugate).trace
```

2.623 PhyslibAlpha.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Matrix

PhyslibAlpha/SpaceAndTime/SpaceTime/SpaceTimeAlgebra/Matrix.lean on GitHub.

lemma mapMatrix_constantCoeff_star [source]

Entrywise evaluation at the base point commutes with the conjugate transpose.

```
lemma mapMatrix_constantCoeff_star {n : Type} [Fintype n] [DecidableEq n]
    (A : Matrix n n SpaceTimeAlgebra) :
    (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix (star A) =
      star ((constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix A)
```

lemma coeff_sum_X_smul_map_pderiv [source]

The Euler operator on matrices of jets acts entrywise on Taylor coefficients as multiplication by the total degree.

```
lemma coeff_sum_X_smul_map_pderiv {κ : Type}
    (M : Matrix κ κ SpaceTimeAlgebra) (p : (Fin 1 ⊕ Fin 3) →₀ ℕ) (i j : κ) :
    coeff p ((∑ ρ, (X ρ : SpaceTimeAlgebra) • M.map (pderiv ρ)) i j) =
      ((Finsupp.degree p : ℕ) : ℂ) * coeff p (M i j)
```

lemma matrix_eq_zero_of_euler_eq_mul_add_mul [source]

The vanishing principle for the Euler operator: a matrix of jets vanishing at the base point and satisfying `E W = A W + W B` with `A`, `B` vanishing at the base point is zero. Each Taylor coefficient of `W` is a multiple of coefficients of strictly smaller degree, so all vanish by strong induction on the degree.

```
lemma matrix_eq_zero_of_euler_eq_mul_add_mul {κ : Type} [Fintype κ]
    {W : Matrix κ κ SpaceTimeAlgebra} (A B : Matrix κ κ SpaceTimeAlgebra)
    (hA : ∀ i j, constantCoeff (A i j) = 0) (hB : ∀ i j, constantCoeff (B i j) = 0)
    (h0 : ∀ i j, constantCoeff (W i j) = 0)
    (hW : ∑ ρ, (X ρ : SpaceTimeAlgebra) • W.map (pderiv ρ) = A * W + W * B) :
    W = 0
```

lemma exists_matrix_eulerTransport [source]

The Euler (radial) transport of a jet matrix `R` vanishing at the base point: a fundamental solution of the radial system `E U = R U` based at the identity, built order-by-order by the Euler recursion.

```
lemma exists_matrix_eulerTransport {κ : Type} [Fintype κ] [DecidableEq κ]
    (R : Matrix κ κ SpaceTimeAlgebra) (hR0 : ∀ i j, constantCoeff (R i j) = 0) :
    ∃ U : Matrix κ κ SpaceTimeAlgebra, (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix U = 1
     ∧
      ∑ ρ, (X ρ : SpaceTimeAlgebra) • U.map (pderiv ρ) = R * U
```

lemma coeff_entry_eq_zero_of_map_pderiv_eq_mul [source]

The matrix form of `coeff_eq_zero_of_pderiv_eq_mul`: the entries of a matrix of power series satisfying `∂_ρ A = X_ρ A`, with the `X_ρ` vanishing below degree `n`, have no coefficients in nonzero degree up to `n`.

```
lemma coeff_entry_eq_zero_of_map_pderiv_eq_mul {κ : Type} [Fintype κ] {n : ℕ}
    {A : Matrix κ κ SpaceTimeAlgebra} {X : (Fin 1 ⊕ Fin 3) → Matrix κ κ SpaceTimeAlgebra}
    (hd : ∀ ρ, A.map (pderiv ρ) = X ρ * A)
    (hX : ∀ (ρ : Fin 1 ⊕ Fin 3) (q : (Fin 1 ⊕ Fin 3) →₀ ℕ), Finsupp.degree q < n →
      ∀ i j, coeff q (X ρ i j) = 0)
```

```
    (i j : κ) {p : (Fin 1 ⊕ Fin 3) →₀ ℕ} (hp : p ≠ 0) (hpn : Finsupp.degree p ≤ n) :
    coeff p (A i j) = 0
```

lemma matrix_map_truncation_eq_one [source]

A matrix of power series with identity value and no coefficients in nonzero degree up to `n` truncates to the identity.

```
lemma matrix_map_truncation_eq_one {κ : Type} [Fintype κ] [DecidableEq κ] {n : ℕ}
    {A : Matrix κ κ SpaceTimeAlgebra}
        (h0 : (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix A = 1)
    (hA : ∀ (i j : κ) (p : (Fin 1 ⊕ Fin 3) →₀ ℕ), p ≠ 0 → Finsupp.degree p ≤ n →
      coeff p (A i j) = 0) :
    A.map (SpaceTimeAlgebra.truncation n) = (1 : Matrix κ κ SpaceTimeAlgebra).map
        (SpaceTimeAlgebra.truncation n)
```

lemma matrix_map_pderiv_mul [source]

The entrywise Leibniz rule for matrix products of jets.

```
lemma matrix_map_pderiv_mul {κ : Type} [Fintype κ] (ρ : Fin 1 ⊕ Fin 3)
    (M N : Matrix κ κ SpaceTimeAlgebra) :
    (M * N).map (pderiv ρ) = M.map (pderiv ρ) * N + M * N.map (pderiv ρ)
```

lemma sum_X_smul_map_pderiv_mul [source]

The Euler operator on matrices of jets is a derivation.

```
lemma sum_X_smul_map_pderiv_mul {κ : Type} [Fintype κ]
    (M N : Matrix κ κ SpaceTimeAlgebra) :
    ∑ ρ, (X ρ : SpaceTimeAlgebra) • (M * N).map (pderiv ρ) =
      (∑ ρ, (X ρ : SpaceTimeAlgebra) • M.map (pderiv ρ)) * N +
        M * ∑ ρ, (X ρ : SpaceTimeAlgebra) • N.map (pderiv ρ)
```

lemma sum_X_smul_map_pderiv_star [source]

The Euler operator commutes with the conjugate transpose.

```
lemma sum_X_smul_map_pderiv_star {κ : Type}
    (M : Matrix κ κ SpaceTimeAlgebra) :
    ∑ ρ, (X ρ : SpaceTimeAlgebra) • (star M).map (pderiv ρ) =
      star (∑ ρ, (X ρ : SpaceTimeAlgebra) • M.map (pderiv ρ))
```

lemma sum_X_smul_map_pderiv_one [source]

The Euler operator kills the identity matrix.

```
lemma sum_X_smul_map_pderiv_one {κ : Type} [DecidableEq κ] :
    ∑ ρ, (X ρ : SpaceTimeAlgebra) • (1 : Matrix κ κ SpaceTimeAlgebra).map (pderiv ρ) = 0
```

lemma eulerTransport_mul_star [source]

A fundamental solution of the radial system `E U = R U` based at the identity is unitary when `R` is anti-hermitian: `U U† − 1` vanishes at the base point and satisfies a homogeneous linear radial system, so it vanishes identically.

```
lemma eulerTransport_mul_star {κ : Type} [Fintype κ] [DecidableEq κ]
    {R U : Matrix κ κ SpaceTimeAlgebra} (hRstar : star R = -R)
    (hR0 : ∀ i j, constantCoeff (R i j) = 0)
    (hU0 : (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix U = 1)
    (hEU : ∑ ρ, (X ρ : SpaceTimeAlgebra) • U.map (pderiv ρ) = R * U) :
    U * star U = 1
```

lemma sum_X_smul_mcMatrix_of_eulerTransport [source]

The radial Maurer–Cartan component of a unitary fundamental solution of the radial system `E V = −i P V` is `P`: `∑ μ x_μ · i (∂_μ V) V† = P`.

```
lemma sum_X_smul_mcMatrix_of_eulerTransport {κ : Type} [Fintype κ] [DecidableEq κ]
    {P V : Matrix κ κ SpaceTimeAlgebra} (hVu : V * star V = 1)
    (hEV : ∑ μ, (X μ : SpaceTimeAlgebra) • V.map (pderiv μ) = ((-Complex.I) • P) * V) :
    ∑ μ, (X μ : SpaceTimeAlgebra) • (Complex.I • (V.map (pderiv μ) * star V)) = P
```

lemma eulerTransport_det [source]

A fundamental solution of the radial system `E U = R U` based at the identity has determinant one when `R` is traceless: by Jacobi's formula the determinant is killed by the Euler operator, so it is the constant `1`.

```
lemma eulerTransport_det {κ : Type} [Fintype κ] [DecidableEq κ]
    {R U : Matrix κ κ SpaceTimeAlgebra}
    (hjac : ∀ (M : Matrix κ κ SpaceTimeAlgebra) (μ : Fin 1 ⊕ Fin 3),
      pderiv μ M.det = (M.map (pderiv μ) * M.adjugate).trace)
    (hRtr : R.trace = 0)
    (hU0 : (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix U = 1)
    (hEU : ∑ ρ, (X ρ : SpaceTimeAlgebra) • U.map (pderiv ρ) = R * U) :
    U.det = 1
```

lemma matrix_multiset_sum_apply [source]

The entry of a multiset sum of matrices is the multiset sum of the entries.

```
lemma matrix_multiset_sum_apply {κ α : Type*} [AddCommMonoid α]
    (m : Multiset (Matrix κ κ α)) (i j : κ) :
    m.sum i j = (m.map fun A => A i j).sum
```

lemma matrix_constantCoeff_iteratedPDeriv_mul [source]

The matrix Leibniz rule at the base point: the base-point Taylor coefficients of a product of matrices of jets are the antidiagonal convolution of the base-point coefficients of the factors.

```
lemma matrix_constantCoeff_iteratedPDeriv_mul {κ : Type} [Fintype κ]
    (s : Multiset (Fin 1 ⊕ Fin 3)) (M N : Matrix κ κ SpaceTimeAlgebra) :
    ((M * N).map fun f => constantCoeff (iteratedPDeriv s f))
      = (s.antidiagonal.map fun p =>
          (M.map fun f => constantCoeff (iteratedPDeriv p.1 f)) *
            (N.map fun f => constantCoeff (iteratedPDeriv p.2 f))).sum
```

lemma mapMatrix_C_star [source]

Entrywise inclusion of constants commutes with the conjugate transpose.

```
lemma mapMatrix_C_star [Fintype κ] [DecidableEq κ] (A : Matrix κ κ ℂ) :
    (C : ℂ →+* SpaceTimeAlgebra).mapMatrix (star A) = star
        ((C : ℂ →+* SpaceTimeAlgebra).mapMatrix A)
```

lemma mapMatrix_C_smul [source]

Entrywise inclusion of constants commutes with complex scalars.

```
lemma mapMatrix_C_smul [Fintype κ] [DecidableEq κ] (c : ℂ) (M : Matrix κ κ ℂ) :
    (C : ℂ →+* SpaceTimeAlgebra).mapMatrix (c • M) = c •
        (C : ℂ →+* SpaceTimeAlgebra).mapMatrix M
```

lemma mapMatrix_constantCoeff_smul [source]

The entrywise constant coefficient commutes with complex scalars.

```
lemma mapMatrix_constantCoeff_smul [Fintype κ] [DecidableEq κ] (c : ℂ)
    (M : Matrix κ κ SpaceTimeAlgebra) :
    (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix (c • M)
      = c • (constantCoeff : SpaceTimeAlgebra →+* ℂ).mapMatrix M
```

lemma star_map_pderiv [source]

The entrywise derivative commutes with the conjugate transpose.

```
lemma star_map_pderiv (μ : Fin 1 ⊕ Fin 3) (A : Matrix κ κ SpaceTimeAlgebra) :
    star (A.map (pderiv μ)) = (star A).map (pderiv μ)
```

lemma map_pderiv_smul [source]

Pulling a complex scalar out of the entrywise derivative.

```
lemma map_pderiv_smul (μ : Fin 1 ⊕ Fin 3) (c : ℂ) (M : Matrix κ κ SpaceTimeAlgebra) :
    (c • M).map (pderiv μ) = c • M.map (pderiv μ)
```

lemma map_pderiv_sub [source]

The entrywise derivative of a difference.

```
lemma map_pderiv_sub (μ : Fin 1 ⊕ Fin 3) (M N : Matrix κ κ SpaceTimeAlgebra) :
    (M - N).map (pderiv μ) = M.map (pderiv μ) - N.map (pderiv μ)
```

lemma map_pderiv_star_of_unitary [source]

The entrywise derivative of the conjugate transpose of a unitary matrix, through the differentiated unitarity relation.

```
lemma map_pderiv_star_of_unitary [Fintype κ] [DecidableEq κ] (μ : Fin 1 ⊕ Fin 3)
    {U : Matrix κ κ SpaceTimeAlgebra}
    (hU : U * star U = 1) (hU' : star U * U = 1) :
    (star U).map (pderiv μ) = -(star U * U.map (pderiv μ) * star U)
```

lemma star_mcMatrix [source]

The Maurer–Cartan matrix `i (∂_μ U) U†` of a unitary matrix of jets is hermitian.

```
lemma star_mcMatrix [Fintype κ] [DecidableEq κ] (μ : Fin 1 ⊕ Fin 3)
    {U : Matrix κ κ SpaceTimeAlgebra}
    (hU : U * star U = 1) (hU' : star U * U = 1) :
    star (Complex.I • (U.map (pderiv μ) * star U)) = Complex.I • (U.map (pderiv μ) * star U)
```

2.624 PhyslibAlpha.SpaceAndTime.SpaceTime.SpaceTimeAlgebra.Taylor

PhyslibAlpha/SpaceAndTime/SpaceTime/SpaceTimeAlgebra/Taylor.lean on GitHub.

lemma star_ofDerivValues [source]

```
lemma star_ofDerivValues (f : Multiset (Fin 1 ⊕ Fin 3) → ℂ) :
    star (ofDerivValues f) = ofDerivValues fun s => star (f s)
```

lemma ofDerivValues_sum [source]

```
lemma ofDerivValues_sum {ι : Type} (t : Finset ι) (f : ι → Multiset (Fin 1 ⊕ Fin 3) → ℂ) :
    ofDerivValues (fun s => ∑ i ∈ t, f i s) = ∑ i ∈ t, ofDerivValues (f i)
```

def taylorMatrix [source]

The matrix of jets with prescribed base-point Taylor data `M`, entrywise.

```
noncomputable def taylorMatrix {κ : Type} (M : Multiset (Fin 1 ⊕ Fin 3) → Matrix κ κ ℂ) :
    Matrix κ κ SpaceTimeAlgebra
```

lemma taylorMatrix_apply [source]

```
lemma taylorMatrix_apply {κ : Type} (M : Multiset (Fin 1 ⊕ Fin 3) → Matrix κ κ ℂ) (i j : κ) :
    taylorMatrix M i j = ofDerivValues fun s => M s i j
```

lemma star_taylorMatrix [source]

```
lemma star_taylorMatrix {κ : Type} {M : Multiset (Fin 1 ⊕ Fin 3) → Matrix κ κ ℂ}
    (hM : ∀ s, star (M s) = M s) : star (taylorMatrix M) = taylorMatrix M
```

lemma trace_taylorMatrix [source]

```
lemma trace_taylorMatrix {κ : Type} [Fintype κ] {M : Multiset (Fin 1 ⊕ Fin 3) → Matrix κ κ ℂ}
    (hM : ∀ s, (M s).trace = 0) : (taylorMatrix M).trace = 0
```

lemma map_constantCoeff_iteratedPDeriv_taylorMatrix [source]

The base-point Taylor data of `taylorMatrix M` are `M`, entrywise.

```
lemma map_constantCoeff_iteratedPDeriv_taylorMatrix {κ : Type}
    (M : Multiset (Fin 1 ⊕ Fin 3) → Matrix κ κ ℂ) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    (taylorMatrix M).map (fun f => constantCoeff (iteratedPDeriv s f)) = M s
```

2.625 PhyslibAlpha.SpaceAndTime.SpaceTime.SpaceTimeDerivAlgebra

PhyslibAlpha/SpaceAndTime/SpaceTime/SpaceTimeDerivAlgebra.lean on GitHub.

lemma basis_apply_symmetricAlgebra [source]

The basis vector at a multiset of derivative indices is the corresponding basis monomial of the symmetric algebra of dual symbols. This is `basis_apply` of `Physlib.SpaceAndTime.SpaceTime.Spa` in the symmetric-algebra form, rather than through `SymmetricAlgebra.equivMvPolynomial`.

```
lemma basis_apply_symmetricAlgebra (s : Multiset (Fin 1 ⊕ Fin 3)) :
    basis s =
      Lorentz.ComplexCoVector.basis.dualBasis.symmetricAlgebra (Multiset.toFinsupp s)
```

lemma basis_nil [source]

The basis vector at the empty multiset is the unit of the algebra: the zeroth-order symbol carries no derivatives.

```
lemma basis_nil :
    basis ({} : Multiset (Fin 1 ⊕ Fin 3)) = 1
```

def deriv [source]

The derivative of an element in `SpaceTimeDerivAlgebraℂ` taking e.g. `∂_s` to `∂_μ ∂_s`.

```
noncomputable def deriv (μ : Fin 1 ⊕ Fin 3) : SpaceTimeDerivAlgebraℂ →ₗ[ℂ]
    SpaceTimeDerivAlgebraℂ
```

lemma deriv_basis [source]

```
lemma deriv_basis (μ : Fin 1 ⊕ Fin 3) (m : (Fin 1 ⊕ Fin 3) →₀ ℕ) :
    deriv μ (Lorentz.ComplexCoVector.basis.dualBasis.symmetricAlgebra m) =
      Lorentz.ComplexCoVector.basis.dualBasis.symmetricAlgebra (m + Finsupp.single μ 1)
```

lemma deriv_comm_apply [source]

```
lemma deriv_comm_apply (μ ν : Fin 1 ⊕ Fin 3) (x : SpaceTimeDerivAlgebraℂ) :
    deriv μ (deriv ν x) = deriv ν (deriv μ x)
```

lemma deriv_basis_multiset [source]

The derivative operator on the multiset basis: appending the derivative index to the multiset.

```
lemma deriv_basis_multiset (μ : Fin 1 ⊕ Fin 3) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    deriv μ (basis s) = basis (s + {μ})
```

lemma deriv_apply_eq_mul [source]

The derivative operator is right multiplication by the first-order derivative symbol.

```
lemma deriv_apply_eq_mul (μ : Fin 1 ⊕ Fin 3) (a : SpaceTimeDerivAlgebraℂ) :
    deriv μ a = a * basis ({μ} : Multiset (Fin 1 ⊕ Fin 3))
```

lemma pderiv_comm [source]

Formal partial derivatives of a multivariate power series commute: each is given on coefficients by a shift and a multiplication, and the two shifts commute.

```
lemma _root_.MvPowerSeries.pderiv_comm {σ R : Type*} [CommSemiring R] (i j : σ)
    (f : MvPowerSeries σ R) :
    MvPowerSeries.pderiv i (MvPowerSeries.pderiv j f) =
      MvPowerSeries.pderiv j (MvPowerSeries.pderiv i f)
```

instance RightCommutative (fun (f : SpaceTimeAlgebra) (μ : Fin 1 ⊕ Fin 3) => MvP… [source]

Differentiating a jet along a multiset of directions is well defined: the partial derivatives commute, so the fold over a multiset does not depend on the order.

```
instance : RightCommutative
    (fun (f : SpaceTimeAlgebra) (μ : Fin 1 ⊕ Fin 3) => MvPowerSeries.pderiv μ f) where
```

def eval [source]

The evaluation map taking a function `f : SpaceTimeAlgebra` to `∂_μ f`.

```
noncomputable def eval : SpaceTimeDerivAlgebraℂ →ₗ[ℂ] SpaceTimeAlgebra →ₗ[ℂ] ℂ
```

lemma eval_basis [source]

```
@[simp]
lemma eval_basis (m : (Fin 1 ⊕ Fin 3) →₀ ℕ) (f : SpaceTimeAlgebra) :
    eval (Lorentz.ComplexCoVector.basis.dualBasis.symmetricAlgebra m) f =
      (∏ μ, (m μ)! : ℕ) • MvPowerSeries.coeff m f
```

lemma eval_monomial [source]

```
lemma eval_monomial (p : SpaceTimeDerivAlgebraℂ) (m : (Fin 1 ⊕ Fin 3) →₀ ℕ) :
    eval p (MvPowerSeries.monomial m 1) =
      ((∏ μ, (m μ)! : ℕ) : ℂ) *
        Lorentz.ComplexCoVector.basis.dualBasis.symmetricAlgebra.repr p m
```

lemma eval_injective [source]

```
lemma eval_injective {p q : SpaceTimeDerivAlgebraℂ}
    (h : ∀ f, eval p f = eval q f) : p = q
```

lemma eval_deriv [source]

Adjointness: the shift of derivative symbols is the transpose of the formal partial derivative under the divided-power pairing.

```
lemma eval_deriv (ν : Fin 1 ⊕ Fin 3) (p : SpaceTimeDerivAlgebraℂ) (f : SpaceTimeAlgebra) :
    eval (deriv ν p) f = eval p (MvPowerSeries.pderiv ν f)
```

lemma eval_one [source]

The pairing of the unit derivative symbol with a jet is its value at the base point: the empty derivative multiset reads off the constant term.

```
lemma eval_one (f : SpaceTimeAlgebra) :
    eval (1 : SpaceTimeDerivAlgebraℂ) f = MvPowerSeries.constantCoeff f
```

lemma eval_basis_eq_constantCoeff_iteratedPDeriv [source]

Iterating adjointness: the pairing of the basis monomial `∂_s` with a jet is the constant term of the iterated formal partial derivative `∂_s f`. This is the concrete description of the divided-power pairing that `eval_deriv` encodes one derivative at a time.

```
lemma eval_basis_eq_constantCoeff_iteratedPDeriv (s : Multiset (Fin 1 ⊕ Fin 3))
    (f : SpaceTimeAlgebra) :
    eval (basis s) f =
      MvPowerSeries.constantCoeff (SpaceTimeAlgebra.iteratedPDeriv s f)
```

def jetRingAction [source]

The action of `χ` on the derivatives, this takes `∂_μ ·` to `∂_μ (χ ·)`, expanded out explicitly.

```
noncomputable def jetRingAction (χ : SpaceTimeAlgebra) :
    SpaceTimeDerivAlgebraℂ →ₗ[ℂ] SpaceTimeDerivAlgebraℂ
```

lemma jetRingAction_basis [source]

```
lemma jetRingAction_basis (χ : SpaceTimeAlgebra) (m : (Fin 1 ⊕ Fin 3) →₀ ℕ) :
    jetRingAction χ (Lorentz.ComplexCoVector.basis.dualBasis.symmetricAlgebra m) =
      ∑ p ∈ Finset.antidiagonal m,
        ((∏ μ, (m μ).descFactorial (p.1 μ) : ℕ) : ℂ) • MvPowerSeries.coeff p.1 χ •
          Lorentz.ComplexCoVector.basis.dualBasis.symmetricAlgebra p.2
```

lemma eval_jetRingAction [source]

```
lemma eval_jetRingAction (χ f : SpaceTimeAlgebra) (p : SpaceTimeDerivAlgebraℂ) :
    eval (jetRingAction χ p) f = eval p (χ * f)
```

lemma jetRingAction_basis_multiset [source]

The action of a jet on a derivative monomial, in multiset form: `∂_s` is sent to the all-orders Leibniz convolution, each splitting `s = s₁ + s₂` of the derivative multiset contributing the base-point Taylor coefficient `(∂_{s₁} χ)(0)` against the lower monomial `∂_{s₂}`.

This is `jetRingAction_basis` with the `Nat.choose` bookkeeping traded for the divided-power pairing: both sides are compared through `eval`, where the identity is the Leibniz rule `SpaceTimeAlgebra.constantCoeff_iteratedPDeriv_mul` at the base point. It is the form in which the transformation law of a matter field is stated.

```
lemma jetRingAction_basis_multiset (χ : SpaceTimeAlgebra) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    jetRingAction χ (basis s) =
      (s.antidiagonal.map fun p =>
```

```
        MvPowerSeries.constantCoeff
            (SpaceTimeAlgebra.iteratedPDeriv p.1 χ) • basis p.2).sum
```

lemma jetRingAction_C [source]

Constant jets act on the derivative symbols by their value: `C c` has no derivative coordinates.

```
@[simp]
lemma jetRingAction_C (c : ℂ) :
    jetRingAction (MvPowerSeries.C c) = c • LinearMap.id
```

lemma jetRingAction_one [source]

```
@[simp]
lemma jetRingAction_one : jetRingAction (1 : SpaceTimeAlgebra) = LinearMap.id
```

lemma jetRingAction_mul [source]

The derivative action is multiplicative: it is the transpose of multiplication in the commutative jet ring. This makes `χ ↦ χ(∂)` a monoid homomorphism and hence yields representations of the jet gauge group on polynomial jet spaces.

```
lemma jetRingAction_mul (χ ψ : SpaceTimeAlgebra) :
    jetRingAction (χ * ψ) = jetRingAction χ ∘ₗ jetRingAction ψ
```

lemma jetRingAction_zero [source]

```
@[simp]
lemma jetRingAction_zero : jetRingAction (0 : SpaceTimeAlgebra) = 0
```

lemma jetRingAction_add [source]

```
lemma jetRingAction_add (χ ψ : SpaceTimeAlgebra) :
    jetRingAction (χ + ψ) = jetRingAction χ + jetRingAction ψ
```

def jetRingActionHom [source]

The derivative action as a ring homomorphism from the jet ring to the endomorphisms of the algebra of derivative symbols: the module structure of the jet ring on its graded dual.

```
noncomputable def jetRingActionHom : SpaceTimeAlgebra →+* Module.End ℂ SpaceTimeDerivAlgebraℂ
    where
```

lemma jetRingAction_comm [source]

The actions of two jets commute: the jet ring is commutative.

```
lemma jetRingAction_comm (χ ψ : SpaceTimeAlgebra) (a : SpaceTimeDerivAlgebraℂ) :
    jetRingAction χ (jetRingAction ψ a) = jetRingAction ψ (jetRingAction χ a)
```

lemma jetRingAction_apply_one [source]

The derivative action on the zeroth-order (field) symbol: it is scaled by the value of the jet at the base point.

```
@[simp]
lemma jetRingAction_apply_one (χ : SpaceTimeAlgebra) :
    jetRingAction χ (1 : SpaceTimeDerivAlgebraℂ) =
      MvPowerSeries.constantCoeff χ • 1
```

lemma jetRingAction_apply_ι [source]

The derivative action on a first-order derivative symbol implements the Leibniz rule: `∂_μ ↦ χ(0) ∂_μ + (∂_μχ)(0) 1`. The value of the jet multiplies the first-derivative symbol, and its first derivative feeds the zeroth-order symbol.

```
lemma jetRingAction_apply_ι (χ : SpaceTimeAlgebra) (μ : Fin 1 ⊕ Fin 3) :
    jetRingAction χ (SymmetricAlgebra.ι ℂ (Module.Dual ℂ Lorentz.ComplexCoVector)
        (Lorentz.ComplexCoVector.basis.dualBasis μ)) =
      MvPowerSeries.constantCoeff χ •
        SymmetricAlgebra.ι ℂ (Module.Dual ℂ Lorentz.ComplexCoVector)
          (Lorentz.ComplexCoVector.basis.dualBasis μ) +
        MvPowerSeries.coeff (Finsupp.single μ 1) χ • 1
```

lemma jetRingAction_deriv [source]

The commutation of the jet-ring action with the derivative operator: acting by `χ` after differentiating equals differentiating after acting, plus the action of the derivative `∂_ν χ`. This is the operator form of the Leibniz rule `∂_ν (χ f) = χ ∂_ν f + (∂_ν χ) f` under the divided-power pairing.

```
lemma jetRingAction_deriv (χ : SpaceTimeAlgebra) (ν : Fin 1 ⊕ Fin 3) (a : SpaceTimeDerivAlgebra
    ℂ) :
    jetRingAction χ (deriv ν a) =
      deriv ν (jetRingAction χ a) + jetRingAction (MvPowerSeries.pderiv ν χ) a
```

def SpaceTimeDerivAlgebraℂ.repLorentzGroup [source]

The representation of the Lorentz group `SL(2,ℂ)` on the algebra of derivative symbols, extending the dual covector representation multiplicatively.

```
noncomputable def repLorentzGroup : Representation ℂ SL(2,ℂ) SpaceTimeDerivAlgebraℂ where
```

lemma SpaceTimeDerivAlgebraℂ.repLorentzGroup_apply_one [source]

```
@[simp]
lemma repLorentzGroup_apply_one (Λ : SL(2,ℂ)) : repLorentzGroup Λ 1 = 1
```

lemma SpaceTimeDerivAlgebraℂ.repLorentzGroup_apply_mul [source]

```
lemma repLorentzGroup_apply_mul (Λ : SL(2,ℂ)) (a b : SpaceTimeDerivAlgebraℂ) :
    repLorentzGroup Λ (a * b) = repLorentzGroup Λ a * repLorentzGroup Λ b
```

lemma SpaceTimeDerivAlgebraℂ.repLorentzGroup_apply_ι [source]

The Lorentz action on a generator.

```
@[simp]
lemma repLorentzGroup_apply_ι (Λ : SL(2,ℂ)) (x : Module.Dual ℂ Lorentz.ComplexCoVector) :
    repLorentzGroup Λ (SymmetricAlgebra.ι ℂ (Module.Dual ℂ Lorentz.ComplexCoVector) x) =
      SymmetricAlgebra.ι ℂ (Module.Dual ℂ Lorentz.ComplexCoVector)
        (Lorentz.ComplexCoVector.SL2CRep.dual Λ x)
```

lemma repLorentzGroup_deriv [source]

The Lorentz action on a derivative: the derivative symbol transforms as a covector, mixing the spacetime directions by the components of `Λ` in the dual covector representation.

```
lemma repLorentzGroup_deriv (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) (a : SpaceTimeDerivAlgebraℂ) :
    repLorentzGroup Λ (deriv μ a) =
      ∑ ν, (Lorentz.ComplexCoVector.SL2CRep.dual Λ (Lorentz.ComplexCoVector.basis.dualBasis μ))
        (Lorentz.ComplexCoVector.basis ν) • deriv ν (repLorentzGroup Λ a)
```

lemma repLorentzGroup_basis_singleton [source]

The Lorentz action on the singleton derivative monomial: the derivative slot transforms by the columns of the Lorentz matrix.

```
lemma repLorentzGroup_basis_singleton (Λ : SL(2,ℂ)) (μ : Fin 1 ⊕ Fin 3) :
    repLorentzGroup Λ (basis ({μ} : Multiset (Fin 1 ⊕ Fin 3))) =
      ∑ ν, (((Lorentz.SL2C.toLorentzGroup Λ).1 ν μ : ℝ) : ℂ) •
        basis ({ν} : Multiset (Fin 1 ⊕ Fin 3))
```

lemma repLorentzGroup_basis_ofFn [source]

The Lorentz action on the derivative monomial of an ordered tuple of directions: every slot mixes by the columns of the Lorentz matrix, one factor per slot.

```
lemma repLorentzGroup_basis_ofFn (Λ : SL(2,ℂ)) {n : ℕ} (l : Fin n → (Fin 1 ⊕ Fin 3)) :
    repLorentzGroup Λ (basis (List.ofFn l)) =
      ∑ p : Fin n → (Fin 1 ⊕ Fin 3),
        (∏ i, (((Lorentz.SL2C.toLorentzGroup Λ).1 (p i) (l i) : ℝ) : ℂ)) • basis (List.ofFn p)
```

def SpaceTimeDerivAlgebraℂ.gradeScale [source]

The derivative-degree scaling on the algebra of derivative symbols: the algebra map multiplying each generator by `t`, hence each degree-`n` monomial by `t ^ n`.

```
noncomputable def gradeScale (t : ℂ) : SpaceTimeDerivAlgebraℂ →ₐ[ℂ] SpaceTimeDerivAlgebraℂ
```

lemma SpaceTimeDerivAlgebraℂ.gradeScale_ι [source]

```
@[simp]
lemma gradeScale_ι (t : ℂ) (x : Module.Dual ℂ Lorentz.ComplexCoVector) :
    gradeScale t (SymmetricAlgebra.ι ℂ (Module.Dual ℂ Lorentz.ComplexCoVector) x) =
      t • SymmetricAlgebra.ι ℂ (Module.Dual ℂ Lorentz.ComplexCoVector) x
```

lemma gradeScale_basis [source]

The degree scaling multiplies the basis monomial at `s` by `t ^ |s|`.

```
lemma gradeScale_basis (t : ℂ) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    gradeScale t (basis s) = t ^ s.card • basis s
```

lemma SpaceTimeDerivAlgebraℂ.gradeScale_repLorentzGroup [source]

The degree scaling commutes with the Lorentz action: the Lorentz action preserves the derivative degree.

```
lemma gradeScale_repLorentzGroup (t : ℂ) (Λ : SL(2,ℂ)) (a : SpaceTimeDerivAlgebraℂ) :
    gradeScale t (repLorentzGroup Λ a) = repLorentzGroup Λ (gradeScale t a)
```

def SpaceTimeDerivAlgebraℂ.gradePoly [source]

The derivative-degree polynomial on the algebra of derivative symbols: the linear map sending the basis monomial `∂_s` to `X ^ |s|` times itself. It is `gradeScale` with the scalar replaced by the formal variable `X`.

```
noncomputable def gradePoly : SpaceTimeDerivAlgebraℂ →ₗ[ℂ] Polynomial SpaceTimeDerivAlgebraℂ
```

lemma gradePoly_basis [source]

The derivative-degree polynomial of a basis monomial is the monomial of degree `|s|`.

```
@[simp]
lemma gradePoly_basis (s : Multiset (Fin 1 ⊕ Fin 3)) :
    gradePoly (basis s) = Polynomial.monomial (Multiset.card s) (basis s)
```

lemma SpaceTimeDerivAlgebraℂ.eval_algebraMap_gradePoly [source]

Evaluating the derivative-degree polynomial at a scalar is the derivative-degree scaling by that scalar: the two descriptions of the grading agree.

```
lemma eval_algebraMap_gradePoly (t : ℂ) (a : SpaceTimeDerivAlgebraℂ) :
    (gradePoly a).eval (algebraMap ℂ SpaceTimeDerivAlgebraℂ t) = gradeScale t a
```

lemma SpaceTimeDerivAlgebraℂ.gradePoly_eval_one [source]

Setting the formal variable to one recovers the original element: the pieces of a graded decomposition sum to the element.

```
lemma gradePoly_eval_one (a : SpaceTimeDerivAlgebraℂ) : (gradePoly a).eval 1 = a
```

abbrev SpaceTimeDerivAlgebraℝ [source]

The $\mathbb{R}$-algebra of derivative symbols in spacetime.

```
abbrev SpaceTimeDerivAlgebraℝ
```

def SpaceTimeDerivAlgebraℝ.repLorentzGroup [source]

The representation of the Lorentz group `SL(2,ℂ)` on the algebra of derivative symbols, extending the dual covector representation multiplicatively.

```
noncomputable def repLorentzGroup : Representation ℝ SL(2,ℂ) SpaceTimeDerivAlgebraℝ where
```

lemma SpaceTimeDerivAlgebraℝ.repLorentzGroup_apply_ι [source]

The Lorentz action on a generator of the real derivative algebra.

```
@[simp]
lemma repLorentzGroup_apply_ι (Λ : SL(2,ℂ)) (x : Module.Dual ℝ Lorentz.CoVector) :
    repLorentzGroup Λ (SymmetricAlgebra.ι ℝ (Module.Dual ℝ Lorentz.CoVector) x) =
      SymmetricAlgebra.ι ℝ (Module.Dual ℝ Lorentz.CoVector)
        (Lorentz.CoVector.sl2Rep.dual Λ x)
```

lemma SpaceTimeDerivAlgebraℝ.repLorentzGroup_apply_mul [source]

The real derivative-algebra representation is multiplicative: it is the lift of a linear map to the symmetric algebra.

```
lemma repLorentzGroup_apply_mul (Λ : SL(2,ℂ))
    (a b : SpaceTimeDerivAlgebraℝ) :
    SpaceTimeDerivAlgebraℝ.repLorentzGroup Λ (a * b) =
      SpaceTimeDerivAlgebraℝ.repLorentzGroup Λ a * SpaceTimeDerivAlgebraℝ.repLorentzGroup Λ b
```

lemma SpaceTimeDerivAlgebraℝ.repLorentzGroup_apply_one [source]

```
@[simp]
lemma repLorentzGroup_apply_one (Λ : SL(2,ℂ)) :
    SpaceTimeDerivAlgebraℝ.repLorentzGroup Λ 1 = 1
```

def SpaceTimeDerivAlgebraℝ.gradeScale [source]

The derivative-degree scaling on the real algebra of derivative symbols: the algebra map multiplying each generator by `t`.

```
noncomputable def gradeScale (t : ℝ) : SpaceTimeDerivAlgebraℝ →ₐ[ℝ] SpaceTimeDerivAlgebraℝ
```

lemma SpaceTimeDerivAlgebraℝ.gradeScale_ι [source]

```
@[simp]
lemma gradeScale_ι (t : ℝ) (x : Module.Dual ℝ Lorentz.CoVector) :
    gradeScale t (SymmetricAlgebra.ι ℝ (Module.Dual ℝ Lorentz.CoVector) x) =
      t • SymmetricAlgebra.ι ℝ (Module.Dual ℝ Lorentz.CoVector) x
```

lemma SpaceTimeDerivAlgebraℝ.gradeScale_repLorentzGroup [source]

The degree scaling commutes with the Lorentz action on the real derivative symbols.

```
lemma gradeScale_repLorentzGroup (t : ℝ) (Λ : SL(2,ℂ)) (a : SpaceTimeDerivAlgebraℝ) :
    gradeScale t (repLorentzGroup Λ a) = repLorentzGroup Λ (gradeScale t a)
```

def basisMultiset [source]

The basis of the symmetric algebra of dual real jet slots, indexed by multisets of spacetime indices.

```
noncomputable def basisMultiset :
    Basis (Multiset (Fin 1 ⊕ Fin 3)) ℝ (SymmetricAlgebra ℝ (Module.Dual ℝ Lorentz.CoVector))
```

lemma basisMultiset_apply [source]

The multiset basis of the dual derivative symbols, as a basis vector of the symmetric algebra at the corresponding multi-index.

```
lemma basisMultiset_apply (s : Multiset (Fin 1 ⊕ Fin 3)) :
    basisMultiset s =
      Lorentz.CoVector.basis.dualBasis.symmetricAlgebra (Multiset.toFinsupp s)
```

lemma basisMultiset_mul [source]

The multiset basis vectors of the real dual derivative slots multiply by adding the multisets.

```
lemma basisMultiset_mul (s t : Multiset (Fin 1 ⊕ Fin 3)) :
    basisMultiset s * basisMultiset t =
      basisMultiset (s + t)
```

lemma basisMultiset_nil [source]

The multiset basis of the real dual derivative slots at the empty multiset is the unit.

```
lemma basisMultiset_nil :
    basisMultiset (0 : Multiset (Fin 1 ⊕ Fin 3)) = 1
```

lemma basisMultiset_singleton [source]

The multiset basis of the real dual derivative slots at a singleton index.

```
lemma basisMultiset_singleton (μ : Fin 1 ⊕ Fin 3) :
    basisMultiset ({μ} : Multiset (Fin 1 ⊕ Fin 3)) =
      SymmetricAlgebra.ι ℝ (Module.Dual ℝ Lorentz.CoVector)
        (Lorentz.CoVector.basis.dualBasis μ)
```

lemma gradeScale_basisMultiset [source]

The degree scaling multiplies the real basis monomial at `s` by `t ^ |s|`.

```
lemma gradeScale_basisMultiset (t : ℝ) (s : Multiset (Fin 1 ⊕ Fin 3)) :
    gradeScale t (basisMultiset s) = t ^ Multiset.card s • basisMultiset s
```

def SpaceTimeDerivAlgebraℝ.gradePoly [source]

The derivative-degree polynomial on the real algebra of derivative symbols: the linear map sending the basis monomial `∂_s` to `X ^ |s|` times itself.

```
noncomputable def gradePoly : SpaceTimeDerivAlgebraℝ →ₗ[ℝ] Polynomial SpaceTimeDerivAlgebraℝ
```

lemma gradePoly_basisMultiset [source]

The derivative-degree polynomial of a real basis monomial is the monomial of degree |s|.

```
@[simp]
lemma gradePoly_basisMultiset (s : Multiset (Fin 1 ⊕ Fin 3)) :
    gradePoly (basisMultiset s) =
      Polynomial.monomial (Multiset.card s) (basisMultiset s)
```

lemma SpaceTimeDerivAlgebraℝ.eval_algebraMap_gradePoly [source]

Evaluating the real derivative-degree polynomial at a scalar is the derivative-degree scaling by that scalar.

```
lemma eval_algebraMap_gradePoly (t : ℝ) (a : SpaceTimeDerivAlgebraℝ) :
    (gradePoly a).eval (algebraMap ℝ SpaceTimeDerivAlgebraℝ t) = gradeScale t a
```

lemma SpaceTimeDerivAlgebraℝ.gradePoly_eval_one [source]

Setting the formal variable to one recovers the original element of the real derivative algebra.

```
lemma gradePoly_eval_one (a : SpaceTimeDerivAlgebraℝ) : (gradePoly a).eval 1 = a
```

2.626 QuantumInfo.ClassicalInfo.Entropy

QuantumInfo/ClassicalInfo/Entropy.lean on GitHub.

lemma H_s_eq_expect_val [source]

Shannon entropy is the expected negative logarithm of the outcome probability. Zero-probability outcomes contribute zero to the expectation.

```
lemma Hₛ_eq_expect_val (d : ProbDistribution α) :
    Hₛ d = ProbDistribution.expect_val ⟨fun x ↦ -Real.log (d x), d⟩
```

lemma H_s_concave [source]

Shannon entropy is concave under convex mixtures of distributions.

```
lemma Hₛ_concave (d₁ d₂ : ProbDistribution α) (p : Prob) :
    p[Hₛ d₁ ↔ Hₛ d₂] ≤ Hₛ (p[d₁ ↔ d₂])
```

2.627 QuantumInfo.ClassicalInfo.Prob

QuantumInfo/ClassicalInfo/Prob.lean on GitHub.

lemma negLog_strictAnti [source]

Smaller probabilities have strictly greater negative logarithms, including at zero.

```
lemma negLog_strictAnti : StrictAnti negLog
```

lemma negLog_surjective [source]

Every extended nonnegative value is the negative logarithm of a probability.

```
lemma negLog_surjective : Function.Surjective negLog
```

def negLogOrderIso [source]

The negative logarithm identifies probabilities with extended nonnegative values in reverse order, sending zero to infinity and one to zero.

```
def negLogOrderIso : Prob ≃o ENNReal^od
```

lemma negLogOrderIso_apply [source]

```
@[simp]
lemma negLogOrderIso_apply (p : Prob) : negLogOrderIso p = OrderDual.toDual (negLog p)
```

2.628 QuantumInfo.ForMathlib.ExchangeRelations

QuantumInfo/ForMathlib/ExchangeRelations.lean on GitHub.

lemma prod_apply_update [source]

Changing coordinate `i` of `x` to `a` changes only the factor indexed by `i` in `∏ j, v j (x j)`.

```
lemma Finset.prod_apply_update (v : (i : ι) → κ i → K) (x : (i : ι) → κ i) (i : ι)
    (a : κ i) : ∏ j, v j (update x i a j) = v i a * ∏ j ∈ Finset.univ.erase i, v j (x j)
```

lemma prod_update_const_mul [source]

Multiplying the function indexed by `i₀` by `c` multiplies `∏ i, v i (x i)` by `c`.

```
lemma Finset.prod_update_const_mul (v : (i : ι) → κ i → K) (c : K) (i₀ : ι)
    (x : (i : ι) → κ i) :
    ∏ i, update v i₀ (fun a ↦ c * v i₀ a) i (x i) = c * ∏ i, v i (x i)
```

lemma prod_apply_update_mul_prod_apply_update [source]

A product `x ↦ ∏ i, v i (x i)` of functions of one variable satisfies the exchange relations.

```
lemma prod_apply_update_mul_prod_apply_update (v : (i : ι) → κ i → K) (x y : (i : ι) → κ i)
    (i : ι) (a b : κ i) :
    (∏ j, v j (update x i a j)) * ∏ j, v j (update y i b j) =
      (∏ j, v j (update x i b j)) * ∏ j, v j (update y i a j)
```

lemma eq_mul_prod_of_mul_update [source]

If `T` satisfies the exchange relations and `T x₀ ≠ 0`, then every `x` that agrees with `x₀` outside a finite set `s` of coordinates satisfies `T x = T x₀ * ∏ j ∈ s, T (update x₀ j (x j)) / T x₀`.

```
lemma eq_mul_prod_of_mul_update {T : ((i : ι) → κ i) → K}
    (hT : ∀ i (a b : κ i) x y, T (update x i a) * T (update y i b) =
      T (update x i b) * T (update y i a))
    {x₀ : (i : ι) → κ i} (h₀ : T x₀ ≠ 0) (s : Finset ι) :
    ∀ x : (i : ι) → κ i, (∀ j ∉ s, x j = x₀ j) →
      T x = T x₀ * ∏ j ∈ s, T (update x₀ j (x j)) / T x₀
```

lemma exists_prod_of_mul_update [source]

If ι is nonempty, a function on a finite product that satisfies the exchange relations factors as a product of functions of one variable.

```
lemma exists_prod_of_mul_update [Fintype ι] [Nonempty ι] {T : ((i : ι) → κ i) → K}
    (hT : ∀ i (a b : κ i) x y, T (update x i a) * T (update y i b) =
      T (update x i b) * T (update y i a)) :
    ∃ v : (i : ι) → κ i → K, ∀ x, T x = ∏ i, v i (x i)
```

lemma exists_prod_iff_mul_update [source]

If ι is nonempty, a function on a finite product factors as a product of functions of one variable if and only if it satisfies the exchange relations.

```
lemma exists_prod_iff_mul_update [Fintype ι] [Nonempty ι] {T : ((i : ι) → κ i) → K} :
    (∃ v : (i : ι) → κ i → K, ∀ x, T x = ∏ i, v i (x i)) ↔
      ∀ i (a b : κ i) x y, T (update x i a) * T (update y i b) =
        T (update x i b) * T (update y i a)
```

2.629 QuantumInfo.ForMathlib.HermitianMat.LogExp

QuantumInfo/ForMathlib/HermitianMat/LogExp.lean on GitHub.

lemma log_left' [source]

Primed because `Commute.log_left` refers to `NormedSpace.log` instead of `HermitianMat.log`.

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
lemma _root_.Commute.log_left' (hAB : Commute A.mat B.mat) :
    Commute (A.log).mat B.mat
```

lemma log_right' [source]

Primed because `Commute.log_right` refers to `NormedSpace.log` instead of `HermitianMat.log`.

```
@[aesop unsafe apply 50% (rule_sets := [Commutes])]
lemma _root_.Commute.log_right' (hAB : Commute A.mat B.mat) :
    Commute A.mat (B.log).mat
```

2.630 QuantumInfo.States.Mixed.MState

QuantumInfo/States/Mixed/MState.lean on GitHub.

lemma relabel_pure [source]

Relabeling the pure state associated to a ket gives the pure state associated to the relabeled ket.

```
@[simp]
lemma relabel_pure (ψ : Ket d₁) (e : d₂ ≃ d₁) :
    (pure ψ).relabel e = pure (ψ.relabel e)
```

lemma pure_piProd [source]

The pure state of `Ket.piProd ψ` is `MState.piProd` of the pure states `pure (ψ i)`, generalizing `MState.pure_prod_pure`.

```
lemma pure_piProd (ψ : (i : ι) → Ket (dI i)) :
    pure (Ket.piProd ψ) = piProd (fun i ↦ pure (ψ i))
```

lemma isPiProd_iff_exists_pure_eq_piProd [source]

With at least one party, a ket satisfies `Ket.IsPiProd` if and only if its associated pure state `MState.pure ψ` is an `MState.piProd` of pure states.

```
lemma isPiProd_iff_exists_pure_eq_piProd [Nonempty ι] (ψ : Ket ((i : ι) → dI i)) :
    ψ.IsPiProd ↔ ∃ φ : (i : ι) → Ket (dI i), pure ψ = piProd (fun i ↦ pure (φ i))
```

2.631 QuantumInfo.States.Pure.Braket

QuantumInfo/States/Pure/Braket.lean on GitHub.

def relabel [source]

Relabel a ket along an equivalence of basis index types. Given `e : d₂ ≃ d₁`, the amplitude of `ψ.relabel e` at `i : d₂` is the amplitude of `ψ` at `e i`.

```
def Ket.relabel (ψ : Ket d₁) (e : d₂ ≃ d₁) : Ket d₂ where
```

lemma relabel_apply [source]

```
@[simp]
lemma Ket.relabel_apply (ψ : Ket d₁) (e : d₂ ≃ d₁) (i : d₂) :
    ψ.relabel e i = ψ (e i)
```

def piProd [source]

The tensor product of a finite family of kets, generalizing `Ket.prod`. Party `i` has basis index type `d i`, and a joint index `j` selects one index `j i` for each party; the amplitude at `j` is `∏ i, ψ i (j i)`.

```
def Ket.piProd {ι : Type*} [DecidableEq ι] [Fintype ι] {d : ι → Type*}
    [∀ i, Fintype (d i)] (ψ : (i : ι) → Ket (d i)) : Ket ((i : ι) → d i) where
```

lemma piProd_apply [source]

```
@[simp]
lemma Ket.piProd_apply {ι : Type*} [DecidableEq ι] [Fintype ι] {d : ι → Type*}
    [∀ i, Fintype (d i)] (ψ : (i : ι) → Ket (d i)) (j : (i : ι) → d i) :
```

```
    Ket.piProd ψ j = ∏ i, ψ i (j i)
```

def IsPiProd [source]

A ket on the joint system of a finite family of parties is a product state if it is the `Ket.piProd` of one ket for each party, generalizing `Ket.IsProd`.

```
def Ket.IsPiProd {ι : Type*} [DecidableEq ι] [Fintype ι] {d : ι → Type*}
    [∀ i, Fintype (d i)] (ψ : Ket ((i : ι) → d i)) : Prop
```

lemma isPiProd_piProd [source]

Every `Ket.piProd` state satisfies `Ket.IsPiProd`, generalizing `Ket.IsProd_prod`.

```
@[simp]
lemma Ket.isPiProd_piProd {ι : Type*} [DecidableEq ι] [Fintype ι] {d : ι → Type*}
    [∀ i, Fintype (d i)] (φ : (i : ι) → Ket (d i)) : (Ket.piProd φ).IsPiProd
```

lemma isPiProd_iff_isProd_relabel [source]

For two parties, `Ket.IsPiProd` agrees with `Ket.IsProd` after relabeling the joint index type `(i : Fin 2) → d i` as `d 0 × d 1`.

```
lemma Ket.isPiProd_iff_isProd_relabel {d : Fin 2 → Type*} [∀ i, Fintype (d i)]
    (ψ : Ket ((i : Fin 2) → d i)) :
    ψ.IsPiProd ↔ (ψ.relabel (piFinTwoEquiv d).symm).IsProd
```

lemma of_phaseEquiv [source]

If a ket is phase-equivalent to a `Ket.IsPiProd` state, then it is also a `Ket.IsPiProd` state, provided there is at least one party. The global phase is absorbed into one tensor factor.

```
lemma Ket.IsPiProd.of_phaseEquiv [Nonempty ι] {ψ ψ' : Ket ((i : ι) → d i)}
    (h' : ψ'.IsPiProd) (h : Ket.PhaseEquiv.r ψ ψ') : ψ.IsPiProd
```

2.632 `scripts.alpha.github.pr_summary`

`scripts/alpha/github/pr_summary.lean` on GitHub.

def dropChars [source]

`s` without its first `n` characters.

```
def dropChars (s : String) (n : Nat) : String
```

def trim [source]

`s` without surrounding whitespace.

```
def trim (s : String) : String
```

def trimStart [source]

`s` without leading whitespace.

```
def trimStart (s : String) : String
```

def trimEnd [source]

`s` without trailing whitespace.

```
def trimEnd (s : String) : String
```

def plural [source]

`n` and `noun`, pluralised unless `n` is one: `1 file`, `2 files`.

```
def plural (n : Nat) (noun : String) : String
```

def tally [source]

The non-zero counts of `items` with their labels, as in `2 added, 1 removed`.

```
def tally (items : Array (Nat × String)) : String
```

def code [source]

`s` as a Markdown code span, or an em dash if `s` is empty.

```
def code (s : String) : String
```

def truncate [source]

The first `limit` of `lines`, then a line counting the rest. GitHub caps a comment at 65536 characters.

```
def truncate (lines : Array String) (limit : Nat) : Array String
```

def repository [source]

The repository the links point into.

```
def repository : String
```

structure Source [source]

The two commits being compared. A removed line is linked in `base`, since it is no longer in `head`; everything else is linked in `head`.

```
structure Source where
  base : String := ""
  head : String := ""
  deriving Repr, DecidableEq, Inhabited
```

def Source.link [source]

`text` linked to `file` in `repository`, at line `line` unless that is zero, in `src.head` if `added` and in `src.base` otherwise.

```
def Source.link (src : Source) (text file : String) (added : Bool) (line : Nat := 0) : String
```

inductive FileStatus [source]

How a file changed, as `git diff --name-status` reports it.

```
inductive FileStatus where
  | added
  | removed
  | modified
  | renamed (oldPath : String)
  | copied (oldPath : String)
  deriving Repr, DecidableEq, Inhabited
```

def FileStatus.rank [source]

The order in which the statuses are reported, as in leaff: additions, removals, then modifications.

```
def FileStatus.rank : FileStatus → Nat
  | .added => 0
  | .removed => 1
  | .renamed .. => 2
  | .copied .. => 3
  | .modified => 4
```

structure FileChange [source]

A file the pull-request changes.

```
structure FileChange where
  path : String
  status : FileStatus
  insertions : Option Nat := none
  deletions : Option Nat := none
  namespaces : Array String := #[]
  addedNamespaces : Array String := #[]
  removedNamespaces : Array String := #[]
  deriving Repr, DecidableEq, Inhabited
```

def FileChange.lt [source]

Files are reported by status, then by path.

```
def FileChange.lt (c d : FileChange) : Bool
```

def oldPath [source]

The path of the file in the base revision: its old path if it was renamed or copied.

```
def FileChange.oldPath (c : FileChange) : String
```

def paths [source]

The paths the diff may give for lines of the file: its path, and, if it was renamed or copied, its old path.

```
def FileChange.paths (c : FileChange) : List String
```

def parseNameStatusLine [source]

The file change one line records, or `none` for a line which is not one.

```
def parseNameStatusLine (line : String) : Option FileChange
```

def parseNameStatus [source]

The file changes recorded by the lines of `git diff --name-status`.

```
def parseNameStatus (lines : Array String) : Array FileChange
```

def parseNumstat [source]

The new path and the two counts of each file in the fields of `git diff --numstat -z`.

```
def parseNumstat (fields : Array String) : Array (String × Option Nat × Option Nat)
```

def withLineCounts [source]

`changes` with the counts of `counts` filled in by path.

```
def withLineCounts (changes : Array FileChange)
    (counts : Array (String × Option Nat × Option Nat)) : Array FileChange
```

def modifiers [source]

The modifiers which may precede a declaration keyword.

```
def modifiers : Array String
```

def openBrackets [source]

The brackets which open a group.

```
def openBrackets : List Char
```

def closeBrackets [source]

The brackets which close a group.

```
def closeBrackets : List Char
```

def afterCloseBracket [source]

`cs` after the bracket which closes the group `depth` brackets deep, or `none` if the group is not closed in `cs`.

```
def afterCloseBracket (cs : List Char) (depth : Nat) : Option (List Char)
```

def dropBalanced? [source]

`s` without its leading bracketed group, if it starts with a group which is closed.

```
def dropBalanced? (s : String) : Option String
```

def dropAttribute? [source]

`s` without its leading `@[…]`, if it has one.

```
def dropAttribute? (s : String) : Option String
```

def dropModifier? [source]

`s` without its leading modifier, if it has one.

```
def dropModifier? (s : String) : Option String
```

def dropScopePrefix? [source]

`s` without its leading `open … in` or `set_option … in`, if it has one.

```
def dropScopePrefix? (s : String) : Option String
```

def normalise [source]

`s` with every leading attribute, modifier and `… in` prefix removed.

```
def normalise (s : String) : String
```

def declKeywords [source]

The keywords which start a declaration. `remark`, `informal_definition` and `informal_lemma` are Physlib's own.

```
def declKeywords : Array String
```

def nameStop [source]

The characters which end a name.

```
def nameStop : List Char
```

def firstToken [source]

```
def firstToken (s : String) : String
```

def isIdentifier [source]

```
def isIdentifier (s : String) : Bool
```

def beforeAssignment [source]

```
def beforeAssignment (s : String) : String
```

def signature [source]

```
def signature (s : String) : String
```

structure DeclHead [source]

```
structure DeclHead where
  keyword : String
  /-- The name, or, for an anonymous `instance`, the signature. -/
  name : String
  /-- Whether `name` is a signature rather than a name. -/
  anonymous : Bool := false
  deriving Repr, DecidableEq, Inhabited
```

def display [source]

```
def DeclHead.display (h : DeclHead) : String
```

def splitKeyword? [source]

```
def splitKeyword? (line : String) : Option (String × String)
```

def namedHead [source]

```
def namedHead (keyword rest : String) : Array DeclHead
```

def aliasHeads [source]

```
def aliasHeads (rest : String) : Array DeclHead
```

def instanceHead [source]

```
def instanceHead (rest : String) : Array DeclHead
```

def declHeads [source]

```
def declHeads (line : String) : Array DeclHead
```

structure Location [source]

```
structure Location where
  file : String
  /-- Counted from one. -/
  line : Nat
  /-- Whether the line is in the head revision (`+`) rather than the base revision (`-`). -/
  added : Bool
  deriving Repr, DecidableEq, Inhabited
```

structure DeclLine [source]

```
structure DeclLine where
  loc : Location
  decl : DeclHead
  /-- The line without its `+` or `-`; two equal texts mean the declaration moved. -/
  text : String
  deriving Repr, DecidableEq, Inhabited
```

def diffPath [source]

```
def diffPath (s : String) : String
```

def parseHunkHeader [source]

```
def parseHunkHeader (line : String) : Option (Nat × Nat)
```

def parseDiff [source]

```
def parseDiff (lines : Array String) : Array DeclLine
```

inductive DeclChange [source]

How a declaration changed.

```
inductive DeclChange where
  | added (loc : Location)
  | restated (loc : Location)
  | moved (old new : Location)
  | removed (loc : Location)
  deriving Repr, DecidableEq, Inhabited
```

def DeclChange.rank [source]

The order in which the changes are reported, as in leaff: additions, removals, then modifications.

```
def DeclChange.rank : DeclChange → Nat
  | .added .. => 0
  | .removed .. => 1
  | .restated .. => 2
  | .moved .. => 3
```

def isNew [source]

Whether the head revision has a statement the base did not: an added or a restated one.

```
def DeclChange.isNew : DeclChange → Bool
  | .added .. | .restated .. => true
  | .moved .. | .removed .. => false
```

def target [source]

Where to read the declaration now, or, if it is gone, where it was.

```
def DeclChange.target : DeclChange → Location
  | .added loc | .restated loc | .removed loc => loc
  | .moved _ new => new
```

structure DeclDiff [source]

A declaration and how it changed.

```
structure DeclDiff where
  decl : DeclHead
  change : DeclChange
  deriving Repr, DecidableEq, Inhabited
```

def DeclDiff.lt [source]

Declarations are reported by change, then keyword, then name.

```
def DeclDiff.lt (c d : DeclDiff) : Bool
```

def DeclDiff.link [source]

The declaration as a code span linked to the line to read it on.

```
def DeclDiff.link (src : Source) (d : DeclDiff) : String
```

def declDiff [source]

The declaration lines grouped by name and classified: a name only on added lines was added, only on removed lines was removed, on both with the same texts was moved, and on both with different texts was restated. Namespaces are not considered.

```
def declDiff (lines : Array DeclLine) : Array DeclDiff
```

structure FileContents [source]

A file as one revision has it.

```
structure FileContents where
  path : String
  added : Bool
  lines : Array String
  deriving Repr, DecidableEq, Inhabited
```

def namespacesOf [source]

The namespaces the lines of a `.lean` file open, in the order they first open them.

```
def namespacesOf (lines : Array String) : Array String
```

def withNamespaces [source]

`changes` with the namespaces of each file filled in from `contents`, and, for a file in both revisions, the namespaces it gained and lost.

```
def withNamespaces (changes : Array FileChange) (contents : Array FileContents) :
    Array FileChange
```

def moduleOf [source]

The module of a `.lean` file, or the path of any other file.

```
def moduleOf (path : String) : String
```

def kindOf [source]

`module` for a `.lean` file, `file` otherwise.

```
def kindOf (path : String) : String
```

def lineCounts [source]

`(+12 -3)`, or `(binary)`.

```
def lineCounts (c : FileChange) : String
```

def FileChange.line [source]

The line of the block for a file change.

```
def FileChange.line (c : FileChange) : String
```

def DeclDiff.line [source]

The line of the block for a declaration change.

```
def DeclDiff.line (d : DeclDiff) : String
```

def diffBlock [source]

The block: each file in order of status then path, under a header, followed by its namespaces, the namespaces it gained and lost, and the changes to its declarations in order of importance; at most `limit` lines; then the number of changes. A moved declaration is listed under the file it is now in.

```
def diffBlock (files : Array FileChange) (decls : Array DeclDiff) (limit : Nat := 300) :
    String
```

def theoremsFile [source]

The file listing every result written with `theorem`.

```
def theoremsFile : String
```

def newTheorems [source]

The declarations written with `theorem` which are new or restated.

```
def newTheorems (decls : Array DeclDiff) : Array DeclDiff
```

def theoremCallout [source]

The call-out, or the empty string if the pull-request writes no `theorem`. It also says whether the pull-request changes the theorems file.

```
def theoremCallout (src : Source) (files : Array FileChange) (decls : Array DeclDiff) : String
```

def caveat [source]

The closing sentence, so a reader knows how far to trust the block.

```
def caveat : String
```

def render [source]

The call-out, if any, then the block in a `diff` fence, then the caveat.

```
def render (src : Source) (files : Array FileChange) (decls : Array DeclDiff) : String
```

def summarise [source]

The whole summary, from the lines of `git diff --name-status` and `git diff --unified=0`, the fields of `git diff --numstat -z`, and the changed `.lean` files in each revision. This is the entire behaviour of the script except for running `git`, and is what G tests.

```
def summarise (src : Source) (nameStatus numstat diff : Array String)
    (contents : Array FileContents) : String
```

def git [source]

The standard output of `git args`, run in the current directory. Throws if `git` fails.

```
def git (args : Array String) : IO String
```

def gitLines [source]

The output of `git args` as lines, without the final empty one.

```
def gitLines (args : Array String) : IO (Array String)
```

def gitFields [source]

The NUL-terminated fields of the output of `git args`, run with `-z`.

```
def gitFields (args : Array String) : IO (Array String)
```

def gitContents [source]

The changed `.lean` files of `nameStatus` in each revision they are in: the head revision of every file but a removed one, and the base revision, at its old path, of every file but an added one.

```
def gitContents (src : Source) (nameStatus : Array String) : IO (Array FileContents)
```

def commitOf [source]

The commit `revision` names. A link must use a commit: a link to `HEAD` would follow the branch rather than point at what was summarised.

```
def commitOf (revision : String) : IO String
```

def mergeBase [source]

The commit which `git diff base...head` compares `head` with: the merge base of the two, or `base` itself if they have none.

```
def mergeBase (base head : String) : IO String
```

def gitSummary [source]

The summary of the changes `head` makes to `base`. `-M` makes `git` report a renamed file as renamed rather than as removed and added, so its declarations are not all reported as moved and its lines are counted against its old self.

```
def gitSummary (base head : String) : IO String
```

def revisions [source]

The revisions from the arguments: `origin/master` and `HEAD` unless given.

```
def revisions : List String → Except String (String × String)
  | [] => .ok ("origin/master", "HEAD")
  | [base] => .ok (base, "HEAD")
  | [base, head] => .ok (base, head)
  | args => .error s!"Expected at most two revisions, got {args.length}."
```

def testSource [source]

Two stand-in commits.

```
def testSource : Source
```

def shortLinks [source]

`s` with `https://github.com/<repository>/blob/` shortened to `…/`.

```
def shortLinks (s : String) : String
```

2.633 scripts.alpha.lint.required.api_map_lint

`scripts/alpha/lint/required/api_map_lint.lean` on GitHub.

structure LeanReference [source]

One entry of `## Lean References`: its number, its Lean file and the declarations it claims.

```
structure LeanReference where
  number : Nat
  file : String
  claims : List String
```

structure TextReference [source]

One entry of `## References`: its number and its text, which cites the literature or another API map.

```
structure TextReference where
  number : Nat
  text : String
```

def trim [source]

`s` with leading and trailing whitespace removed.

```
private def trim (s : String) : String
```

def containsStr [source]

Whether `needle` occurs in `haystack`.

```
private def containsStr (haystack needle : String) : Bool
```

def isWordChar [source]

Whether `c` is a word character in the sense of a regular expression's `\{}w`: an ASCII letter or digit, the underscore, or any non-ASCII character that is not whitespace.

```
private def isWordChar (c : Char) : Bool
```

def words [source]

The whitespace-separated words of `s`.

```
private def words (s : String) : List String
```

def quote [source]

`s` in single quotes, for error messages.

```
private def quote (s : String) : String
```

def numbered [source]

The number and the rest of a line opening a numbered entry `n. rest`.

```
private def numbered (line : String) : Option (Nat × String)
```

def LeanReference.ofLine? [source]

Reads an entry `n. File.lean — a, b``.

```
def LeanReference.ofLine? (line : String) : Option LeanReference
```

def TextReference.ofLine? [source]

Reads an entry `n. text`.

```
def TextReference.ofLine? (line : String) : Option TextReference
```

def mapPaths [source]

The paths of API maps named in a text reference.

```
def TextReference.mapPaths (reference : TextReference) : List String
```

def parse [source]

Reads an API map from its text. Returns the map, together with the problems found that the map cannot hold: text between the title and the first section, and reference entries that cannot be read.

```
def parse (path text : String) : APIMapData × Array String
```

def read [source]

Reads the API map at the repository-relative `path`, together with the problems found that the map cannot hold.

```
def read (repo : System.FilePath) (path : String) : IO (APIMapData × Array String)
```

def findPaths [source]

The repository-relative paths of the files called `name` under `relative`, skipping hidden directories such as `.lake` and `.git`.

```
partial def findPaths (repo : System.FilePath) (name : String) (relative : String := "") :
    IO (Array String)
```

def isFile [source]

Whether `path` is an existing file.

```
def isFile (path : System.FilePath) : IO Bool
```

def proseSections [source]

The prose sections of a map, with their headings, in order: the abstract, the alpha abstract, the content and the alpha content.

```
def proseSections (map : APIMapData) : List (String × Array String)
```

def isAlpha [source]

Whether a map is alpha, that is, has not yet been through the stringent review.

```
def isAlpha (map : APIMapData) : Bool
```

inductive Piece [source]

A piece of prose: a character of text, or a citation.

```
inductive Piece
  | text (c : Char)
  | lean (n : Nat)
  | todo
  | textCitation (n : Nat)
```

def pieces [source]

A paragraph split into text and the citations `[n]`, `[TODO]` and `{n}`.

```
partial def pieces (paragraph : String) : List Piece
```

structure LeanCitation [source]

A run of adjacent Lean citations, which closes one claim of the prose: the reference numbers cited, and whether the run contains `[TODO]`.

```
structure LeanCitation where
  numbers : List Nat
  todo : Bool
```

def leanCitationsOf [source]

The Lean citations of a paragraph, one per run of adjacent citations. Text citations may sit inside a run; a run made only of text citations is not a claim.

```
def leanCitationsOf (paragraph : String) : Array LeanCitation
```

def textCitationsOf [source]

The text citations `{n}` of a paragraph.

```
def textCitationsOf (paragraph : String) : List Nat
```

def leanCitations [source]

The Lean citations in the prose of a map, one per claim.

```
def leanCitations (map : APIMapData) : Array LeanCitation
```

def textCitations [source]

The text citations in the prose of a map.

```
def textCitations (map : APIMapData) : List Nat
```

def leanNumbers [source]

The numbers of the Lean references, in order.

```
def leanNumbers (map : APIMapData) : List Nat
```

def textNumbers [source]

The numbers of the text references, in order.

```
def textNumbers (map : APIMapData) : List Nat
```

inductive ClaimKind [source]

What a claim in a Lean reference names: a declaration, a notation, or an instance.

```
inductive ClaimKind
  | name
  | notation
  | inst
```

def classifyClaim [source]

What a claim names, and the token to look for in the source.

```
def LeanReference.classifyClaim (claim : String) : ClaimKind × String
```

def claimIsPresent [source]

Whether the token of a claim is present in the source. A notation is present when each of its fragments is; a name is present verbatim, or as its last component on word boundaries.

```
def LeanReference.claimIsPresent (source : String) (kind : ClaimKind) (token : String) : Bool
```

structure Counts [source]

The counts reported for a map.

```
structure Counts where
  claims : Nat := 0
  done : Nat := 0
  todo : Nat := 0
  checked : Nat := 0
  namesOk : Nat := 0
  missingFile : Nat := 0
  missingName : Nat := 0
  needLean : Nat := 0
  leanInProse : Nat := 0
```

def add [source]

The sum of two counts.

```
def Counts.add (a b : Counts) : Counts where
```

structure Report [source]

The outcome of a check: the errors, which fail the run, the notes, which are shown only with `--verbose`, and the counts.

```
structure Report where
  errors : Array String := #[]
  notes : Array String := #[]
  counts : Counts := {}
```

def append [source]

Two reports, one after the other.

```
def Report.append (a b : Report) : Report where
```

def ofErrors [source]

A report with only errors.

```
def Report.ofErrors (errors : Array String) : Report
```

def headerRank [source]

The rank of each header in the required order, or `none` for an unknown header.

```
def headerRank : String → Option Nat
  | "Abstract" | "Abstract (Alpha)" => some 0
  | "Content" => some 1
  | "Content (Alpha)" => some 2
  | "Lean References" => some 3
  | "References" => some 4
  | _ => none
```

def checkHeadings [source]

Problems with the title and the headers of a map: exactly one title, only known headers, each once, an abstract, some content and the Lean references, in the required order.

```
def checkHeadings (map : APIMapData) : Array String
```

def checkProse [source]

Problems with the prose of a map: an abstract must be a single paragraph without citations, a content section must not be empty, and neither may contain links, `[planned]` or grouped citations.

```
partial def checkProse (map : APIMapData) : Array String
```

def checkNumbering [source]

Problems with the numbering of a reference list, which must run 1, 2, …, n.

```
def checkNumbering (label : String) (numbers : List Nat) : Array String
```

def checkMapPaths [source]

Problems with the text references of a map: the API maps they name must exist.

```
def checkMapPaths (map : APIMapData) (repo : System.FilePath) : IO (Array String)
```

def checkCitations [source]

Checks the citations in the prose of a map: every citation resolves, and every reference is cited. Also counts the claims, and notes Lean names left in the content.

```
def checkCitations (map : APIMapData) : Report
```

def checkDeclarations [source]

Checks the Lean references of a map: each file exists and contains the declarations claimed of it.

```
def checkDeclarations (map : APIMapData) (repo : System.FilePath) : IO Report
```

def lint [source]

Lints a map: its headings and prose, its reference lists, its citations and its declarations.

```
def lint (map : APIMapData) (repo : System.FilePath) : IO Report
```

2.634 scripts.alpha.website.APIMapIndex

scripts/alpha/website/APIMapIndex.lean on GitHub.

structure APIMapReference [source]

One entry of the `## Lean References` list: the number it is cited by, the Lean file, and the declarations claimed to be in it.

```
structure APIMapReference where
  index : Nat
  file : String
  declarations : List String
```

def locationOf [source]

The `file (names)` form of a requirement's location, built from the entries it cites, or `N/A` when it cites none.

```
def APIMapData.locationOf (map : APIMapData) (requirement : APIMapRequirement) : String
```

def trim [source]

`value` with leading and trailing whitespace removed.

```
def trim (value : String) : String
```

def spanBetween [source]

The text between the first pair of `delimiters`, and the remainder after the closing one.

```
private def spanBetween (value : String) (delimiter : String) : Option (String × String)
```

def backtickedSpans [source]

Every backtick-delimited span of a line, in order.

```
partial def backtickedSpans (value : String) : List String
```

inductive ProseToken [source]

A piece of content prose: plain text, or one citation `[n, m]` or `[TODO]`.

```
inductive ProseToken
  | text (value : String)
  | cite (numbers : List Nat) (todo : Bool)
```

def citationOf [source]

Reads the inside of a pair of square brackets as a citation, if it is one.

```
def citationOf (inner : String) : Option ProseToken
```

def tokenize [source]

Splits a paragraph into text and citations. Square brackets that are not citations, such as the text of a markdown link, stay in the text.

```
def tokenize (paragraph : String) : List ProseToken
```

def removeLiterature [source]

Removes the literature citations `{n}` from a paragraph.

```
def removeLiterature (paragraph : String) : String
```

def stripLiterature [source]

Removes the literature citations `{n}` from a paragraph, together with the space a citation leaves before a following full stop or comma.

```
def stripLiterature (paragraph : String) : String
```

def claimsOf [source]

The claims of a content paragraph. Each run of adjacent citations closes a claim, whose description is the prose since the previous run; a run is done unless it contains `[TODO]`. Prose after the last run cites nothing and is not a claim.

```
def claimsOf (alpha : Bool) (paragraph : String) : List APIMapRequirement
```

def declarationEntry [source]

Reads a `## Lean References` entry, `n. File.lean — a, b``.

```
def declarationEntry (line : String) : Option APIMapReference
```

inductive Section [source]

The section a line belongs to.

```
inductive Section
  | preamble
  | abstract
  | content (alpha : Bool)
  | references
  | literature
  | unknown
```

def sectionOf [source]

The section a `##` heading opens.

```
def sectionOf (heading : String) : Section
```

def toYAML [source]

A Lean reference as a YAML list item.

```
def APIMapReference.toYAML (reference : APIMapReference) : String
```

def requirementYAML [source]

A claim of a map as a YAML list item, with its location.

```
def APIMapData.requirementYAML (map : APIMapData) (requirement : APIMapRequirement) : String
```

2.635 scripts.dev.get_cache

`scripts/dev/get_cache.lean` on GitHub.

def removeRevisionMappings [source]

Deletes Lake's local copies of Physlib's revision mappings. Lake saves them at `<cache dir>/revisions/Physlib/<rev>.jsonl` whatever the `--scope`, so the same path is shared by Physlib and PhyslibAlpha.

```
def removeRevisionMappings : IO Unit
```

2.636 scripts.lint.required.auxillary-script-test

`scripts/lint/required/auxillary-script-test.lean` on GitHub.

def checkAPIMapIndex [source]

Checks `lake exe api_map_index mkFile`: it should make `docs/_data/APIMap.yml`, with a `Total:` section whose map count matches the non-zero number of maps listed under `APIMap:`.

```
def checkAPIMapIndex (ctx : Ctx) (cwd : FilePath) : Check Unit
```

2.637 scripts.lint.required.forMathlib_lint

scripts/lint/required/forMathlib_lint.lean on GitHub.

def forMathlibPrefix [source]

The module name prefix of the files in `Physlib/Mathematics/ForMathlib/`.

```
def forMathlibPrefix : Name
```

def libraryDirs [source]

The library directories whose files are read by the linter.

```
def libraryDirs : List String
```

def isForMathlib [source]

Whether a module lives in `Physlib/Mathematics/ForMathlib/`.

```
def isForMathlib (n : Name) : Bool
```

def isLibraryModule [source]

Whether a module lives in one of the libraries `Physlib`, `PhyslibAlpha` or `QuantumInfo`.

```
def isLibraryModule (n : Name) : Bool
```

def moduleNameOfPath [source]

The module name of a `.lean` file, given its path relative to the repository root.

```
def moduleNameOfPath (path : FilePath) : Name
```

def libraryImports [source]

The modules of `libraryDirs`, each paired with the modules of `libraryDirs` it imports.

```
def libraryImports : IO (Array (Name × Array Name))
```

def outsideImports [source]

The pairs `(m, i)` of a module `m` in `ForMathlib` importing a module `i` of `libraryDirs` which is not in `ForMathlib`.

```
def outsideImports (graph : Array (Name × Array Name)) : Array (Name × Name)
```

def unusedModules [source]

The modules in `ForMathlib` which are not imported, directly or through other modules in `ForMathlib`, by any module outside of `ForMathlib`.

```
def unusedModules (graph : Array (Name × Array Name)) : Array Name
```

def sortNames [source]

Sorts an array of names alphabetically.

```
def sortNames (ns : Array Name) : Array Name
```

2.638 scripts.lint.required.module_doc_lint

`scripts/lint/required/module_doc_lint.lean` on GitHub.

def strip [source]

`s` with leading and trailing whitespace removed.

```
def strip (s : String) : String
```

def rstrip [source]

`s` with trailing whitespace removed.

```
def rstrip (s : String) : String
```

def moduleDocLines [source]

The lines of module documentation in a file, paired with their (1-indexed) line numbers. Lines inside code fences, and the fence markers themselves, are left out.

```
def moduleDocLines (lines : Array String) : Array (Nat × String)
```

structure Heading [source]

A heading in the module documentation.

```
structure Heading where
  line : Nat
  level : Nat
  text : String
  raw : String
  spaced : Bool
```

def parseHeading [source]

```
def parseHeading (n : Nat) (line : String) : Option Heading
```

inductive ErrorKind [source]

```
inductive ErrorKind where
  | noModuleDoc
  | titleHead
  | extraTitle
  | overviewHead
  | keyResultsHead
  | tableOfContentsHead
  | referencesHead
  | sectionOrder
  | noSections
  | sectionTag
  | duplicateTag
  | headingFullStop
  | tableOfContentsCorrect
```

def all [source]

All kinds of errors, in the order they are reported.

```
def ErrorKind.all : List ErrorKind
```

def name [source]

```
def ErrorKind.name : ErrorKind → String
  | .noModuleDoc => "No module documentation headings"
  | .titleHead => "Missing or malformed title"
  | .extraTitle => "Extra title headings"
  | .overviewHead => "Missing or malformed overview section"
  | .keyResultsHead => "Missing or malformed key results section"
  | .tableOfContentsHead => "Missing or malformed table of contents section"
  | .referencesHead => "Missing or malformed references section"
  | .sectionOrder => "Standard sections out of order"
  | .noSections => "No section headings"
  | .sectionTag => "Malformed section tags"
  | .duplicateTag => "Duplicate section tags"
  | .headingFullStop => "Headings ending in a full stop"
  | .tableOfContentsCorrect => "Table of contents does not match headings"
```

def hint [source]

```
def ErrorKind.hint : ErrorKind → String
  | .noModuleDoc => "Add module documentation `/-! … -/` with the standard headings."
  | .titleHead => "Add a title heading starting with '# ' for the whole module, as the first
    heading."
  | .extraTitle => "Only the module title should use '# '; use '## A.', '### A.1.' etc. for
    sections."
  | .overviewHead => "Add an overview section '## i. Overview' after the title heading."
  | .keyResultsHead => "Add a key results section '## ii. Key results' after the overview
    section."
  | .tableOfContentsHead => "Add a table of contents section '## iii. Table of contents' after
    the key results section. This can be filled in later."
  | .referencesHead => "Add a references section '## iv. References' after the table of
    contents section."
  | .sectionOrder => "The headings should start: title, '## i. Overview', '## ii. Key results',
     '## iii. Table of contents', '## iv. References'."
```

```
  | .noSections => "Add other headings for sections and subsections using e.g. '## A.', '### A
    .1.', '#### A.1.2' etc."
  | .sectionTag => "Section tags end in a dot and have one dot fewer than the heading has '#'s,
     e.g. '## A.', '### A.1.', '#### A.1.2.'."
  | .duplicateTag => "Each section tag should be used only once."
  | .headingFullStop => "Ensure all headings do not end in a full stop."
  | .tableOfContentsCorrect => "Fix the table of contents to match the headings in the file."
```

structure StandardSection [source]

One of the standard sections following the title.

```
structure StandardSection where
  kind : ErrorKind
  expected : String
  key : String
```

def standardSections [source]

```
def standardSections : List StandardSection
```

def key [source]

The text of a heading with a leading roman numeral, case and a trailing dot ignored.

```
def Heading.key (h : Heading) : String
```

def hashes [source]

```
def hashes (n : Nat) : String
```

def tocMismatch [source]

The first difference between the given table of contents entries (with line numbers) and the expected ones, as an optional line number and a message.

```
def tocMismatch : List (Nat × String) → List String → Option (Option Nat × String)
  | [], [] => none
  | (n, x) :: gs, y :: es =>
    if x == y then tocMismatch gs es else some (some n, s!"Entry '{x}' should be '{y}'")
  | [], y :: es =>
    some (none, s!"Missing entry '{y}'" ++ if es.isEmpty then "" else s!" (and {es.length} more
    )")
  | (n, x) :: gs, [] =>
    some (some n, s!"Unexpected entry '{x}'" ++ if gs.isEmpty then "" else s!" (and {gs.length}
     more)")
```

2.639 scripts.lint.required.theorem_lint

scripts/lint/required/theorem_lint.lean on GitHub.

def libraries [source]

The libraries which are linted.

```
def libraries : Array Name
```

def theoremsFile [source]

The file listing the theorems of the libraries, relative to the root of the repository.

```
def theoremsFile : System.FilePath
```

inductive ErrorKind [source]

The kinds of error the linter reports.

```
inductive ErrorKind where
  | unknownSection (text : String)
  | beforeFirstSection (text : String)
  | notAName (text : String)
  | duplicate (name : Name) (firstLine : Nat)
  | noDocString (name : Name)
  | notListed (library name : Name)
  | writtenWithLemma (name : Name) (file : System.FilePath) (pos : Position)
  | notADeclaration (name : Name)
  | wrongLibrary (library name module : Name)
  | notATheorem (name : Name)
```

def rank [source]

The place of the kind of `kind` in the order in which the errors are reported.

```
def ErrorKind.rank : ErrorKind → Nat
  | .unknownSection .. => 0
  | .beforeFirstSection .. => 1
  | .notAName .. => 2
  | .duplicate .. => 3
  | .noDocString .. => 4
  | .notListed .. => 5
  | .writtenWithLemma .. => 6
  | .notADeclaration .. => 7
  | .wrongLibrary .. => 8
  | .notATheorem .. => 9
```

def title [source]

The heading of the errors of the kind of `kind`.

```
def ErrorKind.title : ErrorKind → String
  | .unknownSection .. => "Unknown sections"
  | .beforeFirstSection .. => "Names listed before the first section"
  | .notAName .. => "Lines which are not Lean names"
  | .duplicate .. => "Names listed twice"
  | .noDocString .. => "Theorems without a doc-string"
  | .notListed .. => s!"Theorems not listed in {theoremsFile}"
  | .writtenWithLemma .. => "Listed results written with `lemma`"
  | .notADeclaration .. => "Listed names which are not declarations"
  | .wrongLibrary .. => "Names listed under the wrong library"
  | .notATheorem .. => "Listed names which are not theorems"
```

def hint [source]

How to fix the errors of the kind of `kind`.

```
def ErrorKind.hint : ErrorKind → String
  | .unknownSection .. => s!"The sections are \
    {", ".intercalate (libraries.map (s!"`[{·}]`")).toList}."
  | .beforeFirstSection .. => "List each name under the section `[L]` of its library `L`."
  | .notAName .. => "Each line is a section `[L]`, a fully qualified name, or a comment
  starting \
    with `--`."
  | .duplicate .. => "Remove the repeated entries."
  | .noDocString .. => "Add a doc-string `/-- … -/` to each of these theorems."
  | .notListed .. => "Use `lemma`. Only list a result, under the section of its library, if it
  \
    is a significant result in physics, usually known by name."
  | .writtenWithLemma .. => "Use `theorem`, or remove the name from the theorems file."
  | .notADeclaration .. => "Give the fully qualified name of each result."
  | .wrongLibrary .. => "Move each name to the section of the library declaring it."
  | .notATheorem .. => "Only results written with `theorem` can be listed."
```

def detail [source]

The details of the error `kind`, shown after its location.

```
def ErrorKind.detail : ErrorKind → String
  | .unknownSection text | .beforeFirstSection text | .notAName text => text
  | .duplicate name firstLine => s!"{name}, first listed on line {firstLine}"
  | .noDocString name | .notListed _ name | .notADeclaration name | .notATheorem name =>
    name.toString
  | .writtenWithLemma name file pos =>
    s!"{name}, written with `lemma` at {file}:{pos.line}:{pos.column}"
  | .wrongLibrary library name module =>
    s!"{name}, listed under `[{library}]` but declared in {module}"
```

structure LintError [source]

An error of the linter, located in a source file.

```
structure LintError where
  file : System.FilePath
  pos : Position
  kind : ErrorKind
```

def theoremsFileError [source]

An error of kind `kind` located at the line `line` of the theorems file.

```
def theoremsFileError (line : Nat) (kind : ErrorKind) : LintError
```

def lt [source]

The order of errors: by kind, then by file, then by position.

```
def LintError.lt (e f : LintError) : Bool
```

def report [source]

The report of `errors`, grouped by kind. Each group has a heading with its number of errors, how to fix them, and the location and details of each error. With `colour`, the headings and hints are coloured for a terminal.

```
def report (errors : Array LintError) (colour : Bool := true) : String
```

def parseNames [source]

The names listed in the theorems file, given as its lines, each with the library whose section lists it and its line.

```
def parseNames (lines : Array String) : Except LintError (Array (Name × Name × Nat))
```

structure WrittenTheorem [source]

A theorem of the library together with how it is written in the source.

```
structure WrittenTheorem where
  name : Name
  file : System.FilePath
  pos : Position
  keyword : String
  hasDocString : Bool
```

def moduleFile [source]

The source file of a module, relative to the root of the repository.

```
def moduleFile (module : Name) : System.FilePath
```

def headTokens [source]

Parses, with Lean's parser in the environment `env`, the declaration modifiers at `pos` followed by two tokens, and returns these tokens. For a `theorem` or `lemma` declaration they are its keyword and its declaration identifier.

```
def headTokens (env : Environment) (ictx : InputContext) (pos : String.Pos.Raw) :
    Option (Syntax × Syntax)
```

def declares [source]

Whether the declaration identifier `ident`, written in some namespace, declares the constant `name`: `name` is `ident` prefixed by the namespace, or `ident` with `_root_` removed.

```
def declares (ident name : Name) : Bool
```

def writtenTheorems [source]

The theorems declared in `module`, written with `theorem` or `lemma`.

```
def writtenTheorems (env : Environment) (module : Name) (constants : Array ConstantInfo) :
    CoreM (Array WrittenTheorem)
```

def lint [source]

The errors of the linter for `library`, whose section of the theorems file lists `names`: theorems not listed in that section or without a doc-string, and listed names which are not theorems.

```
def lint (library : Name) (names : Array (Name × Nat)) : CoreM (Array LintError)
```

def printParse [source]

Prints the names `parseNames` reads from the lines `lines`, or the error it reports.

```
def printParse (lines : Array String) : IO Unit
```

def headTokensOf [source]

The keyword and identifier which `headTokens` reads at the start of `source`, in the current environment.

```
def headTokensOf (source : String) : CoreM (Option (String × Name))
```